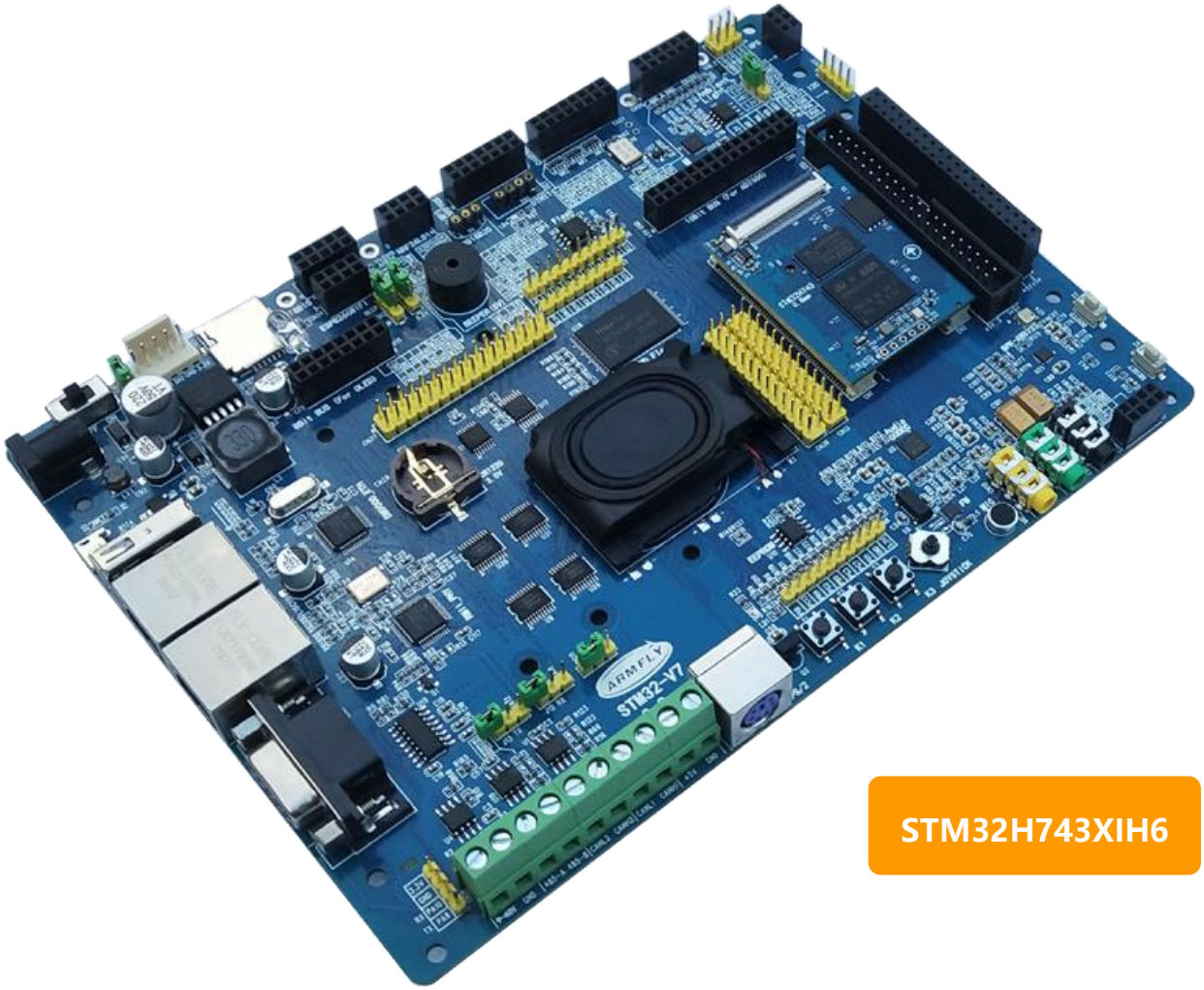
STM32-V7 开发板

用户手册



STM32H743XIH6

武汉安富莱电子有限公司

本文档适用的开发板：

序号	型号配置	说明
1	STM32-V7+4.3寸电阻屏	主板+4.3寸RGB屏（480*320/电阻触摸） 有亚克力保护
2	STM32-V7+4.3寸电容屏	主板+4.3寸RGB屏（480*272/电容触摸） 有亚克力保护
3	STM32-V7+5.0寸电阻屏	主板+5.0寸RGB屏（800*480/电阻触摸） 有亚克力保护
4	STM32-V7+5.0寸电容屏	主板+5.0寸RGB屏（800*480/电容触摸） 有亚克力保护
5	STM32-V7+7.0寸电阻屏	主板+7.0寸RGB屏（800*480/电阻触摸） 有亚克力保护
6	STM32-V7+7.0寸电容屏	主板+7.0寸RGB屏（800*480/电容触摸） 有亚克力保护
7	STM32-V7主板	仅主板 无显示屏，无亚克力保护

(1) 开发板配套的显示模块可以单独购买。

(2) 所有配置中的主板和光盘都是相同的，差异仅在于显示模块和亚克力板。

点击此处直接购买

销售QQ：1295744630　销售旺旺：armfly　微信公众号：安富莱电子
销售电话：13638617262　邮箱：armfly@qq.com
公司网址：www.armfly.com　技术支持论坛：www.armbbs.cn
淘宝直销：armfly.taobao.com
武汉安富莱电子有限公司　专业开发板、显示模块制造商　承接项目开发（提供生产供货服务）

第1章 初学 STM32H7 的准备工作

俗话说万事开头难，学习一门新的知识，难的往往不是知识本身，而是如何快速上手，需要什么资料和开发环境。一旦上手后，深入的学习就相对容易些了。

1.1 初学者重要提示
1.2 开发环境说明
1.3 STM32H7 和 F1，F4 系列的区别
1.4 STM32H7 开发资源查找
1.5 HAL 库介绍
1.6 CMSIS 软件包
1.7 STM32CubeMX 图形平台
1.8 STM32H7 调试方法
1.9 STM32H7 出现硬件异常的解决办法
1.10 总结

1.1 初学者重要提示

- 关于学习方法问题，可以看附件章节 A。
- 学会 STM32H7 相关资源的获取方法，做到心中有数，否则心里老是没底。
- 关于 MDK 和 IAR 两种编译器，推荐都掌握，以其中一个为主，另一个为辅。因为很多时候我们需要参考的工程代码不是自己熟悉的编译器，就会很被动。
- 相对于 F1、F4 系列，H7 最大的区别的还是 MPU 和 Cache 的配置，这两个配置会贯穿整个教程为大家做讲解，而外设的学习大同小异。
- H7 携带了 DTCM 和 ITCM，ITCM 用于运行指令，也就是程序代码，DTCM 用于数据存取，特点是跟内核速度一样，而片上的其它 RAM 主频都是 200MHz，所以要充分发挥 TCM 的性能。
- STM32H7 系列只有 HAL 库，没有再配套标准库了，这点要知道。HAL 库的优点是配置方便，特别是配合图形开发工具 STM32CubeMX 时，缺点是源代码稍显臃肿，封装的有点多。
- 这几年涌现出好几款非常棒的调试工具（如 Event Recoder，SEGGER RTT），教程的后面章节会为大家做讲解。

1.2 开发环境说明

- IDE：支持两种 IDE 开发环境，MDK 和 IAR
  - MDK 推荐使用 MDK5.26 正式版及其以上版本。
  - IAR 固定使用 IAR8.3 版本，由于 IAR 向下兼容性稍差，其它版本未做支持。
- 调试器使用 JLINK，CMSIS-DAP，ULINK 或者 STLINK 均可。
- 配套开发板是安富莱的：STM32-V7 开发板，MCU 是 STM32H743XIH6。

1.3 STM32H7 和 F1，F4 系列的区别

使用 STM32H7 跟 F1，F4 系列的区别。

- 最大的区别 H7 是多了一个 L1 Cache 一级缓存，这个缓存在为低速存储器带来加速的同时，也为程序设计带来了一些问题，其中最为主要的是数据一致性问题。
- STM32H7 的自带外设比较之前的任何 STM32 型号都要生猛，算是大换血了，换了 ADC，DMA，USART 等重要外设，性能比之前要强劲很多。比如 ADC 换成了 3.6Msps 16 位分辨率，DMA 支持任意互联了，USART 也支持波特率自适应。
- 到了 STM32H7 系列，ST 官方仅提供了 HAL 库，没有再提供标准库，而对于 F1，F4 系列，标准库和 HAL 库都是有的。
- F1 是 M3 内核，F4 是 M4 内核，而 STM32H7 是 M7 内核，从编程的角度来说，几乎没有区别。而性能上区别可以看此贴：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=21850

总的来说，主要有上面这四点不同，其它地方与使用 F1，F4 系列是相同的。

1.4 STM32H7 开发资源查找

学习一款新的芯片，需要大家从官方获取两方面的资料，一个是相关的技术文档，比如参考手册、数据手册、应用笔记等；另一个是软件包，官方在软件包中提供了外设驱动库和基于此库的大量例程。

1.4.1 开发文档

学习STM32H7主要下载哪些相关手册呢？主要有以下几个，这几个手册是我们经常要使用到的，不光学习STM32需要这类手册，学习FPGA、DSP也是这些类型的手册，熟练查阅和使用这些手册也是电子工程师必备的知识之一。

- 参考手册（Reference Manual）

对芯片每个外设的具体描述和功能介绍，比如我们要查 USART，SPI，DMA 相关寄存器和功能的介绍就可以使用这个手册。

- 数据手册（Data Sheet）

在我们要画 PCB 的时候用到这个手册的情况比较多，这个手册上面有关于这个系列芯片的引脚定义、电气特性、机械封装、料号定义等信息。

- 勘误手册（Errata Sheet）

描述了芯片某些功能的局限性，并给出解决办法。这个手册也比较重要，有时候我们觉得有些地方调试老是出问题，就需要查找一下，看看是否是硬件 bug。

- 闪存编程手册（Flash Programming Manual）

芯片的片上 Flash 操作指南，比如芯片的擦除，编程，闪存读写保护，选项字节信息等。

- 内核编程手册（Cortex-M Programming Manual）

对内核的系统控制块的介绍。这个手册有时候也要用到，比如我们需要了解 NVIC 和 SysTick 相关的寄存器，就需要使用这个手册。这个手册可以在 ARM 官方网站下载，也可以到 ST 官网下载，区别是 ARM 官网下载的手册是通用的，而 ST 的是针对自家芯片做的。有时候在参考手册上面找不到相关寄存器的信息时，就需要用到这个手册。

- 应用笔记（Application Note）

针对不同应用主题的描述性文档，部分笔记还会有配套的固件例程。应用笔记的重要性不言而喻，很多时候官方对一些应用做出了解决方案，都会以应用笔记的形式发布。

- 用户手册（User Manual）

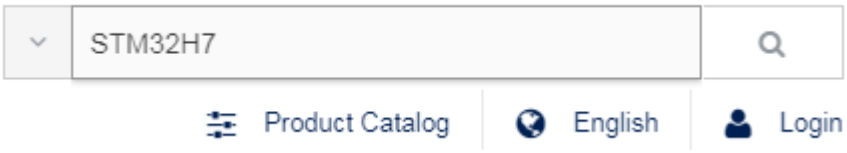
一般是对某个软件库的说明文档。

- Cortex-M3/M4 权威指南

这也是非常重要的参考资料，对于有兴趣了解 M3/M4 内核的同学，这个资料相当重要，了解了内核才能更好的利用 M3/M4。虽然是 STM32H7 是 M7 内核，大部分内容跟 M3/M4 都是一样的。论坛下载地址：http://www.armbbs.cn/forum.php?mod=viewthread&tid=2161 。

了解了这些手册的作用以后，我们学习如何在官网上面查找这些文档。前几年 ST 官方升级后，通过页面超链接的方式查找非常不方便，当前推荐直接在官方右上角的方框里面检索即可，比如使用的是 STM32H7，直接输入 STM32H7 检索：

- 第 1 步：进入 ST 官方地址 www.st.com，右上角输入 STM32H7 检索。



- 第 2 步：检索后会弹出所有 STM32H7 的芯片（随着时间推移，会增加新型号）。

Part Number	Status	Description
STM32H743XI	Active	High-performance and DSP with DP-FPU, ARM Cortex-M7 MCU with 2MBytes Flash, 1MB RAM, 400 MHz CPU, Art Accelerator, L1 cache, external memory interface, large set of peripherals
STM32H743ZI	Active	High-performance and DSP with DP-FPU, ARM Cortex-M7 MCU with 2MBytes Flash, 1MB RAM, 400 MHz CPU, Art Accelerator, L1 cache, external memory interface, large set of peripherals
STM32H753XI	Active	High-performance and DSP with DP-FPU, ARM Cortex-M7 MCU with 2MBytes Flash, 1MB RAM, 400 MHz CPU, Art Accelerator, L1 cache, external memory interface, large set of peripherals including a Crypto accelerator
STM32H753ZI	Active	High-performance and DSP with DP-FPU, ARM Cortex-M7 MCU with 2MBytes Flash, 1MB RAM, 400 MHz CPU, Art Accelerator, L1 cache, external memory interface, large set of peripherals including a Crypto accelerator
STM32H743AI	Active	High-performance and DSP with DP-FPU, ARM Cortex-M7 MCU with 2MBytes Flash, 1MB RAM, 400 MHz CPU, Art Accelerator, L1 cache, external memory interface, large set of peripherals
STM32H753AI	Active	High-performance and DSP with DP-FPU, ARM Cortex-M7 MCU with 2MBytes Flash, 1MB RAM, 400 MHz CPU, Art Accelerator, L1 cache, external memory interface, large set of peripherals including a Crypto accelerator
STM32H753BI	Active	High-performance and DSP with DP-FPU, ARM Cortex-M7 MCU with 2MBytes Flash, 1MB RAM, 400 MHz CPU, Art Accelerator, L1 cache, external memory interface, large set of peripherals including a Crypto accelerator
STM32H743BI	Active	High-performance and DSP with DP-FPU, ARM Cortex-M7 MCU with 2MBytes Flash, 1MB RAM, 400 MHz CPU, Art Accelerator, L1 cache, external memory interface, large set of peripherals
STM32H743II	Active	High-performance and DSP with DP-FPU, ARM Cortex-M7 MCU with 2MBytes Flash, 1MB RAM, 400 MHz CPU, Art Accelerator, L1 cache, external memory interface, large set of peripherals
STM32H743VI	Active	High-performance and DSP with DP-FPU, ARM Cortex-M7 MCU with 2MBytes Flash, 1MB RAM, 400 MHz CPU, Art Accelerator, L1 cache, external memory interface, large set of peripherals
STM32H753VI	Active	High-performance and DSP with DP-FPU, ARM Cortex-M7 MCU with 2MBytes Flash, 1MB RAM, 400 MHz CPU, Art Accelerator, L1 cache, external memory interface, large set of peripherals including a Crypto accelerator
STM32H753II	Active	High-performance and DSP with DP-FPU, ARM Cortex-M7 MCU with 2MBytes Flash, 1MB RAM, 400 MHz CPU, Art Accelerator, L1 cache, external memory interface, large set of peripherals including a Crypto accelerator

◆ 第 3 步：比如我们用的是 STM32H743XI，点击进入第 2 步截图中的 STM32H743XI 选项即可。进入后，所有的相关资源就都在这个页面下了。

数据手册标识：

Product Specifications

	Description
	DS12110: 32-bit Arm® Cortex®-M7 400MHz MCUs, up to 2MB Flash, 1MB RAM, 46 com. and analog interfaces

应用笔记标识：

Application Notes

	Description
	AN3126: Audio and waveform generation using the DAC in STM32 microcontrollers

参考手册标识：

Reference Manuals	
	Description
	RM0433: STM32H7x3 advanced ARM®-based 32-bit MCUs

编程手册标识：

Programming Manuals	
	Description
	PM0253: STM32F7 Series and STM32H7 Series Cortex®-M7 processor programming manual

勘误手册标识：

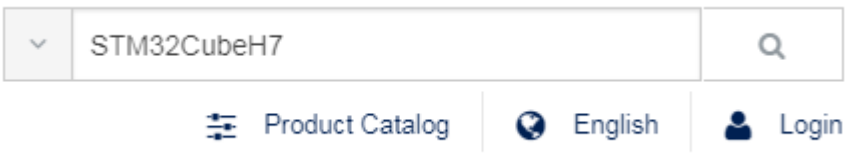
Errata Sheets	
	Description
	ES0392: STM32H743xI rev Y device limitations

基本上大家所需的开发文档都在这个页面下了。

1.4.2 软件包

通过上面小节整理完毕相关文档后，就是 STM32H7 软件包的下载了。软件包也比较好找，同样推荐 1.4.1 小节的方式。

◆ 第 1 步：进入 ST 官方地址 www.st.com ，右上角输入 STM32CubeH7 检索。



◆ 第 2 步：检索后会弹出 STM32CubeH7 的软件包选项。

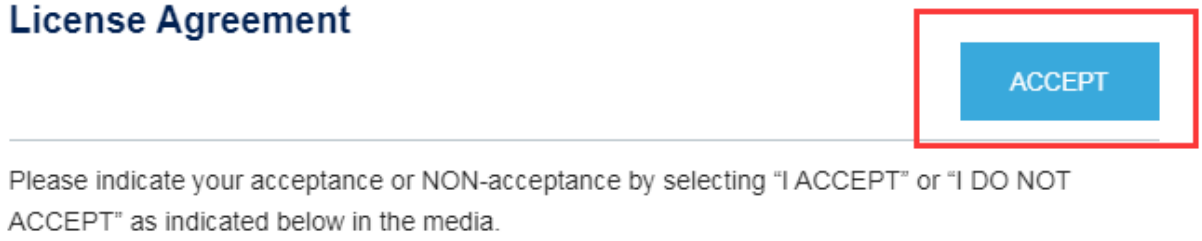
Part Number	Type	Category	Description
STM32CubeH7 这是超链接，点击可进入	Embedded Software	MCUs Embedded Software	STM32Cube MCU Package for STM32H7 series (HAL low level drivers, USB, TCP/IP, File system, RTOS, Graphic - coming with examples running on ST boards: STM32 Nucleo, Discovery kits and Evaluation boards)

◆ 第 3 步：进入 STM32CubeH7 的页面地址后，就在页面的最底端看到这个软件包了（随着时间推移，版本会不断更新）。

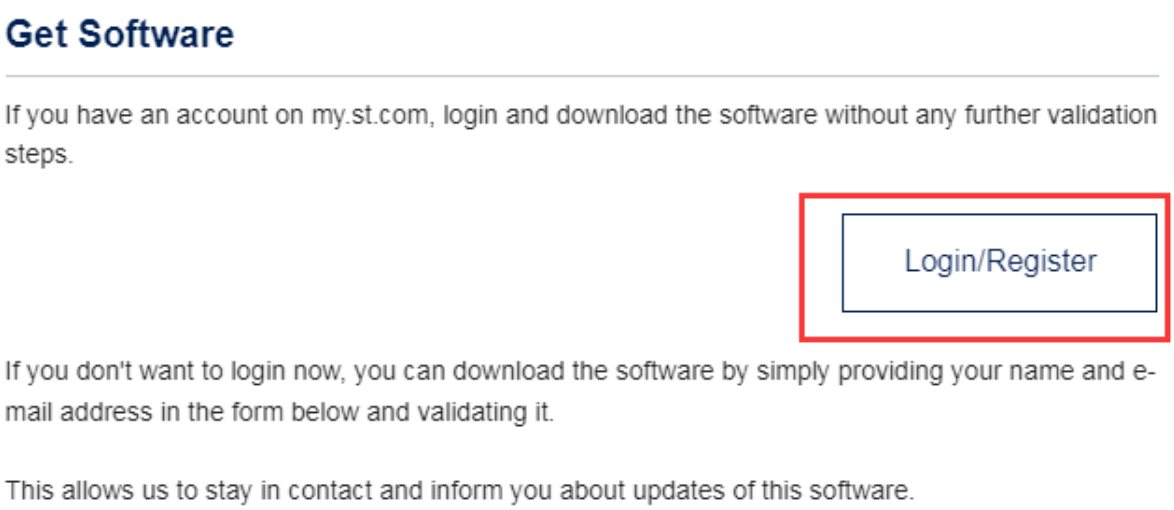
Part Number	Software Version	Marketing Status	Supplier	Order from ST
STM32CubeH7	1.2.0	Active	ST	Get Software

点击这个按钮可以获取

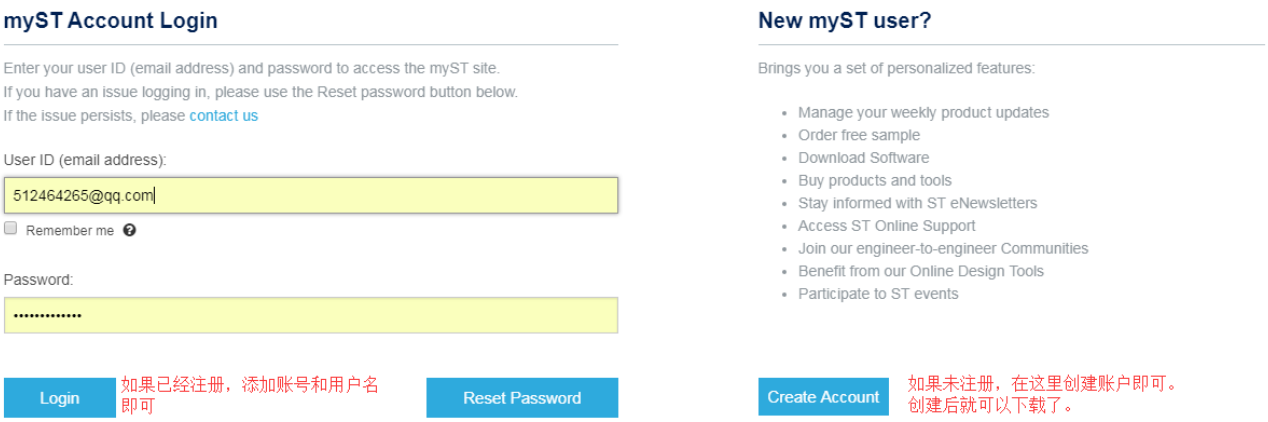
◆ 第 4 步：点击按钮“Get Software”后，弹出如下界面



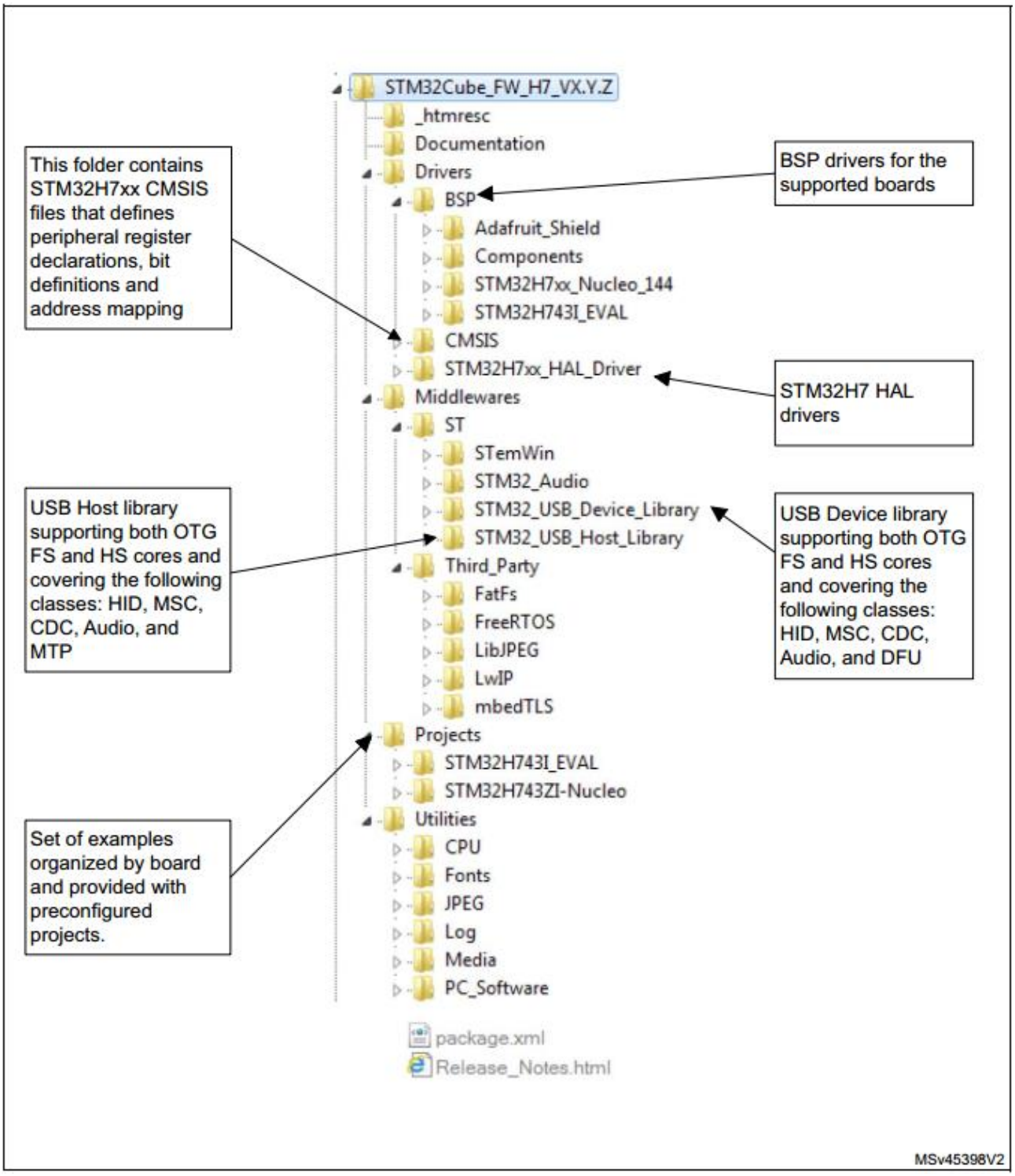
点击 ACCEPT 进入下面界面：



点击 Login/Register

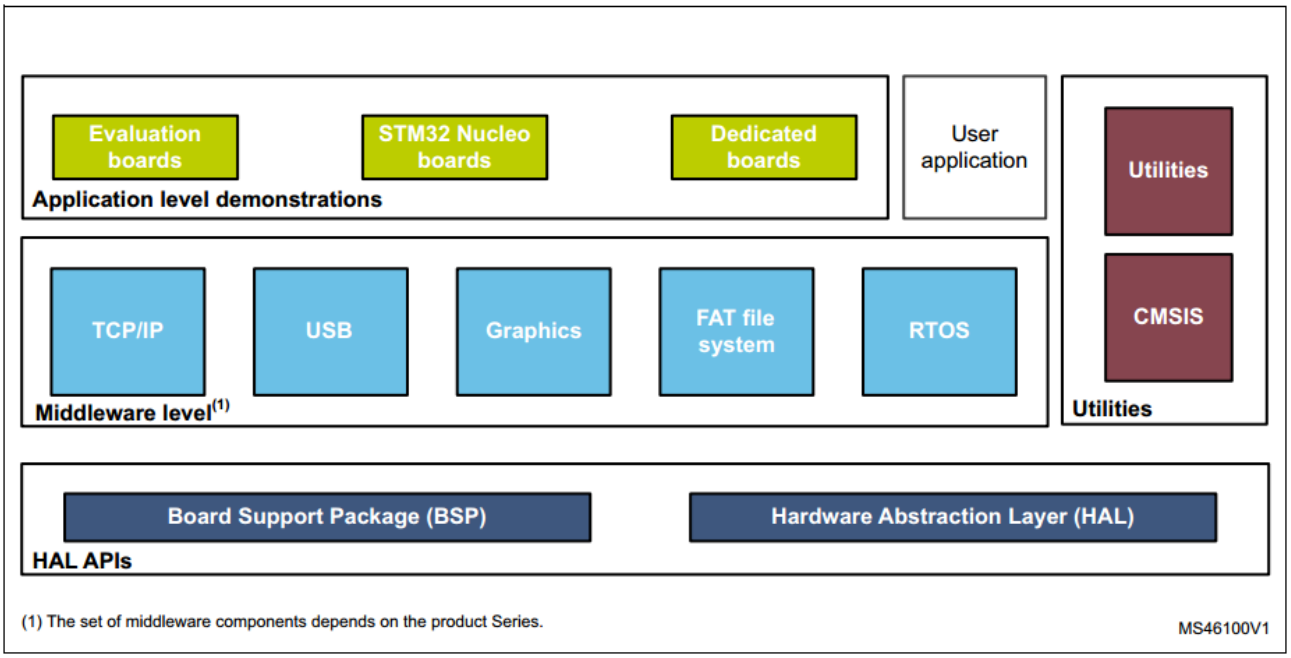


通过上面四步就获取了 STM32H7 的软件包。软件包的目录结构如下：

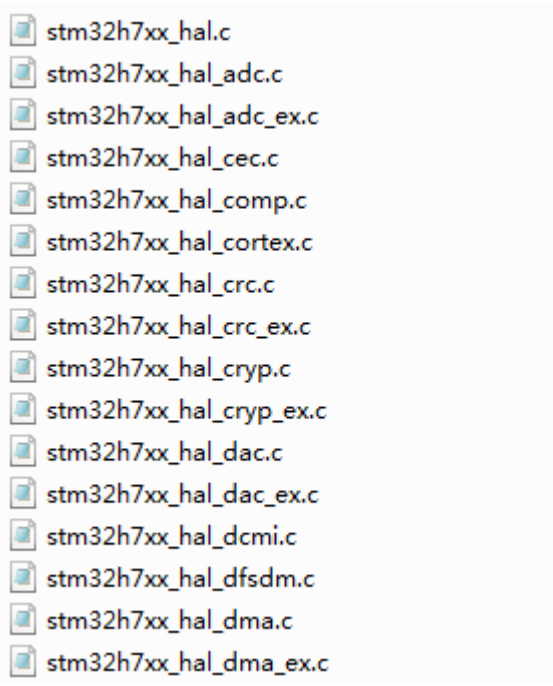


1.5 HAL 库介绍

HAL 库就包含在大家下载的 STM32CubeH7 软件包里面。软件包的框图如下：



HAL 库全称 Hardware Abstraction Layer，即硬件抽象层，其实就是 STM32H7 的外设驱动包。代码文件位于路径：Drivers\STM32H7xx_HAL_Driver。如下是部分截图：



单从人性化角度，这些外设驱动写的还是比较用心的，特别是每个 C 文件开头的使用说明。比如文件 stm32h7xx_hal_gpio.c 开头的说明：

```
  ==============================================================================
                ##### GPIO Peripheral features #####
  ==============================================================================
  [..]
  Subject to the specific hardware characteristics of each I/O port listed in the datasheet, each
  port bit of the General Purpose IO (GPIO) Ports, can be individually configured by software
  in several modes:
  (+) Input mode
```

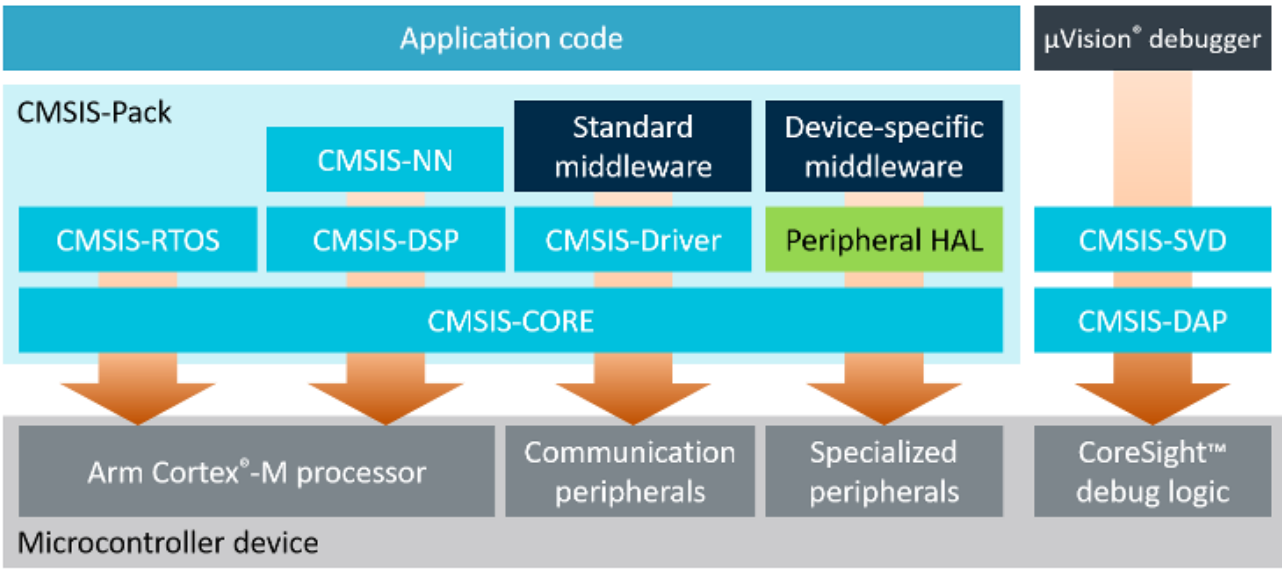
```
  (+) Analog mode
  (+) Output mode
  (+) Alternate function mode
  (+) External interrupt/event lines

  [..]
  During and just after reset, the alternate functions and external interrupt
       lines are not active and the I/O ports are configured in input floating mode.

  [..]
  All GPIO pins have weak internal pull-up and pull-down resistors, which can be
       activated or not.

  [..]
  In Output or Alternate mode, each IO can be configured on open-drain or push-pull
       type and the IO speed can be selected depending on the VDD value.

  [..]
  All ports have external interrupt/event capability. To use external interrupt
       lines, the port must be configured in input mode. All available GPIO pins are
       connected to the 16 external interrupt/event lines from EXTI0 to EXTI15.

  [..]
  The external interrupt/event controller consists of up to 23 edge detectors
       (16 lines are connected to GPIO) for generating event/interrupt requests (each
       input line can be independently configured to select the type (interrupt or event)
       and the corresponding trigger event (rising or falling or both). Each line can
       also be masked independently.

                    ##### How to use this driver #####
  ==============================================================================
  [..]
    (#) Enable the GPIO AHB clock using the following function: __HAL_RCC_GPIOx_CLK_ENABLE().

    (#) Configure the GPIO pin(s) using HAL_GPIO_Init().
        (++) Configure the IO mode using "Mode" member from GPIO_InitTypeDef structure
        (++) Activate Pull-up, Pull-down resistor using "Pull" member from GPIO_InitTypeDef
             structure.
        (++) In case of Output or alternate function mode selection: the speed is
             configured through "Speed" member from GPIO_InitTypeDef structure.
        (++) In alternate mode is selection, the alternate function connected to the IO
             is configured through "Alternate" member from GPIO_InitTypeDef structure.
        (++) Analog mode is required when a pin is to be used as ADC channel
             or DAC output.
        (++) In case of external interrupt/event selection the "Mode" member from
             GPIO_InitTypeDef structure select the type (interrupt or event) and
             the corresponding trigger event (rising or falling or both).

    (#) In case of external interrupt/event mode selection, configure NVIC IRQ priority
        mapped to the EXTI line using HAL_NVIC_SetPriority() and enable it using
        HAL_NVIC_EnableIRQ().

    (#) To get the level of a pin configured in input mode use HAL_GPIO_ReadPin().

    (#) To set/reset the level of a pin configured in output mode use
        HAL_GPIO_WritePin()/HAL_GPIO_TogglePin().

   (#) To lock pin configuration until next reset use HAL_GPIO_LockPin().
```

```
(#) During and just after reset, the alternate functions are not
    active and the GPIO pins are configured in input floating mode (except JTAG
    pins).

(#) The LSE oscillator pins OSC32_IN and OSC32_OUT can be used as general purpose
    (PC14 and PC15, respectively) when the LSE oscillator is off. The LSE has
    priority over the GPIO function.

(#) The HSE oscillator pins OSC_IN/OSC_OUT can be used as
    general purpose PH0 and PH1, respectively, when the HSE oscillator is off.
    The HSE has priority over the GPIO function.
```

HAL 库的使用方法跟之前 F1，F4 系列的标准库差不多，只是 HAL 库封装的稍显臃肿。事情都是两面的，代码臃肿了，易用性会好些。

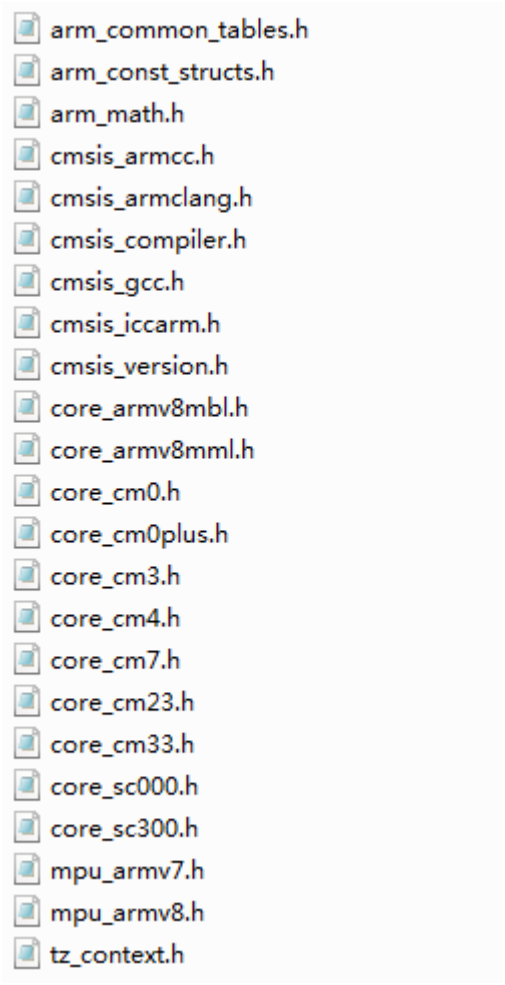
1.6 CMSIS 软件包介绍

CMSIS（微控制器软件接口标准，Cortex Microcontroller Software Interface Standard）是 ARM 官方设计的驱动包，框图如下：



ARM 推出 CMSIS 软件包意在统一各大芯片厂商的外设驱动，DSP 数字信号处理，下载器和各个主流 RTOS 的 API 统一。几年下来，各个厂商一直是各自为战，所以 CMSIS 的驱动一直没有被各个芯片厂商采用。而且 ARM 做得也不够完善，没有 ADC、DAC、定时器之类的外设驱动。

这两年情况好了不少，特别是 ARM 为 ST 做的 CMSIS-Driver 明显完善了很多。针对我们这个教程来说，当前还用不到这些东西，主要用到 CMSIS 软件包里面的如下头文件即可（不同版本，截图中的文件可能不同，这个软件包是一直在更新中的，下面的截图的版本是 V5.3.0）：



这个软件包可以在三个地方获取：

- STM32CubeH7 软件包里面。

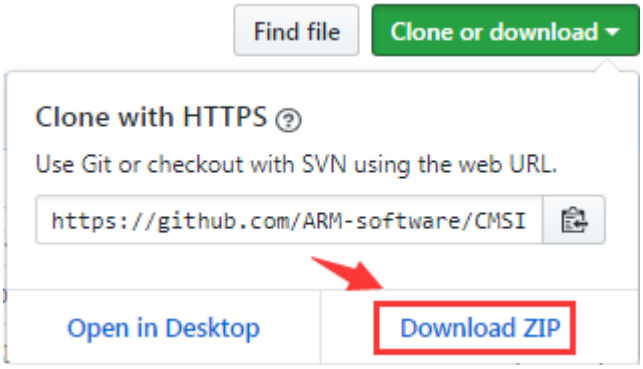
  每个版本的 Cube 软件包都会携带 CMSIS 文件夹。

- MDK 安装目录（下面是 5.3.0 版本的路径）。

  大家安装了新版 MDK 后，CMSIS 软件包会存在于路径：ARM\PACK\ARM\CMSIS\5.3.0\CMSIS。

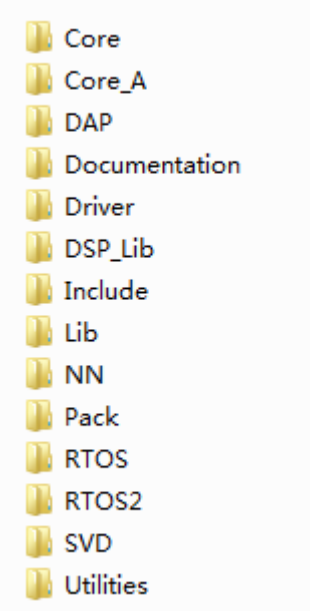
- GitHub。

  通过 GitHub 获取也比较方便，地址：https://github.com/ARM-software/CMSIS_5 。点击右上角就可以下载 CMSIS 软件包了。



当然，也可以在 ARM 官网下载，只是这两年 ARM 官网升级得非常频繁，通过检索功能找资料非常麻烦。所以不推荐大家到 ARM 官网下载资料了。

下面为大家简单介绍下 CMSIS 软件包里面这几个文件夹：



- **Core**

  Cortex-M 处理器内核和外设的 API。 它为 Cortex-M0，Cortex-M0 +，Cortex-M3，Cortex-M4，Cortex-M7，Cortex-M23，Cortex-M33，SC000 和 SC300 提供了标准化接口。 还包括用于 Cortex-M4，Cortex-M7 和 Cortex-M33 的 SIMD 指令。当前这个文件下只有一个示例文件，还用不上。

- **Core_A**

  同上，只是用于 Cortex-A5/A7/A9。

- **DAP**

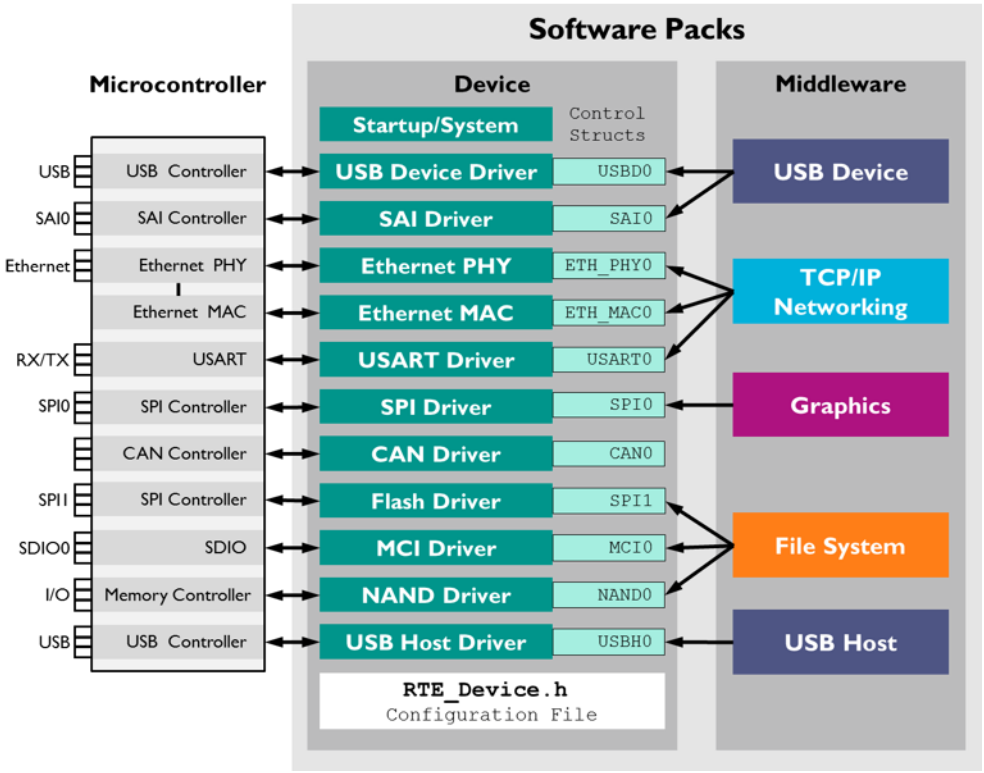
  这个是 ARM 官方推出的下载器固件，也就是大家所说的 CMSIS-DAP 下载器。

- **Documentation**

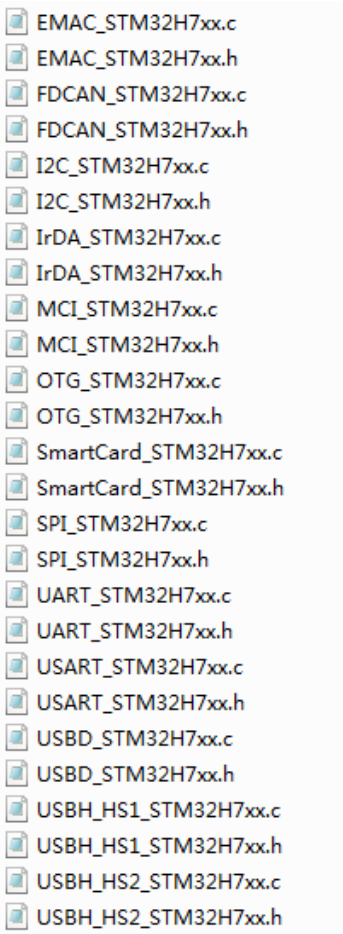
  这个是 CMSIS 软件包的 Help 文档，打开后效果如下：

- **Driver**

  这个是 ARM 做好的驱动框架，支持的外设如下：



针对不同厂商，ARM 会出一个完整的驱动包，比如 STM32H7 系列，在 MDK 安装目录的此路径下（前提是大家安装了 STM32H7 软件包）：ARM\PACK\Keil\STM32H7xx_DFP\2.1.0\CMSIS\Driver。



ARM 做的这个驱动跟 HAL 库有什么区别呢？ARM 做的这个库要调用到 HAL 的一些 API，然后封装了一些比较好用的 API，方便用户调用。

- **DSP_Lib**

  这个是 ARM 提供的 DSP 库，此库支持以 CM0、CM3、CM4 以及 CM7 为内核的所有 MCU，含源码。详细介绍可以看我们的 DSP 教程：

  http://www.armbbs.cn/forum.php?mod=viewthread&tid=3886 。

- **Include**

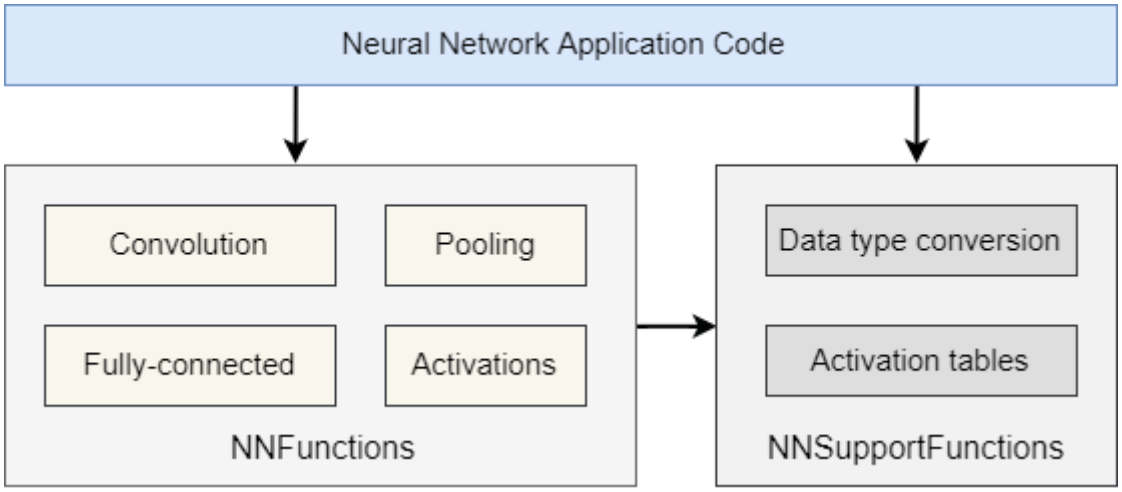
  这个文件比较重要，虽然是头文件，但是封装了很多内核方面的 API，是大家工程里面务必包含的路径。

- **Lib**

  这个文件是 GCC 和 MDK 格式的 DSP 库文件。

- **NN**

  这个是 ARM 新出的神经网络库，框图如下：



- **Packs**

  这个文件没什么用，大家不用管。

- **RTOS**

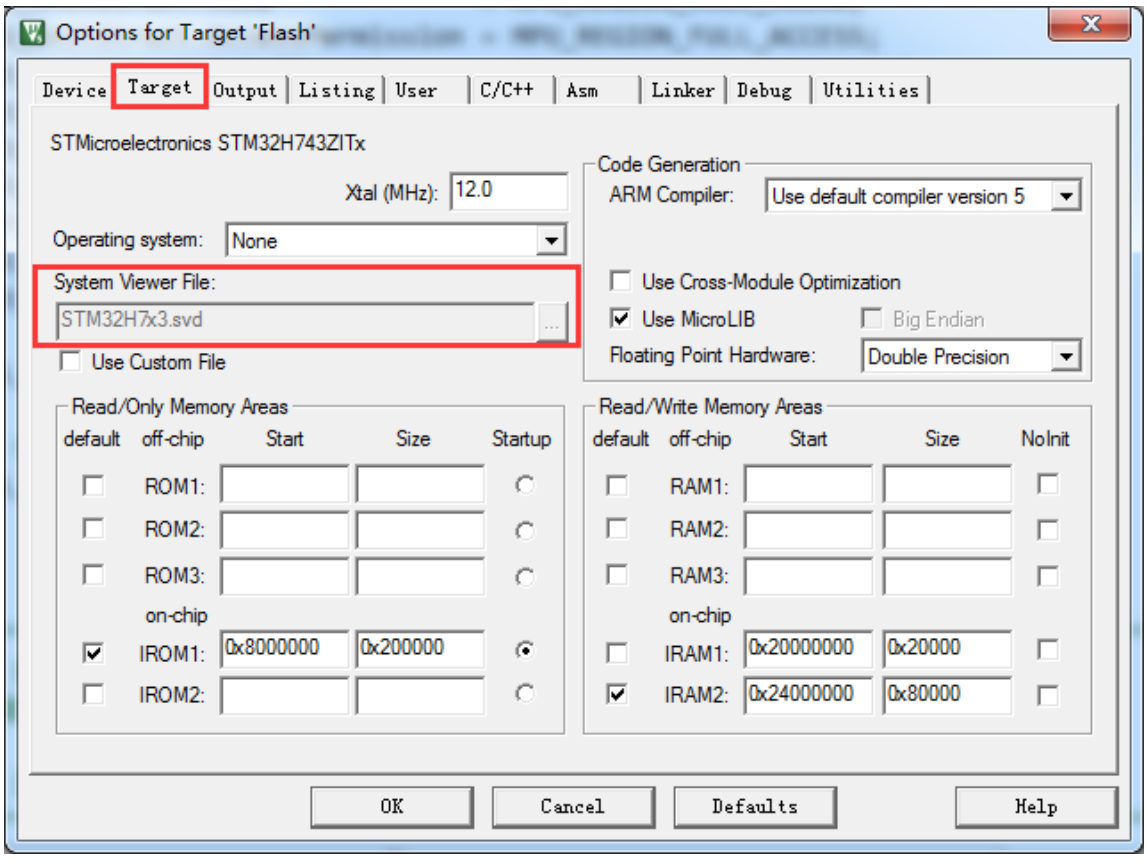
  这个是 RTX4 以及 CMSIS-RTOS V1 封装层，含源码，免费，Apache-2.0 授权。

- **RTOS2**

  这个是 RTX5 以及 CMSIS-RTOS V2 封装层，含源码，免费，Apache-2.0 授权。

- **SVD**

  SVD 的全称是 System View Description，系统视图描述。对芯片的外设、存储器等进行了详细描述，编译器要用到这个文件，不同系列芯片有不同的 SVD 文件。以 STM32H7 为例，在 MDK 的 option 选项里面可以看到以 svd 为后缀的文件被调用。



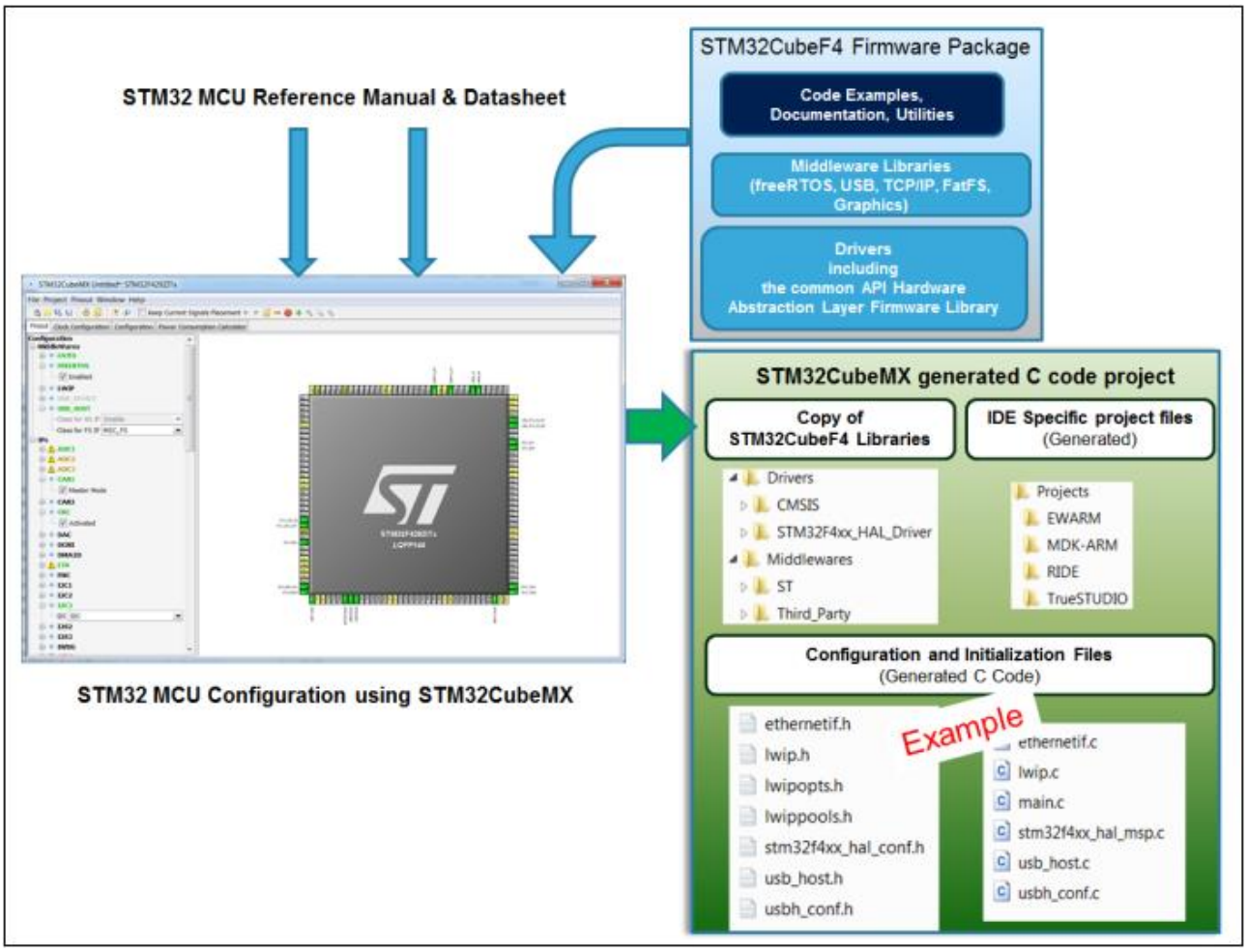
◆ **Utilities**

这个文件里面提供了一些实用的小软件或者文件。

关于 CMSIS 软件包就为大家介绍这么多，后面用到哪个文件时，再为大家详细介绍。

1.7 STM32CubeMX 开发平台

STM32CubeMX 是 ST 在 2014 年推出的图形开发软件，方便用户配置时钟、外设、引脚以及 RTOS 和各种中间件。整体框图如下：



通过这个图形软件，可以让大家方便地生成工程代码，支持 MDK，IAR，TrueSTUDIO 等编译器。针对 STM32CubeMX 的使用，后面会专门做几期专题教程。

1.8 STM32H7 调试方法

STM32H7 的调试方法主要分为两大类：

- MDK 和 IAR 编译器自带的调试

  MDK 调试方法在第 5 章进行了详细讲解。

  IAR 调试方法在第 7 章进行了详细讲解。

- 终极调试组件 Event Recoder 的使用方法。

  在 8 章节进行了详细讲解。

1.9 STM32H7 出现硬件异常的解决办法

大家做项目时，经常会遇到硬件异常问题，所以专门为此做了一个章节（具体在 11 章节进行了详细讲解）。

1.10 总结

本章节就为大家讲解这么多，建议初学者花些时间对 STM32H7 的开发文档的章节结构了解一下，随着以后的学习最好可以达到熟练查看这些开发文档的程度。

第2章 STM32H7 的开发环境搭建

本章主要为大家讲解 V7 开发板的开发板环境搭建，涉及 IDE，下载器和相关小软件的驱动安装。

2.1 初学者重要提示
2.2 常用的开发工具
2.3 MDK5 的安装和注册
2.4 IAR8 的安装和注册
2.5 JLINK 驱动安装
2.6 STLINK 驱动安装
2.7 CH340 驱动安装（RS232 转 USB）
2.8 串口软件（终端仿真软件 SecureCRT）
2.9 三合一下载软件 STM32CubeProgrammer
2.10 硬件开发环境搭建好的效果
2.11 总结

2.1 初学者重要提示

- 如果使用 JLINK 调试下载 STM32H7，务必使用 JLINK V9 或 V10，之前的版本已经不支持。
- 如果使用 STLINK 调试下载 STM32H7，推荐使用最新的电脑端驱动和对应的固件，详情见本章 2.6 小节。
- MDK5 的工程创建，调试方法以及字体设置，多核编译，语法提示等设置在第 4 章和第 5 章有详细说明。
- IAR8 的工程创建，调试方法以及字体设置，语法提示等设置在第 6 章和第 7 章有详细说明。

2.2 常用的开发工具

开发STM32软件，需要配置一个仿真器（也可以叫下载器），同时必须在PC机上安装一系列软件。可以使用JLINK、CMSIS-DAP、ULINK或者STLINK仿真器来下载和调试程序，虽然STM32支持串口ISP下载程序，但是失败率高，下载时间长达数分钟，开发效率太低。

序号	软件类别	可选的软件

1	集成开发环境（IDE）	**Keil公司的RealView MDK** KEIL公司的MDK的界面美观，易用性好，并且KEIL已被ARM公司收购，STM32的Crotex内核也是ARM公司设计的。 **IAR公司的EWARM** EWARM的开发商IAR公司是一个老牌IDE开发商，有很多其他单片机的IDE产品，这些IDE操作方法大体一致，适合于很多转型过来的用户。
2	源文件编辑器	**UltraEdit** UltraEdit编辑源代码非常方便，远胜过IDE内置的编辑器； **SourceInsight** 非常适合于源代码浏览，鼠标一点就可以直接看到函数变量定义。 **IDE自带的编辑器** 这是大多数人常用的方式，不过KEIL和IAR自带的编辑器使用起来不如前面两种软件方便。
3	仿真器	**J-Link** J-Link仿真器可用于MDK和EWARM，下载和跟踪调试非常方便，有独立的下载器软件J-Flash。如果开发H7需要使用J-Link V9，V10版本。 **ULINK** ULINK是KEIL公司主推的仿真器，只能用于MDK。无法在IAR下使用，没有独立的下载器软件。 **ST-Link** 这是ST公司生产的仿真器，可以调试STM8和STM32，可以在MDK和IAR下使用。也有独立的下载器软件STVP。 **CMSIS-DAP** 这是ARM搞的开源下载器方案。
4	串口工具	**SecureCRT** SecureCRT是一个专业的超级终端工具，支持很多接口（包括串口）和协议，推荐使用。 **串口助手** 串口助手之类的软件很多，偶尔用一下也可以，不过稳定性欠佳。

2.3 MDK5 的安装和注册

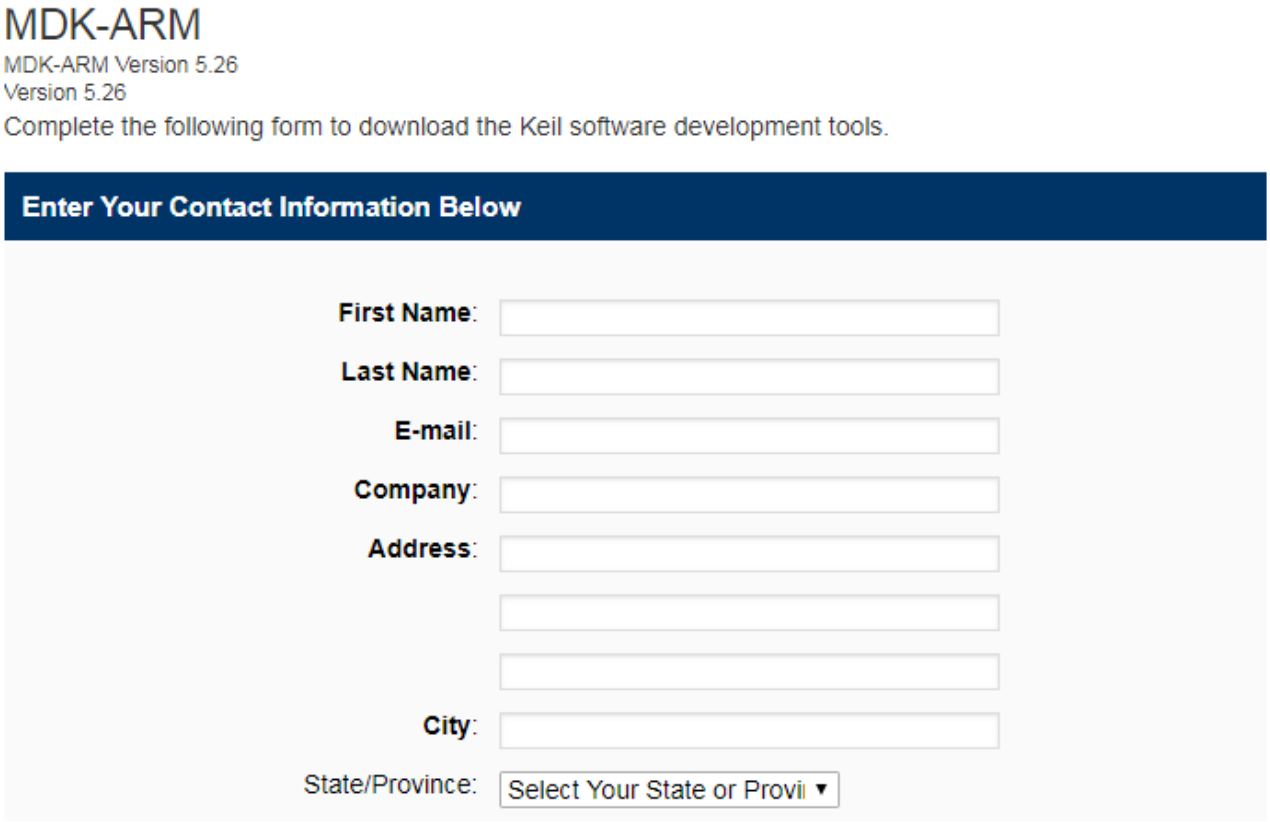
这里以 MDK5.26 为例进行说明。

2.3.1 下载 MDK5

（注，当前时间的官方版本还是 MDK5.26）

推荐直接在官方网站下载，地址：https://www.keil.com/download/product/

点击 MDK-Arm，来到下面地址后，需要大家简单填写注册信息后即可下载：

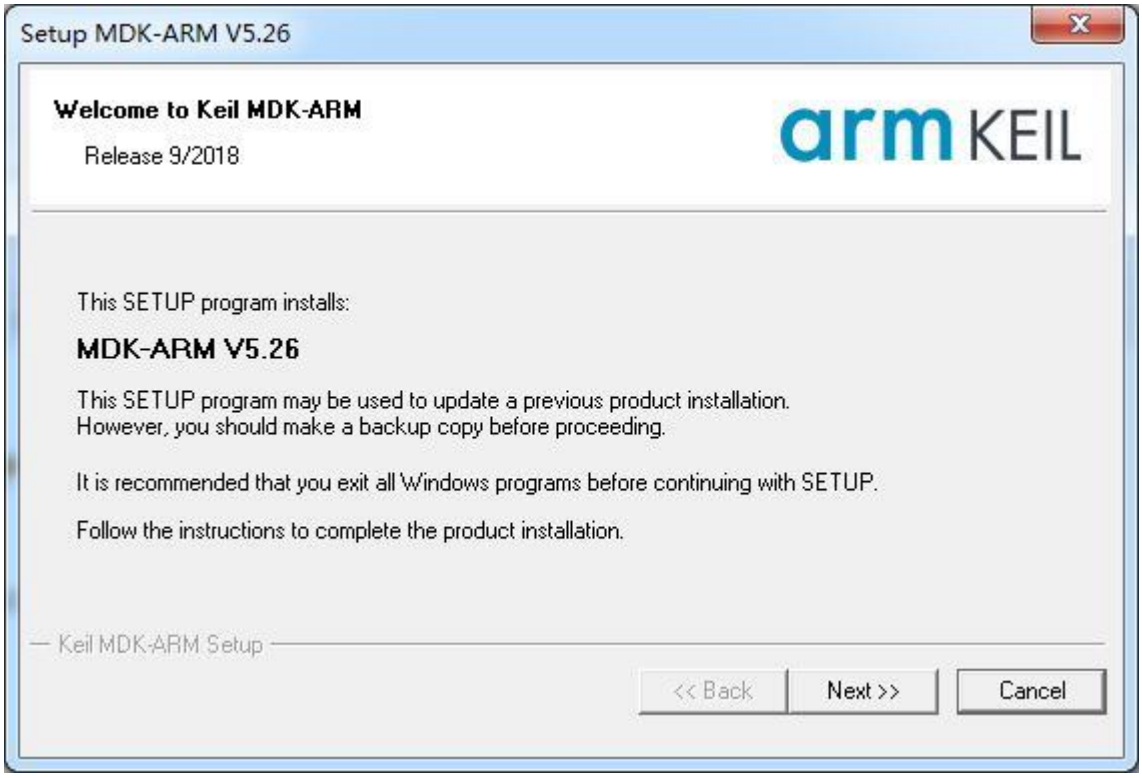


也可以直接在我们论坛下载：http://www.armbbs.cn/forum.php?mod=viewthread&tid=89403 。帖子里面直接提供了 MDK 的原始下载地址。

2.3.2 安装 MDK5

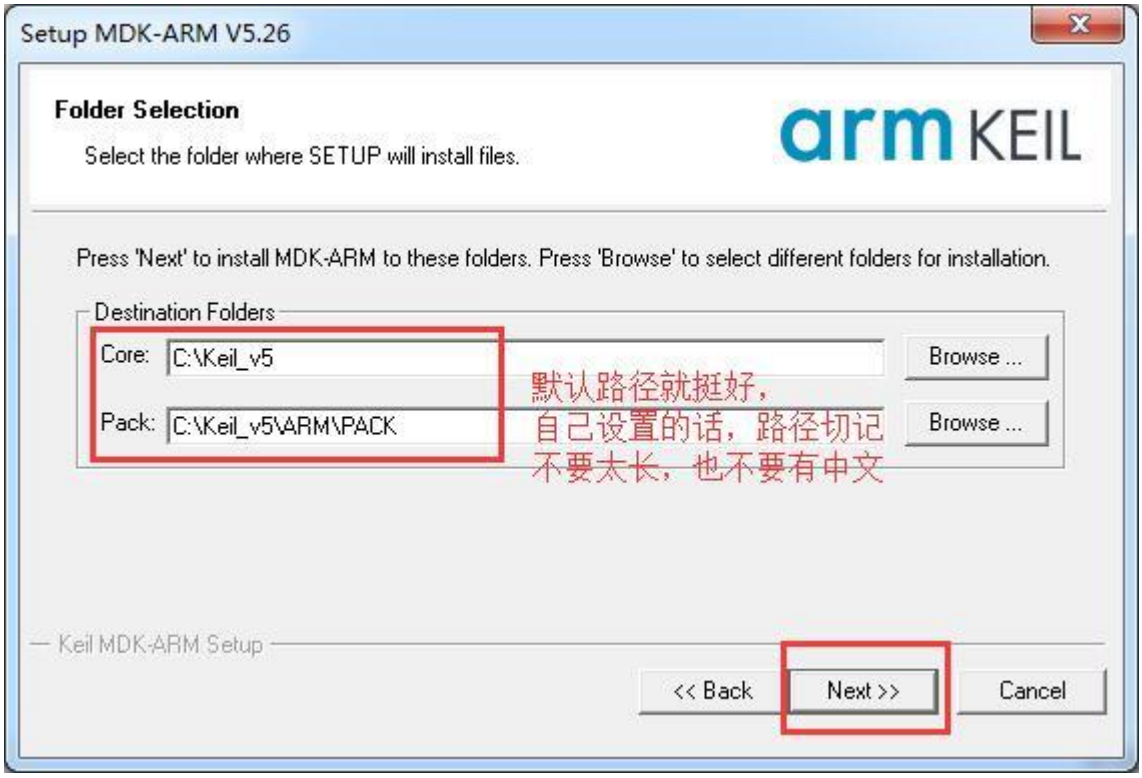
安装的时候注意不要有中文路径，路径越短越好。安装过程比较简单，一直下一步即可。

1. 下载完毕后，点击安装。



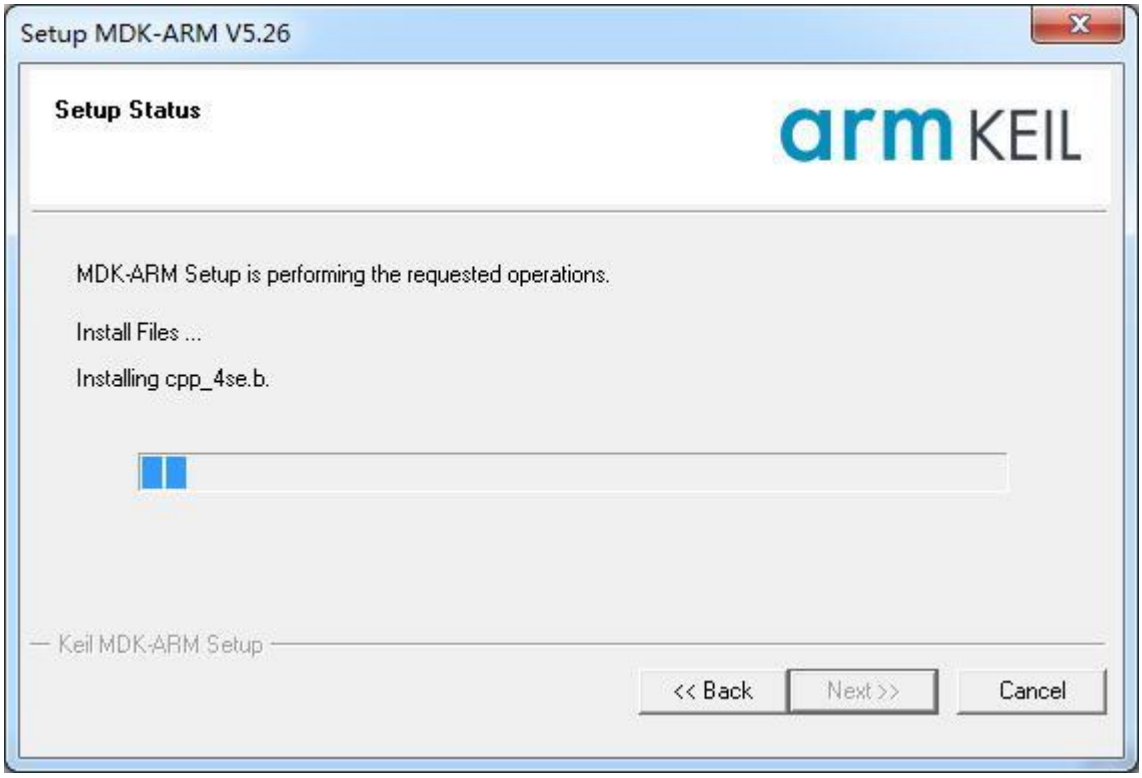
2. 勾上同意，Next。

3. 注意安装路径。



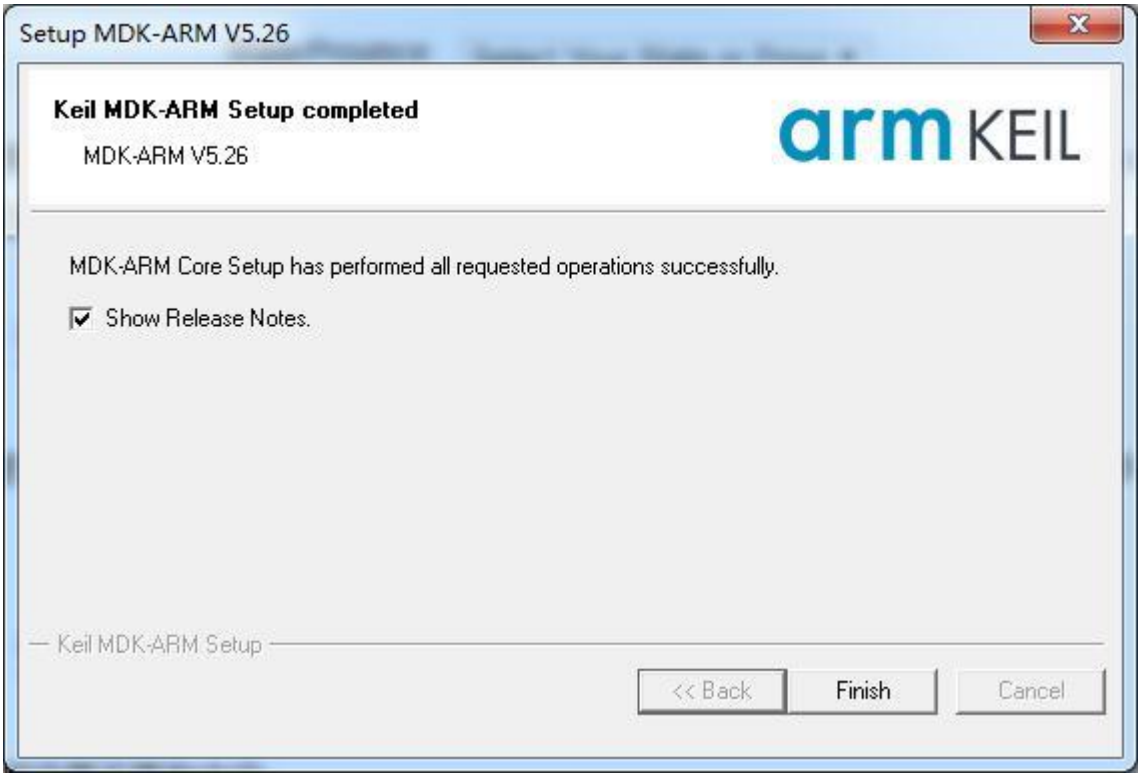
4. 红色方框的两项随意填写，Next。

5. 剩下就是时间略长的安装过程。



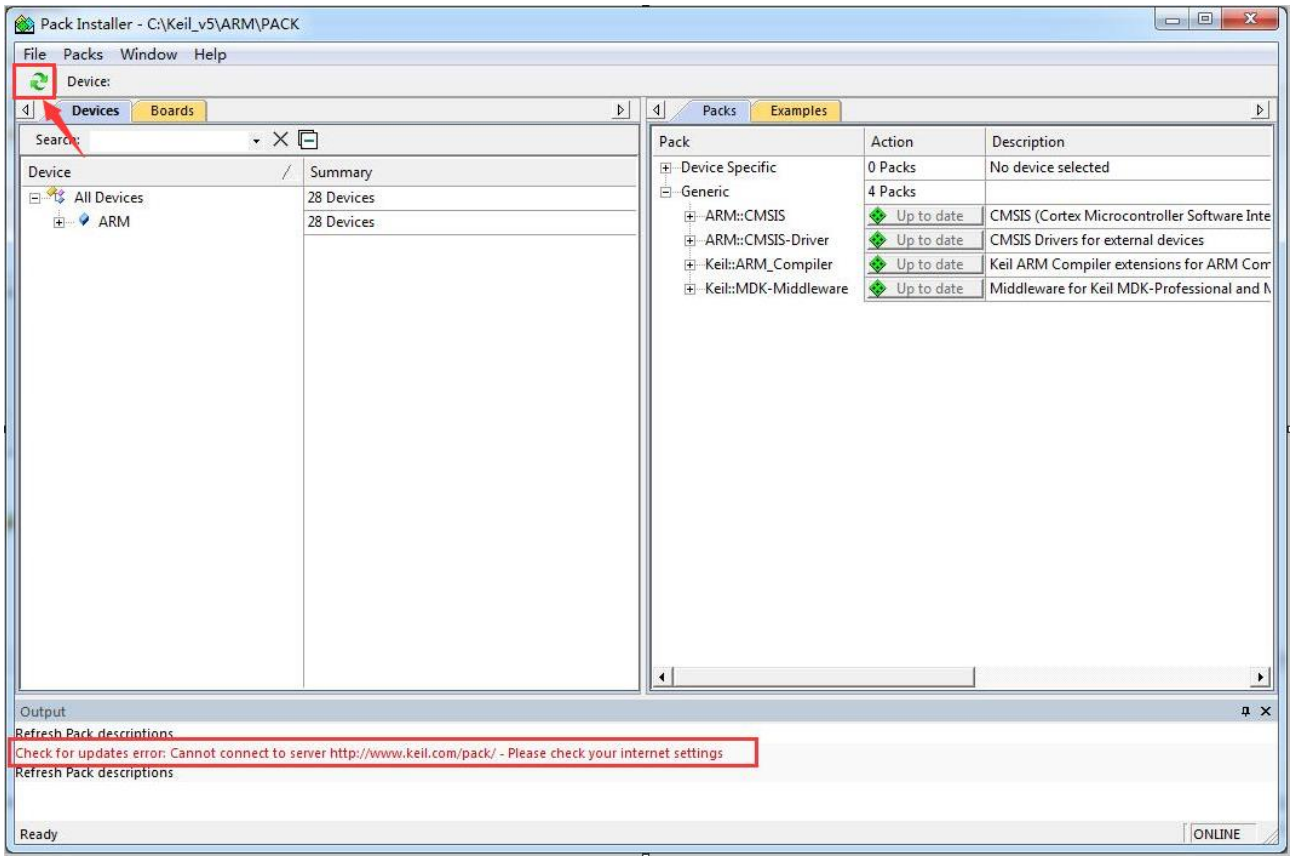
6. 安装结束前会提示是否安装 ULINK 驱动，安装即可。

7. 至此，MDK 就安装完毕了。

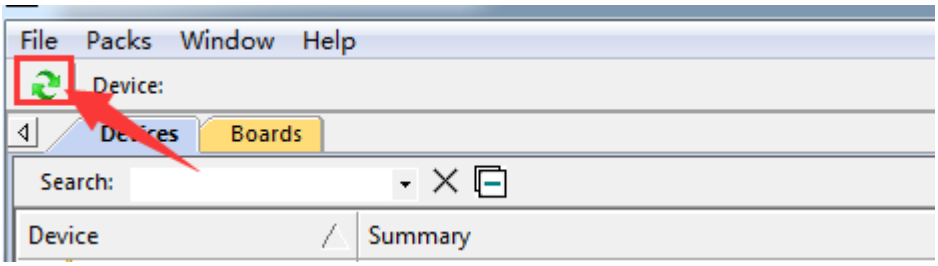


安装完毕后，如果弹出下面界面，是因为要更新安装包列表，这个要连接 MDK 服务器的。

首次打开 MDK 会弹出下面的界面，点击左上角的刷新图标，如果是下面效果，表示的确无法连接到 MDK 服务器，此时可以重启电脑试试。

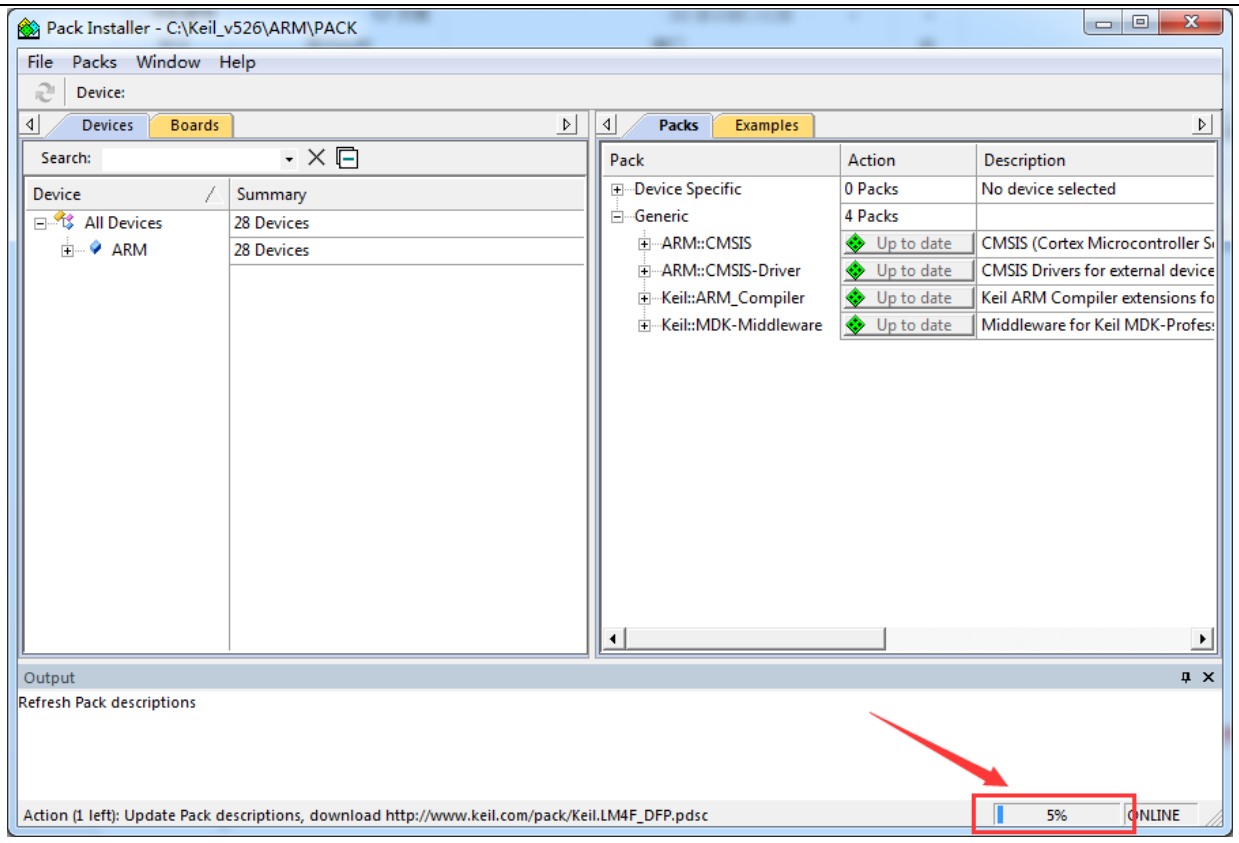


再次打开 MDK 后，点击这里弹出软件包安装界面，看看是否可以刷新成功：

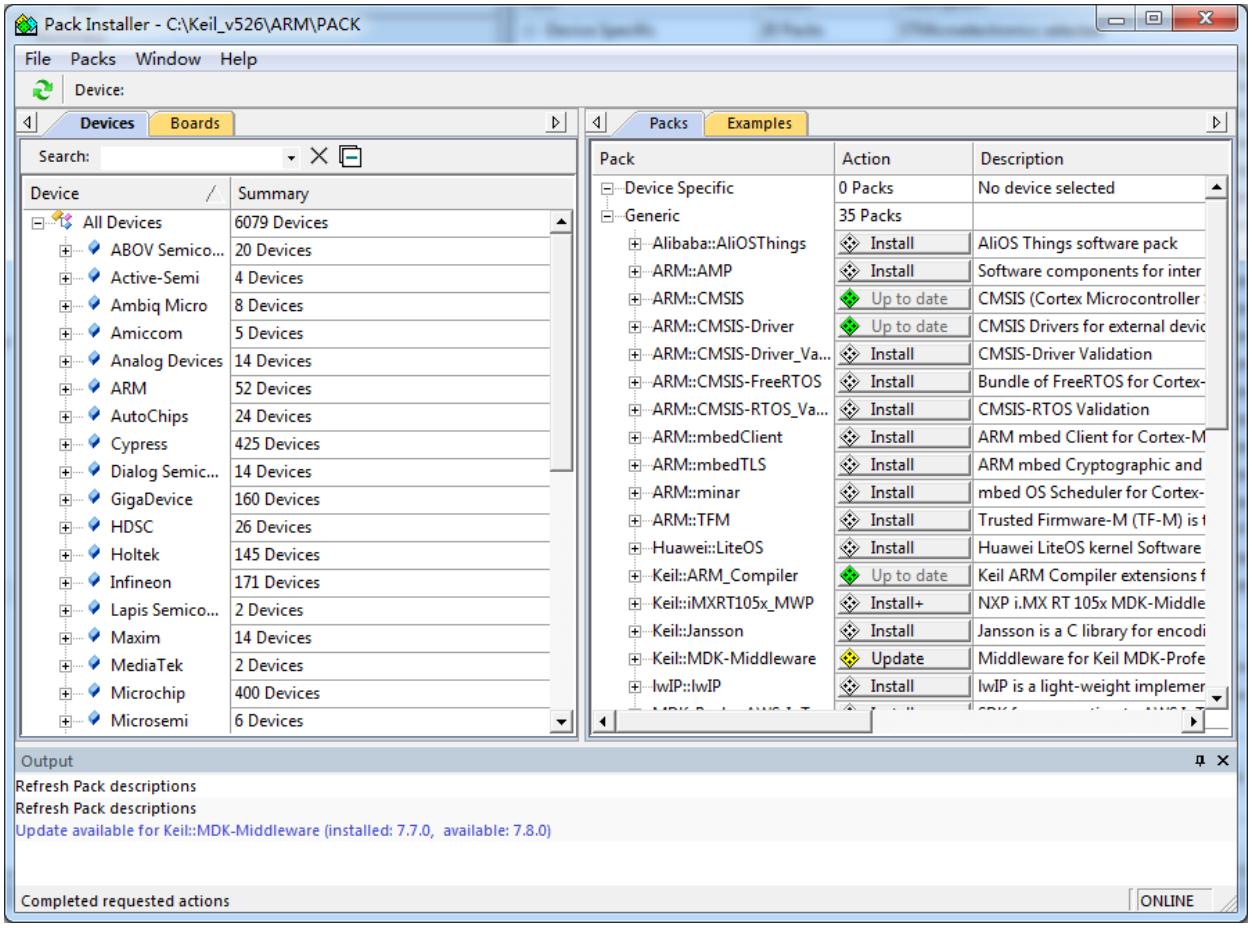


如果还是有问题的话，基本不用管了，直接去官网下载相应的软件包，然后导入即可（见下面 H7 软件包的安装）。

正常更新的时候，右下角有个更新进度：


Pack Installer - C:\Keil_v526\ARM\PACK
File Packs Window Help
Device:
Devices Boards
Search:
Device Summary
All Devices 28 Devices
ARM 28 Devices
Packs Examples
Pack Action Description
Device Specific 0 Packs No device selected
Generic 4 Packs
ARM::CMSIS Up to date CMSIS (Cortex Microcontroller S
ARM::CMSIS-Driver Up to date CMSIS Drivers for external device
Keil::ARM_Compiler Up to date Keil ARM Compiler extensions fo
Keil::MDK-Middleware Up to date Middleware for Keil MDK-Profes
Output
Refresh Pack descriptions
Action (1 left): Update Pack descriptions, download http://www.keil.com/pack/Keil.LM4F_DFP.pdsc
5%
ONLINE

更新完毕后的效果如下：

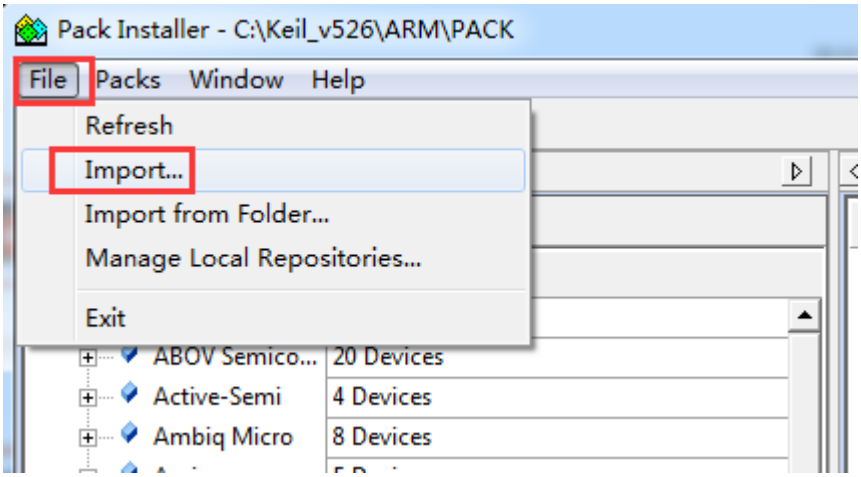

Pack Installer - C:\Keil_v526\ARM\PACK
File Packs Window Help
Device:
Devices Boards
Search:
Device Summary
All Devices 6079 Devices
ABOV Semico... 20 Devices
Active-Semi 4 Devices
Ambiq Micro 8 Devices
Amiccom 5 Devices
Analog Devices 14 Devices
ARM 52 Devices
AutoChips 24 Devices
Cypress 425 Devices
Dialog Semic... 14 Devices
GigaDevice 160 Devices
HDSC 26 Devices
Holtek 145 Devices
Infineon 171 Devices
Lapis Semico... 2 Devices
Maxim 14 Devices
MediaTek 2 Devices
Microchip 400 Devices
Microsemi 6 Devices
Packs Examples
Pack Action Description
Device Specific 0 Packs No device selected
Generic 35 Packs
Alibaba::AliOSThings Install AliOS Things software pack
ARM::AMP Install Software components for inter
ARM::CMSIS Up to date CMSIS (Cortex Microcontroller
ARM::CMSIS-Driver Up to date CMSIS Drivers for external devic
ARM::CMSIS-Driver_Va... Install CMSIS-Driver Validation
ARM::CMSIS-FreeRTOS Install Bundle of FreeRTOS for Cortex-
ARM::CMSIS-RTOS_Va... Install CMSIS-RTOS Validation
ARM::mbedClient Install ARM mbed Client for Cortex-M
ARM::mbedTLS Install ARM mbed Cryptographic and
ARM::minar Install mbed OS Scheduler for Cortex-
ARM::TFM Install Trusted Firmware-M (TF-M) is
Huawei::LiteOS Install Huawei LiteOS kernel Software
Keil::ARM_Compiler Up to date Keil ARM Compiler extensions f
Keil::iMXRT105x_MWP Install+ NXP i.MX RT 105x MDK-Middle
Keil::Jansson Install Jansson is a C library for encodi
Keil::MDK-Middleware Update Middleware for Keil MDK-Profe
lwIP::lwIP Install lwIP is a light-weight implemer
Output
Refresh Pack descriptions
Refresh Pack descriptions
Update available for Keil::MDK-Middleware (installed: 7.7.0, available: 7.8.0)
Completed requested actions
ONLINE

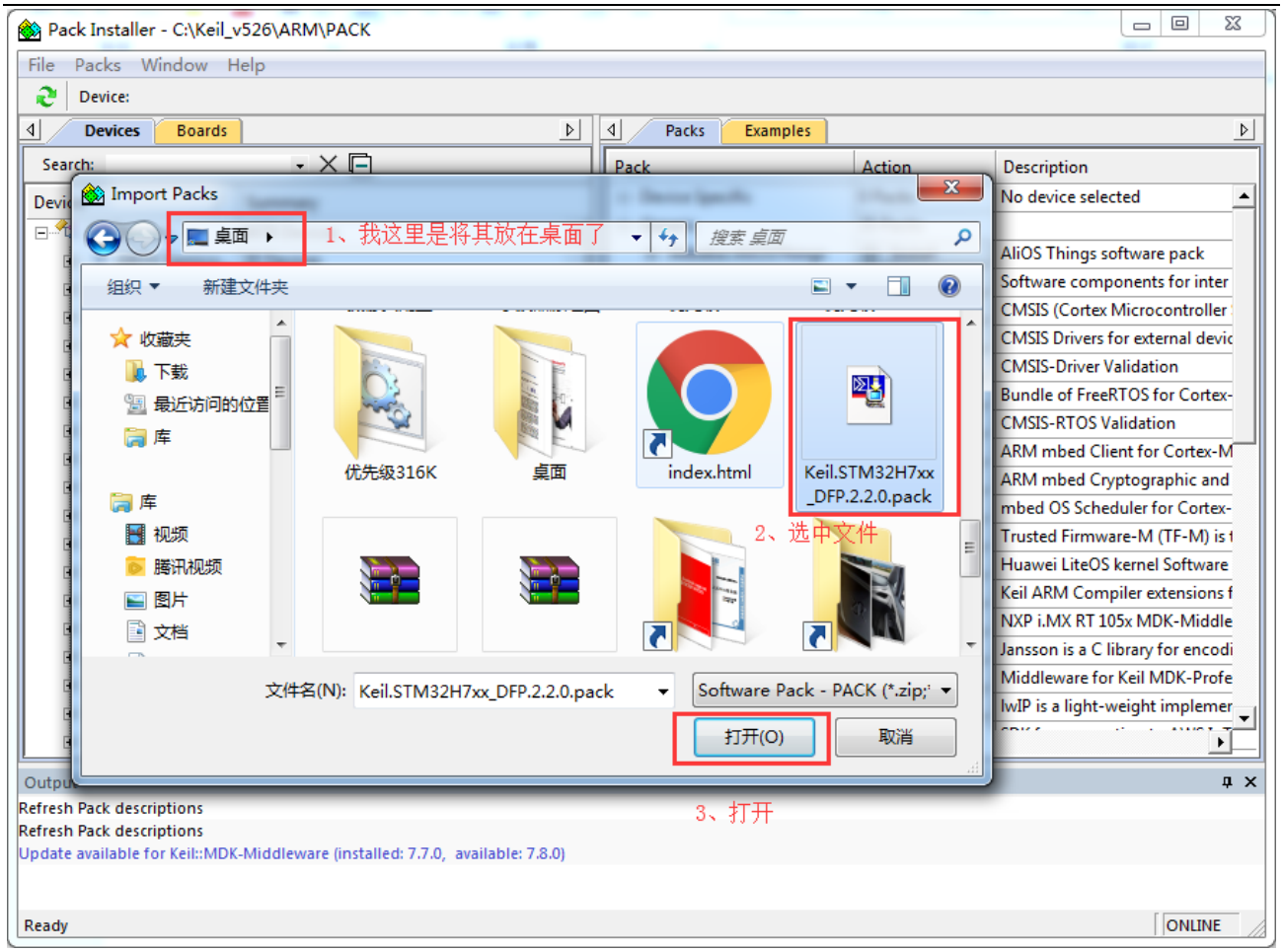
2.3.3 安装 STM32H7 的软件包

这位为大家介绍两种方法，一种是直接去 KEIL 网站下载，另一种是用 MDK 自带的下载功能安装。

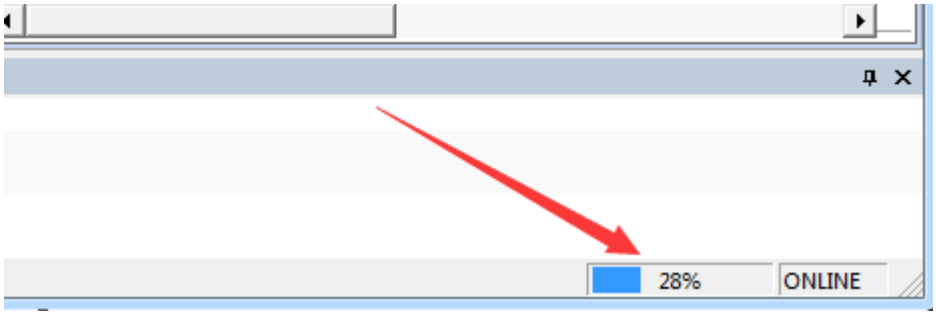
- 方式一：推荐直接在官方地址：http://www.keil.com/dd2/Pack/ 下载。

下载完毕后，导入即可：

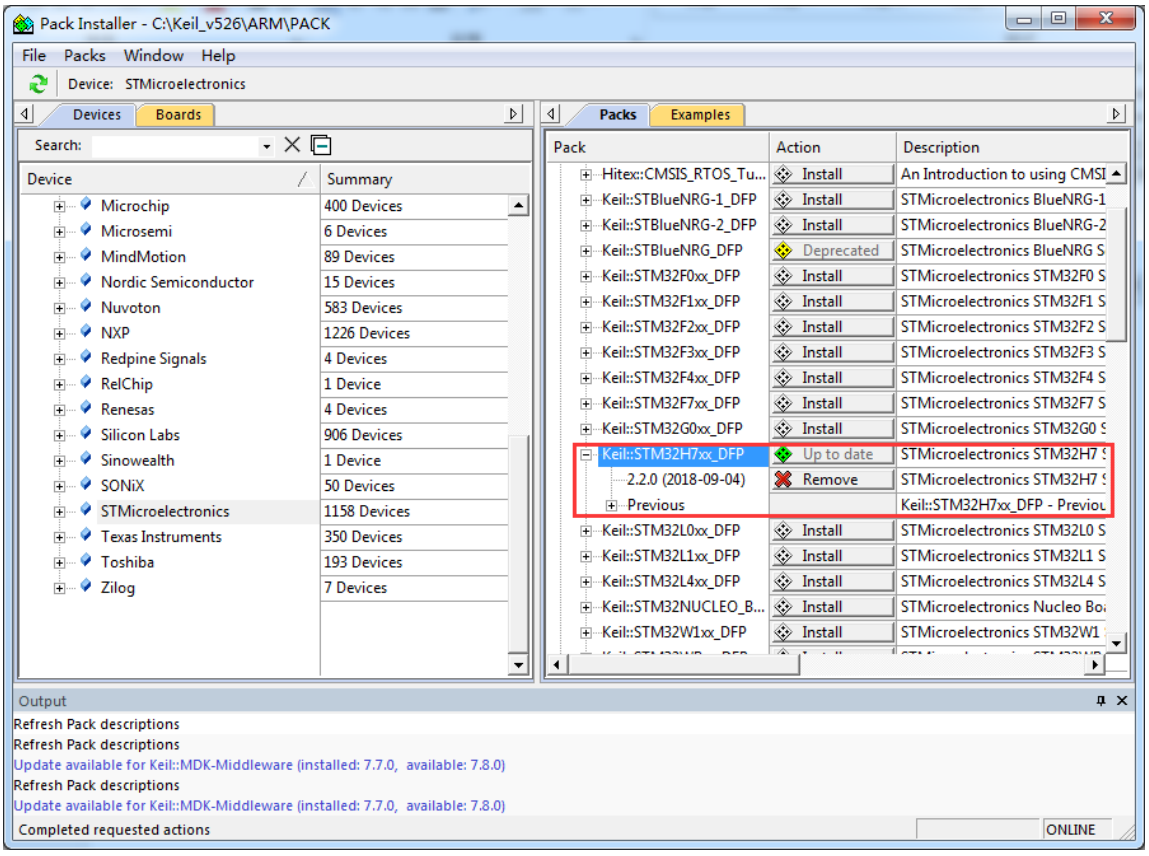




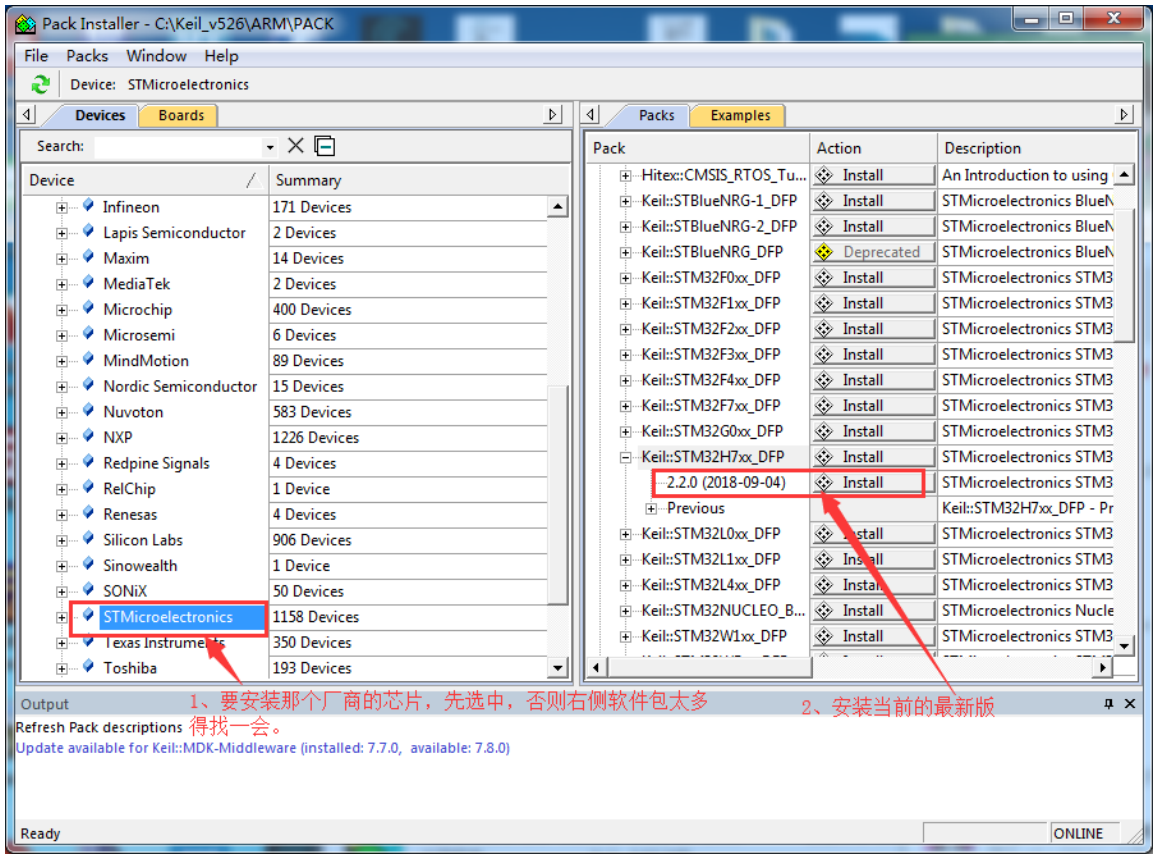
导入的时候，右下角会有一个进度：



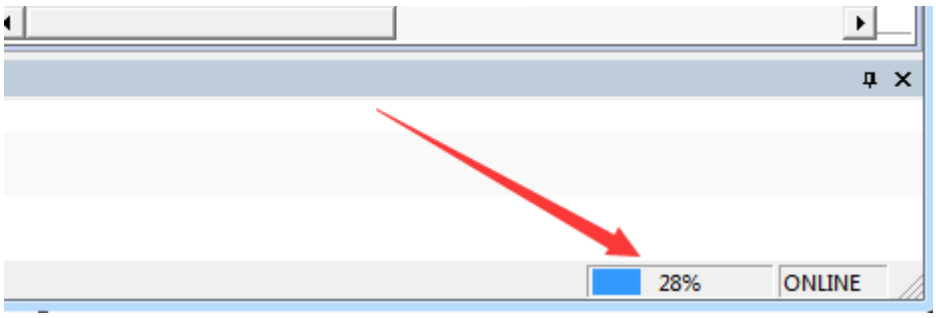
导入成功后，可以看到软件包已经安装上：



◆ 方式二，直接使用 Pack Installer 安装，这个必须要联网才能使用。



下载和安装的时候，右下角也都有进度：

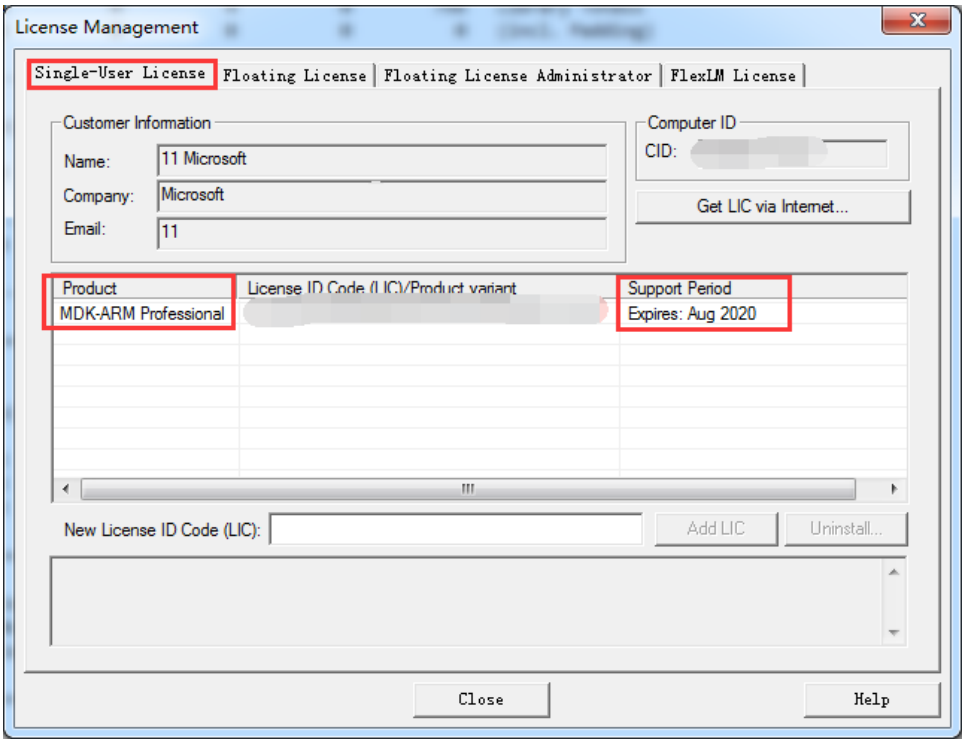


安装完成后，跟方式一的效果一样。

2.3.4 注册 MDK5

1、下载第 2 个注册机：http://www.armbbs.cn/forum.php?mod=viewthread&tid=2346 。
2、运行 MDK5，打开”File“下的”License Management...“，复制 CID 编号（11 字符）。
3、运行破解软件 Keil_ARM_MDK_5.00_Keygen_serial_Crack.exe。有些杀毒软件（如卡巴斯基）会误报为病毒程序，请临时屏蔽掉杀毒软件后再运行。
4、Keil_ARM_MDK_5.00_Keygen_serial_Crack 的“target”下拉列表框选择“ARM”。
5、第 2 行下拉列表框选择“MDK Professional”。
6、将 MDK 的 CID 编号粘贴到 KEIL_lic 中的 CID 编辑框内，然后点击“Generate”按钮。
7、把方框中生成的 30 位注册码，拷贝到 MDK 中的“New License ID Code”内。
8、点击“AddLIC”完成注册。如果截止日期太短的话，就继续用 Keil_ARM_MDK_5.00_Keygen_serial_Crack 生成注册码重新注册。

注册成功后的效果：



2.4 IAR8 的安装和注册

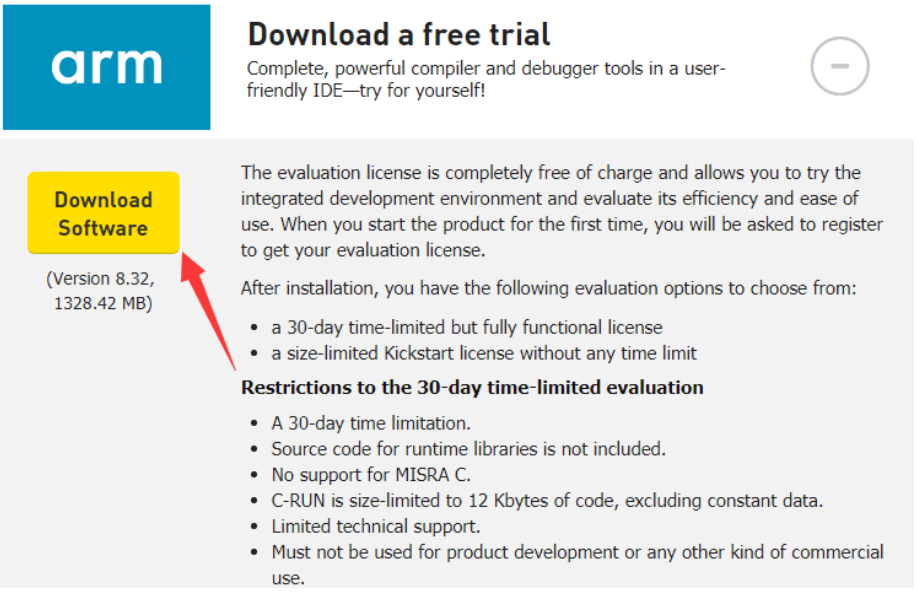
IAR8 的安装就比 MDK5 简单很多了，所有软件都已经集成在 exe 文件里面。

2.4.1 下载 IAR8

（注，当前时间的官方版本还是 IAR8.32）

推荐直接在官方网站下载，地址：

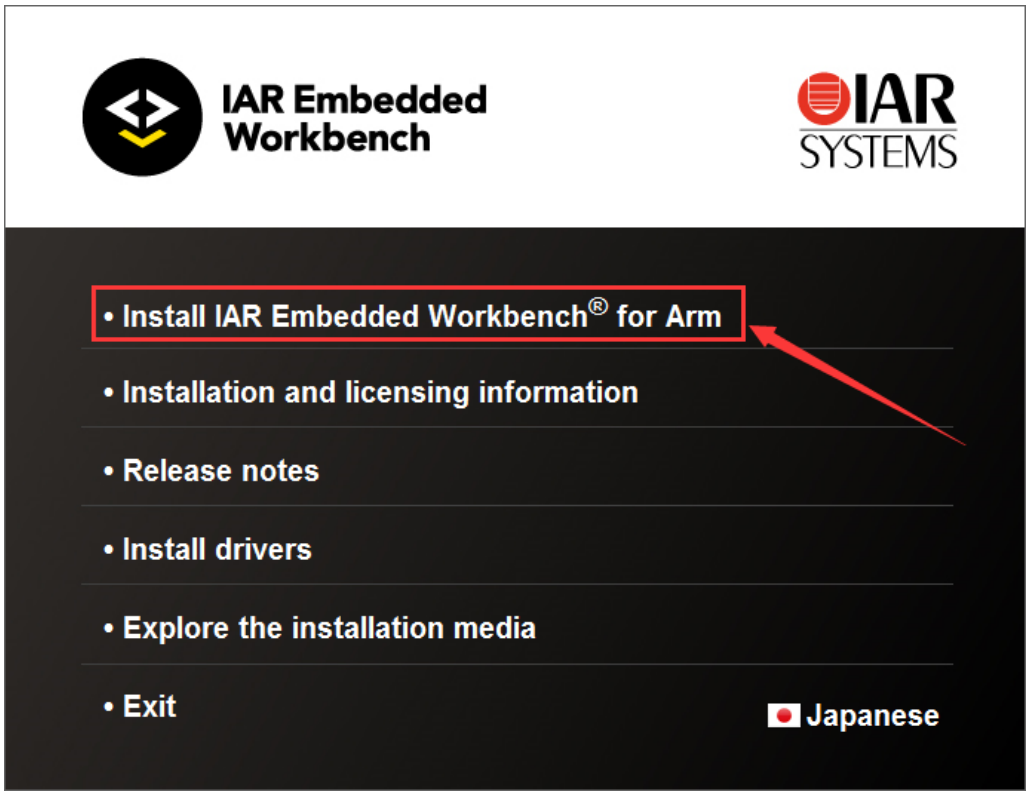
https://www.iar.com/iar-embedded-workbench/#!?architecture=Arm



也可以直接在我们论坛下载：http://www.armbbs.cn/forum.php?mod=viewthread&tid=87220 。帖子里面直接提供的 IAR8.30 的原始下载地址。运行 V7 的例子，使用 IAR8.30 及其以上版本均可。例子是在 IAR8.30 的基础上创建的。

2.4.2 安装 IAR8

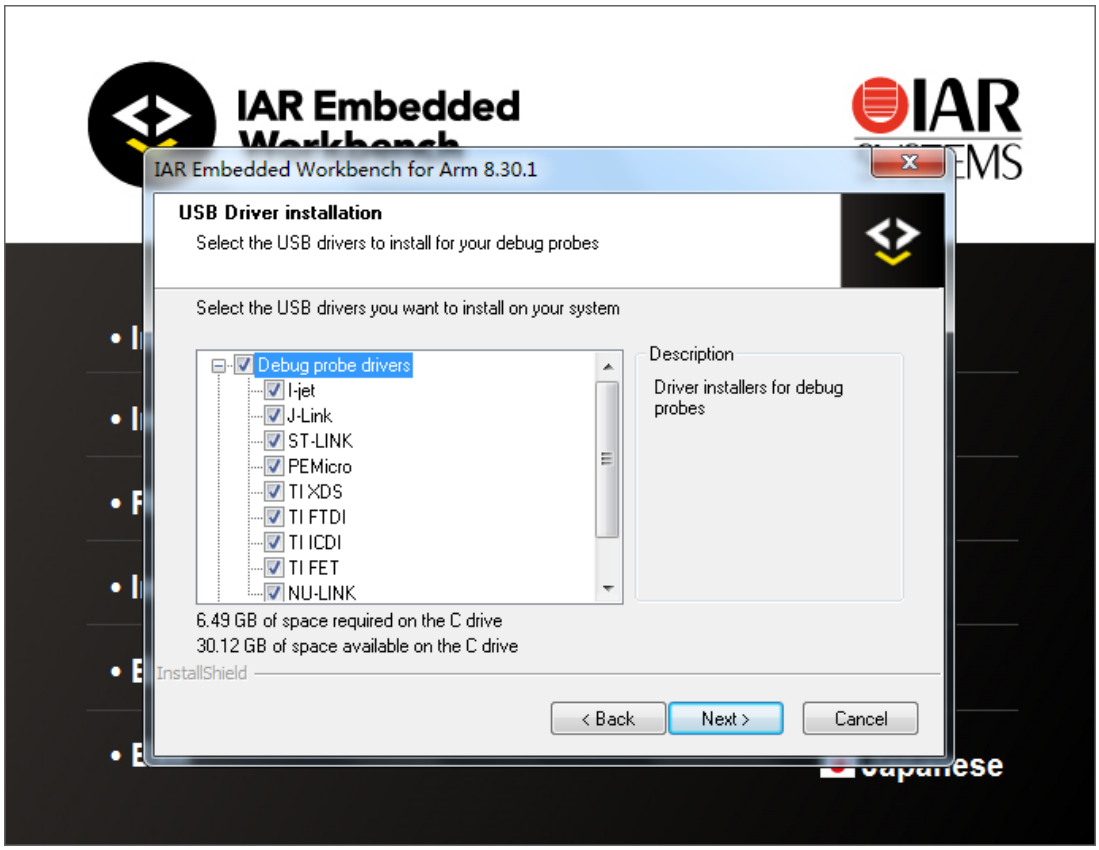
IAR8 的安装比较简单，按照操作提示点击下一步即可。



选择同意：

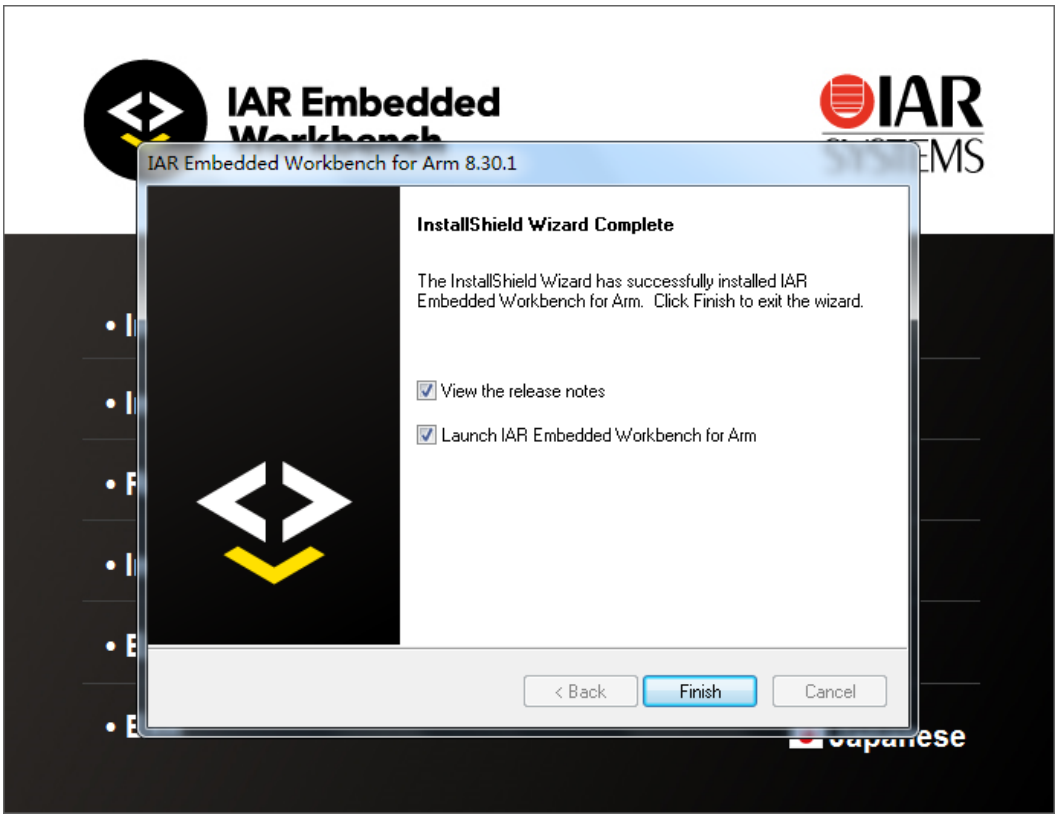
选择安装目录，注意不要有中文：

这里是提示需要安装哪些下载器驱动，如果都安装就全选上。在 IAR 即将安装结束前，会依次提示用户安装这些下载器的驱动。之后就开始软件包的漫长安装过程：



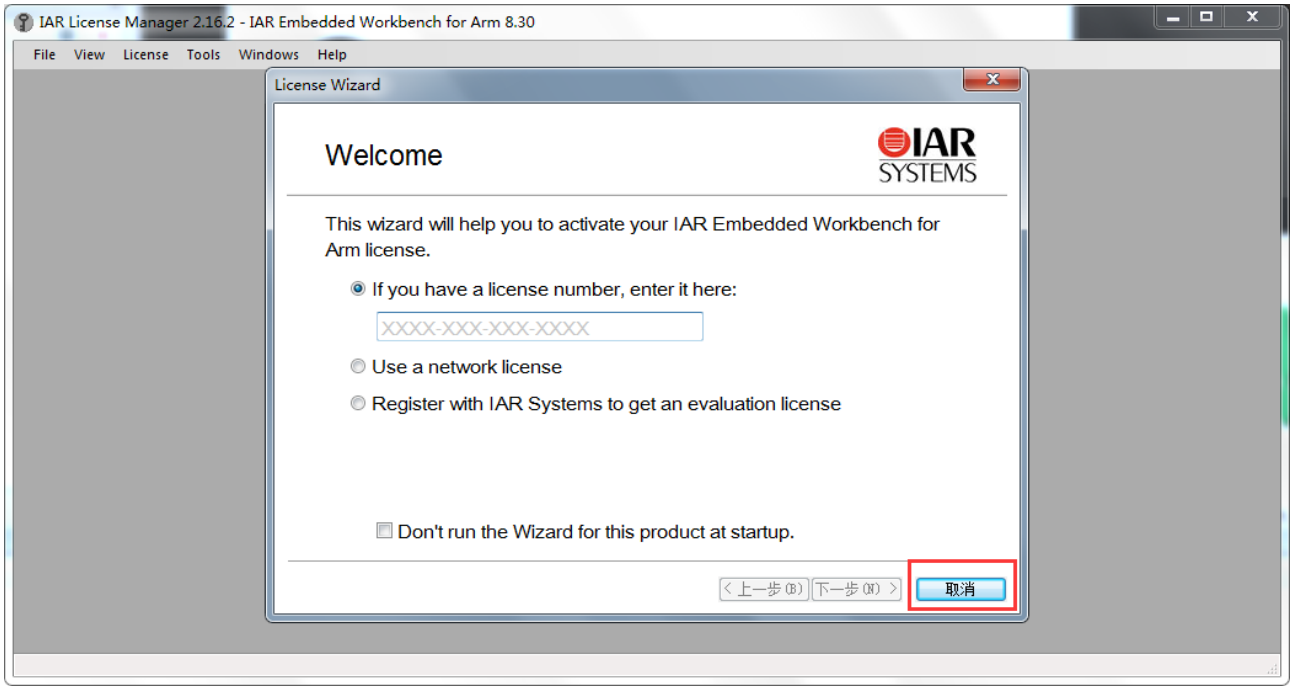
安装各种驱动的时候，不可以将下载器插入电脑：

到这里，IAR 基本就安装完成了：

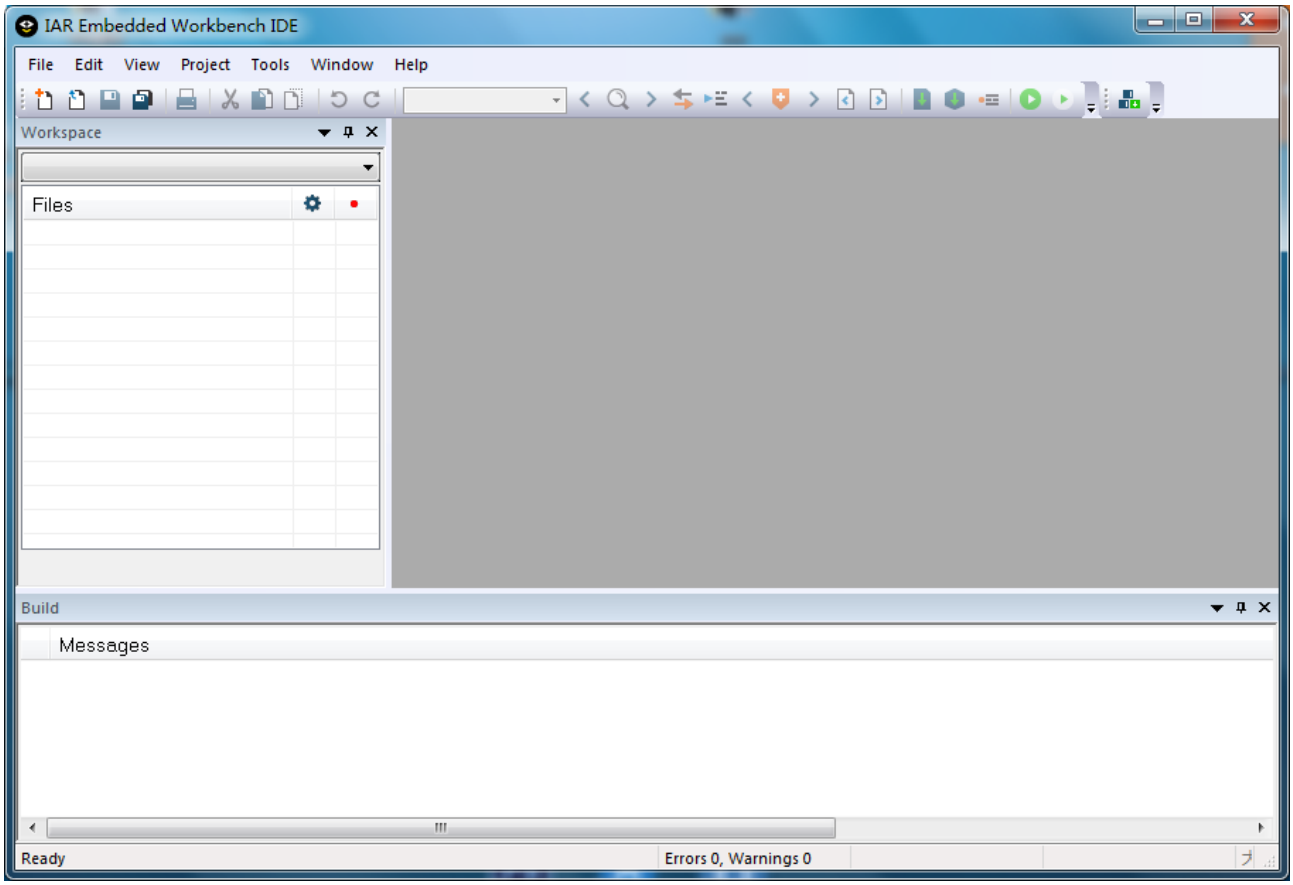


之后就是提示用户是否安装之前选择的各种下载器驱动，一般安装 CMSIS-DAP、STLINK、JLINK 即可。

注册问题先不用管，看本章 2.4.3 小节单独注册就行。



至此，IAR8 就安装完毕了，整体效果如下：



2.4.3 注册 IAR8

IAR8 的注册在此贴的压缩包里面有详细的图示步骤：
http://www.armbbs.cn/forum.php?mod=viewthread&tid=87220 。

2.5 JLINK 驱动安装

调试下载 STM32H7，务必使用 JLINK V9 或 V10，之前的版本已经不支持。

- 下载 JLINK 当前最新版本驱动（V6.44b）：

  可以在 SEGGER 官网下载：
  https://www.segger.com/downloads/jlink/#J-LinkSoftwareAndDocumentationPack 。
  也可以在论坛下载：JLINK 电脑端驱动 V6.44b
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=91727 。

- 安装 JLINK 电脑端驱动：

  下载完毕后，直接安装即可，安装比较简单。

- 安装完毕后，为了验证安装是否成功，看电脑端的设备管理器是否识别出来即可（要将 JLINK 插到电脑端）：

  另外，如果驱动没有安装成功，JLINK 灯会一直闪烁；灯不闪烁了，说明安装成功了。

- 使用 JLINK 前，推荐大家看下此贴：

JLINK 固件，JLINK 驱动和 JLINK 硬件版本之间的关系，以及固件升级方法
http://www.armbbs.cn/forum.php?mod=viewthread&tid=18896 。

- 调试中经常会遇到各种各样的问题，推荐大家看此贴：

JLINK 无法下载解决思路以及常见问题整理
http://www.armbbs.cn/forum.php?mod=viewthread&tid=21708 。

2.6 STLINK 驱动安装

调试下载 STM32H7，推荐使用最新版驱动和对应的固件。

- 为了方便大家下载，已经在论坛将这些软件都整理好，大家可以直接下载：

STLINK 驱动支持 XP，WIN7，WIN8 和 WIN10 的四合一版本已经发布，含 32bit 和 64bit
http://www.armbbs.cn/forum.php?mod=viewthread&tid=26101 。

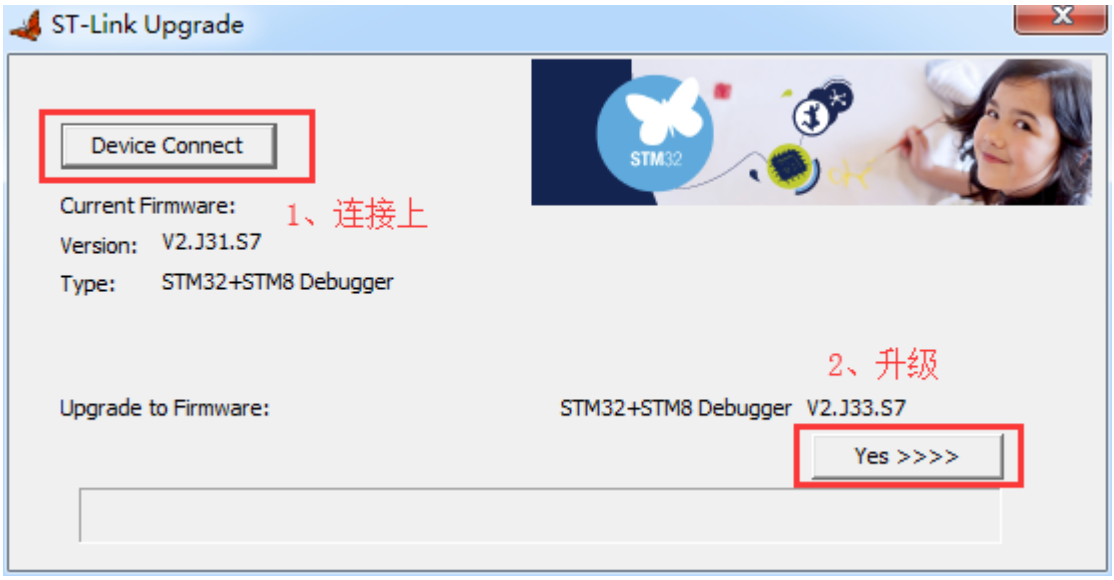
- 安装 STLINK 电脑端驱动：

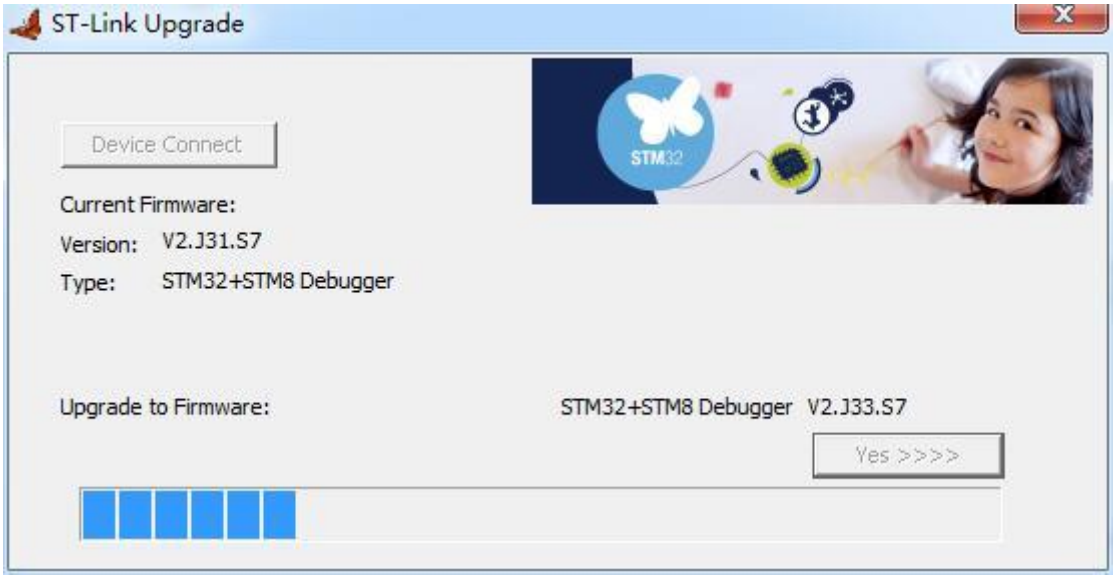
下载完毕后，直接安装即可，安装比较简单。

- 安装完毕后，为了验证安装是否成功，看电脑端的设备管理器是否识别出来即可（要将 STLink 插到电脑端）：

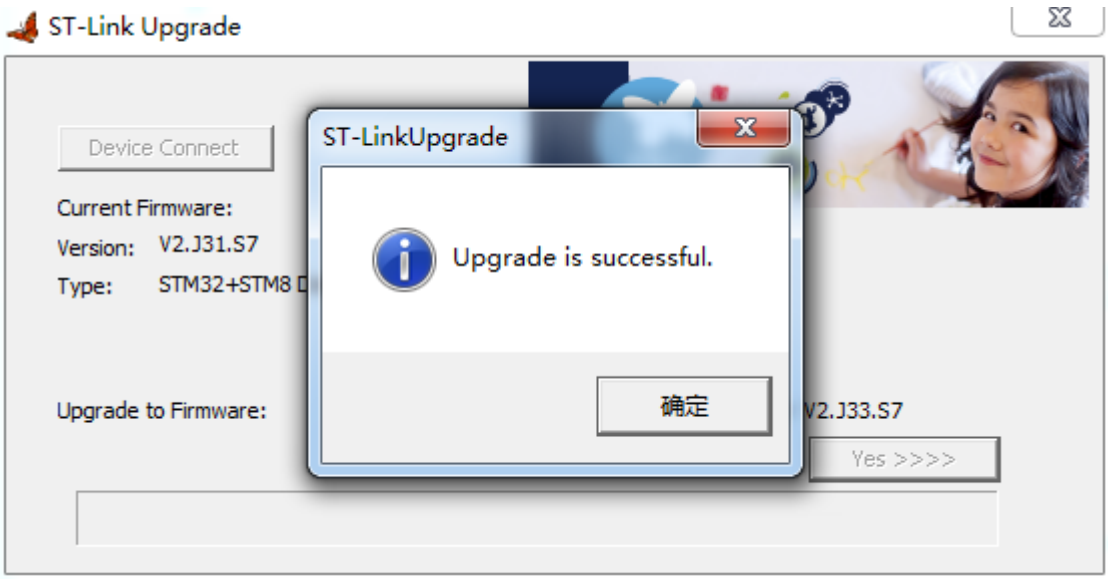
另外，如果驱动没有安装成功，STLINK 灯会一直闪烁；灯不闪烁了，说明安装成功了。

- 推荐用下载的最新版本固件升级软件，将 STLINK 的固件升级到最新版本：





下面就表示升级成功。如果升级失败，重新插拔，再重新升级即可。



2.7 CH340 驱动安装（RS232 转 USB）

开发板发货时，基本都有送一根 R232 转 USB 线，控制芯片是用的 CH340。

- 下载当前最新版本驱动：

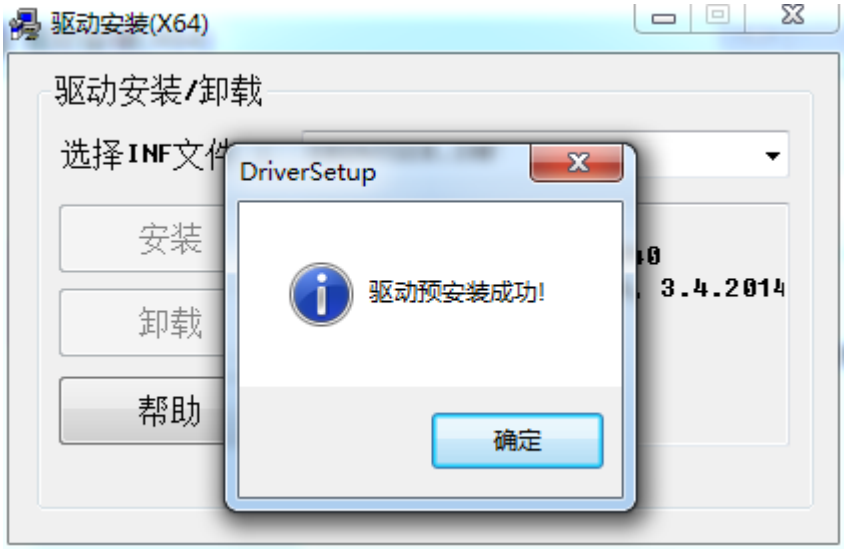
  CH340/CH341 的 USB 转串口 WINDOWS 驱动程序的安装包，支持 32/64 位 Windows 10/8.1/8/7

  http://www.armbbs.cn/forum.php?mod=viewthread&tid=32826 。

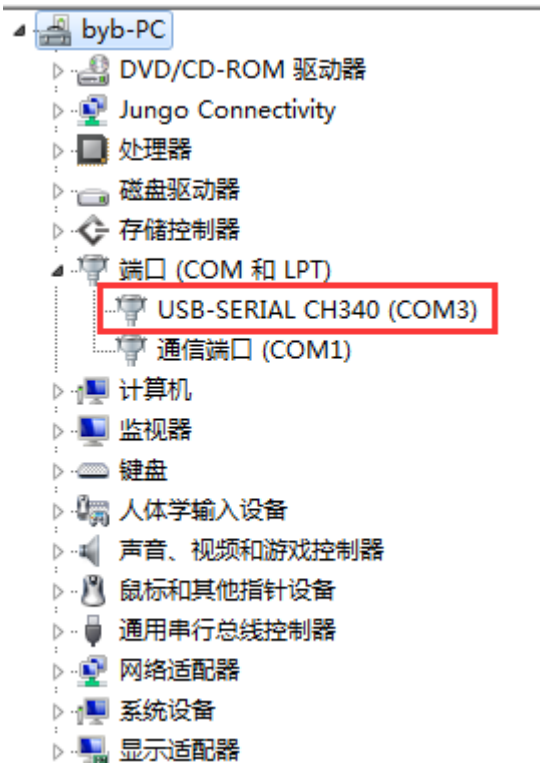
- 安装驱动：

  下载后，打开 exe 小软件，点击安装即可：

安装成功后效果：



- 安装完毕后，为了验证安装是否成功，看电脑端的设备管理器是否识别出来即可（要将 RS232 转 USB 线插到电脑端）：



◆ 进一步验证串口线是否可以用的简单办法

第 30 章的 3.2 小节有详细说明。

2.8 串口软件（终端仿真软件 SecureCRT）

串口软件有很多，大家选择自己顺手的即可，本教程配套的例子是一直用的 SecureCRT 测试的。这个软件最大的好处就是稳定不死机，即使大家的串口死机了，它都没问题。

◆ 汉化绿色版下载，无需安装，解压出来即可使用：

论坛下载地址：

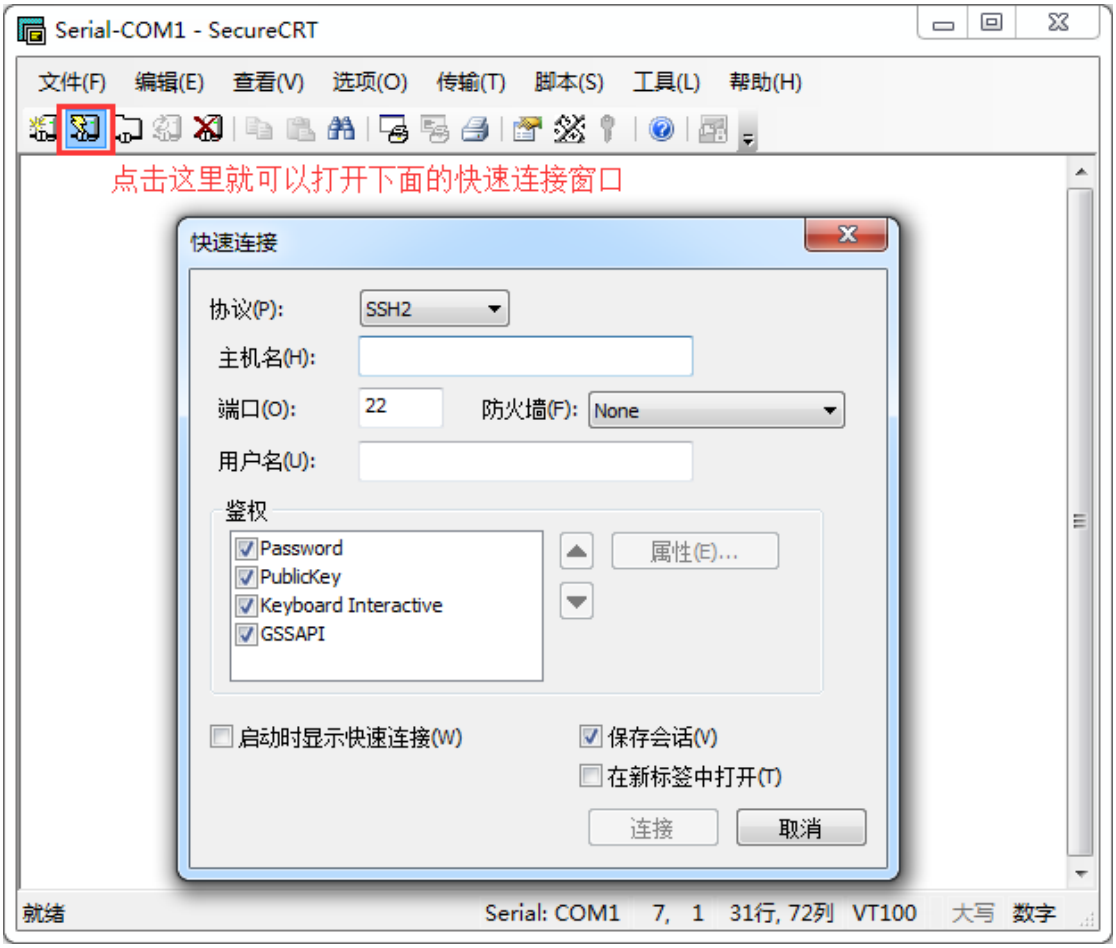
http://www.armbbs.cn/forum.php?mod=viewthread&tid=91718 。

◆ SecureCRT 使用方法：

SecureCRT是一款支持SSH和串口的终端仿真程序，功能非常强大，我们只使用其中的串口部分即可。

下面我们只介绍如何设置串口。

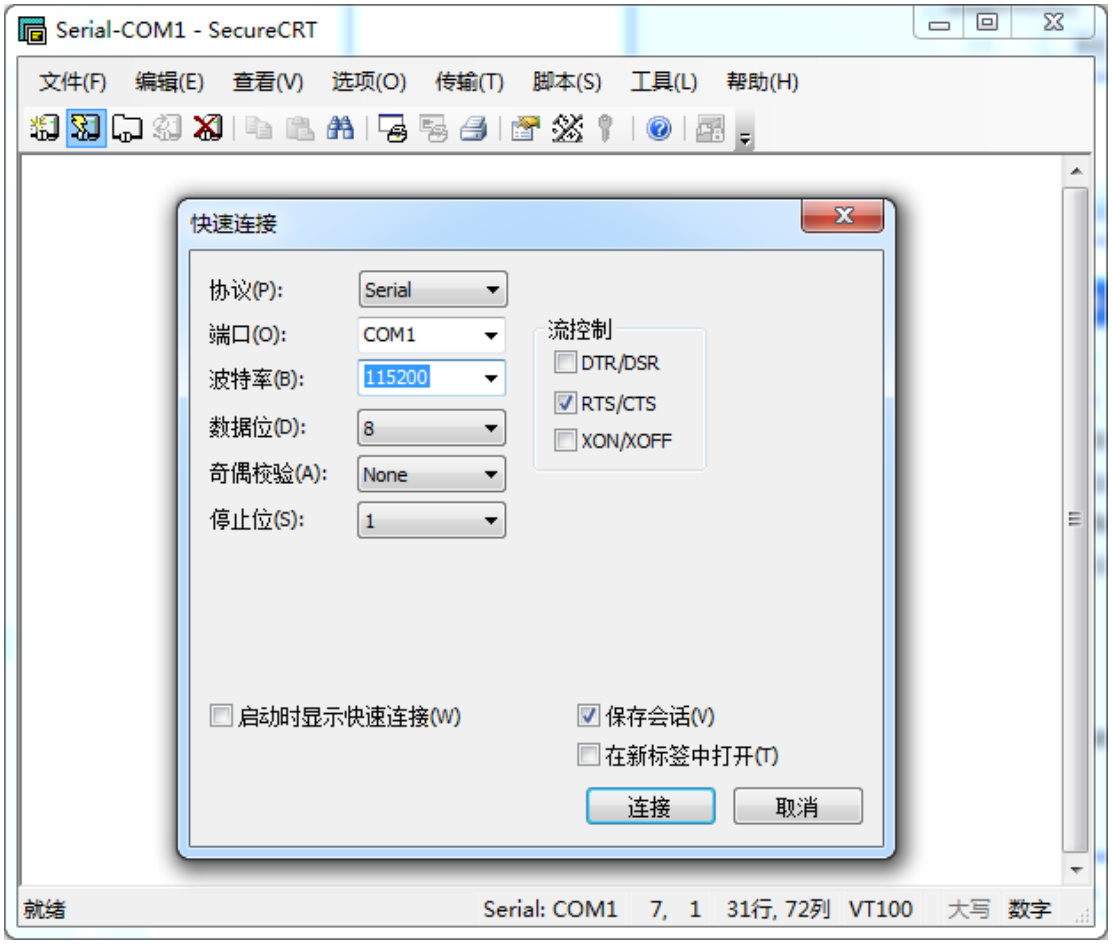
启动SecureCRT软件，第一次使用时，会弹出“快速连接”窗口：



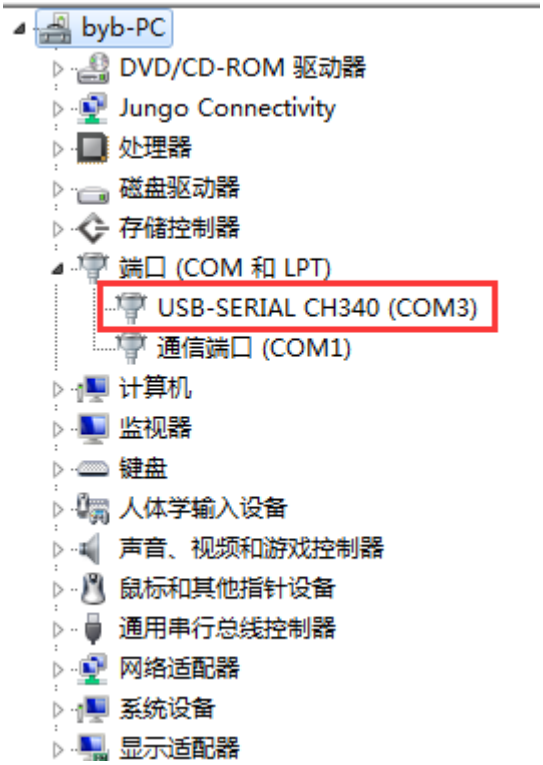
在快速连接窗口中设置如下：

- 协议栏选择：Serial
- 端口选择USB虚拟串口对应的COM号 (如果不知道COM号，请打开设备管理器->端口进行查阅)
- 波特率选择115200
- 数据位按缺省值：8位
- 奇偶校验选择无（None）
- 停止位选择缺省值：1
- 流控都不勾选

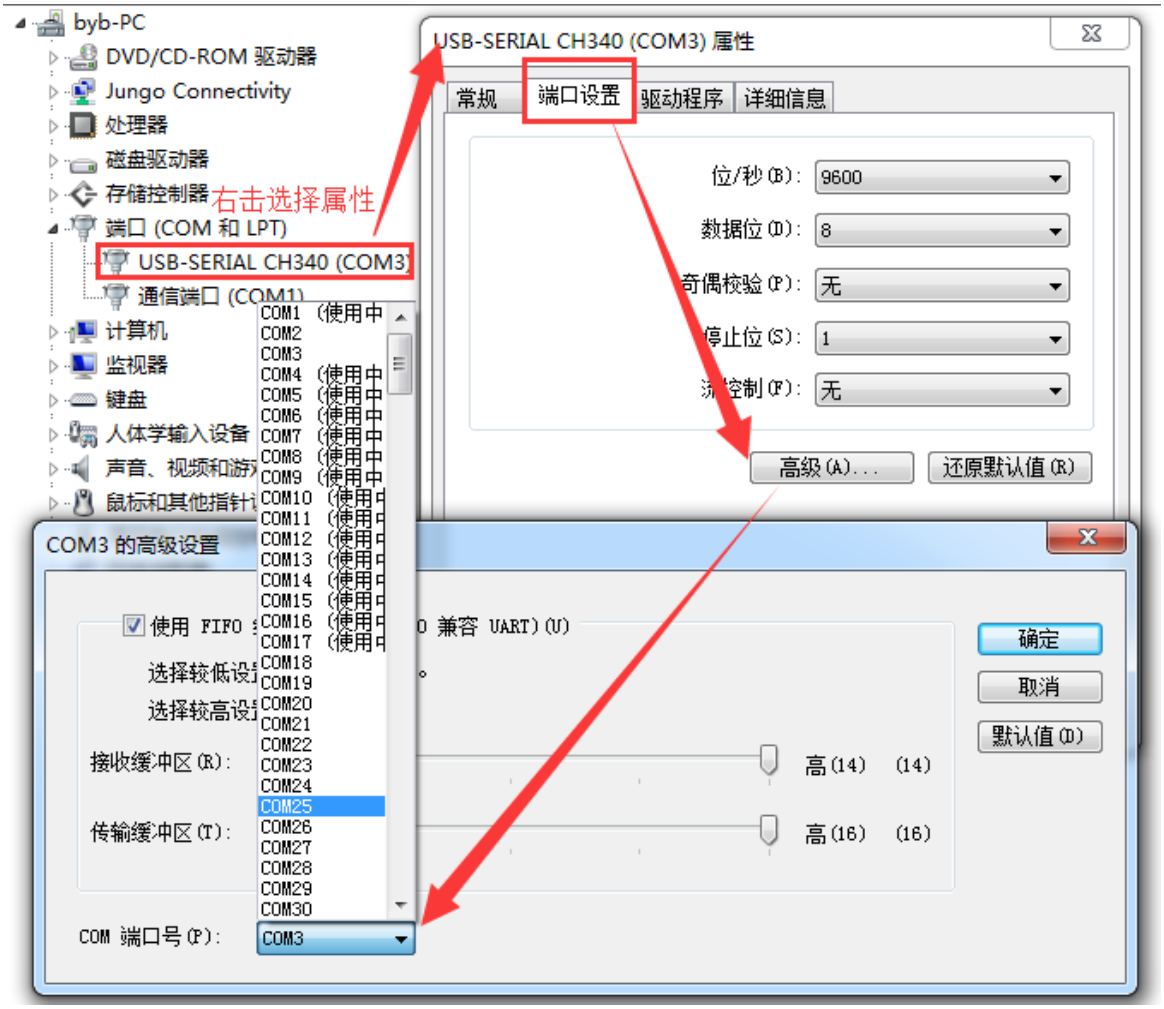
保存会话前面打勾，这样下次启动就不需要再设置了。



如果要查看串口号，需要大家查看设备管理器，特别是用RS232转串口线时，有时候换个电脑端的USB接口插入，都会导致串口号改变了。比如下面使用的COM3：



如果大家要自己更改串口号也是可以的：



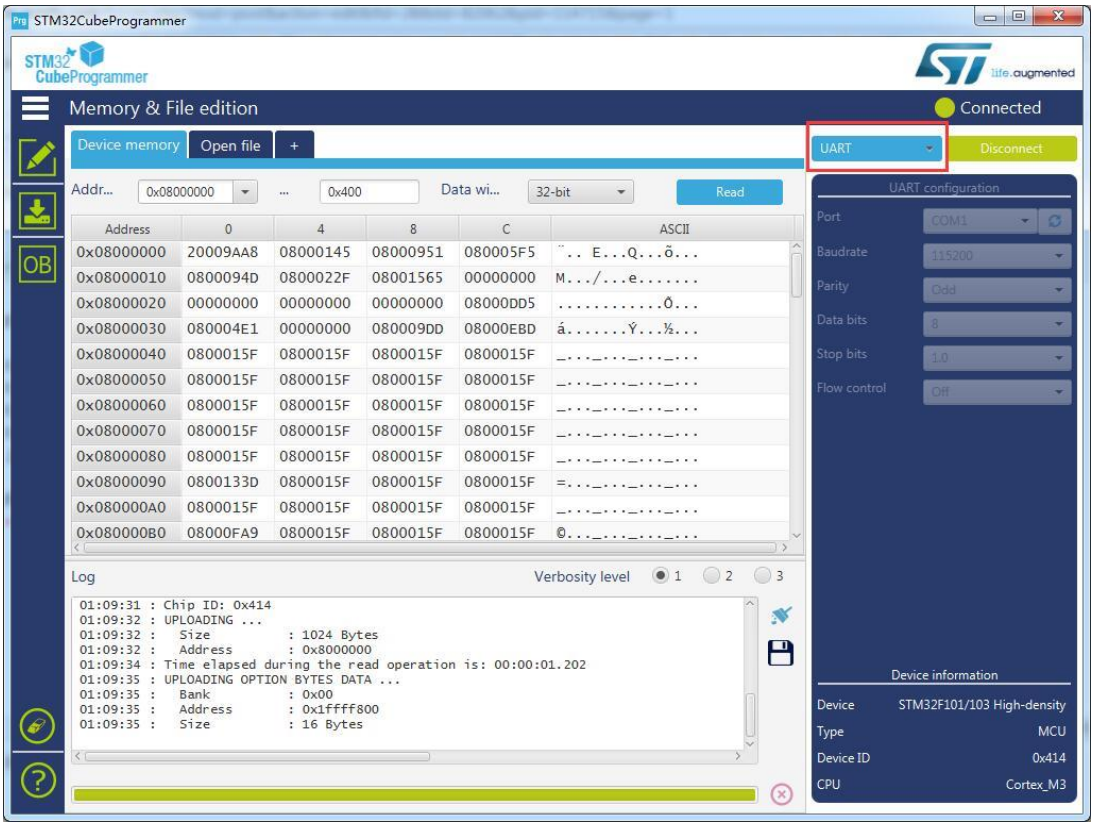
2.9 三合一下载软件 STM32CubeProgrammer

此软件实现了之前的 USB DFU，Flashloader（串口 ISP）和 STLINK 小软件的三合一，测试了下 STLINK 的，效果不错，并且支持外部 EEPROM，NOR Flash，SPI Flash，NAND Flash 等的烧写。也支持 OTP 编程；支持 STLINK 升级；支持 Window，Linux 和 macOS 上使用。

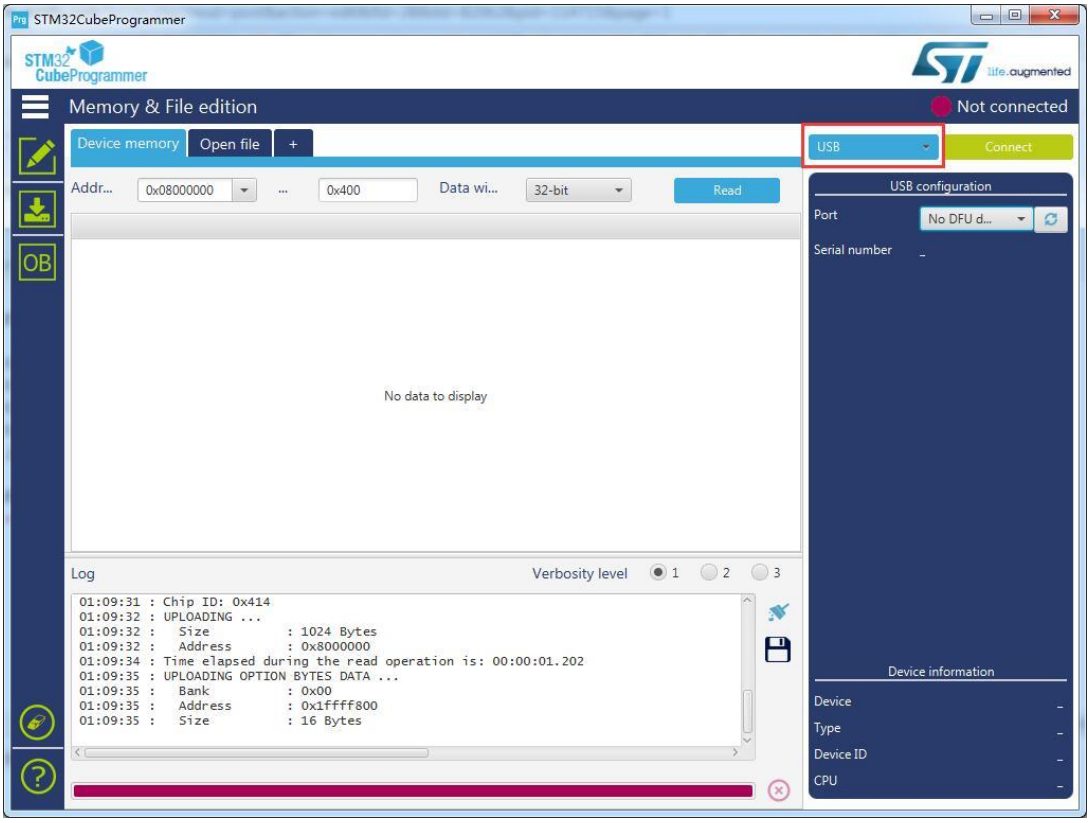
- 论坛下载地址（帖子里面有官方地址）：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=82062 。

- 软件效果：
  这个软件集成的功能有点多，后面会专门做一个专题教程。

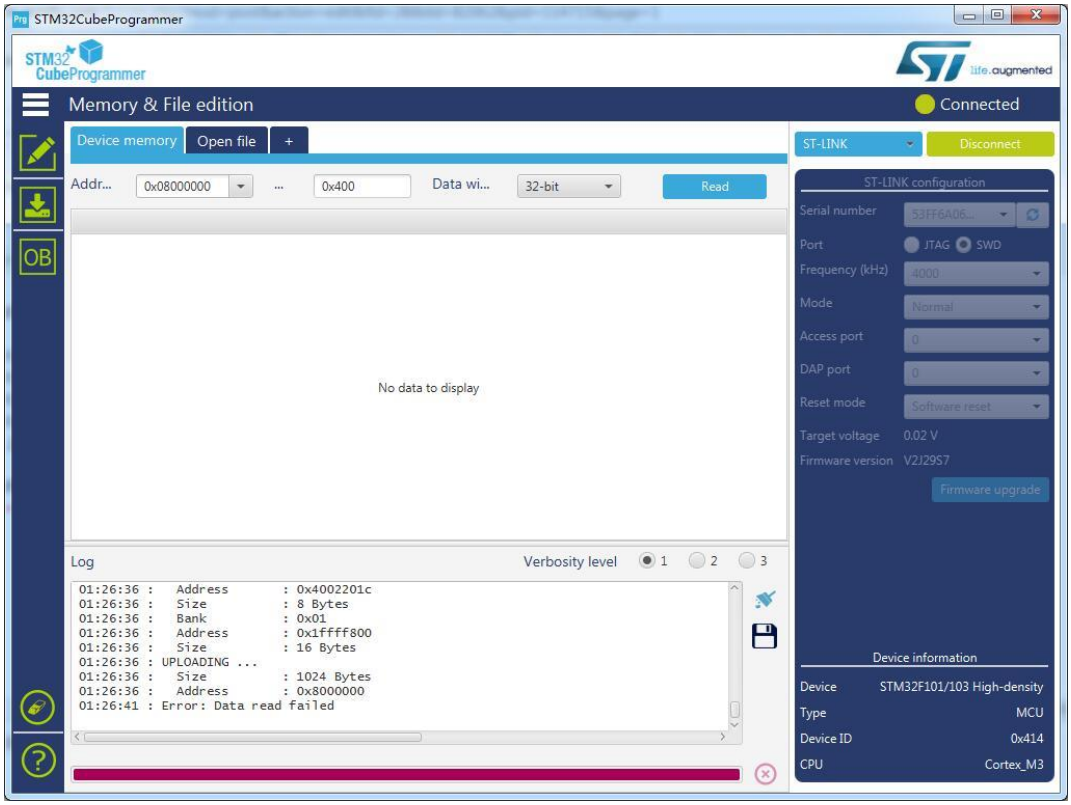
串口下载效果：



USB DFU 下载效果：

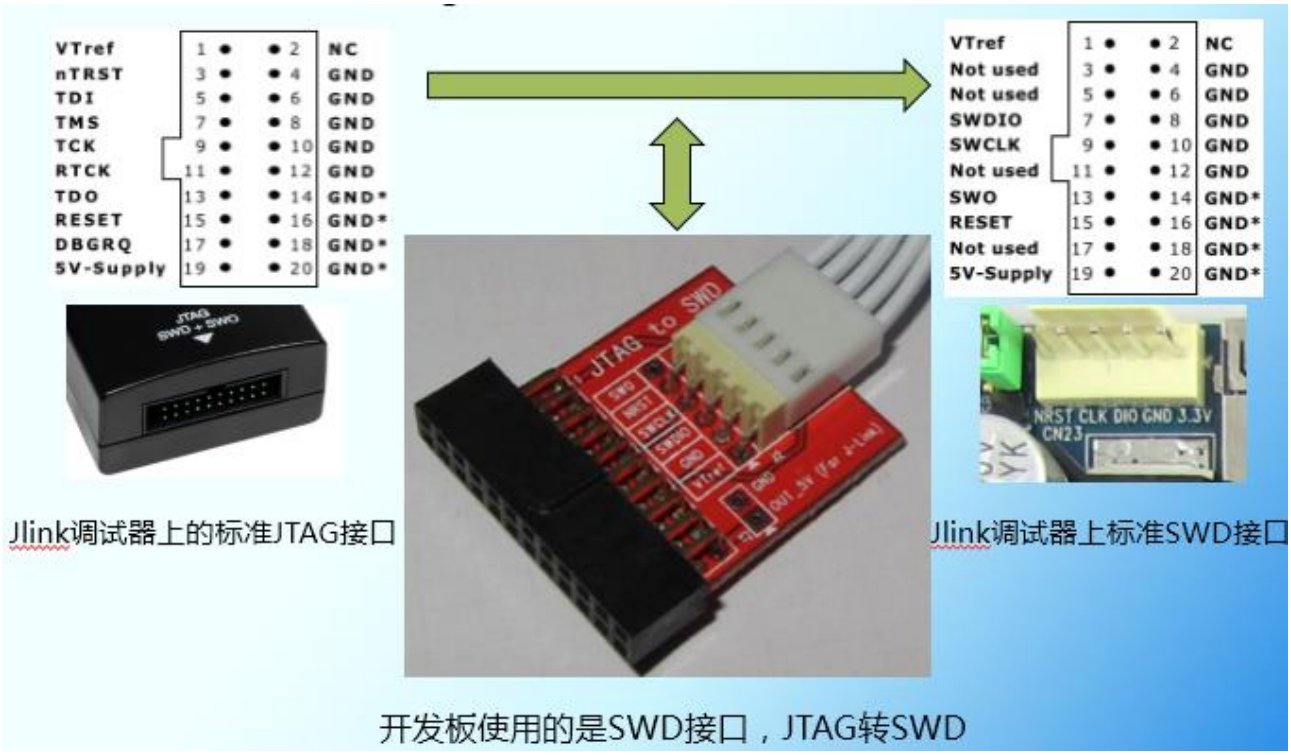


STLINK 下载效果：

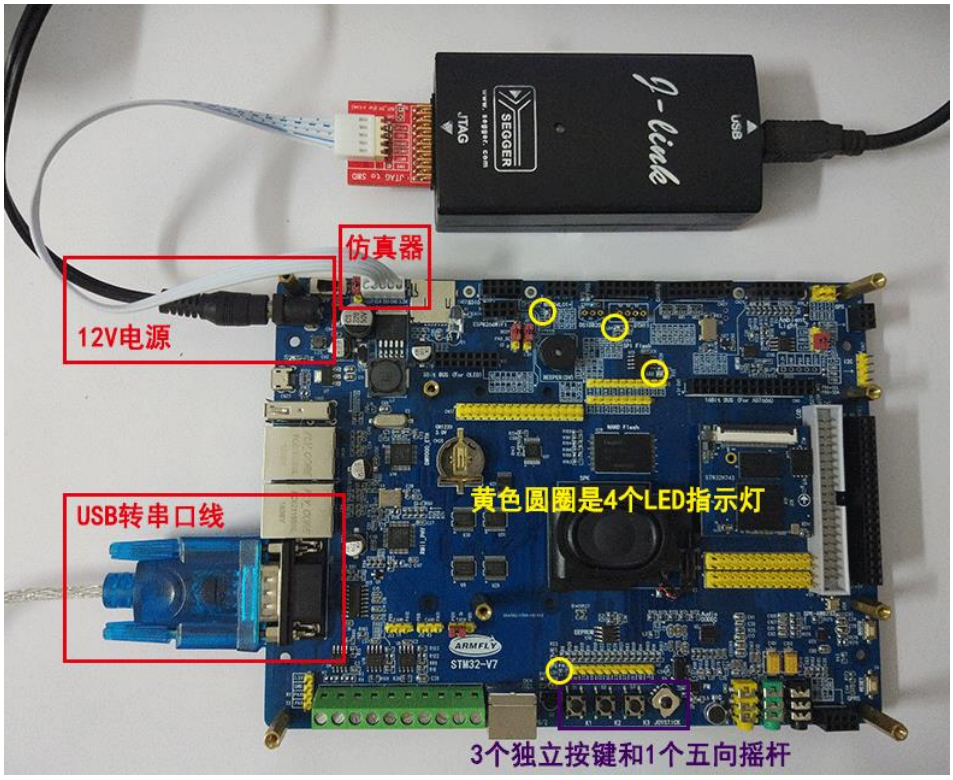


2.10 硬件开发环境搭建好的效果

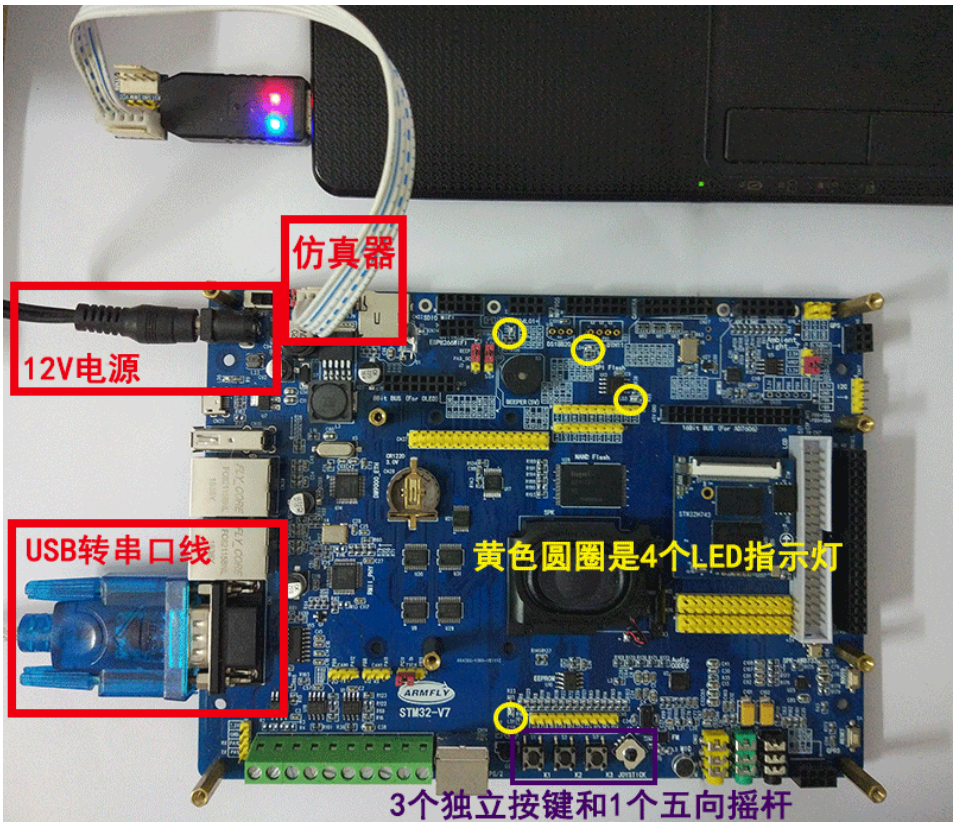
使用 JLINK 的话，我们有一个专门的转接板，可以插到 JLINK 上：



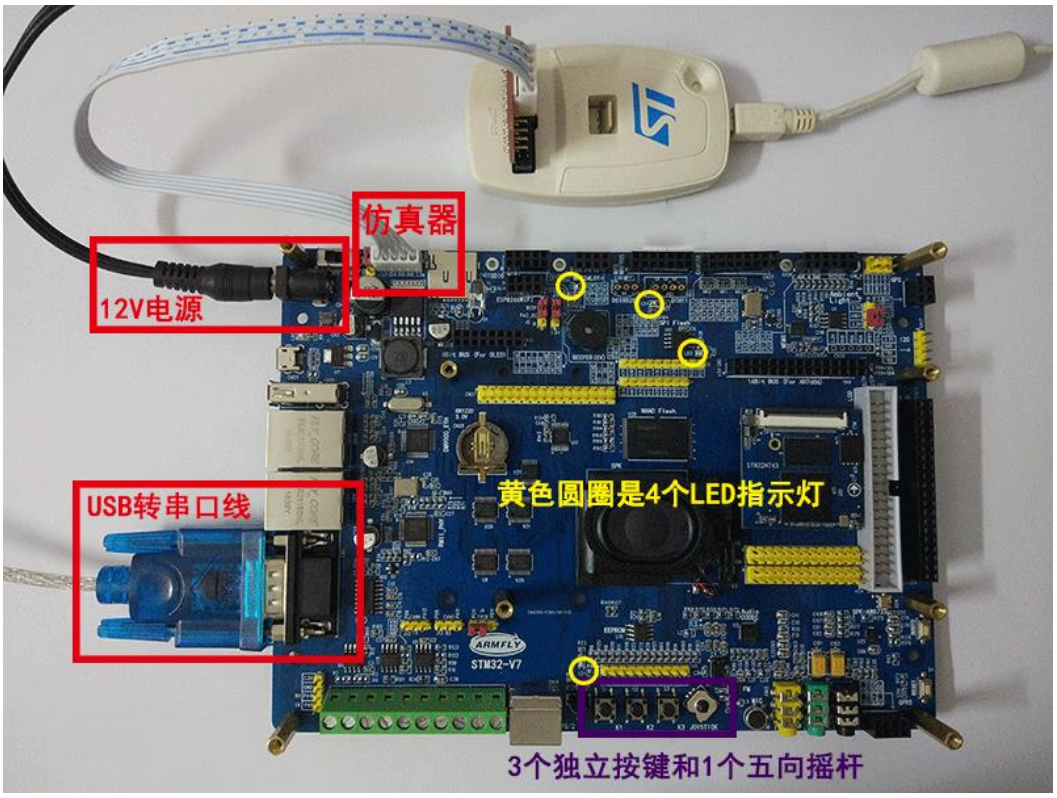
下面是整体的搭建效果：



使用下面这种 STLINK 的话，下面是整体的搭建效果：



使用下面这种 STLINK 的话，也可以使用转接板，下面是整体的搭建效果：



2.11 总结

通过本章节将 STM32 开发环境搭建好之后就可以开启 STM32H7 的学习之旅了。

第3章 STM32H7 整体把控

初学 STM32H7 一定要优先整体把控芯片的框架，不要急于了解单个外设的功能。

3.1 初学者重要提示
3.1 STM32H7 框图
3.2 STM32H7 各个型号的区别
3.2 STM32H7 总线框图和时钟
3.3 STM32H7 的 AXI 总线
3.4 STM32H7 的 FLASH
3.5 STM32H7 的 RAM
3.6 总结

3.1 初学者重要提示

- 学习一款新的芯片，优先掌握系统框架是比较重要的，建议逐渐养成这种学习习惯，然后各个击破即可。
- 本章节提供了多张 STM32H7 的框图，这些框图都非常具有代表性。很多时候记忆知识点比较费脑子，记录这些框图是一种非常好的方式。
- 对于本章节提供的部分知识点，无法理解透彻，暂时没有关系。随着后面的深入学习，基本都可以掌握。
- 重要的 MPU 和 Cache 知识分别放在了第 23 章和第 24 章。

3.2 STM32H7 硬件框图

学习一款新的芯片，需要优先了解一下它的整体功能设计。需要的资料主要是来自官网和数据手册，比如我们 V7 开发板使用的 STM32H743XIH6，直接在官方地址：链接（这是超链接）就可以看到对此芯片所做的介绍，页面中有一个如下的框图，对于了解 STM32H7 整体设计非常方便。

再稍微详细点，就需要大家读页面上的”Key Features”，就是下图所示的内容：

Key Features

- Core
  - 32-bit Arm® Cortex®-M7 core with double-precision FPU and L1 cache: 16 Kbytes of data and 16 Kbytes of instruction cache; frequency up to 400 MHz, MPU, 856 DMIPS/ 2.14 DMIPS/MHz (Dhrystone 2.1), and DSP instructions
- Memories
  - Up to 2 Mbytes of Flash memory with read-while-write support
  - 1 Mbyte of RAM: 192 Kbytes of TCM RAM (inc. 64 Kbytes of ITCM RAM + 128 Kbytes of DTCM RAM for time critical routines), 864 Kbytes of user SRAM, and 4 Kbytes of SRAM in Backup domain
  - Dual mode Quad-SPI memory interface running up to 133 MHz
  - Flexible external memory controller with up to 32-bit data bus: SRAM, PSRAM, SDRAM/LPSDR SDRAM, NOR/NAND Flash memory clocked up to 133 MHz in Synchronous mode
  - CRC calculation unit
- Security
  - ROP, PC-ROP, active tamper

或者直接看数据手册开头的章节即可，也进行了介绍，内容基本都是差不多的，如下图所示（部分截图）：

life.augmented

STM32H743xI

32-bit Arm® Cortex®-M7 400MHz MCUs, up to 2MB Flash, 1MB RAM, 46 com. and analog interfaces

Datasheet - production data

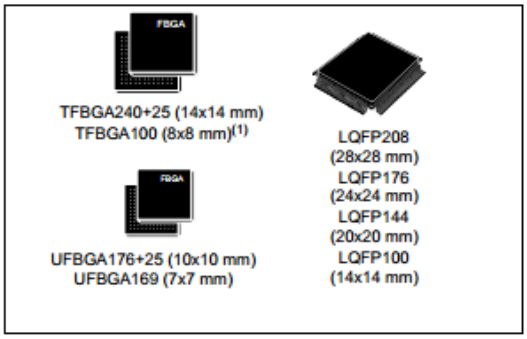
Features

Core

- 32-bit Arm® Cortex®-M7 core with double-precision FPU and L1 cache: 16 Kbytes of data and 16 Kbytes of instruction cache; frequency up to 400 MHz, MPU, 856 DMIPS/ 2.14 DMIPS/MHz (Dhrystone 2.1), and DSP instructions

Memories

- Up to 2 Mbytes of Flash memory with read-while-write support
- 1 Mbyte of RAM: 192 Kbytes of TCM RAM (inc. 64 Kbytes of ITCM RAM + 128 Kbytes of DTCM RAM for time critical routines), 864 Kbytes of user SRAM, and 4 Kbytes of SRAM in Backup domain
- Dual mode Quad-SPI memory interface running up to 133 MHz
- Flexible external memory controller with up to 32-bit data bus: SRAM, PSRAM, SDRAM/LPSDR SDRAM, NOR/NAND Flash memory clocked up to 133 MHz in Synchronous mode
- CRC calculation unit



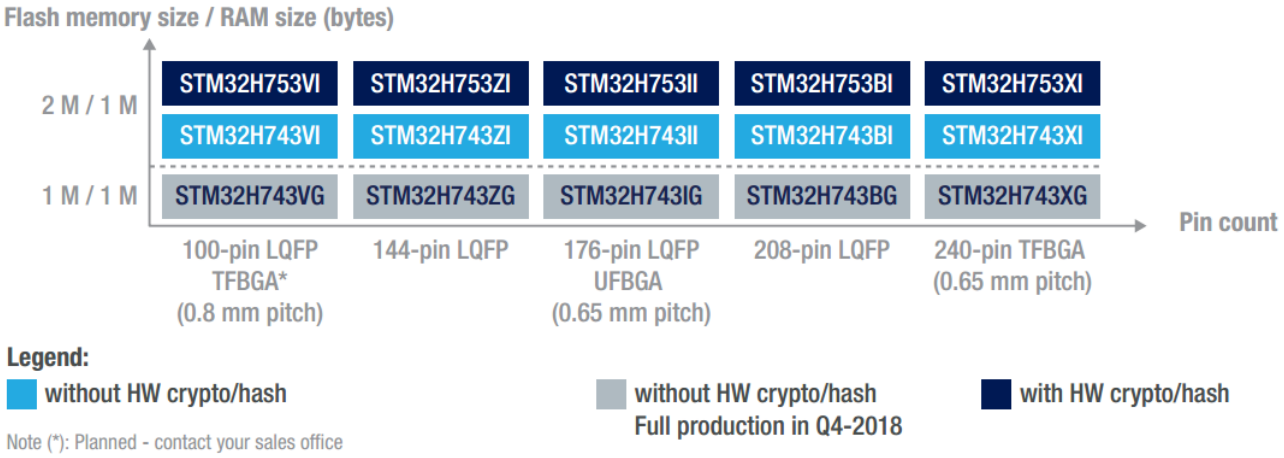
- 1.62 to 3.6 V application supply and I/Os
- POR, PDR, PVD and BOR
- Dedicated USB power embedding a 3.3 V internal regulator to supply the internal PHYs
- Embedded regulator (LDO) with configurable scalable output to supply the digital circuitry
- Voltage scaling in Run and Stop mode (5 configurable ranges)
- Backup regulator (~0.9 V)
- Voltage reference for analog peripheral/V_{REF+}
- Low-power modes: Sleep, Stop, Standby and V_{BAT} supporting battery charging

Low-power consumption

通过框图和 Key Features，大家可以方便地了解 STM32H7 的 FLASH、RAM 大小以及各种自带外设的信息。

3.3 STM32H7 各个型号的区别

涉及到芯片选型的时候，需要大家了解各个型号的区别。对此 ST 有一个专门的文件 STM32H7x3 MCUs High-performance line，在链接（这是一个超链接）里面可以找到。此文件里面有简单的对比，只是内容比较简单，仅两页，不过也言简意赅。最主要的是下面的这个截图：



通过这个截图可以方便地了解不同型号的引脚数、封装、FLASH 大小、RAM 大小以及是否带 HW CRYPTO 硬件加密的区别。

需要了解更详细的对比信息，可以看数据手册。任意下载一个型号的数据手册，在数据手册的的 Table 2 里面有详细的对比，如下图所示（部分截图）：

Table 2. STM32H743xI features and peripheral counts

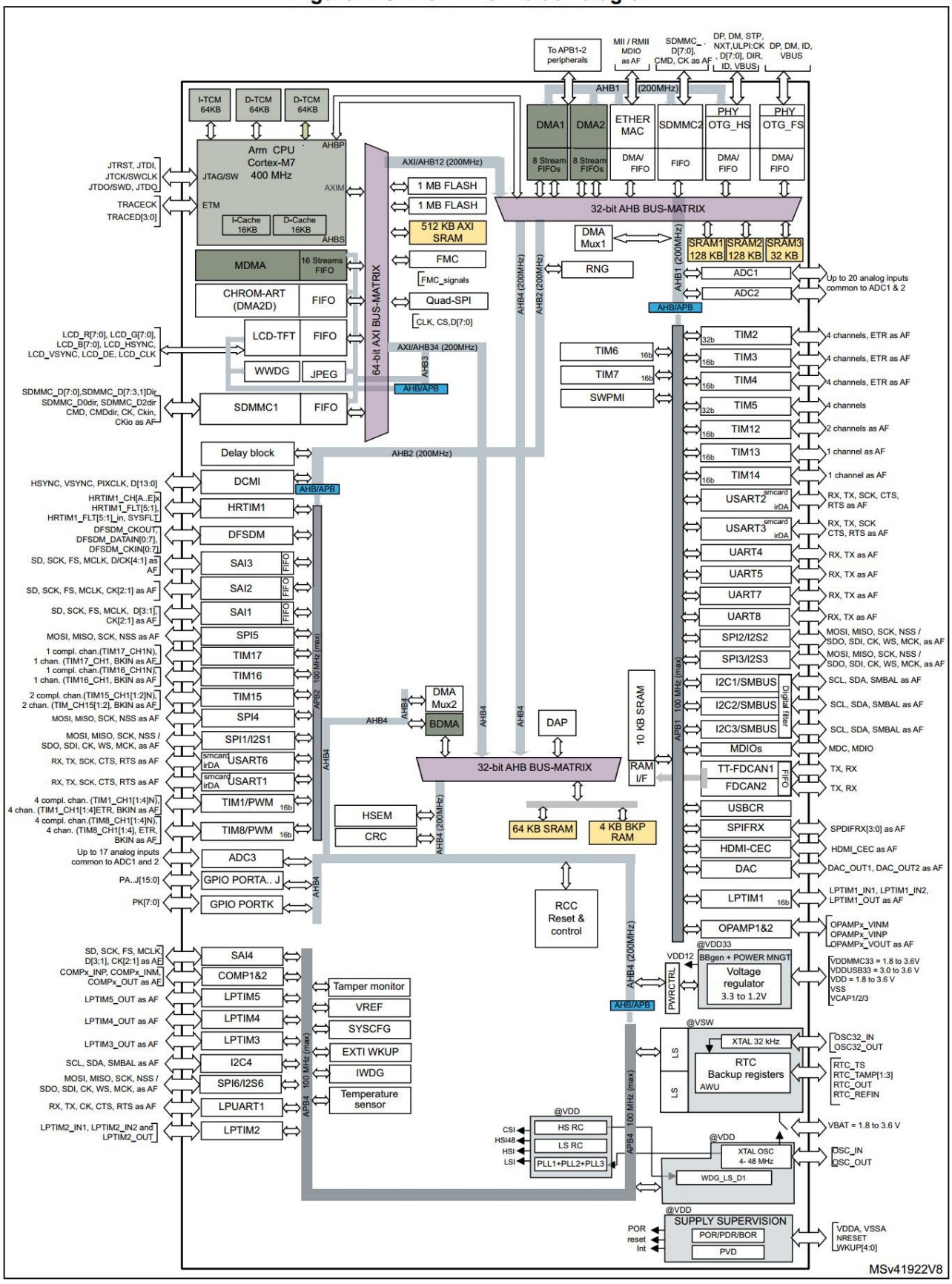
Peripherals		STM32H743VI	STM32H743ZI	STM32H743AI	STM32H743II	STM32H743BI	STM32H743XI
Flash memory in Kbytes		2048					
SRAM in Kbytes	SRAM mapped onto AXI bus	512					
	SRAM1 (D2 domain)	128					
	SRAM2 (D2 domain)	128					
	SRAM3 (D2 domain)	32					
	SRAM4 (D3 domain)	64					
TCM RAM in Kbytes	ITCM RAM (instruction)	64					
	DTCM RAM (data)	128					
Backup SRAM (Kbytes)		4					
FMC		Yes					
Quad-SPI		Yes					
Ethernet		Yes					

使用 ST 提供的软件 STMCUFinder 或者 STM32CubeMX 也可以做对比，只是没有上面的表格这么方便，可以一目了然。

3.4 STM32H7 总线框图和时钟

STM32H7 的数据手册里面提供了一张非常棒的框图，大家可以方便地查看每个总线的时钟速度和这个总线所挂的外设。这个在大家配置外设时钟分频的时候还是非常有用的，因为外设的时钟分频就是建立在所挂的总线速度（大家直接在数据手册里面检索 Figure 1 就可以找到）。

Figure 1. STM32H743xI block diagram



比如我们想得到不同定时器的主频，通过上面的框图，可以方便地获得如下信息：

SYSCLK(Hz) = 400000000 (CPU Clock)
HCLK(Hz) = 200000000 (AXI and AHBs Clock)
AHB Prescaler = 2
D1 APB3 Prescaler = 2 (APB3 Clock 100MHz)
D2 APB1 Prescaler = 2 (APB1 Clock 100MHz)
D2 APB2 Prescaler = 2 (APB2 Clock 100MHz)
D3 APB4 Prescaler = 2 (APB4 Clock 100MHz)

因为 APB1 prescaler != 1，所以 APB1 上的 TIMxCLK = APB1 x 2 = 200MHz;
因为 APB2 prescaler != 1，所以 APB2 上的 TIMxCLK = APB2 x 2 = 200MHz;
APB4 上面的 TIMxCLK 没有分频，所以就是 100MHz;

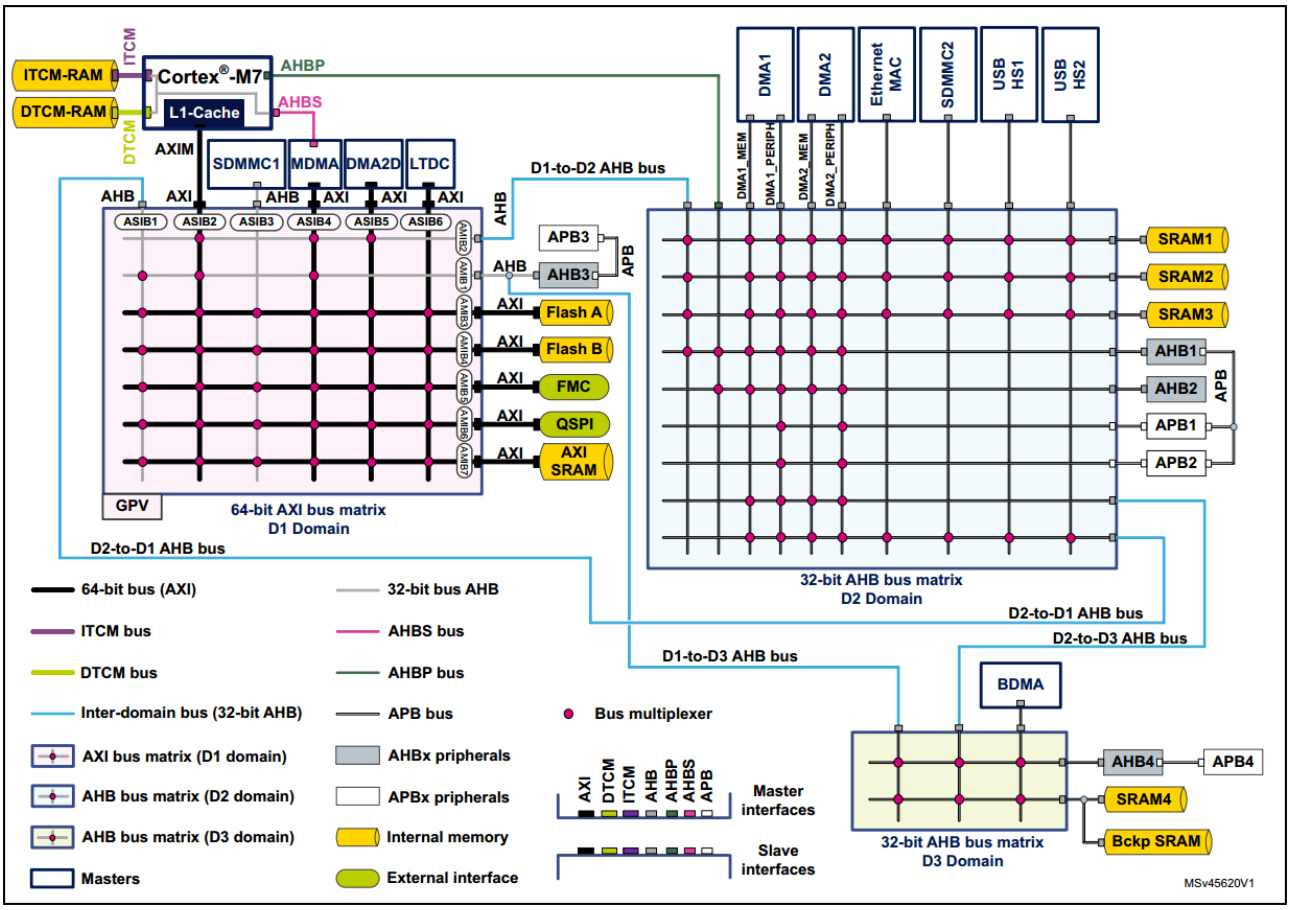
APB1 定时器有 TIM2, TIM3 ,TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14，LPTIM1
APB2 定时器有 TIM1, TIM8 , TIM15, TIM16，TIM17
APB4 定时器有 LPTIM2，LPTIM3，LPTIM4，LPTIM5

3.5 STM32H7 的 AXI 总线

AXI 总线在 STM32H7 中有着举足轻重的作用。高并发性全靠这个总线了，先来看下 AXI 总线的框架：

3.5.1 总线系统框架

下面这个截图比较有代表性，可以帮助大家理解 STM32H7 总线系统。



这个图可以方便识别总线所外挂的外设，共分为三个域：D1 Domain，D2 Domain 和 D3 Domain。

- D1 Domain

  D1 域中的各个外设是挂在 64 位 AXI 总线组成 6*7 的矩阵上。

  - 6 个从接口端 ASIB1 到 ASIB6

    外接的主控是 LTDC，DMA2D，MDMA，SDMMC1，AXIM 和 D2-to-D1 AHB 总线。

  - 7 个主接口端 AMIB1 到 AMIB7

    外接的从设备是 AHB3 总线，Flash A，Flash B，FMC 总线，QSPI 和 AXI SRAM。另外 AHB3 也是由 AXI 总线分支出来的，然后再由 AHB3 分支出 APB3 总线。

- D2 Domain

  D2 域的各个外设是挂在 32 位 AHB 总线组成 10*9 的矩阵上。

  - 10 个从接口

    外接的主控是 D1-to-D2 AHB 总线，AHBP 总线，DMA1，DMA2，Ethernet MAC，SDMMC2，USB HS1 和 USB HS2。

  - 9 个主接口

    外接的从设备是 SRAM1，SRMA2，SRAM3，AHB1，AHB2，APB1，APB2，D2-to-D1 AHB 总线和 D2-to-D3 AHB 总线。

- D3 Domain

D3 域的各个外设是挂在 32 位 AHB 总线组成 3*2 的矩阵上。

- 3 个从接口

  外接的主控 D1-to-D3 AHB 总线，D2-to-D3 AHB 总线和 BDMA。

- 2 个主接口

  外接的从设备是 AHB4，SRAM4 和 Bckp SRAM。另外 AHB4 也是这个总线矩阵分支出来的，然后再由 AHB4 分支出 APB4 总线。

这三个域之间也是有互联的，可以是：

- D1 域到 D2 域的 D1-to-D2 AHB bus

  允许 D1 域中的主接口外设访问 D2 域里面的从接口外设。比如 D1 域里面的 DMA2D 访问 D2 域里面的 SRAM1。

- D2 域到 D1 域的 D2-to-D1 AHB bus

  允许 D2 域中的主接口外设访问 D1 域里面的从接口外设。比如 D2 域里面的 DMA2 访问 D1 域里面的 AXI SRAM。

- D1 域到 D3 域的 D1-to-D3 AHB bus

  允许 D1 域中的主接口外设访问 D3 域里面的从接口外设。比如 D1 域里面的 MDMA 访问 D3 域里面的 SRAM4。

- D2 域到 D3 域的 D2-to-D3 AHB bus。

  允许 D2 域中的主接口外设访问 D3 域里面的从接口外设。比如 D2 域里面的 DMA2 访问 D3 域里面的 SRAM4。

有了这些知识后，下面我们重点了解 AXI 总线矩阵。

3.5.2 AXI 总线特色

AXI 支持高频率、高性能的系统设计：

- 支持高带宽，低延迟设计。
- 提供高频操作，无需复杂的总线桥。
- 满足各种组件的接口需求。
- 适用于具有高初始访问延迟的内存控制器。
- 为互连架构的实现提供了灵活性。
- 与现有的 AHB 和 APB 接口向后兼容。

AXI 总线的关键特性：

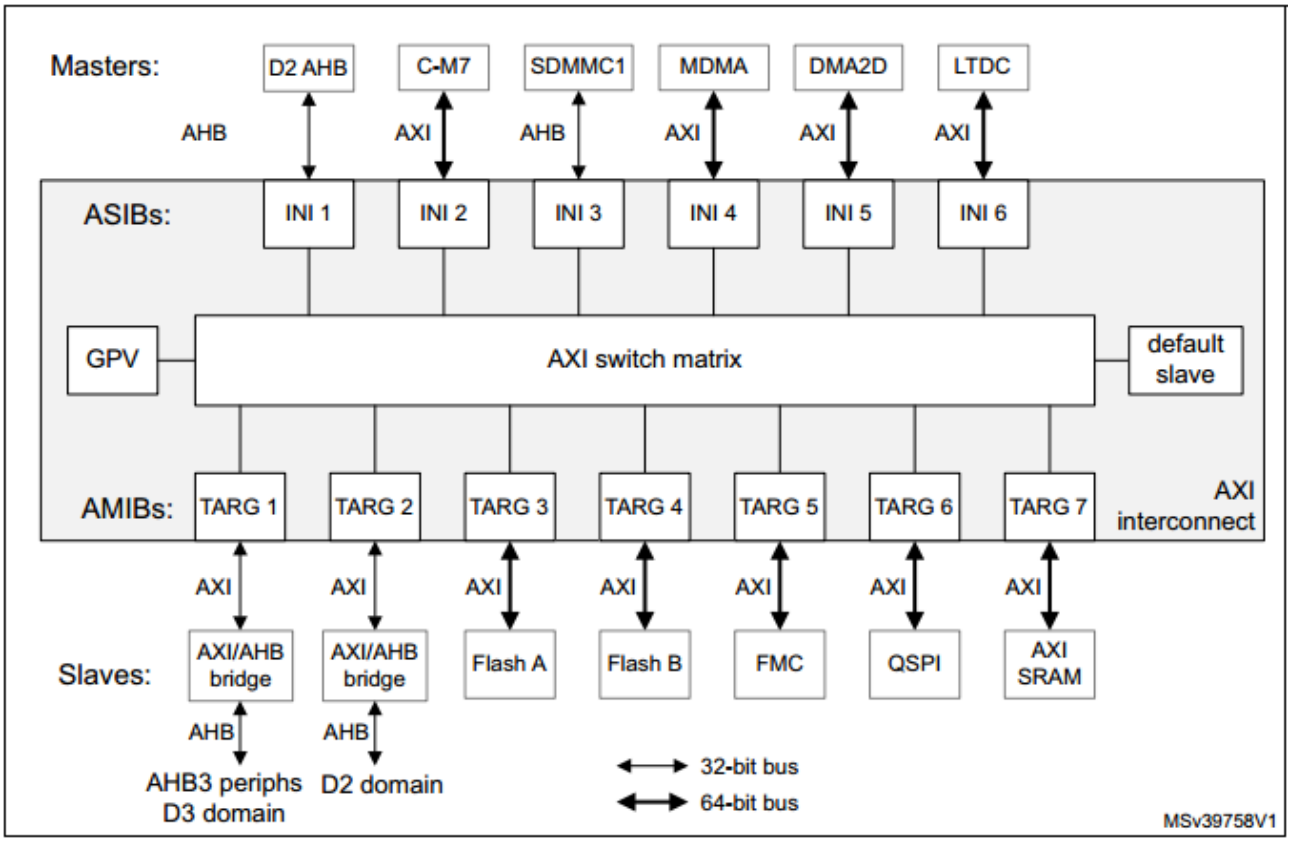
- 独立的地址、控制和数据线。
- 支持非字节对齐方式传输。
- 基于起始地址的突发传输。
- 分开的读和写数据通道，且提供 DMA 传输。
- 支持发起多个地址。
- 支持无序传输。
- 允许添加寄存器，以提供时序收敛。

- AXI 总线

关于 AXI 总线，ARM 有一篇古文观止级文档《AXI4_specification》，已经将其放在了论坛：http://www.armbbs.cn/forum.php?mod=viewthread&tid=87340 。

3.5.3 AXI 总线简介

通过下面的框图，我们再进一步的认识一下 AXI 总线。



通过上面的截图，我们可以看到，AXI 总线有 6 个从接口 ASIBs（AMBA slave interface blocks）和 7 个主控接口 AMIBs（AMBA master interface blocks）。

针对从接口 ASIBs，描述如下：

ASIB	连接的主控	协议	总线带宽	读/写发起能力
IN1	D2 域的 AHB 总线	AHB-lite	32	1/4
IN2	Cortex-M7	AXI4	64	7/32
IN3	SDMMC1	AHB-lite	32	1/4
IN4	MDMA	AXI4	64	4/1
IN5	DMA2D	AXI4	64	2/1
IN6	LTDC	AXI4	64	1/1

重点注意最后一列，STM32H7 参考手册里面原始的描述是 R/W issuing，这里将其翻译为读/写发起能力。比如输入通道 IN5 连接的主控 DMA2D，支持的读发起能力是 2，写发起能力是 1。读发起能力是 2 该如何理解呢？这里的含义是存在两路读信号同时进行（因为 AXI 接口有一个 FIFO 的功能，可供同时进行，更深入的认识有待研究），反映到 DMA2D 的实际应用中，就是 DMA2D 同时读取前景色和背景色的缓存区做 Alpha 融合之类的操作。写操作同理，DMA2D 的写发起能力仅支持一路。

针对主控接口 AMIBs，描述如下：

ASIB	连接的从设备	协议	总线带宽	读/写发起能力/总接收能力
TARG1	D3 域的 AHB 总线	AXI4	32	1/1/1
TARG2	D2 域的 AHB 总线	AXI4	32	1/1/1
TARG3	Flash A	AXI4	64	3/2/5
TARG4	Flash B	AXI4	64	3/2/5
TARG5	FMC	AXI4	64	3/3/6
TARG6	QUADSPI	AXI4	64	2/1/6
TARG7	AXI SRAM	AXI3	64	2/2/2

跟上面表格的含义是一样的，同样重点注意最后一列，这里多了一个总接收能力（Total acceptance），也就是读发起能力和写发起能力同时执行的情况。

3.5.4 AXI 总线优先级编程

由于存在多个 ASIB 从接口访问 AMIB 主控的问题，这就涉及到谁先谁后等问题。所以 AXI 总线矩阵就做了一个基于优先级的仲裁方案。每个 ASIB 接口支持读通道和写通道分别设置，优先级从 0 到 15。数值越大，优先级越高，默认情况都是优先级 0。如果有两个传输同时到达 AMIB 主控接口，那么优先级高的 ASIB 接口传输优先处理；如果优先级相同的话，根据 LUR 方案选择（least recently-used 最近最少使用情况）。

大家在实际应用中，可以根据实际情况进行设置，一般情况下使用默认值即可。

3.6 STM32H7 的总线互联

STM32H7 的总线矩阵四通八达，但不是任意 Bus Master 总线主控端和 Bus Slave 设备端都可以相互通信的：

	Bus master / type(1)																			
	Cortex-M7 - AXIM	Cortex-M7 - AHBP	**Cortex-M7 - ITCM**	**Cortex-M7-DTCM**	SDMMC1	**MDMA - AXI**	MDMA - AHBS	**DMA2D**	**LTDC**	DMA1 - MEM	DMA1 - PERIPH	DMA2 - MEM	DMA2 - PERIPH	Eth. MAC - AHB	SDMMC2 - AHB	USBHS1 - AHB	USBHS2 - AHB	BDMA - AHB		
Bus slave / type(1)	**Interconnect path and type(2)**																			
ITCM	-	-	D	-	-	-	7	-	-	-	-	-	-	-	-	-	-	-	-	-
DTCM	-	-	-	D	-	-	7	-	-	-	-	-	-	-	-	-	-	-	-	-
AHB3 peripherals	1	-	-	-	-	1	-	-	-	21	21	-	21	21	-	21	21	21	21	-
APB3 peripherals	14	-	-	-	-	14	-	-	-	214	214	-	214	214	-	214	214	214	214	-
Flash A	**1**	-	-	-	*1*	**1**	-	**1**	**1**	*21*	*21*	-	*21*	*21*	-	*21*	*21*	*21*	*21*	-
Flash B	**1**	-	-	-	*1*	**1**	-	**1**	**1**	*21*	*21*	-	*21*	*21*	-	*21*	*21*	*21*	*21*	-
AXI SRAM	**1**	-	-	-	*1*	**1**	-	**1**	**1**	*21*	*21*	-	*21*	*21*	-	*21*	*21*	*21*	*21*	-
QUADSPI	**1**	-	-	-	*1*	**1**	-	**1**	**1**	*21*	*21*	-	*21*	*21*	-	*21*	*21*	*21*	*21*	-
FMC	**1**	-	-	-	*1*	**1**	-	**1**	**1**	*21*	*21*	-	*21*	*21*	-	*21*	*21*	*21*	*21*	-
SRAM 1	12	-	-	-	-	12	-	12	-	2	2	-	2	2	-	2	2	2	2	-
SRAM 2	12	-	-	-	-	12	-	12	-	2	2	-	2	2	-	2	2	2	2	-
SRAM 3	12	-	-	-	-	12	-	12	-	2	2	-	2	2	-	2	2	2	2	-
AHB1 peripherals	12	2	-	-	-	12	-	12	-	2	2	-	2	2	-	-	-	-	-	-
APB1 peripherals	125	25	-	-	-	125	-	125	-	25	25	2	25	25	2	-	-	-	-	-
AHB2 peripherals	-	2	-	-	-	-	-	-	-	2	2	-	2	2	-	-	-	-	-	-
APB2 peripherals	125	25	-	-	-	125	-	125	-	25	25	2	25	25	2	-	-	-	-	-
AHB4 peripherals	13	-	-	-	-	13	-	-	-	23	23	-	23	23	-	213	213	213	213	3
APB4 peripherals	136	-	-	-	-	136	-	-	-	236	236	-	236	236	-	213	213	213	213	36
SRAM4	13	-	-	-	-	13	-	-	-	23	23	-	23	23	-	213	213	213	213	3
Backup RAM	13	-	-	-	-	13	-	-	-	23	23	-	23	23	-	213	213	213	213	3

- 黑色加粗字体是 64 位总线（ITCM，DTCM，Flash A，Flash，AXI SRAM，FMC 等），普通字体是 32 位总线。
- 访问通路（每个小方块里面的字符）

  任何有数字的表示有访问通路。

  短横杠“-”表示不可访问。

  有灰色阴影的表示有实用价值的访问通路。
- 表格中具体数值所代表的含义

  D=direct

  1=via AXI bus matrix

2=via AHB bus matrix in D2

3=via AHB bus matrix in D3

4=via AHB/APB bridge in D1

5=via AHB/APB bridge in D2

6=via AHB/APB bridge in D3

7=via AHBS bus of Cortex-M7

多个数值组合 = 互连路径以数字的顺序经过多个矩阵或/和桥。

- 总线访问类型

普通字体表示 32 位总线。

斜体表示 32 位总线主机端/ 64 位总线从机端。

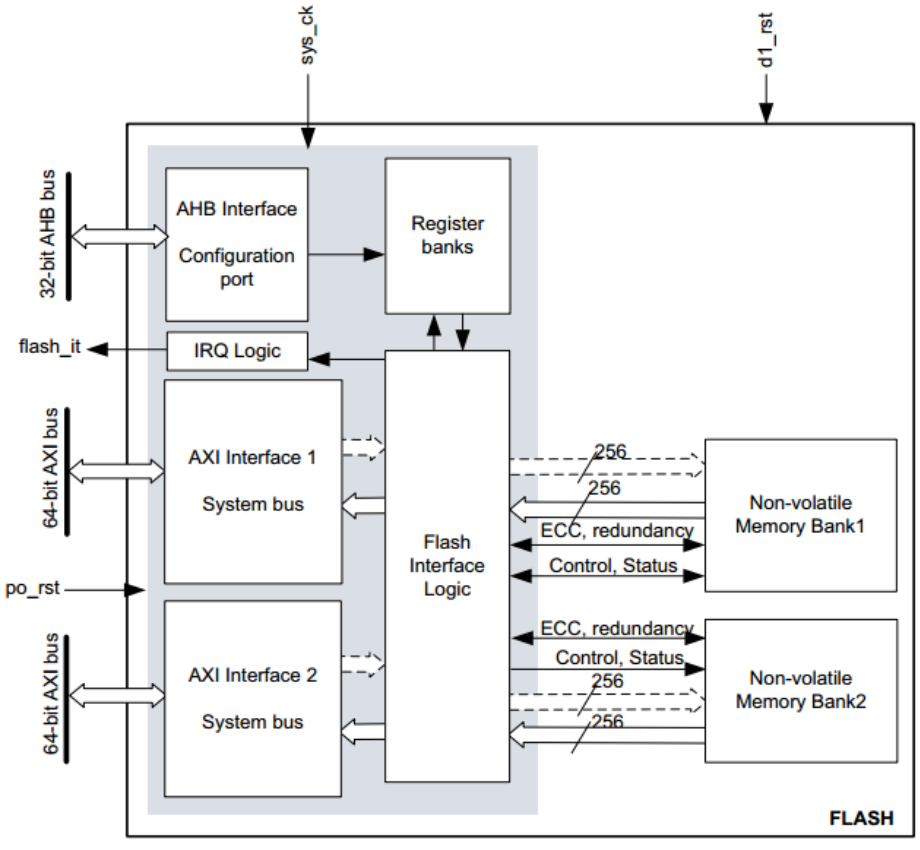
粗体表示 64 位总线。

当前要对这个图有个了解，后面章节讲解各个外设的时候要用到，比如 DTCM 和 ITCM 不支持 DMA1，DMA2 和 BDMA，仅支持 MDMA。

3.7 STM32H7 的 FLASH

首次学习 STM32H7，要掌握以下几点认识即可：

- 双 BANK，每个 BANK 的带宽都是 64bits，如下图所示：



◆ H7 中 Flash 的延迟和主频关系。

H7 中已经没有 F1 和 F4 系列中的 ART Chrome 加速，通过 H7 中的 Cache 加速即可。具体延迟数值和主频关系如下：

Number of wait states (LATENCY)	Programming delay (WRHIGHFREQ)	AXI Interface clock frequency vs V_{CORE} range		
		VOS3 range 0.95 V - 1.05 V	VOS2 range 1.05 V - 1.15 V	VOS1 range 1.15 V - 1.26 V
0 WS (1 FLASH clock cycle)	00	[0 ; 45 MHz]	[0 ; 55 MHz]	[0 ; 70 MHz]
1 WS (2 FLASH clock cycles)	01	]45 MHz ; 90 MHz]	]55 MHz ; 110 MHz]	]70 MHz ; 140 MHz]
2 WS (3 FLASH clock cycles)	01	]90 MHz ; 135 MHz]	]110 MHz ; 165 MHz]	]140 MHz ; 185 MHz]
	10	-	-	]185 MHz ; 210 MHz]
3 WS (4 FLASH clock cycles)	10	]135 MHz ; 180 MHz]	]165 MHz ; 225 MHz]	]210 MHz ; 225 MHz]
4 WS (5 FLASH clock cycles)	10	]180 MHz ; 225 MHz]	225 MHz	-

对于上面的表格，大家可以看到，当延迟等待设置为 0 的时候，即无等待，单周期访问，速度可以做到 70MHz。增加 1 个 Flash 周期后，访问速度可以做到 140MHz。当增加到 3 个或 4 个 Flash 周期后，最高速度可以做到 225MHz。

◆ Flash 编程操作（写）最好以 256bits 为单位进行，应用中也可以小于 256bits，但是容易造成 ECC 校验出问题，所以不推荐。Flash 读操作支持 64bits，32bits，16bits 和 8bits。

◆ Flash 支持 ECC 校验，每 256bits 配 10bit 的 ECC 位，可以检测到 1 个 bit 并纠正或者检测 2 个 bit。随着芯片的制造工艺水平越高，带电粒子能产生的位翻转就越多，此时的 ECC 是必须要有的，一般可以纠正 1-2 个 bit。安全等级高的 Flash 类存储器和 RAM 类都是必须要带 ECC 的。

- ECC 相关知识

关于 ECC 方面的知识，专门整理了一个帖子：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=86777 。

3.8 STM32H7 的 RAM

（注，学习本小节，优先回顾本章节 3.5.1 小节的框图，另外 H7 的 RAM 区也是支持硬件 ECC 的）

STM32H7 的 RAM 区分为好几个部分，下面分别进行说明：

◆ TCM 区

TCM : Tightly-Coupled Memory 紧密耦合内存 。ITCM 用于运行指令，也就是程序代码，DTCM 用于数据存取，特点是跟内核速度一样，而片上 RAM 的速度基本都达不到这个速度，所以有降频处理。

速度：400MHz。

DTCM 地址：0x2000 0000，大小 128KB。

ITCM 地址：0x0000 0000，大小 64KB。

- AXI SRAM 区

位于 D1 域，数据带宽是 64bit，挂在 AXI 总线上。除了 D3 域中的 BDMB 主控不能访问，其它都可以访问此 RAM 区。

速度：200MHz。

地址：0x2400 0000，大小 512KB。

用途：用途不限，可以用于用户应用数据存储或者 LCD 显存。

- SRAM1，SRAM2 和 SRAM3 区

位于 D2 域，数据带宽是 32bit，挂在 AHB 总线上。除了 D3 域中的 BDMB 主控不能访问这三块 SRAM，其它都可以访问这几个 RAM 区。

速度：200MHz。

SRAM1：地址 0x3000 0000，大小 128KB，用途不限，可用于 D2 域中的 DMA 缓冲，也可以当 D1 域断电后用于运行程序代码。

SRAM2：地址 0x3002 0000，大小 128KB，用途不限，可用于 D2 域中的 DMA 缓冲，也可以用于用户数据存取。

SRAM3：地址 0x3004 0000，大小 32KB，用途不限，主要用于以太网和 USB 的缓冲。

- SRAM4 区

位于 D3 域，数据带宽是 32bit，挂在 AHB 总线上，大部分主控都能访这块 SRAM 区。

速度：200MHz。

地址：0x3800 0000，大小 64KB。

用途：用途不限，可以用于 D3 域中的 DMA 缓冲，也可以当 D1 和 D2 域进入 DStandby 待机方式后，继续保存用户数据。

- Backup SRAM 区

备份 RAM 区，位于 D3 域，数据带宽是 32bit，挂在 AHB 总线上，大部分主控都能访问这块 SRAM 区。

速度：200MHz。

地址：0x3880 0000，大小 4KB。

用途：用途不限，主要用于系统进入低功耗模式后，继续保存数据（Vbat 引脚外接电池）。

3.9 总结

本章节就为大家讲解这么多，让大家对 STM32H7 有个整体的认识，后面章节将逐个进行学习。

第4章 STM32H7 工程模板建立（MDK5）

本章教程为大家介绍如何创建 MDK5 工程，配置方法以及经常用到的一些配置项。

4.1 初学者重要提示
4.2 第 1 步，创建工程文件夹
4.3 第 2 步，创建新工程
4.4 第 3 步，设置工程名和工程分组名
4.5 第 4 步，添加工程文件
4.6 第 5 步，Target Options 配置
4.7 第 6 步，最后一步验证是否创建成功
4.8 MDK 的字体，多核并行编译，语法提示等设置
4.9 总结

4.1 初学者重要提示

- KEIL 官方有一个专门的 MDK 上手文档 mdk5-getting-started，在 MDK 的安装目录里面，坛友翻译了整个文档：http://www.armbbs.cn/forum.php?mod=viewthread&tid=31288 。
- MDK5 的工程创建过程还是比较重要的，务必实际上手操作一遍。

4.2 第 1 步，创建工程文件夹

创建一个文件夹，里面包含如下文件（直接将跑马灯例子的 Project 文件里面内容全部删除，然后整个工程复制进来即可）。

- Doc 文件夹

  用于记录例程的修改记录和例子简介。

- Libraries 文件夹

  在第一章详细介绍从官网的下载方法。

- Project 文件夹

  这个是本章的重点，在这个文件夹里面再建立两个文件夹，MDK-ARM(uV5)文件夹里面用于存放 MDK 工程，EWARMv8 文件夹用于存放 IAR 工程，方便以后创建 IAR 的工程。

- User 文件夹

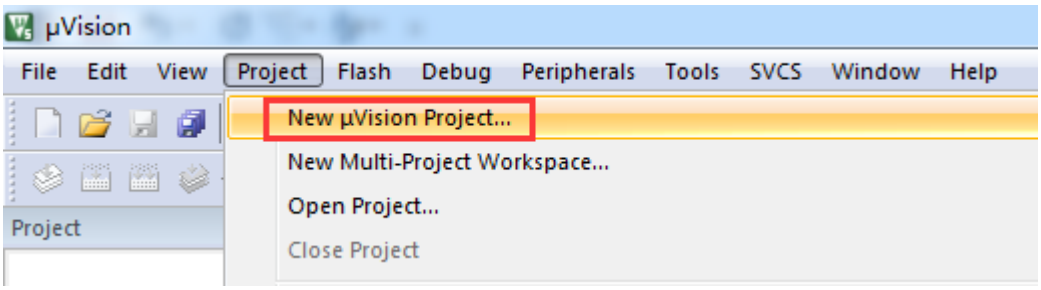
  这个是要用户提供的，开发板已经都提供好，bsp 文件夹里面是开发板各个外设的驱动，segger 文件夹是硬件异常分析代码。

- 删除目标文件.bat

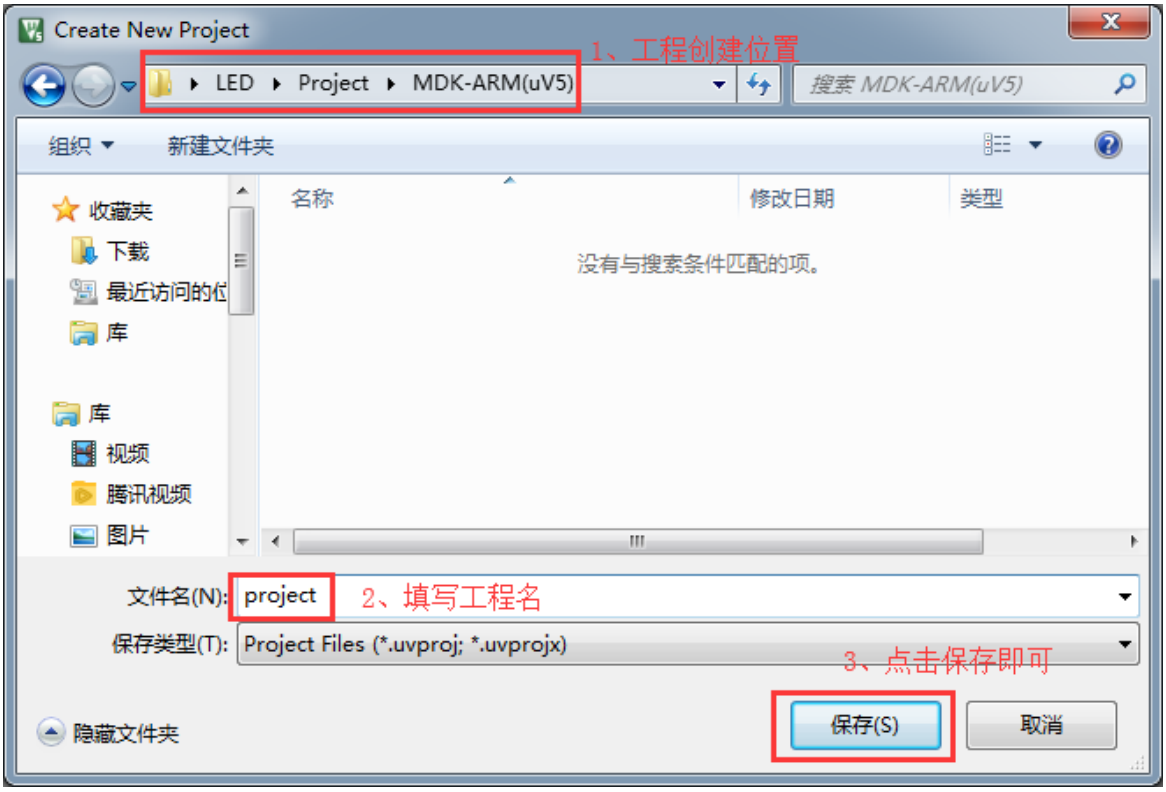
  这个是批处理文件，用于删除 MDK 和 IAR 工程编译后生成的 obj 和 list 临时文件。

4.3 第 2 步，创建新工程

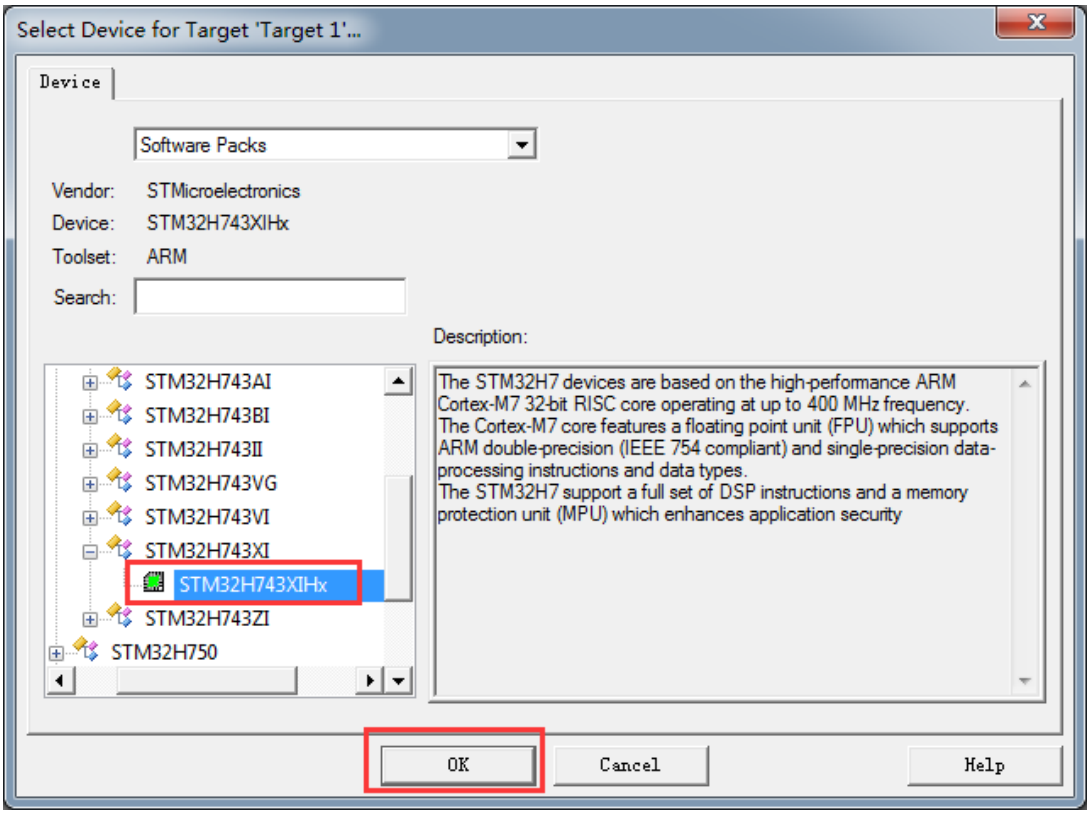
打开 MDK，选择 Project ->New uVision Project：



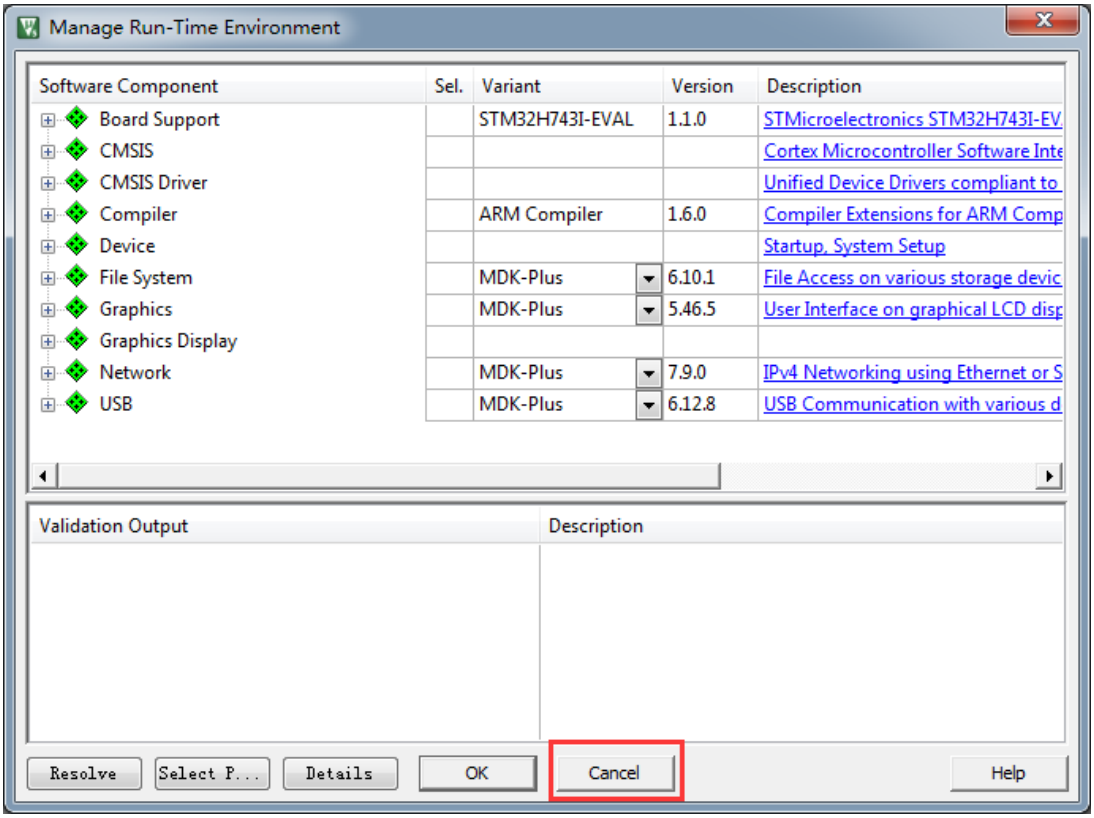
工程名设置为 project：



选择器件：STM32H743XIHx

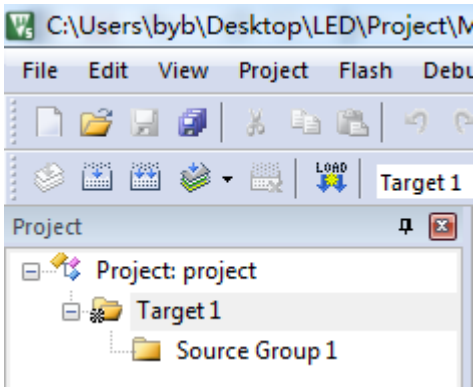


点击 OK 后弹出如下界面，点击 Cancel 取消，当前我们还不使用 RTE 环境创建，

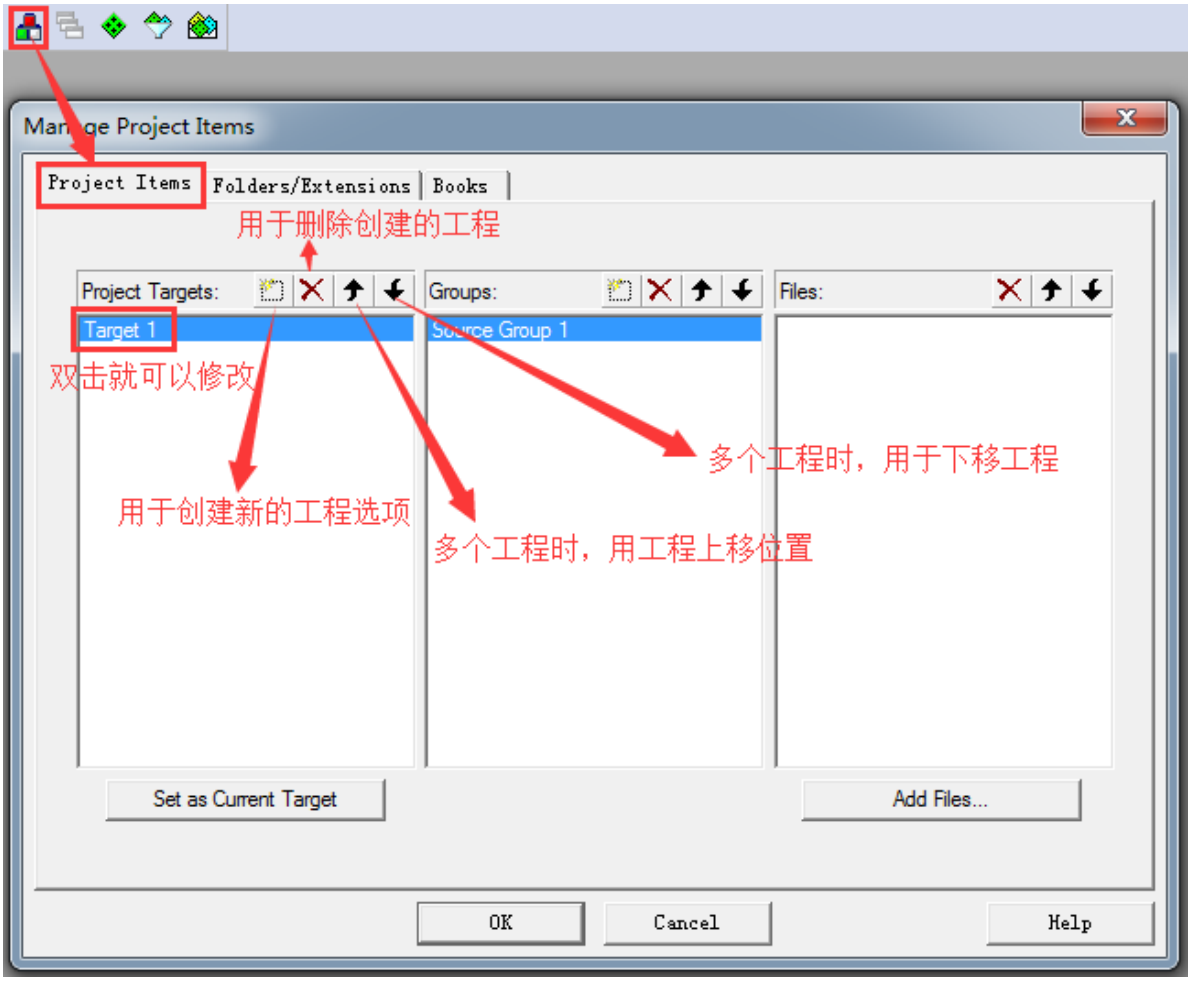


4.4 第 3 步，设置工程名和工程分组名

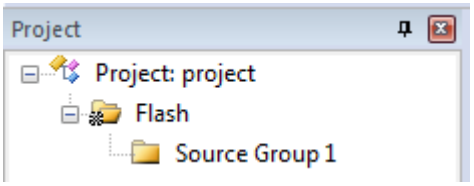
默认新建的工程是下面这样的：



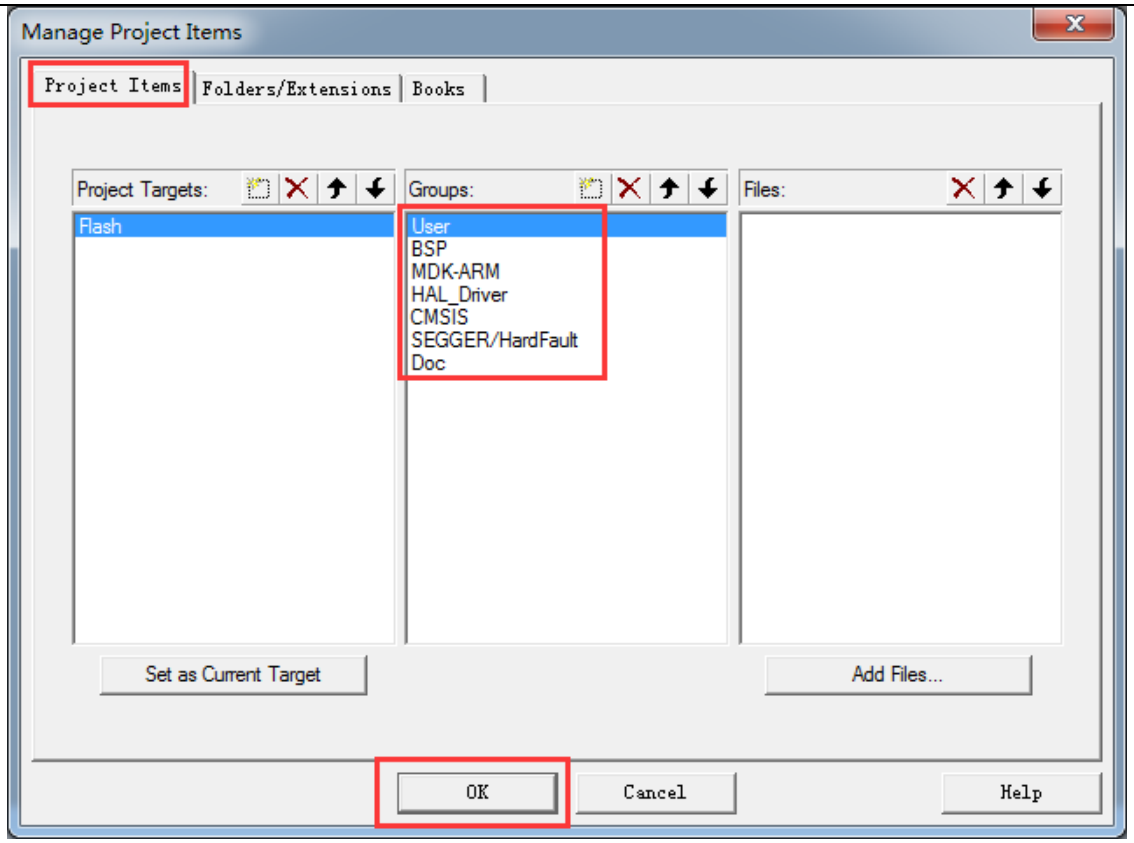
打开 Manage Project Items，双击 Targer1 修改名字为 Flash，或者任何其它方便自己管理的名称，设置完毕后点击 OK。



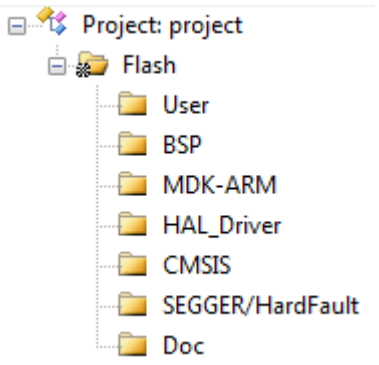
可以看到已经修改为 Flash：



再次打开 Manage Project Items，添加如下分组内容：

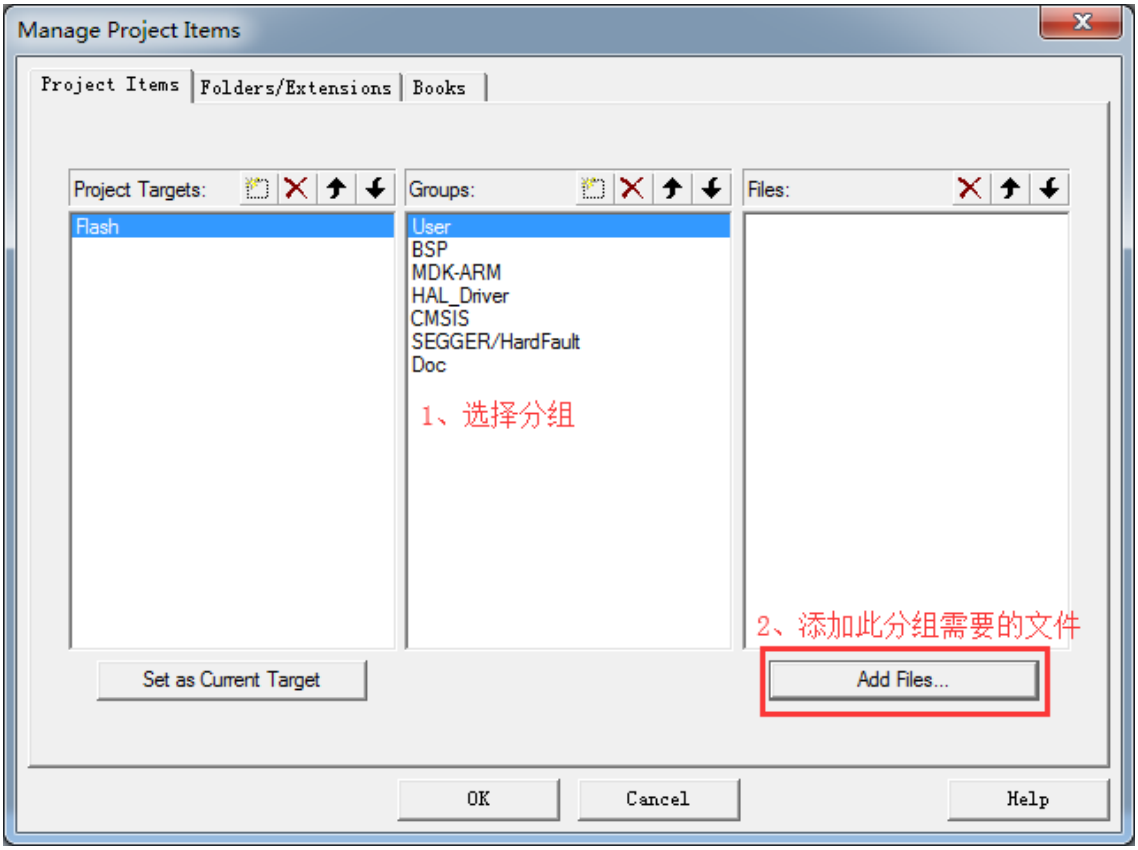


可以看到左侧工程列表已经更新：

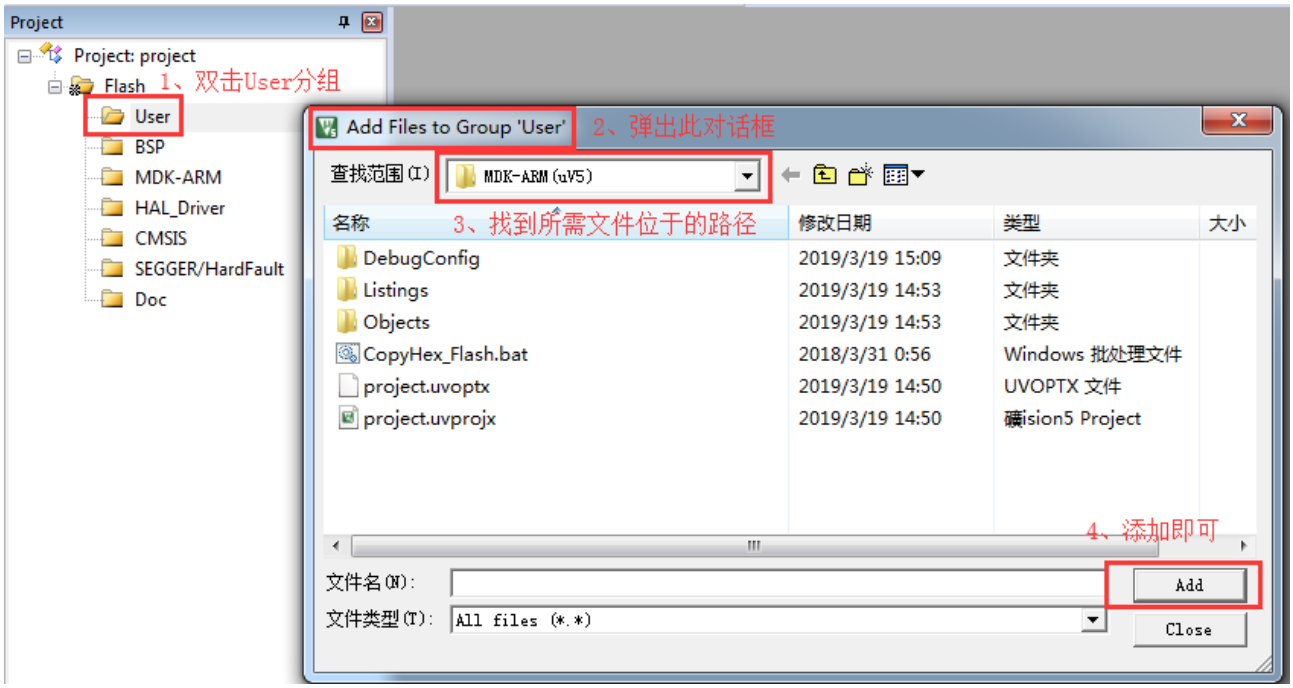


4.5 第 4 步，添加工程文件

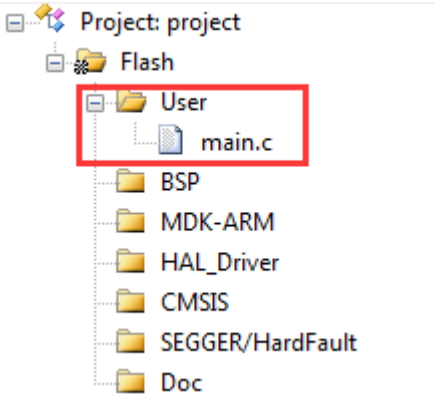
添加方法跟第 2 步，第 3 步一样，可以通过 Manage Project Items 添加：



也可以直接双击工程列表里面的分组添加（注意文件类型里面推荐设置为 ALL files 所有文件）：

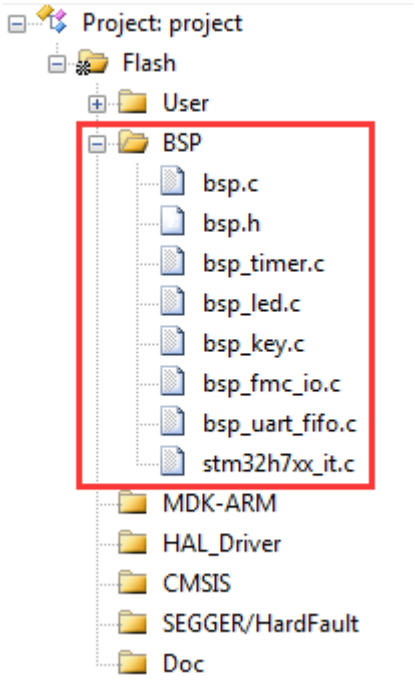


◆ 添加 User 分组需要的文件



main.c 文件位于文件夹 User，实现了一个简单的跑马灯效果。

- 添加 BSP 分组需要的内容



bsp.c 位于路径\User\bsp，主要用于 MPU、Cache、时钟和外设初始化。

bsp.h 位于路径\User\bsp，此文件是各种头文件的集中汇总贴，方便管理。

bsp_led.c 位于路径\User\bsp\src，用于 LED 驱动。

bsp_key.c 位于路径\User\bsp\src，用于按键驱动。

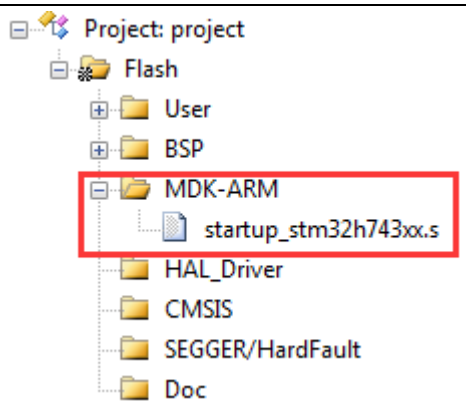
bsp_timer.c 位于路径\User\bsp\src，用于软件定时器组和微妙级单次延迟。

bsp_fmc_io.c 位于路径\User\bsp\src，用于 IO 扩展，这里 LED 要用到。

bsp_uart_fifo.c 位于路径\User\bsp\src，用于 8 串口 FIFO 驱动。

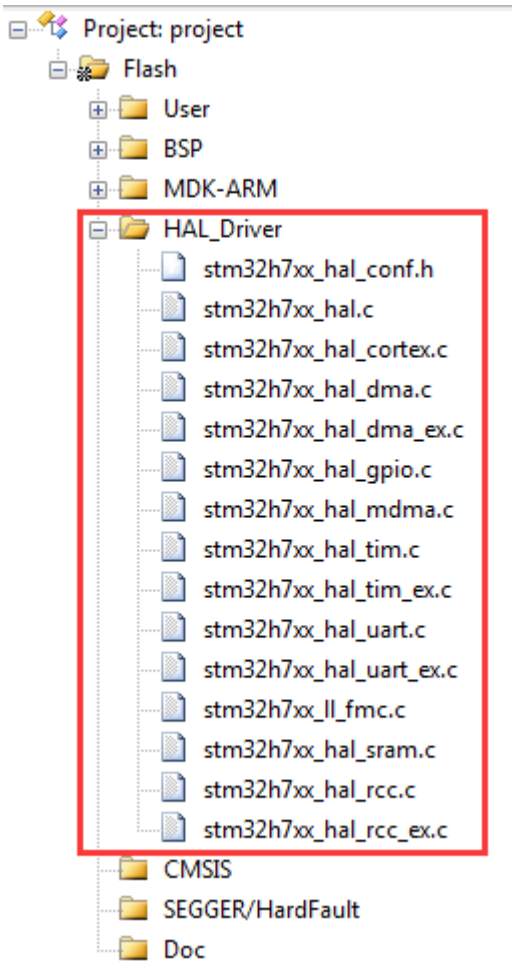
stm32h7xx_it.c 位于路径 User\bsp，用于部分系统中断服务程序。

- 添加 MDK-ARM 分组需要的内容



startup_stm32h743xx.s 位于路径\Libraries\CMSIS\Device\ST\STM32H7xx\Source\Templates\arm，主要用于堆和栈空间设置，中断向量表也在这个文件。

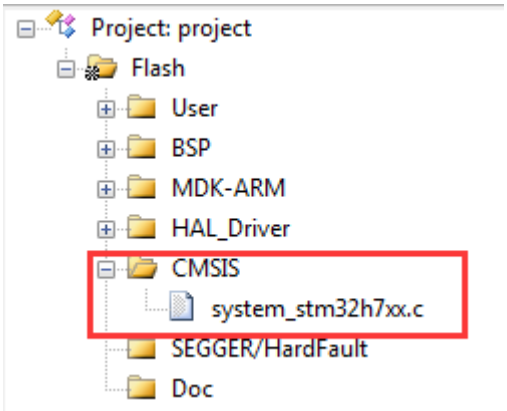
◆ 添加 HAL_Driver 分组需要的内容



stm32h7xx_hal_conf.h 位于路径\User\bsp，主要用于 HAL 库的配置。

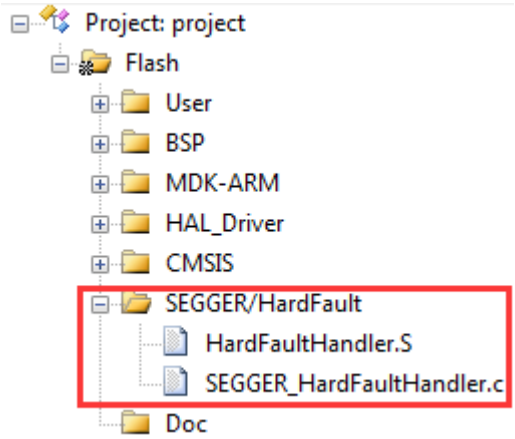
其它文件路径：\Libraries\STM32H7xx_HAL_Driver\Src，用到什么外设添加什么驱动文件即可，有些外设驱动跟 DMA 也有关联，直接添加进来即可。

◆ 添加 CMSIS 分组需要的内容



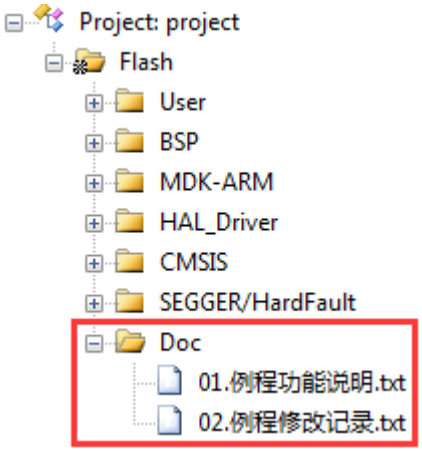
system_stm32h7xx.c 位于路径\Libraries\CMSIS\Device\ST\STM32H7xx\Source\Templates，主要用于复位 RCC 相关寄存器和中断向量表位置设置。

◆ 添加 SEGGER/HardFault 分组需要的内容



这两个文件位于路径：\User\segger\HardFaultHandlerMDK，主要用于硬件异常分析。

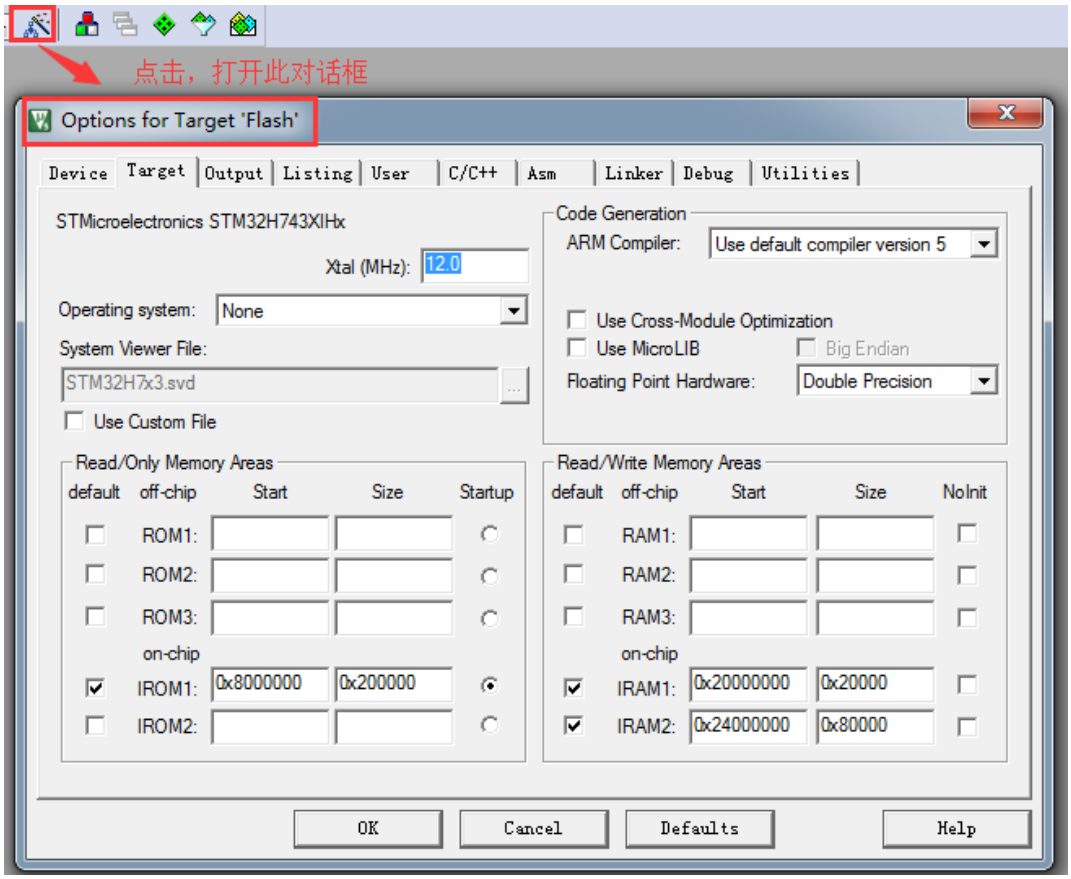
◆ 添加 Doc 分组需要的内容



这两个文件位于 Doc 文件夹，主要用于说明例程的功能和更新记录。

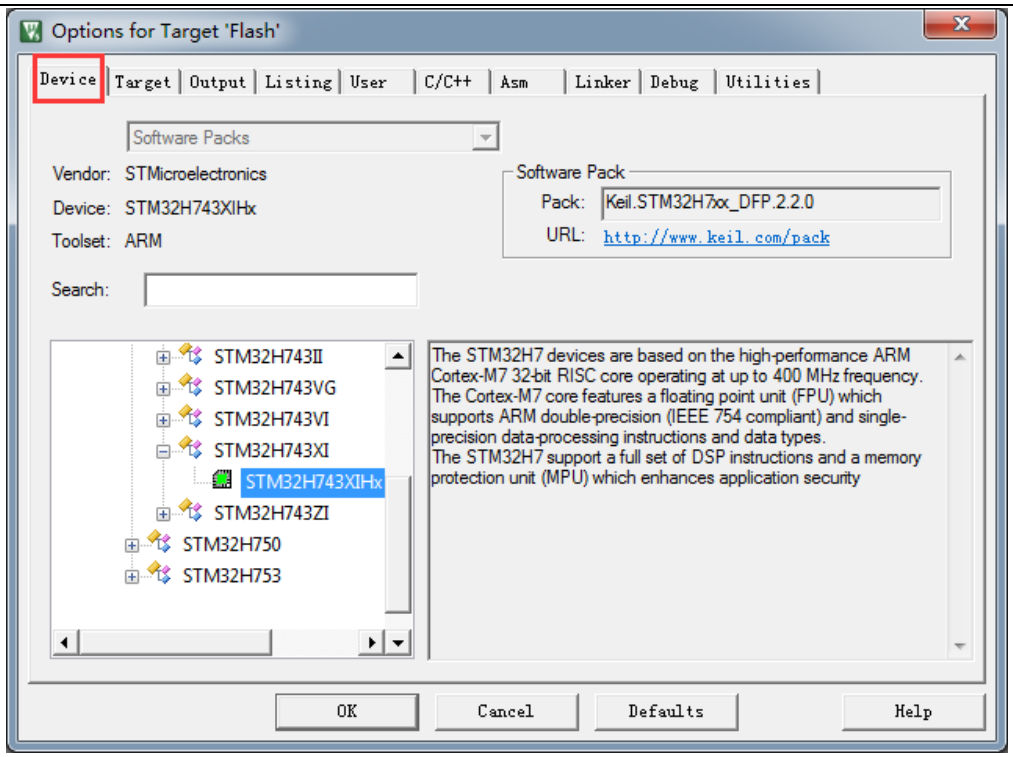
4.6 第 5 步，Target Options 配置

MDK 的配置基本都集中在 Target Options 对话框里面：

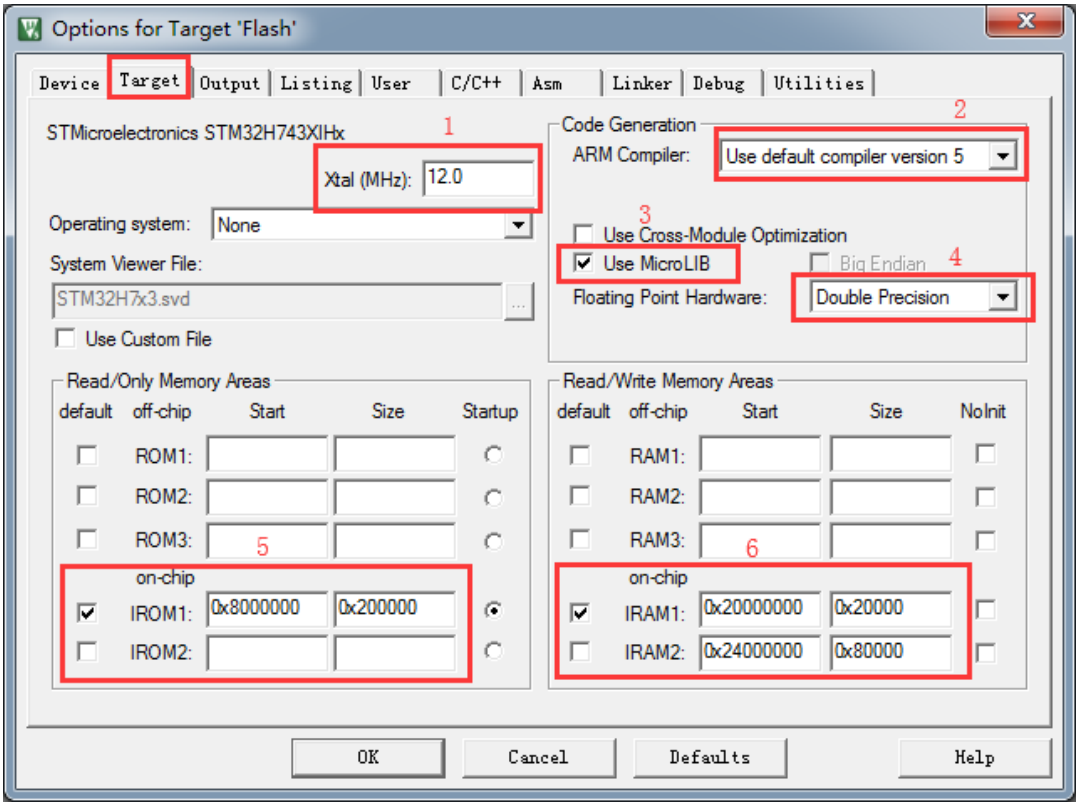


4.6.1 Device 选项

这个选项不用管，因为我们在创建的时候已经选择了 STM32H743XIHx

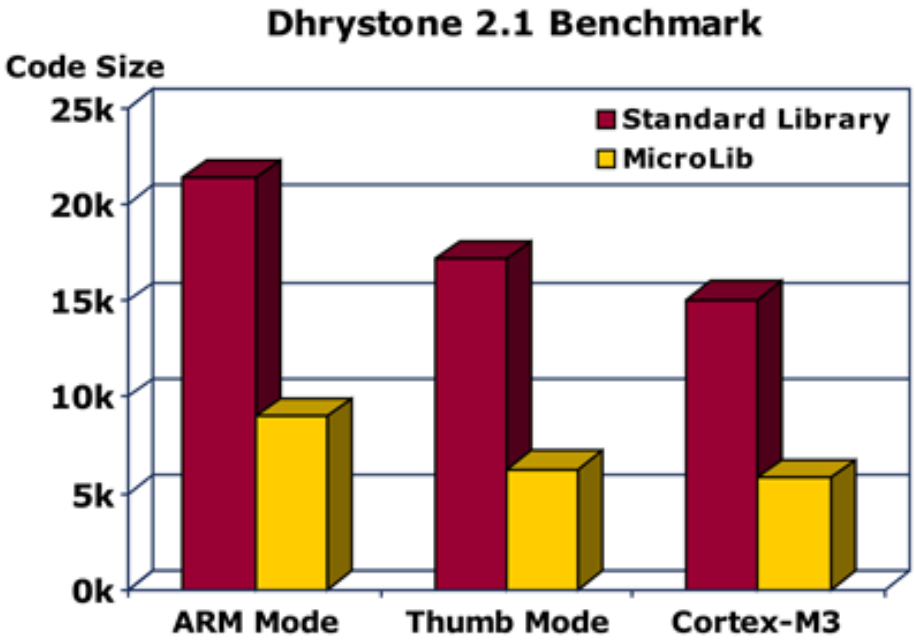


4.6.2 Target 选项



1. 因为我们一般不用软件仿真，这个选项不用管，也不推荐大家用 MDK 提供的软件仿真，问题比较多。

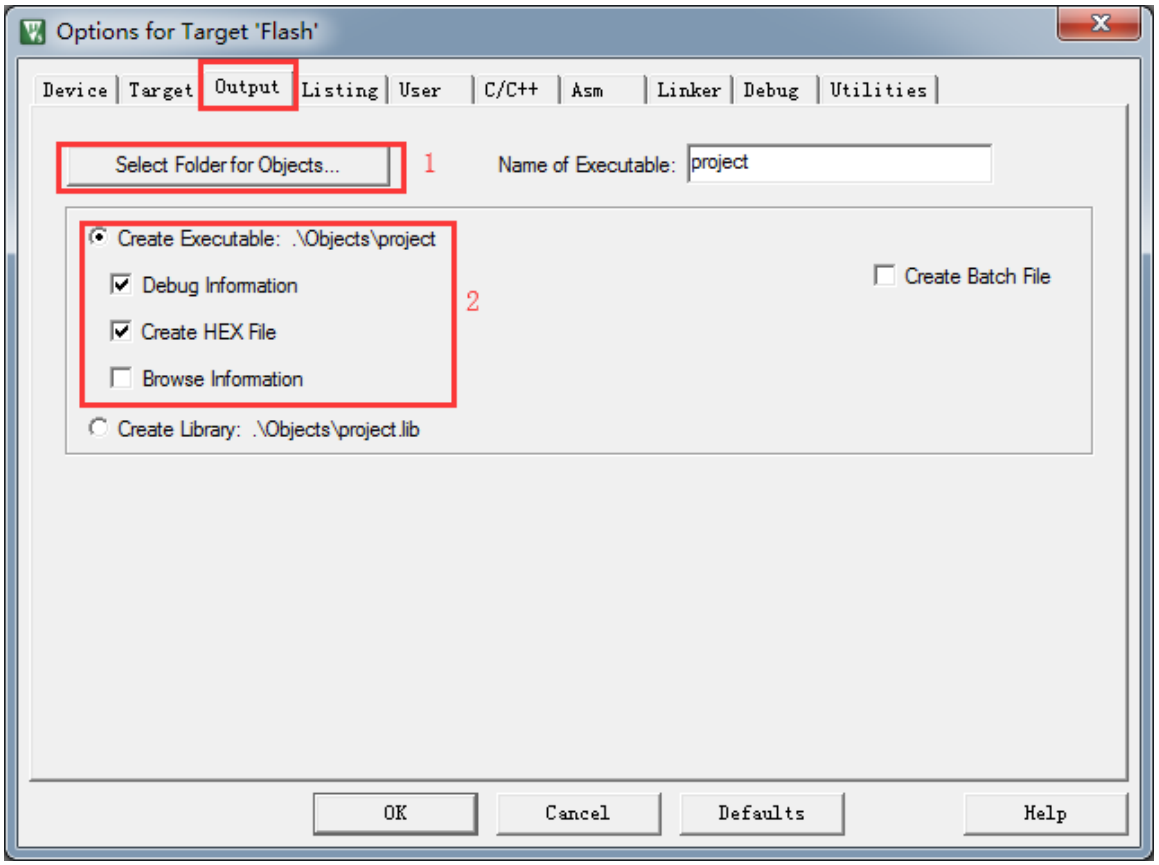
2. 选择编译器，使用默认的 AC5 即可。当前 AC6 支持的还不够好，仅 MDK 自家的软件都支持了，第三方的问题稍多。
3. 我们这里使用微库 MicroLib，这个库针对嵌入式应用，MicroLIB 做了深度优化，比使用 C 标准库所需的 RAM 和 FLASH 空间都大大减小。比如调用：<math.h>，<stdlib.h>，<stdio.h>，<string.h>。



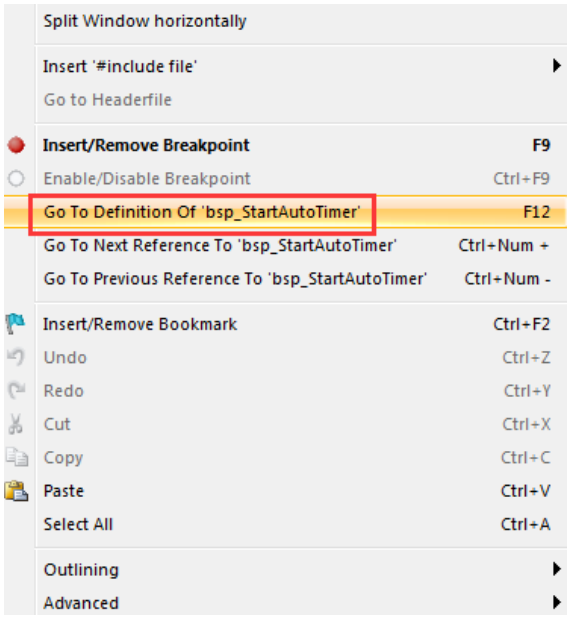
更多这方面的信息可以看 KEIL 官网：http://www.keil.com/arm/microlib.asp 。

4. STM32H7 支持双精度浮点，所以这里选择 Double Precision 双精度浮点。
5. STM32H743XIH 的 Flash 首地址是 0x0800 0000，大小 0x200000，即 2MB。
6. 默认的 RAM 空间选择 DTCM，首地址 0x2000 0000，大小 0x20000，即 128KB。地址 0x2400 0000，大小 0x80000，即 512KB 是 AXI SRAM 的空间。

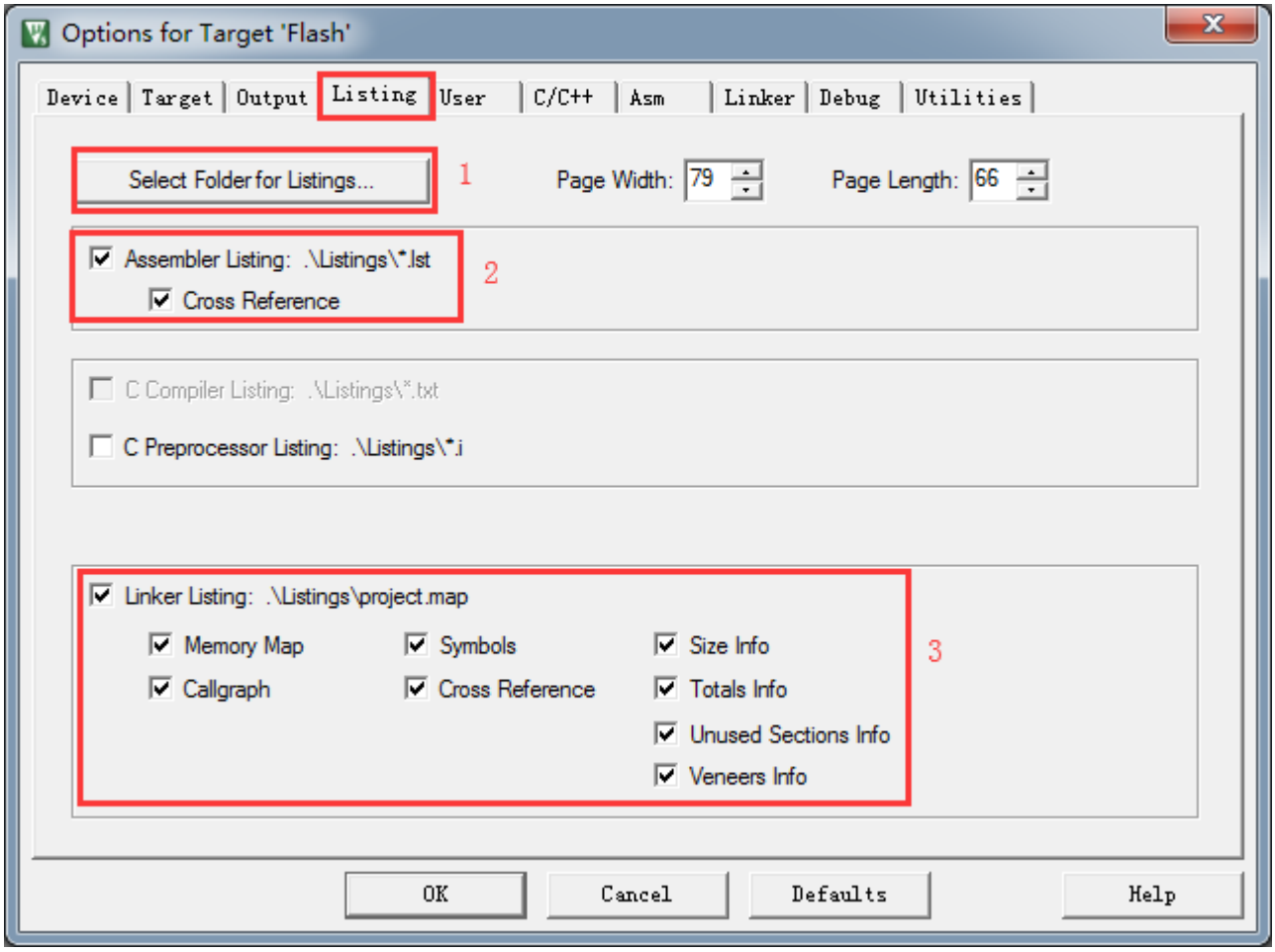
4.6.3 Output 选项



1. 用于存放 MDK 编译过程中生成的 obj 临时文件路径，MDK 在创建的时候已经帮我们创建好文件夹 Objects，这里无需修改，使用默认即可。
2. Debug Information 表示调试信息，选上。
3. Create HEX File 表示是否生成 hex 文件，选上。
4. Browse Information 表示是需要 go to def 的支持，使用 HAL 库话，推荐先不要选上，因为 MDK5 使用 AC5 编译 HAL 库时间比较长.



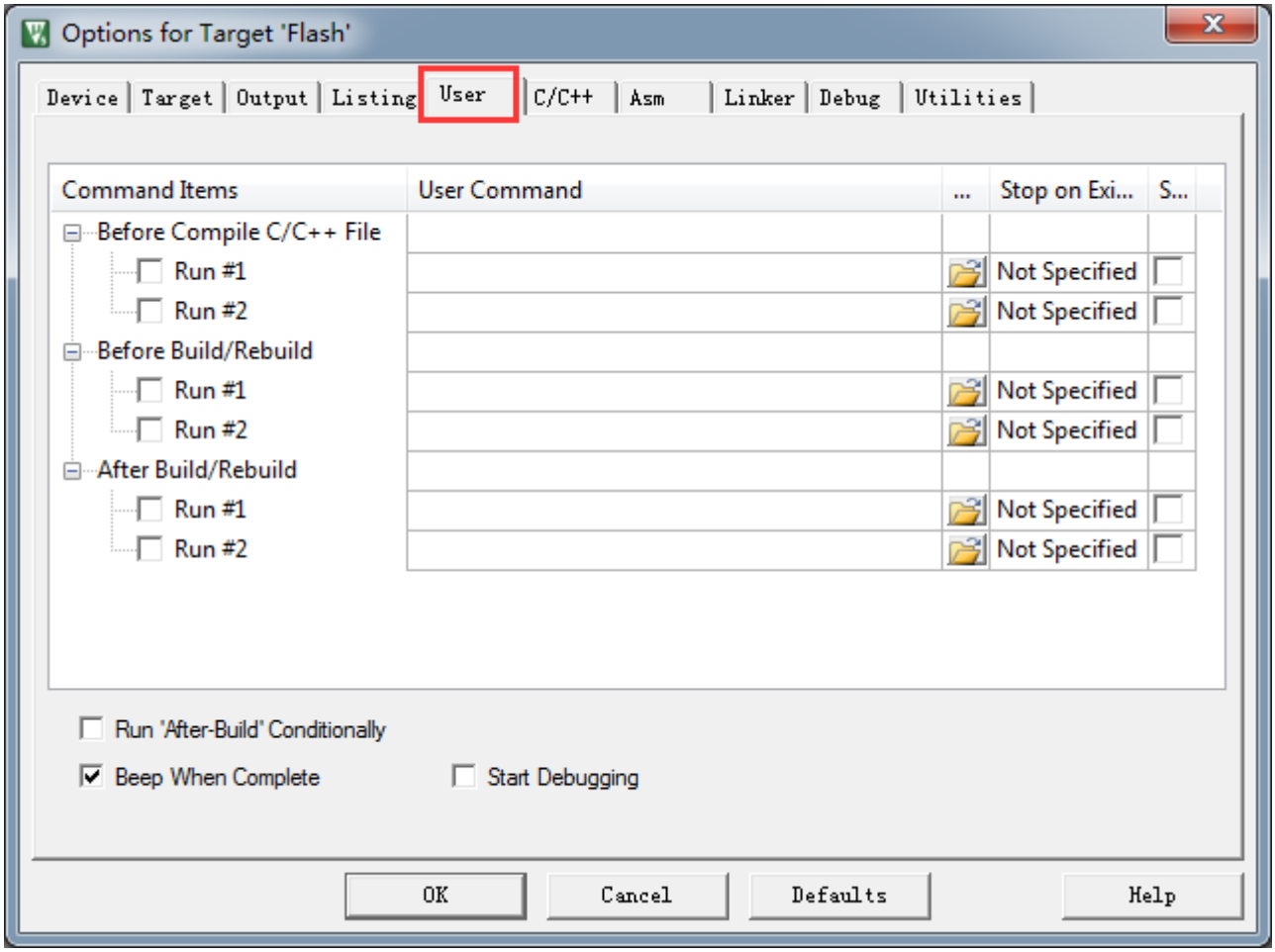
4.6.4 Listing 选项



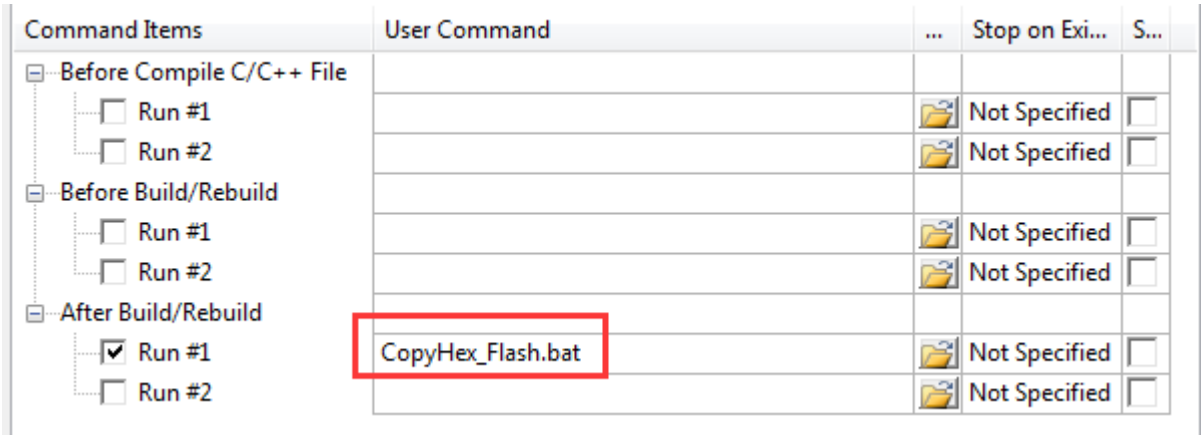
1. 用于存放 MDK 编译过程中生成的 list 临时文件路径，MDK 在创建的时候已经帮我们创建好文件夹 Listings，这里无需修改，使用默认即可。

2. 是否生成*.lst 文件，选上。
3. 查看 MAP 文件要用上，全部选上。

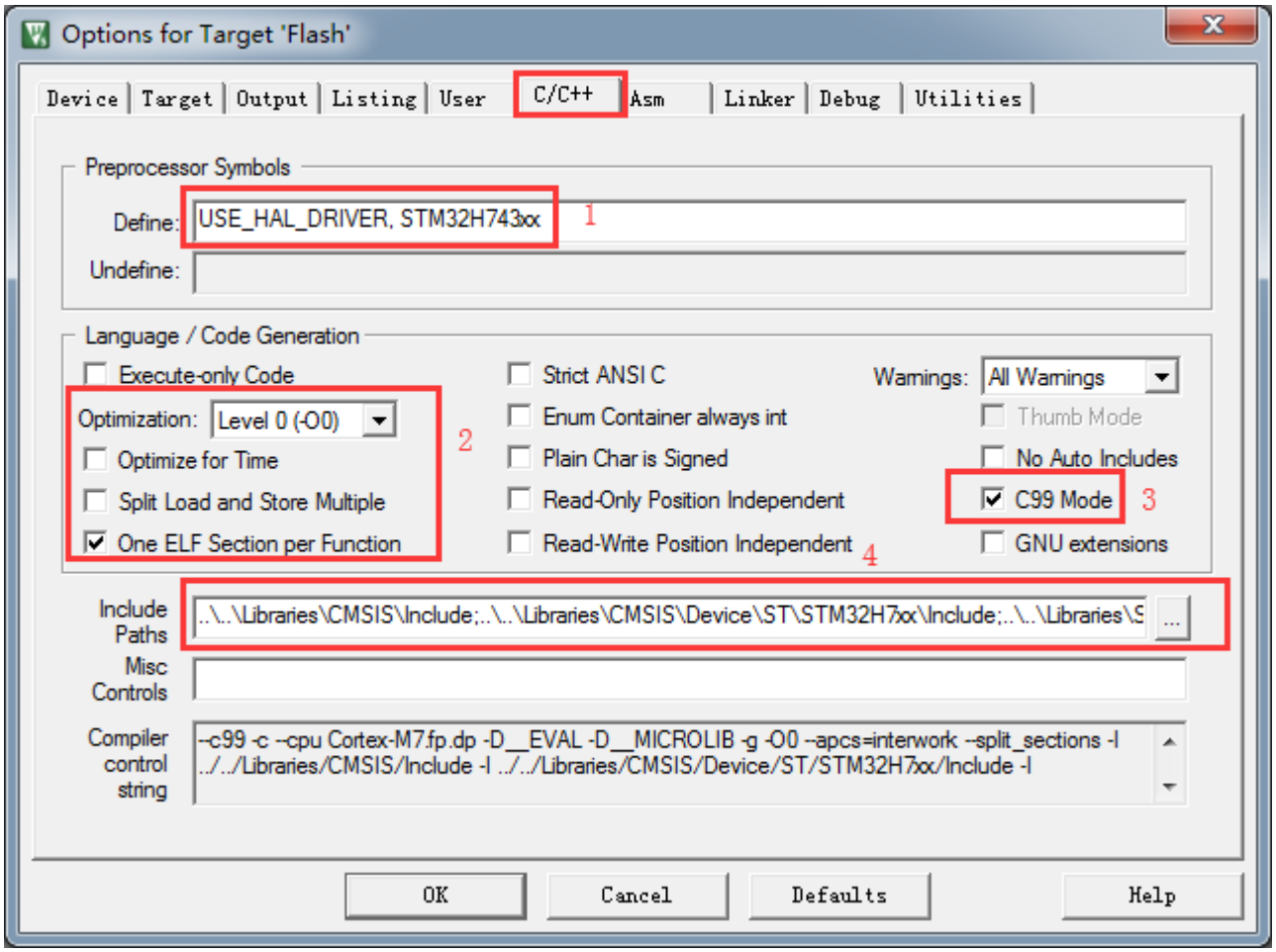
4.6.5 User 选项



这个选项基本用不上，可以不用管，主要是用来指定编译之前和编译之后执行的用户程序。对于本教程配套的例子来说在这里添加了一个批处理功能，用来实现 Objects 文件夹里面的 hex 文件复制到路径 Project\MDK-ARM(uV5)里面，方便查找。



4.6.6 C/C++选项



1. 预定义宏

   USE_HAL_DRIVER 表示使用 HAL 库。

   STM32H743xx 表示使用 H743 系列。

2. 选择 0 级优化，优化等级低，方便调试。

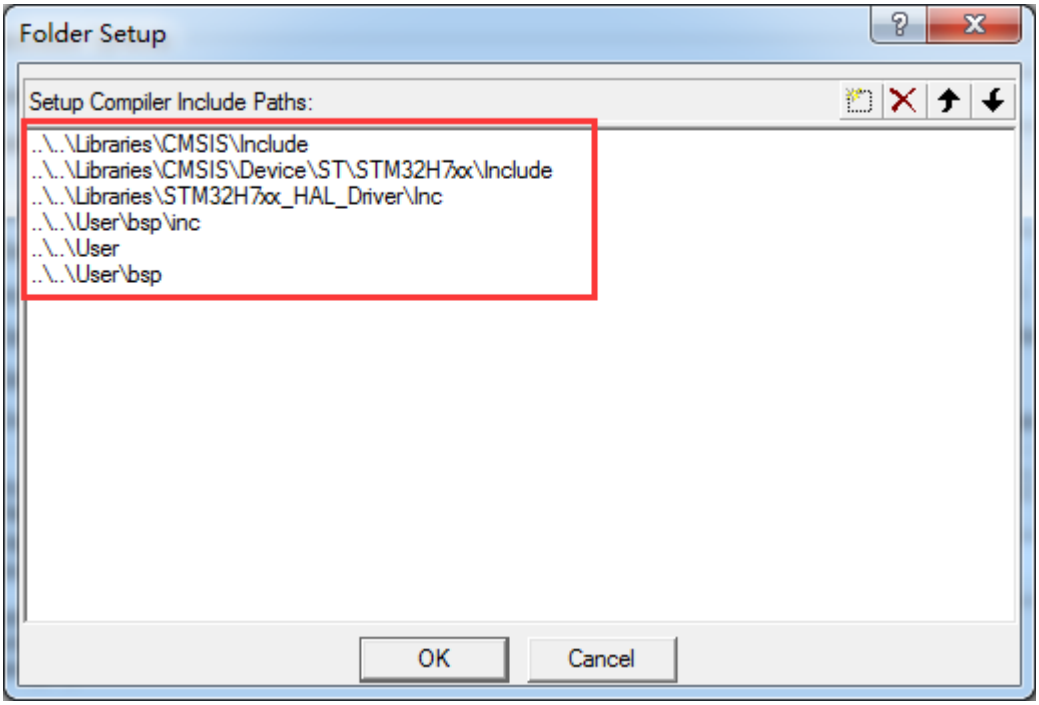
   - 如何做 MDK 编译器的代码最小优化和性能最佳优化

   MDK 之前做的应用笔记：http://www.armbbs.cn/forum.php?mod=viewthread&tid=1794 。

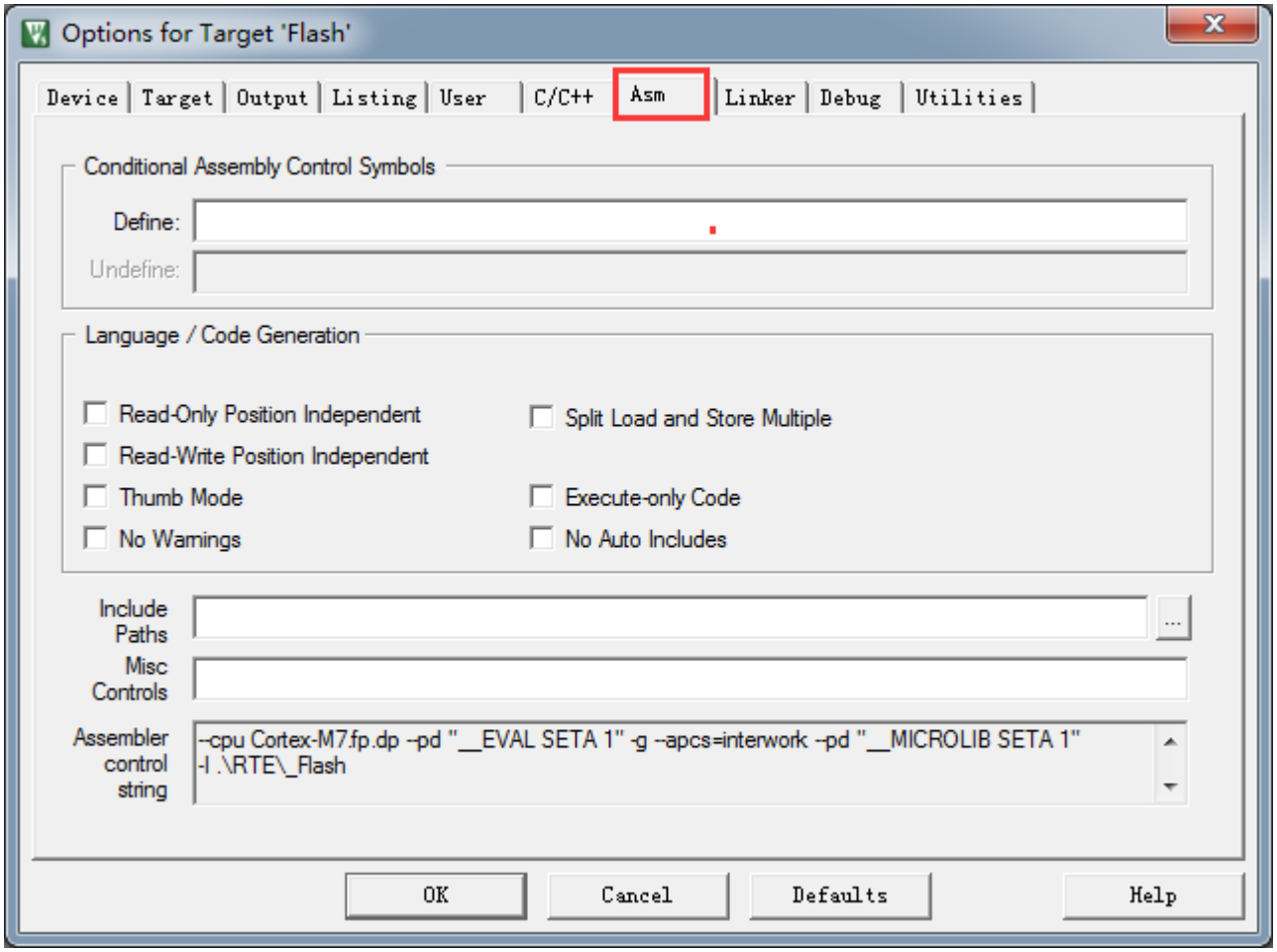
   选项 One ELF Section per Function 用于删除工程中没有用到的冗余函数。

3. 使用 C99 规范，在第 9 章有介绍。

4. 添加路径，把用到的头文件路径包含进来，点击最右侧的小按钮，弹出如下界面进行添加即可：

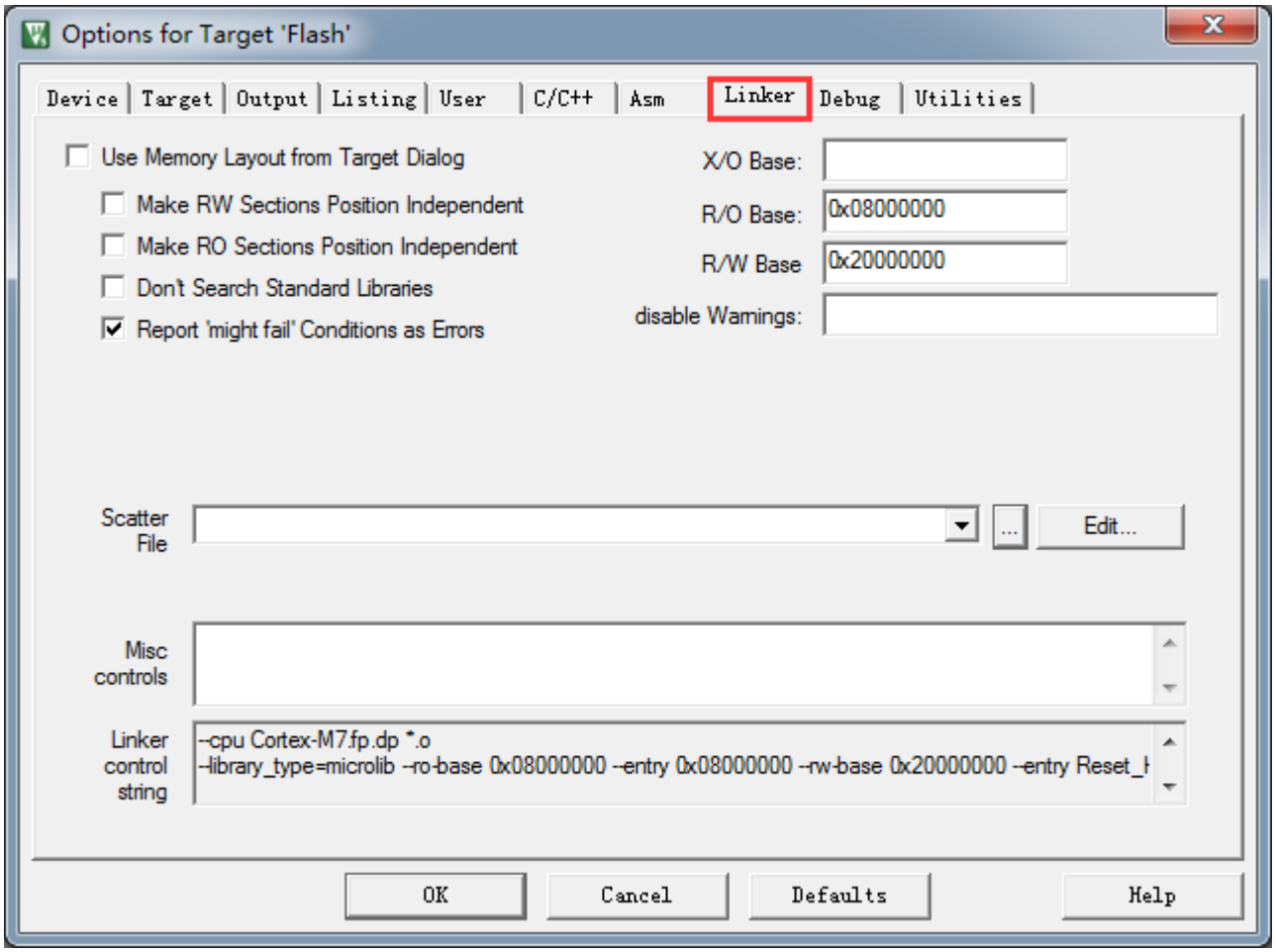

Folder Setup
Setup Compiler Include Paths:
..\..\Libraries\CMSIS\Include
..\..\Libraries\CMSIS\Device\ST\STM32H7xx\Include
..\..\Libraries\STM32H7xx_HAL_Driver\Inc
..\..\User\bsp\inc
..\..\User
..\..\User\bsp
OK
Cancel

4.6.7 Asm 选项


Options for Target 'Flash'
Device
Target
Output
Listing
User
C/C++
Asm
Linker
Debug
Utilities
Conditional Assembly Control Symbols
Define:
Undefine:
Language / Code Generation
Read-Only Position Independent
Split Load and Store Multiple
Read-Write Position Independent
Thumb Mode
Execute-only Code
No Warnings
No Auto Includes
Include Paths
Misc Controls
Assembler control string
--cpu Cortex-M7.fp.dp -pd "__EVAL SETA 1" -g --apcs=interwork -pd "__MICROLIB SETA 1"
-I .\RTE_Flash
OK
Cancel
Defaults
Help

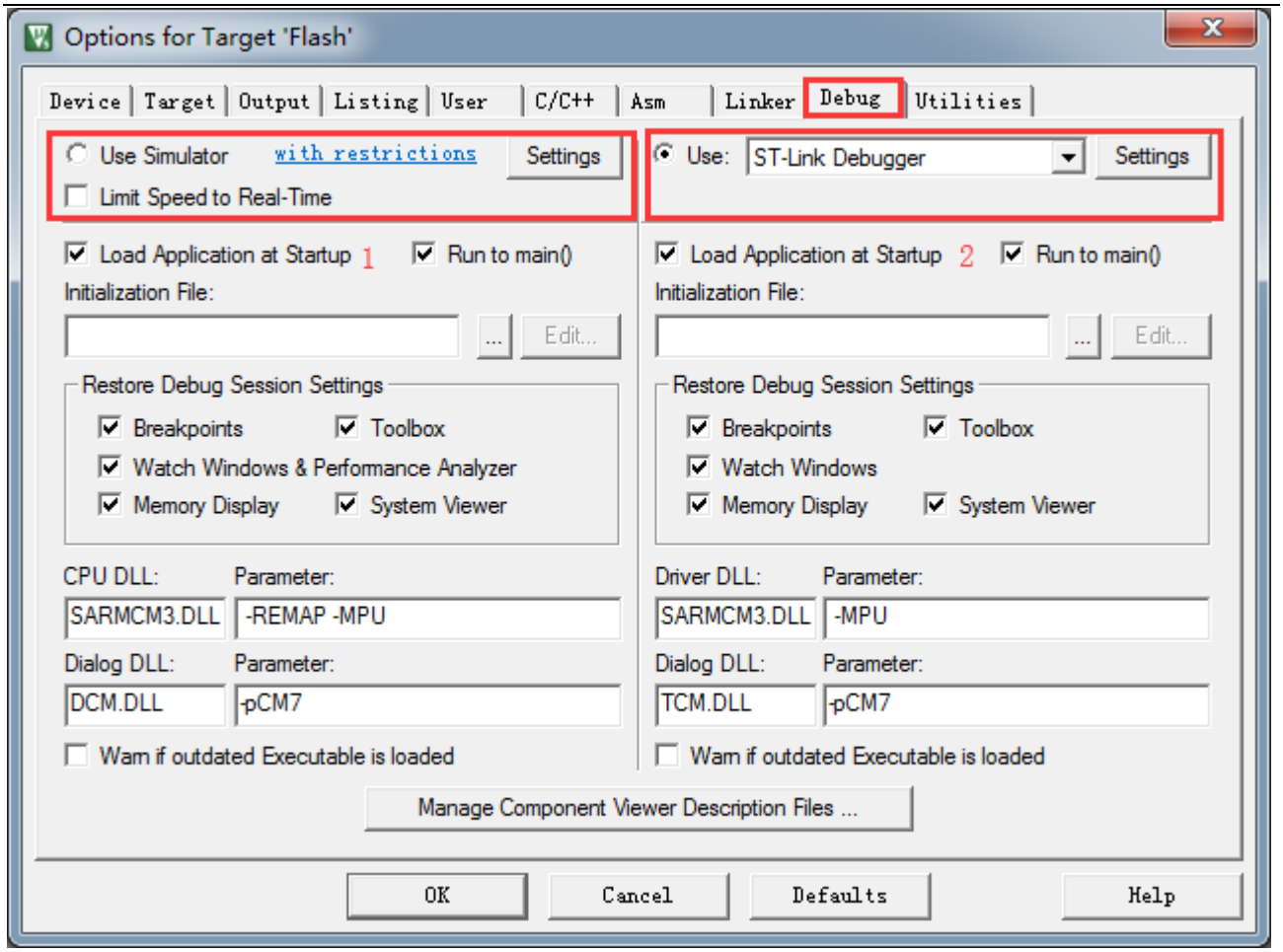
这个选项用不到，不用管。

4.6.8 Linker 选项



这个选项用不到，不用管。

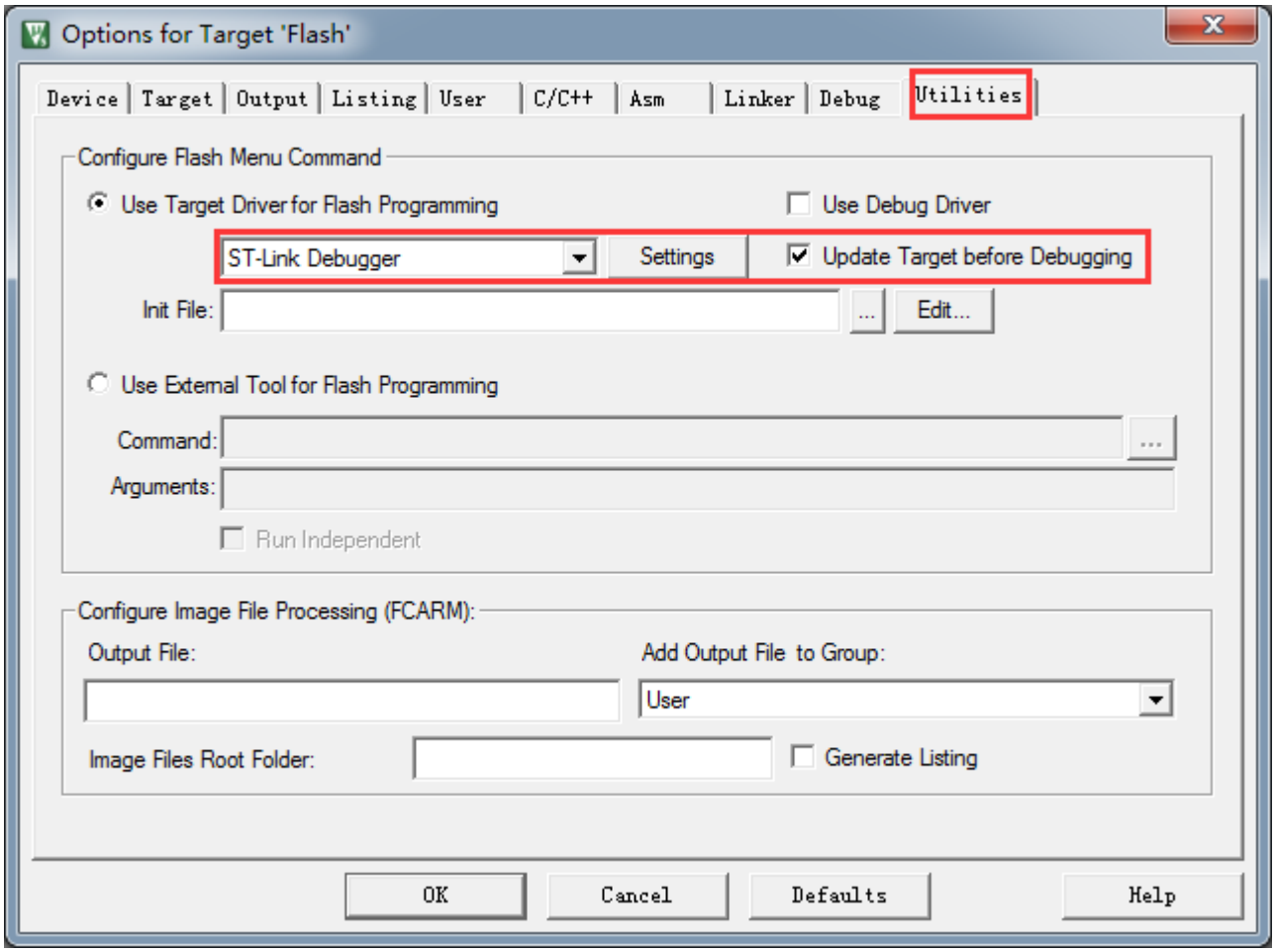
4.6.9 Debug 选项



1. 软件仿真不推荐大家用，问题比较多。
2. 硬件仿真，这里选择的 STLINK，大家根据自己用的那种调试器选择相应型号即可。

具体的调试设置在下一章有详细说明，本章主要是保证创建的工程能够正常编译。

4.6.10 Utilities 选项

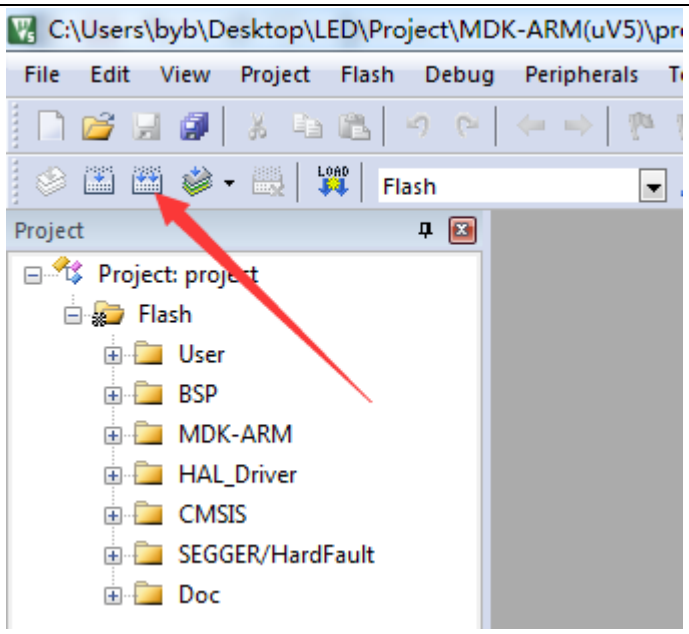


这个选项主要用于编程工具选择，大家根据自己用的那种下载器选择相应型号即可。

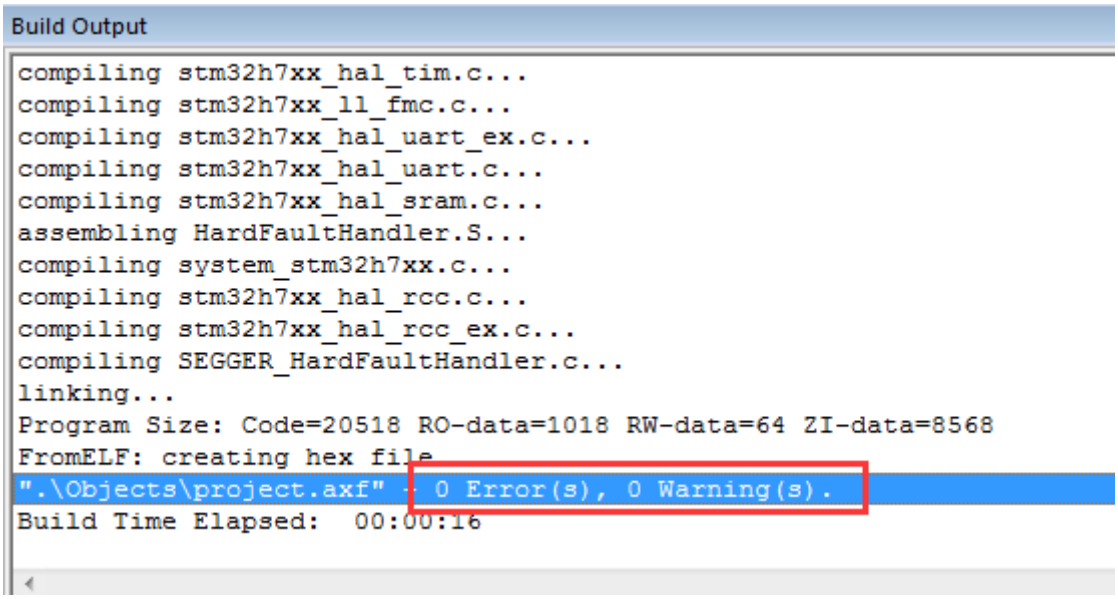
Update Target before Debugging 表示调试前更新芯片的程序。

4.7 第 6 步，最后一步验证是否创建成功

点击全编译：



编译结果：0 错误，0 警告，编译成功。



4.8 MDK 的字体，多核并行编译，语法提示等设置

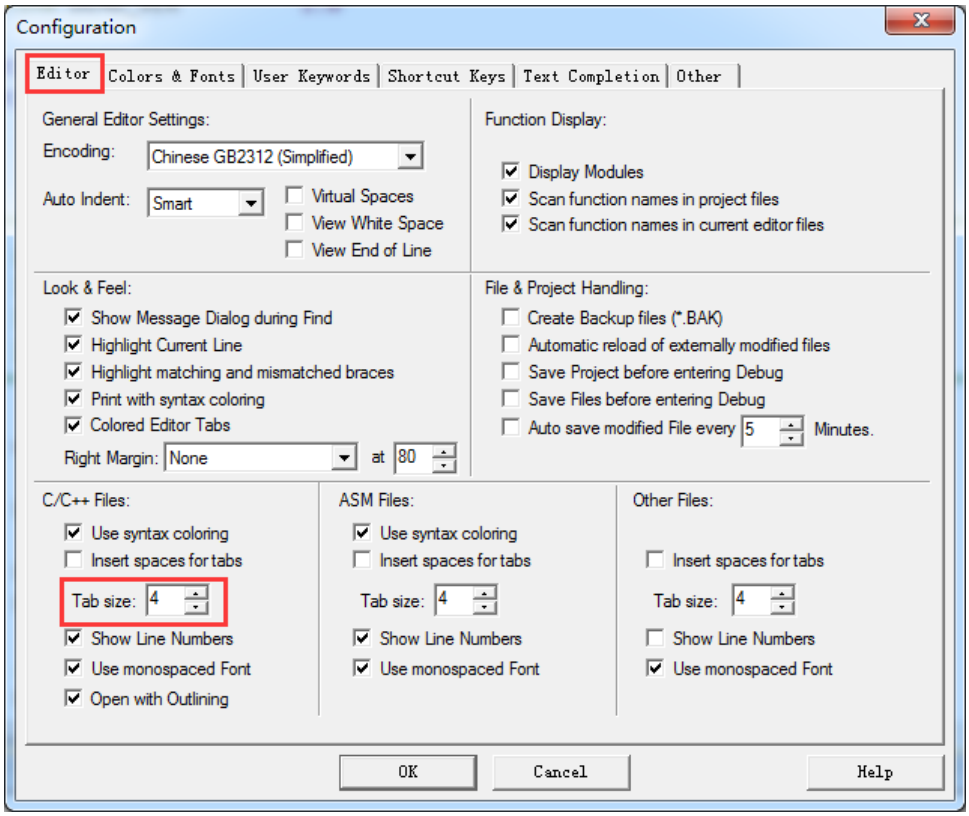
这里将 MDK 经常用到的设置做个说明。

4.8.1 Tab Size 设置

教程配套例子的 Tab size 要设置为 4，否则代码显示不整齐：

选择此图标：

弹出的窗口里面设置 Tab size 为 4：

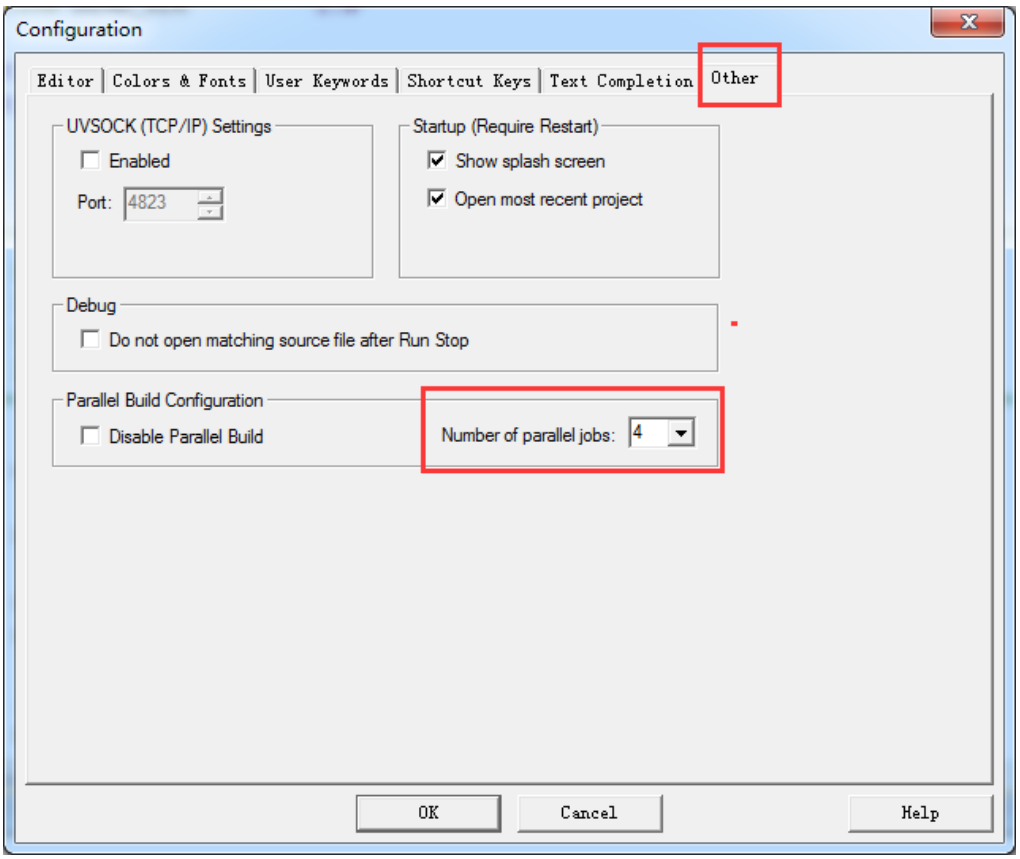


4.8.2 使能多核并行编译

电脑有几个核，都选上，加快编译速度。

选择此图标：

我这里是四核：



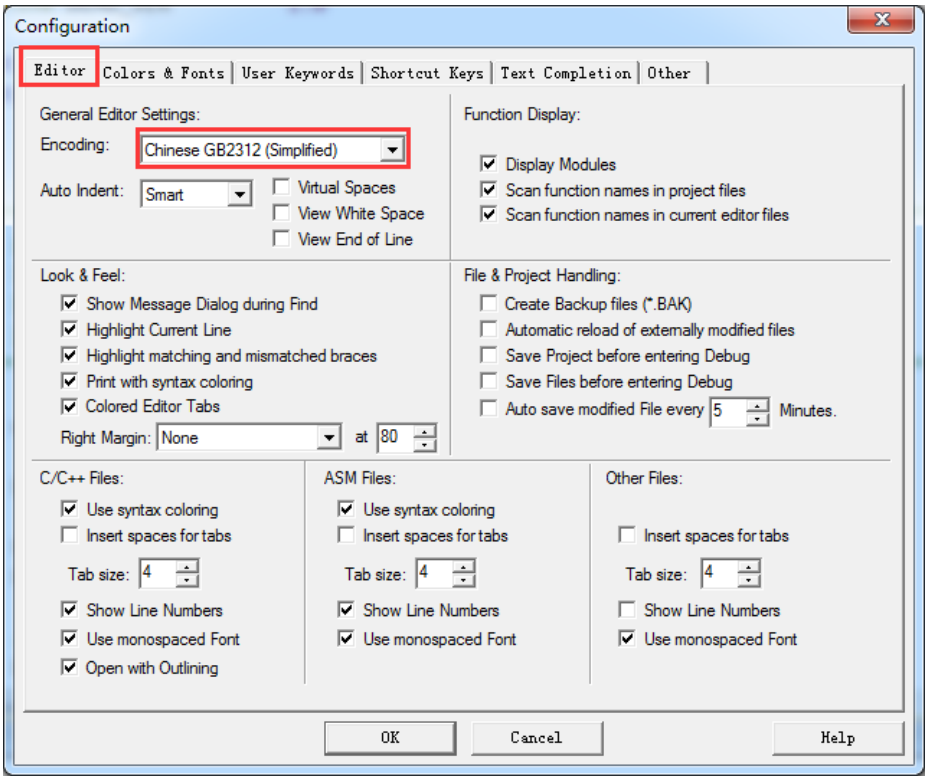
4.8.3 字体设置

推荐大家用下微软雅黑，而且可以防止复制粘贴的时候出现乱码问题，字体下载在此贴：http://www.armbbs.cn/forum.php?mod=viewthread&tid=2293 。

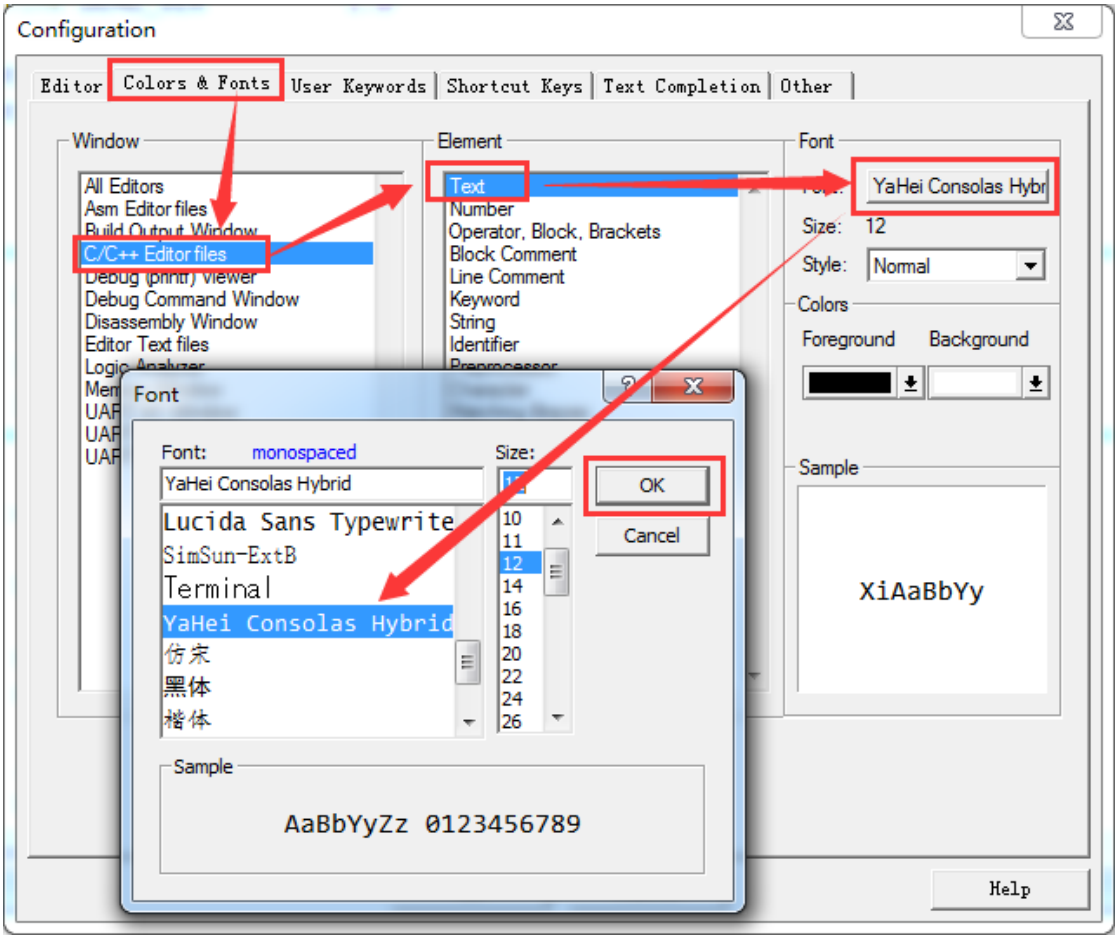
安装微软雅黑的时候要注意，电脑端默认自带了一个微软雅黑，安装的时候会出现字体是躺着的。

选择此图标：

选择中文简体：


Configuration
Editor
Colors & Fonts
User Keywords
Shortcut Keys
Text Completion
Other
General Editor Settings:
Encoding:
Chinese GB2312 (Simplified)
Auto Indent:
Smart
Virtual Spaces
View White Space
View End of Line
Function Display:
Display Modules
Scan function names in project files
Scan function names in current editor files
Look & Feel:
Show Message Dialog during Find
Highlight Current Line
Highlight matching and mismatched braces
Print with syntax coloring
Colored Editor Tabs
Right Margin:
None
at
80
File & Project Handling:
Create Backup files (*.BAK)
Automatic reload of externally modified files
Save Project before entering Debug
Save Files before entering Debug
Auto save modified File every
5
Minutes.
C/C++ Files:
Use syntax coloring
Insert spaces for tabs
Tab size:
4
Show Line Numbers
Use monospaced Font
Open with Outlining
ASM Files:
Use syntax coloring
Insert spaces for tabs
Tab size:
4
Show Line Numbers
Use monospaced Font
Other Files:
Insert spaces for tabs
Tab size:
4
Show Line Numbers
Use monospaced Font
OK
Cancel
Help

选择安装微软雅黑字体：


Configuration
Editor
Colors & Fonts
User Keywords
Shortcut Keys
Text Completion
Other
Window
All Editors
Asm Editor files
C/C++ Editor files
Debug Command Window
Disassembly Window
Editor Text files
Element
Text
Number
Operator, Block, Brackets
Block Comment
Line Comment
Keyword
String
Identifier
Font
YaHei Consolas Hybr
Size:
12
Style:
Normal
Colors
Foreground
Background
Sample
XiAaBbYy
Font
Font:
monospaced
YaHei Consolas Hybrid
Lucida Sans Typewrite
SimSun-ExtB
Terminal
YaHei Consolas Hybrid
仿宋
黑体
楷体
Size:
10
11
12
14
16
18
20
22
24
26
OK
Cancel
Sample
AaBbYyZz 0123456789
Help

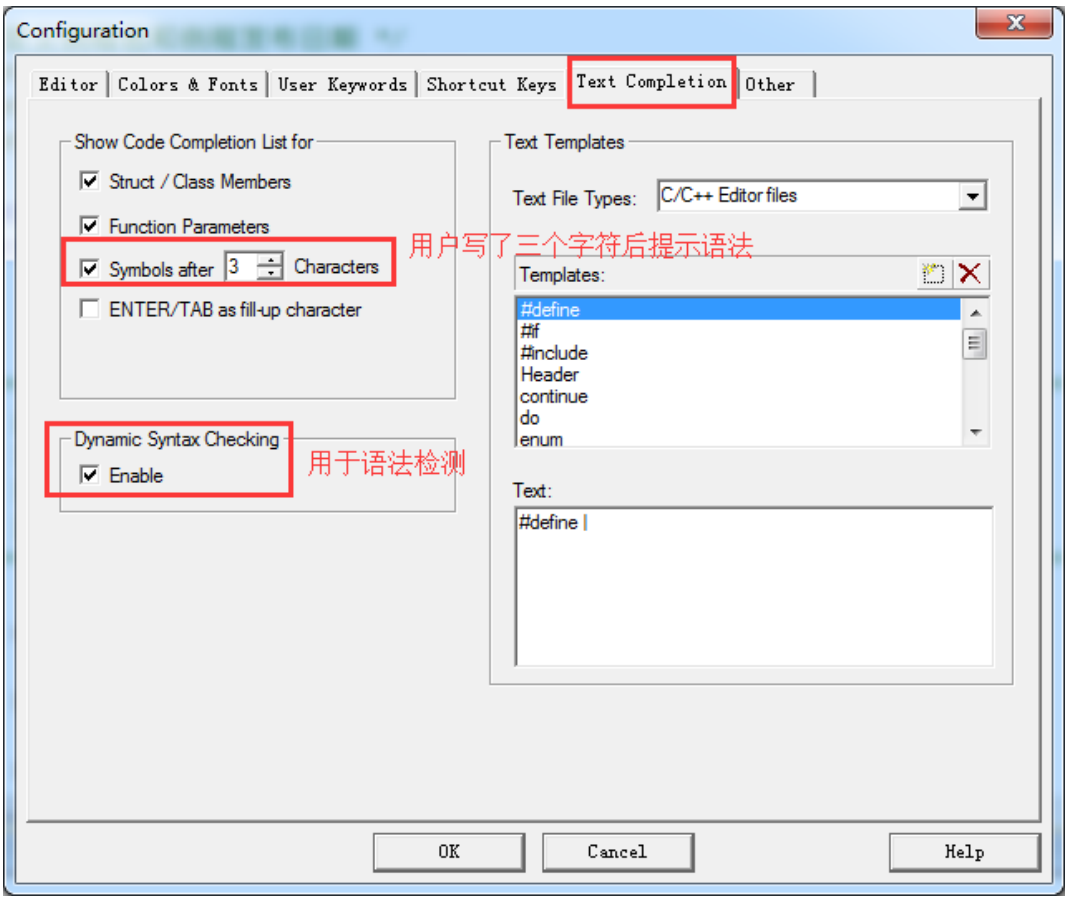
效果如下：

```
30  /* 定义例程名和例程发布日期 */
31  #define EXAMPLE_NAME    "V7-跑马灯"
32  #define EXAMPLE_DATE    "2018-12-12"
33  #define DEMO_VER        "1.0"
34
35  static void PrintfLogo(void);
36  static void PrintfHelp(void);
37
38
39  /*
40  *********************************************************************************************************
41  *   函 数 名: main
42  *   功能说明: c程序入口
43  *   形    参: 无
44  *   返 回 值: 错误代码(无需处理)
45  *********************************************************************************************************
46  */
47  int main(void)
48  {
49      bsp_Init();     /* 硬件初始化 */
50
51      PrintfLogo();   /* 打印例程名称和版本等信息 */
```

4.8.4 语法提示和语法检测功能

选择此图标：

MDK 的语法提示和语法检测功能还行，大家可以根据需要开启：



4.9 总结

对于初学者来说，务必要实际动手操作一遍，以后移植其它程序代码大有裨益。

第5章 STM32H7 下载和调试方法（MDK5）

本章教程为大家介绍如何通过 MDK5 下载程序到芯片里面以及 MDK5 的调试和配置方法。

5.1 初学者重要提示

5.2 使用 MDK 调试和下载程序设置（JLINK）

5.3 使用 MDK 调试和下载程序设置（STLINK）

5.4 MDK 调试容易崩溃的解决办法

5.5 MDK 程序调试

5.6 总结

5.1 初学者重要提示

- 如果使用 JLINK 调试下载 STM32H7，务必使用 JLINK V9 或 V10，之前的版本已经不支持。
- 如果使用 STLINK 调试下载 STM32H7，推荐使用最新的电脑端驱动和对应的固件，详情见第 2 章的 2.6 小节。
- MDK 曾发布的 STM32F1，F407，F429 和 F7 的所有调试方法的设置细节及其注意事项，同样适用于 STM32H7：http://www.armbbs.cn/forum.php?mod=viewthread&tid=14896 。
- MDK5 调试过程中容易崩溃的问题，请看本章 5.4 小节，比较重要。
- JLINK 无法下载解决思路以及常见问题整理，适用于其它 LINK：http://www.armbbs.cn/forum.php?mod=viewthread&tid=21708 。

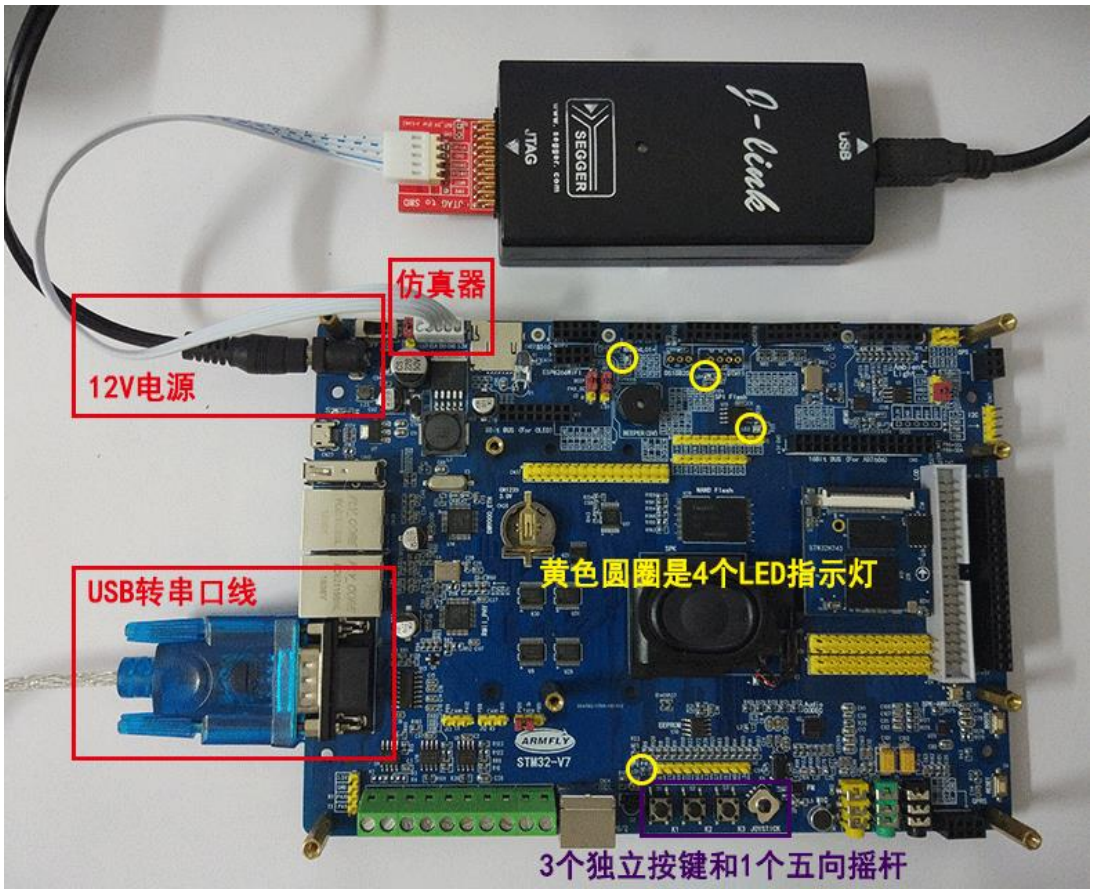
5.2 使用 MDK 调试和下载程序设置（JLINK）

调试下载 STM32H7，务必使用 JLINK V9 或 V10，之前的版本已经不支持，JLINK 的驱动安装等相关文件已经在本教程第 2 章的 2.5 章节有说明。

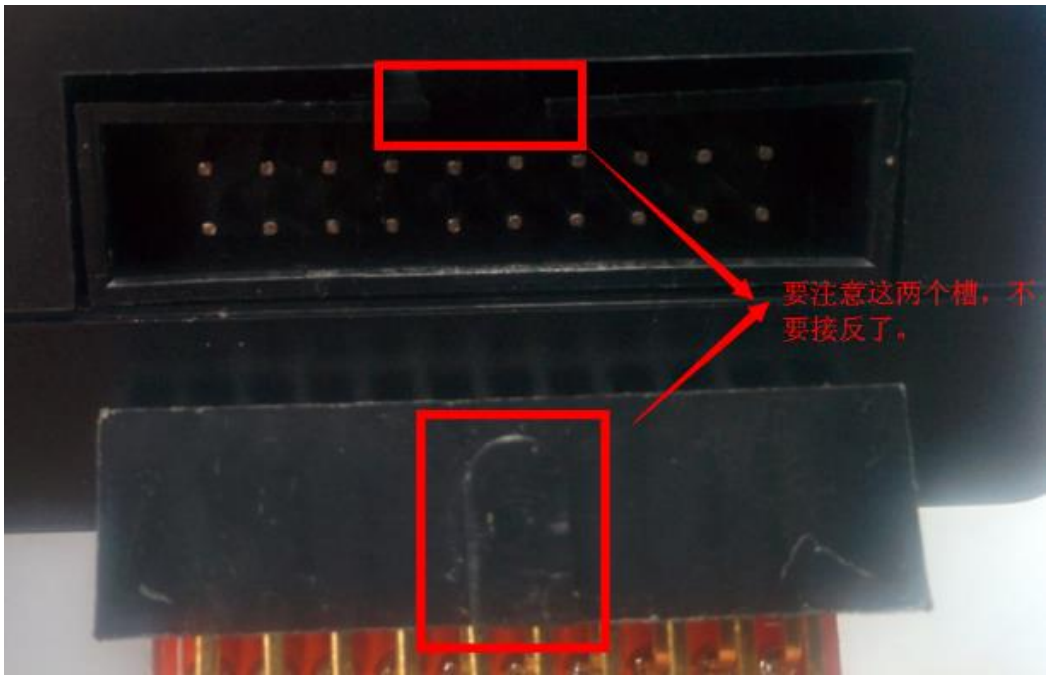
在上个章节里面，我们已经将 Options 对话框里面大部分设置选项都做了说明，本章节主要是调试下载的设置。

5.2.1 开发板和 JLINK 的接线图

开发板接上 JLINK 的效果图如下，JLINK 端需要插入开发板赠送的 JTAG 转 SWD 转接板：

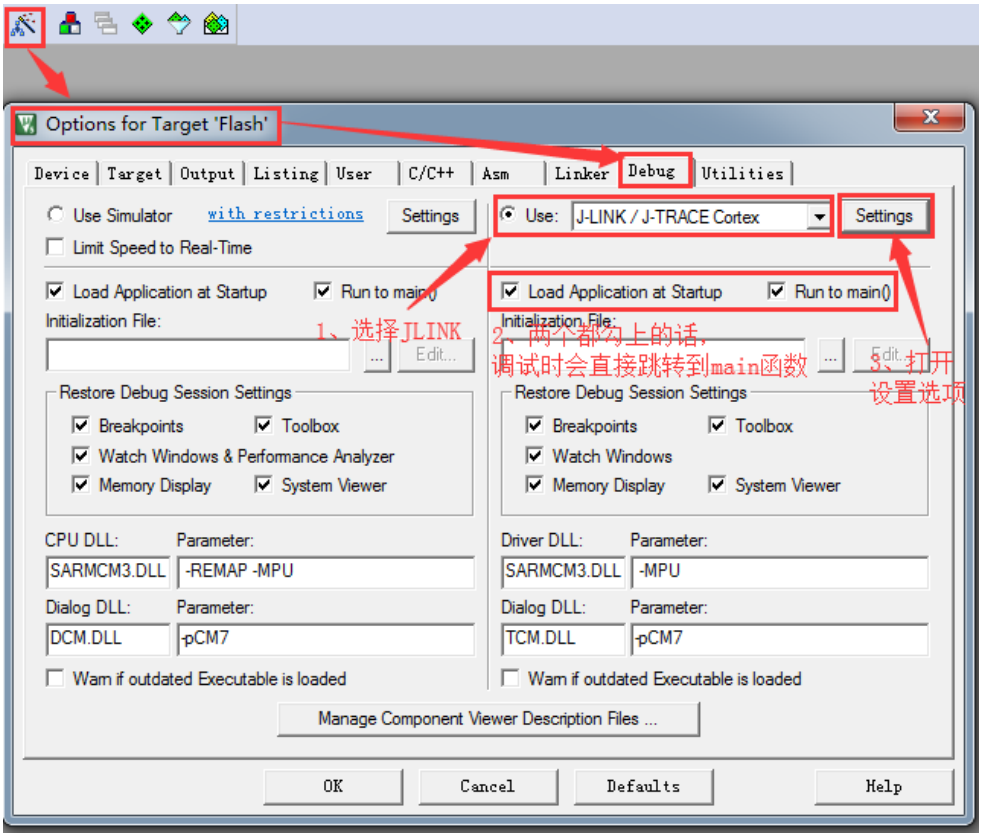

仿真器
12V电源
USB转串口线
黄色圆圈是4个LED指示灯
3个独立按键和1个五向摇杆

注意转接板不要插反了：

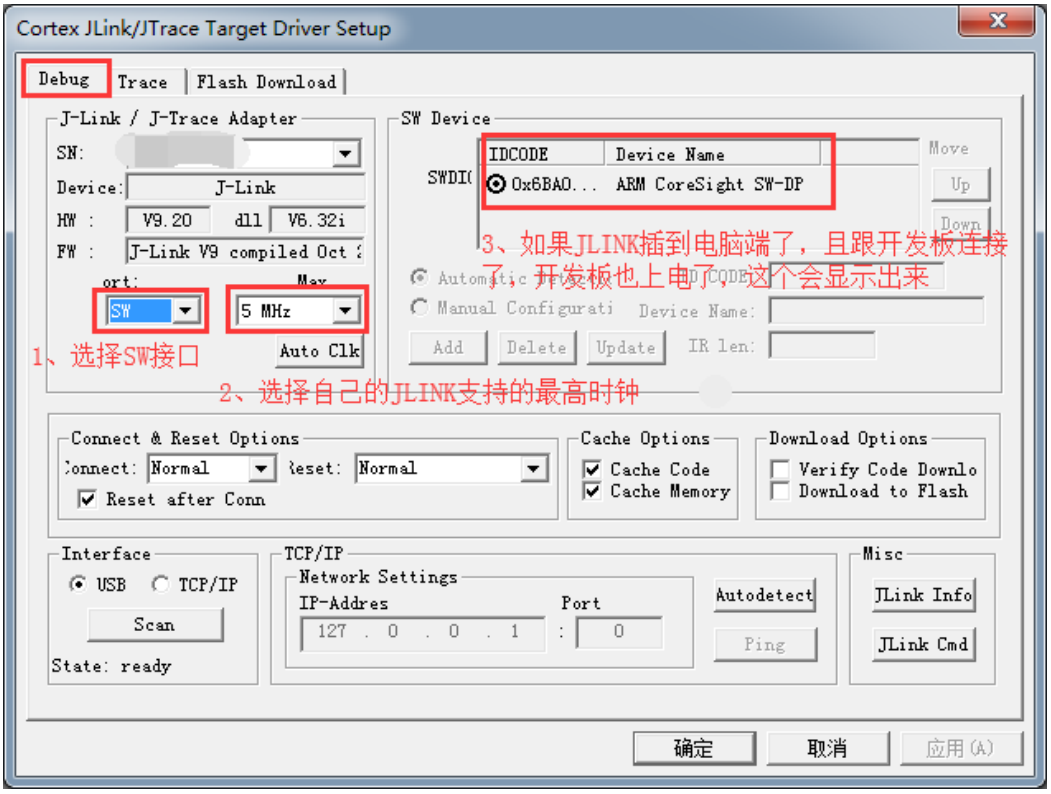

要注意这两个槽，不要接反了。

5.2.2 设置 Options 里面的 Debug 选项

Debug 选项主要用于调试设置。

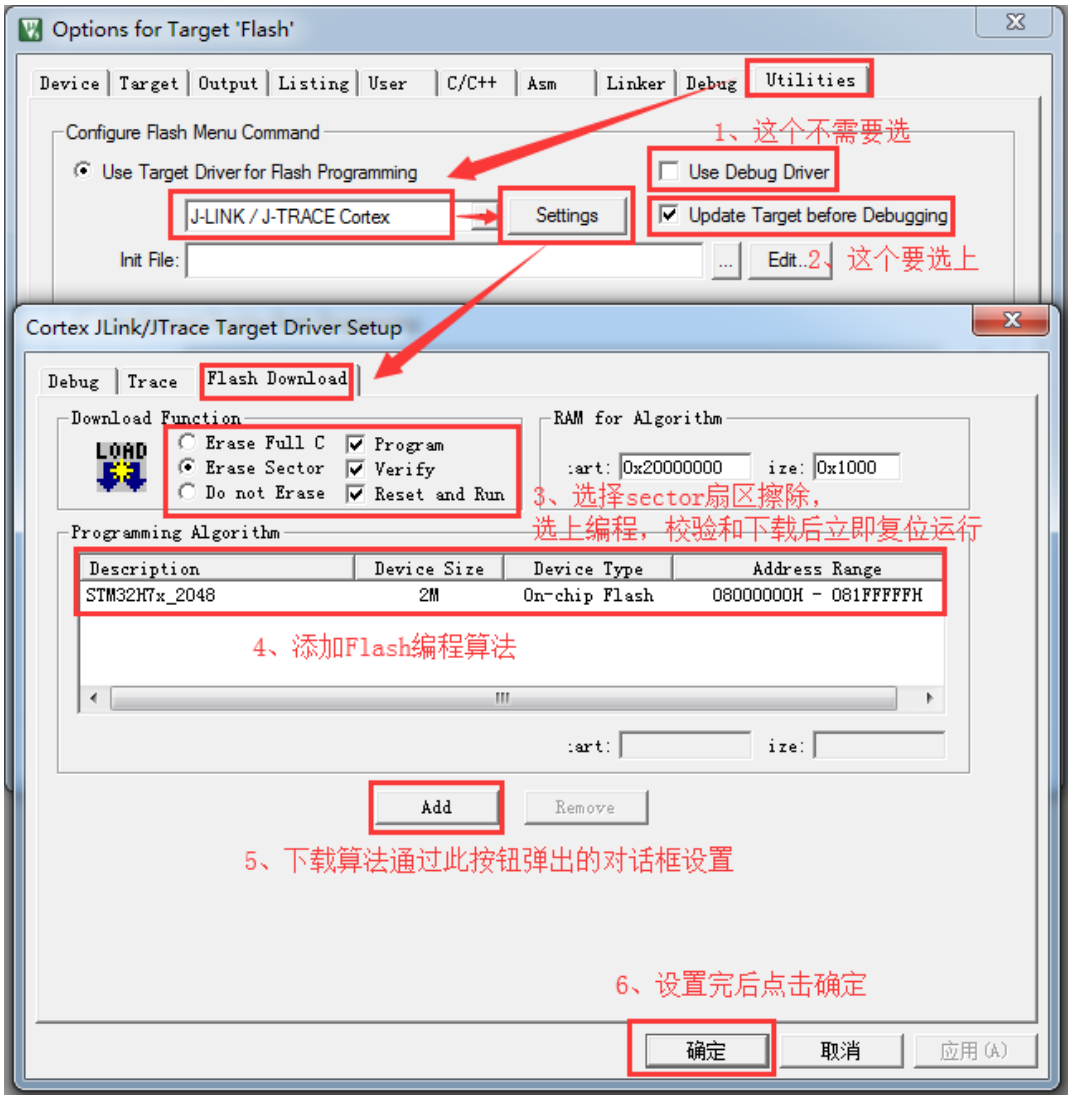


打开 Settings 设置选项，设置完毕后别忘了点击确定。



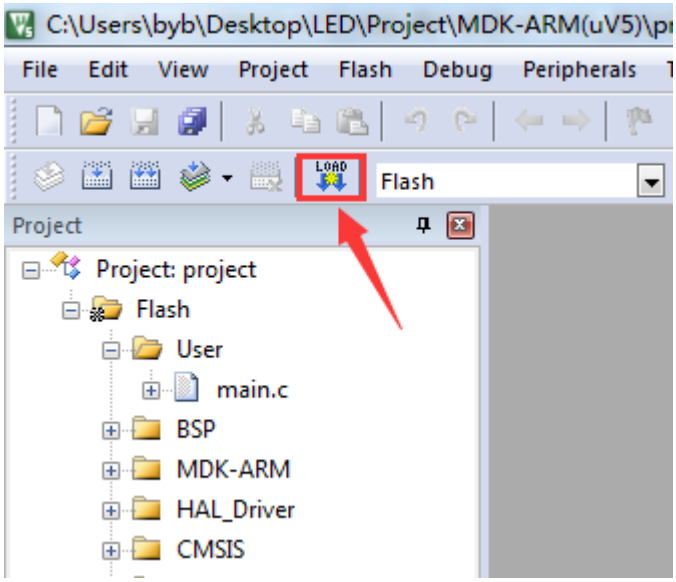
5.2.3 设置 Options 里面的 Utilities 选项

Utilities 选项主要用于程序下载设置。

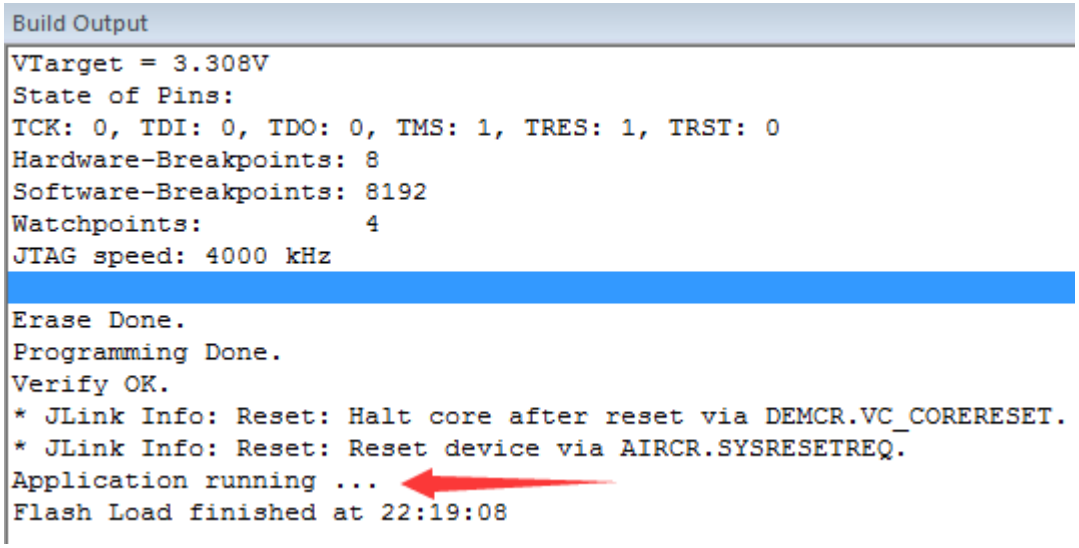


5.2.4 程序下载

前两步设置好之后，JLINK 插到电脑端并跟开发板连接好，将板子上电，就可以下载了。点击这里就可以下载了：



下载成功后可以看到开发板上的 LED 开始闪烁，MDK 的也有信息展示：



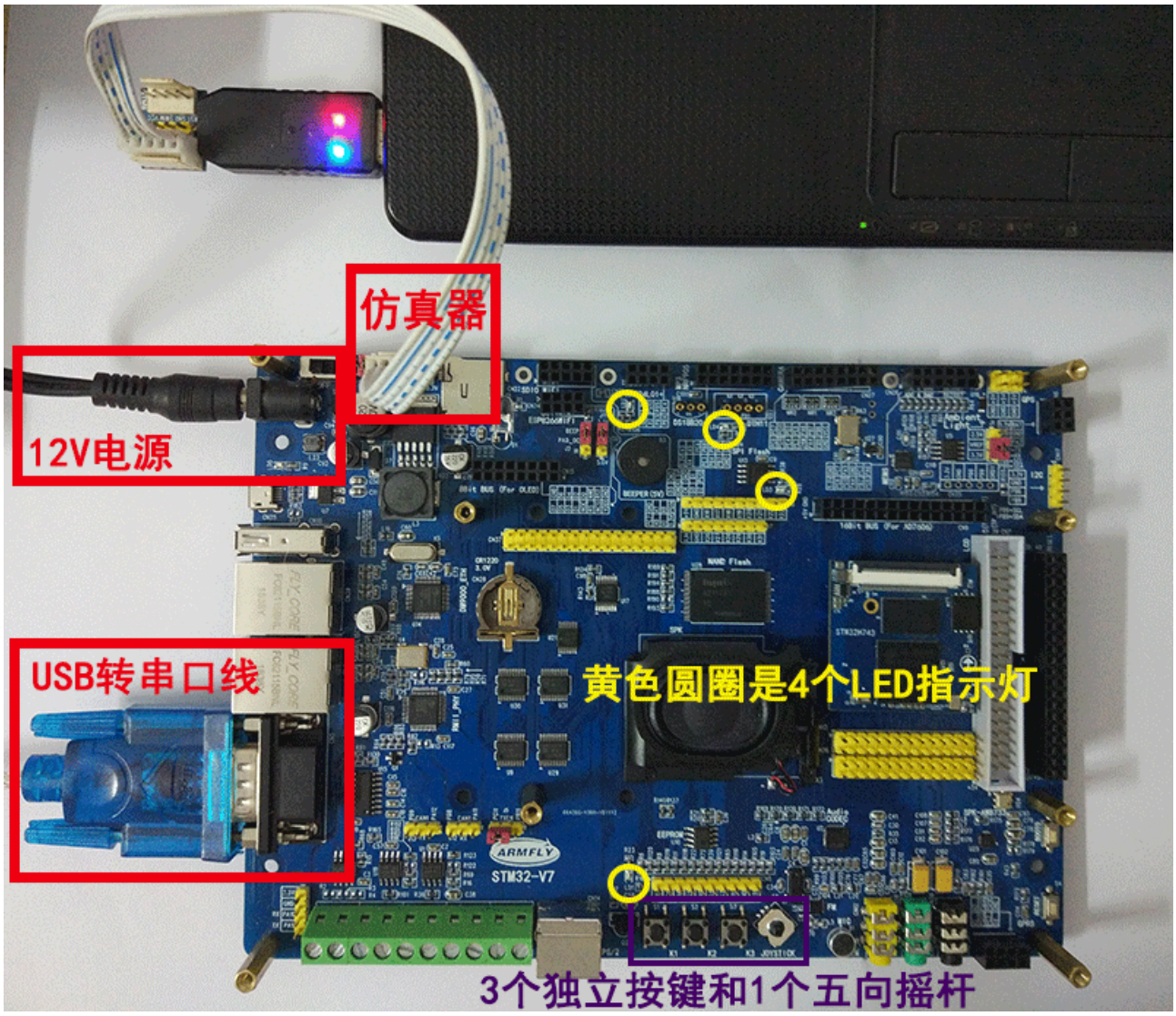
5.3 使用 MDK 调试和下载程序设置（STLINK）

调试下载 STM32H7，请将 STLINK 固件升级到最新版，关于固件升级和驱动安装等相关文件已经在本教程第 2 章的 2.6 章节有说明。

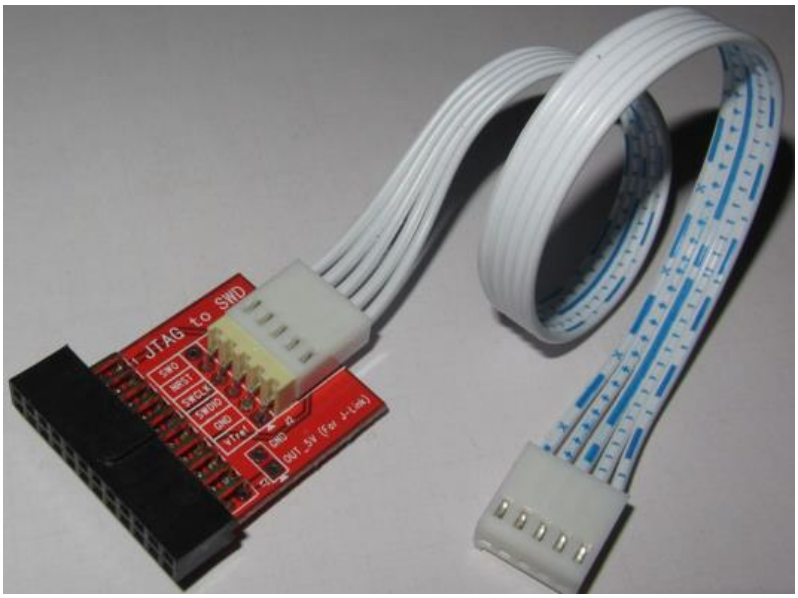
在上个章节里面，我们已经将 Options 对话框里面大部分设置选项都做了说明，本章节主要是调试下载的设置。

5.3.1 开发板和 STLINK 的接线图

开发板接上 STLINK 的效果图如下：

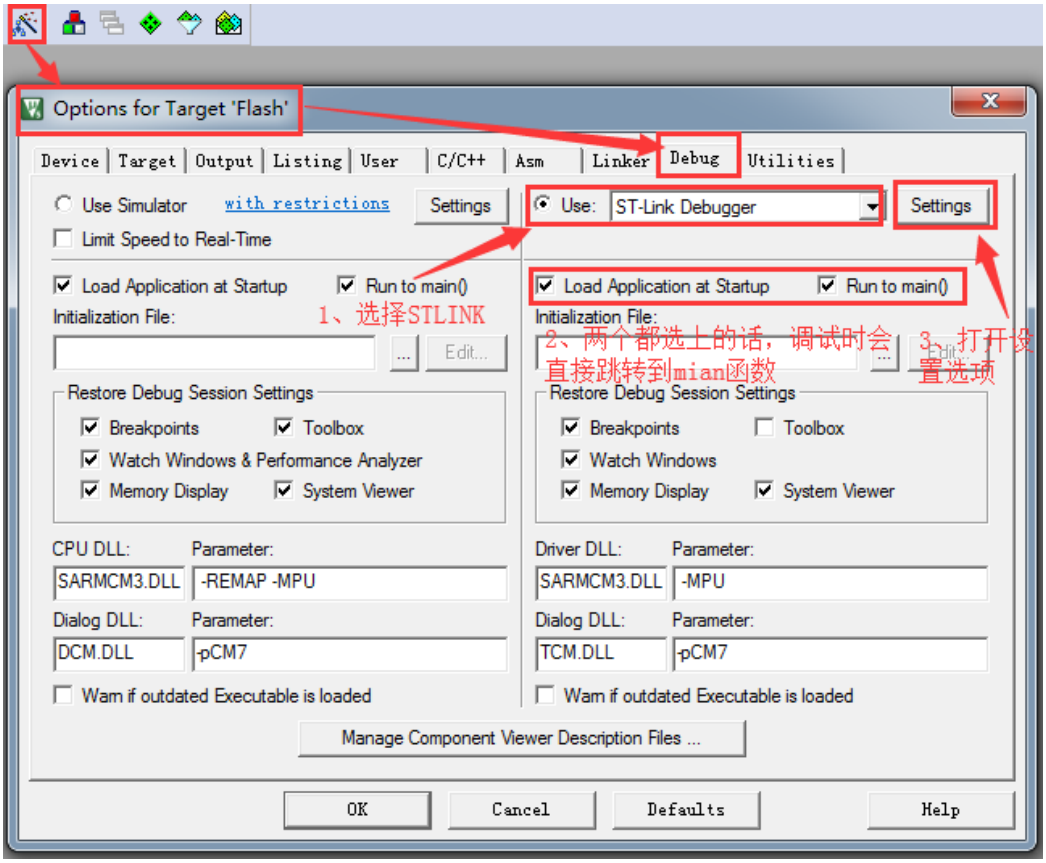


如果是下面这种 STLINK，需要接上转接板：

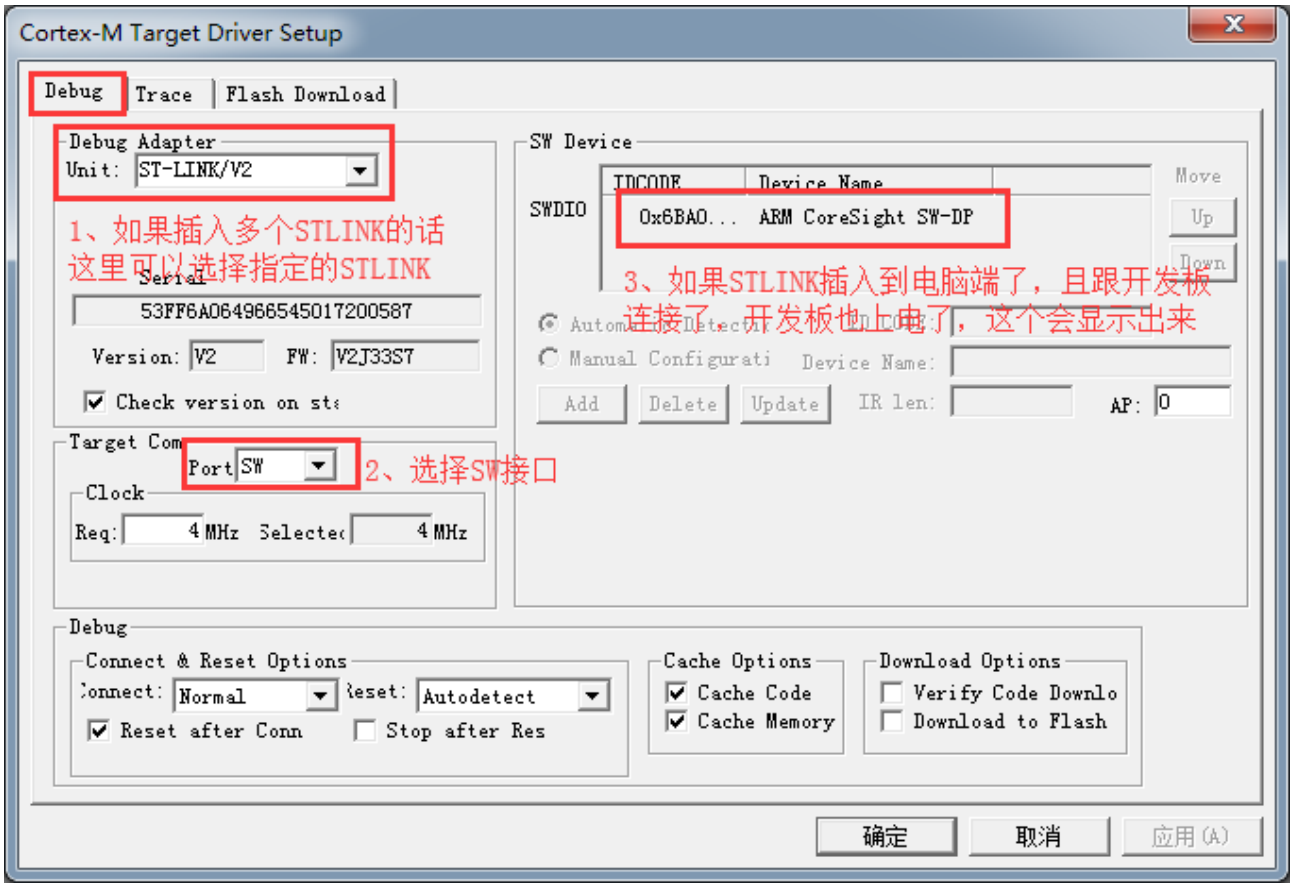


5.3.2 设置 Options 里面的 Debug 选项

Debug 选项主要用于调试设置：

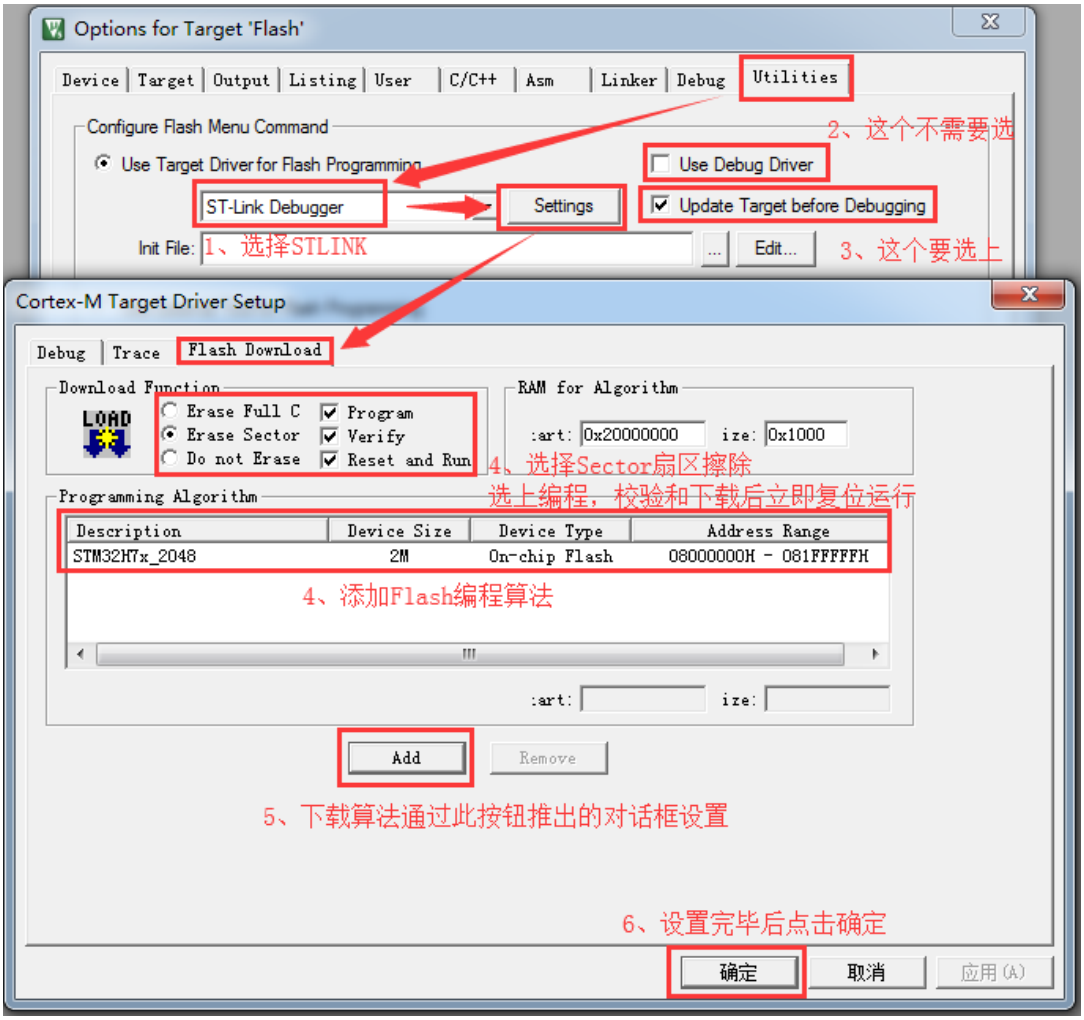


打开 Settings 设置选项，设置完毕后别忘了点击确定。



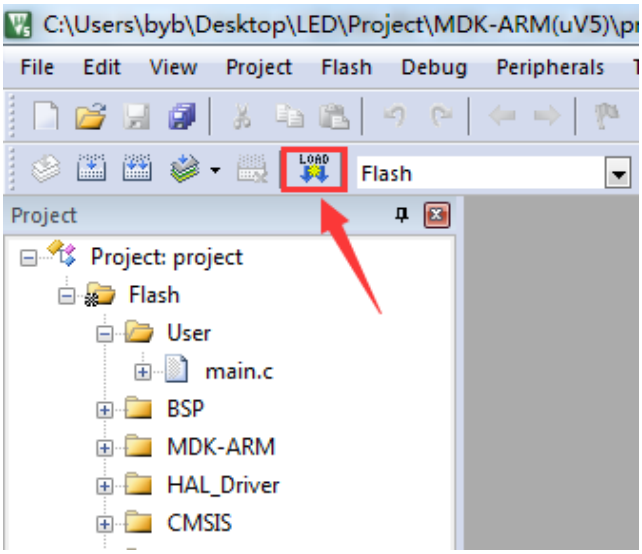
5.3.3 设置 Options 里面的 Utilities 选项

Utilities 选项主要用于程序下载设置。



5.3.4 程序下载

前两步设置好之后，JLINK 插到电脑端并跟开发板连接好，将板子上电就可以下载了。点击这里即可：

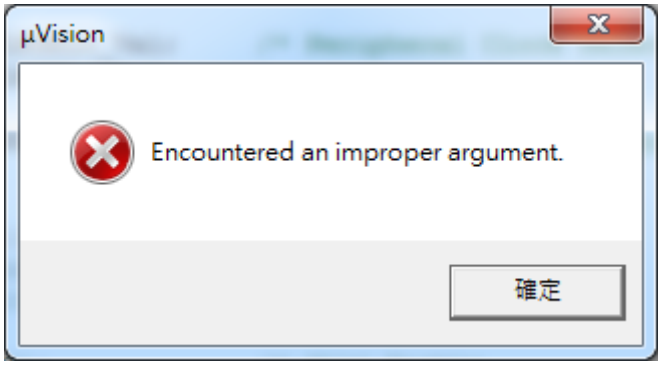


下载成功后可以看到开发板上的 LED 开始闪烁，MDK 的也有信息展示：

```
Build Output
Load "C:\\Users\\byb\\Desktop\\LED\\Project\\MDK-ARM(uV5)\\Objects\\project.axf"
Erase Done.
Programming Done.
Verify OK.
Application running ...
Flash Load finished at 23:31:53
```

5.4 MDK 调试容易崩溃的解决办法

进入到 MDK5.2X 后，MDK 调试的时候很容易崩溃，出现下面这个错误



出现这个错误的话，要注意下面两个问题：

- 如果出现这种情况的话，将工程放在桌面路径下，工程名不要有中文会好很多。路径深或者有中文，都会造成这种情况。
- 退出调试的时候，如果使用断点了，务必清除所有断点，再退出，否则也会造成这种情况。

针对这个问题，KEIL 官方也有说明：http://www.keil.com/support/docs/4036.htm 。

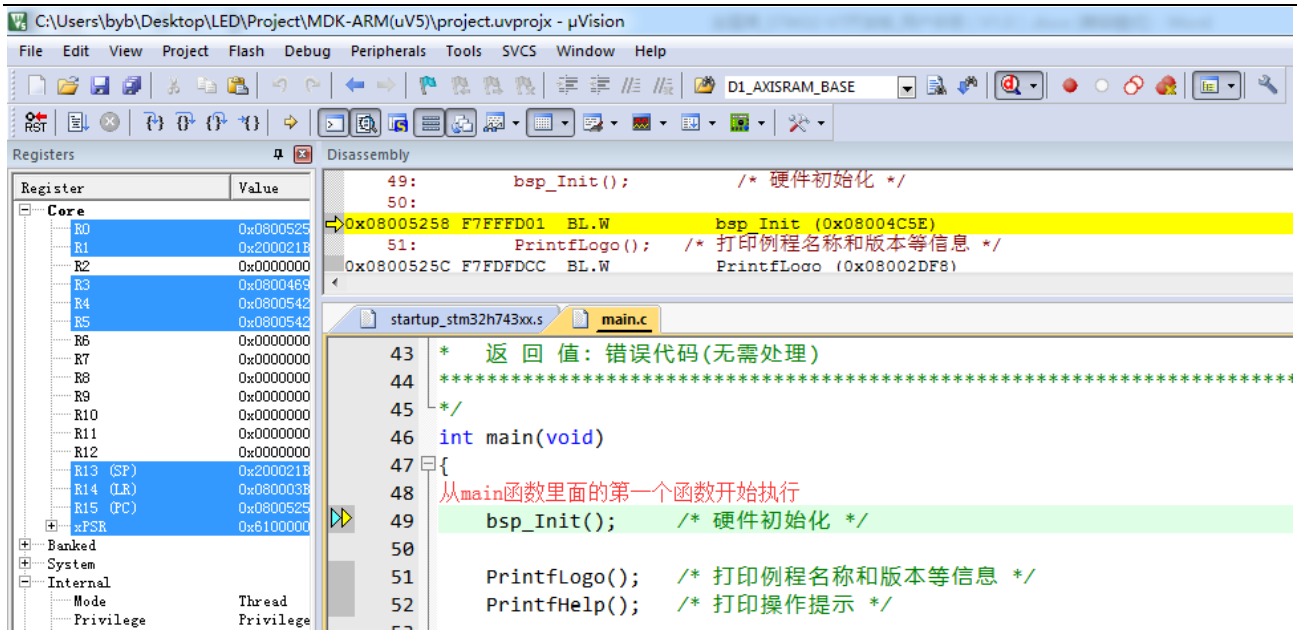
5.5 MDK 程序调试

JLINK 或者 STLINK 设置完毕后，调试是一样的。

5.5.1 全速运行和单步调试

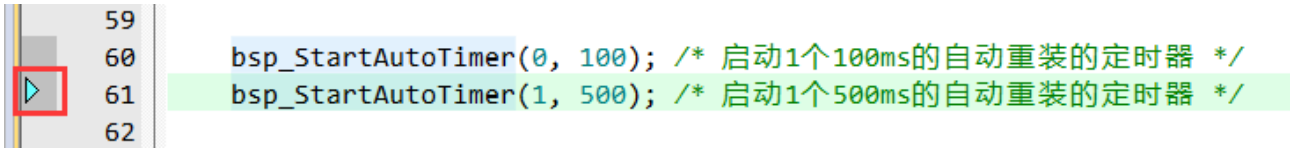
点击如下按钮可以进入调试状态：

进入调试状态后的效果如下：

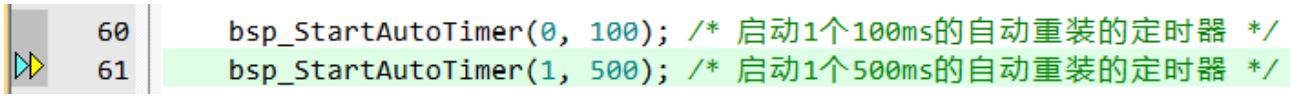


首先要认识下这 7 个图标的含义：

- 第 1 个图标是复位调试。
- 第 2 个图标是全速运行。
- 第 3 个图标是进入全速运行状态后，通过这个图标可以停止全速运行。
- 第 4 个图标是 Step（Step one line），单步调试，每次点击运行一行代码，所以只要有子函数就会进入。
- 第 5 个图标是 Step Over（Step over the current line），每次点击以函数为单位运行，不会进入函数内部。
- 第 6 个图标是 Step out（Step out of the current line），点击后退出当前运行的函数，执行下一个函数。
- 第 7 个图标是 Run to Cursor Line（Run to the current cursor line），直接运行到用户设置的行，设置方法比较简单，用户仅需在那行使用鼠标左键点击一下，可以看到前面有个箭头



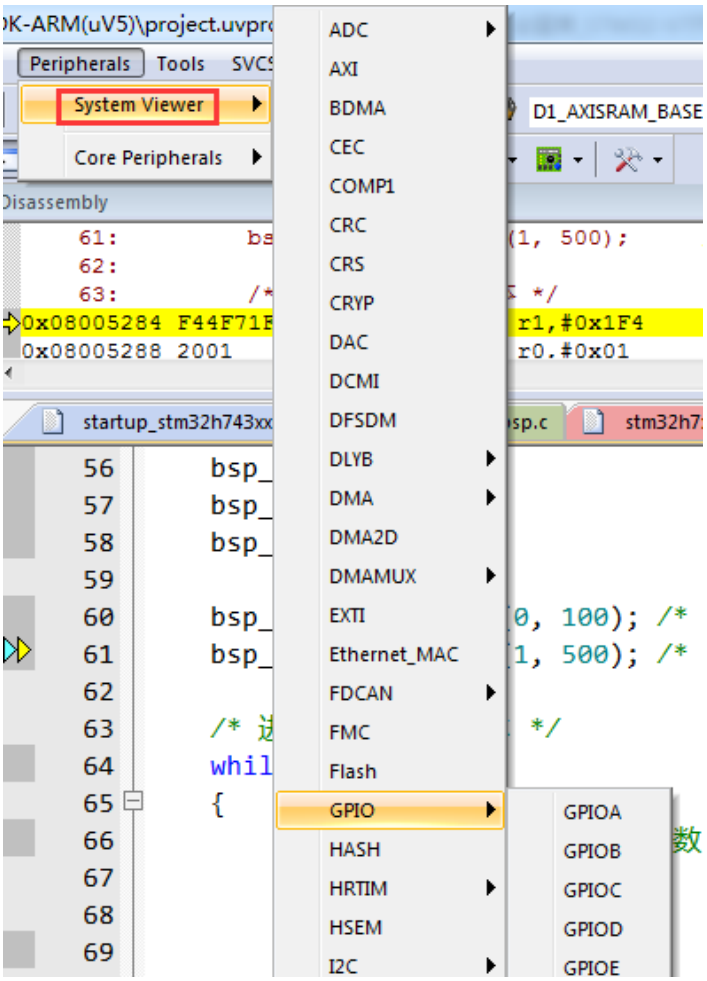
然后点击此图标即可，可以看到前面多了一个黄色的小箭头，说明已经运行到此行



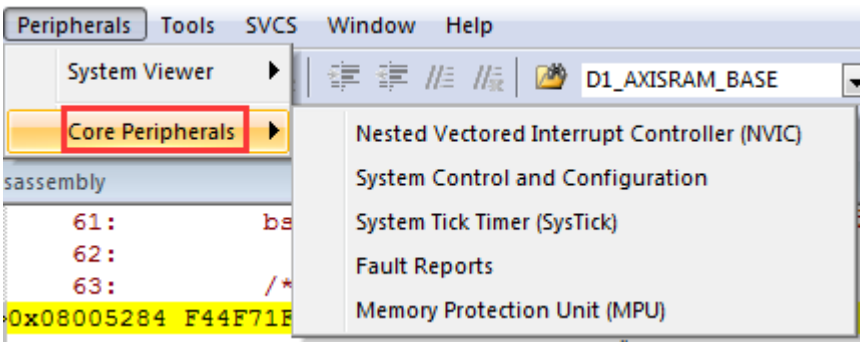
前面几个图标的功能，大家多测试几次就熟练了。

5.5.2 外设寄存器的查看

要查看的外设寄存器内容在这个菜单里面：



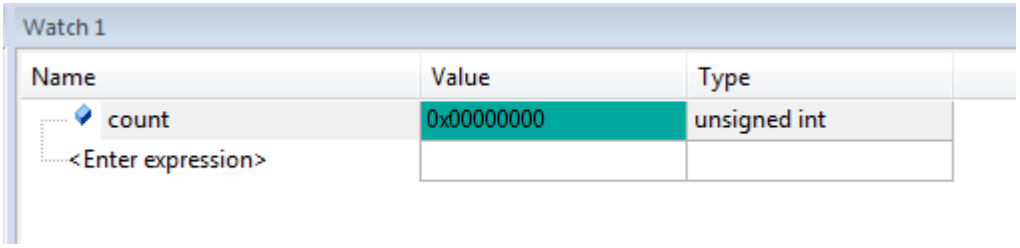
如果要查看内核外设寄存器，在这里：



5.5.3 全局变量的查看

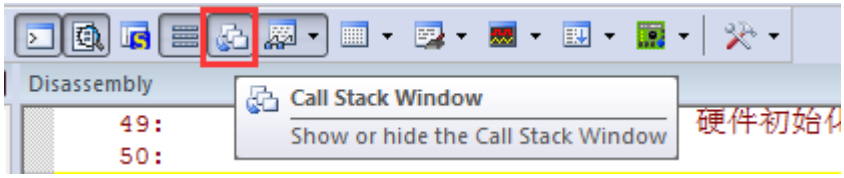
针对全局变量，MDK 专门提供了 Watch1 和 Watch2 两个窗口，每个窗口可以添加很多变量。调试过程中，这些窗口里面添加的变量是会更新的。

也可以从菜单 View ->Watch Windows 进行添加。比如下面添加一个简单的全局计数变量。

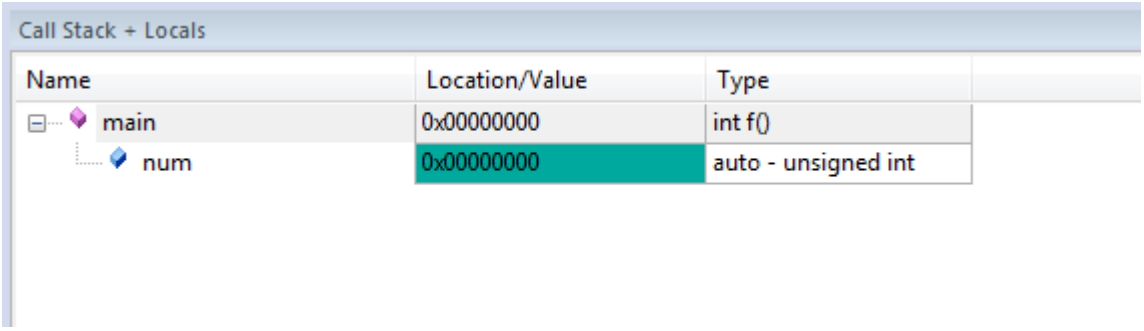


5.5.4 局部变量的查看

针对局部变量，MDK 专门提供了一个栈窗口，用于查看函数中各种局部变量，这个不需要用户添加，MDK 会自动添加。

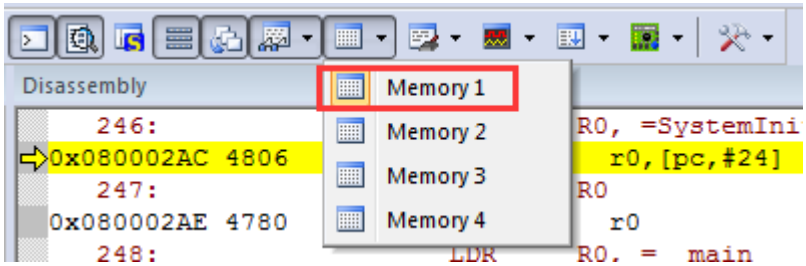


也可以从菜单 View ->Call Stack Window 进行添加。局部变量需要进入到那个函数里面才可以查看，这点要特别注意，因为局部变量是来自栈空间的，进入函数申请占空间，退出函数后，栈空间就会被释放掉。比如 main 函数里面一个 num 局部变量：

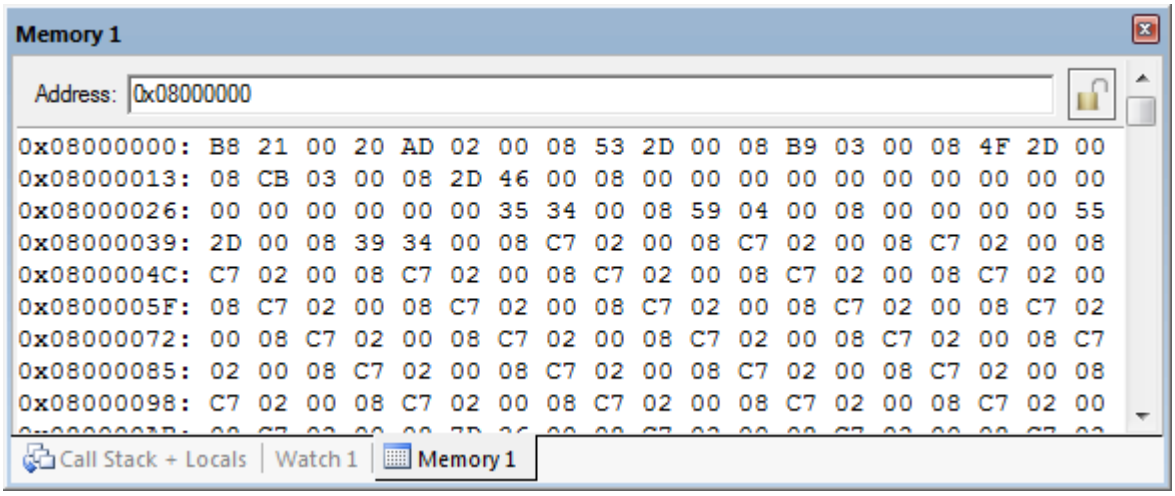


5.5.5 Flash 区或者 RAM 区的查看

如果要查看 Flash 区或者 RAM 区的内容，MDK 提供了 4 个 Memory 窗口。

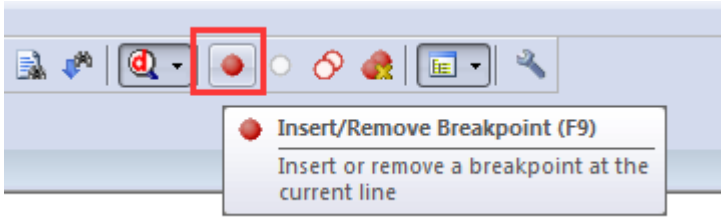


也可以从菜单 View ->Memory Windows 进行添加。比如要查看 0x0800 0000 地址开始的内容，直接输入即可，这个地址是 STM32H7 的 Flash 首地址：



5.5.6 断点调试

断点调试比较重要，进入调整状态后，如果大家想让程序在指定的位置停止运行，可以添加断点，通过这个小红点来添加。



比如想在 84 行添加断点。

```
80          /* 判断定时器超时时间 */
81          if (bsp_CheckTimer(1))
82          {
83              /* 每隔500ms 进来一次 */
84              bsp_LedToggle(2);
85              bsp_LedToggle(3);
86              bsp_LedToggle(4);
87          }
```

用户仅需在那行使用鼠标左键点击一下，可以看到前面有个箭头：

```
80          /* 判断定时器超时时间 */
81          if (bsp_CheckTimer(1))
82          {
83              /* 每隔500ms 进来一次 */
84              bsp_LedToggle(2);
85              bsp_LedToggle(3);
86              bsp_LedToggle(4);
87          }
88      }
```

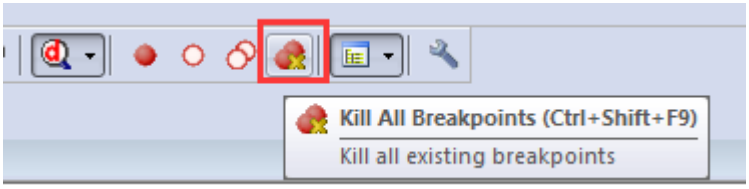
然后点击那个小红点，可以看到已经添加上：

```
80          /* 判断定时器超时时间 */
81          if (bsp_CheckTimer(1))
82          {
83              /* 每隔500ms 进来一次 */
84              bsp_LedToggle(2);
85              bsp_LedToggle(3);
86              bsp_LedToggle(4);
87          }
88      }
```

然后点击全速运行，如果程序有执行到这个地方，就会停在这里。可以看到黄色小箭头也显示在左侧了，说明程序执行这个地方了：

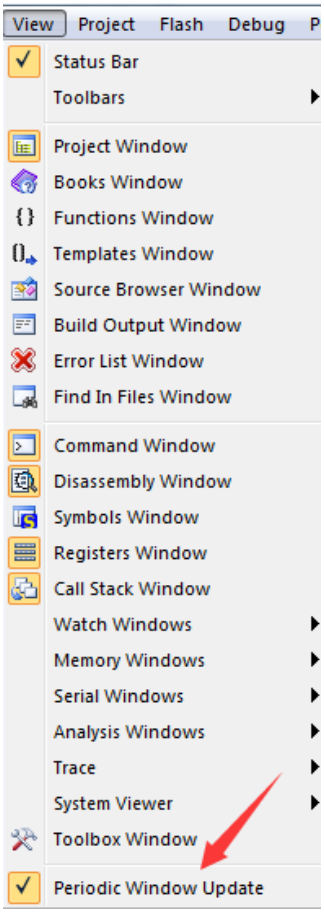
```
80          /* 判断定时器超时时间 */
81          if (bsp_CheckTimer(1))
82          {
83              /* 每隔500ms 进来一次 */
84              bsp_LedToggle(2);
85              bsp_LedToggle(3);
86              bsp_LedToggle(4);
87          }
88      }
89  }
```

正常情况下，断点可以加到任何程序代码可以执行到的地方。另外注意，退出调试前，点击这个按钮取消所有断点，防止 MDK 调试期间崩溃：



5.5.7 寄存器，全局变量和局部变量的动态更新

MDK 有个周期性更新选项，如果大家要周期性的观察一些寄存器、全局变量的更新、需要把菜单 View -> Periodic Window Update 选上。



这个周期不能设置，大概 0.5s 左右更新一次。

5.6 总结

对于初学者来说，务必要实际动手操作一遍。特别是调试方法，要熟练掌握，因为以后工程代码调试时，用到的最多。

第6章 STM32H7 工程模板建立（IAR8）

本章教程为大家介绍如何通过 IAR 下载程序到芯片里面以及 IAR 的调试和配置方法。。

6.1 初学者重要提示
6.2 第 1 步，创建工程文件夹
6.3 第 2 步，创建新工程
6.4 第 3 步，设置工程分组名
6.5 第 4 步，添加工程文件
6.6 第 5 步，保存创建的 Workspace
6.7 第 6 步，Options 配置
6.8 第 7 步，最后一步验证是否创建成功
6.9 IAR 的字体，多核并行编译，语法提示等设置
6.10 总结

6.1 初学者重要提示

- IAR 的工程创建过程还是比较重要的，务必实际上手操作一遍。

6.2 第 1 步，创建工程文件夹

创建一个文件夹，里面包含如下文件（直接将跑马灯例子的 Project 文件里面内容全部删除，然后整个工程复制进来即可）。

- Doc 文件夹

  用于记录例程的修改记录和例子简介。

- Libraries 文件夹

  在第一章详细介绍从官网的下载方法。

- Project 文件夹

  这个是本章的重点，在这个文件夹里面再建立两个文件夹，EWARMv8 文件夹用于存放 IAR 工程，MDK-ARM(uV5)文件夹里面用于存放 MDK 工程，方便以后创建 MDK 的工程。

- User 文件夹

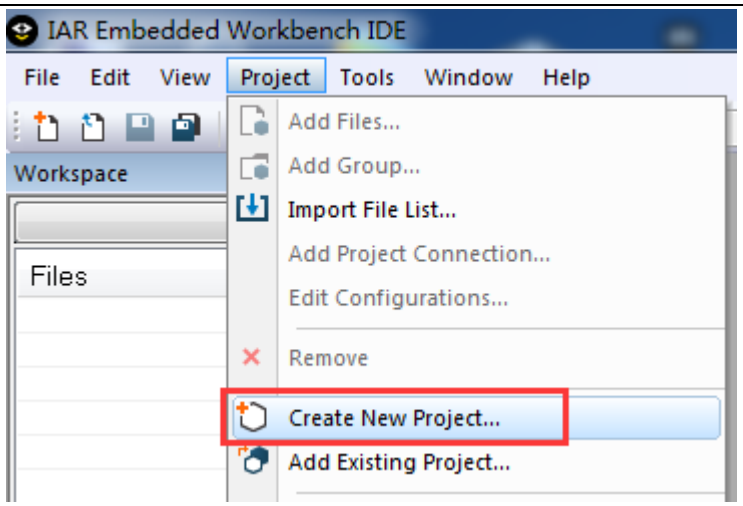
  这个是要用户提供的，开发板已经都提供好，bsp 文件夹里面是开发板各个外设的驱动，segger 文件夹是硬件异常分析代码。

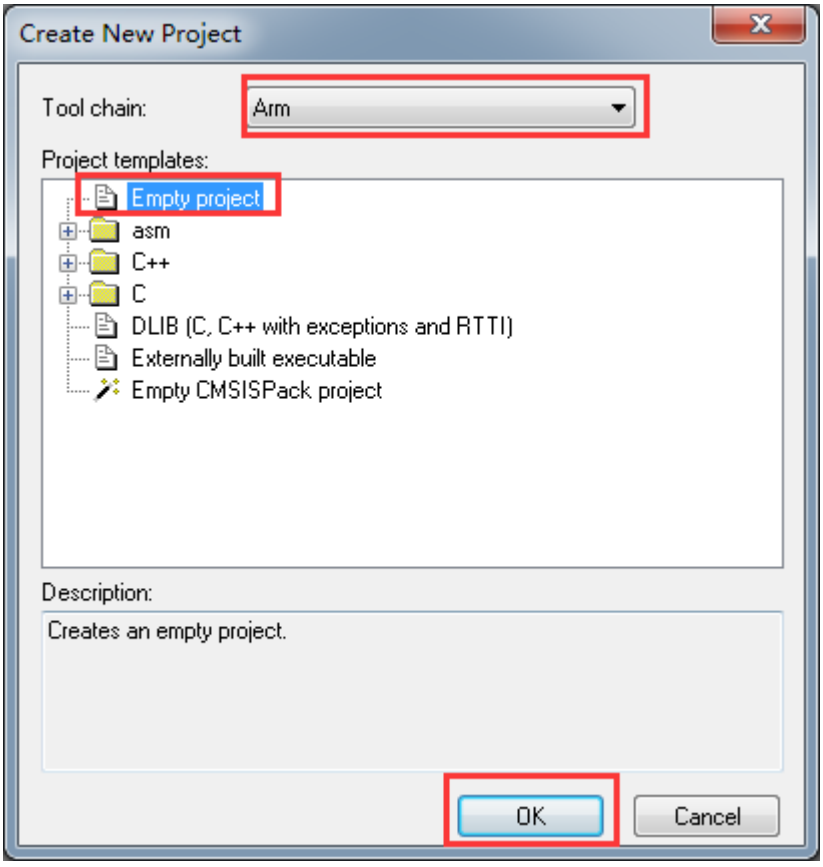
- 删除目标文件.bat

  这个是批处理文件，用于删除 MDK 和 IAR 工程编译后生成的 obj 和 list 临时文件。

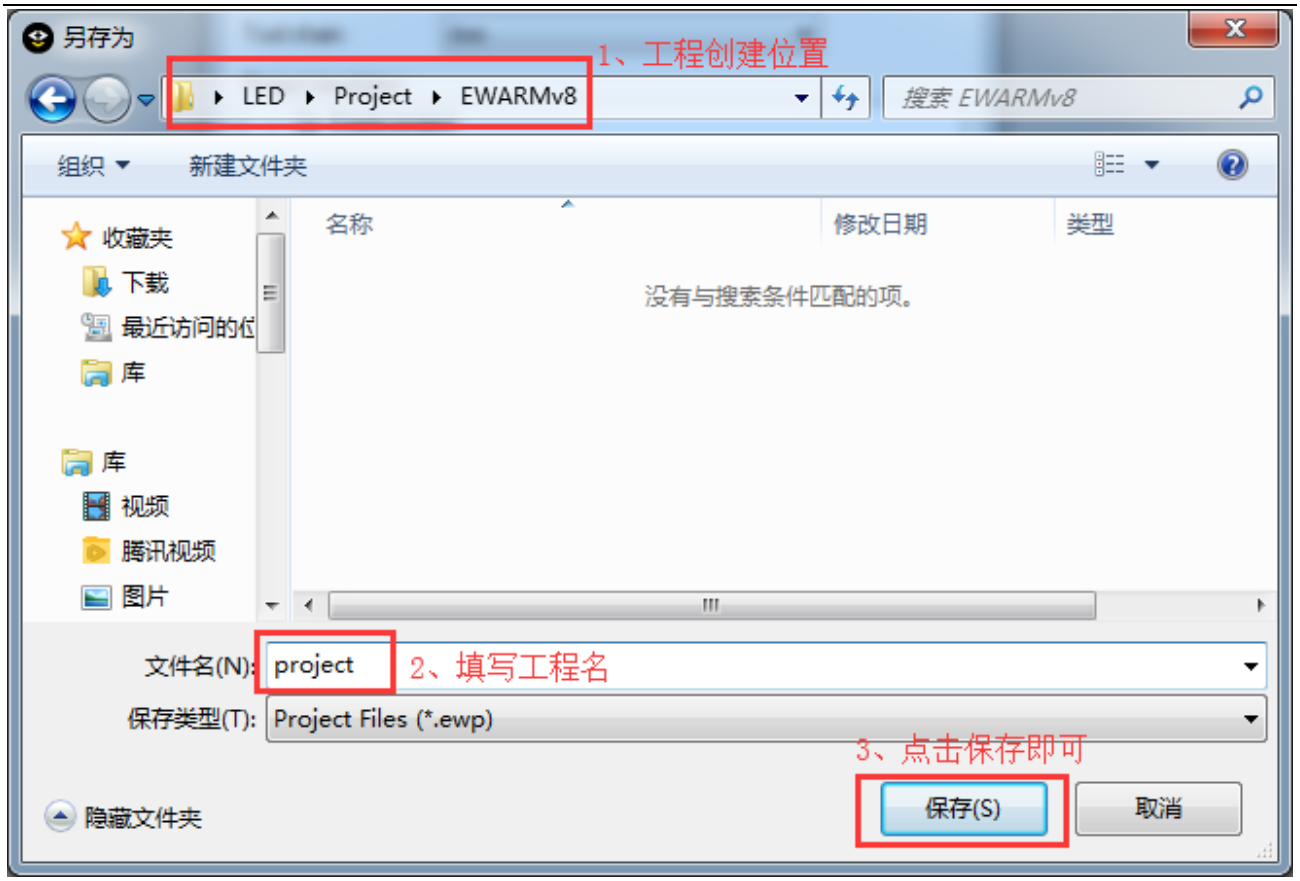
6.3 第 2 步，创建新工程

打开 IAR，选择 Project ->Create New Project

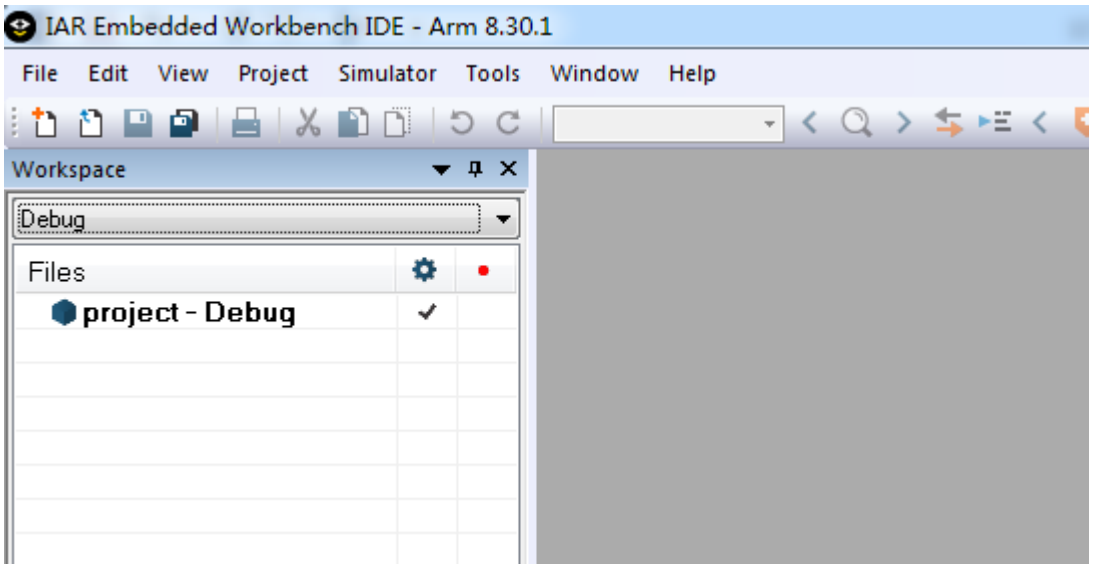




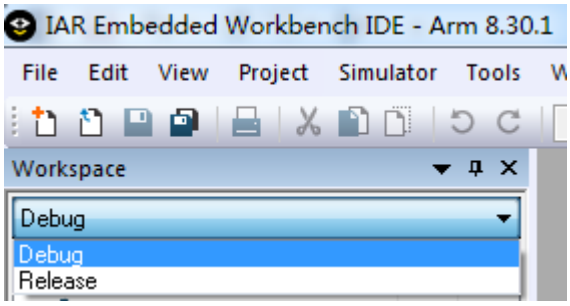
工程名设置为 project：



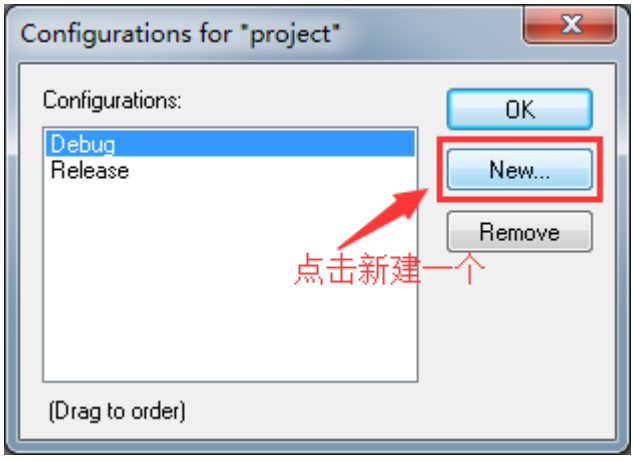
创建后的效果如下：



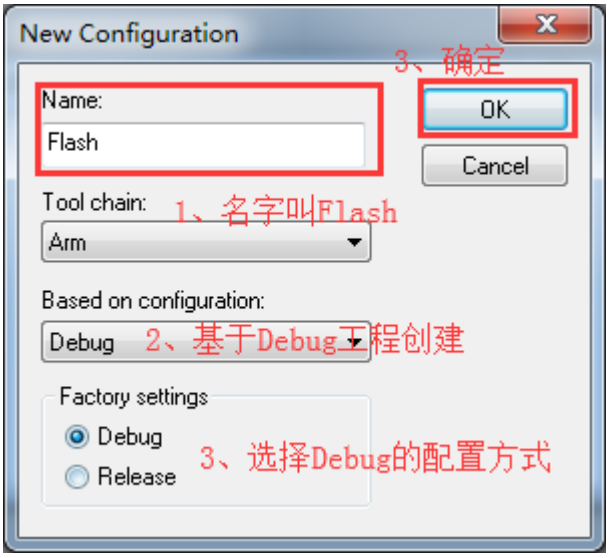
默认的情况下这里是创建了两个工程，一个用于 Debug，一个用于 Release。这两个其实是用的同一个工程，最大的区别是优化等级有所不同。



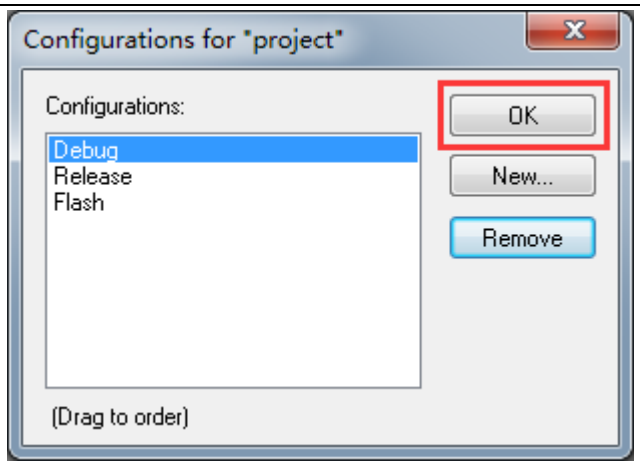
我们这里设置一下，将这两个都删掉，新创建一个 Debug 方式的，只是仅换了一个名字（主要目的是让大家看下操作方法）。打开菜单 Project->EDIT Configration：



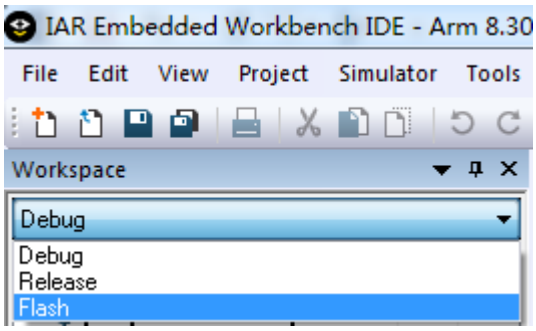
先新建后再删除其它两个，因为当前 IAR 使用的一个是无法删除的。



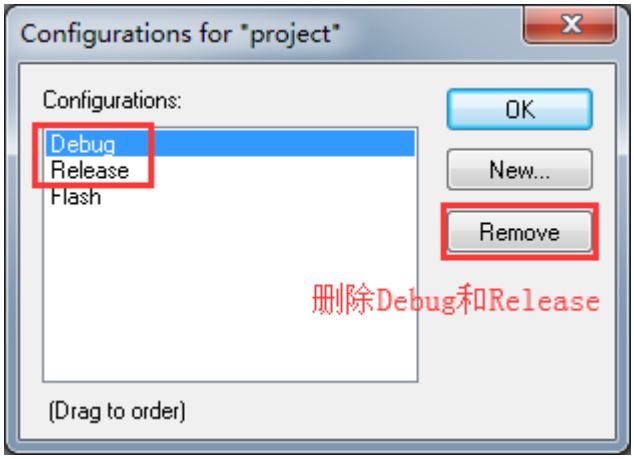
点击确定后，再次点击确定：



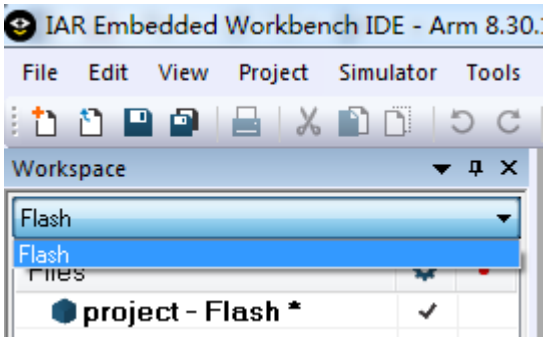
可以看到左上角已经有三个了，这我们选择 Flash，将其激活，也就是使用这个：



再次打开菜单 Project->EDIT Configration 就可以将 Debug 和 Release 删除掉了：

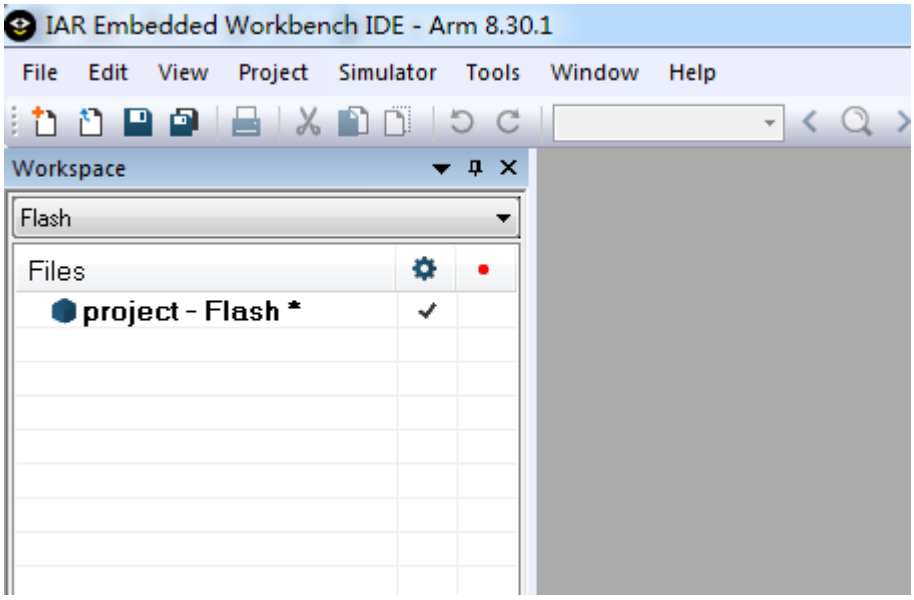


现在左上角就只有一个 Flash 了：

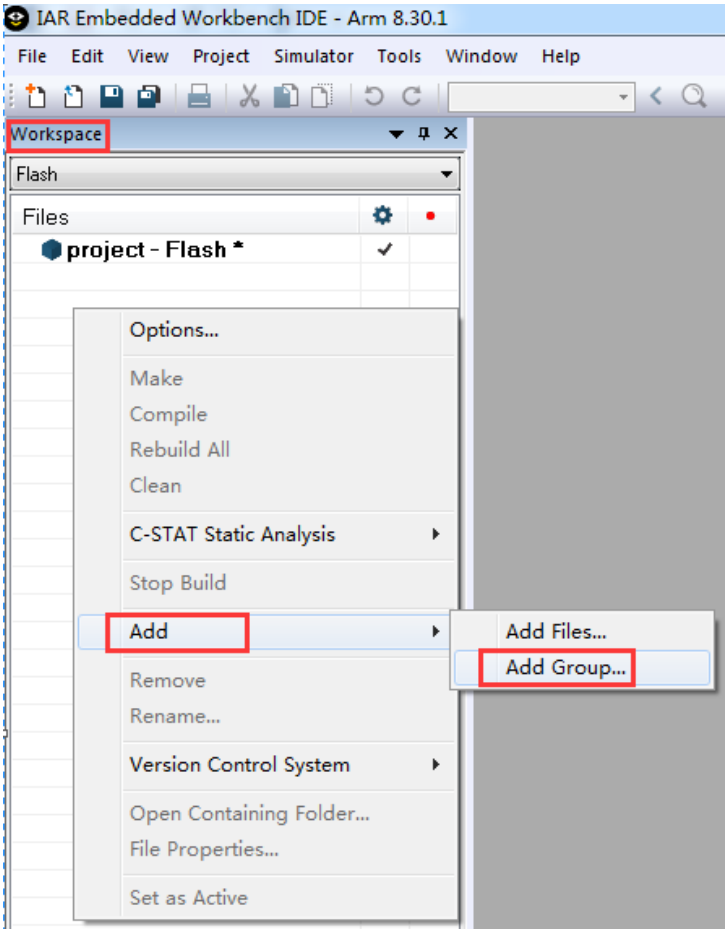


6.4 第 3 步，设置工程分组名

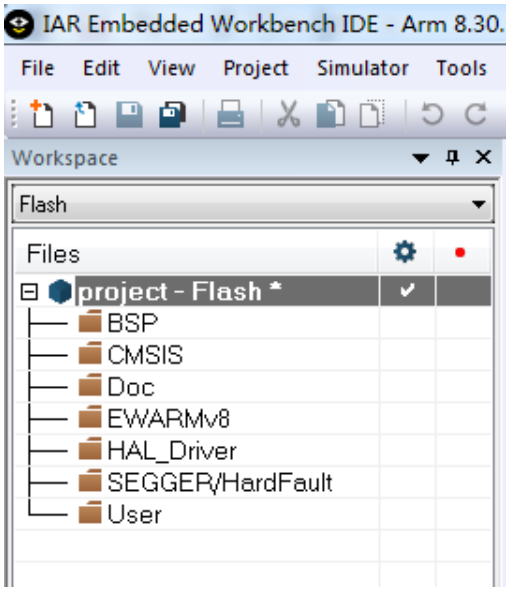
默认新建的工程是下面这样的：



在 Workspace 栏的空白地方鼠标右击选择 Add->Add Group（如果有多个 project 同时在里面的话，要鼠标左击选中 project - Flash，然后右击添加）：

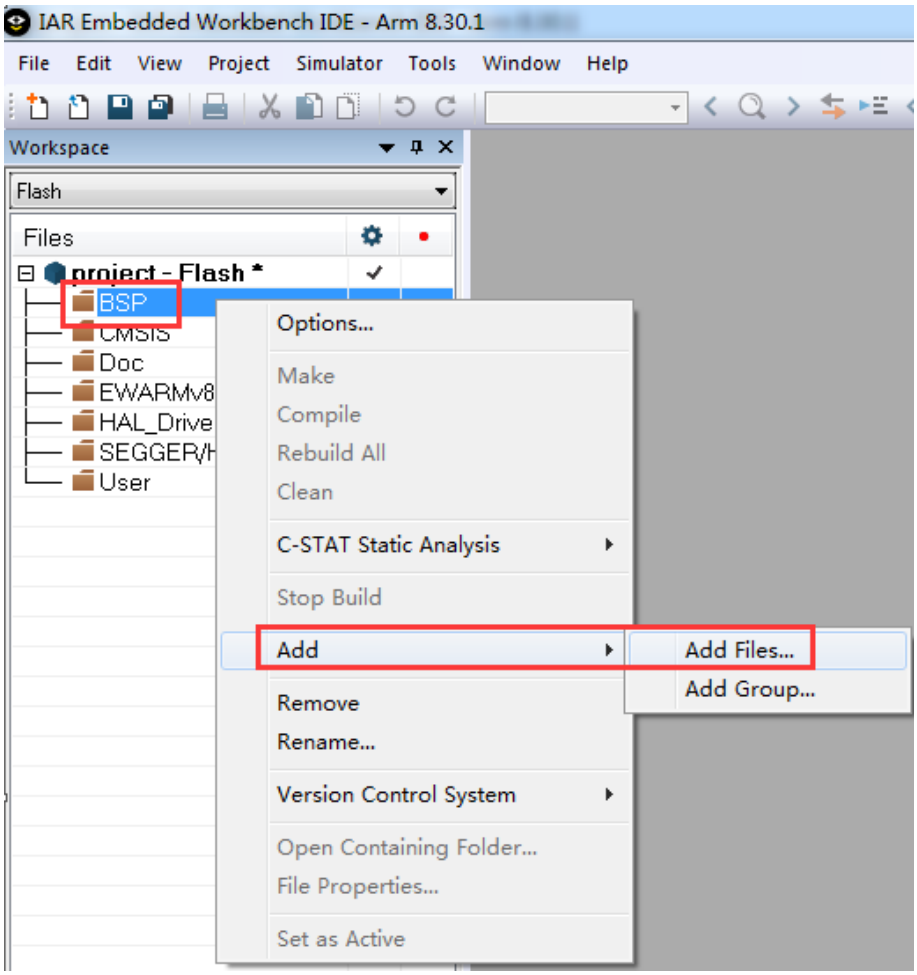


通过这种方式，我们依次创建如下 7 个分组：

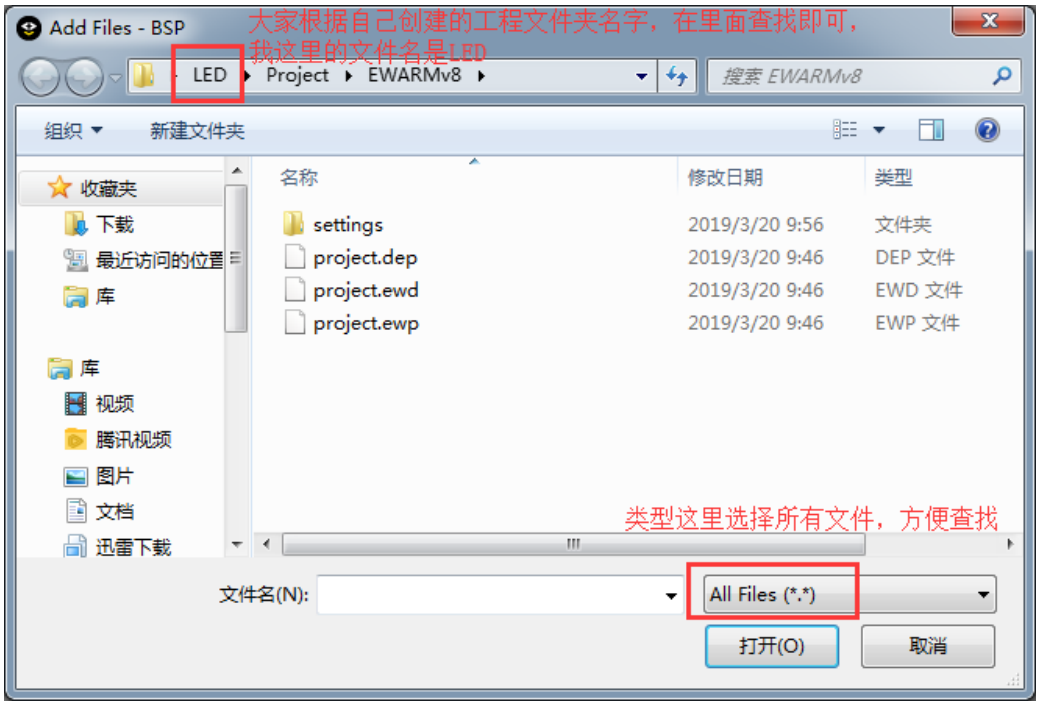


6.5 第 4 步，添加工程文件

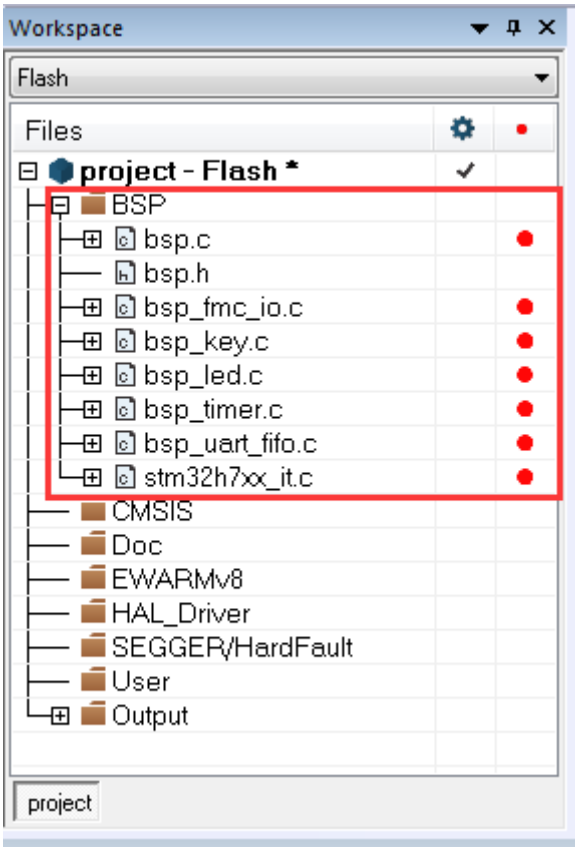
下面我们为各个分组添加上具体的文件。鼠标左击选中 BSP，然后右击选择 Add->Add Files



然后在新建的工程文件夹里面查找需要添加的文件即可。



◆ 添加 BSP 分组需要的文件



bsp.c 位于路径\User\bsp，主要用于 MPU、Cache、时钟和外设初始化。

bsp.h 位于路径\User\bsp，此文件是各种头文件的集中汇总贴，方便管理。

bsp_led.c 位于路径\User\bsp\src，用于 LED 驱动。

bsp_key.c 位于路径\User\bsp\src，用于按键驱动。

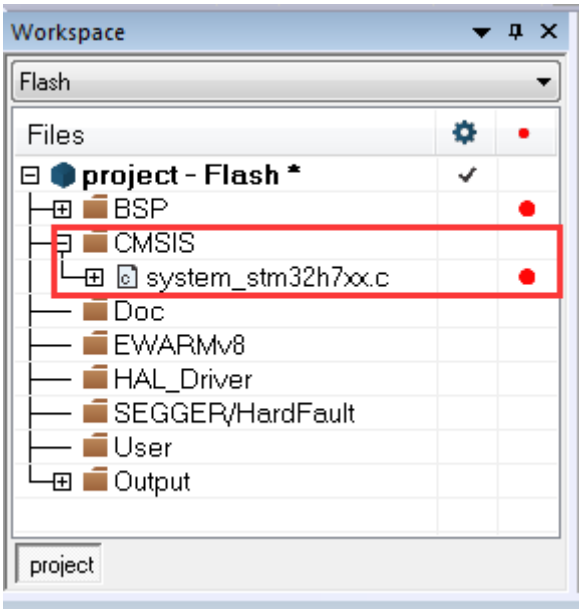
bsp_timer.c 位于路径\User\bsp\src，用于软件定时器组和微妙级单次延迟。

bsp_fmc_io.c 位于路径\User\bsp\src，用于 IO 扩展，这里 LED 要用到。

bsp_uart_fifo.c 位于路径\User\bsp\src，用于 8 串口 FIFO 驱动。

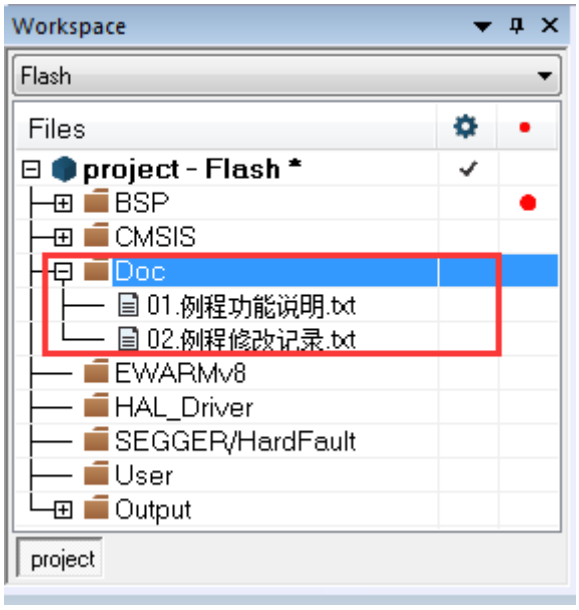
stm32h7xx_it.c 位于路径 User\bsp，用于部分系统中断服务程序。

◆ 添加 CMSIS 分组需要的内容



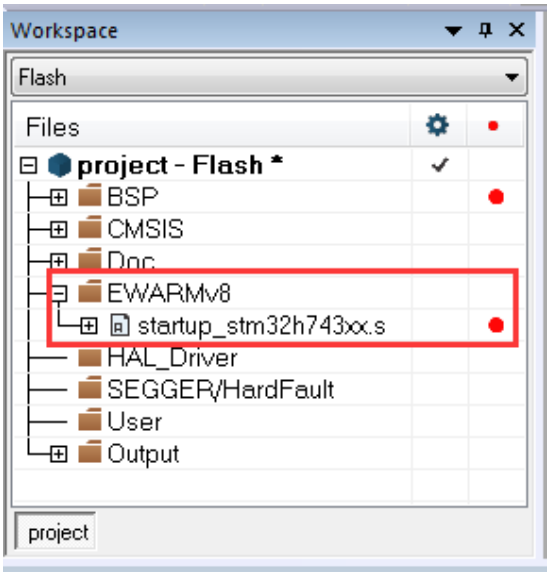
system_stm32h7xx.c 位于路径\Libraries\CMSIS\Device\ST\STM32H7xx\Source\Templates，主要用于复位 RCC 相关寄存器和中断向量表位置设置。

◆ 添加 Doc 分组需要的内容



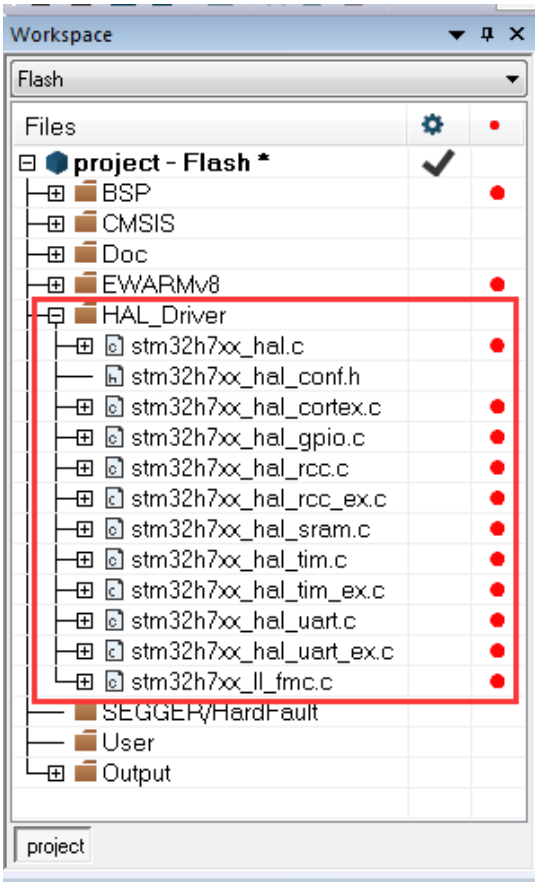
这两个文件位于 Doc 文件夹，主要用于说明例程的功能和更新记录。

◆ 添加 EWARMv8 分组需要的内容



startup_stm32h743xx.s 位于路径\Libraries\CMSIS\Device\ST\STM32H7xx\Source\Templates\iar，主要用于堆和栈空间设置，中断向量表也在这个文件。

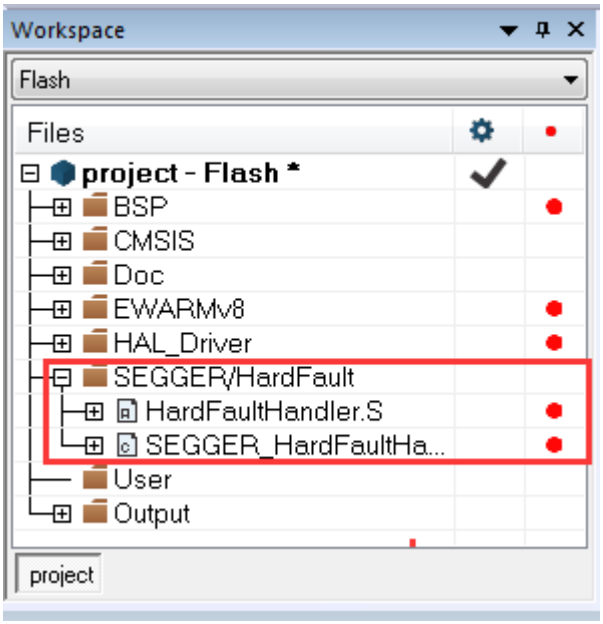
◆ 添加 HAL_Driver 分组需要的内容



stm32h7xx_hal_conf.h 位于路径\User\bsp，主要用于 HAL 库的配置。

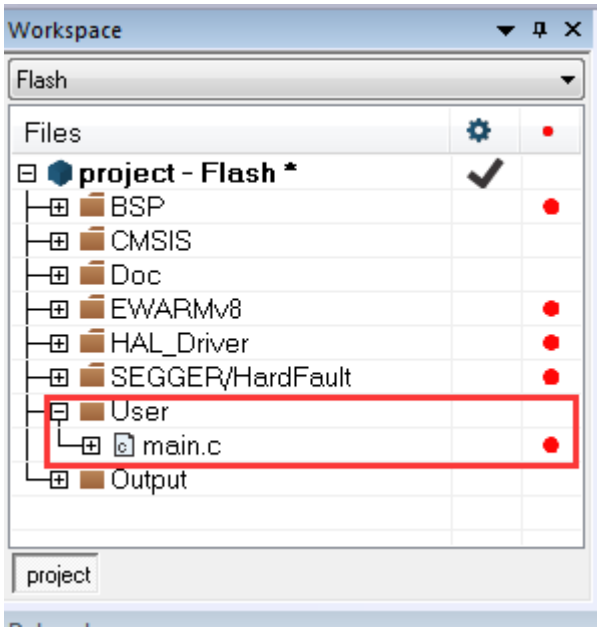
其它文件路径：\Libraries\STM32H7xx_HAL_Driver\Src，用到什么外设添加什么驱动文件即可，有些外设驱动跟 DMA 也有关联，直接添加进来即可。

◆ 添加 SEGGER/HardFault 分组需要的内容



这两个文件位于路径：\User\segger\HardFaultHandlerMDK，主要用于硬件异常分析。

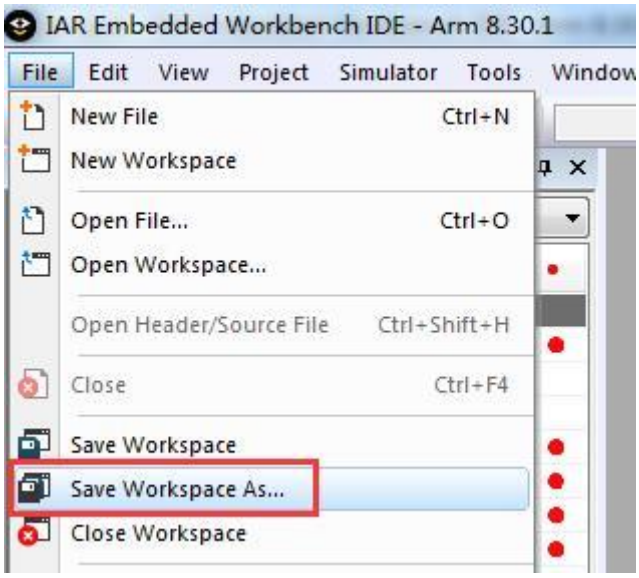
◆ 添加 User 分组需要的内容



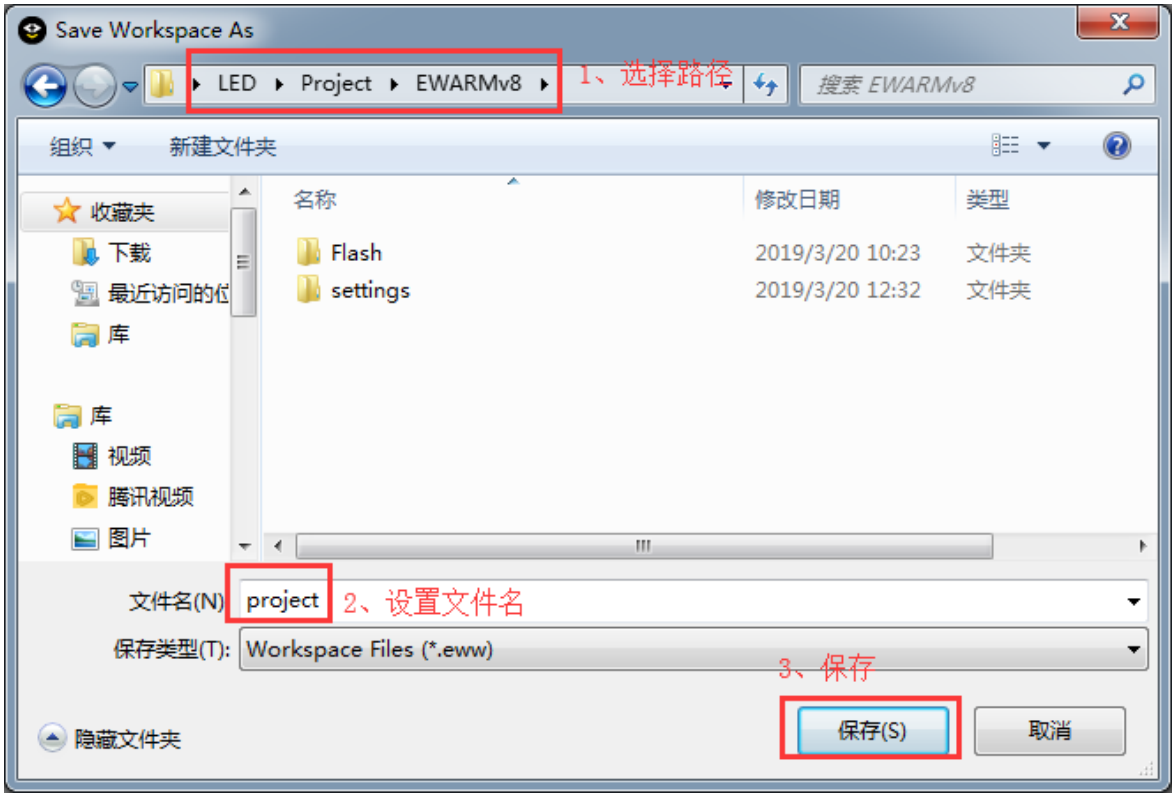
main.c 文件位于文件夹 User，实现了一个简单的跑马灯效果。

6.6 第 5 步，保存创建的 Workspace

文件添加完毕后要保存 Workspace，点击菜单 File->Save Workspace As



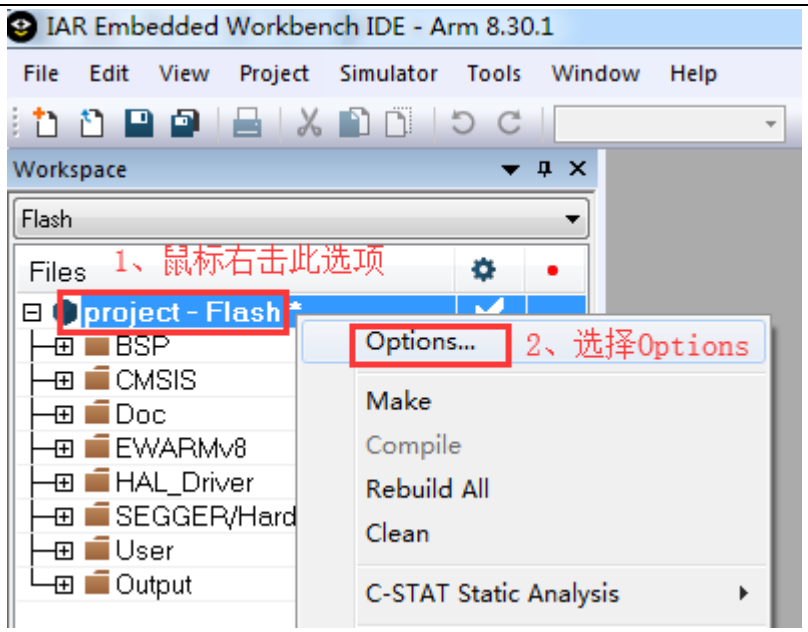
弹出的对话框里面选择路径，设置文件名后，保存即可：

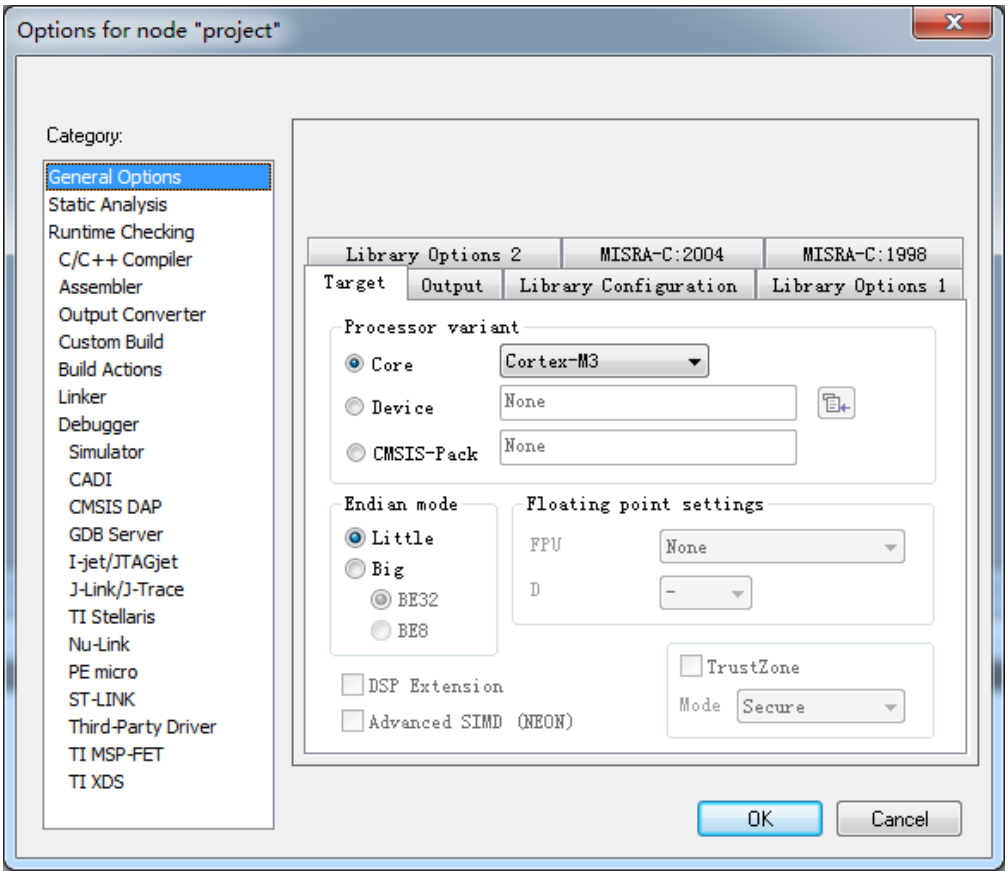


剩下就是工程的配置工作了。

6.7 第 6 步，Options 配置

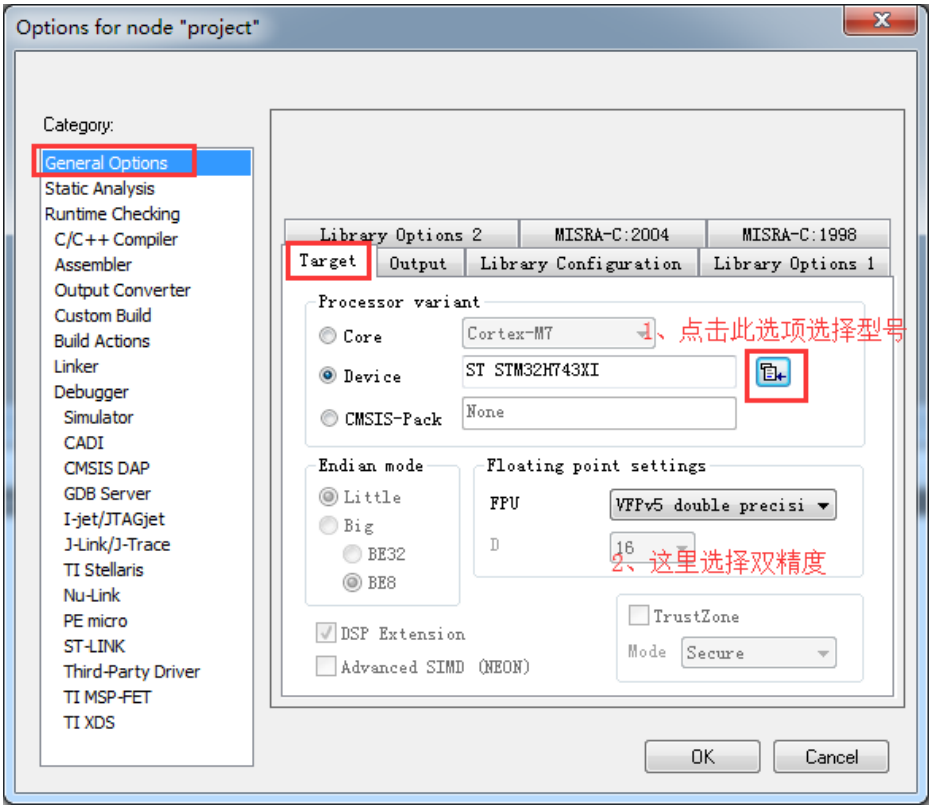
IAR 的配置基本都集中在 Options 对话框里面：



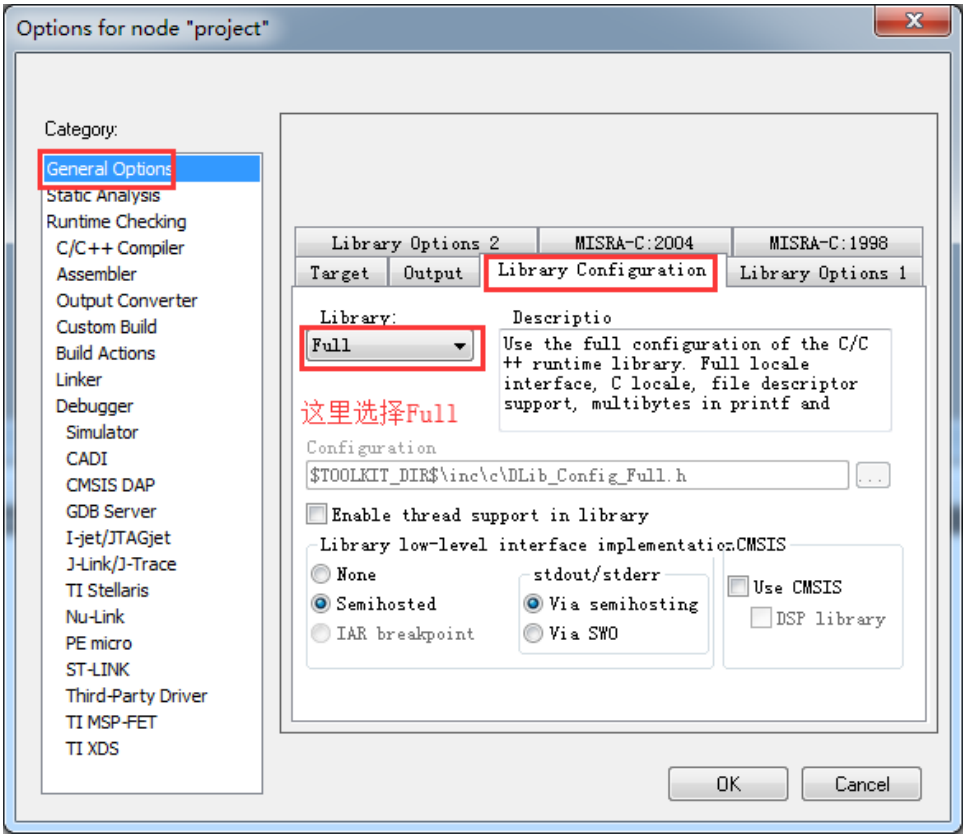


6.7.1 General Options 选项

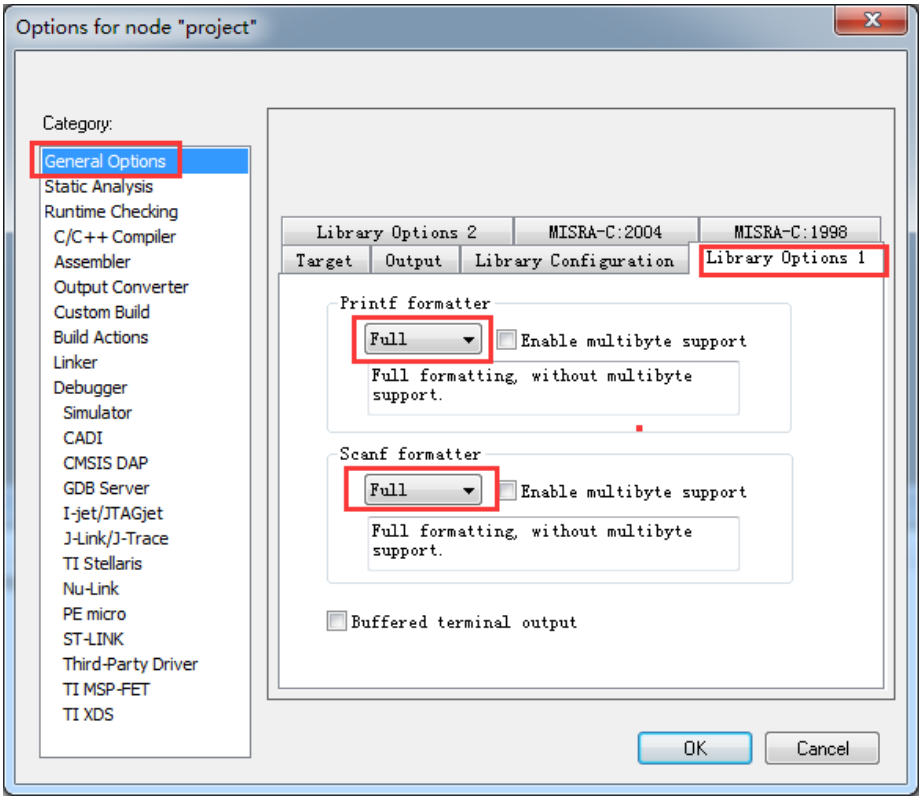
这部分主要用于通用设置。选择 ST 的 STM32H743XI：



Library Configration 选择全功能库：

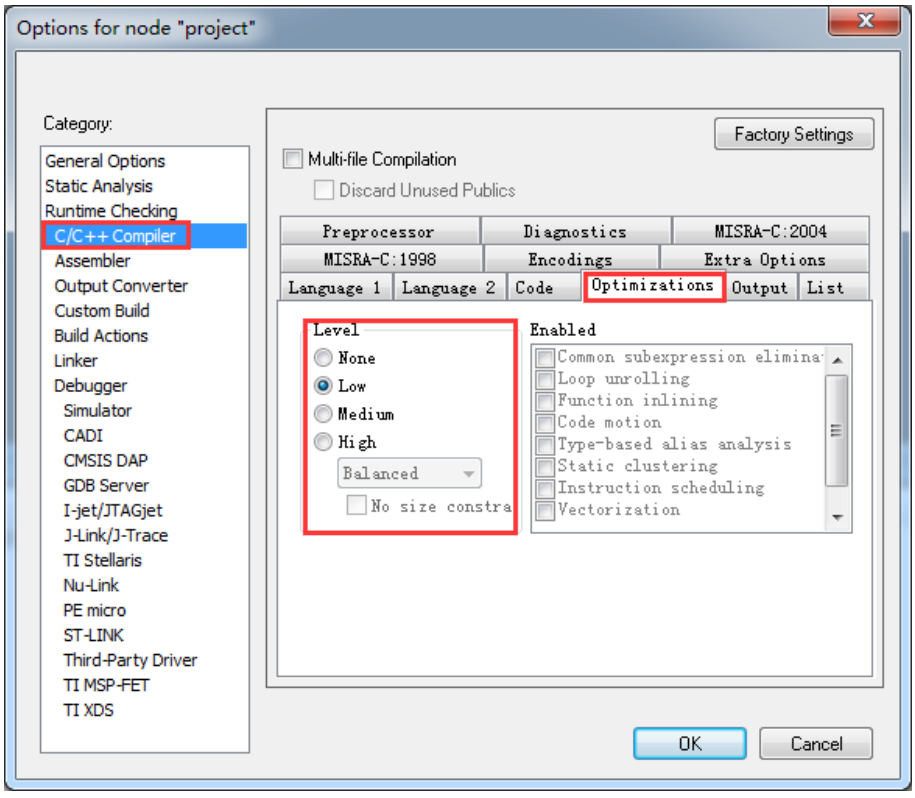


Library Options1 里面的 printf 和 scanf 都选择支持全功能格式：

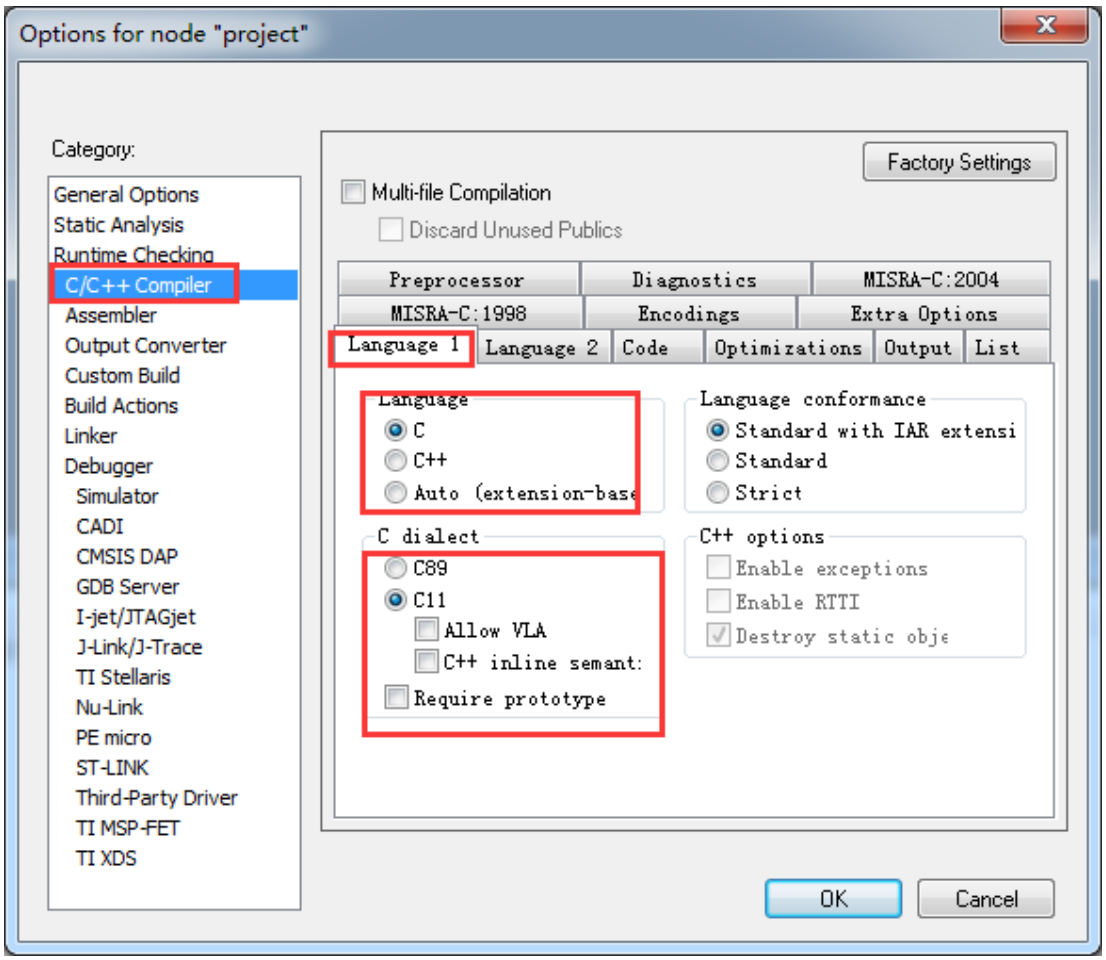


6.7.2 C/C++ Compiler 选项

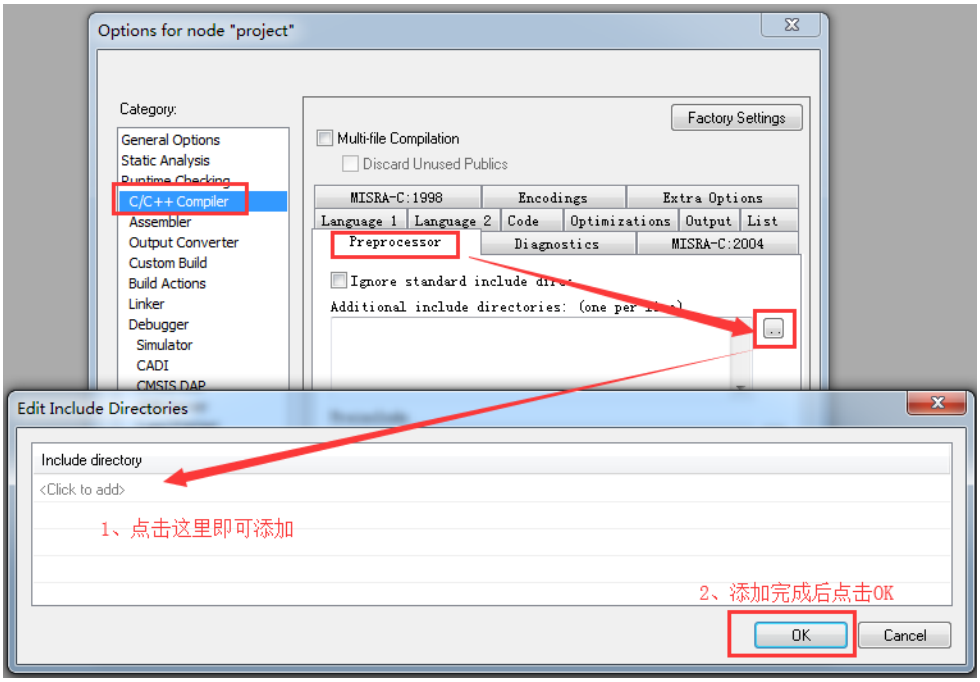
这部分主要用于编译器设置，优化等级选择默认的低优先级：



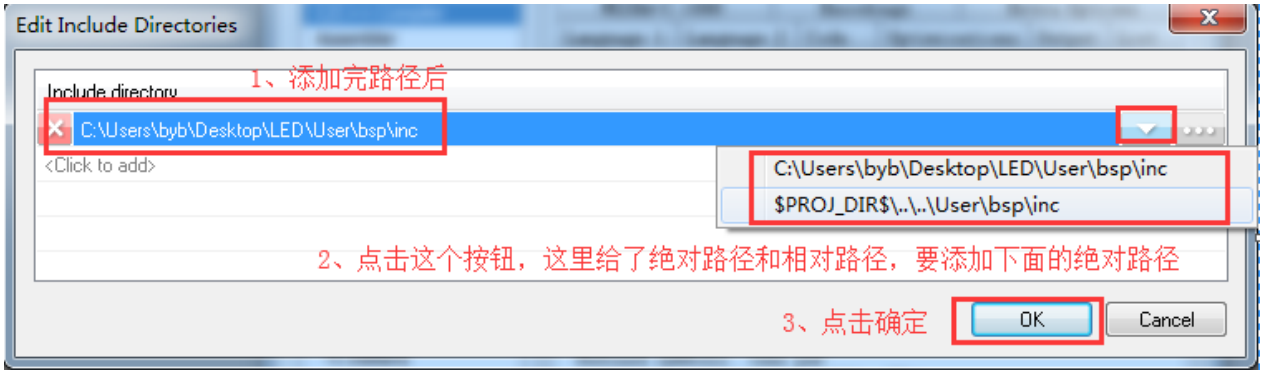
Language 1 选择 C 语言和 C11 规范（在第 9 章有介绍）：



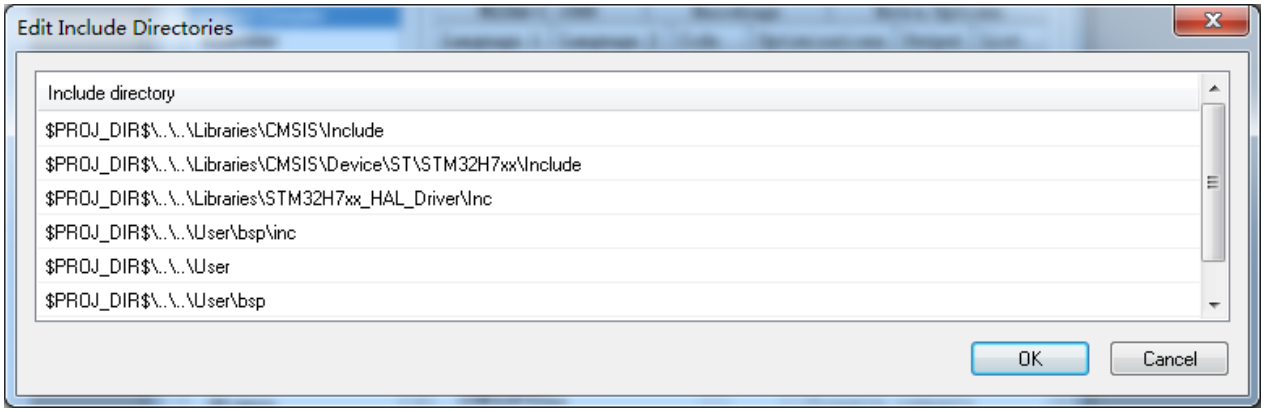
然后就是最重要的头文件路径添加，IAR 提供的添加方式都是相对路径：



注意，添加的时候要添加相对路径，使用绝对路径的话，工程换个位置就无法使用了：



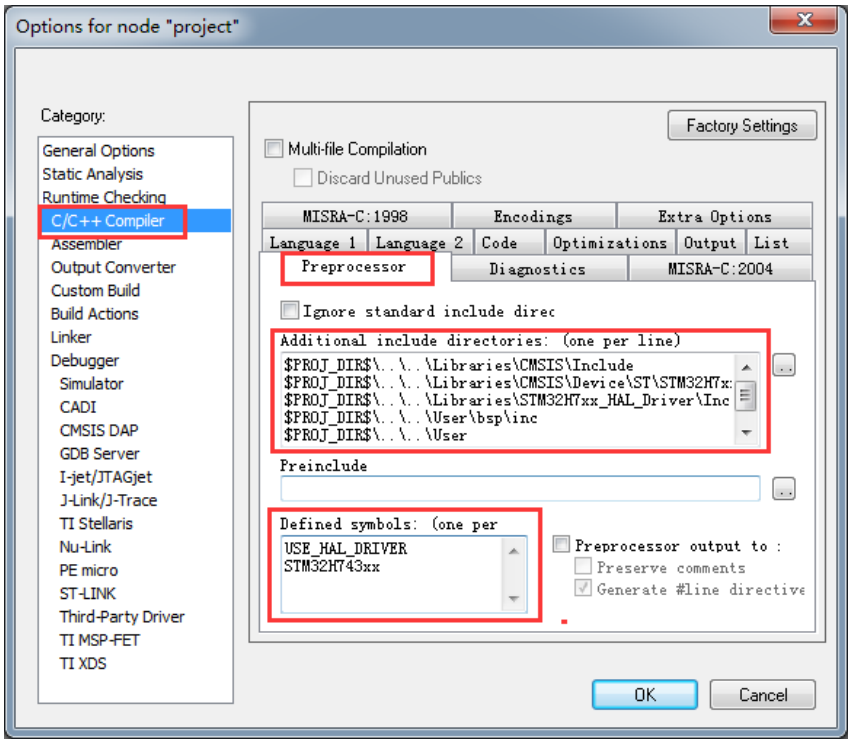
将下面路径都添加进去：



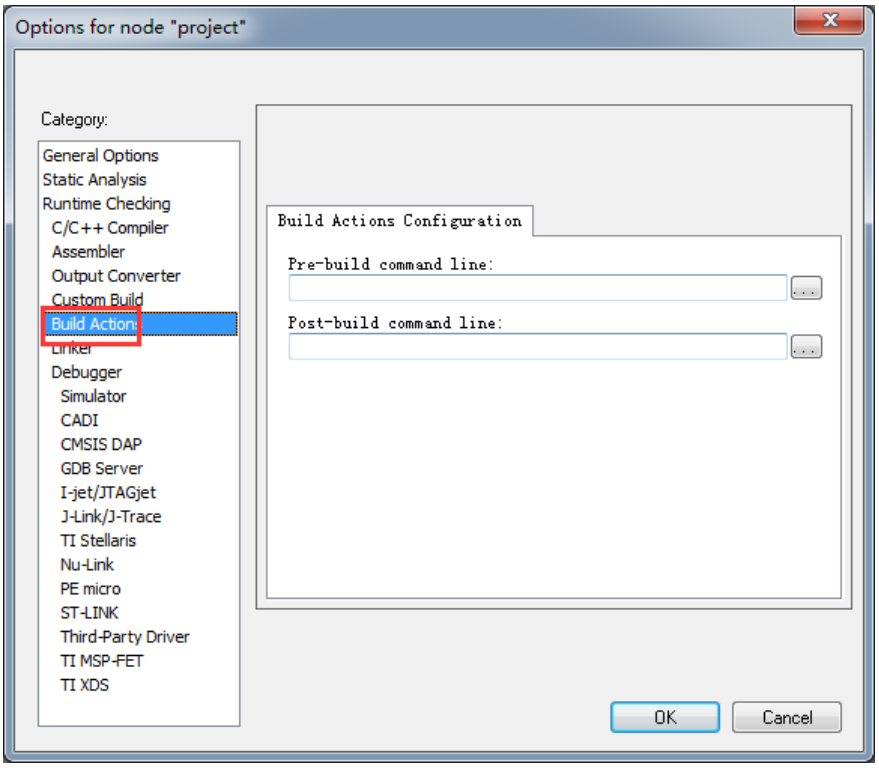
同时在预定义宏选项里面添加如下宏定义：

USE_HAL_DRIVER 表示使用 HAL 库。

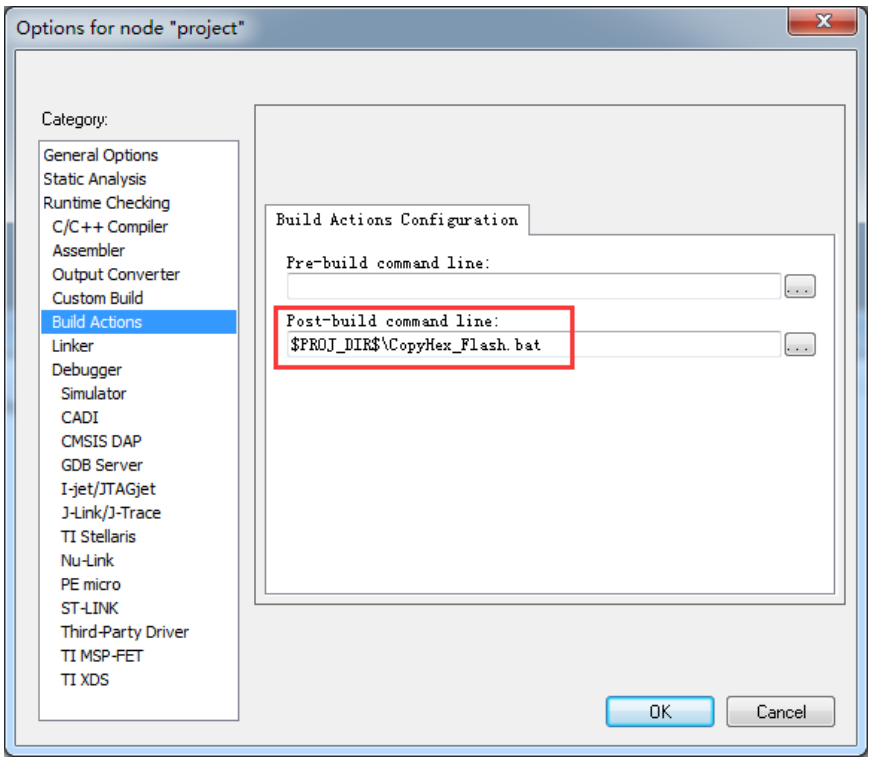
STM32H743xx 表示使用 H743 系列。



6.7.3 Build Actions 选项

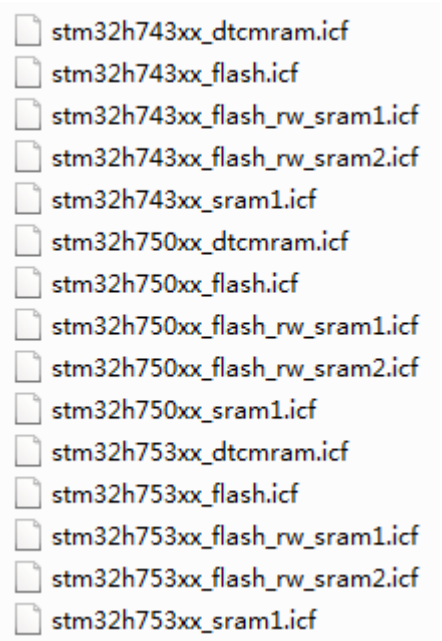


这个选项基本用不上，可以不用管，主要是用来指定编译之前和编译之后执行的用户程序。对于本教程配套的例子来说在这里添加了一个批处理功能，用来实现 Objects 文件夹里面的 hex 文件复制到路径 Project\EWARMv8 里面，方便查找。

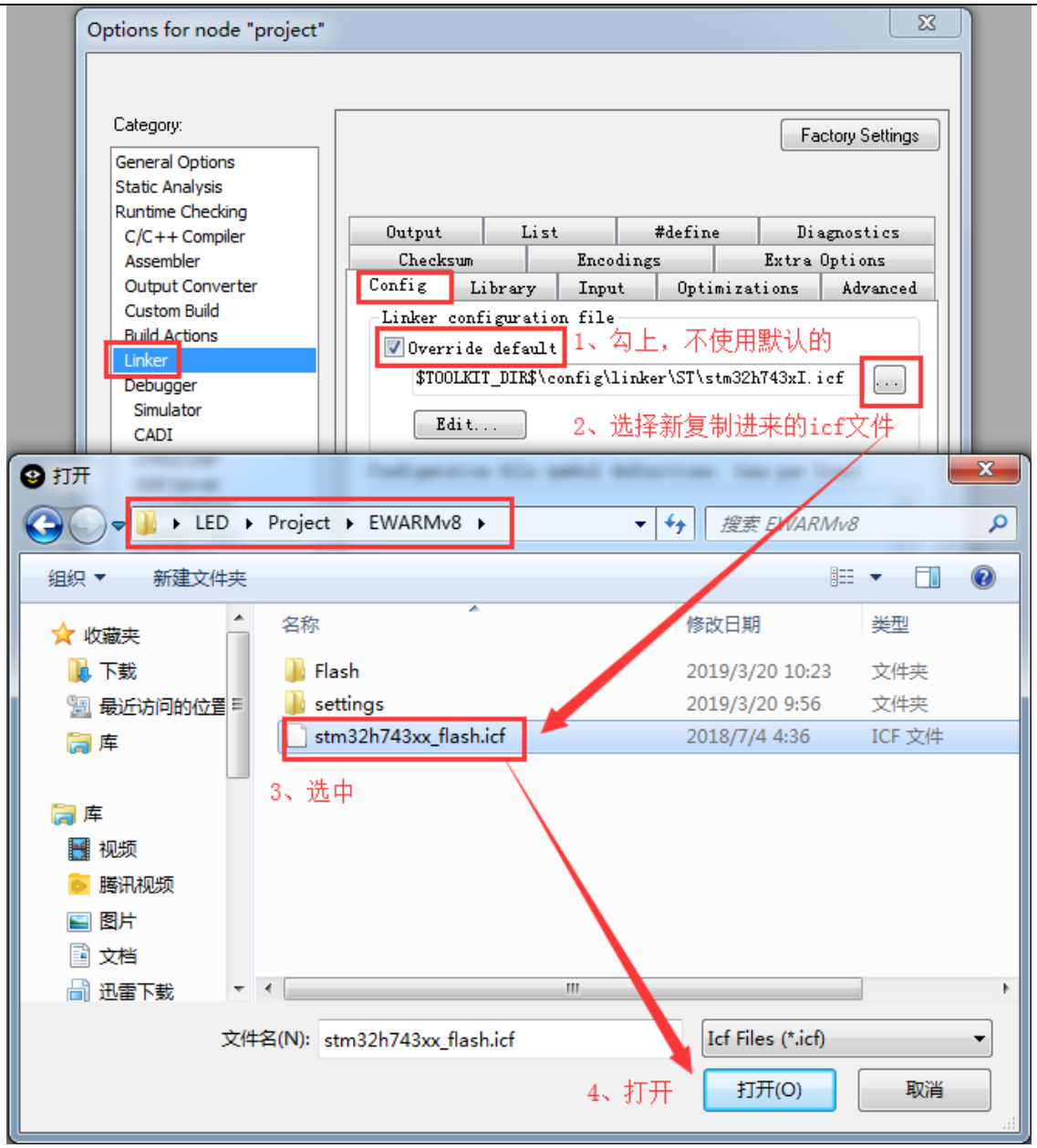


6.7.4 Linker 选项

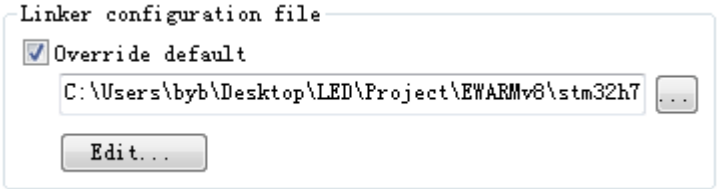
Linker 选项主要是链接器的配置，其中最主要的是 icf 文件配置 Flash 和 RAM 区。在 HAL 库的软件包的路径：\Drivers\CMSIS\Device\ST\STM32H7xx\Source\Templates\iar\linker 里面提供了很多配置方式：



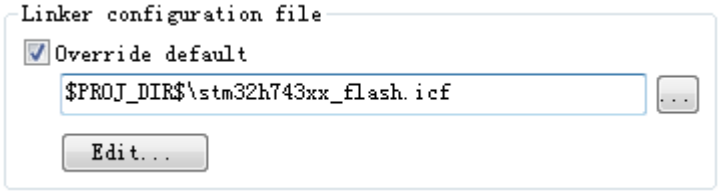
教程配套例子使用的是 stm32h743xx_flash.icf，将这个文件复制到所创建工程的路径\Project\EWARMv8 里面。然后在 Linker 选项里面将其添加进来：



添加进来后是个绝对路径：

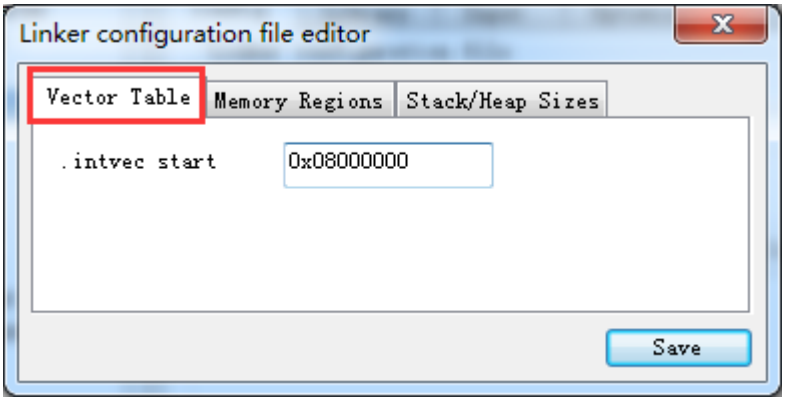


要修改为相对路径：

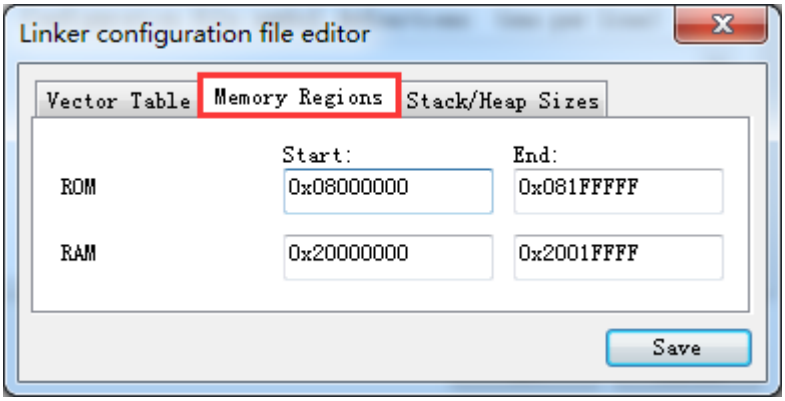


添加进来后要设置下中断向量表首地址，堆和栈大小，点击上面截图左下角的 Edit 按键进行配置。

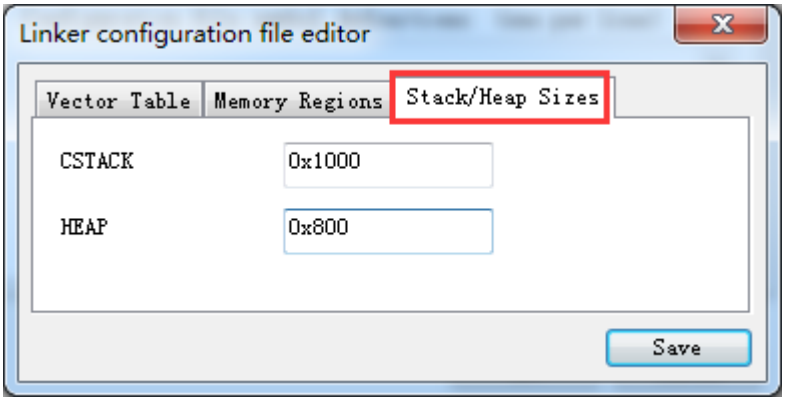
中断向量表首地址设置为 0x0800 0000:



STM32H743XIH 的 Flash 首地址是 0x0800 0000，大小 0x200000，即 2MB。
默认的 RAM 空间选择 DTCM，首地址 0x2000 0000，大小 0x20000，即 128KB。



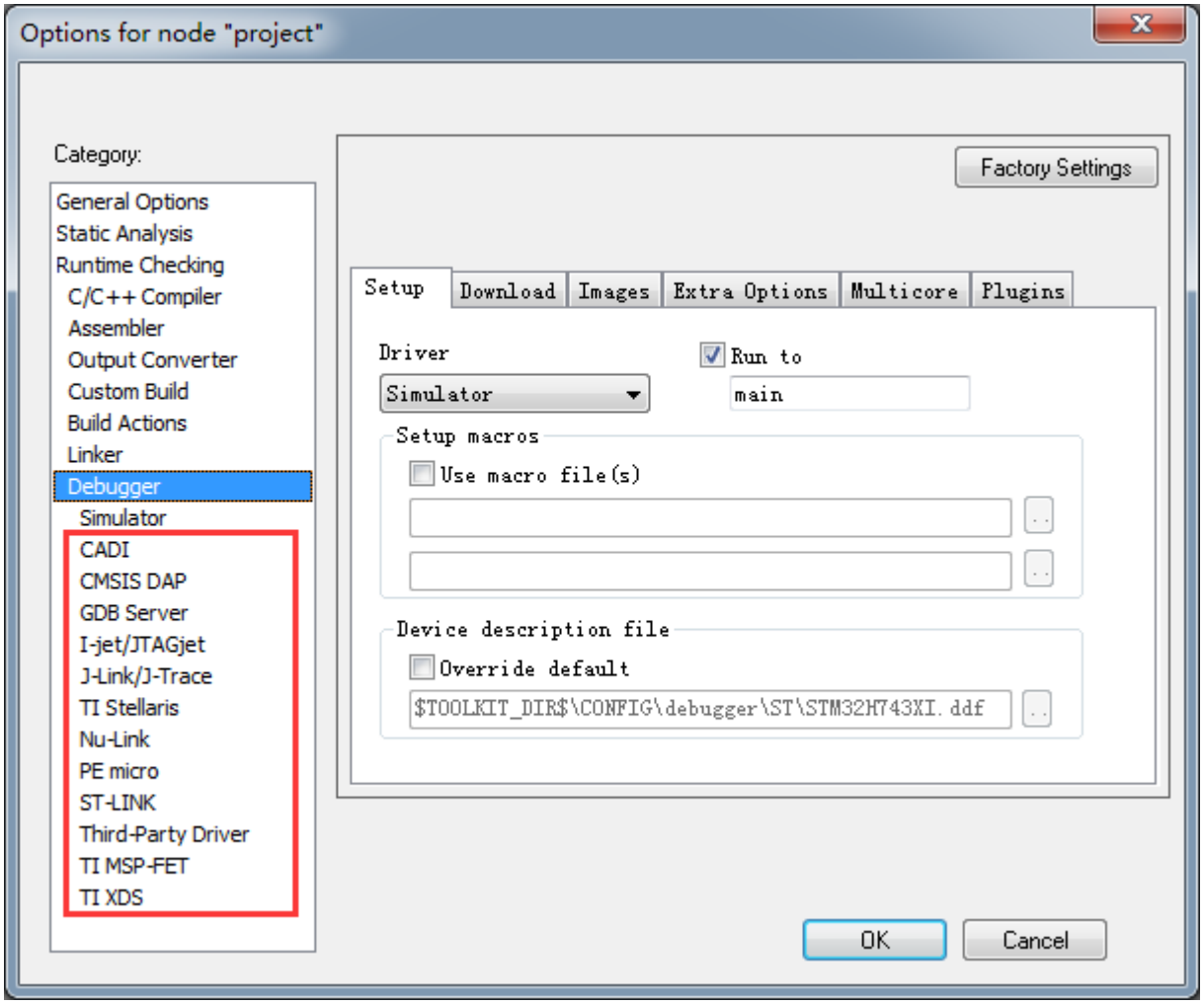
堆和栈大小分别设置为 0x1000 和 0x800：



设置完毕后记得点击保存。

6.7.5 Debugger 选项

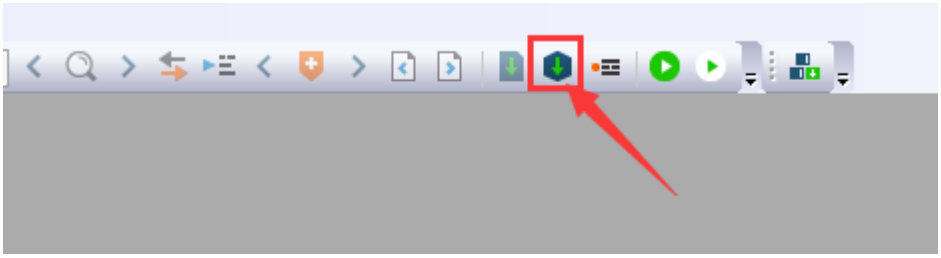
Debugger 选项主要用于调试下载，当前 IAR 支持如下调试工具：



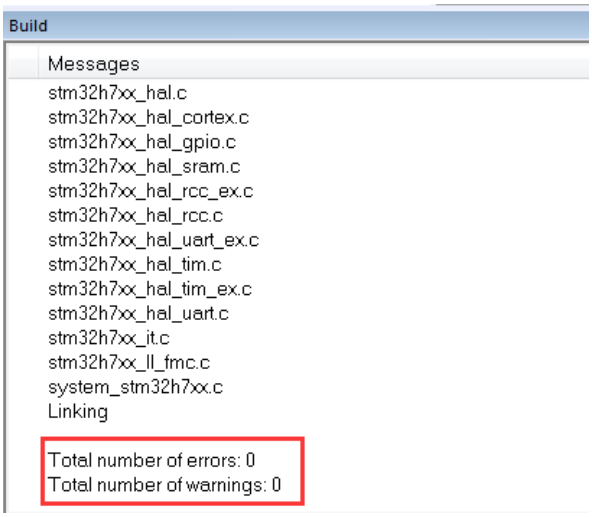
具体的调试设置在下一章有详细说明，本章主要是保证创建的工程能够正常编译。

6.8 第 7 步，最后一步验证是否创建成功

点击全编译：



编译结果：0 错误，0 警告，编译成功。

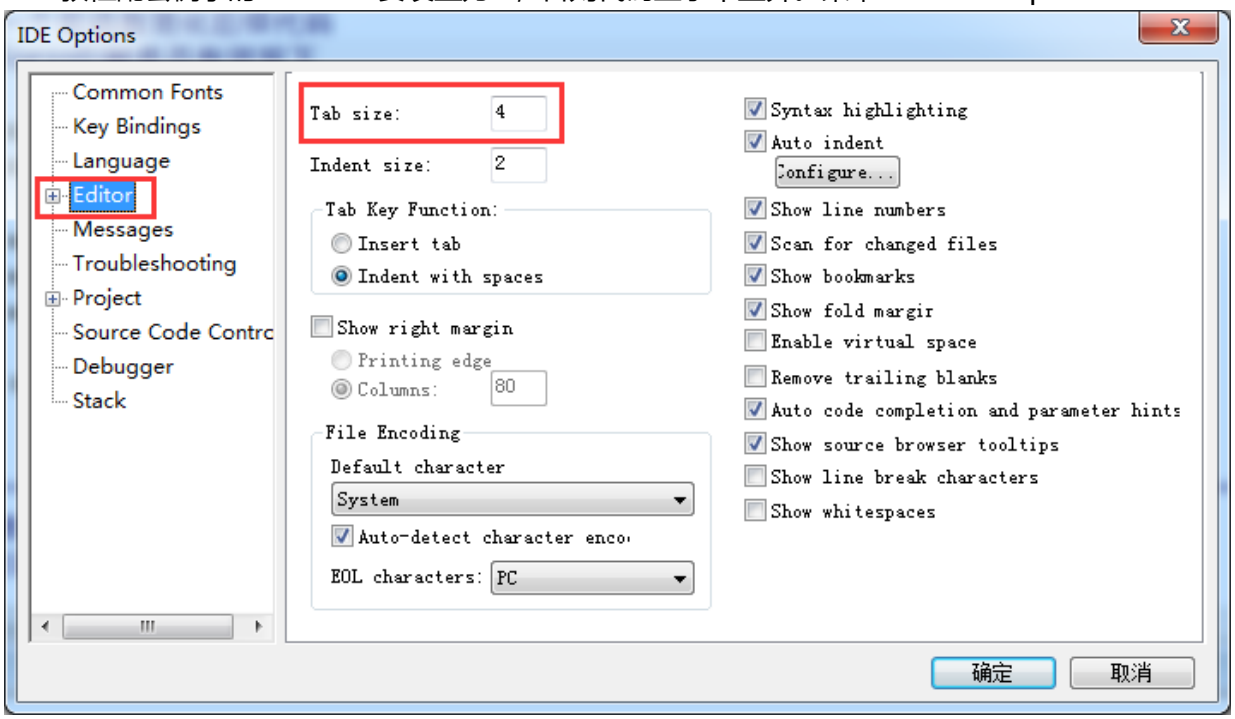


6.9 IAR 的字体，多核并行编译，语法提示等设置

这里将 IAR 经常用到的设置做个说明。

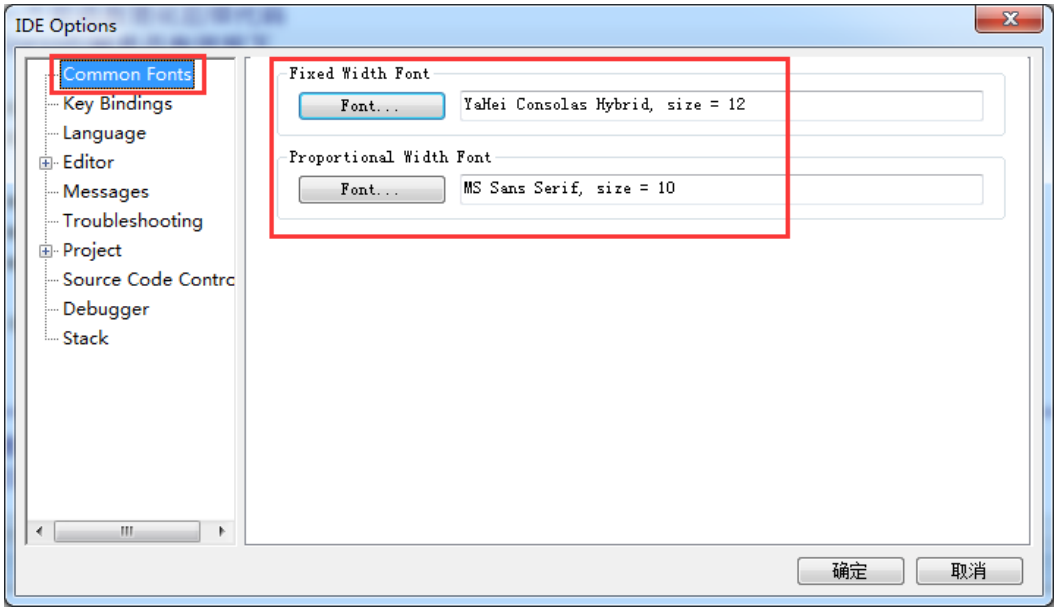
6.9.1 Tab Size 设置

教程配套例子的 Tab size 要设置为 4，否则代码显示不整齐。菜单 Tools -> Options：



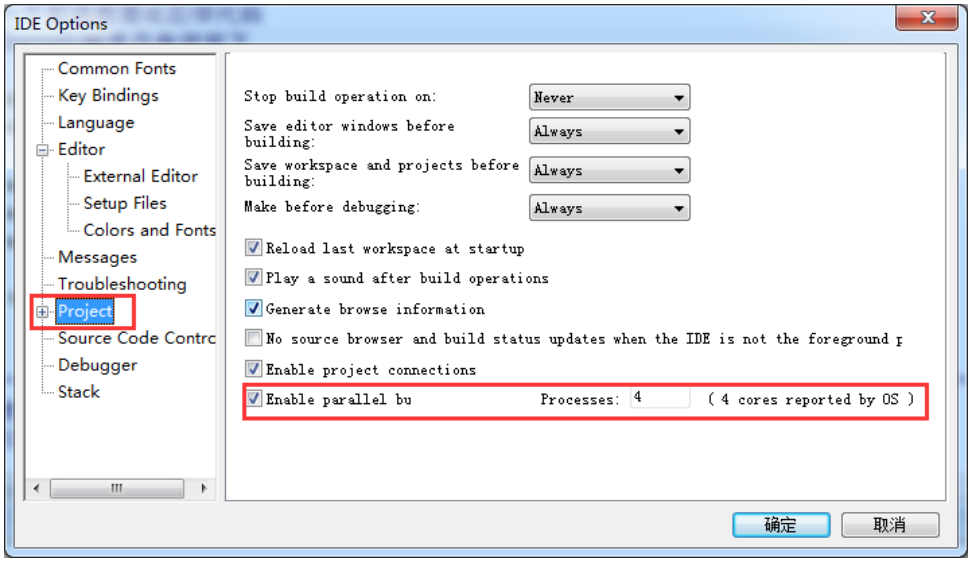
6.9.2 字体设置

菜单 Tools -> Options，支持比例字体和等宽字体选择：



6.9.3 多核并行编译

电脑有几个核都选上，加快编译速度，我这里是四核：



6.10 总结

对于初学者来说，务必要实际动手操作一遍，以后移植其它程序代码大有裨益。

第7章 STM32H7 下载和调试方法（IAR8）

本章教程为大家介绍如何创建 IAR8 工程，配置方法以及经常用到的一些配置项。

7.1 初学者重要提示
7.2 使用 IAR 调试和下载程序设置（JLINK）
7.3 使用 IAR 调试和下载程序设置（STLINK）
7.4 IAR 程序调试
7.5 总结

7.1 初学者重要提示

- 如果使用 JLINK 调试下载 STM32H7，务必使用 JLINK V9 或 V10，之前的版本已经不支持。
- 如果使用 STLINK 调试下载 STM32H7，推荐使用最新的电脑端驱动和对应的固件，详情见第 2 章的 2.6 小节。
- JLINK 无法下载解决思路以及常见问题整理，适用于其它 LINK：http://www.armbbs.cn/forum.php?mod=viewthread&tid=21708 。

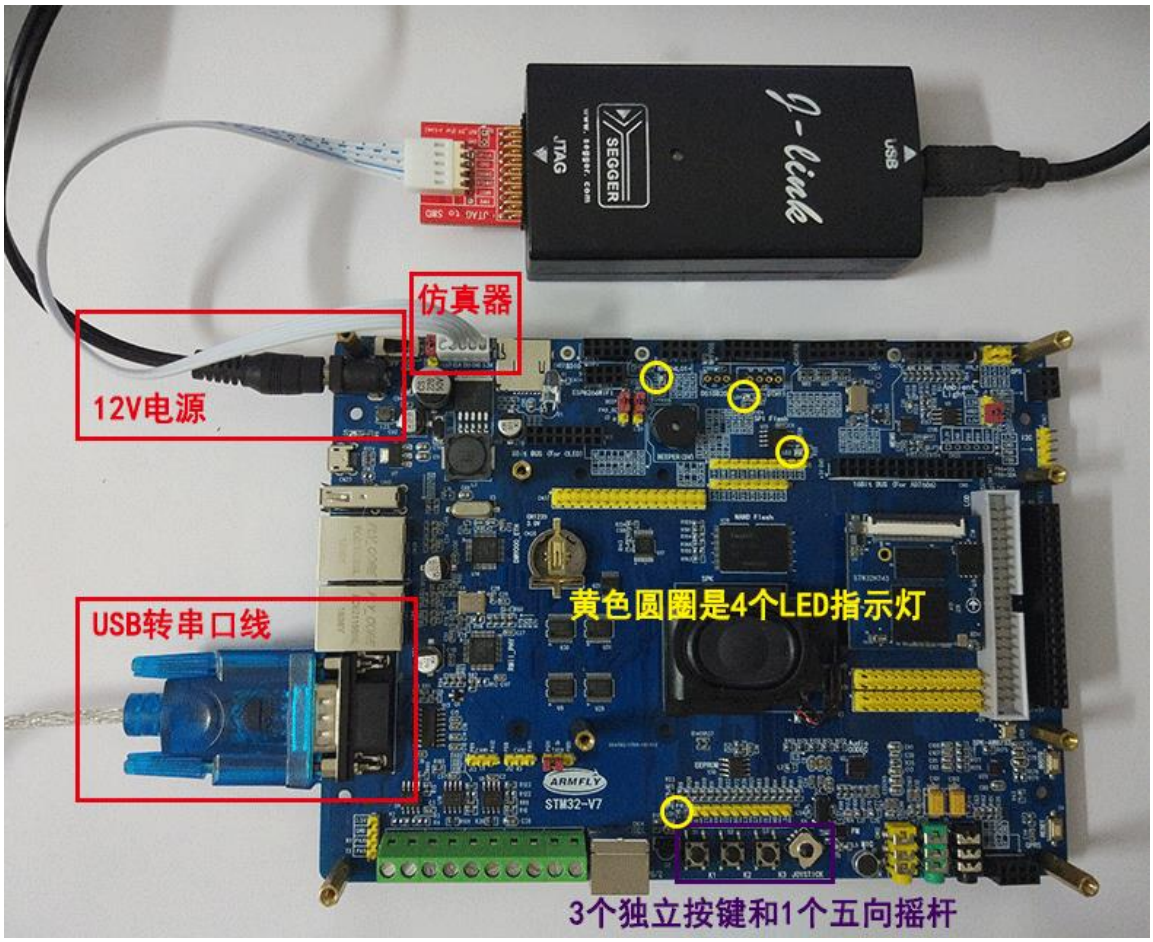
7.2 使用 IAR 调试和下载程序设置（JLINK）

调试下载 STM32H7，务必使用 JLINK V9 或 V10，之前的版本已经不支持，JLINK 的驱动安装等相关文件已经在本教程第 2 章的 2.5 章节有说明。

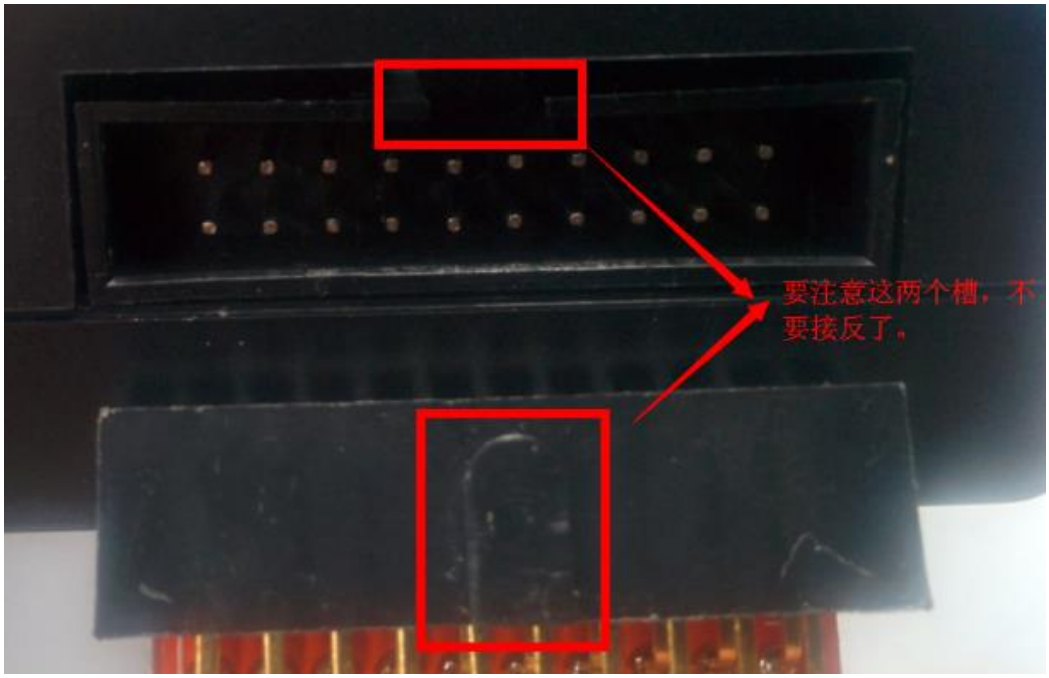
在上个章节里面，我们已经将 Options 对话框里面大部分设置选项都做了说明，本章节主要是调试下载的设置。

7.2.1 开发板和 JLINK 的接线图

开发板接上 JLINK 的效果图如下，JLINK 端需要插入开发板赠送的 JTAG 转 SWD 转接板：

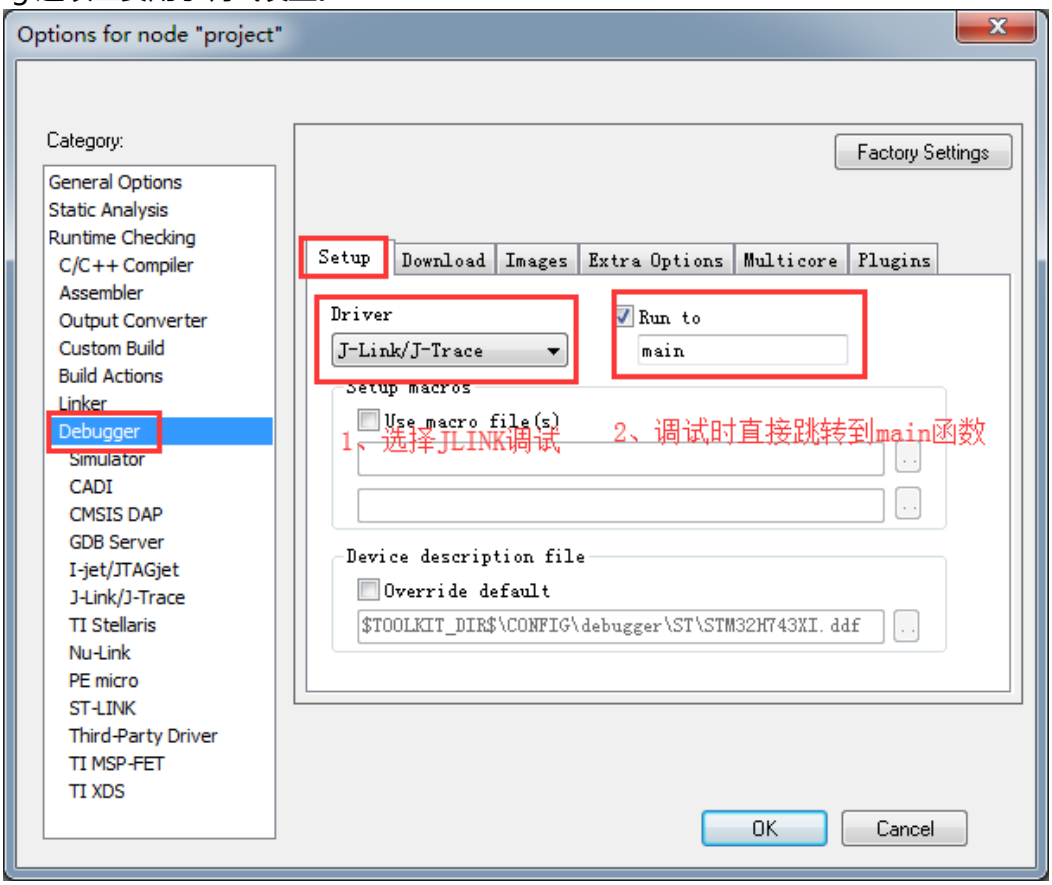

仿真器
12V电源
USB转串口线
黄色圆圈是4个LED指示灯
3个独立按键和1个五向摇杆

注意转接板不要插反了：


要注意这两个槽，不要接反了。

7.2.2 设置 Options 里面的 Degubber 选项

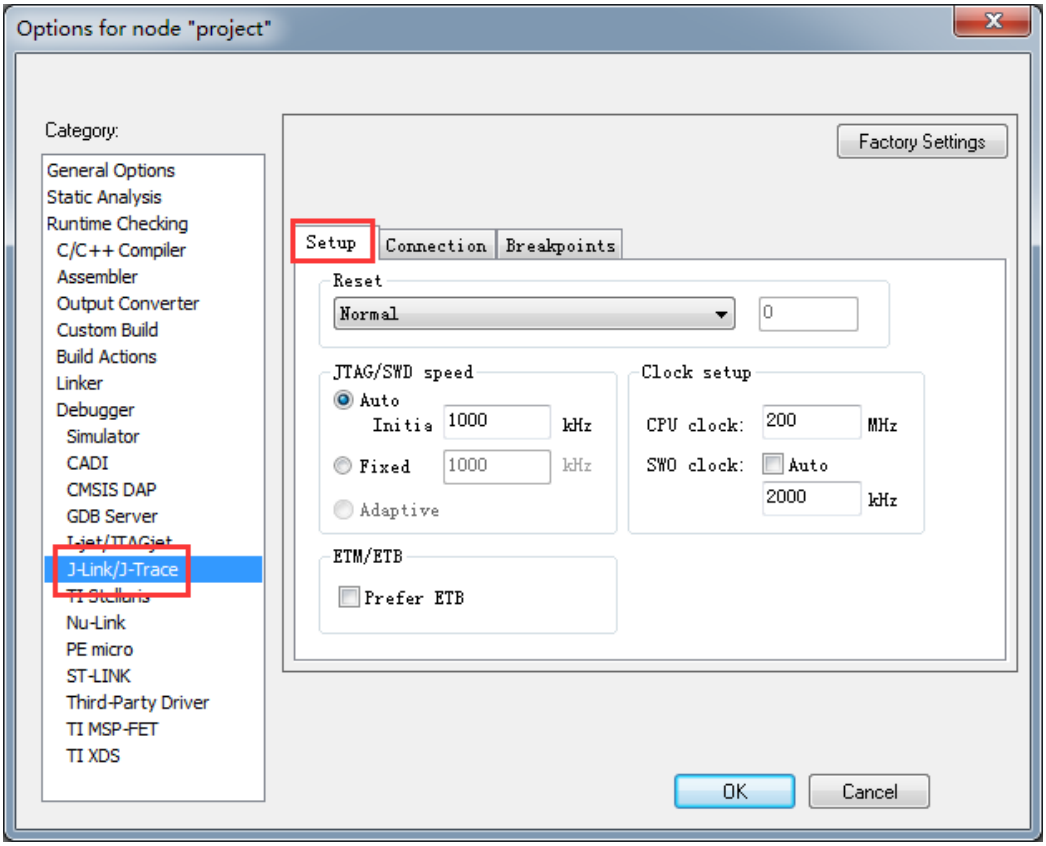
Debug 选项主要用于调试设置。



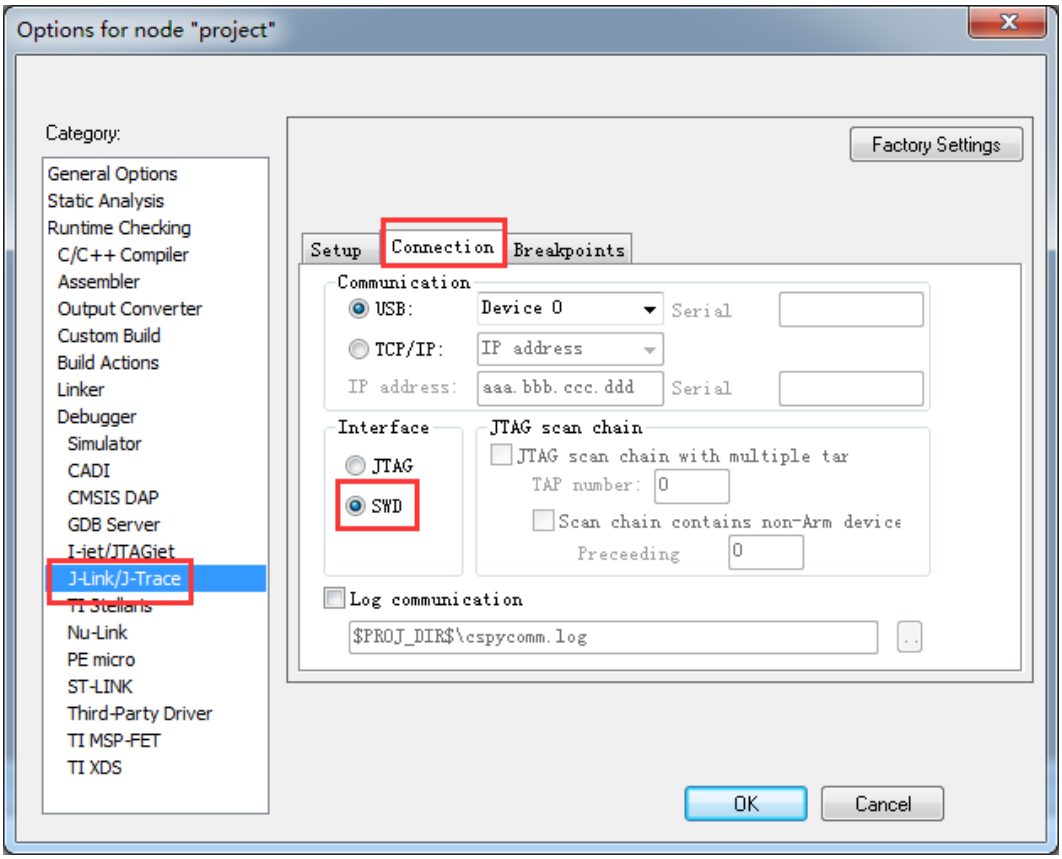
设置完毕后，别忘了点击 OK。

7.2.3 设置 Options 里面的 JLINK 选项

注意，在 Debugger 选择里面选择了 JLINK 后，才可以设置此选项。Setup 部分使用默认配置即可，无需修改：



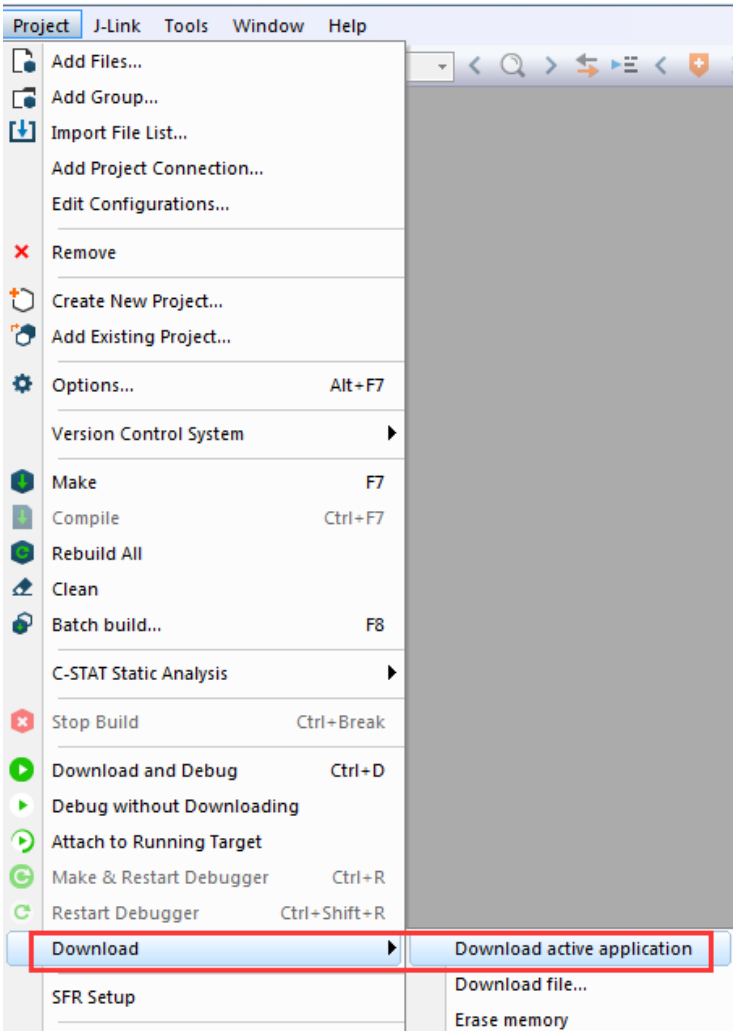
Connection 部分要选择 SWD 接口：



设置完毕后别忘了点击 OK。

7.2.4 程序下载

前两步设置好之后，JLINK 插到电脑端并跟开发板连接好，将板子上电，就可以下载了。菜单 Project->download->download Active Application：



下载成功后将开发板重新上电，可以看到开发板上的 LED 开始闪烁，IAR 的 Debug log 窗口也有信息展示：

```
Debug Log
Log
Wed Mar 20, 2019 14:15:47: ROMTbl[1][1]: E0001000, CID: B105E00D, PID: 000BB002 DWT
Wed Mar 20, 2019 14:15:47: ROMTbl[1][2]: E0002000, CID: B105E00D, PID: 000BB00E FPB-M7
Wed Mar 20, 2019 14:15:47: ROMTbl[1][3]: E0000000, CID: B105E00D, PID: 000BB001 ITM
Wed Mar 20, 2019 14:15:47: ROMTbl[0][1]: E0041000, CID: B105900D, PID: 001BB975 ETM-M7
Wed Mar 20, 2019 14:15:47: ROMTbl[0][2]: E0043000, CID: B105900D, PID: 004BB906 CTI
Wed Mar 20, 2019 14:15:47: Cache: Separate I- and D-cache.
Wed Mar 20, 2019 14:15:47: I-Cache L1: 16 KB, 256 Sets, 32 Bytes/Line, 2-Way
Wed Mar 20, 2019 14:15:47: D-Cache L1: 16 KB, 128 Sets, 32 Bytes/Line, 4-Way
Wed Mar 20, 2019 14:15:47: Reset: Halt core after reset via DEMCR.VC_CORERESET.
Wed Mar 20, 2019 14:15:48: Reset: Reset device via AIRCR.SYSRESETREQ.
Wed Mar 20, 2019 14:15:48: Hardware reset with strategy 0 was performed
Wed Mar 20, 2019 14:15:48: Initial reset was performed
Wed Mar 20, 2019 14:15:48: 770 bytes downloaded (8.09 Kbytes/sec)
Wed Mar 20, 2019 14:15:48: Loaded debugee: D:\Program Files (x86)\IAR Systems\Embedded Workbench 8.1\arm\config\flashloader\ST\FlashSTM32H7xxx_512kB.out
Wed Mar 20, 2019 14:15:48: Target reset
Wed Mar 20, 2019 14:15:49: Unloaded macro file: D:\Program Files (x86)\IAR Systems\Embedded Workbench 8.1\arm\config\flashloader\ST\FlashSTM32H7xxx.mac
Wed Mar 20, 2019 14:15:49: Downloaded C:\Users\byb\Desktop\LED\Project\EWARMv8\Flash\Exe\project.out to flash memory.
Wed Mar 20, 2019 14:15:50: IAR Embedded Workbench 8.30.1 (D:\Program Files (x86)\IAR Systems\Embedded Workbench 8.1\arm\bin\armproc.dll)
```

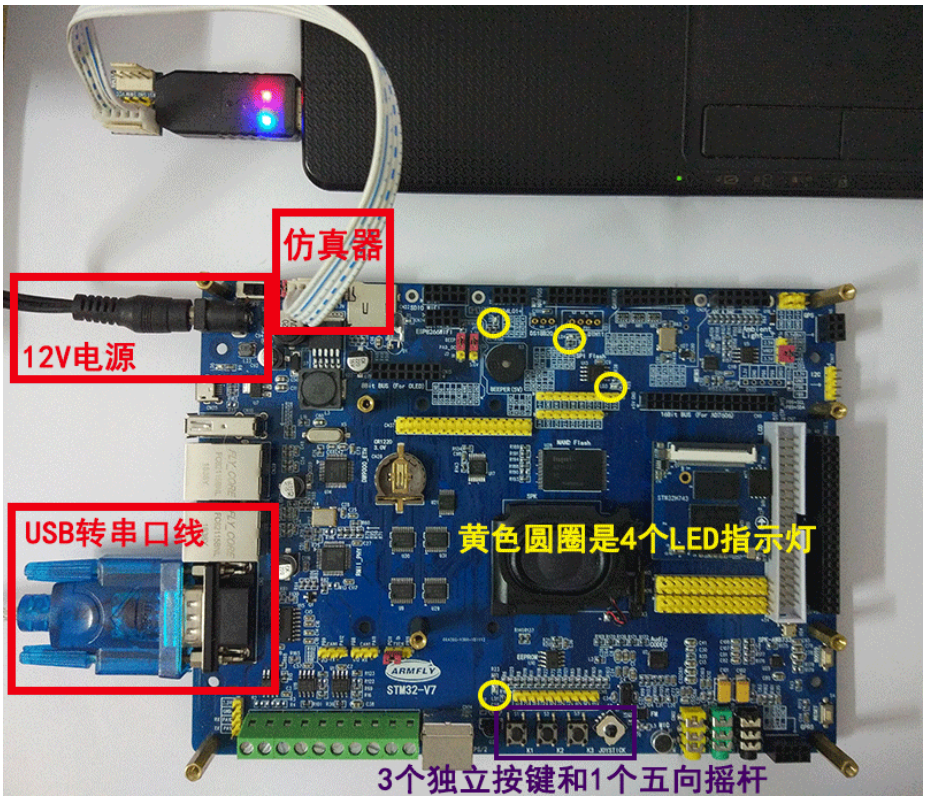
7.3 使用 IAR 调试和下载程序设置（STLINK）

调试下载 STM32H7，请将 STLINK 固件升级到最新版，关于固件升级和驱动安装等相关文件已经在本教程第 2 章的 2.6 章节有说明。

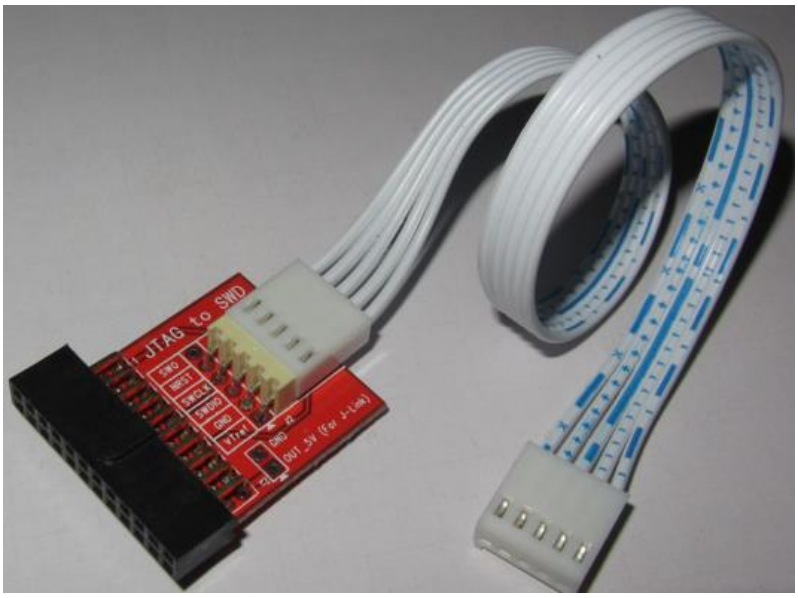
在上个章节里面，我们已经将 Options 对话框里面大部分设置选项都做了说明，本章节主要是调试下载的设置。

7.3.1 开发板和 STLINK 的接线图

开发板接上 STLINK 的效果图如下：

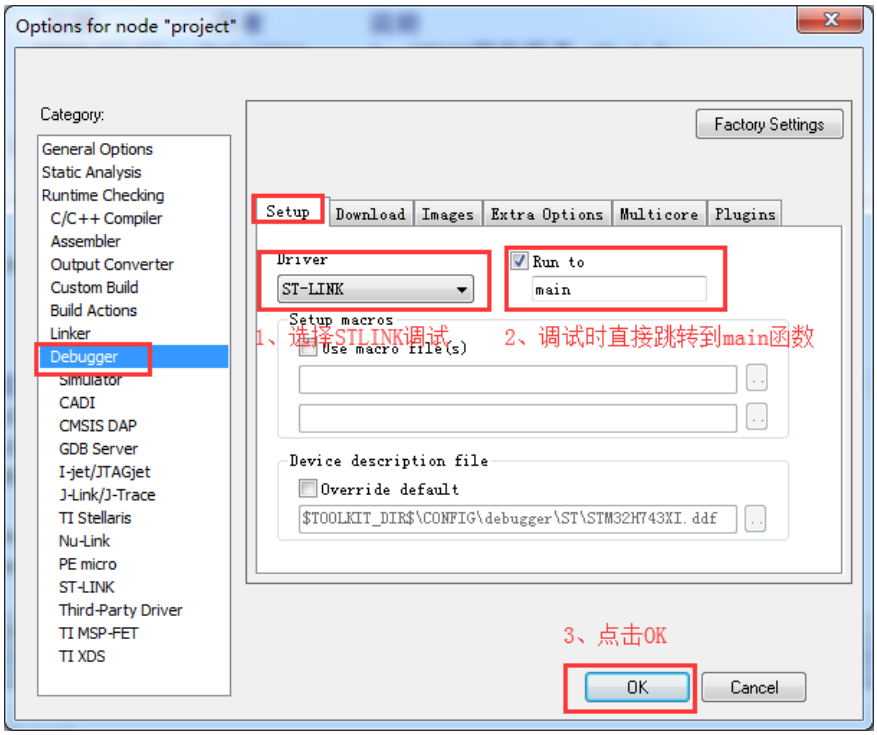


如果是下面这种 STLINK，需要接上转接板：



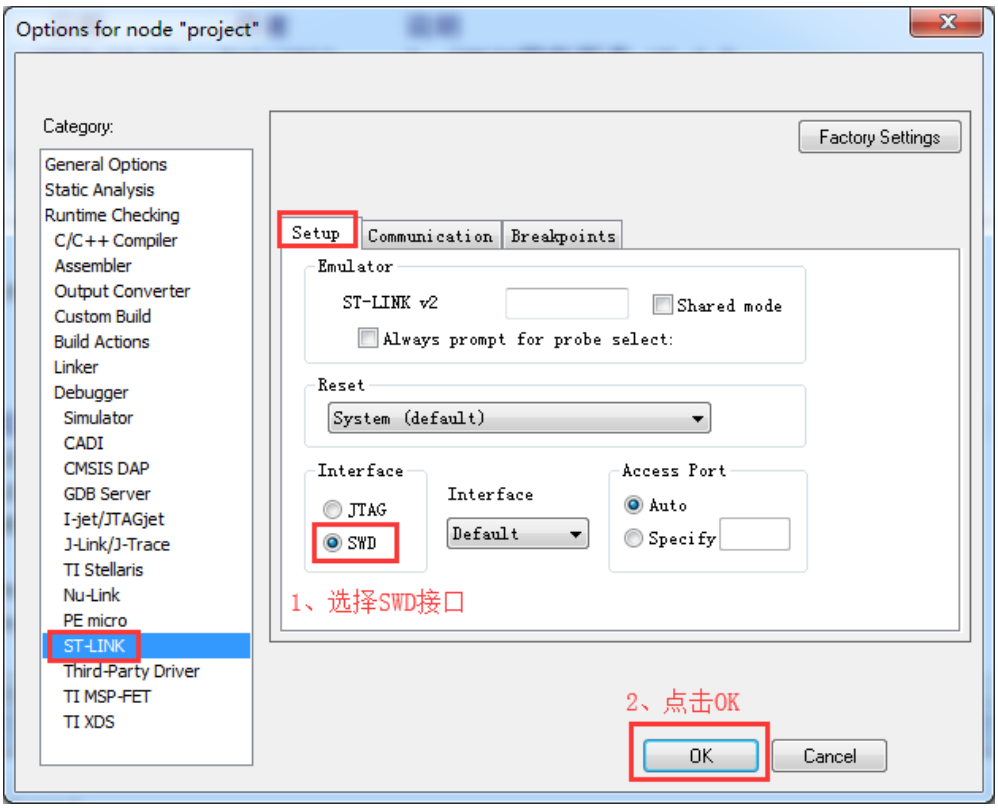
7.3.2 设置 Options 里面的 Debug 选项

Debug 选项主要用调试：



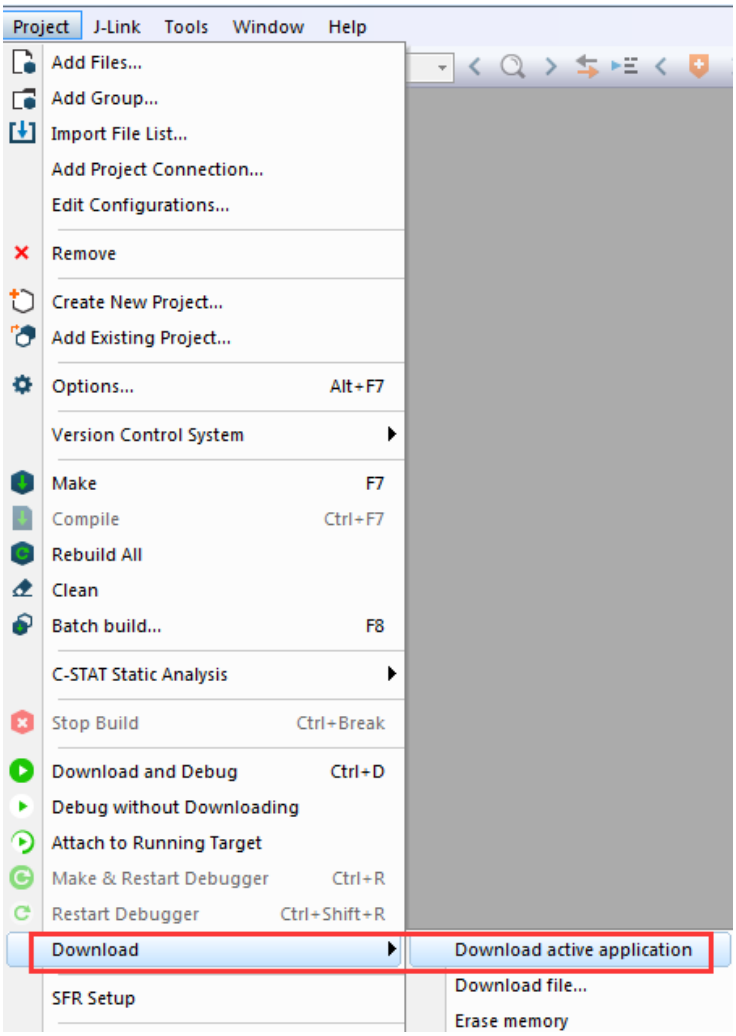
7.3.3 设置 Options 里面的 STLINK 选项

注意，在 Debugger 选择里面选择了 STLINK 后，才可以设置此选项。Setup 部分选择 SWD 接口，其它默认即可：

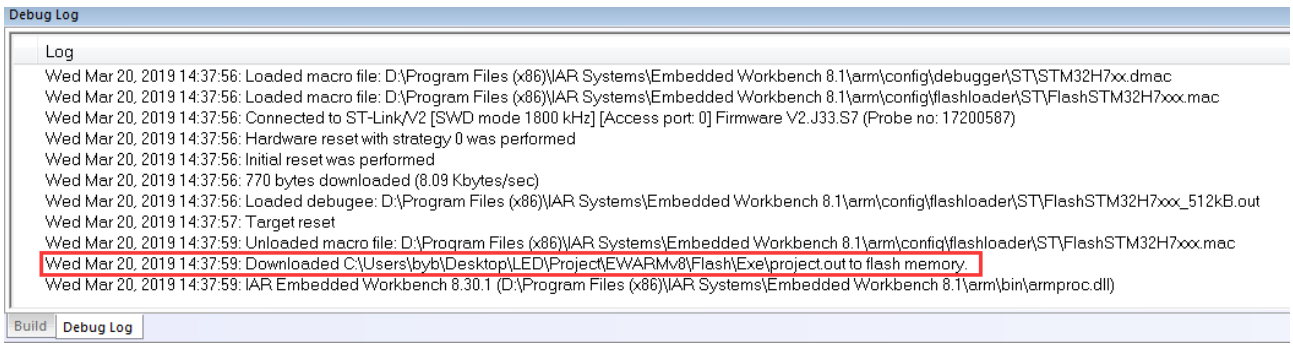


7.3.4 程序下载

前两步设置好之后，STLINK 插到电脑端并跟开发板连接好，将板子上电，就可以下载了。点击这里即可：



下载成功后将开发板重新上电，可以看到开发板上的 LED 开始闪烁，IAR 的 Debug log 窗口也有信息展示。

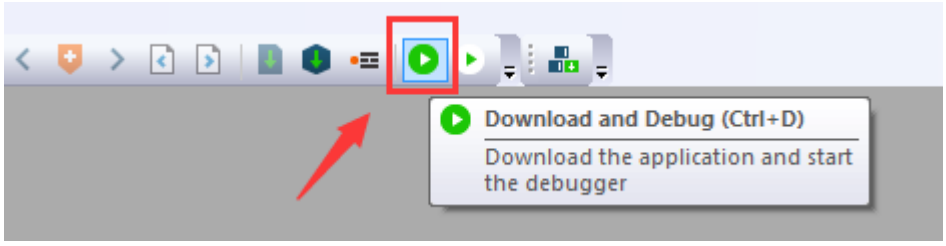


7.4 IAR 程序调试

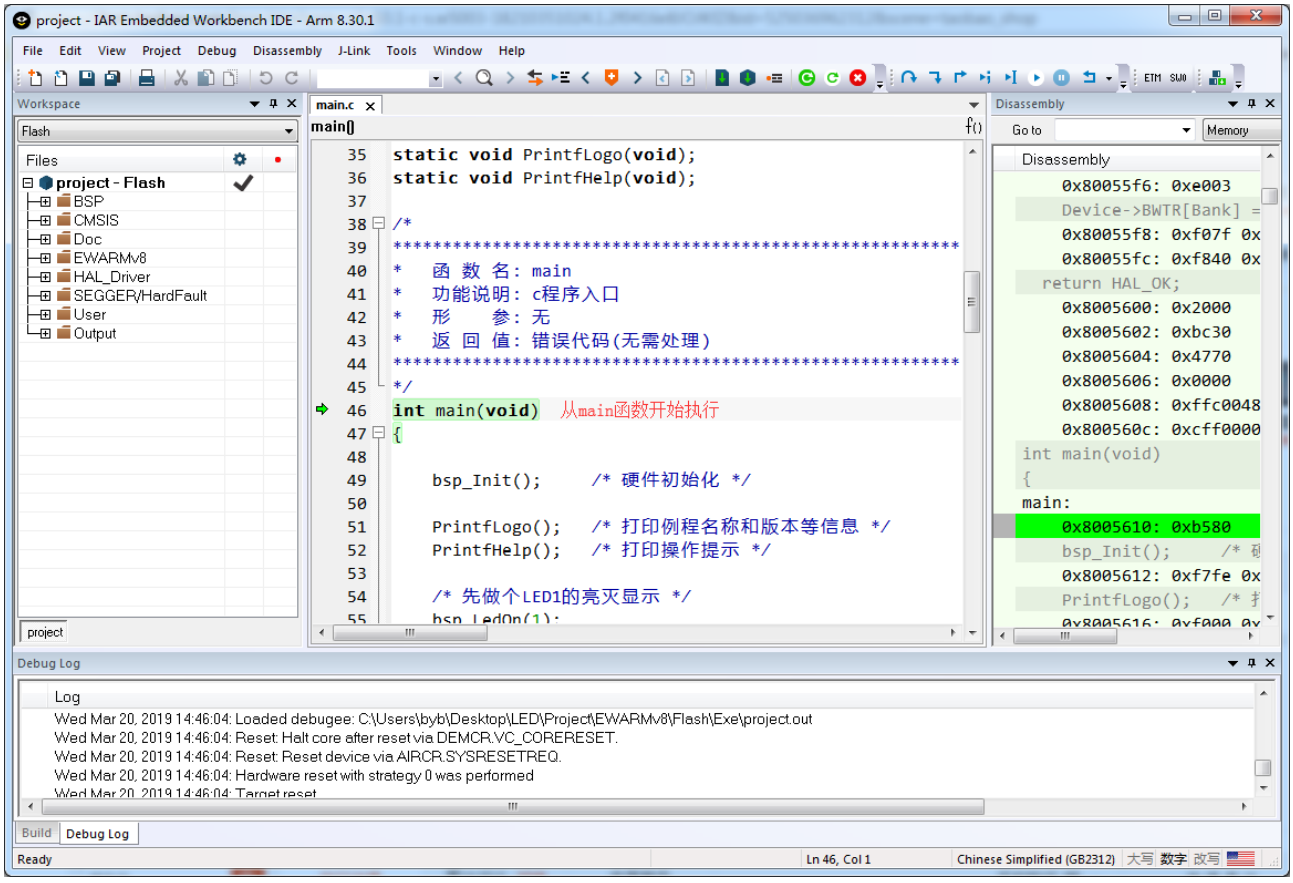
JLINK 或者 STLINK 设置完毕后，调试是一样的。

7.4.1 全速运行和单步调试

点击如下按钮可以进入调试状态（下载程序到芯片同时开启调试）：



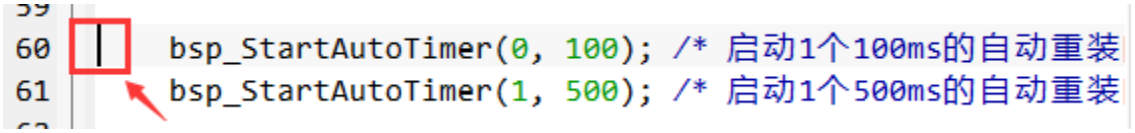
进入调试状态后的效果如下：



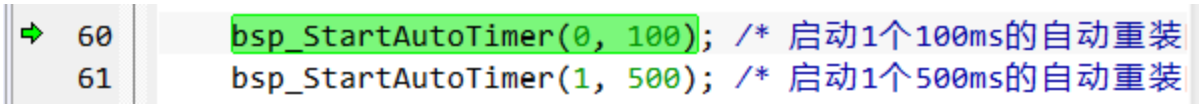
首先要认识下这 11 个图标的含义（进入调试状态后，鼠标放在这几个图标上会有提示）：

- 第 1 个图标是重新编译例程并进入调试状态。
- 第 2 个图标是重新进入调试状态。

- 第 3 个图标是停止调试。
- 第 4 个图标是 Step Over（Step over the current step point），每次点击以函数为单位运行，不会进入函数内部。
- 第 5 个图标是 Step Into（Step into the current step point），单步调试，每次点击运行一行代码，所以只要有子函数就会进入。
- 第 6 个图标是 Step out（Step out of the current line），点击后退出当前运行的函数，执行下一个函数。
- 第 7 个图标是 Next Statement（Step to the next statement），进入下一个状态，跟第 4 个图标功能差不多。
- 第 8 个图标是 Run to Cursor（Execute to the current cursor position），直接运行到用户设置的行，设置方法比较简单，用户仅需在那行使用鼠标左键点击一下，可以看到前面有光标即可（注意，程序不能倒着执行，也就是能往回设置）



然后点击此图标即可，可以看到前面多了一个绿色的小箭头，说明已经运行到此行

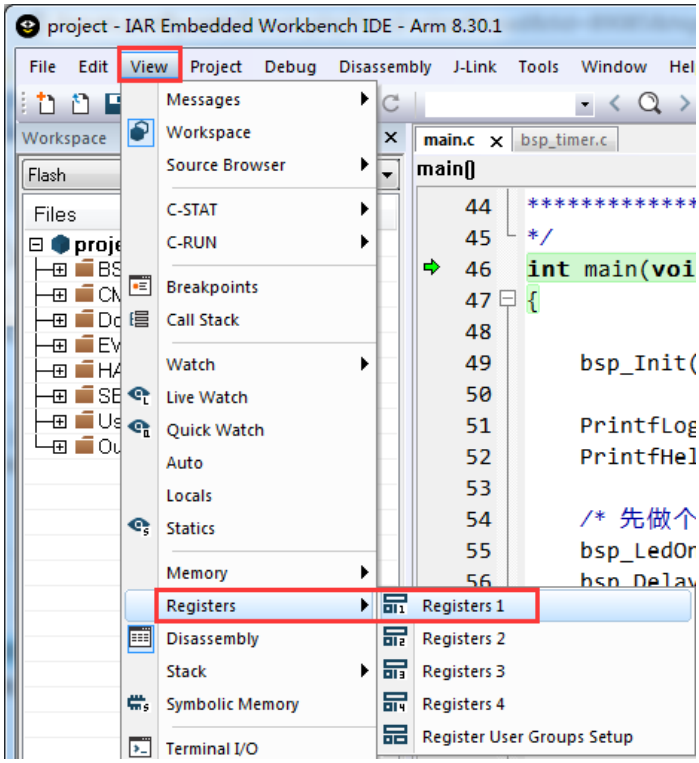


- 第 9 个图标是全速运行。
- 第 10 个图标是用于暂停全速运行。
- 第 11 个图标是复位调试，重新回到 main 函数开始执行。

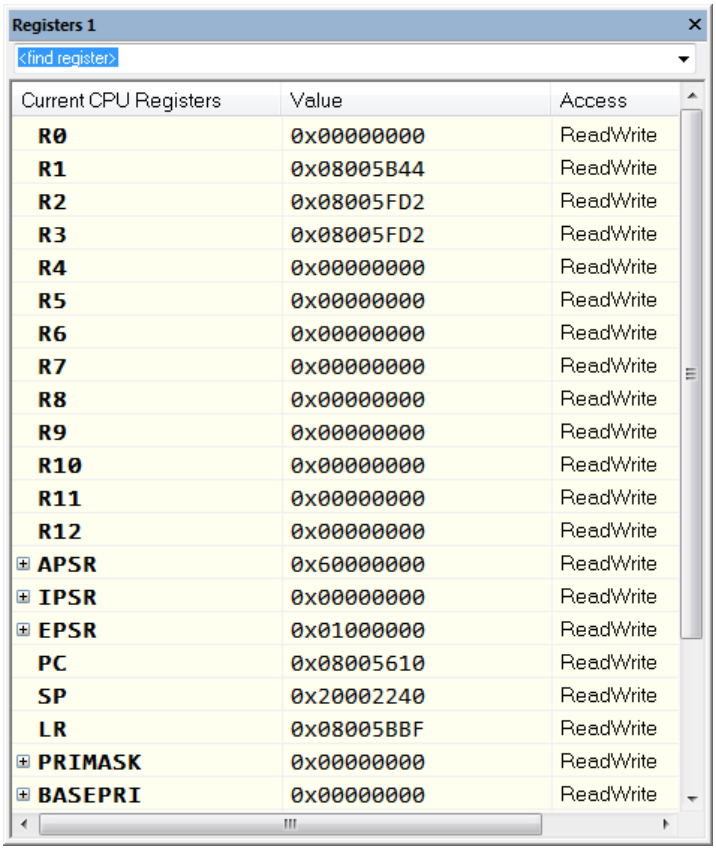
前面几个图标的功能，大家多测试几次就熟练了。

7.4.2 外设寄存器的查看

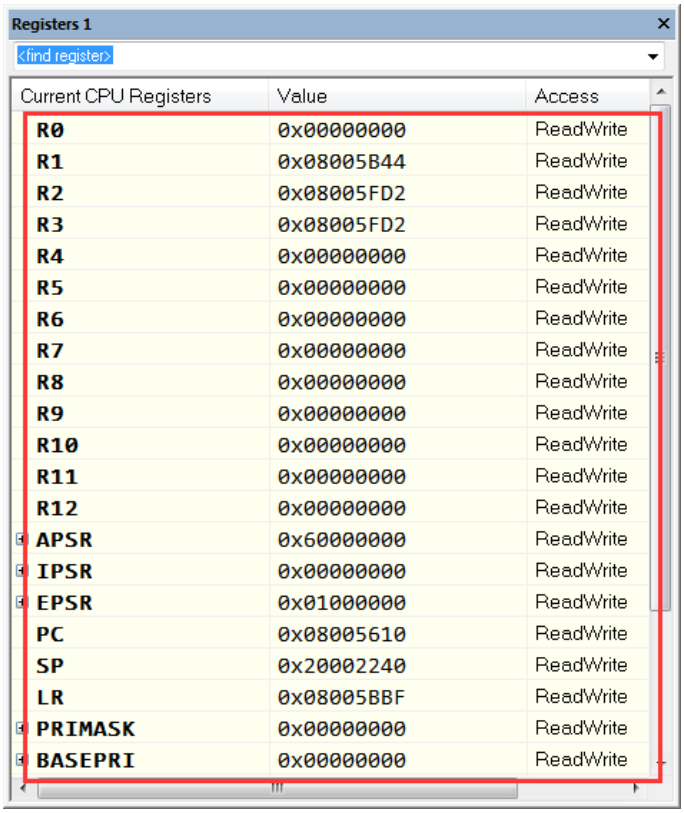
初次使用 IAR8 的话，容易出现找不到在哪里查看寄存器的问题，首先需要点击这里 View ->Register ->Register 1（共提供了四个 register 窗口，打开那个都行，要查看多个外设时，可以同时打开多个窗口查看）：

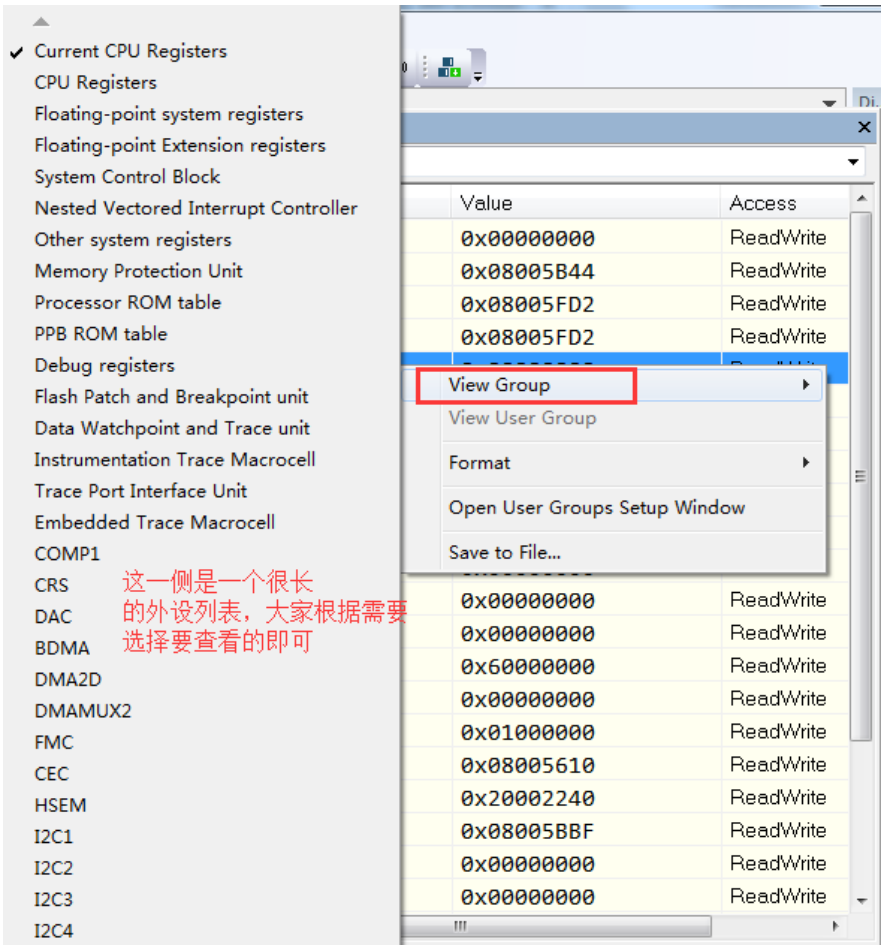


给出的都是 CPU 寄存器：

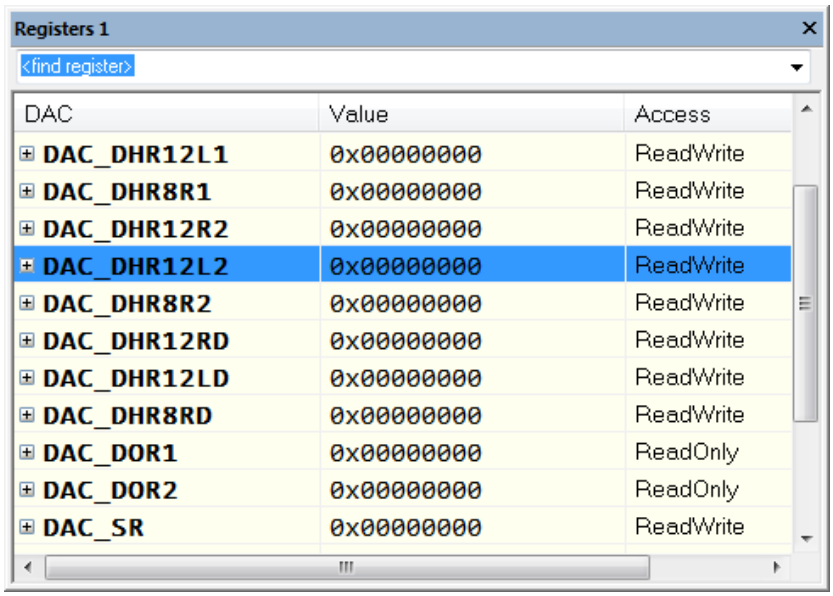


这个并不是我们想要的，还有继续下一步，在红色的方框区域右击：


Registers 1
<find register>
Current CPU Registers
Value
Access
R0 0x00000000 ReadWrite
R1 0x08005B44 ReadWrite
R2 0x08005FD2 ReadWrite
R3 0x08005FD2 ReadWrite
R4 0x00000000 ReadWrite
R5 0x00000000 ReadWrite
R6 0x00000000 ReadWrite
R7 0x00000000 ReadWrite
R8 0x00000000 ReadWrite
R9 0x00000000 ReadWrite
R10 0x00000000 ReadWrite
R11 0x00000000 ReadWrite
R12 0x00000000 ReadWrite
APSR 0x60000000 ReadWrite
IPSR 0x00000000 ReadWrite
EPSR 0x01000000 ReadWrite
PC 0x08005610 ReadWrite
SP 0x20002240 ReadWrite
LR 0x08005BBF ReadWrite
PRIMASK 0x00000000 ReadWrite
BASEPRI 0x00000000 ReadWrite

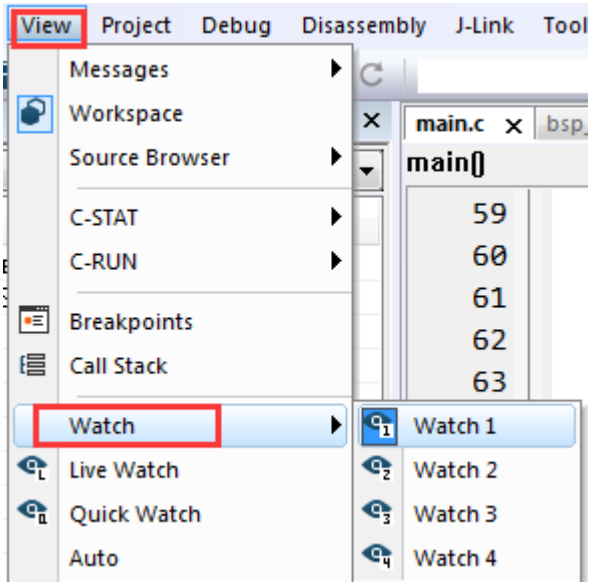

Current CPU Registers
CPU Registers
Floating-point system registers
Floating-point Extension registers
System Control Block
Nested Vectored Interrupt Controller
Other system registers
Memory Protection Unit
Processor ROM table
PPB ROM table
Debug registers
Flash Patch and Breakpoint unit
Data Watchpoint and Trace unit
Instrumentation Trace Macrocell
Trace Port Interface Unit
Embedded Trace Macrocell
COMP1
CRS
DAC
BDMA
DMA2D
DMAMUX2
FMC
CEC
HSEM
I2C1
I2C2
I2C3
I2C4
这一侧是一个很长的外设列表，大家根据需要选择要查看的即可
Value
Access
0x00000000 ReadWrite
0x08005B44 ReadWrite
0x08005FD2 ReadWrite
0x08005FD2 ReadWrite
View Group
View User Group
Format
Open User Groups Setup Window
Save to File...
0x00000000 ReadWrite
0x00000000 ReadWrite
0x60000000 ReadWrite
0x00000000 ReadWrite
0x01000000 ReadWrite
0x08005610 ReadWrite
0x20002240 ReadWrite
0x08005BBF ReadWrite
0x00000000 ReadWrite
0x00000000 ReadWrite

比如查看 DAC：

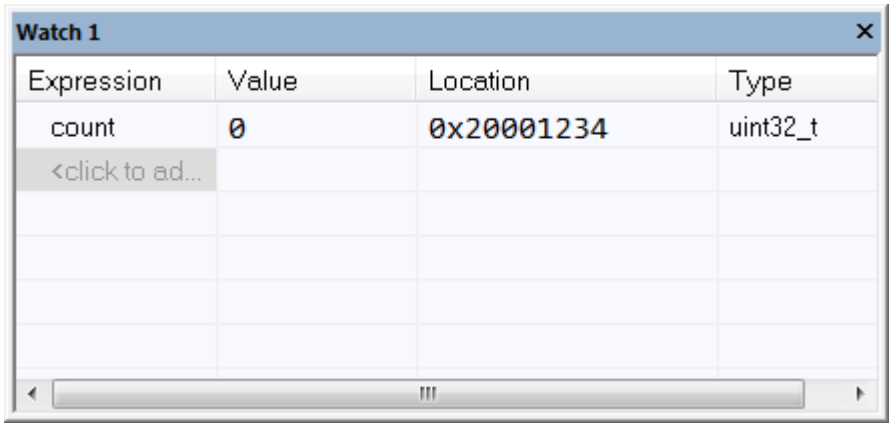


7.4.3 全局变量的查看

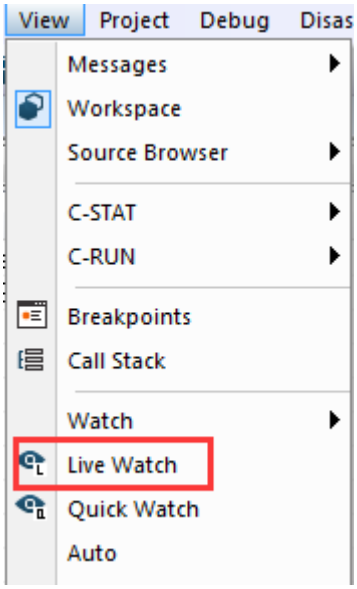
针对全局变量，IAR 提供了专门提供了 Watch1， Watch2，Watch3 和 Watch4 窗口，每个窗口可以添加很多变量。从菜单 View ->Watch 进行添加：



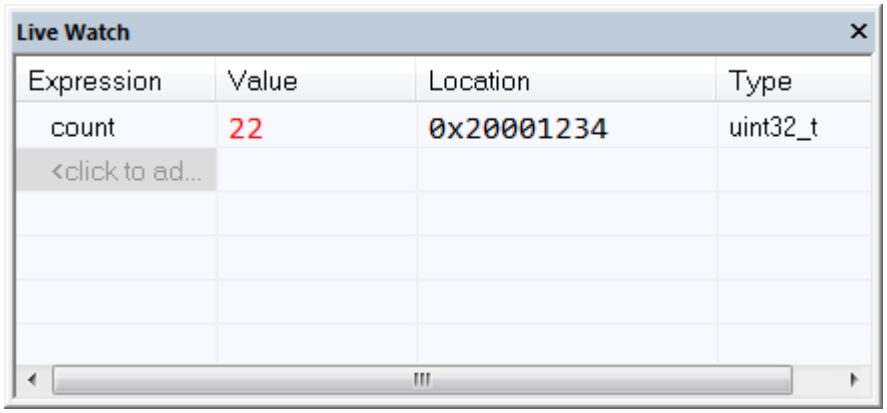
比如定义了一个全局的 32 位变量 count，用户可以直接在 Watch1 窗口里面输入此变量：



也可以在源文件里面，鼠标右击此变量进行添加。调试过程中，这些窗口里面添加的变量是会更新的。但是全速运行状态，这些变量就不会更新，为了解决这个问题，IAR 有一个 Live Watch 可以在全速运行状态查看变化：

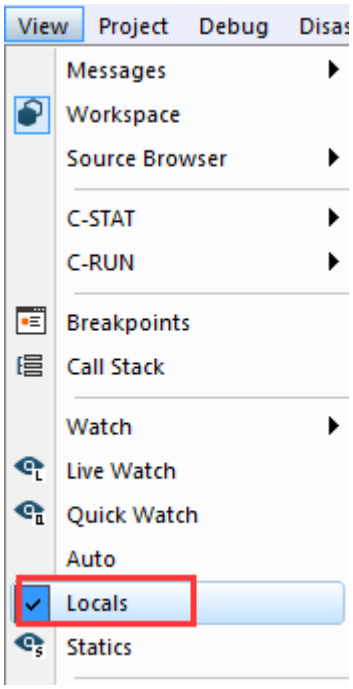


用户可以直接在 Live Watch 窗口里面输入变量查看，如果程序里面做了一个简单的加法计数，就可以在全速调试状态查看此变量的动态变化：

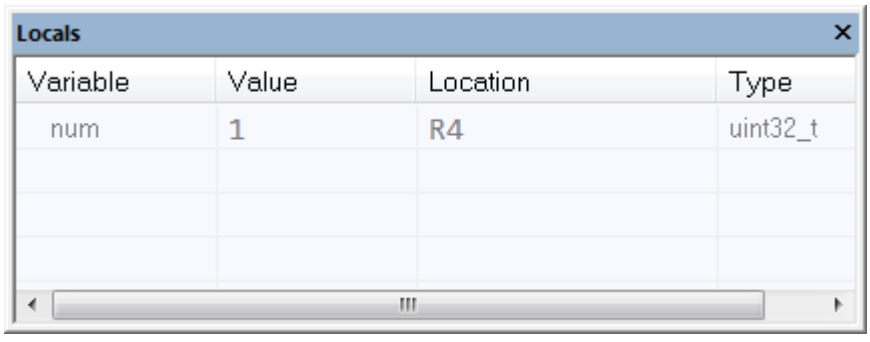


7.4.4 局部变量的查看

针对局部变量，IAR 专门提供了一个栈窗口，用于查看函数中各种局部变量，这个不需要用户添加，IAR 会自动添加。从菜单 View ->Locals 进行添加窗口：

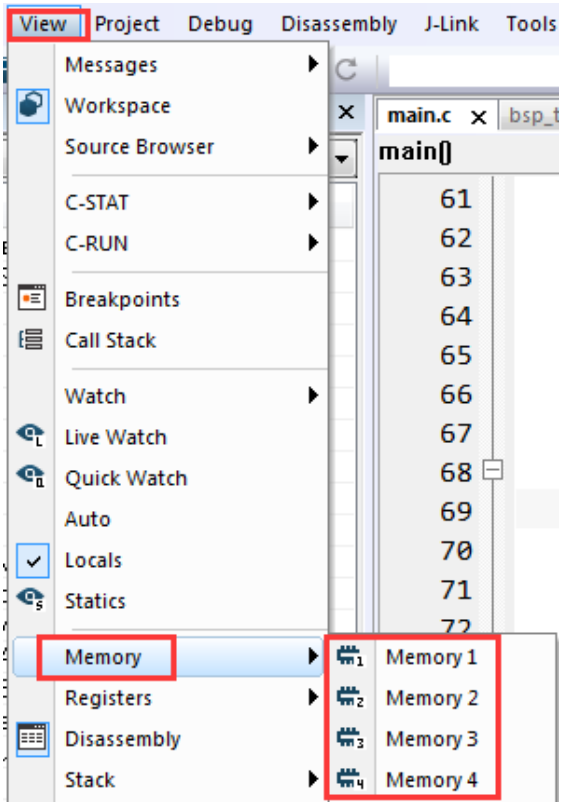


局部变量需要进入到那个函数里面才可以查看，这点要特别注意，因为局部变量是来自栈空间的，进入函数申请栈空间，函数退出后，栈空间就会被释放掉。比如 main 函数里面一个 num 局部变量

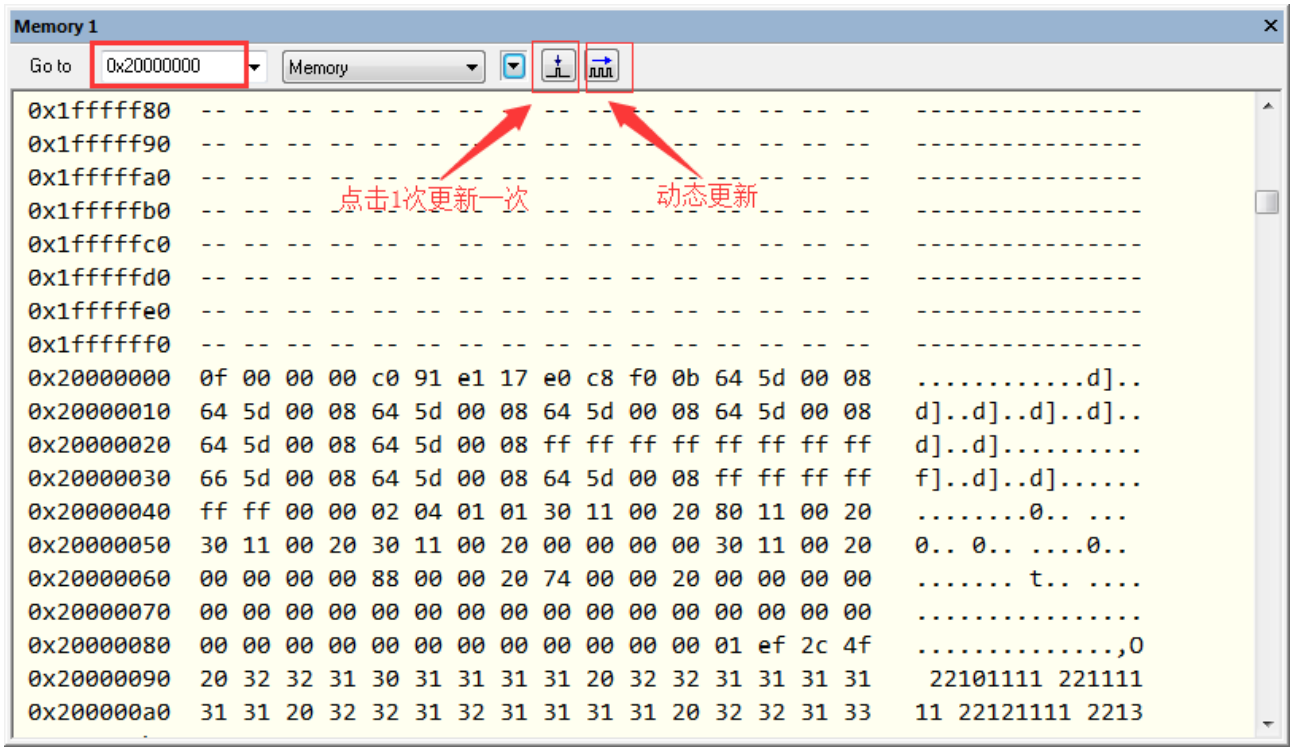


7.4.5 Flash 区或者 RAM 区的查看

如果要查看 Flash 区或者 RAM 区的内容，IAR 提供了 4 个 Memory 窗口。从菜单 View ->Memory 进行添加：

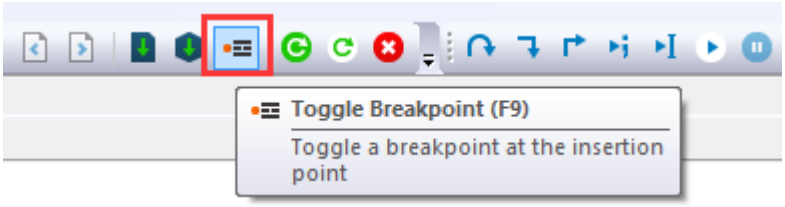


比如要查看 0x2000 0000 地址开始的内容，直接输入即可，这个地址是 STM32H7 的 DTCM 首地址



7.4.6 断点调试

断点调试比较重要，进入调整状态后，如果大家想让程序在指定的位置停止运行，可以添加断点，通过这个小红点来添加（注意，取消断点也是用的这个图标）。



比如想在 79 行添加断点。

```
75          /* 判断定时器超时时间 */
76          if (bsp_CheckTimer(1))
77          {
78              /* 每隔500ms 进来一次 */
79              bsp_LedToggle(2);
80              bsp_LedToggle(3);
81              bsp_LedToggle(4);
82          }
```

用户仅需在那行使用鼠标左键点击一下，然后单击断点图标即可，断点设置好的效果如下：

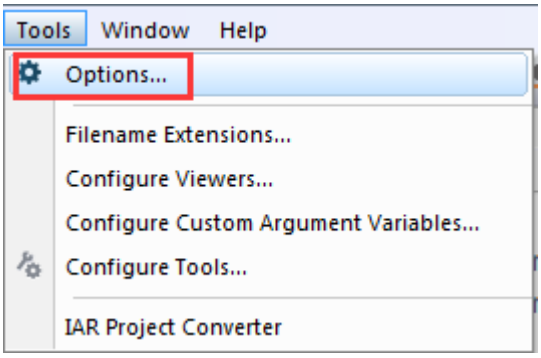
```
75          /* 判断定时器超时时间 */
76          if (bsp_CheckTimer(1))
77          {
78              /* 每隔500ms 进来一次 */
79              bsp_LedToggle(2);
80              bsp_LedToggle(3);
81              bsp_LedToggle(4);
82          }
```

然后点击全速运行，如果程序有执行到这个地方，就会停在这里。可以看到绿色小箭头也显示在左侧了，说明程序执行到这个地方了（此时还没有执行该语句）：

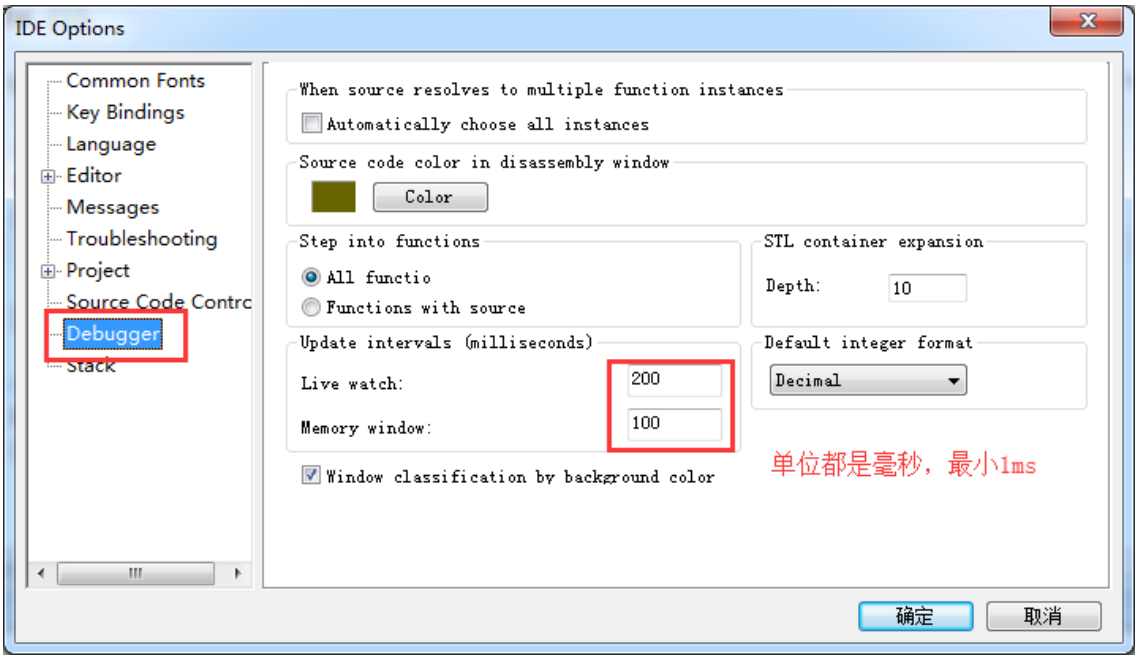
```
75          /* 判断定时器超时时间 */
76          if (bsp_CheckTimer(1))
77          {
78              /* 每隔500ms 进来一次 */
79              bsp_LedToggle(2);
80              bsp_LedToggle(3);
81              bsp_LedToggle(4);
82          }
```

7.4.7 动态更新频率设置

IAR 有个周期性更新频率设置选项，菜单 Tools->Options：



单位都是毫秒，最小 1ms：



7.5 总结

对于初学者来说，务必要实际动手操作一遍。特别是调试方法，要熟练掌握，因为以后工程代码调试时，用到的最多。

第8章 STM32H7 的终极调试组件 Event Recorder

本章节为大家介绍终极调试方案 Event Recoder，之所以叫终极解决方案，是因为所有 Link 通吃，支持时间测量，功耗测量，printf 打印，RTX5 及其所有中间件调试信息展示。

8.1 重要提示（必读）
8.2 Event Recorder 简介
8.3 创建工程模板和注意事项
8.4 Event Recorder 事件记录的实现
8.5 Event Recoder 实现 printf 重定向
8.6 Event Statistics 时间测量功能的实现
8.7 Event Statistics 功耗测量功能的实现
8.8 Event Recoder 对 RTX5 及其所有中间件的支持
8.9 JLINK 配置说明
8.10 STLINK 配置说明
8.11 CMSIS-DAP 配置说明
8.12 ULINK 配置说明
8.13 配套例子
8.14 总结

8.1 重要提示（必读）

- 只要是 MDK 支持的调试下载器，基本都支持 Event Recorder，本教程测试了 JLINK，STLINK 和 CMSIS-DAP。
- 务必使用 MDK5.25 及其以上版本。
- 使用 ARM_Compiler 软件包 V1.4.0 及其以上版本。详情看此贴：http://www.armbbs.cn/forum.php?mod=viewthread&tid=87175 。
- CMSIS 软件包 要是使用 V5.3.0 及其以上版本，详情本教程 8.3 小节末尾的说明。
- 如果大家的 MDK5.X 应用不是很熟练的话，可以看论坛网友翻译的 MDK5.X 入门手册：http://www.armbbs.cn/forum.php?mod=viewthread&tid=31288。

如果觉得看手册上手慢的话，可以直接看 KEIL 官方做的 MDK 入门系列视频，带中文字幕：http://www.armbbs.cn/forum.php?mod=viewthread&tid=82667 。

- 为了实现 Event Recorder 组件的最高性能，最好将下载器的时钟速度设置到所支持的最大值，另外，根据需要加大 EventRecorderConf.h 文件中的缓冲大小，默认可以缓冲 64 个消息 (动态更新的 FIFO 空间)。
- 此调试组件不需要用到 SWO 引脚，使用标准的下载接口即可。以我们的开发板为例，用到 VCC，GND，SWDIO，SWCLK 和 NRST。大家使用三线 JLINK-OB 也是没问题的，仅需用到 GND，SWDIO 和 SWCLK。

8.2 Event Recorder 简介

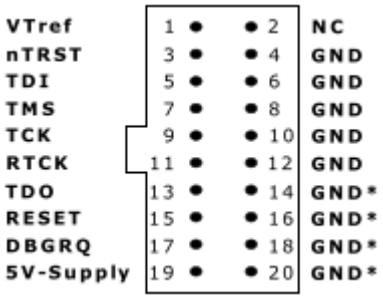
前面的专题教程中为大家讲解了使用 SEGGER 的 RTT 功能来替代串口打印，比较方便。只是这种方法限制用户必须使用 JLINK 才可以。而使用 Event Recorder 的话，无此限制，各种 LINK 通吃。只要是 MDK 支持的即可。

Event Recorder 是 MDK 在 5.22 版本的时增加的功能，到了 5.25 版本后，这个功能就更加完善了，增加了时间测量和功耗测量的功能。

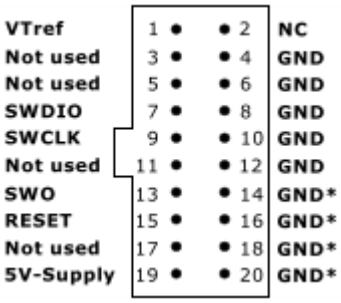
此调试组件不需要用到 SWO 引脚，使用标准的下载接口即可。以我们的开发板为例，用到 VCC，GND，SWDIO，SWCLK 和 NRST。大家使用三线 JLINK-OB 也是没问题的，仅需用到 GND，SWDIO 和 SWCLK。

- JTAG 接口和 SWD 接口区别

下图分别是 20pin 的标准 JTAG 引脚和 SWD（Serial Wire Debug）引脚，一般 SWD 接口仅需要 Vref，SWDIO，SWCLK，RESET 和 GND 五个引脚即可，SWO（Serial Wire Output）引脚是可选的。有了 SWO 引脚才可以实现数据从芯片到电脑端的数据发送。



JTAG 接口



SWD 接口

- 词条 SWV（Serial Wire Viewer）

SWV 是由仪器化跟踪宏单元 ITM(Instrumentation Trace Macrocell)和 SWO 构成的。SWV 实现了一种从 MCU 内部获取信息的低成本方案，SWO 接口支持输出两种格式的跟踪数据，但是任意时刻只能使用一种。两种格式的数据编码分别是 UART（串行）和 Manchester（曼彻斯特）。当前 JLINK 仅支持 UART 编码，SWO 引脚可以根据不同的信息发送不同的数据包。当前 M3/M4 可以通过 SWO 引脚输出以下三种

信息：

1. ITM 支持 printf 函数的 debug 调用（工程需要做一下接口重定向即可）。ITM 有 32 个通道，如果使用 MDK 的话，通道 0 用于输出调试字符或者实现 printf 函数，通道 31 用于 Event Viewer，这就是为什么实现 Event Viewer 需要配置 SWV 的原因。
2. 数据观察点和跟踪 DWT(Data Watchpoint and Trace)可用于变量的实时监测和 PC 程序计数器采样。
3. ITM 还附带了一个时间戳的功能：当一个新的跟踪数据包进入了 ITM 的 FIFO 时，ITM 就会把一个差分的时间戳数据包插入到跟踪数据流中。跟踪捕获设备在得到了这些时间戳后，就可以找出各跟踪数据之间的时间相关信息。另外，在时间戳计数器溢出时也会发送时间戳数据包。

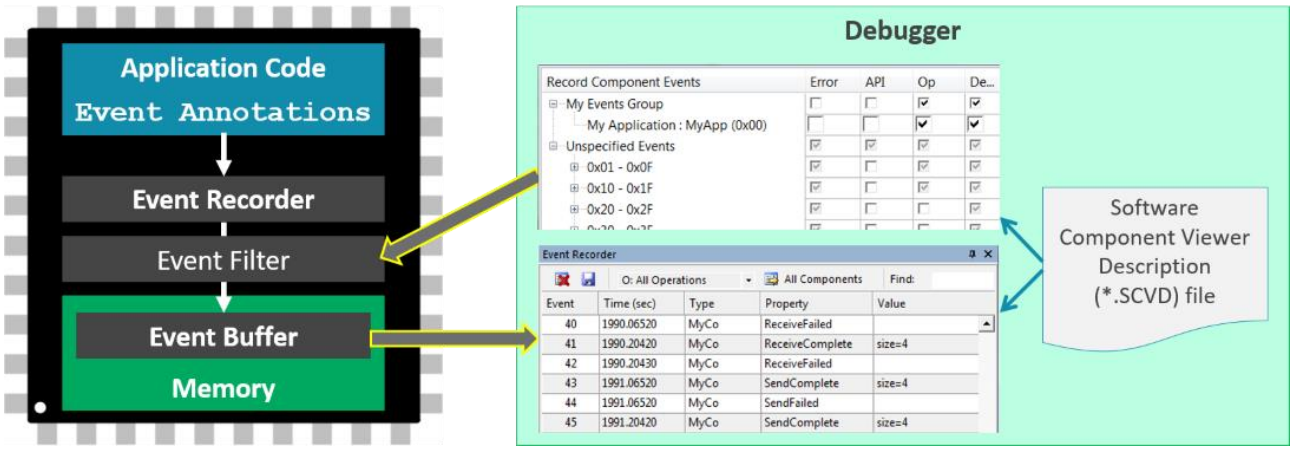
8.2.1 Event Recorder 的特色

Event Recorder 的特色主要有以下几点：

1. 提升应用程序动态执行期间的检测能力。
2. 支持的事件类型滤除机制，比如运行错误、API 调用、内部操作和操作信息的区分。
3. 可以在任务中、RTOS 内核中和中断服务程序中任意调用。
4. 对于带 ITM 功能的 Cortex-M3/M4/M7/M33 内核芯片，执行记录期间，全程无需开关中断操作。对于不带 ITM 功能的 Cortex-M0/M0+/M23，是需要开关中断的。
5. 支持 printf 重定向。
6. 各种 link 通吃，支持 SWD 接口或者 JTAG 接口方式的 JLINK、STLINK、ULINK 和 CMSIS-DAP。
7. 对于带 DWT 时钟周期计数器功能的 Cortex-M3/M4/M7/M33 内核芯片，创建时间戳时，可以有效降低系统负担，无需专用定时器来实现。
8. Event Recorder 执行时间具有时间确定性，即执行的时间是确定的，而且执行速度超快，因此，实际产品中的代码依然可以带有这部分，无需创建 debug 和 release 两种版本。
9. RTX5 及其所有中间件都支持 Event Recorder 调试。

8.2.2 Event Recorder 是如何工作的

首先来看下面这张图：

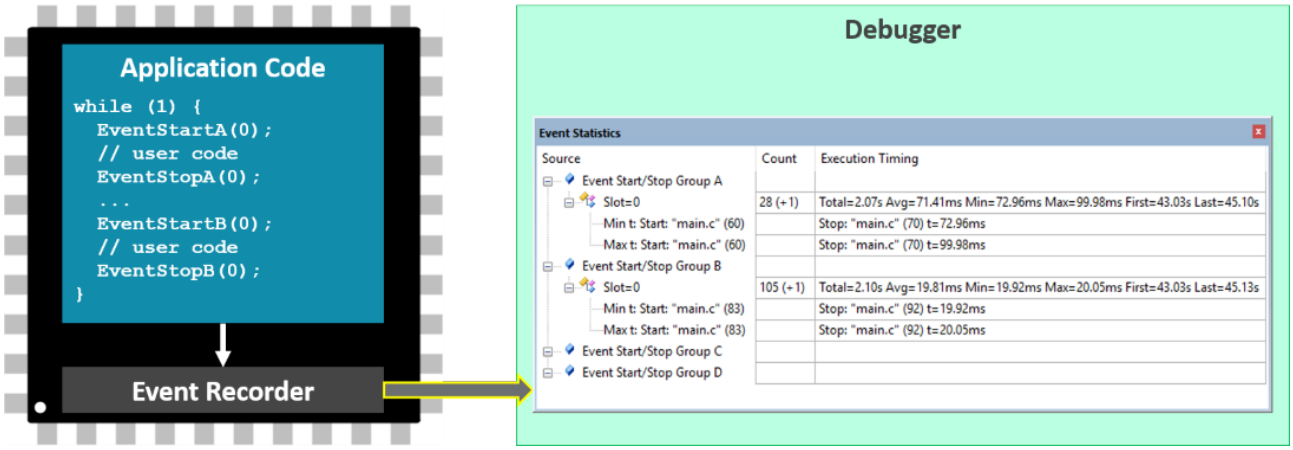


在截图的左下角有个 Memory 内存区，在这个内存区里面有一个缓冲 Event Buffer，其实就是一个大数组。MDK 通过访问这个数组实现消息的图形化展示。为了正确的图形化展示，数组缓冲里面的数据就得有一定的数据格式。而这个数据格式就是通过左侧截图里面的 Event Recorder 和 Event Filter 来实现的。Event Recorder 的 API 实现数据记录和整理，Event Filter 的 API 实现数据的筛选，从而可以选择哪些数据可以在 MDK 的 Event Recorder 调试组件里面展示出来。

这就是 Event Recorder 的基本工作流程。

8.2.3 Event Statistics 时间测量功能

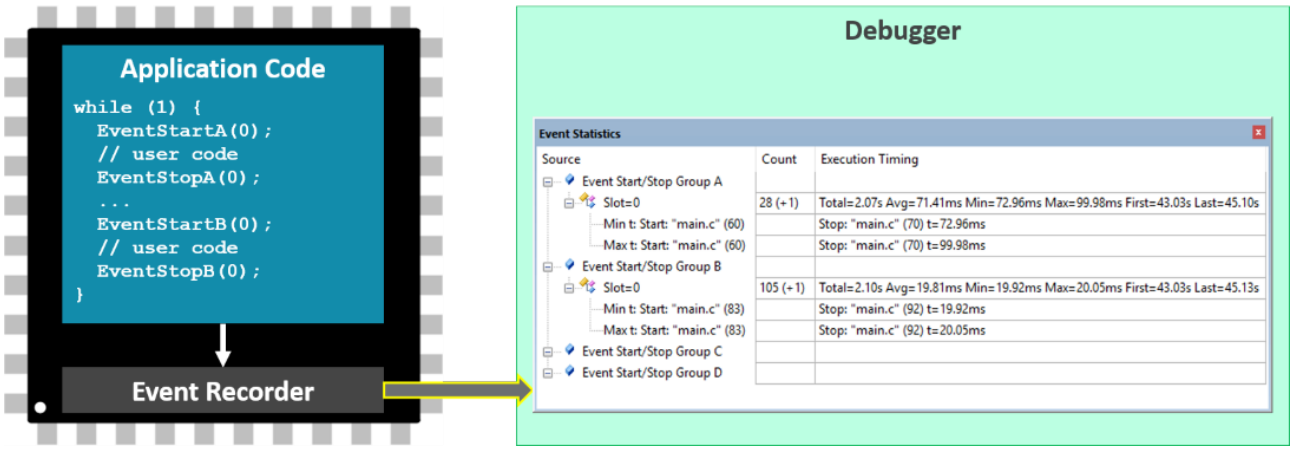
Event Statistics 提供的时间测量功能简单易用，在测试代码前后加上测量函数即可：



在本章教程程的 8.6 小节为大家详细进行了讲解。通过这个时间测量功能，用户可以方便测试代码的执行时间，从而根据需要，进行合理的优化，提高代码执行效率。

8.2.4 Event Statistics 功耗测量功能

Event Statistics 提供的功耗测量功能，当前只有 KEIL 的 ULINKplus 支持此功能，由于 ULINKplus 价格不便宜，一套 5000 多，大家作为了解即可，实际效果如下：



8.2.5 Event Recorder 的实现原理

每条 Event Recorder 消息是由 16 字节的数据组成，32 位的 ID，32 位的时间戳，两个 32 位的数据，共计 16 个字节。其中 32 位 ID 最重要，格式如下：

ID	位	描述
Message number	Bit[0:7]	标识软件组件的事件消息信息，其实就是消息 ID
Component number	Bit[8:15]	标识软件组件（也用于消息筛选）
Level	Bit[16:17]	指定用于消息筛选的类别
保留	Bit[18:31]	设置为 0

Level 指定消息分类，主要用于消息筛选：

Level	描述
EventLevelError = 0	运行错误
EventLevelAPI = 1	API 函数调用
EventLevelOp = 2	内部操作
EventLevelDetail = 3	更多详细的操作信息

Component number 指定事件消息所属的软件组件，也可用于过滤：

Component number	描述
0x0 .. 0x3F (0 .. 63)	用于用户应用程序的软件组件
0x40 .. 0x7F (64 .. 127)	第三方中间件
0x80 .. 0xEE (128 .. 238)	MDK 中间件
0xEF (239)	用于 Event Statistics 的启动和停止事件
0xF0 .. 0xFC (240 .. 253)	RTOS 内核
0xFD (253)	进程间通信层（用于多核系统）
0xFE (254)	printf 重定向

0xFF (255)	Event Recorder 消息

看了下 Event Recorder 的源码，每条消息大体是一样的：

```
typedef struct {
  uint32_t ts;                  // Timestamp (32-bit, Toggle bit instead of MSB)
  uint32_t val1;                // Value 1   (32-bit, Toggle bit instead of MSB)
  uint32_t val2;                // Value 2   (32-bit, Toggle bit instead of MSB)
  uint32_t info;                // Record Information
                                //  [ 7.. 0]: Message ID (8-bit)
                                //  [15.. 8]: Component ID (8-bit)
                                //  [18..16]: Data Length (1..8) / Event Context
                                //      [19]: IRQ Flag
                                //  [23..20]: Sequence Number
                                //      [24]: First Record
                                //      [25]: Last Record
                                //      [26]: Locked Record
                                //      [27]: Valid Record
                                //      [28]: Timestamp MSB
                                //      [29]: Value 1 MSB
                                //      [30]: Value 2 MSB
                                //      [31]: Toggle bit
} EventRecord_t;
```

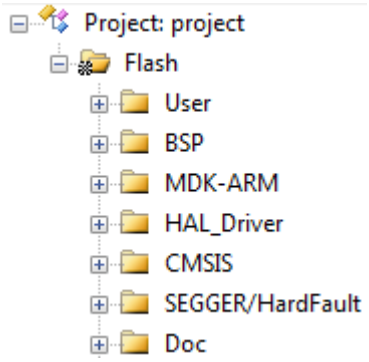
其中参数成员 info 最重要，也就是前面说的 32 位 ID，这里的说明与前面的说明稍有不同。这里是经过处理后，实际存储到 Event Recorder 缓冲里面的数据。

对于 Event Recorder，大家了解了这些知识点基本就够用了。

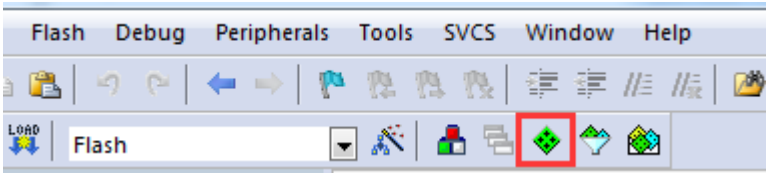
8.3 创建工程模板和注意事项

Event Recorder 工程的创建比较简单，这里分步为大家做个介绍。

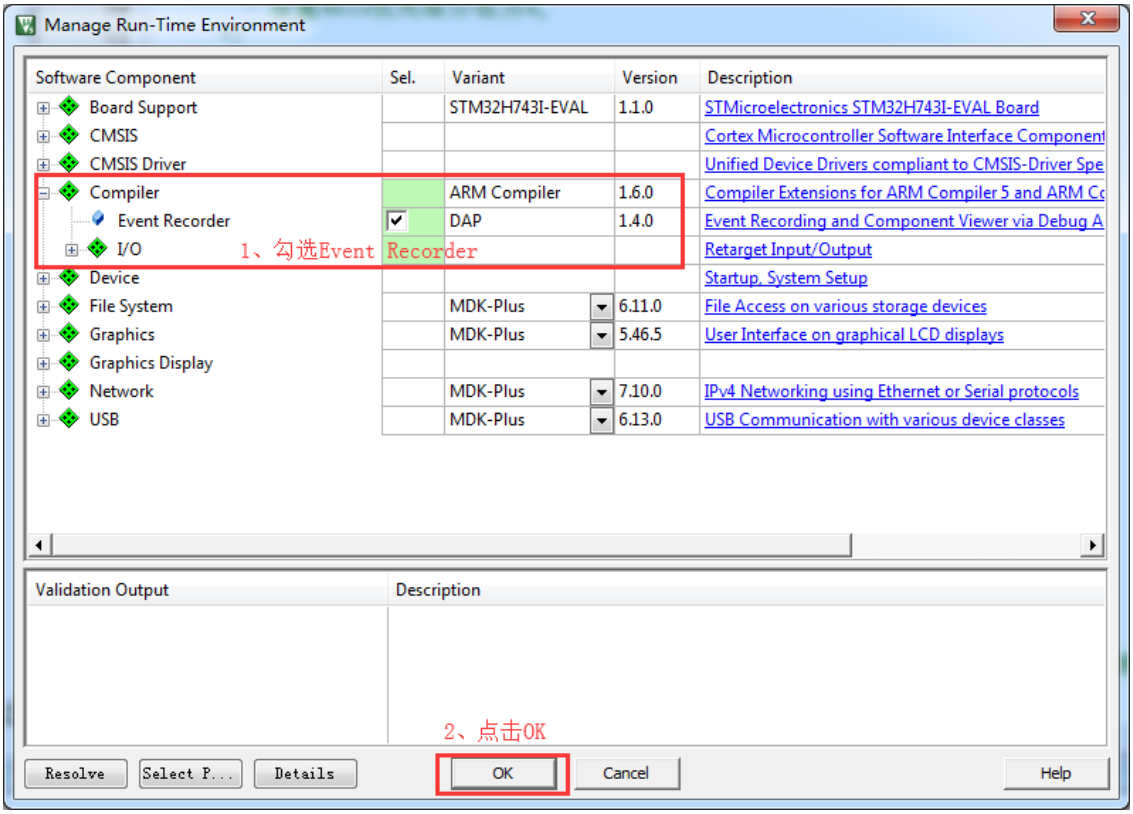
- 第 1 步：准备好一个使用 MDK5.25 或以上版本创建的工程模板。



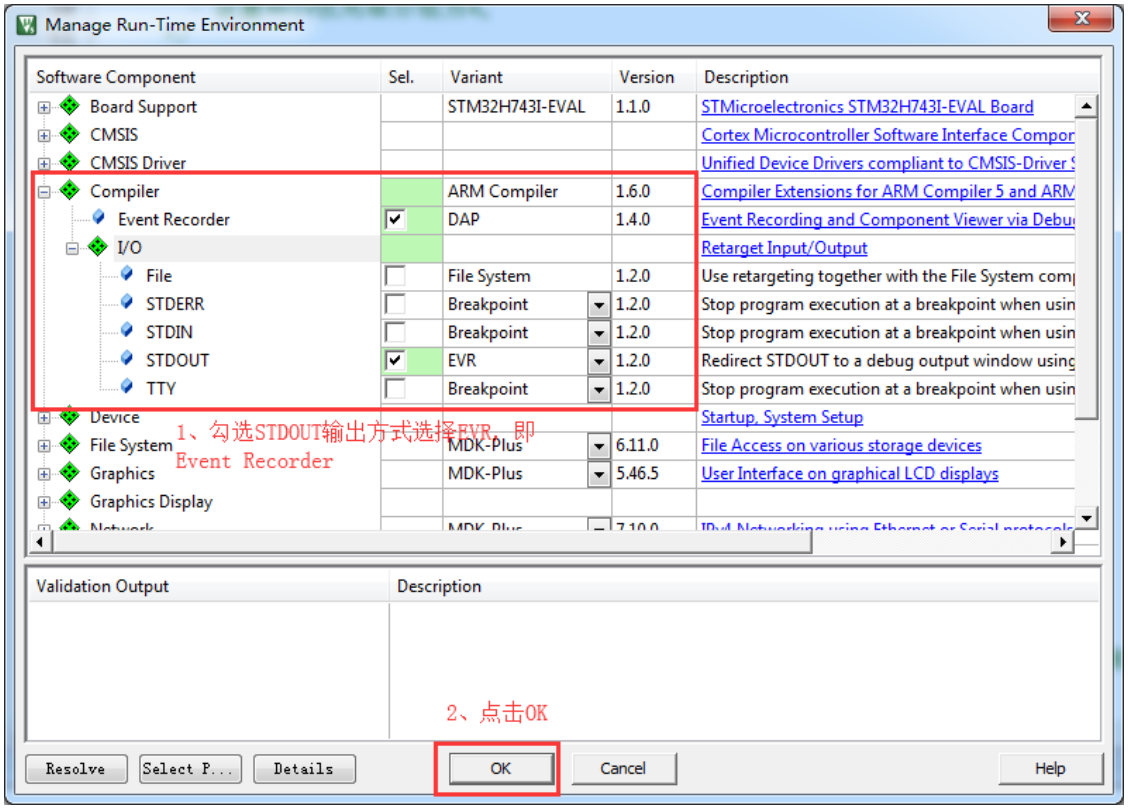
- 第 2 步：安装 ARM_Compiler V1.4.0 或以上版本（如果有最新版，直接安装最新的），详情见帖子：http://www.armbbs.cn/forum.php?mod=viewthread&tid=87175 。
- 第 3 步：打开 MDK5.25 或以上版本创建的 RTE 环境。



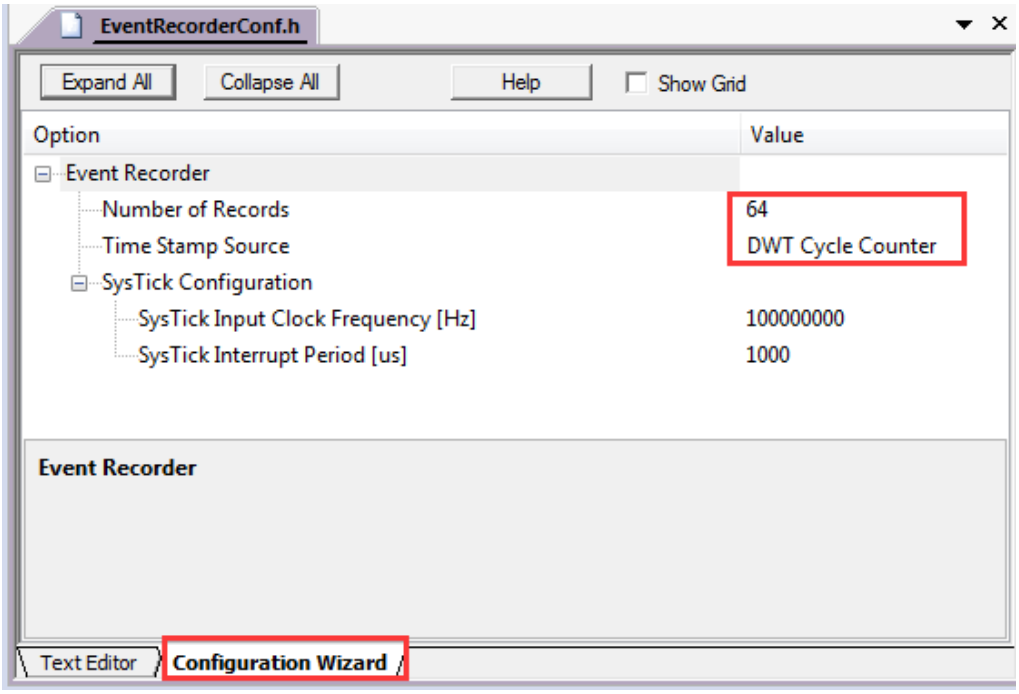
◆ 第 4 步：通过 RTE 环境，为工程添加 Event Recorder 功能。



◆ 第 5 步：为了实现 printf 重定向，我们需要将 STDOUT 的输出方式改为 Event Recorder，即选项里面的 EVR。



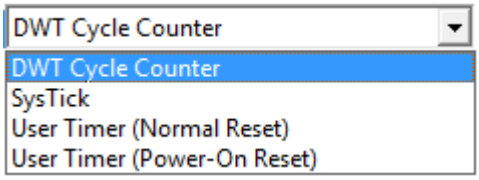
◆ 第 6 步：打开通过 RTE 环境为工程添加的文件 EventRecorderConf.h，配置如下：



这里主要设置方框里面的两个参数。

Number of Records：表示 Event Recorder 缓冲可以记录的消息条数。

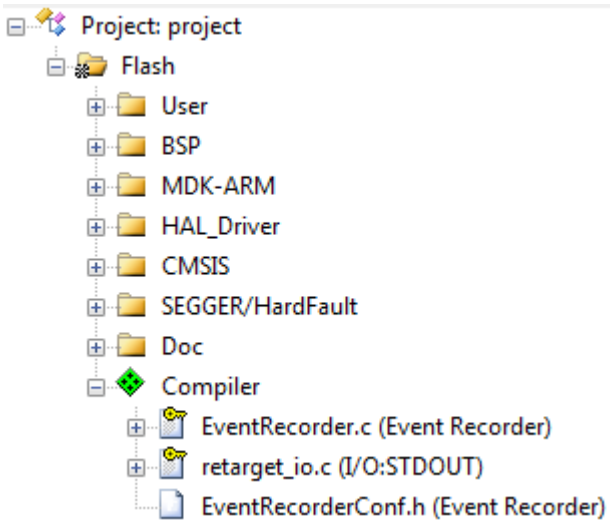
Time Stamp Source：表示时间戳来源，有如下四种可以选择，我们这里使用 DWT 时钟周期计数器。



由于选择的是 DWT，因此 EventRecorderCong.h 文件中的 Systick Configuration 配置就不用管了。

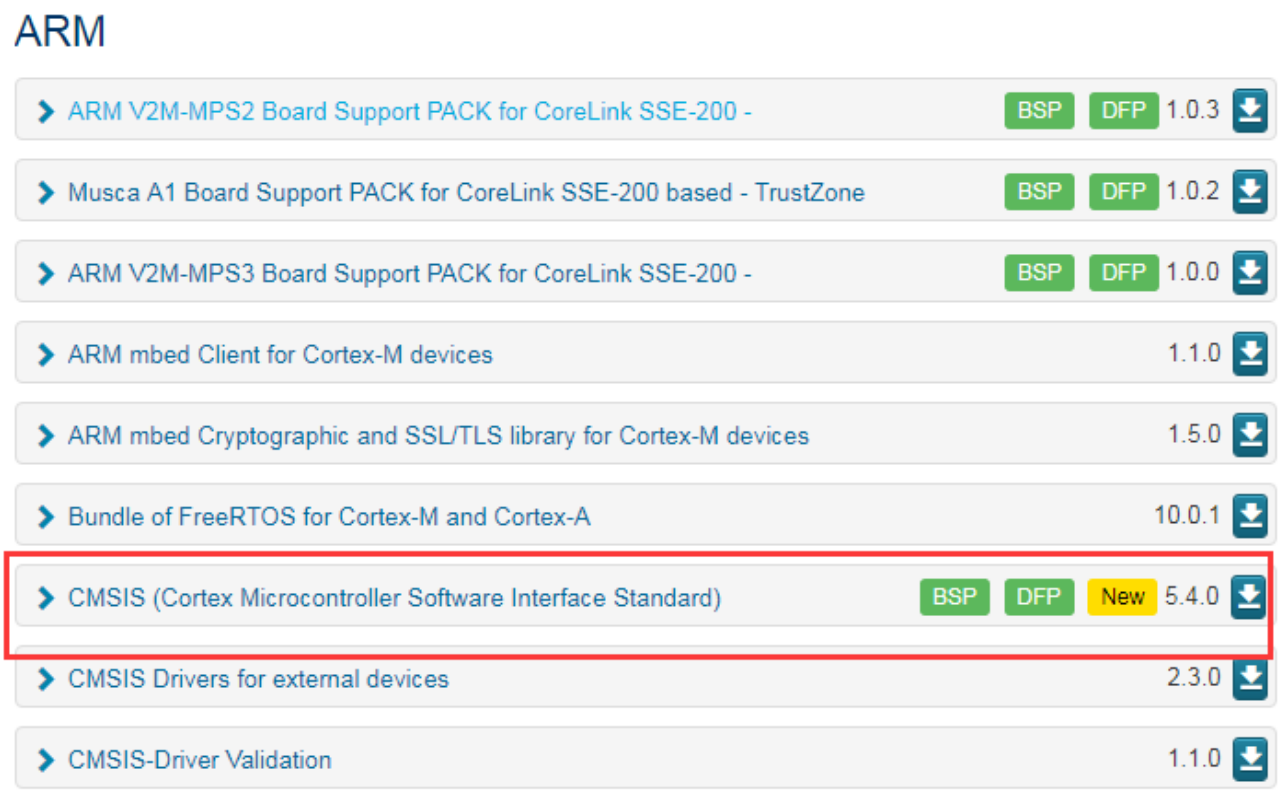
==========

通过上面的 6 步就完成了 Event Recorder 功能的添加，效果如下：



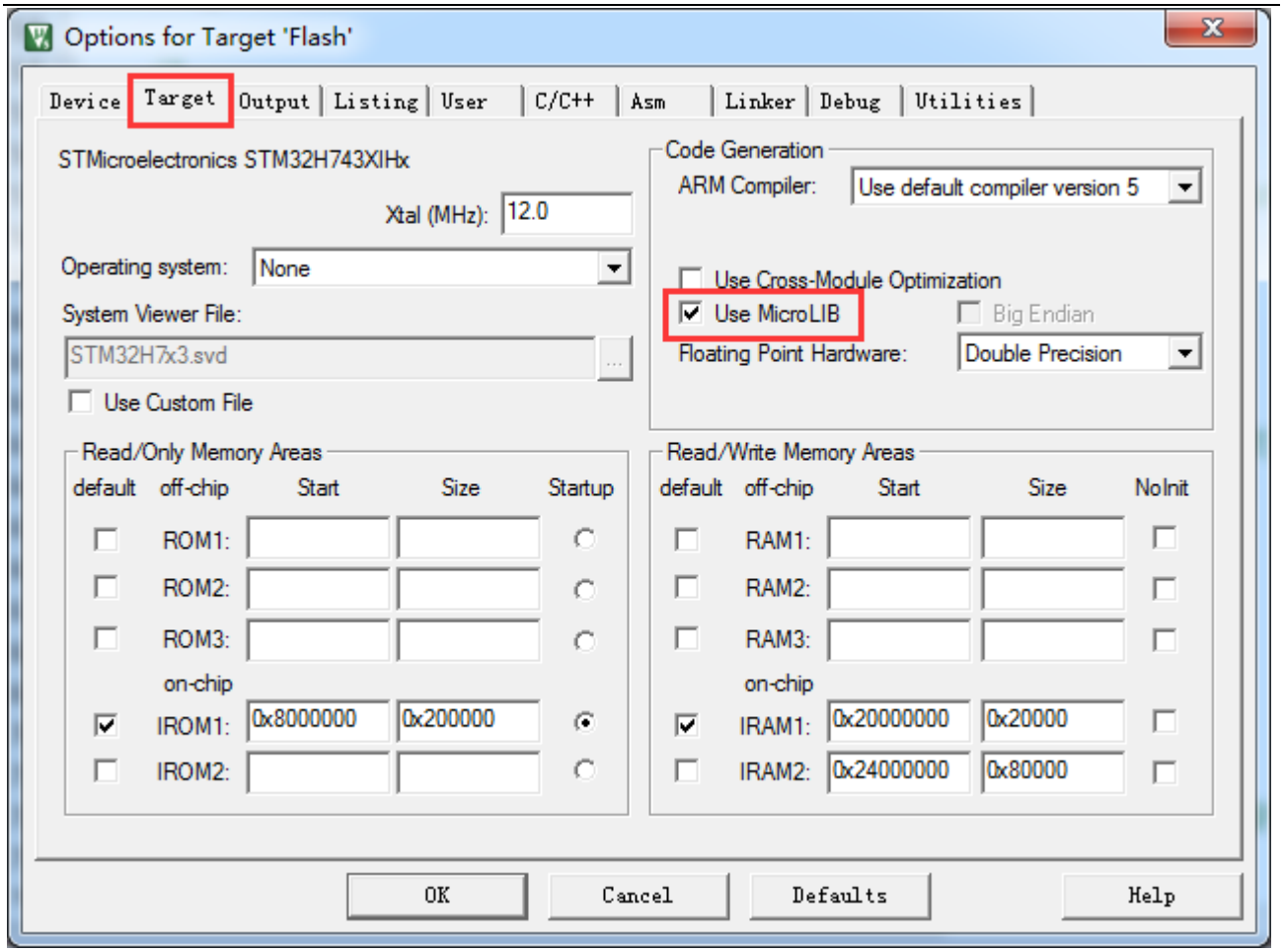
添加完成后，还有非常重要的两点要特别注意：

- 第 1 点：一定要使用当前最新的 CMSIS 软件包，当前是 V5.4.0（随着时间的推移，如果升级了新版本，直接使用新版即可）。大家可以从这里下载：
  http://www.keil.com/dd2/pack/ 。



下载并导入到 MDK 后，需要大家更新自己现有工程 CMSIS 文件里面的头文件，可以直接将 CMSIS 文件夹中 Include 文件里面的所有文件全部删掉，替换为 MDK 安装目录如下路径里面的所有头文件：ARM\PACK\ARM\CMSIS\5.4.0\CMSIS\Include。保证头文件都是最新的 5.4.0 版本。

- 第 2 点：由于使能了 printf 重定向，大家的工程里面一定不要再做重定向了，比如 fpuc，fgetc。另外当前选择了微库 MicroLib：



注意这两点后，就可以使用 Event Recorder 的功能了。

8.4 Event Recorder 事件记录的实现

Event Recorder 的使用也比较省事，这里也分步为大家进行说明：

◆ 第 1 步：初始化，仅需添加如下两行代码即可。

```
/* 初始化 EventRecorder 并开启 */
EventRecorderInitialize(EventRecordAll, 1U);
EventRecorderStart();
```

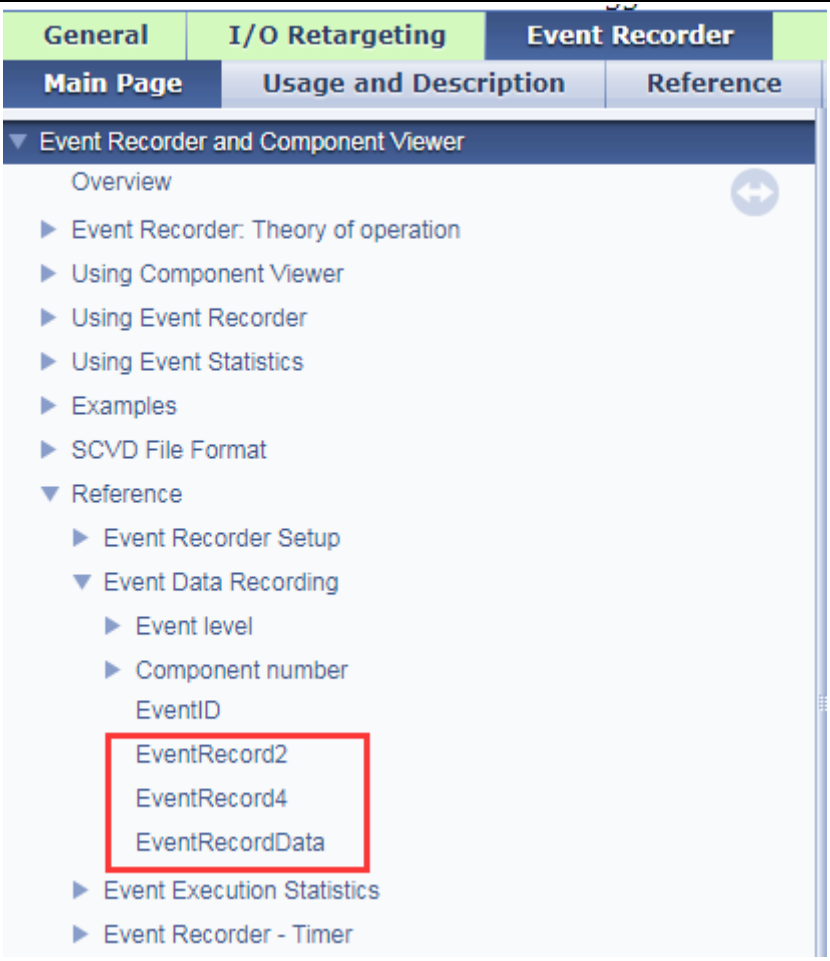
◆ 第 2 步：调用 Event Recorder 的 API 就可以使用了，主要有以下三个 API：

EventRecord2：可以发送两个 32 位数据。

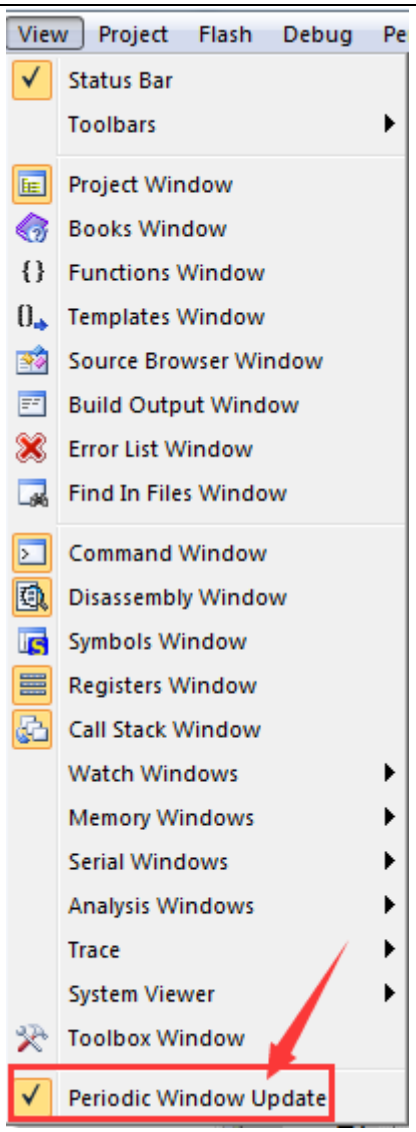
EventRecord4：可以发送四个 32 位数据。

EventRecordData：可以发送字符串。

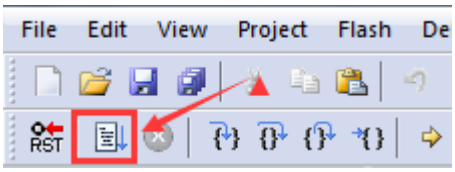
显然这三个函数没有 printf 使用方便，所以对于这三个函数，大家做个简单的了解即可。教程配套例子里面有调用到这三个函数，可以操作熟悉下。这三个 API 的说明是在对应的 help 文档中，即 MDK 安装目录路径：/ARM/PACK/Keil/ARM_Compiler/1.6.0/Doc/General/html/index.html。



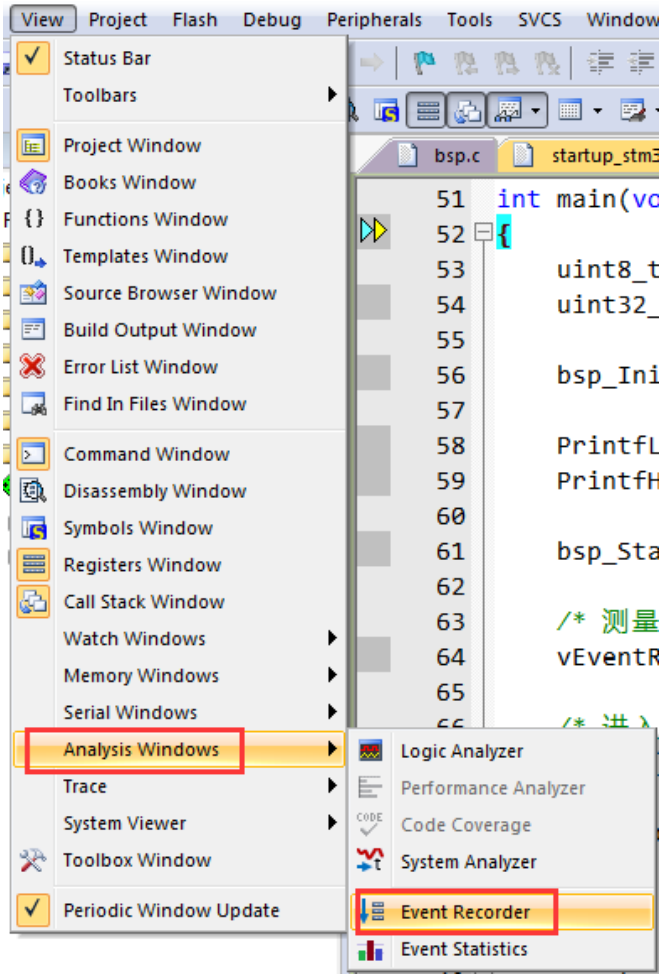
◆ 第 3 步：进入调试状态，选上周期更新：



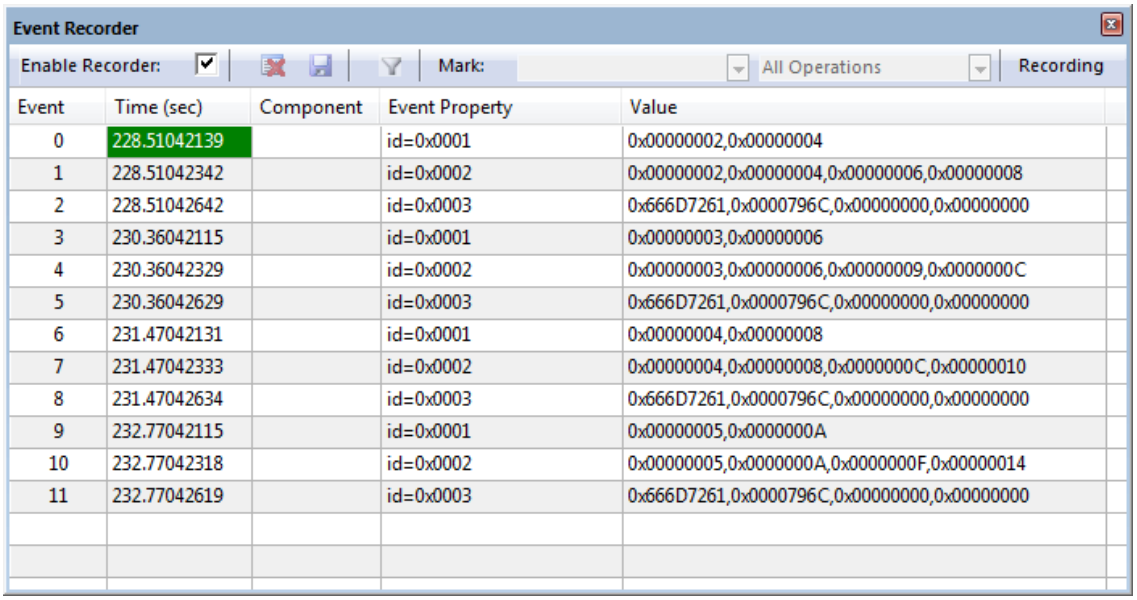
点击全速运行：



然后将 Event Recorder 调试组件展示出来：

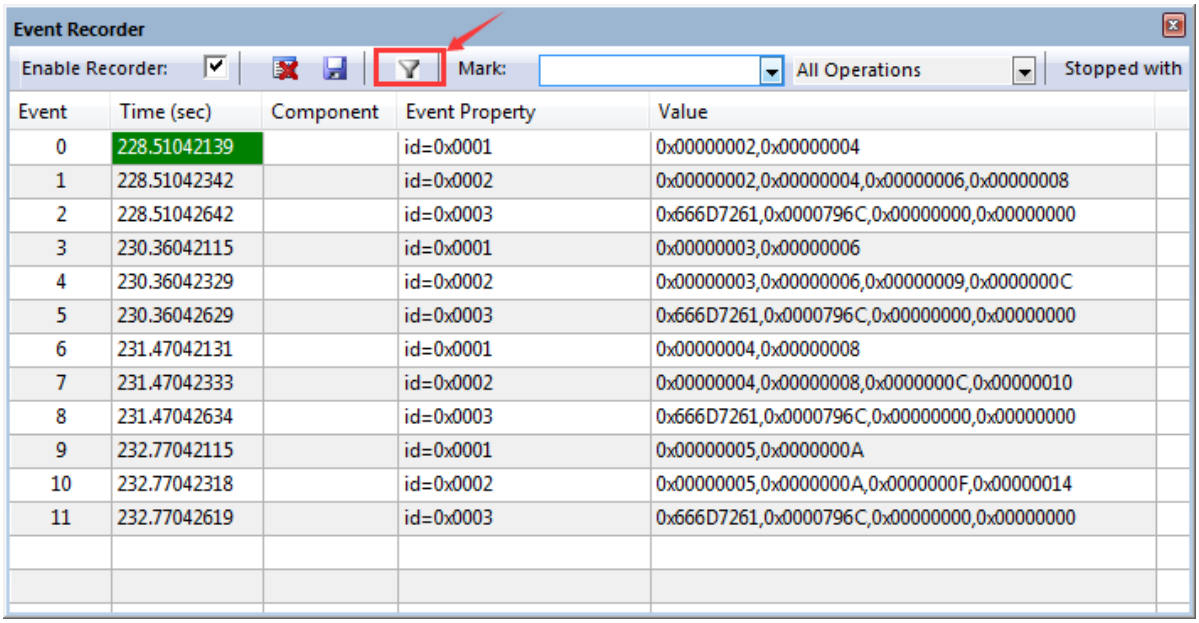


效果如下：



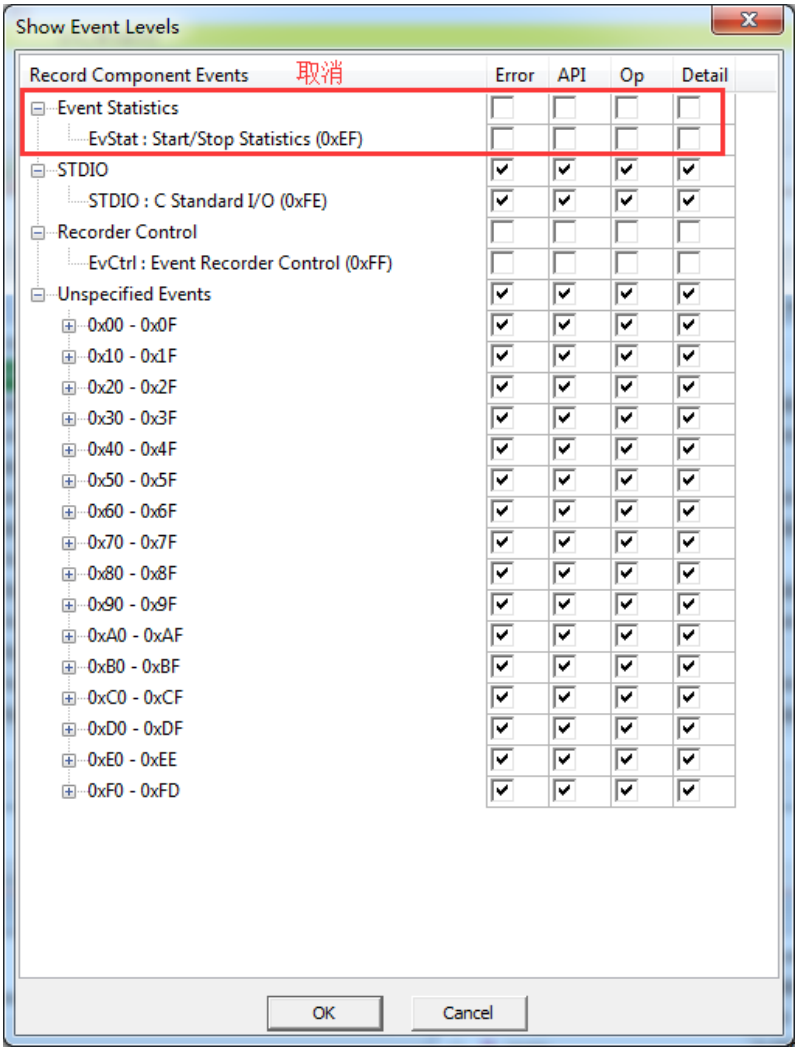
Event	Time (sec)	Component	Event Property	Value
0	228.51042139		id=0x0001	0x00000002,0x00000004
1	228.51042342		id=0x0002	0x00000002,0x00000004,0x00000006,0x00000008
2	228.51042642		id=0x0003	0x666D7261,0x0000796C,0x00000000,0x00000000
3	230.36042115		id=0x0001	0x00000003,0x00000006
4	230.36042329		id=0x0002	0x00000003,0x00000006,0x00000009,0x0000000C
5	230.36042629		id=0x0003	0x666D7261,0x0000796C,0x00000000,0x00000000
6	231.47042131		id=0x0001	0x00000004,0x00000008
7	231.47042333		id=0x0002	0x00000004,0x00000008,0x0000000C,0x00000010
8	231.47042634		id=0x0003	0x666D7261,0x0000796C,0x00000000,0x00000000
9	232.77042115		id=0x0001	0x00000005,0x0000000A
10	232.77042318		id=0x0002	0x00000005,0x0000000A,0x0000000F,0x00000014
11	232.77042619		id=0x0003	0x666D7261,0x0000796C,0x00000000,0x00000000

另外，这里有个知识点需要大家了解下，如果程序里面也调用了 Event Statistics 时间测量函数，那么也会在这个界面里面展示消息的，如何才能仅展示大家想看的功能呢？这就需要用到 Event Recorder 支持的筛选功能。使用这个功能需要大家先暂停全速运行，然后点击下面这个选项：

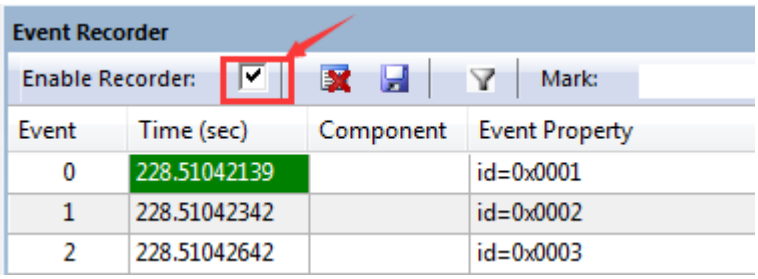


Event	Time (sec)	Component	Event Property	Value
0	228.51042139		id=0x0001	0x00000002,0x00000004
1	228.51042342		id=0x0002	0x00000002,0x00000004,0x00000006,0x00000008
2	228.51042642		id=0x0003	0x666D7261,0x0000796C,0x00000000,0x00000000
3	230.36042115		id=0x0001	0x00000003,0x00000006
4	230.36042329		id=0x0002	0x00000003,0x00000006,0x00000009,0x0000000C
5	230.36042629		id=0x0003	0x666D7261,0x0000796C,0x00000000,0x00000000
6	231.47042131		id=0x0001	0x00000004,0x00000008
7	231.47042333		id=0x0002	0x00000004,0x00000008,0x0000000C,0x00000010
8	231.47042634		id=0x0003	0x666D7261,0x0000796C,0x00000000,0x00000000
9	232.77042115		id=0x0001	0x00000005,0x0000000A
10	232.77042318		id=0x0002	0x00000005,0x0000000A,0x0000000F,0x00000014
11	232.77042619		id=0x0003	0x666D7261,0x0000796C,0x00000000,0x00000000

弹出的界面里面可以设置哪些选项显示，哪些选项不显示（勾上表示显示），我们这里取消 Event Statistics 的显示，设置完毕后记得点击 OK 按钮。



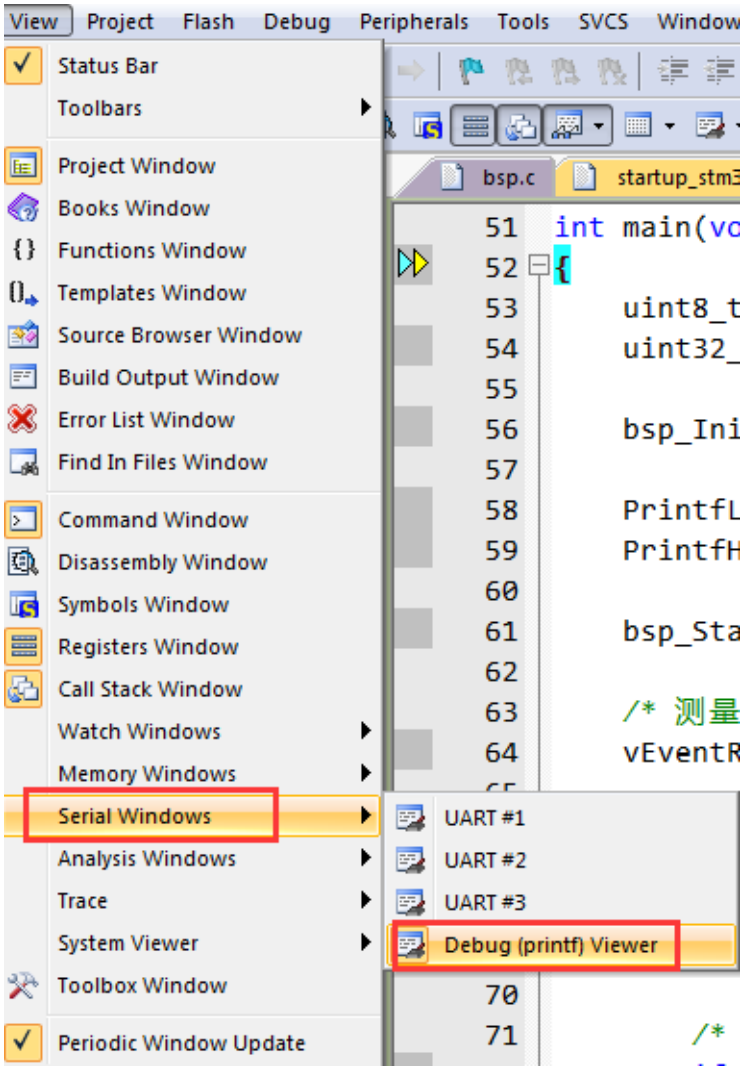
这就不展示 Event Statistics 的内容了。再次启动全速运行前，下面这个选项的对勾别忘了勾上。



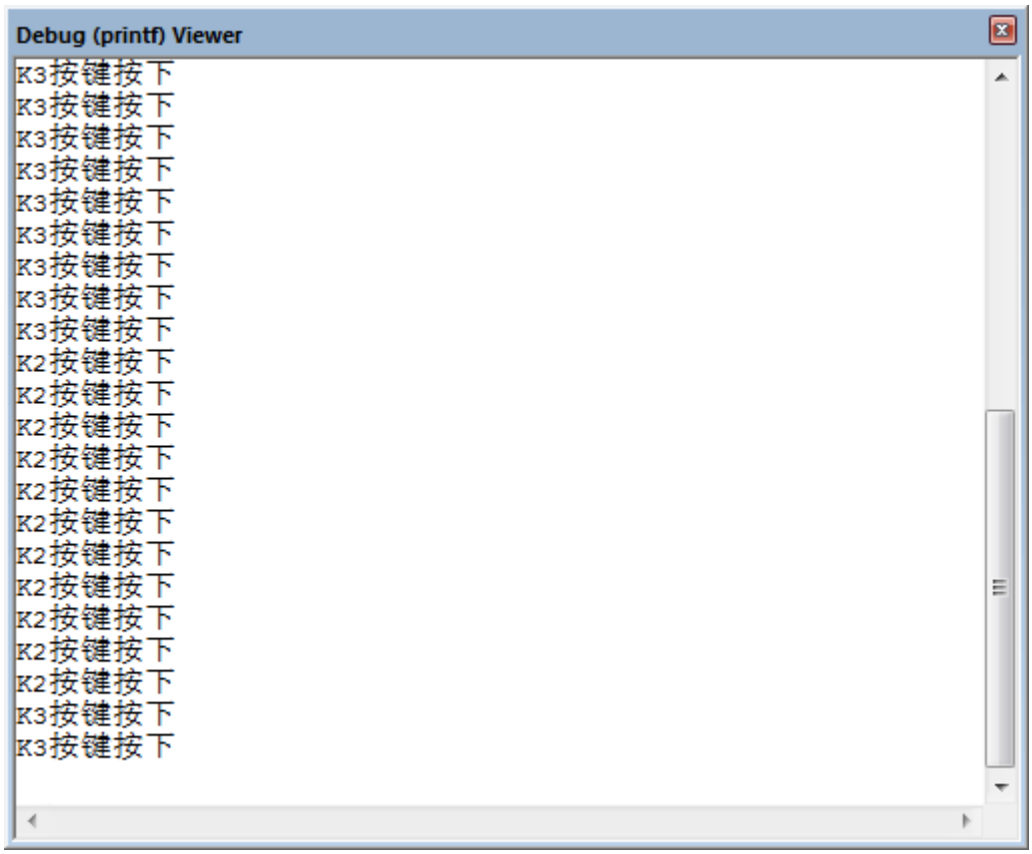
8.5 Event Recorder 实现 printf 重定向

实现 printf 输出需要用到 MDK 调试组件中的 Debug(printf) Viewer，输出效果就跟大家使用串口调试软件一样，可以输出中文和英文。

MDK 的 printf 调试组件使用方法跟本章 8.4 小节中的说明一样，点击调试，选中周期运行，然后显示 Debug(printf) Viewer 调试组件：



效果如下：



另外，还有一个知识点需要给大家做个补充，使用 SWD 接口的 SWO 引脚也是可以做串口打印的，并且也是通过这个调试组件 Debug(printf) Viewer 进行输出。只是这种方式的性能没有 Event Viewer 强，而且要多占用一个 SWO 引脚。

关于 SWO 输出方式可以看此贴：http://www.armbbs.cn/forum.php?mod=viewthread&tid=526 。

8.6 Event Statistics 时间测量功能的实现

时间测量功能简单易用，仅需一个起始函数，一个停止函数即可。当前支持 4 组，每组支持 16 路测量，也就是可以同时测量 64 路。

时间测量的 API 函数支持多任务和中断里面随意调用。

- 测量起始函数：EventStartG (slot) 或者 EventStartGv (slot, val1, val2)
  - 函数中的字母 G 是表示分组 A，B，C，D，即实际调用函数为 EventStartA，EventStartB，EventStartC 和 EventStartD。
  - 函数的第一个形参 slot 的范围是 0-15，也就是每个分组可以测试 16 路。
  - 函数后面的两个形象 val1 和 val2 是 32 位变量，用户可以用这两个形参来传递变量数值给 Event Statistics 调试组件里面，方便图形化展示。简单的说，这两个变量仅仅起到一个传递变量数值的作用。

◆ 测量停止函数：EventStopG (slot) 或者 EventStopGv (slot, val1, val2)

- 函数中的字母 G 是表示分组 A,B,C,D,即实际调用函数为 EventStopA,EventStopB,EventStopC 和 EventStopD。
- 函数的第一个形参 slot 的范围是 0-15，也就是每个分组可以测试 16 路。
- 函数后面的两个形象 val1 和 val2 是 32 位变量，用户可以用这两个形参来传递变量数值给 Event Statistics 调试组件里面，方便图形化展示。简单的说，这两个变量仅仅起到一个传递变量数值的作用。

这里也分步为大家说明 Event Statistics 时间测量功能的使用方法。

◆ 第 1 步：初始化，仅需添加如下两行代码即可。

```
/* 初始化 EventRecorder 并开启 */
EventRecorderInitialize(EventRecordAll, 1U);
EventRecorderStart();
```

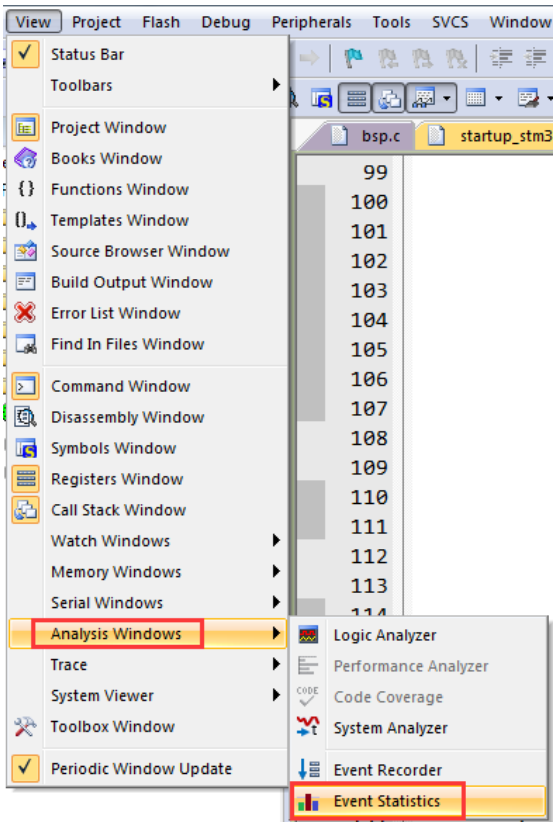
◆ 第 2 步：在要测量的代码前后加上起始和结束时间。

```
EventStartA(0);
测量的代码部分
EventStopA(0);
```

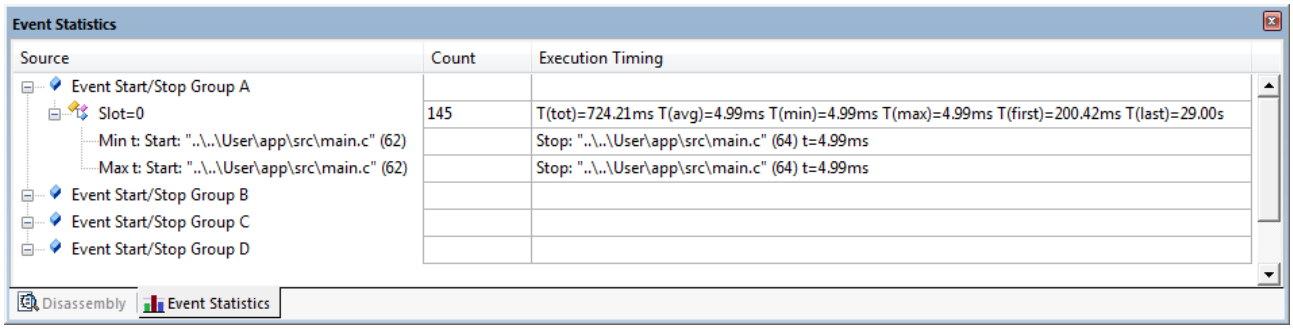
这里是用分组 A 的测量通道 0。

◆ 第 3 步：跟本章 8.4 小节讲解的一样，点击调试，选择周期更新选项，然后全速运行。

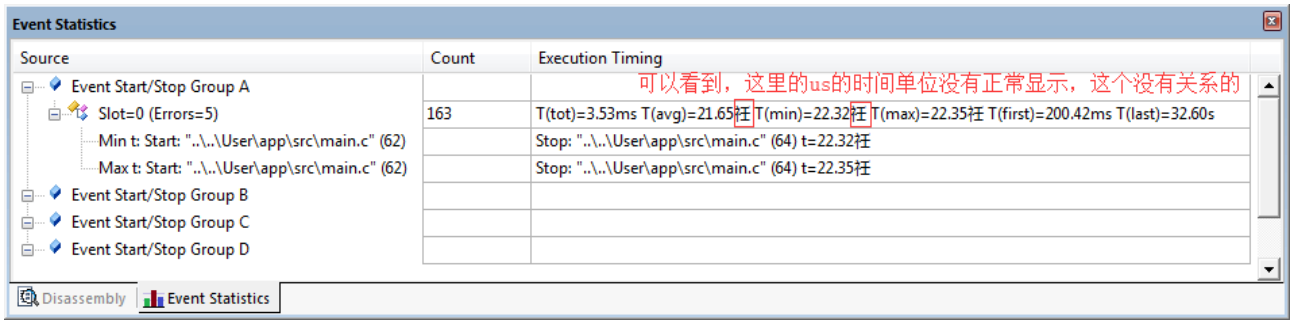
◆ 第 4 步：全速运行后，显示 Event Statistics 调试组件。



比如我这里简单的测试了一个 5ms 的延迟函数，效果如下（测量时间是动态更新的）：



另外要注意一点，微秒的时间单位 us 可能无法正常显示，这个是没有关系的：



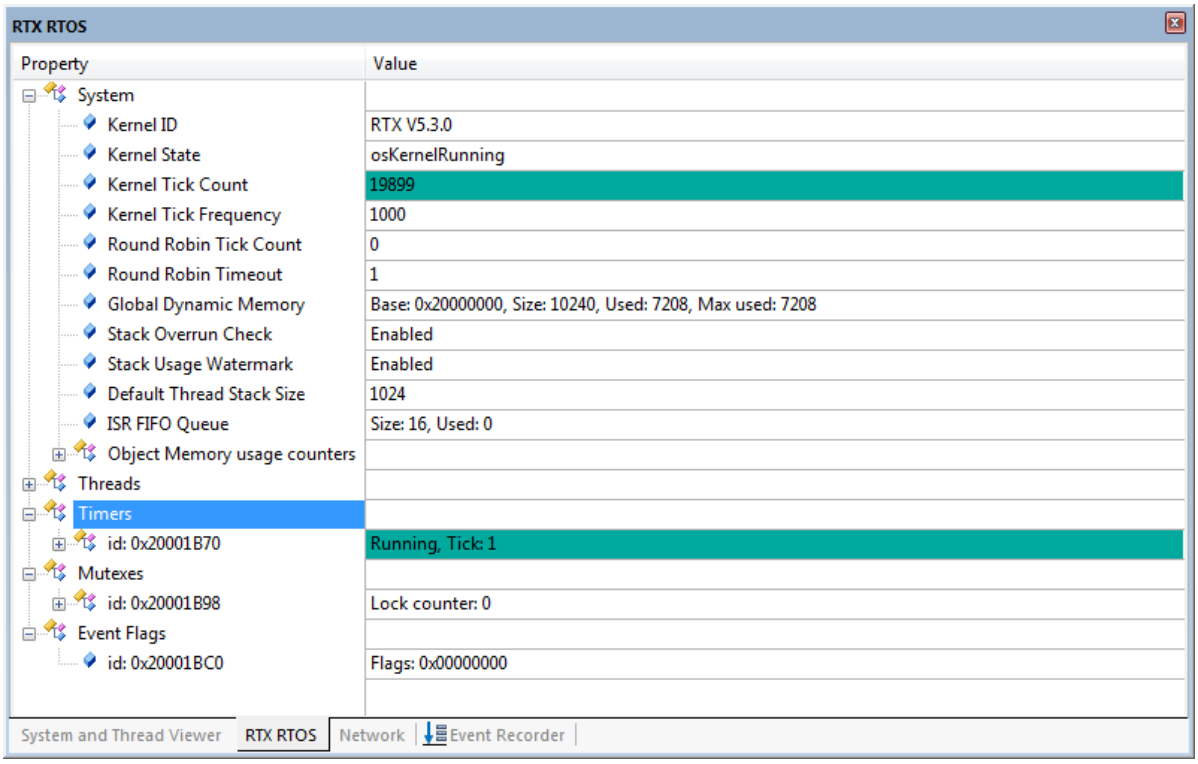
8.7 Event Statistics 功耗测量功能的实现

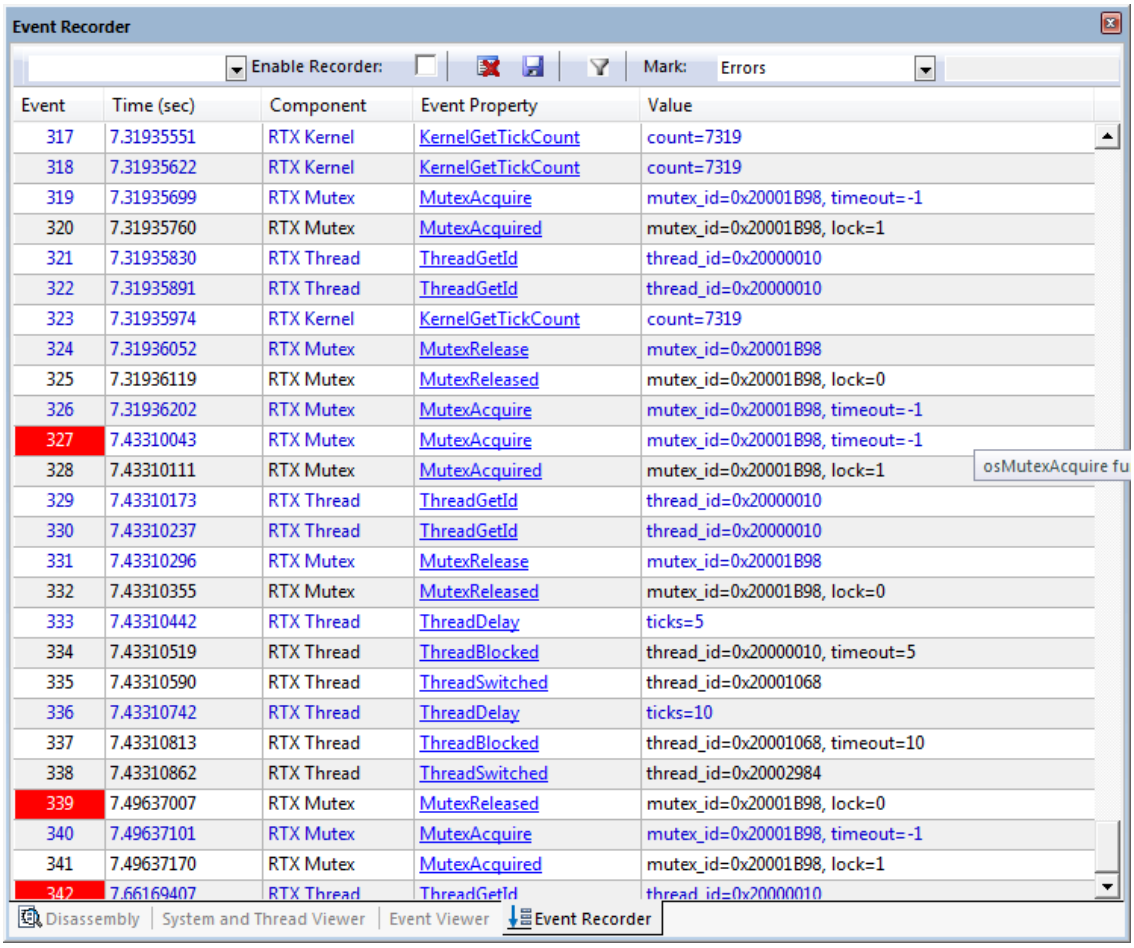
当前仅 KEIL 自家的 ULINKplus 支持功耗测量功能，这款下载器不便宜，一套 5000 多，大家有个了解即可，我们这里就不做讲解了。

8.8 Event Recorder 对 RTX5 及其所有中间件的支持

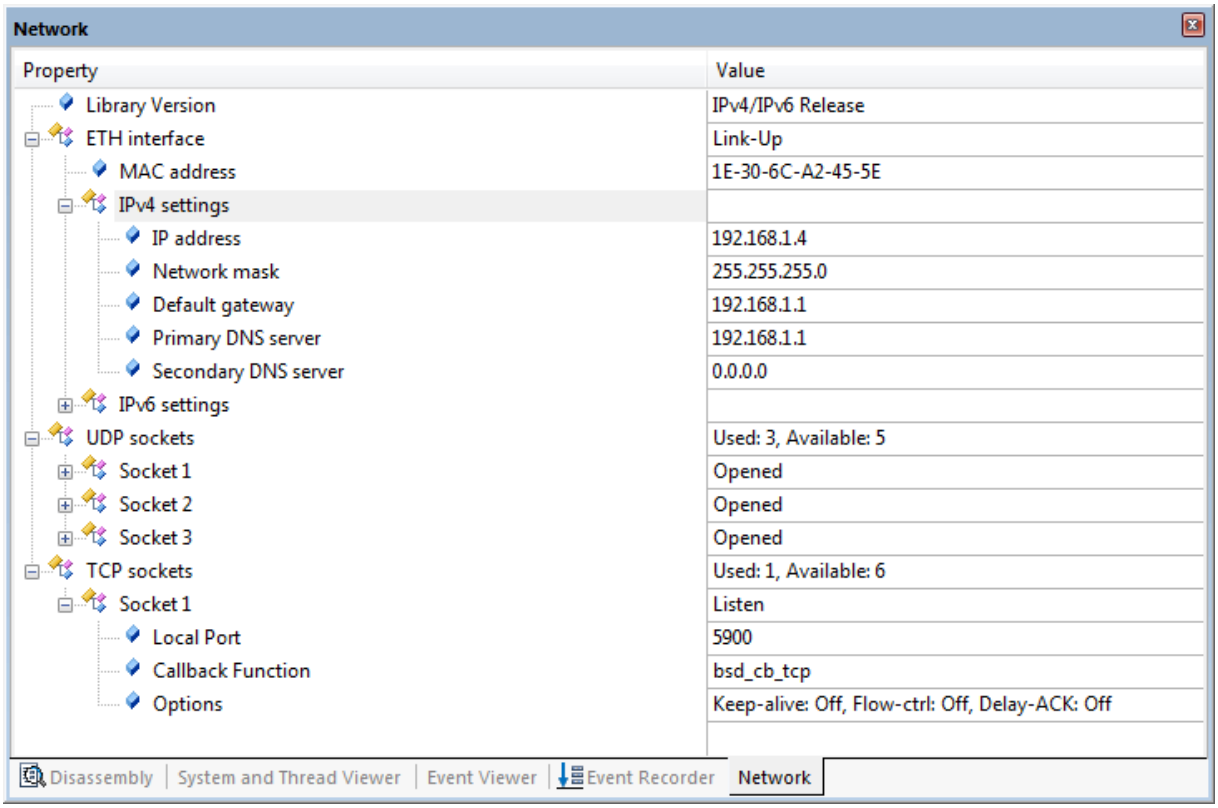
后面做 RTX5 及其所有中间件的教程时会为大家做讲解，这里让大家看下效果：

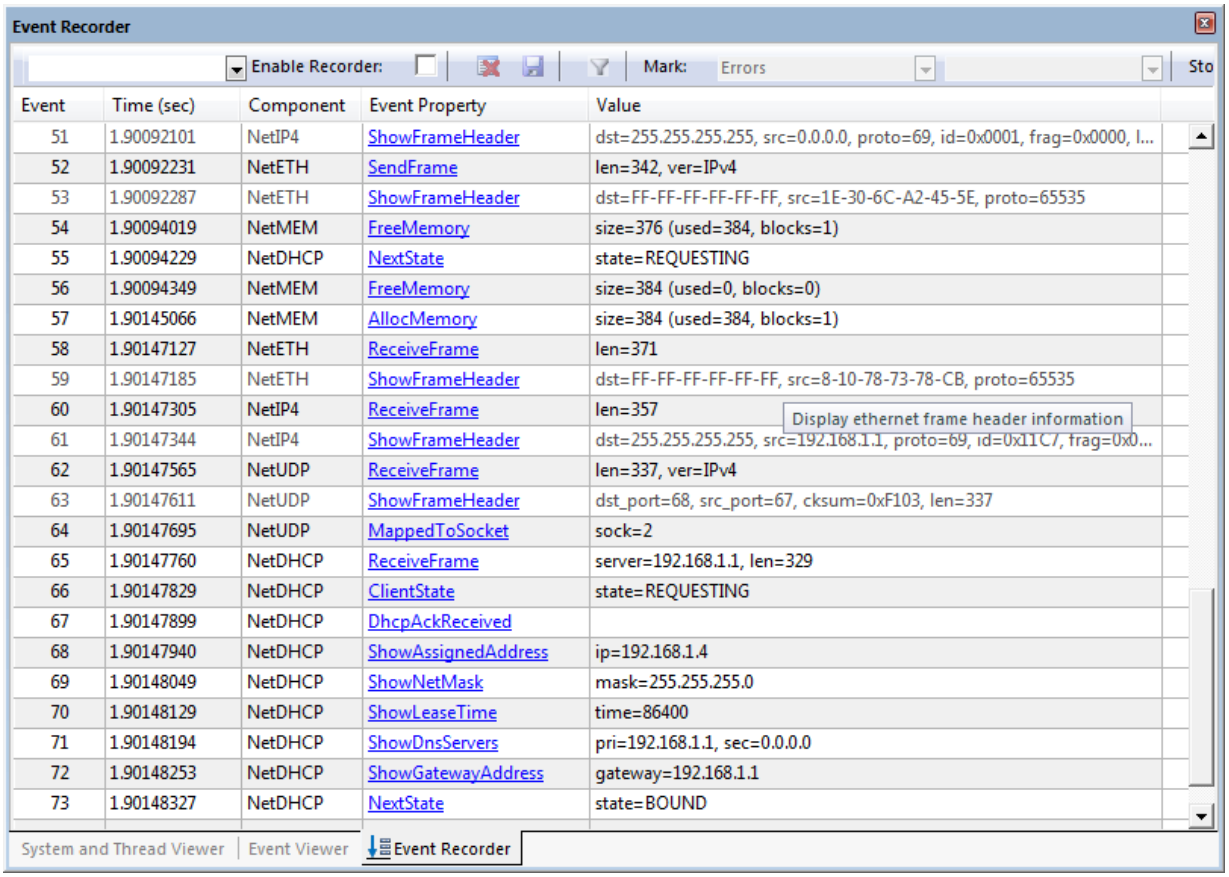
- RTX5 组件和使用 Event Recoder 的效果：


RTX RTOS
Property
Value
System
Kernel ID
RTX V5.3.0
Kernel State
osKernelRunning
Kernel Tick Count
19899
Kernel Tick Frequency
1000
Round Robin Tick Count
0
Round Robin Timeout
1
Global Dynamic Memory
Base: 0x20000000, Size: 10240, Used: 7208, Max used: 7208
Stack Overrun Check
Enabled
Stack Usage Watermark
Enabled
Default Thread Stack Size
1024
ISR FIFO Queue
Size: 16, Used: 0
Object Memory usage counters
Threads
Timers
id: 0x20001B70
Running, Tick: 1
Mutexes
id: 0x20001B98
Lock counter: 0
Event Flags
id: 0x20001BC0
Flags: 0x00000000
System and Thread Viewer
RTX RTOS
Network
Event Recorder

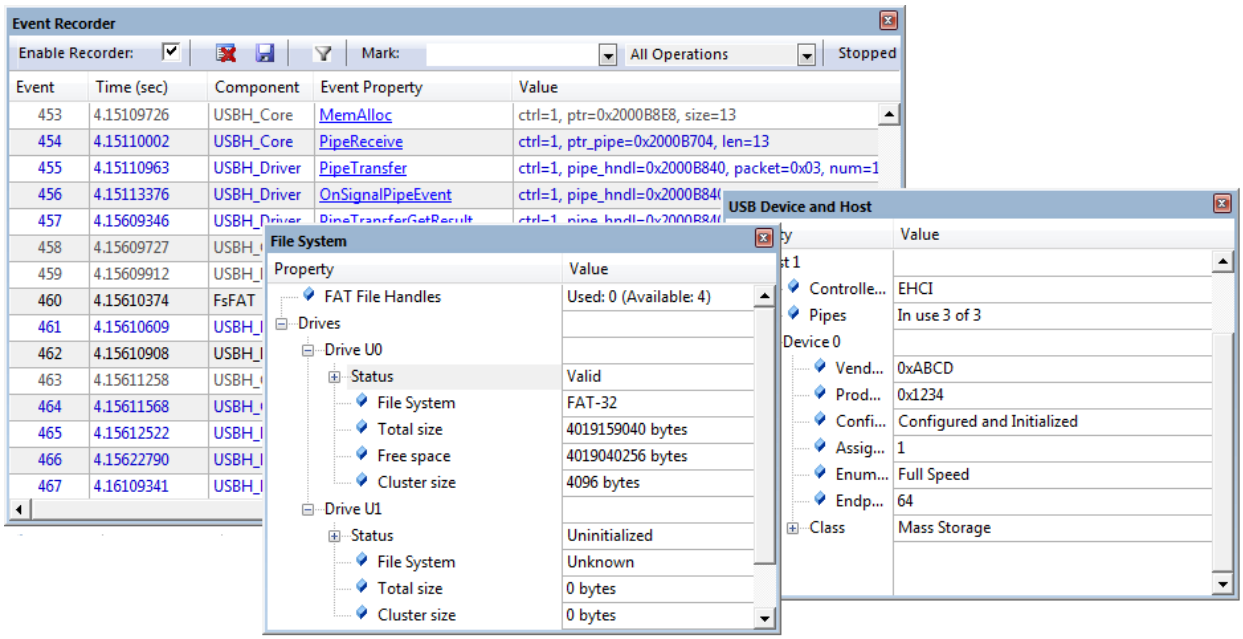

Event Recorder
Enable Recorder:
Mark:
Errors
Event
Time (sec)
Component
Event Property
Value
317 7.31935551 RTX Kernel KernelGetTickCount count=7319
318 7.31935622 RTX Kernel KernelGetTickCount count=7319
319 7.31935699 RTX Mutex MutexAcquire mutex_id=0x20001B98, timeout=-1
320 7.31935760 RTX Mutex MutexAcquired mutex_id=0x20001B98, lock=1
321 7.31935830 RTX Thread ThreadGetId thread_id=0x20000010
322 7.31935891 RTX Thread ThreadGetId thread_id=0x20000010
323 7.31935974 RTX Kernel KernelGetTickCount count=7319
324 7.31936052 RTX Mutex MutexRelease mutex_id=0x20001B98
325 7.31936119 RTX Mutex MutexReleased mutex_id=0x20001B98, lock=0
326 7.31936202 RTX Mutex MutexAcquire mutex_id=0x20001B98, timeout=-1
327 7.43310043 RTX Mutex MutexAcquire mutex_id=0x20001B98, timeout=-1
328 7.43310111 RTX Mutex MutexAcquired mutex_id=0x20001B98, lock=1
329 7.43310173 RTX Thread ThreadGetId thread_id=0x20000010
330 7.43310237 RTX Thread ThreadGetId thread_id=0x20000010
331 7.43310296 RTX Mutex MutexRelease mutex_id=0x20001B98
332 7.43310355 RTX Mutex MutexReleased mutex_id=0x20001B98, lock=0
333 7.43310442 RTX Thread ThreadDelay ticks=5
334 7.43310519 RTX Thread ThreadBlocked thread_id=0x20000010, timeout=5
335 7.43310590 RTX Thread ThreadSwitched thread_id=0x20001068
336 7.43310742 RTX Thread ThreadDelay ticks=10
337 7.43310813 RTX Thread ThreadBlocked thread_id=0x20001068, timeout=10
338 7.43310862 RTX Thread ThreadSwitched thread_id=0x20002984
339 7.49637007 RTX Mutex MutexReleased mutex_id=0x20001B98, lock=0
340 7.49637101 RTX Mutex MutexAcquire mutex_id=0x20001B98, timeout=-1
341 7.49637170 RTX Mutex MutexAcquired mutex_id=0x20001B98, lock=1
342 7.66169407 RTX Thread ThreadGetId thread_id=0x20000010
osMutexAcquire fu
Disassembly
System and Thread Viewer
Event Viewer
Event Recorder

◆ 网络调试组件效果展示:





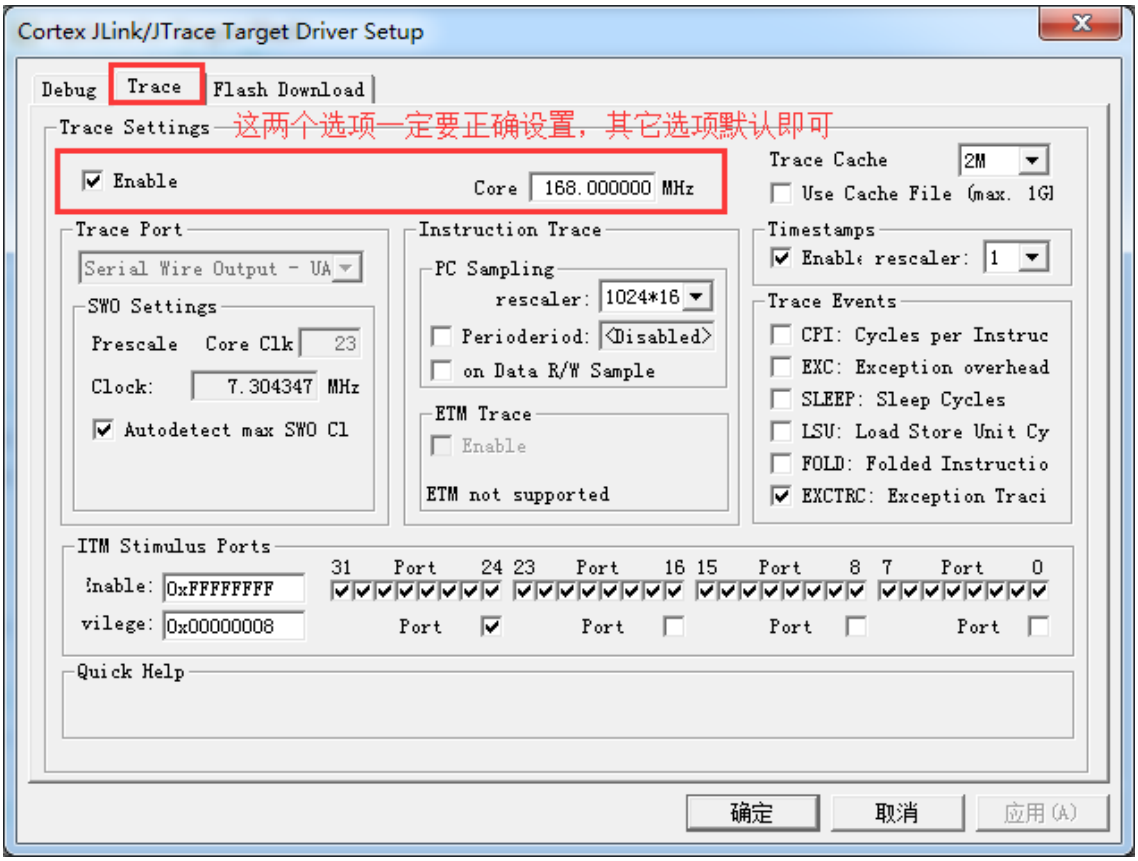
◆ 文件系统和 USB 协议栈的效果展示::



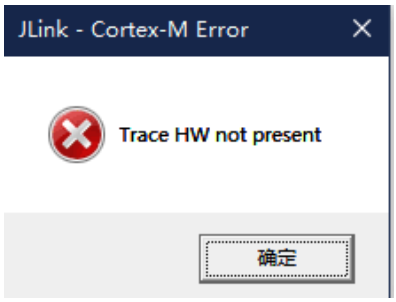
8.9 JLINK 配置说明

为了帮助大家更好的使用 JLINK，这里将 JLINK 配置中关键的几个地方做个说明。

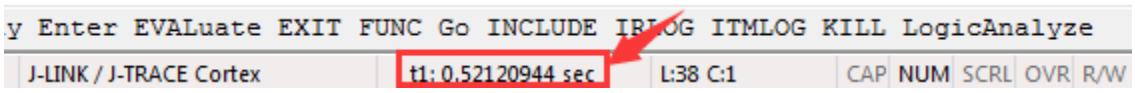
- 下面这个地方最重要，一定要正确设置当前系统工作的主频，如果不正确，会导致 Event Statistics 的时间统计不正确（对于 H7，Core 部分要填 400MHz）。



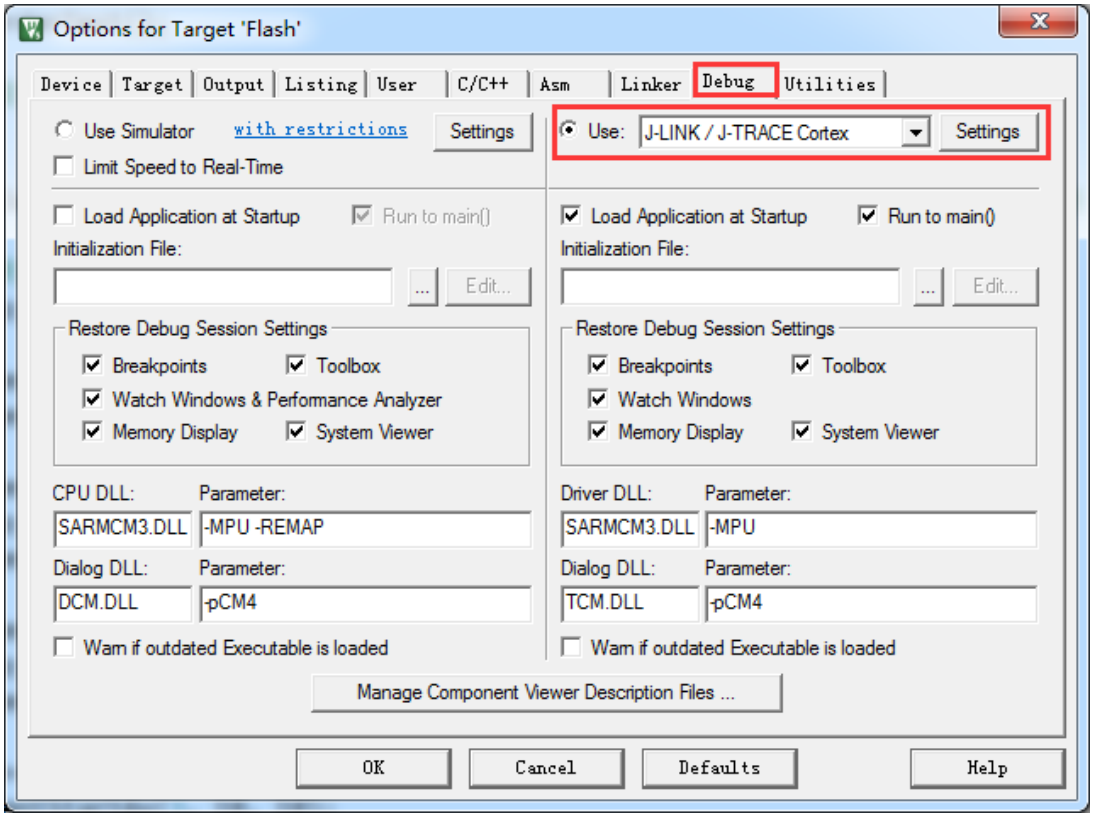
注：如果大家调试状态弹出 SWD 配置时钟超出范围的问题或者下面错误，可以考虑将上面截图中的 Enable 选项的对勾取消掉即可，但内核时钟一定要修改为芯片的主频。

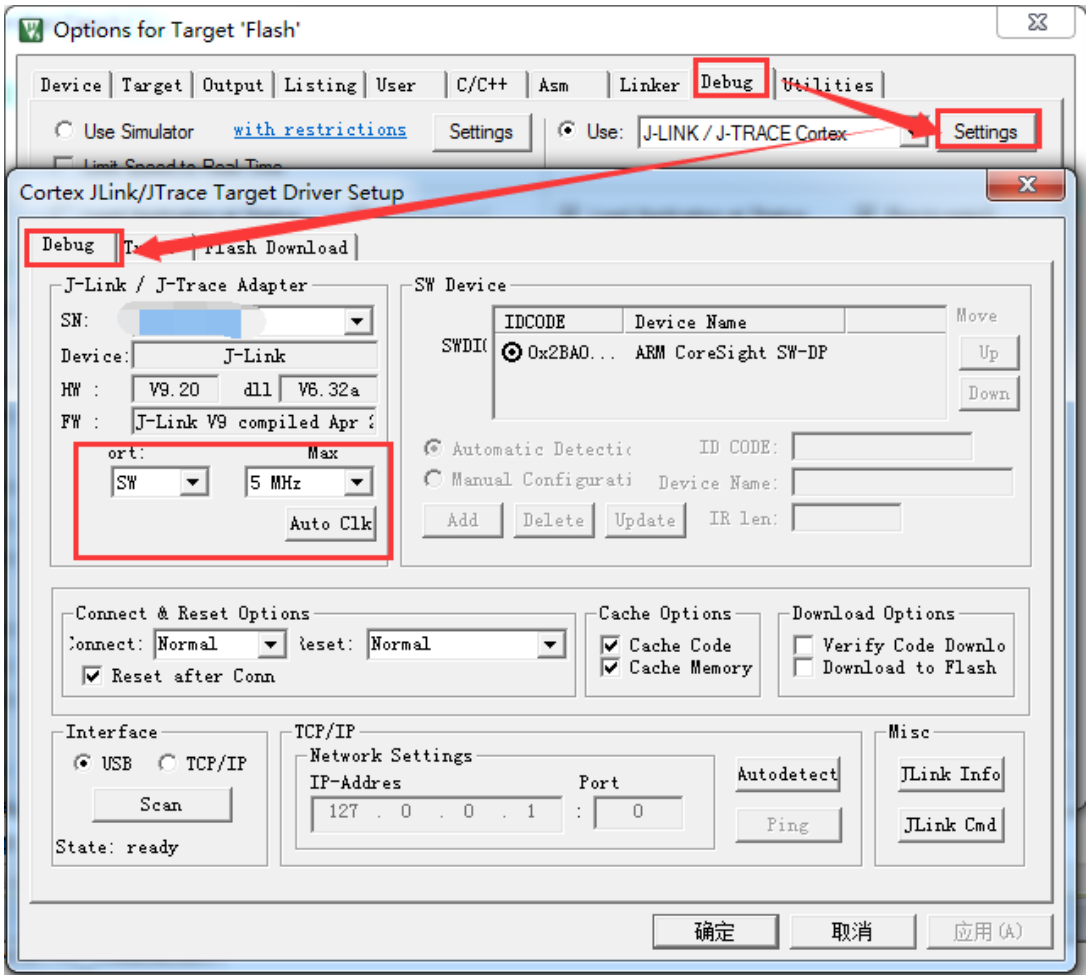


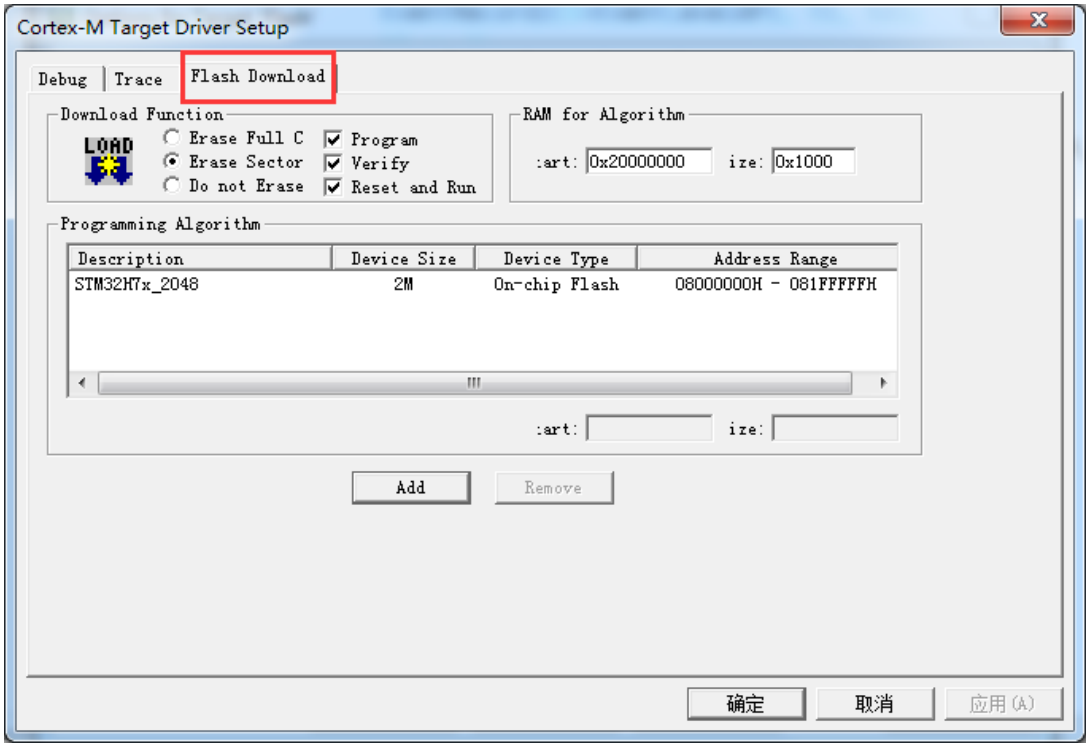
另外，进入调试状态后，右下角的时间是否正常更新都没有关系：

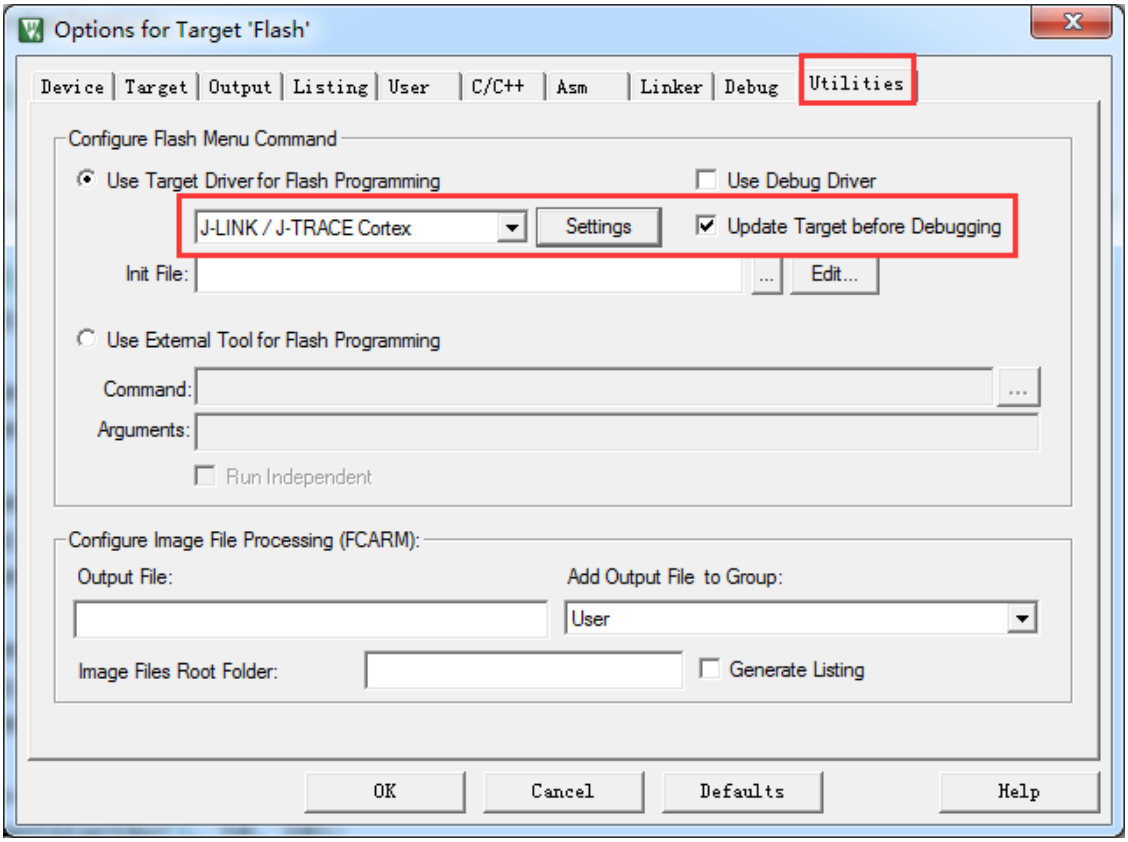


◆ 其它选项配置如下（只要大家的工程能够正常调试，配置就是没问题的）：




Options for Target 'Flash'
Device
Target
Output
Listing
User
C/C++
Asm
Linker
Debug
Utilities
Use Simulator
with restrictions
Settings
Use:
J-LINK / J-TRACE Cortex
Settings
Cortex JLink/JTrace Target Driver Setup
Debug
Flash Download
J-Link / J-Trace Adapter
SN:
Device:
J-Link
HW :
V9.20
dll
V6.32a
FW :
J-Link V9 compiled Apr
Max
SW
5 MHz
Auto Clk
SW Device
IDCODE
Device Name
SWDI(
0x2BA0...
ARM CoreSight SW-DP
Move
Up
Down
Automatic Detecti
ID CODE:
Manual Configurati
Device Name:
Add
Delete
Update
IR len:
Connect & Reset Options
Normal
Normal
Reset after Conn
Cache Options
Cache Code
Cache Memory
Download Options
Verify Code Downlo
Download to Flash
Interface
USB
TCP/IP
Scan
State: ready
TCP/IP
Network Settings
IP-Addres
Port
127 . 0 . 0 . 1
0
Autodetect
Ping
Misc
JLink Info
JLink Cmd
确定
取消
应用 (A)

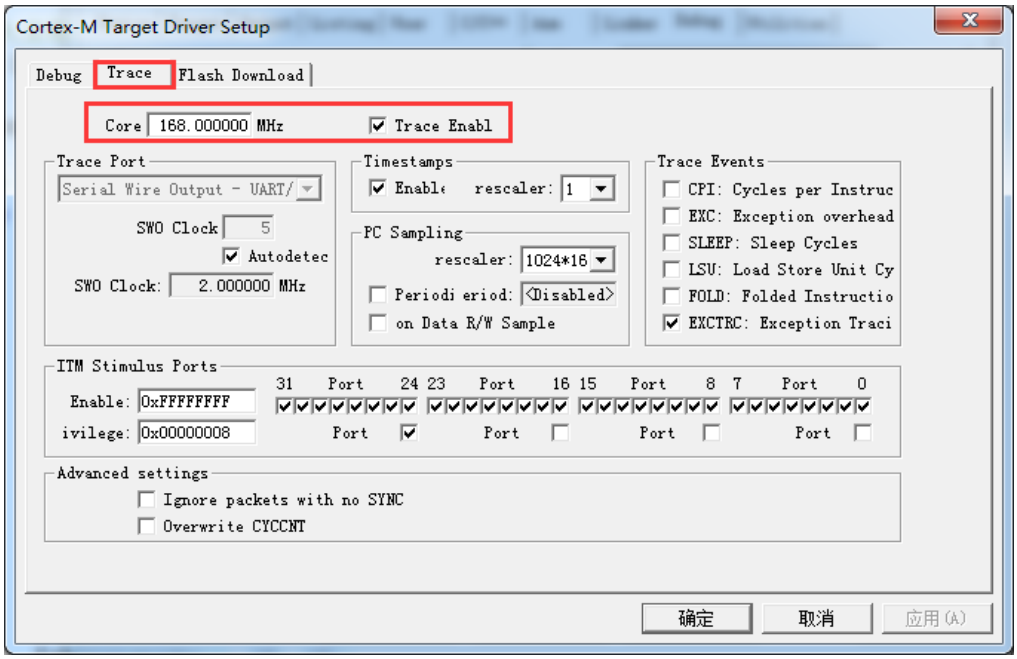

Cortex-M Target Driver Setup
Debug
Trace
Flash Download
Download Function
LOAD
Erase Full C
Erase Sector
Do not Erase
Program
Verify
Reset and Run
RAM for Algorithm
Start: 0x20000000
Size: 0x1000
Programming Algorithm
Description
Device Size
Device Type
Address Range
STM32H7x_2048
2M
On-chip Flash
08000000H - 081FFFFFH
Start:
Size:
Add
Remove
确定
取消
应用 (A)



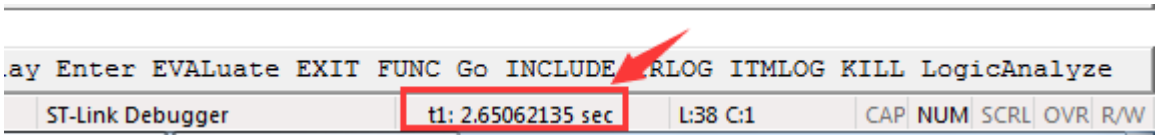
8.10 STLINK 配置说明

为了帮助大家更好的使用 STLINK，这里将 STLINK 配置中关键的几个地方做个说明。

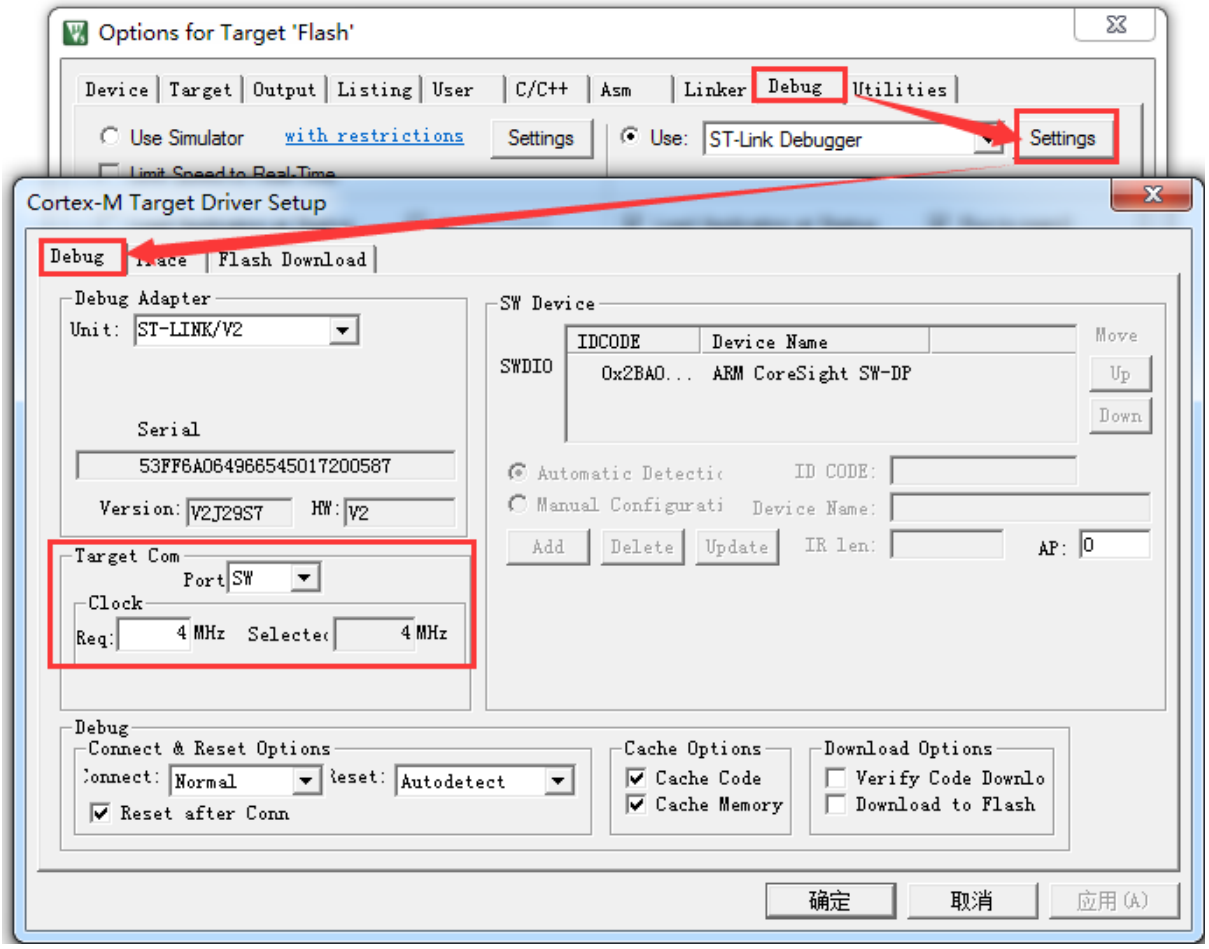
- 下面这个地方最重要，一定要正确设置当前系统工作的主频，如果不正确，会导致 Event Statistics 的时间统计是不正确的（对于 H7，Core 部分要填 400MHz）。

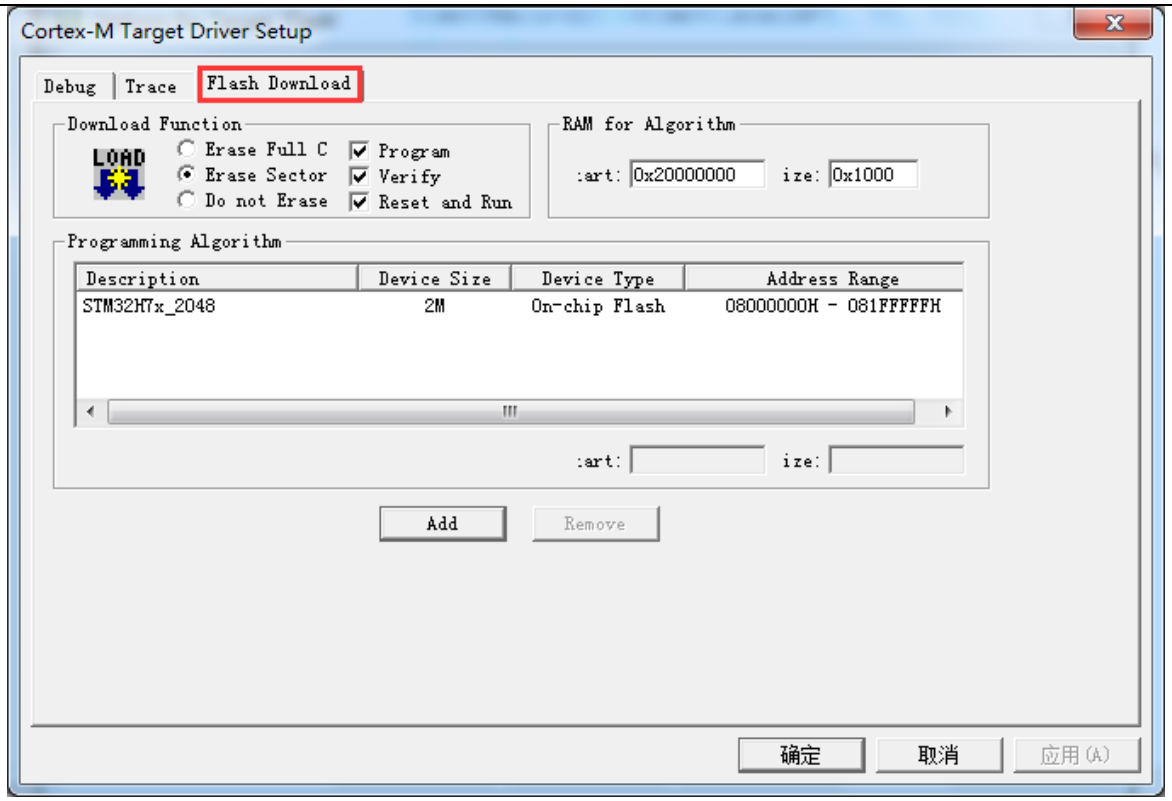


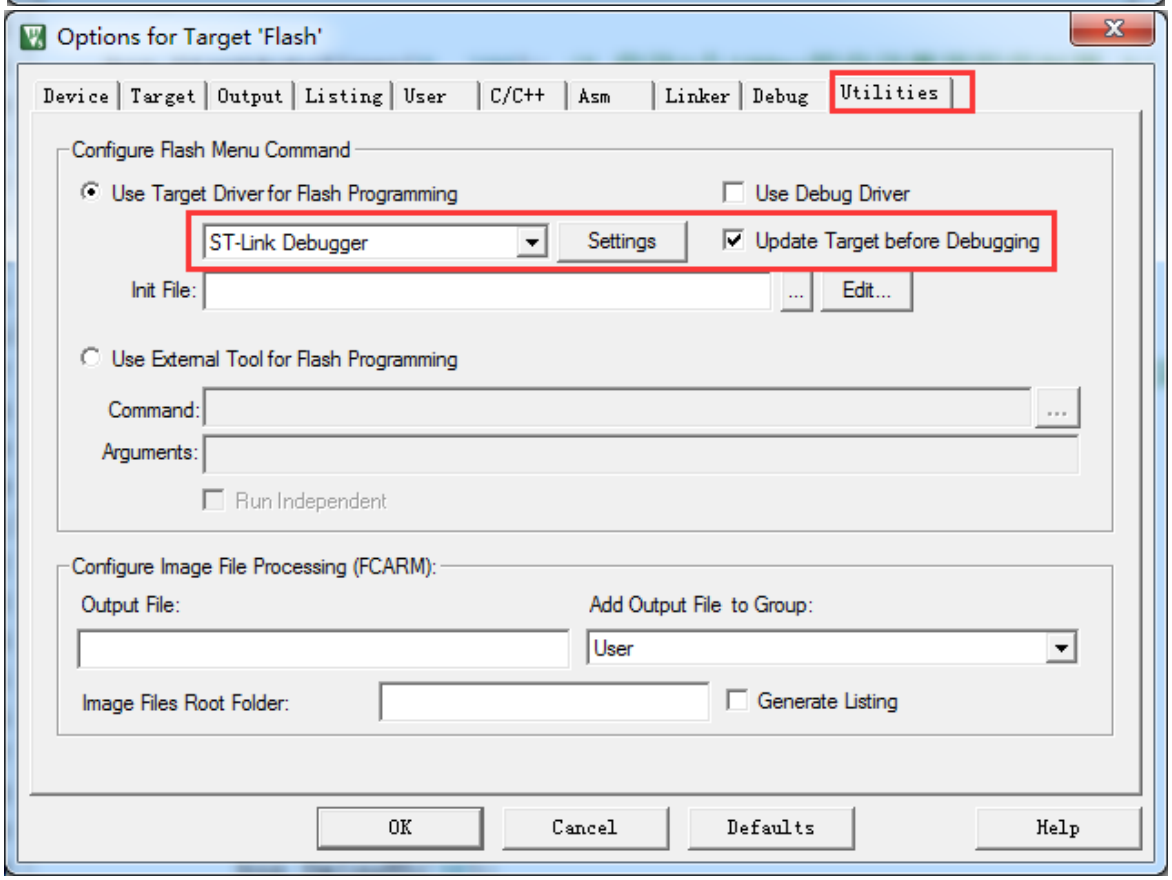
另外注意，进入调试状态后，右下角的时间是否正常更新都没有关系：



◆ 其它选项配置如下（只要大家的工程能够正常调试，配置就是没问题的）：



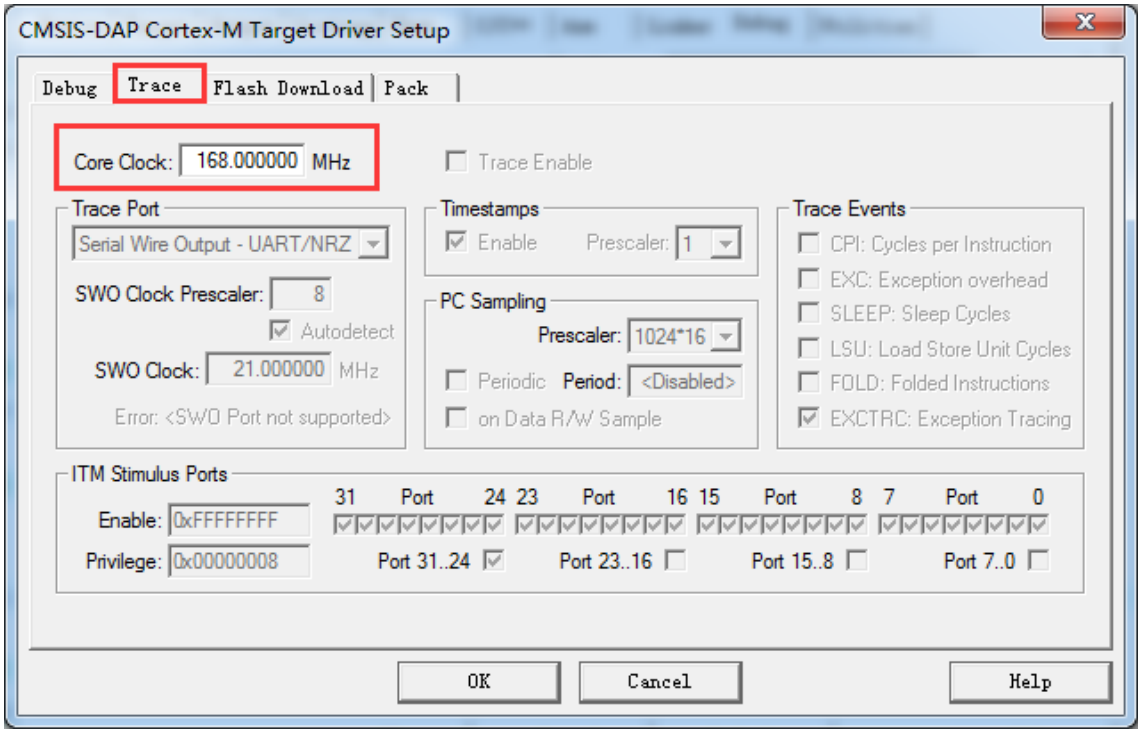




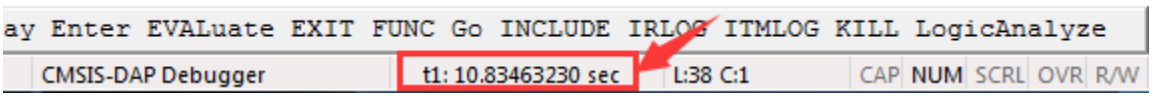
8.11 CMSIS-DAP 配置说明

为了帮助大家更好的使用 CMSIS-DAP，这里将 CMSIS-DAP 配置中关键的几个地方做个说明。

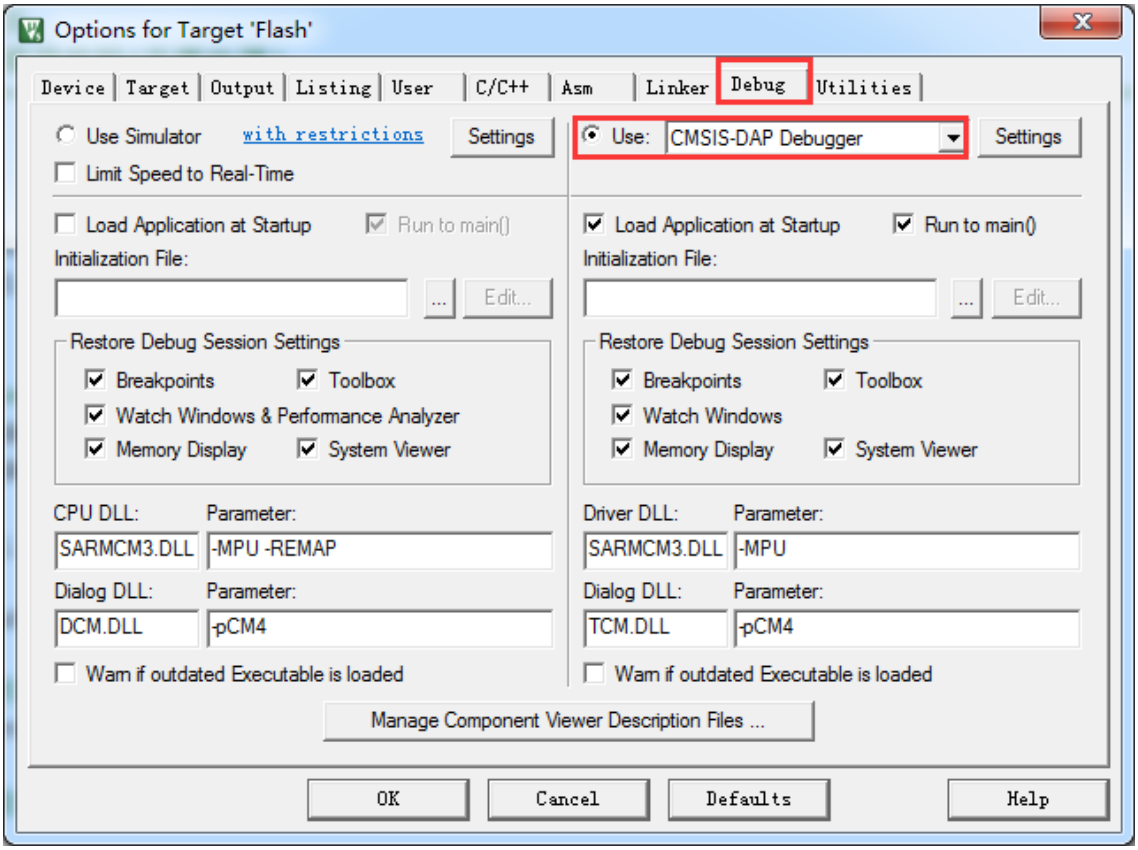
- 下面这个地方最重要，一定要正确设置当前系统工作的主频，如果不正确，会导致 Event Statistics 的时间统计不正确（对于 H7，Core 部分要填 400MHz）。

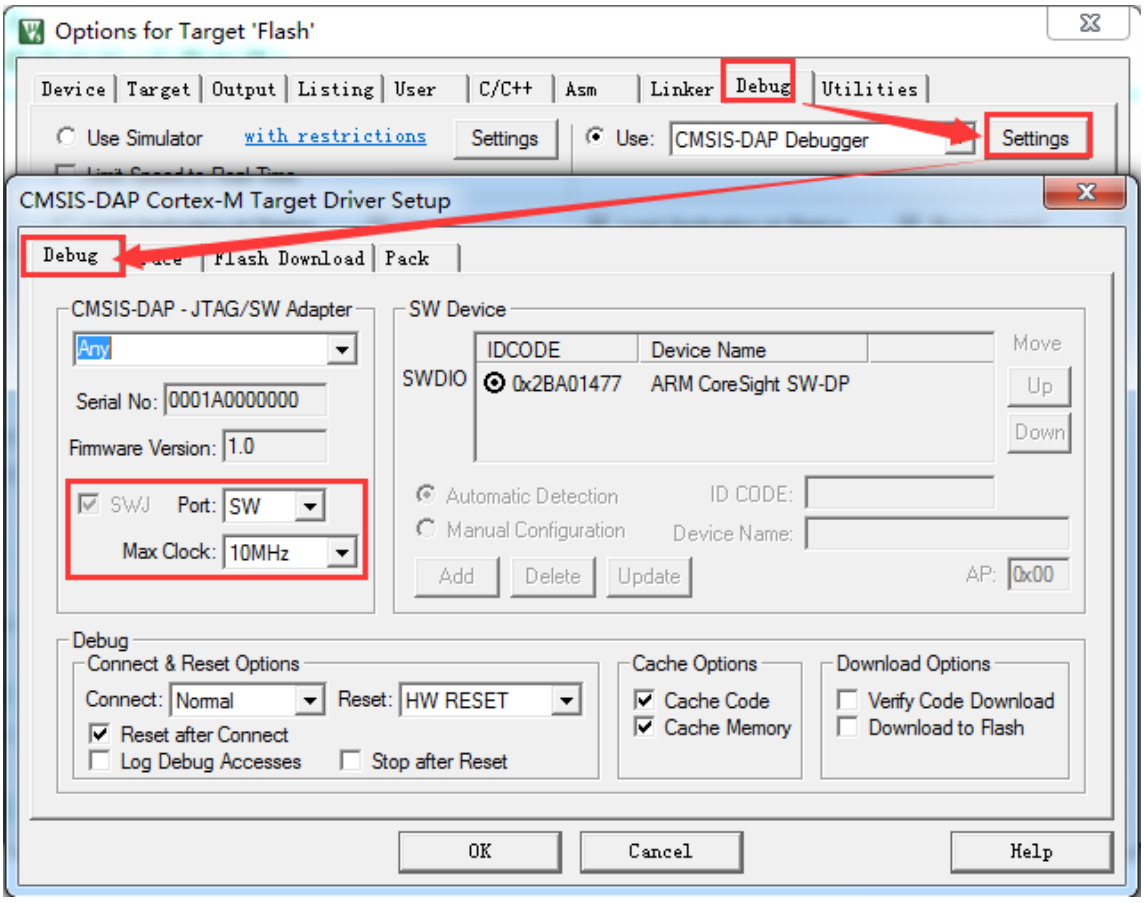


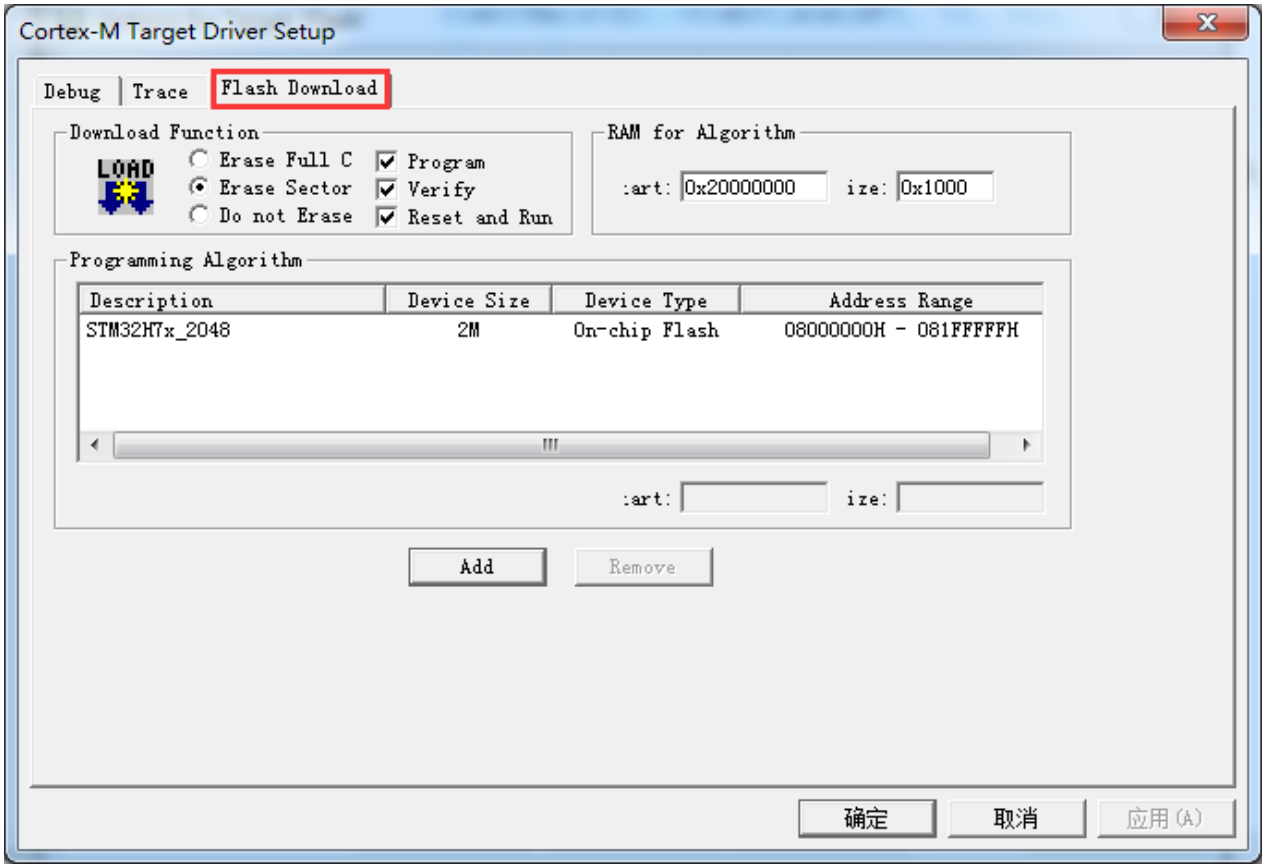
另外注意，进入调试状态后，右下角的时间是否正常更新都没有关系：

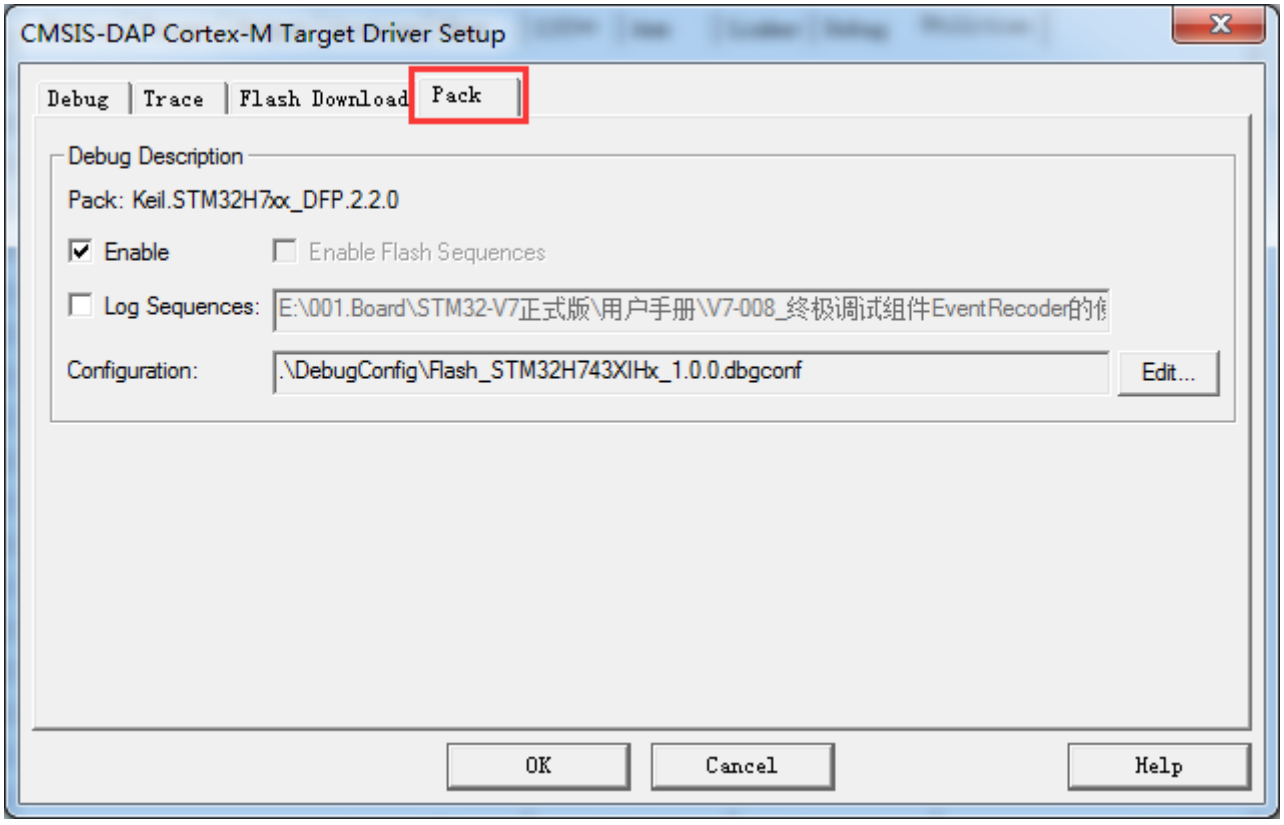


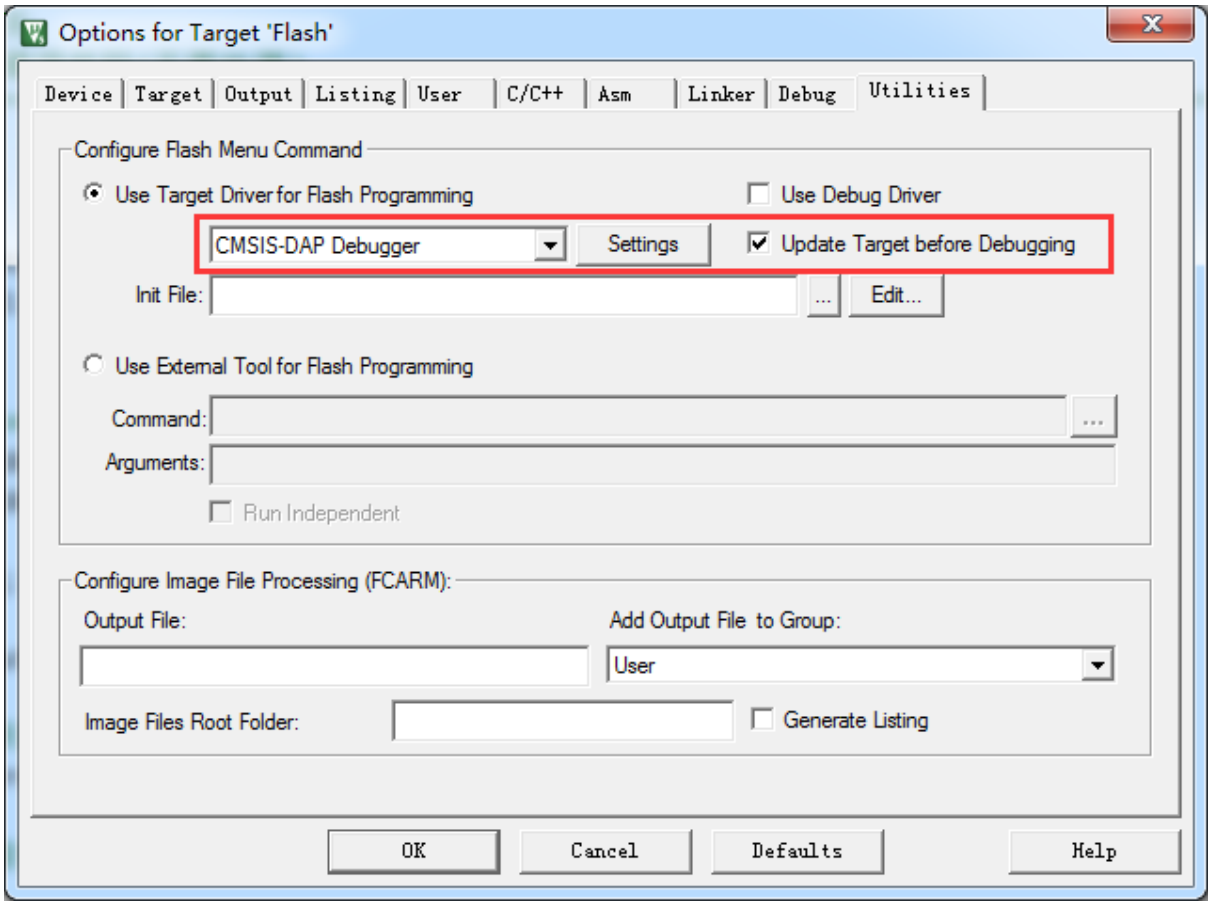
- 其它选项配置如下（只要大家的工程能够正常调试，配置就是没问题的）：


Options for Target 'Flash'
Device
Target
Output
Listing
User
C/C++
Asm
Linker
Debug
Utilities
Use Simulator
with restrictions
Settings
Use:
CMSIS-DAP Debugger
Settings
Limit Speed to Real-Time
Load Application at Startup
Run to main()
Initialization File:
Edit...
Restore Debug Session Settings
Breakpoints
Toolbox
Watch Windows & Performance Analyzer
Watch Windows
Memory Display
System Viewer
CPU DLL:
Parameter:
SARMCM3.DLL
-MPU -REMAP
Driver DLL:
-MPU
Dialog DLL:
DCM.DLL
-pCM4
TCM.DLL
Warn if outdated Executable is loaded
Manage Component Viewer Description Files ...
OK
Cancel
Defaults
Help


Options for Target 'Flash'
CMSIS-DAP Cortex-M Target Driver Setup
Debug
Flash Download
Pack
CMSIS-DAP - JTAG/SW Adapter
Any
Serial No:
0001A0000000
Firmware Version:
1.0
SWJ
Port:
SW
Max Clock:
10MHz
SW Device
SWDIO
IDCODE
Device Name
0x2BA01477
ARM CoreSight SW-DP
Move
Up
Down
Automatic Detection
Manual Configuration
ID CODE:
Device Name:
Add
Delete
Update
AP:
0x00
Debug
Connect & Reset Options
Connect:
Normal
Reset:
HW RESET
Reset after Connect
Log Debug Accesses
Stop after Reset
Cache Options
Cache Code
Cache Memory
Download Options
Verify Code Download
Download to Flash
OK
Cancel
Help

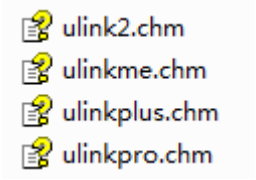

Cortex-M Target Driver Setup
Debug
Trace
Flash Download
Download Function
Erase Full C
Erase Sector
Do not Erase
Program
Verify
Reset and Run
RAM for Algorithm
:art: 0x20000000
ize: 0x1000
Programming Algorithm
Description
Device Size
Device Type
Address Range
STM32H7x_2048
2M
On-chip Flash
08000000H - 081FFFFFH
:art:
ize:
Add
Remove
确定
取消
应用(A)


CMSIS-DAP Cortex-M Target Driver Setup
Debug
Trace
Flash Download
Pack
Debug Description
Pack: Keil.STM32H7xx_DFP.2.2.0
Enable
Enable Flash Sequences
Log Sequences:
E:\001.Board\STM32-V7正式版\用户手册\V7-008_终极调试组件EventRecoder的使
Configuration:
.\DebugConfig\Flash_STM32H743XIHx_1.0.0.dbgconf
Edit...
OK
Cancel
Help



8.12 ULINK 配置说明

由于手头没有 ULINK，这里就不做讲解了。如果大家需要相关配置，按照前面小节三款 LINK 的配置照葫芦画瓢搞一下即可，或者在 MDK 安装目录的路径 ARM\Hlp 下有对应的文档说明：



8.13 配套例子

本章节教程配套了如下例程，仅 MDK 版本。

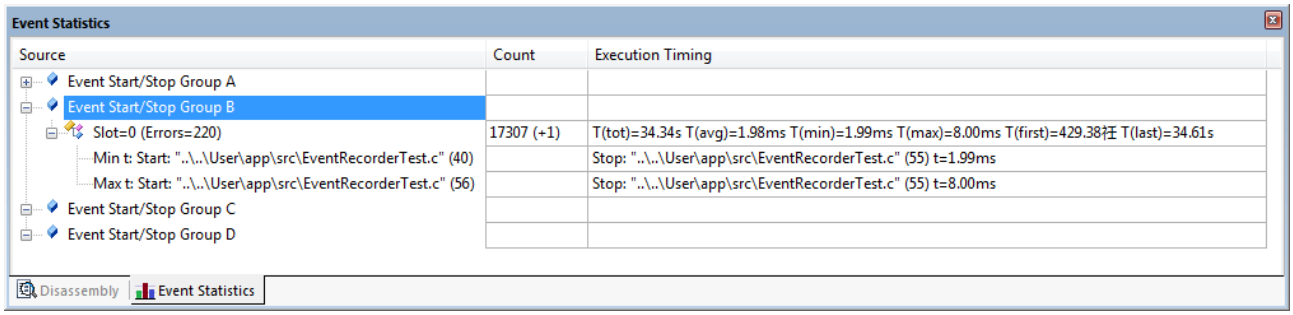
- V7-008_终极调试组件 EventRecoder 的使用

具体代码实现也比较简单，以 V7 开发板为例，定义一个 TIM6 的中断，中断频率是 500Hz，通过 Event Statistics 测量中断的执行频率。代码如下：

```
#include "bsp.h"
#include "EventRecorder.h"
```

```
/* 定时器频率，500Hz */
#define  timerINTERRUPT_FREQUENCY     500

/* 中断优先级 */
#define  timerHIGHEST_PRIORITY        10


/*
*********************************************************************************************************
*    函 数 名: vEventRecorderTest
*    功能说明: 创建定时器
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void vEventRecorderTest(void)
{
    bsp_SetTIMforInt(TIM6, timerINTERRUPT_FREQUENCY, timerHIGHEST_PRIORITY, 0);
    EventStartB(0);
}

/*
*********************************************************************************************************
*    函 数 名: TIM6_DAC_IRQHandler
*    功能说明: TIM6 中断服务程序。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void TIM6_DAC_IRQHandler( void )
{
    if((TIM6->SR & TIM_FLAG_UPDATE) != RESET)
    {
        EventStopB(0);
        EventStartB(0);

        /* 清除更新标志 */
        TIM6->SR = ~ TIM_FLAG_UPDATE;
    }
}
```

效果如下，测量的平均频率是 1.98ms，与我们设计的 500Hz 基本符合：



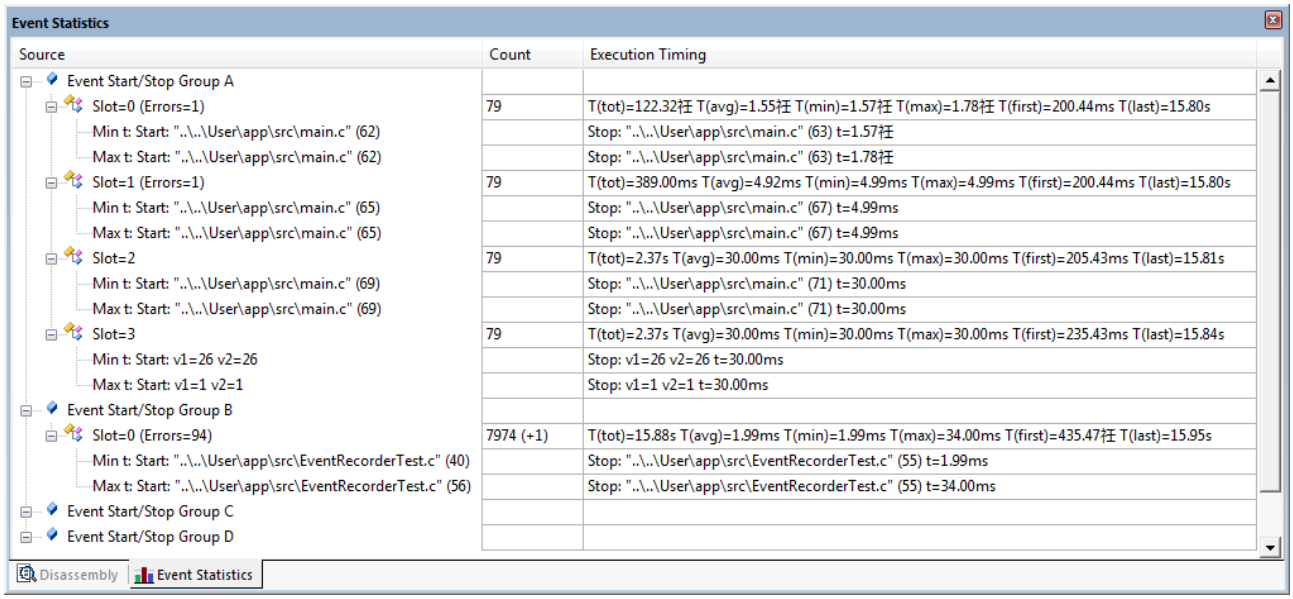
应用程序的设计如下：

```
#include "bsp.h"            /* 底层硬件驱动 */
#include "EventRecorder.h"


/*
*********************************************************************************************************
*                                              函数和变量
*********************************************************************************************************
*/
extern void vEventRecorderTest(void);
uint8_t s_ucBuf[10] = "armfly";


/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */
    uint32_t t0 = 0, t1 = 0, t2 = 0, t3 = 0, t4 = 0;


    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();

    bsp_Init();         /* 硬件初始化 */

    bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器 */

    /* 测量中断周期 */
    vEventRecorderTest();

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            EventStartA(0);
            EventStopA(0);

            EventStartA(1);
            bsp_DelayMS(5);
            EventStopA(1);

            EventStartA(2);
            bsp_DelayMS(30);
            EventStopA(2);
```

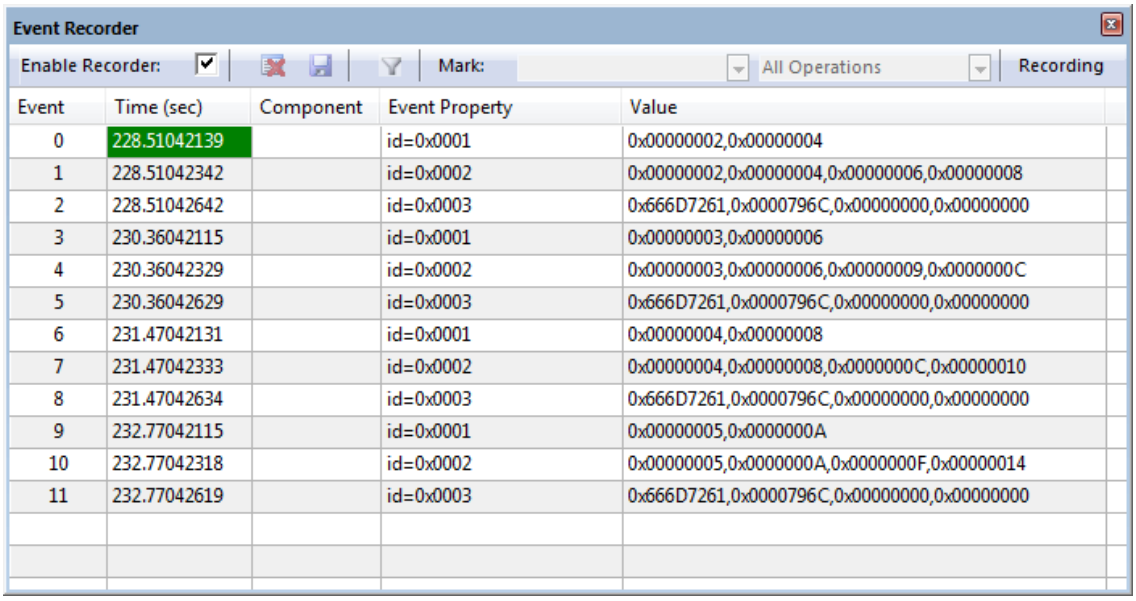
```
            t0++;
            EventStartAv(3, t0, t0);
            bsp_DelayMS(30);
            EventStopAv(3, t0, t0);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值，无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下 */
                    t1 += 1;
                    t2 += 2;
                    EventRecord2(1+EventLevelAPI, t1, t2);
                    t3 += 3;
                    t4 += 4;
                    EventRecord4(2+EventLevelOp, t1, t2, t3, t4);
                    EventRecordData(3+EventLevelOp, s_ucBuf, sizeof(s_ucBuf));
                    break;

                case KEY_DOWN_K2:           /* K2 键按下 */
                    printf("K2 按键按下\r\n");
                    break;

                case KEY_DOWN_K3:           /* K3 键按下 */
                    printf("K3 按键按下\r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

应用程序里面主要实现了三个功能：

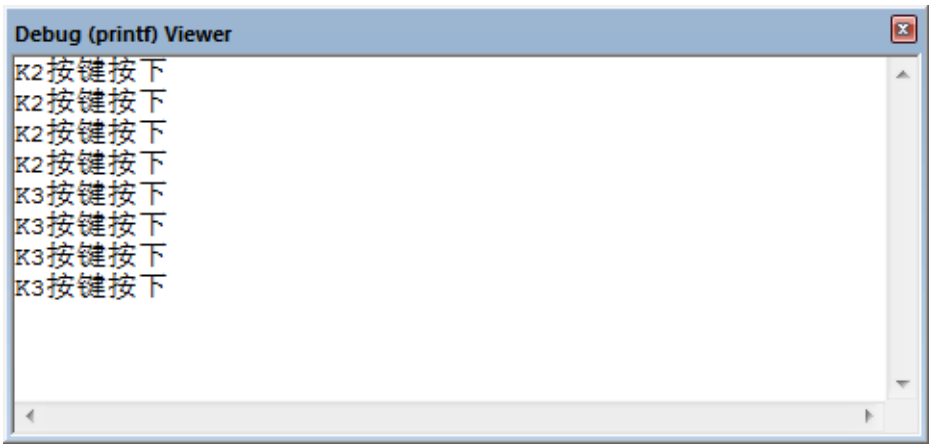
- 利用测量分组 A 实现 4 路时间的测量（第 1 路什么也没有测量，可以用来表示这两个函数本身执行占用的时间）。每 100ms 测量一次时间，效果如下：



◆ 利用函数 EventRecord2，EventRecord4 和 EventRecordData 发送消息事件。按下按键 K1 进行更新，效果如下：



◆ 基于 Event Recorder 的 printf 重定向。按下按键 K2 或者 K3 会打印消息，效果如下：



8.14 总结

Event Recoder 还是非常实用的，建议大家多使用几次，熟练掌握。基本用上几次就上瘾，离不开了，的确是工程调试的利器。

第9章 STM32H7 重要知识点数据类型，变量和堆栈

本章教程为大家介绍数据类型，变量和堆栈的相关知识。

9.1 初学者重要提示
9.2 数据类型
9.3 局部变量和全局变量
9.4 堆栈
9.5 局部变量，全局变量和堆栈实例
9.6 总结

9.1 初学者重要提示

- 如果对C语言不熟练的话，可以阅读C语言书：C Primer Plus（第五版）中文版.pdf
  论坛下载：http://www.armbbs.cn/forum.php?mod=viewthread&tid=91219。
- 为了更好的学习本章知识点，可以看之前做的视频教程第 10 章，针对 H7 也将在今年发布视频教程：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=15408 。

9.2 数据类型

了解数据类型之前要对 ANSI C 和 ISO C 的发展史有个了解，特别是 C89，C99 和 C11 的由来。

9.2.1 ANSI C 和 ISO C 历史

- 1983 年，美国国家标准协会（ANSI）组成了一个委员会来创立 C 语言的标准。因为这个标准是 1989 年发布的，所以一般简称 C89 标准。有些人也把 C89 标准叫做 ANSI C。
- 在 1990 年，ANSI C89 标准被国际标准化组织(ISO)和国际电工委员会(IEC)采纳为国际标准，名叫 ISO/IEC 9899:1990 - Programming languages　C，有些人简称 C90 标准。因此，C89 和 C90 通常指同一个标准，一般更常用 C89 这种说法。
- 在 2000 年 3 月，国际标准化组织(ISO)和国际电工委员会(IEC)采纳了第二个 C 语言标准，名叫 ISO/IEC 9899:1999 - Programming languages -- C，简称 C99 标准。

- 在 2011 年 12 月，国际标准化组织(ISO)和国际电工委员会(IEC)采纳了第三个 C 语言标准，名叫 ISO/IEC 9899:2011 - Information technology -- Programming languages -- C,简称 C11 标准。它是 C 程序语言的最新标准。

对于我们常用的编译器 MDK 和 IAR 而已，C89，C99 和 C11 均支持。

9.2.2 ARM 架构（含 Cortex-M 系列）数据类型

ARM 架构（含 Cortex-M 系列）的数据类型定义如下：

Table 2.2 Size and Range of Data Types in ARM Architecture Including Cortex-M Processors

C and C99 (stdint.h) Data Type	Number of Bits	Range (Signed)	Range (Unsigned)
char, int8_t, uint8_t	8	−128 to 127	0 to 255
short int16_t, uint16_t	16	−32768 to 32767	0 to 65535
int, int32_t, uint32_t	32	−2147483648 to 2147483647	0 to 4294967295
Long	32	−2147483648 to 2147483647	0 to 4294967295
long long, int64_t, uint64_t	64	−(2^63) to (2^63 - 1)	0 to (2^64 - 1)
Float	32	$-3.4028234 \times 10^{38}$ to 3.4028234×10^{38}	
Double	64	$-1.7976931348623157 \times 10^{308}$ to $1.7976931348623157 \times 10^{308}$	
long double	64	$-1.7976931348623157 \times 10^{308}$ to $1.7976931348623157 \times 10^{308}$	
Pointers	32	0x0 to 0xFFFFFFFF	
Enum	8 / 16/ 32	Smallest possible data type, except when overridden by compiler option	
bool (C++ only), _Bool (C only)	8	True or false	
wchar_t	16	0 to 65535	

9.2.3 头文件 stdint.h 对数据类型的定义

stdint.h 是 C99 中引进的一个标准 C 库的头文件。目前大部分单片机 C 编译器均支持，IAR 和 MDK 都支持。下面是部分内容（来自 MDK）。

```
/* exact-width signed integer types */
typedef   signed          char int8_t;
typedef   signed short     int int16_t;
typedef   signed           int int32_t;
typedef   signed       __INT64 int64_t;

/* exact-width unsigned integer types */
```

```
typedef unsigned          char uint8_t;
typedef unsigned short     int uint16_t;
typedef unsigned           int uint32_t;
typedef unsigned       __INT64 uint64_t;

/* 7.18.1.2 */

/* smallest type of at least n bits */
/* minimum-width signed integer types */
typedef   signed          char int_least8_t;
typedef   signed short     int int_least16_t;
typedef   signed           int int_least32_t;
typedef   signed       __INT64 int_least64_t;

/* minimum-width unsigned integer types */
typedef unsigned          char uint_least8_t;
typedef unsigned short     int uint_least16_t;
typedef unsigned           int uint_least32_t;
typedef unsigned       __INT64 uint_least64_t;

/* 7.18.1.3 */

/* fastest minimum-width signed integer types */
typedef   signed           int int_fast8_t;
typedef   signed           int int_fast16_t;
typedef   signed           int int_fast32_t;
typedef   signed       __INT64 int_fast64_t;

    /* fastest minimum-width unsigned integer types */
typedef unsigned           int uint_fast8_t;
typedef unsigned           int uint_fast16_t;
typedef unsigned           int uint_fast32_t;
typedef unsigned       __INT64 uint_fast64_t;
```

以 MDK5.26 为例，stdint.h 位于如下路径：

\Keil_v526\ARM\ARMCC\include

以 IAR8.X 为，stdint.h 位于如下路径：

\IAR Systems\Embedded Workbench 8.1\arm\inc\c

9.2.4 程序中推荐的变量命名方式

看程序的时候，经常会看到各种各样的变量命名方式，例如声明一个 8 位无符号整数，常见的有如下几种写法：

u8 err;

U8 err;

INT8U err;

UINT8 err;

uint8 err;

uint8_t err;

当大家阅读别人写的程序时，往往会看到风格各异的定义方式，移植部分程序时也不知道采用哪种方式更合适。

我们推荐大家采用最后一种定义方式，这种方法符合 C99 规范，ST 的固件库都是采用的这种类型定义方式。像我们开发板配套的 STM32 例程，从 2009 年最初的版本开始就一直沿用 C99 的标准写法来定义整数。

- 知识点拓展

针对变量声明问题，之前专门发过一个详细的帖子：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=501

9.3 局部变量和全局变量

9.3.1 局部变量

在一个函数内部定义的变量是内部变量，它只在本函数范围内有效，也就是说只有在本函数内才能使用它们，在此函数以外是不能使用这些变量的，这称为局部变量。

使用局部变量注意以下问题：

- 不同函数中可以使用相同名字的变量，它们代表不同的对象，互不干扰。
- 形式参数也是局部变量。
- 局部变量的作用域在函数内部。

9.3.2 全局变量

在函数内部定义的变量是局部变量，而在函数之外定义的变量称为外部变量，也就是全局变量。使用全局变量的注意事项：

- 全局变量可以为本文件中其他函数所共用。它的有效范围为从定义变量的位置开始到本源文件结束。
- 设置全局变量的作用是增加了函数间数据联系的渠道。
- 如果在同一个源文件中，外部变量和局部变量同名，则在局部变量的作用范围内，外部变量被“屏蔽”，即外部变量将不起作用。

9.3.3 使用全局变量的缺点

程序设计中，建议不要创建太多的全局变量，主要是出于以下三点考虑：

- 全局变量在程序的执行过程中都占用存储单元，而不是仅在需要时才占用存储单元。

- 函数的通用性降低了，因为函数在执行时要依赖于其所在的外部变量。如果将一个函数移植到另一个文件中，还要将有关的外部变量及其值一起移植过去。
- 使用全局变量过多，会降低程序的清晰性，特别是多个函数都调用此变量时。

9.3.4 变量的存储类别

从变量的作用域来分，可以分为全局变量和局部变量，而从变量值存在的时间来看，可以分为静态存储方式和动态存储方式。

- 静态存储方式：指在程序运行期间由系统分配固定的存储空间方式。
- 动态存储方式：在程序运行期间根据需要进行动态的分配存储空间方式。

全局变量存储在静态存储区中，动态存储区可以存放以下数据：

- 函数形式参数，在调用函数时给形参分配存储空间。
- 局部变量（未加 static 声明的局部变量）。
- 函数调用时的现场保护和返回地址等。

9.3.5 用 static 声明局部或者全局变量

有时候希望函数中的局部变量的值在函数调用结束后不消失而保留原值，即占用的存储单元不释放，在下一次该函数调用时，该变量已有值，就是上一次函数调用结束时的值。这时可以使用关键字 static 进行声明。

用 static 声明一个变量的作用：

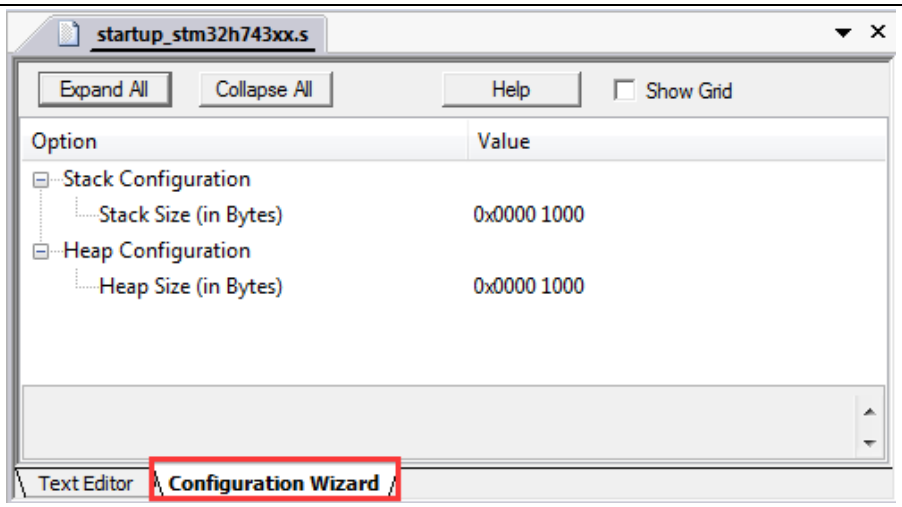
- 对局部变量用 static 声明，则使用该变量在整个程序执行期间不释放，为其分配的的空间始终存在。
- 全局变量用 static 声明，则该变量的作用域只限于本文件模块（即被声明的文件中）。

9.4 堆栈

9.4.1 堆栈作用

栈(stack)空间，用于局部变量，函数调时现场保护和返回地址，函数的形参等。

堆(heap)空间，主要用于动态内存分配，也就是说用 malloc，calloc, realloc 等函数分配的变量空间是在堆上。以 STM32H7 为例，堆栈是在 startup_stm32h743xx.s 文件里面设置：



9.4.2 寄存器组（堆栈指针寄存器）

Cortex – M7/M4/M3 处理器拥有 R0-R15 的通用寄存器组。其中 R13 作为堆栈指针 SP。SP 有两个，但在同一时刻只能有一个可以用。

- 主堆栈指针（MSP）：这是缺省的堆栈指针，它由 OS 内核、异常服务例程以及所有需要特权访问的应用程序代码来使用。
- 进程堆栈指针（PSP）：用于常规的应用程序代码（不处于异常服务例程中时）。

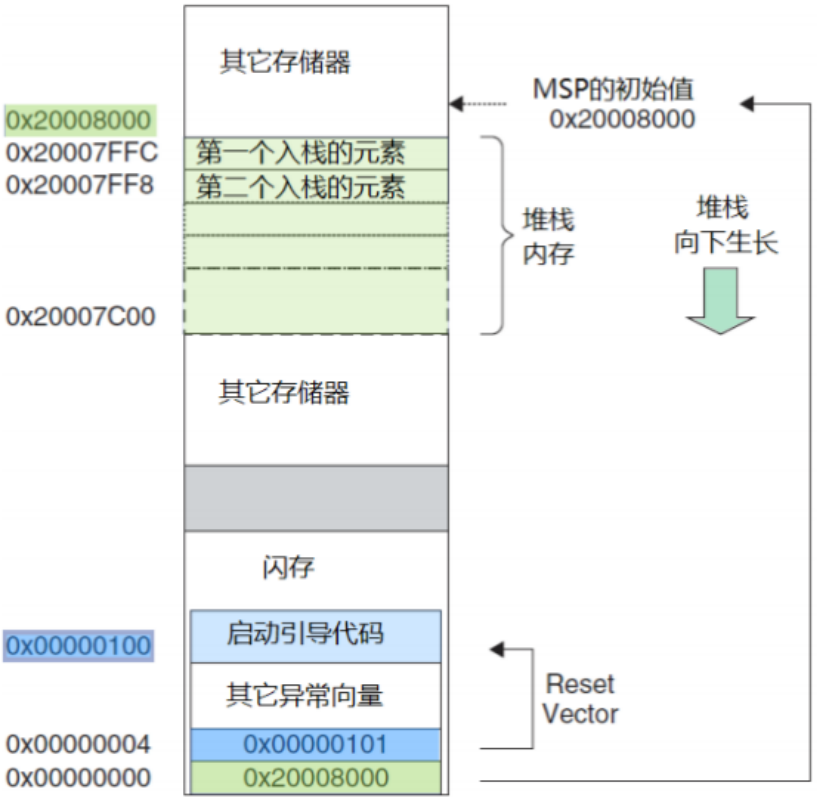
另外以下两点要注意：

- 大多数情况下的应用，只需使用指针 MSP，而 PSP 多用于 RTOS 中。
- R13 的最低两位被硬线连接到 0，并且总是读出 0，这意味着堆栈总是 4 字节对齐的。

R0	通用寄存器
R1	通用寄存器
R2	通用寄存器
R3	通用寄存器
R4	通用寄存器
R5	通用寄存器
R6	通用寄存器
R7	通用寄存器
R8	通用寄存器
R9	通用寄存器
R10	通用寄存器
R11	通用寄存器
R12	通用寄存器
R13(MSP,PSP)	堆栈指针（SP）
R14	连接寄存器（LR）
R15	程序计数器（PC）

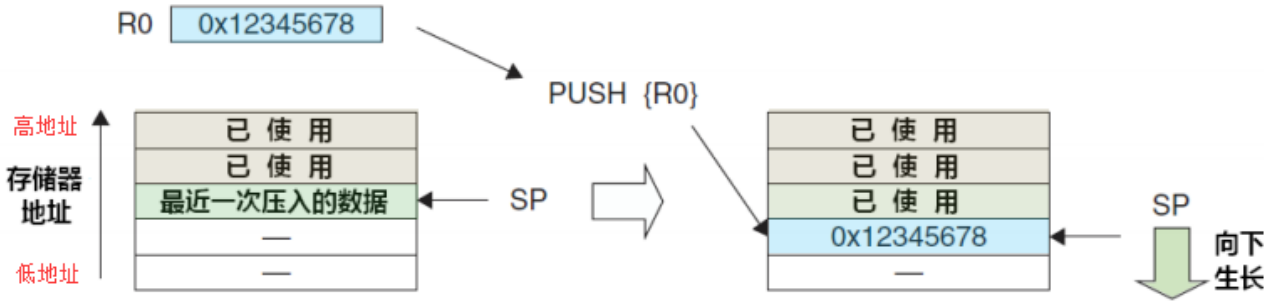
9.4.3 Cortex-M7/M4/M3 向下生长的满栈

这个知识点在以后用 H7 移植 RTOS 时，非常有用。

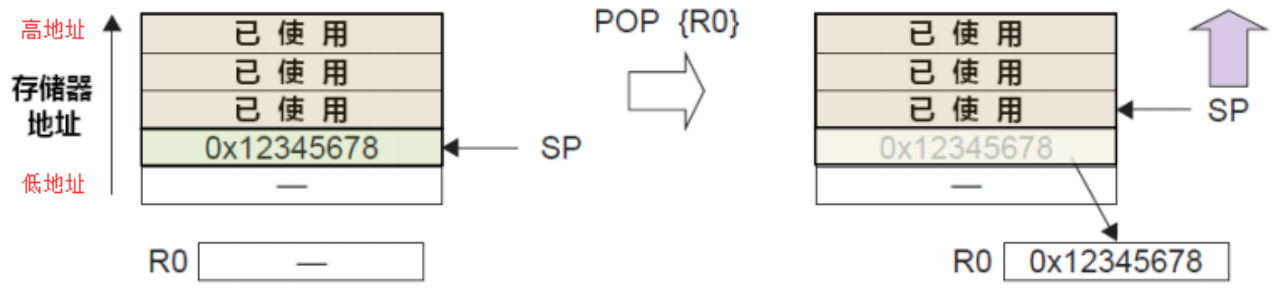


9.4.4 堆栈的基本操作

这里对入栈和出栈做个简单的介绍。PUSH 入栈操作：SP 先自减 4，再存入新的数值：



POP 出栈操作：先从 SP 指针处读出上一次被压入的值，再把 SP 指针自增 4：



9.5 局部变量，全局变量和堆栈实例

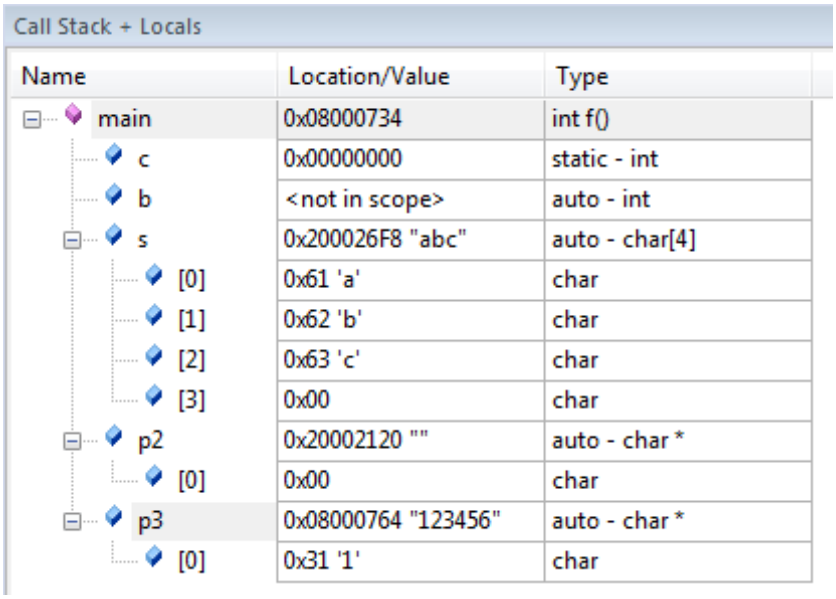
通过下面的实例可以对局部变量，全局变量和堆栈有个感性的认识：

```
uint32_t a = 0;            //全局初始化区，可以被其他 c 文件 extern 引用
static uint32_t ss = 0;    //静态变量，只允许在本文件使用
uint8_t *p1;               //全局未初始化区
int main(void)
{
    uint32_t b;             //栈
    uint8_t s[] = "abc";    //栈
    uint8_t *p2;            //栈
    uint8_t *p3 = "123456"; //123456\0 在常量区，p3 在栈上。
    static uint32_t c =0;   //全局（静态）初始化区

    p1 = (uint8_t *)malloc(10); //在堆区申请了 10 个字节空间
    p2 = (uint8_t *)malloc(20); //在堆区申请了 20 个字节空间
    strcpy(p1, "123456");       /* 123456 字符串（结束符号是 0(\0)，总长度 7）放在常量区，
                                   编译器可能会将它与 p3 所指向的"123456"优化成一个地方 */
}
```

通过查看 MAP 文件，可以看全局变量在 RAM 中的位置：

```
Symbol Name                     Value     Ov Type        Size     Object(Section)
a                               0x20000000   Data          4         main.o(.data)
p1                              0x2000000c   Data          4         main.o(.data)
ss                              0x20000004   Data          4         main.o(.data)
```

而局部变量要调整状态进入 main 函数里面查看：



9.6 总结

C 语言的基础知识点要掌握牢靠，对于后面学习 HAL 库源码大有裨益。

第10章 STM32H7 的 FLASH，RAM 和栈使用情况（map 和 htm 文件）

本章为大家介绍编译器生成的 map 和 htm 文件进行解析，通过这两个文件可以让大家对工程代码的认识程度提升一个档次。

10.1 初学者重要提示
10.2 MAP 文件分析
10.3 HTM 文件分析
10.4 总结

10.1 初学者重要提示

- 为了更好的学习本章知识点，可以看之前做的视频教程第 11 章，针对 H7 也将在今年发布视频教程：http://www.armbbs.cn/forum.php?mod=viewthread&tid=15408 。
- 本章节以 MDK 为例进行说明，使用 IAR 同理。

10.2 MAP 文件分析

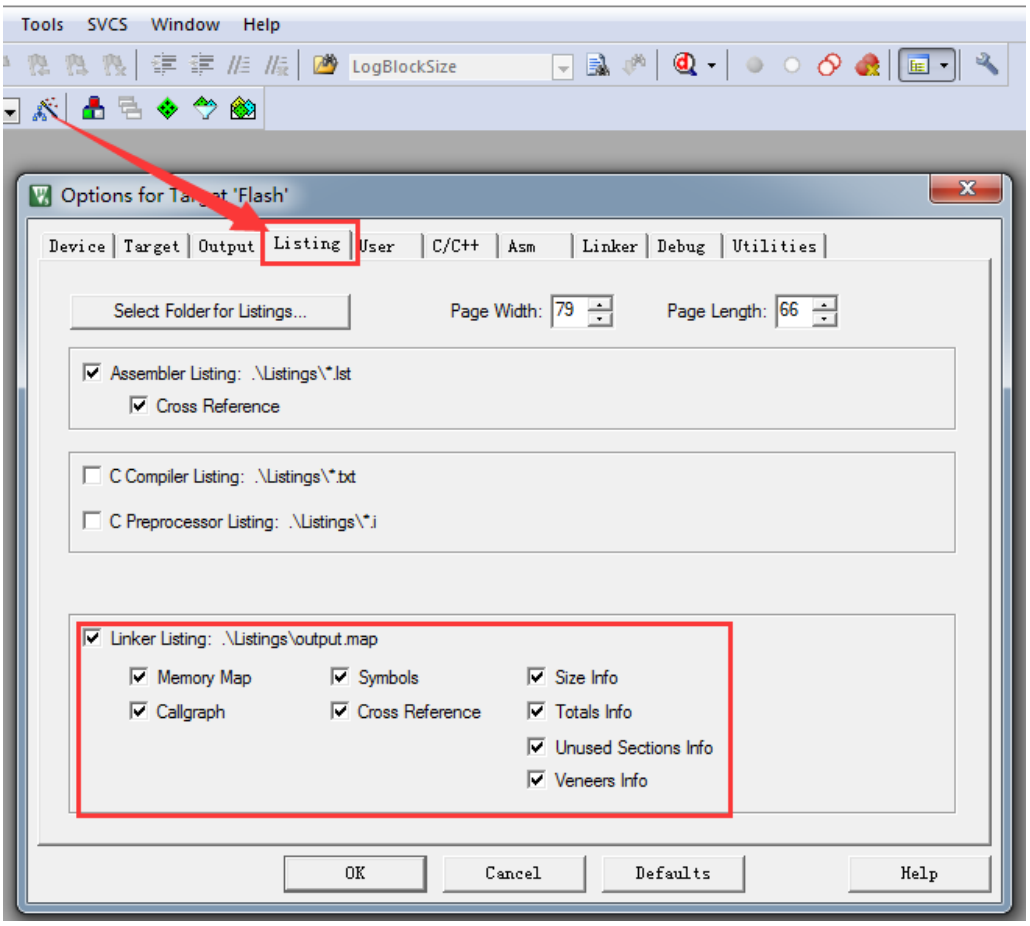
通过 map 文件，可以方便的查看工程 ROM/FLASH 和 RAM 的占用情况，包括单个源文件，甚至具体到每个函数的 rom 大小都给出了，通过这些信息可以很好的进行代码优化。MAP 文件的内容可分为如下几部分：

- Section Cross References
- Removing Unused input sections from the image
- Image Symbol Table (Local Symbols Global Symbols)
- Memory Map of the image
- Image component sizes

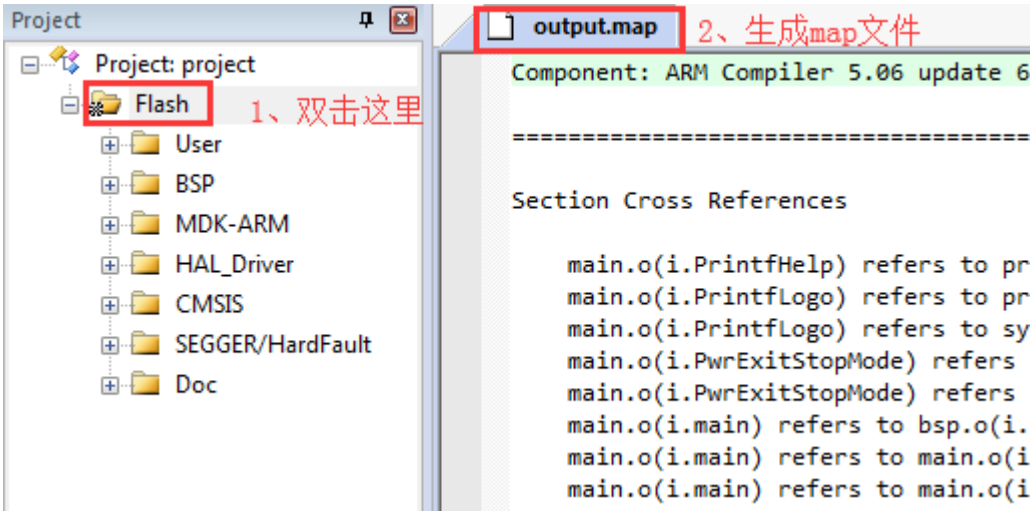
下面将这个几个部分的含义逐一为大家做个说明。

10.2.1 MDK 配置

要生成 MAP 文件，MDK 中如下选项要选上：



将工程全编译，且没有错误后，双击这里就可以看到生成的 map 文件了：



10.2.2 MAP 文件相关概念

认识 MAP 文件前先熟悉如下概念：

- 段(section) ： 描述映像文件的代码和数据块。

- RO： Read-Only 的缩写，包括 RO-data(只读数据)和 RO-code(代码)。
- RW：Read-Write 的缩写，主要是 RW-data，RW-data 由程序初始化初始值。
- ZI： Zero-initialized 的缩写，主要是 ZI-data，由编译器初始化为 0。
- .text：与 RO-code 同义。
- .constdata：与 RO-data 同义。
- .bss： 与 ZI-data 同义。
- .data：与 RW-data 同义

10.2.3 Section Cross References

这部分主要是不同文件中函数的调用关系（部分截图）

```
Section Cross References

    main.o(i.PrintfHelp) refers to printf8.o(i.__0printf$8) for __2printf
    main.o(i.PrintfLogo) refers to printf8.o(i.__0printf$8) for __2printf
    main.o(i.PrintfLogo) refers to system_stm32h7xx.o(.data) for SystemCoreClock
    main.o(i.PwrExitStopMode) refers to memseta.o(.text) for __aeabi_memclr4
    main.o(i.PwrExitStopMode) refers to stm32h7xx_hal_gpio.o(i.HAL_GPIO_Init) for HAL_GPIO_Init
    main.o(i.main) refers to bsp.o(i.bsp_Init) for bsp_Init
    main.o(i.main) refers to main.o(i.PwrExitStopMode) for PwrExitStopMode
    main.o(i.main) refers to main.o(i.PrintfLogo) for PrintfLogo
    main.o(i.main) refers to main.o(i.PrintfHelp) for PrintfHelp
    main.o(i.main) refers to bsp_timer.o(i.bsp_StartAutoTimer) for bsp_StartAutoTimer
    main.o(i.main) refers to bsp.o(i.bsp_Idle) for bsp_Idle
    main.o(i.main) refers to bsp_timer.o(i.bsp_CheckTimer) for bsp_CheckTimer
    main.o(i.main) refers to bsp_led.o(i.bsp_LedToggle) for bsp_LedToggle
    main.o(i.main) refers to bsp_key.o(i.bsp_GetKey) for bsp_GetKey
    main.o(i.main) refers to printf8.o(i.__0printf$8) for __2printf
    main.o(i.main) refers to stm32h7xx_hal_pwr.o(i.HAL_PWR_EnterSTOPMode) for HAL_PWR_EnterSTOPMode
    main.o(i.main) refers to bsp.o(i.SystemClock_Config) for SystemClock_Config
    bsp.o(i.CPU_CACHE_Enable) refers to bsp.o(i.SCB_EnableICache) for SCB_EnableICache
    bsp.o(i.CPU_CACHE_Enable) refers to bsp.o(i.SCB_EnableDCache) for SCB_EnableDCache
    bsp.o(i.MPU_Config) refers to stm32h7xx_hal_cortex.o(i.HAL_MPU_Disable) for HAL_MPU_Disable
    bsp.o(i.MPU_Config) refers to stm32h7xx_hal_cortex.o(i.HAL_MPU_ConfigRegion) for HAL_MPU_ConfigRegion
    bsp.o(i.MPU_Config) refers to stm32h7xx_hal_cortex.o(i.HAL_MPU_Enable) for HAL_MPU_Enable
```

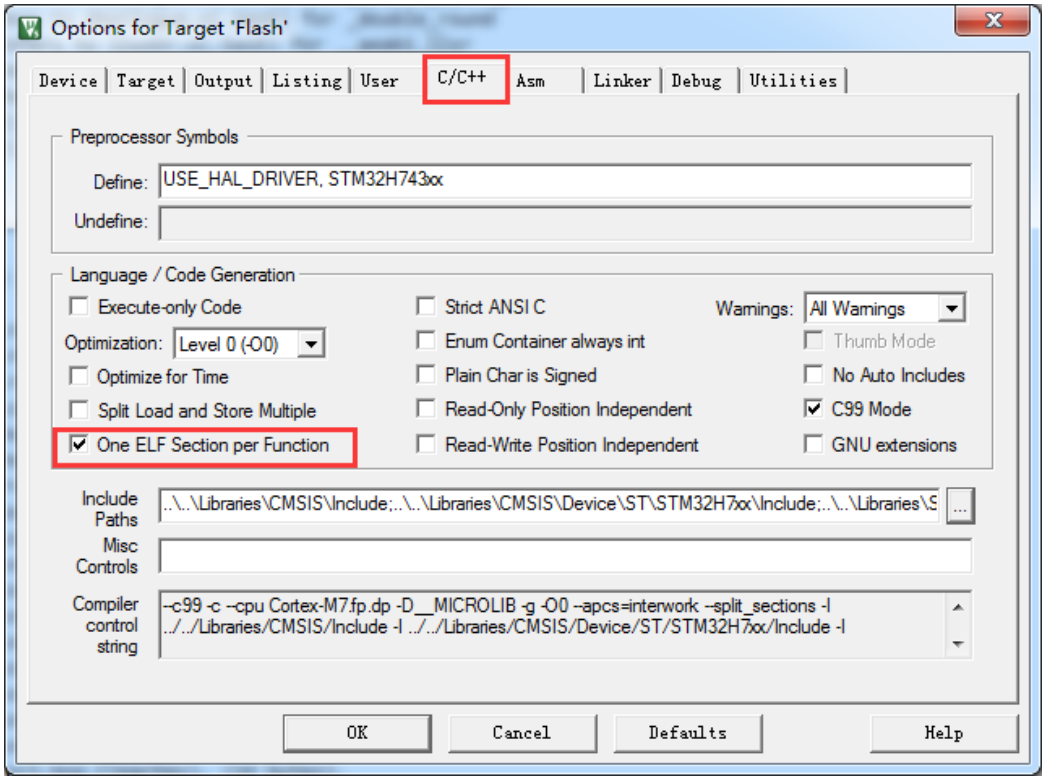
比如这句：main.o(i.main) refers to main.o(i.PrintfLogo) for PrintfLogo 表示 main.c 文件中的 main 函数调用了 PrintfLogo。

10.2.4 Removing Unused input sections from the image

这部分主要是被删除的冗余函数，也就是添加到工程里面，但是没有调用到的，下面是部分被删除冗余函数的截图效果：

```
Removing Unused input sections from the image.

    Removing main.o(.rev16_text), (4 bytes).
    Removing main.o(.revsh_text), (4 bytes).
    Removing main.o(.rrx_text), (6 bytes).
    Removing main.o(.constdata), (48 bytes).
    Removing bsp.o(.rev16_text), (4 bytes).
    Removing bsp.o(.revsh_text), (4 bytes).
    Removing bsp.o(.rrx_text), (6 bytes).
    Removing bsp.o(.constdata), (48 bytes).
    Removing bsp_led.o(.rev16_text), (4 bytes).
    Removing bsp_led.o(.revsh_text), (4 bytes).
    Removing bsp_led.o(.rrx_text), (6 bytes).
    Removing bsp_led.o(i.bsp_IsLedOn), (58 bytes).
    Removing bsp_led.o(i.bsp_LedOn), (62 bytes).
    Removing bsp_led.o(.constdata), (48 bytes).
    Removing bsp_key.o(.rev16_text), (4 bytes).
    Removing bsp_key.o(.revsh_text), (4 bytes).
    Removing bsp_key.o(.rrx_text), (6 bytes).
```

对于这个部分功能，用户最好将 MDK 中这个选项勾上，然后全编译工程，效果会比较好：



10.2.5 Image Symbol Table

Image Symbol Table 主要分为两类，分别是 Local Symbols 和 Global Symbols。

- Local Symbols

Local Symbols 记录了用 static 声明的全局变量地址和大小，C 文件中函数的地址和用 static 声明的函数代码大小，汇编文件中的标号地址（作用域限本文件），下面是部分截图：

```
Image Symbol Table

    Local Symbols

      符号名                                地址      符号类型      大小  所在对象(段)名
    Symbol Name                             Value     Ov Type       Size  Object(Section)

    cdrcmple.s                              0x00000000   Number         0  cdrcmple.o ABSOLUTE
    dc.s                                    0x00000000   Number         0  dc.o ABSOLUTE
    handlers.s                              0x00000000   Number         0  handlers.o ABSOLUTE
    init.s                                  0x00000000   Number         0  init.o ABSOLUTE
    RESET                                   0x08000000   Section      664  startup_stm32h743xx.o(RESET)
    .ARM.Collect$$$$00000000                0x08000298   Section        0  entry.o(.ARM.Collect$$$$00000000)
    .ARM.Collect$$$$00000001                0x08000298   Section        4  entry2.o(.ARM.Collect$$$$00000001)
    .ARM.Collect$$$$00000004                0x0800029c   Section        4  entry5.o(.ARM.Collect$$$$00000004)
    .ARM.Collect$$$$00000008                0x080002a0   Section        0  entry7b.o(.ARM.Collect$$$$00000008)
    .ARM.Collect$$$$0000000A                0x080002a0   Section        0  entry8b.o(.ARM.Collect$$$$0000000A)
    .ARM.Collect$$$$0000000B                0x080002a0   Section        8  entry9a.o(.ARM.Collect$$$$0000000B)
    .ARM.Collect$$$$0000000D                0x080002a8   Section        0  entry10a.o(.ARM.Collect$$$$0000000D)
    .ARM.Collect$$$$0000000F                0x080002a8   Section        0  entry11a.o(.ARM.Collect$$$$0000000F)
    .ARM.Collect$$$$00002712                0x080002a8   Section        4  entry2.o(.ARM.Collect$$$$00002712)
    __lit__00000000                         0x080002a8   Data           4  entry2.o(.ARM.Collect$$$$00002712)
    .text                                   0x080002ac   Section       36  startup_stm32h743xx.o(.text)
    $v0                                     0x080002ac   Number         0  startup_stm32h743xx.o(.text)
    .text                                   0x080002d0   Section        0  uldiv.o(.text)
    .text                                   0x08000332   Section        0  memseta.o(.text)
    .text                                   0x08000356   Section        0  llshl.o(.text)
    .text                                   0x08000374   Section        0  llushr.o(.text)
    .text                                   0x08000394   Section       36  init.o(.text)
    OSKERNEL                                0x080003b8   Section       18  hardfaulthandler.o(OSKERNEL)
    $v0                                     0x080003b8   Number         0  hardfaulthandler.o(OSKERNEL)
    i.BusFault_Handler                      0x080003ca   Section        0  stm32h7xx_it.o(i.BusFault_Handler)
    i.CPU_CACHE_Enable                      0x080003ce   Section        0  bsp.o(i.CPU_CACHE_Enable)
```

◆ Global Symbols

Global Symbols 记录了全局变量的地址和大小，C 文件中函数的地址及其代码大小，汇编文件中的标号地址（作用域全工程），下面是部分截图：

```
Global Symbols

  符号名                                地址      符号类型      大小  所在对象(段)名
Symbol Name                             Value     Ov Type       Size  Object(Section)

HAL_GPIO_Init                           0x08000521   Thumb Code   534  stm32h7xx_hal_gpio.o(i.HAL_GPIO_Init)
HAL_GetTick                             0x0800076d   Thumb Code     6  stm32h7xx_hal.o(i.HAL_GetTick)
HAL_IncTick                             0x08000779   Thumb Code    16  stm32h7xx_hal.o(i.HAL_IncTick)
HAL_Init                                0x08000791   Thumb Code    28  stm32h7xx_hal.o(i.HAL_Init)
HAL_InitTick                            0x080007ad   Thumb Code    64  stm32h7xx_hal.o(i.HAL_InitTick)
HAL_LPTIM_Init                          0x080007f9   Thumb Code   202  stm32h7xx_hal_lptim.o(i.HAL_LPTIM_Init)
HAL_LPTIM_MspInit                       0x080008d5   Thumb Code     2  stm32h7xx_hal_lptim.o(i.HAL_LPTIM_MspInit)
HAL_LPTIM_PWM_Start                     0x080008d7   Thumb Code    64  stm32h7xx_hal_lptim.o(i.HAL_LPTIM_PWM_Start)
HAL_MPU_ConfigRegion                    0x08000919   Thumb Code    88  stm32h7xx_hal_cortex.o(i.HAL_MPU_ConfigRegion)
HAL_MPU_Disable                         0x08000975   Thumb Code    38  stm32h7xx_hal_cortex.o(i.HAL_MPU_Disable)
HAL_MPU_Enable                          0x080009a1   Thumb Code    58  stm32h7xx_hal_cortex.o(i.HAL_MPU_Enable)
HAL_MspInit                             0x080009e1   Thumb Code     2  stm32h7xx_hal.o(i.HAL_MspInit)
HAL_NVIC_EnableIRQ                      0x080009e3   Thumb Code    32  stm32h7xx_hal_cortex.o(i.HAL_NVIC_EnableIRQ)
HAL_NVIC_SetPriority                    0x08000a03   Thumb Code   122  stm32h7xx_hal_cortex.o(i.HAL_NVIC_SetPriority)
HAL_NVIC_SetPriorityGrouping            0x08000a7d   Thumb Code    32  stm32h7xx_hal_cortex.o(i.HAL_NVIC_SetPriorityGrouping)
HAL_PWR_EnterSTOPMode                   0x08000aa5   Thumb Code   126  stm32h7xx_hal_pwr.o(i.HAL_PWR_EnterSTOPMode)
HAL_RCCEx_GetD1SysClockFreq             0x08000b2d   Thumb Code    44  stm32h7xx_hal_rcc_ex.o(i.HAL_RCCEx_GetD1SysClockFreq)
HAL_RCCEx_GetD3PCLK1Freq                0x08000b65   Thumb Code    34  stm32h7xx_hal_rcc_ex.o(i.HAL_RCCEx_GetD3PCLK1Freq)
HAL_RCCEx_GetPLL2ClockFreq              0x08000b91   Thumb Code   532  stm32h7xx_hal_rcc_ex.o(i.HAL_RCCEx_GetPLL2ClockFreq)
HAL_RCCEx_GetPLL3ClockFreq              0x08000dbd   Thumb Code   532  stm32h7xx_hal_rcc_ex.o(i.HAL_RCCEx_GetPLL3ClockFreq)
HAL_RCCEx_PeriphCLKConfig               0x08000fe9   Thumb Code  2868  stm32h7xx_hal_rcc_ex.o(i.HAL_RCCEx_PeriphCLKConfig)
```

10.2.6 Memory Map of the image

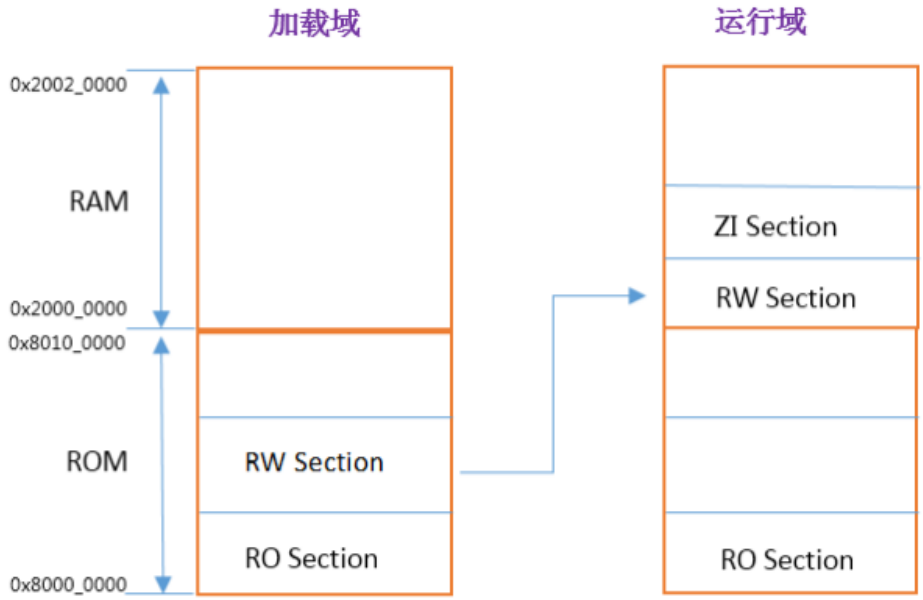
映像文件可以分为加载域（Load Region）和运行域（Execution Region）：加载域反映了 ARM 可执行映像文件的各个段存放在存储器中的位置关系。下面是部分截图，另外映像中的入口点就是程序开始执行的位置。

```
Memory Map of the image

  Image Entry point : 0x08000299

  Load Region LR_IROM1 (Base: 0x08000000, Size: 0x000055c8, Max: 0x00200000, ABSOLUTE)

    Execution Region ER_IROM1 (Exec base: 0x08000000, Load base: 0x08000000, Size: 0x00005588, Max: 0x00200000, ABSOLUTE)

    Exec Addr    Load Addr    Size         Type   Attr      Idx    E Section Name        Object

    0x08000000   0x08000000   0x00000298   Data   RO         1190    RESET               startup_stm32h743xx.o
    0x08000298   0x08000298   0x00000000   Code   RO         4380  * .ARM.Collect$$$$00000000  mc_w.l(entry.o)
    0x08000298   0x08000298   0x00000004   Code   RO         4650    .ARM.Collect$$$$00000001  mc_w.l(entry2.o)
    0x0800029c   0x0800029c   0x00000004   Code   RO         4653    .ARM.Collect$$$$00000004  mc_w.l(entry5.o)
    0x080002a0   0x080002a0   0x00000000   Code   RO         4655    .ARM.Collect$$$$00000008  mc_w.l(entry7b.o)
    0x080002a0   0x080002a0   0x00000000   Code   RO         4657    .ARM.Collect$$$$0000000A  mc_w.l(entry8b.o)
    0x080002a0   0x080002a0   0x00000008   Code   RO         4658    .ARM.Collect$$$$0000000B  mc_w.l(entry9a.o)
    0x080002a8   0x080002a8   0x00000000   Code   RO         4660    .ARM.Collect$$$$0000000D  mc_w.l(entry10a.o)
    0x080002a8   0x080002a8   0x00000000   Code   RO         4662    .ARM.Collect$$$$0000000F  mc_w.l(entry11a.o)
    0x080002a8   0x080002a8   0x00000004   Code   RO         4651    .ARM.Collect$$$$00002712  mc_w.l(entry2.o)
    0x080002ac   0x080002ac   0x00000024   Code   RO         1191    .text               startup_stm32h743xx.o
    0x080002d0   0x080002d0   0x00000062   Code   RO         4383    .text               mc_w.l(uldiv.o)
    0x08000332   0x08000332   0x00000024   Code   RO         4385    .text               mc_w.l(memseta.o)
    0x08000356   0x08000356   0x0000001e   Code   RO         4667    .text               mc_w.l(llshl.o)
    0x08000374   0x08000374   0x00000020   Code   RO         4669    .text               mc_w.l(llushr.o)
    0x08000394   0x08000394   0x00000024   Code   RO         4682    .text               mc_w.l(init.o)
    0x080003b8   0x080003b8   0x00000012   Code   RO         4349    OSKERNEL            hardfaulthandler.o
    0x080003ca   0x080003ca   0x00000004   Code   RO         1032    i.BusFault_Handler  stm32h7xx_it.o
```

运行域反映了 ARM 可执行映像文件各个段真正执行时在存储器中的位置关系：

```
Execution Region RW_IRAM1 (Exec base: 0x20000000, Load base: 0x08005588, Size: 0x000021b8, Max: 0x00020000, ABSOLUTE)

Exec Addr    Load Addr    Size         Type   Attr      Idx    E Section Name        Object

0x20000000   0x08005588   0x0000001e   Data   RW          614    .data               bsp_timer.o
0x2000001e   0x080055a6   0x00000002   PAD
0x20000020   0x080055a8   0x00000004   Data   RW          792    .data               bsp_fmc_io.o
0x20000024   0x080055ac   0x00000009   Data   RW         1248    .data               stm32h7xx_hal.o
0x2000002d   0x080055b5   0x00000003   PAD
0x20000030   0x080055b8   0x00000008   Data   RW         4318    .data               system_stm32h7xx.o
0x20000038   0x080055c0   0x00000004   Data   RW         4358    .data               segger_hardfaulthandler.o
0x2000003c   0x080055c4   0x00000004   Data   RW         4664    .data               mc_w.l(stdout.o)
0x20000040       -        0x000000ad   Zero   RW          506    .bss                bsp_key.o
0x200000ed   0x080055c8   0x00000003   PAD
0x200000f0       -        0x00000030   Zero   RW          612    .bss                bsp_timer.o
0x20000120       -        0x00001058   Zero   RW          877    .bss                bsp_uart_fifo.o
0x20001178       -        0x00000040   Zero   RW         4356    .bss                segger_hardfaulthandler.o
0x200011b8       -        0x00001000   Zero   RW         1188    STACK               startup_stm32h743xx.o
```

简单的说，加载域就是程序在 Flash 中的实际存储，而运行域是芯片上电后的运行状态，通过下面的框图可以有一个感性的认识：



通过上面的框图可以看出，RW 区也是要存储到 ROM/Flash 里面的，在执行映像之前，必须将已初始化的 RW 数据从 ROM 中复制到 RAM 中的执行地址并创建 ZI Section（初始化为 0 的变量区）。

注：0x8010_0000 和 0x8000_0000 要改成 0x0801_0000 和 0x0800_0000。

10.2.7 Image component sizes

Image component sizes 映像组件大小比较重要，比如下面内容代表的含义：

```
Image component sizes

    Code (inc. data)   RO Data    RW Data    ZI Data      Debug   Object Name

         774         48          0          0          0      41351   bsp.o
         540         88          0          4          0       4044   bsp_fmc_io.o
         840         40        112          0        173       7608   bsp_key.o
         158          0          0          0          0       1887   bsp_led.o
         288         50          0          0          0       1310   bsp_lptim_pwm.o
        1002        212         96         30         48      48853   bsp_timer.o
        1244        138          0          0       4184      11455   bsp_uart_fifo.o
          18          0          0          0          0        352   hardfaulthandler.o
         940        636          0          0          0    1253247   main.o
         264         62          0          4         64       3699   segger_hardfaulthandler.o
          36          8        664          0       4096        972   startup_stm32h743xx.o
         162         32          0          9          0       4945   stm32h7xx_hal.o
         502         38          0          0          0      42289   stm32h7xx_hal_cortex.o
         588         54          0          0          0       1948   stm32h7xx_hal_gpio.o
         286         18          0          0          0       2783   stm32h7xx_hal_lptim.o
         136         10          0          0          0        808   stm32h7xx_hal_pwr.o
        3048         86          0          0          0       6408   stm32h7xx_hal_rcc.o
        4692        118          0          0          0       7875   stm32h7xx_hal_rcc_ex.o
         106          4          0          0          0       1971   stm32h7xx_hal_sram.o
         288         32          0          0          0       3409   stm32h7xx_hal_tim.o
        3048        130         48          0          0       6800   stm32h7xx_hal_uart.o
          20          0          0          0          0       3684   stm32h7xx_it.o
         294         10          0          0          0       3044   stm32h7xx_ll_fmc.o
         148         18         64          8          0       1690   system_stm32h7xx.o

       ----------------------------------------------------------------------
       19442       1832       1016         60       8568    1462432   Object Totals
           0          0         32          0          0          0   (incl. Generated)
          20          0          0          5          3          0   (incl. Padding)
```

- Code (inc. Data) ：显示代码占用了多少字节。 在此映像中，有 19442 字节的代码， 其中包括 1832 字节的内联数据 (inc. data)，例如文字池和短字符串。
- RO Data ：显示只读数据占用了多少字节 (比如 const char buf[] = "123456")。这是除 Code (inc. data) 列中包括的内联数据之外的数据。
- RW Data ： 显示读写数据占用了多少字节。
- ZI Data ： 显示零初始化的数据占用了多少字节。
- Debug ： 显示调试数据占用了多少字节，例如，调试输入节以及符号和字符串。
- Object Totals ： 显示链接到一起以生成映像的对象占用了多少字节。
- (incl. Generated)：链接器会生成的映像内容，例如，交互操作中间代码。 如果 Object Totals 行包含此类型的数据，则会显示在该行中。本例中共有 1016 字节的 RO 数据，其中 32 字节是链接器生成的 RO 数据。
- (incl. Padding) ： 链接器根据需要插入填充，以强制字节对齐。

下面的 Library Totals 显示已提取并作为单个对象添加到映像中的库成员占用了多少字节。

Code (inc. data)	RO Data	RW Data	ZI Data	Debug	Library Member Name
0	0	0	0	0	entry.o
0	0	0	0	0	entry10a.o
0	0	0	0	0	entry11a.o
8	4	0	0	0	entry2.o
4	0	0	0	0	entry5.o
0	0	0	0	0	entry7b.o
0	0	0	0	0	entry8b.o
8	4	0	0	0	entry9a.o
30	0	0	0	0	handlers.o
36	8	0	0	68	init.o
30	0	0	0	68	llshl.o
32	0	0	0	68	llushr.o
36	0	0	0	108	memseta.o
1154	54	0	0	352	printf8.o
0	0	0	4	0	stdout.o
98	0	0	0	92	uldiv.o
1438	70	0	4	0	756 Library Totals
2	0	0	0	0	(incl. Padding)

下面是我们经常要查看的内容：

```
==============================================================================

      Code (inc. data)   RO Data    RW Data    ZI Data      Debug

     20880       1902       1016         64       8568    1452680   Grand Totals
     20880       1902       1016         64       8568    1452680   ELF Image Totals
     20880       1902       1016         64          0          0   ROM Totals

==============================================================================

    Total RO  Size (Code + RO Data)                21896 (  21.38kB)
    Total RW  Size (RW Data + ZI Data)              8632 (   8.43kB)
    Total ROM Size (Code + RO Data + RW Data)      21960 (  21.45kB)

==============================================================================
```

- Grand Totals：显示映像的真实大小。
- ELF Image Totals：ELF(Executable and Linking Format)可执行链接格式映像文件大小。
- ROM Totals：显示包含映像所需的 ROM 的最小大小。这不包括 ZI 数据和存储在 ROM 中的调试信息。

10.3 HTM 文件分析

MDK 将工程全编译，且没有错误后，就会生成此文件。以开发板配套例子为例，htm 文件位于路径：\Project\MDK-ARM(uV5)\Objects。

此文件的最大作用就是基本统计了所有被调用函数的栈 stack 使用情况（不考虑中断嵌套）。下面是整个工程的最大栈需求：

Maximum Stack Usage = 504 bytes + Unknown(Functions without stacksize, Cycles, Untraceable Function Pointers)

Call chain for Maximum Stack Depth:

main ⇒ bsp_Init ⇒ bsp_InitUart ⇒ InitHardUart ⇒ bsp_SetUartParam ⇒ HAL_UART_Init ⇒ UART_SetConfig ⇒ __aeabi_uldivmod

具体到每个函数也给出最大的栈深度 Max Depth，同时也给出函数本身的代码量大小和使用的栈大小，比如函数 bsp_InitExtIO，最大栈深度是 152 字节。函数本身占用代码大小（Thumb 指令集）28 字节，使用栈 8 字节。

bsp_InitExtIO (Thumb, 28 bytes, Stack size 8 bytes, bsp_fmc_io.o(i.bsp_InitExtIO))

[Stack]

- Max Depth = 152
- Call Chain = bsp_InitExtIO ⇒ HC574_ConfigFMC ⇒ HAL_SRAM_Init ⇒ FMC_NORSRAM_Extended_Timing_Init

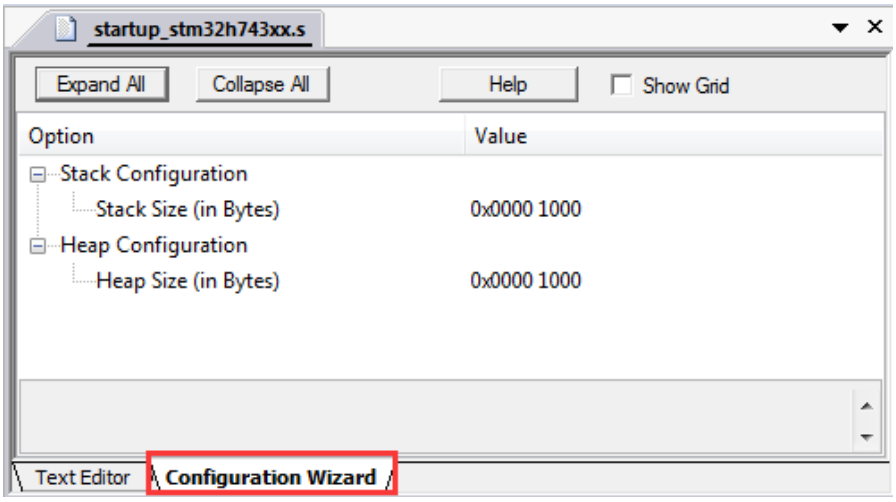
[Calls]

- >> HC574_ConfigGPIO
- >> HC574_ConfigFMC

[Called By]

- >> bsp_Init

通过这个文件，我们可以合理的配置启动文件里面的 stack 大小：



10.4 总结

本章节的知识点比较重要，务必要熟练掌握。平时开发工程项目时，可以多关注 map 文件和 htm 文件。

第11章 STM32H7 移植 SEGGER 的硬件异常分析

本章节为大家介绍 SEGGER 的硬件异常分析代码在 MDK 和 IAR 中的使用方法，在实际项目中比较有实用价值，因为我们经常会遇到进入硬件异常的情况。

11.1 初学者重要提示
11.2 移植方法
11.3 MDK 锁定硬件异常位置方法
11.4 IAR 锁定硬件异常位置方法
11.5 硬件异常原因分析
11.6 IAR 注释自带的硬件异常
11.7 实验例程
11.8 总结

11.1 初学者重要提示

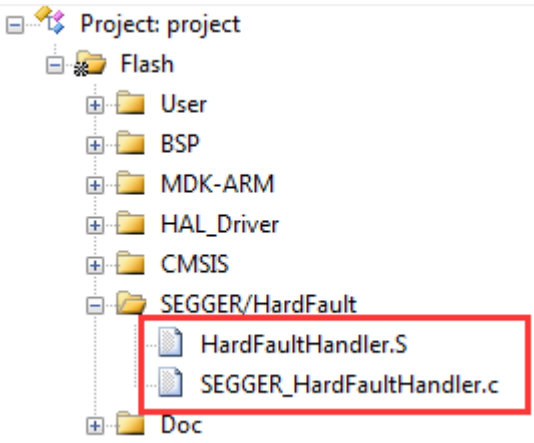
- MDK 本身也是支持硬件异常分析的，就是不够直观，此贴是 MDK 的硬件异常分析文档：http://www.armbbs.cn/forum.php?mod=viewthread&tid=21940 。
- IAR8 带的硬件异常分析比较好用，在本章 11.6 小节有说明。

11.2 移植方法

直接移植 SEGGER 的硬件异常代码会有错误警告，这里针对 IAR 和 MDK 版本做了些简单修改，方便大家移植到自己的工程里面。

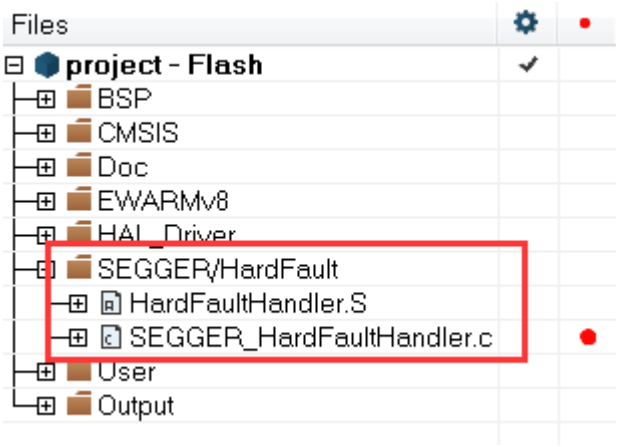
- MDK 版本移植

源文件位于本章配套例子的\User\segger\HardFaultHandlerMDK 文件夹，添加如下两个文件到工程里面即可。



◆ IAR 版本移植

源文件位于本章配套例子的\User\segger\HardFaultHandlerIAR 文件夹，添加如下两个文件到工程里面即可。



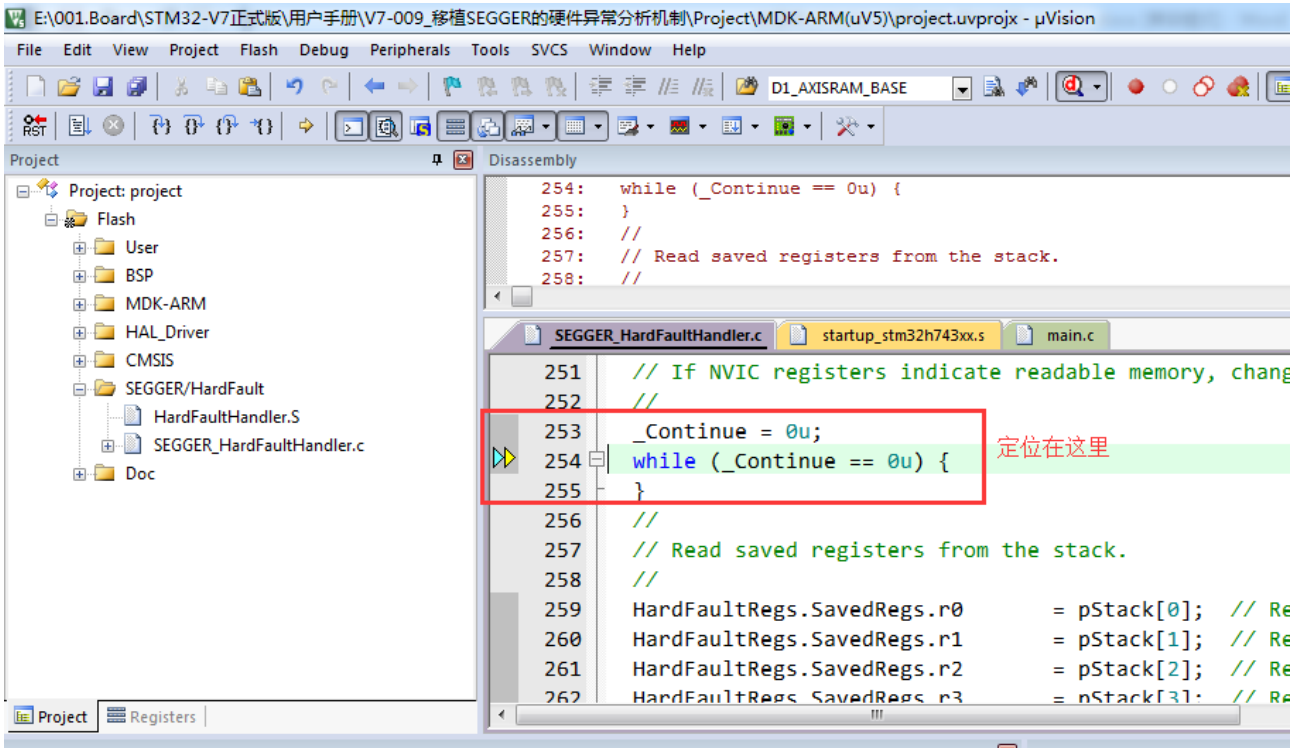
在文件 SEGGER_HardFaultHandler.c 里面都添加了串口打印功能，方便不用编译器的调试功能时，通过串口打印提示是否进入硬件异常。

```
#define ERR_INFO "\r\nEnter HardFault_Handler, System Halt.\r\n"

#if 1
{
    const char *pError = ERR_INFO;
    uint8_t i;

    for (i = 0; i < strlen(ERR_INFO); i++)
    {
        USART1->TDR = pError[i];
        /* 等待发送结束 */
        while((USART1->ISR & USART_ISR_TC) == 0);
    }
}
#endif
```

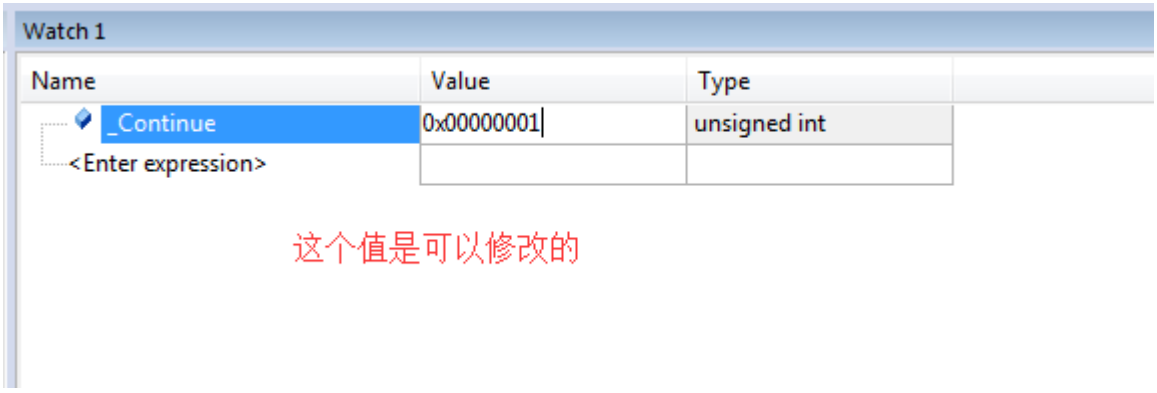
11.3 MDK 锁定硬件异常位置方法

以本章配套的例子为大家做个说明。

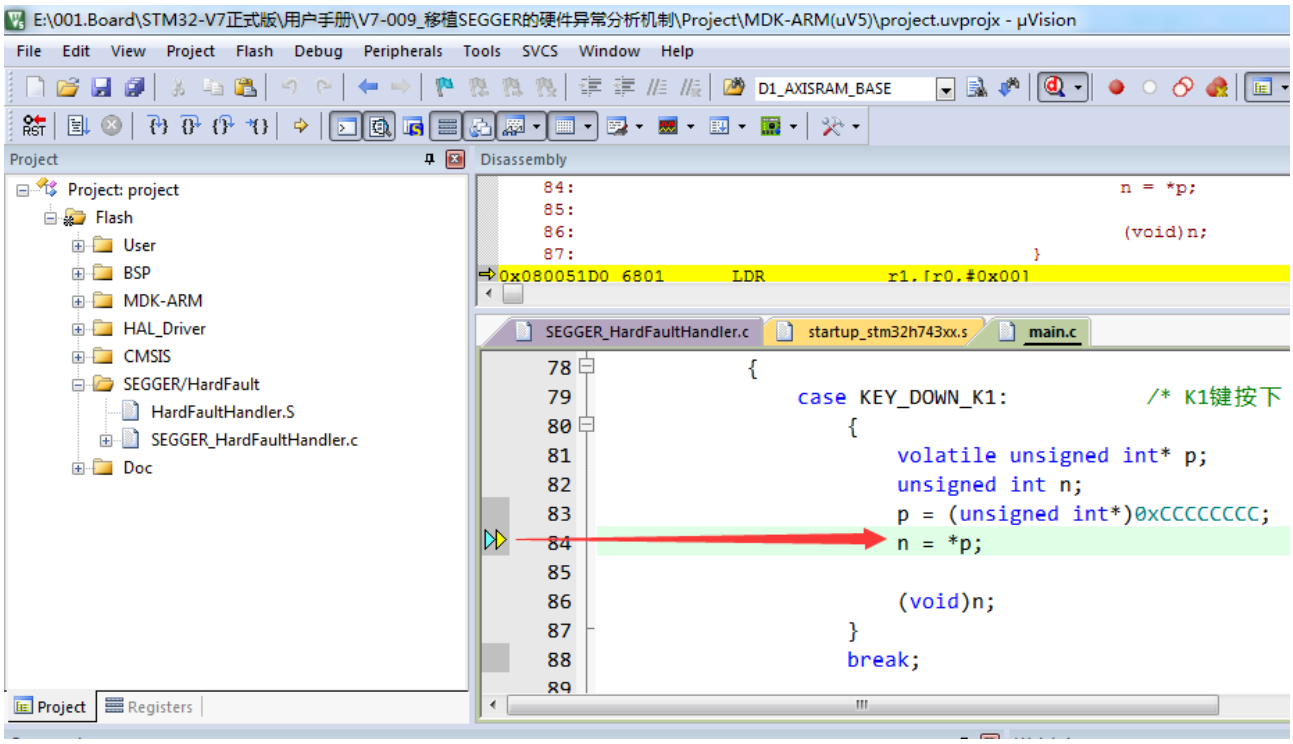
◆ 第 1 步：测试方法比较简单，进入调试状态，全速运行，然后按下 K1 按键，就会进入硬件异常中断，此时停止调试，程序就会自动定位到如下位置：



2、在 Watch1 窗口添加变量_Continue



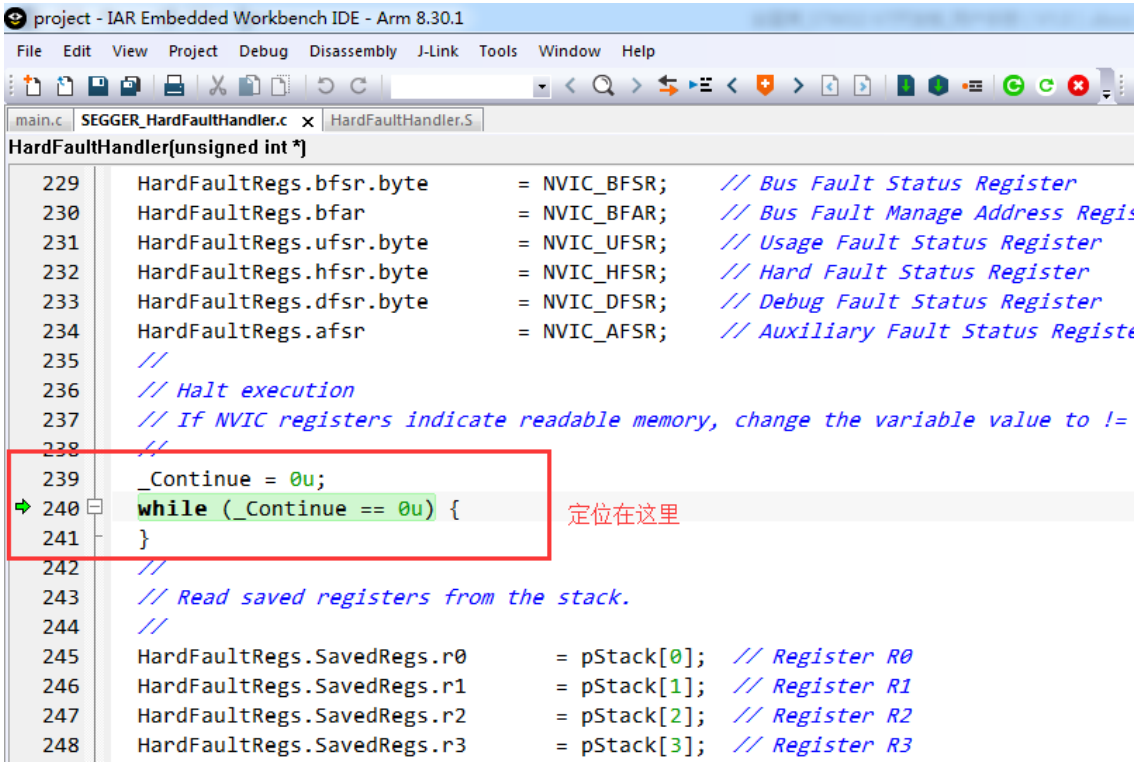
3、修改为任何非 0 数值，就可以继续单步调试。这个代码后面还有一个第 1 步中的 while 循环，也可以继续采用第 2 步的方法修改。退出硬件异常后就是大家进入硬件异常前下一条要执行的指令（可能还是这个函数本身，因为一个函数由多个指令完成）。定位到出问题的位置：



11.4 IAR 锁定硬件异常位置方法

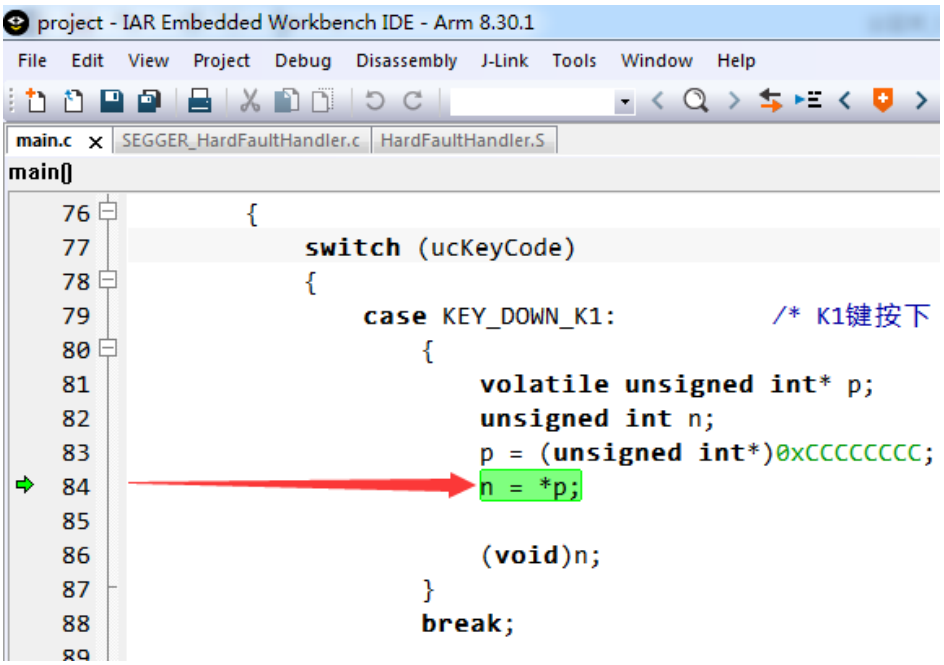
以本章配套的例子为大家做个说明。

- 第 1 步：测试方法比较简单，进入调试状态，全速运行，然后按下 K1 按键，就会进入硬件异常中断，此时停止调试，程序就会自动定位到如下位置：



2、在 Watch1 窗口添加变量_Continue

3、修改为任何非 0 数值，就可以继续单步调试。这个代码后面还有一个第 1 步中的 while 循环，也可以继续采用第 2 步的方法修改。退出硬件异常后就是大家进入硬件异常前下一条要执行的指令（可能还是这个函数本身，因为一个函数由多个指令完成）。定位到出问题的位置：



11.5 硬件异常原因分析

SEGGER 提供的这个机制查找出问题的位置比较方便，具体原因需要继续在调试界面里面添加 HardFaultRegs 结构变量，这个结构体变量添加了所有大家想看的东西。下面是 MDK 调试状态查看部分结构体数值：

Name	Value	Type
_Continue	0x00000001	unsigned int
HardFaultRegs	0x200000D0 &HardFaultRegs	struct <untagged>
SavedRegs	0x200000D0 &HardFaultRegs	struct <untagged>
r0	0xCCCCCCCC	unsigned int
r1	0x00000001	unsigned int
r2	0x20000050	unsigned int
r3	0x20000060	unsigned int
r12	0x0000001C	unsigned int
lr	0x08002957	unsigned int
pc	0x0800296C	unsigned int
psr	0x200000EC	union <untagged>
byte	0x61000000	unsigned int
bits	0x200000EC	struct <untagged>
IPSR	0x00000000	unsigned int bitfield o:=0, w=8
EPSR	0x00010000	unsigned int bitfield o:=8, w=19
APSR	0x0000000C	unsigned int bitfield o:=27, w=5
syshndctrl	0x200000F0	union <untagged>
byte	0x00010000	unsigned int
bits	0x200000F0	struct <untagged>
MEMFAULTACT	0x00000000	unsigned int bitfield o:=0, w=1
BUSFAULTACT	0x00000000	unsigned int bitfield o:=1, w=1
UnusedBits1	0x00000000	unsigned int bitfield o:=2, w=1
USGFAULTACT	0x00000000	unsigned int bitfield o:=3, w=1

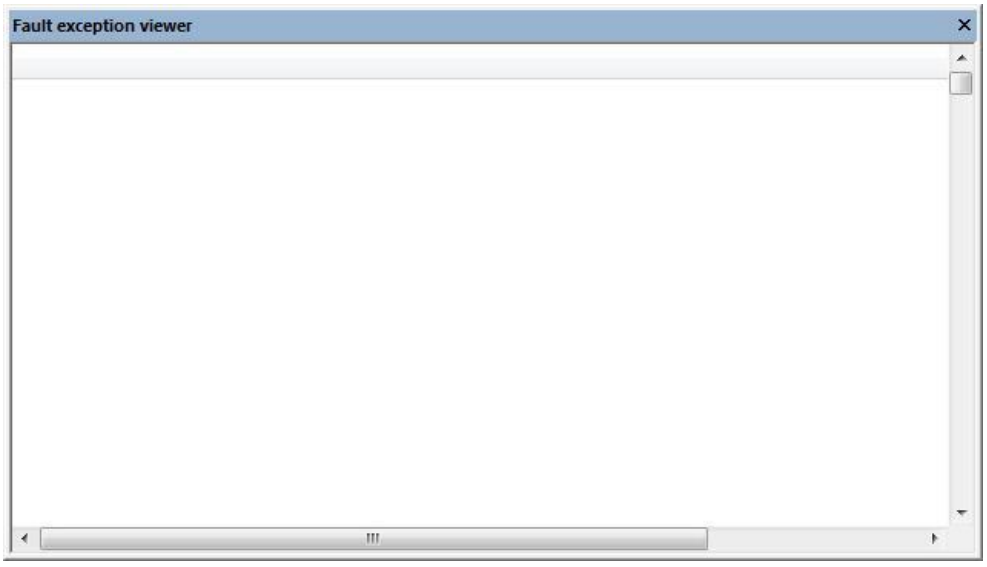
具体上面的变量代表什么含义呢，代码里面有注释，查阅起来没有 IAR 自带的硬件异常提示方便（注意，下面的代码用到了位域）。

```
#if DEBUG
static volatile unsigned int _Continue;  // Set this variable to 1 to run further

static struct {
  struct {
    volatile unsigned int r0;            // Register R0
    volatile unsigned int r1;            // Register R1
    volatile unsigned int r2;            // Register R2
    volatile unsigned int r3;            // Register R3
    volatile unsigned int r12;           // Register R12
    volatile unsigned int lr;            // Link register
    volatile unsigned int pc;            // Program counter
    union {
      volatile unsigned int byte;
      struct {
        unsigned int IPSR : 8;           // Interrupt Program Status register (IPSR)
        unsigned int EPSR : 19;          // Execution Program Status register (EPSR)
        unsigned int APSR : 5;           // Application Program Status register (APSR)
      } bits;
    } psr;                               // Program status register.
  } SavedRegs;

  union {
```

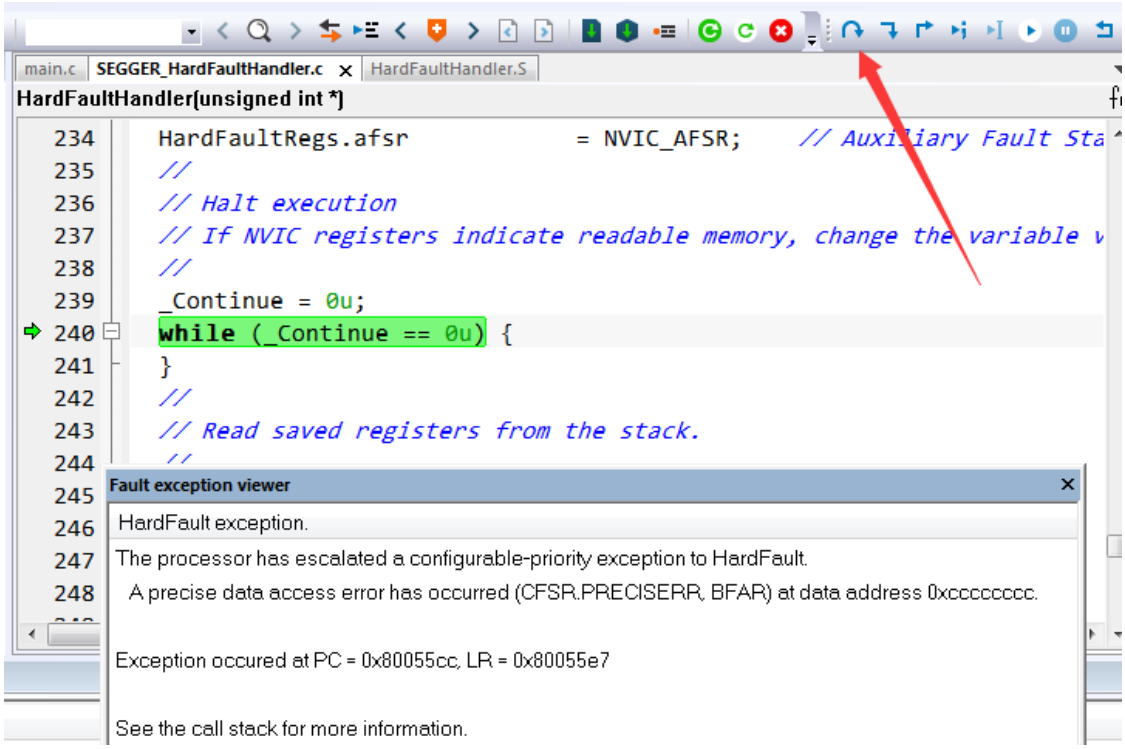
```
    volatile unsigned int byte;
    struct {
      unsigned int MEMFAULTACT    : 1;   // Read as 1 if memory management fault is active
      unsigned int BUSFAULTACT    : 1;   // Read as 1 if bus fault exception is active
      unsigned int UnusedBits1    : 1;
      unsigned int USGFAULTACT    : 1;   // Read as 1 if usage fault exception is active
      unsigned int UnusedBits2    : 3;
      unsigned int SVCALLACT      : 1;   // Read as 1 if SVC exception is active
      unsigned int MONITORACT     : 1;   // Read as 1 if debug monitor exception is active
      unsigned int UnusedBits3    : 1;
      unsigned int PENDSVACT      : 1;   // Read as 1 if PendSV exception is active
      unsigned int SYSTICKACT     : 1;   // Read as 1 if SYSTICK exception is active
      unsigned int USGFAULTPENDED : 1;   // Usage fault pended; usage fault started but was replaced by a
                                            higher-priority exception
      unsigned int MEMFAULTPENDED : 1;   //  Memory management fault pended; memory management fault started
                                             but was replaced by a higher-priority exception
      unsigned int BUSFAULTPENDED : 1;   // Bus fault pended; bus fault handler was started but was replaced
                                            by a higher-priority exception
      unsigned int SVCALLPENDED   : 1;   // SVC pended; SVC was started but was replaced by a higher-priority
                                            exception
      unsigned int MEMFAULTENA    : 1;   // Memory management fault handler enable
      unsigned int BUSFAULTENA    : 1;   // Bus fault handler enable
      unsigned int USGFAULTENA    : 1;   // Usage fault handler enable
    } bits;
  } syshndctrl;                          // System Handler Control and State Register (0xE000ED24)

  /* 省略未写 */

  volatile unsigned int afsr;            // Auxiliary Fault Status Register (0xE000ED3C), Vendor controlled
(optional)
} HardFaultRegs;
#endif
```

11.6 IAR 自带的硬件异常分析

还以本章配套的例子为例，进入调试状态，全速运行，然后按下 K1 按键，就会进入硬件异常中断，此时停止调试，IAR 还会弹出一个硬件异常错误分析，刚进来的时候也许是个空白



单步调试刷新下就出来了：



指出了问题的原因是操作的数据地址有问题。

11.7 实验例程

专门为本章节配套了一个例子：V7-009_移植 SEGGER 的硬件异常分析机制。大家可以按照本章教程提供的方法进行测试。

11.8 总结

除了 SEGGER 的硬件异常分析方案，建议也测试下 MDK 和 IAR 的，以后遇到硬件异常问题，解决起来可以得心应手。

第12章 STM32H7 的 HAL 库框架设计学习

通过本章节，主要是想让大家对 HAL 库程序设计的基本套路有个了解，防止踩坑。

12.1 初学者重要提示
12.2 HAL 库的配置文件
12.3 HAL 库的时间基准
12.4 HAL 库的启动流程
12.5 HAL 库初始化外设
12.6 HAL 库的中断处理思路
12.7 HAL 库的 DMA 处理思路
12.8 总结

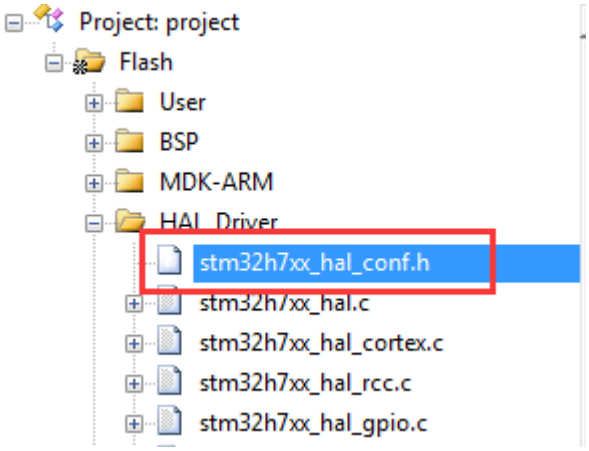
12.1 初学者重要提示

- 学习使用 HAL 库前，有必要对他们的基本设计框架有所了解，然后深入学习，效果更好。
- 为了方便调用，HAL 库为各种外设基本都配了三套 API，查询，中断和 DMA。

12.2 HAL 库的配置文件

HAL 库有一个专门的配置文件叫 stm32h7xx_hal_conf.h，这个文件里面有一个很重要的参数，就是 HSE_VALUE,大家所设计板子使用的实际晶振大小一定要与这个数值一致。比如 V7 的外置晶振是 25MHz，那么这里就务必配置宏定义为：

```
#define HSE_VALUE    ((uint32_t)25000000)
```



完整的代码如下：

```
/* ########################## Module Selection ############################## */
/**
  * @brief This is the list of modules to be used in the HAL driver
  */
#define HAL_MODULE_ENABLED
#define HAL_ADC_MODULE_ENABLED
#define HAL_CEC_MODULE_ENABLED

 /* 省略未写 */

/* ########################## Oscillator Values adaptation ####################*/
/**
  * @brief Adjust the value of External High Speed oscillator (HSE) used in your application.
  *        This value is used by the RCC HAL module to compute the system frequency
  *        (when HSE is used as system clock source, directly or through the PLL).
  */
#if !defined  (HSE_VALUE)
#define HSE_VALUE    ((uint32_t)25000000) /*!< Value of the External oscillator in Hz */
#endif /* HSE_VALUE */

#if !defined  (HSE_STARTUP_TIMEOUT)
  #define HSE_STARTUP_TIMEOUT    ((uint32_t)5000)   /*!< Time out for HSE start up, in ms */
#endif /* HSE_STARTUP_TIMEOUT */

/**
  * @brief Internal  oscillator (CSI) default value.
  *        This value is the default CSI value after Reset.
  */
#if !defined  (CSI_VALUE)
  #define CSI_VALUE    ((uint32_t)4000000) /*!< Value of the Internal oscillator in Hz*/
#endif /* CSI_VALUE */

/**
  * @brief Internal High Speed oscillator (HSI) value.
  *        This value is used by the RCC HAL module to compute the system frequency
  *        (when HSI is used as system clock source, directly or through the PLL).
  */
#if !defined  (HSI_VALUE)
  #define HSI_VALUE    ((uint32_t)64000000) /*!< Value of the Internal oscillator in Hz*/
#endif /* HSI_VALUE */

/**
  * @brief External Low Speed oscillator (LSE) value.
  *        This value is used by the UART, RTC HAL module to compute the system frequency
  */
#if !defined  (LSE_VALUE)
  #define LSE_VALUE    ((uint32_t)32768) /*!< Value of the External oscillator in Hz*/
#endif /* LSE_VALUE */


#if !defined  (LSE_STARTUP_TIMEOUT)
  #define LSE_STARTUP_TIMEOUT    ((uint32_t)5000)   /*!< Time out for LSE start up, in ms */
#endif /* LSE_STARTUP_TIMEOUT */

/**
  * @brief External clock source for I2S peripheral
  *        This value is used by the I2S HAL module to compute the I2S clock source
  *        frequency, this source is inserted directly through I2S_CKIN pad.
```

```
59.    */
60.  #if !defined  (EXTERNAL_CLOCK_VALUE)
61.    #define EXTERNAL_CLOCK_VALUE    12288000U /*!< Value of the External clock in Hz*/
62.  #endif /* EXTERNAL_CLOCK_VALUE */
63.
64.  /* Tip: To avoid modifying this file each time you need to use different HSE,
65.     ===  you can define the HSE value in your toolchain compiler preprocessor. */
66.
67.  /* ########################### System Configuration ######################### */
68.  /**
69.    * @brief This is the HAL system configuration section
70.    */
71.  #define  VDD_VALUE                    ((uint32_t)3300) /*!< Value of VDD in mv */
72.  #define  TICK_INT_PRIORITY            ((uint32_t)0x0F) /*!< tick interrupt priority */
73.  #define  USE_RTOS                     0
74.  /* #define  USE_SD_TRANSCEIVER           1U   */            /*!< use uSD Transceiver */
75.
76.  /* ########################### Ethernet Configuration ######################### */
77.  #define ETH_TX_DESC_CNT         4  /* number of Ethernet Tx DMA descriptors */
78.  #define ETH_RX_DESC_CNT         4  /* number of Ethernet Rx DMA descriptors */
79.
80.  #define ETH_MAC_ADDR0    ((uint8_t)0x02)
81.  #define ETH_MAC_ADDR1    ((uint8_t)0x00)
82.  #define ETH_MAC_ADDR2    ((uint8_t)0x00)
83.  #define ETH_MAC_ADDR3    ((uint8_t)0x00)
84.  #define ETH_MAC_ADDR4    ((uint8_t)0x00)
85.  #define ETH_MAC_ADDR5    ((uint8_t)0x00)
86.
87.  /* ########################### Assert Selection ############################## */
88.  /**
89.    * @brief Uncomment the line below to expanse the "assert_param" macro in the
90.    *        HAL drivers code
91.    */
92.  /* #define USE_FULL_ASSERT    1 */
93.
94.
95.  /* ################### SPI peripheral configuration ########################## */
96.  /**
97.    * @brief Used to activate CRC feature inside HAL SPI Driver
98.    *        Activated   (1U): CRC code is compiled within HAL SPI driver
99.    *        Deactivated (0U): CRC code excluded from HAL SPI driver
100.   */
101.
102. #define USE_SPI_CRC                   1U
103.
104.
105. /* Includes ------------------------------------------------------------------*/
106. /**
107.   * @brief Include module's header file
108.   */
109.
110. #ifdef HAL_RCC_MODULE_ENABLED
111.   #include "stm32h7xx_hal_rcc.h"
112. #endif /* HAL_RCC_MODULE_ENABLED */
113. /* 省略未写 */
114.
115.
116. /* Exported macro ------------------------------------------------------------*/
```

```
117. #ifdef  USE_FULL_ASSERT
118. /**
119.   * @brief  The assert_param macro is used for function's parameters check.
120.   * @param  expr: If expr is false, it calls assert_failed function
121.   *         which reports the name of the source file and the source
122.   *         line number of the call that failed.
123.   *         If expr is true, it returns no value.
124.   * @retval None
125.   */
126.   #define assert_param(expr) ((expr) ? (void)0 : assert_failed((uint8_t *)__FILE__, __LINE__))
127. /* Exported functions ------------------------------------------------------- */
128.   void assert_failed(uint8_t* file, uint32_t line);
129. #else
130.   #define assert_param(expr) ((void)0)
131. #endif /* USE_FULL_ASSERT */
```

除了 HSE_VALUE，下面几个也要作为了解：

- 第 72 行，滴答定时器的优先级设置。这个优先级的设置至关重要。因为 HAL 库中各个外设驱动里面的延迟实现是基于此文件提供的时间基准。

  如果在中断服务程序里面调用基于此时间基准的延迟函数 HAL_Delay 要特别注意，因为这个函数的时间基准是基于滴答定时器或者其他通用定时器实现，实现方式是滴答定时器或者其他通用定时器里面对变量计数。如此以来，结果是显而易见的，如果其他中断服务程序调用了此函数，且中断优先级高于滴答定时器，会导致滴答定时器中断服务程序一直得不到执行，从而卡死在里面。所以滴答定时器的中断优先级一定要比他们高。

  另外这个时间基准既可以使用滴答定时器实现也可以使用通用的定时器实现，默认情况下是用的滴答定时器。
- 第 73 行，当前 HAL 库还不支持 RTOS 方式。
- 第 74 行，STM32H7 的 SDIO 外接支持 UHS-I 模式 (SDR12, SDR25, SDR50, SDR104 和 DDR50) 的 SD 卡，需要 1.8 的电平转换器，此选项就是来使能此功能用的。
- 第 92 行，用于使能断言功能，在 HAL 库的 API 里面都有用到，用来判断函数形参是否有效。默认情况下是关闭的。
- 第 126 行，使能断言功能后，实际对应的代码位置。这里没有对函数 void assert_failed(uint8_t* file, uint32_t line)做具体实现，大家可以根据自己的需求去实现，这里提供一个参考：

```
/*
    ST 库函数使用了 C 编译器的断言功能，如果定义了 USE_FULL_ASSERT，那么所有的 ST 库函数将检查函数形参
    是否正确。如果不正确将调用 assert_failed() 函数，这个函数是一个死循环，便于用户检查代码。

    关键字 __LINE__ 表示源代码行号。
    关键字__FILE__表示源代码文件名。

    断言功能使能后将增大代码大小，推荐用户仅在调试时使能，在正式发布软件是禁止。

    用户可以选择是否使能 ST 固件库的断言供能。使能断言的方法有两种：
    (1) 在 C 编译器的预定义宏选项中定义 USE_FULL_ASSERT。
    (2) 在本文件取消"#define USE_FULL_ASSERT    1"行的注释。
*/
```

```
/*
*********************************************************************************************************
*    函 数 名: assert_failed
*    形    参：file : 源代码文件名称。关键字__FILE__表示源代码文件名。
*              line : 代码行号。关键字 __LINE__ 表示源代码行号
*    返 回 值: 无
*********************************************************************************************************
*/
void assert_failed(uint8_t* file, uint32_t line)
{
    /*
        用户可以添加自己的代码报告源代码文件名和代码行号，比如将错误文件和行号打印到串口
        printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    */

    /* 这是一个死循环，断言失败时程序会在此处死机，以便于用户查错 */
    while (1)
    {
    }
}
```

12.3 HAL 库的时间基准

为了方便各种外设延迟的实现，HAL 库专门搞了一个时间基准，默认来源是滴答定时器，也可以通过重定向使用其他定时器实现。相关函数全部集中在 stm32h7xx_hal.c 文件里面实现，关于这些函数在本教程的第 16 章有详细讲解。

12.4 HAL 库的启动流程

这里通过 V7 板子初始化流程做个说明：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: bsp_Init
4.  *    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。
5.  *              只需要调用一次
6.  *    形    参：无
7.  *    返 回 值: 无
8.  *********************************************************************************************************
9.  */
10. void bsp_Init(void)
11. {
12.     /* 配置 MPU */
13.     MPU_Config();
14.
15.     /* 使能 L1 Cache */
16.     CPU_CACHE_Enable();
17.
18.     /*
19.       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
20.       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
21.       - 设置 NVIV 优先级分组为 4。
```

```
22.       */
23.      HAL_Init();
24.
25.      /*
26.        配置系统时钟到 400MHz
27.        - 切换使用 HSE。
28.        - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
29.      */
30.      SystemClock_Config();
31.
32.
33.      bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
34.      bsp_InitTimer();   /* 初始化滴答定时器 */
35.      bsp_InitUart();    /* 初始化串口 */
36.      bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
37.      bsp_InitLed();     /* 初始化 LED */
38.
39.      bsp_InitLPTIMOutPWM(); /* 低功耗定时器 PWM 输出 */
40.  }
```

- 第 13 – 16 行，配置 MPU 和使能 Cache，对于 H7 而已，这两个函数要优先执行，因为 Flash 速度，SRAM 速度跟 CPU 和 TCM 有些差距的，所以要使能 Cache。而不同存储区的 Cache 策略要通过 MPU 分别进行配置。
- 第 23 行，调用函数 HAL_Init 时，系统依然使用的 64MHz HSI 时钟，这点要特别注意。此函数会调用函数 HAL_InitTick，初始化滴答时钟中断 1ms，并设置 NVIV 优先级分组为 4。
  这里就隐含了一个知识点，就是它会开启滴答定时器中断，如果用户也要使用滴答定时器中断，此问题就要引起注意，我们的 bsp_timer.C 文件解决办法如下：

```
/*
*********************************************************************************************************
*    函 数 名: SysTick_Handler
*    功能说明: 系统嘀嗒定时器中断服务程序。启动文件中引用了该函数。
*    形    参:  无
*    返 回 值: 无
*********************************************************************************************************
*/
void SysTick_Handler(void)
{
    HAL_IncTick();     /* ST HAL 库的滴答定时中断服务程序 */

    if (g_ucEnableSystickISR == 0) /* 做了一个变量标志，调用了函数 bsp_InitTimer 才置位此变量 */
    {
        return;
    }

    SysTick_ISR();     /* 安富莱 bsp 库的滴答定时中断服务程序 */
}
```

- 第 30 行，通过此函数切换 HSI 到外部高速时钟 HSE，并配置系统时钟到 400MHz。调用这个函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
- 前面几步执行完毕后就可以初始化外设了。

12.5 HAL 库初始化外设

HAL 库为外设初始化提供了一套框架，这里以串口为例进行说明，调用函数 HAL_UART_Init 初始化串口，此函数就会调用 HAL_UART_MspInit，这个函数是弱定义的，在 stm32h7xx_hal_uart.c 文件里面：

```
/**
  * @brief Initialize the UART MSP.
  * @param huart: UART handle.
  * @retval None
  */
__weak void HAL_UART_MspInit(UART_HandleTypeDef *huart)
{
  /* Prevent unused argument(s) compilation warning */
  UNUSED(huart);

  /* NOTE : This function should not be modified, when the callback is needed,
            the HAL_UART_MspInit can be implemented in the user file
   */
}
```

如果要初始化，直接将此函数在其它源文件里面实现即可，如果用到了中断和 DMA，也是直接在这里填写。

```
void HAL_UART_MspInit(UART_HandleTypeDef* huart)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    if(huart->Instance==USART1)
    {
        __HAL_RCC_USART1_CLK_ENABLE();

        __HAL_RCC_GPIOB_CLK_ENABLE();

        GPIO_InitStruct.Pin = GPIO_PIN_7;
        GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
        GPIO_InitStruct.Pull = GPIO_NOPULL;
        GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
        GPIO_InitStruct.Alternate = GPIO_AF7_USART1;
        HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

        GPIO_InitStruct.Pin = GPIO_PIN_14;
        GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
        GPIO_InitStruct.Pull = GPIO_NOPULL;
        GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
        GPIO_InitStruct.Alternate = GPIO_AF4_USART1;
        HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
    }
    else if(huart->Instance==USART2)
    {
        __HAL_RCC_USART2_CLK_ENABLE();

        __HAL_RCC_GPIOD_CLK_ENABLE();
        __HAL_RCC_GPIOA_CLK_ENABLE();

        GPIO_InitStruct.Pin = GPIO_PIN_6;
        GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
        GPIO_InitStruct.Pull = GPIO_NOPULL;
```

```
        GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
        GPIO_InitStruct.Alternate = GPIO_AF7_USART2;
        HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);

        GPIO_InitStruct.Pin = GPIO_PIN_2;
        GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
        GPIO_InitStruct.Pull = GPIO_NOPULL;
        GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
        GPIO_InitStruct.Alternate = GPIO_AF7_USART2;
        HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    }
}
```

由于所有串口都是通过函数 HAL_UART_Init 做初始化，所以函数 HAL_UART_MspInit 也是共用的。因此需要大家在这个里面区分是配置的哪个串口。

12.6 HAL 库的中断处理思路

为了加强对中断的管理，HAL 库也为各种外设中断也配套一个函数，比如串口就是函数 HAL_UART_IRQHandler。

```
void USART1_IRQHandler(void)
{
    /* USER CODE BEGIN USART1_IRQn 0 */
        此处可以添加用户代码
    /* USER CODE END USART1_IRQn 0 */

    /* 参数是串口句柄 */
    HAL_UART_IRQHandler(&huart1);

    /* USER CODE BEGIN USART1_IRQn 1 */
        此处可以添加用户代码
    /* USER CODE END USART1_IRQn 1 */
}
```

那么问题来了，如果要实现功能，用户的应用程序怎么写入中断？用户可以直接在函数 HAL_UART_IRQHandler 的前面或者后面添加新代码，也可以直接在 HAL_UART_IRQHandler 调用的各种回调函数里面执行，这些回调都是弱定义的，方便用户直接在其它文件里面重定义（下面回调主要是用于串口 DMA 时调用的）：

```
HAL_UART_TxHalfCpltCallback()
HAL_UART_TxCpltCallback()
HAL_UART_RxHalfCpltCallback()
HAL_UART_RxCpltCallback()
HAL_UART_ErrorCallback()
```

12.7 HAL 库的 DMA 处理思路

为了方便各种外设直接启动 DMA，HAL 库专门为支持 DMA 操作的外设都提供了对应的 DMA 函数，比如串口的：

```
HAL_UART_Transmit_DMA()
HAL_UART_Receive_DMA()
```

```
HAL_UART_DMAPause()
HAL_UART_DMAResume()
HAL_UART_DMAStop()
```

这里特别注意一点，针对外设的 DMA 函数基本都有开启中断，如果用户使能此外设的 NVIC，使用中务必别忘了写 DMA 的中断服务程序，比如使用 DMA1_Stream1：

```
void DMA1_Stream1_IRQHandler(void)
{
    /* USER CODE BEGIN USART1_IRQn 0 */
        此处可以添加用户代码
    /* USER CODE END USART1_IRQn 0 */

     /* 参数是 DMA 句柄 */
     HAL_DMA_IRQHandler(&hdma_usart1_tx);

    /* USER CODE BEGIN USART1_IRQn 1 */
        此处可以添加用户代码
    /* USER CODE END USART1_IRQn 1 */
}
```

如果要在 DMA 传输完成，半传输完成等中断里面执行功能，也是通过 HAL_DMA_IRQHandler 调用的各种回调函数里面实现，这些回调都是弱定义的，方便用户直接在其它文件里面重定义：

```
HAL_UART_TxHalfCpltCallback()
HAL_UART_TxCpltCallback()
HAL_UART_RxHalfCpltCallback()
HAL_UART_RxCpltCallback()
```

12.8 总结

先从整体框架上把握 HAL 库，然后具体外设具体学习，后面做应用会得心应手些。

第13章 STM32H7 启动过程详解

本章教程主要跟大家讲 STM32H7 的启动过程，这里的启动过程是指从 CPU 上电复位执行第 1 条指令开始（汇编文件）到进入 C 程序 main()函数入口之间的部分。启动过程相对来说还是比较重要的，理解了这个过程，对于以后分析程序还是有些帮助的，要不每次看到这个启动过程都会跳过，直接去看主程序了。还有就是以后打算学习 RTOS 的话，对于这个过程必须有个了解，因为移植的时候涉及到中断向量表。对初学者来说，看这个可能有些吃力，不过不要紧，随着自己做过一些简单的应用之后再来看这章，应该会有很多的帮助，由于我们的 V7 板子是基于 STM32H7XXX，所以我们这里主要针对 H7 系列的启动过程做一下分析，对于 F1，F4 系列也是大致相同的。

13.1 初学者重要提示
13.2 各个版本的启动文件介绍
13.3 启动文件分析
13.4 BOOT 启动模式
13.5 总结

13.1 初学者重要提示

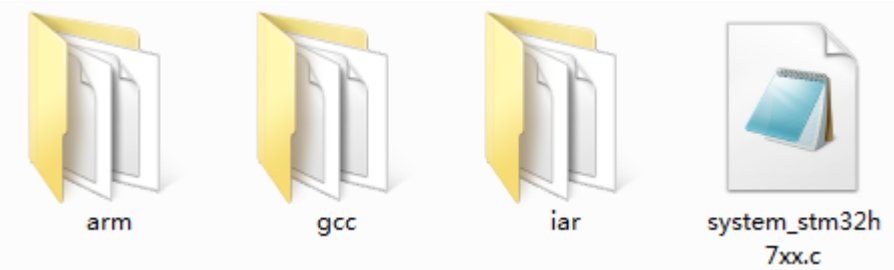
- 如果觉得学习本章节吃力的话，推荐看我们早期做的入门视频教程第 8 章，同样适用于 STM32H7。http://www.armbbs.cn/forum.php?mod=viewthread&tid=15408 。
- 相比 F1，F4 的启动方式，H7 的启动方式更灵活些，只需一个 boot 引脚即可。但是一个引脚只能区分出两个状态，为了解决这个问题，H7 专门配套了两个 option bytes 选项字节来解决此问题。

13.2 各个版本的启动文件介绍

这里各个版本的意思是指不同的编译器、不同的 H7 系列对应的启动文件。

13.2.1 不同编译器对应的启动文件

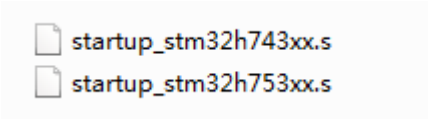
打开我们为本教程提供的工程文件，路径如下：\Libraries\CMSIS\Device\ST\STM32H7xx\Source\Templates 在这个文件里面有 ST 官方为各个编译器提供的启动文件。



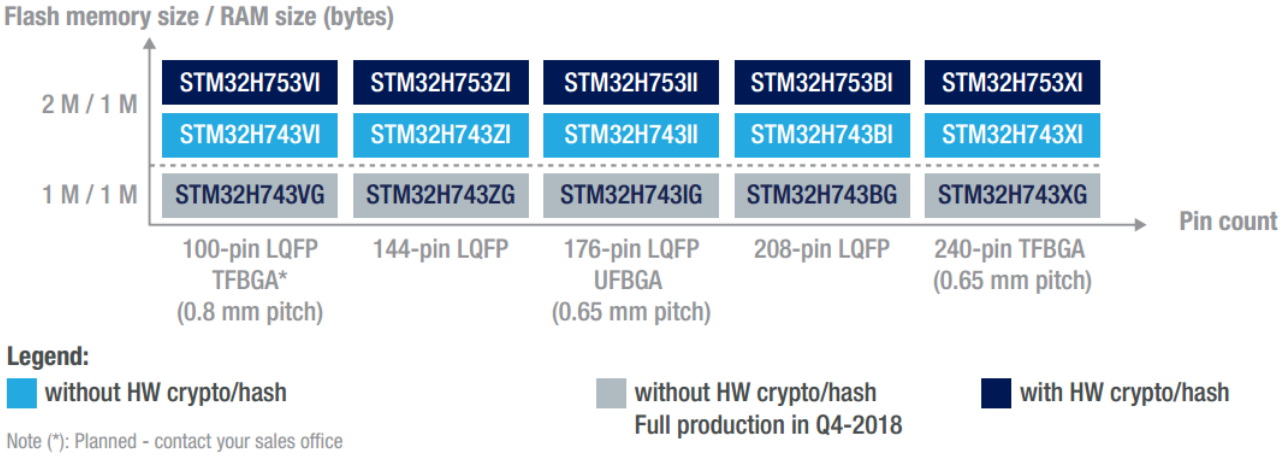
看了上面的截图，大家会问怎么没有 KEIL MDK 呢？其实已经被放在了文件夹 arm 里面，KEIL 公司已经在 2005 年被 ARM 公司收购了。开发板大部分例程都是配套了 MDK 和 IAR 两个版本，这里重点给大家分析一下 MDK 的启动文件分析，IAR 和 MDK 的大同小异。

13.2.2 不同 H7 系列对应的启动文件

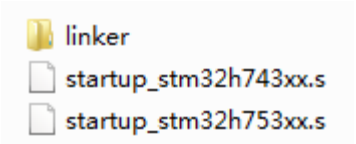
先来看一下 ARM 文件夹里面的文件（2018-07-03，当前只有如下两个系列，后期 ST 会增加新的型号，相应的启动文件也会添加进来）：



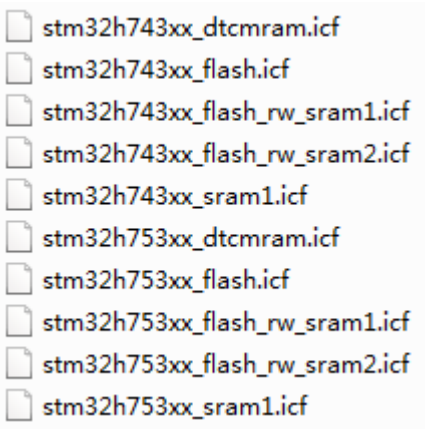
如果是 H743 系列，就使用 startup_stm32h743xx.s 文件，如果是 H753 系列，就使用 startup_stm32h753xx 文件。当前 H743 和 753 系列对应的型号如下：



我们再来打开 IAR 文件夹里面的文件：



多了一个 linker 文件夹，用于 IAR 配置的 ICF 文件：

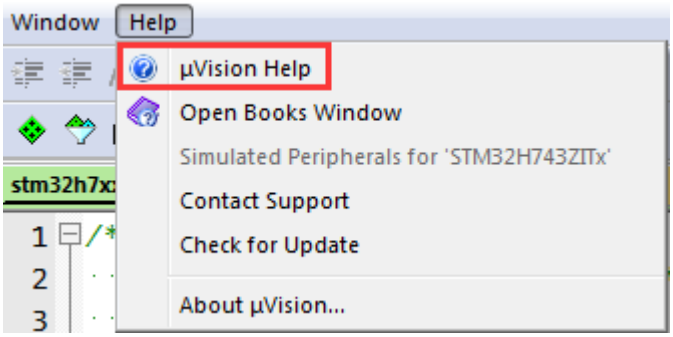


而启动文件跟 MDK 里面的一样，一个是用 H743 系列，另一个是用于 H753 系列。

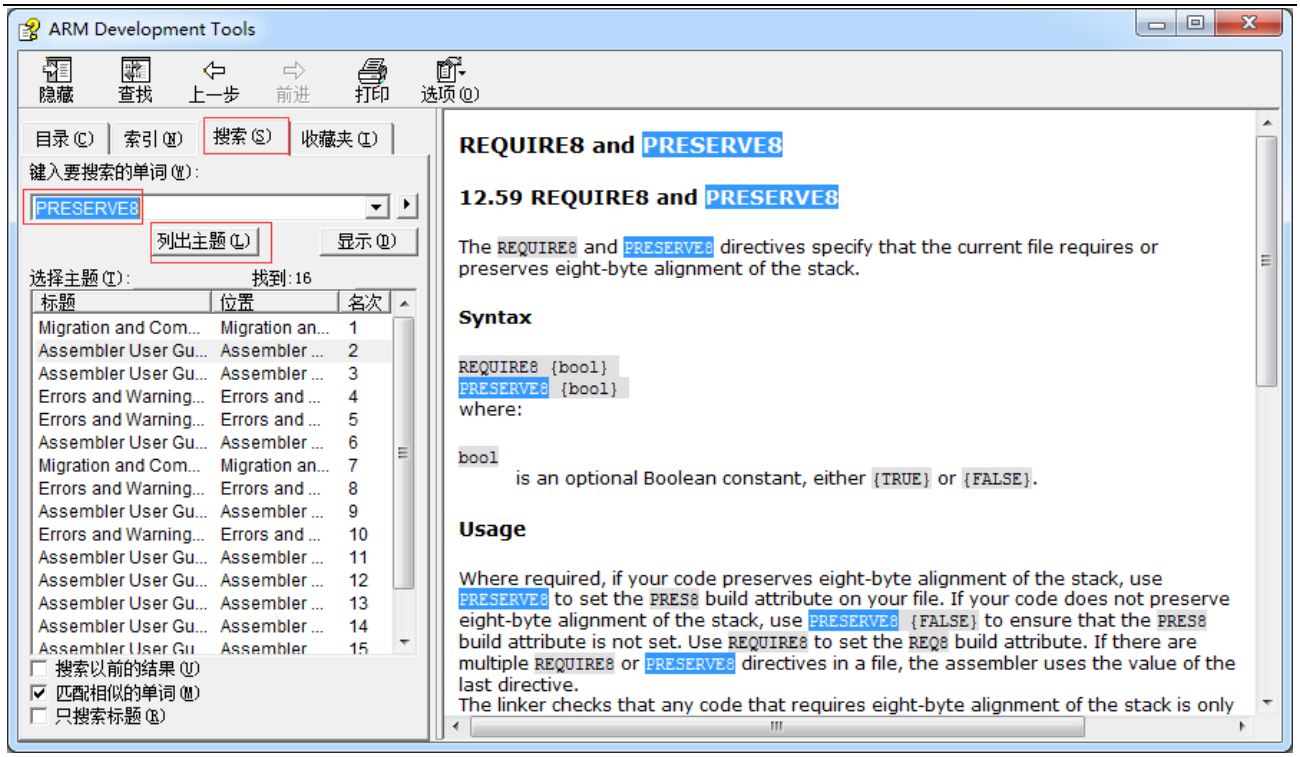
13.3 启动文件分析

鉴于 V7 开发板使用的是 STM32H743XI，下面我们详细的分析一下启动文件 startup_stm32h743xx.s。分析前，先掌握一个小技能，遇到不认识的指令或者关键词可以检索。

- 启动 MDK 软件，在 Help 菜单点击 uVision Help



- 点击后弹出如下文件



在搜索栏输入你需要查询的单词进行查询，然后点击“列出主题”按钮，会将相关的知识点都罗列出来。此功能非常实用，建议熟练掌握。

下面先来看启动文件前面的介绍 （固件库版本：V1.2.0）

```
;******************** (C) COPYRIGHT 2017 STMicroelectronics ********************
;* File Name          : startup_stm32h743xx.s
;* @author  MCD Application Team
;* version            : V1.2.0
;* Date               : 29-December-2017
;* Description        : STM32H7xx devices vector table for MDK-ARM toolchain.
;*                      This module performs:
;*                      - Set the initial SP
;*                      - Set the initial PC == Reset_Handler
;*                      - Set the vector table entries with the exceptions ISR address
;*                      - Branches to __main in the C library (which eventually
;*                        calls main()).
;*                      After Reset the Cortex-M processor is in Thread mode,
;*                      priority is Privileged, and the Stack is set to Main.
;* <<< Use Configuration Wizard in Context Menu >>>
;*******************************************************************************
;
; Licensed under MCD-ST Liberty SW License Agreement V2, (the "License");
; You may not use this file except in compliance with the License.
; You may obtain a copy of the License at:
;
;        http://www.st.com/software_license_agreement_liberty_v2
;
; Unless required by applicable law or agreed to in writing, software
; distributed under the License is distributed on an "AS IS" BASIS,
; WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
; See the License for the specific language governing permissions and
```

```
; limitations under the License.
;
;*******************************************************************************
```

启动文件是后缀为.s 的汇编语言文本文件，每行前面的分号表示此行是注释行。

启动文件主要完成如下工作，即程序执行过程：

- 设置堆栈指针 SP = __initial_sp。
- 设置 PC 指针 = Reset_Handler。
- 设置中断向量表。
- 配置系统时钟。
- 配置外部 SRAM/SDRAM 用于程序变量等数据存储（这是可选的）。
- 跳转到 C 库中的 __main ，最终会调用用户程序的 main()函数。

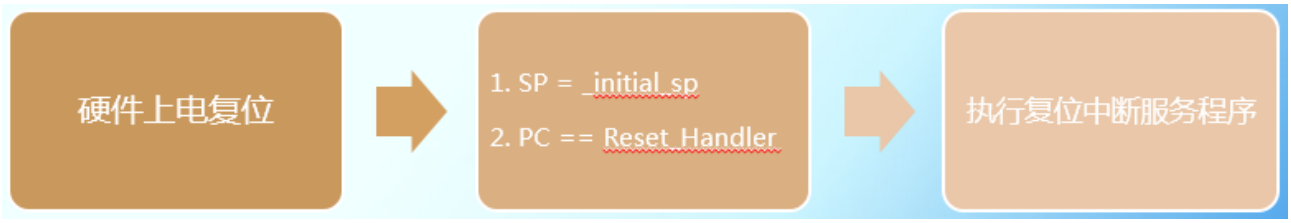
Cortex-M 内核处理器复位后，处于线程模式，指令权限是特权级别（最高级别），堆栈设置为使用主堆栈 MSP。

13.3.1复位序列

硬件复位之后,CPU 内的时序逻辑电路首先完成如下两个工作(程序代码下载到内部 flash 为例,flash 首地址 0x0800 0000)

- 将 0x08000000 位置存放的堆栈栈顶地址存放到 SP 中(MSP)。
- 将 0x08000004 位置存放的向量地址装入 PC 程序计数器。

CPU 从 PC 寄存器指向的物理地址取出第 1 条指令开始执行程序,也就是开始执行复位中断服务程序 Reset_Handler。



复位中断服务程序会调用 SystemInit()函数来配置系统时钟、配置 FMC 总线上的外部 SRAM/SDRAM，然后跳转到 C 库中__main 函数。由 C 库中的__main 函数完成用户程序的初始化工作（比如：变量赋初值等），最后由__main 函数调用用户写的 main()函数开始执行 C 程序。

13.3.2代码分析

- 第 1 部分代码分析

下面的代码实现开辟栈(stack)空间，用于局部变量、函数调用、函数的参数等。

```
1.   ; Amount of memory (in bytes) allocated for Stack
2.   ; Tailor this value to your application needs
```

```
3.  ; <h> Stack Configuration
4.  ;   <o> Stack Size (in Bytes) <0x0-0xFFFFFFFF:8>
5.  ; </h>
6.
7.  Stack_Size      EQU     0x00000400
8.
9.                  AREA    STACK, NOINIT, READWRITE, ALIGN=3
10. Stack_Mem       SPACE   Stack_Size
11. __initial_sp
```

第 7 行：EQU 是表示宏定义的伪指令，类似于 C 语言中的#define。伪指令的意思是指这个“指令”并不会生成二进制程序代码，也不会引起变量空间分配。

0x00000400 表示栈大小，注意这里是以字节为单位。

第 9 行：开辟一段数据空间可读可写，段名 STACK，按照 8 字节对齐。ARER 伪指令表示下面将开始定义一个代码段或者数据段。此处是定义数据段。ARER 后面的关键字表示这个段的属性。

STACK ：表示这个段的名字，可以任意命名。

NOINIT：表示此数据段不需要填入初始数据。

READWRITE：表示此段可读可写。

ALIGN=3 ：表示首地址按照 2 的 3 次方对齐，也就是按照 8 字节对齐(地址对 8 求余数等于 0)。

第 10 行：SPACE 这行指令告诉汇编器给 STACK 段分配 0x00000400 字节的连续内存空间。

第 11 行： __initial_sp 紧接着 SPACE 语句放置，表示了栈顶地址。__initial_sp 只是一个标号，标号主要用于表示一片内存空间的某个位置，等价于 C 语言中的“地址”概念。地址仅仅表示存储空间的一个位置，从 C 语言的角度来看，变量的地址，数组的地址或是函数的入口地址在本质上并无区别。

◆ 第 2 部分代码分析

下面的代码实现开辟堆(heap)空间，主要用于动态内存分配，也就是说用 malloc，calloc, realloc 等函数分配的变量空间是在堆上。

```
1.  ; <h> Heap Configuration
2.  ;   <o>  Heap Size (in Bytes) <0x0-0xFFFFFFFF:8>
3.  ; </h>
4.
5.  Heap_Size       EQU     0x00000200
6.
7.                  AREA    HEAP, NOINIT, READWRITE, ALIGN=3
8.  __heap_base
9.  Heap_Mem        SPACE   Heap_Size
10. __heap_limit
```

这几行语句和上面第 1 部分代码类似。分配一片连续的内存空间给名字叫 HEAP 的段，也就是分配堆空间。堆的大小为 0x00000200。

__heap_base 表示堆的开始地址。

__heap_limit 表示堆的结束地址。

◆ 第 3 部分代码分析

```
1.                  PRESERVE8
2.                  THUMB
3.
4.
5.   ; Vector Table Mapped to Address 0 at Reset
6.                  AREA    RESET, DATA, READONLY
7.                  EXPORT  __Vectors
8.                  EXPORT  __Vectors_End
9.                  EXPORT  __Vectors_Size
```

第 1 行：PRESERVE8 指定当前文件保持堆栈八字节对齐。

第 2 行：THUMB 表示后面的指令是 THUMB 指令集 ， CM7 采用的是 THUMB - 2 指令集。

第 6 行：AREA 定义一块数据段，只读，段名字是 RESET。READONLY 表示只读，缺省就表示代码段了。

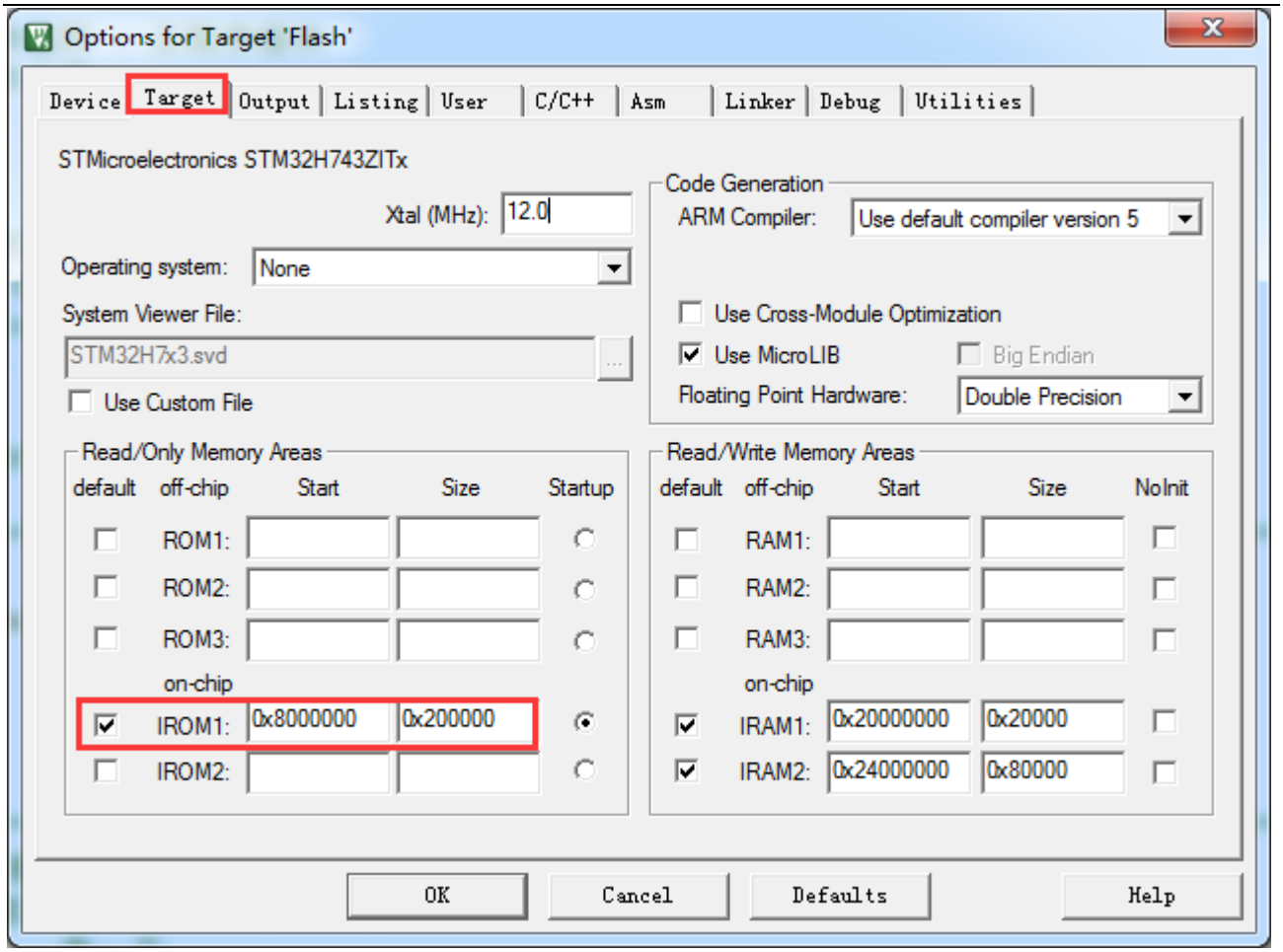
第 7-9 行：3 行 EXPORT 语句将 3 个标号申明为可被外部引用， 主要提供给链接器用于连接库文件或其他文件。

◆ 第 4 部分代码分析

```
1.   __Vectors       DCD     __initial_sp                ; Top of Stack
2.                   DCD     Reset_Handler               ; Reset Handler
3.                   DCD     NMI_Handler                 ; NMI Handler
4.                   DCD     HardFault_Handler           ; Hard Fault Handler
5.
6.                   中间部分省略未写
7.
8.                   DCD     0                           ; Reserved
9.                   DCD     WAKEUP_PIN_IRQHandler       ; Interrupt for all 6 wake-up pins
10.
11.
12.  __Vectors_End
13.
14.  __Vectors_Size  EQU  __Vectors_End - __Vectors
```

上面的这段代码是建立中断向量表，中断向量表定位在代码段的最前面。具体的物理地址由链接器的配置参数（IROM1 的地址）决定。如果程序在 Flash 运行，则中断向量表的起始地址是 0x08000000。

以 MDK 为例，就是如下配置选项：



DCD 表示分配 1 个 4 字节的空间。每行 DCD 都会生成一个 4 字节的二进制代码。中断向量表存放的实际上是中断服务程序的入口地址。当异常（也即是中断事件）发生时，CPU 的中断系统会将相应的入口地址赋值给 PC 程序计数器，之后就开始执行中断服务程序。

◆ 第 5 部分代码分析

```
1.                  AREA    |.text|, CODE, READONLY
2.
3.  ; Reset handler
4.  Reset_Handler    PROC
5.                   EXPORT  Reset_Handler                   [WEAK]
6.          IMPORT  SystemInit
7.          IMPORT  __main
8.
9.                   LDR     R0, =SystemInit
10.                  BLX     R0
11.                  LDR     R0, =__main
12.                  BX      R0
13.                  ENDP
```

第 1 行：AREA 定义一块代码段，只读，段名字是 .text 。READONLY 表示只读。

第 4 行：利用 PROC、ENDP 这一对伪指令把程序段分为若干个过程，使程序的结构加清晰。

第 5 行：WEAK 声明其他的同名标号优先于该标号被引用,就是说如果外面声明了的话会调用外面的。 这个声明很重要，它让我们可以在 C 文件中任意地方放置中断服务程序，只要保证 C 函数的名字和向量表中的名字一致即可。

第 6 行：IMPORT：伪指令用于通知编译器要使用的标号在其他的源文件中定义。但要在当前源文件中引用，而且无论当前源文件是否引用该标号，该标号均会被加入到当前源文件的符号表中。

第 9 行：SystemInit 函数在文件 system_stm32h7xx.c 里面，主要实现 RCC 相关寄存器复位和中断向量表位置设置。

第 11 行：__main 标号表示 C/C++标准实时库函数里的一个初始化子程序__main 的入口地址。该程序的一个主要作用是初始化堆栈(跳转__user_initial_stackheap 标号进行初始化堆栈的，下面会讲到这个标号)，并初始化映像文件，最后跳转到 C 程序中的 main 函数。这就解释了为何所有的 C 程序必须有一个 main 函数作为程序的起点。因为这是由 C/C++标准实时库所规，并且不能更改。

◆ 第 6 部分代码分析

代码如下：

```
1.   ; Dummy Exception Handlers (infinite loops which can be modified)
2.
3.   NMI_Handler     PROC
4.                   EXPORT  NMI_Handler                     [WEAK]
5.                   B       .
6.                   ENDP
7.   HardFault_Handler\
8.                   PROC
9.                   EXPORT  HardFault_Handler               [WEAK]
10.                  B       .
11.                  ENDP
12.
13.                  中间部分省略未写
14.  Default_Handler PROC
15.
16.                  EXPORT  WWDG_IRQHandler                  [WEAK]
17.                  EXPORT  PVD_AVD_IRQHandler               [WEAK]
18.                  EXPORT  TAMP_STAMP_IRQHandler            [WEAK]
19.                  中间部分省略未写
20.  SAI4_IRQHandler
21.  WAKEUP_PIN_IRQHandler
22.
23.                  B       .
24.
25.                  ENDP
26.
27.                  ALIGN
```

第 5 行：死循环，用户可以在此实现自己的中断服务程序。不过很少在这里实现中断服务程序，一般多是在其它的 C 文件里面重新写一个同样名字的中断服务程序，因为这里是 WEEK 弱定义的。如果没有在其它文件中写中断服务器程序，且使能了此中断，进入到这里后，会让程序卡在这个地方。

第 14 行：缺省中断服务程序（开始）

第 23 行：死循环，如果用户使能中断服务程序，而没有在 C 文件里面写中断服务程序的话，都会进入到这里。比如在程序里面使能了串口 1 中断，而没有写中断服务程序 USART1_IRQHandle，那么串口中断来了，会进入到这个死循环。

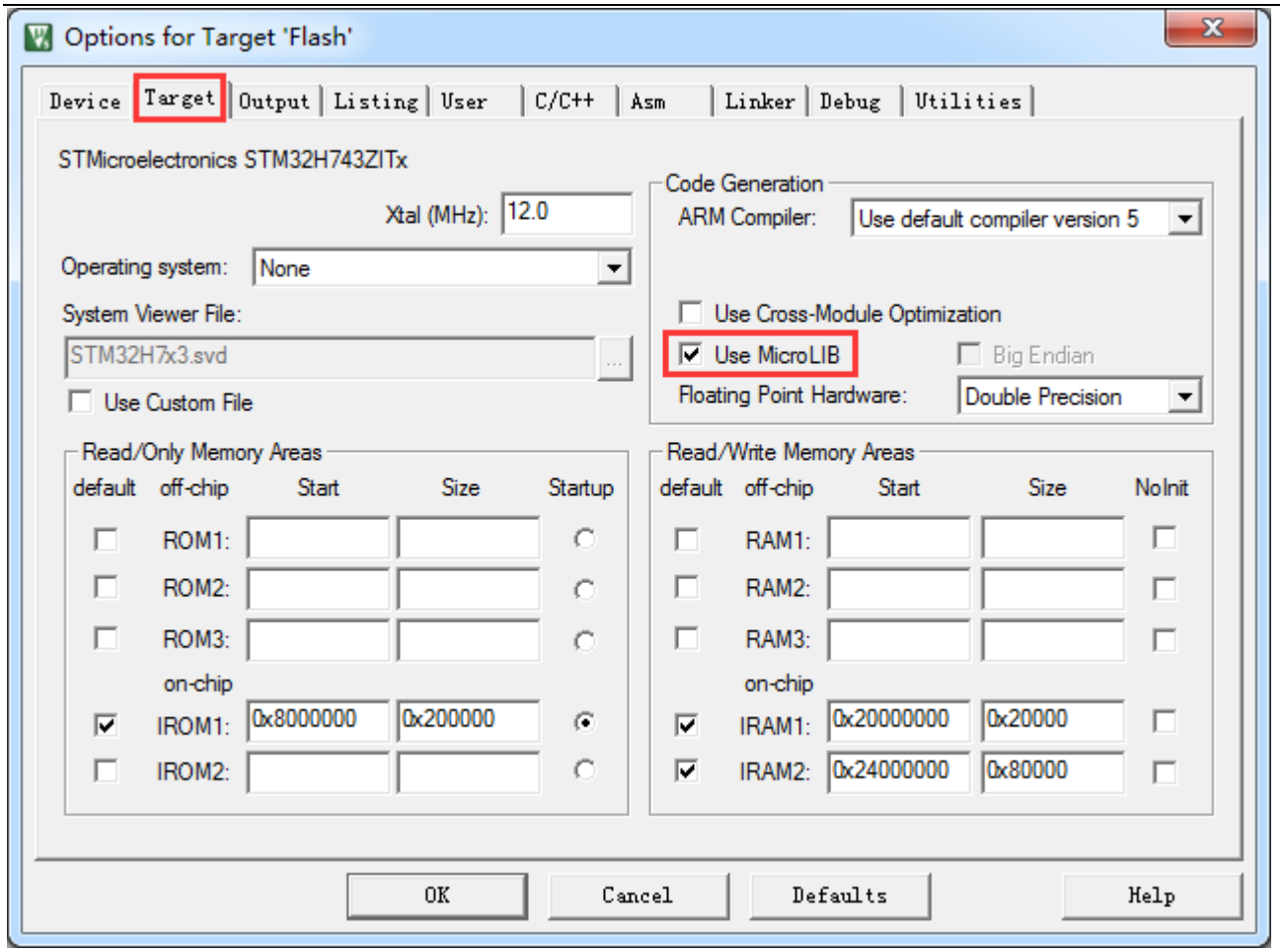
第 25 行：缺省中断服务程序（结束）。

◆ 第 7 部分代码分析

启动代码的最后一部分：

```
1.  ;*******************************************************************************
2.  ; User Stack and Heap initialization
3.  ;*******************************************************************************
4.                  IF      :DEF:__MICROLIB
5.
6.                  EXPORT  __initial_sp
7.                  EXPORT  __heap_base
8.                  EXPORT  __heap_limit
9.
10.                 ELSE
11.
12.                 IMPORT  __use_two_region_memory
13.                 EXPORT  __user_initial_stackheap
14.
15. __user_initial_stackheap
16.
17.                 LDR     R0, =  Heap_Mem
18.                 LDR     R1, =(Stack_Mem + Stack_Size)
19.                 LDR     R2, = (Heap_Mem +  Heap_Size)
20.                 LDR     R3, = Stack_Mem
21.                 BX      LR
22.
23.                 ALIGN
24.
25.                 ENDIF
26.
27.                 END
```

第 4 行：简单的汇编语言实现 IF.....ELSE.....语句。如果定义了 MICROLIB，那么程序是不会执行 ELSE 分支的代码。__MICROLIB 可能大家并不陌生，就在 MDK 的 Target Option 里面设置。



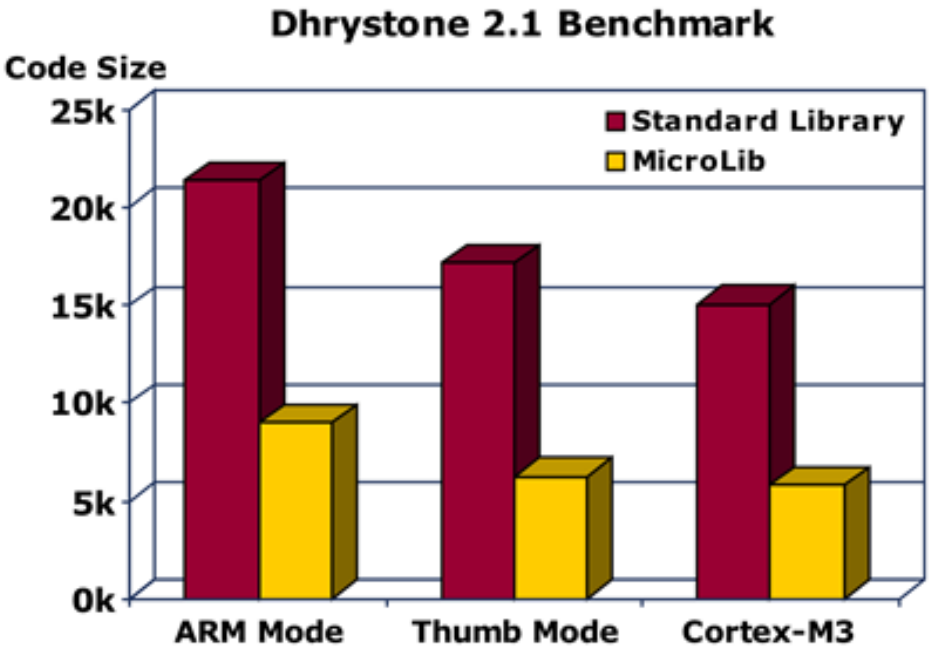
第 5 行： __user_initial_stackheap 将由__main 函数进行调用。

- MicroLib

MicroLib 是 MDK 里面带的微库，针对嵌入式应用，MicroLIB 做了深度优化，比使用 C 标准库所需的 RAM 和 FLASH 空间都大大减小比如调用：

<math.h>，<stdlib.h>，<stdio.h>，<string.h>

更多相关知识可以地址：http://www.keil.com/arm/microlib.asp。另外注意 microlib 只有库，没有源文件。下图是标准库和微库生成代码的比较。



13.4 BOOT 启动模式

相比 F1，F4 的启动方式，H7 的启动方式更灵活些，只需一个 boot 引脚即可。但是一个引脚只能区分出两个状态，为了解决这个问题，H7 专门配套了两个 option bytes 选项字节配置，如此以来就可以方便设置各种存储器地址了。

BOOT 模式选择		**BOOT 区域**
BOOT 引脚	BOOT 地址，选项字节	
0	BOOT_ADD0[15:0]	启动地址由 BOOT_ADD0 定义，默认值是 0x0800，对应 Flash 首地址 0x0800 0000。
1	BOOT_ADD1[15:0]	启动地址由 BOOT_ADD1 定义，默认值是 0x1FF0，对应系统 bootloader 的首地址 0x1FF0 0000。

BOOT_ADD0 和 BOOT_ADD1 对应 32 位地址到高 16 位，这点要特别注意。通过这两个选项字节，所有 0x0000 0000 到 0x3FFF 0000 的存储器地址都可以设置，包括：

- 所有 Flash 地址空间。
- 所有 RAM 地址空间，ITCM，DTCM 和 SRAM。

设置了选项字节后，掉电不会丢失，下次上电或者复位后，会根据 BOOT 引脚状态从 BOOT_ADD0，或 BOOT_ADD1 所设置的地址进行启动。

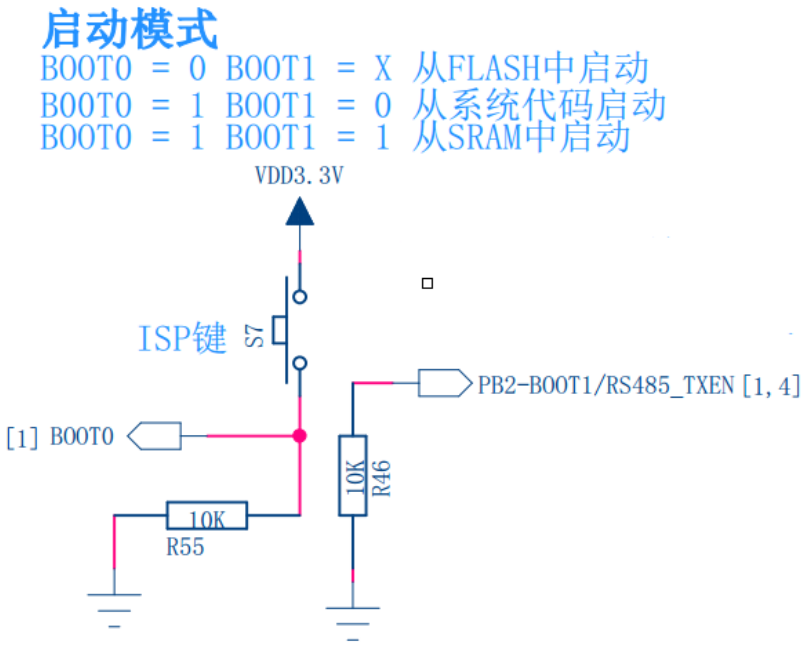
使用 BOOT 功能，注意以下几个问题：

- 如果用户不慎，设置的地址范围不在有效的存储器地址，那么 BOOT = 0 时，会从 Flash 首地址 0x0800 0000 启动，BOOT = 1 时，会从 ITCM 首地址 0x0000 0000 启动。

◆ 如果用户使能了 Flash Level 2 保护，那么只能从 Flash 地址空间进行启动。

- F1,F4 的启动方式

作为对比，这里补充 F1，F4 的启动方式，由 BOOT0 和 BOOT1 引脚共同决定。



13.5 总结

本章节讲解的启动过程分析还是比较重要的，忘初学者务必掌握。

第14章 STM32H7 的电源，复位和时钟系统

本章教程继续为大家讲解学习 STM32H7 的必备知识点电源，复位和时钟系统。掌握这三方面的知识点对后面的学习大有裨益。

14.1 初学者重要提示
14.2 电源
14.3 硬件复位
14.4 软件复位
14.5 RCC 时钟控制
14.6 总结

14.1 初学者重要提示

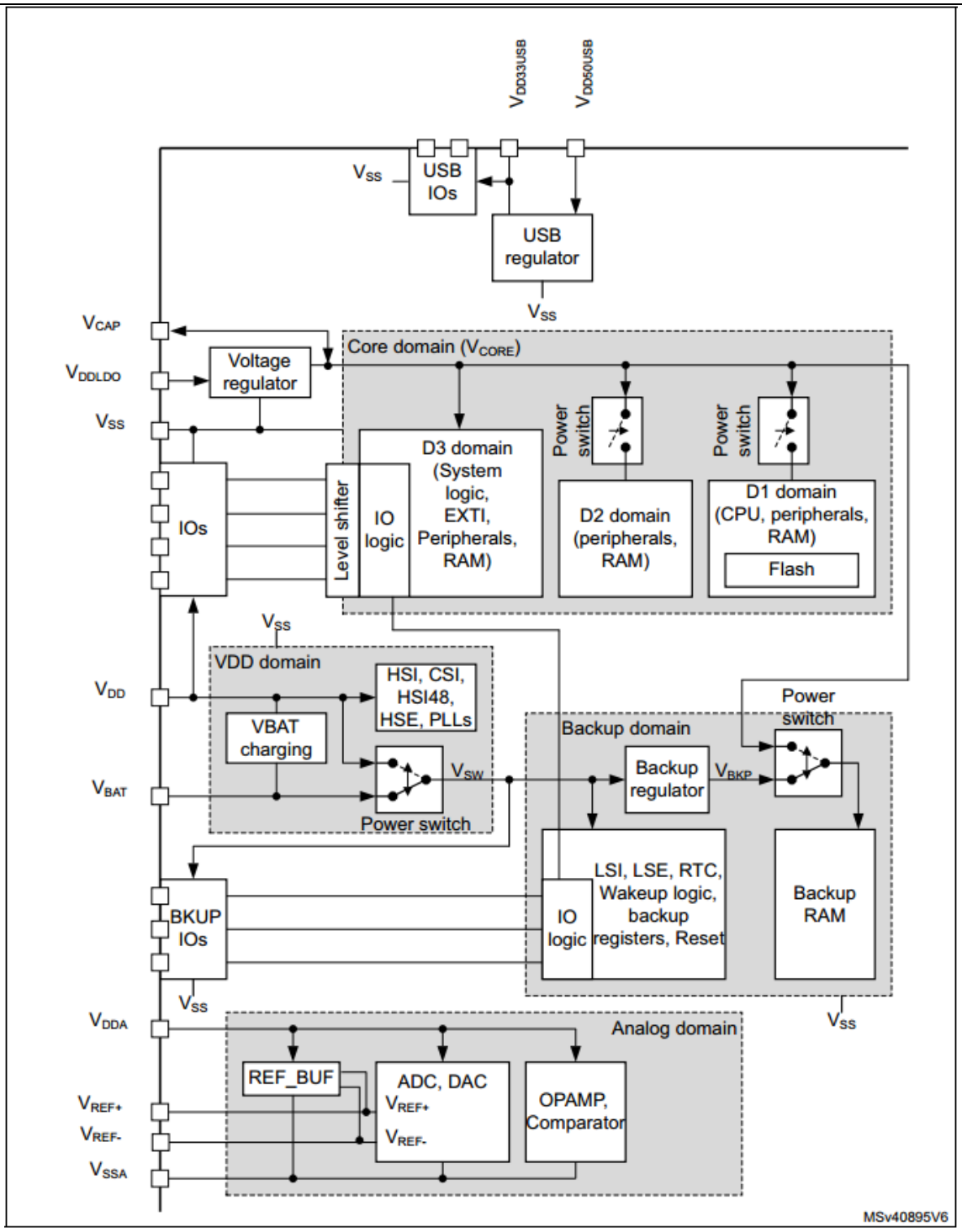
- 电源管理部分涉及到的各种低功耗方式会在后面章节中为大家讲解，当前阶段仅需了解低功耗属于电源管理部分即可。
- 电源管理部分最繁琐的就是 CPU，D1，D2，D3 域的各种运行，待机，停机状态切换，这部分知识点也放在后面低功耗章节学习。

14.2 电源

电源是系统稳定运行的根本，主要分为以下几个知识点，电源供电、供电监控、电源管理和低功耗。当前阶段主要了解电源供电和硬件上电时序。

14.2.1 电源供电

学习 STM32H7 的电源供电，往往被一堆电源标识 Vdd，Vdda，Vcap，Vss 等搞迷糊，这些标识整明白了，电源供电部分也就理解了，首先看下面的框图：



这些常用标识的解释如下：

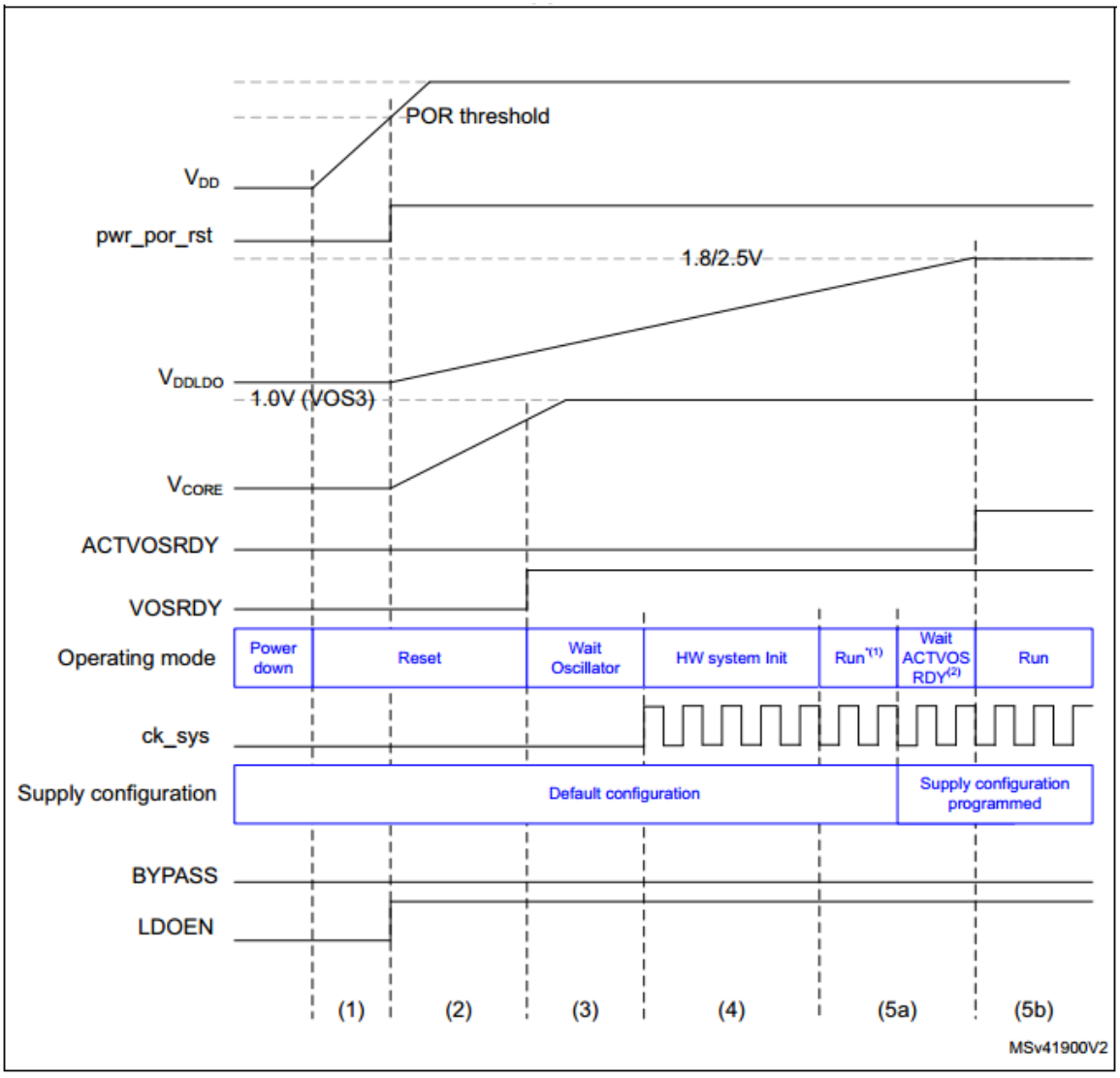
引脚名（标识）	描述
V_{DD}	用于 I/O 引脚和系统模拟部分供电，比如复位、电源管理、振荡器等。
V_{DDA}	用于 ADC、DAC、运放、比较器和电压基准供电，这部分供电是独立的。
V_{REF+}，V_{REF-}	用于 ADC 和 DAC 的基准电压，当使能了 STM32H7 内部的电压基准，将使用内部基准供 V_{REF+}，V_{REF-}。如果没有使能的话，通过外置电压基

	准提供。
V_{BAT}	当 V_{DD} 不供电的时候，由 V_{BAT} 为备份域供电。
V_{DDLDO}	电压稳压器供电。
V_{CAP}	数字核心域供电。
$V_{DD50USB}$	USB 稳压器供电。
$V_{DD33USB}$	USB 稳压器输出。
V_{SS}	所有电源和模拟稳压器的地端。

对于电源供电部分了解了这些知识点就够用。

14.2.2 系统上电启动

系统上电到程序开始运行期间，H7 都做哪些工作，一张时序图可以说明问题：



对于上面的截图，主要看 Operating mode 部分，依次是 Power down –> Reset -> Wait Oscillator ->HW system init -> Run -> Wait ACTVOS RDY –> Run，即断电状态 -> 复位状态 -> 等待 HSI 就绪->硬件初始化 -> 运行 -> 等 ACTVOS 位就绪 -> 正式运行。

详细些的执行流程如下：

- 当系统上电后，POR（Power on reset 上电复位）会检测 V_{DD} 供电，当 V_{DD} 大于 POR 设置的阀值时，将使能电压稳压器，注意看 V_{DD} 那条线的变化。
- 看 V_{CORE} 那条曲线，只要 VOSRDY 未就绪，就会一直处于复位状态。
- 一旦 V_{CORE} 正常输出，系统将走出复位状态，内部高速 RC 振荡器 HSI 将使能。
- HSI 稳定后，将开始系统初始化，主要是 Flash 和可选字节的加载，这些都是由硬件完成的，CPU 也将以受限的方式运行（主要是指不允许对 RAM 进行写操作）。
- 软件程序初始化系统，包括供电配置。当供电配置完成后，等待 ACTVOSRDY 位置 1，完成置 1 后，CPU 就进入正常的运行的模式，允许读写 RAM 了。

14.2.3 电源管理

关于电源管理部分，H7 参考手册中讲解的还挺复杂的，当前阶段我们仅需了解几个重要的工作状态即可，看到这几个单词了要认识，因为官方文档中多处要用这几个标识。

为了实现各种低功耗模式，CPU 和 D1，D2，D3 域支持的各种模式如下：

- **CPU 模式**

  CRun：运行状态，CPU 和 CPU 子系统外设正常运行。

  CSleep：休眠状态，CPU 时钟停止运行，CPU 子系统外设正常运行。

  CStop：停止状态，CPU 和 CPU 子系统外设都停止运行。

- **D1 域模式**

  DRun：运行状态，D1 域的总线矩阵正常运行，CPU 子系统运行在 CRun 或者 CSleep 模式。

  DStop：停机状态，D1 域的总线矩阵时钟停止运行，CPU 子系统运行在 CStop 模式，PDDS_D1 位选择 DStop 模式。

  DStandby：待机状态，D1 域的总线矩阵断电，CPU 子系统运行在 CStop 模式，PDDS_D1 位选择 DStandby 模式。

- **D2 域模式**

  DRun：运行状态，D2 域的总线矩阵正常运行，CPU 子系统在 D2 域中有分配的外设，CPU 子系统运行在 CRun 或者 CSleep 模式。

  DStop：停机状态，D2 域的总线矩阵时钟停止运行，CPU 子系统没有在 D2 域分配外设，PDDS_D1 位选择 DStop 模式。或者 CPU 子系统在 D2 域中有分配的外设，CPU 子系统运行在 CStop 模式，PDDS_D1 位选择 DStop 模式。

DStandby：待机状态，D2 域的总线矩阵断电，CPU 子系统没有在 D2 域分配外设，PDDS_D1 位选择 DStandby 模式。或者 CPU 子系统在 D2 域中有分配的外设，CPU 子系统运行在 CStop 模式，PDDS_D1 位选择 DStandby 模式。

- **系统/D3 域模式**

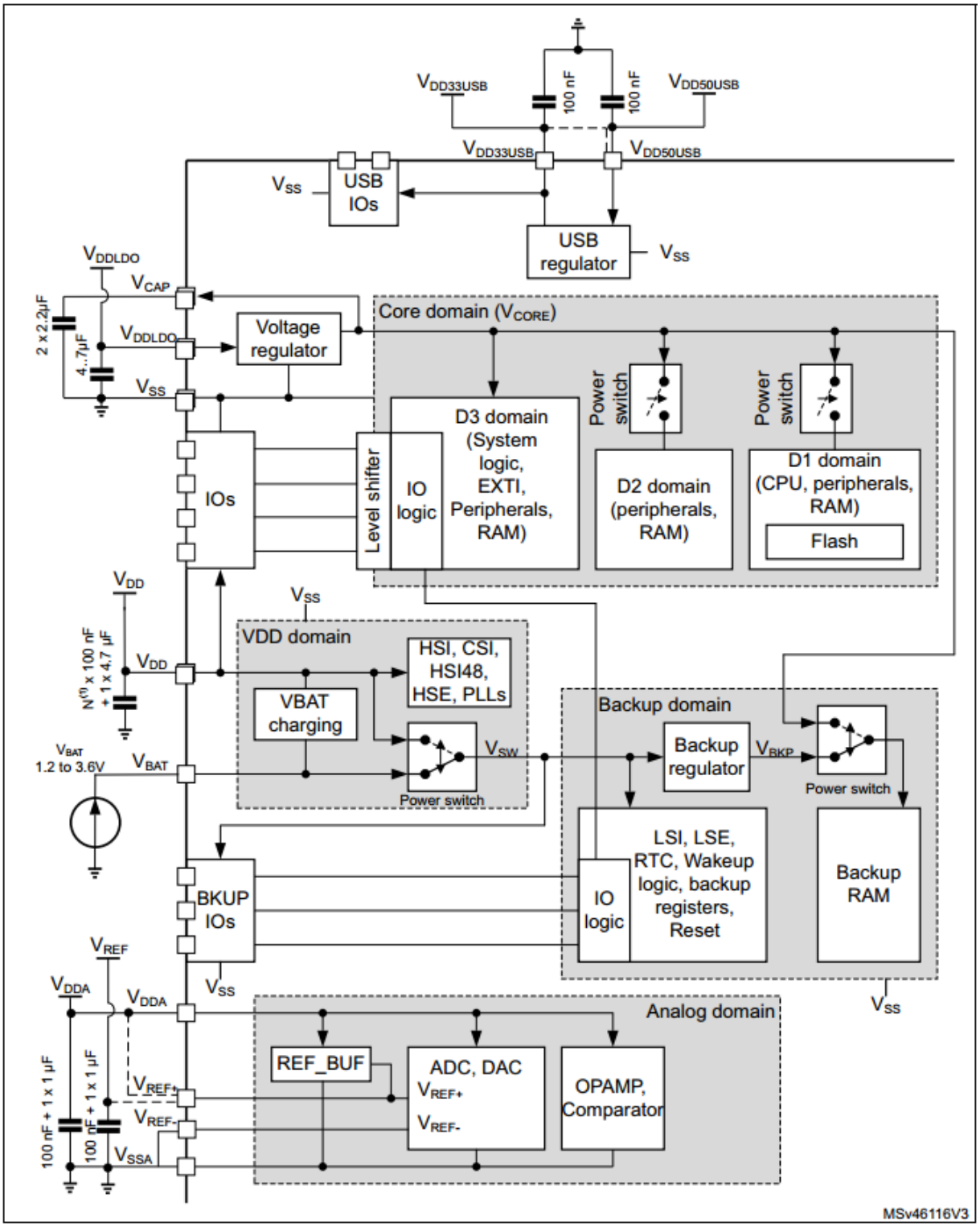
Run：运行状态，系统时钟和 D3 域总线矩阵时钟处于运行状态。CPU 子系统处于 CRun 和 CSleep 模式，或者一个唤醒信号处于激活状态。

Stop：停止状态，系统时钟和 D3 域总线矩阵时钟处于停止状态，CPU 子系统处于 CStop 模式。所有的唤醒信号都处于非激活状态，并且至少某个域的一个 PDDS_Dn 位选择了 Stop 模式。

Standby：待机状态，系统处于断电状态，CPU 子系统处于 CStop 模式，所有的唤醒信号都处于非激活状态，并且所有域的所有 PDDS_Dn 位选择 Standby 模式。

14.2.4电源去耦电容的选择

每个电源对 （VDD/VSS， VDDA/VSSA ...）必须使用下述的滤波陶瓷电容去耦。这些电容必须尽量靠近芯片引脚，以确保器件正常工作。不建议去掉滤波电容来降低 PCB 尺寸或成本，这可能导致器件工作不正常。



14.3 硬件复位

所有数字计算机系统都是由某种形式的震荡时钟电路驱动的。这种电路被称为系统的“脉搏”，是系统正确运行的关键。如果振荡器失灵，系统将完全无法运行，如果振荡器运行不规律，系统执行的所有与时间有关的计算都会有误差。

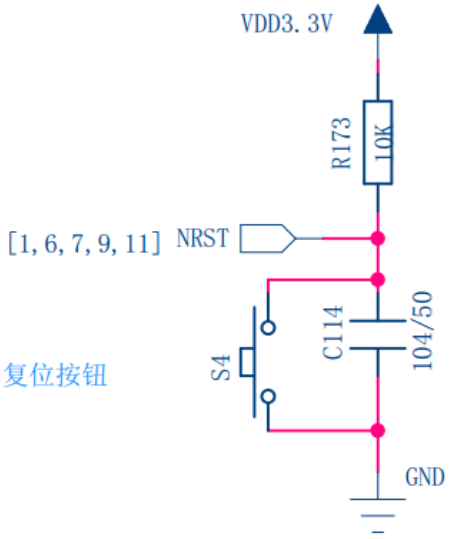
所有微控制器的启动流程都不通用。由于硬件的复杂性，必须运行一段由厂家定义的短小的“复位程序”来使硬件处于一种正确的状态，然后再开始执行用户程序。运行这个复位程序需要时间并且要求微控制器的振荡器已经运行。

当系统由可靠的电源供电时，一旦通电，电源迅速地达到额定输出电压，一旦断电，电源迅速地下降到 0V，并且在接通的时候，电压不会降低。这时能够可靠地使用基于一个电容和一个电阻的低成本硬件复位。这种形式的复位电路称为阻容复位。

如果电源不够可靠，而涉及安全性，这种简单的阻容解决方案就不合适了。

14.3.1 上电复位和手动复位

STM32H7 开发板的硬件复位原理图如下：



- STM32 这款 CPU 的复位引脚是低电平有效，即 NRST 为低电平时，CPU 处于复位状态。
- R173 单的 RC 复位电路。当系统上电瞬间，C114 电容两端电压可以认为是 0，CPU 处于复位状态。3.3V 电源通过 R173 给 C114 充电，当 C114 的电压升到 CPU 的高电平门槛电压时，CPU 退出复位状态转入运行状态。
- 在设计电路时，需要选择适当的 R 值和 C 值，以保证 NRST 低电平持续时间满足 CPU 复位最小脉宽的要求。
- 当按下 S4 轻触开关时，C114 两端被短路接地，可实现手动复位 CPU。

注，根据需要，大家也可以使用 STM32H7 NRST 引脚的内部上拉：
http://www.armbbs.cn/forum.php?mod=viewthread&tid=93144

14.3.2 复位序列

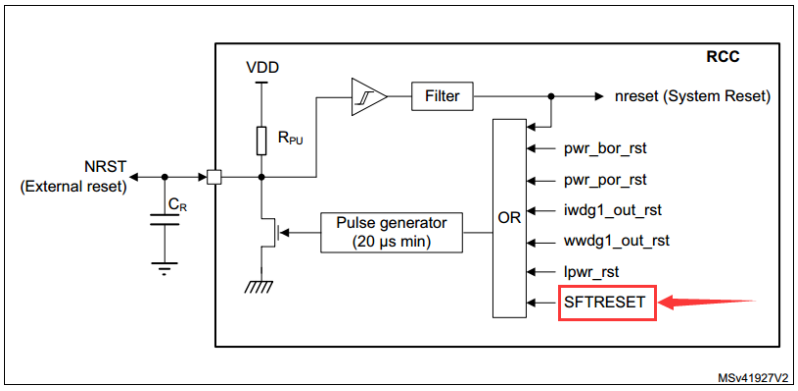
前面第 11 章的 13.3.1 小节讲解了复位系列的相关知识，再结合本章节的上电复位和下电复位，大家会对其有一个较全面的认识，更多复位序列的知识直接看 13.3.1 小节即可。

14.4 软件复位

除了上电和手动复位，程序设计设置中还经常要用到软件复位，即调用一条函数就可以实现复位功能。此函数已经由 CMSIS 软件包中的 core_cm7.h 文件提供，函数如下：

```
/**
  \brief   System Reset
  \details Initiates a system reset request to reset the MCU.
 */
__STATIC_INLINE void __NVIC_SystemReset(void)
{
  __DSB();                                          /* Ensure all outstanding memory accesses included
                                                       buffered write are completed before reset */
  SCB->AIRCR  = (uint32_t)((0x5FAUL << SCB_AIRCR_VECTKEY_Pos)      |
                           (SCB->AIRCR & SCB_AIRCR_PRIGROUP_Msk) |
                            SCB_AIRCR_SYSRESETREQ_Msk    );         /* Keep priority group unchanged */
  __DSB();                                                          /* Ensure completion of memory access */

  for(;;)                                                           /* wait until reset */
  {
    __NOP();
  }
}
```

软件复位反映到实际硬件上，就是给硬件复位部分发一个复位信号：



14.5 RCC 时钟控制

STM32H7 有如下六种时钟可供使用：

- HSI (High-speed internal oscillator) ：

HSI 是内部的高速 RC 振荡器，频率 64MHz，可被用于系统时钟。优势是低成本，无需外部时钟，快速启动（仅需几个微秒），缺点是精度差，即使经过校准。

- HSE (High-speed external oscillator):

  HSE 是外部的高速振荡器，通过外接时钟源，有源或者无源晶振驱动，时钟范围 4-48MHz。优势是精度高，缺点是增加成本。

- LSE (Low-speed external oscillator)

  LSE 是外部的低速振荡器，通过外接时钟源，有源或者无源晶振驱动，一般接 32.768KHz，主要用于 RTC 实时时钟。

- LSI (Low-speed internal oscillator)

  LSI 是内部的低速 RC 振荡器，频率约是 32KHz，主要用于独立看门狗和自动唤醒，也可以用于 RTC 实时时钟。

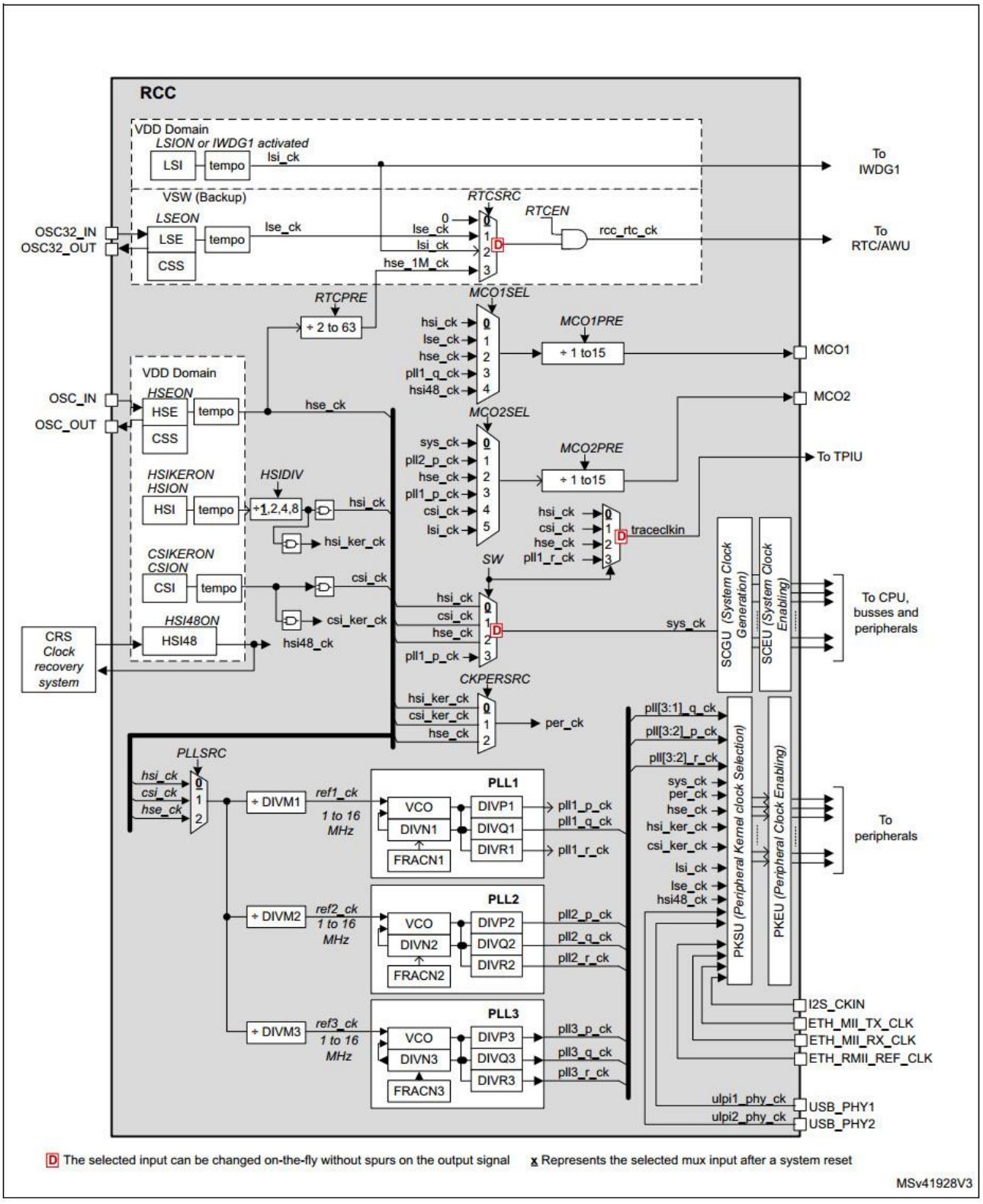
- CSI (Low-power internal oscillator)

  CSI 是内部的低速振荡器，频率约是 4MHz，相比 64MHz 的 HSI，主要用于低功耗。

- HSI48 (High-speed 48 MHz internal oscillator)

  HSI48 是内部高速振荡器，频率约是 48MHz，用于给特定的外设提供时钟，比如 USB。

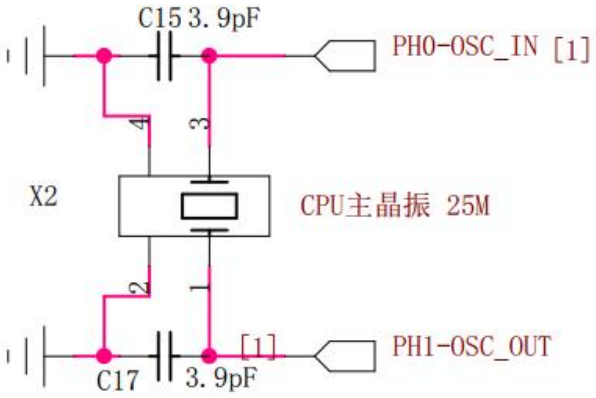
通过下面的时钟树再进一步的认识这几个时钟：



14.5.1 HSE 和 LSE 硬件设计

◆ **HSE 时钟**

当前 V7 开发板是用的 25MHz 晶振为 HSE 提供时钟，硬件设计如下：

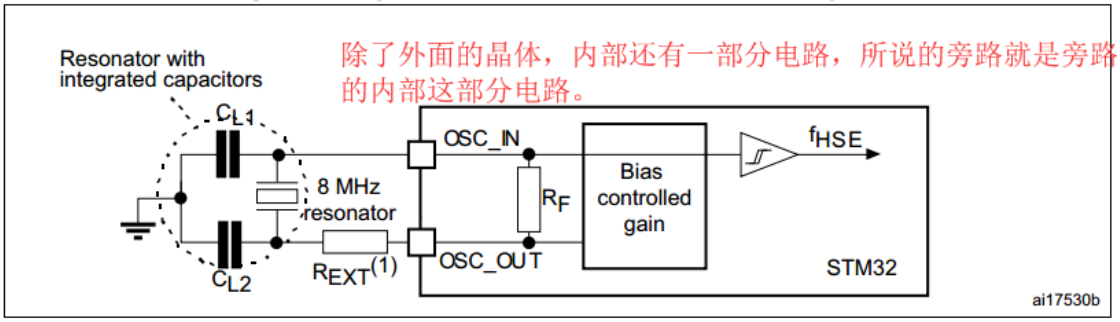


NX3225HA-25MHz-EXS00A-CH00343

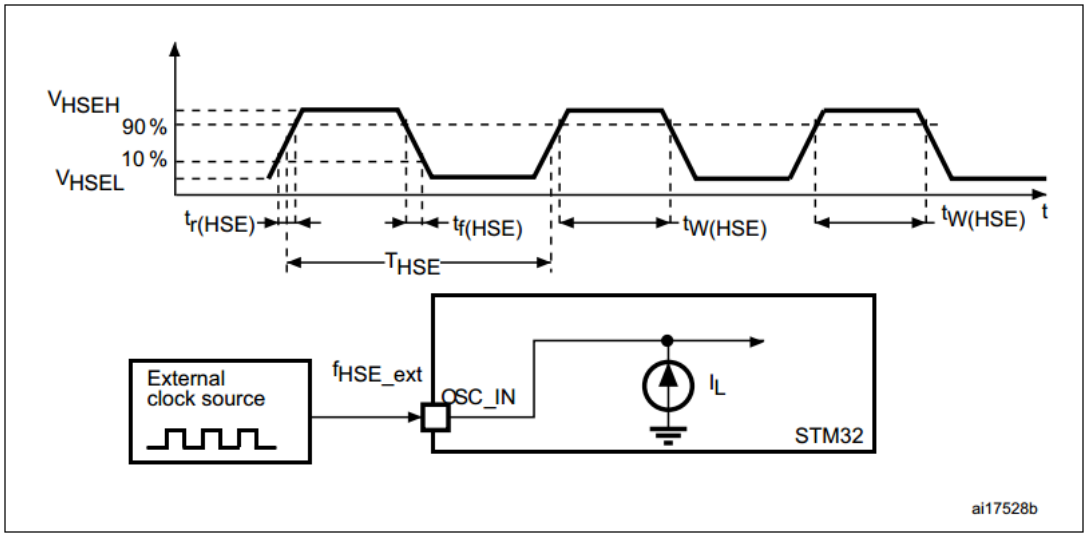
晶振和负载电容需要尽可能近地靠近 H7 的晶振引脚，以减小输出失真和启动稳定时间。负载电容值必须根据选定的晶振进行调节。□□

对于 C15 和 C17，我们推荐使用高质量陶瓷电容，这种电容是设计用于需要高频率的场合，并且可以满足晶体或谐振器的需求。C15 和 C17 通常具有相同的值。

这里再额外补充一个知识点，HSE 旁路时钟和外置晶振区别：当前 V7 板子是采用的外置晶振模式，高速外部（HSE）时钟可以使用一个 4 到 48MHz 的晶振 / 陶瓷谐振振荡器产生：

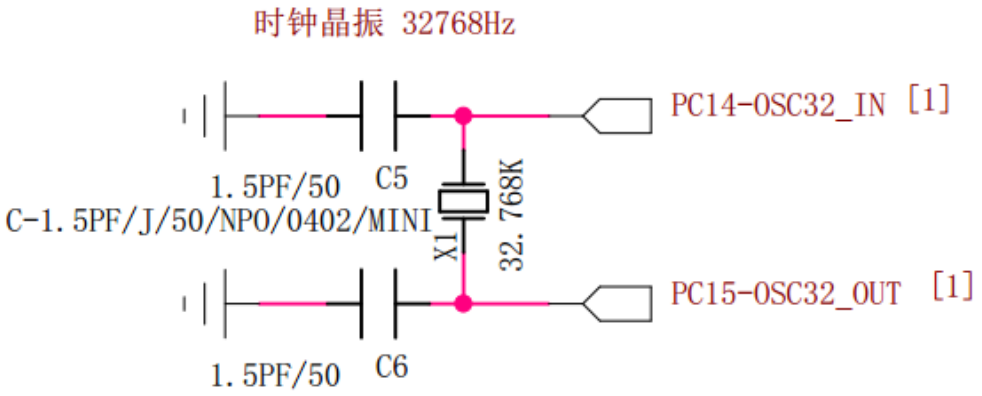


而 bypass 旁路的意思就是不使用它，绕过它。具体到 HSE 旁路的话，用户直接提供 4-50MHz 的时钟源即可，可以使用有源晶振或者 FPGA 提供时钟等方式：



◆ **LSE 时钟**

当前 V7 开发板是用的 32768Hz 晶振为 LSE 提供时钟，硬件设计如下：



STM32 的 LSE 晶振起振难（又称 RTC 起振）是老毛病了，选取晶振和配套电容比较讲究，最好按照 ST 提供的厂家和配套电容选取：http://www.armbbs.cn/forum.php?mod=viewthread&tid=87673 。

14.5.2 时钟配置

STM32H7 开发板使用的外部晶振频率是 25MHz，下面分步说明如何让其通过这个频率工作到 400MHz 的主频。

- 第 1 步：在 stm32h7xx_hal_conf.h 文件配置 HSE_VALUE

  配置的大小要跟板子的实际晶振大小匹配。

```
#if !defined  (HSE_VALUE)
#define HSE_VALUE    ((uint32_t)8000000) /*!< Value of the External oscillator in Hz */
#endif /* HSE_VALUE */
```

- 第 2 步：系统上电后，在启动文件 startup_stm32h743xx.s 的复位中断服务程序里面会调用函数 SystemInit。

```
Reset_Handler    PROC
                 EXPORT  Reset_Handler                    [WEAK]
        IMPORT  SystemInit
        IMPORT  __main

                 LDR     R0, =SystemInit
                 BLX     R0
                 LDR     R0, =__main
                 BX      R0
                 ENDP
```

以往 STM32F1 和 STM32F4 系列都会在函数 SystemInit 里面配置 PLL 锁相环，使用了 HAL 后，需要在 main 函数里面配置。当前 SystemInit 函数实现的功能如下：

```
1.  /**
2.    * @brief  Setup the microcontroller system
3.    *         Initialize the FPU setting, vector table location and External memory
4.    *         configuration.
5.    * @param  None
6.    * @retval None
7.    */
8.  void SystemInit (void)
```

```
9.   {
10.    /* FPU settings ------------------------------------------------------------*/
11.    #if (__FPU_PRESENT == 1) && (__FPU_USED == 1)
12.      SCB->CPACR |= ((3UL << 10*2)|(3UL << 11*2));  /* set CP10 and CP11 Full Access */
13.    #endif
14.    /* Reset the RCC clock configuration to the default reset state ------------*/
15.    /* Set HSION bit */
16.    RCC->CR |= RCC_CR_HSION;
17.
18.    /* Reset CFGR register */
19.    RCC->CFGR = 0x00000000;
20.
21.    /* Reset HSEON, CSSON , CSION,RC48ON, CSIKERON PLL1ON, PLL2ON and PLL3ON bits */
22.    RCC->CR &= (uint32_t)0xEAF6ED7F;
23.
24.    /* Reset D1CFGR register */
25.    RCC->D1CFGR = 0x00000000;
26.
27.    /* Reset D2CFGR register */
28.    RCC->D2CFGR = 0x00000000;
29.
30.    /* Reset D3CFGR register */
31.    RCC->D3CFGR = 0x00000000;
32.
33.    /* Reset PLLCKSELR register */
34.    RCC->PLLCKSELR = 0x00000000;
35.
36.    /* Reset PLLCFGR register */
37.    RCC->PLLCFGR = 0x00000000;
38.    /* Reset PLL1DIVR register */
39.    RCC->PLL1DIVR = 0x00000000;
40.    /* Reset PLL1FRACR register */
41.    RCC->PLL1FRACR = 0x00000000;
42.
43.    /* Reset PLL2DIVR register */
44.    RCC->PLL2DIVR = 0x00000000;
45.
46.    /* Reset PLL2FRACR register */
47.
48.    RCC->PLL2FRACR = 0x00000000;
49.    /* Reset PLL3DIVR register */
50.    RCC->PLL3DIVR = 0x00000000;
51.
52.    /* Reset PLL3FRACR register */
53.    RCC->PLL3FRACR = 0x00000000;
54.
55.    /* Reset HSEBYP bit */
56.    RCC->CR &= (uint32_t)0xFFFBFFFF;
57.
58.    /* Disable all interrupts */
59.    RCC->CIER = 0x00000000;
60.
61.    /* Change  the switch matrix read issuing capability to 1 for the AXI SRAM target (Target 7) */
62.    *((__IO uint32_t*)0x51008108) = 0x00000001;
63.
64.  #if defined (DATA_IN_ExtSRAM) || defined (DATA_IN_ExtSDRAM)
65.    SystemInit_ExtMemCtl();
66.  #endif /* DATA_IN_ExtSRAM || DATA_IN_ExtSDRAM */
```

```
67.
68.    /* Configure the Vector Table location add offset address ------------------*/
69.  #ifdef VECT_TAB_SRAM
70.    SCB->VTOR = D1_AXISRAM_BASE  | VECT_TAB_OFFSET; /* Vector Table Relocation in Internal ITCMSRAM */
71.  #else
72.    SCB->VTOR = FLASH_BANK1_BASE | VECT_TAB_OFFSET; /* Vector Table Relocation in Internal FLASH */
73.  #endif
74.
75.  }
```

第 12 行：使能 FPU 单元。

第 16 – 59 行：复位 RCC 相关寄存器。

第 62 行：这条语句在这里的作用涉及到 STM32H7 硬件版本 Y 的一个 bug 解决方案，详情可以看这个帖子：http://www.armbbs.cn/forum.php?mod=viewthread&tid=87346 。

第 69 – 73 行：设置中断向量表的位置。

- 第 3 步：在 main 函数的外设驱动初始化函数里面完成时钟初始化，主要是 PLL 锁相环，让芯片最终工作到 400MHz。

14.6 总结

本章节就为大家讲解这么多，本章节的知识点相对比较多，比较杂，不容易一下子都掌握了。随着后面章节的进行，还会深入的讲解这些知识点。

第15章 STM32H7 的 GPIO 基础知识（重要）

本章教程为大家讲解 GPIO（General-purpose I/Os）基础知识。GPIO 配置是所有外设驱动的基础，大家务必要熟练掌握。

15.1 初学者重要提示
15.2 GPIO 功能简介
15.3 GPIO 功能模式分析（重要）
15.4 GPIO 的拉电流负载和灌电流负载能力
15.5 IO 补偿单元，用于高速
15.6 GPIO 兼容 CMOS 和 TTL 电平
15.7 不使用的引脚推荐设为模拟模式
15.8 总结

15.1 初学者重要提示

- 学习本章节务必要认真学习并掌握 15.3 小节 GPIO 功能模式分析，学好它们，对后续章节中外设的学习大有裨益。
- 对于不使用的引脚，推荐设置为模拟模式，悬空即可。
- GPIO 的速度等级高的时候，最好使能 IO 补偿单元。
- GPIO 还涉及到一个注入电流的问题，此贴可以作为了解：http://www.armbbs.cn/forum.php?mod=viewthread&tid=87675 。

15.2 GPIO 功能简介

STM32H7 的 GPIO 特性如下：

- 输出状态：开漏/推挽 + 上拉/下拉电阻。
- 通过输出数据寄存器（GPIOx_ODR）或者外设（GPIO 设置为复用模式时）输出数据。
- GPIO 速度等级设置。
- 输入状态：浮空，上拉/下拉，模拟。
- 通过输入数据寄存器（GPIOx_IDR）或者外设（GPIO 设置为复用模式）输入数据。
- 通过寄存器 GPIOx_BSRR 实现对寄存器 GPIOx_ODR 的位操作。
- 通过配置寄存器 GPIOx_LCKR 的锁机制，实现冻结 IO 口配置。

- 每两个时钟周期就可以翻转一次 IO。
- 高度灵活的引脚复用功能，允许 IO 引脚既可以做 GPIO 也可以做功能复用。

15.3 GPIO 功能模式分析（重要）

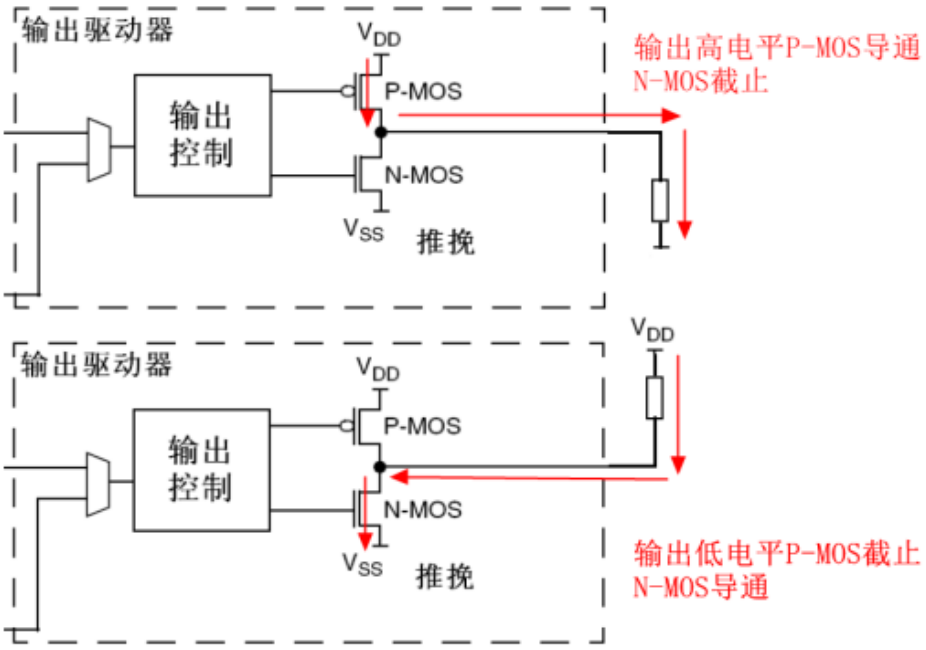
STM32H7 的 GPIO 端口可以配置为如下的 8 种模式：

- 输入浮空
- 输入上拉
- 输入下拉
- 模拟功能
- 具有上拉或下拉功能的开漏输出
- 具有上拉或下拉功能的推挽输出
- 具有上拉或下拉功能的复用功能推挽
- 具有上拉或下拉功能的复用功能开漏

由于上拉和下拉是可选配置，对应的 HAL 库配置使用下面 6 种就可以表示：

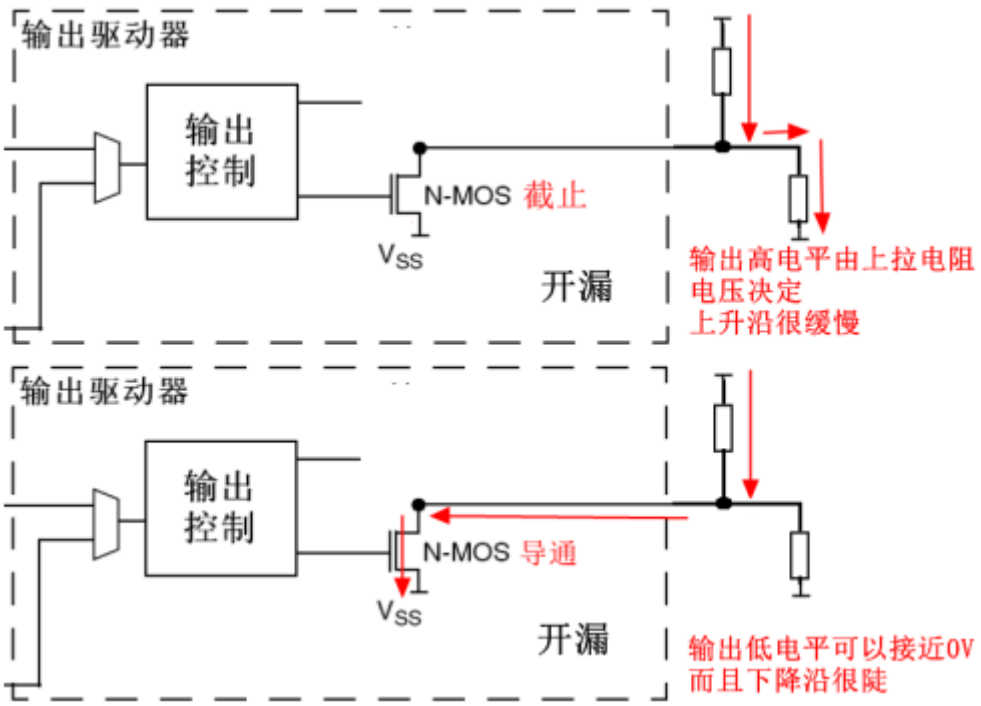
- GPIO_MODE_INPUT 输入模式
- GPIO_MODE_OUTPUT_PP 推挽输出
- GPIO_MODE_OUTPUT_OD 开漏输出
- GPIO_MODE_AF_PP 复用推挽
- GPIO_MODE_AF_OD 复用开漏
- GPIO_MODE_ANALOG 模拟模式

15.3.1推挽输出



推挽电路是两个参数相同的三极管或 MOSFET，以推挽方式存在于电路中。 电路工作时，两只对称的开关管每次只有一个导通，导通损耗小、效率高。输出既可以向负载灌电流，也可以从负载抽取电流。推拉式输出级提高电路的负载能力。 相对于开漏输出模式，推挽输出最大优势是输出高电平时，上升时间快，电压驱动能力强。

15.3.2开漏输出



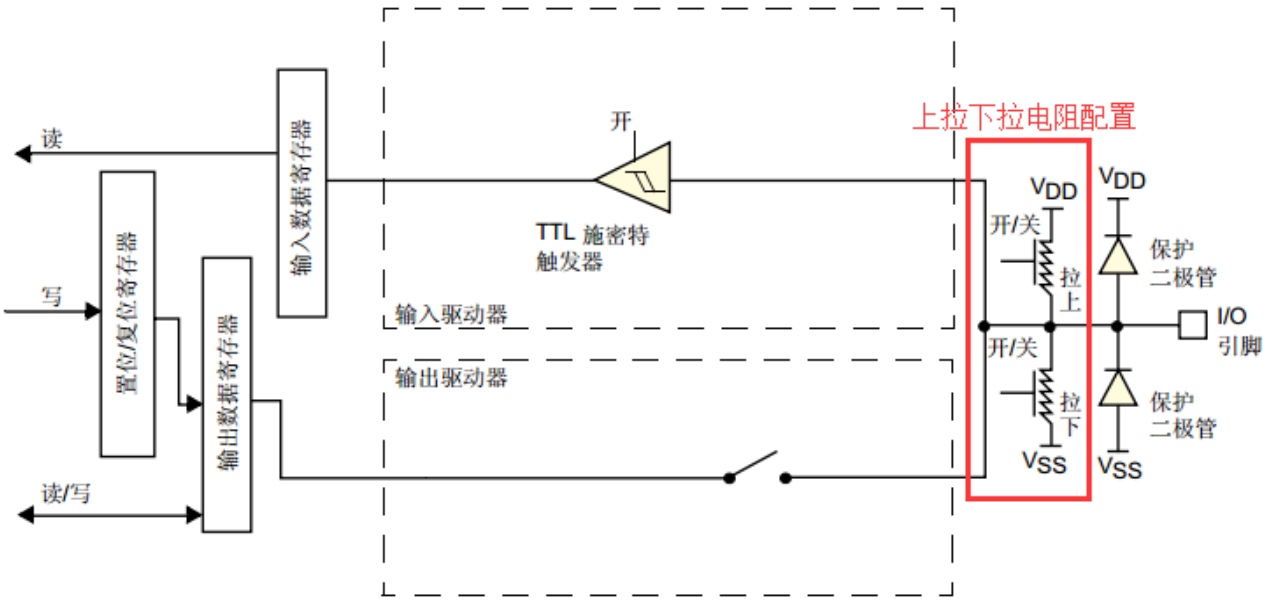
开漏端相当于 MOS 管的漏极（三极管的集电极），要得到高电平状态必须外接上拉电阻才行，因此输出高电平的驱动能力完全由外接上拉电阻决定，但是其输出低电平的驱动能力很强。开漏形式的电路有以下几个特点：

1. 输出高电平时利用外部电路的驱动能力，减少 IC 内部的驱动。
2. 开漏是用来连接不同电平的器件，匹配电平用的，因为开漏引脚不连接外部的上拉电阻时，只能输出低电平。如果需要同时具备输出高电平的功能，则需要接上拉电阻，很好的一个优点是通过改变上拉电源的电压，便可以改变传输电平。上拉电阻的阻值决定了逻辑电平转换的速度。阻值越大，速度越低，功耗越小。
3. 开漏输出提供了灵活的输出方式，但是也有其弱点，就是带来上升沿的延时。因为上升沿是通过外接上拉无源电阻对负载充电，所以当电阻选择小时延时就小，但功耗大；反之延时大功耗小。所以如果对延时有要求，则建议用下降沿输出。
4. 可以将多个开漏输出连接到一条线上。通过一只上拉电阻，在不增加任何器件的情况下，形成“与逻辑”关系，即“线与”。可以简单的理解为：在所有引脚连在一起时，外接一上拉电阻，如果有一个引脚输出为逻辑 0，相当于接地，与之并联的回路“相当于被一根导线短路”，所以外电路逻辑电平便为 0，只有都为高电平时，与的结果才为逻辑 1。

15.3.3复用推挽和开漏

复用指的是 GPIO 切换到 CPU 内部设备（比如 SPI,I2C,UART 等电路），也就是 GPIO 不是作为普通 IO 使用，是由内部设备直接驱动。推挽和开漏的特征同上。

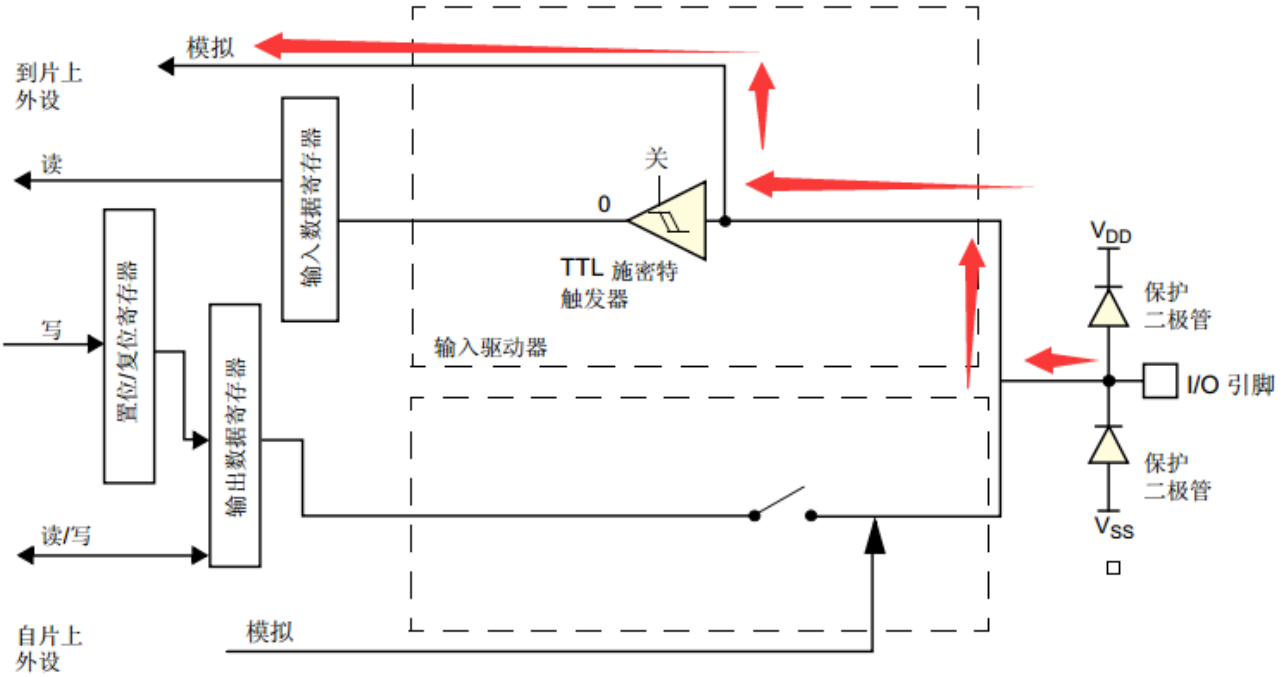
15.3.4四种输入模式



通过上面的引脚结构图可以得到如下三种方式

- 浮空输入：CPU 内部的上拉电阻、下拉电阻均断开的输入模式。
- 下拉输入：CPU 内部的下拉电阻使能、上拉电阻断开的输入模式。
- 上拉输入：CPU 内部的上拉电阻使能、下拉电阻断开的输入模式。

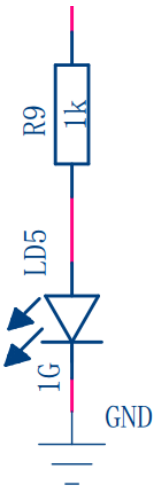
而模拟输入模式是 GPIO 引脚连接内部 ADC。



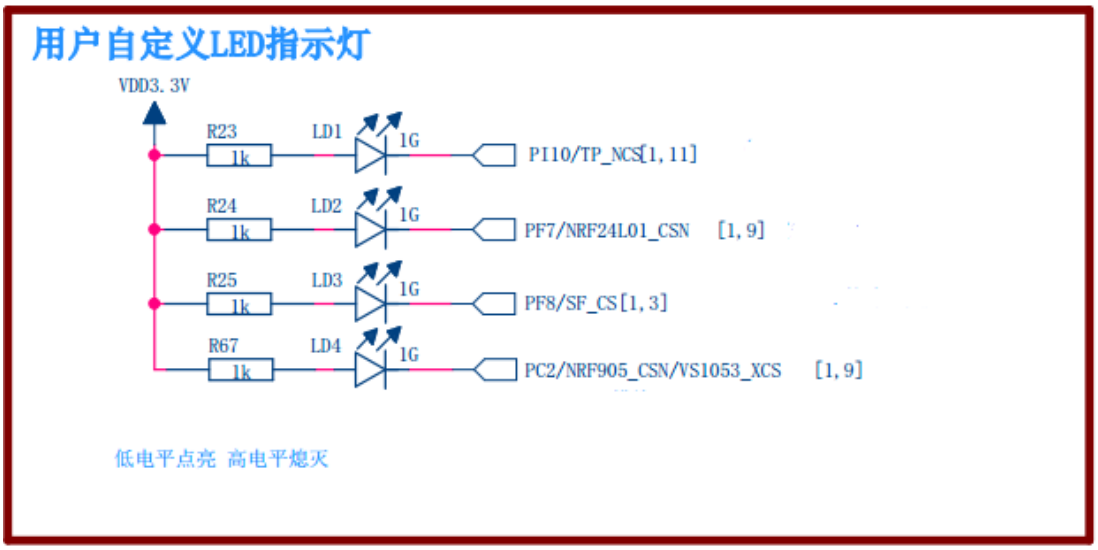
15.4 GPIO 的拉电流负载和灌电流负载能力

这里先普及点小知识，什么是拉电流负载，什么是灌电流负载。

- **拉电流负载**：一种负载电流从驱动门流向外电路，称为拉电流负载。比如使用 STM32H7 的 GPIO 直接驱动 LED 就是拉电流形式。



- **灌电流负载**：负载电流从外电路流入驱动门，称为灌电流负载。比如下面这种形式的 LED 驱动电路



有了上面这些知识后再来看 STM32H7 的 IO 驱动能力（截图来自 STM32H7 数据手册）：

Symbols	Ratings	Max	Unit
ΣIV_{DD}	Total current into sum of all V_{DD} power lines (source)(1)	620	mA
ΣIV_{SS}	Total current out of sum of all V_{SS} ground lines (sink)(1)	620	
IV_{DD}	Maximum current into each V_{DD} power pin (source)(1)	100	
IV_{SS}	Maximum current out of each V_{SS} ground pin (sink)(1)	100	
I_{IO}	Output current sunk by any I/O and control pin	20	
$\Sigma I_{(PIN)}$	Total output current sunk by sum of all I/Os and control pins(2)	140	
	Total output current sourced by sum of all I/Os and control pins(2)	140	
$I_{INJ(PIN)}$(3)(4)	Injected current on FT_xxx, TT_xx, RST and B pins except PA4, PA5	-5/+0	
	Injected current on PA4, PA5	-0/0	
$\Sigma I_{INJ(PIN)}$	Total injected current (sum of all I/Os and control pins)(5)	±25	

通过上面的截图可知：STM32H7 总的拉电流和灌电流不可超过 140mA，单个引脚最大不可超过 20mA，这个知识点，大家要知道。

15.5 IO 补偿单元，用于高速

IO 补偿单元用于控制 I/O 通信压摆率（t_{fall} / t_{rise}）以此来降低 I/O 噪声。当前 STM32H7 的速度等级可以配置为以下四种：

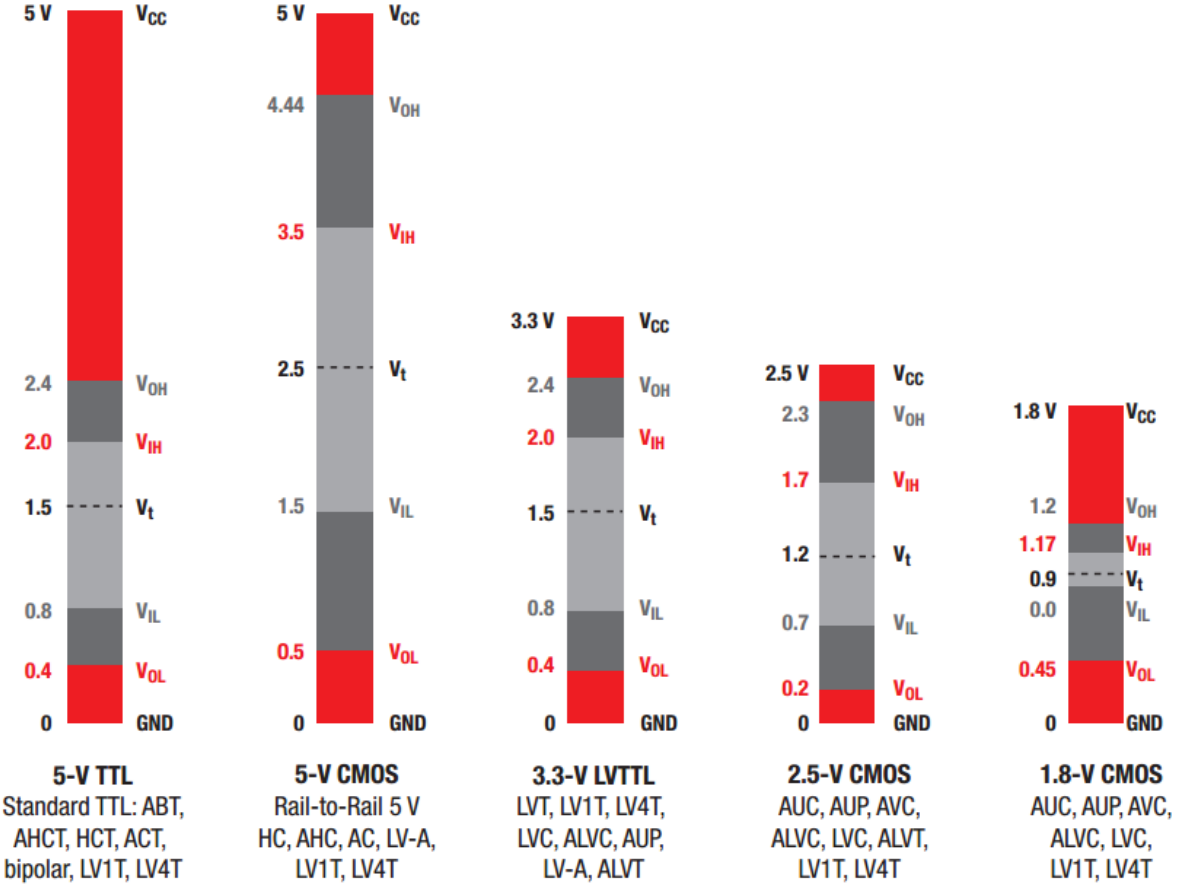
```
/** @defgroup GPIO_speed_define  GPIO speed define
  * @brief GPIO Output Maximum frequency
  * @{
```

```
  */
#define  GPIO_SPEED_FREQ_LOW         ((uint32_t)0x00000000U)  /*!< Low speed     */
#define  GPIO_SPEED_FREQ_MEDIUM      ((uint32_t)0x00000001U)  /*!< Medium speed  */
#define  GPIO_SPEED_FREQ_HIGH        ((uint32_t)0x00000002U)  /*!< Fast speed    */
#define  GPIO_SPEED_FREQ_VERY_HIGH   ((uint32_t)0x00000003U)  /*!< High speed    */
```

使用后两种速度等级的话，最好使能 IO 补偿单元。另外不同速度等级下，IO 补偿使能与否对 IO 最大速度的影响可以看此贴：http://www.armbbs.cn/forum.php?mod=viewthread&tid=87677 。

15.6 GPIO 兼容 CMOS 和 TTL 电平

CMOS 和 TTL 电平兼容问题也是一个比较重要的知识点，关于这方面的知识已经在论坛进行了总结（由于截图较多，较长，就不整理到教程里面了）：
http://www.armbbs.cn/forum.php?mod=viewthread&tid=87676 。



15.7 不使用的引脚推荐设置为模拟模式

主要从功耗和防干扰考虑。

- 所有用作带上拉电阻输入的 I/O 都会在引脚外部保持为低时产生电流消耗。此电流消耗的值可通过使用的静态特性中给出的上拉 / 下拉电阻值简单算出。
- 对于输出引脚，还必须考虑任何外部下拉电阻或外部负载以估计电流消耗。

- 若外部施加了中间电平，则额外的 I/O 电流消耗是因为配置为输入的 I/O。此电流消耗是由用于区分输入值的输入施密特触发器电路导致。除非应用需要此特定配置，否则可通过将这些 I/O 配置为模拟模式以避免此供电电流消耗。 ADC 输入引脚应配置为模拟输入就是这种情况。
- 任何浮空的输入引脚都可能由于外部电磁噪声，成为中间电平或意外切换。为防止浮空引脚相关的电流消耗，它们必须配置为模拟模式，或内部强制为确定的数字值。这可通过使用上拉 / 下拉电阻或将引脚配置为输出模式做到。

综上考虑，不使用的引脚设置为模拟模式，悬空即可。

15.8 总结

本章节就为大家讲解这么多，其中 GPIO 功能模式小节（本章 15.3）最重要，大家务必要掌握。

第16章 STM32H7 必备的 HAL 库 API（重要）

本章教程为大家讲解制作一个 STM32H7 的例子所需的最基本 API 函数，对于一些常用的 API 函数，一定要熟练掌握这些函数都是实现了什么功能，不常用的函数有个了解即可，用到的时候再去学。

16.1 初学者重要提示
16.2 那些是必备的 API
16.3 源文件 stm32h7xx_hal.c
16.4 stm32h7xx_hal_rcc.c
16.5 stm32h7xx_hal_cortex.c
16.6 总结

16.1 初学者重要提示

- 对于一些常用的函数，大家一定要熟练的掌握都实现了什么功能，比如 HAL_Init，HAL_RCC_OscConfig，HAL_RCC_ClockConfig 等。最好的办法是把这些函数的源码读一遍。

16.2 那些是必备的 API

这里我们通过一个简单的初始化流程来了解 STM32H7 的工程模板所必备的库文件和 API：

- **第 1 步：**系统上电复位，进入启动文件 startup_stm32h743xx.s，在这个文件里面执行复位中断服务程序。
  - 在复位中断服务程序里面执行函数 SystemInit，此函数在文件 system_stm32h7xx.c 里面。
  - 之后是调用编译器封装好的函数，比如用于 MDK 的启动文件是调用__main，最终进入到 main 函数。
- **第 2 步：**进入到 main 函数就可以开始用户应用程序编程了。在这个函数里面要做几个重要的初始化，依次是：
  - MPU 初始化，需要用到库文件 stm32h7xx_hal_cortex.c 和 stm32h7xx_hal_cortex.h。
  - Cache 初始化，需要用到 core_cm7.h 文件。
  - HAL 库初始化函数 HAL_Init，需要用到文件 stm32h7xx_hal.c。
  - 系统时钟初始化，需要用到库文件 stm32h7xx_hal_rcc.c。

前面的两步完成后，就可以开始做用户需要的按键、串口等方面的初始化和应用代码的实现了。这里把我们需要学习的几个库文件整理出来，依次有：

- startup_stm32h743xx.s
- system_stm32h7xx.c
- stm32h7xx_hal.c
- stm32h7xx_hal_cortex.c
- stm32h7xx_hal_rcc.c
- core_cm7.h

其中 startup_stm32h743xx.s 和 system_stm32h7xx.c 已经在第 13 章为大家讲解过，这里不再赘述。而 MPU 和 Cache 涉及到的文件 core_cm7.h 在第 23 和 24 章为大家讲解。本章教程重点为大家讲解文件 stm32h7xx_hal.c、stm32h7xx_hal_cortex.c 和 sm32h7xx_hal_rcc.c。

16.3 源文件 stm32h7xx_hal.c（重要）

这个文件比较杂，像基准电压大小配置，EXTI 配置，IO 补偿配置等都在这个文件里面设置。学习这个文件注意事项：

- HAL 库中各个外设驱动里面的延迟实现是基于此文件提供的时间基准，而这个时间基准既可以使用滴答定时器实现也可以使用通用的定时器实现，默认情况下是用的滴答定时器。
- 函数 HAL_Init 里面会调用时间基准初始化函数 HAL_InitTick，而调用函数 HAL_RCC_ClockConfig 也会调用时间基准初始化函数 HAL_InitTick。
- 如果在中断服务程序里面调用延迟函数 HAL_Delay 要特别注意，因为这个函数的时间基准是基于滴答定时器或者其他通用定时器实现，实现方式是滴答定时器或者其他通用定时器里面做了个变量计数。如此一来，结果是显而易见的，如果其他中断服务程序调用了此函数，且中断优先级高于滴答定时器，会导致滴答定时器中断服务程序一直得不到执行，从而卡死在里面。所以滴答定时器的中断优先级一定要比它们高。

16.3.1 函数 HAL_Init

函数原型：

```
HAL_StatusTypeDef HAL_Init(void)
{
  /* 设置中断优先级分组 */
  HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_4);

  /* 使用滴答定时器做为默认时基，配置为 1ms 滴答，另外系统上电后默认使用的 HIS 时钟 */
  if(HAL_InitTick(TICK_INT_PRIORITY) != HAL_OK)
  {
    return HAL_ERROR;
  }
```

```
  /* 初始化底层硬件 */
  HAL_MspInit();

  /* 返回函数状态 */
  return HAL_OK;
}
```

函数描述：

此函数用于初始化 HAL 库，此函数主要实现如下功能：

- 设置 NVIC 优先级分组是 4。
- 设置滴答定时器的每 1ms 中断一次。
- HAL 库不像之前的标准库，在系统启动函数 SystemInit 里面做了 RCC 初始化，HAL 库是没有做的，所以进入到 main 函数后，系统还在用内部高速时钟 HSI，对于 H7 来说，HSI 主频是 64MHz。
- 函数 HAL_Init 里面调用的 HAL_MspInit 一般在文件 stm32h7xx_hal_msp.c 里面做具体实现，主要用于底层初始化。当前此函数也在文件 stm32h7xx_hal.c 里面，只是做了弱定义。

函数参数：

- 返回值，返回 HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

注意事项：

1. 必须在 main 函数里面优先调用此函数。
2. 用户务必保证每 1ms 一次滴答中断。
3. 关于优先级分组的设置可以看第 21 章节。

使用举例：

此函数的使用比较简单，上电后优先调用即可。

16.3.2 函数 HAL_DeInit

函数原型：

```
HAL_StatusTypeDef HAL_DeInit(void)
{
  /* 复位所有外设 */__set_PRIMASK
  __HAL_RCC_AHB3_FORCE_RESET();
  __HAL_RCC_AHB3_RELEASE_RESET();

  /* 省略未写 */

  __HAL_RCC_APB4_FORCE_RESET();
  __HAL_RCC_APB4_RELEASE_RESET();

  /* 复位底层硬件初始化 */
  HAL_MspDeInit();

  /* 返回值 */
  return HAL_OK;
```

```
}
```

函数描述：

此函数用于复位 HAL 库和滴答时钟。

- 复位了 AHB1,2,3,4 的时钟以及 APB1L，APB1H，APB2,3,4 的时钟。
- 函数 HAL_DeInit 里面调用的 HAL_MspDeInit 一般在文件 stm32h7xx_hal_msp.c 里面做具体实现，主要用于底层初始化，跟函数 HAL_Init 里面调用的 HAL_MspInit 是一对。当前此函数也在文件 stm32h7xx_hal.c 里面，只是做了弱定义。

使用举例：

此函数的使用比较简单，需要调用的时候直接调用即可。

16.3.3 函数 HAL_InitTick

函数原型：

```
__weak HAL_StatusTypeDef HAL_InitTick(uint32_t TickPriority)
{
  /* Configure the SysTick to have interrupt in 1ms time basis*/
  if (HAL_SYSTICK_Config(SystemCoreClock / (1000U / uwTickFreq)) > 0U)
  {
    return HAL_ERROR;
  }

  /* Configure the SysTick IRQ priority */
  if (TickPriority < (1UL << __NVIC_PRIO_BITS))
  {
    HAL_NVIC_SetPriority(SysTick_IRQn, TickPriority, 0U);
    uwTickPrio = TickPriority;
  }
  else
  {
    return HAL_ERROR;
  }

  /* Return function status */
  return HAL_OK;
}
```

函数描述：

此函数用于初始化滴答时钟，此函数相关问题如下：

- 此函数有个前缀__weak ，表示弱定义，用户可以重定义。
- 此函数用于初始化滴答时钟 1ms 中断一次，并且为滴答中断配置一个用户指定的优先级。
- 此函数由 HAL_Init 调用，或者任何其它地方调用函数 HAL_RCC_ClockConfig 配置 RCC 的时候也会调用 HAL_InitTick。
- 调用基于此函数实现的 HAL_Delay 要特别注意，因为这个函数的时间基准是基于滴答定时器或者其他通用定时器实现，实现方式是滴答定时器或者其他通用定时器里面做了个变量计数。如此一来，结果是显而易见的，如果其他中断服务程序调用了此函数，且中断优先级高于滴答定时器，会导致滴答

定时器中断服务程序一直得不到执行，从而卡死在里面。所以滴答定时器的中断优先级一定要比它们高。

函数参数：

- 形参 TickPriority 用于设置滴答定时器优先级。
- 返回值，返回 HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

使用举例：

此函数由 HAL_Init 调用，无需用户操作，除非需要重定义。

16.3.4Systick 的相关函数

调用了函数 HAL_Init 后，Systick 相关的函数就可以使用了。这些函数如下：

函数原型：

```
__weak void HAL_IncTick(void)
__weak uint32_t HAL_GetTick(void)
uint32_t HAL_GetTickPrio(void)
HAL_StatusTypeDef HAL_SetTickFreq(HAL_TickFreqTypeDef Freq)
HAL_TickFreqTypeDef HAL_GetTickFreq(void)
__weak void HAL_Delay(uint32_t Delay)
__weak void HAL_SuspendTick(void)
__weak void HAL_ResumeTick(void)
```

函数描述：

这些函数就比较简单了，下面把这些函数实现的功能做个简单的说明：

- 函数 HAL_IncTick 在滴答定时器中断里面被调用，实现一个简单的计数功能，因为一般滴答定时器中断都是配置的 1ms，所以计数全局变量 uwTick 每毫秒加 1。
- 函数 HAL_GetTick 用于获取全局变量 uwTick 当前的计数。
- 函数 HAL_GetTickPrio 用于获取滴答时钟优先级。
- 函数 HAL_SetTickFreq 和 HAL_GetTickFreq 是一对，前者用于设置滴答中断频率，后再用于获取滴答中断频率。
- 函数 HAL_Delay 用于阻塞式延迟，默认单位是 ms。
- 函数 HAL_SuspendTick 和 HAL_ResumeTick 是一对，前者用于挂起滴答定时器，后者用于恢复。

注意事项：

- 函数有个前缀__weak ，表示弱定义，用户可以重定义。

使用举例：

这些函数都比较简单，这里就不举例了。需要的时候，直接调用即可。

16.3.5 函数 HAL_SYSCFG_VREFBUF_VoltageScalingConfig

函数原型：

```
void HAL_SYSCFG_VREFBUF_VoltageScalingConfig(uint32_t VoltageScaling)
```

函数描述：

此函数用于配置 STM32H7 内部电压基准大小。

- 当形参 VoltageScaling = SYSCFG_VREFBUF_VOLTAGE_SCALE0 时
  输出基准是 2.048 V，条件是 VDDA >= 2.4V。
- 当形参 VoltageScaling = SYSCFG_VREFBUF_VOLTAGE_SCALE1 时
  输出基准是 2.5 V，条件是 VDDA >= 2.8V。
- 当形参 VoltageScaling = SYSCFG_VREFBUF_VOLTAGE_SCALE2 时
  输出基准是 1.5 V，条件是 VDDA >= 1.8V。
- 当形参 VoltageScaling = SYSCFG_VREFBUF_VOLTAGE_SCALE3 时
  输出基准是 1.8 V，条件是 VDDA >= 2.1V。

16.3.6 H7 自带电压基准相关函数

函数原型：

```
void HAL_SYSCFG_VREFBUF_HighImpedanceConfig(uint32_t Mode)
void HAL_SYSCFG_VREFBUF_TrimmingConfig(uint32_t TrimmingValue)
HAL_StatusTypeDef HAL_SYSCFG_EnableVREFBUF(void)
void HAL_SYSCFG_DisableVREFBUF(void)
```

函数描述：

- 函数 HAL_SYSCFG_VREFBUF_HighImpedanceConfig
  此函数用于配置 STM32H7 内部电压基准是否在芯片内部与 VREF+引脚接通。
  - 形参为 SYSCFG_VREFBUF_HIGH_IMPEDANCE_DISABLE 时，表示导通。
  - 形参为 SYSCFG_VREFBUF_HIGH_IMPEDANCE_ENABLE 时，表示高阻，即不导通。
- 函数 HAL_SYSCFG_VREFBUF_TrimmingConfig
  此函数用于内部电压基准的校准调节。
- 函数 HAL_SYSCFG_EnableVREFBUF 和 HAL_SYSCFG_DisableVREFBUF 是一对，分别用于内部电压参考基准的禁止和使能。

16.3.7 函数 HAL_SYSCFG_AnalogSwitchConfig

函数原型：

```
void HAL_SYSCFG_AnalogSwitchConfig(uint32_t SYSCFG_AnalogSwitch , uint32_t SYSCFG_SwitchState )
```

函数描述：

- 引脚 PA0，PA1，PC2，PC3 用于 ADC 时，还有一组对应的可选引脚 PA0_C，PA1_C，PC2_C 和 PC3_C。此函数的作用就是切换可选引脚。关于这个问题的详情可看此贴：http://www.armbbs.cn/forum.php?mod=viewthread&tid=87707 。

16.3.8 BOOST 的使能和禁止（用于 ADC）

函数原型：

```
void HAL_SYSCFG_EnableBOOST(void)
void HAL_SYSCFG_DisableBOOST(void)
```

函数描述：

- 这两个函数用于使能或者禁止 Booster。如果使能了 booster 的话，在供电电压低于 2.7V 时，可以减少模拟开关总的谐波失真，这样的话，模拟开关的性能跟正常供电电压时的全范围测量一样。

16.3.9 函数 HAL_SYSCFG_CM7BootAddConfig

函数原型：

```
void HAL_SYSCFG_CM7BootAddConfig(uint32_t BootRegister, uint32_t BootAddress)
```

函数描述：

用于配置 BOOT = 0 或者 BOOT = 1 时的启动地址，详情可以看第 13 章的 13.4 小节。

16.3.10 IO 补偿相关函数

函数原型：

```
void HAL_EnableCompensationCell(void)
void HAL_DisableCompensationCell(void)
void HAL_SYSCFG_EnableIOSpeedOptimize(void)
void HAL_SYSCFG_DisableIOSpeedOptimize(void)
void HAL_SYSCFG_CompensationCodeSelect(uint32_t SYSCFG_CompCode)
void HAL_SYSCFG_CompensationCodeConfig(uint32_t SYSCFG_PMOSCode, uint32_t SYSCFG_NMOSCode )
```

函数描述：

- 函数 HAL_EnableCompensationCell 和 HAL_DisableCompensationCell
  分别用于使能或者禁止 IO 补偿，只有在供电电压范围是 2.4V 到 3.6V 之间时，使用此功能才有意义。
- 函数 HAL_SYSCFG_EnableIOSpeedOptimize 和 HAL_SYSCFG_DisableIOSpeedOptimize
  分别用于优化 IO 速度或者禁止优化，不过仅在供电电压低于 2.5V 时可用，高于 2.5V 是不可以使用的，另外使用这个功能的前提是用户使能了 PRODUCT_BELOW_25V（是可选字节配置选项里面的一个 bit）。
- 函数 HAL_SYSCFG_CompensationCodeSelect

IO 补偿单元的选择，函数形参可以是 SYSCFG_CELL_CODE，即寄存器 SYSCFG_CCVR，也可以是 SYSCFG_REGISTER_CODE，即寄存器 SYSCFG_CCCR。

- 函数 HAL_SYSCFG_CompensationCodeConfig
  用于设置 GPIO 内部构造中 NMOS 和 PMOS 的补偿值，两个形参的范围都是 0-15。根据用户调用函数 HAL_SYSCFG_CompensationCodeSelect 选择的寄存器,这里仅有一个形参的设置是有效的。

16.3.11 低功耗和 EXTI 相关函数

低功耗和 EXTI 相关的函数暂时不做讲解，后面章节用到时候再说明。

16.4源文件 stm32h7xx_hal_rcc.c

这个文件主要是实现内部和外部时钟（HSE、HSI、LSE、CSI、LSI、HSI48、PLL、CSS、MCO）以及总线时钟（SYSCLK、AHB3、 AHB1、AHB2、AHB4、APB3、APB1L、APB1H、APB2、 APB4）的配置。

学习这个文件注意事项：

- 系统上电复位后，通过内部高速时钟 HSI 运行（主频 64MHz），Flash 工作在 0 等待周期，所有外设除了 SRAM、Flash、JTAG 和 PWR，时钟都是关闭的。这里特别注意 SRAM，官方手册里面说的太笼统，这个 SRAM 到底是指的哪个 SRAM。关于这个问题，详情看此贴：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=87784 。
  - AHB 和 APB 总线无分频，所有挂载这两类总线上的外设都是以 HSI 频率运行。
  - 所有的 GPIO 都是模拟模式，除了 JTAG 相关的几个引脚。
- 系统上电复位后，用户需要完成以下工作：
  - 选择用于驱动系统时钟的时钟源。
  - 配置系统时钟频率和 Flash 设置。
  - 配置分频器。
  - 使能外设时钟。
  - 配置外设时钟源，部分外设的时钟可以不来自系统时钟，此时通过配置寄存器 RCC_D1CCIPR、RCC_D2CCIP1R、RCC_D2CCIP2R 和 RCC_D3CCIPR 实现。

RCC 局限性：

使能了外设时钟后，不能立即操作对应的寄存器，要加延迟。不同外设延迟不同：

- 如果是 AHB 的外设，使能了时钟后，需要等待 2 个 AHB 时钟周期才可以操作这个外设的寄存器。
- 如果是 APB 的外设，使能了时钟后，需要等待 2 个 APB 时钟周期才可以操作这个外设的寄存器。

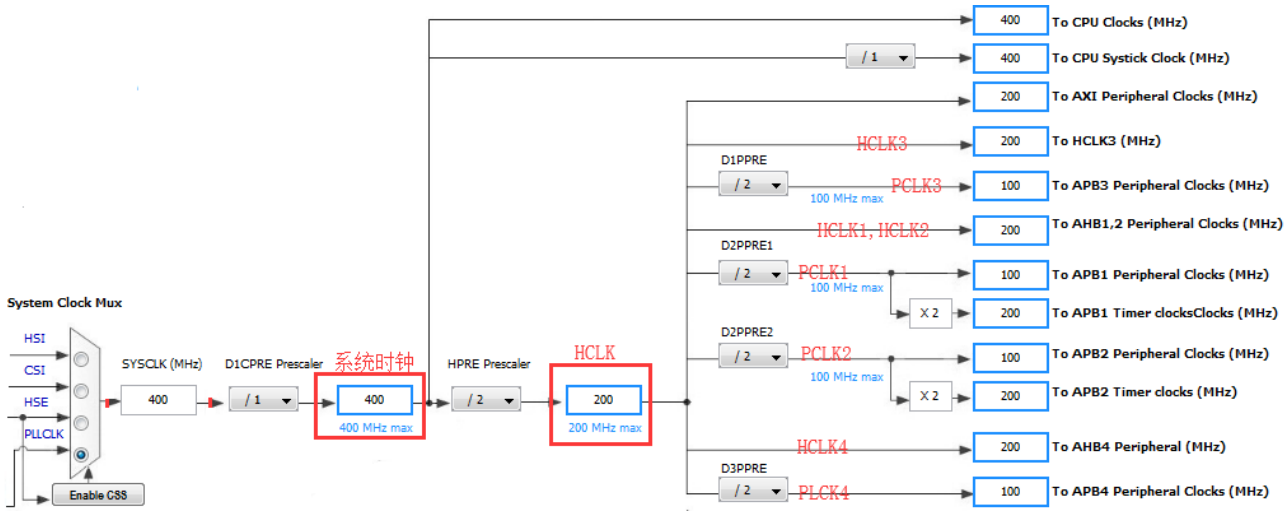
当前 HAL 库的解决方案是在使能了外设时钟后，再搞一个读操作，算是当做延迟用。

比如下面使能 GPIOA 的时钟：

```
#define __HAL_RCC_GPIOA_CLK_ENABLE()   do { \
                                         __IO uint32_t tmpreg; \
                                         SET_BIT(RCC->AHB4ENR, RCC_AHB4ENR_GPIOAEN);\
                                         /* Delay after an RCC peripheral clock enabling */ \
                                         tmpreg = READ_BIT(RCC->AHB4ENR, RCC_AHB4ENR_GPIOAEN);\
                                         UNUSED(tmpreg); \
                                        } while(0)
```

◆ 关于时钟方面的知识点补充

1. 正确理解 HCLK1，2，3，4 以及 PCLK1，2，3，4 对应哪些总线的时钟，下面一张图可以说明问题：



由上图可以得出：

HCLK1，2，3，4 对应的是 AHB 总线 AHB1，AHB2，AHB3 和 AHB4 时钟。

PCLK1、2、3、4 对应的是 APB 总线 APB1，APB2，APB3 和 APB4 时钟。

2. 内部和外部时钟配置：

- HSI (high-speed internal)

  高速内部 RC 振荡器，可以直接或者通过 PLL 倍频后做系统时钟源。缺点是精度差些，即使经过校准。

- CSI (Low-power internal oscillator)

  低功耗内部 RC 振荡器，作用同 HSI，不过主要用于低功耗。

- LSI (low-speed internal)

  低速内部时钟，主要用于独立看门狗和 RTC 的时钟源。

- HSE (high-speed external)

  高速外部晶振，可接 4 - 48MHz 的晶振，可以直接或者通过 PLL 倍频后做系统时钟源，也可以做 RTC 的是时钟源。

- LSE (low-speed external)

低速外部晶振，主要用于 RTC。

- CSS (Clock security system)

  时钟安全系统，一旦使能后，如果 HSE 启动失败（不管是直接作为系统时钟源还是通过 PLL 输出后做系统时钟源），系统时钟将切换到 HSI。如果使能了中断的话，将进入不可屏蔽中断 NMI。

- MCO1 (micro controller clock output)

  可以在 PA8 引脚输出 HSI, LSE, HSE, PLL1(PLL1_Q) 或 HSI48 时钟。

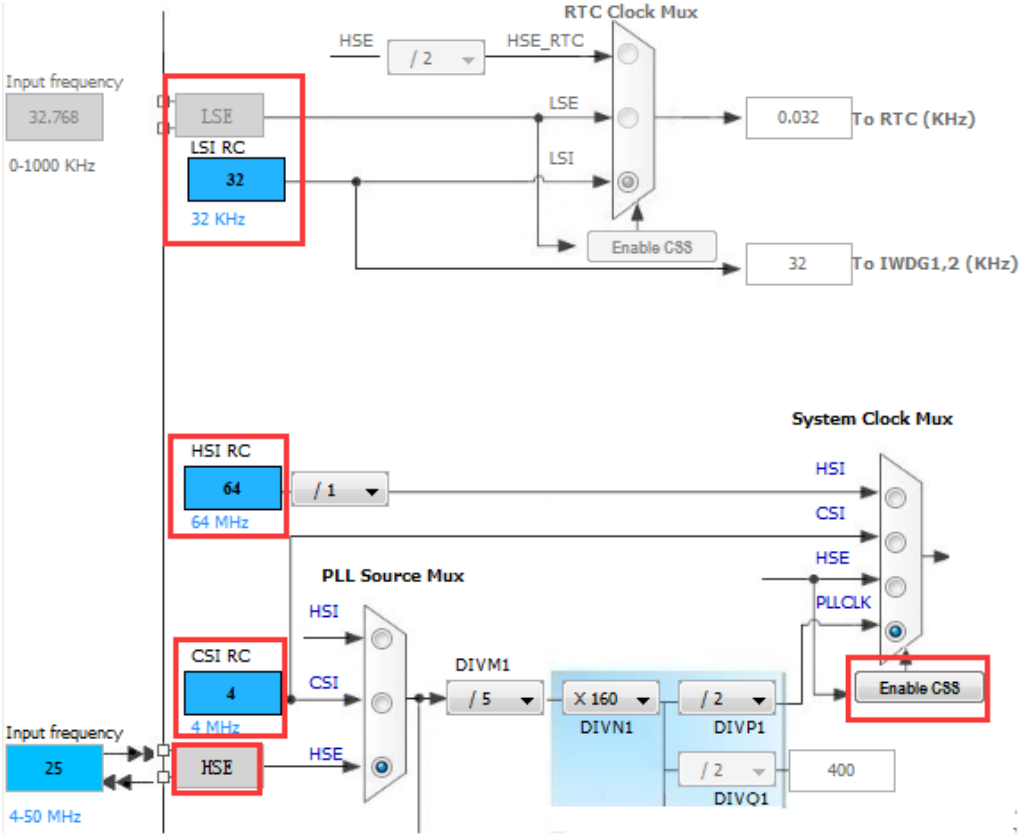
- MCO2 (micro controller clock output)

  可以在 PC9 引脚输出 HSE, PLL2(PLL2_P), SYSCLK, LSI, CSI 或 PLL1(PLL1_P) 时钟。

- PLL 锁相环，时钟输入来自 HSI , HSE 或者 CSI

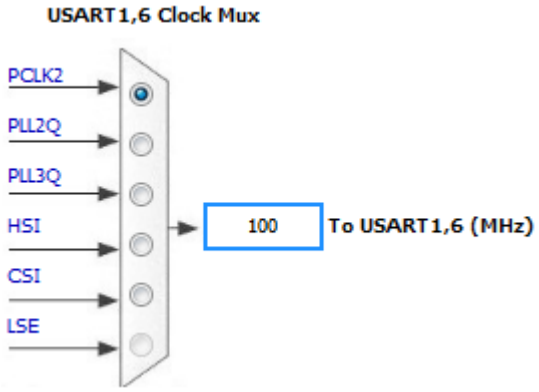
  主锁相环 PLL1，用于给 CPU 和部分外设提供时钟。

  专用锁相环 PLL2 和 PLL3，用于给部分外设提供时钟。



3. System, AHB 和 APB 总线时钟配置：

- 系统总线时钟源可以来自 CSI，HIS，HSE 或 PLL。
- AHB 总线时钟是通过系统时钟分频产生的，用于给 AXI，AHB1，2，3，4 和 APB1，2，3，4 九个总线上的外设提供时钟。另外用户可以通过函数 HAL_RCC_GetSysClockFreq 获取系统时钟。
- 所有外设时钟都可以通过系统时钟获取，并且部分外设还支持多个时钟源，比如下面串口 1,6



16.4.1 函数 HAL_RCC_DeInit

函数原型：

```
void HAL_RCC_DeInit(void)
{
  /* Set HSION bit */
  SET_BIT(RCC->CR, RCC_CR_HSION);

  /* Reset CFGR register */
  CLEAR_REG(RCC->CFGR);

  /* 省略未写 */

  /* Reset HSEBYP bit */
  CLEAR_BIT(RCC->CR, RCC_CR_HSEBYP);

  /* Disable all interrupts */
  CLEAR_REG(RCC->CICR);
}
```

函数描述：

此函数用于 RCC 复位函数，主要实现如下功能：

- HSI 打开作为系统时钟。
- HSE, PLL1, PLL2 和 PLL3 关闭。
- AHB, APB 总线无分频 。
- CSS, MCO1 和 MCO2 关闭。
- 所有中断关闭。

注意事项：

1. 此函数不会修改外设时钟，LSI、LSE 和 RTC 时钟。

使用举例：

此函数的使用比较简单，需要调用的时候直接调用即可。

16.4.2 函数 HAL_RCC_OscConfig

函数原型：

```
272  __weak HAL_StatusTypeDef HAL_RCC_OscConfig(RCC_OscInitTypeDef  *RCC_OscInitStruct)
273  {
274    uint32_t tickstart = 0;
275
276    /* Check the parameters */
277    assert_param(IS_RCC_OSCILLATORTYPE(RCC_OscInitStruct->OscillatorType));
278    /*------------------------------ HSE Configuration ------------------------*/
279    if(((RCC_OscInitStruct->OscillatorType) & RCC_OSCILLATORTYPE_HSE) == RCC_OSCILLATORTYPE_HSE)
280    {
327    /*------------------------------ HSI Configuration ------------------------*/
328    if(((RCC_OscInitStruct->OscillatorType) & RCC_OSCILLATORTYPE_HSI) == RCC_OSCILLATORTYPE_HSI)
329    {
406    /*------------------------------ CSI Configuration ------------------------*/
407    if(((RCC_OscInitStruct->OscillatorType) & RCC_OSCILLATORTYPE_CSI) == RCC_OSCILLATORTYPE_CSI)
408    {
470    /*------------------------------ LSI Configuration ------------------------*/
471    if(((RCC_OscInitStruct->OscillatorType) & RCC_OSCILLATORTYPE_LSI) == RCC_OSCILLATORTYPE_LSI)
472    {
513    /*------------------------------ HSI48 Configuration ----------------------*/
514    if(((RCC_OscInitStruct->OscillatorType) & RCC_OSCILLATORTYPE_HSI48) == RCC_OSCILLATORTYPE_HSI48)
515    {
555    /*------------------------------ LSE Configuration ------------------------*/
556    if(((RCC_OscInitStruct->OscillatorType) & RCC_OSCILLATORTYPE_LSE) == RCC_OSCILLATORTYPE_LSE)
557    {
607    /*------------------------------ PLL Configuration ------------------------*/
608    /* Check the parameters */
609    assert_param(IS_RCC_PLL(RCC_OscInitStruct->PLL.PLLState));
610    if ((RCC_OscInitStruct->PLL.PLLState) != RCC_PLL_NONE)
611    {
707    return HAL_OK;
708  }
```

函数描述：

通过上面函数原型，我们可以一目了然的看出此函数的作用，配置了 HSE、HSI、CSI、LSI、HSI48、LSE 和 PLL。

函数参数：

- 函数的形参是 RCC_OscInitTypeDef 类型结构体变量，这个结构体的定义如下（主要是 HSE、HSI、CSI、LSI、HSI48、LSE 和 PLL 的配置变量）：

```
/**
  * @brief  RCC Internal/External Oscillator (HSE, HSI, CSI, LSE and LSI) configuration structure definition
  */
typedef struct
{
  uint32_t OscillatorType;
  uint32_t HSEState;
  uint32_t LSEState;
  uint32_t HSIState;
  uint32_t HSICalibrationValue;
  uint32_t LSIState;
  uint32_t HSI48State;
  uint32_t CSIState;
  uint32_t CSICalibrationValue;
  RCC_PLLInitTypeDef PLL;
}RCC_OscInitTypeDef;
```

注意事项：

1. LSE Bypass 切换到 LSE On 或者 LSE On 切换到 LSE Bypass 都不支持，用需要先关闭 LSE，然后才可以切换到 LSE Bypass 或者 LSE On。
2. HSE Bypass 切换到 HSE On 或者 LSE On 切换到 LSE Bypass 都不支持，用需要先关闭 HSE，然后才可以切换到 HSE Bypass 或者 HSE On。

使用举例：

```
RCC_OscInitTypeDef RCC_OscInitStruct;
HAL_StatusTypeDef ret = HAL_OK;

/* 使能 HSE，并选择 HSE 作为 PLL 时钟源 */
RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
RCC_OscInitStruct.HSEState = RCC_HSE_ON;
RCC_OscInitStruct.HSIState = RCC_HSI_OFF;
RCC_OscInitStruct.CSIState = RCC_CSI_OFF;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;

RCC_OscInitStruct.PLL.PLLM = 5;
RCC_OscInitStruct.PLL.PLLN = 160;
RCC_OscInitStruct.PLL.PLLP = 2;
RCC_OscInitStruct.PLL.PLLR = 2;
RCC_OscInitStruct.PLL.PLLQ = 4;

RCC_OscInitStruct.PLL.PLLVCOSEL = RCC_PLL1VCOWIDE;
RCC_OscInitStruct.PLL.PLLRGE = RCC_PLL1VCIRANGE_2;
ret = HAL_RCC_OscConfig(&RCC_OscInitStruct);
if(ret != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

16.4.3 函数 HAL_RCC_ClockConfig

函数原型：

```
HAL_StatusTypeDef HAL_RCC_ClockConfig(RCC_ClkInitTypeDef  *RCC_ClkInitStruct, uint32_t FLatency)
{
  {uint32_t tickstart = 0;
  /* To correctly read data from FLASH memory, the number of wait states (LATENCY)
  /* Increasing the CPU frequency */
  if(FLatency > (FLASH->ACR & FLASH_ACR_LATENCY))
  {
   /*------------------------- HCLK Configuration --------------------------*/
    if(((RCC_ClkInitStruct->ClockType) & RCC_CLOCKTYPE_HCLK) == RCC_CLOCKTYPE_HCLK)
    {
    /*------------------------- SYSCLK Configuration -------------------------*/
    if(((RCC_ClkInitStruct->ClockType) & RCC_CLOCKTYPE_SYSCLK) == RCC_CLOCKTYPE_SYSCLK)
    {
  /* Decreasing the number of wait states because of lower CPU frequency */
  if(FLatency < (FLASH->ACR & FLASH_ACR_LATENCY))
  {
  /*------------------------- D1PCLK1 Configuration -------------------------*/
  if(((RCC_ClkInitStruct->ClockType) & RCC_CLOCKTYPE_D1PCLK1) == RCC_CLOCKTYPE_D1PCLK1)
  {
  /*------------------------- PCLK1 Configuration --------------------------*/
  if(((RCC_ClkInitStruct->ClockType) & RCC_CLOCKTYPE_PCLK1) == RCC_CLOCKTYPE_PCLK1)
  {
  /*------------------------- PCLK2 Configuration --------------------------*/
  if(((RCC_ClkInitStruct->ClockType) & RCC_CLOCKTYPE_PCLK2) == RCC_CLOCKTYPE_PCLK2)
  {
  /*------------------------- D3PCLK1 Configuration -------------------------*/
  if(((RCC_ClkInitStruct->ClockType) & RCC_CLOCKTYPE_D3PCLK1) == RCC_CLOCKTYPE_D3PCLK1)
  {
    {  /* Update the SystemCoreClock global variable */
}
```

函数描述：

通过上面函数原型，我们可以一目了然的看出此函数的作用，配置了 CPU、AHB 和 APB 的所有总线时钟，即 HCLK，SYSTICK，D1PCLK1，PCLK1，PCLK2 和 D3PCLK1 时钟。这里的 D1PCLK1 其实就是 PCLK3，而 D3PCLK1 是 PCLK4。

函数参数：

- 第 1 个参数是 RCC_ClkInitTypeDef 类型的结构体变量。
- 第 2 个参数是 Flash 的延迟设置。
- 返回值，返回 HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

注意事项：

1. 此函数会更新全局变量 SystemCoreClock 的主频值，并且会再次调用函数 HAL_InitTick 更新系统滴答时钟，这点要特别注意。
2. 系统上电复位或者从停机、待机模式唤醒后，使用的是 HSI 作为系统时钟。以防使用 HSE 直接或者通过 PLL 输出后做系统时钟时失败（如果使能了 CSS）。
3. 目标时钟就绪后才可以从当前的时钟源往这个目标时钟源切换，如果目标时钟源没有就绪，就会等待直到时钟源就绪才可以切换。
4. 根据设备的供电范围，必须正确设置 D1CPRE[3:0]位的范围，防止超过允许的最大频率。

使用举例：

```
RCC_ClkInitTypeDef RCC_ClkInitStruct;
HAL_StatusTypeDef ret = HAL_OK;
```

```
    /*
       选择 PLL 的输出作为系统时钟
       配置 RCC_CLOCKTYPE_SYSCLK 系统时钟
       配置 RCC_CLOCKTYPE_HCLK 时钟，对应 AHB1，AHB2，AHB3 和 AHB4 总线
       配置 RCC_CLOCKTYPE_PCLK1 时钟，对应 APB1 总线
       配置 RCC_CLOCKTYPE_PCLK2 时钟，对应 APB2 总线
       配置 RCC_CLOCKTYPE_D1PCLK1 时钟，对应 APB3 总线
       配置 RCC_CLOCKTYPE_D3PCLK1 时钟，对应 APB4 总线
    */
    RCC_ClkInitStruct.ClockType = (RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_D1PCLK1 |
                                   RCC_CLOCKTYPE_PCLK1 | \
                                  RCC_CLOCKTYPE_PCLK2  | RCC_CLOCKTYPE_D3PCLK1);

    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.SYSCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB3CLKDivider = RCC_APB3_DIV2;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_APB1_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_APB2_DIV2;
    RCC_ClkInitStruct.APB4CLKDivider = RCC_APB4_DIV2;

    /* 此函数会更新 SystemCoreClock，并重新配置 HAL_InitTick */
    ret = HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4);
    if(ret != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
```

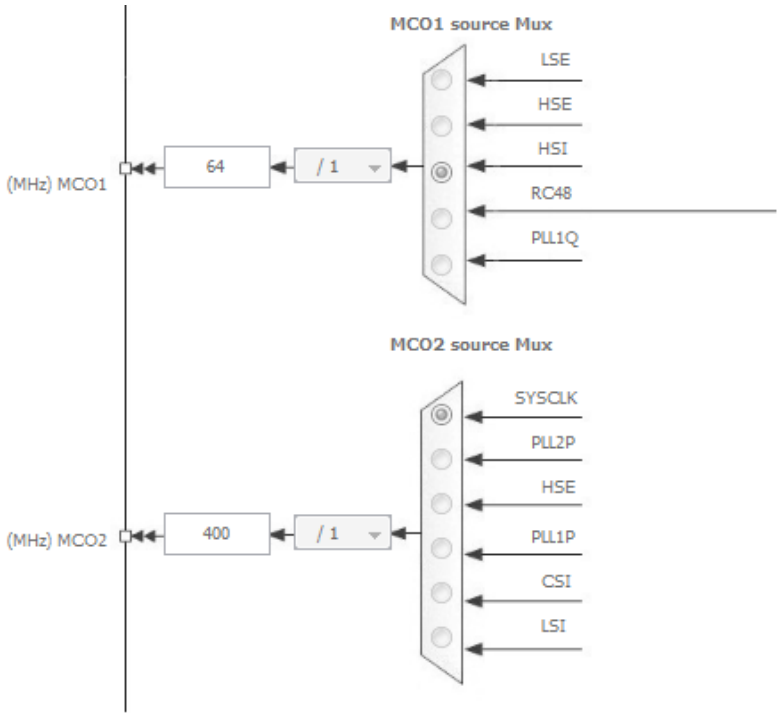
16.4.4函数 HAL_RCC_MCOConfig

函数原型：

```
void HAL_RCC_MCOConfig(uint32_t RCC_MCOx, uint32_t RCC_MCOSource, uint32_t RCC_MCODiv)
{
  GPIO_InitTypeDef GPIO_InitStruct;
  /* Check the parameters */
  assert_param(IS_RCC_MCO(RCC_MCOx));
  assert_param(IS_RCC_MCODIV(RCC_MCODiv));

  /* RCC_MCO1 */
  if(RCC_MCOx == RCC_MCO1)
  {
     /* 省略未写 */
  }
  else
  {
    /* 省略未写 */
  }
}
```

函数描述：

此函数的作用是配置 MCO1（PA8 引脚）和 MCO2（PC9 引脚）的时钟输出以及选择的时钟源，通过下面的截图可以很好的说明此函数的作用：



函数参数：

- 第 1 个形参用于选择输出的引脚，可选择 RCC_MCO1（PA8 引脚）或者 RCC_MCO2（PC9 引脚）。
- 第 2 个形参用于选择输出的时钟源，MCO1 可选择的时钟源如下：
  - RCC_MCO1SOURCE_HSI
  - RCC_MCO1SOURCE_LSE
  - RCC_MCO1SOURCE_HSE
  - RCC_MCO1SOURCE_PLL1QCLK
  - RCC_MCO1SOURCE_HSI48

  MCO1 可选择的时钟源如下：
  - RCC_MCO2SOURCE_SYSCLK
  - RCC_MCO2SOURCE_PLL2PCLK
  - RCC_MCO2SOURCE_HSE
  - RCC_MCO2SOURCE_PLLCLK
  - RCC_MCO2SOURCE_CSICLK
  - RCC_MCO2SOURCE_LSICLK
- 第 3 个参数用于设置输出分频，范围从 RCC_MCODIV_1 到 RCC_MCODIV_15。

使用举例：

此函数的使用比较简单，需要在 MCO 引脚输出时钟的时候，直接调用即可。

16.4.5 RCC 相关函数

函数原型：

```
void HAL_RCC_EnableCSS(void)
uint32_t HAL_RCC_GetSysClockFreq(void)
uint32_t HAL_RCC_GetHCLKFreq(void)
uint32_t HAL_RCC_GetPCLK1Freq(void)
uint32_t HAL_RCC_GetPCLK2Freq(void)
void HAL_RCC_GetOscConfig(RCC_OscInitTypeDef  *RCC_OscInitStruct)
void HAL_RCC_GetClockConfig(RCC_ClkInitTypeDef  *RCC_ClkInitStruct, uint32_t *pFLatency)
```

剩下的这些函数都比较简单了，我们这里就不做讲解了。

16.5 源文件 stm32h7xx_hal_cortex.c

这个库文件主要功能是 NVIC，MPU 和 Systick 的配置。此文件有个臃肿的地方，里面的 API 其实就是将 ARM 的 CMSIS 库各种 API 重新封装了一遍。这么做的好处是保证了 HAL 的 API 都是以字母 HAL 开头。

更多 NVIC 相关知识需要大家看第 21 章节。

更多 SysTick 相关知识需要大家看第 22 章节。

更多 MPU 相关知识需要大家看第 23 章节。

16.5.1 NVIC 相关函数

函数原型：

```
void HAL_NVIC_SetPriorityGrouping(uint32_t PriorityGroup)
void HAL_NVIC_SetPriority(IRQn_Type IRQn, uint32_t PreemptPriority, uint32_t SubPriority)
void HAL_NVIC_EnableIRQ(IRQn_Type IRQn)
void HAL_NVIC_DisableIRQ(IRQn_Type IRQn)
void HAL_NVIC_SystemReset(void)
uint32_t HAL_NVIC_GetPriorityGrouping(void)
void HAL_NVIC_GetPriority(IRQn_Type IRQn, uint32_t PriorityGroup, uint32_t *pPreemptPriority, uint32_t
*pSubPriority)
void HAL_NVIC_SetPendingIRQ(IRQn_Type IRQn)
uint32_t HAL_NVIC_GetPendingIRQ(IRQn_Type IRQn)
void HAL_NVIC_ClearPendingIRQ(IRQn_Type IRQn)
uint32_t HAL_NVIC_GetActive(IRQn_Type IRQn)
```

这些函数的操作就比较容易了，这里就不做讲解了，用到时调用即可。

16.5.2 SysTick 相关函数

函数原型：

```
uint32_t HAL_SYSTICK_Config(uint32_t TicksNumb)
void HAL_SYSTICK_CLKSourceConfig(uint32_t CLKSource)
```

```
void HAL_SYSTICK_IRQHandler(void)
__weak void HAL_SYSTICK_Callback(void)
```

这些函数的操作就比较容易了，这里就不做讲解了，用到时调用即可。

16.5.3 函数 HAL_MPU_ConfigRegion

函数原型：

```
void HAL_MPU_ConfigRegion(MPU_Region_InitTypeDef *MPU_Init)
{
   /* 部分省略未写 */

  /* Set the Region number */
  MPU->RNR = MPU_Init->Number;

  if ((MPU_Init->Enable) != RESET)
  {
    /* 部分省略未写 */

    MPU->RBAR = MPU_Init->BaseAddress;
    MPU->RASR = ((uint32_t)MPU_Init->DisableExec             << MPU_RASR_XN_Pos)   |
                ((uint32_t)MPU_Init->AccessPermission        << MPU_RASR_AP_Pos)   |
                ((uint32_t)MPU_Init->TypeExtField            << MPU_RASR_TEX_Pos)  |
                ((uint32_t)MPU_Init->IsShareable             << MPU_RASR_S_Pos)    |
                ((uint32_t)MPU_Init->IsCacheable             << MPU_RASR_C_Pos)    |
                ((uint32_t)MPU_Init->IsBufferable            << MPU_RASR_B_Pos)    |
                ((uint32_t)MPU_Init->SubRegionDisable        << MPU_RASR_SRD_Pos)  |
                ((uint32_t)MPU_Init->Size                    << MPU_RASR_SIZE_Pos) |
                ((uint32_t)MPU_Init->Enable                  << MPU_RASR_ENABLE_Pos);
  }
  else
  {
    MPU->RBAR = 0x00;
    MPU->RASR = 0x00;
  }
}
```

此函数在本教程第 23 章有专门的讲解。

16.6 总结

本章节就为大家讲解这么多，对于一些常用的函数，望大家务必要掌握。

第17章 STM32H7 之 GPIO 的 HAL 库 API

本章教程为大家讲解 GPIO（General-purpose I/Os）的 API 使用和注意事项。GPIO 是所有外设里面较容易掌握的，但也是用到最多的。

配合第 15 章讲解的各种 IO 模式再学习本章，更容易理解透彻。

17.1 初学者重要提示
17.2 GPIO 涉及到的寄存器
17.3 源文件 stm32h7xx_hal_gpio.c
17.4 如何使用 HAL 库的 GPIO 驱动
17.5 总结

17.1 初学者重要提示

- 如何阅读 HAL 库源码的问题

  HAL 库实现的函数有复杂的，也有简单的，简单的可以直接阅读代码。复杂的代码阅读起来比较耗时间，如果再配合参考手册抠每个寄存器的配置，那就更消耗时间了。所以对于这种函数，用户仅需了解每个部分实行的功能即可，而且 HAL 库都做了关键注释，以说明这部分实现的功能。所以用户没有必要去抠每个配置是如何实现的，仅需知道实现了什么功能。以后工程项目有需要了解具体配置时，再看即可。
- 学习本章节前，务必保证已经学习了第 15 章。

17.2 GPIO 涉及到的寄存器

GPIO 外设涉及到的寄存器比较少，也容易理解，推荐大家阅读 GPIO 源码的时候将参考手册中对应的寄存器功能做一个了解。

很多时候，我们会直接调用 GPIO 的寄存器进行配置，而不使用 HAL 进行调用，以提高执行效率，特别是中断里面执行时。

17.3 源文件 stm32h7xx_hal_gpio.c

这个文件主要是实现 GPIO 的引脚配置，学习这个文件注意事项：

- 系统上电后，引脚默认状态是模拟模式。

- 所有的引脚有弱上拉和弱下拉电阻，阻值范围 30-50KΩ。其中配置为模拟模式时，上拉和下拉被硬件禁止，其它的输入、输出和复用都可以配置上拉和下拉。
- 在输出或者复用模式，每个引脚可以配置成推挽或者开漏，且有 GPIO 速度等级可配置。另外注意，不同的供电范围，实际速度等级是有些区别的。
- 每个 GPIO 都可以配置成外部中断/事件模式，但要特别注意，引脚要配置成输入模式，在芯片的内部有个多路选择器，选择引脚与 16 个外部中断/事件 EXTI0 - EXTI15 中的那个导通。这就决定了，每个外部中断/事件只能与一个引脚导通，如果用户配置了多个引脚 PA0，PB0，PC0 等，那么只有一个能够与 EXTI0 导通。

17.3.1 函数 HAL_GPIO_Init

函数原型：

```
void HAL_GPIO_Init(GPIO_TypeDef  *GPIOx, GPIO_InitTypeDef *GPIO_Init)
{
  /* 部分省略未写 */

  /* 配置 GPIO 引脚，这些采用 16 个引脚的循环检测模式 */
  for(position = 0; position < GPIO_NUMBER; position++)
  {
    /* 部分省略未写 */
    if(iocurrent == ioposition)
    {
      /*--------------------- GPIO 模式配置 -----------------------*/

      /*--------------------- EXTI 模式配置 -----------------------*/

    }
  }
}
```

函数描述：

此函数用于初始化 GPIO，此函数主要实现如下功能：

- GPIO 功能配置。
- 设置 EXTI 功能。

函数参数：

- 第 1 个参数用于填写使用的端口号，可以是：

  #define GPIOA ((GPIO_TypeDef *) GPIOA_BASE)
  #define GPIOB ((GPIO_TypeDef *) GPIOB_BASE)
  #define GPIOC ((GPIO_TypeDef *) GPIOC_BASE)
  #define GPIOD ((GPIO_TypeDef *) GPIOD_BASE)
  #define GPIOE ((GPIO_TypeDef *) GPIOE_BASE)
  #define GPIOF ((GPIO_TypeDef *) GPIOF_BASE)

```
#define GPIOG      ((GPIO_TypeDef *) GPIOG_BASE)
#define GPIOH      ((GPIO_TypeDef *) GPIOH_BASE)
#define GPIOI      ((GPIO_TypeDef *) GPIOI_BASE)
#define GPIOJ      ((GPIO_TypeDef *) GPIOJ_BASE)
#define GPIOK      ((GPIO_TypeDef *) GPIOK_BASE)
```

◆ 第 2 个形参是 GPIO_InitTypeDef 类型的结构体变量，这个变量比较重要，要熟练掌握，定义如下：

```
typedef struct
{
  uint32_t Pin;
  uint32_t Mode;
  uint32_t Pull;
  uint32_t Speed;
  uint32_t Alternate;
}GPIO_InitTypeDef;
```

下面将结构体每个成员做个说明：

- 成员 Pin 用于配置选择的引脚，范围 GPIO_PIN_0 到 GPIO_PIN_15，额外还可以选择 GPIO_PIN_All 和 GPIO_PIN_MASK。
- 成员 Mode 可以选择：

```
GPIO_MODE_INPUT                 /* 输入模式  */
GPIO_MODE_OUTPUT_PP             /* 推挽输出  */
GPIO_MODE_OUTPUT_OD             /* 开漏输出  */
GPIO_MODE_AF_PP                 /* 复用推挽  */
GPIO_MODE_AF_OD                 /* 复用开漏  */

GPIO_MODE_ANALOG                /* 模拟模式  */
GPIO_MODE_IT_RISING             /* 外部中断，上升沿触发检测 */
GPIO_MODE_IT_FALLING            /* 外部中断，下降沿触发检测 */
GPIO_MODE_IT_RISING_FALLING     /* 外部中断，双沿触发检测   */

GPIO_MODE_EVT_RISING            /* 外部事件模式，上升沿触发检测  */
GPIO_MODE_EVT_FALLING           /* 外部事件模式，下降沿触发检测  */
GPIO_MODE_EVT_RISING_FALLING    /* 外部事件模式，双沿触发检测 */
```

- 成员 Pull 用于配置上拉下拉电阻：

```
GPIO_NOPULL          /* 无上拉和下拉电阻 */
```

GPIO_PULLUP /* 带上拉电阻 */
GPIO_PULLDOWN /* 带下拉电阻 */

- 成员 Speed 用于配置 GPIO 速度等级，有下面四种可选：
  GPIO_SPEED_FREQ_LOW /* 低速 */
  GPIO_SPEED_FREQ_MEDIUM /* 中等速度 */
  GPIO_SPEED_FREQ_HIGH /* 快速 */
  GPIO_SPEED_FREQ_VERY_HIGH /* 高速 */
- 成员 Alternate 用于配置引脚复用，可选择的复用方式在文件 stm32h7xx_hal_gpio_ex.h 里面进行了定义，比如串口复用：
  GPIO_AF7_USART1
  GPIO_AF7_USART2
  GPIO_AF7_USART3
  GPIO_AF7_USART6
  GPIO_AF7_UART7

注意事项：

1. 与 F1，F4 系列的标准库不同，H7 的 HAL 库已经没有单独的 EXTI 外部中断设置文件，是将其整合到此函数里面了。
2. 函数 HAL_GPIO_Init 对引脚的初始化是把同组 16 个引脚 for 循环检测了一遍，效率稍低。所以不推荐下面这种初始化：

```
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
GPIO_InitStruct.Pin = GPIO_PIN_0;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);   //这里会执行 16 次 for 查询

GPIO_InitStruct.Pin = GPIO_PIN_1;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);   //这里会执行 16 次 for 查询

GPIO_InitStruct.Pin = GPIO_PIN_2;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);  //这里会执行 16 次 for 查询
```

如果是程序运行期间的引脚状态切换，最好采用下面的方式或者直接寄存器操作：

```
GPIO_InitStruct.Pin = GPIO_PIN_0 |GPIO_PIN_1 | GPIO_PIN_2 ;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;

HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);  //这里会执行 16 次 for 查询
```

使用举例：

```
GPIO_InitTypeDef  GPIO_InitStruct;

GPIO_InitStruct.Pin = GPIO_PIN_0;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;      /* 推挽输出 */
GPIO_InitStruct.Pull = GPIO_NOPULL;                /* 无上拉和下拉电阻 */
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH; /* GPIO 速度等级最高 */
```

```
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
```

17.3.2 函数 HAL_GPIO_DeInit

函数原型：

```
void HAL_GPIO_DeInit(GPIO_TypeDef  *GPIOx, uint32_t GPIO_Pin)
{
  for(position = 0; position < GPIO_NUMBER; position++)
  {
     /* 部分省略未写 */
    if(iocurrent == ioposition)
    {
      /*------------------------ GPIO Mode Configuration --------------------*/
      /* 配置为模拟模式 */
      GPIOx->MODER |= (GPIO_MODER_MODER0 << (position * 2));

      /* 配置复用模式为 AF0，即作为通用 IO */
      GPIOx->AFR[position >> 3] &= ~((uint32_t)0xF << ((uint32_t)(position & (uint32_t)0x07) * 4)) ;

      /* 配置到最低速度 */
      GPIOx->OSPEEDR &= ~(GPIO_OSPEEDER_OSPEEDR0 << (position * 2));

      /* 输出类型是推挽，如果 IO 模式被设置为模拟，此选项对其没有影响 */
      GPIOx->OTYPER  &= ~(GPIO_OTYPER_OT_0 << position) ;

      /* 无上拉和下拉电阻 */
      GPIOx->PUPDR &= ~(GPIO_PUPDR_PUPDR0 << (position * 2));

      /*------------------------ EXTI 模式配置 --------------------*/

    }
  }
}
```

函数描述：

此函数用于复位 IO 到初始化状态，具体状态看函数原型中的注释即可。

函数参数：

- 第 1 个参数用于填写使用的端口号，可以是：

  #define GPIOA ((GPIO_TypeDef *) GPIOA_BASE)
  #define GPIOB ((GPIO_TypeDef *) GPIOB_BASE)
  #define GPIOC ((GPIO_TypeDef *) GPIOC_BASE)
  #define GPIOD ((GPIO_TypeDef *) GPIOD_BASE)
  #define GPIOE ((GPIO_TypeDef *) GPIOE_BASE)
  #define GPIOF ((GPIO_TypeDef *) GPIOF_BASE)
  #define GPIOG ((GPIO_TypeDef *) GPIOG_BASE)
  #define GPIOH ((GPIO_TypeDef *) GPIOH_BASE)

```
#define GPIOI          ((GPIO_TypeDef *) GPIOI_BASE)
#define GPIOJ          ((GPIO_TypeDef *) GPIOJ_BASE)
#define GPIOK          ((GPIO_TypeDef *) GPIOK_BASE)
```

- 第 2 个参数是配置选择的引脚，范围 GPIO_PIN_0 到 GPIO_PIN_15。

使用举例：

此函数的使用比较简单，需要调用的时候直接调用即可。

17.3.3 函数 HAL_GPIO_ReadPin

函数原型：

```
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
{
  GPIO_PinState bitstatus;

  /* Check the parameters */
  assert_param(IS_GPIO_PIN(GPIO_Pin));

  if((GPIOx->IDR & GPIO_Pin) != (uint32_t)GPIO_PIN_RESET)
  {
    bitstatus = GPIO_PIN_SET;
  }
  else
  {
    bitstatus = GPIO_PIN_RESET;
  }
  return bitstatus;
}
```

函数描述：

此函数用于读取引脚状态，通过 GPIO 的 IDR 寄存器读取。

函数参数：

- 第 1 个参数用于填写使用的端口号，从 GPIOA 到 GPIOK。
- 第 2 个参数是配置选择的引脚，范围 GPIO_PIN_0 到 GPIO_PIN_15。

使用举例：

此函数的使用比较简单，需要调用的时候直接调用即可。

17.3.4 函数 HAL_GPIO_WritePin

函数原型：

```
void HAL_GPIO_WritePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)
{
  /* Check the parameters */
  assert_param(IS_GPIO_PIN(GPIO_Pin));
  assert_param(IS_GPIO_PIN_ACTION(PinState));
```

```
  if(PinState != GPIO_PIN_RESET)
  {
    GPIOx->BSRRL = GPIO_Pin;
  }
  else
  {
    GPIOx->BSRRH = GPIO_Pin ;
  }
}
```

函数描述：

此函数用于设置引脚输出高电平或者低电平。使用 GPIO 的 BSRR 寄存器进行设置，使用这个寄存器的好处是支持原子操作，由硬件支持的。原子操作的含义是操作过程不会被中断打断，而我们使用 GPIO 中另一个设置输出的寄存 ODR 是会被中断打断的。大家看下寄存器赋值操作对应的反汇编，是由多条汇编指令组成的。

函数参数：

- 第 1 个参数用于填写使用的端口号，从 GPIOA 到 GPIAK。
- 第 2 个参数是配置选择的引脚，范围 GPIO_PIN_0 到 GPIO_PIN_15。
- 第 3 个参数用于设置引脚输出高电平还是低电平，GPIO_PIN_RESET 表示低电平，GPIO_PIN_SET 表示高电平。

使用举例：

此函数的使用比较简单，需要调用的时候直接调用即可。

17.3.5 函数 HAL_GPIO_TogglePin

函数原型：

```
void HAL_GPIO_TogglePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
{
  /* Check the parameters */
  assert_param(IS_GPIO_PIN(GPIO_Pin));

  GPIOx->ODR ^= GPIO_Pin;
}
```

函数描述：

此函数用于设置引脚的电平翻转，使用 GPIO 的 ODR 寄存器进行设置。

函数参数：

- 第 1 个参数用于填写使用的端口号，从 GPIOA 到 GPIAK。
- 第 2 个参数是配置选择的引脚，范围 GPIO_PIN_0 到 GPIO_PIN_15。

使用举例：

此函数的使用比较简单，需要调用的时候直接调用即可。

17.3.6函数 HAL_GPIO_LockPin

函数原型：

```
HAL_StatusTypeDef HAL_GPIO_LockPin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
{
  __IO uint32_t tmp = GPIO_LCKR_LCKK;

  assert_param(IS_GPIO_LOCK_INSTANCE(GPIOx));
  assert_param(IS_GPIO_PIN(GPIO_Pin));

  /* 应用 IO 锁的写入顺序 */
  tmp |= GPIO_Pin;
  /* 设置 LCKx bit(s): LCKK='1' + LCK[15-0] */
  GPIOx->LCKR = tmp;
  /* 复位 LCKx bit(s): LCKK='0' + LCK[15-0] */
  GPIOx->LCKR = GPIO_Pin;
  /* 设置 LCKx bit(s): LCKK='1' + LCK[15-0] */
  GPIOx->LCKR = tmp;
  /* 复位 LCKK bit*/
  tmp = GPIOx->LCKR;

 if((GPIOx->LCKR & GPIO_LCKR_LCKK) != RESET)
  {
    return HAL_OK;
  }
  else
  {
    return HAL_ERROR;
  }
}
```

函数描述：

此函数用于锁住 GPIO 引脚所涉及到的寄存器，这些寄存器包括 GPIOx_MODER，GPIOx_OTYPER，GPIOx_OSPEEDR，GPIOx_PUPDR，GPIOx_AFRL 和 GPIOx_AFRH。

函数参数：

- 第 1 个参数用于填写使用的端口号，从 GPIOA 到 GPIAK。
- 第 2 个参数是配置选择的引脚，范围 GPIO_PIN_0 到 GPIO_PIN_15。

注意事项：

1. 此函数是锁住用户设置的引脚所对应的寄存器某些位，并不是把整个寄存器都锁住了。
2. 一旦锁住后，就不能再修改，只有复位后才可以重新配置。

使用举例：

此函数的使用比较简单，需要调用的时候直接调用即可。

17.4如何使用 HAL 库的 GPIO 驱动

使用方法由 HAL 库提供（本章 17.3.1 小节提供的例子就是这种方式）：

- 第 1 步：使能 GPIO 所在总线的 AHB 时钟，__HAL_RCC_GPIOx_CLK_ENABLE()。
- 第 2 步：通过函数 HAL_GPIO_Init()配置 GPIO。
  (1) 通过结构体 GPIO_InitTypeDef 的成员 Mode 配置输入、输出、模拟等模式。
  (2) 通过结构体 GPIO_InitTypeDef 的成员 Pull 配置上拉、下拉电阻。
  (3) 通过结构体 GPIO_InitTypeDef 的成员 Speed 配置 GPIO 速度等级。
  (4) 如果选择了复用模式，那么就需要配置结构体 GPIO_InitTypeDef 的成员 Alternate。
  (5) 如果引脚功能用于 ADC、DAC 的话，需要配置引脚为模拟模式。
  (6) 如果是用于外部中断/事件，结构体 GPIO_InitTypeDef 的成员 Mode 可以配置相应模式，相应的上升沿、下降沿或者双沿触发也可以选择。
- 第 3 步：如果配置了外部中断/事件，可以通过函数 HAL_NVIC_SetPriority 设置优先级，然后调用函数 HAL_NVIC_EnableIRQ 使能此中断。
- 第 4 步：输入模式读取引脚状态可以使用函数 HAL_GPIO_ReadPin。
- 第 5 步：输出模式设置引脚状态可以调用函数 HAL_GPIO_WritePin()和 HAL_GPIO_TogglePin。

另外注意下面三个问题：

- 系统上电复位后，GPIO 默认是模拟模式，除了 JTAG 相关引脚。
- 关闭 LSE 的话，用到的两个引脚 OSC32_IN 和 OSC32_OUT（分别是 PC14，PC15）可以用在通用 IO，如果开启了，就不能再做 GPIO。
- 关闭 HSE 的话，用到的两个引脚 OSC_IN 和 OSC_OUT（分别是 PH0，PH1）可以用在通用 IO，如果开启了，就不能再做 GPIO。

17.5 总结

本章节就为大家讲解这么多，建议大家将 GPIO 的驱动源码结合参考手册中的寄存器通读一遍，对于我们后面章节的学习大有裨益。

第18章 STM32H7 的 GPIO 应用之跑马灯

本章教程为大家介绍 STM32H7 的 GPIO 应用之跑马灯，跑马灯作为经典的测试例程，可以让大家对 STM32H7 应用有个简单的整体认识。

18.1 初学者重要提示
18.2 跑马灯硬件设计
18.3 跑马灯软件驱动设计
18.4 跑马灯板级支持包（bsp_led.c）
18.5 实验例程设计框架
18.6 跑马灯驱动移植和使用
18.7 实验例程说明（MDK）
18.8 实验例程说明（IAR）
18.9 总结

18.1 初学者重要提示

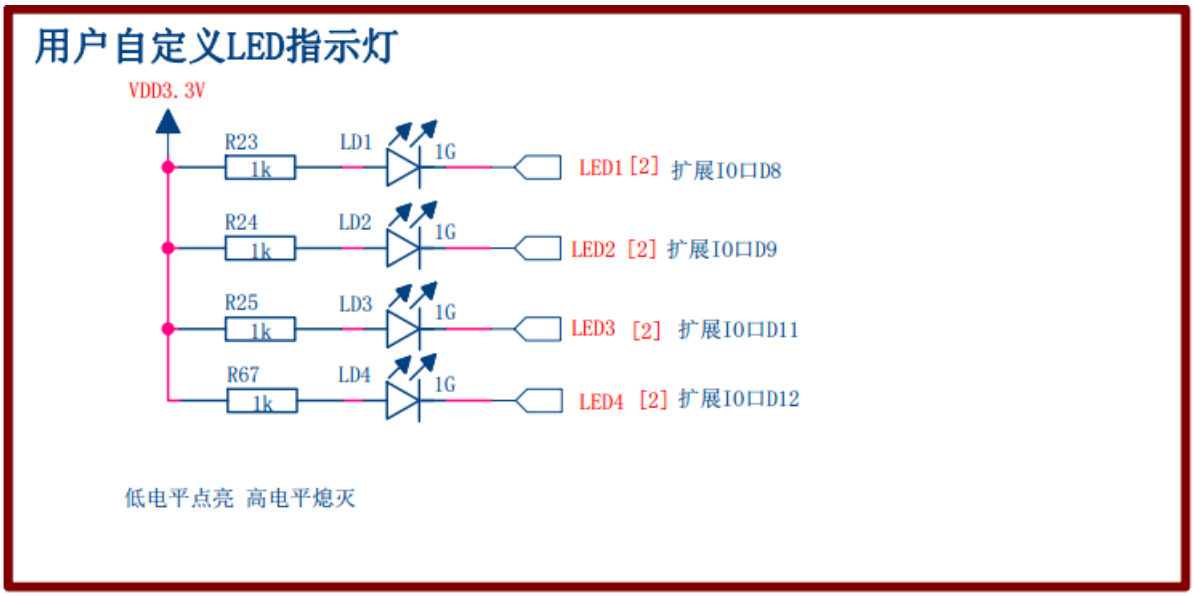
- 学习本章节前，务必保证已经学习了第 15，16 和 17 章。
- 虽然是跑马灯的初级例程，但有必要掌握程序的基本设计框架，后面的例子都是建立在这个框架的基础上。
- LED 不是用 CPU 的 IO 直接驱动，而是由 74HC574 驱动的，74HC574 是一个 8 路并口缓冲器，挂在 FMC 总线上，实现 IO 扩展。也许初学者会问为什么要做 IO 扩展，不是已经用了 240 脚的 STM32H743XIH6 吗？因为开发板使用了 32 位 SDRAM 和 RGB888 硬件接口，消耗 IO 巨大，所以必须得扩展了。
- 对于初学者来说，仅需掌握 LED 驱动的实现方法和对应的 API 调用即可，需要深入的理解 IO 扩展部分，会在后面的第 48 章节进行详细讲解。
- FMC 总线扩展 32 路高速 IO 理解成 GPIO 的 ODR 寄存器就很简单了，其实就是一个东西。
  FMC 扩展 IO 是对地址 0x60001000 的 32bit 数据空间的 0 和 1 的操作。GPIOA 的 ODR 寄存器是对地址 0x40000000 + 0x18020000 + 0x14 空间的操作。但只能操作 16 个引脚。
  使用总线的优势就在这里了，相当于在 GPIOA 到 GPIOK 的基础上，又扩展出 GPIOL 和 GPIOM。

```
#define PERIPH_BASE           ((uint32_t)0x40000000)
#define D3_AHB1PERIPH_BASE    (PERIPH_BASE + 0x18020000)
#define GPIOA_BASE            (D3_AHB1PERIPH_BASE + 0x0000)
#define GPIOA                 ((GPIO_TypeDef *) GPIOA_BASE)
```

```
typedef struct
{
  __IO uint32_t MODER;    /*!< GPIO port mode register,               Address offset: 0x00      */
  __IO uint32_t OTYPER;   /*!< GPIO port output type register,        Address offset: 0x04      */
  __IO uint32_t OSPEEDR;  /*!< GPIO port output speed register,       Address offset: 0x08      */
  __IO uint32_t PUPDR;    /*!< GPIO port pull-up/pull-down register,  Address offset: 0x0C      */
  __IO uint32_t IDR;      /*!< GPIO port input data register,         Address offset: 0x10      */
  __IO uint32_t ODR;      /*!< GPIO port output data register,        Address offset: 0x14      */
  __IO uint16_t BSRRL;    /*!< GPIO port bit set/reset low register,  Address offset: 0x18      */
  __IO uint16_t BSRRH;    /*!< GPIO port bit set/reset high register, Address offset: 0x1A      */
  __IO uint32_t LCKR;     /*!< GPIO port configuration lock register, Address offset: 0x1C      */
  __IO uint32_t AFR[2];   /*!< GPIO alternate function registers,     Address offset: 0x20-0x24 */
} GPIO_TypeDef;
```

18.2 跑马灯硬件设计

跑马灯的硬件设计如下：



通过这个硬件设计，有如下四点需要学习：

18.2.1 灌电流驱动方式

关于拉电流、灌电流和相关的电气特性，在第 15 章的 15.4 小节做了专门的讲解。对于 STM32H7 来说，使用拉电流和灌电流驱动 LED 都是可以的，因为拉电流和灌电流时，STM32H7 总的拉电流和灌电流都是不可超过 140mA，单个引脚最大不可超过 20mA。

开发板这里是采用的灌电流方式。

18.2.2 LED 的压降和驱动电流

这种采用的是灌电流方式，而流经 LED 的电流大小是多少呢？ 这里需要先补充一个基础的知识点。

直插超亮发光二极管压降，主要有三种颜色，然而三种发光二极管的压降都不相同，具体压降参考值如下：

- 红色发光二极管的压降为 2.0V-2.2V。
- 黄色发光二极管的压降为 1.8V-2.0V。
- 绿色发光二极管的压降为 3.0V-3.2V。
- 正常发光时的额定电流约为 20mA。

贴片 LED 压降：

- 红色的压降为 1.82-1.88V，电流 5-8mA。
- 绿色的压降为 1.75-1.82V，电流 3-5mA。
- 橙色的压降为 1.7-1.8V，电流 3-5mA。
- 蓝色的压降为 3.1-3.3V，电流 8-10mA。
- 白色的压降为 3-3.2V，电流 10-15mA。

实际测试开发板红色贴片 LED 的压降的确是 1.8V 左右，那么流过 LED 的电流就是

$$(3.3 - 1.8) / 1K = 1.4mA$$

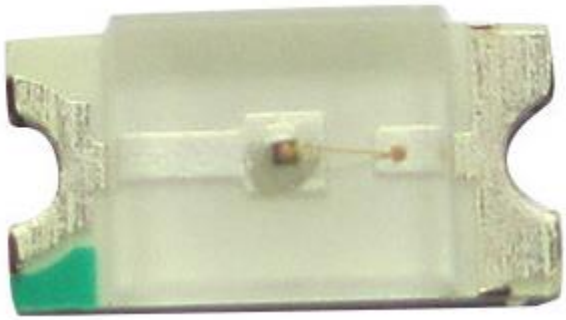
在不考虑二极管本身电阻的情况下，流过 LED 的电流就是 1.4mA。

18.2.3总线扩展

在教程第 48 章节详细讲解了这个问题，对于初学者来说，可以先不用看，等后面学习了 FMC 总线后再去看，就容易掌握多了。

18.2.4贴片 LED 的正负极区分

仔细查看开发板版上面所使用的贴片 LED，会发现一端有绿点，有绿点的这端是负极，而另一端就是正级了。



18.3 跑马灯软件驱动设计

跑马灯的软件驱动实现比较简单，主要是 IO 初始化，LED 亮，LED 灭，LED 翻转。对应的驱动文件也是实现了这几个功能，没有特别的技巧，所以大家看源代码也比较省事。

18.4 跑马灯板级支持包（bsp_led.c）

LED 驱动文件 bsp_led.c 主要实现了如下几个 API：

- bsp_InitLed
- bsp_LedOn
- bsp_LedOff
- bsp_LedToggle
- bsp_IsLedOn

下面将这几个 API 逐一进行说明。

18.4.1 函数 bsp_InitLed

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitLed
*    功能说明: 配置 LED 指示灯相关的 GPIO,  该函数被 bsp_Init() 调用。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitLed(void)
{
    bsp_LedOff(1);
    bsp_LedOff(2);
    bsp_LedOff(3);
    bsp_LedOff(4);
}
```

函数描述：

此函数主要用于 LED 初始化。由于将 GPIO 设置为输出时，GPIO 输出寄存器的值缺省是 0，因此会驱动 LED 点亮，因此在改变 GPIO 为输出前，先关闭 LED 指示灯。

注意事项：

1. 大家会有疑惑，为什么这里没有初始化 GPIO。这是因为 V7 开发板是由 74HC574 驱动的，不是用 CPU 的 IO 直接驱动，74HC574 是一个 8 路并口缓冲器，挂在 FMC 总线上，实现 IO 扩展。
2. 通过 FMC 总线扩展出的 IO 来驱动，不是 GPIO 直接驱动。

3. 调用此函数前，要优先调用函数 bsp_InitExtIO()，此函数用于初始化 FMC 扩展接口，关于这方面的知识在 48 章节专门做了讲解

使用举例：

调用此函数前，务必优先调用函数 bsp_InitExtIO()。这里底层驱动初始化一般都是在 bsp.c 文件的函数 bsp_Init 里面调用。

18.4.2 函数 bsp_LedOn

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_LedOn
*    功能说明: 点亮指定的 LED 指示灯。
*    形    参:  _no : 指示灯序号，范围 1 - 4
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_LedOn(uint8_t _no)
{
    if (_no == 1)
    {
        HC574_SetPin(LED1, 0);
    }
    else if (_no == 2)
    {
        HC574_SetPin(LED2, 0);
    }
    else if (_no == 3)
    {
        HC574_SetPin(LED3, 0);
    }
    else if (_no == 4)
    {
        HC574_SetPin(LED4, 0);
    }
}
```

函数描述：

此函数主要用于点亮 LED。

函数参数：

◆ 第 1 个参数用于指定点亮那个 LED，范围 1-4。

使用举例：

此函数的使用比较简单，需要调用的时候直接调用即可。另外使用前记得先调用函数 bsp_InitExtIO()和 bsp_InitLed。

18.4.3函数 bsp_LedOff

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_LedOff
*    功能说明: 熄灭指定的 LED 指示灯。
*    形    参:  _no 指示灯序号，范围 1 - 4
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_LedOff(uint8_t _no)
{
    if (_no == 1)
    {
        HC574_SetPin(LED1, 1);
    }
    else if (_no == 2)
    {
        HC574_SetPin(LED2, 1);
    }
    else if (_no == 3)
    {
        HC574_SetPin(LED3, 1);
    }
    else if (_no == 4)
    {
        HC574_SetPin(LED4, 1);
    }
}
```

函数描述：

此函数主要用于熄灭 LED。

函数参数：

- 第 1 个参数用于指定熄灭那个 LED，范围 1-4。

使用举例：

此函数的使用比较简单，需要调用的时候直接调用即可。另外使用前记得先调用函数 bsp_InitExtIO()和 bsp_InitLed。

18.4.4函数 bsp_LedToggle

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_LedToggle
*    功能说明: 翻转指定的 LED 指示灯。
*    形    参:  _no 指示灯序号，范围 1 - 4
*    返 回 值: 按键代码
*********************************************************************************************************
```

```
*/
void bsp_LedToggle(uint8_t _no)
{
    uint32_t pin;

    if (_no == 1)
    {
        pin = LED1;
    }
    else if (_no == 2)
    {
        pin = LED2;
    }
    else if (_no == 3)
    {
        pin = LED3;
    }
    else if (_no == 4)
    {
        pin = LED4;
    }
    else
    {
        return;
    }

    if (HC574_GetPin(pin))
    {
        HC574_SetPin(pin, 0);
    }
    else
    {
        HC574_SetPin(pin, 1);
    }
}
```

函数描述：

此函数主要用于翻转 LED。

函数参数：

◆ 第 1 个参数用于指定翻转那个 LED，范围 1-4。

使用举例：

此函数的使用比较简单，需要调用的时候直接调用即可。另外使用前记得先调用函数 bsp_InitExtIO()和 bsp_InitLed。

18.4.5 函数 bsp_IsLedOn

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_IsLedOn
*    功能说明: 判断 LED 指示灯是否已经点亮。
```

```
*    形    参:  _no 指示灯序号，范围 1 - 4
*    返 回 值: 1 表示已经点亮，0 表示未点亮
*********************************************************************************************************
*/
uint8_t bsp_IsLedOn(uint8_t _no)
{
    uint32_t pin;

    if (_no == 1)
    {
        pin = LED1;
    }
    else if (_no == 2)
    {
        pin = LED2;
    }
    else if (_no == 3)
    {
        pin = LED3;
    }
    else if (_no == 4)
    {
        pin = LED4;
    }
    else
    {
        return 0;
    }

    if (HC574_GetPin(pin))
    {
        return 0;/* 灭 */
    }
    else
    {
        return 1;/* 亮 */
    }
}
```

函数描述:

此函数主要用于获取 LED 亮灭状态。

函数参数:

◆ 第 1 个参数用于指定获取那个 LED 的亮灭状态，范围 1-4。

使用举例:

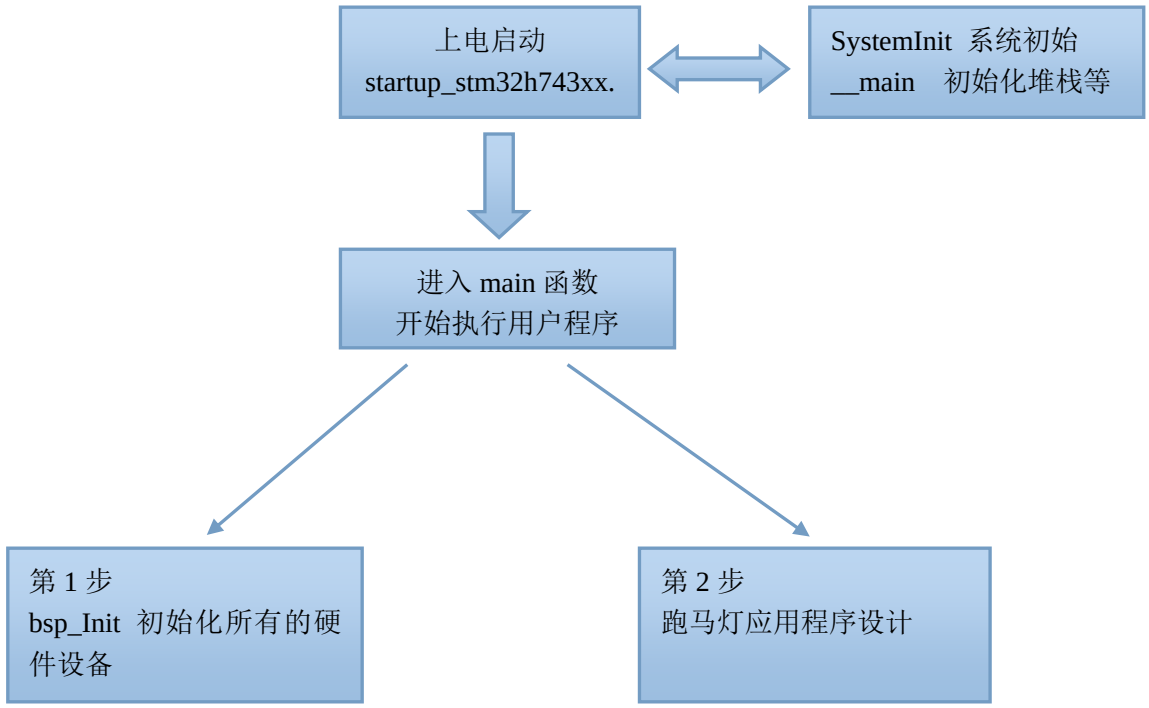
此函数的使用比较简单，需要调用的时候直接调用即可。另外使用前记得先调用函数 bsp_InitExtIO()和 bsp_InitLed。

18.5 跑马灯驱动移植和使用

跑马灯控制是基于 FMC 扩展 IO 实现的，所以跑马灯的移植需要看第 48 章的移植方式。

18.6 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器和 LED。
  - 第 2 部分，应用程序设计部分，实现了一个简易的跑马灯效果。

18.7 实验例程说明（MDK）

配套例子：

V7-001_跑马灯

实验目的：

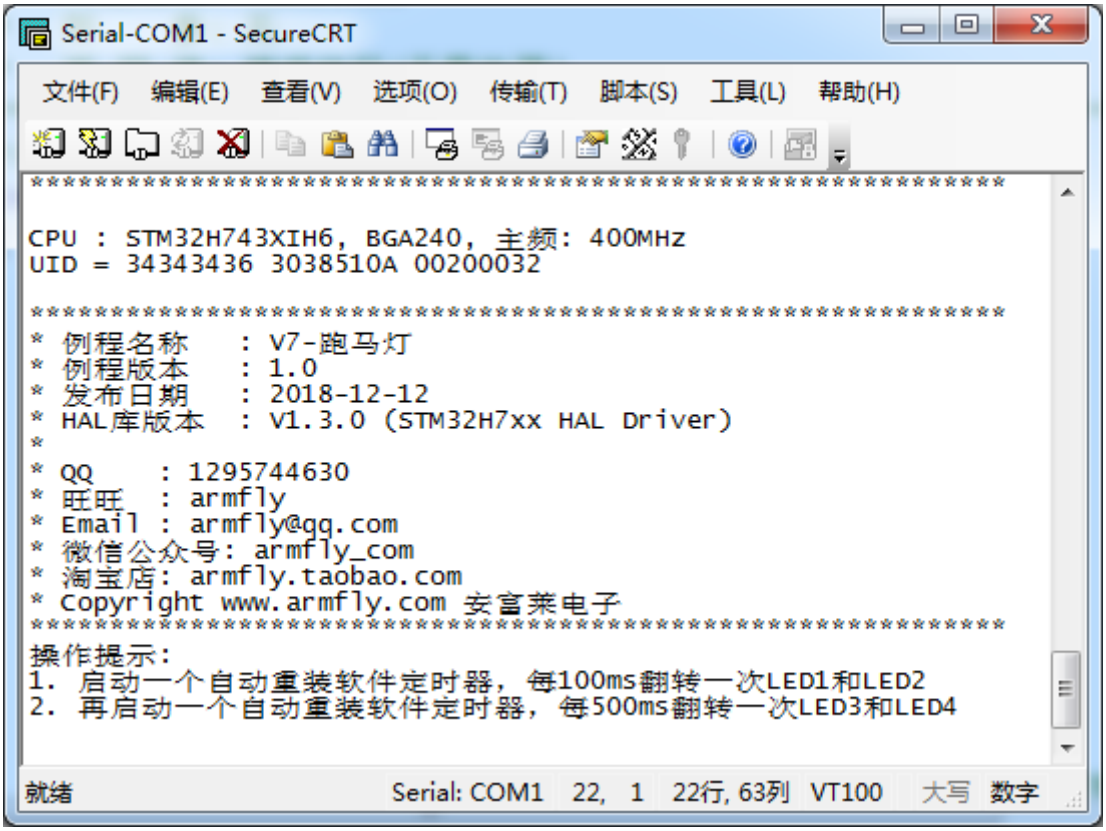
1. 学习 H7 平台的跑马灯实现。

实验内容：

1. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED1 和 LED2。
2. 再启动一个自动重装软件定时器，每 500ms 翻转一次 LED3 和 LED4。

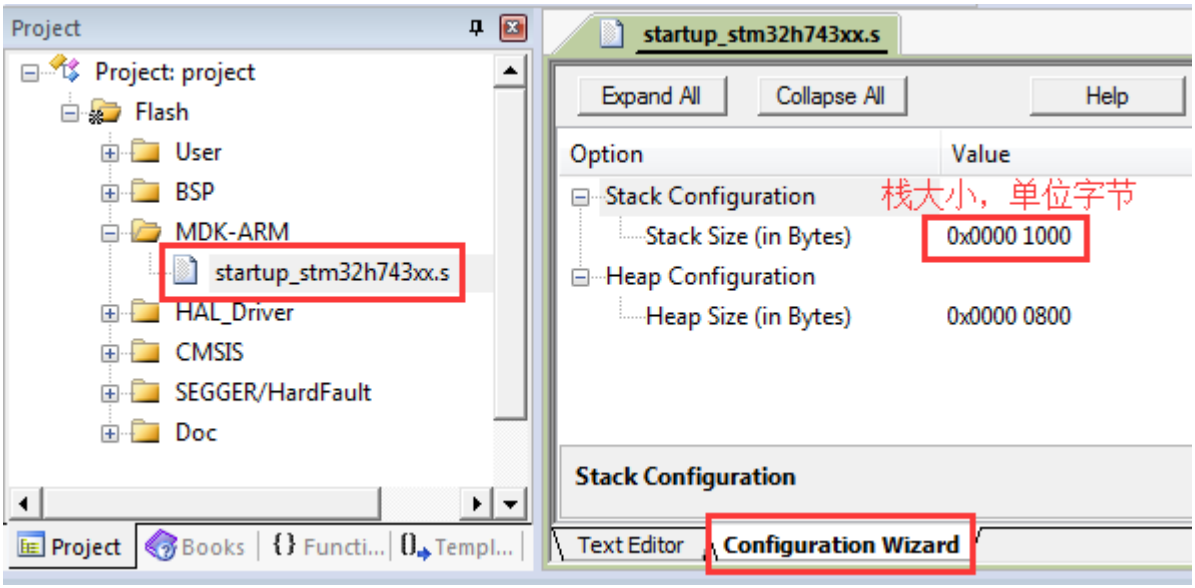
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

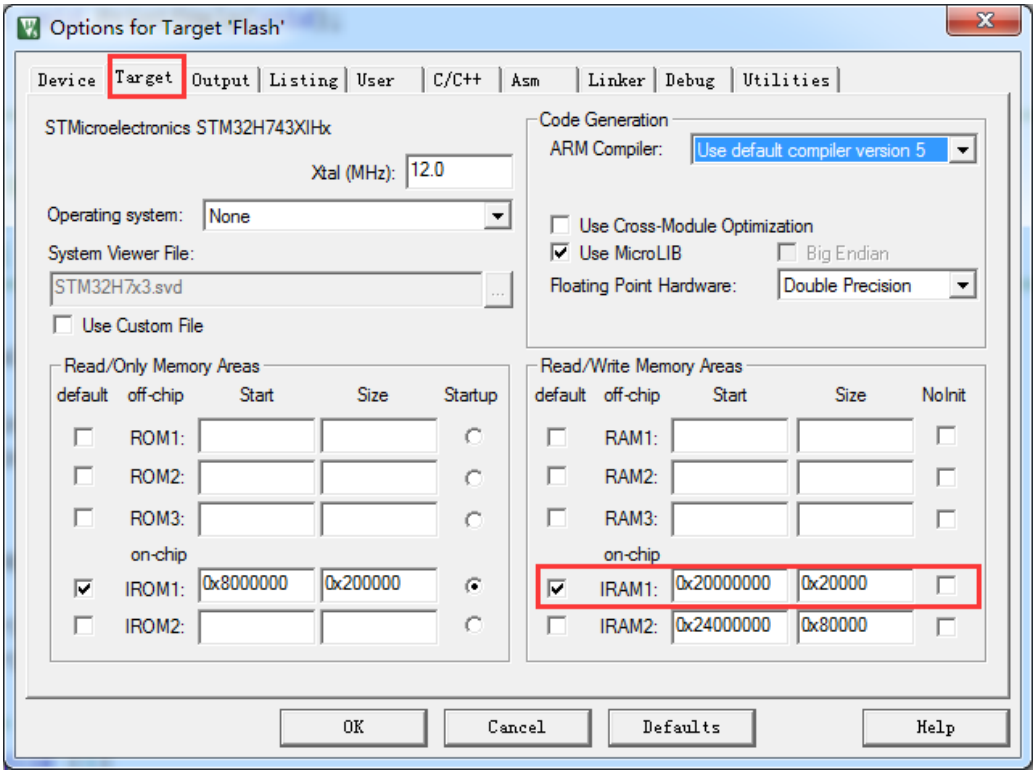


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
```

```
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **主功能：**

主功能的实现主要分为两部分：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED1 和 LED2。
- 再启动一个自动重装软件定时器，每 500ms 翻转一次 LED3 和 LED4。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{

    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    /* 先做个 LED1 的亮灭显示 */
    bsp_LedOn(1);
    bsp_DelayMS(100);
    bsp_LedOff(1);
```

```
    bsp_DelayMS(100);

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */
    bsp_StartAutoTimer(1, 500); /* 启动 1 个 500ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(1);
        }

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(1))
        {
            /* 每隔 500ms 进来一次 */
            bsp_LedToggle(2);
            bsp_LedToggle(3);
            bsp_LedToggle(4);
        }
    }
}
```

18.8 实验例程说明（IAR）

配套例子：

V7-001_跑马灯

实验目的：

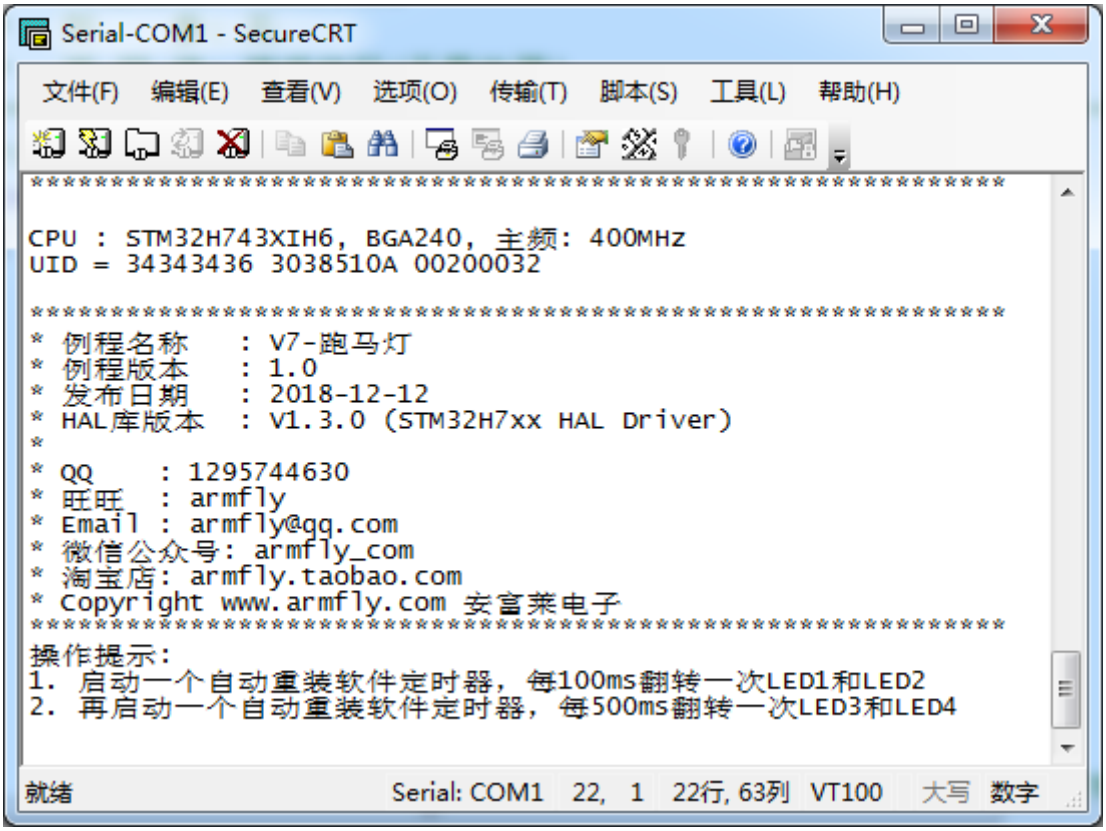
1. 学习 H7 平台的跑马灯实现。

实验内容：

1. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED1 和 LED2。
2. 再启动一个自动重装软件定时器，每 500ms 翻转一次 LED3 和 LED4。

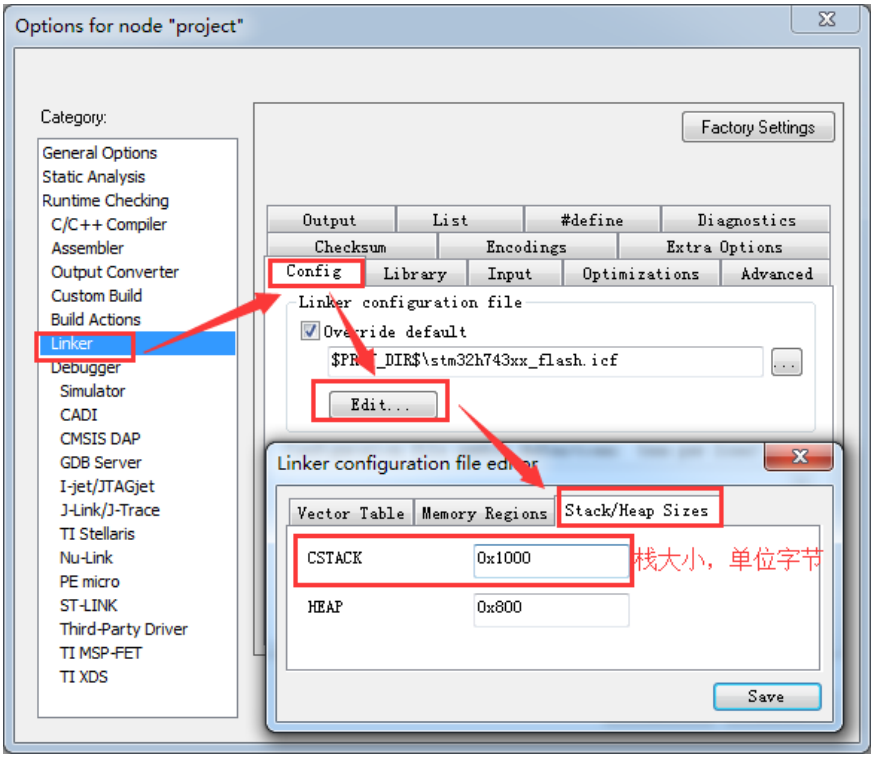
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

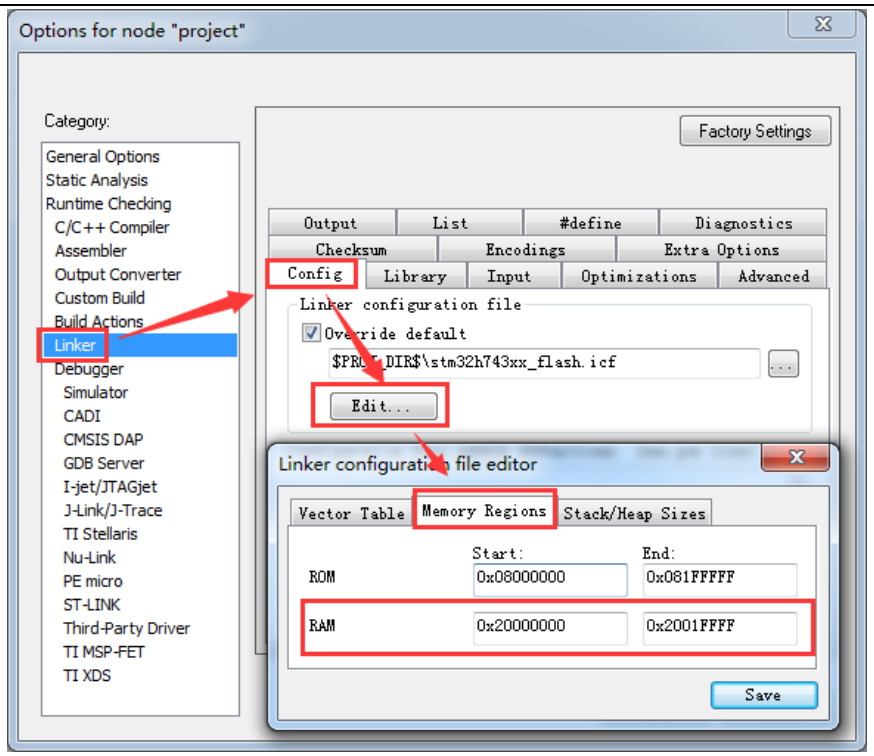


程序设计：

◆ **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
```

```
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主功能的实现主要分为两部分：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED1 和 LED2。
- 再启动一个自动重装软件定时器，每 500ms 翻转一次 LED3 和 LED4。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{

    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    /* 先做个 LED1 的亮灭显示 */
    bsp_LedOn(1);
    bsp_DelayMS(100);
    bsp_LedOff(1);
    bsp_DelayMS(100);
```

```
    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */
    bsp_StartAutoTimer(1, 500); /* 启动 1 个 500ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(1);
        }

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(1))
        {
            /* 每隔 500ms 进来一次 */
            bsp_LedToggle(2);
            bsp_LedToggle(3);
            bsp_LedToggle(4);
        }
    }
}
```

18.9 总结

虽然是跑马灯的初级例程，但有必要掌握程序的基本设计框架，后面的例子都是建立在这个框架的基础上。建议初学者掌握好。

第19章 STM32H7 的 GPIO 应用之按键 FIFO

本章教程为大家介绍 STM32H7 的 GPIO 应用之按键 FIFO，这个方案已经在实际项目中千锤百炼，比较实用。

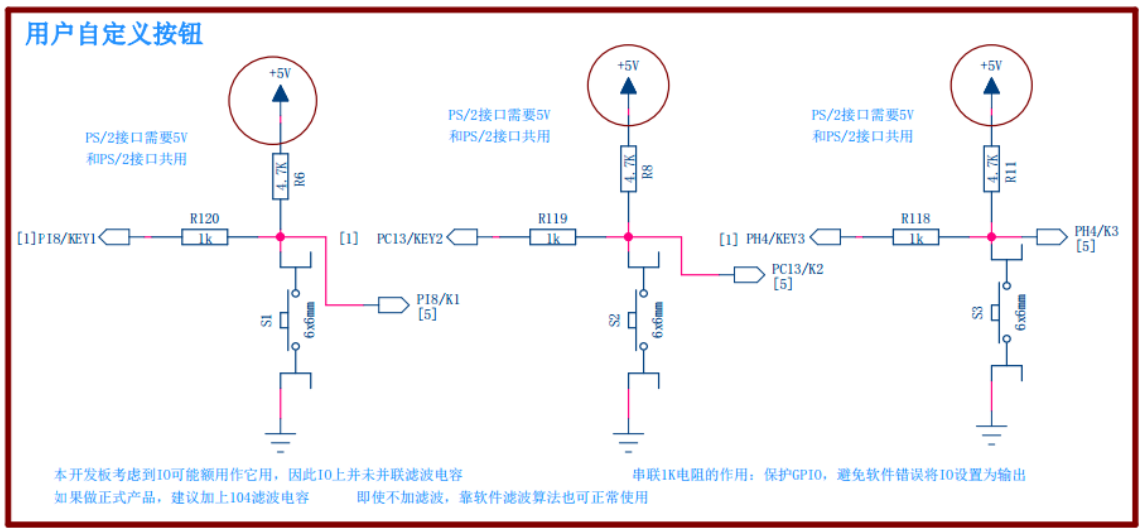
19.1 初学者重要提示
19.2 按键硬件检测
19.3 按键 FIFO 的驱动设计
19.4 按键板级支持包（bsp_key.c）
19.5 按键 FIFO 驱动移植和使用
19.6 实验例程设计框架
19.7 实验例程说明（MDK）
19.8 实验例程说明（IAR）
19.9 总计

19.1 初学者重要提示

- 学习本章节前，务必保证已经学习了第 15，16 和 17 章。
- 按键 FIFO 驱动扩展和移植更简单，组合键也更好用。支持按下、弹起、长按和组合键。

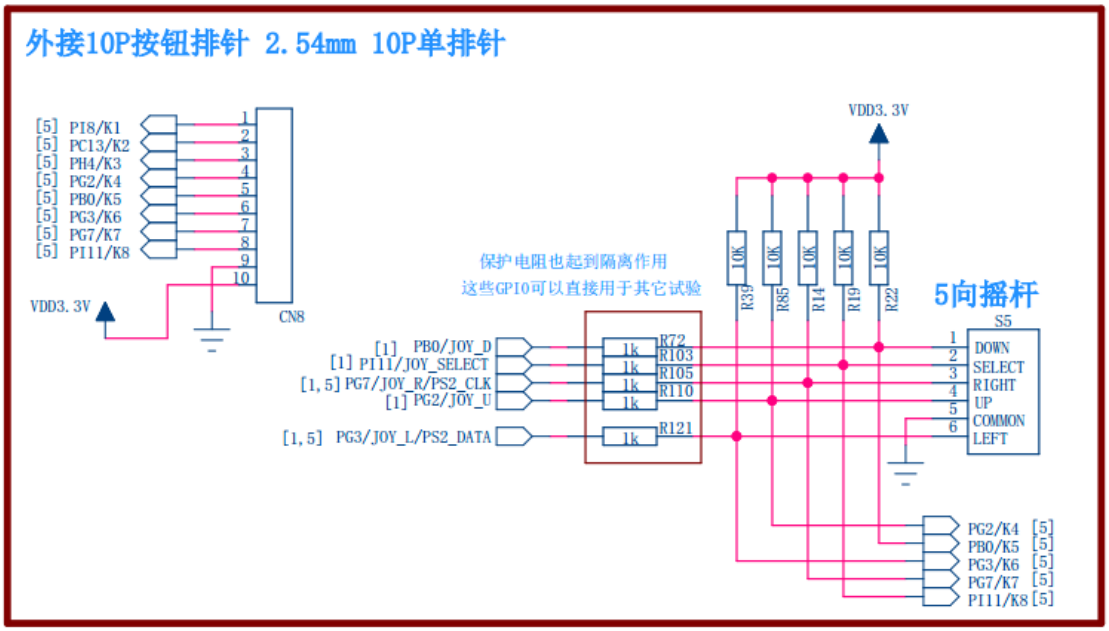
19.2 按键硬件设计

V7 开发板有三个独立按键和一个五向摇杆，下面是三个独立按键的原理图：



注意，K1（S1）、K2(S2)和 K3(S3)按键的上拉电阻是接在 5V 电压上，因为这三个按键被复用为 PS/2 键盘鼠标接口，而 PS/2 是需要 5V 供电的（注，V5 和 V6 开发板做了 PS/2 复用，而 V7 没有使用，这里只是为了兼容之前的板子）。实际测试，K1、K2、K3 按键和 PS/2 键盘是可以同时工作的。

下面是五向摇杆的原理图：



通过这个硬件设计，有如下两个知识点为大家做介绍：

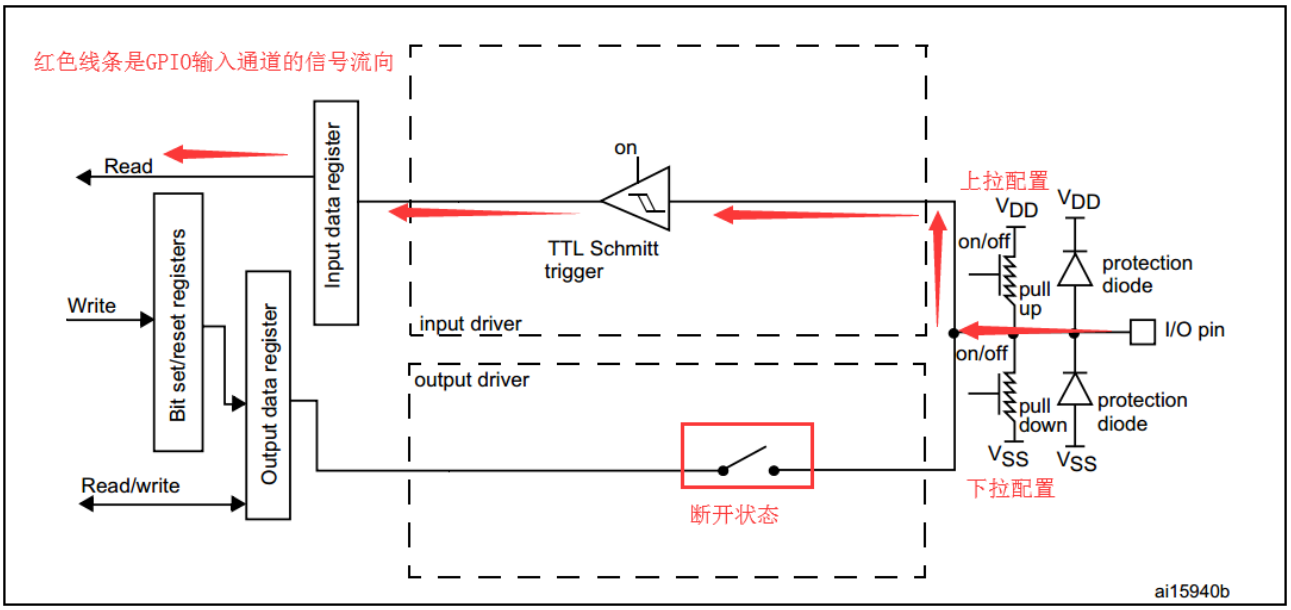
19.2.1 硬件设计

按键和 CPU 之间串联的电阻起保护作用。按键肯定是存在机械抖动的，开发板上面的硬件没有做硬件滤波处理，即使设计了硬件滤波电路，软件上还是需要进行滤波。

- 保护 GPIO，避免软件错误将 IO 设置为输出，如果设置为低电平还好，如果设置输出的是高电平，按键按下会直接跟 GND(低电平)连接，从而损坏 MCU。
- 保护电阻也起到按键隔离作用，这些 GPIO 可以直接用于其它实验。

19.2.2 GPIO 内部结构分析按键

详细的 GPIO 模式介绍，请参考第 15 章的 15.3 小节，本章仅介绍输入模式。下面我们通过一张图来简单介绍 GPIO 的结构。



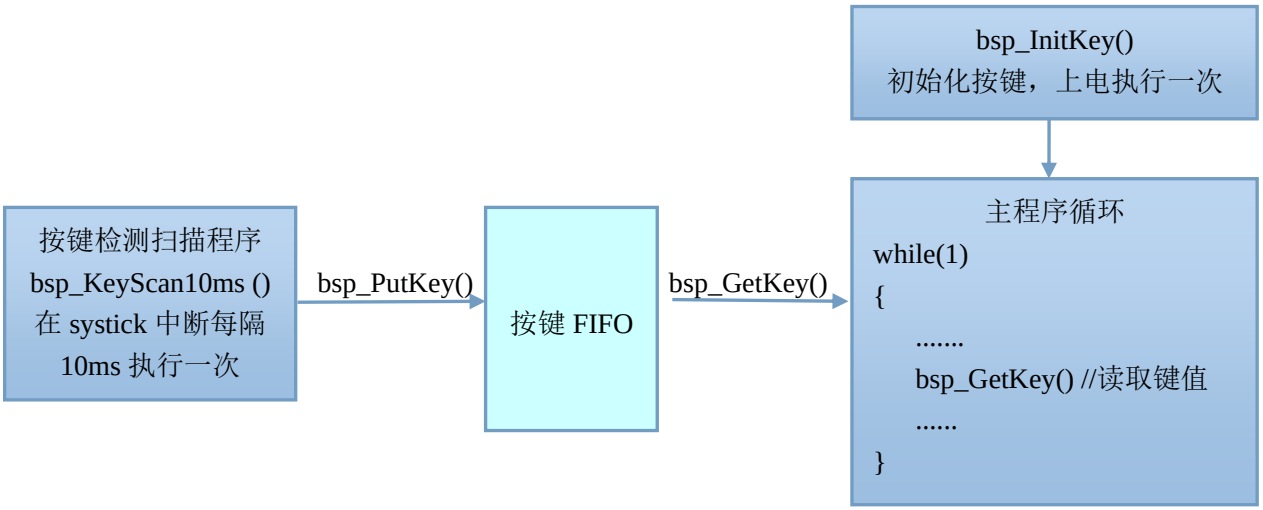
红色的线条是 GPIO 输入通道的信号流向，作为按键检测 IO，这些需要配置为浮空输入。按键已经做了 5V 上拉，因此 GPIO 内部的上下拉电阻都选择关闭状态。

19.3 按键 FIFO 的驱动设计

bsp_key 按键驱动程序用于扫描独立按键，具有软件滤波机制，采用 FIFO 机制保存键值。可以检测如下事件：

- 按键按下。
- 按键弹起。
- 长按键。
- 长按时自动连发。

我们将按键驱动分为两个部分来介绍，一部分是 FIFO 的实现，一部分是按键检测的实现。



bsp_key.c 文件包含按键检测和按键 FIFO 的实现代码。

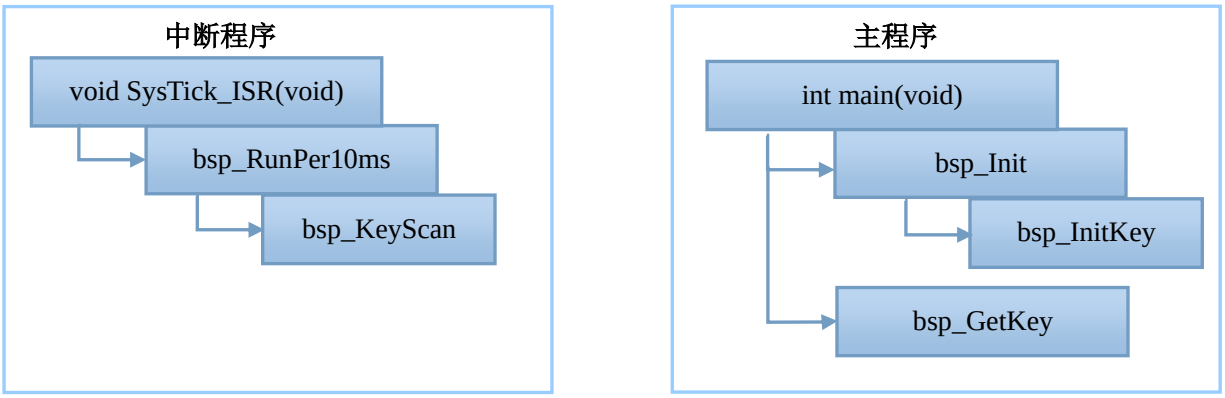
bsp.c 文件会调用 bsp_InitKey()初始化函数。

bsp.c 文件会调用 bsp_KeyScan 按键扫描函数。

bsp_timer.c 中的 Systick 中断服务程序调用 bsp_RunPer10ms。

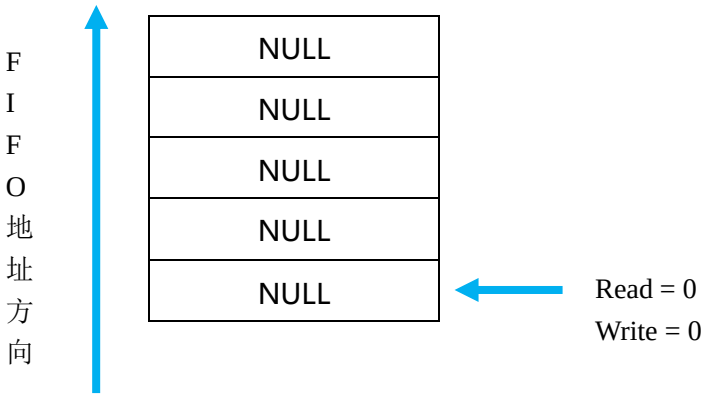
中断程序和主程序通过 FIFO 接口函数进行信息传递。

函数调用关系图：

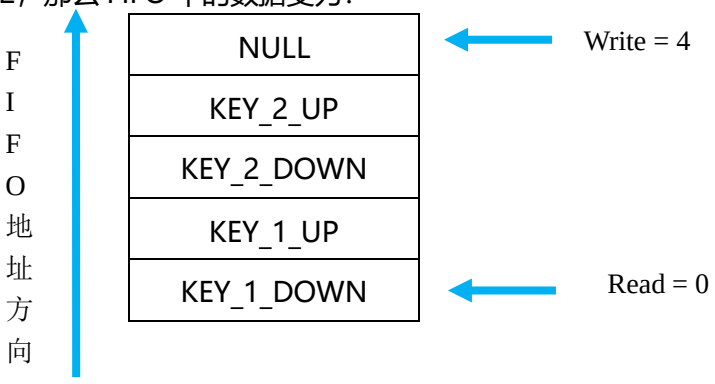


19.3.1 按键 FIFO 的原理

FIFO 是 First Input First Output 的缩写，先入先出队列。我们这里以 5 个字节的 FIFO 空间进行说明。Write 变量表示写位置，Read 变量表示读位置。初始状态时，Read = Write = 0。

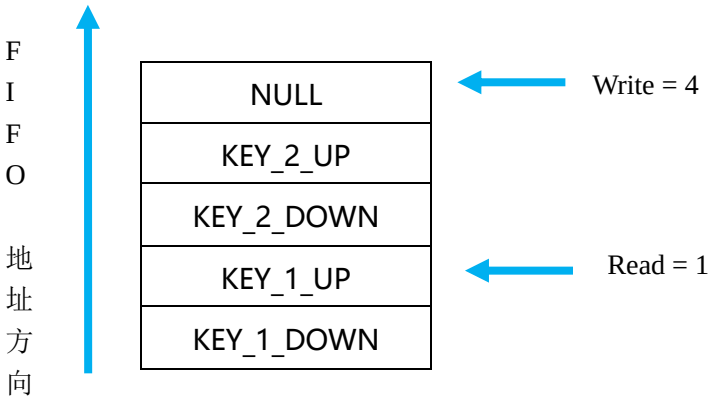


我们依次按下按键 K1，K2，那么 FIFO 中的数据变为：



如果 Write！= Read，则我们认为有新的按键事件。

我们通过函数 bsp_GetKey 读取一个按键值进行处理后，Read 变量变为 1。Write 变量不变。



我们继续通过函数 bsp_GetKey 读取 3 个按键值进行处理后，Read 变量变为 4。此时 Read = Write = 4。两个变量已经相等，表示已经没有新的按键事件需要处理。

有一点要特别的注意，如果 FIFO 空间写满了，Write 会被重新赋值为 0，也就是重新从第一个字节空间填数据进去，如果这个地址空间的数据还没有被及时读取出来，那么会被后来的数据覆盖掉，这点要引起大家的注意。我们的驱动程序开辟了 10 个字节的 FIFO 缓冲区，对于一般的应用足够了。

设计按键 FIFO 主要有三个方面的好处：

- 可靠地记录每一个按键事件，避免遗漏按键事件。特别是需要实现按键的按下、长按、自动连发、弹起等事件时。
- 读取按键的函数可以设计为非阻塞的，不需要等待按键抖动滤波处理完毕。
- 按键 FIFO 程序在嘀嗒定时器中定期的执行检测，不需要在主程序中一直做检测，这样可以有效地降低系统资源消耗。

19.3.2 按键 FIFO 的实现

在 bsp_key.h 中定了结构体类型 KEY_FIFO_T。这只是类型声明，并没有分配变量空间。

```
#define KEY_FIFO_SIZE   10
typedef struct
{
    uint8_t Buf[KEY_FIFO_SIZE];      /* 键值缓冲区 */
    uint8_t Read;                    /* 缓冲区读指针 1 */
    uint8_t Write;                   /* 缓冲区写指针 */
    uint8_t Read2;                   /* 缓冲区读指针 2 */
}KEY_FIFO_T;
```

在 bsp_key.c 中定义 s_tKey 结构变量，此时编译器会分配一组变量空间。

```
static KEY_FIFO_T s_tKey;        /* 按键 FIFO 变量,结构体 */
```

一般情况下，只需要一个写指针 Write 和一个读指针 Read。在某些情况下，可能有两个任务都需要访问按键缓冲区，为了避免键值被其中一个任务取空，我们添加了第 2 个读指针 Read2。出厂程序在

bsp_Idle()函数中实现的按 K1K2 组合键截屏的功能就使用的第 2 个读指针。

当检测到按键事件发生后，可以调用 bsp_PutKey 函数将键值压入 FIFO。下面的代码是函数的实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_PutKey
*    功能说明: 将 1 个键值压入按键 FIFO 缓冲区。可用于模拟一个按键。
*    形    参: _KeyCode : 按键代码
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_PutKey(uint8_t _KeyCode)
{
    s_tKey.Buf[s_tKey.Write] = _KeyCode;

    if (++s_tKey.Write  >= KEY_FIFO_SIZE)
    {
        s_tKey.Write = 0;
    }
}
```

这个 bsp_PutKey 函数除了被按键检测函数调用外，还可以被其他底层驱动调用。比如红外遥控器的按键检测，也共用了同一个按键 FIFO。遥控器的按键代码和主板实体按键的键值统一编码，保持键值唯一即可实现两套按键同时控制程序的功能。

应用程序读取 FIFO 中的键值，是通过 bsp_GetKey 函数和 bsp_GetKey2 函数实现的。我们来看下这两个函数的实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_GetKey
*    功能说明: 从按键 FIFO 缓冲区读取一个键值。
*    形    参:  无
*    返 回 值: 按键代码
*********************************************************************************************************
*/
uint8_t bsp_GetKey(void)
{
    uint8_t ret;

    if (s_tKey.Read == s_tKey.Write)
    {
        return KEY_NONE;
    }
    else
    {
        ret = s_tKey.Buf[s_tKey.Read];

        if (++s_tKey.Read >= KEY_FIFO_SIZE)
        {
            s_tKey.Read = 0;
        }
        return ret;
    }
}

/*
```

```
*********************************************************************************************************
*    函 数 名: bsp_GetKey2
*    功能说明: 从按键 FIFO 缓冲区读取一个键值。独立的读指针。
*    形    参:  无
*    返 回 值: 按键代码
*********************************************************************************************************
*/
uint8_t bsp_GetKey2(void)
{
    uint8_t ret;

    if (s_tKey.Read2 == s_tKey.Write)
    {
        return KEY_NONE;
    }
    else
    {
        ret = s_tKey.Buf[s_tKey.Read2];

        if (++s_tKey.Read2 >= KEY_FIFO_SIZE)
        {
            s_tKey.Read2 = 0;
        }
        return ret;
    }
}
```

返回值 KEY_NONE = 0，表示按键缓冲区为空，所有的按键时间已经处理完毕。按键的键值定义在 bsp_key.h 文件，下面是具体内容：

```
typedef enum
{
    KEY_NONE = 0,           /* 0 表示按键事件 */

    KEY_1_DOWN,             /* 1 键按下 */
    KEY_1_UP,               /* 1 键弹起 */
    KEY_1_LONG,             /* 1 键长按 */

    KEY_2_DOWN,             /* 2 键按下 */
    KEY_2_UP,               /* 2 键弹起 */
    KEY_2_LONG,             /* 2 键长按 */

    KEY_3_DOWN,             /* 3 键按下 */
    KEY_3_UP,               /* 3 键弹起 */
    KEY_3_LONG,             /* 3 键长按 */

    KEY_4_DOWN,             /* 4 键按下 */
    KEY_4_UP,               /* 4 键弹起 */
    KEY_4_LONG,             /* 4 键长按 */

    KEY_5_DOWN,             /* 5 键按下 */
    KEY_5_UP,               /* 5 键弹起 */
    KEY_5_LONG,             /* 5 键长按 */

    KEY_6_DOWN,             /* 6 键按下 */
    KEY_6_UP,               /* 6 键弹起 */
    KEY_6_LONG,             /* 6 键长按 */
```

```
    KEY_7_DOWN,             /* 7 键按下 */
    KEY_7_UP,               /* 7 键弹起 */
    KEY_7_LONG,             /* 7 键长按 */

    KEY_8_DOWN,             /* 8 键按下 */
    KEY_8_UP,               /* 8 键弹起 */
    KEY_8_LONG,             /* 8 键长按 */

    /* 组合键 */
    KEY_9_DOWN,             /* 9 键按下 */
    KEY_9_UP,               /* 9 键弹起 */
    KEY_9_LONG,             /* 9 键长按 */

    KEY_10_DOWN,            /* 10 键按下 */
    KEY_10_UP,              /* 10 键弹起 */
    KEY_10_LONG,            /* 10 键长按 */
}KEY_ENUM;
```

必须按次序定义每个键的按下、弹起和长按事件，即每个按键对象 (组合键也算 1 个) 占用 3 个数值。我们推荐使用枚举 enum，不用#define 的原因：

- 便于新增键值,方便调整顺序。
- 使用{ } 将一组相关的定义封装起来便于理解。
- 编译器可帮我们避免键值重复。

我们来看红外遥控器的键值定义，在 bsp_ir_decode.h 文件。因为遥控器按键和主板按键共用同一个 FIFO，因此在这里我们先贴出这段定义代码，让大家有个初步印象。

```
/* 定义红外遥控器按键代码， 和 bsp_key.h 的物理按键代码统一编码 */
typedef enum
{
    IR_KEY_STRAT  = 0x80,
    IR_KEY_POWER  = IR_KEY_STRAT + 0x45,
    IR_KEY_MENU   = IR_KEY_STRAT + 0x47,
    IR_KEY_TEST   = IR_KEY_STRAT + 0x44,
    IR_KEY_UP     = IR_KEY_STRAT + 0x40,
    IR_KEY_RETURN = IR_KEY_STRAT + 0x43,
    IR_KEY_LEFT   = IR_KEY_STRAT + 0x07,
    IR_KEY_OK     = IR_KEY_STRAT + 0x15,
    IR_KEY_RIGHT  = IR_KEY_STRAT + 0x09,
    IR_KEY_0      = IR_KEY_STRAT + 0x16,
    IR_KEY_DOWN   = IR_KEY_STRAT + 0x19,
    IR_KEY_C      = IR_KEY_STRAT + 0x0D,
    IR_KEY_1      = IR_KEY_STRAT + 0x0C,
    IR_KEY_2      = IR_KEY_STRAT + 0x18,
    IR_KEY_3      = IR_KEY_STRAT + 0x5E,
    IR_KEY_4      = IR_KEY_STRAT + 0x08,
    IR_KEY_5      = IR_KEY_STRAT + 0x1C,
    IR_KEY_6      = IR_KEY_STRAT + 0x5A,
    IR_KEY_7      = IR_KEY_STRAT + 0x42,
    IR_KEY_8      = IR_KEY_STRAT + 0x52,
    IR_KEY_9      = IR_KEY_STRAT + 0x4A,
}IR_KEY_E;
```

我们下面来看一段简单的应用。这个应用的功能是：主板 K1 键控制 LED1 指示灯；遥控器的 POWER 键和 MENU 键控制 LED2 指示灯。

```
#include "bsp.h"
```

```
int main(void)
{
    uint8_t ucKeyCode;

    bsp_Init();

    IRD_StartWork();   /* 启动红外解码 */

    while(1)
    {
        bsp_Idle();

        /* 处理按键事件 */
        ucKeyCode = bsp_GetKey();
        if (ucKeyCode > 0)
        {
            /* 有键按下 */
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:       /* K1 键按下 */
                    bsp_LedOn(1);       /* 点亮 LED1 */
                    break;

                case KEY_UP_K1:         /* K1 键弹起 */
                    bsp_LedOff(1);      /* 熄灭 LED1 */
                    break;

                case IR_KEY_POWER:      /* 遥控器 POWER 键按下 */
                    bsp_LedOn(1);       /* 点亮 LED2 */
                    break;

                case IR_KEY_MENU:       /* 遥控器 MENU 键按下 */
                    bsp_LedOff(1);      /* 熄灭 LED2 */
                    break;

                case MSG_485_RX:        /* 通信程序的发来的消息 */
                    /* 执行通信程序的指令 */
                    break;

                default:
                    break;
            }
        }
    }
}
```

看到这里，想必你已经意识到 bsp_PutKey 函数的强大之处了，可以将不相关的硬件输入设备统一为一个相同的接口函数。

在上面的应用程序中，我们特意添加了一段红色的代码来解说更高级的用法。485 通信程序收到有效的命令后通过 bsp_PutKey（MSG_485_RX）函数可以通知 APP 应用程序进行进一步加工处理（比如显示接收成功）。这是一种非常好的任务间信息传递方式，它不会破坏程序结构。不必新增全局变量来做这种事情，你只需要添加一个键值代码。

对于简单的程序，可以借用按键 FIFO 来进行少量的信息传递。对于复杂的应用，我们推荐使用

bsp_msg 专门来做这种任务间的通信。因为 bsp_msg 除了传递消息代码外，还可以传递参数结构。

19.3.3 按键检测程序分析

在 bsp_key.h 中定了结构体类型 KEY_T。

```
#define KEY_COUNT    10          /* 按键个数, 8 个独立建 + 2 个组合键 */

typedef struct
{
    /* 下面是一个函数指针，指向判断按键手否按下的函数 */
    uint8_t (*IsKeyDownFunc)(void); /* 按键按下的判断函数,1 表示按下 */

    uint8_t  Count;         /* 滤波器计数器 */
    uint16_t LongCount;     /* 长按计数器 */
    uint16_t LongTime;      /* 按键按下持续时间, 0 表示不检测长按 */
    uint8_t  State;         /* 按键当前状态（按下还是弹起） */
    uint8_t  RepeatSpeed;   /* 连续按键周期 */
    uint8_t  RepeatCount;   /* 连续按键计数器 */
}KEY_T;
```

在 bsp_key.c 中定义 s_tBtn 结构体数组变量。

```
static KEY_T s_tBtn[KEY_COUNT];
static KEY_FIFO_T s_tKey;       /* 按键 FIFO 变量,结构体 */
```

每个按键对象都分配一个结构体变量，这些结构体变量以数组的形式存在将便于我们简化程序代码行数。

使用函数指针 IsKeyDownFunc 可以将每个按键的检测以及组合键的检测代码进行统一管理。

因为函数指针必须先赋值，才能被作为函数执行。因此在定时扫描按键之前，必须先执行一段初始化函数来设置每个按键的函数指针和参数。这个函数是 void bsp_InitKey(void)。它由 bsp_Init()调用。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitKey
*    功能说明: 初始化按键. 该函数被 bsp_Init() 调用。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitKey(void)
{
    bsp_InitKeyVar();       /* 初始化按键变量 */
    bsp_InitKeyHard();      /* 初始化按键硬件 */
}
```

下面是 bsp_InitKeyVar 函数的定义：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitKeyVar
*    功能说明: 初始化按键变量
*    形    参:  无
*    返 回 值: 无
*********************************************************************************************************
*/
static void bsp_InitKeyVar(void)
```

```
{
    uint8_t i;

    /* 对按键 FIFO 读写指针清零 */
    s_tKey.Read = 0;
    s_tKey.Write = 0;
    s_tKey.Read2 = 0;

    /* 给每个按键结构体成员变量赋一组缺省值 */
    for (i = 0; i < KEY_COUNT; i++)
    {
        s_tBtn[i].LongTime = KEY_LONG_TIME;            /* 长按时间 0 表示不检测长按键事件 */
        s_tBtn[i].Count = KEY_FILTER_TIME / 2;         /* 计数器设置为滤波时间的一半 */
        s_tBtn[i].State = 0;                           /* 按键缺省状态，0 为未按下 */
        s_tBtn[i].RepeatSpeed = 0;                     /* 按键连发的速度，0 表示不支持连发 */
        s_tBtn[i].RepeatCount = 0;                     /* 连发计数器 */
    }

    /* 如果需要单独更改某个按键的参数，可以在此单独重新赋值 */

    /* 摇杆上下左右，支持长按 1 秒后，自动连发 */
    bsp_SetKeyParam(KID_JOY_U, 100, 6);
    bsp_SetKeyParam(KID_JOY_D, 100, 6);
    bsp_SetKeyParam(KID_JOY_L, 100, 6);
    bsp_SetKeyParam(KID_JOY_R, 100, 6);
}
```

注意下一下 Count 这个成员变量，没有设置为 0。为了避免主板上电的瞬间，检测到一个无效的按键按下或弹起事件。我们将这个滤波计数器的初值设置为正常值的 1/2。bsp_key.h 中定义了滤波时间和长按时间。

```
/*
    按键滤波时间 50ms, 单位 10ms。
    只有连续检测到 50ms 状态不变才认为有效，包括弹起和按下两种事件
    即使按键电路不做硬件滤波，该滤波机制也可以保证可靠地检测到按键事件
*/
#define KEY_FILTER_TIME   5
#define KEY_LONG_TIME     100            /* 单位 10ms， 持续 1 秒，认为长按事件 */
```

uint8_t KeyPinActive(uint8_t _id)（会调用函数 KeyPinActive 判断状态）函数就是最底层的 GPIO 输入状态判断函数。

```
/*
*********************************************************************************************************
*    函 数 名: KeyPinActive
*    功能说明: 判断按键是否按下
*    形    参: 无
*    返 回 值: 返回值 1 表示按下(导通)，0 表示未按下（释放）
*********************************************************************************************************
*/
static uint8_t KeyPinActive(uint8_t _id)
{
    uint8_t level;

    if ((s_gpio_list[_id].gpio->IDR & s_gpio_list[_id].pin) == 0)
    {
        level = 0;
    }
```

```
    else
    {
        level = 1;
    }

    if (level == s_gpio_list[_id].ActiveLevel)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}

/*
*********************************************************************************************************
*    函 数 名: IsKeyDownFunc
*    功能说明: 判断按键是否按下。单键和组合键区分。单键事件不允许有其他键按下。
*    形    参: 无
*    返 回 值: 返回值 1 表示按下(导通)，0 表示未按下（释放）
*********************************************************************************************************
*/
static uint8_t IsKeyDownFunc(uint8_t _id)
{
    /* 实体单键 */
    if (_id < HARD_KEY_NUM)
    {
        uint8_t i;
        uint8_t count = 0;
        uint8_t save = 255;

        /* 判断有几个键按下 */
        for (i = 0; i < HARD_KEY_NUM; i++)
        {
            if (KeyPinActive(i))
            {
                count++;
                save = i;
            }
        }

        if (count == 1 && save == _id)
        {
            return 1;/* 只有 1 个键按下时才有效 */
        }

        return 0;
    }

    /* 组合键 K1K2 */
    if (_id == HARD_KEY_NUM + 0)
    {
        if (KeyPinActive(KID_K1) && KeyPinActive(KID_K2))
        {
            return 1;
        }
        else
```

```
        {
            return 0;
        }
    }

    /* 组合键 K2K3 */
    if (_id == HARD_KEY_NUM + 1)
    {
        if (KeyPinActive(KID_K2) && KeyPinActive(KID_K3))
        {
            return 1;
        }
        else
        {
            return 0;
        }
    }

    return 0;
}
```

在使用 GPIO 之前，我们必须对 GPIO 进行配置，比如打开 GPIO 时钟，设置 GPIO 输入输出方向，设置上下拉电阻。下面是配置 GPIO 的代码，也就是 bsp_InitKeyHard（）函数：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitKeyHard
*    功能说明: 配置按键对应的 GPIO
*    形    参:  无
*    返 回 值: 无
*********************************************************************************************************
*/
static void bsp_InitKeyHard(void)
{
    GPIO_InitTypeDef gpio_init;
    uint8_t i;

    /* 第 1 步：打开 GPIO 时钟 */
    ALL_KEY_GPIO_CLK_ENABLE();

    /* 第 2 步：配置所有的按键 GPIO 为浮动输入模式(实际上 CPU 复位后就是输入状态) */
    gpio_init.Mode = GPIO_MODE_INPUT;                /* 设置输入 */
    gpio_init.Pull = GPIO_NOPULL;                    /* 上下拉电阻不使能 */
    gpio_init.Speed = GPIO_SPEED_FREQ_VERY_HIGH;     /* GPIO 速度等级 */

    for (i = 0; i < HARD_KEY_NUM; i++)
    {
        gpio_init.Pin = s_gpio_list[i].pin;
        HAL_GPIO_Init(s_gpio_list[i].gpio, &gpio_init);
    }
}
```

我们再来看看按键是如何执行扫描检测的。

按键扫描函数 bsp_KeyScan10ms ()每隔 10ms 被执行一次。bsp_RunPer10ms 函数在 systick 中断服务程序中执行。

```
void bsp_RunPer10ms(void)
{
```

```
    bsp_KeyScan10ms();      /* 扫描按键 */
}
```

bsp_KeyScan10ms ()函数的实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_KeyScan10ms
*    功能说明: 扫描所有按键。非阻塞，被 systick 中断周期性的调用，10ms 一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_KeyScan10ms(void)
{
    uint8_t i;

    for (i = 0; i < KEY_COUNT; i++)
    {
        bsp_DetectKey(i);
    }
}
```

每隔 10ms 所有的按键 GPIO 均会被扫描检测一次。bsp_DetectKey 函数实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_DetectKey
*    功能说明: 检测一个按键。非阻塞状态，必须被周期性的调用。
*    形    参: IO 的 id，  从 0 开始编码
*    返 回 值: 无
*********************************************************************************************************
*/
static void bsp_DetectKey(uint8_t i)
{
    KEY_T *pBtn;

    pBtn = &s_tBtn[i];
    if (IsKeyDownFunc(i))
    {
        if (pBtn->Count < KEY_FILTER_TIME)
        {
            pBtn->Count = KEY_FILTER_TIME;
        }
        else if(pBtn->Count < 2 * KEY_FILTER_TIME)
        {
            pBtn->Count++;
        }
        else
        {
            if (pBtn->State == 0)
            {
                pBtn->State = 1;

                /* 发送按钮按下的消息 */
                bsp_PutKey((uint8_t)(3 * i + 1));
            }

            if (pBtn->LongTime > 0)
            {
```

```
                if (pBtn->LongCount < pBtn->LongTime)
                {
                    /* 发送按钮持续按下的消息 */
                    if (++pBtn->LongCount == pBtn->LongTime)
                    {
                        /* 键值放入按键 FIFO */
                        bsp_PutKey((uint8_t)(3 * i + 3));
                    }
                }
                else
                {
                    if (pBtn->RepeatSpeed > 0)
                    {
                        if (++pBtn->RepeatCount >= pBtn->RepeatSpeed)
                        {
                            pBtn->RepeatCount = 0;
                            /* 常按键后，每隔 RepeatSpeed * 10ms 发送 1 个按键 */
                            bsp_PutKey((uint8_t)(3 * i + 1));
                        }
                    }
                }
            }
        }
    }
    else
    {
        if(pBtn->Count > KEY_FILTER_TIME)
        {
            pBtn->Count = KEY_FILTER_TIME;
        }
        else if(pBtn->Count != 0)
        {
            pBtn->Count--;
        }
        else
        {
            if (pBtn->State == 1)
            {
                pBtn->State = 0;

                /* 发送按钮弹起的消息 */
                bsp_PutKey((uint8_t)(3 * i + 2));
            }
        }

        pBtn->LongCount = 0;
        pBtn->RepeatCount = 0;
    }
}
```

对于初学者，这个函数看起来比较吃力，我们拆分进行分析。

```
pBtn = &s_tBtn[i];
```

读取相应按键的结构体地址，程序里面每个按键都有自己的结构体。

```
static KEY_T s_tBtn[KEY_COUNT];
if (IsKeyDownFunc(i))
{
    这个里面执行的是按键按下的处理
```

```
}
else
{
    这个里面执行的是按键松手的处理或者按键没有按下的处理
}
```

执行函数 IsKeyDownFunc(i)做按键状态判断。

```
/*
*********************************************************************************
下面这个 if 语句主要是用于按键滤波前给 Count 设置一个初值，前面说按键初始化的时候
已经设置了 Count = KEY_FILTER_TIME/2
*********************************************************************************
*/
if (pBtn->Count < KEY_FILTER_TIME)
{
    pBtn->Count = KEY_FILTER_TIME;
}

/*
*********************************************************************************
这里实现 KEY_FILTER_TIME 时间长度的延迟
*********************************************************************************
*/
else if(pBtn->Count < 2 * KEY_FILTER_TIME)
{
    pBtn->Count++;
}
/*
*********************************************************************************
这里实现 KEY_FILTER_TIME 时间长度的延迟
*********************************************************************************
*/
else
{
/*
*********************************************************************************
这个 State 变量是有其实际意义的，如果按键按下了，这里就将其设置为 1，如果没有按下这个
变量的值就会一直是 0，这样设置的目的可以有效的防止一种情况的出现：比如按键 K1 在某个
时刻检测到了按键有按下，那么它就会做进一步的滤波处理，但是在滤波的过程中，这个按键
按下的状态消失了，这个时候就会进入到上面第二步 else 语句里面，然后再做按键松手检测滤波
，滤波结束后判断这个 State 变量，如果前面就没有检测到按下，这里就不会记录按键弹起。
*********************************************************************************
*/
    if (pBtn->State == 0)
    {
        pBtn->State = 1;

        /* 发送按钮按下的消息 */
        bsp_PutKey((uint8_t)(3 * i + 1));
    }

    if (pBtn->LongTime > 0)
    {
        if (pBtn->LongCount < pBtn->LongTime)
        {
            /* 发送按钮持续按下的消息 */
            if (++pBtn->LongCount == pBtn->LongTime)
            {
```

```
                /* 键值放入按键 FIFO */
                bsp_PutKey((uint8_t)(3 * i + 3));
            }
        }
        else
        {
            if (pBtn->RepeatSpeed > 0)
            {
                if (++pBtn->RepeatCount >= pBtn->RepeatSpeed)
                {
                    pBtn->RepeatCount = 0;
                    /* 长按键后，每隔 10ms 发送 1 个按键 */
                    bsp_PutKey((uint8_t)(3 * i + 1));
                }
            }
        }
    }
}
```

19.3.4按键检测采用中断方式还是查询方式

检测按键有中断方式和 GPIO 查询方式两种。我们推荐大家用 GPIO 查询方式。

从裸机的角度分析

中断方式：中断方式可以快速地检测到按键按下，并执行相应的按键程序，但实际情况是由于按键的机械抖动特性，在程序进入中断后必须进行滤波处理才能判定是否有效的按键事件。如果每个按键都是独立的接一个 IO 引脚，需要我们给每个 IO 都设置一个中断，程序中过多的中断会影响系统的稳定性。中断方式跨平台移植困难。

查询方式：查询方式有一个最大的缺点就是需要程序定期的去执行查询，耗费一定的系统资源。实际上耗费不了多大的系统资源，因为这种查询方式也只是查询按键是否按下，按键事件的执行还是在主程序里面实现。

从 OS 的角度分析

中断方式：在 OS 中要尽可能少用中断方式，因为在 RTOS 中过多的使用中断会影响系统的稳定性和可预见性（抢占式调度的 OS 基本没有可预见性）。只有比较重要的事件处理需要用中断的方式。

查询方式：对于用户按键推荐使用这种查询方式来实现，现在的 OS 基本都带有 CPU 利用率的功能，这个按键 FIFO 占用的还是很小的，基本都在 1%以下。

19.4按键板级支持包（bsp_key.c)

按键驱动文件 bsp_key.c 主要实现了如下几个 API：

- KeyPinActive
- IsKeyDownFunc

- bsp_InitKey
- bsp_InitKeyHard
- bsp_InitKeyVar
- bsp_PutKey
- bsp_GetKey
- bsp_GetKey2
- bsp_GetKeyState
- bsp_SetKeyParam
- bsp_ClearKey
- bsp_DetectKey
- bsp_DetectFastIO
- bsp_KeyScan10ms
- bsp_KeyScan1ms

所有这些函数在本章的 19.3 小节都进行了详细讲解，本小节主要是把需要用户调用的三个函数做个说明。

19.4.1 函数 bsp_InitKeyHard

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitKey
*    功能说明: 初始化按键. 该函数被 bsp_Init() 调用。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitKey(void)
{
    bsp_InitKeyVar();      /* 初始化按键变量 */
    bsp_InitKeyHard();     /* 初始化按键硬件 */
}
```

函数描述：

此函数主要用于按键的初始化。

使用举例：

底层驱动初始化直接在 bsp.c 文件的函数 bsp_Init 里面调用即可。

19.4.2 函数 bsp_GetKey

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_GetKey
*    功能说明: 从按键 FIFO 缓冲区读取一个键值。
*    形    参: 无
*    返 回 值: 按键代码
*********************************************************************************************************
*/
uint8_t bsp_GetKey(void)
{
    uint8_t ret;

    if (s_tKey.Read == s_tKey.Write)
    {
        return KEY_NONE;
    }
    else
    {
        ret = s_tKey.Buf[s_tKey.Read];

        if (++s_tKey.Read >= KEY_FIFO_SIZE)
        {
            s_tKey.Read = 0;
        }
        return ret;
    }
}
```

函数描述：

此函数用于从 FIFO 中读取键值。

使用举例：

调用此函数前，务必优先调用函数 bsp_InitKey 进行初始化。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */

    bsp_Init();         /* 硬件初始化 */

    /* 进入主程序循环体 */
    while (1)
    {
        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
```

```
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下 */
                    printf("K1 键按下\r\n");
                    break;

                case KEY_DOWN_K2:           /* K2 键按下 */
                    printf("K2 键按下\r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

19.4.3 函数 bsp_KeyScan10ms

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_KeyScan10ms
*    功能说明: 扫描所有按键。非阻塞，被 systick 中断周期性的调用，10ms 一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_KeyScan10ms(void)
{
    uint8_t i;

    for (i = 0; i < KEY_COUNT; i++)
    {
        bsp_DetectKey(i);
    }
}
```

函数描述：

此函数是按键的主处理函数，用于检测和存储按下、松手、长按等状态。

使用举例：

调用此函数前，务必优先调用函数 bsp_InitKey 进行初始化。

另外，此函数需要周期性调用，每 10ms 调用一次。

- 如果是裸机使用，将此函数放在 bsp.c 文件的 bsp_RunPer10ms 函数里面即可，这个函数是由滴答定时器调用的，也就是说，大家要使用按键，定时器的初始化函数 bsp_InitTimer 一定要调用。
- 如果是 RTOS 使用，需要开启一个 10ms 为周期的任务调用函数 bsp_KeyScan10ms。

19.5 按键 FIFO 驱动移植和使用

按键移植步骤如下：

◆ 第 1 步：复制 bsp_key.c 和 bsp_key.c 到自己的工程。

◆ 第 2 步：根据自己使用的独立按键个数和组合键个数，修改几个地方。

```
#define HARD_KEY_NUM    8                /* 实体按键个数 */
#define KEY_COUNT    (HARD_KEY_NUM + 2)    /* 8 个独立建 + 2 个组合按键 */
```

◆ 第 3 步：根据使用的引脚时钟，修改下面函数：

```
/* 使能 GPIO 时钟 */
#define ALL_KEY_GPIO_CLK_ENABLE() {  \
          __HAL_RCC_GPIOB_CLK_ENABLE();    \
          __HAL_RCC_GPIOC_CLK_ENABLE();    \
          __HAL_RCC_GPIOG_CLK_ENABLE();    \
          __HAL_RCC_GPIOH_CLK_ENABLE();    \
          __HAL_RCC_GPIOI_CLK_ENABLE();    \
    };
```

◆ 第 4 步：根据使用的具体引脚，修改如下函数，第 3 列参数低电平表示按下或者高电平表示按下：

```
/* GPIO 和 PIN 定义 */
static const X_GPIO_T s_gpio_list[HARD_KEY_NUM] = {
    {GPIOI, GPIO_PIN_8, 0},     /* K1 */
    {GPIOC, GPIO_PIN_13, 0},    /* K2 */
    {GPIOH, GPIO_PIN_4, 0},     /* K3 */
    {GPIOG, GPIO_PIN_2, 0},     /* JOY_U */
    {GPIOB, GPIO_PIN_0, 0},     /* JOY_D */
    {GPIOG, GPIO_PIN_3, 0},     /* JOY_L */
    {GPIOG, GPIO_PIN_7, 0},     /* JOY_R */
    {GPIOI, GPIO_PIN_11, 0},    /* JOY_OK */
};
```

◆ 第 5 步：根据使用的组合键个数，在函数 IsKeyDownFunc 里面添加相应个数的函数：

```
    /* 组合键 K1K2 */
    if (_id == HARD_KEY_NUM + 0)
    {
        if (KeyPinActive(KID_K1) && KeyPinActive(KID_K2))
        {
            return 1;
        }
        else
        {
            return 0;
        }
    }
```

第 2 行 ID 表示 HARD_KEY_NUM + 0 的组合键，HARD_KEY_NUM + 1 表示下一个组合键，以此类推。另外就是，函数 KeyPinActive 的参数是表示检测哪两个按键，设置 0 的时候表示第 4 步里面的第 1 组按键，设置为 1 表示第 2 组按键，以此类推。

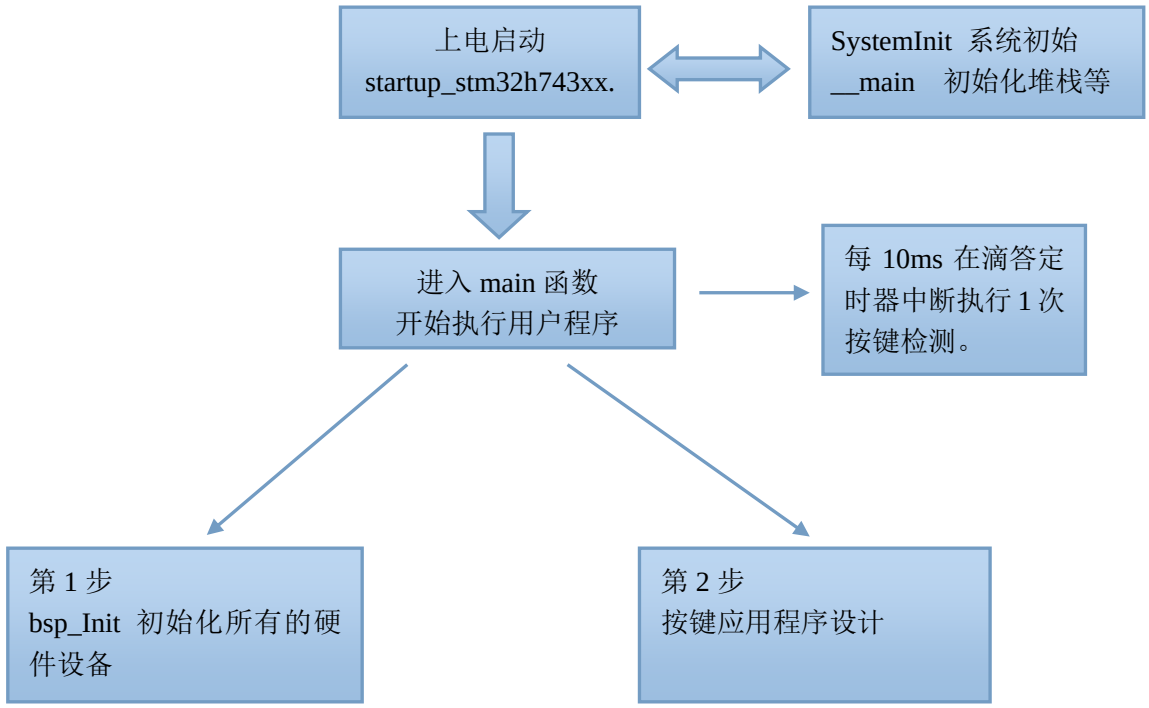
◆ 第 6 步：主要用到 HAL 库的 GPIO 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。

◆ 第 7 步：移植完整，应用方法看本章节配套例子即可。

特别注意，别忘了每 10ms 调用一次按键检测函数 bsp_KeyScan10ms。

19.6 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU、Cache、HAL 库、系统时钟、滴答定时器、按键等。
  - 第 2 部分，应用程序设计部分，实现了一个按键应用。
  - 第 3 部分，按键扫描程序每 10ms 在滴答定时中断执行一次。

19.7 实验例程说明（MDK）

配套例子：

V7-002_按键检测（软件滤波，FIFO 机制）

实验目的：

1. 学习按键的按下，弹起，长按和组合键的实现。

实验内容：

1. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。

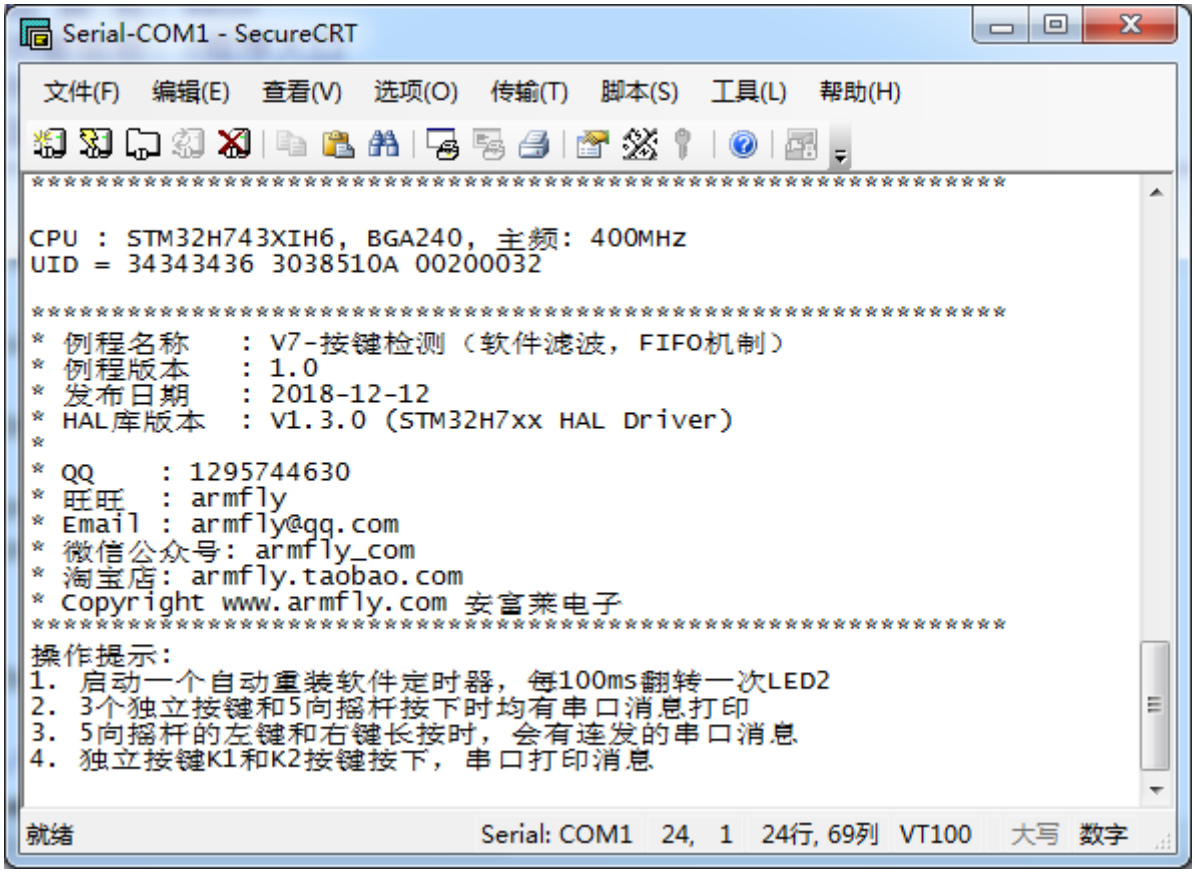
实验操作：

1. 3 个独立按键和 5 向摇杆按下时均有串口消息打印。

2. 5 向摇杆的左键和右键长按时，会有连发的串口消息。
3. 独立按键 K1 和 K2 按键按下，串口打印消息。

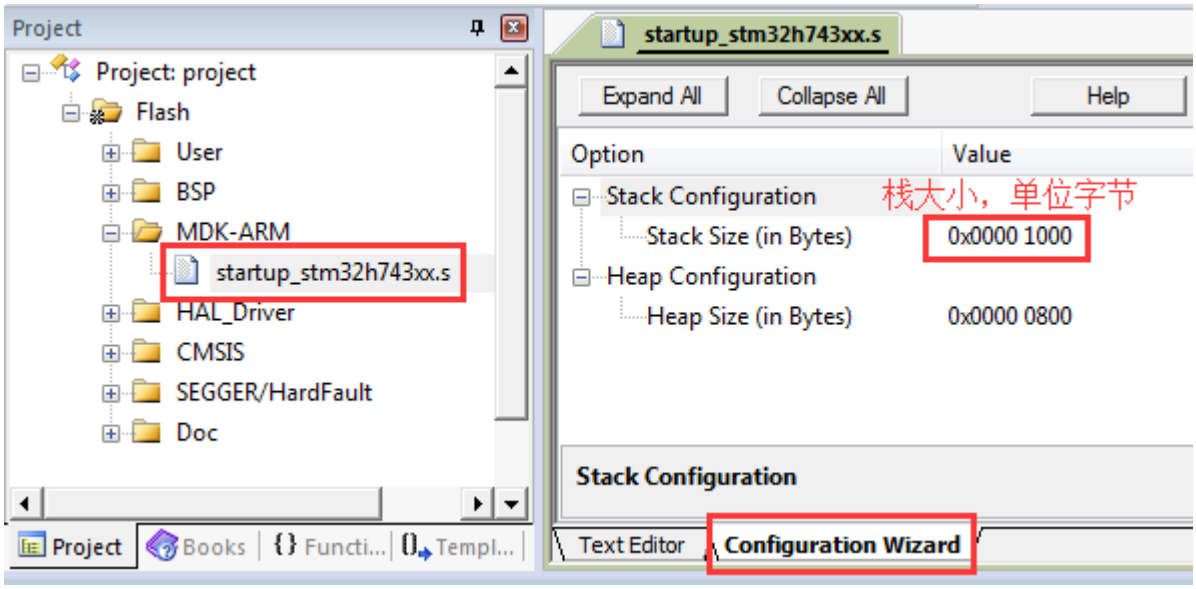
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

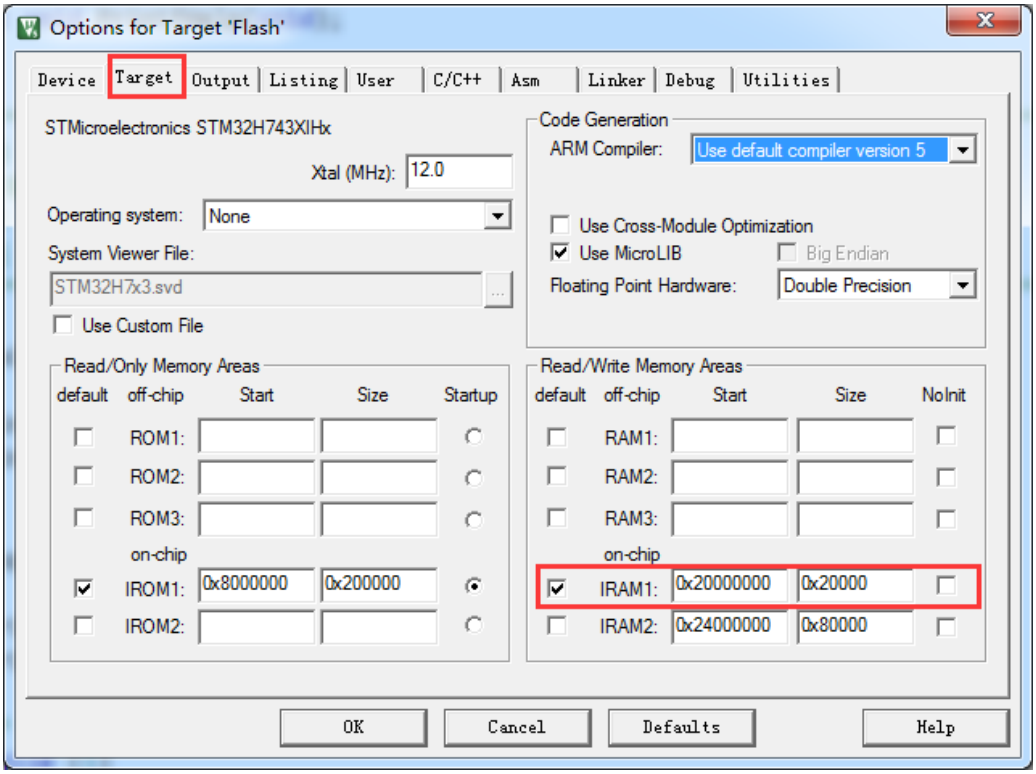


程序设计：

◆ **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
```

```
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ 每 10ms 调用一次按键检测：

按键检测是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ 主功能：

主功能的实现主要分为两部分：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2
- 按键消息的读取，检测到按下后，做串口打印。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
```

```
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */

    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */


    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下 */
                    printf("K1 键按下\r\n");
                    break;

                case KEY_UP_K1:                 /* K1 键弹起 */
                    printf("K1 键弹起\r\n");
                    break;

                case KEY_DOWN_K2:           /* K2 键按下 */
                    printf("K2 键按下\r\n");
                    break;

                case KEY_UP_K2:                 /* K2 键弹起 */
                    printf("K2 键弹起\r\n");
                    break;

                case KEY_DOWN_K3:           /* K3 键按下 */
                    printf("K3 键按下\r\n");
                    break;

                case KEY_UP_K3:                 /* K3 键弹起 */
                    printf("K3 键弹起\r\n");
                    break;
```

```
                case JOY_DOWN_U:            /* 摇杆 UP 键按下 */
                    printf("摇杆上键按下\r\n");
                    break;

                case JOY_DOWN_D:            /* 摇杆 DOWN 键按下 */
                    printf("摇杆下键按下\r\n");
                    break;

                case JOY_DOWN_L:            /* 摇杆 LEFT 键按下 */
                    printf("摇杆左键按下\r\n");
                    break;

                case JOY_LONG_L:            /* 摇杆 LEFT 键长按 */
                    printf("摇杆左键长按\r\n");
                    break;

                case JOY_DOWN_R:            /* 摇杆 RIGHT 键按下 */
                    printf("摇杆右键按下\r\n");
                    break;

                case JOY_LONG_R:            /* 摇杆 RIGHT 键长按 */
                    printf("摇杆右键长按\r\n");
                    break;

                case JOY_DOWN_OK:           /* 摇杆 OK 键按下 */
                    printf("摇杆 OK 键按下\r\n");
                    break;

                case JOY_UP_OK:             /* 摇杆 OK 键弹起 */
                    printf("摇杆 OK 键弹起\r\n");
                    break;

              case SYS_DOWN_K1K2:           /* 摇杆 OK 键弹起 */
                    printf("K1 和 K2 组合键按下\r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }

        }
    }
}
```

19.8 实验例程说明（IAR）

配套例子：

V7-002_按键检测（软件滤波，FIFO 机制）

实验目的：

1. 学习按键的按下，弹起，长按和组合键的实现。

实验内容：

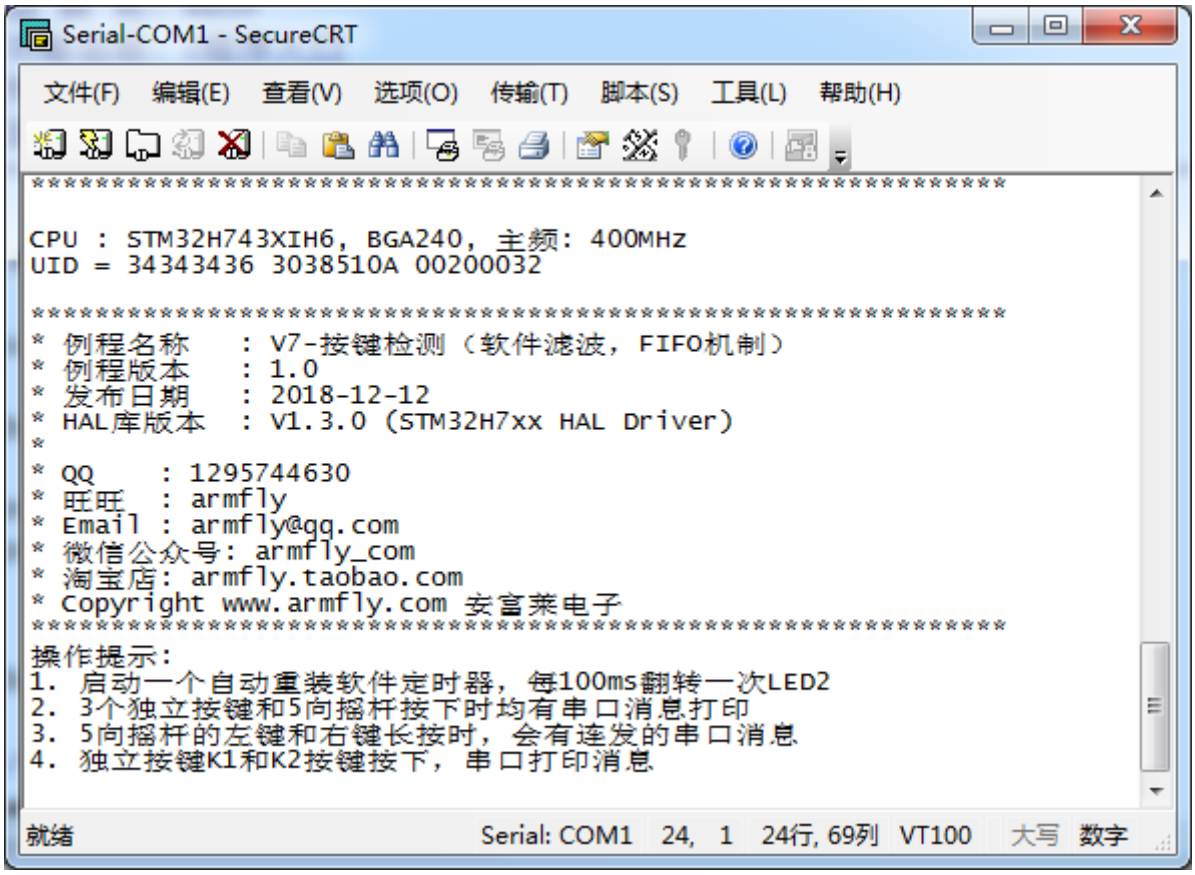
1. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。

实验操作：

1. 3 个独立按键和 5 向摇杆按下时均有串口消息打印。
2. 5 向摇杆的左键和右键长按时，会有连发的串口消息。
3. 独立按键 K1 和 K2 按键按下，串口打印消息。

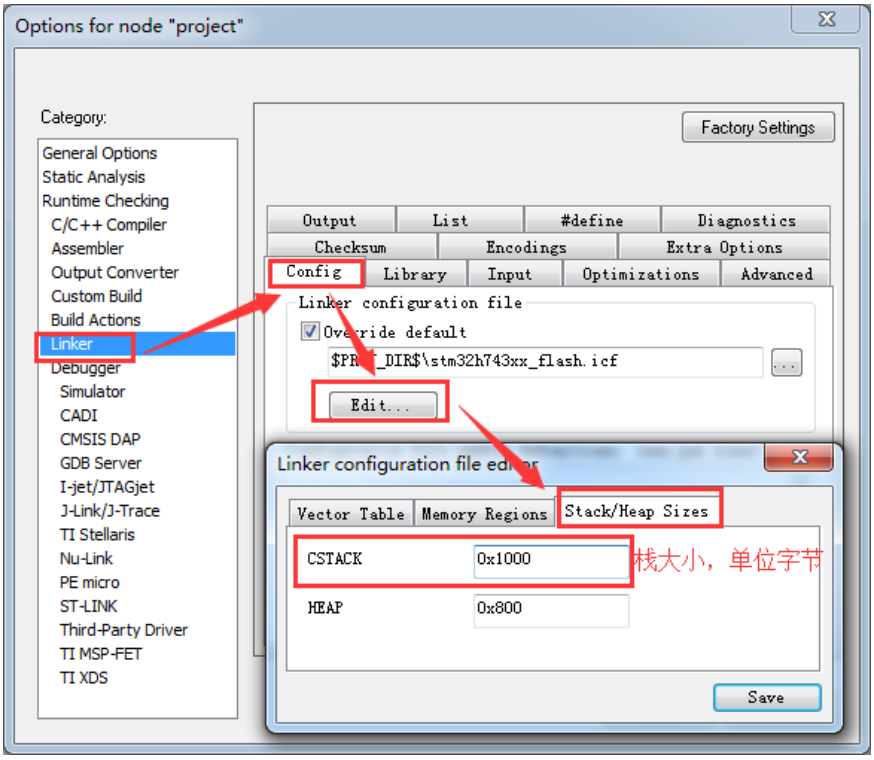
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

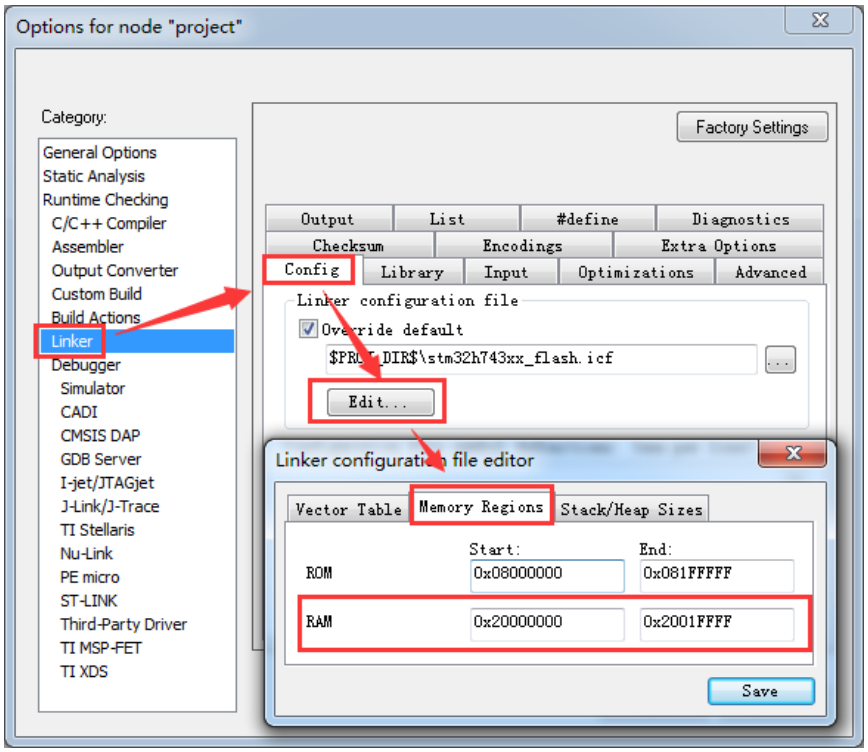


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
```

```
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
```

```
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **每 10ms 调用一次按键检测：**

按键检测是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能：**

主功能的实现主要分为两部分：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2
- 按键消息的读取，检测到按下后，做串口打印。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */

    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */


    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
```

```
{
    switch (ucKeyCode)
    {
        case KEY_DOWN_K1:           /* K1 键按下 */
            printf("K1 键按下\r\n");
            break;

        case KEY_UP_K1:             /* K1 键弹起 */
            printf("K1 键弹起\r\n");
            break;

        case KEY_DOWN_K2:           /* K2 键按下 */
            printf("K2 键按下\r\n");
            break;

        case KEY_UP_K2:             /* K2 键弹起 */
            printf("K2 键弹起\r\n");
            break;

        case KEY_DOWN_K3:           /* K3 键按下 */
            printf("K3 键按下\r\n");
            break;

        case KEY_UP_K3:             /* K3 键弹起 */
            printf("K3 键弹起\r\n");
            break;

        case JOY_DOWN_U:            /* 摇杆 UP 键按下 */
            printf("摇杆上键按下\r\n");
            break;

        case JOY_DOWN_D:            /* 摇杆 DOWN 键按下 */
            printf("摇杆下键按下\r\n");
            break;

        case JOY_DOWN_L:            /* 摇杆 LEFT 键按下 */
            printf("摇杆左键按下\r\n");
            break;

        case JOY_LONG_L:            /* 摇杆 LEFT 键长按 */
            printf("摇杆左键长按\r\n");
            break;

        case JOY_DOWN_R:            /* 摇杆 RIGHT 键按下 */
            printf("摇杆右键按下\r\n");
            break;

        case JOY_LONG_R:            /* 摇杆 RIGHT 键长按 */
            printf("摇杆右键长按\r\n");
            break;

        case JOY_DOWN_OK:           /* 摇杆 OK 键按下 */
            printf("摇杆 OK 键按下\r\n");
            break;

        case JOY_UP_OK:             /* 摇杆 OK 键弹起 */
            printf("摇杆 OK 键弹起\r\n");
            break;
```

```
                case SYS_DOWN_K1K2:             /* 摇杆 OK 键弹起 */
                    printf("K1 和 K2 组合键按下\r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }

        }
    }
}
```

19.9 总结

这个方案在实际项目中已经经过千锤百炼，大家可以放心使用。建议熟练掌握其用法。

第20章 STM32H7 的 GPIO 应用之无源蜂鸣器

本章教程为大家介绍STM32H7的GPIO应用之无源蜂鸣器,蜂鸣器也是GPIO控制的经典测试例程,可以让大家对 STM32H7 应用有个简单的整体认识。

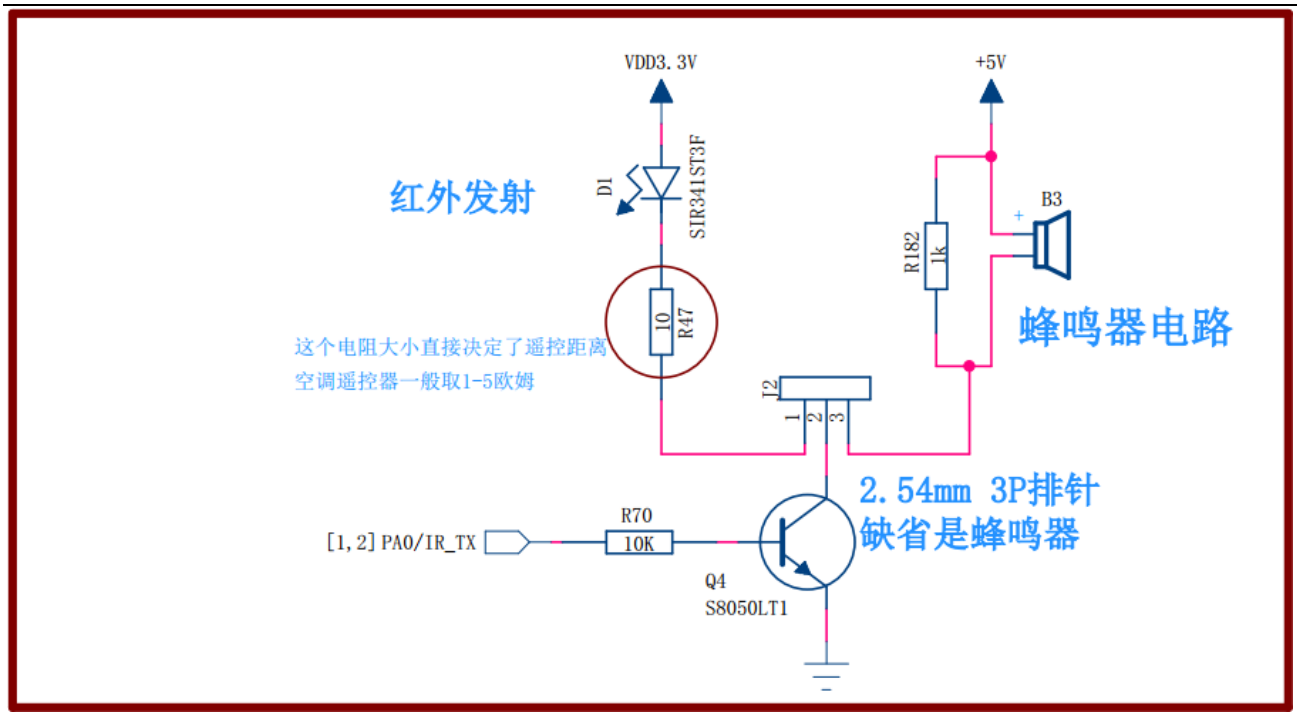
20.1 初学者重要提示
20.2 蜂鸣器硬件设计
20.3 蜂鸣器软件驱动设计
20.4 蜂鸣器板级支持包（bsp_beep.c）
20.5 蜂鸣器驱动移植和使用（裸机版）
20.6 使用例程设计框架
20.7 实验例程说明（MDK）
20.8 实验例程说明（IAR）
20.9 总结

20.1 初学者重要提示

- 学习本章节前，务必保证已经学习了第 13，14 和 15 章。
- 注意有源蜂鸣器和无源蜂鸣器的区别，本章教程的 17.2.1 小节有专门说明。
- 开发板是采用的无源蜂鸣器，需要 PWM 驱动，而截至本章节还没有讲到 PWM，会在 34 章节专门为大家讲解，程序中是通过一个宏定义控制使能和关闭，所以对于初学者来说，当前阶段仅需了解到使能和关闭方法即可，后面学习到 PWM 章节了，再深入了解。
- 无源蜂鸣器的控制采用的非阻塞方式，实际项目中比较实用。

20.2 蜂鸣器硬件设计

蜂鸣器的硬件设计如下：



通过这个硬件设计，有如下两点需要学习：

20.2.1 蜂鸣器分类

蜂鸣器主要有电磁式和电压式两种，而且都有无源蜂鸣器和有源蜂鸣器两类。开发板使用的是电磁式无源蜂鸣器，而有源和无源的区别是有源蜂鸣器内部自带振荡器，给个电压就能发声，但频率是固定的，只能发出一种声音，而无源蜂鸣器频率可控，给个方波才可以发声，并且根据不同频率发出不同的声音效果。

- 拓展知识

关于有源蜂鸣器和无源蜂鸣器区别：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=89764 。

20.2.2 硬件设计

关于硬件驱动，这里主要有三点需要大家认识到：

- S8050TL1 是 NPN 型三极管，这里是当开关使用，PA8 输出高电平的时候三极管导通，输出低电平，三极管关闭。
- 电阻 R70 起到限流的作用。
- 电阻 R47 在这里有特别的作用，首先要普及一个知识点，这里使用的是电磁式蜂鸣器，属于感性负载，切断这种负载必须要注意，如果电流消失，电感两端的电压将急剧上升，这种感应冲击足以损坏逻辑门电路或者其它形式的负载驱动电路，为了保护这个电路，可以用一个二极管或者电阻吸收感应冲击。

- 拓展知识

STM32H7 的 GPIO 控制三极管驱动各种负载的安全措施和注意事项：
http://www.armbbs.cn/forum.php?mod=viewthread&tid=89776 。

20.3 蜂鸣器软件驱动设计

软件驱动对有源蜂鸣器和无源蜂鸣器都做了支持，默认情况下用的是无源蜂鸣器。我们使用蜂鸣器的话，大部分情况下可以配置鸣叫次数、鸣叫的时间和停止的时间。本驱动设计就是基于这种应用方式实现，基本可以满足大部分应用情况。

设计这个软件驱动的关键之处是如何避免采用阻塞式的实现方式，比如要实现鸣叫 1 秒，停止 1 秒，循环 5 次，如果是阻塞方式等待 1 秒执行完毕，那就时间太长了。鉴于这种情况，程序里面实现了一种非阻塞的方式，通过滴答定时器中断每 10ms 调用一次蜂鸣器处理函数来实现鸣叫次数、鸣叫的时间和停止的时间的更新。

20.4 蜂鸣器板级支持包（bsp_beep.c）

蜂鸣器驱动文件 bsp_beep.c 主要实现了如下几个 API：

- BEEP_InitHard
- BEEP_Start
- BEEP_Stop
- BEEP_Pause
- BEEP_Resume
- BEEP_KeyTone
- BEEP_Pro

这里我们重点讲解函数 BEEP_InitHard、BEEP_Sart 和 BEEP_Pro。

函数 BEEP_Stop、BEEP_Pause 和 BEEP_Resume 测试效果不够好，推荐直接使用 BEEP_Sart 即可，设置鸣叫时间、停止鸣叫时间和循环次数。而 BEEP_KeyTone 是基于 BEEP_Start 实现的，直接调用的 BEEP_Start(5, 1, 1); /* 鸣叫 50ms，停 10ms， 1 次 */

20.4.1 宏定义设置

此文件的开头有一个宏定义选择，用户可以选择使用有源蜂鸣器或者无源蜂鸣器。

```
//#define BEEP_HAVE_POWER       /* 定义此行表示有源蜂鸣器，直接通过 GPIO 驱动，无需 PWM */

#ifdef    BEEP_HAVE_POWER       /* 有源蜂鸣器 */
```

```
    /* PA8 */
    #define GPIO_RCC_BEEP   RCC_AHB1Periph_GPIOA
    #define GPIO_PORT_BEEP  GPIOA
    #define GPIO_PIN_BEEP   GPIO_Pin_8

    #define BEEP_ENABLE()   GPIO_PORT_BEEP->BSRRL = GPIO_PIN_BEEP          /* 使能蜂鸣器鸣叫 */
    #define BEEP_DISABLE()  GPIO_PORT_BEEP->BSRRH = GPIO_PIN_BEEP          /* 禁止蜂鸣器鸣叫 */
#else         /* 无源蜂鸣器 */
    /* PA8 ---> TIM1_CH1 */

    /* 1500 表示频率 1.5KHz，5000 表示 50.00%的占空比 */
    #define BEEP_ENABLE()   bsp_SetTIMOutPWM(GPIOA, GPIO_Pin_8, TIM1, 1, 1500, 5000);

    /* 禁止蜂鸣器鸣叫 */
    #define BEEP_DISABLE()  bsp_SetTIMOutPWM(GPIOA, GPIO_Pin_8, TIM1, 1, 1500, 0);
#endif
```

- 使能了宏定义 BEEP_HAVE_POWER 就可以选择使用有源蜂鸣器，默认是无源的。
- 使用无源蜂鸣器时，需要用到定时器的 PWM 功能，这个功能会在 34 章节专门讲解，这里仅需只知道配置了一个 PWM 来驱动蜂鸣器即可。

20.4.2 蜂鸣器结构体变量

为了方便蜂鸣器的控制，专门封装了一个结构体变量：

```
typedef struct _BEEP_T
{
    uint8_t ucEnalbe;
    uint8_t ucState;
    uint16_t usBeepTime;
    uint16_t usStopTime;
    uint16_t usCycle;
    uint16_t usCount;
    uint16_t usCycleCount;
    uint8_t ucMute;
}BEEP_T;
```

- 成员 ucEnalbe：用于使能或者禁止蜂鸣器。
- 成员 ucState：状态变量，用于蜂鸣器鸣叫和停止的区分。
- 成员 usBeepTime：鸣叫时间，单位 10ms。
- 成员 usStopTime：停止鸣叫时间，单位 10ms。
- 成员 usCycle：鸣叫和停止的循环次数。
- 成员 usCount：用于鸣叫和停止时的计数。
- 成员 usCycleCount：用于循环次数计数。
- 成员 ucMute：用于静音。

20.4.3函数 BEEP_InitHard

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: BEEP_InitHard
*    功能说明: 初始化蜂鸣器硬件
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void BEEP_InitHard(void)
{
#ifdef    BEEP_HAVE_POWER        /* 有源蜂鸣器 */
    GPIO_InitTypeDef GPIO_InitStructure;

    /* 打开 GPIOF 的时钟 */
    RCC_AHB1PeriphClockCmd(GPIO_RCC_BEEP, ENABLE);

    BEEP_DISABLE();

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;       /* 设为输出口 */
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;      /* 设为推挽模式 */
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;    /* 上下拉电阻不使能 */
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;   /* IO 口最大速度 */

    GPIO_InitStructure.GPIO_Pin = GPIO_PIN_BEEP;
    GPIO_Init(GPIO_PORT_BEEP, &GPIO_InitStructure);
#endif

    g_tBeep.ucMute = 0;     /* 关闭静音 */
}
```

函数描述：

此函数主要用于蜂鸣器的初始化，代码比较好理解。条件编译实现了一个无源蜂鸣器的初始化，配置引脚为推挽输出模式。由于 V7 开发板使用的无源蜂鸣器，所有没有开启宏定义 BEEP_HAVE_POWER。

使用举例：

底层驱动初始化直接在 bsp.c 文件的函数 bsp_Init 里面调用即可。

20.4.4函数 BEEP_Start

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: BEEP_Start
*    功能说明: 启动蜂鸣音。
*    形    参: _usBeepTime : 蜂鸣时间，单位 10ms; 0 表示不鸣叫
*              _usStopTime : 停止时间，单位 10ms; 0 表示持续鸣叫
*              _usCycle : 鸣叫次数， 0 表示持续鸣叫
*    返 回 值: 无
```

```
*********************************************************************************************************
*/
void BEEP_Start(uint16_t _usBeepTime, uint16_t _usStopTime, uint16_t _usCycle)
{
    if (_usBeepTime == 0 || g_tBeep.ucMute == 1)
    {
        return;
    }

    g_tBeep.usBeepTime = _usBeepTime;
    g_tBeep.usStopTime = _usStopTime;
    g_tBeep.usCycle = _usCycle;
    g_tBeep.usCount = 0;
    g_tBeep.usCycleCount = 0;
    g_tBeep.ucState = 0;
    g_tBeep.ucEnalbe = 1;   /* 设置完全局参数后再使能发声标志 */

    BEEP_ENABLE();          /* 开始发声 */
}
```

函数描述：

此函数主要用于蜂鸣器的初始化，代码比较好理解。条件编译实现了一个无源蜂鸣器的初始化，配置引脚为推挽输出模式。由于 V7 开发板使用的无源蜂鸣器，所有没有开启宏定义 BEEP_HAVE_POWER。

函数参数：

- 第 1 个参数_usBeepTime 用于设置蜂鸣时间，单位 10ms，配置为 0 表示不鸣叫。
- 第 2 个参数_usStopTime 用于设置蜂鸣时间，单位 10ms，配置为 0 表示不鸣叫。
- 第 3 个参数_ _usCycle 用于鸣叫次数，配置为 0 表示持续鸣叫。

使用举例：

调用此函数前，务必优先调用函数 BEEP_InitHard 进行初始化。比如要实现鸣叫 50ms，停 10ms， 1 次，就是 BEEP_Start(5, 1, 1);

20.4.5 函数 BEEP_Pro

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: BEEP_Pro
*    功能说明: 每隔 10ms 调用 1 次该函数，用于控制蜂鸣器发声。该函数在 bsp_timer.c 中被调用。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void BEEP_Pro(void)
{
    if ((g_tBeep.ucEnalbe == 0) || (g_tBeep.usStopTime == 0) || (g_tBeep.ucMute == 1))
    {
        return;
    }
```

```
    if (g_tBeep.ucState == 0)
    {
        if (g_tBeep.usStopTime > 0) /* 间断发声 */
        {
            if (++g_tBeep.usCount >= g_tBeep.usBeepTime)
            {
                BEEP_DISABLE();         /* 停止发声 */
                g_tBeep.usCount = 0;
                g_tBeep.ucState = 1;
            }
        }
        else
        {
            ;    /* 不做任何处理，连续发声 */
        }
    }
    else if (g_tBeep.ucState == 1)
    {
        if (++g_tBeep.usCount >= g_tBeep.usStopTime)
        {
            /* 连续发声时，直到调用 stop 停止为止 */
            if (g_tBeep.usCycle > 0)
            {
                if (++g_tBeep.usCycleCount >= g_tBeep.usCycle)
                {
                    /* 循环次数到，停止发声 */
                    g_tBeep.ucEnalbe = 0;
                }

                if (g_tBeep.ucEnalbe == 0)
                {
                    g_tBeep.usStopTime = 0;
                    return;
                }
            }

            g_tBeep.usCount = 0;
            g_tBeep.ucState = 0;

            BEEP_ENABLE();              /* 开始发声 */
        }
    }
}
```

函数描述：

此函数是蜂鸣器的主处理函数，用于实现鸣叫时间、停止鸣叫时间和循环次数的处理。

使用举例：

调用此函数前，务必优先调用函数 BEEP_InitHard 进行初始化。

另外，此函数需要周期性调用，每 10ms 调用一次。

- 如果是裸机使用，将此函数放在 bsp.c 文件的 bsp_RunPer10ms 函数里面即可，这个函数是由滴答定时器调用的，也就是说，大家要使用蜂鸣器，定时器的初始化函数 bsp_InitTimer 一定要调用。
- 如果是 RTOS 使用，需要开启一个 10ms 为周期的任务调用函数 BEEP_Pro。

20.5 蜂鸣器驱动移植和使用

按键移植步骤如下：

- 第 1 步：复制 bsp_beep.c，bsp_beep.h，bsp_tim_pwm.c 和 bsp_tim_pwm.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：根据自己使用的蜂鸣器驱动引脚和频率，修改下面的宏定义即可

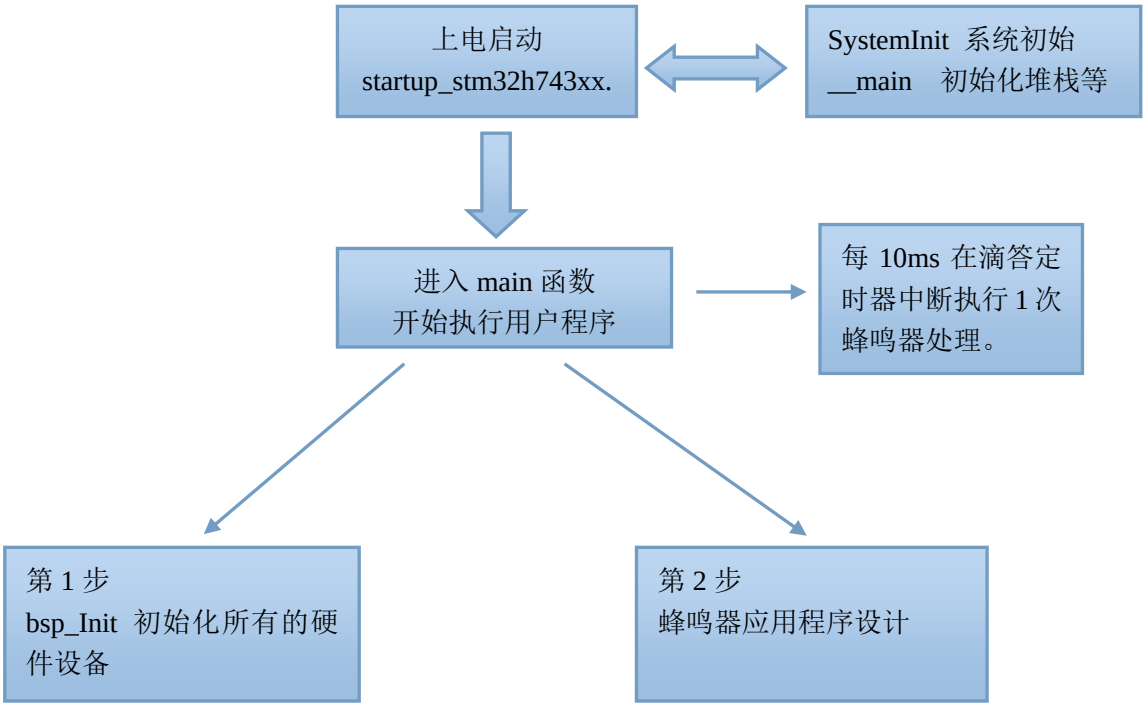
```
#ifdef    BEEP_HAVE_POWER        /* 有源蜂鸣器 */

    /* PA8 */
    #define GPIO_RCC_BEEP   RCC_AHB1Periph_GPIOA
    #define GPIO_PORT_BEEP  GPIOA
    #define GPIO_PIN_BEEP   GPIO_PIN_8

    #define BEEP_ENABLE()   GPIO_PORT_BEEP->BSRRL = GPIO_PIN_BEEP          /* 使能蜂鸣器鸣叫 */
    #define BEEP_DISABLE()  GPIO_PORT_BEEP->BSRRH = GPIO_PIN_BEEP          /* 禁止蜂鸣器鸣叫 */
#else         /* 无源蜂鸣器 */
    /* PA0 ---> TIM5_CH1 */

    /* 1500 表示频率 1.5KHz，5000 表示 50.00%的占空比 */
    #define BEEP_ENABLE()   bsp_SetTIMOutPWM(GPIOA, GPIO_PIN_0, TIM5, 1, 1500, 5000);

    /* 禁止蜂鸣器鸣叫 */
    #define BEEP_DISABLE()  bsp_SetTIMOutPWM(GPIOA, GPIO_PIN_0, TIM5, 1, 1500, 0);
#endif
```

- 第 3 步：这几个驱动文件主要用到 HAL 库的 GPIO 和 TIM 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 4 步，应用方法看本章节配套例子即可。

  特别注意，别忘了每 10ms 调用一次按键检测函数 BEEP_Pro()。

20.6 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，蜂鸣器等。
  - 第 2 部分，应用程序设计部分，实现了一个蜂鸣器应用。
  - 第 3 部分，蜂鸣器程序每 10ms 在滴答定时中断执行一次。

20.7 实验例程说明（MDK）

配套例子：

V7-003_无源蜂鸣器

实验目的：

1. 学习无源蜂鸣器的控制实现。

实验内容：

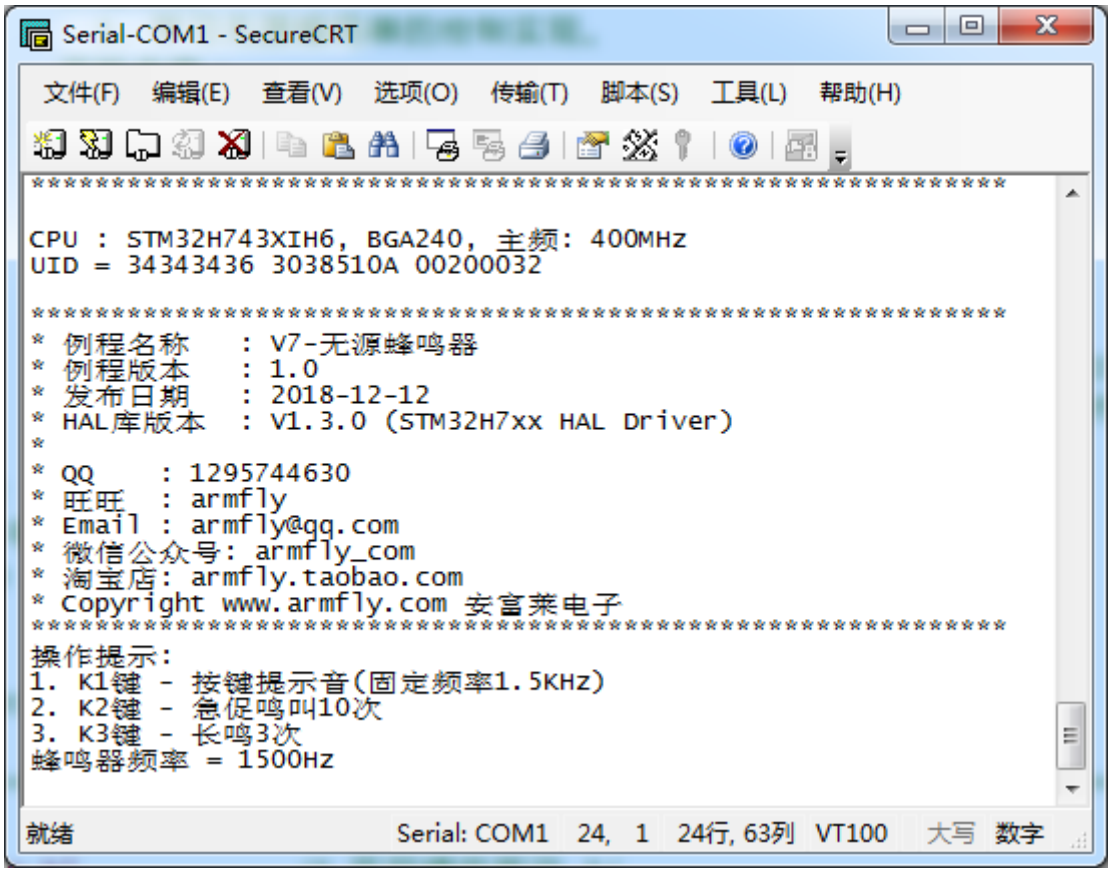
1. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。

实验操作：

1. K1 键按下，按键提示音(固定频率 1.5KHz)。
2. K2 键按下，急促鸣叫 10 次。
3. K3 键按下，长鸣 3 次。

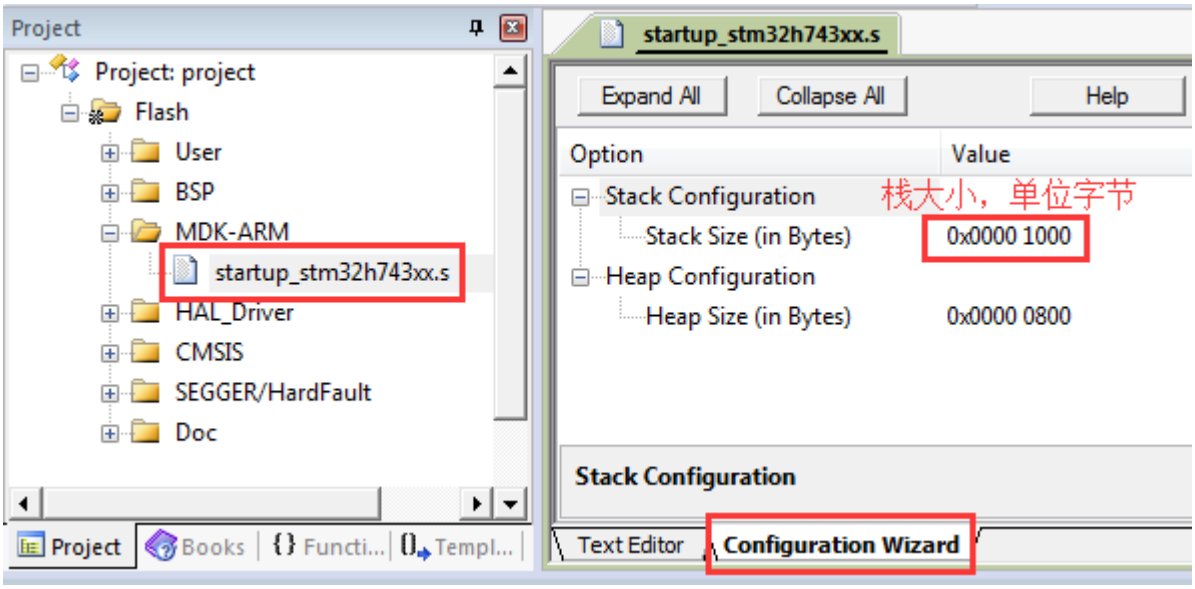
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

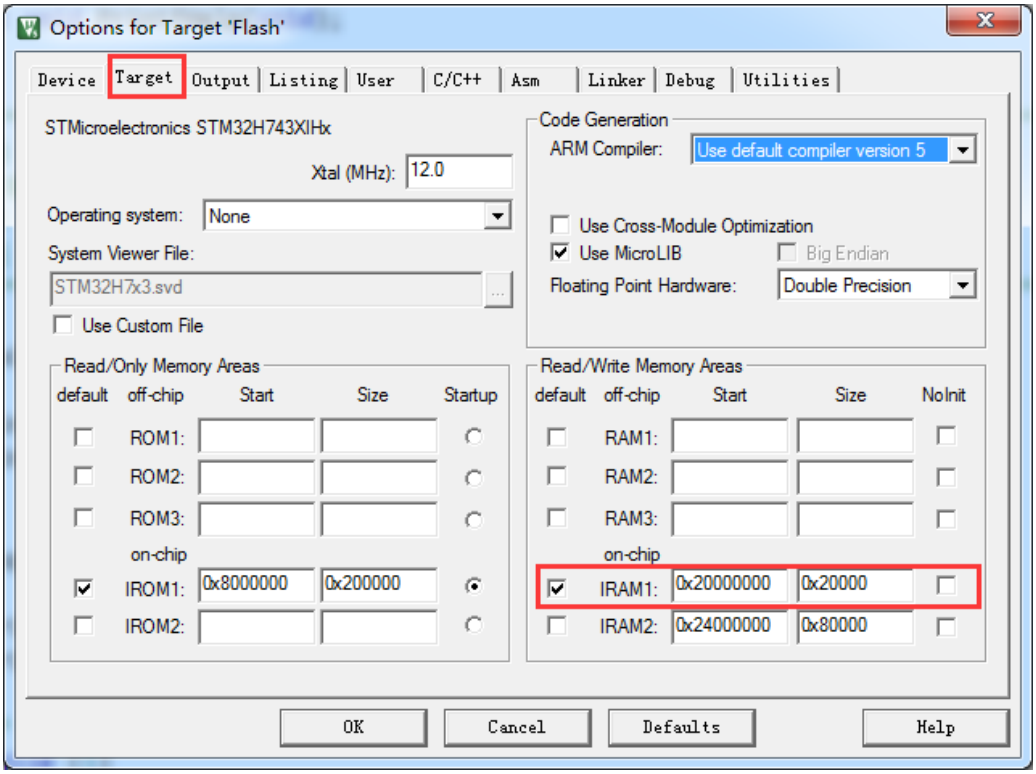


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
    BEEP_InitHard();   /* 初始化蜂鸣器 */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
```

```
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ 每 10ms 调用一次蜂鸣器处理：

蜂鸣器处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
    BEEP_Pro();
}
```

◆ 主功能：

主功能的实现主要分为两部分：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- 通过按键做蜂鸣器演示。

```
/*
*********************************************************************************************************
```

```
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;
    uint32_t freq = 1500;

    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    printf("蜂鸣器频率 = %dHz\r\n", freq);

    /* 主程序大循环 */
    while (1)
    {
        bsp_Idle();        /* CPU 空闲时执行的函数，在 bsp.c */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 处理按键事件 */
        ucKeyCode = bsp_GetKey();
        if (ucKeyCode > 0)
        {
            /* 有键按下 */
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:         /* K1 按键按下，提示音 */
                    BEEP_KeyTone();
                    printf("1 按键按下，提示音(固定频率 1.5KHz)\r\n");
                    break;

                case KEY_DOWN_K2:         /* K2 按键按下，急促鸣叫 10 次*/
                    BEEP_Start(5, 5, 10); /* 鸣叫 50ms，停止 50ms，10 次*/
                    printf("K2 按键按下，急促鸣叫 10 次\r\n");
                    break;

                case KEY_DOWN_K3:          /* K3 按键按下，长鸣 3 次*/
                    BEEP_Start(50, 50, 3); /* 鸣叫 500ms，停止 500ms，3 次*/
                    printf("K3 按键按下，长鸣 3 次\r\n");
                    break;

                default:
                    break;
            }
        }
```

```
    }
}
```

20.8 实验例程说明（IAR）

配套例子：

V7-003_无源蜂鸣器

实验目的：

1. 学习无源蜂鸣器的控制实现。

实验内容：

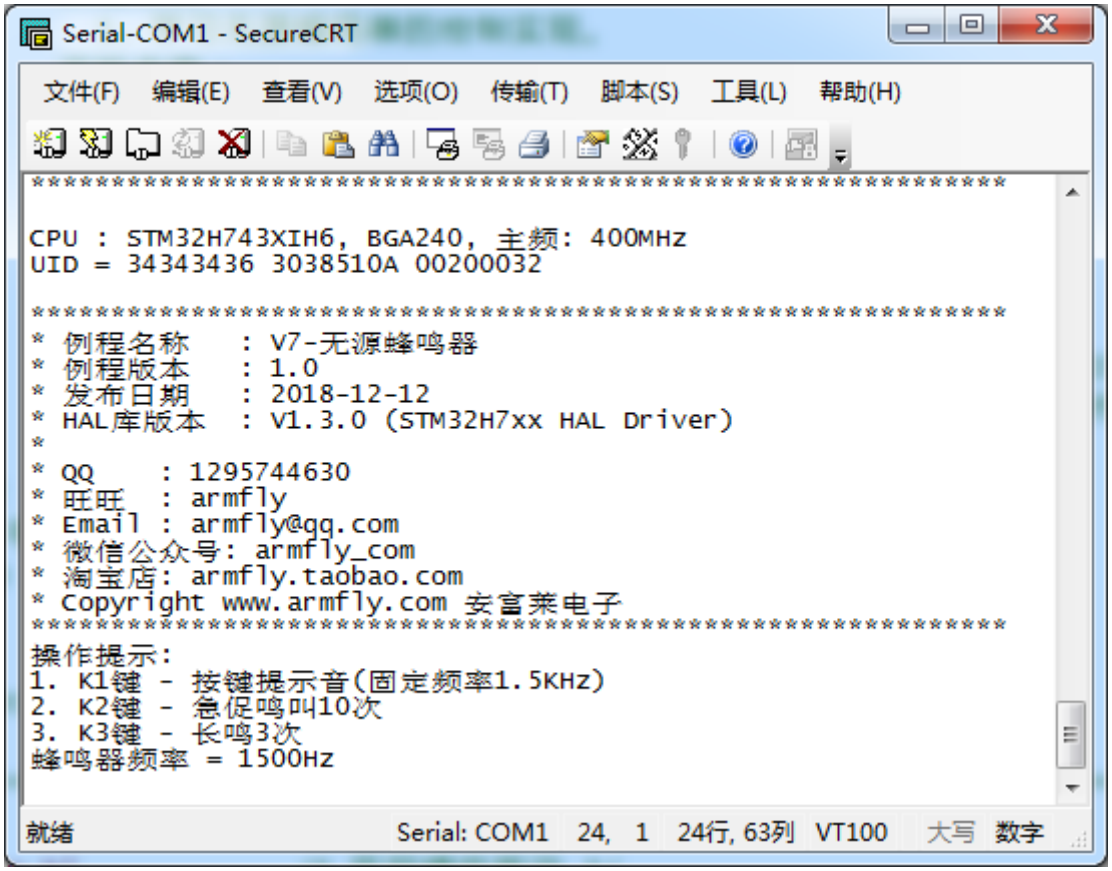
2. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。

实验操作：

1. K1 键按下，按键提示音(固定频率 1.5KHz)。
2. K2 键按下，急促鸣叫 10 次。
3. K3 键按下，长鸣 3 次。

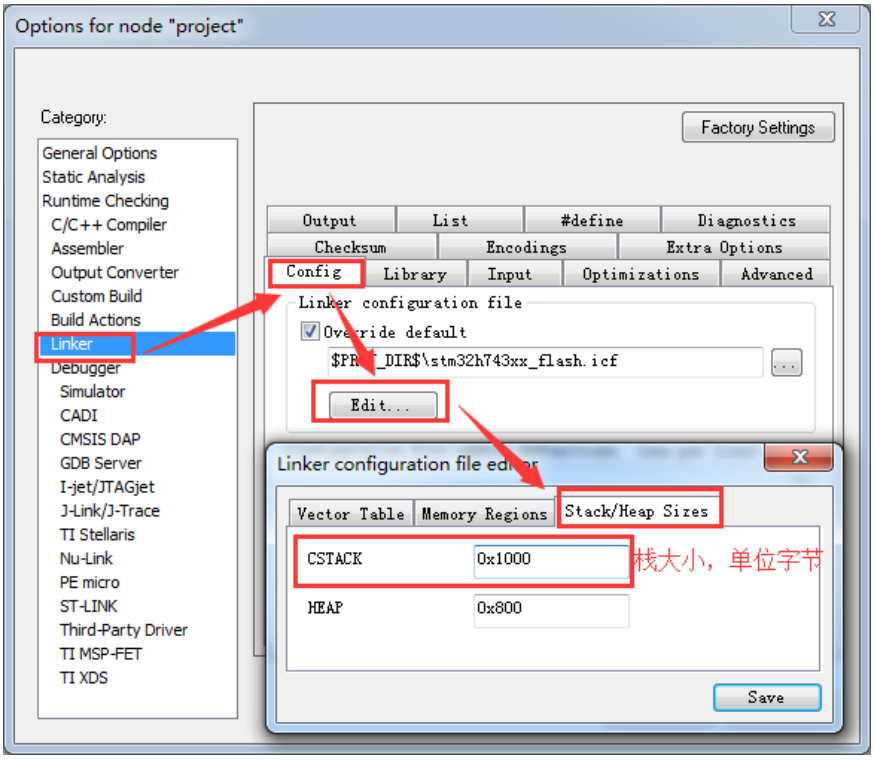
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

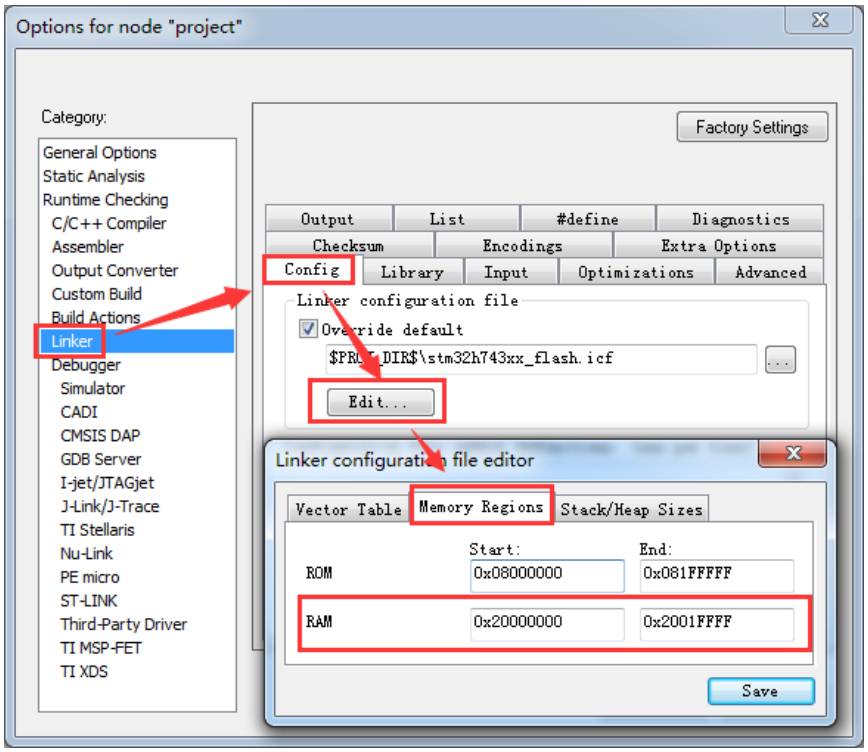


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
```

```
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
    BEEP_InitHard();   /* 初始化蜂鸣器 */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
```

```
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **每 10ms 调用一次蜂鸣器处理：**

蜂鸣器处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
    BEEP_Pro();
}
```

◆ **主功能：**

主功能的实现主要分为两部分：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- 通过按键做蜂鸣器演示。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;
    uint32_t freq = 1500;

    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    printf("蜂鸣器频率 = %dHz\r\n", freq);

    /* 主程序大循环 */
    while (1)
    {
        bsp_Idle();        /* CPU 空闲时执行的函数，在 bsp.c */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }
```

```
        /* 处理按键事件 */
        ucKeyCode = bsp_GetKey();
        if (ucKeyCode > 0)
        {
            /* 有键按下 */
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:         /* K1 按键按下，提示音 */
                    BEEP_KeyTone();
                    printf("1 按键按下，提示音(固定频率 1.5KHz)\r\n");
                    break;

                case KEY_DOWN_K2:         /* K2 按键按下，急促鸣叫 10 次*/
                    BEEP_Start(5, 5, 10); /* 鸣叫 50ms，停止 50ms，10 次*/
                    printf("K2 按键按下，急促鸣叫 10 次\r\n");
                    break;

                case KEY_DOWN_K3:          /* K3 按键按下，长鸣 3 次*/
                    BEEP_Start(50, 50, 3); /* 鸣叫 500ms，停止 500ms，3 次*/
                    printf("K3 按键按下，长鸣 3 次\r\n");
                    break;

                default:
                    break;
            }
        }
    }
}
```

20.9 总结

本章节为大家介绍的无源蜂鸣器方案还是比较实用的，采用的非阻塞方式，实际项目中可以放心使用。

第21章 STM32H7 的 NVIC 中断分组和配置（重要）

本章节为大家讲解 STM32H7 的中断优先级配置。此章节非常重要，初学者经常在这里犯迷糊，因此很有必要整明白。

21.1 NVIC 基础知识

21.2 常用的 NVIC API

21.3 MDK 中查看实际优先级配置

21.4 开关中断

21.5 总结

21.1 NVIC 基础知识

NVIC 的全称是 Nested vectored interrupt controller，即嵌套向量中断控制器。

对于 M3/M4/M7 内核的 MCU，每个中断的优先级都是用寄存器中的 8 位来设置的。8 位的话就可以设置 2^8 = 256 级中断，实际中用不了这么多，所以芯片厂商根据自己生产的芯片做出了调整。比如 ST 的 STM32F1xx，F4xx 和 H7 只使用了这个 8 位中的高四位[7:4]，低四位取零，这样 2^4=16，只能表示 16 级中断嵌套。

对于这个 NVIC，有个重要的知识点就是优先级分组、抢占优先级和子优先级。STM32F1xx，F4xx 和 H7 都是只使用了这个 8 位寄存器的高四位[7:4]。

优先级分组	抢占优先级	子优先级	高 4 位使用情况描述
NVIC_PriorityGroup_0	0 级抢占优先级	0-15 级子优先级	0bit 用于抢占优先级 4bit 全用于子优先级
NVIC_PriorityGroup_1	0-1 级抢占优先级	0-7 级子优先级	1bit 用于抢占优先级 3bit 用于子优先级
NVIC_PriorityGroup_2	0-3 级抢占优先级	0-3 级子优先级	2bit 用于抢占优先级 2bit 用于子优先级
NVIC_PriorityGroup_3	0-7 级抢占优先级	0-1 级子优先级	3bit 用于抢占优先级 1bit 用于子优先级

NVIC_PriorityGroup_4	0-15 级抢占优先级	0 级子优先级	4bit 全用于抢占优先级 0bit 用于子优先级

从上面的表格可以看出,STM32 支持 5 种优先级分组。系统上电复位后,默认使用的是优先级分组 0,也就是没有抢占式优先级，只有子优先级。关于这个抢占优先级和子优先级有几点一定要说清楚。

- 具有高抢占式优先级的中断可以在具有低抢占式优先级的中断服务程序执行过程中被响应，即中断嵌套，或者说高抢占式优先级的中断可以抢占低抢占式优先级的中断的执行。
- 在抢占式优先级相同的情况下，有几个子优先级不同的中断同时到来，那么高子优先级的中断优先被响应。
- 在抢占式优先级相同的情况下，如果有低子优先级中断正在执行，高子优先级的中断要等待已被响应的低子优先级中断执行结束后才能得到响应，即子优先级不支持中断嵌套。
- Reset、NMI、Hard Fault 优先级为负数，高于普通中断优先级，且优先级不可配置。
- 初学者还有一个比较纠结的问题，就是系统中断（比如：PendSV，SVC，SysTick）是不是一定比外部中断（比如 SPI,USART）要高。答案：不是的，它们是在同一个 NVIC 下面设置的。

掌握了这些基础知识基本就够用了。另外特别注意一点，配置抢占优先级和子优先级，它们合并成的 4bit 数字的数值越小，优先级越高，这一点千万不要搞错了。

21.2 常用的 NVIC API

下面将外设配置中经常用到的两个函数做个说明。

- HAL_NVIC_SetPriority
- HAL_NVIC_EnableIRQ

21.2.1 函数 HAL_NVIC_SetPriority

函数原型：

```
void HAL_NVIC_SetPriority(IRQn_Type IRQn, uint32_t PreemptPriority, uint32_t SubPriority)
```

函数描述：

此函数主要用于设置中断的抢占优先级和子优先级。

函数参数：

- 第 1 个参数 IRQn 是中断号，由 IRQn_Type 定义的枚举类型，原始定义在 stm32h743x.h。

```
typedef enum
{
  /* 部分定义，其它省略未写 */
  PendSV_IRQn                 = -2,
  SysTick_IRQn                = -1,
```

```
  DMA1_Stream3_IRQn           = 14,
  DMA1_Stream4_IRQn           = 15,
  DMA1_Stream5_IRQn           = 16,
  DMA1_Stream6_IRQn           = 17,
  ADC_IRQn                    = 18,
} IRQn_Type;
```

- 第 2 个参数 PreemptPriority 用于设置抢占优先级，范围 0 - 15。
- 第 3 个参数 SubPriority 用于设置子优先级，范围 0 -15。

使用举例：

此函数的调用比较简单，比如设置 ADC 中断的抢占优先级是 1，子优先级是 0，那么此函数的设置就是：HAL_NVIC_SetPriority(ADC_IRQn, 1, 0)。

21.2.2 函数 HAL_NVIC_EnableIRQ

函数原型：

```
void HAL_NVIC_EnableIRQ(IRQn_Type IRQn)
```

函数描述：

此函数主要用于使能中断。

函数参数：

- 第 1 个参数 IRQn 是中断号，由 IRQn_Type 定义的枚举类型，原始定义在 stm32h743x.h。

```
typedef enum
{
  /* 部分定义，其它省略未写 */
  PendSV_IRQn                 = -2,
  SysTick_IRQn                = -1,
  DMA1_Stream3_IRQn           = 14,
  DMA1_Stream4_IRQn           = 15,
  DMA1_Stream5_IRQn           = 16,
  DMA1_Stream6_IRQn           = 17,
  ADC_IRQn                    = 18,
} IRQn_Type;
```

注意事项：

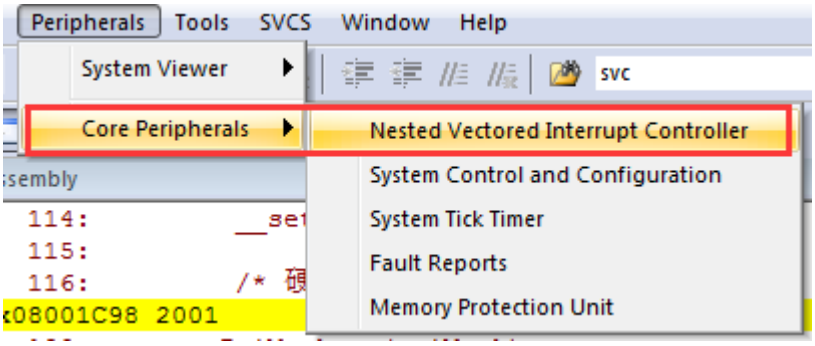
调用此函数前要先调用优先级分组设置函数 NVIC_PriorityGroupConfig，而这个函数会在 HAL_Init 里面被调用。函数基本都会在 main 函数里面优先被调用，所以保证 HAL_Init 优先被调用即可。

使用举例：

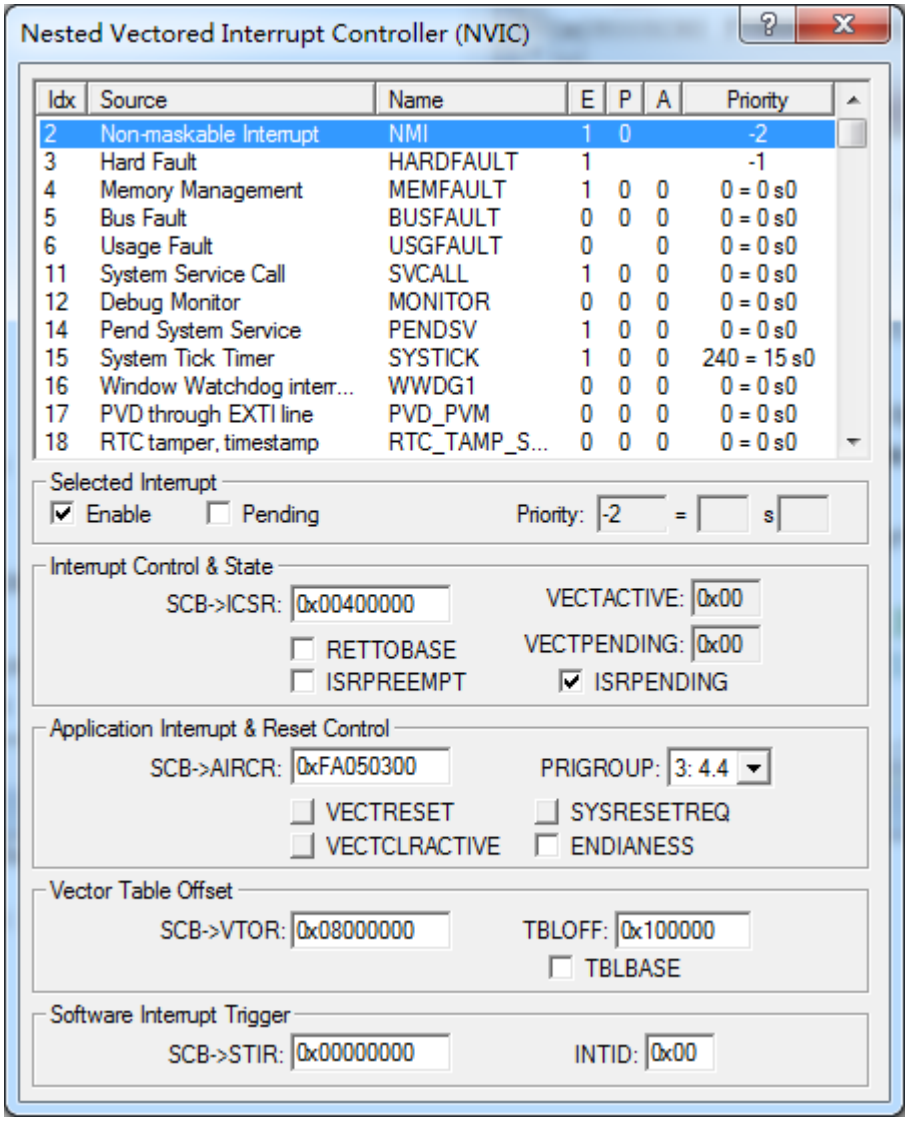
此函数的调用比较省事，比如使能 ADC 中断，那么此函数的设置就是：
HAL_NVIC_EnableIRQ (ADC_IRQn,)。

21.3 MDK 中查看实际优先级配置

如果想查看配置的外设优先级是否正确设置了，MDK 进入调试状态后，先点击全速运行，然后查看如下调试组件：



弹出如下窗口，在这个窗口里面就可以查看实际优先级了，比如滴答定时器 Systick 的优先级就是 240，即 240>>4 = 15。



21.4开关中断

V7 开发板的 BSP 驱动包在 bsp.h 文件里面对开关中断做了一个宏定义：

```
/* 开关全局中断的宏 */
#define ENABLE_INT()    __set_PRIMASK(0)   /* 使能全局中断 */
```

```
#define DISABLE_INT()   __set_PRIMASK(1)   /* 禁止全局中断 */
```

__set_PRIMASK 就是对寄存器 primask 做的开关设置，如下表所示：

名字	功能描述
primask	这是个只有 1 个 bit 的寄存器。 在它被置 1 后，就关掉所有可屏蔽的异常，只剩下 NMI 和硬 fault 可以响应。它的缺省值是 0，表示没有关中断。
faultmask	这是个只有 1 个 bit 的寄存器。当它置 1 时，只有 NMI 才能响应，所有其它的异常，甚至是硬 fault，也通通闭嘴。它的缺省值也是 0，表示没有关异常。
basepri	这个寄存器最多有 9 位 (由表达优先级的位数决定)。它定义了被屏蔽优先级的阈值。当它被设成某个值后，所有优先级号大于等于此值的中断都被关（优先级号越大，优先级越低)。但若被设成 0，则不关闭任何中断， 0 也是缺省值。

对于寄存器 basepri 我们举一个例子，帮助大家理解。比我们配置寄存器 basepri（对于 STM32H7，范围 0-255）的数值为 16，所有优先级数值大于等于 16 的中断都会被关闭，优先级数值小于 16 的中断不会被关闭。但 0 比较特殊，对寄存器 basepri 寄存器赋值 0，那么被关闭的中断会被打开。

21.5 总结

最后，我们再强调一下，在 NVIC 分组为 4 的情况下，抢占优先级可配置范围是 0-15，那么数值越小，抢占优先级的级别越高，即 0 代表最高优先级，15 代表最低优先级。

第22章 STM32H7 的 SysTick 实现多组软件定时器

本章节为大家讲解嘀嗒定时器 SysTick，嘀嗒定时器比较容易掌握，其实大家只要知道它是一个 24 位的递减计数器，支持中断就可以了。

22.1 初学者重要提示
22.2 Systick 基础知识
22.3 多组软件定时器驱动设计
22.4 多组软件定时器板级支持包（bsp_timer.c）
22.6 多组软件定时器驱动移植和使用
22.7 实验例程设计框架
22.8 实验例程说明（MDK）
22.9 实验例程说明（IAR）
22.10 总结

22.1 初学者重要提示

- 比通用定时器要容易掌握很多，因为嘀嗒定时器的功能比较的单一，根据 ARM 的说法，此定时器就是专门为 RTOS 的系统时钟节拍而设计。
- 本章节为大家讲解的多组软件定时器实现方案非常实用，建议初学者熟练掌握。

22.2 Systick 基础知识

关于滴答定时器，初学者仅需了解到以下几点知识就够了。

- Systick 是 Cortex-M7 内核自带的组件，其它几个常用的硬件异常 HardFault，SVC 和 PendSV 也都是是内核自带的，其中 Systick，SVC 和 PendSV 的中断优先级是可编程的，跟 SPI 中断、ADC 中断、UART 中断等一样，都在同一个 NVIC 下配置的。而 HardFault 是不可编程的，且优先级要比可编程的都要高。
- Systick 是一个 24 位的递减计数器，用户仅需掌握 ARM 的 CMSIS 软件提供的一个函数 SysTick_Config 即可，原代码如下：

```
1.  /**
2.    \brief   System Tick Configuration
```

```
3.     \details Initializes the System Timer and its interrupt, and starts the System Tick Timer.
4.              Counter is in free running mode to generate periodic interrupts.
5.     \param [in]  ticks  Number of ticks between two interrupts.
6.     \return          0  Function succeeded.
7.     \return          1  Function failed.
8.     \note    When the variable <b>__Vendor_SysTickConfig</b> is set to 1, then the
9.              function <b>SysTick_Config</b> is not included. In this case, the file <b><i>device</i>.h</b>
10.             must contain a vendor-specific implementation of this function.
11.   */
12.  __STATIC_INLINE uint32_t SysTick_Config(uint32_t ticks)
13.  {
14.    if ((ticks - 1UL) > SysTick_LOAD_RELOAD_Msk)
15.    {
16.      return (1UL);                                           /* Reload value impossible */
17.    }
18.
19.    SysTick->LOAD  = (uint32_t)(ticks - 1UL);                 /* set reload register */
20.    NVIC_SetPriority (SysTick_IRQn, (1UL << __NVIC_PRIO_BITS) - 1UL)/*set Priority for Systick Interrupt */
21.    SysTick->VAL   = 0UL;                                     /* Load the SysTick Counter Value */
22.    SysTick->CTRL  = SysTick_CTRL_CLKSOURCE_Msk |
23.                     SysTick_CTRL_TICKINT_Msk   |
24.                     SysTick_CTRL_ENABLE_Msk;                /* Enable SysTick IRQ and SysTick Timer */
25.    return (0UL);                                             /* Function successful */
26.  }
```

- 第 12 行,函数的形参用于配置滴答定时器 LOAD 寄存器的数值,由于滴答定时器是一个递减计数器,启动后是将 LOAD 寄存器的数值赋给 VAL 寄存器，然后 VAL 寄存器做递减操作，等递减到 0 的时候重新加载 LOAD 寄存器的数值继续做递减操作。

  函数的形参表示内核时钟多少个周期后触发一次 Systick 定时中断，比如形参配置为如下数值。

  -- SystemCoreClock / 1000　表示定时频率为 1000Hz，　也就是定时周期为　1ms。

  -- SystemCoreClock / 500　表示定时频率为 500Hz，　也就是定时周期为　2ms。

  -- SystemCoreClock / 2000　表示定时频率为 2000Hz，　也就是定时周期为　500us。

  注：SystemCoreClock 是 STM32H7 的系统主频 400MHz。

- 第 20 行，此函数设置滴答定时器为最低优先级。
- 第 22 行，配置滴答定时器的控制寄存器，使能滴答定时器中断。滴答定时器的中断服务程序实现比较简单，没有清除中断标志这样的操作，仅需填写用户要实现的功能即可。

控制及其状态寄存器的位定义：

位段	名称	类型	复位值	描述
16	COUNTFLAG	R	0	如果在上次读取本寄存器后，SysTick 已经计到了 0，则该位为 1。如果读取该位，该位将自动清零
2	CLKSOURCE	R/W	0	0=外部时钟源(STCLK) 1=内核时钟(FCLK)
1	TICKINT	R/W	0	1=SysTick 倒数计数到 0 时产生 SysTick 异常请求 0=数到 0 时无动作
0	ENABLE	R/W	0	SysTick 定时器的使能位

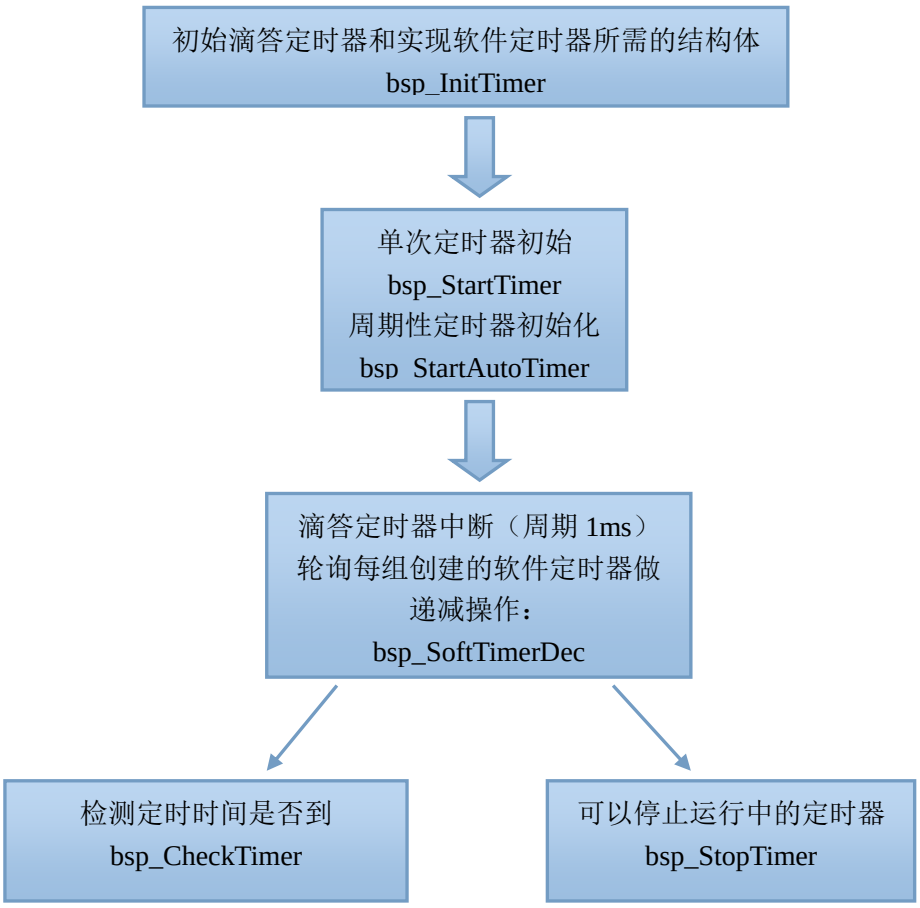
重装寄存器定义，最大值 2^24 – 1 = 16777215，配置的时候注意别超出范围了。

位段	名称	类型	复位值	描述
23:0	RELOAD	R/W	0	当倒数计数至零时，将被重装载的值

22.3 多组软定时器驱动设计

22.3.1 软件定时器框架

为了方便大家理解，先来看下软件定时器的实现框图：



- 第 1 阶段，初始化：

  通过函数 bsp_InitTimer 初始化滴答定时器和实现软件定时器所需的结构体。

- 第 2 阶段，软件定时器初始化：
  - 可以通过函数 bsp_StartTimer 做单次定时器初始化，单次定时器执行一次就结束。下次还想使用，需要重新创建。
  - 可以通过函数 bsp_StartAutoTimer 做周期性定时器初始化，可以周期性的一直运行下去。
- 第 3 阶段，滴答定时器中断里面更新每个软件定时器的计数：

在滴答定时器中断里面通过调用函数 bsp_SoftTimerDec 实现每个软件定时器的计数更新。

- 第 4 阶段，检测时间到和停止运行
  - 通过函数 bsp_CheckTimer 可以检测单次或者周期定时器的时间是否到。时间到后就可以执行用户任务。
  - 如果不想某个单次或者周期性定时器执行，直接调用函数 bsp_StopTimer 停止即可。

22.3.2 程序分析之相关的变量定义

在 bsp_timer.h 中定义了结构体类型 SOFT_TMR。

```
#define TMR_COUNT  4              /* 软件定时器的个数 （定时器 ID 范围 0 - 3) */

typedef enum
{
    TMR_ONCE_MODE = 0,      /* 一次工作模式 */
    TMR_AUTO_MODE = 1       /* 自动定时工作模式 */
}TMR_MODE_E;

/* 定时器结构体，成员变量必须增加__IO 即 volatile，因为这个变量在中断和主程序中同时被访问，
   有可能造成编译器错误优化。
*/
typedef struct
{
    volatile uint8_t Mode;      /* 计数器模式，1 次性 */
    volatile uint8_t Flag;      /* 定时到达标志  */
    volatile uint32_t Count;    /* 计数器 */
    volatile uint32_t PreLoad;  /* 计数器预装值 */
}SOFT_TMR;
```

在 bsp_timer.c 中定义 SOFT_TMR 结构体数组变量。

```
/* 定于软件定时器结构体变量 */
static SOFT_TMR s_tTmr[TMR_COUNT];
```

每个软件定时器对象都分配一个结构体变量，这些结构体变量以数组的形式存在将便于我们简化程序代码行数。

22.3.3 程序分析之初始化

初始化函数如下：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: bsp_InitTimer
4.  *    功能说明: 配置 systick 中断，并初始化软件定时器变量
5.  *    形    参:  无
6.  *    返 回 值: 无
7.  *********************************************************************************************************
8.  */
9.  void bsp_InitTimer(void)
10. {
```

```
11.       uint8_t i;
12.
13.       /* 清零所有的软件定时器 */
14.       for (i = 0; i < TMR_COUNT; i++)
15.       {
16.            s_tTmr[i].Count = 0;
17.            s_tTmr[i].PreLoad = 0;
18.            s_tTmr[i].Flag = 0;
19.            s_tTmr[i].Mode = TMR_ONCE_MODE;  /* 缺省是 1 次性工作模式 */
20.       }
21.
22.       /*
23.            配置 systic 中断周期为 1ms，并启动 systick 中断。
24.
25.       SystemCoreClock 是固件中定义的系统内核时钟，对于 STM32H7,一般为 400MHz
26.
27.       SysTick_Config() 函数的形参表示内核时钟多少个周期后触发一次 Systick 定时中断.
28.            -- SystemCoreClock / 1000   表示定时频率为 1000Hz,  也就是定时周期为  1ms
29.            -- SystemCoreClock / 500    表示定时频率为 500Hz,   也就是定时周期为  2ms
30.            -- SystemCoreClock / 2000   表示定时频率为 2000Hz,  也就是定时周期为  500us
31.
32.       对于常规的应用，我们一般取定时周期 1ms。对于低速 CPU 或者低功耗应用，可以设置定时周期为 10ms
33.      */
34.       SysTick_Config(SystemCoreClock / 1000);
35.  }
```

- 第 14-20 行是软件定时器结构体的初始化部分，设置初始值。实际创建软件定时器会重新做初始化。
- 第 32 行是本章 22.2 小节已经讲解。

22.3.4程序分析之单次定时器创建

单次定时器创建函数 bsp_StartTime。

```
1.   /*
2.   *********************************************************************************************************
3.   *    函 数 名: bsp_StartTimer
4.   *    功能说明: 启动一个定时器，并设置定时周期。
5.   *    形    参:  _id   : 定时器 ID，值域【0,TMR_COUNT-1】。用户必须自行维护定时器 ID，以避免定时器 ID 冲突。
6.   *              _period : 定时周期，单位 1ms
7.   *    返 回 值: 无
8.   *********************************************************************************************************
9.   */
10.  void bsp_StartTimer(uint8_t _id, uint32_t _period)
11.  {
12.       if (_id >= TMR_COUNT)
13.       {
14.            /* 打印出错的源代码文件名、函数名称 */
15.            BSP_Printf("Error: file %s, function %s()\r\n", __FILE__, __FUNCTION__);
16.            while(1); /* 参数异常，死机等待看门狗复位 */
17.       }
18.
19.       DISABLE_INT();                  /* 关中断 */
20.
21.       s_tTmr[_id].Count = _period;          /* 实时计数器初值 */
22.       s_tTmr[_id].PreLoad = _period;        /* 计数器自动重装值，仅自动模式起作用 */
23.       s_tTmr[_id].Flag = 0;                 /* 定时时间到标志 */
```

```
24.       s_tTmr[_id].Mode = TMR_ONCE_MODE;     /* 1 次性工作模式 */
25.
26.       ENABLE_INT();                         /* 开中断 */
27.   }
```

- 第 12-17 行是为了防止用户设置的 ID 参数超过范围。

  其中 BSP_Printf 是在 bsp.h 文件定义的，用于调试阶段排错。

  #define BSP_Printf   printf   /* 使用这个宏定义的话，正常执行 printf */

  #define BSP_Printf(...)   /* 如果使用这个宏定义的话，什么都不执行 */

- 第 19-26 行是临界段，结构体变量赋值前后做了开关中断操作。因为此结构体变量在滴答定时器中断里面也要调用，防止变量赋值出问题。

  开关中断函数也是在 bsp.h 文件里面定义的。

  #define ENABLE_INT()   __set_PRIMASK(0)   /* 使能全局中断 */

  #define DISABLE_INT()   __set_PRIMASK(1)   /* 禁止全局中断 */

22.3.5 程序分析之周期性定时器创建

周期性定时器创建函数 bsp_StartAutoTimer。

```
1.   /*
2.   *********************************************************************************************************
3.   *    函 数 名: bsp_StartAutoTimer
4.   *    功能说明: 启动一个自动定时器，并设置定时周期。
5.   *    形    参:  _id   : 定时器 ID，值域【0,TMR_COUNT-1】。用户必须自行维护定时器 ID，以避免定时器 ID 冲突。
6.   *              _period : 定时周期，单位 10ms
7.   *    返 回 值: 无
8.   *********************************************************************************************************
9.   */
10.  void bsp_StartAutoTimer(uint8_t _id, uint32_t _period)
11.  {
12.      if (_id >= TMR_COUNT)
13.      {
14.          /* 打印出错的源代码文件名、函数名称 */
15.          BSP_Printf("Error: file %s, function %s()\r\n", __FILE__, __FUNCTION__);
16.          while(1); /* 参数异常，死机等待看门狗复位 */
17.      }
18.
19.      DISABLE_INT();          /* 关中断 */
20.
21.      s_tTmr[_id].Count = _period;          /* 实时计数器初值 */
22.      s_tTmr[_id].PreLoad = _period;        /* 计数器自动重装值，仅自动模式起作用 */
23.      s_tTmr[_id].Flag = 0;                 /* 定时时间到标志 */
24.      s_tTmr[_id].Mode = TMR_AUTO_MODE;     /* 自动工作模式 */
25.
26.      ENABLE_INT();                /* 开中断 */
27.  }
```

- 这个函数跟前面 22.3.4 小节中讲的单次定时器是一样的，仅第 24 行的赋值不同，这个函数是周期性的，而 22.3.4 小节里面的是单次定时器。

22.3.6程序分析之停止定时器运行

定时器停止运行函数是 bsp_StopTimer。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_StopTimer
*    功能说明: 停止一个定时器
*    形    参: _id   : 定时器 ID，值域【0,TMR_COUNT-1】。用户必须自行维护定时器 ID，以避免定时器 ID 冲突。
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_StopTimer(uint8_t _id)
{
    if (_id >= TMR_COUNT)
    {
        /* 打印出错的源代码文件名、函数名称 */
        BSP_Printf("Error: file %s, function %s()\r\n", __FILE__, __FUNCTION__);
        while(1); /* 参数异常，死机等待看门狗复位 */
    }

    DISABLE_INT();     /* 关中断 */

    s_tTmr[_id].Count = 0;               /* 实时计数器初值 */
    s_tTmr[_id].Flag = 0;                /* 定时时间到标志 */
    s_tTmr[_id].Mode = TMR_ONCE_MODE;    /* 自动工作模式 */

    ENABLE_INT();          /* 开中断 */
}
```

- 这个函数跟前面 22.3.4 和 22.3.5 小节中的函数框架一样，仅是把结构体变量中的计数器和时间到标志都置位成 0，从而让软件定时器停止运行。

22.3.7程序分析之检测定时器时间到

检测定时器时间到的函数是 bsp_CheckTimer。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_CheckTimer
*    功能说明: 检测定时器是否超时
*    形    参:  _id   : 定时器 ID，值域【0,TMR_COUNT-1】。用户必须自行维护定时器 ID，以避免定时器 ID 冲突。
*              _period : 定时周期，单位 1ms
*    返 回 值: 返回 0 表示定时未到，  1 表示定时到
*********************************************************************************************************
*/
uint8_t bsp_CheckTimer(uint8_t _id)
{
    if (_id >= TMR_COUNT)
    {
        return 0;
    }

    if (s_tTmr[_id].Flag == 1)
```

```
58.       {
59.           s_tTmr[_id].Flag = 0;
60.           return 1;
61.       }
62.       else
63.       {
64.           return 0;
65.       }
66.   }
```

- 第 12 到 15 行是检查 ID 是否有效。
- 第 17 到 25 行是判断时间到标志值 Flag 是否置位，如果置位表示时间已经到，如果为 0，表示时间还没有到。

22.3.8程序分析之滴答定时器中断的处理

软件定时器的主要功能是通过滴答定时器中断实现的，函数的调用关系是滴答定时器中断函数 SysTick_Handler 调用 SysTick_ISR，而 SysTick_ISR 调用 bsp_SoftTimerDec。

```
1.   /*
2.   *********************************************************************************************************
3.   *    函 数 名: SysTick_Handler
4.   *    功能说明: 系统嘀嗒定时器中断服务程序。启动文件中引用了该函数。
5.   *    形    参:  无
6.   *    返 回 值: 无
7.   *********************************************************************************************************
8.   */
9.   void SysTick_Handler(void)
10.  {
11.      SysTick_ISR();
12.  }
13.
14.  /*
15.  *********************************************************************************************************
16.  *    函 数 名: SysTick_ISR
17.  *    功能说明: SysTick 中断服务程序，每隔 1ms 进入 1 次
18.  *    形    参:  无
19.  *    返 回 值: 无
20.  *********************************************************************************************************
21.  */
22.  void SysTick_ISR(void)
23.  {
24.      static uint8_t s_count = 0;
25.      uint8_t i;
26.
27.      HAL_IncTick();
28.
29.      /* 每隔 1ms 进来 1 次 （仅用于 bsp_DelayMS） */
30.      if (s_uiDelayCount > 0)
31.      {
32.          if (--s_uiDelayCount == 0)
33.          {
34.              s_ucTimeOutFlag = 1;
35.          }
```

```
36.       }
37.
38.       /* 每隔 1ms，对软件定时器的计数器进行减一操作 */
39.       for (i = 0; i < TMR_COUNT; i++)
40.       {
41.           bsp_SoftTimerDec(&s_tTmr[i]);
42.       }
43.
44.       /* 全局运行时间每 1ms 增 1 */
45.       g_iRunTime++;
46.       if (g_iRunTime == 0x7FFFFFFF)    /* 这个变量是 int32_t 类型，最大数为 0x7FFFFFFF */
47.       {
48.           g_iRunTime = 0;
49.       }
50.
51.       bsp_RunPer1ms();        /* 每隔 1ms 调用一次此函数，此函数在 bsp.c */
52.
53.       if (++s_count >= 10)
54.       {
55.           s_count = 0;
56.
57.           bsp_RunPer10ms();  /* 每隔 10ms 调用一次此函数，此函数在 bsp.c */
58.       }
59.   }
60.
61.   /*
62.   *********************************************************************************************************
63.   *    函 数 名: bsp_SoftTimerDec
64.   *    功能说明: 每隔 1ms 对所有定时器变量减 1。必须被 SysTick_ISR 周期性调用。
65.   *    形    参:  _tmr : 定时器变量指针
66.   *    返 回 值: 无
67.   *********************************************************************************************************
68.   */
69.   static void bsp_SoftTimerDec(SOFT_TMR *_tmr)
70.   {
71.       if (_tmr->Count > 0)
72.       {
73.           /* 如果定时器变量减到 1 则设置定时器到达标志 */
74.           if (--_tmr->Count == 0)
75.           {
76.               _tmr->Flag = 1;
77.
78.               /* 如果是自动模式，则自动重装计数器 */
79.               if(_tmr->Mode == TMR_AUTO_MODE)
80.               {
81.                   _tmr->Count = _tmr->PreLoad;
82.               }
83.           }
84.       }
85.   }
```

- 第 39 到 42 行是实现的关键，滴答定时器中断会每毫秒执行一次，依次扫描所有需要创建的软件定时器。软件定时器个数由 bsp_timer.h 文件中定义的 TMR_COUNT 决定。
- 第 69 到 85 行是软件定时器的实际处理函数，首先判断计数器_tmr->Count 的数值是否为 0，如果不为 0，则减 1，直到计数器的数值为 0。计数器的数值达到 0 后设置定时器到达标志_tmr->Flag。

如果是周期性定时器，将计数器_tmr->Count 设置为初始化时配置的预装值_tmr->PreLoad，这样就能以_tmr->PreLoad 为周期进行计数，从而实现周期性定时器功能。

22.4 多组软件定时器板级支持包（bsp_timer.c)

滴答定时器驱动文件 bsp_timer.c 主要实现了如下几个 API：

- bsp_InitTimer
- SysTick_ISR
- bsp_SoftTimerDec
- bsp_DelayMS
- bsp_DelayUS
- bsp_StartTimer
- bsp_StartAutoTimer
- bsp_StopTimer
- bsp_CheckTimer
- bsp_GetRunTime
- bsp_CheckRunTime
- SysTick_Handler

软件定时器涉及到的几个函数在本章的 22.3 小节都进行了详细讲解，本小节主要是把需要用户调用的五个函数做个应用说明。

22.4.1 函数 bsp_InitTimer

函数原型：

```
void bsp_InitTimer(void)
```

函数描述：

此函数主要用于软件定时器的初始化，使用所有其它 API 之前，务必优先调用此函数。

注意事项：

此函数的解读在本章 20.3.3 小节。

使用举例：

软件定时器的初始化函数在 bsp.c 文件的 bsp_Init 函数里面调用。

22.4.2 函数 bsp_StartTimer

函数原型：

```
void bsp_StartTimer(uint8_t _id, uint32_t _period)
```

函数描述：

此函数用于启动一个单次定时器，并设置定时时间。

函数参数：

- 第 1 个参数_id 是定时器 ID，值域【0,TMR_COUNT-1】，其中软件定时器个数 TMR_COUNT 在 bsp_timer.h 文件里面定义，用户必须自行维护定时器 ID，以避免定时器 ID 冲突。
- 第 2 个参数_period 用于定时周期设置，单位 1ms。

注意事项：

此函数的解读在本章 22.3.4 小节。

使用举例：

调用此函数前，务必优先调用函数 bsp_InitTimer 进行初始化。

比如实现软件定时 2 单次定时 200ms 就是 bsp_StartTimer(2, 200)。

22.4.3 函数 bsp_StartAutoTimer

函数原型：

```
void bsp_StartAutoTimer(uint8_t _id, uint32_t _period)
```

函数描述：

此函数用于启动一个周期性定时器，并设置定时周期。

函数参数：

- 第 1 个参数_id 是定时器 ID，值域【0,TMR_COUNT-1】，其中软件定时器个数 TMR_COUNT 在 bsp_timer.h 文件里面定义，用户必须自行维护定时器 ID，以避免定时器 ID 冲突。
- 第 2 个参数_period 用于定时周期设置，单位 1ms。

注意事项：

此函数的解读在本章 22.3.5 小节。

使用举例：

调用此函数前，务必优先调用函数 bsp_InitTimer 进行初始化。

比如使用软件定时器 0 创建启动 1 个 100ms 的自动重装的定时器，就是 bsp_StartAutoTimer(0, 100)。

22.4.4 函数 bsp_StopTimer

函数原型：

```
void bsp_StopTimer(uint8_t _id)
```

函数描述：

此函数用于停止运行中的周期性定时器。

函数参数：

- 第 1 个参数_id 是定时器 ID，值域【0,TMR_COUNT-1】，其中软件定时器个数 TMR_COUNT 在 bsp_timer.h 文件里面定义，用户必须自行维护定时器 ID，以避免定时器 ID 冲突。

注意事项：

此函数的解读在本章 22.3.6 小节。

使用举例：

调用此函数前，务必优先调用函数 bsp_InitTimer 进行初始化。

比如停止软件定时 0 就是 bsp_StopTimer(0)。

22.4.5 函数 bsp_CheckTimer

函数原型：

```
uint8_t bsp_CheckTimer(uint8_t _id)
```

函数描述：

此函数用于检测软件定时器的定时时间是否到。

函数参数：

- 第 1 个参数_id 是定时器 ID，值域【0,TMR_COUNT-1】，其中软件定时器个数 TMR_COUNT 在 bsp_timer.h 文件里面定义，用户必须自行维护定时器 ID，以避免定时器 ID 冲突。
- 返回值，返回 0 表示定时未到，1 表示定时到。

注意事项：

此函数的解读在本章 22.3.7 小节。

使用举例：

调用此函数前，务必优先调用函数 bsp_InitTimer 进行初始化。比如检测软件定时 0 的时间是否到：

```
if (bsp_CheckTimer(0))
{
    /* 每隔 100ms 进来一次 */
    bsp_LedToggle(1);
}
```

22.5 多组软定时器驱动移植和使用

按键移植步骤如下：

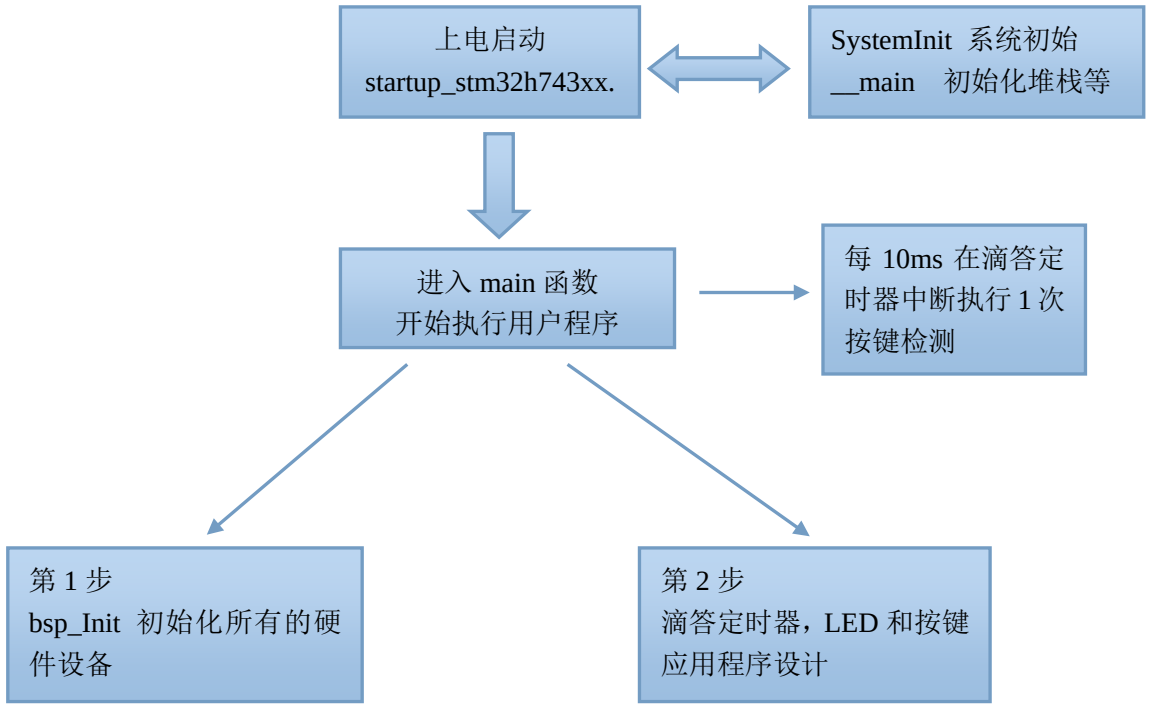
- 第 1 步：复制 bsp_timer.c 和 bsp_timer.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：根据需要的宏定义个数，修改下面的宏定义即可

```
#define TMR_COUNT  4        /* 软件定时器的个数 （定时器 ID 范围 0 - 3） */
```

- 第 3 步：这几个驱动文件主要用到 HAL 库的 TIM 驱动文件（源文件里面还封装了定时器），简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 4 步，应用方法看本章节配套例子即可。

22.6 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。
- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，蜂鸣器等。
  - 第 2 部分，应用程序设计部分，实现滴答定时器，LED 和按键应用程序设计。
  - 第 3 部分，按键检测每 10ms 在滴答定时中断执行一次。

22.7 实验例程说明（MDK）

配套例子：

V7-004_基于 Systick 滴答定时器的多组软件定时器实现

实验目的：

1. 学习基于 Systick 滴答定时器的多组软件定时器实现。

实验内容：

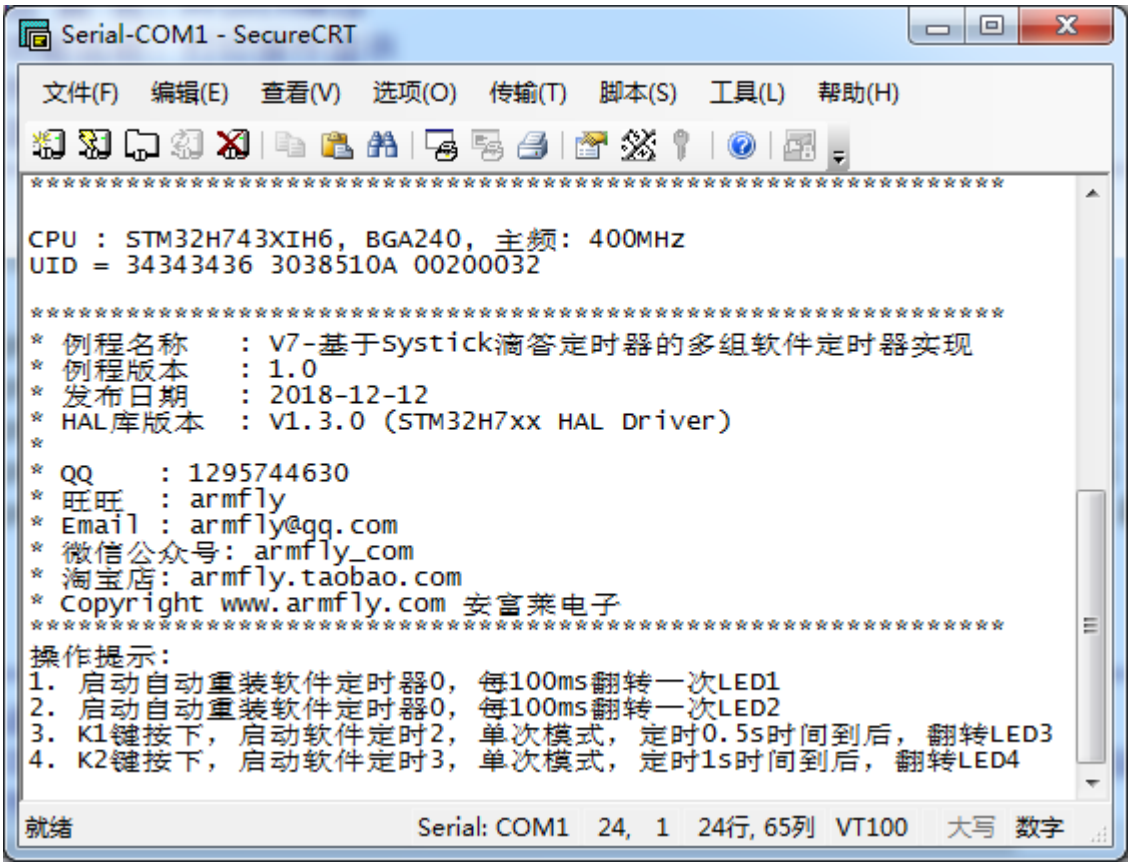
1. 启动自动重装软件定时器 0，每 100ms 翻转一次 LED1。
2. 启动自动重装软件定时器 1，每 100ms 翻转一次 LED2。

实验操作：

1. K1 键按下，启动软件定时 2，单次模式，定时 0.5s 时间到后，翻转 LED3。
2. K2 键按下，启动软件定时 3，单次模式，定时 1s 时间到后，翻转 LED4。

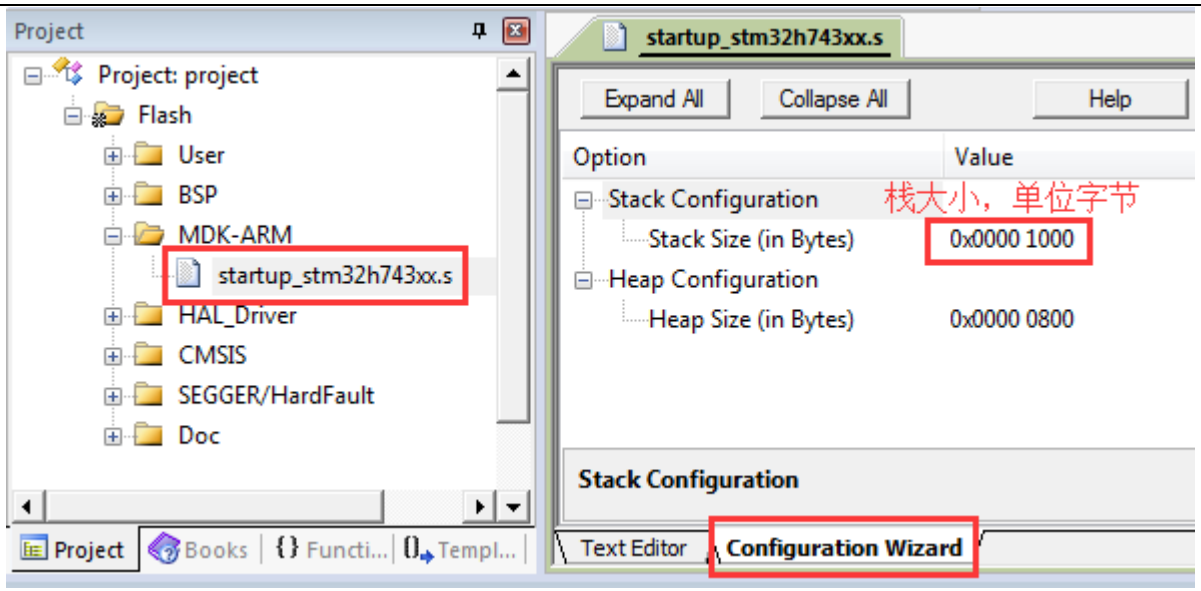
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

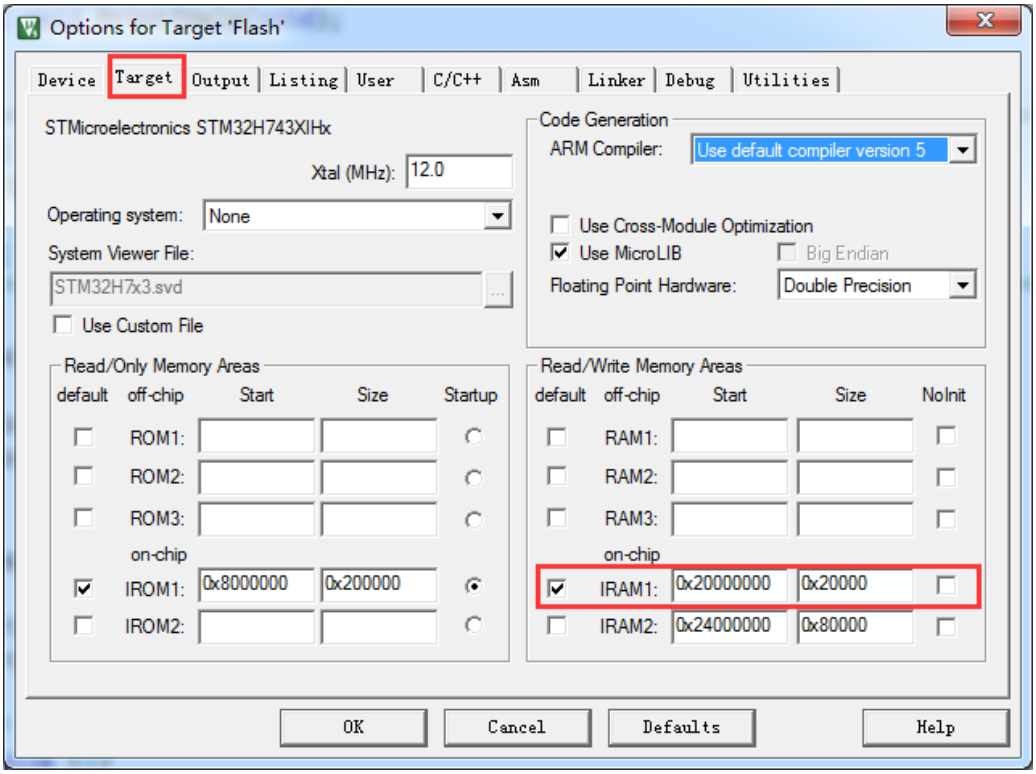


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();
```

```
    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **主功能：**

  主功能的实现主要分为两部分：

  - 启动自动重装软件定时器 0，每 100ms 翻转一次 LED1。
  - 启动自动重装软件定时器 1，每 100ms 翻转一次 LED2。

- K1 键按下，启动软件定时 2，单次模式，定时 0.5s 时间到后，翻转 LED3。
- K2 键按下，启动软件定时 3，单次模式，定时 1s 时间到后，翻转 LED4。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */

    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */


    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器，软件定时器 0 */
    bsp_StartAutoTimer(1, 100); /* 启动 1 个 100ms 的自动重装的定时器，软件定时器 1 */


    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断软件定时器 0 是否超时 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(1);
        }

        /* 判断软件定时器 1 超时 */
        if (bsp_CheckTimer(1))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 判断软件定时器 2 是否超时 */
        if (bsp_CheckTimer(2))
        {
            /* 单次模式，按下 K1 按键后，定时 1 秒进入 */
            bsp_LedToggle(3);
        }

        /* 判断软件定时器 3 是否超时 */
        if (bsp_CheckTimer(3))
        {
            /* 单次模式，按下 K2 按键后，定时 2 秒进入 */
            bsp_LedToggle(4);
```

```
        }


        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值，无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，启动软件定时 2，单次模式，定时 0.5 时间 */
                    printf("K1 键按下\r\n");
                    bsp_StartTimer(2, 500);
                    break;

                case KEY_DOWN_K2:           /* K2 键按下，启动软件定时 3，单次模式，定时 1s 时间  */
                    printf("K2 键按下\r\n");
                    bsp_StartTimer(3, 1000);
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }

        }
    }
}
```

22.8 实验例程说明（IAR）

配套例子：

V7-004_基于 Systick 滴答定时器的多组软件定时器实现

实验目的：

1. 学习基于 Systick 滴答定时器的多组软件定时器实现。

实验内容：

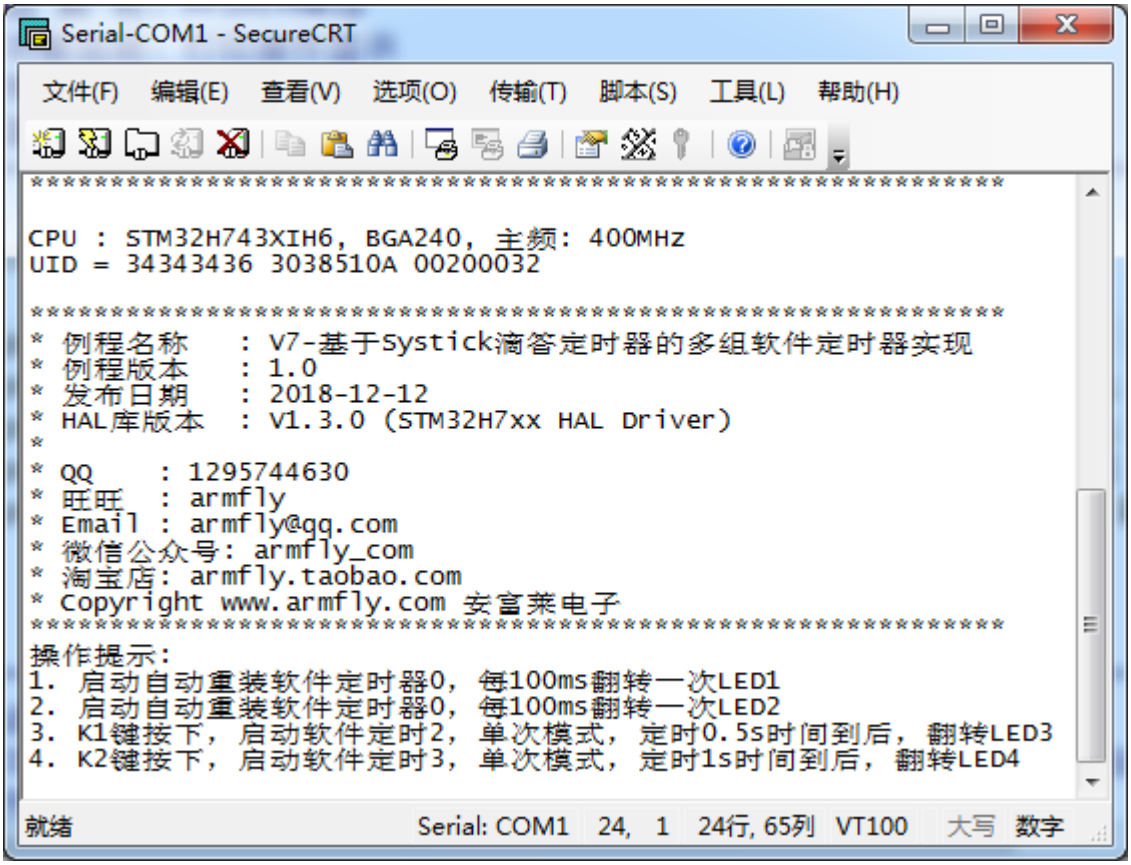
1. 启动自动重装软件定时器 0，每 100ms 翻转一次 LED1。
2. 启动自动重装软件定时器 1，每 100ms 翻转一次 LED2。

实验操作：

1. K1 键按下，启动软件定时 2，单次模式，定时 0.5s 时间到后，翻转 LED3。
2. K2 键按下，启动软件定时 3，单次模式，定时 1s 时间到后，翻转 LED4。

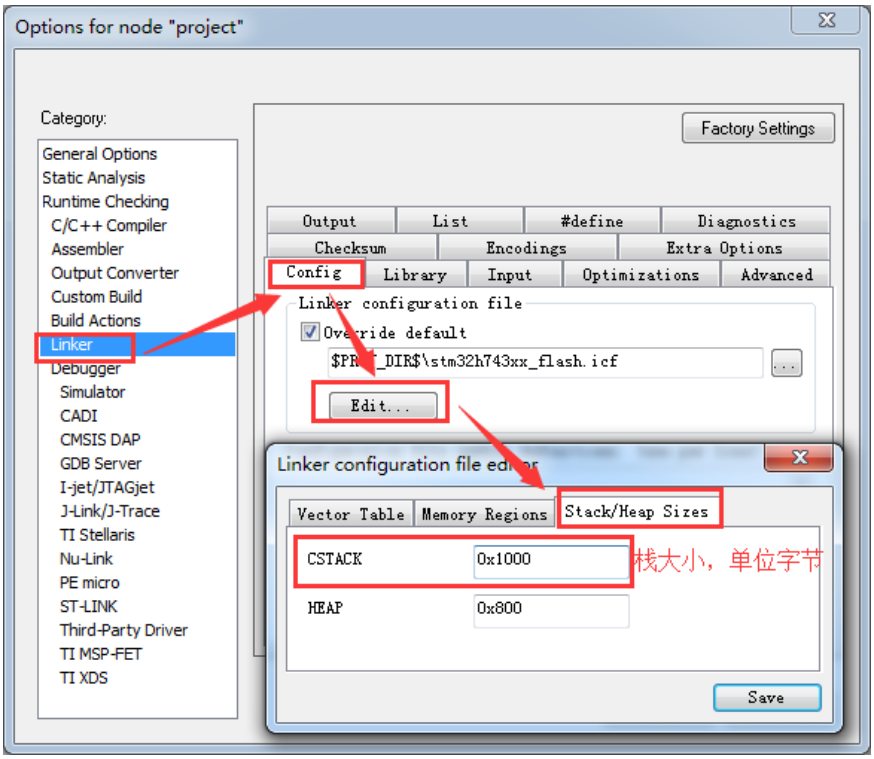
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

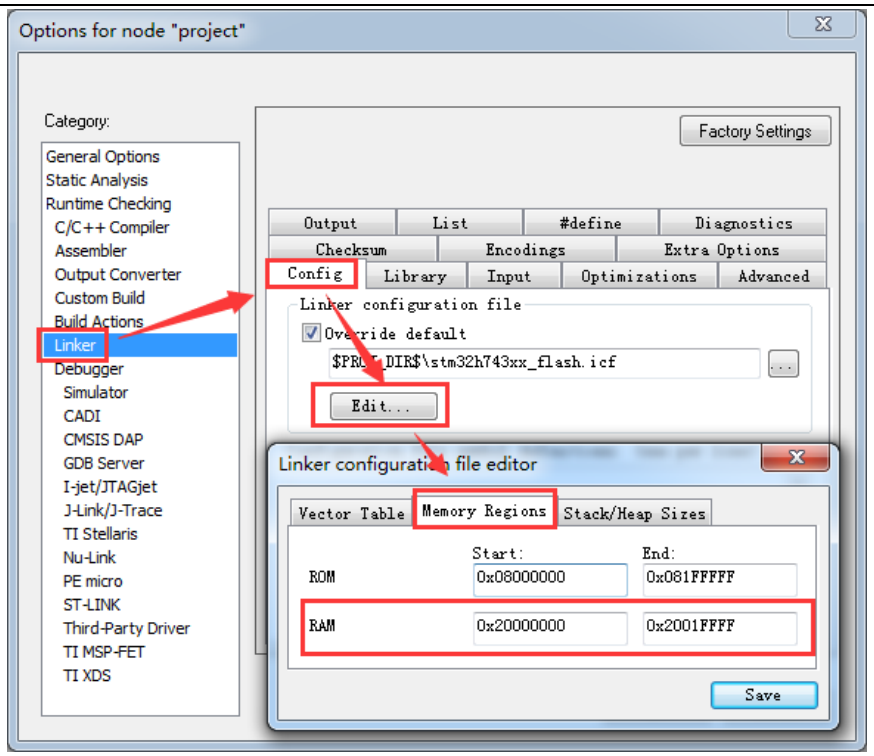


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
```

```
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主功能的实现主要分为两部分：

- 启动自动重装软件定时器 0，每 100ms 翻转一次 LED1。
- 启动自动重装软件定时器 1，每 100ms 翻转一次 LED2。

- K1 键按下，启动软件定时 2，单次模式，定时 0.5s 时间到后，翻转 LED3。
- K2 键按下，启动软件定时 3，单次模式，定时 1s 时间到后，翻转 LED4。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */

    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */
```

```
bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器，软件定时器 0 */
bsp_StartAutoTimer(1, 100); /* 启动 1 个 100ms 的自动重装的定时器，软件定时器 1 */


/* 进入主程序循环体 */
while (1)
{
    bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

    /* 判断软件定时器 0 是否超时 */
    if (bsp_CheckTimer(0))
    {
        /* 每隔 100ms 进来一次 */
        bsp_LedToggle(1);
    }

    /* 判断软件定时器 1 超时 */
    if (bsp_CheckTimer(1))
    {
        /* 每隔 100ms 进来一次 */
        bsp_LedToggle(2);
    }

    /* 判断软件定时器 2 是否超时 */
    if (bsp_CheckTimer(2))
    {
        /* 单次模式，按下 K1 按键后，定时 1 秒进入 */
        bsp_LedToggle(3);
    }

    /* 判断软件定时器 3 是否超时 */
    if (bsp_CheckTimer(3))
    {
        /* 单次模式，按下 K2 按键后，定时 2 秒进入 */
        bsp_LedToggle(4);
    }


    /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
    ucKeyCode = bsp_GetKey();   /* 读取键值，无键按下时返回 KEY_NONE = 0 */
    if (ucKeyCode != KEY_NONE)
    {
        switch (ucKeyCode)
        {
            case KEY_DOWN_K1:           /* K1 键按下，启动软件定时 2，单次模式，定时 0.5 时间 */
                printf("K1 键按下\r\n");
                bsp_StartTimer(2, 500);
                break;

            case KEY_DOWN_K2:           /* K2 键按下，启动软件定时 3，单次模式，定时 1s 时间  */
                printf("K2 键按下\r\n");
                bsp_StartTimer(3, 1000);
                break;

            default:
                /* 其它的键值不处理 */
                break;
```

```
                }

            }
        }
}
```

22.9 总结

本章节为大家介绍的多组软件定时器实现方案用到的地方比较多，而且实用。后续章节中多个外部设备驱动都会用到。

第23章 STM32H7 的 MPU 内存保护单元(重要)

本章节为大家讲解 STM32H7 学习中的一个重要知识点 MPU（Memory Protection Unit，内存保护单元），早在 STM32F1 和 F4 芯片上面也是有这个功能的，但是基本用不上。但是到了 H7 就得用上了，因为要设置 Cache。

23.1 初学者重要提示
23.2 MPU 简介
23.3 MPU 的功能实现
23.4 MPU 可以配置的三种内存类型
23.5 MPU 的寄存器和对应的库参数
23.6 MPU 的配置函数
23.7 总结

23.1 初学者重要提示

- 本章节主要是为下一章 Cache 的讲解做个铺垫，需要初学者把本章节涉及到的基础知识点掌握了。
- 初学 MPU 时，可能有些知识点无法一下子就搞明白，在后续章节的各种应用后，会有一个较深的认识。这个知识点基本会贯穿整个教程。

23.2 MPU 简介

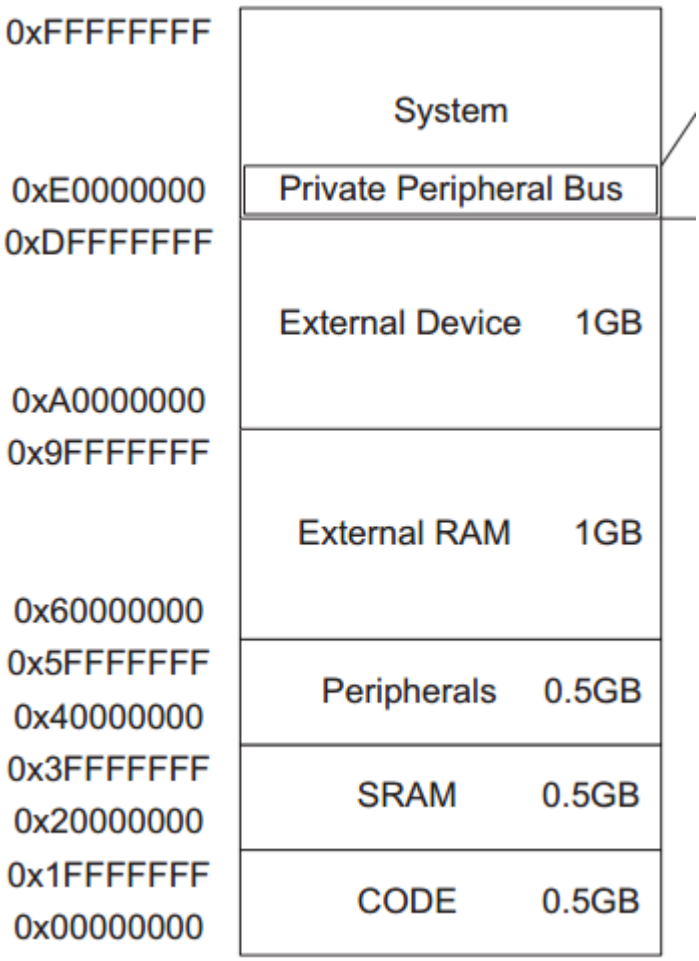
MPU 可以将 memory map 内存映射区分为多个具有一定访问规则的区域，通过这些规则可以实现如下功能：

- 防止不受信任的应用程序访问受保护的内存区域。
- 防止用户应用程序破坏操作系统使用的数据。
- 通过阻止任务访问其它任务的数据区。
- 允许将内存区域定义为只读，以便保护重要数据。
- 检测意外的内存访问。

简单的说就是内存保护、外设保护和代码访问保护。

- 内存映射

内存映射就是 32 位的 CM7 内核整体可以寻址的 0 到 2^32 -1 共计 4GB 的寻址空间。通过这些地址可以访问 RAM、Flash、外设等。下面是内存映射的轮廓图，IC 厂家使用时，再做细分，添加相应的硬件功能。

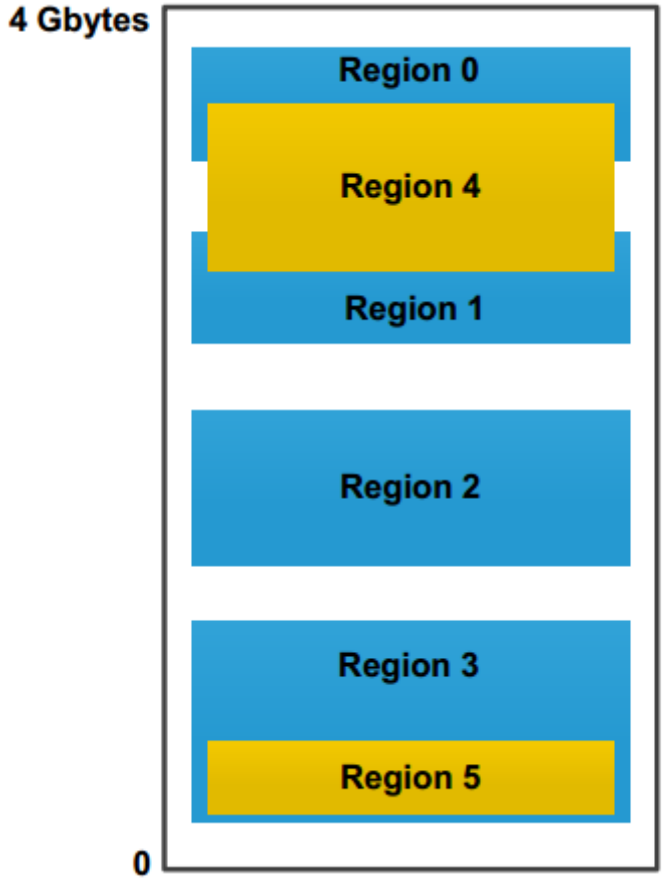


23.3 MPU 的功能实现

MPU 可以配置保护 16 个内存区域（这 16 个内存域是独立配置的），每个区域最小要求 256 字节，每个区域还可以配置为 8 个子区域。由于子区域一般都相同大小，这样每个子区域的大小就是 32 字节，正好跟 Cache 的 Cache Line 大小一样。

MPU 可以配置的 16 个内存区的序号范围是 0 到 15，还有默认区 default region，也叫作背景区，序号-1。由于这些内存区可以嵌套和重叠，所以这些区域在嵌套或者重叠的时候有个优先级的问题。序号 15 的优先级最高，以此递减，序号-1，即背景区的优先级最低。这些优先级是固定的。

下面通过一个具体的实例帮助大家理解。如下所示共有 7 个区，背景区和序号 0-5 的区。内存区 4 跟内存区 0 和 1 有重叠部分，那么重叠部分将按照内存区 4 的配置规则执行；内存区 5 被完全包含在内存区 3 里面，那么这部分内存区将按照内存区 5 的配置规则执行。



23.4 MPU 可以配置的三种内存类型

MPU 可以配置的三种内存类型如下：

- Normal memory

  CPU 以最高效的方式加载和存储字节、半字和字，对于这种内存区，CPU 的加载或存储不一定要按照程序列出的顺序执行。

- Device memory

  对于这种类型的内存区，加载和存储要严格按照次序进行，这样是为了确保寄存器按照正确顺序设置。

- Strongly ordered memory

  程序完全按照代码顺序执行，CPU 需要等待当前的加载/存储指令执行完毕后才执行下一条指令。这样会导致性能下降。

23.5 MPU 的寄存器和对应的库参数

关于 MPU 的寄存器介绍在 STM32H7 的编程手册有专门的讲解说明，我们这里重点讲解寄存器 MPU_RASR 和控制寄存器，此寄存器的定义如下：

Bits	Name	Function
[31:29]	-	Reserved.
[28]	XN	Instruction access disable bit: 0: Instruction fetches enabled. 1: Instruction fetches disabled.
[27]	-	Reserved.
[26:24]	AP	Access permission field, see *Table 93 on page 229*.
[23:22]	-	Reserved.
[21:19, 17, 16]	TEX, C, B	Memory access attributes, see *Table 91 on page 228*.
[18]	S	Shareable bit, see *Table 91 on page 228*.
[15:8]	SRD	Subregion disable bits. For each bit in this field: 0: Corresponding sub-region is enabled. 1: Corresponding sub-region is disabled. See *Subregions on page 231* for more information. Region sizes of 128 bytes and less do not support subregions. When writing the attributes for such a region, write the SRD field as 0x00.
[7:6]	-	Reserved.
[5:1]	SIZE	Specifies the size of the MPU protection region. The minimum permitted value is 4 (0b00100), see *SIZE field values* for more information.
[0]	ENABLE	Region enable bit.

23.5.1 RASR 寄存器的 XN 位

XN=0 表示使能指令提取，即这块内存区可以执行程序代码，XN=1 表示禁止指令提取，即这块内存区禁止执行程序代码。

对应的 HAL 库 MPU 参数如下：

```
/** @defgroup CORTEX_MPU_Instruction_Access CORTEX MPU Instruction Access
  * @{
  */
#define  MPU_INSTRUCTION_ACCESS_ENABLE      ((uint8_t)0x00)
#define  MPU_INSTRUCTION_ACCESS_DISABLE     ((uint8_t)0x01)
```

23.5.2 RASR 寄存器的 AP 位

AP 的具体定义如下：

AP[2:0]	Privileged permissions	Unprivileged permissions	Description
000	No access	No access	All accesses generate a permission fault
001	RW	No access	Access from privileged software only
010	RW	RO	Writes by unprivileged software generate a permission fault
011	RW	RW	Full access
100	Unpredictable	Unpredictable	Reserved
101	RO	No access	Reads by privileged software only
110	RO	RO	Read only, by privileged or unprivileged software
111	RO	RO	Read only, by privileged or unprivileged software

这几个参数对应的 HAL 库 MPU 参数如下:

```
/** @defgroup CORTEX_MPU_Region_Permission_Attributes CORTEX MPU Region Permission Attributes
  * @{
  */
#define  MPU_REGION_NO_ACCESS      ((uint8_t)0x00)
#define  MPU_REGION_PRIV_RW        ((uint8_t)0x01)
#define  MPU_REGION_PRIV_RW_URO    ((uint8_t)0x02)
#define  MPU_REGION_FULL_ACCESS    ((uint8_t)0x03)
#define  MPU_REGION_PRIV_RO        ((uint8_t)0x05)
#define  MPU_REGION_PRIV_RO_URO    ((uint8_t)0x06)
```

23.5.3 RASR 寄存器的 TEX，C，B 和 S 位

(注，这几个位非常重要，当前先了解知识点即可，下个章节专门讲解具体的作用)

TEX，C，B 和 S 的定义如下，这仅关注 TEX = 0b000 和 0b001，其它的 TEX 配置基本用不到。

TEX	C	B	S	Memory type	Shareability	Other attributes
0b000	0	0	x(1)	Strongly-ordered	Shareable	-
		1	x(1)	Device	Shareable	-
	1	0	0	Normal	Not shareable	Outer and inner write-through. No write allocate.
			1		Shareable	
		1	0	Normal	Not shareable	Outer and inner write-back. No write allocate.
			1		Shareable	
0b001	0	0	0	Normal	Not shareable	Outer and inner noncacheable.
			1		Shareable	
		1	x(1)	Reserved encoding		-
	1	0	x(1)	Implementation defined attributes.		-
		1	0	Normal	Not shareable	Outer and inner write-back. Write and read allocate.
			1		Shareable	

TEX 用于配置 Cache 策略，支持如下四种情况，需要配合 C 和 B 位的配置才能实现。

Corresponding cache policy
Non-cacheable
Write back, write and read allocate
Write through, no write allocate
Write back, no write allocate

TEX 对应的 HAL 库 MPU 参数给了三个，实际应用中仅用到前两个 MPU_TEX_LEVEL0 和 MPU_TEX_LEVEL1

```
/** @defgroup CORTEX_MPU_TEX_Levels MPU TEX Levels
  * @{
  */
#define  MPU_TEX_LEVEL0    ((uint8_t)0x00)
#define  MPU_TEX_LEVEL1    ((uint8_t)0x01)
#define  MPU_TEX_LEVEL2    ((uint8_t)0x02)
```

C 位对应的 HAL 库 MPU 参数如下，用于使能或者禁止 Cache。

```
/** @defgroup CORTEX_MPU_Access_Cacheable CORTEX MPU Instruction Access Cacheable
  * @{
  */
#define  MPU_ACCESS_CACHEABLE         ((uint8_t)0x01)
#define  MPU_ACCESS_NOT_CACHEABLE     ((uint8_t)0x00)
```

B 位对应的 HAL 库 MPU 参数如下，用于配合 C 位实现 Cache 模式下是否使用缓冲。

```
/** @defgroup CORTEX_MPU_Access_Bufferable CORTEX MPU Instruction Access Bufferable
  * @{
```

```
  */
#define  MPU_ACCESS_BUFFERABLE         ((uint8_t)0x01)
#define  MPU_ACCESS_NOT_BUFFERABLE     ((uint8_t)0x00)
```

S 位对应的 HAL 库 MPU 参数如下，用于解决多总线或者多核访问的共享问题。

```
/** @defgroup CORTEX_MPU_Access_Shareable CORTEX MPU Instruction Access Shareable
  * @{
  */
#define  MPU_ACCESS_SHAREABLE        ((uint8_t)0x01)
#define  MPU_ACCESS_NOT_SHAREABLE    ((uint8_t)0x00)
```

23.5.4 RASR 寄存器的 SRD 位

这个位用于控制内存区的子区域，使用的是 bit[15:8]，共计 8 个 bit，一个 bit 控制一个子区域，0 表示使能此子区域，1 表示禁止此子区域。

一般情况，基本不使用子区域的禁止功能，所以配置 HAL 库的 SubRegionDisable 参数时，直接取值 0x00 即可，表示 8 个子区域均使能。

23.5.5 RASR 寄存器的 SIZE 位

SIZE 位使用的是 bit[5:1]，共计 5 个 bit，可以表示 2^5 = 32 种大小。 对应的 HAL 库给出了可以配置的 28 个参数：

```
/** @defgroup CORTEX_MPU_Region_Size CORTEX MPU Region Size
  * @{
  */
#define   MPU_REGION_SIZE_32B      ((uint8_t)0x04)
#define   MPU_REGION_SIZE_64B      ((uint8_t)0x05)
#define   MPU_REGION_SIZE_128B     ((uint8_t)0x06)
#define   MPU_REGION_SIZE_256B     ((uint8_t)0x07)
#define   MPU_REGION_SIZE_512B     ((uint8_t)0x08)
#define   MPU_REGION_SIZE_1KB      ((uint8_t)0x09)
#define   MPU_REGION_SIZE_2KB      ((uint8_t)0x0A)
#define   MPU_REGION_SIZE_4KB      ((uint8_t)0x0B)
#define   MPU_REGION_SIZE_8KB      ((uint8_t)0x0C)
#define   MPU_REGION_SIZE_16KB     ((uint8_t)0x0D)
#define   MPU_REGION_SIZE_32KB     ((uint8_t)0x0E)
#define   MPU_REGION_SIZE_64KB     ((uint8_t)0x0F)
#define   MPU_REGION_SIZE_128KB    ((uint8_t)0x10)
#define   MPU_REGION_SIZE_256KB    ((uint8_t)0x11)
#define   MPU_REGION_SIZE_512KB    ((uint8_t)0x12)
#define   MPU_REGION_SIZE_1MB      ((uint8_t)0x13)
#define   MPU_REGION_SIZE_2MB      ((uint8_t)0x14)
#define   MPU_REGION_SIZE_4MB      ((uint8_t)0x15)
#define   MPU_REGION_SIZE_8MB      ((uint8_t)0x16)
#define   MPU_REGION_SIZE_16MB     ((uint8_t)0x17)
#define   MPU_REGION_SIZE_32MB     ((uint8_t)0x18)
#define   MPU_REGION_SIZE_64MB     ((uint8_t)0x19)
#define   MPU_REGION_SIZE_128MB    ((uint8_t)0x1A)
#define   MPU_REGION_SIZE_256MB    ((uint8_t)0x1B)
#define   MPU_REGION_SIZE_512MB    ((uint8_t)0x1C)
```

```
#define   MPU_REGION_SIZE_1GB      ((uint8_t)0x1D)
#define   MPU_REGION_SIZE_2GB      ((uint8_t)0x1E)
#define   MPU_REGION_SIZE_4GB      ((uint8_t)0x1F)
```

23.5.6 CTRL 寄存器的各个位

控制寄存器各个位定义如下如下：

Bits	Name	Function
[31:3]	-	Reserved.
[2]	PRIVDEFENA	Enables privileged software access to the default memory map: 0: If the MPU is enabled, disables use of the default memory map. Any memory access to a location not covered by any enabled region causes a fault. 1: If the MPU is enabled, enables use of the default memory map as a background region for privileged software accesses. When enabled, the background region acts as if it is region number -1. Any region that is defined and enabled has priority over this default map. If the MPU is disabled, the processor ignores this bit.
[1]	HFNMIENA	Enables the operation of MPU during hard fault, NMI, and FAULTMASK handlers. When the MPU is enabled: 0: MPU is disabled during hard fault, NMI, and FAULTMASK handlers, regardless of the value of the ENABLE bit. 1: The MPU is enabled during hard fault, NMI, and FAULTMASK handlers. When the MPU is disabled, if this bit is set to 1 the behavior is Unpredictable.
[0]	ENABLE	Enables the MPU: 0: MPU disabled. 1: MPU enabled.

23.6 MPU 的配置函数

HAL 库的 stm32h7xx_hal_cortex.c 文件为 MPU 的配置提供了三个函数：

- HAL_MPU_Disable
- HAL_MPU_Enable
- HAL_MPU_ConfigRegion

三个函数的使用都比较简单，但是要让配置的内存区最大限度的发挥性能是需要大量的经验积累和测试的。具体工程要具体分析。

23.6.1 函数 HAL_MPU_Disable

函数原型：

```
void HAL_MPU_Disable(void)
{
  /* Make sure outstanding transfers are done */
  __DMB();

  /* Disable fault exceptions */
  SCB->SHCSR &= ~SCB_SHCSR_MEMFAULTENA_Msk;

  /* Disable the MPU and clear the control register*/
  MPU->CTRL = 0;
}
```

函数描述：

此函数用于禁止 MPU。配置 MPU 前要优先调用此函数禁止 MPU，然后才可以配置。

23.6.2 函数 HAL_MPU_Enable

函数原型：

```
void HAL_MPU_Enable(uint32_t MPU_Control)
{
  /* Enable the MPU */
  MPU->CTRL = MPU_Control | MPU_CTRL_ENABLE_Msk;

  /* Enable fault exceptions */
  SCB->SHCSR |= SCB_SHCSR_MEMFAULTENA_Msk;

  /* Ensure MPU setting take effects */
  __DSB();
  __ISB();
}
```

函数描述：

此函数用于使能 MPU，一般情况使用参数 MPU_PRIVILEGED_DEFAULT。

函数参数：

此函数支持以下几个参数：

- MPU_HFNMI_PRIVDEF_NONE ((uint32_t)0x00000000)
  - 此参数设置 MPU 的 CTL 控制寄存器的 PRIVDEFENA 位为 0。
    表示禁止了背景区，访问任何未使能 MPU 的区域均会造成内存异常 MemFault。
  - 此参数设置 MPU 的 CTL 控制寄存器的 HFNMIENA 位为 0。
    表示 NMI 不可屏蔽中断服务程序和硬件异常中断服务程序执行期间会关闭 MPU。
- MPU_HARDFAULT_NMI ((uint32_t)0x00000002)
  - 此参数设置 MPU 的 CTL 控制寄存器的 PRIVDEFENA 位为 0。
    表示使能了背景区，访问任何未使能 MPU 的区域均会造成内存异常 MemFault。
  - 此参数设置 MPU 的 CTL 控制寄存器的 HFNMIENA 位为 1。
    表示 NMI 不可屏蔽中断服务程序和硬件异常中断服务程序执行期间会保持继续开启 MPU。
- MPU_PRIVILEGED_DEFAULT ((uint32_t)0x00000004)

- 此参数设置 MPU 的 CTL 控制寄存器的 PRIVDEFENA 位为 1。
  表示使能了背景区，特权级模式可以正常访问任何未使能 MPU 的区域。
- 此参数设置 MPU 的 CTL 控制寄存器的 HFNMIENA 位为 0。
  表示 NMI 不可屏蔽中断服务程序和硬件异常中断服务程序执行期间会关闭 MPU。

◆ MPU_HFNMI_PRIVDEF ((uint32_t)0x00000006)

- 此参数设置 MPU 的 CTL 控制寄存器的 PRIVDEFENA 位为 1。
  表示禁止了背景区，访问任何未使能 MPU 的区域均会造成内存异常 MemFault。
- 此参数设置 MPU 的 CTL 控制寄存器的 HFNMIENA 位为 1。
  表示 NMI 不可屏蔽中断服务程序和硬件异常中断服务程序执行期间会保持继续开启 MPU。

23.6.3 函数 HAL_MPU_ConfigRegion

函数原型：

```
void HAL_MPU_ConfigRegion(MPU_Region_InitTypeDef *MPU_Init)
{
  /* Check the parameters */
  assert_param(IS_MPU_REGION_NUMBER(MPU_Init->Number));
  assert_param(IS_MPU_REGION_ENABLE(MPU_Init->Enable));

  /* Set the Region number */
  MPU->RNR = MPU_Init->Number;

  if ((MPU_Init->Enable) != RESET)
  {
    /* Check the parameters */
    assert_param(IS_MPU_INSTRUCTION_ACCESS(MPU_Init->DisableExec));
    assert_param(IS_MPU_REGION_PERMISSION_ATTRIBUTE(MPU_Init->AccessPermission));
    assert_param(IS_MPU_TEX_LEVEL(MPU_Init->TypeExtField));
    assert_param(IS_MPU_ACCESS_SHAREABLE(MPU_Init->IsShareable));
    assert_param(IS_MPU_ACCESS_CACHEABLE(MPU_Init->IsCacheable));
    assert_param(IS_MPU_ACCESS_BUFFERABLE(MPU_Init->IsBufferable));
    assert_param(IS_MPU_SUB_REGION_DISABLE(MPU_Init->SubRegionDisable));
    assert_param(IS_MPU_REGION_SIZE(MPU_Init->Size));

    MPU->RBAR = MPU_Init->BaseAddress;
    MPU->RASR = ((uint32_t)MPU_Init->DisableExec             << MPU_RASR_XN_Pos)   |
                ((uint32_t)MPU_Init->AccessPermission        << MPU_RASR_AP_Pos)   |
                ((uint32_t)MPU_Init->TypeExtField            << MPU_RASR_TEX_Pos)  |
                ((uint32_t)MPU_Init->IsShareable             << MPU_RASR_S_Pos)    |
                ((uint32_t)MPU_Init->IsCacheable             << MPU_RASR_C_Pos)    |
                ((uint32_t)MPU_Init->IsBufferable            << MPU_RASR_B_Pos)    |
                ((uint32_t)MPU_Init->SubRegionDisable        << MPU_RASR_SRD_Pos)  |
                ((uint32_t)MPU_Init->Size                    << MPU_RASR_SIZE_Pos) |
                ((uint32_t)MPU_Init->Enable                  << MPU_RASR_ENABLE_Pos);
  }
  else
  {
    MPU->RBAR = 0x00;
    MPU->RASR = 0x00;
```

```
  }
}
```

函数描述：

此函数用于配置 MPU。

函数参数：

此函数的形参是一个 MPU_Region_InitTypeDef 类型的结构体变量，定义如下：

```
typedef struct
{
  uint8_t               Enable;
  uint8_t               Number;
  uint32_t              BaseAddress;
  uint8_t               Size;
  uint8_t               SubRegionDisable;
  uint8_t               TypeExtField;
  uint8_t               AccessPermission;
  uint8_t               DisableExec;
  uint8_t               IsShareable;
  uint8_t               IsCacheable;
  uint8_t               IsBufferable;
}MPU_Region_InitTypeDef;
```

除了参数 Number 和 BaseAddress，其它几个参数在本章 23.5 小节进行了说明。

- 结构体成员 Number

  这个成员是用来设置内存区序号的，用户配置的时候，推荐从 Number0 开始配置，最多到 Number15，共计 16 个。对应的 HAL 库参数如下：

```
/** @defgroup CORTEX_MPU_Region_Number CORTEX MPU Region Number
  * @{
  */
#define  MPU_REGION_NUMBER0    ((uint8_t)0x00)
#define  MPU_REGION_NUMBER1    ((uint8_t)0x01)
#define  MPU_REGION_NUMBER2    ((uint8_t)0x02)
#define  MPU_REGION_NUMBER3    ((uint8_t)0x03)
#define  MPU_REGION_NUMBER4    ((uint8_t)0x04)
#define  MPU_REGION_NUMBER5    ((uint8_t)0x05)
#define  MPU_REGION_NUMBER6    ((uint8_t)0x06)
#define  MPU_REGION_NUMBER7    ((uint8_t)0x07)
#define  MPU_REGION_NUMBER8    ((uint8_t)0x08)
#define  MPU_REGION_NUMBER9    ((uint8_t)0x09)
#define  MPU_REGION_NUMBER10   ((uint8_t)0x0A)
#define  MPU_REGION_NUMBER11   ((uint8_t)0x0B)
#define  MPU_REGION_NUMBER12   ((uint8_t)0x0C)
#define  MPU_REGION_NUMBER13   ((uint8_t)0x0D)
#define  MPU_REGION_NUMBER14   ((uint8_t)0x0E)
#define  MPU_REGION_NUMBER15   ((uint8_t)0x0F)
```

- 结构体成员 BaseAddress

  这个结构体成员用来设置内存区的首地址。这个参数跟结构体成员 Size 的配置是比较考究的，一定要保证首地址跟内存区的大小对齐，比如配置的是 64KB 大小的内存区，那么设置的首地址务必是 64KB 对齐的，也就是这个地址对 64KB，即 0x00010000 求余数等于 0，切记。

使用举例：

下面配置了两组。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}
```

23.7 总结

本章节就为大家讲解这么多，还是本章开头那句话，可能有些知识点无法一下子就搞明白，在学习了后续章节的各种应用后，会有一个较深的认识。

第24章 STM32H7 的 Cache 解读（非常重要）

本章教程为大家讲解 STM32H7 初学过程中最重要的一个知识点 Cache。Cache 在 STM32H7 的高性能发挥中占着举足轻重的作用。所以掌握好 Cache 是提升 STM32H7 性能的关键一步。

24.1 初学者重要提示
24.2 引出问题
24.3 支持的 Cache 配置
24.4 四种 Cache（MPU）配置的读写操作流程
24.5 面对繁冗复杂的 Cache 配置，推荐方式和安全隐患解决办法
24.6 Cache 的相关函数
24.7 总结

24.1 初学者重要提示

- 学习本章节前，务必保证已经学习了第 23 章的 MPU 知识。
- 本章是半年的实践经验总结，非常具有参考价值，而且是入门 STM32H7 的必学章节。
- Cache 的熟练运用需要不断的经验积累。对于初学者来说，可能无法一下子理解所有知识点，但是一定要的花时间多读几遍，随着后面章节的不断运用，认识会不断的深入。

24.2 引出问题

当前芯片厂商出的 M7 内核芯片基本都做了一级 Cache 支持，Cache 又分数据缓存 D-Cache 和指令缓冲 I-Cache，STM32H7 的数据缓存和指令缓存大小都是 16KB。对于指令缓冲，用户不用管，这里主要说的是数据缓存 D-Cache。以 STM32H7 为例，主频是 400MHz，除了 TCM 和 Cache 以 400MHz 工作，其它 AXI SRAM，SRAM1，SRAM2 等都是以 200MHz 工作。数据缓存 D-Cache 就是解决 CPU 加速访问 SRAM。

如果每次 CPU 要读写 SRAM 区的数据，都能够在 Cache 里面进行，自然是最好的，实现了 200MHz 到 400MHz 的飞跃，实际是做不到的，因为数据 Cache 只有 16KB 大小，总有用完的时候。

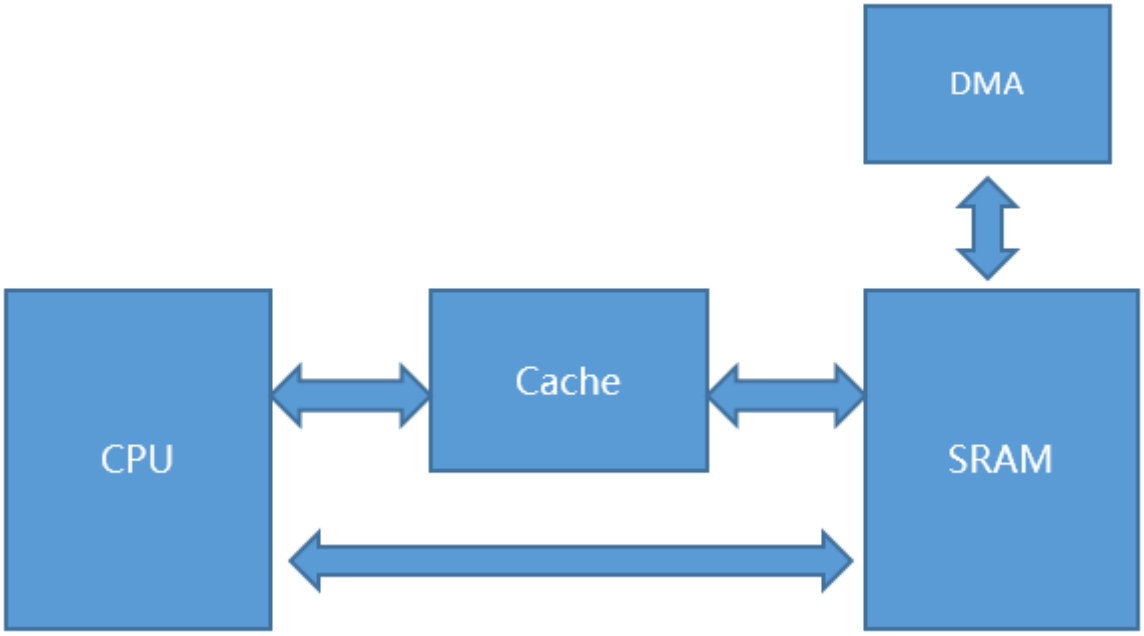
对于使能了 Cache 的 SRAM 区，要分读写两种情况考虑。

- 读操作：

如果 CPU 要读取的 SRAM 区数据在 Cache 中已经加载好，这就叫读命中（Cache hit），如果 Cache 里面没有怎么办，这就是所谓的读 Cache Miss。

◆ 写操作：

如果 CPU 要写的 SRAM 区数据在 Cache 中已经开辟了对应的区域（专业词汇叫 Cache Line，以 32 字节为单位），这就叫写命中（Cache hit），如果 Cache 里面没有开辟对应的区域怎么办，这就是所谓的写 Cache Miss。



24.3 支持的 Cache 配置

（这个知识点在上一章节进行了详细说明，这里再简述下核心内容）

Cache 的配置是通过 MPU 来设置的，通常只用到下几种方式。

Table 91. TEX, C, B, and S encoding

TEX	C	B	S	Memory type	Shareability	Other attributes
0b000	0	0	x(1)	Strongly-ordered	Shareable	-
		1	x(1)	Device	Shareable	-
	1	0	0	Normal	Not shareable	Outer and inner write-through. No write allocate.
			1		Shareable	
		1	0	Normal	Not shareable	Outer and inner write-back. No write allocate.
			1		Shareable	
0b001	0	0	0	Normal	Not shareable	Outer and inner noncacheable.
			1		Shareable	
		1	x(1)	Reserved encoding		-
	1	0	x(1)	Implementation defined attributes.		-
		1	0	Normal	Not shareable	Outer and inner write-back. Write and read allocate.
			1		Shareable	

其中的 TEX 是用来设置 Cache 策略的，C 是 Cache，B 是缓冲用来配合 Cache 设置的，而 S 是共享，用来解决多总线或者多核访问时的同步问题。MPU 配置的时候，最主要的也是配置这几个参数。

Cache 支持的策略有如下四种：

Corresponding cache policy
Non-cacheable
Write back, write and read allocate
Write through, no write allocate
Write back, no write allocate

有了这四种方式，就可以正式进入本章的主题，Cache 的读写操作是如何工作的，下面分这四种情况做介绍。

24.4 四种 Cache（MPU）配置的读写操作流程

24.4.1 配置 Non-cacheable

这个最好理解，就是正常的读写操作，无 Cache。

对应四种 MPU 配置如下：

- TEX = 000 C=0 B=0 S=忽略此位，强制为共享
- TEX = 000 C=0 B=1 S=忽略此位，强制为共享
- TEX = 001 C=0 B=0 S=0
- TEX = 001 C=0 B=0 S=1

24.4.2配置 Write through，read allocate，no write allocate

注意，M7 内核只要开启了 Cache，read allocate 就是开启的。

- 使能了此配置的 SRAM 缓冲区写操作

如果 CPU 要写的 SRAM 区数据在 Cache 中已经开辟了对应的区域，那么会同时写到 Cache 里面和 SRAM 里面；如果没有，就用到配置 no write allocate 了，意思就是 CPU 会直接往 SRAM 里面写数据，而不再需要在 Cache 里面开辟空间了。

在写 Cache 命中的情况下，这个方式的优点是 Cache 和 SRAM 的数据同步更新了，没有多总线访问造成的数据一致性问题。缺点也明显，Cache 在写操作上无法有效发挥性能。

- 使能了此配置的 SRAM 缓冲区读操作

如果 CPU 要读取的 SRAM 区数据在 Cache 中已经加载好，就可以直接从 Cache 里面读取。如果没有，就用到配置 read allocate 了，意思就是在 Cache 里面开辟区域，将 SRAM 区数据加载进来，后续的操作，CPU 可以直接从 Cache 里面读取，从而时间加速。

安全隐患，如果 Cache 命中的情况下，DMA 写操作也更新了 SRAM 区的数据，CPU 直接从 Cache 里面读取的数据就是错误的。

- 对应的两种 MPU 配置如下：

TEX = 000 C=1 B=0 S=1

TEX = 000 C=1 B=0 S=0

24.4.3配置 Write back，read allocate，no write allocate

注意，M7 内核只要开启了 Cache，read allocate 就是开启的。

- 使能了此配置的 SRAM 缓冲区写操作

如果 CPU 要写的 SRAM 区数据在 Cache 中已经开辟了对应的区域，那么会写到 Cache 里面，而不会立即更新 SRAM；如果没有，就用到配置 no write allocate 了，意思就是 CPU 会直接往 SRAM 里面写数据，而不再需要在 Cache 里面开辟空间了。

安全隐患，如果 Cache 命中的情况下，此时仅 Cache 更新了，而 SRAM 没有更新，那么 DMA 直接从 SRAM 里面读出来的就是错误的。

◆ 使能了此配置的 SRAM 缓冲区读操作

如果 CPU 要读取的 SRAM 区数据在 Cache 中已经加载好，就可以直接从 Cache 里面读取。如果没有，就用到配置 read allocate 了，意思就是在 Cache 里面开辟区域，将 SRAM 区数据加载进来，后续的操作，CPU 可以直接从 Cache 里面读取，从而时间加速。

安全隐患，如果 Cache 命中的情况下，DMA 写操作也更新了 SRAM 区的数据，CPU 直接从 Cache 里面读取的数据就是错误的。

◆ 对应两种 MPU 配置如下：

TEX = 000 C=1 B=1 S=1

TEX = 000 C=1 B=1 S=0

24.4.4配置 Write back，read allocate，write allocate

注意，M7 内核只要开启了 Cache，read allocate 就是开启的。

◆ 使能了此配置的 SRAM 缓冲区写操作

如果 CPU 要写的 SRAM 区数据在 Cache 中已经开辟了对应的区域，那么会写到 Cache 里面，而不会立即更新 SRAM；如果没有，就用到配置 write allocate 了，意思就是 CPU 写到往 SRAM 里面的数据，会同步在 Cache 里面开辟一个空间将 SRAM 中写入的数据加载进来，如果此时立即读此 SRAM 区，那么就会有很大的速度优势。

安全隐患，如果 Cache 命中的情况下，此时仅 Cache 更新了，而 SRAM 没有更新，那么 DMA 直接从 SRAM 里面读出来的就是错误的。

◆ 使能了此配置的 SRAM 缓冲区读操作

如果 CPU 要读取的 SRAM 区数据在 Cache 中已经加载好，就可以直接从 Cache 里面读取。如果没有，就用到配置 read allocate 了，意思就是在 Cache 里面开辟区域，将 SRAM 区数据加载进来，后续的操作，CPU 可以直接从 Cache 里面读取，从而时间加速。

安全隐患，如果 Cache 命中的情况下，DMA 写操作也更新了 SRAM 区的数据，CPU 直接从 Cache 里面读取的数据就是错误的。

这个配置被誉为可以最大程度发挥 Cache 性能，不过具体应用仍需具体分析。

◆ 对应两种 MPU 配置如下：

TEX = 001 C=1 B=1 S=1

TEX = 001 C=1 B=1 S=0

24.4.5共享配置是个隐形的大坑

STM32H7 编程手册对其的描述是多核共享。

Table 83. Memory attributes summary (continued)

Memory type	Shareability	Other attributes	Description
Normal	Shared	Non-cacheable Write-through Cacheable Write-back Cacheable	Normal memory that is shared between several processors.
-	Non-shared	Non-cacheable Write-through Cacheable Write-back Cacheable	Normal memory that only a single processor uses.

而 H7 的应用笔记对齐的描述是开启共享基本等同于关闭 Cache。

> The S field is for a shareable memory region: the memory system provides data synchronization between bus masters in a system with multiple bus masters, for example, a processor with a DMA controller. Strongly-ordered memory is always shareable. If multiple bus masters can access a non-shareable memory region, the software must ensure the data coherency between the bus masters. The STM32F7 Series and STM32H7 Series do not support hardware coherency. the S field is equivalent to non-cacheable memory.

实际测试下面四种开 Cache 的情况，开关共享对缓冲区的大批量数据的读操作影响很大，基本差出两倍，而写操作基本没有影响，也许这就是所谓的多总线同步读造成的。另外共享开关仅对开启了 Cache 的情况下有影响，而对于关闭了 Cache 的情况是没有影响的，开不开没关系。

24.4.6 总结这几种方式的几个关键知识点

1. Cortex-M7 内核的 L1 Cache 由多行内存区组成，每行有 32 字节，每行都配有一个地址标签。数据缓冲 DCache 是每 4 行为一组，称为 4-way set associative。而指令缓冲区 ICache 是 2 行为一组，这样节省地址标签，不用每个行都标记一个地址。
2. 对于读操作，只有在第 1 次访问指定地址时才会加载到 Cache，而写操作的话，可以直接写到内存中（write-through 模式）或者放到 Cache 里面，后面再写入（write-back 模式）。
3. 如果采用的是 Write back，Cache line 会被标为 dirty，等到此行被 evicted 时，才会执行实际的写操作，将 Cache Line 里面的数据写入到相应的存储区。
4. Cache 命中是访问的地址落在了给定的 Cache Line 里面，所以硬件需要做少量的地址比较工作，以检查此地址是否被缓存。如果命中了，将用于缓存读操作或者写操作。如果没有命中，则分配和标记新行，填充新的读写操作。如果所有行都分配完毕了，Cache 控制器将支持 eviction 操作。根据 Cache Line 替换算法，一行将被清除 Clean，无效化 Invalid 或者重新配置。数据缓存和指令缓存是采用的伪随机替换算法。
5. Cache 支持的 4 种基本操作，使能，禁止，清空和无效化。Clean 清空操作是将 Cache Line 中标记为 dirty 的数据写入到内存里面，而无效化 Invalid 是将 Cache Line 标记为无效，即删除操作。

24.5 面对繁冗复杂的 Cache 配置，推荐方式和安全隐患解决办法

- 推荐使用 128KB 的 TCM 作为主 RAM 区，其它的专门用于大缓冲和 DMA 操作等。
- Cache 问题主要是 CPU 和 DMA 都操作这个缓冲区时容易出现，使用时要注意。
- Cache 配置的选择，优先考虑的是 WB，然后是 WT 和关闭 Cache，其中 WB 和 WT 的使用中可以配合 ARM 提供的函数解决上面说到的隐患问题（见本章 24.6 小节）。但不是万能的，在不起作用的时候，直接暴力选择函数 SCB_CleanInvlaidateDCache 解决。关于这个问题，在分别配置以太网 MAC 的描述符缓冲区，发送缓冲区和接收缓冲区时尤其突出。

24.6 Cache 的相关函数

CMSIS 软件包的 core_cm7.h 文件为 Cache 的配置提供了 11 个函数：

- SCB_EnableICache
- SCB_DisableICache
- SCB_InvalidateICache
- SCB_EnableDCache
- SCB_DisableDCache
- SCB_InvalidateDCache
- SCB_CleanDCache
- SCB_CleanInvalidateDCache
- SCB_InvalidateDCache_by_Addr
- SCB_CleanDCache_by_Addr
- SCB_CleanInvalidateDCache_by_Addr

下面将这几个函数依次做个讲解。其中前三个函数是指令 Cache，比较容易掌握。重点是后面几个数据 Cache 函数。由于函数 SCB_CleanInvalidateDCache，SCB_CleanDCache 和 SCB_InvalidateDCache 是对整个 Cache 的操作，所以比最后的三个函数 SCB_InvalidateDCache_by_Addr，SCB_CleanDCache_by_Addr 和 SCB_CleanInvalidateDCache_by_Addr 要耗时，当然，如果用户操作的存储器超过了数据 Cache 的大小，即 16KB，那么就跟前三个函数没有区别了。

24.6.1 函数 SCB_EnableICache

函数原型：

```
__STATIC_INLINE void SCB_EnableICache (void)
{
  #if defined (__ICACHE_PRESENT) && (__ICACHE_PRESENT == 1U)
    __DSB();
```

```
    __ISB();
    SCB->ICIALLU = 0UL;                     /* invalidate I-Cache */
    __DSB();
    __ISB();
    SCB->CCR |=  (uint32_t)SCB_CCR_IC_Msk;  /* enable I-Cache */
    __DSB();
    __ISB();
  #endif
}
```

函数描述：

此函数用于使能指令 Cache，系统上电后优先初始化即可。

注意事项：

- __STATIC_INLINE：

  表示内联函数，这种类型函数的作用就是将函数直接嵌入到调用此函数的代码中，从而降低调用此函数所占用的时间。

- __DMB 指令：

  Data Memory Barrier（数据存储器隔离），DMB 指令保证所有在它前面的存储器访问操作都执行完毕后，才提交在它后面的存储器访问操作。

- __DSB 指令：

  Data Synchronization Barrier（数据同步隔离），比 DMB 严格，当所有在它前面的存储器访问操作都执行完毕后，才执行在它后面的指令。

- __ISB 指令：

  Instruction Synchronization Barrier（指令同步隔离），它会清洗流水线，以保证所有它前面的指令都执行完毕之后，才执行它后面的指令。

24.6.2 函数 SCB_DisableICache

函数原型：

```
__STATIC_INLINE void SCB_DisableICache (void)
{
  #if defined (__ICACHE_PRESENT) && (__ICACHE_PRESENT == 1U)
    __DSB();
    __ISB();
    SCB->CCR &= ~(uint32_t)SCB_CCR_IC_Msk;  /* disable I-Cache */
    SCB->ICIALLU = 0UL;                     /* invalidate I-Cache */
    __DSB();
    __ISB();
  #endif
}
```

函数描述：

此函数用于禁止指令 Cache。

注意事项：

__STATIC_INLINE，__DMB，__DSB 和__ISB 的作用看本章 24.6.1 小节的说明。

24.6.3函数 SCB_InvalidateICache

函数原型：

```
__STATIC_INLINE void SCB_InvalidateICache (void)
{
  #if defined (__ICACHE_PRESENT) && (__ICACHE_PRESENT == 1U)
    __DSB();
    __ISB();
    SCB->ICIALLU = 0UL;
    __DSB();
    __ISB();
  #endif
}
```

函数描述：

此函数用于将指令 Cache 无效化，无效化的意思是将 Cache Line 标记为无效，等同于删除操作。这样 Cache 空间就都腾出来了，可以加载新的指令。

注意事项：

__STATIC_INLINE，__DMB，__DSB 和__ISB 的作用看本章 24.6.1 小节的说明。

24.6.4函数 SCB_EnableDCache

函数原型：

```
__STATIC_INLINE void SCB_EnableDCache (void)
{
  #if defined (__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
    uint32_t ccsidr;
    uint32_t sets;
    uint32_t ways;

    SCB->CSSELR = 0U; /*(0U << 1U) | 0U;*/  /* Level 1 data cache */
    __DSB();

    ccsidr = SCB->CCSIDR;

                                          /* invalidate D-Cache */
    sets = (uint32_t)(CCSIDR_SETS(ccsidr));
    do {
      ways = (uint32_t)(CCSIDR_WAYS(ccsidr));
      do {
        SCB->DCISW = (((sets << SCB_DCISW_SET_Pos) & SCB_DCISW_SET_Msk) |
                      ((ways << SCB_DCISW_WAY_Pos) & SCB_DCISW_WAY_Msk)  );
        #if defined ( __CC_ARM )
          __schedule_barrier();
        #endif
      } while (ways-- != 0U);
    } while(sets-- != 0U);
    __DSB();
```

```
    SCB->CCR |=  (uint32_t)SCB_CCR_DC_Msk;  /* enable D-Cache */

    __DSB();
    __ISB();
  #endif
}
```

函数描述：

此函数用于使能数据 Cache，系统上电后优先初始化即可。

注意事项：

__STATIC_INLINE，__DMB，__DSB 和__ISB 的作用看本章 24.6.1 小节的说明。

24.6.5 函数 SCB_DisableDCache

函数原型：

```
__STATIC_INLINE void SCB_DisableDCache (void)
{
  #if defined (__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
    register uint32_t ccsidr;
    register uint32_t sets;
    register uint32_t ways;

    SCB->CSSELR = 0U; /*(0U << 1U) | 0U;*/  /* Level 1 data cache */
    __DSB();

    SCB->CCR &= ~(uint32_t)SCB_CCR_DC_Msk;  /* disable D-Cache */
    __DSB();

    ccsidr = SCB->CCSIDR;

                                            /* clean & invalidate D-Cache */
    sets = (uint32_t)(CCSIDR_SETS(ccsidr));
    do {
      ways = (uint32_t)(CCSIDR_WAYS(ccsidr));
      do {
        SCB->DCCISW = (((sets << SCB_DCCISW_SET_Pos) & SCB_DCCISW_SET_Msk) |
                       ((ways << SCB_DCCISW_WAY_Pos) & SCB_DCCISW_WAY_Msk)  );
        #if defined ( __CC_ARM )
          __schedule_barrier();
        #endif
      } while (ways-- != 0U);
    } while(sets-- != 0U);

    __DSB();
    __ISB();
  #endif
}
```

函数描述：

此函数用于禁止数据 Cache。

注意事项：

__STATIC_INLINE，__DMB，__DSB 和__ISB 的作用看本章 24.6.1 小节的说明

24.6.6 函数 SCB_InvalidateDCache

函数原型：

```
__STATIC_INLINE void SCB_CleanDCache (void)
{
  #if defined (__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
    uint32_t ccsidr;
    uint32_t sets;
    uint32_t ways;

     SCB->CSSELR = 0U; /*(0U << 1U) | 0U;*/  /* Level 1 data cache */
   __DSB();

    ccsidr = SCB->CCSIDR;

                                          /* clean D-Cache */
    sets = (uint32_t)(CCSIDR_SETS(ccsidr));
    do {
      ways = (uint32_t)(CCSIDR_WAYS(ccsidr));
      do {
        SCB->DCCSW = (((sets << SCB_DCCSW_SET_Pos) & SCB_DCCSW_SET_Msk) |
                      ((ways << SCB_DCCSW_WAY_Pos) & SCB_DCCSW_WAY_Msk)  );
        #if defined ( __CC_ARM )
          __schedule_barrier();
        #endif
      } while (ways-- != 0U);
    } while(sets-- != 0U);

    __DSB();
    __ISB();
  #endif
}
```

函数描述：

此函数用于将数据 Cache 无效化，无效化的意思是将 Cache Line 标记为无效，等同于删除操作。这样 Cache 空间就都腾出来了，可以加载新的数据。

注意事项：

__STATIC_INLINE，__DMB，__DSB 和__ISB 的作用看本章 24.6.1 小节的说明。

24.6.7 函数 SCB_CleanDCache

函数原型：

```
__STATIC_INLINE void SCB_CleanDCache (void)
{
  #if defined (__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
    uint32_t ccsidr;
    uint32_t sets;
    uint32_t ways;
```

```
    SCB->CSSELR = 0U; /*(0U << 1U) | 0U;*/  /* Level 1 data cache */
  __DSB();

    ccsidr = SCB->CCSIDR;

                                          /* clean D-Cache */
    sets = (uint32_t)(CCSIDR_SETS(ccsidr));
    do {
      ways = (uint32_t)(CCSIDR_WAYS(ccsidr));
      do {
        SCB->DCCSW = (((sets << SCB_DCCSW_SET_Pos) & SCB_DCCSW_SET_Msk) |
                      ((ways << SCB_DCCSW_WAY_Pos) & SCB_DCCSW_WAY_Msk)  );
        #if defined ( __CC_ARM )
          __schedule_barrier();
        #endif
      } while (ways-- != 0U);
    } while(sets-- != 0U);

    __DSB();
    __ISB();
  #endif
}
```

函数描述：

此函数用于将数据 Cache 清除，清除的意思是将 Cache Line 中标记为 dirty 的数据写入到相应的存储区。

注意事项：

__STATIC_INLINE，__DMB，__DSB 和__ISB 的作用看本章 24.6.1 小节的说明。

24.6.8函数 SCB_CleanInvalidateDCache

函数原型：

```
__STATIC_INLINE void SCB_CleanInvalidateDCache (void)
{
  #if defined (__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
    uint32_t ccsidr;
    uint32_t sets;
    uint32_t ways;

    SCB->CSSELR = 0U; /*(0U << 1U) | 0U;*/  /* Level 1 data cache */
    __DSB();

    ccsidr = SCB->CCSIDR;

                                          /* clean & invalidate D-Cache */
    sets = (uint32_t)(CCSIDR_SETS(ccsidr));
    do {
      ways = (uint32_t)(CCSIDR_WAYS(ccsidr));
      do {
        SCB->DCCISW = (((sets << SCB_DCCISW_SET_Pos) & SCB_DCCISW_SET_Msk) |
                       ((ways << SCB_DCCISW_WAY_Pos) & SCB_DCCISW_WAY_Msk)  );
        #if defined ( __CC_ARM )
          __schedule_barrier();
```

```
        #endif
      } while (ways-- != 0U);
    } while(sets-- != 0U);

    __DSB();
    __ISB();
  #endif
}
```

函数描述：

此函数是前面两个函数 SCB_InvalidateDCache 和 SCB_CleanDCache 的二合一。将 Cache Line 中标记为 dirty 的数据写入到相应的存储区后，再将 Cache Line 标记为无效，表示删除。这样 Cache 空间就都腾出来了，可以加载新的数据。

注意事项：

__STATIC_INLINE，__DMB，__DSB 和__ISB 的作用看本章 24.6.1 小节的说明。

24.6.9 函数 SCB_InvalidateDCache_by_Addr

函数原型：

```
__STATIC_INLINE void SCB_InvalidateDCache_by_Addr (uint32_t *addr, int32_t dsize)
{
  #if defined (__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
     int32_t op_size = dsize;
    uint32_t op_addr = (uint32_t)addr;
     int32_t linesize = 32;           /* in Cortex-M7 size of cache line is fixed to 8 words (32 bytes) */

    __DSB();

    while (op_size > 0) {
      SCB->DCIMVAC = op_addr;
      op_addr += (uint32_t)linesize;
      op_size -=           linesize;
    }

    __DSB();
    __ISB();
  #endif
}
```

函数描述：

此函数与本章 24.6.6 小节中讲解的函数作用一样，区别是这里可以指定地址和存储区大小。用于将数据 Cache 无效化，无效化的意思是将 Cache Line 标记为无效，等同于删除操作。这样 Cache 空间就都腾出来了，可以加载新的数据。

注意事项：

- __STATIC_INLINE，__DMB，__DSB 和__ISB 的作用看本章 24.6.1 小节的说明。
- 第 1 个参数 addr ： 操作的地址一定要是 32 字节对齐的，即这个地址对 32 求余数等于 0。
- 第 2 个参数 dsize ： 一定要是 32 字节的整数倍。

24.6.10 函数 SCB_CleanDCache_by_Addr

函数原型：

```
__STATIC_INLINE void SCB_CleanDCache_by_Addr (uint32_t *addr, int32_t dsize)
{
  #if defined (__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
     int32_t op_size = dsize;
    uint32_t op_addr = (uint32_t) addr;
     int32_t linesize = 32;            /* in Cortex-M7 size of cache line is fixed to 8 words (32 bytes) */

    __DSB();

    while (op_size > 0) {
      SCB->DCCMVAC = op_addr;
      op_addr += (uint32_t)linesize;
      op_size -=           linesize;
    }

    __DSB();
    __ISB();
  #endif
}
```

函数描述：

此函数与本章 24.6.7 小节中讲解的函数作用一样，区别是这里可以指定地址和存储区大小。用于将数据 Cache 清除，清除的意思是将 Cache Line 中标记为 dirty 的数据写入到相应的存储区。

注意事项：

- __STATIC_INLINE，__DMB，__DSB 和__ISB 的作用看本章 24.6.1 小节的说明。
- 第 1 个参数 addr ： 操作的地址一定要是 32 字节对齐的，即这个地址对 32 求余数等于 0。
- 第 2 个参数 dsize ： 一定要是 32 字节的整数倍。

24.6.11 函数 SCB_CleanInvalidateDCache_by_Addr

函数原型：

```
__STATIC_INLINE void SCB_CleanInvalidateDCache_by_Addr (uint32_t *addr, int32_t dsize)
{
  #if defined (__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
     int32_t op_size = dsize;
    uint32_t op_addr = (uint32_t) addr;
     int32_t linesize = 32;          /* in Cortex-M7 size of cache line is fixed to 8 words (32 bytes) */

    __DSB();

    while (op_size > 0) {
      SCB->DCCIMVAC = op_addr;
      op_addr += (uint32_t)linesize;
      op_size -=           linesize;
    }
```

```
    __DSB();
    __ISB();
  #endif
}
```

函数描述：

此函数与本章 24.6.8 小节中讲解的函数作用一样，区别是这里可以指定地址和存储区大小。将 Cache Line 中标记为 dirty 的数据写入到相应的存储区后，再将 Cache Line 标记为无效，表示删除。这样 Cache 空间就都腾出来了，可以加载新的数据。

注意事项：

- __STATIC_INLINE，__DMB，__DSB 和__ISB 的作用看本章 24.6.1 小节的说明。
- 第 1 个参数 addr ： 操作的地址一定要是 32 字节对齐的，即这个地址对 32 求余数等于 0。
- 第 2 个参数 dsize ： 一定要是 32 字节的整数倍。

24.7 总结

本章节就为大家讲解这么多，务必要反复多读几遍，能理解多少算多少，随着后续章节的学习再回过头来再学习。

第25章 STM32H7 的 TCM，SRAM 等五块内存基础知识

本章教程为大家介绍 STM32H7 带的 ITCM，DTCM，AXI SRAM，SRAM1，SRAM2，SRAM3，SRAM4 和备份 SRAM 的基础知识。

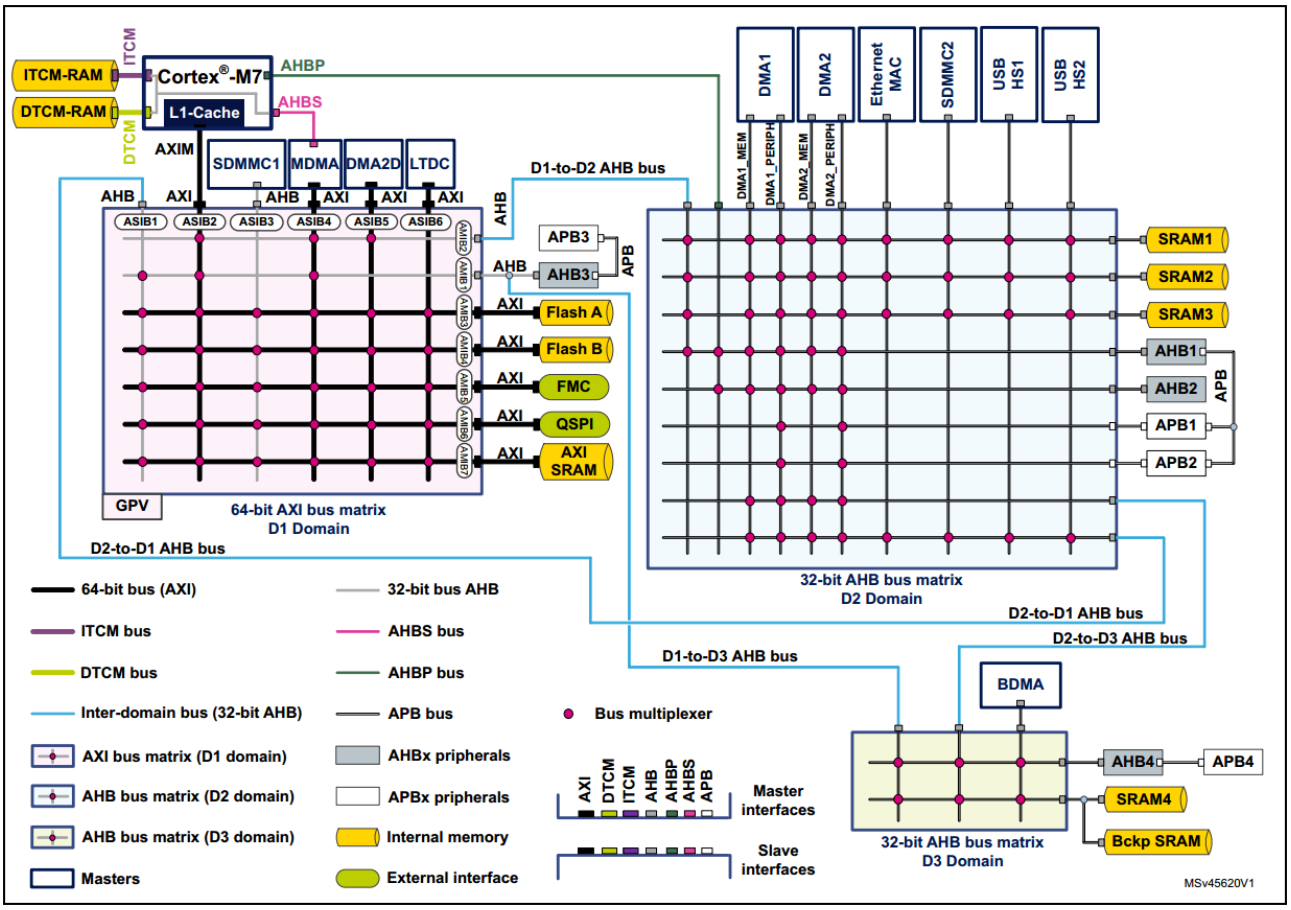
25.1 初学者重要提示
25.2 各块 RAM 在总线中的位置
25.3 各块 RAM 的特性
25.4 各块 RAM 的时钟问题
25.5 各块 RAM 的 DMA 问题
25.6 实际工程推荐的 RAM 分配方案
25.7 总结

25.1 初学者重要提示

- TCM：Tightly-Coupled Memory 紧密耦合内存 。ITCM 用于指令，DTCM 用于数据，特点是跟内核速度一样，而片上 RAM 的速度基本都达不到这个速度。
- 特别注意本章 25.5 小节里面各块 RAM 的 DMA 操作问题。

25.2 各块 RAM 在总线中的位置

这个知识点在前面章节做过介绍，本章再次梳理下这个知识点。



这个图可以方便识别总线所外挂的外设，共分为三个域：D1 Domain，D2 Domain 和 D3 Domain。

- ITCM 和 DTCM

  这两个是直连 CPU 的。

- D1 Domain

  D1 域中的各个外设是挂在 64 位 AXI 总线组成 6*7 的矩阵上。

  - 6 个从接口端 ASIB1 到 ASIB6

    外接的主控是 LTDC，DMA2D，MDMA，SDMMC1，AXIM 和 D2-to-D1 AHB 总线。

  - 7 个主接口端 AMIB1 到 AMIB7

    外接的从设备是 AHB3 总线，Flash A，Flash B，FMC 总线，QSPI 和 AXI SRAM。另外 AHB3 也是由 AXI 总线分支出来的，然后再由 AHB3 分支出 APB3 总线。

- D2 Domain

  D2 域的各个外设是挂在 32 位 AHB 总线组成 10*9 的矩阵上。

  - 10 个从接口

    外接的主控是 D1-to-D2 AHB 总线，AHBP 总线，DMA1，DMA2，Ethernet MAC，SDMMC2，USB HS1 和 USB HS2。

  - 9 个主接口

外接的从设备是 SRAM1，SRMA2，SRAM3，AHB1，AHB2，APB1，APB2，D2-to-D1 AHB 总线和 D2-to-D3 AHB 总线。

- D3 Domain

D3 域的各个外设是挂在 32 位 AHB 总线组成 3*2 的矩阵上。

- 3 个从接口

外接的主控 D1-to-D3 AHB 总线，D2-to-D3 AHB 总线和 BDMA。

- 2 个主接口

外接的从设备是 AHB4，SRAM4 和 Bckp SRAM。另外 AHB4 也是这个总线矩阵分支出来的，然后再由 AHB4 分支出 APB4 总线

25.3 各块 RAM 特性

各块 RAM 的特性对比如下，特别注意他们支持的最大速度和容量大小。

- TCM 区

TCM : Tightly-Coupled Memory 紧密耦合内存 。ITCM 用于运行指令，也就是程序代码，DTCM 用于数据存取，特点是跟内核速度一样，而片上 RAM 的速度基本都达不到这个速度，所以有降频处理。

速度：400MHz。

DTCM 地址：0x2000 0000，大小 128KB。

ITCM 地址：0x0000 0000，大小 64KB。

- AXI SRAM 区

位于 D1 域，数据带宽是 64bit，挂在 AXI 总线上。除了 D3 域中的 BDMB 主控不能访问，其它都可以访问此 RAM 区。

速度：200MHz。

地址：0x2400 0000，大小 512KB。

用途：用途不限，可以用于用户应用数据存储或者 LCD 显存。

- SRAM1，SRAM2 和 SRAM3 区

位于 D2 域,数据带宽是 32bit,挂在 AHB 总线上。除了 D3 域中的 BDMB 主控不能访问这三块 SRAM，其它都可以访问这几个 RAM 区。

速度：200MHz。

SRAM1：地址 0x3000 0000，大小 128KB，用途不限，可用于 D2 域中的 DMA 缓冲，也可以当 D1 域断电后用于运行程序代码。

SRAM2：地址 0x3002 0000，大小 128KB，用途不限，可用于 D2 域中的 DMA 缓冲，也可以用于用户数据存取。

SRAM3：地址 0x3004 0000，大小 32KB，用途不限，主要用于以太网和 USB 的缓冲。

- SRAM4 区

  位于 D3 域，数据带宽是 32bit，挂在 AHB 总线上，大部分主控都能访这块 SRAM 区。

  速度：200MHz。

  地址：0x3800 0000，大小 64KB。

  用途：用途不限，可以用于 D3 域中的 DMA 缓冲，也可以当 D1 和 D2 域进入 DStandby 待机方式后，继续保存用户数据。

- Backup SRAM 区

  备份 RAM 区，位于 D3 域，数据带宽是 32bit，挂在 AHB 总线上，大部分主控都能访问这块 SRAM 区。

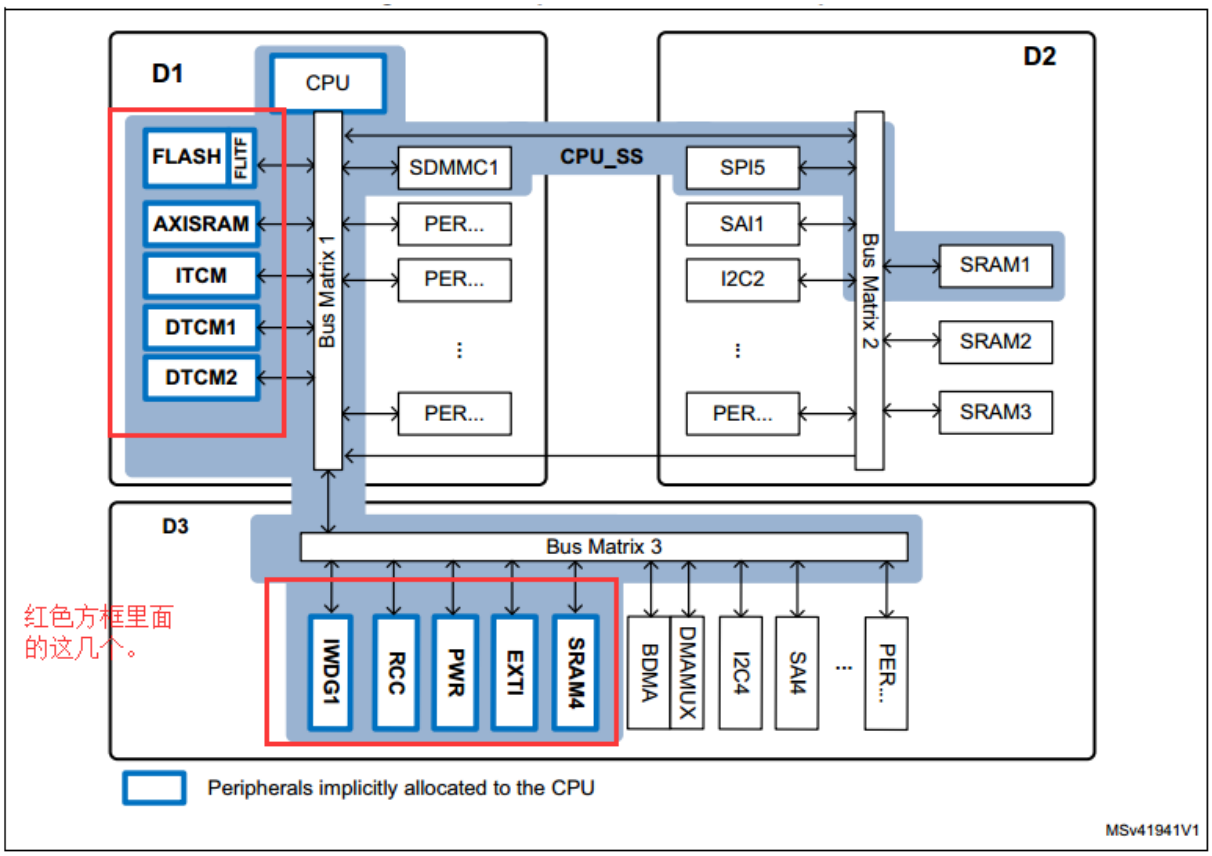
  速度：200MHz。

  地址：0x3880 0000，大小 4KB。

  用途：用途不限，主要用于系统进入低功耗模式后，继续保存数据（Vbat 引脚外接电池）。

25.4 各块 RAM 的时钟问题

正常情况下，系统上电后，CPU 要访问的外设是需要使能对应的时钟位，但是下面这几个，CPU 上电即可访问，而且芯片没有对应的寄存器使能位。



也就是说 AXI SRAM，SRAM4，ITCM 和 DTCM 可以在上电后直接使用。而 SRAM1，SRAM2，SRAM3 是需要使能的，但是实际测试发现，不使能也可以正常使用。不过，建议用到时候开启下时钟，防止意想不到的问题发生。

对于 V7 板子配套的例子，在 bsp.c 文件的函数 SystemClock_Config 末尾做了个条件编译，大家可以根据需要来开启这三个时钟：

```
#if 0
    __HAL_RCC_D2SRAM1_CLK_ENABLE();
    __HAL_RCC_D2SRAM2_CLK_ENABLE();
    __HAL_RCC_D2SRAM3_CLK_ENABLE();
#endif
```

25.5 各块 RAM 的 DMA 问题

了解这个问题之前，要先看下面的 Bus Master 总线主控端和 Bus Slave 设备端的控制互联：

	Bus master / type(1)																			
	Cortex-M7 - AXIM	Cortex-M7 - AHBP	**Cortex-M7 - ITCM**	**Cortex-M7-DTCM**	SDMMC1	**MDMA - AXI**	MDMA - AHBS	**DMA2D**	**LTDC**	DMA1 - MEM	DMA1 - PERIPH	DMA2 - MEM	DMA2 - PERIPH	Eth. MAC - AHB	SDMMC2 - AHB	USBHS1 - AHB	USBHS2 - AHB	BDMA - AHB		
Bus slave / type(1)	Interconnect path and type(2)																			
ITCM	-	-	D	-	-	-	7	-	-	-	-	-	-	-	-	-	-	-	-	-
DTCM	-	-	-	D	-	-	7	-	-	-	-	-	-	-	-	-	-	-	-	-
AHB3 peripherals	1	-	-	-	-	1	-	-	-	21	21	-	21	21	-	21	21	21	21	-
APB3 peripherals	14	-	-	-	-	14	-	-	-	214	214	-	214	214	-	214	214	214	214	-
Flash A	1	-	-	-	1	1	-	1	1	21	21	-	21	21	-	21	21	21	21	-
Flash B	1	-	-	-	1	1	-	1	1	21	21	-	21	21	-	21	21	21	21	-
AXI SRAM	1	-	-	-	1	1	-	1	1	21	21	-	21	21	-	21	21	21	21	-
QUADSPI	1	-	-	-	1	1	-	1	1	21	21	-	21	21	-	21	21	21	21	-
FMC	1	-	-	-	1	1	-	1	1	21	21	-	21	21	-	21	21	21	21	-
SRAM 1	12	-	-	-	-	12	-	12	-	2	2	-	2	2	-	2	2	2	2	-
SRAM 2	12	-	-	-	-	12	-	12	-	2	2	-	2	2	-	2	2	2	2	-
SRAM 3	12	-	-	-	-	12	-	12	-	2	2	-	2	2	-	2	2	2	2	-
AHB1 peripherals	12	2	-	-	-	12	-	12	-	2	2	-	2	2	-	-	-	-	-	-
APB1 peripherals	125	25	-	-	-	125	-	125	-	25	25	2	25	25	2	-	-	-	-	-
AHB2 peripherals	-	2	-	-	-	-	-	-	-	2	2	-	2	2	-	-	-	-	-	-
APB2 peripherals	125	25	-	-	-	125	-	125	-	25	25	2	25	25	2	-	-	-	-	-
AHB4 peripherals	13	-	-	-	-	13	-	-	-	23	23	-	23	23	-	213	213	213	213	3
APB4 peripherals	136	-	-	-	-	136	-	-	-	236	236	-	236	236	-	213	213	213	213	36
SRAM4	13	-	-	-	-	13	-	-	-	23	23	-	23	23	-	213	213	213	213	3
Backup RAM	13	-	-	-	-	13	-	-	-	23	23	-	23	23	-	213	213	213	213	3

- 加粗字体是 64 位总线（ITCM，DTCM，Flash A，Flash，AXI SRAM，FMC 等），普通字体是 32 位总线。

- 访问通路（每个小方块里面的字符）

  任何有数字的表示有访问通路。

  短横杠“-”表示不可访问。

  有灰色阴影的表示有实用价值的访问通路。

- 表格中具体数值所代表的含义

  D=direct,

  1=via AXI bus matrix,

  2=via AHB bus matrix in D2,

  3=via AHB bus matrix in D3,

  4=via AHB/APB bridge in D1,

  5=via AHB/APB bridge in D2,

  6=via AHB/APB bridge in D3,

  7=via AHBS bus of Cortex-M7,

  多个数值组合 = 互连路径以数字的顺序经过多个矩阵或/和桥。

- 总线访问类型

  普通字体表示 32 位总线。

  斜体表示 32 位总线主机端/ 64 位总线从机端。

  粗体表示 64 位总线。

通过这个总线互联图，要了解到下面三个重要知识点：

- DTCM 和 ITCM 不支持 DMA1，DMA2 和 BDMA，仅支持 MDMA。
- AXI SRAM，SRAM1，SRAM2，SRAM3 不支持 BDMA，支持 MDMA，DMA1 和 DMA2。
- SRAM4 支持所有 DMA，即 MDMA，DMA1，DMA2 和 BDMA。

25.6 实际工程推荐的 RAM 分配方案

鉴于 DTCM 是 400MHz 的，而其它的 RAM 都是 200MHz，推荐工程的主 RAM 空间采用 TCM，而其它需要大 RAM 或者 DMA 的场合，使用剩余 RAM 空间。

本教程配套的例子基本都是采用的这个方案，让 TCM 的性能得到最大发挥。

25.7 总结

本章节就为大家讲解这么多，通过本章节主要是为后面三个章节的学习做铺垫。

第26章 STM32H7 的 TCM，SRAM 等五块内存的超方便使用方式

本章教程为大家分享一种快捷的 DTCM，SRAM1，SRAM2，SRAM3 和 SRAM4 的使用方式。MDK 和 IAR 均支持这种方式。

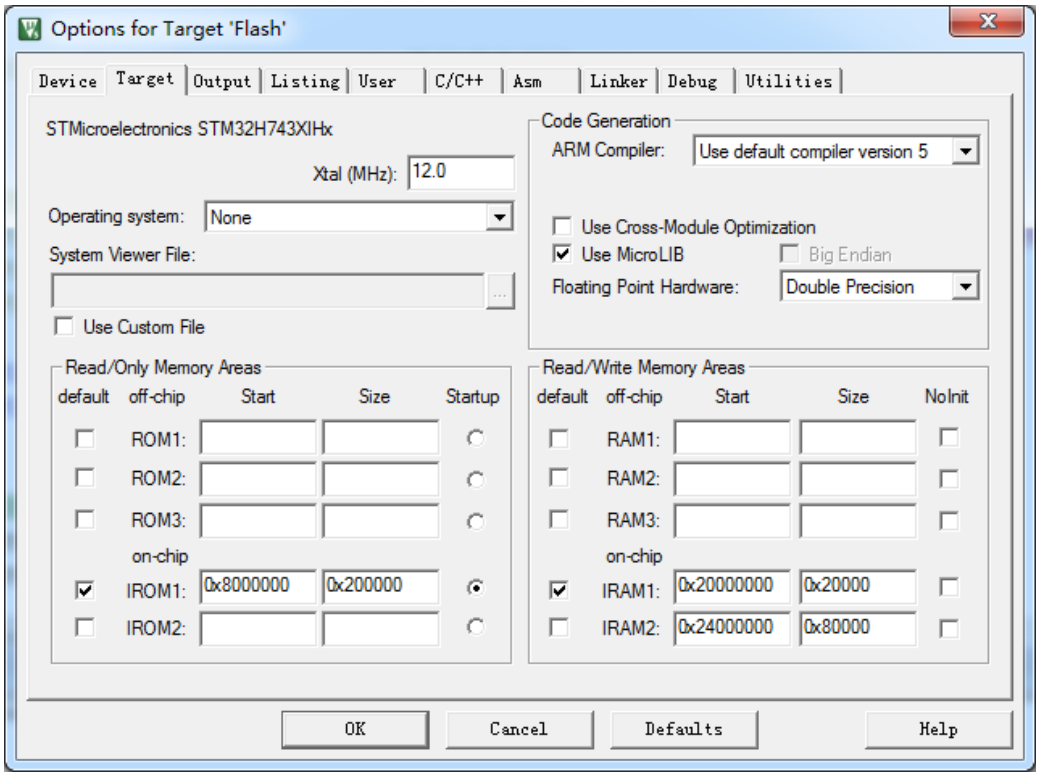
26.1 初学者重要提示
26.2 MDK 分散加载方式管理多块内存区方法
26.3 MDK 分散加载文件解读
26.4 IAR 的 ICF 文件设置
26.5 实验例程说明（MDK）
26.6 实验例程说明（IAR）
26.7 总结

26.1 初学者重要提示

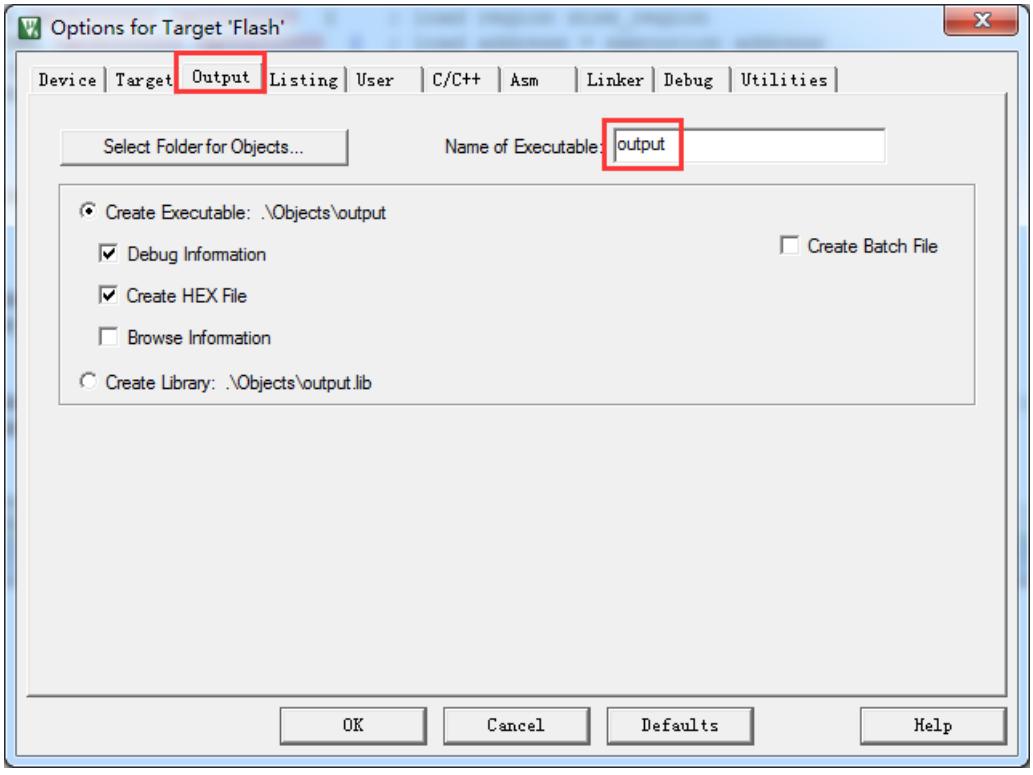
- 学习本章节前，务必优先学习第 25 章，了解 TCM，SRAM 等五块内存区的基础知识，比较重要。
- 本章的管理方式比较容易实现，仅需添加一个分散加载文件即可，对应的分散加载内容也比较好理解。

26.2 MDK 分散加载方式管理多块内存区方法

默认情况下，我们都是通过 MDK 的 option 选项设置 Flash 和 RAM 大小：



这种情况下，所有管理工作都是编译来处理的。针对这个配置，在路径\Project\MDK-ARM(uV5)\Objects（本教程配套例子的路径）里面会自动生成一个后缀为 sct 的文件 output.sct。文件名由下面这个选项决定的：



output.sct 文件生成的内容如下：

```
; *************************************************************
```

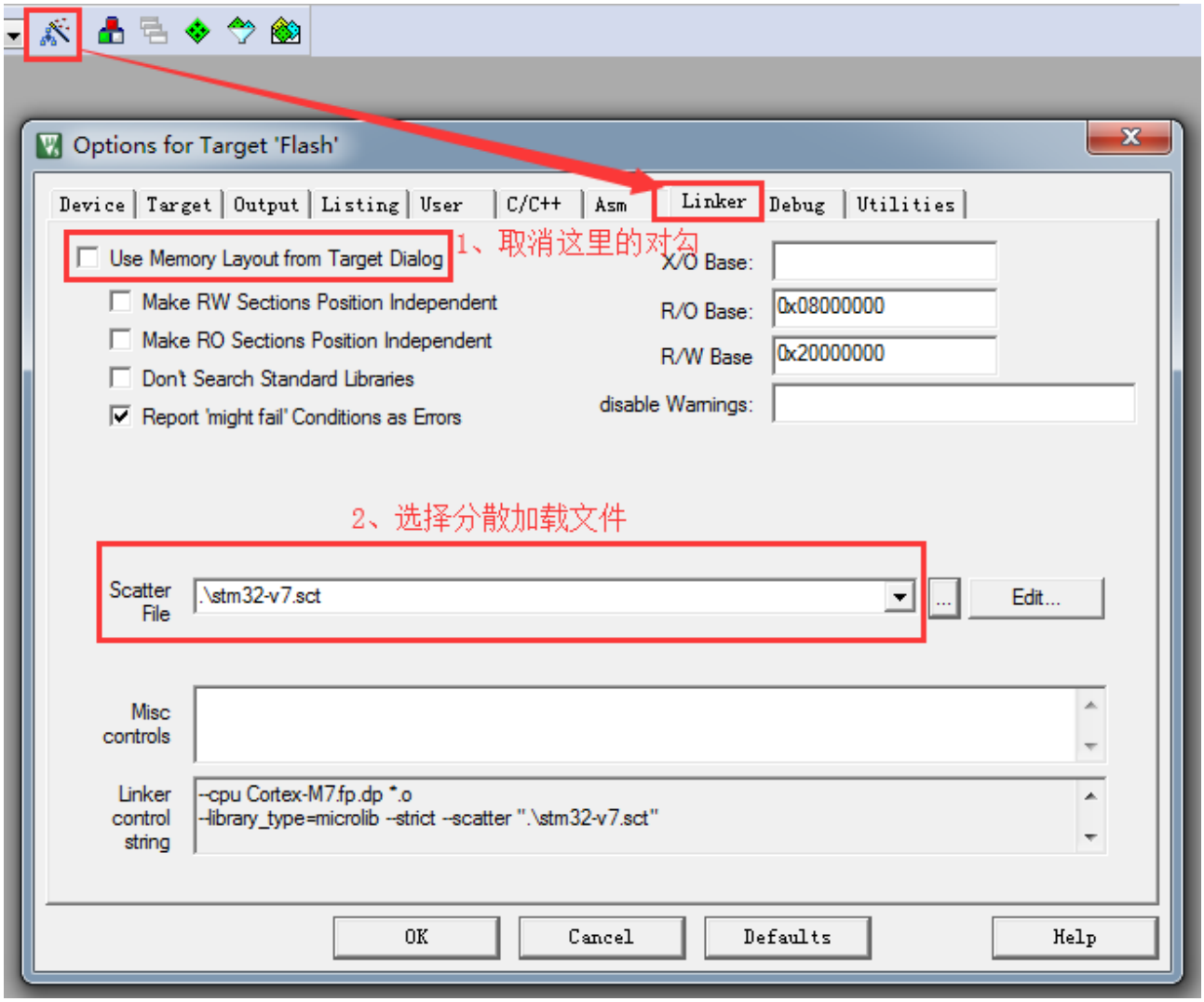
```
; *** Scatter-Loading Description File generated by uVision ***
; ************************************************************

LR_IROM1 0x08000000 0x00200000  {    ; load region size_region
  ER_IROM1 0x08000000 0x00200000  {  ; load address = execution address
   *.o (RESET, +First)
   *(InRoot$$Sections)
   .ANY (+RO)
   .ANY (+XO)
  }
  RW_IRAM1 0x20000000 0x00020000  {  ; RW data
   .ANY (+RW +ZI)
  }
}
```

不方便用户将变量定义到指定的CCM 或者SDRAM中。而使用__attribute__指定具体地址又不方便管理。

针对这种情况，使用一个脚本文件即可解决，脚本定义如下：

```
LR_IROM1 0x08000000 0x00200000  {    ; load region size_region
  ER_IROM1 0x08000000 0x00200000  {  ; load address = execution address
   *.o (RESET, +First)
   *(InRoot$$Sections)
   .ANY (+RO)
  }

  ; RW data - 128KB DTCM
  RW_IRAM1 0x20000000 0x00020000  {
   .ANY (+RW +ZI)
  }

  ; RW data - 512KB AXI SRAM
  RW_IRAM2 0x24000000 0x00080000  {
   *(.RAM_D1)
  }

  ; RW data - 128KB SRAM1(0x30000000) + 128KB SRAM2(0x3002 0000) + 32KB SRAM3(0x30040000)
  RW_IRAM3 0x30000000 0x00048000  {
   *(.RAM_D2)
  }

  ; RW data - 64KB SRAM4(0x38000000)
  RW_IRAM4 0x38000000 0x00010000  {
   *(.RAM_D3)
  }
}
```

同时配置 option 的链接选项使用此分散加载文件：



使用方法很简单，依然是使用__attribute__，但是不指定具体地址了，指定 RAM 区，方法如下，仅需加个前缀即可：

```
/* 定义在 512KB AXI SRAM 里面的变量 */
__attribute__((section (".RAM_D1"))) uint32_t AXISRAMBuf[10];
__attribute__((section (".RAM_D1"))) uint16_t AXISRAMCount;

/* 定义在 128KB SRAM1(0x30000000) + 128KB SRAM2(0x30020000) + 32KB SRAM3(0x30040000)里面的变量 */
__attribute__((section (".RAM_D2"))) uint32_t D2SRAMBuf[10];
__attribute__((section (".RAM_D2"))) uint16_t D2SRAMount;

/* 定义在 64KB SRAM4(0x38000000)里面的变量 */
__attribute__((section (".RAM_D3"))) uint32_t D3SRAMBuf[10];
__attribute__((section (".RAM_D3"))) uint16_t D3SRAMCount;
```

26.3 MDK 分散加载文件解读

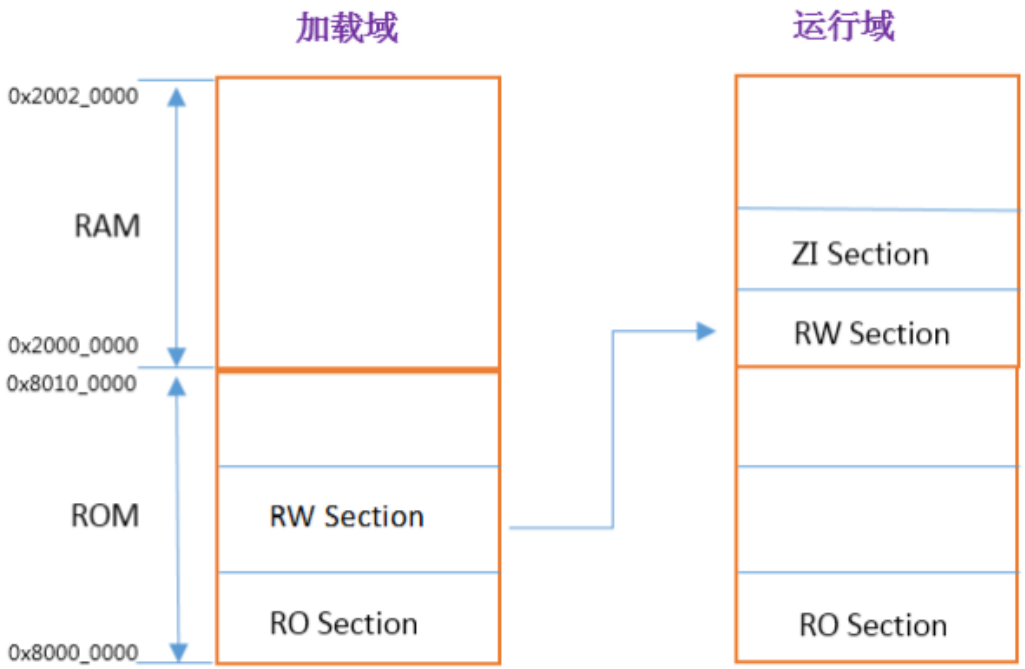
这里将分散加载文件的内容为大家做个解读，方便以后自己修改：

```
1.   LR_IROM1 0x08000000 0x00200000  {    ; load region size_region
2.     ER_IROM1 0x08000000 0x00200000  {  ; load address = execution address
3.      *.o (RESET, +First)
4.      *(InRoot$$Sections)
```

```
5.       .ANY (+RO)
6.     }
7.
8.     ; RW data - 128KB DTCM
9.     RW_IRAM1 0x20000000 0x00020000  {
10.     .ANY (+RW +ZI)
11.    }
12.
13.    ; RW data - 512KB AXI SRAM
14.    RW_IRAM2 0x24000000 0x00080000  {
15.     *(.RAM_D1)
16.    }
17.
18.    ; RW data - 128KB SRAM1(0x30000000) + 128KB SRAM2(0x3002 0000) + 32KB SRAM3(0x30040000)
19.    RW_IRAM3 0x30000000 0x00048000  {
20.     *(.RAM_D2)
21.    }
22.
23.    ; RW data - 64KB SRAM4(0x38000000)
24.    RW_IRAM4 0x38000000 0x00010000  {
25.     *(.RAM_D3)
26.    }
27. }
```

- 第 1 – 2 行，LR_IROM1 是 Load Region 加载域，ER_IROM1 是 Execution Region 执行域。首地址都是 0x0800 0000，大小都是 0x0020 0000，即 STM32H7 的 Flash 地址和对应大小。

  加载域就是程序在 Flash 中的实际存储，而运行域是芯片上电后的运行状态，通过下面的框图可以有一个感性的认识：



  通过上面的框图可以看出，RW 区也是要存储到 ROM/Flash 里面的，在执行映像之前，必须将已初始化的 RW 数据从 ROM 中复制到 RAM 中的执行地址并创建 ZI Section(初始化为 0 的变量区)。

- 第 3 行的*.o (RESET, +First)

在启动文件 startup_stm32h743xx.s 有个段名为 RESET 的代码段，主要存储了中断向量表。这里是将其存放在 Flash 的首地址。

- 第 4 行的*(InRoot$$Sections)
  这里是将 MDK 的一些库文件全部放在根域，比如__main.o， _scatter*.o， _dc*.o。
- 第 5 行.ANY (+RO)
  将目标文件中所有具有 RO 只读属性的数据放在这里，即 ER_IROM1。
- 第 9-11 行，RW_IRAM1 是执行域，配置的是 DTCM，首地址 0x2000 0000，大小 128KB。
  将目标文件中所有具有 RW 和 ZI 数据放在这里。
- 第 14-16 行，RW_IRAM2 是执行域，配置的是 AXI SRAM，首地址 0x24000000，大小 512KB。给这个域专门配了一个名字 .RAM_D1。这样就可以通过__attribute__((section("name")))将其分配到这个 RAM 域。
- 第 19-21 行，RW_IRAM3 是执行域，配置的是 D2 域的 SRAM1，SRAM2 和 SRAM3，首地址 0x30000000，共计大小 288KB。给这个域专门配了一个名字 .RAM_D2。这样就可以通过__attribute__((section("name")))将其分配到这个 RAM 域。
- 第 24-26 行，RW_IRAM3 是执行域，配置的是 D3 域的 SRAM4，首地址 0x38000000，共计大小 64KB。给这个域专门配了一个名字 .RAM_D3。这样就可以通过__attribute__((section("name")))将其分配到这个 RAM 域。

26.4 IAR 的 ICF 文件设置

IAR 相比 MDK 的设置要简单一些，仅需在 IAR 的配置文件 stm32h743xx_flash.icf 中添加如下代码即可：

```
define region RAM_D1_region  = mem:[from 0x24000000 to 0x24080000];
define region RAM_D2_region  = mem:[from 0x30000000 to 0x30048000];
define region RAM_D3_region  = mem:[from 0x38000000 to 0x38010000];
place in RAM_D1_region {section .RAM_D1};
place in RAM_D2_region {section .RAM_D2};
place in RAM_D3_region {section .RAM_D3};
```

用户的使用方法如下：

```
/* 定义在 512KB AXI SRAM 里面的变量 */
#pragma location = ".RAM_D1"
uint32_t AXISRAMBuf[10];
#pragma location = ".RAM_D1"
uint16_t AXISRAMCount;

/* 定义在 128KB SRAM1(0x30000000) + 128KB SRAM2(0x30020000) + 32KB SRAM3(0x30040000)里面的变量 */
#pragma location = ".RAM_D2"
uint32_t D2SRAMBuf[10];
#pragma location = ".RAM_D2"
uint16_t D2SRAMount;

/* 定义在 64KB SRAM4(0x38000000)里面的变量 */
#pragma location = ".RAM_D3"
```

```
uint32_t D3SRAMBuf[10];
#pragma location = ".RAM_D3"
uint16_t D3SRAMCount;
```

26.5 实验例程说明（MDK）

配套例子：

V7-005_TCM，SRAM 等五块内存的超方便使用方式

实验目的：

1. 学习 TCM，SRAM 等五块内存的超方便使用方式。

实验内容：

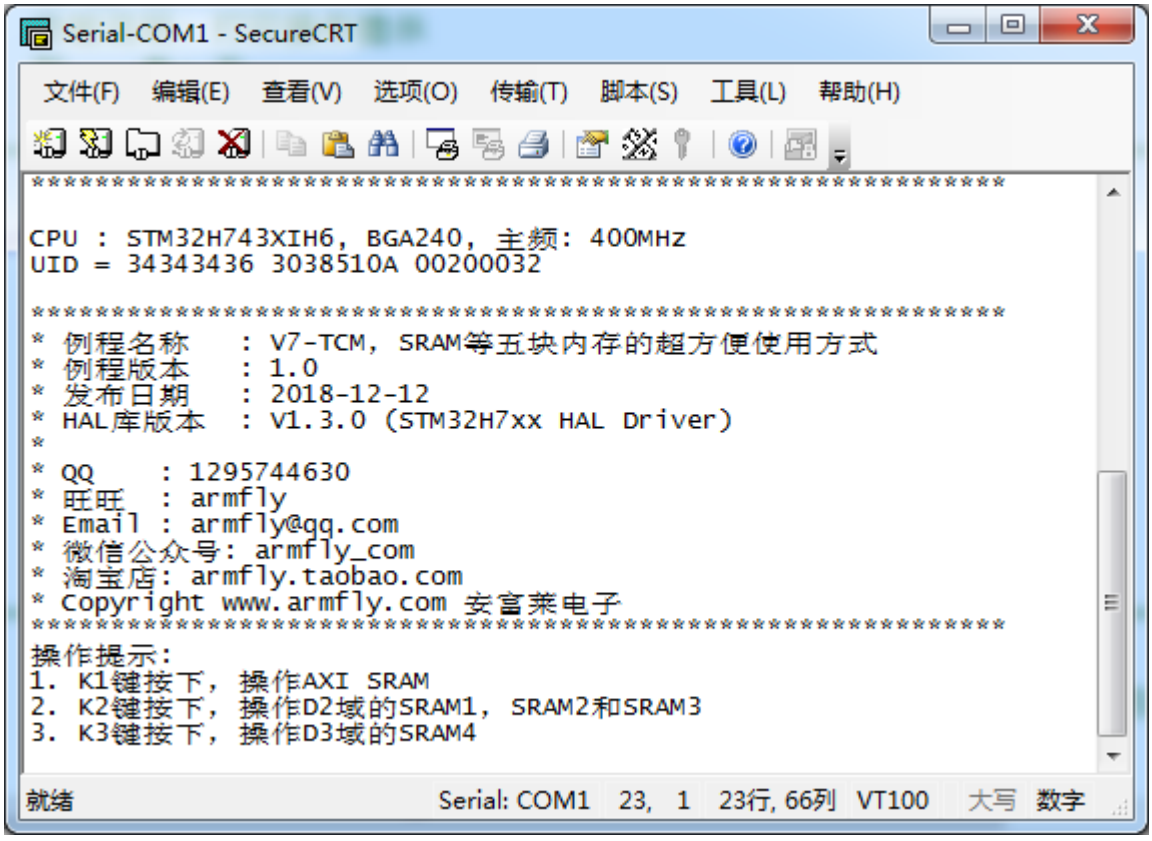
1. 启动自动重装软件定时器 0，每 100ms 翻转一次 LED2。

实验操作：

1. K1 键按下，操作 AXI SRAM。
2. K2 键按下，操作 D2 域的 SRAM1，SRAM2 和 SRAM3。
3. K3 键按下，操作 D3 域的 SRAM4。

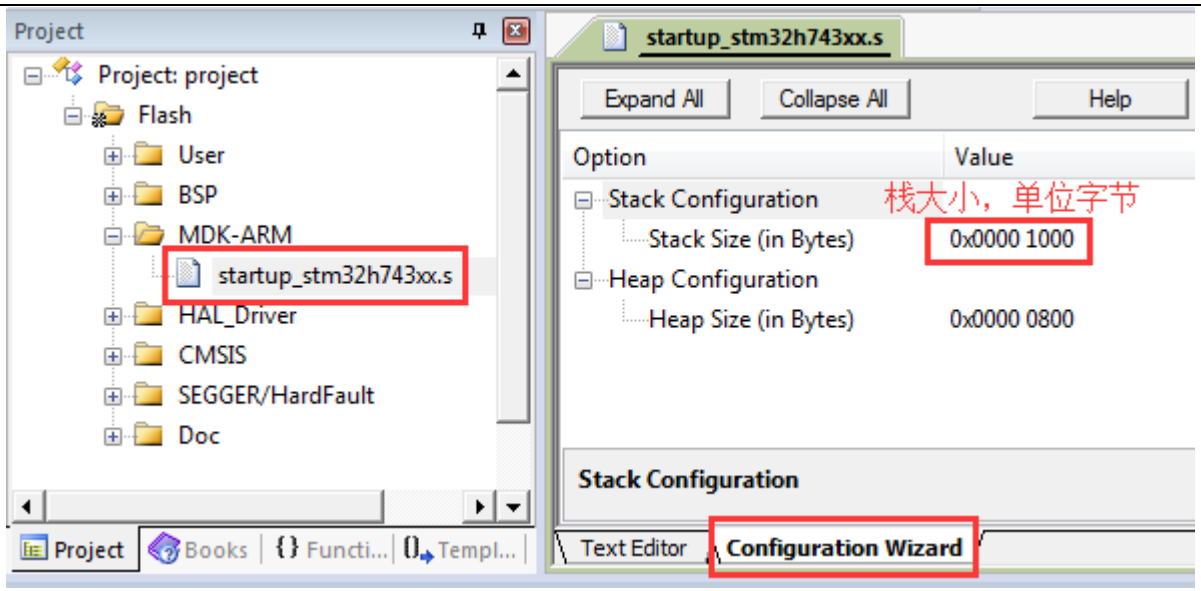
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

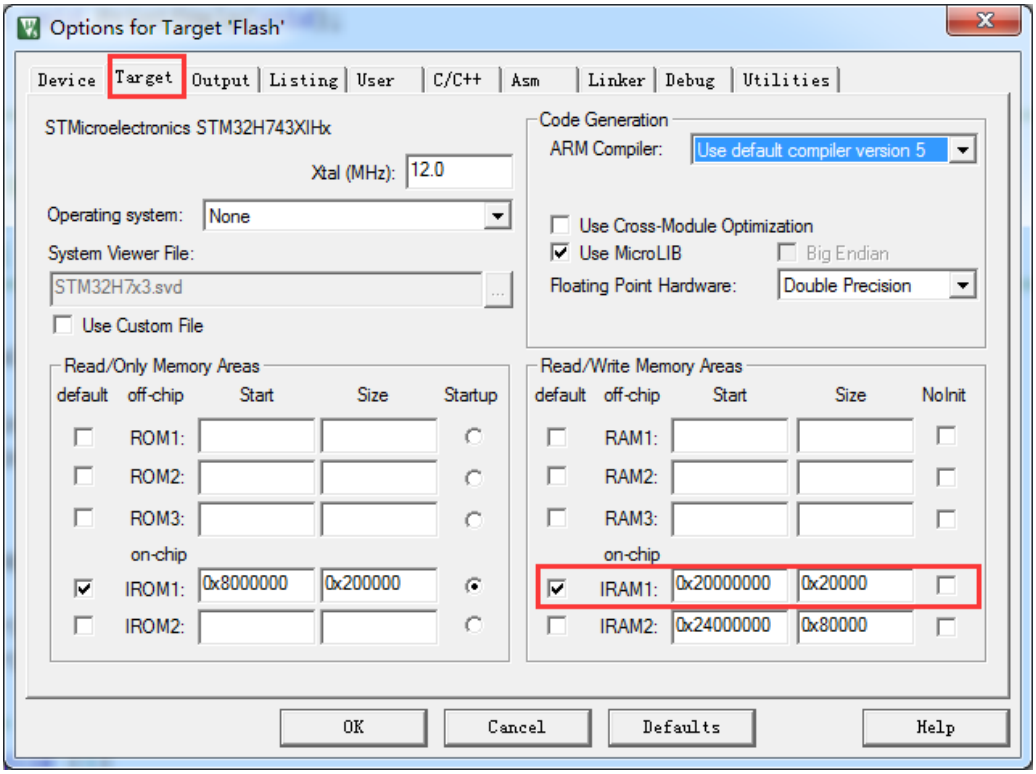


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。

  AXI SRAM 的 MPU 属性:

  Write back, Read allocate，Write allocate。

  FMC 的扩展 IO 的 MPU 属性:

  必须 Device 或者 Strongly Ordered。

  D2 SRAM1，SRAM2 和 SRAM3 的 MPU 属性:

  Write through, read allocate，no write allocate。

  D3 SRAM4 的 MPU 属性:

  Write through, read allocate，no write allocate。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
```

```
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM1 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x30000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_128KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

```
    /* 配置 SRAM2 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x30020000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_128KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER3;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 SRAM3 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x30040000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_32KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER4;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 SRAM4 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER5;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主功能的实现主要分为两部分：

- 启动自动重装软件定时器 0，每 100ms 翻转一次 LED2。

- K1 键按下，操作 AXI SRAM。
- K2 键按下，操作 D2 域的 SRAM1，SRAM2 和 SRAM3。
- K3 键按下，操作 D3 域的 SRAM4

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    AXISRAMCount = 0;
    D2SRAMount = 0;
    D3SRAMCount = 0;

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
```

```
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下, 操作 AXI SRAM */
                    AXISRAMBuf[0] = AXISRAMCount++;
                    AXISRAMBuf[5] = AXISRAMCount++;
                    AXISRAMBuf[9] = AXISRAMCount++;
                    printf("K1 键按下, AXISRAMBuf[0] = %d, AXISRAMBuf[5] = %d, AXISRAMBuf[9] = %d\r\n",
                                                                                AXISRAMBuf[0],
                                                                                AXISRAMBuf[5],
                                                                                AXISRAMBuf[9]);
                    break;

                case KEY_DOWN_K2:           /* K2 键按下, 操作 D2 域的 SRAM1, SRAM2 和 SRAM3 */
                    D2SRAMBuf[0] = D2SRAMount++;
                    D2SRAMBuf[5] = D2SRAMount++;
                    D2SRAMBuf[9] = D2SRAMount++;
                    printf("K2 键按下, D2SRAMBuf[0] = %d, D2SRAMBuf[5] = %d, D2SRAMBuf[9] = %d\r\n",
                                                                                D2SRAMBuf[0],
                                                                                D2SRAMBuf[5],
                                                                               D2SRAMBuf[9]);
                  break;

                case KEY_DOWN_K3:           /* K3 键按下, 操作 D3 域的 SRAM4 */
                D3SRAMBuf[0] = D3SRAMCount++;
                    D3SRAMBuf[5] = D3SRAMCount++;
                    D3SRAMBuf[9] = D3SRAMCount++;
                    printf("K3 键按下, D3SRAMBuf[0] = %d, D3SRAMBuf[5] = %d, D3SRAMBuf[9] = %d\r\n",
                                                                                D3SRAMBuf[0],
                                                                                D3SRAMBuf[5],
                                                                               D3SRAMBuf[9]);
                break;

                default:
                  /* 其它的键值不处理 */
                  break;
            }
        }
    }
}
```

26.6 实验例程说明（IAR）

配套例子：

V7-005_TCM，SRAM 等五块内存的超方便使用方式

实验目的：

1. 学习 TCM，SRAM 等五块内存的超方便使用方式。

实验内容：

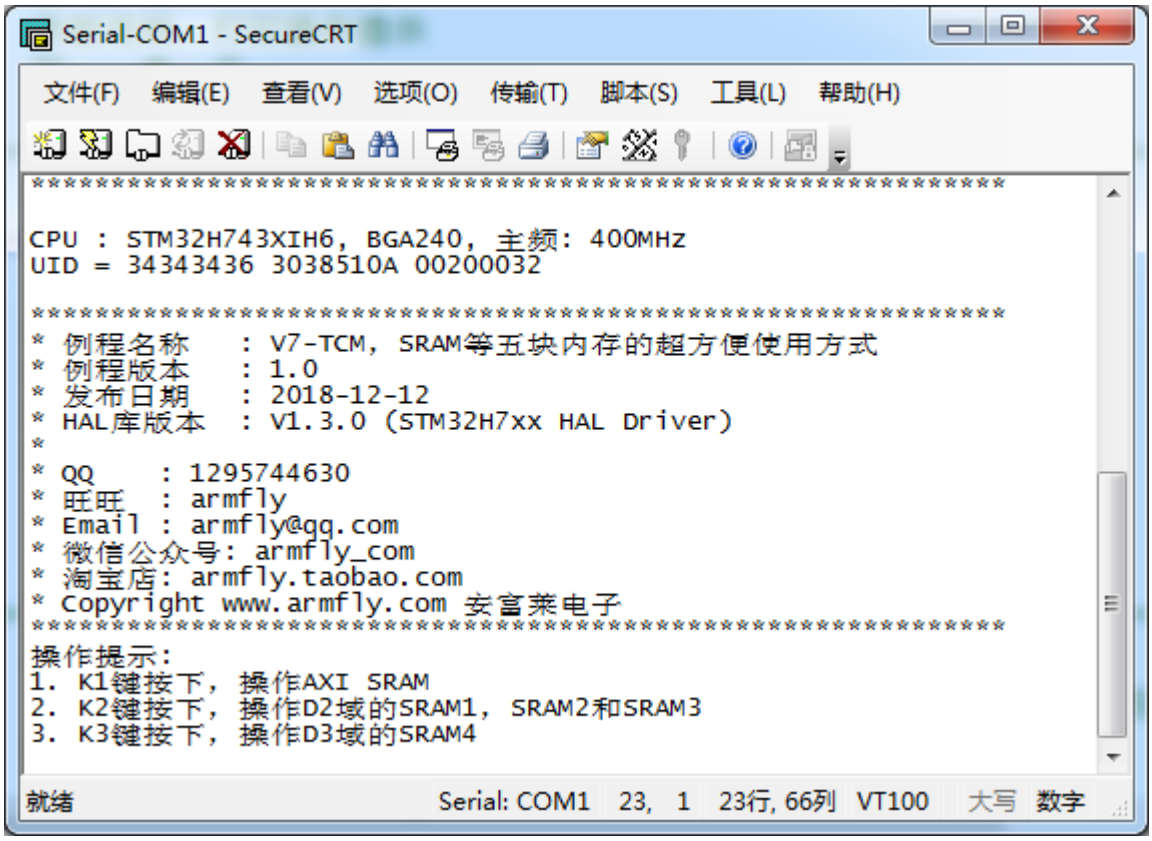
1. 启动自动重装软件定时器 0，每 100ms 翻转一次 LED2。

实验操作：

1. K1 键按下，操作 AXI SRAM。
2. K2 键按下，操作 D2 域的 SRAM1，SRAM2 和 SRAM3。
3. K3 键按下，操作 D3 域的 SRAM4。

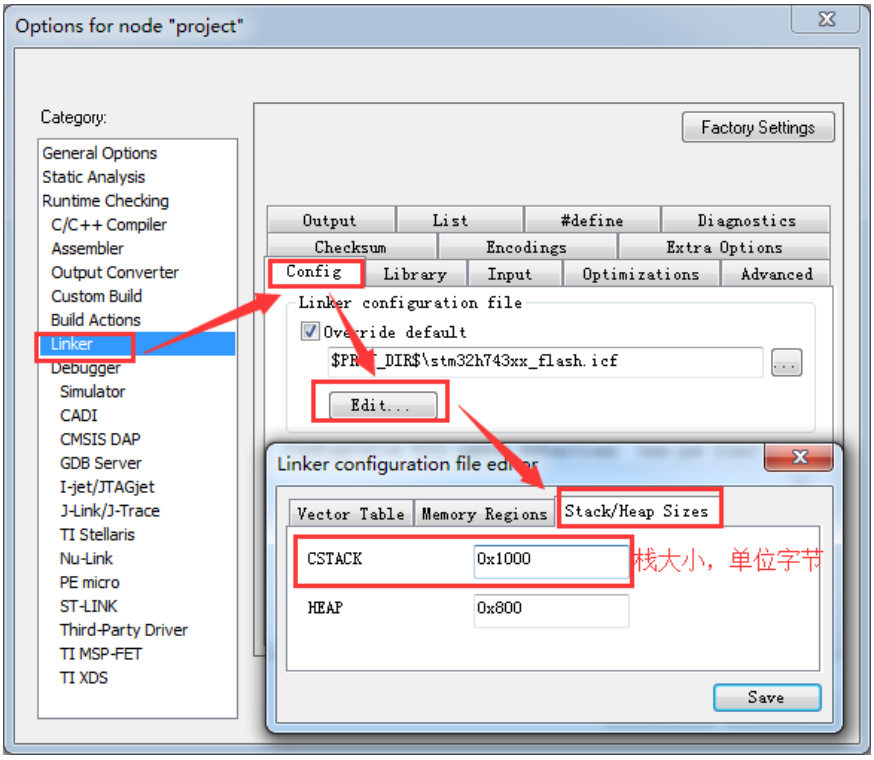
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

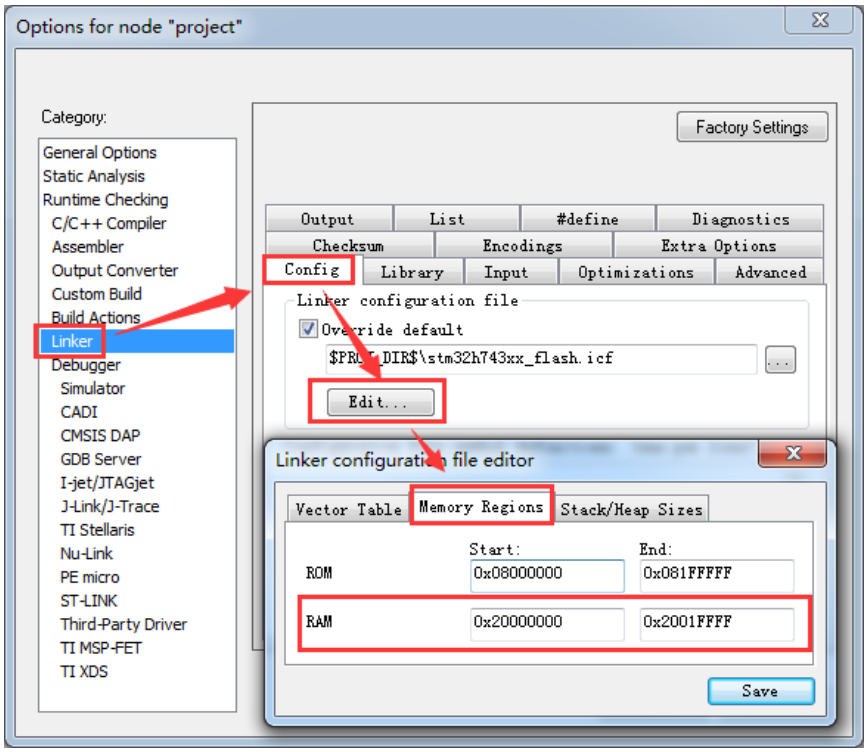


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
```

```
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。

  AXI SRAM 的 MPU 属性:

  Write back, Read allocate，Write allocate。

  FMC 的扩展 IO 的 MPU 属性:

  必须 Device 或者 Strongly Ordered。

  D2 SRAM1，SRAM2 和 SRAM3 的 MPU 属性:

  Write through, read allocate，no write allocate。

D3 SRAM4 的 MPU 属性：

Write through, read allocate，no write allocate。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM1 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x30000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_128KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
```

```
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM2 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x30020000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_128KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER3;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 SRAM3 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x30040000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_32KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER4;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 SRAM4 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER5;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}
```

```
/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主功能的实现主要分为两部分：

- 启动自动重装软件定时器 0，每 100ms 翻转一次 LED2。

- K1 键按下，操作 AXI SRAM。
- K2 键按下，操作 D2 域的 SRAM1，SRAM2 和 SRAM3。
- K3 键按下，操作 D3 域的 SRAM4

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    AXISRAMCount = 0;
    D2SRAMount = 0;
    D3SRAMCount = 0;

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
```

```
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，操作 AXI SRAM */
                    AXISRAMBuf[0] = AXISRAMCount++;
                    AXISRAMBuf[5] = AXISRAMCount++;
                    AXISRAMBuf[9] = AXISRAMCount++;
                    printf("K1 键按下, AXISRAMBuf[0] = %d, AXISRAMBuf[5] = %d, AXISRAMBuf[9] = %d\r\n",
                                                                                AXISRAMBuf[0],
                                                                                AXISRAMBuf[5],
                                                                                AXISRAMBuf[9]);
                    break;

                case KEY_DOWN_K2:           /* K2 键按下，操作 D2 域的 SRAM1，SRAM2 和 SRAM3 */
                    D2SRAMBuf[0] = D2SRAMount++;
                    D2SRAMBuf[5] = D2SRAMount++;
                    D2SRAMBuf[9] = D2SRAMount++;
                    printf("K2 键按下, D2SRAMBuf[0] = %d, D2SRAMBuf[5] = %d, D2SRAMBuf[9] = %d\r\n",
                                                                                D2SRAMBuf[0],
                                                                                D2SRAMBuf[5],
                                                                               D2SRAMBuf[9]);
                  break;

                case KEY_DOWN_K3:           /* K3 键按下，操作 D3 域的 SRAM4 */
                D3SRAMBuf[0] = D3SRAMCount++;
                    D3SRAMBuf[5] = D3SRAMCount++;
                    D3SRAMBuf[9] = D3SRAMCount++;
                    printf("K3 键按下, D3SRAMBuf[0] = %d, D3SRAMBuf[5] = %d, D3SRAMBuf[9] = %d\r\n",
                                                                                D3SRAMBuf[0],
                                                                                D3SRAMBuf[5],
                                                                               D3SRAMBuf[9]);
                break;

                default:
                  /* 其它的键值不处理 */
                  break;
            }
        }
    }
}
```

26.7 总结

本章节为大家介绍的方案比较实用，建议在实际项目中多用用，从而熟练掌握。

第27章 STM32H7 的 TCM，SRAM 等五块内存的动态内存分配实现

本章教程为大家分享一种 DTCM，SRAM1，SRAM2，SRAM3 和 SRAM4 可以独立管理的动态内存管理方案，在实际项目中有一定的实用价值，比如 MP3 编解码，JPEG 编解码，视频播放器，矢量字体等需要动态内存的场合。

27.1 初学者重要提示
27.2 动态内存管理移植
27.3 动态内存的使用方法
27.4 实验例程说明（MDK）
27.5 实验例程说明（IAR）
27.6 总结

27.1 初学者重要提示

- 学习本章节前，务必优先学习第 25 章，了解 TCM，SRAM 等五块内存区的基础知识，比较重要。
- 将 RTX5 系统的动态内存管理整理了出来，可以同时管理多个分区。如果其它 RTOS 中使用，记得做互斥保护或者加个调度锁均可。
- 支持动态内存使用情况统计。

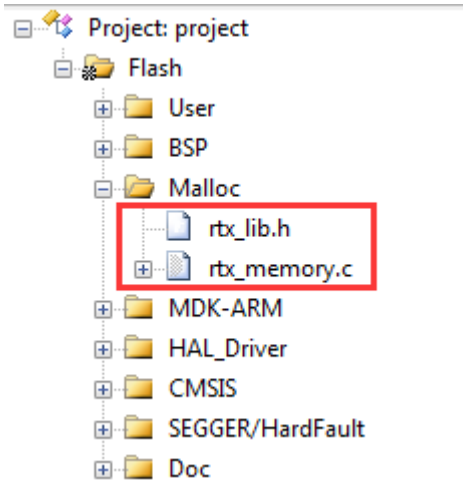
27.2 动态内存管理移植

移植比较简单，仅需添加两个文件到工程即可。

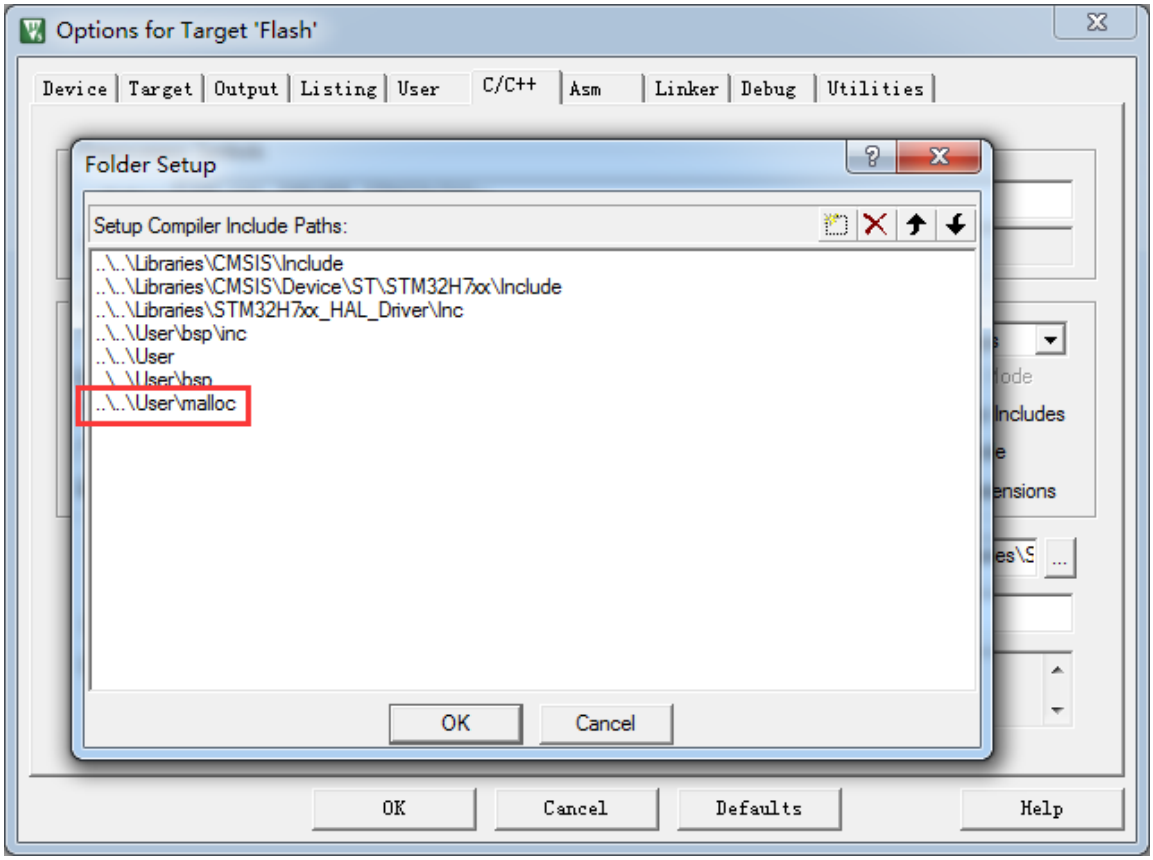
27.2.1 MDK 版的移植

- 第 1 步，添加如下两个文件到 MDK 中

  注，以本章配套例子为例，这两个文件的路径\User\malloc。



◆ 第 2 步，添加路径。



◆ 第 3 步，添加头文件。

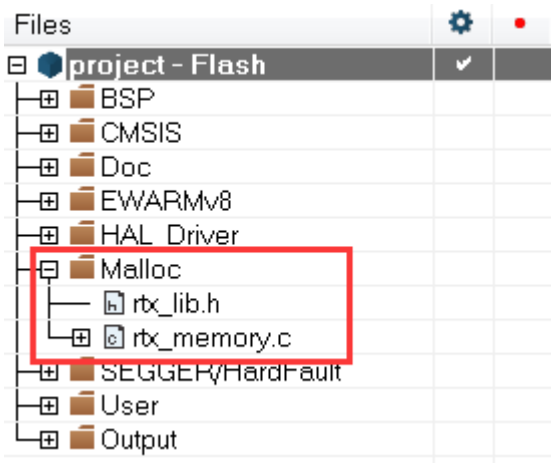
如果哪个源文件要用到动态内存，包含 rtx_lib.h 即可，本章配套例子是直接将其放在了 bsp.h 文件里面，哪个源文件要用到动态内存，直接包含 bsp.h 头文件即可。

通过这简单的三步就完成了 MDK 的移植。

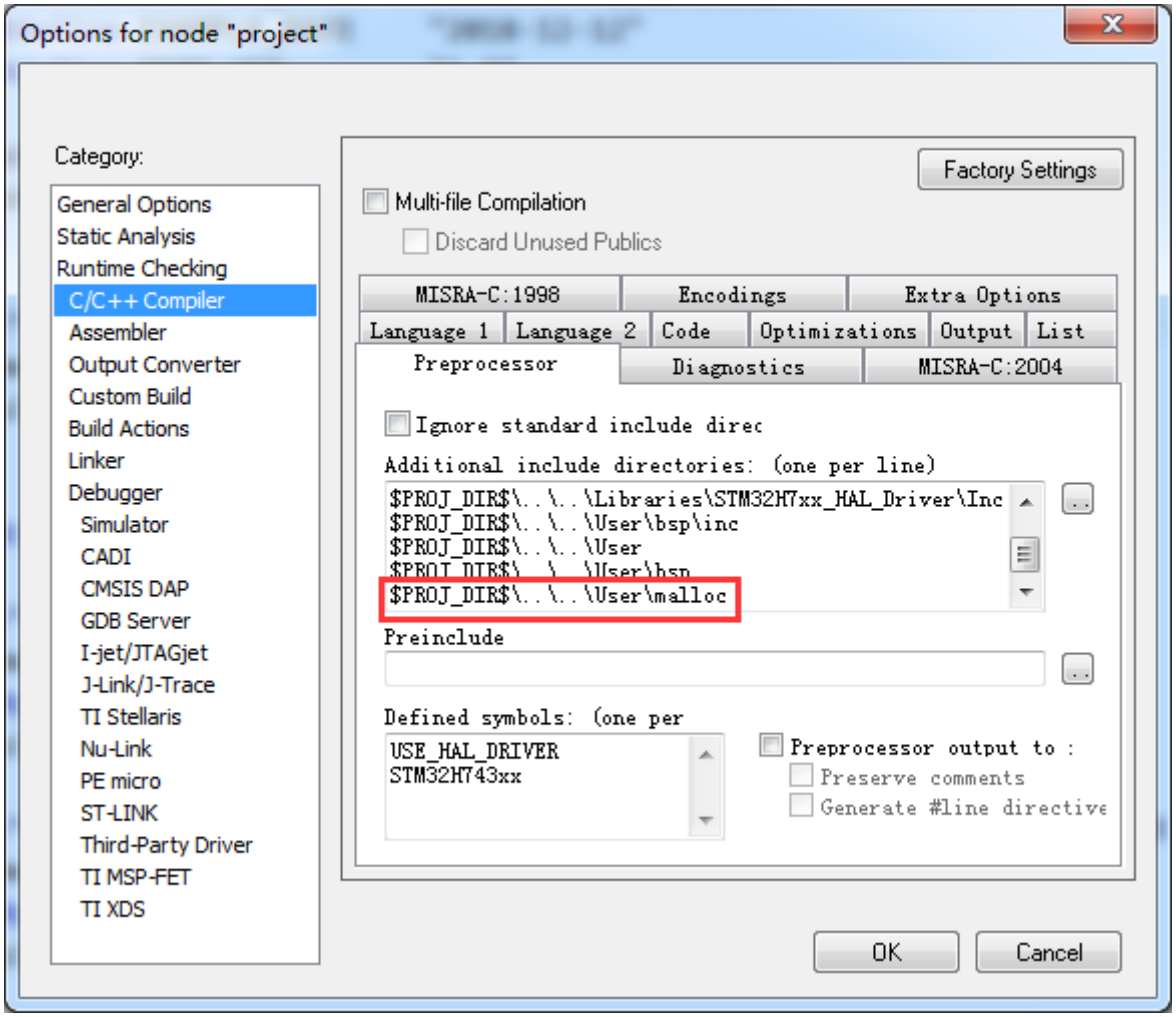
27.2.2 IAR 版的移植

- 第 1 步，添加如下两个文件到 IAR 中

  注，以本章配套例子为例，这两个文件的路径\User\malloc。



- 第 2 步，添加路径。



◆ 第 3 步，添加头文件。

如果哪个源文件要用到动态内存，包含 rtx_lib.h 即可，本章配套例子是直接将其放在了 bsp.h 文件里面，哪个源文件要用到动态内存，直接包含 bsp.h 头文件即可。

通过这简单的三步就完成了 IAR 的移植。

27.3 动态内存的使用方法

下面分别以 MDK 和 IAR 为例进行说明：

27.3.1 MDK 上的动态内存用法

◆ **定义动态内存区**

比如当前的主 RAM 用的 DTCM，我们就可以直接定义一块大的数组作为动态内存空间：

```
/* DTCM, 64KB */
/* 用于获取当前使用的空间大小 */
mem_head_t *DTCMUsed;
/* 定义为 64 位变量，首地址是 8 字节对齐 */
uint64_t AppMallocDTCM[64*1024/8];
```

如果要使用 AXI SRAM 作为动态内存空间，可以使用__attribute__((at( )))指定地址。

```
/* D1 域, AXI SRAM, 512KB */
/* 用于获取当前使用的空间大小 */
mem_head_t *AXISRAMUsed;
/* 定义为 64 位变量，首地址是 8 字节对齐 */
uint64_t AppMallocAXISRAM[512*1024/8]__attribute__((at(0x24000000)));
```

◆ **初始化动态内存区**

调用动态内存管理提供的函数 osRtxMemoryInit 即可做初始化：

```
osRtxMemoryInit(AppMallocDTCM,    sizeof(AppMallocDTCM));
osRtxMemoryInit(AppMallocAXISRAM, sizeof(AppMallocAXISRAM));
```

◆ **申请动态内存**

通过函数 void *osRtxMemoryAlloc (void *mem, uint32_t size, uint32_t type)做动态内存申请。

第 1 个参数填写内存区首地址，比如申请的 AppMallocDTCM，就填 AppMallocDTCM 即可。

第 2 个参数填写申请的字节大小，单位字节。

第 3 个参数固定填 0 即可。

返回值是所申请缓冲区的首地址，如果没有空间可用，将返回 NULL，这点要特别注意！

举个例子：

```
uint32_t *DTCM_Addres0, *AXISRAM_Addres0;

/* 从 DTCM 申请 280 字节空间，使用指针变量 DTCM_Addres0 操作这些空间时不要超过 280 字节大小 */
DTCM_Addres0 = osRtxMemoryAlloc(AppMallocDTCM, 280, 0);
DTCMUsed = MemHeadPtr(AppMallocDTCM);
printf("DTCM 总大小 = %d 字节，申请大小 = 0280 字节，当前共使用大小 = %d 字节\r\n",
                                                DTCMUsed->size, DTCMUsed->used);
```

```
/* 从 AXI SRAM 申请 160 字节空间，使用指针变量 AXISRAM_Addres0 操作这些空间时不要超过 160 字节大小 */
AXISRAM_Addres0 = osRtxMemoryAlloc(AppMallocAXISRAM, 160, 0);
AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
printf("AXI SRAM 总大小 = %d 字节，申请大小 = 0162 字节，当前共使用大小 = %d 字节\r\n",
                                                AXISRAMUsed->size, AXISRAMUsed->used);
```

申请了空间后，就可以直接使用了。另外注意红色字体部分，通过 DTCMUsed->used 和 AXISRAMUsed->used 可以获取当前使用的空间大小。

◆ **释放动态内存**

通过函数 uint32_t osRtxMemoryFree (void *mem, void *block)做动态内存释放。

第 1 个参数填写内存区首地址，比如释放的 AppMallocDTCM，就填 AppMallocDTCM 即可。

第 2 个参数填写申请内存时所获取的内存区首地址，这里用于释放。

返回值，返回 1 表示成功，返回 0 表示失败。

举个例子：

```
/* 释放从 DTCM 申请的 280 字节空间 */
osRtxMemoryFree(AppMallocDTCM, DTCM_Addres0);
DTCMUsed = MemHeadPtr(AppMallocDTCM);
printf("释放 DTCM 动态内存区申请的 0280 字节，当前共使用大小 = %d 字节\r\n", DTCMUsed->used);

/* 释放从 AXI SRAM 申请的 160 字节空间 */
osRtxMemoryFree(AppMallocAXISRAM, AXISRAM_Addres0);
AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
printf("释放 AXI SRAM 动态内存区申请的 0160 字节，当前共使用大小 = %d 字节\r\n", AXISRAMUsed->used);
```

27.3.2 IAR 上的动态内存用法

注：IAR 使用这个动态内存管理，仅在定义时跟 MDK 略有不同，其它地方是一样的。

◆ **定义动态内存区**

比如当前的主 RAM 用的 DTCM，我们就可以直接定义一块大的数组作为动态内存空间：

```
/* DTCM, 64KB */
/* 用于获取当前使用的空间大小 */
mem_head_t *DTCMUsed;
/* 定义为 64 位变量，首地址是 8 字节对齐 */
uint64_t AppMallocDTCM[64*1024/8];
```

如果要使用 AXI SRAM 作为动态内存空间，可以使用__attribute__((at( )))指定地址。

```
/* D1 域, AXI SRAM, 512KB */
/* 用于获取当前使用的空间大小 */
mem_head_t *AXISRAMUsed;
/* 指定下面数组的地址为 0x24000000 */
#pragma location = 0x24000000
uint64_t AppMallocAXISRAM[512*1024/8];
```

◆ **初始化动态内存区**

调用动态内存管理提供的函数 osRtxMemoryInit 即可做初始化：

```
osRtxMemoryInit(AppMallocDTCM,    sizeof(AppMallocDTCM));
osRtxMemoryInit(AppMallocAXISRAM, sizeof(AppMallocAXISRAM));
```

◆ **申请动态内存**

通过函数 void *osRtxMemoryAlloc (void *mem, uint32_t size, uint32_t type)做动态内存申请。

第 1 个参数填写内存区首地址，比如申请的 AppMallocDTCM，就填 AppMallocDTCM 即可。

第 2 个参数填写申请的字节大小，单位字节。

第 3 个参数固定填 0 即可。

返回值是所申请缓冲区的首地址，如果没有空间可用，将返回 NULL，这点要特别注意！

举个例子：

```
uint32_t *DTCM_Addres0, *AXISRAM_Addres0;

/* 从 DTCM 申请 280 字节空间，使用指针变量 DTCM_Addres0 操作这些空间时不要超过 280 字节大小 */
DTCM_Addres0 = osRtxMemoryAlloc(AppMallocDTCM, 280, 0);
DTCMUsed = MemHeadPtr(AppMallocDTCM);
printf("DTCM 总大小 = %d 字节，申请大小 = 0280 字节，当前共使用大小 = %d 字节\r\n",
                                                DTCMUsed->size, DTCMUsed->used);

/* 从 AXI SRAM 申请 160 字节空间，使用指针变量 AXISRAM_Addres0 操作这些空间时不要超过 160 字节大小 */
AXISRAM_Addres0 = osRtxMemoryAlloc(AppMallocAXISRAM, 160, 0);
AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
printf("AXI SRAM 总大小 = %d 字节，申请大小 = 0162 字节，当前共使用大小 = %d 字节\r\n",
                                                AXISRAMUsed->size, AXISRAMUsed->used);
```

申请了空间后，就可以直接使用了。另外注意红色字体部分，通过 DTCMUsed->used 和 AXISRAMUsed->used 可以获取当前使用的空间大小。

◆ **释放动态内存**

通过函数 uint32_t osRtxMemoryFree (void *mem, void *block)做动态内存释放。

第 1 个参数填写内存区首地址，比如释放的 AppMallocDTCM，就填 AppMallocDTCM 即可。

第 2 个参数填写申请内存时所获取的内存区首地址，这里用于释放。

返回值，返回 1 表示成功，返回 0 表示失败。

举个例子：

```
/* 释放从 DTCM 申请的 280 字节空间 */
osRtxMemoryFree(AppMallocDTCM, DTCM_Addres0);
DTCMUsed = MemHeadPtr(AppMallocDTCM);
printf("释放 DTCM 动态内存区申请的 0280 字节，当前共使用大小 = %d 字节\r\n", DTCMUsed->used);

/* 释放从 AXI SRAM 申请的 160 字节空间 */
osRtxMemoryFree(AppMallocAXISRAM, AXISRAM_Addres0);
AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
printf("释放 AXI SRAM 动态内存区申请的 0160 字节，当前共使用大小 = %d 字节\r\n", AXISRAMUsed->used)
```

27.4 实验例程说明（MDK）

配套例子：

V7-006_TCM，SRAM 等五块内存的动态内存分配实现

实验目的：

1. 学习 TCM，SRAM 等五块内存的动态内存分配实现。

实验内容：

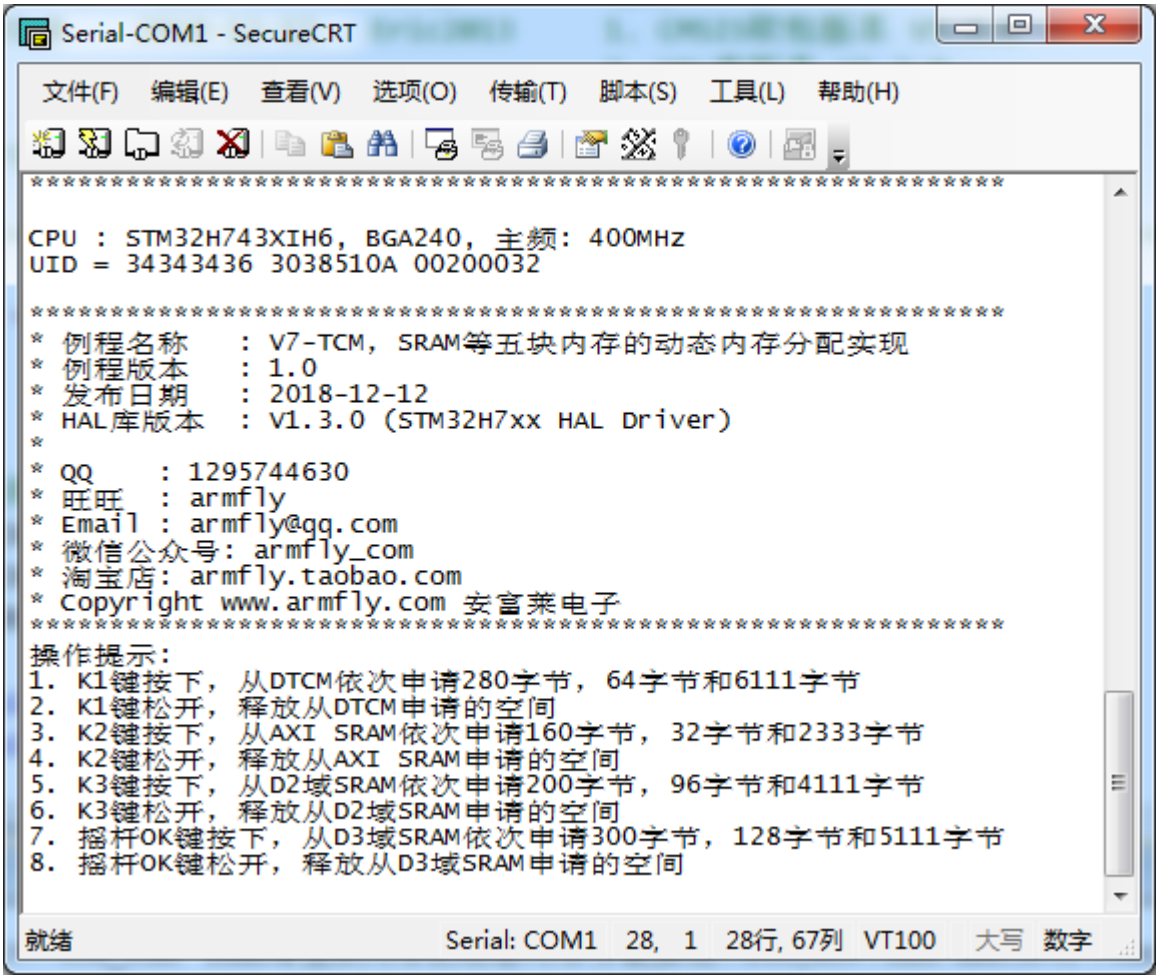
1. 启动自动重装软件定时器 0，每 100ms 翻转一次 LED2。

实验操作：

1. K1 键按下，从 DTCM 依次申请 280 字节，64 字节和 6111 字节。
2. K1 键松开，释放从 DTCM 申请的空间。
3. K2 键按下，从 AXI SRAM 依次申请 160 字节，32 字节和 2333 字节。
4. K2 键松开，释放从 AXI SRAM 申请的空间。
5. K3 键按下，从 D2 域 SRAM 依次申请 200 字节，96 字节和 4111 字节。
6. K3 键松开，释放从 D2 域 SRAM 申请的空间。
7. 摇杆 OK 键按下，从 D3 域 SRAM 依次申请 300 字节，128 字节和 5111 字节。
8. 摇杆 OK 键松开，释放从 D3 域 SRAM 申请的空间。

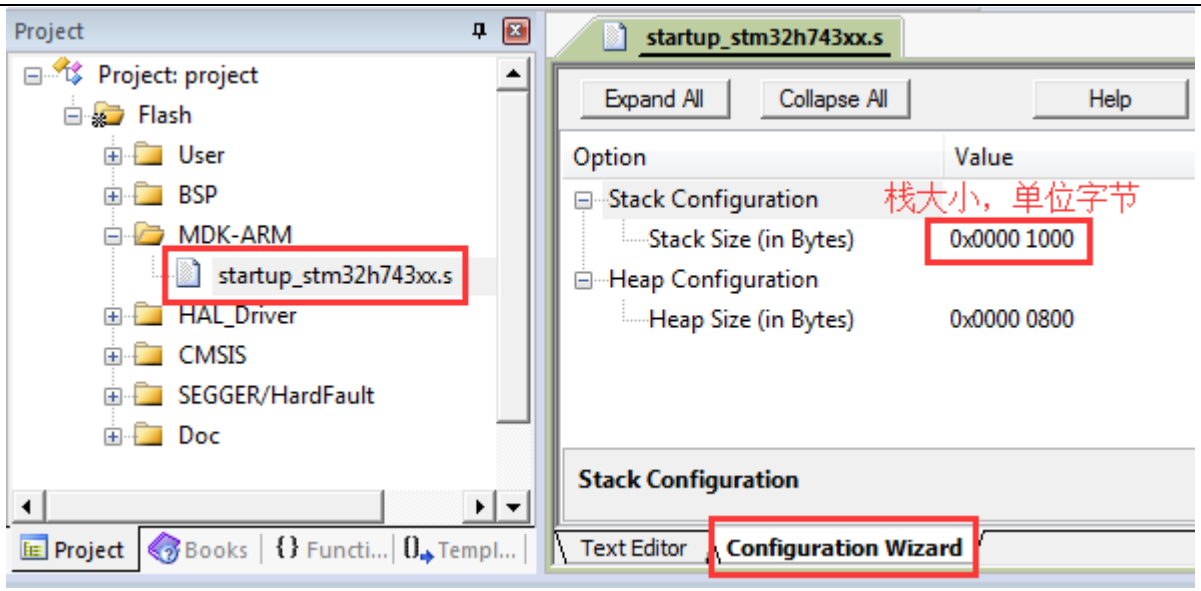
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

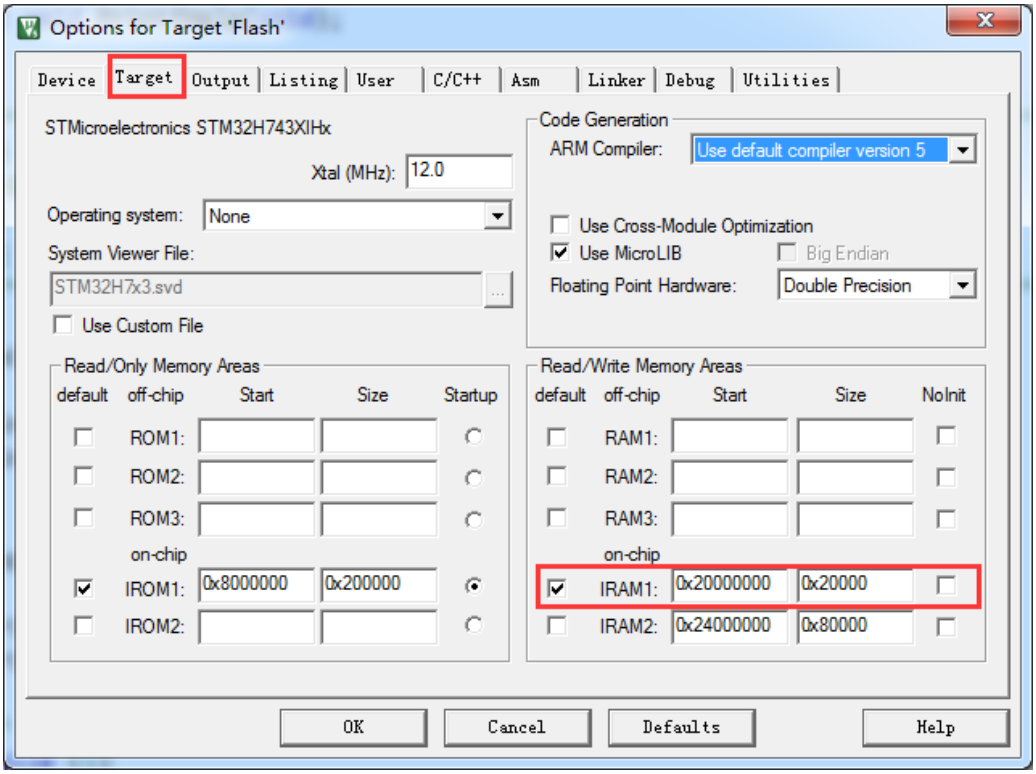


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。

  AXI SRAM 的 MPU 属性:

  Write back, Read allocate，Write allocate。

  FMC 的扩展 IO 的 MPU 属性:

  必须 Device 或者 Strongly Ordered。

  D2 SRAM1，SRAM2 和 SRAM3 的 MPU 属性:

  Write through, read allocate，no write allocate。

  D3 SRAM4 的 MPU 属性:

  Write through, read allocate，no write allocate。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
```

```
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM1 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x30000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_128KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

```
    /* 配置 SRAM2 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x30020000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_128KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER3;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 SRAM3 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x30040000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_32KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER4;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 SRAM4 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER5;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- 启动自动重装软件定时器 0，每 100ms 翻转一次 LED2。
- K1 键按下，从 DTCM 依次申请 280 字节，64 字节和 6111 字节。
- K1 键松开，释放从 DTCM 申请的空间。
- K2 键按下，从 AXI SRAM 依次申请 160 字节，32 字节和 2333 字节。
- K2 键松开，释放从 AXI SRAM 申请的空间。
- K3 键按下，从 D2 域 SRAM 依次申请 200 字节，96 字节和 4111 字节。
- K3 键松开，释放从 D2 域 SRAM 申请的空间。
- 摇杆 OK 键按下，从 D3 域 SRAM 依次申请 300 字节，128 字节和 5111 字节。
- 摇杆 OK 键松开，释放从 D3 域 SRAM 申请的空间。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */
    uint32_t *DTCM_Addres0, *AXISRAM_Addres0, *SRAM1_Addres0, *SRAM4_Addres0;
    uint16_t *DTCM_Addres1, *AXISRAM_Addres1, *SRAM1_Addres1, *SRAM4_Addres1;
    uint8_t  *DTCM_Addres2, *AXISRAM_Addres2, *SRAM1_Addres2, *SRAM4_Addres2;


    bsp_Init();        /* 硬件初始化 */

    /* 初始化动态内存空间 */
    osRtxMemoryInit(AppMallocDTCM,    sizeof(AppMallocDTCM));
    osRtxMemoryInit(AppMallocAXISRAM, sizeof(AppMallocAXISRAM));
    osRtxMemoryInit(AppMallocSRAM1,   sizeof(AppMallocSRAM1));
    osRtxMemoryInit(AppMallocSRAM4,   sizeof(AppMallocSRAM4));

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */
```

```
    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
              /* 从 DTCM 依次申请 280 字节，64 字节和 6111 字节 */
                case KEY_DOWN_K1:
                 /* 从DTCM申请280字节空间,使用指针变量DTCM_Addres0操作这些空间时不要超过280字节大小 */
                    printf("=========================================================\r\n");
                    DTCM_Addres0 = osRtxMemoryAlloc(AppMallocDTCM, 280, 0);
                    DTCMUsed = MemHeadPtr(AppMallocDTCM);
                    printf("DTCM 总大小 = %d 字节，申请大小 = 0280 字节，当前共使用大小 = %d 字节\r\n",
                                                            DTCMUsed->size, DTCMUsed->used);

            /* 从 DTCM 申请 64 字节空间，使用指针变量 DTCM_Addres1 操作这些空间时不要超过 64 字节大小 */
                    DTCM_Addres1 = osRtxMemoryAlloc(AppMallocDTCM, 64, 0);
                    DTCMUsed = MemHeadPtr(AppMallocDTCM);
                    printf("DTCM 总大小 = %d 字节，申请大小 = 0064 字节，当前共使用大小 = %d 字节\r\n",
                                                              DTCMUsed->size, DTCMUsed->used);

               /* 从 DTCM 申请 6111 字节空间，使用指针变量 DTCM_Addres2 操作这些空间时不要超过 6111 字节大小 */
                    DTCM_Addres2 = osRtxMemoryAlloc(AppMallocDTCM, 6111, 0);
                    DTCMUsed = MemHeadPtr(AppMallocDTCM);
                    printf("DTCM 总大小 = %d 字节，申请大小 = 6111 字节，当前共使用大小 = %d 字节\r\n",
                                                            DTCMUsed->size, DTCMUsed->used);
                    break;

                /* 释放从 DTCM 申请的空间 */
                case KEY_UP_K1:
                    /* 释放从 DTCM 申请的 280 字节空间 */
                    osRtxMemoryFree(AppMallocDTCM, DTCM_Addres0);
                    DTCMUsed = MemHeadPtr(AppMallocDTCM);
                    printf("释放 DTCM 动态内存区申请的 0280 字节，当前共使用大小 = %d 字节\r\n",
                                                                    DTCMUsed->used);

                    /* 释放从 DTCM 申请的 64 字节空间 */
                    osRtxMemoryFree(AppMallocDTCM, DTCM_Addres1);
                    DTCMUsed = MemHeadPtr(AppMallocDTCM);
                    printf("释放 DTCM 动态内存区申请的 0064 字节，当前共使用大小 = %d 字节\r\n",
                                                                    DTCMUsed->used);

                    /* 释放从 DTCM 申请的 6111 字节空间 */
                    osRtxMemoryFree(AppMallocDTCM, DTCM_Addres2);
                    DTCMUsed = MemHeadPtr(AppMallocDTCM);
                    printf("释放 DTCM 动态内存区申请的 6111 字节，当前共使用大小 = %d 字节\r\n",
```

```
                                                                    DTCMUsed->used);
                    break;

                /* 从 AXI SRAM 依次申请 160 字节，32 字节和 2333 字节 */
                case KEY_DOWN_K2:
            /* 从 AXI SRAM 申请 160 字节空间,使用指针变量 AXISRAM_Addres0 操作这些空间时不要超过 160 字节大小 */
                    printf("=========================================================\r\n");
                    AXISRAM_Addres0 = osRtxMemoryAlloc(AppMallocAXISRAM, 160, 0);
                    AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
                    printf("AXI SRAM 总大小 = %d 字节，申请大小 = 0162 字节，当前共使用大小 = %d 字节\r\n",
                                                        AXISRAMUsed->size, AXISRAMUsed->used);

             /* 从 AXI SRAM 申请 32 字节空间,使用指针变量 AXISRAM_Addres1 操作这些空间时不要超过 32 字节大小 */
                    AXISRAM_Addres1 = osRtxMemoryAlloc(AppMallocAXISRAM, 32, 0);
                    AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
                    printf("AXI SRAM 总大小 = %d 字节，申请大小 = 0032 字节，当前共使用大小 = %d 字节\r\n",
                                                      AXISRAMUsed->size, AXISRAMUsed->used);

/* 从 AXI SRAM 申请 2333 字节空间，使用指针变量 AXISRAM_Addres2 操作这些空间时不要超过 2333 字节大小 */
                    AXISRAM_Addres2 = osRtxMemoryAlloc(AppMallocAXISRAM, 2333, 0);
                    AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
                    printf("AXI SRAM 总大小 = %d 字节，申请大小 = 2333 字节，当前共使用大小 = %d 字节\r\n",
                                                        AXISRAMUsed->size, AXISRAMUsed->used);
                    break;

                /* 释放从 AXI SRAM 申请的空间 */
                case KEY_UP_K2:
                    /* 释放从 AXI SRAM 申请的 160 字节空间 */
                    osRtxMemoryFree(AppMallocAXISRAM, AXISRAM_Addres0);
                    AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
                    printf("释放 AXI SRAM 动态内存区申请的 0160 字节，当前共使用大小 = %d 字节\r\n",
                                                                    AXISRAMUsed->used);

                    /* 释放从 AXI SRAM 申请的 32 字节空间 */
                    osRtxMemoryFree(AppMallocAXISRAM, AXISRAM_Addres1);
                    AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
                    printf("释放 AXI SRAM 动态内存区申请的 0032 字节，当前共使用大小 = %d 字节\r\n",
                                                                    AXISRAMUsed->used);

                    /* 释放从 AXI SRAM 申请的 2333 字节空间 */
                    osRtxMemoryFree(AppMallocAXISRAM, AXISRAM_Addres2);
                    AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
                    printf("释放 AXI SRAM 动态内存区申请的 2333 字节，当前共使用大小 = %d 字节\r\n",
                                                                    AXISRAMUsed->used);
                    break;

                /* 从 D2 域 SRAM 依次申请 200 字节，96 字节和 4111 字节 */
                case KEY_DOWN_K3:
        /* 从 D2 域的 SRAM 申请 200 字节空间，使用指针变量 SRAM1_Addres0 操作这些空间时不要超过 200 字节大小 */
                    printf("=========================================================\r\n");
                    SRAM1_Addres0 = osRtxMemoryAlloc(AppMallocSRAM1, 200, 0);
                    SRAM1Used = MemHeadPtr(AppMallocSRAM1);
                    printf("D2 域 SRAM 总大小 = %d 字节，申请大小 = 0200 字节，当前共使用大小 = %d 字节\r\n",
                                                        SRAM1Used->size, SRAM1Used->used);

      /* 从 D2 域的 SRAM 申请 96 字节空间，使用指针变量 SRAM1_Addres1 操作这些空间时不要超过 96 字节大小 */
                    SRAM1_Addres1 = osRtxMemoryAlloc(AppMallocSRAM1, 96, 0);
                    SRAM1Used = MemHeadPtr(AppMallocSRAM1);
```

```
                printf("D2 域 SRAM 总大小 =%d 字节，申请大小 =0096 字节，当前共使用大小 =%d 字节\r\n",
                                                        SRAM1Used->size, SRAM1Used->used);

    /* 从 D2 域的 SRAM 申请 4111 字节空间，使用指针变量 SRAM1_Addres2 操作这些空间时不要超过 4111 字节大小 */
                SRAM1_Addres2 = osRtxMemoryAlloc(AppMallocSRAM1, 4111, 0);
                SRAM1Used = MemHeadPtr(AppMallocSRAM1);
                printf("D2 域 SRAM 总大小 =%d 字节，申请大小 =4111 字节，当前共使用大小 =%d 字节\r\n",
                                                        SRAM1Used->size, SRAM1Used->used);
                break;

            /* 释放从 D2 域 SRAM 申请的空间 */
            case KEY_UP_K3:
                /* 释放从 D2 域的 SRAM 申请的 200 字节空间 */
                osRtxMemoryFree(AppMallocSRAM1, SRAM1_Addres0);
                SRAM1Used = MemHeadPtr(AppMallocSRAM1);
                printf("释放 D2 域 SRAM 动态内存区申请的 0200 字节，当前共使用大小 = %d 字节\r\n",
                                                                SRAM1Used->used);

                /* 释放从 D2 域的 SRAM 申请的 96 字节空间 */
                osRtxMemoryFree(AppMallocSRAM1, SRAM1_Addres1);
                SRAM1Used = MemHeadPtr(AppMallocSRAM1);
                printf("释放 D2 域 SRAM 动态内存区申请的 0096 字节，当前共使用大小 = %d 字节\r\n",
                                                                SRAM1Used->used);

                /* 释放从 D2 域的 SRAM 申请的 4111 字节空间 */
                osRtxMemoryFree(AppMallocSRAM1, SRAM1_Addres2);
                SRAM1Used = MemHeadPtr(AppMallocSRAM1);
                printf("释放 D2 域 SRAM 动态内存区申请的 4111 字节，当前共使用大小 = %d 字节\r\n",
                                                                SRAM1Used->used);
                break;

            /* 从 D3 域 SRAM 依次申请 300 字节，128 字节和 5111 字节 */
            case JOY_DOWN_OK:
        /* 从D3域的SRAM申请300字节空间,使用指针变量SRAM4_Addres0操作这些空间时不要超过300字节大小 */
                printf("=========================================================\r\n");
                SRAM4_Addres0 = osRtxMemoryAlloc(AppMallocSRAM4, 300, 0);
                SRAM4Used = MemHeadPtr(AppMallocSRAM4);
                printf("D3 域 SRAM 总大小 =%d 字节，申请大小 =0300 字节，当前共使用大小 =%d 字节\r\n",
                                                        SRAM4Used->size, SRAM4Used->used);

    /* 从 D3 域的 SRAM 申请 96 字节空间，使用指针变量 SRAM4_Addres1 操作这些空间时不要超过 96 字节大小 */
                SRAM4_Addres1 = osRtxMemoryAlloc(AppMallocSRAM4, 128, 0);
                SRAM4Used = MemHeadPtr(AppMallocSRAM4);
                printf("D3 域 SRAM 总大小 =%d 字节，申请大小 =0128 字节，当前共使用大小 =%d 字节\r\n",
                                                        SRAM4Used->size, SRAM4Used->used);

/* 从 D3 域的 SRAM 申请 5111 字节空间，使用指针变量 SRAM4_Addres2 操作这些空间时不要超过 5111 字节大小 */
                SRAM4_Addres2 = osRtxMemoryAlloc(AppMallocSRAM4, 5111, 0);
                SRAM4Used = MemHeadPtr(AppMallocSRAM4);
                printf("D3 域 SRAM 总大小 =%d 字节，申请大小 =5111 字节，当前共使用大小 =%d 字节\r\n",
                                                        SRAM4Used->size, SRAM4Used->used);
                break;

            /* 释放从 D3 域 SRAM 申请的空间 */
            case JOY_UP_OK:
                /* 释放从 D3 域的 SRAM 申请的 300 字节空间 */
                osRtxMemoryFree(AppMallocSRAM4, SRAM4_Addres0);
                SRAM4Used = MemHeadPtr(AppMallocSRAM4);
```

```
                    printf("释放 D3 域 SRAM 动态内存区申请的 0300 字节，当前共使用大小 = %d 字节\r\n",
                                                                    SRAM4Used->used);

                    /* 释放从 D3 域的 SRAM 申请的 128 字节空间 */
                    osRtxMemoryFree(AppMallocSRAM4, SRAM4_Addres1);
                    SRAM4Used = MemHeadPtr(AppMallocSRAM4);
                    printf("释放 D3 域 SRAM 动态内存区申请的 0128 字节，当前共使用大小 = %d 字节\r\n",
                                                                SRAM4Used->used);

                    /* 释放从 D3 域的 SRAM 申请的 5111 字节空间 */
                    osRtxMemoryFree(AppMallocSRAM4, SRAM4_Addres2);
                    SRAM4Used = MemHeadPtr(AppMallocSRAM4);
                    printf("释放 D3 域 SRAM 动态内存区申请的 5111 字节，当前共使用大小 = %d 字节\r\n",
                                                                    SRAM4Used->used);
                    break;

                default:
                  /* 其它的键值不处理 */
                  break;
            }
        }
    }
}
```

27.5 实验例程说明（IAR）

配套例子：

V7-006_TCM，SRAM 等五块内存的动态内存分配实现

实验目的：

1. 学习 TCM，SRAM 等五块内存的动态内存分配实现。

实验内容：

1. 启动自动重装软件定时器 0，每 100ms 翻转一次 LED2。

实验操作：

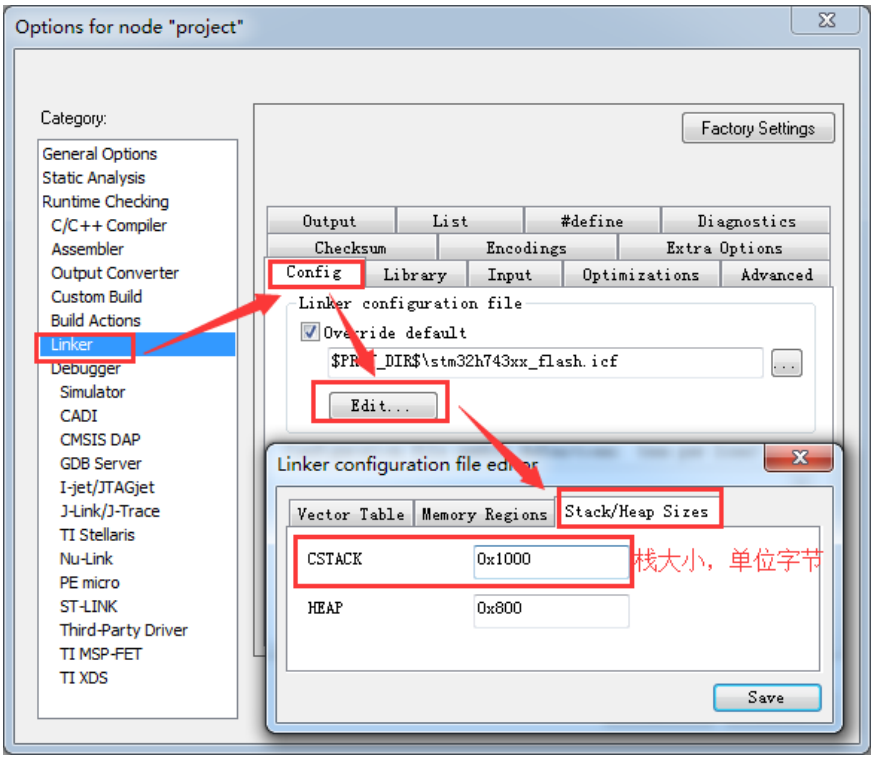
1. K1 键按下，从 DTCM 依次申请 280 字节，64 字节和 6111 字节。
2. K1 键松开，释放从 DTCM 申请的空间。
3. K2 键按下，从 AXI SRAM 依次申请 160 字节，32 字节和 2333 字节。
4. K2 键松开，释放从 AXI SRAM 申请的空间。
5. K3 键按下，从 D2 域 SRAM 依次申请 200 字节，96 字节和 4111 字节。
6. K3 键松开，释放从 D2 域 SRAM 申请的空间。
7. 摇杆 OK 键按下，从 D3 域 SRAM 依次申请 300 字节，128 字节和 5111 字节。
8. 摇杆 OK 键松开，释放从 D3 域 SRAM 申请的空间。

上电后串口打印的信息：

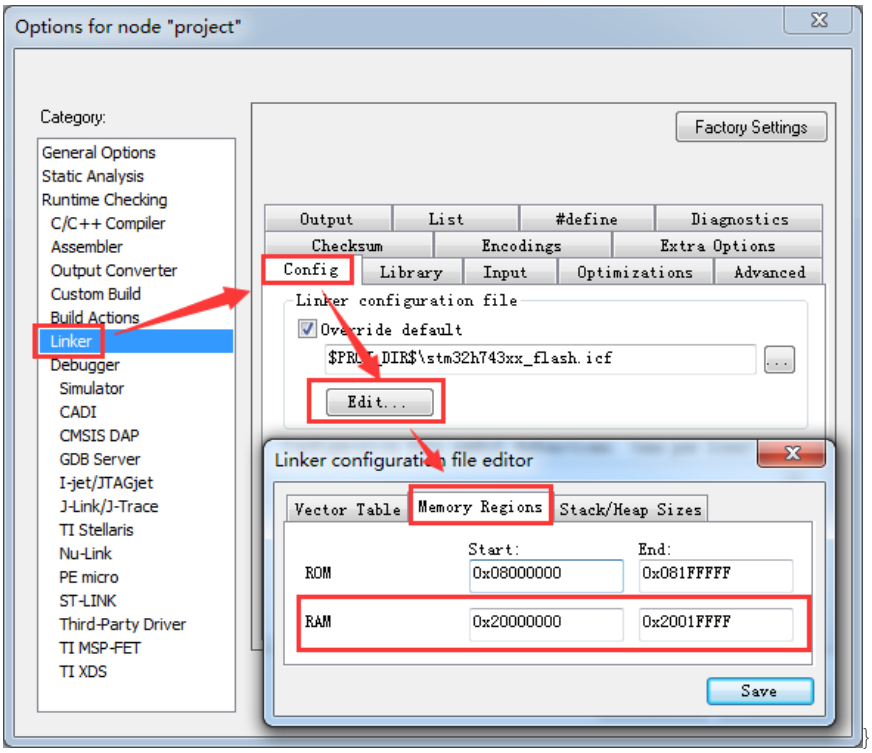
波特率 115200，数据位 8，奇偶校验位无，停止位 1

程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。

  AXI SRAM 的 MPU 属性：

  Write back, Read allocate，Write allocate。

  FMC 的扩展 IO 的 MPU 属性：

  必须 Device 或者 Strongly Ordered。

  D2 SRAM1，SRAM2 和 SRAM3 的 MPU 属性：

  Write through, read allocate，no write allocate。

  D3 SRAM4 的 MPU 属性：

  Write through, read allocate，no write allocate。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
```

```
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM1 的属性为 Write through, read allocate, no write allocate */
   MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
   MPU_InitStruct.BaseAddress      = 0x30000000;
   MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_128KB;
   MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
   MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
   MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
   MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
   MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
   MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
   MPU_InitStruct.SubRegionDisable = 0x00;
   MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

   HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM2 的属性为 Write through, read allocate, no write allocate */
   MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
   MPU_InitStruct.BaseAddress      = 0x30020000;
   MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_128KB;
   MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
   MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
   MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
   MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
   MPU_InitStruct.Number           = MPU_REGION_NUMBER3;
   MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
   MPU_InitStruct.SubRegionDisable = 0x00;
   MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

   HAL_MPU_ConfigRegion(&MPU_InitStruct);


   /* 配置 SRAM3 的属性为 Write through, read allocate, no write allocate */
   MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
   MPU_InitStruct.BaseAddress      = 0x30040000;
   MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_32KB;
```

```
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER4;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 SRAM4 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER5;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **主功能：**

  主程序实现如下操作：

  - 启动自动重装软件定时器 0，每 100ms 翻转一次 LED2。
  - K1 键按下，从 DTCM 依次申请 280 字节，64 字节和 6111 字节。
  - K1 键松开，释放从 DTCM 申请的空间。
  - K2 键按下，从 AXI SRAM 依次申请 160 字节，32 字节和 2333 字节。

- K2 键松开，释放从 AXI SRAM 申请的空间。
- K3 键按下，从 D2 域 SRAM 依次申请 200 字节，96 字节和 4111 字节。
- K3 键松开，释放从 D2 域 SRAM 申请的空间。
- 摇杆 OK 键按下，从 D3 域 SRAM 依次申请 300 字节，128 字节和 5111 字节。
- 摇杆 OK 键松开，释放从 D3 域 SRAM 申请的空间。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */
    uint32_t *DTCM_Addres0, *AXISRAM_Addres0, *SRAM1_Addres0, *SRAM4_Addres0;
    uint16_t *DTCM_Addres1, *AXISRAM_Addres1, *SRAM1_Addres1, *SRAM4_Addres1;
    uint8_t  *DTCM_Addres2, *AXISRAM_Addres2, *SRAM1_Addres2, *SRAM4_Addres2;


    bsp_Init();        /* 硬件初始化 */

    /* 初始化动态内存空间 */
    osRtxMemoryInit(AppMallocDTCM,    sizeof(AppMallocDTCM));
    osRtxMemoryInit(AppMallocAXISRAM, sizeof(AppMallocAXISRAM));
    osRtxMemoryInit(AppMallocSRAM1,   sizeof(AppMallocSRAM1));
    osRtxMemoryInit(AppMallocSRAM4,   sizeof(AppMallocSRAM4));

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */


    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
              /* 从 DTCM 依次申请 280 字节，64 字节和 6111 字节 */
```

```
            case KEY_DOWN_K1:
              /* 从DTCM申请280字节空间,使用指针变量DTCM_Addres0操作这些空间时不要超过280字节大小 */
                printf("=========================================================\r\n");
                DTCM_Addres0 = osRtxMemoryAlloc(AppMallocDTCM, 280, 0);
                DTCMUsed = MemHeadPtr(AppMallocDTCM);
                printf("DTCM 总大小 = %d 字节, 申请大小 = 0280 字节, 当前共使用大小 = %d 字节\r\n",
                                                        DTCMUsed->size, DTCMUsed->used);

            /* 从 DTCM 申请 64 字节空间, 使用指针变量 DTCM_Addres1 操作这些空间时不要超过 64 字节大小 */
                DTCM_Addres1 = osRtxMemoryAlloc(AppMallocDTCM, 64, 0);
                DTCMUsed = MemHeadPtr(AppMallocDTCM);
                printf("DTCM 总大小 = %d 字节, 申请大小 = 0064 字节, 当前共使用大小 = %d 字节\r\n",
                                                            DTCMUsed->size, DTCMUsed->used);

            /* 从 DTCM 申请 6111 字节空间, 使用指针变量 DTCM_Addres2 操作这些空间时不要超过 6111 字节大小 */
                DTCM_Addres2 = osRtxMemoryAlloc(AppMallocDTCM, 6111, 0);
                DTCMUsed = MemHeadPtr(AppMallocDTCM);
                printf("DTCM 总大小 = %d 字节, 申请大小 = 6111 字节, 当前共使用大小 = %d 字节\r\n",
                                                        DTCMUsed->size, DTCMUsed->used);
                break;

            /* 释放从 DTCM 申请的空间 */
            case KEY_UP_K1:
                /* 释放从 DTCM 申请的 280 字节空间 */
                osRtxMemoryFree(AppMallocDTCM, DTCM_Addres0);
                DTCMUsed = MemHeadPtr(AppMallocDTCM);
                printf("释放 DTCM 动态内存区申请的 0280 字节, 当前共使用大小 = %d 字节\r\n",
                                                                DTCMUsed->used);

                /* 释放从 DTCM 申请的 64 字节空间 */
                osRtxMemoryFree(AppMallocDTCM, DTCM_Addres1);
                DTCMUsed = MemHeadPtr(AppMallocDTCM);
                printf("释放 DTCM 动态内存区申请的 0064 字节, 当前共使用大小 = %d 字节\r\n",
                                                                DTCMUsed->used);

                /* 释放从 DTCM 申请的 6111 字节空间 */
                osRtxMemoryFree(AppMallocDTCM, DTCM_Addres2);
                DTCMUsed = MemHeadPtr(AppMallocDTCM);
                printf("释放 DTCM 动态内存区申请的 6111 字节, 当前共使用大小 = %d 字节\r\n",
                                                                DTCMUsed->used);
                break;

            /* 从 AXI SRAM 依次申请 160 字节, 32 字节和 2333 字节 */
            case KEY_DOWN_K2:
          /* 从AXI SRAM 申请160字节空间,使用指针变量AXISRAM_Addres0操作这些空间时不要超过160字节大小 */
                printf("=========================================================\r\n");
                AXISRAM_Addres0 = osRtxMemoryAlloc(AppMallocAXISRAM, 160, 0);
                AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
                printf("AXI SRAM 总大小 = %d 字节, 申请大小 = 0162 字节, 当前共使用大小 = %d 字节\r\n",
                                                        AXISRAMUsed->size, AXISRAMUsed->used);

            /* 从AXI SRAM 申请32字节空间,使用指针变量AXISRAM_Addres1操作这些空间时不要超过32字节大小 */
                AXISRAM_Addres1 = osRtxMemoryAlloc(AppMallocAXISRAM, 32, 0);
                AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
                printf("AXI SRAM 总大小 = %d 字节, 申请大小 = 0032 字节, 当前共使用大小 = %d 字节\r\n",
                                                        AXISRAMUsed->size, AXISRAMUsed->used);

/* 从 AXI SRAM 申请 2333 字节空间, 使用指针变量 AXISRAM_Addres2 操作这些空间时不要超过 2333 字节大小 */
```

```
                AXISRAM_Addres2 = osRtxMemoryAlloc(AppMallocAXISRAM, 2333, 0);
                AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
                printf("AXI SRAM 总大小 = %d 字节, 申请大小 = 2333 字节, 当前共使用大小 = %d 字节\r\n",
                                                        AXISRAMUsed->size, AXISRAMUsed->used);
                break;

            /* 释放从 AXI SRAM 申请的空间 */
            case KEY_UP_K2:
                /* 释放从 AXI SRAM 申请的 160 字节空间 */
                osRtxMemoryFree(AppMallocAXISRAM, AXISRAM_Addres0);
                AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
                printf("释放 AXI SRAM 动态内存区申请的 0160 字节, 当前共使用大小 = %d 字节\r\n",
                                                                AXISRAMUsed->used);

                /* 释放从 AXI SRAM 申请的 32 字节空间 */
                osRtxMemoryFree(AppMallocAXISRAM, AXISRAM_Addres1);
                AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
                printf("释放 AXI SRAM 动态内存区申请的 0032 字节, 当前共使用大小 = %d 字节\r\n",
                                                                AXISRAMUsed->used);

                /* 释放从 AXI SRAM 申请的 2333 字节空间 */
                osRtxMemoryFree(AppMallocAXISRAM, AXISRAM_Addres2);
                AXISRAMUsed = MemHeadPtr(AppMallocAXISRAM);
                printf("释放 AXI SRAM 动态内存区申请的 2333 字节, 当前共使用大小 = %d 字节\r\n",
                                                                AXISRAMUsed->used);
                break;

            /* 从 D2 域 SRAM 依次申请 200 字节, 96 字节和 4111 字节 */
            case KEY_DOWN_K3:
  /* 从 D2 域的 SRAM 申请 200 字节空间, 使用指针变量 SRAM1_Addres0 操作这些空间时不要超过 200 字节大小 */
                printf("=========================================================\r\n");
                SRAM1_Addres0 = osRtxMemoryAlloc(AppMallocSRAM1, 200, 0);
                SRAM1Used = MemHeadPtr(AppMallocSRAM1);
                printf("D2 域 SRAM 总大小 = %d 字节, 申请大小 = 0200 字节, 当前共使用大小 = %d 字节\r\n",
                                                        SRAM1Used->size, SRAM1Used->used);

  /* 从 D2 域的 SRAM 申请 96 字节空间, 使用指针变量 SRAM1_Addres1 操作这些空间时不要超过 96 字节大小 */
                SRAM1_Addres1 = osRtxMemoryAlloc(AppMallocSRAM1, 96, 0);
                SRAM1Used = MemHeadPtr(AppMallocSRAM1);
                printf("D2 域 SRAM 总大小 = %d 字节, 申请大小 = 0096 字节, 当前共使用大小 = %d 字节\r\n",
                                                            SRAM1Used->size, SRAM1Used->used);

  /* 从 D2 域的 SRAM 申请 4111 字节空间, 使用指针变量 SRAM1_Addres2 操作这些空间时不要超过 4111 字节大小 */
                SRAM1_Addres2 = osRtxMemoryAlloc(AppMallocSRAM1, 4111, 0);
                SRAM1Used = MemHeadPtr(AppMallocSRAM1);
                printf("D2 域 SRAM 总大小 = %d 字节, 申请大小 = 4111 字节, 当前共使用大小 = %d 字节\r\n",
                                                        SRAM1Used->size, SRAM1Used->used);
                break;

            /* 释放从 D2 域 SRAM 申请的空间 */
            case KEY_UP_K3:
                /* 释放从 D2 域的 SRAM 申请的 200 字节空间 */
                osRtxMemoryFree(AppMallocSRAM1, SRAM1_Addres0);
                SRAM1Used = MemHeadPtr(AppMallocSRAM1);
                printf("释放 D2 域 SRAM 动态内存区申请的 0200 字节, 当前共使用大小 = %d 字节\r\n",
                                                                SRAM1Used->used);

                /* 释放从 D2 域的 SRAM 申请的 96 字节空间 */
```

```
                osRtxMemoryFree(AppMallocSRAM1, SRAM1_Addres1);
                SRAM1Used = MemHeadPtr(AppMallocSRAM1);
                printf("释放 D2 域 SRAM 动态内存区申请的 0096 字节，当前共使用大小 = %d 字节\r\n",
                                                                    SRAM1Used->used);

                /* 释放从 D2 域的 SRAM 申请的 4111 字节空间 */
                osRtxMemoryFree(AppMallocSRAM1, SRAM1_Addres2);
                SRAM1Used = MemHeadPtr(AppMallocSRAM1);
                printf("释放 D2 域 SRAM 动态内存区申请的 4111 字节，当前共使用大小 = %d 字节\r\n",
                                                                    SRAM1Used->used);
                break;

            /* 从 D3 域 SRAM 依次申请 300 字节，128 字节和 5111 字节 */
            case JOY_DOWN_OK:
        /* 从D3域的SRAM申请300字节空间,使用指针变量SRAM4_Addres0操作这些空间时不要超过300字节大小 */
                printf("=============================================================\r\n");
                SRAM4_Addres0 = osRtxMemoryAlloc(AppMallocSRAM4, 300, 0);
                SRAM4Used = MemHeadPtr(AppMallocSRAM4);
                printf("D3 域 SRAM 总大小 =%d 字节，申请大小 =0300 字节，当前共使用大小 =%d 字节\r\n",
                                                    SRAM4Used->size, SRAM4Used->used);

    /* 从 D3 域的 SRAM 申请 96 字节空间，使用指针变量 SRAM4_Addres1 操作这些空间时不要超过 96 字节大小 */
                SRAM4_Addres1 = osRtxMemoryAlloc(AppMallocSRAM4, 128, 0);
                SRAM4Used = MemHeadPtr(AppMallocSRAM4);
                printf("D3 域 SRAM 总大小 =%d 字节，申请大小 =0128 字节，当前共使用大小 =%d 字节\r\n",
                                                        SRAM4Used->size, SRAM4Used->used);

/* 从 D3 域的 SRAM 申请 5111 字节空间，使用指针变量 SRAM4_Addres2 操作这些空间时不要超过 5111 字节大小 */
                SRAM4_Addres2 = osRtxMemoryAlloc(AppMallocSRAM4, 5111, 0);
                SRAM4Used = MemHeadPtr(AppMallocSRAM4);
                printf("D3 域 SRAM 总大小 =%d 字节，申请大小 =5111 字节，当前共使用大小 =%d 字节\r\n",
                                                    SRAM4Used->size, SRAM4Used->used);
                break;

            /* 释放从 D3 域 SRAM 申请的空间 */
            case JOY_UP_OK:
                /* 释放从 D3 域的 SRAM 申请的 300 字节空间 */
                osRtxMemoryFree(AppMallocSRAM4, SRAM4_Addres0);
                SRAM4Used = MemHeadPtr(AppMallocSRAM4);
                printf("释放 D3 域 SRAM 动态内存区申请的 0300 字节，当前共使用大小 = %d 字节\r\n",
                                                                    SRAM4Used->used);

                /* 释放从 D3 域的 SRAM 申请的 128 字节空间 */
                osRtxMemoryFree(AppMallocSRAM4, SRAM4_Addres1);
                SRAM4Used = MemHeadPtr(AppMallocSRAM4);
                printf("释放 D3 域 SRAM 动态内存区申请的 0128 字节，当前共使用大小 = %d 字节\r\n",
                                                                    SRAM4Used->used);

                /* 释放从 D3 域的 SRAM 申请的 5111 字节空间 */
                osRtxMemoryFree(AppMallocSRAM4, SRAM4_Addres2);
                SRAM4Used = MemHeadPtr(AppMallocSRAM4);
                printf("释放 D3 域 SRAM 动态内存区申请的 5111 字节，当前共使用大小 = %d 字节\r\n",
                                                                    SRAM4Used->used);
                break;

            default:
              /* 其它的键值不处理 */
              break;
```

```
                }
            }
        }
}
```

27.6 总结

本章节就为大家讲解这么多，还是比较有项目实用价值的，特别是 MP3 编解码，JPEG 编解码，视频播放器，矢量字体等需要动态内存的场合。

第28章 STM32H7 时间关键代码在 ITCM 执行的超简单方法

本章教程为大家分享一种时间关键代码在 ITCM 执行的简单方法，同时中断向量表和变量放 DTCM。

28.1 初学者重要提示
28.2 简单实现方法
28.3 实验例程说明（MDK）
28.4 总结

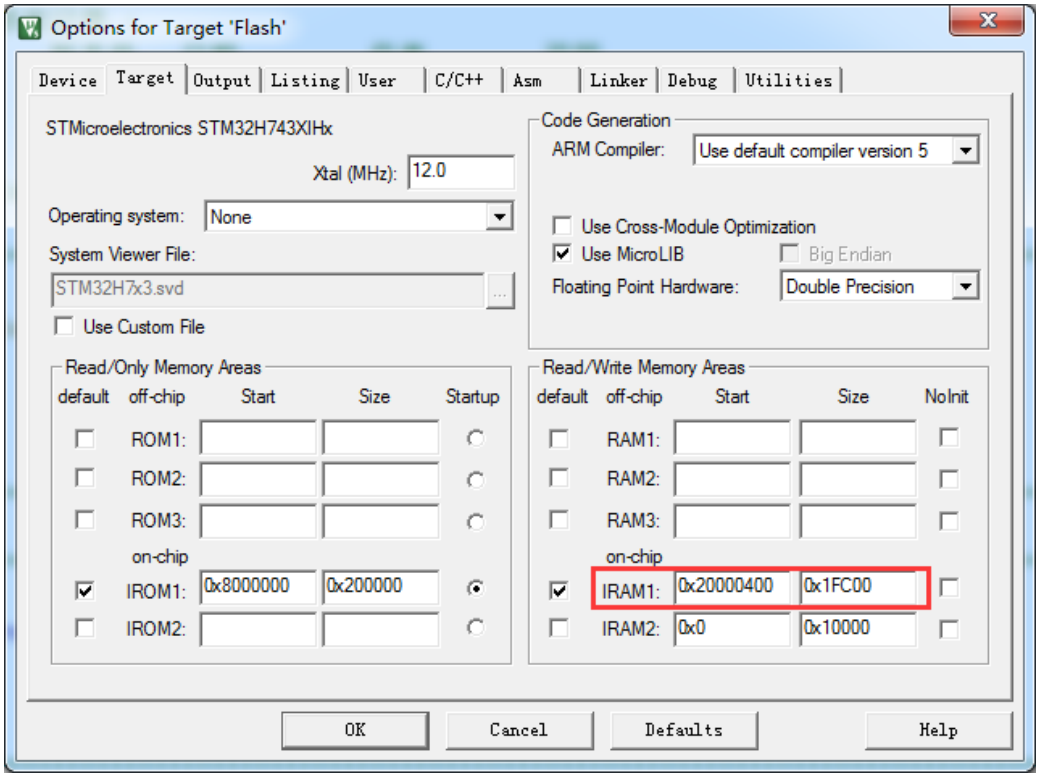
28.1 初学者重要提示

- 学习本章节前，务必优先学习第 25 章，了解 TCM，SRAM 等五块内存区的基础知识，比较重要。
- TCM : Tightly-Coupled Memory 紧密耦合内存 。ITCM 用于指令，DTCM 用于数据，特点是跟内核速度一样（400MHz），而片上 RAM 的速度基本都达不到这个速度（200MHz）。很多时候我们希望将需要实时性的程序和变量分别放在 ITCM 和 DTCM 里面执行，本章就是解决这个问题。
- 实现方法比较简单，基于 MDK 的 Option 选项设置下即可，无需操作分散加载。使用分散加载的好处是灵活，在设置复杂工程的内存映射方面比较方便。
- 实现这个功能的关键是要把所有程序都下载到 Flash，系统上电后让 MDK 中的库函数去将所需的程序加载到 RAM 里面，用户不要自己去加载，太麻烦。如果用户自己去加载就得搞个 bootloader 加载应用程序到 ITCM。这里所说的库函数是 MDK 里面的__main 封装起来了。

28.2 简单实现方法

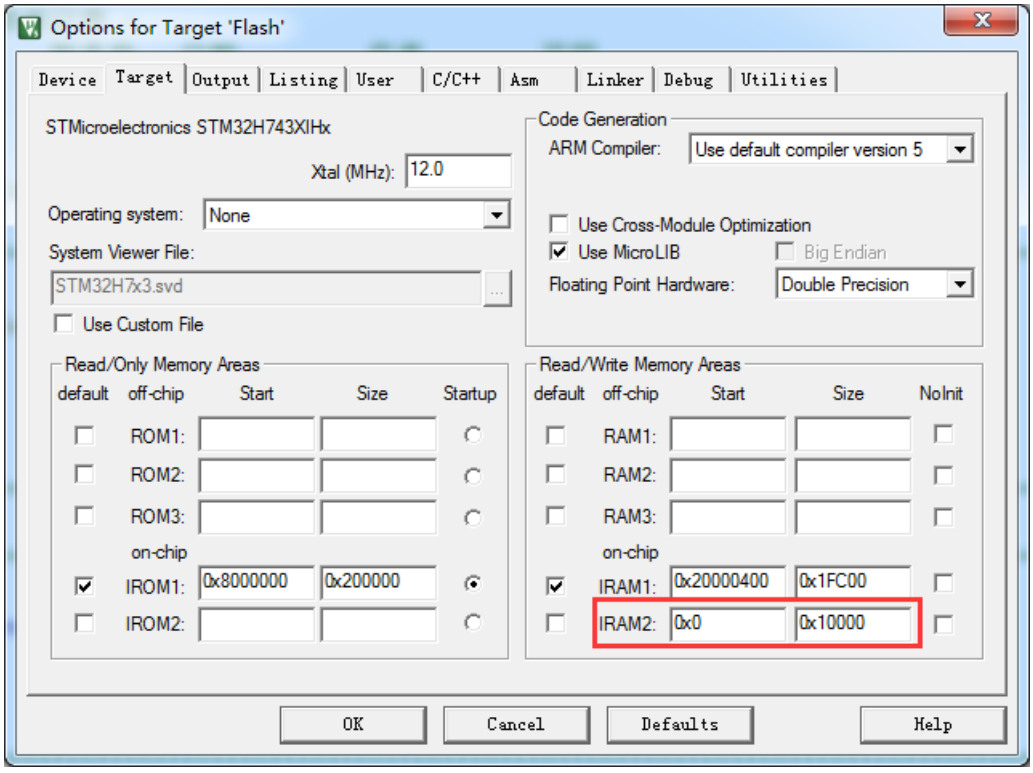
28.2.1 第 1 步，设置 DTCM

设置 DTCM 空间，前 0x400 大小的空间用于中断向量表，所以这里从 0x20000400 开始，用于各种变量需求：



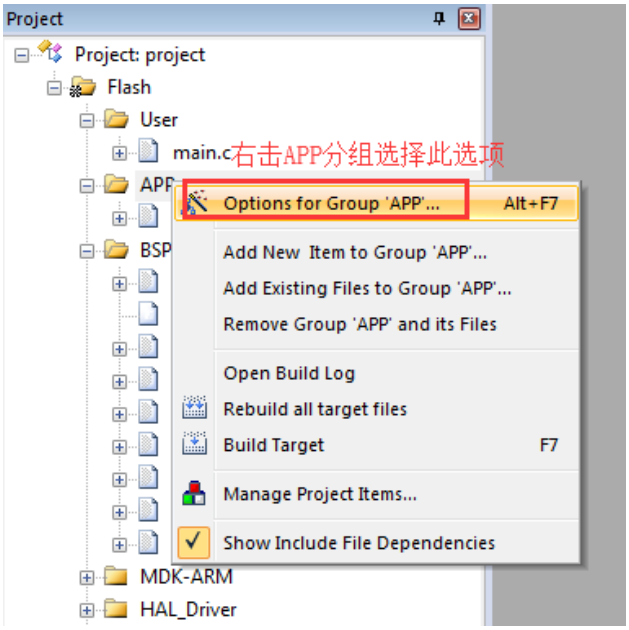
28.2.2第 2 步，添加 ITCM

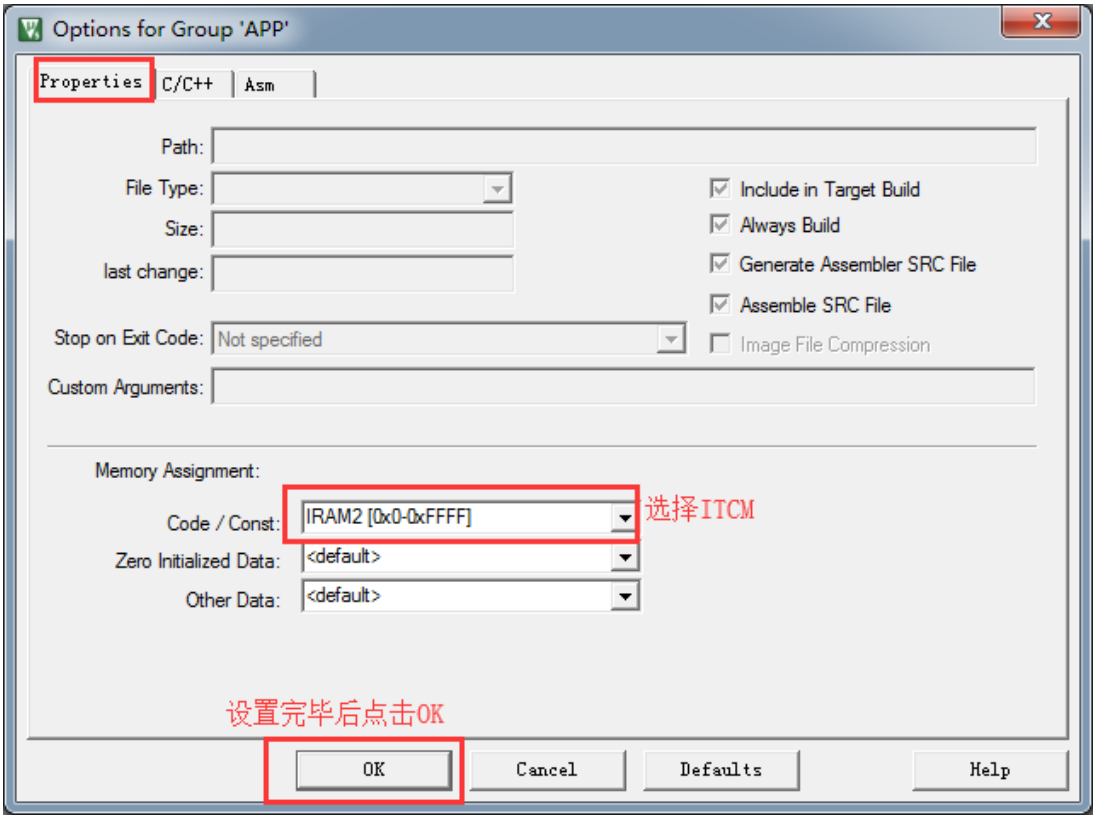
ITCM 的首地址是 0x0000 0000，大小 64KB：



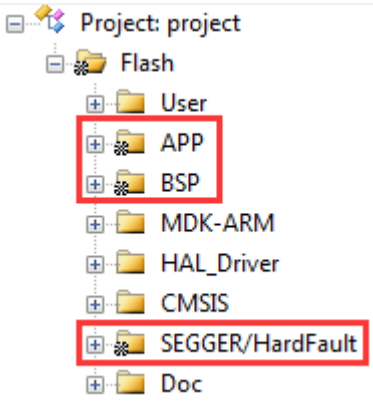
28.2.3第 3 步，选择在 ITCM 执行的代码

右击 MDK 分组，选择使用 ITCM，这里设置了 APP 分组、BSP 分组和 SEGGER/HardFault 分组。以 APP 分组为例，设置方法如下：





BSP 分组和 SEGGER/HardFault 分组也设置完毕后，可以看到小雪花标识



而进入 main 函数之前的所有代码，含 main 函数所在的文件 main.c 切不要设置，这个之前的代码我们都需要在 flash 里面执行。这些代码仅执行一次以后不会执行，所以不用管他们，之后的所有代码都可以放在 ITCM 里面。

28.2.4第 4 步，复制中断向量表到 DTCM

前面三步设置完毕后，将中断向量表从 flash 中复制到 DTCM，主要存储的 DTCM 地址要 0x200 对齐。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: 标准 c 程序入口。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
int main(void)
{
    uint32_t *SouceAddr = (uint32_t *)FLASH_BANK1_BASE;
    uint32_t *DestAddr =  (uint32_t *)D1_DTCMRAM_BASE;

    memcpy(DestAddr, SouceAddr, 0x400);
```

```
    /* 设置中断向量表到 ITCM 里面 */
    SCB->VTOR = D1_DTCMRAM_BASE;

    MainRAM();
}
```

至此就设置完毕了，另外注意以下两点：

- 不限制设置分组，单独设置一个 C 文件也是可以的。
- 如果大家将 HAL_Driver 分组也放在了 ITCM 里面，会有如下警告，这个不用管，是删除了冗余函数。

```
Build Output
compiling stm32h7xx_hal_sram.c...
compiling stm32h7xx_hal_uart_ex.c...
compiling stm32h7xx_hal_tim_ex.c...
compiling stm32h7xx_hal_dma.c...
compiling stm32h7xx_hal_mdma.c...
assembling HardFaultHandler.S...
compiling system_stm32h7xx.c...
compiling SEGGER_HardFaultHandler.c...
linking...
.\Objects\output.sct(34): warning: L6329W: Pattern stm32h7xx_hal_dma.o(RO) only matches removed unused sections.
.\Objects\output.sct(35): warning: L6329W: Pattern stm32h7xx_hal_uart_ex.o(RO) only matches removed unused sections.
.\Objects\output.sct(36): warning: L6329W: Pattern stm32h7xx_hal_tim_ex.o(RO) only matches removed unused sections.
.\Objects\output.sct(37): warning: L6329W: Pattern stm32h7xx_hal_mdma.o(RO) only matches removed unused sections.
```

28.3 实验例程说明（MDK）

配套例子：

V7-007_时间关键代码在 ITCM 执行的超简单方法

实验目的：

1. 学习时间关键代码在 ITCM 执行的超简单方法，同时中断向量表和变量放 DTCM。

实验内容：

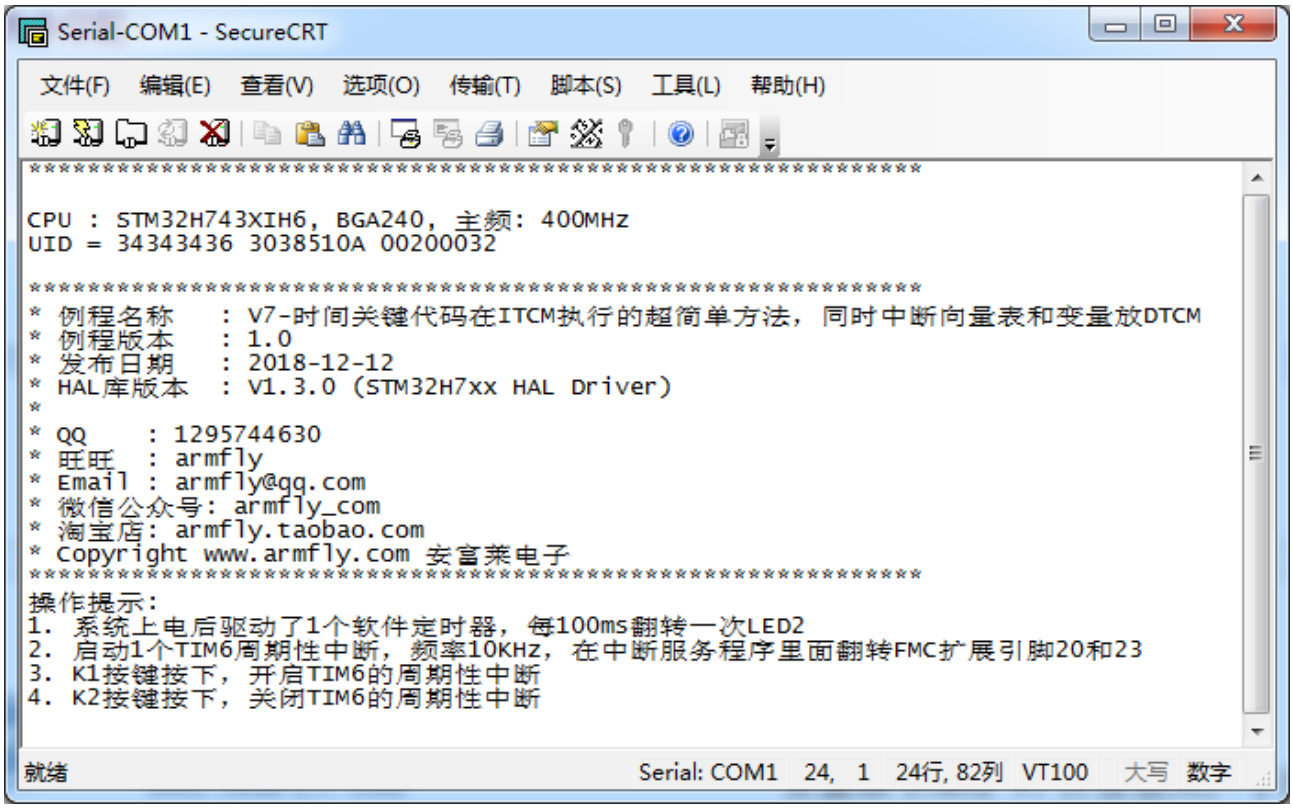
1. 系统上电后驱动了 1 个软件定时器，每 100ms 翻转一次 LED2。
2. 启动 1 个 TIM6 周期性中断，频率 10KHz，在中断服务程序里面翻转 FMC 扩展引脚 20 和 23。

实验操作：

1. K1 按键按下，开启 TIM6 的周期性中断。
2. K2 按键按下，关闭 TIM6 的周期性中断。

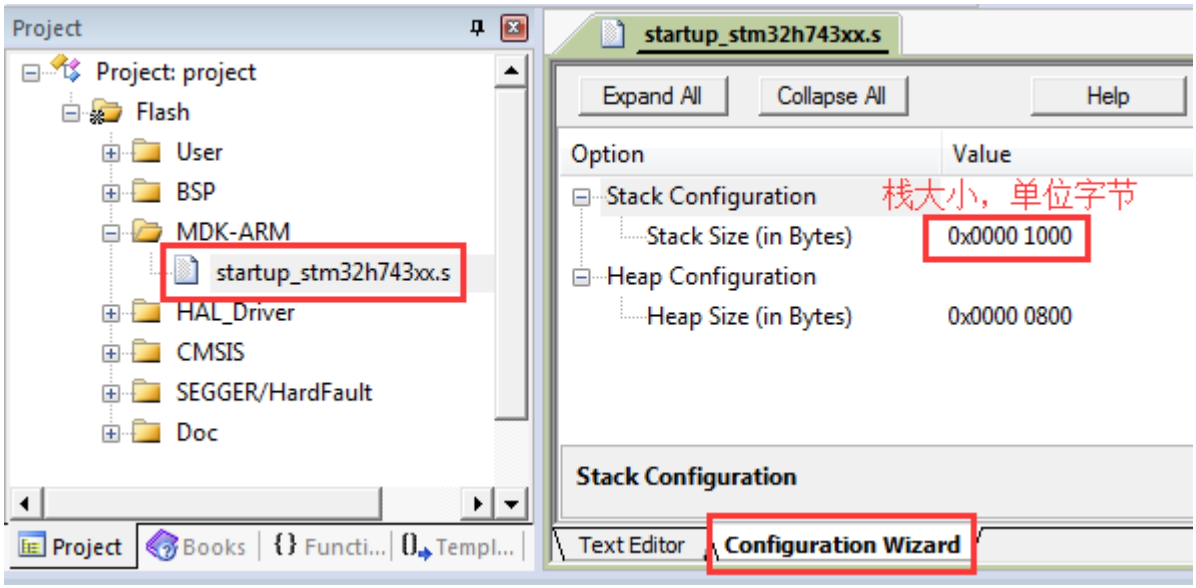
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

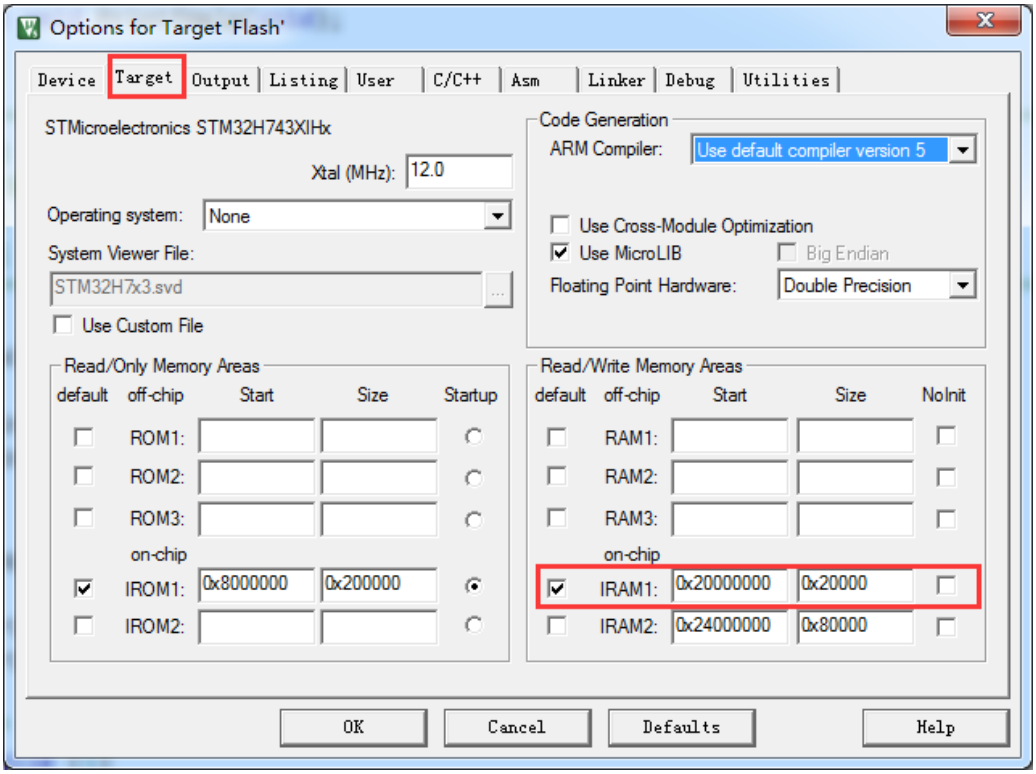


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
```

```
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- 系统上电后驱动了 1 个软件定时器，每 100ms 翻转一次 LED2。
- 启动 1 个 TIM6 周期性中断，频率 10KHz，在中断服务程序里面翻转 FMC 扩展引脚 20 和 23。
- K1 按键按下，开启 TIM6 的周期性中断。
- K2 按键按下，关闭 TIM6 的周期性中断。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: 标准 c 程序入口。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
int main(void)
{
    uint32_t *SouceAddr = (uint32_t *)FLASH_BANK1_BASE;
    uint32_t *DestAddr =  (uint32_t *)D1_DTCMRAM_BASE;

    memcpy(DestAddr, SouceAddr, 0x400);

    /* 设置中断向量表到 ITCM 里面 */
    SCB->VTOR = D1_DTCMRAM_BASE;
```

```
    MainRAM();
}

/*
*********************************************************************************************************
*    函 数 名: MainRAM
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int MainRAM(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    bsp_SetTIMforInt(TIM6, 10000, 2, 0); /* 设置为 10KHz 频率定时器中断*/

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，开启 TIM6 的周期性中断*/
                    TIM6->DIER |= TIM_IT_UPDATE;
                    break;

                case KEY_DOWN_K2:           /* K2 键按下，关闭 TIM6 的周期性中断*/
                    TIM6->DIER &= ~TIM_IT_UPDATE;
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
```

```
}

/*
*********************************************************************************************************
*    函 数 名: TIM6_DAC_IRQHandler
*    功能说明: TIM6 定时中断服务程序
*    返 回 值: 无
*********************************************************************************************************
*/
void TIM6_DAC_IRQHandler(void)
{
    if((TIM6->SR & TIM_FLAG_UPDATE) != RESET)
    {
        /* 清除更新标志 */
        TIM6->SR = ~ TIM_FLAG_UPDATE;

        /* 翻转 FMC 扩展引脚 20 和 23 脚 */
        HC574_TogglePin(GPIO_PIN_23);
        HC574_TogglePin(GPIO_PIN_20);
    }
}
```

28.4 总结

本章节就为大家交流这么多，对速度有要求的应用部分，建议使用 ITCM 和 DTCM 来达到最高性能。

第29章 STM32H7 的 USART 串口基础知识和 HAL 库 API

本章节为大家讲解 USART（Universal synchronous asynchronous receiver transmitter，通用同步异步收发器）的基础知识和对应的 HAL 库 API。相比之前的 F1 和 F4 系列，增加了不少新特性。

29.1 初学者重要提示
29.2 串口基础知识
29.3 串口的 HAL 库用法
29.4 源文件 stm32h7xx_hal_uart.c
29.5 总结

29.1 初学者重要提示

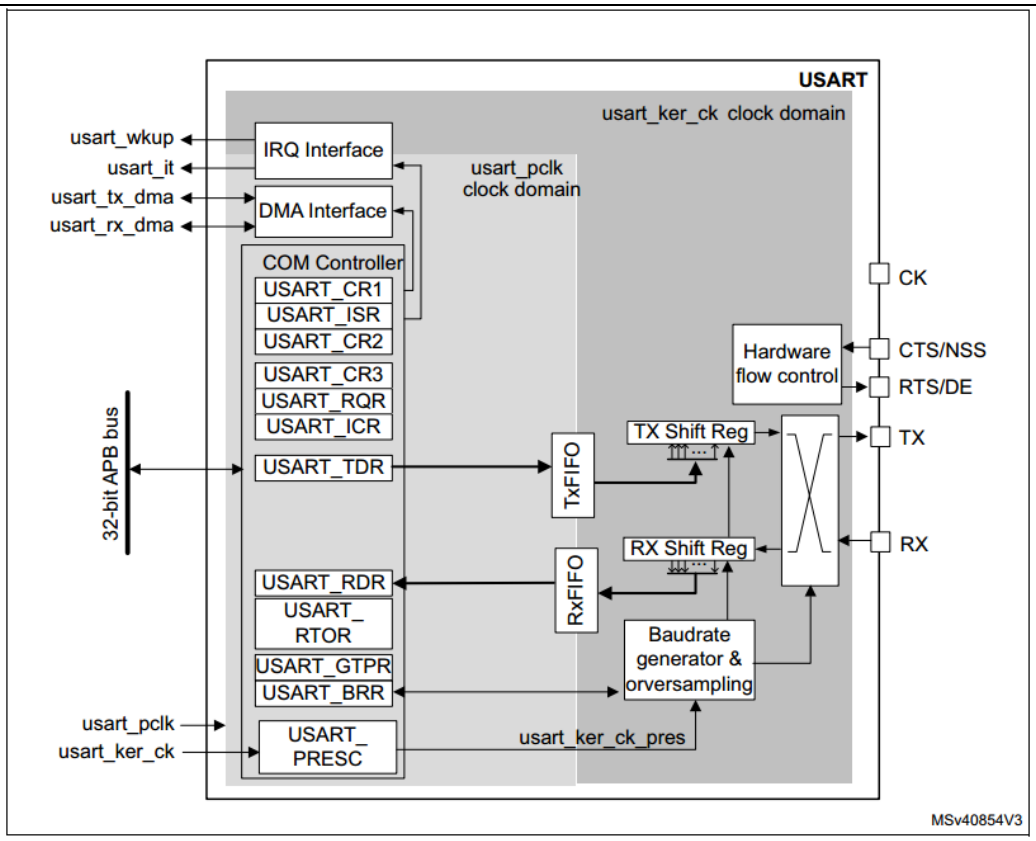
- 学习串口外设推荐从硬件框图开始了解基本的功能特性，然后逐步深入了解各种特性，这种方式方便记忆和以后查阅。而串口的通信学习，推荐看时序图。
- STM32H7 的串口比 STM32F4 和 F1 的串口支持了更多高级特性。比如超时接收检测、自适应波特率、TX 和 RX 引脚互换等功能。
- 部分中断标志是可以通过操作发送数据寄存器 TDR 或者接收数据寄存器 RDR 实现清除，这点要特别注意，详情看本章 29.3.4 小节。
- 初次使用 USART，还是有不少注意事项的，详情看本章 29.3.3 小节和 29.4.1 小节。

29.2 串口基础知识

USART 的全称是 Universal synchronous asynchronous receiver transmitter，中文意思是通用同步异步收发器。我们经常使用串口是异步串口，简称 UART。

29.2.1 串口的硬件框图

认识一个外设，最好的方式就是看它的框图，方便我们快速的了解串口的基本功能，然后再看手册了解细节。



通过这个框图，我们可以得到如下信息：

- IRQ Interface 中断接口

  用于实现中断方式的串口唤醒 usart_wkup 和串口的相关中断 usart_it。

- DMA Interface DMA 接口

  实现串口发送 usart_tx_dma 和接收 usart_rx_dma 的 DMA 方式。

- COM Contronller 串口控制器

  串口相关的寄存器基本都在这部分。

- TxFIFO 和 RxFIFO

  串口的发送和接收都支持了硬件 FIFO 功能。

- TX 和 RX 引脚的互换功能

  发送偏移寄存器（TX Shift Reg）和接收偏移寄存器（RX Shift Reg）与 TX 引脚，RX 引脚之间弄了个交叉连接，这里的意思是支持了引脚互换功能，这样大家在设计 PCB 的时候就可以比较随性了，接反了也没有关系。

- 发送过程经过的寄存器

  依次是 USART_TDR -> TxFIFO ->Tx Shift Reg 偏移寄存器 –> TX 或者 RX 引脚。

- 接收经过的寄存器

  依次是 TX 或者 RX 引脚-> Rx Shift Reg 偏移寄存器->RxFIFO –>USART_RDR。

- 两个时钟 usart_pclk 和 usart_ker_ck

  这两个时钟是独立的，作用如下：

- usart_pclk
  用于为外设总线提供时钟。
- usart_ker_ck
  串口外设的时钟源。

29.2.2 串口的基本功能

STM32 的串口功能很强大，支持太多的模式。我们只需关心我们最常用的特性即可。我们的串口驱动使用的串口中断+FIFO 结构，没有使用 DMA。因此我们只讨论和串口中断、串口常规参数有关的知识。

STM32 串口的优越特性：（只列了举常用的）

- 任意波特率。硬件采用分数波特率发生器系统，可以设置任意的波特率，最高达 4.5Mbits/s。这一点很重要。比如 ES8266 串口 WIFI 芯片，上电时有个奇怪的波特率 74880bps，当然 STM32 是可以支持的。
- 可编程数据字长度，支持 7bit，8bit 和 9bit。
- 可配置的停止位。支持 1 或 2 个停止位。
- 发送器和接收器可以单独使能。比如 GPS 应用只需要串口接收，那么发送的 GPIO 就可以节省出来用作其他功能。
- 检测标志和中断：
  - 接收缓冲器满，可产生中断。串口中断服务程序据此判断是否接收到数据。
  - 发送缓冲器空，可产生中断。串口中断服务程序据此启动发送下一个数据。
  - 传输结束标志，可产生中断。用于 RS485 通信，等最后一个字节发送完毕后，需要控制 RS485 收发器芯片切换为接收模式。

其它中断不常用，包括：CTS 改变、LIN 断开符检测、检测到总线为空闲（在 DMA 不定长接收方式会用到）、溢出错误、帧错误、噪音错误、校验错误。

29.2.3 串口的高级特性

相比 F1 和 F4 系列，H7 系列的串口支持了一些高级特性，比如：

- 数据逻辑电平翻转。
- RX 和 TX 引脚交换。
- 超时接收特性。
- MSB 位先发送。
- 自适应波特率。
- 外接 485 的 PHY 芯片时，硬件支持收发切换，无需用户手动控制 DE 引脚。

通过下面的表格，可以对串口 1-8 支持的功能有个全面的认识：

Table 6. USART features

USART modes/features(1)	USART1/2/3/6	UART4/5/7/8
Hardware flow control for modem	X	X
Continuous communication using DMA	X	X
Multiprocessor communication	X	X
Synchronous mode (Master/Slave)	X	-
Smartcard mode	X	-
Single-wire Half-duplex communication	X	X
IrDA SIR ENDEC block	X	X
LIN mode	X	X
Dual clock domain and wakeup from low power mode	X	X
Receiver timeout interrupt	X	X
Modbus communication	X	X
Auto baud rate detection	X	X
Driver Enable	X	X
USART data length	7, 8 and 9 bits	
Tx/Rx FIFO	X	X
Tx/Rx FIFO size	16	

1. X = supported.

29.2.4串口的自适应波特率

这里简单为大家介绍下高级特性里面的自适应波特率：

- 应用场合：
  - 系统的通信速度未知。
  - 系统使用相对低精度的时钟源，并且该机制能够在没有测量时钟偏差的情况下获得正确的波特率。
- 测量范围：

  注，usart_ker_ck_pres 在不做串口分频的情况下，是 100MHz。
  - 8 倍过采样的情况下，测量速度范围是 usart_ker_ck_pres/65535 到 usart_ker_ck_pres/8。最高串口速度是 100MHz / 8 = 12.5Mbps。
  - 16 倍过采样的情况下，速度范围是 usart_ker_ck_pres/65535 到 usart_ker_ck_pres/16。最高串口速度是 100MHz / 16 = 6.25Mbsp。
- 测量方法：

根据不同的字符特征，支持四种自适应方法。自适应波特率在同步数据接收期间会测量多次，而且每次测量都会跟前一次做比较。

当前支持四种字符样式进行识别，识别成功后会将中断状态寄存的 ABRF 位置 1，其中模式 2 发送几次 0x7F 基本都可以适应成功，相对好用，模式 3 跟模式 2 差不多，而模式 0 检测起始位的持续时间和模式 1 检测起始位以及第 1 个 bit 的数据位持续时间，这两种模式不好用。

针对自适应模式，专门制作过一个例子，供大家参考：
http://www.armbbs.cn/forum.php?mod=viewthread&tid=86289 。

29.2.5 串口的数据帧格式

串口支持的帧格式如下（M 和 PCE 都是 USART_CR1 寄存器的位，其中 M 位用于控制帧长度，PCE 用于使能奇偶校验位）：

M bits	PCE bit	USART frame(1)
00	0	| SB | 8 bit data | STB |
00	1	| SB | 7-bit data | PB | STB |
01	0	| SB | 9-bit data | STB |
01	1	| SB | 8-bit data PB | STB |
10	0	| SB | 7bit data | STB |
10	1	| SB | 6-bit data | PB | STB |

1. Legends: SB: start bit, STB: stop bit, PB: parity bit. In the data register, the PB is always taking the MSB position (8th or 7th, depending on the M bit value).

这里特别注意奇偶校验位，用户在配置的时候可以选择奇校验和偶校验，校验位是占据的最高位。比如选择 M=00，PCE=1，即 7bit 的数据位。

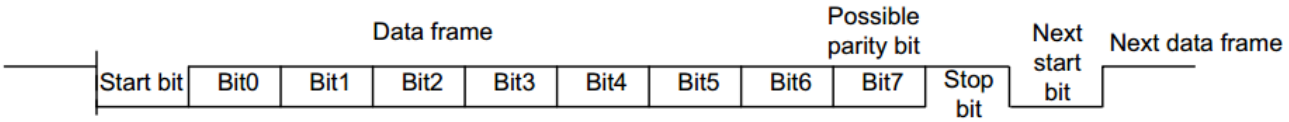
- 串口发送数据：

  如果发送的 7bit 数据是 111 0011，这个里面有奇数个 1，那么选择偶校验的情况下，校验位 = 1，凑够偶数个 1，而选择奇校验的情况下，校验位 = 0，因为已经是奇数个 1。校验位不需要用户去计算，是硬件自动生成的。

- 串口接收数据：

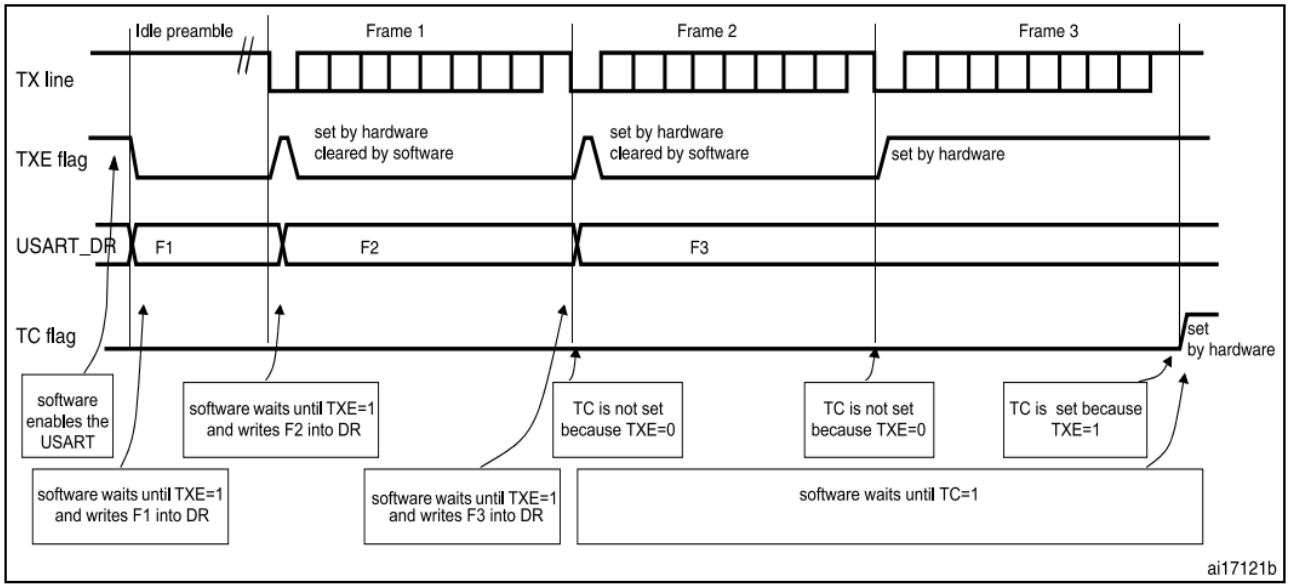
  根据用户设置的奇校验或者偶校验类型，串口硬件会对接收到的数据做校验，如果失败，USART_ISR 寄存器的 PE 位会被置 1。如果使能了对应的中断 PEIE，那么失败的时候还会产生中断。

了解到帧格式后，再来看一下实际数据发送时，数据位的先后顺序：



29.2.6串口发送时序图

这个时序图非常具有代表性，可以帮助大家很好的理解 TC 发送完成中断和 TXE 空中断。



Note: When FIFO management is enabled, the TXFNF flag is used for data transmission.

29.2.7同步串口和异步串口的区别

异步通信是按字符传输的。每传输一个字符就用起始位来进行收、发双方的同步，不会因收发双方的时钟频率的小的偏差导致错误。这种传输方式利用每一帧的起、止信号来建立发送与接收之间的同步。

异步的特点是：每帧内部各位均采用固定的时间间隔，而帧与帧之间的间隔是随机的。接收机完全靠每一帧的起始位和停止位来识别字符是正在进行传输还是传输结束。

同步通信的发送和接收双方要保持完全的同步，因此要求接收和发送设备必须使用同一时钟。优点是可以实现高速度、大容量的数据传送；缺点是要求发生时钟和接收时钟保持严格同步，同时硬件复杂。

可以这样说，不管是异步通信还是同步通信都需要进行同步，只是异步通信通过传送字符内的起始位来进行同步，而同步通信采用共用外部时钟来进行同步。所以，可以说前者是自同步，后者是外同步。

29.2.8单工，半双工和全双工通讯

单工：在一个单工的串行通讯系统中，一般至少有两根线（信号线和地线），数据传送只有一个方向，例如可以使用单工数据传送将数据从一个简单的数据监测系统传送到 PC 上。

半双工：在半双工串行通信系统中，一般同样要求至少有两根线。这里的数据传送是双向的。然而，

同一个时刻只能为一个方向。在上面的数据监测的例子中做了一些变化，可以使用半双工通讯机制发送信息到嵌入式模块（来设置参数，比如采样率）。此外，在其他时候，可以使用这个种连接将嵌入式装置上的数据下载到 PC 中。

全双工：在一个全双工的串行通信系统中，一般要求至少有三根线（信号线 A，信号线 B 和地线）。信号线 A 将传输一个方向上的数据，同时信号线 B 传送另一个方向上的数据。

29.3 串口的 HAL 库用法

串口的 HAL 库用法其实就是几个结构体变量成员的配置和使用，然后配置 GPIO、时钟，并根据需要配置 NVIC、中断和 DMA。下面我们逐一展开为大家做个说明。

29.3.1 串口寄存器结构体 USART_TypeDef

USART 相关的寄存器是通过 HAL 库中的结构体 USART_TypeDef 定义的，在 stm32h743xx.h 中可以找到这个类型定义：

```
typedef struct
{
  __IO uint32_t CR1;    /*!< USART Control register 1,                 Address offset: 0x00 */
  __IO uint32_t CR2;    /*!< USART Control register 2,                 Address offset: 0x04 */
  __IO uint32_t CR3;    /*!< USART Control register 3,                 Address offset: 0x08 */
  __IO uint32_t BRR;    /*!< USART Baud rate register,                 Address offset: 0x0C */
  __IO uint16_t GTPR;   /*!< USART Guard time and prescaler register,  Address offset: 0x10 */
  uint16_t  RESERVED2;  /*!< Reserved, 0x12                                                 */
  __IO uint32_t RTOR;   /*!< USART Receiver Time Out register,         Address offset: 0x14 */
  __IO uint16_t RQR;    /*!< USART Request register,                   Address offset: 0x18 */
  uint16_t  RESERVED3;  /*!< Reserved, 0x1A                                                 */
  __IO uint32_t ISR;    /*!< USART Interrupt and status register,      Address offset: 0x1C */
  __IO uint32_t ICR;    /*!< USART Interrupt flag Clear register,      Address offset: 0x20 */
  __IO uint16_t RDR;    /*!< USART Receive Data register,              Address offset: 0x24 */
  uint16_t  RESERVED4;  /*!< Reserved, 0x26                                                 */
  __IO uint16_t TDR;    /*!< USART Transmit Data register,             Address offset: 0x28 */
  uint16_t  RESERVED5;  /*!< Reserved, 0x2A                                                 */
  __IO uint32_t PRESC;  /*!< USART clock Prescaler register,           Address offset: 0x2C */
} USART_TypeDef;
```

这个结构体的成员名称和排列次序和 CPU 的 USART 寄存器是一 一对应的。

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

下面我们看下 USART1、USART2 ... UART8 的定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE           ((uint32_t)0x40000000)
#define D2_APB1PERIPH_BASE    PERIPH_BASE
#define D2_APB2PERIPH_BASE    (PERIPH_BASE + 0x00010000)

#define USART1_BASE           (D2_APB2PERIPH_BASE + 0x1000)
#define USART2_BASE           (D2_APB1PERIPH_BASE + 0x4400)
```

```
#define USART3_BASE             (D2_APB1PERIPH_BASE + 0x4800)
#define UART4_BASE              (D2_APB1PERIPH_BASE + 0x4C00)
#define UART5_BASE              (D2_APB1PERIPH_BASE + 0x5000)
#define USART6_BASE             (D2_APB2PERIPH_BASE + 0x1400)
#define UART7_BASE              (D2_APB1PERIPH_BASE + 0x7800)
#define UART8_BASE              (D2_APB1PERIPH_BASE + 0x7C00)

#define USART1                  ((USART_TypeDef *) USART1_BASE) <----- 展开这个宏，(USART_TypeDef *) 0x40011000
#define USART2                  ((USART_TypeDef *) USART2_BASE)
#define USART3                  ((USART_TypeDef *) USART3_BASE)
#define UART4                   ((USART_TypeDef *) UART4_BASE)
#define UART5                   ((USART_TypeDef *) UART5_BASE)
#define USART6                  ((USART_TypeDef *) USART6_BASE)
#define UART7                   ((USART_TypeDef *) UART7_BASE)
#define UART8                   ((USART_TypeDef *) UART8_BASE)
```

我们访问 USART1 的 CR1 寄存器可以采用这种形式：USART1->CR1 = 0。

29.3.2 串口句柄结构体 UART_HandleTypeDef

HAL 库在 USART_TypeDef 的基础上封装了一个结构体 UART_HandleTypeDef，定义如下：

```
typedef struct
{
  USART_TypeDef            *Instance;        /*!< UART registers base address        */
  UART_InitTypeDef         Init;             /*!< UART communication parameters      */
  UART_AdvFeatureInitTypeDef AdvancedInit;   /*!< UART Advanced Features initialization parameters */
  uint8_t                  *pTxBuffPtr;      /*!< Pointer to UART Tx transfer Buffer */
  uint16_t                 TxXferSize;       /*!< UART Tx Transfer size              */
  __IO uint16_t            TxXferCount;      /*!< UART Tx Transfer Counter           */
  uint8_t                  *pRxBuffPtr;      /*!< Pointer to UART Rx transfer Buffer */
  uint16_t                 RxXferSize;       /*!< UART Rx Transfer size              */
  __IO uint16_t            RxXferCount;      /*!< UART Rx Transfer Counter           */
  uint16_t                 Mask;             /*!< UART Rx RDR register mask          */
  DMA_HandleTypeDef        *hdmatx;          /*!< UART Tx DMA Handle parameters      */
  DMA_HandleTypeDef        *hdmarx;          /*!< UART Rx DMA Handle parameters      */
  HAL_LockTypeDef          Lock;             /*!< Locking object                     */
  __IO HAL_UART_StateTypeDef    gState;      /*!< UART state information related to global Handle management
                                                and also related to Tx operations.
                                                This parameter can be a value of @ref HAL_UART_StateTypeDef */
  __IO HAL_UART_StateTypeDef    RxState;     /*!< UART state information related to Rx operations.
                                                This parameter can be a value of @ref HAL_UART_StateTypeDef */
  __IO uint32_t            ErrorCode;        /*!< UART Error code                    */
}UART_HandleTypeDef;
```

这里重点介绍前三个参数，其它参数主要是 HAL 库内部使用的。

- USART_TypeDef *Instance

  这个参数是寄存器的例化，方便操作寄存器，比如使能串口的发送空中断。

  SET_BIT(huart->Instance->CR1， USART_CR1_TXEIE)。

- UART_InitTypeDef Init

  这个参数是用户接触最多的，用于配置串口的基本参数，像波特率、奇偶校验、停止位等。

  UART_InitTypeDef 结构体的定义如下：

```
typedef struct
```

```
{
  uint32_t BaudRate;       /* 波特率 */
  uint32_t WordLength;     /* 数据位长度 */
  uint32_t StopBits;       /* 停止位 */
  uint32_t Parity;         /* 奇偶校验位 */
  uint32_t Mode;           /* 发送模式和接收模式使能 */
  uint32_t HwFlowCtl;      /* 硬件流控制 */
  uint32_t OverSampling;   /* 过采样，可以选择 8 倍和 16 倍过采样 */
  uint32_t Prescaler;      /* 串口分频 */
  uint32_t FIFOMode;         /* 串口 FIFO 使能 */
  uint32_t TXFIFOThreshold;  /* 发送 FIFO 的阀值 */
  uint32_t RXFIFOThreshold;  /* 接收 FIFO 的阀值 */
}UART_InitTypeDef;
```

- UART_AdvFeatureInitTypeDef AdvancedInit

这个参数用于配置串口的高级特性。具体支持的功能参数如下：

```
typedef struct
{
  uint32_t AdvFeatureInit;       /* 初始化的高级特性类别 */
  uint32_t TxPinLevelInvert;     /* Tx 引脚电平翻转 */
  uint32_t RxPinLevelInvert;     /* Rx 引脚电平翻转 */
  uint32_t DataInvert;           /* 数据逻辑电平翻转 */
  uint32_t Swap;                 /* Tx 和 Rx 引脚交换 */
  uint32_t OverrunDisable;       /* 接收超时检测禁止 */
  uint32_t DMADisableonRxError;  /* 接收出错，禁止 DMA */
  uint32_t AutoBaudRateEnable;   /* 自适应波特率使能 */
  uint32_t AutoBaudRateMode;     /* 自适应波特率的四种检测模式选择 */
  uint32_t MSBFirst;             /* 发送或者接收数据时，高位在前 */
} UART_AdvFeatureInitTypeDef;
```

配置串口参数，其实就是配置结构体 UART_HandleTypeDef 的成员。比如下面配置为波特率 115200，8 个数据位，无奇偶校验，1 个停止位。

```
UART_HandleTypeDef UartHandle;

/* USART3 工作在 UART 模式 */
/* 配置如下:
  - 数据位 = 8 Bits
  - 停止位 = 1 bit
  - 奇偶校验位 = 无
  - 波特率 = 115200bsp
  - 硬件流控制 (RTS 和 CTS 信号) */
UartHandle.Instance        = USART3;

UartHandle.Init.BaudRate     = 115200;
UartHandle.Init.WordLength   = UART_WORDLENGTH_8B;
UartHandle.Init.StopBits     = UART_STOPBITS_1;
UartHandle.Init.Parity       = UART_PARITY_NONE;
UartHandle.Init.HwFlowCtl    = UART_HWCONTROL_NONE;
UartHandle.Init.Mode         = UART_MODE_TX_RX;
UartHandle.Init.OverSampling = UART_OVERSAMPLING_16;
UartHandle.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;

if(HAL_UART_Init(&UartHandle) != HAL_OK)
{
    Error_Handler();
}
```

29.3.3 串口的底层配置（GPIO、时钟、中断等）

串口外设的基本参数配置完毕后还不能使用，还需要配置 GPIO、时钟、中断等参数，比如下面配置串口 1，使用引脚 PA9 和 PA10。

```
/* 串口 1 的 GPIO  PA9, PA10 */
#define USART1_CLK_ENABLE()              __HAL_RCC_USART1_CLK_ENABLE()
#define USART1_TX_GPIO_CLK_ENABLE()      __HAL_RCC_GPIOA_CLK_ENABLE()
#define USART1_TX_GPIO_PORT              GPIOA
#define USART1_TX_PIN                    GPIO_PIN_9
#define USART1_TX_AF                     GPIO_AF7_USART1

/*
*********************************************************************************************************
*    函 数 名: InitHardUart
*    功能说明: 配置串口的硬件参数和底层
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void InitHardUart(void)
{
    GPIO_InitTypeDef  GPIO_InitStruct;
    RCC_PeriphCLKInitTypeDef RCC_PeriphClkInit;

#if UART1_FIFO_EN == 1      /* 串口 1 */
    /* 使能 GPIO TX/RX 时钟 */
    USART1_TX_GPIO_CLK_ENABLE();
    USART1_RX_GPIO_CLK_ENABLE();

    /* 使能 USARTx 时钟 */
    USART1_CLK_ENABLE();

    /* 配置 TX 引脚 */
    GPIO_InitStruct.Pin       = USART1_TX_PIN;
    GPIO_InitStruct.Mode      = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Pull      = GPIO_PULLUP;
    GPIO_InitStruct.Speed     = GPIO_SPEED_FREQ_VERY_HIGH;
    GPIO_InitStruct.Alternate = USART1_TX_AF;
    HAL_GPIO_Init(USART1_TX_GPIO_PORT, &GPIO_InitStruct);

    /* 配置 RX 引脚 */
    GPIO_InitStruct.Pin = USART1_RX_PIN;
    GPIO_InitStruct.Alternate = USART1_RX_AF;
    HAL_GPIO_Init(USART1_RX_GPIO_PORT, &GPIO_InitStruct);

    /* 配置 NVIC the NVIC for UART */
    HAL_NVIC_SetPriority(USART1_IRQn, 0, 1);
    HAL_NVIC_EnableIRQ(USART1_IRQn);

    /* 配置波特率、奇偶校验 */
    bsp_SetUartParam(USART1,  UART1_BAUD, UART_PARITY_NONE, UART_MODE_TX_RX);

    SET_BIT(USART1->ICR, USART_ICR_TCCF);  /* 清除 TC 发送完成标志 */
    SET_BIT(USART1->RQR, USART_RQR_RXFRQ); /* 清除 RXNE 接收标志 */
    SET_BIT(USART1->CR1, USART_CR1_RXNEIE);/* 使能 PE. RX 接受中断 */
```

```
#endif
}
```

总结下来就是以下几点：

- 配置 GPIO 引脚时钟。
- 配置 USART 时钟。
- 配置 USART 的发送和接收引脚。
- 通过 NVIC 配置中断。
- 配置波特率，奇偶校验等，在上一小节有讲。
- 清除 TC 和 RXNE 标志，使能接收中断。

关于这个底层配置有以下几点要着重说明下：

- 串口发送和接收引脚的复用模式选择已经被 HAL 库定义好，放在了 stm32h7xx_hal_gpio_ex.h 文件里面。比如串口 1 有两个复用

```
#define GPIO_AF4_USART1     ((uint8_t)0x04)  /* USART1 Alternate Function mapping */
#define GPIO_AF7_USART1     ((uint8_t)0x07)  /* USART1 Alternate Function mapping */
```

具体使用那个，要看数据手册，比如我们这里使用引脚 PA9 和 PA10，对应的复用如下：

	Port	AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7	AF8	AF9	AF10	AF11	AF12	AF13	AF14	AF15
		SYS	TIM1/2/16/17/LPTIM1/HRTIM1	SAI1/TIM3/4/5/12/HRTIM1	LPUART/TIM8/LPTIM2/3/4/5/HRTIM1/DFSDM	I2C1/2/3/4/USART1/TIM15/LPTIM2/DFSDM/CEC	SPI1/2/3/4/5/6/CEC	SPI2/3/SAI1/3/I2C4/UART4/DFSDM	SPI2/3/6/USART1/2/3/6/UART7/SDMMC1	SPI6/SAI2/4/UART4/5/8/LPUART/SDMMC1/SPDIFRX	SAI4/FDCAN1/2/TIM13/14/QUADSPI/FMC/SDMMC2/LCD/SPDIFRX	SAI2/4/TIM8/QUADSPI/SDMMC2/OTG1_HS/OTG2_FS/LCD	I2C4/UART7/SWPMI1/TIM1/8/DFSDM/SDMMC2/MDIOS/ETH	TIM1/8/FMC/SDMMC1/MDIOS/OTG1_FS/LCD	TIM1/DCMI/LCD/COMP	UART5/LCD	SYS
Port A	PA0	-	TIM2_CH1/TIM2_ETR	TIM5_CH1	TIM8_ETR	TIM15_BKIN	-	-	USART2_CTS_NSS	UART4_TX	SDMMC2_CMD	SAI2_SD_B	ETH_MII_CRS	-	-	-	EVENT-OUT
	PA1	-	TIM2_CH2	TIM5_CH2	LPTIM3_OUT	TIM15_CH1N	-	-	USART2_RTS	UART4_RX	QUADSPI_BK1_IO3	SAI2_MCK_B	ETH_MII_RX_CLK/ETH_RMII_REF_CLK	-	-	LCD_R2	EVENT-OUT
	PA2	-	TIM2_CH3	TIM5_CH3	LPTIM4_OUT	TIM15_CH1	-	-	USART2_TX	SAI2_SCK_B	-	-	ETH_MDIO	MDIOS_MDIO	-	LCD_R1	EVENT-OUT
	PA3	-	TIM2_CH4	TIM5_CH4	LPTIM5_OUT	TIM15_CH2	-	-	USART2_RX	-	LCD_B2	OTG_HS_ULPI_D0	ETH_MII_COL	-	-	LCD_B5	EVENT-OUT
	PA4	-	-	TIM5_ETR	-	-	SPI1_NSS/I2S1_WS	SPI3_NSS/I2S3_WS	USART2_CK	SPI6_NSS	-	-	-	OTG_HS_SOF	DCMI_HSYNC	LCD_VSYNC	EVENT-OUT
	PA5	-	TIM2_CH1/TIM2_ETR	-	TIM8_CH1N	-	SPI1_SCK/I2S1_CK	-	-	SPI6_SCK	-	OTG_HS_ULPI_CK	-	-	-	LCD_R4	EVENT-OUT
	PA6	-	TIM1_BKIN	TIM3_CH1	TIM8_BKIN	-	SPI1_MISO/I2S1_SDI	-	-	SPI6_MISO	TIM13_CH1	TIM8_BKIN_COMP12	MDIOS_MDC	TIM1_BKIN_COMP12	DCMI_PIXCLK	LCD_G2	EVENT-OUT
	PA7	-	TIM1_CH1N	TIM3_CH2	TIM8_CH1N	-	SPI1_MOSI/I2S1_SDO	-	-	SPI6_MOSI	TIM14_CH1	-	ETH_MII_RX_DV/ETH_RMII_CRS_DV	FMC_SDNWE	-	-	EVENT-OUT
	PA8	MCO1	TIM1_CH1	HRTIM_CHB2	TIM8_BKIN2	I2C3_SCL	-	-	USART1_CK	-	-	OTG_FS_SOF	UART7_RX	TIM8_BKIN2_COMP12	LCD_B3	LCD_R6	EVENT-OUT
	PA9	-	TIM1_CH2	HRTIM_CHC1	LPUART1_TX	I2C3_SMBA	SPI2_SCK/I2S2_CK	-	USART1_TX	-	FDCAN1_RXFD_MODE	-	ETH_TX_ER	-	DCMI_D0	LCD_R5	EVENT-OUT
	PA10	-	TIM1_CH3	HRTIM_CHC2	LPUART1_RX	-	-	-	USART1_RX	-	FDCAN1_TXFD_MODE	OTG_FS_ID	MDIOS_MDIO	LCD_B4	DCMI_D1	LCD_B1	EVENT-OUT

那么使用 GPIO_AF7_USART1 即可。

- 根据情况要清除 TC 发送完成标志和 RXNE 接收数据标志，因为这两个标志位在使能了串口后就已经置位，所以当用户使用了 TC 或者 RX 中断后，就会进入一次中断服务程序，这点要特别注意。
- HAL 库有个自己的底层初始化回调函数 HAL_UART_MspInit，是弱定义的，用户可以在其它的 C 文件里面实现，并将相对的底层初始化在里面实现。当用户调用 HAL_UART_Init 后，会在此函数里面调用 HAL_UART_MspInit，对应的底层复位函数 HAL_UART_MspDeInit 是在函数

HAL_UART_DeInit 里面被调用的。

当然，用户也可以自己初始化，不限制必须在两个函数里面实现。

◆ 上面举的例子里面没有用到 DMA，如果用到了 DMA，也是要初始化的。

29.3.4串口的状态标志清除问题

注，早前使用 F1 和 F4 时候，经常会有网友咨询为什么串口中断服务程序里面没有做清除标志。

下面我们介绍__HAL_USART_GET_FLAG 函数。这个函数用来检查 USART 标志位是否被设置。

```
/** @brief  Check whether the specified USART flag is set or not.
  * @param  __HANDLE__: specifies the USART Handle
  * @param  __FLAG__: specifies the flag to check.
  *        This parameter can be one of the following values:
  *           @arg USART_FLAG_TXFT: TXFIFO threshold flag
  *           @arg USART_FLAG_RXFT: RXFIFO threshold flag
  *           @arg USART_FLAG_RXFF:  RXFIFO Full flag
  *           @arg USART_FLAG_TXFE:  TXFIFO Empty flag
  *           @arg USART_FLAG_REACK: Receive enable ackowledge flag
  *           @arg USART_FLAG_TEACK: Transmit enable ackowledge flag
  *           @arg USART_FLAG_BUSY:  Busy flag
  *           @arg USART_FLAG_TXE:   Transmit data register empty flag
  *           @arg USART_FLAG_TC:    Transmission Complete flag
  *           @arg USART_FLAG_RXNE:  Receive data register not empty flag
  *           @arg USART_FLAG_IDLE:  Idle Line detection flag
  *           @arg USART_FLAG_ORE:   OverRun Error flag
  *           @arg USART_FLAG_UDR:   UnderRun Error flag
  *           @arg USART_FLAG_NE:    Noise Error flag
  *           @arg USART_FLAG_FE:    Framing Error flag
  *           @arg USART_FLAG_PE:    Parity Error flag
  * @retval The new state of __FLAG__ (TRUE or FALSE).
  */
#define __HAL_USART_GET_FLAG(__HANDLE__, __FLAG__) (((__HANDLE__)->Instance->ISR & (__FLAG__)) == (__FLAG__))
```

USART_FLAG 有如下几种取值：

USART_FLAG_CTS	CTS 标志 (CTS flag) 如果设置了CTSE位，当nCTS输入变化状态时，该位被硬件置高，由软件将其清零。如果USART_CR3中的CTSIE为‘1’，则产生中断。 0： nCTS状态线上没有变化； 1： nCTS状态线上发生变化。 注： 异步串口不支持这一位。
USART_FLAG_LBD	LBD: LIN断开检测标志 (LIN break detection flag) 当探测到LIN断开时，该位由硬件置‘1’，由软件清‘0’(将USART_ICR的LBDCF位置1)。如果USART_CR2中的LBDIE = 1，则产生中断。 0：没有检测到LIN断开； 1：检测到LIN断开。 注意：若LBDIE=1，当LBD为‘1’时要产生中断。

USART_FLAG_TXE **(必须了解的)**	TXE:发送数据寄存器空（Transmit data register empty) 当TDR寄存器中的数据被硬件转移到移位寄存器的时候，该位被硬件置位。如果USART_CR1寄存器中的TXEIE为1，则产生中断。对USART_TDR的写操作，将该位清零，也可以将USART_RQR的TXFRQ位置1实现清除标志。 0：数据还没有被转移到移位寄存器； 1：数据已经被转移到移位寄存器。 注意：单缓冲器传输中使用该位。
USART_FLAG_TC **(必须了解的)**	TC：发送完成（Transmission complete) 当包含有数据的一帧发送完成后，并且TXE=1时，由硬件将该位置‘1’。如果USART_CR1中的TCIE为‘1’，则产生中断。对USART_TDR的写操作可以清零该位，也可以将USART_ICR的TCCF位置1实现清除标志。 0：发送还未完成； 1：发送完成。
USART_FLAG_RXNE **(必须了解的)**	RXNE：接收数据寄存器非空（Read data register not empty) 当RDR移位寄存器中的数据被转移到USART_DR寄存器中，该位被硬件置位。如果USART_CR1寄存器中的RXNEIE为1，则产生中断。对USART_RDR的读操作可以将该位清零，也可以将USART_RQR的RXFRQ位置1实现清除标志。 0：数据没有收到； 1：收到数据，可以读出。
USART_FLAG_IDLE	IDLE：监测到总线空闲（IDLE line detected) 当检测到总线空闲时，该位被硬件置位。如果USART_CR1中的IDLEIE为‘1’，则产生中断。将USART_ICR的IDLECF位置1实现清除标志。 0：没有检测到空闲总线； 1：检测到空闲总线。 注意： IDLE位不会再次被置高直到RXNE位被置起(即又检测到一次空闲总线)
USART_FLAG_ORE	ORE：过载错误（Overrun error) 当RXNE仍然是‘1’的时候，当前被接收在移位寄存器中的数据，需要传送至RDR寄存器时，硬件将该位置位。如果USART_CR1中的RXNEIE为‘1’的话，则产生中断。将USART_ICR的ORECF位置1实现清除标志。 0：没有过载错误； 1：检测到过载错误。 注意：该位被置位时， RDR寄存器中的值不会丢失，但是移位寄存器中的数据会被覆盖。如果设置了EIE位，在多缓冲器通信模式下， ORE标志置位会

	产生中断的。
USART_FLAG_NE	NE: 噪声错误标志 (Noise error flag) 在接收到的帧检测到噪音时，由硬件对该位置位。将USART_ICR的NFCF位置1实现清除标志。 0：没有检测到噪声； 1：检测到噪声。 注意：该位不会产生中断，因为它和RXNE一起出现，硬件会在设置RXNE标志时产生中断。在多缓冲区通信模式下，如果设置了EIE位，则设置NE标志时会产生中断。
USART_FLAG_FE	FE: 帧错误 (Framing error) 当检测到同步错位，过多的噪声或者检测到断开符，该位被硬件置位。将USART_ICR的FECF位置1实现清除标志。 0：没有检测到帧错误； 1：检测到帧错误或者break符。
USART_FLAG_PE	PE: 校验错误 (Parity error) 在接收模式下，如果出现奇偶校验错误，硬件对该位置位。将USART_ICR的PECF位置1实现清除标志。如果USART_CR1中的PEIE为‘1’，则产生中断。 0：没有奇偶校验错误； 1：奇偶校验错误。

请大家重点关注上表中红字部分，USART 标志是需要软件主动清零的。清零有两种方式：一种是调用__HAL_USART_CLEAR_FLAG 函数，另一种是操作相关寄存器后自动清零。

```
/** @brief  Clear the specified USART pending flag.
  * @param  __HANDLE__: specifies the USART Handle.
  * @param  __FLAG__: specifies the flag to check.
  *          This parameter can be any combination of the following values:
  *            @arg USART_FLAG_TXFT:  TXFIFO threshold flag
  *            @arg USART_FLAG_RXFT:  RXFIFO threshold flag
  *            @arg USART_FLAG_RXFF:  RXFIFO Full flag
  *            @arg USART_FLAG_TXFE:  TXFIFO Empty flag
  *            @arg USART_FLAG_REACK: Receive enable ackowledge flag
  *            @arg USART_FLAG_TEACK: Transmit enable ackowledge flag
  *            @arg USART_FLAG_WUF:   Wake up from stop mode flag
  *            @arg USART_FLAG_RWU:   Receiver wake up flag (is the USART in mute mode)
  *            @arg USART_FLAG_SBKF:  Send Break flag
  *            @arg USART_FLAG_CMF:   Character match flag
  *            @arg USART_FLAG_BUSY:  Busy flag
  *            @arg USART_FLAG_ABRF:  Auto Baud rate detection flag
  *            @arg USART_FLAG_ABRE:  Auto Baud rate detection error flag
  *            @arg USART_FLAG_RTOF:  Receiver timeout flag
  *            @arg USART_FLAG_LBD:   LIN Break detection flag
  *            @arg USART_FLAG_TXE:   Transmit data register empty flag
  *            @arg USART_FLAG_TC:    Transmission Complete flag
```

```
  *           @arg USART_FLAG_RXNE:  Receive data register not empty flag
  *           @arg USART_FLAG_IDLE:  Idle Line detection flag
  *           @arg USART_FLAG_ORE:   OverRun Error flag
  *           @arg USART_FLAG_NE:    Noise Error flag
  *           @arg USART_FLAG_FE:    Framing Error flag
  *           @arg USART_FLAG_PE:    Parity Error flag
  * @retval The new state of __FLAG__ (TRUE or FALSE).
  */
#define __HAL_USART_CLEAR_FLAG(__HANDLE__, __FLAG__) ((__HANDLE__)->Instance->ICR = (__FLAG__))
```

上面介绍的 USART 标志大部分能够设置为产生中断，也就是有对应的 USART 中断标志。我们只介绍几个串口驱动要用到的中断标志：

USART_IT_TXE： TXE:发送数据寄存器空（此时数据可能正在发送）。

USART_IT_TC： 发送完成 。

USART_IT_RXNE： 接收数据寄存器非空。

中断缺省都是关闭的，通过__HAL_USART_ENABLE_IT 函数可以使能相应的中断标志。函数定义如下：

```
/** @brief  Enable the specified USART interrupt.
  * @param  __HANDLE__: specifies the USART Handle.
  * @param  __INTERRUPT__: specifies the USART interrupt source to enable.
  *          This parameter can be one of the following values:
  *            @arg USART_IT_RXFF: RXFIFO Full interrupt
  *            @arg USART_IT_TXFE: TXFIFO Empty interrupt
  *            @arg USART_IT_RXFT: RXFIFO threshold interrupt
  *            @arg USART_IT_TXFT: TXFIFO threshold interrupt
  *            @arg USART_IT_TXE : Transmit Data Register empty interrupt
  *            @arg USART_IT_TC  : Transmission complete interrupt
  *            @arg USART_IT_RXNE: Receive Data register not empty interrupt
  *            @arg USART_IT_IDLE: Idle line detection interrupt
  *            @arg USART_IT_PE  : Parity Error interrupt
  *            @arg USART_IT_ERR : Error interrupt(Frame error, noise error, overrun error)
  * @retval None
  */
#define __HAL_USART_ENABLE_IT(__HANDLE__, __INTERRUPT__)   (((((uint8_t)(__INTERRUPT__)) >> 5U) == 1)?
                       ((__HANDLE__)->Instance->CR1 |= (1U << ((__INTERRUPT__) & USART_IT_MASK))): \
                                                       ((((uint8_t)(__INTERRUPT__)) >> 5U) == 2)?
                        ((__HANDLE__)->Instance->CR2 |= (1U << ((__INTERRUPT__) & USART_IT_MASK))): \
                         ((__HANDLE__)->Instance->CR3 |= (1U << ((__INTERRUPT__) & USART_IT_MASK))))
```

STM32 一个串口的中断服务程序入口地址只有一个，进入中断服务程序后，我们需要判断是什么原因进入的中断，因此需要调用一个函数来检测中断标志。函数原型如下：

```
#define __HAL_USART_GET_IT(__HANDLE__, __IT__) ((__HANDLE__)->Instance->ISR & ((uint32_t)1 << ((__IT__)>>
0x08)))
```

中断处理完毕后，必须软件清除中断标志，否则中断返回后，会重入中断。清中断标志位的函数为：

```
#define __HAL_USART_CLEAR_IT(__HANDLE__, __IT_CLEAR__) ((__HANDLE__)->Instance->ICR =
(uint32_t)(__IT_CLEAR__))
```

正如前面介绍的，不是所有的标志都需要用这个函数清零。

注意：操作串口的寄存器不限制必须要用 HAL 库提供的 API，比如要操作寄存器 CR1，直接调用 USART1->CR1 操作即可。

29.3.5 串口初始化流程总结

使用方法由 HAL 库提供：

- 第 1 步：定义 UART_HandleTypeDef 类型串口结构体变量，比如 UART_HandleTypeDef huart。
- 第 2 步：使用函数 HAL_UART_MspInit 初始化串口底层，不限制一定要用此函数里面初始化，用户也可以自己实现。
  - 使能串口时钟。
  - 引脚配置。
    a、使能串口所使用的 GPIO 时钟。
    b、配置 GPIO 的复用模式。
  - 如果使用中断方式函数 HAL_UART_Transmit_IT 和 HAL_UART_Receive_IT 需要做如下配置。
    a、配置串口中断优先级。
    b、使能串口中断。
  - 串口中断的开关是通过函数__HAL_UART_ENABLE_IT() 和 __HAL_UART_DISABLE_IT()来实现，这两个函数被嵌套到串口的发送和接收函数中调用。
  - 如果使用 DMA 方式函数 HAL_UART_Transmit_DMA 和 HAL_UART_Receive_DMA 需要做如下配置。
    a、声明串口的发送和接收 DMA 结构体变量，注意发送和接收是独立的，如果都使用，那就都需要配置。
    b、使能 DMA 接口时钟。
    c、配置串口的发送和接收 DMA 结构体变量。
    d、配置 DMA 发送和接收通道。
    e、关联 DMA 和串口的句柄。
    f、配置发送 DMA 和接收 DMA 的传输完成中断和中断优先级。
- 第 3 步：配置串口的波特率，位长，停止位，奇偶校验位，流控制和发送接收模式。
- 第 4 步：如果需要，可以编程高级特性，比如 TX/RX 交换引脚，自动波特率检测。通过第 1 步串口结构体变量 huart 的结构体成员 AdvancedInit 来设置。
- 第 5 步：串口初始化调用的函数 HAL_UART_Init 初始化。

29.4 源文件 stm32h7xx_hal_uart.c

此文件涉及到的函数较多，这里把几个常用的函数做个说明：

- HAL_UART_Init
- HAL_UART_Transmit
- HAL_UART_Receive

- HAL_UART_Transmit_IT
- HAL_UART_Receive_IT
- HAL_UART_Transmit_DMA
- HAL_UART_Receive_DMA

其实 V7 开发板设计的 8 个串口 FIFO 驱动文件 bsp_uart_fifo.c 仅用到了函数 HAL_UART_Init，其它函数都没有用到，不过这里也为大家做个说明。

29.4.1 函数 HAL_UART_Init

函数原型：

```
HAL_StatusTypeDef HAL_UART_Init(UART_HandleTypeDef *huart)
{

   /* 省略 */

  if(huart->gState == HAL_UART_STATE_RESET)
  {
    huart->Lock = HAL_UNLOCKED;

    /* 初始化硬件: GPIO, CLOCK */
    HAL_UART_MspInit(huart);
  }

  huart->gState = HAL_UART_STATE_BUSY;

  /* 禁止串口 */
  __HAL_UART_DISABLE(huart);

  /* 配置串口参数 */
  if (UART_SetConfig(huart) == HAL_ERROR)
  {
    return HAL_ERROR;
  }

   /* 配置串口高级特性 */
  if (huart->AdvancedInit.AdvFeatureInit != UART_ADVFEATURE_NO_INIT)
  {
    UART_AdvFeatureConfig(huart);
  }

  /* 清寄存器的一些标志位 */
  CLEAR_BIT(huart->Instance->CR2, (USART_CR2_LINEN | USART_CR2_CLKEN));
  CLEAR_BIT(huart->Instance->CR3, (USART_CR3_SCEN | USART_CR3_HDSEL | USART_CR3_IREN));

  /* 使能串口 */
  __HAL_UART_ENABLE(huart);

  return (UART_CheckIdleState(huart));
}
```

函数描述：

此函数用于初始化串口的基础特性和高级特性。

函数参数：

- 第 1 个参数是 UART_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

注意事项：

1. 函数 HAL_UART_MspInit 用于初始化 USART 的底层时钟、引脚等功能。需要用户自己在此函数里面实现具体的功能。由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
2. 如果形参 huart 的结构体成员 gState 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 UART_HandleTypeDef UartHandle。

   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_UART_STATE_RESET = 0x00U。

   解决办法有三

   方法 1：用户自己初始串口和涉及到的 GPIO 等。

   方法 2：定义 UART_HandleTypeDef UartHandle 为全局变量。

   方法 3：下面的方法

```
if(HAL_UART_DeInit(&UartHandle) != HAL_OK)
{
    Error_Handler();
}
if(HAL_UART_Init(&UartHandle) != HAL_OK)
{
    Error_Handler();
}
```

3. 注意串口的中断状态寄存器 USART_ISR 复位后，TC 发送完成状态和 RXNE 接收状态都被置 1，如果用户使能这两个中断前，最好优先清除中断标志。

使用举例：

```
UART_HandleTypeDef UartHandle;

/* USART3 工作在 UART 模式 */
/* 配置如下:
  - 数据位 = 8 Bits
  - 停止位 = 1 bit
  - 奇偶校验位 = 无
  - 波特率 = 115200bsp
  - 硬件流控制 (RTS 和 CTS 信号) */
UartHandle.Instance        = USART3;

UartHandle.Init.BaudRate     = 115200;
UartHandle.Init.WordLength   = UART_WORDLENGTH_8B;
UartHandle.Init.StopBits     = UART_STOPBITS_1;
UartHandle.Init.Parity       = UART_PARITY_NONE;
```

```
UartHandle.Init.HwFlowCtl    = UART_HWCONTROL_NONE;
UartHandle.Init.Mode         = UART_MODE_TX_RX;
UartHandle.Init.OverSampling = UART_OVERSAMPLING_16;
UartHandle.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;

if(HAL_UART_Init(&UartHandle) != HAL_OK)
{
    Error_Handler();
}
```

29.4.2 函数 HAL_UART_Transmit

函数原型：

```
HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout)
{
  /* 省略 */

  if(huart->gState == HAL_UART_STATE_READY)
  {
    /* 省略 */

    while(huart->TxXferCount > 0U)
    {
      huart->TxXferCount--;
      /* 等待发送空中断标志 */
      if(UART_WaitOnFlagUntilTimeout(huart, UART_FLAG_TXE, RESET, tickstart, Timeout) != HAL_OK)
      {
        return HAL_TIMEOUT;
      }
    }
    /* 等待发送完成中断 */
    if(UART_WaitOnFlagUntilTimeout(huart, UART_FLAG_TC, RESET, tickstart, Timeout) != HAL_OK)
    {
      return HAL_TIMEOUT;
    }

    /* 省略 */
    return HAL_OK;
  }
  else
  {
    return HAL_BUSY;
  }
}
```

函数描述：

此函数以查询的方式发送指定字节。看源码的话，程序里面最重要的就是上面代码中置红的两个标志，发送空标志和发送完成标志。发送空标志表示发送数据寄存器为空，数据还在移位寄存器里面，而发送完成标志表示数据已经从移位寄存器发送出去。

函数参数：

- 第 1 个参数是 UART_HandleTypeDef 类型结构体指针变量。

- 第 2 个参数是要发送的数据地址。
- 第 3 个参数是要发送的数据大小，单位字节。
- 第 4 个参数是溢出时间，单位 ms。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: fputc
*    功能说明: 重定义 putc 函数，这样可以使用 printf 函数从串口 1 打印输出
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
int fputc(int ch, FILE *f)
{
    HAL_UART_Transmit(&UartHandle, (uint8_t *)&ch, 1, HAL_MAX_DELAY);

    return ch;
}
```

29.4.3 函数 HAL_UART_Receive

函数原型：

```
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout)
{

  /* 省略 */
  if(huart->RxState == HAL_UART_STATE_READY)
  {

    /* 省略 */
    while(huart->RxXferCount > 0U)
    {
      huart->RxXferCount--;
      if(UART_WaitOnFlagUntilTimeout(huart, UART_FLAG_RXNE, RESET, tickstart, Timeout) != HAL_OK)
      {
        return HAL_TIMEOUT;
      }


    }
    /* 省略 */
    return HAL_OK;
  }
  else
  {
    /* 省略 */
    return HAL_BUSY;
  }
}
```

函数描述：

此函数以查询的方式接收指定字节。这个函数相对比较好理解，就是等待上面程序中的 RXNE 标志，置位了表示接收数据寄存器已经存入数据。

函数参数：

- 第 1 个参数是 UART_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是要接收的数据地址。
- 第 3 个参数是要接收的数据大小，单位字节。
- 第 4 个参数是溢出时间，单位 ms。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: fgetc
*    功能说明: 重定义 getc 函数，这样可以使用 scanff 函数从串口 1 输入数据
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
int fgetc(FILE *f)
{
    int ret;

    HAL_UART_Receive(&UartHandle, (uint8_t *)&ret, 1, HAL_MAX_DELAY);

    return ret;
}
```

29.4.4函数 HAL_UART_Transmit_IT

函数原型：

```
HAL_StatusTypeDef HAL_UART_Transmit_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)
{

  if(huart->gState == HAL_UART_STATE_READY)
  {
    if((pData == NULL ) || (Size == 0U))
    {
      return HAL_ERROR;
    }

    __HAL_LOCK(huart);

    huart->pTxBuffPtr = pData;
    huart->TxXferSize = Size;
    huart->TxXferCount = Size;

    huart->ErrorCode = HAL_UART_ERROR_NONE;
```

```
    huart->gState = HAL_UART_STATE_BUSY_TX;


    __HAL_UNLOCK(huart);

    if (READ_BIT(huart->Instance->CR1, USART_CR1_FIFOEN) != RESET)
    {
      /* 使能 FIFO 发送中断 */
      SET_BIT(huart->Instance->CR3, USART_CR3_TXFTIE);
    }
    else
    {
      /* 使能发空中断 */
      SET_BIT(huart->Instance->CR1, USART_CR1_TXEIE);
    }

    return HAL_OK;
  }
  else
  {
    return HAL_BUSY;
  }
}
```

函数描述：

此函数以中断的方式发送指定字节，可以选择使能 FIFO 中断方式或者发送空中断方式。具体数据的发送是在中断处理函数 HAL_UART_IRQHandler 里面实现。

函数参数：

- 第 1 个参数是 UART_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是要发送的数据地址。
- 第 3 个参数是要发送的数据大小，单位字节。
- 返回值，返回 HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

使用中断方式要使能串口中断，此贴有完整例子：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=86245 。

```
UART_HandleTypeDef UartHandle;
uint8_t s_ucBuf[5];

/* 数据发送 */
HAL_UART_Transmit_IT(&UartHandle, s_ucBuf, 1);
HAL_UART_Transmit_IT(&UartHandle, (uint8_t*)"KEY_DOWN_K1\r\n", 13);
```

29.4.5 函数 HAL_UART_Receive_IT

函数原型：

```
HAL_StatusTypeDef HAL_UART_Receive_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)
{
```

```
  if(huart->RxState == HAL_UART_STATE_READY)
  {
    if((pData == NULL ) || (Size == 0U))
    {
      return HAL_ERROR;
    }

    __HAL_LOCK(huart);

    huart->pRxBuffPtr = pData;
    huart->RxXferSize = Size;
    huart->RxXferCount = Size;

    UART_MASK_COMPUTATION(huart);

    huart->ErrorCode = HAL_UART_ERROR_NONE;
    huart->RxState = HAL_UART_STATE_BUSY_RX;

    __HAL_UNLOCK(huart);

    /* 使能错误中断: (Frame error, noise error, overrun error) */
    SET_BIT(huart->Instance->CR3, USART_CR3_EIE);

    if (READ_BIT(huart->Instance->CR1, USART_CR1_FIFOEN) != RESET)
    {
      /* 使能奇偶校验失败中断 */
      SET_BIT(huart->Instance->CR1, USART_CR1_PEIE);
      /* 使能 FIFO 接收中断 */
      SET_BIT(huart->Instance->CR3, USART_CR3_RXFTIE);
    }
    else
    {
      /* 使能奇偶校验失败中断和接收中断 */
      SET_BIT(huart->Instance->CR1, USART_CR1_PEIE | USART_CR1_RXNEIE);
    }

    return HAL_OK;
  }
  else
  {
    return HAL_BUSY;
  }
}
```

函数描述:

此函数以中断的方式接收指定字节，可以选择使能 FIFO 中断方式或者普通中断方式，两种方式使能了奇偶校验中断失败和错误中断。具体数据的接收是在中断处理函数 HAL_UART_IRQHandler 里面实现。

函数参数:

- 第 1 个参数是 UART_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是要接收的数据地址。
- 第 3 个参数是要接收的数据大小，单位字节。

- 返回值，返回 HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

使用中断方式要使能串口中断，此贴有完整例子：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=86245 。

```
UART_HandleTypeDef UartHandle;
uint8_t s_ucBuf[5];

/* 数据接收*/
HAL_UART_Receive_IT(&UartHandle, s_ucBuf, 1);
```

29.4.6函数 HAL_UART_Transmit_DMA

函数原型：

```
HAL_StatusTypeDef HAL_UART_Transmit_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)
{
  if(huart->gState == HAL_UART_STATE_READY)
  {
    if((pData == NULL ) || (Size == 0U))
    {
      return HAL_ERROR;
    }

    __HAL_LOCK(huart);

    huart->pTxBuffPtr = pData;
    huart->TxXferSize = Size;
    huart->TxXferCount = Size;

    huart->ErrorCode = HAL_UART_ERROR_NONE;
    huart->gState = HAL_UART_STATE_BUSY_TX;

    /* 注册各种 DMA 回调函数 */
    huart->hdmatx->XferCpltCallback = UART_DMATransmitCplt;
    huart->hdmatx->XferHalfCpltCallback = UART_DMATxHalfCplt;
    huart->hdmatx->XferErrorCallback = UART_DMAError;
    huart->hdmatx->XferAbortCallback = NULL;

    /* 使能串口发送 DMA 通道 */
    HAL_DMA_Start_IT(huart->hdmatx, (uint32_t)huart->pTxBuffPtr, (uint32_t)&huart->Instance->TDR, Size);

    /* 清除传输 TC 完成标志 */
    __HAL_UART_CLEAR_FLAG(huart, UART_CLEAR_TCF);

    __HAL_UNLOCK(huart);

    /* 使能串口发送 DMA 传输 */
    SET_BIT(huart->Instance->CR3, USART_CR3_DMAT);

    return HAL_OK;
  }
  else
```

```
  {
    return HAL_BUSY;
  }
}
```

函数描述：

此函数以 DMA 的方式发送指定字节。这里是用的 DMA 中断方式 HAL_DMA_Start_IT 进行的发送。所以使用此函数的话，不要忘了写 DMA 中断服务程序。而且 DMA 的配置也是需要用户实现的，可以直接在函数 HAL_UART_MspInit 里面实现，也可以放在其它位置。

函数参数：

- 第 1 个参数是 UART_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是要发送的数据地址。
- 第 3 个参数是要发送的数据大小，单位字节。
- 返回值，返回 HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

使用 DMA 方式，此贴有完整例子：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=86271 。

29.4.7 函数 HAL_UART_Receive_DMA

函数原型：

```
HAL_StatusTypeDef HAL_UART_Receive_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)
{
  if(huart->RxState == HAL_UART_STATE_READY)
  {
    if((pData == NULL ) || (Size == 0U))
    {
      return HAL_ERROR;
    }

    __HAL_LOCK(huart);

    huart->pRxBuffPtr = pData;
    huart->RxXferSize = Size;

    huart->ErrorCode = HAL_UART_ERROR_NONE;
    huart->RxState = HAL_UART_STATE_BUSY_RX;

    /* 注册各种 DMA 回调函数 */
    huart->hdmarx->XferCpltCallback = UART_DMAReceiveCplt;

    huart->hdmarx->XferHalfCpltCallback = UART_DMARxHalfCplt;
    huart->hdmarx->XferErrorCallback = UART_DMAError;
    huart->hdmarx->XferAbortCallback = NULL;

    /* 使能串口接收 DMA 通道 */
```

```
    HAL_DMA_Start_IT(huart->hdmarx, (uint32_t)&huart->Instance->RDR, (uint32_t)huart->pRxBuffPtr, Size);

    __HAL_UNLOCK(huart);

    /* 使能串口校验错误中断 */
    SET_BIT(huart->Instance->CR1, USART_CR1_PEIE);

    /* 使能串口错误中断:(Frame error, noise error, overrun error) */
    SET_BIT(huart->Instance->CR3, USART_CR3_EIE);

    /* 使能串口接收 DMA 传输 */
    SET_BIT(huart->Instance->CR3, USART_CR3_DMAR);

    return HAL_OK;
  }
  else
  {
    return HAL_BUSY;
  }
}
```

函数描述:

此函数以 DMA 的方式接收指定字节。这里是用的 DMA 中断方式 HAL_DMA_Start_IT 进行的接收。所以使用此函数的话，不要忘了写 DMA 中断服务程序。而且 DMA 的配置也是需要用户实现的，可以直接在函数 HAL_UART_MspInit 里面实现，也可以放在其它位置。

函数参数:

- 第 1 个参数是 UART_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是要接收的数据地址。
- 第 3 个参数是要接收的数据大小，单位字节。
- 返回值，返回 HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例:

使用 DMA 方式，此贴有完整例子:

http://www.armbbs.cn/forum.php?mod=viewthread&tid=86271 。

29.5 总结

本章节就为大家讲解这么多，涉及到的知识点和 API 函数比较多，需要花点时间消化，后面用到的多了，就可以熟练掌握了。

第30章 STM32H7 的 USART 应用之八个串口 FIFO 实现

本章节为大家讲解 STM327 的 8 个串口的 FIFO 驱动实现，后面的 ESP8266，GPS，RS485，GPRS 等试验都是建立在这个驱动的基础上实现。

除了串口 FIFO 的驱动实现，RS232 通信也通过本章节做个讲解。

30.1 初学者重要提示
30.2 硬件设计
30.3 串口驱动设计
30.4 串口 FIFO 板级支持包（bsp_uart_fifo.c）
30.5 串口 FIFO 驱动移植和使用
30.6 实验例程设计框架
30.7 实验例程说明（MDK）
30.8 实验例程说明（IAR）
30.9 总结

30.1 初学者重要提示

- 学习本章节前，务必优先学习第 29 章。
- 串口 FIFO 的实现跟前面章节按键 FIFO 的机制是一样的。
- 本章节比较重要，因为后面的 ESP8266，GPS，RS485，GPRS 等试验都是建立在这个驱动的基础上实现。
- 大家自己做的板子，测试串口收发是乱码的话，重点看 stm32h7xx_hal_conf.h 文件中的 HSE_VALUE 的大小跟板子上实际晶振大小是否一致，然后再看 PLL 配置。
- CH340/CH341 的 USB 转串口 Windows 驱动程序的安装包，支持 32/64 位 Windows 10/8.1/8/7。http://www.armbbs.cn/forum.php?mod=viewthread&tid=32826 。

30.2 硬件设计

STM32H743XIH6 最多可以支持 8 个独立的串口。其中串口 4 和串口 5 和 SDIO 的 GPIO 是共用的，也就是说，如果要用到 SD 卡，那么串口 4 和串口 5 将不能使用。串口 7 和 SPI3 共用，串口 8 和 RGB 硬

件接口共用。串口功能可以分配到不同的 GPIO。我们常用的引脚分配如下：

串口 USART1　TX = PA9，　RX = PA10

串口 USART2　TX = PA2，　RX = PA3

串口 USART3　TX = PB10，　RX = PB11

串口 UART4　TX = PC10，　RX = PC11　(和 SDIO 共用)

串口 UART5　TX = PC12，　RX = PD2　(和 SDIO 共用)

串口 USART6　TX = PG14，　RX = PC7

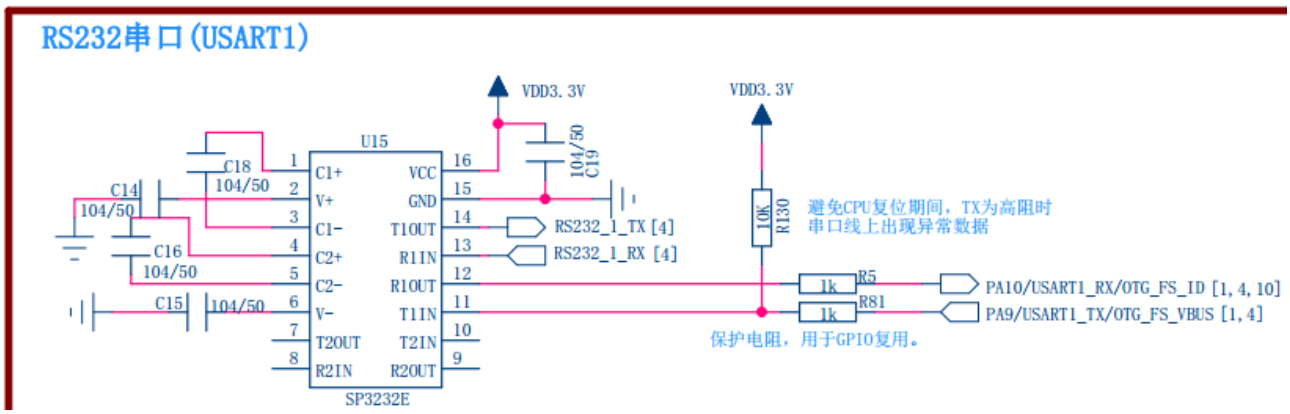
串口 UART7　TX = PB4，　RX = PB3　(和 SPI1/3 共用)

串口 UART8　TX = PJ8，　RX =PJ9　(和 RGB 硬件接口共用)

STM32-V7 开发板使用了 4 个串口设备。

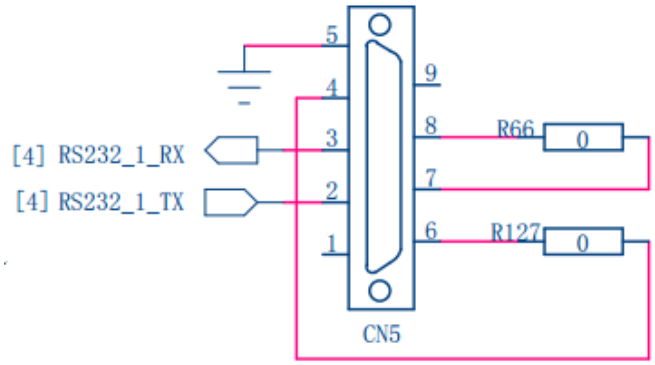
- 串口 1 用于 RS232 接口，很多例子的 pritnf 结果就是输出到串口 1
- 串口 2 用于 GPS
- 串口 3 用于 RS485 接口
- 串口 6 用于 TTL 串口插座，板子上有 GPRS 插座和串口 WIFI 插座。

下面是 RS232 的原理图：



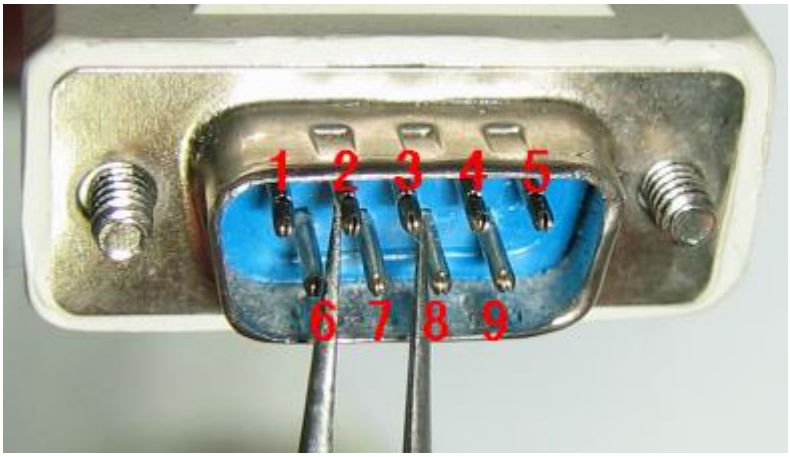
关于 232 的 PHY 芯片 SP3232E 要注意以下几个问题：

- SP3232E 的作用是 TTL 电平转 RS232 电平。
- 电阻 R130 的作用是避免 CPU 复位期间，TX 为高阻时串口线上出现异常数据。
- 检测 SP3232E 的好坏可以采用回环的方式，即短接 T1OUT 和 R1IN，对应到 DB9 插座上就是短接引脚 2 和引脚 3。



DB9 RS232插座 9孔

实际效果如下：



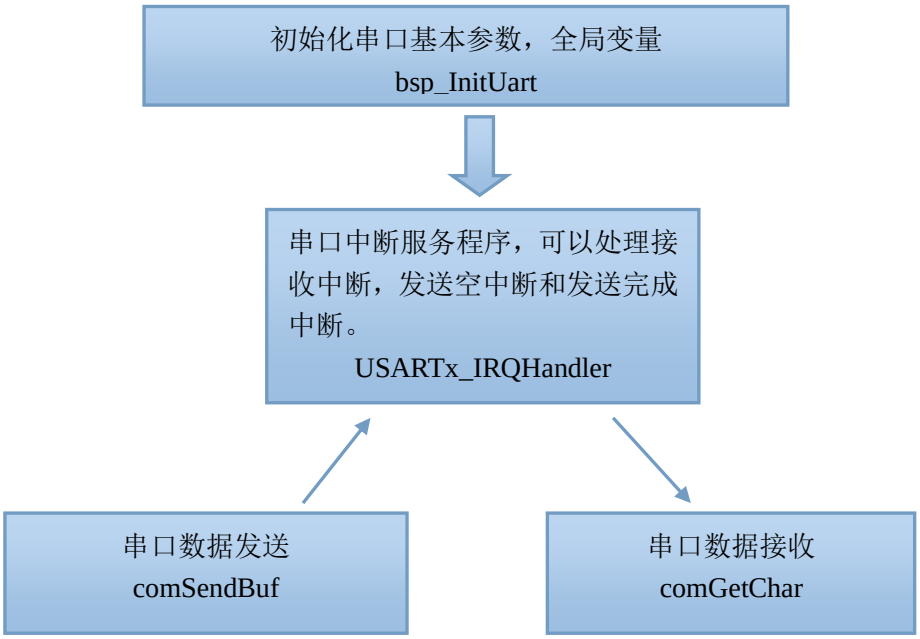
通过这种方式，可以在应用程序中通过串口发送几个字符，查看是否可以正确接收来判断 232 PHY 芯片是否有问题。

- 由于这里是 TTL 转 RS232，如果电脑端自带 DB9 串口，可以找根交叉线直接接上。如果电脑端没有，就需要用 RS232 转 USB 的串口线。这里要注意是 RS232 转 USB，不是 TTL 转 USB。像我们用的 CH340 就是 RS232 转 USB 芯片。
- 检测串口线的好坏跟板子上的 232 PHY 一样，将电脑端的串口助手打开，串口线接到电脑端并短接串口线的 2 脚和 3 脚，然后使用串口助手进行自收发测试即可。

30.3 串口 FIFO 驱动设计

30.3.1 串口 FIFO 框架

为了方便大家理解，先来看下串口 FIFO 的实现框图：



- 第 1 阶段，初始化：

  通过函数 bsp_InitUart 初始化串口结构体，串口硬件参数。

- 第 2 阶段，串口中断服务程序：
  - 接收中断是一直开启的。
  - 做了发送空中断和发送完成中断的消息处理。
- 第 3 阶段，串口数据的收发：
  - 串口发送函数会开启发送空中断。
  - 串口接收中断接收到函数后，可以使用函数 comGetChar 获取数据。

30.3.2 串口 FIFO 之相关的变量定义

串口驱动的核心文件为：bsp_uart_fifo.c, bsp_uart_fifo.h。

这里面包括有串口硬件的配置函数、中断处理函数，以及串口的读写接口函数。还有 printf 函数的实现。

每个串口都有 2 个 FIFO 缓冲区，一个是用于发送数据的 TX_FIFO，一个用于保存接收数据的 RX_FIFO。

我们来看下这个 FIFO 的定义，在 bsp_uart_fifo.h 文件。

```
/* 定义串口波特率和 FIFO 缓冲区大小，分为发送缓冲区和接收缓冲区，支持全双工 */
#if UART1_FIFO_EN == 1
    #define UART1_BAUD          115200
    #define UART1_TX_BUF_SIZE   1*1024
    #define UART1_RX_BUF_SIZE   1*1024
#endif

/* 串口设备结构体 */
typedef struct
```

```
{
    USART_TypeDef *uart;        /* STM32 内部串口设备指针 */
    uint8_t *pTxBuf;            /* 发送缓冲区 */
    uint8_t *pRxBuf;            /* 接收缓冲区 */
    uint16_t usTxBufSize;       /* 发送缓冲区大小 */
    uint16_t usRxBufSize;       /* 接收缓冲区大小 */
    __IO uint16_t usTxWrite;    /* 发送缓冲区写指针 */
    __IO uint16_t usTxRead;     /* 发送缓冲区读指针 */
    __IO uint16_t usTxCount;    /* 等待发送的数据个数 */

    __IO uint16_t usRxWrite;    /* 接收缓冲区写指针 */
    __IO uint16_t usRxRead;     /* 接收缓冲区读指针 */
    __IO uint16_t usRxCount;    /* 还未读取的新数据个数 */

    void (*SendBefor)(void);    /* 开始发送之前的回调函数指针（主要用于 RS485 切换到发送模式） */
    void (*SendOver)(void);     /* 发送完毕的回调函数指针（主要用于 RS485 将发送模式切换为接收模式） */
    void (*ReciveNew)(uint8_t _byte);    /* 串口收到数据的回调函数指针 */
    uint8_t Sending;            /* 正在发送中 */
}UART_T;
```

bsp_uart_fifo.c 文件定义变量。我们以串口 1 为例，其他的串口都是一样的代码。

```
/* 定义每个串口结构体变量 */
#if UART1_FIFO_EN == 1
    static UART_T g_tUart1;
    static uint8_t g_TxBuf1[UART1_TX_BUF_SIZE];        /* 发送缓冲区 */
    static uint8_t g_RxBuf1[UART1_RX_BUF_SIZE];        /* 接收缓冲区 */
#endif
```

关于 FIFO 的机制，我们在按键 FIFO 驱动已经做过详细的介绍，这个地方就不赘述了。每个串口有两个 FIFO 缓冲区，每个 FIFO 对应一个写指针和一个读指针。这个结构中还有三个回调函数。回调函数就是一个通过函数指针调用的函数。如果你把函数的指针（地址）作为参数传递给另一个函数，当这个指针被用为调用它所指向的函数时，我们就说这是回调函数。

30.3.3 串口 FIFO 初始化

串口的初始化代码如下；

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitUart
*    功能说明: 初始化串口硬件，并对全局变量赋初值.
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitUart(void)
{

    UartVarInit();     /* 必须先初始化全局变量,再配置硬件 */

    InitHardUart();    /* 配置串口的硬件参数(波特率等) */

    RS485_InitTXE();   /* 配置 RS485 芯片的发送使能硬件，配置为推挽输出 */
}
```

下面将初始化代码实现的功能依次为大家做个说明。

◆ 函数 UartVarInit

这个函数实现的功能比较好理解，主要是串口设备结构体变量的初始化，代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: UartVarInit
*    功能说明: 初始化串口相关的变量
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void UartVarInit(void)
{
#if UART1_FIFO_EN == 1
    g_tUart1.uart = USART1;                     /* STM32 串口设备 */
    g_tUart1.pTxBuf = g_TxBuf1;                 /* 发送缓冲区指针 */
    g_tUart1.pRxBuf = g_RxBuf1;                 /* 接收缓冲区指针 */
    g_tUart1.usTxBufSize = UART1_TX_BUF_SIZE;   /* 发送缓冲区大小 */
    g_tUart1.usRxBufSize = UART1_RX_BUF_SIZE;   /* 接收缓冲区大小 */
    g_tUart1.usTxWrite = 0;                     /* 发送 FIFO 写索引 */
    g_tUart1.usTxRead = 0;                      /* 发送 FIFO 读索引 */
    g_tUart1.usRxWrite = 0;                     /* 接收 FIFO 写索引 */
    g_tUart1.usRxRead = 0;                      /* 接收 FIFO 读索引 */
    g_tUart1.usRxCount = 0;                     /* 接收到的新数据个数 */
    g_tUart1.usTxCount = 0;                     /* 待发送的数据个数 */
    g_tUart1.SendBefor = 0;                     /* 发送数据前的回调函数 */
    g_tUart1.SendOver = 0;                      /* 发送完毕后的回调函数 */
    g_tUart1.ReciveNew = 0;                     /* 接收到新数据后的回调函数 */
    g_tUart1.Sending = 0;                       /* 正在发送中标志 */
#endif
    /* 串口 2-8 的初始化省略未写 */
}
```

◆ 函数 InitHardUart

此函数主要用于串口的 GPIO，中断和相关参数的配置。

```
1.  /* 串口 1 的 GPIO  PA9, PA10   RS323 DB9 接口 */
2.  #define USART1_CLK_ENABLE()              __HAL_RCC_USART1_CLK_ENABLE()
3.
4.  #define USART1_TX_GPIO_CLK_ENABLE()      __HAL_RCC_GPIOA_CLK_ENABLE()
5.  #define USART1_TX_GPIO_PORT              GPIOA
6.  #define USART1_TX_PIN                    GPIO_PIN_9
7.  #define USART1_TX_AF                     GPIO_AF7_USART1
8.
9.  #define USART1_RX_GPIO_CLK_ENABLE()      __HAL_RCC_GPIOA_CLK_ENABLE()
10. #define USART1_RX_GPIO_PORT              GPIOA
11. #define USART1_RX_PIN                    GPIO_PIN_10
12. #define USART1_RX_AF                     GPIO_AF7_USART1
13.
14. /* 串口 2-8 的引脚和时钟宏定义未写 */
15.
16. /*
17. *********************************************************************************************************
18. *    函 数 名: InitHardUart
19. *    功能说明: 配置串口的硬件参数（波特率，数据位，停止位，起始位，校验位，中断使能）适合于 STM32-H7 开
20. *              发板
```

```
21. *    形    参: 无
22. *    返 回 值: 无
23. *********************************************************************************************************
24. */
25. static void InitHardUart(void)
26. {
27.     GPIO_InitTypeDef  GPIO_InitStruct;
28.     RCC_PeriphCLKInitTypeDef RCC_PeriphClkInit;
29.
30.     /*
31.        下面这个配置可以注释掉，预留下来是为了方便以后选择其它时钟使用
32.        默认情况下，USART1 和 USART6 选择的 PCLK2，时钟 100MHz。
33.        USART2，USART3，UART4，UART5，UART6，UART7 和 UART8 选择的时钟是 PLCK1，时钟 100MHz。
34.     */
35.     RCC_PeriphClkInit.PeriphClockSelection = RCC_PERIPHCLK_USART16;
36.     RCC_PeriphClkInit.Usart16ClockSelection = RCC_USART16CLKSOURCE_D2PCLK2;
37.     HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphClkInit);
38.
39. #if UART1_FIFO_EN == 1       /* 串口 1 */
40.     /* 使能 GPIO TX/RX 时钟 */
41.     USART1_TX_GPIO_CLK_ENABLE();
42.     USART1_RX_GPIO_CLK_ENABLE();
43.
44.     /* 使能 USARTx 时钟 */
45.     USART1_CLK_ENABLE();
46.
47.     /* 配置 TX 引脚 */
48.     GPIO_InitStruct.Pin       = USART1_TX_PIN;
49.     GPIO_InitStruct.Mode      = GPIO_MODE_AF_PP;
50.     GPIO_InitStruct.Pull      = GPIO_PULLUP;
51.     GPIO_InitStruct.Speed     = GPIO_SPEED_FREQ_VERY_HIGH;
52.     GPIO_InitStruct.Alternate = USART1_TX_AF;
53.     HAL_GPIO_Init(USART1_TX_GPIO_PORT, &GPIO_InitStruct);
54.
55.     /* 配置 RX 引脚 */
56.     GPIO_InitStruct.Pin = USART1_RX_PIN;
57.     GPIO_InitStruct.Alternate = USART1_RX_AF;
58.     HAL_GPIO_Init(USART1_RX_GPIO_PORT, &GPIO_InitStruct);
59.
60.     /* 配置 NVIC the NVIC for UART */
61.     HAL_NVIC_SetPriority(USART1_IRQn, 0, 1);
62.     HAL_NVIC_EnableIRQ(USART1_IRQn);
63.
64.     /* 配置波特率、奇偶校验 */
65.     bsp_SetUartParam(USART1,  UART1_BAUD, UART_PARITY_NONE, UART_MODE_TX_RX);
66.
67.     SET_BIT(USART1->ICR, USART_ICR_TCCF);   /* 清除 TC 发送完成标志 */
68.     SET_BIT(USART1->RQR, USART_RQR_RXFRQ);  /* 清除 RXNE 接收标志 */
69.     // USART_CR1_PEIE | USART_CR1_RXNEIE
70.     SET_BIT(USART1->CR1, USART_CR1_RXNEIE); /* 使能 PE. RX 接受中断 */
71. #endif
72. /* 串口 2-8 的初始化省略未写 */
73. }
```

- 第 2-12 行，以宏定义的方式设置串口 1-8 的 GPIO 时钟、引脚和串口时钟，方便修改。
- 第 35-37 行,这里的配置可以注释掉,预留下来仅仅是为了方便以后选择其它时钟使用。默认情况下，USART1 和 USART6 选择的 PCLK2，时钟 100MHz。

USART2,USART3,UART4,UART5,UART6,UART7 和 UART8 选择的时钟是 PLCK1,时钟 100MHz。

- 第 61-62 行，配置串口中断优先级并使能串口中断，用户可以根据实际工程修改优先级大小。
- 第 65 行，配置串口的基本参数，具体配置在函数里面有注释。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_SetUartParam
*    功能说明: 配置串口的硬件参数（波特率，数据位，停止位，起始位，校验位，中断使能）适合于 STM32- H7 开发板
*    形    参: Instance   USART_TypeDef 类型结构体
*              BaudRate   波特率
*              Parity     校验类型，奇校验或者偶校验
*              Mode       发送和接收模式使能
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_SetUartParam(USART_TypeDef *Instance,  uint32_t BaudRate, uint32_t Parity, uint32_t Mode)
{
    UART_HandleTypeDef UartHandle;

    /*##-1- 配置串口硬件参数 ######################################*/
    /* 异步串口模式 (UART Mode) */
    /* 配置如下:
      - 字长    = 8 位
      - 停止位  = 1 个停止位
      - 校验    = 参数 Parity
      - 波特率  = 参数 BaudRate
      - 硬件流控制关闭 (RTS and CTS signals) */

    UartHandle.Instance        = Instance;

    UartHandle.Init.BaudRate   = BaudRate;
    UartHandle.Init.WordLength = UART_WORDLENGTH_8B;
    UartHandle.Init.StopBits   = UART_STOPBITS_1;
    UartHandle.Init.Parity     = Parity;
    UartHandle.Init.HwFlowCtl  = UART_HWCONTROL_NONE;
    UartHandle.Init.Mode       = Mode;
    UartHandle.Init.OverSampling = UART_OVERSAMPLING_16;
    UartHandle.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
    UartHandle.Init.Prescaler = UART_PRESCALER_DIV1;
    UartHandle.Init.FIFOMode = UART_FIFOMODE_DISABLE;
    UartHandle.Init.TXFIFOThreshold = UART_TXFIFO_THRESHOLD_1_8;
    UartHandle.Init.RXFIFOThreshold = UART_RXFIFO_THRESHOLD_1_8;
    UartHandle.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;

    if (HAL_UART_Init(&UartHandle) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

- 函数 RS485_InitTXE

此函数主要用于 485 PHY 芯片的发送使能，直接配置引脚为推挽输出模式即可使用。具体代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: RS485_InitTXE
*    功能说明: 配置 RS485 发送使能口线 TXE
```

```
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void RS485_InitTXE(void)
{
    GPIO_InitTypeDef gpio_init;

    /* 打开 GPIO 时钟 */
    RS485_TXEN_GPIO_CLK_ENABLE();

    /* 配置引脚为推挽输出 */
    gpio_init.Mode = GPIO_MODE_OUTPUT_PP;          /* 推挽输出 */
    gpio_init.Pull = GPIO_NOPULL;                  /* 上下拉电阻不使能 */
    gpio_init.Speed = GPIO_SPEED_FREQ_VERY_HIGH;   /* GPIO 速度等级 */
    gpio_init.Pin = RS485_TXEN_PIN;
    HAL_GPIO_Init(RS485_TXEN_GPIO_PORT, &gpio_init);
}
```

30.3.4串口中断服务程序工作流程

串口中断服务程序是最核心的部分，主要实现如下三个功能

- 收到新的数据后，会将数据压入 RX_FIFO。
- 检测到发送缓冲区空后，会从 TX_FIFO 中取下一个数据并发送。
- 如果是 RS485 半双工串口，发送前会设置一个 GPIO=1 控制 RS485 收发器进入发送状态，当最后一个字节的最后一个 bit 传送完毕后，设置这个 GPIO=0 让 RS485 收发器进入接收状态。

下面我们分析一下串口中断处理的完整过程。

当产生串口中断后，CPU 会查找中断向量表，获得中断服务程序的入口地址。入口函数为 USART1_IRQHandler，这个函数在启动文件 startup_stm32h743xx.s 汇编代码中已经有实现。我们在 c 代码中需要重写一个同样名字的函数就可以重载它。如果不重载，启动文件中缺省的中断服务程序就是一个死循环，等于 while(1);

我们将串口中断服务程序放在 bsp_uart_fifo.c 文件，没有放到 stm32h7xx_it.c。当应用不需要串口功能时，直接从工程中删除 bsp_uart_fifo.c 接口，不必再去整理 stm32h7xx_it.c 这个文件。下面展示的代码是 8 个串口的中断服务程序：

```
#if UART1_FIFO_EN == 1
void USART1_IRQHandler(void)
{
    UartIRQ(&g_tUart1);
}
#endif

#if UART2_FIFO_EN == 1
void USART2_IRQHandler(void)
{
    UartIRQ(&g_tUart2);
```

```
}
#endif

#if UART3_FIFO_EN == 1
void USART3_IRQHandler(void)
{
	UartIRQ(&g_tUart3);
}
#endif

#if UART4_FIFO_EN == 1
void UART4_IRQHandler(void)
{
	UartIRQ(&g_tUart4);
}
#endif

#if UART5_FIFO_EN == 1
void UART5_IRQHandler(void)
{
	UartIRQ(&g_tUart5);
}
#endif

#if UART6_FIFO_EN == 1
void USART6_IRQHandler(void)
{
	UartIRQ(&g_tUart6);
}
#endif

#if UART7_FIFO_EN == 1
void UART7_IRQHandler(void)
{
	UartIRQ(&g_tUart7);
}
#endif

#if UART8_FIFO_EN == 1
void UART8_IRQHandler(void)
{
	UartIRQ(&g_tUart8);
}
#endif
```

大家可以看到，这 8 个中断服务程序都调用了同一个处理函数 UartIRQ。我们只需要调通一个串口 FIFO 驱动，那么其他的串口驱动也就都通了。

下面，我们来看看 UartIRQ 函数的实现代码。

```
/*
*********************************************************************************************************
*	函 数 名: UartIRQ
*	功能说明: 供中断服务程序调用，通用串口中断处理函数
*	形	参: _pUart : 串口设备
*	返 回 值: 无
*********************************************************************************************************
*/
static void UartIRQ(UART_T *_pUart)
```

```
{
    uint32_t isrflags   = READ_REG(_pUart->uart->ISR);
    uint32_t crlits     = READ_REG(_pUart->uart->CR1);
    uint32_t cr3its     = READ_REG(_pUart->uart->CR3);

    /* 处理接收中断  */
    if ((isrflags & USART_ISR_RXNE) != RESET)
    {
        /* 从串口接收数据寄存器读取数据存放到接收 FIFO */
        uint8_t ch;

        ch = READ_REG(_pUart->uart->RDR);               /* 读串口接收数据寄存器 */
        _pUart->pRxBuf[_pUart->usRxWrite] = ch;         /* 填入串口接收 FIFO */
        if (++_pUart->usRxWrite >= _pUart->usRxBufSize) /* 接收 FIFO 的写指针+1 */
        {
            _pUart->usRxWrite = 0;
        }
        if (_pUart->usRxCount < _pUart->usRxBufSize)    /* 统计未处理的字节个数 */
        {
            _pUart->usRxCount++;
        }

        /* 回调函数,通知应用程序收到新数据,一般是发送 1 个消息或者设置一个标记 */
        //if (_pUart->usRxWrite == _pUart->usRxRead)
        //if (_pUart->usRxCount == 1)
        {
            if (_pUart->ReciveNew)
            {
                _pUart->ReciveNew(ch); /* 比如，交给 MODBUS 解码程序处理字节流 */
            }
        }
    }

    /* 处理发送缓冲区空中断 */
    if ( ((isrflags & USART_ISR_TXE) != RESET) && (crlits & USART_CR1_TXEIE) != RESET)
    {
        //if (_pUart->usTxRead == _pUart->usTxWrite)
        if (_pUart->usTxCount == 0)  /* 发送缓冲区已无数据可取 */
        {
        /* 发送缓冲区的数据已取完时， 禁止发送缓冲区空中断 （注意：此时最后 1 个数据还未真正发送完毕）*/
            //USART_ITConfig(_pUart->uart, USART_IT_TXE, DISABLE);
            CLEAR_BIT(_pUart->uart->CR1, USART_CR1_TXEIE);

            /* 使能数据发送完毕中断 */
            //USART_ITConfig(_pUart->uart, USART_IT_TC, ENABLE);
            SET_BIT(_pUart->uart->CR1, USART_CR1_TCIE);
        }
        Else  /*  还有数据等待发送 */
        {
            _pUart->Sending = 1;

            /* 从发送 FIFO 取 1 个字节写入串口发送数据寄存器 */
            //USART_SendData(_pUart->uart, _pUart->pTxBuf[_pUart->usTxRead]);
            _pUart->uart->TDR = _pUart->pTxBuf[_pUart->usTxRead];
            if (++_pUart->usTxRead >= _pUart->usTxBufSize)
            {
                _pUart->usTxRead = 0;
            }
```

```
            _pUart->usTxCount--;
        }

    }
    /* 数据 bit 位全部发送完毕的中断 */
    if (((isrflags & USART_ISR_TC) != RESET) && ((cr1its & USART_CR1_TCIE) != RESET))
    {
        //if (_pUart->usTxRead == _pUart->usTxWrite)
        if (_pUart->usTxCount == 0)
        {
            /* 如果发送 FIFO 的数据全部发送完毕，禁止数据发送完毕中断 */
            //USART_ITConfig(_pUart->uart, USART_IT_TC, DISABLE);
            CLEAR_BIT(_pUart->uart->CR1, USART_CR1_TCIE);

            /* 回调函数, 一般用来处理 RS485 通信，将 RS485 芯片设置为接收模式，避免抢占总线 */
            if (_pUart->SendOver)
            {
                _pUart->SendOver();
            }

            _pUart->Sending = 0;
        }
        else
        {
            /* 正常情况下，不会进入此分支 */

            /* 如果发送 FIFO 的数据还未完毕，则从发送 FIFO 取 1 个数据写入发送数据寄存器 */
            //USART_SendData(_pUart->uart, _pUart->pTxBuf[_pUart->usTxRead]);
            _pUart->uart->TDR = _pUart->pTxBuf[_pUart->usTxRead];
            if (++_pUart->usTxRead >= _pUart->usTxBufSize)
            {
                _pUart->usTxRead = 0;
            }
            _pUart->usTxCount--;
        }
    }

    /* 清除中断标志 */
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_PEF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_FEF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_NEF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_OREF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_IDLEF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_TCF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_LBDF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_CTSF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_CMF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_WUF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_TXFECF);
}
```

中断服务程序的处理主要分为两部分，接收数据的处理和发送数据的处理，详情看程序注释即可，已经比较详细，下面重点把思路说一下。

◆ 接收数据处理

接收数据的处理是判断 ISR 寄存器的 USART_ISR_RXNE 标志是否置位，如果置位表示 RDR 接收寄

存器已经存入数据。然后将数据读入到接收 FIFO 空间。

特别注意里面的 ReciveNew 处理，这个在 Modbus 协议里面要用到。

◆ 发送数据处理

发送数据主要是发送空中断 TEX 和发送完成中断 TC 的处理，当 TXE=1 时，只是表示发送数据寄存器为空了，此时可以填充下一个准备发送的数据了。当为 TDR 发送寄存器赋值后，硬件启动发送，等所有的 bit 传送完毕后，TC 标志设置为 1。如果是 RS232 全双工通信，可以只用 TXE 标志控制发送过程。如果是 RS485 半双工通信，就需要利用 TC 标志了，因为在最后一个 bit 传送完毕后，需要设置 RS485 收发器进入到接收状态。

30.3.5 串口数据发送

串口数据的发送主要涉及到下面三个函数：

```
/*
*********************************************************************************************************
*    函 数 名: comSendBuf
*    功能说明: 向串口发送一组数据。数据放到发送缓冲区后立即返回，由中断服务程序在后台完成发送
*    形    参: _ucPort: 端口号(COM1 - COM8)
*              _ucaBuf: 待发送的数据缓冲区
*              _usLen : 数据长度
*    返 回 值: 无
*********************************************************************************************************
*/
void comSendBuf(COM_PORT_E _ucPort, uint8_t *_ucaBuf, uint16_t _usLen)
{
    UART_T *pUart;

    pUart = ComToUart(_ucPort);
    if (pUart == 0)
    {
        return;
    }

    if (pUart->SendBefor != 0)
    {
        pUart->SendBefor();         /* 如果是 RS485 通信，可以在这个函数中将 RS485 设置为发送模式 */
    }

    UartSend(pUart, _ucaBuf, _usLen);
}

/*
*********************************************************************************************************
*    函 数 名: comSendChar
*    功能说明: 向串口发送 1 个字节。数据放到发送缓冲区后立即返回，由中断服务程序在后台完成发送
*    形    参: _ucPort: 端口号(COM1 - COM8)
*              _ucByte: 待发送的数据
*    返 回 值: 无
*********************************************************************************************************
*/
void comSendChar(COM_PORT_E _ucPort, uint8_t _ucByte)
```

```
{
    comSendBuf(_ucPort, &_ucByte, 1);
}
/*
*********************************************************************************************************
*    函 数 名: UartSend
*    功能说明: 填写数据到 UART 发送缓冲区,并启动发送中断。中断处理函数发送完毕后，自动关闭发送中断
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void UartSend(UART_T *_pUart, uint8_t *_ucaBuf, uint16_t _usLen)
{
    uint16_t i;

    for (i = 0; i < _usLen; i++)
    {
        /* 如果发送缓冲区已经满了，则等待缓冲区空 */
        while (1)
        {
            __IO uint16_t usCount;

            DISABLE_INT();
            usCount = _pUart->usTxCount;
            ENABLE_INT();

            if (usCount < _pUart->usTxBufSize)
            {
                break;
            }
            else if(usCount == _pUart->usTxBufSize)/* 数据已填满缓冲区 */
            {
                if((_pUart->uart->CR1 & USART_CR1_TXEIE) == 0)
                {
                    SET_BIT(_pUart->uart->CR1, USART_CR1_TXEIE);
                }
            }
        }

        /* 将新数据填入发送缓冲区 */
        _pUart->pTxBuf[_pUart->usTxWrite] = _ucaBuf[i];

        DISABLE_INT();
        if (++_pUart->usTxWrite >= _pUart->usTxBufSize)
        {
            _pUart->usTxWrite = 0;
        }
        _pUart->usTxCount++;
        ENABLE_INT();
    }

    SET_BIT(_pUart->uart->CR1, USART_CR1_TXEIE);   /* 使能发送中断（缓冲区空） */
}
```

函数 comSendChar 是发送一个字节，通过调用函数 comSendBuf 实现，而函数 comSendBuf 又是通过调用函数 UartSend 实现，这个函数是重点。

函数 UartSend 的作用就是把要发送的数据填到发送缓冲区里面，并使能发送空中断。

- 如果要发送的数据没有超过发送缓冲区大小，实现起来还比较容易，直接把数据填到 FIFO 里面，并使能发送空中断即可。
- 如果超过了 FIFO 大小，就需要等待有空间可用，针对这种情况有个重要的知识点，就是当缓冲刚刚填满的时候要判断发送空中断是否开启了，如果填满了还没有开启，就会卡死在 while 循环中，所以多了一个刚填满时的判断，填满了还没有开启发送空中断，要开启下。

注意：由于函数 UartSend 做了 static 作用域限制，仅可在 bsp_uart_fifo.c 文件中调用。函数 comSendChar 和 comSendBuf 是供用户调用的。

函数 comSendBuf 中调用了一个函数 pUart = ComToUart(_ucPort)，这个函数是将整数的 COM 端口号转换为 UART 结构体指针。

```
/*
*********************************************************************************************************
*    函 数 名: ComToUart
*    功能说明: 将 COM 端口号转换为 UART 指针
*    形    参: _ucPort: 端口号(COM1 - COM8)
*    返 回 值: uart 指针
*********************************************************************************************************
*/
UART_T *ComToUart(COM_PORT_E _ucPort)
{
    if (_ucPort == COM1)
    {
        #if UART1_FIFO_EN == 1
            return &g_tUart1;
        #else
            return 0;
        #endif
    }
    else if (_ucPort == COM2)
    {
        #if UART2_FIFO_EN == 1
            return &g_tUart2;
        #else
            return 0;
        #endif
    }
    else if (_ucPort == COM3)
    {
        #if UART3_FIFO_EN == 1
            return &g_tUart3;
        #else
            return 0;
        #endif
    }
    else if (_ucPort == COM4)
    {
        #if UART4_FIFO_EN == 1
            return &g_tUart4;
        #else
```

```
            return 0;
        #endif
    }
    else if (_ucPort == COM5)
    {
        #if UART5_FIFO_EN == 1
            return &g_tUart5;
        #else
            return 0;
        #endif
    }
    else if (_ucPort == COM6)
    {
        #if UART6_FIFO_EN == 1
            return &g_tUart6;
        #else
            return 0;
        #endif
    }
    else if (_ucPort == COM7)
    {
        #if UART7_FIFO_EN == 1
            return &g_tUart7;
        #else
            return 0;
        #endif
    }
    else if (_ucPort == COM8)
    {
        #if UART8_FIFO_EN == 1
            return &g_tUart8;
        #else
            return 0;
        #endif
    }
    else
    {
        Error_Handler(__FILE__, __LINE__);
        return 0;
    }
}
```

30.3.6 串口数据接收

下面我们再来看看接收的函数：

```
/*
*********************************************************************************************************
*    函 数 名: comGetChar
*    功能说明: 从接收缓冲区读取 1 字节，非阻塞。无论有无数据均立即返回。
*    形    参: _ucPort: 端口号(COM1 - COM8)
*              _pByte: 接收到的数据存放在这个地址
*    返 回 值: 0 表示无数据, 1 表示读取到有效字节
*********************************************************************************************************
*/
uint8_t comGetChar(COM_PORT_E _ucPort, uint8_t *_pByte)
```

```
{
    UART_T *pUart;

    pUart = ComToUart(_ucPort);
    if (pUart == 0)
    {
        return 0;
    }

    return UartGetChar(pUart, _pByte);
}

/*
*********************************************************************************************************
*    函 数 名: UartGetChar
*    功能说明: 从串口接收缓冲区读取 1 字节数据 （用于主程序调用）
*    形    参: _pUart : 串口设备
*              _pByte : 存放读取数据的指针
*    返 回 值: 0 表示无数据  1 表示读取到数据
*********************************************************************************************************
*/
static uint8_t UartGetChar(UART_T *_pUart, uint8_t *_pByte)
{
    uint16_t usCount;

    /* usRxWrite 变量在中断函数中被改写，主程序读取该变量时，必须进行临界区保护 */
    DISABLE_INT();
    usCount = _pUart->usRxCount;
    ENABLE_INT();

    /* 如果读和写索引相同，则返回 0 */
    //if (_pUart->usRxRead == usRxWrite)
    if (usCount == 0)  /* 已经没有数据 */
    {
        return 0;
    }
    else
    {
        *_pByte = _pUart->pRxBuf[_pUart->usRxRead];        /* 从串口接收 FIFO 取 1 个数据 */

        /* 改写 FIFO 读索引 */
        DISABLE_INT();
        if (++_pUart->usRxRead >= _pUart->usRxBufSize)
        {
            _pUart->usRxRead = 0;
        }
        _pUart->usRxCount--;
        ENABLE_INT();
        return 1;
    }
}
```

函数 comGetChar 是专门供用户调用的，用于从接收 FIFO 中读取 1 个数据。具体代码的实现也比较好理解，主要是接收 FIFO 的空间调整。

注意：由于函数 UartGetChar 做了 static 作用域限制，仅可在 bsp_uart_fifo.c 文件中调用。

30.3.7串口 printf 实现

printf 函数是标准 c 库函数。最原来的意思是打印输出到显示器。在单片机，我们常用它来打印调试信息到串口，通过计算机上运行的串口软件来监视程序的运行状态。

为什么要用 printf 函数，而不用串口发送的函数。因为 printf 函数的形参功能很强大，它支持各种数值转换。比如将整数、浮点数转换为字符串，支持整数左对齐、右对齐显示等。

我们设计的很多裸机例子都是用 printf 函数输出运行结果的。因为如果加上显示屏驱动后，会将程序搞的很复杂，显示部分的代码量超过了例程本身要演示的核心功能代码。用串口做输出，移植很方便，现在很少有不带串口的单片机。

实现 printf 输出到串口，只需要在工程中添加两个函数：

```
/*
*********************************************************************************************************
*    函 数 名: fputc
*    功能说明: 重定义 putc 函数，这样可以使用 printf 函数从串口 1 打印输出
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
int fputc(int ch, FILE *f)
{
#if 1    /* 将需要 printf 的字符通过串口中断 FIFO 发送出去，printf 函数会立即返回 */
    comSendChar(COM1, ch);

    return ch;
#else    /* 采用阻塞方式发送每个字符,等待数据发送完毕 */
    /* 写一个字节到 USART1 */
    USART1->TDR = ch;

    /* 等待发送结束 */
    while((USART1->ISR & USART_ISR_TC) == 0)
    {}

    return ch;
#endif
}

/*
*********************************************************************************************************
*    函 数 名: fgetc
*    功能说明: 重定义 getc 函数，这样可以使用 getchar 函数从串口 1 输入数据
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
int fgetc(FILE *f)
{

#if 1    /* 从串口接收 FIFO 中取 1 个数据, 只有取到数据才返回 */
    uint8_t ucData;

    while(comGetChar(COM1, &ucData) == 0);
```

```
    return ucData;
#else
    /* 等待接收到数据 */
    while((USART1->ISR & USART_ISR_RXNE) == 0)
    {}

    return (int)USART1->RDR;
#endif
}
```

printf 函数是非阻塞的，执行后会立即返回，串口中断服务程序会陆续将数据发送出去。

30.4 串口 FIFO 板级支持包（bsp_uart_fifo.c）

串口驱动文件 bsp_uart_fifo.c 主要实现了如下几个 API 供用户调用：

- bsp_InitUart
- comSendBuf
- comSendChar
- comGetChar

30.4.1 函数 bsp_InitUart

函数原型：

```
void bsp_InitUart(void)
```

函数描述：

此函数主要用于串口的初始化，使用所有其它 API 之前，务必优先调用此函数。

使用举例：

串口的初始化函数在 bsp.c 文件的 bsp_Init 函数里面调用。

30.4.2 函数 comSendBuf

函数原型：

```
void comSendBuf(COM_PORT_E _ucPort, uint8_t *_ucaBuf, uint16_t _usLen);
```

函数描述：

此函数用于向串口发送一组数据，非阻塞方式，数据放到发送缓冲区后立即返回，由中断服务程序在后台完成发送。

函数参数：

- 第 1 个参数_ucPort 是端口号，范围 COM1 - COM8。
- 第 2 个参数_ucaBuf 是待发送的数据缓冲区地址。
- 第 3 个参数_usLen 是要发送数据的字节数。

注意事项：

- 此函数的解读在本章 30.3.5 小节。
- 发送的数据最好不要超过 bsp_uart_fifo.h 文件中定义的发送缓冲区大小，从而实现最优的工作方式。因为超过后需要在发送函数等待有发送空间可用。

使用举例：

调用此函数前，务必优先调用函数 bsp_InitUart 进行初始化。

```
const char buf1[] = "接收到串口命令 1\r\n";
comSendBuf(COM1, (uint8_t *)buf1, strlen(buf1));
```

30.4.3 函数 comSendChar

函数原型：

```
void comSendChar(COM_PORT_E _ucPort, uint8_t _ucByte);
```

函数描述：

此函数用于向串口发送 1 个字节，非阻塞方式，数据放到发送缓冲区后立即返回，由中断服务程序在后台完成发送。此函数是通过调用函数 comSendBuf 实现的。

函数参数：

- 第 1 个参数_ucPort 是端口号，范围 COM1 - COM8。
- 第 2 个参数_ucByte 是待发送的数据。

注意事项：

- 此函数的解读在本章 30.3.2 小节。

使用举例：

调用此函数前，务必优先调用函数 bsp_InitUart 进行初始化。比如通过串口 1 发送一个字符 c：comSendChar(COM1, 'c')。

30.4.4 函数 comGetChar

函数原型：

```
uint8_t comGetChar(COM_PORT_E _ucPort, uint8_t *_pByte)
```

函数描述：

此函数用于从接收缓冲区读取 1 字节，非阻塞。无论有无数据均立即返回。

函数参数：

- 第 1 个参数_ucPort 是端口号，范围 COM1 - COM8。
- 第 2 个参数_pByte 用于存放接收到的数据。
- 返回值，返回 0 表示无数据，1 表示读取到有效字节。

注意事项：

- 此函数的解读在本章 30.3.6 小节。

使用举例：

调用此函数前，务必优先调用函数 bsp_InitUart 进行初始化。

比如从串口 1 读取一个字符就是：comGetChar(COM1, &read)。

30.5 串口 FIFO 驱动移植和使用

串口 FIFO 移植步骤如下：

- 第 1 步：复制 bsp_uart_fifo.h 和 bsp_uart_fifo.c 到自己的工程目录，并添加到工程里面。
- 第 2 步：根据自己要使用的串口和收发缓冲大小，修改下面的宏定义即可。

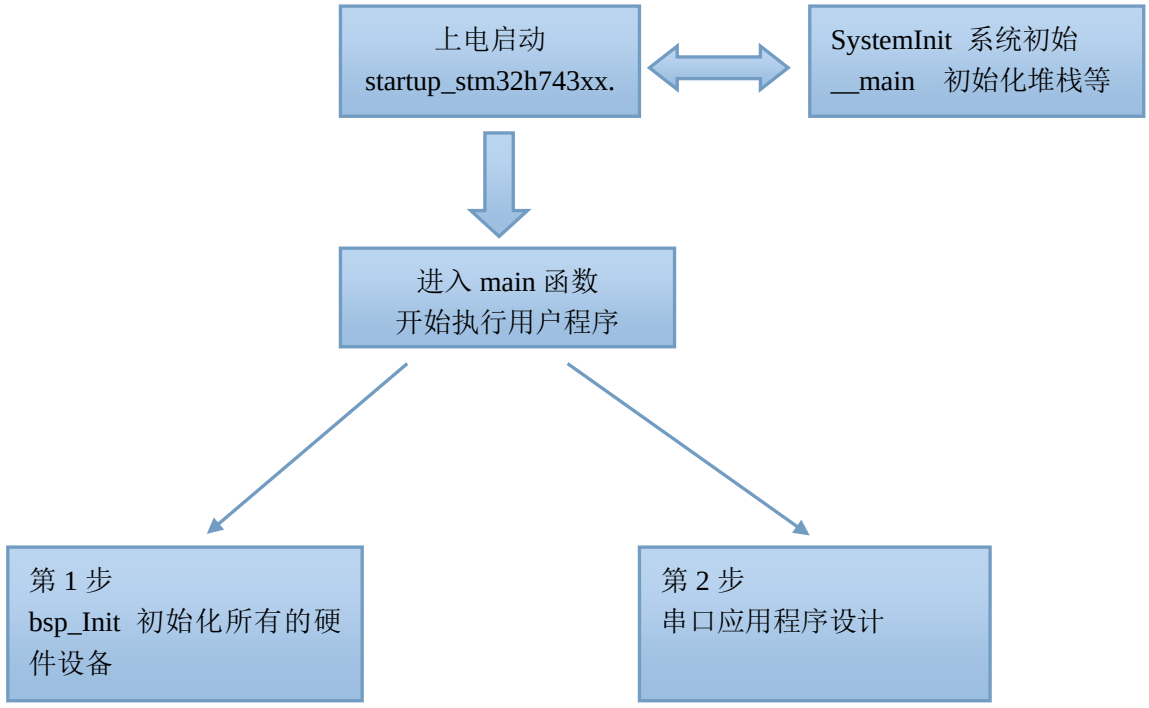
```
#define  UART1_FIFO_EN 1
#define  UART2_FIFO_EN 0
#define  UART3_FIFO_EN 0
#define  UART4_FIFO_EN 0
#define  UART5_FIFO_EN 0
#define  UART6_FIFO_EN 0
#define  UART7_FIFO_EN 0
#define  UART8_FIFO_EN 0

/* 定义串口波特率和 FIFO 缓冲区大小，分为发送缓冲区和接收缓冲区，支持全双工 */
#if UART1_FIFO_EN == 1
    #define UART1_BAUD          115200
    #define UART1_TX_BUF_SIZE   1*1024
    #define UART1_RX_BUF_SIZE   1*1024
#endif

#if UART2_FIFO_EN == 1
    #define UART2_BAUD          9600
    #define UART2_TX_BUF_SIZE   10
    #define UART2_RX_BUF_SIZE   2*1024
#endif

#if UART3_FIFO_EN == 1
    #define UART3_BAUD          9600
    #define UART3_TX_BUF_SIZE   1*1024
    #define UART3_RX_BUF_SIZE   1*1024
#endif

#if UART4_FIFO_EN == 1
    #define UART4_BAUD          115200
    #define UART4_TX_BUF_SIZE   1*1024
    #define UART4_RX_BUF_SIZE   1*1024
#endif

#if UART5_FIFO_EN == 1
    #define UART5_BAUD          115200
    #define UART5_TX_BUF_SIZE   1*1024
    #define UART5_RX_BUF_SIZE   1*1024
#endif

#if UART6_FIFO_EN == 1
    #define UART6_BAUD          115200
```

```
    #define UART6_TX_BUF_SIZE   1*1024
    #define UART6_RX_BUF_SIZE   1*1024
#endif

#if UART7_FIFO_EN == 1
    #define UART7_BAUD          115200
    #define UART7_TX_BUF_SIZE   1*1024
    #define UART7_RX_BUF_SIZE   1*1024
#endif

#if UART8_FIFO_EN == 1
    #define UART8_BAUD          115200
    #define UART8_TX_BUF_SIZE   1*1024
    #define UART8_RX_BUF_SIZE   1*1024
#endif
```

- 第 3 步：这几个驱动文件主要用到 HAL 库的 GPIO 和串口驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 4 步，应用方法看本章节配套例子即可。

30.6 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器和 LED。

- 第 2 部分，应用程序设计部分，实现了一个串口接收命令，返回消息的简单功能。

30.7 实验例程说明（MDK）

配套例子：

V7-015_串口和 PC 机通信（驱动支持 8 串口 FIFO）

实验目的：

1. 学习串口与 PC 通信。

实验内容：

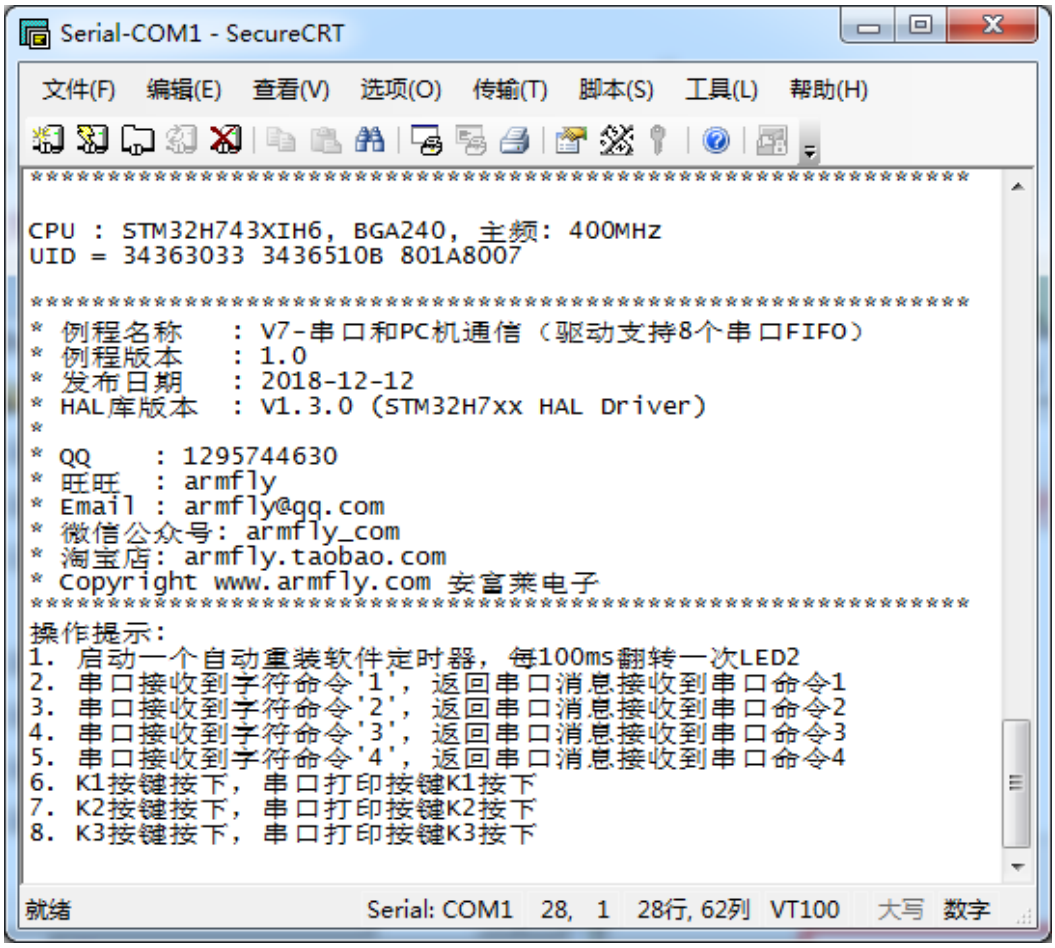
1. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。

实验操作：

1. 串口接收到字符命令'1'，返回串口消息"接收到串口命令 1"。
2. 串口接收到字符命令'2'，返回串口消息"接收到串口命令 2"。
3. 串口接收到字符命令'3'，返回串口消息"接收到串口命令 3"。
4. 串口接收到字符命令'4'，返回串口消息"接收到串口命令 4"。
5. K1 按键按下，串口打印"按键 K1 按下"。
6. K2 按键按下，串口打印"按键 K2 按下"。
7. K3 按键按下，串口打印"按键 K3 按下"。

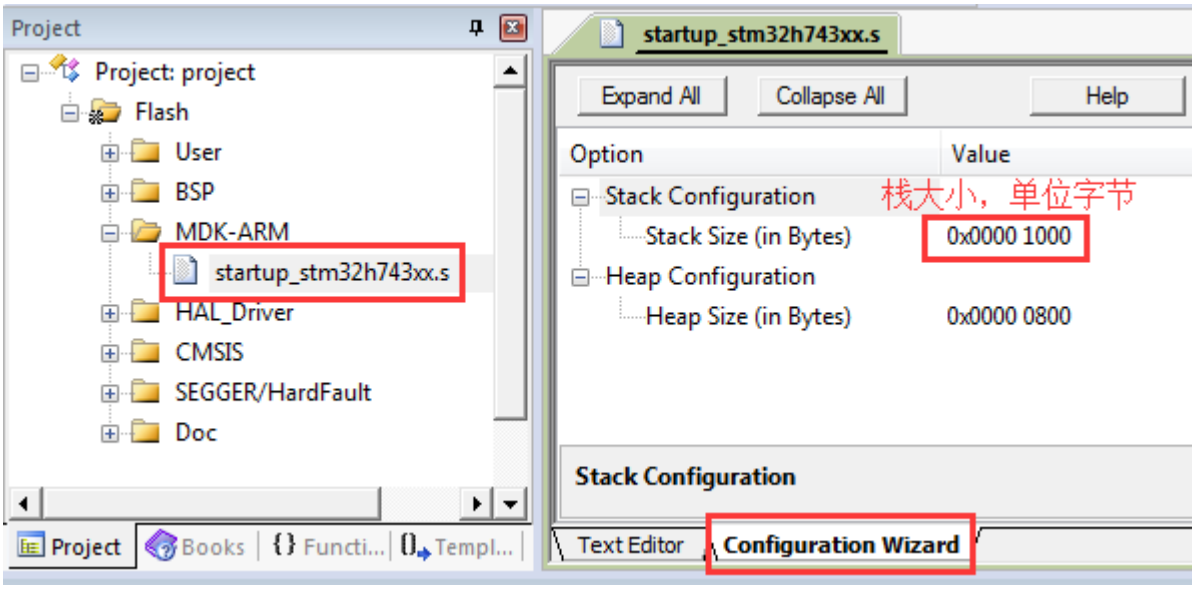
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

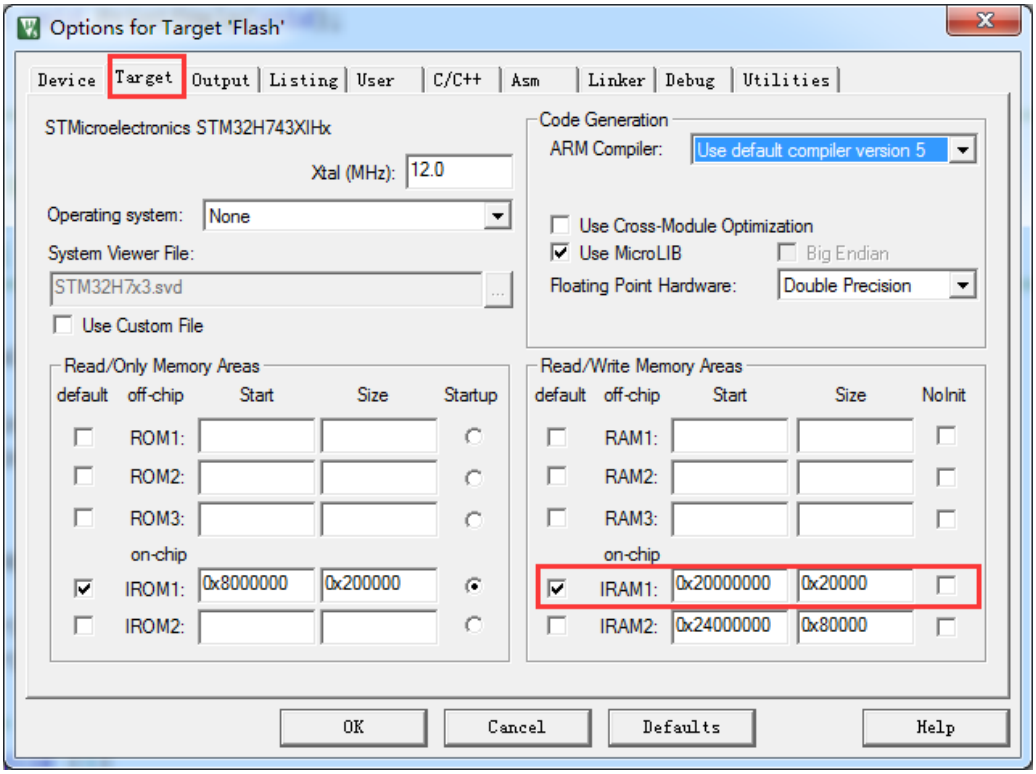


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
```

```
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **每 10ms 调用一次蜂鸣器处理：**

蜂鸣器处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能：**

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- 串口接收到字符命令'1'，返回串口消息"接收到串口命令 1"。
- 串口接收到字符命令'2'，返回串口消息"接收到串口命令 2"。
- 串口接收到字符命令'3'，返回串口消息"接收到串口命令 3"。

- 串口接收到字符命令'4'，返回串口消息"接收到串口命令 4"。
- K1 按键按下，串口打印"按键 K1 按下"。
- K2 按键按下，串口打印"按键 K2 按下"。
- K3 按键按下，串口打印"按键 K3 按下"。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;
    uint8_t read;
    const char buf1[] = "接收到串口命令 1\r\n";
    const char buf2[] = "接收到串口命令 2\r\n";
    const char buf3[] = "接收到串口命令 3\r\n";
    const char buf4[] = "接收到串口命令 4\r\n";


    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 主程序大循环 */
    while (1)
    {
        /* CPU 空闲时执行的函数，在 bsp.c */
        bsp_Idle();

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            /* 翻转 LED2 的状态 */
            bsp_LedToggle(2);
        }

        /* 接收到的串口命令处理 */
        if (comGetChar(COM1, &read))
        {
            switch (read)
            {
                case '1':
                    comSendBuf(COM1, (uint8_t *)buf1, strlen(buf1));
                    break;

                case '2':
                    comSendBuf(COM1, (uint8_t *)buf2, strlen(buf2));
                    break;
```

```
                case '3':
                    comSendBuf(COM1, (uint8_t *)buf3, strlen(buf3));
                    break;

                case '4':
                    comSendBuf(COM1, (uint8_t *)buf4, strlen(buf4));
                    break;

                default:
                    break;
            }
        }

        /* 处理按键事件 */
        ucKeyCode = bsp_GetKey();
        if (ucKeyCode > 0)
        {
            /* 有键按下 */
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:       /* 按键 K1 键按下 */
                    printf("按键 K1 按下\r\n");
                    bsp_LedToggle(1);
                    break;

                case KEY_DOWN_K2:       /* 按键 K2 键按下 */
                    printf("按键 K2 按下\r\n");
                    bsp_LedToggle(3);
                    break;

                case KEY_DOWN_K3:       /* 按键 K3 键按下 */
                    printf("按键 K3 按下\r\n");
                    bsp_LedToggle(4);
                    break;

                default:
                    break;
            }
        }
    }
}
```

30.8 实验例程说明（IAR）

配套例子：

V7-015_串口和 PC 机通信（驱动支持 8 串口 FIFO）

实验目的：

1. 学习串口与 PC 通信。

实验内容：

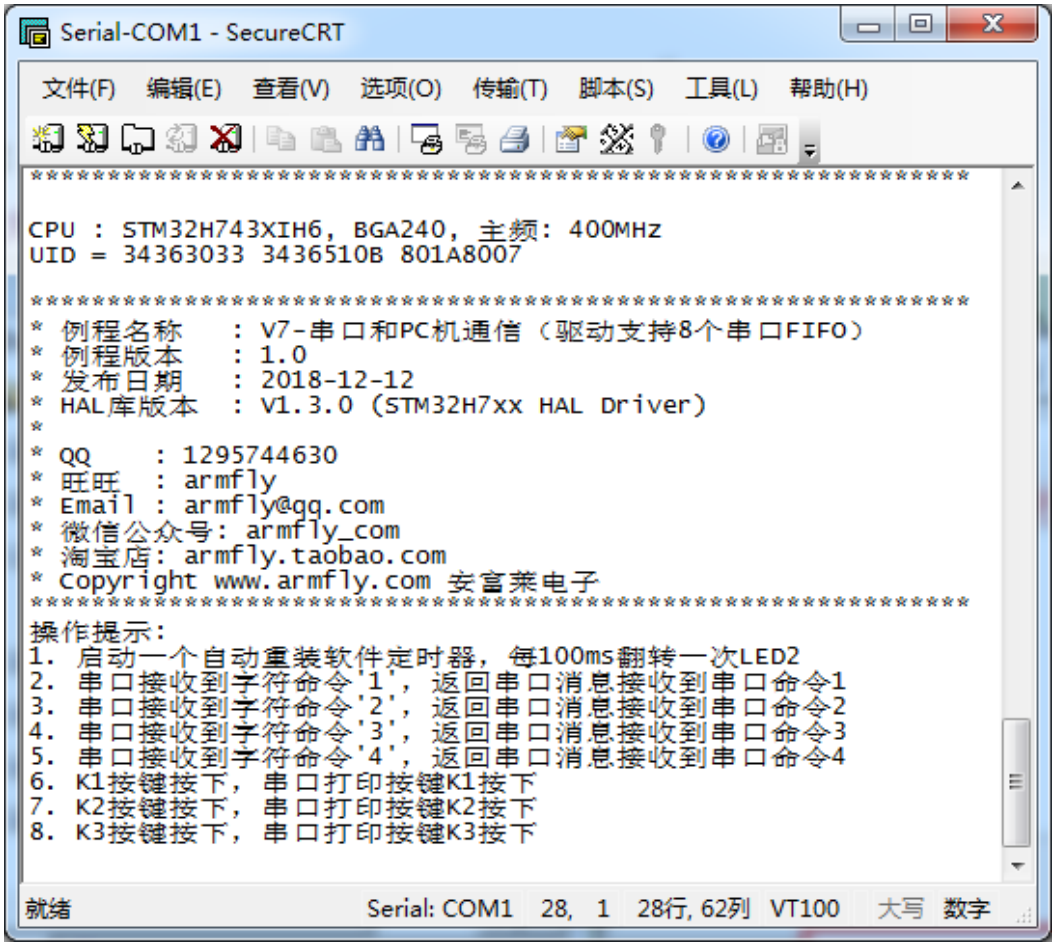
1. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。

实验操作：

1. 串口接收到字符命令'1'，返回串口消息"接收到串口命令 1"。
2. 串口接收到字符命令'2'，返回串口消息"接收到串口命令 2"。
3. 串口接收到字符命令'3'，返回串口消息"接收到串口命令 3"。
4. 串口接收到字符命令'4'，返回串口消息"接收到串口命令 4"。
5. K1 按键按下，串口打印"按键 K1 按下"。
6. K2 按键按下，串口打印"按键 K2 按下"。
7. K3 按键按下，串口打印"按键 K3 按下"。

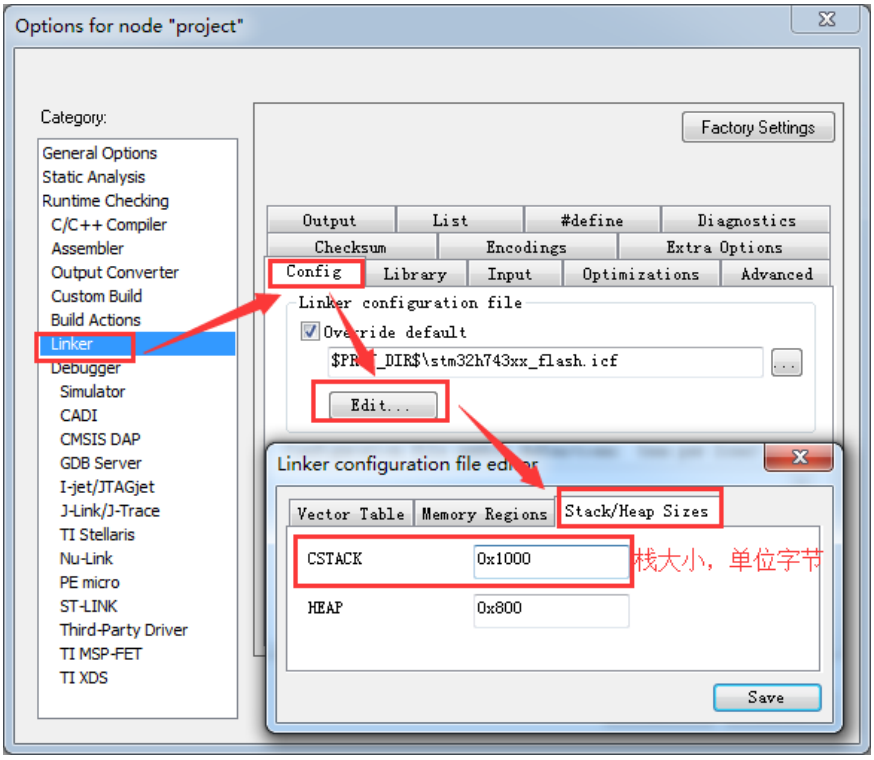
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

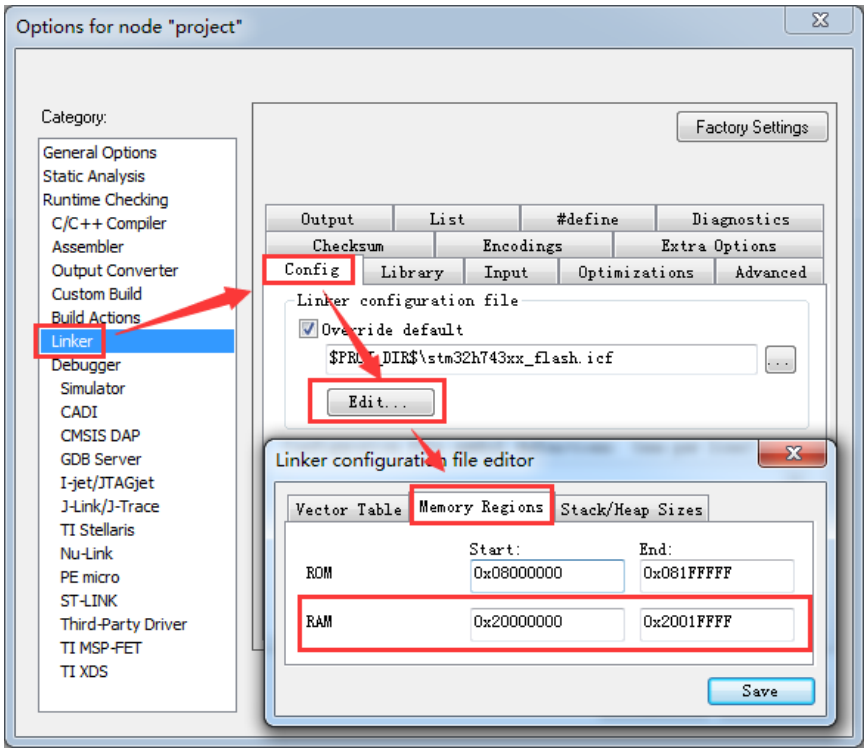


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
```

```
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
```

```
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ 每 10ms 调用一次蜂鸣器处理：

蜂鸣器处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ 主功能：

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- 串口接收到字符命令'1'，返回串口消息"接收到串口命令 1"。
- 串口接收到字符命令'2'，返回串口消息"接收到串口命令 2"。
- 串口接收到字符命令'3'，返回串口消息"接收到串口命令 3"。
- 串口接收到字符命令'4'，返回串口消息"接收到串口命令 4"。
- K1 按键按下，串口打印"按键 K1 按下"。
- K2 按键按下，串口打印"按键 K2 按下"。
- K3 按键按下，串口打印"按键 K3 按下"。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;
    uint8_t read;
    const char buf1[] = "接收到串口命令 1\r\n";
    const char buf2[] = "接收到串口命令 2\r\n";
    const char buf3[] = "接收到串口命令 3\r\n";
    const char buf4[] = "接收到串口命令 4\r\n";


    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */
```

```
    /* 主程序大循环 */
    while (1)
    {
        /* CPU 空闲时执行的函数，在 bsp.c */
        bsp_Idle();

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            /* 翻转 LED2 的状态 */
            bsp_LedToggle(2);
        }

        /* 接收到的串口命令处理 */
        if (comGetChar(COM1, &read))
        {
            switch (read)
            {
                case '1':
                    comSendBuf(COM1, (uint8_t *)buf1, strlen(buf1));
                    break;

                case '2':
                    comSendBuf(COM1, (uint8_t *)buf2, strlen(buf2));
                    break;

                case '3':
                    comSendBuf(COM1, (uint8_t *)buf3, strlen(buf3));
                    break;

                case '4':
                    comSendBuf(COM1, (uint8_t *)buf4, strlen(buf4));
                    break;

                default:
                    break;
            }
        }

        /* 处理按键事件 */
        ucKeyCode = bsp_GetKey();
        if (ucKeyCode > 0)
        {
            /* 有键按下 */
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:       /* 按键 K1 键按下 */
                    printf("按键 K1 按下\r\n");
                    bsp_LedToggle(1);
                    break;

                case KEY_DOWN_K2:       /* 按键 K2 键按下 */
                    printf("按键 K2 按下\r\n");
                    bsp_LedToggle(3);
                    break;

                case KEY_DOWN_K3:       /* 按键 K3 键按下 */
```

```
                        printf("按键 K3 按下\r\n");
                        bsp_LedToggle(4);
                        break;

                default:
                        break;
            }
        }
    }
}
```

30.9 总结

本章节就为大家讲解这么多， 重点是 8 串口 FIFO 的实现，而且移植也比较简单，可放心用于项目实战。

第31章 STM32H7 的 USART 应用之 RS485

本章教程为大家讲解 USART 应用之 485 总线。虽然这几年无线网络的使用率有所上升，有线的串行网络仍然提供最有力、最可靠的通信，特别是在恶劣的环境中。在需要抗噪、抗静电、抗电压故障的工业，建筑自动化领域仍然是有线通信的天下。

31.1 初学者重要提示
31.2 RS485 基础知识
31.3 RS485 硬件设计
31.4 RS485 驱动设计
31.5 RS485 板级支持包（bsp_uart_fifo.c）
31.6 RS485 驱动移植和使用
31.7 使用例程设计框架
31.8 实验例程说明（MDK）
31.9 实验例程说明（IAR）
31.10 总结

31.1 初学者重要提示

- 学习本章节前，务必优先学习第 30 章，RS485 用到的串口 FIFO 也是建立在 30 章的基础上。
- 了解了本章 31.2 和 31.3 小节的基础知识后，强烈推荐看此贴的两个文档，对 RS485 讲解的比较透彻，中文版：http://www.armbbs.cn/forum.php?mod=viewthread&tid=90753 。
- STM32H7 支持 RS485 的硬件流控制，即有一个专门的引脚来控制 485 PHY 的收发状态切换。V7 开发板用的 USART3，需要用 PD12 来控制，而这个引脚要用于 FMC，所以用的是一个通用 IO。
- 经常会有网友咨询为什么程序里面收发切换没有做延迟处理，这里就涉及到一个关键的知识点 TXE 发送空中断和 TC 发送完成中断的区别，详细看教程中说明即可。

31.2 RS485 的基础知识

- **背景知识（了解即可）**

智能仪表是随着 80 年代初单片机技术的成熟而发展起来的，现在世界仪表市场基本被智能仪表所垄断。究其原因就是企业信息化的需要，企业在仪表选型时其中的一个必要条件就是要具有联网通讯接口。最初是数据模拟信号输出简单过程量，后来仪表接口是 RS232 接口，这种接口可以实现点对点的通信方

式，但这种方式不能实现联网功能。随后出现的 RS485 解决了这个问题。

EIA-485 (过去叫做 RS-485 或者 RS485) 是隶属于 OSI 模型物理层的电气特性规定为 2 线、半双工、平衡传输线多点通信的标准，是由电信行业协会（TIA）及电子工业联盟（EIA）联合发布的标准。实现此标准的数字通信网可以在有电子噪声的环境下进行长距离有效率的通信。在线性多点总线的配置下，可以在一个网络上有多个接收器。因此适用在工业环境中。

EIA 一开始将 RS (Recommended Standard) 做为标准的前缀，不过后来为了便于识别标准的来源，已将 RS 改为 EIA/TIA。电子工业联盟（EIA）已结束运作，此标准目前是电信行业协会（TIA）维护，名称为 TIA-485，但工程师仍继续用 RS-485 来称呼此协议。

◆ **RS485 电气特性**

RS-485 的数据最高传输速率为 10Mbsp。

RS-485 接口是采用平衡驱动器和差分接收器的组合，抗共模干扰能力增强，即抗干扰噪声性好。

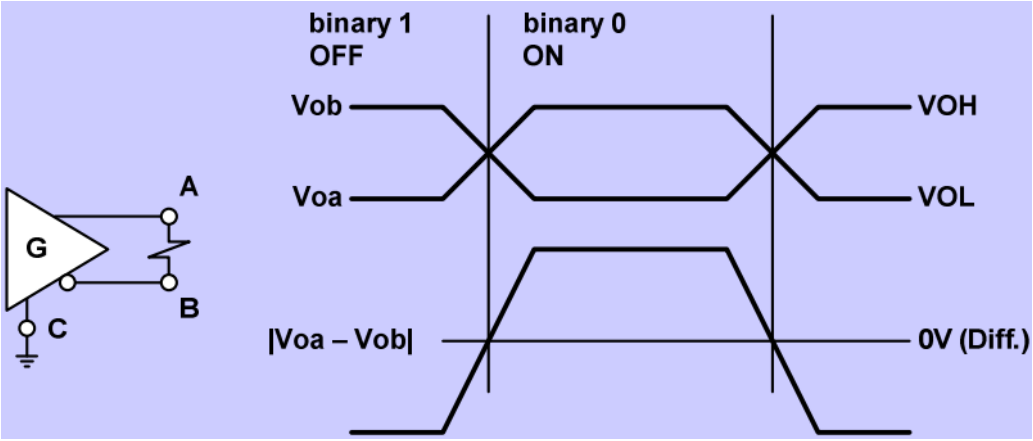
RS-485 最大的通信距离约为 1219m，最高传输速率为 10Mbsp，传输速率与传输距离成反比，在 100Kb/S 的传输速率下，才可以达到最大的通信距离，如果需传输更长的距离，需要加 485 中继器。RS-485 总线一般最大支持 32 个节点，如果使用特制的 485 芯片，可以达到 128 个或者 256 个节点，最大的可以支持到 400 个节点。

关于 RS485 的逻辑状态，不同厂家的芯片的定义可能不同，但不影响正常的数据收发，这里以 TI 的为例做个说明，TI 的定义方式如下：

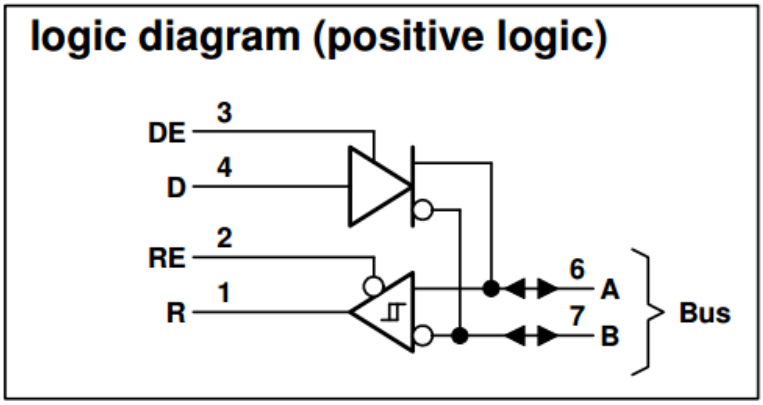
A 表示非反向输出 non-inverting output，B 表示反向输出 inverting output。

当 $V_A > V_B$ 的时候表示逻辑状态 0，被称为 ON。

当 $V_A < V_B$ 的时候表示逻辑状态 1，被称为 OFF。



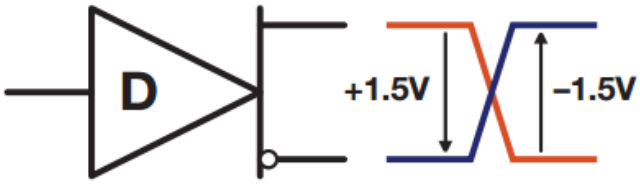
对应到实际芯片框图上就是下面这样 (DE 发送使能，D 是发送数据端，RE 是接收使能，R 是接收数据端)：

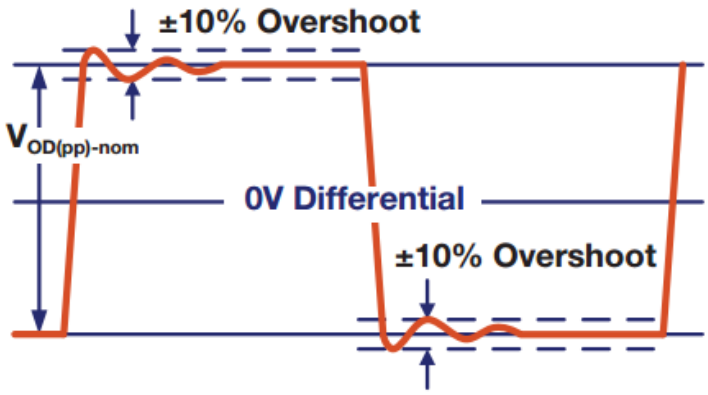


当用户在 D（Driver）引脚输入逻辑高电平时，将在 485 总线上实现逻辑状态 0，即 ON 状态。接收端 R（Receiver）将收到逻辑高电平。

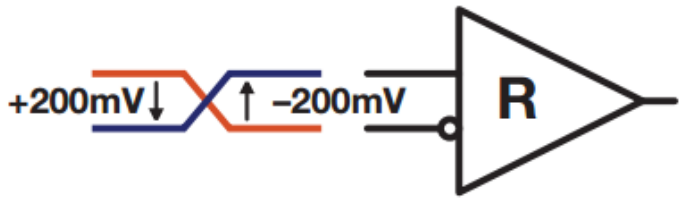
当用户在 D（Driver）引脚输入逻辑低电平时，将在 485 总线上实现逻辑状态 1，即 OFF 状态。接收端 R（Receiver）将收到逻辑低电平。

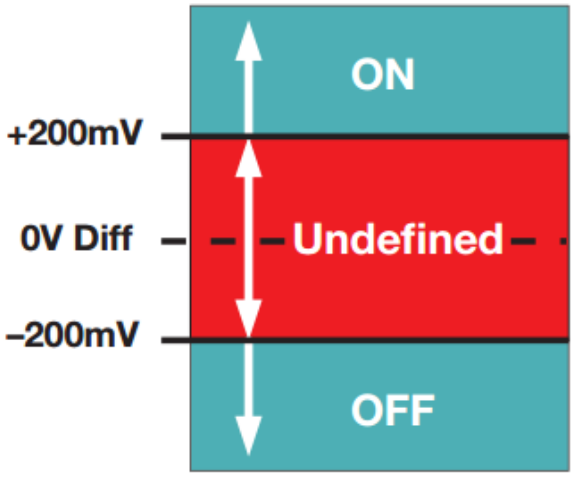
发送状态下，大于|±1.5V |可以有效表示逻辑状态 1 和逻辑状态 0：





接收状态下，大于|±200mv|可以有效表示逻辑状态 1 和逻辑状态 0：





31.3 RS485 硬件设计

STM32H743XIH6 最多可以支持 8 个独立的串口。其中串口 4 和串口 5 和 SDIO 的 GPIO 是共用的，也就是说，如果要用到 SD 卡，那么串口 4 和串口 5 将不能使用。串口 7 和 SPI3 共用，串口 8 和 RGB 硬件接口共用。串口功能可以分配到不同的 GPIO。我们常用的管脚分配如下：

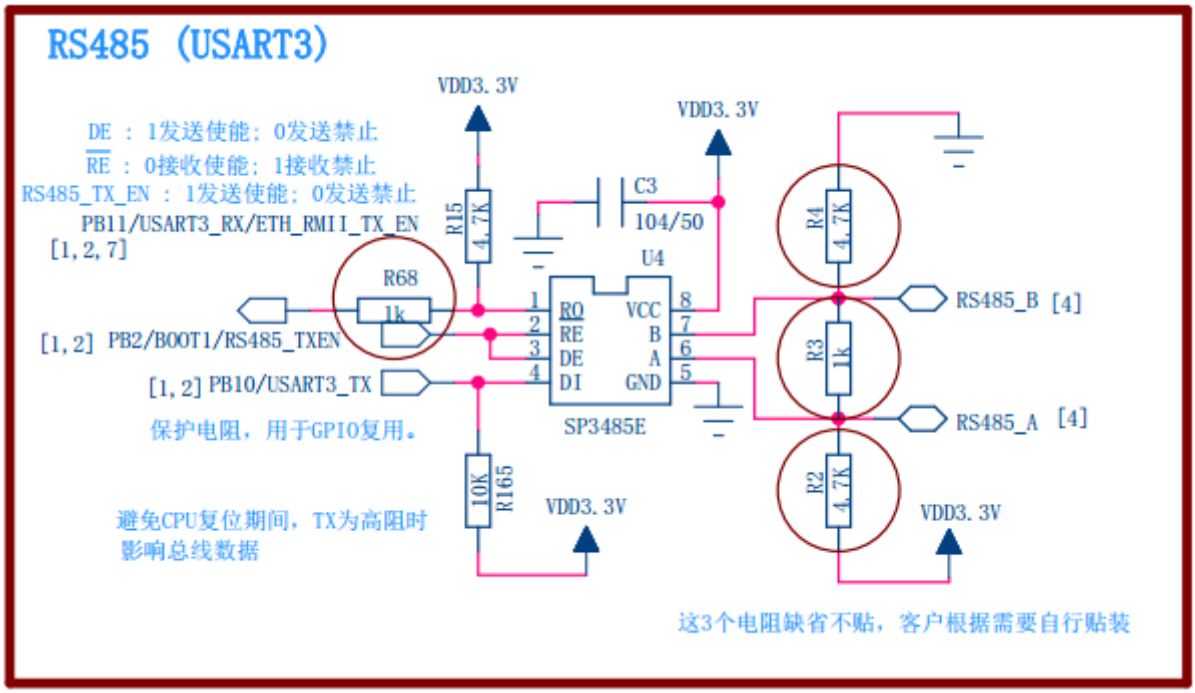
串口 USART1　TX = PA9，　RX = PA10
串口 USART2　TX = PA2，　RX = PA3
串口 USART3　TX = PB10，　RX = PB11
串口 UART4　TX = PC10，　RX = PC11　(和 SDIO 共用)
串口 UART5　TX = PC12，　RX = PD2　(和 SDIO 共用)
串口 USART6　TX = PG14，　RX = PC7
串口 UART7　TX = PB4，　RX = PB3　(和 SPI1/3 共用)
串口 UART8　TX = PJ8，　RX =PJ9　(和 RGB 硬件接口共用)

STM32-V7 开发板使用了 4 个串口设备。

- 串口 1 用于 RS232 接口，很多例子的 pritnf 结果就是输出到串口 1
- 串口 2 用于 GPS
- 串口 3 用于 RS485 接口
- 串口 6 用于 TTL 串口插座，板子上有 GPRS 插座和串口 WIFI 插座。

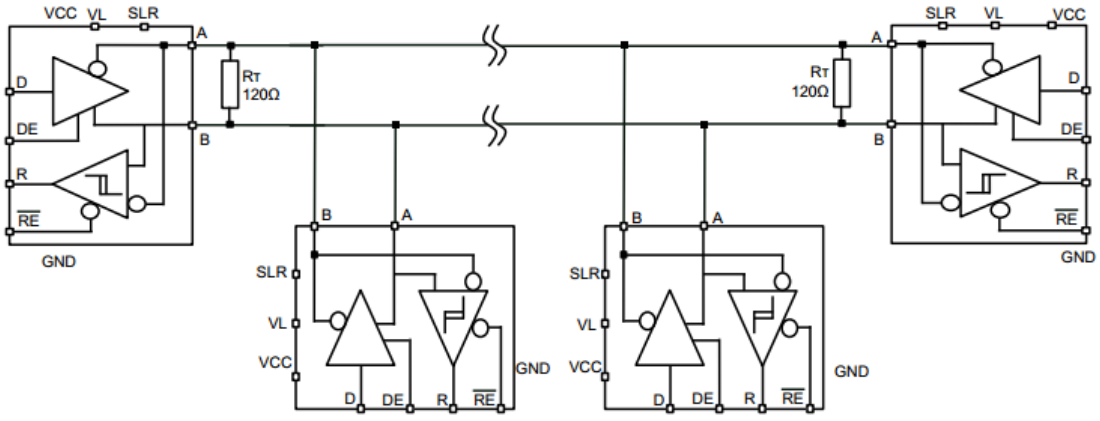
下面是相关的原理图：

串口 3，RS485



关于 485 的 PHY 芯片 SP3485E 要注意以下几个问题：

- SP3485E 允许在同一总线上连接 32 个收发器。
- PB2（RS485_TX_EN）引脚控制收发状态，高电平表示使能发送，低电平表示用于接收。
- 由于 485 的 RO 是输出引脚，如果软件上将 PB2 配置为输出就会冲突（PB2 输出高，RO 输出低就会短路）。加电阻的话，PB2 就可以做其他用途。主要是开发板才这样弄，正式产品不加。
- 电阻 R15 和 R165 的作用是避免 CPU 复位期间，TX 为高阻时影响总线数据。
- 电阻 R4 和 R2 是保证空闲时处于确定的逻辑状态，提供可靠性。
- 电阻 R3 是终端电阻。使用终端电阻是为了阻抗匹配，防止不匹配引起的噪声干扰。一般在 300 米以下，19200bps 不需要接终端电阻。终端电阻要接在传输总线的两端。



31.4 RS485 驱动设计

RS485 的驱动实现是建立在第 31 章讲解的串口 FIFO 基础上，关键的知识点已经在第 31 章节做了详

细讲解，这里把 485 驱动涉及到的两个关键地方做个说明。

31.4.1 RS485 驱动初始化

RS485 驱动的初始化要对收发控制引脚进行配置，这点要注意，对应的代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitUart
*    功能说明: 初始化串口硬件，并对全局变量赋初值.
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitUart(void)
{

    UartVarInit();     /* 必须先初始化全局变量,再配置硬件 */

    InitHardUart();    /* 配置串口的硬件参数(波特率等) */

    RS485_InitTXE();   /* 配置 RS485 芯片的发送使能硬件，配置为推挽输出 */
}
/*
*********************************************************************************************************
*    函 数 名: RS485_InitTXE
*    功能说明: 配置 RS485 发送使能口线 TXE
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void RS485_InitTXE(void)
{
    GPIO_InitTypeDef gpio_init;

    /* 打开 GPIO 时钟 */
    RS485_TXEN_GPIO_CLK_ENABLE();

    /* 配置引脚为推挽输出 */
    gpio_init.Mode = GPIO_MODE_OUTPUT_PP;          /* 推挽输出 */
    gpio_init.Pull = GPIO_NOPULL;                  /* 上下拉电阻不使能 */
    gpio_init.Speed = GPIO_SPEED_FREQ_VERY_HIGH;   /* GPIO 速度等级 */
    gpio_init.Pin = RS485_TXEN_PIN;
    HAL_GPIO_Init(RS485_TXEN_GPIO_PORT, &gpio_init);
}
```

31.4.2 RS485 驱动回调函数初始化

由于 RS485 是半双工，收发要做切换，初始化的时候专门配套了回调函数：

```
/*
*********************************************************************************************************
*    函 数 名: UartVarInit
*    功能说明: 初始化串口相关的变量
```

```
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void UartVarInit(void)
{

#if UART3_FIFO_EN == 1
    g_tUart3.uart = USART3;                     /* STM32 串口设备 */
    g_tUart3.pTxBuf = g_TxBuf3;                 /* 发送缓冲区指针 */
    g_tUart3.pRxBuf = g_RxBuf3;                 /* 接收缓冲区指针 */
    g_tUart3.usTxBufSize = UART3_TX_BUF_SIZE;   /* 发送缓冲区大小 */
    g_tUart3.usRxBufSize = UART3_RX_BUF_SIZE;   /* 接收缓冲区大小 */
    g_tUart3.usTxWrite = 0;                     /* 发送 FIFO 写索引 */
    g_tUart3.usTxRead = 0;                      /* 发送 FIFO 读索引 */
    g_tUart3.usRxWrite = 0;                     /* 接收 FIFO 写索引 */
    g_tUart3.usRxRead = 0;                      /* 接收 FIFO 读索引 */
    g_tUart3.usRxCount = 0;                     /* 接收到的新数据个数 */
    g_tUart3.usTxCount = 0;                     /* 待发送的数据个数 */
    g_tUart3.SendBefor = RS485_SendBefor;       /* 发送数据前的回调函数 */
    g_tUart3.SendOver = RS485_SendOver;         /* 发送完毕后的回调函数 */
    g_tUart3.ReciveNew = RS485_ReciveNew;       /* 接收到新数据后的回调函数 */
    g_tUart3.Sending = 0;                       /* 正在发送中标志 */
#endif

}
```

上面代码中置红的部分是专用于 485 总线的，对应的代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: RS485_SendBefor
*    功能说明: 发送数据前的准备工作。对于 RS485 通信，请设置 RS485 芯片为发送状态，
*              并修改 UartVarInit()中的函数指针等于本函数名，比如 g_tUart2.SendBefor = RS485_SendBefor
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void RS485_SendBefor(void)
{
    RS485_TX_EN();      /* 切换 RS485 收发芯片为发送模式 */
}

/*
*********************************************************************************************************
*    函 数 名: RS485_SendOver
*    功能说明: 发送一串数据结束后的善后处理。对于 RS485 通信，请设置 RS485 芯片为接收状态，
*              并修改 UartVarInit()中的函数指针等于本函数名，比如 g_tUart2.SendOver = RS485_SendOver
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void RS485_SendOver(void)
{
    RS485_RX_EN();      /* 切换 RS485 收发芯片为接收模式 */
}

/*
*********************************************************************************************************
```

```
*    函 数 名: RS485_ReciveNew
*    功能说明: 接收到新的数据
*    形    参: _byte 接收到的新数据
*    返 回 值: 无
*********************************************************************************************************
*/
//extern void MODH_ReciveNew(uint8_t _byte);
void RS485_ReciveNew(uint8_t _byte)
{
//   MODH_ReciveNew(_byte);
}
```

- 函数 RS485_SendBefor

  发送数据前的准备工作。对于 RS485 通信，请设置 RS485 芯片为发送状态。

- 函数 RS485_SendOver

  发送一串数据结束后的善后处理。对于 RS485 通信，请设置 RS485 芯片为接收状态。

- 函数 RS485_ReciveNew

  接收到新的数据，用于 Modbus 通信协议。这里未用到 Modbus，所以将对应的内容注释掉了。

31.4.3 RS485 发送处理

串口数据的发送主要涉及到下面四个函数，调用关系是如下：

RS485_SendStr –> RS485_SendBuf –> comSendBuf -> UartSend

实际应用中，大家调用函数 RS485_SendStr，RS485_SendBuf 或者 comSendBuf 均可。另外特别注意代码中置红的部分，用于设置 485 发送使能。

```
/*
*********************************************************************************************************
*    函 数 名: RS485_SendBuf
*    功能说明: 通过 RS485 芯片发送一串数据。注意，本函数不等待发送完毕。
*    形    参: _ucaBuf : 数据缓冲区
*              _usLen : 数据长度
*    返 回 值: 无
*********************************************************************************************************
*/
void RS485_SendBuf(uint8_t *_ucaBuf, uint16_t _usLen)
{
    comSendBuf(COM3, _ucaBuf, _usLen);
}

/*
*********************************************************************************************************
*    函 数 名: RS485_SendStr
*    功能说明: 向 485 总线发送一个字符串，0 结束。
*    形    参: _pBuf 字符串，0 结束
*    返 回 值: 无
*********************************************************************************************************
*/
void RS485_SendStr(char *_pBuf)
{
    RS485_SendBuf((uint8_t *)_pBuf, strlen(_pBuf));
```

```
}

/*
*********************************************************************************************************
*    函 数 名: comSendBuf
*    功能说明: 向串口发送一组数据。数据放到发送缓冲区后立即返回，由中断服务程序在后台完成发送
*    形    参: _ucPort: 端口号(COM1 - COM8)
*              _ucaBuf: 待发送的数据缓冲区
*              _usLen : 数据长度
*    返 回 值: 无
*********************************************************************************************************
*/
void comSendBuf(COM_PORT_E _ucPort, uint8_t *_ucaBuf, uint16_t _usLen)
{
    UART_T *pUart;

    pUart = ComToUart(_ucPort);
    if (pUart == 0)
    {
        return;
    }

    if (pUart->SendBefor != 0)
    {
        pUart->SendBefor();         /* 如果是 RS485 通信，可以在这个函数中将 RS485 设置为发送模式 */
    }

    UartSend(pUart, _ucaBuf, _usLen);
}

/*
*********************************************************************************************************
*    函 数 名: UartSend
*    功能说明: 填写数据到 UART 发送缓冲区,并启动发送中断。中断处理函数发送完毕后，自动关闭发送中断
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void UartSend(UART_T *_pUart, uint8_t *_ucaBuf, uint16_t _usLen)
{
    uint16_t i;

    for (i = 0; i < _usLen; i++)
    {
        /* 如果发送缓冲区已经满了，则等待缓冲区空 */
        while (1)
        {
            __IO uint16_t usCount;

            DISABLE_INT();
            usCount = _pUart->usTxCount;
            ENABLE_INT();

            if (usCount < _pUart->usTxBufSize)
            {
                break;
            }
            else if(usCount == _pUart->usTxBufSize)/* 数据已填满缓冲区 */
```

```
            {
                if((_pUart->uart->CR1 & USART_CR1_TXEIE) == 0)
                {
                    SET_BIT(_pUart->uart->CR1, USART_CR1_TXEIE);
                }
            }
        }

        /* 将新数据填入发送缓冲区 */
        _pUart->pTxBuf[_pUart->usTxWrite] = _ucaBuf[i];

        DISABLE_INT();
        if (++_pUart->usTxWrite >= _pUart->usTxBufSize)
        {
            _pUart->usTxWrite = 0;
        }
        _pUart->usTxCount++;
        ENABLE_INT();
    }

    SET_BIT(_pUart->uart->CR1, USART_CR1_TXEIE);   /* 使能发送中断（缓冲区空） */
}
```

函数 UartSend 的作用就是把要发送的数据填到发送缓冲区里面，并使能发送空中断。

- 如果要发送的数据没有超过发送缓冲区大小，实现起来还比较容易，直接把数据填到 FIFO 里面，并使能发送空中断即可。
- 如果超过了 FIFO 大小，就需要等待有空间可用，针对这种情况有个重要的知识点，就是当缓冲刚刚填满的时候要判断发送空中断是否开启了，如果填满了还没有开启，就会卡死在 while 循环中，所以多了一个刚填满时的判断，填满了还没有开启发送空中断，要开启下。

注意：由于函数 UartSend 做了 static 作用域限制，仅可在 bsp_uart_fifo.c 文件中调用。函数 RS485_SendStr，RS485_SendBuf 或者 comSendBuf 是供用户调用的。

31.4.4 RS485 数据接收

下面我们再来看看接收的函数：

```
/*
*********************************************************************************************************
*    函 数 名: comGetChar
*    功能说明: 从接收缓冲区读取 1 字节，非阻塞。无论有无数据均立即返回。
*    形    参: _ucPort: 端口号(COM1 - COM8)
*              _pByte: 接收到的数据存放在这个地址
*    返 回 值: 0 表示无数据, 1 表示读取到有效字节
*********************************************************************************************************
*/
uint8_t comGetChar(COM_PORT_E _ucPort, uint8_t *_pByte)
{
    UART_T *pUart;

    pUart = ComToUart(_ucPort);
```

```
    if (pUart == 0)
    {
        return 0;
    }

    return UartGetChar(pUart, _pByte);
}

/*
*********************************************************************************************************
*    函 数 名: UartGetChar
*    功能说明: 从串口接收缓冲区读取 1 字节数据 （用于主程序调用）
*    形    参: _pUart : 串口设备
*              _pByte : 存放读取数据的指针
*    返 回 值: 0 表示无数据  1 表示读取到数据
*********************************************************************************************************
*/
static uint8_t UartGetChar(UART_T *_pUart, uint8_t *_pByte)
{
    uint16_t usCount;

    /* usRxWrite 变量在中断函数中被改写，主程序读取该变量时，必须进行临界区保护 */
    DISABLE_INT();
    usCount = _pUart->usRxCount;
    ENABLE_INT();

    /* 如果读和写索引相同，则返回 0 */
    //if (_pUart->usRxRead == usRxWrite)
    if (usCount == 0)  /* 已经没有数据 */
    {
        return 0;
    }
    else
    {
        *_pByte = _pUart->pRxBuf[_pUart->usRxRead];        /* 从串口接收 FIFO 取 1 个数据 */

        /* 改写 FIFO 读索引 */
        DISABLE_INT();
        if (++_pUart->usRxRead >= _pUart->usRxBufSize)
        {
            _pUart->usRxRead = 0;
        }
        _pUart->usRxCount--;
        ENABLE_INT();
        return 1;
    }
}
```

函数 comGetChar 是专门供用户调用的，用于从接收 FIFO 中读取 1 个数据。具体代码的实现也比较好理解，主要是接收 FIFO 的调整。

注意：由于函数 UartGetChar 做了 static 作用域限制，仅可在 bsp_uart_fifo.c 文件中调用。

31.4.5 RS485 驱动中断服务程序的处理

串口中断服务程序是实现 RS485 驱动的关键部分，主要实现如下三个功能：

- 收到新的数据后，会将数据压入 RX_FIFO。
- 检测到发送缓冲区空后，会从 TX_FIFO 中取下一个数据并发送。
- 对于 RS485 半双工串口，发送前会设置一个 GPIO=1 控制 RS485 收发器进入发送状态，当最后一个字节的最后一个 bit 传送完毕后，设置这个 GPIO=0 让 RS485 收发器进入接收状态。

下面我们分析一下串口中断处理的完整过程。

当产生串口中断后，CPU 会查找中断向量表，获得中断服务程序的入口地址。入口函数为 USART1_IRQHandler，这个函数在启动文件 startup_stm32h743xx.s 汇编代码中已经有实现。我们在 c 代码中需要重写一个同样名字的函数就可以重载它。如果不重载，启动文件中缺省的中断服务程序就是一个死循环，等于 while(1);

我们将串口中断服务程序放在 bsp_uart_fifo.c 文件，没有放到 stm32h7xx_it.c。当应用不需要串口功能时，直接从工程中删除 bsp_uart_fifo.c 接口，不必再去整理 stm32h7xx_it.c 这个文件。下面展示的代码是 8 个串口的中断服务程序，RS485 用的 USART3。

```
#if UART1_FIFO_EN == 1
void USART1_IRQHandler(void)
{
    UartIRQ(&g_tUart1);
}
#endif

#if UART2_FIFO_EN == 1
void USART2_IRQHandler(void)
{
    UartIRQ(&g_tUart2);
}
#endif

#if UART3_FIFO_EN == 1
void USART3_IRQHandler(void)
{
    UartIRQ(&g_tUart3);
}
#endif

#if UART4_FIFO_EN == 1
void UART4_IRQHandler(void)
{
    UartIRQ(&g_tUart4);
}
#endif

#if UART5_FIFO_EN == 1
void UART5_IRQHandler(void)
{
```

```
    UartIRQ(&g_tUart5);
}
#endif

#if UART6_FIFO_EN == 1
void USART6_IRQHandler(void)
{
    UartIRQ(&g_tUart6);
}
#endif

#if UART7_FIFO_EN == 1
void UART7_IRQHandler(void)
{
    UartIRQ(&g_tUart7);
}
#endif

#if UART8_FIFO_EN == 1
void UART8_IRQHandler(void)
{
    UartIRQ(&g_tUart8);
}
#endif
```

大家可以看到，这 8 个中断服务程序都调用了同一个处理函数 UartIRQ。我们只需要调通一个串口 FIFO 驱动，那么其他的串口驱动也就都通了。

下面，我们来看看 UartIRQ 函数的实现代码。

```
/*
*********************************************************************************************************
*    函 数 名: UartIRQ
*    功能说明: 供中断服务程序调用，通用串口中断处理函数
*    形    参: _pUart : 串口设备
*    返 回 值: 无
*********************************************************************************************************
*/
static void UartIRQ(UART_T *_pUart)
{
    uint32_t isrflags   = READ_REG(_pUart->uart->ISR);
    uint32_t crlits     = READ_REG(_pUart->uart->CR1);
    uint32_t cr3its     = READ_REG(_pUart->uart->CR3);

    /* 处理接收中断  */
    if ((isrflags & USART_ISR_RXNE) != RESET)
    {
        /* 从串口接收数据寄存器读取数据存放到接收 FIFO */
        uint8_t ch;

        ch = READ_REG(_pUart->uart->RDR);               /* 读串口接收数据寄存器 */
        _pUart->pRxBuf[_pUart->usRxWrite] = ch;         /* 填入串口接收 FIFO */
        if (++_pUart->usRxWrite >= _pUart->usRxBufSize) /* 接收 FIFO 的写指针+1 */
        {
            _pUart->usRxWrite = 0;
        }
        if (_pUart->usRxCount < _pUart->usRxBufSize)    /* 统计未处理的字节个数 */
        {
            _pUart->usRxCount++;
```

```
        }

        /* 回调函数,通知应用程序收到新数据,一般是发送 1 个消息或者设置一个标记 */
        //if (_pUart->usRxWrite == _pUart->usRxRead)
        //if (_pUart->usRxCount == 1)
        {
            if (_pUart->ReciveNew)
            {
                _pUart->ReciveNew(ch); /* 比如，交给 MODBUS 解码程序处理字节流 */
            }
        }
    }

    /* 处理发送缓冲区空中断 */
    if ( ((isrflags & USART_ISR_TXE) != RESET) && (crlits & USART_CR1_TXEIE) != RESET)
    {
        //if (_pUart->usTxRead == _pUart->usTxWrite)
        if (_pUart->usTxCount == 0)  /* 发送缓冲区已无数据可取 */
        {
        /* 发送缓冲区的数据已取完时， 禁止发送缓冲区空中断 （注意：此时最后 1 个数据还未真正发送完毕）*/
            //USART_ITConfig(_pUart->uart, USART_IT_TXE, DISABLE);
            CLEAR_BIT(_pUart->uart->CR1, USART_CR1_TXEIE);

            /* 使能数据发送完毕中断 */
            //USART_ITConfig(_pUart->uart, USART_IT_TC, ENABLE);
            SET_BIT(_pUart->uart->CR1, USART_CR1_TCIE);
        }
        Else  /*  还有数据等待发送 */
        {
            _pUart->Sending = 1;

            /* 从发送 FIFO 取 1 个字节写入串口发送数据寄存器 */
            //USART_SendData(_pUart->uart, _pUart->pTxBuf[_pUart->usTxRead]);
            _pUart->uart->TDR = _pUart->pTxBuf[_pUart->usTxRead];
            if (++_pUart->usTxRead >= _pUart->usTxBufSize)
            {
                _pUart->usTxRead = 0;
            }
            _pUart->usTxCount--;
        }

    }
    /* 数据 bit 位全部发送完毕的中断 */
    if (((isrflags & USART_ISR_TC) != RESET) && ((crlits & USART_CR1_TCIE) != RESET))
    {
        //if (_pUart->usTxRead == _pUart->usTxWrite)
        if (_pUart->usTxCount == 0)
        {
            /* 如果发送 FIFO 的数据全部发送完毕，禁止数据发送完毕中断 */
            //USART_ITConfig(_pUart->uart, USART_IT_TC, DISABLE);
            CLEAR_BIT(_pUart->uart->CR1, USART_CR1_TCIE);

            /* 回调函数, 一般用来处理 RS485 通信，将 RS485 芯片设置为接收模式，避免抢占总线 */
            if (_pUart->SendOver)
            {
                _pUart->SendOver();
            }
```

```
            _pUart->Sending = 0;
        }
        else
        {
            /* 正常情况下，不会进入此分支 */

            /* 如果发送 FIFO 的数据还未完毕，则从发送 FIFO 取 1 个数据写入发送数据寄存器 */
            //USART_SendData(_pUart->uart, _pUart->pTxBuf[_pUart->usTxRead]);
            _pUart->uart->TDR = _pUart->pTxBuf[_pUart->usTxRead];
            if (++_pUart->usTxRead >= _pUart->usTxBufSize)
            {
                _pUart->usTxRead = 0;
            }
            _pUart->usTxCount--;
        }
    }

    /* 清除中断标志 */
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_PEF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_FEF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_NEF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_OREF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_IDLEF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_TCF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_LBDF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_CTSF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_CMF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_WUF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_TXFECF);
}
```

中断服务程序的处理主要分为两部分，接收数据的处理和发送数据的处理，详情看程序注释即可，已经比较详细，下面重点把思路说一下：

- 接收数据处理

接收数据的处理是判断 ISR 寄存器的 USART_ISR_RXNE 标志是否置位，如果置位表示 RDR 接收寄存器已经存入数据。然后将数据读入到接收 FIFO 空间。

特别注意里面的 ReciveNew 处理，这个在 Modbus 协议里面要用到。

- 发送数据处理

发送数据主要是发送空中断 TEX 和发送完成中断 TC 的处理，当 TXE=1 时，只是表示发送数据寄存器为空了，此时可以填充下一个准备发送的数据了。当为 TDR 发送寄存器赋值后，硬件启动发送，等所有的 bit 传送完毕后，TC 标志设置为 1。如果是 RS232 全双工通信，可以只用 TXE 标志控制发送过程。如果是 RS485 半双工通通信，就需要利用 TC 标志了，因为在最后一个 bit 传送完毕后，需要设置 RS485 收发器进入到接收状态。

31.5 RS485 板级支持包（bsp_uart_fifo.c)

串口驱动文件 bsp_uart_fifo.c 主要实现了如下几个 API 供用户调用：

- bsp_InitUart

- comSendBuf
- comSendChar
- comGetChar

31.5.1 函数 bsp_InitUart

函数原型：

```
void bsp_InitUart(void)
```

函数描述：

此函数主要用于串口的初始化，使用所有其它 API 之前，务必优先调用此函数。

使用举例：

串口的初始化函数在 bsp.c 文件的 bsp_Init 函数里面调用。

31.5.2 函数 comSendBuf

函数原型：

```
void comSendBuf(COM_PORT_E _ucPort, uint8_t *_ucaBuf, uint16_t _usLen);
```

函数描述：

此函数用于向串口发送一组数据，非阻塞方式，数据放到发送缓冲区后立即返回，由中断服务程序在后台完成发送。

函数参数：

- 第 1 个参数_ucPort 是端口号，范围 COM1 - COM8。
- 第 2 个参数_ucaBuf 是待发送的数据缓冲区地址。
- 第 3 个参数_usLen 是要发送数据的字节数。

注意事项：

- 此函数的解读在第 30 章 30.3.5 小节。
- 发送的数据最好不要超过 bsp_uart_fifo.h 文件中定义的发送缓冲区大小，从而实现最优的工作方式。因为超过后需要在发送函数等待有发送空间可用。

使用举例：

调用此函数前，务必优先调用函数 bsp_InitUart 进行初始化。

```
const char buf1[] = "接收到串口命令 1\r\n";

comSendBuf(COM1, (uint8_t *)buf1, strlen(buf1));
```

31.5.3 函数 RS485_SendBuf

函数原型：

```
void RS485_SendBuf(uint8_t *_ucaBuf, uint16_t _usLen)
```

函数描述：

此函数用于向 RS485 总线发送一组数据，非阻塞方式，数据放到发送缓冲区后立即返回，由中断服务程序在后台完成发送。此函数是通过调用函数 comSendBuf 实现的。

函数参数：

- 第 1 个参数_ucaBuf 是数据缓冲区。
- 第 2 个参数_usLen 是数据长度。

注意事项：

调用此函数前，务必优先调用函数 bsp_InitUart 进行初始化。

31.5.4 函数 RS485_SendStr

函数原型：

```
void RS485_SendStr(char *_pBuf)
```

函数描述：

此函数用于向 RS485 总线发送一个字符串，非阻塞方式，数据放到发送缓冲区后立即返回，由中断服务程序在后台完成发送。此函数是通过调用函数 RS485_SendBuf 实现的。

函数参数：

- 第 1 个参数_pBuf 是字符串地址，0 结束。

注意事项：

调用此函数前，务必优先调用函数 bsp_InitUart 进行初始化。

31.5.5 函数 comGetChar

函数原型：

```
uint8_t comGetChar(COM_PORT_E _ucPort, uint8_t *_pByte)
```

函数描述：

此函数用于从接收缓冲区读取 1 字节，非阻塞。无论有无数据均立即返回。

函数参数：

- 第 1 个参数_ucPort 是端口号，范围 COM1 - COM8。
- 第 2 个参数_pByte 用于存放接收到的数据。
- 返回值，返回 0 表示无数据，1 表示读取到有效字节。

注意事项：

- 此函数的解读在第 30 章 30.3.6 小节。

使用举例：

调用此函数前，务必优先调用函数 bsp_InitUart 进行初始化。

比如从串口 1 读取一个字符就是：comGetChar(COM1, &read)

31.6 RS485 驱动移植和使用

RS485 移植步骤如下：

- 第 1 步：复制 bsp_uart_fifo.h 和 bsp_uart_fifo.c 到自己的工程目录，并添加到工程里面。
- 第 2 步：根据 485 要使用的串口和收发缓冲大小，修改下面的宏定义即可。

```
#define  UART1_FIFO_EN 1
#define  UART2_FIFO_EN 0
#define  UART3_FIFO_EN 0
#define  UART4_FIFO_EN 0
#define  UART5_FIFO_EN 0
#define  UART6_FIFO_EN 0
#define  UART7_FIFO_EN 0
#define  UART8_FIFO_EN 0

/* 定义串口波特率和 FIFO 缓冲区大小，分为发送缓冲区和接收缓冲区，支持全双工 */
#if UART1_FIFO_EN == 1
    #define UART1_BAUD          115200
    #define UART1_TX_BUF_SIZE   1*1024
    #define UART1_RX_BUF_SIZE   1*1024
#endif

#if UART2_FIFO_EN == 1
    #define UART2_BAUD          9600
    #define UART2_TX_BUF_SIZE   10
    #define UART2_RX_BUF_SIZE   2*1024
#endif

#if UART3_FIFO_EN == 1
    #define UART3_BAUD          9600
    #define UART3_TX_BUF_SIZE   1*1024
    #define UART3_RX_BUF_SIZE   1*1024
#endif

#if UART4_FIFO_EN == 1
    #define UART4_BAUD          115200
    #define UART4_TX_BUF_SIZE   1*1024
    #define UART4_RX_BUF_SIZE   1*1024
#endif

#if UART5_FIFO_EN == 1
    #define UART5_BAUD          115200
    #define UART5_TX_BUF_SIZE   1*1024
    #define UART5_RX_BUF_SIZE   1*1024
#endif

#if UART6_FIFO_EN == 1
    #define UART6_BAUD          115200
    #define UART6_TX_BUF_SIZE   1*1024
    #define UART6_RX_BUF_SIZE   1*1024
#endif

#if UART7_FIFO_EN == 1
```

```
    #define UART7_BAUD          115200
    #define UART7_TX_BUF_SIZE   1*1024
    #define UART7_RX_BUF_SIZE   1*1024
#endif

#if UART8_FIFO_EN == 1
    #define UART8_BAUD          115200
    #define UART8_TX_BUF_SIZE   1*1024
    #define UART8_RX_BUF_SIZE   1*1024
#endif
```

◆ 第 3 步：配置 485 PHY 的收发切换引脚

```
/* PB2 控制 RS485 芯片的发送使能 */
#define RS485_TXEN_GPIO_CLK_ENABLE()      __HAL_RCC_GPIOB_CLK_ENABLE()
#define RS485_TXEN_GPIO_PORT              GPIOB
#define RS485_TXEN_PIN                    GPIO_PIN_2

#define RS485_RX_EN()   RS485_TXEN_GPIO_PORT->BSRRH = RS485_TXEN_PIN
#define RS485_TX_EN()   RS485_TXEN_GPIO_PORT->BSRRL = RS485_TXEN_PIN
```

◆ 第 4 步：根据使用的串口，配置下面函数的几个参数成员，下面是以串口 3 为例说明（置红部分）：

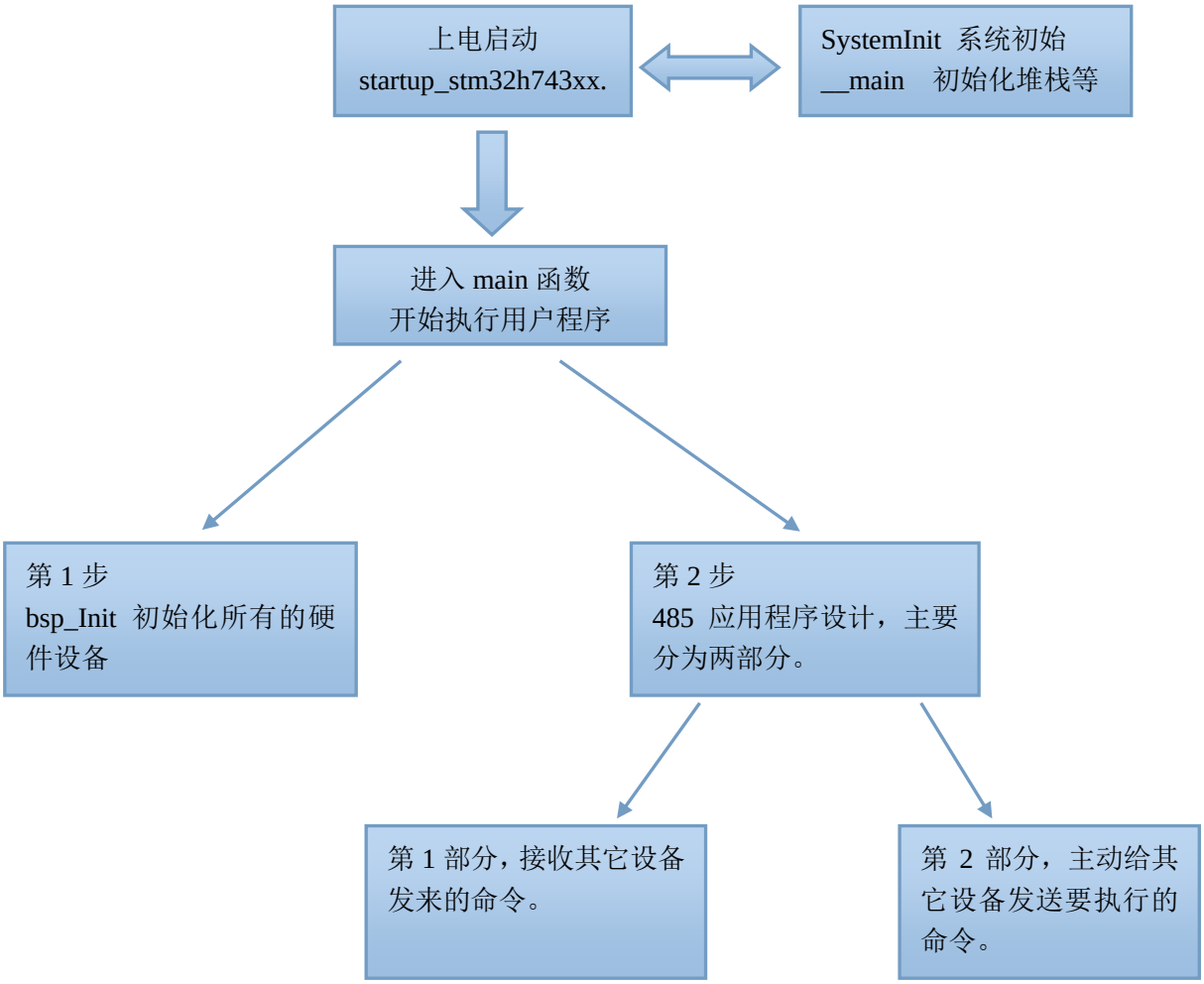
```
static void UartVarInit(void)
{

#if UART3_FIFO_EN == 1
    g_tUart3.uart = USART3;                     /* STM32 串口设备 */
    g_tUart3.pTxBuf = g_TxBuf3;                 /* 发送缓冲区指针 */
    g_tUart3.pRxBuf = g_RxBuf3;                 /* 接收缓冲区指针 */
    g_tUart3.usTxBufSize = UART3_TX_BUF_SIZE;   /* 发送缓冲区大小 */
    g_tUart3.usRxBufSize = UART3_RX_BUF_SIZE;   /* 接收缓冲区大小 */
    g_tUart3.usTxWrite = 0;                     /* 发送 FIFO 写索引 */
    g_tUart3.usTxRead = 0;                      /* 发送 FIFO 读索引 */
    g_tUart3.usRxWrite = 0;                     /* 接收 FIFO 写索引 */
    g_tUart3.usRxRead = 0;                      /* 接收 FIFO 读索引 */
    g_tUart3.usRxCount = 0;                     /* 接收到的新数据个数 */
    g_tUart3.usTxCount = 0;                     /* 待发送的数据个数 */
    g_tUart3.SendBefor = RS485_SendBefor;       /* 发送数据前的回调函数 */
    g_tUart3.SendOver = RS485_SendOver;         /* 发送完毕后的回调函数 */
    g_tUart3.ReciveNew = RS485_ReciveNew;       /* 接收到新数据后的回调函数 */
    g_tUart3.Sending = 0;                       /* 正在发送中标志 */
#endif
}
```

◆ 第 5 步：这几个驱动文件主要用到 HAL 库的 GPIO 和串口驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。

◆ 第 6 步，应用方法看本章节配套例子即可。

31.7 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED 和串口。
  - 第 2 步，485 应用程序设计部分，具体分为两个部分，接收命令和发送命令。

31.8 实验例程说明（MDK）

配套例子：

V7-016_RS485 多机通讯

实验目的：

1. 学习 RS485 多机通讯的实现。

实验内容：

1. 由于通信距离较短，SP3485E 芯片上缺省未贴的电阻不需要贴上，大家可以根据需要贴上电阻做测试。

2. 本例子支持多个 485 节点，不需要设置主设备和从设备，所有节点下载此程序即可。
3. 开发板的 485-A 端子连接到一起，485-B 端子连接到一起，具体连接看工程 Doc 文件夹中的截图。

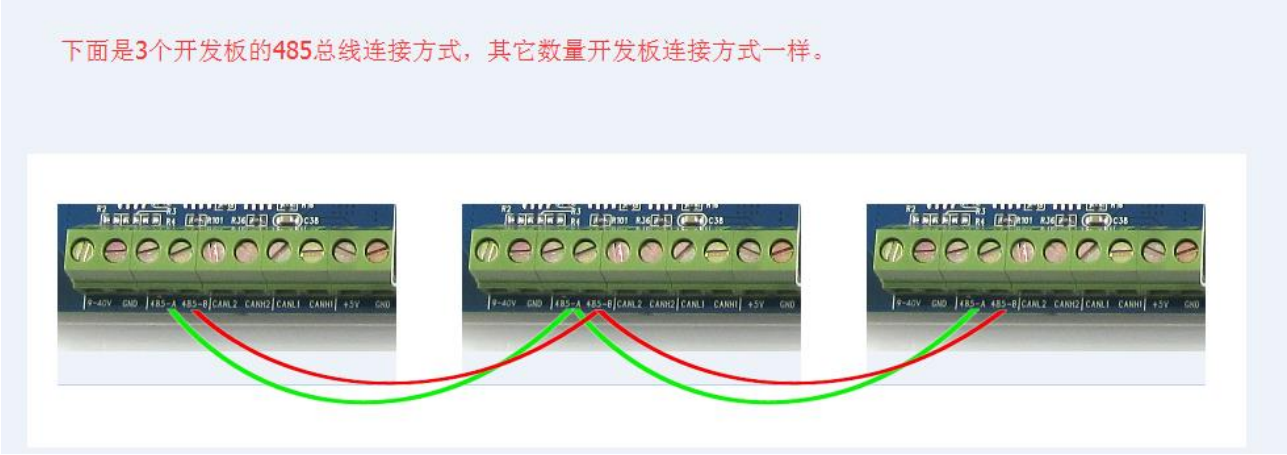
实验操作：

1. 按下开发板上的 K1 键点亮 LED1，松开熄灭 LED1，同时打印按键事件到串口 1。485 总线上的其它开发板做相同的动作。
2. 按下开发板上的 K2 键，启动 50ms 的自动重装定时器，每隔 50ms 翻转 LED2，并向 485 总线上的其它开发板发送按键 K2 按下消息，从而也实现每隔 50ms 翻转 LED2。
3. 按下开发板上的 K3 按键，停止 K2 按键启动的 50ms 自动重载定时器，485 总线上的其它开发板做相同的动作。
4. 按下开发板上的摇杆（上下左右，OK 共 5 种），会通过串口 1 打印摇杆事件。485 总线上的其它开发板做相同的动作。

注意事项：

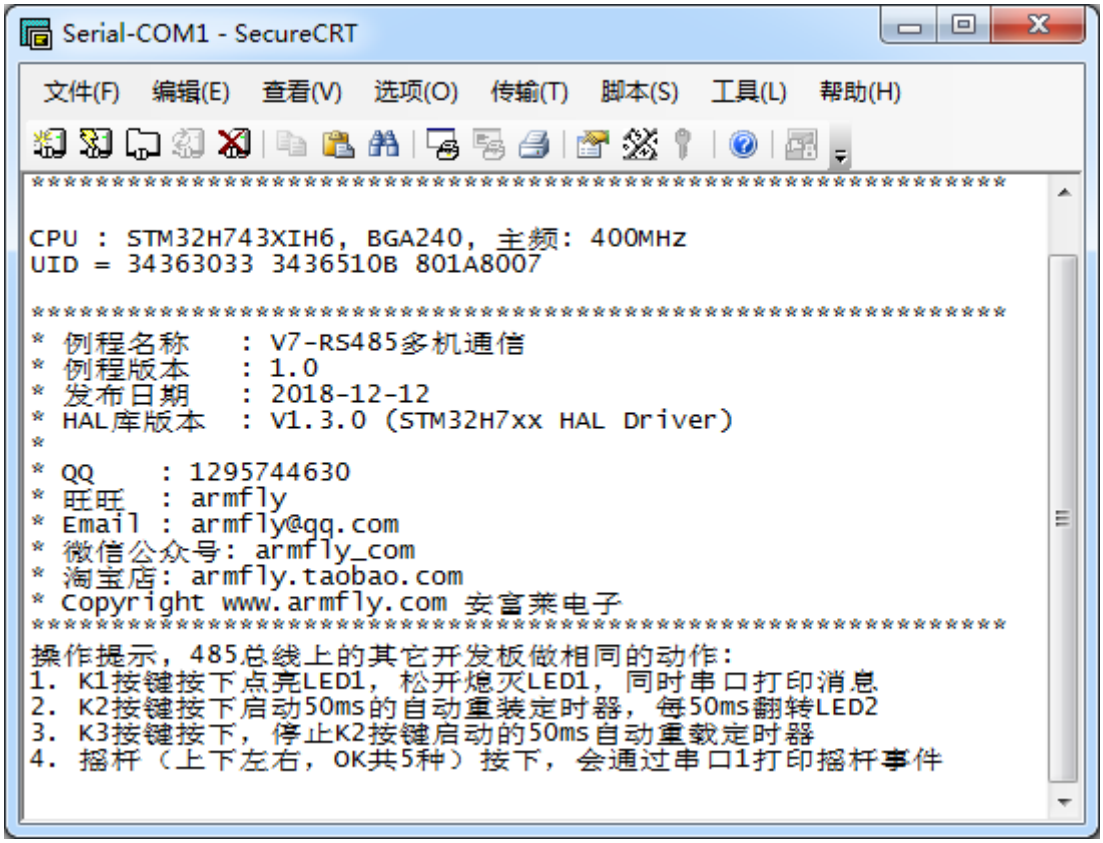
1. 将板子 J5 处的跳线帽拔掉。

多机接线效果：



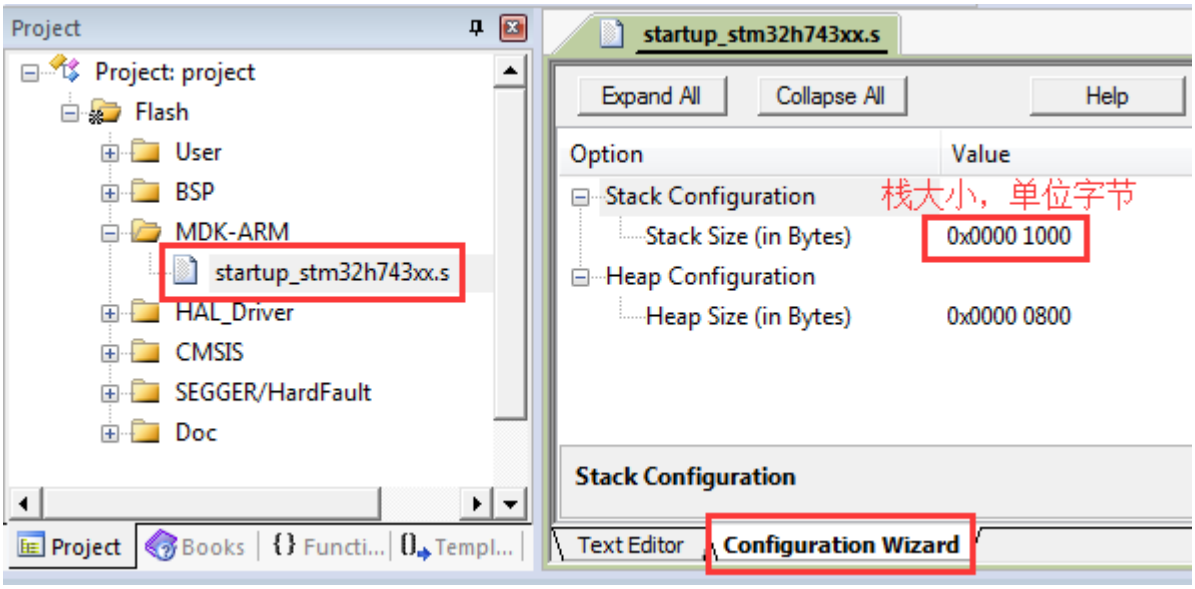
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

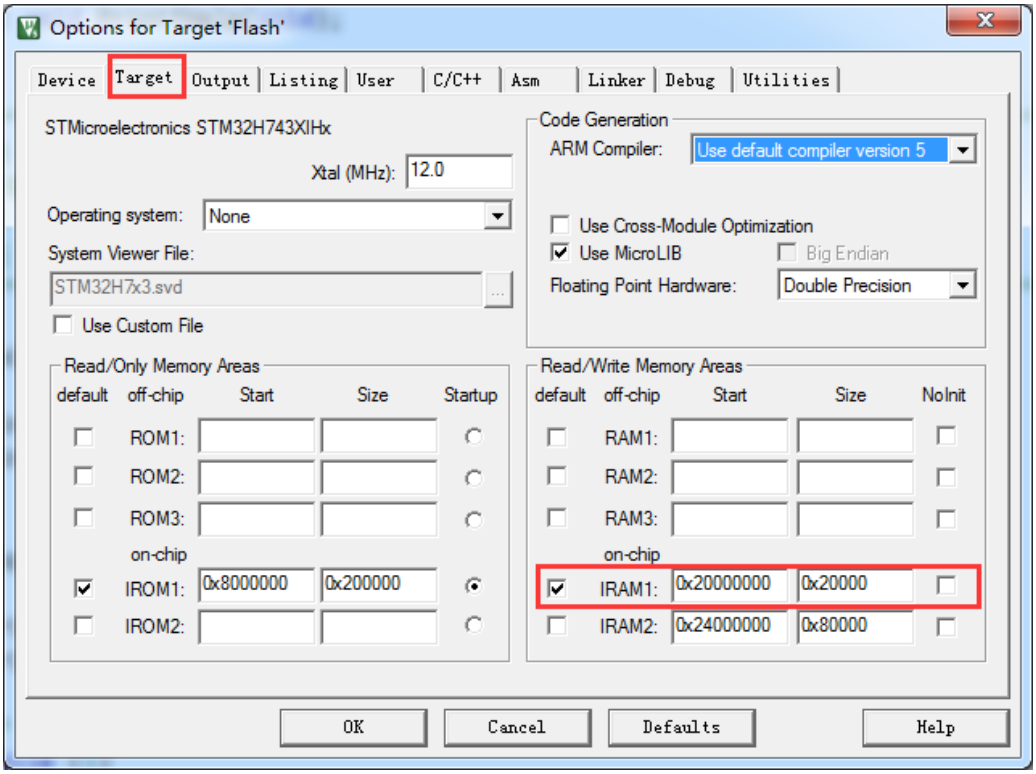


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
```

```
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主功能的实现主要分为两部分：

- 获取其它 RS485 设备发来的命令，并执行相应功能。

  通过函数 comGetChar(COM3, &ucDataTravel)以查询、非阻塞方式获取其它开发板发来的数据。

- 根据不同的按键消息，向其它 485 设备发送命令

  通过函数 comSendChar(COM3, ucDataTravel)以非阻塞方式向其它 485 设备发送命令，以此来执行同样的功能。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */
    uint8_t ucDataTravel;   /* 发送变量 */
```

```
    uint8_t ucDataRec;      /* 接收变量 */

    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 获取其它开发板通过 485 总线发来的数据 */
        if(comGetChar(COM3, &ucDataRec))
        {
            switch (ucDataRec)
            {
                case KEY_DOWN_K1:           /* 获得 K1 键按下消息 */
                    bsp_LedOn(1);
                    printf("K1 键按下, LED1 点亮\r\n");
                    break;

                case KEY_UP_K1:             /* 获得 K1 键释放消息 */
                    bsp_LedOff(1);
                    printf("K1 键弹起, LED1 熄灭\r\n");
                    break;

                case KEY_DOWN_K2:           /* 获得 K1 键按下消息 */
                    bsp_LedToggle(2);
                    break;

                case JOY_DOWN_U:            /* 获得摇杆 UP 键按下 */
                    printf("摇杆上键按下\r\n");
                    break;

                case JOY_DOWN_D:            /* 获得摇杆 DOWN 键按下 */
                    printf("摇杆下键按下\r\n");
                    break;

                case JOY_DOWN_L:            /* 获得摇杆 LEFT 键按下 */
                    printf("摇杆左键按下\r\n");
                    break;

                case JOY_DOWN_R:            /* 获得摇杆 RIGHT 键按下 */
                    printf("摇杆右键按下\r\n");
                    break;

                case JOY_DOWN_OK:           /* 获得摇杆 OK 键按下 */
                    printf("摇杆 OK 键按下\r\n");
                    break;

                case JOY_UP_OK:                 /* 获得摇杆 OK 键弹起 */
                    printf("摇杆 OK 键弹起\r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
```

```
        }
    }

    /* 判断定时器超时时间 */
    if (bsp_CheckTimer(0))
    {
        /* 每隔 50ms 进来一次 */
        bsp_LedToggle(2);
        /* 向其它开发板发送按键 K2 按下的消息 */
        ucDataTravel = KEY_DOWN_K2;
        comSendChar(COM3, ucDataTravel);
    }

    /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
    ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
    if (ucKeyCode != KEY_NONE)
    {
        switch (ucKeyCode)
        {
            case KEY_DOWN_K1:           /* K1 键按下 */
                ucDataTravel = KEY_DOWN_K1;
                comSendChar(COM3, ucDataTravel);
                bsp_LedOn(1);
                printf("K1 键按下, LED1 点亮\r\n");
                break;

            case KEY_UP_K1:             /* K1 键弹起 */
                ucDataTravel = KEY_UP_K1;
                comSendChar(COM3, ucDataTravel);
                bsp_LedOff(1);
                printf("K1 键弹起, LED1 熄灭\r\n");
                break;

            case KEY_DOWN_K2:           /* K2 键按下 */
                bsp_StartAutoTimer(0, 50);  /* 启动 1 个 50ms 的自动重装的定时器 */
                break;

            case KEY_DOWN_K3:           /* K3 键按下 */
                bsp_StopTimer(0);       /* 停止自动重装的定时器 */
                break;

            case JOY_DOWN_U:            /* 摇杆 UP 键按下 */
                ucDataTravel = JOY_DOWN_U;
                comSendChar(COM3, ucDataTravel);
                printf("摇杆上键按下\r\n");
                break;

            case JOY_DOWN_D:            /* 摇杆 DOWN 键按下 */
                ucDataTravel = JOY_DOWN_D;
                comSendChar(COM3, ucDataTravel);
                printf("摇杆下键按下\r\n");
                break;

            case JOY_DOWN_L:            /* 摇杆 LEFT 键按下 */
                ucDataTravel = JOY_DOWN_L;
                comSendChar(COM3, ucDataTravel);
                printf("摇杆左键按下\r\n");
                break;
```

```
                case JOY_DOWN_R:            /* 摇杆 RIGHT 键按下 */
                    ucDataTravel = JOY_DOWN_R;
                    comSendChar(COM3, ucDataTravel);
                    printf("摇杆右键按下\r\n");
                    break;

                case JOY_DOWN_OK:           /* 摇杆 OK 键按下 */
                    ucDataTravel = JOY_DOWN_OK;
                    comSendChar(COM3, ucDataTravel);
                    printf("摇杆 OK 键按下\r\n");
                    break;

                case JOY_UP_OK:                 /* 摇杆 OK 键弹起 */
                    ucDataTravel = JOY_UP_OK;
                    comSendChar(COM3, ucDataTravel);
                    printf("摇杆 OK 键弹起\r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

31.9 实验例程说明（IAR）

配套例子：

V7-016_RS485 多机通讯

实验目的：

1. 学习 RS485 多机通讯的实现。

实验内容：

1. 由于通信距离较短，SP3485E 芯片上缺省未贴的电阻不需要贴上，大家可以根据需要贴上电阻做测试。
2. 本例子支持多个 485 节点，不需要设置主设备和从设备，所有节点下载此程序即可。
3. 开发板的 485-A 端子连接到一起，485-B 端子连接到一起，具体连接看工程 Doc 文件夹中的截图。

实验操作：

1. 按下开发板上的 K1 键点亮 LED1，松开熄灭 LED1，同时打印按键事件到串口 1。485 总线上的其它开发板做相同的动作。
2. 按下开发板上的 K2 键，启动 50ms 的自动重装定时器，每隔 50ms 翻转 LED2，并向 485 总线上的其它开发板发送按键 K2 按下消息，从而也实现每隔 50ms 翻转 LED2。
3. 按下开发板上的 K3 按键，停止 K2 按键启动的 50ms 自动重载定时器，485 总线上的其它开发

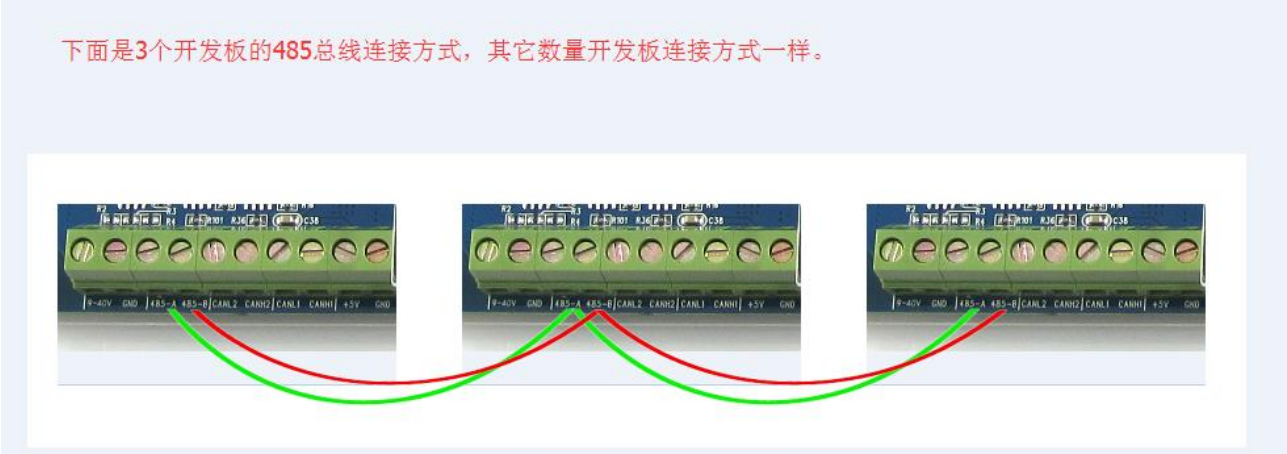
板做相同的动作。

4. 按下开发板上的摇杆（上下左右，OK 共 5 种），会通过串口 1 打印摇杆事件。485 总线上的其它开发板做相同的动作。

注意事项：

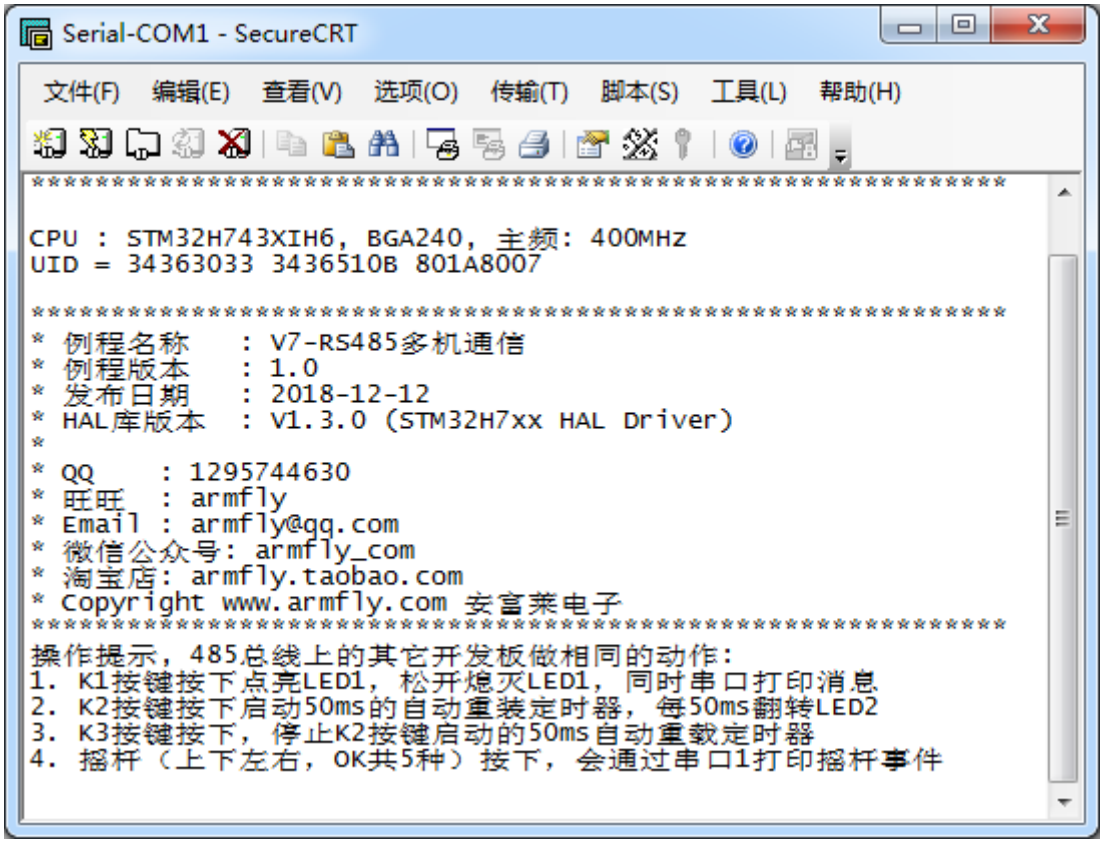
1. 将板子 J5 处的跳线帽拔掉。

多机接线效果：



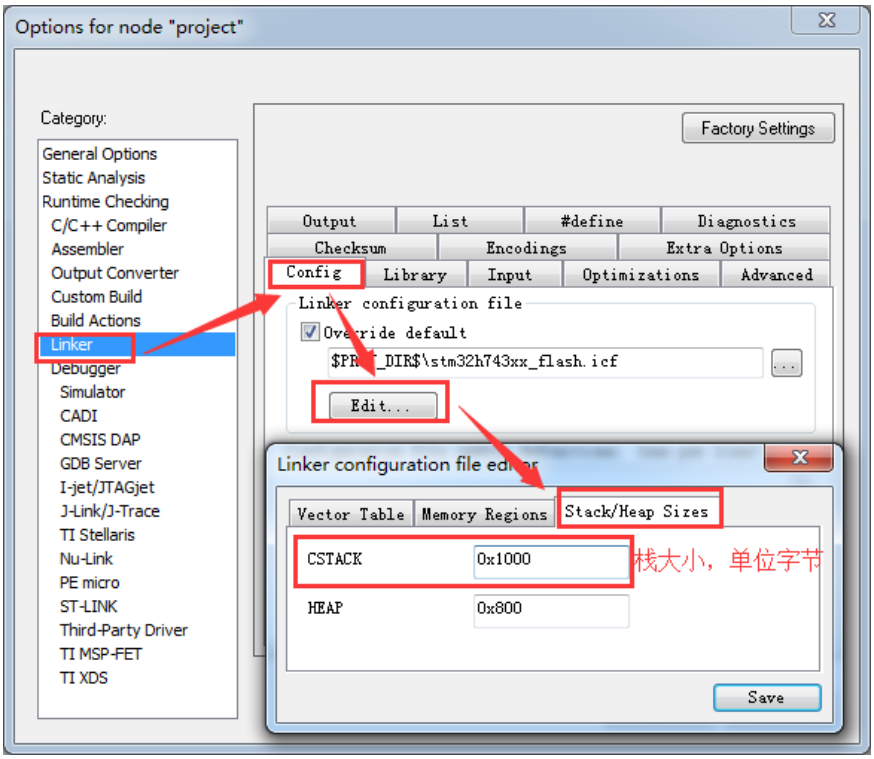
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

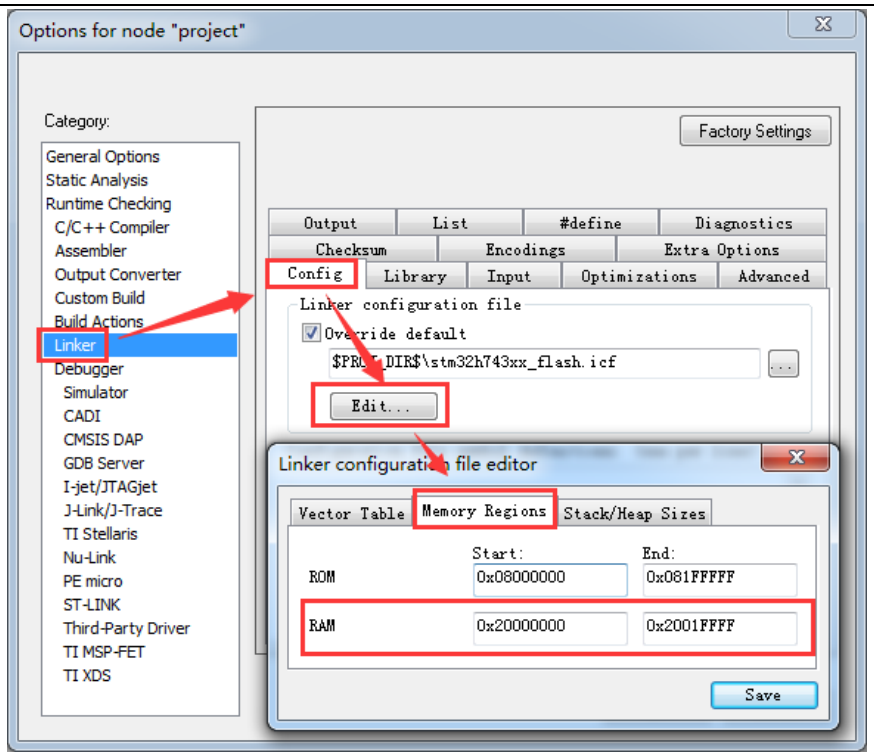


程序设计：

◆ 系统栈大小分配：



◆ RAM 空间用的 DTCM：



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
```

```
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能:**

主功能的实现主要分为两部分：

- 获取其它 RS485 设备发来的命令，并执行相应功能。

  通过函数 comGetChar(COM3, &ucDataTravel)以查询、非阻塞方式获取其它开发板发来的数据。

- 根据不同的按键消息，向其它 485 设备发送命令

  通过函数 comSendChar(COM3, ucDataTravel)以非阻塞方式向其它 485 设备发送命令，以此来执行同样的功能。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */
    uint8_t ucDataTravel;   /* 发送变量 */
    uint8_t ucDataRec;      /* 接收变量 */
```

```
    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 获取其它开发板通过 485 总线发来的数据 */
        if(comGetChar(COM3, &ucDataRec))
        {
            switch (ucDataRec)
            {
                case KEY_DOWN_K1:           /* 获得 K1 键按下消息 */
                    bsp_LedOn(1);
                    printf("K1 键按下, LED1 点亮\r\n");
                    break;

                case KEY_UP_K1:             /* 获得 K1 键释放消息 */
                    bsp_LedOff(1);
                    printf("K1 键弹起, LED1 熄灭\r\n");
                    break;

                case KEY_DOWN_K2:           /* 获得 K1 键按下消息 */
                    bsp_LedToggle(2);
                    break;

                case JOY_DOWN_U:            /* 获得摇杆 UP 键按下 */
                    printf("摇杆上键按下\r\n");
                    break;

                case JOY_DOWN_D:            /* 获得摇杆 DOWN 键按下 */
                    printf("摇杆下键按下\r\n");
                    break;

                case JOY_DOWN_L:            /* 获得摇杆 LEFT 键按下 */
                    printf("摇杆左键按下\r\n");
                    break;

                case JOY_DOWN_R:            /* 获得摇杆 RIGHT 键按下 */
                    printf("摇杆右键按下\r\n");
                    break;

                case JOY_DOWN_OK:           /* 获得摇杆 OK 键按下 */
                    printf("摇杆 OK 键按下\r\n");
                    break;

                case JOY_UP_OK:                 /* 获得摇杆 OK 键弹起 */
                    printf("摇杆 OK 键弹起\r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
```

```
/* 判断定时器超时时间 */
if (bsp_CheckTimer(0))
{
    /* 每隔 50ms 进来一次 */
    bsp_LedToggle(2);
    /* 向其它开发板发送按键 K2 按下的消息 */
    ucDataTravel = KEY_DOWN_K2;
    comSendChar(COM3, ucDataTravel);
}

/* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
if (ucKeyCode != KEY_NONE)
{
    switch (ucKeyCode)
    {
        case KEY_DOWN_K1:           /* K1 键按下 */
            ucDataTravel = KEY_DOWN_K1;
            comSendChar(COM3, ucDataTravel);
            bsp_LedOn(1);
            printf("K1 键按下, LED1 点亮\r\n");
            break;

        case KEY_UP_K1:             /* K1 键弹起 */
            ucDataTravel = KEY_UP_K1;
            comSendChar(COM3, ucDataTravel);
            bsp_LedOff(1);
            printf("K1 键弹起, LED1 熄灭\r\n");
            break;

        case KEY_DOWN_K2:           /* K2 键按下 */
            bsp_StartAutoTimer(0, 50);  /* 启动 1 个 50ms 的自动重装的定时器 */
            break;

        case KEY_DOWN_K3:           /* K3 键按下 */
            bsp_StopTimer(0);       /* 停止自动重装的定时器 */
            break;

        case JOY_DOWN_U:            /* 摇杆 UP 键按下 */
            ucDataTravel = JOY_DOWN_U;
            comSendChar(COM3, ucDataTravel);
            printf("摇杆上键按下\r\n");
            break;

        case JOY_DOWN_D:            /* 摇杆 DOWN 键按下 */
            ucDataTravel = JOY_DOWN_D;
            comSendChar(COM3, ucDataTravel);
            printf("摇杆下键按下\r\n");
            break;

        case JOY_DOWN_L:            /* 摇杆 LEFT 键按下 */
            ucDataTravel = JOY_DOWN_L;
            comSendChar(COM3, ucDataTravel);
            printf("摇杆左键按下\r\n");
            break;

        case JOY_DOWN_R:            /* 摇杆 RIGHT 键按下 */
```

```
                ucDataTravel = JOY_DOWN_R;
                comSendChar(COM3, ucDataTravel);
                printf("摇杆右键按下\r\n");
                break;

            case JOY_DOWN_OK:           /* 摇杆 OK 键按下 */
                ucDataTravel = JOY_DOWN_OK;
                comSendChar(COM3, ucDataTravel);
                printf("摇杆 OK 键按下\r\n");
                break;

            case JOY_UP_OK:                 /* 摇杆 OK 键弹起 */
                ucDataTravel = JOY_UP_OK;
                comSendChar(COM3, ucDataTravel);
                printf("摇杆 OK 键弹起\r\n");
                break;

            default:
                /* 其它的键值不处理 */
                break;
        }
    }
}
```

31.10 总结

本章节就为大家讲解这么多，485 通信依然在实际项目中非常实用。有线的串行网络仍然提供最有力、最可靠的通信，特别是在恶劣的环境中。

第32章 STM32H7 的 TIM 定时器基础知识和 HAL 库 API

本章节为大家讲解 TIM1 – TIM17（STM32H7 没有 TIM9，TIM10 和 TIM11）共计 14 个定时器的基础知识和对应的 HAL 库 API。

32.1 初学者重要提示

32.2 定时器基础知识

32.3 定时器的 HAL 库用户

32.4 源文件 stm32h7xx_hal_tim.c

32.5 总结

32.1 初学者重要提示

- 学习定时器外设推荐从硬件框图开始了解基本的功能特性，然后逐步深入了解各种特性，这种方式方便记忆和以后查阅。
- STM32H7 的定时器输出 100MHz 方波是完全没问题。
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=86434 。
- STM32H7 定时器进出中断的速度能跑到 12.5MHz，所有程序在 TCM 和 Flash 运行没差别。
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=90836 。
- STM32H7 的定时器输入捕获可以实现 12MHz 方波的双边沿捕获，单边沿可以做到 24MHz。
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=91283。
- 特别注意 STM32H7 的 TIM1,8,15,16,17 才有 RCR 重复计数器，其它都没用的。
- STM32H7 的单个定时器中不同通道可以配置不同频率 PWM。
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=89008 。
- STM32H7 的 TIM1-TIM17 中断入口函数名使用时要注意，别搞错了：

```
TIM1_BRK_IRQHandler
TIM1_UP_IRQHandler
TIM1_TRG_COM_IRQHandler
TIM1_CC_IRQHandler
TIM2_IRQHandler
TIM3_IRQHandler
TIM4_IRQHandler
TIM5_IRQHandler
TIM6_DAC_IRQHandler          <----------------要注意
```

```
TIM7_IRQHandler
TIM8_BRK_TIM12_IRQHandler      <------------------要注意，定时器 12 也是用的这个
TIM8_UP_TIM13_IRQHandler       <------------------要注意，定时器 13 也是用的这个
TIM8_TRG_COM_TIM14_IRQHandler  <------------------要注意，定时器 14 也是用的这个
TIM8_CC_IRQHandler
TIM15_IRQHandler
TIM16_IRQHandler
TIM17_IRQHandler
```

32.2 定时器基础知识

注，不同定时支持的功能略有区别，基础定时器功能较少，TIM1 和 TIM8 高级定时器功能多些。

- TIM2 和 TIM5 是 32 位定时器，其它定时器都是 16 位定时器。16 位和 32 位的区别是 CNT 计数器范围不同，32 位的范围是 0 到 2^32 – 1，而 16 位的是 0 到 65535；它们支持的分频是范围是一样的，都是 1 到 65535。
- 计数器支持递增、递减和递增/递减二合一。
- 多个独立通道，可用于：
  - 输入捕获。
  - 输出比较。
  - PWM 生成（边沿和中心对齐模式）。
  - 单脉冲模式输出。
- 带死区插入，断路功能和 PWM 互补输出，效果可看此贴：http://www.armbbs.cn/forum.php?mod=viewthread&tid=88997 。
- 发生如下事件时生成中断/DMA 请求：
  - 更新：计数器上溢/下溢、计数器初始化（通过软件或内部/外部触发）
  - 触发事件（计数器启动、停止、初始化或通过内部/外部触发计数）
  - 输入捕获
  - 输出比较
- 支持增量式编码器和霍尔传感器。

32.2.1 定时器 TIM1-TIM17 的区别

STM32H7 支持的定时器有点多，要简单的区分下。STM32H7 支持 TIM1-TIM8，TIM12-TIM17 共 14 个定时器，而中间的 TIM9，TIM10，TIM11 是不存在的，这点要注意。

粗略的比较如下：

Table 5. Timer feature comparison

Timer type	Timer	Counter resolution	Counter type	Prescaler factor	DMA request generation	Capture/ compare channels	Comple-mentary output	Max interface clock (MHz)	Max timer clock (MHz) (1)
High-resolution timer	HRTIM1	16-bit	Up	/1 /2 /4 (x2 x4 x8 x16 x32, with DLL)	Yes	10	Yes	400	400
Advanced -control	TIM1, TIM8	16-bit	Up, Down, Up/down	Any integer between 1 and 65536	Yes	4	Yes	100	200
General purpose	TIM2, TIM5	32-bit	Up, Down, Up/down	Any integer between 1 and 65536	Yes	4	No	100	200
	TIM3, TIM4	16-bit	Up, Down, Up/down	Any integer between 1 and 65536	Yes	4	No	100	200
	TIM12	16-bit	Up	Any integer between 1 and 65536	No	2	No	100	200
	TIM13, TIM14	16-bit	Up	Any integer between 1 and 65536	No	1	No	100	200
	TIM15	16-bit	Up	Any integer between 1 and 65536	Yes	2	1	100	200
	TIM16, TIM17	16-bit	Up	Any integer between 1 and 65536	Yes	1	1	100	200

Table 5. Timer feature comparison (continued)

Timer type	Timer	Counter resolution	Counter type	Prescaler factor	DMA request generation	Capture/ compare channels	Comple-mentary output	Max interface clock (MHz)	Max timer clock (MHz) (1)
Basic	TIM6, TIM7	16-bit	Up	Any integer between 1 and 65536	Yes	0	No	100	200
Low-power timer	LPTIM1, LPTIM2, LPTIM3, LPTIM4, LPTIM5	16-bit	Up	1, 2, 4, 8, 16, 32, 64, 128	No	0	No	100	200

1. The maximum timer clock is up to 400 MHz depending on TIMPRE bit in the RCC_CFGR register and D2PRE1/2 bits in RCC_D2CFGR register.

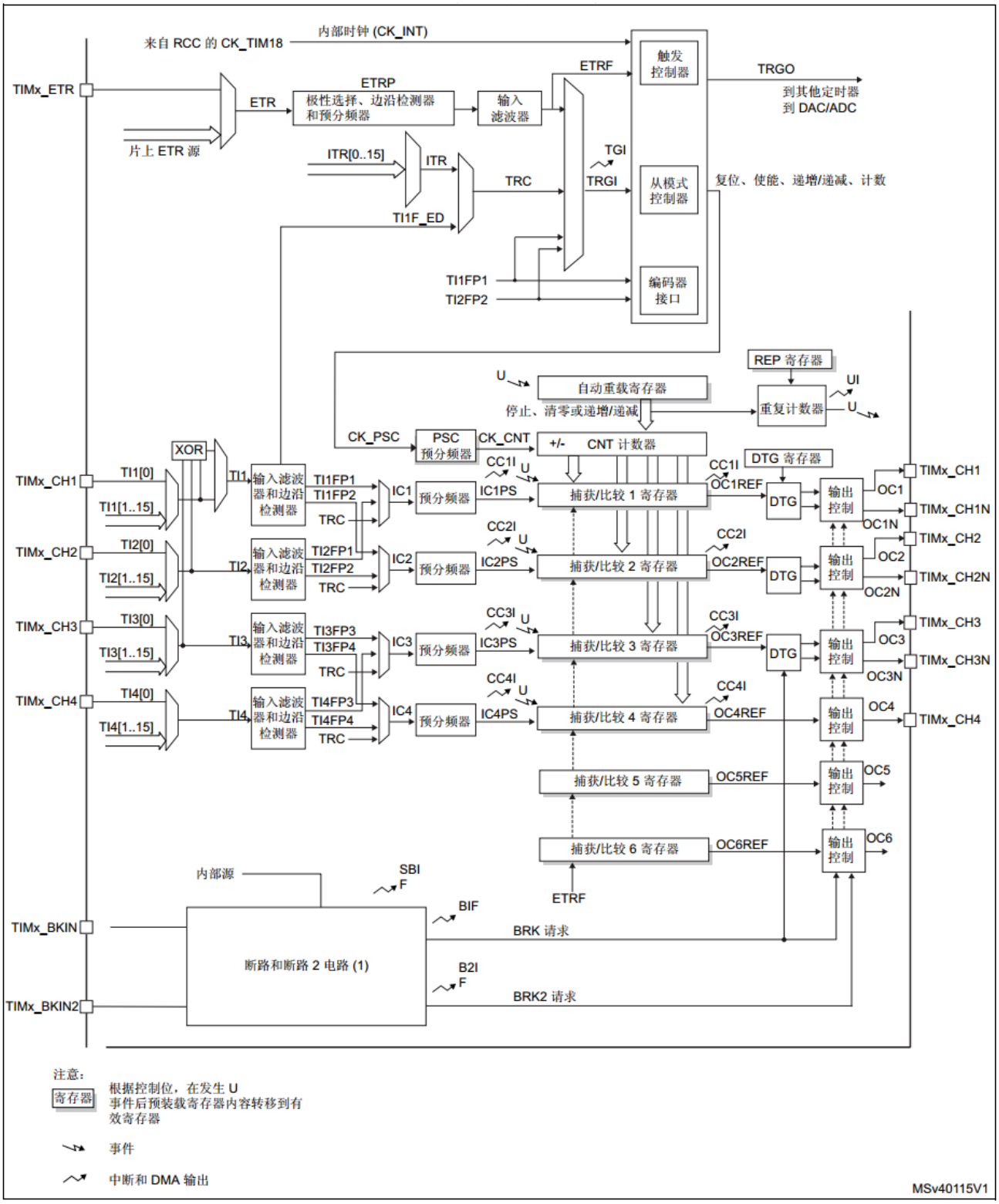
通过上面的表格，至少要了解到以下两点：

- STM32H7 的定时器主要分为高级定时器，通用定时器，基础定时器和低功耗定时器。
- TIM2 和 TIM5 是 32 位定时器，其它都是 16 位定时器。

32.2.2 定时器的硬件框图

认识一个外设，最好的方式就是看他的框图，方便我们快速的了解定时器的基本功能，然后再看手册了解细节。

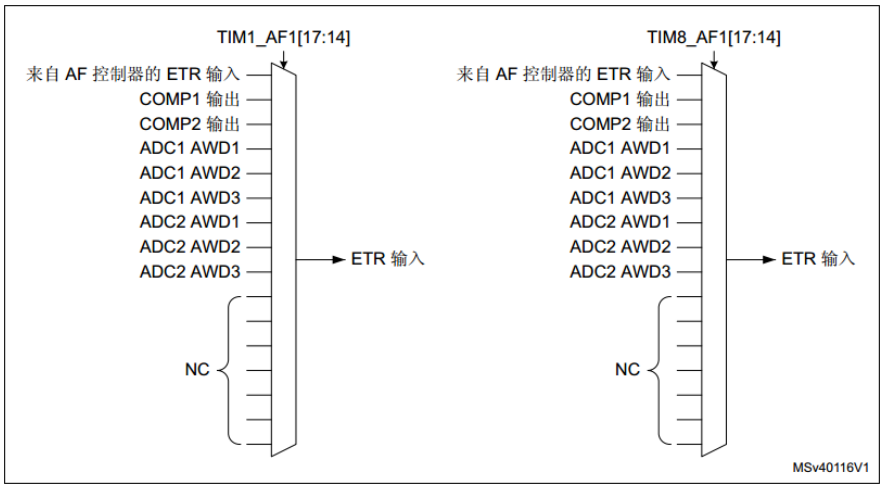
下面我们直接看最复杂的高级定时器 TIM1&TIM8 框图：



通过这个框图，我们可以得到如下信息：

- TIMx_ETR 接口

  外部触发输入接口。ETR 支持多种输入源：输入引脚（默认配置）、比较器输出和模拟看门狗。



- 截图左侧的 TIMx_CH1，TIMx_CH2，TIMx_CH3 和 TIMx_CH4 接口

  这四个通道主要用于输入捕获，可以计算波形频率和脉宽。

- TIMx_BKIN 和 TIMx_BKIN2 接口

  断路功能，主要用于保护由 TIM1 和 TIM8 定时器产生的 PWM 信号所驱动的功率开关

- TRGO 内部输出通道

  主要用于定时器级联，ADC 和 DAC 的定时器触发。

- 6 组输出比较单元 OC1 到 OC6

  OC1 到 OC4 有对应的输出引脚，而 OC5 和 OC6 没有对应的输出引脚，主要用于内部控制。

- 截图右侧的输出比较通道 TIMx_CH1，TIMx_CH1N，TIMx_CH2，TIMx_CH2N，TIMx_CH3，TIMx_CH3N 和 TIMx_CH4

  主要用于 PWM 输出，注意 CH1 到 CH3 有互补输出，而 CH4 没有互补输出。

- 其它框图里面未展示出来功能

  定时器 TIM1&TIM8 还支持的其它功能在用到的时候再做说明。

32.2.3定时器的时基单元

定时器要工作就需要一个基本时基单元，而基本的时基单元是由下面几个寄存器组成的：

- 预分频器寄存器（TIMx_PSC）

  用于设置定时器的分频，比如定时器的主频是 200MHz，通过此寄存器可以将其设置为 100MHz，50MHz，25MHz 等分频值。

  注：预分频器有个缓冲功能，可以让用户实时更改，新的预分频值将在下一个更新事件发生时被采用（以递增计数模式为例，就是 CNT 计数值达到 ARR 自动重装寄存器的数值时会产生更新事件）。

- 计数器寄存器（TIMx_CNT）

  计数器是最基本的计数单元，计数值是建立在分频的基础上面，比如通过 TIMx_PSC 设置分频后的频率为 100MHz，那么计数寄存器计一次数就是 10ns。

- 自动重载寄存器（TIMx_ARR）

  自动重装寄存器是 CNT 计数寄存器能达到的最大计数值，以递增计数模式为例，就是 CNT 计数器达到 ARR 寄存器数值时，重新从 0 开始计数。

  注，自动重载寄存器是预装载的。对自动重载寄存器执行写入或读取操作时会访问预装载寄存器。预装载寄存器的内容既可以立即传送到影子寄存器（让设置立即起到效果的寄存器），也可以在每次发生更新事件时传送到影子寄存器。简单的说就是让 ARR 寄存器的数值立即更新还是更新事件发送的时候更新。

- 重复计数器寄存器（TIMx_RCR）

  以递增计数模式为例，当 CNT 计数器数值达到 ARR 自动重载数值时，重复计数器的数值加 1，重复次数达到 TIMx_RCR+ 1 后就，将生成更新事件。

  注，只有 TIM1，TIM8，TIM15，TIM16，TIM17 有此寄存器。

比如我们要配置定时器实现周期性的中断，主要使用这几个寄存器即可。

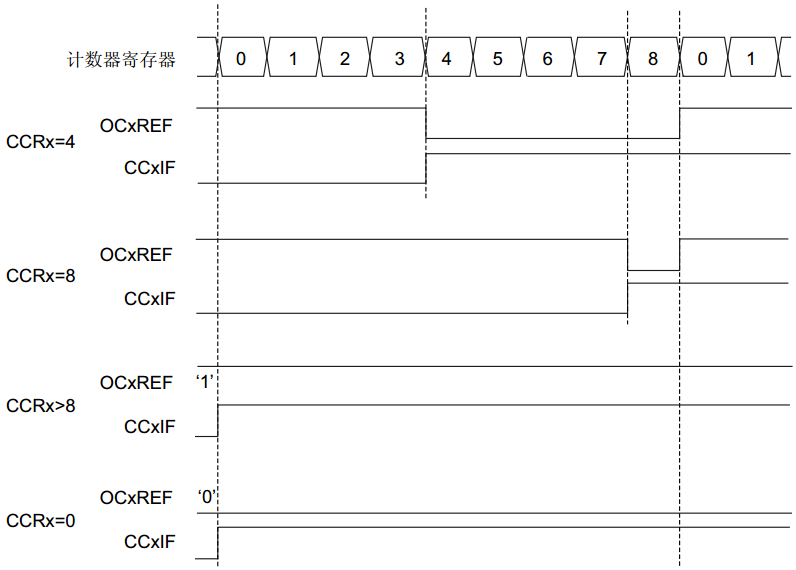
32.2.4定时器输出比较（PWM）

使用定时器时基单元的那几个寄存器仅仅能设置周期，还不能设置占空比。针对这个问题，还需要比较捕获寄存 CCR 的参与，这样就可以设置占空比了。

为了方便大家理解，以 PWM 边沿对齐模式，递增计数配置为例：

- 当计数器 TIMx_CNT < 比较捕获寄存器 TIMx_CCRx 期间，PWM 参考信号 OCxREF 输出高电平。
- 当计数器 TIMx_CNT >= 比较捕获寄存器 TIMx_CCRx 期间，PWM 参考信号 OCxREF 输出低电平。
- 当比较捕获寄存器 TIMx_CCRx > 自动重载寄存器 TIMx_ARR，OCxREF 保持为 1。
- 当比较捕获寄存器 TIMx_CCRx = 0，则 OCxRef 保持为 0。

下面是 TIMx_ARR=8 的波形效果：



32.2.5 定时器输入捕获

与 PWM 一样，使用定时器实现输入捕获，仅靠时基单元的那几个寄存器是不行的，我们需要一个寄存器来记录发生捕获时的具体时间，这个寄存器依然由比较捕获寄存器 TIMx_CCRx 来实现。

比如我们要测量一路方波的周期：

- 配置定时器为输入捕获模式，上升沿触发，设置分频，自动重装等寄存器，比如设置的 CNT 计数器计数 1 次是 1 微秒。
- 当有上升沿触发的时候，TIMx_CCRx 寄存器就会自动记录当前的 CNT 数值，然后用户就可以通过 CC 中断，在中断复位程序里面保存当前的 TIMx_CCRx 寄存器数值。等下次再检测到上升沿触发，两次时间求差就可以得到方波的周期。

  不过这里要特别注意一点，如果 CNT 发生溢出（比如 16 位定时器，计数到 65535 就溢出了）就需要特别处理下，将 CNT 计数溢出考虑进来。

32.3 定时器的 HAL 库用法

定时器的 HAL 库用法其实就是几个结构体变量成员的配置和使用，然后配置 GPIO、时钟，并根据需要配置 NVIC、中断和 DMA。下面我们逐一展开为大家做个说明。

32.3.1 定时器寄存器结构体 TIM_TypeDef

定时器相关的寄存器是通过 HAL 库中的结构体 TIM_TypeDef 定义的，在 stm32h743xx.h 中可以找到这个类型定义：

```
typedef struct
{
  __IO uint16_t CR1;         /*!< TIM control register 1,                 Address offset: 0x00 */
  uint16_t      RESERVED0;   /*!< Reserved, 0x02                                               */
  __IO uint32_t CR2;         /*!< TIM control register 2,                 Address offset: 0x04 */
  __IO uint32_t SMCR;        /*!< TIM slave mode control register,        Address offset: 0x08 */
  __IO uint32_t DIER;        /*!< TIM DMA/interrupt enable register,      Address offset: 0x0C */
  __IO uint32_t SR;          /*!< TIM status register,                    Address offset: 0x10 */
  __IO uint32_t EGR;         /*!< TIM event generation register,          Address offset: 0x14 */
  __IO uint32_t CCMR1;       /*!< TIM capture/compare mode register 1,    Address offset: 0x18 */
  __IO uint32_t CCMR2;       /*!< TIM capture/compare mode register 2,    Address offset: 0x1C */
  __IO uint32_t CCER;        /*!< TIM capture/compare enable register,    Address offset: 0x20 */
  __IO uint32_t CNT;         /*!< TIM counter register,                   Address offset: 0x24 */
  __IO uint16_t PSC;         /*!< TIM prescaler,                          Address offset: 0x28 */
  uint16_t      RESERVED9;   /*!< Reserved, 0x2A                                               */
  __IO uint32_t ARR;         /*!< TIM auto-reload register,               Address offset: 0x2C */
  __IO uint16_t RCR;         /*!< TIM repetition counter register,        Address offset: 0x30 */
  uint16_t      RESERVED10;  /*!< Reserved, 0x32                                               */
  __IO uint32_t CCR1;        /*!< TIM capture/compare register 1,         Address offset: 0x34 */
  __IO uint32_t CCR2;        /*!< TIM capture/compare register 2,         Address offset: 0x38 */
  __IO uint32_t CCR3;        /*!< TIM capture/compare register 3,         Address offset: 0x3C */
  __IO uint32_t CCR4;        /*!< TIM capture/compare register 4,         Address offset: 0x40 */
```

```
  __IO uint32_t BDTR;        /*!< TIM break and dead-time register,         Address offset: 0x44 */
  __IO uint16_t DCR;         /*!< TIM DMA control register,                 Address offset: 0x48 */
  uint16_t      RESERVED12;  /*!< Reserved, 0x4A                                                 */
  __IO uint16_t DMAR;        /*!< TIM DMA address for full transfer,        Address offset: 0x4C */
  uint16_t      RESERVED13;  /*!< Reserved, 0x4E                                                 */
  uint16_t      RESERVED14;  /*!< Reserved, 0x50                                                 */
  __IO uint32_t CCMR3;       /*!< TIM capture/compare mode register 3,      Address offset: 0x54 */
  __IO uint32_t CCR5;        /*!< TIM capture/compare register5,            Address offset: 0x58 */
  __IO uint32_t CCR6;        /*!< TIM capture/compare register6,            Address offset: 0x5C */
  __IO uint32_t AF1;         /*!< TIM alternate function option register 1, Address offset: 0x60 */
  __IO uint32_t AF2;         /*!< TIM alternate function option register 2, Address offset: 0x64 */
  __IO uint32_t TISEL;       /*!< TIM Input Selection register,             Address offset: 0x68 */
} TIM_TypeDef;
```

这个结构体的成员名称和排列次序和 CPU 的定时器寄存器是一 一对应的。

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

下面我们看下定时器的定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE           ((uint32_t)0x40000000)
#define D2_APB1PERIPH_BASE    PERIPH_BASE
#define D2_APB2PERIPH_BASE    (PERIPH_BASE + 0x00010000)

/*!< D2_APB1PERIPH 外设 */
#define TIM2_BASE             (D2_APB1PERIPH_BASE + 0x0000) <----- 展开这个宏，(TIM_TypeDef *) 0x40000000
#define TIM3_BASE             (D2_APB1PERIPH_BASE + 0x0400)
#define TIM4_BASE             (D2_APB1PERIPH_BASE + 0x0800)
#define TIM5_BASE             (D2_APB1PERIPH_BASE + 0x0C00)
#define TIM6_BASE             (D2_APB1PERIPH_BASE + 0x1000)
#define TIM7_BASE             (D2_APB1PERIPH_BASE + 0x1400)
#define TIM12_BASE            (D2_APB1PERIPH_BASE + 0x1800)
#define TIM13_BASE            (D2_APB1PERIPH_BASE + 0x1C00)
#define TIM14_BASE            (D2_APB1PERIPH_BASE + 0x2000)

/*!< D2_APB1PERIPH 外设 */
#define TIM1_BASE             (D2_APB2PERIPH_BASE + 0x0000)
#define TIM8_BASE             (D2_APB2PERIPH_BASE + 0x0400)
#define TIM15_BASE            (D2_APB2PERIPH_BASE + 0x4000)
#define TIM16_BASE            (D2_APB2PERIPH_BASE + 0x4400)
#define TIM17_BASE            (D2_APB2PERIPH_BASE + 0x4800)

#define TIM1                ((TIM_TypeDef *) TIM1_BASE)
#define TIM2                ((TIM_TypeDef *) TIM2_BASE)
#define TIM3                ((TIM_TypeDef *) TIM3_BASE)
#define TIM4                ((TIM_TypeDef *) TIM4_BASE)
#define TIM5                ((TIM_TypeDef *) TIM5_BASE)
#define TIM6                ((TIM_TypeDef *) TIM6_BASE)
#define TIM7                ((TIM_TypeDef *) TIM7_BASE)
#define TIM8                ((TIM_TypeDef *) TIM8_BASE)
#define TIM12               ((TIM_TypeDef *) TIM12_BASE)
#define TIM13               ((TIM_TypeDef *) TIM13_BASE)
#define TIM14               ((TIM_TypeDef *) TIM14_BASE)
#define TIM15               ((TIM_TypeDef *) TIM15_BASE)
#define TIM16               ((TIM_TypeDef *) TIM16_BASE)
#define TIM17               ((TIM_TypeDef *) TIM17_BASE)
```

我们访问 TIM2 的 CR1 寄存器可以采用这种形式：TIM2->CR1 = 0;

32.3.2 定时器句柄结构体 TIM_HandleTypeDef

HAL 库在 TIM_TypeDef 的基础上封装了一个结构体 TIM_HandleTypeDef，定义如下：

```
typedef struct
{
  TIM_TypeDef              *Instance;     /*!< Register base address             */
  TIM_Base_InitTypeDef     Init;          /*!< TIM Time Base required parameters */
  HAL_TIM_ActiveChannel    Channel;       /*!< Active channel                    */

  /*!< DMA Handlers array This array is accessed by a @ref DMA_Handle_index */
  DMA_HandleTypeDef        *hdma[7];
  HAL_LockTypeDef          Lock;          /*!< Locking object                    */
  __IO HAL_TIM_StateTypeDef   State;      /*!< TIM operation state               */
}TIM_HandleTypeDef;
```

这里重点介绍前四个参数，其它参数主要是 HAL 库内部使用的。

- **TIM_TypeDef *Instance**

  这个参数是寄存器的例化，方便操作寄存器，比如使能定时器的计数器。

  SET_BIT(huart->Instance->CR1, TIM_CR1_CEN)。

- **TIM_Base_InitTypeDef Init**

  这个参数是用户接触最多的，用于配置定时器的基本参数。

  TIM_Base_InitTypeDef 结构体的定义如下：

```
typedef struct
{
  uint32_t Prescaler;
  uint32_t CounterMode;
  uint32_t Period;
  uint32_t ClockDivision;
  uint32_t RepetitionCounter;
  uint32_t AutoReloadPreload;
} TIM_Base_InitTypeDef;
```

  - 成员 Prescaler

    用于设置定时器分频，对于 32 位的 TIM2 和 TIM5 范围是 0 到 0xFFFFFFFF，其它定时器是 0 到 0xFFFF。

  - 成员 CounterMode

    用于设置计数模式，向上计数模式、向下计数模式和中心对齐模式。

```
#define TIM_COUNTERMODE_UP                ((uint32_t)0x0000U)   /*!< Up counting mode */
#define TIM_COUNTERMODE_DOWN               TIM_CR1_DIR          /*!< Down counting mode */
#define TIM_COUNTERMODE_CENTERALIGNED1     TIM_CR1_CMS_0        /*!< Center-aligned counting mode 1 */
#define TIM_COUNTERMODE_CENTERALIGNED2     TIM_CR1_CMS_1        /*!< Center-aligned counting mode 2 */
#define TIM_COUNTERMODE_CENTERALIGNED3     TIM_CR1_CMS          /*!< Center-aligned counting mode 3 */
```

  - 成员 Period

用于设置定时器周期，对于 32 位的 TIM2 和 TIM5 范围是 0 到 0xFFFFFFFF，其它定时器是 0 到 0xFFFF。

- 成员 ClockDivision

用于指示定时器时钟（CK_INT）频率与死区发生器以及数字滤波器（ETR、TIx）所使用的死区及采样时钟（tDTS）之间的分频比。

```
#define TIM_CLOCKDIVISION_DIV1        ((uint32_t)0x0000U)        /*!< Clock Division DIV1 */
#define TIM_CLOCKDIVISION_DIV2        (TIM_CR1_CKD_0)            /*!< Clock Division DIV2 */
#define TIM_CLOCKDIVISION_DIV4        (TIM_CR1_CKD_1)             /*!< Clock Division DIV4 */
```

- 成员 RepetitionCounter

用于设置重复计数器，仅 TIM1 和 TIM8 有，其它定时器没有。作用是每当计数器上溢/下溢时，重复计数器减 1，当减到零时，才会生成更新事件，这个在生成 PWM 时比较有用。

- 成员 AutoReloadPreload

用于设置定时器的 ARR 自动重装寄存器是更新事件产生时写入有效还是立即写入有效。如果使能了表示更新事件产生时写入有效，否则反之。

```
#define TIM_AUTORELOAD_PRELOAD_DISABLE    ((uint32_t)0x0000U)    /*!< TIMx_ARR register is not buffered */
#define TIM_AUTORELOAD_PRELOAD_ENABLE     (TIM_CR1_ARPE)         /*!< TIMx_ARR register is buffered */
```

◆ **HAL_TIM_ActiveChannel Channel;**

用于设置定时器通道，比如 TIM1 和 TIM8 都是 6 个通道。

```
typedef enum
{
  HAL_TIM_ACTIVE_CHANNEL_1        = 0x01U,    /*!< The active channel is 1     */
  HAL_TIM_ACTIVE_CHANNEL_2        = 0x02U,    /*!< The active channel is 2     */
  HAL_TIM_ACTIVE_CHANNEL_3        = 0x04U,    /*!< The active channel is 3     */
  HAL_TIM_ACTIVE_CHANNEL_4        = 0x08U,    /*!< The active channel is 4     */
  HAL_TIM_ACTIVE_CHANNEL_5        = 0x10U,    /*!< The active channel is 5     */
  HAL_TIM_ACTIVE_CHANNEL_6        = 0x20U,    /*!< The active channel is 6     */
  HAL_TIM_ACTIVE_CHANNEL_CLEARED  = 0x00U     /*!< All active channels cleared */
}HAL_TIM_ActiveChannel;
```

◆ **DMA_HandleTypeDef *hdma[7];**

用于关联 DMA。

配置定时器参数，其实就是配置结构体 TIM_HandleTypeDef 的成员。

```
TIM_HandleTypeDef    TimHandle = {0};

/*
    定时器中断更新周期 = TIMxCLK / usPrescaler + 1）/usPeriod + 1）
*/
TimHandle.Instance = TIMx;
TimHandle.Init.Prescaler         = usPrescaler;
TimHandle.Init.Period            = usPeriod;
TimHandle.Init.ClockDivision     = 0;
TimHandle.Init.CounterMode       = TIM_COUNTERMODE_UP;
TimHandle.Init.RepetitionCounter = 0;
TimHandle.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
if (HAL_TIM_Base_Init(&TimHandle) != HAL_OK)
{
```

```
    Error_Handler(__FILE__, __LINE__);
}
```

32.3.3 定时器输出比较结构体 TIM_OC_InitTypeDef

此结构体主要用于定时器的输出比较，定义如下：

```
typedef struct
{
  uint32_t OCMode;
  uint32_t Pulse;
  uint32_t OCPolarity;
  uint32_t OCNPolarity;
  uint32_t OCFastMode;
  uint32_t OCIdleState;
  uint32_t OCNIdleState;
} TIM_OC_InitTypeDef;
```

下面将这几个参数一 一做个说明。

- **OCMode**

用于配置输出比较模式，支持的模式较多：

```
/*!< TIM Output timing mode */
#define TIM_OCMODE_TIMING                ((uint32_t)0x0000U)

/*!< TIM Output Active mode */
#define TIM_OCMODE_ACTIVE                ((uint32_t)TIM_CCMR1_OC1M_0)

 /*!< TIM Output Inactive mode */
#define TIM_OCMODE_INACTIVE              ((uint32_t)TIM_CCMR1_OC1M_1)

 /*!< TIM Output Toggle mode */
#define TIM_OCMODE_TOGGLE                ((uint32_t)TIM_CCMR1_OC1M_1 | TIM_CCMR1_OC1M_0)

 /*!< TIM PWM mode 1 */
#define TIM_OCMODE_PWM1                  ((uint32_t)TIM_CCMR1_OC1M_2 | TIM_CCMR1_OC1M_1)

/*!< TIM PWM mode 2 */
#define TIM_OCMODE_PWM2                  ((uint32_t)TIM_CCMR1_OC1M_2 | TIM_CCMR1_OC1M_1 | TIM_CCMR1_OC1M_0)

 /*!< TIM Forced Active mode */
#define TIM_OCMODE_FORCED_ACTIVE         ((uint32_t)TIM_CCMR1_OC1M_2 | TIM_CCMR1_OC1M_0)

 /*!< TIM Forced Inactive mode */
#define TIM_OCMODE_FORCED_INACTIVE       ((uint32_t)TIM_CCMR1_OC1M_2)

/*!< TIM Rettrigerrable OPM mode 1 */
#define TIM_OCMODE_RETRIGERRABLE_OPM1    ((uint32_t)TIM_CCMR1_OC1M_3)

/*!< TIM Rettrigerrable OPM mode 2 */
#define TIM_OCMODE_RETRIGERRABLE_OPM2    ((uint32_t)TIM_CCMR1_OC1M_3 | TIM_CCMR1_OC1M_0)

/*!< TIM Combined PWM mode 1 */
#define TIM_OCMODE_COMBINED_PWM1         ((uint32_t)TIM_CCMR1_OC1M_3 | TIM_CCMR1_OC1M_2)

/*!< TIM Combined PWM mode 2 */
```

```
#define TIM_OCMODE_COMBINED_PWM2           ((uint32_t)TIM_CCMR1_OC1M_3 | TIM_CCMR1_OC1M_0 | TIM_CCMR1_OC1M_2)

/*!< TIM Asymetruc PWM mode 1 */
#define TIM_OCMODE_ASSYMETRIC_PWM1         ((uint32_t)TIM_CCMR1_OC1M_3 | TIM_CCMR1_OC1M_1 | TIM_CCMR1_OC1M_2)

 /*!< TIM Asymetruc PWM mode 2 */
#define TIM_OCMODE_ASSYMETRIC_PWM2         ((uint32_t)TIM_CCMR1_OC1M_3 | TIM_CCMR1_OC1M)
```

- **Pulse**

可用于设置占空比，对应定时器的 CCR 寄存器，32 位的 TIM2 和 TIM5 范围是 0 到 0xFFFFFFFF。

- **OCPolarity**

设置输出极性，可选高电平或低电平有效。

```
#define TIM_OCPOLARITY_HIGH                ((uint32_t)0x0000U)
#define TIM_OCPOLARITY_LOW                 (TIM_CCER_CC1P)
```

- **OCNPolarity**

互补输出极性设置，可选高电平或者低电平有效。

```
#define TIM_OCNPOLARITY_HIGH               ((uint32_t)0x0000U)
#define TIM_OCNPOLARITY_LOW                (TIM_CCER_CC1NP)
```

- **OCFastMode**

快速输出模式使能，仅 OCMode 配置为 PWM1 或者 PWM2 模式时才有意义。

```
#define TIM_OCFAST_DISABLE                 ((uint32_t)0x0000U)
#define TIM_OCFAST_ENABLE                  (TIM_CCMR1_OC1FE)
```

- **OCIdleState**

空闲状态时，设置输出比较引脚的电平状态。

```
#define TIM_OCIDLESTATE_SET                (TIM_CR2_OIS1)
#define TIM_OCIDLESTATE_RESET              ((uint32_t)0x0000U)
```

- **OCNIdleState**

空闲状态时，设置互补输出引脚的电平状态。

```
#define TIM_OCNIDLESTATE_SET               (TIM_CR2_OIS1N)
#define TIM_OCNIDLESTATE_RESET             ((uint32_t)0x0000U)
```

32.3.4定时器输入捕获结构体 TIM_IC_InitTypeDef

此结构体主要用于定时器的输入捕获，定义如下：

```
typedef struct
{
  uint32_t ICPolarity;
  uint32_t ICSelection;
  uint32_t ICPrescaler;
  uint32_t ICFilter;
} TIM_IC_InitTypeDef;
```

下面将这几个参数一 一做个说明。

- **ICPolarity**

输入触发极性，可以选择上升沿，下降沿或者双沿触发。

```
#define  TIM_ICPOLARITY_RISING             TIM_INPUTCHANNELPOLARITY_RISING
#define  TIM_ICPOLARITY_FALLING            TIM_INPUTCHANNELPOLARITY_FALLING
```

```
#define  TIM_ICPOLARITY_BOTHEDGE          TIM_INPUTCHANNELPOLARITY_BOTHEDGE
```

- **ICSelection**

  输入捕获通道选择，可以选择直接输入（即 CC1 选择 TI1，CC2 选择 TI2 等），间接输入（CC1 选择 TI2，CC3 选择 TI4 等）或者 TRC。

```
#define TIM_ICSELECTION_DIRECTTI        (TIM_CCMR1_CC1S_0)
#define TIM_ICSELECTION_INDIRECTTI      (TIM_CCMR1_CC1S_1)
#define TIM_ICSELECTION_TRC             (TIM_CCMR1_CC1S)
```

- **ICPrescaler**

  输入捕获分频，表示每捕获 1，2，4 或 8 个事件后表示一次捕获。

```
#define TIM_ICPSC_DIV1        ((uint32_t)0x0000U)
#define TIM_ICPSC_DIV2        (TIM_CCMR1_IC1PSC_0)
#define TIM_ICPSC_DIV4        (TIM_CCMR1_IC1PSC_1)
#define TIM_ICPSC_DIV8        (TIM_CCMR1_IC1PSC)
```

- **ICFilter**

  输入捕获滤波器，可以定义采样频率和多少个连续事件才视为有效的触发，参数范围 0 到 15。具体定义如下，其中 f_{CK_INT} 表示定时器时钟，f_{DTS} 表示死区时间采样率，N 表示这么多个事件代表一次有效边沿。

```
0000：无滤波器，按 fDTS 频率进行采样
0001：  fSAMPLING=fCK_INT，  N=2
0010：  fSAMPLING=fCK_INT，  N=4
0011：  fSAMPLING=fCK_INT，  N=8
0100：  fSAMPLING=fDTS/2，  N=6
0101：  fSAMPLING=fDTS/2，  N=8
0110：  fSAMPLING=fDTS/4，  N=6
0111：  fSAMPLING=fDTS/4，  N=8
1000：  fSAMPLING=fDTS/8，  N=6
1001：  fSAMPLING=fDTS/8，  N=8
1010：  fSAMPLING=fDTS/16，  N=5
1011：  fSAMPLING=fDTS/16，  N=6
1100：  fSAMPLING=fDTS/16，  N=8
1101：  fSAMPLING=fDTS/32，  N=5
1110：  fSAMPLING=fDTS/32，  N=6
```

32.3.5 定时器的底层配置（GPIO，时钟，中断等）

HAL 库有个自己的底层初始化回调函数，比如调用函数 HAL_TIM_Base_Init 就会调用 HAL_TIM_Base_MspInit，此函数是弱定义的。

```
__weak void HAL_TIM_Base_MspInit(TIM_HandleTypeDef *htim)
{
  /* Prevent unused argument(s) compilation warning */
  UNUSED(htim);
  /* NOTE : This function Should not be modified, when the callback is needed,
            the HAL_TIM_Base_MspDeInit could be implemented in the user file
   */
}
```

用户可以在其它的 C 文件重定向，并将相对的底层初始化在里面实现。对应的底层复位函数 HAL_TIM_Base_DeInit 是在函数 HAL_TIM_Base_MspDeInit 里面被调用的，也是弱定义的。

当然，用户也可以自己初始化，不限制必须在两个函数里面实现。

定时器外设的基本参数配置完毕后还不能使用，还需要配置 GPIO、时钟、中断等参数，比如下面配置 TIM1 使用 PA8 做 PWM 输出。

```
void HAL_TIM_PWM_MspInit(TIM_HandleTypeDef *htim)
{
  GPIO_InitTypeDef   GPIO_InitStruct;

  /* 使能 TIM1 时钟 */
  __HAL_RCC_TIM1_CLK_ENABLE ();

  /* 使能 GPIOA 时钟 */
  __HAL_RCC_GPIOA_CLK_ENABLE ();

  /* 设置 TIM1 使用 PA8 做 PWM 输出引脚，将其配置为输出，推挽，复用模式 */
  GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;

  GPIO_InitStruct.Alternate = GPIO_AF1_TIM1;
  GPIO_InitStruct.Pin = GPIO_PIN_8;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

}
```

总结下来就是以下几点：

- 配置 TIM 时钟。
- 配置 TIM 所用到引脚和对应的 GPIO 时钟。
- 如果用到定时器中断，还需要通过 NVIC 配置中断。
- 如果用到 DMA，还要配置 DMA。

关于这个底层配置有以下几点要着重说明下：

- 定时器所使用引脚的复用模式选择已经被 HAL 库定义好，放在了 stm32h7xx_hal_gpio_ex.h 文件里面。比如 TIM1 有一个复用，

```
#define GPIO_AF1_TIM1      ((uint8_t)0x01)  /* TIM1 Alternate Function mapping */
```

但是却有 4 个输出通道，每个通道都有几个支持的输出引脚：

```
TIM1_CH1,    PA8     PE9     PK1
TIM1_CH2,    PA9     PE11
TIM1_CH3,    PA10    PE13    PJ9
TIM1_CH4,    PA11    PE14    PJ11
```

具体使用哪个，配置对应引脚的复用即可：

Port		AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7	AF8	AF9	AF10	AF11	AF12	AF13	AF14	AF15
		SYS	TIM1/2/16/17/LPTIM1/HRTIM1	SAI1/TIM3/4/5/12/HRTIM1	LPUART/TIM8/LPTIM2/3/4/5/HRTIM1/DFSDM	I2C1/2/3/4/USART1/TIM15/LPTIM2/DFSDM/CEC	SPI1/2/3/4/5/6/CEC	SPI2/3/SAI1/3/I2C4/UART4/DFSDM	SPI2/3/6/USART1/2/3/6/UART7/SDMMC1	SPI6/SAI2/4/UART4/5/8/LPUART/SDMMC1/SPDIFRX	SAI4/FDCAN1/2/TIM13/14/QUADSPI/FMC/SDMMC2/LCD/SPDIFRX	SAI2/4/TIM8/QUADSPI/SDMMC2/OTG1_HS/OTG2_FS/LCD	I2C4/UART7/SWPMI1/TIM1/8/DFSDM/SDMMC2/MDIOS/ETH	TIM1/8/FMC/SDMMC1/MDIOS/OTG1_FS/LCD	TIM1/DCMI/LCD/COMP	UART5/LCD	SYS
Port A	PA0	-	TIM2_CH1/TIM2_ETR	TIM5_CH1	TIM8_ETR	TIM15_BKIN	-	-	USART2_CTS_NSS	UART4_TX	SDMMC2_CMD	SAI2_SD_B	ETH_MII_CRS	-	-	-	EVENT-OUT
	PA1	-	TIM2_CH2	TIM5_CH2	LPTIM3_OUT	TIM15_CH1N	-	-	USART2_RTS	UART4_RX	QUADSPI_BK1_IO3	SAI2_MCK_B	ETH_MII_RX_CLK/ETH_RMII_REF_CLK	-	-	LCD_R2	EVENT-OUT
	PA2	-	TIM2_CH3	TIM5_CH3	LPTIM4_OUT	TIM15_CH1	-	-	USART2_TX	SAI2_SCK_B	-	-	ETH_MDIO	MDIOS_MDIO	-	LCD_R1	EVENT-OUT
	PA3	-	TIM2_CH4	TIM5_CH4	LPTIM5_OUT	TIM15_CH2	-	-	USART2_RX	-	LCD_B2	OTG_HS_ULPI_D0	ETH_MII_COL	-	-	LCD_B5	EVENT-OUT
	PA4	-	-	TIM5_ETR	-	-	SPI1_NSS/I2S1_WS	SPI3_NSS/I2S3_WS	USART2_CK	SPI6_NSS	-	-	-	OTG_HS_SOF	DCMI_HSYNC	LCD_VSYNC	EVENT-OUT
	PA5	-	TIM2_CH1/TIM2_ETR	-	TIM8_CH1N	-	SPI1_SCK/I2S1_CK	-	-	SPI6_SCK	-	OTG_HS_ULPI_CK	-	-	-	LCD_R4	EVENT-OUT
	PA6	-	TIM1_BKIN	TIM3_CH1	TIM8_BKIN	-	SPI1_MISO/I2S1_SDI	-	-	SPI6_MISO	TIM13_CH1	TIM8_BKIN_COMP12	MDIOS_MDC	TIM1_BKIN_COMP12	DCMI_PIXCLK	LCD_G2	EVENT-OUT
	PA7	-	TIM1_CH1N	TIM3_CH2	TIM8_CH1N	-	SPI1_MOSI/I2S1_SDO	-	-	SPI6_MOSI	TIM14_CH1	-	ETH_MII_RX_DV/ETH_RMII_CRS_DV	FMC_SDNWE	-	-	EVENT-OUT
	PA8	MCO1	TIM1_CH1	HRTIM_CHB2	TIM8_BKIN2	I2C3_SCL	-	-	USART1_CK	-	-	OTG_FS_SOF	UART7_RX	TIM8_BKIN2_COMP12	LCD_B3	LCD_R6	EVENT-OUT
	PA9	-	TIM1_CH2	HRTIM_CHC1	LPUART1_TX	I2C3_SMBA	SPI2_SCK/I2S2_CK	-	USART1_TX	-	FDCAN1_RXFD_MODE	-	ETH_TX_ER	-	DCMI_D0	LCD_R5	EVENT-OUT
	PA10	-	TIM1_CH3	HRTIM_CHC2	LPUART1_RX	-	-	-	USART1_RX	-	FDCAN1_TXFD_MODE	OTG_FS_ID	MDIOS_MDIO	LCD_B4	DCMI_D1	LCD_B1	EVENT-OUT
	PA11	-	TIM1_CH4	HRTIM_CHD1	LPUART1_CTS	-	SPI2_NSS/I2S2_WS	UART4_RX	USART1_CTS_NSS	-	FDCAN1_RX	OTG_FS_DM	-	-	-	LCD_R4	EVENT-OUT
	PA12	-	TIM1_ETR	HRTIM_CHD2	LPUART1_RTS	-	SPI2_SCK/I2S2_CK	UART4_TX	USART1_RTS	SAI2_FS_B	FDCAN1_TX	OTG_FS_DP	-	-	-	LCD_R5	EVENT-OUT
	PA13	JTMS-SWDIO	-	-	-	-	-	-	-	-	-	-	-	-	-	-	EVENT-OUT

32.3.6定时器的状态标志清除问题

下面我们介绍__HAL_TIM_GET_FLAG 函数。这个函数用来检查定时器标志位是否被设置。

```
/** @brief  Check whether the specified TIM interrupt flag is set or not.
  * @param  __HANDLE__: specifies the TIM Handle.
  * @param  __FLAG__: specifies the TIM interrupt flag to check.
  *        This parameter can be one of the following values:
  *           @arg TIM_FLAG_UPDATE: Update interrupt flag
  *           @arg TIM_FLAG_CC1: Capture/Compare 1 interrupt flag
  *           @arg TIM_FLAG_CC2: Capture/Compare 2 interrupt flag
  *           @arg TIM_FLAG_CC3: Capture/Compare 3 interrupt flag
  *           @arg TIM_FLAG_CC4: Capture/Compare 4 interrupt flag
  *           @arg TIM_FLAG_CC5: Compare 5 interrupt flag
  *           @arg TIM_FLAG_CC6: Compare 6 interrupt flag
  *           @arg TIM_FLAG_COM:  Commutation interrupt flag
  *           @arg TIM_FLAG_TRIGGER: Trigger interrupt flag
  *           @arg TIM_FLAG_BREAK: Break interrupt flag
  *           @arg TIM_FLAG_BREAK2: Break 2 interrupt flag
  *           @arg TIM_FLAG_SYSTEM_BREAK: System Break interrupt flag
  *           @arg TIM_FLAG_CC1OF: Capture/Compare 1 overcapture flag
  *           @arg TIM_FLAG_CC2OF: Capture/Compare 2 overcapture flag
  *           @arg TIM_FLAG_CC3OF: Capture/Compare 3 overcapture flag
  *           @arg TIM_FLAG_CC4OF: Capture/Compare 4 overcapture flag
  * @retval The new state of __FLAG__ (TRUE or FALSE).
  */
#define __HAL_TIM_GET_FLAG(__HANDLE__, __FLAG__)  (((__HANDLE__)->Instance->SR &(__FLAG__)) == (__FLAG__))
```

前 5 个是比较常用的中断标志。

- TIM_FLAG_UPDATE

  定时器更新标准，配置一个周期性的定时器中断要用到。

- TIM_FLAG_CC1

  TIM_FLAG_CC2

  TIM_FLAG_CC3

  TIM_FLAG_CC4

  捕获/比较标志，配置了捕获/比较中断要用到。

与标志获取函数__HAL_TIM_GET_FLAG 对应的清除函数是__HAL_TIM_CLEAR_FLAG：

```
/** @brief  Clear the specified TIM interrupt flag.
  * @param  __HANDLE__: specifies the TIM Handle.
  * @param  __FLAG__: specifies the TIM interrupt flag to clear.
  *        This parameter can be one of the following values:
  *            @arg TIM_FLAG_UPDATE: Update interrupt flag
  *            @arg TIM_FLAG_CC1: Capture/Compare 1 interrupt flag
  *            @arg TIM_FLAG_CC2: Capture/Compare 2 interrupt flag
  *            @arg TIM_FLAG_CC3: Capture/Compare 3 interrupt flag
  *            @arg TIM_FLAG_CC4: Capture/Compare 4 interrupt flag
  *            @arg TIM_FLAG_CC5: Compare 5 interrupt flag
  *            @arg TIM_FLAG_CC6: Compare 6 interrupt flag
  *            @arg TIM_FLAG_COM:  Commutation interrupt flag
  *            @arg TIM_FLAG_TRIGGER: Trigger interrupt flag
  *            @arg TIM_FLAG_BREAK: Break interrupt flag
  *            @arg TIM_FLAG_BREAK2: Break 2 interrupt flag
  *            @arg TIM_FLAG_SYSTEM_BREAK: System Break interrupt flag
  *            @arg TIM_FLAG_CC1OF: Capture/Compare 1 overcapture flag
  *            @arg TIM_FLAG_CC2OF: Capture/Compare 2 overcapture flag
  *            @arg TIM_FLAG_CC3OF: Capture/Compare 3 overcapture flag
  *            @arg TIM_FLAG_CC4OF: Capture/Compare 4 overcapture flag
  * @retval The new state of __FLAG__ (TRUE or FALSE).
  */
#define __HAL_TIM_CLEAR_FLAG(__HANDLE__, __FLAG__)       ((__HANDLE__)->Instance->SR = ~(__FLAG__))
```

清除标志函数所支持的参数跟获取函数是一 一对应的。除了这两个函数，还是定时器的中断开启和中断关闭函数用的也比较多。

```
/** @brief  Enable the specified TIM interrupt.
* @param  __HANDLE__: specifies the TIM Handle.
* @param  __INTERRUPT__: specifies the TIM interrupt source to enable.
*          This parameter can be one of the following values:
*            @arg TIM_IT_UPDATE: Update interrupt
*            @arg TIM_IT_CC1:   Capture/Compare 1 interrupt
*            @arg TIM_IT_CC2:  Capture/Compare 2 interrupt
*            @arg TIM_IT_CC3:  Capture/Compare 3 interrupt
*            @arg TIM_IT_CC4:  Capture/Compare 4 interrupt
*            @arg TIM_IT_COM:   Commutation interrupt
*            @arg TIM_IT_TRIGGER: Trigger interrupt
*            @arg TIM_IT_BREAK: Break interrupt
* @retval None
*/
#define __HAL_TIM_ENABLE_IT(__HANDLE__, __INTERRUPT__)  ((__HANDLE__)->Instance->DIER |= (__INTERRUPT__))
```

```
  /** @brief  Disable the specified TIM interrupt.
  * @param  __HANDLE__: specifies the TIM Handle.
  * @param  __INTERRUPT__: specifies the TIM interrupt source to disable.
  *          This parameter can be one of the following values:
  *            @arg TIM_IT_UPDATE: Update interrupt
  *            @arg TIM_IT_CC1:   Capture/Compare 1 interrupt
  *            @arg TIM_IT_CC2:  Capture/Compare 2 interrupt
  *            @arg TIM_IT_CC3:  Capture/Compare 3 interrupt
  *            @arg TIM_IT_CC4:  Capture/Compare 4 interrupt
  *            @arg TIM_IT_COM:   Commutation interrupt
  *            @arg TIM_IT_TRIGGER: Trigger interrupt
  *            @arg TIM_IT_BREAK: Break interrupt
  * @retval None
  */
#define __HAL_TIM_DISABLE_IT(__HANDLE__, __INTERRUPT__)   ((__HANDLE__)->Instance->DIER &= ~(__INTERRUPT__))
```

常用的也是前五个参数，1 个定时器更新中断以及 4 个 CC 中断 。

注意：操作定时器的寄存器不限制必须要用 HAL 库提供的 API，比如要操作寄存器 CR1，直接调用 TIM1->CR1 操作即可。

32.3.7 定时器初始化流程总结

使用方法由 HAL 库提供：

- 第 1 步：通过下面几个函数配置定时器工作在相应的模式
  - HAL_TIM_Base_Init
    简单的定时器时基础功能
  - HAL_TIM_OC_Init 和 HAL_TIM_OC_ConfigChannel
    配置定时器产生输出比较信号
  - HAL_TIM_PWM_Init 和 HAL_TIM_PWM_ConfigChannel
    配置定时器产生 PWM 信号
  - HAL_TIM_IC_Init 和 HAL_TIM_IC_ConfigChannel
    配置定时器测量外部信号
  - HAL_TIM_OnePulse_Init 和 HAL_TIM_OnePulse_ConfigChannel
    配置定时器工作在单脉冲模式
  - HAL_TIM_Encoder_Init
    配置定时器使用编码器接口
- 第 2 步：定时器几个常用功能的底层初始化 API，这个里面需要用户自己填
  第 1 步里面的几个函数会调用下面的 API。
  - 定时器基本功能 ：HAL_TIM_Base_MspInit()
  - 输入捕获 ：HAL_TIM_IC_MspInit()

- 输出比较：HAL_TIM_OC_MspInit()
- PWM 输出：HAL_TIM_PWM_MspInit()
- 单脉冲输出模式: HAL_TIM_OnePulse_MspInit()
- 编码器模式：HAL_TIM_Encoder_MspInit()

◆ 第 3 步：底层初始化具体实现

第 2 步中函数的具体实现。

- 使用函数__HAL_RCC_TIMx_CLK_ENABLE()使能定时器时钟。
- 使用函数__HAL_RCC_GPIOx_CLK_ENABLE()使能定时器使用到的引脚时钟。
- 使用函数 HAL_GPIO_Init()配置 GPIO 的复用功能。
- 如果使能了定时器中断，调用函数 HAL_NVIC_SetPriority 和 HAL_NVIC_EnableIRQ 配置。
- 如果使能了 DMA，还需要做 DMA 的配置。
- 定时器默认使用 APB 时钟，如果使用外部时钟，调用函数 HAL_TIM_ConfigClockSource 可以配置。

◆ 第 4 步：启动定时器外设

- 定时器基础功能:
  HAL_TIM_Base_Start()
  HAL_TIM_Base_Start_DMA()
  HAL_TIM_Base_Start_IT()
- 输入捕获：
  HAL_TIM_IC_Start()
  HAL_TIM_IC_Start_DMA()
  HAL_TIM_IC_Start_IT()
- 输出比较：
  HAL_TIM_OC_Start()
  HAL_TIM_OC_Start_DMA()
  HAL_TIM_OC_Start_IT()
- PWM 输出:
  HAL_TIM_PWM_Start()
  HAL_TIM_PWM_Start_DMA()
  HAL_TIM_PWM_Start_IT()
- 单脉冲模式:
  HAL_TIM_OnePulse_Start()
  HAL_TIM_OnePulse_Start_IT().
- 编码器模式:

HAL_TIM_Encoder_Start()

HAL_TIM_Encoder_Start_DMA()

HAL_TIM_Encoder_Start_IT().

- 第 5 步：定时器的 DMA 突发使用下面两个函数
  - HAL_TIM_DMABurst_WriteStart()
  - HAL_TIM_DMABurst_ReadStart()

定时器常用的功能，通过上面这几步即可实现。

32.4 源文件 stm32h7xx_hal_tim.c

此文件涉及到的函数非常多，这里把几个常用的函数做个说明：

- HAL_TIM_Base_Init
- HAL_TIM_Base_Start
- HAL_TIM_PWM_Init
- HAL_TIM_PWM_ConfigChannel
- HAL_TIM_PWM_Start
- HAL_TIM_IC_Init
- HAL_TIM_IC_ConfigChannel
- HAL_TIM_IC_Start_IT
- HAL_TIM_OC_Init
- HAL_TIM_OC_ConfigChannel
- HAL_TIM_OC_Start

32.4.1 函数 HAL_TIM_Base_Init

函数原型：

```
HAL_StatusTypeDef HAL_TIM_Base_Init(TIM_HandleTypeDef *htim)
{
  /* 检测是否是有效句柄 */
  if(htim == NULL)
  {
    return HAL_ERROR;
  }

  /* 检测参数 */
  assert_param(IS_TIM_INSTANCE(htim->Instance));
  assert_param(IS_TIM_COUNTER_MODE(htim->Init.CounterMode));
  assert_param(IS_TIM_CLOCKDIVISION_DIV(htim->Init.ClockDivision));

  if(htim->State == HAL_TIM_STATE_RESET)
```

```
  {
    /* 默认取消锁 */
    htim->Lock = HAL_UNLOCKED;
    /* 初始化底层硬件 : GPIO, CLOCK, NVIC */
    HAL_TIM_Base_MspInit(htim);
  }

  /* 设置 TIM 状态 */
  htim->State= HAL_TIM_STATE_BUSY;

  /* 基本参数配置 */
  TIM_Base_SetConfig(htim->Instance, &htim->Init);

  /* 设置 TIM 就绪 */
  htim->State= HAL_TIM_STATE_READY;

  return HAL_OK;
}
```

函数描述：

此函数用于初始化定时器用于 PWM。

函数参数：

- 第 1 个参数是 TIM_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 函数 HAL_TIM_Base_MspInit 用于初始化定时器的底层时钟、引脚等功能。需要用户自己在此函数里面实现具体的功能，由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
2. 如果形参 htim 的结构体成员 gState 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 TIM_HandleTypeDef TimHandle。

   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_TIM_STATE_RESET = 0x00U。

   解决办法有三：

   方法 1：用户自己初始定时器和涉及到的 GPIO 等。

   方法 2：定义 TIM_HandleTypeDef TimHandle 为全局变量。

   方法 3：下面的方法

```
if(HAL_TIM_Base_DeInit(&TimHandle) != HAL_OK)
{
    Error_Handler();
}
if(HAL_TIM_Base_Init(&TimHandle) != HAL_OK)
{
    Error_Handler();
}
```

使用举例：

```
TIM_HandleTypeDef    TimHandle = {0};

/*
    定时器中断更新周期 = TIMxCLK / usPrescaler + 1）/usPeriod + 1）
*/
TimHandle.Instance = TIMx;
TimHandle.Init.Prescaler         = usPrescaler;
TimHandle.Init.Period            = usPeriod;
TimHandle.Init.ClockDivision     = 0;
TimHandle.Init.CounterMode       = TIM_COUNTERMODE_UP;
TimHandle.Init.RepetitionCounter = 0;
TimHandle.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
if (HAL_TIM_Base_Init(&TimHandle) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

32.4.2 函数 HAL_TIM_Base_Start

函数原型：

```
HAL_StatusTypeDef HAL_TIM_Base_Start(TIM_HandleTypeDef *htim)
{
  /* 检测参数状态 */
  assert_param(IS_TIM_INSTANCE(htim->Instance));

  /* 设置定时器状态 */
  htim->State= HAL_TIM_STATE_BUSY;

  /* 使能定时器 */
  __HAL_TIM_ENABLE(htim);

  /* 设置定时器状态 */
  htim->State= HAL_TIM_STATE_READY;

  /* 返回 HAL_OK */
  return HAL_OK;
}
```

函数描述：

此函数比较简单，调用函数 HAL_TIM_Base_Init 配置了基础功能后，启动定时器。

函数参数：

- 第 1 个参数是 TIM_HandleTypeDef 类型结构体指针变量。
- 返回值，固定返回 HAL_OK，表示初始化成功。

使用举例：

```
TIM_HandleTypeDef    TimHandle = {0};

/*
    定时器中断更新周期 = TIMxCLK / usPrescaler + 1）/usPeriod + 1）
*/
TimHandle.Instance = TIMx;
TimHandle.Init.Prescaler         = usPrescaler;
TimHandle.Init.Period            = usPeriod;
```

```
TimHandle.Init.ClockDivision     = 0;
TimHandle.Init.CounterMode       = TIM_COUNTERMODE_UP;
TimHandle.Init.RepetitionCounter = 0;
TimHandle.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
if (HAL_TIM_Base_Init(&TimHandle) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}

/* 启动定时器 */
if (HAL_TIM_Base_Start(&TimHandle) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

32.4.3函数 HAL_TIM_PWM_Init

函数原型：

```
HAL_StatusTypeDef HAL_TIM_PWM_Init(TIM_HandleTypeDef *htim)
{
  /* 检查句柄是否有效 */
  if(htim == NULL)
  {
    return HAL_ERROR;
  }

  /* 检测参数 */
  assert_param(IS_TIM_INSTANCE(htim->Instance));
  assert_param(IS_TIM_COUNTER_MODE(htim->Init.CounterMode));
  assert_param(IS_TIM_CLOCKDIVISION_DIV(htim->Init.ClockDivision));

  if(htim->State == HAL_TIM_STATE_RESET)
  {
    /* 默认取消锁 */
    htim->Lock = HAL_UNLOCKED;

    /* 初始底层 : GPIO, CLOCK, NVIC 和 DMA */
    HAL_TIM_PWM_MspInit(htim);
  }

  /* 设置定时状态 */
  htim->State= HAL_TIM_STATE_BUSY;

  /* 配置定时器用于 PWM */
  TIM_Base_SetConfig(htim->Instance, &htim->Init);

  /* 设置定时器状态 */
  htim->State= HAL_TIM_STATE_READY;

  return HAL_OK;
}
```

函数描述：

此函数用于初始化定时为 PWM 方式。

函数参数:

- 第 1 个参数是 TIM_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项:

1. 函数 HAL_TIM_PWM_MspInit 用于初始化定时器的底层时钟、引脚等功能。需要用户自己在此函数里面实现具体的功能，由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
2. 如果形参 htim 的结构体成员 gState 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 TIM_HandleTypeDef TimHandle。

   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_TIM_STATE_RESET = 0x00U。

   解决办法有三:

   方法 1：用户自己初始定时器和涉及到的 GPIO 等。

   方法 2：定义 TIM_HandleTypeDef TimHandle 为全局变量。

   方法 3：下面的方法

```
if(HAL_TIM_PWM_DeInit(&TimHandle) != HAL_OK)
{
    Error_Handler();
}
if(HAL_TIM_PWM_Init(&TimHandle) != HAL_OK)
{
    Error_Handler();
}
```

使用举例:

```
TIM_HandleTypeDef  TimHandle = {0};

/*  PWM 频率 = TIMxCLK / usPrescaler + 1) /usPeriod + 1) */
TimHandle.Instance = TIM1;
TimHandle.Init.Prescaler         = usPrescaler;
TimHandle.Init.Period            = usPeriod;
TimHandle.Init.ClockDivision     = 0;
TimHandle.Init.CounterMode       = TIM_COUNTERMODE_UP;
TimHandle.Init.RepetitionCounter = 0;
TimHandle.Init.AutoReloadPreload = 0;
if (HAL_TIM_PWM_Init(&TimHandle) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

32.4.4函数 HAL_TIM_PWM_ConfigChannel

函数原型：

```
HAL_StatusTypeDef HAL_TIM_PWM_ConfigChannel(TIM_HandleTypeDef *htim,
                                            TIM_OC_InitTypeDef* sConfig,
                                            uint32_t Channel)
{
   /* 省略 */

  /* 开锁 */
  __HAL_LOCK(htim);

  htim->State = HAL_TIM_STATE_BUSY;

  switch (Channel)
  {
    case TIM_CHANNEL_1:
    {
      /* 检查参数 */
      assert_param(IS_TIM_CC1_INSTANCE(htim->Instance));

      /* 配置通道 1 的 PWM 模式 */
      TIM_OC1_SetConfig(htim->Instance, sConfig);

      /*预装载使能，更新事件产生时写入有效 */
      htim->Instance->CCMR1 |= TIM_CCMR1_OC1PE;

      /* 配置是快速输出模式 */
      htim->Instance->CCMR1 &= ~TIM_CCMR1_OC1FE;
      htim->Instance->CCMR1 |= sConfig->OCFastMode;
    }
    break;

    case TIM_CHANNEL_2:
    {
       /* 省略 */
    }
    break;

    case TIM_CHANNEL_3:
    {
       /* 省略 */
    }
    break;

    case TIM_CHANNEL_4:
    {
      /* 省略 */
    }
    break;

    case TIM_CHANNEL_5:
    {
       /* 省略 */
    }
```

```
    break;

    case TIM_CHANNEL_6:
    {
        /* 省略 */
    }
    break;

    default:
    break;
  }

  htim->State = HAL_TIM_STATE_READY;

  /* 关锁 */
  __HAL_UNLOCK(htim);

  return HAL_OK;
}
```

函数描述：

此函数用于配置定时器的 PWM 通道。

函数参数：

- 第 1 个参数是 TIM_HandleTypeDef 类型结构体指针变量，用于定时器基本参数配置。
- 第 2 个参数是 TIM_OC_InitTypeDef 类型结构体指定变量，用于定时器输出比较参数配置。
- 第 3 个参数是通道设置，支持以下参数：

  TIM_CHANNEL_1

  TIM_CHANNEL_2

  TIM_CHANNEL_3

  TIM_CHANNEL_4

  TIM_CHANNEL_5

  TIM_CHANNEL_6
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
TIM_HandleTypeDef  TimHandle = {0};

/*  PWM 频率 = TIMxCLK / usPrescaler + 1) /usPeriod + 1) */
TimHandle.Instance = TIM1;
TimHandle.Init.Prescaler         = usPrescaler;
TimHandle.Init.Period            = usPeriod;
TimHandle.Init.ClockDivision     = 0;
TimHandle.Init.CounterMode       = TIM_COUNTERMODE_UP;
TimHandle.Init.RepetitionCounter = 0;
TimHandle.Init.AutoReloadPreload = 0;
if (HAL_TIM_PWM_Init(&TimHandle) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

```
/* 配置定时器 PWM 输出通道 */
sConfig.OCMode       = TIM_OCMODE_PWM1;
sConfig.OCPolarity   = TIM_OCPOLARITY_HIGH;
sConfig.OCFastMode   = TIM_OCFAST_DISABLE;
sConfig.OCNPolarity  = TIM_OCNPOLARITY_HIGH;
sConfig.OCNIdleState = TIM_OCNIDLESTATE_RESET;
sConfig.OCIdleState  = TIM_OCIDLESTATE_RESET;

/* 占空比 */
sConfig.Pulse = pulse;
if (HAL_TIM_PWM_ConfigChannel(&TimHandle, &sConfig, TimChannel[_ucChannel]) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

32.4.5 函数 HAL_TIM_PWM_Start

函数原型：

```
HAL_StatusTypeDef HAL_TIM_PWM_Start(TIM_HandleTypeDef *htim, uint32_t Channel)
{
  /* 检测参数 */
  assert_param(IS_TIM_CCX_INSTANCE(htim->Instance, Channel));

  /* 使能捕获比较通道 */
  TIM_CCxChannelCmd(htim->Instance, Channel, TIM_CCx_ENABLE);

  if(IS_TIM_BREAK_INSTANCE(htim->Instance) != RESET)
  {
    /* 使能主输出 */
    __HAL_TIM_MOE_ENABLE(htim);
  }

  /* 使能定时器 */
  __HAL_TIM_ENABLE(htim);

  /* 返回状态*/
  return HAL_OK;
}
```

函数描述：

此函数用于启动 PWM。

函数参数：

- 第 1 个参数是 TIM_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。
- 第 2 个参数是通道设置，支持以下参数：

  TIM_CHANNEL_1

  TIM_CHANNEL_2

  TIM_CHANNEL_3

  TIM_CHANNEL_4

- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
TIM_HandleTypeDef  TimHandle = {0};

/*  PWM 频率 = TIMxCLK / usPrescaler + 1) /usPeriod + 1) */
TimHandle.Instance = TIM1;
TimHandle.Init.Prescaler         = usPrescaler;
TimHandle.Init.Period            = usPeriod;
TimHandle.Init.ClockDivision     = 0;
TimHandle.Init.CounterMode       = TIM_COUNTERMODE_UP;
TimHandle.Init.RepetitionCounter = 0;
TimHandle.Init.AutoReloadPreload = 0;
if (HAL_TIM_PWM_Init(&TimHandle) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}

/* 配置定时器 PWM 输出通道 */
sConfig.OCMode       = TIM_OCMODE_PWM1;
sConfig.OCPolarity   = TIM_OCPOLARITY_HIGH;
sConfig.OCFastMode   = TIM_OCFAST_DISABLE;
sConfig.OCNPolarity  = TIM_OCNPOLARITY_HIGH;
sConfig.OCNIdleState = TIM_OCNIDLESTATE_RESET;
sConfig.OCIdleState  = TIM_OCIDLESTATE_RESET;

/* 占空比 */
sConfig.Pulse = pulse;
if (HAL_TIM_PWM_ConfigChannel(&TimHandle, &sConfig, TimChannel[_ucChannel]) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}

/* 启动 PWM 输出 */
if (HAL_TIM_PWM_Start(&TimHandle, TimChannel[_ucChannel]) != HAL_OK)
{
     Error_Handler(__FILE__, __LINE__);
}
```

32.4.6函数 HAL_TIM_IC_Init

函数原型：

```
HAL_StatusTypeDef HAL_TIM_IC_Init(TIM_HandleTypeDef *htim)
{
  /* 检测形参是否有效 */
  if(htim == NULL)
  {
    return HAL_ERROR;
  }

  /* 检测参数 */
  assert_param(IS_TIM_INSTANCE(htim->Instance));
  assert_param(IS_TIM_COUNTER_MODE(htim->Init.CounterMode));
```

```
  assert_param(IS_TIM_CLOCKDIVISION_DIV(htim->Init.ClockDivision));

  if(htim->State == HAL_TIM_STATE_RESET)
  {
    /* 默认取消锁 */
    htim->Lock = HAL_UNLOCKED;

     /* 初始底层 : GPIO, CLOCK, NVIC 和 DMA */
    HAL_TIM_IC_MspInit(htim);
  }

  /* 设置定时器状态 */
  htim->State= HAL_TIM_STATE_BUSY;

  /* 配置定时器为输入捕获模式 */
  TIM_Base_SetConfig(htim->Instance, &htim->Init);

  /* 设置定时器状态 */
  htim->State= HAL_TIM_STATE_READY;

  return HAL_OK;
}
```

函数描述：

此函数用于定时器输入捕获初始化。

函数参数：

- 第 1 个参数是 TIM_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 函数 HAL_TIM_IC_MspInit 用于初始化定时器的底层时钟、引脚等功能。需要用户自己在此函数里面实现具体的功能，由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
2. 如果形参 htim 的结构体成员 gState 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 TIM_HandleTypeDef TimHandle。

   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_TIM_STATE_RESET = 0x00U。

   解决办法有三：

   方法 1：用户自己初始定时器和涉及到的 GPIO 等。

   方法 2：定义 TIM_HandleTypeDef TimHandle 为全局变量。

   方法 3；下面的方法

```
if(HAL_TIM_IC_DeInit(&UartHandle) != HAL_OK)
{
    Error_Handler();
}
if(HAL_TIM_IC_Init(&UartHandle) != HAL_OK)
{
```

```
    Error_Handler();
}
```

32.4.7函数 HAL_TIM_IC_ConfigChannel

函数原型：

```
HAL_StatusTypeDef HAL_TIM_IC_ConfigChannel(TIM_HandleTypeDef *htim, TIM_IC_InitTypeDef* sConfig, uint32_t
Channel)
{
  /* 检查参数 */
  assert_param(IS_TIM_CC1_INSTANCE(htim->Instance));
  assert_param(IS_TIM_IC_POLARITY(sConfig->ICPolarity));
  assert_param(IS_TIM_IC_SELECTION(sConfig->ICSelection));
  assert_param(IS_TIM_IC_PRESCALER(sConfig->ICPrescaler));
  assert_param(IS_TIM_IC_FILTER(sConfig->ICFilter));

  /* 开锁 */
  __HAL_LOCK(htim);

  htim->State = HAL_TIM_STATE_BUSY;

  if (Channel == TIM_CHANNEL_1)
  {
    /* 配置输入通道 1 */
    TIM_TI1_SetConfig(htim->Instance,
                sConfig->ICPolarity,
                sConfig->ICSelection,
                sConfig->ICFilter);

    /* 清零 IC1PSC 位 */
    htim->Instance->CCMR1 &= ~TIM_CCMR1_IC1PSC;

    /* 根据用户配置，设置分频 */
    htim->Instance->CCMR1 |= sConfig->ICPrescaler;
  }
  else if (Channel == TIM_CHANNEL_2)
  {
     /* 省略 */
  }
  else if (Channel == TIM_CHANNEL_3)
  {
     /* 省略 */
  }
  else
  {
     /* 省略 */
  }

  htim->State = HAL_TIM_STATE_READY;

  __HAL_UNLOCK(htim);

  return HAL_OK;
}
```

函数描述：

此函数用于配置定时器的输入捕获通道。

函数参数：

- 第 1 个参数是 TIM_HandleTypeDef 类型结构体指针变量，用于定时器基本参数配置
- 第 2 个参数是 TIM_IC_InitTypeDef 类型结构体指定变量，用于定时器输出比较参数配置。
- 第 3 个参数是通道设置，支持以下参数：

  TIM_CHANNEL_1

  TIM_CHANNEL_2

  TIM_CHANNEL_3

  TIM_CHANNEL_4
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

32.4.8函数 HAL_TIM_IC_Start_IT

函数原型：

```
HAL_StatusTypeDef HAL_TIM_IC_Start_IT (TIM_HandleTypeDef *htim, uint32_t Channel)
{
  /* 检查参数 */
  assert_param(IS_TIM_CCX_INSTANCE(htim->Instance, Channel));

  switch (Channel)
  {
    case TIM_CHANNEL_1:
    {
      /* 使能 CC1 (Capture/Compare 1) 中断 */
      __HAL_TIM_ENABLE_IT(htim, TIM_IT_CC1);
    }
    break;

    case TIM_CHANNEL_2:
    {
        /* 省略 */
    }
    break;

    case TIM_CHANNEL_3:
    {
       /* 省略 */
    }
    break;

    case TIM_CHANNEL_4:
    {
       /* 省略 */
    }
    break;
```

```
    default:
    break;
  }
  /* 使能输入捕获通道 */
  TIM_CCxChannelCmd(htim->Instance, Channel, TIM_CCx_ENABLE);

  /* 使能定时器 */
  __HAL_TIM_ENABLE(htim);

  /* 返回状态 */
  return HAL_OK;
}
```

函数描述：

此函数用于启动定时器输入捕获模式，采用定时器方式。

函数参数：

- 第 1 个参数是 TIM_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。
- 第 2 个参数是通道设置，支持以下参数：

  TIM_CHANNEL_1

  TIM_CHANNEL_2

  TIM_CHANNEL_3

  TIM_CHANNEL_4
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

32.4.9函数 HAL_TIM_OC_Init

函数原型：

```
HAL_StatusTypeDef HAL_TIM_OC_Init(TIM_HandleTypeDef* htim)
{
  /* 检测形参是否有效 Check */
  if(htim == NULL)
  {
    return HAL_ERROR;
  }

  /* 检查参数 */
  assert_param(IS_TIM_INSTANCE(htim->Instance));
  assert_param(IS_TIM_COUNTER_MODE(htim->Init.CounterMode));
  assert_param(IS_TIM_CLOCKDIVISION_DIV(htim->Init.ClockDivision));

  if(htim->State == HAL_TIM_STATE_RESET)
  {
    /* 默认取消锁 */
    htim->Lock = HAL_UNLOCKED;

    /* 初始底层 : GPIO, CLOCK, NVIC 和 DMA */
    HAL_TIM_OC_MspInit(htim);
```

```
  }

  /* 设置定时器状态 Set the TIM state */
  htim->State= HAL_TIM_STATE_BUSY;

  /* 配置定时器为输出比较模式 Init the base time for the Output Compare */
  TIM_Base_SetConfig(htim->Instance,  &htim->Init);

  /* 设置定时器状态 */
  htim->State= HAL_TIM_STATE_READY;

  return HAL_OK;
}
```

函数描述：

此函数用于定时器输入捕获初始化。

函数参数：

- 第 1 个参数是 TIM_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 函数 HAL_TIM_OC_MspInit 用于初始化定时器的底层时钟、引脚等功能。需要用户自己在此函数里面实现具体的功能，由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
2. 如果形参 htim 的结构体成员 gState 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 TIM_HandleTypeDef TimHandle。

   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_TIM_STATE_RESET = 0x00U。

   解决办法有三：

   方法 1：用户自己初始定时器和涉及到的 GPIO 等。

   方法 2：定义 TIM_HandleTypeDef TimHandle 为全局变量。

   方法 3；下面的方法

```
if(HAL_TIM_OC_DeInit(&UartHandle) != HAL_OK)
{
    Error_Handler();
}
if(HAL_TIM_OC_Init(&UartHandle) != HAL_OK)
{
    Error_Handler();
}
```

32.4.10 函数 HAL_TIM_OC_ConfigChannel

函数原型：

```
HAL_StatusTypeDef HAL_TIM_OC_ConfigChannel(TIM_HandleTypeDef *htim,
                                           TIM_OC_InitTypeDef* sConfig,
                                           uint32_t Channel)
{
  /* Check the parameters */
  assert_param(IS_TIM_CHANNELS(Channel));
  assert_param(IS_TIM_OC_MODE(sConfig->OCMode));
  assert_param(IS_TIM_OC_POLARITY(sConfig->OCPolarity));

  /* Process Locked */
  __HAL_LOCK(htim);

  htim->State = HAL_TIM_STATE_BUSY;

  switch (Channel)
  {
    case TIM_CHANNEL_1:
    {
      /* 检测参数 Check the parameters */
      assert_param(IS_TIM_CC1_INSTANCE(htim->Instance));

     /* 配置定时器输出比较通道 1 */
      TIM_OC1_SetConfig(htim->Instance, sConfig);
    }
    break;

    case TIM_CHANNEL_2:
    {
      /* 省略 */
    }
    break;

    case TIM_CHANNEL_3:
    {
      /* 省略 */
    }
    break;

    case TIM_CHANNEL_4:
    {
      /* 省略 */
    }
    break;

    case TIM_CHANNEL_5:
    {
      /* 省略 */
    }
    break;

    case TIM_CHANNEL_6:
    {
      /* 省略 */
    }
    break;

    default:
    break;
```

```
  }

  htim->State = HAL_TIM_STATE_READY;

  __HAL_UNLOCK(htim);

  return HAL_OK;
}
```

函数描述：

此函数用于初始化串口的基础特性和高级特性。

函数参数：

- 第 1 个参数是 TIM_HandleTypeDef 类型结构体指针变量，用于定时器基本参数配置。
- 第 2 个参数是 TIM_OC_InitTypeDef 类型结构体指定变量，用于定时器输出比较参数配置。
- 第 3 个参数是通道设置，支持以下参数：

  TIM_CHANNEL_1

  TIM_CHANNEL_2

  TIM_CHANNEL_3

  TIM_CHANNEL_4

  TIM_CHANNEL_5

  TIM_CHANNEL_6
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

32.4.11 函数 HAL_TIM_OC_Start

函数原型：

```
HAL_StatusTypeDef HAL_TIM_OC_Start(TIM_HandleTypeDef *htim, uint32_t Channel)
{
  /* 检查参数 */
  assert_param(IS_TIM_CCX_INSTANCE(htim->Instance, Channel));

  /* 使能输出比较通道 */
  TIM_CCxChannelCmd(htim->Instance, Channel, TIM_CCx_ENABLE);

  if(IS_TIM_BREAK_INSTANCE(htim->Instance) != RESET)
  {
    /* 使能主输出 */
    __HAL_TIM_MOE_ENABLE(htim);
  }

  /* 使能定时器 */
  __HAL_TIM_ENABLE(htim);

  /* 返回状态 */
  return HAL_OK;
}
```

函数描述：

此函数用于启动定时器输出比较模式。

函数参数：

- 第 1 个参数是 TIM_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。
- 第 2 个参数是通道设置，支持以下参数：

  TIM_CHANNEL_1

  TIM_CHANNEL_2

  TIM_CHANNEL_3

  TIM_CHANNEL_4
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

32.5 总结

本章节就为大家讲解这么多，建议大家将 GPIO 的驱动源码结合参考手册中的寄存器通读一遍，对于我们后面章节的学习大有裨益。

第33章 STM32H7 的定时器应用之 TIM1-TIM17 的中断实现

本章教程为大家讲解定时器应用之 TIM1 – TIM17 所有定时器的周期性中断实现。实际项目中用到的地方较多，特别是周期性的事件查询。

33.1 初学者重要提示
33.2 定时器中断的驱动设计
33.3 定时器板级支持包（bsp_tim_pwm.c）
33.4 定时器驱动移植和使用
33.5 实验例程设计框架
33.6 实验例程说明（MDK）
33.7 实验例程说明（IAR）
33.8 总结

33.1 初学者重要提示

- 学习本章节前，务必优先学习第 32 章，HAL 库的几个常用 API 均作了讲解和举例。
- STM32H7 支持 TIM1-TIM8，TIM12-TIM17 共 14 个定时器，而中间的 TIM9，TIM10，TIM11 是不存在的，这点要注意。
- STM32H7 的进出中断的速度能跑到 12.5MHz，所有程序在 TCM 和 Flash 运行没差别，详情可看本章 2.3 小节。
- 实际应用中，中断入口函数名称不要写错，有些中断的入口函数名称比较特殊，详情可看本章的 2.2 小节。

33.2 定时器中断的驱动设计

定时器中断的实现相对比较简单，仅需一个函数即可实现 TIM1-TIM17 定时器的中断更新配置。

33.2.1 定时器中断初始化

实现代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RCC_TIM_Enable
*    功能说明: 使能 TIM RCC 时钟
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RCC_TIM_Enable(TIM_TypeDef* TIMx)
{
    if (TIMx == TIM1) __HAL_RCC_TIM1_CLK_ENABLE();
    else if (TIMx == TIM2) __HAL_RCC_TIM2_CLK_ENABLE();
    else if (TIMx == TIM3) __HAL_RCC_TIM3_CLK_ENABLE();
    else if (TIMx == TIM4) __HAL_RCC_TIM4_CLK_ENABLE();
    else if (TIMx == TIM5) __HAL_RCC_TIM5_CLK_ENABLE();
    else if (TIMx == TIM6) __HAL_RCC_TIM6_CLK_ENABLE();
    else if (TIMx == TIM7) __HAL_RCC_TIM7_CLK_ENABLE();
    else if (TIMx == TIM8) __HAL_RCC_TIM8_CLK_ENABLE();
//  else if (TIMx == TIM9) __HAL_RCC_TIM9_CLK_ENABLE();
//  else if (TIMx == TIM10) __HAL_RCC_TIM10_CLK_ENABLE();
//  else if (TIMx == TIM11) __HAL_RCC_TIM11_CLK_ENABLE();
    else if (TIMx == TIM12) __HAL_RCC_TIM12_CLK_ENABLE();
    else if (TIMx == TIM13) __HAL_RCC_TIM13_CLK_ENABLE();
    else if (TIMx == TIM14) __HAL_RCC_TIM14_CLK_ENABLE();
    else if (TIMx == TIM15) __HAL_RCC_TIM15_CLK_ENABLE();
    else if (TIMx == TIM16) __HAL_RCC_TIM16_CLK_ENABLE();
    else if (TIMx == TIM17) __HAL_RCC_TIM17_CLK_ENABLE();
    else
    {
        Error_Handler(__FILE__, __LINE__);
    }
}

/*
*********************************************************************************************************
*    函 数 名: bsp_SetTIMforInt
*    功能说明: 配置 TIM 和 NVIC，用于简单的定时中断，开启定时中断。另外注意中断服务程序需要由用户应
*              用程序实现。
*    形    参: TIMx : 定时器
*              _ulFreq : 定时频率 （Hz）。 0 表示关闭。
*              _PreemptionPriority : 抢占优先级
*              _SubPriority : 子优先级
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_SetTIMforInt(TIM_TypeDef* TIMx, uint32_t _ulFreq, uint8_t _PreemptionPriority,
                      uint8_t _SubPriority)
{
    TIM_HandleTypeDef   TimHandle = {0};
    uint16_t usPeriod;
    uint16_t usPrescaler;
    uint32_t uiTIMxCLK;

    /* 使能 TIM 时钟 */
    bsp_RCC_TIM_Enable(TIMx);

   /*-----------------------------------------------------------------------
        bsp.c 文件中 void SystemClock_Config(void) 函数对时钟的配置如下:
```

```
59.
60.         System Clock source       = PLL (HSE)
61.         SYSCLK(Hz)                = 400000000 (CPU Clock)
62.         HCLK(Hz)                  = 200000000 (AXI and AHBs Clock)
63.         AHB Prescaler             = 2
64.         D1 APB3 Prescaler         = 2 (APB3 Clock  100MHz)
65.         D2 APB1 Prescaler         = 2 (APB1 Clock  100MHz)
66.         D2 APB2 Prescaler         = 2 (APB2 Clock  100MHz)
67.         D3 APB4 Prescaler         = 2 (APB4 Clock  100MHz)
68.
69.         因为 APB1 prescaler != 1, 所以 APB1 上的 TIMxCLK = APB1 x 2 = 200MHz;
70.         因为 APB2 prescaler != 1, 所以 APB2 上的 TIMxCLK = APB2 x 2 = 200MHz;
71.         APB4 上面的 TIMxCLK 没有分频，所以就是 100MHz;
72.
73.         APB1 定时器有 TIM2, TIM3 ,TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14，LPTIM1
74.         APB2 定时器有 TIM1, TIM8 , TIM15, TIM16，TIM17
75.
76.         APB4 定时器有 LPTIM2，LPTIM3，LPTIM4，LPTIM5
77.       ------------------------------------------------------------------------ */
78.       if ((TIMx == TIM1) || (TIMx == TIM8) || (TIMx == TIM15) || (TIMx == TIM16) || (TIMx == TIM17))
79.       {
80.           /* APB2 定时器时钟 = 200M */
81.           uiTIMxCLK = SystemCoreClock / 2;
82.       }
83.       else
84.       {
85.           /* APB1 定时器 = 200M */
86.           uiTIMxCLK = SystemCoreClock / 2;
87.       }
88.
89.       if (_ulFreq < 100)
90.       {
91.           usPrescaler = 10000 - 1;                      /* 分频比 = 10000 */
92.           usPeriod =  (uiTIMxCLK / 10000) / _ulFreq  - 1;         /* 自动重装的值 */
93.       }
94.       else if (_ulFreq < 3000)
95.       {
96.           usPrescaler = 100 - 1;                    /* 分频比 = 100 */
97.           usPeriod =  (uiTIMxCLK / 100) / _ulFreq  - 1;       /* 自动重装的值 */
98.       }
99.       else /* 大于 4K 的频率，无需分频 */
100.      {
101.          usPrescaler = 0;                      /* 分频比 = 1 */
102.          usPeriod = uiTIMxCLK / _ulFreq - 1;  /* 自动重装的值 */
103.      }
104.
105.      /*
106.        定时器中断更新周期 = TIMxCLK / usPrescaler + 1）/usPeriod + 1）
107.      */
108.      TimHandle.Instance = TIMx;
109.      TimHandle.Init.Prescaler         = usPrescaler;
110.      TimHandle.Init.Period            = usPeriod;
111.      TimHandle.Init.ClockDivision     = 0;
112.      TimHandle.Init.CounterMode       = TIM_COUNTERMODE_UP;
113.      TimHandle.Init.RepetitionCounter = 0;
114.      TimHandle.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
115.      if (HAL_TIM_Base_Init(&TimHandle) != HAL_OK)
116.      {
```

```
117.          Error_Handler(__FILE__, __LINE__);
118.      }
119.
120.      /* 使能定时器中断  */
121.      __HAL_TIM_ENABLE_IT(&TimHandle, TIM_IT_UPDATE);
122.
123.
124.      /* 配置 TIM 定时更新中断 (Update) */
125.      {
126.          uint8_t irq = 0;   /* 中断号, 定义在 stm32h7xx.h */
127.
128.          if (TIMx == TIM1) irq = TIM1_UP_IRQn;
129.          else if (TIMx == TIM2) irq = TIM2_IRQn;
130.          else if (TIMx == TIM3) irq = TIM3_IRQn;
131.          else if (TIMx == TIM4) irq = TIM4_IRQn;
132.          else if (TIMx == TIM5) irq = TIM5_IRQn;
133.          else if (TIMx == TIM6) irq = TIM6_DAC_IRQn;
134.          else if (TIMx == TIM7) irq = TIM7_IRQn;
135.          else if (TIMx == TIM8) irq = TIM8_UP_TIM13_IRQn;
136.          else if (TIMx == TIM12) irq = TIM8_BRK_TIM12_IRQn;
137.          else if (TIMx == TIM13) irq = TIM8_UP_TIM13_IRQn;
138.          else if (TIMx == TIM14) irq = TIM8_TRG_COM_TIM14_IRQn;
139.          else if (TIMx == TIM15) irq = TIM15_IRQn;
140.          else if (TIMx == TIM16) irq = TIM16_IRQn;
141.          else if (TIMx == TIM16) irq = TIM17_IRQn;
142.          else
143.          {
144.              Error_Handler(__FILE__, __LINE__);
145.          }
146.          HAL_NVIC_SetPriority((IRQn_Type)irq, _PreemptionPriority, _SubPriority);
147.          HAL_NVIC_EnableIRQ((IRQn_Type)irq);
148.      }
149.
150.      HAL_TIM_Base_Start(&TimHandle);
151. }
```

程序中的注释已经比较详细，这里把几个关键的地方再阐释下：

- 第 9- 32 行，函数 bsp_RCC_TIM_Enable 用于获取要使能的定时器时钟。
- 第 49 行，HAL 库的定时器句柄变量要初始化为 0，这个问题在教程上一章的 4.1 小节有专门说明。
- 第 78 – 103 行，计算出要配置的分频和周期。这里要注意一点，因为除了 TIM2 和 TIM5，其它定时器都是 16 位的，相关寄存器大部分也都是 16 位的，配置的时候不可以超出 0 -65535。这里分频变量 usPrescaler 和周期变量 usPeriod 统一按照 16 位计算，所以有了这几行代码做频率区分，防止超出范围。
- 第 108 – 118 行，通过函数 HAL_TIM_Base_Init 初始化定时器的基本功能。
- 第 125 – 148 行，配置定时器中断的优先级，并使能中断。

33.2.2 定时器中断服务程序

定时器初始化完毕了，定时器中断服务程序不要忘了写。比如使用定时器 6 的中断。

```
/*
```

```
*********************************************************************************************************
*    函 数 名: TIM6_DAC_IRQHandler
*    功能说明: TIM6 定时中断服务程序
*    返 回 值: 无
*********************************************************************************************************
*/
void TIM6_DAC_IRQHandler(void)
{
    if((TIM6->SR & TIM_FLAG_UPDATE) != RESET)
    {
        /* 清除标志 */
        TIM6->SR = ~ TIM_FLAG_UPDATE;
        /* 添加用户程序 */
    }
}
```

使用定时器中断不要把中断入口函数的名字写错了,比如这个定时器 6,很容易错搞成 TIM6__IRQHandler。

TIM1 – TIM17 中断入口名如下（在 startup_stm32h743xx.s 文件里面有弱定义）：

```
TIM1_BRK_IRQHandler
TIM1_UP_IRQHandler
TIM1_TRG_COM_IRQHandler
TIM1_CC_IRQHandler
TIM2_IRQHandler
TIM3_IRQHandler
TIM4_IRQHandler
TIM5_IRQHandler
TIM6_DAC_IRQHandler
TIM7_IRQHandler
TIM8_BRK_TIM12_IRQHandler         /* 注意这里是 TIM8 BRK 和 TIM12 公用 */
TIM8_UP_TIM13_IRQHandler          /* 注意这里是 TIM8 UP 和 TIM13 公用 */
TIM8_TRG_COM_TIM14_IRQHandler     /* 注意这里是 TIM8 TRG COM 和 TIM14 公用 */
TIM8_CC_IRQHandler
TIM15_IRQHandler
TIM16_IRQHandler
TIM17_IRQHandler
```

33.2.3 定时器中断的最高频率

有时候我们希望定时器的中断服务程序执行频率越快越好，这样可以方便的在中断里面执行一些特定功能，比如控制引脚输出指定个数的 IO 脉冲，使用定时器中断就可以方便的实现。

测试时开启 MDK 的最高等级优化和时间优化。

测试下面情况下，性能没差别：

- 程序在 Flash 运行，变量在 DTCM，开启 Cache。
- 程序和变量都在 DTCM 运行。

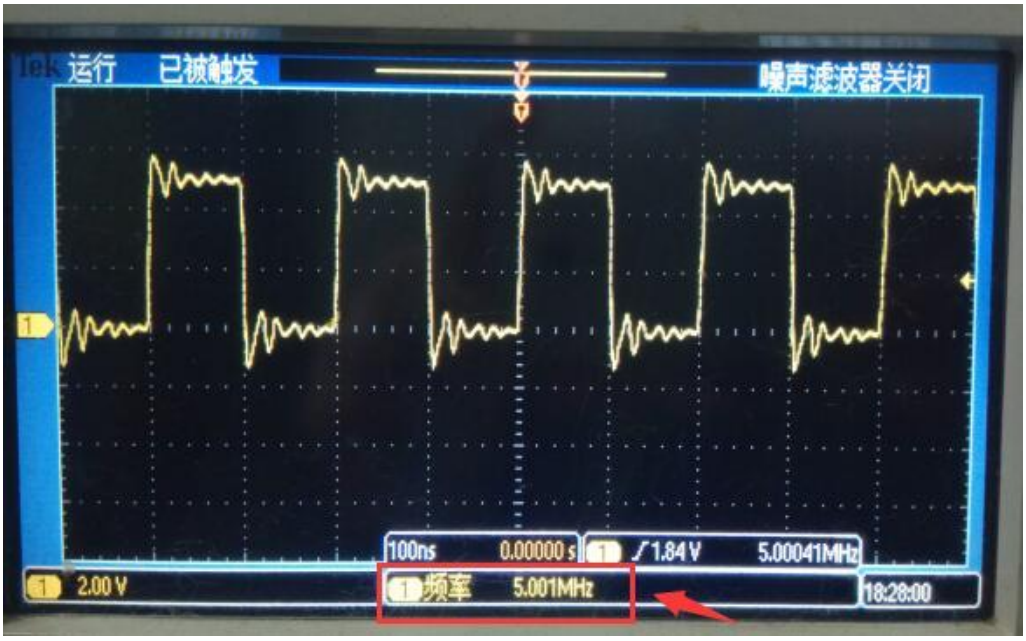
中断部分的测试程序：

```
/*
*********************************************************************************************************
*        函 数 名: TIM6_DAC_IRQHandler
*        功能说明: TIM6 定时中断服务程序
*        返 回 值: 无
*********************************************************************************************************
```

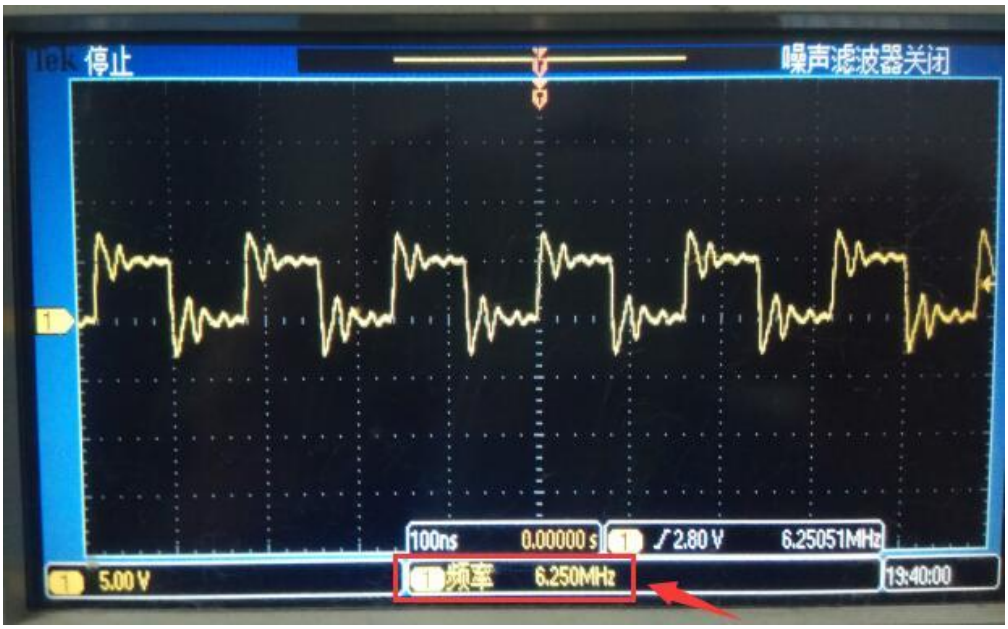
```
*/
void TIM6_DAC_IRQHandler(void)
{
	TIM6->SR = ~TIM_FLAG_UPDATE;
	//GPIOB->ODR ^= GPIO_PIN_1;    /* 使用通用 GPIO */
	HC574_TogglePin(GPIO_PIN_23); /* 使用的 FMC 扩展 IO */
}
```

测试结果是 STM32H7 的进出中断的速度能跑到 12.5MHz，所有程序在 TCM 和 Flash 运行没差别。

IO 翻转 10MHz，方波频率 5MHz：



IO 翻转 12.5MHz，方波频率 6.25MHz：



12.5Hz 是最高频率，实际应用中别跑这么高，因为这个频率下，程序基本一直在执行中断服务程序。实际应用做个 1MHz 及其以下还是没问题的。

33.3 定时器板级支持包（bsp_tim_pwm.c)

定时器驱动文件 bsp_tim_pwm.c 主要实现了如下两个 API 供用户调用：

- bsp_SetTIMOutPWM
- bsp_SetTIMforInt

这两个函数都是 TIM1-TIM17 所有定时器都支持，函数 bsp_SetTIMOutPWM 用于 PWM，下个章节为大家讲解，本小节主要把函数 bsp_SetTIMforInt 做个说明。

33.3.1 函数 bsp_SetTIMforInt

函数原型：

```
void bsp_SetTIMforInt(TIM_TypeDef* TIMx, uint32_t _ulFreq, uint8_t _PreemptionPriority, uint8_t _SubPriority)
```

函数描述：

此函数主要用配置定时器周期性中断。

函数参数：

- 第 1 个参数用于指定使用那个定时器，参数可以是 TIM1 – TIM17 所有定时器（不含 TIM9，TIM10 和 TIM11，因为 STM32H7 不支持这三个定时器）。
- 第 2 个参数是要实现的定时器中断频率，单位 Hz，如果填 0 的话，表示关闭。
- 第 3 个参数是定时器抢占式优先级，范围 0 – 15。
- 第 4 个参数是定时器子优先级，范围 0 – 15。

注意事项：

1. 定时器中断频率最好别超过 10MHz，本章 2.3 小节有说明。
2. 初始化后，别忘了写对应的中断服务程序。

使用举例：

比如使用定时器 6 设置为 20Hz 频率，周期 0.05 秒定时中断：

```
bsp_SetTIMforInt(TIM6, 20, 2, 0);
```

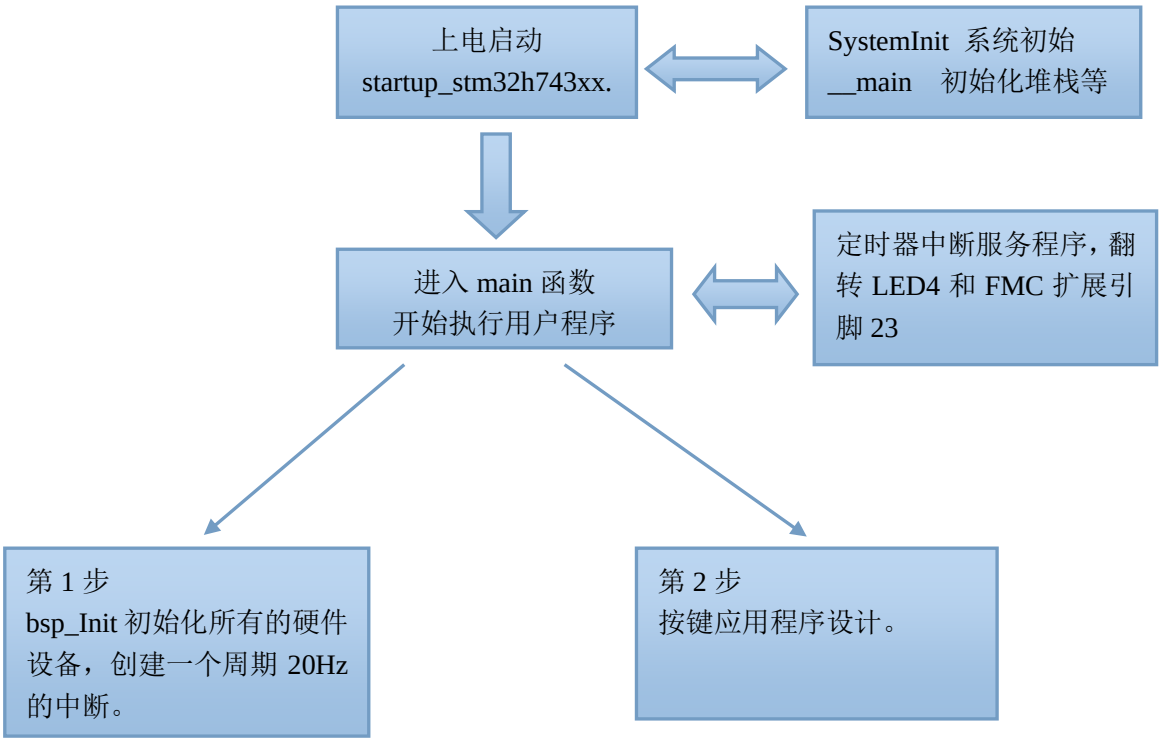
33.4 定时器驱动移植和使用

定时器的移植比较简单：

- 第 1 步：复制 bsp_tim_pwm.c 和 bsp_tim_pwm.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：这几个驱动文件主要用到 HAL 库的 GPIO 和 TIM 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 3 步，应用方法看本章节配套例子即可。

33.5 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED 和串口。
  - 第 2 步，按键应用程序设计部分。
  - 定时器中断服务程序里面实现翻转 LED4 和 FMC 扩展引脚 23。

33.6 实验例程说明（MDK）

配套例子：

V7-018_定时器周期性中断（驱动支持 TIM1-TIM17）

实验目的：

1. 学习定时器周期性中断实现，支持 TIM1-TIM17 所有定时器。

实验内容：

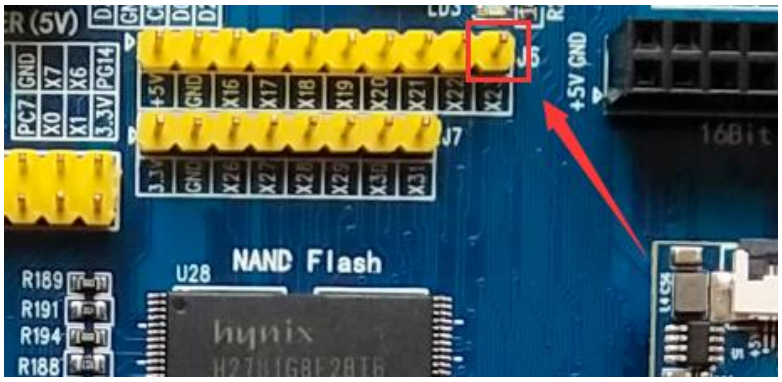
1. 系统上电后驱动了 1 个软件定时器，每 100ms 翻转一次 LED2，同时启动 1 个 TIM6 周期性中断，每 50ms 执行一次，在中断服务程序里面翻转 LED4 和 FMC 扩展引脚 23。

2. STM32H7 支持 TIM1-TIM8，TIM12-TIM17 共 14 个定时器，而中间的 TIM9，TIM10，TIM11 是不存在的。
3. STM32H7 的进出中断的速度能跑到 12.5MHz，所有程序在 TCM 和 Flash 运行没差别，实际应用中最好别超过 1MHz。
4. 中断入口函数名称不要写错，有些中断的入口函数名称比较特殊，详情可看 V7 开发板用户手册。

实验操作：

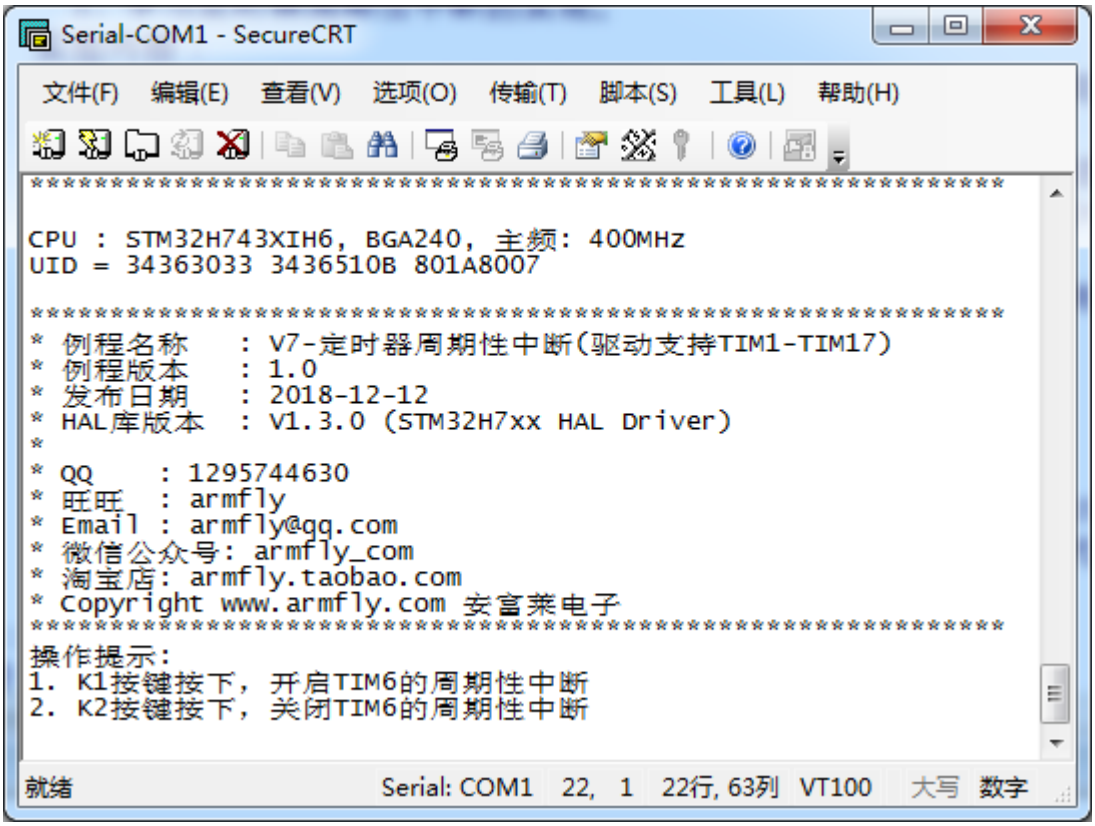
1. K1 按键按下，开启 TIM6 的周期性中断。
2. K2 按键按下，关闭 TIM6 的周期性中断。

FMC 扩展引脚 23 的位置：



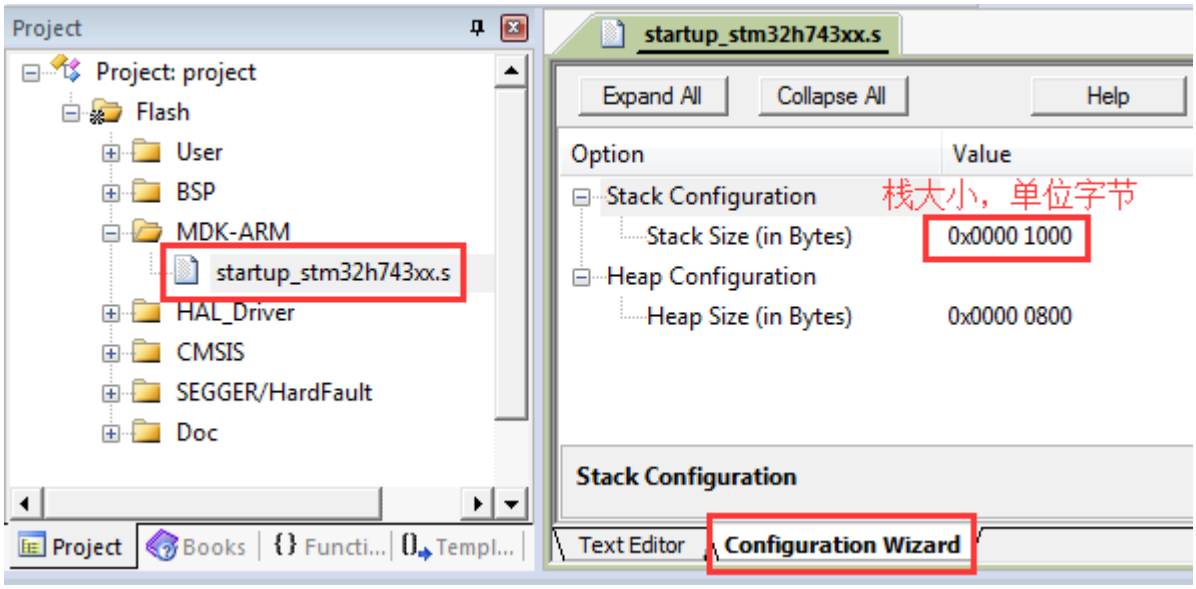
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

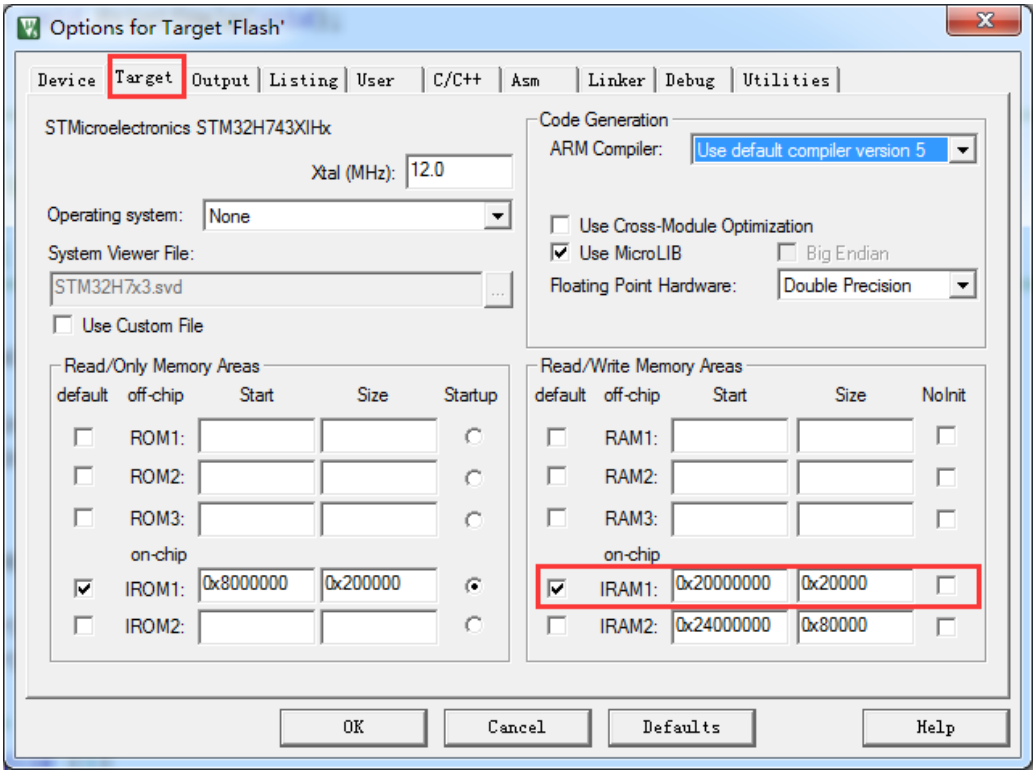


程序设计：

◆ **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
```

```
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **定时器中断服务程序：**

定时器 6 的中断服务程序如下，主要实现了 LED4 翻转和 FMC 扩展引脚 23 的翻转：

```
/*
```

```
*********************************************************************************************************
*    函 数 名: TIM6_DAC_IRQHandler
*    功能说明: TIM6 定时中断服务程序
*    返 回 值: 无
*********************************************************************************************************
*/
void TIM6_DAC_IRQHandler(void)
{
    if((TIM6->SR & TIM_FLAG_UPDATE) != RESET)
    {
        /* 清除更新标志 */
        TIM6->SR = ~ TIM_FLAG_UPDATE;

        /* 翻转 LED4 和 FMC 扩展引脚 23 */
        bsp_LedToggle(4);
        HC574_TogglePin(GPIO_PIN_23);
    }
}
```

◆ **主功能：**

主程序实现如下操作：

- K1 按键按下，开启 TIM6 的周期性中断。
- K2 按键按下，关闭 TIM6 的周期性中断。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode; /* 按键代码 */


    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    bsp_SetTIMforInt(TIM6, 20, 2, 0);     /* 设置为 20Hz 频率, 周期 0.05 秒定时中断*/

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 50ms 进来一次 */
            bsp_LedToggle(2);
        }
```

```
        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值，无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下 */
                    TIM6->DIER |= TIM_IT_UPDATE;
                    break;

                case KEY_DOWN_K2:           /* K2 键按下 */
                    TIM6->DIER &= ~TIM_IT_UPDATE;
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

33.7 实验例程说明（IAR）

配套例子：

V7-018_定时器周期性中断（驱动支持 TIM1-TIM17）

实验目的：

1. 学习定时器周期性中断实现，支持 TIM1-TIM17 所有定时器。

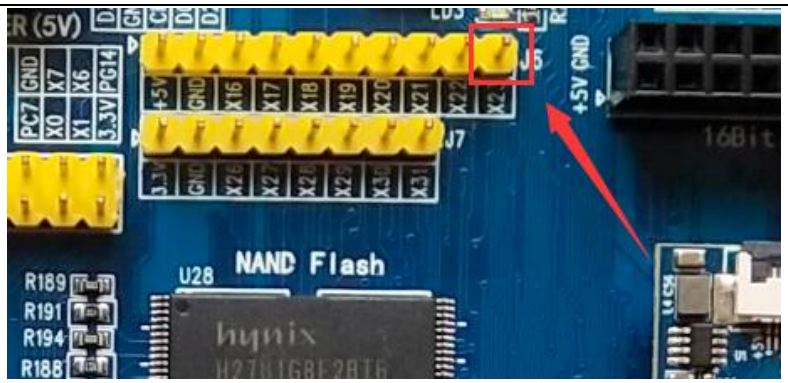
实验内容：

1. 系统上电后驱动了 1 个软件定时器，每 100ms 翻转一次 LED2，同时启动 1 个 TIM6 周期性中断，每 50ms 执行一次，在中断服务程序里面翻转 LED4 和 FMC 扩展引脚 23。
2. STM32H7 支持 TIM1-TIM8，TIM12-TIM17 共 14 个定时器，而中间的 TIM9，TIM10，TIM11 是不存在的。
3. STM32H7 的进出中断的速度能跑到 12.5MHz，所有程序在 TCM 和 Flash 运行没差别，实际应用中最好别超过 1MHz。
4. 中断入口函数名称不要写错，有些中断的入口函数名称比较特殊，详情可看 V7 开发板用户手册。

实验操作：

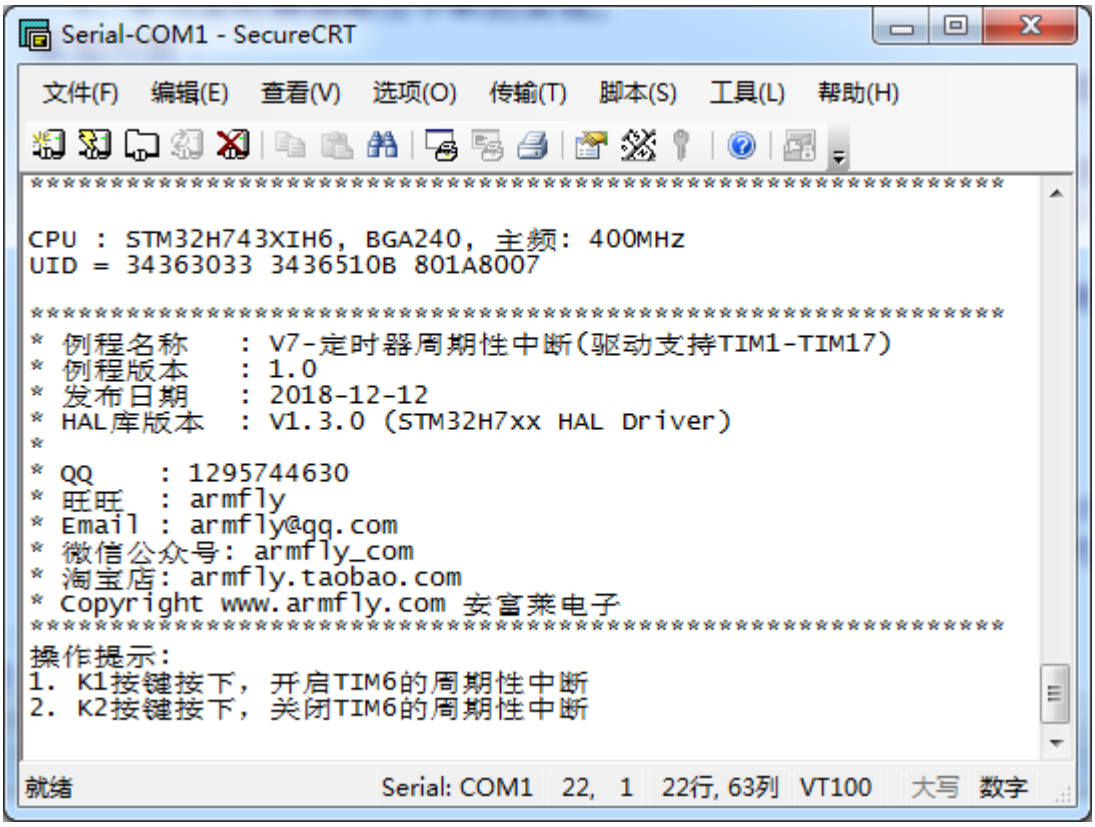
1. K1 按键按下，开启 TIM6 的周期性中断。
2. K2 按键按下，关闭 TIM6 的周期性中断。

FMC 扩展引脚 23 的位置：



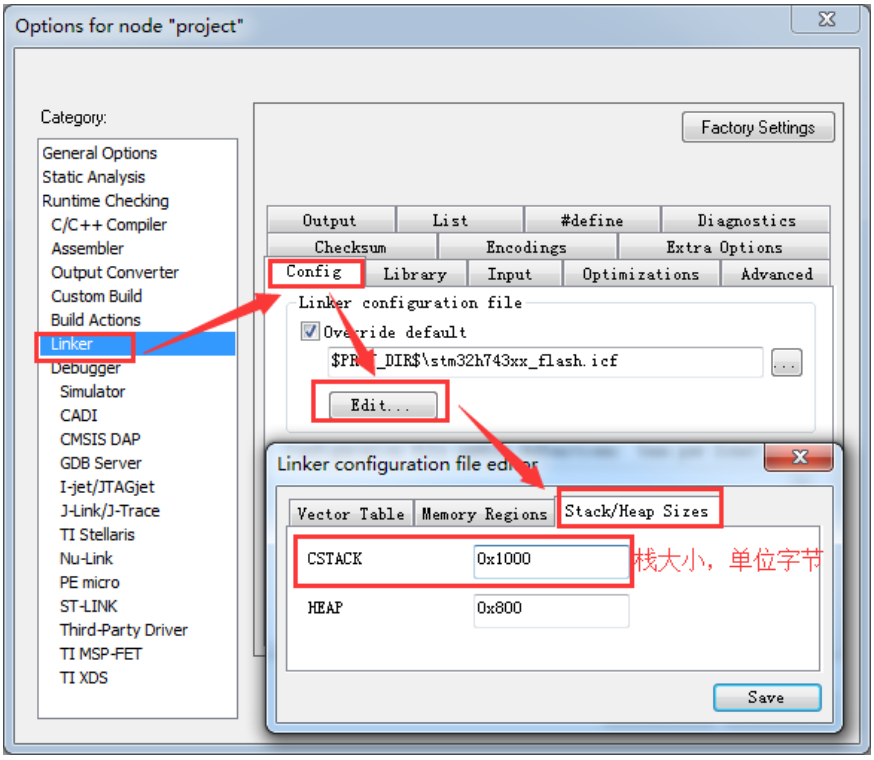
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

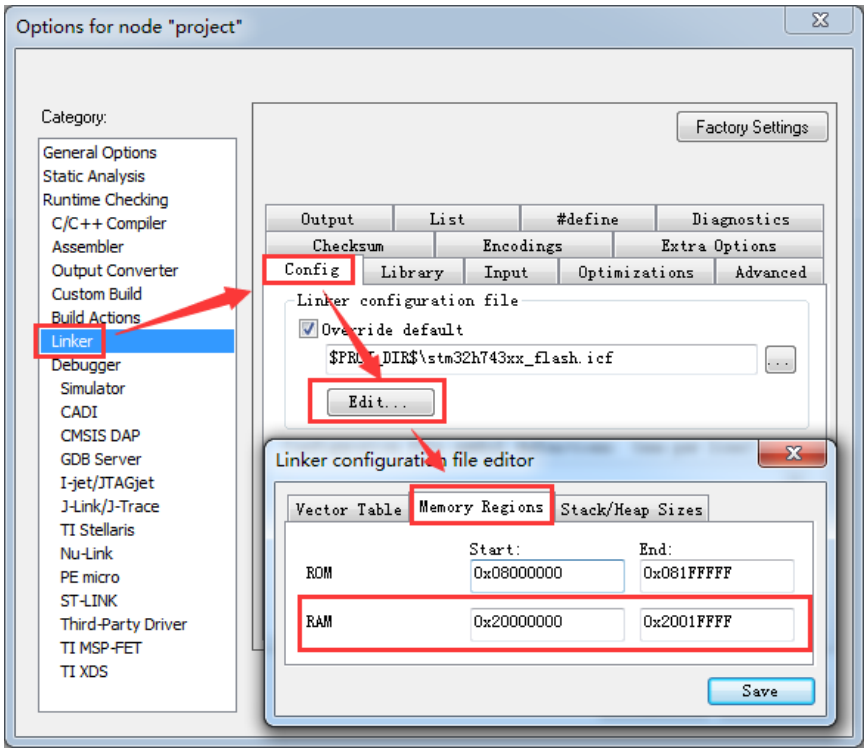


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
```

```
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
```

```
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **定时器中断服务程序：**

定时器 6 的中断服务程序如下，主要实现了 LED4 翻转和 FMC 扩展引脚 23 的翻转：

```
/*
*********************************************************************************************************
*    函 数 名: TIM6_DAC_IRQHandler
*    功能说明: TIM6 定时中断服务程序
*    返 回 值: 无
*********************************************************************************************************
*/
void TIM6_DAC_IRQHandler(void)
{
    if((TIM6->SR & TIM_FLAG_UPDATE) != RESET)
    {
        /* 清除更新标志 */
        TIM6->SR = ~ TIM_FLAG_UPDATE;

        /* 翻转 LED4 和 FMC 扩展引脚 23 */
        bsp_LedToggle(4);
        HC574_TogglePin(GPIO_PIN_23);
    }
}
```

◆ **主功能：**

主程序实现如下操作：

- K1 按键按下，开启 TIM6 的周期性中断。
- K2 按键按下，关闭 TIM6 的周期性中断。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode; /* 按键代码 */


    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    bsp_SetTIMforInt(TIM6, 20, 2, 0);     /* 设置为 20Hz 频率, 周期 0.05 秒定时中断*/

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
```

```
        {
            /* 每隔 50ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下 */
                    TIM6->DIER |= TIM_IT_UPDATE;
                    break;

                case KEY_DOWN_K2:           /* K2 键按下 */
                    TIM6->DIER &= ~TIM_IT_UPDATE;
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

33.8 总结

本章节就为大家讲解这么多，相对比较容易掌握，望初学者熟练运用。

第34章 STM32H7 的定时器应用之 TIM1-TIM17 的 PWM 实现

本章教程为大家讲解定时器应用之 TIM1 – TIM17 所有定时器的 PWM 实现。实际项目中用到的地方较多，如电机控制、无源蜂鸣器、显示屏背光等场合。

34.1 初学者重要提示
34.2 定时器 PWM 驱动设计
34.3 定时器板级支持包（bsp_tim_pwm.c）
34.4 定时器驱动移植和使用
34.5 实验例程设计框架
34.6 实验例程说明（MDK）
34.7 实验例程说明（IAR）
34.8 总结

34.1 初学者重要提示

- 学习本章节前，务必优先学习第 32 章，HAL 库的几个常用 API 均作了讲解和举例。
- 如果配置的 GPIO 引脚无法正确输出，注意本章 2.1 小节，保证是定时器复用支持的引脚。
- STM32H7 支持 TIM1-TIM8，TIM12-TIM17 共 14 个定时器，而中间的 TIM9，TIM10，TIM11 是不存在的，这点要注意。
- STM32H7 的 PWM 输出 100MHz 也是没问题的。输出效果见本章 2.3 小节。

34.2 定时器 PWM 的驱动设计

针对 STM32H7 的定时器 PWM 功能，专门设置了一个超级函数，用户可以方便的配置 TIM1-TIM17 所有定时器的 PWM 输出。

34.2.1 定时器 PWM 输出支持的引脚

STM32H7 支持的 PWM 输出引脚如下（未整理互补输出引脚）：

```
TIM1_CH1,  PA8   PE9   PK1
TIM1_CH2,  PA9   PE11
```

```
    TIM1_CH3,  PA10  PE13  PJ9
    TIM1_CH4,  PA11  PE14  PJ11

    TIM2_CH1,  PA0   PA5   PA15
    TIM2_CH2,  PA1   PB3
    TIM2_CH3,  PA2
    TIM2_CH4,  PA3   PB11

    TIM3_CH1,  PA6   PC6  PB4
    TIM3_CH2,  PA7   PC7  PB5
    TIM3_CH3,  PB0   PC8
    TIM3_CH4,  PB1   PC9

    TIM4_CH1,  PB6   PD12
    TIM4_CH2,  PB7   PD13
    TIM4_CH3,  PB8   PD14
    TIM4_CH4,  PB9   PD15

    TIM5_CH1,  PA0   PH10
    TIM5_CH2,  PA1   PH11
    TIM5_CH3,  PA2   PH12
    TIM5_CH4,  PA3   PI0

    TIM8_CH1,  PC6   PI5  PJ8
    TIM8_CH2,  PC7   PI6  PJ10
    TIM8_CH3,  PC8   PI7  PK0
    TIM8_CH4,  PC9

    TIM12_CH1,  PB14  PH6
    TIM12_CH2,  PB15  PH9

    TIM13_CH1,  PF8

    TIM14_CH1,  PF9

    TIM15_CH1,  PE5
    TIM15_CH2,  PE6

    TIM16_CH1,  PB8   PF6
    TIM16_CH2,  PF7

    TIM17_CH1,  PB9
```

使用时，直接配置定时器 PWM 模式，并配置相应引脚即可使用。

34.2.2 定时器 PWM 初始化

下面函数的作用是根据使用的是 GPIO，使能相应的 GPIO 时钟。

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: bsp_RCC_GPIO_Enable
4.  *    功能说明: 使能 GPIO 时钟
5.  *    形    参: GPIOx GPIOA - GPIOK
6.  *    返 回 值: 无
7.  *********************************************************************************************************
```

```
*/
void bsp_RCC_GPIO_Enable(GPIO_TypeDef* GPIOx)
{
    if (GPIOx == GPIOA) __HAL_RCC_GPIOA_CLK_ENABLE();
    else if (GPIOx == GPIOB) __HAL_RCC_GPIOB_CLK_ENABLE();
    else if (GPIOx == GPIOC) __HAL_RCC_GPIOC_CLK_ENABLE();
    else if (GPIOx == GPIOD) __HAL_RCC_GPIOD_CLK_ENABLE();
    else if (GPIOx == GPIOE) __HAL_RCC_GPIOE_CLK_ENABLE();
    else if (GPIOx == GPIOF) __HAL_RCC_GPIOF_CLK_ENABLE();
    else if (GPIOx == GPIOG) __HAL_RCC_GPIOG_CLK_ENABLE();
    else if (GPIOx == GPIOH) __HAL_RCC_GPIOH_CLK_ENABLE();
    else if (GPIOx == GPIOI) __HAL_RCC_GPIOI_CLK_ENABLE();
    else if (GPIOx == GPIOJ) __HAL_RCC_GPIOJ_CLK_ENABLE();
    else if (GPIOx == GPIOK) __HAL_RCC_GPIOK_CLK_ENABLE();
}
```

下面函数的作用是根据使用的定时器，使能和禁止相应的定时器时钟。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RCC_TIM_Enable
*    功能说明: 使能 TIM RCC 时钟
*    形    参: TIMx TIM1 - TIM17
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RCC_TIM_Enable(TIM_TypeDef* TIMx)
{
    if (TIMx == TIM1) __HAL_RCC_TIM1_CLK_ENABLE();
    else if (TIMx == TIM2) __HAL_RCC_TIM2_CLK_ENABLE();
    else if (TIMx == TIM3) __HAL_RCC_TIM3_CLK_ENABLE();
    else if (TIMx == TIM4) __HAL_RCC_TIM4_CLK_ENABLE();
    else if (TIMx == TIM5) __HAL_RCC_TIM5_CLK_ENABLE();
    else if (TIMx == TIM6) __HAL_RCC_TIM6_CLK_ENABLE();
    else if (TIMx == TIM7) __HAL_RCC_TIM7_CLK_ENABLE();
    else if (TIMx == TIM8) __HAL_RCC_TIM8_CLK_ENABLE();
//  else if (TIMx == TIM9) __HAL_RCC_TIM9_CLK_ENABLE();
//  else if (TIMx == TIM10) __HAL_RCC_TIM10_CLK_ENABLE();
//  else if (TIMx == TIM11) __HAL_RCC_TIM11_CLK_ENABLE();
    else if (TIMx == TIM12) __HAL_RCC_TIM12_CLK_ENABLE();
    else if (TIMx == TIM13) __HAL_RCC_TIM13_CLK_ENABLE();
    else if (TIMx == TIM14) __HAL_RCC_TIM14_CLK_ENABLE();
    else if (TIMx == TIM15) __HAL_RCC_TIM15_CLK_ENABLE();
    else if (TIMx == TIM16) __HAL_RCC_TIM16_CLK_ENABLE();
    else if (TIMx == TIM17) __HAL_RCC_TIM17_CLK_ENABLE();
    else
    {
        Error_Handler(__FILE__, __LINE__);
    }
}

/*
*********************************************************************************************************
*    函 数 名: bsp_RCC_TIM_Disable
*    功能说明: 关闭 TIM RCC 时钟
*    形    参: TIMx TIM1 - TIM17
*    返 回 值: TIM 外设时钟名
*********************************************************************************************************
*/
```

```
void bsp_RCC_TIM_Disable(TIM_TypeDef* TIMx)
{
    /*
       APB1 定时器有 TIM2, TIM3 ,TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14
       APB2 定时器有 TIM1, TIM8 , TIM15, TIM16，TIM17
    */
    if (TIMx == TIM1) __HAL_RCC_TIM3_CLK_DISABLE();
    else if (TIMx == TIM2) __HAL_RCC_TIM2_CLK_DISABLE();
    else if (TIMx == TIM3) __HAL_RCC_TIM3_CLK_DISABLE();
    else if (TIMx == TIM4) __HAL_RCC_TIM4_CLK_DISABLE();
    else if (TIMx == TIM5) __HAL_RCC_TIM5_CLK_DISABLE();
    else if (TIMx == TIM6) __HAL_RCC_TIM6_CLK_DISABLE();
    else if (TIMx == TIM7) __HAL_RCC_TIM7_CLK_DISABLE();
    else if (TIMx == TIM8) __HAL_RCC_TIM8_CLK_DISABLE();
//  else if (TIMx == TIM9) __HAL_RCC_TIM9_CLK_DISABLE();
//  else if (TIMx == TIM10) __HAL_RCC_TIM10_CLK_DISABLE();
//  else if (TIMx == TIM11) __HAL_RCC_TIM11_CLK_DISABLE();
    else if (TIMx == TIM12) __HAL_RCC_TIM12_CLK_DISABLE();
    else if (TIMx == TIM13) __HAL_RCC_TIM13_CLK_DISABLE();
    else if (TIMx == TIM14) __HAL_RCC_TIM14_CLK_DISABLE();
    else if (TIMx == TIM15) __HAL_RCC_TIM15_CLK_DISABLE();
    else if (TIMx == TIM16) __HAL_RCC_TIM16_CLK_DISABLE();
    else if (TIMx == TIM17) __HAL_RCC_TIM17_CLK_DISABLE();
    else
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

配置定时器的 PWM 功能时，要是设置引脚的复用模式，下面函数就是起到这个作用。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_GetAFofTIM
*    功能说明: 根据 TIM 得到 AF 寄存器配置
*    形    参: TIMx TIM1 - TIM17
*    返 回 值: AF 寄存器配置
*********************************************************************************************************
*/
uint8_t bsp_GetAFofTIM(TIM_TypeDef* TIMx)
{
    uint8_t ret = 0;

    if (TIMx == TIM1) ret = GPIO_AF1_TIM1;
    else if (TIMx == TIM2) ret = GPIO_AF1_TIM2;
    else if (TIMx == TIM3) ret = GPIO_AF2_TIM3;
    else if (TIMx == TIM4) ret = GPIO_AF2_TIM4;
    else if (TIMx == TIM5) ret = GPIO_AF2_TIM5;
    else if (TIMx == TIM8) ret = GPIO_AF3_TIM8;
    else if (TIMx == TIM12) ret = GPIO_AF2_TIM12;
    else if (TIMx == TIM13) ret = GPIO_AF9_TIM13;
    else if (TIMx == TIM14) ret = GPIO_AF9_TIM14;
    else if (TIMx == TIM15) ret = GPIO_AF4_TIM15;
    else if (TIMx == TIM16) ret = GPIO_AF1_TIM16;
    else if (TIMx == TIM17) ret = GPIO_AF1_TIM17;
    else
    {
        Error_Handler(__FILE__, __LINE__);
    }
```

```
29.
30.     return ret;
31. }
```

下面函数的作用是配置用于 PWM 输出的引脚：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: bsp_ConfigTimGpio
4.  *    功能说明: 配置 GPIO 和 TIM 时钟， GPIO 连接到 TIM 输出通道
5.  *    形    参: GPIOx : GPIOA - GPIOK
6.  *              GPIO_PinX : GPIO_PIN_0 - GPIO__PIN_15
7.  *              TIMx : TIM1 - TIM17
8.  *    返 回 值: 无
9.  *********************************************************************************************************
10. */
11. void bsp_ConfigTimGpio(GPIO_TypeDef* GPIOx, uint16_t GPIO_PinX, TIM_TypeDef* TIMx)
12. {
13.     GPIO_InitTypeDef   GPIO_InitStruct;
14.
15.     /* 使能 GPIO 时钟 */
16.     bsp_RCC_GPIO_Enable(GPIOx);
17.
18.     /* 使能 TIM 时钟 */
19.     bsp_RCC_TIM_Enable(TIMx);
20.
21.     GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
22.     GPIO_InitStruct.Pull = GPIO_PULLUP;
23.     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
24.     GPIO_InitStruct.Alternate = bsp_GetAFofTIM(TIMx);
25.     GPIO_InitStruct.Pin = GPIO_PinX;
26.     HAL_GPIO_Init(GPIOx, &GPIO_InitStruct);
27. }
```

当占空比是 0%或者 100%时，直接设置引脚的高低电平状态。

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: bsp_ConfigGpioOut
4.  *    功能说明: 配置 GPIO 为推挽输出。主要用于 PWM 输出，占空比为 0 和 100 的情况。
5.  *    形    参: GPIOx : GPIOA - GPIOK
6.  *              GPIO_PinX : GPIO_PIN_0 - GPIO__PIN_15
7.  *    返 回 值: 无
8.  *********************************************************************************************************
9.  */
10. void bsp_ConfigGpioOut(GPIO_TypeDef* GPIOx, uint16_t GPIO_PinX)
11. {
12.     GPIO_InitTypeDef   GPIO_InitStruct;
13.
14.     bsp_RCC_GPIO_Enable(GPIOx);      /* 使能 GPIO 时钟 */
15.
16.     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
17.     GPIO_InitStruct.Pull = GPIO_NOPULL;
18.     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
19.     GPIO_InitStruct.Pin = GPIO_PinX;
20.     HAL_GPIO_Init(GPIOx, &GPIO_InitStruct);
21. }
```

下面的函数是实现 TIM1 – TIM17 进行 PWM 输出的核心，也是专门供用户调用的。

```
22. /*
23. *********************************************************************************************************
```

```
*    函 数 名: bsp_SetTIMOutPWM
*    功能说明: 设置引脚输出的 PWM 信号的频率和占空比.  当频率为 0,并且占空为 0 时,关闭定时器,GPIO 输出 0;
*              当频率为 0，占空比为 100%时，GPIO 输出 1.
*    形    参: GPIOx : GPIOA - GPIOK
*              GPIO_Pin : GPIO_PIN_0 - GPIO__PIN_15
*              TIMx : TIM1 - TIM17
*             _ucChannel：使用的定时器通道，范围 1 - 4
*              _ulFreq : PWM 信号频率，单位 Hz （实际测试，可以输出 100MHz）. 0 表示禁止输出
*              _ulDutyCycle : PWM 信号占空比，单位: 万分之一。如 5000，表示 50.00%的占空比
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_SetTIMOutPWM(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, TIM_TypeDef* TIMx, uint8_t _ucChannel,
      uint32_t _ulFreq, uint32_t _ulDutyCycle)
{
    TIM_HandleTypeDef  TimHandle = {0};
    TIM_OC_InitTypeDef sConfig = {0};
    uint16_t usPeriod;
    uint16_t usPrescaler;
    uint32_t pulse;
    uint32_t uiTIMxCLK;
    const uint16_t TimChannel[6+1] = {0, TIM_CHANNEL_1, TIM_CHANNEL_2, TIM_CHANNEL_3, TIM_CHANNEL_4,
                                      TIM_CHANNEL_5, TIM_CHANNEL_6};

    if (_ucChannel > 6)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    if (_ulDutyCycle == 0)
    {
        //bsp_RCC_TIM_Disable(TIMx);          /* 关闭 TIM 时钟, 可能影响其他通道 */
        bsp_ConfigGpioOut(GPIOx, GPIO_Pin);   /* 配置 GPIO 为推挽输出 */
        GPIOx->BSRRH = GPIO_Pin;          /* PWM = 0 */
        return;
    }
    else if (_ulDutyCycle == 10000)
    {
        //bsp_RCC_TIM_Disable(TIMx);          /* 关闭 TIM 时钟, 可能影响其他通道 */
        bsp_ConfigGpioOut(GPIOx, GPIO_Pin);   /* 配置 GPIO 为推挽输出 */
        GPIOx->BSRRL = GPIO_Pin;          /* PWM = 1*/
        return;
    }

    /* 下面是 PWM 输出 */

    bsp_ConfigTimGpio(GPIOx, GPIO_Pin, TIMx); /* 使能 GPIO 和 TIM 时钟，并连接 TIM 通道到 GPIO */

   /*----------------------------------------------------------------------
        bsp.c 文件中 void SystemClock_Config(void) 函数对时钟的配置如下:

      System Clock source       = PLL (HSE)
      SYSCLK(Hz)                = 400000000 (CPU Clock)
      HCLK(Hz)                  = 200000000 (AXI and AHBs Clock)
      AHB Prescaler             = 2
      D1 APB3 Prescaler         = 2 (APB3 Clock  100MHz)
      D2 APB1 Prescaler         = 2 (APB1 Clock  100MHz)
      D2 APB2 Prescaler         = 2 (APB2 Clock  100MHz)
```

```
82.          D3 APB4 Prescaler         = 2 (APB4 Clock  100MHz)
83.
84.          因为 APB1 prescaler != 1, 所以 APB1 上的 TIMxCLK = APB1 x 2 = 200MHz;
85.          因为 APB2 prescaler != 1, 所以 APB2 上的 TIMxCLK = APB2 x 2 = 200MHz;
86.          APB4 上面的 TIMxCLK 没有分频，所以就是 100MHz;
87.
88.          APB1 定时器有 TIM2, TIM3 ,TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14，LPTIM1
89.          APB2 定时器有 TIM1, TIM8 , TIM15, TIM16，TIM17
90.
91.          APB4 定时器有 LPTIM2，LPTIM3，LPTIM4，LPTIM5
92.
93.      ------------------------------------------------------------------------ */
94.      if ((TIMx == TIM1) || (TIMx == TIM8) || (TIMx == TIM15) || (TIMx == TIM16) || (TIMx == TIM17))
95.      {
96.          /* APB2 定时器时钟 = 200M */
97.          uiTIMxCLK = SystemCoreClock / 2;
98.      }
99.      else
100.     {
101.         /* APB1 定时器 = 200M */
102.         uiTIMxCLK = SystemCoreClock / 2;
103.     }
104.
105.     if (_ulFreq < 100)
106.     {
107.         usPrescaler = 10000 - 1;                    /* 分频比 = 10000 */
108.         usPeriod =  (uiTIMxCLK / 10000) / _ulFreq  - 1;         /* 自动重装的值 */
109.     }
110.     else if (_ulFreq < 3000)
111.     {
112.         usPrescaler = 100 - 1;                  /* 分频比 = 100 */
113.         usPeriod =  (uiTIMxCLK / 100) / _ulFreq  - 1;       /* 自动重装的值 */
114.     }
115.     else /* 大于 4K 的频率，无需分频 */
116.     {
117.         usPrescaler = 0;                    /* 分频比 = 1 */
118.         usPeriod = uiTIMxCLK / _ulFreq - 1;  /* 自动重装的值 */
119.     }
120.     pulse = (_ulDutyCycle * usPeriod) / 10000;
121.
122.
123.     HAL_TIM_PWM_DeInit(&TimHandle);
124.
125.     /*  PWM 频率 = TIMxCLK / usPrescaler + 1) /usPeriod + 1) */
126.     TimHandle.Instance = TIMx;
127.     TimHandle.Init.Prescaler         = usPrescaler;
128.     TimHandle.Init.Period            = usPeriod;
129.     TimHandle.Init.ClockDivision     = 0;
130.     TimHandle.Init.CounterMode       = TIM_COUNTERMODE_UP;
131.     TimHandle.Init.RepetitionCounter = 0;
132.     TimHandle.Init.AutoReloadPreload = 0;
133.     if (HAL_TIM_PWM_Init(&TimHandle) != HAL_OK)
134.     {
135.         Error_Handler(__FILE__, __LINE__);
136.     }
137.
138.     /* 配置定时器 PWM 输出通道 */
139.     sConfig.OCMode       = TIM_OCMODE_PWM1;
```

```
140.      sConfig.OCPolarity   = TIM_OCPOLARITY_HIGH;
141.      sConfig.OCFastMode   = TIM_OCFAST_DISABLE;
142.      sConfig.OCNPolarity  = TIM_OCNPOLARITY_HIGH;
143.      sConfig.OCNIdleState = TIM_OCNIDLESTATE_RESET;
144.      sConfig.OCIdleState  = TIM_OCIDLESTATE_RESET;
145.
146.      /* 占空比 */
147.      sConfig.Pulse = pulse;
148.      if (HAL_TIM_PWM_ConfigChannel(&TimHandle, &sConfig, TimChannel[_ucChannel]) != HAL_OK)
149.      {
150.          Error_Handler(__FILE__, __LINE__);
151.      }
152.
153.      /* 启动 PWM 输出 */
154.      if (HAL_TIM_PWM_Start(&TimHandle, TimChannel[_ucChannel]) != HAL_OK)
155.      {
156.          Error_Handler(__FILE__, __LINE__);
157.      }
158. }
```

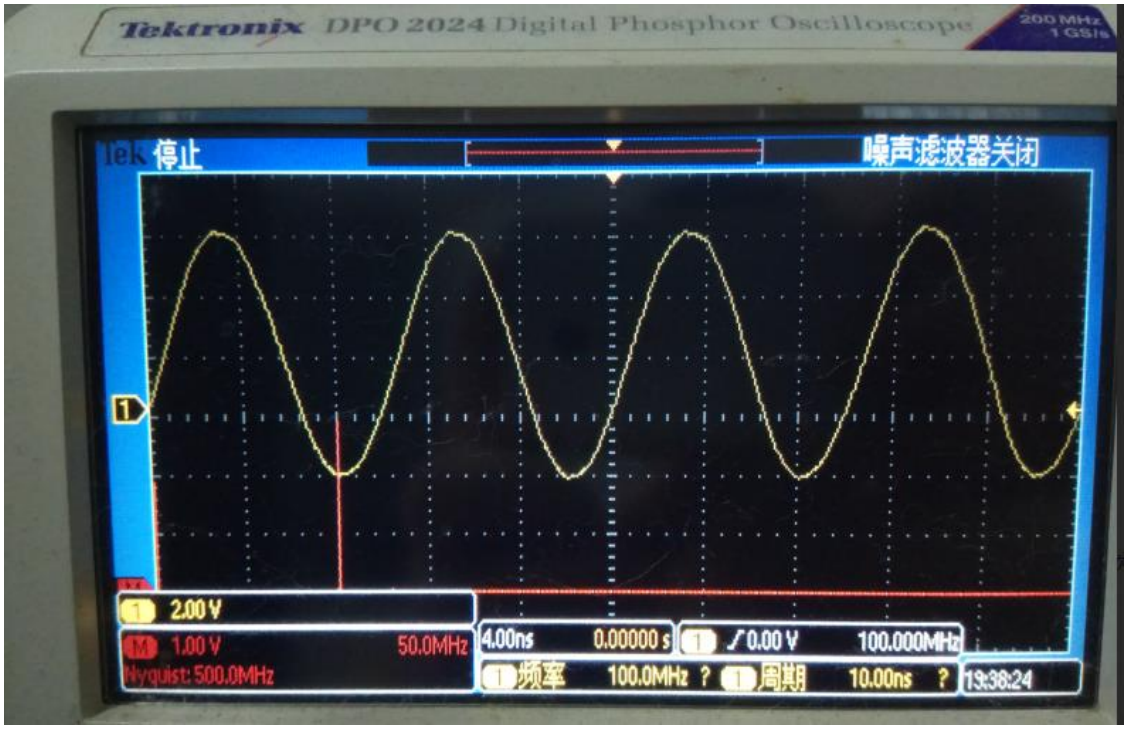
程序中的注释已经比较详细，这里把几个关键的地方再阐释下：

- 第 39 -40 行，HAL 库的这两个结构体变量要初始化为 0，这个问题在第 32 章的的 4.3 和 4.4 小节有专门说明。
- 第 94 – 120 行，计算出要配置的分频和周期。这里要注意一点，因为除了 TIM2 和 TIM5，其它定时器都是 16 位的，相关寄存器大部分也都是 16 位的，配置的时候不可以超出 0 -65535。这里分频变量 usPrescaler 和周期变量 usPeriod 统一按照 16 位计算，所以有了这几行代码做频率区分，防止超出范围。
- 第 126 – 136 行，通过函数 HAL_TIM_PWM_Init 配置了 PWM 频率。
- 第 139 – 151 行，配置定时器的 PWM 输出通道，关于结构体成员代表的含义和函数 HAL_TIM_PWM_ConfigChannel 的用法分别看第 32 章的 3.3 和 4.4 小节。
- 第 154 行，启动定时器 PWM 输出。

34.2.3定时器 PWM 输出 100MHz 的效果

测试 PWM 输出 100MHz 方波的效果，因为我的示波器是 200MHz 带宽，1Gsps 采样率的，用来采样 100MHz 方波的话，仅可以采集到基波（一次谐波，100MHz），而三次谐波（300MHz），五次谐波（500MHz），以此类推都是采集不到的，所以最终的采集应该就是一个标准的 100MHz 正弦波，实际测试效果完美，就是个 100MHz 的正弦波。

黄色的是波形，红色的是 FFT 幅值谱。



实现这个高频率，代码要特别配置，实现如下，注意红字部分：

```
/*##-1- 配置定时器外设 ##########################################*/
htim1.Instance = TIM1;
htim1.Init.Prescaler = 0;
htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
htim1.Init.Period = 1;
htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim1.Init.RepetitionCounter = 0;
htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;

/*##-2- 使能定时器 ##############################################*/
if (HAL_TIM_Base_Init(&htim1) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}

/* 配置模式 */
sConfigOC.OCMode = TIM_OCMODE_PWM1;
sConfigOC.Pulse = 1;
sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
sConfigOC.OCNPolarity = TIM_OCNPOLARITY_HIGH;
sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
sConfigOC.OCIdleState = TIM_OCIDLESTATE_RESET;
sConfigOC.OCNIdleState = TIM_OCNIDLESTATE_RESET;

/* 配置 PWM 通道 */
if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_1) != HAL_OK)
{
      Error_Handler(__FILE__, __LINE__);
}

/* 开启 PWM 输出 */
if (HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1) != HAL_OK)
```

```
{
	Error_Handler(__FILE__, __LINE__);
}
```

34.3 定时器板级支持包（bsp_tim_pwm.c）

定时器驱动文件 bsp_tim_pwm.c 主要实现了如下两个 API 供用户调用：

- bsp_SetTIMOutPWM
- bsp_SetTIMforInt

这个两个函数都是 TIM1-TIM17 所有定时器都支持，函数 bsp_SetTIMforInt 用于定时器周期性中断，下个章节为大家讲解，本小节主要把函数 bsp_SetTIMOutPWM 做个说明。

34.3.1 函数 bsp_SetTIMOutPWM

函数原型：

```
void bsp_SetTIMOutPWM(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, TIM_TypeDef* TIMx, uint8_t _ucChannel,
                      uint32_t _ulFreq, uint32_t _ulDutyCycle)
```

函数描述：

此函数主要用配置定时器的 PWM 输出。

函数参数：

- 第 1 个参数 GPIO 分组，范围 GPIOA – GPIOK。
- 第 2 个参数是具体的 GPIO 引脚，范围 GPIO_PIN_0 - GPIO__PIN_15。
- 第 3 个参数用于指定使用哪个定时器，参数可以是 TIM1 – TIM17 所有定时器（不含 TIM9，TIM10 和 TIM11，因为 STM32H7 不支持这三个定时器）。
- 第 4 个参数是使用的定时器通道，范围 1-4，分别表示通道 1，通道 2，通道 3 和通道 4。
- 第 5 个参数是要实现的定时器中断频率，单位 Hz，如果填 0 的话，表示关闭。
- 第 6 个参数是 PWM 信号占空比，单位: 万分之一。如 5000，表示 50.00%的占空比。

注意事项：

1. PWM 频率最好别超过 50MHz，因为此函数的源码实现超过 50MHz 后，计算的已经不准确。10MHz 以下基本都是没问题的。

使用举例：

比如配置 PB3 硬件输出 1KHz 方波，占空比 50%

bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_3, TIM3, 4, 1000, 5000)

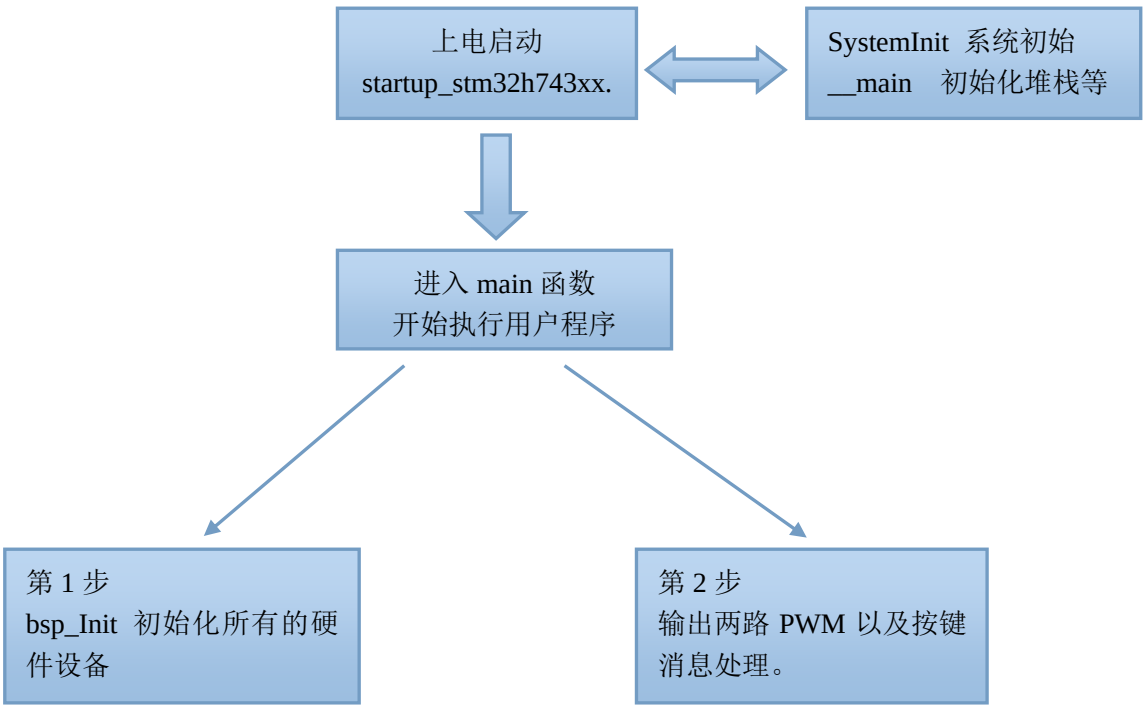
34.4 定时器驱动移植和使用

定时器的移植比较简单：

- 第 1 步：复制 bsp_tim_pwm.c 和 bsp_tim_pwm.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：这几个驱动文件主要用到 HAL 库的 GPIO 和 TIM 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 3 步，应用方法看本章节配套例子即可。

34.5 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED 和串口。
  - 第 2 步，输出两路 PWM 以及按键消息处理。

34.6 实验例程说明（MDK）

配套例子：

V7-019_定时器 PWM 输出（驱动支持 TIM1-TIM17）

实验目的：

1. 学习定时器 PWM 输出。

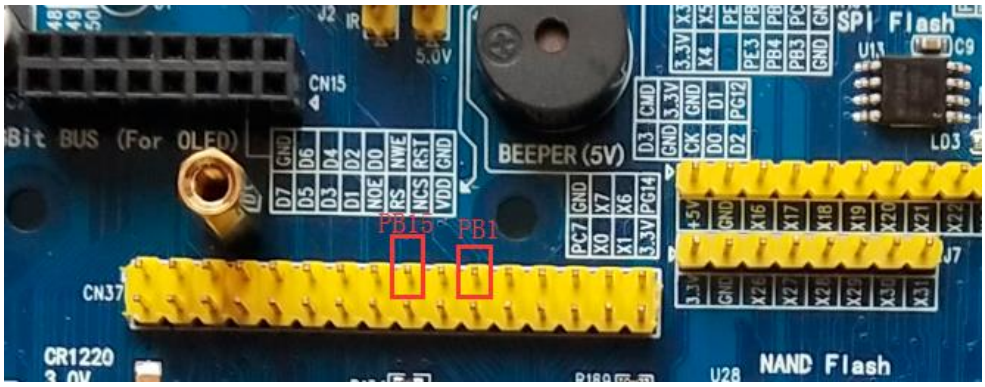
实验内容：

1. 系统上电后驱动了 1 个软件定时器，每 100ms 翻转一次 LED2，同时 PB3 和 PB15 输出 1KHz 方波，占空比 50% 。
2. TM32H7 支持 TIM1-TIM8，TIM12-TIM17 共 14 个定时器，而中间的 TIM9，TIM10，TIM11 是不存在的。

实验操作：

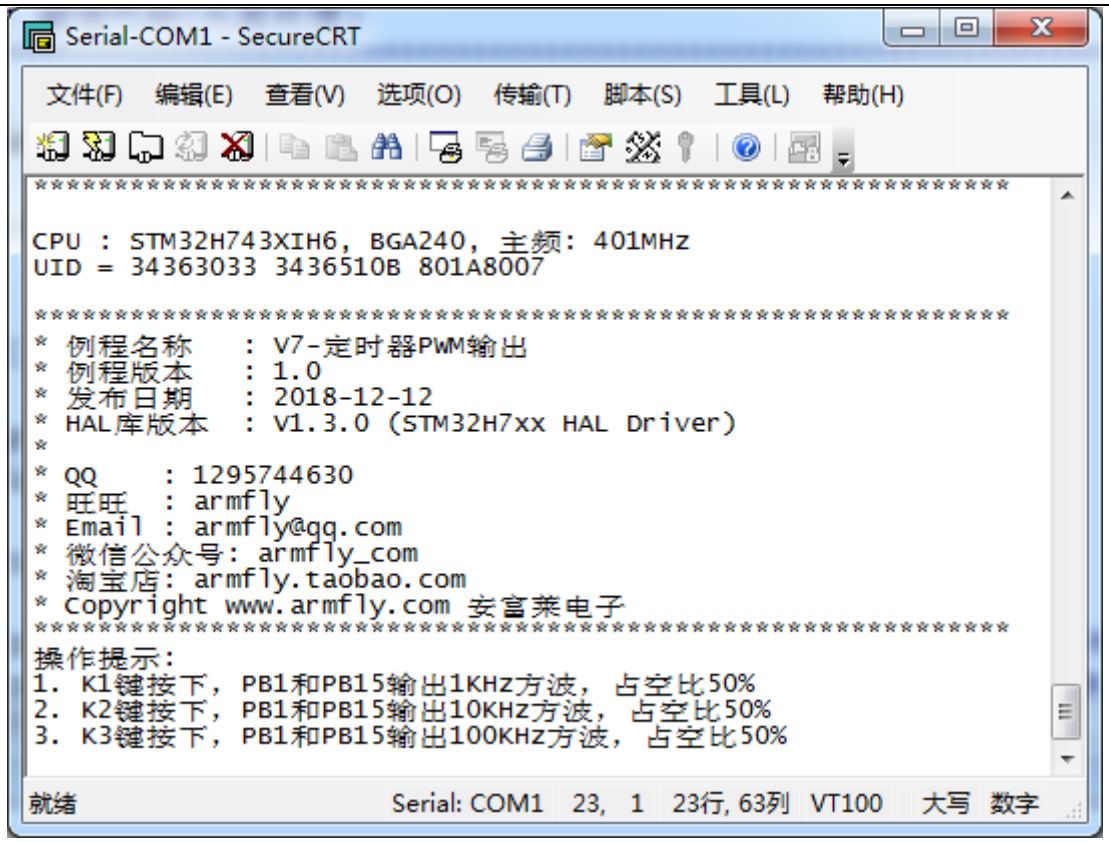
1. K1 键按下，PB1 和 PB15 输出 1KHz 方波，占空比 50%。
2. K2 键按下，PB1 和 PB15 输出 10KHz 方波，占空比 50%。
3. K3 键按下，PB1 和 PB15 输出 100KHz 方波，占空比 50%

PWM 输出引脚 PB1 和 PB15 的位置：



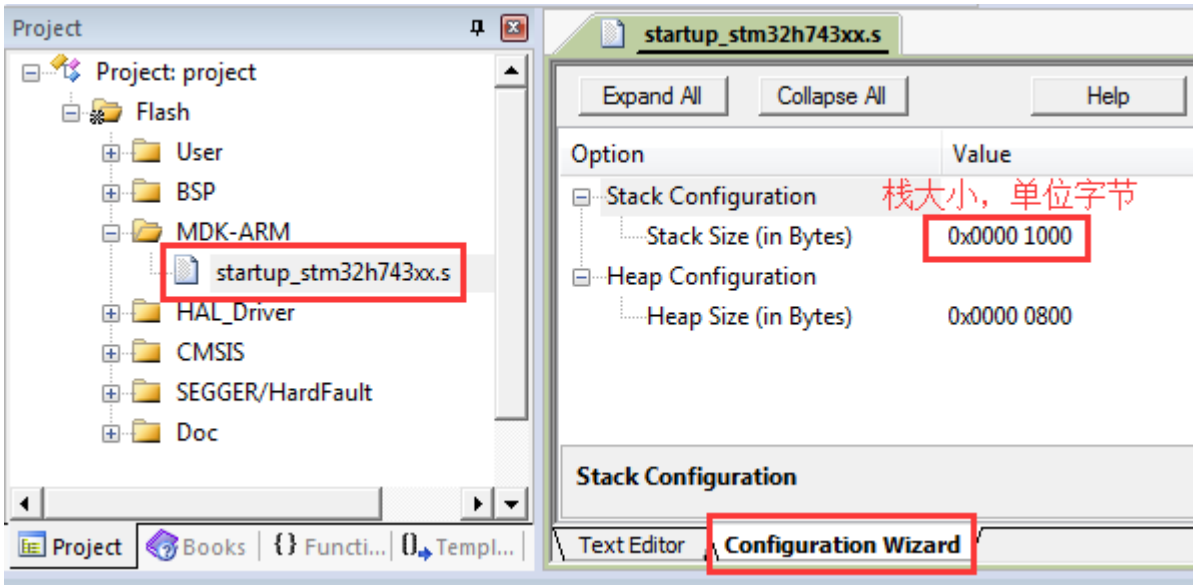
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

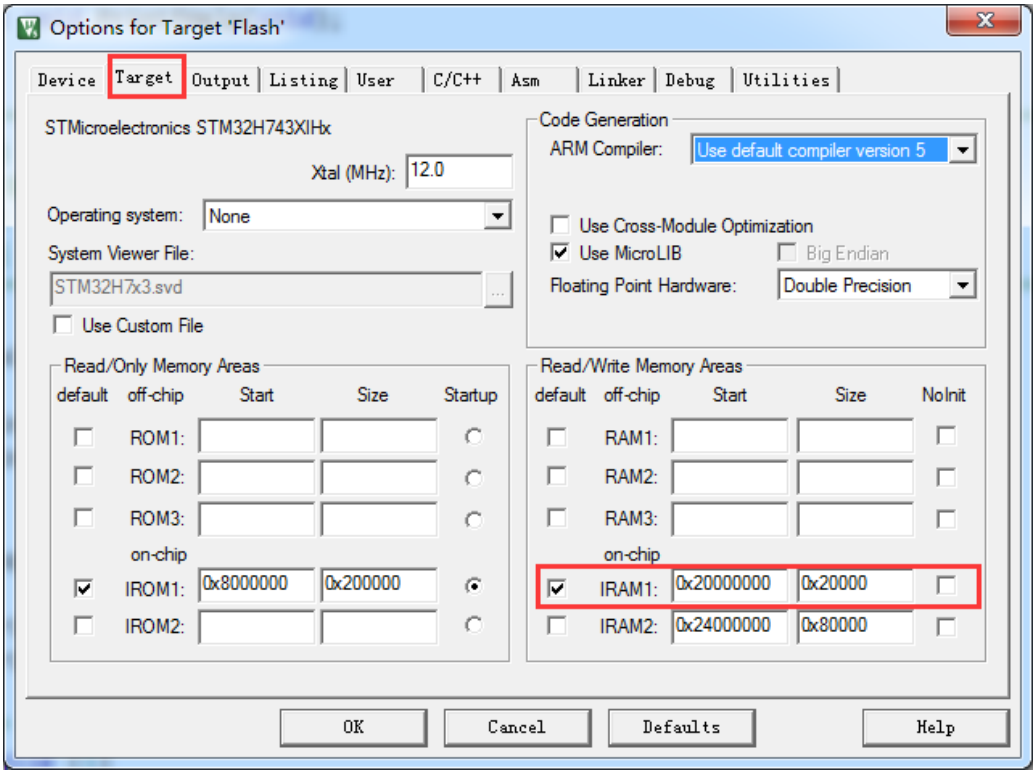


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
```

```
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- K1 键按下，PB1 和 PB15 输出 1KHz 方波，占空比 50%。
- K2 键按下，PB1 和 PB15 输出 10KHz 方波，占空比 50%。
- K3 键按下，PB1 和 PB15 输出 100KHz 方波，占空比 50%。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */
```

```
    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_1,  TIM3,  4, 1000, 5000); /* PB3 硬件输出 1KHz 方波，占空比 50% */
    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_15, TIM12, 2, 1000, 5000); /* PB15 硬件输出 1KHz 方波，占空比 50% */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 50ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值，无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，PB1 和 PB15 输出 1KHz 方波，占空比 50% */
                    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_1,  TIM3,  4, 1000, 5000);
                    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_15, TIM12, 2, 1000, 5000);
                    break;

                case KEY_DOWN_K2:           /* K2 键按下，PB1 和 PB15 输出 10KHz 方波，占空比 50% */
                    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_1,  TIM3,  4, 10000, 5000);
                    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_15, TIM12, 2, 10000, 5000);
                    break;

                case KEY_DOWN_K3:           /* K3 键按下，PB1 和 PB15 输出 100KHz 方波，占空比 50% */
                    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_1,  TIM3,  4, 100000, 5000);
                    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_15, TIM12, 2, 100000, 5000);
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

34.7 实验例程说明（IAR）

配套例子：

V7-019_定时器 PWM 输出（驱动支持 TIM1-TIM17）

实验目的：

2. 学习定时器 PWM 输出。

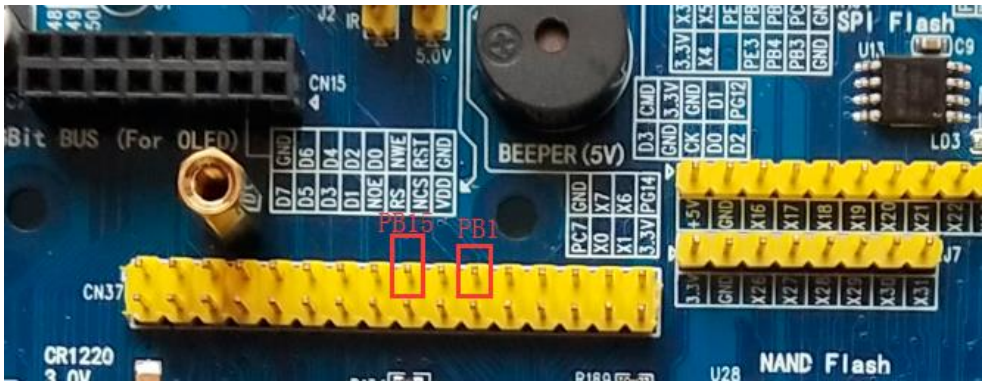
实验内容：

3. 系统上电后驱动了 1 个软件定时器，每 100ms 翻转一次 LED2，同时 PB3 和 PB15 输出 1KHz 方波，占空比 50% 。
4. TM32H7 支持 TIM1-TIM8，TIM12-TIM17 共 14 个定时器，而中间的 TIM9，TIM10，TIM11 是不存在的。

实验操作：

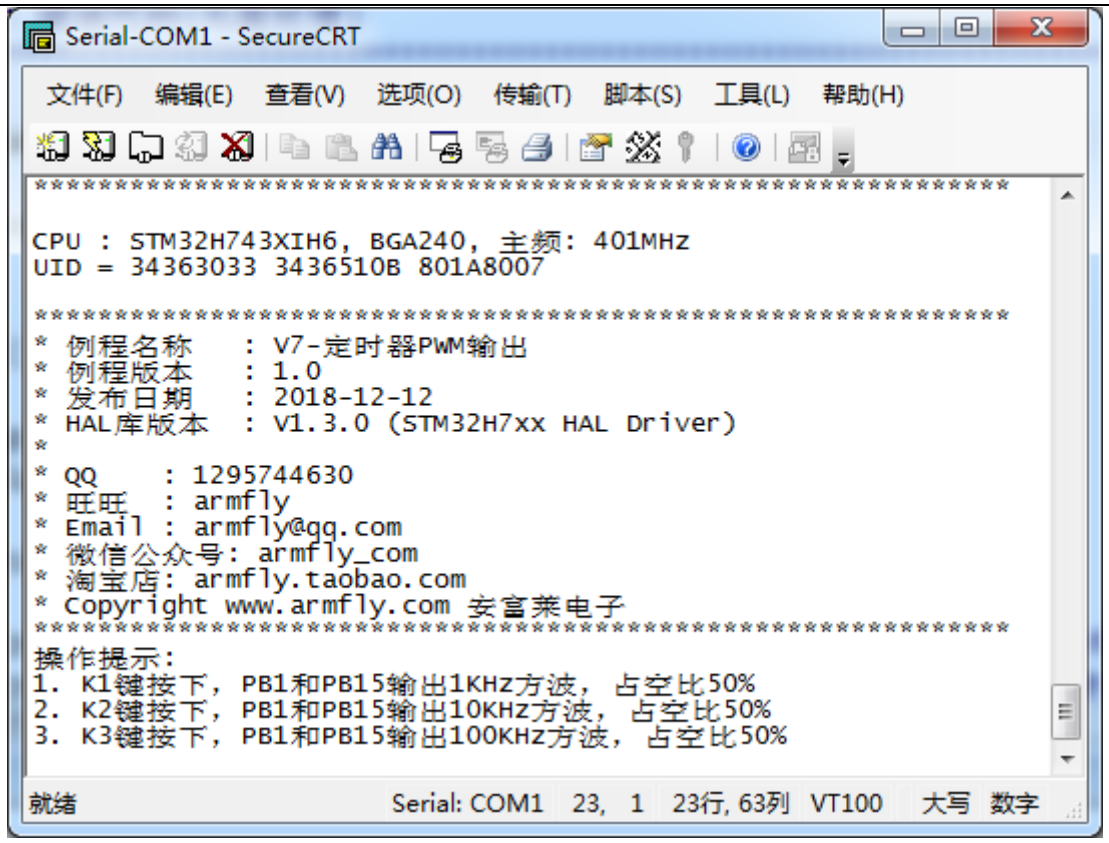
4. K1 键按下，PB1 和 PB15 输出 1KHz 方波，占空比 50%。
5. K2 键按下，PB1 和 PB15 输出 10KHz 方波，占空比 50%。
6. K3 键按下，PB1 和 PB15 输出 100KHz 方波，占空比 50%

PWM 输出引脚 PB1 和 PB15 的位置：



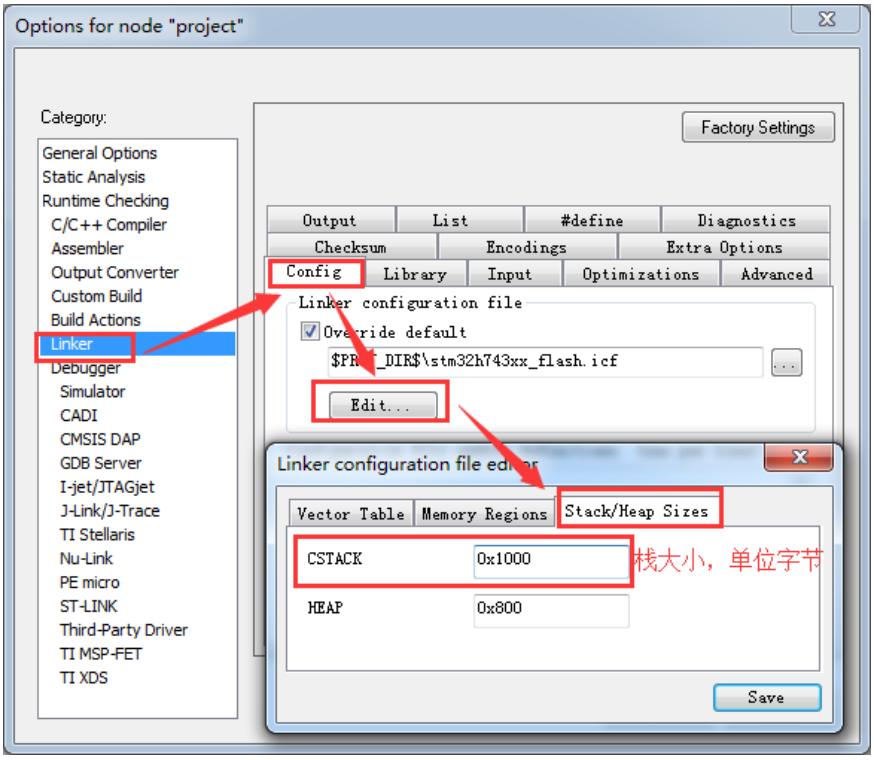
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

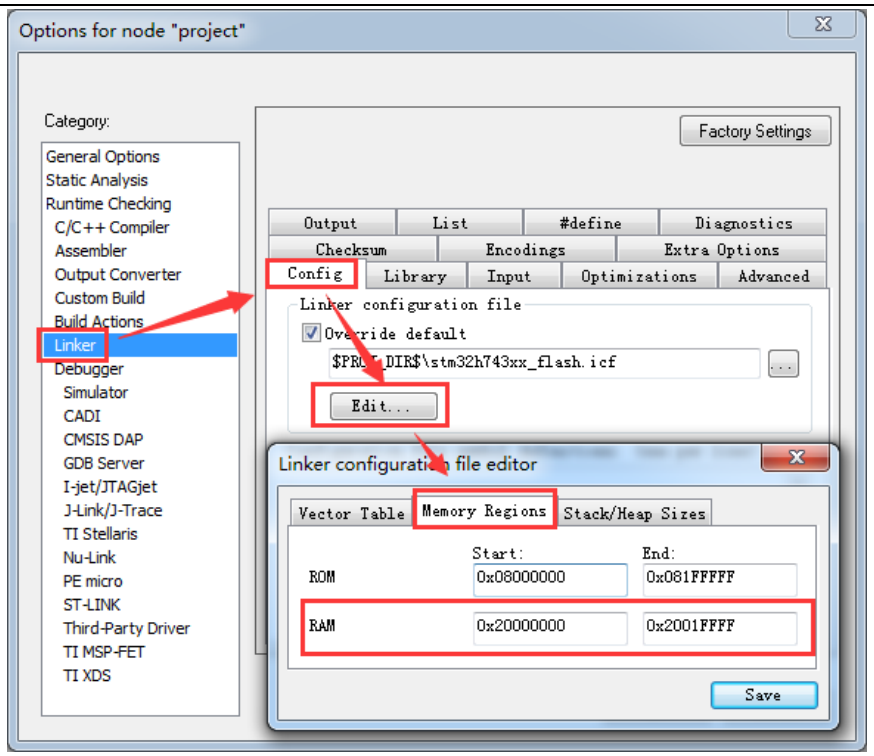


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
```

```
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能:**

主程序实现如下操作:

- K1 键按下，PB1 和 PB15 输出 1KHz 方波，占空比 50%。
- K2 键按下，PB1 和 PB15 输出 10KHz 方波，占空比 50%。
- K3 键按下，PB1 和 PB15 输出 100KHz 方波，占空比 50%。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */
```

```
    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_1,  TIM3,  4, 1000, 5000); /* PB3 硬件输出 1KHz 方波，占空比 50% */
    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_15, TIM12, 2, 1000, 5000); /* PB15 硬件输出 1KHz 方波，占空比 50% */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 50ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，PB1 和 PB15 输出 1KHz 方波，占空比 50% */
                    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_1,  TIM3,  4, 1000, 5000);
                    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_15, TIM12, 2, 1000, 5000);
                    break;

                case KEY_DOWN_K2:           /* K2 键按下，PB1 和 PB15 输出 10KHz 方波，占空比 50% */
                    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_1,  TIM3,  4, 10000, 5000);
                    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_15, TIM12, 2, 10000, 5000);
                    break;

                case KEY_DOWN_K3:           /* K3 键按下，PB1 和 PB15 输出 100KHz 方波，占空比 50% */
                    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_1,  TIM3,  4, 100000, 5000);
                    bsp_SetTIMOutPWM(GPIOB, GPIO_PIN_15, TIM12, 2, 100000, 5000);
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

34.8 总结

本章节就为大家讲解这么多，相对比较容易掌握，望初学者熟练运用。

第35章 STM32H7 的定时器应用之高精度单次延迟实现（支持 TIM2,3,4 和 5）

本章教程为大家讲解定时器应用之高精度单次延迟实现，支持 TIM2，TIM3，TIM4 和 TIM5。实际项目中用到的地方较多，如 Modbus 帧符间隔，定时采集一段时间波形等。

35.1 初学者重要提示
35.2 定时器单次延迟驱动设置
35.3 定时器板级支持包（bsp_timer.c）
35.4 定时器驱动移植和使用
35.5 实验例程框架
35.6 实验例程说明（MDK）
35.7 实验例程说明（IAR）
35.8 总结

35.1 初学者重要提示

- 学习本章节前，务必优先学习第 32 章，HAL 库的几个常用 API 均作了讲解和举例。
- STM32H7 支持 TIM1-TIM8，TIM12-TIM17 共 14 个定时器，而中间的 TIM9，TIM10，TIM11 是不存在的，这点要注意。
- 在不需要任何补偿的情况下，误差可以做到正负 1 微秒以内。
- TIM2 和 TIM5 是 32 位定时器，而 TIM3 和 TIM4 是 16 位定时器。

35.2 定时器单次延迟驱动设计

单次定时器要实现 1us 的精度，可以直接将定时器时钟设置为 1MHz，这样定时器每计数 1 次就是 1us。对于 16 位定时器最大值就是 0xFFFF 微秒，而 32 位定时器就是 0xFFFFFFFF 微秒。

剩下的问题就是单次延迟时间到了可以及时执行相应功能，那么就可以开启一个 CC 捕获比较中断。而延迟时间可以直接通过设置 CCR 比较捕获寄存器实现。比如当前定时器的计数值是 1000，我们要实现 10us 的单次延迟，我们就可以直接设置 CCR 的数值为 1000 + 10 =1010 即可，等 1010 的计数值到了，就会触发 CC 捕获比较中断。

35.2.1 定时器单次延迟宏定义

单次延迟支持 TIM2，TIM3，TIM4 和 TIM5，其中 TIM2 和 TIM5 是 32 位定时器，而 TIM3 和 TIM4 是 16 位定时器。每个定时器都有 4 个通道，可以独立配置使用，互不影响。

```
1.  /*
2.       定义用于硬件定时器的 TIM,  可以使 TIM2 - TIM5
3.  */
4.  #define USE_TIM2
5.  //#define USE_TIM3
6.  //#define USE_TIM4
7.  //#define USE_TIM5
8.
9.  #ifdef USE_TIM2
10.      #define TIM_HARD                        TIM2
11.      #define  RCC_TIM_HARD_CLK_ENABLE()      __HAL_RCC_TIM2_CLK_ENABLE()
12.      #define TIM_HARD_IRQn                   TIM2_IRQn
13.      #define TIM_HARD_IRQHandler             TIM2_IRQHandler
14. #endif
15.
16. #ifdef USE_TIM3
17.      #define TIM_HARD                        TIM3
18.      #define  RCC_TIM_HARD_CLK_ENABLE()      __HAL_RCC_TIM3_CLK_ENABLE()
19.      #define TIM_HARD_IRQn                   TIM3_IRQn
20.      #define TIM_HARD_IRQHandler             TIM3_IRQHandler
21. #endif
22.
23. #ifdef USE_TIM4
24.      #define TIM_HARD                        TIM4
25.      #define  RCC_TIM_HARD_CLK_ENABLE()      __HAL_RCC_TIM4_CLK_ENABLE()
26.      #define TIM_HARD_IRQn                   TIM4_IRQn
27.      #define TIM_HARD_IRQHandler             TIM4_IRQHandler
28. #endif
29.
30. #ifdef USE_TIM5
31.      #define TIM_HARD                        TIM5
32.      #define  RCC_TIM_HARD_CLK_ENABLE()      __HAL_RCC_TIM5_CLK_ENABLE()
33.      #define TIM_HARD_IRQn                   TIM5_IRQn
34.      #define TIM_HARD_IRQHandler             TIM5_IRQHandler
35. #endif
36.
37. /* 保存 TIM 定时中断到后执行的回调函数指针 */
38. static void (*s_TIM_CallBack1)(void);
39. static void (*s_TIM_CallBack2)(void);
40. static void (*s_TIM_CallBack3)(void);
41. static void (*s_TIM_CallBack4)(void);
```

这里把几个关键的地方阐释下：

- 第 4- 7 行，用于选择要使用的定时器，使用哪个定时器，使能那个宏定义即可。
- 第 9 - 14 行，用于配置定时器的四个宏定义，这里是配置的 TIM2，后面 TIM3，TIM4，TIM5 的配置同理。
- 第 38 – 40 行，定义 4 个函数指针，用于保存定时器 CC 比较捕获中断执行后的回调函数指针。

35.2.2 定时器单次延迟初始化

单次定时器的初始化代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitHardTimer
*    功能说明: 配置 TIMx，用于 us 级别硬件定时。TIMx 将自由运行，永不停止.
*              TIMx 可以用 TIM2 - TIM5 之间的 TIM, 这些 TIM 有 4 个通道, 挂在 APB1 上，输入时钟
*              =SystemCoreClock / 2
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitHardTimer(void)
{
    TIM_HandleTypeDef  TimHandle = {0};
     uint32_t usPeriod;
     uint16_t usPrescaler;
     uint32_t uiTIMxCLK;
     TIM_TypeDef* TIMx = TIM_HARD;

     RCC_TIM_HARD_CLK_ENABLE();       /* 使能 TIM 时钟 */

    /*-----------------------------------------------------------------------
          bsp.c 文件中 void SystemClock_Config(void) 函数对时钟的配置如下:

        System Clock source       = PLL (HSE)
        SYSCLK(Hz)                = 400000000 (CPU Clock)
        HCLK(Hz)                  = 200000000 (AXI and AHBs Clock)
        AHB Prescaler             = 2
        D1 APB3 Prescaler         = 2 (APB3 Clock  100MHz)
        D2 APB1 Prescaler         = 2 (APB1 Clock  100MHz)
        D2 APB2 Prescaler         = 2 (APB2 Clock  100MHz)
        D3 APB4 Prescaler         = 2 (APB4 Clock  100MHz)

        因为 APB1 prescaler != 1, 所以 APB1 上的 TIMxCLK = APB1 x 2 = 200MHz;
        因为 APB2 prescaler != 1, 所以 APB2 上的 TIMxCLK = APB2 x 2 = 200MHz;
        APB4 上面的 TIMxCLK 没有分频，所以就是 100MHz;

        APB1 定时器有 TIM2, TIM3 ,TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14，LPTIM1
        APB2 定时器有 TIM1, TIM8 , TIM15, TIM16，TIM17

        APB4 定时器有 LPTIM2，LPTIM3，LPTIM4，LPTIM5

    ----------------------------------------------------------------------- */
    if ((TIMx == TIM1) || (TIMx == TIM8) || (TIMx == TIM15) || (TIMx == TIM16) || (TIMx == TIM17))
    {
        /* APB2 定时器时钟 = 200M */
        uiTIMxCLK = SystemCoreClock / 2;
    }
    else
    {
        /* APB1 定时器 = 200M */
        uiTIMxCLK = SystemCoreClock / 2;
    }
```

```
53.
54.     usPrescaler = uiTIMxCLK / 1000000 - 1;    /* 分频比 = 1 */
55.
56.     if (TIMx == TIM2 || TIMx == TIM5)
57.     {
58.         usPeriod = 0xFFFFFFFF;
59.     }
60.     else
61.     {
62.         usPeriod = 0xFFFF;
63.     }
64.
65.     /*
66.       设置分频为 usPrescaler 后，那么定时器计数器计 1 次就是 1us
67.       而参数 usPeriod 的值是决定了最大计数：
68.       usPeriod = 0xFFFF 表示最大 0xFFFF 微妙。
69.       usPeriod = 0xFFFFFFFF 表示最大 0xFFFFFFFF 微妙。
70.     */
71.     TimHandle.Instance = TIMx;
72.     TimHandle.Init.Prescaler         = usPrescaler;
73.     TimHandle.Init.Period            = usPeriod;
74.     TimHandle.Init.ClockDivision     = 0;
75.     TimHandle.Init.CounterMode       = TIM_COUNTERMODE_UP;
76.     TimHandle.Init.RepetitionCounter = 0;
77.     TimHandle.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
78.
79.     if (HAL_TIM_Base_Init(&TimHandle) != HAL_OK)
80.     {
81.         Error_Handler(__FILE__, __LINE__);
82.     }
83.
84.     /* 配置定时器中断，给 CC 捕获比较中断使用 */
85.     {
86.         HAL_NVIC_SetPriority(TIM_HARD_IRQn, 0, 2);
87.         HAL_NVIC_EnableIRQ(TIM_HARD_IRQn);
88.     }
89.
90.     /* 启动定时器 */
91.     HAL_TIM_Base_Start(&TimHandle);
92. }
```

这里把几个关键的地方阐释下：

- 第 13 行，HAL 库的这个结构体变量要初始化为 0，此问题在第 32 章的的 4.1 小节有专门说明。
- 第 43 – 52 行，获取定时器的时钟频率，TIM2，TIM3，TIM4 和 TIM5 都是用的 APB1，因为 APB1 prescaler != 1，所以 APB1 上的 TIMxCLK = APB1 x 2 = 200MHz。
- 第 54 行，设置分频参数，定时器分频的频率是 1MHz。
- 第 71 - 82 行，设置分频为 usPrescaler 后，那么定时器计数器计 1 次就是 1us，而参数 usPeriod 的值是决定了最大计数：

  usPeriod = 0xFFFF 表示最大 0xFFFF 微秒。

  usPeriod = 0xFFFFFFFF 表示最大 0xFFFFFFFF 微秒。
- 第 86 – 87 行，这里要特别注意，此处是开启定时器的 NVIC 是供 CC 捕获比较中断使用，而不是更

新中断。

- 第 91 行，启动定时器。

35.2.3 定时器单次延迟启动

下面是定时器的启动代码，使用 TIM2-5 做单次定时器使用，定时时间到后执行回调函数。可以同时启动 4 个定时器，互不干扰。

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: bsp_StartHardTimer
4.  *    功能说明: 使用 TIM2-5 做单次定时器使用，定时时间到后执行回调函数。可以同时启动 4 个定时器，互不干扰。
5.  *              定时精度正负 1us （主要耗费在调用本函数的执行时间，函数内部进行了补偿减小误差）
6.  *                TIM2 和 TIM5 是 32 位定时器。定时范围很大
7.  *                TIM3 和 TIM4 是 16 位定时器。
8.  *    形    参: _CC : 捕获通道几，1，2，3，4
9.  *          _uiTimeOut : 超时时间，单位 1us. 对于 16 位定时器，最大 65.5ms；对于 32 位定时器，最大 4294 秒
10. *          _pCallBack : 定时时间到后，被执行的函数
11. *    返 回 值: 无
12. *********************************************************************************************************
13. */
14. void bsp_StartHardTimer(uint8_t _CC, uint32_t _uiTimeOut, void * _pCallBack)
15. {
16.     uint32_t cnt_now;
17.     uint32_t cnt_tar;
18.     TIM_TypeDef* TIMx = TIM_HARD;
19.
20.     /* H743 速度较快，无需补偿延迟，实测精度正负 1us */
21.
22.     cnt_now = TIMx->CNT;
23.     cnt_tar = cnt_now + _uiTimeOut;           /* 计算捕获的计数器值 */
24.     if (_CC == 1)
25.     {
26.         s_TIM_CallBack1 = (void (*)(void))_pCallBack;
27.
28.         TIMx->CCR1 = cnt_tar;               /* 设置捕获比较计数器 CC1 */
29.         TIMx->SR = (uint16_t)~TIM_IT_CC1;   /* 清除 CC1 中断标志 */
30.         TIMx->DIER |= TIM_IT_CC1;           /* 使能 CC1 中断 */
31.     }
32.     else if (_CC == 2)
33.     {
34.         s_TIM_CallBack2 = (void (*)(void))_pCallBack;
35.
36.         TIMx->CCR2 = cnt_tar;               /* 设置捕获比较计数器 CC2 */
37.         TIMx->SR = (uint16_t)~TIM_IT_CC2;   /* 清除 CC2 中断标志 */
38.         TIMx->DIER |= TIM_IT_CC2;           /* 使能 CC2 中断 */
39.     }
40.     else if (_CC == 3)
41.     {
42.         s_TIM_CallBack3 = (void (*)(void))_pCallBack;
43.
44.         TIMx->CCR3 = cnt_tar;               /* 设置捕获比较计数器 CC3 */
45.         TIMx->SR = (uint16_t)~TIM_IT_CC3;   /* 清除 CC3 中断标志 */
46.         TIMx->DIER |= TIM_IT_CC3;           /* 使能 CC3 中断 */
```

```
47.     }
48.     else if (_CC == 4)
49.     {
50.         s_TIM_CallBack4 = (void (*)(void))_pCallBack;
51.
52.          TIMx->CCR4 = cnt_tar;                /* 设置捕获比较计数器 CC4 */
53.          TIMx->SR = (uint16_t)~TIM_IT_CC4;    /* 清除 CC4 中断标志 */
54.          TIMx->DIER |= TIM_IT_CC4;            /* 使能 CC4 中断 */
55.     }
56.     else
57.     {
58.         return;
59.     }
60. }
```

这里把几个关键的地方阐释下：

- 第 22 行，获取定时器的计数值，赋给 32 位变量。
- 第 23 行，将当前的计数值和延迟的计数值求和，这里有个隐含的知识点，就是两个数求和会有溢出的情况，溢出了会不会出问题，答案是不会的。
  - 对于 32 位定时器，如果两个 32 位变量求和超过范围，那么变量 cnt_tar 最终结果是超出的那部分。而定时器的配置也是向上计数的，计数满 32 位后，也是从 0 开始重新计数，记到 cnt_tar 就是我们所设置的_uiTimeOut 时间。为了方便大家理解，举个例子，比如 cnt_now = TIMx->CNT = 0xffffffff0， _uiTimeOut = 0x20。那么 cnt_tar = 0x10，定时器从 0xffffffff0 计数到 0xffffffff 后，再从 0 开始计数到 0x10，时间差就是_uiTimeOut。
  - 对于 16 位定时器，cnt_now = TIMx->CNT 获取的数值是小于等于 0xffff 的，执行第 23 行的函数后，变量 cnt_tar 的数值是有可能会大于 0xffff 的，这也没有关系的，因为 16 位定时器对应的 CCR 寄存器是 16 位的，执行效果跟 32 位定时器溢出的效果一样。比如 cnt_now = TIMx->CNT = 0xfff0， _uiTimeOut = 0x20。那么 cnt_tar = 0x10010，将这个数值赋值给 16 位的 CCR 寄存器效果就是 CCR = 0x10。定时器从 0xfff0 计数到 0xffff 后，再从 0 开始计数到 0x10，时间差就_uiTimeOut。
- 第 24 行，_CC = 1 表示通道 1，_CC = 2 表示通道 2，_CC = 3 表示通道 3，_CC = 4 表示通道 4。
- 第 26 行，参数_pCallBack 前的 void (*)(void)是函数指针的强制类型转换，防止警告。
- 第 28 – 30 行，设置捕获比较寄存器 CCR，清除 CC 中断并开启 CC 中断。
- 第 32 – 55 行，其它通道的处理。跟通道 1 的处理方式相同。

看了源码后，也许会有读者会问，程序里面直接将定时器计数器 CNT 清零后设置新的计数是否可行。答案是不行的，因为我们要实现四个通道可以同时使用，如果 CNT 清零，将影响其它通道的使用。

35.2.4定时器中断处理

定时器中断服务程序主要用于处理 CC 捕获比较中断，启动单次延迟后，时间到了将执行中断服务程序里面的回调函数。用户可以在这个回调函数里面实现要做的功能。

```
/*
*********************************************************************************************************
*    函 数 名: TIMx_IRQHandler
*    功能说明: TIM 中断服务程序
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void TIM_HARD_IRQHandler(void)
{
    uint16_t itstatus = 0x0, itenable = 0x0;
    TIM_TypeDef* TIMx = TIM_HARD;


    itstatus = TIMx->SR & TIM_IT_CC1;
    itenable = TIMx->DIER & TIM_IT_CC1;

    if ((itstatus != (uint16_t)RESET) && (itenable != (uint16_t)RESET))
    {
        TIMx->SR = (uint16_t)~TIM_IT_CC1;
        TIMx->DIER &= (uint16_t)~TIM_IT_CC1;      /* 禁能 CC1 中断 */

       /* 先关闭中断，再执行回调函数。因为回调函数可能需要重启定时器 */
       s_TIM_CallBack1();
    }

    itstatus = TIMx->SR & TIM_IT_CC2;
    itenable = TIMx->DIER & TIM_IT_CC2;
    if ((itstatus != (uint16_t)RESET) && (itenable != (uint16_t)RESET))
    {
        TIMx->SR = (uint16_t)~TIM_IT_CC2;
        TIMx->DIER &= (uint16_t)~TIM_IT_CC2;      /* 禁能 CC2 中断 */

       /* 先关闭中断，再执行回调函数。因为回调函数可能需要重启定时器 */
       s_TIM_CallBack2();
    }

    itstatus = TIMx->SR & TIM_IT_CC3;
    itenable = TIMx->DIER & TIM_IT_CC3;
    if ((itstatus != (uint16_t)RESET) && (itenable != (uint16_t)RESET))
    {
        TIMx->SR = (uint16_t)~TIM_IT_CC3;
        TIMx->DIER &= (uint16_t)~TIM_IT_CC3;      /* 禁能 CC2 中断 */

       /* 先关闭中断，再执行回调函数。因为回调函数可能需要重启定时器 */
       s_TIM_CallBack3();
    }

    itstatus = TIMx->SR & TIM_IT_CC4;
    itenable = TIMx->DIER & TIM_IT_CC4;
    if ((itstatus != (uint16_t)RESET) && (itenable != (uint16_t)RESET))
```

```
52.       {
53.           TIMx->SR = (uint16_t)~TIM_IT_CC4;
54.           TIMx->DIER &= (uint16_t)~TIM_IT_CC4;      /* 禁能 CC4 中断 */
55.
56.          /* 先关闭中断，再执行回调函数。因为回调函数可能需要重启定时器 */
57.          s_TIM_CallBack4();
58.       }
59.   }
```

中断服务程序里面四个通道的处理方式是一样的，这里以通道 1 为例进行说明。

- 第 15 – 18 行，获取是否使能了 CC 中断且 CC 中断标志被置位。
- 第 20 – 24 行，清除 CC 中断标志，关闭 CC 中断，并执行回调函数。
- 第 27 - 58 行，其它通道的处理。跟通道 1 的处理方式相同。

35.3 定时器板级支持包（bsp_timer.c)

定时器单次延迟驱动文件 bsp_timer.c 供用户调用的主要是如下两个函数：

- bsp_InitHardTimer
- bsp_StartHardTimer

注意，当用户调用了函数 bsp_InitTimer，此函数里面会调用 bsp_InitHardTimer，用户无需再单独调用进行初始化。

35.3.1 函数 bsp_InitHardTimer

函数原型：

```
void bsp_InitHardTimer(void)
```

函数描述：

此函数主要用于初始化定时器的单次延迟功能。us 级别硬件定时，TIMx 将自由运行，永不停止。

注意事项：

1. 当用户调用了函数 bsp_InitTimer，此函数也会被调用，无需用户再单独调用。

35.3.2 函数 bsp_StartHardTimer

函数原型：

```
void bsp_StartHardTimer(uint8_t _CC, uint32_t _uiTimeOut, void * _pCallBack)
```

函数描述：

使用 TIM2-5 做单次定时器使用，定时时间到后执行回调函数。可以同时启动 4 个定时器通道，互不干扰。定时精度正负 1us（主要耗费在调用本函数的执行时间）。

函数参数：

- 第 1 个参数表示使用的捕获比较通道，数值范围 1，2，3， 4，分别表示通道 1，通道 2，通道 3 和通道 4。
- 第 2 个参数是超时时间，单位 1us。对于 16 位定时器，最大 0xFFFF 微秒，即 65.5 毫秒，对于 32 位定时器，最大 0xFFFFFFFF 微秒，即 4294 秒。
- 第 3 个参数是超时时间到后，被执行的回调函数。

注意事项：

1. 根据使用的 16 位定时器或 32 位定时器，设置的超时时间不可超出范围。

使用举例：

可以看本章节配套的实例。

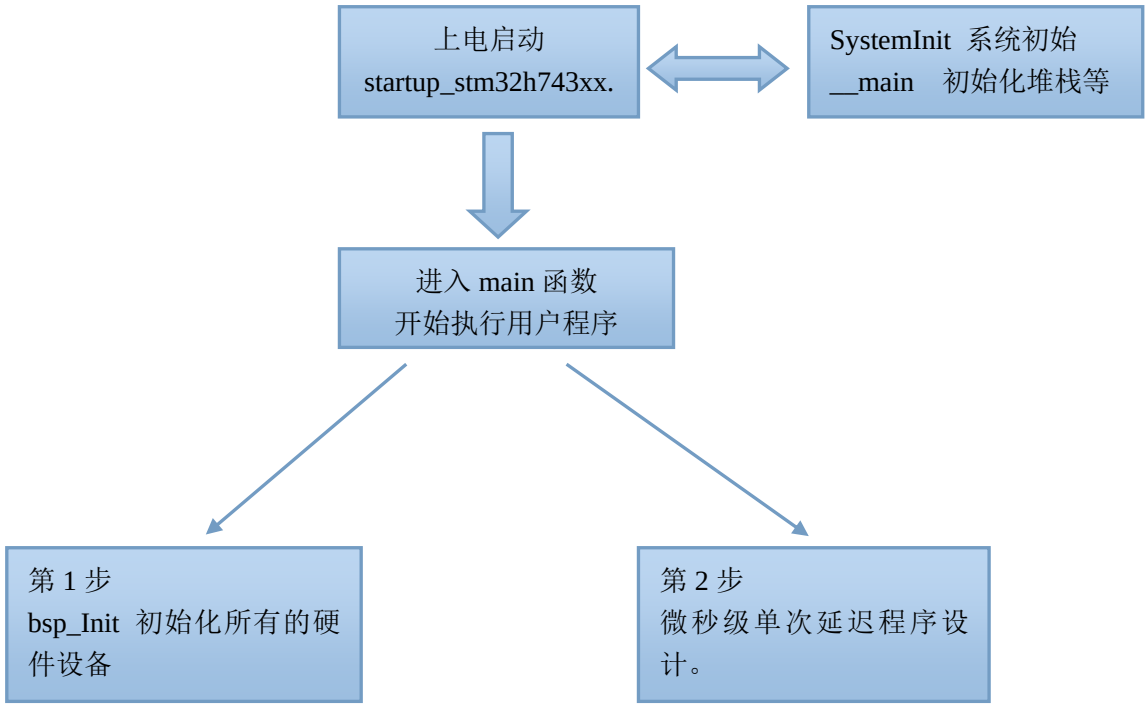
35.4 定时器驱动移植和使用

定时器的移植比较简单：

- 第 1 步：复制 bsp_timer.c 和 bsp_timer.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：这几个驱动文件主要用到 HAL 库的 GPIO 和 TIM 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 3 步，应用方法看本章节配套例子即可。

35.5 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED 和串口。
  - 第 2 步，借助按键消息，方便用户测量不同微秒延迟实际耗时。

35.6 实验例程说明（MDK）

配套例子：

V7-020_定时器四个比较捕获通道实现微妙级单次延迟（驱动支持 TIM2-TIM5）

实验目的：

1. 学习定时器实现微秒级单次延迟。

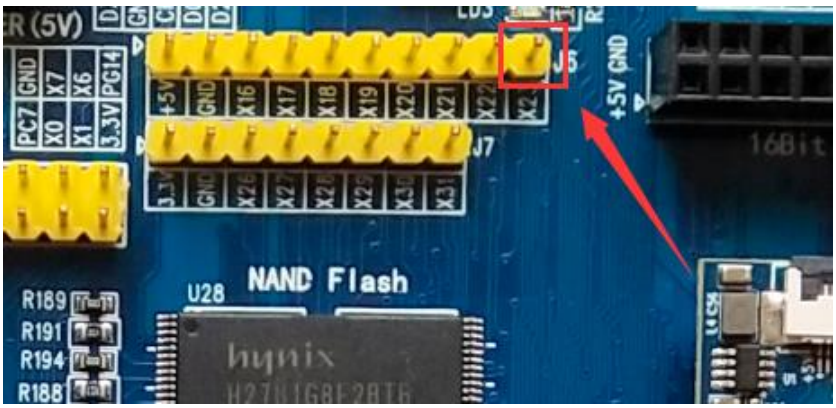
实验内容：

1. 系统上电后驱动了 1 个软件定时器，每 100ms 翻转一次 LED2。
2. STM32H7 支持 TIM1-TIM8，TIM12-TIM17 共 14 个定时器，而中间的 TIM9，TIM10，TIM11 是不存在的。
3. 在不需要任何补偿的情况下，误差可以做到正负 1 微秒以内。
4. 通过测量 FMC 扩展引脚 23，可以测试单次延迟的实际执行时间。

实验操作：

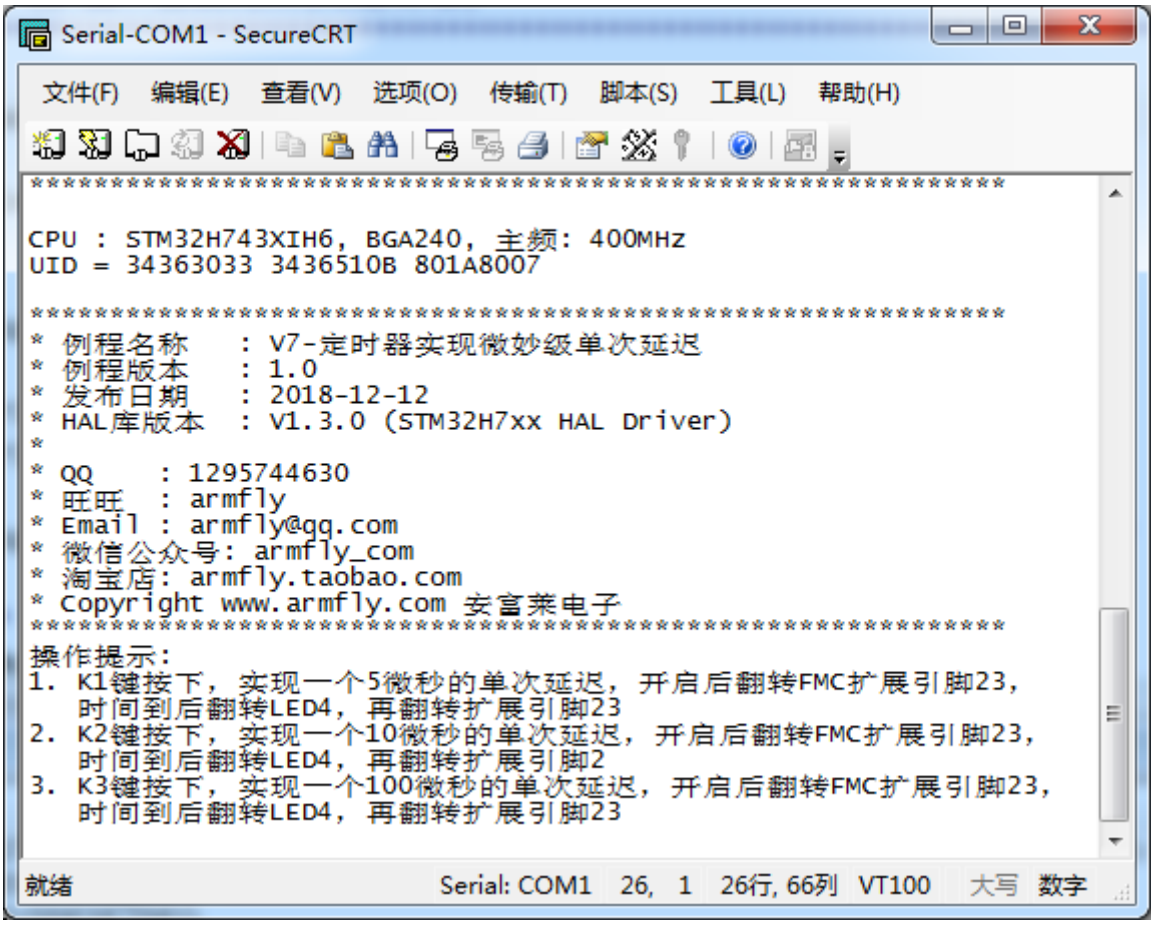
1. K1 键按下，实现一个 5 微秒的单次延迟，开启后翻转 FMC 扩展引脚 23，时间到后翻转 LED4，再翻转扩展引脚 23。
2. K2 键按下，实现一个 10 微秒的单次延迟，开启后翻转 FMC 扩展引脚 23，时间到后翻转 LED4，再翻转扩展引脚 23。
3. K3 键按下，实现一个 100 微秒的单次延迟，开启后翻转 FMC 扩展引脚 23，时间到后翻转 LED4，再翻转扩展引脚 23。

FMC 扩展引脚 23 的位置：



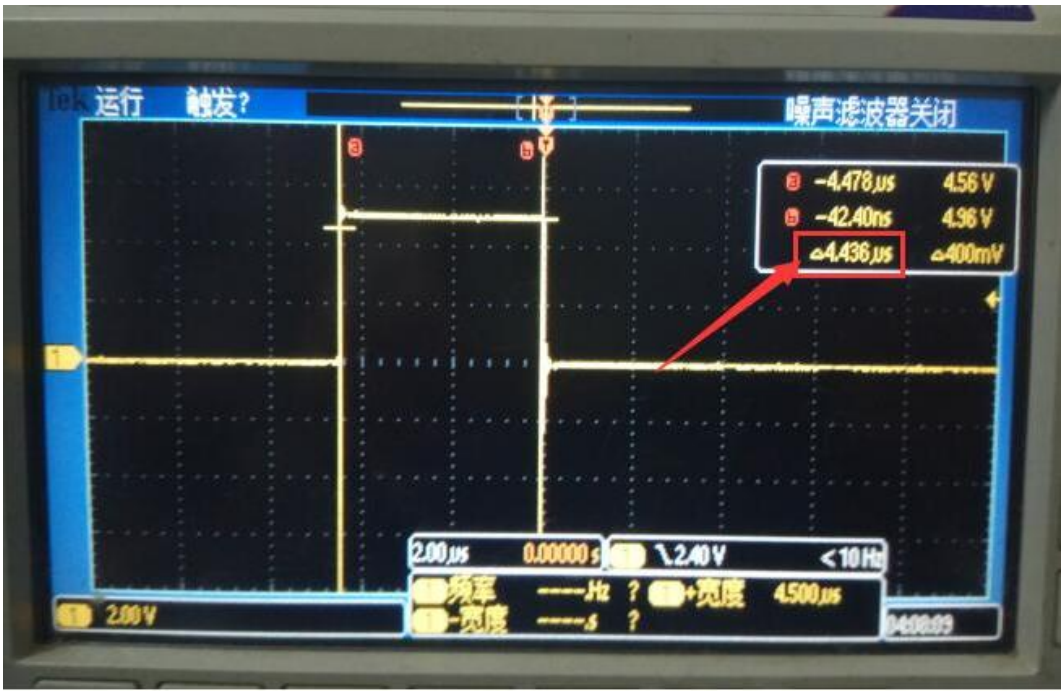
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

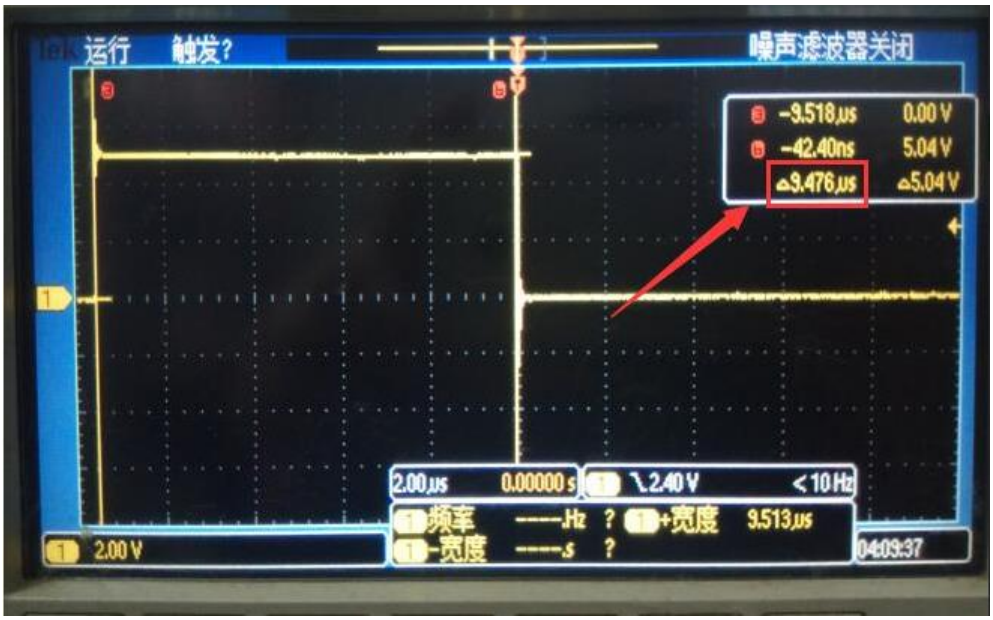


实际执行时间测量：

在不做任何误差补偿的情况下，误差在正负 1 微妙内，下面是延迟 5 微妙的实际执行时间：

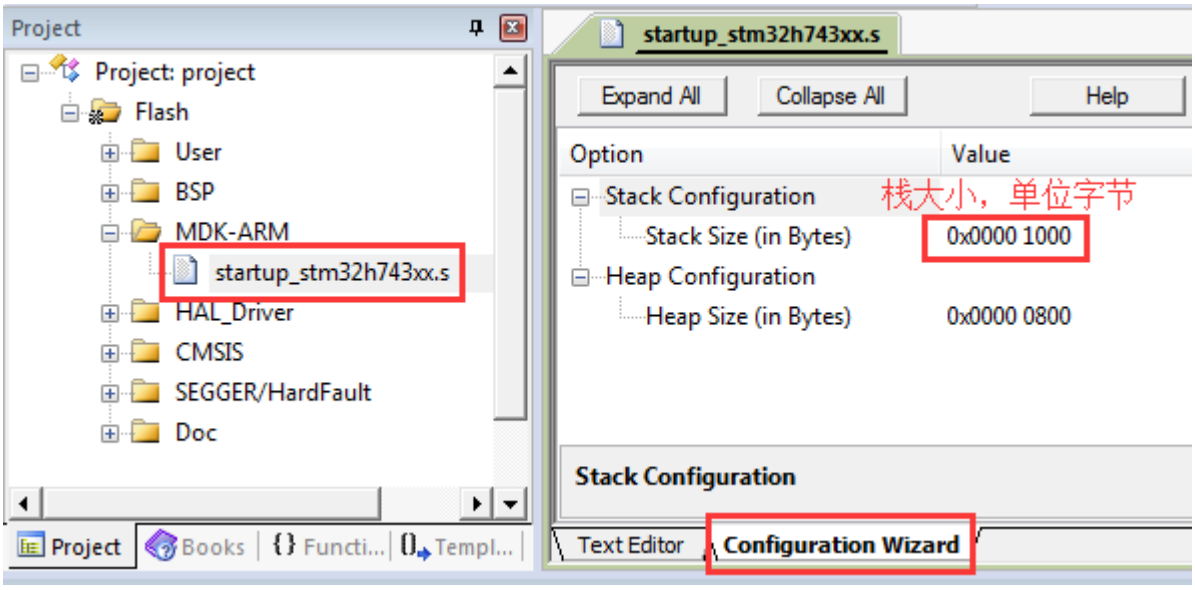


下面是延迟 10 微妙的实际执行时间：

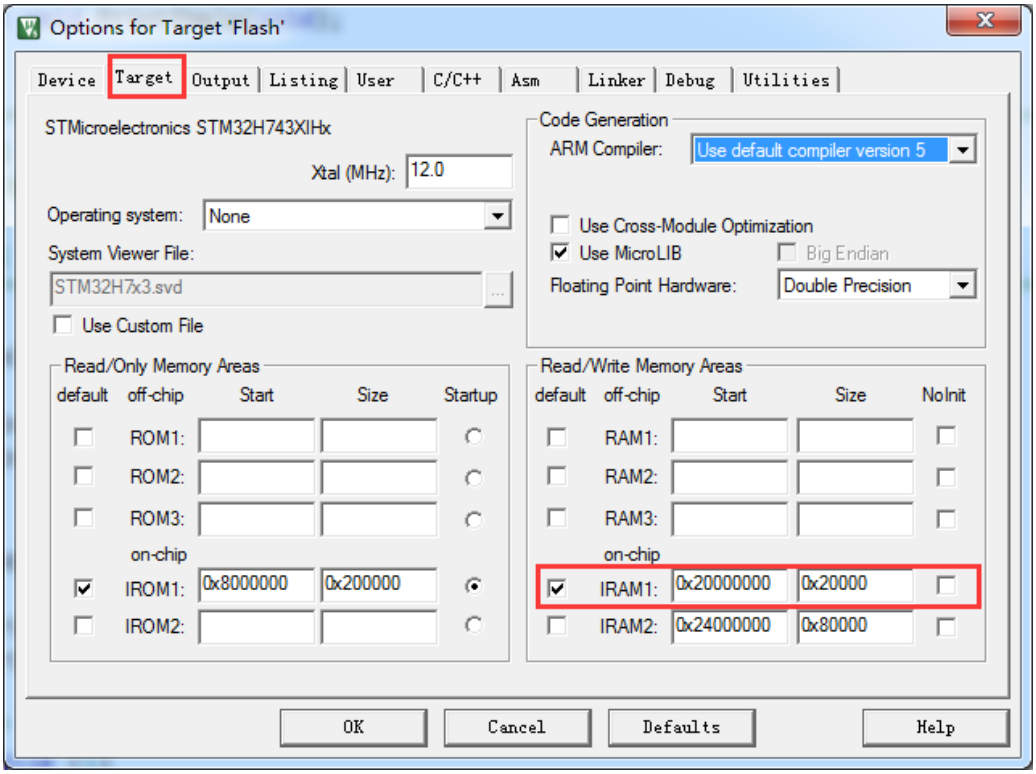


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
```

```
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- K1 键按下，实现一个 5 微秒的单次延迟，开启后翻转 FMC 扩展引脚 23，时间到后翻转 LED4，再翻转扩展引脚 23。
- K2 键按下，实现一个 10 微秒的单次延迟，开启后翻转 FMC 扩展引脚 23，时间到后翻转 LED4，再翻转扩展引脚 23。
- K3 键按下，实现一个 100 微秒的单次延迟，开启后翻转 FMC 扩展引脚 23，时间到后翻转 LED4，再翻转扩展引脚 23。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */
```

```
    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，实现一个 5 微秒的单次延迟 */
                    bsp_StartHardTimer(1 ,5, (void *)TIM_CallBack2);
                 HC574_TogglePin(GPIO_PIN_23);
                    break;

                case KEY_DOWN_K2:           /* K2 键按下，实现一个 10 微秒的单次延迟 */
                    bsp_StartHardTimer(1 ,10, (void *)TIM_CallBack2);
                 HC574_TogglePin(GPIO_PIN_23);
                    break;

                case KEY_DOWN_K3:           /* K3 键按下，实现一个 100 微秒的单次延迟 */
                    bsp_StartHardTimer(1 ,100, (void *)TIM_CallBack2);
                 HC574_TogglePin(GPIO_PIN_23);
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

注意回调函数的处理：

```
/*
*********************************************************************************************************
*    函 数 名: TIM_CallBack2
*    功能说明: 定时器中断的回调函数，此函数被 bsp_StartHardTimer 所调用。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
static void TIM_CallBack2(void)
{
    HC574_TogglePin(GPIO_PIN_23);
    bsp_LedToggle(4);
}
```

35.7 实验例程说明（IAR）

配套例子：

V7-020_定时器四个比较捕获通道实现微妙级单次延迟（驱动支持 TIM2-TIM5）

实验目的：

1. 学习定时器实现微秒级单次延迟。

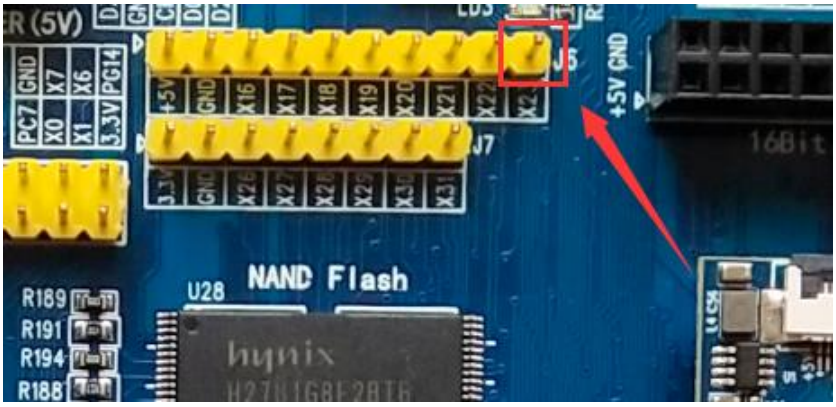
实验内容：

1. 系统上电后驱动了 1 个软件定时器，每 100ms 翻转一次 LED2。
2. STM32H7 支持 TIM1-TIM8，TIM12-TIM17 共 14 个定时器，而中间的 TIM9，TIM10，TIM11 是不存在的。
3. 在不需要任何补偿的情况下，误差可以做到正负 1 微秒以内。
4. 通过测量 FMC 扩展引脚 23，可以测试单次延迟的实际执行时间。

实验操作：

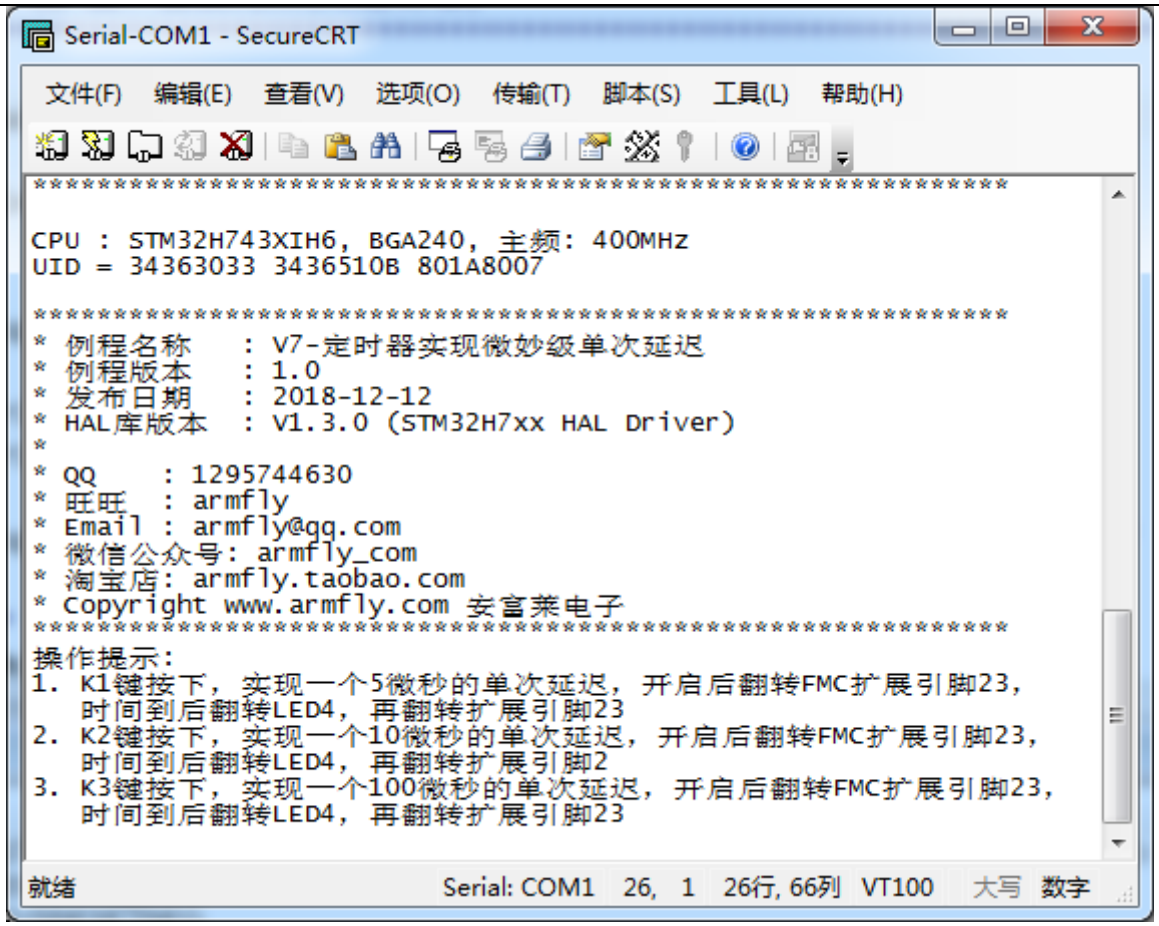
1. K1 键按下，实现一个 5 微秒的单次延迟，开启后翻转 FMC 扩展引脚 23，时间到后翻转 LED4，再翻转扩展引脚 23。
2. K2 键按下，实现一个 10 微秒的单次延迟，开启后翻转 FMC 扩展引脚 23，时间到后翻转 LED4，再翻转扩展引脚 23。
3. K3 键按下，实现一个 100 微秒的单次延迟，开启后翻转 FMC 扩展引脚 23，时间到后翻转 LED4，再翻转扩展引脚 23。

FMC 扩展引脚 23 的位置：



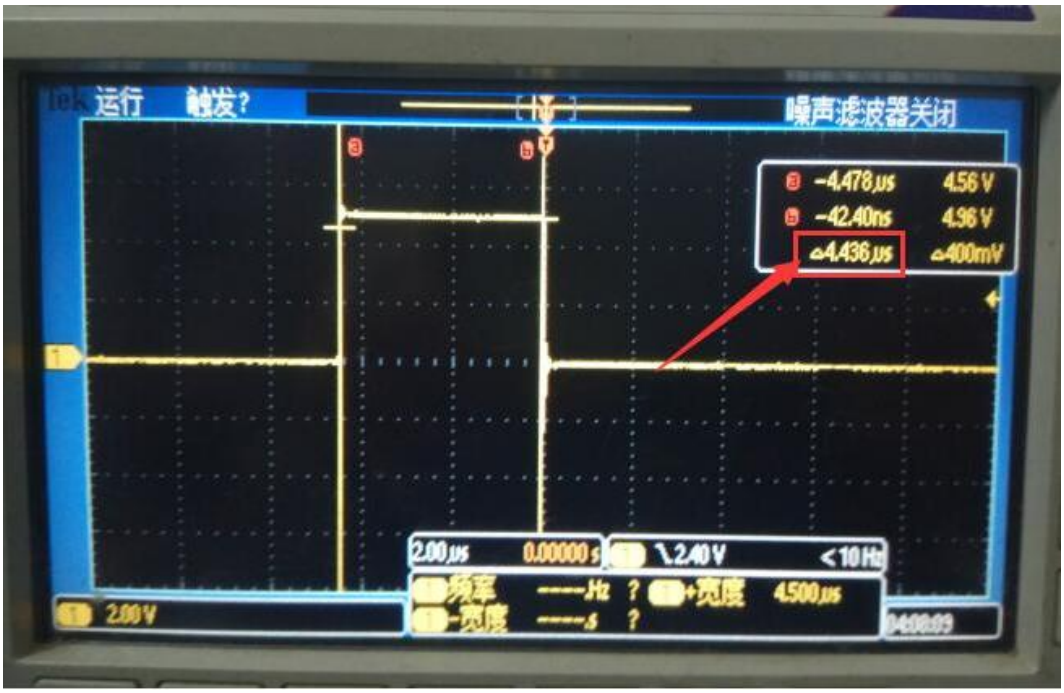
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

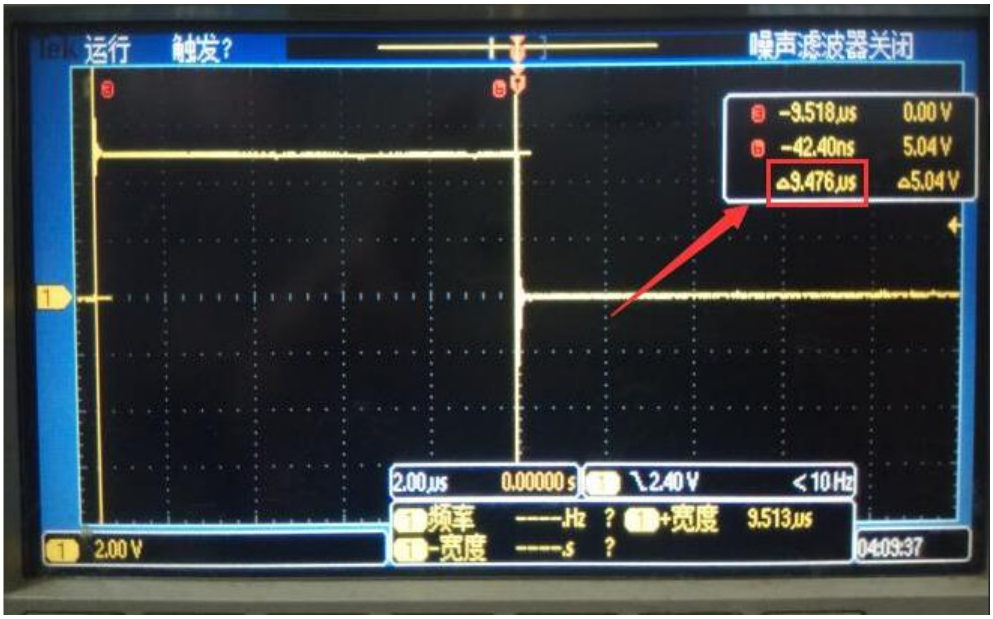


实际执行时间测量：

在不做任何误差补偿的情况下，误差在正负 1 微妙内，下面是延迟 5 微妙的实际执行时间：

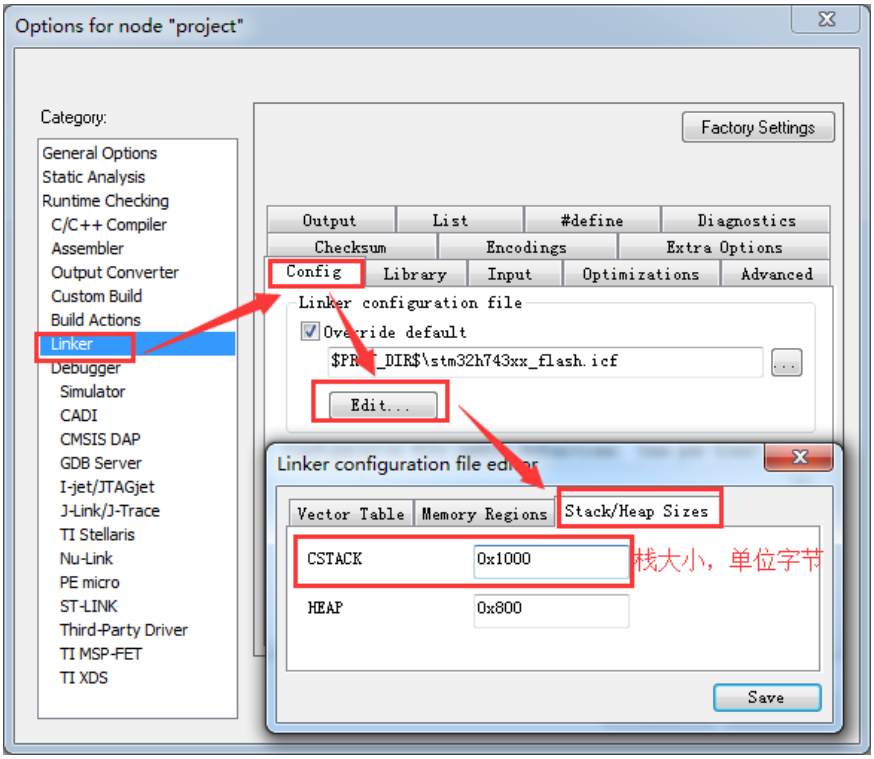


下面是延迟 10 微妙的实际执行时间：

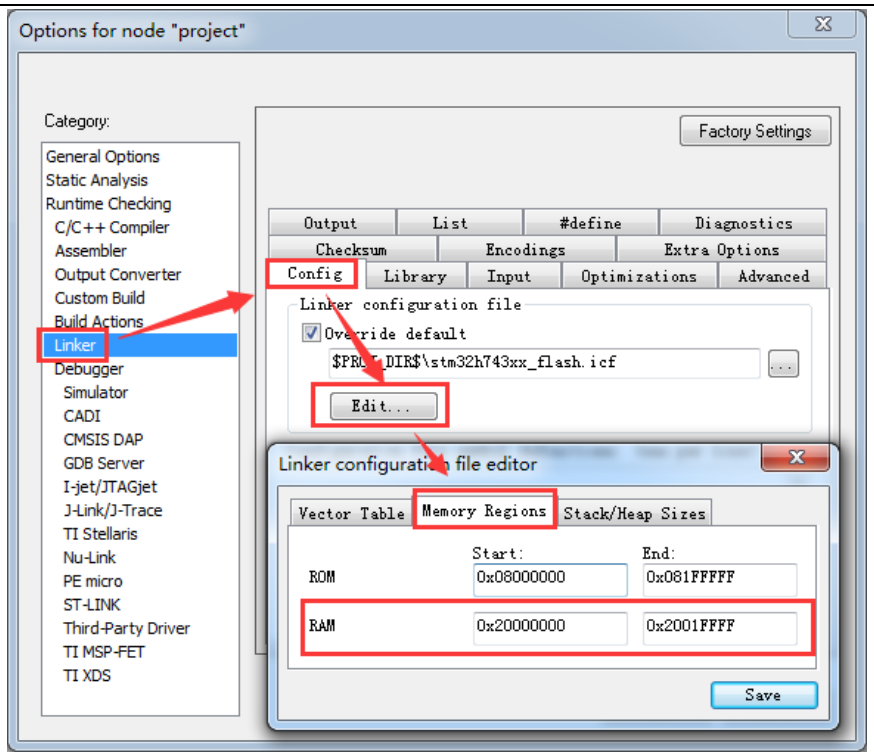


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
```

```
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能:**

主程序实现如下操作:

- K1 键按下，实现一个 5 微秒的单次延迟，开启后翻转 FMC 扩展引脚 23，时间到后翻转 LED4，再翻转扩展引脚 23。
- K2 键按下，实现一个 10 微秒的单次延迟，开启后翻转 FMC 扩展引脚 23，时间到后翻转 LED4，再翻转扩展引脚 23。
- K3 键按下，实现一个 100 微秒的单次延迟，开启后翻转 FMC 扩展引脚 23，时间到后翻转 LED4，再翻转扩展引脚 23。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


    bsp_Init();         /* 硬件初始化 */
```

```
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，实现一个 5 微秒的单次延迟 */
                    bsp_StartHardTimer(1 ,5, (void *)TIM_CallBack2);
                 HC574_TogglePin(GPIO_PIN_23);
                    break;

                case KEY_DOWN_K2:           /* K2 键按下，实现一个 10 微秒的单次延迟 */
                    bsp_StartHardTimer(1 ,10, (void *)TIM_CallBack2);
                 HC574_TogglePin(GPIO_PIN_23);
                    break;

                case KEY_DOWN_K3:           /* K3 键按下，实现一个 100 微秒的单次延迟 */
                    bsp_StartHardTimer(1 ,100, (void *)TIM_CallBack2);
                 HC574_TogglePin(GPIO_PIN_23);
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

注意回调函数的处理：

```
/*
*********************************************************************************************************
*    函 数 名: TIM_CallBack2
*    功能说明: 定时器中断的回调函数，此函数被 bsp_StartHardTimer 所调用。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void TIM_CallBack2(void)
```

```
{
    HC574_TogglePin(GPIO_PIN_23);
    bsp_LedToggle(4);
}
```

35.8 总结

本章节就为大家讲解这么多，单次延迟在实际项目中用到的地方较多，如 Modbus 帧符间隔，定时采集一段时间波形等，望初学者务必掌握。

第36章 STM32H7 的 LPTIM 低功耗定时器基础知识和 HAL 库 API

本章节为大家讲解 LPTIM1 – LPTIM5 共计 5 个定时器的基础知识和对应的 HAL 库 API。

36.1 初学者重要提示

36.2 低功耗定时器基础知识

36.3 低功耗定时器的 HAL 库用户

36.4 源文件 stm32h7xx_hal_lptim.c

36.5 总结

36.1 初学者重要提示

- 使用 LPTIM 的好处是系统处于睡眠、停机状态依然可以正常工作（除了待机模式）。停机状态可以正常工作的关键是 LSE，LSI 时钟不会被关闭，同时也可以选择使用外部时钟源。
- 重点学习本章的 2.9 小节，对于 LPTIM 的认识起到至关重要的作用。

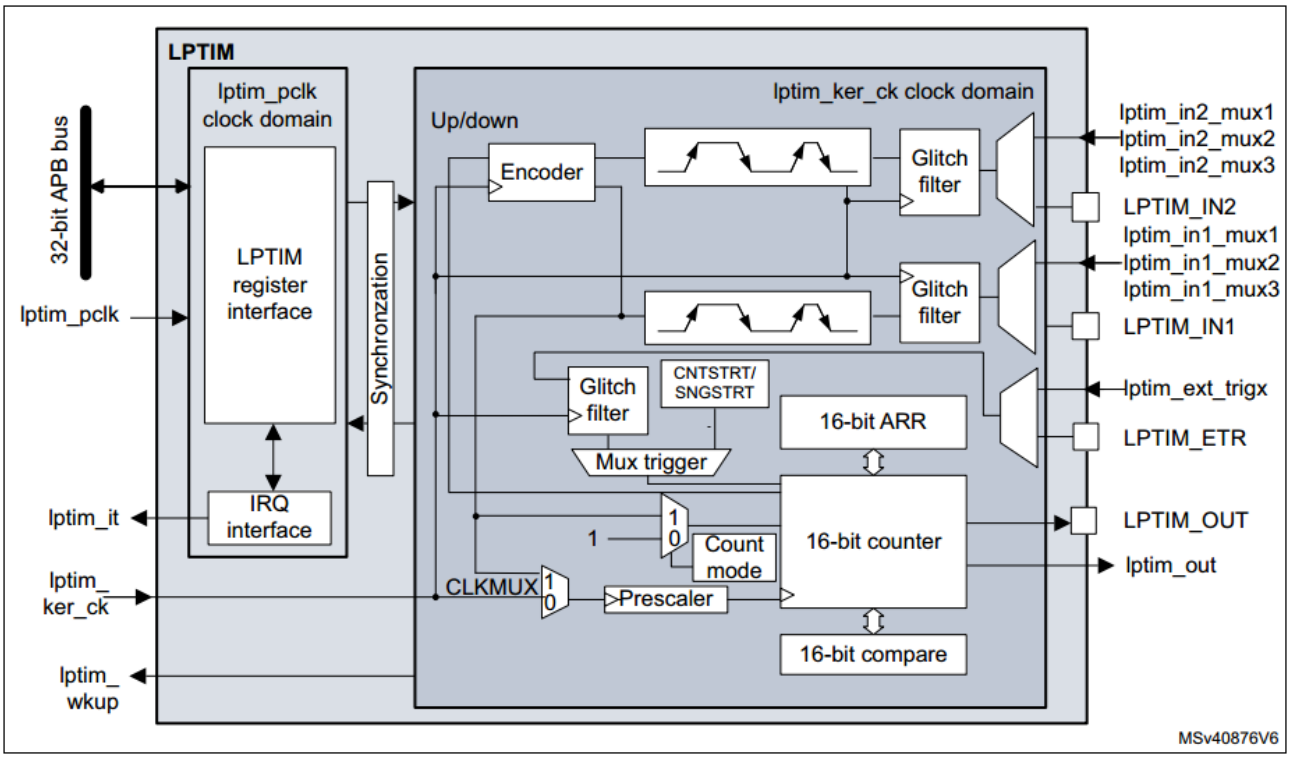
36.2 低功耗定时器基础知识

下面将低功耗定时器应用中要用到的基础知识做个介绍。

36.2.1 定时器的硬件框图

认识一个外设，最好的方式就是看它的框图，方便我们快速的了解定时器的基本功能，然后再看手册了解细节。

下面我们直接看最复杂的 LPTIM1&LPTIM2 框图：



通过这个框图，我们可以得到如下信息：

- lptim_pclk 接口

  主要用于 LPTIM 的寄存器提供时钟，寄存器操作需要高速时钟，不能像 LPTIM 的其它部分一样使用 LSI，LSE 等低速时钟。

- lptim_it 接口

  用于触发中断。

- lptim_ker_ck 接口

  内核时钟，供 lptim 使用。lptim_ker_tim 接入到 CLKMUX 双路选择器的一个输入端，另一个输入端是 LPTIM_IN1 或者 LPTIM_IN2 的输入。也就是说 LPTIM 的计数器可以选择 lptim_ker_ck，也可以选择 LPTIM_IN1 或者 LPTIM_IN。

  注意：CLKMUX 双路选择器对应的是 CFGR 寄存器的 bit0:CKSEL，用于控制内核时钟选择由内部时钟源（APB 时钟或 LSE、LSI 和 HSI 等任何其他内置振荡器）提供时钟。也可以选择由外部时钟源通过 LPTIM 外部 Input 提供时钟。

  Count mode 对应的是 CFGR 寄存器的 bit23:COUNTMODE 计数模式位,用于选择 LPTIM 使用哪个时钟源来为计数器提供时钟。可以选择计数器在每个内部时钟脉冲后递增，或者在 LPTIM 外部 Input 上的每个有效时钟脉冲后递增。

- lptim_wkup

  用于将系统从睡眠或者停机模式唤醒。

- lptim_out 和 LPTIM_OUT

lptim_out 表示计数器输出，用于内部触发。

LPTIM_OUT 表示 GPIO 的输出通道，用于 PWM。

- LPTIM_ETR

  通过 GPIO 为 LPTIM 提供外部触发。

- lptim_ext_trigx

  LPTIM 的计数器既可以通过软件启动，也可以通过外部触发启动，有 8 种触发方式可供选择，以 LPTIM1 为例，支持的触发如下：

Table 344. LPTIM1 external trigger connection

TRIGSEL	External trigger
lptim_ext_trig0	GPIO pin as LPTIM1_ETR alternate function
lptim_ext_trig1	RTC_ALARMA
lptim_ext_trig2	RTC_ALARMB
lptim_ext_trig3	RTC_TAMP1_OUT
lptim_ext_trig4	RTC_TAMP2_OUT
lptim_ext_trig5	RTC_TAMP3_OUT

Table 344. LPTIM1 external trigger connection (continued)

TRIGSEL	External trigger
lptim_ext_trig6	COMP1_OUT
lptim_ext_trig7	COMP2_OUT

- LPTIM_IN1，lptim_in1_mux1，lptim_in1_mux2 和 lptim_in1_mux3

  LPTIM_IN2 实际对应的是多路选择器的 mux0，通过 GPIO 输入。

  lptim_in1_mux1 到 lptim_in1_mux3 对应的是内部输入，以 LPTIM1 为例，支持的输入信号如下：

Table 349. LPTIM1 input 1 connection

lptim_in1_mux	LPTIM1 input 1 connected to
lptim_in1_mux0	GPIO pin as LPTIM1_IN1 alternate function
lptim_in1_mux1	COMP1_OUT
lptim_in1_mux2	Not connected
lptim_in1_mux3	Not connected

- LPTIM_IN2，lptim_in2_mux1，lptim_in2_mux2 和 lptim_in2_mux3

  LPTIM_IN2 实际对应的是多路选择器的 mux0，通过 GPIO 输入。

  lptim_in2_mux1 到 lptim_in2_mux3 对应的是内部输入，以 LPTIM1 为例，支持的输入信号如下：

Table 350. LPTIM1 input 2 connection

Iptim_in2_mux	LPTIM1 input 2 connected to
Iptim_int2_mux0	GPIO pin as LPTIM1_IN2 alternate function
Iptim_in2_mux1	COMP2_OUT
Iptim_in2_mux2	Not connected
Iptim_in2_mux3	Not connected

◆ Glitch filter（干扰滤波器）

从框图中可以看到有三组 Glitch filter，LPTIM_IN1 和 LPTIM_IN2 接入多路选择器后各有一组，LPTIM_ETR 接入后，也有一组。Glitch filter 的作用是避免任何毛刺和噪声干扰在 LPTIM 内部传播，从而防止产生意外计数或触发。

注意：使用 Glitch filter 要向 LPTIM 提供内部时钟源。

36.2.2 低功耗定时器的基本功能

LPTIM1 – LPTIM5 都是 16 位的低功耗定时器(自动重载寄存器、比较寄存器和计数器都是 16 位的)，相比 TIM1 – TIM17 这种通用定时器，在睡眠或者停机模式下依然可以工作（待机模式除外）。低功耗模式下要工作，就必然要支持低速时钟 LSI、LSE 或者外部输入时钟，这点是与通用定时器的本质区别。同时 LPTIM 的中断还可以唤醒停机模式，这点比较重要（休眠模式是任何中断都可以唤醒的，而停机模式可以 LPTIM 中断唤醒）。

以下几点是大家在使用中必须要了解到的：

◆ TIM1 – TIM17 有专门的分频寄存器，而 LPTIM1 – LPTIM5 的分频是几种固定的值。

Table 354. Prescaler division ratios

programming	dividing factor
000	/1
001	/2
010	/4
011	/8
100	/16
101	/32
110	/64
111	/128

◆ 低功耗定时器支持以下 6 种模式：

- PWM 模式

- 单脉冲模式
- 单次模式

  在此模式下，当满足匹配条件时，输出可以切换高低电平（如果输出极性配置为高，则为低电平至高电平变化，反之亦然）。
- 编码器模式
- 超时模式

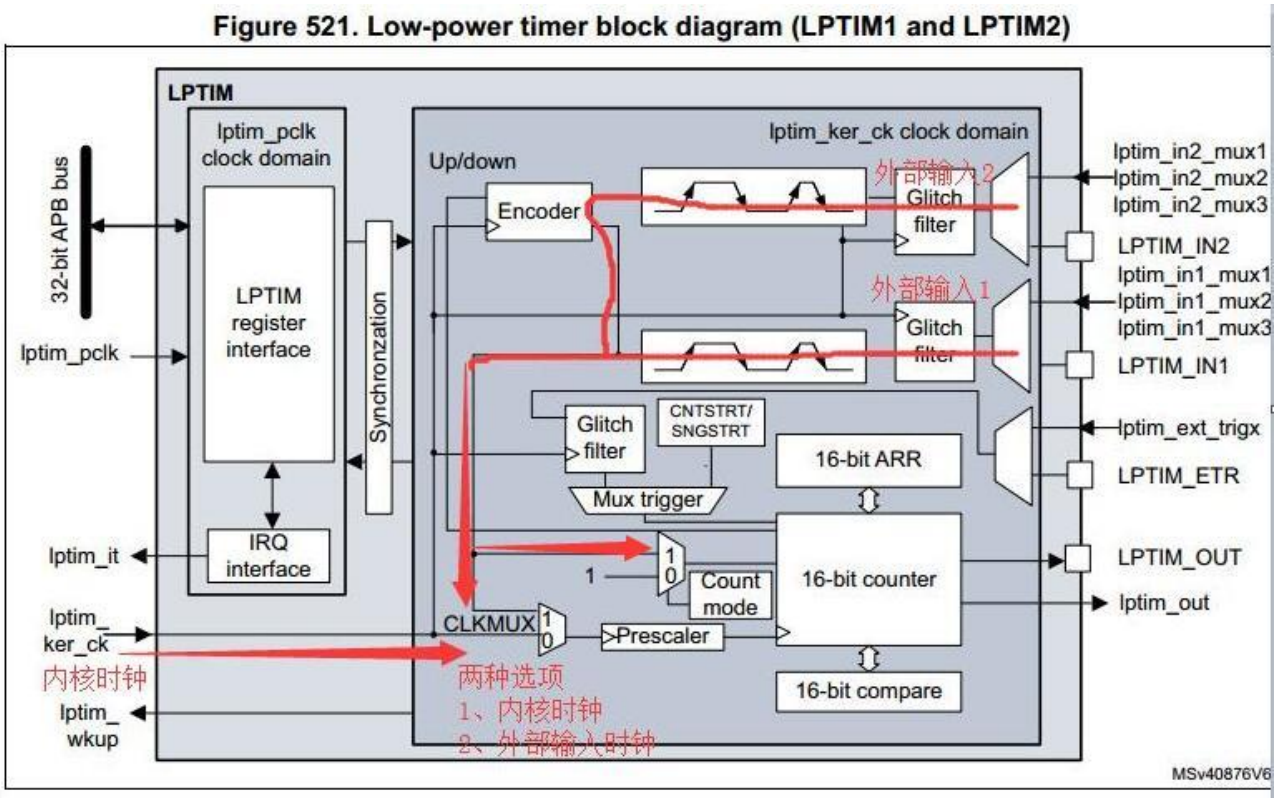
  有效的边沿触发输入可复位定时器。第一个触发事件将启动计时器，任何连续触发事件将重置计数器并重新开始。
- 计数器模式：

  计数器可用于计算来自 Input1 的外部事件或用于计算内部时钟周期。

36.2.3 低功耗定时器时钟选择问题（重要）

这个知识点比较重要，可以帮助大家更好的理解 LPTIM。下面先看框图：

Figure 521. Low-power timer block diagram (LPTIM1 and LPTIM2)



首先将框图里面两个最重要的标识跟寄存器对上号。

◆ lptim_ker_ck 接口

内核时钟，供 lptim 使用。lptim_ker_tim 接入到 CLKMUX 双路选择器的一个输入端，另一个输入端是 LPTIM_IN1 或者 LPTIM_IN2 的输入。也就是说 LPTIM 的计数器可以选择 lptim_ker_ck，也可以选择 LPTIM_IN1 或者 LPTIM_IN。

- 最关键的地方来了
  - CLKMUX 多路选择器对应的是 CFGR 寄存器的 bit0:CKSEL
    - 用于控制内核时钟选择由内部时钟源(APB 时钟或 LSE、LSI 和 HSI 等任何其他内置振荡器)提供时钟。
    - 也可以选择由外部时钟源通过 LPTIM 外部 Input 提供时钟。
  - Count mode 对应的是 CFGR 寄存器的 bit23:COUNTMODE 计数模式位,用于选择 LPTIM 使用哪个时钟源来为计数器提供时钟。
    - 可以选择计数器在每个内部时钟脉冲后递增。
    - 或者在 LPTIM 外部 Input 上的每个有效时钟脉冲后递增。
- 应用的时候，我们可以选择
  - CKSEL = 0 ， COUNTMODE = 0
    表示 LPTIM 内核时钟使用的内部时钟源，计数器通过内部时钟脉计数。
  - CKSEL = 0 ， COUNTMODE = 1
    表示 LPTIM 内核时钟使用的内部时钟源，计数器通过外部输入脉冲计数。
  - CKSEL = 1 ， COUNTMODE = x
    表示 LPTIM 内核时钟使用的外部时钟源，计数器通过外部输入脉冲计数。

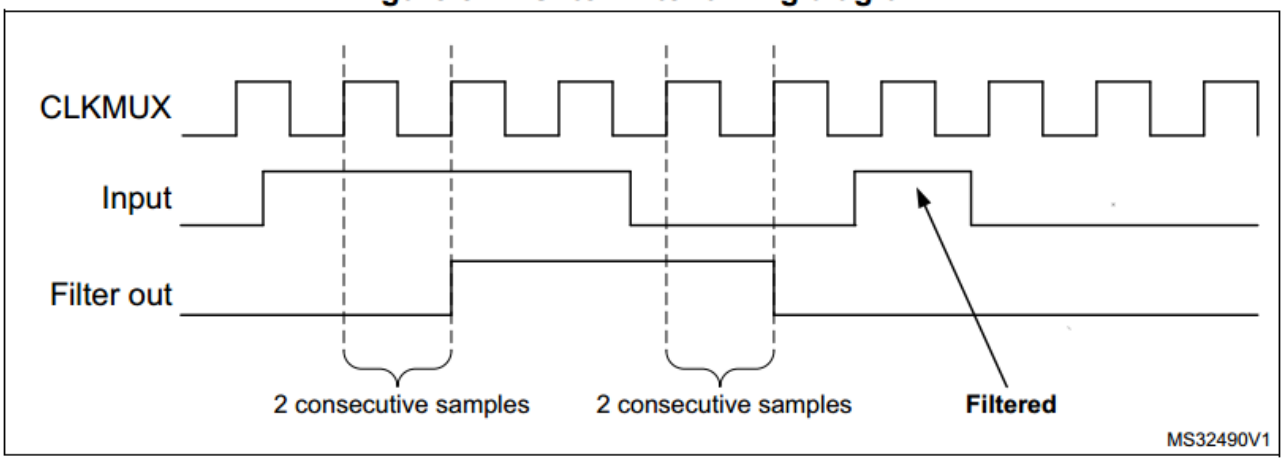
36.2.4干扰滤波器（Glitch filter）

Glitch filter 干扰滤波器的作用是避免任何毛刺和噪声干扰在 LPTIM 内部传播，从而防止产生意外计数或触发。

实现原理就是 LPTIM 的 CFGR 寄存器有专门的控制位 TRGFLT[1:0]（用于滤波外部触发信号）和 CKFLT[1:0]（用于滤波外部输入时钟）来控制信号，其有效电平变化必须至少稳定 2/4/8 个时钟周期才能将其视为有效触发。

比如下面的截图，配置为稳定 2 个时钟周期才算有效信号。

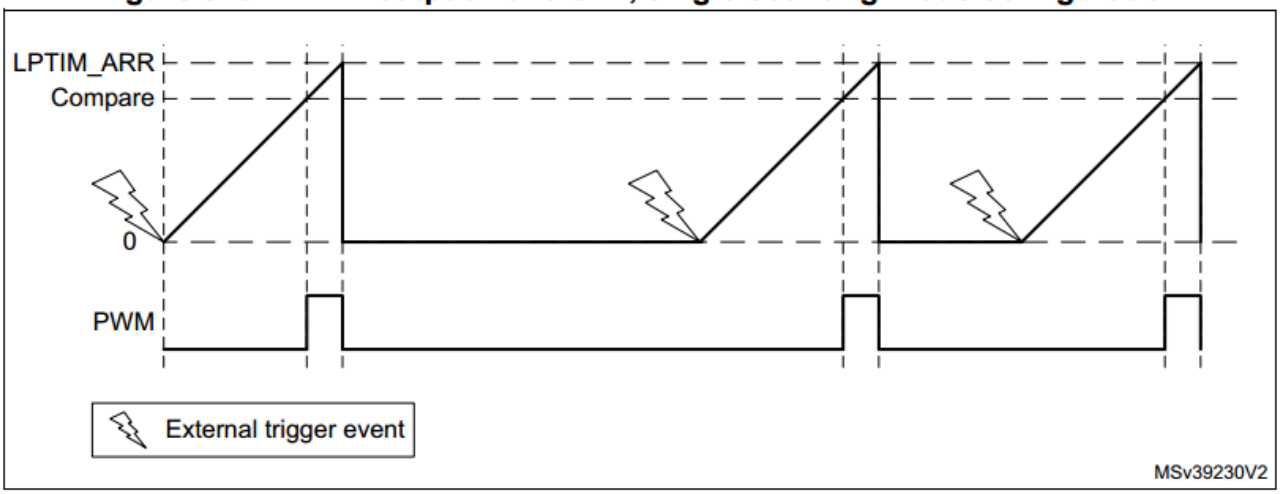
Figure 524. Glitch filter timing diagram



36.2.5 单次触发和连续模式

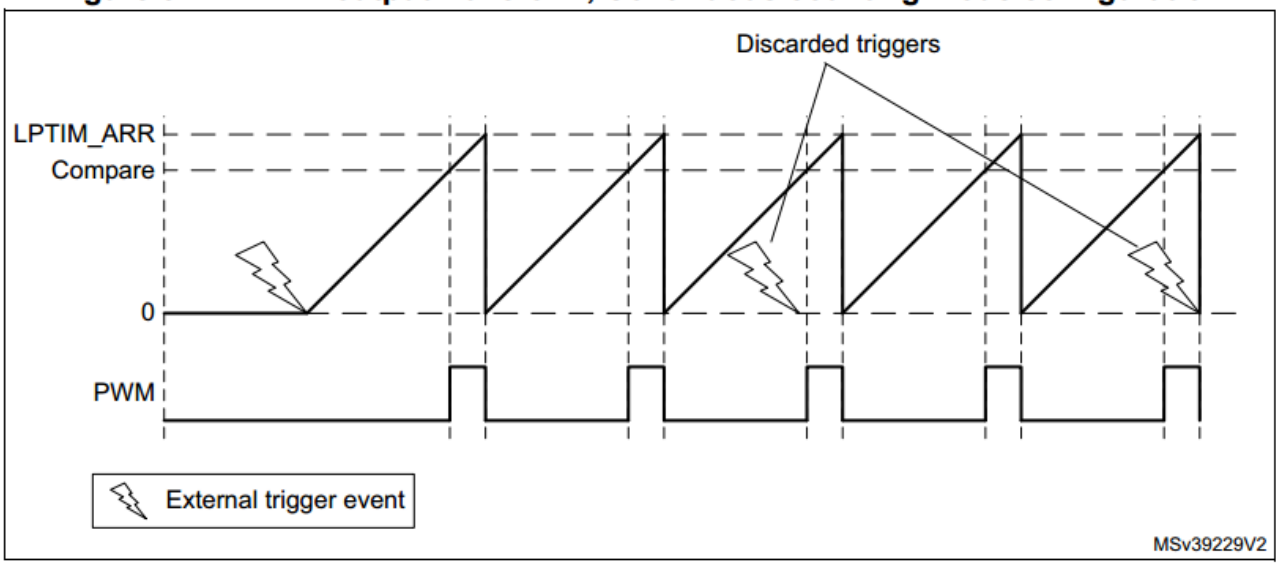
单次触发的含义就是定时器由触发事件启动，当达到 ARR 值时停止，效果如下：

Figure 525. LPTIM output waveform, single counting mode configuration



连续模式的含义是定时器由触发事件启动，并且直到被禁止才会停止，效果如下：

Figure 527. LPTIM output waveform, Continuous counting mode configuration



36.2.6 溢出模式

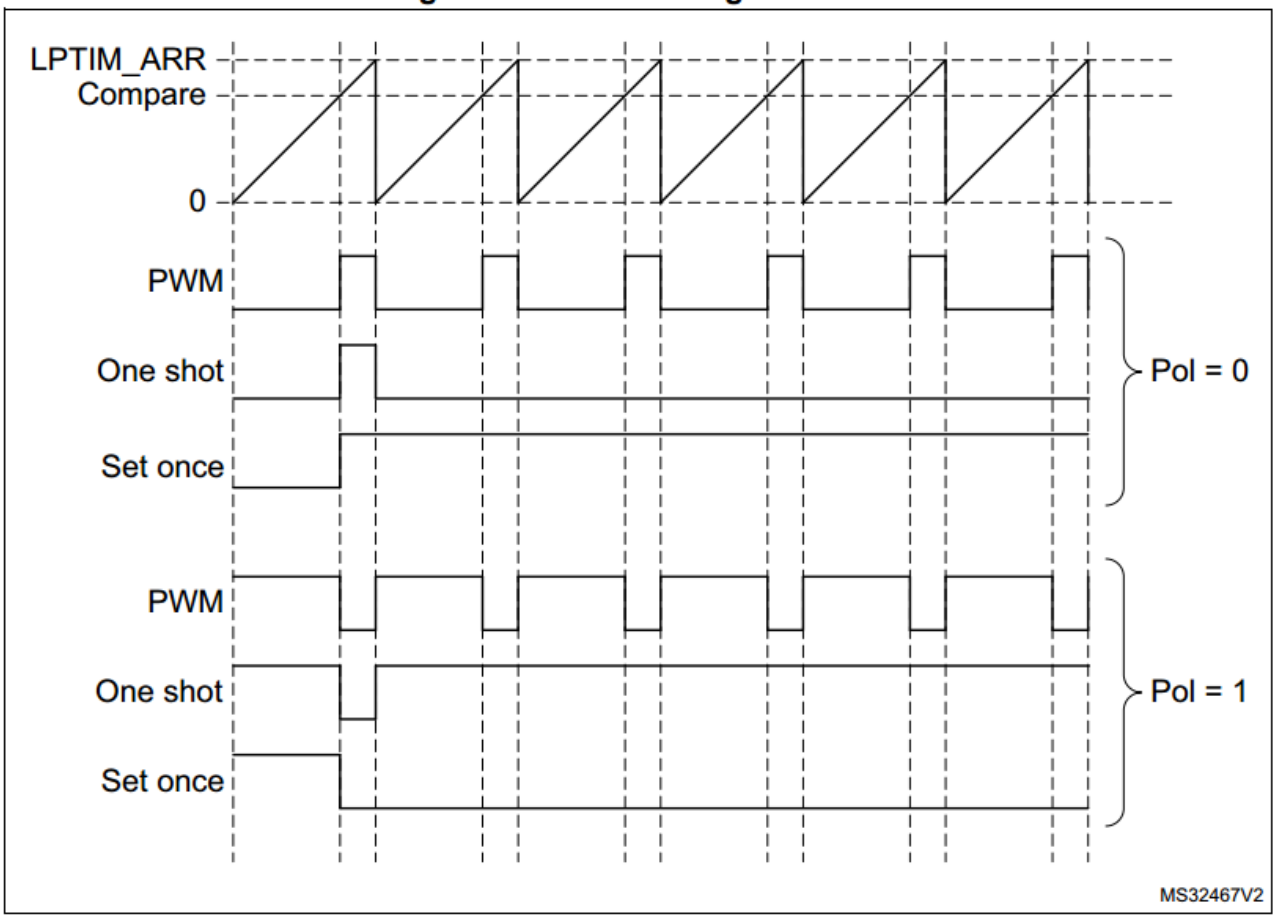
注：这个模式用来做停机模式唤醒比较方便。

检测引脚第 1 次检查到触发信号，LPTIM 就开始工作了，在溢出时间内检测到的触发信号都将复位计数，定时器重新开始工作。如果溢出内没有再接收到触发信号，仅进入溢出中断。

36.2.7 波形输出

通过下面的截图，可以让大家对低功耗定时器的波形输出效果有个全面认识。

Figure 528. Waveform generation



LPTIM_ARR 是自动重载寄存器，Compare 是比较寄存器。当定时器的计数器达到 Compare 后，GPIO 输出高电平还是低电平，是由 CFGR 寄存器的 bit2:1：WAVPOL 波形极性决定的。

以 PWM 输出为例：

- 如果 WAVPOL = 0 表示计数器的数值介于 Compare 和 LPTIM_ARR 之间时，GPIO 输出高电平。其它时间是低电平。
- 如果 WAVPOL = 1 表示计数器的数值介于 Compare 和 LPTIM_ARR 之间时，GPIO 输出低电平。其它时间是高电平。

One–Shot 效果跟 PWM 一样，不过 GPIO 仅输出 1 次脉冲。

Set–Once 特殊些，计数到 ARR 后，GPIO 输出结果将一直保持达到 Compare 寄存器数值的输出电平。

36.2.8低功耗定时器 LPTIM1 – LPTIM5 的区别

关于这五个低功耗定时器的区别，可以直接通过参考手册里面的框图看它们的区别。我们这里也简单整理下：

- LPTIM1 和 LPTIM2 的功能是一样的，且支持编码器模式，而 LPTIM3，LPTIM4 和 LPTIM5 均不支持。
- LPTIM3 跟 LPTIM1 的区别是仅有 1 组 LPTIM_IN 输入，且不支持 LTPTIM_ETR。
- LPTIM4 和 LPTIM5 的功能是一样的,这两个功能最弱。跟 LPTIM1 的区别是没有 LPTIM_IN 输入端，也不支持 LPTIM_ETR，仅有一个内部触发 lptim_ext_trigx。

36.3 低功耗定时器的 HAL 库用法

低功耗定时器的 HAL 库用法其实就是几个结构体变量成员的配置和使用，然后配置 GPIO、时钟，并根据需要配置 NVIC，中断和 DMA。下面我们逐一展开为大家做个说明。

36.3.1定时器寄存器结构体 LPTIM_TypeDef

低功耗定时器相关的寄存器是通过 HAL 库中的结构体 LPTIM_TypeDef 定义的，在 stm32h743xx.h 中可以找到这个类型定义：

```
typedef struct
{
  __IO uint32_t ISR;      /*!< LPTIM Interrupt and Status register,         Address offset: 0x00 */
  __IO uint32_t ICR;      /*!< LPTIM Interrupt Clear register,              Address offset: 0x04 */
  __IO uint32_t IER;      /*!< LPTIM Interrupt Enable register,             Address offset: 0x08 */
  __IO uint32_t CFGR;     /*!< LPTIM Configuration register,                Address offset: 0x0C */
  __IO uint32_t CR;       /*!< LPTIM Control register,                      Address offset: 0x10 */
  __IO uint32_t CMP;      /*!< LPTIM Compare register,                      Address offset: 0x14 */
  __IO uint32_t ARR;      /*!< LPTIM Autoreload register,                   Address offset: 0x18 */
  __IO uint32_t CNT;      /*!< LPTIM Counter register,                      Address offset: 0x1C */
  uint16_t  RESERVED1;    /*!< Reserved, 0x20                                                    */
  __IO uint32_t CFGR2;    /*!< LPTIM Option register,                     Address offset: 0x24 */
} LPTIM_TypeDef;
```

这个结构体的成员名称和排列次序和 CPU 的定时器寄存器是一 一对应的。

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

下面我们看下 LPTIM1，LPTIM2，LPTIM3，LPTIM4 和 LPTIM5 的定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE           ((uint32_t)0x40000000)
#define D2_APB2PERIPH_BASE    (PERIPH_BASE + 0x00010000)
#define D3_APB1PERIPH_BASE    (PERIPH_BASE + 0x18000000)

#define LPTIM1_BASE           (D2_APB1PERIPH_BASE + 0x2400)
```

```
#define LPTIM2_BASE             (D3_APB1PERIPH_BASE + 0x2400)
#define LPTIM3_BASE             (D3_APB1PERIPH_BASE + 0x2800)
#define LPTIM4_BASE             (D3_APB1PERIPH_BASE + 0x2C00)
#define LPTIM5_BASE             (D3_APB1PERIPH_BASE + 0x3000)


#define LPTIM1                ((LPTIM_TypeDef *) LPTIM1_BASE) <----- 展开这个宏，(TIM_TypeDef *) 0x40012400
#define LPTIM2                ((LPTIM_TypeDef *) LPTIM2_BASE)
#define LPTIM3                ((LPTIM_TypeDef *) LPTIM3_BASE)
#define LPTIM4                ((LPTIM_TypeDef *) LPTIM4_BASE)
#define LPTIM5                ((LPTIM_TypeDef *) LPTIM5_BASE)
```

我们访问 LPTIM 的 ISR 寄存器可以采用这种形式：LPTIM->ISR = 0。

36.3.2 定时器句柄结构体 LPTIM_HandleTypeDef

HAL 库在 LPTIM_TypeDef 的基础上封装了一个结构体 LPTIM_HandleTypeDef，定义如下：

```
typedef struct
{
      LPTIM_TypeDef              *Instance;         /*!< Register base address     */
      LPTIM_InitTypeDef           Init;             /*!< LPTIM required parameters */
      HAL_StatusTypeDef           Status;           /*!< LPTIM peripheral status   */
      HAL_LockTypeDef             Lock;             /*!< LPTIM locking object      */
   __IO  HAL_LPTIM_StateTypeDef   State;            /*!< LPTIM peripheral state    */

}LPTIM_HandleTypeDef;
```

这里重点介绍前两个参数，其它参数主要是 HAL 库内部使用的。

- **TIM_TypeDef *Instance**

  这个参数是寄存器的例化，方便操作寄存器，比如使能定时器的计数器。

  SET_BIT(huart->Instance->CR, LPTIM_CR_CNTSTRT)。

- **LPTIM_InitTypeDef Init**

  这个参数是用户接触最多的，用于配置低功耗定时器的基本参数。

  LPTIM_InitTypeDef 结构体的定义如下：

```
typedef struct
{
  LPTIM_ClockConfigTypeDef     Clock;
  LPTIM_ULPClockConfigTypeDef  UltraLowPowerClock;
  LPTIM_TriggerConfigTypeDef   Trigger;
  uint32_t                     OutputPolarity;
  uint32_t                     UpdateMode;
  uint32_t                     CounterSource;
  uint32_t                     Input1Source;
  uint32_t                     Input2Source;
}LPTIM_InitTypeDef;
```

  - 成员 Clock

    用于设置时钟源和时钟分频，结构体变量 LPTIM_ClockConfigTypeDef 的定义如下。

```
typedef struct
{
  uint32_t Source;
```

```
  uint32_t Prescaler;
}LPTIM_ClockConfigTypeDef;
```

时钟源参数 Source 可以选择如下两种。

（1）#define LPTIM_CLOCKSOURCE_APBCLOCK_LPOSC ((uint32_t)0x00U)

表示 LPTIM 由内部时钟源（APB 时钟或 APB 或 LSE、LSI 和 HSI 等）提供时钟。

（2）#define LPTIM_CLOCKSOURCE_ULPTIM LPTIM_CFGR_CKSEL

表示 LPTIM 由外部时钟源通过 LPTIM 外部 Input1 提供时钟。

分配参数 Prescaler 可以选择如下八种。

```
#define LPTIM_PRESCALER_DIV1              ((uint32_t)0x000000U)
#define LPTIM_PRESCALER_DIV2               LPTIM_CFGR_PRESC_0
#define LPTIM_PRESCALER_DIV4               LPTIM_CFGR_PRESC_1
#define LPTIM_PRESCALER_DIV8               ((uint32_t)(LPTIM_CFGR_PRESC_0 | LPTIM_CFGR_PRESC_1))
#define LPTIM_PRESCALER_DIV16              LPTIM_CFGR_PRESC_2
#define LPTIM_PRESCALER_DIV32             ((uint32_t)(LPTIM_CFGR_PRESC_0 | LPTIM_CFGR_PRESC_2))
#define LPTIM_PRESCALER_DIV64             ((uint32_t)(LPTIM_CFGR_PRESC_1 | LPTIM_CFGR_PRESC_2))
#define LPTIM_PRESCALER_DIV128            ((uint32_t)LPTIM_CFGR_PRESC)
```

- 成员 UltraLowPowerClock

此参数仅在使用超低功耗时钟源时使用，用于设置所选择的外部时钟，结构体变量 LPTIM_ULPClockConfigTypeDef 定义如下：

```
typedef struct
{
  uint32_t Polarity;
  uint32_t SampleTime;
}LPTIM_ULPClockConfigTypeDef;
```

时钟极性参数 Polarity 用于选择有效的时钟极性，如果使能了双边沿，Auxiliary Clock（一种低功耗振荡器）必须处于激活状态。

采样时间参数 SampleTime 用于配置时钟干扰滤波器。可以配置的参数如下：

```
#define LPTIM_CLOCKSAMPLETIME_DIRECTTRANSITION  ((uint32_t)0x00000000U)
#define LPTIM_CLOCKSAMPLETIME_2TRANSITIONS      LPTIM_CFGR_CKFLT_0
#define LPTIM_CLOCKSAMPLETIME_4TRANSITIONS      LPTIM_CFGR_CKFLT_1
#define LPTIM_CLOCKSAMPLETIME_8TRANSITIONS      LPTIM_CFGR_CKFLT
```

- 成员 Trigger

用于配置触发参数，结构体变量 LPTIM_TriggerConfigTypeDef 的定义如下：

```
typedef struct
{
  uint32_t Source;
  uint32_t ActiveEdge;
  uint32_t SampleTime;

}LPTIM_TriggerConfigTypeDef;
```

触发源参数 Source 支持的选择如下：

```
#define LPTIM_TRIGSOURCE_SOFTWARE               ((uint32_t)0x0000FFFFU)
#define LPTIM_TRIGSOURCE_0                      ((uint32_t)0x00000000U)
#define LPTIM_TRIGSOURCE_1                      ((uint32_t)LPTIM_CFGR_TRIGSEL_0)
#define LPTIM_TRIGSOURCE_2                      LPTIM_CFGR_TRIGSEL_1
#define LPTIM_TRIGSOURCE_3                      ((uint32_t)LPTIM_CFGR_TRIGSEL_0 | LPTIM_CFGR_TRIGSEL_1)
#define LPTIM_TRIGSOURCE_4                      LPTIM_CFGR_TRIGSEL_2
```

```
#define LPTIM_TRIGSOURCE_5                  ((uint32_t)LPTIM_CFGR_TRIGSEL_0 | LPTIM_CFGR_TRIGSEL_2)
#define LPTIM_TRIGSOURCE_6                  ((uint32_t)LPTIM_CFGR_TRIGSEL_1 | LPTIM_CFGR_TRIGSEL_2)
#define LPTIM_TRIGSOURCE_7                  LPTIM_CFGR_TRIGSEL
```

参数 ActiveEdge 用于设置有效的触发边沿，可以选择上升沿，下降沿或者双边沿触发。

```
#define LPTIM_ACTIVEEDGE_RISING             LPTIM_CFGR_TRIGEN_0
#define LPTIM_ACTIVEEDGE_FALLING            LPTIM_CFGR_TRIGEN_1
#define LPTIM_ACTIVEEDGE_RISING_FALLING     LPTIM_CFGR_TRIGEN
```

- 成员 OutputPolarity

  用于配置输出极性，可选择高电平或者低电平输出：

```
#define LPTIM_OUTPUTPOLARITY_HIGH           ((uint32_t)0x00000000U)
#define LPTIM_OUTPUTPOLARITY_LOW            (LPTIM_CFGR_WAVPOL)
```

- 成员 UpdateMode

  用于配置是否立即更新自动重装寄存器和比较寄存器，可以选择立即更新，或者当前周期结束后更新。

```
#define LPTIM_UPDATE_IMMEDIATE              ((uint32_t)0x00000000U)
#define LPTIM_UPDATE_ENDOFPERIOD            LPTIM_CFGR_PRELOAD
```

- 成员 CounterSource

  用于配置定时器计数器在每个内部事件或者外部事件后递增计数。可以选择内部或者外部。

```
#define LPTIM_COUNTERSOURCE_INTERNAL        ((uint32_t)0x00000000U)
#define LPTIM_COUNTERSOURCE_EXTERNAL        LPTIM_CFGR_COUNTMODE
```

- 成员 Input1Source

  用于配置 Input1 的输入源，可以选择 GPIO，比较器输出或者 SAI FSA/FSB。

```
#define LPTIM_INPUT1SOURCE_GPIO         ((uint32_t)0x00000000U)  /*!< For LPTIM1, LPTIM2 and LPTIM3 */
#define LPTIM_INPUT1SOURCE_COMP1        LPTIM_CFGR2_IN1_SEL0     /*!< For LPTIM1 and LPTIM2 */
#define LPTIM_INPUT1SOURCE_COMP2        LPTIM_CFGR2_IN1_SEL1     /*!< For LPTIM2 and LPTIM2 */
#define LPTIM_INPUT1SOURCE_COMP1_COMP2  (LPTIM_CFGR2_IN1_SEL0|LPTIM_CFGR2_IN1_SEL1)  /*!< For LPTIM2 */
#define LPTIM_INPUT1SOURCE_SAI1_FSA     LPTIM_CFGR2_IN1_SEL0                         /*!< For LPTIM3 */
#define LPTIM_INPUT1SOURCE_SAI1_FSB     LPTIM_CFGR2_IN1_SEL1                         /*!< For LPTIM3 */
```

- 成员 Input2ource

  用于配置 Input2 的输入源，可以选择 GPIO 和比较器。

  注意，此参数仅用于编码器模式，也就是说仅支持 LPTIM1 和 LPTIM2 的例化。

```
#define LPTIM_INPUT2SOURCE_GPIO         ((uint32_t)0x00000000U)       /*!< For LPTIM1 and LPTIM2 */
#define LPTIM_INPUT2SOURCE_COMP2        LPTIM_CFGR2_IN2_SEL0          /*!< For LPTIM1 and LPTIM2 */
```

下面是 LPTIM1 的配置例子：

```
LPTIM_HandleTypeDef     LptimHandle = {0};

LptimHandle.Instance = LPTIM1;

/* 对应寄存器 CKSEL，选择内部时钟源 */
LptimHandle.Init.Clock.Source    = LPTIM_CLOCKSOURCE_APBCLOCK_LPOSC;
LptimHandle.Init.Clock.Prescaler = LPTIM_PRESCALER_DIV1;        /* 设置 LPTIM 时钟分频 */
LptimHandle.Init.CounterSource   = LPTIM_COUNTERSOURCE_INTERNAL;/* LPTIM 计数器对内部时钟源计数 */
LptimHandle.Init.Trigger.Source  = LPTIM_TRIGSOURCE_SOFTWARE;   /* 软件触发 */
/* 计数器计数到比较寄存器和 ARR 自动重载寄存器之间数值，输出高电平 */
LptimHandle.Init.OutputPolarity  = LPTIM_OUTPUTPOLARITY_HIGH;
/* 比较寄存器和 ARR 自动重载寄存器选择更改后立即更新 */
LptimHandle.Init.UpdateMode      = LPTIM_UPDATE_IMMEDIATE;
```

```
LptimHandle.Init.Input1Source    = LPTIM_INPUT1SOURCE_GPIO;
LptimHandle.Init.Input2Source    = LPTIM_INPUT2SOURCE_GPIO;

if (HAL_LPTIM_Init(&LptimHandle) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

36.3.3 定时器的底层配置（GPIO、时钟、中断等）

HAL 库有个自己的底层初始化回调函数，比如调用函数 HAL_LPTIM_Init 就会调用 HAL_LPTIM_MspInit，此函数是弱定义的。

```
__weak void HAL_LPTIM_MspInit(LPTIM_HandleTypeDef *hlptim)
{
  /* Prevent unused argument(s) compilation warning */
  UNUSED(hlptim);

  /* NOTE : This function Should not be modified, when the callback is needed,
            the HAL_LPTIM_MspInit could be implemented in the user file
   */
}
```

用户可以在其它的 C 文件重定向，并将相对的底层初始化在里面实现。对应的底层复位函数 HAL_LPTIM_MspDeInit 是在函数 HAL_LPTIM_DeInit 里面被调用的，也是弱定义的。

当然，用户也可以自己初始化，不限制必须在两个函数里面实现。

定时器外设的基本参数配置完毕后还不能使用，还需要配置 GPIO、时钟、中断等参数，比如下面配置 LPTIM1 使用 PD13 做 PWM 输出。

```
void HAL_TIM_PWM_MspInit(TIM_HandleTypeDef *htim)
{
  GPIO_InitTypeDef     GPIO_InitStruct;

  /* 使能 LPTIM 时钟 */
  __HAL_RCC_LPTIM1_CLK_ENABLE();

  /* 使能 GPIO 时钟 Enable GPIO PORT */
  __HAL_RCC_GPIOD_CLK_ENABLE();

  /* 配置 PD13 */
  GPIO_InitStruct.Pin = GPIO_PIN_13;
  GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_MEDIUM;
  GPIO_InitStruct.Alternate = GPIO_AF1_LPTIM1;
  HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
}
```

总结下来就是以下几点：

- 配置 LPTIM 时钟。
- 配置 LPTIM 所用到引脚和对应的 GPIO 时钟。
- 如果用到定时器中断，还需要通过 NVIC 配置中断。

关于这个底层配置有以下几点要着重说明下：

◆ 定时器所使用引脚的复用模式选择已经被 HAL 库定义好，放在了 stm32h7xx_hal_gpio_ex.h 文件里面。比如 TIM1 有一个复用，

```
#define GPIO_AF1_LPTIM1         ((uint8_t)0x01)  /* LPTIM1 Alternate Function mapping */
```

但是却有 4 个输出通道，每个通道都有几个支持的输出引脚：

```
LPTIM1_OUT  PG13
LPTIM1_OUT  PD13
```

具体使用哪个，配置对应引脚的复用即可：

	Port	AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7	AF8	AF9	AF10	AF11	AF12	AF13	AF14	AF15
		SYS	TIM1/2/16/17/LPTIM1/HRTIM1	SAI1/TIM3/4/5/12/HRTIM1	LPUART/TIM8/LPTIM2/3/4/5/HRTIM1/DFSDM	I2C1/2/3/4/USART1/TIM15/LPTIM2/DFSDM/CEC	SPI1/2/3/4/5/6/CEC	SPI2/3/SAI1/3/I2C4/UART4/DFSDM	SPI2/3/6/USART1/2/3/6/UART7/SDMMC1	SPI6/SAI2/4/UART4/5/8/LPUART/SDMMC1/SPDIFRX	SAI4/FDCAN1/2/TIM13/14/QUADSPI/FMC/SDMMC2/LCD/SPDIFRX	SAI2/4/TIM8/QUADSPI/SDMMC2/OTG1_HS/OTG2_FS/LCD	I2C4/UART7/SWPMI1/TIM1/8/DFSDM/SDMMC2/MDIOS/ETH	TIM1/8/FMC/SDMMC1/MDIOS/OTG1_FS/LCD	TIM1/DCMI/LCD/COMP	UART5/LCD	SYS
Port D	PD0	-	-	-	DFSDM_CKIN6	-	-	SAI3_SCK_A	-	UART4_RX	FDCAN1_RX	-	-	FMC_D2/FMC_DA2	-	-	EVENT-OUT
	PD1	-	-	-	DFSDM_DATIN6	-	-	SAI3_SD_A	-	UART4_TX	FDCAN1_TX	-	-	FMC_D3/FMC_DA3	-	-	EVENT-OUT
	PD2	TRACED2	-	TIM3_ETR	-	-	-	-	-	UART5_RX	-	-	-	SDMMC1_CMD	DCMI_D11	-	EVENT-OUT
	PD3	-	-	-	DFSDM_CKOUT	-	SPI2_SCK/I2S2_CK	-	USART2_CTS_NSS	-	-	-	-	FMC_CLK	DCMI_D5	LCD_G7	EVENT-OUT
	PD4	-	-	HRTIM_FLT3	-	-	-	SAI3_FS_A	USART2_RTS	-	FDCAN1_RXFD_MODE	-	-	FMC_NOE	-	-	EVENT-OUT
	PD5	-	-	HRTIM_EEV3	-	-	-	-	USART2_TX	-	FDCAN1_TXFD_MODE	-	-	FMC_NWE	-	-	EVENT-OUT
	PD6	-	-	SAI1_D1	DFSDM_CKIN4	DFSDM_DATIN1	SPI3_MOSI/I2S3_SDO	SAI1_SD_A	USART2_RX	SAI4_SD_A	FDCAN2_RXFD_MODE	SAI4_D1	SDMMC2_CK	FMC_NWAIT	DCMI_D10	LCD_B2	EVENT-OUT
	PD7	-	-	-	DFSDM_DATIN4	-	SPI1_MOSI/I2S1_SDO	DFSDM_CKIN1	USART2_CK	-	SPDIFRX_IN0	-	SDMMC2_CMD	FMC_NE1	-	-	EVENT-OUT
	PD8	-	-	-	DFSDM_CKIN3	-	-	SAI3_SCK_B	USART3_TX	-	SPDIFRX_IN1	-	-	FMC_D13/FMC_DA13	-	-	EVENT-OUT
	PD9	-	-	-	DFSDM_DATIN3	-	-	SAI3_SD_B	USART3_RX	-	FDCAN2_RXFD_MODE	-	-	FMC_D14/FMC_DA14	-	-	EVENT-OUT
	PD10	-	-	-	DFSDM_CKOUT	-	-	SAI3_FS_B	USART3_CK	-	FDCAN2_TXFD_MODE	-	-	FMC_D15/FMC_DA15	-	LCD_B3	EVENT-OUT
	PD11	-	-	-	LPTIM2_IN2	I2C4_SMBA	-	-	USART3_CTS_NSS	-	QUADSPI_BK1_IO0	SAI2_SD_A	-	FMC_A16	-	-	EVENT-OUT
	PD12	-	LPTIM1_IN1	TIM4_CH1	LPTIM2_IN1	I2C4_SCL	-	-	USART3_RTS	-	QUADSPI_BK1_IO1	SAI2_FS_A	-	FMC_A17	-	-	EVENT-OUT
	PD13	-	LPTIM1_OUT	TIM4_CH2	-	I2C4_SDA	-	-		-	QUADSPI_BK1_IO3	SAI2_SCK_A	-	FMC_A18	-	-	EVENT-OUT
	PD14	-	-	TIM4_CH3	-	-	-	SAI3_MCLK_B	-	UART8_CTS	-	-	-	FMC_D0/FMC_DA0	-	-	EVENT-OUT
	PD15	-	-	TIM4_CH4	-	-	-	SAI3_MCLK_A	-	UART8_RTS	-	-	-	FMC_D1/FMC_DA1	-	-	EVENT-OUT

36.3.4定时器的状态标志清除问题

下面我们介绍__HAL_LPTIM_GET_FLAG 函数。这个函数用来检查定时器标志位是否被设置。

```
/**
  * @brief  Check whether the specified LPTIM flag is set or not.
  * @param  __HANDLE__: LPTIM handle
  * @param  __FLAG__  : LPTIM flag to check
  *           This parameter can be a value of:
  *           @arg LPTIM_FLAG_DOWN    : Counter direction change up Flag.
  *           @arg LPTIM_FLAG_UP      : Counter direction change down to up Flag.
  *           @arg LPTIM_FLAG_ARROK   : Autoreload register update OK Flag.
  *           @arg LPTIM_FLAG_CMPOK   : Compare register update OK Flag.
  *           @arg LPTIM_FLAG_EXTTRIG : External trigger edge event Flag.
  *           @arg LPTIM_FLAG_ARRM    : Autoreload match Flag.
```

```
  *           @arg LPTIM_FLAG_CMPM    : Compare match Flag.
  * @retval The state of the specified flag (SET or RESET).
  */
#define __HAL_LPTIM_GET_FLAG(__HANDLE__, __FLAG__)  (((__HANDLE__)->Instance->ISR &(__FLAG__)) == (__FLAG__))
```

当前做的应用程序，这几个中断标志暂时都还没有被用到。

与标志获取函数__HAL_TIM_GET_FLAG 对应的清除函数是__HAL_LPTIM_CLEAR_FLAG:

```
/**
  * @brief  Clear the specified LPTIM flag.
  * @param  __HANDLE__: LPTIM handle.
  * @param  __FLAG__  : LPTIM flag to clear.
  *           This parameter can be a value of:
  *           @arg LPTIM_FLAG_DOWN    : Counter direction change up Flag.
  *           @arg LPTIM_FLAG_UP      : Counter direction change down to up Flag.
  *           @arg LPTIM_FLAG_ARROK   : Autoreload register update OK Flag.
  *           @arg LPTIM_FLAG_CMPOK   : Compare register update OK Flag.
  *           @arg LPTIM_FLAG_EXTTRIG : External trigger edge event Flag.
  *           @arg LPTIM_FLAG_ARRM    : Autoreload match Flag.
  *           @arg LPTIM_FLAG_CMPM    : Compare match Flag.
  * @retval None.
  */
#define __HAL_LPTIM_CLEAR_FLAG(__HANDLE__, __FLAG__)        ((__HANDLE__)->Instance->ICR  = (__FLAG__))
```

清除标志函数所支持的参数跟获取函数是一 一对应的。除了这两个函数，还是定时器的中断开启和中断关闭函数，有时候也要用到。

```
/**
  * @brief  Enable the specified LPTIM interrupt.
  * @param  __HANDLE__     : LPTIM handle.
  * @param  __INTERRUPT__ : LPTIM interrupt to set.
  *           This parameter can be a value of:
  *           @arg LPTIM_IT_DOWN    : Counter direction change up Interrupt.
  *           @arg LPTIM_IT_UP      : Counter direction change down to up Interrupt.
  *           @arg LPTIM_IT_ARROK   : Autoreload register update OK Interrupt.
  *           @arg LPTIM_IT_CMPOK   : Compare register update OK Interrupt.
  *           @arg LPTIM_IT_EXTTRIG : External trigger edge event Interrupt.
  *           @arg LPTIM_IT_ARRM    : Autoreload match Interrupt.
  *           @arg LPTIM_IT_CMPM    : Compare match Interrupt.
  * @retval None.
  */
#define __HAL_LPTIM_ENABLE_IT(__HANDLE__, __INTERRUPT__)    ((__HANDLE__)->Instance->IER  |=
                                                                (__INTERRUPT__))

 /**
  * @brief  Disable the specified LPTIM interrupt.
  * @param  __HANDLE__     : LPTIM handle.
  * @param  __INTERRUPT__ : LPTIM interrupt to set.
  *           This parameter can be a value of:
  *           @arg LPTIM_IT_DOWN    : Counter direction change up Interrupt.
  *           @arg LPTIM_IT_UP      : Counter direction change down to up Interrupt.
  *           @arg LPTIM_IT_ARROK   : Autoreload register update OK Interrupt.
  *           @arg LPTIM_IT_CMPOK   : Compare register update OK Interrupt.
  *           @arg LPTIM_IT_EXTTRIG : External trigger edge event Interrupt.
  *           @arg LPTIM_IT_ARRM    : Autoreload match Interrupt.
  *           @arg LPTIM_IT_CMPM    : Compare match Interrupt.
  * @retval None.
  */
#define __HAL_LPTIM_DISABLE_IT(__HANDLE__, __INTERRUPT__)   ((__HANDLE__)->Instance->IER  &=
```

注意：操作定时器的寄存器不限制必须要用 HAL 库提供的 API，比如要操作寄存器 CR，直接调用 LPTIM1->CR 操作即可。

36.3.5 定时器初始化流程总结

使用方法由 HAL 库提供：

◆ 第 1 步：通过函数 HAL_LPTIM_Init()做初始化，主要配置的内容如下：

- 支持 LPTIM1 – LPTIM5
- Clock 计数时钟
  （1） Source，可以选择 ULPTIM input (IN1)时钟输入，或者内部时钟，如 APB, LSE, LSI ,MSI 等。
  （2） Prescaler，设置分频。
- UltraLowPowerClock 超低功耗时钟
  只有在 Clock 计数时钟源选择了 ULPTIM input，此参数才有意义。
  （1） Polarity，计数单元有效的边沿极性。
  （2） SampleTime，配置时钟干扰滤波器的时钟。
- Trigger 计数器的触发
  （1） Source，可以选择硬件触发或者软件触发。
  （2） ActiveEdge，仅用于硬件触发，用来设置触发边沿（上升沿、下降沿或者双沿）。
  （3） SampleTime，采样时间，用于配置触发干扰滤波器的时钟。
- OutputPolarity 输出极性配置。
  UpdateMode 更新模式，用于配置是否立即更新自动重装寄存器和比较寄存器，可以选择立即更新，或者当前周期结束后更新。
- Input1Source 用于配置 Input1 的输入源，可以选择 GPIO，比较器输出或者 SAI FSA/FSB。
- Input2Source 用于配置 Input2 的输入源，可以选择 GPIO 和比较器。
  注意，此参数仅用于编码器模式，也就是说仅支持 LPTIM1 和 LPTIM2 的例化。

◆ 第 2 步：低功耗定时器的底层配置是通过函数 HAL_LPTIM_MspInit()实现：

- 使用函数__HAL_RCC_LPTIMx_CLK_ENABLE()使能定时器时钟。
- 使用函数__HAL_RCC_GPIOx_CLK_ENABLE 使能 GPIO 时钟。
- 使用函数 HAL_GPIO_Init 配置 GPIO。

- 如果调用函数 HAL_LPTIM_PWM_Start_IT()使能了中断，需要调用 HAL_NVIC_SetPriority()设置中断优先级，调用函数 HAL_NVIC_EnableIRQ()使能中断，在中断服务程序里面调用 HAL_LPTIM_IRQHandler。

◆ 第 3 步：低功耗定时器支持的 6 种工作模式：

- PWM 模式
  启动此模式可调用 HAL_LPTIM_PWM_Start（）或 HAL_LPTIM_PWM_Start_IT（）用于中断方式。
- 单脉冲模式
  启动此模式可调用 HAL_LPTIM_OnePulse_Start（）或 HAL_LPTIM_OnePulse_Start_IT（）用于中断方式。
- 单次模式
  在此模式下，当满足匹配条件时，输出可以切换高低电平（如果输出极性配置为高，则从低电平切至高电平，反之亦然）。启动此模式可调用 HAL_LPTIM_SetOnce_Start（）或 HAL_LPTIM_SetOnce_Start_IT（）用于中断方式。
- 编码器模式
  启动此模式可调用 HAL_LPTIM_Encoder_Start（）或 HAL_LPTIM_Encoder_Start_IT（）用于中断方式。
- 超时模式
  有效的边沿触发输入可复位定时器。第一个触发事件将启动计时器，任何连续触发事件将重置计数器并重新开始。启动此模式可调用 HAL_LPTIM_TimeOut_Start（）或 HAL_LPTIM_TimeOut_Start_IT（）用于中断方式。
- 计数器模式
  计数器可用于计算来自 Input1 的外部事件或用于计算内部时钟周期。启动此模式可调用 HAL_LPTIM_Counter_Start（）或 HAL_LPTIM_Counter_Start_IT（）用于中断方式。

◆ 第 4 步：停止任何模式：

用户可以通过调用相应的 API 来停止任何模式： HAL_LPTIM_Xxx_Stop 或 HAL_LPTIM_Xxx_Stop_IT（如果此模式已经在中断方式下启动）。

低功耗定时器常用的功能，通过上面这几步即可实现。

36.4 源文件 stm32h7xx_hal_lptim.c

此文件涉及到的函数比较多，这里把我们几个常用的函数做个说明：

- HAL_LPTIM_Init
- HAL_LPTIM_PWM_Start
- HAL_LPTIM_TimeOut_Start_IT
- HAL_LPTIM_TimeOut_Stop_IT

36.4.1 函数 HAL_LPTIM_Init

函数原型：

```
HAL_StatusTypeDef HAL_LPTIM_Init(LPTIM_HandleTypeDef *hlptim)
{
  uint32_t tmpcfgr = 0;

  /* 检测是否是有效句柄 */
  if(hlptim == NULL)
  {
    return HAL_ERROR;
  }

  /* 省略 */

  if(hlptim->State == HAL_LPTIM_STATE_RESET)
  {
    /* 默认取消锁 */
    hlptim->Lock = HAL_UNLOCKED;

    /* 初始化底层硬件 : GPIO, CLOCK, NVIC */
    HAL_LPTIM_MspInit(hlptim);
  }

  /* 更改 LPTIM 状态 */
  hlptim->State = HAL_LPTIM_STATE_BUSY;

  /* 获取 LPTIMx CFGR 数值 */
  tmpcfgr = hlptim->Instance->CFGR;

  if ((hlptim->Init.Clock.Source) ==  LPTIM_CLOCKSOURCE_ULPTIM)
  {
    tmpcfgr &= (uint32_t)(~(LPTIM_CFGR_CKPOL | LPTIM_CFGR_CKFLT));
  }
  if ((hlptim->Init.Trigger.Source) !=  LPTIM_TRIGSOURCE_SOFTWARE)
  {
    tmpcfgr &= (uint32_t)(~ (LPTIM_CFGR_TRGFLT | LPTIM_CFGR_TRIGSEL));
  }

  /* 清除 CKSEL, PRESC, TRIGEN, TRGFLT, WAVPOL, PRELOAD & COUNTMODE 位 */
  tmpcfgr &= (uint32_t)(~(LPTIM_CFGR_CKSEL | LPTIM_CFGR_TRIGEN | LPTIM_CFGR_PRELOAD |
                          LPTIM_CFGR_WAVPOL | LPTIM_CFGR_PRESC | LPTIM_CFGR_COUNTMODE ));

  /* 设置初始化参数 */
  tmpcfgr |= (hlptim->Init.Clock.Source    |
              hlptim->Init.Clock.Prescaler |
              hlptim->Init.OutputPolarity  |
```

```
              hlptim->Init.UpdateMode       |
              hlptim->Init.CounterSource);

  if ((hlptim->Init.Clock.Source) ==  LPTIM_CLOCKSOURCE_ULPTIM)
  {
    tmpcfgr |=  (hlptim->Init.UltraLowPowerClock.Polarity |
                 hlptim->Init.UltraLowPowerClock.SampleTime);
  }

  if ((hlptim->Init.Trigger.Source) !=  LPTIM_TRIGSOURCE_SOFTWARE)
  {
    /* Enable External trigger and set the trigger source */
    tmpcfgr |= (hlptim->Init.Trigger.Source      |
                hlptim->Init.Trigger.ActiveEdge |
                hlptim->Init.Trigger.SampleTime);
  }

  /* 写入到配置寄存器 LPTIMx CFGR */
  hlptim->Instance->CFGR = tmpcfgr;

 /* 配置 LPTIM input 时钟源 */
  if((hlptim->Instance == LPTIM1) || (hlptim->Instance == LPTIM2))
  {
    assert_param(IS_LPTIM_INPUT1_SOURCE(hlptim->Instance,hlptim->Init.Input1Source));
    assert_param(IS_LPTIM_INPUT2_SOURCE(hlptim->Instance,hlptim->Init.Input2Source));

    /* 配置 LPTIM1/2 Input1 和 Input2 的时钟源 */
    hlptim->Instance->CFGR2 = (hlptim->Init.Input1Source | hlptim->Init.Input2Source);
  }
  else
  {
    if(hlptim->Instance == LPTIM3)
    {
       assert_param(IS_LPTIM_INPUT1_SOURCE(hlptim->Instance,hlptim->Init.Input1Source));

      /* 注，H7 库 V1.3.0 版本这里是个 bug，LPTIM3 应该配置 CFGR3 寄存器 */
      hlptim->Instance->CFGR2 = hlptim->Init.Input1Source;
    }
  }
  /* 更改 LPTIM 状态 */
  hlptim->State = HAL_LPTIM_STATE_READY;

  /* 返回 HAL_OK */
  return HAL_OK;
}
```

函数描述：

此函数用于初始化低功耗定时器的基本参数。

函数参数：

- 第 1 个参数是 LPTIM_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。结构体变量成员的详细介绍看本章 3.2 小节。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 函数 HAL_LPTIM_MspInit 用于初始化定时器的底层时钟、引脚等功能，需要用户自己在此函数里面实现具体的功能。由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
2. 如果形参 hlptim 的结构体成员 gState 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 LPTIM_HandleTypeDef　LptimHandle。

   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_LPTIM_STATE_RESET = 0x00U。

   解决办法有四

   方法 1：用户自己初始定时器和涉及到的 GPIO 等。

   方法 2：定义 LPTIM_HandleTypeDef　LptimHandle 为全局变量。

   方法 3：定义为局部变量要赋初始值 LPTIM_HandleTypeDef　LptimHandle = {0}。

   方法 4：下面的方法

```
if(HAL_LPTIM_DeInit(&LptimHandle)!= HAL_OK)
{
    Error_Handler();
}
if(HAL_LPTIM_Init(&LptimHandle)!= HAL_OK)
{
    Error_Handler();
}
```

使用举例：

```
LPTIM_HandleTypeDef     LptimHandle = {0};

LptimHandle.Instance = LPTIM1;

/* 对应寄存器 CKSEL，选择内部时钟源 */
LptimHandle.Init.Clock.Source    = LPTIM_CLOCKSOURCE_APBCLOCK_LPOSC;
LptimHandle.Init.Clock.Prescaler = LPTIM_PRESCALER_DIV1;         /* 设置 LPTIM 时钟分频 */
LptimHandle.Init.CounterSource   = LPTIM_COUNTERSOURCE_INTERNAL;/* LPTIM 计数器对内部时钟源计数 */
LptimHandle.Init.Trigger.Source  = LPTIM_TRIGSOURCE_SOFTWARE;    /* 软件触发 */
/* 计数器计数到比较寄存器和 ARR 自动重载寄存器之间数值，输出高电平 */
LptimHandle.Init.OutputPolarity  = LPTIM_OUTPUTPOLARITY_HIGH;
/* 比较寄存器和 ARR 自动重载寄存器选择更改后立即更新 */
LptimHandle.Init.UpdateMode      = LPTIM_UPDATE_IMMEDIATE;
LptimHandle.Init.Input1Source    = LPTIM_INPUT1SOURCE_GPIO;
LptimHandle.Init.Input2Source    = LPTIM_INPUT2SOURCE_GPIO;

if (HAL_LPTIM_Init(&LptimHandle) != HAL_OK)
{
     Error_Handler(__FILE__, __LINE__);
}
```

36.4.2 函数 HAL_LPTIM_PWM_Start

函数原型：

```
HAL_StatusTypeDef HAL_LPTIM_PWM_Start(LPTIM_HandleTypeDef *hlptim, uint32_t Period, uint32_t Pulse)
{
  /* 检查参数 */
  assert_param(IS_LPTIM_INSTANCE(hlptim->Instance));
  assert_param(IS_LPTIM_PERIOD(Period));
  assert_param(IS_LPTIM_PULSE(Pulse));

  /* 设置 LPTIM 的状态 */
  hlptim->State= HAL_LPTIM_STATE_BUSY;

  /* 复位 PWM 模式的 WAVE 位 */
  hlptim->Instance->CFGR &= ~LPTIM_CFGR_WAVE;

  /* 使能 LPTIM */
  __HAL_LPTIM_ENABLE(hlptim);

  /* 设置自动重载寄存器 ARR 数值 */
  __HAL_LPTIM_AUTORELOAD_SET(hlptim, Period);

  /* 设置比较寄存器数值，用于 PWM 占空比配置 */
  __HAL_LPTIM_COMPARE_SET(hlptim, Pulse);

  /* 定时器以连续模式运行 */
  __HAL_LPTIM_START_CONTINUOUS(hlptim);

  /* 更改定时器状态 */
  hlptim->State= HAL_LPTIM_STATE_READY;

  /* 返回 HAL_OK */
  return HAL_OK;
}
```

函数描述：

调用函数 HAL_LPTIM_Init 配置了基础功能后，就可以调用此函数启动定时器 PWM 输出了。

函数参数：

- 第 1 个参数是 LPTIM_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是低功耗定时器的周期配置，范围 0 – 0xFFFF。
- 第 3 个参数是低功耗定时器 PWM 的脉宽配置，范围 0 -0xFFFF。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 这里重点说下配置了第 2 个参数后，低功耗定时器的 PWM 输出频率是多少。比如 LPTIM 使用的是 LSE 时钟，频率 32768Hz，第 2 个参数 Period = 9。那么

   PWM 频率 = LSE / （Period + 1） = 32768 / （9 + 1） = 327Hz。

   当第 3 个参数 Pluse = 5

   PWM 占空比 = 1 – （Pluse + 1）/（Period + 1） = 1 – 5/10 = 50%

使用举例：

```
LptimHandle.Instance = LPTIM1;
```

```
/* 对应寄存器 CKSEL，选择内部时钟源 */
LptimHandle.Init.Clock.Source    = LPTIM_CLOCKSOURCE_APBCLOCK_LPOSC;
LptimHandle.Init.Clock.Prescaler = LPTIM_PRESCALER_DIV1;        /* 设置 LPTIM 时钟分频 */
LptimHandle.Init.CounterSource   = LPTIM_COUNTERSOURCE_INTERNAL;/* LPTIM 计数器对内部时钟源计数 */
LptimHandle.Init.Trigger.Source  = LPTIM_TRIGSOURCE_SOFTWARE;   /* 软件触发 */
/* 计数器计数到比较寄存器和 ARR 自动重载寄存器之间数值，输出高电平 */
LptimHandle.Init.OutputPolarity  = LPTIM_OUTPUTPOLARITY_HIGH;
/* 比较寄存器和 ARR 自动重载寄存器选择更改后立即更新 */
LptimHandle.Init.UpdateMode      = LPTIM_UPDATE_IMMEDIATE;
LptimHandle.Init.Input1Source    = LPTIM_INPUT1SOURCE_GPIO;
LptimHandle.Init.Input2Source    = LPTIM_INPUT2SOURCE_GPIO;

if (HAL_LPTIM_Init(&LptimHandle) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}

    /*
       ARR 是自动重装寄存器，对应函数 HAL_LPTIM_PWM_Start 的第 2 个参数
       Compare 是比较寄存器，对应函数 HAL_LPTIM_PWM_Start 的第 3 个参数

       ----------------------
       LSE = 32768Hz
       分频设置为 LPTIM_PRESCALER_DIV1，即未分频
       ARR 自动重载寄存器 = 31
       那么 PWM 频率 = LSE / （ARR + 1） = 32768Hz / (31 + 1) = 1024Hz

       占空比 = 1 - (Comprare + 1)/ (ARR + 1)
              = 1 - (15 + 1)/(31 + 1)
              = 50%

       占空比这里为什么要 1 减操作，而不是直接的(Comprare + 1)/ (ARR + 1)，这是因为前面的配置中
       计数器计数到比较寄存器和 ARR 自动重载寄存器之间数值，输出高电平。
    */
if (HAL_LPTIM_PWM_Start(&LptimHandle, 31, 15) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

36.4.3 函数 HAL_LPTIM_TimeOut_Start_IT

函数原型：

```
HAL_StatusTypeDef HAL_LPTIM_TimeOut_Start_IT(LPTIM_HandleTypeDef *hlptim, uint32_t Period, uint32_t Timeout)
{
  /* 检查参数 */
  assert_param(IS_LPTIM_INSTANCE(hlptim->Instance));
  assert_param(IS_LPTIM_PERIOD(Period));
  assert_param(IS_LPTIM_PULSE(Timeout));

  /* 设置 LPTIM 的状态 */
  hlptim->State= HAL_LPTIM_STATE_BUSY;

  /* 使能超时位 TIMOUT */
  hlptim->Instance->CFGR |= LPTIM_CFGR_TIMOUT;
```

```
  /* 使能比匹配中断 */
  __HAL_LPTIM_ENABLE_IT(hlptim, LPTIM_IT_CMPM);

  /* 使能低功耗定时器 */
  __HAL_LPTIM_ENABLE(hlptim);

  /* 设置自动重载寄存器 ARR 的数值 */
  __HAL_LPTIM_AUTORELOAD_SET(hlptim, Period);

  /* 设置比较寄存器数值 */
  __HAL_LPTIM_COMPARE_SET(hlptim, Timeout);

  /* 低功耗定时器以连续模式运行 */
  __HAL_LPTIM_START_CONTINUOUS(hlptim);

  /* 设置定时器的状态 */
  hlptim->State= HAL_LPTIM_STATE_READY;

  /* 返回 HAL_OK */
  return HAL_OK;
}
```

函数描述：

调用函数 HAL_LPTIM_Init 配置了基础功能后，就可以调用此函数启动定时器的超时功能。

函数参数：

- 第 1 个参数是 LPTIM_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是低功耗定时器的周期配置，范围 0 – 0xFFFF。
- 第 3 个参数是低功耗定时器的超时时间配置，范围 0 -0xFFFF。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。
- **注意事项：**

1. 超时配置用不到第 2 个参数。
2. 此函数开启的是比较匹配中断，所以实际的超时时间由 Compare 寄存器决定，即第 3 个参数。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_StartLPTIM
*    功能说明: 启动 LPTIM
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_StartLPTIM(void)
{
    /*
       ARR 是自动重装寄存器，对应函数 HAL_LPTIM_TimeOut_Start_IT 的第 2 个参数
       Compare 是比较寄存器，对应函数 HAL_LPTIM_TimeOut_Start_IT 的第 3 个参数

       ---------------------
       LSE = 32768Hz
       分频设置为 LPTIM_PRESCALER_DIV8，即 8 分频（函数 bsp_InitLPTIM 里面做的初始化配置）
```

```
        ARR 自动重载寄存器 = 32768
        实际测试发现溢出中断与 ARR 寄存器无关，全部由第 3 个参数，Compare 寄存器决定

        LPTIM 的计数器计数 1 次的时间是 1 / (32768 / 8) = 8 /32768。
        第三个参数配置的是 32767，那么计数到 32767 就是 (32767 + 1)*（8 /32768） = 8 秒，计算的时候要加 1。
    */
    if (HAL_LPTIM_TimeOut_Start_IT(&LptimHandle, 0, 32767) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}

/*
*********************************************************************************************************
*    函 数 名: LPTIM1_IRQHandler
*    功能说明: LPTIM1 中断服务程序
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void LPTIM1_IRQHandler(void)
{
    if((LPTIM1->ISR & LPTIM_FLAG_CMPM) != RESET)
    {
        /* 清除比较匹配中断 */
        LPTIM1->ICR = LPTIM_FLAG_CMPM;

        /* 关闭溢出中断 */
        HAL_LPTIM_TimeOut_Stop_IT(&LptimHandle);

        bsp_LedToggle(4);
    }
}
```

36.4.4函数 HAL_LPTIM_TimeOut_Stop_IT

函数原型：

```
HAL_StatusTypeDef HAL_LPTIM_TimeOut_Stop_IT(LPTIM_HandleTypeDef *hlptim)
{
  /* 检查参数 */
  assert_param(IS_LPTIM_INSTANCE(hlptim->Instance));

  /* 设置 LPTIM 状态 */
  hlptim->State= HAL_LPTIM_STATE_BUSY;

  /* 禁止低功耗定时器 */
  __HAL_LPTIM_DISABLE(hlptim);

  /* 关闭 TIMOUT 位 */
  hlptim->Instance->CFGR &= ~LPTIM_CFGR_TIMOUT;

  /* 关闭比较匹配中断 */
  __HAL_LPTIM_DISABLE_IT(hlptim, LPTIM_IT_CMPM);

  /* 设置 LPTIM 状态 */
```

```
  hlptim->State= HAL_LPTIM_STATE_READY;

  /* 返回 HAL_OK */
  return HAL_OK;
}
```

函数描述：

此函数用于关闭低功耗定时器的超时模式。

函数参数：

◆ 第 1 个参数是 TIM_LPHandleTypeDef 类型结构体指针变量。

注意事项：

1. 推荐在低功耗定时器的中断服务程序里面调用此函数进行关闭，这样可以及时的关闭。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_StartLPTIM
*    功能说明: 启动 LPTIM
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_StartLPTIM(void)
{
    /*
       ARR 是自动重装寄存器，对应函数 HAL_LPTIM_TimeOut_Start_IT 的第 2 个参数
       Compare 是比较寄存器，对应函数 HAL_LPTIM_TimeOut_Start_IT 的第 3 个参数

       ----------------------
       LSE = 32768Hz
       分频设置为 LPTIM_PRESCALER_DIV8，即 8 分频（函数 bsp_InitLPTIM 里面做的初始化配置）
       ARR 自动重载寄存器 = 32768
       实际测试发现溢出中断与 ARR 寄存器无关，全部由第 3 个参数，Compare 寄存器决定

       LPTIM 的计数器计数 1 次的时间是 1 / (32768 / 8) = 8 /32768。
       第三个参数配置的是 32767，那么计数到 32767 就是 (32767 + 1)*（8 /32768） = 8 秒，计算的时候要加 1。
    */
    if (HAL_LPTIM_TimeOut_Start_IT(&LptimHandle, 0, 32767) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}

/*
*********************************************************************************************************
*    函 数 名: LPTIM1_IRQHandler
*    功能说明: LPTIM1 中断服务程序
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void LPTIM1_IRQHandler(void)
{
    if((LPTIM1->ISR & LPTIM_FLAG_CMPM) != RESET)
    {
```

```
        /* 清除比较匹配中断 */
        LPTIM1->ICR = LPTIM_FLAG_CMPM;

        /* 关闭溢出中断 */
        HAL_LPTIM_TimeOut_Stop_IT(&LptimHandle);

        bsp_LedToggle(4);
    }
}
```

36.5 总结

本章节就为大家讲解这么多，低功耗定时器在低功耗场合比较有用，如果有低功耗方面的项目需求，可以考虑使用这个定时器。

第37章 STM32H7 的 LPTIM 低功耗定时器应用之 PWM

本章教程为大家讲解低功耗定时器的 PWM 输出。使用 LPTIM 的好处是系统处于睡眠、停机状态依然可以正常工作（除了待机模式）。实际项目中对于功耗有要求的场合，可以使用这种方式，可以一定程度上较低功耗。

37.1 初学者重要提示
37.2 低功耗定时器 PWM 驱动设计
37.3 低功耗定时器板级支持包（bsp_lptim_pwm.c）
37.4 低功耗定时器驱动移植和使用
37.5 实验例程设计框架
37.6 实验例程说明（MDK）
37.7 实验例程说明（IAR）
37.8 总结

37.1 初学者重要提示

- 学习本章节前，务必优先学习第 36 章，HAL 库的几个常用 API 均作了讲解和举例。
- 使用 LPTIM 的好处是系统处于睡眠、停机状态依然可以正常工作（除了待机模式）。停机状态可以正常工作的关键是 LSE，LSI 时钟不会被关闭，同时也可以选择使用外部时钟源。
- STM32H7 从停机模式唤醒后要重新配置系统时钟，这点跟 F1，F4 系列一样。

37.2 低功耗定时器 PWM 驱动设计

低功耗定时器 LPTIM1 – LPTIM5 均支持 PWM 输出。

37.2.1 低功耗定时器 PWM 输出支持的引脚

STM32H7 的低功耗定时器 LPTIM1 - LPTIM5 可以输出到 GPIO 的 TIM 通道整理：

```
LPTIM1_IN1    PD12   PG12
LPTIM1_IN2    PH2    PE1
LPTIM1_OUT    PG13
LPTIM1_OUT    PD13
```

```
LPTIM1_ETR   PG14  PE0

LPTIM2_IN1   PB10  PD12
LPTIM2_IN2   PD11
LPTIM2_OUT   PB13
LPTIM2_ETR   PB11 PE0

LPTIM3_OUT   PA1
LPTIM4_OUT   PA2
LPTIM5_OUT   PA3
```

37.2.2 低功耗定时器时钟选择

由前面的第 36 章节，我们知道 LPTIM1 的时钟可以由 LSE，LSI，APB 或者外部输入时钟提供。使用 LSE，LSI 或者外部输入的好处是停机状态下，LPTIM1 也可以正常工作。

- V7 开发板使用的 LSE 晶振是 32768Hz。
- STM32H743 的 LSI 频率约 32KHz。
- LPTIM1 – LPTIM5 的频率都是 100MHz。

```
System Clock source       = PLL (HSE)
SYSCLK(Hz)                = 400000000 (CPU Clock)
HCLK(Hz)                  = 200000000 (AXI and AHBs Clock)
AHB Prescaler             = 2
D1 APB3 Prescaler         = 2 (APB3 Clock  100MHz)
D2 APB1 Prescaler         = 2 (APB1 Clock  100MHz)
D2 APB2 Prescaler         = 2 (APB2 Clock  100MHz)
D3 APB4 Prescaler         = 2 (APB4 Clock  100MHz)

因为 APB1 prescaler != 1, 所以 APB1 上的 TIMxCLK = APB1 x 2 = 200MHz; 不含这个总线下的 LPTIM1
因为 APB2 prescaler != 1, 所以 APB2 上的 TIMxCLK = APB2 x 2 = 200MHz;

APB4 上面的 TIMxCLK 没有分频，所以就是 100MHz;

APB1 定时器有 TIM2, TIM3 ,TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14，LPTIM1
APB2 定时器有 TIM1, TIM8 , TIM15, TIM16，TIM17

APB4 定时器有 LPTIM2，LPTIM3，LPTIM4，LPTIM5
```

下面为大家讲解下使用 LSE，LSI 或者 APB 时钟的配置方法。

- 选择 LSE 的配置如下:

```
#define LPTIM_CLOCK_SOURCE_LSE     /* LSE 时钟 32768Hz */

RCC_PeriphCLKInitTypeDef   RCC_PeriphCLKInitStruct = {0};
RCC_OscInitTypeDef RCC_OscInitStruct = {0};

RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_LSE;
RCC_OscInitStruct.LSEState = RCC_LSE_ON;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;

if (HAL_RCC_OscConfig(&RCC_OscInitStruct)!= HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

```
RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPTIM1;
RCC_PeriphCLKInitStruct.Lptim1ClockSelection = RCC_LPTIM1CLKSOURCE_LSE;
HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
```

特别注意程序中置红的地方，这几个地方很容易配置错。配置后 LPTIM1 就会将 LSE 作为系统时钟。

◆ 选择 LSI 的配置如下：

```
//#define LPTIM_CLOCK_SOURCE_LSI    /* LSI 时钟约 32KHz */
RCC_PeriphCLKInitTypeDef   RCC_PeriphCLKInitStruct = {0};
RCC_OscInitTypeDef RCC_OscInitStruct = {0};

RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_LSI;
RCC_OscInitStruct.LSIState = RCC_LSI_ON;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;

if (HAL_RCC_OscConfig(&RCC_OscInitStruct)!= HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}

RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPTIM1;
RCC_PeriphCLKInitStruct.Lptim1ClockSelection = RCC_LPTIM1CLKSOURCE_LSI;
HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
```

使用 LSI 作为 LPTIM1 的系统是要注意两点：

1、LSI 的实现有一定的误差，具体范围在数据手册有给出，由于不支持温补，温度对其也是有影响的。

Symbol	Parameter	Conditions	Min	Typ	Max	Unit
f_{LSI} (1)	LSI frequency	V_{DD} = 3.3 V, T_J = 25 °C (after calibration)	31.4	32	32.6	kHz
		T_J = –40 to 105 °C, V_{DD} = 1.62 to 3.6 V	29.76	-	33.60	
$t_{su(LSI)}$ (2)	LSI oscillator startup time	-	-	80	130	µs
$t_{stab(LSI)}$ (2)	LSI oscillator stabilization time (5% of final value)	-	-	120	170	
$I_{DD(LSI)}$ (2)	LSI oscillator power consumption	-	-	130	280	nA

2、特别注意程序中置红的地方，这几个地方很容易跟 LSE 搞混淆（复制粘贴的时候容易搞错）。

◆ 选择 APB 时钟的配置如下：

```
RCC_PeriphCLKInitTypeDef   RCC_PeriphCLKInitStruct = {0};

RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPTIM1;
RCC_PeriphCLKInitStruct.Lptim1ClockSelection = RCC_LPTIM1CLKSOURCE_D2PCLK1;
HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
```

使用 APB 作为 LPTIM 系统时钟注意以下两点：

1、 LPTIM1 – LPTIM5 的最高主频都是 100MHz。

2、 注意参数 RCC_LPTIM1CLKSOURCE_D2PCLK1。

LPTIM1 使用的 RCC_LPTIM1CLKSOURCE_D2PCLK1。

LPTIM2 使用的 RCC_LPTIM2CLKSOURCE_D3PCLK1。

LPTIM3-LPTIM5 使用的 RCC_LPTIM345CLKSOURCE_D3PCLK1。

37.2.3 低功耗定时器的 PWM 配置

下面通过低功耗定时器实现一个频率为 1024Hz，占空比 50%，使用 LSE 做系统时钟的配置。PWM 输出引脚采用 PD13。

```
/* 选择 LPTIM 的时钟源 */
#define LPTIM_CLOCK_SOURCE_LSE     /* LSE 时钟 32768Hz */
//#define LPTIM_CLOCK_SOURCE_LSI   /* LSI 时钟约 32KHz */
//#define LPTIM_CLOCK_SOURCE_PCLK  /* PCLK 时钟 100MHz */
/*
*********************************************************************************************************
*    函 数 名: bsp_InitTIMOutPWM
*    功能说明: LPTIM1 时钟默认选择的 LSE，而 PWM 输出使用的 PD13 引脚，频率 1024Hz。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitTIMOutPWM(void)
{
    LPTIM_HandleTypeDef        LptimHandle = {0};
    RCC_PeriphCLKInitTypeDef   RCC_PeriphCLKInitStruct = {0};
    GPIO_InitTypeDef           GPIO_InitStruct = {0};

    /* ## - 1 - 使能 LPTIM 时钟和 GPIO 时钟 ########################################## */
    __HAL_RCC_LPTIM1_CLK_ENABLE();

    __HAL_RCC_GPIOD_CLK_ENABLE();

    /* ## - 2 - 配置 PD13 做 PWM 输出 ############################################### */
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Pull = GPIO_PULLUP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    GPIO_InitStruct.Alternate = GPIO_AF1_LPTIM1;
    GPIO_InitStruct.Pin = GPIO_PIN_13;
    HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);

    /* ## - 3 - 配置 LPTIM 时钟，可以选择 LSE，LSI 或者 PCLK ######################### */
#if defined (LPTIM_CLOCK_SOURCE_LSE)
    {
        RCC_OscInitTypeDef RCC_OscInitStruct = {0};

        RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_LSE;
        RCC_OscInitStruct.LSEState = RCC_LSE_ON;
        RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;

        if (HAL_RCC_OscConfig(&RCC_OscInitStruct)!= HAL_OK)
        {
            Error_Handler(__FILE__, __LINE__);
        }

        RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPTIM1;
        RCC_PeriphCLKInitStruct.Lptim1ClockSelection = RCC_LPTIM1CLKSOURCE_LSE;
        HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
```

```
    }
#elif defined (LPTIM_CLOCK_SOURCE_LSI)
    {
        RCC_OscInitTypeDef RCC_OscInitStruct = {0};

        RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_LSI;
        RCC_OscInitStruct.LSIState = RCC_LSI_ON;
        RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;

        if (HAL_RCC_OscConfig(&RCC_OscInitStruct)!= HAL_OK)
        {
            Error_Handler(__FILE__, __LINE__);
        }

        RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPTIM1;
        RCC_PeriphCLKInitStruct.Lptim1ClockSelection = RCC_LPTIM1CLKSOURCE_LSI;
        HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
    }
#elif defined (LPTIM_CLOCK_SOURCE_PCLK)
    RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPTIM1;
    RCC_PeriphCLKInitStruct.Lptim1ClockSelection = RCC_LPTIM1CLKSOURCE_D2PCLK1;
    HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
#else
    #error Please select the LPTIM Clock source inside the bsp_lptim_pwm.c file
#endif

    /* ## - 4 - 配置 LPTIM ############################################################ */
    LptimHandle.Instance = LPTIM1;
    /* 对应寄存器 CKSEL，选择内部时钟源 */
    LptimHandle.Init.Clock.Source    = LPTIM_CLOCKSOURCE_APBCLOCK_LPOSC;
    /* 设置 LPTIM 时钟分频 */
    LptimHandle.Init.Clock.Prescaler = LPTIM_PRESCALER_DIV1;
    /* LPTIM 计数器对内部时钟源计数 */
    LptimHandle.Init.CounterSource   = LPTIM_COUNTERSOURCE_INTERNAL
    /* 软件触发 */
    LptimHandle.Init.Trigger.Source  = LPTIM_TRIGSOURCE_SOFTWARE;
    /* 计数器计数到比较寄存器和 ARR 自动重载寄存器之间数值，输出高电平 */
    LptimHandle.Init.OutputPolarity  = LPTIM_OUTPUTPOLARITY_HIGH;
    /* 比较寄存器和 ARR 自动重载寄存器选择更改后立即更新 */
    LptimHandle.Init.UpdateMode      = LPTIM_UPDATE_IMMEDIATE;
    /* 外部输入 1，本配置未使用 */
    LptimHandle.Init.Input1Source    = LPTIM_INPUT1SOURCE_GPIO;
    /* 外部输入 2，本配置未使用 */
    LptimHandle.Init.Input2Source    = LPTIM_INPUT2SOURCE_GPIO;

    if (HAL_LPTIM_Init(&LptimHandle) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }


    /* ## - 5 - 启动 LPTIM 的 PWM 模式 ############################################################ */
    /*
       ARR 是自动重装寄存器，对应函数 HAL_LPTIM_PWM_Start 的第 2 个参数
       Compare 是比较寄存器，对应函数 HAL_LPTIM_PWM_Start 的第 3 个参数

       ---------------------
       LSE = 32768Hz
```

```
107.          分频设置为 LPTIM_PRESCALER_DIV1，即未分频
108.          ARR 自动重载寄存器 = 31
109.          那么 PWM 频率 = LSE / （ARR + 1） = 32768Hz / (31 + 1) = 1024Hz
110.
111.          占空比 = 1 - (Comprare + 1)/ (ARR + 1)
112.                 = 1 - (15 + 1)/(31 + 1)
113.                 = 50%
114.
115.          占空比这里为什么要 1 减操作，而不是直接的(Comprare + 1)/ (ARR + 1)，这是因为前面的配置中
116.          计数器计数到比较寄存器和 ARR 自动重载寄存器之间数值，输出高电平。
117.      */
118.      if (HAL_LPTIM_PWM_Start(&LptimHandle, 31, 15) != HAL_OK)
119.      {
120.          Error_Handler(__FILE__, __LINE__);
121.      }
122. }
```

这里把几个关键的地方阐释下：

- 第 2 行，LPTIM1 的系统时钟选项 LSE，频率 32768Hz。
- 第 15 – 17 行，HAL 库的这个结构体变量要初始化为 0，此问题在第 36 章的 4.1 小节有专门说明。
- 第 76 – 97 行，第 36 章的 3.2 小节对这些参数成员有详细描述。
- 第 118 行，启动 PWM 输出，特别注意 PWM 的频率和占空比的计算，在前面的注释中已经讲解的比较清楚。

37.2.4低功耗定时器待机模式下正常运行

这里先补充三个知识点。

- LPTIM 的好处是系统处于睡眠，停机状态依然可以正常工作，但停机模式不能再正常工作。
- 对于睡眠模式，任何受 NVIC 控制的中断都可以唤醒休眠模式。进入睡眠模式调用函数 HAL_PWR_EnterSLEEPMode 即可。
- 在系统停止模式下,1.2V 供电域中的所有时钟都停止,PLL,HSI 和 HSE RC 振荡器被禁用。内部 SRAM 和寄存器内容保留。而 LSE 和 LSI 是可以正常工作的，所以 LPTIM 系统时钟使用 LSE 或者 LSI 依然可以在停机模式下工作。

  进入停机模式调用函数 HAL_PWR_EnterSTOPMode 即可。

对于停机模式，本章节配套的例子是通过 GPIO 的 EXTI Event 唤醒。配套如下：

```
/*
*********************************************************************************************************
*    函 数 名: PwrExitStopMode
*    功能说明: 起初按键 K3 的 GPIO 配置为输入模式用于按键后,再次配置 EXTI Event 模式后,按键功能依然可以正常用。
*              配置为 EXTI Event 是因为可以唤醒停机模式。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void PwrExitStopMode(void)
```

```
{
    GPIO_InitTypeDef               GPIO_InitStruct = {0};

    GPIO_InitStruct.Pin = GPIO_PIN_4;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    GPIO_InitStruct.Mode = GPIO_MODE_EVT_FALLING; /* 配置为下降沿触发 Event */
    HAL_GPIO_Init(GPIOH, &GPIO_InitStruct);
}
```

37.3 低功耗定时器板级支持包（bsp_lptim_pwm.c)

低功耗定时器驱动文件 bsp_lptim_pwm.c 供用户调用的只有如下一个函数：

- bsp_InitLPTIMOutPWM

下面将这个函数的使用为大家做个说明。

37.3.1 函数 bsp_InitLPTIMOutPWM

函数原型：

```
void bsp_InitLPTIMOutPWM(void)
```

函数描述：

使用低功耗定时器 LPTIM1 实现一个频率为 1024Hz，占空比 50%，使用 LSE 做 LPTIM1 的系统时钟。

注意事项：

1. 关于此函数的相关注意事项在本章的 37.2 小节有详细说明。

使用举例：

初始化函数在 bsp.c 文件的 bsp_Init 函数里面调用。

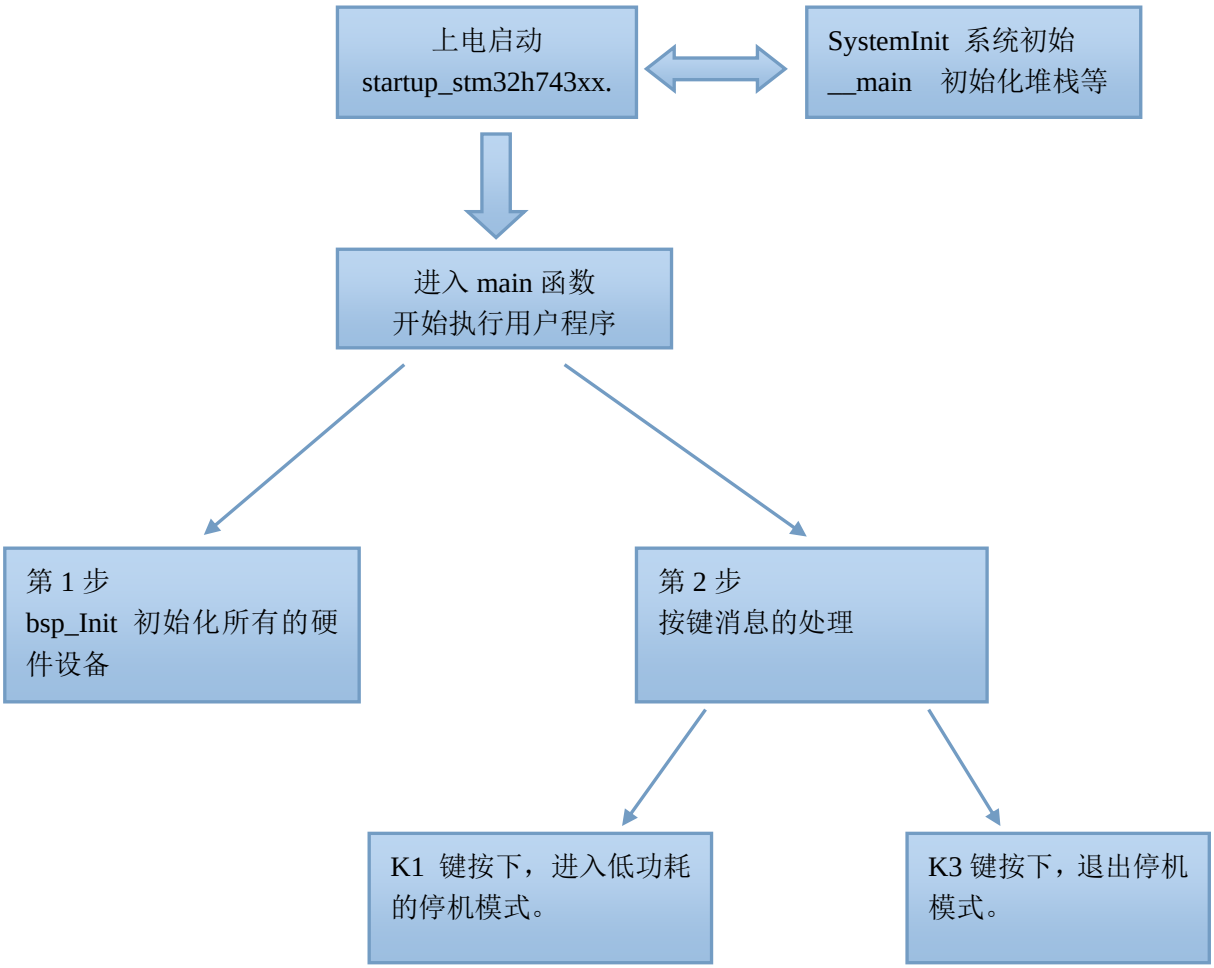
37.4 低功耗定时器驱动移植和使用

低功耗定时器的移植比较简单：

- 第 1 步：复制 bsp_lptim_pwm.c 和 bsp_lptim_pwm.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：这几个驱动文件主要用到 HAL 库的 GPIO 和 LPTIM 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 3 步，应用方法看本章节配套例子即可。如果用到按键唤醒的话，看 main.c 文件里面的函数 PwrExitStopMode 即可。

37.5 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED 和串口。
  - 第 2 步，借助按键消息实现低功耗定时器的效果测试。

37.6 实验例程说明（MDK）

配套例子：

V7-021_低功耗定时器 PWM 输出

实验目的：

1. 学习低功耗定时器 PWM 输出。

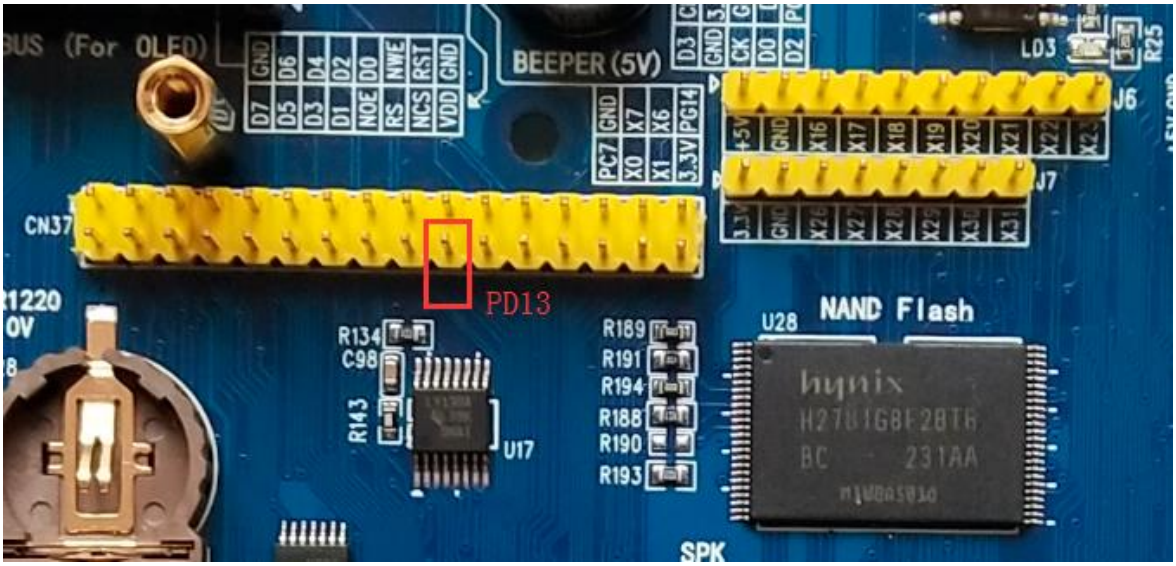
实验内容：

1. 使用 LPTIM 的好处是系统处于睡眠，停机状态依然可以正常工作（除了待机模式）。停机状态可以正常工作的关键是 LSE，LSI 时钟不会被关闭，同时也可以选择使用外部时钟源。
2. 例子默认用的 LSE 时钟供 LPTIM1 使用，大家可以通过 bsp_lptim_pwm.c 文件开头宏定义切换到 LSI 或者 PLCK。
3. PWM 输出引脚采用的 PD13，输出频谱 1024Hz，占空比 50%。

实验操作：

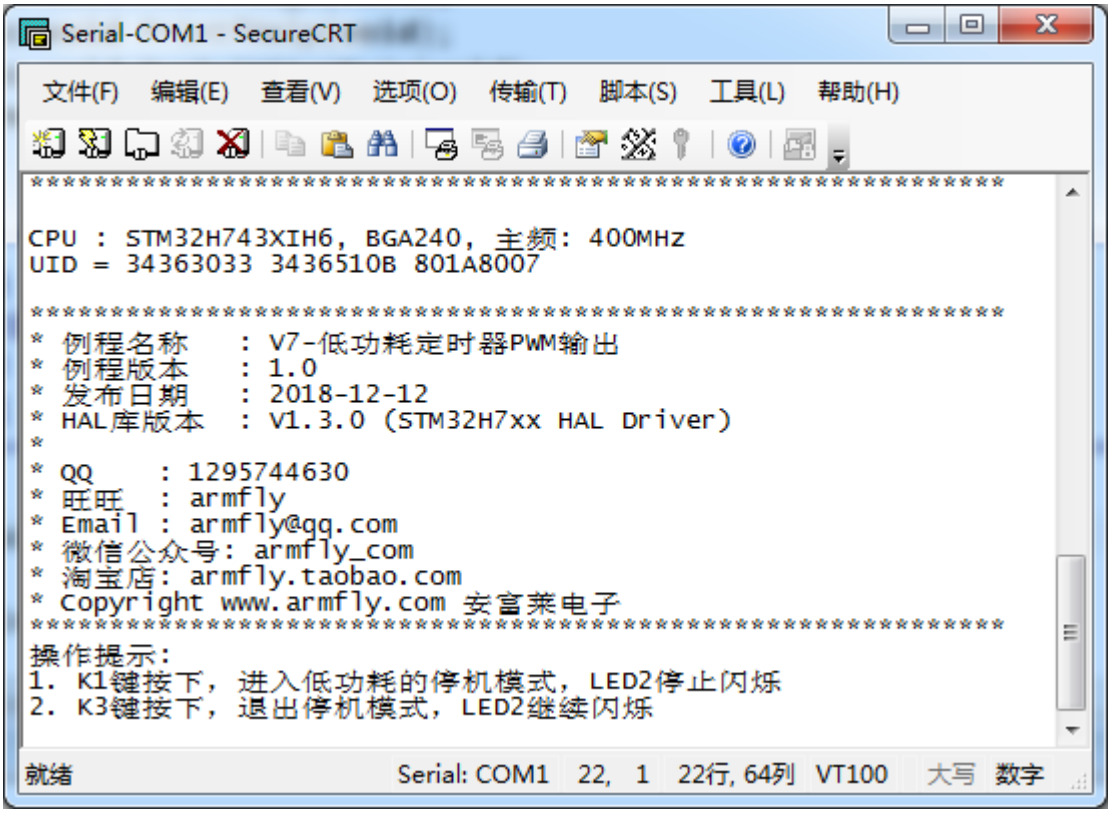
1. K1 键按下，进入低功耗的停机模式，LED2 停止闪烁。
2. K3 键按下，退出停机模式，LED2 继续闪烁。

PD13 的位置：



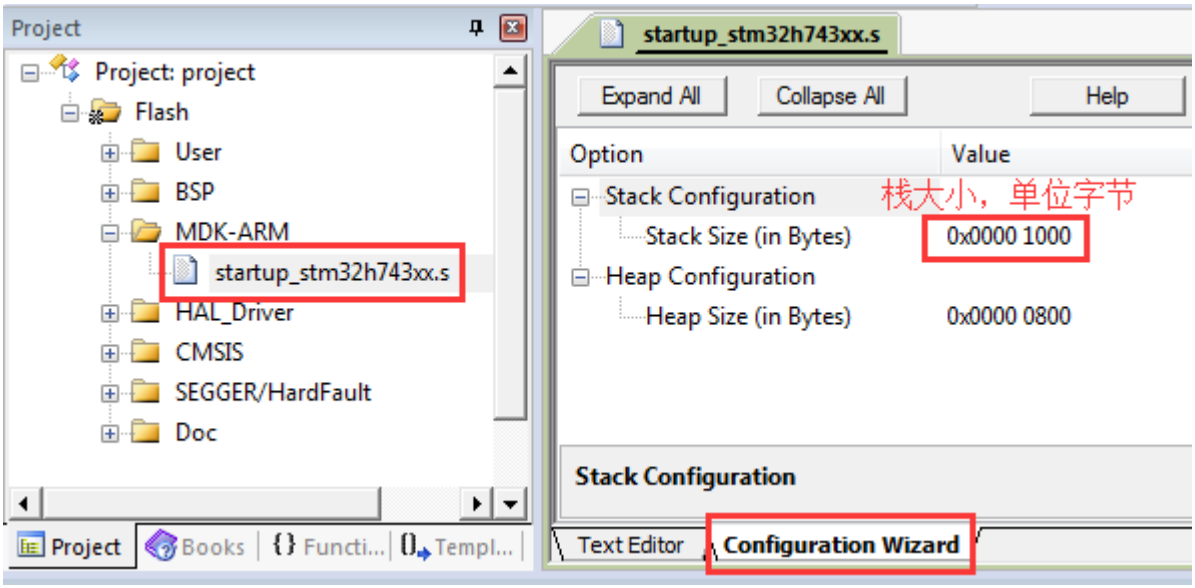
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

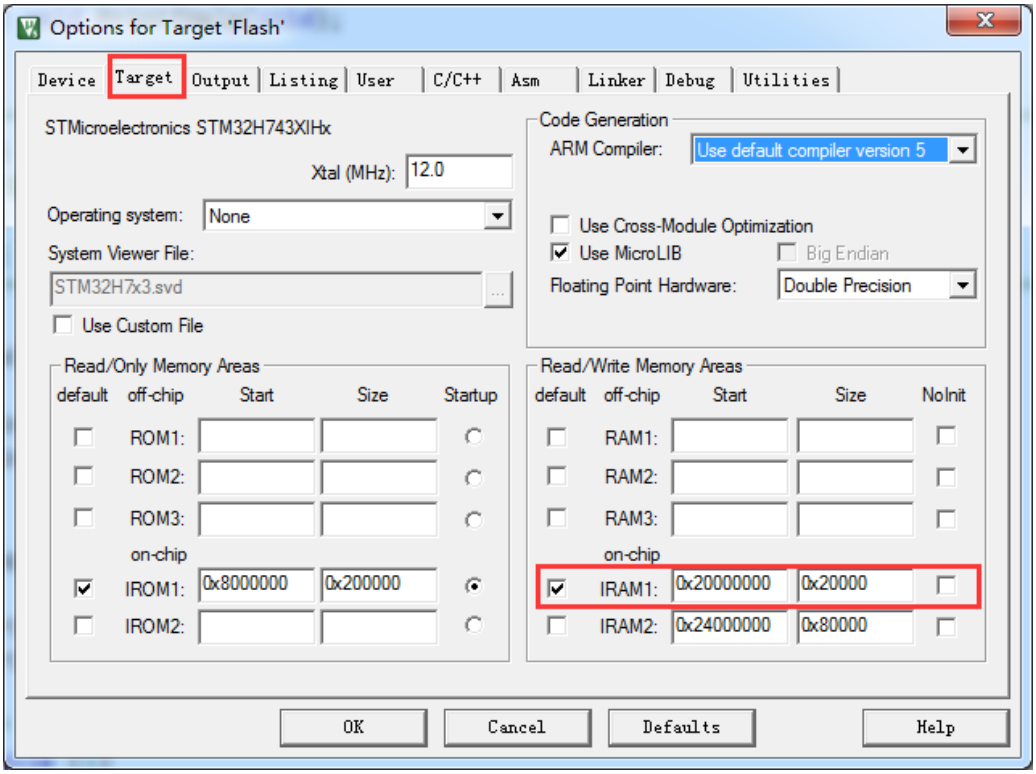


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
```

```
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **主功能：**

主程序实现如下操作：

- K1 键按下，进入低功耗的停机模式，LED2 停止闪烁。
- K3 键按下，退出停机模式，LED2 继续闪烁。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


    bsp_Init();           /* 硬件初始化 */
    PwrExitStopMode(); /* 配置 K3 按键用于唤醒停机模式 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */
```

```
    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，进入停机模式 */
                    printf("--进入停机模式\r\n");
                    HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);
                    SystemClock_Config();   /* 特别注意，退出后要重新配置系统时钟 */
                    break;

                case KEY_DOWN_K3:           /* K3 键按下，唤醒停机模式 */
                    printf("--退出停机模式\r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

通过 GPIO 为 EXTI Event 可以唤醒停机模式：

```
/*
*********************************************************************************************************
*    函 数 名: PwrExitStopMode
*    功能说明: 起初按键 K3 的 GPIO 配置为输入模式用于按键后，再次配置 EXTI Event 模式后，按键功能依然可以正常使
*              用。配置为 EXTI Event 是因为可以唤醒停机模式。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void PwrExitStopMode(void)
{
    GPIO_InitTypeDef              GPIO_InitStruct = {0};

    GPIO_InitStruct.Pin = GPIO_PIN_4;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    GPIO_InitStruct.Mode = GPIO_MODE_EVT_FALLING; /* 配置为下降沿触发 Event */
    HAL_GPIO_Init(GPIOH, &GPIO_InitStruct);
```

```
}
```

37.7 实验例程说明（IAR）

配套例子：

V7-021_低功耗定时器 PWM 输出

实验目的：

1. 学习低功耗定时器 PWM 输出。

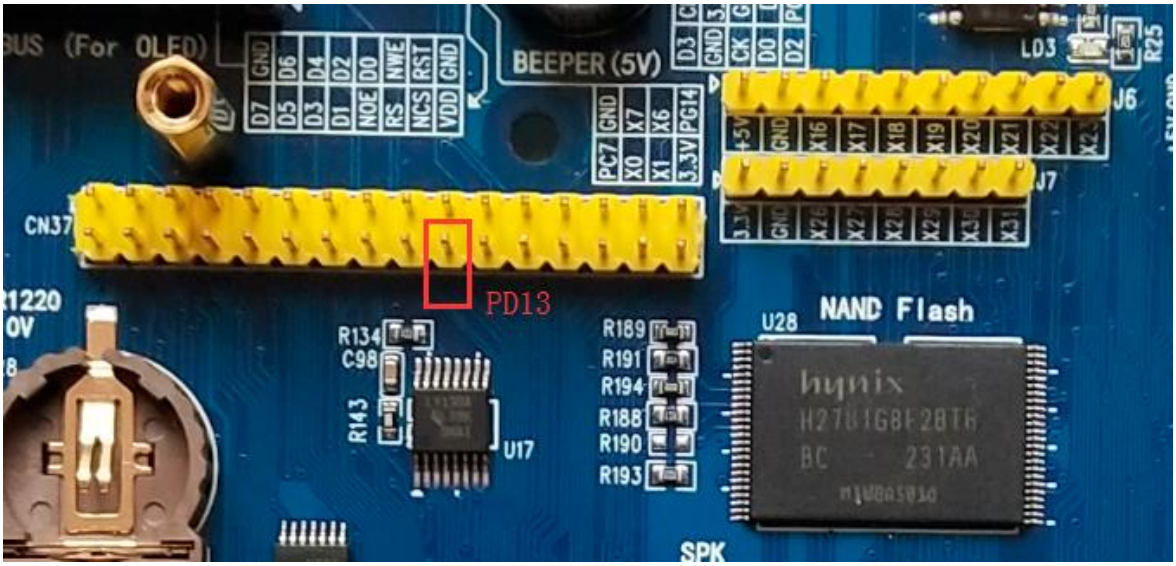
实验内容：

1. 使用 LPTIM 的好处是系统处于睡眠，停机状态依然可以正常工作（除了待机模式）。停机状态可以正常工作的关键是 LSE，LSI 时钟不会被关闭，同时也可以选择使用外部时钟源。
2. 例子默认用的 LSE 时钟供 LPTIM1 使用，大家可以通过 bsp_lptim_pwm.c 文件开头宏定义切换到 LSI 或者 PLCK。
3. PWM 输出引脚采用的 PD13，输出频谱 1024Hz，占空比 50%。

实验操作：

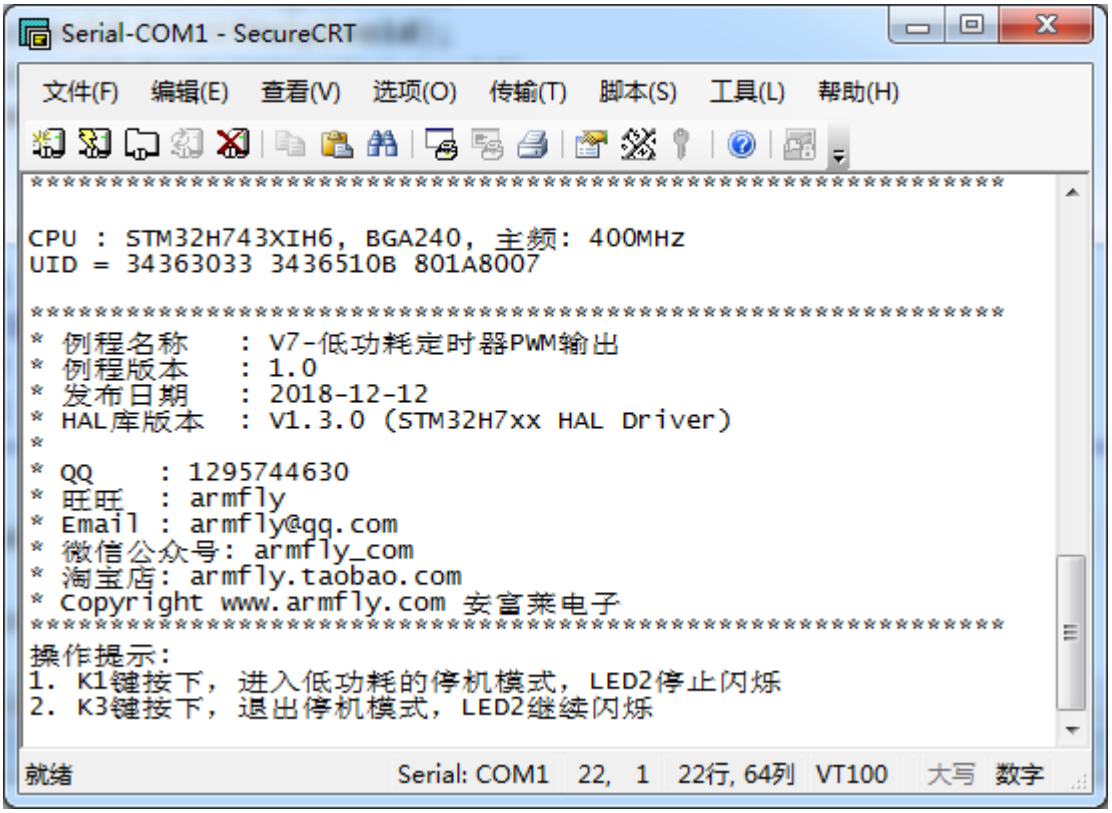
1. K1 键按下，进入低功耗的停机模式，LED2 停止闪烁。
2. K3 键按下，退出停机模式，LED2 继续闪烁。

PD13 的位置：



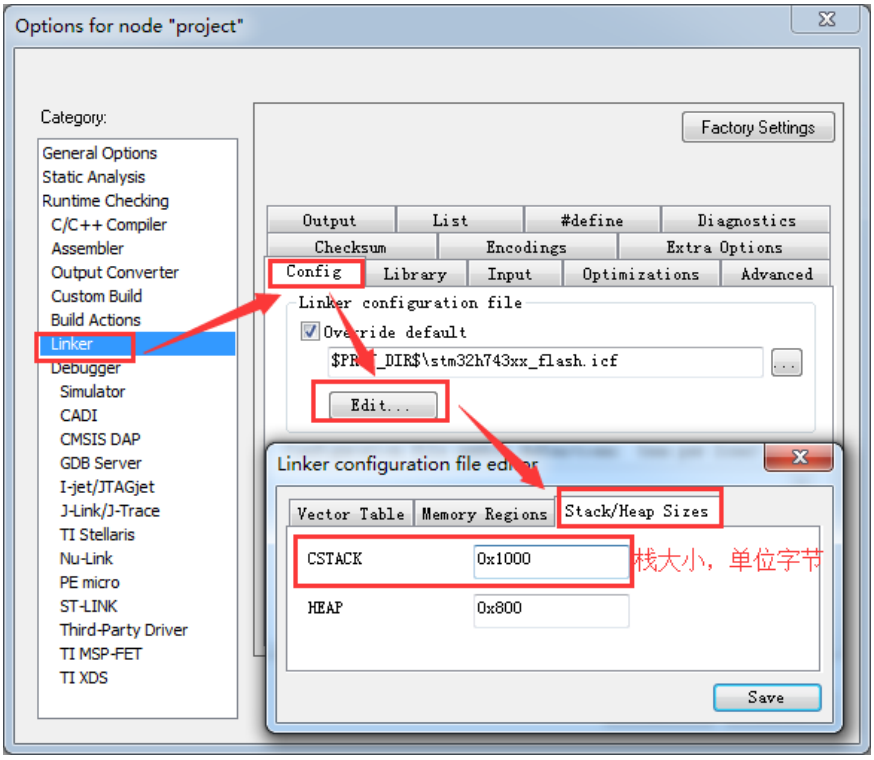
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

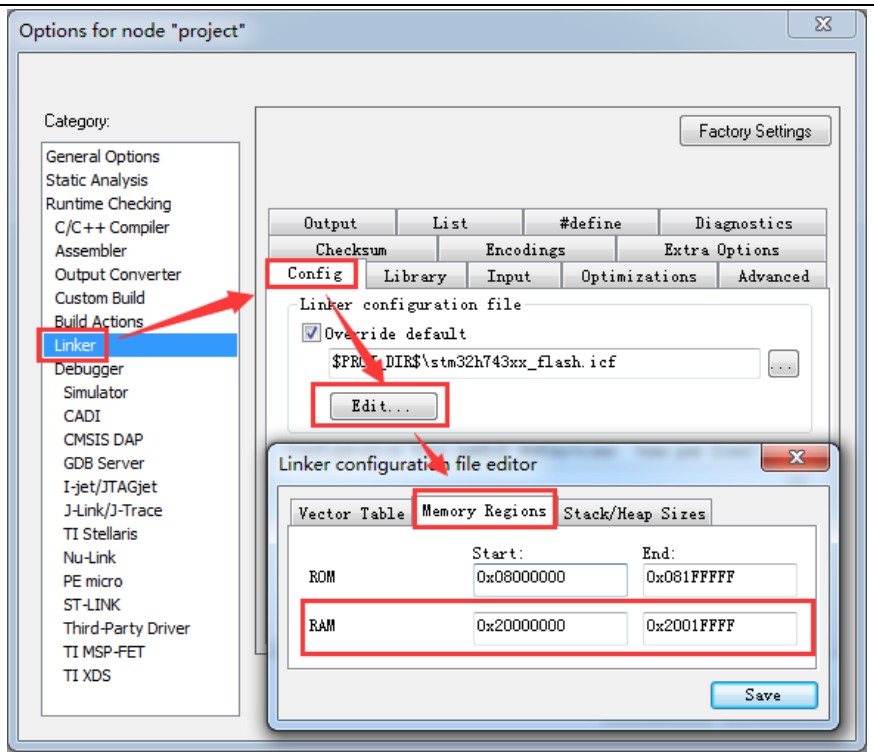


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
```

```
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- K1 键按下，进入低功耗的停机模式，LED2 停止闪烁。
- K3 键按下，退出停机模式，LED2 继续闪烁。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


    bsp_Init();           /* 硬件初始化 */
    PwrExitStopMode(); /* 配置 K3 按键用于唤醒停机模式 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */
```

```
    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，进入停机模式 */
                    printf("--进入停机模式\r\n");
                    HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);
                    SystemClock_Config();   /* 特别注意，退出后要重新配置系统时钟 */
                    break;

                case KEY_DOWN_K3:           /* K3 键按下，唤醒停机模式 */
                    printf("--退出停机模式\r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

通过 GPIO 为 EXTI Event 可以唤醒停机模式：

```
/*
*********************************************************************************************************
*    函 数 名: PwrExitStopMode
*    功能说明: 起初按键 K3 的 GPIO 配置为输入模式用于按键后，再次配置 EXTI Event 模式后，按键功能依然可以正常使
*              用。配置为 EXTI Event 是因为可以唤醒停机模式。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void PwrExitStopMode(void)
{
    GPIO_InitTypeDef               GPIO_InitStruct = {0};

    GPIO_InitStruct.Pin = GPIO_PIN_4;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    GPIO_InitStruct.Mode = GPIO_MODE_EVT_FALLING; /* 配置为下降沿触发 Event */
    HAL_GPIO_Init(GPIOH, &GPIO_InitStruct);
}
```

37.8 总结

本章节就为大家讲解这么多，低功耗定时器在低功耗场合比较有用，如果有低功耗方面的项目需求，可以考虑是这个定时器实现 PWM。

第38章 STM32H7 的 LPTIM 低功耗定时器应用之超时唤醒

本章教程为大家讲解定时器应用之超时模式的停机唤醒，实际项目中有一定的使用价值，可以方便的配置系统在停机模式运行一段时间，时间到了可以自动唤醒。

38.1 初学者重要提示
38.2 低功耗定时器超时唤醒驱动设计
38.3 低功耗定时器板级支持包（bsp_lptim_pwm.c）
38.4 低功耗定时器驱动移植和使用
38.5 实验例程设计框架
38.6 实验例程说明（MDK）
38.7 实验例程说明（IAR）
38.8 总结

38.1 初学者重要提示

- 学习本章节前，务必优先学习第 36 章，HAL 库的几个常用 API 均作了讲解和举例。
- 使用 LPTIM 的好处是系统处于睡眠、停机状态依然可以正常工作（除了待机模式）。停机状态可以正常工作的关键是 LSE，LSI 时钟不会被关闭，同时也可以选择使用外部时钟源。
- LPTIM 的任何中断都可以唤醒停机模式。
- STM32H7 从停机模式唤醒后要重新配置系统时钟，这点跟 F1，F4 系列一样。
- 测试发现 STM32H7 的 LPTIM1 的中断可以唤醒停机模式，其它几个 LPTIM2-5 无法唤醒。详情记录看此贴：http://www.armbbs.cn/forum.php?mod=viewthread&tid=91064

38.2 低功耗定时器超时唤醒驱动设计

低功耗定时器超时唤醒驱动设计中有几个要注意的事项，下面逐一为大家做个说明。

38.2.1 低功耗定时器时钟选择

由前面的第 36 章节，我们知道 LPTIM1 的时钟可以由 LSE，LSI，APB 或者外部输入时钟提供。使用

LSE，LSI 或者外部输入的好处是停机状态下，LPTIM1 也可以正常工作。

- V7 开发板使用的 LSE 晶振是 32768Hz。
- STM32H743 的 LSI 频率约 32KHz。
- LPTIM1 – LPTIM5 的频率都是 100MHz。

```
System Clock source       = PLL (HSE)
SYSCLK(Hz)                = 400000000 (CPU Clock)
HCLK(Hz)                  = 200000000 (AXI and AHBs Clock)
AHB Prescaler             = 2
D1 APB3 Prescaler         = 2 (APB3 Clock  100MHz)
D2 APB1 Prescaler         = 2 (APB1 Clock  100MHz)
D2 APB2 Prescaler         = 2 (APB2 Clock  100MHz)
D3 APB4 Prescaler         = 2 (APB4 Clock  100MHz)

因为 APB1 prescaler != 1, 所以 APB1 上的 TIMxCLK = APB1 x 2 = 200MHz; 不含这个总线下的 LPTIM1
因为 APB2 prescaler != 1, 所以 APB2 上的 TIMxCLK = APB2 x 2 = 200MHz;

APB4 上面的 TIMxCLK 没有分频，所以就是 100MHz;

APB1 定时器有 TIM2, TIM3 ,TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14，LPTIM1
APB2 定时器有 TIM1, TIM8 , TIM15, TIM16，TIM17

APB4 定时器有 LPTIM2，LPTIM3，LPTIM4，LPTIM5
```

下面为大家讲解下使用 LSE，LSI 或者 APB 时钟的配置方法。

- 选择 LSE 的配置如下：

```
#define LPTIM_CLOCK_SOURCE_LSE     /* LSE 时钟 32768Hz */

RCC_PeriphCLKInitTypeDef   RCC_PeriphCLKInitStruct = {0};
RCC_OscInitTypeDef RCC_OscInitStruct = {0};

RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_LSE;
RCC_OscInitStruct.LSEState = RCC_LSE_ON;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;

if (HAL_RCC_OscConfig(&RCC_OscInitStruct)!= HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}

RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPTIM1;
RCC_PeriphCLKInitStruct.Lptim1ClockSelection = RCC_LPTIM1CLKSOURCE_LSE;
HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
```

特别注意程序中置红的地方，这几个地方很容易配置错。配置后 LPTIM1 就会将 LSE 作为系统时钟。

- 选择 LSI 的配置如下：

```
//#define LPTIM_CLOCK_SOURCE_LSI    /* LSI 时钟约 32KHz */
RCC_PeriphCLKInitTypeDef   RCC_PeriphCLKInitStruct = {0};
RCC_OscInitTypeDef RCC_OscInitStruct = {0};

RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_LSI;
RCC_OscInitStruct.LSIState = RCC_LSI_ON;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;

if (HAL_RCC_OscConfig(&RCC_OscInitStruct)!= HAL_OK)
```

```
{
    Error_Handler(__FILE__, __LINE__);
}

RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPTIM1;
RCC_PeriphCLKInitStruct.Lptim1ClockSelection = RCC_LPTIM1CLKSOURCE_LSI;
HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
```

使用 LSI 作为 LPTIM1 的系统是要注意两点：

1、LSI 的实现有一定的误差，具体范围在数据手册有给出，由于不支持温补，温度对其也是有影响的。

Symbol	Parameter	Conditions	Min	Typ	Max	Unit
f_{LSI} (1)	LSI frequency	V_{DD} = 3.3 V, T_J = 25 °C (after calibration)	31.4	32	32.6	kHz
		T_J = –40 to 105 °C, V_{DD} = 1.62 to 3.6 V	29.76	-	33.60	
$t_{su(LSI)}$ (2)	LSI oscillator startup time	-	-	80	130	µs
$t_{stab(LSI)}$ (2)	LSI oscillator stabilization time (5% of final value)	-	-	120	170	
$I_{DD(LSI)}$ (2)	LSI oscillator power consumption	-	-	130	280	nA

2、特别注意程序中置红的地方，这几个地方很容易跟 LSE 搞混淆（复制粘贴的时候容易搞错）。

◆ 选择 APB 时钟的配置如下：

```
RCC_PeriphCLKInitTypeDef   RCC_PeriphCLKInitStruct = {0};

RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPTIM1;
RCC_PeriphCLKInitStruct.Lptim1ClockSelection = RCC_LPTIM1CLKSOURCE_D2PCLK1;
HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
```

使用 APB 作为 LPTIM 系统时钟注意以下两点：

1、 LPTIM1 – LPTIM5 的最高主频都是 100MHz。

2、 注意参数 RCC_LPTIM1CLKSOURCE_D2PCLK1。

LPTIM1 使用的 RCC_LPTIM1CLKSOURCE_D2PCLK1。

LPTIM2 使用的 RCC_LPTIM2CLKSOURCE_D3PCLK1。

LPTIM3-LPTIM5 使用的 RCC_LPTIM345CLKSOURCE_D3PCLK1。

38.2.2 低功耗定时器超时模式配置

下面使用 LSE 做低功耗定时器的系统时钟，做了 8 分频，并开启 LPTIM1 的超时中断。

```
1.  /* 选择 LPTIM 的时钟源 */
2.  #define LPTIM_CLOCK_SOURCE_LSE     /* LSE 时钟 32768Hz */
3.  //#define LPTIM_CLOCK_SOURCE_LSI   /* LSI 时钟 32768Hz */
4.  //#define LPTIM_CLOCK_SOURCE_PCLK  /* PCLK 时钟 100MHz */
5.
6.  LPTIM_HandleTypeDef     LptimHandle = {0};
7.
8.  /*
```

```
9.  *********************************************************************************************************
10. *    函 数 名: bsp_InitLPTIM
11. *    功能说明: 初始化 LPTIM
12. *    形    参: 无
13. *    返 回 值: 无
14. *********************************************************************************************************
15. */
16. void bsp_InitLPTIM(void)
17. {
18.     RCC_PeriphCLKInitTypeDef   RCC_PeriphCLKInitStruct = {0};
19. 
20. 
21.     /* ## - 1 - 使能 LPTIM 时钟和 GPIO 时钟 ######################################### */
22.     __HAL_RCC_LPTIM1_CLK_ENABLE();
23. 
24.     /* ## - 2 - 配置 LPTIM 时钟，可以选择 LSE，LSI 或者 PCLK ######################### */
25. #if defined (LPTIM_CLOCK_SOURCE_LSE)
26.     {
27.         RCC_OscInitTypeDef RCC_OscInitStruct = {0};
28. 
29.         RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_LSE;
30.         RCC_OscInitStruct.LSEState = RCC_LSE_ON;
31.         RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
32. 
33.         if (HAL_RCC_OscConfig(&RCC_OscInitStruct)!= HAL_OK)
34.         {
35.             Error_Handler(__FILE__, __LINE__);
36.         }
37. 
38.         RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPTIM1;
39.         RCC_PeriphCLKInitStruct.Lptim1ClockSelection = RCC_LPTIM1CLKSOURCE_LSE;
40.         HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
41.     }
42. #elif defined (LPTIM_CLOCK_SOURCE_LSI)
43.     {
44.         RCC_OscInitTypeDef RCC_OscInitStruct = {0};
45. 
46.         RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_LSI;
47.         RCC_OscInitStruct.LSIState = RCC_LSI_ON;
48.         RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
49. 
50.         if (HAL_RCC_OscConfig(&RCC_OscInitStruct)!= HAL_OK)
51.         {
52.             Error_Handler(__FILE__, __LINE__);
53.         }
54. 
55.         RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPTIM1;
56.         RCC_PeriphCLKInitStruct.Lptim1ClockSelection = RCC_LPTIM1CLKSOURCE_LSI;
57.         HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
58.     }
59. #elif defined (LPTIM_CLOCK_SOURCE_PCLK)
60.     RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPTIM1;
61.     RCC_PeriphCLKInitStruct.Lptim1ClockSelection = RCC_LPTIM1CLKSOURCE_LSE;
62.     HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
63. #else
64.     #error Please select the LPTIM Clock source inside the bsp_lptim_pwm.c file
65. #endif
66. 
```

```
67.       /* ## - 3 - 配置 LPTIM ############################################################ */
68.       LptimHandle.Instance = LPTIM1;
69.       /* 对应寄存器 CKSEL，选择内部时钟源 */
70.       LptimHandle.Init.Clock.Source    = LPTIM_CLOCKSOURCE_APBCLOCK_LPOSC;
71.       /* 设置 LPTIM 时钟分频 */
72.       LptimHandle.Init.Clock.Prescaler = LPTIM_PRESCALER_DIV8;
73.       /* LPTIM 计数器对内部时钟源计数 */
74.       LptimHandle.Init.CounterSource   = LPTIM_COUNTERSOURCE_INTERNAL;
75.       /* 软件触发 */
76.       LptimHandle.Init.Trigger.Source  = LPTIM_TRIGSOURCE_SOFTWARE;
77.       /* 超时模式用不到这个配置 */
78.       LptimHandle.Init.OutputPolarity  = LPTIM_OUTPUTPOLARITY_HIGH;
79.       /* 比较寄存器和 ARR 自动重载寄存器选择更改后立即更新 */
80.       LptimHandle.Init.UpdateMode      = LPTIM_UPDATE_IMMEDIATE;
81.       /* 外部输入 1，本配置未使用 */
82.       LptimHandle.Init.Input1Source    = LPTIM_INPUT1SOURCE_GPIO;
83.       /* 外部输入 2，本配置未使用 */
84.       LptimHandle.Init.Input2Source    = LPTIM_INPUT2SOURCE_GPIO;
85.
86.       if (HAL_LPTIM_Init(&LptimHandle) != HAL_OK)
87.       {
88.           Error_Handler(__FILE__, __LINE__);
89.       }
90.
91.       /* ## - 4 - 配置 LPTIM ############################################################ */
92.       /* 配置中断优先级并使能中断 */
93.       HAL_NVIC_SetPriority(LPTIM1_IRQn, 1, 0);
94.       HAL_NVIC_EnableIRQ(LPTIM1_IRQn);
95.   }
```

这里把几个关键的地方阐释下：

- 第 2 行，LPTIM1 的系统时钟选项 LSE，频率 32768Hz。
- 第 68 – 89 行，第 36 章的 3.2 小节对这些参数成员有详细描述。
- 第 76 行，低功耗定时器的超时模式使用软件触发或者外部触发均可以正常工作。
- 第 93 -94 行，配置 LPTIM1 的中断优先级并使能中断。

38.2.3 低功耗定时器超时模式启动

启动低功耗定时器：

```
1.   /*
2.   *********************************************************************************************************
3.   *    函 数 名: bsp_StartLPTIM
4.   *    功能说明: 启动 LPTIM
5.   *    形    参: 无
6.   *    返 回 值: 无
7.   *********************************************************************************************************
8.   */
9.   void bsp_StartLPTIM(void)
10.  {
11.      /*
12.         ARR 是自动重装寄存器，对应函数 HAL_LPTIM_TimeOut_Start_IT 的第 2 个参数
13.         Compare 是比较寄存器，对应函数 HAL_LPTIM_TimeOut_Start_IT 的第 3 个参数
14.
```

```
15.          ----------------------
16.          LSE = 32768Hz
17.          分频设置为 LPTIM_PRESCALER_DIV8，即 8 分频（函数 bsp_InitLPTIM 里面做的初始化配置）
18.          ARR 自动重载寄存器 = 32768
19.          实际测试发现溢出中断与 ARR 寄存器无关，全部由第 3 个参数，Compare 寄存器决定
20.
21.          LPTIM 的计数器计数 1 次的时间是 1 / (32768 / 8) = 8 /32768。
22.          第三个参数配置的是 32767，那么计数到 32767 就是 (32767 + 1)*（8 /32768） = 8 秒，计算的时候要加 1。
23.      */
24.      if (HAL_LPTIM_TimeOut_Start_IT(&LptimHandle, 0, 32767) != HAL_OK)
25.      {
26.          Error_Handler(__FILE__, __LINE__);
27.      }
28.  }
```

这里把几个关键的地方阐释下：

- 程序里面的注释已经比较详细，特别注意函数 HAL_LPTIM_TimeOut_Start_IT 的第 2 个参数在超时模式没有任何作用。主要是通过第 3 个参数配置超时时间。
- 函数 HAL_LPTIM_TimeOut_Start_IT 开启的是比较匹配中断,所以实际的超时时间由 Compare 寄存器决定。

中断事件	说明
比较匹配	当计数器寄存器 (LPTIM_CNT) 的内容与比较寄存器 (LPTIM_CMP) 的内容匹配时，生成中断标志。
自动重载匹配	当计数器寄存器 (LPTIM_CNT) 的内容与自动重新加载寄存器 (LPTIM_ARR) 的内容匹配时，生成中断标志。
外部触发事件	当检测到外部触发事件时，会生成中断标志。
自动重载寄存器写操作完成	当对 LPTIM_ARR 寄存器的写操作完成时，生成中断标志。
比较寄存器写操作完成	当对 LPTIM_CMP 寄存器的写操作完成时，生成中断标志。
方向更改	用于编码器模式。嵌入两个中断标志以发出方向更改的信号： – UP 标志发出递增计数方向更改的信号 – DOWN 标志发出递减计数方向更改的信号

38.2.4低功耗定时器中断处理

低功耗定时器中断的实现如下：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: LPTIM1_IRQHandler
4.  *    功能说明: LPTIM1 中断服务程序
5.  *    形    参: 无
6.  *    返 回 值: 无
7.  *********************************************************************************************************
8.  */
9.  void LPTIM1_IRQHandler(void)
10. {
11.     if((LPTIM1->ISR & LPTIM_FLAG_CMPM) != RESET)
```

```
12.     {
13.         /* 清除比较匹配中断 */
14.         LPTIM1->ICR = LPTIM_FLAG_CMPM;
15.
16.         /* 关闭溢出中断 */
17.         HAL_LPTIM_TimeOut_Stop_IT(&LptimHandle);
18.
19.         bsp_LedToggle(4);
20.     }
21. }
```

这里把几个关键的地方阐释下：

- 程序中没有使用 HAL 整理的中断处理函数 HAL_LPTIM_IRQHandler，而是直接使用寄存器判断的方式，效果高些。
- 第 14 行，函数 HAL_LPTIM_TimeOut_Start_IT 开启的是比较匹配中断，所以这里要清除对应的标识。
- 第 17 行，关闭超时中断，下次使用时再开启。

38.2.5 低功耗定时器超时唤醒注意事项

这里再强调下低功耗定时器唤醒的三个注意事项。

- LPTIM 的任何中断都可以唤醒停机模式。
- STM32H7 从停机模式唤醒后要重新配置系统时钟，这点跟 F1，F4 系列一样。
- 测试发现 STM32H7 的 LPTIM1 的中断可以唤醒停机模式，其它几个 LPTIM2-5 无法唤醒。详情记录看此贴：http://www.armbbs.cn/forum.php?mod=viewthread&tid=91064

38.3 低功耗定时器板级支持包（bsp_lptim_pwm.c）

低功耗定时器驱动文件 bsp_lptim_pwm.c 供用户调用的两个函数：

- bsp_InitLPTIM
- bsp_StartLPTIM

下面将这两个函数的使用为大家做个说明。

38.3.1 函数 bsp_InitLPTIM

函数原型：

```
void bsp_InitLPTIM(void)
```

函数描述：

此函数使用 LSE 做低功耗定时器的系统时钟，做了 8 分频，并开启 LPTIM1 的超时中断。

注意事项：

1. 关于此函数的相关注意事项在本章的 38.2.2 小节有详细说明。

使用举例：

初始化函数在 bsp.c 文件的 bsp_Init 函数里面调用。

38.3.2 函数 bsp_StartLPTIM

函数原型：

```
void bsp_StartLPTIM(void)
```

函数描述：

此函数通过调用函数 HAL_LPTIM_TimeOut_Start_IT 来启动低功耗定时器的超时模式，并开启了相应中断。

注意事项：

1. 关于此函数的相关注意事项和解读在本章的 38.2.3 小节有详细说明。

使用举例：

调用此函数前优先调用初始化函数 bsp_InitLPTIM 即可。

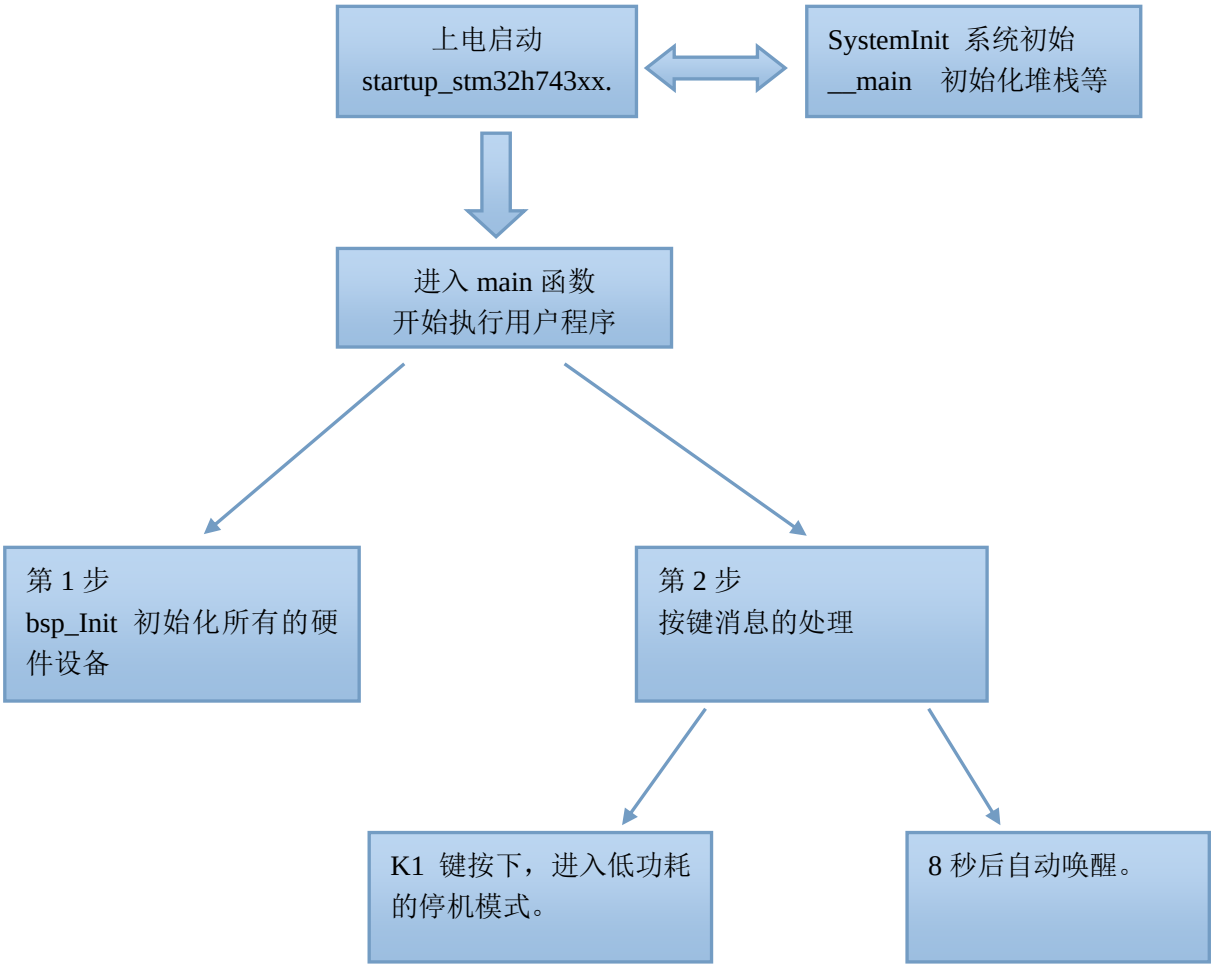
38.4 低功耗定时器驱动移植和使用

低功耗定时器的移植比较简单：

- 第 1 步：复制 bsp_lptim_pwm.c 和 bsp_lptim_pwm.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：这几个驱动文件主要用到 HAL 库的 GPIO 和 LPTIM 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 3 步，应用方法看本章节配套例子即可。

38.5 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED 和串口
  - 第 2 步，借助按键消息实现低功耗定时器的效果测试。

38.6 实验例程说明（MDK）

配套例子：

V7-022_低功耗定时器超时模式的停机唤醒

实验目的：

1. 习低功耗定时器超时模式的停机唤醒。

实验内容：

1. 使用 LPTIM 的好处是系统处于睡眠，停机状态依然可以正常工作（除了待机模式）。停机状态可以正常工作的关键是 LSE，LSI 时钟不会被关闭，同时也可以选择使用外部时钟源。
2. 例子默认用的 LSE 时钟供 LPTIM1 使用，大家可以通过 bsp_lptim_pwm.c 文件开头宏定义切换到 LSI 或者 PLCK。

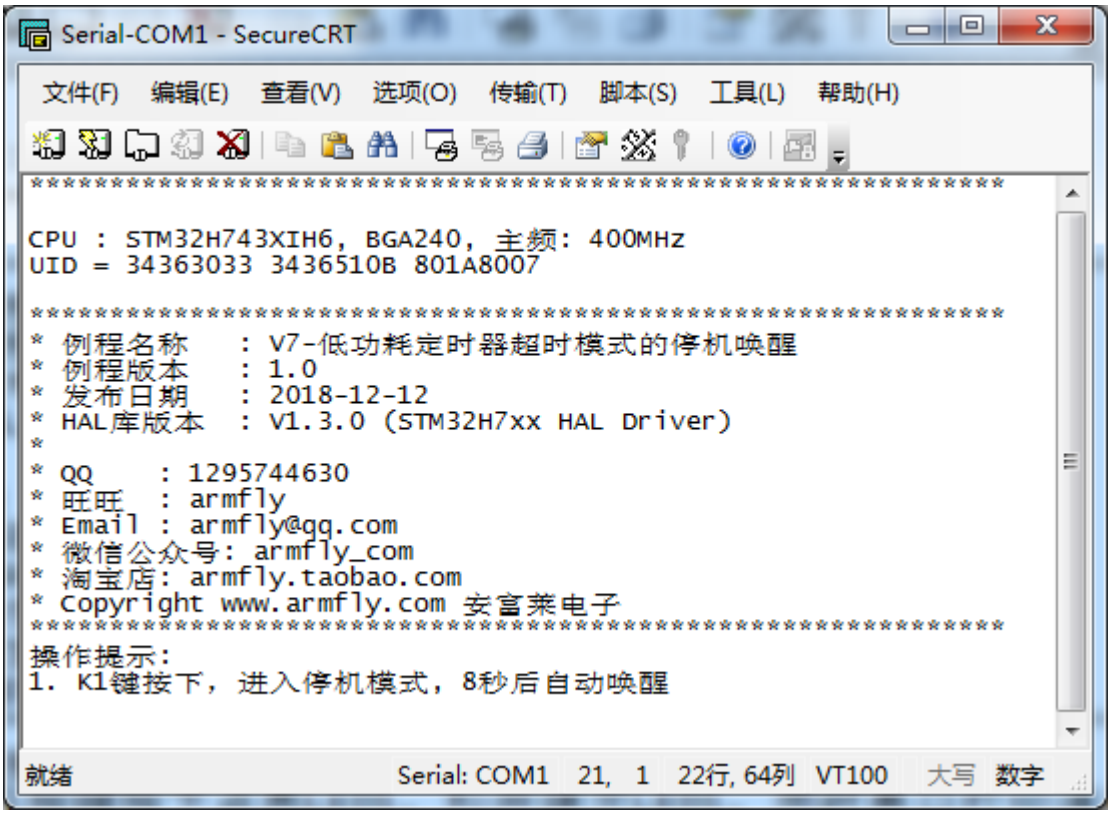
3. 本例子利用 LPTIM 的超时功能，配置 8 秒后唤醒停机模式。
4. 板子正常运行时 LED2 闪烁，进入停机模式后，LED2 停止闪烁。
5. 进入 LPTIM 的超时中断后，LED4 翻转，方便查看是否正常执行。

实验操作：

1. K1 键按下，进入停机模式，8 秒后自动唤醒。

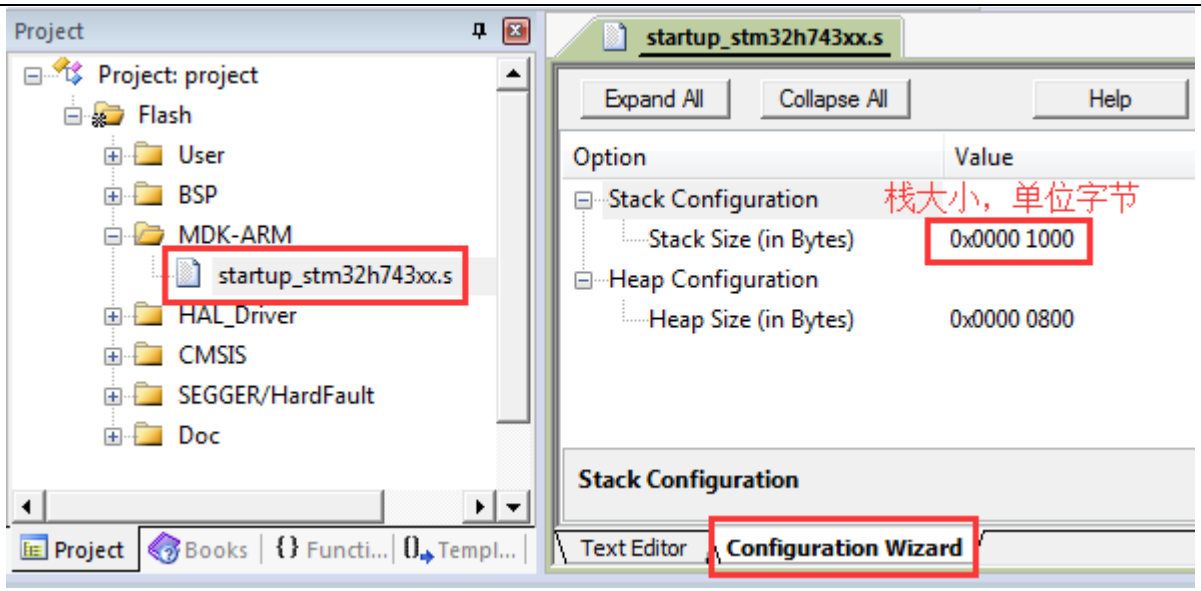
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

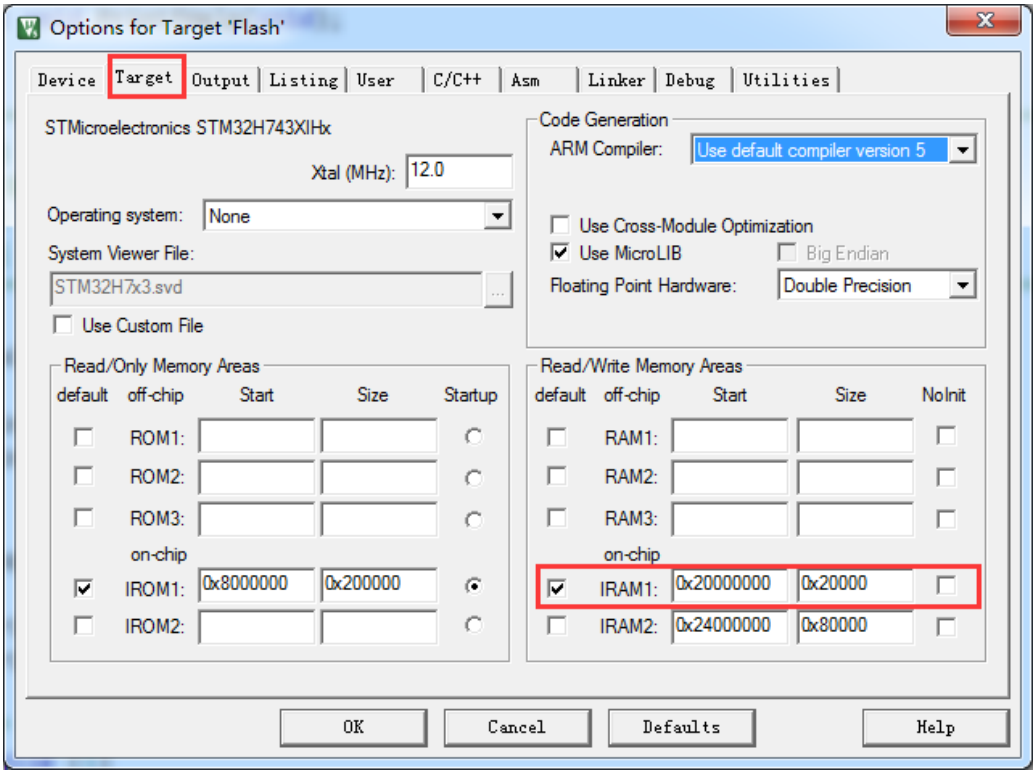


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();
```

```
    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **主功能：**

  主程序实现如下操作：

  - K1 键按下，进入停机模式，8 秒后自动唤醒。

```
/*
```

```
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */



    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，进入停机模式，8 秒后自动唤醒 */
                    printf("--进入停机模式，8 秒后唤醒\r\n");
                    bsp_StartLPTIM();
                    HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);
                    SystemClock_Config();   /* 特别注意，退出后要重新配置系统时钟 */
                    printf("--退出停机模式\r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

38.7 实验例程说明（IAR）

配套例子：

V7-022_低功耗定时器超时模式的停机唤醒

实验目的：

1. 习低功耗定时器超时模式的停机唤醒。

实验内容：

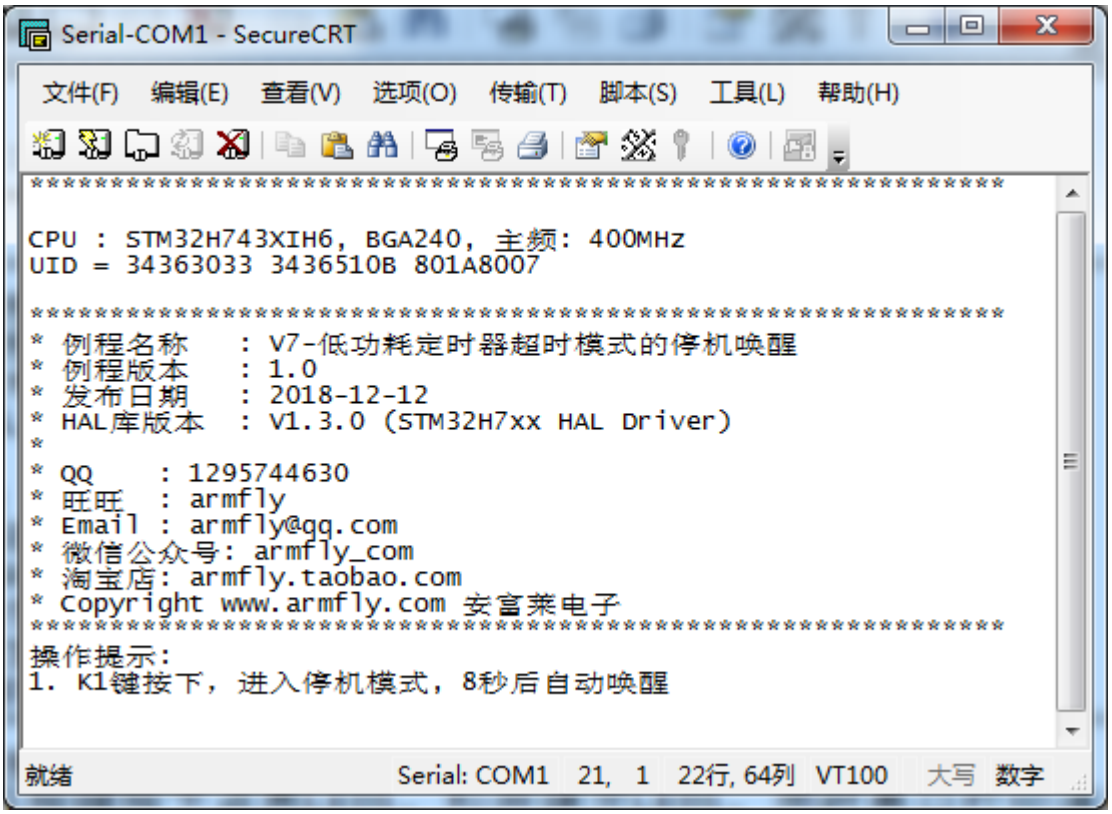
1. 使用 LPTIM 的好处是系统处于睡眠，停机状态依然可以正常工作（除了待机模式）。停机状态可以正常工作的关键是 LSE，LSI 时钟不会被关闭，同时也可以选择使用外部时钟源。
2. 例子默认用的 LSE 时钟供 LPTIM1 使用，大家可以通过 bsp_lptim_pwm.c 文件开头宏定义切换到 LSI 或者 PLCK。
3. 本例子利用 LPTIM 的超时功能，配置 8 秒后唤醒停机模式。
4. 板子正常运行时 LED2 闪烁，进入停机模式后，LED2 停止闪烁。
5. 进入 LPTIM 的超时中断后，LED4 翻转，方便查看是否正常执行。

实验操作：

1. K1 键按下，进入停机模式, 8 秒后自动唤醒。

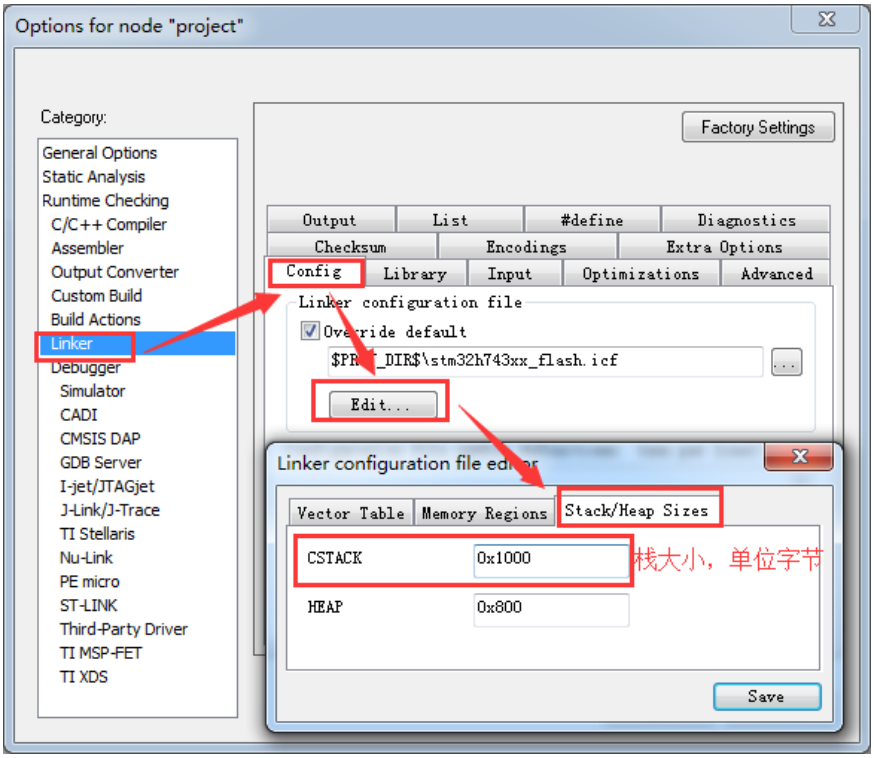
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

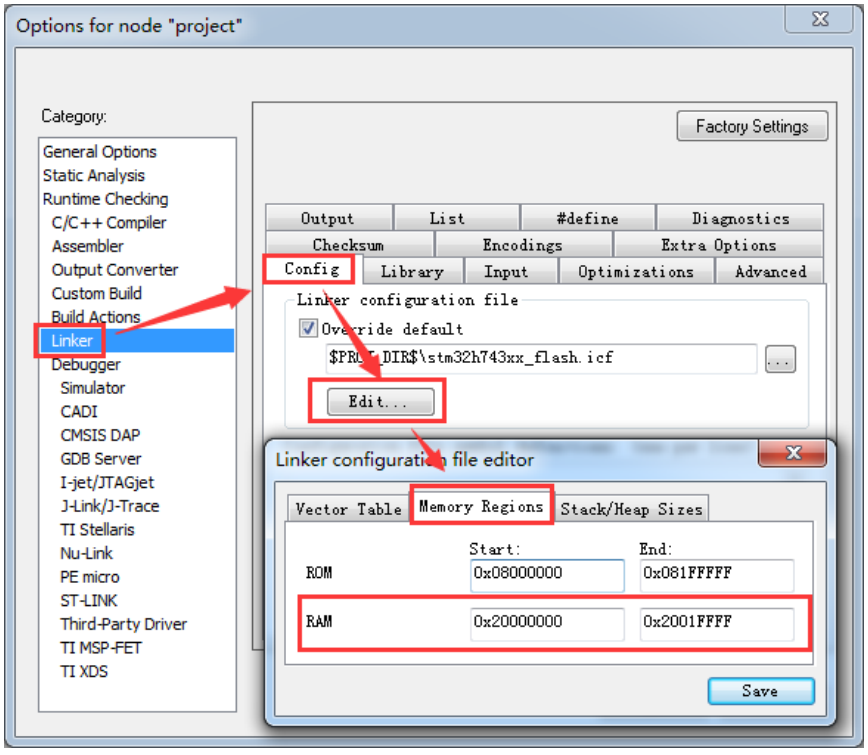


程序设计：

◆ **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
```

```
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
```

```
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主功能的实现主要分为两部分：

- 获取其它 RS485 设备发来的命令，并执行相应功能。

  通过函数 comGetChar(COM3, &ucDataTravel)以查询、非阻塞方式获取其它开发板发来的数据。

- 根据不同的按键消息，向其它 485 设备发送命令

  通过函数 comSendChar(COM3, ucDataTravel)以非阻塞方式向其它 485 设备发送命令，以此来执行同样的功能。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */



    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，进入停机模式，8 秒后自动唤醒 */
                    printf("--进入停机模式，8 秒后唤醒\r\n");
                    bsp_StartLPTIM();
                    HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);
```

```
                        SystemClock_Config();   /* 特别注意，退出后要重新配置系统时钟 */
                        printf("--退出停机模式\r\n");
                        break;

                    default:
                        /* 其它的键值不处理 */
                        break;
                }
            }
        }
    }
```

38.8 总结

本章节就为大家讲解这么多，低功耗定时器在低功耗场合比较有用，如果有低功耗方面的项目需求，可以考虑使用这个定时器实现低功耗唤醒。

第39章 STM32H7 的 DMAMUX 基础知识（重要）

本章教程为大家讲解 DMAMUX（Direct memory access request multiplexer，直接存储器访问请求复用器），本章知识点非常重要，是掌握好 DMA1，DMA2 和 BDMA 的关键一步。

39.1 初学者重要提示
39.2 DMAMUX 基础知识
39.3 DMAMUX 的 HAL 库用法
39.4 源文件 stm32h7xx_hal_dma_ex.c
39.5 总结

39.1 初学者重要提示

- DMAMUX 其实就是 DMA 控制器前一级的多路选择器，有了这个选择器就不用再像 F1，F4 系列那样每个通道（数据流）要固定选择指定的外设，有了多路选择器就可以任意选择，外设使用 DMA 方式时无需再选择指定的 DMA 通道（数据流），任意通道（数据流）都可以。

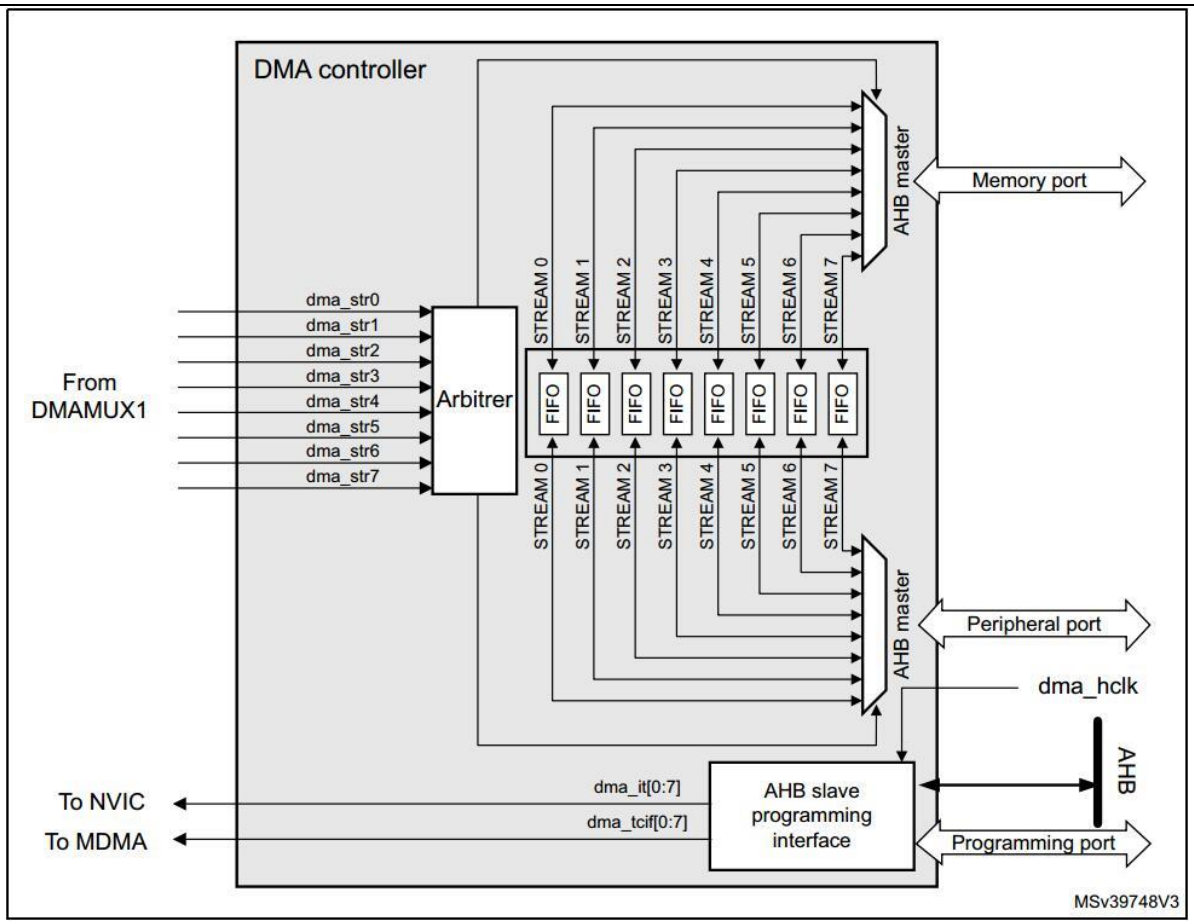
39.2 DMAMUX 基础知识

当前STM32H7有两路DMAMUX，分别是DMAMUX1和DMAMUX2，其中DMAMUX1负责DMA1和 DMA2，而 DMAMUX2 负责 BDMA。

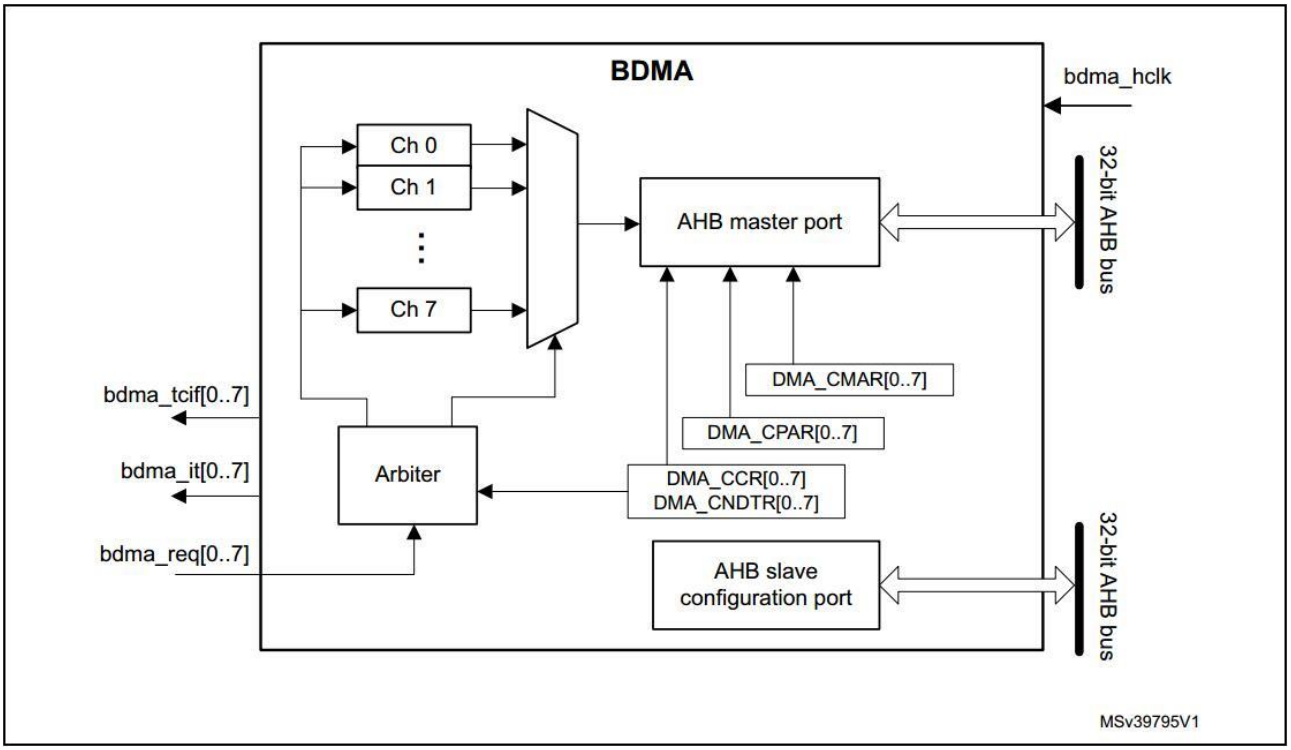
39.2.1 DMAMUX 和 DMA 的连接关系

认识一个外设，最好的方式就是看它的框图，方便我们快速的了解 DMAMUX 的基本功能，然后再看手册了解细节。首先来看下 DMAMUX 与 DMA 之间的连接方式，从整体上把握下，可以更好的理解 DMAMUX 的作用。

DMAMUX1 有 16 个输出通道，前 8 个通道分别连接 DMA1 Stream0 到 Stream7，而后 8 个通道分别连接DMA2 Stream0到Stream7。下面DMAMUX1的前8路接到DMA1的8路数据流通道Stream0到 Stream7 中。

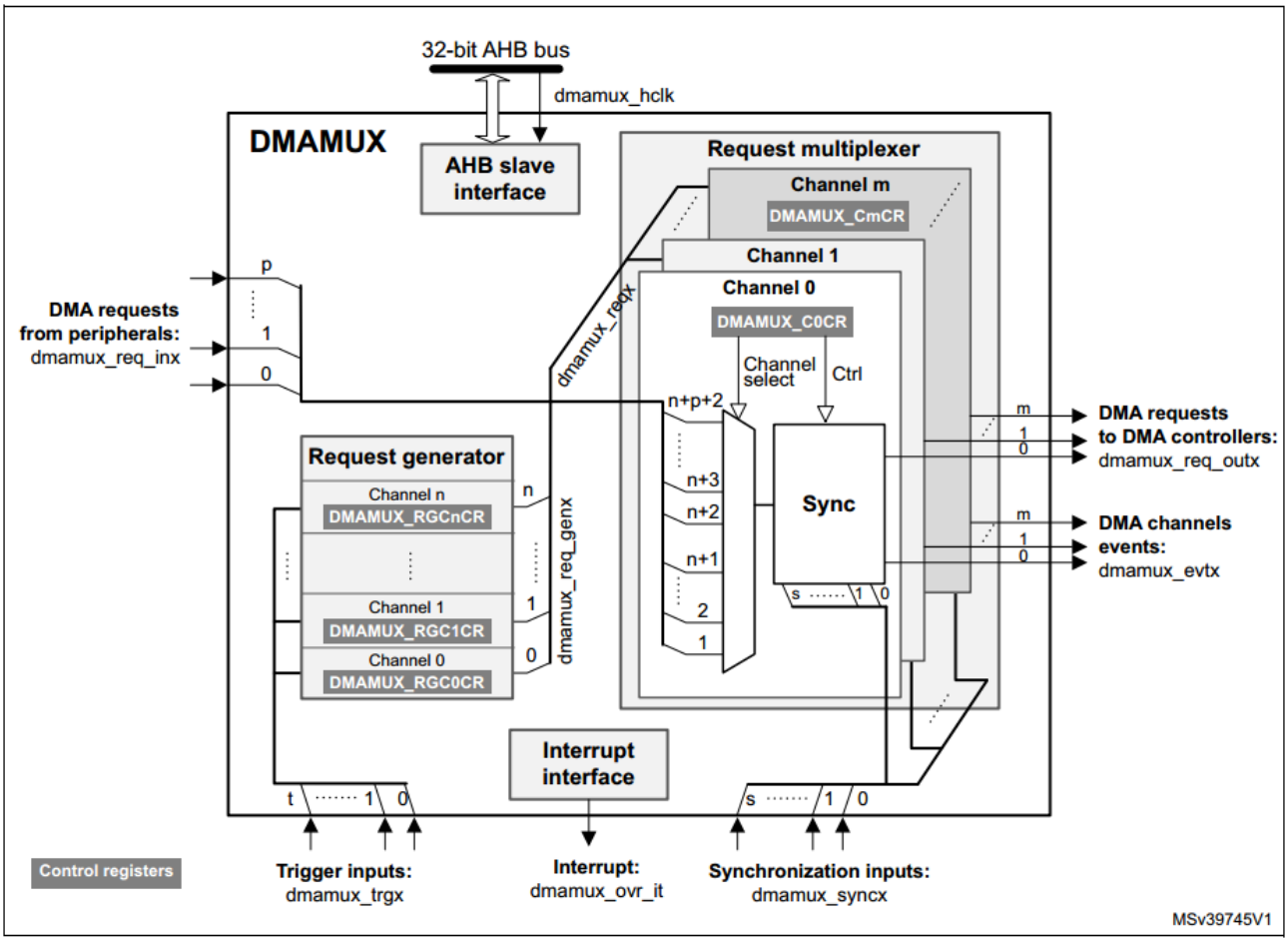


DMAMUX2 有 8 个输出通道，连接 BDMA 的 8 个输入通道:



39.2.2 DMAMUX 的硬件框图

这个框图对于理解 DMAMUX 至关重要。



通过这个框图，我们可以得到如下信息：

- DMA requests from peripherals 接口

对于 DMAMUX1 来说，这个接口支持 107 个 DMA 外设请求，供 DMA1 和 DMA2 的数据流使用：

```
#define DMA_REQUEST_ADC1                9U  /*!< DMAMUX1 ADC1 request */
#define DMA_REQUEST_ADC2                10U /*!< DMAMUX1 ADC2 request */

#define DMA_REQUEST_TIM1_CH1            11U  /*!< DMAMUX1 TIM1 CH1 request  */
#define DMA_REQUEST_TIM1_CH2            12U  /*!< DMAMUX1 TIM1 CH2 request  */
#define DMA_REQUEST_TIM1_CH3            13U  /*!< DMAMUX1 TIM1 CH3 request  */
#define DMA_REQUEST_TIM1_CH4            14U  /*!< DMAMUX1 TIM1 CH4 request  */
#define DMA_REQUEST_TIM1_UP             15U  /*!< DMAMUX1 TIM1 UP request   */
#define DMA_REQUEST_TIM1_TRIG           16U  /*!< DMAMUX1 TIM1 TRIG request */
#define DMA_REQUEST_TIM1_COM            17U  /*!< DMAMUX1 TIM1 COM request  */

中间部分省略未写

#define DMA_REQUEST_TIM16_CH1          109U  /*!< DMAMUX1 TIM16 CH1 request  */
#define DMA_REQUEST_TIM16_UP           110U  /*!< DMAMUX1 TIM16 UP request   */

#define DMA_REQUEST_TIM17_CH1          111U  /*!< DMAMUX1 TIM17 CH1 request  */
```

```
#define DMA_REQUEST_TIM17_UP         112U  /*!< DMAMUX1 TIM17 UP request   */

#define DMA_REQUEST_SAI3_A           113U  /*!< DMAMUX1 SAI3 A request  */
#define DMA_REQUEST_SAI3_B           114U  /*!< DMAMUX1 SAI3 B request  */

#define DMA_REQUEST_ADC3             115U  /*!< DMAMUX1 ADC3  request  */
```

对于 DMAMUX2 来说，这个接口支持 9 个 DMA 外设请求，供 BDMA 通道使用：

```
#define BDMA_REQUEST_LPUART1_RX       9U  /*!< DMAMUX2 LP_UART1_RX request */
#define BDMA_REQUEST_LPUART1_TX      10U  /*!< DMAMUX2 LP_UART1_TX request */
#define BDMA_REQUEST_SPI6_RX         11U  /*!< DMAMUX2 SPI6 RX request     */
#define BDMA_REQUEST_SPI6_TX         12U  /*!< DMAMUX2 SPI6 TX request     */
#define BDMA_REQUEST_I2C4_RX         13U  /*!< DMAMUX2 I2C4 RX request     */
#define BDMA_REQUEST_I2C4_TX         14U  /*!< DMAMUX2 I2C4 TX request     */
#define BDMA_REQUEST_SAI4_A          15U  /*!< DMAMUX2 SAI4 A request      */
#define BDMA_REQUEST_SAI4_B          16U  /*!< DMAMUX2 SAI4 B request      */
#define BDMA_REQUEST_ADC3            17U  /*!< DMAMUX2 ADC3 request        */
```

这里特别注意一点，DMA1，DMA2 和 BDMA 都支持存储区到存储区的传输。

◆ Trigger inputs 接口

除了正常的 DMA 请求可以输入到 DMAMUX 里面，通过设置触发条件也可以生成 DMA 触发请求。这样就比较灵活了，不支持 DMA 的外设也可以通过 Trigger inputs 接口触发 DMA 传输，比如我们可以将 RAM 中的数据通过定时器触发直接输出到 GPIO，这样就可以产生各种脉冲效果。

DMAMUX1 支持 8 种触发输入，供 DMA1 和 DMA2 使用：

```
#define HAL_DMAMUX1_REQ_GEN_DMAMUX1_CH0_EVT   0U
#define HAL_DMAMUX1_REQ_GEN_DMAMUX1_CH1_EVT   1U
#define HAL_DMAMUX1_REQ_GEN_DMAMUX1_CH2_EVT   2U
#define HAL_DMAMUX1_REQ_GEN_LPTIM1_OUT        3U
#define HAL_DMAMUX1_REQ_GEN_LPTIM2_OUT        4U
#define HAL_DMAMUX1_REQ_GEN_LPTIM3_OUT        5U
#define HAL_DMAMUX1_REQ_GEN_EXTI0             6U
#define HAL_DMAMUX1_REQ_GEN_TIM12_TRGO        7U
```

DMAMUX2 支持的 30 种触发输入，供 BDMA 使用：

```
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH0_EVT   0U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH1_EVT   1U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH2_EVT   2U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH3_EVT   3U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH4_EVT   4U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH5_EVT   5U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH6_EVT   6U
#define HAL_DMAMUX2_REQ_GEN_LPUART1_RX_WKUP   7U
#define HAL_DMAMUX2_REQ_GEN_LPUART1_TX_WKUP   8U
#define HAL_DMAMUX2_REQ_GEN_LPTIM2_WKUP       9U
#define HAL_DMAMUX2_REQ_GEN_LPTIM2_OUT       10U
#define HAL_DMAMUX2_REQ_GEN_LPTIM3_WKUP      11U
#define HAL_DMAMUX2_REQ_GEN_LPTIM3_OUT       12U
#define HAL_DMAMUX2_REQ_GEN_LPTIM4_WKUP      13U
#define HAL_DMAMUX2_REQ_GEN_LPTIM5_WKUP      14U
#define HAL_DMAMUX2_REQ_GEN_I2C4_WKUP        15U
#define HAL_DMAMUX2_REQ_GEN_SPI6_WKUP        16U
#define HAL_DMAMUX2_REQ_GEN_COMP1_OUT        17U
#define HAL_DMAMUX2_REQ_GEN_COMP2_OUT        18U
#define HAL_DMAMUX2_REQ_GEN_RTC_WKUP         19U
#define HAL_DMAMUX2_REQ_GEN_EXTI0            20U
```

```
#define HAL_DMAMUX2_REQ_GEN_EXTI2            21U
#define HAL_DMAMUX2_REQ_GEN_I2C4_IT_EVT      22U
#define HAL_DMAMUX2_REQ_GEN_SPI6_IT          23U
#define HAL_DMAMUX2_REQ_GEN_LPUART1_TX_IT    24U
#define HAL_DMAMUX2_REQ_GEN_LPUART1_RX_IT    25U
#define HAL_DMAMUX2_REQ_GEN_ADC3_IT          26U
#define HAL_DMAMUX2_REQ_GEN_ADC3_AWD1_OUT    27U
#define HAL_DMAMUX2_REQ_GEN_BDMA_CH0_IT      28U
#define HAL_DMAMUX2_REQ_GEN_BDMA_CH1_IT      29U
```

- Interrupt 接口

  用于触发中断。

- Synchronization inputs 接口

  同步输入接口可以用来控制 DMAMUX 的输入端的 DMA 外设请求到输出端的同步控制，其实就是控制何时输出。

  DMAMUX1 支持的 8 种同步输入，供 DMA1 和 DMA2 使用：

```
#define HAL_DMAMUX1_SYNC_DMAMUX1_CH0_EVT   0U
#define HAL_DMAMUX1_SYNC_DMAMUX1_CH1_EVT   1U
#define HAL_DMAMUX1_SYNC_DMAMUX1_CH2_EVT   2U
#define HAL_DMAMUX1_SYNC_LPTIM1_OUT        3U
#define HAL_DMAMUX1_SYNC_LPTIM2_OUT        4U
#define HAL_DMAMUX1_SYNC_LPTIM3_OUT        5U
#define HAL_DMAMUX1_SYNC_EXTI0             6U
#define HAL_DMAMUX1_SYNC_TIM12_TRGO        7U
```

DMAMUX2 支持的 16 种同步输入，供 BDMA 使用：

```
#define HAL_DMAMUX2_SYNC_DMAMUX2_CH0_EVT   0U
#define HAL_DMAMUX2_SYNC_DMAMUX2_CH1_EVT   1U
#define HAL_DMAMUX2_SYNC_DMAMUX2_CH2_EVT   2U
#define HAL_DMAMUX2_SYNC_DMAMUX2_CH3_EVT   3U
#define HAL_DMAMUX2_SYNC_DMAMUX2_CH4_EVT   4U
#define HAL_DMAMUX2_SYNC_DMAMUX2_CH5_EVT   5U
#define HAL_DMAMUX2_SYNC_LPUART1_RX_WKUP   6U
#define HAL_DMAMUX2_SYNC_LPUART1_TX_WKUP   7U
#define HAL_DMAMUX2_SYNC_LPTIM2_OUT        8U
#define HAL_DMAMUX2_SYNC_LPTIM3_OUT        9U
#define HAL_DMAMUX2_SYNC_I2C4_WKUP        10U
#define HAL_DMAMUX2_SYNC_SPI6_WKUP        11U
#define HAL_DMAMUX2_SYNC_COMP1_OUT        12U
#define HAL_DMAMUX2_SYNC_RTC_WKUP         13U
#define HAL_DMAMUX2_SYNC_EXTI0            14U
#define HAL_DMAMUX2_SYNC_EXTI2            15U
```

- DMA Channels event 接口

  DMAMUX 的事件输出。

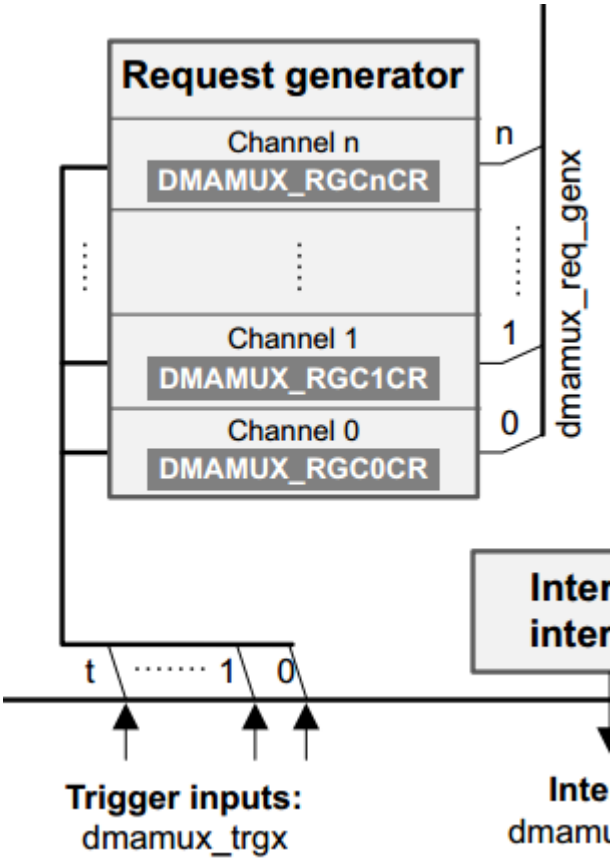
- DMA requests to DMA controllers 接口

  DMAMUX 的输出，发往 DMA1，DMA2 或者 BDMA。

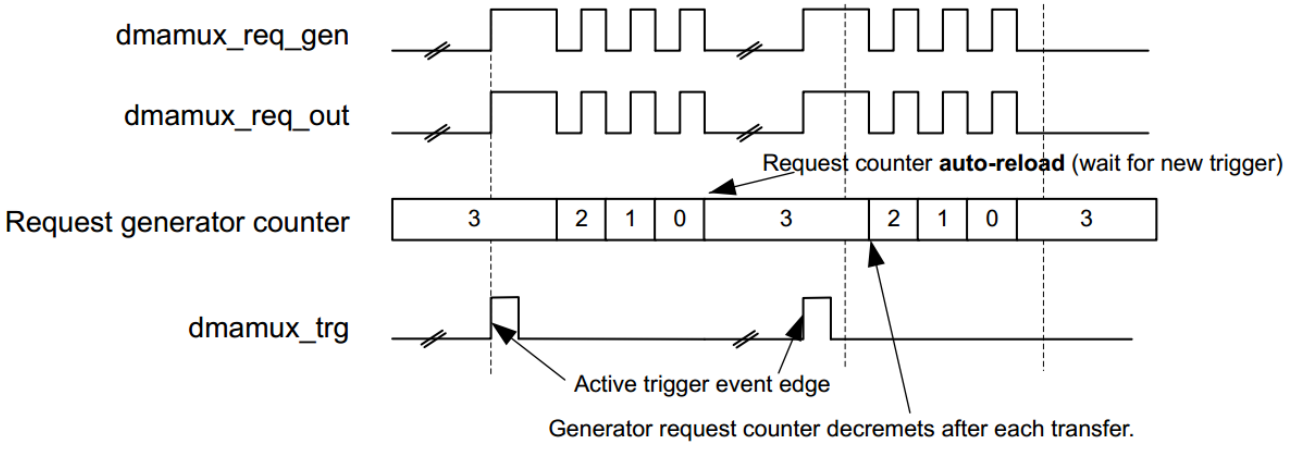
39.2.3请求发生器（Request Generator）

请求触发器最大的优势就是可以让不支持 DMA 传输的外设也可以通过 Trigger inputs 接口触发 DMA 传输，比如我们可以将 RAM 中的数据通过定时器触发直接输出到 GPIO，这样就可以产生各种脉冲效果，这样就比较灵活了。

这里我们再进一步的认识下请求发生器，通过下面框图可以看出请求发生器有 n 个通道，并且每个通道都支持 t 个触发。这里有一个关键知识点，所有这些通道可以选择同一个触发源。



了解了这些之后，我们要对它的工作过程有一个简单的认识，看下面的时序图：



◆ dmamux_req_gen 信号请求发生器生成的 DMA 请求。

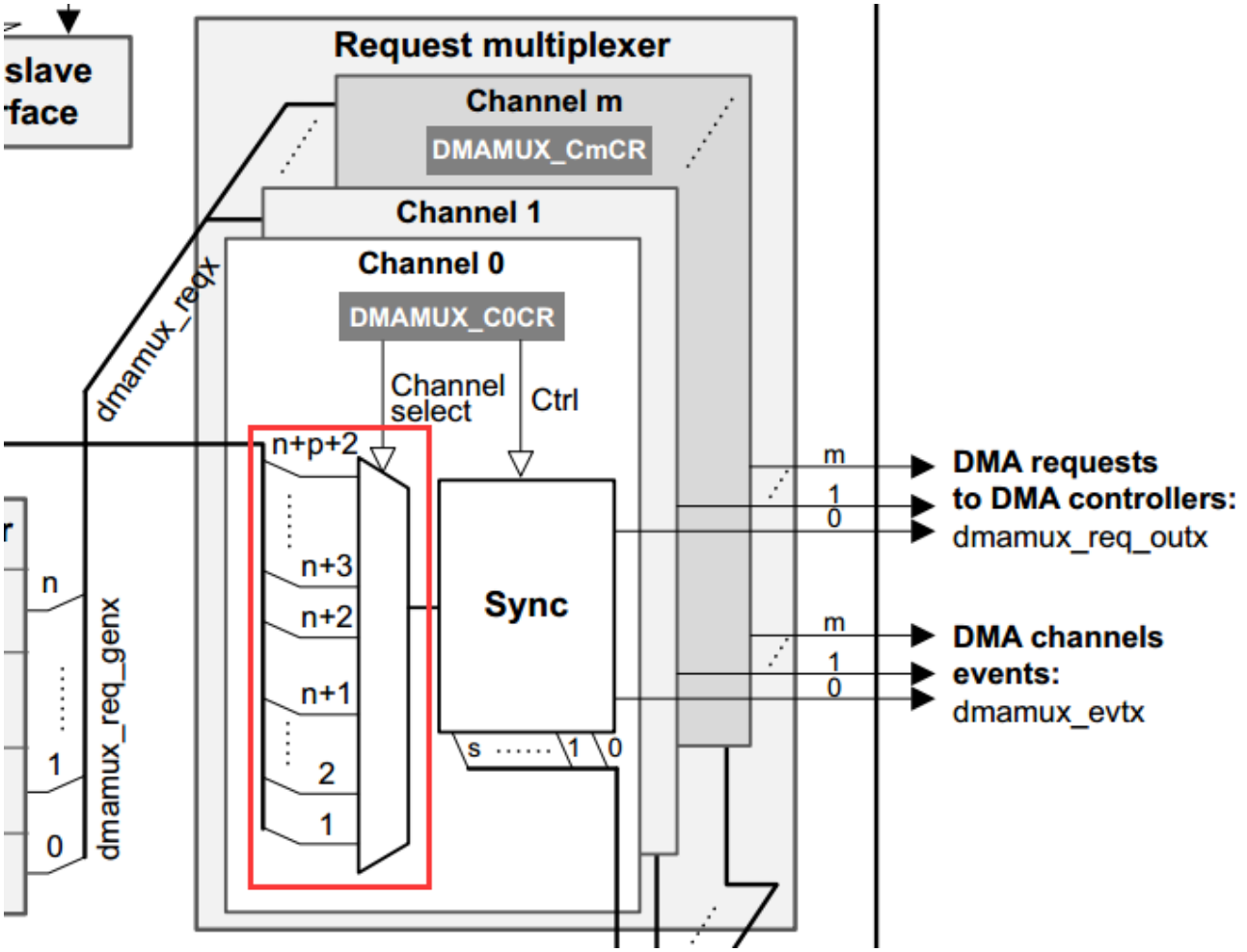
- dmamux_req_out 信号是 DMAMUX 的输出，供 DMA1，DMA2 或者 BDMA 使用。
- Request generator counter 这个计数器比较重要，它的意思是一次 dmamux_trg 触发信号，可以连续执行的 DMA 请求，最大 32 次。这里是以 DMA 可以执行的最快速度进行响应的。每执行一次，计数器减 1，减到 0 后自动加载用户设置的最大次数，等待下次触发，依次进行。如果计数器还没有减到 0 就再次触发，请求发生器的中断状态寄存器 DMAMUX_RGSR 的标志将被置位，如果使能了中断，将会被触发。
- dmamux_trg 信号是触发源。

39.2.4同步触发和请求复用器（Request multiplexer）

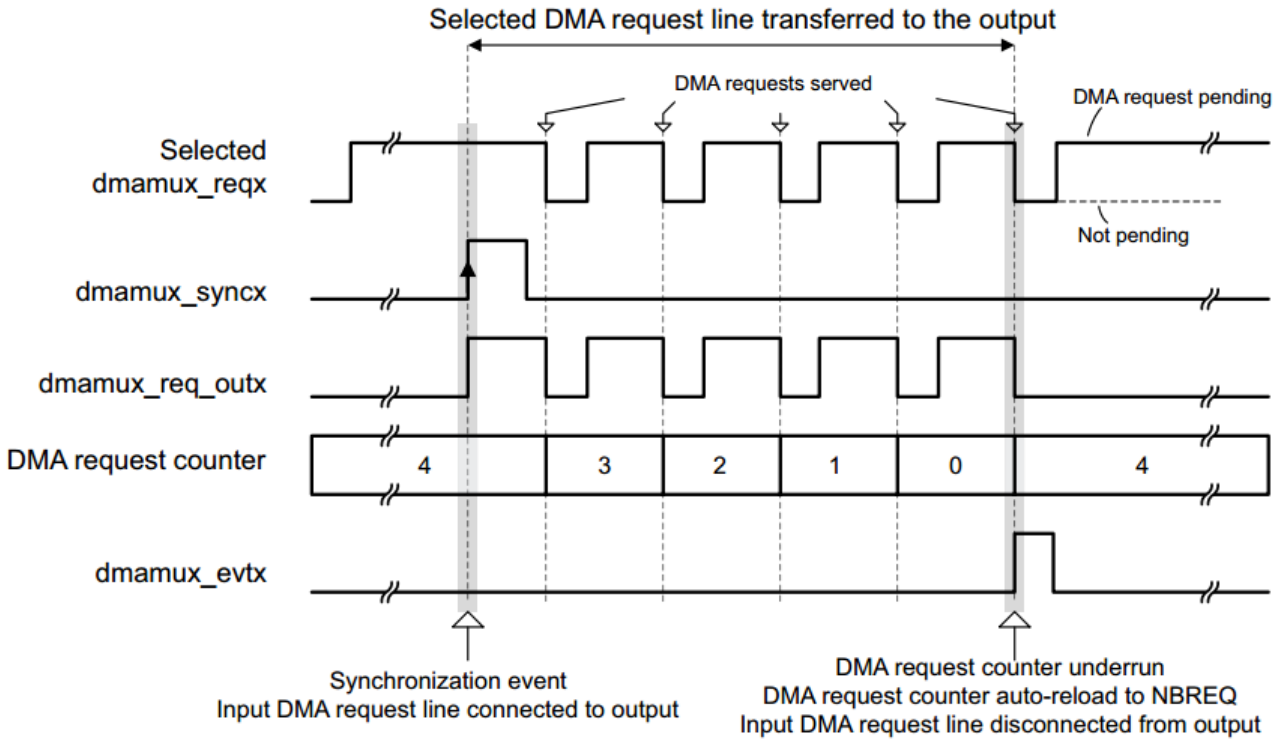
同步输入接口可以用来控制 DMAMUX 的输入端的 DMA 外设请求到输出端的同步控制，其实就是控制何时输出。

这里我们再进一步的认识下请求复用器，从下面的框图中可以看出请求发生器有 m 个通道，并且每个通道都支持 n+p+2 个 DMA 请求，但是每个通道不可以选择相同的 DMA 请求。

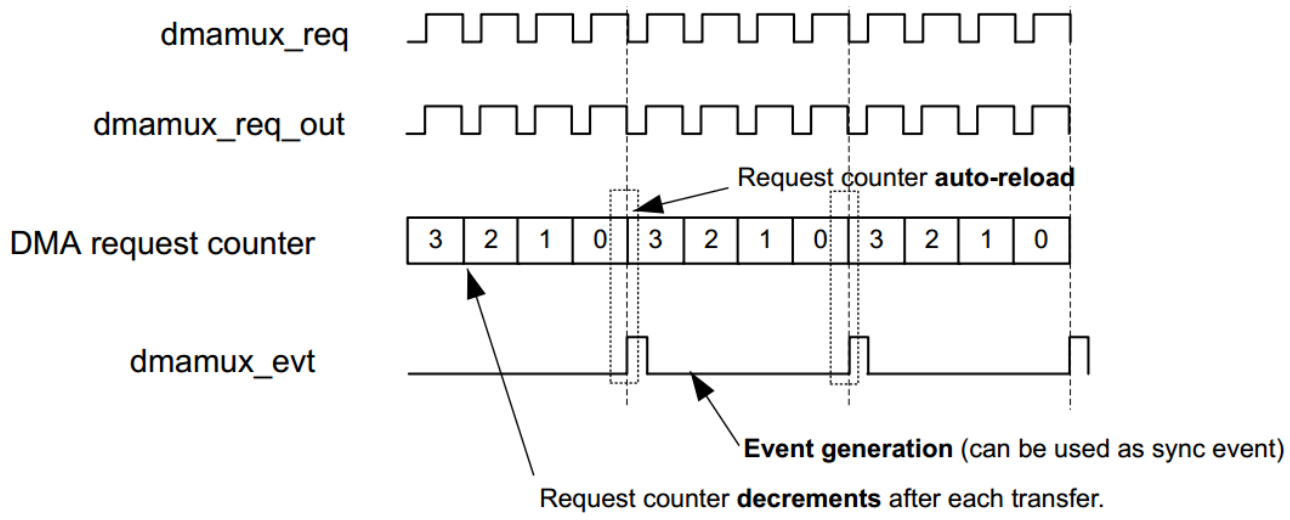
特别注意红色方框的地方，请求发生器的 n 个 DMA 请求和 p 个 DMA 外设请求全都汇集于此，可供这里的多路选择器选择。



了解了这些之后，我们要对它的工作过程有一个简单的认识，看下面的时序图：



下面连续同步三次的效果，每次产生 4 次 DMA 请求：



- dmamux_reqx 信号是请求复用器的输入端。
- dmamux_syncx 是同步触发信号。
- dmamux_req_outx 信号是 DMAMUX 的输出，供 DMA1，DMA2 或者 BDMA 使用。
- DMA request counter 这个计数器比较重要，他的意思是一次 dmamux_syncx 触发信号，可以连续执行的 DMA 请求，最大 32 次。这里是以 DMA 请求可以执行的最快速度进行响应的。每执行一次，计数器减 1，减到 0 后自动加载用户设置的最大次数，等待下次触发，依次进行。如果计数器还没有

减到 0 就再次触发，请求发生器的中断状态寄存器 DMAMUX_CSR 的标志将被置位，如果使能了中断，将会被触发。

- dmamux_evt 信号是输出事件。

39.2.5 正常的 DMA 请求方式

除了前面说的请求发生器产生的 DMA 请求和同步触发产生的 DMA 请求，关闭了这两种方式后也可以通过 DMAMUX 正常发出 DMA 请求的，这种情况和之前使用 F1 和 F4 系列是一样的。

39.3 DMAMUX 的 HAL 库用法

DMAMUX 的 HAL 库用法其实就是几个结构体变量成员的配置和使用，然后配置 GPIO、时钟，并根据需要配置 NVIC、中断和 DMA。下面我们逐一展开为大家做个说明。

39.3.1 请求发生器结构体

HAL_DMA_MuxRequestGeneratorConfigTypeDef

HAL_DMA_MuxRequestGeneratorConfigTypeDef 的定义如下：

```
typedef struct
{
 uint32_t SignalID;  /*!< Specifies the ID of the signal used for DMAMUX request generator
                         This parameter can be a value of @ref DMAEx_MUX_SignalGeneratorID_selection */

  uint32_t Polarity; /*!< Specifies the polarity of the signal on which the request is generated.
                      This parameter can be a value of @ref DMAEx_MUX_RequestGeneneratorPolarity_selection */

  uint32_t RequestNumber;  /*!< Specifies the number of DMA request that will be generated after a signal event.
                            This parameters can be in the range 1 to 32 */

}HAL_DMA_MuxRequestGeneratorConfigTypeDef;
```

下面将这三个结构体成员一 一为大家做个说明。

- **SingnalID**

请求发生器的触发源 ID 选择，DMAMUX1 可以选择的触发源 ID 如下：

```
#define HAL_DMAMUX1_REQ_GEN_DMAMUX1_CH0_EVT   0U
#define HAL_DMAMUX1_REQ_GEN_DMAMUX1_CH1_EVT   1U
#define HAL_DMAMUX1_REQ_GEN_DMAMUX1_CH2_EVT   2U
#define HAL_DMAMUX1_REQ_GEN_LPTIM1_OUT        3U
#define HAL_DMAMUX1_REQ_GEN_LPTIM2_OUT        4U
#define HAL_DMAMUX1_REQ_GEN_LPTIM3_OUT        5U
#define HAL_DMAMUX1_REQ_GEN_EXTI0             6U
#define HAL_DMAMUX1_REQ_GEN_TIM12_TRGO        7U
```

DMAMUX2 可以选择的触发源 ID 如下：

```
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH0_EVT   0U
```

```
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH1_EVT    1U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH2_EVT    2U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH3_EVT    3U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH4_EVT    4U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH5_EVT    5U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH6_EVT    6U
#define HAL_DMAMUX2_REQ_GEN_LPUART1_RX_WKUP    7U
#define HAL_DMAMUX2_REQ_GEN_LPUART1_TX_WKUP    8U
#define HAL_DMAMUX2_REQ_GEN_LPTIM2_WKUP        9U
#define HAL_DMAMUX2_REQ_GEN_LPTIM2_OUT        10U
#define HAL_DMAMUX2_REQ_GEN_LPTIM3_WKUP       11U
#define HAL_DMAMUX2_REQ_GEN_LPTIM3_OUT        12U
#define HAL_DMAMUX2_REQ_GEN_LPTIM4_WKUP       13U
#define HAL_DMAMUX2_REQ_GEN_LPTIM5_WKUP       14U
#define HAL_DMAMUX2_REQ_GEN_I2C4_WKUP         15U
#define HAL_DMAMUX2_REQ_GEN_SPI6_WKUP         16U
#define HAL_DMAMUX2_REQ_GEN_COMP1_OUT         17U
#define HAL_DMAMUX2_REQ_GEN_COMP2_OUT         18U
#define HAL_DMAMUX2_REQ_GEN_RTC_WKUP          19U
#define HAL_DMAMUX2_REQ_GEN_EXTI0             20U
#define HAL_DMAMUX2_REQ_GEN_EXTI2             21U
#define HAL_DMAMUX2_REQ_GEN_I2C4_IT_EVT       22U
#define HAL_DMAMUX2_REQ_GEN_SPI6_IT           23U
#define HAL_DMAMUX2_REQ_GEN_LPUART1_TX_IT     24U
#define HAL_DMAMUX2_REQ_GEN_LPUART1_RX_IT     25U
#define HAL_DMAMUX2_REQ_GEN_ADC3_IT           26U
#define HAL_DMAMUX2_REQ_GEN_ADC3_AWD1_OUT     27U
#define HAL_DMAMUX2_REQ_GEN_BDMA_CH0_IT       28U
#define HAL_DMAMUX2_REQ_GEN_BDMA_CH1_IT       29U
```

- **Polarity**

  触发信号的极性设置，可以是上升沿触发、下降沿触发或者双沿触发。

```
#define HAL_DMAMUX_REQ_GEN_NO_EVENT         0x00000000U
#define HAL_DMAMUX_REQ_GEN_RISING           DMAMUX_RGxCR_GPOL_0
#define HAL_DMAMUX_REQ_GEN_FALLING          DMAMUX_RGxCR_GPOL_1
#define HAL_DMAMUX_REQ_GEN_RISING_FALLING   DMAMUX_RGxCR_GPOL
```

- **RequestNumber**

  每次触发信号后，可以执行的 DMA 请求次数，这个在本章的 2.3 小节已经进行了详细说明。

39.3.2同步触发结构体 HAL_DMA_MuxSyncConfigTypeDef

HAL_DMA_MuxSyncConfigTypeDef 的定义如下：：

```
typedef struct
{
  uint32_t SyncSignalID;
  uint32_t SyncPolarity;
  FunctionalState SyncEnable;
  FunctionalState EventEnable;
  uint32_t RequestNumber;
}HAL_DMA_MuxSyncConfigTypeDef;
```

下面将这五个结构体成员一 一为大家做个说明。

- **SyncSingnalID**

同步触发源的 ID 选择，DMAMUX1 可以选择的触发源 ID 如下：

```
#define HAL_DMAMUX1_SYNC_DMAMUX1_CH0_EVT    0U
#define HAL_DMAMUX1_SYNC_DMAMUX1_CH1_EVT    1U
#define HAL_DMAMUX1_SYNC_DMAMUX1_CH2_EVT    2U
#define HAL_DMAMUX1_SYNC_LPTIM1_OUT         3U
#define HAL_DMAMUX1_SYNC_LP TIM2_OUT        4U
#define HAL_DMAMUX1_SYNC_LPTIM3_OUT         5U
#define HAL_DMAMUX1_SYNC_EXTI0              6U
#define HAL_DMAMUX1_SYNC_TIM12_TRGO         7U
```

DMAMUX2 可以选择的触发源 ID 如下：

```
#define HAL_DMAMUX2_SYNC_DMAMUX2_CH0_EVT    0U
#define HAL_DMAMUX2_SYNC_DMAMUX2_CH1_EVT    1U
#define HAL_DMAMUX2_SYNC_DMAMUX2_CH2_EVT    2U
#define HAL_DMAMUX2_SYNC_DMAMUX2_CH3_EVT    3U
#define HAL_DMAMUX2_SYNC_DMAMUX2_CH4_EVT    4U
#define HAL_DMAMUX2_SYNC_DMAMUX2_CH5_EVT    5U
#define HAL_DMAMUX2_SYNC_LPUART1_RX_WKUP    6U
#define HAL_DMAMUX2_SYNC_LPUART1_TX_WKUP    7U
#define HAL_DMAMUX2_SYNC_LPTIM2_OUT         8U
#define HAL_DMAMUX2_SYNC_LPTIM3_OUT         9U
#define HAL_DMAMUX2_SYNC_I2C4_WKUP         10U
#define HAL_DMAMUX2_SYNC_SPI6_WKUP         11U
#define HAL_DMAMUX2_SYNC_COMP1_OUT         12U
#define HAL_DMAMUX2_SYNC_RTC_WKUP          13U
#define HAL_DMAMUX2_SYNC_EXTI0             14U
#define HAL_DMAMUX2_SYNC_EXTI2             15U
```

- **SyncPolarity**

同步触发信号的极性设置，可以是上升沿触发、下降沿触发或者双沿触发。

```
#define HAL_DMAMUX_SYNC_NO_EVENT         0x00000000U
#define HAL_DMAMUX_SYNC_RISING           DMAMUX_CxCR_SPOL_0
#define HAL_DMAMUX_SYNC_FALLING          DMAMUX_CxCR_SPOL_1
#define HAL_DMAMUX_SYNC_RISING_FALLING   DMAMUX_CxCR_SPOL
```

- **SyncEnable**

用于使能同步触发，参数可以是 ENABLE 或者 DISABLE。

- **EventEnable**

同步触发事件输出使能，当参数 RequestNumber 的数值减到 0 的时候才会输出。

- **RequestNumber**

每次同步触发信号后，可以执行的 DMA 请求次数，这个在本章的 2.4 小节已经进行了详细说明。

39.3.3 DMAMUX 的状态标志清除问题

DMAMUX 有两个中断，一个是请求复用器通道发生同步事件溢出，另一个是请求发生器通道发生触发事件溢出。

当用户调用了函数 HAL_DMA_Start_IT，程序代码中会根据用户是否使能了请求复用器或者请求发生器通道来使能这两个中断。

```
HAL_StatusTypeDef HAL_DMA_Start_IT(DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t
```

```
DataLength)
{
    /* 省略未写 */

    if((hdma->DMAmuxChannel->CCR & DMAMUX_CxCR_SE) != 0U)
    {
      /* 使能同步溢出中断 */
      hdma->DMAmuxChannel->CCR |= DMAMUX_CxCR_SOIE;
    }

    if(hdma->DMAmuxRequestGen != 0U)
    {
      /* 使能请求发生器的溢出中断 */
      hdma->DMAmuxRequestGen->RGCR |= DMAMUX_RGxCR_OIE;
    }

    /* 省略未写 */

}
```

这两个中断产生的中断标志是在函数 HAL_DMAEx_MUX_IRQHandler 里面做的清除，如果大家不使用 HAL 库提供的这个中断处理函数，就需要自己编写代码清除。

```
void HAL_DMAEx_MUX_IRQHandler(DMA_HandleTypeDef *hdma)
{
  /* 检测 DMAMUX 同步溢出 */
  if((hdma->DMAmuxChannelStatus->CSR & hdma->DMAmuxChannelStatusMask) != 0U)
  {
    /* 禁止同步溢出中断 */
    hdma->DMAmuxChannel->CCR &= ~DMAMUX_CxCR_SOIE;

    /* 清除 DMAMUX 同步溢出标志 */
    hdma->DMAmuxChannelStatus->CFR = hdma->DMAmuxChannelStatusMask;

    /* Update error code */
    hdma->ErrorCode |= HAL_DMA_ERROR_SYNC;

    if(hdma->XferErrorCallback != NULL)
    {
      /* 回调函数 */
      hdma->XferErrorCallback(hdma);
    }
  }

  if(hdma->DMAmuxRequestGen != 0)
  {
   /* 检测请求发生器溢出 */
    if((hdma->DMAmuxRequestGenStatus->RGSR & hdma->DMAmuxRequestGenStatusMask) != 0U)
    {
      /* 禁止请求发生器溢出中断 */
      hdma->DMAmuxRequestGen->RGCR &= ~DMAMUX_RGxCR_OIE;

      /* 清除 DMAMUX 请求发生器溢出标志 */
      hdma->DMAmuxRequestGenStatus->RGCFR = hdma->DMAmuxRequestGenStatusMask;

      /* 更新错误代码标识 */
      hdma->ErrorCode |= HAL_DMA_ERROR_REQGEN;
```

```
      if(hdma->XferErrorCallback != NULL)
      {
        /* 回调函数 */
        hdma->XferErrorCallback(hdma);
      }
    }
  }
}
```

39.3.4 DMAMUX 初始化流程总结

DMAMUX 没有单独的初始化流程，要结合第 40 章和 42 章的函数一起使用。

39.4 源文件 stm32h7xx_hal_dma_ex.c

DMAMUX 用到如下四个说明，这里把这四个函数的使用为大家做个说明：

- HAL_DMAEx_ConfigMuxSync
- HAL_DMAEx_ConfigMuxRequestGenerator
- HAL_DMAEx_EnableMuxRequestGenerator
- HAL_DMAEx_DisableMuxRequestGenerator

39.4.1 函数 HAL_DMAEx_ConfigMuxSync

函数原型：

```
HAL_StatusTypeDef HAL_DMAEx_ConfigMuxSync(DMA_HandleTypeDef *hdma, HAL_DMA_MuxSyncConfigTypeDef *pSyncConfig)
{
  uint32_t syncSignalID = 0;
  uint32_t syncPolarity = 0;

  /* 检测参数 */
  assert_param(IS_DMA_STREAM_ALL_INSTANCE(hdma->Instance));
  assert_param(IS_DMAMUX_SYNC_STATE(pSyncConfig->SyncEnable));
  assert_param(IS_DMAMUX_SYNC_EVENT(pSyncConfig->EventEnable));
  assert_param(IS_DMAMUX_SYNC_REQUEST_NUMBER(pSyncConfig->RequestNumber));

  /* 检测是否使能了同步触发 */
  if(pSyncConfig->SyncEnable == ENABLE)
  {
    assert_param(IS_DMAMUX_SYNC_POLARITY(pSyncConfig->SyncPolarity));

    if(IS_D2_DMA_INSTANCE(hdma) != 0U)
    {
      assert_param(IS_D2_DMAMUX_SYNC_SIGNAL_ID(pSyncConfig->SyncSignalID));
    }
    else
    {
      assert_param(IS_D3_DMAMUX_SYNC_SIGNAL_ID(pSyncConfig->SyncSignalID));
    }
```

```
    syncSignalID = pSyncConfig->SyncSignalID;
    syncPolarity = pSyncConfig->SyncPolarity;
  }

  /* 检测 DMA 是否处于就绪态 */
  if(hdma->State == HAL_DMA_STATE_READY)
  {
    /* 上锁 */
    __HAL_LOCK(hdma);

    /* 应用新的配置前禁止同步和产生同步事件 */
    CLEAR_BIT(hdma->DMAmuxChannel->CCR,(DMAMUX_CxCR_SE | DMAMUX_CxCR_EGE));

    /* 配置新的参数，同时 DMAMUX_CxCR_DMAREQ_ID 位保持不变 */
    MODIFY_REG( hdma->DMAmuxChannel->CCR, \
                (~DMAMUX_CxCR_DMAREQ_ID) , \
                (syncSignalID << POSITION_VAL(DMAMUX_CxCR_SYNC_ID))       | \
                ((pSyncConfig->RequestNumber - 1U) << POSITION_VAL(DMAMUX_CxCR_NBREQ)) | \
                syncPolarity | (pSyncConfig->SyncEnable << DMAMUX_POSITION_CxCR_SE)    | \
                (pSyncConfig->EventEnable << DMAMUX_POSITION_CxCR_EGE));

      /* 开锁 */
    __HAL_UNLOCK(hdma);

    return HAL_OK;
  }
  else
  {
    /* 设置错误标志 */
    hdma->ErrorCode = HAL_DMA_ERROR_BUSY;

    /* 返回状态 HAL_ERROR */
    return HAL_ERROR;
  }
}
```

函数描述：

此函数用于配置 DMAMUX 的同步触发，可以用来控制 DMAMUX 的输入端的 DMA 外设请求到输出端的同步控制，其实就是控制何时输出。

函数参数：

- 第 1 个参数是 DMA_HandleTypeDef 类型结构体指针变量，用于配置 DMA 的初始化参数。
- 第 2 个参数是 HAL_DMA_MuxSyncConfigTypeDef 类型结构体变量，参数成员的含义在本章的 3.2 小节有讲解说明。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 第 1 个参数的结构体成员介绍在第 40 章进行了讲解。
2. 调用此函数前要先调用函数 HAL_DMA_Init 配置好 DMA。

使用举例：

此函数的举例可以看此贴:

http://www.armbbs.cn/forum.php?mod=viewthread&tid=88925 。

39.4.2 函数 HAL_DMAEx_ConfigMuxRequestGenerator

函数原型:

```
HAL_StatusTypeDef HAL_DMAEx_ConfigMuxRequestGenerator (DMA_HandleTypeDef *hdma,
HAL_DMA_MuxRequestGeneratorConfigTypeDef *pRequestGeneratorConfig)
{
  HAL_StatusTypeDef status = HAL_OK;

  /* 检查参数 */
  assert_param(IS_DMA_STREAM_ALL_INSTANCE(hdma->Instance));

  if(IS_D2_DMA_INSTANCE(hdma) != 0U)
  {
    assert_param(IS_D2_DMAMUX_REQUEST_GEN_SIGNAL_ID(pRequestGeneratorConfig->SignalID));
  }
  else
  {
    assert_param(IS_D3_DMAMUX_REQUEST_GEN_SIGNAL_ID(pRequestGeneratorConfig->SignalID));
  }

  assert_param(IS_DMAMUX_REQUEST_GEN_POLARITY(pRequestGeneratorConfig->Polarity));
  assert_param(IS_DMAMUX_REQUEST_GEN_REQUEST_NUMBER(pRequestGeneratorConfig->RequestNumber));

  /* 如果 DMA 配置中未使用 DMAMUX 的请求发生器，返回 HAL_ERROR */
  if(hdma->DMAmuxRequestGen == 0U)
  {
    /* 设置参数错误 */
    hdma->ErrorCode = HAL_DMA_ERROR_PARAM;

    /* 设置错误状态 */
    status = HAL_ERROR;
  }
  /* 必须保证请求发生器是关闭的才可以配置 */
  else if((hdma->State == HAL_DMA_STATE_READY) && ((hdma->DMAmuxRequestGen->RGCR & DMAMUX_RGxCR_GE) == 0))
  {

    /* 上锁 */
    __HAL_LOCK(hdma);

    /* 设置新参数 */
    hdma->DMAmuxRequestGen->RGCR = pRequestGeneratorConfig->SignalID | \
                                   ((pRequestGeneratorConfig->RequestNumber - 1U) <<
                                    POSITION_VAL(DMAMUX_RGxCR_NBREQ))| \
                                   pRequestGeneratorConfig->Polarity;
    /* 解锁 */
    __HAL_UNLOCK(hdma);

    return HAL_OK;
  }
  else
  {
```

```
    /* 设置错误标志 */
    hdma->ErrorCode = HAL_DMA_ERROR_BUSY;

    /* 设置错误状态 */
    status = HAL_ERROR;
  }

  return status;
}
```

函数描述：

此函数用于配置 DMAMUX 的发生器。请求触发器最大的优势就是可以让不支持 DMA 传输的外设也可以通过 Trigger inputs 接口触发 DMA 传输，比如我们可以将 RAM 中的数据通过定时器触发直接输出到 GPIO，就可以产生各种脉冲效果，这样就比较灵活了。

函数参数：

- 第 1 个参数是 DMA_HandleTypeDef 类型结构体指针变量，用于配置 DMA 的初始化参数。
- 第 2 个参数是 HAL_DMA_MuxRequestGeneratorConfigTypeDef f 类型结构体变量，参数成员的含义在本章的 3.1 小节有讲解说明。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 第 1 个参数的结构体成员介绍在第 40 章进行了讲解。
2. 调用此函数前要先调用函数 HAL_DMA_Init 配置好 DMA。

使用举例：

```
HAL_DMA_MuxRequestGeneratorConfigTypeDef dmamux_ReqGenParams ={0}

dmamux_ReqGenParams.SignalID  = HAL_DMAMUX2_REQ_GEN_LPTIM2_OUT;   /* 请求触发器选择 LPTIM2_OUT */
dmamux_ReqGenParams.Polarity  = HAL_DMAMUX_REQ_GEN_RISING_FALLING; /* LPTIM2 输出的上升沿和下降沿均可触发  */
dmamux_ReqGenParams.RequestNumber = 1;                             /* 触发后，传输进行 1 次 DMA 传输 */
HAL_DMAEx_ConfigMuxRequestGenerator(&DMA_Handle, &dmamux_ReqGenParams); /* 配置 DMAMUX */
HAL_DMAEx_EnableMuxRequestGenerator (&DMA_Handle);                      /* 使能 DMAMUX 请求发生器 */
```

39.4.3 函数 HAL_DMAEx_EnableMuxRequestGenerator

函数原型：

```
HAL_StatusTypeDef HAL_DMAEx_EnableMuxRequestGenerator (DMA_HandleTypeDef *hdma)
{
  /* 检测参数 */
  assert_param(IS_DMA_STREAM_ALL_INSTANCE(hdma->Instance));

  /* 如果 DMA 配置中使用了 DMAMUX 的请求发生器，则将其使能并返回 HAL_OK */
  if((hdma->State != HAL_DMA_STATE_RESET) && (hdma->DMAmuxRequestGen != 0U))
  {

    /* 使能请求发生器 */
    hdma->DMAmuxRequestGen->RGCR |= DMAMUX_RGxCR_GE;
```

```
    return HAL_OK;
  }
  else
  {
    return HAL_ERROR;
  }
}
```

函数描述：

调用函数 HAL_DMAEx_ConfigMuxRequestGenerator 配置了请求发生器后，就可以调用此函数使能请求发生器。跟禁止函数 HAL_DMAEx_DisableMuxRequestGenerator 是一对。

函数参数：

- 第 1 个参数是 DMA_HandleTypeDef 类型结构体指针变量。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
HAL_DMA_MuxRequestGeneratorConfigTypeDef dmamux_ReqGenParams ={0}

dmamux_ReqGenParams.SignalID  = HAL_DMAMUX2_REQ_GEN_LPTIM2_OUT;   /* 请求触发器选择 LPTIM2_OUT */
dmamux_ReqGenParams.Polarity  =HAL_DMAMUX_REQ_GEN_RISING_FALLING; /*LPTIM2 输出的上升沿和下降沿均可触发  */
dmamux_ReqGenParams.RequestNumber = 1;                            /* 触发后，传输进行 1 次 DMA 传输 */
HAL_DMAEx_ConfigMuxRequestGenerator(&DMA_Handle, &dmamux_ReqGenParams); /* 配置 DMAMUX */
HAL_DMAEx_EnableMuxRequestGenerator (&DMA_Handle);                      /* 使能 DMAMUX 请求发生器 */
```

39.4.4函数 HAL_DMAEx_DisableMuxRequestGenerator

函数原型：

```
HAL_StatusTypeDef HAL_DMAEx_DisableMuxRequestGenerator (DMA_HandleTypeDef *hdma)
{
  /* 检查参数 */
  assert_param(IS_DMA_STREAM_ALL_INSTANCE(hdma->Instance));

  /* 如果 DMA 配置中使用了 DMAMUX 的请求发生器，则将其禁止并返回 HAL_OK */
  if((hdma->State != HAL_DMA_STATE_RESET) && (hdma->DMAmuxRequestGen != 0U))
  {

    /* 禁止请求发生器 */
    hdma->DMAmuxRequestGen->RGCR &= ~DMAMUX_RGxCR_GE;

    return HAL_OK;
  }
  else
  {
    return HAL_ERROR;
  }
}
```

函数描述：

此函数用于禁止请求发生器，跟使能函数 HAL_DMAEx_EnableMuxRequestGenerator 是一对，

函数参数：

◆ 第 1 个参数是 DMA_HandleTypeDef 类型结构体指针变量。

使用举例：

此函数跟前面的 HAL_DMAEx_EnableMuxRequestGenerator 是一对，使用时直接调用即可。

39.5 总结

本章节就为大家讲解这么多，DMXMUX 用到的地方比较多，这几个常用的函数要熟练掌握。

第40章 STM32H7 的 BDMA 基础知识和 HAL 库 API

本章节为大家讲解 BDMA（Basic direct memory access controller，基本直接存储器访问控制器），相比通用的 DMA1 和 DMA2，BDMA 功能稍弱，支持一些基本的 DMA 功能。

40.1 初学者重要提示
40.2 BDMA 基础知识
40.3 BDMA 的 HAL 库用法
40.4 源文件 stm32h7xx_hal_dma.c
40.5 总结

40.1 初学者重要提示

- BDMA 只能操作 D3 域的存储器和外设，这点比较重要，操作的时候容易被遗忘。详情看本章 2.6 小节。
- BDMA 支持 8 路通道。虽然是 8 路，但这 8 路不是并行工作的，而是由 BDMA 的仲裁器决定当前处理哪一路。
- BDMA 不支持硬件 FIFO，但是支持双缓冲。
- BDMA 不支持突发模式。
- BDMA 最大传输次数 65535 次，每次传输单位可以是字节、半字和字。
- BDMA 的循环模式不可用于存储器到存储器模式。
- HAL 库没有配套 BDMA 的双缓冲，因为当前的 HAL 库 V1.3.0 版本没有对双缓冲进行支持，详情看此贴：http://www.armbbs.cn/forum.php?mod=viewthread&tid=91149 。

40.2 BDMA 基础知识

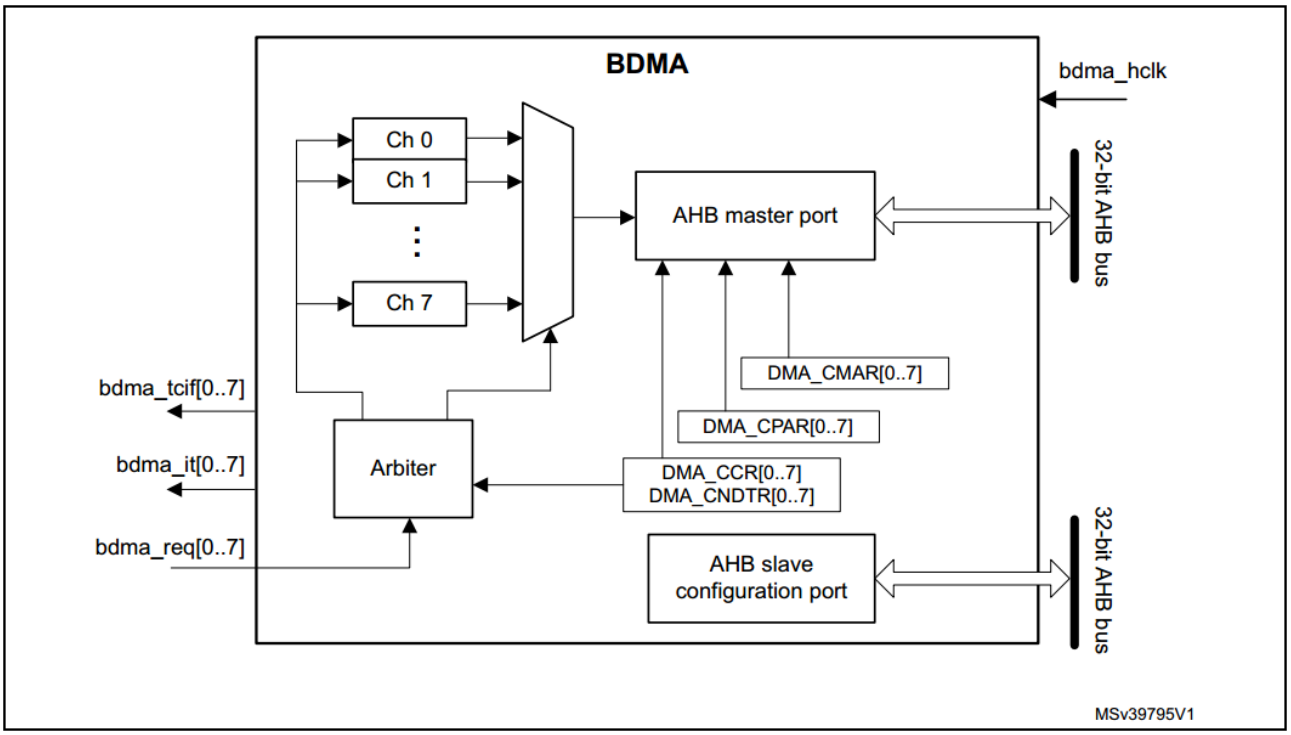
BDMA 的几个关键知识点放在开头说：

- 由于总线矩阵的存在，各个主控的道路四通八达，从而可以让 DMA 和 CPU 同时开工，但是注意一点，如果他们同时访问的同一个外设，会有一点性能影响的。
- BDMA 支持存储器到外设，外设到存储器，存储器到存储器和外设到外设的传输，其中外设到外设的传输，DMA1 和 DMA2 是不支持的，这个模式在低功耗模式下比较有用。

- BDMA 只有一个 AHB 总线主控，而 DMA1 和 DMA2 是有两个的，可以分别用于源地址和目的地址的传输。
- 源地址和目的地址的数据宽度可以不同，但是数据地址必须要跟其数据类型对齐。比如源地址是 uint32 类型的，那么此数组的地址必须 4 字节对齐。
- BDMA 主要有两种模式，一个是 Normal 正常模式，传输一次后就停止传输；另一种是 Circular 循环模式，会一直循环的传输下去，即使有 DMA 中断，传输也是一直在进行的。
- BDMA 的通道请求（Channel0 – Channel7）的优先级可编程，分为四级 Very high priority，High priority，Medium priority 和 Low priority。通道的优先级配置相同的情况下，如果同时产生请求，会优先响应编号低的，即 Channel0 优先响应。

40.2.1 BDMA 硬件框图

认识一个外设，最好的方式就是看他的框图，方便我们快速的了解 BDMA 的基本功能，然后再看手册了解细节。框图如下所示：



（注：ST 做的 DMA 框图没有其它外设做的好，不够详细）

通过这个框图，我们可以得到如下信息：

- bdma_tcif[0:7]接口

  通道 0 – 通道 7 的传输完成标志。

- bdma_it[0:7]接口

  通道 0 – 通道 7 的中断触发。

- bdma_req[0:7]接口

  通道 0 –通道 7 的请求信号接口。

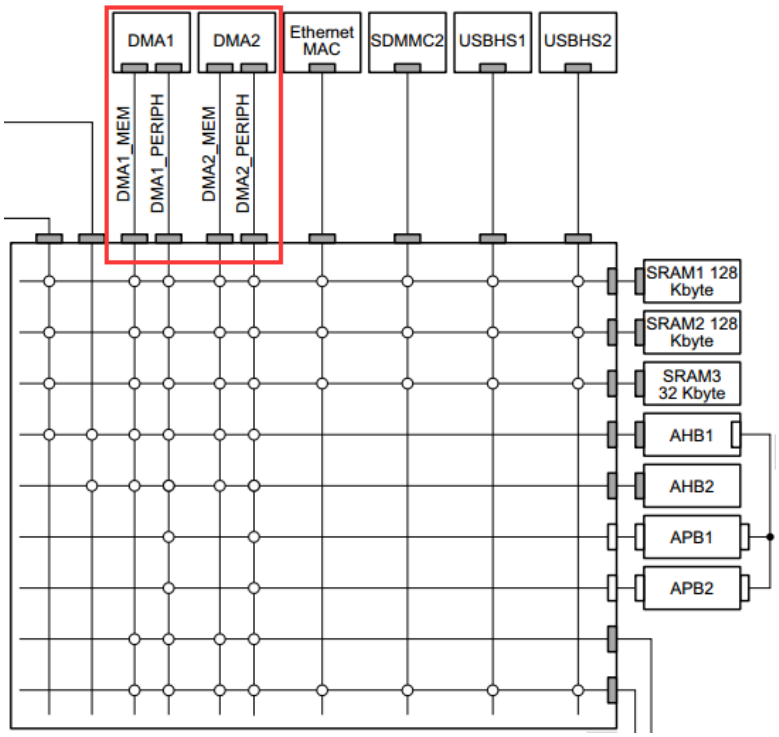
- Arbiter 仲裁器

  用于仲裁当期要处理的 DMA 请求。通过这里我们可以看出虽然是 8 路，但这 8 路不是并行工作的，而是由 BDMA 的仲裁器决定当前处理哪一路。

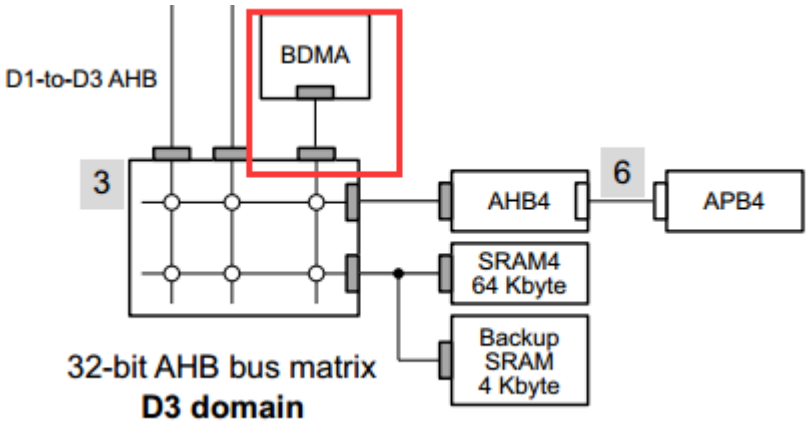
- AHB 总线接口

  BDMA 只有一个 AHB 总线主控，而 DMA1 和 DMA2 是有两个的，可以分别用于源地址和目的地址的传输。

  DMA1 和 DMA2:



BDMA:



40.2.2 BDMA 传输

BDMA 支持如下几种传输模式：

- 存储器到外设。
- 外设到存储器。
- 存储器到存储器。
- 外设到外设的传输。

其中外设到外设的传输，DMA1 和 DMA2 是不支持的，这个模式在低功耗模式下比较有用。

关于这几种传输方式要注意以下两个问题：

- 源地址和目的地址的数据宽度可以不同，但是数据地址必须要跟其数据类型对齐。比如源地址是 uint32 类型的，那么此数组的地址必须 4 字节对齐。
- BDMA 不可以操作 TCM，AXI SRAM，SRAM1，SRAM2，SRAM3，仅可以操作 SRAM4，在第 25 章专门讲解过这个问题。

- 拓展知识

  MDK 中全局变量的数据对齐问题说明：

  http://www.armbbs.cn/forum.php?mod=viewthread&tid=13511 。

40.2.3 BDMA 的循环模式和正常模式

BDMA 主要有两种模式，一个是 Normal 正常模式，传输一次后就停止传输；另一种是 Circular 循环模式，会一直循环的传输下去，即使有 DMA 中断，传输也是一直在进行的。

这两种模式各有用途。

- Normal 正常模式

  适合用于单次传输，比如存储器到存储器的数据复制粘贴，又比如串口的数据单次发送，下次还需要发送的时候，使能下即可。

- Circular 循环模式

  适合用于需要连续传输的场合，比如定时器触发 BDMA 实现任意 IO 的 PWM 输出。

另外特别注意，循环模式不可用于存储器到存储器模式。

40.2.4 BDMA 数据封装和解封

独立的源和目标传输宽度（字节、半字、字）：源和目标的数据宽度不相等时， DMA 自动封装/解封必要的传输数据来优化带宽。无需像 F1 系列那样强行要求数据缓冲的 4 字节对齐。下面是各种源地址和目的地址数据宽度传输 4 次的效果，可以帮助大家更好的理解。

源端口宽度	目标端口宽度	要传输的数据项的数目(NDT)	源内容：地址或/数据	传输操作	目标内容：地址/数据
8	8	4	@0x0/B0 @0x1/B1 @0x2/B2 @0x3/B3	1：读取 B0[7:0] @0x0，然后写入 B0[7:0] @0x0 2：读取 B1[7:0] @0x1，然后写入 B1[7:0] @0x1 3：读取 B2[7:0] @0x2，然后写入 B2[7:0] @0x2 4：读取 B3[7:0] @0x3，然后写入 B3[7:0] @0x3	@0x0/B0 @0x1/B1 @0x2/B2 @0x3/B3
8	16	4	@0x0/B0 @0x1/B1 @0x2/B2 @0x3/B3	1：读取 B0[7:0] @0x0，然后写入 00B0[15:0] @0x0 2：读取 B1[7:0] @0x1，然后写入 00B1[15:0] @0x2 3：读取 B2[7:0] @0x2，然后写入 00B2[15:0] @0x4 4：读取 B3[7:0] @0x3，然后写入 00B3[15:0] @0x6	@0x0/00B0 @0x2/00B1 @0x4/00B2 @0x6/00B3
8	32	4	@0x0/B0 @0x1/B1 @0x2/B2 @0x3/B3	1：读取 B0[7:0] @0x0，然后写入 000000B0[31:0] @0x0 2：读取 B1[7:0] @0x1，然后写入 000000B1[31:0] @0x4 3：读取 B2[7:0] @0x2，然后写入 000000B2[31:0] @0x8 4：读取 B3[7:0] @0x3，然后写入 000000B3[31:0] @0xC	@0x0/000000B0 @0x4/000000B1 @0x8/000000B2 @0xC/000000B3
16	8	4	@0x0/B1B0 @0x2/B3B2 @0x4/B5B4 @0x6/B7B6	1：读取 B1B0[15:0] @0x0，然后写入 B0[7:0] @0x0 2：读取 B3B2[15:0] @0x2，然后写入 B2[7:0] @0x1 3：读取 B5B4[15:0] @0x4，然后写入 B4[7:0] @0x2 4：读取 B7B6[15:0] @0x6，然后写入 B6[7:0] @0x3	@0x0/B0 @0x1/B2 @0x2/B4 @0x3/B6
16	16	4	@0x0/B1B0 @0x2/B3B2 @0x4/B5B4 @0x6/B7B6	1：读取 B1B0[15:0] @0x0，然后写入 B1B0[15:0] @0x0 2：读取 B3B2[15:0] @0x2，然后写入 B3B2[15:0] @0x2 3：读取 B5B4[15:0] @0x4，然后写入 B5B4[15:0] @0x4 4：读取 B7B6[15:0] @0x6，然后写入 B7B6[15:0] @0x6	@0x0/B1B0 @0x2/B3B2 @0x4/B5B4 @0x6/B7B6
16	32	4	@0x0/B1B0 @0x2/B3B2 @0x4/B5B4 @0x6/B7B6	1：读取 B1B0[15:0] @0x0，然后写入 0000B1B0[31:0] @0x0 2：读取 B3B2[15:0] @0x2，然后写入 0000B3B2[31:0] @0x4 3：读取 B5B4[15:0] @0x4，然后写入 0000B5B4[31:0] @0x8 4：读取 B7B6[15:0] @0x6，然后写入 0000B7B6[31:0] @0xC	@0x0/0000B1B0 @0x4/0000B3B2 @0x8/0000B5B4 @0xC/0000B7B6
32	8	4	@0x0/B3B2B1B0 @0x4/B7B6B5B4 @0x8/BBBAB9B8 @0xC/BFBEBDBC	1：读取 B3B2B1B0[31:0] @0x0，然后写入 B0[7:0] @0x0 2：读取 B7B6B5B4[31:0] @0x4，然后写入 B4[7:0] @0x1 3：读取 BBBAB9B8[31:0] @0x8，然后写入 B8[7:0] @0x2 4：读取 BFBEBDBC[31:0] @0xC，然后写入 BC[7:0] @0x3	@0x0/B0 @0x1/B4 @0x2/B8 @0x3/BC
32	16	4	@0x0/B3B2B1B0 @0x4/B7B6B5B4 @0x8/BBBAB9B8 @0xC/BFBEBDBC	1：读取 B3B2B1B0[31:0] @0x0，然后写入 B1B0[15:0] @0x0 2：读取 B7B6B5B4[31:0] @0x4，然后写入 B5B4[15:0] @0x2 3：读取 BBBAB9B8[31:0] @0x8，然后写入 B9B8[15:0] @0x4 4：读取 BFBEBDBC[31:0] @0xC，然后写入 BDBC[15:0] @0x6	@0x0/B1B0 @0x2/B5B4 @0x4/B9B8 @0x6/BDBC
32	32	4	@0x0/B3B2B1B0 @0x4/B7B6B5B4 @0x8/BBBAB9B8 @0xC/BFBEBDBC	1：读取 B3B2B1B0[31:0] @0x0，然后写入 B3B2B1B0[31:0] @0x0 2：读取 B7B6B5B4[31:0] @0x4，然后写入 B7B6B5B4[31:0] @0x4 3：读取 BBBAB9B8[31:0] @0x8，然后写入 BBBAB9B8[31:0] @0x8 4：读取 BFBEBDBC[31:0] @0xC，然后写入 BFBEBDBC[31:0] @0xC	@0x0/B3B2B1B0 @0x4/B7B6B5B4 @0x8/BBBAB9B8 @0xC/BFBEBDBC

40.2.5 BDMA 双缓冲

BDMA 也是支持双缓冲模式的，双缓冲的含义是源地址或者目的地址可以设置两个缓冲区，这种方式的好处是一个缓冲区在接收或者发送数据的时候，另一个缓冲区可以动态更新数据或者处理已经接收到的数据。

当用户开启了 BDMA 传输完成中断后，通过寄存器 CCRx 的 CT 位判断当前使用的是哪个缓冲区

- 如果 CT = 1 表示当前正在使用缓冲区 1，即寄存器 BDMA_CM1ARx 记录的地址。

◆ 如果 CT = 0 表示当前正在使用缓冲区 0，即寄存器 BDMA_CM0ARx 记录的地址。

另外注意，存储器到存储器的 BDMA 传输不支持双缓冲模式，仅可以用于存储器到外设或者外设到存储器。

40.2.6 BDMA 可以操作的区域

根据第 3 章的总线互联方式，BDMA 仅可以操作：AHB4，APB4 的外设以及 SRAM4，Backup RAM。实际应用的时候要特别注意，防止操作错误。

	Bus master / type(1)																	
	Cortex-M7 - AXIM	Cortex-M7 - AHBP	**Cortex-M7 - ITCM**	**Cortex-M7 -DTCM**	SDMMC1	**MDMA - AXI**	MDMA - AHBS	**DMA2D**	**LTDC**	DMA1 - MEM	DMA1 - PERIPH	DMA2 - MEM	DMA2 - PERIPH	Eth. MAC - AHB	SDMMC2 - AHB	USBHS1 - AHB	USBHS2 - AHB	BDMA - AHB
Bus slave / type(1)	Interconnect path and type(2)																	
ITCM	-	-	x	-	-	-	x	-	-	-	-	-	-	-	-	-	-	-
DTCM	-	-	-	x	-	-	x	-	-	-	-	-	-	-	-	-	-	-
AHB3 peripherals	x	-	-	-	-	x	-	-	-	x	x	x	x	x	x	x	x	-
APB3 peripherals	x	-	-	-	-	x	-	-	-	x	x	x	x	x	x	x	x	-
Flash bank 1	x	-	-	-	x	x	-	x	x	x	x	x	x	x	x	x	x	-
Flash bank 2(3)	x	-	-	-	x	x	-	x	x	x	x	x	x	x	x	x	x	-
AXI SRAM	x	-	-	-	x	x	-	x	x	x	x	x	x	x	x	x	x	-
QUADSPI	x	-	-	-	x	x	-	x	x	x	x	x	x	x	x	x	x	-
FMC	x	-	-	-	x	x	-	x	x	x	x	x	x	x	x	x	x	-
SRAM 1	x	-	-	-	-	x	-	x	-	x	x	x	x	x	x	x	x	-
SRAM 2	x	-	-	-	-	x	-	x	-	x	x	x	x	x	x	x	x	-
SRAM 3	x	-	-	-	-	x	-	x	-	x	x	x	x	x	x	x	x	-
AHB1 peripherals	-	x	-	-	-	x	-	x	-	x	x	x	x	-	-	-	-	-
APB1 peripherals	-	x	-	-	-	x	-	x	-	x	x	x	x	-	-	-	-	-
AHB2 peripherals	-	x	-	-	-	-	-	-	-	x	x	x	x	-	-	-	-	-
APB2 peripherals	-	x	-	-	-	x	-	x	-	x	x	x	x	-	-	-	-	-
AHB4 peripherals	x	-	-	-	-	x	-	-	-	x	x	x	x	x	x	x	x	x
APB4 peripherals	x	-	-	-	-	x	-	-	-	x	x	x	x	x	x	x	x	x
SRAM4	x	-	-	-	-	x	-	-	-	x	x	x	x	x	x	x	x	x
Backup RAM	x	-	-	-	-	x	-	-	-	x	x	x	x	x	x	x	x	x

40.3 BDMA 的 HAL 库用法

BDMA 的 HAL 库用法其实就是几个结构体变量成员的配置和使用，然后配置 GPIO、时钟，并根据需要配置 NVIC、中断和 DMA。下面我们逐一展开为大家做个说明。

40.3.1 BDMA 寄存器结构体

BDMA 相关的寄存器是通过 HAL 库中的结构体 DMA_TypeDef 和 DMA_Stream_TypeDef 定义的，在 stm32h743xx.h 中可以找到这个类型定义：

注：当前的 HAL 库版本 V1.3.0 对 CMAR 寄存器的定义不完善，详情看此贴：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=91207 。

```
typedef struct
{
  __IO uint32_t ISR;    /*!< DMA interrupt status register,       Address offset: 0x00 */
  __IO uint32_t IFCR;   /*!< DMA interrupt flag clear register,   Address offset: 0x04 */
} BDMA_TypeDef;

typedef struct
{
  __IO uint32_t CCR; /*!< DMA channel x configuration register  Addr offset: 0x08 + 0x14 * x,x = 0 to 7 */
  __IO uint32_t CNDTR;/*!< DMA channel x number of data register Addr offset: 0x0C + 0x14 * x,x = 0 to 7 */
  __IO uint32_t CPAR; /*!< DMA channel x peripheral address register Addr offset: 0x10 + 0x14 * x, x = 0 to 7*/
  __IO uint32_t CMAR; /*!< DMA channel x memory address register, Addr offset: 0x14 + 0x14 * x, x = 0 to 7 */
} BDMA_Channel_TypeDef;
```

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

与其它外设的的定义方式不同，BDMA 有 8 组通道，每个通道都有一组 BDMA_Channel_TypeDef 结构体所定义的寄存器。

解决这个问题的办法就是定义一套 BDMA_Channel0 - BDMA_Channel7 来解决，定义在 stm32h743xx.h 文件。

```
#define PERIPH_BASE          ((uint32_t)0x40000000)
#define D3_AHB1PERIPH_BASE   (PERIPH_BASE + 0x18020000)
#define BDMA_BASE            (D3_AHB1PERIPH_BASE + 0x5400)
#define BDMA                 ((BDMA_TypeDef *) BDMA_BASE)

#define BDMA_Channel0_BASE    (BDMA_BASE + 0x0008)
#define BDMA_Channel1_BASE    (BDMA_BASE + 0x001C)
#define BDMA_Channel2_BASE    (BDMA_BASE + 0x0030)
#define BDMA_Channel3_BASE    (BDMA_BASE + 0x0044)
#define BDMA_Channel4_BASE    (BDMA_BASE + 0x0058)
#define BDMA_Channel5_BASE    (BDMA_BASE + 0x006C)
#define BDMA_Channel6_BASE    (BDMA_BASE + 0x0080)
#define BDMA_Channel7_BASE    (BDMA_BASE + 0x0094)
```

```
<-----展开下面的宏定义，(BDMA_Channel_TypeDef *) 0x58025408
#define BDMA_Channel0  ((BDMA_Channel_TypeDef *) BDMA_Channel0_BASE)
#define BDMA_Channel1  ((BDMA_Channel_TypeDef *) BDMA_Channel1_BASE)
#define BDMA_Channel2  ((BDMA_Channel_TypeDef *) BDMA_Channel2_BASE)
#define BDMA_Channel3  ((BDMA_Channel_TypeDef *) BDMA_Channel3_BASE)
#define BDMA_Channel4  ((BDMA_Channel_TypeDef *) BDMA_Channel4_BASE)
#define BDMA_Channel5  ((BDMA_Channel_TypeDef *) BDMA_Channel5_BASE)
#define BDMA_Channel6  ((BDMA_Channel_TypeDef *) BDMA_Channel6_BASE)
#define BDMA_Channel7  ((BDMA_Channel_TypeDef *) BDMA_Channel7_BASE)
```

我们访问 BDMA 的 ISR 寄存器可以采用这种形式：BDMA->ISR = 0，而访问通道 0 的 CCR 就可以采用这种形式 BDMA_Channel0->CCR = 0。

40.3.2 BDMA 句柄结构体 DMA_HandleTypeDef

HAL 库在 DMA_TypeDef 的基础上封装了一个结构体 DMA_HandleTypeDef，定义如下：

```
typedef struct __DMA_HandleTypeDef
{
  void                          *Instance;
  DMA_InitTypeDef               Init;
  HAL_LockTypeDef               Lock;
  __IO HAL_DMA_StateTypeDef     State;
  void                          *Parent;
  void                          (* XferCpltCallback)( struct __DMA_HandleTypeDef * hdma);
  void                          (* XferHalfCpltCallback)( struct __DMA_HandleTypeDef * hdma);
  void                          (* XferM1CpltCallback)( struct __DMA_HandleTypeDef * hdma);
  void                          (* XferM1HalfCpltCallback)( struct __DMA_HandleTypeDef * hdma);
  void                          (* XferErrorCallback)( struct __DMA_HandleTypeDef * hdma);
  void                          (* XferAbortCallback)( struct __DMA_HandleTypeDef * hdma);
 __IO uint32_t                  ErrorCode;
 uint32_t                       StreamBaseAddress;
 uint32_t                       StreamIndex;
 DMAMUX_Channel_TypeDef         *DMAmuxChannel;
 DMAMUX_ChannelStatus_TypeDef   *DMAmuxChannelStatus;
 uint32_t                       DMAmuxChannelStatusMask;
 DMAMUX_RequestGen_TypeDef      *DMAmuxRequestGen;
 DMAMUX_RequestGenStatus_TypeDef *DMAmuxRequestGenStatus;
 uint32_t                       DMAmuxRequestGenStatusMask;
}DMA_HandleTypeDef;
```

这里重点介绍前几个参数，其它参数主要是 HAL 库内部使用的。

- **void *Instance**

  用于 BDMA，DMA1 和 DMA2 的例化，主要是相关寄存器的操作。

  因为 DMA1，DMA2 和 BDMA 都使用的这个结构体句柄，而 DMA1，DMA2 与 BDMA 的寄存器结构体封装是不同的，这里的定义比较巧妙， 定义为 void *空类型后，就可以直接使用 DMA1，DMA2 和 BDMA 的结构体定义了。

  比如操作 DMA1 Stream1 的寄存器 CR：

  DMA_HandleTypeDef DMA_Handle;

  DMA_Handle.Instance = DMA1_Stream1;

((DMA_Stream_TypeDef *) DMA_Handle ->Instance)->CR =0;

又比如操作 BDMA Channel1 的寄存器 CCR：

DMA_HandleTypeDef BDMA_Handle;

BDMA_Handle.Instance = BDMA_Channel1;

((BDMA_Channel_TypeDef *) DMA_Handle ->Instance)->CCR =0;

◆ **DMA_InitTypeDef Init;**

这个参数是用户接触最多的，用于配置 BDMA 的基本参数。

DMA_InitTypeDef 结构体的定义如下：

```
typedef struct
{
  uint32_t Request;
  uint32_t Direction;
  uint32_t PeriphInc;
  uint32_t MemInc;
  uint32_t PeriphDataAlignment;
  uint32_t MemDataAlignment;
  uint32_t Mode;
  uint32_t Priority;
  uint32_t FIFOMode;
  uint32_t FIFOThreshold;
  uint32_t MemBurst;
  uint32_t PeriphBurst;
}DMA_InitTypeDef;
```

- 成员 Request

用于设置支持的 DMA 请求，对于 BDAM 来说，主要来自 DMAMUX2。

```
/* D3 Domain : DMAMUX2 requests */
#define BDMA_REQUEST_MEM2MEM          0U  /*!< memory to memory transfer   */
#define BDMA_REQUEST_GENERATOR0       1U  /*!< DMAMUX2 request generator 0 */
#define BDMA_REQUEST_GENERATOR1       2U  /*!< DMAMUX2 request generator 1 */
#define BDMA_REQUEST_GENERATOR2       3U  /*!< DMAMUX2 request generator 2 */
#define BDMA_REQUEST_GENERATOR3       4U  /*!< DMAMUX2 request generator 3 */
#define BDMA_REQUEST_GENERATOR4       5U  /*!< DMAMUX2 request generator 4 */
#define BDMA_REQUEST_GENERATOR5       6U  /*!< DMAMUX2 request generator 5 */
#define BDMA_REQUEST_GENERATOR6       7U  /*!< DMAMUX2 request generator 6 */
#define BDMA_REQUEST_GENERATOR7       8U  /*!< DMAMUX2 request generator 7 */
#define BDMA_REQUEST_LPUART1_RX       9U  /*!< DMAMUX2 LP_UART1_RX request */
#define BDMA_REQUEST_LPUART1_TX      10U  /*!< DMAMUX2 LP_UART1_TX request */
#define BDMA_REQUEST_SPI6_RX         11U  /*!< DMAMUX2 SPI6 RX request     */
#define BDMA_REQUEST_SPI6_TX         12U  /*!< DMAMUX2 SPI6 TX request     */
#define BDMA_REQUEST_I2C4_RX         13U  /*!< DMAMUX2 I2C4 RX request     */
#define BDMA_REQUEST_I2C4_TX         14U  /*!< DMAMUX2 I2C4 TX request     */
#define BDMA_REQUEST_SAI4_A          15U  /*!< DMAMUX2 SAI4 A request      */
#define BDMA_REQUEST_SAI4_B          16U  /*!< DMAMUX2 SAI4 B request      */
#define BDMA_REQUEST_ADC3            17U  /*!< DMAMUX2 ADC3 request        */
```

- 成员 Direction

用于设置传输方向，外设到存储器、存储器到外设或者存储器到存储器，具体支持的参数如下：

```
#define DMA_PERIPH_TO_MEMORY         ((uint32_t)0x00000000U)     /*!< Peripheral to memory direction */
#define DMA_MEMORY_TO_PERIPH         ((uint32_t)DMA_SxCR_DIR_0)  /*!< Memory to peripheral direction */
#define DMA_MEMORY_TO_MEMORY         ((uint32_t)DMA_SxCR_DIR_1)  /*!< Memory to memory direction     */
```

- 成员 PeriphInc

  用于设置外设地址是否使能递增，即每完成一次传输，外设地址自增，增加的大小由参数 PeriphDataAlignment 决定。具体支持的参数如下：

```
#define DMA_PINC_ENABLE         ((uint32_t)DMA_SxCR_PINC)  /*!< Peripheral increment mode enable  */
#define DMA_PINC_DISABLE        ((uint32_t)0x00000000U)    /*!< Peripheral increment mode disable */
```

- 成员 MemInc

  用于设置存储器地址是否使能递增，即每完成一次传输，存储器地址自增，增加的大小由参数 MemDataAlignment 决定。具体支持的参数如下：

```
#define DMA_MINC_ENABLE         ((uint32_t)DMA_SxCR_MINC)  /*!< Memory increment mode enable  */
#define DMA_MINC_DISABLE        ((uint32_t)0x00000000U)    /*!< Memory increment mode disable */
```

- 成员 PeriphDataAlignment

  用于设置外设支持的数据宽度，可以选择字节、半字和字进行传输。

```
#define DMA_PDATAALIGN_BYTE     ((uint32_t)0x00000000U)      /*!< Peripheral data alignment: Byte     */
#define DMA_PDATAALIGN_HALFWORD (uint32_t)DMA_SxCR_PSIZE_0)  /*!< Peripheral data alignment: HalfWord */
#define DMA_PDATAALIGN_WORD     ((uint32_t)DMA_SxCR_PSIZE_1) /*!< Peripheral data alignment: Word     */
```

- 成员 MemDataAlignment

  用于设置存储器支持的数据宽度，可以选择字节、半字和字进行传输。

```
#define DMA_MDATAALIGN_BYTE         ((uint32_t)0x00000000U)       /*!< Memory data alignment: Byte     */
#define DMA_MDATAALIGN_HALFWORD     ((uint32_t)DMA_SxCR_MSIZE_0)  /*!< Memory data alignment: HalfWord */
#define DMA_MDATAALIGN_WORD         ((uint32_t)DMA_SxCR_MSIZE_1)  /*!< Memory data alignment: Word     */
```

- 成员 Mode

  用于设置正常模式、循环模式和流控制，对于 BDMA 而言，仅支持正常模式和循环模式。

```
#define DMA_NORMAL          ((uint32_t)0x00000000U)       /*!< Normal mode                  */
#define DMA_CIRCULAR        ((uint32_t)DMA_SxCR_CIRC)    /*!< Circular mode               */
#define DMA_PFCTRL          ((uint32_t)DMA_SxCR_PFCTRL)  /*!< Peripheral flow control mode */
```

- 成员 Priority

  用于 BDMA 通道进行传输时的优先级设置，控制多通道同时请求时优先响应谁。支持四种优先级设置。

```
#define DMA_PRIORITY_LOW            ((uint32_t)0x00000000U)    /*!< Priority level: Low       */
#define DMA_PRIORITY_MEDIUM         ((uint32_t)DMA_SxCR_PL_0)  /*!< Priority level: Medium    */
#define DMA_PRIORITY_HIGH           ((uint32_t)DMA_SxCR_PL_1)  /*!< Priority level: High      */
#define DMA_PRIORITY_VERY_HIGH      ((uint32_t)DMA_SxCR_PL)    /*!< Priority level: Very High */
```

- 成员 FIFOMode

  BDMA 不支持 FIFO。

- 成员 FIFOThreshold

  FIFO 阀值设置，BDMA 不支持此参数。

- 成员 MemBurst

  存储器突发配置，BDMA 不支持此参数。

- 成员 PeriphBurst

  外设突发配置，BDMA 不支持此参数。

- **HAL_LockTypeDef Lock**

  __IO HAL_DMA_StateTypeDef State

  这两个变量主要供函数内部使用。Lock 用于设置锁状态，而 State 用于设置 DMA 状态。

- **void (* XferCpltCallback)(struct __DMA_HandleTypeDef * hdma);**

  void (* XferHalfCpltCallback)(struct __DMA_HandleTypeDef * hdma);

  void (* XferM1CpltCallback)(struct __DMA_HandleTypeDef * hdma);

  void (* XferM1HalfCpltCallback)(struct __DMA_HandleTypeDef * hdma);

  void (* XferErrorCallback)(struct __DMA_HandleTypeDef * hdma);

  void (* XferAbortCallback)(struct __DMA_HandleTypeDef * hdma);

这里是定义了六个回调函数指针，分别用于配置传输完成回调，半传输完成回调，Memory1 传输完成回调，Memory1 半传输完成回调，传输错误回调和传输终止回调。

40.3.3 BDMA 的状态标志清除问题

下面我们介绍__HAL_DMA_GET_FLAG 函数。这个函数用来检查 BDMA 标志位是否被设置。

```
/**
  * @brief  Get the DMA Stream pending flags.
  * @param  __HANDLE__: DMA handle
  * @param  __FLAG__: Get the specified flag.
  *          This parameter can be any combination of the following values:
  *            @arg DMA_FLAG_TCIFx: Transfer complete flag.
  *            @arg DMA_FLAG_HTIFx: Half transfer complete flag.
  *            @arg DMA_FLAG_TEIFx: Transfer error flag.
  *            @arg DMA_FLAG_DMEIFx: Direct mode error flag.
  *            @arg DMA_FLAG_FEIFx: FIFO error flag.
  *         Where x can be 0_4, 1_5, 2_6 or 3_7 to select the DMA Stream flag.
  * @retval The state of FLAG (SET or RESET).
  */
#define __HAL_DMA_GET_FLAG(__HANDLE__, __FLAG__)\
(((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA2_Stream7)? (BDMA->ISR & (__FLAG__))  :\
 ((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA2_Stream3)? (DMA2->HISR & (__FLAG__)) :\
 ((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA1_Stream7)? (DMA2->LISR & (__FLAG__)) :\
 ((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA1_Stream3)? (DMA1->HISR & (__FLAG__)) : (DMA1->LISR &
                                                                                            (__FLAG__)))
```

对于 BDMA，主要是前三个中断标志。

- DMA_FLAG_TCIFx

  传输完成标志。

- DMA_FLAG_HTIFx

  半传输完成标志。

- DMA_FLAG_TEIFx

  传输错误标志。

- DMA_FLAG_DMEIFx

直接模式错误标志。

- DMA_FLAG_FEIFx

  FIFO 错误标志。

BDMA 支持的标志参数如下：

```
/** @defgroup BDMA_flag_definitions BDMA flag definitions
  * @brief    BDMA flag definitions
  * @{
  */
#define BDMA_FLAG_GL0                    ((uint32_t)0x00000001)
#define BDMA_FLAG_TC0                    ((uint32_t)0x00000002)
#define BDMA_FLAG_HT0                    ((uint32_t)0x00000004)
#define BDMA_FLAG_TE0                    ((uint32_t)0x00000008)
#define BDMA_FLAG_GL1                    ((uint32_t)0x00000010)
#define BDMA_FLAG_TC1                    ((uint32_t)0x00000020)
#define BDMA_FLAG_HT1                    ((uint32_t)0x00000040)
#define BDMA_FLAG_TE1                    ((uint32_t)0x00000080)
#define BDMA_FLAG_GL2                    ((uint32_t)0x00000100)
#define BDMA_FLAG_TC2                    ((uint32_t)0x00000200)
#define BDMA_FLAG_HT2                    ((uint32_t)0x00000400)
#define BDMA_FLAG_TE2                    ((uint32_t)0x00000800)
#define BDMA_FLAG_GL3                    ((uint32_t)0x00001000)
#define BDMA_FLAG_TC3                    ((uint32_t)0x00002000)
#define BDMA_FLAG_HT3                    ((uint32_t)0x00004000)
#define BDMA_FLAG_TE3                    ((uint32_t)0x00008000)
#define BDMA_FLAG_GL4                    ((uint32_t)0x00010000)
#define BDMA_FLAG_TC4                    ((uint32_t)0x00020000)
#define BDMA_FLAG_HT4                    ((uint32_t)0x00040000)
#define BDMA_FLAG_TE4                    ((uint32_t)0x00080000)
#define BDMA_FLAG_GL5                    ((uint32_t)0x00100000)
#define BDMA_FLAG_TC5                    ((uint32_t)0x00200000)
#define BDMA_FLAG_HT5                    ((uint32_t)0x00400000)
#define BDMA_FLAG_TE5                    ((uint32_t)0x00800000)
#define BDMA_FLAG_GL6                    ((uint32_t)0x01000000)
#define BDMA_FLAG_TC6                    ((uint32_t)0x02000000)
#define BDMA_FLAG_HT6                    ((uint32_t)0x04000000)
#define BDMA_FLAG_TE6                    ((uint32_t)0x08000000)
#define BDMA_FLAG_GL7                    ((uint32_t)0x10000000)
#define BDMA_FLAG_TC7                    ((uint32_t)0x20000000)
#define BDMA_FLAG_HT7                    ((uint32_t)0x40000000)
#define BDMA_FLAG_TE7                    ((uint32_t)0x80000000)
```

与标志获取函数__HAL_DMA_GET_FLAG 对应的清除函数是__HAL_DMA_CLEAR_FLAG：

```
/**
  * @brief  Clear the DMA Stream pending flags.
  * @param  __HANDLE__: DMA handle
  * @param  __FLAG__: specifies the flag to clear.
  *          This parameter can be any combination of the following values:
  *            @arg DMA_FLAG_TCIFx: Transfer complete flag.
  *            @arg DMA_FLAG_HTIFx: Half transfer complete flag.
  *            @arg DMA_FLAG_TEIFx: Transfer error flag.
  *            @arg DMA_FLAG_DMEIFx: Direct mode error flag.
  *            @arg DMA_FLAG_FEIFx: FIFO error flag.
  *         Where x can be 0_4, 1_5, 2_6 or 3_7 to select the DMA Stream flag.
  * @retval None
```

```
  */
#define __HAL_DMA_CLEAR_FLAG(__HANDLE__, __FLAG__) \
(((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA2_Stream7)? (BDMA->IFCR = (__FLAG__))  :\
 ((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA2_Stream3)? (DMA2->HIFCR = (__FLAG__)) :\
 ((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA1_Stream7)? (DMA2->LIFCR = (__FLAG__)) :\
 ((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA1_Stream3)? (DMA1->HIFCR = (__FLAG__)) : (DMA1->LIFCR =
(__FLAG__)))
```

清除标志函数所支持的参数跟获取函数是一 一对应的。除了这两个函数，还有 BDMA 的中断开启和中断关闭函数，有时候也要用到。

```
/**
  * @brief  Enable the specified DMA Stream interrupts.
  * @param  __HANDLE__: DMA handle
  * @param  __INTERRUPT__: specifies the DMA interrupt sources to be enabled or disabled.
  *        This parameter can be one of the following values:
  *           @arg DMA_IT_TC: Transfer complete interrupt mask.
  *           @arg DMA_IT_HT: Half transfer complete interrupt mask.
  *           @arg DMA_IT_TE: Transfer error interrupt mask.
  *           @arg DMA_IT_FE: FIFO error interrupt mask.
  *           @arg DMA_IT_DME: Direct mode error interrupt.
  * @retval None
  */
#define __HAL_DMA_ENABLE_IT(__HANDLE__, __INTERRUPT__) ((IS_D2_DMA_INSTANCE(__HANDLE__))?\
                                                        (__HAL_DMA_D2_ENABLE_IT((__HANDLE__),
                                                                                (__INTERRUPT__))) :\
                                                        (__HAL_DMA_D3_ENABLE_IT((__HANDLE__),
                                                                                (__INTERRUPT__))))

/**
  * @brief  Disable the specified DMA Stream interrupts.
  * @param  __HANDLE__: DMA handle
  * @param  __INTERRUPT__: specifies the DMA interrupt sources to be enabled or disabled.
  *         This parameter can be one of the following values:
  *            @arg DMA_IT_TC: Transfer complete interrupt mask.
  *            @arg DMA_IT_HT: Half transfer complete interrupt mask.
  *            @arg DMA_IT_TE: Transfer error interrupt mask.
  *            @arg DMA_IT_FE: FIFO error interrupt mask.
  *            @arg DMA_IT_DME: Direct mode error interrupt.
  * @retval None
  */
#define __HAL_DMA_DISABLE_IT(__HANDLE__, __INTERRUPT__) ((IS_D2_DMA_INSTANCE(__HANDLE__))?\
                                                         (__HAL_DMA_D2_DISABLE_IT((__HANDLE__),
                                                                                 (__INTERRUPT__))) :\
                                                         (__HAL_DMA_D3_DISABLE_IT((__HANDLE__),
                                                                                 (__INTERRUPT__))))
```

注意：操作 DMA 的寄存器不限制必须要用 HAL 库提供的 API，比如要操作寄存器 ISR，直接调用 BDMA->ISR 操作即可。

40.3.4BDMA 初始化流程总结

使用方法由 HAL 库提供：

◆ 第 1 步：通过函数 HAL_DMA_Init 配置各项参数。

◆ 第 2 步：BDMA 查询方式。

- 配置了源地址、目的地址和数据长度后，调用函数 HAL_DMA_Start()启动传输。
- 使用函数 HAL_DMA_PollForTransfer()查询当前传输是否结束，用户还可以给此函数配置超时等待时间。

◆ 第 3 步：BDMA 中断方式。

- 使用函数 HAL_NVIC_SetPriority 配置 BDMA 优先级。
- 使用函数 HAL_NVIC_EnableIRQ 使能 BDMA 中断。
- 配置了源地址、目的地址和数据长度后，调用函数 HAL_DMA_Start_IT()可以启动传输（注，此函数会使能 BDMA 中断）。
- 将函数 HAL_DMA_IRQHandler()填到中断服务程序 BDMA_Channelx_IRQHandler 里面。
- 传输结束后会调用函数 HAL_DMA_IRQHandler()，此函数里面会执行回调函数，即用户需要为 XferCpltCallback，XferErrorCallback 等函数配置实体（如果用到的话）。

◆ 第 4 步：使用函数 HAL_DMA_GetState()可以获得 DMA 状态，函数 HAL_DMA_GetError()获取错误类型。

◆ 第 5 步：使用函数 HAL_DMA_Abort()可以终止 DMA 传输。

- 存储器到存储器方式，不支持循环模式。
- DMA FIFO 的作用是降低对总线的需求和源地址，目的地址不同数据宽度的传输。
- 当 FIFO 禁止后，不允许配置源数据和目的数据宽度不同，此时将统一使用外设数据宽度。

◆ 第 6 步：下面是几个常用的 DMA 宏定义。

- __HAL_DMA_ENABLE: 使能指定的 DMA Stream
- __HAL_DMA_DISABLE: 禁止指定的 DMA Stream
- __HAL_DMA_GET_FS: 返回当前 DMA Stream FIFO 填充情况
- __HAL_DMA_ENABLE_IT: 使能指定的 DMA Stream 中断
- __HAL_DMA_DISABLE_IT: 禁止指定的 DMA Stream 中断
- __HAL_DMA_GET_IT_SOURCE: 检查指定的 DMA Stream 中断是否使能

40.4 源文件 stm32h7xx_hal_dma.c

此文件涉及到的函数比较多，这里把我们几个常用的函数做个说明：

- HAL_DMA_Init
- HAL_DMA_Start
- HAL_DMA_Start_IT

40.4.1 函数 HAL_DMA_Init

函数原型：

```
HAL_StatusTypeDef HAL_DMA_Init(DMA_HandleTypeDef *hdma)
{
  uint32_t registerValue = 0U;
  uint32_t tickstart = HAL_GetTick();
  DMA_Base_Registers *regs = NULL;

   /* 省略 */

  /* DMA1 或者 DMA2 的初始化 */
  if(IS_D2_DMA_INSTANCE(hdma) != RESET)
  {
     /* 省略 */
  }

  /* BDMA 的初始 */
  else if(IS_D3_DMA_INSTANCE(hdma) != RESET) /*<BDMA channel , D3 domain*/
  {
     /* 省略 */
  }
  else
  {
    hdma->ErrorCode = HAL_DMA_ERROR_PARAM;
    hdma->State     = HAL_DMA_STATE_ERROR;

    return HAL_ERROR;
  }

  /* 初始化 DMAMUX */
  DMA_CalcDMAMUXChannelBaseAndMask(hdma);

  if(hdma->Init.Direction == DMA_MEMORY_TO_MEMORY)
  {
     /* 如果是内存到内存模式，强制使用请求 DMA_REQUEST_MEM2MEM */
     hdma->Init.Request = DMA_REQUEST_MEM2MEM;
  }


  /* 设置外设请求 */
  hdma->DMAmuxChannel->CCR = (hdma->Init.Request & DMAMUX_CxCR_DMAREQ_ID);

  /* 清除 DMAMUX 同步溢出标志 */
```

```
  hdma->DMAmuxChannelStatus->CFR = hdma->DMAmuxChannelStatusMask;

  /* 如果请求类型是 DMA_REQUEST_GENERATOR0 到 DMA_REQUEST_GENERATOR7，那么设置请求发生器 */
  if((hdma->Init.Request >= DMA_REQUEST_GENERATOR0) && (hdma->Init.Request <= DMA_REQUEST_GENERATOR7))
  {
    /* 省略 */
  }
  else
  {
    hdma->DMAmuxRequestGen = 0U;
    hdma->DMAmuxRequestGenStatus = 0U;
    hdma->DMAmuxRequestGenStatusMask = 0U;
  }

  /* 无错误 */
  hdma->ErrorCode = HAL_DMA_ERROR_NONE;

  /* 设置 DMA 就绪 */
  hdma->State = HAL_DMA_STATE_READY;

  return HAL_OK;
}
```

函数描述：

此函数用于初始化 DMA1，DMA2 和 BDMA。

函数参数：

- 第 1 个参数是 DMA_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。结构体变量成员的详细介绍看本章 3.2 小节。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 第 1 个参数的结构体成员介绍在本章的 3.2 小节进行了详细说明。

使用举例：

```
DMA_HandleTypeDef DMA_Handle = {0};

DMA_Handle.Instance                 = BDMA_Channel0;            /* 使用的 BDMA 通道 0 */
DMA_Handle.Init.Request             = BDMA_REQUEST_GENERATOR0; /* 请求类型采用的 DMAMUX 请求发生器通道 0 */
DMA_Handle.Init.Direction           = DMA_MEMORY_TO_PERIPH;     /* 传输方向是从存储器到外设 */
DMA_Handle.Init.PeriphInc           = DMA_PINC_DISABLE;         /* 外设地址自增禁止 */
DMA_Handle.Init.MemInc              = DMA_MINC_ENABLE;          /* 存储器地址自增使能 */
DMA_Handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;      /* 外设数据传输位宽选择字，即 32bit */
DMA_Handle.Init.MemDataAlignment    = DMA_MDATAALIGN_WORD;      /* 存储器数据传输位宽选择字，即 32bit */
DMA_Handle.Init.Mode                = DMA_CIRCULAR;             /* 循环模式 */
DMA_Handle.Init.Priority            = DMA_PRIORITY_LOW;         /* 优先级低 */
DMA_Handle.Init.FIFOMode            = DMA_FIFOMODE_DISABLE;     /* BDMA 不支持 FIFO */
DMA_Handle.Init.FIFOThreshold       = DMA_FIFO_THRESHOLD_FULL; /* BDMA 不支持 FIFO 阈值设置 */
DMA_Handle.Init.MemBurst            = DMA_MBURST_SINGLE;        /* BDMA 不支持存储器突发 */
DMA_Handle.Init.PeriphBurst         = DMA_PBURST_SINGLE;        /* BDMA 不支持外设突发 */

HAL_DMA_Init(&DMA_Handle);
```

40.4.2 函数 HAL_DMA_Start

函数原型：

```
HAL_StatusTypeDef HAL_DMA_Start(DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t DataLength)
{
  HAL_StatusTypeDef status = HAL_OK;

  /* 检测参数 */
  assert_param(IS_DMA_BUFFER_SIZE(DataLength));

  /* 检测句柄 */
  if(hdma == NULL)
  {
    return HAL_ERROR;
  }

  /* 上锁 */
  __HAL_LOCK(hdma);

  if(HAL_DMA_STATE_READY == hdma->State)
  {
    /* 设置 DMA 状态 */
    hdma->State = HAL_DMA_STATE_BUSY;

    /* 无错误 */
    hdma->ErrorCode = HAL_DMA_ERROR_NONE;

    /* 禁止 DMA */
    __HAL_DMA_DISABLE(hdma);

    /* 配置源地址，目的地址和数据长度 */
    DMA_SetConfig(hdma, SrcAddress, DstAddress, DataLength);

    /* 使能 DMA */
    __HAL_DMA_ENABLE(hdma);
  }
  else
  {
    /* 解锁 */
    __HAL_UNLOCK(hdma);

    /* 设置忙 */
    hdma->ErrorCode = HAL_DMA_ERROR_BUSY;

    /* 返回 HAL_ERROR */
    status = HAL_ERROR;
  }
  return status;
}
```

函数描述：

调用函数 HAL_DMA_Init 配置了基础功能后，就可以调用此函数启动 BDMA 了。DMA1，DMA2 和 BDMA 都是用的这个函数。

函数参数：

- 第 1 个参数是 DMA_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是 BDMA 传输的源地址。
- 第 3 个参数是 BDMA 传输的目的地址。
- 第 4 个参数是传输的数据长度。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 第 1 个参数的结构体成员介绍在本章的 3.2 小节进行了详细说明。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitTimBDMA
*    功能说明: 配置 DMAMUX 的定时器触+DMA 控制任意 IO 做 PWM 和脉冲数控制
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitTimBDMA(void)
{
    GPIO_InitTypeDef  GPIO_InitStruct;
    DMA_HandleTypeDef DMA_Handle = {0};
    HAL_DMA_MuxRequestGeneratorConfigTypeDef dmamux_ReqGenParams ={0};


     /*##-1- 配置 PB1 用于 PWM 输出#########################################*/
    __HAL_RCC_GPIOB_CLK_ENABLE();

    GPIO_InitStruct.Pin = GPIO_PIN_1;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);


    /*##-2- 配置 DMA ######################################################*/
    __HAL_RCC_BDMA_CLK_ENABLE();

    DMA_Handle.Instance                 = BDMA_Channel0;            /* 使用的 BDMA 通道 0 */
    DMA_Handle.Init.Request             = BDMA_REQUEST_GENERATOR0; /* 请求类型采用的 DMAMUX 请求发生器通道 0
*/
    DMA_Handle.Init.Direction           = DMA_MEMORY_TO_PERIPH;     /* 传输方向是从存储器到外设 */
    DMA_Handle.Init.PeriphInc           = DMA_PINC_DISABLE;         /* 外设地址自增禁止 */
    DMA_Handle.Init.MemInc              = DMA_MINC_ENABLE;          /* 存储器地址自增使能 */
    DMA_Handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;      /* 外设数据传输位宽选择字，即 32bit */
    DMA_Handle.Init.MemDataAlignment    = DMA_MDATAALIGN_WORD;      /* 存储器数据传输位宽选择字，即 32bit */
    DMA_Handle.Init.Mode                = DMA_CIRCULAR;             /* 循环模式 */
    DMA_Handle.Init.Priority            = DMA_PRIORITY_LOW;         /* 优先级低 */
    DMA_Handle.Init.FIFOMode            = DMA_FIFOMODE_DISABLE;     /* BDMA 不支持 FIFO */
    DMA_Handle.Init.FIFOThreshold       = DMA_FIFO_THRESHOLD_FULL; /* BDMA 不支持 FIFO 阈值设置 */
    DMA_Handle.Init.MemBurst            = DMA_MBURST_SINGLE;        /* BDMA 不支持存储器突发 */
```

```
    DMA_Handle.Init.PeriphBurst         = DMA_PBURST_SINGLE;        /* BDMA 不支持外设突发 */

    HAL_DMA_Init(&DMA_Handle);

    /* 开启 BDMA Channel0 的中断 */
    HAL_NVIC_SetPriority(BDMA_Channel0_IRQn, 2, 0);
    HAL_NVIC_EnableIRQ(BDMA_Channel0_IRQn);

    /*##-3- 配置 DMAMUX ######################################################*/
    dmamux_ReqGenParams.SignalID  = HAL_DMAMUX2_REQ_GEN_LPTIM2_OUT;     /* 请求触发器选择 LPTIM2_OUT */
    dmamux_ReqGenParams.Polarity  = HAL_DMAMUX_REQ_GEN_RISING_FALLING; /* LPTIM2 输出的上升沿和下降沿均可触
                                                                          发  */
    dmamux_ReqGenParams.RequestNumber = 1;                              /* 触发后，传输进行 1 次 DMA 传输 */
    HAL_DMAEx_ConfigMuxRequestGenerator(&DMA_Handle, &dmamux_ReqGenParams); /* 配置 DMAMUX */
    HAL_DMAEx_EnableMuxRequestGenerator (&DMA_Handle);                  /* 使能 DMAMUX 请求发生器 */

    /*##-4- 启动 DMA 传输 ####################################################*/
    HAL_DMA_Start(&DMA_Handle, (uint32_t)IO_Toggle, (uint32_t)&GPIOB->BSRRL, 8);
}
```

40.4.3 函数 HAL_DMA_Start_IT

函数原型：

```
HAL_StatusTypeDef HAL_DMA_Start_IT(DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t
DataLength)
{
  HAL_StatusTypeDef status = HAL_OK;

  /* 检测参数 */
  assert_param(IS_DMA_BUFFER_SIZE(DataLength));

  /* 检测句柄 */
  if(hdma == NULL)
  {
    return HAL_ERROR;
  }

  /* 上锁  */
  __HAL_LOCK(hdma);

  if(HAL_DMA_STATE_READY == hdma->State)
  {
    /* 设置 DMA 忙 */
    hdma->State = HAL_DMA_STATE_BUSY;

    /* 设置无错误 */
    hdma->ErrorCode = HAL_DMA_ERROR_NONE;

    /* 禁止 DMA */
    __HAL_DMA_DISABLE(hdma);

    /* 配置 DMA 源地址，目的地址和数据长度 */
    DMA_SetConfig(hdma, SrcAddress, DstAddress, DataLength);

    /* DMA1 和 DMA2 配置 */
```

```
    if(IS_D2_DMA_INSTANCE(hdma) != RESET)
    {
      /* 使能 TC，TE 和 DME 中断 */
      MODIFY_REG(((DMA_Stream_TypeDef   *)hdma->Instance)->CR, (DMA_IT_TC | DMA_IT_TE | DMA_IT_DME |
                                                 DMA_IT_HT), (DMA_IT_TC | DMA_IT_TE | DMA_IT_DME));
      ((DMA_Stream_TypeDef   *)hdma->Instance)->FCR |= DMA_IT_FE;

      if(hdma->XferHalfCpltCallback != NULL)
      {
        /* 如何设置了半传输回调函数，同时开启半传输中断 */
        ((DMA_Stream_TypeDef   *)hdma->Instance)->CR  |= DMA_IT_HT;
      }
    }
    else /* BDMA 配置 */
    {
      /* 使能 TC 和 TC 中断 */
      MODIFY_REG(((BDMA_Channel_TypeDef   *)hdma->Instance)->CCR, (BDMA_CCR_TCIE | BDMA_CCR_HTIE |
                                                   BDMA_CCR_TEIE), (BDMA_CCR_TCIE | BDMA_CCR_TEIE));

      if(hdma->XferHalfCpltCallback != NULL)
      {
         /* 如何设置了半传输回调函数，同时开启半传输中断 */
        ((BDMA_Channel_TypeDef   *)hdma->Instance)->CCR  |= BDMA_CCR_HTIE;
      }
    }

    /* 检测是否使能 DMAMUX 同步传输 */
    if((hdma->DMAmuxChannel->CCR & DMAMUX_CxCR_SE) != 0U)
    {
      /* 使能了的话，开启同步溢出中断 */
      hdma->DMAmuxChannel->CCR |= DMAMUX_CxCR_SOIE;
    }

    if(hdma->DMAmuxRequestGen != 0U)
    {
      /* 如果使用了 DMAMUX 请求发生器，使能请求发生器溢出中断 */
      hdma->DMAmuxRequestGen->RGCR |= DMAMUX_RGxCR_OIE;

    }

    /* 使能 DMA */
    __HAL_DMA_ENABLE(hdma);
  }
  else
  {
    /* 解锁 */
    __HAL_UNLOCK(hdma);

    /* 设置 DMA 忙 */
    hdma->ErrorCode = HAL_DMA_ERROR_BUSY;

    /* 返回 HAL_ERROR */
    status = HAL_ERROR;
  }

  return status;
}
```

函数描述：

调用函数 HAL_DMA_Init 配置了基础功能后，就可以调用此函数启动 BDMA 了，采用的中断方式。DMA1，DMA2 和 BDMA 都是用的这个函数。

函数参数：

- 第 1 个参数是 LPTIM_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是 BDMA 传输的源地址。
- 第 3 个参数是 BDMA 传输的目的地址。
- 第 4 个参数是传输的数据长度。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 第 1 个参数的结构体成员介绍在本章的 3.2 小节进行了详细说明。
2. 对于 DMA1 和 DMA2，这个函数会开启 TC，TE 和 MDE 中断，如果注册了半传输完成回调函数，还会开启半传输中断。
3. 对于 BDMA，这个函数会开始 TC 和 TE 中断，如果注册了半传输完成回调函数，还会开启半传输中断。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitTimBDMA
*    功能说明: 配置 DMAMUX 的定时器触+DMA 控制任意 IO 做 PWM 和脉冲数控制
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitTimBDMA(void)
{
    GPIO_InitTypeDef  GPIO_InitStruct;
    DMA_HandleTypeDef DMA_Handle = {0};
    HAL_DMA_MuxRequestGeneratorConfigTypeDef dmamux_ReqGenParams ={0};


     /*##-1- 配置 PB1 用于 PWM 输出#########################################*/
    __HAL_RCC_GPIOB_CLK_ENABLE();

    GPIO_InitStruct.Pin = GPIO_PIN_1;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);


    /*##-2- 配置 DMA ######################################################*/
    __HAL_RCC_BDMA_CLK_ENABLE();

    DMA_Handle.Instance                 = BDMA_Channel0;            /* 使用的 BDMA 通道 0 */
```

```
    DMA_Handle.Init.Request             = BDMA_REQUEST_GENERATOR0; /* 请求类型采用的 DMAMUX 请求发生器通道 0
*/
    DMA_Handle.Init.Direction           = DMA_MEMORY_TO_PERIPH;    /* 传输方向是从存储器到外设 */
    DMA_Handle.Init.PeriphInc           = DMA_PINC_DISABLE;        /* 外设地址自增禁止 */
    DMA_Handle.Init.MemInc              = DMA_MINC_ENABLE;         /* 存储器地址自增使能 */
    DMA_Handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;     /* 外设数据传输位宽选择字，即 32bit */
    DMA_Handle.Init.MemDataAlignment    = DMA_MDATAALIGN_WORD;     /* 存储器数据传输位宽选择字，即 32bit */
    DMA_Handle.Init.Mode                = DMA_CIRCULAR;            /* 循环模式 */
    DMA_Handle.Init.Priority            = DMA_PRIORITY_LOW;        /* 优先级低 */
    DMA_Handle.Init.FIFOMode            = DMA_FIFOMODE_DISABLE;    /* BDMA 不支持 FIFO */
    DMA_Handle.Init.FIFOThreshold       = DMA_FIFO_THRESHOLD_FULL; /* BDMA 不支持 FIFO 阀值设置 */
    DMA_Handle.Init.MemBurst            = DMA_MBURST_SINGLE;       /* BDMA 不支持存储器突发 */
    DMA_Handle.Init.PeriphBurst         = DMA_PBURST_SINGLE;       /* BDMA 不支持外设突发 */

    HAL_DMA_Init(&DMA_Handle);

    /* 开启 BDMA Channel0 的中断 */
    HAL_NVIC_SetPriority(BDMA_Channel0_IRQn, 2, 0);
    HAL_NVIC_EnableIRQ(BDMA_Channel0_IRQn);

    /*##-3- 配置 DMAMUX ######################################################*/
    dmamux_ReqGenParams.SignalID  = HAL_DMAMUX2_REQ_GEN_LPTIM2_OUT;    /* 请求触发器选择 LPTIM2_OUT */
    dmamux_ReqGenParams.Polarity  = HAL_DMAMUX_REQ_GEN_RISING_FALLING; /* LPTIM2 输出的上升沿和下降沿均可触
                                                                          发 */
    dmamux_ReqGenParams.RequestNumber = 1;                             /* 触发后，传输进行 1 次 DMA 传输 */
    HAL_DMAEx_ConfigMuxRequestGenerator(&DMA_Handle, &dmamux_ReqGenParams); /* 配置 DMAMUX */
    HAL_DMAEx_EnableMuxRequestGenerator (&DMA_Handle);                 /* 使能 DMAMUX 请求发生器 */

    /*##-4- 启动 DMA 传输 ####################################################*/
    HAL_DMA_Start_IT(&DMA_Handle, (uint32_t)IO_Toggle, (uint32_t)&GPIOB->BSRRL, 8);
}
```

40.5 总结

本章节就为大家讲解这么多，BDMA 作为一个重要的外设，务必要熟练掌握。

第41章 STM32H7 的 BDMA 应用之控制任意 IO 做 PWM 和脉冲数控制

本章教程为大家讲解定时器触发 DMAMUX，控制 BDMA 让 GPIO 输出 PWM 以及脉冲数的控制，实际项目中有一定的使用价值。

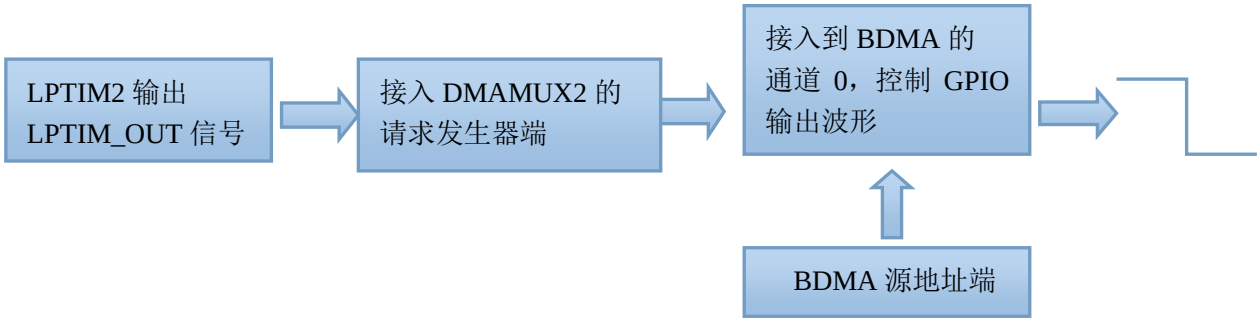
41.1 初学者重要提示
41.2 定时器触发 BDMA 驱动设计
41.3 BDMA 板级支持包（bsp_tim_dma.c）
41.4 BDMA 驱动移植和使用
41.5 实验例程设计框架
41.6 实验例程说明（MDK）
41.7 实验例程说明（IAR）
41.8 总结

41.1 初学者重要提示

- 学习本章节前，务必优先学习第 39 章和 40 章，需要对 DMAMUX，BDMA 的基础知识和 HAL 库的几个常用 API 有个认识。
- 使用半传输完成中断和传输完成中断实现的双缓冲效果跟 BDMA 本身支持的双缓冲模式实现的效果是一样的。只是最大传输个数只能达到 32767 次。
- 相比定时器本身支持的 PWM，这种方式更加灵活，可以让任意 IO 都可以输出 PWM，而且方便运行中动态修改输出状态。

41.2 定时器触发 BDMA 驱动设计

定时器触发 DMAMUX，控制 BDMA 让 GPIO 输出 PWM 的实现思路框图如下：



下面将程序设计中的相关问题逐一为大家做个说明。

41.2.1 定时器选择

使用 BDMA 的话，请求信号都是来自 DMAMUX2，而控制 DMA 做周期性传输的话，可以使用定时器触发，这样的话就可以使用 DMAMUX 的请求发生器功能，支持如下几种触发：

```
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH0_EVT    0U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH1_EVT    1U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH2_EVT    2U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH3_EVT    3U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH4_EVT    4U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH5_EVT    5U
#define HAL_DMAMUX2_REQ_GEN_DMAMUX2_CH6_EVT    6U
#define HAL_DMAMUX2_REQ_GEN_LPUART1_RX_WKUP    7U
#define HAL_DMAMUX2_REQ_GEN_LPUART1_TX_WKUP    8U
#define HAL_DMAMUX2_REQ_GEN_LPTIM2_WKUP        9U
#define HAL_DMAMUX2_REQ_GEN_LPTIM2_OUT         10U
#define HAL_DMAMUX2_REQ_GEN_LPTIM3_WKUP        11U
#define HAL_DMAMUX2_REQ_GEN_LPTIM3_OUT         12U
#define HAL_DMAMUX2_REQ_GEN_LPTIM4_WKUP        13U
#define HAL_DMAMUX2_REQ_GEN_LPTIM5_WKUP        14U
#define HAL_DMAMUX2_REQ_GEN_I2C4_WKUP          15U
#define HAL_DMAMUX2_REQ_GEN_SPI6_WKUP          16U
#define HAL_DMAMUX2_REQ_GEN_COMP1_OUT          17U
#define HAL_DMAMUX2_REQ_GEN_COMP2_OUT          18U
#define HAL_DMAMUX2_REQ_GEN_RTC_WKUP           19U
#define HAL_DMAMUX2_REQ_GEN_EXTI0              20U
#define HAL_DMAMUX2_REQ_GEN_EXTI2              21U
#define HAL_DMAMUX2_REQ_GEN_I2C4_IT_EVT        22U
#define HAL_DMAMUX2_REQ_GEN_SPI6_IT            23U
#define HAL_DMAMUX2_REQ_GEN_LPUART1_TX_IT      24U
#define HAL_DMAMUX2_REQ_GEN_LPUART1_RX_IT      25U
#define HAL_DMAMUX2_REQ_GEN_ADC3_IT            26U
#define HAL_DMAMUX2_REQ_GEN_ADC3_AWD1_OUT      27U
#define HAL_DMAMUX2_REQ_GEN_BDMA_CH0_IT        28U
#define HAL_DMAMUX2_REQ_GEN_BDMA_CH1_IT        29U
```

我们这里使用的是 LPTIM2_OUT，因为 BDMA，LPTIM2 和 GPIO 都在 D3 域。

接下来就是 LPTIM 的时钟配置问题，由前面的 LPTIM 章节，我们知道 LPTIM2 的时钟可以由 LSE，LSI，APB 或者外部输入时钟提供。使用 LSE，LSI 或者外部输入的好处是停机状态下，LPTIM1 也可以正

常工作。

- V7 开发板使用的 LSE 晶振是 32768Hz。
- STM32H743 的 LSI 频率约 32KHz。
- LPTIM1 – LPTIM5 的频率都是 100MHz。

```
System Clock source       = PLL (HSE)
SYSCLK(Hz)                = 400000000 (CPU Clock)
HCLK(Hz)                  = 200000000 (AXI and AHBs Clock)
AHB Prescaler             = 2
D1 APB3 Prescaler         = 2 (APB3 Clock  100MHz)
D2 APB1 Prescaler         = 2 (APB1 Clock  100MHz)
D2 APB2 Prescaler         = 2 (APB2 Clock  100MHz)
D3 APB4 Prescaler         = 2 (APB4 Clock  100MHz)

因为 APB1 prescaler != 1, 所以 APB1 上的 TIMxCLK = APB1 x 2 = 200MHz; 不含这个总线下的 LPTIM1
因为 APB2 prescaler != 1, 所以 APB2 上的 TIMxCLK = APB2 x 2 = 200MHz;

APB4 上面的 TIMxCLK 没有分频，所以就是 100MHz;

APB1 定时器有 TIM2, TIM3 ,TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14，LPTIM1
APB2 定时器有 TIM1, TIM8 , TIM15, TIM16，TIM17

APB4 定时器有 LPTIM2，LPTIM3，LPTIM4，LPTIM5
```

如果选择 APB 时钟的话，配置如下：

```
RCC_PeriphCLKInitTypeDef   RCC_PeriphCLKInitStruct = {0};

RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPTIM2;
RCC_PeriphCLKInitStruct.Lptim2ClockSelection = RCC_LPTIM2CLKSOURCE_D3PCLK1;
HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
```

使用 APB 作为 LPTIM 系统时钟注意以下两点：

- LPTIM1 – LPTIM5 的最高主频都是 100MHz。
- 注意参数 RCC_LPTIM2CLKSOURCE_D3PCLK1。

  LPTIM1 使用的 RCC_LPTIM1CLKSOURCE_D2PCLK1。

  LPTIM2 使用的 RCC_LPTIM2CLKSOURCE_D3PCLK1。

  LPTIM3-LPTIM5 使用的 RCC_LPTIM345CLKSOURCE_D3PCLK1。

LPTIM2 的配置代码如下：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: LPTIM_Config
4.  *    功能说明: 配置 LPTIM，用于触发 DMAMUX 的请求发生器
5.  *    形    参: 无
6.  *    返 回 值: 无
7.  *********************************************************************************************************
8.  */
9.  void LPTIM_Config(void)
10. {
11.
12.     RCC_PeriphCLKInitTypeDef  PeriphClkInitStruct;
13.
```

```
14.
15.     /*##-1- 配置 LPTIM2 使用 PCLK 时钟 ###################################################*/
16.     PeriphClkInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPTIM2;
17.     PeriphClkInitStruct.Lptim2ClockSelection = RCC_LPTIM2CLKSOURCE_D3PCLK1;
18.     HAL_RCCEx_PeriphCLKConfig(&PeriphClkInitStruct);
19.
20.
21.     /*##-2- 使能 LPTIM2 时钟并配置 ########################################################*/
22.     __HAL_RCC_LPTIM2_CLK_ENABLE();
23.
24.     LptimHandle.Instance                             = LPTIM2;
25.     LptimHandle.Init.CounterSource                   = LPTIM_COUNTERSOURCE_INTERNAL;
26.     LptimHandle.Init.UpdateMode                      = LPTIM_UPDATE_ENDOFPERIOD;
27.     LptimHandle.Init.OutputPolarity                  = LPTIM_OUTPUTPOLARITY_HIGH;
28.     LptimHandle.Init.Clock.Source                    = LPTIM_CLOCKSOURCE_APBCLOCK_LPOSC;
29.     LptimHandle.Init.Clock.Prescaler                 = LPTIM_PRESCALER_DIV1;
30.     LptimHandle.Init.UltraLowPowerClock.Polarity     = LPTIM_CLOCKPOLARITY_RISING;
31.     LptimHandle.Init.UltraLowPowerClock.SampleTime = LPTIM_CLOCKSAMPLETIME_DIRECTTRANSITION;
32.     LptimHandle.Init.Trigger.Source                  = LPTIM_TRIGSOURCE_SOFTWARE;
33.     LptimHandle.Init.Trigger.ActiveEdge              = LPTIM_ACTIVEEDGE_RISING;
34.     LptimHandle.Init.Trigger.SampleTime              = LPTIM_TRIGSAMPLETIME_DIRECTTRANSITION;
35.
36.     /*##-3- 初始化 LPTIM2 ################################################################*/
37.     if(HAL_LPTIM_Init(&LptimHandle) != HAL_OK)
38.     {
39.         Error_Handler(__FILE__, __LINE__);
40.     }
41.
42.     /*##-4- 启动 LPTIM2 的 PWM 模式，但使用输出引脚，仅用于 DMAMUX 的触发 ###############*/
43.     /* LPTIM2 的时钟主频是 100MHz，这里配置触发是 100MHz / (10000 - 1 + 1) = 10KHz */
44.     if (HAL_LPTIM_PWM_Start(&LptimHandle, 10000-1, 5000 - 1) != HAL_OK)
45.     {
46.         Error_Handler(__FILE__, __LINE__);
47.     }
48. }
```

这里把几个关键的地方阐释下：

- 第 16 – 18 行，配置 LPTIM2 使用 APB 时钟。
- 第 22 – 40 行，配置 LPTIM2 的相关参数,具体每个参数代表的含义可以看前面 LPTIM 章节的讲解。
- 第 44 – 47 行，配置 LPTIM2 工作在 PWM 模式，频率 10KHz，占空比 50%。这里仅仅是用到 LPTIM2_OUT 的输出信号作为 DMAMUX 的请求发生器触发源，所以用不到 PWM 的输出引脚。

41.2.2 DMMUX 和 BDMA 配置

完整配置如下：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: bsp_InitTimBDMA
4.  *    功能说明: 配置 DMAMUX 的定时器触+DMA 控制任意 IO 做 PWM 和脉冲数控制
5.  *    形    参: 无
6.  *    返 回 值: 无
7.  *********************************************************************************************************
8.  */
```

```
void bsp_InitTimBDMA(void)
{
    GPIO_InitTypeDef  GPIO_InitStruct;
    DMA_HandleTypeDef DMA_Handle = {0};
    HAL_DMA_MuxRequestGeneratorConfigTypeDef dmamux_ReqGenParams ={0};


     /*##-1-  ###################################################*/
    __HAL_RCC_GPIOB_CLK_ENABLE();

    GPIO_InitStruct.Pin = GPIO_PIN_1;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);


    /*##-2- Configure the DMA ###################################################*/
    __HAL_RCC_BDMA_CLK_ENABLE();

    DMA_Handle.Instance                 = BDMA_Channel0;            /* 使用的 BDMA 通道 0 */
    DMA_Handle.Init.Request             =BDMA_REQUEST_GENERATOR0; /* 请求类型采用的 DMAMUX 请求发生器通道 0 */
    DMA_Handle.Init.Direction           = DMA_MEMORY_TO_PERIPH;     /* 传输方向是从存储器到外设 */
    DMA_Handle.Init.PeriphInc           = DMA_PINC_DISABLE;         /* 外设地址自增禁止 */
    DMA_Handle.Init.MemInc              = DMA_MINC_ENABLE;          /* 存储器地址自增使能 */
    DMA_Handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;   /* 外设数据传输位宽选择字，即 32bit */
    DMA_Handle.Init.MemDataAlignment    =DMA_MDATAALIGN_WORD;    /* 存储器数据传输位宽选择字,即 32bit */
    DMA_Handle.Init.Mode                = DMA_CIRCULAR;             /* 循环模式 */
    DMA_Handle.Init.Priority            = DMA_PRIORITY_LOW;         /* 优先级低 */
    DMA_Handle.Init.FIFOMode            = DMA_FIFOMODE_DISABLE;     /* BDMA 不支持 FIFO */
    DMA_Handle.Init.FIFOThreshold       = DMA_FIFO_THRESHOLD_FULL; /* BDMA 不支持 FIFO 阀值设置 */
    DMA_Handle.Init.MemBurst            = DMA_MBURST_SINGLE;        /* BDMA 不支持存储器突发 */
    DMA_Handle.Init.PeriphBurst         = DMA_PBURST_SINGLE;        /* BDMA 不支持外设突发 */

    HAL_DMA_Init(&DMA_Handle);

    /* 开启 BDMA Channel0 的中断 */
    HAL_NVIC_SetPriority(BDMA_Channel0_IRQn, 2, 0);
    HAL_NVIC_EnableIRQ(BDMA_Channel0_IRQn);

    /*##-3- 配置 DMAMUX ###################################################*/
    dmamux_ReqGenParams.SignalID = HAL_DMAMUX2_REQ_GEN_LPTIM2_OUT;      /* 请求触发器选择 LPTIM2_OUT */
    dmamux_ReqGenParams.Polarity  = HAL_DMAMUX_REQ_GEN_RISING_FALLING; /* 上升沿和下降沿均可触发  */
    dmamux_ReqGenParams.RequestNumber = 1;                              /* 触发后，传输进行 1 次 DMA 传输 */

    HAL_DMAEx_ConfigMuxRequestGenerator(&DMA_Handle, &dmamux_ReqGenParams); /* 配置 DMAMUX */

    HAL_DMAEx_EnableMuxRequestGenerator (&DMA_Handle);                      /* 使能 DMAMUX 请求发生器 */

    /*##-4- 启动 DMA 传输 ###################################################*/
    HAL_DMA_Start_IT(&DMA_Handle, (uint32_t)IO_Toggle, (uint32_t)&GPIOB->BSRRL, 8);

    /*
       默认情况下，用户通过注册回调函数 DMA_Handle.XferHalfCpltCallback，然后函数 HAL_DMA_Init 会开启半
       传输完成中断，
       由于这里没有使用 HAL 库默认的中断管理函数 HAL_DMA_IRQHandler，直接手动开启。
    */
    BDMA_Channel0->CCR |= BDMA_CCR_HTIE;
```

```
67.
68.     LPTIM_Config(); /* 配置 LPTIM 触发 DMAMUX */
69. }
```

这里把几个关键的地方阐释下：

- 第 12 - 13 行，对作为局部变量的 HAL 库结构体做初始化，防止不确定值配置时出问题。
- 第 17 - 23 行，配置 PB1 推挽输出。
- 第 27 – 43 行，配置 BDMA 的基本参数，注释较详细。
- 第 46 – 47 行，配置 BDMA 的中断优先级，并使能。
- 第 50 – 56 行，配置 DMAMUX 的请求发生器触发源选择的 LPTIM2_OUT，上升沿和下降沿均触发 BDMA 传输。
- 第 59 行，采用中断方式启动 BDAM 传输，这里中断注意第 2 个参数和第 3 个参数。第 2 个原地址，定义如下：

```
uint32_t IO_Toggle[8]   ={
                          0x00000002U,
                          0x00020000U,
                          0x00000002U,
                          0x00020000U,
                          0x00000002U,
                          0x00020000U,
                          0x00000002U,
                          0x00020000U,
                        };
```

定义了 8 个 uint32_t 类型的变量。第 3 个参数非常考究，这里使用的 GPIO 的 BSRR 寄存器，这个寄存器的特点就是置 1 有效，而清零操作对其无效。

10.4.7 GPIO port bit set/reset register (GPIOx_BSRR) (x = A to K)

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

高 16 位用于控制 GPIO 的输出低电平，而低 16 位用于输出高电平工作，所以我们这里设置

GPIOB_BSRR = 0x00000002 时，表示 PB1 输出高电平。

GPIOB_BSRR = 0x00020000 时，表示 PB1 输出低电平。

通过这种方式就实现了 PB1 引脚的高低电平控制。

- 第 66 行,这里比较特殊,默认情况下,用户通过注册回调函数 DMA_Handle.XferHalfCpltCallback,然后函数 HAL_DMA_Init 会开启半传输完成中断,由于这里没有使用 HAL 库默认的中断管理函数 HAL_DMA_IRQHandler,直接手动开启。
- 第 68 行,调用 LPTIM 的初始化配置。

41.2.3 BDMA 存储器选择注意事项

由于 STM32H7 Cache 的存在,凡是 CPU 和 DMA 都会操作到的存储器,我们都要注意数据一致性问题。对于本章节要实现的功能,如果不需要运行中动态修改 BDMA 源地址中的数据,可以不用管这个问题,如果要动态修改,就得注意 Cache 所带来的的数据一致性问题,这里提供两种解决办法:

- 方法一:

  设置 BDMA 所使用 SRAM3 存储区的 Cache 属性为 Write through, read allocate, no write allocate。保证写入的数据会立即更新到 SRAM3 里面。

```
/* 配置 SRAM3 的属性为 Write through, read allocate, no write allocate */
MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress      = 0x38000000;
MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

- 方法二:

  设置 SRAM3 的缓冲区做 32 字节对齐,大小最好也是 32 字节整数倍,然后调用函数 SCB_CleanDCache_by_Addr 做 Clean 操作即可,保证 BDMA 读取到的数据是刚更新好的。

本章节配套例子是直接使用的方法一。例子中变量的定义方式如下:

```
/* 方便 Cache 类的 API 操作, 做 32 字节对齐 */
#if defined ( __ICCARM__ )
#pragma location = 0x38000000
uint32_t IO_Toggle[8]  =
                      {
                          0x00000002U,
                          0x00020000U,
                          0x00000002U,
                          0x00020000U,
                          0x00000002U,
                          0x00020000U,
                          0x00000002U,
                          0x00020000U,
                      };
```

```
#elif defined ( __CC_ARM )
ALIGN_32BYTES(__attribute__((section (".RAM_D3"))) uint32_t IO_Toggle[8]) =
                                                        {
                                                            0x00000002U,
                                                            0x00020000U,
                                                            0x00000002U,
                                                            0x00020000U,
                                                            0x00000002U,
                                                            0x00020000U,
                                                            0x00000002U,
                                                            0x00020000U,
                                                        };
#endif
```

对于 IAR 需要#pragma location 指定位置，而 MDK 通过分散加载即可实现，详情看前面第 26 章，有详细讲解。

41.2.4 BDMA 中断处理

前面的配置中开启了 BDMA 的传输完成中断、半传输完成中断和传输错误中断。通过半传输完整中断和传输完成中断可以实现双缓冲的效果：

```
/*
*********************************************************************************************************
*    函 数 名: BDMA_Channel0_IRQHandler
*    功能说明: BDMA 通道 0
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void BDMA_Channel0_IRQHandler(void)
{
    /* 传输完成中断 */
    if((BDMA->ISR & BDMA_FLAG_TC0) != RESET)
    {
        BDMA->IFCR = BDMA_FLAG_TC0;

        /*
           1、传输完成开始使用 DMA 缓冲区的前半部分，此时可以动态修改后半部分数据
              比如缓冲区大小是 IO_Toggle[0] 到 IO_Toggle[7]
              那么此时可以修改 IO_Toggle[4] 到 IO_Toggle[7]
           2、变量所在的 SRAM 区已经通过 MPU 配置为 WT 模式，更新变量 IO_Toggle 会立即写入。
           3、不配置 MPU 的话，也可以通过 Cahce 的函数 SCB_CleanDCache_by_Addr 做 Clean 操作。
        */
    }

    /* 半传输完成中断 */
    if((BDMA->ISR & BDMA_FLAG_HT0) != RESET)
    {
        BDMA->IFCR = BDMA_FLAG_HT0;

        /*
           1、半传输完成开始使用 DMA 缓冲区的后半部分，此时可以动态修改前半部分数据
              比如缓冲区大小是 IO_Toggle[0] 到 IO_Toggle[7]
```

```
            那么此时可以修改 IO_Toggle[0] 到 IO_Toggle[3]
        2、变量所在的 SRAM 区已经通过 MPU 配置为 WT 模式，更新变量 IO_Toggle 会立即写入。
        3、不配置 MPU 的话，也可以通过 Cahce 的函数 SCB_CleanDCache_by_Addr 做 Clean 操作。
    */
  }

  /* 传输错误中断 */
  if((BDMA->ISR & BDMA_FLAG_TEO) != RESET)
  {
    BDMA->IFCR = BDMA_FLAG_TEO;
  }
}
```

注释的比较清楚。如果输出的 PWM 频率较高，建议将 BDMA 的缓冲区设置的大些，防止 BDMA 中断的执行频率较高。

41.2.5 BDMA 脉冲个数控制

借助本章 2.4 小节的知识点，如果要实现脉冲个数的控制，在 BDMA 中断服务程序里面动态修改缓冲区即可。比如我们配置：

- BDMA 开启半传输完成中断和传输完成中断。
- BDMA 传输 16 次为一轮，每两次传输算一个周期的脉冲。

  如果要实现 100 个脉冲，我们就可以在 12 轮，即 12*8=96 个脉冲后的传输完成中断里面修改后半部分输出低电平即可，进入半传输完成中断后再修改前半部分数据输出低电平。

41.3 BDMA 板级支持包（bsp_tim_dma.c）

BDMA 驱动文件 bsp_pwm_dma.c 提供了如下两个函数：

- LPTIM_Config
- bsp_InitTimBDMA

函数 LPTIM_Config 是文件内部调用的，而函数 bsp_InitTimBDMA 是供用户调用的。

41.3.1 函数 LPTIM_Config

函数原型：

```
static void LPTIM_Config(void)
```

函数描述：

此函数用于配置 LPTIM2 工作在 PWM 模式，但不初始化 GPIO，使用内部的 LPTIM2_OUT 即可作为 BDMA 请求发生器的触发源。

注意事项：

1. 函数前面 static 用于限制作用域，表示仅在本文件里面调用。
2. 关于此函数的讲解在本章的 2.1 小节

41.3.2 函数 bsp_InitTimBDMA

函数原型：

```
void bsp_InitTimBDMA(void)
```

函数描述：

此函数用于配置定时器触发 BDMA，可以实现任意 IO 做 PWM 输出。

注意事项：

1. 关于此函数的讲解在本章 2.2 小节。

使用举例：

作为初始化函数，直接在 bsp.c 文件的 bsp_Init 函数里面调用即可。

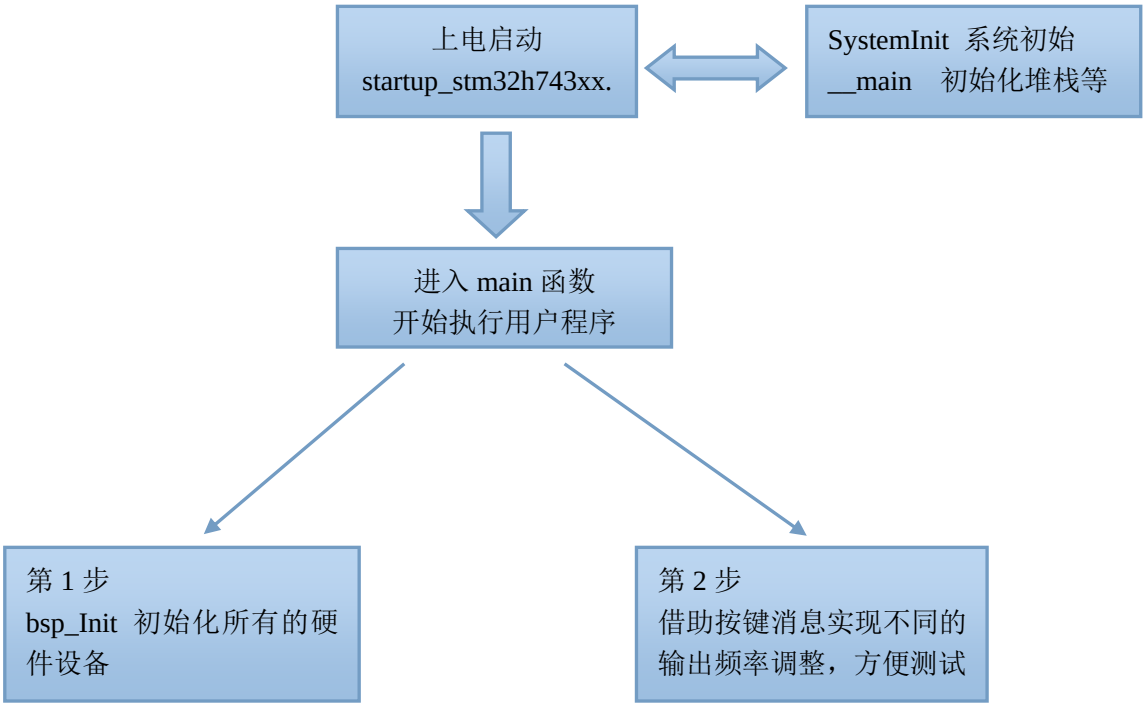
41.4 BDMA 驱动移植和使用

低功耗定时器的移植比较简单：

- 第 1 步：复制 bsp_tim_dma.c 和 bsp_tim_dma.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：这几个驱动文件主要用到 HAL 库的 GPIO、LPTIM 和 DMA 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 3 步，应用方法看本章节配套例子即可。

41.5 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED 和串口。
  - 第 2 步，借助按键消息实现不同的输出频率调整，方便测试。

41.6 实验例程说明（MDK）

配套例子：

V7-010_DMAMUX 的定时器触+BDMA 控制任意 IO 做 PWM 和脉冲数控制

实验目的：

1. 学习 DMAMUX 的定时器触+BDMA 控制任意 IO 做 PWM 和脉冲数控制。

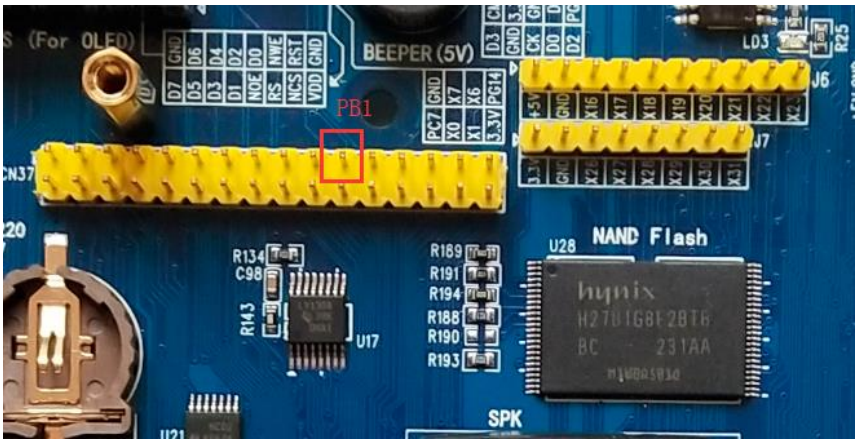
实验内容：

1. 通过 LPTIM2 触发 DMAMUX 的请求发生器，控制 DMA 给任意 IO 做 PWM 输出。

实验操作：

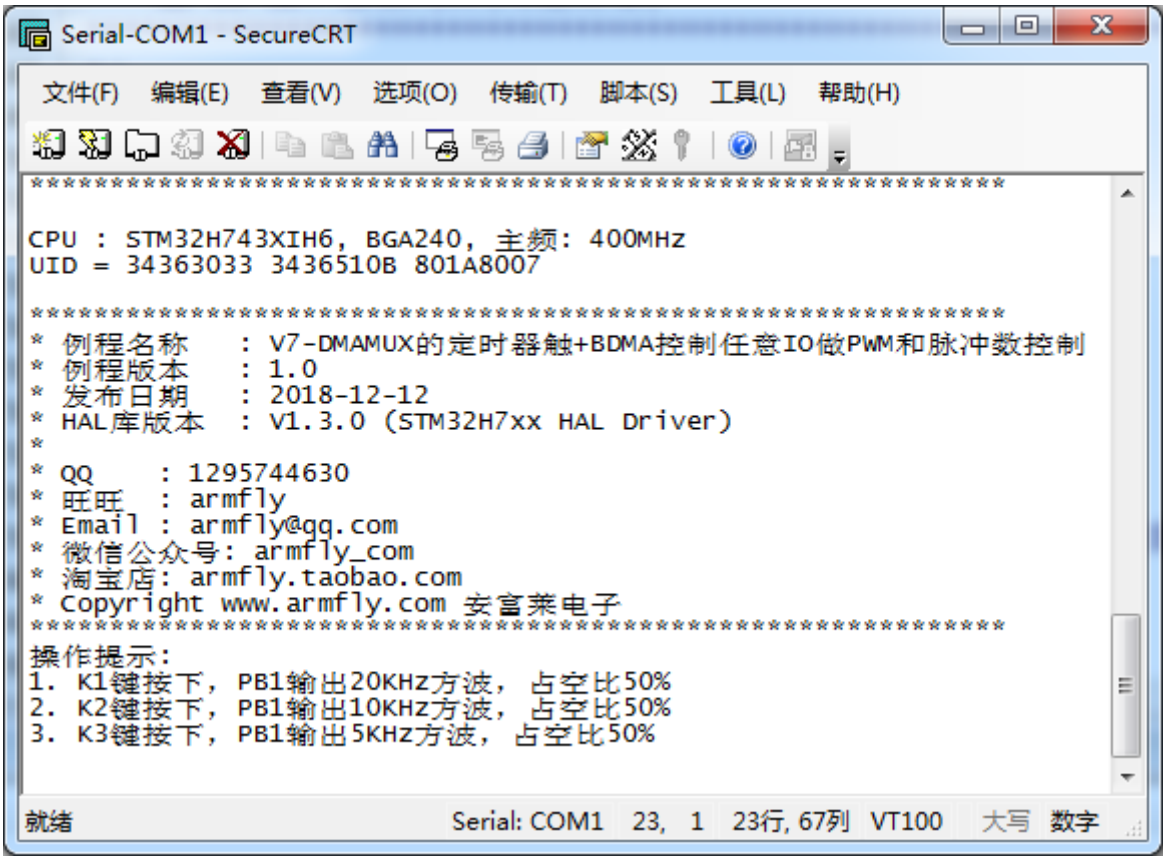
1. K1 键按下，PB1 输出 20KHz 方波，占空比 50%。
2. K2 键按下，PB1 输出 10KHz 方波，占空比 50%。
3. K3 键按下，PB1 输出 5KHz 方波，占空比 50%。

PB1 的位置：



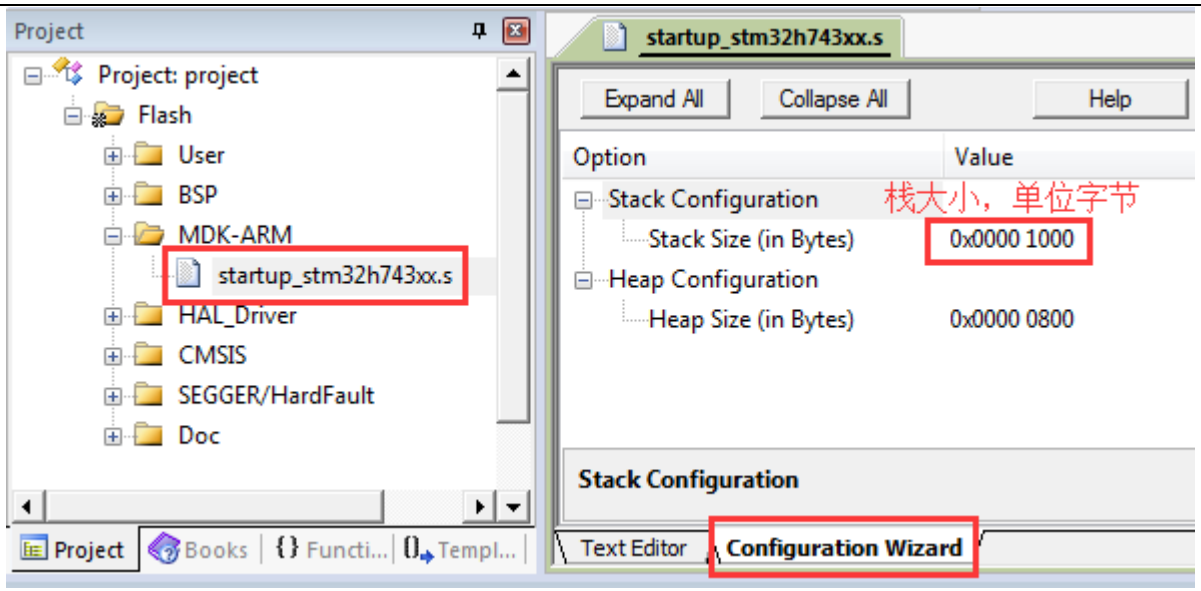
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

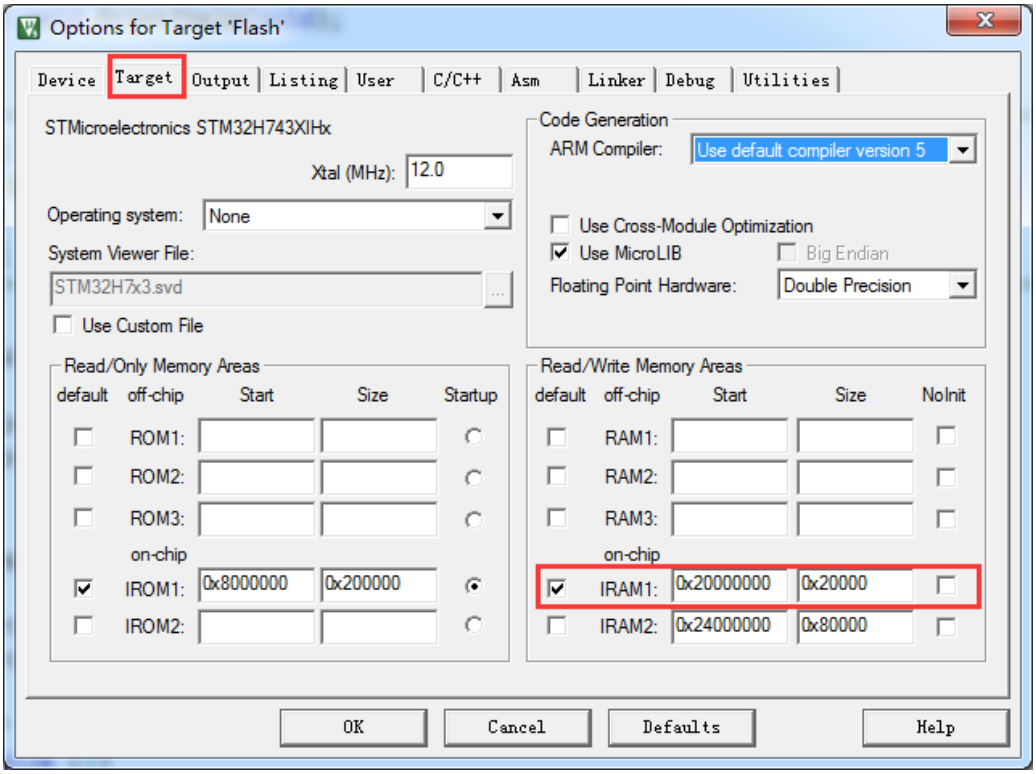


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */

    bsp_InitTimBDMA();  /* 初始化 BDMA 控制 PB1 做的 PWM 输出 */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SRAM4。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
```

```
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SRAM4 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
```

```
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- K1 键按下，PB1 输出 20KHz 方波，占空比 50%。
- K2 键按下，PB1 输出 10KHz 方波，占空比 50%。
- K3 键按下，PB1 输出 5KHz 方波，占空比 50%。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
```

```
                case KEY_DOWN_K1:           /* K1 键按下，PB1 输出 20KHz 方波，占空比 50% */
                if (HAL_LPTIM_PWM_Start(&LptimHandle, 5000-1, 2500 - 1) != HAL_OK)
                {
                    Error_Handler(__FILE__, __LINE__);
                }
                  break;

                case KEY_DOWN_K2:           /* K2 键按下，PB1 输出 10KHz 方波，占空比 50% */
                if (HAL_LPTIM_PWM_Start(&LptimHandle, 10000-1, 5000 - 1) != HAL_OK)
                {
                    Error_Handler(__FILE__, __LINE__);
                }
                break;

                case KEY_DOWN_K3:           /* K3 键按下，PB1 输出 5KHz 方波，占空比 50% */
                if (HAL_LPTIM_PWM_Start(&LptimHandle, 20000-1, 10000 - 1) != HAL_OK)
                {
                    Error_Handler(__FILE__, __LINE__);
                }
                break;

                default:
                  /* 其它的键值不处理 */
                  break;
            }
        }
    }
}
```

41.7 实验例程说明（IAR）

配套例子：

V7-010_DMAMUX 的定时器触+BDMA 控制任意 IO 做 PWM 和脉冲数控制

实验目的：

1. 学习 DMAMUX 的定时器触+BDMA 控制任意 IO 做 PWM 和脉冲数控制。

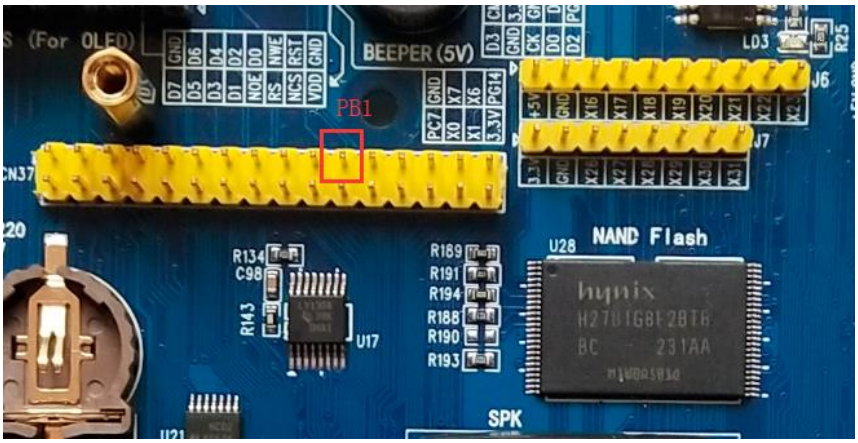
实验内容：

1. 通过 LPTIM2 触发 DMAMUX 的请求发生器，控制 DMA 给任意 IO 做 PWM 输出。

实验操作：

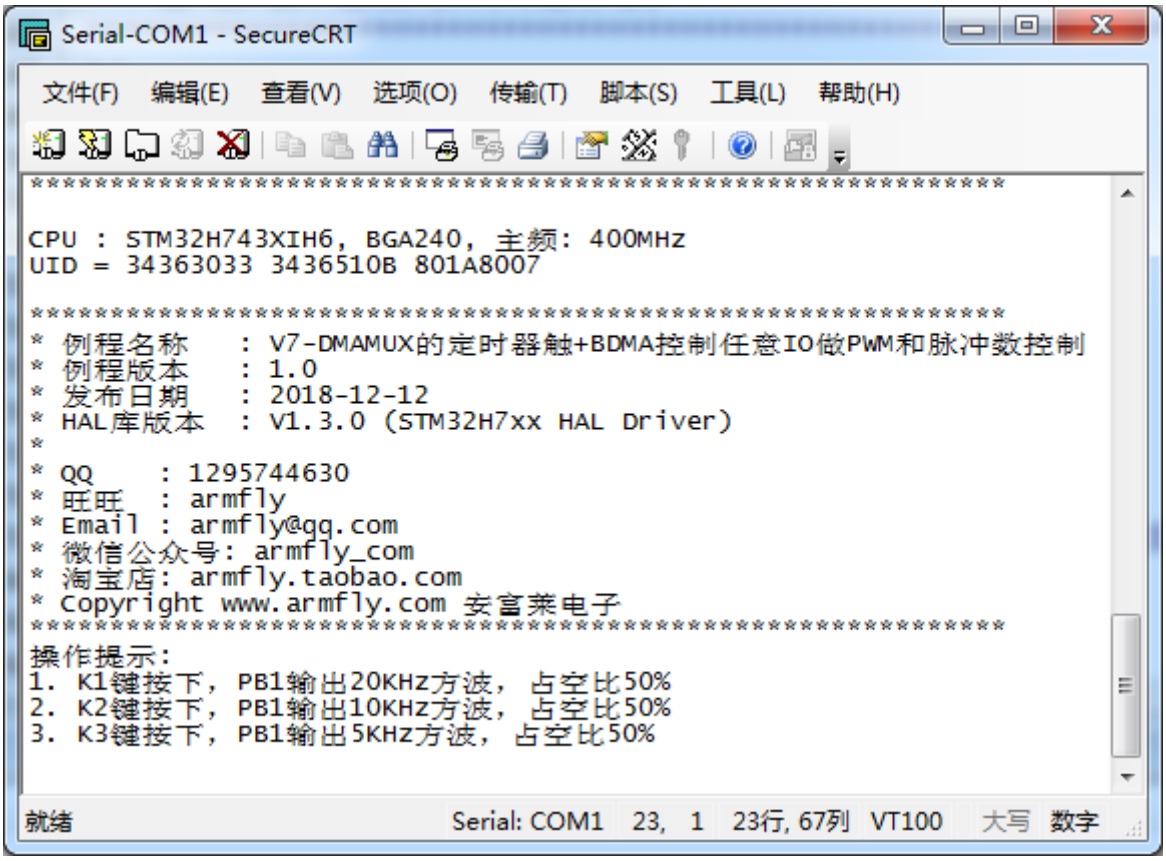
1. K1 键按下，PB1 输出 20KHz 方波，占空比 50%。
2. K2 键按下，PB1 输出 10KHz 方波，占空比 50%。
3. K3 键按下，PB1 输出 5KHz 方波，占空比 50%。

PB1 的位置：



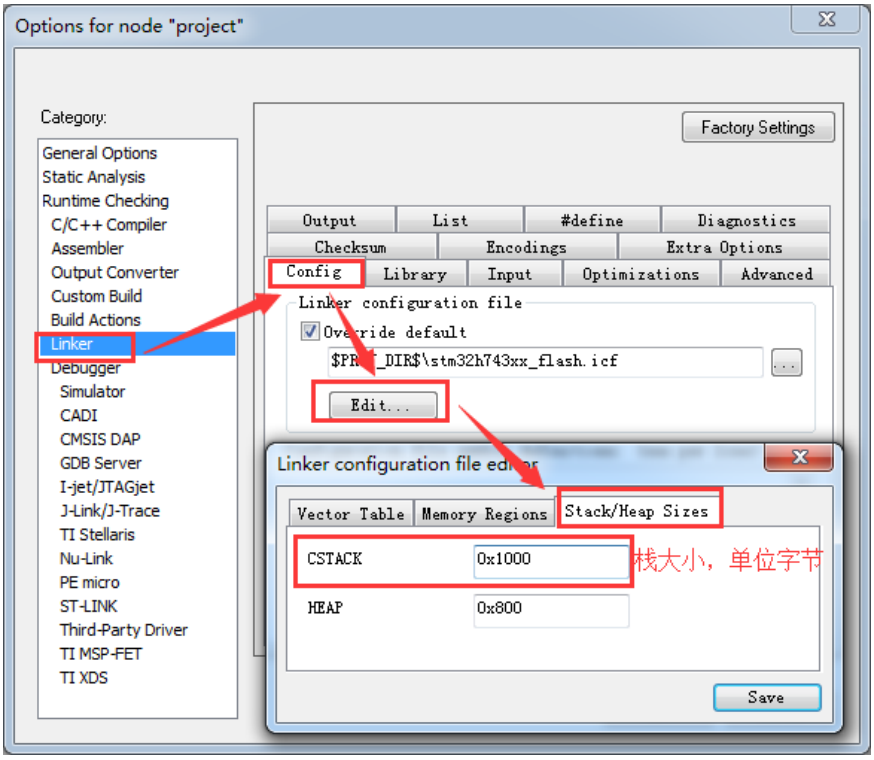
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

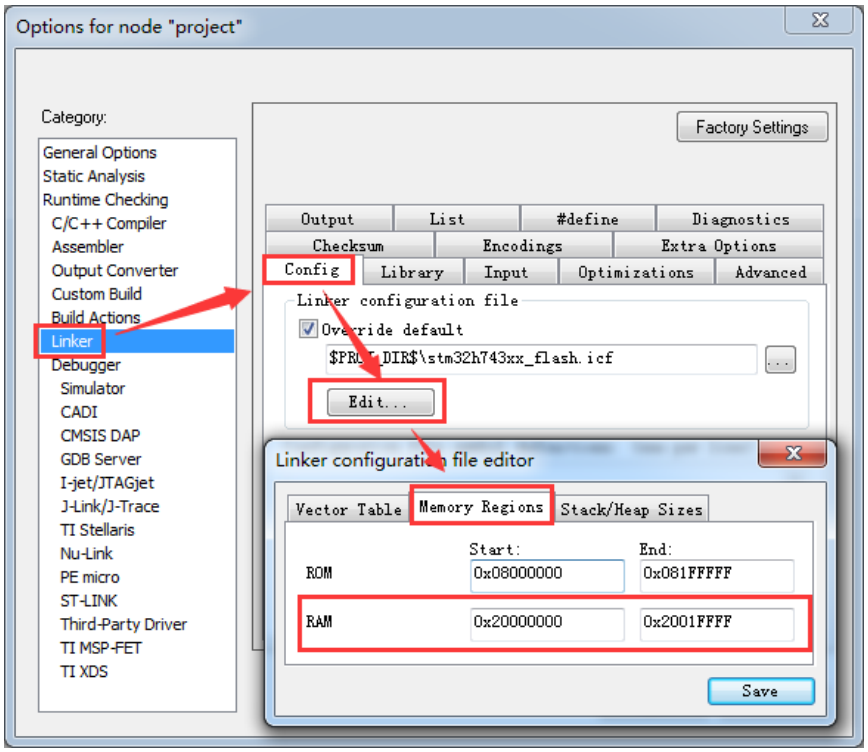


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
```

```
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */

    bsp_InitTimBDMA();  /* 初始化 BDMA 控制 PB1 做的 PWM 输出 */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SRAM4。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SRAM4 的属性为 Write through, read allocate, no write allocate */
   MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
   MPU_InitStruct.BaseAddress      = 0x38000000;
   MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
   MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
   MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
   MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
   MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
   MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
   MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
   MPU_InitStruct.SubRegionDisable = 0x00;
   MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

   HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}
```

```
/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- K1 键按下，PB1 输出 20KHz 方波，占空比 50%。
- K2 键按下，PB1 输出 10KHz 方波，占空比 50%。
- K3 键按下，PB1 输出 5KHz 方波，占空比 50%。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }
```

```
        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值，无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，PB1 输出 20KHz 方波，占空比 50% */
                if (HAL_LPTIM_PWM_Start(&LptimHandle, 5000-1, 2500 - 1) != HAL_OK)
                {
                    Error_Handler(__FILE__, __LINE__);
                }
                  break;

                case KEY_DOWN_K2:           /* K2 键按下，PB1 输出 10KHz 方波，占空比 50% */
                if (HAL_LPTIM_PWM_Start(&LptimHandle, 10000-1, 5000 - 1) != HAL_OK)
                {
                    Error_Handler(__FILE__, __LINE__);
                }
                break;

                case KEY_DOWN_K3:           /* K3 键按下，PB1 输出 5KHz 方波，占空比 50% */
                if (HAL_LPTIM_PWM_Start(&LptimHandle, 20000-1, 10000 - 1) != HAL_OK)
                {
                    Error_Handler(__FILE__, __LINE__);
                }
                break;

                default:
                  /* 其它的键值不处理 */
                  break;
            }
        }
    }
}
```

41.8 总结

本章节就为大家讲解这么多，控制 BDMA 让 GPIO 输出 PWM 以及脉冲数的控制，实际项目中有一定的实用价值，望初学者熟练掌握。

第42章 STM32H7 的 DMA 基础知识和 HAL 库 API

本章节为大家讲解 DMA1（Direct memory access controller，直接存储器访问控制器）和 DMA2，相比前面章节的 BDMA，功能要强些，属于通用型 DMA。

42.1 初学者重要提示
42.2 DMA 基础知识
42.3 DMA 的 HAL 库用法
42.4 源文件 stm32h7xx_hal_dma.c
42.5 总结

42.1 初学者重要提示

- DMA1 和 DMA2 均支持 8 路通道。虽然是 8 路，但这 8 路不是并行工作的，而是由 DMA 的仲裁器决定当前处理那一路。
- DMA 最大传输次数 65535 次，每次传输单位可以是字节、半字和字。
- DMA 的循环模式不可用于存储器到存储器模式。
- DMA1 和 DMA2 带的 FIFO 是 4 个 32bit 的空间，即 16 字节。
- 使用 DMA 的 FIFO 和突发需要注意的问题较多，详情可看本章 2.7 小节。
- STM32H7 的参数手册 DMA 章节对存储器到存储器，外设到存储器，外设到存储器模式的传输过程进行了讲解，推荐大家看完本章节后读一下。

42.2 DMA 基础知识

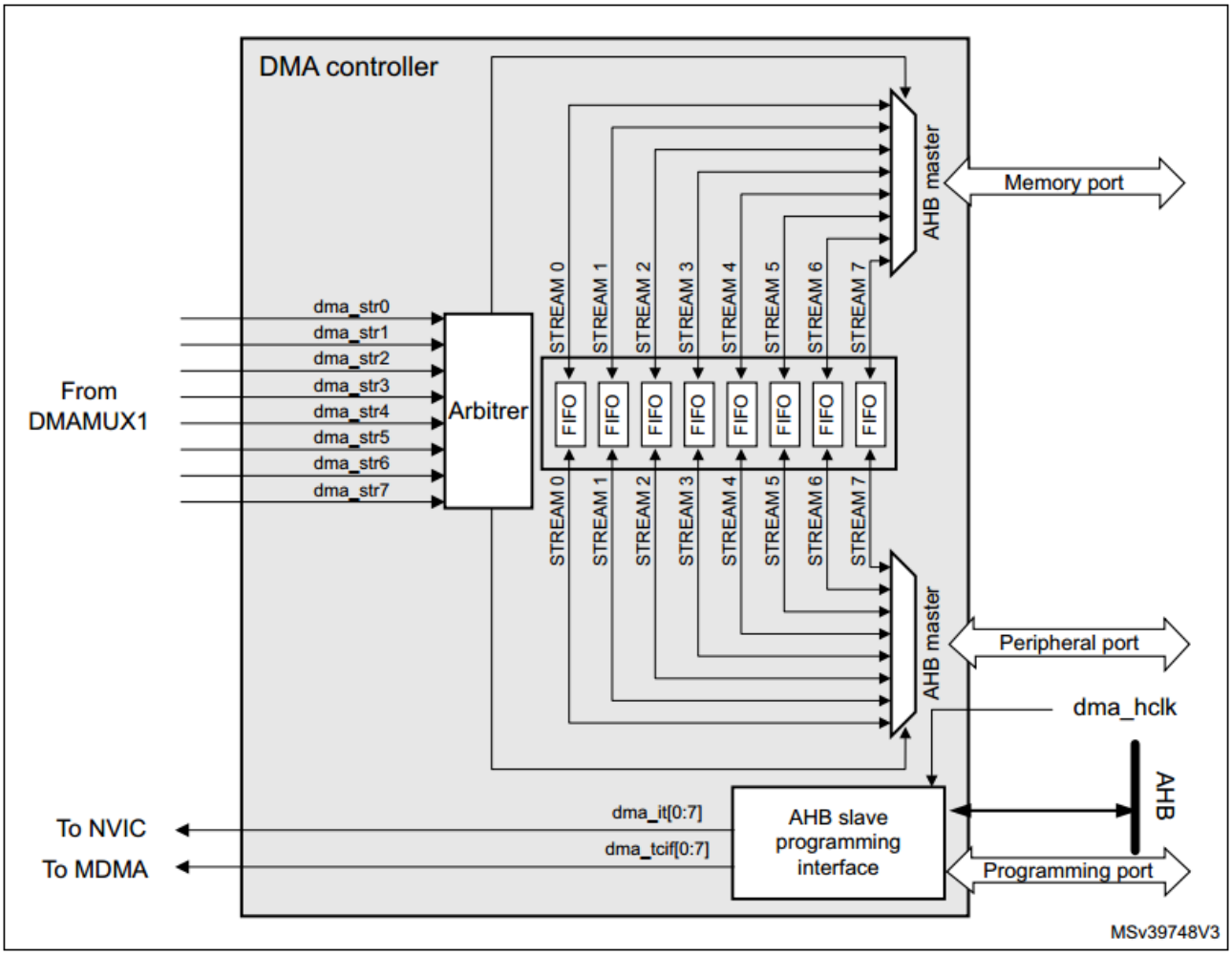
DMA 的几个关键知识点放在开头说：

- 由于总线矩阵的存在，各个主控的道路四通八达，从而可以让 DMA 和 CPU 同时开工，但是注意一点，如果他们同时访问的同一个外设，会有一点性能影响的。
- DMA 支持存储器到外设，外设到存储器和存储器到存储器的传输，不支持外设到外设的传输，而 BDMA 是支持的，这个模式在低功耗模式下比较有用。
- DMA1 和 DMA2 是有两个 AHB 总线主控，可以分别用于源地址和目的地址的传输。

- 源地址和目的地址的数据宽度可以不同，但是数据地址必须要跟其数据类型对齐。比如源地址是 uint32 类型的，那么此数组的地址必须 4 字节对齐。
- DMA 主要有两种模式，一个是 Normal 正常模式，传输一次后就停止传输；另一种是 Circular 循环模式，会一直循环的传输下去，即使有 DMA 中断，传输也是一直在进行的。
- DMA 的数据流请求（Stream0 – Stream7）的优先级可编程，分为四级 Very high priority，High priority，Medium priority 和 Low priority。通道的优先级配置相同的情况下，如果同时产生请求，会优先响应编号低的，即 Stream0 优先响应。

42.2.1 DMA 硬件框图

认识一个外设，最好的方式就是看他的框图，方便我们快速的了解 DMA 的基本功能，然后再看手册了解细节。框图如下所示（DMA1 和 DMA2 是一样的）：



通过这个框图，我们可以得到如下信息：

- dma_str0 – dma_str7

  这里是 8 路来自 DMAMUX1 的 DMA 请求信号。

- dma_it[0:7]接口

通道 0 – 通道 7 的中断触发。

- dma_tcif[0:7]接口

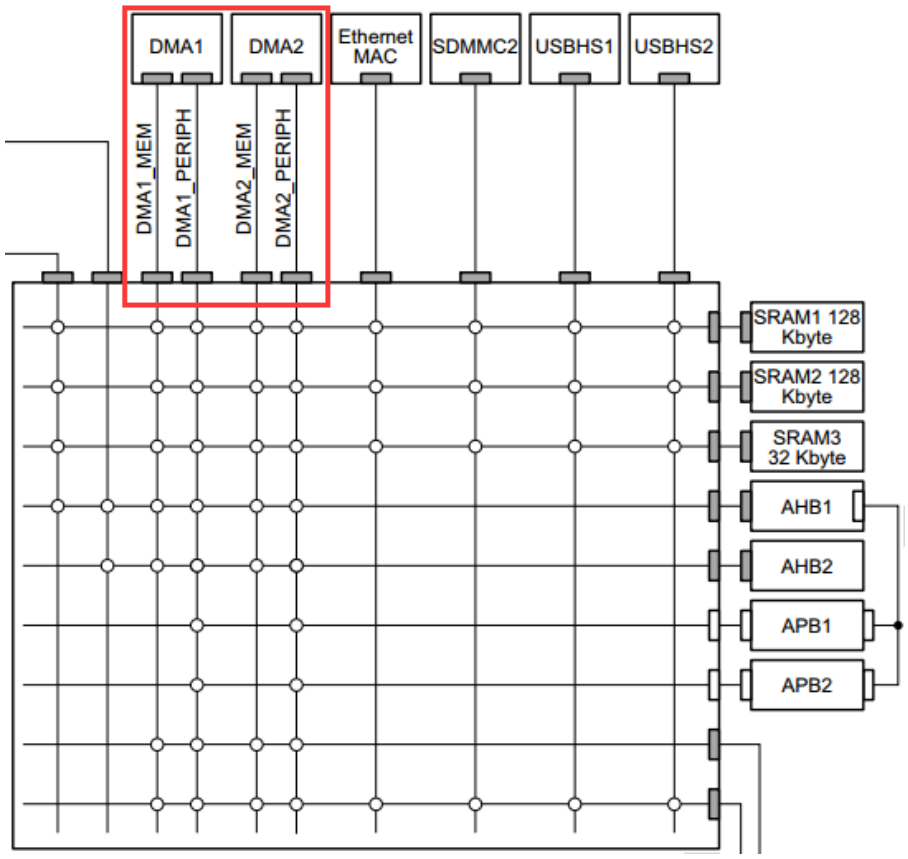
  通道 0 – 通道 7 的传输完成标志，可以用于触发 MDMA 。

- Arbiter 仲裁器

  用于仲裁当期要处理的 DMA 请求。通过这里我们可以看出虽然是 8 路，但这 8 路不是并行工作的，而是由 DMA 的仲裁器决定当前处理哪一路。

- AHB 总线接口

  DMA1 和 DMA2 有两个接口，可以分别用于源地址和目的地址的传输。



42.2.2 DMA 传输

DMA 支持如下几种传输模式：

- 存储器到外设。
- 外设到存储器。
- 存储器到存储器。

关于这几种传输方式要注意以下两个问题：

- 源地址和目的地址的数据宽度可以不同，但是数据地址必须要跟其数据类型对齐。比如源地址是

uint32 类型的，那么此数组的地址必须 4 字节对齐。

- DMA1 和 DMA2 是不支持外设到外设的传输，BDMA 是支持的，这个模式在低功耗模式下比较有用。
- 使用存储器到存储器模式不支持循环传输模式，同时必须开启 FIFO，即不支持直接模式（关闭了 FIFO 就是直接模式 Direct mode）。

- 拓展知识

  MDK 中全局变量的数据对齐问题说明：

  http://www.armbbs.cn/forum.php?mod=viewthread&tid=13511 。

42.2.3 DMA 的循环模式和正常模式

BDMA 主要有两种模式，一个是 Normal 正常模式，传输一次后就停止传输；另一种是 Circular 循环模式，会一直循环的传输下去，即使有 DMA 中断，传输也是一直在进行的。

这两种模式各有用途。

- Normal 正常模式

  适合用于单次传输，比如存储器到存储器的数据复制粘贴，又比如串口的数据单次发送，下次还需要发送的时候，使能下即可。

- Circular 循环模式

  适合用于需要连续传输的场合，比如定时器触发 BDMA 实现任意 IO 的 PWM 输出。

另外特别注意，循环模式不可用于存储器到存储器模式。

42.2.4 DMA 数据封装和解封

DMA1 和 DMA2 可以实现源和目标数据宽度不等时的传输，实现的关键是务必开启 FIFO。无需像 F1 系列那样强行要求数据缓冲的 4 字节对齐。下面是各种源地址和目的地址数据宽度传输的效果，可以帮助大家更好的理解。

AHB 存储器端口宽度	AHB 外设端口宽度	要传输的数据项的数目 (NDT)	存储器传输数目	存储器端口地址/字节通道	外设传输数目	外设端口地址/字节通道	
						PINCOS = 1	PINCOS = 0
8	8	4	1 2 3 4	0x0/B0[7:0] 0x1/B1[7:0] 0x2/B2[7:0] 0x3/B3[7:0]	1 2 3 4	0x0/B0[7:0] 0x4/B1[7:0] 0x8/B2[7:0] 0xC/B3[7:0]	0x0/B0[7:0] 0x1/B1[7:0] 0x2/B2[7:0] 0x3/B3[7:0]
8	16	2	1 2 3 4	0x0/B0[7:0] 0x1/B1[7:0] 0x2/B2[7:0] 0x3/B3[7:0]	1 2	0x0/B1|B0[15:0] 0x4/B3|B2[15:0]	0x0/B1|B0[15:0] 0x2/B3|B2[15:0]
8	32	1	1 2 3 4	0x0/B0[7:0] 0x1/B1[7:0] 0x2/B2[7:0] 0x3/B3[7:0]	1	0x0/B3|B2|B1|B0[31:0]	0x0/B3|B2|B1|B0[31:0]
16	8	4	1 2	0x0/B1|B0[15:0] 0x2/B3|B2[15:0]	1 2 3 4	0x0/B0[7:0] 0x4/B1[7:0] 0x8/B2[7:0] 0xC/B3[7:0]	0x0/B0[7:0] 0x1/B1[7:0] 0x2/B2[7:0] 0x3/B3[7:0]
16	16	2	1 2	0x0/B1|B0[15:0] 0x2/B1|B0[15:0]	1 2	0x0/B1|B0[15:0] 0x4/B3|B2[15:0]	0x0/B1|B0[15:0] 0x2/B3|B2[15:0]
16	32	1	1 2	0x0/B1|B0[15:0] 0x2/B3|B2[15:0]	1	0x0/B3|B2|B1|B0[31:0]	0x0/B3|B2|B1|B0[31:0]
32	8	4	1	0x0/B3|B2|B1|B0[31:0]	1 2 3 4	0x0/B0[7:0] 0x4/B1[7:0] 0x8/B2[7:0] 0xC/B3[7:0]	0x0/B0[7:0] 0x1/B1[7:0] 0x2/B2[7:0] 0x3/B3[7:0]
32	16	2	1	0x0/B3|B2|B1|B0[31:0]	1 2	0x0/B1|B0[15:0] 0x4/B3|B2[15:0]	0x0/B1|B0[15:0] 0x2/B3|B2[15:0]
32	32	1	1	0x0/B3|B2|B1|B0 [31:0]	1	0x0/B3|B2|B1|B0 [31:0]	0x0/B3|B2|B1|B0[31:0]

这里为大家说下这个图应该如何查看：

- 首先看 NDT（要传输的数据项目列），这一列跟外设传输的项目数是一样的。对应到外设端口地址列，PINCOS=1 和 PINCOS=0 时的传输次数也是一致的。
  注：PINGCOS=1 表示外设地址偏移固定为 4，PINCOS=0 表示外设地址的偏移量由用户配置的外设数据宽度，即 PSIZE 决定。
- 有了要传输的数据个数，外设端口宽度和存储器端口宽度后
  存储器的传输数目 = NDT * 外设端口宽度 / 存储器端口宽度。

有了这两条，大家看上面的表格就方便很多了。最后就是注意 PINCOS 配置所代表的含义。

42.2.5 DMA 双缓冲

DMA1 和 DMA2 支持双缓冲模式的，双缓冲的含义是源地址或者目的地址可以设置两个缓冲区，这种方式的好处是一个缓冲区在接收或者发送数据的时候，另一个缓冲区可以动态更新数据或者处理已经接收到的数据。

当用户开启了 DMA 传输完成中断后，通过寄存器 CCRx 的 CT 位判断当前使用的是哪个缓冲区：

- 如果 CT = 1 表示当前正在使用缓冲区 1，即寄存器 DMA_SxM1AR 记录的地址。
- 如果 CT = 0 表示当前正在使用缓冲区 0，即寄存器 DMA_SxM0AR 记录的地址。

另外注意，存储器到存储器的 DMA 传输不支持双缓冲模式，仅可以用于存储器到外设或者外设到存储器。

42.2.6 DMA 的 FIFO 和突发支持

使用 DMA 的 FIFO 主要有两个作用，一个是降低总线带宽的需求，另一个是前面说的源地址数据宽度和目的地址数据宽度不同时的数据传输。

而突发传输的含义是每个 DMA 请求后可以连续传输的数据项目数，支持 4 次，8 次和 16 次。

了解到以上两点就够用了，现在重点讲解下使用中的注意事项，使用 FIFO 要注意的事项较多。

- 禁止 FIFO 的情况下，即 STM32H7 参考手册里面所说的直接模式 Direct Mode，务必要保证外设数据宽度和内存数据宽度是一样的，而且禁止了 FIFO 的情况下，不支持突发，即使配置了，也是无效的。
- 禁止了 FIFO 的情况下，也不可用于存储器到存储器的数据传输，仅支持外设到存储器或者存储器到外设方式。
- 使能 FIFO 的情况下，可以使用突发模式，也可以不使用。
- 独立的源和目标传输宽度（字节、半字、字）：源和目标的数据宽度不相等时， DMA 自动封装/解封必要的传输数据来优化带宽。这个特性仅在 FIFO 模式下可用。这个特性非常重要，在 H7 使用 SDIO 时要用到。无需像 F1 系列那样强行数据缓冲的 4 字节对齐要求。

最后要特别注意一点，一般应用中最好关闭 FIFO，实际测试发现容易有一些比较奇怪的问题，大家测试的时候要注意。

42.2.7 DMA 支持的各种配置

不像 BDMA，配置 DMA1 和 DMA2 时要注意的事项较多，通过下面的图可以帮助大家方便的验证配置选项是否合理：

DMA 传输模式	源	目标	流控制器	循环模式	传输类型	直接模式	双缓冲区模式
外设到存储器	AHB 外设端口	AHB 存储器端口	DMA	允许	单独	允许	允许
					突发	禁止	
			外设	禁止	单独	允许	禁止
					突发	禁止	
存储器到外设	AHB 存储器端口	AHB 外设端口	DMA	允许	单独	允许	允许
					突发	禁止	
			外设	禁止	单独	允许	禁止
					突发	禁止	
存储器到存储器	AHB 外设端口	AHB 存储器端口	仅 DMA	禁止	单独	禁止	禁止
					突发		

42.3 DMA 的 HAL 库用法

DMA 的 HAL 库用法其实就是几个结构体变量成员的配置和使用，然后配置时钟，并根据需要配置 NVIC、中断和 DMA。下面我们逐一展开为大家做个说明。

42.3.1 DMA 寄存器结构体

DMA 相关的寄存器是通过 HAL 库中的结构体 DMA_TypeDef 和 DMA_Stream_TypeDef 定义的，在 stm32h743xx.h 中可以找到这个类型定义：

```
typedef struct
{
  __IO uint32_t LISR;   /*!< DMA low interrupt status register,      Address offset: 0x00 */
  __IO uint32_t HISR;   /*!< DMA high interrupt status register,     Address offset: 0x04 */
  __IO uint32_t LIFCR;  /*!< DMA low interrupt flag clear register,  Address offset: 0x08 */
  __IO uint32_t HIFCR;  /*!< DMA high interrupt flag clear register, Address offset: 0x0C */
} DMA_TypeDef;

typedef struct
{
  __IO uint32_t CR;     /*!< DMA stream x configuration register      */
  __IO uint32_t NDTR;   /*!< DMA stream x number of data register     */
  __IO uint32_t PAR;    /*!< DMA stream x peripheral address register */
  __IO uint32_t M0AR;   /*!< DMA stream x memory 0 address register   */
  __IO uint32_t M1AR;   /*!< DMA stream x memory 1 address register   */
  __IO uint32_t FCR;    /*!< DMA stream x FIFO control register       */
} DMA_Stream_TypeDef;
```

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

与其它外设的的定义方式不同，DMA 有 8 组通道，每个通道都有一组 DMA_Stream_TypeDef 结构

体所定义的寄存器。

解决这个问题的办法就是定义一套 DMA1_Stream0 - DMA_Stream7 来解决，定义在 stm32h743xx.h 文件。

```
#define PERIPH_BASE           ((uint32_t)0x40000000)
#define D2_AHB1PERIPH_BASE    (PERIPH_BASE + 0x00020000)
#define DMA1_BASE             (D2_AHB1PERIPH_BASE + 0x0000)
#define DMA2_BASE             (D2_AHB1PERIPH_BASE + 0x0400)
#define DMA1                  ((DMA_TypeDef *) DMA1_BASE)
#define DMA2                  ((DMA_TypeDef *) DMA2_BASE)

#define DMA1_Stream0_BASE     (DMA1_BASE + 0x010)
#define DMA1_Stream1_BASE     (DMA1_BASE + 0x028)
#define DMA1_Stream2_BASE     (DMA1_BASE + 0x040)
#define DMA1_Stream3_BASE     (DMA1_BASE + 0x058)
#define DMA1_Stream4_BASE     (DMA1_BASE + 0x070)
#define DMA1_Stream5_BASE     (DMA1_BASE + 0x088)
#define DMA1_Stream6_BASE     (DMA1_BASE + 0x0A0)
#define DMA1_Stream7_BASE     (DMA1_BASE + 0x0B8)

<-----展开下面 DMA1_Stream0 的宏定义，(DMA_Stream_TypeDef *) 0x40020010
#define DMA1_Stream0          ((DMA_Stream_TypeDef *) DMA1_Stream0_BASE)
#define DMA1_Stream1          ((DMA_Stream_TypeDef *) DMA1_Stream1_BASE)
#define DMA1_Stream2          ((DMA_Stream_TypeDef *) DMA1_Stream2_BASE)
#define DMA1_Stream3          ((DMA_Stream_TypeDef *) DMA1_Stream3_BASE)
#define DMA1_Stream4          ((DMA_Stream_TypeDef *) DMA1_Stream4_BASE)
#define DMA1_Stream5          ((DMA_Stream_TypeDef *) DMA1_Stream5_BASE)
#define DMA1_Stream6          ((DMA_Stream_TypeDef *) DMA1_Stream6_BASE)
#define DMA1_Stream7          ((DMA_Stream_TypeDef *) DMA1_Stream7_BASE)

#define DMA1_Stream0_BASE     (DMA1_BASE + 0x010)
#define DMA1_Stream1_BASE     (DMA1_BASE + 0x028)
#define DMA1_Stream2_BASE     (DMA1_BASE + 0x040)
#define DMA1_Stream3_BASE     (DMA1_BASE + 0x058)
#define DMA1_Stream4_BASE     (DMA1_BASE + 0x070)
#define DMA1_Stream5_BASE     (DMA1_BASE + 0x088)
#define DMA1_Stream6_BASE     (DMA1_BASE + 0x0A0)
#define DMA1_Stream7_BASE     (DMA1_BASE + 0x0B8)
#define DMA2_Stream0          ((DMA_Stream_TypeDef *) DMA2_Stream0_BASE)
#define DMA2_Stream1          ((DMA_Stream_TypeDef *) DMA2_Stream1_BASE)
#define DMA2_Stream2          ((DMA_Stream_TypeDef *) DMA2_Stream2_BASE)
#define DMA2_Stream3          ((DMA_Stream_TypeDef *) DMA2_Stream3_BASE)
#define DMA2_Stream4          ((DMA_Stream_TypeDef *) DMA2_Stream4_BASE)
#define DMA2_Stream5          ((DMA_Stream_TypeDef *) DMA2_Stream5_BASE)
#define DMA2_Stream6          ((DMA_Stream_TypeDef *) DMA2_Stream6_BASE)
#define DMA2_Stream7          ((DMA_Stream_TypeDef *) DMA2_Stream7_BASE)
```

我们访问 DMA1 的 LISR 寄存器可以采用这种形式：DMA1->LISR = 0，而访问 DMA1 Stream0 的 CR 就可以采用这种形式 DMA1_Stream0->CR = 0。

42.3.2 DMA 句柄结构体 DMA_HandleTypeDef

HAL 库在 DMA_TypeDef 的基础上封装了一个结构体 DMA_HandleTypeDef，定义如下：

```
typedef struct __DMA_HandleTypeDef
{
  void                          *Instance;
  DMA_InitTypeDef               Init;
  HAL_LockTypeDef               Lock;
  __IO HAL_DMA_StateTypeDef     State;
  void                          *Parent;
  void                          (* XferCpltCallback)( struct __DMA_HandleTypeDef * hdma);
  void                          (* XferHalfCpltCallback)( struct __DMA_HandleTypeDef * hdma);
  void                          (* XferM1CpltCallback)( struct __DMA_HandleTypeDef * hdma);
  void                          (* XferM1HalfCpltCallback)( struct __DMA_HandleTypeDef * hdma);
  void                          (* XferErrorCallback)( struct __DMA_HandleTypeDef * hdma);
  void                          (* XferAbortCallback)( struct __DMA_HandleTypeDef * hdma);
 __IO uint32_t                  ErrorCode;
 uint32_t                       StreamBaseAddress;
 uint32_t                       StreamIndex;
 DMAMUX_Channel_TypeDef         *DMAmuxChannel;
 DMAMUX_ChannelStatus_TypeDef   *DMAmuxChannelStatus;
 uint32_t                       DMAmuxChannelStatusMask;
 DMAMUX_RequestGen_TypeDef      *DMAmuxRequestGen;
 DMAMUX_RequestGenStatus_TypeDef *DMAmuxRequestGenStatus;
 uint32_t                       DMAmuxRequestGenStatusMask;
}DMA_HandleTypeDef;
```

这里重点介绍前几个参数，其它参数主要是 HAL 库内部使用的。

- **void *Instance**

  用于 BDMA，DMA1 和 DMA2 的例化，主要是相关寄存器的操作。

  因为 DMA1，DMA2 和 BDMA 都使用的这个结构体句柄，而 DMA1，DMA2 与 BDMA 的寄存器结构体封装是不同的，这里的定义比较巧妙， 定义为 void *空类型后，就可以直接使用 DMA1，DMA2 和 BDMA 的结构体定义了。

  比如操作 DMA1 Stream1 的寄存器 CR：

  DMA_HandleTypeDef DMA_Handle;

  DMA_Handle.Instance = DMA1_Stream1;

  ((DMA_Stream_TypeDef *) DMA_Handle ->Instance)->CR =0;

  又比如操作 BDMA Channel1 的寄存器 CCR：

  DMA_HandleTypeDef BDMA_Handle;

  BDMA_Handle.Instance = BDMA_Channel1;

  ((BDMA_Channel_TypeDef *) DMA_Handle ->Instance)->CCR =0;

- **DMA_InitTypeDef Init;**

  这个参数是用户接触最多的，用于配置 DMA 的基本参数，像波特率，奇偶校验，停止位等。

  DMA_InitTypeDef 结构体的定义如下：

```
typedef struct
{
  uint32_t Request;
  uint32_t Direction;
  uint32_t PeriphInc;
  uint32_t MemInc;
```

```
  uint32_t PeriphDataAlignment;
  uint32_t MemDataAlignment;
  uint32_t Mode;
  uint32_t Priority;
  uint32_t FIFOMode;
  uint32_t FIFOThreshold;
  uint32_t MemBurst;
  uint32_t PeriphBurst;
}DMA_InitTypeDef;
```

- 成员 Request

用于设置支持的 DMA 请求，对于 DMA 来说，主要来自 DMAMUX1。

```
/* D2 Domain : DMAMUX1 requests */
#define DMA_REQUEST_MEM2MEM          0U  /*!< memory to memory transfer   */

#define DMA_REQUEST_GENERATOR0       1U  /*!< DMAMUX1 request generator 0 */
#define DMA_REQUEST_GENERATOR1       2U  /*!< DMAMUX1 request generator 1 */
#define DMA_REQUEST_GENERATOR2       3U  /*!< DMAMUX1 request generator 2 */
#define DMA_REQUEST_GENERATOR3       4U  /*!< DMAMUX1 request generator 3 */
#define DMA_REQUEST_GENERATOR4       5U  /*!< DMAMUX1 request generator 4 */
#define DMA_REQUEST_GENERATOR5       6U  /*!< DMAMUX1 request generator 5 */
#define DMA_REQUEST_GENERATOR6       7U  /*!< DMAMUX1 request generator 6 */
#define DMA_REQUEST_GENERATOR7       8U  /*!< DMAMUX1 request generator 7 */

#define DMA_REQUEST_ADC1             9U  /*!< DMAMUX1 ADC1 request */
#define DMA_REQUEST_ADC2             10U /*!< DMAMUX1 ADC2 request */

#define DMA_REQUEST_TIM1_CH1         11U  /*!< DMAMUX1 TIM1 CH1 request  */
#define DMA_REQUEST_TIM1_CH2         12U  /*!< DMAMUX1 TIM1 CH2 request  */
#define DMA_REQUEST_TIM1_CH3         13U  /*!< DMAMUX1 TIM1 CH3 request  */
#define DMA_REQUEST_TIM1_CH4         14U  /*!< DMAMUX1 TIM1 CH4 request  */
#define DMA_REQUEST_TIM1_UP          15U  /*!< DMAMUX1 TIM1 UP request   */
#define DMA_REQUEST_TIM1_TRIG        16U  /*!< DMAMUX1 TIM1 TRIG request */
#define DMA_REQUEST_TIM1_COM         17U  /*!< DMAMUX1 TIM1 COM request  */

中间部分省略未写

#define DMA_REQUEST_TIM16_CH1        109U  /*!< DMAMUX1 TIM16 CH1 request  */
#define DMA_REQUEST_TIM16_UP         110U  /*!< DMAMUX1 TIM16 UP request   */

#define DMA_REQUEST_TIM17_CH1        111U  /*!< DMAMUX1 TIM17 CH1 request  */
#define DMA_REQUEST_TIM17_UP         112U  /*!< DMAMUX1 TIM17 UP request   */

#define DMA_REQUEST_SAI3_A           113U  /*!< DMAMUX1 SAI3 A request  */
#define DMA_REQUEST_SAI3_B           114U  /*!< DMAMUX1 SAI3 B request  */

#define DMA_REQUEST_ADC3             115U  /*!< DMAMUX1 ADC3  request  */
```

- 成员 Direction

用于设置传输方向，外设到存储器，存储器到外设或者存储器到存储器，具体支持的参数如下：

```
#define DMA_PERIPH_TO_MEMORY         ((uint32_t)0x00000000U)     /*!< Peripheral to memory direction */
#define DMA_MEMORY_TO_PERIPH         ((uint32_t)DMA_SxCR_DIR_0)  /*!< Memory to peripheral direction */
#define DMA_MEMORY_TO_MEMORY         ((uint32_t)DMA_SxCR_DIR_1)  /*!< Memory to memory direction     */
```

- 成员 PeriphInc

用于设置外设地址是否使能递增，即每完成一次传输，外设地址自增，增加的大小由参数

PeriphDataAlignment 决定。具体支持的参数如下：

```
#define DMA_PINC_ENABLE         ((uint32_t)DMA_SxCR_PINC)  /*!< Peripheral increment mode enable  */
#define DMA_PINC_DISABLE        ((uint32_t)0x00000000U)    /*!< Peripheral increment mode disable */
```

- 成员 MemInc

  用于设置存储器地址是否使能递增，即每完成一次传输，存储器地址自增，增加的大小由参数 MemDataAlignment 决定。具体支持的参数如下：

```
#define DMA_MINC_ENABLE         ((uint32_t)DMA_SxCR_MINC)  /*!< Memory increment mode enable  */
#define DMA_MINC_DISABLE        ((uint32_t)0x00000000U)    /*!< Memory increment mode disable */
```

- 成员 PeriphDataAlignment

  用于设置外设支持的数据宽度，可以选择字节，半字和字进行传输。

```
#define DMA_PDATAALIGN_BYTE     ((uint32_t)0x00000000U)      /*!< Peripheral data alignment: Byte     */
#define DMA_PDATAALIGN_HALFWORD (uint32_t)DMA_SxCR_PSIZE_0)  /*!< Peripheral data alignment: HalfWord */
#define DMA_PDATAALIGN_WORD     ((uint32_t)DMA_SxCR_PSIZE_1) /*!< Peripheral data alignment: Word     */
```

- 成员 MemDataAlignment

  用于设置存储器支持的数据宽度，可以选择字节，半字和字进行传输。

```
#define DMA_MDATAALIGN_BYTE          ((uint32_t)0x00000000U)       /*!< Memory data alignment: Byte     */
#define DMA_MDATAALIGN_HALFWORD      ((uint32_t)DMA_SxCR_MSIZE_0)  /*!< Memory data alignment: HalfWord */
#define DMA_MDATAALIGN_WORD          ((uint32_t)DMA_SxCR_MSIZE_1)  /*!< Memory data alignment: Word     */
```

- 成员 Mode

  用于设置正常模式、循环模式和流控制，对于 BDMA 而言，仅支持正常模式和循环模式。

```
#define DMA_NORMAL          ((uint32_t)0x00000000U)      /*!< Normal mode                  */
#define DMA_CIRCULAR        ((uint32_t)DMA_SxCR_CIRC)    /*!< Circular mode                */
#define DMA_PFCTRL          ((uint32_t)DMA_SxCR_PFCTRL)  /*!< Peripheral flow control mode */
```

- 成员 Priority

  用于 DMA 通道进行传输时的优先级设置，控制多通道同时请求时优先响应谁。支持四种优先级设置：

```
#define DMA_PRIORITY_LOW            ((uint32_t)0x00000000U)    /*!< Priority level: Low       */
#define DMA_PRIORITY_MEDIUM         ((uint32_t)DMA_SxCR_PL_0)  /*!< Priority level: Medium    */
#define DMA_PRIORITY_HIGH           ((uint32_t)DMA_SxCR_PL_1)  /*!< Priority level: High      */
#define DMA_PRIORITY_VERY_HIGH      ((uint32_t)DMA_SxCR_PL)    /*!< Priority level: Very High */
```

- 成员 FIFOMode

  用于配置是否使能 FIFO，BDMA 不支持 FIFO，仅 DMA1 和 DMA2 支持。

```
#define DMA_FIFOMODE_DISABLE        ((uint32_t)0x00000000U)      /*!< FIFO mode disable */
#define DMA_FIFOMODE_ENABLE         ((uint32_t)DMA_SxFCR_DMDIS)  /*!< FIFO mode enable  */
```

- 成员 FIFOThreshold

  使能了 FIFO 后，用于 FIFO 阀值设置，BDMA 不支持此参数，仅 DMA1 和 DMA2 支持。可以设置 FIFO 空间的四分之一，四分之二，四分之三和使用所有空间作为阀值（FIFO 总大小是 16 字节）。

```
/*!< FIFO threshold 1 quart full configuration  */
#define DMA_FIFO_THRESHOLD_1QUARTERFULL  ((uint32_t)0x00000000U)

/*!< FIFO threshold half full configuration     */
#define DMA_FIFO_THRESHOLD_HALFFULL      ((uint32_t)DMA_SxFCR_FTH_0)

/*!< FIFO threshold 3 quarts full configuration */
```

```
#define DMA_FIFO_THRESHOLD_3QUARTERSFULL ((uint32_t)DMA_SxFCR_FTH_1)

/*!< FIFO threshold full configuration          */
#define DMA_FIFO_THRESHOLD_FULL               ((uint32_t)DMA_SxFCR_FTH)
```

- 成员 MemBurst

  存储器突发配置，必须使能了 FIFO 才有效，否则设置此参数没有意义，BDMA 不支持此参数，仅 DMA1，DMA2 支持。SINGLE 表示每个 DMA 请求进行 1 次，INC4 表示每个 DMA 请求进行 4 次 DMA 传输。

```
#define DMA_MBURST_SINGLE       ((uint32_t)0x00000000U)
#define DMA_MBURST_INC4         ((uint32_t)DMA_SxCR_MBURST_0)
#define DMA_MBURST_INC8         ((uint32_t)DMA_SxCR_MBURST_1)
#define DMA_MBURST_INC16        ((uint32_t)DMA_SxCR_MBURST)
```

- 成员 PeriphBurst

  外设突发配置，必须使能了 FIFO 才有效，否则设置此参数没有意义，BDMA 不支持此参数，仅 DMA1，DMA2 支持。SINGLE 表示每个 DMA 请求进行 1 次，INC4 表示每个 DMA 请求进行 4 次 DMA 传输。

```
#define DMA_PBURST_SINGLE       ((uint32_t)0x00000000U)
#define DMA_PBURST_INC4         ((uint32_t)DMA_SxCR_PBURST_0)
#define DMA_PBURST_INC8         ((uint32_t)DMA_SxCR_PBURST_1)
#define DMA_PBURST_INC16        ((uint32_t)DMA_SxCR_PBURST)
```

- **HAL_LockTypeDef Lock**

  __IO HAL_DMA_StateTypeDef State

  这两个变量主要供函数内部使用。Lock 用于设置锁状态，而 State 用于设置 DMA 状态。

- **void (* XferCpltCallback)(struct __DMA_HandleTypeDef * hdma);**

  void (* XferHalfCpltCallback)(struct __DMA_HandleTypeDef * hdma);

  void (* XferM1CpltCallback)(struct __DMA_HandleTypeDef * hdma);

  void (* XferM1HalfCpltCallback)(struct __DMA_HandleTypeDef * hdma);

  void (* XferErrorCallback)(struct __DMA_HandleTypeDef * hdma);

  void (* XferAbortCallback)(struct __DMA_HandleTypeDef * hdma);

这里是定义了六个回调函数指针，分别用于配置传输完成回调，半传输完成回调，Memory1 传输完成回调，Memory1 半传输完成回调，传输错误回调和传输终止回调。

42.3.3 DMA 的状态标志清除问题

下面我们介绍__HAL_DMA_GET_FLAG 函数。这个函数用来检查定时器标志位是否被设置。

```
/**
  * @brief  Get the DMA Stream pending flags.
  * @param  __HANDLE__: DMA handle
  * @param  __FLAG__: Get the specified flag.
  *          This parameter can be any combination of the following values:
  *            @arg DMA_FLAG_TCIFx: Transfer complete flag.
  *            @arg DMA_FLAG_HTIFx: Half transfer complete flag.
```

```
  *            @arg DMA_FLAG_TEIFx: Transfer error flag.
  *            @arg DMA_FLAG_DMEIFx: Direct mode error flag.
  *            @arg DMA_FLAG_FEIFx: FIFO error flag.
  *          Where x can be 0_4, 1_5, 2_6 or 3_7 to select the DMA Stream flag.
  * @retval The state of FLAG (SET or RESET).
  */
#define __HAL_DMA_GET_FLAG(__HANDLE__, __FLAG__)\
(((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA2_Stream7)? (BDMA->ISR & (__FLAG__))  :\
 ((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA2_Stream3)? (DMA2->HISR & (__FLAG__)) :\
 ((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA1_Stream7)? (DMA2->LISR & (__FLAG__)) :\
 ((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA1_Stream3)? (DMA1->HISR & (__FLAG__)) : (DMA1->LISR &
                                                                                            (__FLAG__)))
```

对于 BDMA，主要是前三个中断标志。

- DMA_FLAG_TCIFx

  传输完成标志。

- DMA_FLAG_HTIFx

  半传输完成标志。

- DMA_FLAG_TEIFx

  传输错误标志。

- DMA_FLAG_DMEIFx

  直接模式错误标志。

- DMA_FLAG_FEIFx

  FIFO 错误标志。

DMA 支持的标志参数如下：

```
/** @defgroup DMA_flag_definitions DMA flag definitions
  * @brief    DMA flag definitions
  * @{
  */
#define DMA_FLAG_FEIF0_4                 ((uint32_t)0x00800001U)
#define DMA_FLAG_DMEIF0_4                ((uint32_t)0x00800004U)
#define DMA_FLAG_TEIF0_4                 ((uint32_t)0x00000008U)
#define DMA_FLAG_HTIF0_4                 ((uint32_t)0x00000010U)
#define DMA_FLAG_TCIF0_4                 ((uint32_t)0x00000020U)
#define DMA_FLAG_FEIF1_5                 ((uint32_t)0x00000040U)
#define DMA_FLAG_DMEIF1_5                ((uint32_t)0x00000100U)
#define DMA_FLAG_TEIF1_5                 ((uint32_t)0x00000200U)
#define DMA_FLAG_HTIF1_5                 ((uint32_t)0x00000400U)
#define DMA_FLAG_TCIF1_5                 ((uint32_t)0x00000800U)
#define DMA_FLAG_FEIF2_6                 ((uint32_t)0x00010000U)
#define DMA_FLAG_DMEIF2_6                ((uint32_t)0x00040000U)
#define DMA_FLAG_TEIF2_6                 ((uint32_t)0x00080000U)
#define DMA_FLAG_HTIF2_6                 ((uint32_t)0x00100000U)
#define DMA_FLAG_TCIF2_6                 ((uint32_t)0x00200000U)
#define DMA_FLAG_FEIF3_7                 ((uint32_t)0x00400000U)
#define DMA_FLAG_DMEIF3_7                ((uint32_t)0x01000000U)
#define DMA_FLAG_TEIF3_7                 ((uint32_t)0x02000000U)
#define DMA_FLAG_HTIF3_7                 ((uint32_t)0x04000000U)
#define DMA_FLAG_TCIF3_7                 ((uint32_t)0x08000000U)
```

这里要注意以下几点：

- 比如 DMA_FLAG_FEIF0_4，表示 DMA1 或者 DMA2 的 Stream0 和 Stream4 的标志，而不是 Stream0 到 Stream4，同理 1 和 5 就是 Srteam1 和 Stream5。
- DMA1 和 DMA2 的 Stream0 到 Stream3 标志是通过 LISR 寄存器查看（DMA1 和 DMA2 的寄存器是相同的，只是位于不同的地址）。
  Stream4 到 Stream7 的标志是通过 HISR 寄存器查看。
- #define DMA_FLAG_FEIF0_4 ((uint32_t)0x00800001U)
  #define DMA_FLAG_DMEIF0_4 ((uint32_t)0x00800004U)

  置红的数字 8 是什么意义？看了参考手册，原来这个 bit 没有定义，实际上这 bit 在 DMA 的库代码中也没有被用到过。

14.5.1 DMA low interrupt status register (DMA_LISR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	TCIF3	HTIF3	TEIF3	DMEIF3	Res.	FEIF3	TCIF2	HTIF2	TEIF2	DMEIF2	Res.	FEIF2
				r	r	r	r		r	r	r	r	r		r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	TCIF1	HTIF1	TEIF1	DMEIF1	Res.	FEIF1	TCIF0	HTIF0	TEIF0	DMEIF0	Res.	FEIF0
				r	r	r	r		r	r	r	r	r		r

14.5.2 DMA high interrupt status register (DMA_HISR)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	TCIF7	HTIF7	TEIF7	DMEIF7	Res.	FEIF7	TCIF6	HTIF6	TEIF6	DMEIF6	Res.	FEIF6
				r	r	r	r		r	r	r	r	r		r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	TCIF5	HTIF5	TEIF5	DMEIF5	Res.	FEIF5	TCIF4	HTIF4	TEIF4	DMEIF4	Res.	FEIF4
				r	r	r	r		r	r	r	r	r		r

与标志获取函数__HAL_TIM_GET_FLAG 对应的清除函数是__HAL_LPTIM_CLEAR_FLAG：

```
/**
  * @brief  Clear the DMA Stream pending flags.
  * @param  __HANDLE__: DMA handle
  * @param  __FLAG__: specifies the flag to clear.
  *          This parameter can be any combination of the following values:
  *            @arg DMA_FLAG_TCIFx: Transfer complete flag.
  *            @arg DMA_FLAG_HTIFx: Half transfer complete flag.
  *            @arg DMA_FLAG_TEIFx: Transfer error flag.
  *            @arg DMA_FLAG_DMEIFx: Direct mode error flag.
  *            @arg DMA_FLAG_FEIFx: FIFO error flag.
```

```
  *         Where x can be 0_4, 1_5, 2_6 or 3_7 to select the DMA Stream flag.
  * @retval None
  */
#define __HAL_DMA_CLEAR_FLAG(__HANDLE__, __FLAG__) \
(((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA2_Stream7)? (BDMA->IFCR = (__FLAG__))  :\
 ((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA2_Stream3)? (DMA2->HIFCR = (__FLAG__)) :\
 ((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA1_Stream7)? (DMA2->LIFCR = (__FLAG__)) :\
 ((uint32_t)((__HANDLE__)->Instance) > (uint32_t)DMA1_Stream3)? (DMA1->HIFCR = (__FLAG__)) : (DMA1->LIFCR =
(__FLAG__)))
```

清除标志函数所支持的参数跟获取函数是一 一对应的。除了这两个函数，还有 DMA 的中断开启和中断关闭函数，有时候也要用到。

```
/**
  * @brief  Enable the specified DMA Stream interrupts.
  * @param  __HANDLE__: DMA handle
  * @param  __INTERRUPT__: specifies the DMA interrupt sources to be enabled or disabled.
  *        This parameter can be one of the following values:
  *           @arg DMA_IT_TC: Transfer complete interrupt mask.
  *           @arg DMA_IT_HT: Half transfer complete interrupt mask.
  *           @arg DMA_IT_TE: Transfer error interrupt mask.
  *           @arg DMA_IT_FE: FIFO error interrupt mask.
  *           @arg DMA_IT_DME: Direct mode error interrupt.
  * @retval None
  */
#define __HAL_DMA_ENABLE_IT(__HANDLE__, __INTERRUPT__) ((IS_D2_DMA_INSTANCE(__HANDLE__))?\
                                                        (__HAL_DMA_D2_ENABLE_IT((__HANDLE__),
                                                                                (__INTERRUPT__))) :\
                                                        (__HAL_DMA_D3_ENABLE_IT((__HANDLE__),
                                                                                (__INTERRUPT__))))

/**
  * @brief  Disable the specified DMA Stream interrupts.
  * @param  __HANDLE__: DMA handle
  * @param  __INTERRUPT__: specifies the DMA interrupt sources to be enabled or disabled.
  *         This parameter can be one of the following values:
  *            @arg DMA_IT_TC: Transfer complete interrupt mask.
  *            @arg DMA_IT_HT: Half transfer complete interrupt mask.
  *            @arg DMA_IT_TE: Transfer error interrupt mask.
  *            @arg DMA_IT_FE: FIFO error interrupt mask.
  *            @arg DMA_IT_DME: Direct mode error interrupt.
  * @retval None
  */
#define __HAL_DMA_DISABLE_IT(__HANDLE__, __INTERRUPT__) ((IS_D2_DMA_INSTANCE(__HANDLE__))?\
                                                         (__HAL_DMA_D2_DISABLE_IT((__HANDLE__),
                                                                                 (__INTERRUPT__))) :\
                                                         (__HAL_DMA_D3_DISABLE_IT((__HANDLE__),
                                                                                 (__INTERRUPT__))))
```

注意：操作 DMA 的寄存器不限制必须要用 HAL 库提供的 API，比如要操作 DMA1 的寄存器 LISR，直接调用 DMA1->LISR 操作即可。

42.3.4DMA 初始化流程总结

使用方法由 HAL 库提供：

- 第 1 步：通过函数 HAL_DMA_Init 配置各项参数。

- 第 2 步：DMA 查询方式。
  - 配置了源地址、目的地址和数据长度后，调用函数 HAL_DMA_Start()启动传输。
  - 使用函数 HAL_DMA_PollForTransfer()查询当前传输是否结束，用户还可以给此函数配置超时等待时间。

- 第 3 步：DMA 中断方式。
  - 使用函数 HAL_NVIC_SetPriority 配置 DMA 优先级。
  - 使用函数 HAL_NVIC_EnableIRQ 使能 DMA 中断。
  - 配置了源地址、目的地址和数据长度后，调用函数 HAL_DMA_Start_IT()可以启动传输（注，此函数会使能 BDMA 中断）。
  - 将函数 HAL_DMA_IRQHandler()填到中断服务程序 DMAx_Streamx_IRQHandler 里面。
  - 传输结束后会调用函数 HAL_DMA_IRQHandler()，此函数里面会执行回调函数，即用户需要为 XferCpltCallback，XferErrorCallback 等函数配置实体（如果用到的话）

- 第 4 步：使用函数 HAL_DMA_GetState()可以获得 DMA 状态，函数 HAL_DMA_GetError()获取获取错误类型。

- 第 5 步：使用函数 HAL_DMA_Abort()可以终止 DMA 传输。
  - 存储器到存储器方式，不支持循环模式。
  - DMA FIFO 的作用是降低对总线的需求和源地址，目的地址不同数据宽度的传输。
  - 当 FIFO 禁止后，不允许配置源数据和目的数据宽度不同，此时将统一使用外设数据宽度。

- 第 6 步：下面是几个常用的 DMA 宏定义。
  - __HAL_DMA_ENABLE: 使能指定的 DMA Stream
  - __HAL_DMA_DISABLE: 禁止指定的 DMA Stream
  - __HAL_DMA_GET_FS: 返回当前 DMA Stream FIFO 填充情况
  - __HAL_DMA_ENABLE_IT: 使能指定的 DMA Stream 中断
  - __HAL_DMA_DISABLE_IT: 禁止指定的 DMA Stream 中断
  - __HAL_DMA_GET_IT_SOURCE: 检查指定的 DMA Stream 中断是否使能

42.4 源文件 stm32h7xx_hal_dma.c

此文件涉及到的函数比较多，这里把我们几个常用的函数做个说明：

- HAL_DMA_Init
- HAL_DMA_Start
- HAL_DMA_Start_IT
- HAL_DMAEx_MultiBufferStart
- HAL_DMAEx_MultiBufferStart_IT

42.4.1 函数 HAL_DMA_Init

函数原型：

```
HAL_StatusTypeDef HAL_DMA_Init(DMA_HandleTypeDef *hdma)
{
  uint32_t registerValue = 0U;
  uint32_t tickstart = HAL_GetTick();
  DMA_Base_Registers *regs = NULL;

   /* 省略 */

  /* DMA1 或者 DMA2 的初始化 */
  if(IS_D2_DMA_INSTANCE(hdma) != RESET)
  {
     /* 省略 */
  }

  /* BDMA 的初始 */
  else if(IS_D3_DMA_INSTANCE(hdma) != RESET) /*<BDMA channel , D3 domain*/
  {
     /* 省略 */
  }
  else
  {
    hdma->ErrorCode = HAL_DMA_ERROR_PARAM;
    hdma->State     = HAL_DMA_STATE_ERROR;

    return HAL_ERROR;
  }

  /* 初始化 DMAMUX */
  DMA_CalcDMAMUXChannelBaseAndMask(hdma);

  if(hdma->Init.Direction == DMA_MEMORY_TO_MEMORY)
  {
     /* 如果是内存到内存模式，强制使用请求 DMA_REQUEST_MEM2MEM */
     hdma->Init.Request = DMA_REQUEST_MEM2MEM;
  }


  /* 设置外设请求 */
```

```
  hdma->DMAmuxChannel->CCR = (hdma->Init.Request & DMAMUX_CxCR_DMAREQ_ID);

  /* 清除 DMAMUX 同步溢出标志 */
  hdma->DMAmuxChannelStatus->CFR = hdma->DMAmuxChannelStatusMask;

  /* 如果请求类型是 DMA_REQUEST_GENERATOR0 到 DMA_REQUEST_GENERATOR7，那么设置请求发生器 */
  if((hdma->Init.Request >= DMA_REQUEST_GENERATOR0) && (hdma->Init.Request <= DMA_REQUEST_GENERATOR7))
  {
    /* 省略 */
  }
  else
  {
    hdma->DMAmuxRequestGen = 0U;
    hdma->DMAmuxRequestGenStatus = 0U;
    hdma->DMAmuxRequestGenStatusMask = 0U;
  }

  /* 无错误 */
  hdma->ErrorCode = HAL_DMA_ERROR_NONE;

  /* 设置 DMA 就绪 */
  hdma->State = HAL_DMA_STATE_READY;

  return HAL_OK;
}
```

函数描述：

此函数用于初始化 DMA1，DMA2 和 BDMA。

函数参数：

- 第 1 个参数是 DMA_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。结构体变量成员的详细介绍看本章 3.2 小节。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 第 1 个参数的结构体成员介绍在本章的 3.2 小节进行了详细说明。

使用举例：

```
DMA_HandleTypeDef DMA_Handle = {0};

DMA_Handle.Instance                 = BDMA_Channel0;            /* 使用的 BDMA 通道 0 */
DMA_Handle.Init.Request             = BDMA_REQUEST_GENERATOR0; /* 请求类型采用的 DMAMUX 请求发生器通道 0 */
DMA_Handle.Init.Direction           = DMA_MEMORY_TO_PERIPH;    /* 传输方向是从存储器到外设 */
DMA_Handle.Init.PeriphInc           = DMA_PINC_DISABLE;        /* 外设地址自增禁止 */
DMA_Handle.Init.MemInc              = DMA_MINC_ENABLE;         /* 存储器地址自增使能 */
DMA_Handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;     /* 外设数据传输位宽选择字，即 32bit */
DMA_Handle.Init.MemDataAlignment    = DMA_MDATAALIGN_WORD;     /* 存储器数据传输位宽选择字，即 32bit */
DMA_Handle.Init.Mode                = DMA_CIRCULAR;            /* 循环模式 */
DMA_Handle.Init.Priority            = DMA_PRIORITY_LOW;        /* 优先级低 */
DMA_Handle.Init.FIFOMode            = DMA_FIFOMODE_DISABLE;    /* BDMA 不支持 FIFO */
DMA_Handle.Init.FIFOThreshold       = DMA_FIFO_THRESHOLD_FULL; /* BDMA 不支持 FIFO 阈值设置 */
DMA_Handle.Init.MemBurst            = DMA_MBURST_SINGLE;       /* BDMA 不支持存储器突发 */
DMA_Handle.Init.PeriphBurst         = DMA_PBURST_SINGLE;       /* BDMA 不支持外设突发 */
```

```
HAL_DMA_Init(&DMA_Handle);
```

42.4.2 函数 HAL_DMA_Start

函数原型：

```
HAL_StatusTypeDef HAL_DMA_Start(DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t DataLength)
{
  HAL_StatusTypeDef status = HAL_OK;

  /* 检测参数 */
  assert_param(IS_DMA_BUFFER_SIZE(DataLength));

  /* 检测句柄 */
  if(hdma == NULL)
  {
    return HAL_ERROR;
  }

  /* 上锁 */
  __HAL_LOCK(hdma);

  if(HAL_DMA_STATE_READY == hdma->State)
  {
    /* 设置 DMA 状态 */
    hdma->State = HAL_DMA_STATE_BUSY;

    /* 无错误 */
    hdma->ErrorCode = HAL_DMA_ERROR_NONE;

    /* 禁止 DMA */
    __HAL_DMA_DISABLE(hdma);

    /* 配置源地址，目的地址和数据长度 */
    DMA_SetConfig(hdma, SrcAddress, DstAddress, DataLength);

    /* 使能 DMA */
    __HAL_DMA_ENABLE(hdma);
  }
  else
  {
    /* 解锁 */
    __HAL_UNLOCK(hdma);

    /* 设置忙 */
    hdma->ErrorCode = HAL_DMA_ERROR_BUSY;

    /* 返回 HAL_ERROR */
    status = HAL_ERROR;
  }
  return status;
}
```

函数描述：

调用函数 HAL_DMA_Init 配置了基础功能后，就可以调用此函数启动 DMA 了。DMA1，DMA2 和 BDMA 都是用的这个函数。

函数参数：

- 第 1 个参数是 DMA_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是 DMA 传输的源地址。
- 第 3 个参数是 DMA 传输的目的地址。
- 第 4 个参数是传输的数据长度。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 第 1 个参数的结构体成员介绍在本章的 3.2 小节进行了详细说明。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitTimBDMA
*    功能说明: 配置 DMAMUX 的定时器触+DMA 控制任意 IO 做 PWM 和脉冲数控制
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitTimBDMA(void)
{
    GPIO_InitTypeDef  GPIO_InitStruct;
    DMA_HandleTypeDef DMA_Handle = {0};
    HAL_DMA_MuxRequestGeneratorConfigTypeDef dmamux_ReqGenParams ={0};


     /*##-1- 配置 PB1 用于 PWM 输出#########################################*/
    __HAL_RCC_GPIOB_CLK_ENABLE();

    GPIO_InitStruct.Pin = GPIO_PIN_1;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);


    /*##-2- 配置 DMA ######################################################*/
    __HAL_RCC_BDMA_CLK_ENABLE();

    DMA_Handle.Instance                 = BDMA_Channel0;            /* 使用的 BDMA 通道 0 */
    DMA_Handle.Init.Request             = BDMA_REQUEST_GENERATOR0; /* 请求类型采用的 DMAMUX 请求发生器通道 0
*/
    DMA_Handle.Init.Direction           = DMA_MEMORY_TO_PERIPH;     /* 传输方向是从存储器到外设 */
    DMA_Handle.Init.PeriphInc           = DMA_PINC_DISABLE;         /* 外设地址自增禁止 */
    DMA_Handle.Init.MemInc              = DMA_MINC_ENABLE;          /* 存储器地址自增使能 */
    DMA_Handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;      /* 外设数据传输位宽选择字，即 32bit */
    DMA_Handle.Init.MemDataAlignment    = DMA_MDATAALIGN_WORD;      /* 存储器数据传输位宽选择字，即 32bit */
```

```
    DMA_Handle.Init.Mode                = DMA_CIRCULAR;             /* 循环模式 */
    DMA_Handle.Init.Priority            = DMA_PRIORITY_LOW;         /* 优先级低 */
    DMA_Handle.Init.FIFOMode            = DMA_FIFOMODE_DISABLE;     /* BDMA 不支持 FIFO */
    DMA_Handle.Init.FIFOThreshold       = DMA_FIFO_THRESHOLD_FULL; /* BDMA 不支持 FIFO 阀值设置 */
    DMA_Handle.Init.MemBurst            = DMA_MBURST_SINGLE;        /* BDMA 不支持存储器突发 */
    DMA_Handle.Init.PeriphBurst         = DMA_PBURST_SINGLE;        /* BDMA 不支持外设突发 */

    HAL_DMA_Init(&DMA_Handle);

    /* 开启 BDMA Channel0 的中断 */
    HAL_NVIC_SetPriority(BDMA_Channel0_IRQn, 2, 0);
    HAL_NVIC_EnableIRQ(BDMA_Channel0_IRQn);

    /*##-3- 配置 DMAMUX ######################################################*/
    dmamux_ReqGenParams.SignalID  = HAL_DMAMUX2_REQ_GEN_LPTIM2_OUT;     /* 请求触发器选择 LPTIM2_OUT */
    dmamux_ReqGenParams.Polarity  = HAL_DMAMUX_REQ_GEN_RISING_FALLING; /* LPTIM2 输出的上升沿和下降沿均可触
                                                                          发 */
    dmamux_ReqGenParams.RequestNumber = 1;                             /* 触发后，传输进行 1 次 DMA 传输 */
    HAL_DMAEx_ConfigMuxRequestGenerator(&DMA_Handle, &dmamux_ReqGenParams); /* 配置 DMAMUX */
    HAL_DMAEx_EnableMuxRequestGenerator (&DMA_Handle);                 /* 使能 DMAMUX 请求发生器 */

    /*##-4- 启动 DMA 传输 ####################################################*/
    HAL_DMA_Start(&DMA_Handle, (uint32_t)IO_Toggle, (uint32_t)&GPIOB->BSRRL, 8);
}
```

42.4.3 函数 HAL_DMA_Start_IT

函数原型：

```
HAL_StatusTypeDef HAL_DMA_Start_IT(DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t
DataLength)
{
  HAL_StatusTypeDef status = HAL_OK;

  /* 检测参数 */
  assert_param(IS_DMA_BUFFER_SIZE(DataLength));

  /* 检测句柄 */
  if(hdma == NULL)
  {
    return HAL_ERROR;
  }

  /* 上锁  */
  __HAL_LOCK(hdma);

  if(HAL_DMA_STATE_READY == hdma->State)
  {
    /* 设置 DMA 忙 */
    hdma->State = HAL_DMA_STATE_BUSY;

    /* 设置无错误 */
    hdma->ErrorCode = HAL_DMA_ERROR_NONE;

    /* 禁止 DMA */
    __HAL_DMA_DISABLE(hdma);
```

```
  /* 配置 DMA 源地址，目的地址和数据长度 */
  DMA_SetConfig(hdma, SrcAddress, DstAddress, DataLength);

  /* DMA1 和 DMA2 配置 */
  if(IS_D2_DMA_INSTANCE(hdma) != RESET)
  {
    /* 使能 TC，TE 和 DME 中断 */
    MODIFY_REG(((DMA_Stream_TypeDef   *)hdma->Instance)->CR, (DMA_IT_TC | DMA_IT_TE | DMA_IT_DME |
                                                DMA_IT_HT), (DMA_IT_TC | DMA_IT_TE | DMA_IT_DME));
    ((DMA_Stream_TypeDef   *)hdma->Instance)->FCR |= DMA_IT_FE;

    if(hdma->XferHalfCpltCallback != NULL)
    {
      /* 如何设置了半传输回调函数，同时开启半传输中断 */
      ((DMA_Stream_TypeDef   *)hdma->Instance)->CR  |= DMA_IT_HT;
    }
  }
  else /* BDMA 配置 */
  {
    /* 使能 TC 和 TC 中断 */
    MODIFY_REG(((BDMA_Channel_TypeDef   *)hdma->Instance)->CCR, (BDMA_CCR_TCIE | BDMA_CCR_HTIE |
                                                  BDMA_CCR_TEIE), (BDMA_CCR_TCIE | BDMA_CCR_TEIE));

    if(hdma->XferHalfCpltCallback != NULL)
    {
       /* 如何设置了半传输回调函数，同时开启半传输中断 */
      ((BDMA_Channel_TypeDef   *)hdma->Instance)->CCR  |= BDMA_CCR_HTIE;
    }
  }

  /* 检测是否使能 DMAMUX 同步传输 */
  if((hdma->DMAmuxChannel->CCR & DMAMUX_CxCR_SE) != 0U)
  {
    /* 使能了的话，开启同步溢出中断 */
    hdma->DMAmuxChannel->CCR |= DMAMUX_CxCR_SOIE;
  }

  if(hdma->DMAmuxRequestGen != 0U)
  {
    /* 如果使用了 DMAMUX 请求发生器，使能请求发生器溢出中断 */
    hdma->DMAmuxRequestGen->RGCR |= DMAMUX_RGxCR_OIE;

  }

  /* 使能 DMA */
  __HAL_DMA_ENABLE(hdma);
}
else
{
  /* 解锁 */
  __HAL_UNLOCK(hdma);

  /* 设置 DMA 忙 */
  hdma->ErrorCode = HAL_DMA_ERROR_BUSY;

  /* 返回 HAL_ERROR */
  status = HAL_ERROR;
```

```
  }

  return status;
}
```

函数描述：

调用函数 HAL_DMA_Init 配置了基础功能后，就可以调用此函数启动 DMA 了。DMA1，DMA2 和 BDMA 都是用的这个函数。

函数参数：

- 第 1 个参数是 DMA_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是 DMA 传输的源地址。
- 第 3 个参数是 DMA 传输的目的地址。
- 第 4 个参数是传输的数据长度。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 第 1 个参数的结构体成员介绍在本章的 3.2 小节进行了详细说明。
2. 对于 DMA1 和 DMA2，这个函数会开启 TC，TE 和 MDE 中断，如果注册了半传输完成回调函数，还会开启半传输中断。
3. 对于 BDMA，这个函数会开始 TC 和 TE 中断，如果注册了半传输完成回调函数，还会开启半传输中断。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitTimBDMA
*    功能说明: 配置 DMAMUX 的定时器触+DMA 控制任意 IO 做 PWM 和脉冲数控制
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitTimBDMA(void)
{
    GPIO_InitTypeDef  GPIO_InitStruct;
    DMA_HandleTypeDef DMA_Handle = {0};
    HAL_DMA_MuxRequestGeneratorConfigTypeDef dmamux_ReqGenParams ={0};


     /*##-1- 配置 PB1 用于 PWM 输出#########################################*/
    __HAL_RCC_GPIOB_CLK_ENABLE();

    GPIO_InitStruct.Pin = GPIO_PIN_1;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
```

```
    /*##-2- 配置 DMA ##################################################*/
    __HAL_RCC_BDMA_CLK_ENABLE();

    DMA_Handle.Instance                 = BDMA_Channel0;            /* 使用的 BDMA 通道 0 */
    DMA_Handle.Init.Request             = BDMA_REQUEST_GENERATOR0; /* 请求类型采用的 DMAMUX 请求发生器通道 0
*/
    DMA_Handle.Init.Direction           = DMA_MEMORY_TO_PERIPH;     /* 传输方向是从存储器到外设 */
    DMA_Handle.Init.PeriphInc           = DMA_PINC_DISABLE;         /* 外设地址自增禁止 */
    DMA_Handle.Init.MemInc              = DMA_MINC_ENABLE;          /* 存储器地址自增使能 */
    DMA_Handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;      /* 外设数据传输位宽选择字，即 32bit */
    DMA_Handle.Init.MemDataAlignment    = DMA_MDATAALIGN_WORD;      /* 存储器数据传输位宽选择字，即 32bit */
    DMA_Handle.Init.Mode                = DMA_CIRCULAR;             /* 循环模式 */
    DMA_Handle.Init.Priority            = DMA_PRIORITY_LOW;         /* 优先级低 */
    DMA_Handle.Init.FIFOMode            = DMA_FIFOMODE_DISABLE;     /* BDMA 不支持 FIFO */
    DMA_Handle.Init.FIFOThreshold       = DMA_FIFO_THRESHOLD_FULL; /* BDMA 不支持 FIFO 阀值设置 */
    DMA_Handle.Init.MemBurst            = DMA_MBURST_SINGLE;        /* BDMA 不支持存储器突发 */
    DMA_Handle.Init.PeriphBurst         = DMA_PBURST_SINGLE;        /* BDMA 不支持外设突发 */

    HAL_DMA_Init(&DMA_Handle);

    /* 开启 BDMA Channel0 的中断 */
    HAL_NVIC_SetPriority(BDMA_Channel0_IRQn, 2, 0);
    HAL_NVIC_EnableIRQ(BDMA_Channel0_IRQn);

    /*##-3- 配置 DMAMUX ###############################################################*/
    dmamux_ReqGenParams.SignalID  = HAL_DMAMUX2_REQ_GEN_LPTIM2_OUT;     /* 请求触发器选择 LPTIM2_OUT */
    dmamux_ReqGenParams.Polarity  = HAL_DMAMUX_REQ_GEN_RISING_FALLING; /* LPTIM2 输出的上升沿和下降沿均可触
                                                                          发 */
    dmamux_ReqGenParams.RequestNumber = 1;                              /* 触发后，传输进行 1 次 DMA 传输 */
    HAL_DMAEx_ConfigMuxRequestGenerator(&DMA_Handle, &dmamux_ReqGenParams); /* 配置 DMAMUX */
    HAL_DMAEx_EnableMuxRequestGenerator (&DMA_Handle);                 /* 使能 DMAMUX 请求发生器 */

    /*##-4- 启动 DMA 传输 ##############################################################*/
    HAL_DMA_Start_IT(&DMA_Handle, (uint32_t)IO_Toggle, (uint32_t)&GPIOB->BSRRL, 8);
}
```

42.4.4函数 HAL_DMAEx_MultiBufferStart

函数原型：

```
HAL_StatusTypeDef HAL_DMAEx_MultiBufferStart(DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t DstAddress,
uint32_t SecondMemAddress, uint32_t DataLength)
{
  HAL_StatusTypeDef status = HAL_OK;
  __IO uint32_t *ifcRegister_Base = NULL;

  /* 检测参数 */
  assert_param(IS_DMA_BUFFER_SIZE(DataLength));

  /* 如果是存储器到存储器的传输，不支持双缓冲 */
  if ( (IS_D2_DMA_INSTANCE(hdma) == 0U) || (hdma->Init.Direction == DMA_MEMORY_TO_MEMORY))
  {
    hdma->ErrorCode = HAL_DMA_ERROR_NOT_SUPPORTED;
    status = HAL_ERROR;
  }
```

```
  else
  {
    /* 上锁 */
    __HAL_LOCK(hdma);

    if(HAL_DMA_STATE_READY == hdma->State)
    {
      /* 设置 DMA 忙 */
      hdma->State = HAL_DMA_STATE_BUSY;

      /* 设置无错误 */
      hdma->ErrorCode = HAL_DMA_ERROR_NONE;

      /* 使能 DMA 双缓冲 */
      ((DMA_Stream_TypeDef   *)hdma->Instance)->CR |= (uint32_t)DMA_SxCR_DBM;

      /* 配置 DMA Stream 的另一个缓冲地址 */
      ((DMA_Stream_TypeDef   *)hdma->Instance)->M1AR = SecondMemAddress;

      /* 配置源地址，目的地址和数据长度 */
      DMA_MultiBufferSetConfig(hdma, SrcAddress, DstAddress, DataLength);

      /* 计算寄存器 IFCR 地址，因为 DMA1 和 DMA2 的这寄存器地址不同，所以要获取下 */
      ifcRegister_Base = (uint32_t *)((uint32_t)(hdma->StreamBaseAddress + 8U));

      /* 清除所有标志 */
      *ifcRegister_Base = 0x3FU << hdma->StreamIndex;

      /* 清除 DMAMUX 同步溢出标志 */
      hdma->DMAmuxChannelStatus->CFR = hdma->DMAmuxChannelStatusMask;

      if(hdma->DMAmuxRequestGen != 0U)
      {
        /* 如果使用了 DMAMUX 的请求发生器，清除请求发生器溢出标志 */
        hdma->DMAmuxRequestGenStatus->RGCFR = hdma->DMAmuxRequestGenStatusMask;
      }

      /* 使能 DMA */
      __HAL_DMA_ENABLE(hdma);
    }
    else
    {
      /* 设置 DMA 错误标志，DMA 忙 */
      hdma->ErrorCode = HAL_DMA_ERROR_BUSY;

      /* 返回错误状态 */
      status = HAL_ERROR;
    }
  }
  return status;
}
```

函数描述：

调用函数 HAL_DMA_Init 配置了基础功能后，就可以调用此函数启动 DMA 双缓冲了。

函数参数：

◆ 第 1 个参数是 DMA_HandleTypeDef 类型结构体指针变量。

- 第 2 个参数是 DMA 传输的源地址。
- 第 3 个参数是 DMA 传输的目的地址。
- 第 4 个参数是双缓冲模式时，另一个缓冲的地址。
- 第 5 个参数是传输的数据长度。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 第 1 个参数的结构体成员介绍在本章的 3.2 小节进行了详细说明。
2. 存储器到存储器的传输方式不支持双缓冲，仅存储器到外设或者外设到存储器支持双缓冲。
3. 当前的 HAL 库 V1.3.0 版本未支持 BDMA 的双缓冲，仅对 DMA1 和 DMA2 做了支持。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitTimDMA
*    功能说明: 配置 DMAMUX 的定时器触+DMA 双缓冲控制任意 IO 做 PWM 和脉冲数控制
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitTimDMA(void)
{
    GPIO_InitTypeDef  GPIO_InitStruct;
    DMA_HandleTypeDef DMA_Handle = {0};
    HAL_DMA_MuxRequestGeneratorConfigTypeDef dmamux_ReqGenParams = {0};

    /*##-1- 配置 PB1 用于 PWM 输出 ###########################################################*/
    __HAL_RCC_GPIOB_CLK_ENABLE();
    GPIO_InitStruct.Pin = GPIO_PIN_1;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    /*##-2- 使能 DMA1 时钟并配置 ##############################################################*/
    __HAL_RCC_DMA1_CLK_ENABLE();
    DMA_Handle.Instance        = DMA1_Stream1;            /* 使用的 DMA1 Stream1 */
    DMA_Handle.Init.Request    = DMA_REQUEST_GENERATOR0;  /* 请求类型采用的 DMAMUX 请求发生器通道 0 */
    DMA_Handle.Init.Direction           = DMA_MEMORY_TO_PERIPH;    /* 传输方向是从存储器到外设 */
    DMA_Handle.Init.PeriphInc           = DMA_PINC_DISABLE;        /* 外设地址自增禁止 */
    DMA_Handle.Init.MemInc              = DMA_MINC_ENABLE;         /* 存储器地址自增使能 */
    DMA_Handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;     /* 外设数据传输位宽选择字，即 32bit */
    DMA_Handle.Init.MemDataAlignment    = DMA_MDATAALIGN_WORD;     /* 存储器数据传输位宽选择字，即 32bit */
    DMA_Handle.Init.Mode                = DMA_CIRCULAR;            /* 循环模式 */
    DMA_Handle.Init.Priority            = DMA_PRIORITY_LOW;        /* 优先级低 */
    DMA_Handle.Init.FIFOMode            = DMA_FIFOMODE_DISABLE;    /* 禁止 FIFO*/
    DMA_Handle.Init.FIFOThreshold       = DMA_FIFO_THRESHOLD_FULL; /* 禁止 FIFO 此位不起作用，用于设置阀值 */
    DMA_Handle.Init.MemBurst        = DMA_MBURST_SINGLE;       /* 禁止 FIFO 此位不起作用，用于存储器突发 */
    DMA_Handle.Init.PeriphBurst     = DMA_PBURST_SINGLE;       /* 禁止 FIFO 此位不起作用，用于外设突发 */

    /* 初始化 DMA */
```

```
    if(HAL_DMA_Init(&DMA_Handle) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 开启 DMA1 Stream1 的中断 */
    HAL_NVIC_SetPriority(DMA1_Stream1_IRQn, 2, 0);
    HAL_NVIC_EnableIRQ(DMA1_Stream1_IRQn);


    /*##-4- 配置 DMAMUX ######################################################*/
    dmamux_ReqGenParams.SignalID  = HAL_DMAMUX1_REQ_GEN_TIM12_TRGO;        /* 请求触发器选择 LPTIM2_OUT */
    dmamux_ReqGenParams.Polarity  = HAL_DMAMUX_REQ_GEN_RISING;             /* 上升沿触发  */
    dmamux_ReqGenParams.RequestNumber = 1;                                 /* 触发后,传输进行 1 次 DMA 传输 */

    HAL_DMAEx_ConfigMuxRequestGenerator(&DMA_Handle, &dmamux_ReqGenParams); /* 配置 DMAMUX */
    HAL_DMAEx_EnableMuxRequestGenerator (&DMA_Handle);                      /* 使能 DMAMUX 请求发生器 */


    /*##-4- 启动 DMA 双缓冲传输 ##############################################*/
    /*
        1、此函数会开启 DMA 的 TC，TE 和 DME 中断
        2、如果用户配置了回调函数 DMA_Handle.XferHalfCpltCallback，那么函数 HAL_DMA_Init 会开启半传输完成中断。
        3、如果用户使用了 DMAMUX 的同步模式，此函数会开启同步溢出中断。
        4、如果用户使用了 DMAMUX 的请求发生器，此函数会开始请求发生器溢出中断。
    */
    HAL_DMAEx_MultiBufferStart(&DMA_Handle, (uint32_t)IO_Toggle,
                                    (uint32_t)&GPIOB->BSRRL,(uint32_t)IO_Toggle1, 8);

}
```

42.4.5 函数 HAL_DMAEx_MultiBufferStart_IT

函数原型：

```
HAL_StatusTypeDef HAL_DMAEx_MultiBufferStart_IT(DMA_HandleTypeDef *hdma, uint32_t SrcAddress, uint32_t
DstAddress, uint32_t SecondMemAddress, uint32_t DataLength)
{
  HAL_StatusTypeDef status = HAL_OK;
  __IO uint32_t *ifcRegister_Base = NULL;

  /* 检测参数 */
  assert_param(IS_DMA_BUFFER_SIZE(DataLength));

  /* 存储器到存储器的传输不支持双缓冲 */
  if( (IS_D2_DMA_INSTANCE(hdma) == 0U) || (hdma->Init.Direction == DMA_MEMORY_TO_MEMORY))
  {
    hdma->ErrorCode = HAL_DMA_ERROR_NOT_SUPPORTED;
    return HAL_ERROR;
  }

  /* 上锁 */
  __HAL_LOCK(hdma);

  if(HAL_DMA_STATE_READY == hdma->State)
  {
```

```
    /* 设置 DMA 忙 */
    hdma->State = HAL_DMA_STATE_BUSY;

    /* 设置无错误 */
    hdma->ErrorCode = HAL_DMA_ERROR_NONE;

    /* 使能双缓冲模式 */
    ((DMA_Stream_TypeDef   *)hdma->Instance)->CR |= (uint32_t)DMA_SxCR_DBM;

    /* 设置 DMA Stream 另一个缓冲区地址 */
    ((DMA_Stream_TypeDef   *)hdma->Instance)->M1AR = SecondMemAddress;

    * 配置源地址，目的地址和数据长度 */
    DMA_MultiBufferSetConfig(hdma, SrcAddress, DstAddress, DataLength);

    /* 计算寄存器 IFCR 地址，因为 DMA1 和 DMA2 的这寄存器地址不同，所以要获取下 */
    ifcRegister_Base = (uint32_t *)((uint32_t)(hdma->StreamBaseAddress + 8U));

   /* 清除所有标志 */
    *ifcRegister_Base = 0x3FU << hdma->StreamIndex;

    /* 清除 DMAMUX 同步溢出标志 */
    hdma->DMAmuxChannelStatus->CFR = hdma->DMAmuxChannelStatusMask;

    if(hdma->DMAmuxRequestGen != 0U)
    {
      /* 如果使用了 DMAMUX 的请求发生器，清除请求发生器溢出标志 */
      hdma->DMAmuxRequestGenStatus->RGCFR = hdma->DMAmuxRequestGenStatusMask;
    }

    /* 使能 TC，TE 和 DME 中断 */
    MODIFY_REG(((DMA_Stream_TypeDef   *)hdma->Instance)->CR, (DMA_IT_TC | DMA_IT_TE | DMA_IT_DME | DMA_IT_HT),
(DMA_IT_TC | DMA_IT_TE | DMA_IT_DME));
    ((DMA_Stream_TypeDef   *)hdma->Instance)->FCR |= DMA_IT_FE;

    if(hdma->XferHalfCpltCallback != NULL)
    {
      /* 如何设置了半传输回调函数，同时开启半传输中断 */
      ((DMA_Stream_TypeDef   *)hdma->Instance)->CR  |= DMA_IT_HT;
    }

    /* 检测 DMAMUX 同步是否已经使能 */
    if((hdma->DMAmuxChannel->CCR & DMAMUX_CxCR_SE) != 0)
    {
      /* 使能 DMAMUX 同步溢出中断 */
      hdma->DMAmuxChannel->CCR |= DMAMUX_CxCR_SOIE;
    }

    if(hdma->DMAmuxRequestGen != 0U)
    {
      /* 如果使用 DMAMUX 请求发生器，使能 DMAMUX 请求发生器溢出中断 */
      hdma->DMAmuxRequestGen->RGCR |= DMAMUX_RGxCR_OIE;
    }

    /* 使能 DMA */
    __HAL_DMA_ENABLE(hdma);
  }
  else
```

```
  {
    /* 设置错误代码，DMA 忙 */
    hdma->ErrorCode = HAL_DMA_ERROR_BUSY;

    /* 返回 HAL_ERROR */
    status = HAL_ERROR;
  }
  return status;
}
```

函数描述：

调用函数 HAL_DMA_Init 配置了基础功能后，就可以调用此函数启动 DMA 双缓冲了，中断方式。

函数参数：

- 第 1 个参数是 DMA_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是 DMA 传输的源地址。
- 第 3 个参数是 DMA 传输的目的地址。
- 第 4 个参数是双缓冲模式时，另一个缓冲的地址。
- 第 5 个参数是传输的数据长度。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 第 1 个参数的结构体成员介绍在本章的 3.2 小节进行了详细说明。
2. 对于 DMA1 和 DMA2，这个函数会开启 TC，TE 和 MDE 中断，如果注册了半传输完成回调函数，还会开启半传输中断。
3. 如果使用了 DMAMUX 同步模式，还会开启同步溢出中断。
4. 如果使用了 DMAMUX 请求发生器，会开启请求发生器溢出中断。
5. 存储器到存储器的传输方式不支持双缓冲，仅存储器到外设或者外设到存储器支持双缓冲。
6. 当前的 HAL 库 V1.3.0 版本未支持 BDMA 的双缓冲，仅对 DMA1 和 DMA2 做了支持。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitTimDMA
*    功能说明: 配置 DMAMUX 的定时器触+DMA 双缓冲控制任意 IO 做 PWM 和脉冲数控制
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitTimDMA(void)
{
    GPIO_InitTypeDef  GPIO_InitStruct;
    DMA_HandleTypeDef DMA_Handle = {0};
    HAL_DMA_MuxRequestGeneratorConfigTypeDef dmamux_ReqGenParams = {0};

    /*##-1- 配置 PB1 用于 PWM 输出 #################################################*/
    __HAL_RCC_GPIOB_CLK_ENABLE();
    GPIO_InitStruct.Pin = GPIO_PIN_1;
```

```
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    /*##-2- 使能 DMA1 时钟并配置 ###########################################################*/
    __HAL_RCC_DMA1_CLK_ENABLE();
    DMA_Handle.Instance         = DMA1_Stream1;             /* 使用的 DMA1 Stream1 */
    DMA_Handle.Init.Request     = DMA_REQUEST_GENERATOR0;   /* 请求类型采用的 DMAMUX 请求发生器通道 0 */
    DMA_Handle.Init.Direction           = DMA_MEMORY_TO_PERIPH;     /* 传输方向是从存储器到外设 */
    DMA_Handle.Init.PeriphInc           = DMA_PINC_DISABLE;         /* 外设地址自增禁止 */
    DMA_Handle.Init.MemInc              = DMA_MINC_ENABLE;          /* 存储器地址自增使能 */
    DMA_Handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;      /* 外设数据传输位宽选择字，即 32bit */
    DMA_Handle.Init.MemDataAlignment    = DMA_MDATAALIGN_WORD;      /* 存储器数据传输位宽选择字，即 32bit */
    DMA_Handle.Init.Mode                = DMA_CIRCULAR;             /* 循环模式 */
    DMA_Handle.Init.Priority            = DMA_PRIORITY_LOW;         /* 优先级低 */
    DMA_Handle.Init.FIFOMode            = DMA_FIFOMODE_DISABLE;     /* 禁止 FIFO*/
    DMA_Handle.Init.FIFOThreshold       = DMA_FIFO_THRESHOLD_FULL; /* 禁止 FIFO 此位不起作用，用于设置阀值 */
    DMA_Handle.Init.MemBurst        = DMA_MBURST_SINGLE;        /* 禁止 FIFO 此位不起作用，用于存储器突发 */
    DMA_Handle.Init.PeriphBurst     = DMA_PBURST_SINGLE;        /* 禁止 FIFO 此位不起作用，用于外设突发 */

    /* 初始化 DMA */
    if(HAL_DMA_Init(&DMA_Handle) != HAL_OK)
    {
         Error_Handler(__FILE__, __LINE__);
    }

    /* 开启 DMA1 Stream1 的中断 */
    HAL_NVIC_SetPriority(DMA1_Stream1_IRQn, 2, 0);
    HAL_NVIC_EnableIRQ(DMA1_Stream1_IRQn);


    /*##-4- 配置 DMAMUX ####################################################################*/
    dmamux_ReqGenParams.SignalID  = HAL_DMAMUX1_REQ_GEN_TIM12_TRGO;         /* 请求触发器选择 LPTIM2_OUT */
    dmamux_ReqGenParams.Polarity  = HAL_DMAMUX_REQ_GEN_RISING;              /* 上升沿触发  */
    dmamux_ReqGenParams.RequestNumber = 1;                                  /* 触发后,传输进行 1 次 DMA 传输 */

    HAL_DMAEx_ConfigMuxRequestGenerator(&DMA_Handle, &dmamux_ReqGenParams); /* 配置 DMAMUX */
    HAL_DMAEx_EnableMuxRequestGenerator (&DMA_Handle);                       /* 使能 DMAMUX 请求发生器 */


    /*##-4- 启动 DMA 双缓冲传输 ############################################################*/
    /*
        1、此函数会开启 DMA 的 TC，TE 和 DME 中断
        2、如果用户配置了回调函数 DMA_Handle.XferHalfCpltCallback,那么函数 HAL_DMA_Init 会开启半传输完成中断。
        3、如果用户使用了 DMAMUX 的同步模式，此函数会开启同步溢出中断。
        4、如果用户使用了 DMAMUX 的请求发生器，此函数会开始请求发生器溢出中断。
    */
    HAL_DMAEx_MultiBufferStart_IT(&DMA_Handle, (uint32_t)IO_Toggle,
                                  (uint32_t)&GPIOB->BSRRL,(uint32_t)IO_Toggle1, 8);
}
```

42.5 总结

本章节就为大家讲解这么多，BDMA 作为一个重要的外设，务必要熟练掌握。另外注意 DMA1，DMA2 和 BDMA 的区别。

第43章 STM32H7 的 DMA 应用之双缓冲控制任意 IO 和脉冲数控制

本章教程为大家讲解定时器触发 DMAMUX，控制 DMA 让 GPIO 输出 PWM 以及脉冲数的控制，实际项目中有一定的使用价值。

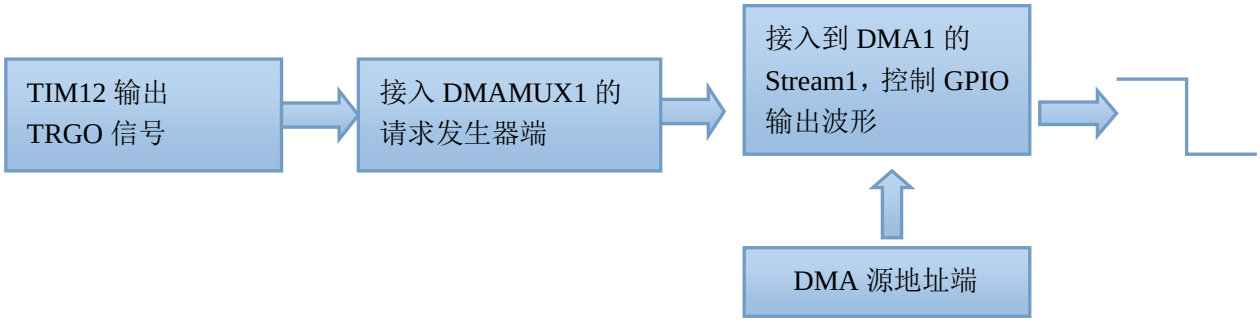
43.1 初学者重要提示
43.2 定时器触发 DMA 驱动设计
43.3 DMA 板级支持包（bsp_tim_dma.c）
43.4 DMA 驱动移植和使用
43.5 实验例程设计框架
43.6 实验例程说明（MDK）
43.7 实验例程说明（IAR）
43.8 总结

43.1 初学者重要提示

- 学习本章节前，务必优先学习第 39 章和 42 章，需要对 DMAMUX，DMA 的基础知识和 HAL 库的几个常用 API 有个认识。
- 相比定时器本身支持的 PWM，这种方式更加灵活，可以让任意 IO 都可以输出 PWM，而且方便运行中动态修改输出状态。

43.2 定时器触发 DMA 驱动设计

定时器触发 DMAMUX，控制 DMA 让 GPIO 输出 PWM 的实现思路框图如下：



下面将程序设计中的相关问题逐一为大家做个说明。

43.2.1 定时器选择

使用 DMA 的话，请求信号都是来自 DMAMUX2，而控制 DMA 做周期性传输的话，可以使用定时器触发，这样的话就可以使用 DMAMUX 的请求发生器功能，支持如下几种触发：

```
#define HAL_DMAMUX1_REQ_GEN_DMAMUX1_CH0_EVT   0U
#define HAL_DMAMUX1_REQ_GEN_DMAMUX1_CH1_EVT   1U
#define HAL_DMAMUX1_REQ_GEN_DMAMUX1_CH2_EVT   2U
#define HAL_DMAMUX1_REQ_GEN_LPTIM1_OUT        3U
#define HAL_DMAMUX1_REQ_GEN_LPTIM2_OUT        4U
#define HAL_DMAMUX1_REQ_GEN_LPTIM3_OUT        5U
#define HAL_DMAMUX1_REQ_GEN_EXTI0             6U
#define HAL_DMAMUX1_REQ_GEN_TIM12_TRGO        7U
```

我们这里使用的是 TIM12_TRGO。

接下来就是 TIM12 的时钟配置问题，代码如下：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: TIM12_Config
4.  *    功能说明: 配置 TIM12，用于触发 DMAMUX 的请求发生器
5.  *    形    参: _Mode
6.  *              0 配置为 100KHz 触发频率，如果 DMAMUX 配置为单边沿触发，那么输出 PWM 频率是 50KHz，双边沿是
7.                  100KHz。
8.  *              1 配置为 10KHz 触发频率,如果 DMAMUX 配置为单边沿触发,那么输出 PWM 频率是 5KHz,双边沿是 10KHz。
9.  *    返 回 值: 无
10. *********************************************************************************************************
11. */
12. void TIM12_Config(uint8_t _Mode)
13. {
14.     TIM_HandleTypeDef  htim ={0};
15.     TIM_MasterConfigTypeDef sMasterConfig = {0};
16.     TIM_OC_InitTypeDef sConfig = {0};
17.     uint32_t Period[2] = {1999, 19999};
18.     uint32_t Pulse[2]  = {1000, 10000};
19.
20.      /* 使能时钟 */
21.      __HAL_RCC_TIM12_CLK_ENABLE();
22.
23.     /*----------------------------------------------------------------------
24.          bsp.c 文件中 void SystemClock_Config(void) 函数对时钟的配置如下:
25.
```

```
26.          System Clock source       = PLL (HSE)
27.          SYSCLK(Hz)                = 400000000 (CPU Clock)
28.          HCLK(Hz)                  = 200000000 (AXI and AHBs Clock)
29.          AHB Prescaler             = 2
30.          D1 APB3 Prescaler         = 2 (APB3 Clock  100MHz)
31.          D2 APB1 Prescaler         = 2 (APB1 Clock  100MHz)
32.          D2 APB2 Prescaler         = 2 (APB2 Clock  100MHz)
33.          D3 APB4 Prescaler         = 2 (APB4 Clock  100MHz)
34.
35.          因为 APB1 prescaler != 1, 所以 APB1 上的 TIMxCLK = APB1 x 2 = 200MHz;
36.          因为 APB2 prescaler != 1, 所以 APB2 上的 TIMxCLK = APB2 x 2 = 200MHz;
37.          APB4 上面的 TIMxCLK 没有分频，所以就是 100MHz;
38.
39.          APB1 定时器有 TIM2, TIM3 ,TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14，LPTIM1
40.          APB2 定时器有 TIM1, TIM8 , TIM15, TIM16，TIM17
41.
42.          APB4 定时器有 LPTIM2，LPTIM3，LPTIM4，LPTIM5
43.
44.      TIM12CLK = 200MHz/(Period + 1) / (Prescaler + 1)
45.      函数 bsp_InitTimDMA1 中 DMAMUX1 选择的是单边沿触发，每个时钟可以触发两次。
46.      ---------------------------------------------------------------------- */
47.      HAL_TIM_Base_DeInit(&htim);
48.
49.      htim.Instance = TIM12;
50.       htim.Init.Period            = Period[_Mode];
51.       htim.Init.Prescaler         = 0;
52.       htim.Init.ClockDivision     = 0;
53.       htim.Init.CounterMode       = TIM_COUNTERMODE_UP;
54.       htim.Init.RepetitionCounter = 0;
55.       HAL_TIM_Base_Init(&htim);
56.
57.      sConfig.OCMode     = TIM_OCMODE_PWM1;
58.      sConfig.OCPolarity = TIM_OCPOLARITY_LOW;
59.
60.      /* 占空比 50% */
61.      sConfig.Pulse = Pulse[_Mode];
62.      if(HAL_TIM_OC_ConfigChannel(&htim, &sConfig, TIM_CHANNEL_1) != HAL_OK)
63.      {
64.           Error_Handler(__FILE__, __LINE__);
65.      }
66.
67.      /* 启动 OC1 */
68.      if(HAL_TIM_OC_Start(&htim, TIM_CHANNEL_1) != HAL_OK)
69.      {
70.           Error_Handler(__FILE__, __LINE__);
71.      }
72.
73.      /* TIM12 的 TRGO 用于触发 DMAMUX 的请求发生器 */
74.      sMasterConfig.MasterOutputTrigger = TIM_TRGO_OC1REF;
75.      sMasterConfig.MasterOutputTrigger2 = TIM_TRGO2_RESET;
76.      sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
77.
78.       HAL_TIMEx_MasterConfigSynchronization(&htim, &sMasterConfig);
79.  }
```

这里把几个关键的地方阐释下：

- 第 14 – 16 行，对作为局部变量的 HAL 库结构体做初始化，防止不确定值配置时出问题。

- 第 17 – 18 行，定义了两组周期变量和占空比变量，用来设置 TIM12。
- 第 20 – 71 行，注释已经比较详细。

  当选择第 1 组配置时，

  TIM12CLK = 200MHz / (Period + 1) / (Prescaler + 1) = 200MHz/（1999+1） = 100KHz

  占空比 = Pulse / (Period + 1) = 1000 / （1999+1） = 50%

  当选择第 2 组配置时，

  TIM12CLK = 200MHz / (Period + 1) / (Prescaler + 1) = 200MHz/（19999+1） = 10KHz

  占空比 = Pulse / (Period + 1) = 10000 / （19999+1） = 50%
- 第 22 – 40 行， TIM12 的 TRGO 用于触发 DMAMUX 的请求发生器。

这些知识点在前面的定时器章节有更详细的说明。

43.2.2 DMAMUX 和 DMA 配置

完整配置如下：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: bsp_InitTimDMA
4.  *    功能说明: 配置 DMAMUX 的定时器触+DMA 双缓冲控制任意 IO 做 PWM 和脉冲数控制
5.  *    形    参: 无
6.  *    返 回 值: 无
7.  *********************************************************************************************************
8.  */
9.  void bsp_InitTimDMA(void)
10. {
11.     GPIO_InitTypeDef  GPIO_InitStruct;
12.     DMA_HandleTypeDef DMA_Handle = {0};
13.     HAL_DMA_MuxRequestGeneratorConfigTypeDef dmamux_ReqGenParams = {0};
14.
15.     /*##-1- 配置 PB1 用于 PWM 输出 ###########################################################*/
16.     __HAL_RCC_GPIOB_CLK_ENABLE();
17.     GPIO_InitStruct.Pin = GPIO_PIN_1;
18.     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
19.     GPIO_InitStruct.Pull = GPIO_NOPULL;
20.     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
21.     HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
22.
23.     /*##-2- 使能 DMA1 时钟并配置 ##############################################################*/
24.     __HAL_RCC_DMA1_CLK_ENABLE();
25.     DMA_Handle.Instance          = DMA1_Stream1;            /* 使用的 DMA1 Stream1 */
26.     DMA_Handle.Init.Request      = DMA_REQUEST_GENERATOR0;  /* 请求类型采用的 DMAMUX 请求发生器通道 0 */
27.     DMA_Handle.Init.Direction           = DMA_MEMORY_TO_PERIPH;/* 传输方向是从存储器到外设 */
28.     DMA_Handle.Init.PeriphInc           = DMA_PINC_DISABLE;    /* 外设地址自增禁止 */
29.     DMA_Handle.Init.MemInc              = DMA_MINC_ENABLE;     /* 存储器地址自增使能 */
30.     DMA_Handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD; /* 外设数据传输位宽选择字，即 32bit */
31.     DMA_Handle.Init.MemDataAlignment    =DMA_MDATAALIGN_WORD;  /* 存储器数据传输位宽选择字，即 32bit */
32.     DMA_Handle.Init.Mode                = DMA_CIRCULAR;        /* 循环模式 */
33.     DMA_Handle.Init.Priority            = DMA_PRIORITY_LOW;    /* 优先级低 */
```

```
34.     DMA_Handle.Init.FIFOMode      = DMA_FIFOMODE_DISABLE;      /* 禁止 FIFO*/
35.     DMA_Handle.Init.FIFOThreshold = DMA_FIFO_THRESHOLD_FULL; /* 禁止 FIFO 此位不起作用，用于设置阀值 */
36.     DMA_Handle.Init.MemBurst      = DMA_MBURST_SINGLE; /* 禁止 FIFO 此位不起作用，用于存储器突发 */
37.     DMA_Handle.Init.PeriphBurst   = DMA_PBURST_SINGLE; /* 禁止 FIFO 此位不起作用，用于外设突发 */
38.
39.     /* 初始化 DMA */
40.     if(HAL_DMA_Init(&DMA_Handle) != HAL_OK)
41.     {
42.         Error_Handler(__FILE__, __LINE__);
43.     }
44.
45.     /* 开启 DMA1 Stream1 的中断 */
46.     HAL_NVIC_SetPriority(DMA1_Stream1_IRQn, 2, 0);
47.     HAL_NVIC_EnableIRQ(DMA1_Stream1_IRQn);
48.
49.
50.     /*##-4- 配置 DMAMUX ######################################################*/
51.     dmamux_ReqGenParams.SignalID  = HAL_DMAMUX1_REQ_GEN_TIM12_TRGO;  /* 请求触发器选择 LPTIM2_OUT */
52.     dmamux_ReqGenParams.Polarity  = HAL_DMAMUX_REQ_GEN_RISING;       /* 上升沿触发  */
53.     dmamux_ReqGenParams.RequestNumber = 1;                          /* 触发后，传输进行 1 次 DMA 传输 */
54.
55.     HAL_DMAEx_ConfigMuxRequestGenerator(&DMA_Handle, &dmamux_ReqGenParams);/* 配置 DMAMUX */
56.     HAL_DMAEx_EnableMuxRequestGenerator (&DMA_Handle);                    /* 使能 DMAMUX 请求发生器 */
57.
58.
59.     /*##-4- 启动 DMA 双缓冲传输 ##############################################*/
60.     /*
61.         1、此函数会开启 DMA 的 TC，TE 和 DME 中断
62.         2、如果用户配置了回调函数 DMA_Handle.XferHalfCpltCallback，那么函数 HAL_DMA_Init 会开启半传输完
63.            成中断。
64.         3、如果用户使用了 DMAMUX 的同步模式，此函数会开启同步溢出中断。
65.         4、如果用户使用了 DMAMUX 的请求发生器，此函数会开始请求发生器溢出中断。
66.     */
67.     HAL_DMAEx_MultiBufferStart_IT(&DMA_Handle, (uint32_t)IO_Toggle,
68.                                   (uint32_t)&GPIOB->BSRRL,(uint32_t)IO_Toggle1, 8);
69.
70.     /* 用不到的中断可以直接关闭 */
71.     //DMA1_Stream1->CR &= ~DMA_IT_DME;
72.     //DMA1_Stream1->CR &= ~DMA_IT_TE;
73.     //DMAMUX1_RequestGenerator0->RGCR &= ~DMAMUX_RGxCR_OIE;
74.
75.     TIM12_Config(0);
76. }
```

这里把几个关键的地方阐释下：

- 第 11 - 13 行，对作为局部变量的 HAL 库结构体做初始化，防止不确定值配置时出问题。
- 第 16 - 21 行，配置 PB1 推挽输出。
- 第 24 – 43 行，配置 DMA 的基本参数，注释较详细。
- 第 46 – 47 行，配置 DMA 的中断优先级，并使能。
- 第 50 – 56 行，配置 DMAMUX 的请求发生器触发源选择的 TIM12 TRGO，上升沿触发 DMA 传输。
- 第 59 行，采用双缓冲的中断方式启动 DMA 传输，这里中断注意第 2 个参数，第 3 个参数和第 4 个参数。第 2 个原地址，即缓冲 0 地址和第 4 个缓冲 1 的地址定义如下：

  uint32_t IO_Toggle[8] ={

```
                0x00000002U,
                0x00020000U,
                0x00000002U,
                0x00020000U,
                0x00000002U,
                0x00020000U,
                0x00000002U,
                0x00020000U,
            };
uint32_t IO_Toggle1[8]   =
            {
                0x00000002U,
                0x00020000U,
                0x00000002U,
                0x00020000U,
                0x00000002U,
                0x00020000U,
                0x00000002U,
                0x00020000U,
            };
```

都是定义了 8 个 uint32_t 类型的变量。第 3 个参数非常考究，这里使用的 GPIO 的 BSRR 寄存器，这个寄存器的特点就是置 1 有效，而清零操作对其无效。

10.4.7 GPIO port bit set/reset register (GPIOx_BSRR) (x = A to K)

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

高 16 位用于控制 GPIO 的输出低电平，而低 16 位用于输出高电平工作，所以我们这里设置

GPIOB_BSRR = 0x00000002 时，表示 PB1 输出高电平。

GPIOB_BSRR = 0x00020000 时，表示 PB1 输出低电平。

通过这种方式就实现了 PB1 引脚的高低电平控制。

◆ 第 71 – 73 行，由于函数 HAL_DMAEx_MultiBufferStart_IT 会开启好几个中断（函数前面的注释比

较详细），大家可以在这里关闭不需要开启的中断。

- 第 75 行，调用 TIM12 的初始化配置，默认 TIM12 配置的触发频率是 100KHz，实现 50KHz 的方波输出（两次触发是一个周期）。

43.2.3 DMA 存储器选择注意事项

由于 STM32H7 Cache 的存在，凡是 CPU 和 DMA 都会操作到的存储器，我们都要注意数据一致性问题。对于本章节要实现的功能，如果不需要运行中动态修改 DMA 源地址中的数据，可以不用管这个问题，如果要动态修改就得注意 Cache 所带来的的数据一致性问题，这里提供两种解决办法：

- 方法一：

  设置 DMA 所使用 SRAM3 存储区的 Cache 属性为 Write through, read allocate, no write allocate。保证写入的数据会立即更新到 SRAM3 里面。

```
/* 配置 SRAM3 的属性为 Write through, read allocate, no write allocate */
MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress      = 0x38000000;
MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

- 方法二：

  设置 SRAM3 的缓冲区做 32 字节对齐，大小最好也是 32 字节整数倍，然后调用函数 SCB_CleanDCache_by_Addr 做 Clean 操作即可，保证 BDMA 读取到的数据是刚更新好的。

本章节配套例子是直接使用的方法一。例子中变量的定义方式如下：

```
/* 方便 Cache 类的 API 操作，做 32 字节对齐 */
#if defined ( __ICCARM__ )
#pragma location = 0x38000000
uint32_t IO_Toggle[8]  =
                      {
                          0x00000002U,
                          0x00020000U,
                          0x00000002U,
                          0x00020000U,
                          0x00000002U,
                          0x00020000U,
                          0x00000002U,
                          0x00020000U,
                      };
```

```
#pragma location = 0x38000020
uint32_t IO_Toggle1[8]  =
                      {
                          0x00000002U,
                          0x00020000U,
                          0x00000002U,
                          0x00020000U,
                          0x00000002U,
                          0x00020000U,
                          0x00000002U,
                          0x00020000U,
                      };

#elif defined ( __CC_ARM )
ALIGN_32BYTES(__attribute__((section (".RAM_D3"))) uint32_t IO_Toggle[8]) =
                                                    {
                                                        0x00000002U,
                                                        0x00020000U,
                                                        0x00000002U,
                                                        0x00020000U,
                                                        0x00000002U,
                                                        0x00020000U,
                                                        0x00000002U,
                                                        0x00020000U,
                                                    };

ALIGN_32BYTES(__attribute__((section (".RAM_D3"))) uint32_t IO_Toggle1[8]) =
                                                    {
                                                        0x00000002U,
                                                        0x00020000U,
                                                        0x00000002U,
                                                        0x00020000U,
                                                        0x00000002U,
                                                        0x00020000U,
                                                        0x00000002U,
                                                        0x00020000U,
                                                    };
#endif
```

对于 IAR 需要#pragma location 指定位置，而 MDK 通过分散加载即可实现，详情看前面第 26 章，有详细讲解。

43.2.4DMA 中断处理

前面的配置中开启了 DMA 的传输完成中断、传输错误中断和直接模式错误中断（半传输完成中断未做开启），其中传输完成中断里面可以实现双缓冲的处理：

```
/*
*********************************************************************************************************
*    函 数 名: DMA1_Stream1_IRQHandler
*    功能说明: DMA1 Stream1 中断服务程序
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
```

```
void DMA1_Stream1_IRQHandler(void)
{
    /* 传输完成中断 */
    if((DMA1->LISR & DMA_FLAG_TCIF1_5) != RESET)
    {
        /* 清除标志 */
        DMA1->LIFCR = DMA_FLAG_TCIF1_5;

        /* 当前使用的缓冲 0 */
        if((DMA1_Stream1->CR & DMA_SxCR_CT) == RESET)
        {
            /*
                1、当前正在使用缓冲 0，此时可以动态修改缓冲 1 的数据。
                   比如缓冲区 0 是 IO_Toggle，缓冲区 1 是 IO_Toggle1，那么此时就可以修改 IO_Toggle1。
                2、变量所在的 SRAM 区已经通过 MPU 配置为 WT 模式，更新变量 IO_Toggle 会立即写入。
                3、不配置 MPU 的话，也可以通过 Cahce 的函数 SCB_CleanDCache_by_Addr 做 Clean 操作。
            */

        }
        /* 当前使用的缓冲 1 */
        else
        {
             /*
               1、当前正在使用缓冲 1，此时可以动态修改缓冲 0 的数据。
                  比如缓冲区 0 是 IO_Toggle，缓冲区 1 是 IO_Toggle1，那么此时就可以修改 IO_Toggle。
               2、变量所在的 SRAM 区已经通过 MPU 配置为 WT 模式，更新变量 IO_Toggle 会立即写入。
               3、不配置 MPU 的话，也可以通过 Cahce 的函数 SCB_CleanDCache_by_Addr 做 Clean 操作。
            */

        }
    }

    /* 半传输完成中断 */
    if((DMA1->LISR & DMA_FLAG_HTIF1_5) != RESET)
    {
        /* 清除标志 */
        DMA1->LISR = DMA_FLAG_HTIF1_5;
    }

    /* 传输错误中断 */
    if((DMA1->LISR & DMA_FLAG_TEIF1_5) != RESET)
    {
        /* 清除标志 */
        DMA1->LISR = DMA_FLAG_TEIF1_5;
    }

    /* 直接模式错误中断 */
    if((DMA1->LISR & DMA_FLAG_DMEIF1_5) != RESET)
    {
        /* 清除标志 */
        DMA1->LISR = DMA_FLAG_DMEIF1_5;
    }
}
```

注释的比较清楚。如果输出的 PWM 频率较高，建议将 DMA 的缓冲区设置的大些，防止 DMA 中断的执行频率较高。

由于使用了 DMAMUX 的请求发生器功能，函数 HAL_DMAEx_MultiBufferStart_IT 还会开启请求发

生器溢出中断，对应的中断服务程序处理如下：

```
/*
*********************************************************************************************************
*    函 数 名: DMAMUX1_OVR_IRQHandler
*    功能说明: DMAMUX 的中断服务程序，这里用于处理请求发生器的溢出。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DMAMUX1_OVR_IRQHandler(void)
{
    if(DMAMUX1_RequestGenStatus->RGSR & DMAMUX_RGSR_OF0 != RESET)
    {
       /* 关闭溢出中断 */
       DMAMUX1_RequestGenerator0->RGCR &= ~DMAMUX_RGxCR_OIE;

       /* 清除标志 */
       DMAMUX1_RequestGenStatus->RGCFR = DMAMUX_RGSR_OF0;
    }
}
```

处理比较简单，检测到溢出中断后关闭溢出中断，同时清除标志。这个中断主要用于检测。另外，如果大家用到 DMAMUX 的同步模式，同步溢出中断也是在 DMAMUX1_OVR_IRQHandler 里面处理。

43.2.5 DMA 脉冲个数控制

借助本章 2.4 小节的知识点，如果要实现脉冲个数的控制，在 DMA 中断服务程序里面动态修改缓冲区即可。比如我们配置：

- DMA 开启双缓冲模式。
- DMA 传输 16 次为一轮，每两次传输算一个周期的脉冲。

  如果要实现 100 个脉冲，我们就可以在 12 轮，即 12*8=96 个脉冲的传输完成中断里面修改缓冲区 1 出低电平即可，再次进入传输完成中断后再缓冲区 0 输出低电平。

43.3 DMA 板级支持包（bsp_tim_dma.c）

DMA 驱动文件 bsp_pwm_dma.c 提供了如下两个函数：

- TIM12_Config
- bsp_InitTimDMA

43.3.1 函数 TIM12_Config

函数原型：

```
void TIM12_Config(uint8_t _Mode)
```

函数描述：

此函数用于配置 TIM12 工作在 OC 输出比较模式，使用 TIM12 的 TRGO 作为 BDMA 请求发生器的触发源。

函数参数：

- 第 1 个参数用于配置 TIM12 的输出频率。

  配置为数值 0：表示配置为 100KHz 触发频率,配置为 100KHz 触发频率，如果 DMAMUX 配置为单边沿触发，那么输出 PWM 频。

  配置为数值 1：表示配置为 10KHz 触发频率，如果 DMAMUX 配置为单边沿触发，那么输出 PWM 频率是 5KHz，双边沿是 10KHz。

注意事项：

1. 关于此函数的讲解在本章的 2.1 小节。

43.3.2 函数 bsp_InitTimDMA

函数原型：

```
void bsp_InitTimBDMA(void)
```

函数描述：

此函数用于配置定时器触发 DMA，可以实现任意 IO 做 PWM 输出。

注意事项：

1. 关于此函数的讲解在本章 2.2 小节。

使用举例：

作为初始化函数，直接在 bsp.c 文件的 bsp_Init 函数里面调用即可。

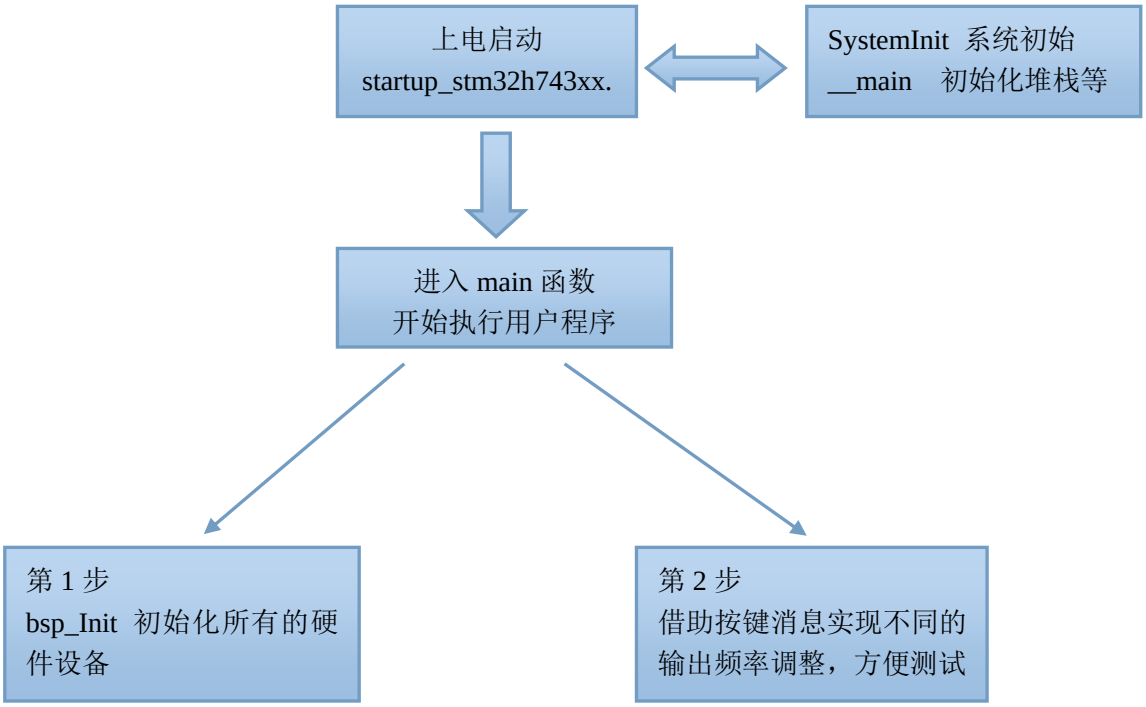
43.4 DMA 驱动移植和使用

低功耗定时器的移植比较简单：

- 第 1 步：复制 bsp_tim_dma.c 和 bsp_tim_dma.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：这几个驱动文件主要用到 HAL 库的 GPIO、TIM 和 DMA 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 3 步，应用方法看本章节配套例子即可。

43.5 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：

  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED 和串口。第 2 步，借助按键消息实现不同的输出频率调整，方便测试。

43.6 实验例程说明（MDK）

配套例子：

V7-011_DMAMUX 的定时器触+DMA 双缓冲控制任意 IO 做 PWM 和脉冲数控制

实验目的：

1. 学习 DMAMUX 的定时器触+DMA 双缓冲控制任意 IO 做 PWM 和脉冲数控制。

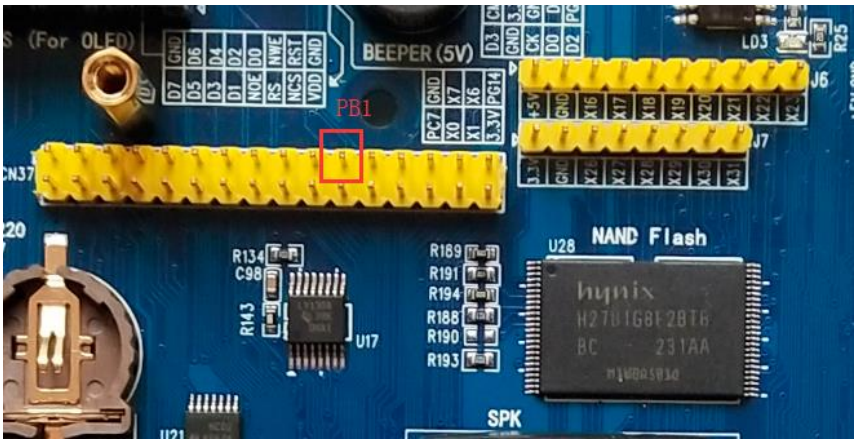
实验内容：

1. 通过 TIM12 触发 DMAMUX 的请求发生器，控制 DMA 给任意 IO 做 PWM 输出。

实验操作：

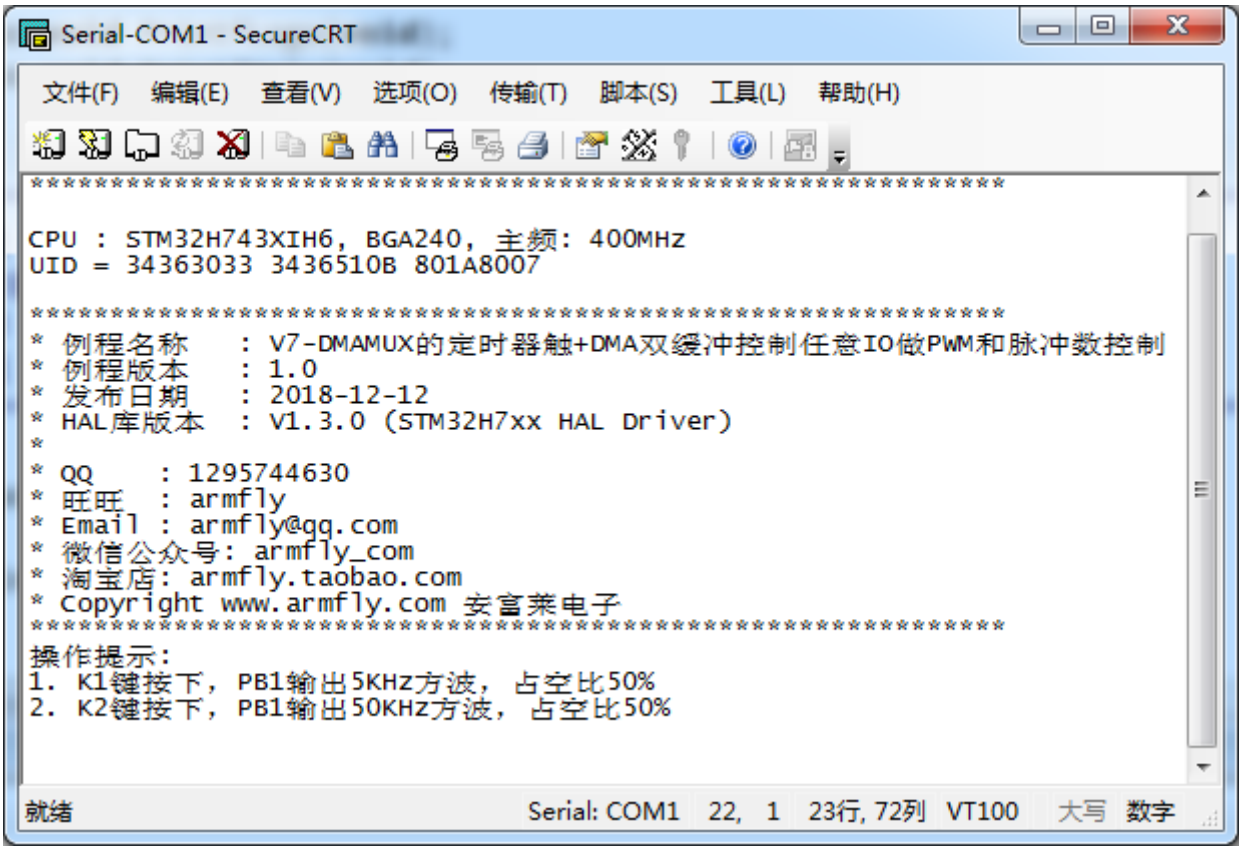
1. K2 键按下，PB1 输出 5KHz 方波，占空比 50%
2. K2 键按下，PB1 输出 50KHz 方波，占空比 50%

PB1 的位置：



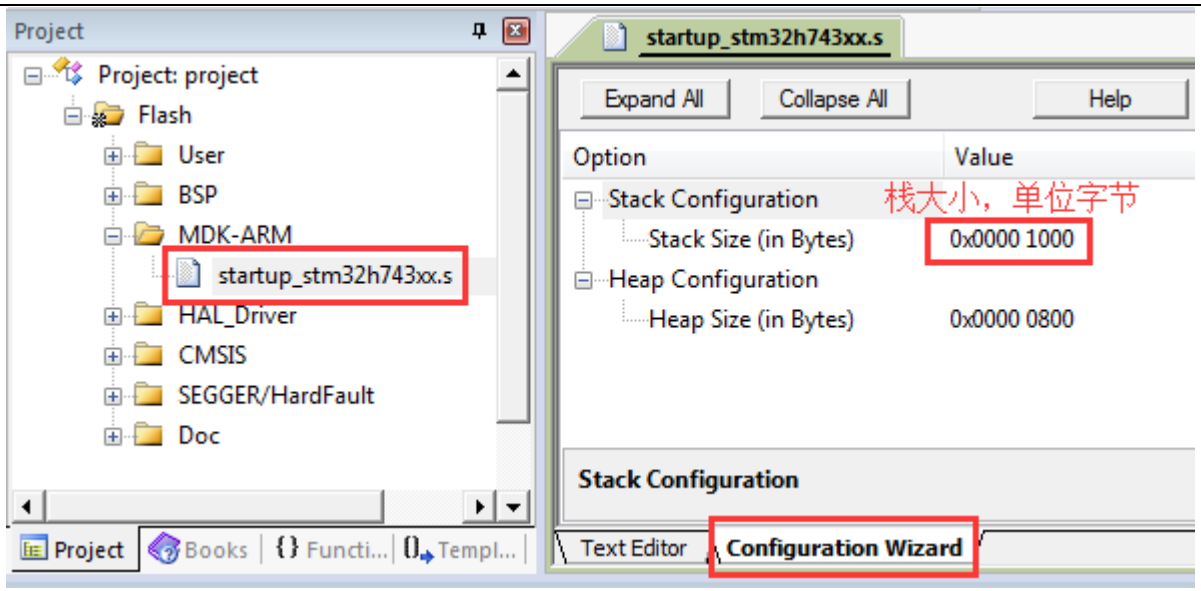
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

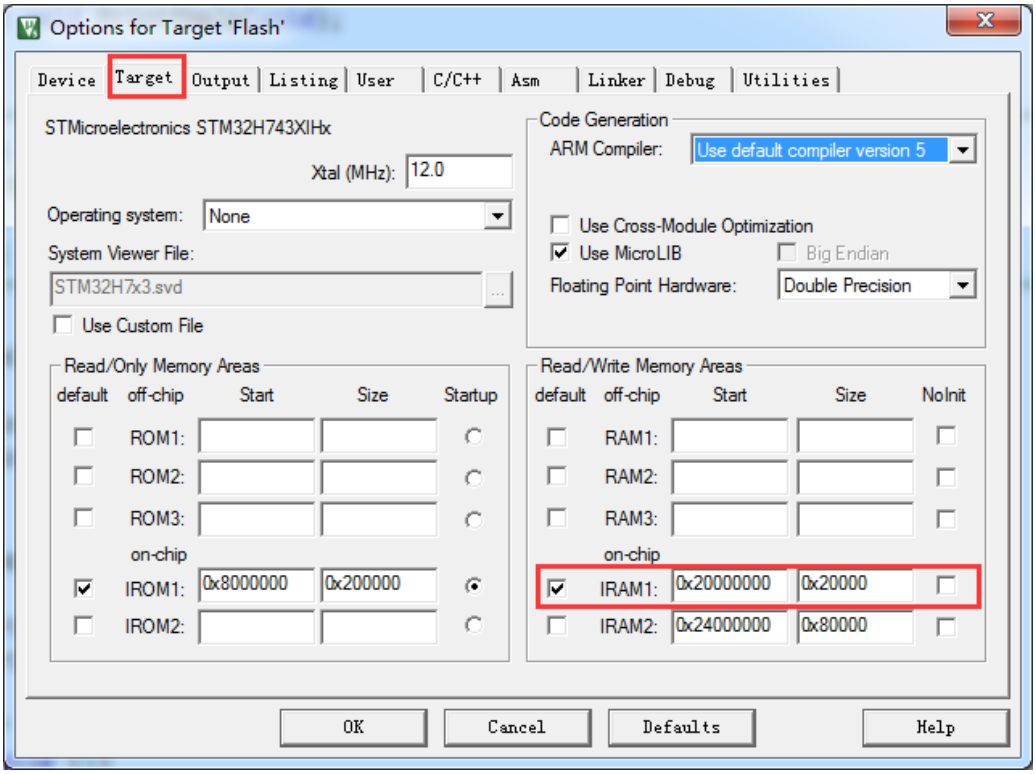


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
    bsp_InitTimDMA(); /* 初始化定时器触发 DMA */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SRAM4。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();
```

```
    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SRAM4 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- K1 键按下，PB1 输出 5KHz 方波，占空比 50%。
- K2 键按下，PB1 输出 50KHz 方波，占空比 50%。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */


    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 50ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，PB1 输出 5KHz 方波，占空比 50% */
                    TIM12_Config(1);
```

```
                    break;

                case KEY_DOWN_K2:           /* K2 键按下，PB1 输出 50KHz 方波，占空比 50% */
                    TIM12_Config(0);
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

43.7 实验例程说明（IAR）

配套例子：

V7-011_DMAMUX 的定时器触+DMA 双缓冲控制任意 IO 做 PWM 和脉冲数控制

实验目的：

1. 学习 DMAMUX 的定时器触+DMA 双缓冲控制任意 IO 做 PWM 和脉冲数控制。

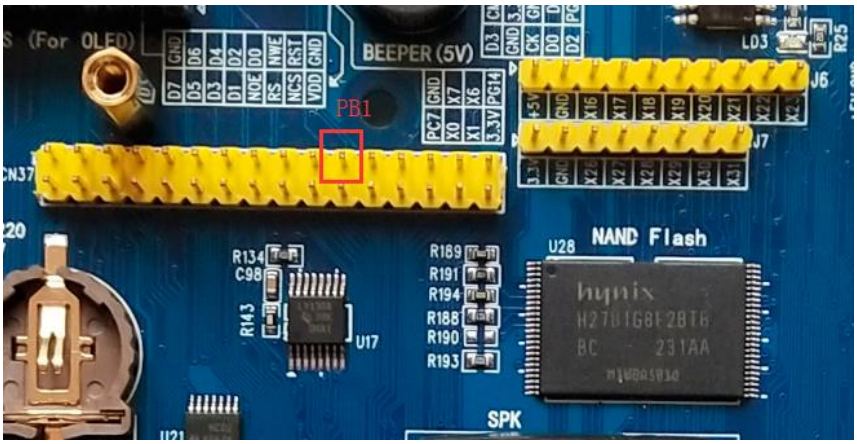
实验内容：

1. 通过 TIM12 触发 DMAMUX 的请求发生器，控制 DMA 给任意 IO 做 PWM 输出。

实验操作：

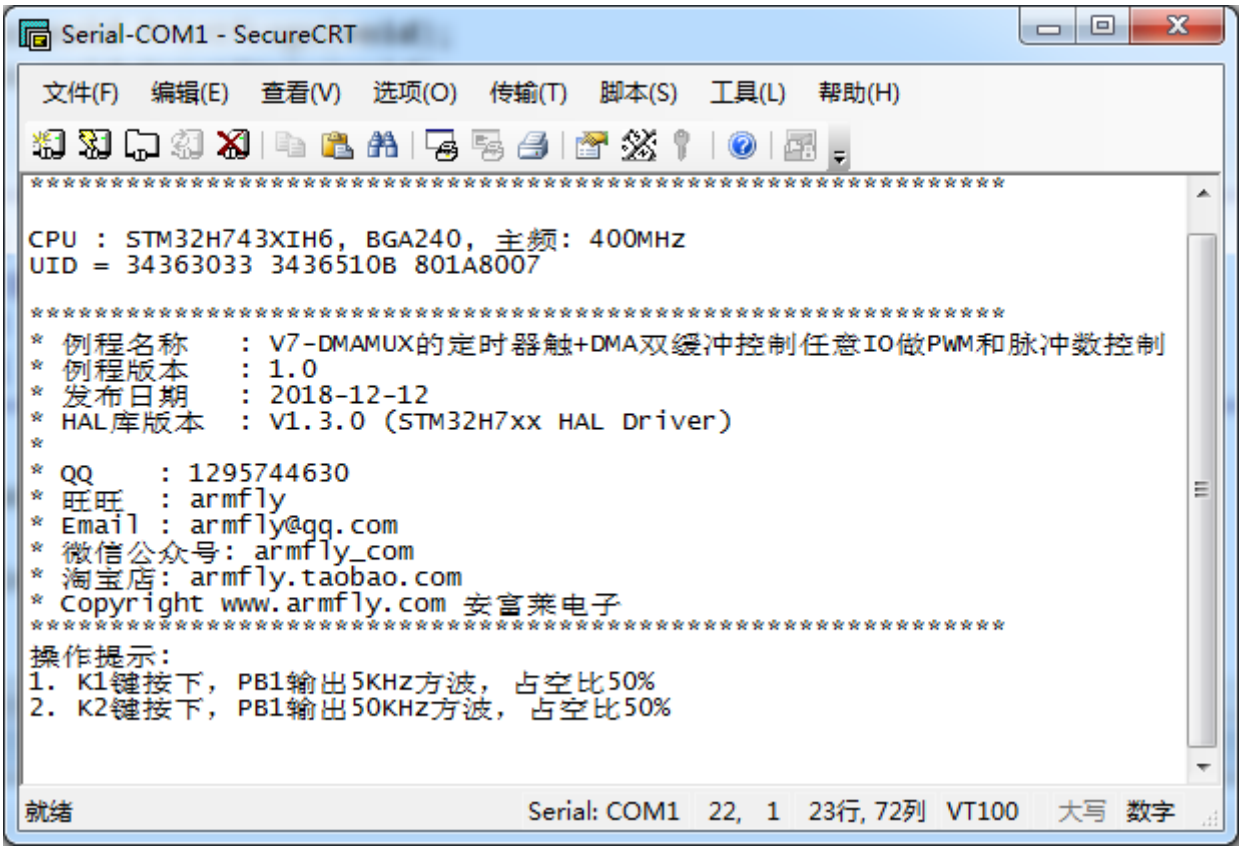
1. K2 键按下，PB1 输出 5KHz 方波，占空比 50%
2. K2 键按下，PB1 输出 50KHz 方波，占空比 50%

PB1 的位置：



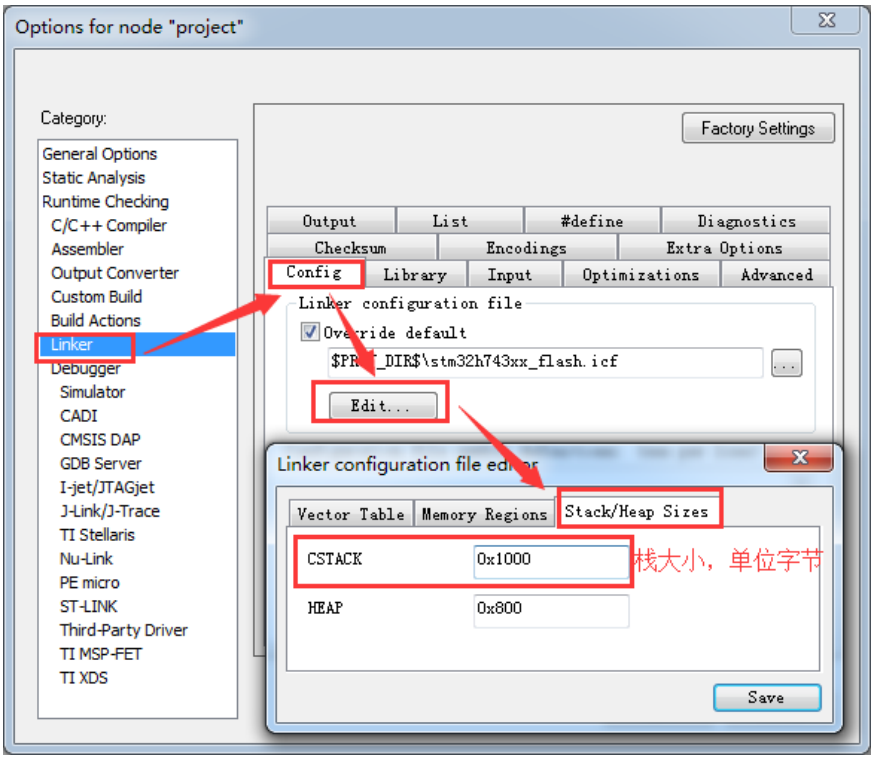
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

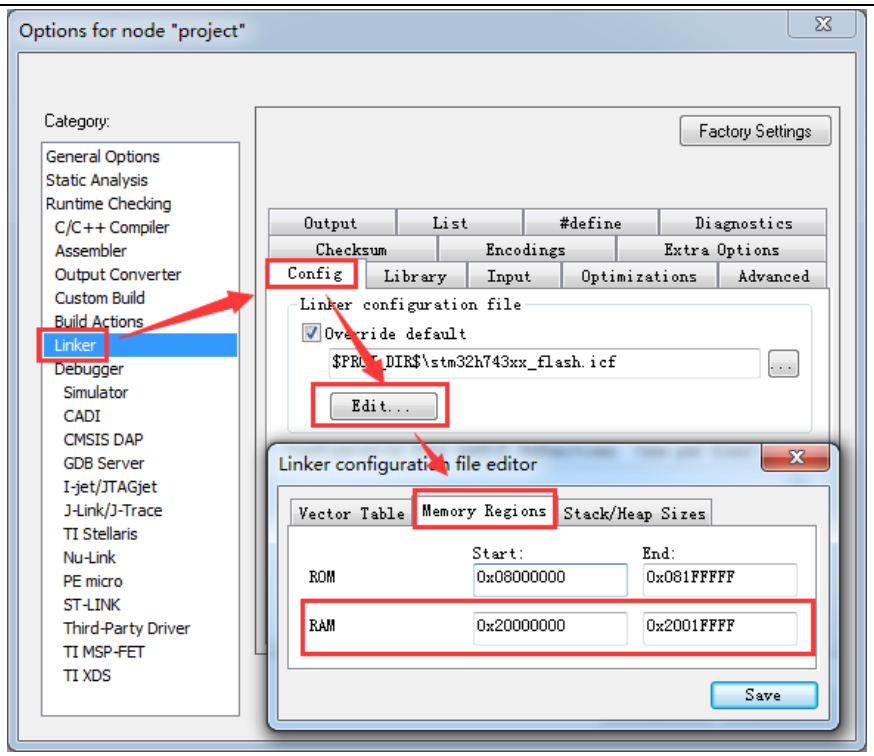


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
    bsp_InitTimDMA(); /* 初始化定时器触发 DMA */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SRAM4。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
```

```
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

  /* 配置 SRAM4 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **主功能：**

  主程序实现如下操作：

  - K1 键按下，PB1 输出 5KHz 方波，占空比 50%。
  - K2 键按下，PB1 输出 50KHz 方波，占空比 50%。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
```

```
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */


    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 50ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，PB1 输出 5KHz 方波，占空比 50% */
                    TIM12_Config(1);
                    break;

                case KEY_DOWN_K2:           /* K2 键按下，PB1 输出 50KHz 方波，占空比 50% */
                    TIM12_Config(0);
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

43.8 总结

本章节就为大家讲解这么多，控制 DMA 让 GPIO 输出 PWM 以及脉冲数的控制，实际项目中有一定的实用价值，望初学者熟练掌握。

第44章 STM32H7 的 ADC 基础知识和 HAL 库 API

本章节为大家讲解 ADC（Analog-to-digital converters，模数转换器），极具项目使用价值，因为 STM32H7 的 ADC 已经高达 16 位分辨率，支持 3.6Msps 采样率。

44.1 初学者重要提示
44.2 ADC 基础知识
44.3 ADC 的 HAL 库用法
44.4 源文件 stm32h7xx_hal_adc.c
44.5 总结

44.1 初学者重要提示

- STM32H7 虽然支持差分，但不支持负压测量。
- STM32H7 的 ADC 采集通道体验快速通道 Fast Channels 和低速通道 Slow Channels 的区别，详情看本章 2.12 小节的电气特性。
- STM32H7 的 ADC 支持过采样，通过过采样技术可以做到 26 位分辨率。
- ADC 的专业术语诠释文档，推荐大家看看：http://www.armbbs.cn/forum.php?mod=viewthread&tid=89414 。

44.2 ADC 基础知识

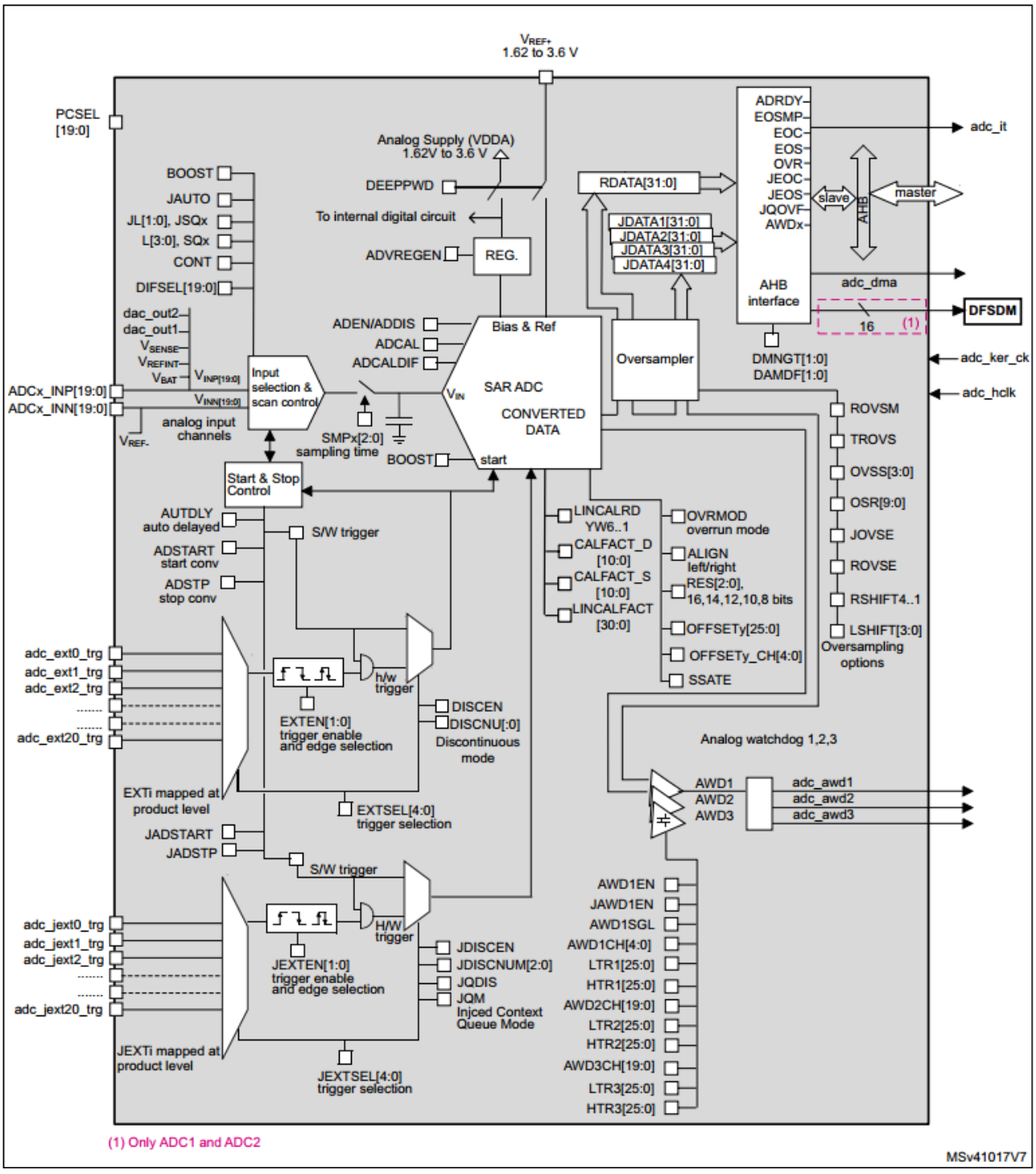
ADC 的几个关键知识点放在开头说：

- STM32H7 支持三路 ADC，分别是 ADC1，ADC2 和 ADC3。其中 ADC1 和 ADC2 可以组成双 ADC 模式，ADC3 是独立的。这个跟 STM32F4 有所不同，F4 的 ADC1，ADC2 和 ADC3 可以组成三 ADC 模式。
- 可以配置为 16bit，14bit，12bit，10bit 或者 8bit 分辨率，分辨率越低可以做到的采样率越高，因为转换时间要短。
- 每个 ADC 都支持 20 路采样通道。其中有 6 路快速通道和 14 路慢速通道，慢速和快速的区别主要是支持的最高采样率不同，慢速通道要比快速通道低。
- 支持单独输入和差分输入，其中差分输入不支持负压测量。

- 支持偏移校准和线性度校准，STM32F1 的时候还带校准功能，到了 STM32F4 取消掉了，H7 又恢复了校准功能。
- 支持规则通道和注入通道两种采样方式。
- 支持低功耗特性，系统在低频工作时保持最佳 ADC 性能（提供自动延迟插入）。
- 具有五条专用的内部通道，内部参考电压 VrefInt，内部温度传感器和 VBAT 监测通道 VBAT/4 都是连接到 ADC3。另外内部 DAC 通道 1 和通道 2，连接到 ADC2。
- 支持过采样，最高可以调整到 26bit 采样率。
- ADC 采样的数据可接入 DFSDM 数字滤波器进行后期处理。
- 每个 ADC 支持三路模拟看门狗。

44.2.1 ADC 硬件框图

认识一个外设，最好的方式就是看他的框图，方便我们快速的了解 ADC 的基本功能，然后再看手册了解细节。框图如下所示（ADC1 和 ADC2）：



相比前面章节讲解的外设，ADC 的框图相对较复杂，因为涉及到控制寄存器较多。通过这个框图，我们可以得到如下信息：

- ADC_INP[0:19]和 ADC_INN[0:19]

  INP 是差分正向输入，INN 是差分反向输入。

  ADC_INP[0:5]和 ADC_INN[0:5]是快速通道。

  ADC_INP[6:19]和 ADC_INN[6:19]是慢速通道。

- adc_ext_trg[20:0]

共有 21 路触发用于规则通道，ADC1 和 ADC2 共用的，而 ADC3 是独立的。

- adc_jext_trg[20:0]

  共有 21 路触发用于注入通道，ADC1 和 ADC2 共用的，而 ADC3 是独立的。

- adc_awd1，adc_awd2 和 adc_awd3

  每个 ADC 都支持三个模拟看门狗。

- adc_it

  ADC 中断。

- adc_hclk

  ADC 的 AHB 时钟。

- adc_ker_ck

  ADC 的内核时钟。

- adc_dma

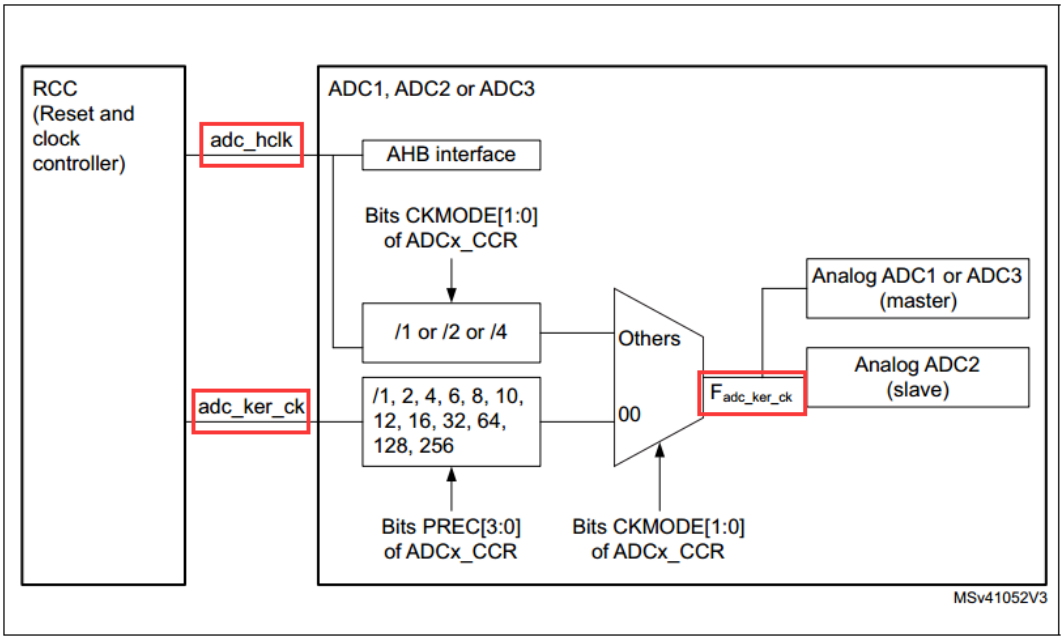
  用于 ADC 的 DMA 请求。

- dac_out1，dac_out2，Vsense，Vrefint 和 Vbat

  五条专用的内部通道，内部参考电压 VrefInt，内部温度传感器和 VBAT 监测通道 VBAT/4 都是连接到 ADC3。另外内部 DAC 通道 1 和通道 2，连接到 ADC2。

44.2.2 ADC 时钟源选择

ADC 有两种时钟源可供选择，可以使用来自 AHB 总线的系统时钟（属于同步时钟，对应下面框图的 adc_hclk），也可以使用 PLL2，PLL3，HSE，HSI 或者 CSI 时钟（属于异步时钟，对应下面框图的 adc_ker_ck）。



结合上面的框图，ADC 的时钟源要注意以下几个问题：

- ADC1，ADC2 和 ADC3 共用选择的时钟。
- ADC 的时钟源使用 AHB 时钟，且使用注入模式，那么在 16bit，14bit，12bit 或者 10bit 分辨率时，ADC 的时钟不能超过 AHB 时钟的四分之一。8bit 模式时，不能超过 AHB 时钟的三分之一。
- 选择 AHB 时钟的话，ADC 的配置中提供了不分频，二分频和四分频。如果选择了不分频，那么配置 AHB 的时钟输出时也不可以设置分频，即 RCC 的 CFGR 寄存器配置不可分频。
- 如果使用 PLL 时钟，运行期间要一直开启，不可关闭。

最后特别注意一点，如果 STM32H7 工作在 400MHz，ADC 使用 AHB 做时钟源，超频是不可避免的。ADC1 和 ADC2 位于 200MHz 的 AHB1 总线时钟，而 ADC3 位于 200MHz 的 AHB4 下。根据上面的框图，ADCx_CCR 寄存器的 CKMODE 最高可以选择 4 分频，那么就是 50MHz，而 ADC 数据手册限制最高是 36MHz，也就是说已经超频了。

使用 AHB 作为时钟源的好处就是定时器等外部触发方式的效果好。

44.2.3 ADC 的采样时间和转换时间

STM32H7 的 ADC 采样速度，即转换时间 = 采样时间 + 逐次逼近时间。

采样时间是可配置的，通过 ADCx_SMPR1 和 ADCx_SMPR2 寄存器中的 SMP[2:0] 位就可以编程所有 ADC 通道，可选采样时间值如下：

- SMP = 000： 1.5 个 ADC 时钟周期
- SMP = 001： 2.5 个 ADC 时钟周期
- SMP = 010： 8.5 个 ADC 时钟周期
- SMP = 011： 16.5 个 ADC 时钟周期
- SMP = 100： 32.5 个 ADC 时钟周期
- SMP = 101： 64.5 个 ADC 时钟周期
- SMP = 110： 387.5 个 ADC 时钟周期
- SMP = 111： 810.5 个 ADC 时钟周期

不同 ADC 分辨率对应的逐次逼近时间不同，具体数值如下：

RES	T_{SAR} ADC 时钟周期	T_{SAR} (ns), F_{ADC}=24 MHz	T_{ADC}（ADC 时钟周期）（采样时间 = 1.5 个 ADC 时钟周期）	T_{ADC} (ns), F_{ADC}=24 MHz
16	8.5 个 ADC 时钟周期	354.2	10 个 ADC 时钟周期	416.7
14	7.5 个 ADC 时钟周期	312.5	9 个 ADC 时钟周期	375
12	6.5 个 ADC 时钟周期	270.8	8 个 ADC 时钟周期	333.3
10	5.5 个 ADC 时钟周期	229.2	7 个 ADC 时钟周期	291.7
8	4.5 个 ADC 时钟周期	187.5	6 个 ADC 时钟周期	250.0

比如配置 SMP = 110，采用 16 位分辨率，那么：

ADC 的转换时间 =采样时间 + 逐次逼近时间

= 387.5 个 ADC 时钟周期 + 8.5 个 ADC 时钟周期

= 396 个 ADC 时钟周期。

44.2.4 ADC 单次转换和连续转换

STM32H7 的 ADC 支持单次转换和连续转换。

- 单次转换

  在单次转换模式下，ADC 会将通道的所有转换执行一次。

- 连续转换

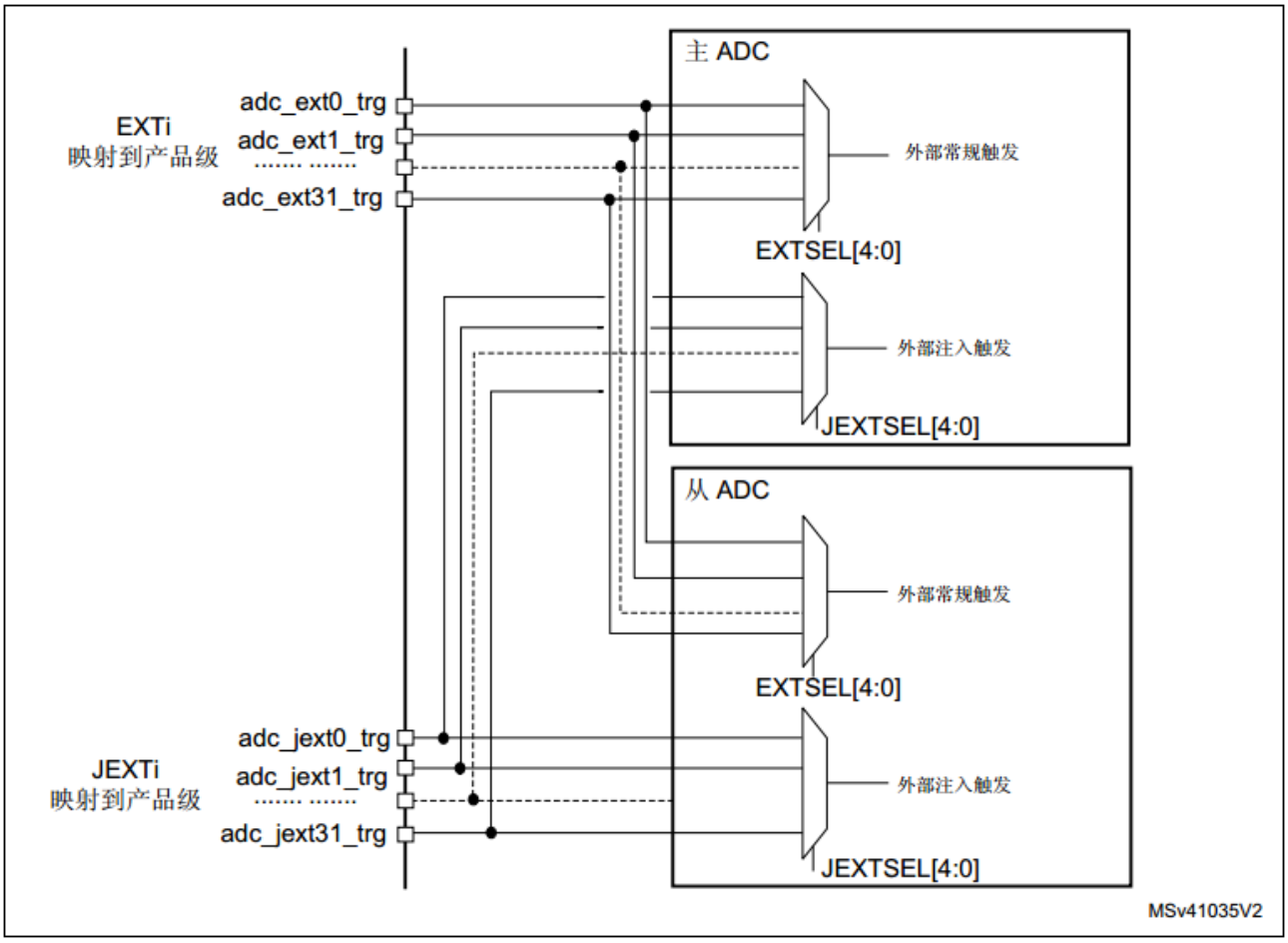
  该模式仅适用于常规通道。

  在连续转换模式下，如果发生软件或硬件触发，ADC 会执行所有常规通道的转换，随后会自动重启并继续执行每个通道的转换。

44.2.5 ADC 外部触发采样

STM32H7 既可以选择软件触发也可以选择外部硬件触发，并且可以设置触发边沿。

- 这里有一点要特别注意，对于 ADC1 和 ADC2 是共用相同的规则通道触发和注入通道触发：



◆ 外部触发支持上升沿、下降沿和双沿触发。

◆ 规则通道支持的外部触发源如下：

```
#define ADC_EXTERNALTRIG_T1_CC1          ((uint32_t)0x00000000)
#define ADC_EXTERNALTRIG_T1_CC2          ((uint32_t)ADC_CFGR_EXTSEL_0)
#define ADC_EXTERNALTRIG_T1_CC3          ((uint32_t)ADC_CFGR_EXTSEL_1)
#define ADC_EXTERNALTRIG_T2_CC2          ((uint32_t)(ADC_CFGR_EXTSEL_1 | ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_T3_TRGO         ((uint32_t)ADC_CFGR_EXTSEL_2)
#define ADC_EXTERNALTRIG_T4_CC4          ((uint32_t)(ADC_CFGR_EXTSEL_2 | ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_EXT_IT11        ((uint32_t)(ADC_CFGR_EXTSEL_2 | ADC_CFGR_EXTSEL_1))
#define ADC_EXTERNALTRIG_T8_TRGO         ((uint32_t)(ADC_CFGR_EXTSEL_2 | ADC_CFGR_EXTSEL_1 |
                                                     ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_T8_TRGO2        ((uint32_t) ADC_CFGR_EXTSEL_3)
#define ADC_EXTERNALTRIG_T1_TRGO         ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_T1_TRGO2        ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_1))
#define ADC_EXTERNALTRIG_T2_TRGO         ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_1 |
                                                     ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_T4_TRGO         ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_2))
#define ADC_EXTERNALTRIG_T6_TRGO         ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_2 |
                                                     ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_T15_TRGO        ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_2 |
                                                     ADC_CFGR_EXTSEL_1))
#define ADC_EXTERNALTRIG_T3_CC4          ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_2 |
                                                     ADC_CFGR_EXTSEL_1 | ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_HR1_ADCTRG1     ((uint32_t) ADC_CFGR_EXTSEL_4)
#define ADC_EXTERNALTRIG_HR1_ADCTRG3     ((uint32_t) (ADC_CFGR_EXTSEL_4 | ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_LPTIM1_OUT      ((uint32_t) (ADC_CFGR_EXTSEL_4 | ADC_CFGR_EXTSEL_1))
```

```
#define ADC_EXTERNALTRIG_LPTIM2_OUT       ((uint32_t) (ADC_CFGR_EXTSEL_4 | ADC_CFGR_EXTSEL_1|
                                                       ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_LPTIM3_OUT       ((uint32_t) (ADC_CFGR_EXTSEL_4 | ADC_CFGR_EXTSEL_2))
```

◆ 注入通道支持的外部触发源如下：

```
#define ADC_EXTERNALTRIGINJEC_T1_TRGO       ((uint32_t)0x00000000)
#define ADC_EXTERNALTRIGINJEC_T1_CC4        ((uint32_t)ADC_JSQR_JEXTSEL_0)
#define ADC_EXTERNALTRIGINJEC_T2_TRGO       ((uint32_t)ADC_JSQR_JEXTSEL_1)
#define ADC_EXTERNALTRIGINJEC_T2_CC1        ((uint32_t)(ADC_JSQR_JEXTSEL_1 | ADC_JSQR_JEXTSEL_0))
#define ADC_EXTERNALTRIGINJEC_T3_CC4        ((uint32_t)ADC_JSQR_JEXTSEL_2)
#define ADC_EXTERNALTRIGINJEC_T4_TRGO       ((uint32_t)(ADC_JSQR_JEXTSEL_2 | ADC_JSQR_JEXTSEL_0))
#define ADC_EXTERNALTRIGINJEC_EXT_IT15      ((uint32_t)(ADC_JSQR_JEXTSEL_2 | ADC_JSQR_JEXTSEL_1))
#define ADC_EXTERNALTRIGINJEC_T8_CC4        ((uint32_t)(ADC_JSQR_JEXTSEL_2 | ADC_JSQR_JEXTSEL_1 |
                                                        ADC_JSQR_JEXTSEL_0))
#define ADC_EXTERNALTRIGINJEC_T1_TRGO2      ((uint32_t)ADC_JSQR_JEXTSEL_3)
#define ADC_EXTERNALTRIGINJEC_T8_TRGO       ((uint32_t)(ADC_JSQR_JEXTSEL_3 | ADC_JSQR_JEXTSEL_0))
#define ADC_EXTERNALTRIGINJEC_T8_TRGO2      ((uint32_t)(ADC_JSQR_JEXTSEL_3 | ADC_JSQR_JEXTSEL_1))
#define ADC_EXTERNALTRIGINJEC_T3_CC3        ((uint32_t)(ADC_JSQR_JEXTSEL_3 | ADC_JSQR_JEXTSEL_1 |
                                                        ADC_JSQR_JEXTSEL_0))
#define ADC_EXTERNALTRIGINJEC_T3_TRGO       ((uint32_t)(ADC_JSQR_JEXTSEL_3 | ADC_JSQR_JEXTSEL_2))
#define ADC_EXTERNALTRIGINJEC_T3_CC1        ((uint32_t)(ADC_JSQR_JEXTSEL_3 | ADC_JSQR_JEXTSEL_2 |
                                                        ADC_JSQR_JEXTSEL_0))
#define ADC_EXTERNALTRIGINJEC_T6_TRGO       ((uint32_t)(ADC_JSQR_JEXTSEL_3 | ADC_JSQR_JEXTSEL_2 |
                                                        ADC_JSQR_JEXTSEL_1))
#define ADC_EXTERNALTRIGINJEC_T15_TRGO      ((uint32_t)(ADC_JSQR_JEXTSEL_3 | ADC_JSQR_JEXTSEL_2 |
                                                        ADC_JSQR_JEXTSEL_1 | ADC_JSQR_JEXTSEL_0))
#define ADC_EXTERNALTRIGINJEC_HR1_ADCTRG2   ((uint32_t)ADC_JSQR_JEXTSEL_4)
#define ADC_EXTERNALTRIGINJEC_HR1_ADCTRG4   ((uint32_t)(ADC_JSQR_JEXTSEL_4 | ADC_JSQR_JEXTSEL_0))
#define ADC_EXTERNALTRIGINJEC_LPTIM1_OUT    ((uint32_t)(ADC_JSQR_JEXTSEL_4 | ADC_JSQR_JEXTSEL_1))
#define ADC_EXTERNALTRIGINJEC_LPTIM2_OUT    ((uint32_t)(ADC_JSQR_JEXTSEL_4 | ADC_JSQR_JEXTSEL_1 |
                                                        ADC_JSQR_JEXTSEL_0))
#define ADC_EXTERNALTRIGINJEC_LPTIM3_OUT    ((uint32_t)(ADC_JSQR_JEXTSEL_4 | ADC_JSQR_JEXTSEL_2))
```

44.2.6 ADC 多通道连接方式

ADC1，ADC2 和 ADC3 均支持 20 条通道扫描采样（注意，部分引脚是多个 ADC 共用的）：

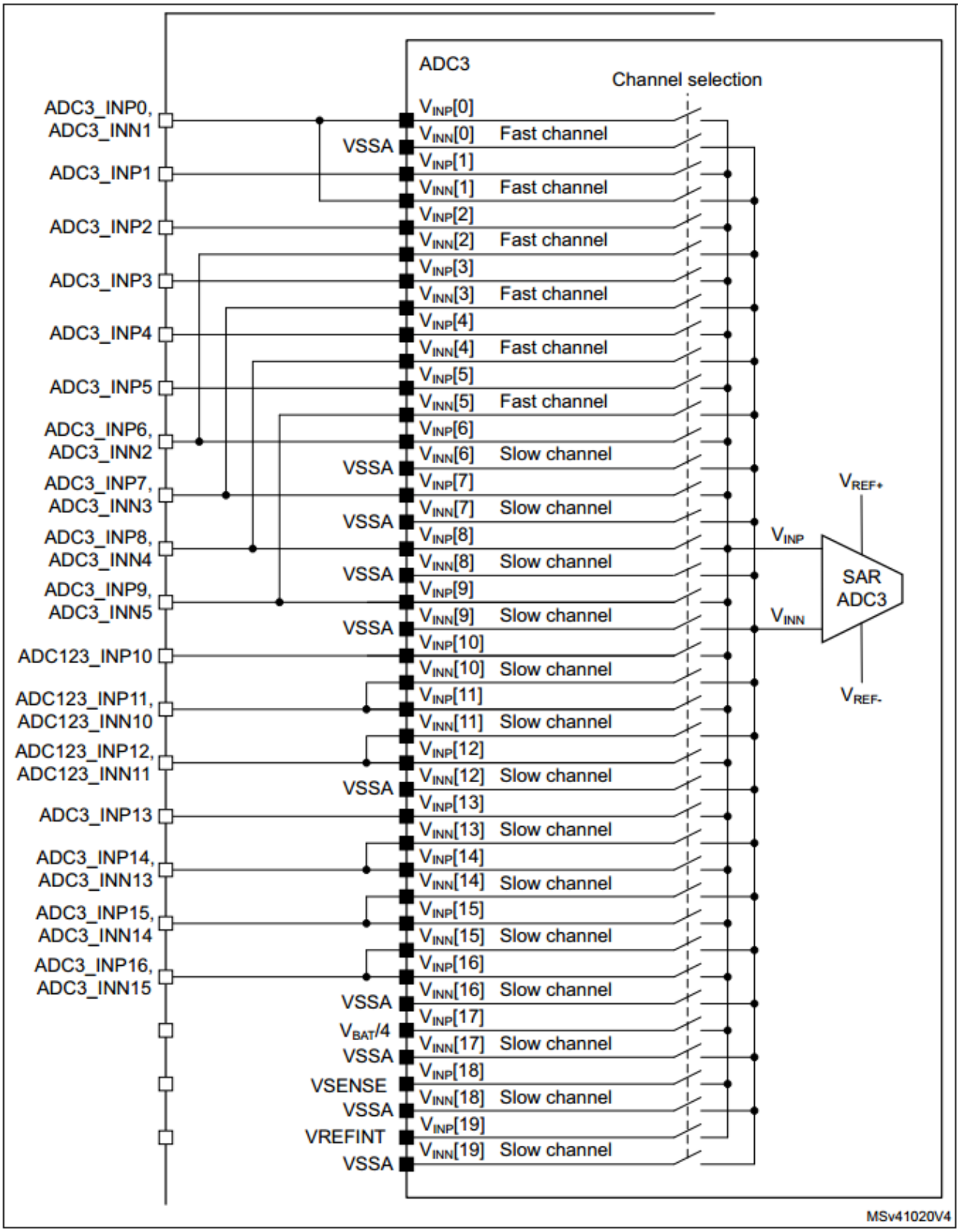
◆ 6 路快速模拟输入 (ADCx_INP[0]/INN[0] 到 ADCx_INP[5]/INN[5])

◆ 14 路慢速模拟输入 (ADCx_INP[6]/INN[6] 到 ADCx_INP[19]/INN[19])

◆ ADC 连接至 5 路内部模拟输入：

- 内部温度传感器 (VSENSE) 连接到 ADC3_INP18
- 内部参考电压 (VREFINT) 连接到 ADC3_INP19
- VBAT 监测通道 (VBAT/4) 连接到 ADC3_INP17
- DAC 内部通道 1 连接到 ADC2_INP16
- DAC 内部通道 2 连接到 ADC2_INP17

反映到硬件上，这些通道的连接方式就是下面这样（以 ADC3 为例）：


ADC3
Channel selection
ADC3_INP0, ADC3_INN1
ADC3_INP1
ADC3_INP2
ADC3_INP3
ADC3_INP4
ADC3_INP5
ADC3_INP6, ADC3_INN2
ADC3_INP7, ADC3_INN3
ADC3_INP8, ADC3_INN4
ADC3_INP9, ADC3_INN5
ADC123_INP10
ADC123_INP11, ADC123_INN10
ADC123_INP12, ADC123_INN11
ADC3_INP13
ADC3_INP14, ADC3_INN13
ADC3_INP15, ADC3_INN14
ADC3_INP16, ADC3_INN15
VSSA
VBAT/4
VSSA
VSENSE
VSSA
VREFINT
VSSA
VINP[0]
VINN[0] Fast channel
VINP[1]
VINN[1] Fast channel
VINP[2]
VINN[2] Fast channel
VINP[3]
VINN[3] Fast channel
VINP[4]
VINN[4] Fast channel
VINP[5]
VINN[5] Fast channel
VINP[6]
VINN[6] Slow channel
VINP[7]
VINN[7] Slow channel
VINP[8]
VINN[8] Slow channel
VINP[9]
VINN[9] Slow channel
VINP[10]
VINN[10] Slow channel
VINP[11]
VINN[11] Slow channel
VINP[12]
VINN[12] Slow channel
VINP[13]
VINN[13] Slow channel
VINP[14]
VINN[14] Slow channel
VINP[15]
VINN[15] Slow channel
VINP[16]
VINN[16] Slow channel
VINP[17]
VINN[17] Slow channel
VINP[18]
VINN[18] Slow channel
VINP[19]
VINN[19] Slow channel
VREF+
VINP
VINN
SAR ADC3
VREF-
MSv41020V4

44.2.7 ADC 多通道扫描时序

ADC 的多通道采样过程是单个 ADC 通过多路选择器不断切换不同的通道进行采样的，也就是说当前通道采集完成后才会进行下一个通道的采样。

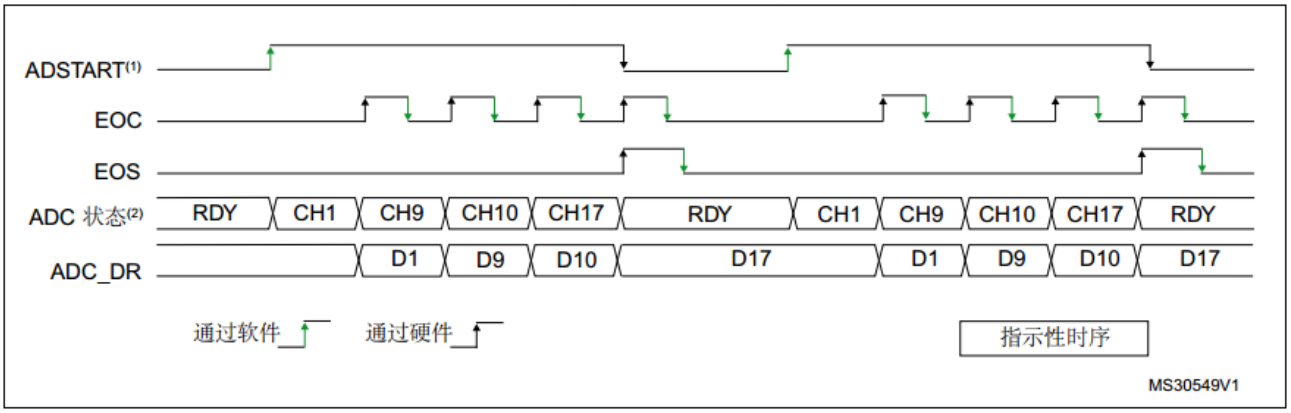
通过下面这四幅时序图可以让大家有个感性的认识：

◆ **单次转换序列，软件触发：**

ADSTART 表示软件启动转换。

EOC 表示一个通道转换结束。

EOS 表示所有通道转换结束。



关于这个时序图的解读：

- 配置为单次转换的话，每次软件启动，所有通道仅进行一次转换，如果需要再次转换，需要再启动一次。
- 每个通过转换完毕有个 EOC 标志，所有通道转换完毕有个 EOS 标志。

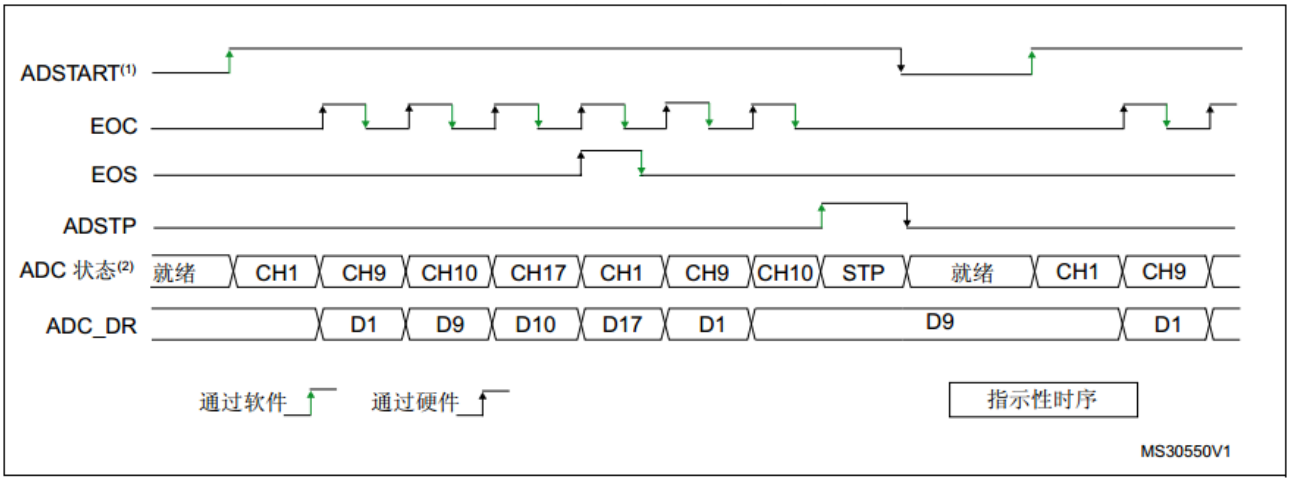
◆ **连续转换序列，软件触发：**

ADSTART 表示软件启动转换。

ADSTP 表示停止转换。

EOC 表示一个通道转换结束。

EOS 表示所有通道转换结束。



关于这个时序图的解读：

- 配置为连续转换的话，软件启动 ADSTART 会开启所有通道转换，全部转换完毕后，继续进行下一轮转换。调用了停止转换 ADSTP 后，会停止转换。
- 每个通过转换完毕有个 EOC 标志，所有通道转换完毕有个 EOS 标志。

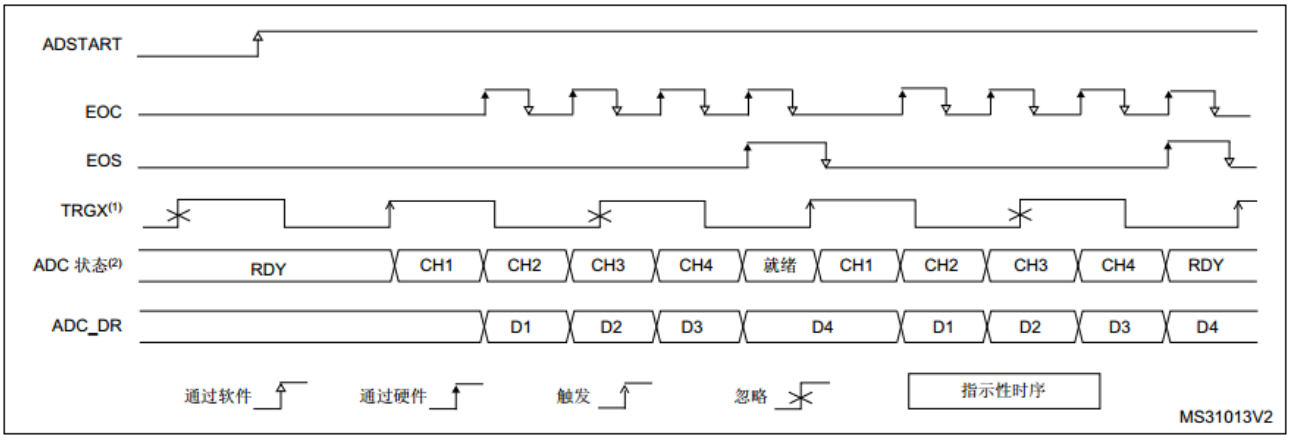
◆ **单次转换序列，硬件触发：**

ADSTART 表示软件启动转换。

EOC 表示一个通道转换结束。

EOS 表示所有通道转换结束。

TRGX 表示硬件上升沿触发。



关于这个时序图的解读：

- 软件启动 ADSTART 后还不会开启转换，TRGX 硬件上升沿触发才会启动转换，全部转换完毕后，再来一个 TRGX 硬件上升沿触发会继续进行下一轮转换。如果所有通道转换期间有个 TRGX 硬件上升沿会被忽略。
- 每个通过转换完毕有个 EOC 标志，所有通道转换完毕有个 EOS。

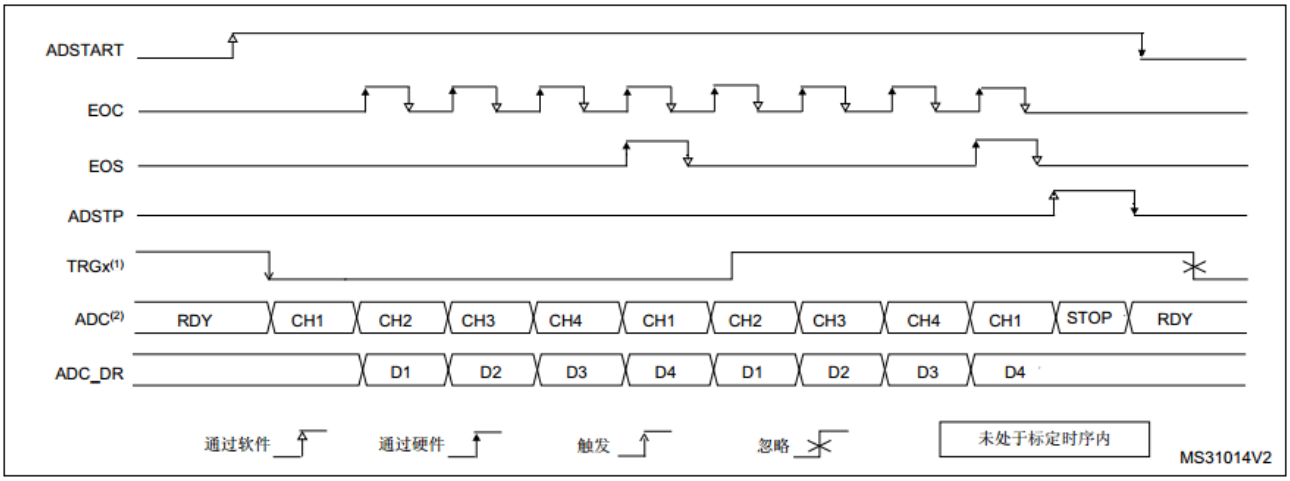
◆ **连续转换序列，硬件触发：**

ADSTART 表示软件启动转换。

EOC 表示一个通道转换结束。

EOS 表示所有通道转换结束。

TRGX 表示硬件上升沿触发。



关于这个时序图的解读：

- 软件启动 ADSTART 后还不会开启转换，TRGX 硬件下降沿触发才会启动转换，一旦开启转换，所有通道的转换会一轮接一轮的进行下去，也就是说 TRGX 硬件下降沿会被忽略。调用了停止转换 ADSTP 后，会停止转换。停止后再有 TRGX 硬件下降沿触发不会再转换，必须再次启动 ADSTART 才行。
- 每个通过转换完毕有个 EOC 标志，所有通道转换完毕有个 EOS。

44.2.8 ADC 单端和差分的支持

初学的话，容易有几个概念搞不清楚，单极性，双极性，真差分和伪差分。在此贴里面有一个专门的截图：http://www.armbbs.cn/forum.php?mod=viewthread&tid=89397 （暂未找到这个图片的原始出处，所以先不放在教程里面）。

单极性，双极性比较好理解，就是单电源供电或者双电源供电，这里的双电源是指的正负电压供电。

- 单端输入是第 1 幅图的效果，ADC 读取 VIN 和 GND 的差值。
- 伪差分 AIP-AIN 就是第 5 幅图，内部 ADC 读取 AIP 和 AIN 的差值，但允许 AIN 上有一个很小的共模电压，比如正负 0.3V。
- 真差分是 AIP-AIN 就是第 2 幅或者第 5 幅图，其内部 AIP 和 AIN 分别有一个 ADC，分别读取转换 AIP-GND，和 AIN-GND，再对这两个数字值做差，所以 AIN 上也可以接收很大的共模值。

STM32H7 的差分属于单极性真差分，也就是不可以测量负压。另外要注意下面内容：

- 单端输入模式下，通道 i 转换的模拟电压是 VINP[i]正向复用引脚与 VREF-之差。
- 差分输入模式下，通道 i 转换的模拟电压是 VINP[i]正向复用引脚与 VINN[i]反向复用引脚之差。

- 差分模式的输出数据是无符号数据。当 VINP[i]为 VREF-、VINN[i]为 VREF+时，输出数据为 0x0000（16 位分辨率模式）；当 VINP[i]为 VREF+、VINN[i]为 VREF-时，输出数据为 0xFFFF。对应的公式如下：

$$转换后的值 = \frac{ADC_Full_Scale}{2} \times \left[1 + \frac{V_{INP} - V_{INN}}{V_{REF+}}\right]$$

（1）当 ADC 配置为差分模式时，两路输入的偏置电压均为 Vref+/2。

（2）输入信号应为差分信号且共模电压应固定。

44.2.9 ADC 过采样机制

过采样的意思就是提高单位时间的采样次数，比如原来每秒采集 1 次，那么 16 倍过采样就是每秒要采集 16 次。

STM32H7 最高支持 1024 倍过采样，1024 次采样数据累加后存到 ADC 的数据寄存器里面。如果想求 1024 次采集的平均数，也不需要用户参与计算，ADC 的 CFGR2 寄存器 OVSS[3:0]位支持右移操作（右移 1 位到 11 位均可配置），可以方便的求平均。

这个功能在实际项目还是非常实用的。比如下面的测试：

- 测试条件

  做了一个 ADC3+DMA 的多通道采样。

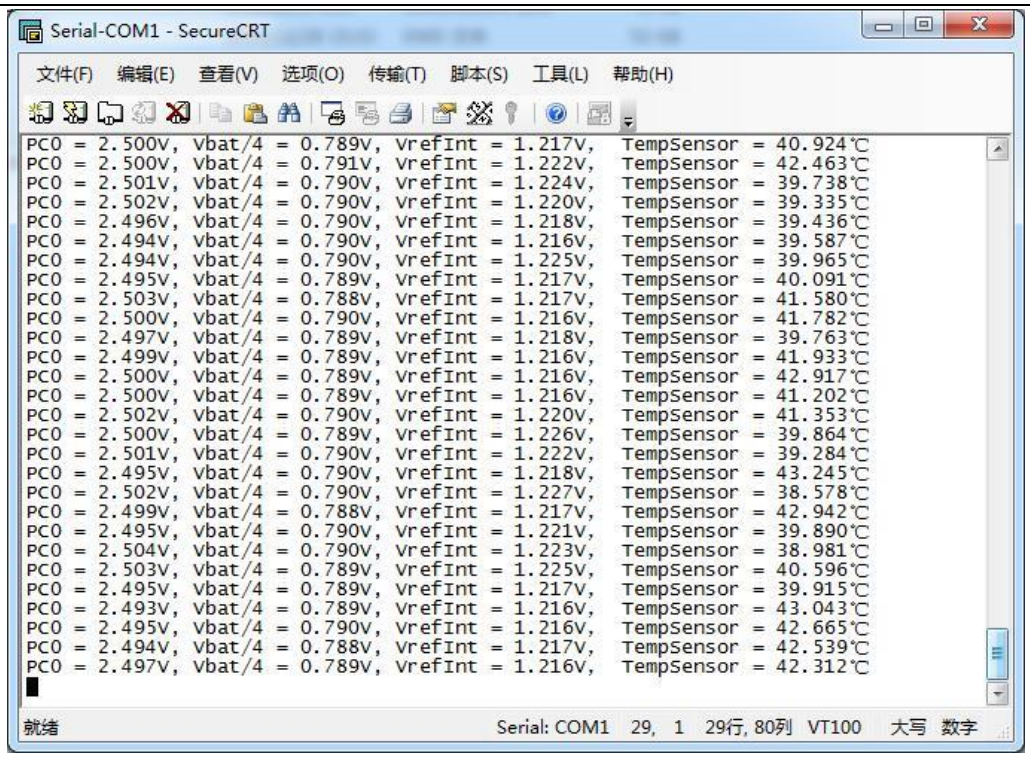
  通道 1：PC0 采集 2.5V 的稳压基准。

  通道 2：Vbat/4。

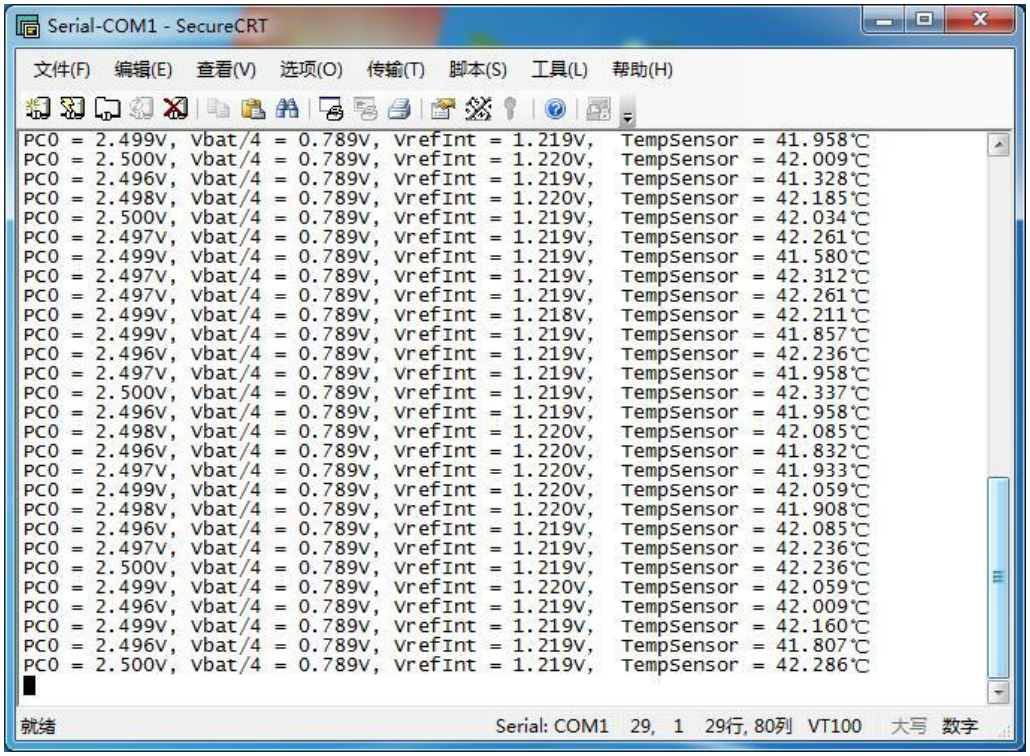
  通道 3：VrefInt。

  通道 4：温度。

- 不做任何处理的效果



◆ 16 倍过采样求平均后的效果。

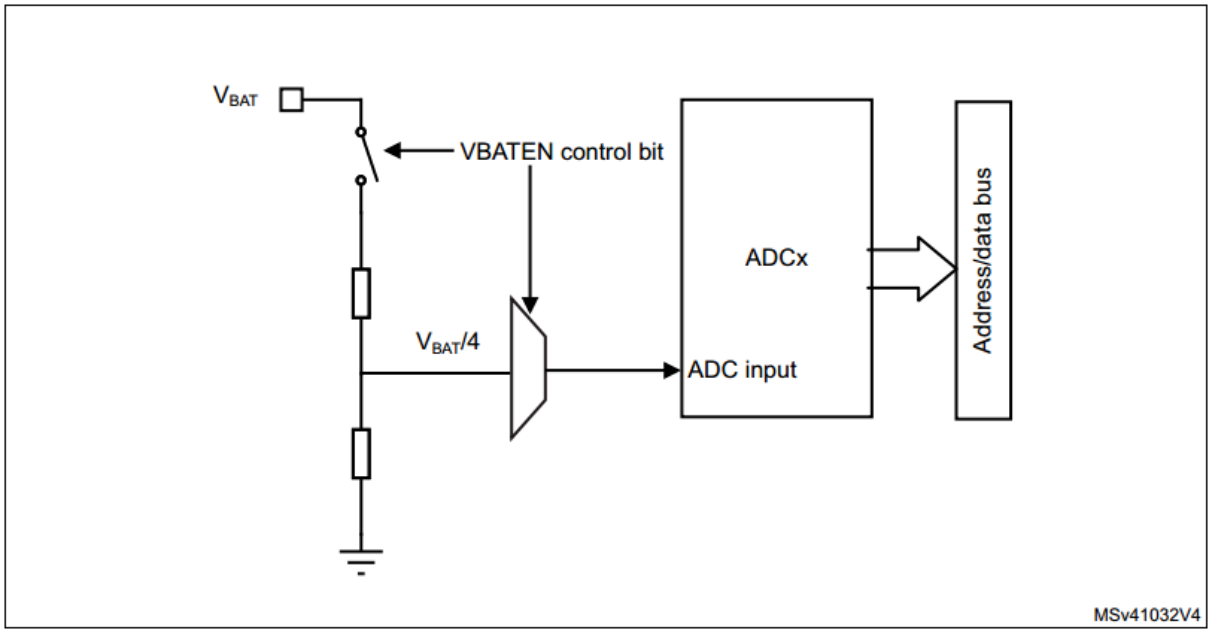


44.2.10 ADC 的 Vbat/4，VrefInt 和温度采样

◆ Vbat/4 电池监测电压

Vbat/4 连接至 ADC3_INP17，所以可以使用 ADC3 的通道 17 进行测量。为什么不是直接测试 Vbat，

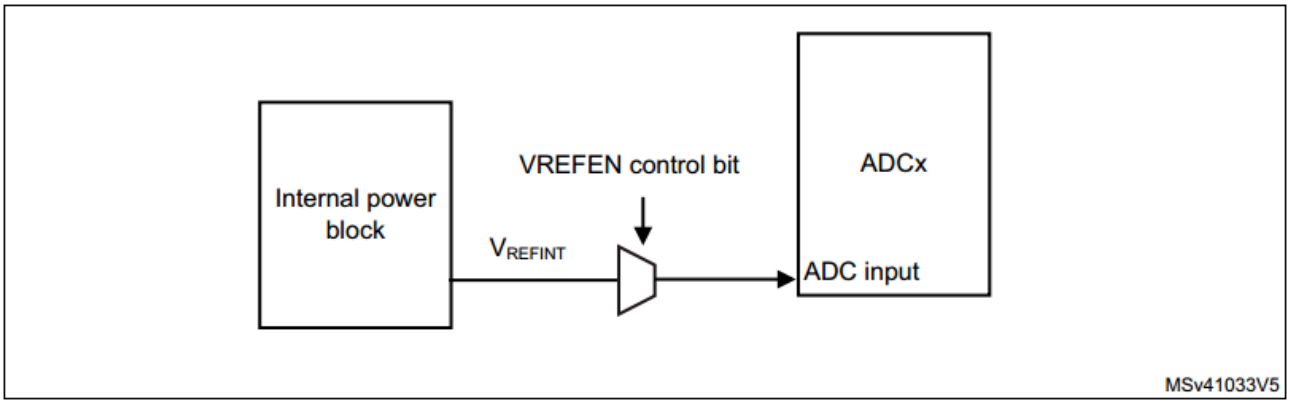
因为 Vbat 电压有可能高于 Vdda，导致 ADC3 测量电压超出范围。Vbat 的测量框图如下：



注意：必须将 VBATEN 位置 1 才能使能内部通道 ADC3_INP17 采集。

◆ VrefInt 内部电源模块参考电压

VrefInt 连接至 ADC3_INP19，所以可以使用 ADC3 的通道 19 进行测量。可以通过监测内部电源模块参考电压 VrefInt 来评估 ADC Vref+电压的参考值。VrefInt 的测量框图如下：



注意：必须将 ADCx_CCR 寄存器中的 VREFEN 位置 1 才能使能内部通道 ADC3_INP19 采集。

◆ 温度测量

STM32H7 带有温度传感器，可以使用 ADC3_INP18 进行测量，不过读取出来的还是个电压值，需要将其转换为温度值，调用下面的转换公式即可：

$$\text{Temperature (in °C)} = \frac{110\ °C - 30\ °C}{TS_CAL2 - TS_CAL1} \times (TS_DATA - TS_CAL1) + 30\ °C$$

TS_CAL1 = *(__IO uint16_t *)(0x1FF1E820);

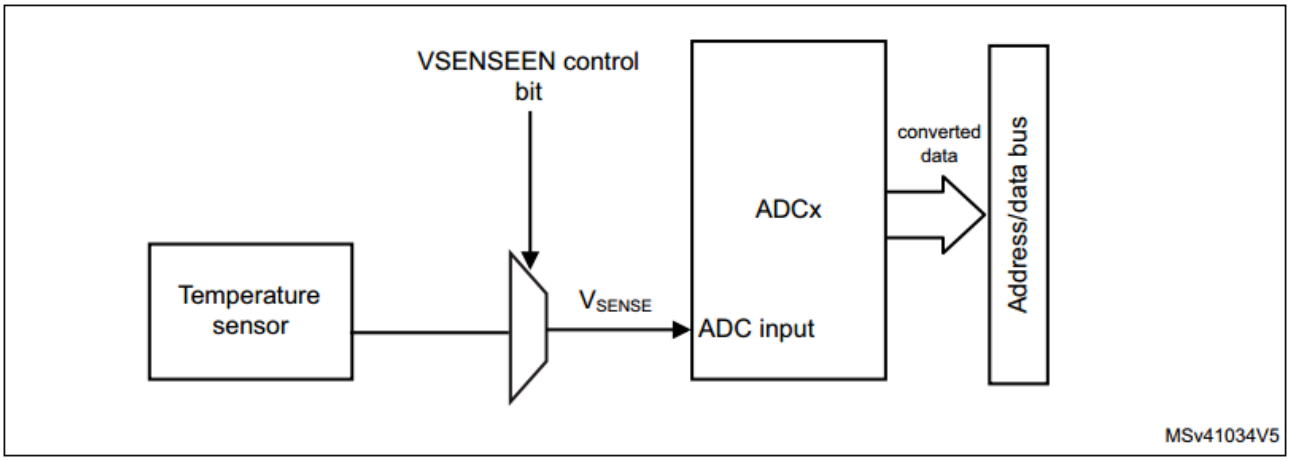
TS_CAL2 = *(__IO uint16_t *)(0x1FF1E840);

TS_CAL2 表示温度 110℃时的 ADC 测量值，读取地址 0x1FF1E820 可以获得。

TS_CAL1 表示温度 30℃时的 ADC 测量值，读取地址 0x1FF1E840 可以获得。

TS_DATA 表示当前的测量值，获得当前的测试值代入上面公式就可以获取温度。

温度测量的框图如下：



注意：必须将 VSENSEEN 位置 1 才能使能内部通道 ADC3_VINP18 采集。

44.2.11 ADC 校准问题

STM32H7 的 ADC 支持偏移校准和线性度校准，两种校准实现都比较方便，HAL 库已经为我们做好了，直接调用 API 即可，但是使用中务必注意此贴的问题：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=91436 。

44.2.12 ADC 电气特性（重要）

如果使用 ADC 的话，部分电气特性一定要了解：

Symbol	Parameter	Conditions		Min	Typ	Max	Unit
V_{DDA}	Analog power supply	-		1.62	-	3.6	V
V_{REF+}	Positive reference voltage	$V_{DDA} \geq 2$ V		2	-	V_{DDA}	
		$V_{DDA} < 2$ V		V_{DDA}			
V_{REF-}	Negative reference voltage	-		V_{SSA}			
f_{ADC}	ADC clock frequency	2 V ≤ V_{DDA} ≤ 3.3 V	BOOST = 1	-	-	36	MHz
			BOOST = 0	-	-	20	
f_S	Sampling rate for Fast channels, BOOST = 1, f_{ADC} = 36 MHz	16-bit resolution		-	-	3.60	MSPS
		14-bit resolution		-	-	4.00	
		12-bit resolution		-	-	4.50	
		10-bit resolution		-	-	5.00	
		8-bit resolution				6.00	
	Sampling rate for Fast channels, BOOST = 0, f_{ADC} = 20 MHz	16-bit resolution		-	-	2.00	
		14-bit resolution		-	-	2.20	
		12-bit resolution		-	-	2.50	
		10-bit resolution		-	-	2.80	
		8-bit resolution				3.30	
	Sampling rate for Slow channels, BOOST = 0, f_{ADC} = 10 MHz	16-bit resolution		-	-	1.00	
		14-bit resolution		-	-	1.00	
		12-bit resolution		-	-	1.00	
		10-bit resolution		-	-	1.00	
		8-bit resolution				1.00	

Symbol	Parameter	Conditions	Min	Typ	Max	Unit
f_{TRIG}	External trigger frequency	f_{ADC} = 36 MHz	-	-	3.6	MHz
		16-bit resolution	-	-	10	$1/f_{ADC}$
V_{AIN}(2)	Conversion voltage range	-	0	-	V_{REF+}	V
V_{CMIV}	Common mode input voltage	-	$V_{REF}/2-10\%$	$V_{REF}/2$	$V_{REF}/2+10\%$	
R_{AIN}	External input impedance	-	-	-	50	kΩ
C_{ADC}	Internal sample and hold capacitor	-	-	4	-	pF
t_{ADCREG_STUP}	ADC LDO startup time	-	-	5	10	µs
t_{STAB}	ADC power-up time	LDO already started	1			conversion cycle

通过上面的截图，我们要了解到以下几点：

- BOOST=1 的时候，ADC 的最高采样率 36MHz。（BOOST 位可以通过寄存器配置）
- BOOST=0 的时候，ADC 的最高采样率 20MHz。
- 对于快速通道（Fast Channels），配置的分辨率越低，支持的最高采样率越高。

- 对于慢速通道（Slow Channels），不同分辨率支持的最高采样率一样。
- 支持的测量范围是 0 – Vref+，不支持负压测量。以我们 V7 开发板为例，稳压基准 Vref 是通过跳线帽设置的，可以选择 2.5V，也可以选择 3.3V。
- 差分测量时，共模电压的典型值是 Vref / 2。
- ADC 的输入阻抗最大值是 50KΩ。

44.3 ADC 的 HAL 库用法

ADC 的 HAL 库用法其实就是几个结构体变量成员的配置和使用，然后配置时钟，并根据需要配置 NVIC、中断和 DMA。

只是 ADC 涉及到定义非常多，下面我们逐一展开为大家做个说明。

44.3.1 ADC 寄存器结构体 ADC_TypeDef

ADC 相关的寄存器是通过 HAL 库中的结构体 ADC_TypeDef 和 ADC_Common_TypeDef 定义的，在 stm32h743xx.h 中可以找到它们的具体定义如下：

```
/**
  * @brief Analog to Digital Converter
  */
typedef struct
{
  __IO uint32_t ISR;        /*!< ADC Interrupt and Status Register,                Address offset: 0x00 */
  __IO uint32_t IER;        /*!< ADC Interrupt Enable Register,                    Address offset: 0x04 */
  __IO uint32_t CR;         /*!< ADC control register,                             Address offset: 0x08 */
  __IO uint32_t CFGR;       /*!< ADC Configuration register,                       Address offset: 0x0C */
  __IO uint32_t CFGR2;      /*!< ADC Configuration register 2,                     Address offset: 0x10 */
  __IO uint32_t SMPR1;      /*!< ADC sample time register 1,                       Address offset: 0x14 */
  __IO uint32_t SMPR2;      /*!< ADC sample time register 2,                       Address offset: 0x18 */
  __IO uint32_t PCSEL;      /*!< ADC pre-channel selection,                        Address offset: 0x1C */
  __IO uint32_t LTR1;       /*!< ADC watchdog Lower threshold register 1,          Address offset: 0x20 */
  __IO uint32_t HTR1;       /*!< ADC watchdog higher threshold register 1,         Address offset: 0x24 */
  uint32_t      RESERVED1;  /*!< Reserved, 0x028                                                        */
  uint32_t      RESERVED2;  /*!< Reserved, 0x02C                                                        */
  __IO uint32_t SQR1;       /*!< ADC regular sequence register 1,                  Address offset: 0x30 */
  __IO uint32_t SQR2;       /*!< ADC regular sequence register 2,                  Address offset: 0x34 */
  __IO uint32_t SQR3;       /*!< ADC regular sequence register 3,                  Address offset: 0x38 */
  __IO uint32_t SQR4;       /*!< ADC regular sequence register 4,                  Address offset: 0x3C */
  __IO uint32_t DR;         /*!< ADC regular data register,                        Address offset: 0x40 */
  uint32_t      RESERVED3;  /*!< Reserved, 0x044                                                        */
  uint32_t      RESERVED4;  /*!< Reserved, 0x048                                                        */
  __IO uint32_t JSQR;       /*!< ADC injected sequence register,                   Address offset: 0x4C */
  uint32_t      RESERVED5[4]; /*!< Reserved, 0x050 - 0x05C                                              */
  __IO uint32_t OFR1;       /*!< ADC offset register 1,                            Address offset: 0x60 */
  __IO uint32_t OFR2;       /*!< ADC offset register 2,                            Address offset: 0x64 */
  __IO uint32_t OFR3;       /*!< ADC offset register 3,                            Address offset: 0x68 */
  __IO uint32_t OFR4;       /*!< ADC offset register 4,                            Address offset: 0x6C */
  uint32_t      RESERVED6[4]; /*!< Reserved, 0x070 - 0x07C                                              */
  __IO uint32_t JDR1;       /*!< ADC injected data register 1,                     Address offset: 0x80 */
```

```
  __IO uint32_t JDR2;       /*!< ADC injected data register 2,                    Address offset: 0x84 */
  __IO uint32_t JDR3;       /*!< ADC injected data register 3,                    Address offset: 0x88 */
  __IO uint32_t JDR4;       /*!< ADC injected data register 4,                    Address offset: 0x8C */
  uint32_t      RESERVED7[4]; /*!< Reserved, 0x090 - 0x09C                                            */
  __IO uint32_t AWD2CR;     /*!< ADC  Analog Watchdog 2 Configuration Register,   Address offset: 0xA0 */
  __IO uint32_t AWD3CR;     /*!< ADC  Analog Watchdog 3 Configuration Register,   Address offset: 0xA4 */
  uint32_t      RESERVED8;  /*!< Reserved, 0x0A8                                                    */
  uint32_t      RESERVED9;  /*!< Reserved, 0x0AC                                                    */
  __IO uint32_t LTR2;       /*!< ADC watchdog Lower threshold register 2,         Address offset: 0xB0 */
  __IO uint32_t HTR2;       /*!< ADC watchdog Higher threshold register 2,        Address offset: 0xB4 */
  __IO uint32_t LTR3;       /*!< ADC watchdog Lower threshold register 3,         Address offset: 0xB8 */
  __IO uint32_t HTR3;       /*!< ADC watchdog Higher threshold register 3,        Address offset: 0xBC */
  __IO uint32_t DIFSEL;     /*!< ADC  Differential Mode Selection Register,       Address offset: 0xC0 */
  __IO uint32_t CALFACT;    /*!< ADC  Calibration Factors,                        Address offset: 0xC4 */
  __IO uint32_t CALFACT2;   /*!< ADC  Linearity Calibration Factors,              Address offset: 0xC8 */
} ADC_TypeDef;

typedef struct
{
__IO uint32_t CSR; /*!< ADC Common status register, Address offset: ADC1/3 base address + 0x300 */
uint32_t RESERVED; /*!< Reserved, ADC1/3 base address + 0x304 */
__IO uint32_t CCR; /*!< ADC common control register, Address offset: ADC1/3 base address + 0x308 */
__IO uint32_t CDR; /*!< ADC common regular data register for dual Address offset: ADC1/3 base address+0x30C */
__IO uint32_t CDR2; /*!< ADC common regular data register for 32-bit dual mode Address offset: ADC1/3 base address
+ 0x310 */
} ADC_Common_TypeDef;
```

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

结构体变量 ADC_TypeDef 用于 ADC1，ADC2 和 ADC3，每个 ADC 都有一组。结构体变量 ADC_Common_TypeDef 是公共寄存器，ADC1 和 ADC2 共用一组，而 ADC3 单独用一组。

下面我们再看 ADC1，ADC2 和 ADC3 以及公共寄存器的定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE           ((uint32_t)0x40000000)
#define D2_AHB1PERIPH_BASE    (PERIPH_BASE + 0x00020000)
#define D3_AHB1PERIPH_BASE    (PERIPH_BASE + 0x18020000)

#define ADC1_BASE             (D2_AHB1PERIPH_BASE + 0x2000)
#define ADC2_BASE             (D2_AHB1PERIPH_BASE + 0x2100)
#define ADC12_COMMON_BASE     (D2_AHB1PERIPH_BASE + 0x2300)
#define ADC3_BASE             (D3_AHB1PERIPH_BASE + 0x6000)
#define ADC3_COMMON_BASE      (D3_AHB1PERIPH_BASE + 0x6300)

#define ADC1                  ((ADC_TypeDef *) ADC1_BASE) <----- 展开这个宏，(ADC_TypeDef *) 0x40022000
#define ADC2                  ((ADC_TypeDef *) ADC2_BASE)
#define ADC3                  ((ADC_TypeDef *) ADC3_BASE)
#define ADC12_COMMON          ((ADC_Common_TypeDef *) ADC12_COMMON_BASE)
#define ADC3_COMMON           ((ADC_Common_TypeDef *) ADC3_COMMON_BASE)
```

我们访问 ADC1 的 ISR 寄存器可以采用这种形式：ADC1->ISR = 0;

44.3.2 ADC 句柄结构体 ADC_HandleTypeDef

HAL 库在 ADC_TypeDef 的基础上封装了一个结构体 ADC_HandleTypeDef，定义如下：

```
typedef struct
{
  ADC_TypeDef                    *Instance;
  ADC_InitTypeDef                Init;
  DMA_HandleTypeDef              *DMA_Handle;
  HAL_LockTypeDef                Lock;
  __IO uint32_t                  State;
  __IO uint32_t                  ErrorCode;
  ADC_InjectionConfigTypeDef     InjectionConfig ;
}ADC_HandleTypeDef;
```

下面将这几个参数逐一做个说明。

- **ADC_TypeDef *Instance**

  这个参数是寄存器的例化，方便操作寄存器，比如使能 ADC 内部稳压器。

  SET_BIT(hadc->Instance->CR, ADC_CR_ADVREGEN);

- **DMA_InitTypeDef Init;**

  这个参数是用户接触最多的，用于配置 ADC 的基本参数，像 ADC 时钟、分辨率、扫描模式、过采样等。ADC_InitTypeDef 结构体的定义如下：

```
typedef struct
{
  uint32_t ClockPrescaler;
  uint32_t Resolution;
  uint32_t ScanConvMode;
  uint32_t EOCSelection;
  FunctionalState LowPowerAutoWait;
  FunctionalState ContinuousConvMode;
  uint32_t NbrOfConversion;
  FunctionalState DiscontinuousConvMode;
  uint32_t NbrOfDiscConversion;
  uint32_t ExternalTrigConv;
  uint32_t ExternalTrigConvEdge;
  uint32_t ConversionDataManagement;
  uint32_t Overrun;
  uint32_t LeftBitShift;
  FunctionalState BoostMode;
  FunctionalState OversamplingMode;
  ADC_OversamplingTypeDef Oversampling;
}ADC_InitTypeDef;
```

具体每个成员的含义在本章 3.3 小节有说明。

- **DMA_HandleTypeDef *DMA_Handle;**

  如果 ADC 使用 DMA 模式的话，此参数用于关联 DMA 的句柄，方便 DMA 的配置。

- **HAL_LockTypeDef Lock**

  __IO uint32_t State;

  __IO uint32_t ErrorCode

这三个变量主要供函数内部使用。Lock 用于设置锁状态，State 用于设置 ADC 通信状态，而 ErrorCode 用于配置代码错误。

- **ADC_InjectionConfigTypeDef InjectionConfig**

  用于配置 ADC 注入模式。

44.3.3 ADC 参数初始化结构体 ADC_InitTypeDef

参数初始化结构体 ADC _InitTypeDef 要注意的事项比较多，所以专门开一个小节单独说明。

跟寄存器结构体 ADC_TypeDef 一样，参数初始化结构体 ADC_InitTypeDef 也是封装在了 ADC 的句柄结构体 ADC_HandleTypeDef 里面，定义如下：

```
typedef struct
{
  uint32_t ClockPrescaler;
  uint32_t Resolution;
  uint32_t ScanConvMode;
  uint32_t EOCSelection;
  FunctionalState LowPowerAutoWait;
  FunctionalState ContinuousConvMode;
  uint32_t NbrOfConversion;
  FunctionalState DiscontinuousConvMode;
  uint32_t NbrOfDiscConversion;
  uint32_t ExternalTrigConv;
  uint32_t ExternalTrigConvEdge;
  uint32_t ConversionDataManagement;
  uint32_t Overrun;
  uint32_t LeftBitShift;
  FunctionalState BoostMode;
  FunctionalState OversamplingMode;
  ADC_OversamplingTypeDef Oversampling;
}ADC_InitTypeDef;
```

下面将这几个参数逐一做说明：

- ClockPrescaler

  用于 ADC 的时钟分频设置，ADC 有两种时钟源可供选择，可以使用来自 AHB 总线的系统时钟（属于同步时钟），也可以使用 PLL2，PLL3，HSE，HSI 或者 CSI 时钟（属于异步时钟）。

  - ADC1，ADC2 和 ADC3 共用选择的时钟。
  - ADC 的时钟源使用 AHB 时钟，且使用注入模式，那么在 16bit，14bit，12bit 或者 10bit 分辨率时，ADC 的时钟不能超过 AHB 时钟的四分之一。8bit 模式时，不能超过 AHB 时钟的三分之一。
  - 选择 AHB 时钟的话，ADC 的配置中提供了不分频，二分频和四分频。如果选择了不分频，那么配置 AHB 的时钟输出时也不可以设置分频，即 RCC 的 CFGR 寄存器配置不可分频。
  - 如果使用 PLL 时钟，运行期间要一直开启，不可关闭。
  - 此参数仅可以在关闭 ADC 的时候配置。

有以下几种参数可供选择：

```
/** @defgroup ADC_ClockPrescaler ADC clock source and clock prescaler
  * @{
  */
#define ADC_CLOCK_SYNC_PCLK_DIV1    ((uint32_t)ADC_CCR_CKMODE_0)
#define ADC_CLOCK_SYNC_PCLK_DIV2    ((uint32_t)ADC_CCR_CKMODE_1)
#define ADC_CLOCK_SYNC_PCLK_DIV4    ((uint32_t)ADC_CCR_CKMODE)

#define ADC_CLOCKPRESCALER_PCLK_DIV1    ADC_CLOCK_SYNC_PCLK_DIV1    /* 这三个仅仅是为了兼容，已经不推荐使用 */
#define ADC_CLOCKPRESCALER_PCLK_DIV2    ADC_CLOCK_SYNC_PCLK_DIV2
#define ADC_CLOCKPRESCALER_PCLK_DIV4    ADC_CLOCK_SYNC_PCLK_DIV4

#define ADC_CLOCK_ASYNC_DIV1        ((uint32_t)0x00000000)
#define ADC_CLOCK_ASYNC_DIV2        ((uint32_t)ADC_CCR_PRESC_0)
#define ADC_CLOCK_ASYNC_DIV4        ((uint32_t)ADC_CCR_PRESC_1)
#define ADC_CLOCK_ASYNC_DIV6        ((uint32_t)(ADC_CCR_PRESC_1|ADC_CCR_PRESC_0))
#define ADC_CLOCK_ASYNC_DIV8        ((uint32_t)(ADC_CCR_PRESC_2))
#define ADC_CLOCK_ASYNC_DIV10       ((uint32_t)(ADC_CCR_PRESC_2|ADC_CCR_PRESC_0))
#define ADC_CLOCK_ASYNC_DIV12       ((uint32_t)(ADC_CCR_PRESC_2|ADC_CCR_PRESC_1))
#define ADC_CLOCK_ASYNC_DIV16       ((uint32_t)(ADC_CCR_PRESC_2|ADC_CCR_PRESC_1|ADC_CCR_PRESC_0))
#define ADC_CLOCK_ASYNC_DIV32       ((uint32_t)(ADC_CCR_PRESC_3))
#define ADC_CLOCK_ASYNC_DIV64       ((uint32_t)(ADC_CCR_PRESC_3|ADC_CCR_PRESC_0))
#define ADC_CLOCK_ASYNC_DIV128      ((uint32_t)(ADC_CCR_PRESC_3|ADC_CCR_PRESC_1))
#define ADC_CLOCK_ASYNC_DIV256      ((uint32_t)(ADC_CCR_PRESC_3|ADC_CCR_PRESC_1|ADC_CCR_PRESC_0))
```

- Resolution

  用于 ADC 的分辨率配置，支持如下几种：

```
/** @defgroup ADC_Resolution ADC Resolution
  * @{
  */
#define ADC_RESOLUTION_16B      ((uint32_t)0x00000000)
#define ADC_RESOLUTION_14B      ((uint32_t)ADC_CFGR_RES_0)
#define ADC_RESOLUTION_12B      ((uint32_t)ADC_CFGR_RES_1)
#define ADC_RESOLUTION_10B      ((uint32_t)(ADC_CFGR_RES_1 | ADC_CFGR_RES_0))
#define ADC_RESOLUTION_8B       ((uint32_t)ADC_CFGR_RES_2)
```

- ScanConvMode

  用于使能或者禁止 ADC 的扫描模式，即多通道转换。此参数配合成员 DiscontinuousConvMode 可以将主转换序列分成多个子系列进行逐步转换。

  - 禁止扫描模式，表示单通道转换，成员 NbrOfConversion 和 InjectedNbrOfConversion 不起作用，等效为 1，即仅进行一个通道转换。
  - 使能扫描模式，表示支持多通道转换，成员 NbrOfConversion 和 InjectedNbrOfConversion 分别用于规则通道和注入通道转换序列数。

支持的参数如下：

```
#define ADC_SCAN_DISABLE          ((uint32_t)0x00000000)
#define ADC_SCAN_ENABLE           ((uint32_t)0x00000001)
```

- EOCSelection

  用于中断或者查询模式时，转换结束标志 EOC (End Of Conversion)的选择。参数可以单通道转换结束或者序列转换结束：

```
#define ADC_EOC_SINGLE_CONV         ((uint32_t) ADC_ISR_EOC)    /*!< End of unitary conversion flag  */
#define ADC_EOC_SEQ_CONV            ((uint32_t) ADC_ISR_EOS)    /*!< End of sequence conversions flag */
```

- LowPowerAutoWait

用于使能或者禁止低功耗自动延迟等待模式。仅当用户调用函数 HAL_ADC_GetValue()获取规则通道数据或者调用函数 HAL_ADCEx_InjectedGetValue()获取注入通道数据后才会开启下一次 ADC 转换。

- 此功能可自动将 ADC 触发频率调整为读取数据速度，即完全由用户决定的自适应模式。这样可以避免溢出，适用于低频应用。
- 此模式仅可用于 ADC 的查询模式，不可用于中断或者 DMA 模式，即调用函数 HAL_ADC_Start_IT()或者 HAL_ADC_Start_DMA()，因为这种模式会立即清除 EOC 转换结束标志，从而释放中断请求向量序列。
- 使用查询模式的调用方法：
  - 调用函数 HAL_ADC_Start 启动转换。
  - 调用函数 HAL_ADC_PollForConversion 等待转换结束，然后调用函数 HAL_ADC_GetValue 可以获取当前的转换值，并且会自动启动下次转换。
  - 如果是注入通道，就是调用函数 HAL_ADCExInjected_Start()启动转换，调用函数 HAL_ADCEx_InjectedGetValue()获取转换数据。

参数可以是使能 ENABLE 或者禁能 DISABLE。

- ContinuousConvMode

用于配置使用单次转换还是连续转换，此参数仅对规则通道有效。触发方式可以选择软件触发或者外部触发。

参数可以是使能 ENABLE，表示连续转换或者 DISABLE 禁能，表示单次转换。

- NbrOfConversion

用于配置规则通道要转换的通道数。

- 如果要进行多个通道的转换，参数成员 ScanConvMode 必须使能。
- 通道数范围 1 到 16。
- 仅当规则通道上没有后续的转换时才可以修改此参数(即 ADC 禁止的情况下或者 ADC 单次转换模式，又或者无外部触发）。

- DiscontinuousConvMode

用于配置 ADC 规则组转换序列的不连续方式。这里的不连续含义是指每次触发进行一个子组的转换。注意跟参数成员 ContinuousConvMode 的含义区分开。

- 使能此参数，要转换的整个序列通道 1，2，6，7，8，10，设置 2 个为一个子组，那么第 1 次触发会进行通道 1 和通道 2 的转换，下次触发进行通道 6 和通道 7 的转换，以此进行了。
- 禁止此参数，要转换的整个序列通道 1，2，6，7，8，10，那么第 1 次触发会进行所有通道的转换。

使用此参数要注意以下两点：

- 只有参数成员 ScanConvMode 使能的情况下，此参数才有使用的意义，否则忽略此参数。
- 只有参数成员 ContinuousConvMode 禁止的情况下，此参数才有使用的意义，否则忽略此参数。也就是说这两个参数不能同时使能。

参数可以是使能 ENABLE 或者禁止 DISABLE。

◆ NbrOfDiscConversion

使能了参数 DiscontinuousConvMode 的情况下，用于设置子组的大小。

- 仅用于规则通道。
- 如果参数 DiscontinuousConvMode 禁止的情况下，忽略此参数。
- 此参数范围 1 到 8。

◆ ExternalTrigConv

用于规则通道外部触发源的选择。如果使能了软件触发，那么外部触发将被关闭，使用软件触发。ADC1，ADC2 和 ADC3 支持的触发源是相同的。具体支持的触发源参数如下：

```
/** @defgroup ADC_regular_external_trigger_source ADC group regular trigger source
  * @{
  */
/* External triggers of regular group for ADC1, ADC2, ADC3 */
#define ADC_EXTERNALTRIG_T1_CC1           ((uint32_t)0x00000000)
#define ADC_EXTERNALTRIG_T1_CC2           ((uint32_t)ADC_CFGR_EXTSEL_0)
#define ADC_EXTERNALTRIG_T1_CC3           ((uint32_t)ADC_CFGR_EXTSEL_1)
#define ADC_EXTERNALTRIG_T2_CC2           ((uint32_t)(ADC_CFGR_EXTSEL_1 | ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_T3_TRGO          ((uint32_t)ADC_CFGR_EXTSEL_2)
#define ADC_EXTERNALTRIG_T4_CC4           ((uint32_t)(ADC_CFGR_EXTSEL_2 | ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_EXT_IT11         ((uint32_t)(ADC_CFGR_EXTSEL_2 | ADC_CFGR_EXTSEL_1))
#define ADC_EXTERNALTRIG_T8_TRGO          ((uint32_t)(ADC_CFGR_EXTSEL_2 | ADC_CFGR_EXTSEL_1 |
                                                      ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_T8_TRGO2         ((uint32_t) ADC_CFGR_EXTSEL_3)
#define ADC_EXTERNALTRIG_T1_TRGO          ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_T1_TRGO2         ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_1))
#define ADC_EXTERNALTRIG_T2_TRGO          ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_1 |
                                                      ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_T4_TRGO          ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_2))
#define ADC_EXTERNALTRIG_T6_TRGO          ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_2 |
                                                      ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_T15_TRGO         ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_2 |
                                                      ADC_CFGR_EXTSEL_1))
#define ADC_EXTERNALTRIG_T3_CC4           ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_2 |
                                                      ADC_CFGR_EXTSEL_1 | ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_HR1_ADCTRG1      ((uint32_t) ADC_CFGR_EXTSEL_4)
#define ADC_EXTERNALTRIG_HR1_ADCTRG3      ((uint32_t) (ADC_CFGR_EXTSEL_4 | ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_LPTIM1_OUT       ((uint32_t) (ADC_CFGR_EXTSEL_4 | ADC_CFGR_EXTSEL_1))
#define ADC_EXTERNALTRIG_LPTIM2_OUT       ((uint32_t) (ADC_CFGR_EXTSEL_4 | ADC_CFGR_EXTSEL_1|
                                                       ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_LPTIM3_OUT       ((uint32_t) (ADC_CFGR_EXTSEL_4 | ADC_CFGR_EXTSEL_2))
```

◆ ExternalTrigConvEdge

如果使用外部触发的话，设置触发沿类型，支持上升沿、下降沿或者双沿触发。

```
#define ADC_EXTERNALTRIGCONVEDGE_NONE           ((uint32_t)0x00000000)
#define ADC_EXTERNALTRIGCONVEDGE_RISING         ((uint32_t)ADC_CFGR_EXTEN_0)
#define ADC_EXTERNALTRIGCONVEDGE_FALLING        ((uint32_t)ADC_CFGR_EXTEN_1)
#define ADC_EXTERNALTRIGCONVEDGE_RISINGFALLING  ((uint32_t)ADC_CFGR_EXTEN)
```

注意，如果使能了软件触发，那么外部触发将被关闭，使用软件触发，此参数已经不起作用。

◆ ConversionDataManagement

此参数成员用于 ADC 采集数据的管理，可以存到 ADC 的 DR 寄存器，传输给 DFSDM，又或者通过 DMA 的单次或者循环模式传输数据到指定地址。

```
/** @defgroup ADC_ConversionDataManagement ADC Conversion Data Management
  * @{
  */
#define ADC_CONVERSIONDATA_DR    ((uint32_t)0x00000000)
#define ADC_CONVERSIONDATA_DFSDM                ((uint32_t)ADC_CFGR_DMNGT_1)
#define ADC_CONVERSIONDATA_DMA_ONESHOT          ((uint32_t)ADC_CFGR_DMNGT_0)
#define ADC_CONVERSIONDATA_DMA_CIRCULAR         ((uint32_t)(ADC_CFGR_DMNGT_0 | ADC_CFGR_DMNGT_1))
```

使用此参数成员注意以下问题：

- 如果参数成员 ContinuousConvMode 配置为连续转换模式，那么 DMA 必须配置为循环模式，否则会造成 DMA 传输溢出。
- 仅当规则通道上没有后续的转换时才可以修改此参数(即 ADC 禁止的情况下或者 ADC 单次转换模式，又或者无外部触发）。

◆ Overrun

用于配置 ADC 转换数据未及时读取，造成溢出时的处理，可以选择继续保留上次转换的数据，也可以选择新转换的数据覆盖，具体支持的参数如下：

```
#define ADC_OVR_DATA_PRESERVED              ((uint32_t)0x00000000)
#define ADC_OVR_DATA_OVERWRITTEN            ((uint32_t)ADC_CFGR_OVRMOD)
```

使用此参数成员注意以下问题：

- 此参数仅可用于规则通道。
- 如果此参数设置为保留上次转换的数据，且使用函数 HAL_ADC_Start_IT 的中断服务程序里面去的清除转换结束标志会导致保存的数据被释放。针对这种情况，用户可以在中断服务程序的回调函数 HAL_ADC_ConvCpltCallback 里面保存转换的数据（此回调函数是在清除转换结束标志前调用）。
- 如果ADC转换采用查询或者中断方式且此参数被设置为保留上次转换的数据,会产生错误报告。此参数设置为覆盖方式，那么用户不读取数据时，不会被视为溢出错误。
- 如果 ADC 转换采用 DMA 方式，不管此参数配置为何种方式，都会报告溢出错误（因为 DMA 是需要处理所有转换的数据）。

◆ LeftBitShift

用于设置 ADC 转换结果的左移位数，使用或者没有使用过采样的情况下，都可以使用此参数。

具体支持的参数如下：

```
#define ADC_LEFTBITSHIFT_NONE  ((uint32_t)0x00000000)
```

```
#define ADC_LEFTBITSHIFT_1     ((uint32_t)ADC_CFGR2_LSHIFT_0)
#define ADC_LEFTBITSHIFT_2     ((uint32_t)ADC_CFGR2_LSHIFT_1)
#define ADC_LEFTBITSHIFT_3     ((uint32_t)(ADC_CFGR2_LSHIFT_1 | ADC_CFGR2_LSHIFT_0))
#define ADC_LEFTBITSHIFT_4     ((uint32_t)ADC_CFGR2_LSHIFT_2)
#define ADC_LEFTBITSHIFT_5     ((uint32_t)(ADC_CFGR2_LSHIFT_2 | ADC_CFGR2_LSHIFT_0))
#define ADC_LEFTBITSHIFT_6     ((uint32_t)(ADC_CFGR2_LSHIFT_2 | ADC_CFGR2_LSHIFT_1))
#define ADC_LEFTBITSHIFT_7     ((uint32_t)(ADC_CFGR2_LSHIFT_2 | ADC_CFGR2_LSHIFT_1 | ADC_CFGR2_LSHIFT_0))
#define ADC_LEFTBITSHIFT_8     ((uint32_t)ADC_CFGR2_LSHIFT_3)
#define ADC_LEFTBITSHIFT_9     ((uint32_t)(ADC_CFGR2_LSHIFT_3 | ADC_CFGR2_LSHIFT_0))
#define ADC_LEFTBITSHIFT_10    ((uint32_t)(ADC_CFGR2_LSHIFT_3 | ADC_CFGR2_LSHIFT_1))
#define ADC_LEFTBITSHIFT_11    ((uint32_t)(ADC_CFGR2_LSHIFT_3 | ADC_CFGR2_LSHIFT_1 | ADC_CFGR2_LSHIFT_0))
#define ADC_LEFTBITSHIFT_12    ((uint32_t)(ADC_CFGR2_LSHIFT_3 | ADC_CFGR2_LSHIFT_2))
#define ADC_LEFTBITSHIFT_13    ((uint32_t)(ADC_CFGR2_LSHIFT_3 | ADC_CFGR2_LSHIFT_2 | ADC_CFGR2_LSHIFT_0))
#define ADC_LEFTBITSHIFT_14    ((uint32_t)(ADC_CFGR2_LSHIFT_3 | ADC_CFGR2_LSHIFT_2 | ADC_CFGR2_LSHIFT_1))
#define ADC_LEFTBITSHIFT_15    ((uint32_t)(ADC_CFGR2_LSHIFT_3 | ADC_CFGR2_LSHIFT_2 | ADC_CFGR2_LSHIFT_1 |
                                ADC_CFGR2_LSHIFT_0))
```

- BoostMode

  用于设置 ADC 的 BOOST 模式，当 ADC 的时钟高于 20MHz 时，必须使能此位。

  参数可以是使能 ENABLE 或者禁止 DISABLE。

- OversamplingMode

  此参数成员用于使能或者禁止过采样模式。只有当 ADC 规则通道或者注入通道没有数据转换时才可以修改此参数。

  参数可以是使能 ENABLE 或者禁止 DISABLE。

- Oversampling

  此参数是 ADC_OversamplingTypeDef 类型结构体变量，用于设置过采样的相关参数。

44.3.4ADC 通道配置结构体 ADC_ChannelConfTypeDef

结构体变量 ADC_ChannelConfTypeDef 用于配置 ADC 规则通道的一些特性，定义如下：

```
typedef struct
{
  uint32_t Channel;
  uint32_t Rank;
  uint32_t SamplingTime;
  uint32_t SingleDiff;
  uint32_t OffsetNumber;
  uint32_t Offset;
  FunctionalState OffsetRightShift;
  FunctionalState OffsetSignedSaturation;
}ADC_ChannelConfTypeDef;
```

下面将这几个参数逐一做说明：

- Channel

  具体支持的通道参数如下：

```
/** @defgroup ADC_channels ADC Channels
  * @{
  */
#define ADC_CHANNEL_0           ((uint32_t)(0x00000000))
```

```
#define ADC_CHANNEL_1           ((uint32_t)(ADC_SQR3_SQ10_0))
#define ADC_CHANNEL_2           ((uint32_t)(ADC_SQR3_SQ10_1))
#define ADC_CHANNEL_3           ((uint32_t)(ADC_SQR3_SQ10_1 | ADC_SQR3_SQ10_0))
#define ADC_CHANNEL_4           ((uint32_t)(ADC_SQR3_SQ10_2))
#define ADC_CHANNEL_5           ((uint32_t)(ADC_SQR3_SQ10_2 | ADC_SQR3_SQ10_0))
#define ADC_CHANNEL_6           ((uint32_t)(ADC_SQR3_SQ10_2 | ADC_SQR3_SQ10_1))
#define ADC_CHANNEL_7           ((uint32_t)(ADC_SQR3_SQ10_2 | ADC_SQR3_SQ10_1 | ADC_SQR3_SQ10_0))
#define ADC_CHANNEL_8           ((uint32_t)(ADC_SQR3_SQ10_3))
#define ADC_CHANNEL_9           ((uint32_t)(ADC_SQR3_SQ10_3 | ADC_SQR3_SQ10_0))
#define ADC_CHANNEL_10          ((uint32_t)(ADC_SQR3_SQ10_3 | ADC_SQR3_SQ10_1))
#define ADC_CHANNEL_11          ((uint32_t)(ADC_SQR3_SQ10_3 | ADC_SQR3_SQ10_1 | ADC_SQR3_SQ10_0))
#define ADC_CHANNEL_12          ((uint32_t)(ADC_SQR3_SQ10_3 | ADC_SQR3_SQ10_2))
#define ADC_CHANNEL_13          ((uint32_t)(ADC_SQR3_SQ10_3 | ADC_SQR3_SQ10_2 | ADC_SQR3_SQ10_0))
#define ADC_CHANNEL_14          ((uint32_t)(ADC_SQR3_SQ10_3 | ADC_SQR3_SQ10_2 | ADC_SQR3_SQ10_1))
#define ADC_CHANNEL_15          ((uint32_t)(ADC_SQR3_SQ10_3 | ADC_SQR3_SQ10_2 | ADC_SQR3_SQ10_1 |
ADC_SQR3_SQ10_0))
#define ADC_CHANNEL_16          ((uint32_t)(ADC_SQR3_SQ10_4))
#define ADC_CHANNEL_17          ((uint32_t)(ADC_SQR3_SQ10_4 | ADC_SQR3_SQ10_0))
#define ADC_CHANNEL_18          ((uint32_t)(ADC_SQR3_SQ10_4 | ADC_SQR3_SQ10_1))
#define ADC_CHANNEL_19          ((uint32_t)(ADC_SQR3_SQ10_4 | ADC_SQR3_SQ10_1| ADC_SQR3_SQ10_0))

/* Note: Vbat/4, TempSensor and VREFINT internal channels are available on ADC3 only */
#define ADC_CHANNEL_VBAT_DIV4    ADC_CHANNEL_17
#define ADC_CHANNEL_TEMPSENSOR   ADC_CHANNEL_18
#define ADC_CHANNEL_VREFINT      ADC_CHANNEL_19

/* Note: DAC1CH1 and DAC1CH2 internal channels is available on ADC2 only  */
/*!< ADC internal channel connected to DAC1 channel 1, channel specific to ADC2 */
#define ADC_CHANNEL_DAC1CH1_ADC2 (ADC_CHANNEL_16)

/*!< ADC internal channel connected to DAC1 channel 2, channel specific to ADC2 */
#define ADC_CHANNEL_DAC1CH2_ADC2 (ADC_CHANNEL_17)
```

这里要特别注意：

- VBAT、温度传感器和 VREFINT 仅 ADC3 可用。
- DAC1 的两个通道在内部连接至 ADC2，也就是说仅 ADC2 可用。

◆ Rank

用于配置规则通道的转换顺序，如果想禁止一个通道或者改变一个通道的顺序，可以使用新配置覆盖。

具体支持的参数如下：

```
/** @defgroup ADC_regular_rank ADC group regular sequencer rank
  * @{
  */
#define ADC_REGULAR_RANK_1    ((uint32_t)0x00000001)       /*!< ADC regular conversion rank 1  */
#define ADC_REGULAR_RANK_2    ((uint32_t)0x00000002)       /*!< ADC regular conversion rank 2  */
#define ADC_REGULAR_RANK_3    ((uint32_t)0x00000003)       /*!< ADC regular conversion rank 3  */
#define ADC_REGULAR_RANK_4    ((uint32_t)0x00000004)       /*!< ADC regular conversion rank 4  */
#define ADC_REGULAR_RANK_5    ((uint32_t)0x00000005)       /*!< ADC regular conversion rank 5  */
#define ADC_REGULAR_RANK_6    ((uint32_t)0x00000006)       /*!< ADC regular conversion rank 6  */
#define ADC_REGULAR_RANK_7    ((uint32_t)0x00000007)       /*!< ADC regular conversion rank 7  */
#define ADC_REGULAR_RANK_8    ((uint32_t)0x00000008)       /*!< ADC regular conversion rank 8  */
#define ADC_REGULAR_RANK_9    ((uint32_t)0x00000009)       /*!< ADC regular conversion rank 9  */
#define ADC_REGULAR_RANK_10   ((uint32_t)0x0000000A)       /*!< ADC regular conversion rank 10 */
#define ADC_REGULAR_RANK_11   ((uint32_t)0x0000000B)       /*!< ADC regular conversion rank 11 */
#define ADC_REGULAR_RANK_12   ((uint32_t)0x0000000C)       /*!< ADC regular conversion rank 12 */
#define ADC_REGULAR_RANK_13   ((uint32_t)0x0000000D)       /*!< ADC regular conversion rank 13 */
```

```
#define ADC_REGULAR_RANK_14    ((uint32_t)0x0000000E)       /*!< ADC regular conversion rank 14 */
#define ADC_REGULAR_RANK_15    ((uint32_t)0x0000000F)       /*!< ADC regular conversion rank 15 */
#define ADC_REGULAR_RANK_16    ((uint32_t)0x00000010)       /*!< ADC regular conversion rank 16 */
```

◆ SamplingTime

用于所选通道的采样时间配置，ADC 的采样速度是由采样时间和转换时间同决定。用于内部通道测量时（VrefInt/Vbat/TempSensor），务必要遵循数据手册要求的参数范围。此参数成员具体支持的定义如下：

```
/** @defgroup ADC_sampling_times ADC Sampling Times
  * @{
  */
#define ADC_SAMPLETIME_1CYCLE_5       ((uint32_t)0x00000000)
#define ADC_SAMPLETIME_2CYCLES_5      ((uint32_t)ADC_SMPR2_SMP10_0)
#define ADC_SAMPLETIME_8CYCLES_5      ((uint32_t)ADC_SMPR2_SMP10_1)
#define ADC_SAMPLETIME_16CYCLES_5     ((uint32_t)(ADC_SMPR2_SMP10_1 | ADC_SMPR2_SMP10_0))
#define ADC_SAMPLETIME_32CYCLES_5     ((uint32_t)ADC_SMPR2_SMP10_2)
#define ADC_SAMPLETIME_64CYCLES_5     ((uint32_t)(ADC_SMPR2_SMP10_2 | ADC_SMPR2_SMP10_0))
#define ADC_SAMPLETIME_387CYCLES_5    ((uint32_t)(ADC_SMPR2_SMP10_2 | ADC_SMPR2_SMP10_1))
#define ADC_SAMPLETIME_810CYCLES_5    ((uint32_t)ADC_SMPR2_SMP10)
```

◆ SingleDiff

此参数成员用于选择单端输入还是差分输入。差分输入要用到输入通道 i（正向输入）和 i+1 反向输入，用户仅需配置通道 i 即可，通道 i+1 会被自动配置。具体支持的定义如下：

```
/** @defgroup ADCEx_SingleDifferential ADC Extended Single-ended/Differential input mode
  * @{
  */
#define ADC_SINGLE_ENDED               ((uint32_t)0x00000000)
#define ADC_DIFFERENTIAL_ENDED         ((uint32_t)ADC_CR_ADCALDIF)
```

使用此参数要注意以下问题：

- 如果用于差分模式，要注意选择的通道是正确的。
- 如果配置了通道 i 为差分模式，不可再配置通道 i+1。
- ADC 禁止期间才可以修改此参数，ADC 使能后修改此参数会被忽略，也不会报错。

◆ OffsetNumber

此参数成员用于选择偏移序号 Offset Number，具体支持的参数定义如下：

```
/** @defgroup ADCEx_OffsetNumber ADC Extended Offset Number
  * @{
  */
#define ADC_OFFSET_NONE   ((uint32_t)0x00)     /*!< No offset correction                          */
#define ADC_OFFSET_1      ((uint32_t)0x01)     /*!< Offset correction to apply to a first channel  */
#define ADC_OFFSET_2      ((uint32_t)0x02)     /*!< Offset correction to apply to a second channel */
#define ADC_OFFSET_3      ((uint32_t)0x03)     /*!< Offset correction to apply to a third channel  */
#define ADC_OFFSET_4      ((uint32_t)0x04)     /*!< Offset correction to apply to a fourth channel */
```

注意，每个通道仅支持一个偏移设置。

◆ Offset

定义要从原始数据中减去的偏移量。

- 偏移量必须是正数。
- 根据用户配置的 ADC 分辨率为 16bit,14bit,12bit,10bit 或者 8bit,偏移量的最小值为 0x0000,

最大值分别为 0xFFFF，0x3FFF, 0xFFF, 0x3FF 和 0xFF。

- 仅当规则通道或者注入通道上没有后续的转换时才可以修改此参数（即 ADC 禁止的情况下或者 ADC 单次转换模式，又或者无外部触发）。

◆ OffsetRightShift

此参数成员用于选择是否使能偏移校准后数据右移，仅适用于 16bit 或者 8bit 分辨率。

支持的参数可以是使能 ENABLE 或者禁止 DISABLE。

◆ OffsetSignedSaturation

此参数成员用于选择是否使能有符号饱和特性，仅适用于 16bit 或者 8bit 分辨率。

支持的参数可以是使能 ENABLE 或者禁止 DISABLE。

44.3.5 ADC 过采样结构体 ADC_OversamplingTypeDef

结构体 ADC_OversamplingTypeDef 主要用于过采样方面参数配置，定义如下：

```
typedef struct
{
  uint32_t Ratio;
  uint32_t RightBitShift;
  uint32_t TriggeredMode;
  uint32_t OversamplingStopReset;
}ADC_OversamplingTypeDef;
```

下面将这几个参数逐一做说明：

◆ Ratio

此参数成员用于配置过采样率。

◆ RightBitShift

此参数用于设置右移，即分频因数。具体支持的参数如下：

```
/** @defgroup ADCEx_Right_Bit_Shift   ADC Extended Oversampling Right Shift
  * @{
  */
#define ADC_RIGHTBITSHIFT_NONE  ((uint32_t)0x00000000)
#define ADC_RIGHTBITSHIFT_1     ((uint32_t)ADC_CFGR2_OVSS_0)
#define ADC_RIGHTBITSHIFT_2     ((uint32_t)ADC_CFGR2_OVSS_1)
#define ADC_RIGHTBITSHIFT_3     ((uint32_t)(ADC_CFGR2_OVSS_1 | ADC_CFGR2_OVSS_0))
#define ADC_RIGHTBITSHIFT_4     ((uint32_t)ADC_CFGR2_OVSS_2)
#define ADC_RIGHTBITSHIFT_5     ((uint32_t)(ADC_CFGR2_OVSS_2 | ADC_CFGR2_OVSS_0))
#define ADC_RIGHTBITSHIFT_6     ((uint32_t)(ADC_CFGR2_OVSS_2 | ADC_CFGR2_OVSS_1))
#define ADC_RIGHTBITSHIFT_7     ((uint32_t)(ADC_CFGR2_OVSS_2 | ADC_CFGR2_OVSS_1 | ADC_CFGR2_OVSS_0))
#define ADC_RIGHTBITSHIFT_8     ((uint32_t)ADC_CFGR2_OVSS_3)
#define ADC_RIGHTBITSHIFT_9     ((uint32_t)(ADC_CFGR2_OVSS_3 | ADC_CFGR2_OVSS_0))
#define ADC_RIGHTBITSHIFT_10    ((uint32_t)(ADC_CFGR2_OVSS_3 | ADC_CFGR2_OVSS_1))
#define ADC_RIGHTBITSHIFT_11    ((uint32_t)(ADC_CFGR2_OVSS_3 | ADC_CFGR2_OVSS_1 | ADC_CFGR2_OVSS_0))
```

◆ TriggeredMode

此参数成员用于过采样的触发模式配置，具体支持的定义如下：

```
/** @defgroup ADCEx_Triggered_Oversampling_Mode   ADC Extended Triggered Regular Oversampling
  * @{
```

```
  */
#define ADC_TRIGGEREDMODE_SINGLE_TRIGGER      ((uint32_t)0x00000000)
#define ADC_TRIGGEREDMODE_MULTI_TRIGGER       ((uint32_t)ADC_CFGR2_TROVS)
```

- OversamplingStopReset

此参数成员用于配置使用注入通道时，过采样的处理。可以选择保持原有过采样缓冲数据，或者缓冲数据清零。具体支持的定义如下：

```
/** @defgroup ADCEx_Regular_Oversampling_Mode   ADC Extended Regular Oversampling Continued or Resumed Mode
  * @{
  */
/*!<  Oversampling buffer maintained during injection sequence */
#define ADC_REGOVERSAMPLING_CONTINUED_MODE    ((uint32_t)0x00000000)

/*!<  Oversampling buffer zeroed during injection sequence     */
#define ADC_REGOVERSAMPLING_RESUMED_MODE      ((uint32_t)ADC_CFGR2_ROVSM)
```

注意，如果规则通道和注入通道同时使用过采样，此参数成员的配置将被忽略，强制设置为 ADC_REGOVERSAMPLING_RESUMED_MODE。

44.3.6 ADC 模拟看门狗结构体 ADC_AnalogWDGConfTypeDef

结构体 ADC_AnalogWDGConfTypeDef 主要用于模拟看门狗参数配置，ADC1，ADC2 和 ADC3 都有三个模拟看门狗，结构体定义如下：

```
typedef struct
{
  uint32_t WatchdogNumber;
  uint32_t WatchdogMode;
  uint32_t Channel;
  FunctionalState ITMode;
  uint32_t HighThreshold;
  uint32_t LowThreshold;
}ADC_AnalogWDGConfTypeDef;
```

下面将这几个参数逐一做说明：

- WatchdogNumber

此参数成员用于配置选择那个看门狗监测通道。

- 模拟看门狗 1 仅可以监测 1 个通道，或者通过成员 WatchdogMode 配置选择监测所有通道。
- 模拟看门狗 2 和 3 可以检测任意指定的通道，调用函数 HAL_ADC_AnalogWDGConfig 配置即可，要监测那个通道，调用一次这个函数。

具体支持的定义如下：

```
/** @defgroup ADCEx_analog_watchdog_number ADC Extended Analog Watchdog Selection
  * @{
  */
#define ADC_ANALOGWATCHDOG_1      ((uint32_t)0x00000001)
#define ADC_ANALOGWATCHDOG_2      ((uint32_t)0x00000002)
#define ADC_ANALOGWATCHDOG_3      ((uint32_t)0x00000003)
```

- WatchdogMode

此参数用于设置模拟看门狗模式。

- 模拟看门狗 1 可以配置监测单个通道或者所有通道，适用于规则通道和注入通道。
- 模拟看门狗 2 和 3 不支持监测所有通道，但可以监测指定的多个通道，每调用一次函数 HAL_ADC_AnalogWDGConfig 可以指定一个通道，如果此参数配置为 ADC_ANALOGWATCHDOG_NONE，那么参数 Channel 配置的通道将被复位。

此参数具体支持的定义如下：

```
/** @defgroup ADCEx_analog_watchdog_mode ADC Extended Analog Watchdog Mode
  * @{
  */
#define ADC_ANALOGWATCHDOG_NONE                 ((uint32_t) 0x00000000)
#define ADC_ANALOGWATCHDOG_SINGLE_REG           ((uint32_t)(ADC_CFGR_AWD1SGL | ADC_CFGR_AWD1EN))
#define ADC_ANALOGWATCHDOG_SINGLE_INJEC         ((uint32_t)(ADC_CFGR_AWD1SGL | ADC_CFGR_JAWD1EN))
#define ADC_ANALOGWATCHDOG_SINGLE_REGINJEC      ((uint32_t)(ADC_CFGR_AWD1SGL | ADC_CFGR_AWD1EN |
                                                            ADC_CFGR_JAWD1EN))
#define ADC_ANALOGWATCHDOG_ALL_REG              ((uint32_t) ADC_CFGR_AWD1EN)
#define ADC_ANALOGWATCHDOG_ALL_INJEC            ((uint32_t) ADC_CFGR_JAWD1EN)
#define ADC_ANALOGWATCHDOG_ALL_REGINJEC         ((uint32_t)(ADC_CFGR_AWD1EN | ADC_CFGR_JAWD1EN))
```

◆ Channel

用于配置要监测的通道。

- 对于模拟看门狗 1，参数成员 WatchdogMode 配置为单个通道时，此参数才有意义。
- 对于模拟看门狗 2 和 3，每调用一次函数 HAL_ADC_AnalogWDGConfig 可以指定一个通道，如果要复位那个通道，将参数 WatchdogMode 配置为 ADC_ANALOGWATCHDOG_NONE 即可。

◆ ITMode

用于配置模拟看门狗为中断方式或者查询方式。

配置为 ENABEL 表示使用中断方式，配置为 DISABLE 表示查询方式。

◆ HighThreshold

用于配置模拟看门狗高阀值。根据配置的 ADC 的分辨率 16, 14, 12, 10 或者 8bit，高阀值最小都是 0x0000，最大值分别是 0xFFFF, 0x3FFF, 0xFFF, 0x3FF 和 0xFF。

◆ LowThreshold

用于配置模拟看门狗低阀值。根据配置的 ADC 的分辨率 16, 14, 12, 10 或者 8bit，高阀值最小都是 0x0000，最大值分别是 0xFFFF, 0x3FFF, 0xFFF, 0x3FF 和 0xFF。

44.3.7 ADC 的状态标志清除问题

下面我们介绍__HAL_ADC_GET_FLAG 函数。这个函数用来检查 ADC 标志位是否被设置。

```
/**
  * @brief Checks whether the specified ADC flag is set or not.
  * @param __HANDLE__: ADC handle
  * @param __FLAG__: ADC flag to check
  *        This parameter can be one of the following values:
  *            @arg ADC_FLAG_RDY     ADC Ready (ADRDY) flag
  *            @arg ADC_FLAG_EOSMP   ADC End of Sampling flag
```

```
  *            @arg ADC_FLAG_EOC     ADC End of Regular Conversion flag
  *            @arg ADC_FLAG_EOS     ADC End of Regular sequence of Conversions flag
  *            @arg ADC_FLAG_OVR     ADC overrun flag
  *            @arg ADC_FLAG_JEOC    ADC End of Injected Conversion flag
  *            @arg ADC_FLAG_JEOS    ADC End of Injected sequence of Conversions flag
  *            @arg ADC_FLAG_AWD1    ADC Analog watchdog 1 flag (main analog watchdog)
  *            @arg ADC_FLAG_AWD2    ADC Analog watchdog 2 flag (additional analog watchdog)
  *            @arg ADC_FLAG_AWD3    ADC Analog watchdog 3 flag (additional analog watchdog)
  *            @arg ADC_FLAG_JQOVF   ADC Injected Context Queue Overflow flag
  * @retval The new state of __FLAG__ (TRUE or FALSE).
  */
#define __HAL_ADC_GET_FLAG(__HANDLE__, __FLAG__) ((((__HANDLE__)->Instance->ISR) & (__FLAG__)) == (__FLAG__))
```

与标志获取函数__HAL_ADC_GET_FLAG 对应的清除函数是__HAL_ADC_CLEAR_FLAG:

```
/**
  * @brief Clear a specified ADC flag
  * @param __HANDLE__: ADC handle
  * @param __FLAG__: ADC flag to clear
  *        This parameter can be one of the following values:
  *            @arg ADC_FLAG_RDY     ADC Ready (ADRDY) flag
  *            @arg ADC_FLAG_EOSMP   ADC End of Sampling flag
  *            @arg ADC_FLAG_EOC     ADC End of Regular Conversion flag
  *            @arg ADC_FLAG_EOS     ADC End of Regular sequence of Conversions flag
  *            @arg ADC_FLAG_OVR     ADC overrun flag
  *            @arg ADC_FLAG_JEOC    ADC End of Injected Conversion flag
  *            @arg ADC_FLAG_JEOS    ADC End of Injected sequence of Conversions flag
  *            @arg ADC_FLAG_AWD1    ADC Analog watchdog 1 flag (main analog watchdog)
  *            @arg ADC_FLAG_AWD2    ADC Analog watchdog 2 flag (additional analog watchdog)
  *            @arg ADC_FLAG_AWD3    ADC Analog watchdog 3 flag (additional analog watchdog)
  *            @arg ADC_FLAG_JQOVF   ADC Injected Context Queue Overflow flag
  * @note: bit cleared bit by writing 1 (writing 0 has no effect on any bit of register ISR)
  * @retval None
  */
#define __HAL_ADC_CLEAR_FLAG(__HANDLE__, __FLAG__) (((__HANDLE__)->Instance->ISR) = (__FLAG__))
```

清除标志函数所支持的参数跟获取函数是一 一对应的。除了这两个函数，还有 ADC 的中断开启和中断关闭函数，有时候也要用到。

```
/**
  * @brief Enable an ADC interrupt.
  * @param __HANDLE__: ADC handle
  * @param __INTERRUPT__: ADC Interrupt to enable
   *          This parameter can be one of the following values:
  *            @arg ADC_IT_RDY    ADC Ready (ADRDY) interrupt source
  *            @arg ADC_IT_EOSMP  ADC End of Sampling interrupt source
  *            @arg ADC_IT_EOC    ADC End of Regular Conversion interrupt source
  *            @arg ADC_IT_EOS    ADC End of Regular sequence of Conversions interrupt source
  *            @arg ADC_IT_OVR    ADC overrun interrupt source
  *            @arg ADC_IT_JEOC   ADC End of Injected Conversion interrupt source
  *            @arg ADC_IT_JEOS   ADC End of Injected sequence of Conversions interrupt source
  *            @arg ADC_IT_AWD1   ADC Analog watchdog 1 interrupt source (main analog watchdog)
  *            @arg ADC_IT_AWD2   ADC Analog watchdog 2 interrupt source (additional analog watchdog)
  *            @arg ADC_IT_AWD3   ADC Analog watchdog 3 interrupt source (additional analog watchdog)
  *            @arg ADC_IT_JQOVF  ADC Injected Context Queue Overflow interrupt source
  * @retval None
  */
#define __HAL_ADC_ENABLE_IT(__HANDLE__, __INTERRUPT__) (((__HANDLE__)->Instance->IER) |= (__INTERRUPT__))

/**
```

```
  * @brief Disable an ADC interrupt.
  * @param __HANDLE__: ADC handle
  * @param __INTERRUPT__: ADC Interrupt to disable
  *            @arg ADC_IT_RDY    ADC Ready (ADRDY) interrupt source
  *            @arg ADC_IT_EOSMP  ADC End of Sampling interrupt source
  *            @arg ADC_IT_EOC    ADC End of Regular Conversion interrupt source
  *            @arg ADC_IT_EOS    ADC End of Regular sequence of Conversions interrupt source
  *            @arg ADC_IT_OVR    ADC overrun interrupt source
  *            @arg ADC_IT_JEOC   ADC End of Injected Conversion interrupt source
  *            @arg ADC_IT_JEOS   ADC End of Injected sequence of Conversions interrupt source
  *            @arg ADC_IT_AWD1   ADC Analog watchdog 1 interrupt source (main analog watchdog)
  *            @arg ADC_IT_AWD2   ADC Analog watchdog 2 interrupt source (additional analog watchdog)
  *            @arg ADC_IT_AWD3   ADC Analog watchdog 3 interrupt source (additional analog watchdog)
  *            @arg ADC_IT_JQOVF  ADC Injected Context Queue Overflow interrupt source
  * @retval None
  */
#define __HAL_ADC_DISABLE_IT(__HANDLE__, __INTERRUPT__) (((__HANDLE__)->Instance->IER) &= ~(__INTERRUPT__))
```

注意：操作 ADC 的寄存器不限制必须要用 HAL 库提供的 API，比如要操作 ADC1 的寄存器 IER，直接调用 DMA1->IER 操作即可。

44.3.8 ADC 初始化流程总结

使用方法由 HAL 库提供：

◆ 第 1 步：ADC 时钟源选择。

两种时钟源可供选择，可以选择同步时钟，来自 AHB；也可以选择异步时钟，来自系统时钟，PLL2 或者 PLL3 的时钟。

比如使用 PLL2:

```
RCC_PeriphCLKInitTypeDef     RCC_PeriphCLKInit;
PeriphCLKInit.PeriphClockSelection = RCC_PERIPHCLK_ADC;
PeriphCLKInit.AdcClockSelection      = RCC_ADCCLKSOURCE_PLL2;
HAL_RCCEx_PeriphCLKConfig(&PeriphCLKInit);
```

◆ 第 2 步：ADC 输入引脚配置。

- 调用函数__HAL_RCC_GPIOx_CLK_ENABLE 使能时钟。
- 调用函数 HAL_GPIO_Init 配置引脚工作在模拟模式。

◆ 第 3 步：ADC 中断配置（如果用到的话）。

- 调用函数 HAL_NVIC_EnableIRQ(ADCx_IRQn)使能 ADC 中断。
- 将 ADC 中断处理函数 HAL_ADC_IRQHandler()填到 ADC 中断服务程序 ADCx_IRQHandler()中。

◆ 第 4 步：ADC 使用 DMA 方式配置（如果用到的话）

- 调用函数 HAL_DMA_Init 配置 DMA 的相关参数。
- 使用函数 HAL_NVIC_SetPriority 配置 DMA 优先级。
- 使用函数 HAL_NVIC_EnableIRQ 使能 DMA 中断。
- 将函数 HAL_DMA_IRQHandler()填到中断服务程序 HAL_DMA_IRQHandler()里面。
- 传输结束后会调用函数 HAL_DMA_IRQHandler()，此函数里面会执行回调函数 HAL_ADC_ConvCpltCallback，HAL_ADC_ConvHalfCpltCallback 等。

◆ 第 5 步：配置 ADC，ADC 通道和模拟看门狗

- 调用函数 HAL_ADC_Init 初始化 ADC。
- 调用函数 HAL_ADC_ConfigChannel 配置 ADC 通道各个参数。
- 调用函数 HAL_ADC_AnalogWDGConfig 配置模拟看门狗。

◆ 第 6 步：调用函数 HAL_ADCEx_Calibration_Start 做自动校准。

◆ 第 7 步：ADC 数值获取的三种方式。

- 查询方式

HAL_ADC_Start()
HAL_ADC_PollForConversion()
HAL_ADC_GetValue()
HAL_ADC_Stop()

- 中断方式

HAL_ADC_Start_IT()
HAL_ADC_ConvCpltCallback()
HAL_ADC_GetValue()
HAL_ADC_Stop_IT()

- DMA 方式

HAL_ADC_Start_DMA()
HAL_ADC_ConvCpltCallback() 和 HAL_ADC_ConvHalfCpltCallback()
HAL_ADC_Stop_DMA()

44.4 源文件 stm32h7xx_hal_adc.c

此文件涉及到的函数比较多，这里把我们几个常用的函数做个说明：

- HAL_ADC_Init
- HAL_ADC_ConfigChannel
- HAL_ADC_Start
- HAL_ADC_Start_DMA
- HAL_ADCEx_Calibration_Start

44.4.1 函数 HAL_ADC_Init

函数原型：

```
HAL_StatusTypeDef HAL_ADC_Init(ADC_HandleTypeDef* hadc)
{
  HAL_StatusTypeDef tmp_hal_status = HAL_OK;

  ADC_Common_TypeDef *tmpADC_Common;
  uint32_t tmpCFGR = 0;
  __IO uint32_t wait_loop_index = 0;

  /* 程序中不重要的部分被清除掉，仅留下关键部分做注释 */

  /* 检测 ADC 句柄 */
  if(hadc == NULL)
  {
    return HAL_ERROR;
  }

  /* 复位状态初始化 */
  if (hadc->State == HAL_ADC_STATE_RESET)
  {
    HAL_ADC_MspInit(hadc);
    ADC_CLEAR_ERRORCODE(hadc);
    hadc->Lock = HAL_UNLOCKED;
  }

  /*  ADC 退出深度掉电模式 */
  if (HAL_IS_BIT_SET(hadc->Instance->CR, ADC_CR_DEEPPWD))
  {
    /* 退出深度掉电模式 Exit deep power down mode */
    CLEAR_BIT(hadc->Instance->CR, ADC_CR_DEEPPWD);

    /* 退出深度掉电模式，一旦 ADC 稳压器使能，必须重新校准或者应用之前保存的校准值 */
  }


  if  (HAL_IS_BIT_CLR(hadc->Instance->CR, ADC_CR_ADVREGEN))
  {
    /* 使能 ADC 内部稳压器 */
    SET_BIT(hadc->Instance->CR, ADC_CR_ADVREGEN);
```

```
  /* 等待 ADC 稳定 */
  wait_loop_index = (ADC_STAB_DELAY_US * (SystemCoreClock / (1000000 * 2)));
  while(wait_loop_index != 0)
  {
    wait_loop_index--;
  }
}


/* 检测 ADC 稳压器是否使能，潜在的时钟稳定会导致使失败  */
if (HAL_IS_BIT_CLR(hadc->Instance->CR, ADC_CR_ADVREGEN))
{
  /* 更新 ADC 状态 */
  SET_BIT(hadc->State, HAL_ADC_STATE_ERROR_INTERNAL);

  /* 设置内部错误 */
  SET_BIT(hadc->ErrorCode, HAL_ADC_ERROR_INTERNAL);

  tmp_hal_status = HAL_ERROR;
}


/* 如果 ADC 之前的配置成功且没有继续进行规则通道的 */
if (HAL_IS_BIT_CLR(hadc->State, HAL_ADC_STATE_ERROR_INTERNAL) &&
    (ADC_IS_CONVERSION_ONGOING_REGULAR(hadc) == RESET)  )
{

  /* Initialize the ADC state */
  SET_BIT(hadc->State, HAL_ADC_STATE_BUSY_INTERNAL);

  /* 配置 ADC 的公共参数 */
  if((hadc->Instance == ADC1) || (hadc->Instance == ADC2))
  {
    tmpADC_Common = ADC12_COMMON_REGISTER(hadc);
  }
  else
  {
    tmpADC_Common = ADC3_COMMON_REGISTER(hadc);
  }

  if ((ADC_IS_ENABLE(hadc) == RESET)   &&
      (ADC_ANY_OTHER_ENABLED(hadc) == RESET) )
  {
    /* 配置 CCR 寄存器 */
    MODIFY_REG(tmpADC_Common->CCR, ADC_CCR_PRESC|ADC_CCR_CKMODE, hadc->Init.ClockPrescaler);
  }


  /* 配置 ADC 参数 */
  tmpCFGR  = ( ADC_CFGR_CONTINUOUS(hadc->Init.ContinuousConvMode)          |
               hadc->Init.Overrun                                          |
               hadc->Init.Resolution                                       |
              ADC_CFGR_REG_DISCONTINUOUS(hadc->Init.DiscontinuousConvMode)  );

  if (hadc->Init.DiscontinuousConvMode == ENABLE)
  {
    tmpCFGR |= ADC_CFGR_DISCONTINUOUS_NUM(hadc->Init.NbrOfDiscConversion);
```

```
  }

  /* 注意，如果参数 ExternalTrigConvEdge 设置为 trigger edge none 等效于软件启动 */
  if ((hadc->Init.ExternalTrigConv != ADC_SOFTWARE_START)
      && (hadc->Init.ExternalTrigConvEdge != ADC_EXTERNALTRIGCONVEDGE_NONE))
  {
    tmpCFGR |= ( hadc->Init.ExternalTrigConv |  hadc->Init.ExternalTrigConvEdge);
  }

  MODIFY_REG(hadc->Instance->CFGR, ADC_CFGR_FIELDS_1, tmpCFGR);


  /* 更新 ADC 参数 */
  if (ADC_IS_CONVERSION_ONGOING_REGULAR_INJECTED(hadc) == RESET)
  {
    tmpCFGR = ( ADC_CFGR_AUTOWAIT(hadc->Init.LowPowerAutoWait)       |
                ADC_CFGR_DMACONTREQ(hadc->Init.ConversionDataManagement) );

    MODIFY_REG(hadc->Instance->CFGR, ADC_CFGR_FIELDS_2, tmpCFGR);

    if (hadc->Init.OversamplingMode == ENABLE)
    {
      if ((hadc->Init.ExternalTrigConv == ADC_SOFTWARE_START)
          || (hadc->Init.ExternalTrigConvEdge == ADC_EXTERNALTRIGCONVEDGE_NONE))
      {
        /* 软件启动不能用于多触发，只能单触发 */
        assert_param((hadc->Init.Oversampling.TriggeredMode == ADC_TRIGGEREDMODE_SINGLE_TRIGGER));
      }


     /* 配置过采样 */
     MODIFY_REG(hadc->Instance->CFGR2, ADC_CFGR2_FIELDS,
                                       ADC_CFGR2_ROVSE                          |
                                       (hadc->Init.Oversampling.Ratio << 16) |
                                       hadc->Init.Oversampling.RightBitShift |
                                       hadc->Init.Oversampling.TriggeredMode |
                                       hadc->Init.Oversampling.OversamplingStopReset);
    }
    else
    {
      /* 禁止规则通道过采样 */
      CLEAR_BIT( hadc->Instance->CFGR2, ADC_CFGR2_ROVSE);
    }

    /* 设置左移参数 */
    MODIFY_REG(hadc->Instance->CFGR2, ADC_CFGR2_LSHIFT, hadc->Init.LeftBitShift);

    /* 是否使能 BOOST 模式 */
    if(hadc->Init.BoostMode == ENABLE)
    {
      SET_BIT(hadc->Instance->CR, ADC_CR_BOOST);
    }
    else
    {
      CLEAR_BIT(hadc->Instance->CR, ADC_CR_BOOST);
    }

  }
```

```
    /* 配置规则通道   */
    if (hadc->Init.ScanConvMode == ADC_SCAN_ENABLE)
    {
      /* 配置规格通道转换个数 */
      MODIFY_REG(hadc->Instance->SQR1, ADC_SQR1_L, (hadc->Init.NbrOfConversion - (uint8_t)1));
    }
    else
    {
      CLEAR_BIT(hadc->Instance->SQR1, ADC_SQR1_L);
    }


    /* 初始 ADC 就绪状态 */
    ADC_STATE_CLR_SET(hadc->State, HAL_ADC_STATE_BUSY_INTERNAL, HAL_ADC_STATE_READY);
  }
  else
  {
    /* 设置 ADC 状态错误 */
    SET_BIT(hadc->State, HAL_ADC_STATE_ERROR_INTERNAL);

    tmp_hal_status = HAL_ERROR;
  }

  /* 返回状态 */
  return tmp_hal_status;
}
```

函数描述：

此函数用于初始化 ADC1，ADC2 或者 ADC3。

函数参数：

- 第 1 个参数是 ADC_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。结构体变量成员的详细介绍看本章 3.2 小节。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 函数 HAL_ADC_MspInit 用于初始化定时器的底层时钟、引脚等功能。需要用户自己在此函数里面实现具体的功能。由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
2. 如果形参 hadc 的结构体成员 State 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 ADC_HandleTypeDef　　AdcHandle。

   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_ADC_STATE_RESET = 0x00U。

   解决办法有四

   方法 1：用户自己初始定时器和涉及到的 GPIO 等。

   方法 2：定义 ADC_HandleTypeDef　AdcHandle 为全局变量。

方法 3：定义为局部变量要赋初始值 ADC_HandleTypeDef AdcHandle = {0}。

方法 4：下面的方法：

```
if(HAL_ADC_DeInit(&AdcHandle)!= HAL_OK)
{
    Error_Handler();
}
if(HAL_ADC_Init(&AdcHandle)!= HAL_OK)
{
    Error_Handler();
}
```

3. ADC 有两种时钟源可供选择,可以使用来自 AHB 总线的系统时钟(属于同步时钟),也可以使用 PLL2，PLL3，HSE，HSI 或者 CSI 时钟（属于异步时钟）。如果使用异步时钟，调用函数 HAL_ADC_Init 前要单独配置。而 AHB 是默认时钟，所以不必单独配置。
4. 如果更新 ADC 的公共寄存器，需要关闭了所有 ADC 时才能更新。

使用举例：

```
ADC_HandleTypeDef    AdcHandle = {0};

__HAL_RCC_ADC12_CLK_ENABLE();

AdcHandle.Instance = ADC1;

/* 采用 AHB 同步时钟，4 分频，即 200MHz/4 = 50MHz */
AdcHandle.Init.ClockPrescaler        = ADC_CLOCK_SYNC_PCLK_DIV4;
AdcHandle.Init.Resolution            = ADC_RESOLUTION_16B;            /* 16 位分辨率 */
AdcHandle.Init.ScanConvMode          = ADC_SCAN_DISABLE;              /* 禁止扫描，因为仅开了一个通道 */
AdcHandle.Init.EOCSelection          = ADC_EOC_SINGLE_CONV;           /* EOC 转换结束标志 */
AdcHandle.Init.LowPowerAutoWait      = DISABLE;                       /* 禁止低功耗自动延迟特性 */
AdcHandle.Init.ContinuousConvMode    = DISABLE;                       /* 禁止自动转换，采用的定时器触发转换
*/
AdcHandle.Init.NbrOfConversion       = 1;                             /* 使用了 1 个转换通道 */
AdcHandle.Init.DiscontinuousConvMode = DISABLE;                       /* 禁止不连续模式 */

 /* 禁止不连续模式后，此参数忽略，此位是用来配置不连续子组中通道数 */
AdcHandle.Init.NbrOfDiscConversion   = 1;
AdcHandle.Init.ExternalTrigConv      = ADC_EXTERNALTRIG_T1_CC1;            /* 定时器 1 的 CC1 触发 */
AdcHandle.Init.ExternalTrigConvEdge  = ADC_EXTERNALTRIGCONVEDGE_RISING;    /* 上升沿触发 */
AdcHandle.Init.ConversionDataManagement = ADC_CONVERSIONDATA_DMA_CIRCULAR;/*DMA 循环模式接收 ADC 转换的数据*/
AdcHandle.Init.BoostMode          = ENABLE;                   /* ADC 时钟超过 20MHz 的话，使能 boost */
AdcHandle.Init.Overrun            = ADC_OVR_DATA_OVERWRITTEN; /* ADC 转换溢出的话，覆盖 ADC 的数据寄存器 */
AdcHandle.Init.OversamplingMode   = DISABLE;                  /* 禁止过采样 */

/* 初始化 DMA */
if(HAL_DMA_Init(&DmaHandle) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

44.4.2 函数 HAL_ADC_ConfigChannel

函数原型：

```
HAL_StatusTypeDef HAL_ADC_ConfigChannel(ADC_HandleTypeDef* hadc, ADC_ChannelConfTypeDef* sConfig)
{
  HAL_StatusTypeDef tmp_hal_status = HAL_OK;

  ADC_Common_TypeDef *tmpADC_Common;
  uint32_t tmpOffsetShifted;
  __IO uint32_t wait_loop_index = 0;

  /* 程序中不重要的部分被清除掉，仅留下关键部分做注释 */
  /* 上锁 */
  __HAL_LOCK(hadc);


  /* 配置 ADC 参数 */
  if (ADC_IS_CONVERSION_ONGOING_REGULAR(hadc) == RESET)
  {
    /* ADC 通道选择 */
    hadc->Instance->PCSEL |= (1U << sConfig->Channel);

    /* Rank 1 to 4 */
    if (sConfig->Rank < 5)
    {
      MODIFY_REG(hadc->Instance->SQR1,
                 ADC_SQR1_RK(ADC_SQR2_SQ5, sConfig->Rank),
                 ADC_SQR1_RK(sConfig->Channel, sConfig->Rank));
    }
    /* For Rank 5 to 9 */
    else if (sConfig->Rank < 10)
    {
      MODIFY_REG(hadc->Instance->SQR2,
                 ADC_SQR2_RK(ADC_SQR2_SQ5, sConfig->Rank),
                 ADC_SQR2_RK(sConfig->Channel, sConfig->Rank));
    }
    /* For Rank 10 to 14 */
    else if (sConfig->Rank < 15)
    {
      MODIFY_REG(hadc->Instance->SQR3,
                 ADC_SQR3_RK(ADC_SQR3_SQ10, sConfig->Rank),
                 ADC_SQR3_RK(sConfig->Channel, sConfig->Rank));
    }
    /* For Rank 15 to 16 */
    else
    {
      MODIFY_REG(hadc->Instance->SQR4,
                 ADC_SQR4_RK(ADC_SQR4_SQ15, sConfig->Rank),
                 ADC_SQR4_RK(sConfig->Channel, sConfig->Rank));
    }


    /* 更新 ADC 参数 */
    if (ADC_IS_CONVERSION_ONGOING_REGULAR_INJECTED(hadc) == RESET)
    {
      /* 通道采样时间配置 */
      /* For channels 10 to 19 */
      if (sConfig->Channel >= ADC_CHANNEL_10)
      {
        MODIFY_REG(hadc->Instance->SMPR2,
                   ADC_SMPR2(ADC_SMPR2_SMP10, sConfig->Channel),
```

```
                ADC_SMPR2(sConfig->SamplingTime, sConfig->Channel));
    }
    else /* For channels 0 to 9 */
    {
      MODIFY_REG(hadc->Instance->SMPR1,
                ADC_SMPR1(ADC_SMPR1_SMP0, sConfig->Channel),
                ADC_SMPR1(sConfig->SamplingTime, sConfig->Channel));
    }

    /* 配置偏移 */
    tmpOffsetShifted = ADC_OFFSET_SHIFT_RESOLUTION(hadc, sConfig->Offset);

    switch (sConfig->OffsetNumber)
    {
      case ADC_OFFSET_1:
         MODIFY_REG(hadc->Instance->OFR1,
                  ADC_OFR_FIELDS,
                  ADC_OFR_CHANNEL(sConfig->Channel) | tmpOffsetShifted);
         MODIFY_REG(hadc->Instance->CFGR2, ADC_CFGR2_RSHIFT1, sConfig->OffsetRightShift);
         if(sConfig->OffsetSignedSaturation != DISABLE)
         {
            SET_BIT(hadc->Instance->OFR1, ADC_OFR1_SSATE);
         }
         else
         {
           CLEAR_BIT(hadc->Instance->OFR1, ADC_OFR1_SSATE);
         }
        break;

      case ADC_OFFSET_2:
        break;

      case ADC_OFFSET_3:
        break;

      case ADC_OFFSET_4:
        break;

      default :
        break;
    }

  }

  /* ADC 参数更新 */
  /*  内部的 Vbat/VrefInt/TempSensor */
  if (ADC_IS_ENABLE(hadc) == RESET)
  {
    /* 配置差分模式 */
    if (sConfig->SingleDiff != ADC_DIFFERENTIAL_ENDED)
    {
      /* 禁止差分 */
      CLEAR_BIT(hadc->Instance->DIFSEL, ADC_DIFSEL_CHANNEL(sConfig->Channel));
    }
    else
    {
      /* 使能差分 */
      SET_BIT(hadc->Instance->DIFSEL, ADC_DIFSEL_CHANNEL(sConfig->Channel));
```

```
      /* 配置通道 ADC_IN+1 (negative input)的采样时间     */
      /* 通道 9 to 15 (ADC1, ADC2) or to 11 (ADC3), SMPR2 寄存器必须配置 */
      if (sConfig->Channel >= ADC_CHANNEL_9)
      {
        MODIFY_REG(hadc->Instance->SMPR2,
                ADC_SMPR2(ADC_SMPR2_SMP10, sConfig->Channel +1),
                ADC_SMPR2(sConfig->SamplingTime, sConfig->Channel +1));
      }
      else /* 对于通道 0 to 8, SMPR1 必须配置 */
      {
        MODIFY_REG(hadc->Instance->SMPR1,
            ADC_SMPR1(ADC_SMPR1_SMP0, sConfig->Channel +1),
             ADC_SMPR1(sConfig->SamplingTime, sConfig->Channel +1));
      }
    }
    /* 内部测量通道配置: Vbat/VrefInt/TempSensor */
    /* 公用寄存器配置 */
    if((hadc->Instance == ADC1) || (hadc->Instance == ADC2))
    {
      tmpADC_Common = ADC12_COMMON_REGISTER(hadc);
    }
    else
    {
      tmpADC_Common = ADC3_COMMON_REGISTER(hadc);
    }

    /* 具体内部通道的配置，省略未写 */
  }

}
else
{
  SET_BIT(hadc->State, HAL_ADC_STATE_ERROR_CONFIG);
  tmp_hal_status = HAL_ERROR;
}

/* 解锁 */
__HAL_UNLOCK(hadc);

/* 返回函数状态 */
return tmp_hal_status;
}
```

函数描述：

调用函数 HAL_ADC_Init 配置了基础功能后，就可以调用此函数配置 ADC 的具体通道了。

函数参数：

- 第 1 个参数是 ADC_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是 ADC_ChannelConfTypeDef 类型结构体指针变量，用于配置 ADC 的采样时间，使用的通道号，单端或者差分方式的配置等。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 第 1 个参数的结构体成员介绍在本章的 3.2 小节进行了详细说明。
2. 第 2 个参数的结构体成员介绍在本章的 3.4 小节进行了详细说明。

使用举例：

```
ADC_ChannelConfTypeDef   sConfig = {0};

/* 配置 ADC 通道  */
sConfig.Channel      = ADC_CHANNEL_10;              /* 配置使用的 ADC 通道 */
sConfig.Rank         = ADC_REGULAR_RANK_1;          /* 采样序列里的第 1 个 */
sConfig.SamplingTime = ADC_SAMPLETIME_1CYCLE_5;     /* 采样周期 */
sConfig.SingleDiff   = ADC_SINGLE_ENDED;            /* 单端输入 */
sConfig.OffsetNumber = ADC_OFFSET_NONE;             /* 无偏移 */
sConfig.Offset = 0;                                 /* 无偏移的情况下，此参数忽略 */

if (HAL_ADC_ConfigChannel(&AdcHandle, &sConfig) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

44.4.3 函数 HAL_ADC_Start

函数原型：

```
HAL_StatusTypeDef HAL_ADC_Start(ADC_HandleTypeDef* hadc)
```

函数描述：

调用函数 HAL_ADC_Init 配置了基础功能后，就可以调用此函数启动 ADC 了。

函数参数：

- 第 1 个参数是 ADC_HandleTypeDef 类型结构体指针变量。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 第 1 个参数的结构体成员介绍在本章的 3.2 小节进行了详细说明。

使用举例：

```
if (HAL_ADC_ConfigChannel(&AdcHandl) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

44.4.4 函数 HAL_ADC_Start_DMA

函数原型：

```
HAL_StatusTypeDef HAL_ADC_Start_DMA(ADC_HandleTypeDef* hadc, uint32_t* pData, uint32_t Length)
```

函数描述：

调用函数 HAL_ADC_Init 配置了基础功能后，就可以调用此函数启动 ADC 的 DMA 方式了。

函数参数：

- 第 1 个参数是 ADC_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是 ADC 采样数据传输的目的地址。
- 第 3 个参数是传输的数据长度。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 第 1 个参数的结构体成员介绍在本章的 3.2 小节进行了详细说明。
2. 这个函数会开启 DMA 的 HT，TC，TE 和 MDE 中断。
3. 此函数用于单 ADC 模式，多 ADC 模式是调用的函数 HAL_ADCEx_MultiModeStart_DMA。

使用举例：

```
uint16_t ADCxValues[4];
ADC_HandleTypeDef    AdcHandle = {0};

/* ADC 和 DMA 的配置部分未贴，函数较多 */
if (HAL_ADC_Start_DMA(&AdcHandle, (uint32_t *)ADCxValues, 4) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

44.4.5 函数 HAL_ADCEx_Calibration_Start

函数原型：

```
HAL_StatusTypeDef HAL_ADCEx_Calibration_Start(ADC_HandleTypeDef* hadc, uint32_t CalibrationMode, uint32_t SingleDiff)
```

函数描述：

调用函数 HAL_ADC_Init 配置了基础功能后，就可以调用此函数启动 ADC 的自校准功能了，支持偏移校准和线性度校准。

函数参数：

- 第 1 个参数是 ADC_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是校准模式选择：
  - ADC_CALIB_OFFSET 表示只运行偏移校准而不运行线性度校准。
  - ADC_CALIB_OFFSET_LINEARITY 表示同时运行偏移校准和线性度校准。
- 第 3 个参数是单端或差分模式选择：
  - ADC_SINGLE_ENDED 表示单端模式。
  - ADC_DIFFERENTIAL_ENDED 表示差分模式。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 第 1 个参数的结构体成员介绍在本章的 3.2 小节进行了详细说明。
2. 必须在函数 HAL_ADC_Start（或者中断和 MDA 方式的启动函数）执行前或者 HAL_ADC_Stop（或者中断和 MDA 方式的停止函数）执行后才可以调用此校准函数。

使用举例：

```
/* 校准 ADC，采用偏移校准 */
if (HAL_ADCEx_Calibration_Start(&AdcHandle, ADC_CALIB_OFFSET, ADC_SINGLE_ENDED) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

44.5 总结

本章节就为大家讲解这么多，由于 ADC 用到的场合比较多，建议将常用的知识点熟练掌握。

第45章 STM32H7 的 ADC 应用之定时器触发配合 DMA 双缓冲

本章教程为大家讲解定时器触发配合 DMA 双缓冲做 ADC 数据采集，实际项目中有一定的使用价值，一个缓冲接收数据的时候，另一个缓冲可以做数据处理。

45.1 初学者重要提示
45.2 ADC 稳压基准硬件设计
45.3 ADC 驱动设计
45.4 ADC 板级支持包（bsp_adc.c）
45.5 ADC 驱动移植和使用
45.6 实验例程设计框架
45.7 实验例程说明（MDK）
45.8 实验例程说明（IAR）
45.9 总结

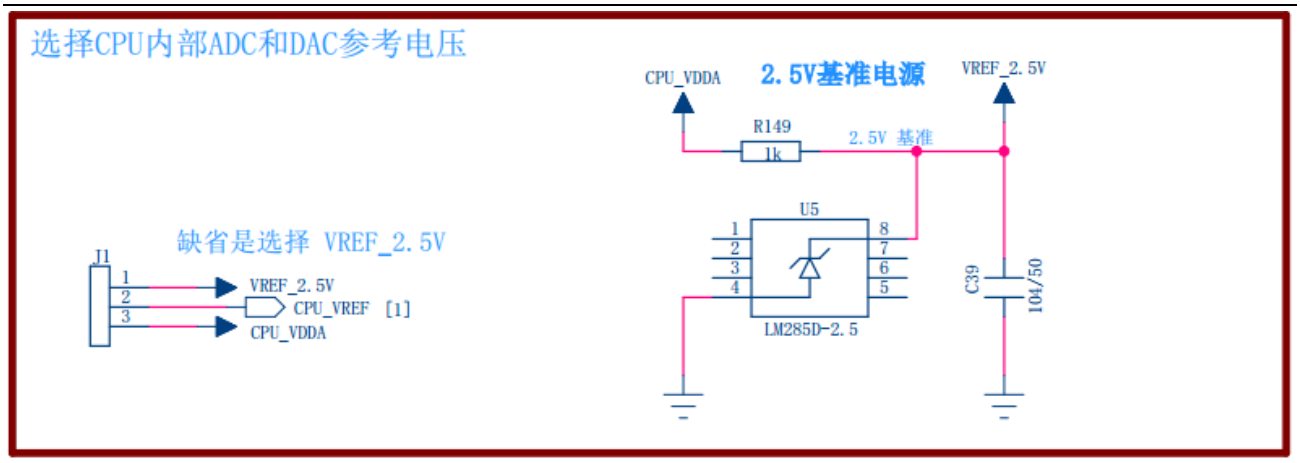
45.1 初学者重要提示

- 学习本章节前，务必优先学习第 44 章，需要对 ADC 的基础知识和 HAL 库的几个常用 API 有个认识。
- 开发板右上角有个跳线帽，可以让 ADC 的稳压基准接 3.3V 或者 2.5V，本章例子是接到 3.3V。
- STM32H7 的 ADC 支持偏移校准和线性度校准。如果使用线性度校准的话，特别要注意此贴的问题：http://www.armbbs.cn/forum.php?mod=viewthread&tid=91436 。
- ADC 的专业术语诠释文档，推荐大家看看：http://www.armbbs.cn/forum.php?mod=viewthread&tid=89414 。

45.2 ADC 稳压基准硬件设计

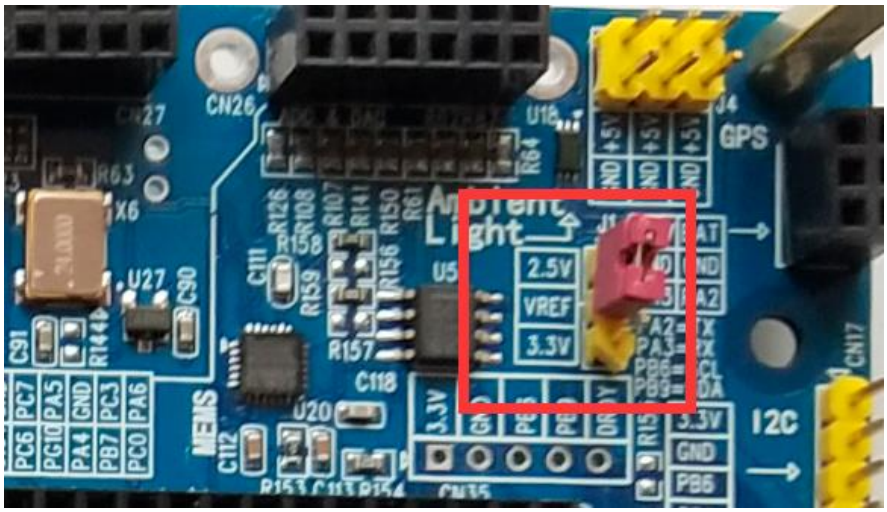
注：学习前务必优先看第 14 章的 2.1 小节，对电源供电框架有个了解。

ADC 要采集的准确，就需要有一个稳定的稳压基准源，V7 开发板使用的 LM285D-2.5，即 2.5V 的基准源。硬件设计如下：



关于这个原理图要注意以下问题：

LM285D-2.5 输出的是 2.5V 的稳压基准，原理图这里做了一个特别的处理，同时接了一个上拉电阻到 VDDA（3.3V），然后用户可以使用开发板右上角的跳线帽设置 Vref 选择 3.3V 稳压还是 2.5V 稳压。



下面再来了解下 LM285 的电气特性：

PARAMETER		TEST CONDITIONS		MIN	TYP	MAX	UNIT
LM285-2.5							
V_Z	Reference voltage	I_Z = 20 µA to 20 mA		2.462	2.5	2.538	V
$^{\alpha}V_Z$	Average temperature coefficient of reference voltage(1)	I_Z = 20 µA to 20 mA	Full range(2)		±20		ppm/°C
ΔV_Z	Change in reference voltage with current	I_Z = 20 µA to 1 mA	T_A = 25°C			1	mV
			Full range			1.5	
		I_Z = 1 mA to 20 mA	T_A = 25°C			10	
			Full range			30	
$\Delta V_Z/\Delta t$	Long-term change in reference voltage	I_Z = 100 µA			±20		ppm/khr
$I_{Z(MIN)}$	Minimum reference current	Full range			8	20	µA
z_z	Reference impedance	I_Z = 100 µA	T_A = 25°C		0.2	0.6	Ω
			Full range			1.5	
V_n	Broadband noise voltage	I_Z = 100 µA, f = 10 Hz to 10 kHz			120		µV

通过这个表，我们要了解以下几点知识：

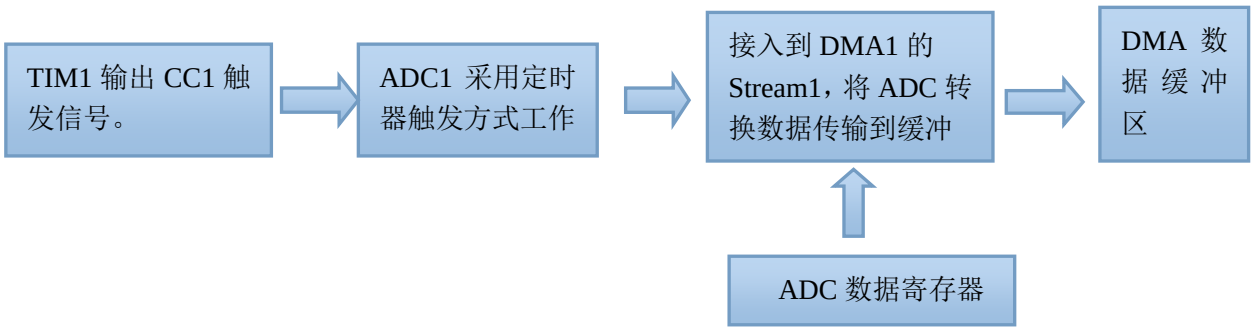
- LM285 的典型值是 2.5V，支持的最小值 2.462V，最大值 2.538V。工作电流是 20uA 到 20mA，温飘是±20ppm/℃。
- Iz 是 Reference current 参考电流的意思：
  - 参考电流是 20uA 到 1mA，温度 25℃，参考电压最大变化 1mV。
  - 参考电流是 20uA 到 1mA，全范围温度（-40°C to 85°C），参考电压最大变化 1.5mV。
  - 参考电流是 1mA 到 20mA，温度 25℃，参考电压最大变化 10mV。
  - 参考电流是 1mA 到 20mA，全范围温度（-40°C to 85°C），参考电压最大变化 30mV。

那么问题来了，V7 开发板上 LM285 的参考电流是多少？ 简单计算就是：

（VDDA – 2.5V） / 1K = （3.3 – 2.5V） / 1K = 0.8mA。

45.3 ADC 驱动设计

定时器触发 ADC 做 DMA 数据传输的实现思路框图如下：



下面将程序设计中的相关问题逐一为大家做个说明。

45.3.1 触发 ADC 的定时器选择和配置

ADC 转换既可以选择外部触发也可以选择软件触发。定时器属于外部触发方式，使用定时器触发的好处是可以设置任何 ADC 能够支持的转换频率。

对于 ADC1，ADC2，ADC3 来说，规则通道支持的外部触发源如下：

```
#define ADC_EXTERNALTRIG_T1_CC1          ((uint32_t)0x00000000)
#define ADC_EXTERNALTRIG_T1_CC2          ((uint32_t)ADC_CFGR_EXTSEL_0)
#define ADC_EXTERNALTRIG_T1_CC3          ((uint32_t)ADC_CFGR_EXTSEL_1)
#define ADC_EXTERNALTRIG_T2_CC2          ((uint32_t)(ADC_CFGR_EXTSEL_1 | ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_T3_TRGO         ((uint32_t)ADC_CFGR_EXTSEL_2)
#define ADC_EXTERNALTRIG_T4_CC4          ((uint32_t)(ADC_CFGR_EXTSEL_2 | ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_EXT_IT11        ((uint32_t)(ADC_CFGR_EXTSEL_2 | ADC_CFGR_EXTSEL_1))
#define ADC_EXTERNALTRIG_T8_TRGO         ((uint32_t)(ADC_CFGR_EXTSEL_2 | ADC_CFGR_EXTSEL_1 |
                                                     ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_T8_TRGO2        ((uint32_t) ADC_CFGR_EXTSEL_3)
```

```
#define ADC_EXTERNALTRIG_T1_TRGO          ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_T1_TRGO2         ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_1))
#define ADC_EXTERNALTRIG_T2_TRGO          ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_1 |
                                                      ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_T4_TRGO          ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_2))
#define ADC_EXTERNALTRIG_T6_TRGO          ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_2 |
                                                      ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_T15_TRGO         ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_2 |
                                                      ADC_CFGR_EXTSEL_1))
#define ADC_EXTERNALTRIG_T3_CC4           ((uint32_t)(ADC_CFGR_EXTSEL_3 | ADC_CFGR_EXTSEL_2 |
                                                      ADC_CFGR_EXTSEL_1 | ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_HR1_ADCTRG1      ((uint32_t) ADC_CFGR_EXTSEL_4)
#define ADC_EXTERNALTRIG_HR1_ADCTRG3      ((uint32_t) (ADC_CFGR_EXTSEL_4 | ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_LPTIM1_OUT       ((uint32_t) (ADC_CFGR_EXTSEL_4 | ADC_CFGR_EXTSEL_1))
#define ADC_EXTERNALTRIG_LPTIM2_OUT       ((uint32_t) (ADC_CFGR_EXTSEL_4 | ADC_CFGR_EXTSEL_1|
                                                       ADC_CFGR_EXTSEL_0))
#define ADC_EXTERNALTRIG_LPTIM3_OUT       ((uint32_t) (ADC_CFGR_EXTSEL_4 | ADC_CFGR_EXTSEL_2))
```

我们这里使用的是 TIM1_CC1。

接下来就是 TIM1 的时钟配置问题，代码如下：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: TIM1_Config
4.  *    功能说明: 配置 TIM1，用于触发 ADC，当前配置的 100KHz 触发频率
5.  *    形    参: 无
6.  *    返 回 值: 无
7.  *********************************************************************************************************
8.  */
9.  static void TIM1_Config(void)
10. {
11.     TIM_HandleTypeDef  htim ={0};
12.     TIM_OC_InitTypeDef sConfig = {0};
13.
14.
15.     /* 使能时钟 */
16.     __HAL_RCC_TIM1_CLK_ENABLE();
17.
18.    /*-----------------------------------------------------------------------
19.         bsp.c 文件中 void SystemClock_Config(void) 函数对时钟的配置如下:
20.
21.        System Clock source       = PLL (HSE)
22.        SYSCLK(Hz)                = 400000000 (CPU Clock)
23.        HCLK(Hz)                  = 200000000 (AXI and AHBs Clock)
24.        AHB Prescaler             = 2
25.        D1 APB3 Prescaler         = 2 (APB3 Clock  100MHz)
26.        D2 APB1 Prescaler         = 2 (APB1 Clock  100MHz)
27.        D2 APB2 Prescaler         = 2 (APB2 Clock  100MHz)
28.        D3 APB4 Prescaler         = 2 (APB4 Clock  100MHz)
29.
30.        因为 APB1 prescaler != 1, 所以 APB1 上的 TIMxCLK = APB1 x 2 = 200MHz;
31.        因为 APB2 prescaler != 1, 所以 APB2 上的 TIMxCLK = APB2 x 2 = 200MHz;
32.        APB4 上面的 TIMxCLK 没有分频，所以就是 100MHz;
33.
34.        APB1 定时器有 TIM2, TIM3 ,TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14，LPTIM1
35.        APB2 定时器有 TIM1, TIM8 , TIM15, TIM16，TIM17
36.
37.        APB4 定时器有 LPTIM2，LPTIM3，LPTIM4，LPTIM5
```

```
38.
39.      TIM12CLK = 200MHz/(Period + 1) / (Prescaler + 1) = 200MHz / 2000 / 1 = 100KHz
40.       ----------------------------------------------------------------------- */
41.       HAL_TIM_Base_DeInit(&htim);
42.
43.       htim.Instance = TIM1;
44.       htim.Init.Period            = 1999;
45.       htim.Init.Prescaler         = 0;
46.       htim.Init.ClockDivision     = 0;
47.       htim.Init.CounterMode       = TIM_COUNTERMODE_UP;
48.       htim.Init.RepetitionCounter = 0;
49.       HAL_TIM_Base_Init(&htim);
50.
51.      sConfig.OCMode     = TIM_OCMODE_PWM1;
52.      sConfig.OCPolarity = TIM_OCPOLARITY_LOW;
53.
54.      /* 占空比 50% */
55.      sConfig.Pulse = 1000;
56.      if(HAL_TIM_OC_ConfigChannel(&htim, &sConfig, TIM_CHANNEL_1) != HAL_OK)
57.      {
58.           Error_Handler(__FILE__, __LINE__);
59.      }
60.
61.      /* 启动 OC1 */
62.      if(HAL_TIM_OC_Start(&htim, TIM_CHANNEL_1) != HAL_OK)
63.      {
64.           Error_Handler(__FILE__, __LINE__);
65.      }
66.  }
```

这里把几个关键的地方阐释下：

- 第 11 – 12 行，对作为局部变量的 HAL 库结构体做初始化，防止不确定值配置时出问题。
- 第 18 – 65 行，注释已经比较详细，配置 TIM1 的频率是 100KHz，这个速度就是 ADC 的触发频率。
  TIM1CLK = 200MHz / (Period + 1) / (Prescaler + 1) = 200MHz/（1999+1）/（0+1） = 100KHz
  占空比 = Pulse / (Period + 1) = 1000 / （1999+1） = 50%

  这些知识点在前面的定时器章节有更详细的说明。

45.3.2 ADC 时钟源选择

根据第 44 章 2.2 小节的讲解，我们知道 ADC 有两种时钟源可供选择，可以使用来自 AHB 总线的系统时钟，也可以使用 PLL2，PLL3，HSE，HSI 或者 CSI 时钟。

如果采用 AHB 时钟，不需要做专门的配置，而采用 PLL2，PLL3 时钟需要特别的配置，下面是使用 AHB 或者 PLL2 时钟的配置。

- 通过宏定义设置选择的时钟源

使用那个时钟源，将另一个注释掉即可：

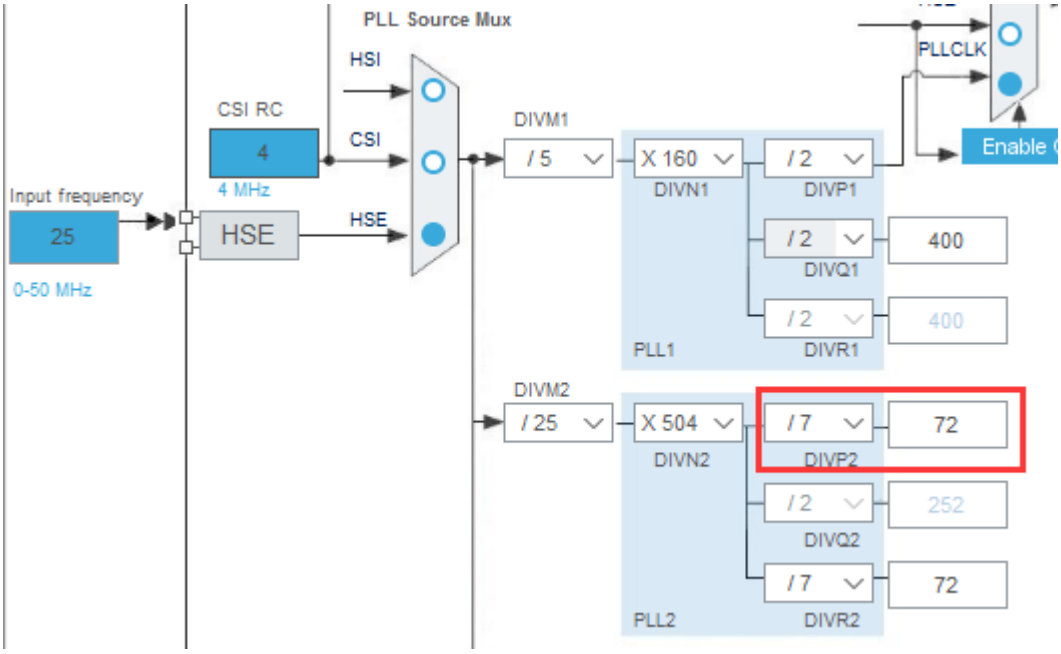
```
/* 选择 ADC 的时钟源 */
#define ADC_CLOCK_SOURCE_AHB     /* 选择 AHB 时钟源 */
```

```
//#define ADC_CLOCK_SOURCE_PLL   /* 选择 PLL 时钟源 */
```

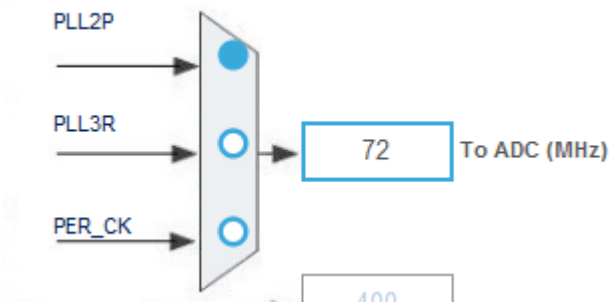
◆ PLL2 或者 AHB 时钟源配置

```
#if defined (ADC_CLOCK_SOURCE_PLL)
    /* 配置 PLL2 时钟为的 72MHz，方便分频产生 ADC 最高时钟 36MHz */
    RCC_PeriphCLKInitTypeDef PeriphClkInitStruct = {0};
    PeriphClkInitStruct.PeriphClockSelection = RCC_PERIPHCLK_ADC;
    PeriphClkInitStruct.PLL2.PLL2M = 25;
    PeriphClkInitStruct.PLL2.PLL2N = 504;
    PeriphClkInitStruct.PLL2.PLL2P = 7;
    PeriphClkInitStruct.PLL2.PLL2Q = 7;
    PeriphClkInitStruct.PLL2.PLL2R = 7;
    PeriphClkInitStruct.PLL2.PLL2RGE = RCC_PLL2VCIRANGE_0;
    PeriphClkInitStruct.PLL2.PLL2VCOSEL = RCC_PLL2VCOWIDE;
    PeriphClkInitStruct.PLL2.PLL2FRACN = 0;
    PeriphClkInitStruct.AdcClockSelection = RCC_ADCCLKSOURCE_PLL2;
    if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInitStruct) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
#elif defined (ADC_CLOCK_SOURCE_AHB)

  /* 使用 AHB 时钟的话，无需配置，默认选择*/

#endif
```

对于 PLL2 的时钟输出，直接使用 STM32CubeMX 里面的时钟树配置即可，效果如下：

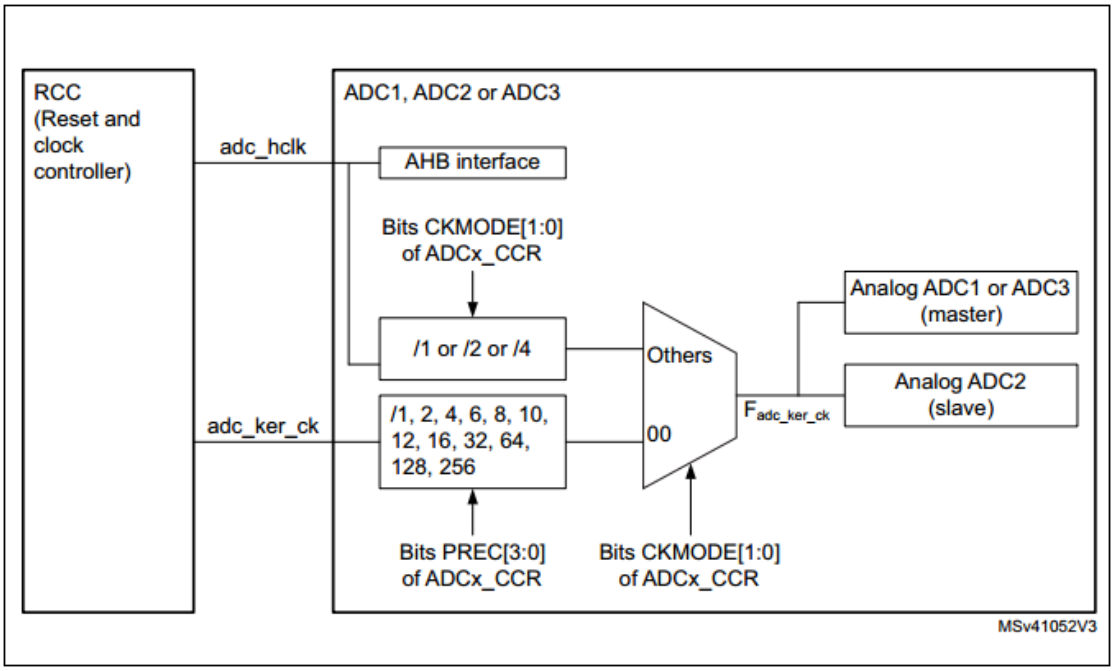


选择 PLL2P 输出作为 ADC 时钟源：



◆ ADC 分频设置

无论是使用 AHB 时钟还是 PLL2 时钟都支持分频设置：



AHB 支持下面三种分频设置：

```
#define ADC_CLOCK_SYNC_PCLK_DIV1    ((uint32_t)ADC_CCR_CKMODE_0)
#define ADC_CLOCK_SYNC_PCLK_DIV2    ((uint32_t)ADC_CCR_CKMODE_1)
#define ADC_CLOCK_SYNC_PCLK_DIV4    ((uint32_t)ADC_CCR_CKMODE)

#define ADC_CLOCKPRESCALER_PCLK_DIV1    ADC_CLOCK_SYNC_PCLK_DIV1    /* 这三个仅仅是为了兼容，已经不推荐使用 */
#define ADC_CLOCKPRESCALER_PCLK_DIV2    ADC_CLOCK_SYNC_PCLK_DIV2
#define ADC_CLOCKPRESCALER_PCLK_DIV4    ADC_CLOCK_SYNC_PCLK_DIV4
```

PLL2 支持下面几种分频设置：

```
#define ADC_CLOCK_ASYNC_DIV1        ((uint32_t)0x00000000)
#define ADC_CLOCK_ASYNC_DIV2        ((uint32_t)ADC_CCR_PRESC_0)
#define ADC_CLOCK_ASYNC_DIV4        ((uint32_t)ADC_CCR_PRESC_1)
#define ADC_CLOCK_ASYNC_DIV6        ((uint32_t)(ADC_CCR_PRESC_1|ADC_CCR_PRESC_0))
#define ADC_CLOCK_ASYNC_DIV8        ((uint32_t)(ADC_CCR_PRESC_2))
#define ADC_CLOCK_ASYNC_DIV10       ((uint32_t)(ADC_CCR_PRESC_2|ADC_CCR_PRESC_0))
#define ADC_CLOCK_ASYNC_DIV12       ((uint32_t)(ADC_CCR_PRESC_2|ADC_CCR_PRESC_1))
#define ADC_CLOCK_ASYNC_DIV16       ((uint32_t)(ADC_CCR_PRESC_2|ADC_CCR_PRESC_1|ADC_CCR_PRESC_0))
#define ADC_CLOCK_ASYNC_DIV32       ((uint32_t)(ADC_CCR_PRESC_3))
#define ADC_CLOCK_ASYNC_DIV64       ((uint32_t)(ADC_CCR_PRESC_3|ADC_CCR_PRESC_0))
#define ADC_CLOCK_ASYNC_DIV128      ((uint32_t)(ADC_CCR_PRESC_3|ADC_CCR_PRESC_1))
```

```
#define ADC_CLOCK_ASYNC_DIV256      ((uint32_t)(ADC_CCR_PRESC_3|ADC_CCR_PRESC_1|ADC_CCR_PRESC_0))
```

有了这些认识后再看实际的分频配置就好理解了：

```
#if defined (ADC_CLOCK_SOURCE_PLL)
/* 采用 PLL 异步时钟，2 分频，即 72MHz/2 = 36MHz */
    AdcHandle.Init.ClockPrescaler        = ADC_CLOCK_ASYNC_DIV2;
/* 采用 AHB 同步时钟，4 分频，即 200MHz/4 = 50MHz */
#elif defined (ADC_CLOCK_SOURCE_AHB)
    AdcHandle.Init.ClockPrescaler        = ADC_CLOCK_SYNC_PCLK_DIV4;
#endif
```

45.3.3 ADC 的 DMA 双缓冲配置

由于函数 HAL_ADC_Start_DMA 封装的 DMA 传输函数是 HAL_DMA_Start_IT,而不是专门的 DMA 双缓冲函数 HAL_DMAEx_MultiBuferStart_IT。所以要实现双缓冲效果的话，可以使用半传输完成中断和传输完成中断配合实现双缓冲效果。这里使用的是 DMA1_Stream1：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: bsp_InitADC
4.  *    功能说明: 初始化 ADC
5.  *    形    参: 无
6.  *    返 回 值: 无
7.  *********************************************************************************************************
8.  */
9.  void bsp_InitADC(void)
10. {
11.     ADC_HandleTypeDef    AdcHandle = {0};
12.     DMA_HandleTypeDef    DmaHandle = {0};
13.     ADC_ChannelConfTypeDef    sConfig = {0};
14.     GPIO_InitTypeDef         GPIO_InitStruct;
15.
16.
17.   /* ## - 1 - 配置 ADC 采样的时钟 ######################################### */
18. #if defined (ADC_CLOCK_SOURCE_PLL)
19.     /* 配置 PLL2 时钟为的 72MHz，方便分频产生 ADC 最高时钟 36MHz */
20.     RCC_PeriphCLKInitTypeDef PeriphClkInitStruct = {0};
21.     PeriphClkInitStruct.PeriphClockSelection = RCC_PERIPHCLK_ADC;
22.     PeriphClkInitStruct.PLL2.PLL2M = 25;
23.     PeriphClkInitStruct.PLL2.PLL2N = 504;
24.     PeriphClkInitStruct.PLL2.PLL2P = 7;
25.     PeriphClkInitStruct.PLL2.PLL2Q = 7;
26.     PeriphClkInitStruct.PLL2.PLL2R = 7;
27.     PeriphClkInitStruct.PLL2.PLL2RGE = RCC_PLL2VCIRANGE_0;
28.     PeriphClkInitStruct.PLL2.PLL2VCOSEL = RCC_PLL2VCOWIDE;
29.     PeriphClkInitStruct.PLL2.PLL2FRACN = 0;
30.     PeriphClkInitStruct.AdcClockSelection = RCC_ADCCLKSOURCE_PLL2;
31.     if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInitStruct) != HAL_OK)
32.     {
33.         Error_Handler(__FILE__, __LINE__);
34.     }
35. #elif defined (ADC_CLOCK_SOURCE_AHB)
36.
37.   /* 使用 AHB 时钟的话，无需配置，默认选择*/
38.
```

```
#endif

    /* ## - 2 - 配置 ADC 采样使用的采集引脚 ######################################### */
    __HAL_RCC_GPIOC_CLK_ENABLE();

    GPIO_InitStruct.Pin = GPIO_PIN_0;
    GPIO_InitStruct.Mode = GPIO_MODE_ANALOG;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

    /* ## - 3 - 配置 ADC 采样使用的时钟 ######################################### */
    __HAL_RCC_DMA1_CLK_ENABLE();
    DmaHandle.Instance                 = DMA1_Stream1;            /* 使用的 DMA1 Stream1 */
    DmaHandle.Init.Request             = DMA_REQUEST_ADC1;         /* 请求类型采用 DMA_REQUEST_ADC1 */
    DmaHandle.Init.Direction           = DMA_PERIPH_TO_MEMORY;    /* 传输方向是从外设到存储器 */
    DmaHandle.Init.PeriphInc           = DMA_PINC_DISABLE;        /* 外设地址自增禁止 */
    DmaHandle.Init.MemInc              = DMA_MINC_ENABLE;         /* 存储器地址自增使能 */
    DmaHandle.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD;/* 外设数据传输位宽选择半字，16bit */
    DmaHandle.Init.MemDataAlignment    = DMA_MDATAALIGN_HALFWORD;/* 存储器数据传输位宽选半字，16bit */
    DmaHandle.Init.Mode                = DMA_CIRCULAR;            /* 循环模式 */
    DmaHandle.Init.Priority            = DMA_PRIORITY_LOW;        /* 优先级低 */
    DmaHandle.Init.FIFOMode            = DMA_FIFOMODE_DISABLE;    /* 禁止 FIFO*/
    DmaHandle.Init.FIFOThreshold = DMA_FIFO_THRESHOLD_FULL; /* 禁止 FIFO 此位不起作用，用于设置阀值 */
    DmaHandle.Init.MemBurst      = DMA_MBURST_SINGLE;       /* 禁止 FIFO 此位不起作用,用于存储器突发 */
    DmaHandle.Init.PeriphBurst   = DMA_PBURST_SINGLE;      /* 禁止 FIFO 此位不起作用，用于外设突发 */

    /* 初始化 DMA */
    if(HAL_DMA_Init(&DmaHandle) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 开启 DMA1 Stream1 的中断 */
    HAL_NVIC_SetPriority(DMA1_Stream1_IRQn, 2, 0);
    HAL_NVIC_EnableIRQ(DMA1_Stream1_IRQn);

    /* 关联 ADC 句柄和 DMA 句柄 */
    __HAL_LINKDMA(&AdcHandle, DMA_Handle, DmaHandle);

    /* ## - 4 - 配置 ADC ######################################################### */
    __HAL_RCC_ADC12_CLK_ENABLE();
    AdcHandle.Instance = ADC1;

#if defined (ADC_CLOCK_SOURCE_PLL)
    AdcHandle.Init.ClockPrescaler  = ADC_CLOCK_ASYNC_DIV2;     /* 采用 PLL 异步时钟，2 分频，即 72MHz/2
                                                                  = 36MHz */
#elif defined (ADC_CLOCK_SOURCE_AHB)
    AdcHandle.Init.ClockPrescaler  = ADC_CLOCK_SYNC_PCLK_DIV4;  /* 采用 AHB 同步时钟，4 分频，即 200MHz/4
                                                                   = 50MHz */
#endif
    AdcHandle.Init.Resolution            = ADC_RESOLUTION_16B;    /* 16 位分辨率 */
    AdcHandle.Init.ScanConvMode          = ADC_SCAN_DISABLE;      /* 禁止扫描，因为仅开了一个通道 */
    AdcHandle.Init.EOCSelection          = ADC_EOC_SINGLE_CONV;   /* EOC 转换结束标志 */
    AdcHandle.Init.LowPowerAutoWait      = DISABLE;               /* 禁止低功耗自动延迟特性 */
    AdcHandle.Init.ContinuousConvMode    = DISABLE;             /* 禁止自动转换，采用的定时器触发转换 */
    AdcHandle.Init.NbrOfConversion       = 1;          /* 使用了 1 个转换通道 */
    AdcHandle.Init.DiscontinuousConvMode = DISABLE;    /* 禁止不连续模式 */
    AdcHandle.Init.NbrOfDiscConversion   = 1;          /* 禁止不连续模式后，此参数忽略，此位是用来配置
```

```
97.                                                     不连续子组中通道数 */
98.       AdcHandle.Init.ExternalTrigConv      =ADC_EXTERNALTRIG_T1_CC1;            /* 定时器 1 的 CC1 触发 */
99.       AdcHandle.Init.ExternalTrigConvEdge  = ADC_EXTERNALTRIGCONVEDGE_RISING;    /* 上升沿触发 */
100.      AdcHandle.Init.ConversionDataManagement = ADC_CONVERSIONDATA_DMA_CIRCULAR; /* DMA 循环模式接收 ADC
101.                                                                                  转换的数据 */
102.      AdcHandle.Init.BoostMode     = ENABLE;                 /* ADC 时钟超过 20MHz 的话，使能 boost */
103.      AdcHandle.Init.Overrun     = ADC_OVR_DATA_OVERWRITTEN; /* ADC 转换溢出的话，覆盖 ADC 的数据寄存器 */
104.      AdcHandle.Init.OversamplingMode          = DISABLE;    /* 禁止过采样 */
105.
106.     /* 初始化 ADC */
107.      if (HAL_ADC_Init(&AdcHandle) != HAL_OK)
108.      {
109.          Error_Handler(__FILE__, __LINE__);
110.      }
111.
112.      /* 校准 ADC，采用偏移校准 */
113.      if (HAL_ADCEx_Calibration_Start(&AdcHandle, ADC_CALIB_OFFSET, ADC_SINGLE_ENDED) != HAL_OK)
114.      {
115.          Error_Handler(__FILE__, __LINE__);
116.      }
117.
118.      /* 配置 ADC 通道  */
119.      sConfig.Channel      = ADC_CHANNEL_10;              /* 配置使用的 ADC 通道 */
120.      sConfig.Rank         = ADC_REGULAR_RANK_1;          /* 采样序列里的第 1 个 */
121.      sConfig.SamplingTime = ADC_SAMPLETIME_1CYCLE_5;     /* 采样周期 */
122.      sConfig.SingleDiff   = ADC_SINGLE_ENDED;            /* 单端输入 */
123.      sConfig.OffsetNumber = ADC_OFFSET_NONE;             /* 无偏移 */
124.      sConfig.Offset = 0;                                 /* 无偏移的情况下，此参数忽略 */
125.
126.      if (HAL_ADC_ConfigChannel(&AdcHandle, &sConfig) != HAL_OK)
127.      {
128.          Error_Handler(__FILE__, __LINE__);
129.      }
130.
131.      /* ## - 5 - 配置 ADC 的定时器触发 ########################################## */
132.      TIM1_Config();
133.
134.      /* ## - 6 - 启动 ADC 的 DMA 方式传输 ########################################## */
135.      if (HAL_ADC_Start_DMA(&AdcHandle, (uint32_t *)ADCxValues, 128) != HAL_OK)
136.      {
137.          Error_Handler(__FILE__, __LINE__);
138.      }
139. }
```

这里把几个关键的地方阐释下：

- 第 11 - 13 行，对作为局部变量的 HAL 库结构体做初始化，防止不确定值配置时出问题。
- 第 18 - 39 行，前面 2.2 小节已经讲解，ADC 时钟源选择 AHB 时钟还是 PLL 时钟。
- 第 42 – 47 行，选择 PC0 作为数据采集引脚。
- 第 50- 69 行，配置 DMA 的基本参数，注释较详细。这里是采用的 ADC 外设到内部 SRAM 的传输方向，数据带宽设置 16bit，循环传输模式。
- 第 72 – 73 行，配置 DMA 的中断优先级，并使能。
- 第 76 行，这行代码比较重要，应用中容易被遗忘，用于关联 ADC 句柄和 DMA 句柄。在用户调用 ADC 的 DMA 传输方式函数 HAL_ADC_Start_DMA 时,此函数内部调用的 HAL_DMA_Start_IT 会用

到 DMA 句柄。

- 第 79 - 110 行，主要是 ADC 的配置，注释较详细，配置 ADC1 为 16bit 模式，采用定时器 1 的 CC1 作为外部触发。
- 第 113 – 116 行，这里的是采用的 ADC 偏移校准，如果要采用线性度校准，务必要注意此贴的问题：http://www.armbbs.cn/forum.php?mod=viewthread&tid=91436 。
- 第 119 -129 行，配置 ADC 通道参数。这里使用的通道 10 是 PC0 引脚的复用功能，不是随意设置的。另外注意这里的采用周期，取的最小值，方便实现 ADC 外部触发的最快速度。
- 第 132 行，配置 ADC 的定时器触发，在本章 2.2 小节有讲解。
- 第 135 – 138 行，启动 ADC 的 DMA 方式数据传输。

45.3.4 DMA 存储器选择注意事项

由于 STM32H7 Cache 的存在，凡是 CPU 和 DMA 都会操作到的存储器，我们都要注意数据一致性问题。对于本章节要实现的功能，要注意读 Cache 问题，防止 DMA 已经更新了缓冲区的数据，而我们读取的却是 Cache 里面缓存的。这里提供两种解决办法：

- 方法一：

  关闭 DMA 所使用 SRAM 存储区。

```
/* 配置 SRAM 的 MPU 属性为 Device 或者 Strongly Ordered，即关闭 Cache */
MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress      = 0x60000000;
MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;
```

- 方法二：

  设置 SRAM 的缓冲区做 32 字节对齐，大小最好也是 32 字节整数倍，然后调用函数 SCB_InvalidateDCache_by_Addr 做无效化操作即可，保证 CPU 读取到的数据是刚更新好的。

本章节配套例子是直接使用的方法二。例子中变量的定义方式如下：

```
/* 方便 Cache 类的 API 操作，做 32 字节对齐 */
#if defined ( __ICCARM__ )
#pragma location = 0x38000000
uint16_t ADCxValues[128];
#elif defined ( __CC_ARM )
ALIGN_32BYTES(__attribute__((section (".RAM_D3")))) uint16_t ADCxValues[128];
#endif
```

对于 IAR 需要#pragma location 指定位置，而 MDK 通过分散加载即可实现，详情看前面第 26 章，有

详细讲解。

45.3.5 DMA 中断处理

调用函数 HAL_ADC_Start_DMA 开启了 DMA 的传输完成中断，半传输完成中断，传输错误中断和直接模式错误中断。通过传输完成中断和半传输完整中断可以实现双缓冲的处理：

```
/*
*********************************************************************************************************
*    函 数 名: DMA1_Stream1_IRQHandler
*    功能说明: DMA1 Stream1 中断服务程序
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DMA1_Stream1_IRQHandler(void)
{
    /* 传输完成中断 */
    if((DMA1->LISR & DMA_FLAG_TCIF1_5) != RESET)
    {
        /* 清除标志 */
        DMA1->LIFCR = DMA_FLAG_TCIF1_5;

        /* 当前使用的缓冲 0 */
        if((DMA1_Stream1->CR & DMA_SxCR_CT) == RESET)
        {
            /*
                1、当前正在使用缓冲 0，此时可以动态修改缓冲 1 的数据。
                   比如缓冲区 0 是 IO_Toggle，缓冲区 1 是 IO_Toggle1，那么此时就可以修改 IO_Toggle1。
                2、变量所在的 SRAM 区已经通过 MPU 配置为 WT 模式，更新变量 IO_Toggle 会立即写入。
                3、不配置 MPU 的话，也可以通过 Cahce 的函数 SCB_CleanDCache_by_Addr 做 Clean 操作。
            */

        }
        /* 当前使用的缓冲 1 */
        else
        {
             /*
               1、当前正在使用缓冲 1，此时可以动态修改缓冲 0 的数据。
                  比如缓冲区 0 是 IO_Toggle，缓冲区 1 是 IO_Toggle1，那么此时就可以修改 IO_Toggle。
               2、变量所在的 SRAM 区已经通过 MPU 配置为 WT 模式，更新变量 IO_Toggle 会立即写入。
               3、不配置 MPU 的话，也可以通过 Cahce 的函数 SCB_CleanDCache_by_Addr 做 Clean 操作。
            */

        }
    }

    /* 半传输完成中断 */
    if((DMA1->LISR & DMA_FLAG_HTIF1_5) != RESET)
    {
        /* 清除标志 */
        DMA1->LISR = DMA_FLAG_HTIF1_5;
    }

    /* 传输错误中断 */
```

```
    if((DMA1->LISR & DMA_FLAG_TEIF1_5) != RESET)
    {
        /* 清除标志 */
        DMA1->LISR = DMA_FLAG_TEIF1_5;
    }

    /* 直接模式错误中断 */
    if((DMA1->LISR & DMA_FLAG_DMEIF1_5) != RESET)
    {
        /* 清除标志 */
        DMA1->LISR = DMA_FLAG_DMEIF1_5;
    }
}

/*
*********************************************************************************************************
*    函 数 名: DMA1_Stream1_IRQHandler
*    功能说明: DMA1 Stream1 中断服务程序
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DMA1_Stream1_IRQHandler(void)
{
    /* 传输完成中断 */
    if((DMA1->LISR & DMA_FLAG_TCIF1_5) != RESET)
    {

       HC574_TogglePin(GPIO_PIN_23);

        /*
           1、使用此函数要特别注意，第 1 个参数地址要 32 字节对齐，第 2 个参数要是 32 字节的整数倍。
           2、进入传输完成中断，当前 DMA 正在使用缓冲区的前半部分，用户可以操作后半部分。
        */
        SCB_InvalidateDCache_by_Addr((uint32_t *)(&ADCxValues[64]), 128);

        s_DmaFlag = 2;

        /* 清除标志 */
        DMA1->LIFCR = DMA_FLAG_TCIF1_5;
    }

    /* 半传输完成中断 */
    if((DMA1->LISR & DMA_FLAG_HTIF1_5) != RESET)
    {
        /*
           1、使用此函数要特别注意，第 1 个参数地址要 32 字节对齐，第 2 个参数要是 32 字节的整数倍。
           2、进入半传输完成中断，当前 DMA 正在使用缓冲区的后半部分，用户可以操作前半部分。
        */
        SCB_InvalidateDCache_by_Addr((uint32_t *)(&ADCxValues[0]), 128);

        s_DmaFlag = 1;

        /* 清除标志 */
        DMA1->LIFCR = DMA_FLAG_HTIF1_5;
    }

    /* 传输错误中断 */
```

```
    if((DMA1->LISR & DMA_FLAG_TEIF1_5) != RESET)
    {
        /* 清除标志 */
        DMA1->LIFCR = DMA_FLAG_TEIF1_5;
    }

    /* 直接模式错误中断 */
    if((DMA1->LISR & DMA_FLAG_DMEIF1_5) != RESET)
    {
        /* 清除标志 */
        DMA1->LIFCR = DMA_FLAG_DMEIF1_5;
    }
}
```

注释的比较清楚。如果输出的 PWM 频率较高，建议将 DMA 的缓冲区设置的大些，防止 DMA 中断的执行频率较高。

传输完成中断里面有个扩展 IO 翻转函数 HC574_TogglePin(GPIO_PIN_23)，大家可以通过示波器测量开发板上丝印为 X23 的排针，从而方便的查看 DMA 中断速度。

传输完成中断和半传输完成中断里面还有一个变量 s_DmaFlag，当 s_DmaFlag= 1 时表示进 DMA 半传输完成中断，s_DmaFlag = 2 表示进入 DMA 传输完成中断。

45.3.6读取 DMA 双缓冲数据

DMA 双缓冲方式配置一款 RTOS 是最方便的，可以在中断服务程序里面发消息给任务，让数据可以得到及时处理。而裸机方式的话，需要用户实时查询变量，检测到有数据了再进行处理。具体实现代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_GetAdcValues
*    功能说明: 获取 ADC 的数据并打印
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_GetAdcValues(void)
{
    uint32_t values;
    float  temp;

    /* 当前 DMA 操作是后半个缓冲，读取前半个缓冲的前 4 个数值求平均 */
    if(s_DmaFlag == 1)
    {
        DISABLE_INT();
        s_DmaFlag = 0;
        values = (ADCxValues[0] + ADCxValues[1] + ADCxValues[2] + ADCxValues[3])/4;

        ENABLE_INT();
    }
    /* 当前 DMA 操作是后前个缓冲，读取后半个缓冲的前 4 个数值求平均 */
    else if(s_DmaFlag == 2)
    {
```

```
        DISABLE_INT();
        s_DmaFlag = 0;
        values = (ADCxValues[64] + ADCxValues[65] + ADCxValues[66] + ADCxValues[67])/4;
        ENABLE_INT();
    }

    /* 打印读出的串口值 */
    temp = values *3.3 / 65536;

    printf("ADCxValues = %d, %5.3f\r\n", values, temp);
}
```

45.4 ADC 板级支持包（bsp_adc.c）

ADC 驱动文件 bsp_adc.c 提供了如下三个函数：

- TIM1_Config
- bsp_InitADC
- bsp_GetAdcValues

45.4.1 函数 TIM1_Config

函数原型：

```
static void TIM1_Config(void)
```

函数描述：

此函数用于配置 TIM1 工作在 OC 输出比较模式，使用 TIM1 的 CC1 作为 ADC 的触发源。

注意事项：

1. 关于此函数的讲解在本章的 2.1 小节。
2. 函数前面的 static 表示限制在 bsp_adc.c 文件里面调用。

45.4.2 函数 bsp_InitADC

函数原型：

```
void bsp_InitADC(void)
```

函数描述：

此函数用于配置定时器触发 ADC 做 DMA 传输。

注意事项：

1. 关于此函数的讲解在本章 2.2 小节。

使用举例：

作为初始化函数，直接在 bsp.c 文件的 bsp_Init 函数里面调用即可。

45.4.3 函数 bsp_GetAdcValues

函数原型：

```
void bsp_GetAdcValues(void)
```

函数描述：

此函数用于获取 ADC 的转换数据。

注意事项：

1. 关于此函数的讲解在本章 2.4，2.5 和 2.6 小节。

使用举例：

如果是裸机工程：要实时调用此函数读取双缓冲里面的数据。

如果是 RTOS 工程：要在 DMA 的中断服务程序里面给 ADC 任务发消息，让 ADC 任务可以及时读取数据。

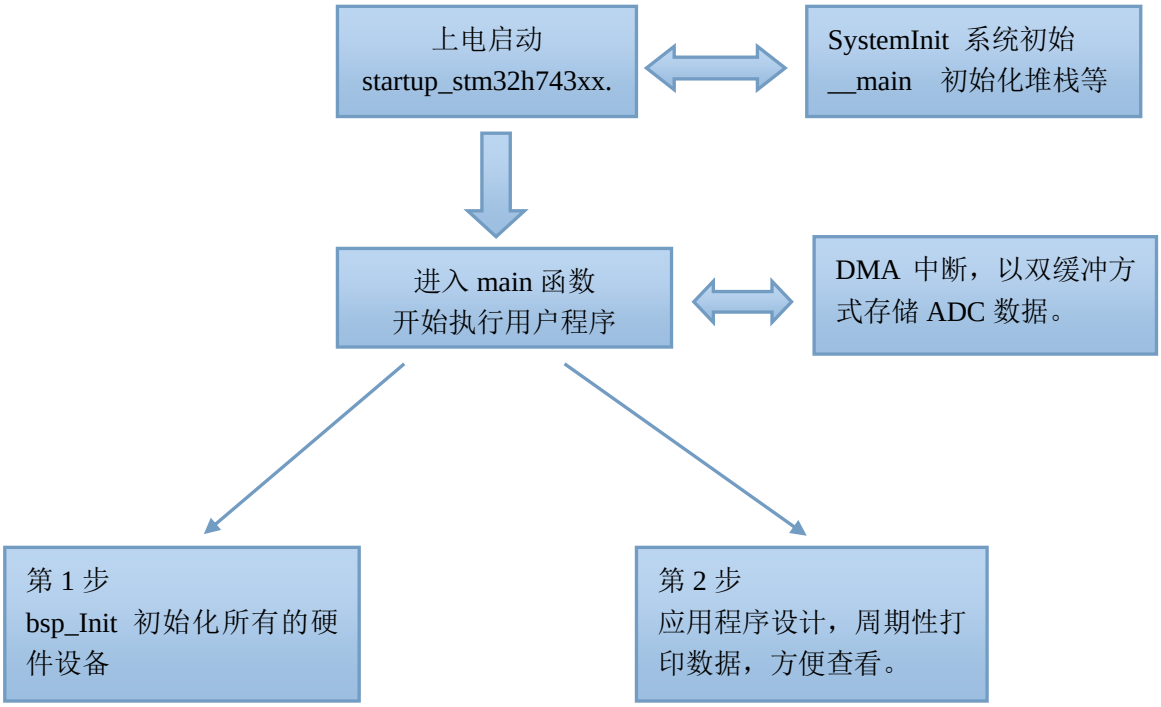
45.5 ADC 驱动移植和使用

ADC 驱动的移植比较方便：

- 第 1 步：复制 bsp_adc.c 和 bsp_adc.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：这几个驱动文件主要用到 HAL 库的 GPIO、TIM，DMA 和 ADC 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 3 步，应用方法看本章节配套例子即可，另外就是根据自己的需要做配置修改。

45.6 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 4 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED，串口和 ADC。
  - 第 2 步，应用程序设计部分，周期性的打印数据，方便查看。
  - 第 3 步，DMA 中断，以双缓冲方式存储 ADC 数据。

45.7 实验例程说明（MDK）

配套例子：

V7-019_ADC 定时器触发+DMA 双缓冲实现

实验目的：

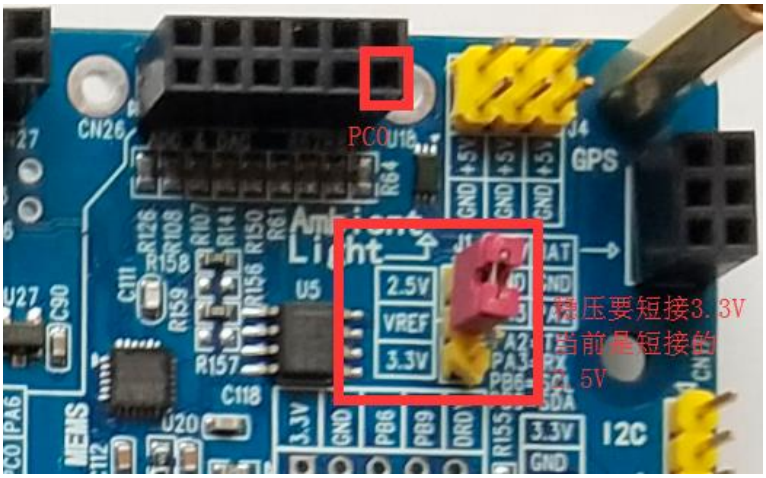
1. 学习 ADC 定时器触发 + DMA 双缓冲的实现。

实验内容：

1. 例子默认用的 AHB 时钟供 ADC 使用，大家可以通过 bsp_adc.c 文件开头宏定义切换到 PLL2 专用时钟。
2. 使用的 TIM1 的 OC1 作为 ADC 的外部触发源，触发速度是 100KHz，即 ADC 的采样率也是 100KHz。
3. 使用 DMA 的半传输完成中断和传输完成中断实现数据的双缓冲更新。
4. 采集引脚使用的 PC0，另外特别注意开发板上的 Vref 稳压基准跳线帽短接的 3.3V。

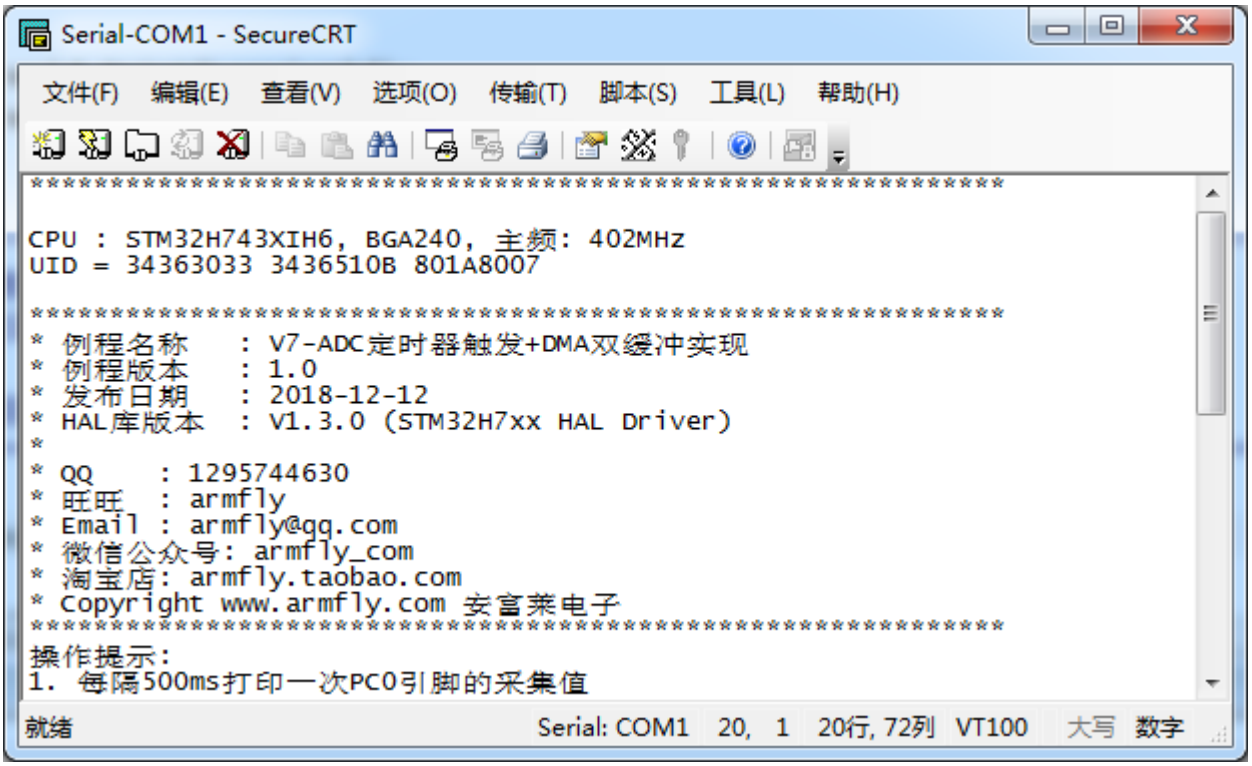
5. 每隔 500ms，串口会打印一次。
6. 板子正常运行时 LED2 闪烁。

PC0 引脚位置（稳压基准要短接 3.3V）：



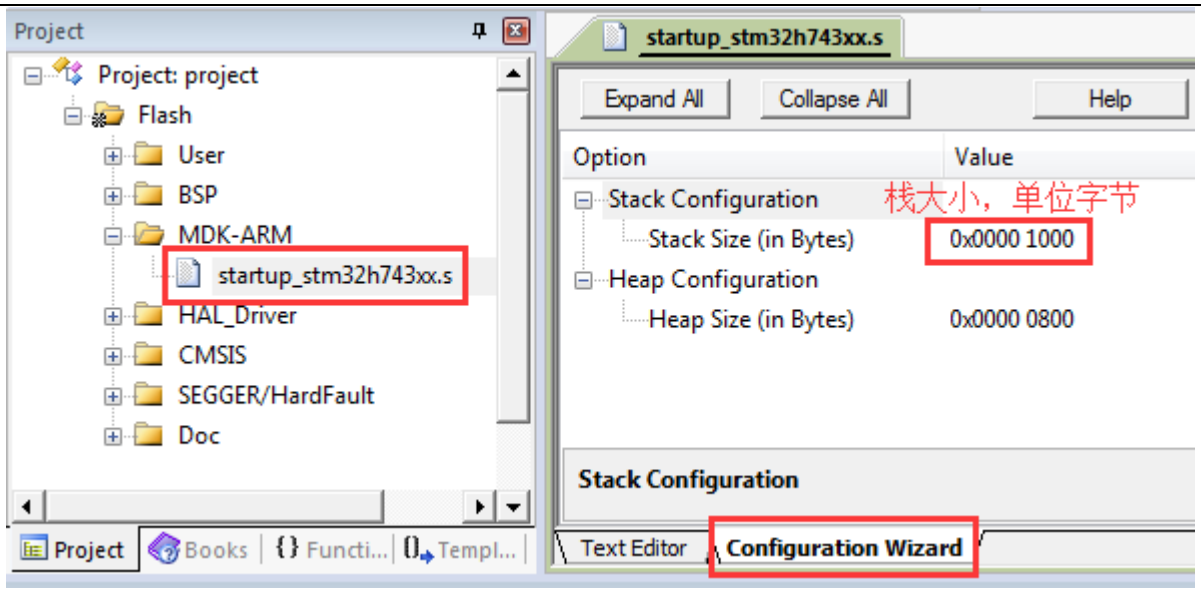
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

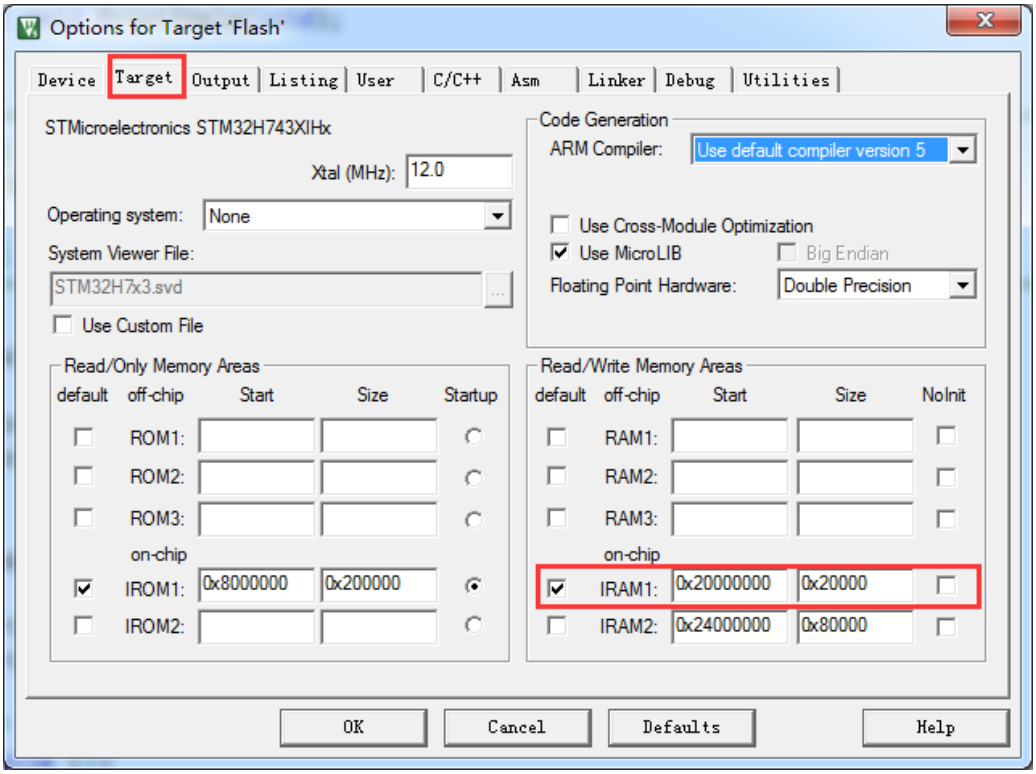


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
    bsp_InitADC();     /* 初始化 ADC */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 D3 域的 SRAM4。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();
```

```
    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SRAM4 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- 每隔 500ms 打印一次 PC0 引脚的采集值。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;  /* 按键代码 */

#if defined ( __CC_ARM )
    TempValues1 = 0; /* 避免 MDK 警告 */
    TempValues2 = 0;
#endif

    bsp_Init();          /* 硬件初始化 */

    PrintfLogo();   /* 打印例程名称和版本等信息 */
    PrintfHelp();   /* 打印操作提示 */

    bsp_StartAutoTimer(0, 500); /* 启动 1 个 500ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 500ms 进来一次 */
            bsp_LedToggle(2);

            /*
               这里仅仅是为了展示方便，500ms 更新一次，如果是实际工程里面应用
               裸机工程：要实时调用下面的函数读取双缓冲里面的数据。
               RTOS 工程：要在 DMA 的中断服务程序里面给 ADC 任务发消息，让 ADC 任务可以及时读取数据。
            */
            bsp_GetAdcValues();
        }
```

```
        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值，无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下 */
                    printf(" K1 键按下 \r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

DMA 中断处理：

```
/*
*********************************************************************************************************
*    函 数 名: DMA1_Stream1_IRQHandler
*    功能说明: DMA1 Stream1 中断服务程序
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DMA1_Stream1_IRQHandler(void)
{
    /* 传输完成中断 */
    if((DMA1->LISR & DMA_FLAG_TCIF1_5) != RESET)
    {

       HC574_TogglePin(GPIO_PIN_23);

        /*
           1、使用此函数要特别注意，第 1 个参数地址要 32 字节对齐，第 2 个参数要是 32 字节的整数倍。
           2、进入传输完成中断，当前 DMA 正在使用缓冲区的前半部分，用户可以操作后半部分。
        */
        SCB_InvalidateDCache_by_Addr((uint32_t *)(&ADCxValues[64]), 128);

        s_DmaFlag = 2;

        /* 清除标志 */
        DMA1->LIFCR = DMA_FLAG_TCIF1_5;
    }

    /* 半传输完成中断 */
    if((DMA1->LISR & DMA_FLAG_HTIF1_5) != RESET)
    {
        /*
           1、使用此函数要特别注意，第 1 个参数地址要 32 字节对齐，第 2 个参数要是 32 字节的整数倍。
           2、进入半传输完成中断，当前 DMA 正在使用缓冲区的后半部分，用户可以操作前半部分。
        */
        SCB_InvalidateDCache_by_Addr((uint32_t *)(&ADCxValues[0]), 128);

        s_DmaFlag = 1;
```

```
        /* 清除标志 */
        DMA1->LIFCR = DMA_FLAG_HTIF1_5;
    }

    /* 传输错误中断 */
    if((DMA1->LISR & DMA_FLAG_TEIF1_5) != RESET)
    {
        /* 清除标志 */
        DMA1->LIFCR = DMA_FLAG_TEIF1_5;
    }

    /* 直接模式错误中断 */
    if((DMA1->LISR & DMA_FLAG_DMEIF1_5) != RESET)
    {
        /* 清除标志 */
        DMA1->LIFCR = DMA_FLAG_DMEIF1_5;
    }
}
```

45.8 实验例程说明（IAR）

配套例子：

V7-019_ADC 定时器触发+DMA 双缓冲实现

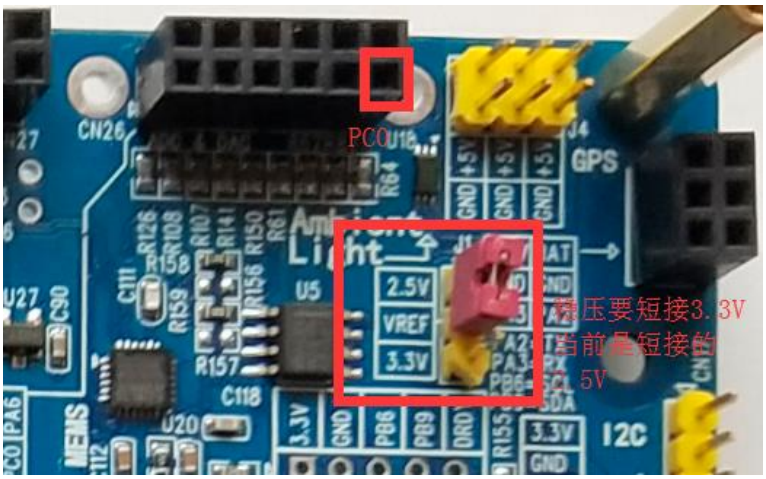
实验目的：

1. 学习 ADC 定时器触发 + DMA 双缓冲的实现。

实验内容：

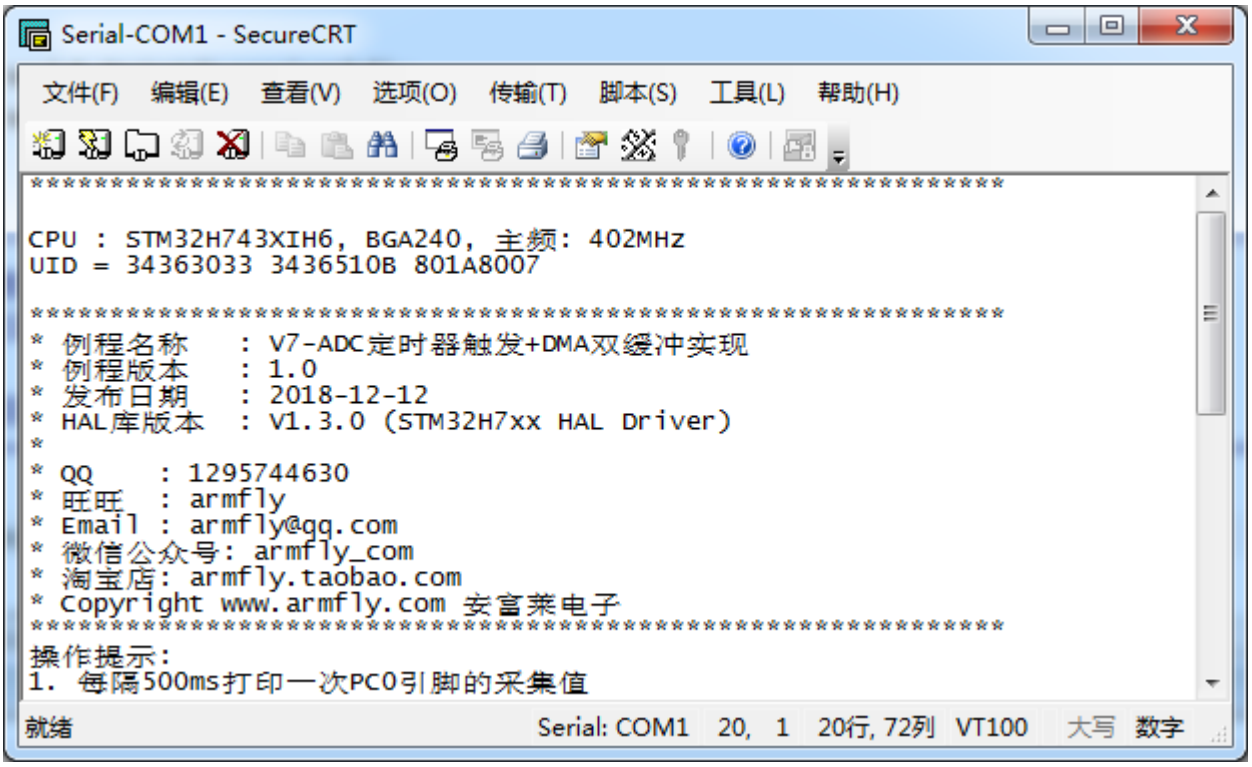
1. 例子默认用的 AHB 时钟供 ADC 使用，大家可以通过 bsp_adc.c 文件开头宏定义切换到 PLL2 专用时钟。
2. 使用的 TIM1 的 OC1 作为 ADC 的外部触发源，触发速度是 100KHz，即 ADC 的采样率也是 100KHz。
3. 使用 DMA 的半传输完成中断和传输完成中断实现数据的双缓冲更新。
4. 采集引脚使用的 PC0，另外特别注意开发板上的 Vref 稳压基准跳线帽短接的 3.3V。
5. 每隔 500ms，串口会打印一次。
6. 板子正常运行时 LED2 闪烁。

PC0 引脚位置（稳压基准要短接 3.3V）：



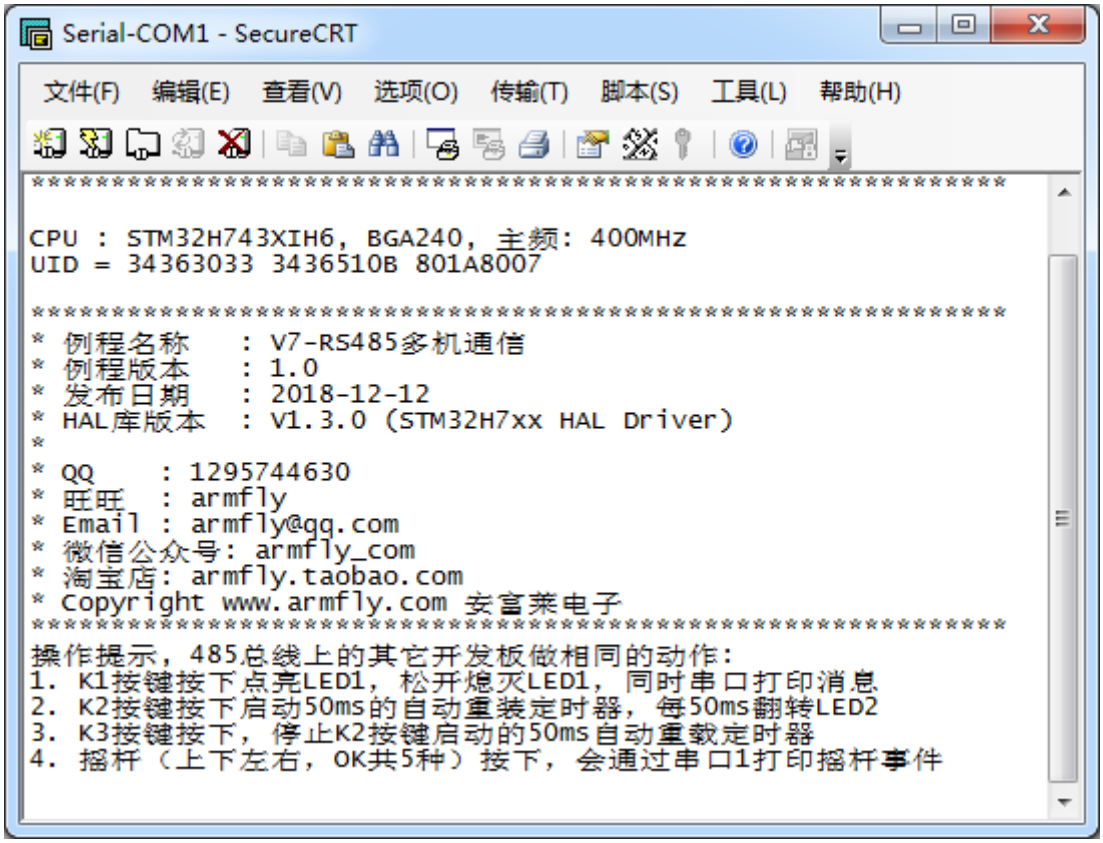
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1



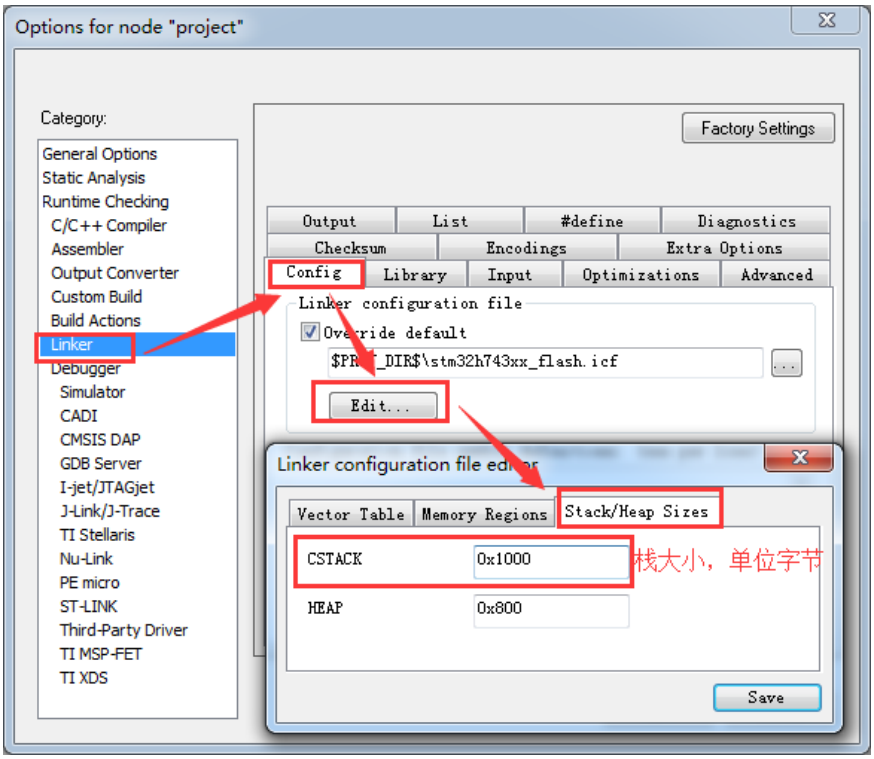
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

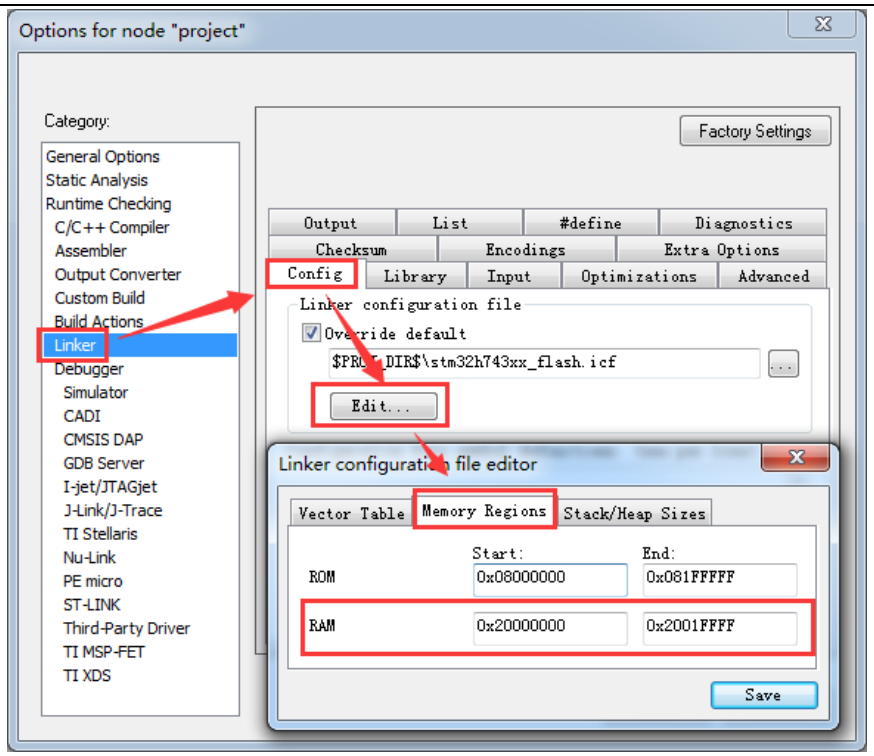


程序设计：

◆ 系统栈大小分配：



◆ RAM 空间用的 DTCM：



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
    bsp_InitADC();     /* 初始化 ADC */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 D3 域的 SRAM4。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
```

```
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM4 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- 每隔 500ms 打印一次 PC0 引脚的采集值。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
```

```
*/
int main(void)
{
    uint8_t ucKeyCode;  /* 按键代码 */

#if defined ( __CC_ARM )
    TempValues1 = 0; /* 避免 MDK 警告 */
    TempValues2 = 0;
#endif

    bsp_Init();          /* 硬件初始化 */

    PrintfLogo();   /* 打印例程名称和版本等信息 */
    PrintfHelp();   /* 打印操作提示 */

    bsp_StartAutoTimer(0, 500); /* 启动 1 个 500ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 500ms 进来一次 */
            bsp_LedToggle(2);

            /*
               这里仅仅是为了展示方便，500ms 更新一次，如果是实际工程里面应用
               裸机工程：要实时调用下面的函数读取双缓冲里面的数据。
               RTOS 工程：要在 DMA 的中断服务程序里面给 ADC 任务发消息，让 ADC 任务可以及时读取数据。
            */
            bsp_GetAdcValues();
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值，无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下 */
                    printf("K1 键按下 \r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

DMA 中断处理：

```
/*
*********************************************************************************************************
```

```
*    函 数 名: DMA1_Stream1_IRQHandler
*    功能说明: DMA1 Stream1 中断服务程序
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DMA1_Stream1_IRQHandler(void)
{
    /* 传输完成中断 */
    if((DMA1->LISR & DMA_FLAG_TCIF1_5) != RESET)
    {

       HC574_TogglePin(GPIO_PIN_23);

        /*
           1、使用此函数要特别注意，第 1 个参数地址要 32 字节对齐，第 2 个参数要是 32 字节的整数倍。
           2、进入传输完成中断，当前 DMA 正在使用缓冲区的前半部分，用户可以操作后半部分。
        */
        SCB_InvalidateDCache_by_Addr((uint32_t *)(&ADCxValues[64]), 128);

        s_DmaFlag = 2;

        /* 清除标志 */
        DMA1->LIFCR = DMA_FLAG_TCIF1_5;
    }

    /* 半传输完成中断 */
    if((DMA1->LISR & DMA_FLAG_HTIF1_5) != RESET)
    {
        /*
           1、使用此函数要特别注意，第 1 个参数地址要 32 字节对齐，第 2 个参数要是 32 字节的整数倍。
           2、进入半传输完成中断，当前 DMA 正在使用缓冲区的后半部分，用户可以操作前半部分。
        */
        SCB_InvalidateDCache_by_Addr((uint32_t *)(&ADCxValues[0]), 128);

        s_DmaFlag = 1;

        /* 清除标志 */
        DMA1->LIFCR = DMA_FLAG_HTIF1_5;
    }

    /* 传输错误中断 */
    if((DMA1->LISR & DMA_FLAG_TEIF1_5) != RESET)
    {
        /* 清除标志 */
        DMA1->LIFCR = DMA_FLAG_TEIF1_5;
    }

    /* 直接模式错误中断 */
    if((DMA1->LISR & DMA_FLAG_DMEIF1_5) != RESET)
    {
        /* 清除标志 */
        DMA1->LIFCR = DMA_FLAG_DMEIF1_5;
    }
}
```

45.9 总结

本章节就为大家讲解这么多，DMA 双缓冲方式记录 ADC 数据还是非常实用的，建议初学者熟练应用这种方案。

第46章 STM32H7 的 ADC 应用之 DMA 方式多通道采样

本章教程为大家讲解 ADC+DMA 方式的多通道数据采集，实际项目中有一定的使用价值，使用一路 ADC 就可以采集多个通道的数据。

46.1 初学者重要提示
46.2 ADC 稳压基准硬件设计
46.3 ADC 驱动设计
46.4 ADC 板级支持包（bsp_adc.c）
46.5 ADC 驱动移植和使用
46.6 实验例程设计框架
46.7 实验例程说明（MDK）
46.8 实验例程说明（IAR）
46.9 总结

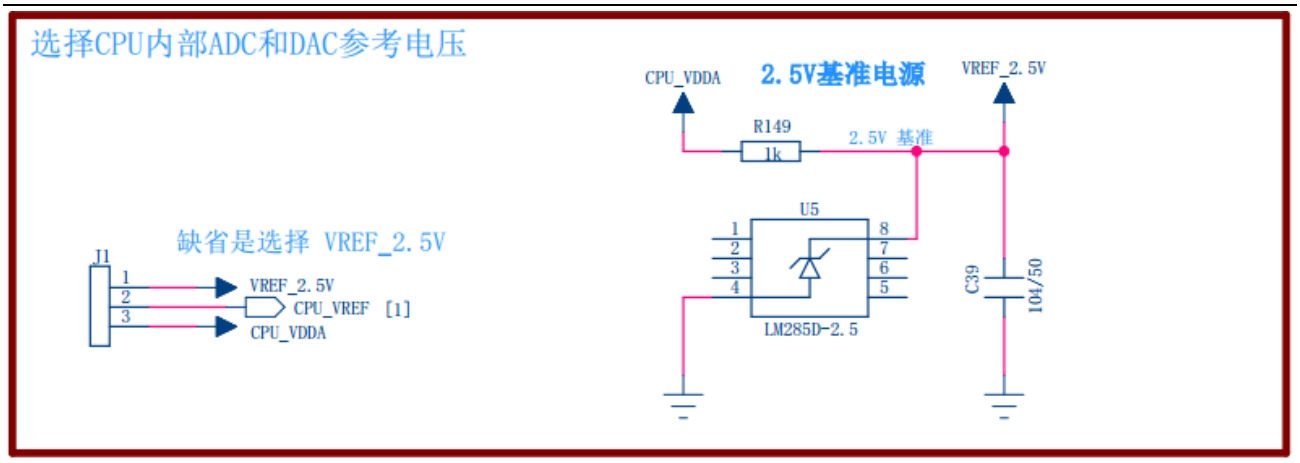
46.1 初学者重要提示

- 学习本章节前，务必优先学习第 44 章，需要对 ADC 的基础知识和 HAL 库的几个常用 API 有个认识。
- 开发板右上角有个跳线帽，可以让 ADC 的稳压基准接 3.3V 或者 2.5V，本章例子是接到 3.3V。
- STM32H7 的 ADC 支持偏移校准和线性度校准。如果使用线性度校准的话，特别要注意此贴的问题：http://www.armbbs.cn/forum.php?mod=viewthread&tid=91436 。
- ADC 的专业术语诠释文档，推荐大家看看：http://www.armbbs.cn/forum.php?mod=viewthread&tid=89414 。
- STM32H7 的 ADC 多通道并不是同步采样的，本质上是通过内部的多路选择器不断切换实现的，一个采集完毕了才会采集另一个。

46.2 ADC 稳压基准硬件设计

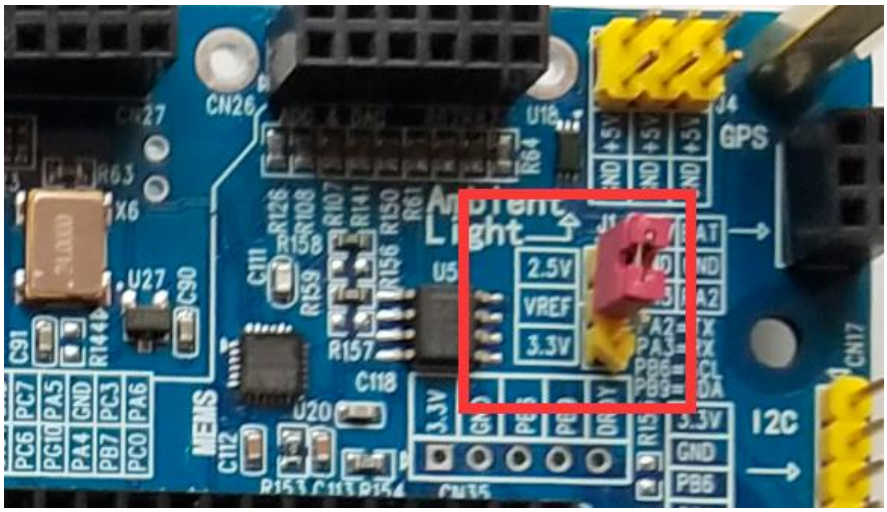
注：学习前务必优先看第 14 章的 2.1 小节，对电源供电框架有个了解。

ADC 要采集的准确，就需要有一个稳定的稳压基准源，V7 开发板使用的 LM285D-2.5，即 2.5V 的基准源。硬件设计如下：



关于这个原理图要注意以下问题：

LM285D-2.5 输出的是 2.5V 的稳压基准，原理图这里做了一个特别的处理，同时接了一个上拉电阻到 VDDA（3.3V），然后用户可以使用开发板右上角的跳线帽设置 Vref 选择 3.3V 稳压还是 2.5V 稳压。



下面再来了解下 LM285 的电气特性：

PARAMETER		TEST CONDITIONS		MIN	TYP	MAX	UNIT
LM285-2.5							
V_Z	Reference voltage	I_Z = 20 µA to 20 mA		2.462	2.5	2.538	V
$^{\alpha}V_Z$	Average temperature coefficient of reference voltage(1)	I_Z = 20 µA to 20 mA	Full range(2)		±20		ppm/°C
ΔV_Z	Change in reference voltage with current	I_Z = 20 µA to 1 mA	T_A = 25°C			1	mV
			Full range			1.5	
		I_Z = 1 mA to 20 mA	T_A = 25°C			10	
			Full range			30	
$\Delta V_Z/\Delta t$	Long-term change in reference voltage	I_Z = 100 µA			±20		ppm/khr
$I_{Z(MIN)}$	Minimum reference current	Full range			8	20	µA
z_z	Reference impedance	I_Z = 100 µA	T_A = 25°C		0.2	0.6	Ω
			Full range			1.5	
V_n	Broadband noise voltage	I_Z = 100 µA, f = 10 Hz to 10 kHz			120		µV

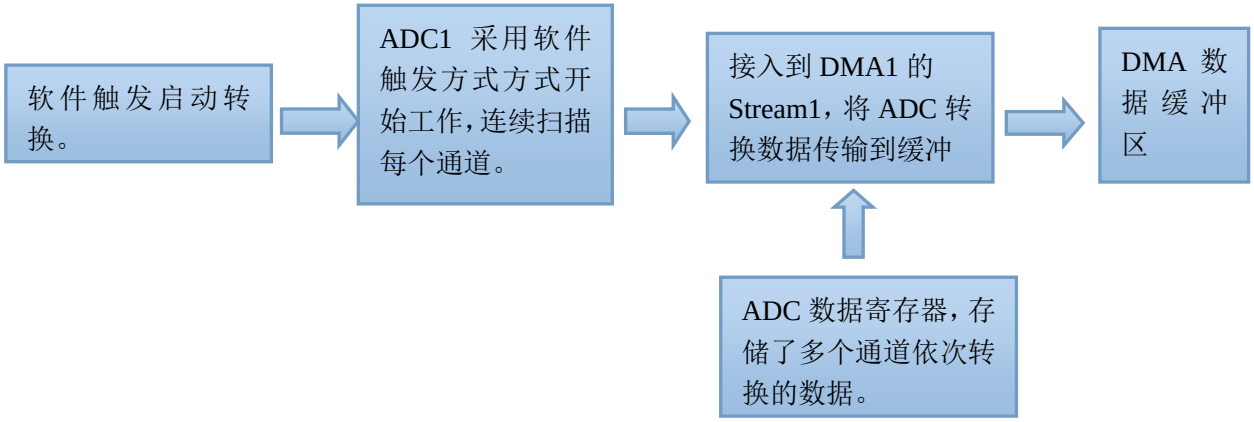
通过这个表，我们要了解以下几点知识：

- LM285 的典型值是 2.5V，支持的最小值 2.462V，最大值 2.538V。工作电流是 20uA 到 20mA，温飘是±20ppm/℃
- Iz 是 Reference current 参考电流的意思：
  - 参考电流是 20uA 到 1mA，温度 25℃，参考电压最大变化 1mV。
  - 参考电流是 20uA 到 1mA，全范围温度（-40°C to 85°C），参考电压最大变化 1.5mV。
  - 参考电流是 1mA 到 20mA，温度 25℃，参考电压最大变化 10mV。
  - 参考电流是 1mA 到 20mA，全范围温度（-40°C to 85°C），参考电压最大变化 30mV。

那么问题来了，V7 开发板上 LM285 的参考电流是多少？ 简单计算就是：
（VDDA – 2.5V） / 1K = （3.3 – 2.5V） / 1K = 0.8mA。

46.3 ADC 驱动设计

ADC 做 DMA 数据传输的实现思路框图如下：



下面将程序设计中的相关问题逐一为大家做个说明。

46.3.1 ADC 软件触发

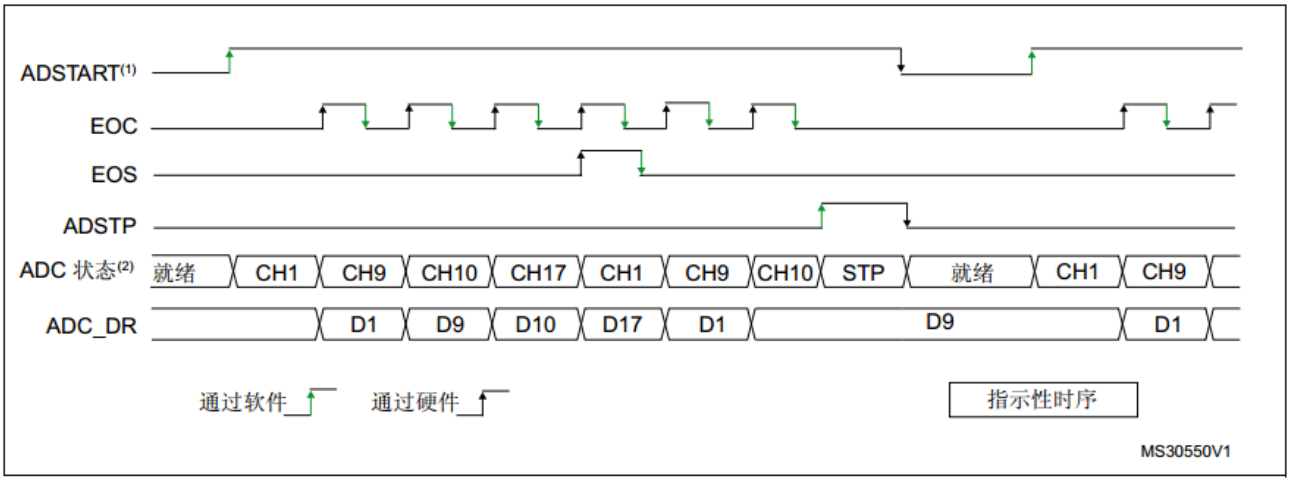
ADC 转换既可以选择外部触发也可以选择软件触发。我们这里选择的是软件触发方式的多通道转换，即连续转换序列，软件触发。对应的时序如下（在第 44 章的 2.7 小节有详细讲解软件触发和硬件触发的时序）：。

ADSTART 表示软件启动转换。

ADSTP 表示停止转换。

EOC 表示一个通道转换结束。

EOS 表示所有通道转换结束。



关于这个时序图的解读：

- 配置为连续转换的话，软件启动 ADSTART 会开启所有通道转换，全部转换完毕后，继续进行下一轮转换。调用了停止转换 ADSTP 后，会停止转换。
- 每个通过转换完毕有个 EOC 标志，所有通道转换完毕有个 EOS 标志。

46.3.2 ADC 时钟源选择

根据第 44 章 2.2 小节的讲解，我们知道 ADC 有两种时钟源可供选择，可以使用来自 AHB 总线的系统时钟，也可以使用 PLL2，PLL3，HSE，HSI 或者 CSI 时钟。

如果采用 AHB 时钟，不需要做专门的配置，而采用 PLL2，PLL3 时钟需要特别的配置，下面是使用 AHB 或者 PLL2 时钟的配置。

◆ 通过宏定义设置选择的时钟源

使用哪个时钟源，将另一个注释掉即可：

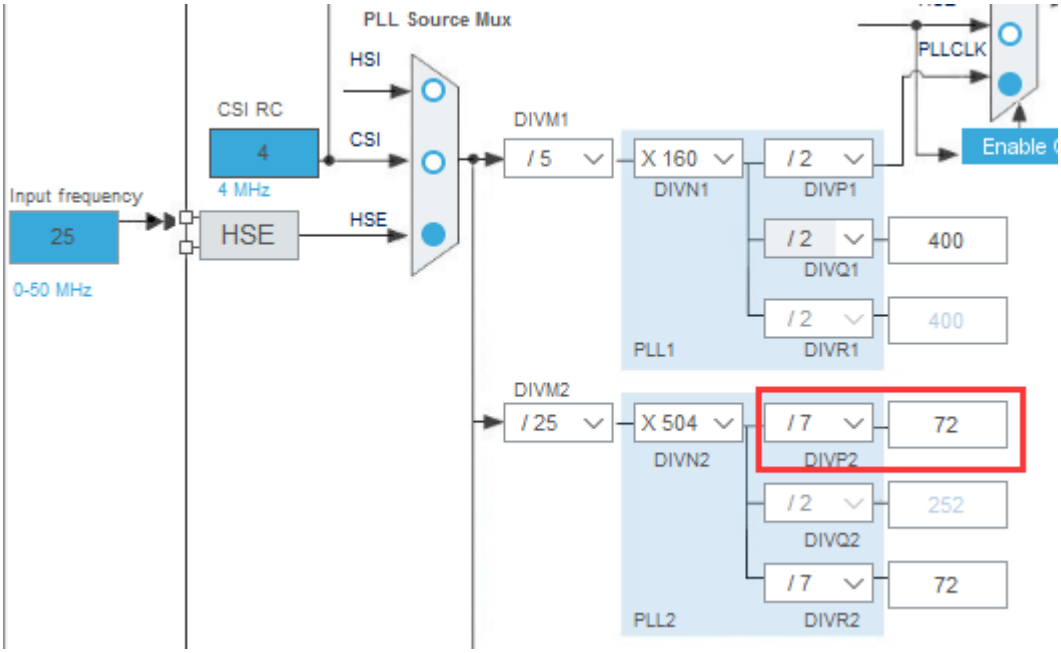
```
/* 选择 ADC 的时钟源 */
#define ADC_CLOCK_SOURCE_AHB     /* 选择 AHB 时钟源 */
//#define ADC_CLOCK_SOURCE_PLL   /* 选择 PLL 时钟源 */
```

◆ PLL2 或者 AHB 时钟源配置

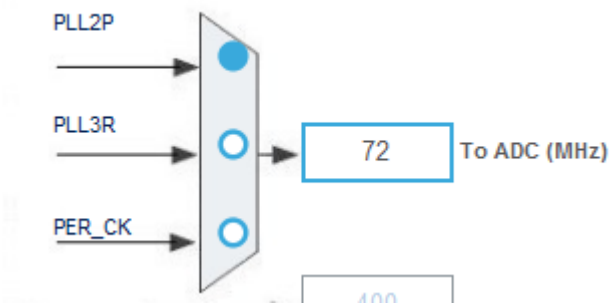
```
#if defined (ADC_CLOCK_SOURCE_PLL)
    /* 配置 PLL2 时钟为的 72MHz，方便分频产生 ADC 最高时钟 36MHz */
    RCC_PeriphCLKInitTypeDef PeriphClkInitStruct = {0};
    PeriphClkInitStruct.PeriphClockSelection = RCC_PERIPHCLK_ADC;
    PeriphClkInitStruct.PLL2.PLL2M = 25;
    PeriphClkInitStruct.PLL2.PLL2N = 504;
    PeriphClkInitStruct.PLL2.PLL2P = 7;
    PeriphClkInitStruct.PLL2.PLL2Q = 7;
    PeriphClkInitStruct.PLL2.PLL2R = 7;
    PeriphClkInitStruct.PLL2.PLL2RGE = RCC_PLL2VCIRANGE_0;
    PeriphClkInitStruct.PLL2.PLL2VCOSEL = RCC_PLL2VCOWIDE;
    PeriphClkInitStruct.PLL2.PLL2FRACN = 0;
    PeriphClkInitStruct.AdcClockSelection = RCC_ADCCLKSOURCE_PLL2;
    if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInitStruct) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
```

```
    }
#elif defined (ADC_CLOCK_SOURCE_AHB)

  /* 使用 AHB 时钟的话，无需配置，默认选择*/

#endif
```

对于 PLL2 的时钟输出，直接使用 STM32CubeMX 里面的时钟树配置即可，效果如下：

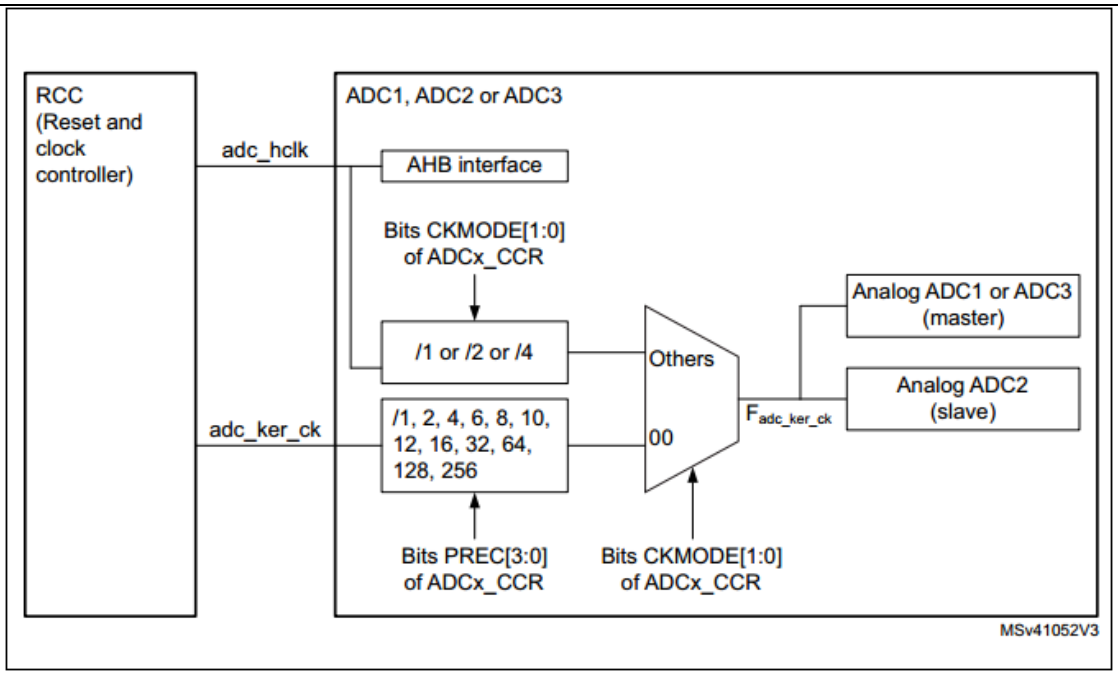


选择 PLL2P 输出作为 ADC 时钟源：



◆ ADC 分频设置

无论是使用 AHB 时钟还是 PLL2 时钟都支持分频设置：



AHB 支持下面三种分频设置：

```
#define ADC_CLOCK_SYNC_PCLK_DIV1    ((uint32_t)ADC_CCR_CKMODE_0)
#define ADC_CLOCK_SYNC_PCLK_DIV2    ((uint32_t)ADC_CCR_CKMODE_1)
#define ADC_CLOCK_SYNC_PCLK_DIV4    ((uint32_t)ADC_CCR_CKMODE)

#define ADC_CLOCKPRESCALER_PCLK_DIV1    ADC_CLOCK_SYNC_PCLK_DIV1    /* 这三个仅仅是为了兼容，已经不推荐使用 */
#define ADC_CLOCKPRESCALER_PCLK_DIV2    ADC_CLOCK_SYNC_PCLK_DIV2
#define ADC_CLOCKPRESCALER_PCLK_DIV4    ADC_CLOCK_SYNC_PCLK_DIV4
```

PLL2 支持下面几种分频设置：

```
#define ADC_CLOCK_ASYNC_DIV1        ((uint32_t)0x00000000)
#define ADC_CLOCK_ASYNC_DIV2        ((uint32_t)ADC_CCR_PRESC_0)
#define ADC_CLOCK_ASYNC_DIV4        ((uint32_t)ADC_CCR_PRESC_1)
#define ADC_CLOCK_ASYNC_DIV6        ((uint32_t)(ADC_CCR_PRESC_1|ADC_CCR_PRESC_0))
#define ADC_CLOCK_ASYNC_DIV8        ((uint32_t)(ADC_CCR_PRESC_2))
#define ADC_CLOCK_ASYNC_DIV10       ((uint32_t)(ADC_CCR_PRESC_2|ADC_CCR_PRESC_0))
#define ADC_CLOCK_ASYNC_DIV12       ((uint32_t)(ADC_CCR_PRESC_2|ADC_CCR_PRESC_1))
#define ADC_CLOCK_ASYNC_DIV16       ((uint32_t)(ADC_CCR_PRESC_2|ADC_CCR_PRESC_1|ADC_CCR_PRESC_0))
#define ADC_CLOCK_ASYNC_DIV32       ((uint32_t)(ADC_CCR_PRESC_3))
#define ADC_CLOCK_ASYNC_DIV64       ((uint32_t)(ADC_CCR_PRESC_3|ADC_CCR_PRESC_0))
#define ADC_CLOCK_ASYNC_DIV128      ((uint32_t)(ADC_CCR_PRESC_3|ADC_CCR_PRESC_1))
#define ADC_CLOCK_ASYNC_DIV256      ((uint32_t)(ADC_CCR_PRESC_3|ADC_CCR_PRESC_1|ADC_CCR_PRESC_0))
```

有了这些认识后再看实际的分频配置就好理解了：

```
#if defined (ADC_CLOCK_SOURCE_PLL)
/* 采用 PLL 异步时钟，2 分频，即 72MHz/2 = 36MHz */
    AdcHandle.Init.ClockPrescaler        = ADC_CLOCK_ASYNC_DIV2;
/* 采用 AHB 同步时钟，4 分频，即 200MHz/4 = 50MHz */
#elif defined (ADC_CLOCK_SOURCE_AHB)
    AdcHandle.Init.ClockPrescaler        = ADC_CLOCK_SYNC_PCLK_DIV4;
#endif
```

46.3.3 ADC 的 DMA 配置

由于函数 HAL_ADC_Start_DMA 封装的 DMA 传输函数是 HAL_DMA_Start_IT。而我们这里仅需要用到 DMA 传输，而用不到中断，所以不开启对应的 NVIC 即可，这里使用的是 DMA1_Stream1，测量了 PC0，Vbat/4，VrefInt 和温度四个通道。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitADC
*    功能说明: 初始化 ADC，采用 DMA 方式进行多通道采样，采集了 PC0, Vbat/4, VrefInt 和温度
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitADC(void)
{
    ADC_HandleTypeDef    AdcHandle = {0};
    DMA_HandleTypeDef    DMA_Handle = {0};
    ADC_ChannelConfTypeDef    sConfig = {0};
    GPIO_InitTypeDef          GPIO_InitStruct;

  /* ## - 1 - 配置 ADC 采样的时钟 ######################################### */
#if defined (ADC_CLOCK_SOURCE_PLL)
    /* 配置 PLL2 时钟为的 72MHz，方便分频产生 ADC 最高时钟 36MHz */
    RCC_PeriphCLKInitTypeDef PeriphClkInitStruct = {0};
    PeriphClkInitStruct.PeriphClockSelection = RCC_PERIPHCLK_ADC;
    PeriphClkInitStruct.PLL2.PLL2M = 25;
    PeriphClkInitStruct.PLL2.PLL2N = 504;
    PeriphClkInitStruct.PLL2.PLL2P = 7;
    PeriphClkInitStruct.PLL2.PLL2Q = 7;
    PeriphClkInitStruct.PLL2.PLL2R = 7;
    PeriphClkInitStruct.PLL2.PLL2RGE = RCC_PLL2VCIRANGE_0;
    PeriphClkInitStruct.PLL2.PLL2VCOSEL = RCC_PLL2VCOWIDE;
    PeriphClkInitStruct.PLL2.PLL2FRACN = 0;
    PeriphClkInitStruct.AdcClockSelection = RCC_ADCCLKSOURCE_PLL2;
    if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInitStruct) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
#elif defined (ADC_CLOCK_SOURCE_AHB)

  /* 使用 AHB 时钟的话，无需配置，默认选择*/

#endif

    /* ## - 2 - 配置 ADC 采样使用的时钟 ######################################### */
    __HAL_RCC_GPIOC_CLK_ENABLE();

    GPIO_InitStruct.Pin = GPIO_PIN_0;
    GPIO_InitStruct.Mode = GPIO_MODE_ANALOG;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

    /* ## - 3 - 配置 ADC 采样使用的时钟 ######################################### */
    __HAL_RCC_DMA1_CLK_ENABLE();
```

```
50.       DMA_Handle.Instance                 = DMA1_Stream1;             /* 使用的 DMA1 Stream1 */
51.       DMA_Handle.Init.Request             = DMA_REQUEST_ADC3;          /* 请求类型采用 DMA_REQUEST_ADC3 */
52.       DMA_Handle.Init.Direction           = DMA_PERIPH_TO_MEMORY;     /* 传输方向是从外设到存储器*/
53.       DMA_Handle.Init.PeriphInc           = DMA_PINC_DISABLE;         /* 外设地址自增禁止 */
54.       DMA_Handle.Init.MemInc              = DMA_MINC_ENABLE;          /* 存储器地址自增使能 */
55.       DMA_Handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD; /* 外设数据位宽选择半字，即 16bit */
56.       DMA_Handle.Init.MemDataAlignment    =DMA_MDATAALIGN_HALFWORD; /* 存储器数据位宽选择半字,即 16bit */
57.       DMA_Handle.Init.Mode                = DMA_CIRCULAR;             /* 循环模式 */
58.       DMA_Handle.Init.Priority            = DMA_PRIORITY_LOW;         /* 优先级低 */
59.       DMA_Handle.Init.FIFOMode            = DMA_FIFOMODE_DISABLE;     /* 禁止 FIFO*/
60.       DMA_Handle.Init.FIFOThreshold = DMA_FIFO_THRESHOLD_FULL; /* 禁止 FIFO 此位不起作用，用于设置阀值 */
61.       DMA_Handle.Init.MemBurst    = DMA_MBURST_SINGLE;       /* 禁止 FIFO 此位不起作用，用于存储器突发 */
62.       DMA_Handle.Init.PeriphBurst = DMA_PBURST_SINGLE;      /* 禁止 FIFO 此位不起作用，用于外设突发 */
63.
64.       /* 初始化 DMA */
65.       if(HAL_DMA_Init(&DMA_Handle) != HAL_OK)
66.       {
67.           Error_Handler(__FILE__, __LINE__);
68.       }
69.
70.       /* 关联 ADC 句柄和 DMA 句柄 */
71.       __HAL_LINKDMA(&AdcHandle, DMA_Handle, DMA_Handle);
72.
73.
74.       /* ## - 4 - 配置 ADC ##################################################################### */
75.       __HAL_RCC_ADC3_CLK_ENABLE();
76.       AdcHandle.Instance = ADC3;
77.
78.  #if defined (ADC_CLOCK_SOURCE_PLL)
79.       AdcHandle.Init.ClockPrescaler    = ADC_CLOCK_ASYNC_DIV8;    /* 采用 PLL 异步时钟，8 分频，即 72MHz/8
80.                                                                      = 36MHz */
81.  #elif defined (ADC_CLOCK_SOURCE_AHB)
82.       AdcHandle.Init.ClockPrescaler    =ADC_CLOCK_SYNC_PCLK_DIV4; /* 采用 AHB 同步时钟，4 分频，即 200MHz/4
83.                                                                      = 50MHz */
84.  #endif
85.
86.       AdcHandle.Init.Resolution            = ADC_RESOLUTION_16B;    /* 16 位分辨率 */
87.       AdcHandle.Init.ScanConvMode          = ADC_SCAN_ENABLE;       /* 禁止扫描，因为仅开了一个通道 */
88.       AdcHandle.Init.EOCSelection          = ADC_EOC_SINGLE_CONV;   /* EOC 转换结束标志 */
89.       AdcHandle.Init.LowPowerAutoWait      = DISABLE;               /* 禁止低功耗自动延迟特性 */
90.       AdcHandle.Init.ContinuousConvMode    = ENABLE;                /* 禁止自动转换，采用的软件触发 */
91.       AdcHandle.Init.NbrOfConversion       = 4;                     /* 使用了 4 个转换通道 */
92.       AdcHandle.Init.DiscontinuousConvMode = DISABLE;               /* 禁止不连续模式 */
93.       AdcHandle.Init.NbrOfDiscConversion   = 1;    /* 禁止不连续模式后，此参数忽略，此位是用来配置不连续
94.                                                       子组中通道数 */
95.
96.       AdcHandle.Init.ExternalTrigConv      = ADC_SOFTWARE_START;              /* 采用软件触发 */
97.       AdcHandle.Init.ExternalTrigConvEdge  = ADC_EXTERNALTRIGCONVEDGE_RISING; /* 软件触发，此位忽略 */
98.       AdcHandle.Init.ConversionDataManagement = ADC_CONVERSIONDATA_DMA_CIRCULAR; /* DMA 循环模式接收*/
99.       AdcHandle.Init.BoostMode  = DISABLE;                /* ADC 时钟低于 20MHz 的话，可以禁止 boost */
100.      AdcHandle.Init.Overrun = ADC_OVR_DATA_OVERWRITTEN;  /* ADC 转换溢出的话，覆盖 ADC 的数据寄存器 */
101.      AdcHandle.Init.OversamplingMode      = DISABLE;     /* 禁止过采样 */
102.
103.     /* 初始化 ADC */
104.      if (HAL_ADC_Init(&AdcHandle) != HAL_OK)
105.      {
106.          Error_Handler(__FILE__, __LINE__);
107.      }
```

```
108.
109.
110.      /* 校准 ADC，采用偏移校准 */
111.      if (HAL_ADCEx_Calibration_Start(&AdcHandle, ADC_CALIB_OFFSET, ADC_SINGLE_ENDED) != HAL_OK)
112.      {
113.          Error_Handler(__FILE__, __LINE__);
114.      }
115.
116.      /* 配置 ADC 通道，序列 1，采样 PC0 引脚 */
117.      /*
118.          采用 PLL2 时钟的话，ADCCLK = 72MHz / 8 = 9MHz
119.          ADC 采样速度，即转换时间 = 采样时间 + 逐次逼近时间
120.                                   = 810.5 + 8.5(16bit)
121.                                   = 820 个 ADC 时钟周期
122.          那么转换速度就是 9MHz / 820 = 10975Hz
123.      */
124.      sConfig.Channel      = ADC_CHANNEL_10;              /* 配置使用的 ADC 通道 */
125.      sConfig.Rank         = ADC_REGULAR_RANK_1;          /* 采样序列里的第 1 个 */
126.      sConfig.SamplingTime = ADC_SAMPLETIME_810CYCLES_5;  /* 采样周期 */
127.      sConfig.SingleDiff   = ADC_SINGLE_ENDED;            /* 单端输入 */
128.      sConfig.OffsetNumber = ADC_OFFSET_NONE;             /* 无偏移 */
129.      sConfig.Offset = 0;                                 /* 无偏移的情况下，此参数忽略 */
130.      sConfig.OffsetRightShift       = DISABLE;           /* 禁止右移 */
131.      sConfig.OffsetSignedSaturation = DISABLE;           /* 禁止有符号饱和 */
132.
133.      if (HAL_ADC_ConfigChannel(&AdcHandle, &sConfig) != HAL_OK)
134.      {
135.          Error_Handler(__FILE__, __LINE__);
136.      }
137.
138.      /* 配置 ADC 通道，序列 2，采样 Vbat/4 */
139.      sConfig.Channel      = ADC_CHANNEL_VBAT_DIV4;       /* 配置使用的 ADC 通道 */
140.      sConfig.Rank         = ADC_REGULAR_RANK_2;          /* 采样序列里的第 1 个 */
141.      sConfig.SamplingTime = ADC_SAMPLETIME_810CYCLES_5;  /* 采样周期 */
142.      sConfig.SingleDiff   = ADC_SINGLE_ENDED;            /* 单端输入 */
143.      sConfig.OffsetNumber = ADC_OFFSET_NONE;             /* 无偏移 */
144.      sConfig.Offset = 0;                                 /* 无偏移的情况下，此参数忽略 */
145.      sConfig.OffsetRightShift       = DISABLE;           /* 禁止右移 */
146.      sConfig.OffsetSignedSaturation = DISABLE;           /* 禁止有符号饱和 */
147.
148.      if (HAL_ADC_ConfigChannel(&AdcHandle, &sConfig) != HAL_OK)
149.      {
150.          Error_Handler(__FILE__, __LINE__);
151.      }
152.
153.      /* 配置 ADC 通道，序列 3，采样 VrefInt */
154.      sConfig.Channel      = ADC_CHANNEL_VREFINT;         /* 配置使用的 ADC 通道 */
155.      sConfig.Rank         = ADC_REGULAR_RANK_3;          /* 采样序列里的第 1 个 */
156.      sConfig.SamplingTime = ADC_SAMPLETIME_810CYCLES_5;  /* 采样周期 */
157.      sConfig.SingleDiff   = ADC_SINGLE_ENDED;            /* 单端输入 */
158.      sConfig.OffsetNumber = ADC_OFFSET_NONE;             /* 无偏移 */
159.      sConfig.Offset = 0;                                 /* 无偏移的情况下，此参数忽略 */
160.      sConfig.OffsetRightShift       = DISABLE;           /* 禁止右移 */
161.      sConfig.OffsetSignedSaturation = DISABLE;           /* 禁止有符号饱和 */
162.
163.      if (HAL_ADC_ConfigChannel(&AdcHandle, &sConfig) != HAL_OK)
164.      {
165.          Error_Handler(__FILE__, __LINE__);
```

```
166.      }
167.
168.      /* 配置 ADC 通道，序列 4，采样温度 */
169.      sConfig.Channel      = ADC_CHANNEL_TEMPSENSOR;      /* 配置使用的 ADC 通道 */
170.      sConfig.Rank         = ADC_REGULAR_RANK_4;          /* 采样序列里的第 1 个 */
171.      sConfig.SamplingTime = ADC_SAMPLETIME_810CYCLES_5;  /* 采样周期 */
172.      sConfig.SingleDiff   = ADC_SINGLE_ENDED;            /* 单端输入 */
173.      sConfig.OffsetNumber = ADC_OFFSET_NONE;             /* 无偏移 */
174.      sConfig.Offset = 0;                                 /* 无偏移的情况下，此参数忽略 */
175.      sConfig.OffsetRightShift       = DISABLE;           /* 禁止右移 */
176.      sConfig.OffsetSignedSaturation = DISABLE;           /* 禁止有符号饱和 */
177.
178.      if (HAL_ADC_ConfigChannel(&AdcHandle, &sConfig) != HAL_OK)
179.      {
180.          Error_Handler(__FILE__, __LINE__);
181.      }
182.
183.
184.      /* ## - 6 - 启动 ADC 的 DMA 方式传输 ########################################## */
185.      if (HAL_ADC_Start_DMA(&AdcHandle, (uint32_t *)ADCxValues, 4) != HAL_OK)
186.      {
187.          Error_Handler(__FILE__, __LINE__);
188.      }
189. }
```

这里把几个关键的地方阐释下：

- 第 11 - 13 行，对作为局部变量的 HAL 库结构体做初始化，防止不确定值配置时出问题。
- 第 17 - 38 行，前面 2.2 小节已经讲解，ADC 时钟源选择 AHB 时钟还是 PLL 时钟。
- 第 41 – 46 行，选择 PC0 作为数据采集引脚。
- 第 49- 68 行，配置 DMA 的基本参数，注释较详细。这里是采用的 ADC 外设到内部 SRAM 的传输方向，数据带宽设置 16bit，循环传输模式。
- 第 71 行，这行代码比较重要，应用中容易被遗忘，用于关联 ADC 句柄和 DMA 句柄。在用户调用 ADC 的 DMA 传输方式函数 HAL_ADC_Start_DMA 时,此函数内部调用的 HAL_DMA_Start_IT 会用到 DMA 句柄。
- 第 75 - 107 行，主要是 ADC 的配置，注释较详细，配置 ADC3 为 16bit 模式，扫描多通道，连续转换，软件触发。
- 第 111 – 114 行，这里的是采用的 ADC 偏移校准，如果要采用线性度校准，务必要注意此贴的问题：http://www.armbbs.cn/forum.php?mod=viewthread&tid=91436 。
- 第 119 -129 行，配置 ADC 多通道采样的第 1 个序列。这里使用的通道 10 是 PC0 引脚的复用功能，不是随意设置的。另外注意转换速度的计算，在程序里面有注释。
- 第 139 – 151 行，配置 ADC 多通道采样的第 2 个序列，采样的 Vbat/4 电压。
- 第 154 – 166 行，配置 ADC 多通道采样的第 3 个序列，采样的 VrefInt 电压。
- 第 169 – 181 行，配置 ADC 多通道采样的第 4 个序列，采样的温度。
- 第 185 – 188 行，启动 ADC 的 DMA 方式数据传输。

46.3.4DMA 存储器选择注意事项

由于 STM32H7 Cache 的存在，凡是 CPU 和 DMA 都会操作到的存储器，我们都要注意数据一致性问题。对于本章节要实现的功能，要注意读 Cache 问题，防止 DMA 已经更新了缓冲区的数据，而我们读取的却是 Cache 里面缓存的。这里提供两种解决办法：

- 方法一：

  关闭 DMA 所使用 SRAM 存储区。

```
/* 配置 SRAM 的 MPU 属性为 Device 或者 Strongly Ordered，即关闭 Cache */
MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress      = 0x60000000;
MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;
```

- 方法二：

  设置 SRAM 的缓冲区做 32 字节对齐，大小最好也是 32 字节整数倍，然后调用函数 SCB_InvalidateDCache_by_Addr 做无效化操作即可，保证 CPU 读取到的数据是刚更新好的。

本章节配套例子是直接使用的方法二。例子中变量的定义方式如下：

```
/* 方便 Cache 类的 API 操作，做 32 字节对齐 */
#if defined ( __ICCARM__ )
#pragma location = 0x38000000
uint16_t ADCxValues[4];
#elif defined ( __CC_ARM )
ALIGN_32BYTES(__attribute__((section (".RAM_D3")))) uint16_t ADCxValues[4];
#endif
```

对于 IAR 需要#pragma location 指定位置，而 MDK 通过分散加载即可实现，详情看前面第 26 章，有详细讲解。

46.3.5读取 DMA 缓冲数据

程序中配置的 DMA 缓冲区可以存储 4 次 ADC 的转换数据，正好 ADCxValues[0]对应 PC0 引脚的采样电压，ADCxValues[1]对应 Vbat/4 电压，ADCxValues[2]对应 VrefInt 采样的电源，ADCxValues[3]对应温度采样值。

具体实现代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_GetAdcValues
```

```
*    功能说明: 获取 ADC 的数据并打印
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_GetAdcValues(void)
{
    float AdcValues[5];
    uint16_t TS_CAL1;
    uint16_t TS_CAL2;

    /*
       使用此函数要特别注意，第 1 个参数地址要 32 字节对齐，第 2 个参数要是 32 字节的整数倍
    */
    SCB_InvalidateDCache_by_Addr((uint32_t *)ADCxValues,  sizeof(ADCxValues));
    AdcValues[0] = ADCxValues[0] * 3.3 / 65536;
    AdcValues[1] = ADCxValues[1] * 3.3 / 65536;
    AdcValues[2] = ADCxValues[2] * 3.3 / 65536;

    /*根据参考手册给的公式计算温度值 */
    TS_CAL1 = *(__IO uint16_t *)(0x1FF1E820);
    TS_CAL2 = *(__IO uint16_t *)(0x1FF1E840);

    AdcValues[3] = (110.0 - 30.0) * (ADCxValues[3] - TS_CAL1)/ (TS_CAL2 - TS_CAL1) + 30;

    printf("PC0 = %5.3fV, Vbat/4 = %5.3fV, VrefInt = %5.3fV,  TempSensor = %5.3f℃\r\n",
            AdcValues[0],  AdcValues[1], AdcValues[2], AdcValues[3]);

}
```

46.4 ADC 板级支持包（bsp_adc.c)

ADC 驱动文件 bsp_adc.c 提供了如下函数：

- bsp_InitADC
- bsp_GetAdcValues

46.4.1 函数 bsp_InitADC

函数原型：

```
void bsp_InitADC(void)
```

函数描述：

此函数用于初始化 ADC，采用 DMA 方式进行多通道采样，采集了 PC0, Vbat/4, VrefInt 和温度。

注意事项：

1. 关于此函数的讲解在本章 2.3 小节。

使用举例：

作为初始化函数，直接在 bsp.c 文件的 bsp_Init 函数里面调用即可。

46.4.2 函数 bsp_GetAdcValues

函数原型：

```
void bsp_GetAdcValues(void)
```

函数描述：

此函数用于获取 ADC 的转换数据。

注意事项：

1. 关于此函数的讲解在本章 2.4 和 2.5 小节。

使用举例：

根据需要，周期性调用即可。

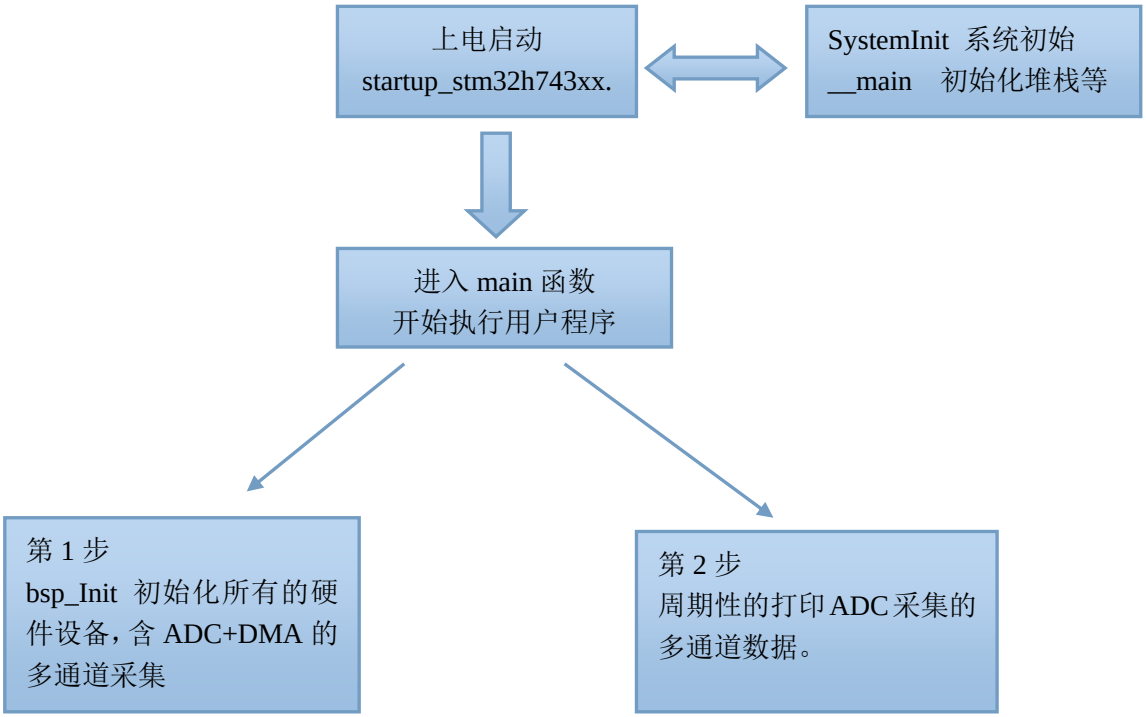
46.5 ADC 驱动移植和使用

ADC 驱动的移植比较方便：

- 第 1 步：复制 bsp_adc.c 和 bsp_adc.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：这几个驱动文件主要用到 HAL 库的 GPIO、TIM，DMA 和 ADC 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 3 步，应用方法看本章节配套例子即可，另外就是根据自己的需要做配置修改。

46.6 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED，串口和 ADC。
  - 第 2 步，周期性的打印 ADC 采集的多通道数据。

46.7 实验例程说明（MDK）

配套例子：

V7-024-ADC+DMA 的多通道采集

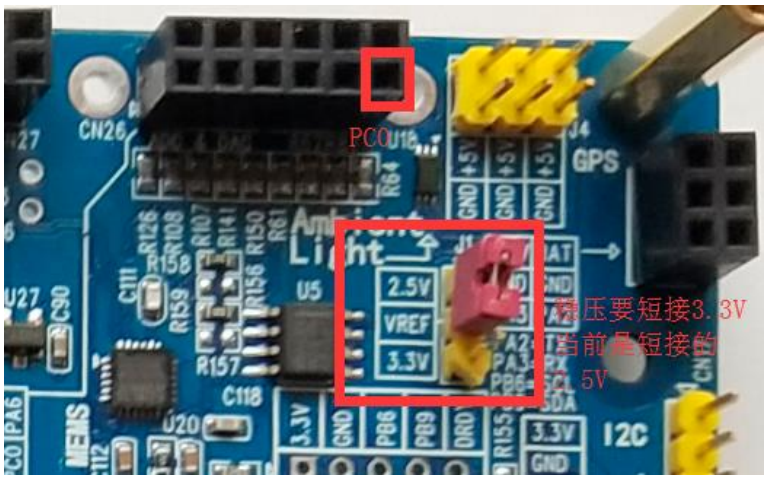
实验目的：

1. 学习 ADC + DMA 的多通道采集实现。

实验内容：

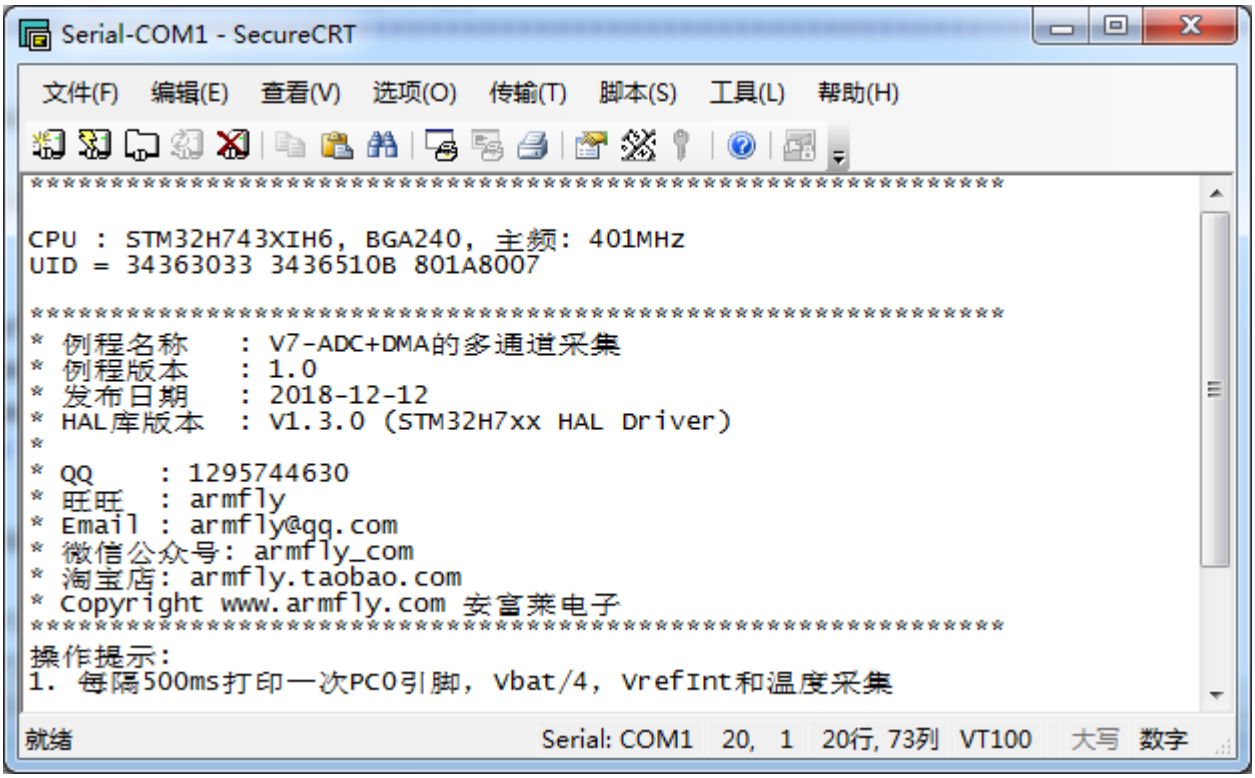
1. 例子默认用的 PLL 时钟供 ADC 使用，大家可以通过 bsp_adc.c 文件开头宏定义切换到 AHB 时钟。
2. 采用 DMA 方式进行多通道采样，采集了 PC0, Vbat/4, VrefInt 和温度。
3. 每隔 500ms，串口会打印一次。
4. 板子正常运行时 LED2 闪烁。

PC0 引脚位置（稳压基准要短接 3.3V）：



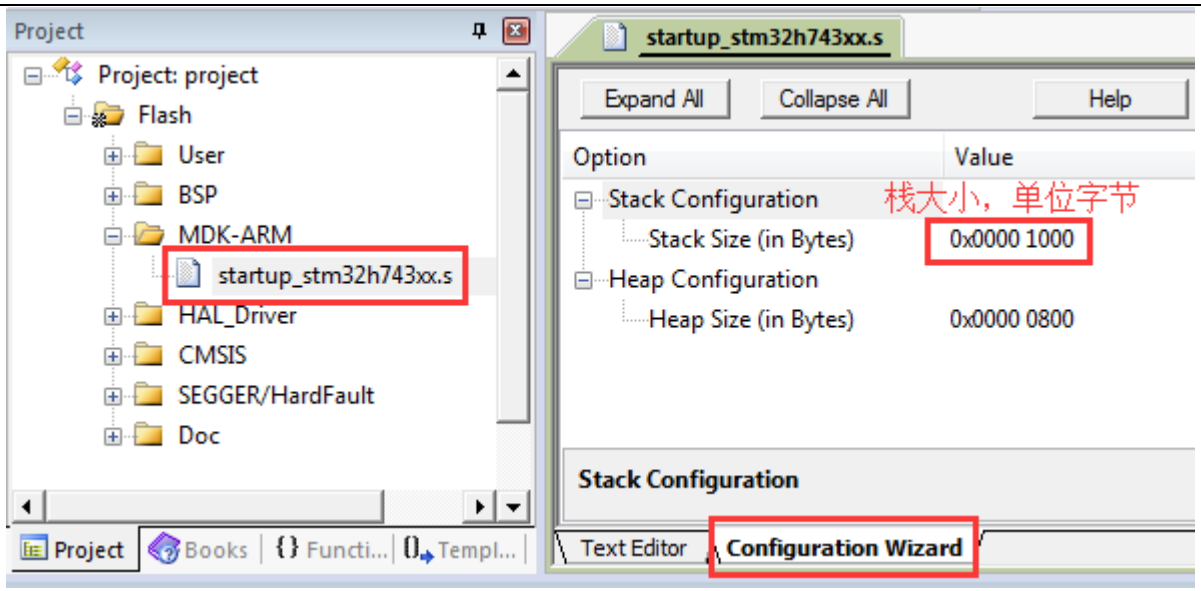
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

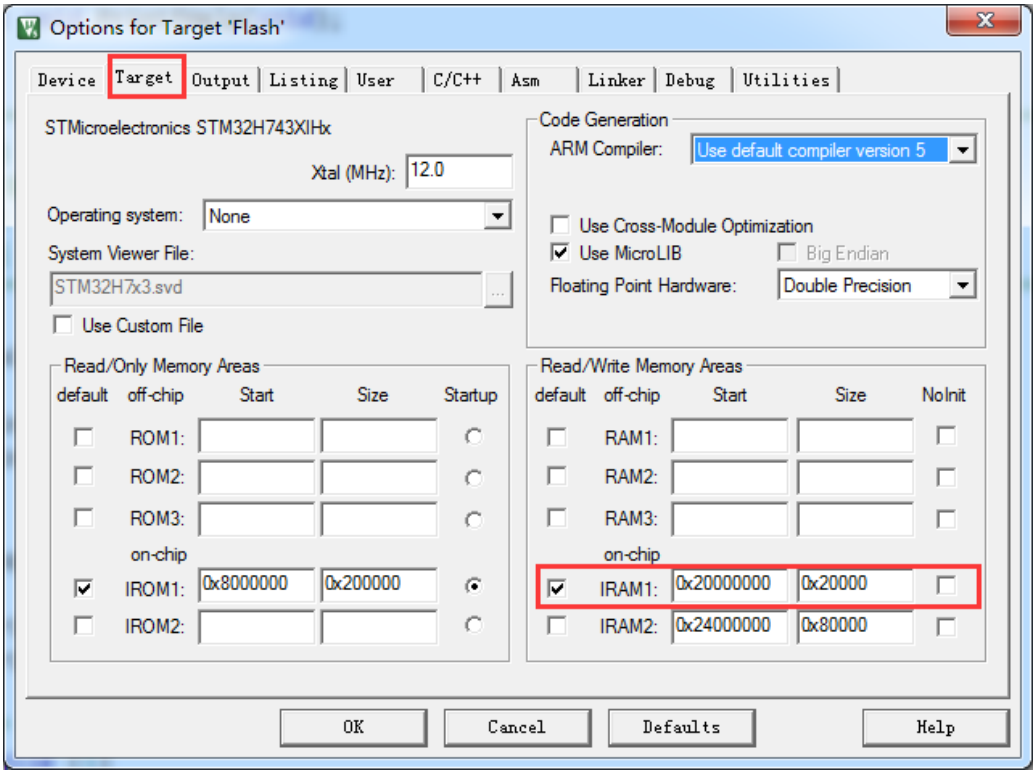


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
    bsp_InitADC();    /* 初始化 ADC */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 D3 域的 SRAM4。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();
```

```
    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM4 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- 每隔 500ms，串口会打印一次 ADC 采集 的 PC0, Vbat/4, VrefInt 和温度。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


#if defined ( __CC_ARM )
    TempValues1 = 0; /* 避免 MDK 警告 */
    TempValues2 = 0;
#endif

    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 500); /* 启动 1 个 500ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
           bsp_GetAdcValues();

            /* 每隔 500ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
```

```
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:       /* K1 键按下 */
                 printf("K1 按键按下\r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

46.8 实验例程说明（IAR）

配套例子：

V7-024-ADC+DMA 的多通道采集

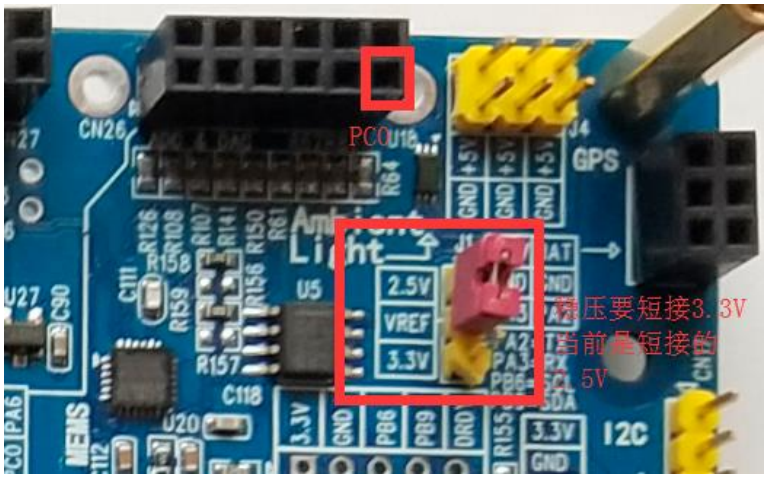
实验目的：

1. 学习 ADC + DMA 的多通道采集实现。

实验内容：

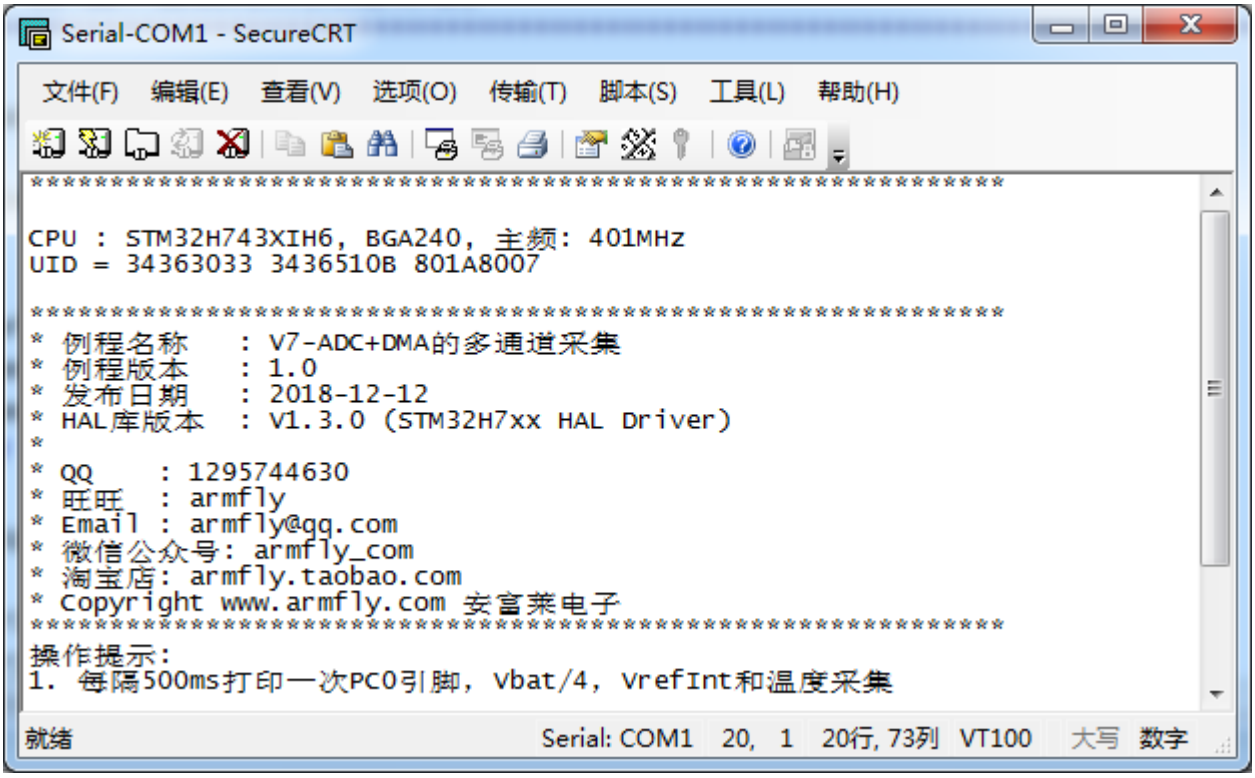
1. 例子默认用的 PLL 时钟供 ADC 使用，大家可以通过 bsp_adc.c 文件开头宏定义切换到 AHB 时钟。
2. 采用 DMA 方式进行多通道采样，采集了 PC0, Vbat/4, VrefInt 和温度。
3. 每隔 500ms，串口会打印一次。
4. 板子正常运行时 LED2 闪烁。

PC0 引脚位置（稳压基准要短接 3.3V）：



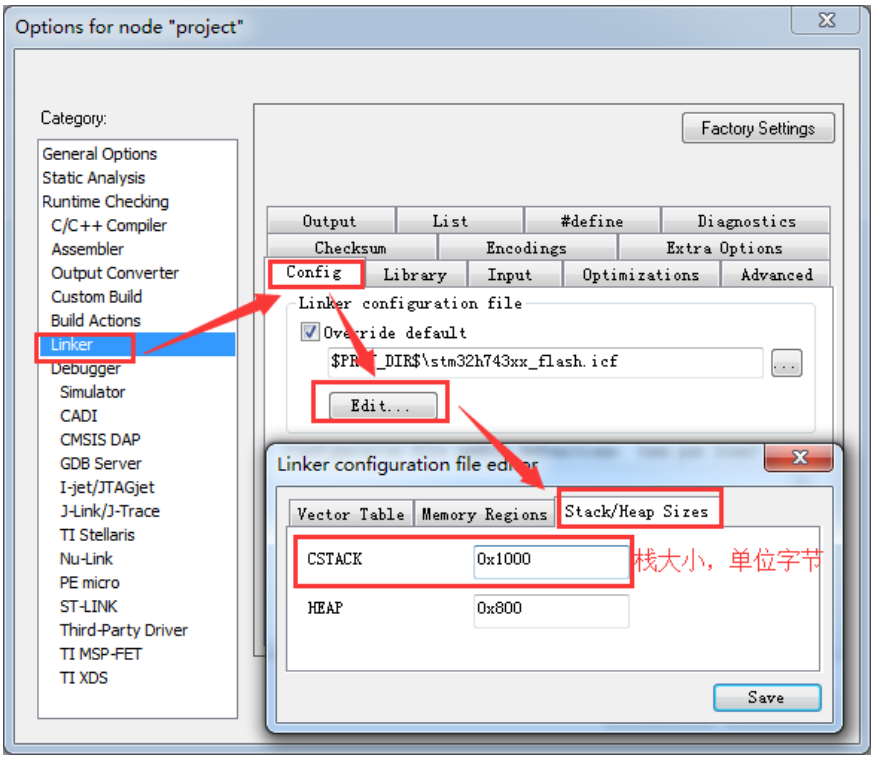
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

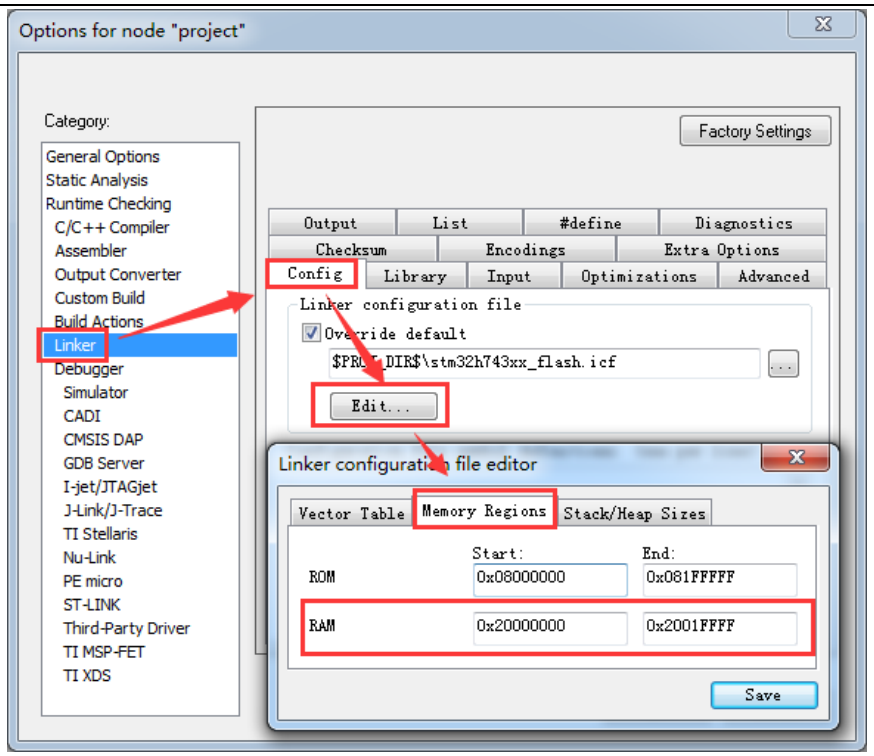


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
    bsp_InitADC();     /* 初始化 ADC */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 D3 域的 SRAM4。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
```

```
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM4 的属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- 每隔 500ms，串口会打印一次 ADC 采集 的 PC0, Vbat/4, VrefInt 和温度。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
```

```
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


#if defined ( __CC_ARM )
    TempValues1 = 0; /* 避免 MDK 警告 */
    TempValues2 = 0;
#endif

    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 500); /* 启动 1 个 500ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
          bsp_GetAdcValues();

            /* 每隔 500ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:       /* K1 键按下 */
                 printf("K1 按键按下\r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

46.9 总结

本章节就为大家讲解这么多，ADC 多通道采样在实际项目中也比较实用，望初学者熟练掌握。

第47章 STM32H7 的 FMC 总线基础知识和 HAL 库 API

本章节为大家讲解并行总线接口 FMC（Flexible memory controller，灵活动态存储器），用到的地方比较多，比如 V7 开发板外接 DM9000，SDRAM，OLED，AD7606，NAND，扩展 IO 等都有用到。

47.1 初学者重要提示
47.2 FMC 基础知识
47.3 FMC 的 HAL 库用法
47.4 源文件 stm32h7xx_ll_fmc.c
47.5 总结

47.1 初学者重要提示

- F1 和 F407 使用的是 FSMC（Flexible static memory controller），跟 F429 和 H7 带的 FMC 区别是不支持 SDRAM，也就是差在字母 static，使用 FMC 可以动态刷新 SDRAM，来保持电量。
- FMC 控制 SRAM 型存储器和 NAND 型存储器是异步控制，而控制 SDRAM 属于同步控制。同步和异步的区别是同步方式需要一个专门的时钟控制引脚。
- FMC 配置中未用到引脚均可以继续用作通用 I/O 模式或者其它复用功能，仅需不配置 FMC 复用即可。
- STM32H7 驱动 32 位 SDRAM 的写速度狂飙 376MB/S，读速度 189MB/S。http://www.armbbs.cn/forum.php?mod=viewthread&tid=91481 。

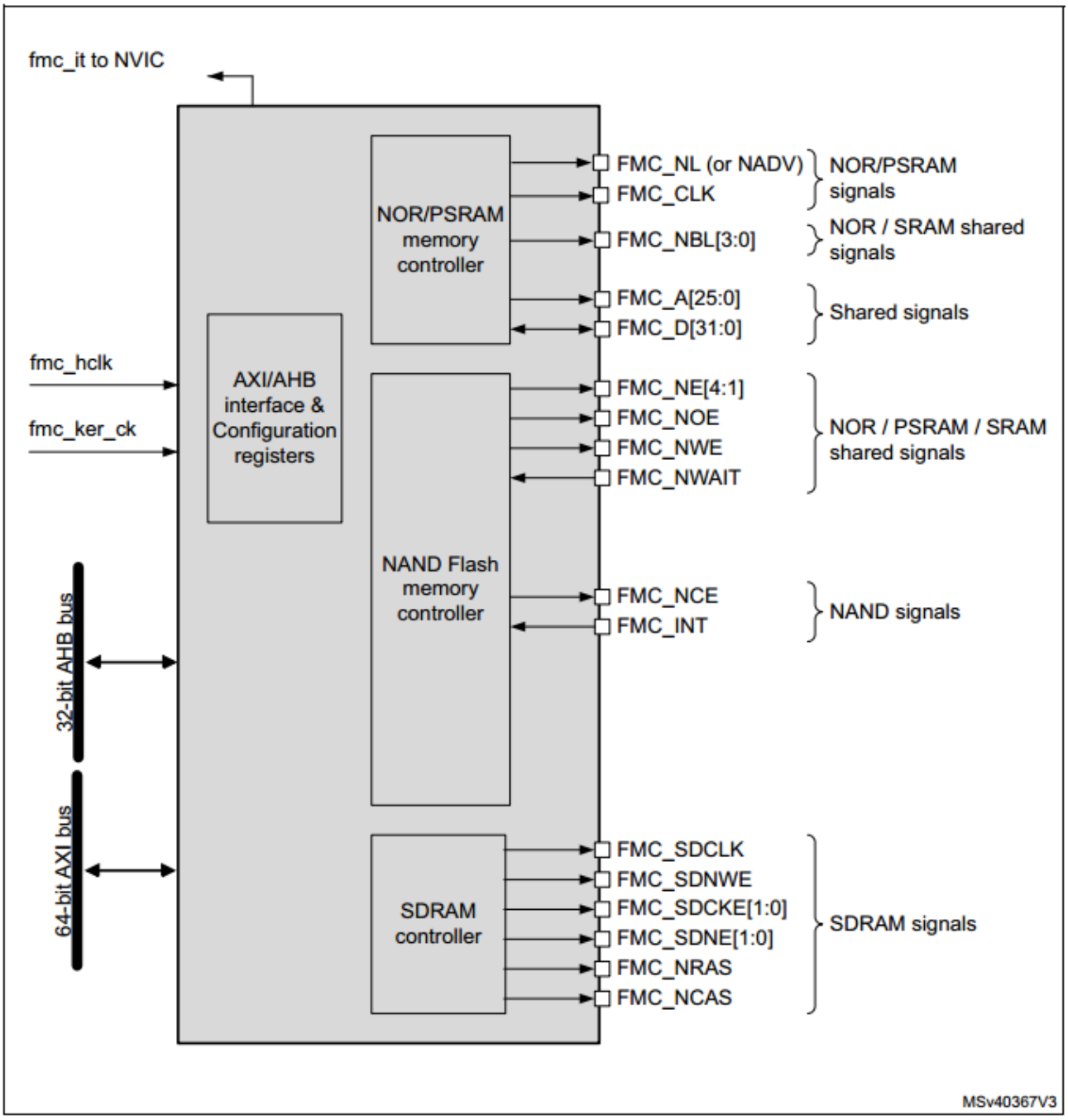
47.2 FMC 基础知识

FMC 的几个关键知识点放在开头说：

- STM32H7 的 FMC 总线是挂载 64 位带宽的 AXI 总线上，F1,F4 和 F7 是挂在 32 位总线上。
- 使用 FMC，可以用来外挂 NOR/PSRAM 型存储器，SRAM 型存储器，NAND 型存储器，SDRAM 存储器等，从而可以用来驱动 AD7606，OLED，DM9000 等并行控制设备。
- 支持 8 位，16 位和 32 位总线带宽控制。
- 每个片选下的存储器空间配置都是独立的，有专门的寄存器，互不影响。这点比较重要，要牢记。

47.2.1 FMC 硬件框图

认识一个外设，最好的方式就是看他的框图，方便我们快速的了解 FMC 的基本功能，然后再看手册了解细节。框图如下所示：



通过这个框图，我们可以得到如下信息：

- fmc_it to NVIC 接口

  用于 FMC 的 NVIC 配置，使能其中断通道。

- fmc_hclk 接口

FMC 的接口时钟。

- fmc_ker_ck 接口

  FMC 的内核时钟。

- SDRAM signals

  驱动 SDRAM 独享的几个控制信号。

- NAND signals

  驱动 NAND 独享的几个控制信号。

- NOR/PSRAM/SRAM shared signals

  驱动 NOR/PSRAM/SRAM 的共享信号，主要是片选、读写控制和等待信号。

- Shared signals

  这个是驱动任何设备都要用到地址和数据线，地址线有 A[0] – A[25]，共计 26 根，能访问的地址空间是 2^26 = 64MB。数据线是 D[0] – D[31]，共计 32 根，也就是说可以支持 32 位带宽的存储器访问模式。

- NOR/SRAM shared signals

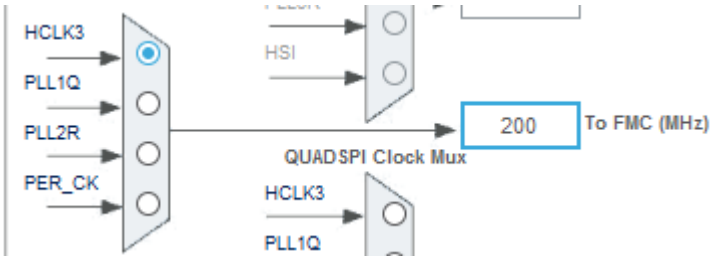
  用于 NOR/SRAM 存储器的字节方式控制。

- NOR/PSRAM signals

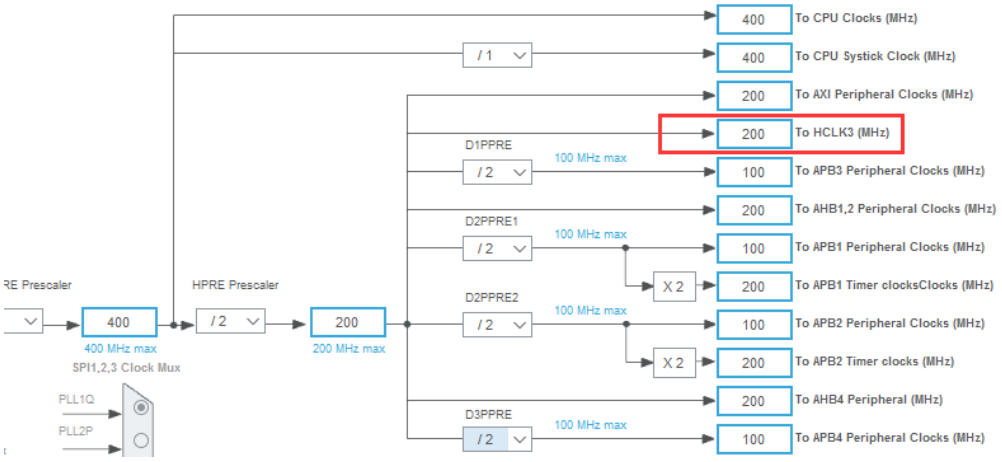
  用于控制需要同步时钟的 NOR/PSRAM 类型存储器。

47.2.2 FMC 时钟选择

使用 FMC 可以选择如下几种时钟源 HCLK3，PLL1Q，PLL2R 和 PER_CK：

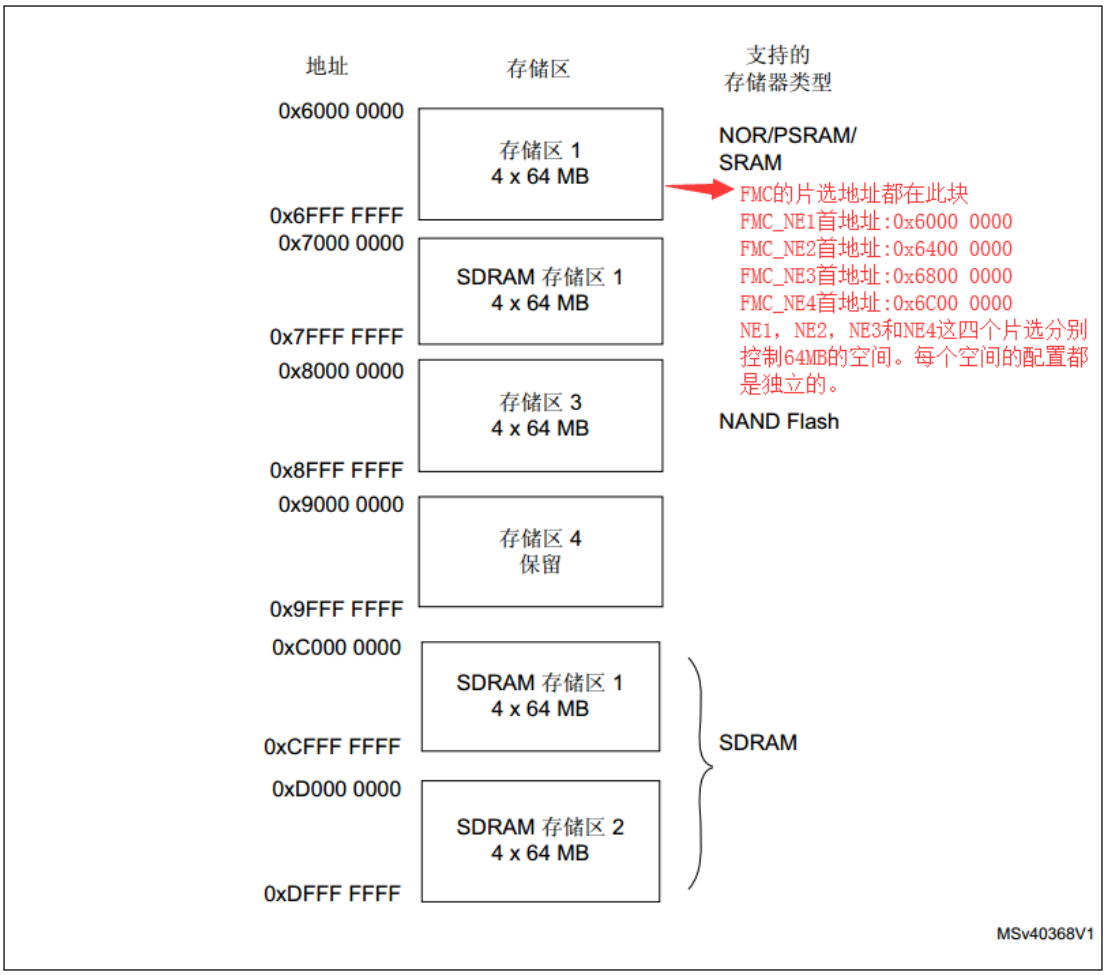


我们这里直接使用 HCLK3，配置 STM32H7 的主频为 400MHz 的时候，HCLK3 输出的 200MHz，这个速度是 FMC 支持的最高时钟，正好用于这里：



47.2.3 FMC 地址区分配

FMC 总线可操作的地址范围 0x60000000 到 0xDFFFFFFF，具体的框图如下：



根据上面这个地址映射图，下面几个知识点要了解到：

◆ 与 F1 和 F4 不同，H7 系列的 FMC 总线接口支持重映射，也就是可以设置这几块存储器的位置。

起始地址 - 结束地址	BMAP[1:0]=00 （默认映射）	BMAP[1:0]=01 交换 NOR/PSRAM 与 SDRAM 存储区域	BMAP[1:0]=10 重映射 SDRAM 存储区域 2
0x6000 0000 - 0x6FFF FFFF	NOR/PSRAM 存储区域	SDRAM 存储区域 1	NOR/PSRAM 存储区域
0x7000 0000 - 0x7FFF FFFF	SDRAM 存储区域 1	SDRAM 存储区域 2	SDRAM 存储区域 2
0x8000 0000 - 0x8FFF FFFF	NAND 存储区域	NAND 存储区域	NAND 存储区域
0x9000 0000 - 0x9FFF FFFF	保留	保留	保留
0xC000 0000 - 0xCFFF FFFF	SDRAM 存储区域 1	NOR/PSRAM 存储区域	SDRAM 存储区域 1
0xD000 0000 - 0xDFFF FFFF	SDRAM 存储区域 2	SDRAM 存储区域 2	SDRAM 存储区域 2

根据 BMAP[0:1]设置的数值 00，01 或者 10，支持三种分配方式。默认情况下，使用 BMAP[0:1] = 00 的分配方式即可，也就是上面截图中的分配方式。

◆ 对于 NOR/PSRAM/SRAM 块区。

这个块区用到的地方最多，像 NAND 和 SDRAM 块区基本只能接 NAND 和 SDRAM，而 NOR/PSRAM/SRAM 区就不同了，除了能接这几种类型的存储器，还可以外接 DM9000，SDRAM，OLED，AD7606 等总线外设。这个块区有 4 路片选，分别是 FMC_NE1，FMC_NE2，FMC_NE3 和 FMC_NE4，这几个片选在芯片上都有对应的引脚，每个片选可以管理 64MB 的访问空间，这个是由 FMC 引出的 26 路地址线 FMC_A[0:25]决定的，2^26 = 64MB。

- FMC_NE1：首地址 0x6000 0000，可以管理的地址范围 0x6000 0000 到 0x63FF FFFF。
- FMC_NE2：首地址 0x6400 0000，可以管理的地址范围 0x6400 0000 到 0x67FF FFFF。
- FMC_NE3：首地址 0x6800 0000，可以管理的地址范围 0x6800 0000 到 0x6BFF FFFF。
- FMC_NE4：首地址 0x6C00 0000，可以管理的地址范围 0x6C00 0000 到 0x6FFF FFFF。

这里特别注意一点，每个片选控制的地址空间都是可以独立配置的，与其它片选控制的空间互不影响。也许大家会问，只有这四路片选不够用啊，而且还有片选引脚被复用做其它功能的情况，这可以通过外接地址译码器解决，V7 开发板就是通过这种方式外接了多个总线设备。

◆ 使用 NAND 块区外接 NAND 的时候，务必要使用 MPU 将这块区域设置为 Device 模式。

47.2.4NOR/PSRAM/SRAM 区地址映射（重要）

FMC 总线除了复用到具体引脚上的 A[25:0]，共计 26 路地址以外，还有两条内部总线 ADDR[27:26]。通过这两路线才区分出了 FMC_NE1，FMC_NE2，FMC_NE3 和 FMC_NE4。即 0x60xx xxxx，0x64xx xxxx，0x68xx xxx 和 0x6Cxx xxxx。

表 139. NOR/PSRAM 存储区域选择

ADDR[27:26](1)	选择的存储区域
00	存储区域 1 NOR/PSRAM 1
01	存储区域 1 NOR/PSRAM 2
10	存储区域 1 NOR/PSRAM 3
11	存储区域 1 NOR/PSRAM 4

1. ADDR 是内部地址线，但也会参与对外部存储器的寻址。

这些地址都是按照字节进行访问的，如果控制 8 位宽度的存储设备比较方便，如果控制 16 位，32 位宽度的存储设备，且不支持字节访问就比较麻烦了。比如 16 位宽度的 NOR Flash，写入仅支持 16 位或者 32 位（通过写入两次 16 位），写入 8 位数据是不支持的。这个时候，为了方便操作，FMC 在硬件设计上就提供了一种解决办法，将内部数据总线 ADDR[25:0]措位后接到 FMC_A 地址引脚上。

表 140. NOR/PSRAM 外部存储器地址

存储器宽度(1)	向存储器发出的数据地址	最大存储器容量（位）
8 位	ADDR[25:0]	64 MB x 8 = 512 Mb
16 位	ADDR[25:1] >> 1	64 MB/2 x 16 = 512 Mb
32 位	ADDR[25:2] >> 2	64 MB/4 x 32 = 512 Mb

配置外部存储器的宽度为 16 位， FMC 将使用内部的 ADDR[25:1]地址来作为对外部存储器的寻址地址 FMC_A[24:0]。如果存储器宽度为 32 位， FMC 将使用内部的 ADDR[25:2]地址进行外部寻址。但是不管存储器的宽度是多少， FMC_A[0]都应连接到外部存储器地址 A[0]。

这里有一点要补充下，如果外接 SDRAM/SRAM 配置为 16 位或者 32 位带宽的时候，如何实现字节方式的读写。针对这个问题，SDRAM/SRAM 都有专门的字节控制引脚，所以操作起来比较方便。

47.2.5 NOR/PSRAM/SRAM 时序控制

F103 和 F407 仅支持 16 位总线访问，等到 F429，H7 已经支持 32 位总线访问。以驱动 SRAM 为例，需要用到下面的数据，地址和控制引脚。配置完毕后，就可以像使用内部 SRAM 一样进行读写了，使用比较方便。

FMC信号名称	I/O	功能
CLK	O	时钟（仅用于 PSRAM 同步访问）
A[25:0]	O	地址总线
D[31:0]	I/O	数据双向总线
NE[x]	O	片选，x = 1..4 （在 PSRAM 应用中被称作 NCE（Cellular RAM，即 CRAM））
NOE	O	输出使能
NWE	O	写入使能
NL(= NADV)	O	仅用于 PSRAM 输入的地址有效信号（存储器信号名称：NADV）
NWAIT	I	FMC 的 PSRAM 等待输入信号
NBL[3:0]	O	字节通道输出。字节 0 到字节 3 控制（高字节和低字节使能）

FMC 支持以下四种时序模式：

```
/** @defgroup FMC_Access_Mode FMC Access Mode
  * @{
  */
#define FMC_ACCESS_MODE_A                ((uint32_t)0x00000000U)
#define FMC_ACCESS_MODE_B                ((uint32_t)0x10000000U)
#define FMC_ACCESS_MODE_C                ((uint32_t)0x20000000U)
#define FMC_ACCESS_MODE_D                ((uint32_t)0x30000000)
```

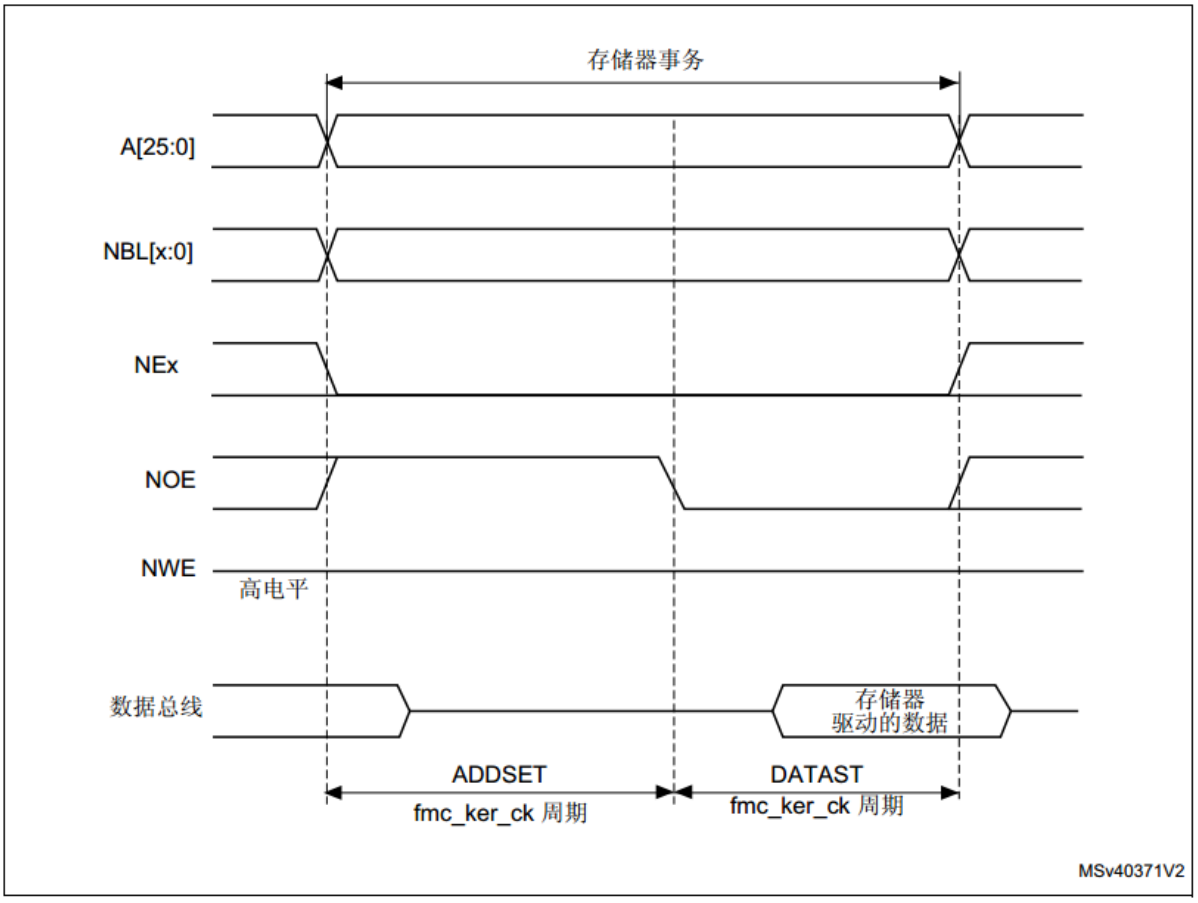
每个模式的时序略有不同，我们这里以常用的模式 A 为例进行说明：

◆ 先来认识如下几个关键参数：

参数	功能	访问模式	单位	最小值	最大值
地址建立	地址建立阶段的持续时间	异步	FMC 时钟周期 (fmc_ker_ck)	0	15
地址保持	地址保持阶段的持续时间	异步，复用 I/O	FMC 时钟周期 (fmc_ker_ck)	1	15
数据建立	数据建立阶段的持续时间	异步	FMC 时钟周期 (fmc_ker_ck)	1	256
总线周转	总线周转阶段的持续时间	异步和同步读取	FMC 时钟周期 (fmc_ker_ck)	0	15
时钟分频比	构建一个存储器时钟周期 (CLK) 所需的 FMC 时钟周期 (fmc_ker_ck) 数量	同步	FMC 时钟周期 (fmc_ker_ck)	2	16
数据延迟	在发出突发的第一个数据前向存储器发出的时钟周期数量	同步	存储器时钟周期 (fmc_ker_ck)	2	17

Parameter	Function	Access mode	Unit	Min.	Max.
Address setup	Duration of the address setup phase	Asynchronous	FMC clock cycle (fmc_ker_ck)	0	15
Address hold	Duration of the address hold phase	Asynchronous, muxed I/Os	FMC clock cycle (fmc_ker_ck)	1	15
Data setup	Duration of the data setup phase	Asynchronous	FMC clock cycle (fmc_ker_ck)	1	256
Bust turn	Duration of the bus turnaround phase	Asynchronous and synchronous read	FMC clock cycle (fmc_ker_ck)	0	15
Clock divide ratio	Number of FMC clock cycles (fmc_ker_ck) to build one memory clock cycle (CLK)	Synchronous	FMC clock cycle (fmc_ker_ck)	2	16
Data latency	Number of clock cycles to issue to the memory before the first data of the burst	Synchronous	Memory clock cycle (fmc_ker_ck)	2	17

了解了这几个参数后再来看模式 A 的读时序：

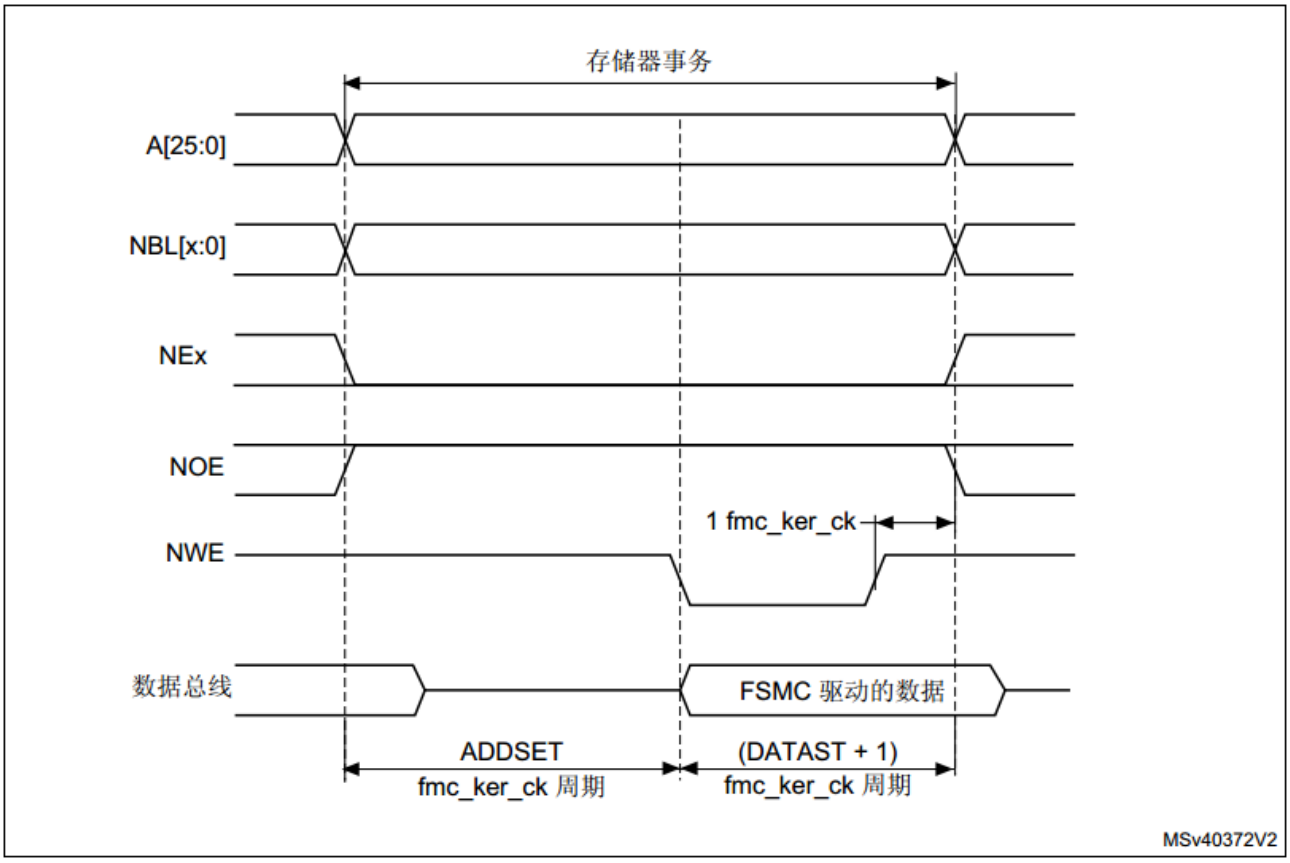


通过读时序图，我们要了解到以下几点知识：

- NBL[x:0]高低字节控制信号，NEx 片选信号，NOE 读使能信号和 NWE 写使能前面的字母 N（Ngative）示这几个信号低电平有效。
- 地址信号 A[25:0]，NBL[x:0]高低字节控制信号，要在 NEx 片选信号使能前准备就绪。整个读取过程中，NWE 写使能信号是不起作用的，被设置为高电平。
- NEx 片选后，NOE 要保持一段时间的高电平，这个时间就是 ADDSET 地址建立时间（通过寄存器 FMC_BTRx 可配置）。之后 NOE 变为低电平，读使能。低电平保持的时间由 DATAST 数据建立时间（通过寄存器 FMC_BTRx 可配置）决定。
- 读取完毕数据后，NOE 变成高电平，NEx 变成高电平。

这里主要一点，A 模式读时序没有用到地址保持时间（Address Hold）。

模式 A 的写时序：



通过写时序图，我们要了解到以下几点知识：

- NBL[x:0]高低字节控制信号，NEx 片选信号，NOE 读使能信号和 NWE 写使能前面的字母 N（Ngative）示这几个信号低电平有效。
- 地址信号 A[25:0]，NBL[x:0]高低字节控制信号，要在 NEx 片选信号使能前准备就绪。整个写入过程中，NOE 读使能信号是不起作用的，被设置为高电平。

- NEx 片选后，NWE 要保持一段时间的高电平，这个时间就是 ADDSET 地址建立时间（通过寄存器 FMC_BTRx 可配置）。之后 NWE 变为低电平，写使能。低电平保持的时间由 DATAST 数据建立时间（通过寄存器 FMC_BTRx 可配置）决定。
- 写入完毕数据后，等待 NWE 置高 1 个 FMC 时钟周期后，NOE 变成低电平，NEx 变成高电平。

这里主要一点，A 模式写时序没有用到地址保持时间（Address Hold）。

47.2.6 NOR/PSRAM/SRAM 的不同位宽通信问题

FMC 是外挂在 64 位带宽的 AXI 总线上，可以 8 位，16 位，32 位或者 64 位方式操作。如果 FMC 外接设备是 16 位带宽，H7 参考手册给出了不同位宽设置的支持情况：

设备	模式	R/W	AXI 数据大小	存储器数据大小	是否允许	注释
NOR Flash（复用 I/O 和非复用 I/O）	异步	R	8	16	是	
	异步	W	8	16	否	
	异步	R	16	16	是	
	异步	W	16	16	是	
	异步	R	32	16	是	分为 2 次 FMC 访问
	异步	W	32	16	是	分为 2 次 FMC 访问
	异步	R	64	16	是	分为 4 次 FMC 访问
	异步	W	64	16	是	分为 4 次 FMC 访问
	异步页	R	-	16	否	不支持该模式
	同步	R	8	16	否	
	同步	R	16	16	是	
	同步	R	32/64	16	是	

根据上面表格，操作并口 NOR Flash 时，比如 8 位数据写入是不支持的，而读取是支持的，这是因为可以读取一个 16 位数据，放弃高字节或者低字节。

设备	模式	R/W	AXI 数据大小	存储器数据大小	是否允许	注释
SRAM 和 ROM	异步	R	8/16	16	是	
	异步	W	8/16	16	是	使用字节通道 NBL[1:0]
	异步	R	32	16	是	分为 2 次 FMC 访问
	异步	W	32	16	是	分为 2 次 FMC 访问，使用字节通道 NBL[1:0]
	异步	R	64	16	是	分为 4 次 FMC 访问
	异步	W	64	16	是	分为 4 次 FMC 访问，使用字节通道 NBL[1:0]

1. NBL[1:0] 同样通过 AXI 写入选通驱动。

根据上面表格，操作 SRAM 就比较方便了，各种数据位宽都可以正常操作 SRAM，因为 SRAM 有专门的，NBL[x:0]高低字节控制引脚。

47.3 FMC 的 HAL 库用法

FMC 的 HAL 库用法其实就是几个结构体变量成员的配置和使用，然后配置 GPIO、时钟，并根据需要配置 NVIC、中断和 DMA。下面我们逐一展开为大家做个说明。

47.3.1 NOR/PSRAM/SRAM 寄存器结构体

FMC 控制 NOR/PSRAM/SRAM 相关的寄存器是通过 HAL 库中的结构体 FMC_NORSRAM_TypeDef 和 FMC_NORSRAM_EXTENDED_TypeDef 定义的，在 stm32h743xx.h 中可以找到这个类型定义：

```
#define FMC_NORSRAM_TypeDef            FMC_Bank1_TypeDef
#define FMC_NORSRAM_EXTENDED_TypeDef   FMC_Bank1E_TypeDef

typedef struct
{
  __IO uint32_t BTCR[8];
} FMC_Bank1_TypeDef;

typedef struct
{
  __IO uint32_t BWTR[7];
} FMC_Bank1E_TypeDef;
```

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

FMC 控制 NOR/PSRAM/SRAM 设备涉及到 FMC_BCRx，FMC_BTRx 和 FMC_BWTRx 这三个寄存器(x的范围是1到4)，共计4路片选NE1 – NE4，每路对应一组FMC_BCRx，FMC_BTRx和FMC_BWTR。

为了方便控制，HAL 库是将 4 个寄存器 FMC_BCRx 和 4 个寄存器 FMC_BTRx 统一由结构体变量 FMC_Bank1_TypeDef 来定义，__IO uint32_t BTCR[8]正好 8 个 32 位变量。

而结构体变量 FMC_Bank1E_TypeDef 的__IO uint32_t BWTR[7]是用来定义 4 个 FMC_BWTRx 寄存器，也许大家会问为什么要用 7 个 32 位变量来定义，这是因为每个 FMC_BWTRx 寄存器的地址间隔 8 个字节。

下面再来看这几个寄存器的具体定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE           ((uint32_t)0x40000000)
#define D1_AHB1PERIPH_BASE    (PERIPH_BASE + 0x12000000)
#define FMC_R_BASE            (D1_AHB1PERIPH_BASE + 0x4000)

#define FMC_Bank1_R_BASE      (FMC_R_BASE + 0x0000)
#define FMC_Bank1E_R_BASE     (FMC_R_BASE + 0x0104)
#define FMC_Bank2_R_BASE      (FMC_R_BASE + 0x0060)
#define FMC_Bank3_R_BASE      (FMC_R_BASE + 0x0080)
#define FMC_Bank5_6_R_BASE    (FMC_R_BASE + 0x0140)

<-----展开下面的宏定义，(FMC_Bank1_TypeDef *) 0x52004000
#define FMC_Bank1             ((FMC_Bank1_TypeDef *) FMC_Bank1_R_BASE)
#define FMC_Bank1E            ((FMC_Bank1E_TypeDef *) FMC_Bank1E_R_BASE)
#define FMC_Bank2             ((FMC_Bank2_TypeDef *) FMC_Bank2_R_BASE)
#define FMC_Bank3             ((FMC_Bank3_TypeDef *) FMC_Bank3_R_BASE)
#define FMC_Bank5_6           ((FMC_Bank5_6_TypeDef *) FMC_Bank5_6_R_BASE)

#define FMC_NORSRAM_DEVICE              FMC_Bank1
#define FMC_NORSRAM_EXTENDED_DEVICE     FMC_Bank1E
#define FMC_NAND_DEVICE                 FMC_Bank3
#define FMC_SDRAM_DEVICE                FMC_Bank5_6
```

我们访问 FMC 的 BCR0 寄存器可以采用这种形式：FMC->BCR[0] = 0。

47.3.2 NOR/PSRAM/SRAM 参数配置结构体 FMC_NORSRAM_InitTypeDef

这个结构体比较重要，FMC 驱动 NOR/PSRAM/SRAM 设备的参数配置，全靠这个结构体实现，具体定义如下：

```
typedef struct
{
  uint32_t NSBank;
  uint32_t DataAddressMux;
  uint32_t MemoryType;
  uint32_t MemoryDataWidth;
  uint32_t BurstAccessMode;
  uint32_t WaitSignalPolarity;
  uint32_t WaitSignalActive;
  uint32_t WriteOperation;
  uint32_t WaitSignal;
  uint32_t ExtendedMode;
  uint32_t AsynchronousWait;
  uint32_t WriteBurst;
```

```
  uint32_t ContinuousClock;
  uint32_t WriteFifo;
  uint32_t PageSize;
}FMC_NORSRAM_InitTypeDef;
```

下面将这几个成员逐一为大家做个介绍：

- uint32_t NSBank

  用于设置使用的 BANK，片选 NE1 对于 BANK1，片选 NE2 对应 BANK2，片选 NE3 对应 BANK3，NE4 对应 BANK4，支持的参数类型如下：

```
#define FMC_NORSRAM_BANK1                ((uint32_t)0x00000000U)
#define FMC_NORSRAM_BANK2                ((uint32_t)0x00000002U)
#define FMC_NORSRAM_BANK3                ((uint32_t)0x00000004U)
#define FMC_NORSRAM_BANK4                ((uint32_t)0x00000006U)
```

- uint32_t DataAddressMux

  用于设置地址线和数据线复用，都使用的 FMC_D[31:0]，可以选择使能或者禁止：

```
#define FMC_DATA_ADDRESS_MUX_DISABLE         ((uint32_t)0x00000000U)
#define FMC_DATA_ADDRESS_MUX_ENABLE          ((uint32_t)0x00000002U)
```

- uint32_t MemoryType

  用于设置使用的存储器类型，具体支持的参数如下：

```
#define FMC_MEMORY_TYPE_SRAM                 ((uint32_t)0x00000000U)
#define FMC_MEMORY_TYPE_PSRAM                ((uint32_t)0x00000004U)
#define FMC_MEMORY_TYPE_NOR                  ((uint32_t)0x00000008U)
```

- uint32_t MemoryDataWidth

  用于设置外接的存储器位宽，具体支持的参数如下：

```
#define FMC_NORSRAM_MEM_BUS_WIDTH_8          ((uint32_t)0x00000000U)
#define FMC_NORSRAM_MEM_BUS_WIDTH_16         ((uint32_t)0x00000010U)
#define FMC_NORSRAM_MEM_BUS_WIDTH_32         ((uint32_t)0x00000020U)
```

- uint32_t BurstAccessMode

  用于使能或者禁止突发模式，仅用于支持同步突发的存储器，具体支持的参数如下：

```
#define FMC_BURST_ACCESS_MODE_DISABLE        ((uint32_t)0x00000000U)
#define FMC_BURST_ACCESS_MODE_ENABLE         ((uint32_t)0x00000100U)
```

- uint32_t WaitSignalPolarity

  用于设置等待信号的极性，仅当使能突发模式时有效，具体支持的参数如下：

```
#define FMC_WAIT_SIGNAL_POLARITY_LOW         ((uint32_t)0x00000000U)
#define FMC_WAIT_SIGNAL_POLARITY_HIGH        ((uint32_t)0x00000200U)
```

- uint32_t WaitSignalActive

  用于在等待状态之前或等待状态期间，存储器是否在一个时钟周期内置位等待信号，仅当使能突发模式时有效，具体支持的参数如下：

```
#define FMC_WAIT_TIMING_BEFORE_WS            ((uint32_t)0x00000000U)
#define FMC_WAIT_TIMING_DURING_WS            ((uint32_t)0x00000800U)
```

- uint32_t WriteOperation

  用于使能或者禁止写保护，具体支持的参数如下：

```
#define FMC_WRITE_OPERATION_DISABLE          ((uint32_t)0x00000000U)
#define FMC_WRITE_OPERATION_ENABLE           ((uint32_t)0x00001000U)
```

- uint32_t WaitSignal

用于使能或者禁止通过等待信号来插入等待状态，仅当使能突发模式时有效，具体支持的参数如下：

```
#define FMC_WAIT_SIGNAL_DISABLE                 ((uint32_t)0x00000000U)
#define FMC_WAIT_SIGNAL_ENABLE                  ((uint32_t)0x00002000U)
```

- uint32_t ExtendedMode

用于使能或者禁止扩展模式，具体支持的参数如下：

```
#define FMC_EXTENDED_MODE_DISABLE               ((uint32_t)0x00000000U)
#define FMC_EXTENDED_MODE_ENABLE                ((uint32_t)0x00004000U)
```

- uint32_t AsynchronousWait

用于异步传输期间，使能或者禁止等待信号，仅操作异步存储器有效，具体支持的参数如下：

```
#define FMC_ASYNCHRONOUS_WAIT_DISABLE           ((uint32_t)0x00000000U)
#define FMC_ASYNCHRONOUS_WAIT_ENABLE            ((uint32_t)0x00008000U)
```

- uint32_t WriteBurst

用于使能或者禁止异步的写突发操作：

```
#define FMC_WRITE_BURST_DISABLE                 ((uint32_t)0x00000000U)
#define FMC_WRITE_BURST_ENABLE                  ((uint32_t)0x00080000U)
```

- uint32_t ContinuousClock

用于使能或者禁止 FMC 同步异步模式的时钟信号输出，此参数仅通过 FMC_BCR1 使能，与 FMC_BCR2,3,4 无关，具体支持的参数如下：

```
#define FMC_CONTINUOUS_CLOCK_SYNC_ONLY          ((uint32_t)0x00000000U) /* 同步模式时钟输出 */
#define FMC_CONTINUOUS_CLOCK_SYNC_ASYNC         ((uint32_t)0x00100000U) /* 同步和异步模式均时钟输出 */
```

- uint32_t WriteFifo

用于使能或者禁止写 FIFO，此参数仅通过 FMC_BCR1 使能，与 FMC_BCR2,3,4 无关，具体支持的参数如下：

```
#define FMC_WRITE_FIFO_DISABLE            ((uint32_t)FMC_BCR1_WFDIS)
#define FMC_WRITE_FIFO_ENABLE             ((uint32_t)0x00000000U)
```

- uint32_t PageSize

用于设置页大小，FMC 操作器件 Cellular RAM 1.5 时要用到，具体支持的参数如下：

```
#define FMC_PAGE_SIZE_NONE            ((uint32_t)0x00000000U)
#define FMC_PAGE_SIZE_128             ((uint32_t)FMC_BCR1_CPSIZE_0)
#define FMC_PAGE_SIZE_256             ((uint32_t)FMC_BCR1_CPSIZE_1)
#define FMC_PAGE_SIZE_512             ((uint32_t)(FMC_BCR1_CPSIZE_0 | FMC_BCR1_CPSIZE_1))
#define FMC_PAGE_SIZE_1024            ((uint32_t)FMC_BCR1_CPSIZE_2)
```

47.3.3 NOR/PSARM/SRAM 时序配置结构体 FMC_NORSRAM_TimingTypeDef

此结构体对于配置 NOR/PSRAM/SRAM 器件的时序参数比较重要，具体定义如下：

```
typedef struct
{
  uint32_t AddressSetupTime;
  uint32_t AddressHoldTime;
  uint32_t DataSetupTime;
  uint32_t BusTurnAroundDuration;
```

```
  uint32_t CLKDivision;
  uint32_t DataLatency;
  uint32_t AccessMode;
}FMC_NORSRAM_TimingTypeDef;
```

下面将这几个参数逐一为大家做个说明（默认情况下 FMC 用的 HCLK3，主频 200MHz）

- uint32_t AddressSetupTime

  此参数用于设置地址建立时间，单位 FMC 时钟周期个数，范围 0 -15。同步 NOR Flash 用不到此参数。

- uint32_t AddressHoldTime

  此参数用于设置地址持续时间，单位 FMC 时钟周期个数，范围 1 -15。同步 NOR Flash 用不到此参数。

- uint32_t DataSetupTime

  此参数用于设置数据建立时间，单位 FMC 时钟周期个数，范围 1 -255。用于 SRAM，异步多路复用 NOR Flash。

- uint32_t BusTurnAroundDuration

  此参数用于设置总线 TurnAround (总线周转阶段) 持续时间，单位 FMC 时钟周期个数，范围 0 -15。仅用于多路复用 NOR Flash。

- uint32_t CLKDivision

  此参数用于设置时钟分频，范围 2 -16，仅用于同步器件。

- uint32_t DataLatency

  对于使能了读/写突发模式的同步访问，此参数定义了读写首个数据前要发送给存储器的时钟周期个数。

  - 操作 CRAM，此参数必须为 0。
  - 异步 NOR/PSRAM/SRAM 器件用不到此参数。
  - 使能了同步突发模式的 NOR Flash，此参数的范围是 2 – 17，单位 FMC 时钟周期个数。

- uint32_t AccessMode

  用于设置 FMC 的访问模式，支持如下四种：

```
#define FMC_ACCESS_MODE_A                       ((uint32_t)0x00000000U)
#define FMC_ACCESS_MODE_B                       ((uint32_t)0x10000000U)
#define FMC_ACCESS_MODE_C                       ((uint32_t)0x20000000U)
#define FMC_ACCESS_MODE_D                       ((uint32_t)0x30000000)
```

47.3.4SRAM 句柄结构体 SRAM_HandleTypeDef

HAL 库在 FMC_NORSRAM_TypeDef 和 FMC_NORSRAM_EXTENDED_TypeDef 的基础上封装了一个结构体 SRAM_HandleTypeDef，定义如下：

```
typedef struct
{
  FMC_NORSRAM_TypeDef           *Instance;
```

```
  FMC_NORSRAM_EXTENDED_TypeDef  *Extended;
  FMC_NORSRAM_InitTypeDef       Init;
  HAL_LockTypeDef               Lock;
  __IO HAL_SRAM_StateTypeDef    State;
  MDMA_HandleTypeDef            *hmdma;
}SRAM_HandleTypeDef;
```

下面将这几个参数逐一为大家做个说明。

- FMC_NORSRAM_TypeDef *Instance

  用于驱动 SRAM 设备时的例化，主要用于寄存器 FMC_BCRx 和 FMC_BTRx 的配置。

- FMC_NORSRAM_EXTENDED_TypeDef *Extended;

  用于驱动 SRAM 设备时的扩展配置，通过设置寄存器 FMC_BWTRx 实现。

- FMC_NORSRAM_InitTypeDef Init

  这个参数是用户接触最多的，用于配置 FMC 外接 SRAM 或者相同时序设备时的基本参数，像使用的 BANK 号，存储器位宽，是否使用地址数据复用等。结构体 FMC_NORSRAM_InitTypeDef 的详细介绍见本章 3.3 小节。

- HAL_LockTypeDef Lock

  __IO HAL_SRAM_StateTypeDef State

  这两个变量主要供函数内部使用。Lock 用于设置锁状态，而 State 用于设置 FMC 状态。

- MDMA_HandleTypeDef *hmdma

  用于关联 MDMA 句柄。

47.3.5 SDRAM 寄存器结构体

FMC 控制 SDRAM 相关的寄存器是通过 HAL 库中的结构体 FMC_SDRAM_TypeDef 定义的，在 stm32h743xx.h 中可以找到这个类型定义：

```
#define FMC_SDRAM_TypeDef     FMC_Bank5_6_TypeDef

typedef struct
{
  __IO uint32_t SDCR[2];     /*!< SDRAM Control registers ,      Address offset: 0x140-0x144  */
  __IO uint32_t SDTR[2];     /*!< SDRAM Timing registers ,       Address offset: 0x148-0x14C  */
  __IO uint32_t SDCMR;       /*!< SDRAM Command Mode register,    Address offset: 0x150  */
  __IO uint32_t SDRTR;       /*!< SDRAM Refresh Timer register,   Address offset: 0x154  */
  __IO uint32_t SDSR;        /*!< SDRAM Status register,          Address offset: 0x158  */
} FMC_Bank5_6_TypeDef;
```

下面再来看这几个寄存器的具体定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE           ((uint32_t)0x40000000)
#define D1_AHB1PERIPH_BASE    (PERIPH_BASE + 0x12000000)
#define FMC_R_BASE            (D1_AHB1PERIPH_BASE + 0x4000)

#define FMC_Bank1_R_BASE      (FMC_R_BASE + 0x0000)
#define FMC_Bank1E_R_BASE     (FMC_R_BASE + 0x0104)
#define FMC_Bank2_R_BASE      (FMC_R_BASE + 0x0060)
#define FMC_Bank3_R_BASE      (FMC_R_BASE + 0x0080)
```

```
#define FMC_Bank5_6_R_BASE     (FMC_R_BASE + 0x0140)

#define FMC_Bank1              ((FMC_Bank1_TypeDef *) FMC_Bank1_R_BASE)
#define FMC_Bank1E             ((FMC_Bank1E_TypeDef *) FMC_Bank1E_R_BASE)
#define FMC_Bank2              ((FMC_Bank2_TypeDef *) FMC_Bank2_R_BASE)
#define FMC_Bank3              ((FMC_Bank3_TypeDef *) FMC_Bank3_R_BASE)
<-----展开下面的宏定义，(FMC_Bank1_TypeDef *) 0x52004140
#define FMC_Bank5_6            ((FMC_Bank5_6_TypeDef *) FMC_Bank5_6_R_BASE)

#define FMC_NORSRAM_DEVICE             FMC_Bank1
#define FMC_NORSRAM_EXTENDED_DEVICE    FMC_Bank1E
#define FMC_NAND_DEVICE                FMC_Bank3
#define FMC_SDRAM_DEVICE               FMC_Bank5_6
```

我们访问 FMC 的 SDRTR 寄存器可以采用这种形式：FMC->SDRTR= 0。

47.3.6SDRAM 参数配置结构体 FMC_SDRAM_InitTypeDef

这个结构体比较重要，FMC 驱动 SDRAM 设备的参数配置，全靠这个结构体实现，具体定义如下：

```
typedef struct
{
  uint32_t SDBank;
  uint32_t ColumnBitsNumber;
  uint32_t RowBitsNumber;
  uint32_t MemoryDataWidth;
  uint32_t InternalBankNumber;
  uint32_t CASLatency;
  uint32_t WriteProtection;
  uint32_t SDClockPeriod;
  uint32_t ReadBurst;
  uint32_t ReadPipeDelay;
}FMC_SDRAM_InitTypeDef;
```

下面将这几个成员逐一为大家做个介绍：

- uint32_t NSBank

用于配置使用的 SDRAM BANK，具体支持的参数如下：

```
#define FMC_SDRAM_BANK1                 ((uint32_t)0x00000000U)
#define FMC_SDRAM_BANK2                 ((uint32_t)0x00000001U)
```

- uint32_t ColumnBitsNumber

用于设置列地址的位数，具体支持的参数如下：

```
#define FMC_SDRAM_COLUMN_BITS_NUM_8     ((uint32_t)0x00000000U)
#define FMC_SDRAM_COLUMN_BITS_NUM_9     ((uint32_t)0x00000001U)
#define FMC_SDRAM_COLUMN_BITS_NUM_10    ((uint32_t)0x00000002U)
#define FMC_SDRAM_COLUMN_BITS_NUM_11    ((uint32_t)0x00000003U)
```

- uint32_t RowBitsNumber

用于设置行地址的位数，具体支持的参数如下：

```
#define FMC_SDRAM_ROW_BITS_NUM_11       ((uint32_t)0x00000000U)
#define FMC_SDRAM_ROW_BITS_NUM_12       ((uint32_t)0x00000004U)
#define FMC_SDRAM_ROW_BITS_NUM_13       ((uint32_t)0x00000008U)
```

- uint32_t MemoryDataWidth

用于设置 SDRAM 的数据带宽，具体支持的参数如下：

```
#define FMC_SDRAM_MEM_BUS_WIDTH_8              ((uint32_t)0x00000000U)
#define FMC_SDRAM_MEM_BUS_WIDTH_16             ((uint32_t)0x00000010U)
#define FMC_SDRAM_MEM_BUS_WIDTH_32             ((uint32_t)0x00000020U)
```

- uint32_t InternalBankNumber

用于设置 SDRAM 的内部 BANK 个数，具体支持的参数如下：

```
#define FMC_SDRAM_INTERN_BANKS_NUM_2           ((uint32_t)0x00000000U)
#define FMC_SDRAM_INTERN_BANKS_NUM_4           ((uint32_t)0x00000040U)
```

- uint32_t CASLatency

用于设置 SDRAM 的 CAS 延迟，具体支持的参数如下：

```
#define FMC_SDRAM_CAS_LATENCY_1                ((uint32_t)0x00000080U)
#define FMC_SDRAM_CAS_LATENCY_2                ((uint32_t)0x00000100U)
#define FMC_SDRAM_CAS_LATENCY_3                ((uint32_t)0x00000180)
```

- uint32_t WriteProtection

用于使能或者禁止 SDRAM 的写保护，具体支持的参数如下：

```
#define FMC_SDRAM_WRITE_PROTECTION_DISABLE     ((uint32_t)0x00000000U)
#define FMC_SDRAM_WRITE_PROTECTION_ENABLE      ((uint32_t)0x00000200U)
```

- uint32_t SDClockPeriod

用于设置 SDRAM 的时钟频率，可以选择主频的 2 分频或者 3 分频。由于 FMC 最高支持 200MHz，所有 SDRAM 最高 100MHz。此成员具体支持的参数如下：

```
#define FMC_SDRAM_CLOCK_DISABLE                ((uint32_t)0x00000000U)
#define FMC_SDRAM_CLOCK_PERIOD_2               ((uint32_t)0x00000800U)
#define FMC_SDRAM_CLOCK_PERIOD_3               ((uint32_t)0x00000C00)
```

- uint32_t ReadBurst

此函数用于使能读突发，借助读突发，SDRAM 可以在 CAS 延迟期间接受下一个读命令并将数据存储在读 FIFO 中。此成员具体支持的参数如下：

```
#define FMC_SDRAM_RBURST_DISABLE               ((uint32_t)0x00000000U)
#define FMC_SDRAM_RBURST_ENABLE                ((uint32_t)0x00001000U)
```

- uint32_t ReadPipeDela

此成员用于定义在 CAS 延时后多少个 SDRAM 时钟周期读取数据，具体支持的参数如下：

```
#define FMC_SDRAM_RPIPE_DELAY_0                ((uint32_t)0x00000000U)
#define FMC_SDRAM_RPIPE_DELAY_1                ((uint32_t)0x00002000U)
#define FMC_SDRAM_RPIPE_DELAY_2                ((uint32_t)0x00004000U)
```

47.3.7SDRAM 时序配置结构体 FMC_SDRAM_TimingTypeDef

此结构体对于配置 SDRAM 的时序参数，比较重要，具体定义如下：

```
typedef struct
{
  uint32_t LoadToActiveDelay;
  uint32_t ExitSelfRefreshDelay;
  uint32_t SelfRefreshTime;
  uint32_t RowCycleDelay;
  uint32_t WriteRecoveryTime;
  uint32_t RPDelay;
```

```
  uint32_t RCDDelay;
}FMC_SDRAM_TimingTypeDef;
```

下面将这几个参数逐一为大家做个说明（默认情况下 FMC 用的 HCLKx 要修改，主频 200MHz）

- uint32_t LoadToActiveDelay

  此成员用于定义加载模式寄存器的命令与激活命令或刷新命令之间的延迟，单位 SDRAM 时钟周期个数，范围 1 – 16。

- uint32_t ExitSelfRefreshDelay

  此成员用于定义从发出自刷新命令到发出激活命令之间的延迟，单位 SDRAM 时钟周期个数，范围 1 – 16。

- uint32_t SelfRefreshTime

  此成员用于定义最短的自刷新周期，单位 SDRAM 时钟周期个数，范围 1 – 16。

- uint32_t RowCycleDelay

  此成员用于定义刷新命令和激活命令之间的延迟，以及两个相邻刷新命令之间的延迟，单位 SDRAM 时钟周期个数，范围 1 – 16。

- uint32_t WriteRecoveryTime

  此成员用于定义在写命令和预充电命令之间的延迟，单位 SDRAM 时钟周期个数，范围 1 – 16。

- uint32_t RPDelay

  此成员用于定义预充电命令与其它命令之间的延迟，单位 SDRAM 时钟周期个数，范围 1 – 16。

- uint32_t RCDDelay

  此成员用于定义激活命令与读/写命令之间的延迟，单位 SDRAM 时钟周期个数，范围 1 – 16。

47.3.8SDRAM 句柄结构体 SDRAM_HandleTypeDef

HAL 库在 FMC_SDRAM_TypeDef 和 FMC_SDRAM_InitTypeDef 的基础上封装了一个结构体 SDRAM_HandleTypeDef，定义如下：

```
typedef struct
{
  FMC_SDRAM_TypeDef             *Instance;
  FMC_SDRAM_InitTypeDef         Init;
  __IO HAL_SDRAM_StateTypeDef   State;
  HAL_LockTypeDef               Lock;
  MDMA_HandleTypeDef           *hmdma;
}SDRAM_HandleTypeDef;
```

下面将这几个参数逐一为大家做个说明。

- FMC_SDRAM_TypeDef *Instance

  用于驱动 SDRAM 设备时的例化，主要用于寄存器配置。

- FMC_SDRAM_InitTypeDef Init;

这个参数是用户接触最多的，用于配置 FMC 外接 SDRAM 时的基本参数，像使用的 BANK 号，存储器位宽，行列个数等。结构体 FMC_NORSRAM_InitTypeDef 的详细介绍见本章 3.6 小节。

- HAL_LockTypeDef Lock
  __IO HAL_SRAM_StateTypeDef State
  这两个变量主要供函数内部使用。Lock 用于设置锁状态，而 State 用于设置 FMC 状态。
- MDMA_HandleTypeDef *hmdma
  用于关联 MDMA 句柄。

47.4 源文件 stm32h7xx_ll_fmc.c

SRAM 类设备的 API 在源文件 stm32h7xx_hal_sram.c 文件里面，而 SDRAM 类设备的 API 在源文件 stm32h7xx_hal_sdram.c 文件里面。

这里把我们把如下两个常用到的函数做个说明：

- HAL_SRAM_Init
- HAL_SDRAM_Init

47.4.1 函数 HAL_SRAM_Init

函数原型：

```
HAL_StatusTypeDef HAL_SRAM_Init(SRAM_HandleTypeDef *hsram, FMC_NORSRAM_TimingTypeDef *Timing,
                                FMC_NORSRAM_TimingTypeDef *ExtTiming)
```

函数描述：

此函数用于初始化 SRAM 类设置，不限制必须是 SRAM。只要时序类似，均可使用。

函数参数：

- 第 1 个参数是 SRAM_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。结构体变量成员的详细介绍看本章 3.4 小节。
- 第 2 个参数是 FMC_NORSRAM_TimingTypeDef 类型结构体指针变量，用于配置 FMC 时序参数。结构体变量成员的详细介绍看本章 3.3 小节。
- 第 3 个参数类型同第 2 个，用于扩展模式。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: HC574_ConfigFMC
*    功能说明: 配置 FMC 并口访问时序
*    形    参:  无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
static void HC574_ConfigFMC(void)
{
    SRAM_HandleTypeDef hsram = {0};
    FMC_NORSRAM_TimingTypeDef SRAM_Timing = {0};

    hsram.Instance  = FMC_NORSRAM_DEVICE;
    hsram.Extended  = FMC_NORSRAM_EXTENDED_DEVICE;

   /* FMC 使用的 HCLK3，主频 200MHz，1 个 FMC 时钟周期就是 5ns */
    /* SRAM 总线时序配置 4-1-2-1-2-2 不稳定，5-2-2-1-2-2 稳定 */
    SRAM_Timing.AddressSetupTime = 5;  /* 5*5ns=25ns，地址建立时间，范围 0 -15 个 FMC 时钟周期个数 */
    SRAM_Timing.AddressHoldTime  = 2;  /* 地址保持时间，配置为模式 A 时，
                                          用不到此参数 范围 1 -15 个时钟周期个数 */
    SRAM_Timing.DataSetupTime          = 2;  /* 2*5ns=10ns，数据保持时间，范围 1 -255 个时钟周期个数 */
    SRAM_Timing.BusTurnAroundDuration  = 1;  /* 此配置用不到这个参数 */
    SRAM_Timing.CLKDivision            = 2;  /* 此配置用不到这个参数 */
    SRAM_Timing.DataLatency            = 2;  /* 此配置用不到这个参数 */
    SRAM_Timing.AccessMode             = FMC_ACCESS_MODE_A; /* 配置为模式 A */

    hsram.Init.NSBank             = FMC_NORSRAM_BANK1;               /* 使用的 BANK1，即使用的片选 FMC_NE1 */
    hsram.Init.DataAddressMux     = FMC_DATA_ADDRESS_MUX_DISABLE;    /* 禁止地址数据复用 */
    hsram.Init.MemoryType         = FMC_MEMORY_TYPE_SRAM;            /* 存储器类型 SRAM */
    hsram.Init.MemoryDataWidth    = FMC_NORSRAM_MEM_BUS_WIDTH_32;     /* 32 位总线宽度 */
    hsram.Init.BurstAccessMode    = FMC_BURST_ACCESS_MODE_DISABLE;   /* 关闭突发模式 */
    hsram.Init.WaitSignalPolarity = FMC_WAIT_SIGNAL_POLARITY_LOW;    /* 用于设置等待信号的极性，关闭突发模式，
                                                                        此参数无效 */
    hsram.Init.WaitSignalActive   = FMC_WAIT_TIMING_BEFORE_WS;       /* 关闭突发模式，此参数无效 */
    hsram.Init.WriteOperation     = FMC_WRITE_OPERATION_ENABLE;      /* 用于使能或者禁止写保护 */
    hsram.Init.WaitSignal         = FMC_WAIT_SIGNAL_DISABLE;         /* 关闭突发模式，此参数无效 */
    hsram.Init.ExtendedMode       = FMC_EXTENDED_MODE_DISABLE;       /* 禁止扩展模式 */
    hsram.Init.AsynchronousWait   = FMC_ASYNCHRONOUS_WAIT_DISABLE;   /* 用于异步传输期间，使能或者禁止等待信
                                                                        号，这里选择关闭 */
    hsram.Init.WriteBurst         = FMC_WRITE_BURST_DISABLE;         /* 禁止写突发 */
    hsram.Init.ContinuousClock    = FMC_CONTINUOUS_CLOCK_SYNC_ONLY; /* 仅同步模式才做时钟输出 */
   hsram.Init.WriteFifo          = FMC_WRITE_FIFO_ENABLE;           /* 使能写 FIFO */

    /* 初始化 SRAM 控制器 */
    if (HAL_SRAM_Init(&hsram, &SRAM_Timing, &SRAM_Timing) != HAL_OK)
    {
        /* 初始化错误 */
        Error_Handler(__FILE__, __LINE__);
    }
}
```

47.4.2 函数 HAL_SDRAM_Init

函数原型：

```
HAL_StatusTypeDef HAL_SDRAM_Init(SDRAM_HandleTypeDef *hsdram, FMC_SDRAM_TimingTypeDef *Timing)
```

函数描述：

此函数主要用于初始化 SDRAM。

函数参数：

- 第 1 个参数是 SDRAM 句柄，具体每个成员的介绍看本章 3.8 小节
- 第 2 个参数是 FMC_SDRAM_TimingTypeDef 类型结构体指针变量，用于配置 SDRAM 时序参数。结构体变量成员的详细介绍看本章 3.7 小节
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: vTaskLED
*    功能说明: LED 闪烁
*    形    参: pvParameters 是在创建该任务时传递的形参
*    返 回 值: 无
*   优 先 级: 2
*********************************************************************************************************
*/
static void vTaskLED(void *pvParameters)
{

/*
*********************************************************************************************************
*    函 数 名: bsp_InitExtSDRAM
*    功能说明: 配置连接外部 SDRAM 的 GPIO 和 FMC
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitExtSDRAM(void)
{
    SDRAM_HandleTypeDef      hsdram = {0};
     FMC_SDRAM_TimingTypeDef  SDRAM_Timing = {0};
     FMC_SDRAM_CommandTypeDef command = {0};


     /* FMC SDRAM 所涉及到 GPIO 配置 */
     SDRAM_GPIOConfig();

     /* SDRAM 配置 */
     hsdram.Instance = FMC_SDRAM_DEVICE;

     /*
       FMC 使用的 HCLK3 时钟，200MHz，用于 SDRAM 的话，至少 2 分频，也就是 100MHz，即 1 个 SDRAM 时钟周期是 10ns
       下面参数单位均为 10ns。
    */
     /* 20ns, TMRD 定义加载模式寄存器的命令与激活命令或刷新命令之间的延迟 */
     SDRAM_Timing.LoadToActiveDelay    = 2;
     SDRAM_Timing.ExitSelfRefreshDelay = 7; /* 70ns, TXSR 定义从发出自刷新命令到发出激活命令之间的延迟 */
     SDRAM_Timing.SelfRefreshTime      = 4; /* 50ns, TRAS 定义最短的自刷新周期 */
     SDRAM_Timing.RowCycleDelay        = 7; /* 70ns, TRC 定义刷新命令和激活命令之间的延迟 */
     SDRAM_Timing.WriteRecoveryTime    = 2; /* 20ns, TWR 定义在写命令和预充电命令之间的延迟 */
     SDRAM_Timing.RPDelay              = 2; /* 20ns, TRP 定义预充电命令与其它命令之间的延迟 */
     SDRAM_Timing.RCDDelay             = 2; /* 20ns, TRCD 定义激活命令与读/写命令之间的延迟 */

     hsdram.Init.SDBank             = FMC_SDRAM_BANK1;              /* 硬件设计上用的 BANK1 */
     hsdram.Init.ColumnBitsNumber   = FMC_SDRAM_COLUMN_BITS_NUM_9;  /* 9 列 */
```

```
    hsdram.Init.RowBitsNumber       = FMC_SDRAM_ROW_BITS_NUM_12;      /* 12 行 */
    hsdram.Init.MemoryDataWidth     = FMC_SDRAM_MEM_BUS_WIDTH_32; /* 32 位带宽 */
    hsdram.Init.InternalBankNumber = FMC_SDRAM_INTERN_BANKS_NUM_4;  /* SDRAM 有 4 个 BANK */
   /* CAS Latency 可以设置 Latency1，2 和 3，实际测试 Latency3 稳定 */
    hsdram.Init.CASLatency          = FMC_SDRAM_CAS_LATENCY_3;
    hsdram.Init.WriteProtection     = FMC_SDRAM_WRITE_PROTECTION_DISABLE; /* 禁止写保护 */
    /* FMC 时钟 200MHz，2 分频后给 SDRAM，即 100MHz */
    hsdram.Init.SDClockPeriod       = SDCLOCK_PERIOD;
    hsdram.Init.ReadBurst           = FMC_SDRAM_RBURST_ENABLE;        /* 使能读突发 */
   /* 此位定 CAS 延时后延后多少个 SDRAM 时钟周期读取数据，实际测此位可以设置无需延迟 */
    hsdram.Init.ReadPipeDelay       = FMC_SDRAM_RPIPE_DELAY_0;

    /* 配置 SDRAM 控制器基本参数 */
    if(HAL_SDRAM_Init(&hsdram, &SDRAM_Timing) != HAL_OK)
    {
        /* Initialization Error */
        Error_Handler(__FILE__, __LINE__);
    }

    /* 完成 SDRAM 序列初始化 */
    SDRAM_Initialization_Sequence(&hsdram, &command);
}
```

47.5 总结

本章节就为大家讲解这么多，FMC 涉及到的知识点比较多，而且实际项目中用到的地方也比较多，望初学者熟练运用。

第48章 STM32H7 的 FMC 总线应用之是 32 路高速 IO 扩展

本章教程为大家讲解利用 STM32H7 的 FMC 总线扩展出 32 路高速 IO，且使用简单，实际项目中也比较有实用价值。

48.1 初学者重要提示
48.2 FMC 扩展 IO 硬件设计
48.3 FMC 扩展 IO 驱动设计
48.4 FMC 扩展 IO 板级支持包（bsp_fmc_io.c）
48.5 FMC 扩展 IO 驱动移植和使用
48.6 实验例程设计框架
48.7 实验例程说明（MDK）
48.8 实验例程说明（IAR）
48.9 总结

48.1 初学者重要提示

- 学习本章节前,务必优先学习第 47 章,需要对 FMC 的基础知识和 HAL 库的几个常用 API 有个认识。
- 为什么要做 IO 扩展，不是已经用了 240 脚的 H743XIH6 吗？因为开发板使用了 32 位 SDRAM 和 RGB888 硬件接口，消耗 IO 巨大，所以必须得扩展了。
- 扩展的 32 路高速 IO 非常实用，且使用简单，只需初始下 FMC，32 路 IO 就可以随意使用了。当前的扩展方式只支持高速输出。
- FMC 总线扩展 32 路高速 IO 理解成 GPIO 的 ODR 寄存器就很简单了，其实就是一个东西。
  FMC 扩展 IO 是对地址 0x60001000 的 32bit 数据空间的 0 和 1 的操作。GPIOA 的 ODR 寄存器是对地址 0x40000000 + 0x18020000 + 0x14 空间的操作。但只能操作 16 个引脚。
  使用总线的优势就在这里了，相当于在 GPIOA 到 GPIOK 的基础上，又扩展出 GPIOL 和 GPIOM。

```
#define PERIPH_BASE           ((uint32_t)0x40000000)
#define D3_AHB1PERIPH_BASE    (PERIPH_BASE + 0x18020000)
#define GPIOA_BASE            (D3_AHB1PERIPH_BASE + 0x0000)
#define GPIOA                 ((GPIO_TypeDef *) GPIOA_BASE)

typedef struct
{
```

```
  __IO uint32_t MODER;    /*!< GPIO port mode register,               Address offset: 0x00      */
  __IO uint32_t OTYPER;   /*!< GPIO port output type register,        Address offset: 0x04      */
  __IO uint32_t OSPEEDR;  /*!< GPIO port output speed register,       Address offset: 0x08      */
  __IO uint32_t PUPDR;    /*!< GPIO port pull-up/pull-down register,  Address offset: 0x0C      */
  __IO uint32_t IDR;      /*!< GPIO port input data register,         Address offset: 0x10      */
  __IO uint32_t ODR;      /*!< GPIO port output data register,        Address offset: 0x14      */
  __IO uint16_t BSRRL;    /*!< GPIO port bit set/reset low register,  Address offset: 0x18      */
  __IO uint16_t BSRRH;    /*!< GPIO port bit set/reset high register, Address offset: 0x1A      */
  __IO uint32_t LCKR;     /*!< GPIO port configuration lock register, Address offset: 0x1C      */
  __IO uint32_t AFR[2];   /*!< GPIO alternate function registers,     Address offset: 0x20-0x24 */
} GPIO_TypeDef;
```

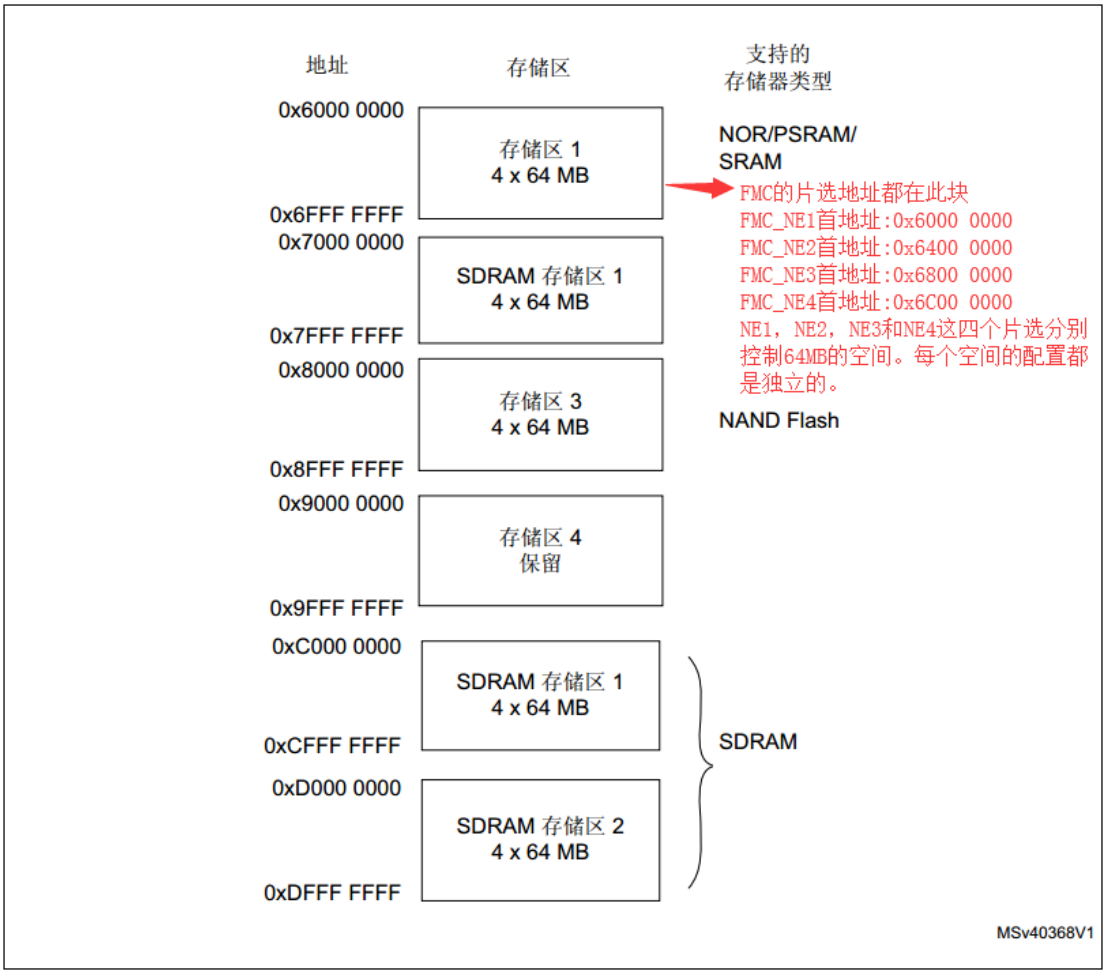
48.2 FMC 扩展 IO 硬件设计

扩展 IO 涉及到的知识点稍多，下面逐一为大家做个说明。

48.2.1 第 1 步，先来看 FMC 的块区分配

注，这个知识点在前面第 47 章的 2.3 小节有详细说明。

FMC 总线可操作的地址范围 0x60000000 到 0xDFFFFFFF，具体的框图如下：

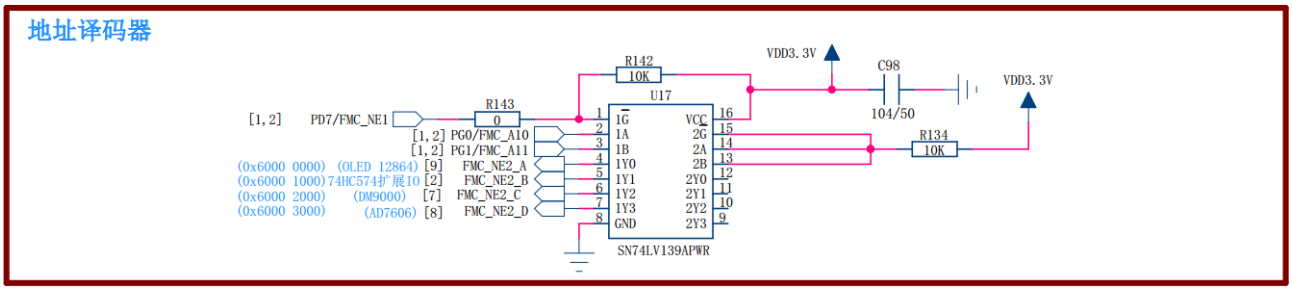


从上面的框图可以看出，NOR/PSRAM/SRAM 块区有 4 个片选 NE1，NE2，NE3 和 NE4，但由于引脚

复用，部分片选对应的引脚要用于其他功能，而且要控制的总线外设较多，导致片选不够用。因此需要增加译码器。

48.2.2 第 2 步，增加译码器及其地址计算

有了前面的认识之后再来看下面的译码器电路：



SN74LVC1G139APWR 是双 2-4 线地址译码器，也就是带了两个译码器。原理图上仅用了一个。下面是 139 的真值表和引脚功能：

INPUTS		OUTPUTS			
B	A	Y_0	Y_1	Y_2	Y_3
L	L	L	H	H	H
L	H	H	L	H	H
H	L	H	H	L	H
H	H	H	H	H	L

PIN		I/O	DESCRIPTION
NAME	NO.		
A	1	I	Adress input, bit 0
B	2	I	Adress input, bit 1
Y_3	3	O	Output 3, low when B is high and A is high
GND	4	—	Ground
Y_2	5	O	Output 2, low when B is high and A is low
Y_1	6	O	Output 1, low when B is low and A is high
Y_0	7	O	Output 0, low when B is low and A is low
V_{CC}	8	—	Power pin

通过上面的原理图和真值表就比较好理解了，真值表的输出是由片选 FMC_NE1 和地址线 FMC_A10、FMC_A11 控制。

FMC_NE1 输出低电平：

- FMC_A11（B），FMC_A10（A） = 00 时，1Y0 输出的低电平，选择的是 OLED。
- FMC_A11（B），FMC_A10（A） = 01 时，1Y1 输出的低电平，选择的是 74HC574。
- FMC_A11（B），FMC_A10（A） = 10 时，1Y2 输出的低电平，选择的是 DM9000。
- FMC_A11（B），FMC_A10（A） = 11 时，1Y3 输出的低电平，选择的是 AD7606。

然后我们再计算译码器的地址，注意，这里地址的计算都是按照 FMC 的 32bit 访问模式计算的，因为我们的 V7 程序中是将 NE1 对应的 FMC 配置为 32bit 模式了。

具体 FMC 的 32bit 访问模式，16bit 访问模式和 8bit 访问模式的区别在第 47 章的 2.4 小节有详细讲解。

表 248. NOR/PSRAM 外部存储器地址

存储器宽度(1)	向存储器发出的数据地址	最大存储器容量（位）
8 位	HADDR[25:0]	64 MB x 8 = 512 Mb
16 位	HADDR[25:1] >> 1	64 MB/2 x 16 = 512 Mb
32 位	HADDR[25:2] >> 2	64 MB/4 x 32 = 512 Mb

1. 如果外部存储器的宽度为 16 位，FMC 将使用内部的 HADDR[25:1] 地址来作为对外部存储器的寻址地址 FMC_A[24:0]。如果存储器宽度为 32 位，FMC 将使用内部的 HADDR[25:2] 地址进行外部寻址。无论外部存储器的宽度是多少，FMC_A[0] 都应连接到外部存储器地址 A[0]。

32bit 模式下，我们计算 A10 和 A11 的时候，实际上需要按 HADDR12 和 HADDR13 计算的。

如果来算 NE1 + HADDR12 + HADDR13 的四种组合地址就是如下：

NE1 + HADDR13 + HADDR12 = 0x60000000 + 0<<13 + 0<<12 = 0x60000000

NE1 + HADDR13 + HADDR12 = 0x60000000 + 0<<13 + 1<<12 = 0x60001000

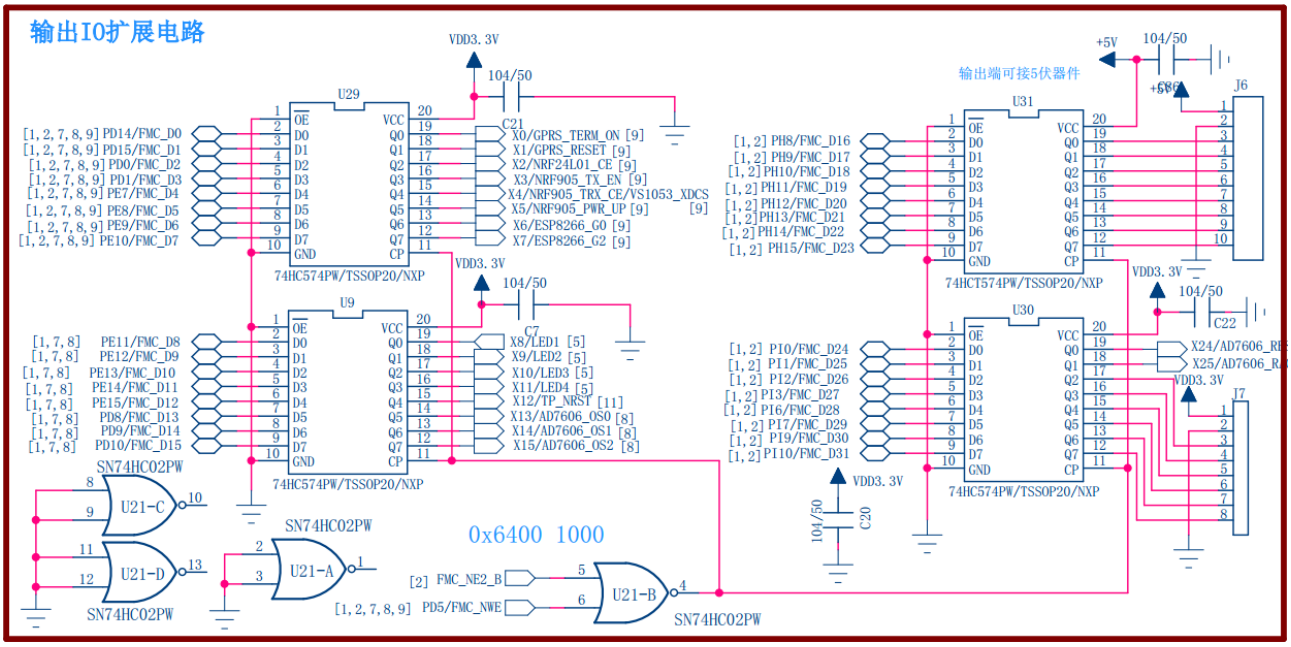
NE1 + HADDR13 + HADDR12 = 0x60000000 + 1<<13 + 0<<12 = 0x60002000

NE1 + HADDR13 + HADDR12 = 0x60000000 + 1<<13 + 1<<12 = 0x60003000

这样一来，原理图里面给的地址就对应上了。同理如果配置为 16 位模式和 8 位模式，大家应该也都会计算了。

48.2.3第 3 步，FMC 的 IO 扩展部分

先来看下 IO 扩展的原理图实现，如果不太了解 FMC 的通信时序和数字逻辑芯片的使用，可能会比较懵，下面逐一为大家说明。



有了这个原理图，首先要做的就是了解 74HC574 和 SN74HC02 的功能。

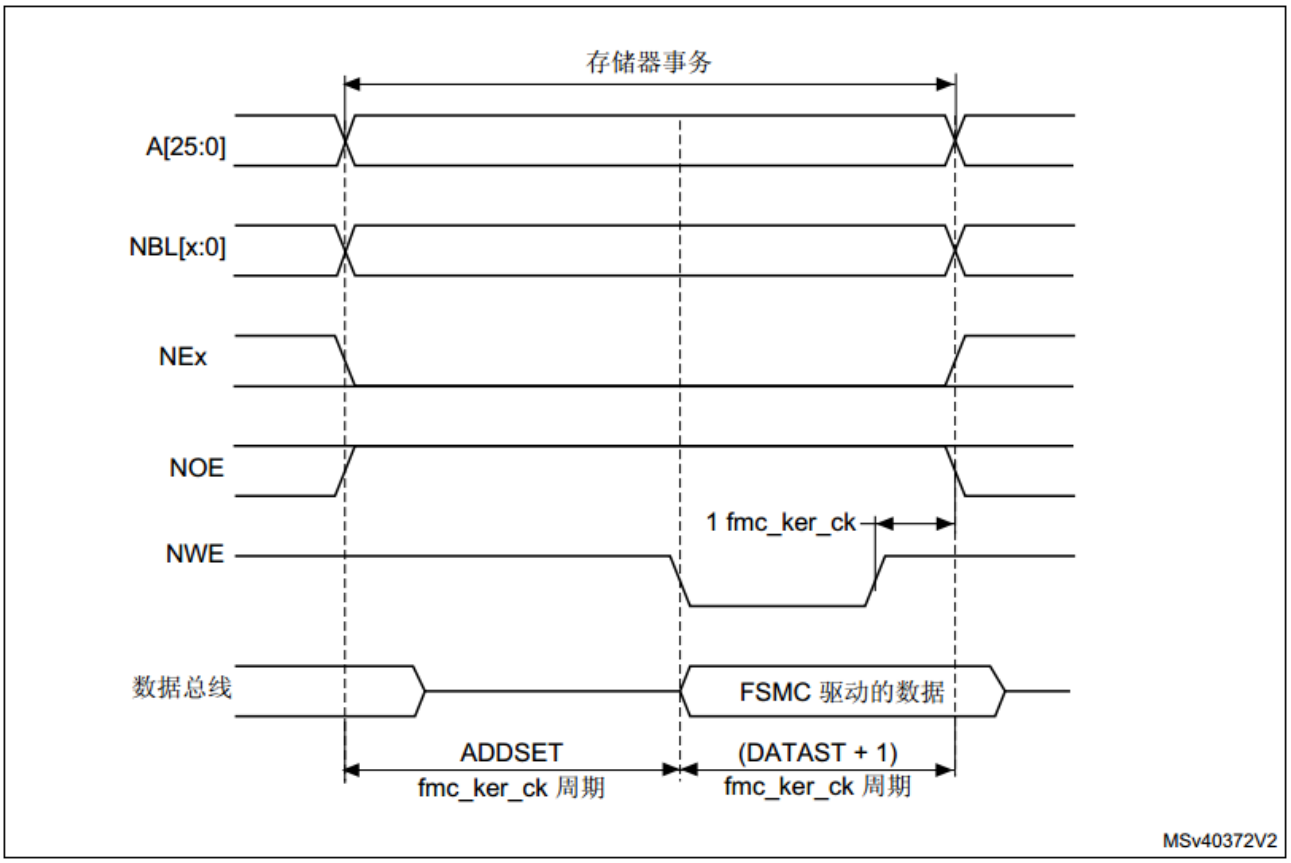
74HC574 是一款 8 位三态 D 触发器，起到锁存的功能，上升沿触发，对应的真值表如下（L 表示低电平，H 表示高电平，Z 表示高阻）：

Operating mode	Input			Internal flip-flop	Output
	OE	CP	Dn		Qn
Load and read register	L	↑	l	L	L
	L	↑	h	H	H
Load register and disable output	H	↑	l	L	Z
	H	↑	h	H	Z

SN74HC02 是一款 2 输入或非门，一个芯片带了四组或非门，对应的真值表如下（L 表示低电平，H 表示高电平）：

Input		Output
nA	nB	nY
L	L	H
X	H	L
H	X	L

有了这个认识后，我们再来看 FMC 的配置，V7 开发板的 BSP 驱动包里面专门做了一个 IO 扩展的 FMC 配置，即文件 bsp_fmc_io.c，配置方式是 FMC_AccessMode_A，这种模式对应的写时序是：



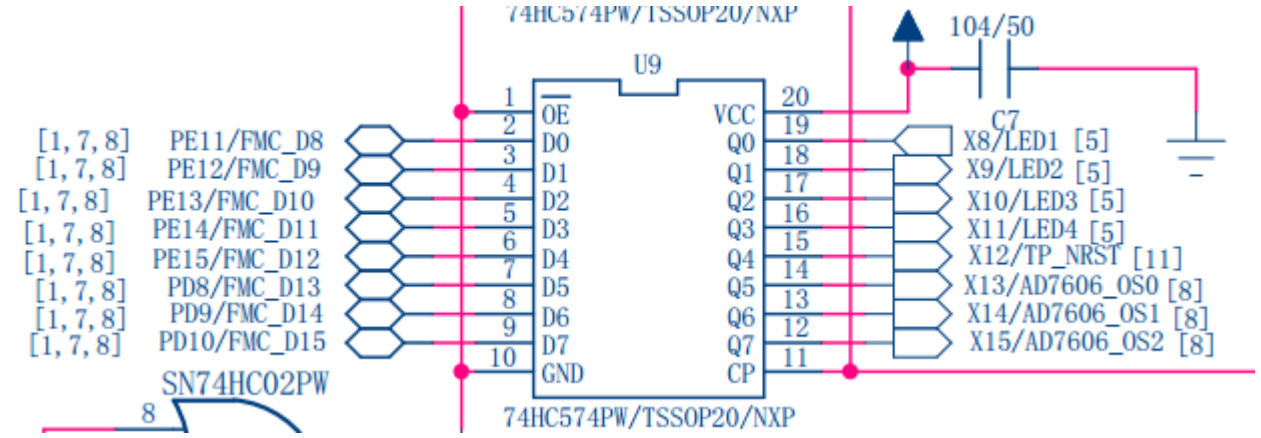
那么问题来了，我们要实现的功能是通过 FMC 输出的数据要锁存在扩展 IO 的输出端，否则 FMC 时序信号消失了，扩展 IO 的输出数据也消失了，就起不到控制作用了。所以就用到 74HC574 的锁存功能，而锁存的实现需要一个上升沿触发，这个上升沿就是通过 74HC02 输出的。

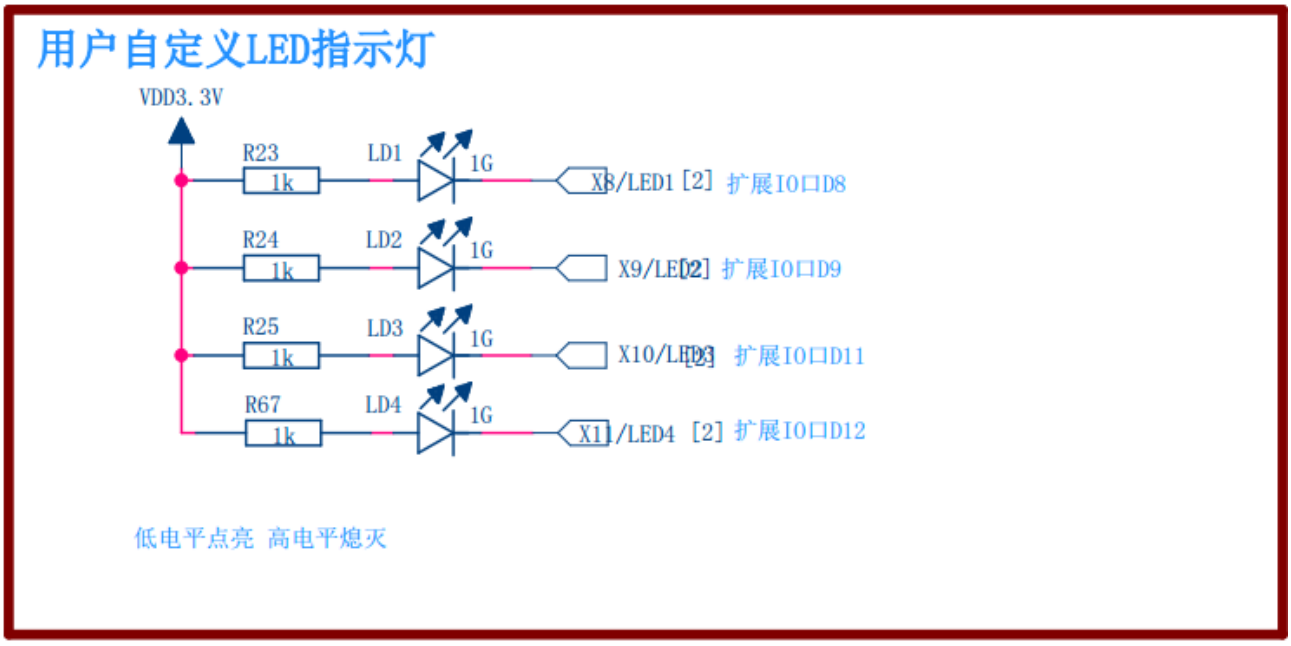
再结合上面 FMC 写时序图，在 NE 片选为低电平，NWE 写使能信号为高电平期间，即地址建立时间段 ADDSET 内，74HC02 是输出的低电平。

进入到 DATAST 数据建立阶段，在 NE 片选为低电平，NWE 写使能信号也为低电平时，74HC02 输出高电平，正好是实现 1 个上升沿的变化，将数据总线上的数据锁存到 74HC574 的输出端了。这里隐含了一个知识点，数据还没有完全建立起来就锁存是不是会有问题。在下面的 3.3 小节配置具体时序参数时再为大家说明。

48.2.4第 4 步，举例扩展 IO 驱动 LED 应用

进行到这里，再回过头来看 LED 驱动就比较好理解了。操作 LED 的亮灭就是操作 FMC 的数据引脚 D8，D9，D10 和 D11。





对地址 0x64001000 发送数据就可以了，但是如何对这个地址发送数据呢？ 反映到 C 语言的实现上就是通过固定地址的指针变量（跟我们操作寄存器是一样的），即

#define HC574_PORT *(uint32_t *)0x64001000

如果要点亮 LED1（低电平点亮），就是 HC574_PORT = 0x0000 0000。

如果要熄灭 LED1 就是 HC574_PORT = 0x0000 0100，即操作 FMC_D8 的高低电平即可。

48.3 FMC 扩展 IO 驱动设计

下面将程序设计中的相关问题逐一为大家做个说明。

48.3.1 FMC 扩展 IO 所涉及到的 GPIO 配置

这里仅需把用到的 GPIO 时钟、FMC 时钟、GPIO 引脚和复用配置好即可：

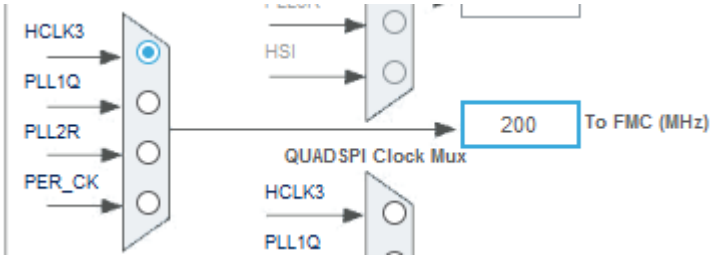
```
/*
```

```
*********************************************************************************************************
*    函 数 名: HC574_ConfigGPIO
*    功能说明: 配置 GPIO，FMC 管脚设置为复用功能
*    形    参:  无
*    返 回 值: 无
*********************************************************************************************************
*/
static void HC574_ConfigGPIO(void)
{
/*
    安富莱 STM32-H7 开发板接线方法：4 片 74HC574 挂在 FMC 32 位总线上。1 个地址端口可以扩展出 32 个 IO
    PD0/FMC_D2
    PD1/FMC_D3
    PD4/FMC_NOE        ---- 读控制信号，OE = Output Enable ，  N 表示低有效
    PD5/FMC_NWE        -XX- 写控制信号，AD7606 只有读，无写信号
    PD8/FMC_D13
    PD9/FMC_D14
    PD10/FMC_D15
    PD14/FMC_D0
    PD15/FMC_D1

    PE7/FMC_D4
    PE8/FMC_D5
    PE9/FMC_D6
    PE10/FMC_D7
    PE11/FMC_D8
    PE12/FMC_D9
    PE13/FMC_D10
    PE14/FMC_D11
    PE15/FMC_D12

    PG0/FMC_A10        --- 和主片选 FMC_NE2 一起译码
    PG1/FMC_A11        --- 和主片选 FMC_NE2 一起译码
    XX --- PG9/FMC_NE2      --- 主片选（OLED, 74HC574, DM9000, AD7606）
     --- PD7/FMC_NE1        --- 主片选（OLED, 74HC574, DM9000, AD7606）

     +------------------+------------------+
     +   32-bits Mode: D31-D16              +
     +------------------+------------------+
     | PH8 <-> FMC_D16   | PI0 <-> FMC_D24  |
     | PH9 <-> FMC_D17   | PI1 <-> FMC_D25  |
     | PH10 <-> FMC_D18  | PI2 <-> FMC_D26  |
     | PH11 <-> FMC_D19  | PI3 <-> FMC_D27  |
     | PH12 <-> FMC_D20  | PI6 <-> FMC_D28  |
     | PH13 <-> FMC_D21  | PI7 <-> FMC_D29  |
     | PH14 <-> FMC_D22  | PI9 <-> FMC_D30  |
     | PH15 <-> FMC_D23  | PI10 <-> FMC_D31 |
     +------------------+------------------+
*/

    GPIO_InitTypeDef gpio_init_structure;

    /* 使能 GPIO 时钟 */
    __HAL_RCC_GPIOD_CLK_ENABLE();
    __HAL_RCC_GPIOE_CLK_ENABLE();
    __HAL_RCC_GPIOG_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOI_CLK_ENABLE();
```

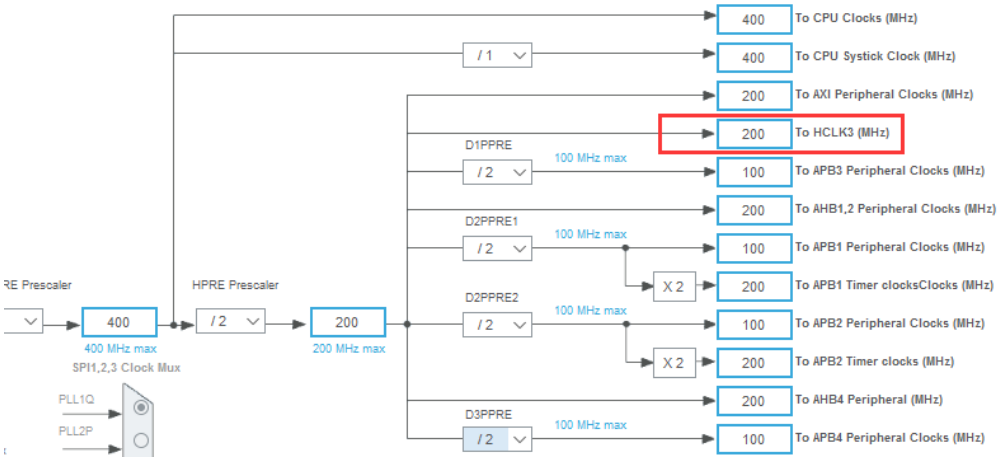
```
    /* 使能 FMC 时钟 */
    __HAL_RCC_FMC_CLK_ENABLE();

    /* 设置 GPIOD 相关的 IO 为复用推挽输出 */
    gpio_init_structure.Mode = GPIO_MODE_AF_PP;
    gpio_init_structure.Pull = GPIO_PULLUP;
    gpio_init_structure.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    gpio_init_structure.Alternate = GPIO_AF12_FMC;

    /* 配置 GPIOD */
    gpio_init_structure.Pin = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_7 |
                              GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 | GPIO_PIN_14 |
                              GPIO_PIN_15;
    HAL_GPIO_Init(GPIOD, &gpio_init_structure);

    /* 配置 GPIOE */
    gpio_init_structure.Pin = GPIO_PIN_7 | GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 |
                              GPIO_PIN_11 | GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14 |
                              GPIO_PIN_15;
    HAL_GPIO_Init(GPIOE, &gpio_init_structure);

    /* 配置 GPIOG */
    gpio_init_structure.Pin = GPIO_PIN_0 | GPIO_PIN_1;
    HAL_GPIO_Init(GPIOG, &gpio_init_structure);

    /* 配置 GPIOH */
    gpio_init_structure.Pin = GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 | GPIO_PIN_11 | GPIO_PIN_12
                            | GPIO_PIN_13 | GPIO_PIN_14 | GPIO_PIN_15;
    HAL_GPIO_Init(GPIOH, &gpio_init_structure);

    /* 配置 GPIOI */
    gpio_init_structure.Pin = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3 | GPIO_PIN_6
                            | GPIO_PIN_7 | GPIO_PIN_9 | GPIO_PIN_10;
    HAL_GPIO_Init(GPIOI, &gpio_init_structure);
}
```

48.3.2 FMC 扩展 IO 时钟源选择

使用 FMC 可以选择如下几种时钟源 HCLK3，PLL1Q，PLL2R 和 PER_CK：

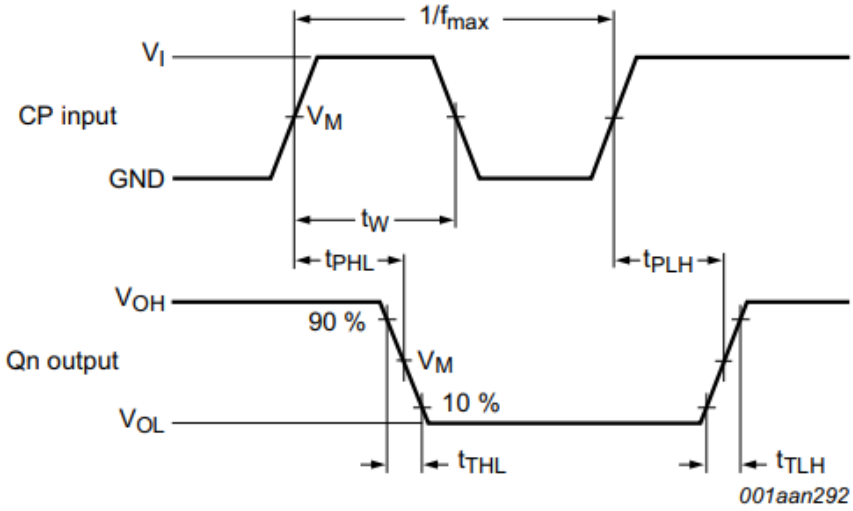


我们这里直接使用 HCLK3，配置 STM32H7 的主频为 400MHz 的时候，HCLK3 输出的 200MHz，这个速度是 FMC 支持的最高时钟，正好用于这里：



48.3.3 时序配置（重要）

这里要补充两个重要的知识点，74HC574 的 CP 端接收到上升沿触发到 Qn 输出的时间参数：



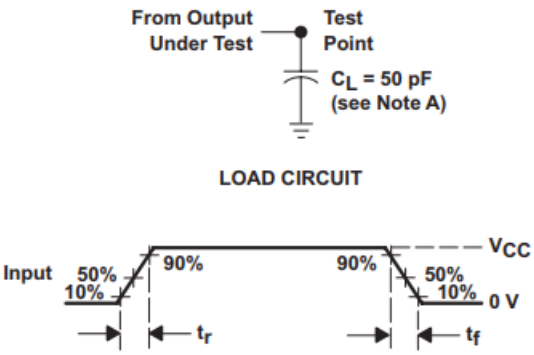
Voltages are referenced to GND (ground = 0 V); C_L = 50 pF unless otherwise specified; for test circuit see Figure 9.

Symbol	Parameter	Conditions	25 °C			–40 °C to +85 °C		–40 °C to +125 °C		Unit
			Min	Typ	Max	Min	Max	Min	Max	
74HC574										
t_{pd}	propagation delay	CP to Qn; see Figure 6 [1]								
		V_{CC} = 2.0 V	-	47	150	-	190	-	225	ns
		V_{CC} = 4.5 V	-	17	30	-	35	-	45	ns
		V_{CC} = 5 V; C_L = 15 pF	-	14	-	-	-	-	-	ns
		V_{CC} = 6.0 V	-	14	26	-	33	-	38	ns

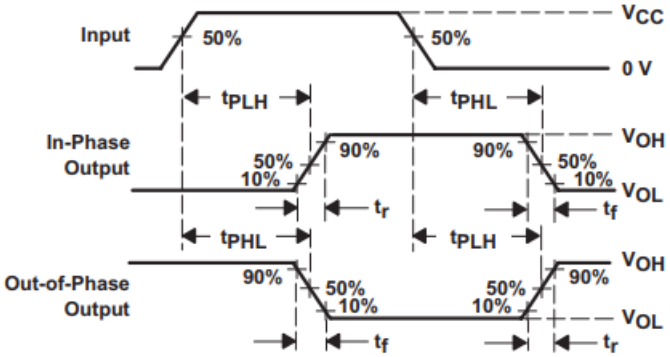
通过时序图和对应的参数要了解到以下几点：

- t_{pd} 传输延迟在这里等效于 t_{PHL} 和 t_{PLH}。
- V7 开发板的 74HC574 有三片是 3.3V 供电，另外一片是 5V 供电。参数表格里面没有给 3.3V 供电时的参数，也没有最小值。

了解了 74HC574，再来看 SN74HC02：



VOLTAGE WAVEFORM
INPUT RISE AND FALL TIMES



VOLTAGE WAVEFORMS
PROPAGATION DELAY AND OUTPUT TRANSITION TIMES

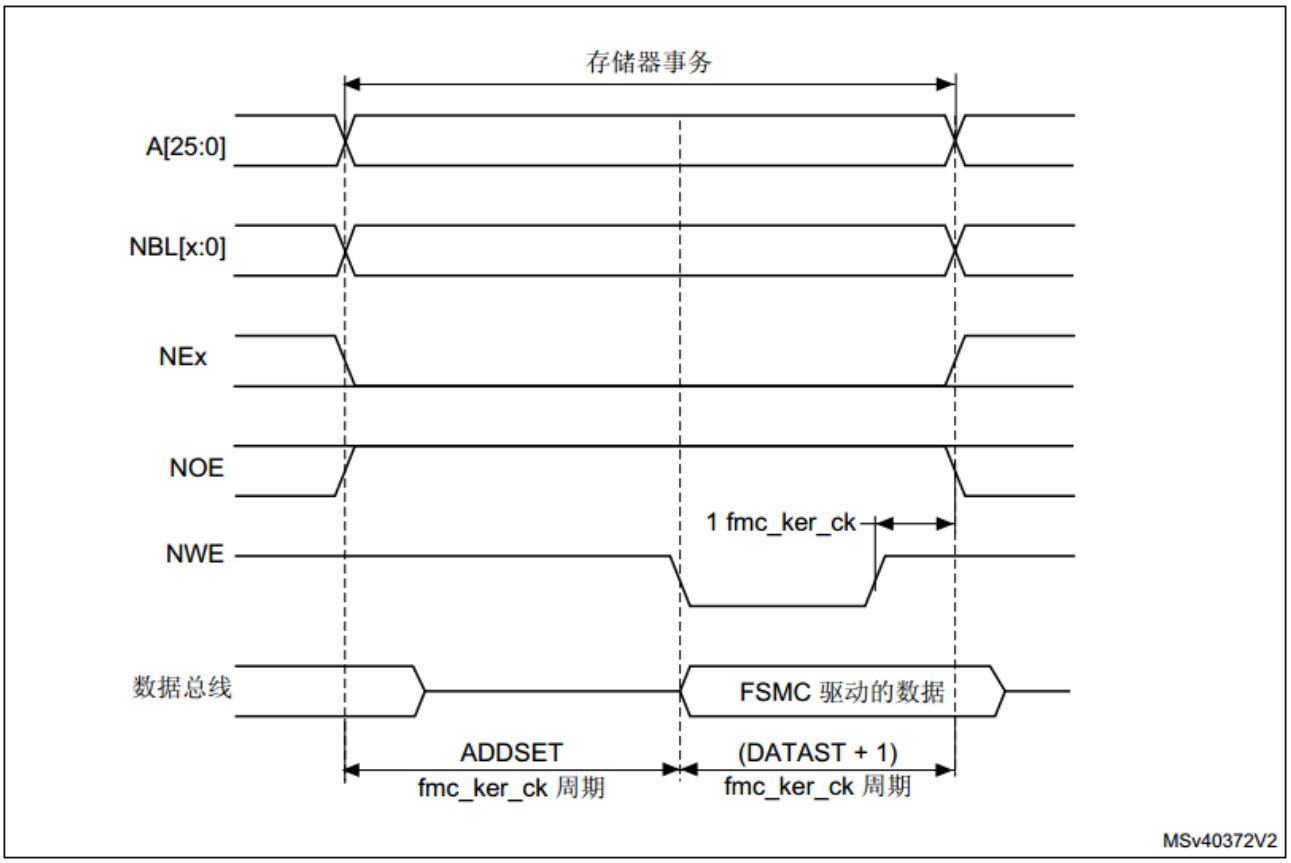
over recommended operating free-air temperature range, C_L = 50 pF (unless otherwise noted) (see Figure 2)

PARAMETER	FROM (INPUT)	TO (OUTPUT)	V_{CC}	T_A	MIN	TYP	MAX	UNIT
t_{pd}	A or B	Y	2 V	T_A = 25°C		45	90	ns
				SN54HC02			135	
				SN74HC02			115	
			4.5 V	T_A = 25°C		9	18	
				SN54HC02			27	
				SN74HC02			23	
			6 V	T_A = 25°C		8	15	
				SN54HC02			23	
				SN74HC02			20	
t_t	A or B	Y	2 V	T_A = 25°C		38	75	ns
				SN54HC02			110	
				SN74HC02			95	
			4.5 V	T_A = 25°C		8	15	
				SN54HC02			22	
				SN74HC02			19	
			6 V	T_A = 25°C		6	13	
				SN54HC02			19	
				SN74HC02			16	

通过时序图和对应的参数要了解到以下几点：

- t_{pd} 传输延迟在这里等效于 t_{PHL} 和 t_{PLH}。
- t_t 过渡时间等效于 t_r 上升沿时间和 t_f 下降沿时间。
- V7 开发板的 74HC574 有两片是 3.3V 供电，另外两片是 5V 供电。参数表格里面没有给 3.3V 和 5V 供电时的参数，也没有最小值。

对应 74HC574 和 74HC02 的时序参数有个了解后，再来看本章 2.3 小节末尾的问题：



当写使能信号 NWE 出现下降沿后，74H02 或非门就会输出一个上升沿，然后触发 74HC574 做锁存。此时我们要考虑到一个重要的知识点，就是使用的数字逻辑芯片有个传输延迟问题，也就是要我们要保证 74HC02 的 t_{pd} 传输延迟时间 + 74HC02 的 t_r 传输延迟时间 + 74HC574 的 t_{pd} 传输延迟时间的这段时间内，数据总线上要有数据，所以保证 DATAST 数据建立时间够大就行。实际测试 FMC 频率在 200MHz 的情况下，2-3 个 FMC 时钟周期就已经可以正常使用。

有了这些认识后，再来看 FMC 的时序配置就比较好理解了：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: HC574_ConfigFMC
4.  *    功能说明: 配置 FMC 并口访问时序
5.  *    形    参: 无
6.  *    返 回 值: 无
7.  *********************************************************************************************************
8.  */
9.  static void HC574_ConfigFMC(void)
10. {
11.     SRAM_HandleTypeDef hsram = {0};
12.     FMC_NORSRAM_TimingTypeDef SRAM_Timing = {0};
13.
14.     hsram.Instance  = FMC_NORSRAM_DEVICE;
15.     hsram.Extended  = FMC_NORSRAM_EXTENDED_DEVICE;
16.
17.    /* FMC 使用的 HCLK3，主频 200MHz，1 个 FMC 时钟周期就是 5ns */
18.     /* SRAM 总线时序配置 4-1-2-1-2-2 不稳定，5-2-2-1-2-2 稳定 */
19.     SRAM_Timing.AddressSetupTime    = 5;  /* 5*5ns=25ns，地址建立时间，范围 0 -15 个 FMC 时钟周期个数 */
20.     SRAM_Timing.AddressHoldTime     = 2;  /* 地址保持时间，配置为模式 A 时，用不到此参数 范围 1 -15 个时
```

```
21.                                                 钟周期个数 */
22.     SRAM_Timing.DataSetupTime          = 2;  /* 2*5ns=10ns，数据保持时间，范围 1 -255 个时钟周期个数 */
23.     SRAM_Timing.BusTurnAroundDuration  = 1;  /* 此配置用不到这个参数 */
24.     SRAM_Timing.CLKDivision            = 2;  /* 此配置用不到这个参数 */
25.     SRAM_Timing.DataLatency            = 2;  /* 此配置用不到这个参数 */
26.     SRAM_Timing.AccessMode             = FMC_ACCESS_MODE_A; /* 配置为模式 A */
27.
28.     hsram.Init.NSBank             = FMC_NORSRAM_BANK1;    /* 使用的 BANK1，即使用的片选 FMC_NE1 */
29.     hsram.Init.DataAddressMux     = FMC_DATA_ADDRESS_MUX_DISABLE;   /* 禁止地址数据复用 */
30.     hsram.Init.MemoryType         = FMC_MEMORY_TYPE_SRAM;           /* 存储器类型 SRAM */
31.     hsram.Init.MemoryDataWidth    = FMC_NORSRAM_MEM_BUS_WIDTH_32;    /* 32 位总线宽度 */
32.     hsram.Init.BurstAccessMode    = FMC_BURST_ACCESS_MODE_DISABLE;  /* 关闭突发模式 */
33.     hsram.Init.WaitSignalPolarity = FMC_WAIT_SIGNAL_POLARITY_LOW;   /* 用于设置等待信号的极性，关闭突
34.                                                                        发模式，此参数无效 */
35.     hsram.Init.WaitSignalActive   = FMC_WAIT_TIMING_BEFORE_WS;      /* 关闭突发模式，此参数无效 */
36.     hsram.Init.WriteOperation     = FMC_WRITE_OPERATION_ENABLE;     /* 用于使能或者禁止写保护 */
37.     hsram.Init.WaitSignal         = FMC_WAIT_SIGNAL_DISABLE;        /* 关闭突发模式，此参数无效 */
38.     hsram.Init.ExtendedMode       = FMC_EXTENDED_MODE_DISABLE;      /* 禁止扩展模式 */
39.     hsram.Init.AsynchronousWait   = FMC_ASYNCHRONOUS_WAIT_DISABLE;  /* 用于异步传输期间，使能或者禁止
40.                                                                        等待信号，这里选择关闭 */
41.     hsram.Init.WriteBurst         = FMC_WRITE_BURST_DISABLE;        /* 禁止写突发 */
42.     hsram.Init.ContinuousClock    = FMC_CONTINUOUS_CLOCK_SYNC_ONLY; /* 仅同步模式才做时钟输出 */
43.     hsram.Init.WriteFifo          = FMC_WRITE_FIFO_ENABLE;          /* 使能写 FIFO */
44.
45.     /* 初始化 SRAM 控制器 */
46.     if (HAL_SRAM_Init(&hsram, &SRAM_Timing, &SRAM_Timing) != HAL_OK)
47.     {
48.         /* 初始化错误 */
49.         Error_Handler(__FILE__, __LINE__);
50.     }
51. }
```

这里把几个关键的地方阐释下：

- 第 11 - 12 行，对作为局部变量的 HAL 库结构体做初始化，防止不确定值配置时出问题。
- 第 19 行，地址建立时间，对于 FMC 的 IO 扩展来说，这个地方取值 0 都可以，因为主要还是 ADDST 数据建立时间起作用。但是考虑到扩展 IO 外接了多个控制设备，这里取值 5 个 FMC 时钟周期，大家可以根据实际情况做减小出来。
- 第 20 行，地址保持时间，对于 FMC 模式 A 来说，此参数用不到。
- 第 22 行，数据建立时间，实际测试 2 个 FMC 时钟周期就可以正常使用，大家可以根据情况加大此数值。
- 第 23 – 25 行，当前配置用不到这三个参数。
- 第 28 行，使用的 BANK1，即使用的片选 FMC_NE1。
- 第 31 行，由于是扩展的 32 路 IO，所以这里要配置为 32 位带宽。

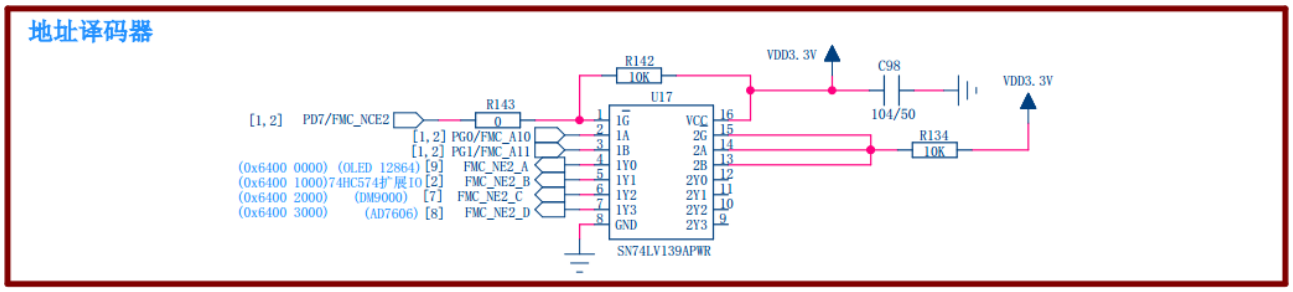
48.3.4 MPU 配置

实际测试发现，使能 FMC_NE1 所管理的存储区的 Cache 功能后，会出现扩展 IO 的 NE 片选和 NWE 信号输出 2 次的问题。经过各种 Cache 方式配置、FMC 带宽配置、操作 FMC 时的数据位宽设置，发现

禁止了 Cache 功能就正常了，也就是说，设置 FMC_NE1 所管理的存储区 MPU 属性为 Device 或者 Strongly Ordered 即可。

```
    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

MPU 配置中直接从 FMC_NE1 的首地址开始配置，设置了 64KB 空间的属性。将 FMC_NE1 通过译码器所管理的所有设备地址全部设置为此配置：



48.3.5 操作数据位宽注意事项

在 bsp_fmc_io.c 文件开头有个宏定义#define HC574_PORT *(uint32_t *)0x60001000。特别注意，这里是要操作地址 0x60001000 上的 32 位数据空间，即做了一个强制转换 uint32_t *，要跟 FMC 配置时设置的位宽一致。这样做的原因，在第 47 章的 2.6 小节有说明。

48.4 FMC 扩展 IO 板级支持包（bsp_fmc_io.c）

驱动文件 bsp_fmc_io.c 提供了如下几个函数供用户调用：

- bsp_InitExtIO
- HC574_SetPin
- HC574_TogglePin
- HC574_GetPin

48.4.1 函数 bsp_InitExtIO

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitExtIO
*    功能说明: 配置扩展 IO 相关的 GPIO. 上电只能执行一次。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitExtIO(void)
{
    HC574_ConfigGPIO();
    HC574_ConfigFMC();

    /* 将开发板一些片选，LED 口设置为高 */
    g_HC574 = (NRF24L01_CE | VS1053_XDCS | LED1 | LED2 | LED3 | LED4);
    HC574_PORT = g_HC574;  /* 写硬件端口，更改 IO 状态 */
}
```

函数描述：

此函数用于初始化 FMC 扩展 IO 所用到的 GPIO 和 FMC 的参数配置。

使用举例：

作为初始化函数，直接在在 bsp.c 文件的 bsp_Init 函数里面调用即可。

48.4.2 函数 HC574_SetPin

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: HC574_SetPin
*    功能说明: 设置 74HC574 端口值
*    形    参: _pin : 管脚号， 0-31; 只能选 1 个，不能多选
*              _value : 设定的值，0 或 1
*    返 回 值: 无
*********************************************************************************************************
*/
void HC574_SetPin(uint32_t _pin, uint8_t _value)
{
    if (_value == 0)
    {
        g_HC574 &= (~_pin);
    }
    else
    {
        g_HC574 |= _pin;
    }
    HC574_PORT = g_HC574;
}
```

函数描述：

此函数用于设置扩展 IO 的输出状态。调用此函数前，要保证调用了函数 bsp_InitExtIO 进行了初始化。

函数参数：

◆ 第 1 个参数是扩展 IO 的引脚,支持的形参如下,每次仅支持调用下面 1 个,不支持多个 IO 一起操作。

```
#define GPIO_PIN_0                 ((uint16_t)0x0001)  /* Pin 0 selected */
#define GPIO_PIN_1                 ((uint16_t)0x0002)  /* Pin 1 selected */
#define GPIO_PIN_2                 ((uint16_t)0x0004)  /* Pin 2 selected */
#define GPIO_PIN_3                 ((uint16_t)0x0008)  /* Pin 3 selected */
#define GPIO_PIN_4                 ((uint16_t)0x0010)  /* Pin 4 selected */
#define GPIO_PIN_5                 ((uint16_t)0x0020)  /* Pin 5 selected */
#define GPIO_PIN_6                 ((uint16_t)0x0040)  /* Pin 6 selected */
#define GPIO_PIN_7                 ((uint16_t)0x0080)  /* Pin 7 selected */
#define GPIO_PIN_8                 ((uint16_t)0x0100)  /* Pin 8 selected */
#define GPIO_PIN_9                 ((uint16_t)0x0200)  /* Pin 9 selected */
#define GPIO_PIN_10                ((uint16_t)0x0400)  /* Pin 10 selected */
#define GPIO_PIN_11                ((uint16_t)0x0800)  /* Pin 11 selected */
#define GPIO_PIN_12                ((uint16_t)0x1000)  /* Pin 12 selected */
#define GPIO_PIN_13                ((uint16_t)0x2000)  /* Pin 13 selected */
#define GPIO_PIN_14                ((uint16_t)0x4000)  /* Pin 14 selected */
#define GPIO_PIN_15                ((uint16_t)0x8000)  /* Pin 15 selected */
#define GPIO_PIN_16                ((uint32_t)0x00010000)  /* Pin 16 selected */
#define GPIO_PIN_17                ((uint32_t)0x00020000)  /* Pin 17 selected */
#define GPIO_PIN_18                ((uint32_t)0x00040000)  /* Pin 18 selected */
#define GPIO_PIN_19                ((uint32_t)0x00080000)  /* Pin 19 selected */
#define GPIO_PIN_20                ((uint32_t)0x00100000)  /* Pin 20 selected */
#define GPIO_PIN_21                ((uint32_t)0x00200000)  /* Pin 21 selected */
#define GPIO_PIN_22                ((uint32_t)0x00400000)  /* Pin 22 selected */
#define GPIO_PIN_23                ((uint32_t)0x00800000)  /* Pin 23 selected */
#define GPIO_PIN_24                ((uint32_t)0x01000000)  /* Pin 24 selected */
#define GPIO_PIN_25                ((uint32_t)0x02000000)  /* Pin 25 selected */
#define GPIO_PIN_26                ((uint32_t)0x04000000)  /* Pin 26 selected */
#define GPIO_PIN_27                ((uint32_t)0x08000000)  /* Pin 27 selected */
#define GPIO_PIN_28                ((uint32_t)0x10000000)  /* Pin 28 selected */
#define GPIO_PIN_29                ((uint32_t)0x20000000)  /* Pin 29 selected */
#define GPIO_PIN_30                ((uint32_t)0x40000000)  /* Pin 30 selected */
#define GPIO_PIN_31                ((uint32_t)0x80000000)  /* Pin 31 selected */
```

◆ 第 2 个参数用于设置指定扩展 IO 的高低电平，0 表示输出低电平，1 表示输出高电平。

使用举例：

比如设置扩展 IO 引脚 GPIO_PIN_23 为高电平：HC574_SetPin(GPIO_PIN_23, 1)。

48.4.3 函数 HC574_TogglePin

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: HC574_TogglePin
*    功能说明: 饭庄 74HC574 端口值
*    形    参: _pin : 管脚号， 0-31; 只能选 1 个，不能多选
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
void HC574_TogglePin(uint32_t _pin)
{
    if (g_HC574 & _pin)
    {
        g_HC574 &= (~_pin);
    }
    else
    {
        g_HC574 |= _pin;
    }
    HC574_PORT = g_HC574;
}
```

函数描述：

此函数用于 FMC 扩展 IO 的翻转。调用此函数前，要保证调用了函数 bsp_InitExtIO 进行了初始化。

函数参数：

◆ 第 1 个参数是扩展 IO 的引脚,支持的形参如下,每次仅支持调用下面 1 个,不支持多个 IO 一起操作。

```
#define GPIO_PIN_0                 ((uint16_t)0x0001)  /* Pin 0 selected */
#define GPIO_PIN_1                 ((uint16_t)0x0002)  /* Pin 1 selected */
#define GPIO_PIN_2                 ((uint16_t)0x0004)  /* Pin 2 selected */
#define GPIO_PIN_3                 ((uint16_t)0x0008)  /* Pin 3 selected */
#define GPIO_PIN_4                 ((uint16_t)0x0010)  /* Pin 4 selected */
#define GPIO_PIN_5                 ((uint16_t)0x0020)  /* Pin 5 selected */
#define GPIO_PIN_6                 ((uint16_t)0x0040)  /* Pin 6 selected */
#define GPIO_PIN_7                 ((uint16_t)0x0080)  /* Pin 7 selected */
#define GPIO_PIN_8                 ((uint16_t)0x0100)  /* Pin 8 selected */
#define GPIO_PIN_9                 ((uint16_t)0x0200)  /* Pin 9 selected */
#define GPIO_PIN_10                ((uint16_t)0x0400)  /* Pin 10 selected */
#define GPIO_PIN_11                ((uint16_t)0x0800)  /* Pin 11 selected */
#define GPIO_PIN_12                ((uint16_t)0x1000)  /* Pin 12 selected */
#define GPIO_PIN_13                ((uint16_t)0x2000)  /* Pin 13 selected */
#define GPIO_PIN_14                ((uint16_t)0x4000)  /* Pin 14 selected */
#define GPIO_PIN_15                ((uint16_t)0x8000)  /* Pin 15 selected */
#define GPIO_PIN_16                ((uint32_t)0x00010000)  /* Pin 16 selected */
#define GPIO_PIN_17                ((uint32_t)0x00020000)  /* Pin 17 selected */
#define GPIO_PIN_18                ((uint32_t)0x00040000)  /* Pin 18 selected */
#define GPIO_PIN_19                ((uint32_t)0x00080000)  /* Pin 19 selected */
#define GPIO_PIN_20                ((uint32_t)0x00100000)  /* Pin 20 selected */
#define GPIO_PIN_21                ((uint32_t)0x00200000)  /* Pin 21 selected */
#define GPIO_PIN_22                ((uint32_t)0x00400000)  /* Pin 22 selected */
#define GPIO_PIN_23                ((uint32_t)0x00800000)  /* Pin 23 selected */
#define GPIO_PIN_24                ((uint32_t)0x01000000)  /* Pin 24 selected */
#define GPIO_PIN_25                ((uint32_t)0x02000000)  /* Pin 25 selected */
#define GPIO_PIN_26                ((uint32_t)0x04000000)  /* Pin 26 selected */
#define GPIO_PIN_27                ((uint32_t)0x08000000)  /* Pin 27 selected */
#define GPIO_PIN_28                ((uint32_t)0x10000000)  /* Pin 28 selected */
#define GPIO_PIN_29                ((uint32_t)0x20000000)  /* Pin 29 selected */
#define GPIO_PIN_30                ((uint32_t)0x40000000)  /* Pin 30 selected */
#define GPIO_PIN_31                ((uint32_t)0x80000000)  /* Pin 31 selected */
```

使用举例：

比如翻转扩展 IO 引脚 GPIO_PIN_23 为高电平：HC574_TogglePin(GPIO_PIN_23)。

48.4.4函数 HC574_GetPin

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: HC574_GetPin
*    功能说明: 判断指定的管脚输出是 1 还是 0
*    形    参: _pin : 管脚号， 0-31; 只能选 1 个，不能多选
*    返 回 值: 0 或 1
*********************************************************************************************************
*/
uint8_t HC574_GetPin(uint32_t _pin)
{
    if (g_HC574 & _pin)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}
```

函数描述：

此函数用于读取 FMC 扩展 IO 的状态。调用此函数前，要保证调用了函数 bsp_InitExtIO 进行了初始化。

函数参数：

◆ 第 1 个参数是扩展 IO 的引脚,支持的形参如下,每次仅支持调用下面 1 个,不支持多个 IO 一起操作。

```
#define GPIO_PIN_0                 ((uint16_t)0x0001)  /* Pin 0 selected */
#define GPIO_PIN_1                 ((uint16_t)0x0002)  /* Pin 1 selected */
#define GPIO_PIN_2                 ((uint16_t)0x0004)  /* Pin 2 selected */
#define GPIO_PIN_3                 ((uint16_t)0x0008)  /* Pin 3 selected */
#define GPIO_PIN_4                 ((uint16_t)0x0010)  /* Pin 4 selected */
#define GPIO_PIN_5                 ((uint16_t)0x0020)  /* Pin 5 selected */
#define GPIO_PIN_6                 ((uint16_t)0x0040)  /* Pin 6 selected */
#define GPIO_PIN_7                 ((uint16_t)0x0080)  /* Pin 7 selected */
#define GPIO_PIN_8                 ((uint16_t)0x0100)  /* Pin 8 selected */
#define GPIO_PIN_9                 ((uint16_t)0x0200)  /* Pin 9 selected */
#define GPIO_PIN_10                ((uint16_t)0x0400)  /* Pin 10 selected */
#define GPIO_PIN_11                ((uint16_t)0x0800)  /* Pin 11 selected */
#define GPIO_PIN_12                ((uint16_t)0x1000)  /* Pin 12 selected */
#define GPIO_PIN_13                ((uint16_t)0x2000)  /* Pin 13 selected */
#define GPIO_PIN_14                ((uint16_t)0x4000)  /* Pin 14 selected */
#define GPIO_PIN_15                ((uint16_t)0x8000)  /* Pin 15 selected */
#define GPIO_PIN_16                ((uint32_t)0x00010000)  /* Pin 16 selected */
#define GPIO_PIN_17                ((uint32_t)0x00020000)  /* Pin 17 selected */
#define GPIO_PIN_18                ((uint32_t)0x00040000)  /* Pin 18 selected */
#define GPIO_PIN_19                ((uint32_t)0x00080000)  /* Pin 19 selected */
#define GPIO_PIN_20                ((uint32_t)0x00100000)  /* Pin 20 selected */
#define GPIO_PIN_21                ((uint32_t)0x00200000)  /* Pin 21 selected */
#define GPIO_PIN_22                ((uint32_t)0x00400000)  /* Pin 22 selected */
#define GPIO_PIN_23                ((uint32_t)0x00800000)  /* Pin 23 selected */
#define GPIO_PIN_24                ((uint32_t)0x01000000)  /* Pin 24 selected */
#define GPIO_PIN_25                ((uint32_t)0x02000000)  /* Pin 25 selected */
#define GPIO_PIN_26                ((uint32_t)0x04000000)  /* Pin 26 selected */
```

```
#define GPIO_PIN_27                 ((uint32_t)0x08000000)  /* Pin 27 selected */
#define GPIO_PIN_28                 ((uint32_t)0x10000000)  /* Pin 28 selected */
#define GPIO_PIN_29                 ((uint32_t)0x20000000)  /* Pin 29 selected */
#define GPIO_PIN_30                 ((uint32_t)0x40000000)  /* Pin 30 selected */
#define GPIO_PIN_31                 ((uint32_t)0x80000000)  /* Pin 31 selected */
```

- 返回值，返回 0 表示低电平，返回 1 表示高电平。

使用举例：

比如获取扩展 IO 的 GPIO_PIN_23 高低电平状态，调用函数 HC574_GetPin(GPIO_PIN_23)获取即可。

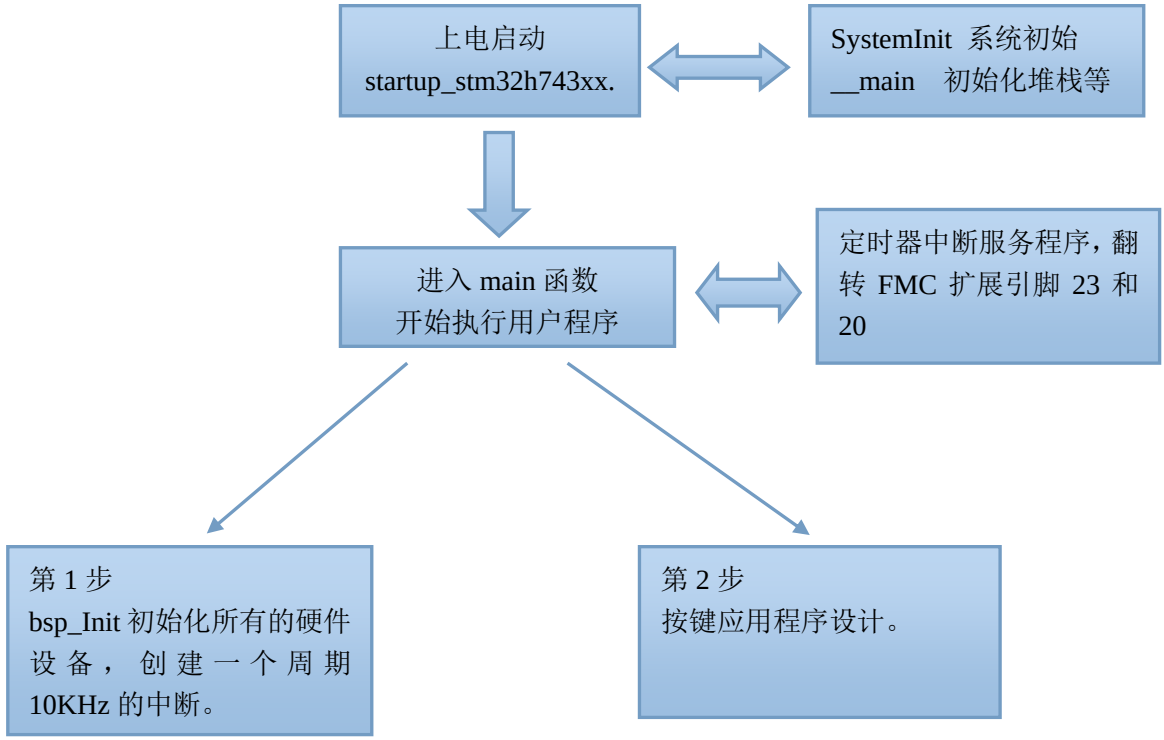
48.5 FMC 扩展 IO 驱动移植和使用

扩展 IO 的移植比较方便：

- 第 1 步：复制 bsp_fmc_io.c 和 bsp_fmc_io.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：这几个驱动文件主要用到 HAL 库的 GPIO 和 FMC 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 3 步，应用方法看本章节配套例子即可，另外就是根据自己的需要做配置修改。

48.6 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED 和串口。
  - 第 2 步，按键应用程序设计部分。定时器中断服务程序里面实现翻转 FMC 扩展引脚 20 和 23。

48.7 实验例程说明（MDK）

配套例子：

V7-027-FMC 总线扩展 32 路高速 IO

实验目的：

1. 学习 FMC 总线扩展 32 路高速 IO。

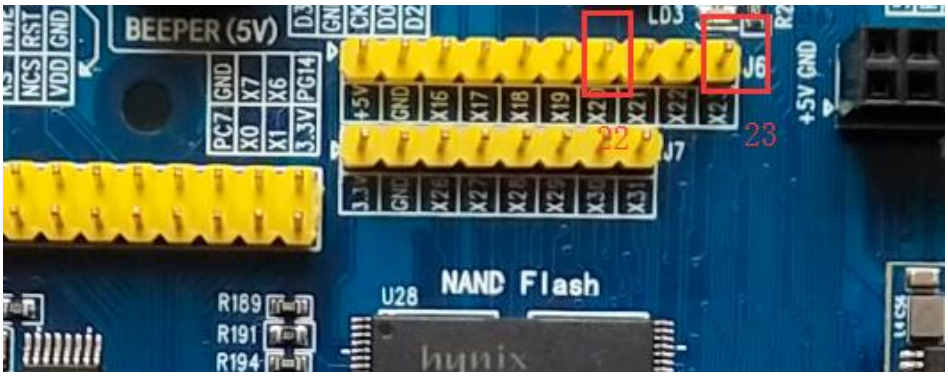
实验内容：

1. 系统上电后驱动了 1 个软件定时器，每 100ms 翻转一次 LED2。
2. 启动 1 个 TIM6 周期性中断，频率 10KHz，在中断服务程序里面翻转 FMC 扩展引脚 20 和 23。

实验操作：

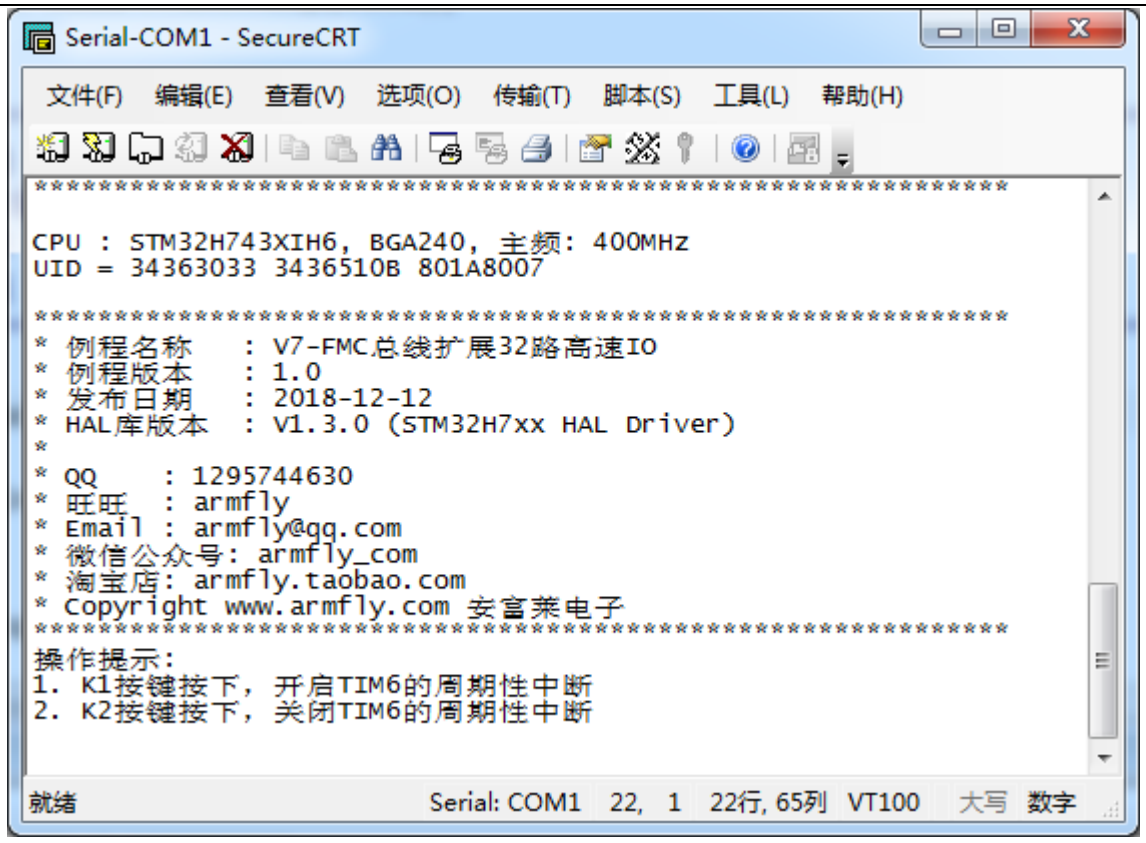
1. K1 按键按下，开启 TIM6 的周期性中断。
2. K2 按键按下，关闭 TIM6 的周期性中断。

FMC 扩展引脚 20 和 23 的位置：



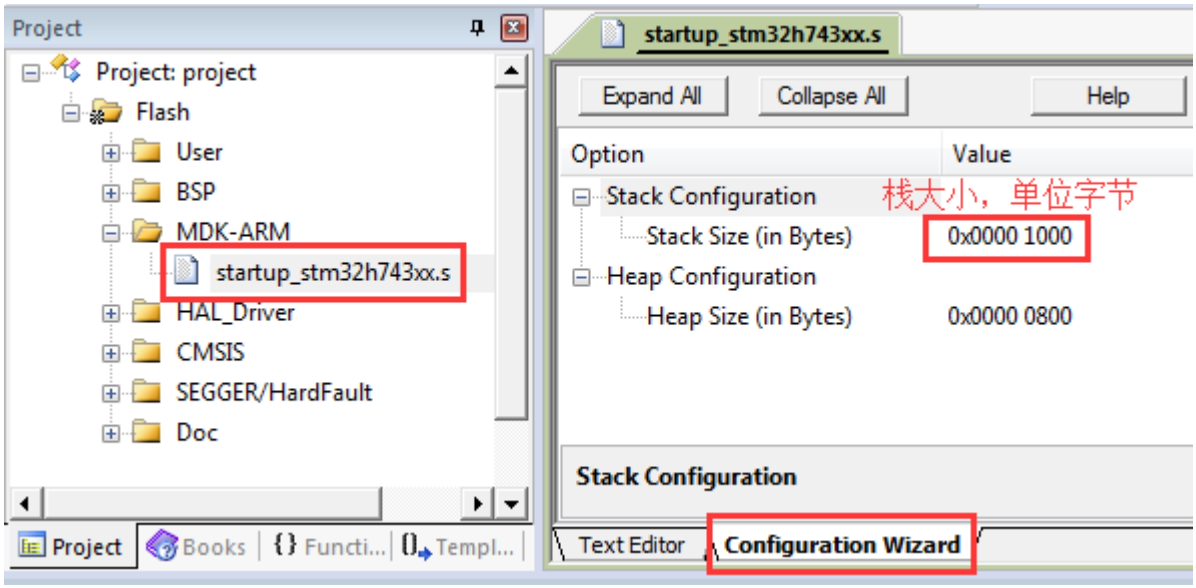
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

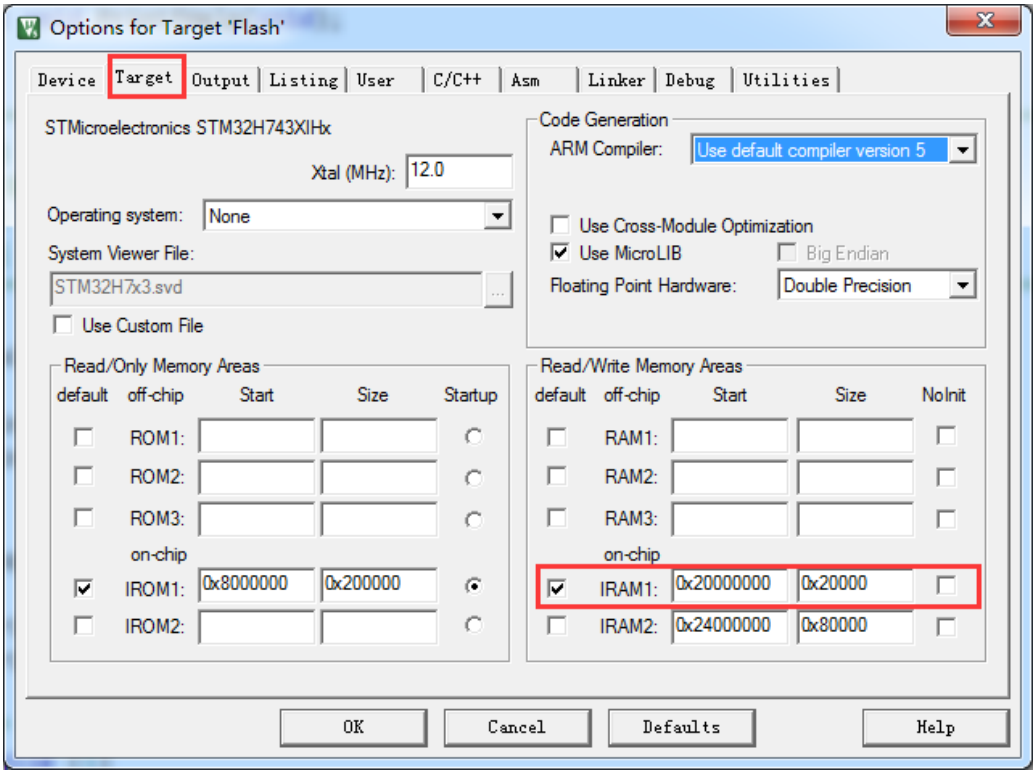


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
```

```
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- K1 按键按下，开启 TIM6 的周期性中断。
- K2 按键按下，关闭 TIM6 的周期性中断。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */
```

```
    bsp_SetTIMforInt(TIM6, 10000, 2, 0); /* 设置为 10KHz 频率定时器中断*/

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，开启 TIM6 的周期性中断*/
                    TIM6->DIER |= TIM_IT_UPDATE;
                    break;

                case KEY_DOWN_K2:           /* K2 键按下，关闭 TIM6 的周期性中断*/
                    TIM6->DIER &= ~TIM_IT_UPDATE;
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

定时器 6 中断服务程序：

```
/*
*********************************************************************************************************
*    函 数 名: TIM6_DAC_IRQHandler
*    功能说明: TIM6 定时中断服务程序
*    返 回 值: 无
*********************************************************************************************************
*/
void TIM6_DAC_IRQHandler(void)
{
    if((TIM6->SR & TIM_FLAG_UPDATE) != RESET)
    {
        /* 清除更新标志 */
        TIM6->SR = ~ TIM_FLAG_UPDATE;

        /* 翻转 FMC 扩展引脚 20 和 23 脚 */
        HC574_TogglePin(GPIO_PIN_23);
        HC574_TogglePin(GPIO_PIN_20);
    }
}
```

48.8 实验例程说明（IAR）

配套例子：

V7-027-FMC 总线扩展 32 路高速 IO

实验目的：

1. 学习 FMC 总线扩展 32 路高速 IO。

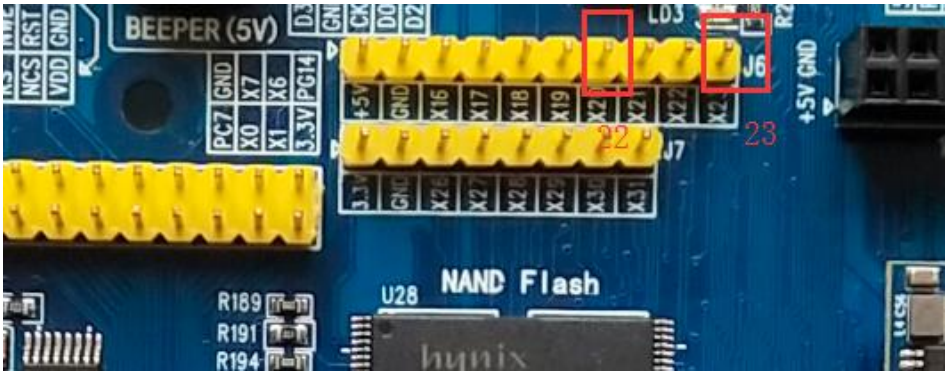
实验内容：

1. 系统上电后驱动了 1 个软件定时器，每 100ms 翻转一次 LED2。
2. 启动 1 个 TIM6 周期性中断，频率 10KHz，在中断服务程序里面翻转 FMC 扩展引脚 20 和 23。

实验操作：

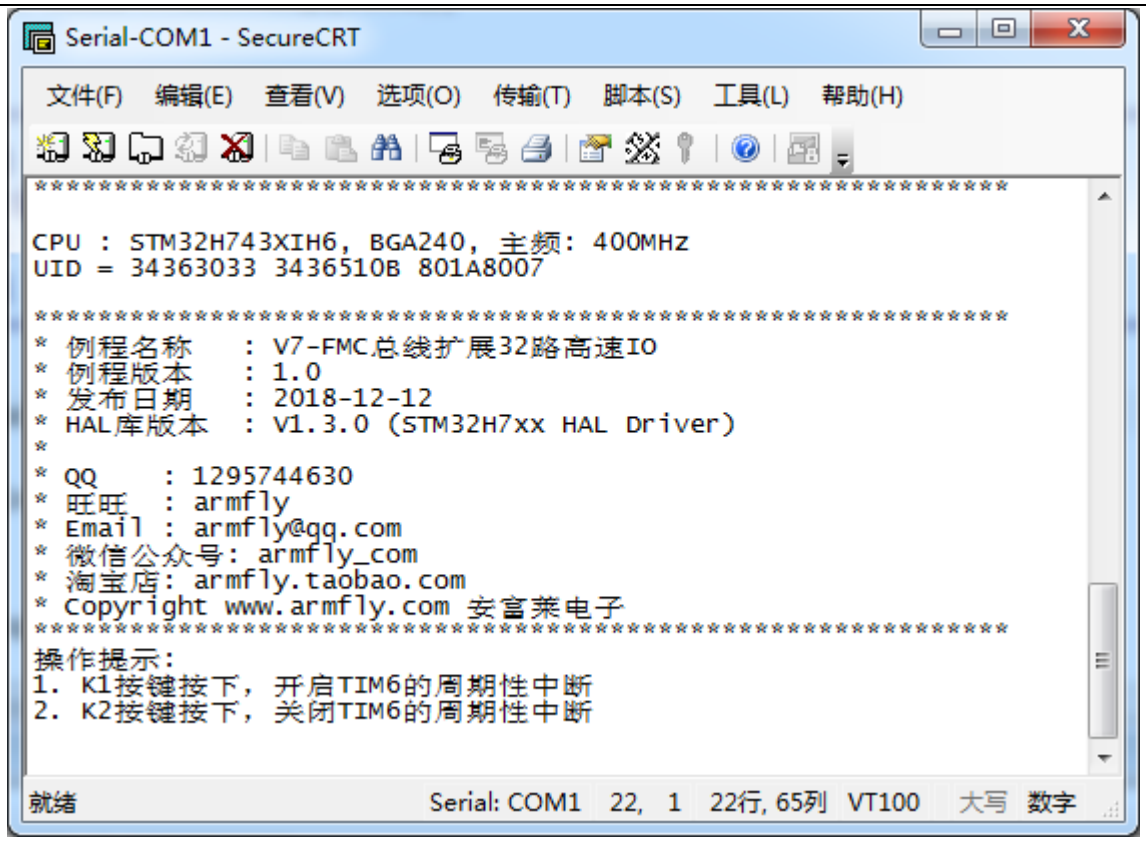
1. K1 按键按下，开启 TIM6 的周期性中断。
2. K2 按键按下，关闭 TIM6 的周期性中断。

FMC 扩展引脚 20 和 23 的位置：



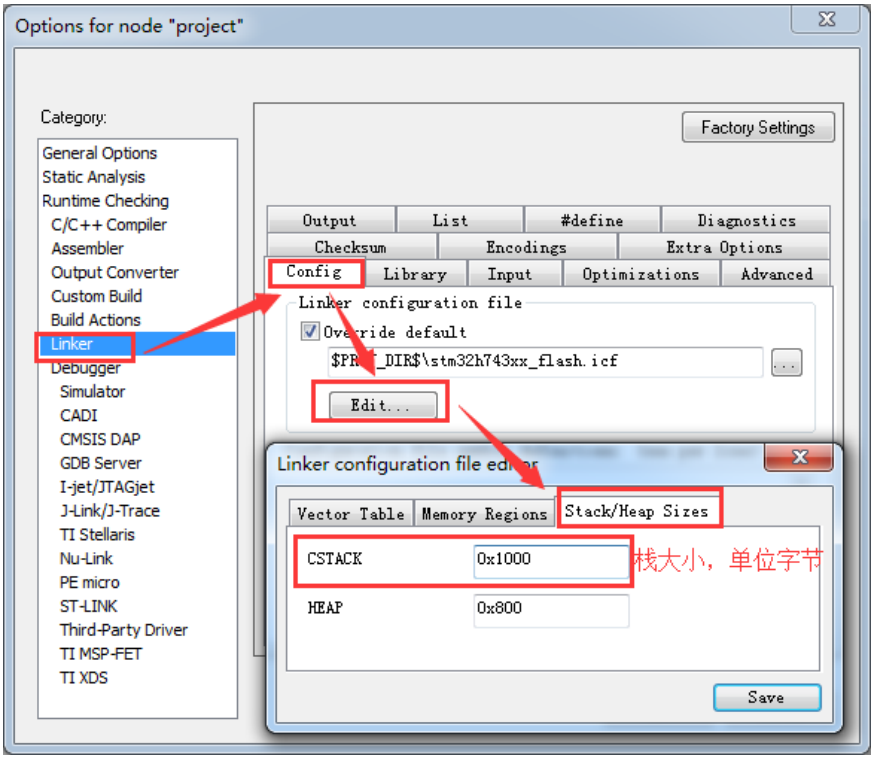
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

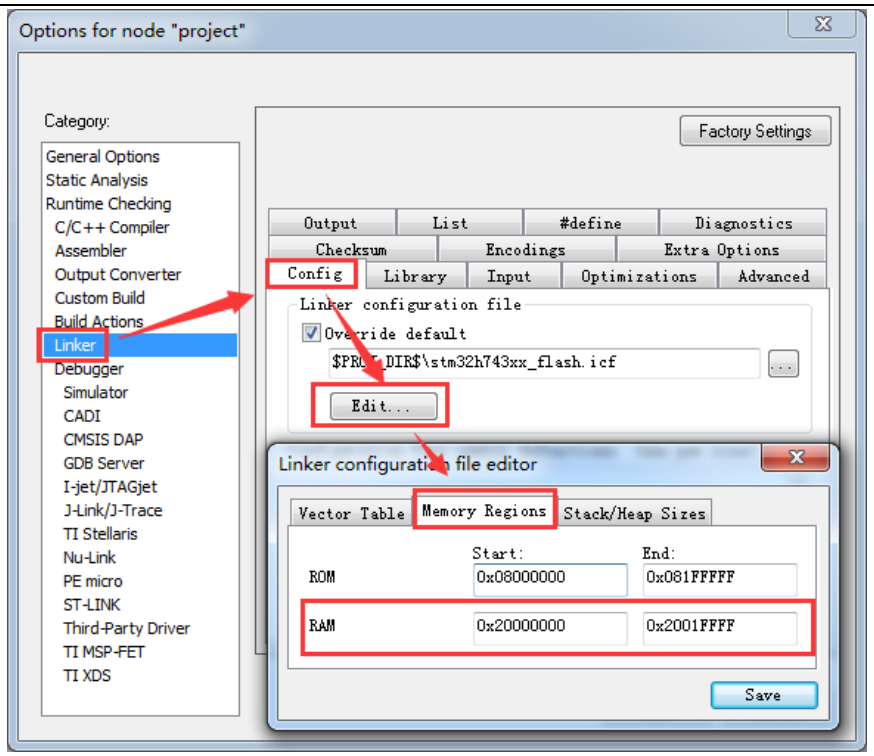


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
```

```
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能:**

主程序实现如下操作:

- K1 按键按下，开启 TIM6 的周期性中断。
- K2 按键按下，关闭 TIM6 的周期性中断。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */


    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    bsp_SetTIMforInt(TIM6, 10000, 2, 0); /* 设置为 10KHz 频率定时器中断*/
```

```
    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，开启 TIM6 的周期性中断*/
                    TIM6->DIER |= TIM_IT_UPDATE;
                    break;

                case KEY_DOWN_K2:           /* K2 键按下，关闭 TIM6 的周期性中断*/
                    TIM6->DIER &= ~TIM_IT_UPDATE;
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

定时器 6 中断服务程序：

```
/*
*********************************************************************************************************
*    函 数 名: TIM6_DAC_IRQHandler
*    功能说明: TIM6 定时中断服务程序
*    返 回 值: 无
*********************************************************************************************************
*/
void TIM6_DAC_IRQHandler(void)
{
    if((TIM6->SR & TIM_FLAG_UPDATE) != RESET)
    {
        /* 清除更新标志 */
        TIM6->SR = ~ TIM_FLAG_UPDATE;

        /* 翻转 FMC 扩展引脚 20 和 23 脚 */
        HC574_TogglePin(GPIO_PIN_23);
        HC574_TogglePin(GPIO_PIN_20);
    }
}
```

48.9 总结

本章节就为大家讲解这么多，由于 FMC 总线可以扩展出 32 路高速 IO 且使用简单，所以实际项目中也比较有实用价值，望初学者熟练掌握。

第49章 STM32H7 的 FMC 总线应用之 SDRAM

本章教程为大家讲解 SDRAM 的驱动实现，后面 LCD 的显存和大数量的存取都要用到。

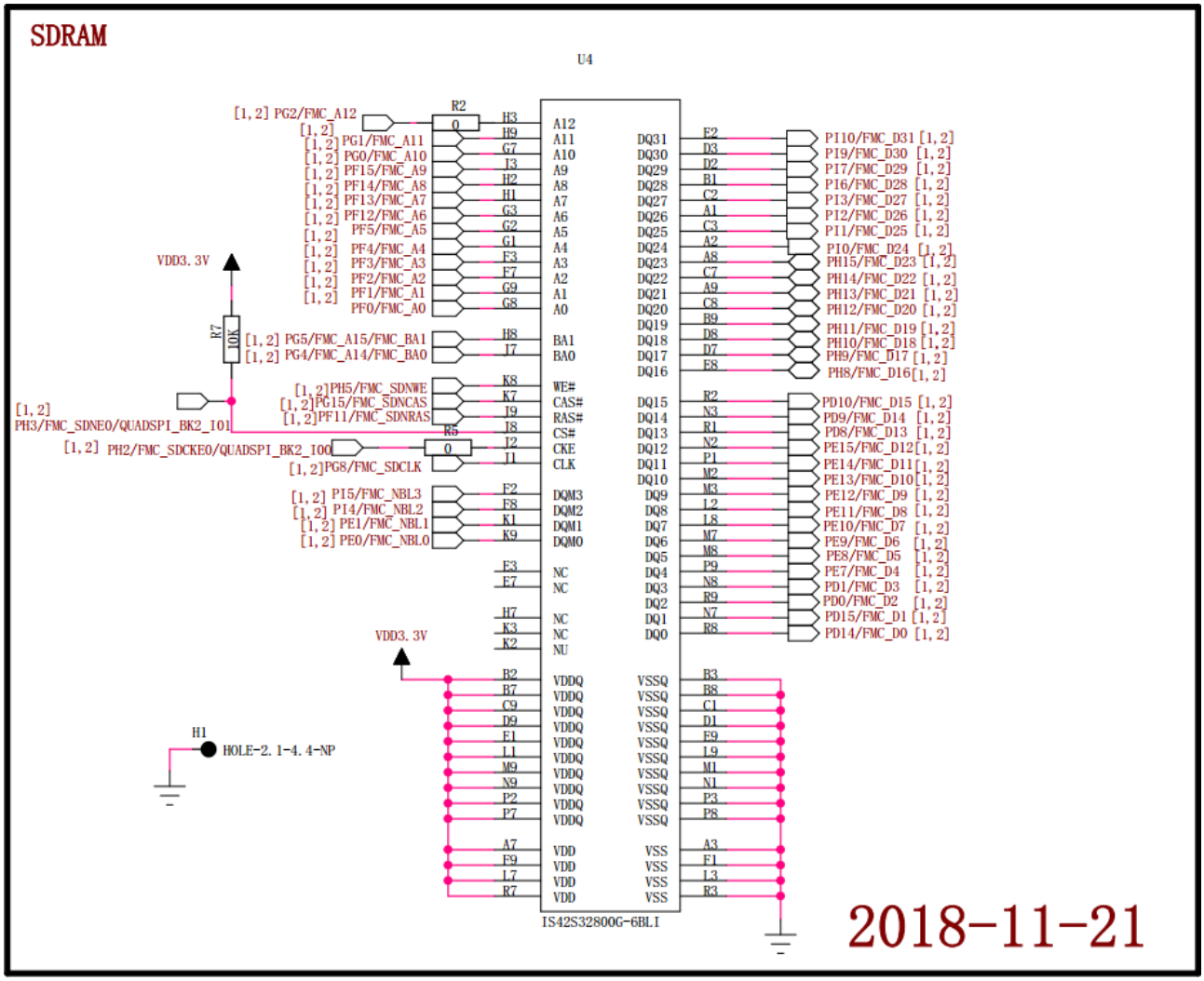
49.1 初学者重要提示
49.2 SDRAM 硬件设计
49.3 SDRAM 驱动设计
49.4 SDRAM 板级支持包（bsp_fmc_sdram.c）
49.5 SDRAM 驱动移植和使用
49.6 实验例程设计框架
49.7 实验例程说明（MDK）
49.8 实验例程说明（IAR）
49.9 总结

49.1 初学者重要提示

- 学习本章节前，务必优先学习第 47 章，需要对 FMC 的基础知识和 HAL 库的几个常用 API 有个认识。
- 学习 SDRAM 前搞清楚两个问题，一个是 SDRAM 的基本原理，还有一个就是那几个关键的参数，参数是 STM32H7 配置 SDRAM 的关键。这几个参数大概了解是什么意思即可，配置的时候，根据 SDRAM 的手册配置一下就完成了。
- 关于 SDRAM 的学习资料，推荐此贴：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=1930。
  特别是《高手进阶，终极内存技术指南——完整/进阶版》，强烈推荐大家一定看下。
- STM32H7 驱动 32 位 SDRAM 的写速度狂飙 376MB/S，读速度 189MB/S。
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=91481 。

49.2 SDRAM 硬件设计

SDRAM 的硬件设计如下：



通过这个硬件设计我们要了解到以下几点知识：

- STM32H7 采用的 32 位 FMC 接口驱动 ISSI 的 SDRAM，型号 IS42S32800G-6BLI，最高支持 166MHz 的时钟，容量 32MB。
- 标准的 SDRAM 一般都是 4 个 BANK，这个芯片也不例外，芯片的总容量：
  2Mbit x 32bit x 4bank = 268,435,456bits = 256Mbit 。
  每个 BANK 由 4096rows x 512columns x 32bits 组成。
  这个比较重要，配置的时候要用到，也就是 12 行 9 列。
- 片选采用的 SDNE0，那么 SDRAM 的首地址是 0xC000 000，控制 32MB 的空间。
- 用到引脚所代表的含义：

表 179. SDRAM 信号

SDRAM 信号	I/O 类型	说明	复用功能
SDCLK	O	SDRAM 时钟	-
SDCKE[1:0]	O	SDCKE0：SDRAM 存储区域 1 时钟使能 SDCKE1：SDRAM 存储区域 2 时钟使能	-
SDNE[1:0]	O	SDNE0：SDRAM 存储区域 1 芯片使能 SDNE1：SDRAM 存储区域 2 芯片使能	-
A[12:0]	O	地址	FMC_A[12:0]
D[31:0]	I/O	双向数据总线	FMC_D[31:0]
BA[1:0]	O	存储区域地址	FMC_A[15:14]
NRAS	O	行地址选通	-
NCAS	O	列地址选通	-
SDNWE	O	写入使能	-
NBL[3:0]	O	写访问的输出字节屏蔽 （存储器信号名称：DQM[3:0]）	FMC_NBL[3:0]

了解这些知识就够了，剩下就是软件配置时的参数设置。

49.3 SDRAM 驱动设计

下面将程序设计中的相关问题逐一为大家做个说明。

49.3.1 第 1 步，配置 SDRAM 的几个重要参数

STM32H7 把这几个关键的参数做到了一个寄存器里面了，这些参数，手册上面有一些说明，但比较的笼统。

SDRAM Timing registers for SDRAM memory bank x (FMC_SDTRx)

Address offset: 0x148 + 4 * (x - 1), (x = 1 to 2)

Reset value: 0x0FFF FFFF

This register contains the timing parameters of each SDRAM bank

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	TRCD				TRP				TWR			
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TRC				TRAS				TXSR				TMRD			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

注：更多的参数介绍可以看本章初学者重要提示部分推荐的文档《高手进阶，终极内存技术指南——完整/进阶版》。

◆ **tRCD（TRCD）：**

在发送列读写命令时必须要与行有效命令有一个间隔，这个间隔被定义为 tRCD，即 RAS to CAS Delay（RAS 至 CAS 延迟），大家也可以理解为行选通周期，这应该是根据芯片存储阵列电子元件响应时间（从一种状态到另一种状态变化的过程）所制定的延迟。tRCD 是 SDRAM 的一个重要时序参数，广义的 tRCD 以时钟周期数为单位，比如 tRCD=2，就代表延迟周期为两个时钟周期。具体到确切的时间，则要根据时钟频率而定，对于 STM32H7 驱动 SDRAM，采用的 200MHz，实际使用要做 2 分频，即 100MHz，那么我们设置 tRCD=2，就代表 20ns 的延迟。

◆ **CL（CAS Latency）：**

在选定列地址后，就已经确定了具体的存储单元，剩下的事情就是数据通过数据 I/O 通道（DQ）输出到内存总线上了。但是在 CAS 发出之后，仍要经过一定的时间才能有数据输出，从 CAS 与读取命令发出到第一笔数据输出的这段时间，被定义为 CL（CAS Latency，CAS 潜伏期）。由于 CL 只在读取时出现，所以 CL 又被称为读取潜伏期（RL，Read Latency）。CL 的单位与 tRCD 一样，为时钟周期数，具体耗时由时钟频率决定。数据写入的操作也是在 tRCD 之后进行，但此时没有了 CL（记住，CL 只出现在读取操作中）。

◆ **tWR（TWR）：**

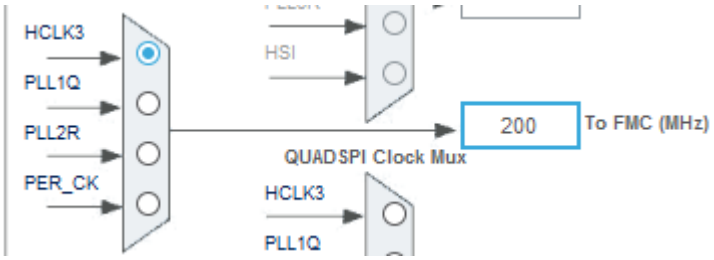
数据并不是即时地写入存储电容，因为选通三极管（就如读取时一样）与电容的充电必须要有一段时间，所以数据的真正写入需要一定的周期。为了保证数据的可靠写入，都会留出足够的写入/校正时间（tWR，WriteRecovery Time），这个操作也被称作写回（Write Back）。

◆ **tRP（TRP）：**

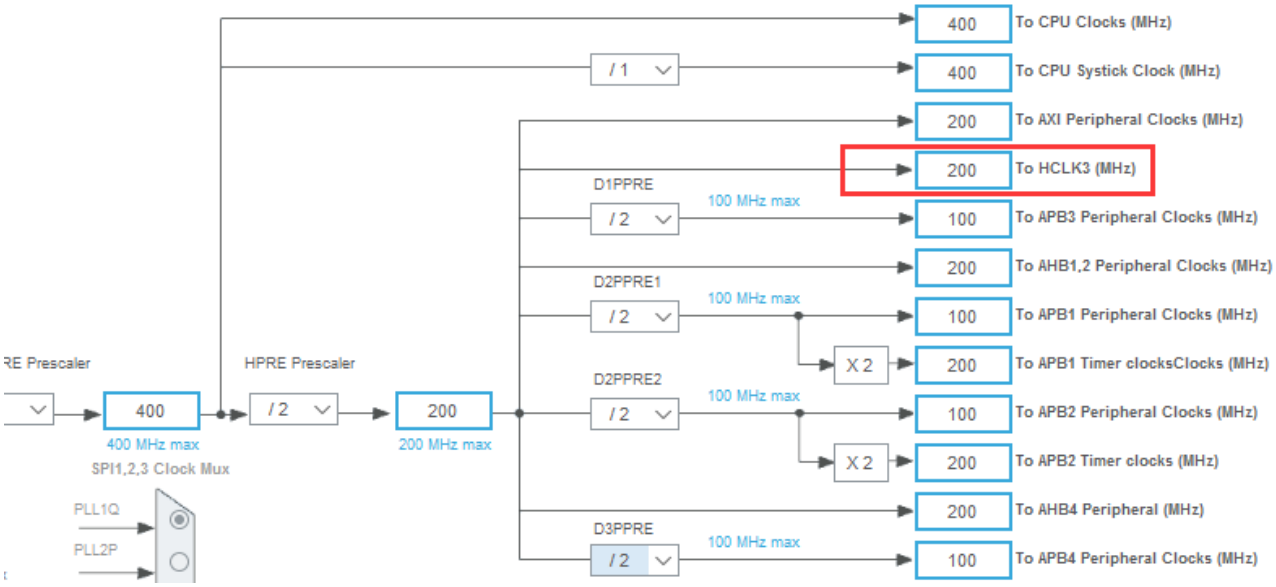
在发出预充电命令之后，要经过一段时间才能允许发送 RAS 行有效命令打开新的工作行，这个间隔被称为 tRP（Precharge command Period，预充电有效周期）。和 tRCD、CL 一样，tRP 的单位也是时钟周期数，具体值视时钟频率而定。

49.3.2 第 2 步，FMC 时钟源选择

使用 FMC 可以选择如下几种时钟源 HCLK3，PLL1Q，PLL2R 和 PER_CK：



我们这里直接使用 HCLK3，配置 STM32H7 的主频为 400MHz 的时候，HCLK3 输出的 200MHz，这个速度是 FMC 支持的最高时钟，正好用于这里：



FMC 驱动 SDRAM 的话，必须对 FMC 的时钟做 2 分频或者 3 分频，而且仅支持这两种分频方式，也就是说，SDRAM 时钟可以选择 200MHz/2 = 100MHz，或者 200MHz/3 = 66MHz。

49.3.3第 3 步，SDRAM 时序参数配置

SDRAM 的时序配置主要是下面几个参数，FMC 时钟是 200MHz，驱动 SDRAM 做了 2 分频，也就是 100MHz，一个 SDRAM 时钟周期就是 10ns，下面参数的单位都是 10ns：

```
SDRAM_Timing.LoadToActiveDelay    = 2;
SDRAM_Timing.ExitSelfRefreshDelay = 7;
SDRAM_Timing.SelfRefreshTime      = 4;
SDRAM_Timing.RowCycleDelay        = 7;
SDRAM_Timing.WriteRecoveryTime    = 2;
SDRAM_Timing.RPDelay              = 2;
SDRAM_Timing.RCDDelay             = 2;
```

下面就把这几个参数逐一为大家做个说明：

- TMRD

  SDRAM_Timing.LoadToActiveDelay = 2;

  TMRD 定义加载模式寄存器的命令与激活命令或刷新命令之间的延迟。SDRAM 手册上提供的是四种速度等级时提供的参数，V7 开发板用的 SDRAM 支持 166MHz，TMRD=2 就是 12ns，而我们实际驱动 SDRAM 是用的 100MHz，TMRD = 1 时是 10ns，超出了性能范围，TMRD=2 时是 20ns，所以这里取值 2。

OPERATING FREQUENCY / LATENCY RELATIONSHIPS

SYMBOL	PARAMETER						UNITS
—	Clock Cycle Time		5	6	7	7	ns
—	Operating Frequency		200	166	143	133	MHz
tCAC	CAS Latency		3	3	3	2	cycle
tRCD	Active Command To Read/Write Command Delay Time		3	3	3	2	cycle
tRAC	RAS Latency (tRCD + tCAC)	CAS Latency = 3	6	6	6	—	cycle
		CAS Latency = 2	—	—	—	4	
tRC	Command Period (REF to REF / ACT to ACT)		11	10	10	9	cycle
tRAS	Command Period (ACT to PRE)		8	7	7	6	cycle
tRP	Command Period (PRE to ACT)		3	3	3	2	cycle
tRRD	Command Period (ACT[0] to ACT [1])		2	2	2	2	cycle
tCCD	Column Command Delay Time (READ, READA, WRIT, WRITA)		1	1	1	1	cycle
tDPL	Input Data To Precharge Command Delay Time		2	2	2	2	cycle
tDAL	Input Data To Active/Refresh Command Delay Time (During Auto-Precharge)		5	5	5	4	cycle
tRBD	Burst Stop Command To Output in HIGH-Z Delay Time (Read)	CAS Latency = 3	3	3	3	—	cycle
		CAS Latency = 2	—	—	—	2	
tWBD	Burst Stop Command To Input in Invalid Delay Time (Write)		0	0	0	0	cycle
tRQL	Precharge Command To Output in HIGH-Z Delay Time (Read)	CAS Latency = 3	3	3	3	—	cycle
		CAS Latency = 2	—	—	—	2	
tWDL	Precharge Command To Input in Invalid Delay Time (Write)		0	0	0	0	cycle
tPQL	Last Output To Auto-Precharge Start Time (Read)	CAS Latency = 3	-2	-2	-2	—	cycle
		CAS Latency = 2	—	—	—	-1	
tQMD	DQM To Output Delay Time (Read)		2	2	2	2	cycle
tDMD	DQM To Input Delay Time (Write)		0	0	0	0	cycle
tMRD	Mode Register Set To Command Delay Time		2	2	2	2	cycle

- TXSR

  SDRAM_Timing.ExitSelfRefreshDelay = 7;

  TXSR 定义从发出自刷新命令到发出激活命令之间的延迟。不管那种 SDRAM 速度等级，此参数都是需要 70ns，实际驱动 SDRAM 是用的 100MHz，TXSR = 7 时正好是 70ns。

			-5		-6		-7		
Symbol	Parameter		Min.	Max.	Min.	Max.	Min.	Max.	Units
t_{CK3}	Clock Cycle Time	CAS Latency = 3	5	—	6	—	7	—	ns
t_{CK2}		CAS Latency = 2	10	—	10	—	7.5	—	ns
t_{AC3}	Access Time From CLK	CAS Latency = 3	—	4.8	—	5.4	—	5.4	ns
t_{AC2}		CAS Latency = 2	—	6.5	—	6.5	—	5.5	ns
t_{CH}	CLK HIGH Level Width		2.5	—	2.5	—	2.5	—	ns
t_{CL}	CLK LOW Level Width		2.5	—	2.5	—	2.5	—	ns
t_{OH3}	Output Data Hold Time	CAS Latency = 3	2.7	—	2.7	—	2.7	—	ns
t_{OH2}		CAS Latency = 2	2.7	—	2.7	—	2.7	—	ns
t_{LZ}	Output LOW Impedance Time		0	—	0	—	0	—	ns
t_{HZ}	Output HIGH Impedance Time		2.7	4.8	2.7	5.4	2.7	5.4	ns
t_{DS}	Input Data Setup Time[2]		1.5	—	1.5	—	1.5	—	ns
t_{DH}	Input Data Hold Time[2]		1.0	—	1.0	—	1.0	—	ns
t_{AS}	Address Setup Time[2]		1.5	—	1.5	—	1.5	—	ns
t_{AH}	Address Hold Time[2]		1.0	—	1.0	—	1.0	—	ns
t_{CKS}	CKE Setup Time[2]		1.5	—	1.5	—	1.5	—	ns
t_{CKH}	CKE Hold Time[2]		1.0	—	1.0	—	1.0	—	ns
t_{CMS}	Command Setup Time ($\overline{CS}$, $\overline{RAS}$, $\overline{CAS}$, $\overline{WE}$, DQM)[2]		1.5	—	1.5	—	1.5	—	ns
t_{CMH}	Command Hold Time ($\overline{CS}$, $\overline{RAS}$, $\overline{CAS}$, $\overline{WE}$, DQM)[2]		1.0	—	1.0	—	1.0	—	ns
t_{RC}	Command Period (REF to REF / ACT to ACT)		55	—	60	—	67.5	—	ns
t_{RAS}	Command Period (ACT to PRE)		40	100K	42	100K	45	100K	ns
t_{RP}	Command Period (PRE to ACT)		15	—	18	—	15	—	ns
t_{RCD}	Active Command To Read / Write Command Delay Time		15	—	18	—	15	—	ns
t_{RRD}	Command Period (ACT [0] to ACT[1])		10	—	12	—	14	—	ns
t_{DPL}	Input Data To Precharge Command Delay time		10	—	12	—	14	—	ns
t_{DAL}	Input Data To Active / Refresh Command Delay time (During Auto-Precharge)		25	—	30	—	30	—	ns
t_{MRD}	Mode Register Program Time		10	—	12	—	14	—	ns
t_{DDE}	Power Down Exit Setup Time		5	—	6	—	7	—	ns
t_{XSR}	Self-Refresh Exit Time[5]		70	—	70	—	70	—	ns
t_T	Transition Time		0.3	1.2	0.3	1.2	0.3	1.2	ns
t_{REF}	Refresh Cycle Time (4096)	$T_A \le 70°C$ Com., Ind., A1, A2	—	64	—	64	—	64	ms
		$T_A \le 85°C$ Ind., A1, A2	—	64	—	64	—	64	ms
		$T_A > 85°C$ A2	—	—	—	16	—	16	ms

- TRAS

  SDRAM_Timing.SelfRefreshTime = 4;

  TRAS 定义最短的自刷新周期。SDRAM 手册上提供的是四种速度等级时提供的参数，V7 开发板用的 SDRAM 支持 166MHz，TRAS=7 就是 42ns，而我们实际驱动 SDRAM 是用的 100MHz，TRAS = 4 时是 40ns，保险起见这里可以设置 TRAS=5，实际测试 40ns 也是稳定的。

OPERATING FREQUENCY / LATENCY RELATIONSHIPS

SYMBOL	PARAMETER						UNITS
—	Clock Cycle Time		5	6	7	7	ns
—	Operating Frequency		200	166	143	133	MHz
t_{CAC}	$\overline{CAS}$ Latency		3	3	3	2	cycle
t_{RCD}	Active Command To Read/Write Command Delay Time		3	3	3	2	cycle
t_{RAC}	$\overline{RAS}$ Latency (t_{RCD} + t_{CAC})	$\overline{CAS}$ Latency = 3 $\overline{CAS}$ Latency = 2	6 —	6 —	6 —	— 4	cycle
t_{RC}	Command Period (REF to REF / ACT to ACT)		11	10	10	9	cycle
t_{RAS}	Command Period (ACT to PRE)		8	7	7	6	cycle
t_{RP}	Command Period (PRE to ACT)		3	3	3	2	cycle
t_{RRD}	Command Period (ACT[0] to ACT [1])		2	2	2	2	cycle
t_{CCD}	Column Command Delay Time (READ, READA, WRIT, WRITA)		1	1	1	1	cycle
t_{DPL}	Input Data To Precharge Command Delay Time		2	2	2	2	cycle
t_{DAL}	Input Data To Active/Refresh Command Delay Time (During Auto-Precharge)		5	5	5	4	cycle
t_{RBD}	Burst Stop Command To Output in HIGH-Z Delay Time (Read)	$\overline{CAS}$ Latency = 3 $\overline{CAS}$ Latency = 2	3 —	3 —	3 —	— 2	cycle
t_{WBD}	Burst Stop Command To Input in Invalid Delay Time (Write)		0	0	0	0	cycle
t_{RQL}	Precharge Command To Output in HIGH-Z Delay Time (Read)	$\overline{CAS}$ Latency = 3 $\overline{CAS}$ Latency = 2	3 —	3 —	3 —	— 2	cycle
t_{WDL}	Precharge Command To Input in Invalid Delay Time (Write)		0	0	0	0	cycle
t_{PQL}	Last Output To Auto-Precharge Start Time (Read)	$\overline{CAS}$ Latency = 3 $\overline{CAS}$ Latency = 2	-2 —	-2 —	-2 —	— -1	cycle
t_{QMD}	DQM To Output Delay Time (Read)		2	2	2	2	cycle
t_{DMD}	DQM To Input Delay Time (Write)		0	0	0	0	cycle
t_{MRD}	Mode Register Set To Command Delay Time		2	2	2	2	cycle

- TRC

  SDRAM_Timing.RowCycleDelay = 7;

  TRC 定义刷新命令和激活命令之间的延迟。SDRAM 手册上提供的是四种速度等级时提供的参数，V7 开发板用的 SDRAM 支持 166MHz，TRC=10 就是 60ns，而我们实际驱动 SDRAM 是用的 100MHz，TRAS = 7 时是 70ns，设置 TRC=6 也是可以的，不过保险起见，设置 TRAS=7。

OPERATING FREQUENCY / LATENCY RELATIONSHIPS

SYMBOL	PARAMETER						UNITS
—	Clock Cycle Time		5	6	7	7	ns
—	Operating Frequency		200	166	143	133	MHz
tCAC	CAS Latency		3	3	3	2	cycle
tRCD	Active Command To Read/Write Command Delay Time		3	3	3	2	cycle
tRAC	RAS Latency (tRCD + tCAC)	CAS Latency = 3	6	6	6	—	cycle
		CAS Latency = 2	—	—	—	4	
tRC	Command Period (REF to REF / ACT to ACT)		11	10	10	9	cycle
tRAS	Command Period (ACT to PRE)		8	7	7	6	cycle
tRP	Command Period (PRE to ACT)		3	3	3	2	cycle
tRRD	Command Period (ACT[0] to ACT [1])		2	2	2	2	cycle
tCCD	Column Command Delay Time (READ, READA, WRIT, WRITA)		1	1	1	1	cycle
tDPL	Input Data To Precharge Command Delay Time		2	2	2	2	cycle
tDAL	Input Data To Active/Refresh Command Delay Time (During Auto-Precharge)		5	5	5	4	cycle
tRBD	Burst Stop Command To Output in HIGH-Z Delay Time (Read)	CAS Latency = 3	3	3	3	—	cycle
		CAS Latency = 2	—	—	—	2	
tWBD	Burst Stop Command To Input in Invalid Delay Time (Write)		0	0	0	0	cycle
tRQL	Precharge Command To Output in HIGH-Z Delay Time (Read)	CAS Latency = 3	3	3	3	—	cycle
		CAS Latency = 2	—	—	—	2	
tWDL	Precharge Command To Input in Invalid Delay Time (Write)		0	0	0	0	cycle
tPQL	Last Output To Auto-Precharge Start Time (Read)	CAS Latency = 3	-2	-2	-2	—	cycle
		CAS Latency = 2	—	—	—	-1	
tQMD	DQM To Output Delay Time (Read)		2	2	2	2	cycle
tDMD	DQM To Input Delay Time (Write)		0	0	0	0	cycle
tMRD	Mode Register Set To Command Delay Time		2	2	2	2	cycle

- TWR

  SDRAM_Timing.WriteRecoveryTime = 2;

  TWR 定义在写命令和预充电命令之间的延迟。SDRAM 手册上提供的是四种速度等级时提供的参数（TWR 等效于 TDPL），V7 开发板用的 SDRAM 支持 166MHz，TWR/TDPL=2 就是 12ns，而我们实际驱动 SDRAM 是用的 100MHz，TWR/TDPL = 1 时是 10ns，超出了性能访问。设置 TWR/TDPL =2 时是 20ns，所以这里取值 2。

OPERATING FREQUENCY / LATENCY RELATIONSHIPS

SYMBOL	PARAMETER						UNITS
—	Clock Cycle Time		5	6	7	7	ns
—	Operating Frequency		200	166	143	133	MHz
tCAC	CAS Latency		3	3	3	2	cycle
tRCD	Active Command To Read/Write Command Delay Time		3	3	3	2	cycle
tRAC	RAS Latency (tRCD + tCAC)	CAS Latency = 3	6	6	6	—	cycle
		CAS Latency = 2	—	—	—	4	
tRC	Command Period (REF to REF / ACT to ACT)		11	10	10	9	cycle
tRAS	Command Period (ACT to PRE)		8	7	7	6	cycle
tRP	Command Period (PRE to ACT)		3	3	3	2	cycle
tRRD	Command Period (ACT[0] to ACT [1])		2	2	2	2	cycle
tCCD	Column Command Delay Time (READ, READA, WRIT, WRITA)		1	1	1	1	cycle
tDPL	Input Data To Precharge Command Delay Time		2	2	2	2	cycle
tDAL	Input Data To Active/Refresh Command Delay Time (During Auto-Precharge)		5	5	5	4	cycle
tRBD	Burst Stop Command To Output in HIGH-Z Delay Time (Read)	CAS Latency = 3	3	3	3	—	cycle
		CAS Latency = 2	—	—	—	2	
tWBD	Burst Stop Command To Input in Invalid Delay Time (Write)		0	0	0	0	cycle
tRQL	Precharge Command To Output in HIGH-Z Delay Time (Read)	CAS Latency = 3	3	3	3	—	cycle
		CAS Latency = 2	—	—	—	2	
tWDL	Precharge Command To Input in Invalid Delay Time (Write)		0	0	0	0	cycle
tPQL	Last Output To Auto-Precharge Start Time (Read)	CAS Latency = 3	-2	-2	-2	—	cycle
		CAS Latency = 2	—	—	—	-1	
tQMD	DQM To Output Delay Time (Read)		2	2	2	2	cycle
tDMD	DQM To Input Delay Time (Write)		0	0	0	0	cycle
tMRD	Mode Register Set To Command Delay Time		2	2	2	2	cycle

- TRP

  SDRAM_Timing.RPDelay = 2;

  TRP 定义预充电命令与其它命令之间的延迟。SDRAM 手册上提供四种速度等级的参数，V7 开发板用的 SDRAM 支持 166MHz，TRP =3 就是 18ns，而我们实际驱动 SDRAM 是用的 100MHz，TRP = 2 时是 20ns，满足要求。

OPERATING FREQUENCY / LATENCY RELATIONSHIPS

SYMBOL	PARAMETER						UNITS
—	Clock Cycle Time		5	6	7	7	ns
—	Operating Frequency		200	166	143	133	MHz
tCAC	CAS Latency		3	3	3	2	cycle
tRCD	Active Command To Read/Write Command Delay Time		3	3	3	2	cycle
tRAC	RAS Latency (tRCD + tCAC)	CAS Latency = 3	6	6	6	—	cycle
		CAS Latency = 2	—	—	—	4	
tRC	Command Period (REF to REF / ACT to ACT)		11	10	10	9	cycle
tRAS	Command Period (ACT to PRE)		8	7	7	6	cycle
tRP	Command Period (PRE to ACT)		3	3	3	2	cycle
tRRD	Command Period (ACT[0] to ACT [1])		2	2	2	2	cycle
tCCD	Column Command Delay Time (READ, READA, WRIT, WRITA)		1	1	1	1	cycle
tDPL	Input Data To Precharge Command Delay Time		2	2	2	2	cycle
tDAL	Input Data To Active/Refresh Command Delay Time (During Auto-Precharge)		5	5	5	4	cycle
tRBD	Burst Stop Command To Output in HIGH-Z Delay Time (Read)	CAS Latency = 3	3	3	3	—	cycle
		CAS Latency = 2	—	—	—	2	
tWBD	Burst Stop Command To Input in Invalid Delay Time (Write)		0	0	0	0	cycle
tRQL	Precharge Command To Output in HIGH-Z Delay Time (Read)	CAS Latency = 3	3	3	3	—	cycle
		CAS Latency = 2	—	—	—	2	
tWDL	Precharge Command To Input in Invalid Delay Time (Write)		0	0	0	0	cycle
tPQL	Last Output To Auto-Precharge Start Time (Read)	CAS Latency = 3	-2	-2	-2	—	cycle
		CAS Latency = 2	—	—	—	-1	
tQMD	DQM To Output Delay Time (Read)		2	2	2	2	cycle
tDMD	DQM To Input Delay Time (Write)		0	0	0	0	cycle
tMRD	Mode Register Set To Command Delay Time		2	2	2	2	cycle

- TRCD

  SDRAM_Timing.RCDDelay = 2;

  TRCD 定义激活命令与读/写命令之间的延迟。SDRAM 手册上提供四种速度等级的参数，V7 开发板用的 SDRAM 支持 166MHz，TRCD =3 就是 18ns，而我们实际驱动 SDRAM 是用的 100MHz，TRCD = 2 时是 20ns，满足要求。

OPERATING FREQUENCY / LATENCY RELATIONSHIPS

SYMBOL	PARAMETER						UNITS
—	Clock Cycle Time		5	6	7	7	ns
—	Operating Frequency		200	166	143	133	MHz
tCAC	$\overline{CAS}$ Latency		3	3	3	2	cycle
tRCD	Active Command To Read/Write Command Delay Time		3	3	3	2	cycle
tRAC	$\overline{RAS}$ Latency (tRCD + tCAC)	$\overline{CAS}$ Latency = 3	6	6	6	—	cycle
		$\overline{CAS}$ Latency = 2	—	—	—	4	
tRC	Command Period (REF to REF / ACT to ACT)		11	10	10	9	cycle
tRAS	Command Period (ACT to PRE)		8	7	7	6	cycle
tRP	Command Period (PRE to ACT)		3	3	3	2	cycle
tRRD	Command Period (ACT[0] to ACT [1])		2	2	2	2	cycle
tCCD	Column Command Delay Time (READ, READA, WRIT, WRITA)		1	1	1	1	cycle
tDPL	Input Data To Precharge Command Delay Time		2	2	2	2	cycle
tDAL	Input Data To Active/Refresh Command Delay Time (During Auto-Precharge)		5	5	5	4	cycle
tRBD	Burst Stop Command To Output in HIGH-Z Delay Time (Read)	$\overline{CAS}$ Latency = 3	3	3	3	—	cycle
		$\overline{CAS}$ Latency = 2	—	—	—	2	
tWBD	Burst Stop Command To Input in Invalid Delay Time (Write)		0	0	0	0	cycle
tRQL	Precharge Command To Output in HIGH-Z Delay Time (Read)	$\overline{CAS}$ Latency = 3	3	3	3	—	cycle
		$\overline{CAS}$ Latency = 2	—	—	—	2	
tWDL	Precharge Command To Input in Invalid Delay Time (Write)		0	0	0	0	cycle
tPQL	Last Output To Auto-Precharge Start Time (Read)	$\overline{CAS}$ Latency = 3	-2	-2	-2	—	cycle
		$\overline{CAS}$ Latency = 2	—	—	—	-1	
tQMD	DQM To Output Delay Time (Read)		2	2	2	2	cycle
tDMD	DQM To Input Delay Time (Write)		0	0	0	0	cycle
tMRD	Mode Register Set To Command Delay Time		2	2	2	2	cycle

49.3.4第 4 步，SDRAM 基本参数配置

SDRAM 的基本参数配置如下：

```
1.  hsdram.Init.SDBank             = FMC_SDRAM_BANK1;
2.  hsdram.Init.ColumnBitsNumber   = FMC_SDRAM_COLUMN_BITS_NUM_9;
3.  hsdram.Init.RowBitsNumber      = FMC_SDRAM_ROW_BITS_NUM_12;
4.  hsdram.Init.MemoryDataWidth    = FMC_SDRAM_MEM_BUS_WIDTH_32;
5.  hsdram.Init.InternalBankNumber = FMC_SDRAM_INTERN_BANKS_NUM_4;
6.  hsdram.Init.CASLatency         = FMC_SDRAM_CAS_LATENCY_3;
7.  hsdram.Init.WriteProtection    = FMC_SDRAM_WRITE_PROTECTION_DISABLE;
8.  hsdram.Init.SDClockPeriod      = SDCLOCK_PERIOD;
9.  hsdram.Init.ReadBurst          = FMC_SDRAM_RBURST_ENABLE;
10. hsdram.Init.ReadPipeDelay      = FMC_SDRAM_RPIPE_DELAY_0;
```

- 第 1 行：硬件设计上用的 BANK1。
- 第 2-3 行：ISSI 的 SDRAM，型号 IS42S32800G-6BLI，12 行 9 列。

- 第 4 行：SDRAM 的带宽是 32 位。
- 第 5 行：SDRAM 有 4 个 BANK。
- 第 6 行：CAS Latency 可以设置 Latency1，Latency2 和 Latency3，实际测试 Latency3 稳定。
- 第 7 行：关闭写保护。
- 第 8 行：设置 FMC 做 2 分频输出给 SDRAM，即 200MHz 做 2 分频，SDRAM 的时钟是 100MHz。这里的 SDCLOCK_PERIOD 是个宏定义：
  #define SDCLOCK_PERIOD　　FMC_SDRAM_CLOCK_PERIOD_2
- 第 9 行：使能读突发。
- 第 10 行：此位定义 CAS 延时后延后多少个 SDRAM 时钟周期读取数据，实际测此位可以设置无需延迟。

49.3.5第 5 步，SDRAM 初始化

SDRAM 的初始化如下：

```
/*
*********************************************************************************************************
*    函 数 名: SDRAM 初始化序列
*    功能说明: 完成 SDRAM 序列初始化
*    形    参: hsdram: SDRAM 句柄
*              Command: 命令结构体指针
*    返 回 值: None
*********************************************************************************************************
*/
static void SDRAM_Initialization_Sequence(SDRAM_HandleTypeDef *hsdram,
                                          FMC_SDRAM_CommandTypeDef *Command)
{
    __IO uint32_t tmpmrd =0;

    /*##-1- 时钟使能命令 ##################################################*/
    Command->CommandMode = FMC_SDRAM_CMD_CLK_ENABLE;
    Command->CommandTarget = FMC_SDRAM_CMD_TARGET_BANK1;;
    Command->AutoRefreshNumber = 1;
    Command->ModeRegisterDefinition = 0;

    /* 发送命令 */
    HAL_SDRAM_SendCommand(hsdram, Command, SDRAM_TIMEOUT);

    /*##-2- 插入延迟，至少 100us ##########################################################*/
    HAL_Delay(1);

    /*##-3- 整个 SDRAM 预充电命令，PALL(precharge all) ##############################*/
    Command->CommandMode = FMC_SDRAM_CMD_PALL;
    Command->CommandTarget = FMC_SDRAM_CMD_TARGET_BANK1;
    Command->AutoRefreshNumber = 1;
    Command->ModeRegisterDefinition = 0;

    /* 发送命令 */
    HAL_SDRAM_SendCommand(hsdram, Command, SDRAM_TIMEOUT);
```

```
35.
36.     /*##-4- 自动刷新命令 ###########################################################*/
37.      Command->CommandMode = FMC_SDRAM_CMD_AUTOREFRESH_MODE;
38.      Command->CommandTarget = FMC_SDRAM_CMD_TARGET_BANK1;
39.      Command->AutoRefreshNumber = 8;
40.      Command->ModeRegisterDefinition = 0;
41.
42.      /* 发送命令 */
43.      HAL_SDRAM_SendCommand(hsdram, Command, SDRAM_TIMEOUT);
44.
45.     /*##-5- 配置 SDRAM 模式寄存器 ###################################################*/
46.      tmpmrd = (uint32_t)SDRAM_MODEREG_BURST_LENGTH_1          |
47.                         SDRAM_MODEREG_BURST_TYPE_SEQUENTIAL   |
48.                         SDRAM_MODEREG_CAS_LATENCY_3           |
49.                         SDRAM_MODEREG_OPERATING_MODE_STANDARD |
50.                         SDRAM_MODEREG_WRITEBURST_MODE_SINGLE;
51.
52.      Command->CommandMode = FMC_SDRAM_CMD_LOAD_MODE;
53.      Command->CommandTarget = FMC_SDRAM_CMD_TARGET_BANK1;
54.      Command->AutoRefreshNumber = 1;
55.      Command->ModeRegisterDefinition = tmpmrd;
56.
57.      /* 发送命令 */
58.      HAL_SDRAM_SendCommand(hsdram, Command, SDRAM_TIMEOUT);
59.
60.     /*##-6- 设置自刷新率 ############################################################*/
61.     /*
62.         SDRAM refresh period / Number of rows）*SDRAM 时钟速度 - 20
63.       = 64ms / 4096 *100MHz - 20
64.       = 1542.5 取值 1543
65.     */
66.      HAL_SDRAM_ProgramRefreshRate(hsdram, REFRESH_COUNT);
67.  }
```

这里把几个关键的地方阐释下：

- 第 16 - 22 行，发送时钟使能命令。
- 第 25 行，插入延迟，这个延迟是必不可少的，如果要自己移植的话，这个地方要特别注意。
- 第 28 – 34 行，发送整个 SDRAM 预充电命令。
- 第 37 - 43 行，发送自刷新命令。
- 第 46 – 58 行，配置 SDRAM 模式寄存器。
- 第 66 行，配置 SDRAM 的刷新率，关于刷新频率的数值是这么得到的。目前公认的标准是 SDRAM 中电容保存数据的上限是 64ms，也就是说每一行刷新的循环周期是 64ms。这样刷新速度就是：64ms / 行数量。我们在看内存规格时，经常会看到 4096 Refresh Cycles/64ms 或 8192 RefreshCycles/64ms 的标识，这里的 4096 与 8192 就代表这个芯片中每个 L-Bank 的行数。刷新命令一次对一行有效，发送间隔也是随总行数而变化，4096 行时为 15.625μs，8192 行时就为 7.8125μs。

  V7 开发板使用的型号 IS42S32800G-6BLI，自刷新规格是 4K / 64ms，即 4096/64ms。刷新一行需要 15.625μs。

  刷新计数 = （SDRAM refresh period / Number of rows）*SDRAM 时钟速度 – 20

= （64ms / 4096）* 100MHz – 20

= 1562.5 – 20

= 1542.5 ，取值 1543

实际上这个数值稍差点，在使用 SDRAM 时，基本都没有影响的。

Parameter		8M x 32
Configuration		2M x 32 x 4 banks
Refresh Count	Com./Ind.	4K / 64ms
	A1	4K / 64ms
	A2	4K / 16ms
Row Addresses		A0 – A11
Column Addresses		A0 – A8
Bank Address Pins		BA0, BA1
Autoprecharge Pins		A10/AP

注：截图里面的 Com 表示 Commercial 商业级，而 Ind 表示 Industrial 工业级。V7 用的是工业级的。

49.3.6第 6 步，SDRAM 使用

进行到这一步，已经可以像使用内部 SRAM 一样使用 SDRAM 了。除了本章节配套例子采用指针方式操作 SDRAM，前面第 26 章的超方便使用方式和第 27 章的动态内存分配也非常推荐。

49.4SDRAM 板级支持包（bsp_fmc_sdram.c)

SDRAM 驱动文件 bsp_fmc_sdram.c 提供了如下三个函数：

- bsp_InitExtSDRAM
- bsp_TestExtSDRAM1
- bsp_TestExtSDRAM2

49.4.1函数 bsp_InitExtSDRAM

函数原型：

```
void bsp_InitExtSDRAM(void)
```

函数描述：

此函数用于初始化 SDRAM，用到的 GPIO、时钟和 FMC 的 SDRAM 控制器都已经进行了初始化，调用了此函数就可以像使用内部 SRAM 一样使用 SDRAM 了。

注意事项：

1. 关于此函数的讲解在本章第 3 小节。

使用举例：

作为初始化函数，直接在 bsp.c 文件的 bsp_Init 函数里面调用即可。

49.4.2 函数 bsp_TestExtSDRAM1

函数原型：

```
uint32_t bsp_TestExtSDRAM1(void)
```

函数描述：

此函数用于扫描测试外部 SDRAM 的全部单元，如果有错误会返回错误单元个数。

函数参数：

- 返回值，返回 0 表示整个 SDRAM 测试通过，返回值大于 0 表示错误的单元个数。

使用举例：

直接调用即可。

49.4.3 函数 bsp_TestExtSDRAM2

函数原型：

```
uint32_t bsp_TestExtSDRAM2(void)
```

函数描述：

此函数用于扫描测试外部 SDRAM，不扫描前面 4M 字节的显存，如果有错误会返回错误单元个数。

函数参数：

- 返回值，返回 0 表示整个 SDRAM 测试通过，返回值大于 0 表示错误的单元个数。

使用举例：

直接调用即可。

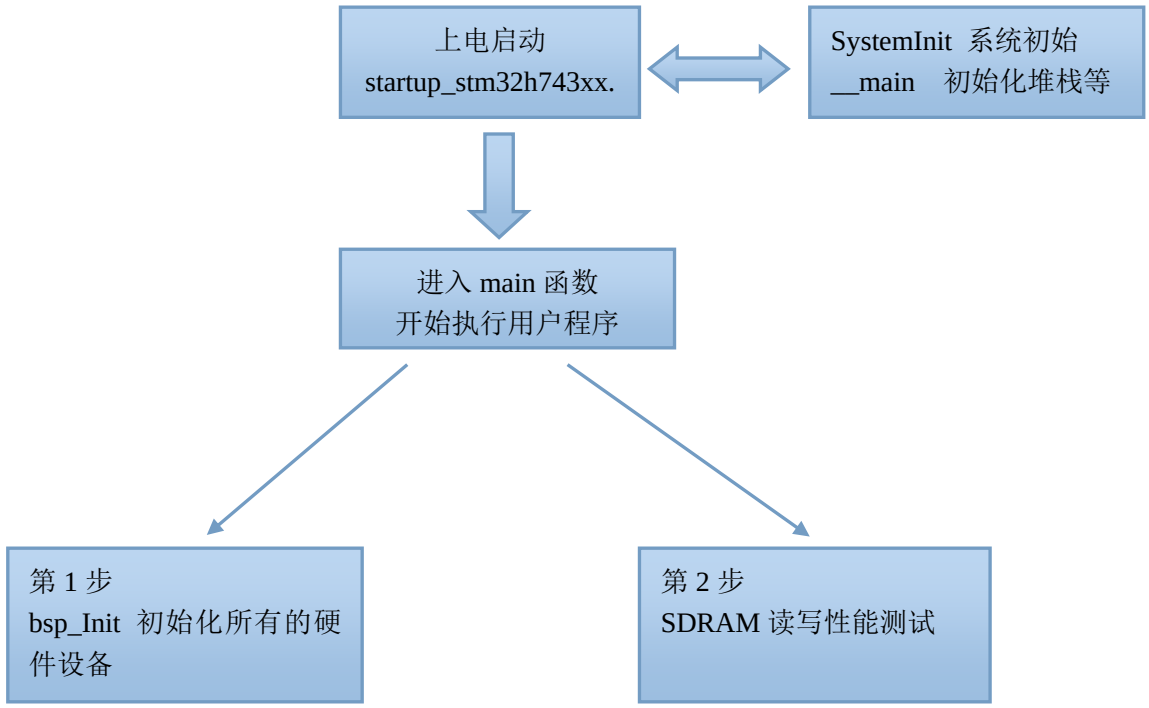
49.5 SDRAM 驱动移植和使用

SDRAM 的驱动移植比较方便：

- 第 1 步：复制 bsp_fmc_sdram.c 和 bsp_fmc_sram.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：这几个驱动文件主要用到 HAL 库的 GPIO 和 FMC 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 3 步，应用方法看本章节配套例子即可，另外就是根据自己所使用 SDRAM 的时序参数修改配置。

49.6 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED 和串口。
  - 第 2 步，SDRAM 的读写性能测试。

49.7 实验例程说明（MDK）

配套例子：

V7-028-外设 32 位带宽 SDRAM 性能测试

实验目的：

1. 学习外部 32 位带宽 SDRAM 性能测试。

实验内容：

1. SDRAM 型号 IS42S32800G-6BLI, 32 位带宽，容量 32MB, 6ns 速度(166MHz)。
2. 开启 Cache
   （1）使用 MDK 和 IAR 的各种优化等级测试，优化对其影响很小。
   （2）写速度 376MB/S，读速度 182MB/S

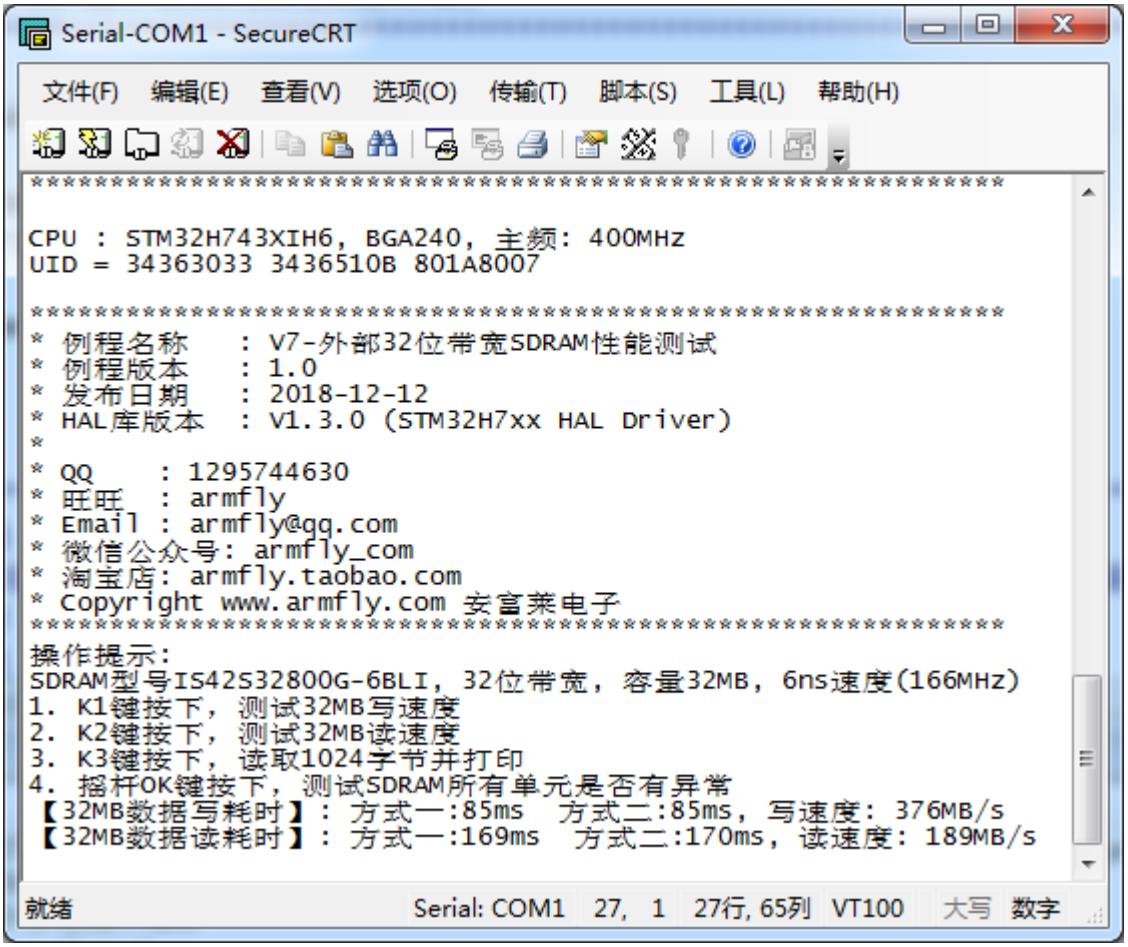
3. 关闭 Cache
   （1）使用 MDK 和 IAR 的各种优化等级测试，优化对其影响很小。
   （2）写速度 307MB/S，读速度 116MB/S
4. IAR 开启最高等级优化，读速度是 189MB/S，比 MDK 的 182MB/S 高点。
5. 对于 MDK，本实验开启了最高等级优化和时间优化。
6. 对 IAR，本实验开启了最高等级速度优化。

实验操作：

1. K1 键按下，测试 32MB 写速度;
2. K2 键按下，测试 32MB 读速度;
3. K3 键按下，读取 1024 字节并打印;
4. 摇杆 OK 键按下，测试 SDRAM 所有单元是否有异常。

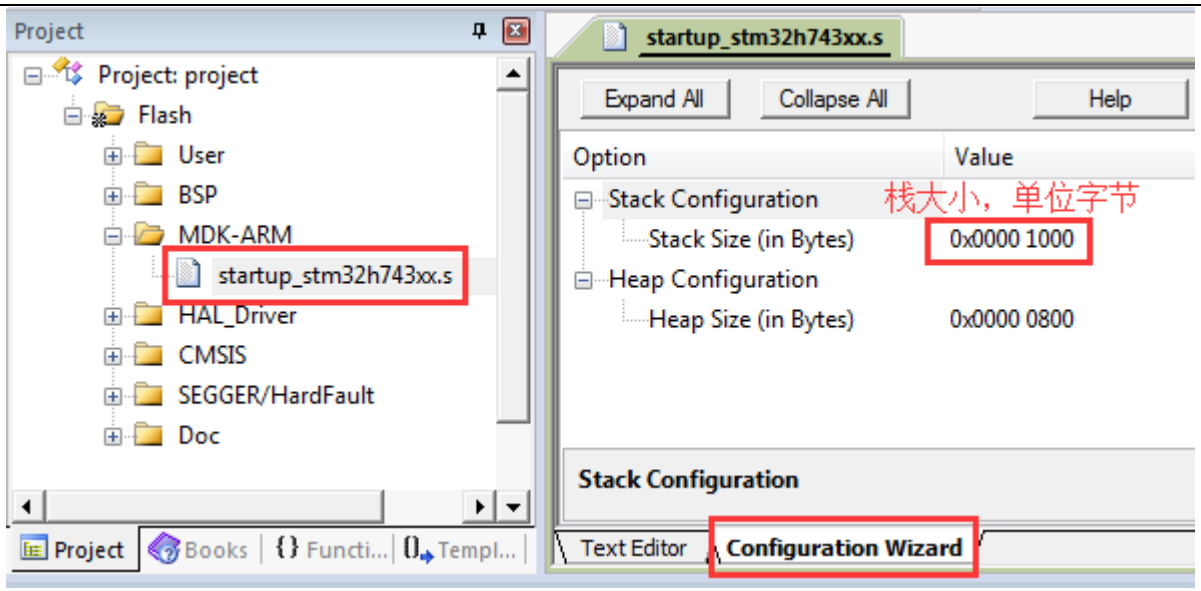
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

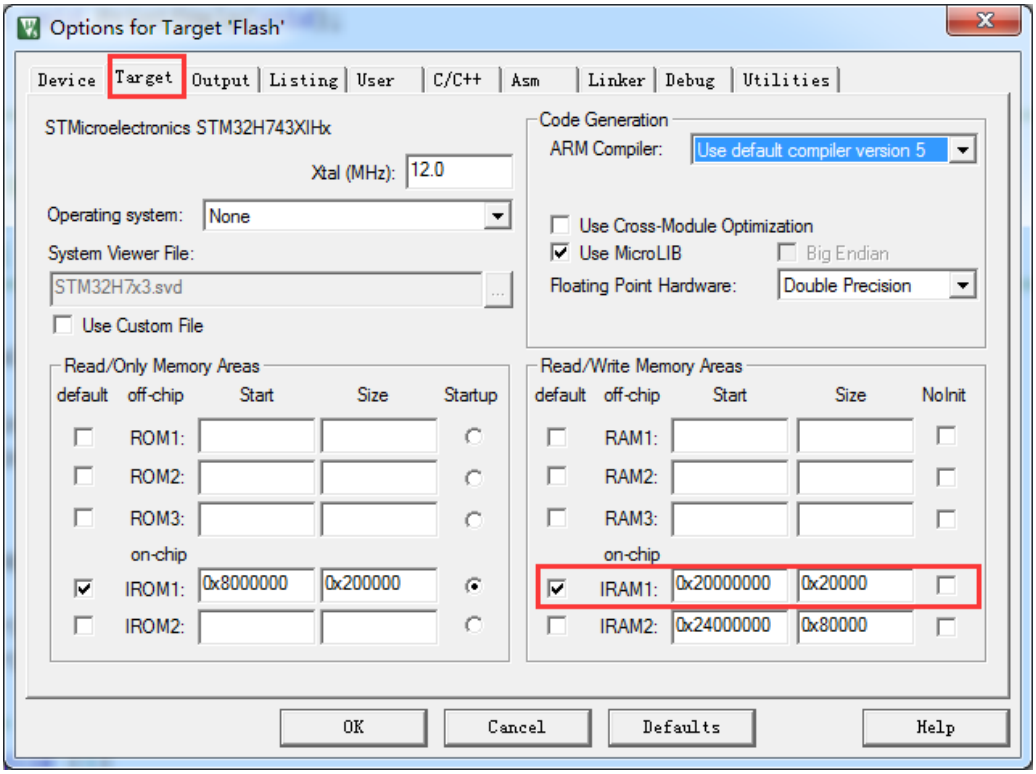


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SDRAM。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();
```

```
    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SDRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0xC0000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_32MB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
```

```
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能:**

主程序实现如下操作:

- K1 键按下，测试 32MB 写速度;
- K2 键按下，测试 32MB 读速度;
- K3 键按下，读取 1024 字节并打印;
- 摇杆 OK 键按下，测试 SDRAM 所有单元是否有异常。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */

    DemoFmcSRAM(); /* 外部 SDRAM 读写测试 */
}

/*
*********************************************************************************************************
*    函 数 名: DemoFmcSRAM
*    功能说明: SDRAM 读写性能测试
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoFmcSRAM(void)
{
    uint8_t ucKeyCode; /* 按键代码 */
    uint32_t err;

    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
```

```
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值，无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，测试 32MB 写速度 */
                    WriteSpeedTest();
                    break;

                case KEY_DOWN_K2:           /* K2 键按下，测试 32MB 读速度*/
                    ReadSpeedTest();
                    break;

                case KEY_DOWN_K3:           /* K3 键按下，读取 1024 字节并打印 */
                    ReadWriteTest();
                    break;

                case JOY_DOWN_OK:           /* 摇杆 OK 键按下，测试 SDRAM 所有单元是否有异常*/
                    err = bsp_TestExtSDRAM1();
                    if (err == 0)
                    {
                        printf("外部 SDRAM 测试通过\r\n");
                    }
                    else
                    {
                        printf("外部 SDRAM 出错，错误单元个数：%d\r\n", err);
                    }
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

49.8 实验例程说明（IAR）

配套例子：

V7-028-外设 32 位带宽 SDRAM 性能测试

实验目的：

1. 学习外部 32 位带宽 SDRAM 性能测试。

实验内容：

1. SDRAM 型号 IS42S32800G-6BLI, 32 位带宽, 容量 32MB, 6ns 速度(166MHz)。

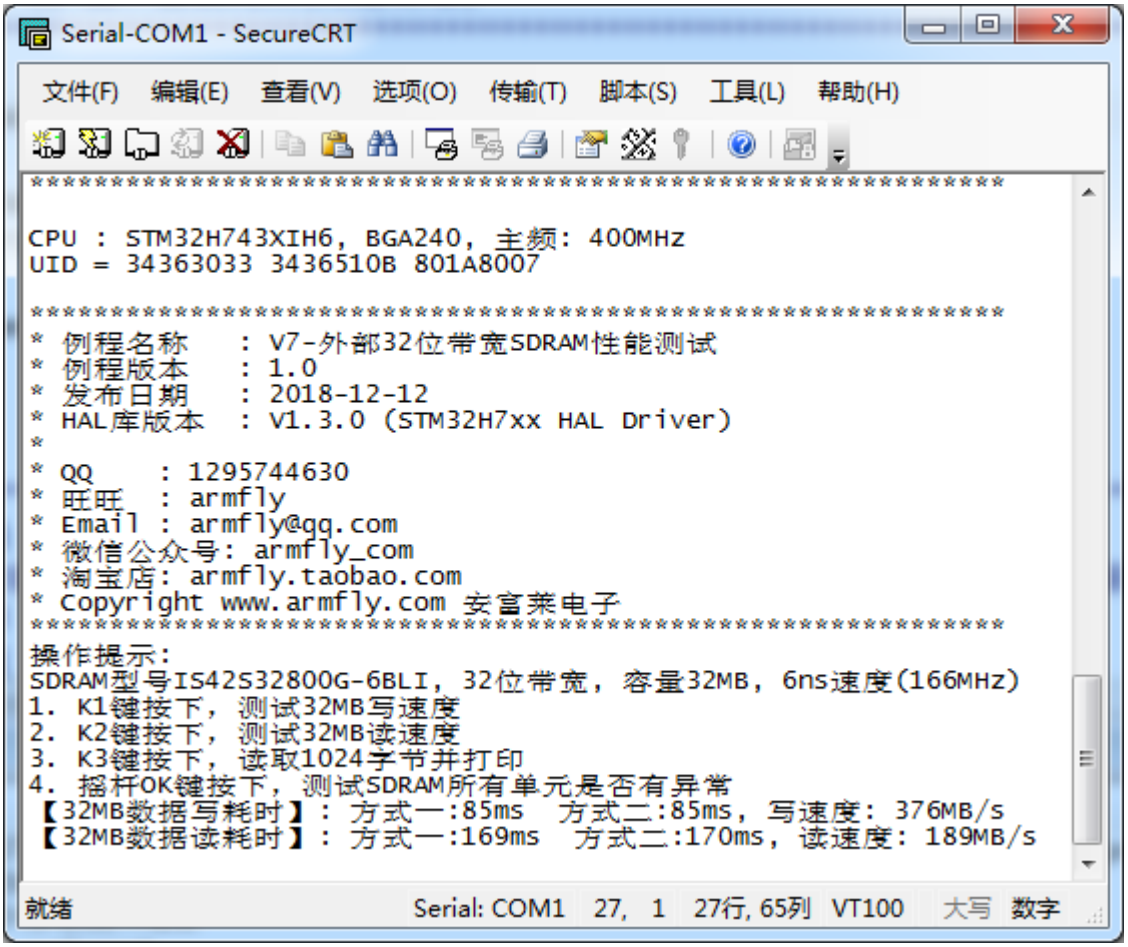
2. 开启 Cache
   （1）使用 MDK 和 IAR 的各种优化等级测试，优化对其影响很小。
   （2）写速度 376MB/S，读速度 182MB/S
3. 关闭 Cache
   （1）使用 MDK 和 IAR 的各种优化等级测试，优化对其影响很小。
   （2）写速度 307MB/S，读速度 116MB/S
4. IAR 开启最高等级优化，读速度是 189MB/S，比 MDK 的 182MB/S 高点。
5. 对于 MDK，本实验开启了最高等级优化和时间优化。
6. 对 IAR，本实验开启了最高等级速度优化。

实验操作：

1. K1 键按下，测试 32MB 写速度;
2. K2 键按下，测试 32MB 读速度;
3. K3 键按下，读取 1024 字节并打印;
4. 摇杆 OK 键按下，测试 SDRAM 所有单元是否有异常。

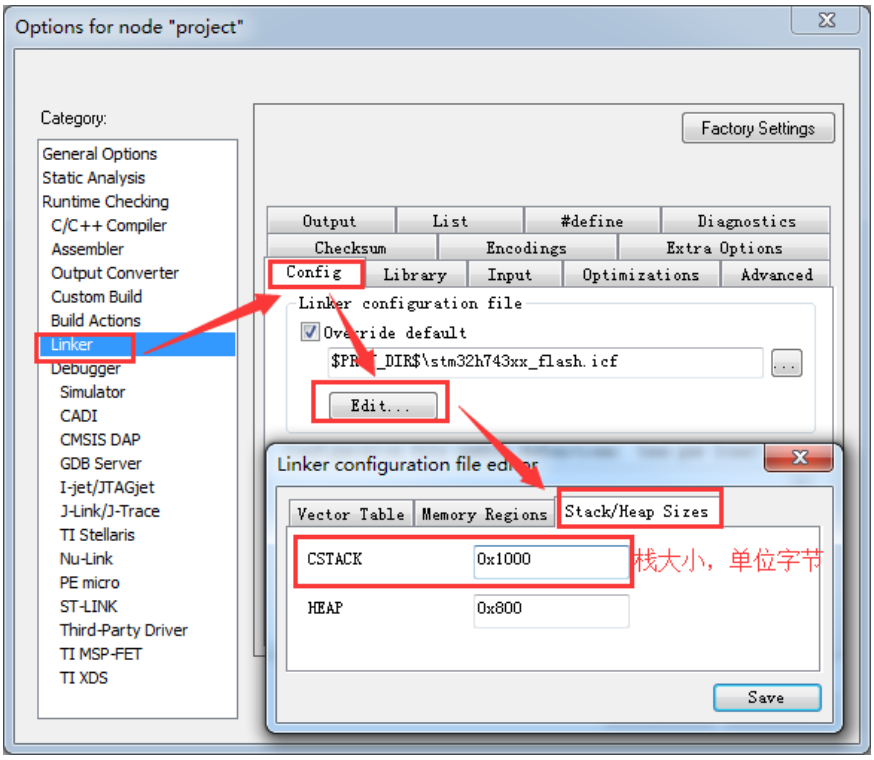
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

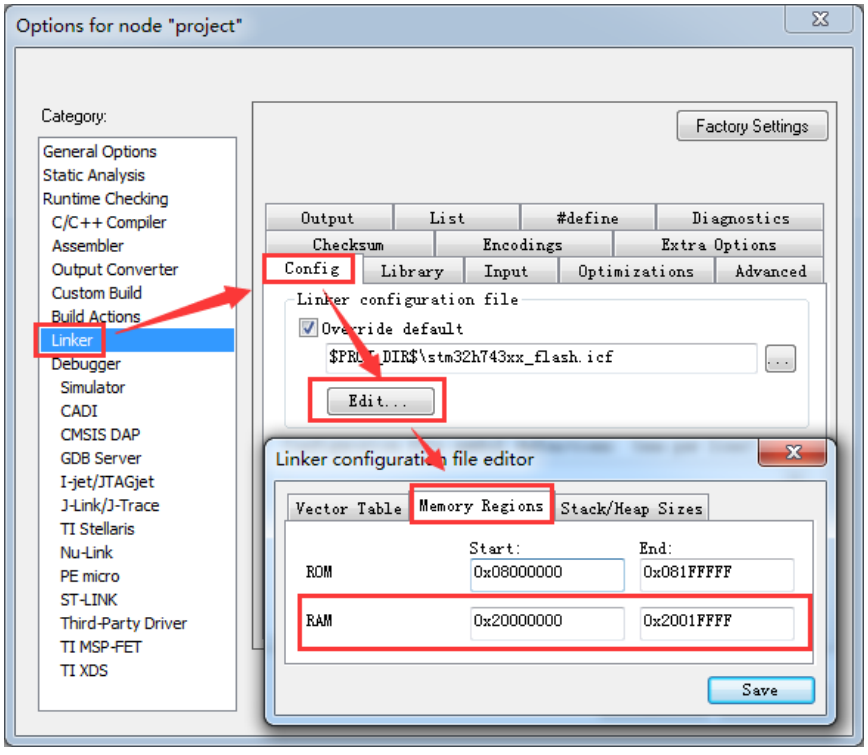


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



- **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SDRAM。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
```

```
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SDRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0xC0000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_32MB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}
```

```
/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- K1 键按下，测试 32MB 写速度;
- K2 键按下，测试 32MB 读速度;
- K3 键按下，读取 1024 字节并打印;
- 摇杆 OK 键按下，测试 SDRAM 所有单元是否有异常。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */

    DemoFmcSRAM(); /* 外部 SDRAM 读写测试 */
}

/*
*********************************************************************************************************
*    函 数 名: DemoFmcSRAM
*    功能说明: SDRAM 读写性能测试
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoFmcSRAM(void)
{
    uint8_t ucKeyCode; /* 按键代码 */
    uint32_t err;

    PrintfHelp(); /* 打印操作提示 */
```

```
    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，测试 32MB 写速度 */
                    WriteSpeedTest();
                    break;

                case KEY_DOWN_K2:           /* K2 键按下，测试 32MB 读速度*/
                    ReadSpeedTest();
                    break;

                case KEY_DOWN_K3:           /* K3 键按下，读取 1024 字节并打印 */
                    ReadWriteTest();
                    break;

                case JOY_DOWN_OK:           /* 摇杆 OK 键按下，测试 SDRAM 所有单元是否有异常*/
                    err = bsp_TestExtSDRAM1();
                    if (err == 0)
                    {
                        printf("外部 SDRAM 测试通过\r\n");
                    }
                    else
                    {
                        printf("外部 SDRAM 出错，错误单元个数：%d\r\n", err);
                    }
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

49.9 总结

本章节就为大家讲解这么多，不同厂家的 SDRAM 驱动基本都是一样的，仅仅是时序参数有些区别，配置时根据 SDRAM 手册上的参数设置即可。

第50章 STM32H7 的 LCD 控制器 LTDC 基础知识和 HAL 库 API

本章节为大家讲解 LCD 控制器 LTDC（LCD-TFT display controller），实际项目中用到显示屏的地方全靠它了，而且性能也比较给力。

50.1 初学者重要提示

50.2 LTDC 基础知识

50.3 LTDC 的 HAL 库用法

50.4 源文件 stm32h7xx_hal_ltdc.c

50.5 总结

50.1 初学者重要提示

- 本章 50.2 小节中的几个知识点比较重要，比如 DE 同步模式和 HV 同步模式的区别，Alpha 混合，水平消隐和垂直消隐等知识点尤其重要，需要大家理解透彻。
- 测评 STM32H7 的 LTDC+DMA2D 性能，100Hz 以上无压力，刷 800*480 图片和色块仅需 2.6ms 一张：http://www.armbbs.cn/forum.php?mod=viewthread&tid=91489 。

50.2 LTDC 基础知识

LTDC 的几个关键知识点放在开头说：

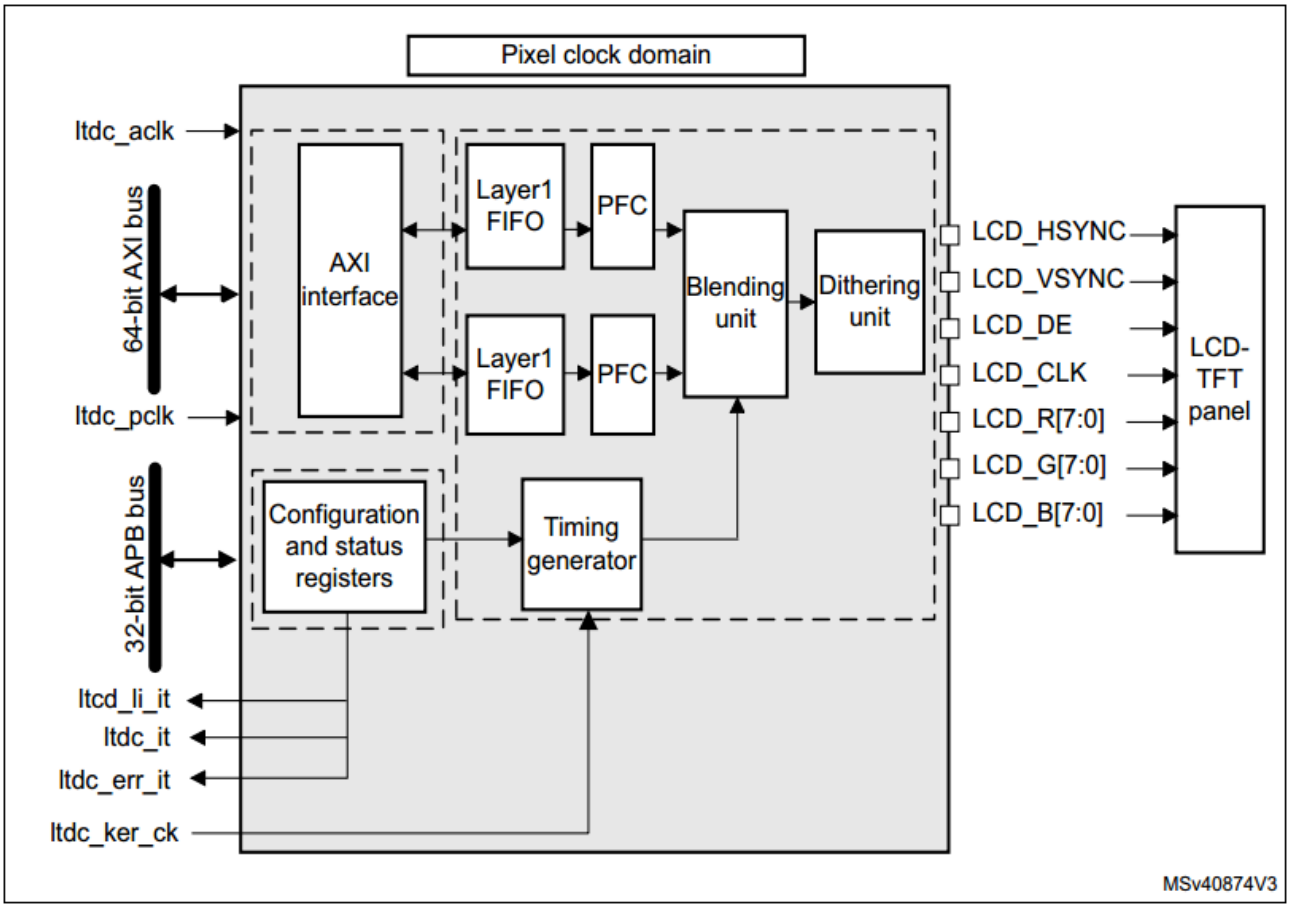
- STM32H7 的 LTDC 最大支持 1024*768 分辨率，且支持硬件双图层。实际支持的分辨率可能比 1024*768 要高一点，因为最终可以支持的最大分辨率是芯片后期定标的。
- 支持 32 位色，24 位色，16 位色和 8 位色。
- 可编程窗口位置和大小，可编程行同步，场同步和数据使能信号的极性。
- 查色表（CLUT，Color look-up table），每个图层最高可记录 256 种 24 位色。
- 支持如下 8 种颜色格式：
  - ARGB8888

    32 位颜色格式，一个像素点占用 4 字节，其中低位 3 字节用于颜色分量，高位字节用于 Alpha 混合。红、绿、蓝和 Alpha 通道（0x00 表示完全透明，0xFF 表示完全不透明）都是 8 位表示。颜色格式：AAAAAAAARRRRRRRRGGGGGGGGBBBBBBBB。

- RGB888
  24 位颜色格式，一个像素点占用 3 字节，分别用于红、绿、蓝。
  颜色格式：RRRRRRRRGGGGGGGGBBBBBBBB。
- RGB565
  16 位颜色格式，一个像素点占用 2 字节，分别用于红、绿、蓝。
  颜色格式：RRRRRGGGGGGBBBBB。
- ARGB1555
  16 位颜色格式，一个像素点占用 2 字节，Alpha 通道使用 1 个位表示，等于 0 的时候表示完全透明，等于 1 的时候表示完全不透明。红、绿、蓝都是用 5 个位表示。
  颜色格式：ARRRRRGGGGGBBBBB。
- ARGB4444
  16 位颜色格式，一个像素点占用 2 字节，Alpha 通道使用 2 个位表示（0x0 表示完全透明，0x3 表示完全不透明）。红、绿、蓝都是用 4 个位表示。
  颜色格式：ARRRRRGGGGGBBBBB。
- L8 (8-bit luminance or CLUT)
  8 位颜色格式,实际上仅仅是 8 位索引值,范围 0–255,而每个索引值的具体颜色值在查色表 CLUT 里面存储。
- AL44 (4-bit alpha + 4-bit luminance)
  8 位颜色格式，实际上是 4 位 Alpha 通道（0x0 表示完全透明，0xF 表示完全不透明）和 4 位的索引值，索引范围 0–15,而每个索引值的具体颜色值在查色表 CLUT 里面存储。
- AL88 (8-bit alpha + 8-bit luminance)
  16 位颜色格式，实际上是 8 位 Alpha 通道（0x00 表示完全透明，0xFF 表示完全不透明）和 8 位的索引值，索引范围 0–255,而每个索引值的具体颜色值在查色表 CLUT 里面存储。

50.2.1 LTDC 硬件框图

认识一个外设，最好的方式就是看它的框图，方便我们快速地了解 LTDC 的基本功能，然后再看手册了解细节。框图如下所示：



通过这个框图，我们可以得到如下信息：

- ltdc_aclk

  为 LTDC 寄存器提供时钟，时钟来自 AXI 时钟域。

- ltdc_pclk

  LTDC 寄存器接口时钟。

- ltdc_ker_ck

  用于生成 LCD_CLK（像素时钟输出）的 LTDC 内核时钟。

- ltdc_li_it

  LTDC 行中断，用于触发 MDMA。

- ltdc_it

  LTDC 全局中断请求。

- ltdc_err_it

  LTDC 全局错误中断请求。

下面是 LCD 接口引脚，用于外接显示屏：

- LCD_CLK

  像素时钟输出。

- LCD_HSYNC

  水平同步信号。

- LCD_VSYN

  垂直同步信号。

- LCD_DE

  数据使能信号。

- LCD_R[7:0]

  8 位红色数据。

- LCD_G[7:0]

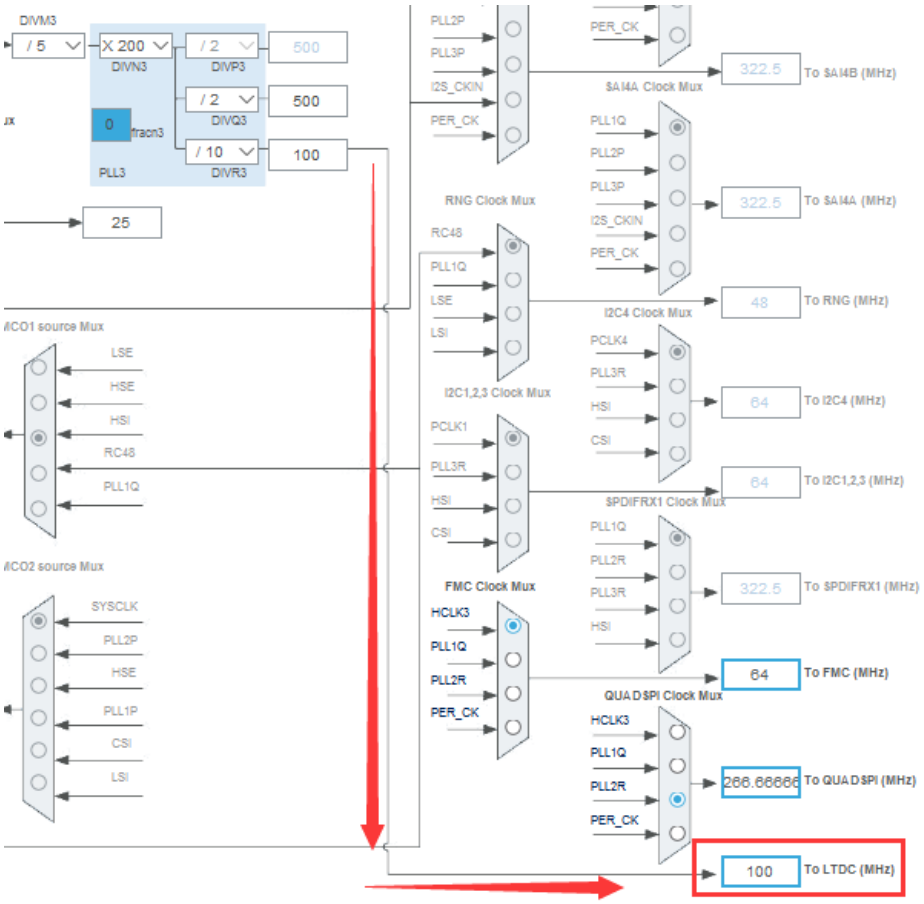
  8 位绿色数据。

- LCD_B[7:0]

  8 位蓝色数据。

50.2.2 LTDC 时钟源选择

LTDC 仅有一个时钟源可供选择，即 PLL3R。



50.2.3 LCD 的 DE 同步模式和 HV 同步模式的区别

一般情况下，STM32H7 都是用 SDRAM 作为 LCD 的显存，LTDC 控制器会从 SDRAM 读取数据刷新到 LCD 显示屏上，具体如何刷新呢？这就涉及到 DE 同步模式和 HV 同步模式。

具体支持哪种模式是由裸屏自带的 Driver IC 决定，比如 V7 板子 7 寸裸屏的 Source Driver IC OTA7001 支持 DE 和 HV 两种模式。现在的大分辨率显示屏一般都是 DE 同步模式，小分辨率的 HV 同步模式多。

- DE 同步模式

DE 模式需要 LCD_DE 和 LCD_CLK 信号来控制刷新。比如一个 800x480 分辨率的裸屏，在 DE 有效信号的时候（高电平或低电平），就有 800 个 LCD_CLK 输出时钟来确认行中 800 个点。每个时钟有效的时候，从显存读取一次 RGB 数据。因为存在回扫信号，所以 DE 是个方波。一个周期的 LCD_DE 信号，裸屏就扫描一行。扫描 480 行后，又从第一行扫描开始。这个规律由裸屏的驱动 IC 所决定的。

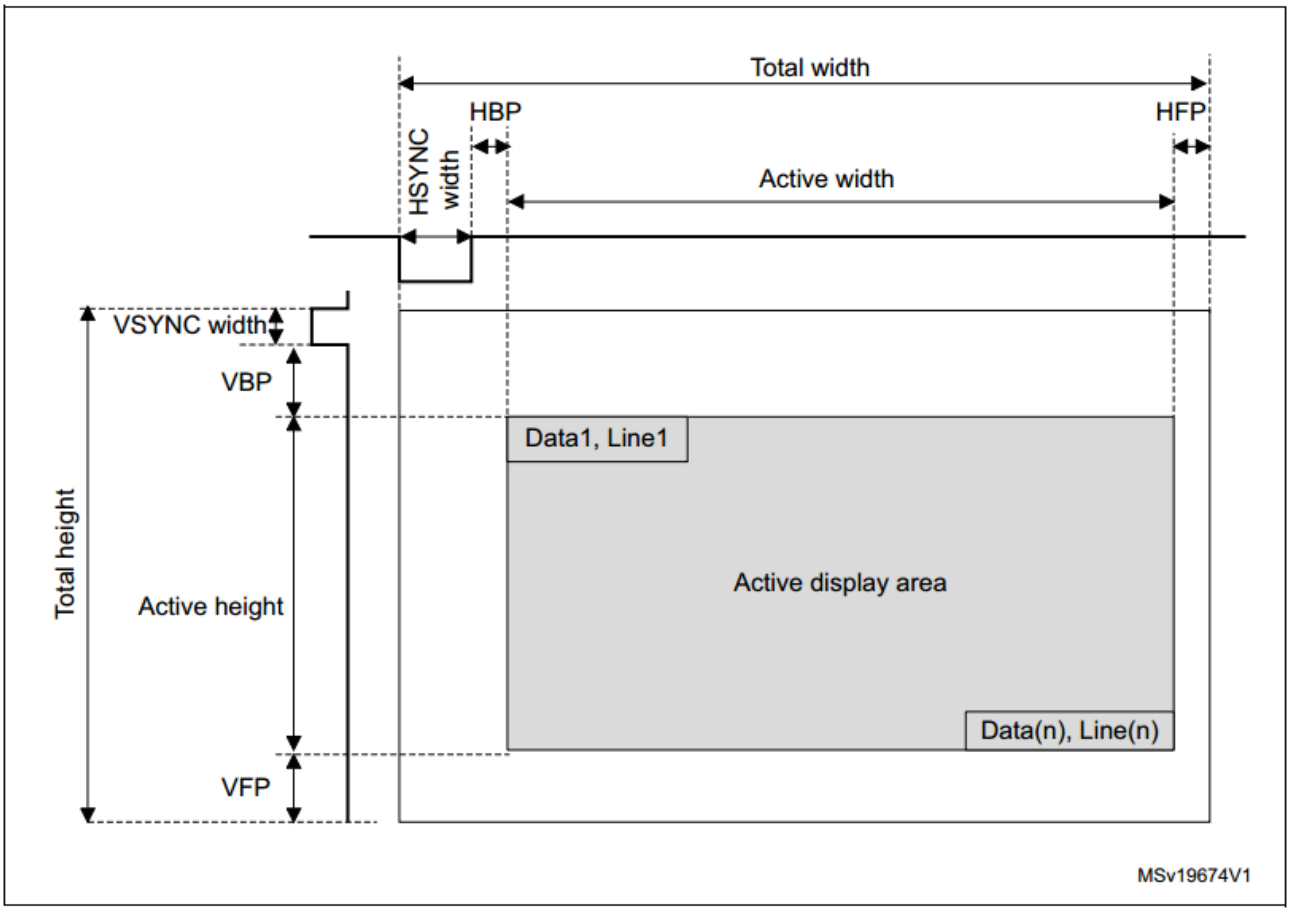
- HV 同步模式

HV 模式需要 LCD_CLK 时钟信号，行同步信号 LCD_HSYNC 和场同步信号 LCD_VSYNC 来控制刷新。比如一个 480x272 分辨率的裸屏，有一个行同步信号 LCD_HSYNC 产生时（高电平或者低电平脉冲），就有 480 个 LCD_CLK 输出时钟来确认行中 480 个点。每个时钟有效的时候，从显存读取一次 RGB 数据。再来一个行同步信号 LCD_HSYNC 产生时（高电平或者低电平脉冲），切换到下一行，继续行同步和时钟输出，扫描 272 行后，发送一个场同步信号 LCD_VSYNC，又重新从第一行扫描开始。

具体的时序效果可以看第 51 章的内容。

50.2.4 LTDC 的时序配置

LTDC 的时序控制就是下面几个参数的设置，这几个参数都可以通过寄存器进行配置。



- HSYNC width

  水平同步宽度设置，以 LCD_CLK 的像素时钟输出为单位。

- HBP（horizontal back porch period）

  水平后沿周期设置，以 LCD_CLK 的像素时钟输出为单位。

- Active width

  有效宽度设置，以 LCD_CLK 的像素时钟输出为单位。以 800*480 分辨率为例，Active width = 800。

- HFP（horizontal front porch period）

  水平前沿周期设置，以 LCD_CLK 的像素时钟输出为单位。

- VSYNC width

  垂直同步宽度设置，以 LCD_CLK 的像素时钟输出为单位。

- VBP（vertical back porch period）

  垂直后沿周期设置，以 LCD_CLK 的像素时钟输出为单位。

- Active height

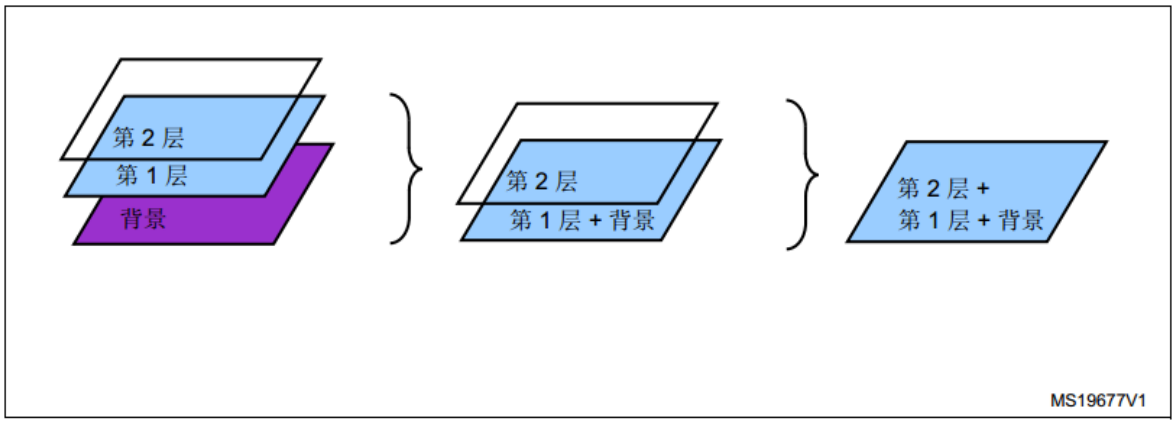
  有效高度设置，以 LCD_CLK 的像素时钟输出为单位。以 800*480 分辨率为例，Active height = 480。

- VFP（vertical front porch period）

  垂直前沿周期设置，以 LCD_CLK 的像素时钟输出为单位。

50.2.5 LTDC 背景层，图层 1，图层 2 和 Alpha 混合

LTDC 除了图层 1 和图层 2 两个硬件图层以外，还有一个背景层。由于背景层的刷新不需要显存空间，所以可以用这个图层验证 LTDC 时序配置是否有问题。



- 对于背景层来说，仅支持单色设置，固定颜色格式 RGB888（LTDC_HandleTypeDef hltdc_F）
  hltdc_F.Init.Backcolor.Blue = 0
  hltdc_F.Init.Backcolor.Green = 0
  hltdc_F.Init.Backcolor.Red = 0
- 对于图层 1 和图层 2 来说，支持如下 8 种颜色格式：
  - ARGB8888
  - RGB888
  - RGB565
  - ARGB1555
  - ARGB4444
  - L8（8 位 Luminance 或 CLUT）
  - AL44（4 位 alpha + 4 位 luminance）
  - AL88（8 位 alpha + 8 位 luminance）
- 实现 Alpha 混合的关键是要有一个变量可以设置各种透明度。对此，STM32H7 准备了两个 Alpha 供使用：
  - 一个是常数 Alpha（0x00 表示完全透明，0xFF 表示完全不透明），所有颜色格式都可以使用。
  - 另一个是像素 Alpha，也就是 ARGB8888，ARGB1555，ARGB4444 等颜色格式的 Alpha 通道数值，也就是我们为图层每个位置绘制的实际颜色值。

==========

了解了这点后就是具体的实现了。STM32H7 的参考手册给出了具体的混合公式

BC = BF1 x C + BF2 x Cs
混合后的颜色= 混合系数 1 x 当前层颜色 + 混合系数 2 x 底层混合后的颜色

◆ 混合系数 1 可以选择：

（1）常数 Alpha

（2）像素 Alpha x 常数 Alpha

◆ 混合系数 2 可以选择：

（1）1 - 常数 Alpha

（2）1 - 像素 Alpha x 常数 Alpha

◆ 底层混合后的颜色：

（1）可以是背景层。

（2）可以是背景层与图层 1 混合后的颜色。

◆ 那么公式就变成如下形式（主要是如下两种）：

混合后的颜色 = 常数 Alpha x 当前层颜色 + （1 - 常数 Alpha） x 底层混合后的颜色。

混合后的颜色 = 像素 Alpha x 常数 Alpha x 当前层颜色 +
（1 - 像素 Alpha x 常数 Alpha） x 底层混合后的颜色。

◆ 再进一步简化，代入 HAL 库参数（LTDC_LayerCfgTypeDef pLayerCfg）：

像素 Alpha 是 ARGB8888，ARGB1555 等颜色格式的 Alpha 数值，也就是我们为图层每个位置绘制的实际颜色值，我们这里用 AlphaValue 表示。

混合后的颜色 = （pLayerCfg.Alpha /255 ） x 当前层颜色 +
（1 - pLayerCfg.Alpha /255 ） x 底层混合后的颜色。

混合后的颜色 = （AlphaValue/255）x （pLayerCfg.Alpha /255 ） x 当前层颜色 +
（1 - （AlphaValue/255）x （pLayerCfg.Alpha /255 ）） x 底层混合后的颜色。

注，Alpha 值要做归一化，Alpha 的范围是 0 - 255，比如 Alpha = 100，那么代入公式的时候就是 100/255。

◆ 举例：

（1）混合因数 1 选择像素 Alpha x 常数 Alpha。

（2）混合因数 2 选择像素 Alpha x 常数 Alpha。

（3）使用图层 1 和背景层，图层 1 使用 ARGB8888 颜色格式。

（4）背景色 = 0xFF0000，图层 1 位置坐标（0,0）颜色值 0x5500FF00。

（5）Alpha 常数 = 0x77

混合后的颜色 = （AlphaValue/255）x （pLayerCfg.Alpha /255 ） x 当前层颜色 +
（1 - （AlphaValue/255）x （pLayerCfg.Alpha /255 ）） x 底层混合后的颜色。
= （0x55/255）x （0x77/255） x 0x00FF00 +
（1 - （0x55/255）x （0x77/255）） x 0xFF0000
= 0xD7809C

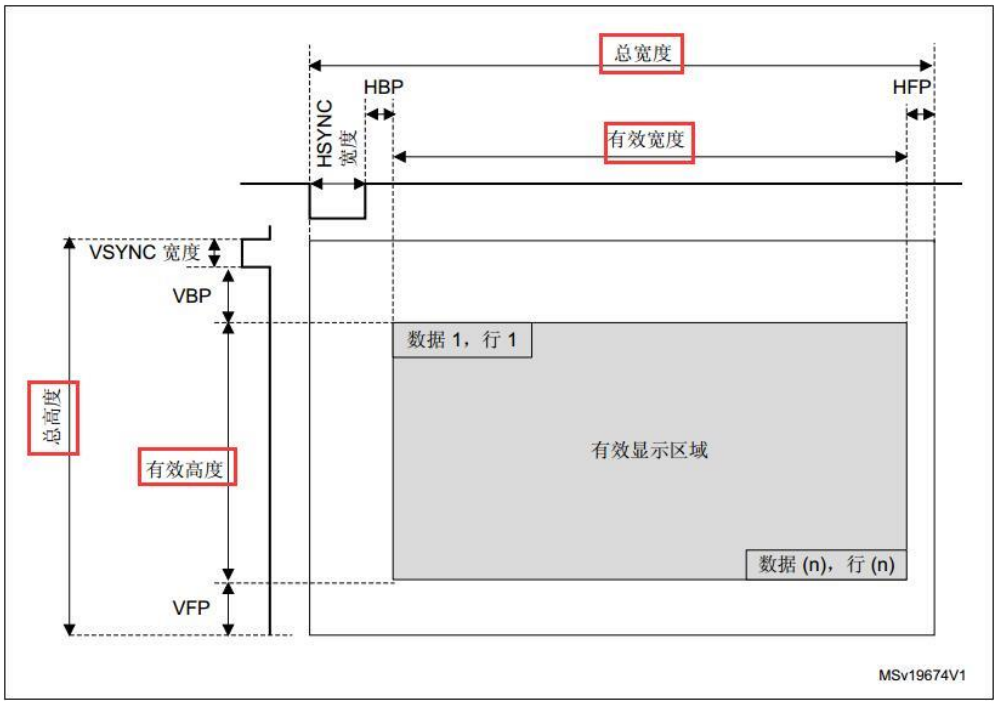
◆ 最后注意一点：

如果大家设置的图层显示区没有显示满整个显示屏，且使用的 ARGB 颜色格式，那么未覆盖的区域会使用图层默认颜色。对此 HAL 库有专门的配置：

```
LTDC_LayerCfgTypeDef        pLayerCfg
pLayerCfg.Alpha0 = 0;
pLayerCfg.Backcolor.Blue = 0;
pLayerCfg.Backcolor.Green = 0;
pLayerCfg.Backcolor.Red = 0;
```

50.2.6 LTDC 的水平消隐和垂直消隐

正常情况下，LCD 的刷新就是从左到右，从上到下进行逐个像素点刷新。但仅刷新有效的显示区是不够的，比如 800*480 分辨率，我们不仅仅要刷 800*480 这段有效区域，边界区也是要刷新的，即下图总宽度以内，有效宽度以外的区域。



水平消隐就是 LCD 扫描一行结束到另一行开始的时间，这段消失的时间就是水平消隐，即 HSYNC 宽度+ HBP + HFP 这段消失的时间。

垂直消隐就是 LCD 扫描最后一行结束到第一行开始的时间,这段消失的时间就是垂直消隐,即 VSYNC 宽度+ VBP + VFP 这段消失的时间。

50.2.7 区分 FPS 帧率和刷新率

FPS 帧率是对 STM32H7 刷到显存，也就是 SDRAM 里面来说的，而是刷新率是实际 LCD 显示的速度。

```
刷新率 = LTDC 输出时钟 /（（Width + HSYNC_W  + HBP  + HFP ）*(Height + VSYNC_W +  VBP  +VFP  ))
```

一般情况下，帧率是远高于刷新率的，但帧数高于刷新率有没有意义？网上普遍的看法是高于刷新率的帧数都是浪费，比如 H7 刷 800*480 分辨率显示屏，帧数可以达到 300 多，而刷新率估计才 108Hz，多出来的不是都浪费了吗（对于那种 FPS 类的游戏，我们不讨论）。

对于这种观点，在一定情况下是成立的。但是有一点，即使是帧数和刷新率都是 108Hz，能保证显示没有撕裂感吗？能保证没有帧延迟吗？能保证流畅的画面吗？这个时候,我们使用 emWin 支持的三缓冲，多余的帧数完全可以输出到其它缓冲区，有效降低撕裂感和帧延迟，保证流畅输出。

50.2.8 避免 LTDC 刷新撕裂感的解决办法

如果用户快速刷新颜色差异比较大的两种界面，容易遇到这种撕裂问题。

- 出现这个问题的原因

用户更新显存数据期间，LTDC 也在不断的读取显存的数据到显示屏上，如果用户才更新了部分界面数据，后面部分还没有更新，LTDC 刷新到显示屏的界面效果出现撕裂感，即下面这种现象：

- 解决这个问题的办法

LTDC 刷新还在垂直消隐期间就将整个界面刷新完成，而我们如何只知道 LTDC 在垂直消隐期，通过函数 HAL_LTDC_ProgramLineEvent 设置刷新到指定行时进入中断即可，一般设置为第 0 行进入中断，然后设置个标志即可。

一旦检测到这个标志，就通过 DMA2D 快速将界面刷新好，这样就有效的避免了撕裂感。

50.3 LTDC 的 HAL 库用法

LTDC 的 HAL 库用法其实就是几个结构体变量成员的配置和使用，然后配置 GPIO、时钟，并根据需要配置 NVIC、中断和 DMA。下面我们逐一展开为大家做个说明。

50.3.1 LTDC 寄存器结构体 LTDC_TypeDef

LTDC 相关的寄存器是通过 HAL 库中的结构体 LTDC_TypeDef 定义的，在 stm32h743xx.h 中可以找到它们的具体定义：

```
typedef struct
{
  uint32_t      RESERVED0[2];/*!< Reserved,                                                    0x00-0x04 */
  __IO uint32_t SSCR;      /*!< LTDC Synchronization Size Configuration Register,    Address offset: 0x08 */
  __IO uint32_t BPCR;      /*!< LTDC Back Porch Configuration Register,              Address offset: 0x0C */
  __IO uint32_t AWCR;      /*!< LTDC Active Width Configuration Register,            Address offset: 0x10 */
  __IO uint32_t TWCR;      /*!< LTDC Total Width Configuration Register,             Address offset: 0x14 */
  __IO uint32_t GCR;       /*!< LTDC Global Control Register,                        Address offset: 0x18 */
  uint32_t      RESERVED1[2]; /*!< Reserved,                                                  0x1C-0x20 */
  __IO uint32_t SRCR;      /*!< LTDC Shadow Reload Configuration Register,           Address offset: 0x24 */
  uint32_t      RESERVED2[1]; /*!< Reserved,                                                       0x28 */
  __IO uint32_t BCCR;      /*!< LTDC Background Color Configuration Register,        Address offset: 0x2C */
  uint32_t      RESERVED3[1];  /*!< Reserved,                                                     0x30 */
  __IO uint32_t IER;       /*!< LTDC Interrupt Enable Register,                      Address offset: 0x34 */
  __IO uint32_t ISR;       /*!< LTDC Interrupt Status Register,                      Address offset: 0x38 */
  __IO uint32_t ICR;       /*!< LTDC Interrupt Clear Register,                       Address offset: 0x3C */
  __IO uint32_t LIPCR;     /*!< LTDC Line Interrupt Position Configuration Register, Address offset: 0x40 */
  __IO uint32_t CPSR;      /*!< LTDC Current Position Status Register,               Address offset: 0x44 */
  __IO uint32_t CDSR;      /*!< LTDC Current Display Status Register,                Address offset: 0x48 */
} LTDC_TypeDef;
```

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

下面我们再看 LTDC 的定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE           ((uint32_t)0x40000000)
#define D1_APB1PERIPH_BASE    (PERIPH_BASE + 0x10000000)

#define LTDC_BASE             (D1_APB1PERIPH_BASE + 0x1000)
#define LTDC                  ((LTDC_TypeDef *)LTDC_BASE) <----- 展开这个宏，(LTDC_TypeDef *) 0x50001000
```

我们访问 LTDC 的 ISR 寄存器可以采用这种形式：LTDC->ISR = 0。

50.3.2 LTDC 参数初始化结构体 LTDC_InitTypeDef

此结构体用于配置 LTDC 的基本参数，具体定义如下：

```
typedef struct
{
  uint32_t            HSPolarity;
  uint32_t            VSPolarity;
  uint32_t            DEPolarity;
  uint32_t            PCPolarity;
  uint32_t            HorizontalSync;
  uint32_t            VerticalSync;
  uint32_t            AccumulatedHBP;
  uint32_t            AccumulatedVBP;
  uint32_t            AccumulatedActiveW;
  uint32_t            AccumulatedActiveH;
  uint32_t            TotalWidth;
  uint32_t            TotalHeigh;
  LTDC_ColorTypeDef   Backcolor;
} LTDC_InitTypeDef;
```

下面将这几个参数逐一为大家做个说明：

- uint32_t HSPolarity

此参数用于设置水平同步信号极性，具体支持的参数如下：

```
#define LTDC_HSPOLARITY_AL   (0x00000000U)            /* 水平同步极性低电平有效 */
#define LTDC_HSPOLARITY_AH    LTDC_GCR_HSPOL          /* 水平同步极性高电平有效 */
```

- uint32_t VSPolarity

此参数用于设置垂直同步信号极性，具体支持的参数如下：

```
#define LTDC_VSPOLARITY_AL    (0x00000000U)           /* 垂直同步极性低电平有效 */
#define LTDC_VSPOLARITY_AH    LTDC_GCR_VSPOL          /* 垂直同步极性高电平有效 */
```

- uint32_t DEPolarity

此参数用于设置数据使能极性，具体支持的参数如下：

```
#define LTDC_DEPOLARITY_AL    (0x00000000U)           /* 数据使能极性低电平有效 */
#define LTDC_DEPOLARITY_AH     LTDC_GCR_DEPOL         /* 数据使能极性高电平有效 */
```

- uint32_t PCPolarity

此参数用于设置像素时钟极性，具体支持的参数如下：

```
#define LTDC_PCPOLARITY_IPC     (0x00000000U)        /* 像素时钟极性低电平有效 */
#define LTDC_PCPOLARITY_IIPC    LTDC_GCR_PCPOL       /* 像素时钟极性告电平有效 */
```

- uint32_t HorizontalSync

此参数用于设置水平同步宽度，范围 0x000 – 0xFFF，单位像素时钟个数。

- uint32_t VerticalSync

此参数用于设置垂直同步宽度，范围 0x000 – 0x7FF，单位像素时钟个数。

- uint32_t AccumulatedHBP

此参数用于设置 HSYNC 水平同步宽度 + HBP 水平后沿之和,范围 HSYNC 水平同步宽度到 0xFFF，单位像素时钟个数。

- uint32_t AccumulatedVBP

此参数用于设置 VSYNC 垂直同步宽度 + VBP 垂直后沿之和，范围 VSYNC 垂直同步宽度到 0x7FF，单位像素时钟个数。

- uint32_t AccumulatedActiveW

  此参数用于设置 HSYNC 水平同步宽度 + HBP 水平后沿 + 有效宽度之和，范围 AccumulatedHBP 到 0xFFF，单位像素时钟个数。

- uint32_t AccumulatedActiveH

  此参数用于设置 VSYNC 垂直同步宽度 + VBP 垂直后沿 + 有效高度之和，范围 AccumulatedVBP 到 0x7FF，单位像素时钟个数。

- uint32_t TotalWidth

  此参数用于设置 HSYNC 水平同步宽度 + HBP 水平后沿 + 有效宽度 + HFP 水平前沿之和，范围 AccumulatedActiveW 到 0xFFF，单位像素时钟个数。

- uint32_t TotalHeigh

  此参数用于设置 VSYNC 垂直同步宽度 + VBP 垂直后沿 + 有效高度 +VFP 垂直前沿之和，范围 AccumulatedActiveH 到 0x7FF，单位像素时钟个数。

- LTDC_ColorTypeDef Backcolor

  此参数用于设置背景层颜色，结构体 LTDC_ColorTypeDef 的定义如下：

```
typedef struct
{
  uint8_t Blue;
  uint8_t Green;
  uint8_t Red;
  uint8_t Reserved;
} LTDC_ColorTypeDef;
```

Bule 用于设置蓝色值，范围 0x00 到 0xFF。

Green 用于设置绿色值，范围 0x00 到 0xFF。

Red 用于设置红色值，范围 0x00 到 0xFF。

50.3.3 LTDC 图层配置结构体 LTDC_LayerCfgTypeDef

此结构体用于配置 LTDC 的图层，图层 1 和图层 2 均通过此结构体配置。

```
typedef struct
{
  uint32_t WindowX0;
  uint32_t WindowX1;
  uint32_t WindowY0;
  uint32_t WindowY1;
  uint32_t PixelFormat;
  uint32_t Alpha;
  uint32_t Alpha0;
  uint32_t BlendingFactor1;
  uint32_t BlendingFactor2;
  uint32_t FBStartAdress;
```

```
  uint32_t ImageWidth;
  uint32_t ImageHeight;
  LTDC_ColorTypeDef   Backcolor;
} LTDC_LayerCfgTypeDef;
```

下面将这几个参数逐一为大家做个说明。

- uint32_t WindowX0

  设置图层水平起始位置，范围 0x000 到 0xFFF。

- uint32_t WindowX1

  设置图层水平结束位置，范围 0x000 到 0xFFF。

- uint32_t WindowY0

  设置图层垂直起始位置，范围 0x000 到 0x7FF。

- uint32_t WindowY1

  设置图层垂直结束位置，范围 0x000 到 0x7FF。

- uint32_t PixelFormat

  设置图层所使用的颜色格式。

- uint32_t Alpha

  设置常数 Alpha，范围 0x00 – 0xFF。

- uint32_t Alpha0

  设置图层默认 Alpha 值，范围 0x00 – 0xFF，与结构体成员 Backcolor 一起使用，组成 ARGB 颜色格式用于图层背景色。

- uint32_t BlendingFactor1

  设置混合因数 1，具体支持的参数如下：

```
#define LTDC_BLENDING_FACTOR1_CA      (0x00000400U)   /* 常数 Alpha */
#define LTDC_BLENDING_FACTOR1_PAxCA   (0x00000600U)   /* 常数 Alpha * 像素 Alpha */
```

- uint32_t BlendingFactor2

  设置混合因数 2，具体支持的参数如下：

```
#define LTDC_BLENDING_FACTOR2_CA      (0x00000400U)   /* 常数 Alpha */
#define LTDC_BLENDING_FACTOR2_PAxCA   (0x00000600U)   /* 常数 Alpha * 像素 Alpha */
```

- uint32_t FBStartAdress

  设置颜色帧缓冲区地址，即图层的显存地址。

- uint32_t ImageWidth

  设置颜色帧缓冲区行长，即要从显存读取一行的长度，范围 0x0000 到 0x1FFF。

- uint32_t ImageHeight

  设置颜色帧缓冲区行数，即要从显存读取的行数，范围 0x000 到 0xFFF。

- LTDC_ColorTypeDef Backcolor

  此参数用于设置图层默认色，结构体 LTDC_ColorTypeDef 的定义如下：

```
typedef struct
{
  uint8_t Blue;
```

```
  uint8_t Green;
  uint8_t Red;
  uint8_t Reserved;
} LTDC_ColorTypeDef;
```

Bule 用于设置蓝色值，范围 0x00 到 0xFF。

Green 用于设置绿色值，范围 0x00 到 0xFF。

Red 用于设置红色值，范围 0x00 到 0xFF。

注：如果大家设置的图层显示区没有显示满整个显示屏，且使用的 ARGB 颜色格式，那么未覆盖的区域会使用图层默认颜色。

50.3.4LTDC 句柄结构体 LTDC_HandleTypeDef

HAL 库在 LTDC_TypeDef， LTDC_InitTypeDef 和 LTDC_LayerCfgTypeDef 的基础上封装了一个结构体 LTDC_HandleTypeDef，定义如下：

```
typedef struct
{
  LTDC_TypeDef                *Instance;
  LTDC_InitTypeDef            Init;
  LTDC_LayerCfgTypeDef        LayerCfg[MAX_LAYER];
  HAL_LockTypeDef             Lock;
  __IO HAL_LTDC_StateTypeDef  State;
  __IO uint32_t               ErrorCode;
} LTDC_HandleTypeDef;
```

下面将这几个参数逐一做个说明。

- **LTDC_TypeDef *Instance**

  这个参数是寄存器的例化，方便操作寄存器，详见本章 3.1 小节。

- **LTDC_InitTypeDef Init;**

  这个参数是用户接触较多的，用于配置 LTDC 的时序参数配置，详见本章 3.2 小节。

- **LTDC_LayerCfgTypeDef LayerCfg[MAX_LAYER]**

  这个参数用于 LTDC 的图层配置，对于 STM32H7 来说，支持双图层，MAX_LAYER=2。详见本章 3.3 小节。

- **HAL_LockTypeDef Lock**
  __IO uint32_t State;
  __IO uint32_t ErrorCode

  这三个变量主要供函数内部使用。Lock 用于设置锁状态，State 用于设置 LTDC 通信状态，而 ErrorCode 用于配置代码错误。

50.3.5 LTDC 初始化流程总结

在下章的 51.4 小节给出了详细的设计步骤。

50.4 源文件 stm32h7xx_hal_ltdc.c

这里把我们把如下几个常用到的函数做个说明：

- HAL_LTDC_Init
- HAL_LTDC_ConfigLayer
- HAL_LTDC_SetAlpha
- HAL_LTDC_Reload
- HAL_LTDC_SetPixelFormat
- HAL_LTDC_SetWindowPosition
- HAL_LTDC_SetWindowSize_NoReload

LTDC 的 API 函数主要分为两类：

- 一类是以_NoReload 作为后缀

  带后缀_NoReload 的函数没有对重装寄存器进行配置。如果要配置的话，可以通过函数 HAL_LTDC_Reload 来立即更新配置或者在垂直消隐期间更新。

- 另一类是没有此后缀

  不带后缀_NoReload 的函数会立即更新配置。

==========

能够实现这种操作的关键是 LTDC 外设有影子寄存器。如果选择不立即更新，可以将用户配置先放到影子寄存器，等垂直消隐期间再更新，这样做的好处是可以整体更新这些寄存器。下面是部分函数：

```
HAL_StatusTypeDef HAL_LTDC_ConfigLayer(LTDC_HandleTypeDef *hltdc, LTDC_LayerCfgTypeDef *pLayerCfg, uint32_t LayerIdx);
HAL_StatusTypeDef HAL_LTDC_ConfigLayer_NoReload(LTDC_HandleTypeDef *hltdc, LTDC_LayerCfgTypeDef *pLayerCfg, uint32_t LayerIdx);

HAL_StatusTypeDef HAL_LTDC_SetWindowSize(LTDC_HandleTypeDef *hltdc, uint32_t XSize, uint32_t YSize, uint32_t LayerIdx);
HAL_StatusTypeDef HAL_LTDC_SetWindowSize_NoReload(LTDC_HandleTypeDef *hltdc, uint32_t XSize, uint32_t YSize, uint32_t LayerIdx);

HAL_StatusTypeDef HAL_LTDC_SetWindowPosition(LTDC_HandleTypeDef *hltdc, uint32_t X0, uint32_t Y0, uint32_t LayerIdx);
HAL_StatusTypeDef HAL_LTDC_SetWindowPosition_NoReload(LTDC_HandleTypeDef *hltdc, uint32_t X0, uint32_t Y0, uint32_t LayerIdx);

HAL_StatusTypeDef HAL_LTDC_SetPixelFormat(LTDC_HandleTypeDef *hltdc, uint32_t Pixelformat, uint32_t LayerIdx);
HAL_StatusTypeDef HAL_LTDC_SetPixelFormat_NoReload(LTDC_HandleTypeDef *hltdc, uint32_t Pixelformat, uint32_t LayerIdx);

HAL_StatusTypeDef HAL_LTDC_SetAlpha(LTDC_HandleTypeDef *hltdc, uint32_t Alpha, uint32_t LayerIdx);
HAL_StatusTypeDef HAL_LTDC_SetAlpha_NoReload(LTDC_HandleTypeDef *hltdc, uint32_t Alpha, uint32_t LayerIdx);

HAL_StatusTypeDef HAL_LTDC_SetAddress(LTDC_HandleTypeDef *hltdc, uint32_t Address, uint32_t LayerIdx);
HAL_StatusTypeDef HAL_LTDC_SetAddress_NoReload(LTDC_HandleTypeDef *hltdc, uint32_t Address, uint32_t LayerIdx);
```

```
HAL_StatusTypeDef HAL_LTDC_SetPitch(LTDC_HandleTypeDef *hltdc, uint32_t LinePitchInPixels, uint32_t LayerIdx);
HAL_StatusTypeDef HAL_LTDC_SetPitch_NoReload(LTDC_HandleTypeDef *hltdc, uint32_t LinePitchInPixels, uint32_t LayerIdx);

HAL_StatusTypeDef HAL_LTDC_ConfigColorKeying(LTDC_HandleTypeDef *hltdc, uint32_t RGBValue, uint32_t LayerIdx);
HAL_StatusTypeDef HAL_LTDC_ConfigColorKeying_NoReload(LTDC_HandleTypeDef *hltdc, uint32_t RGBValue, uint32_t LayerIdx);

HAL_StatusTypeDef HAL_LTDC_EnableColorKeying(LTDC_HandleTypeDef *hltdc, uint32_t LayerIdx);
HAL_StatusTypeDef HAL_LTDC_EnableColorKeying_NoReload(LTDC_HandleTypeDef *hltdc, uint32_t LayerIdx);

HAL_StatusTypeDef HAL_LTDC_DisableColorKeying(LTDC_HandleTypeDef *hltdc, uint32_t LayerIdx);
HAL_StatusTypeDef HAL_LTDC_DisableColorKeying_NoReload(LTDC_HandleTypeDef *hltdc, uint32_t LayerIdx);

HAL_StatusTypeDef HAL_LTDC_EnableCLUT(LTDC_HandleTypeDef *hltdc, uint32_t LayerIdx);
HAL_StatusTypeDef HAL_LTDC_EnableCLUT_NoReload(LTDC_HandleTypeDef *hltdc, uint32_t LayerIdx);

HAL_StatusTypeDef HAL_LTDC_DisableCLUT(LTDC_HandleTypeDef *hltdc, uint32_t LayerIdx);
HAL_StatusTypeDef HAL_LTDC_DisableCLUT_NoReload(LTDC_HandleTypeDef *hltdc, uint32_t LayerIdx);
```

50.4.1 函数 HAL_LTDC_Init

函数原型：

```
HAL_StatusTypeDef HAL_LTDC_Init(LTDC_HandleTypeDef *hltdc)
{
  uint32_t tmp = 0, tmp1 = 0;

  /* 检测参数是否有效 */
  if(hltdc == NULL)
  {
    return HAL_ERROR;
  }

  /* 检查函数参数 */
  assert_param(IS_LTDC_ALL_INSTANCE(hltdc->Instance));
  assert_param(IS_LTDC_HSYNC(hltdc->Init.HorizontalSync));
  assert_param(IS_LTDC_VSYNC(hltdc->Init.VerticalSync));
  assert_param(IS_LTDC_AHBP(hltdc->Init.AccumulatedHBP));
  assert_param(IS_LTDC_AVBP(hltdc->Init.AccumulatedVBP));
  assert_param(IS_LTDC_AAH(hltdc->Init.AccumulatedActiveH));
  assert_param(IS_LTDC_AAW(hltdc->Init.AccumulatedActiveW));
  assert_param(IS_LTDC_TOTALH(hltdc->Init.TotalHeigh));
  assert_param(IS_LTDC_TOTALW(hltdc->Init.TotalWidth));
  assert_param(IS_LTDC_HSPOL(hltdc->Init.HSPolarity));
  assert_param(IS_LTDC_VSPOL(hltdc->Init.VSPolarity));
  assert_param(IS_LTDC_DEPOL(hltdc->Init.DEPolarity));
  assert_param(IS_LTDC_PCPOL(hltdc->Init.PCPolarity));

  if(hltdc->State == HAL_LTDC_STATE_RESET)
  {
    hltdc->Lock = HAL_UNLOCKED;
    /* 初始化 GPIO, NVIC 等 */
    HAL_LTDC_MspInit(hltdc);
  }

  /* 设置 LTDC 外设状态 */
  hltdc->State = HAL_LTDC_STATE_BUSY;
```

```
  /* 配置 HSE,VS，DE 和 PC 极性 */
  hltdc->Instance->GCR &= ~(LTDC_GCR_HSPOL | LTDC_GCR_VSPOL | LTDC_GCR_DEPOL | LTDC_GCR_PCPOL);
  hltdc->Instance->GCR |=  (uint32_t)(hltdc->Init.HSPolarity | hltdc->Init.VSPolarity | \
  hltdc->Init.DEPolarity | hltdc->Init.PCPolarity);

  /* 设置水平同步宽度和垂直同步宽度 */
  hltdc->Instance->SSCR &= ~(LTDC_SSCR_VSH | LTDC_SSCR_HSW);
  tmp = (hltdc->Init.HorizontalSync << 16);
  hltdc->Instance->SSCR |= (tmp | hltdc->Init.VerticalSync);

  /* 设置 HSYNC 水平同步宽度 + HBP 水平后沿之和
     设置 VSYNC 垂直同步宽度 + VBP 垂直后沿之和 */
  hltdc->Instance->BPCR &= ~(LTDC_BPCR_AVBP | LTDC_BPCR_AHBP);
  tmp = (hltdc->Init.AccumulatedHBP << 16);
  hltdc->Instance->BPCR |= (tmp | hltdc->Init.AccumulatedVBP);

  /* 设置 HSYNC 水平同步宽度 + HBP 水平后沿 + 有效宽度之和
     设置 VSYNC 垂直同步宽度 + VBP 垂直后沿 + 有效高度之和 */
  hltdc->Instance->AWCR &= ~(LTDC_AWCR_AAH | LTDC_AWCR_AAW);
  tmp = (hltdc->Init.AccumulatedActiveW << 16);
  hltdc->Instance->AWCR |= (tmp | hltdc->Init.AccumulatedActiveH);

  /* 设置总宽度 */
  hltdc->Instance->TWCR &= ~(LTDC_TWCR_TOTALH | LTDC_TWCR_TOTALW);
  tmp = (hltdc->Init.TotalWidth << 16);
  hltdc->Instance->TWCR |= (tmp | hltdc->Init.TotalHeigh);

  /* 设置背景层颜色 */
  tmp = ((uint32_t)(hltdc->Init.Backcolor.Green) << 8);
  tmp1 = ((uint32_t)(hltdc->Init.Backcolor.Red) << 16);
  hltdc->Instance->BCCR &= ~(LTDC_BCCR_BCBLUE | LTDC_BCCR_BCGREEN | LTDC_BCCR_BCRED);
  hltdc->Instance->BCCR |= (tmp1 | tmp | hltdc->Init.Backcolor.Blue);

  /* 使能传输错误中断和 FIFO 下溢中断 */
  __HAL_LTDC_ENABLE_IT(hltdc, LTDC_IT_TE | LTDC_IT_FU);

  /* 使能 LTDC */
  __HAL_LTDC_ENABLE(hltdc);

  /* 无错误 Initialize the error code */
  hltdc->ErrorCode = HAL_LTDC_ERROR_NONE;

  /* 设置 LTDC 状态 */
  hltdc->State = HAL_LTDC_STATE_READY;

  return HAL_OK;
}
```

函数描述：

此函数用于初始化 LTDC 的基本参数。

函数参数：

- 第 1 个参数是 LTDC_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数，结构体变量成员的详细介绍看本章 3.4 小节。

◆ 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 如果配置了 LTDC 的 NVIC，那么此函数会开启 LTDC 的传输错误中断和 FIFO 下溢中断，所以 LTDC 的中断服务程序别忘了写。
2. 函数 HAL_LTDC_MspInit 用于初始化 LTDC 的底层时钟、引脚等功能。需要用户自己在此函数里面实现具体的功能。由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
3. 如果形参 ltdc 的结构体成员 gState 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 LTDC_HandleTypeDef LtdcHandle。

   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_LTDC_STATE_RESET = 0x00U。

   解决办法有三

   方法 1：用户自己初始 LTDC 和涉及到的 GPIO 等。

   方法 2：定义 LTDC_HandleTypeDef LtdcHandle 为全局变量。

   方法 3：下面的方法

```
if(HAL_LTDC_DeInit(&LtdcHandle) != HAL_OK)
{
    Error_Handler();
}
if(HAL_LTDC_Init(&LtdcHandle) != HAL_OK)
{
    Error_Handler();
}
```

使用举例：

```
static LTDC_HandleTypeDef   hltdc_F;
uint16_t Width, Height, HSYNC_W, HBP, HFP, VSYNC_W, VBP, VFP;

/* 配置信号极性 */
hltdc_F.Init.HSPolarity = LTDC_HSPOLARITY_AL;  /* HSYNC 低电平有效 */
hltdc_F.Init.VSPolarity = LTDC_VSPOLARITY_AL;  /* VSYNC 低电平有效 */
hltdc_F.Init.DEPolarity = LTDC_DEPOLARITY_AL;  /* DE 低电平有效 */
hltdc_F.Init.PCPolarity = LTDC_PCPOLARITY_IPC;

/* 时序配置 */
hltdc_F.Init.HorizontalSync = (HSYNC_W - 1);
hltdc_F.Init.VerticalSync = (VSYNC_W - 1);
hltdc_F.Init.AccumulatedHBP = (HSYNC_W + HBP - 1);
hltdc_F.Init.AccumulatedVBP = (VSYNC_W + VBP - 1);
hltdc_F.Init.AccumulatedActiveH = (Height + VSYNC_W + VBP - 1);
hltdc_F.Init.AccumulatedActiveW = (Width + HSYNC_W + HBP - 1);
hltdc_F.Init.TotalHeigh = (Height + VSYNC_W + VBP + VFP - 1);
hltdc_F.Init.TotalWidth = (Width + HSYNC_W + HBP + HFP - 1);

/* 配置背景层颜色 */
hltdc_F.Init.Backcolor.Blue = 0;
```

```
hltdc_F.Init.Backcolor.Green = 0;
hltdc_F.Init.Backcolor.Red = 0;

hltdc_F.Instance = LTDC;

/* 配置 LTDC  */
if (HAL_LTDC_Init(&hltdc_F) != HAL_OK)
{
    /* 初始化错误 */
    Error_Handler(__FILE__, __LINE__);
}
```

50.4.2 函数 HAL_LTDC_ConfigLayer

函数原型：

```
HAL_StatusTypeDef HAL_LTDC_ConfigLayer(LTDC_HandleTypeDef *hltdc, LTDC_LayerCfgTypeDef *pLayerCfg, uint32_t
LayerIdx)
{
  /* 检测参数 */
  assert_param(IS_LTDC_LAYER(LayerIdx));
  assert_param(IS_LTDC_HCONFIGST(pLayerCfg->WindowX0));
  assert_param(IS_LTDC_HCONFIGSP(pLayerCfg->WindowX1));
  assert_param(IS_LTDC_VCONFIGST(pLayerCfg->WindowY0));
  assert_param(IS_LTDC_VCONFIGSP(pLayerCfg->WindowY1));
  assert_param(IS_LTDC_PIXEL_FORMAT(pLayerCfg->PixelFormat));
  assert_param(IS_LTDC_ALPHA(pLayerCfg->Alpha));
  assert_param(IS_LTDC_ALPHA(pLayerCfg->Alpha0));
  assert_param(IS_LTDC_BLENDING_FACTOR1(pLayerCfg->BlendingFactor1));
  assert_param(IS_LTDC_BLENDING_FACTOR2(pLayerCfg->BlendingFactor2));
  assert_param(IS_LTDC_CFBLL(pLayerCfg->ImageWidth));
  assert_param(IS_LTDC_CFBLNBR(pLayerCfg->ImageHeight));

  /* 上锁 */
  __HAL_LOCK(hltdc);

  /* 设置 LTDC 外设状态 */
  hltdc->State = HAL_LTDC_STATE_BUSY;

  /* 结构体之间直接赋值 */
  hltdc->LayerCfg[LayerIdx] = *pLayerCfg;

  /* 配置 LTDC 图层 */
  LTDC_SetConfig(hltdc, pLayerCfg, LayerIdx);

  /* 设置立即更新 */
  hltdc->Instance->SRCR = LTDC_SRCR_IMR;

  /* 设置 LTDC 就绪 */
  hltdc->State  = HAL_LTDC_STATE_READY;

  /* 解锁 */
  __HAL_UNLOCK(hltdc);

  return HAL_OK;
}
```

函数描述：

此函数主要用于配置 LTDC 的图层。

函数参数：

- 第 1 个参数是 LTDC_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数，结构体变量成员的详细介绍看本章 3.4 小节。
- 第 2 个参数是 LTDC_LayerCfgTypeDef 类型的结构体指针变量，用于图层配置，结构体变量成员的详细介绍看本章 3.3 小节。
- 第 3 个参数用于要配置的图层，LTDC_LAYER_1 表示配置图层 1，LTDC_LAYER_2 表示配置图层 2。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
static LTDC_HandleTypeDef   hltdc_F;
uint16_t Width, Height, HSYNC_W, HBP, HFP, VSYNC_W, VBP, VFP;
LTDC_LayerCfgTypeDef      pLayerCfg;

* 配置信号极性 */
hltdc_F.Init.HSPolarity = LTDC_HSPOLARITY_AL;  /* HSYNC 低电平有效 */
hltdc_F.Init.VSPolarity = LTDC_VSPOLARITY_AL;  /* VSYNC 低电平有效 */
hltdc_F.Init.DEPolarity = LTDC_DEPOLARITY_AL;  /* DE 低电平有效 */
hltdc_F.Init.PCPolarity = LTDC_PCPOLARITY_IPC;

/* 时序配置 */
hltdc_F.Init.HorizontalSync = (HSYNC_W - 1);
hltdc_F.Init.VerticalSync = (VSYNC_W - 1);
hltdc_F.Init.AccumulatedHBP = (HSYNC_W + HBP - 1);
hltdc_F.Init.AccumulatedVBP = (VSYNC_W + VBP - 1);
hltdc_F.Init.AccumulatedActiveH = (Height + VSYNC_W + VBP - 1);
hltdc_F.Init.AccumulatedActiveW = (Width + HSYNC_W + HBP - 1);
hltdc_F.Init.TotalHeigh = (Height + VSYNC_W + VBP + VFP - 1);
hltdc_F.Init.TotalWidth = (Width + HSYNC_W + HBP + HFP - 1);

/* 配置背景层颜色 */
hltdc_F.Init.Backcolor.Blue = 0;
hltdc_F.Init.Backcolor.Green = 0;
hltdc_F.Init.Backcolor.Red = 0;

hltdc_F.Instance = LTDC;

/* 开始配置图层 ------------------------------------------------------*/
/* 窗口显示区设置 */
pLayerCfg.WindowX0 = 0;
pLayerCfg.WindowX1 = Width;
pLayerCfg.WindowY0 = 0;
pLayerCfg.WindowY1 = Height;

/* 配置颜色格式 */
pLayerCfg.PixelFormat = LTDC_PIXEL_FORMAT_RGB565;

/* 显存地址 */
pLayerCfg.FBStartAdress = LCDH7_FRAME_BUFFER;
```

```
/* Alpha 常数 (255 表示完全不透明) */
pLayerCfg.Alpha = 255;

/* 无背景色 */
pLayerCfg.Alpha0 = 0;  /* 完全透明 */
pLayerCfg.Backcolor.Blue = 0;
pLayerCfg.Backcolor.Green = 0;
pLayerCfg.Backcolor.Red = 0;

/* 配置图层混合因数 */
pLayerCfg.BlendingFactor1 = LTDC_BLENDING_FACTOR1_CA;
pLayerCfg.BlendingFactor2 = LTDC_BLENDING_FACTOR2_CA;

/* 配置行列大小 */
pLayerCfg.ImageWidth  = Width;
pLayerCfg.ImageHeight = Height;

/* 配置 LTDC  */
if (HAL_LTDC_Init(&hltdc_F) != HAL_OK)
{
    /* 初始化错误 */
    Error_Handler(__FILE__, __LINE__);
}

/* 配置图层 1 */
if (HAL_LTDC_ConfigLayer(&hltdc_F, &pLayerCfg, LTDC_LAYER_1) != HAL_OK)
{
    /* 初始化错误 */
    Error_Handler(__FILE__, __LINE__);
}
```

50.4.3 函数 HAL_LTDC_SetAlpha

函数原型：

```
HAL_StatusTypeDef HAL_LTDC_SetAlpha(LTDC_HandleTypeDef *hltdc, uint32_t Alpha, uint32_t LayerIdx)
{
  LTDC_LayerCfgTypeDef *pLayerCfg;

  /* 检查参数 */
  assert_param(IS_LTDC_ALPHA(Alpha));
  assert_param(IS_LTDC_LAYER(LayerIdx));

  /* 上锁 */
  __HAL_LOCK(hltdc);

  /* 设置 LTDC 状态 */
  hltdc->State = HAL_LTDC_STATE_BUSY;

  /* 获取图层配置 */
  pLayerCfg = &hltdc->LayerCfg[LayerIdx];

  /* 配置图层 Alpha 值 */
  pLayerCfg->Alpha = Alpha;
```

```
  /* 重新配置 LTDC */
  LTDC_SetConfig(hltdc, pLayerCfg, LayerIdx);

  /* 设置立即更新 */
  hltdc->Instance->SRCR = LTDC_SRCR_IMR;

  /* 设置 LTDC 状态就绪 */
  hltdc->State = HAL_LTDC_STATE_READY;

  /* 解锁 */
  __HAL_UNLOCK(hltdc);

  return HAL_OK;
}
```

函数描述：

此函数用于设置图层的常数 Alpha。

函数参数：

- 第 1 个参数是 LTDC_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数，结构体变量成员的详细介绍看本章 3.4 小节。
- 第 2 个参数是 Alpha 值设置，0x00 表示完全透明，0xFF 表示完全不透明。
- 第 3 个参数用于要配置的图层，LTDC_LAYER_1 表示配置图层 1，LTDC_LAYER_2 表示配置图层 2。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

此函数的使用相对比较简单，直接调用即可。

50.4.4函数 HAL_LTDC_Reload

函数原型：

```
HAL_StatusTypeDef  HAL_LTDC_Reload(LTDC_HandleTypeDef *hltdc, uint32_t ReloadType)
{
  /* 检测参数 */
  assert_param(IS_LTDC_RELOAD(ReloadType));

  /* 上锁 */
  __HAL_LOCK(hltdc);

  /* 设置 LTDC 状态 Change LTDC peripheral state */
  hltdc->State = HAL_LTDC_STATE_BUSY;

  /* 使能 LTDC 重装中断 */
  __HAL_LTDC_ENABLE_IT(hltdc, LTDC_IT_RR);

  /* 设置立即更新或者下一个垂直消隐期更新 Apply Reload type */
  hltdc->Instance->SRCR = ReloadType;

  /* 设置 LTDC 就绪 */
  hltdc->State = HAL_LTDC_STATE_READY;
```

```
  /* 解锁 */
  __HAL_UNLOCK(hltdc);

  return HAL_OK;
}
```

函数描述：

此函数主要用于配合其它以_NoReload 结尾的函数，可以设置立即更新配置或者下一个垂直消隐期更新。

函数参数：

- 第 1 个参数是 LTDC_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数，结构体变量成员的详细介绍看本章 3.4 小节。
- 第 2 个参数设置为 LTDC_RELOAD_IMMEDIATE 表示立即更新。
  设置为 LTDC_RELOAD_VERTICAL_BLANKING 表示下一个垂直消隐期更新。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

此函数的使用相对比较简单，直接调用即可。

50.4.5 函数 HAL_LTDC_SetPixelFormat

函数原型：

```
HAL_StatusTypeDef HAL_LTDC_SetPixelFormat(LTDC_HandleTypeDef *hltdc, uint32_t Pixelformat, uint32_t LayerIdx)
{
  LTDC_LayerCfgTypeDef *pLayerCfg;

  /* 检测参数 */
  assert_param(IS_LTDC_PIXEL_FORMAT(Pixelformat));
  assert_param(IS_LTDC_LAYER(LayerIdx));

  /* 上锁 */
  __HAL_LOCK(hltdc);

  /* 设置 LTDC 状态 */
  hltdc->State = HAL_LTDC_STATE_BUSY;

  /* 获取图层配置 */
  pLayerCfg = &hltdc->LayerCfg[LayerIdx];

  /* 重新配置图层颜色格式 */
  pLayerCfg->PixelFormat = Pixelformat;

  /* 配置 LTDC */
  LTDC_SetConfig(hltdc, pLayerCfg, LayerIdx);

  /* 设置立即更新 */
  hltdc->Instance->SRCR = LTDC_SRCR_IMR;

  /* 设置 LTDC 就绪 */
```

```
  hltdc->State = HAL_LTDC_STATE_READY;

  /* 解锁 */
  __HAL_UNLOCK(hltdc);

  return HAL_OK;
}
```

函数描述：

此函数用于设置图层的颜色格式。

函数参数：

- 第 1 个参数是 LTDC_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数，结构体变量成员的详细介绍看本章 3.4 小节。
- 第 2 个参数是 STM32H7 支持的颜色格式，具体有如下 8 种：

```
#define LTDC_PIXEL_FORMAT_ARGB8888          (0x00000000U)      /*!< ARGB8888 LTDC pixel format */
#define LTDC_PIXEL_FORMAT_RGB888            (0x00000001U)      /*!< RGB888 LTDC pixel format   */
#define LTDC_PIXEL_FORMAT_RGB565            (0x00000002U)      /*!< RGB565 LTDC pixel format   */
#define LTDC_PIXEL_FORMAT_ARGB1555          (0x00000003U)      /*!< ARGB1555 LTDC pixel format */
#define LTDC_PIXEL_FORMAT_ARGB4444          (0x00000004U)      /*!< ARGB4444 LTDC pixel format */
#define LTDC_PIXEL_FORMAT_L8                (0x00000005U)      /*!< L8 LTDC pixel format       */
#define LTDC_PIXEL_FORMAT_AL44              (0x00000006U)      /*!< AL44 LTDC pixel format     */
#define LTDC_PIXEL_FORMAT_AL88              (0x00000007U)      /*!< AL88 LTDC pixel format     */
```

- 第 3 个参数用于要配置的图层，LTDC_LAYER_1 表示配置图层 1，LTDC_LAYER_2 表示配置图层 2。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

此函数的使用相对比较简单，直接调用即可。

50.4.6 函数 HAL_LTDC_SetWindowPosition

函数原型：

```
HAL_StatusTypeDef HAL_LTDC_SetWindowPosition(LTDC_HandleTypeDef *hltdc, uint32_t X0, uint32_t Y0, uint32_t
                                             LayerIdx)
{
  LTDC_LayerCfgTypeDef *pLayerCfg;

  /* 检测参数 */
  assert_param(IS_LTDC_LAYER(LayerIdx));
  assert_param(IS_LTDC_CFBLL(X0));
  assert_param(IS_LTDC_CFBLNBR(Y0));

  /* 上锁 */
  __HAL_LOCK(hltdc);

  /* 设置 LTDC 状态 */
  hltdc->State = HAL_LTDC_STATE_BUSY;

  /* 获取图层配置 */
  pLayerCfg = &hltdc->LayerCfg[LayerIdx];
```

```
  /* 更新水平起始和结束位置 */
  pLayerCfg->WindowX0 = X0;
  pLayerCfg->WindowX1 = X0 + pLayerCfg->ImageWidth;

  /* 更新垂直起始和结束位置 */
  pLayerCfg->WindowY0 = Y0;
  pLayerCfg->WindowY1 = Y0 + pLayerCfg->ImageHeight;

  /* 设置 LTDC 参数 */
  LTDC_SetConfig(hltdc, pLayerCfg, LayerIdx);

  /* 设置立即更新 */
  hltdc->Instance->SRCR = LTDC_SRCR_IMR;

  /* 设置 LTDC 就绪 */
  hltdc->State = HAL_LTDC_STATE_READY;

  /* 解锁 */
  __HAL_UNLOCK(hltdc);

  return HAL_OK;
}
```

函数描述：

此函数用于设置显示区在图层中的起始位置。

函数参数：

- 第 1 个参数是 LTDC_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数，结构体变量成员的详细介绍看本章 3.4 小节。
- 第 2 个参数是设置显示区在图层中的 X 轴起始位置。
- 第 3 个参数是设置显示区在图层中的 Y 轴起始位置。
- 第 4 个参数用于要配置的图层，LTDC_LAYER_1 表示配置图层 1，LTDC_LAYER_2 表示配置图层 2。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

此函数的使用相对比较简单，直接调用即可，注意设置的显示区最好不要超出图层范围。

50.4.7 函数 HAL_LTDC_SetWindowSize_NoReload

函数原型：

```
HAL_StatusTypeDef HAL_LTDC_SetWindowSize_NoReload(LTDC_HandleTypeDef *hltdc, uint32_t XSize, uint32_t YSize,
                                  uint32_t LayerIdx)
{
  LTDC_LayerCfgTypeDef *pLayerCfg;

  /* 检测参数 */
  assert_param(IS_LTDC_LAYER(LayerIdx));
  assert_param(IS_LTDC_CFBLL(XSize));
```

```
  assert_param(IS_LTDC_CFBLNBR(YSize));

  /* 上锁 */
  __HAL_LOCK(hltdc);

  /* 设置 LTDC 状态 */
  hltdc->State = HAL_LTDC_STATE_BUSY;

  /* 获取图层配置 */
  pLayerCfg = &hltdc->LayerCfg[LayerIdx];

  /* 更新水平结束位置 */
  pLayerCfg->WindowX1 = XSize + pLayerCfg->WindowX0;

  /* 更新垂直解锁位置 */
  pLayerCfg->WindowY1 = YSize + pLayerCfg->WindowY0;

  /* 重新配置颜色帧缓冲的行长 */
  pLayerCfg->ImageWidth = XSize;

  /* 重新配置颜色帧缓冲的行数 */
  pLayerCfg->ImageHeight = YSize;

  /* 设置 LTDC 参数 */
  LTDC_SetConfig(hltdc, pLayerCfg, LayerIdx);

  /* 此函数没有配置立即更新配置，绘制下一个垂直消隐期更新  */

  /* 设置 LTDC 就绪 */
  hltdc->State = HAL_LTDC_STATE_READY;

  /* 解锁 */
  __HAL_UNLOCK(hltdc);

  return HAL_OK;
}
```

函数描述：

此函数用于设置图层显示区的大小，但不会立即更新，会在下一个垂直消隐期更新。

函数参数：

- 第 1 个参数是 LTDC_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数，结构体变量成员的详细介绍看本章 3.4 小节。
- 第 2 个参数设置图层显示区的行长，单位像素。
- 第 3 个参数是设置图层显示区的行数，单位像素。
- 第 4 个参数用于要配置的图层，LTDC_LAYER_1 表示配置图层 1，LTDC_LAYER_2 表示配置图层 2。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

此函数的使用相对比较简单，直接调用即可，注意设置的显示区最好不要超出图层范围。

50.5 总结

本章节涉及到的知识点比较多，而且比较重要，特别是第 2 小节中的几个知识点尤其重要，望初学者熟练掌握。

第51章 STM32H7 的 LTDC 应用之 LCD 汉字显示和 2D 图形显示

本章教程为大家讲解 LTDC 应用中最基本的汉字显示和 2D 图形显示功能实现。

51.1 初学者重要提示
51.2 LCD 相关的基础支持
51.3 LCD 硬件设计
51.4 LCD 驱动设计
51.5 LCD 板级支持包（bsp_ltdc_h7.c 和 bsp_tft_lcd.c）
51.6 LCD 的驱动移植和使用
51.7 实验例程设计框架
51.8 实验例程说明（MDK）
51.9 实验例程说明（IAR）
51.10 总结

51.1 初学者重要提示

- 学习本章节前，务必优先学习第 50 章，需要对 LTDC 的基础知识和 HAL 库的几个常用 API 有个认识。
- 本章的第 4 小节 LCD 驱动设计非常重要。
- 如果自己观察的话，LCD 上电会有一个瞬间高亮的问题，在此贴进行了描述：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=82619 。
  这个解决方案已经应用到本章配套的例子上。
- 本章节用到的汉字方案会在下章专门为大家讲解，下面是小字库的制作方法：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=202 。
- 测评 STM32H7 的 LTDC+DMA2D 性能，100Hz 以上无压力，刷 800*480 图片和色块仅需 2.6ms 一张：http://www.armbbs.cn/forum.php?mod=viewthread&tid=91489 。
- 调试状态或者刚下载 LCD 的程序到 H7 里面，屏幕会抖动，这个是正常现象，之前 F429 就有这个问题，详情看此贴：http://www.armbbs.cn/forum.php?mod=viewthread&tid=16892 。

51.2 LCD 相关的基础知识

51.2.1 显示屏相关知识

显示屏的结构有必要给大家普及下，这里我们通过如下三种类型的显示屏进行说明，基本已经涵盖我们常用的方式了。

- RA8875 + RGB 接口裸屏

  首先 RA8875 是一个显示屏控制器，自带显存，它的作用就是让不支持 RGB 接口的 MCU 也可以使用 RGB 接口的大屏。这起到了一个桥接的作用，可以将 RGB 接口屏转换成 8080 总线接口、SPI 接口或者 I2C 接口方式。这种情况下，甚至低速的 51 单片机都可以外接大屏了。另外像 SSD1963 也是同样的作用。

- ili9488 类显示屏

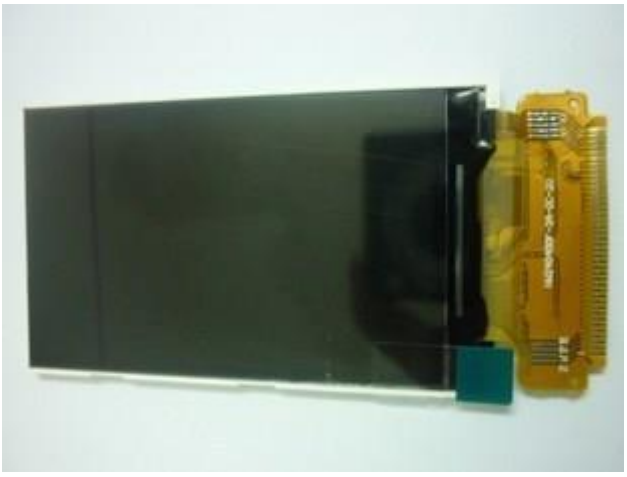
  这种类型是把显示控制器和显示屏都集成好了，支持 8080 总线接口，有些还支持 SPI 或者 I2C 接口，而且显存也都集成了，不过主要是驱动一些小屏。像 ili9341，ili9326，SPFD5420 等也是一样的。此外还要注意，部分这种类型显示屏也是支持 RGB 接口的，像 ST 官方的 STM32F429 探索板外接的 ili9431 就是用的 RGB 接口。

- STM32H7 + SDRAM + RGB 接口裸屏

  这个是我们本章节要讲解的，STM32H7 是自带 LCD 控制器的，再配合 SDRAM 作为显示屏的显存，整体作用跟 RA8875 是一样的，可以直接外接 RGB 接口的屏了。

有了这些认识后，对于裸屏还有些知识点需要了解。首先，裸屏本身不是什么控制芯片都没有，其构成也是比较复杂的，有兴趣了解的话，可以搜索关键字“TFT 结构”进行学习。其次，TFT 裸屏中主要的两个 IC 是 Gate Driver IC 和 Source Driver IC，这两个 IC 的引脚都超级多，基本都是几百个引脚。最后，不管使用的哪种裸屏，一般都有规格书，会给出时序参数，这个在配置 STM32H7 的 LTDC 时要用到，如果规格书没有直接给出时序参数，则会给出使用的 Driver IC 型号，用户可以搜索此 Driver IC 的手册，在手册中会给出。

为了让大家有个感性认识，我们来看一看 TFT 裸屏的实际效果，下面是 SPDF5420 显示屏，400*240 分辨率：



下面是 TFT 裸屏，480*272 分辨率：

下面是 TFT 裸屏，800*480 分辨率：

51.2.2 电阻触摸和电容触摸相关知识

有了 TFT 裸屏后还要配套电阻触摸板或者电容触摸板才可以获取触摸信息。触摸板是贴到 TFT 屏上面的，然后再通过电阻触摸芯片就可以获取电阻触摸板的信息，通过电容触摸芯片采集电容触摸板的信息。教程配套开发板的显示屏使用了三种触摸 IC，电阻触摸 IC 是 STMPE811，电容触摸 IC 是 GT811 和 FT5X06。其中，电阻触摸和电容触摸两者的区别是初学者务必要知道的：

- 电阻触摸芯片 STMPE811 其实就是 ADC，返回的是 ADC 数值，而电容触摸芯片 GT811，GT911 和 FT5X06 返回的是显示屏实际的坐标值。
- 使用电阻触摸芯片 STMPE811 需要做触摸校准，而使用电容触摸芯片 GT811，GT911 和 FT5X06 是自动校准的，无需手动校准。

下面是四线电阻触摸板的效果：

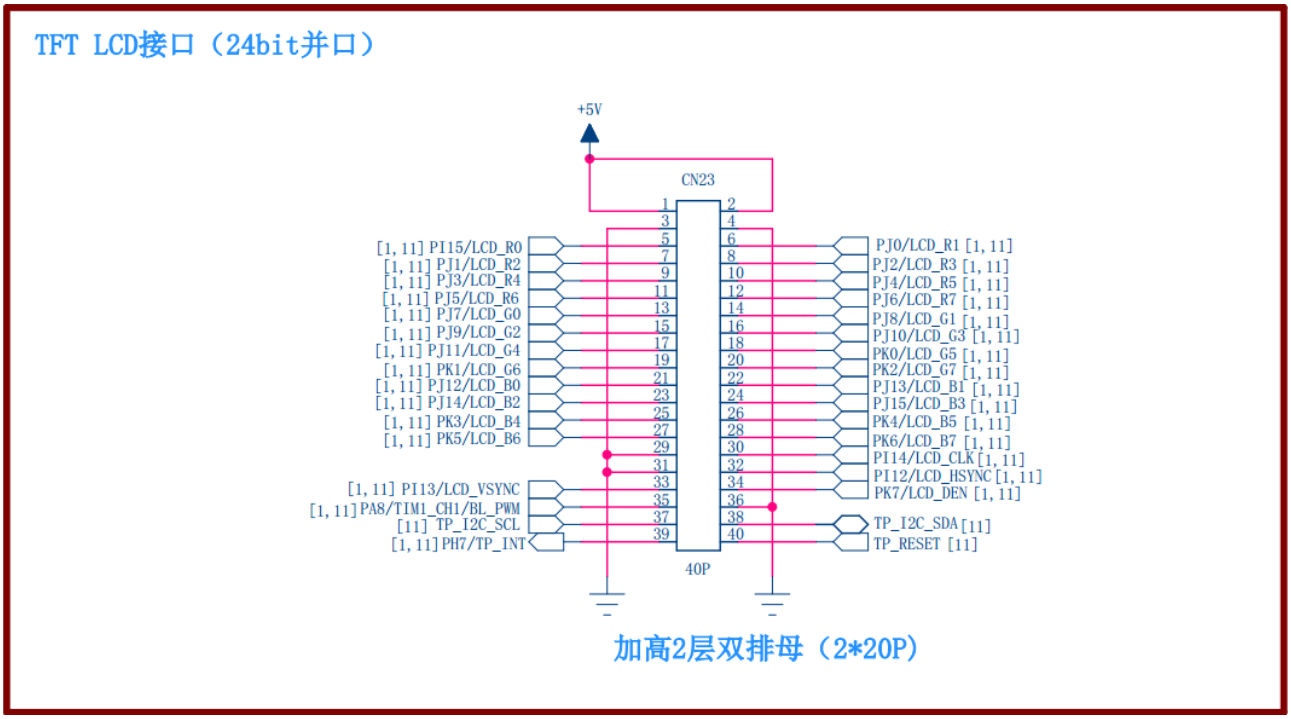
下面是电容触摸板的效果：

了解了这些知识，基本已经够我们本章节使用了，更多电阻触摸和电容触摸的相关知识可以看这个文档，讲解比较全面：http://bbs.armfly.com/read.php?tid=14898 。

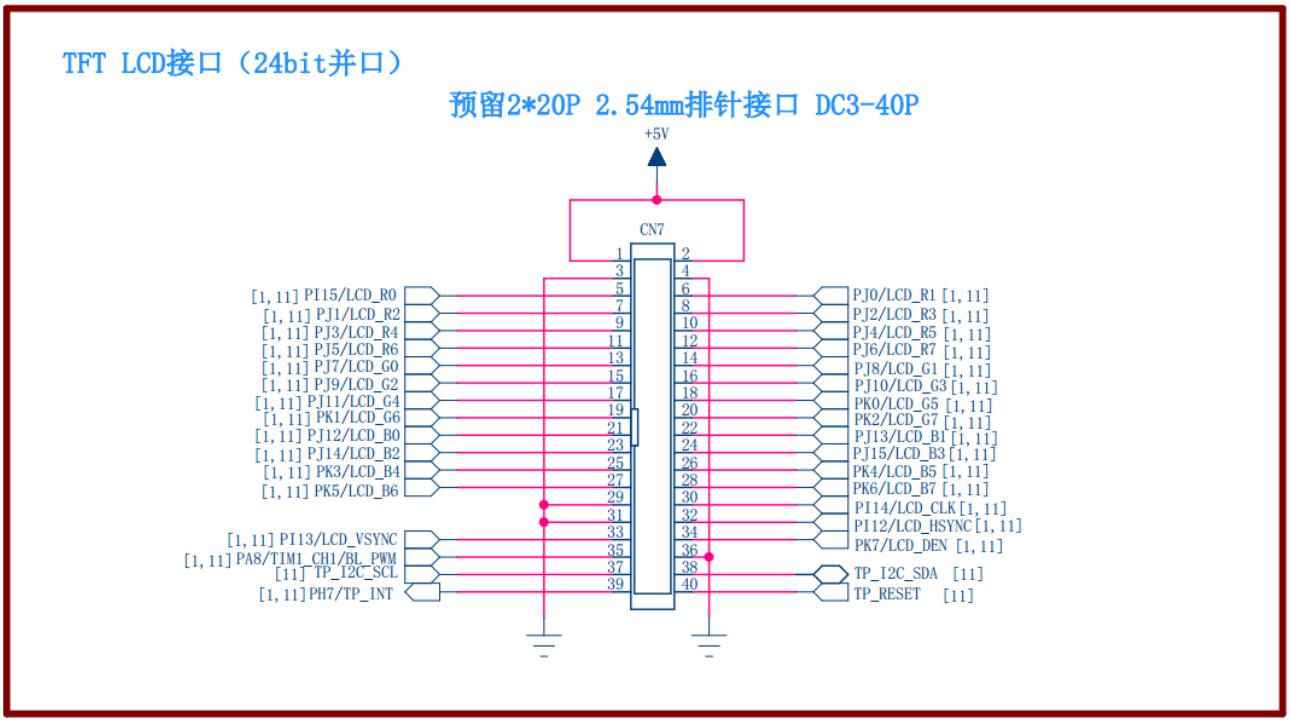
51.3 LCD 硬件设计

下面是 RGB888 硬件接口的原理图，STM32-V7 开发板制作了三个硬件接口。

◆ 加高 2 层的双排母接口

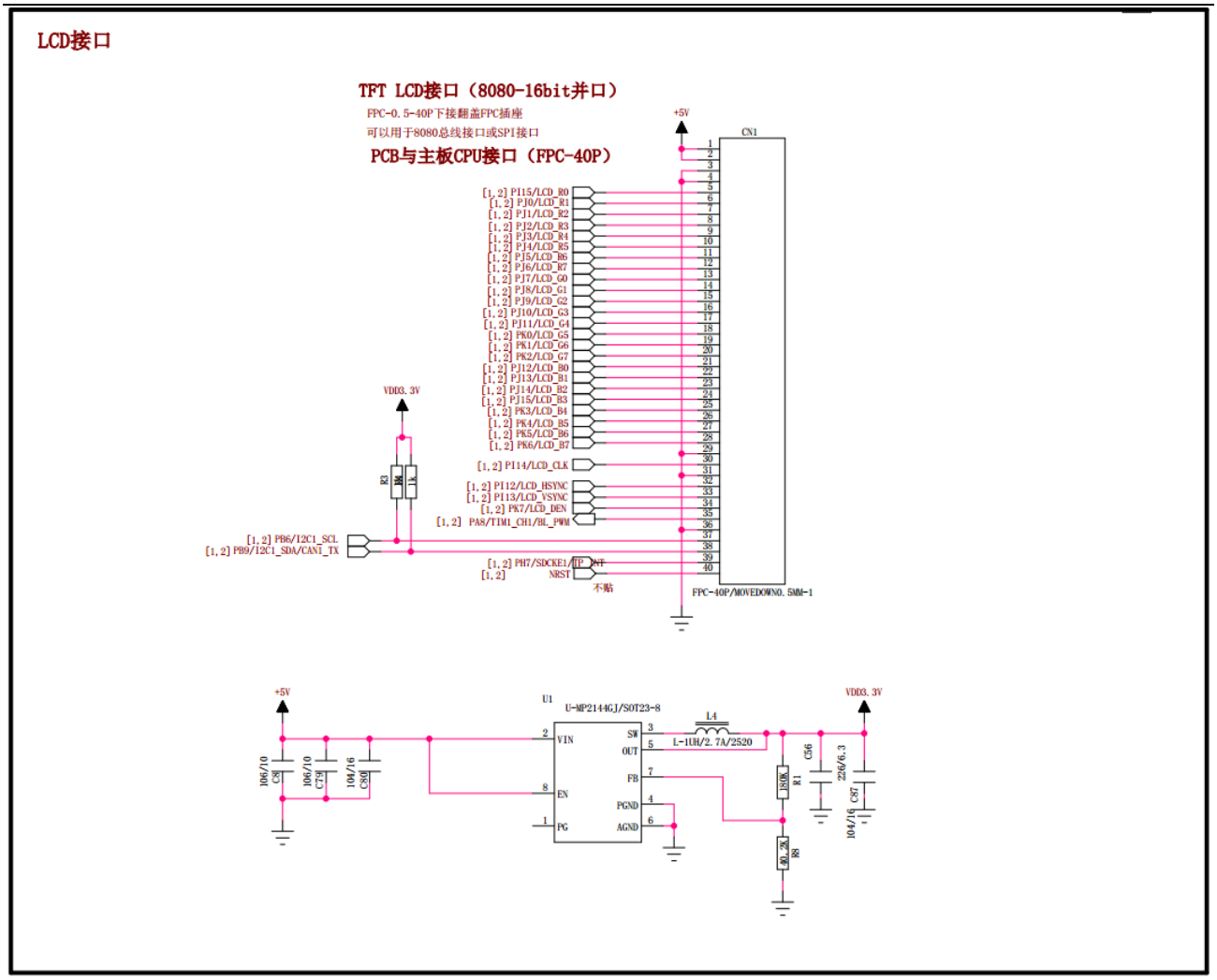


◆ 2.54mm 的排针接口

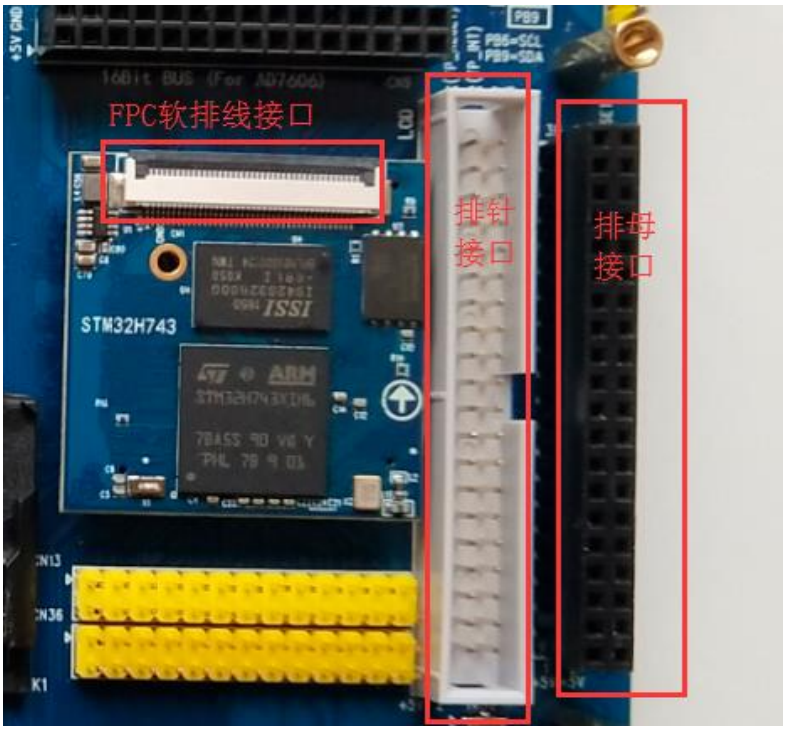


◆ FPC 软排线接口方式

LCD接口



了解了原理图后，再来看下实际的接口效果：



通过上面的原理图，我们要了解以下几个问题：

- V7 开发板采用的是 RGB888 硬件接口，也许大家会问 ARGB8888 这种颜色格式怎么用于这种接口？Alpha 通道是软件编程的时候用的，用于设置透明度，透明度会反应到 RGB 颜色值上。
- STM32H7 支持的 8 种颜色格式都可以在 RGB888 硬件接口上实现。
- 如果大家用的是 16 位色的 RGB565 颜色格式，那么仅需用到 LCD_R[7:3]、LCD_G[7:2] 和 LCD_B[7:3] 引脚即可，没有用到的引脚可以继续用作其它功能。

51.4 LCD 驱动设计

下面将程序设计中的相关问题逐一为大家做个说明。

51.4.1 第 1 步，LTDC 显存使用 SDRAM

设计 LTDC 驱动前，要先保证显存可以正常使用，V7 开发板用的外部 SDRAM 作为显存。所以一定要保证 SDRAM 大批量读写数据时是正常的，SDRAM 的测试可以自己专门做一个工程测试下。对于 SDRAM 的驱动实现，可以学习本教程第 49 章。不管你使用的是镁光的，海力士的，三星的，ISSI 的或者华邦的，实现方法基本都是一样的。

V7 开发板使用 ISSI 的 32 位带宽、32MB 的 SDRAM，如果想最大限度的发挥 STM32H7 驱动 SDRAM 的性能，强烈建议使用 32 位带宽的 SDRAM，或者两个 16 位 SDRAM 组成 32 位带宽的 SDRAM 也是可以的。那 SDRAM 主要起到什么作用呢？作用有二：

- 用作显示屏的显存
  STM32H7 的 LTDC 外接 RGB 接口屏是没有显存的，所以需要 SDRAM 用作显存。如果用户选择 STM32H7 LTDC 的颜色格式是 32 位色 ARGB8888，那么所需要显存大小（单位字节）是：显示屏宽 * 显示屏高 * （32/8），其中 32/8 是表示这种颜色格式的一个像素点需要 4 个字节来表示。又比如配置颜色格式是 16 位色的 RGB565，那么需要的显存大小是：显示屏宽 * 显示屏高 *（16/8），其中 16/8 是表示这种颜色格式的一个像素点需要 2 个字节来表示。其它的颜色格式，依此类推。
- 用作 GUI 动态内存
  如果想要实现炫酷效果，GUI 是极其消耗动态内存的，所以用户可以将 SDRAM 除了用于显存以外的所有内存全部用作 GUI 动态内存。

如果 SDRAM 的驱动测试已经没有问题了，就可以将其添加到工程里面了，V7 使用的 SDRAM 驱动文件是 bsp_fmc_sdram.c。图层 1 占用 2MB，图层 2 占用 2MB，最后 28MB 可做其它使用。也许会有初学者会问，每个图层分配 2MB 是不是有些多了？实际上不多的，因为我们要让不同的颜色格式都通用，这里分配 2MB 的话，教程实例使用很方便。大家实际项目中的使用可以配置成实际大小。具体的配置如下，详情见 bsp_fmc_sdram.h 文件：

```
#define EXT_SDRAM_ADDR      ((uint32_t)0xC0000000)
#define EXT_SDRAM_SIZE      (32 * 1024 * 1024)

/* LCD 显存,第 1 页, 分配 2M 字节 */
#define SDRAM_LCD_BUF1      EXT_SDRAM_ADDR

/* LCD 显存,第 2 页, 分配 2M 字节 */
#define SDRAM_LCD_BUF2      (EXT_SDRAM_ADDR + SDRAM_LCD_SIZE)

#define SDRAM_LCD_SIZE      (2 * 1024 * 1024)       /* 每层 2M */
#define SDRAM_LCD_LAYER     2                       /* 2 层 */

/* 剩下的 28M 字节, 提供给应用程序使用 */
#define SDRAM_APP_BUF       (EXT_SDRAM_ADDR + SDRAM_LCD_SIZE * SDRAM_LCD_LAYER)
#define SDRAM_APP_SIZE      (EXT_SDRAM_SIZE - SDRAM_LCD_SIZE * SDRAM_LCD_LAYER)
```

由于用户要刷新数据到SDRAM,而且LTDC也要从SDRAM读取数据,这就属于多总线访问SDRAM,此时就要注意 Cache 配置。为了使用方便起见，直接将 SDRAM 配置为 WT 模式，这样用户刷新的数据就可以立即写入到 SDRAM，从而不影响 LTDC 刷新。

51.4.2第 2 步，LTDC 涉及到的引脚配置

本章第 3 小节用到了哪些引脚，这些引脚全部要做初始化，初始化时别忘了初始化引脚对应的时钟：

```
static void LCDH7_ConfigLTDC(void)
{
    /* 配置 LCD 相关的 GPIO */
    {
        /* GPIOs Configuration */
        /*
        +------------------------+-----------------------+----------------------------+
        +                       LCD pins assignment                                   +
        +------------------------+-----------------------+----------------------------+
        |  LCDH7_TFT R0 <-> PI.15  |  LCDH7_TFT G0 <-> PJ.07 |  LCDH7_TFT B0 <-> PJ.12     |
        |  LCDH7_TFT R1 <-> PJ.00  |  LCDH7_TFT G1 <-> PJ.08 |  LCDH7_TFT B1 <-> PJ.13     |
        |  LCDH7_TFT R2 <-> PJ.01  |  LCDH7_TFT G2 <-> PJ.09 |  LCDH7_TFT B2 <-> PJ.14     |
        |  LCDH7_TFT R3 <-> PJ.02  |  LCDH7_TFT G3 <-> PJ.10 |  LCDH7_TFT B3 <-> PJ.15     |
        |  LCDH7_TFT R4 <-> PJ.03  |  LCDH7_TFT G4 <-> PJ.11 |  LCDH7_TFT B4 <-> PK.03     |
        |  LCDH7_TFT R5 <-> PJ.04  |  LCDH7_TFT G5 <-> PK.00 |  LCDH7_TFT B5 <-> PK.04     |
        |  LCDH7_TFT R6 <-> PJ.05  |  LCDH7_TFT G6 <-> PK.01 |  LCDH7_TFT B6 <-> PK.05     |
        |  LCDH7_TFT R7 <-> PJ.06  |  LCDH7_TFT G7 <-> PK.02 |  LCDH7_TFT B7 <-> PK.06     |
        -------------------------------------------------------------------------------
        |  LCDH7_TFT HSYNC <-> PI.12  | LCDTFT VSYNC <->  PI.13 |
        |  LCDH7_TFT CLK   <-> PI.14  | LCDH7_TFT DE   <->  PK.07 |
        -----------------------------------------------------
        */
        GPIO_InitTypeDef GPIO_Init_Structure;

        /*##-1- Enable peripherals and GPIO Clocks #################################*/
        /* 使能 LTDC 时钟 */
        __HAL_RCC_LTDC_CLK_ENABLE();
        /* 使能 GPIO 时钟 */
        __HAL_RCC_GPIOI_CLK_ENABLE();
        __HAL_RCC_GPIOJ_CLK_ENABLE();
        __HAL_RCC_GPIOK_CLK_ENABLE();
```

```
        /* GPIOI 配置 */
        GPIO_Init_Structure.Pin       = GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14 | GPIO_PIN_15;
        GPIO_Init_Structure.Mode      = GPIO_MODE_AF_PP;
        GPIO_Init_Structure.Pull      = GPIO_NOPULL;
        GPIO_Init_Structure.Speed     = GPIO_SPEED_FREQ_HIGH;
        GPIO_Init_Structure.Alternate = GPIO_AF14_LTDC;
        HAL_GPIO_Init(GPIOI, &GPIO_Init_Structure);

        /* GPIOJ 配置 */
        GPIO_Init_Structure.Pin       = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3 | \
                                        GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7 | \
                                        GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 | GPIO_PIN_11 | \
                                        GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14 | GPIO_PIN_15;
        GPIO_Init_Structure.Mode      = GPIO_MODE_AF_PP;
        GPIO_Init_Structure.Pull      = GPIO_NOPULL;
        GPIO_Init_Structure.Speed     = GPIO_SPEED_FREQ_HIGH;
        GPIO_Init_Structure.Alternate = GPIO_AF14_LTDC;
        HAL_GPIO_Init(GPIOJ, &GPIO_Init_Structure);

        /* GPIOK 配置 */
        GPIO_Init_Structure.Pin       = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3 | \
                                        GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7;
        GPIO_Init_Structure.Mode      = GPIO_MODE_AF_PP;
        GPIO_Init_Structure.Pull      = GPIO_NOPULL;
        GPIO_Init_Structure.Speed     = GPIO_SPEED_FREQ_HIGH;
        GPIO_Init_Structure.Alternate = GPIO_AF14_LTDC;
        HAL_GPIO_Init(GPIOK, &GPIO_Init_Structure);
    }
  /* 其它省略未写 */

}
```

51.4.3第 3 步，LTDC 时钟和时序配置

LTDC 时序配置主要分三步就可以完成：

1. 行同步，场同步和 DE 的极性配置。
2. CLK 时钟配置。
3. 时序参数配置。

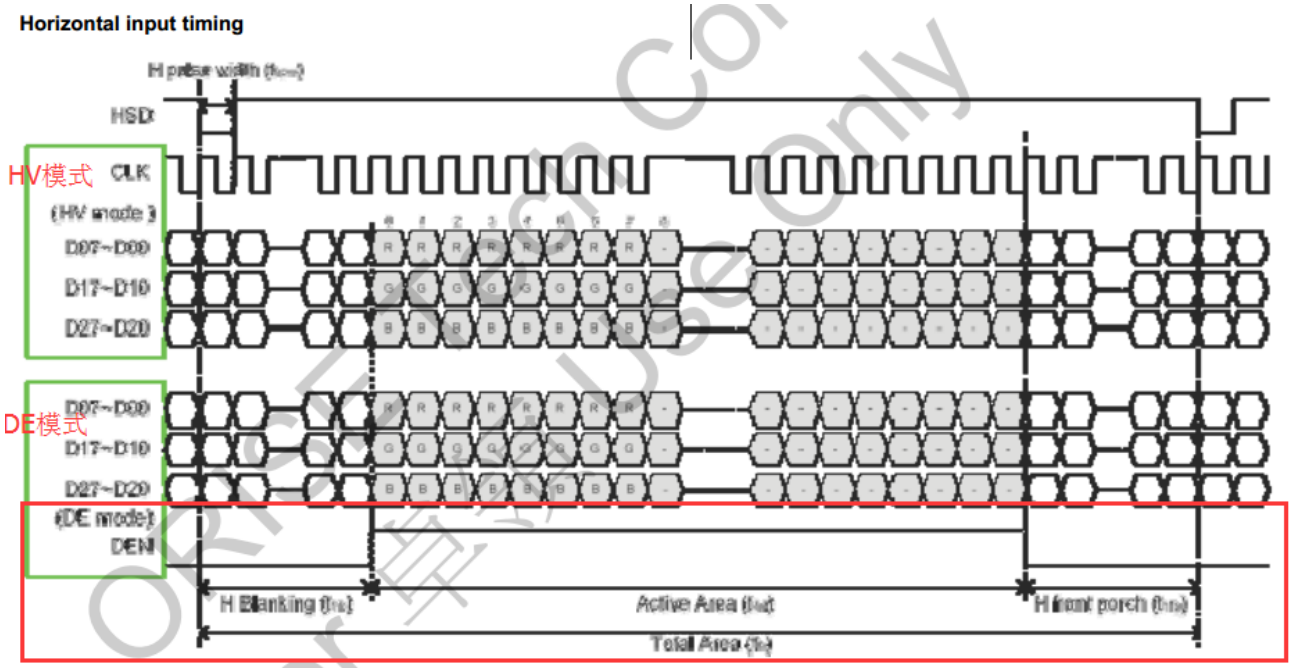
下面将这三点分别做个说明：

◆ **行同步，场同步和 DE 的极性配置**

这里以 V7 开发板 7 寸 RGB 屏使用的 source driver ic OTA7001 为例进行说明（手册下载地址：http://www.armbbs.cn/forum.php?mod=viewthread&tid=18528）。

这几项配置要看 OTA7001 手册上面的时序图，对于 DE 模式，行同步和场同步的极性配置为高或者为低均可。因为我们这里使用的就是 DE 模式，所以主要配置 DE 的极性。这里要特别注意一个小问题，看时序图是 DE 高电平时数据有效，但是配置的时候要设置为低电平才可以。

下面的是 V7 开发板配套的 7 寸裸屏使用的 source driver ic OTA7001 的时序图：

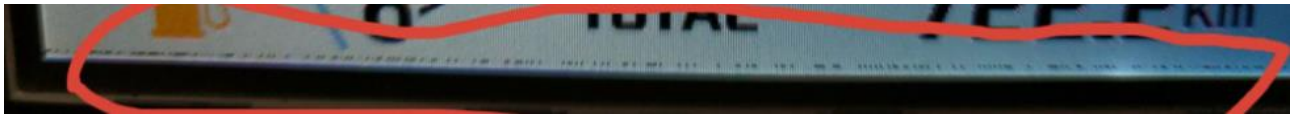


按照我们常规的理解DE是高电平有效

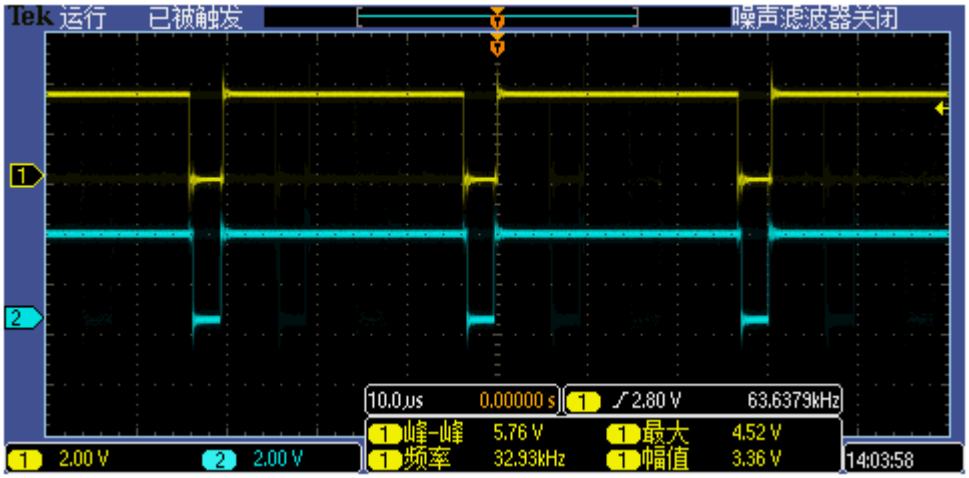
实际配置 STM32H7 的工程时，将 DE 配置为低有效才是上面截图的效果，这个问题的确是有些奇葩了。大家使用的时候也特别注意。

```
/* 配置信号极性 */
hltdc_F.Init.HSPolarity = LTDC_HSPOLARITY_AL;  /* HSYNC 低电平有效 */
hltdc_F.Init.VSPolarity = LTDC_VSPOLARITY_AL;  /* VSYNC 低电平有效 */
hltdc_F.Init.DEPolarity = LTDC_DEPOLARITY_AL;  /* DE 低电平有效 */
hltdc_F.Init.PCPolarity = LTDC_PCPOLARITY_IPC; /* Pixel Clock 或者 Dot Clock 极性 */
```

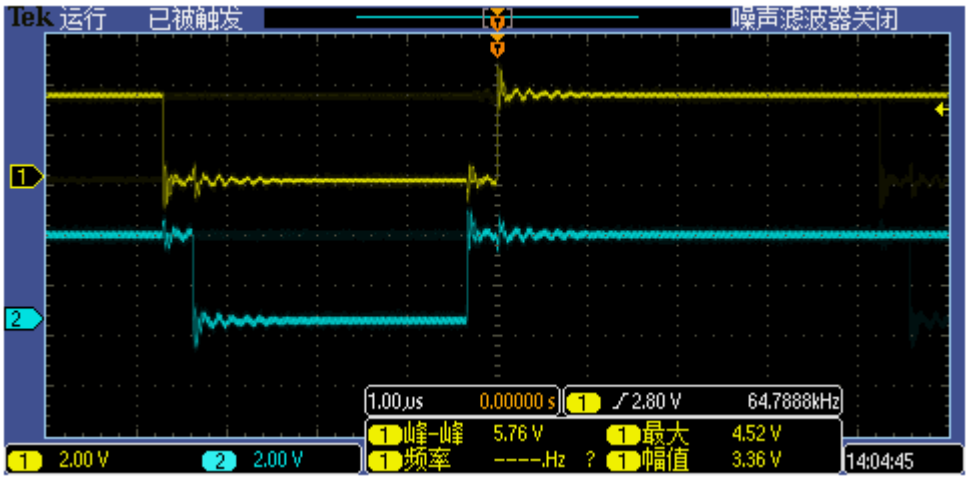
ps：注意 LTDC_PCPOLARITY_IPC 和 LTDC_PCPOLARITY_IIPC 两种极性，选择不当会有一些问题。比如下面这个现象，底边会有黑的。



下面是用示波器实际测量的波形效果，黄色的波形是 DE 信号，另一个是行同步信号 Hsync：



将波形放缩后：



◆ **LTDC 的像素时钟输出配置**

在 OTA7001 手册上面给出了支持的时钟范围：

For 800x480 panel

Horizontal input timing

Parameter		Symbol	Value			Unit
			Min.	Typ.	Max.	
Horizontal display area		thd		800		DCLK
DCLK frequency		fclk	-	30	50	MHz
1 Horizontal Line		th		928		DCLK
HSD pulse width	Min.	thpw		1		
	Typ.			48		
	Max.			-		
HSD Back Porch (Blanking)		thb	-	88	-	
HSD Front Porch		thfp	-	40	-	

由于USB和LTDC都是用的PLL3产生时钟，为了方便起见，直接将产生LTDC时钟的PLL3R设置为24MHz（PLL3Q 输出的 48MHz 时钟供 USB 使用）。

```
首先，输入时钟 PLL3_VCO Input = HSE_VALUE/PLL3M = 25MHz/5 = 5MHz
输出时钟 PLL3_VCO Output = PLL3_VCO Input * PLL3N = 5MHz * 48 = 240MHz
PLLLCDCLK = PLL3_VCO Output/PLL3R = 240 / 10 = 24MHz
最后，LTDC 时钟 LTDC clock frequency = PLLLCDCLK = 24MHz
```

这里再额外补充点知识，LCD_CLK=24MHz 时

刷新率 = 24MHz /((Width + HSYNC_W + HBP + HFP)*(Height + VSYNC_W + VBP + VFP))

= 24000000/((800 + 96 + 10 + 10)*(480 + 2 + 10 + 10))

= 24000000/(916*502)

= 52Hz

根据需要可以加大，100Hz 刷新率完全没问题，设置 PeriphClkInitStruct.PLL3.PLL3N = 100 即可，此时的 LTDC 时钟是 50MHz

刷新率 = 50MHz /（(Width + HSYNC_W+ HBP+ HFP ）*(Height + VSYNC_W +VBP +VFP ））

= 5000000/(916*502)

= 108.7Hz

◆ 时序参数配置

时序参数的配置也比较容易，其实就是先看 STM32H7 参考手册上面的公式说明，说是公式，其实就是简单的加减法。然后将 OTA7001 的参数代到这个公式就可以了。又因为手册一般都是给出了参数的最小值，典型值和最大值，大家可以根据实际情况做简单的调整即可。需要用到的参数：

```
uint16_t Width, Height, HSYNC_W, VSYNC_W, HBP, HFP, VBP, VFP;

Horizontal Synchronization (Hsync)  对应变量 HSYNC_W
Horizontal Back Porch (HBP)         对应变量 HBP
Active Width                        对应变量 Width
Horizontal Front Porch (HFP)        对应变量 HFP
Vertical Synchronization (Vsync)    对应变量 VSYNC_W
Vertical Back Porch (VBP)           对应变量 VBP
Active Heigh                        对应变量 Heigh
Vertical Front Porch (VFP)          对应变量 VFP
```

STM32H7 参考手册上面的公式如下：

```
**********************************************************************************************************
*       LCD_TFT 同步时序配置（整理自官方做的一个截图，言简意赅）：
*       ------------------------------------------------------------------------
*
*                                          Total Width
*                          <-----------------------------------------------------
*                     Hsync width HBP             Active Width                HFP
*                          <--->|<-->|<-----------------------------------><-->
*                       ____    ____|_____________________________________|____
*                           |___|    |                                     |    |
*                                    |                                     |    |
*                       __|          |                                     |    |
*            /|\    /|\  |           |                                     |    |
*             | VSYNC|   |           |                                     |    |
*             |Width\|/  |__         |                                     |    |
*             |     /|\     |        |                                     |    |
*             |  VBP |      |        |                                     |    |
*             |     \|/_____|________|_____________________________________|    |
*             |     /|\     |        | / / / / / / / / / / / / / / / / / / |    |
*             |      |      |        |/ / / / / / / / / / / / / / / / / / /|    |
*    Total    |      |      |        |/ / / / / / / / / / / / / / / / / / /|    |
*    Heigh    |      |      |        |/ / / / / / / / / / / / / / / / / / /|    |
*             |Active|      |        |/ / / / / / / / / / / / / / / / / / /|    |
*             |Heigh |      |        |/ / / / / Active Display Area / / / /|    |
*             |      |      |        |/ / / / / / / / / / / / / / / / / / /|    |
*             |      |      |        |/ / / / / / / / / / / / / / / / / / /|    |
*             |      |      |        |/ / / / / / / / / / / / / / / / / / /|    |
*             |      |      |        |/ / / / / / / / / / / / / / / / / / /|    |
*             |      |      |        |/ / / / / / / / / / / / / / / / / / /|    |
*             |     \|/_____|________|_____________________________________|    |
*             |     /|\     |                                                   |
*             |  VFP |      |                                                   |
*            \|/    \|/_____|___________________________________________________|
*
*
*     每个 LCD 设备都有自己的同步时序值：
```

```
*     Horizontal Synchronization (Hsync)
*     Horizontal Back Porch (HBP)
*     Active Width
*     Horizontal Front Porch (HFP)
*
*     Vertical Synchronization (Vsync)
*     Vertical Back Porch (VBP)
*     Active Heigh
*     Vertical Front Porch (VFP)
*
*     LCD_TFT 窗口水平和垂直的起始以及结束位置 :
*     ------------------------------------------------------------
*
*     HorizontalStart = (Offset_X + Hsync + HBP);
*     HorizontalStop  = (Offset_X + Hsync + HBP + Window_Width - 1);
*     VarticalStart   = (Offset_Y + Vsync + VBP);
*     VerticalStop    = (Offset_Y + Vsync + VBP + Window_Heigh - 1);
*
*********************************************************************************************************
```

OTA7001 手册中已经给出了我们需要的数值：

uint16_t Width, Height, HSYNC_W, HBP, HFP, VSYNC_W, VBP, VFP。

Horizontal input timing

Parameter		Symbol	Value			Unit	
			Min.	Typ.	Max.		
Horizontal display area		thd		800		DCLK	Width = 800
DCLK frequency		fclk	-	30	50	MHz	
1 Horizontal Line		th		928		DCLK	
HSD pulse width	Min.	thpw		1			HSYNC_W = 48
	Typ.			48			
	Max.			-			
HSD Back Porch (Blanking)		thb	-	88	-		HBP = 88
HSD Front Porch		thfp	-	40	-		HFP = 40

Vertical input timing

Parameter	Symbol	Value			Unit	
		Min.	Typ.	Max.		
Vertical display area	tvd		480		H	Height = 480
VSD period time	tv	-	525	-	H	
VSD pulse width	tvpw	-	3	-	H	VSYNC_W = 3
VSD Back Porch (Blanking)	tvb	-	32	-	H	VBP = 32
VSD Front Porch	tvfp	-	13	-	H	VFP = 13

参数设置好了，直接代入公式并与行同步，场同步和 DE 一起初始化：

```
/* 时序配置 */
hltdc_F.Init.HorizontalSync = (HSYNC_W - 1);
hltdc_F.Init.VerticalSync = (VSYNC_W - 1);
hltdc_F.Init.AccumulatedHBP = (HSYNC_W + HBP - 1);
hltdc_F.Init.AccumulatedVBP = (VSYNC_W + VBP - 1);
hltdc_F.Init.AccumulatedActiveH = (Height + VSYNC_W + VBP - 1);
hltdc_F.Init.AccumulatedActiveW = (Width + HSYNC_W + HBP - 1);
hltdc_F.Init.TotalHeigh = (Height + VSYNC_W + VBP + VFP - 1);
hltdc_F.Init.TotalWidth = (Width + HSYNC_W + HBP + HFP - 1);
```

至此，时序配置工作就完成了。

这里特别注意，当前程序中实际使用的参数与本小节的参数略有区别。由于这些参数都有较大的容错范围，所以很多参数都可以正常使用。

51.4.4 第 4 步，如何验证 LTDC 的时序配置是否正确

下面说一个最重要的问题，配置好时序了，怎么检查自己的配置是否成功了?用户仅需在函数 LCDH7_ConfigLTDC 里面的如下代码后面加上两个函数：

```
/* 配置 LTDC  */
if (HAL_LTDC_Init(&hltdc_F) != HAL_OK)
{
    /* 初始化错误 */
    Error_Handler(__FILE__, __LINE__);
}

/* 下面是添加的 */
LCD_SetBackLight(BRIGHT_DEFAULT);
while(1);
```

加上这两行代码后，再将背景层设置为一个合适的颜色，建议设置成红色，方便观察：

```
/* 配置背景层颜色 */
hltdc_F.Init.Backcolor.Blue = 0;
hltdc_F.Init.Backcolor.Green = 0;
hltdc_F.Init.Backcolor.Red = 255;
```

如果背景层可以正常显示红色，说明引脚和时序配置都是没有问题的。如果不成功要从以下几个方面着手检查：

- 首先要清楚一点，当前的配置是否成功与 SDRAM 没有任何关系，因为背景层还用不到 SDRAM，图层 1 和图层 2 才需要 SDRAM 做显存使用。
- 从硬件着手检查，保证 STM32H7 芯片焊接没问题，TFT 接口一定要牢固，防止接触不良，特别是使用 FPC 软排线的时候，测试阶段，软排线越短越好。有时候也可能是显示屏有问题，最好可以备两个显示屏测试。
- 从软件配置着手检查，查看 LTDC 涉及到的所有引脚是否配置，引脚时钟是否使能。有时候无法显示也有可能是板子硬件设计不规范导致干扰较大造成的，此时，可以降低 LTDC 所涉及到 GPIO 的速度等级。
- 如果显示了，但是显示的位置不正确，可以重新调整时序参数即可。

51.4.5第 5 步，LTDC 图层配置

LTDC 的图层配置就比较好理解了，下面是完整的驱动代码：

```
/*
*********************************************************************************************************
*    函 数 名: LCDH7_ConfigLTDC
*    功能说明: 配置 LTDC
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void LCDH7_ConfigLTDC(void)
{
    /* GPIO 初始化部分省略未写 */

    /*##-2- LTDC 初始化 ######################################################################*/
    {
        LTDC_LayerCfgTypeDef      pLayerCfg;
        uint16_t Width, Height, HSYNC_W, HBP, HFP, VSYNC_W, VBP, VFP;
        RCC_PeriphCLKInitTypeDef  PeriphClkInitStruct;

        /* 支持 6 种面板 */
        switch (g_LcdType)
        {
            case LCD_35_480X320:    /* 3.5 寸 480 * 320 */
                Width = 480;
                Height = 272;
                HSYNC_W = 10;
                HBP = 20;
                HFP = 20;
                VSYNC_W = 20;
                VBP = 20;
                VFP = 20;
                break;

            case LCD_43_480X272:        /* 4.3 寸 480 * 272 */
                Width = 480;
                Height = 272;

                HSYNC_W = 40;
                HBP = 2;
                HFP = 2;
                VSYNC_W = 9;
                VBP = 2;
                VFP = 2;

                /* LCD 时钟配置 */
                /* PLL3_VCO Input = HSE_VALUE/PLL3M = 25MHz/5 = 5MHz */
                /* PLL3_VCO Output = PLL3_VCO Input * PLL3N = 5MHz * 48 = 240MHz */
                /* PLLLCDCLK = PLL3_VCO Output/PLL3R = 240 / 10 = 24MHz */
                /* LTDC clock frequency = PLLLCDCLK = 24MHz */
                /*
                    刷新率 = 24MHz /((Width + HSYNC_W  + HBP  + HFP)*(Height + VSYNC_W +  VBP  + VFP))
                           = 24000000/((480 + 40  + 2  + 2)*(272 + 9 +  2  + 2))
                           = 24000000/(524*285)
```

```
                                = 160Hz

                        当前这个配置方便用户使用 PLL3Q 输出的 48MHz 时钟供 USB 使用。
                    */
                    PeriphClkInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LTDC;
                    PeriphClkInitStruct.PLL3.PLL3M = 5;
                    PeriphClkInitStruct.PLL3.PLL3N = 48;
                    PeriphClkInitStruct.PLL3.PLL3P = 2;
                    PeriphClkInitStruct.PLL3.PLL3Q = 5;
                    PeriphClkInitStruct.PLL3.PLL3R = 10;
                    HAL_RCCEx_PeriphCLKConfig(&PeriphClkInitStruct);
                    break;

                case LCD_50_480X272:        /* 5.0 寸 480 * 272 */
                    Width = 480;
                    Height = 272;

                    HSYNC_W = 40;
                    HBP = 2;
                    HFP = 2;
                    VSYNC_W = 9;
                    VBP = 2;
                    VFP = 2;
                    break;

                case LCD_50_800X480:        /* 5.0 寸 800 * 480 */
                case LCD_70_800X480:        /* 7.0 寸 800 * 480 */
                    Width = 800;
                    Height = 480;

                    HSYNC_W = 96;
                    HBP = 10;
                    HFP = 10;
                    VSYNC_W = 2;
                    VBP = 10;
                    VFP = 10;

                    /* LCD 时钟配置 */
                    /* PLL3_VCO Input = HSE_VALUE/PLL3M = 25MHz/5 = 5MHz */
                    /* PLL3_VCO Output = PLL3_VCO Input * PLL3N = 5MHz * 48 = 240MHz */
                    /* PLLLCDCLK = PLL3_VCO Output/PLL3R = 240 / 10 = 24MHz */
                    /* LTDC clock frequency = PLLLCDCLK = 24MHz */
                    /*
                        刷新率 = 24MHz /((Width + HSYNC_W  + HBP  + HFP)*(Height + VSYNC_W +  VBP  + VFP))
                               = 24000000/((800 + 96  + 10  + 10)*(480 + 2 +  10  + 10))
                               = 24000000/(916*502)
                               = 52Hz

                根据需要可以加大，100Hz 刷新率完全没问题，设置 PeriphClkInitStruct.PLL3.PLL3N = 100 即可
                此时的 LTDC 时钟是 50MHz
                刷新率 = 50MHz /（（Width + HSYNC_W  + HBP  + HFP ）*(Height + VSYNC_W +  VBP  +VFP  ))
                       = 5000000/(916*502)
                       = 108.7Hz

                        当前这个配置方便用户使用 PLL3Q 输出的 48MHz 时钟供 USB 使用。
                    */
                    PeriphClkInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LTDC;
                    PeriphClkInitStruct.PLL3.PLL3M = 5;
```

```
111.                    PeriphClkInitStruct.PLL3.PLL3N = 48;
112.                    PeriphClkInitStruct.PLL3.PLL3P = 2;
113.                    PeriphClkInitStruct.PLL3.PLL3Q = 5;
114.                    PeriphClkInitStruct.PLL3.PLL3R = 10;
115.                    HAL_RCCEx_PeriphCLKConfig(&PeriphClkInitStruct);
116.                    break;
117.
118.                case LCD_70_1024X600:       /* 7.0 寸 1024 * 600 */
119.                    /* 实测像素时钟 = 53.7M */
120.                    Width = 1024;
121.                    Height = 600;
122.
123.                    HSYNC_W = 2;
124.                    HBP = 157;
125.                    HFP = 160;
126.                    VSYNC_W = 2;
127.                    VBP = 20;
128.                    VFP = 12;
129.
130.                    PeriphClkInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LTDC;
131.                    PeriphClkInitStruct.PLL3.PLL3M = 5;
132.                    PeriphClkInitStruct.PLL3.PLL3N = 48;
133.                    PeriphClkInitStruct.PLL3.PLL3P = 2;
134.                    PeriphClkInitStruct.PLL3.PLL3Q = 5;
135.                    PeriphClkInitStruct.PLL3.PLL3R = 10;
136.                    HAL_RCCEx_PeriphCLKConfig(&PeriphClkInitStruct);
137.                    break;
138.
139.                default:
140.                    Width = 800;
141.                    Height = 480;
142.
143.                    HSYNC_W = 80;
144.                    HBP = 10;
145.                    HFP = 10;
146.                    VSYNC_W = 10;
147.                    VBP = 10;
148.                    VFP = 10;
149.
150.                    /* LCD 时钟配置 */
151.                    /* PLL3_VCO Input = HSE_VALUE/PLL3M = 25MHz/5 = 5MHz */
152.                    /* PLL3_VCO Output = PLL3_VCO Input * PLL3N = 5MHz * 48 = 240MHz */
153.                    /* PLLLCDCLK = PLL3_VCO Output/PLL3R = 240 / 10 = 24MHz */
154.                    /* LTDC clock frequency = PLLLCDCLK = 24MHz */
155.                    /*
156.                        刷新率 = 24MHz /((Width + HSYNC_W  + HBP  + HFP)*(Height + VSYNC_W +  VBP  + VFP))
157.                               = 24000000/((800 + 96  + 10  + 10)*(480 + 2 +  10  + 10))
158.                               = 24000000/(916*502)
159.                               = 52Hz
160.
161.                根据需要可以加大，100Hz 刷新率完全没问题，设置 PeriphClkInitStruct.PLL3.PLL3N = 100 即可
162.                此时的 LTDC 时钟是 50MHz
163.                刷新率 = 50MHz /（（Width + HSYNC_W  + HBP  + HFP ）*(Height + VSYNC_W +  VBP  +VFP  ））
164.                       = 5000000/(916*502)
165.                       = 108.7Hz
166.
167.                        当前这个配置方便用户使用 PLL3Q 输出的 48MHz 时钟供 USB 使用。
168.                    */
```

```
169.                    PeriphClkInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LTDC;
170.                    PeriphClkInitStruct.PLL3.PLL3M = 5;
171.                    PeriphClkInitStruct.PLL3.PLL3N = 48;
172.                    PeriphClkInitStruct.PLL3.PLL3P = 2;
173.                    PeriphClkInitStruct.PLL3.PLL3Q = 5;
174.                    PeriphClkInitStruct.PLL3.PLL3R = 10;
175.                    HAL_RCCEx_PeriphCLKConfig(&PeriphClkInitStruct);
176.                    break;
177.            }
178.
179.            g_LcdHeight = Height;
180.            g_LcdWidth = Width;
181.
182.            /* 配置信号极性 */
183.            hltdc_F.Init.HSPolarity = LTDC_HSPOLARITY_AL;  /* HSYNC 低电平有效 */
184.            hltdc_F.Init.VSPolarity = LTDC_VSPOLARITY_AL;  /* VSYNC 低电平有效 */
185.            hltdc_F.Init.DEPolarity = LTDC_DEPOLARITY_AL;  /* DE 低电平有效 */
186.            hltdc_F.Init.PCPolarity = LTDC_PCPOLARITY_IPC;
187.
188.            /* 时序配置 */
189.            hltdc_F.Init.HorizontalSync = (HSYNC_W - 1);
190.            hltdc_F.Init.VerticalSync = (VSYNC_W - 1);
191.            hltdc_F.Init.AccumulatedHBP = (HSYNC_W + HBP - 1);
192.            hltdc_F.Init.AccumulatedVBP = (VSYNC_W + VBP - 1);
193.            hltdc_F.Init.AccumulatedActiveH = (Height + VSYNC_W + VBP - 1);
194.            hltdc_F.Init.AccumulatedActiveW = (Width + HSYNC_W + HBP - 1);
195.            hltdc_F.Init.TotalHeigh = (Height + VSYNC_W + VBP + VFP - 1);
196.            hltdc_F.Init.TotalWidth = (Width + HSYNC_W + HBP + HFP - 1);
197.
198.            /* 配置背景层颜色 */
199.            hltdc_F.Init.Backcolor.Blue = 0;
200.            hltdc_F.Init.Backcolor.Green = 0;
201.            hltdc_F.Init.Backcolor.Red = 0;
202.
203.            hltdc_F.Instance = LTDC;
204.
205.            /* 开始配置图层 ------------------------------------------------------*/
206.            /* 窗口显示区设置 */
207.            pLayerCfg.WindowX0 = 0;
208.            pLayerCfg.WindowX1 = Width;
209.            pLayerCfg.WindowY0 = 0;
210.            pLayerCfg.WindowY1 = Height;
211.
212.            /* 配置颜色格式 */
213.            pLayerCfg.PixelFormat = LTDC_PIXEL_FORMAT_RGB565;
214.
215.            /* 显存地址 */
216.            pLayerCfg.FBStartAdress = LCDH7_FRAME_BUFFER;
217.
218.            /* Alpha 常数 (255 表示完全不透明) */
219.            pLayerCfg.Alpha = 255;
220.
221.            /* 无背景色 */
222.            pLayerCfg.Alpha0 = 0;   /* 完全透明 */
223.            pLayerCfg.Backcolor.Blue = 0;
224.            pLayerCfg.Backcolor.Green = 0;
225.            pLayerCfg.Backcolor.Red = 0;
226.
```

```
227.          /* 配置图层混合因数 */
228.          pLayerCfg.BlendingFactor1 = LTDC_BLENDING_FACTOR1_CA;
229.          pLayerCfg.BlendingFactor2 = LTDC_BLENDING_FACTOR2_CA;
230.
231.          /* 配置行列大小 */
232.          pLayerCfg.ImageWidth  = Width;
233.          pLayerCfg.ImageHeight = Height;
234.
235.          /* 配置 LTDC  */
236.          if (HAL_LTDC_Init(&hltdc_F) != HAL_OK)
237.          {
238.              /* 初始化错误 */
239.              Error_Handler(__FILE__, __LINE__);
240.          }
241.
242.          /* 配置图层 1 */
243.          if (HAL_LTDC_ConfigLayer(&hltdc_F, &pLayerCfg, LTDC_LAYER_1) != HAL_OK)
244.          {
245.              /* 初始化错误 */
246.              Error_Handler(__FILE__, __LINE__);
247.          }
248.
249.          #if 0
250.          /* 配置图层 2 */
251.          if (HAL_LTDC_ConfigLayer(&hltdc_F, &pLayerCfg, LTDC_LAYER_2) != HAL_OK)
252.          {
253.              /* 初始化错误 */
254.              Error_Handler(__FILE__, __LINE__);
255.          }
256.         #endif
257.      }
258.
259. #if  1
260.      HAL_NVIC_SetPriority(LTDC_IRQn, 0xE, 0);
261.      HAL_NVIC_EnableIRQ(LTDC_IRQn);
262. #endif
263. }
```

下面将几个关键的地方做个阐释：

- 第 20 行，六种显示面板的 LTDC 输出时钟和时序参数配置，六种面板的识别是在 bsp_touch.c 文件中实现的。大家自己配置时用不到这个，仅需提供一组时序参数和输出时钟即可，除非项目中需要切换不同显示屏。
- 第 78-116 行，这里是 7 寸面板的 LTDC 时钟输出配置和时序参数配置，配置方法在前面几步已经介绍，这里不再赘述，其它面板的设置方法是一样的。
- 第 179-180 行，全局变量 g_LcdWidth 和 g_LcdHeight 在文件 bsp_tft_lcd.c 文件定义，用来记录显示屏的分辨率。
- 第 183-186 行，用来配置行同步，场同步，DE 数据使能和 LTDC 像素时钟的极性。
- 第 189-196 行，配置时序。
- 第 199-201 行，配置背景层颜色。
- 第 207-210 行，用来设置图层在 LCD 显示区的起始位置和结束位置。

- 第 213 行，用来设置图层的颜色格式，STM32H7 支持 8 种颜色格式，这里是设置为 RGB565。
- 第 216 行，用来设置图层的显示首地址。
- 第 219-229 行，在第 50 章的 2.5 小节详细讲解了这行代码的作用。
- 第 232-233 行，用于设置从显存中要读取的行长和行数。
- 第 236-240 行，配置 LTDC 的基本参数。
- 第 243-247 行，配置图层 1。
- 第 251-255 行，配置图层 2，暂时未用到图层 2（通过条件编译取消执行）。
- 第 260-261 行，配置 LTDC 中断的优先级并使能。

51.4.6第 6 步，LCD 背光实现

LCD 的背光是 PWM 驱动方式，涉及到的代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: LCD_SetBackLight
*    功能说明: 初始化控制 LCD 背景光的 GPIO,配置为 PWM 模式。
*            当关闭背光时，将 CPU IO 设置为浮动输入模式（推荐设置为推挽输出，并驱动到低电平）；将 TIM3 关闭 省电
*    形    参: _bright 亮度，0 是灭，255 是最亮
*    返 回 值: 无
*********************************************************************************************************
*/
void LCD_SetBackLight(uint8_t _bright)
{
    s_ucBright =  _bright; /* 保存背光值 */

    LCD_SetPwmBackLight(s_ucBright);
}

/*
*********************************************************************************************************
*    函 数 名: LCD_SetPwmBackLight
*    功能说明: 初始化控制 LCD 背景光的 GPIO,配置为 PWM 模式。
*            当关闭背光时，将 CPU IO 设置为浮动输入模式（推荐设置为推挽输出，并驱动到低电平）；将 TIM3 关闭 省电
*    形    参:  _bright 亮度，0 是灭，255 是最亮
*    返 回 值: 无
*********************************************************************************************************
*/
static void LCD_SetPwmBackLight(uint8_t _bright)
{
    /* 背光有 CPU 输出 PWM 控制，PA0/TIM5_CH1/TIM2_CH1 */
    //bsp_SetTIMOutPWM(GPIOA, GPIO_PIN_0, TIM5, 1, 100, (_bright * 10000) /255);
    //bsp_SetTIMOutPWM(GPIOA, GPIO_PIN_0, TIM5, 1, 20000, (_bright * 10000) /255);
    bsp_SetTIMOutPWM(GPIOA, GPIO_PIN_8, TIM1, 1, 20000, (_bright * 10000) /255);
}
```

函数的注释已经比较详细。另外，背光是基于第 34 章的 API：bsp_SetTIMOutPWM 实现，关于这个函数可以看第 34 章节。

51.5 LCD 板级支持包（bsp_ltdc_h7.c 和 bsp_tft_lcd.c)

bsp_ltdc_h7.c 是 H7 的 LTDC 驱动文件，涉及到的函数比较多。而 bsp_tft_lcd.c 文件是在 LTDC 的 API 基础上面封装出更通用的函数，不仅仅 LTDC，像 RA8875，ili9488 等也可通过此文件进行封装。

本章节主要给几个常用的基本 API 做个介绍：

- LCD_InitHard
- LCD_ClrScr
- LCD_SetBackLight
- LCD_DispStr
- LCD_PutPixel
- LCD_DrawLine
- LCD_DrawRect
- LCD_DrawCircle
- LCD_Fill_Rect

51.5.1 函数 LCD_InitHard

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: LCD_InitHard
*    功能说明: 初始化 LCD
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void LCD_InitHard(void)
{
    LCD_HardReset();

    LCDH7_InitHard();

    LCD_SetDirection(0);

    LCD_ClrScr(CL_BLACK);  /* 清屏，显示全黑 */

//  LCD_SetBackLight(BRIGHT_DEFAULT);
}
```

函数描述：

此函数用于初始化 LCD，配置了 STM32H7 的 LTDC 控制器，设置横向显示，默认清屏为黑色。

注意事项：

1. 调用此函数前，务必优先调用函数 TOUCH_InitHard 识别不同分辨率面板。

2. 通过函数 LCD_SetDirection 可以设置横向，竖向，横向 180 度，竖向 180 度显示。在后面章节再为大家介绍具体的实现方法。

使用举例：

作为初始化函数，直接在 bsp.c 文件的 bsp_Init 函数里面调用即可。

51.5.2 函数 LCD_ClrScr

函数原型：

```
void LCD_ClrScr(uint16_t _usColor)
```

函数描述：

此函数用于清屏操作。

函数参数：

◆ 第 1 个参数是清屏的颜色值设置，在 bsp_tft_lcd.h 文件中定义了几个常用的颜色：

```
/*
    LCD 颜色代码，CL_是 Color 的简写
    16Bit 由高位至低位， RRRR RGGG GGGB BBBB

    下面的 RGB 宏将 24 位的 RGB 值转换为 16 位格式。
    启动 windows 的画笔程序，点击编辑颜色，选择自定义颜色，可以获得的 RGB 值。

    推荐使用迷你取色器软件获得你看到的界面颜色。
*/
#define RGB(R,G,B) (((R >> 3) << 11) | ((G >> 2) << 5) | (B >> 3))/* 将 8 位 R,G,B 转化为 16 位 RGB565 格式 */

/* 解码出 R=8bit G=8bit B=8bit */
#define RGB565_R(x)  ((x >> 8) & 0xF8)
#define RGB565_G(x)  ((x >> 3) & 0xFC)
#define RGB565_B(x)  ((x << 3) & 0xF8)

/* 解码出 R=5bit G=6bit B=5bit */
#define RGB565_R2(x)  ((x >> 11) & 0x1F)
#define RGB565_G2(x)  ((x >> 5) & 0x3F)
#define RGB565_B2(x)  ((x >> 0) & 0x1F)


enum
{
    CL_WHITE        = RGB(255,255,255),   /* 白色 */
    CL_BLACK        = RGB(  0,  0,  0),   /* 黑色 */
    CL_RED          = RGB(255,  0,  0),   /* 红色 */
    CL_GREEN        = RGB(  0,255,  0),   /* 绿色 */
    CL_BLUE         = RGB(  0,  0,255),   /* 蓝色 */
    CL_YELLOW       = RGB(255,255,  0),   /* 黄色 */

    CL_GREY           = RGB( 98, 98, 98), /* 深灰色 */
    CL_GREY1      = RGB( 150, 150, 150),   /* 浅灰色 */
    CL_GREY2      = RGB( 180, 180, 180),  /* 浅灰色 */
    CL_GREY3      = RGB( 200, 200, 200),  /* 最浅灰色 */
    CL_GREY4      = RGB( 230, 230, 230),  /* 最浅灰色 */
```

```
    CL_BUTTON_GREY     = RGB( 220, 220, 220), /* WINDOWS 按钮表面灰色 */

    CL_MAGENTA      = 0xF81F,    /* 红紫色，洋红色 */
    CL_CYAN         = 0x7FFF,    /* 蓝绿色，青色 */

    CL_BLUE1        = RGB(  0,   0, 240),      /* 深蓝色 */
    CL_BLUE2        = RGB(  0,   0, 128),      /* 深蓝色 */
    CL_BLUE3        = RGB(  68,  68, 255),     /* 浅蓝色 1 */
    CL_BLUE4        = RGB(  0,  64, 128),      /* 浅蓝色 1 */

    /* UI 界面 Windows 控件常用色 */
    CL_BTN_FACE     = RGB(236, 233, 216),   /* 按钮表面颜色(灰) */

    CL_BTN_FONT     = CL_BLACK,             /* 按钮字体颜色（黑） */

    CL_BOX_BORDER1  = RGB(172, 168,153),    /* 分组框主线颜色 */
    CL_BOX_BORDER2  = RGB(255, 255,255),    /* 分组框阴影线颜色 */


    CL_MASK         = 0x9999 /* 颜色掩码，用于文字背景透明 */
};
```

使用举例：

比如将 LCD 清为红色，即 LCD_ClrSrc(CL_RED)。

51.5.3 函数 LCD_SetBackLight

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: LCD_SetBackLight
*    功能说明: 初始化控制 LCD 背景光的 GPIO,配置为 PWM 模式。
*            当关闭背光时，将 CPU IO 设置为浮动输入模式（推荐设置为推挽输出，并驱动到低电平)；将 TIM3 关闭 省电
*    形    参: _bright 亮度，0 是灭，255 是最亮
*    返 回 值: 无
*********************************************************************************************************
*/
void LCD_SetBackLight(uint8_t _bright)
{
    s_ucBright =  _bright; /* 保存背光值 */

    LCD_SetPwmBackLight(s_ucBright);
}
```

函数描述：

此函数主要用于 LCD 背光设置。

函数参数：

- 第 1 个参数是背光参数，0 表示灭，255 表示最亮。

使用举例：

比如设置 LCD 最亮 LCD_SetBackLight(255)。

51.5.4函数 LCD_DispStr

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: LCD_DispStr
*    功能说明: 在 LCD 指定坐标（左上角）显示一个字符串
*    形    参:
*        _usX : X 坐标
*        _usY : Y 坐标
*        _ptr  : 字符串指针
*        _tFont : 字体结构体，包含颜色、背景色(支持透明)、字体代码、文字间距等参数
*    返 回 值: 无
*********************************************************************************************************
*/
void LCD_DispStr(uint16_t _usX, uint16_t _usY, char *_ptr, FONT_T *_tFont)
{
    LCD_DispStrEx(_usX, _usY, _ptr, _tFont, 0, 0);
}
```

函数描述：

此函数用于在 LCD 指定位置显示字符串，中英文均支持。由于这个函数涉及到的知识点比较多，下章节会专门为大家讲解。

函数参数：

- 第 1 个参数是 x 轴坐标位置。
- 第 2 个参数是 y 轴坐标位置。
- 第 3 个参数是要显示的字符串。
- 第 4 个参数是 FONT_T 类型结构体，定义如下：

```
/* 字体属性结构，用于 LCD_DispStr() */
typedef struct
{
    FONT_CODE_E FontCode;  /* 字体代码 FONT_CODE_E  */
    uint16_t FrontColor;    /* 字体颜色 */
    uint16_t BackColor;    /* 文字背景颜色，透明 */
    uint16_t Space;        /* 文字间距，单位 = 像素 */
}FONT_T;
```

使用举例：

比如显示 12 点阵和 16 点阵字符。

```
FONT_T tFont12;            /* 定义一个字体结构体变量，用于设置字体参数 */
FONT_T tFont16;            /* 定义一个字体结构体变量，用于设置字体参数 */

/* 设置字体参数 */
{
    tFont12.FontCode = FC_ST_12;     /* 字体代码 12 点阵 */
    tFont12.FrontColor = CL_WHITE;   /* 字体颜色 */
    tFont12.BackColor = CL_BLUE;     /* 文字背景颜色 */
    tFont12.Space = 0;               /* 文字间距，单位 = 像素 */
}

/* 设置字体参数 */
```

```
{
    tFont16.FontCode = FC_ST_16;      /* 字体代码 16 点阵 */
    tFont16.FrontColor = CL_WHITE;    /* 字体颜色 */
    tFont16.BackColor = CL_BLUE;      /* 文字背景颜色 */
    tFont16.Space = 0;                /* 文字间距，单位 = 像素 */
}

LCD_ClrScr(CL_BLUE);

LCD_DispStr(5, 3, "故人西辞黄鹤楼，烟花三月下扬州。", &tFont12);
LCD_DispStr(5, 18, "孤帆远影碧空尽，唯见长江天际流。", &tFont12);
LCD_DispStr(5, 38, "故人西辞黄鹤楼，烟花三月下扬州。", &tFont16);
LCD_DispStr(5, 58, "孤帆远影碧空尽，唯见长江天际流。", &tFont16);
```

51.5.5 函数 LCD_PutPixel

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: LCD_PutPixel
*    功能说明: 画 1 个像素
*    形    参:
*            _usX,_usY : 像素坐标
*            _usColor  : 像素颜色
*    返 回 值: 无
*********************************************************************************************************
*/
void LCD_PutPixel(uint16_t _usX, uint16_t _usY, uint16_t _usColor)
{
    LCDH7_PutPixel(_usX, _usY, _usColor);
}
```

函数描述：

此函数用于在指定位置显示一个像素点。

函数参数：

- 第 1 个参数是 x 轴坐标位置。
- 第 2 个参数是 y 轴坐标位置。
- 第 3 个参数是像素点颜色。

使用举例：

比如在坐标(0, 0)显示红色，那就是 LCD_PutPixel(0, 0, CL_RED)。

51.5.6 函数 LCD_DrawLine

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: LCD_DrawLine
*    功能说明: 采用 Bresenham 算法，在 2 点间画一条直线。
```

```
*    形    参:
*            _usX1, _usY1 : 起始点坐标
*            _usX2, _usY2 : 终止点 Y 坐标
*            _usColor     : 颜色
*    返 回 值: 无
*********************************************************************************************************
*/
void LCD_DrawLine(uint16_t _usX1 , uint16_t _usY1 , uint16_t _usX2 , uint16_t _usY2 , uint16_t _usColor)
{
    LCDH7_DrawLine(_usX1 , _usY1 , _usX2, _usY2 , _usColor);
}
```

函数描述:

此函数用于任意两点间的直线绘制，采用的 Bresenham 算法，关于这个算法的介绍在帖子:

http://www.armbbs.cn/forum.php?mod=viewthread&tid=4048 。

函数参数:

- 第 1 个参数是起始坐标 x 轴位置。
- 第 2 个参数是起始坐标 y 轴位置。
- 第 3 个参数是结束坐标 x 轴位置。
- 第 4 个参数是结束坐标 y 轴位置。
- 第 5 个参数是直线颜色。

使用举例:

比如在坐标(0, 0)到坐标(100, 100)显示一条红色直线，那就是 LCD_DrawLine(0, 0, 100, 100, CL_RED)。

51.5.7 函数 LCD_DrawRect

函数原型:

```
/*
*********************************************************************************************************
*    函 数 名: LCD_DrawRect
*    功能说明: 绘制水平放置的矩形。
*    形    参:
*            _usX,_usY: 矩形左上角的坐标
*            _usHeight : 矩形的高度
*            _usWidth  : 矩形的宽度
*    返 回 值: 无
*********************************************************************************************************
*/
void LCD_DrawRect(uint16_t _usX, uint16_t _usY, uint16_t _usHeight, uint16_t _usWidth, uint16_t _usColor)
{
    LCDH7_DrawRect(_usX, _usY, _usHeight, _usWidth, _usColor);
}
```

函数描述:

此函数用于绘制矩形框。

函数参数:

- 第 1 个参数是左上角 X 轴位置。

- 第 2 个参数是左上角 Y 轴位置。
- 第 3 个参数是矩形框的长度。
- 第 4 个参数是矩形框的高度。
- 第 5 个参数是矩形框的颜色。

使用举例：

比如在坐标(0, 0)绘制一个长度为 100，高度为 50 的红色矩形框，那么就是 LCD_DrawRect(0, 0, 100, 50, CL_RED)。

51.5.8函数 LCD_DrawCircle

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: LCD_DrawCircle
*    功能说明: 绘制一个圆，笔宽为 1 个像素
*    形    参:
*            _usX,_usY  : 圆心的坐标
*            _usRadius  : 圆的半径
*    返 回 值: 无
*********************************************************************************************************
*/
void LCD_DrawCircle(uint16_t _usX, uint16_t _usY, uint16_t _usRadius, uint16_t _usColor)
{
    LCDH7_DrawCircle(_usX, _usY, _usRadius, _usColor);
}
```

函数描述：

此函数显示一个圆圈。

函数参数：

- 第 1 个参数是圆心坐标 x 轴。
- 第 2 个参数是圆心坐标 y 轴。
- 第 3 个参数是圆半径。
- 第 4 个参数是圆颜色。

使用举例：

比如在坐标(200, 200)绘制一个半径为 50 的红色圆圈，那么就是 LCD_DrawCircle (200, 200, 50, CL_RED)。

51.5.9函数 LCD_Fill_Rect

函数原型：

```
/*
*********************************************************************************************************
```

```
*    函 数 名: LCD_Fill_Rect
*    功能说明: 用一个颜色值填充一个矩形。【emWin 中有同名函数 LCD_FillRect，因此加了下划线区分】
*    形    参:
*             _usX,_usY: 矩形左上角的坐标
*             _usHeight : 矩形的高度
*             _usWidth  : 矩形的宽度
*    返 回 值: 无
*********************************************************************************************************
*/
void LCD_Fill_Rect(uint16_t _usX, uint16_t _usY, uint16_t _usHeight, uint16_t _usWidth, uint16_t _usColor)
{
    LCDH7_FillRect(_usX, _usY, _usHeight, _usWidth, _usColor);
}
```

函数描述：

此函数用绘制一个填充的矩形。

函数参数：

- 第 1 个参数是左上角 X 轴位置。
- 第 2 个参数是左上角 Y 轴位置。
- 第 3 个参数是填充矩形的长度。
- 第 4 个参数是填充矩形的高度。
- 第 5 个参数是填充矩形的颜色。

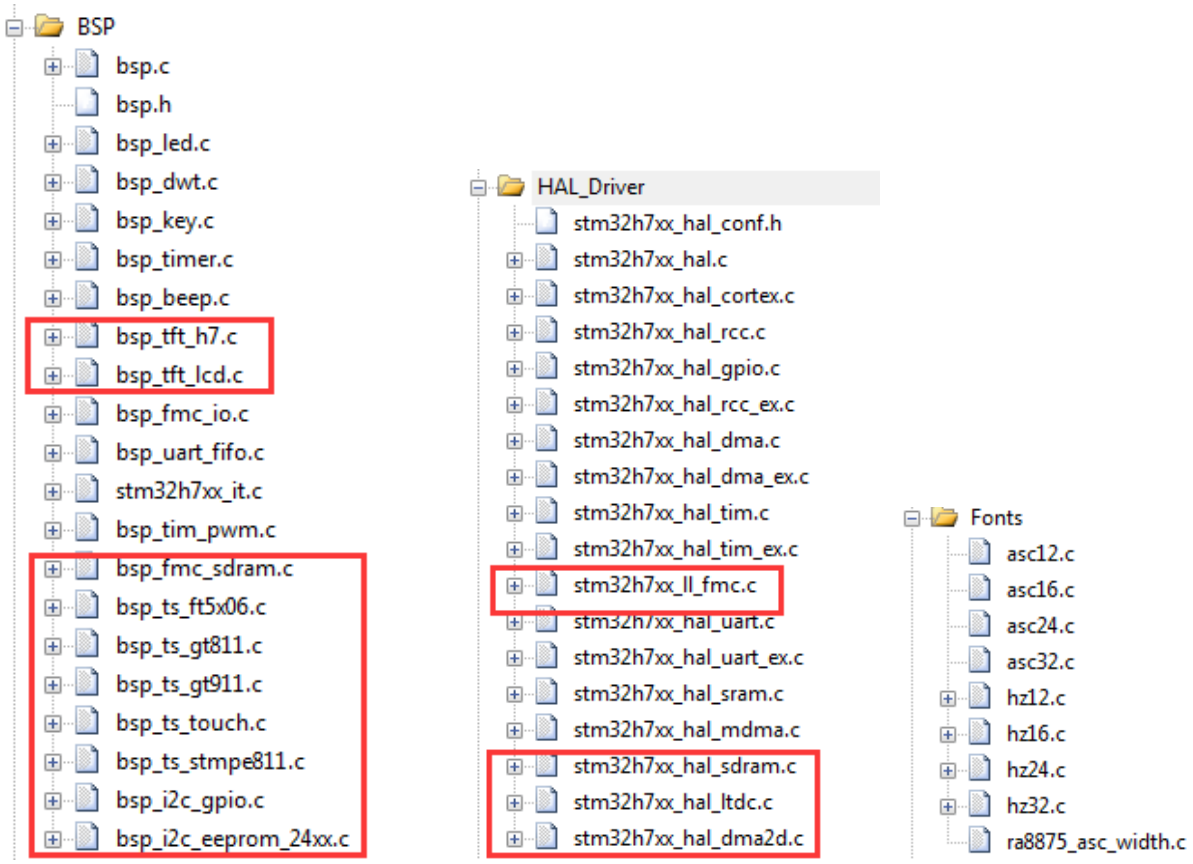
使用举例：

比如在坐标(0, 0)绘制一个长度为 100，高度为 50 的红色填充矩形，那么就是

LCD_Fill_Rect (0, 0, 100, 50, CL_RED)。

51.6 LCD 驱动移植和使用

由于我们开发板要做不同显示屏的自适应，所以关联了好多个文件，所有关于 TFT，触摸，触摸校准参数保存和字体的文件都要添加进来。这里有必要先为大家做个介绍才好移植。



◆ Fonts 分组中的文件

asc12.c ---12 点阵 ASCII 字符字库。

asc16.c---16 点阵 ASCII 字符字库。

hz12.c --- 12 点阵宋体小字库。

hz16.c --- 16 点阵宋体小字库。

hz24.c --- 24 点阵宋体小字库。

hz32.c --- 32 点阵宋体小字库。

ra8875_asc_width.c -- RA8875 ASCII 字体的宽度表。

◆ bsp 分组中的文件

bsp_fmc_sdram.c---用于 TFT 的显存。

bsp_tft_h7.c --- STM32H7 的 LTDC 的驱动文件。

bsp_tft_lcd.c --- TFT 驱动和相关 API 函数汇总文件，比如 RA8875 显示屏，ili9488 显示屏，STM32H7 所带 TFT 控制器驱动显示屏都可以有一个单独的文件，然后将这些显示屏相同功能的函数汇总成一个函数。这个文件就起到这个作用。

bsp_touch.c --- 触摸芯片自适应驱动，根据用户使用的触摸 IC 选择不同的驱动。另外，电阻屏的触摸扫描，触摸校准和触摸滤波也是在这个文件里面实现。

bsp_gt811.c --- 电容触摸芯片 GT811 的驱动以及触摸扫描。

bsp_gt911.c --- 电容触摸芯片 GT911 的驱动以及触摸扫描。

bsp_ft5x06.c --- 电容触摸芯片 FT5X06 的驱动以及触摸扫描。

TOUCH_STMPE811.c --- 电阻触摸芯片 STMPE811 的驱动。

bsp_tim_pwm.c --- 定时器驱动，显示屏的背光要用到 PWM。

bsp_i2c_gpio.c --- I2C 接口驱动，EEPROM，GT811，GT911，STMPE811 和 FT5X06 都要用到，因为他们的接口都是 I2C 方式。

bsp_eeprom_24xx.c --- EEPROM 驱动，用于存储电阻屏触摸校准参数。

- HAL_Driver 分组中的文件

stm32h7xx_ll_fmc.c 和 stm32h7xx_hal_sdram.c ---SDRAM 的驱动文件。

stm32h7xx_hal_tim.c --- 显示屏的背光是用 PWM 驱动的，需要用到这个定时器库文件。

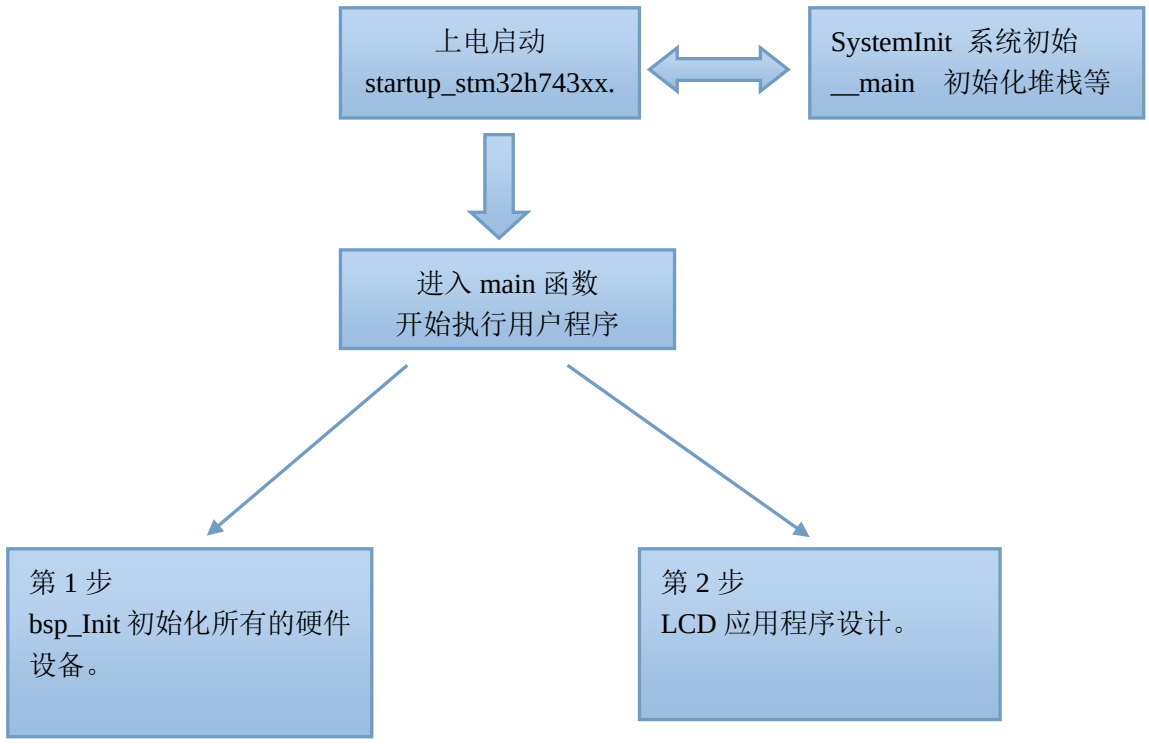
stm32h7xx_hal_ltdc.c --- LTDC 相关的 API 函数需要用到这个库文件。

stm32h7xx_hal_dma2d.c --- DMA2D 相关的 API 函数需要用到这个库文件。

对于本章节的驱动，不推荐单独移植了，建议直接使用本章节配套例子的基础上做修改。

51.7 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED 和串口。
  - 第 2 步，LCD 应用程序设计部分，显示汉字和 2D 图形。

51.8 实验例程说明（MDK）

配套例子：

V7-024_LCD 的汉字显示和 2D 图形显示（小字库）

实验目的：

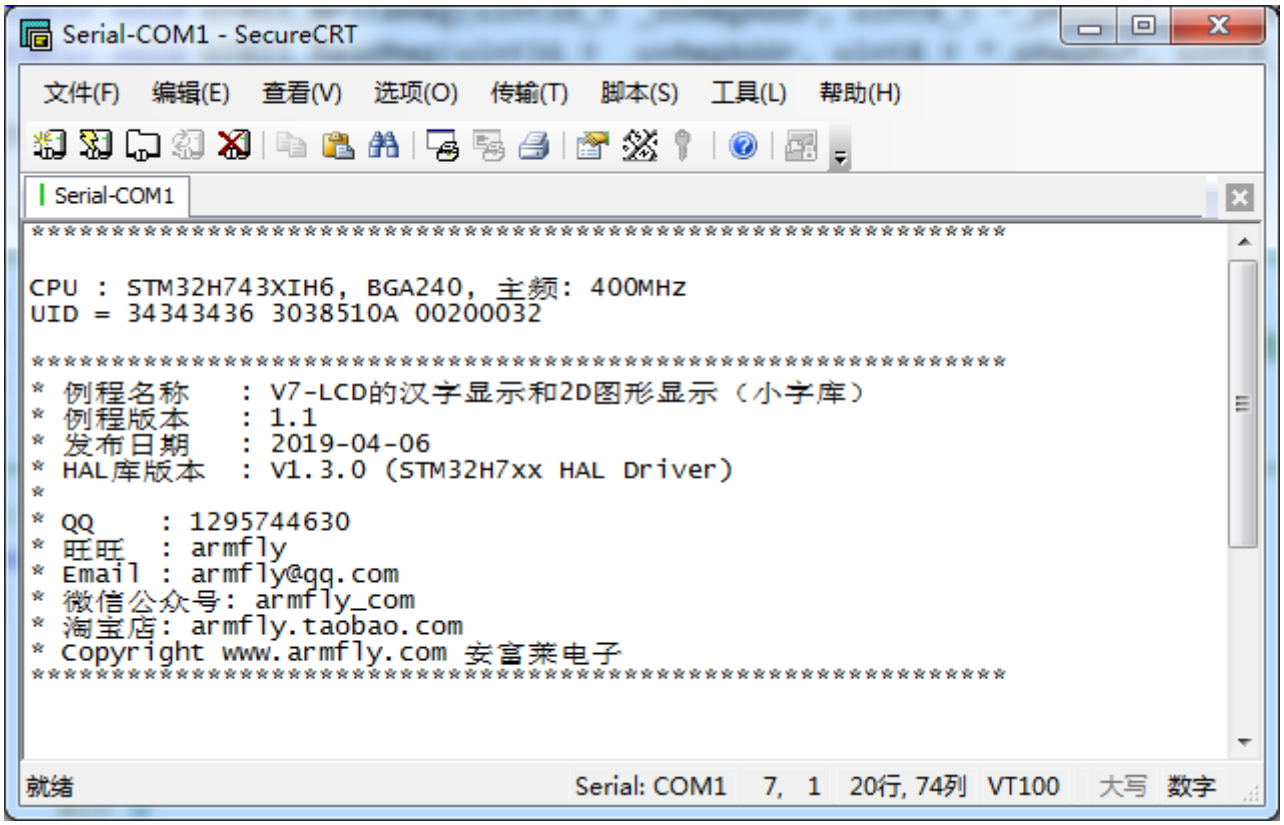
1. 学习 LCD 的小字库实现和 2D 图形显示。

实验内容：

1. 小字库通过此软件生成：
   http://www.armbbs.cn/forum.php?mod=viewthread&tid=202 。
2. LCD 界面上显示了汉字和 2D 图形。
3. 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。
4. 同时在 LCD 界面上实现一个简单计数，每 200ms 加 1，计数到 255 后继续从 0 开始。

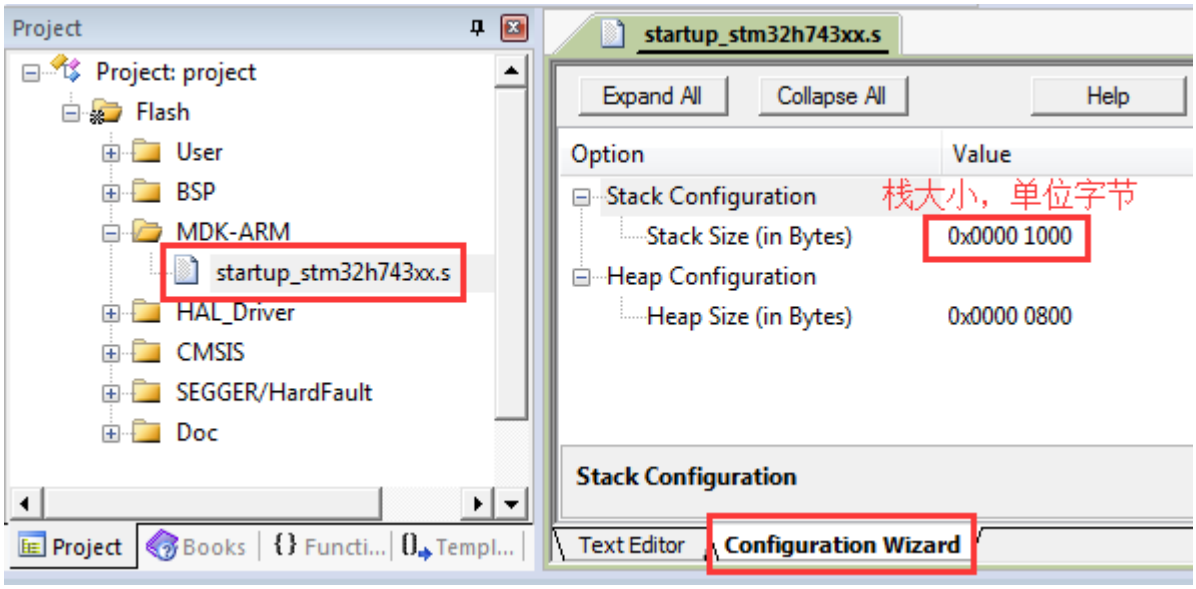
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

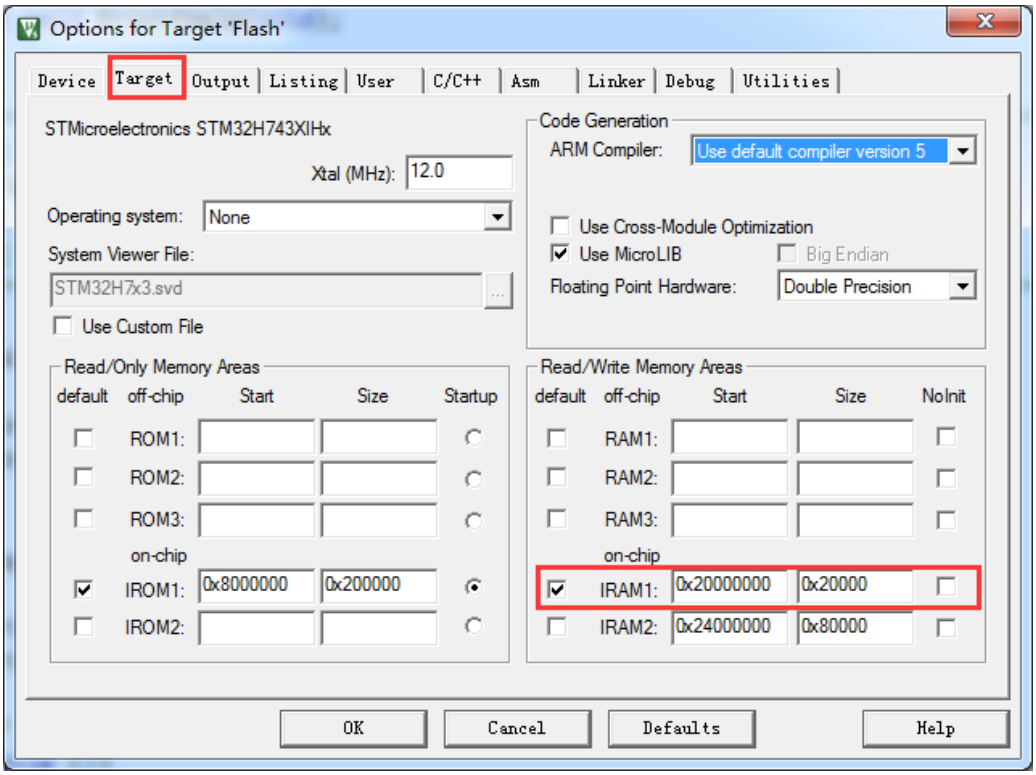


程序设计：

◆ **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
```

```
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */

    bsp_InitI2C();    /* 初始化 I2C 总线 */
    TOUCH_InitHard();  /* 初始化触摸芯片，LCD 面板型号的检查也在此函数，所以要在函数 LCD_InitHard 前调用 */
    LCD_InitHard();    /* 初始化 LCD */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SDRAM。由于 SDRAM 要用于 LCD 的显存，方便起见，直接将其配置为 WT 模式。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SDRAM 的 MPU 属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0xC0000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_32MB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}
```

```
/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- LCD 界面上显示了汉字和 2D 图形。
- 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。
- 同时在 LCD 界面上实现一个简单计数，每 200ms 加 1，计数到 255 后继续从 0 开始。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    FONT_T tFont12;             /* 定义一个字体结构体变量，用于设置字体参数 */
    FONT_T tFont16;             /* 定义一个字体结构体变量，用于设置字体参数 */
    uint8_t buf[100], count = 0;

    /* 设置字体参数 */
    {
        tFont12.FontCode = FC_ST_12;         /* 字体代码 12 点阵 */
        tFont12.FrontColor = CL_WHITE;       /* 字体颜色 */
        tFont12.BackColor = CL_BLUE;         /* 文字背景颜色 */
        tFont12.Space = 0;                   /* 文字间距，单位 = 像素 */
    }

    /* 设置字体参数 */
    {
        tFont16.FontCode = FC_ST_16;        /* 字体代码 16 点阵 */
        tFont16.FrontColor = CL_WHITE;      /* 字体颜色 */
        tFont16.BackColor = CL_BLUE;        /* 文字背景颜色 */
        tFont16.Space = 0;                  /* 文字间距，单位 = 像素 */
    }


    bsp_Init();   /* 硬件初始化 */
```

```
    PrintfLogo(); /* 打印例程名称和版本等信息 */

    /* 延迟 200ms 再点亮背光，避免瞬间高亮 */
    bsp_DelayMS(200);

    /* 清屏 */
    LCD_ClrScr(CL_BLUE);

    /* 显示汉字 */
    LCD_DispStr(5, 3, "故人西辞黄鹤楼，烟花三月下扬州。", &tFont12);
    LCD_DispStr(5, 18, "孤帆远影碧空尽，唯见长江天际流。", &tFont12);
    LCD_DispStr(5, 38, "故人西辞黄鹤楼，烟花三月下扬州。", &tFont16);
    LCD_DispStr(5, 58, "孤帆远影碧空尽，唯见长江天际流。", &tFont16);

    /* 绘制 2D 图形 */
    LCD_DrawLine(5, 120, 100, 220, CL_RED);
    LCD_DrawRect(120, 120, 100, 100, CL_RED);
    LCD_DrawCircle(280, 170, 50, CL_RED);
    LCD_Fill_Rect (340, 120, 100, 100, CL_BUTTON_GREY);

    /* 界面整体显示完毕后，再打开背光，设置为缺省亮度 */
    bsp_DelayMS(100);
    LCD_SetBackLight(BRIGHT_DEFAULT);


    bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */

    while (1)
    {
        /* 判断软件定时器 0 是否超时 */
        if(bsp_CheckTimer(0))
        {
            /* 每隔 200ms 进来一次 */
            bsp_LedToggle(2);

            sprintf((char *)buf, "count = %03d", count++);
            LCD_DispStr(5, 90, (char *)buf, &tFont16);
        }
    }
}
```

51.9 实验例程说明（IAR）

配套例子：

V7-024_LCD 的汉字显示和 2D 图形显示（小字库）

实验目的：

1. 学习 LCD 的小字库实现和 2D 图形显示。

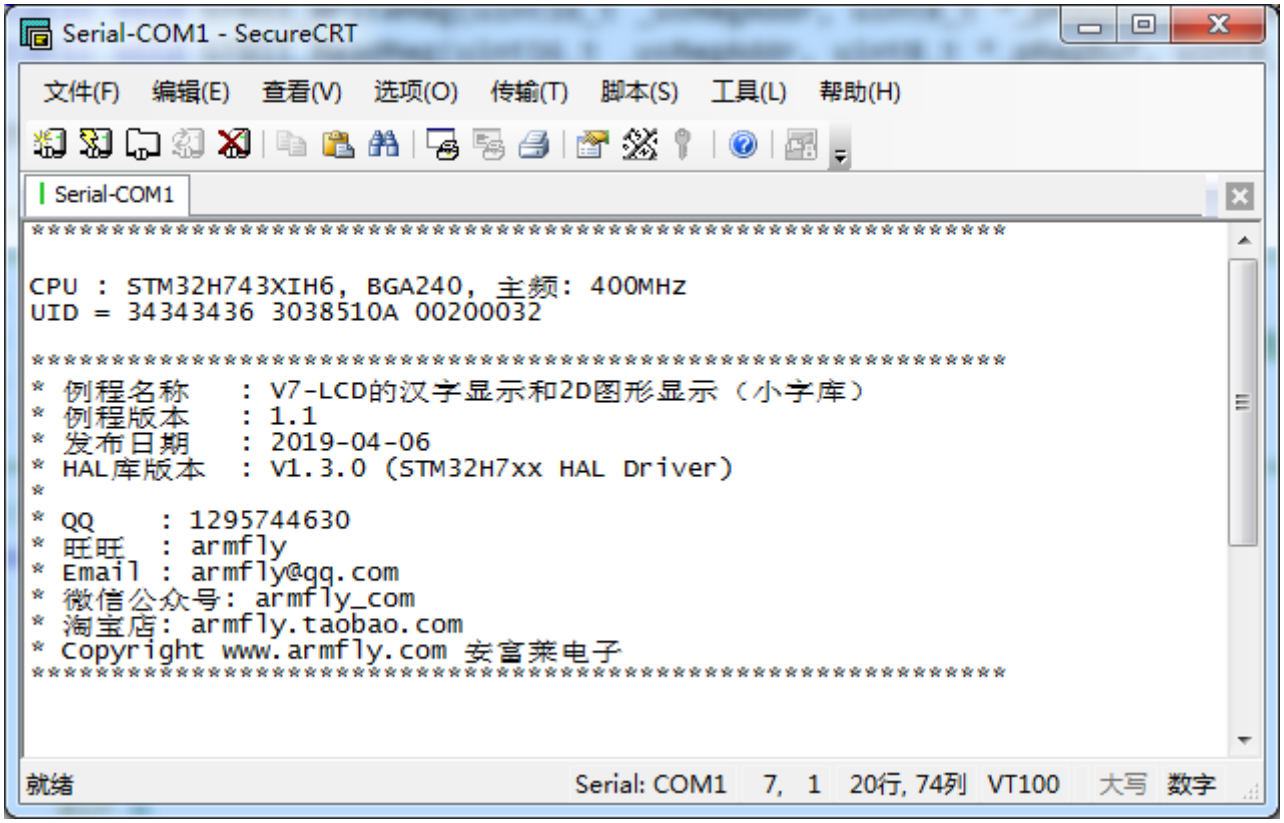
实验内容：

1. 小字库通过此软件生成：
   http://www.armbbs.cn/forum.php?mod=viewthread&tid=202 。
2. LCD 界面上显示了汉字和 2D 图形。

3. 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。
4. 同时在 LCD 界面上实现一个简单计数，每 200ms 加 1，计数到 255 后继续从 0 开始。

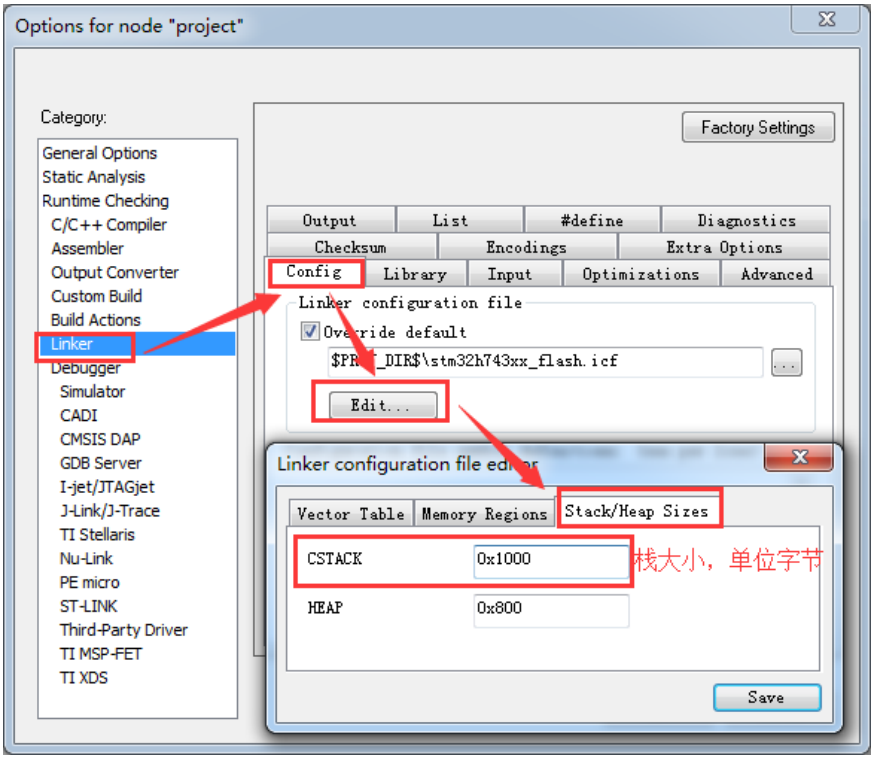
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

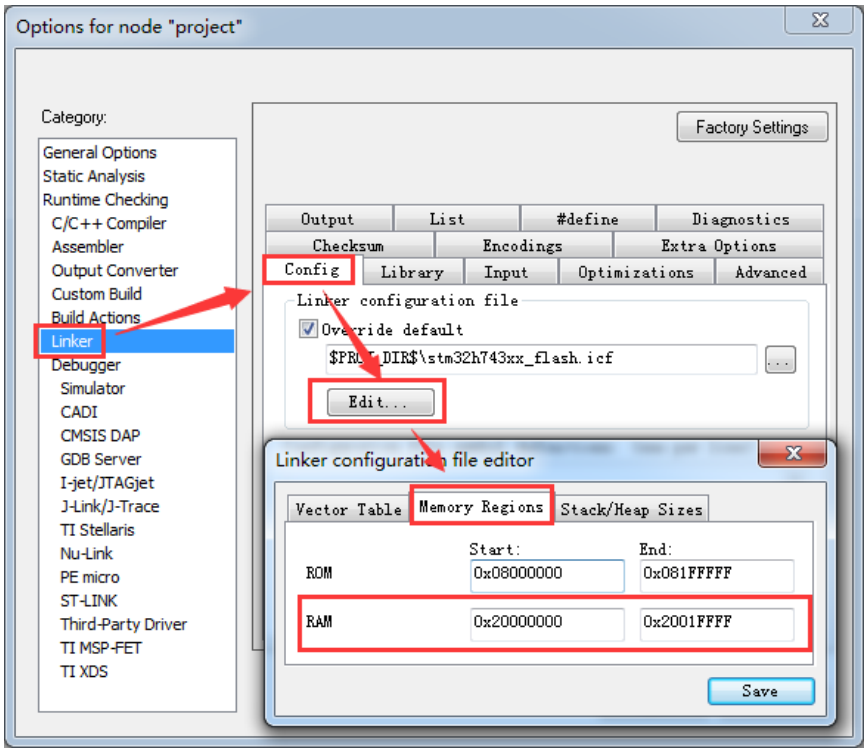


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
```

```
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */

    bsp_InitI2C();    /* 初始化 I2C 总线 */
    TOUCH_InitHard(); /* 初始化触摸芯片，LCD 面板型号的检查也在此函数，所以要在函数 LCD_InitHard 前调用 */
    LCD_InitHard();   /* 初始化 LCD */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SDRAM。由于 SDRAM 要用于 LCD 的显存，方便起见，直接将其配置为 WT 模式。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
```

```
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SDRAM 的 MPU 属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0xC0000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_32MB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
```

```
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- LCD 界面上显示了汉字和 2D 图形。
- 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。
- 同时在 LCD 界面上实现一个简单计数，每 200ms 加 1，计数到 255 后继续从 0 开始。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    FONT_T tFont12;             /* 定义一个字体结构体变量，用于设置字体参数 */
    FONT_T tFont16;             /* 定义一个字体结构体变量，用于设置字体参数 */
    uint8_t buf[100], count = 0;

    /* 设置字体参数 */
    {
        tFont12.FontCode = FC_ST_12;         /* 字体代码 12 点阵 */
        tFont12.FrontColor = CL_WHITE;       /* 字体颜色 */
        tFont12.BackColor = CL_BLUE;         /* 文字背景颜色 */
        tFont12.Space = 0;                   /* 文字间距，单位 = 像素 */
    }

    /* 设置字体参数 */
    {
        tFont16.FontCode = FC_ST_16;        /* 字体代码 16 点阵 */
        tFont16.FrontColor = CL_WHITE;      /* 字体颜色 */
        tFont16.BackColor = CL_BLUE;        /* 文字背景颜色 */
        tFont16.Space = 0;                  /* 文字间距，单位 = 像素 */
    }
```

```
    bsp_Init();   /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */

    /* 延迟 200ms 再点亮背光，避免瞬间高亮 */
    bsp_DelayMS(200);

    /* 清屏 */
    LCD_ClrScr(CL_BLUE);

    /* 显示汉字 */
    LCD_DispStr(5, 3, "故人西辞黄鹤楼，烟花三月下扬州。", &tFont12);
    LCD_DispStr(5, 18, "孤帆远影碧空尽，唯见长江天际流。", &tFont12);
    LCD_DispStr(5, 38, "故人西辞黄鹤楼，烟花三月下扬州。", &tFont16);
    LCD_DispStr(5, 58, "孤帆远影碧空尽，唯见长江天际流。", &tFont16);

    /* 绘制 2D 图形 */
    LCD_DrawLine(5, 120, 100, 220, CL_RED);
    LCD_DrawRect(120, 120, 100, 100, CL_RED);
    LCD_DrawCircle(280, 170, 50, CL_RED);
    LCD_Fill_Rect (340, 120, 100, 100, CL_BUTTON_GREY);

    /* 界面整体显示完毕后，再打开背光，设置为缺省亮度 */
    bsp_DelayMS(100);
    LCD_SetBackLight(BRIGHT_DEFAULT);

    bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */

    while (1)
    {
        /* 判断软件定时器 0 是否超时 */
        if(bsp_CheckTimer(0))
        {
            /* 每隔 200ms 进来一次 */
            bsp_LedToggle(2);

            sprintf((char *)buf, "count = %03d", count++);
            LCD_DispStr(5, 90, (char *)buf, &tFont16);
        }
    }
}
```

51.10 总结

本章节涉及到的知识点比较多，需要大家花点时间去掌握，直至可以独立驱动一个显示屏。

第52章 STM32H7 的 LTDC 应用之点阵字体和字符编码（重要）

本章节主要为大家讲解 ASCII 字符集，ASCII 扩展字符集，GB2312 字符集，GBK 字符集和 Unicode 字符集。其中，字符编码这块涉及到的知识点非常多，特别是 Unicode 字符集的编码，涉及到的知识点极其多。大家如果有精力的话，最好可以花点时间把 Unicode 的发展史捋清楚了。本章节笔者能够做的是把相关的知识点告诉大家，让大家有一个全面的认识。

52.1 初学者重要提示

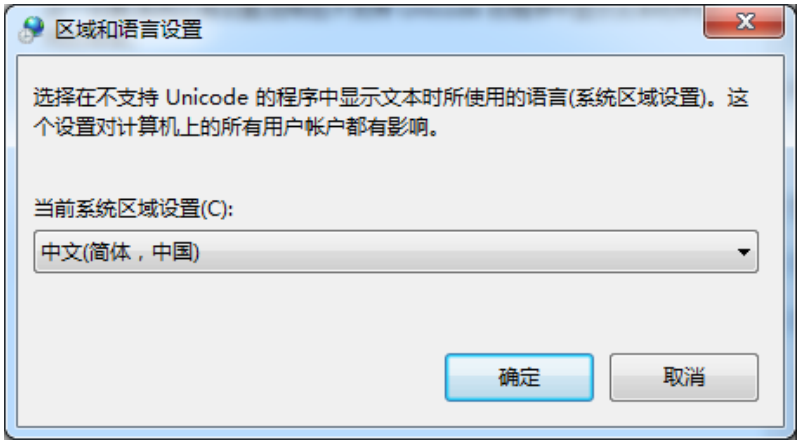
52.2 编码基础知识

52.3 汉字点阵原理

52.4 总结

52.1 初学者重要提示

- 本章节讲解的 GB2312 和 GBK 比较容易理解，而 Unicode 是全球统一码，所以涉及到的知识点比较多，也复杂些。
- 当前 Windows 系统字符管理已经全部采用 Unicode 字符集，编码形式是 UTF-16 LE （LE 表示小端格式）编码。比如我们通过记事本保存的时候选择编码类型 Unicode，其实就是选择的 UTF-16 LE。而我们平时所说的中文版 Windows 系统是指用户在选择不支持 Unicode 的程序中显示文本时所使用的语言。



- 区分清楚什么是字符，字符集。字符(Character)是各种文字和符号的总称，包括各国家文字、标点符号、图形符号、数字等。字符集(Character set)是多个字符的集合，字符集种类较多，每个字符集包

含的字符个数不同，常见字符集名称：ASCII 字符集、GB2312 字符集、BIG5 字符集、 GB18030 字符集、Unicode 字符集等。而 UTF-16，UTF-8 仅仅是 Unicode 字符集的编码形式，他们不是字符集，不要混淆了。

52.2 编码基础知识

关于字符编码的知识，我们这里换一种讲解方式，从字符编码的发展史来讲解，做到知其然知其所以然。

52.2.1 ASCII 字符集及其编码

ASCII 字符随着计算机的发明而诞生，计算机最早是在美国开始使用，当时为了将生活中用到的各种文字和符号能够通过计算机进行存储、显示和传输，人们就将控制字符、空格、标点符号、数字和大小写字母进行编号，从编号 0 到编号 127，共 128 个字符。到了 1967 年，专门为这个方案起了一个名字叫 ASCII（American Standard Code for Information Interchange，美国信息交换标准代码）。

下面是 128 个 ASCII 字符：

dec	oct	hex	ch	dec	oct	hex	ch	dec	oct	hex	ch	dec	oct	hex	ch
0	0	00	NUL (null)	32	40	20	(space)	64	100	40	@	96	140	60	`
1	1	01	SOH (start of header)	33	41	21	!	65	101	41	A	97	141	61	a
2	2	02	STX (start of text)	34	42	22	"	66	102	42	B	98	142	62	b
3	3	03	ETX (end of text)	35	43	23	#	67	103	43	C	99	143	63	c
4	4	04	EOT (end of transmission)	36	44	24	$	68	104	44	D	100	144	64	d
5	5	05	ENQ (enquiry)	37	45	25	%	69	105	45	E	101	145	65	e
6	6	06	ACK (acknowledge)	38	46	26	&	70	106	46	F	102	146	66	f
7	7	07	BEL (bell)	39	47	27	'	71	107	47	G	103	147	67	g
8	10	08	BS (backspace)	40	50	28	(	72	110	48	H	104	150	68	h
9	11	09	HT (horizontal tab)	41	51	29	)	73	111	49	I	105	151	69	i
10	12	0a	LF (line feed - new line)	42	52	2a	*	74	112	4a	J	106	152	6a	j
11	13	0b	VT (vertical tab)	43	53	2b	+	75	113	4b	K	107	153	6b	k
12	14	0c	FF (form feed - new page)	44	54	2c	,	76	114	4c	L	108	154	6c	l

13	15	0d	CR (carriage return)	45	55	2d	-	77	115	4d	M	109	155	6d	m
14	16	0e	SO (shift out)	46	56	2e	.	78	116	4e	N	110	156	6e	n
15	17	0f	SI (shift in)	47	57	2f	/	79	117	4f	O	111	157	6f	o
16	20	10	DLE (data link escape)	48	60	30	0	80	120	50	P	112	160	70	p
17	21	11	DC1 (device control 1)	49	61	31	1	81	121	51	Q	113	161	71	q
18	22	12	DC2 (device control 2)	50	62	32	2	82	122	52	R	114	162	72	r
19	23	13	DC3 (device control 3)	51	63	33	3	83	123	53	S	115	163	73	s
20	24	14	DC4 (device control 4)	52	64	34	4	84	124	54	T	116	164	74	t
21	25	15	NAK (negative acknowledge)	53	65	35	5	85	125	55	U	117	165	75	u
22	26	16	SYN (synchronous idle)	54	66	36	6	86	126	56	V	118	166	76	v
23	27	17	ETB (end of transmission block)	55	67	37	7	87	127	57	W	119	167	77	w
24	30	18	CAN (cancel)	56	70	38	8	88	130	58	X	120	170	78	x
25	31	19	EM (end of medium)	57	71	39	9	89	131	59	Y	121	171	79	y
26	32	1a	SUB (substitute)	58	72	3a	:	90	132	5a	Z	122	172	7a	z
27	33	1b	ESC (escape)	59	73	3b	;	91	133	5b	[	123	173	7b	{
28	34	1c	FS (file separator)	60	74	3c	<	92	134	5c	\	124	174	7c	|
29	35	1d	GS (group separator)	61	75	3d	=	93	135	5d	]	125	175	7d	}
30	36	1e	RS (record separator)	62	76	3e	>	94	136	5e	^	126	176	7e	~
31	37	1f	US (unit separator)	63	77	3f	?	95	137	5f	_	127	177	7f	DEL (delete)

随着计算机的发展，世界各地都开始使用计算机，但是很多国家用的不是英文，他们的字母里有很多是 ASCII 字符集中没有的。为了可以在计算机中保存这些非英文国家的文字和符号，决定采用 127 号之后未编排的空位来表示这些新的字母和符号，并且还加入了很多画表格时需要用到的横线、竖线、以及交叉线等形状，一直把序号编到了最后一个状态 255，从 128 到 255 的字符被称为扩展字符集。具体字符就不整理出来了，电脑端不太好打印出，大家可以看此网站 http://www.ascii-code.com/ 。

52.2.2 GB2312 字符集及其编码

等计算机传到中国来，一个字节（一个字节是 8 位，共可以表达 2^8 = 256 个字符）的所有编码已

经用完，没有可以使用的编码来表示汉字了，而且一个字节不能表示 6000 多个常用的汉字，所以汉字就需要 2 个字节来表示。

第一个字节从编号 0 到编号 127 的字符不变，还是表示 ASCII，而之后的 0xA1 到 0xFE 用于汉字编码，这个字节被称为汉字的区号或者高位字节，0xA1 到 0xFE 换算成区号就是从 01 区到 94 区（换算关系就是对编码值减去 0xA0）。

第二个字节的 0xA1 到 0xFE 用于汉字编码，这个字节被称为汉字的位号或者低位字节，0xA1 到 0xFE 换算成位号就是从位号 01 到位号 94（换算关系就是对编码值减去 0xA0）。根据区号和位号的设置，那么就有 94*94 = 8836 个编码可供使用。在这些编码里，我们还把数学符号、罗马希腊字母、日文的假名都编码进去了，连在 ASCII 里本来就有的数字、标点、字母都统统重新编了两个字节长的编码，即全角字符，而原来在 127 号之前的那些字符称为半角字符。

由这些编码组成的字符，我们为其取了一个名字叫 GB2312。GB2312 是对 ASCII 字符集的简体中文扩展，这个就是 GB2312 的由来。GB2312 编码全称《信息交换用汉字编码字符集——基本集》，由中国国家标准总局 1980 年发布，1981 年 5 月 1 日开始实施的一套国家标准，标准号是 GB 2312—1980。

GB2312 编码适用于汉字处理、汉字通信等系统之间的信息交换，通行于中国大陆，新加坡等地也采用此编码。中国大陆几乎所有的中文系统和国际化的软件都支持 GB2312。基本集共收入汉字 6763 个以及非汉字图形字符 682 个。整个字符集分成 94 个区，每区有 94 个位。每个区位上只有一个字符，因此可用所在的区和位来对汉字进行编码，称为区位码。其中，这 94 个区的分配如下：

- 01-09 区为特殊全角符号共 682 个。
- 16-55 区为一级汉字共 3755 个，按音序排。
- 56-87 区为二级汉字共 3008 个，按部首/笔画排序。
- 10-15 区和 88-94 区共 13 个区暂未编码。

区位码和汉字编码：

为了编码兼容 ASCII 码且机内存储不与 ASCII 码冲突，GB2312 并不直接使用 0x0101-0x5E5E(区位码)对字符进行表示和存储，而是使用实际的编码范围 0XA1A1 到 0XFEFE。例如“啊”字在大多数程序中，会以两个字节 0xB0A1 储存。而区位码就是 0x1001，他们之间的关系就是高位字节和低位字节都差 0xA0。0xB0 - 0x10 = 0xA0，0xA1 - 0x01 = 0xA0。

ASCII 区，GB2312 编码的全角字符区和汉字区的所有字符可以看下这个帖子：
http://bbs.armfly.com/read.php?tid=201 ，初学者务必看下，非常有必要。

特别补充：上面讲解的内容中隐藏了一个小知识，但是在我们做中英文混合显示时很有用，只要高字节超过 127(0x80)就认为是双字节编码，即 GB2312，而不用管低字节是否低于 127(0x80)。

52.2.3 GBK 字符集及其编码

随着计算机使用范围的扩大，发现很多的人名在 GB2312 编码中找不到，还有很多文字和符号将来可能还会用到，我国在 1995 年 12 月完成 GBK 规范，全称汉字内码扩展规范，共收入 21886 个汉字和图形符号，其中汉字（包括部首和构件）21003 个，图形符号 883 个。

GBK 编码也需要两个字节来表示，第一个字节是区号，范围是 81 到 FE，第二个字节是位号，位号是该字在该区中的位置，范围是 40 到 7E 和 80 到 FE，其中 7F 部分是没有编码的，这点要特别注意。下面是 GBK 的编码范围表。

范围	第 1 字节	第 2 字节	编码数	字数
GBK/1	A1 – A9	A1 - FE	846	717
GBK/2	B0 – F7	A1 - FE	6768	6763
GBK/3	81 – A0	40 – FE（7F 除外）	6080	6080
GBK/4	AA - FE	40 –A0（7F 除外）	8160	8160
GBK/5	A8 – A9	40 –A0（7F 除外）	192	166
用户定义	AA - AF	A1 - FE	564	
用户定义	F8 - FE	A1 - FE	658	
用户定义	A1 – A7	40 – A0（7F 除外）	672	
合计:			23940	21886

从上面表格可以看出，GBK 区间是 8140~FEFE，其中 XX7F 部分是要删除的， GBK 里有 FE-81+1=126 个区 ，每个区有 FE-40-1+1=190 个字符，因此一共有 126*190=23940 个字符，与上面的表格编码数是一致的。

特别补充，GBK 编码与 Unicode 字符集中的 UCS-2 编码完全兼容，这一点很重要，后面 emWin 的学习中要用到。

52.2.4 Unicode 字符集及其编码

由于各个国家都像中国这样搞出一套自己的编码标准，结果互相之间谁也不懂谁的编码，谁也不支持别人的编码。基于此，国际组织决定着手解决这个问题，即重新弄一套包括了地球上所有文化、所有字母和符号的编码。他们打算叫它“ Universal Coded Character Set”，简称 UCS，俗称 Unicode。Unicode 有两种编码 UCS-2 和 UCS-4。UCS-2 用两个字节编码，UCS-4 用 4 个字节编码。

Unicode 伴随着通用字符集的标准而发展，同时也以书本的形式对外发表。Unicode 至今仍在不断增修，每个新版本都加入更多新的字符。目前最新的版本为 2016 年 6 月 21 日公布的 9.0.0 版本，已经收入超过十万个字符（第十万个字符在 2005 年获采纳）。Unicode 涵盖的数据除了视觉上的字形、编码方法、标准的字符编码外，还包含了字符特性，如大小写字母。

关于 Unicode 这块，我们这里补充点知识，来自 wiki 百科中文版，非常值得一读，笔者这里将其整理到教程里面来了：

==========

大概来说，Unicode 编码系统可分为编码方式和实现方式两个层次。

◆ 编码方式

统一码的编码方式与 ISO 10646 的通用字符集概念相对应。目前实际应用的统一码版本对应于 UCS-2，使用 16 位的编码空间，也就是每个字符占用 2 个字节。这样理论上一共最多可以表示 2^16，即 65536 个字符，基本能满足各种语言的使用。实际上当前版本的统一码并未完全使用这 16 位编码，而是保留了大量空间以作为特殊使用或将来扩展。

上述 16 位统一码字符构成基本多文种平面。最新（但未实际广泛使用）的统一码版本定义了 16 个辅助平面，两者合起来至少需要占据 21 位的编码空间，比 3 字节略少。但事实上辅助平面字符仍然占用 4 字节编码空间，与 UCS-4 保持一致。未来版本会扩充到 ISO 10646-1 实现级别 3，即涵盖 UCS-4 的所有字符。UCS-4 是一个更大的尚未填充完全的 31 位字符集，加上恒为 0 的首位，共需占据 32 位，即 4 字节。理论上最多能表示 2^31 个字符，完全可以涵盖一切语言所用的符号。

基本多文种平面的字符的编码为 U+hhhh，其中每个 h 代表一个十六进制数字，与 UCS-2 编码完全相同。而其对应的 4 字节 UCS-4 编码后两个字节一致，前两个字节的所有位均为 0。

◆ 实现方式

Unicode 的实现方式不同于编码方式。一个字符的 Unicode 编码是确定的。但是在实际传输过程中，由于不同系统平台的设计不一定一致，以及出于节省空间的目的，对 Unicode 编码的实现方式有所不同。Unicode 的实现方式称为 Unicode 转换格式（Unicode Transformation Format，简称为 UTF）。

例如，一个仅包含基本 7 位 ASCII 字符的 Unicode 文件，如果每个字符都使用 2 字节的原 Unicode 编码传输，其第一字节的 8 位始终为 0。这就造成了比较大的浪费。对于这种情况，可以使用 UTF-8 编码，这是一种变长编码，它将基本 7 位 ASCII 字符仍用 7 位编码表示，占用一个字节（首位补 0）。而遇到与其他 Unicode 字符混合的情况，将按一定算法转换，每个字符使用 1-3 个字节编码，并利用首位为 0 或 1 进行识别。这样对以 7 位 ASCII 字符为主的西文文档就大幅节省了编码长度。类似的，对未来会出现的需要 4 个字节的辅助平面字符和其他 UCS-4 扩充字符，2 字节编码的 UTF-16 也需要通过一定的算法进行转换。

再如，直接使用与 Unicode 编码一致（仅限于 BMP 字符）的 UTF-16 编码，由于每个字符占用了两个字节，在 Mac OS 系统和 Windows 系统上，对字节顺序的理解是不一致的。这时同一字节流可能会被解释为不同内容，如某字符为十六进制编码 4E59，按两个字节拆分为 4E 和 59，在 Mac 上读取时是从低字节开始，那么在 Mac OS 会认为此 4E59 编码为 594E，找到的字符为“奎”，而在 Windows 上从高字节开始读取，则编码为 U+4E59 的字符为“乙”。就是说在 Windows 下以 UTF-16 编码保存一个字符“乙”，在 Mac OS 环境下打开会显示成“奎”。此类情况说明 UTF-16 的编码顺序若不加以人为定义就可能发生混淆，于是在 UTF-16 编码实现方式中使用了大端序（Big-Endian，简写为 UTF-16 BE）、小端序

（Little-Endian，简写为 UTF-16 LE）的概念，以及可附加的字节顺序记号解决方案，目前在 PC 机上的 Windows 系统和 Linux 系统对于 UTF-16 编码默认使用 UTF-16 LE。

此外 Unicode 的实现方式还包括 UTF-7、Punycode、CESU-8、SCSU、UTF-32、GB18030 等，这些实现方式有些仅在一定的国家和地区使用，有些则属于未来的规划方式。目前通用的实现方式是 UTF-16 小端序(LE)、UTF-16 大端序(BE)和 UTF-8。在微软公司 Windows XP 附带的记事本(Notepad)中，“另存为”对话框可以选择的四种编码方式除去非 Unicode 编码的 ANSI（对于英文系统即 ASCII 编码，中文系统则为 GB2312 或 Big5 编码）外，其余三种为“Unicode”（对应 UTF-16 LE）、“Unicode big endian”（对应 UTF-16 BE）和“UTF-8”。

目前辅助平面的工作主要集中在第二和第三平面的中日韩统一表意文字中，因此包括 GBK、GB18030、Big5 等简体中文、繁体中文、日文、韩文以及越南喃字的各种编码与 Unicode 的协调性被重点关注。考虑到 Unicode 最终要涵盖所有的字符，从某种意义而言，这些编码方式也可视作 Unicode 出现之前的实现方式，如同 ASCII 及其扩展 Latin-1 一样，后两者的字符在 16 位 Unicode 编码空间中的编码第一字节各位全为 0，第二字节编码与原编码完全一致。但上述东亚语言编码与 Unicode 编码的对应关系要复杂得多。

==========

更多 Unicode 相关知识，大家可以学习如下四个地址：

官网：http://www.unicode.org/

百度百科：超链接地址

wiki 百科中文版：https://zh.wikipedia.org/wiki/Unicode

wiki 百科英文版（更详细些）：https://en.wikipedia.org/wiki/Unicode

更多 UTF-8 编码相关知识，大家可以学习如下三个地址：

百度百科：超链接地址

wiki 百科中文版：https://zh.wikipedia.org/wiki/UTF-8

wiki 百科英文版（更详细些）： https://en.wikipedia.org/wiki/UTF-8

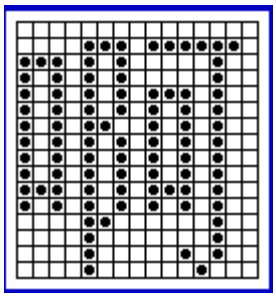
52.3 汉字点阵原理

简单的理解，一个字的点阵是指这个汉字用多少个像素点来描述。下面举一个 16 点阵汉字库（宽度 x 高度 = 16x16）给大家说明一下：

16 点阵每个字型的点阵数据为 16×16 (横行点数×纵列点数)，共 256 个二进制位，32 个字节。汉字 16 点阵字型数据的 32 个字节排列次序是以 0 字节开始到 31 字节结束，均用十六进制表示，其记录格式如下：

列数 / 行数	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0 字节								1 字节							
1	:								:							
2																
3	:								:							
4																
:	:								:							
:																
15	30 字节								31 字节							

以一个点阵汉字“啊”为例给大家说明下：



“啊”是汉字区第 1 个汉字，对应的数据为：

0x00，0x00，0x0E，0xFC，0xEA，0x08，0xAA，0x08，
0xAA，0xE8，0xAA，0xA8，0xAC，0xA8，0xAA，0xA8，
0xAA，0xA8，0xAA，0xA8，0xEA，0xE8，0xAA，0xA8，
0x0C，0x08，0x08，0x08，0x08，0x28，0x08，0x10

大家明白了点阵字库的排列原理，编程实现汉字显示就比较容易了，后面章节的汉字显示要用到。

52.4 总结

本章节主要把大家后面章节要用到的知识点进行了说明，其实字符编码这块涉及到的知识点还是非常多的，如果想完全弄明白还要花一番心思和时间去查阅相关资料。

第53章 STM32H7 的 LTDC 应用之汉字小字库和全字库制作

本章教程为大家讲解汉字小字库和全字库的制作方式，实际项目中用到的地方比较多。

53.1 初学者重要提示
53.2 使用 MakeDot 小软件生成 C 文件格式小字库方法
53.3 使用 MakeDot 小软件生成 C 文件格式全字库方法
53.4 C 文件格式汉字使用方法
53.5 汉字显示方法解析
53.6 LCD 驱动移植和使用
53.7 实验例程设计框架
53.8 实验例程说明（MDK）
53.9 实验例程说明（IAR）
5.10 总结

53.1 初学者重要提示

- 学习本章节前，务必优先学习第 52 章，需要对点阵字体字符编码有个认识。
- LTDC 驱动设计和相关问题在第 51 章有详细说明。
- 本章节为大家讲解的小字库和全字库方法，简单易用，是直接以 C 文件格式存储到内部 Flash 的。支持 12 点阵，16 点阵，24 点阵和 32 点阵的 ASCII 以及 GB2312 编码汉字显示。

53.2 使用 MakeDot 小软件生成 C 文件格式小字库方法

生成方法比较简单，这里做个介绍：

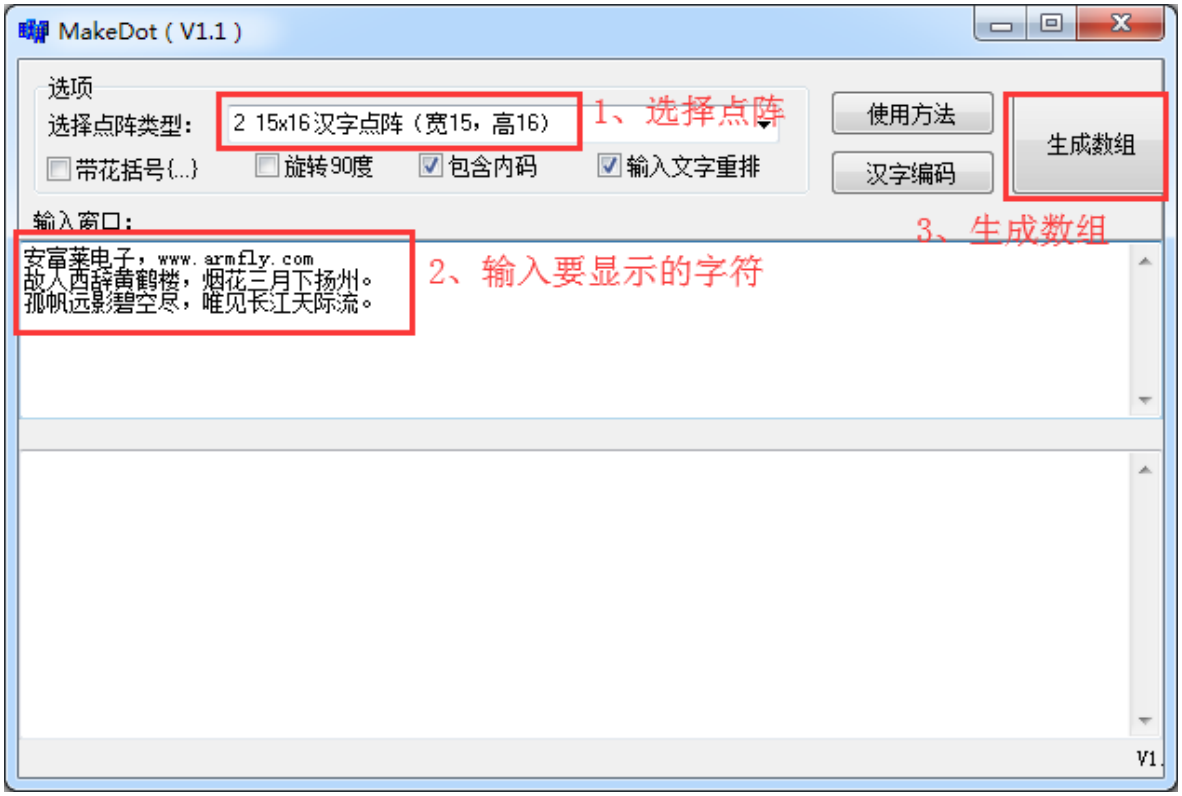
53.2.1 第 1 步，准备好显示的字符

比如要显示如下字符，采用 16 点阵格式：

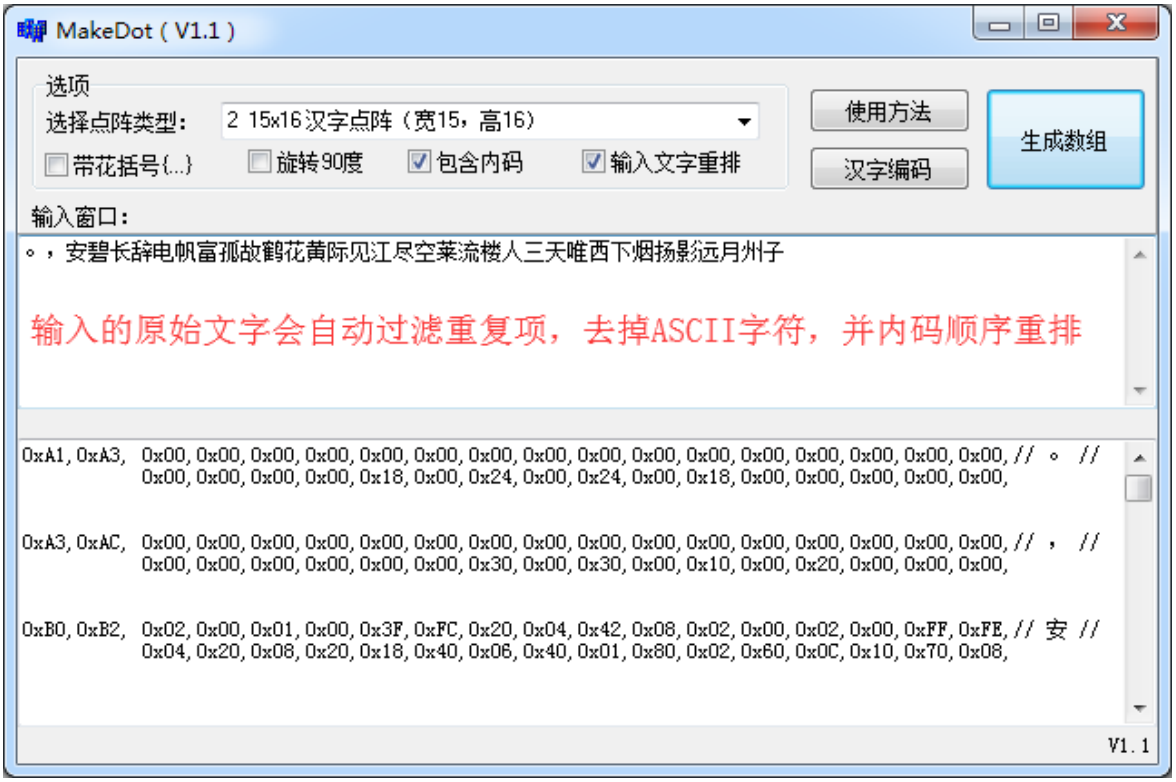
```
安富莱电子，www.armfly.com
故人西辞黄鹤楼，烟花三月下扬州。
孤帆远影碧空尽，唯见长江天际流。
```

53.2.2第 2 步，复制要显示的字符到 MakeDot 小软件

选择 16 点阵，并将要显示的字符复制到输入窗口：

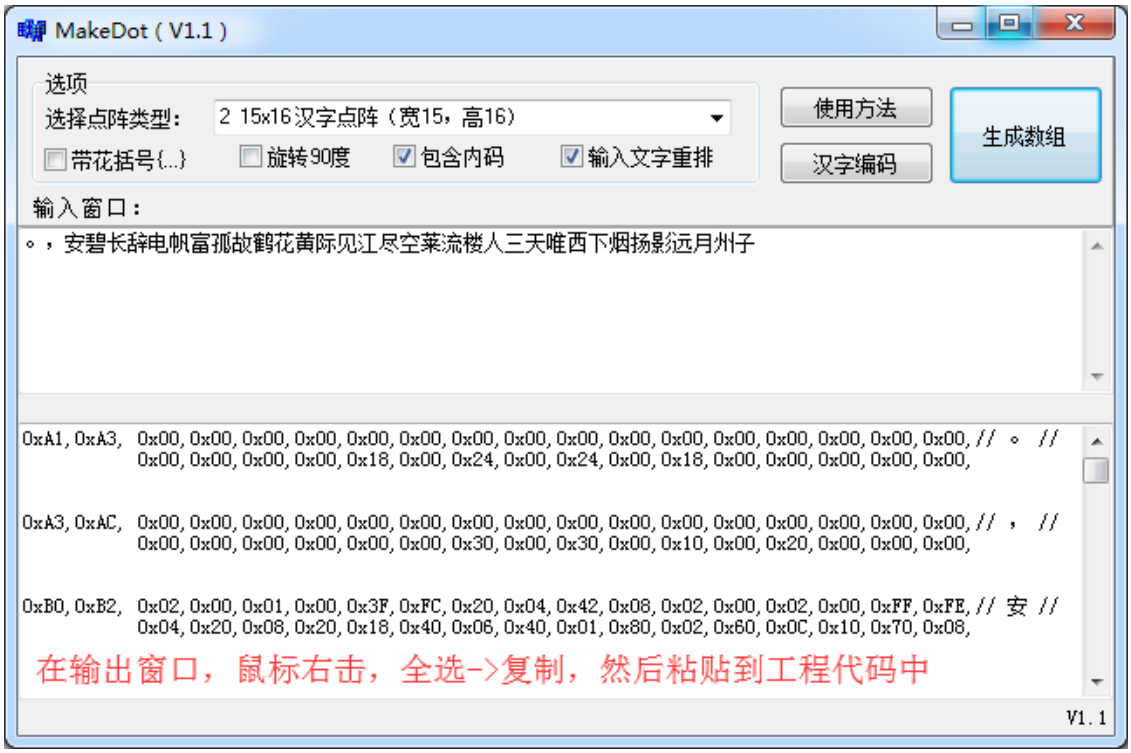


点击生成数组按钮后的效果如下：

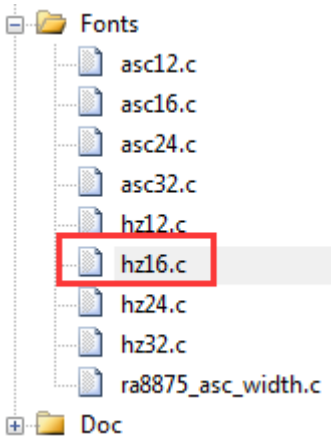


53.2.3第 3 步，复制生成的数组到工程中

在输出窗口鼠标右击，选择“全选”，然后再次鼠标右击选择复制。



这样就可以粘贴到工程的 hz.c 文件里面：



将点阵数据放在相应的文件里面时要注意加上两个 0XFF。hz.c 文件的内容如下：

```
/*
    FLASH 中内嵌小字库，只包括本程序用到的汉字点阵
    每行点阵数据，头 2 字节是汉子的内码，后面是 16 点阵汉子的字模数据。
*/

#ifdef USE_SMALL_FONT

unsigned char const g_Hz16[] = {

0xA1,0xA3,  0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00, // 。 //
```

```
            0x00,0x00,0x00,0x00,0x18,0x00,0x24,0x00,0x24,0x00,0x18,0x00,0x00,0x00,0x00,0x00,


0xA3,0xAC,  0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,// ，  //
            0x00,0x00,0x00,0x00,0x00,0x00,0x30,0x00,0x30,0x00,0x10,0x00,0x20,0x00,0x00,0x00,


0xB0,0xB2,  0x02,0x00,0x01,0x00,0x3F,0xFC,0x20,0x04,0x42,0x08,0x02,0x00,0x02,0x00,0xFF,0xFE,// 安 //
            0x04,0x20,0x08,0x20,0x18,0x40,0x06,0x40,0x01,0x80,0x02,0x60,0x0C,0x10,0x70,0x08,

 /* 中间部分省略未写 */

0xD7,0xD3,  0x00,0x00,0x7F,0xF8,0x00,0x10,0x00,0x20,0x00,0x40,0x01,0x80,0x01,0x00,0xFF,0xFE,// 子 //
            0x01,0x00,0x01,0x00,0x01,0x00,0x01,0x00,0x01,0x00,0x01,0x00,0x05,0x00,0x02,0x00,


/* 最后一行必须用 0xFF,0xFF 结束，这是字库数组结束标志 */
0xFF,0xFF

};

#else
    unsigned char const g_Hz16[] = {0xFF, 0xFF};
#endif
```

添加完毕点阵数据后，在 font.h 文件里面使能使用小字库：

#define USE_SMALL_FONT　　/* 定义此行表示使用小字库， 这个宏只在 bsp_tft+lcd.c 中使用 */

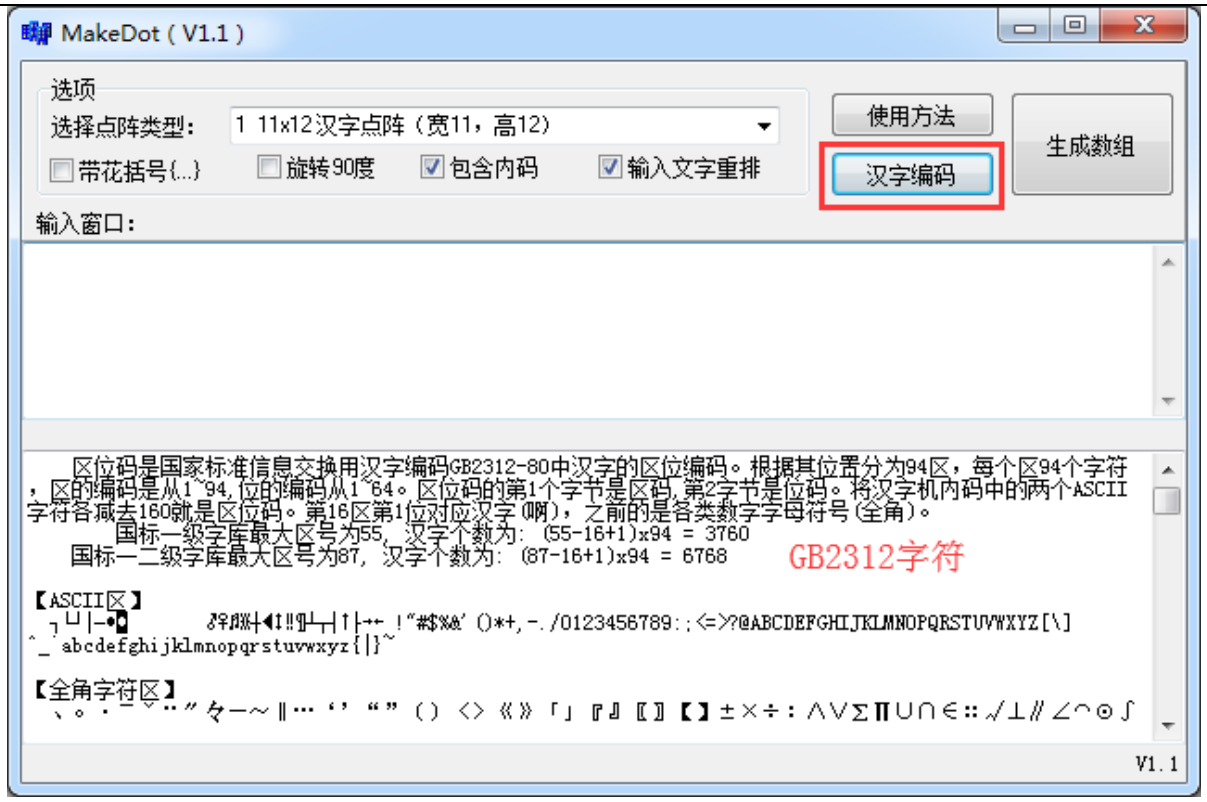
至此就完成了小字库的汉字添加，用户就可以在使用 16 点阵时显示第 1 步中转换的字符了。

53.3 使用 MakeDot 小软件生成 C 文件格式全字库方法

生成方法比较简单，这里做个介绍：

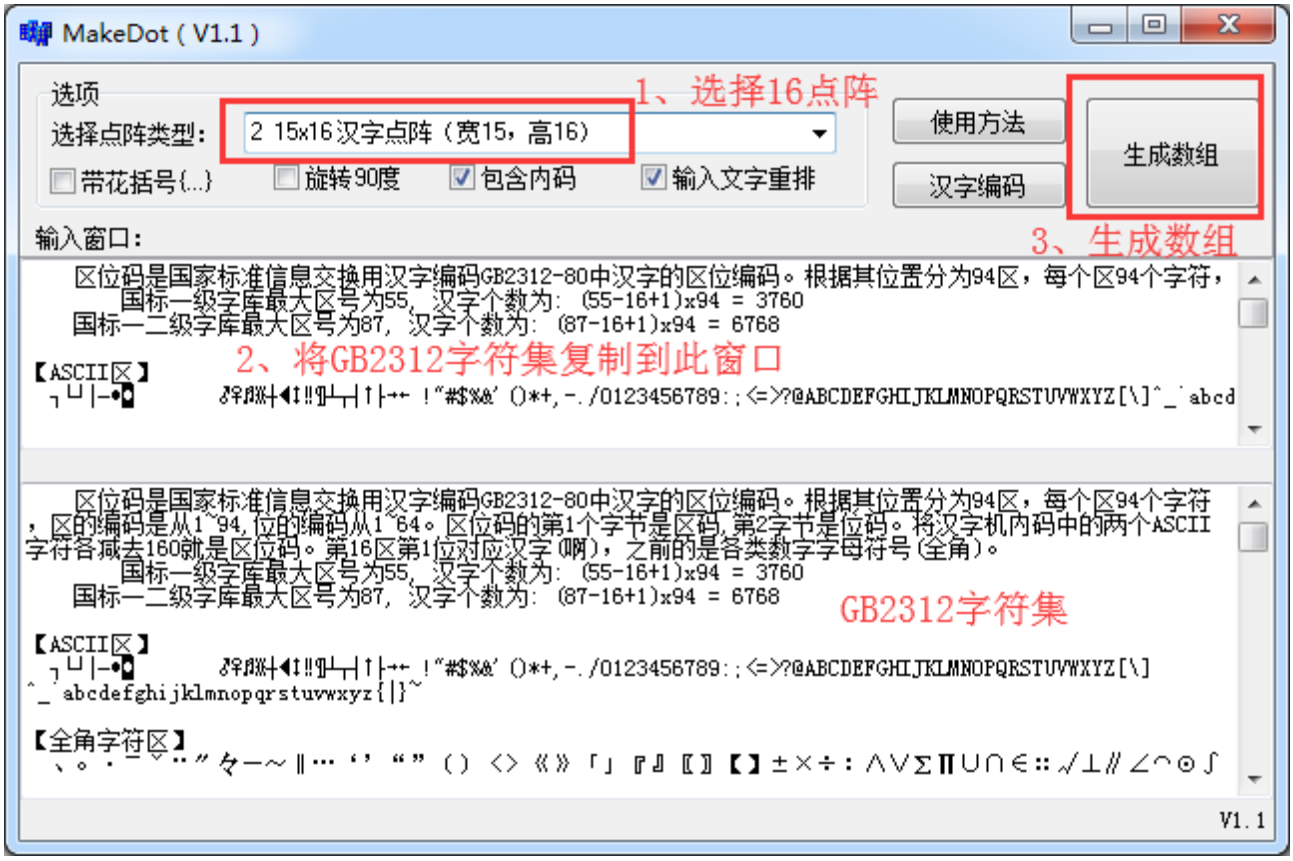
53.3.1 第 1 步，准备好 GB2312 字符集

GB2312 字符集已经在 MakeDot 小软件里面存好，点击汉字编码按钮可以看到：

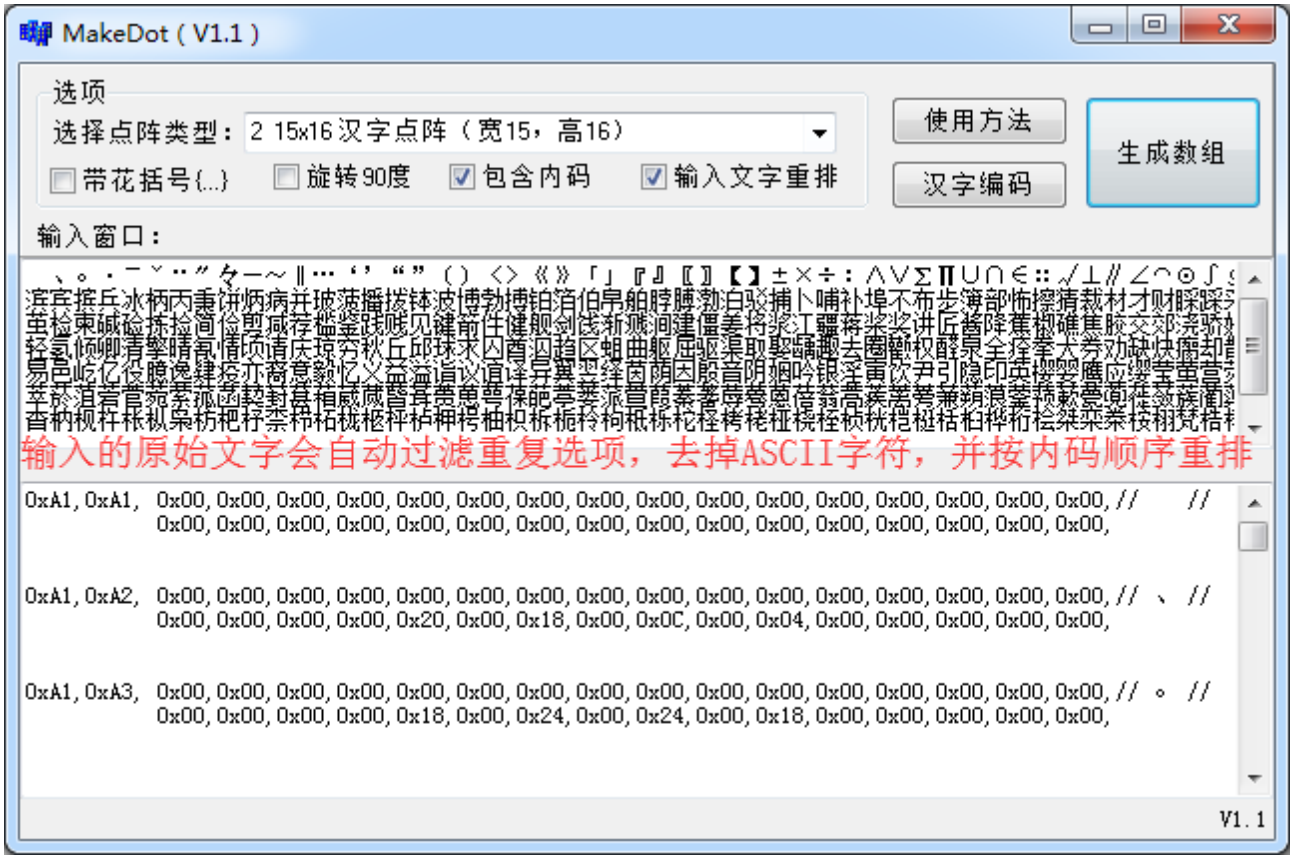


53.3.2第 2 步，复制 GB2312 全部字符到 MakeDot 小软件

复制 MakeDot 小软件中 GB2312 所有字符到“输入窗口区”（在 GB2312 字符显示区，鼠标右击选择全选，之后就可以复制了），

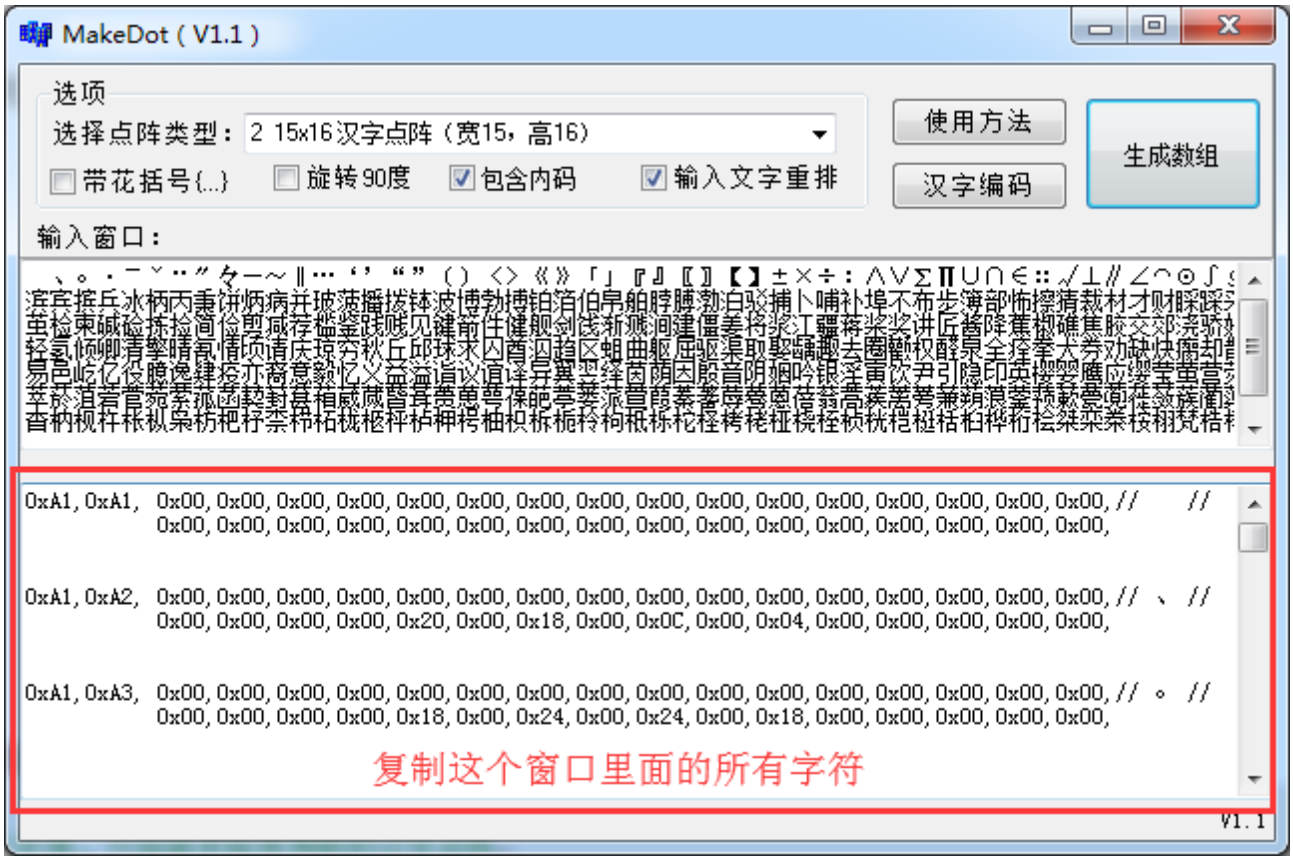


点击生成数组按钮后的效果如下：

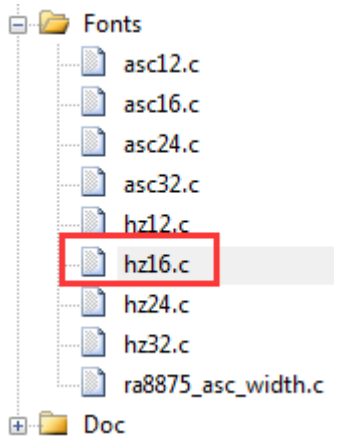


53.3.3第 3 步，复制生成的数组到工程中

在输出窗口鼠标右击，选择“全选”，然后再次鼠标右击选择复制。



这样就可以粘贴到工程的 hz.c 文件里面：



将点阵数据放在相应的文件里面时要注意加上两个 0XFF。hz.c 文件的内容如下：

```
/*
    FLASH 中内嵌小字库，只包括本程序用到的汉字点阵
    每行点阵数据，头 2 字节是汉子的内码，后面是 16 点阵汉子的字模数据。
*/

#ifdef USE_SMALL_FONT
```

```
unsigned char const g_Hz16[] = {



0xA1,0xA1, 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,//    //
           0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,


0xA1,0xA2, 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,// 、 //
           0x00,0x00,0x00,0x00,0x20,0x00,0x18,0x00,0x0C,0x00,0x04,0x00,0x00,0x00,0x00,0x00,0x00,0x00,


0xA1,0xA3, 0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,// 。 //
           0x00,0x00,0x00,0x00,0x18,0x00,0x24,0x00,0x24,0x00,0x18,0x00,0x00,0x00,0x00,0x00,0x00,0x00,

 /* 中间部分省略未写 */

0xF7,0xFB, 0x20,0x0E,0xCE,0xF0,0x82,0x22,0xEE,0x92,0x82,0x44,0x82,0x20,0xFE,0x44,0x00,0xF8,// 龖 //
           0x92,0x10,0x92,0x24,0xDA,0xFE,0x92,0x10,0xDA,0xFE,0x92,0x28,0x93,0x44,0xD9,0x82,


0xF7,0xFC, 0x10,0x20,0x3E,0x20,0x22,0x20,0x3E,0x20,0x22,0xF8,0x3E,0x28,0x00,0x28,0x7F,0x28,// 鼹 //
           0x49,0x28,0x7F,0x28,0x49,0x28,0x7F,0x2A,0x00,0x2A,0xFF,0xCA,0x22,0x46,0x42,0x80,


0xF7,0xFD, 0x10,0x00,0x3E,0x00,0x22,0x7C,0x3E,0x10,0x22,0x10,0x3E,0x10,0x00,0x10,0x7F,0x10,// 鼾 //
           0x49,0xFE,0x7F,0x10,0x49,0x10,0x7F,0x10,0x00,0x10,0xFF,0x90,0x22,0x10,0x42,0x10,


0xF7,0xFE, 0x10,0x10,0x3E,0x10,0x22,0xFE,0x3E,0x38,0x22,0x54,0x3E,0x92,0x00,0x00,0x7F,0x7C,// 齾 //
           0x49,0x44,0x7F,0x7C,0x49,0x44,0x7F,0x7C,0x00,0x44,0xFF,0x80,0x22,0xFE,0x42,0x00,


/* 最后一行必须用 0xFF,0xFF 结束，这是字库数组结束标志 */
0xFF,0xFF

};

#else
    unsigned char const g_Hz16[] = {0xFF, 0xFF};
#endif
```

添加完毕点阵数据后，在 font.h 文件里面使能宏定义：

#define USE_SMALL_FONT　　/*这个宏只在 bsp_tft+lcd.c 中使用 */

至此就完成了全字库的汉字添加，用户就可以使用 16 点阵的汉字了。

53.4C 文件格式汉字使用方法

汉字的显示方法比较简单。

- 定义一个 FONT_T 类型变量：

```
FONT_T tFont12;         /* 定义一个 12 点阵字体结构体变量，用于设置字体参数 */
FONT_T tFont16;         /* 定义一个 16 点阵字体结构体变量，用于设置字体参数 */
FONT_T tFont24;         /* 定义一个 24 点阵字体结构体变量，用于设置字体参数 */
```

FONT_T 的原始定义如下：

```
typedef struct
{
    FONT_CODE_E FontCode;  /* 字体代码 FONT_CODE_E  */
    uint16_t FrontColor;   /* 字体颜色 */
    uint16_t BackColor;    /* 文字背景颜色，透明 */
    uint16_t Space;        /* 文字间距，单位 = 像素 */
}FONT_T;
```

◆ 初始化变量 tFont：

设置 12，16 和 24 点阵。

```
/* 设置字体属性 */
tFont.FontCode = FC_ST_12;       /* 字体选择宋体 12 点阵，高 12 x 宽 11) */
tFont.FrontColor = CL_WHITE;     /* 字体颜色设置为白色 */
tFont.BackColor = CL_MASK;       /* 文字背景颜色，透明 */
tFont.Space = 0;                 /* 字符水平间距，单位 = 像素 */

tFont.FontCode = FC_ST_16;       /* 字体选择宋体 16 点阵，高 16 x 宽 15) */
tFont.FrontColor = CL_WHITE;     /* 字体颜色设置为白色 */
tFont.BackColor = CL_MASK;       /* 文字背景颜色，透明 */
tFont.Space = 0;                 /* 字符水平间距，单位 = 像素 */

tFont.FontCode = FC_ST_24;       /* 字体选择宋体 24 点阵，高 24 x 宽 23) */
tFont.FrontColor = CL_WHITE;     /* 字体颜色设置为白色 */
tFont.BackColor = CL_MASK;       /* 文字背景颜色，透明 */
tFont.Space = 0;                 /* 字符水平间距，单位 = 像素 */
```

◆ 调用函数 LCD_DispStr 显示字符：

下面显示了 12,16 和 24 点阵字符。

```
LCD_DispStr(5, 3,  "故人西辞黄鹤楼，烟花三月下扬州。www.armfly.com", &tFont12);
LCD_DispStr(5, 20, "孤帆远影碧空尽，唯见长江天际流。www.armfly.com", &tFont12);
LCD_DispStr(5, 38, "故人西辞黄鹤楼，烟花三月下扬州。", &tFont16);
LCD_DispStr(5, 68, "孤帆远影碧空尽，唯见长江天际流。", &tFont16);
LCD_DispStr(5, 98, "故人西辞黄鹤楼烟花三月下扬州", &tFont24);
LCD_DispStr(5, 128, "孤帆远影碧空尽唯见长江天际流", &tFont24);
```

53.5 汉字显示方法解析

下面将汉字的显示流程做个说明，几个函数的调用关系如下：

LCD_DispStr ----> LCD_DispStrEx ----->_LCD_ReadAsciiDot

_LCD_ReadHZDot

53.5.1 函数 LCD_DispStr

中英文显示都是调用的如下函数实现：

```
/*
*********************************************************************************************************
*    函 数 名: LCD_DispStr
*    功能说明: 在 LCD 指定坐标（左上角）显示一个字符串
*    形    参:
```

```
*        _usX : X 坐标
*        _usY : Y 坐标
*        _ptr  : 字符串指针
*        _tFont : 字体结构体，包含颜色、背景色(支持透明)、字体代码、文字间距等参数
*    返 回 值: 无
*********************************************************************************************************
*/
void LCD_DispStr(uint16_t _usX, uint16_t _usY, char *_ptr, FONT_T *_tFont)
{
    LCD_DispStrEx(_usX, _usY, _ptr, _tFont, 0, 0);
}
```

这个函数的注释已经比较详细，这里就不再赘述了。而这个函数是通过调用 LCD_DispStrEx 实现。

53.5.2 函数 LCD_DispStrEx

此函数的源码如下：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: LCD_DispStrEx
4.  *    功能说明: 在 LCD 指定坐标（左上角）显示一个字符串。 增强型函数。支持左\中\右对齐，支持定长清屏。
5.  *    形    参:
6.  *        _usX : X 坐标
7.  *        _usY : Y 坐标
8.  *        _ptr  : 字符串指针
9.  *        _tFont : 字体结构体，包含颜色、背景色(支持透明)、字体代码、文字间距等参数。可以指定 RA8875 字库
10. *                 显示汉字。
11. *        _Width : 字符串显示区域的宽度. 0 表示不处理留白区域，此时_Align 无效
12. *        _Align :字符串在显示区域的对齐方式，
13. *                 ALIGN_LEFT = 0,
14. *                 ALIGN_CENTER = 1,
15. *                 ALIGN_RIGHT = 2
16. *    返 回 值: 无
17. *********************************************************************************************************
18. */
19. void LCD_DispStrEx(uint16_t _usX, uint16_t _usY, char *_ptr, FONT_T *_tFont, uint16_t _Width,
20.     uint8_t _Align)
21. {
22.     uint32_t i;
23.     uint8_t code1;
24.     uint8_t code2;
25.     uint8_t buf[32 * 32 / 8];   /* 最大支持 32 点阵汉字 */
26.     uint8_t width;
27.     uint16_t m;
28.     uint8_t font_width = 0;
29.     uint8_t font_height = 0;
30.     uint16_t x, y;
31.     uint16_t offset;
32.     uint16_t str_width;     /* 字符串实际宽度  */
33.
34.     switch (_tFont->FontCode)
35.     {
36.         case FC_ST_12:          /* 12 点阵 */
37.             font_height = 12;
38.             font_width = 12;
```

```
39.                 break;
40.
41.             case FC_ST_16:
42.                 font_height = 16;
43.                 font_width = 16;
44.                 break;
45.
46.             case FC_ST_24:
47.                 font_height = 24;
48.                 font_width = 24;
49.                 break;
50.
51.             case FC_ST_32:
52.                 font_height = 32;
53.                 font_width = 32;
54.                 break;
55.         }
56.
57.         str_width = LCD_GetStrWidth(_ptr, _tFont);/* 计算字符串实际宽度(RA8875 内部 ASCII 点阵宽度为变长 */
58.         offset = 0;
59.         if (_Width > str_width)
60.         {
61.             if (_Align == ALIGN_RIGHT)  /* 右对齐 */
62.             {
63.                 offset = _Width - str_width;
64.             }
65.             else if (_Align == ALIGN_CENTER) /* 居中 */
66.             {
67.                 offset = (_Width - str_width) / 2;
68.             }
69.             else /* 左对齐 ALIGN_LEFT */
70.             {
71.                 ;
72.             }
73.         }
74.
75.         /* 左侧填背景色, 中间对齐和右边对齐  */
76.         if (offset > 0)
77.         {
78.             LCD_Fill_Rect(_usX, _usY, LCD_GetFontHeight(_tFont), offset,  _tFont->BackColor);
79.             _usX += offset;
80.         }
81.
82.         /* 右侧填背景色 */
83.         if (_Width > str_width)
84.         {
85.             LCD_Fill_Rect(_usX + str_width, _usY, LCD_GetFontHeight(_tFont), _Width - str_width - offset,
86.                           _tFont->BackColor);
87.         }
88.
89.         /* 使用 CPU 内部字库. 点阵信息由 CPU 读取 */
90.         {
91.             /* 开始循环处理字符 */
92.             while (*_ptr != 0)
93.             {
94.                 code1 = *_ptr;     /* 读取字符串数据,  该数据可能是 ascii 代码, 也可能汉字代码的高字节 */
95.                 if (code1 < 0x80)
96.                 {
```

```
97.                     /* 将 ascii 字符点阵复制到 buf */
98.                     //memcpy(buf, &pAscDot[code1 * (font_bytes / 2)], (font_bytes / 2));
99.                     _LCD_ReadAsciiDot(code1, _tFont->FontCode, buf);    /* 读取 ASCII 字符点阵 */
100.                    width = font_width / 2;
101.                }
102.                else
103.                {
104.                    code2 = *++_ptr;
105.                    if (code2 == 0)
106.                    {
107.                        break;
108.                    }
109.                    /* 读 1 个汉字的点阵 */
110.                    _LCD_ReadHZDot(code1, code2, _tFont->FontCode, buf);
111.                    width = font_width;
112.                }
113.
114.                y = _usY;
115.                /* 开始刷 LCD */
116.                for (m = 0; m < font_height; m++)     /* 字符高度 */
117.                {
118.                    x = _usX;
119.                    for (i = 0; i < width; i++) /* 字符宽度 */
120.                    {
121.                        if ((buf[m * ((2 * width) / font_width) + i / 8] & (0x80 >> (i % 8 ))) != 0x00)
122.                        {
123.                            LCD_PutPixel(x, y, _tFont->FrontColor);    /* 设置像素颜色为文字色 */
124.                        }
125.                        else
126.                        {
127.                            if (_tFont->BackColor != CL_MASK)     /* 透明色 */
128.                            {
129.                                LCD_PutPixel(x, y, _tFont->BackColor);/* 设置像素颜色为文字背景色 */
130.                            }
131.                        }
132.
133.                        x++;
134.                    }
135.                    y++;
136.                }
137.
138.                if (_tFont->Space > 0)
139.                {
140.                    /* 如果文字底色按_tFont->usBackColor，并且字间距大于点阵的宽度，那么需要在文字之间填
141.                       充(暂时未实现) */
142.                }
143.                _usX += width + _tFont->Space;   /* 列地址递增 */
144.                _ptr++;             /* 指向下一个字符 */
145.            }
146.        }
147. }
```

下面将代码中几个关键地方做个阐释：

- 第 34-55 行，根据使用的的 12，16，24 和 32 点阵字体，设置字体的高度变量 font_height 和宽度变量 font_width。
- 第 57-73 行，通过函数 LCD_GetStrWidth 计算字符串的长度，然后根据设置的字符总宽度和实际宽

度做比较，来实现左对齐，右对齐和居中显示。由于函数 LCD_DispStr 调用 LCD_DispStrEx 时，将形参_Width 设置为 0，所以这部分代码功能未用到。

- 第 76-87 行，用于填充显示字符以外区域的背景色，只有设置的字符总宽度大于实际宽度时才会用到，填充的就是实际宽度以外的区域。
- 第 90-146 行，显示所有字符。
  - 第 95 行，如果编码值小于 0x80，表示 ASCII 字符。
  - 第 99 行，根据编码值读取 ASCII 值对应的点阵数据到数组 buf 里面。
  - 第 100 行，显示 ASCII 字符仅需要一半宽度即可，比如显示 12*12 点阵字符，显示成 ASCII 仅需 6*12 即可。
  - 第 102 行，如果编码值大于等于 0x80，汉字编码在这个范围。
  - 第 104 行，因为 GB 编码需要两个字节表示，所以这里再读取一个字节。
  - 第 110 行，根据汉字编码值对应的点阵数据到数组 buf 里面。
  - 第 116-136 行，采用从左到右，从上到下的方式刷新字符。这里特别注意点阵数据位置的获取：
    buf[m * ((2 * width) / font_width) + i / 8] & (0x80 >> (i % 8)
    对于这个公式，大家通过代数法，代入两次数值就好理解了。

53.5.3 函数_LCD_ReadAsciiDot

此函数的作用是根据 ASCII 编码值，读取对应的点阵数据出来。

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: _LCD_ReadAsciiDot
4.  *    功能说明: 读取 1 个 ASCII 字符的点阵数据
5.  *    形    参:
6.  *          _code : ASCII 字符的编码，1 字节。1-128
7.  *          _fontcode ：字体代码
8.  *          _pBuf : 存放读出的字符点阵数据
9.  *    返 回 值: 文字宽度
10. *********************************************************************************************************
11. */
12. static void _LCD_ReadAsciiDot(uint8_t _code, uint8_t _fontcode, uint8_t *_pBuf)
13. {
14.     const uint8_t *pAscDot;
15.     uint8_t font_bytes = 0;
16.
17.     pAscDot = 0;
18.     switch (_fontcode)
19.     {
20.         case FC_ST_12:          /* 12 点阵 */
21.             font_bytes = 24;
22.             pAscDot = g_Ascii12;
23.             break;
24.
25.         case FC_ST_24:
26.         case FC_ST_32:
```

```
27.             case FC_ST_16:
28.                 /* 缺省是 16 点阵 */
29.                 font_bytes = 32;
30.                 pAscDot = g_Ascii16;
31.                 break;
32.
33.             case FC_RA8875_16:
34.             case FC_RA8875_24:
35.             case FC_RA8875_32:
36.                 return;
37.         }
38.
39.         /* 将 CPU 内部 Flash 中的 ascii 字符点阵复制到 buf */
40.         memcpy(_pBuf, &pAscDot[_code * (font_bytes / 2)], (font_bytes / 2));
41.     }
```

下面将此函数涉及到的知识点为大家做个阐释：

- 第 20-23 行，显示 12 点阵 ASCII，每个字符需要 24 个字节，存储在数组 g_Ascii12 里面。
- 第 25-31 行，显示 16，24 和 32 点阵 ASCII，这里采用同一大小字符进行显示。每个字符需要 32 个字节，存储在数组 g_Ascii16 里面。
- 第 33-35 行，这个是 RA8875 的字库处理，V7 开发板用不到。
- 第 40 行，将 ASCII 点阵数据复制到缓冲_pBuf 里面。

53.5.4函数_LCD_ReadHZDot

此函数的作用是根据 ASCII 编码值，读取对应的点阵数据出来。

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: _LCD_ReadHZDot
4.  *    功能说明: 读取 1 个汉字的点阵数据
5.  *    形    参:
6.  *        _code1, _cod2 : 汉字内码. GB2312 编码
7.  *        _fontcode : 字体代码
8.  *        _pBuf : 存放读出的字符点阵数据
9.  *    返 回 值: 无
10. *********************************************************************************************************
11. */
12. static void _LCD_ReadHZDot(uint8_t _code1, uint8_t _code2,  uint8_t _fontcode, uint8_t *_pBuf)
13. {
14.     #ifdef USE_SMALL_FONT  /* 使用 CPU 内部 Flash 小字库 */
15.         uint8_t *pDot;
16.         uint8_t font_bytes = 0;
17.         uint32_t address;
18.         uint16_t m;
19.
20.         pDot = 0;/* 仅仅用于避免告警 */
21.         switch (_fontcode)
22.         {
23.             case FC_ST_12:          /* 12 点阵 */
24.                 font_bytes = 24;
25.                 pDot = (uint8_t *)g_Hz12;
26.                 break;
```

```
27.
28.                 case FC_ST_16:
29.                     font_bytes = 32;
30.                     pDot = (uint8_t *)g_Hz16;
31.                     break;
32.
33.                 case FC_ST_24:
34.                     font_bytes = 72;
35.                     pDot = (uint8_t *)g_Hz24;
36.                     break;
37.
38.                 case FC_ST_32:
39.                     font_bytes = 128;
40.                     pDot = (uint8_t *)g_Hz32;
41.                     break;
42.
43.                 case FC_RA8875_16:
44.                 case FC_RA8875_24:
45.                 case FC_RA8875_32:
46.                     return;
47.             }
48.
49.             m = 0;
50.             while(1)
51.             {
52.                 address = m * (font_bytes + 2);
53.                 m++;
54.                 if ((_code1 == pDot[address + 0]) && (_code2 == pDot[address + 1]))
55.                 {
56.                     address += 2;
57.                     memcpy(_pBuf, &pDot[address], font_bytes);
58.                     break;
59.                 }
60.                 else if ((pDot[address + 0] == 0xFF) && (pDot[address + 1] == 0xFF))
61.                 {
62.                     /* 字库搜索完毕，未找到，则填充全 FF */
63.                     memset(_pBuf, 0xFF, font_bytes);
64.                     break;
65.                 }
66.             }
67.         #else    /* 用全字库 */
68.             uint8_t *pDot = 0;
69.             uint8_t font_bytes = 0;
70.
71.             switch (_fontcode)
72.             {
73.                 case FC_ST_12:          /* 12 点阵 */
74.                     font_bytes = 24;
75.                     pDot = (uint8_t *)HZK12_ADDR;
76.                     break;
77.
78.                 case FC_ST_16:
79.                     font_bytes = 32;
80.                     pDot = (uint8_t *)HZK16_ADDR;
81.                     break;
82.
83.                 case FC_ST_24:
84.                     font_bytes = 72;
```

```
85.                     pDot = (uint8_t *)HZK24_ADDR;
86.                     break;
87.
88.                 case FC_ST_32:
89.                     font_bytes = 128;
90.                     pDot = (uint8_t *)HZK32_ADDR;
91.                     break;
92.
93.                 case FC_RA8875_16:
94.                 case FC_RA8875_24:
95.                 case FC_RA8875_32:
96.                     return;
97.             }
98.
99.             /* 此处需要根据字库文件存放位置进行修改 */
100.            if (_code1 >=0xA1 && _code1 <= 0xA9 && _code2 >=0xA1)
101.            {
102.                pDot += ((_code1 - 0xA1) * 94 + (_code2 - 0xA1)) * font_bytes;
103.            }
104.            else if (_code1 >=0xB0 && _code1 <= 0xF7 && _code2 >=0xA1)
105.            {
106.                pDot += ((_code1 - 0xB0) * 94 + (_code2 - 0xA1) + 846) * font_bytes;
107.            }
108.            memcpy(_pBuf, pDot, font_bytes);
109.        #endif
110. }
```

下面将此函数涉及到的知识点为大家做个阐释：

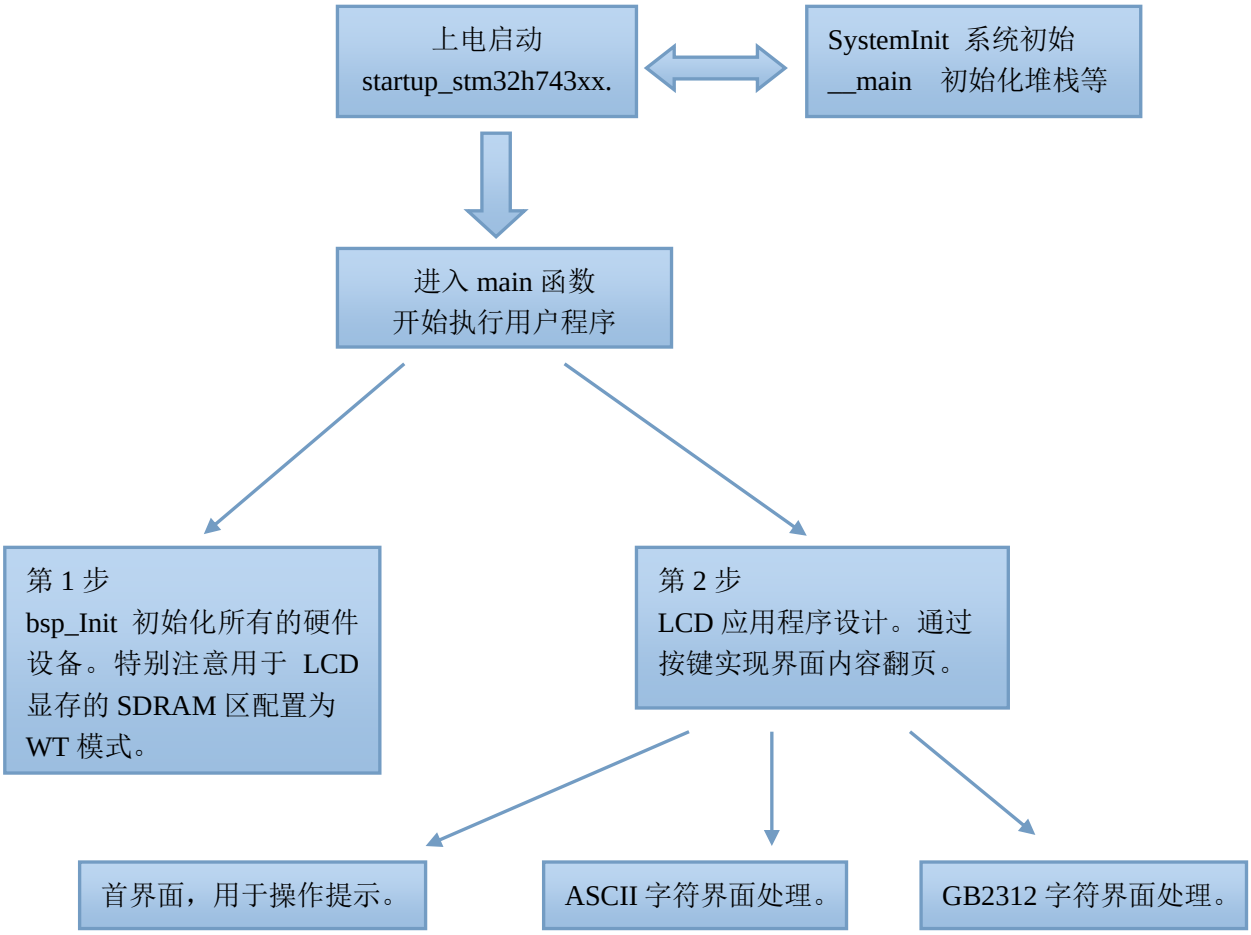
- 第 15-66 行，小字库显示，这个方案既可以显示小字库，也可以显示全字库。
  - 第 23-41 行，获取 12 点阵，16 点阵，24 点阵和 32 点阵汉字显示需要的字节数以及存储点阵数据的缓冲地址。
  - 第 49-66 行，这里是通过比较汉字的编码值找到点阵数据位置，如果遇到两个 0xFF，表示检索到数组末尾了也没有找到汉字点阵数组。找到数据后，将其复制到缓冲_pBuf 里面。
- 第 68-108 行，本章暂时用不到这种方案，后面章节用到这种方案了再为大家做说明。

53.6 LCD 驱动移植和使用

与第 51 章 51.7 小节相同，这里就不再赘述了。

53.7 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED，串口，LCD，SDRAM 等。
  - 第 2 步，LCD 应用程序设计部分，显示汉字。通过按键实现三种界面的处理，其中 GB2312 有几十个界面。

53.8 实验例程说明（MDK）

配套例子：

V7-033_LCD 的汉字小字库和全字库制作实验

实验目的：

1. 学习 LCD 的汉字小字库和全字库制作实验。

实验内容：

1. 小字库和全字库通过此软件生成：

   http://www.armbbs.cn/forum.php?mod=viewthread&tid=202 。

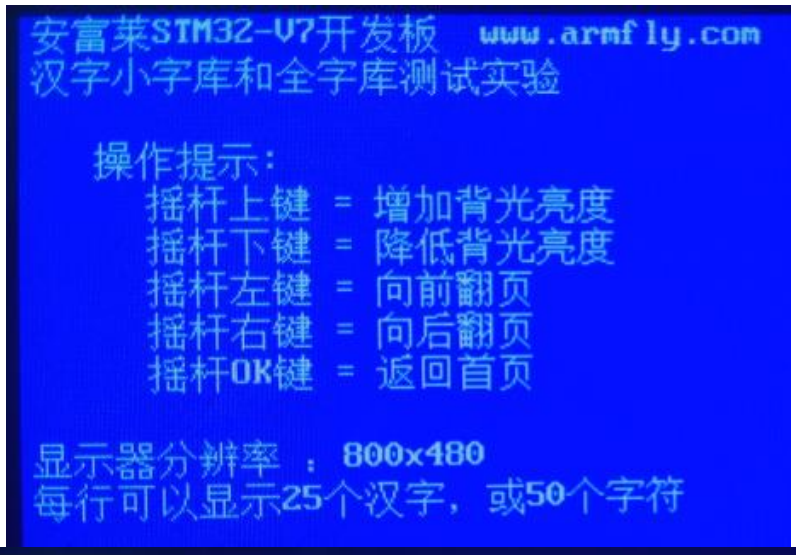
2. LCD 界面上展示 ASCII 字符和 GB2312 编码汉字。
3. 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。

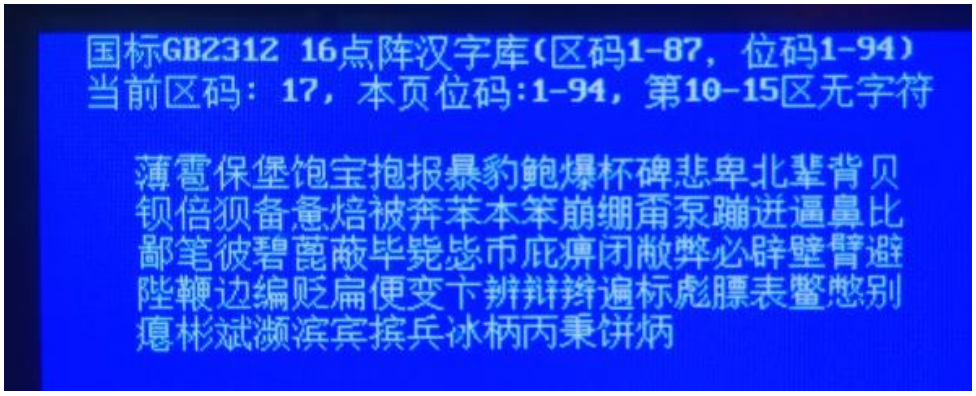
实验操作：

1. 摇杆上键，增加 LCD 背景光亮度。
2. 摇杆下键，降低 LCD 背景光亮度。
3. 摇杆左键，显示上一页汉字。
4. 摇杆右键，显示下一页汉字。
5. 摇杆 OK 键，返回首页。

LCD 的界面显示效果如下：

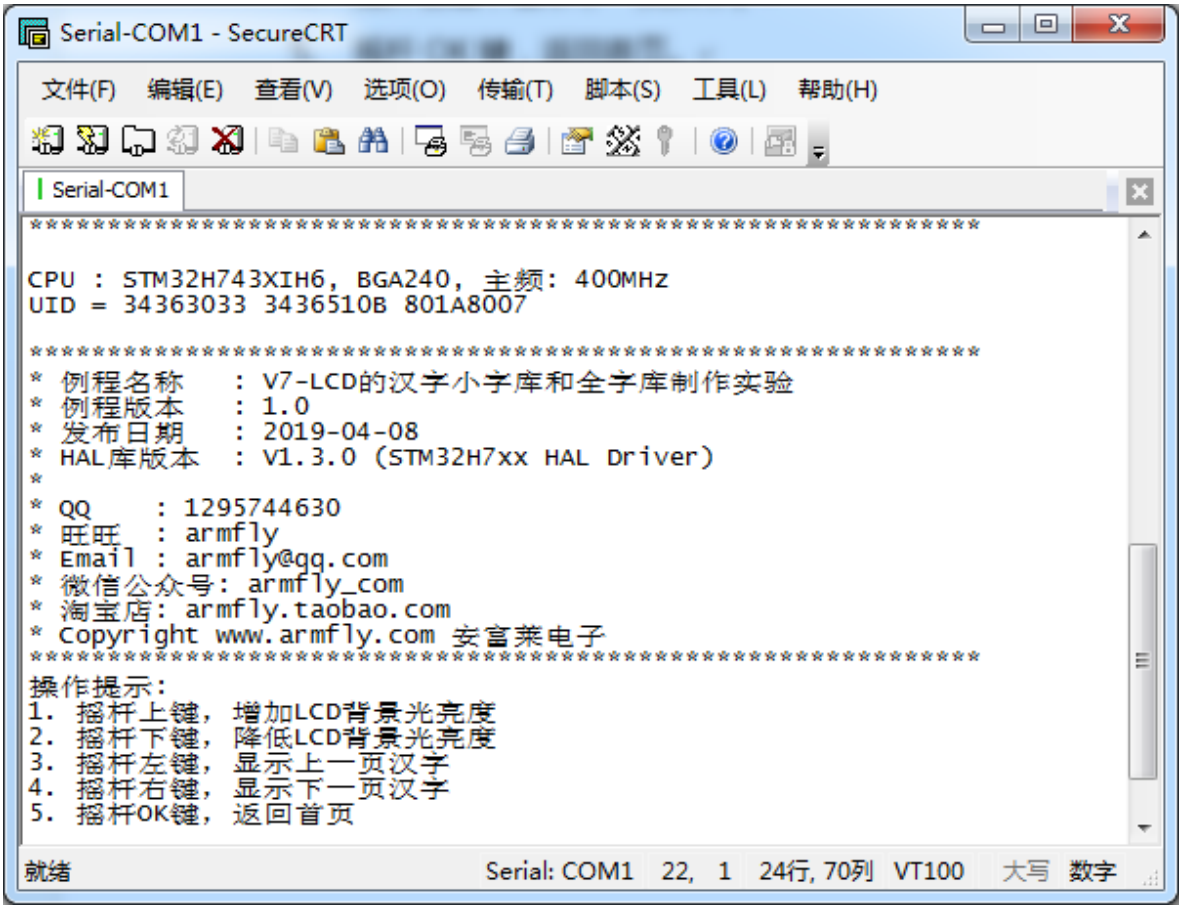
部分截图：





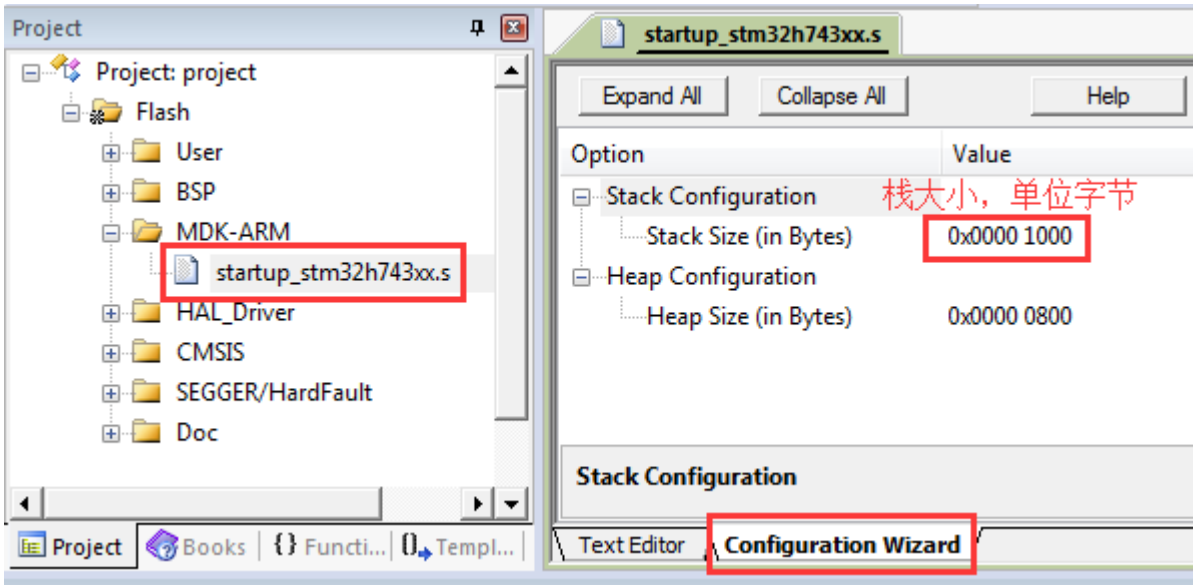
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

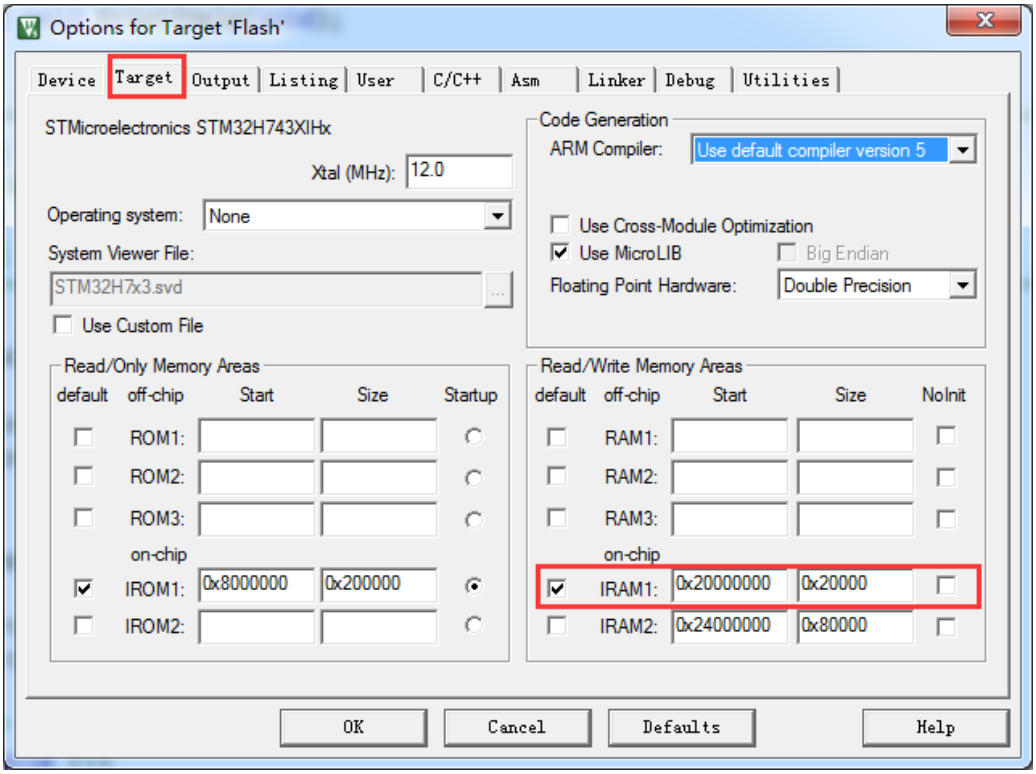


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */

    bsp_InitI2C();      /* 初始化 I2C 总线 */
    TOUCH_InitHard();   /* 初始化触摸芯片，LCD 面板型号的检查也在此函数，所以要在函数 LCD_InitHard 前调用 */
    LCD_InitHard();     /* 初始化 LCD */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SDRAM。由于 SDRAM 要用于 LCD 的显存，方便起见，直接将其配置为 WT 模式。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

```
    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SDRAM 的 MPU 属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0xC0000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_32MB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- LCD 界面上展示 ASCII 字符和 GB2312 编码汉字
- 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。

- 通过按键来实现翻页功能，方便查看所有 GB2312 编码汉字。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint16_t ucBright;      /* 背光亮度(0-255) */
    uint8_t ucKeyCode;      /* 按键代码 */
    uint8_t ucStatus;       /* 主程序状态字 */
    uint8_t fRefresh;       /* 刷屏请求标志,1 表示需要刷新 */


    bsp_Init();   /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    /* 延迟 200ms 再点亮背光，避免瞬间高亮 */
    bsp_DelayMS(200);

    DispFirstPage();   /* 显示第 1 页 */

    /* 界面整体显示完毕后，再打开背光，设置为缺省亮度 */
    bsp_DelayMS(100);
    ucBright = BRIGHT_DEFAULT;
    LCD_SetBackLight(ucBright);

    bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */

    /* 进入主程序循环体 */
    ucStatus = 0;
    fRefresh = 0;
    while (1)
    {
        /* 判断软件定时器 0 是否超时 */
        if(bsp_CheckTimer(0))
        {
            /* 每隔 200ms 进来一次 */
            bsp_LedToggle(2);
        }

        if (fRefresh == 1)
        {
            fRefresh = 0;

            switch (ucStatus)
            {
                case 0:
                    DispFirstPage();   /* 显示第 1 页 */
                    break;

                case 1:
                    DispAsciiDot();         /* 显示 ASCII 点阵 */
```

```
                break;

            default:
                /* 区码范围 ： 1 - 87 */
                if (ucStatus <= 89)
                {
                    DispHZK16(ucStatus);    /* 显示一个区的 94 个汉字 */
                }
                break;
        }
    }

    ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
    if (ucKeyCode != KEY_NONE)
    {
        /* 有键按下 */
        switch (ucKeyCode)
        {
            case JOY_DOWN_L:        /* 摇杆 LEFT 键按下 */
                if (ucStatus > 0)
                {
                    ucStatus--;
                }
                fRefresh = 1;       /* 请求刷新 LCD */
                break;

            case JOY_DOWN_R:        /* 摇杆 RIGHT 键按下 */
                if (ucStatus < DEMO_PAGE_COUNT - 1)
                {
                    ucStatus++;
                }
                fRefresh = 1;       /* 请求刷新 LCD */
                break;

            case  JOY_DOWN_OK:      /* 摇杆 OK 键 */
                ucStatus = 0;       /* 返回首页 */
                fRefresh = 1;       /* 请求刷新 LCD */
                break;

            case JOY_DOWN_U:        /* 摇杆 UP 键按下 */
                ucBright += BRIGHT_STEP;
                if (ucBright > BRIGHT_MAX)
                {
                    ucBright = BRIGHT_MAX;
                }
                LCD_SetBackLight(ucBright);
                printf("当前背景光亮度 : %d\r\n", ucBright);
                break;

            case JOY_DOWN_D:        /* 摇杆 DOWN 键按下 */
                if (ucBright < BRIGHT_STEP)
                {
                    ucBright = 0;
                }
                else
                {
                    ucBright -= BRIGHT_STEP;
                }
```

```
                        LCD_SetBackLight(ucBright);
                        printf("当前背景光亮度 : %d\r\n", ucBright);
                        break;

                    default:
                        break;
                }
            }
        }
```

下面的代码用于 LCD 首页显示:

```
/*
*********************************************************************************************************
*    函 数 名: DispFirstPage
*    功能说明: 显示操作提示
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
static void DispFirstPage(void)
{
    FONT_T tFont;          /* 定义一个字体结构体变量，用于设置字体参数 */
    uint16_t y;            /* Y 坐标 */
    uint16_t usLineCap;    /* 行高 */
    uint8_t buf[50];

    LCD_ClrScr(CL_BLUE);        /* 清屏，背景蓝色 */


    /* 设置字体属性 */
    tFont.FontCode = FC_ST_16;       /* 字体选择宋体 16 点阵，高 16x 宽 15) */
    tFont.FrontColor = CL_WHITE;     /* 字体颜色设置为白色 */
    tFont.BackColor = CL_MASK;       /* 文字背景颜色，透明 */
    tFont.Space = 0;                 /* 字符水平间距，单位 = 像素 */

    y = 0;
    usLineCap = 18; /* 行间距 */
    LCD_DispStr(5, y, "安富莱 STM32-V7 开发板  www.armfly.com", &tFont);
    y += usLineCap;
    LCD_DispStr(5, y, "汉字小字库和全字库测试实验", &tFont);

    y += 2 * usLineCap;
    LCD_DispStr(30, y, "操作提示:", &tFont);

    y += usLineCap;
    LCD_DispStr(50, y, "摇杆上键 = 增加背光亮度", &tFont);

    y += usLineCap;
    LCD_DispStr(50, y, "摇杆下键 = 降低背光亮度", &tFont);

    y += usLineCap;
    LCD_DispStr(50, y, "摇杆左键 = 向前翻页", &tFont);

    y += usLineCap;
    LCD_DispStr(50, y, "摇杆右键 = 向后翻页", &tFont);
```

```
    y += usLineCap;
    LCD_DispStr(50, y, "摇杆 OK 键 = 返回首页", &tFont);

    y += 2 * usLineCap;

    sprintf((char *)buf, "显示器分辨率 : %dx%d", g_LcdWidth, g_LcdHeight);
    LCD_DispStr(5, y, (char *)buf, &tFont);

    y += usLineCap;
    LCD_DispStr(5, y, "每行可以显示 25 个汉字，或 50 个字符", &tFont);
}
```

下面是 ASCII 字符显示：

```
/*
*********************************************************************************************************
*    函 数 名: DispAsciiDot
*    功能说明: 显示 ASCII 点阵
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
static void DispAsciiDot(void)
{
    uint8_t i,k;
    FONT_T tFont;      /* 定义一个字体结构体变量，用于设置字体参数 */
    uint16_t x;        /* X 坐标 */
    uint16_t y;        /* Y 坐标 */
    char buf[32 + 1];
    uint8_t ascii;

    LCD_ClrScr(CL_BLUE);        /* 清屏，背景蓝色 */

    /* 设置字体属性 */
    tFont.FontCode = FC_ST_16;       /* 字体选择宋体 16 点阵，高 16x 宽 15) */
    tFont.FrontColor = CL_WHITE;     /* 字体颜色设置为白色 */
    tFont.BackColor = CL_MASK;       /* 文字背景颜色，透明 */
    tFont.Space = 2;                 /* 字符水平间距，单位 = 像素 */

    LCD_DispStr(50, 0, "16 点阵 ASCII 码字库，代码 1-127", &tFont);

    x = 50;
    y = 40;
    ascii = 0;
    for (i = 0; i < 4; i++)
    {
        for (k = 0; k < 32; k++)
        {
            buf[k] = ascii++;
        }
        buf[32] = 0;
        if (buf[0] == 0)
        {
            buf[0] = ' ';
        }
        LCD_DispStr(x, y, buf, &tFont);
        y += 20;
    }
}
```

下面是 GB2312 编码字符显示:

```
/*
*********************************************************************************************************
*    函 数 名: DispHZK16
*    功能说明: 显示 16 点阵汉字阵
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void DispHZK16(uint8_t _ucIndex)
{
    uint8_t i,k;
    uint16_t x,y;
    char buf[50 + 1];
    uint8_t code1,code2;    /* 汉字内码 */
    uint8_t usLineCap = 18;
    FONT_T tFont;           /* 定义一个字体结构体变量，用于设置字体参数 */

    printf(" Display HZK Area Code = %d\r\n", _ucIndex - 1);

    if (_ucIndex == 2)
    {
        /* 第 1 次清屏，以后显示位置不变，可以不清屏，避免闪烁 */
        LCD_ClrScr(CL_BLUE);        /* 清屏，背景蓝色 */
    }

    /* 设置字体属性 */
    tFont.FontCode = FC_ST_16;      /* 字体选择宋体 16 点阵，高 16x 宽 15) */
    tFont.FrontColor = CL_WHITE;    /* 字体颜色设置为白色 */
    tFont.BackColor = CL_BLUE;      /* 文字背景颜色，蓝色 */
    tFont.Space = 0;                /* 字符水平间距，单位 = 像素 */

    y = 0;
    LCD_DispStr(20, y, "国标 GB2312 16 点阵汉字库(区码 1-87，位码 1-94)", &tFont);

    code1 = _ucIndex - 1; /* 得到区码 */
    code2 = 1;            /* 位码从 1 开始 */

    y += usLineCap;
    sprintf((char *)buf, (char *)"当前区码: %2d, 本页位码:1-94, 第 10-15 区无字符", code1);
    LCD_DispStr(20, y, buf, &tFont);
    y += (2 * usLineCap);

    /*
        机内码高位 = 区码 + 0xA0
        机内码低位 = 位码 + 0xA0

        区码范围 ：1 - 87
        位码范围 : 1 - 94

        每行显示 20 个汉字，一个区是 94 个汉字，需要 5 行显示,第 5 行显示 14 个汉字
    */

    x = 40;
    code1 += 0xA0; /* 换算到内码高位 */
    code2 = 0xA1;  /* 内码低位起始 */
    for (i = 0; i < 5; i++)
```

```
    {
        for (k = 0; k < 20; k++)
        {
            buf[2 * k] = code1;
            buf[2 * k + 1] = code2;
            code2++;
            if ((i == 4) && (k == 13))
            {
                k++;
                break;
            }
        }
        buf[2 * k] = 0;
        LCD_DispStr(x, y, buf, &tFont);
        y += usLineCap;
    }
}
```

53.9 实验例程说明（IAR）

配套例子：

V7-033_LCD 的汉字小字库和全字库制作实验

实验目的：

1. 学习 LCD 的汉字小字库和全字库制作实验。

实验内容：

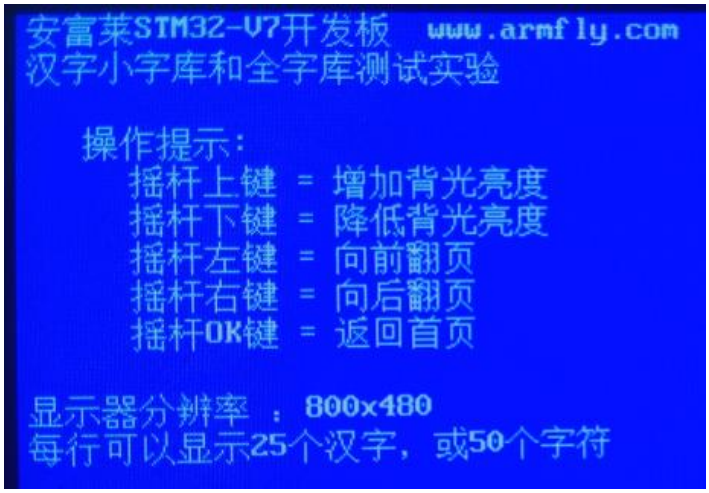
1. 小字库和全字库通过此软件生成：
   http://www.armbbs.cn/forum.php?mod=viewthread&tid=202 。
2. LCD 界面上展示 ASCII 字符和 GB2312 编码汉字。
3. 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。

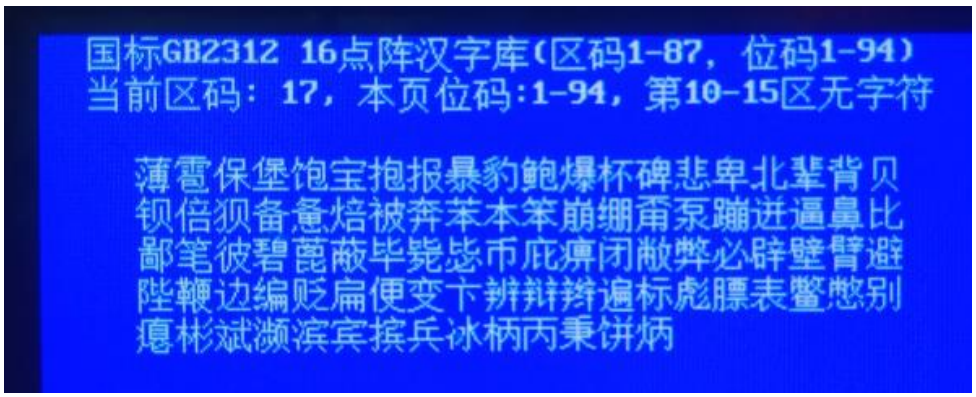
实验操作：

1. 摇杆上键，增加 LCD 背景光亮度。
2. 摇杆下键，降低 LCD 背景光亮度。
3. 摇杆左键，显示上一页汉字。
4. 摇杆右键，显示下一页汉字。
5. 摇杆 OK 键，返回首页。

LCD 的界面显示效果如下：

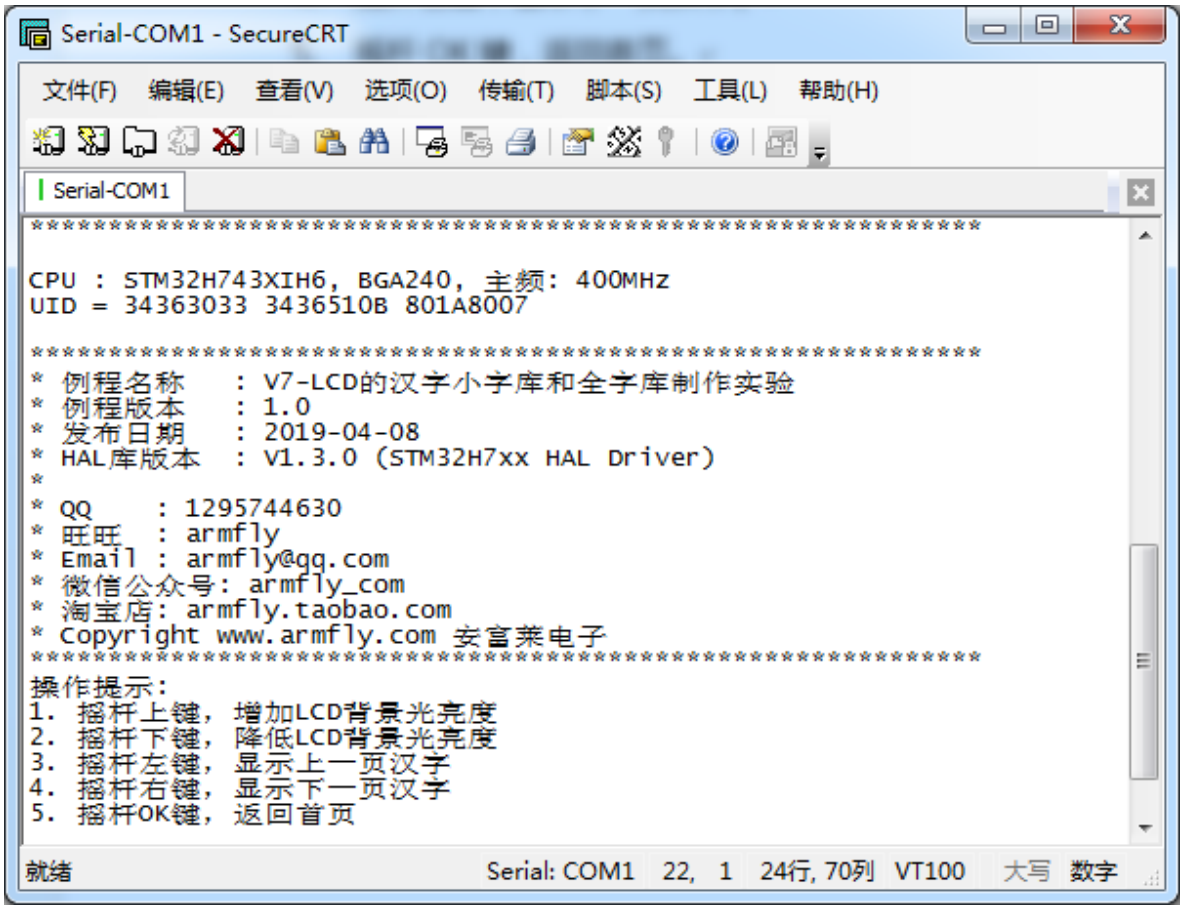
部分截图：





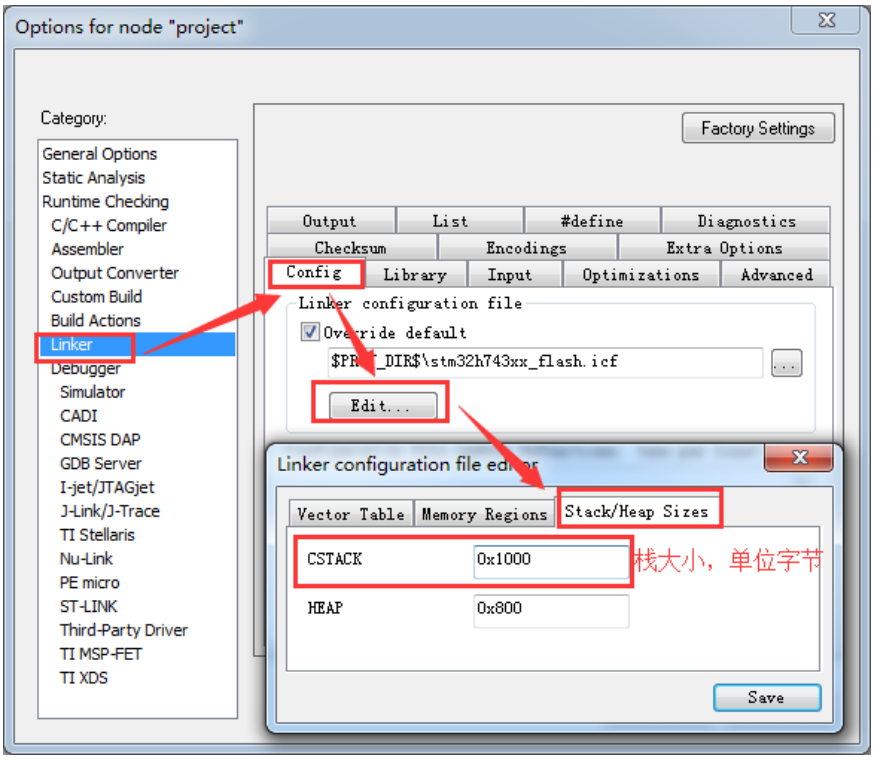
上电后串口打印的信息:

波特率 115200，数据位 8，奇偶校验位无，停止位 1

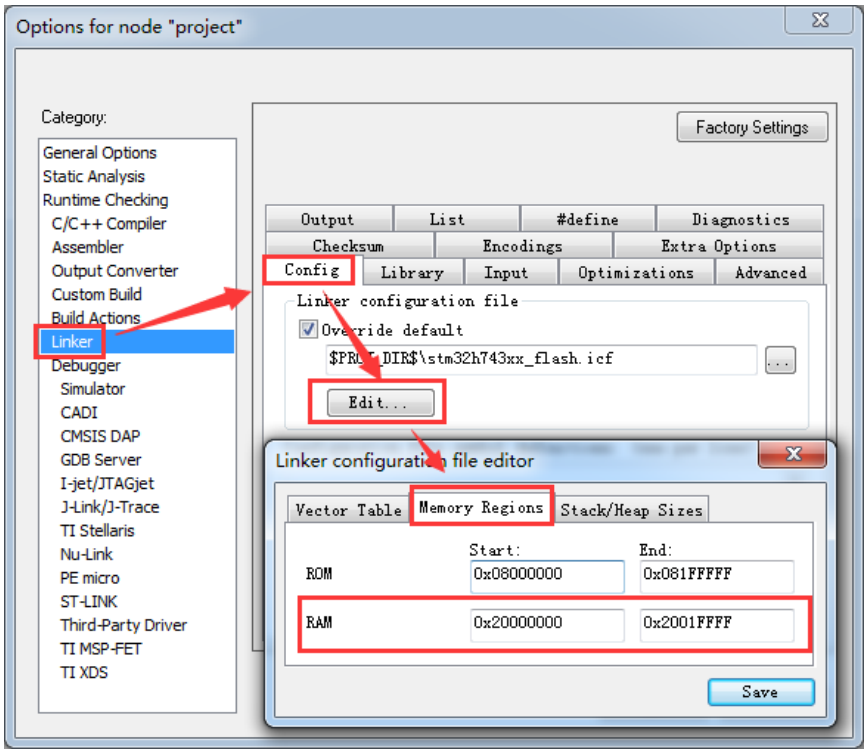


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
```

```
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */

    bsp_InitI2C();      /* 初始化 I2C 总线 */
    TOUCH_InitHard();   /* 初始化触摸芯片，LCD 面板型号的检查也在此函数，所以要在函数 LCD_InitHard 前调用 */
    LCD_InitHard();     /* 初始化 LCD */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SDRAM。由于 SDRAM 要用于 LCD 的显存，方便起见，直接将其配置为 WT 模式。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

```
    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SDRAM 的 MPU 属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0xC0000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_32MB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- LCD 界面上展示 ASCII 字符和 GB2312 编码汉字

- 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。
- 通过按键来实现翻页功能，方便查看所有 GB2312 编码汉字。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint16_t ucBright;      /* 背光亮度(0-255) */
    uint8_t ucKeyCode;      /* 按键代码 */
    uint8_t ucStatus;       /* 主程序状态字 */
    uint8_t fRefresh;       /* 刷屏请求标志,1 表示需要刷新 */


    bsp_Init();   /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    /* 延迟 200ms 再点亮背光，避免瞬间高亮 */
    bsp_DelayMS(200);

    DispFirstPage();   /* 显示第 1 页 */

    /* 界面整体显示完毕后，再打开背光，设置为缺省亮度 */
    bsp_DelayMS(100);
    ucBright = BRIGHT_DEFAULT;
    LCD_SetBackLight(ucBright);

    bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */

    /* 进入主程序循环体 */
    ucStatus = 0;
    fRefresh = 0;
    while (1)
    {
        /* 判断软件定时器 0 是否超时 */
        if(bsp_CheckTimer(0))
        {
            /* 每隔 200ms 进来一次 */
            bsp_LedToggle(2);
        }

        if (fRefresh == 1)
        {
            fRefresh = 0;

            switch (ucStatus)
            {
                case 0:
                    DispFirstPage();   /* 显示第 1 页 */
                    break;
```

```
            case 1:
                DispAsciiDot();         /* 显示 ASCII 点阵 */
                break;

            default:
                /* 区码范围 ： 1 - 87 */
                if (ucStatus <= 89)
                {
                    DispHZK16(ucStatus);    /* 显示一个区的 94 个汉字 */
                }
                break;
        }
    }

    ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
    if (ucKeyCode != KEY_NONE)
    {
        /* 有键按下 */
        switch (ucKeyCode)
        {
            case JOY_DOWN_L:        /* 摇杆 LEFT 键按下 */
                if (ucStatus > 0)
                {
                    ucStatus--;
                }
                fRefresh = 1;       /* 请求刷新 LCD */
                break;

            case JOY_DOWN_R:        /* 摇杆 RIGHT 键按下 */
                if (ucStatus < DEMO_PAGE_COUNT - 1)
                {
                    ucStatus++;
                }
                fRefresh = 1;       /* 请求刷新 LCD */
                break;

            case  JOY_DOWN_OK:      /* 摇杆 OK 键 */
                ucStatus = 0;       /* 返回首页 */
                fRefresh = 1;       /* 请求刷新 LCD */
                break;

            case JOY_DOWN_U:        /* 摇杆 UP 键按下 */
                ucBright += BRIGHT_STEP;
                if (ucBright > BRIGHT_MAX)
                {
                    ucBright = BRIGHT_MAX;
                }
                LCD_SetBackLight(ucBright);
                printf("当前背景光亮度 : %d\r\n", ucBright);
                break;

            case JOY_DOWN_D:        /* 摇杆 DOWN 键按下 */
                if (ucBright < BRIGHT_STEP)
                {
                    ucBright = 0;
                }
                else
                {
```

```
                        ucBright -= BRIGHT_STEP;
                    }
                    LCD_SetBackLight(ucBright);
                    printf("当前背景光亮度 : %d\r\n", ucBright);
                    break;

                default:
                    break;
            }
        }
    }
}
```

下面的代码用于 LCD 首页显示：

```
/*
*********************************************************************************************************
*    函 数 名: DispFirstPage
*    功能说明: 显示操作提示
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
static void DispFirstPage(void)
{
    FONT_T tFont;          /* 定义一个字体结构体变量，用于设置字体参数 */
    uint16_t y;            /* Y 坐标 */
    uint16_t usLineCap;    /* 行高 */
    uint8_t buf[50];

    LCD_ClrScr(CL_BLUE);        /* 清屏，背景蓝色 */


    /* 设置字体属性 */
    tFont.FontCode = FC_ST_16;       /* 字体选择宋体 16 点阵，高 16x 宽 15) */
    tFont.FrontColor = CL_WHITE;     /* 字体颜色设置为白色 */
    tFont.BackColor = CL_MASK;       /* 文字背景颜色，透明 */
    tFont.Space = 0;                 /* 字符水平间距，单位 = 像素 */

    y = 0;
    usLineCap = 18; /* 行间距 */
    LCD_DispStr(5, y, "安富莱 STM32-V7 开发板  www.armfly.com", &tFont);
    y += usLineCap;
    LCD_DispStr(5, y, "汉字小字库和全字库测试实验", &tFont);

    y += 2 * usLineCap;
    LCD_DispStr(30, y, "操作提示:", &tFont);

    y += usLineCap;
    LCD_DispStr(50, y, "摇杆上键 = 增加背光亮度", &tFont);

    y += usLineCap;
    LCD_DispStr(50, y, "摇杆下键 = 降低背光亮度", &tFont);

    y += usLineCap;
    LCD_DispStr(50, y, "摇杆左键 = 向前翻页", &tFont);

    y += usLineCap;
```

```
    LCD_DispStr(50, y, "摇杆右键 = 向后翻页", &tFont);

    y += usLineCap;
    LCD_DispStr(50, y, "摇杆 OK 键 = 返回首页", &tFont);

    y += 2 * usLineCap;

    sprintf((char *)buf, "显示器分辨率 : %dx%d", g_LcdWidth, g_LcdHeight);
    LCD_DispStr(5, y, (char *)buf, &tFont);

    y += usLineCap;
    LCD_DispStr(5, y, "每行可以显示 25 个汉字，或 50 个字符", &tFont);
}
```

下面是 ASCII 字符显示:

```
/*
*********************************************************************************************************
*    函 数 名: DispAsciiDot
*    功能说明: 显示 ASCII 点阵
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
static void DispAsciiDot(void)
{
    uint8_t i,k;
    FONT_T tFont;      /* 定义一个字体结构体变量，用于设置字体参数 */
    uint16_t x;        /* X 坐标 */
    uint16_t y;        /* Y 坐标 */
    char buf[32 + 1];
    uint8_t ascii;

    LCD_ClrScr(CL_BLUE);        /* 清屏，背景蓝色 */

    /* 设置字体属性 */
    tFont.FontCode = FC_ST_16;       /* 字体选择宋体 16 点阵，高 16x 宽 15) */
    tFont.FrontColor = CL_WHITE;     /* 字体颜色设置为白色 */
    tFont.BackColor = CL_MASK;       /* 文字背景颜色，透明 */
    tFont.Space = 2;                 /* 字符水平间距，单位 = 像素 */

    LCD_DispStr(50, 0, "16 点阵 ASCII 码字库，代码 1-127", &tFont);

    x = 50;
    y = 40;
    ascii = 0;
    for (i = 0; i < 4; i++)
    {
        for (k = 0; k < 32; k++)
        {
            buf[k] = ascii++;
        }
        buf[32] = 0;
        if (buf[0] == 0)
        {
            buf[0] = ' ';
        }
        LCD_DispStr(x, y, buf, &tFont);
        y += 20;
```

```
    }
}
```

下面是 GB2312 编码字符显示：

```
/*
*********************************************************************************************************
*    函 数 名: DispHZK16
*    功能说明: 显示 16 点阵汉字阵
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
static void DispHZK16(uint8_t _ucIndex)
{
    uint8_t i,k;
    uint16_t x,y;
    char buf[50 + 1];
    uint8_t code1,code2;    /* 汉字内码 */
    uint8_t usLineCap = 18;
    FONT_T tFont;           /* 定义一个字体结构体变量，用于设置字体参数 */

    printf(" Display HZK Area Code = %d\r\n", _ucIndex - 1);

    if (_ucIndex == 2)
    {
        /* 第 1 次清屏，以后显示位置不变，可以不清屏，避免闪烁 */
        LCD_ClrScr(CL_BLUE);        /* 清屏，背景蓝色 */
    }

    /* 设置字体属性 */
    tFont.FontCode = FC_ST_16;      /* 字体选择宋体 16 点阵，高 16x 宽 15) */
    tFont.FrontColor = CL_WHITE;    /* 字体颜色设置为白色 */
    tFont.BackColor = CL_BLUE;      /* 文字背景颜色，蓝色 */
    tFont.Space = 0;                /* 字符水平间距，单位 = 像素 */

    y = 0;
    LCD_DispStr(20, y, "国标 GB2312 16 点阵汉字库(区码 1-87，位码 1-94)", &tFont);

    code1 = _ucIndex - 1; /* 得到区码 */
    code2 = 1;            /* 位码从 1 开始 */

    y += usLineCap;
    sprintf((char *)buf, (char *)"当前区码: %2d, 本页位码:1-94, 第 10-15 区无字符", code1);
    LCD_DispStr(20, y, buf, &tFont);
    y += (2 * usLineCap);

    /*
        机内码高位 = 区码 + 0xA0
        机内码低位 = 位码 + 0xA0

        区码范围 ：1 - 87
        位码范围 : 1 - 94

        每行显示 20 个汉字，一个区是 94 个汉字，需要 5 行显示,第 5 行显示 14 个汉字
    */

    x = 40;
    code1 += 0xA0; /* 换算到内码高位 */
```

```
    code2 = 0xA1;  /* 内码低位起始 */
    for (i = 0; i < 5; i++)
    {
        for (k = 0; k < 20; k++)
        {
            buf[2 * k] = code1;
            buf[2 * k + 1] = code2;
            code2++;
            if ((i == 4) && (k == 13))
            {
                k++;
                break;
            }
        }
        buf[2 * k] = 0;
        LCD_DispStr(x, y, buf, &tFont);
        y += usLineCap;
    }
}
```

53.10 总结

本章节涉及到的知识点比较多，需要大家花点时间去掌握，直至可以独立驱动一个显示屏。

第54章 STM32H7 的 LTDC 应用之 LCD 电阻触摸和电容触摸

本章教程为大家讲解 LTDC 应用之 LCD 电阻触摸芯片 STMPE811 的 4 点和 2 点触摸校准和电容触摸芯片 FT5X06、GT911 和 GT811 的使用。

54.1 初学者重要提示
54.2 电阻触摸和电容触摸相关知识
54.3 电阻屏硬件设计
54.4 电容屏硬件设计
54.5 电阻触摸驱动设计
54.6 电容触摸驱动设计
54.7 不同触摸 IC 的识别
54.8 LCD 触摸移植和使用
54.9 实验例程设计框架
54.10 实验例程说明（MDK）
54.11 实验例程说明（IAR）
54.12 总结

54.1 初学者重要提示

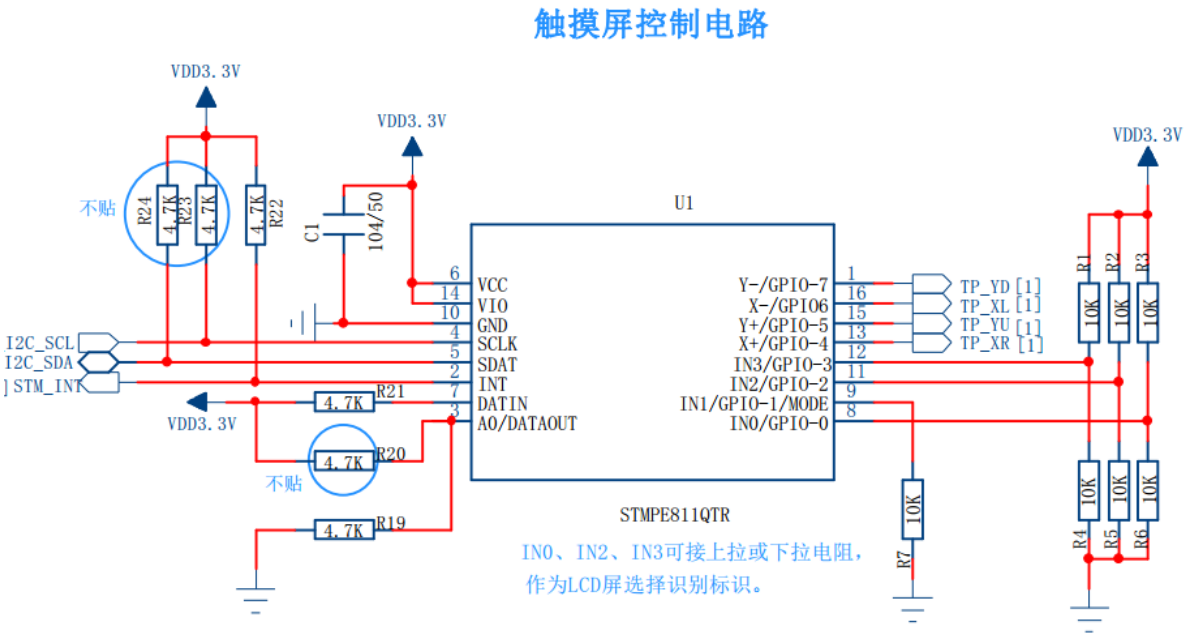
- 学习本章节前，务必优先学习第 50 章，需要对 LTDC 的基础知识和 HAL 库的几个常用 API 有个认识。
- LTDC 驱动设计和相关问题在第 51 章有详细说明。
- 电阻触摸支持 2 点和 4 点校准，而电容屏无需校准。
- 电阻触摸校准解决的是触摸板的线性度问题，而飞点要另外处理，当前程序已经做了支持。总的来说，V7 配套的电阻触摸方案已经比较成熟，可以放心用于项目。

54.2 电阻触摸和电容触摸相关知识

这部分知识点在第 52 章的 2.2 小节有详细说明，必看。

54.3 电阻屏硬件设计

电阻触摸 STMPE811 的原理图如下：

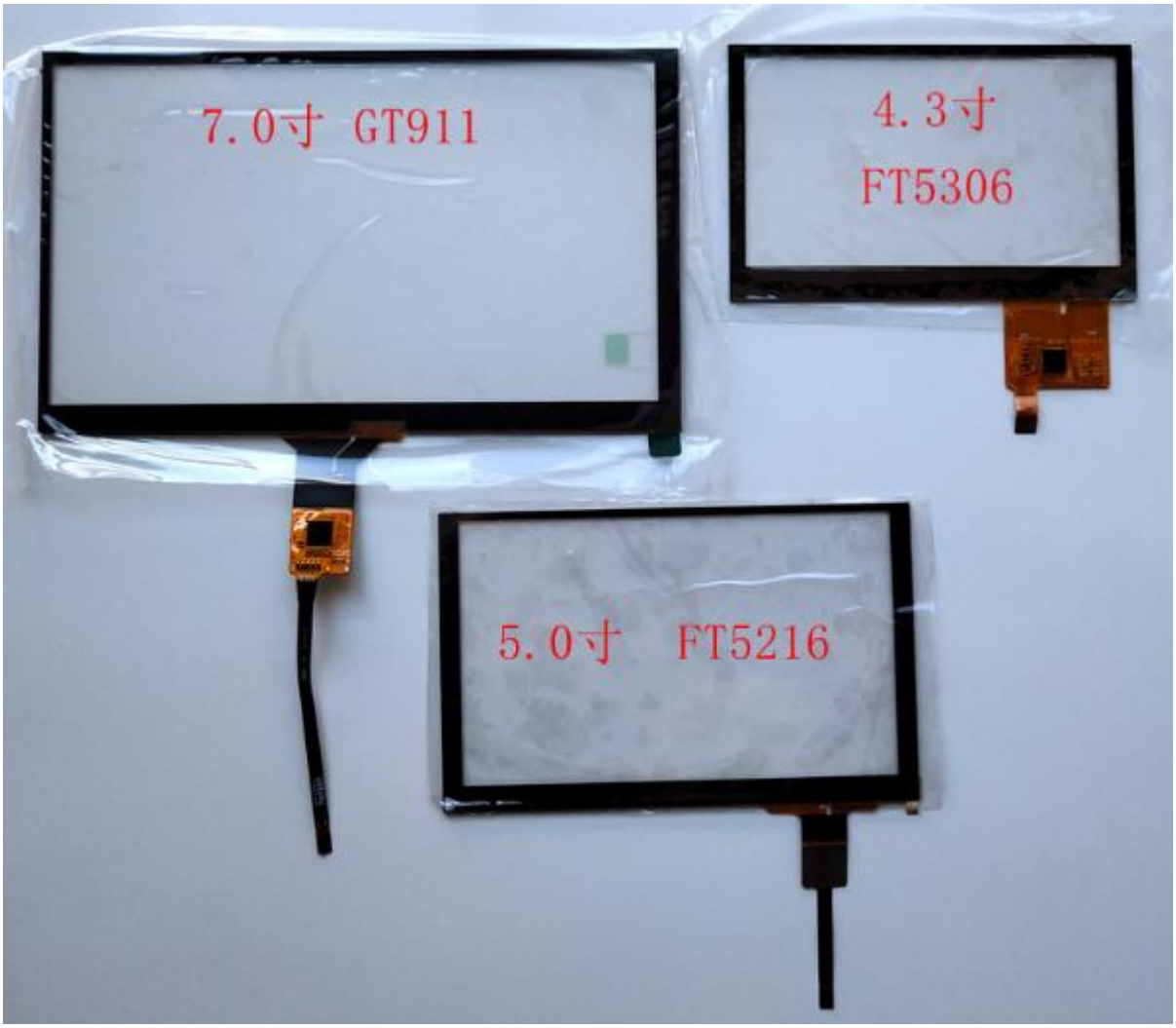


通过 STMPE811 的原理图要了解以下几点：

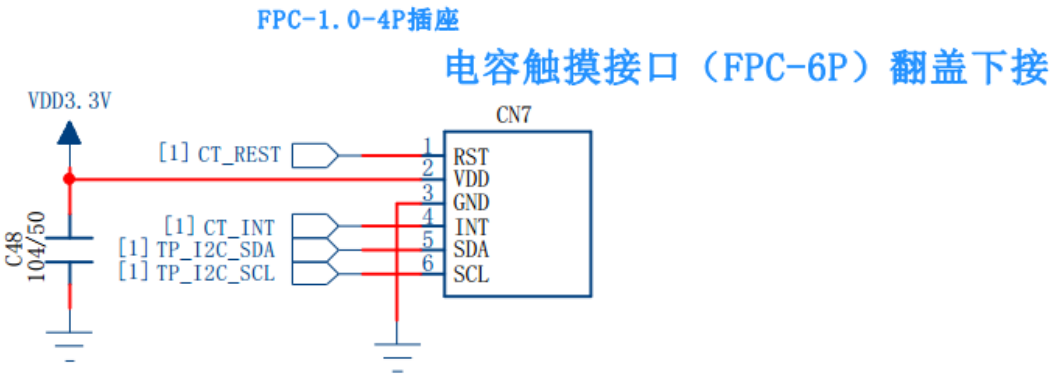
- I2C 的两根通信线 I2C_SCL 和 I2C_SDA 的上拉电阻在 V7 的主板上。
- 原理图右侧的 GPIO-0 到 GPIO-7 可以作为扩展 IO 使用，支持输入和输出。其中 GPIO-4 到 GPIO-7 用于电阻触摸校准（使用那个 IO 是可以配置的）。
- 对于 X-，X+，Y-和 Y+，只要不是 X 和 Y 进行组合，其它组合方式可以随意接，配套的触摸校准算法都可以正常识别。

54.4 电容屏硬件设计

电容触摸主要有三种：FT5X06，GT911 和 GT811，其中 GT811 已经停产。下面是 FT5X06 和 GT911 触摸板效果（触摸板和触摸芯片是一体的）：



触摸芯片已经集成到柔性 PCB 上，且已经校准好。用户使用的话，直接通过 I2C 方式读取数据即可。下面是电容触摸板引出的引脚：



注意 I2C_SDK 和 I2C_SCL 的上拉电阻在 V7 主板上。

54.5 电阻触摸驱动设计

下面将电阻触摸程序设计中的相关问题逐一为大家做个说明。

54.5.1 STMPE811 的驱动实现

电阻触摸要比电容触摸麻烦很多，因为电阻触摸要做校准，还要做滤波，否则采集回来的触摸值会抖动或者出现飞点，出现这种情况的主要原因是电阻触摸板的线性度不够好。开发板电阻屏使用的触摸芯片是 STMPE811，这个芯片其实就是 12 位分辨率的 ADC，用来采集电阻触摸板的 X 轴 ADC 值和 Y 轴 ADC 值，然后按照一定的线性关系将 ADC 值转换为实际的坐标值。其中这个线性关系是通过触摸校准建立起来的，每次采集的 X 轴和 Y 轴 ADC 就可以代入这个线性关系，从而获得实际的坐标值。

总的来说，STMPE811 的驱动不难实现，可以结合 STMPE811 的数据手册：http://bbs.armfly.com/read.php?tid=23306 研究开发板提供的驱动配置。配置好后仅需要提供读取的 X 轴，Y 轴的 ADC 值以及触摸按下状态（判断 STMPE811 的中断输出引脚就可以了，如果有触摸，这个引脚输出低电平，反之，输出高电平。通过判断这个引脚就可以选择是否读取 X 轴，Y 轴的 ADC 值，避免不必要的操作）。这些函数在 bsp_ts_stmpe811.c 文件实现。而触摸值滤波，触摸扫描和触摸校准是在 bsp_touch.c 文件里面实现。

下面是清除触摸中断标志函数和 X 轴，Y 轴的 ADC 值读取函数，这些函数被 bsp_touch.c 文件所调用，而函数 TOUCH_PenInt 是在 bsp_touch.c 文件，这里也贴出来。

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: TOUCH_PenInt
4.  *    功能说明: 判断触摸按下
5.  *    形    参: 无
6.  *    返 回 值: 0 表示无触笔按下，1 表示有触笔按下
7.  *********************************************************************************************************
8.  */
9.  uint8_t TOUCH_PenInt(void)
10. {
11.     if ((TP_INT_GPIO_PORT->IDR & TP_INT_PIN) == 0)
12.     {
13.         return 1;
14.     }
15.     return 0;
16. }
17.
18. /*
19. *********************************************************************************************************
20. *    函 数 名: STMPE811_ClearInt
21. *    功能说明: 清楚触笔中断
22. *    形    参: 无
23. *    返 回 值: 无
24. *********************************************************************************************************
25. */
26. void STMPE811_ClearInt(void)
27. {
28.     STMPE811_WriteReg1(REG811_INT_STA, 0xFF);
29. }
30.
31. /*
32. *********************************************************************************************************
```

```
*    函 数 名: STMPE811_ReadX
*    功能说明: 读取 X 坐标 adc
*    形    参: 无
*    返 回 值: X 坐标值 adc
*********************************************************************************************************
*/
uint16_t STMPE811_ReadX(void)
{
    /* 按照 XY 读取模式，连续读取 3 字节数据，然后分解出 X,Y
     |  byte0   |     byte1      |   byte2  |
     | X[11:4], | X[3:0],Y[11:8] | Y[7:0]   |
    */
    uint8_t buf[3];

#if 0
    STMPE811_ReadBytes(buf, REG811_TSC_DATA1, 3);

    s_AdcX = ((uint16_t)buf[0] << 4) | (buf[1] >> 4);
    s_AdcY = ((uint16_t)(buf[1] & 0xF) << 8) | buf[2];
#else
    if (STMPE811_ReadReg1(REG811_TSC_CTRL) & 0x80)
    {
        STMPE811_ReadBytes(buf, REG811_TSC_DATA1, 3);

        s_AdcX = ((uint16_t)buf[0] << 4) | (buf[1] >> 4);
        s_AdcY = ((uint16_t)(buf[1] & 0xF) << 8) | buf[2];

        #if 0
        /* for debug */
        {
            static int32_t s_t1 = 0;
            int32_t tt;

            tt = bsp_GetRunTime();
            if (tt - s_t1 > 1000)
            {
                printf("\r\n");
                s_t1 = tt;
            }
            printf("(%7d) %5d %5d\r\n", tt, s_AdcX, s_AdcY);
        }
        #endif
    }
    else
    {
        s_AdcX = 0;
        s_AdcY = 0;
    }
#endif

    return s_AdcX;
}

/*
*********************************************************************************************************
*    函 数 名: STMPE811_ReadX
*    功能说明: 读取 Y 坐标 adc
*    形    参: 无
```

```
91.  *    返 回 值: Y 坐标值 adc
92.  *********************************************************************************************************
93.  */
94.  uint16_t STMPE811_ReadY(void)
95.  {
96.      return  s_AdcY;
97.  }
```

下面将程序设计中的关键地方做个阐释：

- 第 9-16 行，通过判断 STMPE811 的中断输出引脚的高低电平来判断触摸板是否被按下，如果有触摸，这个引脚输出低电平，反之，输出高电平。通过判断这个引脚就可以选择是否读取 X 轴，Y 轴的 ADC 值，避免不必要的操作。
- 第 26-29 行，清除触摸中断标志，检测到触摸屏未被按下时，要做清除。
- 第 39-84 行，读取 X 轴 ADC 数值。
- 第 94-97 行，读取 Y 轴 ADC 数值。

54.5.2 电阻触摸扫描函数 TOUCH_Scan

接下来再来看 bsp_touch.c 文件中 STMPE811 触摸扫描函数 TOUCH_Scan 的实现：

```
1.   /*
2.   *********************************************************************************************************
3.   *    函 数 名: TOUCH_Scan
4.   *    功能说明: 触摸板事件检测程序。该函数被周期性调用，每 ms 调用 1 次. 见 bsp_Timer.c
5.   *    形    参:  无
6.   *    返 回 值: 无
7.   *********************************************************************************************************
8.   */
9.   void TOUCH_Scan(void)
10.  {
11.      uint16_t usAdcX;
12.      uint16_t usAdcY;
13.      static uint16_t s_usXBuf[SAMPLE_COUNT];
14.      static uint16_t s_usYBuf[SAMPLE_COUNT];
15.      static uint8_t s_ucPos = 0;
16.      static uint8_t s_count = 0;
17.      static uint8_t s_down = 0;
18.      static uint16_t s_usSaveAdcX, s_usSaveAdcY; /* 用于触笔抬起事件，保存按下和移动的最后采样值 */
19.      static uint8_t s_ms = 0;
20.
21.      if (g_GT811.Enable == 1)
22.      {
23.          GT811_Timer1ms();  /* 电容触摸屏程序计数器 */
24.          return;
25.      }
26.
27.      if (g_GT911.Enable == 1)
28.      {
29.          GT911_Timer1ms();  /* 电容触摸屏程序计数器 */
30.          return;
31.      }
32.
```

```
33.     if (g_tFT5X06.Enable == 1)
34.     {
35.         FT5X06_Timer1ms(); /* 电容触摸屏程序计数器 */
36.         return;
37.     }
38.
39.     /* 下面用于电阻触摸 */
40.
41.     if (g_tTP.Enable == 0)
42.     {
43.         return;
44.     }
45.
46.     if (++s_ms >= 2)
47.     {
48.         return;
49.     }
50.
51.     /* 2ms 进入一次 */
52.     s_ms = 0;
53.
54.     /* 触笔中断发生 */
55.     if (TOUCH_PenInt())
56.     {
57.         /* 获得原始的 ADC 值，未滤波 */
58.         usAdcX = STMPE811_ReadX();
59.         usAdcY = STMPE811_ReadY();
60.
61.         if (TOUCH_PressValid(usAdcX, usAdcY))
62.         {
63.             /* 按压 30ms 之后才开始采集数据 */
64.             if (s_count >= DOWN_VALID / 2)
65.             {
66.                 s_usXBuf[s_ucPos] = usAdcX;
67.                 s_usYBuf[s_ucPos] = usAdcY;
68.
69.                 /* 采集 20ms 数据进行滤波 */
70.                 if (++s_ucPos >= SAMPLE_COUNT / 2)
71.                 {
72.                     s_ucPos = 0;
73.
74.                     /* 对 ADC 采样值进行软件滤波 */
75.                     g_tTP.usAdcNowX = TOUCH_DataFilter(s_usXBuf, SAMPLE_COUNT / 2);
76.                     g_tTP.usAdcNowY = TOUCH_DataFilter(s_usYBuf, SAMPLE_COUNT / 2);
77.
78.                     if (s_down == 0)
79.                     {
80.                         s_down = 1;
81.                         /* 触摸按下事件 */
82.                         TOUCH_PutKey(TOUCH_DOWN, g_tTP.usAdcNowX, g_tTP.usAdcNowY);
83.
84.                         s_usSaveAdcX = g_tTP.usAdcNowX;
85.                         s_usSaveAdcY = g_tTP.usAdcNowY;
86.                     }
87.                     else
88.                     {
89.                 if (TOUCH_MoveValid(s_usSaveAdcX, s_usSaveAdcY, g_tTP.usAdcNowX, g_tTP.usAdcNowY))
90.                         {
```

```
91.                                 /* 触摸移动事件 */
92.                                 TOUCH_PutKey(TOUCH_MOVE, g_tTP.usAdcNowX, g_tTP.usAdcNowY);
93.
94.                                 s_usSaveAdcX = g_tTP.usAdcNowX;
95.                                 s_usSaveAdcY = g_tTP.usAdcNowY;
96.                             }
97.                             else
98.                             {
99.                                 g_tTP.usAdcNowX = 0; /* for debug stop */
100.                            }
101.                        }
102.                    }
103.                }
104.                else
105.                {
106.                    s_count++;
107.                }
108.            }
109.            else
110.            {
111.                if (s_count > 0)
112.                {
113.                    if (--s_count == 0)
114.                    {
115.                        /* 触摸释放事件 */
116.                        //TOUCH_PutKey(TOUCH_RELEASE, g_tTP.usAdcNowX, g_tTP.usAdcNowY);
117.                        TOUCH_PutKey(TOUCH_RELEASE, s_usSaveAdcX, s_usSaveAdcY);
118.
119.                        g_tTP.usAdcNowX = 0;
120.                        g_tTP.usAdcNowY = 0;
121.
122.                        s_count = 0;
123.                        s_down = 0;
124.
125.                        STMPE811_ClearInt();        /* 清触笔中断标志 */
126.                    }
127.                }
128.                s_ucPos = 0;
129.            }
130.        }
131. }
```

下面将程序设计中的关键地方做个阐释：

- 第 9 行，此函数要每 1ms 被调用一次。
- 第 21-37 行，用于 GT811，GT911 和 FT5X06 的程序计数器。
- 第 46-52 行，设置每 2ms 进行一次 STMPE811 检测。
- 第 55 行，这个就是本章前面小节说的利用 STMPE811 的中断输出引脚的高低电平来判断触摸板是否被按下。
- 第 58-59 行，读取 X 轴 ADC 数值和 Y 轴 ADC 数值。
- 第 61 行，通过函数 TOUCH_PressValid 检测刚刚读取的 X 轴，Y 轴数值是否在有效的范围内。函数 TOUCH_PressValid 的具体实现如下，其中全局变量 g_tTP.usMaxAdc = 4095，因为电阻触摸芯片 STMPE811 是 12 位 ADC，最大触摸值就是 2^12 – 1 = 4095。

```
/* 有效 ADC 值的判断门限. 太接近 ADC 临界值的坐标认为无效 */
#define ADC_VALID_OFFSET    2

/*
*********************************************************************************************************
*    函 数 名: TOUCH_PressValid
*    功能说明: 判断按压是否有效，根据 X, Y 的 ADC 值进行大致判断
*    形    参: 无
*    返 回 值: 1 表示有效； 0 表示无效
*********************************************************************************************************
*/
static uint8_t    TOUCH_PressValid(uint16_t _usX, uint16_t _usY)
{
    if ((_usX <= ADC_VALID_OFFSET) || (_usY <= ADC_VALID_OFFSET)
        || (_usX >= g_tTP.usMaxAdc - ADC_VALID_OFFSET)
        || (_usY >= g_tTP.usMaxAdc - ADC_VALID_OFFSET))
    {
        return 0;
    }
    else
    {
        return 1;
    }
}
```

- 第 64 行，DOWN_VALID 的宏定义是

  #define DOWN_VALID 30

  由于是每 2ms 进行一次检测，这里就表示延迟 30ms 后进行触摸数据采集。延迟 30ms 是为了消除触摸抖动。

- 第 70 行，SAMPLE_COUNT 的宏定义是

  #define SAMPLE_COUNT 20

  由于是每 2ms 进行一次检测，这里就表示采集够 10 组数据，即 20ms 后进行下一步操作。

- 第 75-76 行，对 X 轴和 Y 轴的 ADC 数值都进行软件滤波。软件滤波函数 TOUCH_DataFilter 的实现方法是对 10 组数值由小到大进行排序，对第 3 个，第 4 个和第 5 个数值求和，然后求平均，将平均值作为最终的 ADC 数值。

- 第 78-86 行，变量标识 s_down = 0 表示触摸之前是未按下状态，在此条件里面设置 s_down = 1 表示触摸已经按下，并通过函数 TOUCH_TransX（这个函数比较关键，是通过触摸校准函数得到的一个线性关系）将当前的 X 轴和 Y 轴 ADC 数值转换成实际的坐标值，然后调用函数 TOUCH_PutKey 将当前的坐标信息存储到 FIFO 里面。

- 第 89-100 行设置变量标识 s_down = 1 后会进入此条件里面，在这个条件里面通过函数 TOUCH_MoveValid 判断当前是否是有效的移动，如果是，就继续调用函数 TOUCH_PutKey 将当前的坐标信息存储到 FIFO 里面，如果不是，就设置全局变量 g_tTP.usAdcNowX = 0。

- 第 111-128 行，如果通过 STMPE811 的中断输出引脚检测到触摸未按下，然后判断变量 s_count 是否大于 0，如果大于 0 的话，做减减运算，算是做了一个松手延迟，防止抖动。减到 0 的时候，将触摸未按下或者说触摸释放消息通过函数 TOUCH_PutKey 存储到 FIFO 里面。

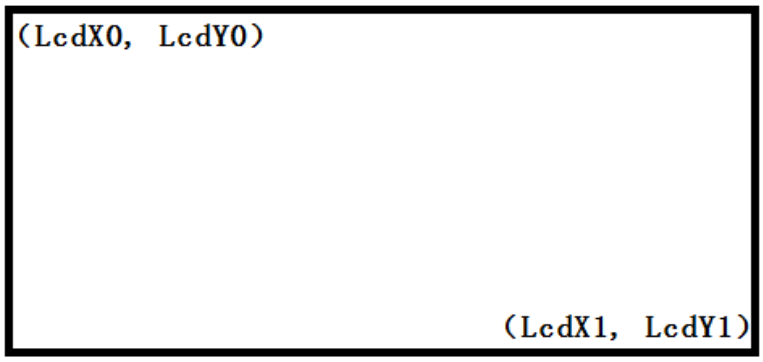
54.5.3 电阻屏触摸校准原理（2 点）

由于不同电阻触摸板的线性度参差不齐，不能直接采用比例关系将电阻触摸芯片 STMPE811 的返回值转换成实际的坐标。比如我们操作的显示屏分辨率是 800*480，电阻触摸芯片采用 STMPE811（12 位 ADC，触摸值范围 0-4095），获得当前的触摸值是（1024, 2048），按照比例关系转换成坐标值就是(1024*800/4096，2048*800/4096)，即（200，400）。采用这种方法效果不好，容易出现点击不准确的问题。

鉴于此原因，需要通过触摸校准在 ADC 数值和显示屏分辨率之间建立一个新的线性关系，简单的说就是由比例关系 y = ax 升级为 y = ax + b。如果有了新的触摸 ADC 数值，代入这个线性关系里面就可以得到当前实际的坐标值，触摸校准的作用就在这里了。

具体实现原理图如下：

在左上角和右下角分别设置两个坐标点（LcdX0， LcdY0）和（LcdX1， LcdY1），然后让用户去点击，会得到两组 ADC 数值（AdcX0，AdcY0）和（AdcX1， AdcY1）。



根据这四个坐标点，可以建立两组方程，一个 X 轴的，一个 Y 轴。

- 将数值（AdcX0， LcdX0）和（AdcX1， LcdX1）代入方程 y = ax + b 得到 X 轴方程 ：
  y = （x - AdcX0）*（LcdX1 - LcdX0）/(AdcX1 - AdcX0) + LcdX0。
- 将数值（AdcY0， LcdY0）和（AdcY1， LcdY1）代入方程 y = ax + b 得到 Y 轴方程
  y = （x - AdcY0）*（LcdY1 - LcdY0）/(AdcY1 - AdcX0) + LcdY0。

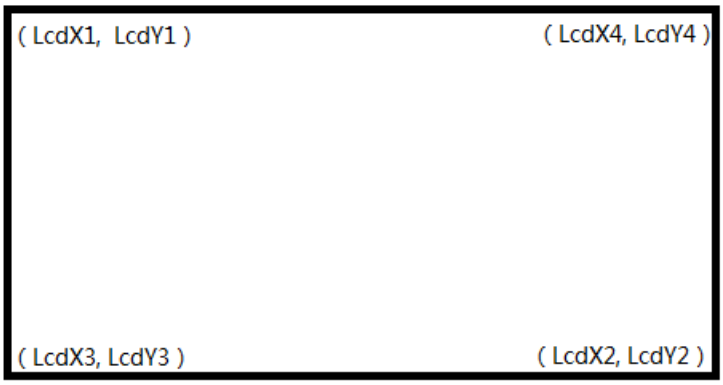
后面采集到的 ADC 数值直接代入上面公式就可以得到校准后的物理坐标值（实际的分辨率坐标）。

54.5.4 电阻屏触摸校准原理（4 点）

4 点触摸校准实现，略复杂，实现原理如下（如果理解起来麻烦的话，会用就行，一般情况下 2 点校准就行）：

在 LCD 的左上角,右上角,左下角和右下角分别标坐标点(LcdX1， LcdY1),(LcdX4, LcdY4),(LcdX3, LcdY3）和（LcdX2, LcdY2)。然后让用户去点击，会得到四组 ADC 数值（AdcX1, AdcY1)，(AdcX4,

AdcY4），（AdcX3, AdcY3）和（AdcX2, AdcY2）。



计算 X 轴:

1. 将数值（AdcX1，AdcY1）和（AdcX2, AdcY2）代入方程 y = ax + b 得到一组方程
   y = （x - AdcX1）*（AdcY2- AdcY1）/(AdcX2- AdcX1) + AdcY1

2. 这里将 AdcX2 用 AdcX3 替换，那么坐标方程就变为
   y = （x - AdcX1）*（AdcY2- AdcY1）/(AdcX3- AdcX1) + AdcY1。

3. 同理，将 AdcX1 用 AdcX4 替换，那么坐标方程就变为
   y = （x - AdcX4）*（AdcY2- AdcY1）/(AdcX3- AdcX4) + AdcY1。
   那么将采集的 X 值代入上面两个方程会得到两个数值，假设数值是 x1 和 x2。

4. 再将(x1， LcdX1)）和（x2， LcdX2）代入方程 y = ax + b 得到一组方程
   y = （x - x1）*（LcdX2- LcdX1）/(x2- x1) + LcdX1。

将采集的 X 轴 ADC 数值再次代入这个方程就得到了最终的物理坐标（实际的分辨率坐标）。

计算 Y 轴:

1. 将数值（AdcX1， AdcY1）和（AdcX2， AdcY2）代入方程 y = ax + b 得到一组方程
   y = （x - AdcX1）*（AdcY2- AdcY1）/(AdcX2- AdcX1) + AdcY1

2. 这里将 AdcY2 用 AdcY4 替换，那么坐标方程就变为
   y = （x - AdcX1）*（AdcY4- AdcY1）/(AdcX2- AdcX1) + AdcY1

3. 同理，将 AdcX1 用 AdcX3 替换，那么坐标方程就变为
   y = （x - AdcX3）*（AdcY2- AdcY1）/(AdcX2- AdcX3) + AdcY1
   那么将采集的 X 值代入上面两个方程会得到两个数值，假设数值是 x1 和 x2。

4. 再将(x1, LcdY1)）和（x2， LcdY2）代入方程 y = ax + b 得到一组方程
   y = （x - x1）*（LcdY2- LcdY1）/(x2- x1) + LcdY1。

将采集的 Y 轴 ADC 数值再次代入这个方程就得到了最终的物理坐标（实际的分辨率坐标）。

54.5.5 电阻屏触摸校准的实现

对 2 点和 4 点触摸校准原理有所了解后，再看代码部分就比较好理解了：

```
/*
*********************************************************************************************************
*    函 数 名: TOUCH_Calibration
*    功能说明: 触摸屏校准
*    形    参: _PointCount : 校准点数，2 或 4.
*    返 回 值: 无
*********************************************************************************************************
*/
void TOUCH_Calibration(uint8_t _PointCount)
{
    uint16_t usAdcX;
    uint16_t usAdcY;
    uint8_t usCount;
    uint8_t I;
    uint32_t n;

    /* 校准点数，2 点或 4 点 */
    if (_PointCount == 4)
    {
        g_tTPParam.CalibPointCount = 4;
    }
    else
    {
        g_tTPParam.CalibPointCount = 2;
    }

    TOUCH_CelarFIFO();      /* 清除无效的触摸事件 */

    for (I = 0; I < g_tTPParam.CalibPointCount; i++)
    {
        TOUCH_DispPoint(i);         /* 显示校准点 */

        TOUCH_WaitRelease();    /* 等待触笔释放 */

        usCount = 0;
        for (n = 0; n < 500; n++)
        {
            usAdcX = TOUCH_ReadAdcX();
            usAdcY = TOUCH_ReadAdcY();

            if (TOUCH_PressValid(usAdcX, usAdcY))
            {
                if (++usCount > 5)
```

```
                    {
                        /* 按压有效，保存校准点 ADC 采样值 */
                        if (I == 0)
                        {
                            g_tTPParam.usAdcX1 = usAdcX;
                            g_tTPParam.usAdcY1 = usAdcY;
                        }
                        else if (I == 1)
                        {
                            g_tTPParam.usAdcX2 = usAdcX;
                            g_tTPParam.usAdcY2 = usAdcY;
                        }
                        else if (I == 2)
                        {
                            g_tTPParam.usAdcX3 = usAdcX;
                            g_tTPParam.usAdcY3 = usAdcY;
                        }
                        else
                        {
                            g_tTPParam.usAdcX4 = usAdcX;
                            g_tTPParam.usAdcY4 = usAdcY;
                        }
                        break;
                    }
                }
                else
                {
                    usCount = 0;
                }
                bsp_DelayMS(10);
            }
            if (n == 500)
            {
                return;
            }
        }

        TOUCH_WaitRelease();    /* 等待触笔释放 */

        /* 识别触摸的 X，Y 和 显示面板的 X，Y 是否需要交换 */
        g_tTPParam.XYChange = 0;          /* 1 表示 X Y 需要交换 */
        if (LCD_GetHeight() < LCD_GetWidth())
        {
           if (TOUCH_Abs(g_tTPParam.usAdcX1 - g_tTPParam.usAdcX2) <
                TOUCH_Abs(g_tTPParam.usAdcY1 - g_tTPParam.usAdcY2))
            {
                g_tTPParam.XYChange = 1;
            }
        }
        else
        {
            if (TOUCH_Abs(g_tTPParam.usAdcX1 - g_tTPParam.usAdcX2) >
                TOUCH_Abs(g_tTPParam.usAdcY1 - g_tTPParam.usAdcY2))
            {
                g_tTPParam.XYChange = 1;
            }
        }

```

```
102.      g_tTPParam.usLcdX1 = TP_X1;
103.      g_tTPParam.usLcdY1 = TP_Y1;
104.      g_tTPParam.usLcdX2 = TP_X2;
105.      g_tTPParam.usLcdY2 = TP_Y2;
106.      g_tTPParam.usLcdX3 = TP_X3;
107.      g_tTPParam.usLcdY3 = TP_Y3;
108.      g_tTPParam.usLcdX4 = TP_X4;
109.      g_tTPParam.usLcdY4 = TP_Y4;
110.
111.      /* 在最后一步，将校准参数保存入 Flash 或者 EEPROM */
112.      TOUCH_SaveParam();
113. }
```

下面将程序设置中的关键地方做个阐释：

- 第 18-25 行，用于标记是 4 点触摸校准还是 2 点触摸校准。
- 第 31 行，显示触摸校准点，2 点触摸校准的话，显示左上角和右下角的校准点位置。4 点触摸校准的话，显示左上角，右上角，左下角和右下角的校准点位置。
- 第 33 行，用于等待触摸笔释放，当校准完毕 1 个点后，等待释放时使用。
- 第 35-79 行，读取 500 次 X 轴和 Y 轴的 ADC 数值，每 10ms 读取 1 次，每个触摸点的最大读取时间就是 5 秒。如果 5 秒内还没有触摸就会进入到第 77 行直接退出触摸校准。

  第 41 行检测到有按下的话，会连续读取 5 次，确保已经按下了，然后标记本次按下得到的 ADC 数值。根据执行的是 4 点触摸校准还是 2 点触摸校准，这个操作会执行 4 次或者 2 次。
- 第 84-100 行，这里涉及到一个知识点，即 X 轴镜像，Y 轴镜像和 XY 交换的处理办法，详情在此贴进行了说明：http://www.armbbs.cn/forum.php?mod=viewthread&tid=93300 。
- 第 112 行，将校准后的参数存储到 EEPROM 里面，下次开发板上电可以直接从 EEPROM 里面读取校准参数。

54.5.6 电阻屏触摸 ADC 值转物理坐标

电阻屏触摸 ADC 值转物理坐标的公式就是由前面 5.4 和 5.6 小节而来。

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: CalTwoPoint
4.  *    功能说明: 根据 2 点直线方程，计算 Y 值
5.  *    形    参:  2 个点的坐标和 x 输入量
6.  *    返 回 值: x 对应的 y 值
7.  *********************************************************************************************************
8.  */
9.  static int32_t CalTwoPoint(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2, uint16_t x)
10. {
11.     return y1 + ((int32_t)(y2 - y1) * (x - x1)) / (x2 - x1);
12. }
13.
14. /*
15. *********************************************************************************************************
16. *    函 数 名: TOUCH_TransX
17. *    功能说明: 将触摸 ADC 值转换为像素坐标
```

```
*    形    参: 无
*    返 回 值: X 坐标值，允许负值
*********************************************************************************************************
*/
static int16_t TOUCH_TransX(uint16_t _usAdcX, uint16_t _usAdcY)
{
    if (g_tTPParam.CalibPointCount == 2)
    {
        uint16_t x;
        int32_t y;

        if (g_tTPParam.XYChange == 0)
        {
            x = _usAdcX;
            if (x == 0)
            {
                y = 0;
            }
            else
            {
                //y = CalTwoPoint(g_tTPParam.usAdcX1, TP_X1, g_tTPParam.usAdcX2, TP_X2, x);
                y = CalTwoPoint(g_tTPParam.usAdcX1, g_tTPParam.usLcdX1, g_tTPParam.usAdcX2,
                                g_tTPParam.usLcdX2, x);
            }
        }
        else
        {
            x = _usAdcY;
            if (x == 0)
            {
                y = 0;
            }
            else
            {
                //y = CalTwoPoint(g_tTPParam.usAdcY1, TP_X1, g_tTPParam.usAdcY2, TP_X2, x);
                y = CalTwoPoint(g_tTPParam.usAdcY1, g_tTPParam.usLcdX1, g_tTPParam.usAdcY2,
                                g_tTPParam.usLcdX2, x);
            }
        }
        return y;
    }
    else /* 4 点校准 */
    {
        uint16_t x, x1, x2;
        int32_t y;

        if (g_tTPParam.XYChange == 0)    /* X Y 坐标不交换 */
        {
            x = _usAdcX;

            /* 根据 Y ADC 实时计算直线方程的参考点 x1, x2
                if  _usAdcY = usAdcY1 then  取点 = (AdcX1, TP_X1, AdcX4, TP_X4, _usAdcY)
                if  _usAdcY = usAdcY2 then  取点 = (AdcX3, TP_X3, AdcX2, TP_X2, _usAdcY)

                其中 TP_X1 = TP_X3;  TP_X4 = TP_X1 , 这是程序设定的校准位置的像素坐标, 是固定的。
                我们仅需要动态计算对第 1 个和第 3 个参数。同样采用 2 点直线方程计算。
            */
            x1 = CalTwoPoint(g_tTPParam.usAdcY1, g_tTPParam.usAdcX1, g_tTPParam.usAdcY2,
```

```
76.                                 g_tTPParam.usAdcX3, _usAdcY);
77.                 x2 = CalTwoPoint(g_tTPParam.usAdcY1, g_tTPParam.usAdcX4, g_tTPParam.usAdcY2,
78.                                 g_tTPParam.usAdcX2, _usAdcY);
79.             }
80.             else                            /* X Y 坐标交换 */
81.             {
82.                 x = _usAdcY;
83.
84.                 /* 根据 X ADC 实时计算直线方程的参考点 x1, x2
85.                     if  _usAdcX = usAdcX1 then  取点 = (AdcY1, TP_X1, AdcY4, TP_X4, _usAdcX)
86.                     if  _usAdcX = usAdcX2 then  取点 = (AdcY3, TP_X3, AdcY2, TP_X2, _usAdcX)
87.
88.                     其中 TP_X1 = TP_X3;  TP_X4 = TP_X1 , 这是程序设定的校准位置的像素坐标, 是固定的。
89.                     我们仅需要动态计算对第 1 个和第 3 个参数。同样采用 2 点直线方程计算。
90.                 */
91.                 x1 = CalTwoPoint(g_tTPParam.usAdcX1, g_tTPParam.usAdcY1, g_tTPParam.usAdcX2,
92.                                 g_tTPParam.usAdcY3, _usAdcX);
93.                 x2 = CalTwoPoint(g_tTPParam.usAdcX1, g_tTPParam.usAdcY4, g_tTPParam.usAdcX2,
94.                                 g_tTPParam.usAdcY2, _usAdcX);
95.             }
96.
97.             if (x == 0)
98.             {
99.                 y = 0;
100.            }
101.            else
102.            {
103.                /* 根据 2 点直线方程，计算坐标 */
104.                //y = CalTwoPoint(x1, TP_X1, x2, TP_X2, x);
105.                y = CalTwoPoint(x1, g_tTPParam.usLcdX1, x2, g_tTPParam.usLcdX2, x);
106.            }
107.            return y;
108.        }
109. }
110.
111. /*
112. *********************************************************************************************************
113. *    函 数 名: TOUCH_TransY
114. *    功能说明: 将触摸 ADC 值转换为像素坐标
115. *    形    参:  无
116. *    返 回 值: Y 坐标值，允许负值
117. *********************************************************************************************************
118. */
119. static int16_t TOUCH_TransY(uint16_t _usAdcX, uint16_t _usAdcY)
120. {
121.     if (g_tTPParam.CalibPointCount == 2) /* 2 点校准 */
122.     {
123.         /* 类似函数 TOUCH_TransX，省略未贴出  */
124.     }
125.     else /* 4 点校准 */
126.     {
127.         /* 类似函数 TOUCH_TransX，省略未贴出  */
128.     }
129. }
```

下面将程序设计中几个关键地方做个阐释：

◆ 第 9-12 行，y =ax+b 类型的直线方程，根据前四个参数输入的两个坐标点可以确定一条直线，然后

输入第 5 个参数 x，可以得到此坐标点对应的 y 值。

- 第 22 行，函数 TOUCH_TransX 的作用是将 X 轴采集的 ADC 值转换为物理坐标值。
- 第 24-58 行，用于处理 2 点触摸校准，对应的公式就是本章 5.4 小节的内容。
- 第 59-108 行，用于处理 4 点触摸校准，对应的公式就是本章 5.5 小节的内容。
- 第 119-129 行，函数 TOUCH_TransY 的作用是将 Y 轴采集的 ADC 值转换为物理坐标值。

这里注意 g_tTPParam.XYChange = 1 情况的处理，之所以会有这种情况，详情看此贴：http://www.armbbs.cn/forum.php?mod=viewthread&tid=93300 。

54.5.7 电阻触摸的使用方法

电阻触摸的使用主要分为三步：

- 第 1 步，使能触摸，在 bsp.c 文件的函数 bsp_Init 里面实现。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 省略未写 */

    bsp_InitI2C();      /* 初始化 I2C 总线 */
    TOUCH_InitHard();   /* 初始化触摸芯片，LCD 面板型号的检查也在此函数，所以要在函数 LCD_InitHard 前调用 */
    LCD_InitHard();     /* 初始化 LCD */
}
```

- 第 2 步，每毫秒调用 1 次触摸扫描。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer1ms
*    功能说明: 该函数每隔 1ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些需要周期性处理
*              的事务可以放在此函数。比如：触摸坐标扫描。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer1ms(void)
{
    TOUCH_Scan(); /* 触摸屏 */
}
```

- 第 3 步，用户程序里面采用下面的框架进行触摸消息处理。

TOUCH_DOWN 表示按下消息。

TOUCH_MOVE 表示触摸移动消息。

TOUCH_RELEASE 表示触摸释放消息。

根据这几个消息，用户可以在程序里面判断当前获取的物理坐标值是否在设置的区域内来执行触摸操作。

```
int16_t tpX, tpY;
uint8_t ucTouch;        /* 触摸事件 */

ucTouch = TOUCH_GetKey(&tpX, &tpY);  /* 读取触摸事件 */
if (ucTouch != TOUCH_NONE)
{
    switch (ucTouch)
    {
        case TOUCH_DOWN:        /* 触笔按下事件 */

            /* 在触笔所在位置显示一个小圈 */
            if ((tpX > 0) && (tpY > 0))
            {

            }
            break;

        case TOUCH_MOVE:        /* 触笔移动事件 */
            /* 实时刷新触摸 ADC 采样值和转换后的坐标 */
            {
                /* 在触笔所在位置显示一个小圈 */
                if ((tpX > 0) && (tpY > 0))
                {

                }
            }
            break;

        case TOUCH_RELEASE:         /* 触笔释放事件 */
            /* 在触笔所在位置显示一个小圈 */
            if ((tpX > 0) && (tpY > 0))
            {

            }
            break;
    }
}
```

54.6 电容触摸驱动设计

电容触摸相比电阻触摸就要简单很多了，因为电容触摸不需要做触摸校准，而且用的是触摸板和触摸芯片一体的，也不需要做寄存器初始化配置，上电后直接读取参数即可。

由于 GT811 已经停产，这里重点把 GT911 和 FT5X06 做个说明。

54.6.1 电容屏触摸 IC---FT5X06

电容触摸 IC 是支持多点触摸的，FT5X06 支持多达 10 点触摸同时按下，并提供了 I2C 和 SPI 两种

通信接口方式，开发板使用的是 I2C 通信接口。更多相关知识学习可以在这里下载 FT5X06 数据手册和应用手册：http://bbs.armfly.com/read.php?tid=16461 。

注意，这个芯片返回的就是实际的坐标值，比如显示屏的分辨率是 800*480，那么返回的就是在这个分辨率范围内的实际坐标，然后通过函数 TOUCH_PutKey 将 FT5X06 读出的实际坐标值存储到 FIFO 中即可，具体实现代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: FT5X06_Scan
*    功能说明: 读取触摸数据。读取全部的数据。放在主程序 bsp_Idle()中执行
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void FT5X06_Scan(void)
{
    uint8_t buf[CFG_POINT_READ_BUF];
    uint8_t i;
    static uint8_t s_tp_down = 0;
    uint16_t x, y;
    static uint16_t x_save, y_save;
    static uint8_t s_count = 0;

    if (g_tFT5X06.Enable == 0)
    {
        return;
    }

    /* 10ms 执行一次 */
    if (g_tFT5X06.TimerCount < 10)
    {
        return;
    }

    g_tFT5X06.TimerCount = 0;

#if 1    /* 方案 1: 检测 INT 引脚电平. */
    if (TOUCH_PenInt() == 0)
    {
#else    /* 方案 2：不用 INT 引脚，读状态寄存器 */
    FT5X06_ReadReg(2, buf, 1);
    if ((buf[0] & 0x07) == 0)
    {
#endif
        /* 持续按下时，INT 电平是脉冲信号。每隔 18ms 出现 1 个宽度 4ms 的高电平。 */
        if (s_tp_down == 1)
        {
            if (++s_count > 2)
            {
                s_count = 0;
                s_tp_down = 0;
                TOUCH_PutKey(TOUCH_RELEASE, x_save, y_save);
            }
        }
        return;
```

```
50.       }
51.       s_count = 0;
52.
53.       /* 有触摸，读取完整的数据 */
54.       FT5X06_ReadReg(0, buf, CFG_POINT_READ_BUF);
55.
56.       g_tFT5X06.Count = buf[2] & 0x07;
57.       if (g_tFT5X06.Count > FT5X06_TOUCH_POINTS)
58.       {
59.           g_tFT5X06.Count = FT5X06_TOUCH_POINTS;
60.       }
61.
62.       g_tFT5X06.Count = 0;
63.       for (i = 0; i < FT5X06_TOUCH_POINTS; i++)
64.       {
65.           uint8_t pointid;
66.
67.           pointid = (buf[5 + 6*i]) >> 4;
68.           if (pointid >= 0x0f)
69.           {
70.               break;
71.           }
72.           else
73.           {
74.             g_tFT5X06.Count++;
75.             g_tFT5X06.X[i] = (int16_t)(buf[3 + 6*i] & 0x0F)<<8 | (int16_t)buf[4 + 6*i];
76.             g_tFT5X06.Y[i] = (int16_t)(buf[5 + 6*i] & 0x0F)<<8 | (int16_t)buf[6 + 6*i];
77.             g_tFT5X06.Event[i] = buf[0x3 + 6*i] >> 6;
78.             g_tFT5X06.id[i] = (buf[5 + 6*i])>>4;
79.           }
80.       }
81.
82.       /* 检测按下 */
83.       {
84.           if (g_tFT5X06.ChipID == 0x55)        /* 4.3 寸 480 * 272 */
85.           {
86.               x = g_tFT5X06.Y[0];
87.               y = g_tFT5X06.X[0];
88.
89.               /* 判断值域 */
90.               if (x > 479)
91.               {
92.                   x = 479;
93.               }
94.
95.               if (y > 271)
96.               {
97.                   y = 271;
98.               }
99.           }
100.          else if (g_tFT5X06.ChipID == 0x0A)    /* 5.0 寸 800 * 480 */
101.          {
102.              x = g_tFT5X06.X[0];
103.              y = g_tFT5X06.Y[0];
104.
105.              /* 判断值域 */
106.              if (x > 799)
107.              {
```

```
108.                    x = 799;
109.                }
110.                if (y > 479)
111.                {
112.                    y = 479;
113.                }
114.            }
115.            else /* id == 0x06 表示 7 寸电容屏（FT 芯片） */
116.            {
117.                x = g_tFT5X06.X[0];
118.                y = g_tFT5X06.Y[0];
119.
120.                /* 判断值域 */
121.                if (x > 799)
122.                {
123.                    x = 799;
124.                }
125.                if (y > 479)
126.                {
127.                    y = 479;
128.                }
129.            }
130.        }
131.
132.        if (s_tp_down == 0)
133.        {
134.            s_tp_down = 1;
135.
136.            TOUCH_PutKey(TOUCH_DOWN, x, y);
137.        }
138.        else
139.        {
140.            TOUCH_PutKey(TOUCH_MOVE, x, y);
141.        }
142.        x_save = x;    /* 保存坐标，用于释放事件 */
143.        y_save = y;
144.
145. #if 0
146.        for (i = 0; i < CFG_POINT_READ_BUF; i++)
147.        {
148.            printf("%02X ", buf[i]);
149.        }
150.        printf("\r\n");
151. #endif
152.
153. #if 0  /* 打印 5 个坐标点数据 */
154.        printf("(%5d,%5d,%3d,%3d) ",  g_tFT5X06.X[0], g_tFT5X06.Y[0], g_tFT5X06.Event[0],
155.                                      g_tFT5X06.id[0]);
156.        printf("(%5d,%5d,%3d,%3d) ",  g_tFT5X06.X[1], g_tFT5X06.Y[1], g_tFT5X06.Event[1],
157.                                      g_tFT5X06.id[1]);
158.        printf("(%5d,%5d,%3d,%3d) ",  g_tFT5X06.X[2], g_tFT5X06.Y[2], g_tFT5X06.Event[2],
159.                                       g_tFT5X06.id[2]);
160.        printf("(%5d,%5d,%3d,%3d) ",  g_tFT5X06.X[3], g_tFT5X06.Y[3], g_tFT5X06.Event[3],
161.                                       g_tFT5X06.id[3]);
162.        printf("(%5d,%5d,%3d,%3d) ",  g_tFT5X06.X[4], g_tFT5X06.Y[4], g_tFT5X06.Event[4],
163.                                      g_tFT5X06.id[4]);
164.        printf("\r\n");
165. #endif
```

```
166. }
```

下面将程序设计中几个关键地方做个阐释：

- 第 32 行，通过判断 FT5X06 的中断输出引脚的高低电平来判断触摸板是否被按下，如果有触摸，这个引脚输出低电平，反之，输出高电平。通过判断这个引脚可以避免不必要的操作。
- 第 35-36 行，从寄存器 2 读取一个数据，判断是否有触摸数据，这种方式就没有直接判断中断引脚方便。
- 第 40-50 行，如果没有触摸数据，而且变量标志 s_tp_down = 1（此变量等于 1 表示之前处于触摸按下或者移动状态，如果等于 0 表示之前处于未被触摸状态），那么此时要通过函数 TOUCH_PutKey 存储松手消息，这一步比较重要，切不可省略。如果变量标志 s_tp_down = 0，直接退出即可。另外，特别注意一点，这里是通过 s_count 变量连续记录到两次松手消息才执行。
- 第 50-80 行，如果有触摸，将所有触摸值全部记录下来（由于支持多点触摸，会有多个触摸值）。
- 第 83-130 行，根据不同的分辨率的显示屏，做值域范围处理，防止超出范围。
- 第 132-137 行，如果变量 s_tp_down = 0 表示之前处于未被触摸状态，这里设置此变量为 1，并通过函数 TOUCH_PutKey 存储按下消息和当前的坐标值。
- 第 138-141 行，如果变量不等于 0（其实这里也就是 1）表示之前处于按下状态，此时触摸处于移动状态，这里不用重复设置此变量了，但通过函数 TOUCH_PutKey 存储按下消息和当前的坐标值。
- 第 153-165 行，这里预留的条件编译主要是方便调试阶段使用。

54.6.2 电容屏触摸 IC---GT911

GT911 支持多达 5 点触摸同时按下，并提供了 I2C 通信接口方式，更多相关知识学习可以在这里下载 GT911 数据手册：http://www.armbbs.cn/forum.php?mod=viewthread&tid=87205 。

GT911 的使用基本跟 FT5X06 是一样（注意，芯片 GT911 返回的就是实际的坐标值，比如显示屏的分辨率是 800*480，那么返回的就是在这个分辨率范围内的实际坐标，跟 FT5X06 也是一样的），具体实现代码如下（这里没有使用多点触摸功能，仅存了一个触摸值，一般情况下已经够用）：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: GT911_Scan
4.  *    功能说明: 读取 GT911 触摸数据。读取全部的数据，需要 720us 左右。放在 bsp_Idle()中执行
5.  *    形    参: 无
6.  *    返 回 值: 无
7.  *********************************************************************************************************
8.  */
9.  void GT911_Scan(void)
10. {
11.     uint8_t buf[48];
12.     static uint8_t s_tp_down = 0;
13.     uint16_t x, y;
14.     static uint16_t x_save, y_save;
15.     uint8_t clear_flag = 0;
16.     static uint8_t s_count = 0;
17.
```

```
    if (g_GT911.Enable == 0)
    {
        return;
    }

    /* 10ms 执行一次 */
    if (g_GT911.TimerCount < 10)
    {
        return;
    }

    g_GT911.TimerCount = 0;

#if 1    /* 方案 1: 检测 INT 引脚电平. */
    if (TOUCH_PenInt() == 0)
    {
#else    /* 方案 2：不用 INT 引脚，读状态寄存器 */
    GT911_ReadReg(GT911_READ_XY_REG, buf, 1);
    if (buf[0] == 0)
    {
#endif
        if (s_tp_down == 1)
        {
            if (++s_count > 1)
            {
                s_count = 0;
                s_tp_down = 0;
                TOUCH_PutKey(TOUCH_RELEASE, x_save, y_save);
            }
        }
        return;
    }
    s_count = 0;

#if 1        /* 一般应用只读 1 点 */
    GT911_ReadReg(GT911_READ_XY_REG, buf, 8);
#else        /* 读 5 个触摸点 */
    GT911_ReadReg(GT911_READ_XY_REG, buf, 40);
#endif

    GT911_WriteReg(GT911_READ_XY_REG, &clear_flag, 1);      /* 读完坐标后必须写 0 清除 */

    /*
        0x814E R/W Bufferstatus Large_Detect number of touch points
            Bit7: Buffer status，1 表示坐标（或按键）已经准备好，主控可以读取；0 表示未就绪，数据无效。
                  当主控读取完坐标后，必须通过 I2C 将此标志（或整个字节）写为 0。
            Bit4: HaveKey, 1 表示有按键，0 表示无按键（已经松键）。
            Bit3~0: Number of touch points, 屏上的坐标点个数

        0x814F R Point1 track id
        0x8150 R Point1Xl 触摸点 1，X 坐标低 8 位
        0x8151 R Point1Xh 触摸点 1，X 坐标高 8 位
        0x8152 R Point1Yl 触摸点 1，Y 坐标低 8 位
        0x8153 R Point1Yh 触摸点 1，Y 坐标高 8 位
        0x8154 R Point1 触摸点 1，触摸面积低 8 位
        0x8155 R Point1 触摸点 1，触摸面积高 8 位
        0x8156 ----

```

```
76.           0x8157 R Point2 track id
77.           0x8158 R Point2Xl 触摸点 2，X 坐标低 8 位
78.           0x8159 R Point2Xh 触摸点 2，X 坐标高 8 位
79.           0x815A R Point2Yl 触摸点 2，Y 坐标低 8 位
80.           0x815B R Point2Yh 触摸点 2，Y 坐标高 8 位
81.           0x815C R Point2 触摸点 2，触摸面积低 8 位
82.           0x815D R Point2 触摸点 2，触摸面积高 8 位
83.           0x815E ----
84.
85.           0x815F R Point3 track id
86.           0x8160 R Point3Xl 触摸点 3，X 坐标低 8 位
87.           0x8161 R Point3Xh 触摸点 3，X 坐标高 8 位
88.           0x8162 R Point3Yl 触摸点 3，Y 坐标低 8 位
89.           0x8163 R Point3Yh 触摸点 3，Y 坐标高 8 位
90.           0x8164 R Point3 触摸点 3，触摸面积低 8 位
91.           0x8165 R Point3 触摸点 3，触摸面积高 8 位
92.           0x8166 ----
93.
94.           0x8167 R Point4 track id
95.           0x8168 R Point4Xl 触摸点 4，X 坐标低 8 位
96.           0x8169 R Point4Xh 触摸点 4，X 坐标高 8 位
97.           0x816A R Point4Yl 触摸点 4，Y 坐标低 8 位
98.           0x816B R Point4Yh 触摸点 4，Y 坐标高 8 位
99.           0x816C R Point4 触摸点 4，触摸面积低 8 位
100.          0x816D R Point4 触摸点 4，触摸面积高 8 位
101.          0x816E ----
102.
103.          0x816F R Point5 track id
104.          0x8170 R Point5Xl 触摸点 5，X 坐标低 8 位
105.          0x8171 R Point5Xh 触摸点 5，X 坐标高 8 位
106.          0x8172 R Point5Yl 触摸点 5，Y 坐标低 8 位
107.          0x8173 R Point5Yh 触摸点 5，Y 坐标高 8 位
108.          0x8174 R Point5 触摸点 5，触摸面积低 8 位
109.          0x8175 R Point5 触摸点 5，触摸面积高 8 位
110.          0x8176 --
111.
112.      */
113.      g_GT911.TouchpointFlag = buf[0];
114.      g_GT911.Touchkeystate = buf[1];
115.
116.      g_GT911.X0 = ((uint16_t)buf[3] << 8) + buf[2];
117.      g_GT911.Y0 = ((uint16_t)buf[5] << 8) + buf[4];
118.      g_GT911.P0 = ((uint16_t)buf[7] << 8) + buf[6];
119.
120.      #if 0    /* 其余 4 点一般不用 */
121.          g_GT911.X1 = ((uint16_t)buf[9] << 8) + buf[10];
122.          g_GT911.Y1 = ((uint16_t)buf[11] << 8) + buf[12];
123.          g_GT911.P1 = ((uint16_t)buf[13] << 8) + buf[14];
124.
125.          g_GT911.X2 = ((uint16_t)buf[17] << 8) + buf[16];
126.          g_GT911.Y2 = ((uint16_t)buf[19] << 8) + buf[18];
127.          g_GT911.P2 = ((uint16_t)buf[21] << 8) + buf[20];
128.
129.          g_GT911.X3 = ((uint16_t)buf[24] << 8) + buf[23];
130.          g_GT911.Y3 = ((uint16_t)buf[26] << 8) + buf[25];
131.          g_GT911.P3 = ((uint16_t)buf[28] << 8) + buf[27];
132.
133.          g_GT911.X4 = ((uint16_t)buf[31] << 8) + buf[30];
```

```
134.          g_GT911.Y4 = ((uint16_t)buf[33] << 8) + buf[32];
135.          g_GT911.P4 = ((uint16_t)buf[35] << 8) + buf[34];
136.      #endif
137.
138.      /* 检测按下 */
139.      {
140.          /* 坐标转换 :
141.               电容触摸板左下角是 (0, 0);  右上角是 (479, 799)
142.               需要转到 LCD 的像素坐标 (左上角是 (0, 0), 右下角是 (799, 479)
143.          */
144.          {
145.               x = g_GT911.X0;
146.               y = g_GT911.Y0;
147.
148.               if (x > 799)
149.               {
150.                    x = 799;
151.               }
152.
153.               if (y > 479)
154.               {
155.                    y = 479;
156.               }
157.          }
158.      }
159.
160.      if (s_tp_down == 0)
161.      {
162.          s_tp_down = 1;
163.
164.          TOUCH_PutKey(TOUCH_DOWN, x, y);
165.      }
166.      else
167.      {
168.          TOUCH_PutKey(TOUCH_MOVE, x, y);
169.      }
170.      x_save = x;   /* 保存坐标，用于释放事件 */
171.      y_save = y;
172.
173. #if 0
174.      {
175.          uint8_t i;
176.
177.          for (i = 0; i < 34; i++)
178.          {
179.               printf("%02X ", buf[i]);
180.          }
181.          printf("\r\n");
182.
183.          printf("(%5d,%5d,%3d) ",  g_GT911.X0, g_GT911.Y0, g_GT911.P0);
184.          printf("(%5d,%5d,%3d) ",  g_GT911.X1, g_GT911.Y1, g_GT911.P1);
185.          printf("(%5d,%5d,%3d) ",  g_GT911.X2, g_GT911.Y2, g_GT911.P2);
186.          printf("(%5d,%5d,%3d) ",  g_GT911.X3, g_GT911.Y3, g_GT911.P3);
187.          printf("(%5d,%5d,%3d) ",  x, y, g_GT911.P4);
188.          printf("\r\n");
189.      }
190. #endif
191. }
```

下面将程序设计中几个关键地方做个阐释：

- 第 18-21 行，使能标志，在文件 bsp_touch.c 文件里面有一个函数 bsp_DetectLcdType 专门检测不同的触摸 IC，如果是 GT911 的话，变量 g_tFT5X06.Enable 会被置 1。
- 第 24-27 行，表示函数 GT911_Scan 每 10ms 执行 1 次。
- 第 32 行，通过判断 GT911 的中断输出引脚的高低电平来判断触摸板是否被按下，如果有触摸，这个引脚输出低电平，反之，输出高电平。通过判断这个引脚可以避免不必要的操作。
- 第 35-36 行，从寄存器 2 读取一个数据，判断是否有触摸数据，这种方式就没有直接判断中断引脚方便。
- 第 39-49 行，如果没有触摸数据，而且变量标志 s_tp_down = 1（此变量等于 1 表示之前处于触摸按下或者移动状态，如果等于 0 表示之前处于未被触摸状态），那么此时要通过函数 TOUCH_PutKey 存储松手消息，这一步比较重要，切不可省略。如果变量标志 s_tp_down = 0，直接退出即可。另外，特别注意一点，这里是通过 s_count 变量连续记录到两次松手消息才执行。
- 第 52-136 行，仅读取一组。这里没有使用多点触摸功能，仅存了一个触摸值，一般情况下已经够用。
- 第 138-158 行，对读取的值域范围处理，防止超出范围。
- 第 160-165 行，如果变量 s_tp_down = 0 表示之前处于未被触摸状态，这里设置此变量为 1，并通过函数 TOUCH_PutKey 存储按下消息和当前的坐标值。
- 第 166-169 行，如果变量不等于 0（其实这里也就是 1）表示之前处于按下状态，此时触摸处于移动状态，这里不用重复设置此变量了，但通过函数 TOUCH_PutKey 存储按下消息和当前的坐标值。
- 第 173-190 行，这里预留的条件编译主要是方便调试阶段使用。

54.6.3 电容触摸的使用方法

电容触摸的使用主要分为三步：

- 第 1 步，使能触摸，在 bsp.c 文件的函数 bsp_Init 里面实现。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 省略未写 */

    bsp_InitI2C();      /* 初始化 I2C 总线 */
    TOUCH_InitHard();   /* 初始化触摸芯片，LCD 面板型号的检查也在此函数，所以要在函数 LCD_InitHard 前调用 */
    LCD_InitHard();     /* 初始化 LCD */
}
```

- 第 2 步，每毫秒调用 1 次触摸扫描。对于电容屏来说，这个函数主要提供了一个计数器功能。并且需

要用户在主任务里面一直调用函数 bsp_Idle，这个函数主要实现电容触摸屏扫描。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer1ms
*    功能说明: 该函数每隔 1ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些需要周期性处理
*              的事务可以放在此函数。比如：触摸坐标扫描。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer1ms(void)
{
    TOUCH_Scan(); /* 触摸屏 */
}

/*
*********************************************************************************************************
*    函 数 名: bsp_Idle
*    功能说明: 空闲时执行的函数。一般主程序在 for 和 while 循环程序体中需要插入 CPU_IDLE() 宏来调用本函数。
*              本函数缺省为空操作。用户可以添加喂狗、设置 CPU 进入休眠模式的功能。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Idle(void)
{
    /* --- 喂狗 */

    /* --- 让 CPU 进入休眠，由 Systick 定时中断唤醒或者其他中断唤醒 */

    /* 例如 emWin 图形库，可以插入图形库需要的轮询函数 */
    //GUI_Exec();

    /* 例如 uIP 协议，可以插入 uip 轮询函数 */
    TOUCH_CapScan();
}
```

◆ 第 3 步，用户程序里面采用下面的框架进行触摸消息处理。

TOUCH_DOWN 表示按下消息。

TOUCH_MOVE 表示触摸移动消息。

TOUCH_RELEASE 表示触摸释放消息。

根据这几个消息，用户可以在程序里面判断当前获取的物理坐标值是否在设置的区域内来执行触摸操作。

```
int16_t tpX, tpY;
uint8_t ucTouch;        /* 触摸事件 */

ucTouch = TOUCH_GetKey(&tpX, &tpY);  /* 读取触摸事件 */
if (ucTouch != TOUCH_NONE)
{
    switch (ucTouch)
    {
        case TOUCH_DOWN:        /* 触笔按下事件 */

            /* 在触笔所在位置显示一个小圈 */
            if ((tpX > 0) && (tpY > 0))
            {
```

```
            }
            break;

        case TOUCH_MOVE:        /* 触笔移动事件 */
            /* 实时刷新触摸 ADC 采样值和转换后的坐标 */
            {
                /* 在触笔所在位置显示一个小圈 */
                if ((tpX > 0) && (tpY > 0))
                {

                }
            }
            break;

        case TOUCH_RELEASE:         /* 触笔释放事件 */
            /* 在触笔所在位置显示一个小圈 */
            if ((tpX > 0) && (tpY > 0))
            {

            }
            break;
    }
}
```

54.7 不同触摸 IC 的识别

由于 V7 开发板要做不同显示屏的自适应，所以关联了多个文件。自适应方法也比较简单，因为触摸芯片 GT911，GT811，FT5X06 和 STMPE811 都是 I2C 接口，通过 I2C 地址就区分开了，具体代码如下：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: bsp_DetectLcdType
4.  *    功能说明: 通过 I2C 触摸芯片，识别 LCD 模组类型。结果存放在全局变量 g_LcdType 和 g_TouchType
5.  *    形    参: 无
6.  *    返 回 值: 无
7.  *********************************************************************************************************
8.  */
9.  void bsp_DetectLcdType(void)
10. {
11.     uint8_t i;
12.
13.     g_TouchType = 0xFF;
14.     g_LcdType = 0xFF;
15.
16.     bsp_DelayUS(5000);
17.
18.     touch_printf("正在识别触摸屏型号\r\n");
19.
20.     /* 50ms，等待 GT811 复位就绪，才能探测 GT811 芯片 ID */
21.     for (i = 0; i < 5; i++)
22.     {
23.         /*
24.             GT811 电容触摸板和 GT911 的 I2C 地址相同
25.             一般就 0x28 、 0xBA 两种。
26.             通过读取触摸 IC 的芯片 ID 来识别。
```

```
27.         */
28.         if (i2c_CheckDevice(0x28) == 0)
29.         {
30.             uint32_t id;
31.
32.             bsp_DelayUS(500);
33.             g_GT911.i2c_addr = 0x28;
34.             id = GT911_ReadID();
35.             if (id == 0x00313139)
36.             {
37.                 g_GT911.i2c_addr = 0x28;
38.                 g_TouchType = CT_GT911;
39.                 g_LcdType = LCD_70_800X480;
40.                 touch_printf("检测到 7.0 寸电容触摸屏 GT911(0x28) 800x480\r\n");
41.             }
42.             else     /* GT811 */
43.             {
44.                 g_GT811.i2c_addr = 0x28;
45.                 g_TouchType = CT_GT811;
46.                 g_LcdType = LCD_70_800X480;
47.                 touch_printf("检测到 7.0 寸电容触摸屏 GT811(0x28) 800x480\r\n");
48.             }
49.             break;
50.         }
51.
52.         if (i2c_CheckDevice(0xBA) == 0)
53.         {
54.             uint32_t id;
55.
56.             bsp_DelayUS(500);
57.             g_GT911.i2c_addr = 0xBA;
58.             id = GT911_ReadID();
59.             if (id == 0x00313139)
60.             {
61.                 g_GT911.i2c_addr = 0xBA;
62.                 g_TouchType = CT_GT911;
63.                 g_LcdType = LCD_70_800X480;
64.                 touch_printf("检测到 7.0 寸电容触摸屏 GT911(0xBA) 800x480\r\n");
65.             }
66.             else     /* GT811 */
67.             {
68.                 g_GT811.i2c_addr = 0xBA;
69.                 g_TouchType = CT_GT811;
70.                 g_LcdType = LCD_70_800X480;
71.                 touch_printf("检测到 7.0 寸电容触摸屏 GT811(0xBA) 800x480\r\n");
72.             }
73.             break;
74.         }
75.
76.         /* FT 系列电容触摸: 4.3 寸 id = 0x55    5.0 寸 id = 0x0A  7.0 寸 id = 0x06 */
77.         if (i2c_CheckDevice(FT5X06_I2C_ADDR) == 0)
78.         {
79.             uint8_t id;
80.
81.             bsp_DelayUS(50000);     /* 延迟 50ms */
82.             id = FT5X06_ReadID();
83.             if (id == 0x55)
84.             {
```

```
85.                     g_TouchType = CT_FT5X06;
86.                     g_LcdType = LCD_43_480X272;
87.                     touch_printf("检测到 4.3 寸电容触摸屏\r\n");
88.                 }
89.                 else if (id == 0x0A)
90.                 {
91.                     g_TouchType = CT_FT5X06;
92.                     g_LcdType = LCD_50_800X480;
93.                     touch_printf("检测到 5.0 寸电容触摸屏\r\n");
94.                 }
95.                 else /* id == 0x06 表示 7 寸电容屏（FT 芯片） */
96.                 {
97.                     g_TouchType = CT_FT5X06;
98.                     g_LcdType = LCD_70_800X480;
99.                     touch_printf("检测到 7.0 寸电容触摸屏 FT\r\n");
100.                }
101.                break;
102.            }
103.
104.            /* 电阻触摸板 */
105.            if (i2c_CheckDevice(STMPE811_I2C_ADDRESS) == 0)
106.            {
107.                /*
108.                    0  = 4.3 寸屏（480X272）
109.                    1  = 5.0 寸屏（480X272）
110.                    2  = 5.0 寸屏（800X480）
111.                    3  = 7.0 寸屏（800X480）
112.                    4  = 7.0 寸屏（1024X600）
113.                    5  = 3.5 寸屏（480X320）
114.                */
115.                uint8_t id;
116.
117.                g_TouchType = CT_STMPE811;  /* 触摸类型 */
118.
119.                STMPE811_InitHard();    /* 必须先配置才能读取 ID */
120.
121.                id = STMPE811_ReadIO();/* 识别 LCD 硬件类型 */
122.
123.                touch_printf("检测到电阻触摸屏, id = %d\r\n", id);
124.                switch (id)
125.                {
126.                    case 0:
127.                        g_LcdType = LCD_43_480X272;
128.                        break;
129.
130.                    case 1:
131.                        g_LcdType = LCD_50_480X272;
132.                        break;
133.
134.                    case 2:
135.                        g_LcdType = LCD_50_800X480;
136.                        break;
137.
138.                    case 3:
139.                        g_LcdType = LCD_70_800X480;
140.                        break;
141.
142.                    case 4:
```

```
143.                        g_LcdType = LCD_70_1024X600;
144.                        break;
145.
146.                    case 5:
147.                        g_LcdType = LCD_35_480X320;
148.                        break;
149.
150.                    default:
151.                        g_LcdType = LCD_35_480X320;
152.                        break;
153.                }
154.                break;
155.            }
156.
157.            bsp_DelayMS(10);
158.        }
159.
160.        if (i == 5)
161.        {
162.            touch_printf("未识别出显示模块\r\n");
163.        }
164. }
```

下面将程序设计中几个关键地方做个阐释：

- 第 21 行，上电后，这几款传感器一定要延迟一段时间才可以正常读取 ID，所以这里做了一个 for 循环做检测。
- 第 28-74 行，GT811 电容触摸板和 GT911 的 I2C 地址相同，一般就 0x28 和 0xBA 两种。程序里面对这两个地址都做了检测，然后通过 ID 区分是 GT811 还是 GT911。
- 第 77-102 行，FT5X06 电容触摸板检测，通过 ID 区分不同分辨率。
- 第 105-158 行，STMPE811 电阻触摸屏检测，通过函数 STMPE811_ReadIO 返回的 IO 状态可以识别 3.5 寸 480*320 分辨率显示屏，4.3 寸 480*272 分辨率显示屏，5 寸 480*272 和 800*480 分辨率显示屏以及 7 寸 800*480 和 1024*600 分辨率显示屏。
- 第 160-163 行，用于调试。

54.8 LCD 触摸移植和使用

LCD 驱动的移植与第 51 章 51.7 小节相同，这里重点说下触摸部分的移植。

通过前面的讲解，移植触摸驱动到自己的板子上，最简单的办法是将开发板上与触摸相关的文件全部移植过来，然后在这些文件的基础上进行修改。下面分两种情况进行说明：

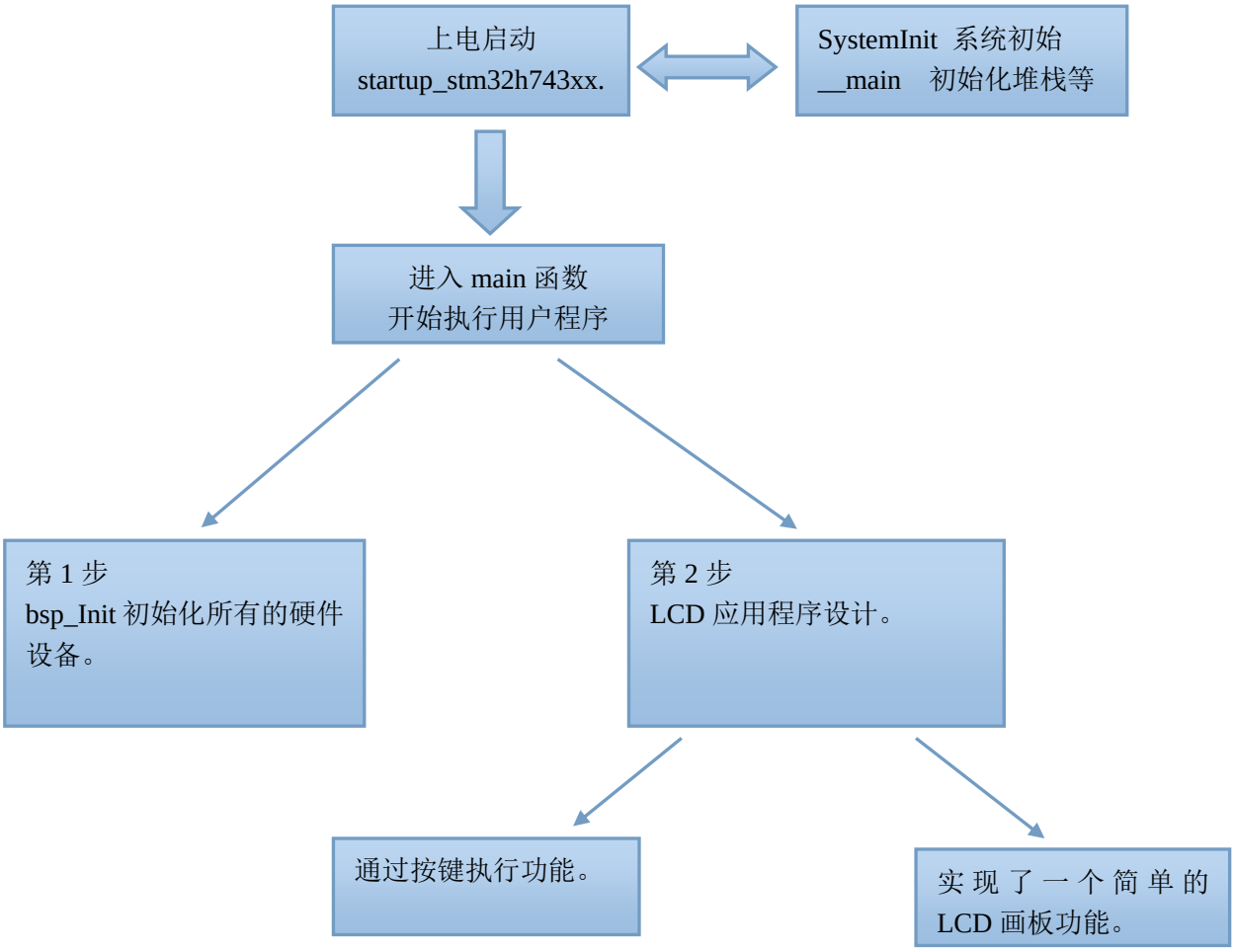
- 电容屏触摸的移植比较简单，如果用户用的触摸 IC 跟开发板一样，直接拿来用即可，如果不一样，需要先将触摸 IC 的驱动实现，然后按照开发板提供的 GT811、GT911 和 FT5X06 的触摸扫描函数，照葫芦画瓢实现一个即可。
- 电阻屏的移植稍麻烦些，如果用户用的触摸 IC 跟开发板一样，直接拿来用即可，如果不一样，需要先将触摸 IC 的驱动实现，然后仿照 TOUCH_STMPE811.c 文件提供触摸按下状态函数，X 轴，Y 轴的 ADC 数值读取函数和清除触摸中断标志函数。最后用重新实现的这几个函数替换 bsp_touch.c 文

件中的原函数即可。另外要注意一点，这种方式实现后，虽然触摸校准依然可以使用，但是开发板的触摸校准参数保存在 EEPROM 中，用户可以根据自己的实际情况选择存储介质。

当然，如果大家不想用开发板实现的方案，想自己重新实现一个，也是没问题的。

54.9 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED，串口，LCD，SDRAM 等。
  - 第 2 步，LCD 应用程序设计部分。通过按键执行功能，同时 LCD 界面做了一个简单的画板功能。

54.10 实验例程说明（MDK）

配套例子：

V7-034_LCD 的电阻触摸和电容触摸（电阻触摸支持 2 点和 4 点校准）

实验目的：

1. 学习 LCD 的电阻触摸和电容触摸。

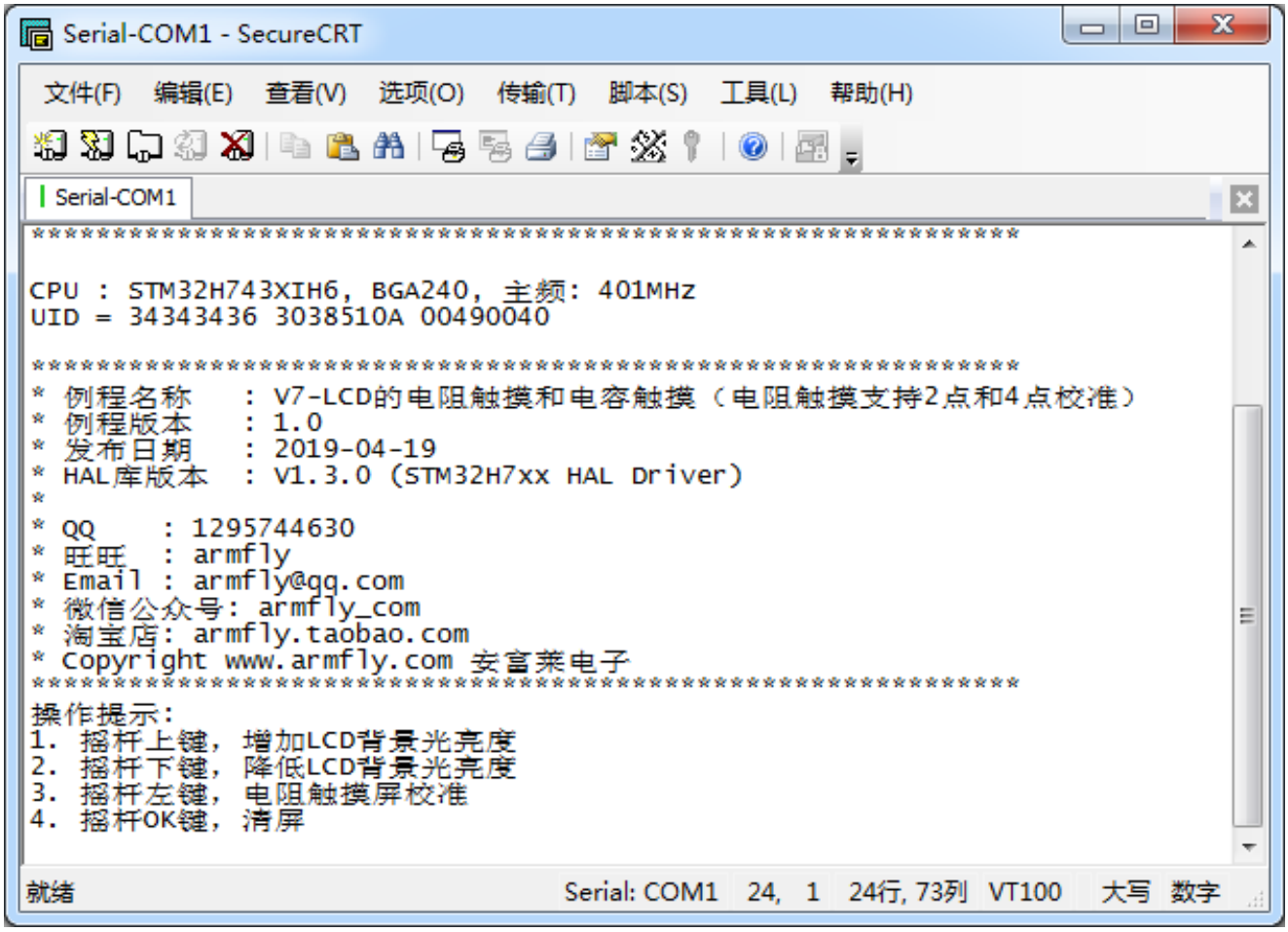
实验内容：

1. 电容屏无需校准，电阻屏需要校准。
2. LCD 界面实现了一个简单的画板功能，方便检测触摸是否准确。
3. 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。

LCD 界面显示效果如下：

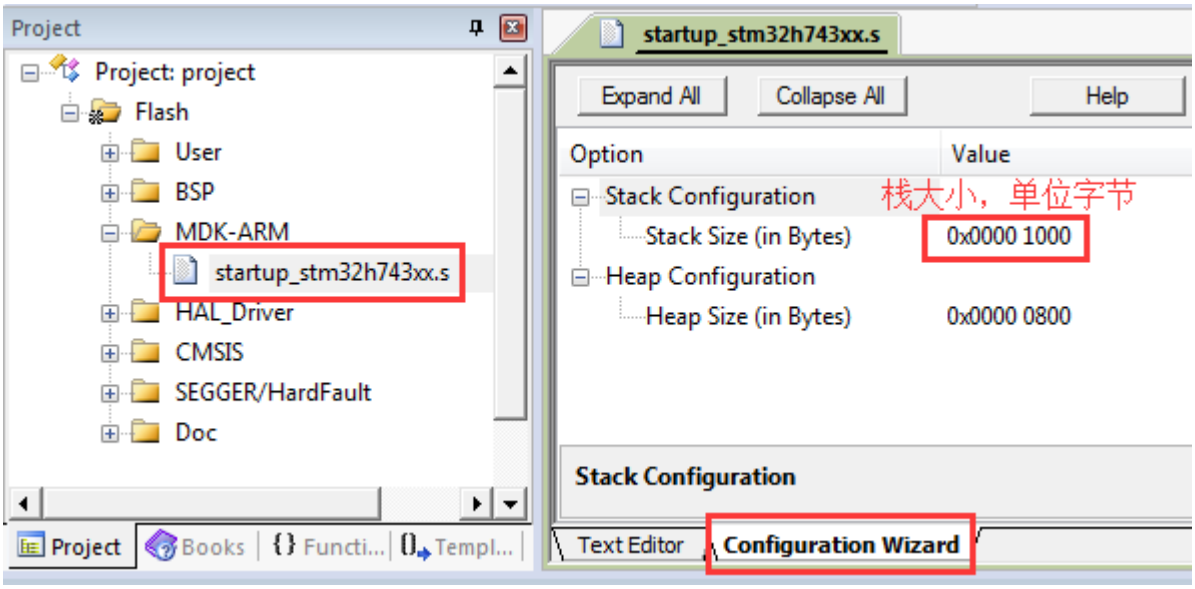
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

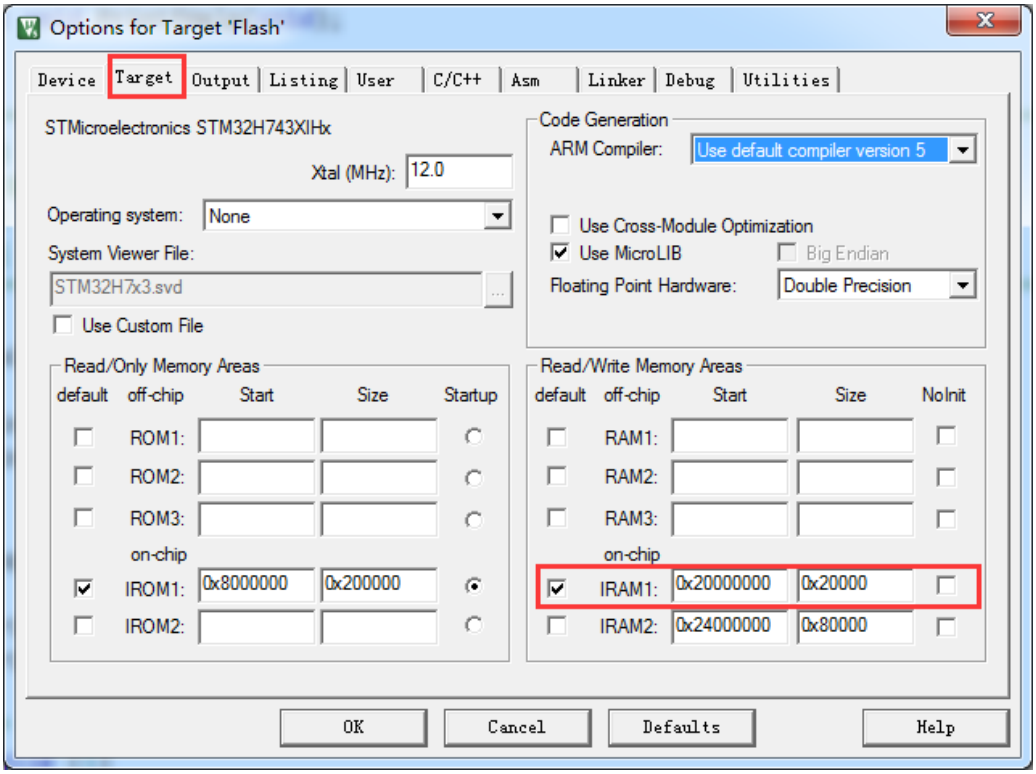


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */

    bsp_InitI2C();      /* 初始化 I2C 总线 */
    TOUCH_InitHard();   /* 初始化触摸芯片，LCD 面板型号的检查也在此函数，所以要在函数 LCD_InitHard 前调用 */
    LCD_InitHard();     /* 初始化 LCD */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SDRAM。由于 SDRAM 要用于 LCD 的显存，方便起见，直接将其配置为 WT 模式。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

```
    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SDRAM 的 MPU 属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0xC0000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_32MB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- LCD 界面实现了一个简单的画板功能，方便检测触摸是否准确。
- 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。

- 摇杆上键，增加 LCD 背景光亮度。
- 摇杆下键，降低 LCD 背景光亮度。
- 摇杆左键，电阻触摸屏校准。
- 摇杆 OK 键，清屏。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint16_t ucBright;      /* 背光亮度(0-255) */
    uint8_t ucKeyCode;      /* 按键代码 */
    uint8_t fRefresh;       /* 刷屏请求标志,1 表示需要刷新 */
   FONT_T tFont;           /* 定义一个字体结构体变量，用于设置字体参数 */
    char buf[64];
   uint16_t usAdcX, usAdcY;
    int16_t tpX, tpY;
   uint8_t ucTouch;        /* 触摸事件 */


    /* 设置字体参数 */
    {
        tFont.FontCode = FC_ST_16;       /* 字体代码 16 点阵 */
        tFont.FrontColor = CL_WHITE;     /* 字体颜色 */
        tFont.BackColor = CL_BLUE;       /* 文字背景颜色 */
        tFont.Space = 0;                 /* 文字间距，单位 = 像素 */
    }

    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    /* 延迟 200ms 再点亮背光，避免瞬间高亮 */
    bsp_DelayMS(200);

    DispFirstPage();   /* 显示第 1 页 */

    /* 界面整体显示完毕后，再打开背光，设置为缺省亮度 */
    bsp_DelayMS(100);
    ucBright = BRIGHT_DEFAULT;
    LCD_SetBackLight(ucBright);

    bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */

    /* 进入主程序循环体 */
    fRefresh = 1;
    while (1)
    {
      bsp_Idle();

        /* 判断软件定时器 0 是否超时 */
```

```
	if(bsp_CheckTimer(0))
	{
		/* 每隔 200ms 进来一次 */
		bsp_LedToggle(2);
	}

	if (fRefresh)
	{
		fRefresh = 0;

		/* 实时刷新触摸 ADC 采样值和转换后的坐标 */
		{

			/* 读取并显示当前 X 轴和 Y 轴的 ADC 采样值 */
			usAdcX = TOUCH_ReadAdcX();
			usAdcY = TOUCH_ReadAdcY();
			sprintf(buf, "触摸 ADC 值 X = %4d, Y = %4d   ", usAdcX, usAdcY);
			LCD_DispStr(5, 40, buf, &tFont);

			/* 读取并显示当前触摸坐标 */
			tpX = TOUCH_GetX();
			tpY = TOUCH_GetY();
			sprintf(buf, "触摸坐标  X = %4d, Y = %4d   ", tpX, tpY);
			LCD_DispStr(5, 60, buf, &tFont);

			/* 在触笔所在位置显示一个小圈 */
			if ((tpX > 0) && (tpY > 0))
			{
				LCD_DrawCircle(tpX, tpY, 2, CL_YELLOW);
			}
		}

		/* 在屏幕边沿绘制 2 个矩形框(用于检测面板边缘像素是否正常) */
		LCD_DrawRect(0, 0, LCD_GetHeight(), LCD_GetWidth(), CL_WHITE);
		LCD_DrawRect(2, 2, LCD_GetHeight() - 4, LCD_GetWidth() - 4, CL_YELLOW);

	   /* 显示背光值 */
		sprintf(buf, "当前背光亮度: %3d", ucBright);
		LCD_DispStr(5, 80, buf, &tFont);
	}

	ucTouch = TOUCH_GetKey(&tpX, &tpY);	/* 读取触摸事件 */
	if (ucTouch != TOUCH_NONE)
	{
		switch (ucTouch)
		{
			case TOUCH_DOWN:		/* 触笔按下事件 */

				/* 在触笔所在位置显示一个小圈 */
				if ((tpX > 0) && (tpY > 0))
				{
					LCD_DrawCircle(tpX, tpY, 3, CL_RED);
				}
				break;

			case TOUCH_MOVE:		/* 触笔移动事件 */
				/* 实时刷新触摸 ADC 采样值和转换后的坐标 */
				{
```

```
                    /* 读取并显示当前 X 轴和 Y 轴的 ADC 采样值 */
                    usAdcX = TOUCH_ReadAdcX();
                    usAdcY = TOUCH_ReadAdcY();
                    sprintf(buf, "触摸 ADC 值 X = %4d, Y = %4d   ", usAdcX, usAdcY);
                    LCD_DispStr(5, 40, buf, &tFont);

                    /* 读取并显示当前触摸坐标 */
                    //tpX = TOUCH_GetX();
                    //tpY = TOUCH_GetY();
                    sprintf(buf, "触摸坐标  X = %4d, Y = %4d   ", tpX, tpY);
                    LCD_DispStr(5, 60, buf, &tFont);

                    /* 在触笔所在位置显示一个小圈 */
                    if ((tpX > 0) && (tpY > 0))
                    {
                        LCD_DrawCircle(tpX, tpY, 2, CL_YELLOW);
                    }
                }
                break;

            case TOUCH_RELEASE:         /* 触笔释放事件 */
                /* 在触笔所在位置显示一个小圈 */
                if ((tpX > 0) && (tpY > 0))
                {
                    LCD_DrawCircle(tpX, tpY, 4, CL_WHITE);
                }
                break;
        }
    }

    ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
    if (ucKeyCode != KEY_NONE)
    {
        /* 有键按下 */
        switch (ucKeyCode)
        {
            case JOY_DOWN_L:        /* 摇杆 LEFT 键按下, 2 点触摸校准 */
                TOUCH_Calibration(2);
                DispFirstPage();
                fRefresh = 1;       /* 请求刷新 LCD */
                break;

            case  JOY_DOWN_OK:      /* 摇杆 OK 键, 清屏 */
                DispFirstPage();
                fRefresh = 1;       /* 请求刷新 LCD */
                break;

            case JOY_DOWN_U:        /* 摇杆 UP 键按下 */
                ucBright += BRIGHT_STEP;
                if (ucBright > BRIGHT_MAX)
                {
                    ucBright = BRIGHT_MAX;
                }
                LCD_SetBackLight(ucBright);
              fRefresh = 1;         /* 请求刷新 LCD */
                break;

            case JOY_DOWN_D:        /* 摇杆 DOWN 键按下 */
```

```
                    if (ucBright < BRIGHT_STEP)
                    {
                        ucBright = 0;
                    }
                    else
                    {
                        ucBright -= BRIGHT_STEP;
                    }
                    LCD_SetBackLight(ucBright);
                fRefresh = 1;         /* 请求刷新 LCD */
                    break;

                default:
                    break;
            }
        }
    }
}
```

LCD 主界面显示内容如下:

```
/*
*********************************************************************************************************
*    函 数 名: DispFirstPage
*    功能说明: 显示操作提示
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
static void DispFirstPage(void)
{
    FONT_T tFont;      /* 定义一个字体结构体变量，用于设置字体参数 */
    uint16_t y;            /* Y 坐标 */
    uint16_t usLineCap;    /* 行高 */
    char buf[50];

    LCD_ClrScr(CL_BLUE);        /* 清屏，背景蓝色 */


    /* 设置字体属性 */
    tFont.FontCode = FC_ST_16;       /* 字体选择宋体 16 点阵，高 16x 宽 15) */
    tFont.FrontColor = CL_WHITE;     /* 字体颜色设置为白色 */
    tFont.BackColor = CL_MASK;       /* 文字背景颜色，透明 */
    tFont.Space = 0;                 /* 字符水平间距，单位 = 像素 */

    y = 4;
    usLineCap = 18; /* 行间距 */
    LCD_DispStr(5, y, "安富莱 STM32-V7 开发板  www.armfly.com", &tFont);
    y += usLineCap;
    LCD_DispStr(5, y, "电阻屏和电容屏实验，电容屏无需校准，电阻需要校准", &tFont);

    y += 5 * usLineCap;
    LCD_DispStr(30, y, "操作提示:", &tFont);

    y += usLineCap;
    LCD_DispStr(50, y, "摇杆上键 = 增加背光亮度", &tFont);

    y += usLineCap;
    LCD_DispStr(50, y, "摇杆下键 = 降低背光亮度", &tFont);
```

```
    y += usLineCap;
    LCD_DispStr(50, y, "摇杆左键 = 电阻屏触摸校准", &tFont);

    y += usLineCap;
    LCD_DispStr(50, y, "摇杆 OK 键 = 清屏", &tFont);

    y += 2 * usLineCap;


    /* 显示 TFT 控制器型号和屏幕分辨率 */
    //LCD_GetChipDescribe(buf);  /* 读取 TFT 驱动芯片型号 */
    if (g_TouchType == CT_GT811)
    {
        strcpy(buf, "STM32H743 + GT811");
    }
    else if (g_TouchType == CT_GT911)
    {
        strcpy(buf, "STM32H743 + GT911");
    }
    else if (g_TouchType == CT_FT5X06)
    {
        strcpy(buf, "STM32H743 + FT5X06");
    }
    else if (g_TouchType == CT_STMPE811)
    {
        strcpy(buf, "STM32H743 + STMPE811");
    }
    else
    {
        strcpy(buf, "STM32H743 + NoTouch");
    }
    sprintf(&buf[strlen(buf)], "   %d x %d", LCD_GetWidth(), LCD_GetHeight());
    LCD_DispStr(5, y, (char *)buf, &tFont);
}
```

54.11 实验例程说明（IAR）

配套例子：

V7-034_LCD 的电阻触摸和电容触摸（电阻触摸支持 2 点和 4 点校准）

实验目的：

1. 学习 LCD 的电阻触摸和电容触摸。

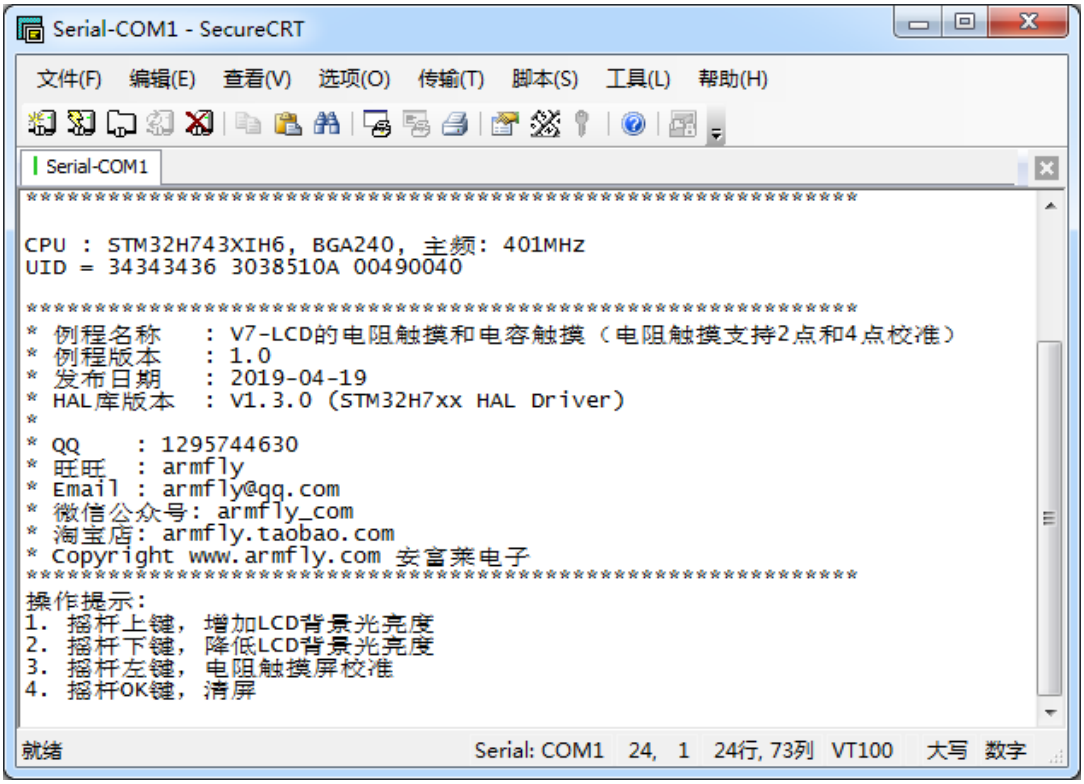
实验内容：

1. 电容屏无需校准，电阻屏需要校准。
2. LCD 界面实现了一个简单的画板功能，方便检测触摸是否准确。
3. 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。

LCD 界面显示效果如下：

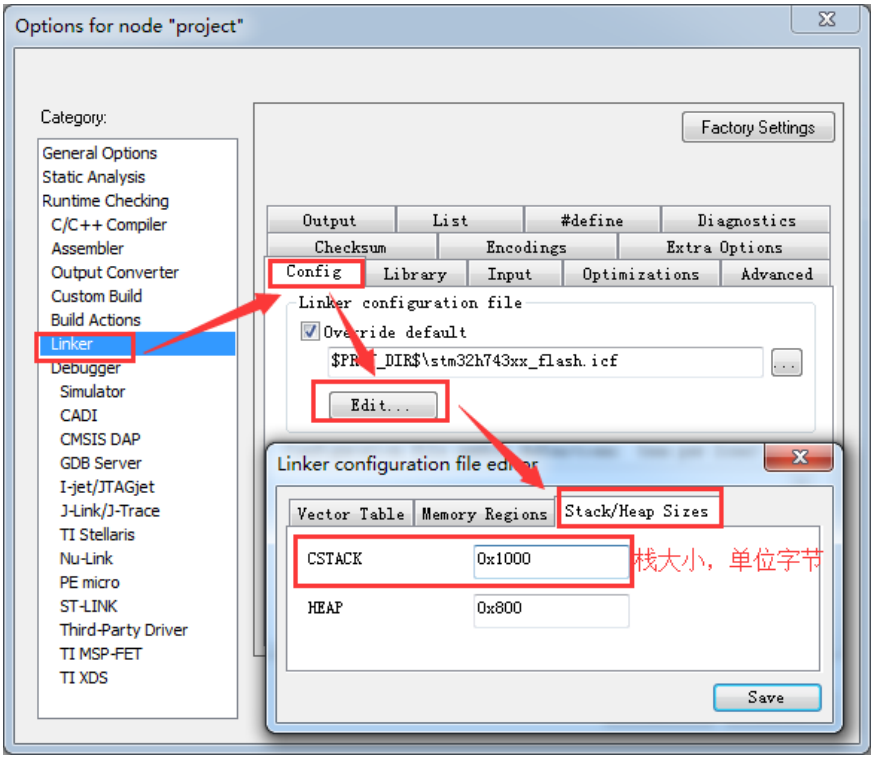
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

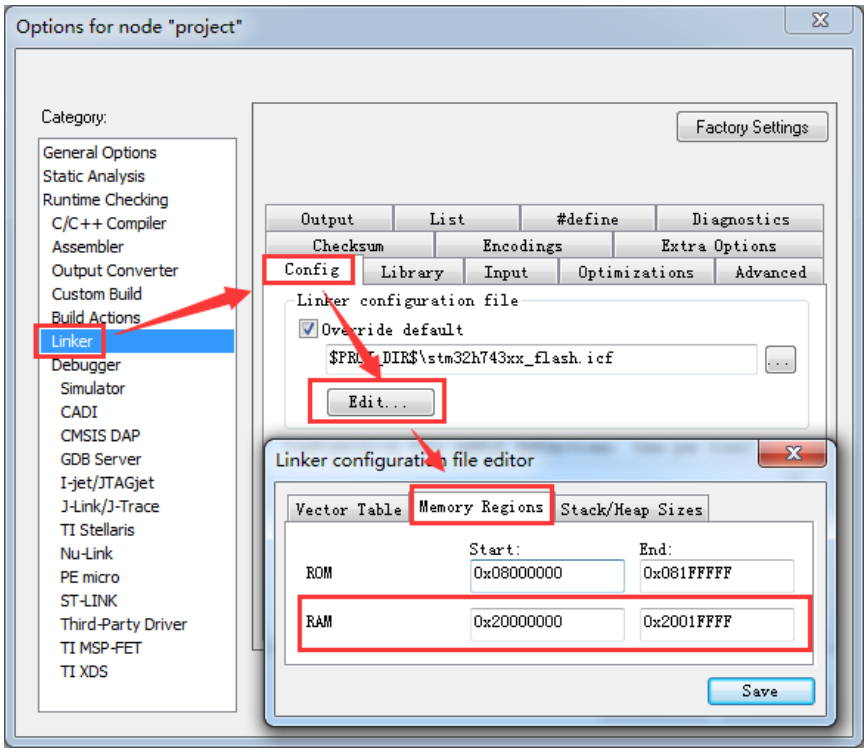


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
```

```
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */

    bsp_InitI2C();    /* 初始化 I2C 总线 */
    TOUCH_InitHard(); /* 初始化触摸芯片，LCD 面板型号的检查也在此函数，所以要在函数 LCD_InitHard 前调用 */
    LCD_InitHard();   /* 初始化 LCD */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SDRAM。由于 SDRAM 要用于 LCD 的显存，方便起见，直接将其配置为 WT 模式。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
```

```
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SDRAM 的 MPU 属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0xC0000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_32MB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
```

```
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能:**

主程序实现如下操作:

- LCD 界面实现了一个简单的画板功能，方便检测触摸是否准确。
- 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。
- 摇杆上键，增加 LCD 背景光亮度。
- 摇杆下键，降低 LCD 背景光亮度。
- 摇杆左键，电阻触摸屏校准。
- 摇杆 OK 键，清屏。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint16_t ucBright;      /* 背光亮度(0-255) */
    uint8_t ucKeyCode;      /* 按键代码 */
    uint8_t fRefresh;       /* 刷屏请求标志,1 表示需要刷新 */
    FONT_T tFont;           /* 定义一个字体结构体变量，用于设置字体参数 */
    char buf[64];
    uint16_t usAdcX, usAdcY;
    int16_t tpX, tpY;
    uint8_t ucTouch;        /* 触摸事件 */


    /* 设置字体参数 */
    {
        tFont.FontCode = FC_ST_16;      /* 字体代码 16 点阵 */
        tFont.FrontColor = CL_WHITE;    /* 字体颜色 */
        tFont.BackColor = CL_BLUE;      /* 文字背景颜色 */
```

```
    tFont.Space = 0;                    /* 文字间距，单位 = 像素 */
}

bsp_Init();         /* 硬件初始化 */
PrintfLogo(); /* 打印例程名称和版本等信息 */
PrintfHelp(); /* 打印操作提示 */

/* 延迟 200ms 再点亮背光，避免瞬间高亮 */
bsp_DelayMS(200);

DispFirstPage();   /* 显示第 1 页 */

/* 界面整体显示完毕后，再打开背光，设置为缺省亮度 */
bsp_DelayMS(100);
ucBright = BRIGHT_DEFAULT;
LCD_SetBackLight(ucBright);

bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */

/* 进入主程序循环体 */
fRefresh = 1;
while (1)
{
  bsp_Idle();

    /* 判断软件定时器 0 是否超时 */
    if(bsp_CheckTimer(0))
    {
        /* 每隔 200ms 进来一次 */
        bsp_LedToggle(2);
    }

   if (fRefresh)
    {
        fRefresh = 0;

        /* 实时刷新触摸 ADC 采样值和转换后的坐标 */
        {

            /* 读取并显示当前 X 轴和 Y 轴的 ADC 采样值 */
            usAdcX = TOUCH_ReadAdcX();
            usAdcY = TOUCH_ReadAdcY();
            sprintf(buf, "触摸 ADC 值 X = %4d, Y = %4d   ", usAdcX, usAdcY);
            LCD_DispStr(5, 40, buf, &tFont);

            /* 读取并显示当前触摸坐标 */
            tpX = TOUCH_GetX();
            tpY = TOUCH_GetY();
            sprintf(buf, "触摸坐标  X = %4d, Y = %4d   ", tpX, tpY);
            LCD_DispStr(5, 60, buf, &tFont);

            /* 在触笔所在位置显示一个小圈 */
            if ((tpX > 0) && (tpY > 0))
            {
                LCD_DrawCircle(tpX, tpY, 2, CL_YELLOW);
            }
        }
```

```
        /* 在屏幕边沿绘制 2 个矩形框(用于检测面板边缘像素是否正常) */
        LCD_DrawRect(0, 0, LCD_GetHeight(), LCD_GetWidth(), CL_WHITE);
        LCD_DrawRect(2, 2, LCD_GetHeight() - 4, LCD_GetWidth() - 4, CL_YELLOW);

      /* 显示背光值 */
        sprintf(buf, "当前背光亮度: %3d", ucBright);
        LCD_DispStr(5, 80, buf, &tFont);
    }

    ucTouch = TOUCH_GetKey(&tpX, &tpY);    /* 读取触摸事件 */
    if (ucTouch != TOUCH_NONE)
    {
        switch (ucTouch)
        {
            case TOUCH_DOWN:        /* 触笔按下事件 */

                /* 在触笔所在位置显示一个小圈 */
                if ((tpX > 0) && (tpY > 0))
                {
                    LCD_DrawCircle(tpX, tpY, 3, CL_RED);
                }
                break;

            case TOUCH_MOVE:        /* 触笔移动事件 */
                /* 实时刷新触摸 ADC 采样值和转换后的坐标 */
                {
                    /* 读取并显示当前 X 轴和 Y 轴的 ADC 采样值 */
                    usAdcX = TOUCH_ReadAdcX();
                    usAdcY = TOUCH_ReadAdcY();
                    sprintf(buf, "触摸 ADC 值 X = %4d, Y = %4d   ", usAdcX, usAdcY);
                    LCD_DispStr(5, 40, buf, &tFont);

                    /* 读取并显示当前触摸坐标 */
                    //tpX = TOUCH_GetX();
                    //tpY = TOUCH_GetY();
                    sprintf(buf, "触摸坐标  X = %4d, Y = %4d   ", tpX, tpY);
                    LCD_DispStr(5, 60, buf, &tFont);

                    /* 在触笔所在位置显示一个小圈 */
                    if ((tpX > 0) && (tpY > 0))
                    {
                        LCD_DrawCircle(tpX, tpY, 2, CL_YELLOW);
                    }
                }
                break;

            case TOUCH_RELEASE:         /* 触笔释放事件 */
                /* 在触笔所在位置显示一个小圈 */
                if ((tpX > 0) && (tpY > 0))
                {
                    LCD_DrawCircle(tpX, tpY, 4, CL_WHITE);
                }
                break;
        }
    }

    ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
    if (ucKeyCode != KEY_NONE)
```

```
        {
            /* 有键按下 */
            switch (ucKeyCode)
            {
                case JOY_DOWN_L:        /* 摇杆 LEFT 键按下, 2 点触摸校准 */
                    TOUCH_Calibration(2);
                    DispFirstPage();
                    fRefresh = 1;       /* 请求刷新 LCD */
                    break;

                case  JOY_DOWN_OK:      /* 摇杆 OK 键, 清屏 */
                    DispFirstPage();
                    fRefresh = 1;       /* 请求刷新 LCD */
                    break;

                case JOY_DOWN_U:        /* 摇杆 UP 键按下 */
                    ucBright += BRIGHT_STEP;
                    if (ucBright > BRIGHT_MAX)
                    {
                        ucBright = BRIGHT_MAX;
                    }
                    LCD_SetBackLight(ucBright);
                  fRefresh = 1;         /* 请求刷新 LCD */
                    break;

                case JOY_DOWN_D:        /* 摇杆 DOWN 键按下 */
                    if (ucBright < BRIGHT_STEP)
                    {
                        ucBright = 0;
                    }
                    else
                    {
                        ucBright -= BRIGHT_STEP;
                    }
                    LCD_SetBackLight(ucBright);
                  fRefresh = 1;         /* 请求刷新 LCD */
                    break;

                default:
                    break;
            }
        }
    }
}
```

LCD 主界面显示内容如下:

```
/*
*********************************************************************************************************
*    函 数 名: DispFirstPage
*    功能说明: 显示操作提示
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void DispFirstPage(void)
{
    FONT_T tFont;       /* 定义一个字体结构体变量, 用于设置字体参数 */
    uint16_t y;             /* Y 坐标 */
```

```
    uint16_t usLineCap;     /* 行高 */
    char buf[50];

    LCD_ClrScr(CL_BLUE);        /* 清屏，背景蓝色 */


    /* 设置字体属性 */
    tFont.FontCode = FC_ST_16;       /* 字体选择宋体 16 点阵，高 16x 宽 15) */
    tFont.FrontColor = CL_WHITE;     /* 字体颜色设置为白色 */
    tFont.BackColor = CL_MASK;       /* 文字背景颜色，透明 */
    tFont.Space = 0;                 /* 字符水平间距，单位 = 像素 */

    y = 4;
    usLineCap = 18; /* 行间距 */
    LCD_DispStr(5, y, "安富莱 STM32-V7 开发板  www.armfly.com", &tFont);
    y += usLineCap;
    LCD_DispStr(5, y, "电阻屏和电容屏实验，电容屏无需校准，电阻需要校准", &tFont);

    y += 5 * usLineCap;
    LCD_DispStr(30, y, "操作提示:", &tFont);

    y += usLineCap;
    LCD_DispStr(50, y, "摇杆上键 = 增加背光亮度", &tFont);

    y += usLineCap;
    LCD_DispStr(50, y, "摇杆下键 = 降低背光亮度", &tFont);

    y += usLineCap;
    LCD_DispStr(50, y, "摇杆左键 = 电阻屏触摸校准", &tFont);

    y += usLineCap;
    LCD_DispStr(50, y, "摇杆 OK 键 = 清屏", &tFont);

    y += 2 * usLineCap;


    /* 显示 TFT 控制器型号和屏幕分辨率 */
    //LCD_GetChipDescribe(buf);  /* 读取 TFT 驱动芯片型号 */
    if (g_TouchType == CT_GT811)
    {
        strcpy(buf, "STM32H743 + GT811");
    }
    else if (g_TouchType == CT_GT911)
    {
        strcpy(buf, "STM32H743 + GT911");
    }
    else if (g_TouchType == CT_FT5X06)
    {
        strcpy(buf, "STM32H743 + FT5X06");
    }
    else if (g_TouchType == CT_STMPE811)
    {
        strcpy(buf, "STM32H743 + STMPE811");
    }
    else
    {
        strcpy(buf, "STM32H743 + NoTouch");
    }
```

```
    sprintf(&buf[strlen(buf)], "   %d x %d", LCD_GetWidth(), LCD_GetHeight());
    LCD_DispStr(5, y, (char *)buf, &tFont);
}
```

54.12 总结

本章节为大家讲解的电阻触摸方案和电容触摸方案都是经过实战总结的，实际项目中比较实用。

第55章 STM32H7 的图形加速器 DMA2D 的基础知识和 HAL 库 API

本章节为大家讲解 DMA2D，实际项目中显示屏的加速全靠它了，而且性能也比较给力。

55.1 初学者重要提示
55.2 DMA2D 基础知识
55.3 DMA2D 的 HAL 库用法
55.4 源文件 stm32h7xx_hal_dma2d.c
55.5 总结

55.1 初学者重要提示

- DMA2D 是专门用于 LCD 加速的，特别是刷单色屏，刷图片，刷 Alpah（透明）混合效果全靠它，而且可以大大降低 CPU 利用率。
- 测评 STM32H7 的 LTDC+DMA2D 性能，100Hz 以上无压力，刷 800*480 图片和色块仅需 2.6ms 一张：http://www.armbbs.cn/forum.php?mod=viewthread&tid=91489 。
- H7 的 DMA2D 与 F429 的 DMA2D 最大区别是支持了 ARGB 和 ABGR 互转，而且支持 H7 的硬解 JEPG 输出格式 YCbCr 转 RGB，方便 LCD 显示。
- 特别注意，大家只需对 HAL 库提供的 DMA2D 操作 API 有个了解即可，实际工程中，并不使用这些 API，我们需要使用更加高效的寄存器直接操作，在下一章节会为大家说明。

55.2 DMA2D 基础知识

DMA2D 主要实现了两个功能，一个是 DMA 数据传输功能，另一个是 2D 图形加速功能。

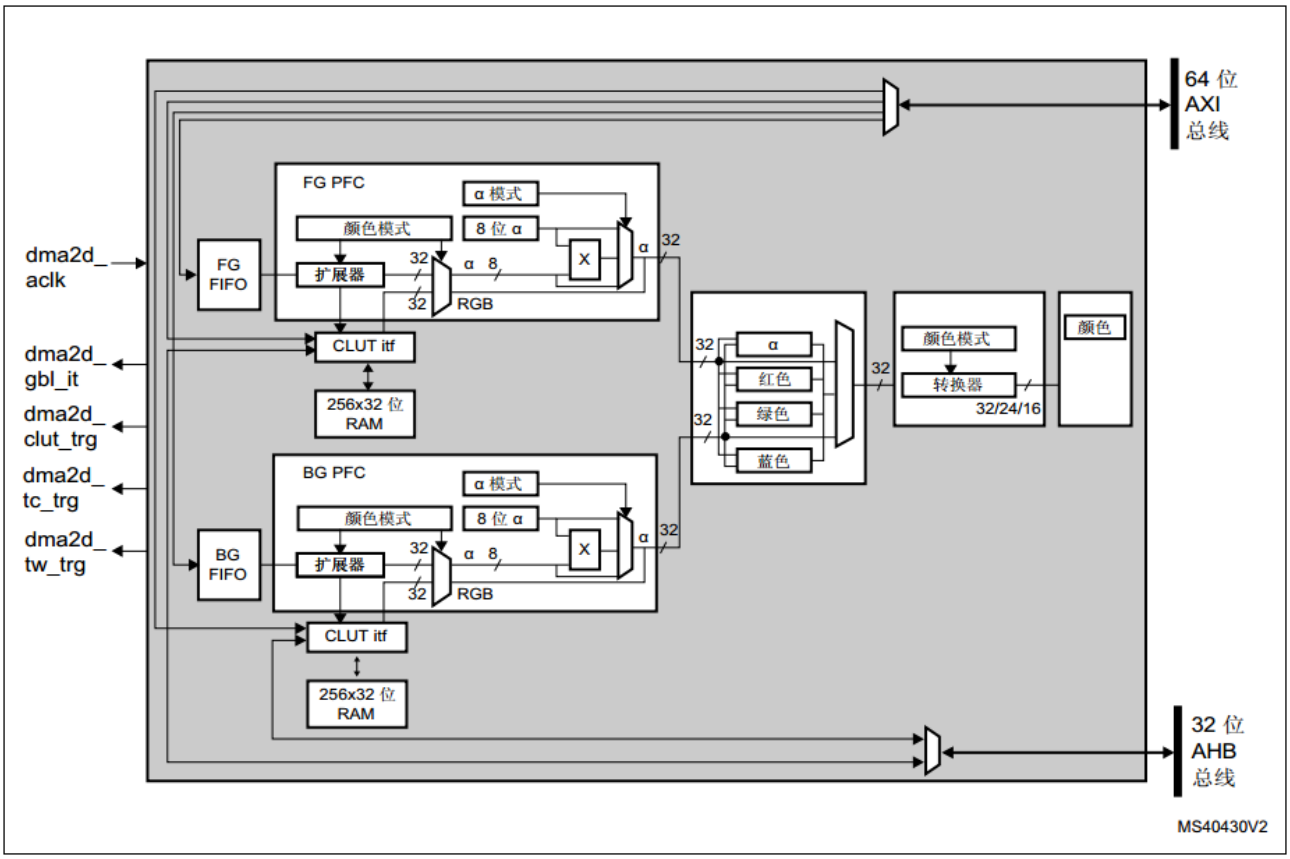
- DMA 数据传输

  主要是两种方式，一个是寄存器到存储器，另一个是存储器到存储器。通过 DMA 可以大大降低 CPU 的利用率。
- 2D 图形加速功能

  支持硬件的颜色格式转换和 Alpha 混合效果。

55.2.1DMA2D 硬件框图

认识一个外设，最好的方式就是看它的框图，方便我们快速地了解 DMA2D 的基本功能，然后再看手册了解细节。框图如下所示：



通过这个框图，我们可以得到如下信息：

- dma2d_aclk

  AXI 总线时钟输入。

- dma2d_gbl_it

  DMA2D 全局中断输出。

- dma2d_clut_trg

  CLUT 传输完成信号输出，可以触发 MDMA。

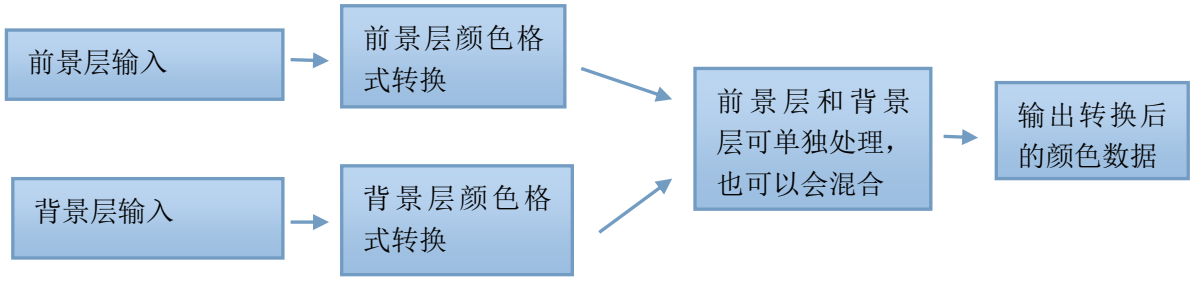
- dma2d_tc_trg

  传输完成信号输出，可以触发 MDMA。

- dma2d_tw_trg

  传输 watermark 信号输出，可以触发 MDMA。

将这个硬件框图简化一下，就是下面这样：



下面按照简化的硬件框图，对每个部分做个说明。

55.2.2 DMA2D 工作模式

DMA2D 支持的工作模式如下：

- 模式 1：寄存器到存储器模式

  这个模式主要用于清屏，也即是将显示屏清为单色效果。

- 模式 2：存储器到存储器模式

  这个模式用于从一个存储器复制一块数据到另一个存储器，比如将摄像头 OV7670 的输出图像复制到 LCD 显存就可以采用这种方式。

- 模式 3：存储器到存储器模式，带颜色格式转换

  这个模式比模式 2 多了一个颜色格式转换，比如我们要显示一幅 RGB888 颜色格式的位图到 RGB565 颜色格式的显示屏，就需要用到这个模式，只需输入端配置为 RGB888，输出端配置 RGB565 即可。位图颜色格式转换后会显示到显示屏上。

- 模式 4：存储器到存储器模式，带颜色格式转换和混合

  这个模式比模式 3 多了一个混合操作，通过混合，可以将两种效果进行混合显示。

- 模式 5：存储器到存储器模式，带颜色格式转换和混合，前景色是固定的

  同模式 4，只是前景色的颜色值是固定的。

55.2.3 前景层和背景层的输入以及颜色格式转换

前景层和背景层是指的用户绘制图形时的前景色和背景色，比如我们显示汉字，字体会有一个颜色，也就是前景色，还有一个背景色。又比如我们绘制两幅图片，想将两幅图片混合，那就可以将一幅图片作为前景层，另一个幅图片作为背景层。

DMA2D 支持的输入颜色格式如下，前景层和背景层一样：

CM[3:0]配置位	颜色格式
0000	ARGB8888

0001	RGB888
0010	RGB565
0011	ARGB1555
0100	ARGB4444
0101	L8
0110	AL44
0111	AL88
1000	L4
1001	A8
1010	A4
1011	YCbCr（仅用于前景层）

前 8 种颜色格式在第 50 章的第 2 小节开头有介绍，这里把后四种做个说明：

- L4 (4-bit luminance or CLUT)

  4 位颜色格式，实际上仅仅是 4 位索引值，范围 0–15,而每个索引值的具体颜色值在查色表 CLUT 里面存储。

- A4 和 A8

  A4 和 A8 用于特定的 Alpha 模式，既不存储颜色信息，也没有索引值。

- YCbCr

  这个是 H7 的硬件 JPEG 输出的颜色格式，后面 JPEG 章节为大家专门做讲解。

这里特别注意一点，输入颜色格式的 Alpha 值是可以设置的，而且颜色格式里面的 R 通道和 B 通道可以交换位置。

55.2.4前景层和背景层混合

DMA2D 混合器用于混合前景色和背景色，这个功能不需要任何配置，仅需要通过 DMA2D_CR 寄存器使能即可。混合公式如下：

$$\text{with } \alpha_{Mult} = \frac{\alpha_{FG} \cdot \alpha_{BG}}{255}$$

$$\alpha_{OUT} = \alpha_{FG} + \alpha_{BG} - \alpha_{Mult}$$

$$C_{OUT} = \frac{C_{FG} \cdot \alpha_{FG} + C_{BG} \cdot \alpha_{BG} - C_{BG} \cdot \alpha_{Mult}}{\alpha_{OUT}} \quad \text{with C = R or G or B}$$

55.2.5 DMA2D 输出颜色格式

DMA2D 支持的输出颜色格式如下：

CM[2:0]配置位	颜色格式
0000	ARGB8888
0001	RGB888
0010	RGB565
0011	ARGB1555
0100	ARGB4444

这里特别注意一点，输出颜色格式的 Alpha 值是可以设置的，而且颜色格式里面的 R 通道和 B 通道可以交换位置。

55.3 DMA2D 的 HAL 库用法

DMA2D 的 HAL 库用法其实就是几个结构体变量成员的配置和使用，然后配置时钟，并根据需要配置 NVIC、中断。下面我们逐一展开为大家做个说明。

55.3.1 DMA2D 寄存器结构体 DMA2D_TypeDef

DMA2D 相关的寄存器是通过 HAL 库中的结构体 DMA2D_TypeDef 定义的，在 stm32h743xx.h 中可以找到它们的具体定义：

```
typedef struct
{
  __IO uint32_t CR;            /*!< DMA2D Control Register,                      Address offset: 0x00 */
  __IO uint32_t ISR;           /*!< DMA2D Interrupt Status Register,             Address offset: 0x04 */
  __IO uint32_t IFCR;          /*!< DMA2D Interrupt Flag Clear Register,         Address offset: 0x08 */
  __IO uint32_t FGMAR;         /*!< DMA2D Foreground Memory Address Register,    Address offset: 0x0C */
  __IO uint32_t FGOR;          /*!< DMA2D Foreground Offset Register,            Address offset: 0x10 */
  __IO uint32_t BGMAR;         /*!< DMA2D Background Memory Address Register,    Address offset: 0x14 */
  __IO uint32_t BGOR;          /*!< DMA2D Background Offset Register,            Address offset: 0x18 */
  __IO uint32_t FGPFCCR;       /*!< DMA2D Foreground PFC Control Register,       Address offset: 0x1C */
  __IO uint32_t FGCOLR;        /*!< DMA2D Foreground Color Register,             Address offset: 0x20 */
  __IO uint32_t BGPFCCR;       /*!< DMA2D Background PFC Control Register,       Address offset: 0x24 */
  __IO uint32_t BGCOLR;        /*!< DMA2D Background Color Register,             Address offset: 0x28 */
  __IO uint32_t FGCMAR;        /*!< DMA2D Foreground CLUT Memory Address Register, Address offset: 0x2C */
  __IO uint32_t BGCMAR;        /*!< DMA2D Background CLUT Memory Address Register, Address offset: 0x30 */
  __IO uint32_t OPFCCR;        /*!< DMA2D Output PFC Control Register,           Address offset: 0x34 */
  __IO uint32_t OCOLR;         /*!< DMA2D Output Color Register,                 Address offset: 0x38 */
  __IO uint32_t OMAR;          /*!< DMA2D Output Memory Address Register,        Address offset: 0x3C */
  __IO uint32_t OOR;           /*!< DMA2D Output Offset Register,                Address offset: 0x40 */
  __IO uint32_t NLR;           /*!< DMA2D Number of Line Register,               Address offset: 0x44 */
  __IO uint32_t LWR;           /*!< DMA2D Line Watermark Register,               Address offset: 0x48 */
  __IO uint32_t AMTCR;         /*!< DMA2D AHB Master Timer Configuration Register, Address offset: 0x4C */
  uint32_t      RESERVED[236]; /*!< Reserved, 0x50-0x3FF */
```

```
  __IO uint32_t FGCLUT[256];    /*!< DMA2D Foreground CLUT,                    Address offset:400-7FF */
  __IO uint32_t BGCLUT[256];    /*!< DMA2D Background CLUT,                    Address offset:800-BFF */
} DMA2D_TypeDef;
```

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

下面我们再看 DMA2D 的定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE              ((uint32_t)0x40000000)
#define D1_AHB1PERIPH_BASE       (PERIPH_BASE + 0x12000000)
#define DMA2D_BASE               (D1_AHB1PERIPH_BASE + 0x1000)
#define DMA2D                    ((DMA2D_TypeDef *) DMA2D_BASE) <----- 展开这个宏，(DMA2D_TypeDef *) 0x52001000
```

我们访问 DMA2D 的 ISR 寄存器可以采用这种形式：DMA2D->ISR = 0。

55.3.2 DMA2D 参数初始化结构体 DMA2D_InitTypeDef

此结构体用于配置 DMA2D 的基本参数，具体定义如下：

```
typedef struct
{
  uint32_t             Mode;
  uint32_t             ColorMode;
  uint32_t             OutputOffset;
  uint32_t             AlphaInverted;
  uint32_t             RedBlueSwap;
} DMA2D_InitTypeDef;
```

下面将这几个参数逐一为大家做个说明：

- uint32_t Mode

此参数用于设置 DMA2D 的传输模式，具体支持的参数如下：

```
#define DMA2D_M2M         ((uint32_t)0x00000000U)  /*存储器到存储传输模式 */
#define DMA2D_M2M_PFC      DMA2D_CR_MODE_0         /*存储器到存储器传输模式，并执行 FPC 像素格式转 */
#define DMA2D_M2M_BLEND    DMA2D_CR_MODE_1         /* 存储器到存储器模式，并执行像素格式转换和混合 */
#define DMA2D_R2M          DMA2D_CR_MODE           /* 寄存器到存储器传输模式 */
```

- uint32_t ColorMode

此参数用于设置 DMA2D 的输出颜色格式，具体支持的参数如下：

```
#define DMA2D_OUTPUT_ARGB8888       ((uint32_t)0x00000000U)                /* ARGB8888 */
#define DMA2D_OUTPUT_RGB888         DMA2D_OPFCCR_CM_0                      /* RGB888 */
#define DMA2D_OUTPUT_RGB565         DMA2D_OPFCCR_CM_1                      /* RGB565 */
#define DMA2D_OUTPUT_ARGB1555       (DMA2D_OPFCCR_CM_0|DMA2D_OPFCCR_CM_1)  /* ARGB1555 */
#define DMA2D_OUTPUT_ARGB4444       DMA2D_OPFCCR_CM_2                      /* ARGB4444 */
```

- uint32_t OutputOffset

此参数用于设置输出位置的偏移值，参数范围 0x0000 到 0x3FFF。

- uint32_t AlphaInverted

此参数用于设置 DMA2D 的输出颜色格式 Alpha 值反转 (即 255-原来数值)，具体支持的参数如下：

```
#define DMA2D_REGULAR_ALPHA      ((uint32_t)0x00000000U)  /* 正常输出 */
#define DMA2D_INVERTED_ALPHA     ((uint32_t)0x00000001U)  /* 反转输出 */
```

- uint32_t RedBlueSwap

此参数用于设置 DMA2D 的输出颜色格式中 R 通道和 B 通道的交换，具体支持的参数如下：

```
#define DMA2D_RB_REGULAR    ((uint32_t)0x00000000U)  /* 不交换(RGB or ARGB) */
#define DMA2D_RB_SWAP       ((uint32_t)0x00000001U)  /* 交换(BGR or ABGR) */
```

55.3.3 DMA2D 的图层结构体 DMA2D_LayerCfgTypeDef

此结构体用于配置前景色和背景色。

```
typedef struct
{
  uint32_t              InputOffset;
  uint32_t              InputColorMode;
  uint32_t              AlphaMode;
  uint32_t              InputAlpha;
  uint32_t              AlphaInverted;
  uint32_t              RedBlueSwap;
  uint32_t              ChromaSubSampling;
} DMA2D_LayerCfgTypeDef;
```

下面将这几个参数逐一为大家做个说明。

- uint32_t InputOffset

  设置前景色或者背景色的输入偏移，范围 0x000 到 0x3FFF。

- uint32_t InputColorMode

  设置前景色或者背景色的输入颜色格式，具体支持的参数如下：

```
#define DMA2D_INPUT_ARGB8888         ((uint32_t)0x00000000U)  /* ARGB8888 */
#define DMA2D_INPUT_RGB888           ((uint32_t)0x00000001U)  /* RGB888   */
#define DMA2D_INPUT_RGB565           ((uint32_t)0x00000002U)  /* RGB565   */
#define DMA2D_INPUT_ARGB1555         ((uint32_t)0x00000003U)  /* ARGB1555 */
#define DMA2D_INPUT_ARGB4444         ((uint32_t)0x00000004U)  /* ARGB4444 */
#define DMA2D_INPUT_L8               ((uint32_t)0x00000005U)  /* L8       */
#define DMA2D_INPUT_AL44             ((uint32_t)0x00000006U)  /* AL44     */
#define DMA2D_INPUT_AL88             ((uint32_t)0x00000007U)  /* AL88     */
#define DMA2D_INPUT_L4               ((uint32_t)0x00000008U)  /* L4       */
#define DMA2D_INPUT_A8               ((uint32_t)0x00000009U)  /* A8       */
#define DMA2D_INPUT_A4               ((uint32_t)0x0000000AU)  /* A4       */
#define DMA2D_INPUT_YCBCR            ((uint32_t)0x0000000BU)  /* YCbCr    */
```

- uint32_t AlphaMode

  设置前景色或者背景色的 Alpha 模式，具体支持的参数如下：

```
#define DMA2D_NO_MODIF_ALPHA    ((uint32_t)0x00000000U)  /* 不修改 Alpha 通道值 */
#define DMA2D_REPLACE_ALPHA     ((uint32_t)0x00000001U)  /* 用新设置的 Alpha 值替换原始 Alpha 值 */
#define DMA2D_COMBINE_ALPHA     ((uint32_t)0x00000002U)  /* 用新设置的 Alpha 值与原始 Alpha 值的乘积替换原始
                                                            Alaha 值*/
```

- uint32_t InputAlpha

  设置前景色或者背景色的 Alpha 值，范围 0x00 到 0xFF，如果颜色格式是 A4 或者 A8，那么此参数的范围是 0x00000000 到 0xFFFFFFFF，标准的 ARGB8888 格式。

- uint32_t AlphaInverted

  设置前景色或者背景色的输入颜色格式 Alpha 值反转（即 255-原来数值），具体支持的参数如下：

```
#define DMA2D_REGULAR_ALPHA      ((uint32_t)0x00000000U)  /* 正常输出 */
```

```
#define DMA2D_INVERTED_ALPHA      ((uint32_t)0x00000001U)  /* 反转输出 */
```

- uint32_t RedBlueSwap

设置前景色或者背景色颜色格式中 R 通道和 B 通道的交换，具体支持的参数如下：

```
#define DMA2D_RB_REGULAR    ((uint32_t)0x00000000U)  /* 不交换(RGB or ARGB) */
#define DMA2D_RB_SWAP       ((uint32_t)0x00000001U)  /* 交换(BGR or ABGR) */
```

- uint32_t ChromaSubSampling

设置前景色或者背景色中 YCbCr 颜色模式的采样格式，具体支持的参数如下：

```
#define DMA2D_NO_CSS                ((uint32_t)0x00000000)  /* 4:4:4 */
#define DMA2D_CSS_422               ((uint32_t)0x00000001)  /* 4:2:2 */
#define DMA2D_CSS_420               ((uint32_t)0x00000002)  /* 4:2:0 */
```

55.3.4DMA2D 句柄结构体 DMA2D_HandleTypeDef

HAL 库在 DMA2D_TypeDef， DMA2D_InitTypeDef 和 DMA2D_LayerCfgTypeDef 的基础上封装了一个结构体 DMA2D_HandleTypeDef，定义如下：

```
typedef struct __DMA2D_HandleTypeDef
{
  DMA2D_TypeDef               *Instance;
  DMA2D_InitTypeDef           Init;
  void                        (* XferCpltCallback)(struct __DMA2D_HandleTypeDef * hdma2d);
  void                        (* XferErrorCallback)(struct __DMA2D_HandleTypeDef * hdma2d);
  DMA2D_LayerCfgTypeDef       LayerCfg[MAX_DMA2D_LAYER];
  HAL_LockTypeDef             Lock;
  __IO HAL_DMA2D_StateTypeDef State;
  __IO uint32_t               ErrorCode;                                          }
DMA2D_HandleTypeDef;
```

下面将这几个参数逐一做个说明。

- **DMA2D_TypeDef *Instance**

这个参数是寄存器的例化，方便操作寄存器，详见本章 3.1 小节。

- **DMA2D_InitTypeDef Init;**

这个参数是用户接触较多的，用于配置 DMA2D 的基本参数，详见本章 3.2 小节。

- **void (* XferCpltCallback)(struct __DMA2D_HandleTypeDef * hdma2d);**
- **void (* XferErrorCallback)(struct __DMA2D_HandleTypeDef * hdma2d);**

DMA2D 中断服务程序里面执行的回调函数，一个是传输完成回调，另一个是传输错误回调。

- **DMA2D_LayerCfgTypeDef LayerCfg[MAX_DMA2D_LAYER]**

这个参数用于前景色和背景色的设置，MAX_DMA2D_LAYER=2，详见本章 3.3 小节。

- **HAL_LockTypeDef Lock**

__IO uint32_t State;

__IO uint32_t ErrorCode

这三个变量主要供函数内部使用。Lock 用于设置锁状态，State 用于设置 DMA2D 通信状态，而 ErrorCode 用于配置代码错误。

55.3.5 DMA2D 初始化流程总结

对于 DMA2D 来说，其实不需要初始化流程，每个功能都可以直接封装出一个函数来，下个章节会为大家专门讲解，也是实际项目比较推荐的方式。

55.4 源文件 stm32h7xx_hal_dma2d.c

这里把我们把如下几个常用到的函数做个说明：

- HAL_DMA2D_Init
- HAL_DMA2D_ConfigLayer
- HAL_DMA2D_Start_IT
- HAL_DMA2D_BlendingStart_IT

55.4.1 函数 HAL_DMA2D_Init

函数原型：

```
HAL_StatusTypeDef HAL_DMA2D_Init(DMA2D_HandleTypeDef *hdma2d)
{

  /*  检测参数是否有效 */
  if(hdma2d == NULL)
  {
     return HAL_ERROR;
  }

  /* 检测函数形参 */
  assert_param(IS_DMA2D_ALL_INSTANCE(hdma2d->Instance));
  assert_param(IS_DMA2D_MODE(hdma2d->Init.Mode));
  assert_param(IS_DMA2D_CMODE(hdma2d->Init.ColorMode));
  assert_param(IS_DMA2D_OFFSET(hdma2d->Init.OutputOffset));

  if(hdma2d->State == HAL_DMA2D_STATE_RESET)
  {
    hdma2d->Lock = HAL_UNLOCKED;
    /* 初始化 GPIO, NVIC 等 */
    HAL_DMA2D_MspInit(hdma2d);
  }

  /* 设置 DAM2D 外设状态 */
  hdma2d->State = HAL_DMA2D_STATE_BUSY;

  /* 设置 DAM2D 工作模式 ---------------------------------------------*/
  MODIFY_REG(hdma2d->Instance->CR, DMA2D_CR_MODE, hdma2d->Init.Mode);

  /* 设置输出颜色格式 ------------------------------------*/
  MODIFY_REG(hdma2d->Instance->OPFCCR, DMA2D_OPFCCR_CM, hdma2d->Init.ColorMode);

  /* 设置输出偏移 ----------------------------------------*/
```

```
  MODIFY_REG(hdma2d->Instance->OOR, DMA2D_OOR_LO, hdma2d->Init.OutputOffset);

  /* 设置输出颜色格式中的 Alpah 值反转 */
  MODIFY_REG(hdma2d->Instance->OPFCCR, DMA2D_OPFCCR_AI, (hdma2d->Init.AlphaInverted <<
                                DMA2D_POSITION_OPFCCR_AI));

  MODIFY_REG(hdma2d->Instance->OPFCCR, DMA2D_OPFCCR_RBS,(hdma2d->Init.RedBlueSwap <<
                                DMA2D_POSITION_OPFCCR_RBS));


  /* 无错误 */
  hdma2d->ErrorCode = HAL_DMA2D_ERROR_NONE;

  /* DAM2D 就绪 */
  hdma2d->State  = HAL_DMA2D_STATE_READY;

  return HAL_OK;
}
```

函数描述：

此函数用于初始化 DMA2D 的工作模式和输出颜色格式。

函数参数：

- 第 1 个参数是 DMA2D_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数，结构体变量成员的详细介绍看本章 3.4 小节。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 函数 HAL_DMA2D_MspInit 用于初始化 DMA2D 的底层时钟、NVIC 等功能。需要用户自己在此函数里面实现具体的功能。由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
2. 如果形参 hdma2d 的结构体成员 State 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 DMA2D_HandleTypeDef Dma2dHandle。

   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_DMA2D_STATE_RESET = 0x00U。

   解决办法有三

   方法 1：用户自己初始 DMA2D 底层。

   方法 2：定义 DMA2D_HandleTypeDef LtdcHandle 为全局变量。

   方法 3：下面的方法

```
if(HAL_DMA2D_DeInit(&Dma2dHandle) != HAL_OK)
{
    Error_Handler();
}
if(HAL_DMA2D_Init(&Dma2dHandle) != HAL_OK)
{
```

```
    Error_Handler();
}
```

使用举例：

```
DMA2D_HandleTypeDef Dma2dHandle;

/*##-1- 配置 DMA 工作模式，输出颜色格式和输出偏移 ##############*/
Dma2dHandle.Instance = DMA2D;

Dma2dHandle.Init.Mode          = DMA2D_M2M;             /* 存储器到存储器模式 */
Dma2dHandle.Init.ColorMode     = DMA2D_OUTPUT_ARGB4444; /* 输出颜色格式 */
Dma2dHandle.Init.OutputOffset  = 0x0;                   /* 无输出偏移 */
Dma2dHandle.Init.RedBlueSwap   = DMA2D_RB_REGULAR;      /* 输出颜色的 R/B 通过不切换 */
Dma2dHandle.Init.AlphaInverted = DMA2D_REGULAR_ALPHA;   /* 输出颜色的 Alpha 通道数值不翻转 */

/*##-2- DMA2D 回调函数配置 ######################################*/
Dma2dHandle.XferCpltCallback  = TransferComplete;
Dma2dHandle.XferErrorCallback = TransferError;

/*##-3- 前景层配置 ##############################################*/
Dma2dHandle.LayerCfg[1].AlphaMode = DMA2D_NO_MODIF_ALPHA;      /* 保持输入颜色格式中的 Alpha 值 */
Dma2dHandle.LayerCfg[1].InputAlpha = 0xFF;                     /* 完全不透明 */
Dma2dHandle.LayerCfg[1].InputColorMode = DMA2D_INPUT_ARGB4444; /* 输入颜色格式 */
Dma2dHandle.LayerCfg[1].InputOffset = 0x0;                     /* 输入无偏移 */
Dma2dHandle.LayerCfg[1].RedBlueSwap   = DMA2D_RB_REGULAR;      /* 输入颜色的 R/B 通过不切换*/
Dma2dHandle.LayerCfg[1].AlphaInverted = DMA2D_REGULAR_ALPHA;   /* 输入颜色的 Alpha 通道数值不翻转*/


/*##-4- DMA2D 初始化 ############################################*/
if(HAL_DMA2D_Init(&Dma2dHandle) != HAL_OK)
{
    Error_Handler();
}
```

55.4.2 函数 HAL_DMA2D_ConfigLayer

函数原型：

```
HAL_StatusTypeDef HAL_DMA2D_ConfigLayer(DMA2D_HandleTypeDef *hdma2d, uint32_t LayerIdx)
{
  DMA2D_LayerCfgTypeDef *pLayerCfg = &hdma2d->LayerCfg[LayerIdx];

  uint32_t regMask = 0, regValue = 0;

  /* 检查参数 */
  assert_param(IS_DMA2D_LAYER(LayerIdx));
  assert_param(IS_DMA2D_OFFSET(pLayerCfg->InputOffset));
  if(hdma2d->Init.Mode != DMA2D_R2M)
  {
    assert_param(IS_DMA2D_INPUT_COLOR_MODE(pLayerCfg->InputColorMode));
    if(hdma2d->Init.Mode != DMA2D_M2M)
    {
      assert_param(IS_DMA2D_ALPHA_MODE(pLayerCfg->AlphaMode));
    }
  }
```

```
/* 上锁 */
__HAL_LOCK(hdma2d);

/* 设置 DMA2D 外设状态 */
hdma2d->State = HAL_DMA2D_STATE_BUSY;

/* 准备好背景层或者前景层 FPC 寄存器配置参数*/
regValue = pLayerCfg->InputColorMode | (pLayerCfg->AlphaMode << DMA2D_POSITION_BGPFCCR_AM) | \
           (pLayerCfg->AlphaInverted << DMA2D_POSITION_BGPFCCR_AI) | \
           (pLayerCfg->RedBlueSwap << DMA2D_POSITION_BGPFCCR_RBS);

regMask  = DMA2D_BGPFCCR_CM | DMA2D_BGPFCCR_AM | DMA2D_BGPFCCR_ALPHA | DMA2D_BGPFCCR_AI | DMA2D_BGPFCCR_RBS;

if ((pLayerCfg->InputColorMode == DMA2D_INPUT_A4) || (pLayerCfg->InputColorMode == DMA2D_INPUT_A8))
{
  regValue |= (pLayerCfg->InputAlpha & DMA2D_BGPFCCR_ALPHA);
}
else
{
  regValue |=  (pLayerCfg->InputAlpha << DMA2D_POSITION_BGPFCCR_ALPHA);
}

/* 配置背景层 */
if(LayerIdx == 0)
{
  /* DMA2D BGPFCCR 寄存器 */
  MODIFY_REG(hdma2d->Instance->BGPFCCR, regMask, regValue);

  /* DMA2D BGOR 寄存器 */
  WRITE_REG(hdma2d->Instance->BGOR, pLayerCfg->InputOffset);

  /* DMA2D BGCOLR 寄存器 */
  if ((pLayerCfg->InputColorMode == DMA2D_INPUT_A4) || (pLayerCfg->InputColorMode == DMA2D_INPUT_A8))
  {
    WRITE_REG(hdma2d->Instance->BGCOLR, pLayerCfg->InputAlpha &
                                        (DMA2D_BGCOLR_BLUE|DMA2D_BGCOLR_GREEN|DMA2D_BGCOLR_RED));
  }
}
/* 配置前景层 */
else
{
  if(pLayerCfg->InputColorMode == DMA2D_INPUT_YCBCR)
  {
    regValue |= (pLayerCfg->ChromaSubSampling << DMA2D_POSITION_FGPFCCR_CSS);
    regMask  |= DMA2D_FGPFCCR_CSS;
  }

   /* DMA2D FGPFCCR 寄存器 */
  MODIFY_REG(hdma2d->Instance->FGPFCCR, regMask, regValue);

  /* DMA2D FGOR 寄存器 */
  WRITE_REG(hdma2d->Instance->FGOR, pLayerCfg->InputOffset);

  /* DMA2D FGCOLR 寄存器 */
  if ((pLayerCfg->InputColorMode == DMA2D_INPUT_A4) || (pLayerCfg->InputColorMode == DMA2D_INPUT_A8))
  {
    WRITE_REG(hdma2d->Instance->FGCOLR, pLayerCfg->InputAlpha &
                                        (DMA2D_FGCOLR_BLUE|DMA2D_FGCOLR_GREEN|DMA2D_FGCOLR_RED));
```

```
    }
  }
  /* DMA2D 就绪 */
  hdma2d->State = HAL_DMA2D_STATE_READY;

  /* 解锁 */
  __HAL_UNLOCK(hdma2d);

  return HAL_OK;
}
```

函数描述：

此函数主要用于配置 DMA2D 要转换的前景层和背景层，即输入颜色配置。而前面的函数 HAL_DMA2D_Init 配置的输出颜色。

函数参数：

- 第 1 个参数是 DMA2D_HandleTypeDef 类型结构体指针变量，用于配置 DMA2D 的基本参数，结构体变量成员的详细介绍看本章 3.4 小节。
- 第 2 个参数用于配置前景层和背景层，0 表示背景层，1 表示前景层。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
DMA2D_HandleTypeDef Dma2dHandle;


/*##-1- 配置 DMA 工作模式，输出颜色格式和输出偏移 ##############*/
Dma2dHandle.Instance = DMA2D;

Dma2dHandle.Init.Mode          = DMA2D_M2M;             /* 存储器到存储器模式 */
Dma2dHandle.Init.ColorMode     = DMA2D_OUTPUT_ARGB4444; /* 输出颜色格式 */
Dma2dHandle.Init.OutputOffset  = 0x0;                   /* 无输出偏移 */
Dma2dHandle.Init.RedBlueSwap   = DMA2D_RB_REGULAR;      /* 输出颜色的 R/B 通过不切换 */
Dma2dHandle.Init.AlphaInverted = DMA2D_REGULAR_ALPHA;   /* 输出颜色的 Alpha 通道数值不翻转 */

/*##-2- DMA2D 回调函数配置 ########################################*/
Dma2dHandle.XferCpltCallback  = TransferComplete;
Dma2dHandle.XferErrorCallback = TransferError;

/*##-3- 前景层配置 ################################################*/
Dma2dHandle.LayerCfg[1].AlphaMode = DMA2D_NO_MODIF_ALPHA;      /* 保持输入颜色格式中的 Alpha 值 */
Dma2dHandle.LayerCfg[1].InputAlpha = 0xFF;                     /* 完全不透明 */
Dma2dHandle.LayerCfg[1].InputColorMode = DMA2D_INPUT_ARGB4444; /* 输入颜色格式 */
Dma2dHandle.LayerCfg[1].InputOffset = 0x0;                     /* 输入无偏移 */
Dma2dHandle.LayerCfg[1].RedBlueSwap   = DMA2D_RB_REGULAR;      /* 输入颜色的 R/B 通过不切换*/
Dma2dHandle.LayerCfg[1].AlphaInverted = DMA2D_REGULAR_ALPHA;   /* 输入颜色的 Alpha 通道数值不翻转*/

/*##-4- DMA2D 初始化 ##############################################*/
if(HAL_DMA2D_Init(&Dma2dHandle) != HAL_OK)
{
    Error_Handler();
}

/* 配置前景层  */
```

```
if(HAL_DMA2D_ConfigLayer(&Dma2dHandle, 1) != HAL_OK)
{
   Error_Handler();
}
```

55.4.3函数 HAL_DMA2D_Start_IT

函数原型：

```
HAL_StatusTypeDef HAL_DMA2D_Start_IT(DMA2D_HandleTypeDef *hdma2d, uint32_t pdata, uint32_t DstAddress,
                                     uint32_t Width,  uint32_t Height)
{
  /* 检测函数形参 */
  assert_param(IS_DMA2D_LINE(Height));
  assert_param(IS_DMA2D_PIXEL(Width));

  /* 上锁 */
  __HAL_LOCK(hdma2d);

  /* 设置 DMA2D 外设状态 */
  hdma2d->State = HAL_DMA2D_STATE_BUSY;

  /* 设置源地址，目的地址和数据大小 */
  DMA2D_SetConfig(hdma2d, pdata, DstAddress, Width, Height);

  /* 使能 DMA2D 的传输完成中断，传输错误中断和配置错误中断 */
  __HAL_DMA2D_ENABLE_IT(hdma2d, DMA2D_IT_TC|DMA2D_IT_TE|DMA2D_IT_CE);

  /* 使能 DMA2D */
  __HAL_DMA2D_ENABLE(hdma2d);

  return HAL_OK;
}
```

函数描述：

此函数用于启动 DMA2D 数据传输。由于采用的中断方式，此函数使能了多个 DMA2D 中断，不要忘记 DMA2D 中断服务程序的处理。

函数参数：

- 第 1 个参数是 DMA2D_HandleTypeDef 类型结构体指针变量，用于配置 DMA2D 的基本参数，结构体变量成员的详细介绍看本章 3.4 小节。
- 第 2 个参数是源数据地址。
- 第 3 个参数是目的数据地址。
- 第 4 个参数是源数据的长度，即每行的像素个数。
- 第 5 个参数是源数据的高度，即行数。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
DMA2D_HandleTypeDef Dma2dHandle;
```

```
if(HAL_DMA2D_Start_IT(&Dma2dHandle,            /* DMA2D 句柄 */
                      (uint32_t)&BufferInput,  /* 源地址 */
                      (uint32_t)&BufferResult, /* 目的地址 */
                      SIZE_X,                  /* 源数据长度，单位像素个数*/
                      SIZE_Y)                  /* 源数据行数 */
   != HAL_OK)
{
   Error_Handler();
}
```

55.4.4函数 HAL_DMA2D_BlendingStart_IT

函数原型：

```
HAL_StatusTypeDef HAL_DMA2D_BlendingStart_IT(DMA2D_HandleTypeDef *hdma2d, uint32_t SrcAddress1, uint32_t
                                SrcAddress2, uint32_t DstAddress, uint32_t Width,  uint32_t Height)
{
  /* 检测参数 */
  assert_param(IS_DMA2D_LINE(Height));
  assert_param(IS_DMA2D_PIXEL(Width));

  /* 上锁 */
  __HAL_LOCK(hdma2d);

  /* 设置 DMA2D 外设状态 */
  hdma2d->State = HAL_DMA2D_STATE_BUSY;

  /* 配置 DMA2D 源地址 2 */
  WRITE_REG(hdma2d->Instance->BGMAR, SrcAddress2);

  /* 配置源地址 1，目的地址和数据大小 */
  DMA2D_SetConfig(hdma2d, SrcAddress1, DstAddress, Width, Height);

  /* 使能 DMA2D 传输完成中断，传输错误中断和配置错误中断 */
  __HAL_DMA2D_ENABLE_IT(hdma2d, DMA2D_IT_TC|DMA2D_IT_TE|DMA2D_IT_CE);

  /* 使能 DMA2D */
  __HAL_DMA2D_ENABLE(hdma2d);

  return HAL_OK;
}
```

函数描述：

此函数用于启动 DMA2D 传输，除了数据传输以外，还支持颜色格式转换和颜色混合。由于采用的中断方式，此函数使能了多个 DMA2D 中断，不要忘记 DMA2D 中断服务程序的处理。

函数参数：

- 第 1 个参数是 DMA2D_HandleTypeDef 类型结构体指针变量，用于配置 DMA2D 的基本参数，结构体变量成员的详细介绍看本章 3.4 小节。
- 第 2 个参数是源数据地址 1。
- 第 3 个参数是源数据地址 2。

- 第 4 个参数是目的数据地址。
- 第 5 个参数是源数据的长度，即每行的像素个数。
- 第 6 个参数是源数据的高度，即行数。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
DMA2D_HandleTypeDef Dma2dHandle;

if(HAL_DMA2D_BlendingStart_IT(&Dma2dHandle,           /* DMA2D 句柄 */
                              (uint32_t)&BufferInput1, /* 源地址 1，前景色 */
                              (uint32_t)&BufferInput2, /* 源地址 2，背景色 */
                              (uint32_t)&BufferResult, /* 目的地址 */
                               SIZE_X,                 /* 源数据长度，单位像素个数*/
                               SIZE_Y)                 /* 源数据行数 */
   != HAL_OK)
{
   Error_Handler();
}
```

55.5 总结

本章节就为大家讲解这么多，DMA2D 功能比较重要，一定要做到熟练使用。

第56章 STM32H7 的 DMA2D 应用之刷色块，位图和 Alpha 混合

本章节为大家讲解 DMA2D 应用中经常用到的刷色块，刷位图，Alpha 混合和图片混合的实现。

56.1 初学者重要提示
56.2 DMA2D 驱动设计
56.3 制作 C 文件格式的位图
56.4 DMA2D 常用操作（重要）
56.5 DMA2D 驱动移植和使用
56.6 实验例程设计框架
56.7 实验例程说明（MDK）
56.8 实验例程说明（IAR）
56.9 总结

56.1 初学者重要提示

- 学习本章节前，务必优先学习第 55 章，需要对 DMA2D 的基础知识有个认识。
- DMA2D 里面有一个重要的概念就是行偏移，这知识点务必要认识到位，详见本章 2.2 小节。
- DMA2D 可以直接绘制 ARGB8888，RGB565 颜色格式位图，并且可以方便的做各种透明效果和图像混合显示。
- LCD 的加速全靠 DMA2D，所有务必要熟练掌握其用法。

56.2 DMA2D 驱动设计

56.2.1 DMA2D 驱动设计思路

DMA2D 的驱动设计比较省事：

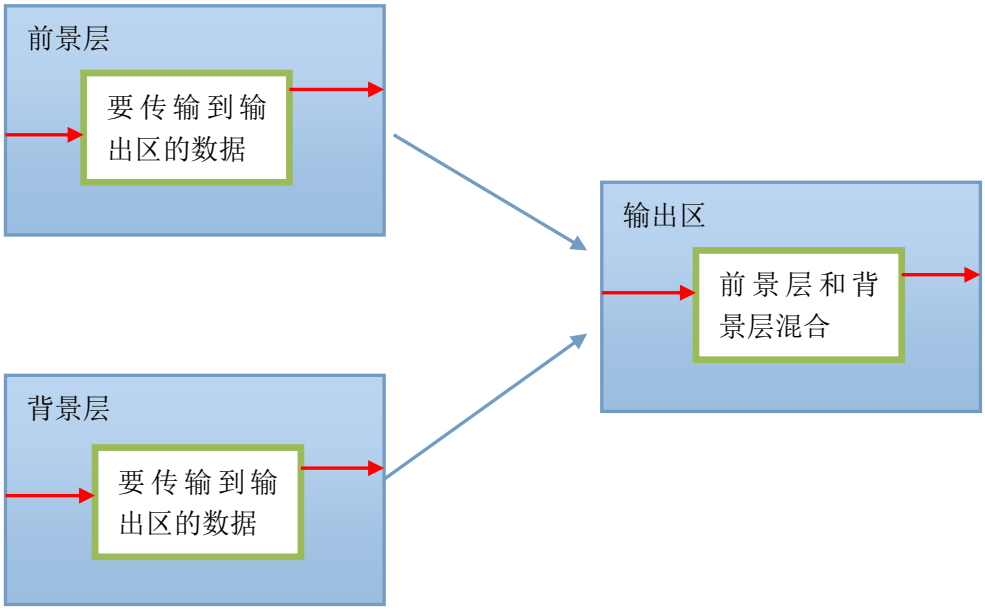
- 用户仅需调用函数__HAL_RCC_DMA2D_CLK_ENABLE 使能 DMA2D 即可使用。
- 默认 DMA2D 的 API 都是采用阻塞式，这种方式在使用 RTOS 的时候比较方便，用户可以将 GUI 任务的优先级设置的仅比空闲任务高即可，这样有高优先级任务要执行，可以及时切换到高优先级任务

里，GUI 任务等待 DMA2D 执行完成即可。从而可以充分利用 DMA2D 和 CPU，使芯片性能得到最大发挥。

- 最重要的一条，所有的 DMA2D 操作，直接采用寄存器方式，不再使用 HAL 库给的 API，让性能得到最大发挥。

56.2.2 行偏移的含义（重要）

理解 DMA2D 传输的关键就是理解行偏移。前景层，背景层和输出区都有一个行偏移的寄存器，为了方便大家理解，这里画一个框图：



条件：

- 前景层，背景层和输出区分辨率都是 800*480 分辨率，颜色格式均为 RGB565。
- 将前景层里面起始坐标（40，30），长 480，高 272 的数据与
  背景层里面起始坐标（50，40），长 480，高 272 的数据复制到
  输出区起始地址（60，50），长 480，高 272 的区域。

引出问题：

那么问题来了，前景层和背景层的起始坐标在各自数据缓冲区的起始位置都比较好计算。比如前景层就是前景层首地址加上 30*800*2 + 40*2,乘以 2 的原因是 RGB565 颜色格式的 1 个像素占用两个字节。

而难点就在如何保证前景层复制完 480 长度的数据后，如何切换到下一行。这个时候，行偏移就派上用场了，行偏移的意思是一行结束到下一行开始的距离，单位像素个数（也就是上面框图中两个红色箭头的总长度）。通过这个行偏移，我们就可以从前景层复制出来 480*272 的数据。

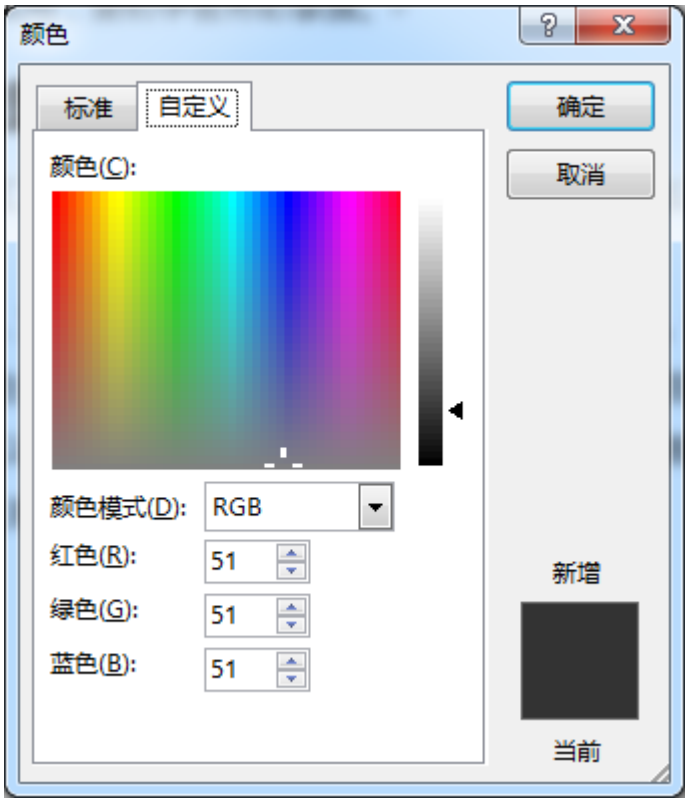
同理，背景层和输出区的行偏移也是这个意思。

56.3 制作 C 文件格式的位图

由于 DMA2D 刷新图片要用到，所以本小节为大家介绍下位图的制作。

56.3.1 什么是位图

位图（bitmap），又称为点阵图，是使用像素阵列来表示图像。位图中每个位置的像素都有自己的颜色值，这些颜色值是由 RGB 组合或者灰度值来表示。其中，RGB 是指的 Red 红色，Green 绿色和 Blue 蓝色，任何颜色都可以由这三种颜色来组成。电脑端绘图类的软件基本都有自定义颜色功能，可以很好的说明 RGB 三原色的作用：



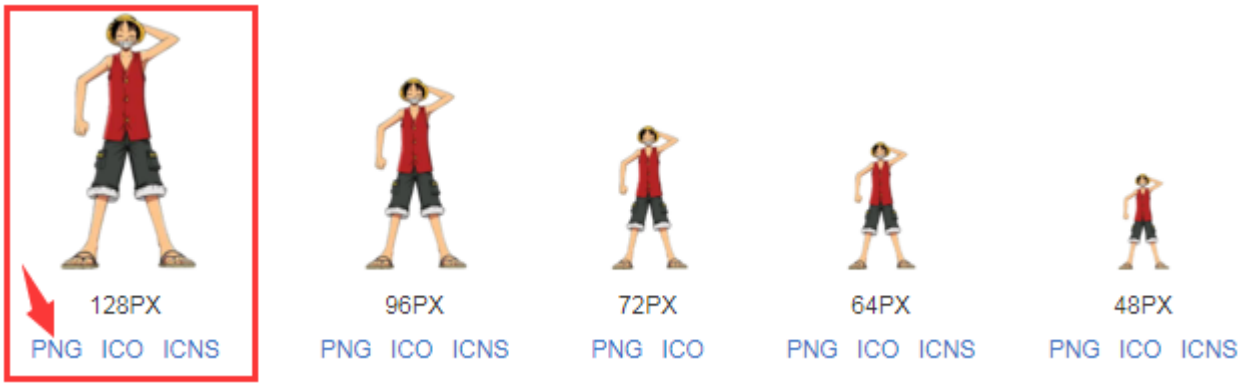
根据位深度，可以将位图分为 1 位（单色），2 位（4 色，CGA），4 位（16 色，VGA），8 位（256 色），16 位（增强色），24 位（真彩色）和 32 位等。

关于位图，还有个概念就是 alpha 通道。所谓 alpha 通道就是指在原有的图片编码方法的基础上，增加像素的透明度信息。图形处理中，通常把 RGB 三种颜色信息称为红通道、绿通道和蓝通道，相应的把透明度称为 Alpha 通道。

56.3.2 图标下载

这里为大家推荐一个图标下载网址：http://www.easyicon.net/ ，此网站非常好用，需要什么图标

直接检索关键字就可以了。这里以关键字 cartoon 进行检索，最终选择地址：https://www.easyicon.net/581941-Lufy_cartoon_icon.html 里面的图标，我们下载 128*128 点阵大小的 PNG 图片（下载的图片已经放在了本章教程配套例子的 Doc 文件夹里面）：

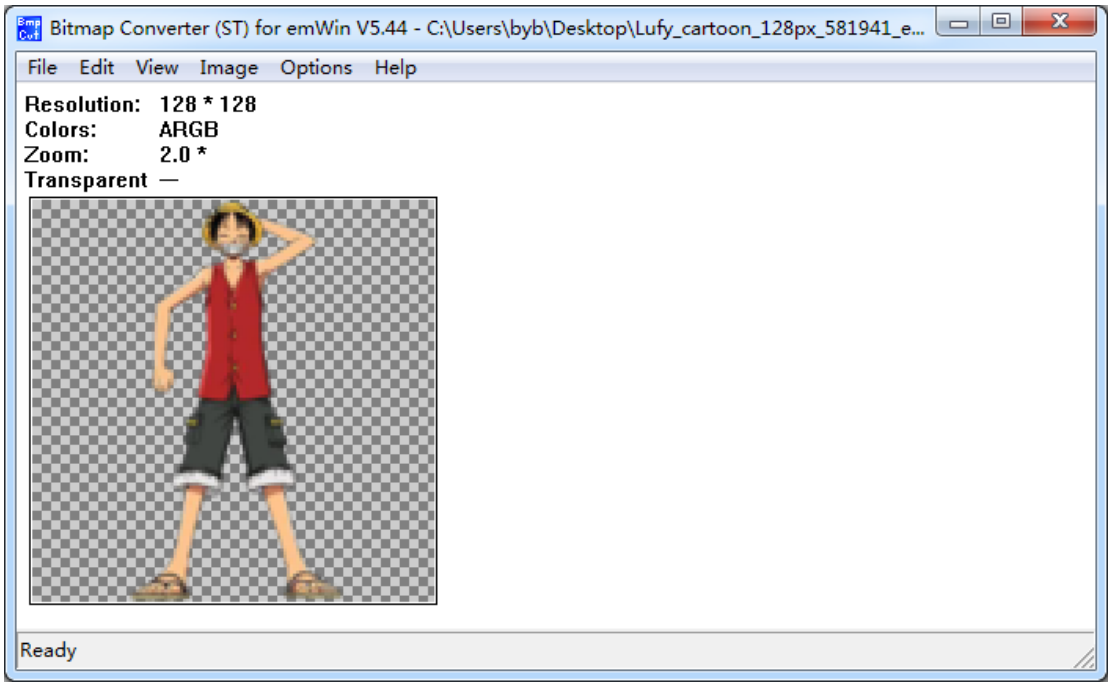


下面我们分两步走，分别将其转换为 ARGB8888 格式位图和 RGB565 格式位图。

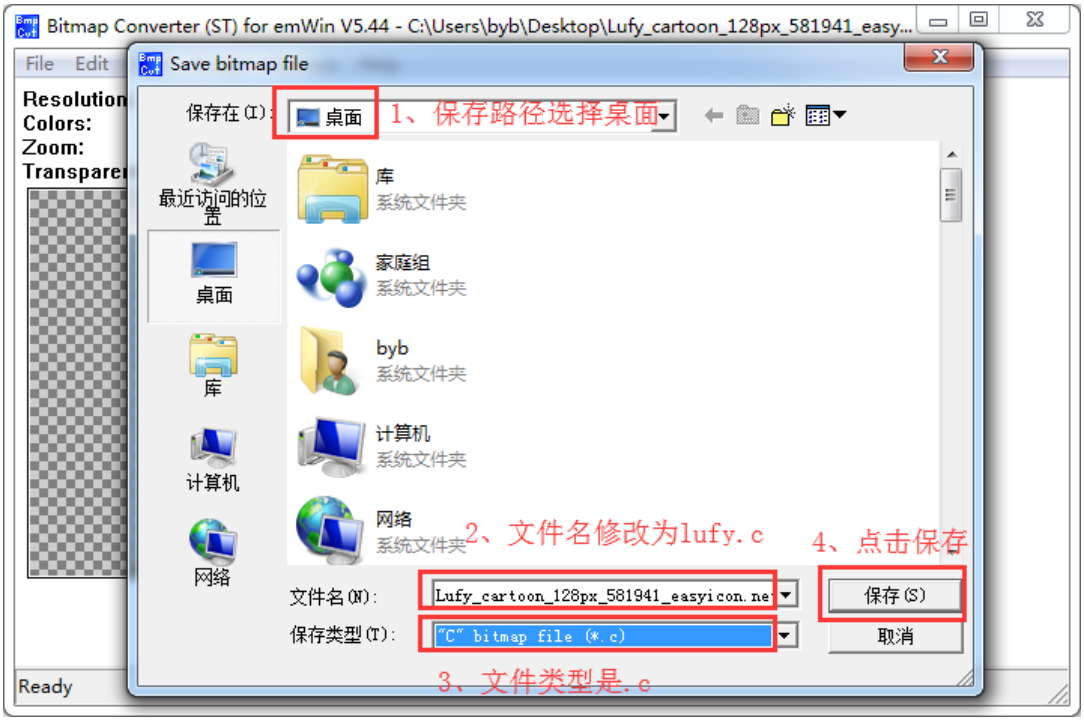
56.3.3 转换 PNG 图片为 ARGB8888 格式位图

下载小软件 BmpCvt：http://www.armbbs.cn/forum.php?mod=viewthread&tid=93478 。

- 第 1 步：打开 BmpCvtST.exe ，直接将 PNG 格式的图片拖到此软件里面即可，或者点击 File->Open 进行加载也是可以的。

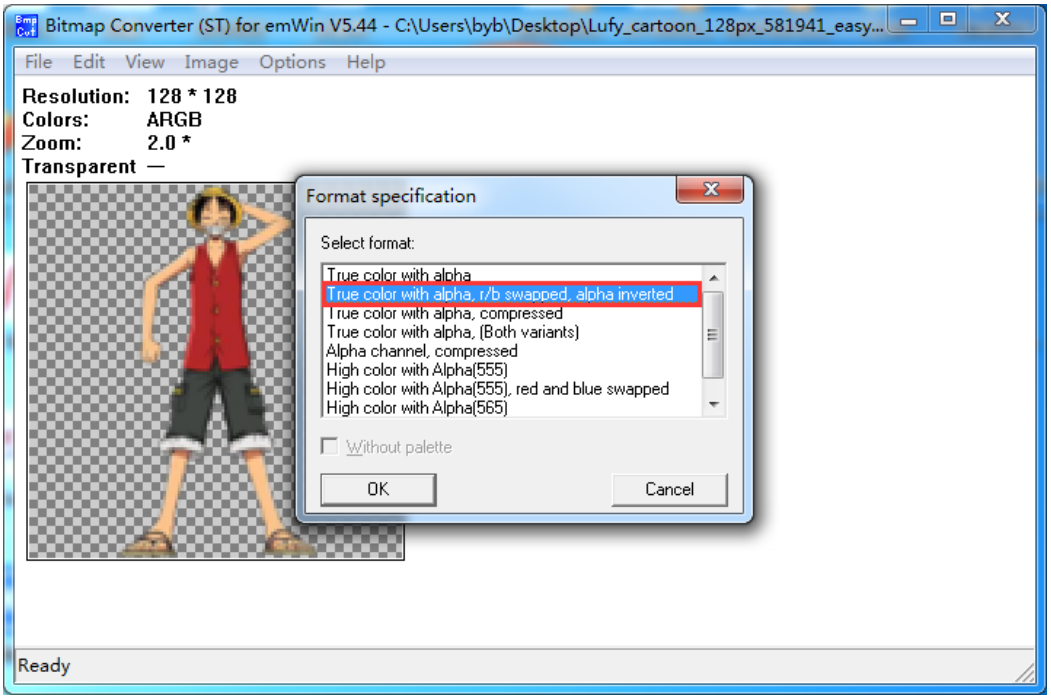


- 第 2 步：点击 File->Save as，弹出如下窗口



上面截图中共分了 4 步进行操作，其中第 2 步修改名字是因为原有的名字太长了，不方便程序代码的调用。

◆ 第 3 步：第 2 步操作完毕后，弹出如下窗口：



第 1 个选项的 True color with alpha 颜色格式是 ABGR，我们这里选择的第 2 个，这个是 ARGB 格式。当然，也可以选择 ABGR 格式，因为 H7 的 DMA2D 可以设置 Alpha 通过翻转和 R 通道与 B 通道交互位置，从而实现 ABGR 转为 ARGB 格式。

点击 OK 按钮后会在桌面出现一个新文件，即 lufy.c，保存在桌面是因为第 2 步中选择的路径是桌面。

打开 lufy.c 文件，代码如下：

```
/*********************************************************************
*                SEGGER Microcontroller GmbH & Co. KG                *
*        Solutions for real time microcontroller applications        *
*                           www.segger.com                           *
**********************************************************************
*                                                                    *
* C-file generated by                                                *
*                                                                    *
*        Bitmap Converter (ST) for emWin V5.32.                      *
*        Compiled Oct  8 2015, 11:58:22                              *
*                                                                    *
*        (c) 1998 - 2015 Segger Microcontroller GmbH & Co. KG        *
*                                                                    *
**********************************************************************
*                                                                    *
* Source file: pic                                                   *
* Dimensions:  64 * 64                                               *
* NumColors:   16bpp: 65536                                          *
*                                                                    *
**********************************************************************
*/

#include <stdlib.h>

#include "GUI.h"

#ifndef GUI_CONST_STORAGE
  #define GUI_CONST_STORAGE const
#endif

extern GUI_CONST_STORAGE GUI_BITMAP bmlufy;

static GUI_CONST_STORAGE unsigned long _aclufy[] = {
  0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
0x00000000, 0x00000000, 0x00000000,
        0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
0x00000000, 0x00000000, 0x00000000,
        0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x0042454E, 0x00494C4E, 0x00393E50,
0x0046402A, 0x00968C62, 0x093E3C26,
        0x2A787653, 0x597F8266, 0x606F6F4D, 0x5F7D7D54, 0x37747B50, 0x0B455035, 0x00745F3F, 0x00615856,
0x00655751, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000, 0x00000000,
0x00000000, 0x00000000, 0x00000000,

       /* 后面的数据未列出 */

};

GUI_CONST_STORAGE GUI_BITMAP bmlufy = {
  128, // xSize
  128, // ySize
  512, // BytesPerLine
  32, // BitsPerPixel
  (unsigned char *)_aclufy,  // Pointer to picture data
```

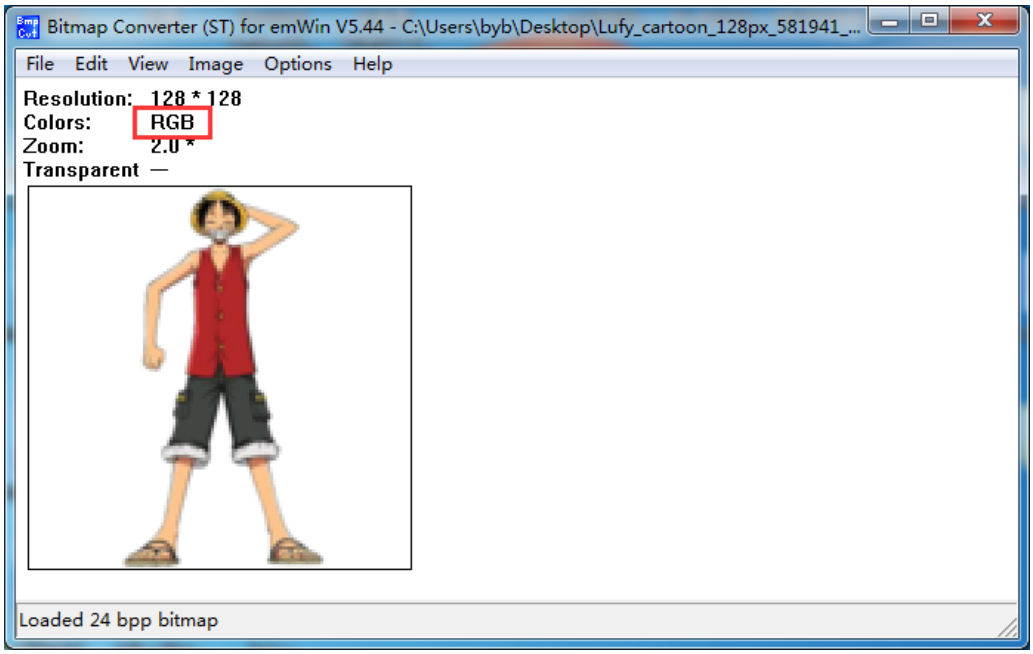
```
  NULL,  // Pointer to palette
  GUI_DRAW_BMPM8888I
};

/*************************** End of file ****************************/
```

对于生成的代码仅需用到数组 static GUI_CONST_STORAGE unsigned long _aclufy[]，这个数值要稍微修改下，将 GUI_CONST_STORAGE 修改为 const 即可用到工程里面。const 表示数组存储到 flash 里面。

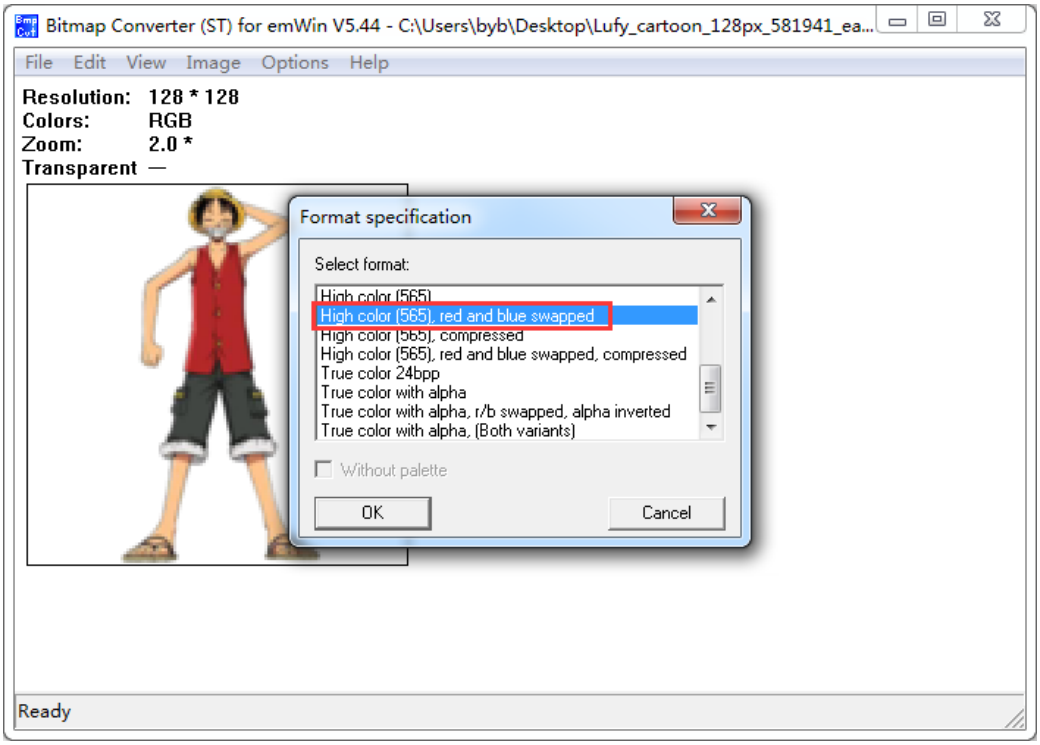
56.3.4 转换 PNG 图片为 RGB565 格式位图

转换方法与 56.4.1 小节相似，主要下面两个地方不同：

- 第 1 点不同：使用电脑端的画图小软件将前面下载的图标转换为 BMP 格式（PNG 图片中的透明通道会滤被掉），再用 BmpCvt 软件打开后的效果如下，已经没有 Alpha 通道。



- 第 2 点不同，转换格式选择如下：



转换后生成的代码如下：

```
/*********************************************************************
*                SEGGER Microcontroller GmbH & Co. KG                *
*        Solutions for real time microcontroller applications        *
*                           www.segger.com                           *
**********************************************************************
*                                                                    *
* C-file generated by                                                *
*                                                                    *
*        Bitmap Converter (ST) for emWin V5.44.                      *
*        Compiled Nov 10 2017, 08:52:20                              *
*                                                                    *
*        (c) 1998 - 2017 Segger Microcontroller GmbH & Co. KG        *
*                                                                    *
**********************************************************************
*                                                                    *
* Source file: Lufy_cartoon_128px_581941_easyiconnet                 *
* Dimensions:  128 * 128                                             *
* NumColors:   16bpp: 65536                                          *
*                                                                    *
**********************************************************************
*/

#include <stdlib.h>

#include "GUI.h"

#ifndef GUI_CONST_STORAGE
  #define GUI_CONST_STORAGE const
#endif

extern GUI_CONST_STORAGE GUI_BITMAP bmLufy_cartoon_128px_581941_easyiconnet;
```

```
static GUI_CONST_STORAGE unsigned short _acLufy_cartoon_128px_581941_easyiconnet[] = {
  0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF,
0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF,
0xFFFF, 0xFFFF, 0xFFFF,
        0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF,
0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF,
0xFFFF, 0xFFFF, 0xF7BE,
        0xE73C, 0xD699, 0xC657, 0xCE77, 0xDF1A, 0xF7BE, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF,
0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF,
0xFFFF, 0xFFFF, 0xFFFF,
        0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF,
0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF,
0xFFFF, 0xFFFF, 0xFFFF,
        0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF, 0xFFFF,

       /* 后面的数据未列出 */

};

GUI_CONST_STORAGE GUI_BITMAP bmlufy = {
  128, // xSize
  128, // ySize
  256, // BytesPerLine
  16, // BitsPerPixel
  (unsigned char *) aclufy,  // Pointer to picture data
  NULL,  // Pointer to palette
  GUI_DRAW_BMPM565
};

/************************* End of file ****************************/
```

这里生成的代码也是仅需用到数组 static GUI_CONST_STORAGE unsigned long _aclufy[]，并将 GUI_CONST_STORAGE 修改为 const 即可用到工程里面。

56.4 DMA2D 常用操作（重要）

DMA2D 的常用 API 要熟练掌握，后面的 GUI 的底层驱动加速，JPEG 硬解，摄像头等部分都要用到。这里为大家介绍如下几个常用 API：

- _DMA2D_Fill
- _DMA2D_Copy
- _DMA2D_MixColorsBulk
- _DMA2D_AlphaBlendingBulk
- _DMA2D_DrawAlphaBitmap

56.4.1 函数_DMA2D_Fill

函数原型：

```
1.  /*
2.  *********************************************************************************************************
```

```
3.  *    函 数 名: _DMA2D_Fill
4.  *    功能说明: DMA2D 颜色填充功能
5.  *    形    参: pDst          颜色数据目的地址
6.  *              xSize         色块 X 轴大小，即每行像素数
7.  *              ySize         色块 Y 轴大小，即行数
8.  *              OffLine       前景层图像的行偏移
9.  *              ColorIndex    色块颜色值
10. *              PixelFormat   目标区颜色格式
11. *    返 回 值: 无
12. *********************************************************************************************************
13. */
14. static void _DMA2D_Fill(void * pDst,
15.                          uint32_t xSize,
16.                         uint32_t ySize,
17.                         uint32_t OffLine,
18.                         uint32_t ColorIndex,
19.                         uint32_t PixelFormat)
20. {
21.
22.     /* DMA2D 采用寄存器到存储器模式，这种模式用不到前景层和背景层 */
23.     DMA2D->CR      = 0x00030000UL | (1 << 9);
24.     DMA2D->OCOLR   = ColorIndex;
25.     DMA2D->OMAR    = (uint32_t)pDst;
26.     DMA2D->OOR     = OffLine;
27.     DMA2D->OPFCCR  = PixelFormat;
28.     DMA2D->NLR     = (uint32_t)(xSize << 16) | (uint16_t)ySize;
29.
30.     /* 启动传输 */
31.     DMA2D->CR   |= DMA2D_CR_START;
32.
33.     /* 等待 DMA2D 传输完成 */
34.     while (DMA2D->CR & DMA2D_CR_START) {}
35. }
```

函数描述：

此函数主要用于 LCD 的颜色填充。

- 第 23 行，设置 DMA2D 采用寄存器往存储器传输数据模式，即 DMA2D 将 OCOLR 寄存器设置颜色值填充到存储器里面。
- 第 24 行，OCOLR 寄存器用于设置输出颜色值，特别注意要跟第 27 行的输出颜色格式匹配。比如设置的是 RGB565，那么设置的颜色值就要是 16 位色的。
- 第 25 行，设置输出填充区的首地址。
- 第 26 行，设置输出行偏移。
- 第 27 行，设置输出颜色格式，如果是输出到 LCD 显存，那么此格式要与 LCD 颜色格式一致，否则显示不正常。
- 第 28 行，高 16 位用于设置每行的像素数，低 16 位用于设置行数，合并起来决定总传输次数。
- 第 31-34 行，启动传输，并等待传输完成。

注意事项：

1. 使用此函数前务必要调用函数__HAL_RCC_DMA2D_CLK_ENABLE 使能 DMA2D 时钟。

使用举例：

起始坐标（24, 20），输出颜色格式 RGB565，绘制一个 128x128 的红色填充区：

```
1.      _DMA2D_Fill((void *)(SDRAM_LCD_BUF1 + g_LcdWidth*20*2 + 24*2),  /* 显示起始地址(24, 20) */
2.                  128,                                                /* 色块长 */
3.                  128,                                                /* 色块高 */
4.                  g_LcdWidth-128,                                     /* 色块行偏移 */
5.                  CL_RED,                                             /* 色块颜色 */
6.                  LTDC_PIXEL_FORMAT_RGB565);                          /* 色块颜色格式 */
```

- 第 1 行，根据设置的起始坐标，计算起始坐标在 LCD 显存中的具体位置。

  SDRAM_LCD_BUF1 表示 LCD 显存首地址。

  g_LcdWidth*20*2 计算的是行数占用的字节数。

  24*2 计算的是所在行的具体地址。

  乘以 2 是因为 RGB565 颜色格式的 1 个像素占用两个字节。

- 第 4 行，输出行偏移的意思就是一行结束到下一行开始的距离，单位像素个数。由于填充区的长度是 128，那么行偏移就是 LCD 的长度减去 128。

56.4.2 函数_DMA2D_Copy

函数原型：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: _DMA2D_Copy
4.  *    功能说明: 通过 DMA2D 从前景层复制指定区域的颜色数据到目标区域
5.  *    形    参: pSrc         颜色数据源地址
6.  *              pDst         颜色数据目的地址
7.  *              xSize        目的区域的 X 轴大小，即每行像素数
8.  *              ySize        目的区域的 Y 轴大小，即行数
9.  *              OffLineSrc   前景层图像的行偏移
10. *              OffLineDst   输出的行偏移
11. *              PixelFormat  目标区颜色格式
12. *    返 回 值: 无
13. *********************************************************************************************************
14. */
15. static void _DMA2D_Copy(void * pSrc,
16.                          void * pDst,
17.                              uint32_t xSize,
18.                              uint32_t ySize,
19.                              uint32_t OffLineSrc,
20.                              uint32_t OffLineDst,
21.                              uint32_t PixelFormat)
22. {
23.
24.     /* DMA2D 采用存储器到存储器模式, 这种模式是前景层作为 DMA2D 输入 */
25.     DMA2D->CR      = 0x00000000UL | (1 << 9);
26.     DMA2D->FGMAR   = (uint32_t)pSrc;
27.     DMA2D->OMAR    = (uint32_t)pDst;
28.     DMA2D->FGOR    = OffLineSrc;
29.     DMA2D->OOR     = OffLineDst;
30.
31.     /* 前景层和输出区域都采用的 RGB565 颜色格式 */
32.     DMA2D->FGPFCCR = LTDC_PIXEL_FORMAT_RGB565;
```

```
33.       DMA2D->OPFCCR  = LTDC_PIXEL_FORMAT_RGB565;
34.
35.       DMA2D->NLR     = (uint32_t)(xSize << 16) | (uint16_t)ySize;
36.
37.       /* 启动传输 */
38.       DMA2D->CR    |= DMA2D_CR_START;
39.
40.       /* 等待 DMA2D 传输完成 */
41.       while (DMA2D->CR & DMA2D_CR_START) {}
42.   }
```

函数描述：

此函数用于从前景层复制指定区域的颜色数据到目标区域。

- 第 25 行，设置 DMA2D 采用存储器往存储器传输数据模式，输入存储器只能是前景层。即 DMA2D 将 FGMAR 寄存器所指向的存储数据传输到寄存器 OMAR 所指向的存储器。
- 第 26 行，设置前景层数据首地址。
- 第 27 行，设置输出存储器首地址。
- 第 28 行，设置前景层行偏移。
- 第 29 行，设置输出区行偏移。
- 第 32-33 行，设置前景层和输出区域都采用 RGB565 颜色格式。另外注意，如果是输出到 LCD 显存，那么输出颜色格式要与 LCD 颜色格式一致，否则显示不正常。
- 第 35 行，高 16 位用于设置每行的像素数，低 16 位用于设置行数，合并起来决定总传输次数。
- 第 38-41 行，启动传输，并等待传输完成。

注意事项：

1. 使用此函数前务必要调用函数__HAL_RCC_DMA2D_CLK_ENABLE 使能 DMA2D 时钟。

使用举例：

将大小 128*128，颜色格式为 RGB565 的位图绘制到 LCD 起始坐标为（328, 20）的区域，输出颜色格式也配置为 RGB565。

```
1.        _DMA2D_Copy((uint32_t *)_acmickey,                                  /* 位图地址 */
2.                    (uint32_t *)(SDRAM_LCD_BUF1 + g_LcdWidth*20*2 + 328*2),  /* 显示起始地址(328, 20) */
3.                    128,                                                    /* 位图长 */
4.                    128,                                                    /* 位图高 */
5.                    0,                                                      /* 位图行偏移 */
6.                    g_LcdWidth-128,                                         /* 目标区行偏移 */
7.                    LTDC_PIXEL_FORMAT_RGB565);                              /* 目标区颜色格式 */
```

- 第 1 行是位图首地址。
- 第 2 行，根据设置的起始坐标，计算起始坐标在 LCD 显存中的具体位置。

  SDRAM_LCD_BUF1 表示 LCD 显存首地址。

  g_LcdWidth*20*2 计算的是行数占用的字节数。

  328*2 计算的是所在行的具体地址。

  乘以 2 是因为 RGB565 颜色格式的 1 个像素占用两个字节。

- 第 5 行是位图的行偏移，行偏移的意思就是一行结束到下一行开始的距离，单位像素个数。由于整个位图都要绘制，所有行偏移就是 0。
- 第 6 行，输出行偏移的，由于输出区需要的长度是 128，那么行偏移就是 LCD 的长度减去 128。

56.4.3 函数_DMA2D_MixColorsBulk

函数原型：

```
/*
*********************************************************************************************************
*    函 数 名: _DMA2D_MixColorsBulk
*    功能说明: 前景层和目标区域的颜色混合
*    形    参: pColorFG     前景层数据源地址
*              OffLineSrcFG  前景层图像的行偏移
*              pColorDst     目标区数据地址
*              OffLineDst    目标区的行偏移
*              xSize         目的区域的 X 轴大小，即每行像素数
*              ySize         目的区域的 Y 轴大小，即行数
*              Intens        设置前景层的透明度，255 表示完全不透明，0 表示完全透明
*    返 回 值: 无
*********************************************************************************************************
*/
static void _DMA2D_MixColorsBulk(uint32_t * pColorFG,
                                 uint32_t OffLineSrcFG,
                                 uint32_t * pColorDst,
                                 uint32_t OffLineDst,
                                 uint32_t xSize,
                                 uint32_t ySize,
                                 uint8_t Intens)
{
/* DMA2D 采用存储器到存储器模式，这种模式前景层和背景层作为 DMA2D 输入，且支持颜色格式转换和颜色混合 */
    DMA2D->CR      = 0x00020000UL | (1 << 9);
    DMA2D->FGMAR   = (uint32_t)pColorFG;
    DMA2D->BGMAR   = (uint32_t)pColorDst;
    DMA2D->OMAR    = (uint32_t)pColorDst;
    DMA2D->FGOR    = OffLineSrcFG;
    DMA2D->BGOR    = OffLineDst;
    DMA2D->OOR     = OffLineDst;

    /* 前景层，背景层和输出区都是用的 RGB565 格式 */
    DMA2D->FGPFCCR = LTDC_PIXEL_FORMAT_RGB565
                   | (1UL << 16)
                   | ((uint32_t)Intens << 24);
    DMA2D->BGPFCCR = LTDC_PIXEL_FORMAT_RGB565;
    DMA2D->OPFCCR  = LTDC_PIXEL_FORMAT_RGB565;

    DMA2D->NLR     = (uint32_t)(xSize << 16) | (uint16_t)ySize;

    /* 启动传输 */
    DMA2D->CR   |= DMA2D_CR_START;

    /* 等待 DMA2D 传输完成 */
    while (DMA2D->CR & DMA2D_CR_START) {}
}
```

函数描述：

此函数用于前景层和目标区域的颜色混合，并将混合后的图像输出到目标区域。

- 第 24 行，DMA2D 采用存储器到存储器模式，这种模式把前景层和背景层作为 DMA2D 输入，且支持颜色格式转换和颜色混合。
- 第 25 行，设置前景层数据首地址。
- 第 26 行，设置背景层数据首地址，这里是把输出区中的数据作为背景层。
- 第 27 行，设置输出存储器首地址。
- 第 28 行，设置前景层行偏移。
- 第 29 行，设置背景层行偏移。
- 第 29 行，设置输出区行偏移。
- 第 33-37 行，前景层，背景层和输出区都采用 RGB565 格式，并且为前景层配置一个 Alpha 值。另外注意，如果是输出到 LCD 显存，那么输出颜色格式要与 LCD 颜色格式一致，否则显示不正常。
- 第 39 行，高 16 位用于设置每行的像素数，低 16 位用于设置行数，合并起来决定总传输次数。
- 第 42-45 行，启动传输，并等待传输完成。

注意事项：

1. 使用此函数前务必要调用函数__HAL_RCC_DMA2D_CLK_ENABLE 使能 DMA2D 时钟。

使用举例：

将大小 128*128，颜色格式为 RGB565 的位图绘制到 LCD 起始坐标为（176, 168）的区域，输出颜色格式也配置为 RGB565，透明度设置为 200（255 表示完全不透明，0 表示完全透明）。

```
1.  _DMA2D_MixColorsBulk((uint32_t *)_achuoying,                                /* 位图地址 */
2.                       0,                                                    /* 位图行偏移 */
3.                       (uint32_t *)(SDRAM_LCD_BUF1 + g_LcdWidth*168*2 + 176*2), /* 显示起始地址(176, 168) */
4.                        g_LcdWidth-128,                                      /* 目标区行偏移 */
5.                        128,                                                 /* 目标区长 */
6.                        128,                                                 /* 目标区高 */
7.                        200);                                                /* 位图显示透明度 200 */
```

- 第 1 行是位图首地址。
- 第 2 行是位图的行偏移，行偏移的意思就是一行结束到下一行开始的距离，单位像素个数。由于整个位图都要绘制，所有行偏移就是 0。
- 第 3 行，根据设置的起始坐标，计算起始坐标在 LCD 显存中的具体位置。

  SDRAM_LCD_BUF1 表示 LCD 显存首地址。

  g_LcdWidth*168*2 计算的是行数占用的字节数。

  176*2 计算的是所在行的具体地址。

  乘以 2 是因为 RGB565 颜色格式的 1 个像素占用两个字节。
- 第 4 行，输出行偏移的，由于输出区需要的长度是 128，那么行偏移就是 LCD 的长度减去 128。

56.4.4函数_DMA2D_AlphaBlendingBulk

函数原型：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: _DMA2D_AlphaBlendingBulk
4.  *    功能说明: 前景层和背景层的颜色混合
5.  *    形    参: pColorFG      前景层源数据地址
6.  *              OffLineSrcFG  前景层源数据行偏移
7.  *              pColorBG      背景层源数据地址
8.  *              OffLineSrcBG  背景层源数据行偏移
9.  *              pColorDst     目标区地址
10. *              OffLineDst    目标区行偏移
11. *              xSize         目标区域的 X 轴大小，即每行像素数
12. *              ySize         目标区域的 Y 轴大小，即行数
13. *    返 回 值: 无
14. *********************************************************************************************************
15. */
16. static void _DMA2D_AlphaBlendingBulk(uint32_t * pColorFG,
17.                                       uint32_t OffLineSrcFG,
18.                                       uint32_t * pColorBG,
19.                                      uint32_t OffLineSrcBG,
20.                                      uint32_t * pColorDst,
21.                                      uint32_t OffLineDst,
22.                                     uint32_t xSize,
23.                                      uint32_t ySize)
24. {
25. /* DMA2D 采用存储器到存储器模式，这种模式前景层和背景层作为 DMA2D 输入，且支持颜色格式转换和颜色混合 */
26.     DMA2D->CR      = 0x00020000UL | (1 << 9);
27.     DMA2D->FGMAR   = (uint32_t)pColorFG;
28.     DMA2D->BGMAR   = (uint32_t)pColorBG;
29.     DMA2D->OMAR    = (uint32_t)pColorDst;
30.     DMA2D->FGOR    = OffLineSrcFG;
31.     DMA2D->BGOR    = OffLineSrcBG;
32.     DMA2D->OOR     = OffLineDst;
33.
34.     /* 前景层，背景层采用 ARGB8888 格式，输出区采用 RGB565 格式 */
35.     DMA2D->FGPFCCR = LTDC_PIXEL_FORMAT_ARGB8888;
36.     DMA2D->BGPFCCR = LTDC_PIXEL_FORMAT_ARGB8888;
37.     DMA2D->OPFCCR  = LTDC_PIXEL_FORMAT_RGB565;
38.     DMA2D->NLR     = (uint32_t)(xSize << 16) | (uint16_t)ySize;
39.
40.     /* 启动传输 */
41.     DMA2D->CR   |= DMA2D_CR_START;
42.
43.     /* 等待 DMA2D 传输完成 */
44.     while (DMA2D->CR & DMA2D_CR_START) {}
45. }
```

函数描述：

此函数用于前景层和背景层的颜色混合，并将混合后的图像输出到目标区域。

- 第 26 行，DMA2D 采用存储器到存储器模式，这种模式把前景层和背景层作为 DMA2D 输入，且支持颜色格式转换和颜色混合。

- 第 27 行，设置前景层数据首地址。
- 第 28 行，设置背景层数据首地址。
- 第 29 行，设置输出存储器首地址。
- 第 30 行，设置前景层行偏移。
- 第 31 行，设置背景层行偏移。
- 第 32 行，设置输出区行偏移。
- 第 35-37 行，前景层，背景层采用 ARGB8888 格式，输出区采用 RGB565 格式。另外注意，如果是输出到 LCD 显存，那么输出颜色格式要与 LCD 颜色格式一致，否则显示不正常。
- 第 38 行，高 16 位用于设置每行的像素数，低 16 位用于设置行数，合并起来决定总传输次数。
- 第 41-44 行，启动传输，并等待传输完成。

注意事项：

1. 使用此函数前务必要调用函数__HAL_RCC_DMA2D_CLK_ENABLE 使能 DMA2D 时钟。

使用举例：

将两个大小 128*128，颜色格式为 ARGB8888 的位图混合后绘制到 LCD 起始坐标为 (24, 168) 的区域，输出颜色格式配置为 RGB565。

```
1.  _DMA2D_AlphaBlendingBulk((uint32_t *)_aclufei,                          /* 前景层位图地址 */
2.                           0,                                            /* 前景层行偏移  */
3.                           (uint32_t *)_acsuolong,                       /* 背景层位图地址  */
4.                           0,                                            /* 背景层行偏移  */
5.                           (uint32_t *)(SDRAM_LCD_BUF1 +  g_LcdWidth*168*2 + 24*2), /* 显示起始地址(24, 168) */
6.                           g_LcdWidth-128,                               /* 目标区行偏移 */
7.                           128,                                          /* 目标区长 */
8.                           128);                                         /* 目标区高 */
```

- 第 1 行是前景层位图首地址。
- 第 2 行是位图的行偏移，行偏移的意思就是一行结束到下一行开始的距离，单位像素个数。由于整个位图都要绘制，所有行偏移就是 0。
- 第 3 行是背景层位图首地址。
- 第 4 行是背景层行偏移。
- 第 5 行，根据设置的起始坐标，计算起始坐标在 LCD 显存中的具体位置。

  SDRAM_LCD_BUF1 表示 LCD 显存首地址。

  g_LcdWidth*168*2 计算的是行数占用的字节数。

  24*2 计算的是所在行的具体地址。

  乘以 2 是因为 RGB565 颜色格式的 1 个像素占用两个字节。
- 第 6 行，输出行偏移的，由于输出区需要的长度是 128，那么行偏移就是 LCD 的长度减去 128。

56.4.5 函数_DMA2D_DrawAlphaBitmap

函数原型：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: _DMA2D_DrawAlphaBitmap
4.  *    功能说明: ARGB8888 格式位图显示
5.  *    形    参: pDst        目标区地址
6.  *              pSrc        源数据地址，即位图首地址
7.  *              xSize       目标区域的 X 轴大小，即每行像素数
8.  *              ySize       目标区域的 Y 轴大小，即行数
9.  *              OffLineSrc  源数据行偏移
10. *              OffLineDst  目标区行偏移
11. *              PixelFormat 目标区颜色格式
12. *    返 回 值: 无
13. *********************************************************************************************************
14. */
15. static void _DMA2D_DrawAlphaBitmap(void  * pDst,
16.                                    void  * pSrc,
17.                                    uint32_t xSize,
18.                                    uint32_t ySize,
19.                                    uint32_t OffLineSrc,
20.                                    uint32_t OffLineDst,
21.                                    uint32_t PixelFormat)
22. {
23. /* DMA2D 采用存储器到存储器模式，这种模式前景层和背景层作为 DMA2D 输入，且支持颜色格式转换和颜色混合 */
24.     DMA2D->CR      = 0x00020000UL | (1 << 9);
25.     DMA2D->FGMAR   = (uint32_t)pSrc;
26.     DMA2D->BGMAR   = (uint32_t)pDst;
27.     DMA2D->OMAR    = (uint32_t)pDst;
28.     DMA2D->FGOR    = OffLineSrc;
29.     DMA2D->BGOR    = OffLineDst;
30.     DMA2D->OOR     = OffLineDst;
31.
32.     /* 前景层颜色格式是 LTDC_PIXEL_FORMAT_ARGB8888，即位图的颜色格式，背景层和输出区颜色格式可配置 */
33.     DMA2D->FGPFCCR = LTDC_PIXEL_FORMAT_ARGB8888;
34.     DMA2D->BGPFCCR = PixelFormat;
35.     DMA2D->OPFCCR  = PixelFormat;
36.     DMA2D->NLR     = (uint32_t)(xSize << 16) | (uint16_t)ySize;
37.
38.     /* 启动传输 */
39.     DMA2D->CR   |= DMA2D_CR_START;
40.
41.     /* 等待 DMA2D 传输完成 */
42.     while (DMA2D->CR & DMA2D_CR_START) {}
43. }
```

函数描述：

此函数用于在指定位置显示 ARGB8888 格式位图。

- 第 24 行，DMA2D 采用存储器到存储器模式，这种模式把前景层和背景层作为 DMA2D 输入，且支持颜色格式转换和颜色混合。
- 第 25 行，设置前景层数据首地址。
- 第 26 行，设置背景层数据首地址，这里是把输出区中的数据作为背景层。
- 第 27 行，设置输出存储器首地址。
- 第 28 行，设置前景层行偏移。
- 第 29 行，设置背景层行偏移。

- 第 30 行，设置输出区行偏移。
- 第 33-35 行，前景层颜色格式是 LTDC_PIXEL_FORMAT_ARGB8888，即位图的颜色格式。背景层和输出区颜色格式可配置。另外注意，如果是输出到 LCD 显存，那么输出颜色格式要与 LCD 颜色格式一致，否则显示不正常。
- 第 36 行，高 16 位用于设置每行的像素数，低 16 位用于设置行数，合并起来决定总传输次数。
- 第 39-42 行，启动传输，并等待传输完成。

注意事项：

1. 使用此函数前务必要调用函数__HAL_RCC_DMA2D_CLK_ENABLE 使能 DMA2D 时钟。

使用举例：

将大小 128*128，颜色格式为 ARGB8888 的位图绘制到 LCD 起始坐标为（176, 20）的区域，输出颜色格式配置为 RGB565。

```
1.  _DMA2D_DrawAlphaBitmap((void *)(SDRAM_LCD_BUF1 + g_LcdWidth*20*2 + 176*2), /* 显示起始地址(176, 20) */
2.                         (void *)_aclufei,                                  /* 位图地址 */
3.                         128,                                               /* 位图长 */
4.                         128,                                               /* 位图高 */
5.                         0,                                                 /* 位图行偏移 */
6.                         g_LcdWidth-128,                                    /* 目标区行偏移 */
7.                         LTDC_PIXEL_FORMAT_RGB565);                         /* 目标区颜色格式 */
```

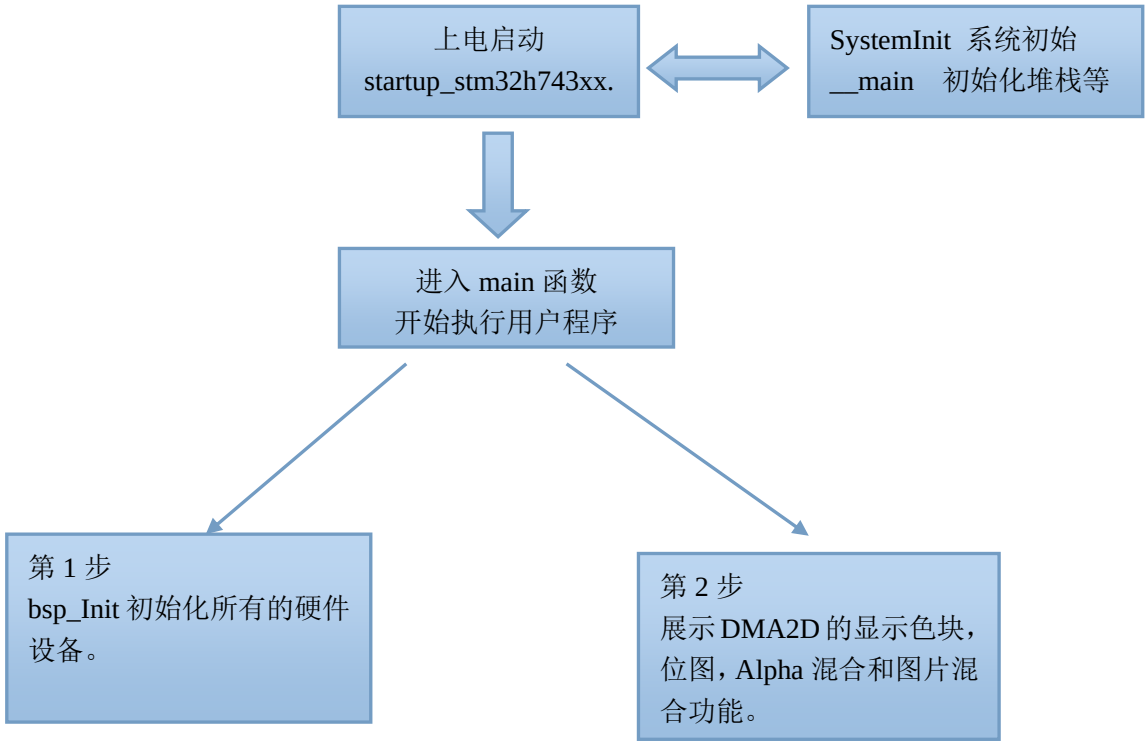
- 第 1 行，根据设置的起始坐标，计算起始坐标在 LCD 显存中的具体位置。
  SDRAM_LCD_BUF1 表示 LCD 显存首地址。
  g_LcdWidth*20*2 计算的是行数占用的字节数。
  176*2 计算的是所在行的具体地址。
  乘以 2 是因为 RGB565 颜色格式的 1 个像素占用两个字节
- 第 2 行是位图首地址。
- 第 5 行是位图的行偏移，行偏移的意思就是一行结束到下一行开始的距离，单位像素个数。由于整个位图都要绘制，所有行偏移就是 0。
- 第 6 行，输出行偏移的，由于输出区需要的长度是 128，那么行偏移就是 LCD 的长度减去 128。

56.5 DMA2D 驱动移植和使用

DMA2D 的驱动移植比较简单，用户仅需调用函数__HAL_RCC_DMA2D_CLK_ENABLE 使能时钟，然后用到哪个函数直接复到工程里面调用即可。

56.6 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED ，LCD，和 SDRAM。
  - 第 2 步，DMA2D 应用程序设计部分，主要展示 DMA2D 刷色块，刷位图，Alpha 混合和图片混合功能。

56.7 实验例程说明（MDK）

配套例子：

V7-035_DMA2D 功能测试（显示色块，位图，Alpha 混合和图片混合等）

实验目的：

1. 学习 DMA2D 显示色块，位图，Alpha 混合和图片混合等。

实验内容：

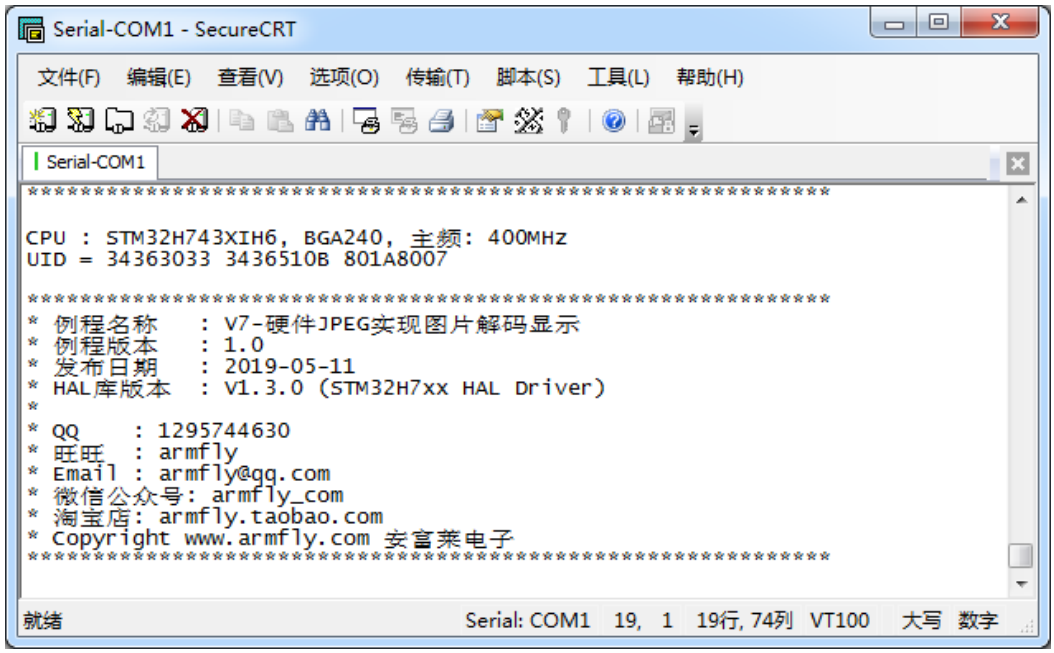
1. 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。
2. 第 1 个图：使用 DMA2D 刷色块。
3. 第 2 个图：显示 ARGB8888 位图。
4. 第 3 个图：显示 RGB565 位图。
5. 第 4 个图：两个位图混合。

6. 第 5 个图：Alpha 透明度 200 的位图显示。
7. 第 6 个图：Alpha 透明度 100 的位图显示。

LCD 界面显示效果如下：

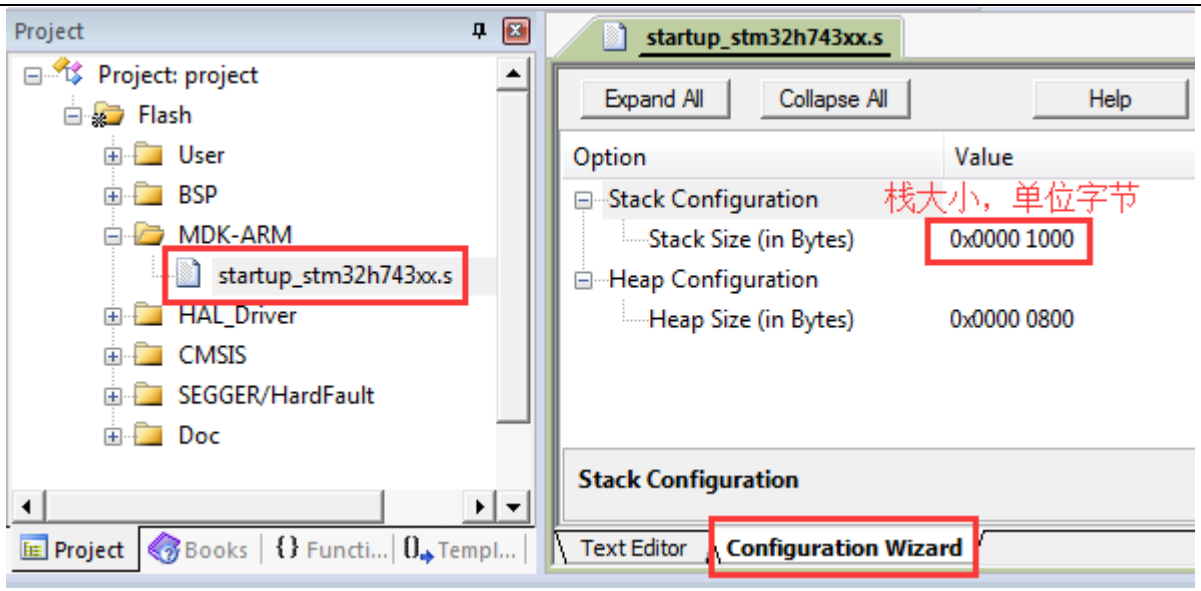
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

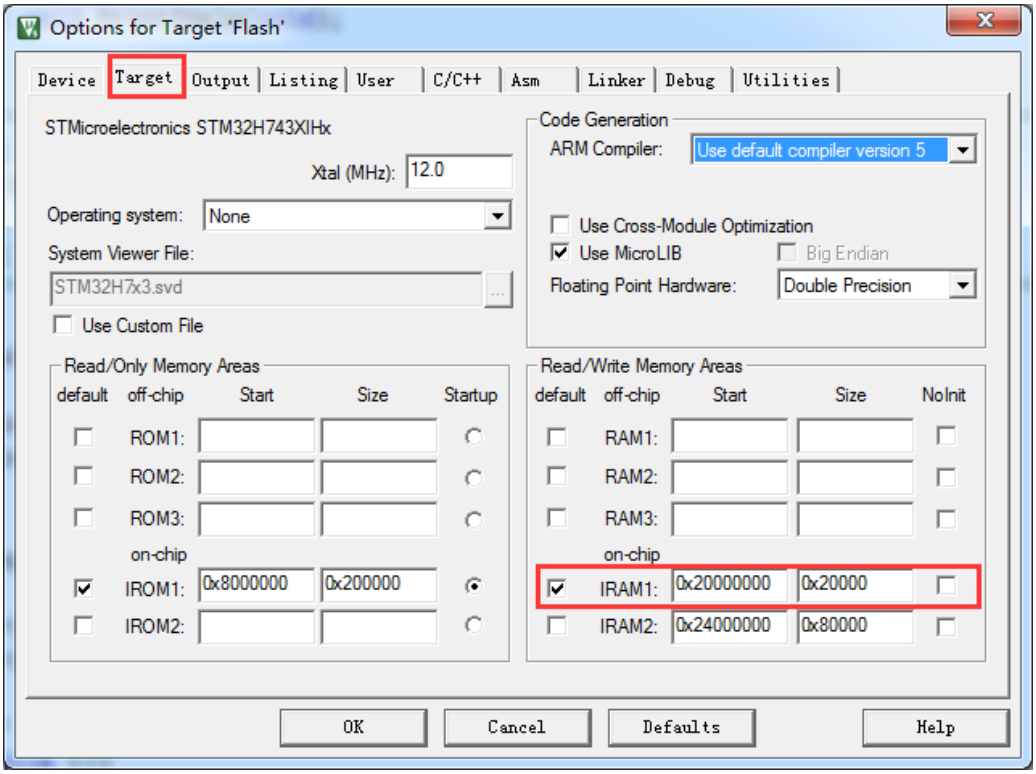


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */

    bsp_InitI2C();     /* 初始化 I2C 总线 */
    TOUCH_InitHard();  /* 初始化触摸芯片，LCD 面板型号的检查也在此函数，所以要在函数 LCD_InitHard 前调用 */
    LCD_InitHard();    /* 初始化 LCD */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SDRAM。由于 SDRAM 要用于 LCD 的显存，方便起见，直接将其配置为 WT 模式。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;
```

```
    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SDRAM 的 MPU 属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0xC0000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_32MB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
```

```
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能:**

主程序实现如下操作:

- 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。
- 解码一张 480*272 大小的 JPEG 图片并显示。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint16_t ucBright;      /* 背光亮度(0-255) */
    FONT_T tFont;           /* 定义一个字体结构体变量，用于设置字体参数 */


    /* 设置字体参数 */
    {
        tFont.FontCode = FC_ST_16;       /* 字体代码 16 点阵 */
        tFont.FrontColor = CL_WHITE;     /* 字体颜色 */
        tFont.BackColor = CL_BLUE;       /* 文字背景颜色 */
        tFont.Space = 0;                 /* 文字间距，单位 = 像素 */
    }

    bsp_Init();         /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    /* 延迟 200ms 再点亮背光，避免瞬间高亮 */
    bsp_DelayMS(200);

    LCD_ClrScr(CL_BLUE);

    /* 界面整体显示完毕后，再打开背光，设置为缺省亮度 */
    bsp_DelayMS(100);
    ucBright = BRIGHT_DEFAULT;
    LCD_SetBackLight(ucBright);

    /* 第 1 个图: 使用 DMA2D 刷色块 ###################################################################*/
    LCD_DispStr(24, 2, "DMA2D 刷色块", &tFont);
```

```
    _DMA2D_Fill((void *)(SDRAM_LCD_BUF1 + g_LcdWidth*20*2 + 24*2), /* 显示起始地址(24, 20) */
                128,                                               /* 色块长 */
                  128,                                               /* 色块高 */
                  g_LcdWidth-128,                                    /* 色块行偏移 */
                  CL_RED,                                            /* 色块颜色 */
                  LTDC_PIXEL_FORMAT_RGB565);                         /* 色块颜色格式 */

    /* 第 2 个图：显示 ARGB8888 位图 ###################################################################*/
    LCD_DispStr(176, 2, "刷 ARGB8888 位图", &tFont);
    _DMA2D_DrawAlphaBitmap((void *)(SDRAM_LCD_BUF1 + g_LcdWidth*20*2 + 176*2), /* 显示起始地址(176, 20) */
                           (void *)_aclufei,                                   /* 位图地址 */
                           128,                                                /* 位图长 */
                           128,                                                /* 位图高 */
                           0,                                                  /* 位图行偏移 */
                           g_LcdWidth-128,                                     /* 目标区行偏移 */
                           LTDC_PIXEL_FORMAT_RGB565);                          /* 目标区颜色格式 */

    /* 第 3 个图：显示 RGB565 位图 ###################################################################*/
    LCD_DispStr(328, 2, "刷 RGB565 位图", &tFont);
    _DMA2D_Copy((uint32_t *)_acmickey,                                  /* 位图地址 */
                (uint32_t *)(SDRAM_LCD_BUF1 + g_LcdWidth*20*2 + 328*2),  /* 显示起始地址(328, 20) */
                128,                                                    /* 位图长 */
                128,                                                    /* 位图高 */
                0,                                                      /* 位图行偏移 */
                g_LcdWidth-128,                                         /* 目标区行偏移 */
                 LTDC_PIXEL_FORMAT_RGB565);                              /* 目标区颜色格式 */


    /* 第 4 个图：两个位图混合 ###################################################################*/
    LCD_DispStr(24, 150, "两个位图混合", &tFont);
    _DMA2D_AlphaBlendingBulk((uint32_t *)_aclufei,                          /* 前景层位图地址 */
                                0,                                          /* 前景层行偏移  */
                             (uint32_t *)_acsuolong,                        /* 背景层位图地址  */
                                0,                                          /* 背景层行偏移  */
                           (uint32_t *)(SDRAM_LCD_BUF1 +  g_LcdWidth*168*2 + 24*2), /* 显示起始地址(24, 168) */
                                g_LcdWidth-128,                             /* 目标区行偏移 */
                                128,                                        /* 目标区长 */
                                128);                                       /* 目标区高 */

    /* 第 5 个图：Alpha 透明度 200 的位图显示 ###################################################################*/
    LCD_DispStr(176, 150, "Alpha 透明度 200", &tFont);
    _DMA2D_MixColorsBulk((uint32_t *)_achuoying,                              /* 位图地址 */
                         0,                                                   /* 位图行偏移 */
                        (uint32_t *)(SDRAM_LCD_BUF1 + g_LcdWidth*168*2 + 176*2), /* 显示起始地址(176, 168) */
                            g_LcdWidth-128,                                   /* 目标区行偏移 */
                            128,                                              /* 目标区长 */
                            128,                                              /* 目标区高 */
                            200);                                             /* 位图显示透明度200 */

    /* 第 6 个图：Alpha 透明度 100 的位图显示 ###################################################################*/
    LCD_DispStr(328, 150, "Alpha 透明度 100", &tFont);
    _DMA2D_MixColorsBulk((uint32_t *)_achuoying,                              /* 位图地址 */
                         0,                                                   /* 位图行偏移 */
                        (uint32_t *)(SDRAM_LCD_BUF1 + g_LcdWidth*168*2 + 328*2), /* 显示起始地址(328, 168) */
                            g_LcdWidth-128,                                   /* 目标区行偏移 */
                            128,                                              /* 目标区长 */
                            128,                                              /* 目标区高 */
                            100);                                             /* 位图显示透明度200 */
```

```
    bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */

    while (1)
    {
      bsp_Idle();

        /* 判断软件定时器 0 是否超时 */
        if(bsp_CheckTimer(0))
        {
            /* 每隔 200ms 进来一次 */
            bsp_LedToggle(2);
        }
    }
}

/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */

   /* 延迟 200ms 再点亮背光，避免瞬间高亮 */
    bsp_DelayMS(200);

    LCD_ClrScr(CL_BLUE);

    /* 界面整体显示完毕后，再打开背光，设置为缺省亮度 */
    bsp_DelayMS(100);
    LCD_SetBackLight(BRIGHT_DEFAULT);

    TestJpeg(); /* JPEG 测试 */
}

/*
*********************************************************************************************************
*    函 数 名: TestJpeg
*    功能说明: 硬件 JPEG 测试
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void TestJpeg(void)
{
    int iTimeStart, iTimeEnd;
    FONT_T tFont;      /* 定义一个字体结构体变量，用于设置字体参数 */
    char buf0[100];
    char buf1[100];
```

```
    /* 设置字体属性 */
    tFont.FontCode = FC_ST_16;       /* 字体选择宋体 16 点阵，高 16x 宽 15) */
    tFont.FrontColor = CL_RED;       /* 字体颜色设置为红色 */
    tFont.BackColor = CL_MASK;       /* 文字背景颜色，透明 */
    tFont.Space = 0;                 /* 字符水平间距，单位 = 像素 */

    /* 第 1 步：JPEG 初始化 ############################################*/
    JPEG_Handle.Instance = JPEG;
    HAL_JPEG_Init(&JPEG_Handle);

    iTimeStart = bsp_GetRunTime();
    JPEG_Decode_DMA(&JPEG_Handle, (uint32_t)_ac1, sizeof(_ac1) , SDRAM_APP_BUF);

    /* 第 2 步：等待 JPEG 解码完成 ############################################*/
    while(Jpeg_HWDecodingEnd == 0){}
   iTimeEnd = bsp_GetRunTime();
   sprintf(buf0, "STM32H7 硬件 JPEG 解码 480*272 图片时间=%dms", iTimeEnd- iTimeStart);

    /* 第 3 步：获取 JPEG 图片信息#############################################*/
    HAL_JPEG_GetInfo(&JPEG_Handle, &JPEG_Info);

    /* 第 4 步：绘制 JPEG 图片到显示屏#########################################*/
   iTimeStart = bsp_GetRunTime();
    DMA2D_Copy_YCbCr_To_RGB((uint32_t *)SDRAM_APP_BUF,  /* JEPG 解码后的数据 */
                            (uint32_t *)SDRAM_LCD_BUF1, /* 这里是显存地址 */
                            0 ,
                            0,
                            JPEG_Info.ImageWidth,
                            JPEG_Info.ImageHeight,
                               LTDC_PIXEL_FORMAT_RGB565,
                            JPEG_Info.ChromaSubsampling);


   iTimeEnd = bsp_GetRunTime();
    LCD_DispStr(0, 0, buf0, &tFont);

   sprintf(buf1, "STM32H7 硬件 JPEG 显示 480*272 图片时间=%dms", iTimeEnd- iTimeStart);
    LCD_DispStr(0, 18, buf1, &tFont);

    bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();

        /* 判断软件定时器 0 是否超时 */
        if(bsp_CheckTimer(0))
        {
            /* 每隔 200ms 进来一次 */
            bsp_LedToggle(2);
        }
    }
}
```

56.8 实验例程说明（IAR）

配套例子：

V7-035_DMA2D 功能测试（显示色块，位图，Alpha 混合和图片混合等）

实验目的：

1. 学习 DMA2D 显示色块，位图，Alpha 混合和图片混合等。

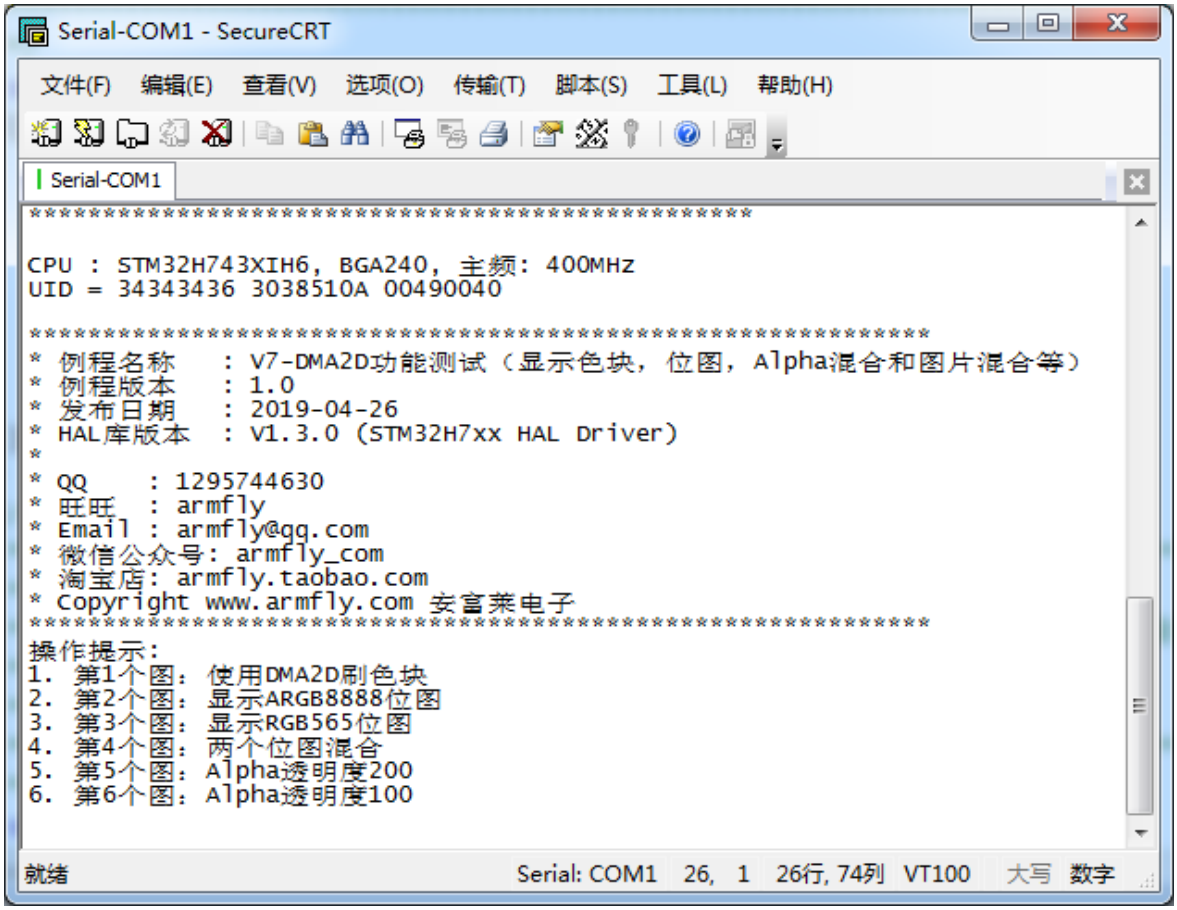
实验内容：

1. 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。
2. 第 1 个图：使用 DMA2D 刷色块。
3. 第 2 个图：显示 ARGB8888 位图。
4. 第 3 个图：显示 RGB565 位图。
5. 第 4 个图：两个位图混合。
6. 第 5 个图：Alpha 透明度 200 的位图显示。
7. 第 6 个图：Alpha 透明度 100 的位图显示。

LCD 界面显示效果如下：

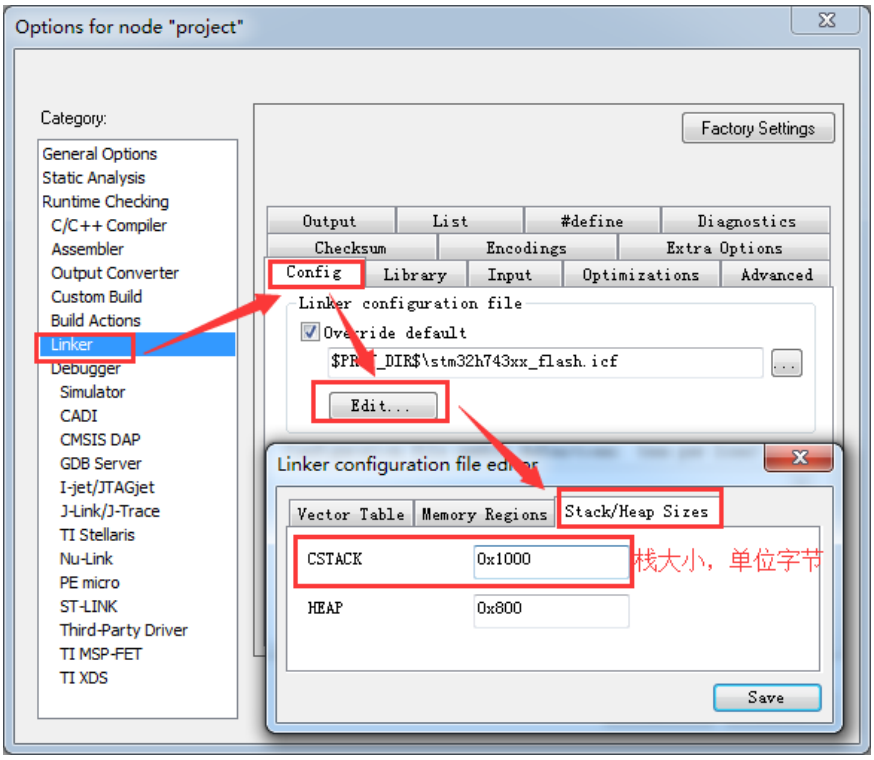
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

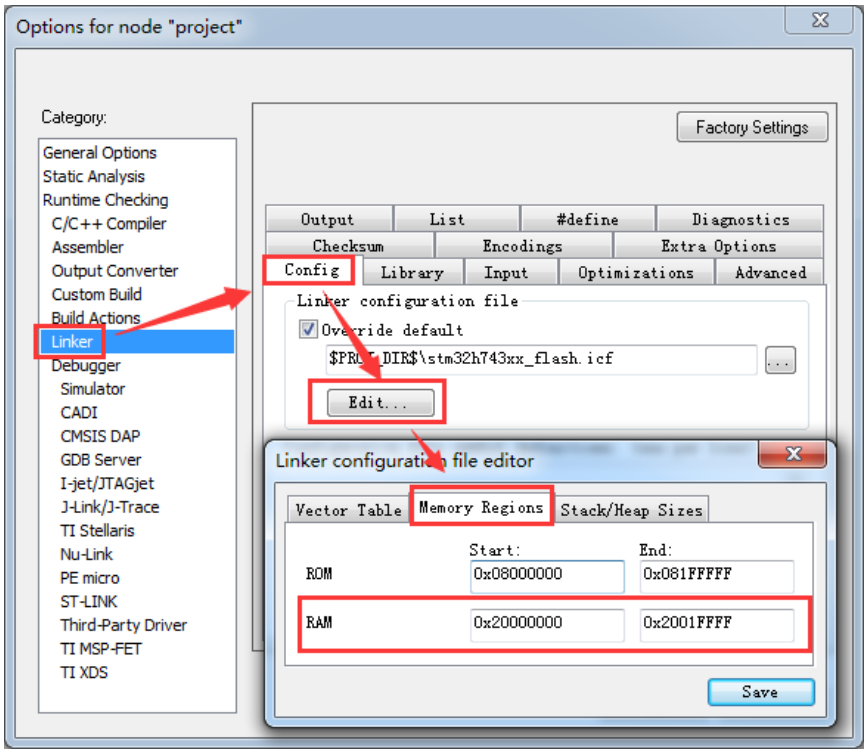


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
```

```
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */

    bsp_InitI2C();      /* 初始化 I2C 总线 */
    TOUCH_InitHard();   /* 初始化触摸芯片，LCD 面板型号的检查也在此函数，所以要在函数 LCD_InitHard 前调用 */
    LCD_InitHard();     /* 初始化 LCD */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SDRAM。由于 SDRAM 要用于 LCD 的显存，方便起见，直接将其配置为 WT 模式。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

```
    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SDRAM 的 MPU 属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0xC0000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_32MB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。

- LCD 第 1 个图：使用 DMA2D 刷色块。
- LCD 第 2 个图：显示 ARGB8888 位图。
- LCD 第 3 个图：显示 RGB565 位图。
- LCD 第 4 个图：两个位图混合。
- LCD 第 5 个图：Alpha 透明度 200 的位图显示。
- LCD 第 6 个图：Alpha 透明度 100 的位图显示。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint16_t ucBright;      /* 背光亮度(0-255) */
    FONT_T tFont;           /* 定义一个字体结构体变量，用于设置字体参数 */


    /* 设置字体参数 */
    {
        tFont.FontCode = FC_ST_16;       /* 字体代码 16 点阵 */
        tFont.FrontColor = CL_WHITE;     /* 字体颜色 */
        tFont.BackColor = CL_BLUE;       /* 文字背景颜色 */
        tFont.Space = 0;                 /* 文字间距，单位 = 像素 */
    }

    bsp_Init();         /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    /* 延迟 200ms 再点亮背光，避免瞬间高亮 */
    bsp_DelayMS(200);

    LCD_ClrScr(CL_BLUE);

    /* 界面整体显示完毕后，再打开背光，设置为缺省亮度 */
    bsp_DelayMS(100);
    ucBright = BRIGHT_DEFAULT;
    LCD_SetBackLight(ucBright);

    /* 第 1 个图：使用 DMA2D 刷色块 ###########################################################*/
    LCD_DispStr(24, 2, "DMA2D 刷色块", &tFont);
    _DMA2D_Fill((void *)(SDRAM_LCD_BUF1 + g_LcdWidth*20*2 + 24*2), /* 显示起始地址(24, 20) */
                128,                                               /* 色块长 */
                  128,                                               /* 色块高 */
                  g_LcdWidth-128,                                    /* 色块行偏移 */
                  CL_RED,                                            /* 色块颜色 */
                  LTDC_PIXEL_FORMAT_RGB565);                         /* 色块颜色格式 */

    /* 第 2 个图：显示 ARGB8888 位图 ###########################################################*/
    LCD_DispStr(176, 2, "刷 ARGB8888 位图", &tFont);
    _DMA2D_DrawAlphaBitmap((void *)(SDRAM_LCD_BUF1 + g_LcdWidth*20*2 + 176*2), /* 显示起始地址(176, 20) */
```

```
                        (void *)_aclufei,                                   /* 位图地址 */
                        128,                                                /* 位图长 */
                        128,                                                /* 位图高 */
                        0,                                                  /* 位图行偏移 */
                        g_LcdWidth-128,                                     /* 目标区行偏移 */
                        LTDC_PIXEL_FORMAT_RGB565);                          /* 目标区颜色格式 */

    /* 第 3 个图：显示 RGB565 位图 ########################################################################*/
    LCD_DispStr(328, 2, "刷 RGB565 位图", &tFont);
    _DMA2D_Copy((uint32_t *)_acmickey,                                      /* 位图地址 */
                (uint32_t *)(SDRAM_LCD_BUF1 + g_LcdWidth*20*2 + 328*2),     /* 显示起始地址(328, 20) */
                128,                                                        /* 位图长 */
                128,                                                        /* 位图高 */
                0,                                                          /* 位图行偏移 */
                g_LcdWidth-128,                                             /* 目标区行偏移 */
                 LTDC_PIXEL_FORMAT_RGB565);                                 /* 目标区颜色格式 */


    /* 第 4 个图：两个位图混合 ########################################################################*/
    LCD_DispStr(24, 150, "两个位图混合", &tFont);
    _DMA2D_AlphaBlendingBulk((uint32_t *)_aclufei,                          /* 前景层位图地址 */
                             0,                                             /* 前景层行偏移  */
                            (uint32_t *)_acsuolong,                         /* 背景层位图地址  */
                             0,                                             /* 背景层行偏移  */
                        (uint32_t *)(SDRAM_LCD_BUF1 +  g_LcdWidth*168*2 + 24*2), /* 显示起始地址(24, 168) */
                             g_LcdWidth-128,                                /* 目标区行偏移 */
                             128,                                           /* 目标区长 */
                             128);                                          /* 目标区高 */

    /* 第 5 个图：Alpha 透明度 200 的位图显示 ########################################################################*/
    LCD_DispStr(176, 150, "Alpha 透明度 200", &tFont);
    _DMA2D_MixColorsBulk((uint32_t *)_achuoying,                            /* 位图地址 */
                         0,                                                 /* 位图行偏移 */
                        (uint32_t *)(SDRAM_LCD_BUF1 + g_LcdWidth*168*2 + 176*2), /* 显示起始地址(176, 168) */
                         g_LcdWidth-128,                                    /* 目标区行偏移 */
                         128,                                               /* 目标区长 */
                         128,                                               /* 目标区高 */
                         200);                                              /* 位图显示透明度200 */

    /* 第 6 个图：Alpha 透明度 100 的位图显示 ########################################################################*/
    LCD_DispStr(328, 150, "Alpha 透明度 100", &tFont);
    _DMA2D_MixColorsBulk((uint32_t *)_achuoying,                            /* 位图地址 */
                         0,                                                 /* 位图行偏移 */
                        (uint32_t *)(SDRAM_LCD_BUF1 + g_LcdWidth*168*2 + 328*2), /* 显示起始地址(328, 168) */
                         g_LcdWidth-128,                                    /* 目标区行偏移 */
                         128,                                               /* 目标区长 */
                         128,                                               /* 目标区高 */
                         100);                                              /* 位图显示透明度200 */


    bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */

    while (1)
    {
      bsp_Idle();

        /* 判断软件定时器 0 是否超时 */
        if(bsp_CheckTimer(0))
```

```
        {
            /* 每隔 200ms 进来一次 */
            bsp_LedToggle(2);
        }
    }
}
```

56.9 总结

本章节涉及到的知识点非常重要，望初学者务必熟练掌握，因为后面章节用到 LCD 加速的地方都要用到 DMA2D。

第57章 STM32H7 硬件 JPEG 编解码基础知识和 HAL 库 API

本章节为大家讲解硬件 JPEG，实际项目用到图像显示的地方比较多，有了硬件 JPEG 可以大大加速 JPEG 图片显示速度。

57.1 初学者重要提示
57.2 硬件 JPEG 基础知识
57.3 硬件 JPEG 的 HAL 库用法
57.4 源文件 stm32h7xx_hal_jpeg.c
57.5 总结

57.1 初学者重要提示

- 由于硬件 JPEG 解码后输出的图像格式是 YCbCr，所以本章对 YCbCr 进行了重点介绍。
- 测试 STM32H7 硬件 JPEG 解码 800*480 图片性能，全部通过 SDRAM 缓存数据，解码 10ms，显示 9ms：http://www.armbbs.cn/forum.php?mod=viewthread&tid=93598 。
- JPEG 涉及到的知识点还是比较多的，如果想深入了解 JPEG 的话，可以看本章 2.6 小节给的参考资料。
- 本章 JPEG 相关概念的介绍参考了 wiki 百科和百度百科。

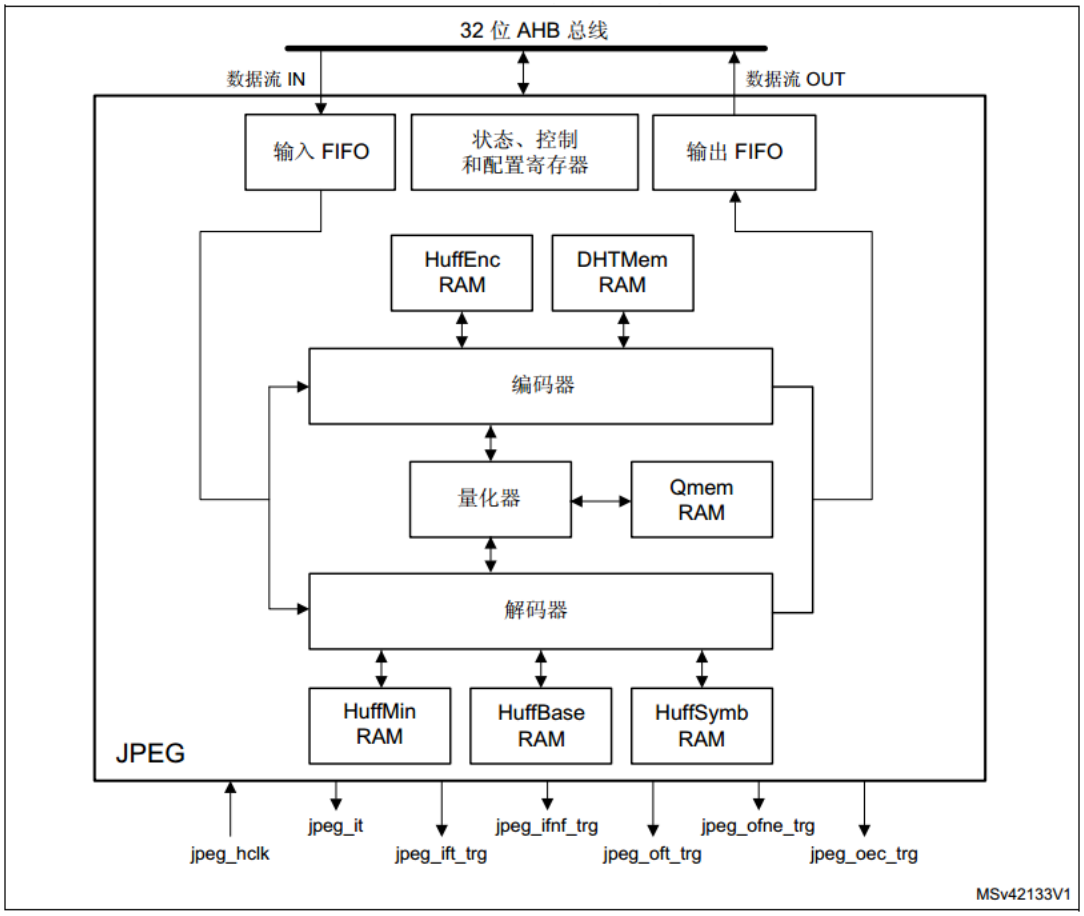
57.2 硬件 JPEG 基础知识

对于 STM32H7 的硬件 JPEG 了解到以下几点即可：

- 支持 JPEG 解码和编码。
- 对每个像素数据进行编解码只需一个时钟周期。
- 支持 RGB、 YCbCr、YCMK 和 BW（灰度）图像色彩模型。
- 编解码时每图像分量 8 位深度。

57.2.1 JPEG 硬件框图

认识一个外设，最好的方式就是看它的框图，方便我们快速地了解 JPEG 的基本功能，然后再看手册了解细节。框图如下所示：



通过这个框图，我们可以得到如下信息：

- JPEG 硬件外设支持编码和解码

  并且对于输入数据和输出数据都有 FIFO 支持。

- jpeg_hclk

  为 JPEG 内核和寄存器提供时钟。

- jpeg_it

  JPEG 全局中断输出。

- jpeg_ift_trg

  JPEG 输入 FIFO 阈值信号，可触发 MDMA。

- jpeg_ifnf_trg

  JPEG 输入 FIFO 未满信号，可触发 MDMA。

- jpeg_oft_trg

  JPEG 输出 FIFO 阀值信号，可触发 MDMA。

- jpeg_ofne_trg

  JPEG 输出 FIFO 非空信号，可触发 MDMA。

- jpeg_oec_trg

  JPEG 转换结束信号，可触发 MDMA。

57.2.2 YCbCr 颜色格式

（注，硬件 JPEG 解码后输出的图像格式是 YCbCr，所以有必要了解下）

正如几何上用坐标空间来描述坐标集，而色彩空间用数学方式来描述颜色集。常见的 3 种色彩模型是 RGB，CMYK 和 YUV。

YCbCr 是 YUV 经过缩放和修改的翻版，只是在表示方法上不同。其中 Y 是指亮度分量，Cb 指蓝色色度分量，而 Cr 指红色色度分量。人眼对视频的 Y 分量更敏感，因此通过对色度分量进行子采样来减少色度分量后，人眼察觉不到的图像质量的变化。

在 YUV 家族中，YCbCr 是在计算机系统中应用最多的成员，其应用领域广泛，JPEG、MPEG 均采用此格式。一般人们所讲的 YUV 大多是指 YCbCr。

57.2.3 YCbCr 采样格式

YCbCr 有许多取样格式，如 YCbCr 4:4:4，YCbCr 4:2:2，YCbCr 4:1:1 和 YCbCr 4:2:0。

- 4:2:0

  表示每 4 个像素有 4 个亮度分量，2 个色度分量（YYYYCbCr），仅采样奇数扫描线，是便携式视频设备（MPEG-4）以及电视会议（H.263）最常用格式。

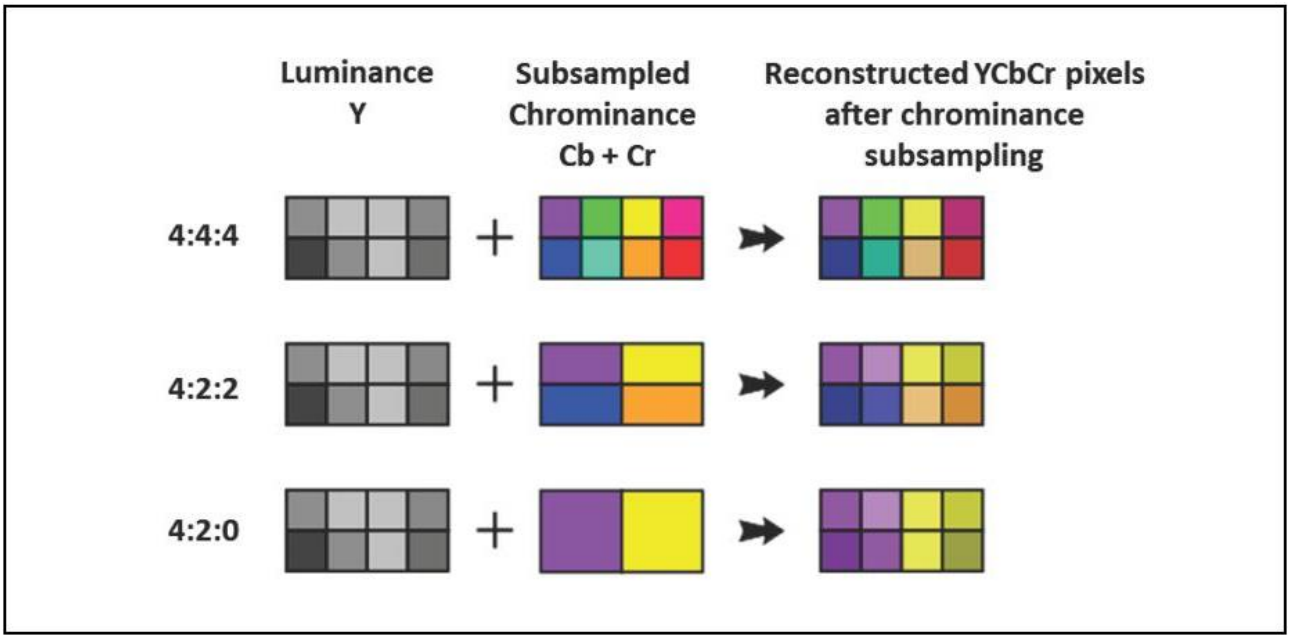
- 4:2:2

  表示每 4 个像素有 4 个亮度分量，4 个色度分量（YYYYCbCrCbCr），是 DVD、数字电视、HDTV 以及其它消费类视频设备的最常用格式。

- 4:4:4

  表示全像素点阵(YYYYCbCrCbCrCbCrCbCr），用于高质量视频应用、演播室以及专业视频产品。

具体的采样方式如下图所示，以 8 个像素为一个单元进行采样：



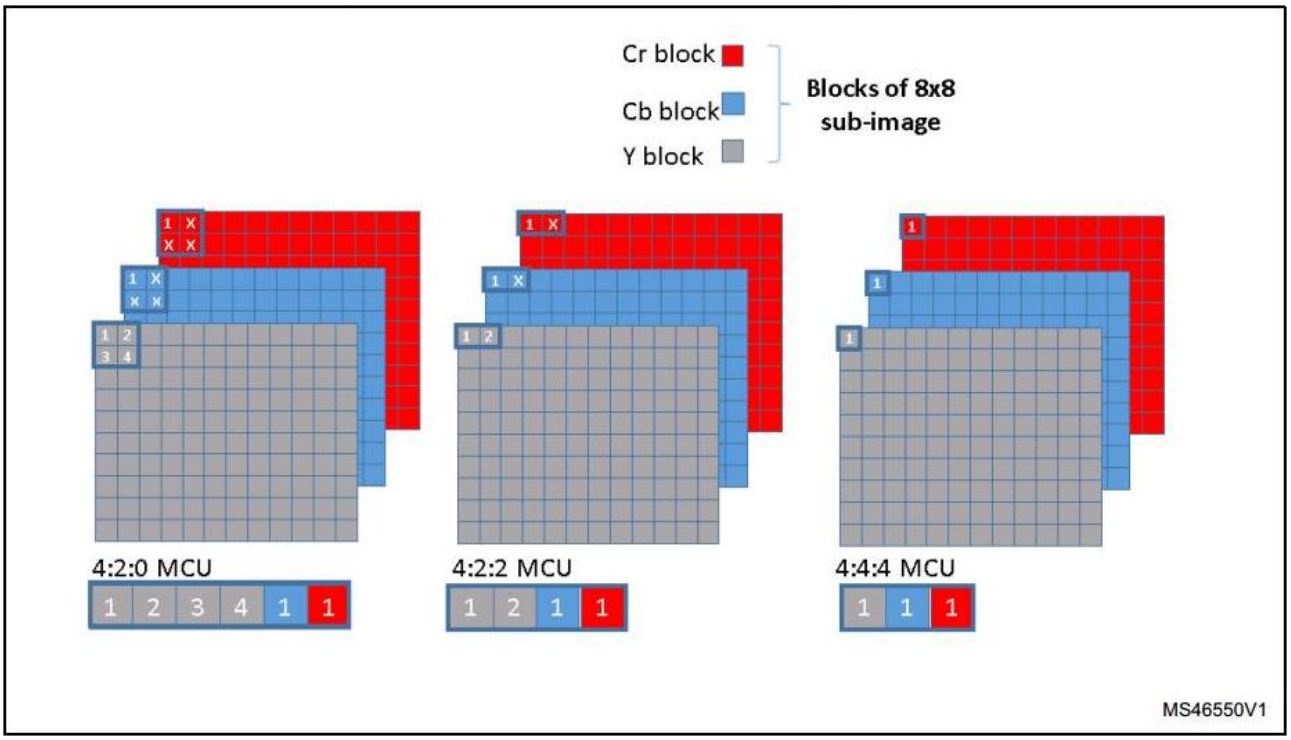
由上面的截图可以了解到：

4:4:4 表示 Y 通道，Cb+Cr 通道全部采样。

4:2:2 表示 Y 通道全部采样，而 Cb+Cr 通道两个像素为一组，统一采用第 1 个颜色值。

4:2:0 表示 Y 通道全部采样，而 Cb+Cr 通道四个像素为一组，统一采用第 1 个颜色值。

下面是整体效果，方便大家更好的理解：

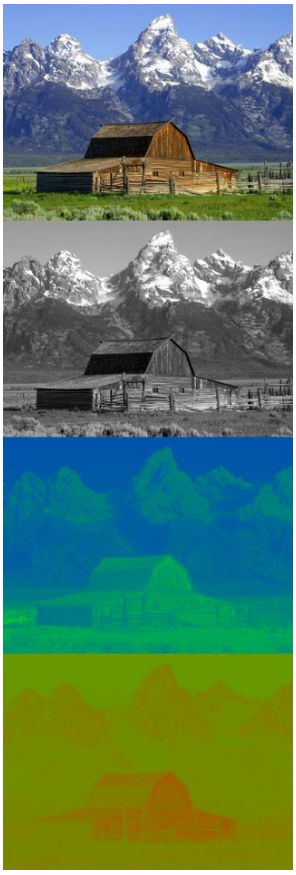


57.2.4 YCbCr 的优势

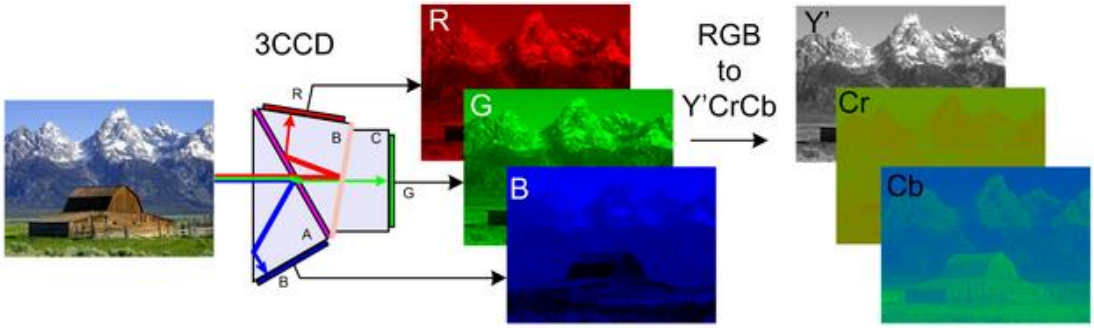
RGB 信号作为存储和传输的效率不高，因为它们具有大量冗余信息。而使用 YCbCr 可以丢弃一些信息以减少带宽，因为人的肉眼对视频的 Y 分量更敏感，因此通过对色度分量进行子采样来减少色度分量后，肉眼察觉不到的图像质量的变化。了解这种人为缺点，NTSC 和 PAL 等标准大大降低了色度通道的带宽。

57.2.5 YCbCr 和 RGB 互转

为了方便大家更好的了解 YCbCr 和 RGB 图像的实际效果，特此搜集整理了两个截图（来自 WIKI 百科）。下面是图像转 YCBCR 的效果：四个图，从上到下依次是原始图像，Y 通道，Cb 通道和 Cr 通道。



下面是一幅图像分别以 R，G，B 通道和 Y，CB，CR 通道的方式展示：



57.2.6 JPEG 编解码知识

JPEG 涉及到的知识点比较多，这里有之前整理的 20 多个专题知识点，大家有兴趣可以了解下（不了解也没有关系，不影响使用硬件 JPEG 外设）：

http://www.armbbs.cn/forum.php?mod=forumdisplay&fid=12&filter=typeid&typeid=71 。

另外还 ST 整理的 JPEG 应用笔记，含中文版：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=93516 。

57.3 硬件 JPEG 的 HAL 库用法

JPEG 的 HAL 库用法其实就是几个结构体变量成员的配置和使用，然后配置时钟，并根据需要配置 NVIC、中断和 MDMA。下面我们逐一展开为大家做个说明。

57.3.1 JPEG 寄存器结构体 JPEG_TypeDef

JPEG 相关的寄存器是通过 HAL 库中的结构体 JPEG_TypeDef 定义的，在 stm32h743xx.h 中可以找到它们的具体定义：

```
typedef struct
{
  __IO uint32_t CONFR0;
  __IO uint32_t CONFR1;
  __IO uint32_t CONFR2;
  __IO uint32_t CONFR3;
  __IO uint32_t CONFR4;
  __IO uint32_t CONFR5;
  __IO uint32_t CONFR6;
  __IO uint32_t CONFR7;
  uint32_t  Reserved20[4];
  __IO uint32_t CR;
  __IO uint32_t SR;
  __IO uint32_t CFR;
  uint32_t  Reserved3c;
  __IO uint32_t DIR;
  __IO uint32_t DOR;
  uint32_t  Reserved48[2];
  __IO uint32_t QMEM0[16];
  __IO uint32_t QMEM1[16];
  __IO uint32_t QMEM2[16];
  __IO uint32_t QMEM3[16];
  __IO uint32_t HUFFMIN[16];
  __IO uint32_t HUFFBASE[32];
  __IO uint32_t HUFFSYMB[84];
  __IO uint32_t DHTMEM[103];
  uint32_t  Reserved4FC;
  __IO uint32_t HUFFENC_AC0[88];
  __IO uint32_t HUFFENC_AC1[88];
  __IO uint32_t HUFFENC_DC0[8];
```

```
  __IO uint32_t HUFFENC_DC1[8];
} JPEG_TypeDef;
```

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

下面我们再看 JPEG 的定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE           ((uint32_t)0x40000000)
#define D1_AHB1PERIPH_BASE    (PERIPH_BASE + 0x12000000)
#define JPEG                  ((JPEG_TypeDef *) JPGDEC_BASE)
#define JPGDEC_BASE           (D1_AHB1PERIPH_BASE + 0x3000) <----- 展开这个宏，(JPEG_TypeDef *) 0x52003000
```

我们访问 JPEG 的 CR 寄存器可以采用这种形式：JPEG->CR = 0。

57.3.2 JPEG 的编解码参数结构体 JPEG_ConfTypeDef

此结构体用于 JPEG 的编解码参数，具体定义如下：

```
typedef struct
{
  uint8_t  ColorSpace;
  uint8_t  ChromaSubsampling;
  uint32_t ImageHeight;
  uint32_t ImageWidth;
  uint8_t  ImageQuality;
}JPEG_ConfTypeDef;
```

下面将这几个参数逐一为大家做个说明：

- uint8_t ColorSpace

此参数用于设置输出数据流中的量化表，具体支持的参数如下：

```
#define JPEG_GRAYSCALE_COLORSPACE     ((uint32_t)0x00000000U)    /* 灰度（1 个量化表）*/
#define JPEG_YCBCR_COLORSPACE         JPEG_CONFR1_COLORSPACE_0   /* YUV（2 个量化表） */
#define JPEG_CMYK_COLORSPACE          JPEG_CONFR1_COLORSPACE     /* CMYK（4 个量化表）*/
```

- uint8_t ChromaSubsampling

此参数用于色度子采样，具体支持的参数如下：

```
#define JPEG_444_SUBSAMPLING     ((uint32_t)0x00000000U)   /* 4:4:4 */
#define JPEG_420_SUBSAMPLING     ((uint32_t)0x00000001U)   /* 4:2:0 */
#define JPEG_422_SUBSAMPLING     ((uint32_t)0x00000002U)   /* 4:2:2 */
```

- uint32_t ImageHeight

此参数用于图像高度。

- uint32_t ImageWidth

此参数用于图像宽度。

- uint8_t ImageQuality

此参数用于图像质量，参数范围 1 – 100，1 最差，100 最好。

57.3.3 JPEG 结构体句柄 JPEG_HandleTypeDef

HAL 库在 JPEG_TypeDef，JPEG_ConfTypeDef 的基础上封装了一个结构体 JPEG_HandleTypeDef，定义如下：

```
typedef struct
{
  JPEG_TypeDef             *Instance;
  JPEG_ConfTypeDef         Conf;
  uint8_t                  *pJpegInBuffPtr;
  uint8_t                  *pJpegOutBuffPtr;
  __IO uint32_t            JpegInCount;
  __IO uint32_t            JpegOutCount;
  uint32_t                 InDataLength;
  uint32_t                 OutDataLength;
  MDMA_HandleTypeDef        *hdmain;
  MDMA_HandleTypeDef        *hdmaout;
  uint8_t                  CustomQuanTable;
  uint8_t                  *QuantTable0;
  uint8_t                  *QuantTable1;
  uint8_t                  *QuantTable2;
  uint8_t                  *QuantTable3;
  HAL_LockTypeDef          Lock;
  __IO  HAL_JPEG_STATETypeDef State;
  __IO  uint32_t           ErrorCode;
  __IO uint32_t Context;
}JPEG_HandleTypeDef;
```

下面将这几个参数逐一做个说明。

- **JPEG_TypeDef *Instance**

  这个参数是寄存器的例化，方便操作寄存器，详见本章 3.1 小节。

- **JPEG_ConfTypeDef Conf**

  这个参数是用户接触较多的，用于 JPEG 的编解码参数，详见本章 3.2 小节。

- **uint8_t *pJpegInBuffPtr**

  JPEG 编解码输入缓冲地址

- **uint8_t *pJpegOutBuffPtr**

  JPEG 编解码输出缓冲地址

- **__IO uint32_t JpegInCount**

  JPEG 内部输入计数。

- **__IO uint32_t JpegOutCount**

  JPEG 内部输出计数。

- **uint32_t InDataLength**

  JPEG 输入缓冲区长度，单位字节

- **uint32_t OutDataLength**

JPEG 输出缓冲区长度，单位字节。

- **MDMA_HandleTypeDef *hdmain**
- **MDMA_HandleTypeDef *hdmaout**

MDMA 句柄结构体指针变量，用于关联 JPEG 句柄，方便调用。

- **uint8_t CustomQuanTable**

如果此参数设置为 1，将使用用户设置的量化表。

- **uint8_t *QuantTable0;**
- **uint8_t *QuantTable1;**
- **uint8_t *QuantTable2;**
- **uint8_t *QuantTable3;**

指定量化表地址。

- **HAL_LockTypeDef Lock**
- **__IO HAL_JPEG_STATETypeDef State**
- **__IO uint32_t ErrorCode**

这三个变量主要供函数内部使用。Lock 用于设置锁状态，State 用于设置 JPEG 状态，而 ErrorCode 用于配置代码错误。

- **__IO uint32_t Context**

JPEG 上下文。

57.3.4JPEG 初始化流程总结

使用方法由 HAL 库提供：

- 第 1 步：调用函数 HAL_JPEG_Init 进行初始化，但这个函数不需要初始化参数。

如果是 JPEG 编码，可以通过函数 HAL_JPEG_ConfigEncoding 设置 JPEG 图像的质量参数，质量越高，生成的 JPEG 文件越大、

- 第 2 步：调用编解码函数
  - 查询式编解码函数

HAL_JPEG_Encode

HAL_JPEG_Decode

  - 中断方式

HAL_JPEG_Encode_IT

HAL_JPEG_Decode_IT

- DMA 方式

HAL_JPEG_Encode_DMA

HAL_JPEG_Decode_DMA

◆ 第 4 步：如果用户之前的数据已经处理完毕，需要插入新数据，会调用函数 HAL_JPEG_GetDataCallback

（1）如果新的数据已经准备好，需要调用函数 HAL_JPEG_ConfigInputBuffer。如果新的数据没有准备好，需要等待插入新数据时，可以调用函数 HAL_JPEG_Pause（参数 XferSelection 被设置为 JPEG_PAUSE_RESUME_INPUT），待数据准备好后，可以调用 HAL_JPEG_ConfigInputBuffer 设置新的输入缓冲和大小，然后调用函数 HAL_JPEG_Resume 恢复 JPEG 编解码。

如果编解码的数据已经处理完毕，可以调用函数 HAL_JPEG_ConfigInputBuffer 设置 InDataLength 参数为 0（此函数是在回调函数 HAL_JPEG_GetDataCallback 里面被调用的）。

（2）函数 HAL_JPEG_ConfigInputBuffer/HAL_JPEG_Pause/HAL_JPEG_Resume 的工作机制允许应用程序以块为单位提供输入数据。如果新的数据块未准备好，可以调用函数 HAL_JPEG_Pause 暂停输入，待数据准备好后，可以调用 HAL_JPEG_ConfigInputBuffer 设置新的输入缓冲和大小，然后调用函数 HAL_JPEG_Resume 恢复 JPEG 编解码。

（3）新的数据块准备好后，可以在回调函数 HAL_JPEG_GetDataCallback 外面调用 HAL_JPEG_ConfigInputBuffer 和 HAL_JPEG_Resume，但是为了保持数据一致性问题，务必在回调函数 HAL_JPEG_GetDataCallback 里面调用 HAL_JPEG_Resume。

◆ 第 5 步：输出缓冲区填充了给定大小的数据后，会调用回调函数 HAL_JPEG_DataReadyCallback

（1）如果有数据空间存储新数据块，需要调用函数 HAL_JPEG_ConfigOutputBuffer 配置新存储位置。如果没有数据空间存储新数据块，需要等待有数据空间可用时，可以调用函数 HAL_JPEG_Pause（参数 XferSelection 被设置为 JPEG_PAUSE_RESUME_INPUT），待有数据空间可用时，可以调用 HAL_JPEG_ConfigOutputBuffe 设置新的输出缓冲，然后调用函数 HAL_JPEG_Resume 恢复 JPEG 编解码。

（2）函数 HAL_JPEG_ConfigOutputBuffe/HAL_JPEG_Pause/HAL_JPEG_Resume 的工作机制允许应用程序以块为单位接收数据。当接收到数据块时，应用程序可以暂停 JPEG 输出来处理这些数据，比如解码时 YCbCr 转 RGB 或者编码时数据存储。

（3）新的数据空间准备好后，可以在回调函数 HAL_JPEG_DataReadyCallback 外面调用 HAL_JPEG_ConfigOutputBuffer 和 HAL_JPEG_Resume，但是为了保持数据一致性问题，务必在回调函数 HAL_JPEG_DataReadyCallback 里面调用 HAL_JPEG_Resume。

- 第 6 步：其它相关函数
  - JPEG 解码时，如果解码成功，会调用回调函数 HAL_JPEG_InfoReadyCallback。
  - JPEG 编码操作结束后会调用回调函数 HAL_JPEG_EncodeCpltCallback。
  - JPEG 解码操作结束后，会调用回调函数 HAL_JPEG_DecodeCpltCallback。
  - 操作过程中出现错误，会调用回调函数 HAL_JPEG_ErrorCallback，用户可以调用函数 HAL_JPEG_GetError 获取错误类型。
  - HAL JPEG 默认使用的是 ISO/IEC 10918-1 规格量化表，如果要修改，可以调用函数 HAL_JPEG_SetUserQuantTables 实现。
  - 通过函数 HAL_JPEG_GetState 可以获取 JPEG 状态。

57.4 源文件 stm32h7xx_hal_jpeg.c

这里把我们把如下几个常用到的函数做个说明：

- HAL_JPEG_Init
- HAL_JPEG_GetInfo
- HAL_JPEG_Decode_DMA
- HAL_JPEG_ConfigInputBuffer
- HAL_JPEG_ConfigOutputBuffer

57.4.1 函数 HAL_JPEG_Init

函数原型：

```
HAL_StatusTypeDef HAL_JPEG_Init(JPEG_HandleTypeDef *hjpeg)
{
  uint32_t acLum_huffmanTableAddr = (uint32_t)(&JPEG_ACLUM_HuffTable);
  uint32_t dcLum_huffmanTableAddr = (uint32_t)(&JPEG_DCLUM_HuffTable);
  uint32_t acChrom_huffmanTableAddr = (uint32_t)(&JPEG_ACCHROM_HuffTable);
  uint32_t dcChrom_huffmanTableAddr = (uint32_t)(&JPEG_DCCHROM_HuffTable);

  /* 检测句柄是否有效 */
  if(hjpeg == NULL)
  {
    return HAL_ERROR;
  }

  if(hjpeg->State == HAL_JPEG_STATE_RESET)
  {
```

```
  hjpeg->Lock = HAL_UNLOCKED;

   /* 初始化 GPIO, NVIC 等 */
  HAL_JPEG_MspInit(hjpeg);
}

/* 设置 JPEG 状态 */
hjpeg->State = HAL_JPEG_STATE_BUSY;

/* 使能 JPEG  */
__HAL_JPEG_ENABLE(hjpeg);

/* 关闭 JPEG 编解码处理 */
hjpeg->Instance->CONFR0 &=  ~JPEG_CONFR0_START;

/* 关闭 JPEG 所有中断 */
__HAL_JPEG_DISABLE_IT(hjpeg,JPEG_INTERRUPT_MASK);

/* 清空输入输入 FIFO 缓冲 */
hjpeg->Instance->CR |= JPEG_CR_IFF;
hjpeg->Instance->CR |= JPEG_CR_OFF;

/* 清除所有标志 */
__HAL_JPEG_CLEAR_FLAG(hjpeg,JPEG_FLAG_ALL);

/* 初始化默认的量化表 */
hjpeg->QuantTable0 = (uint8_t *)((uint32_t)JPEG_LUM_QuantTable);
hjpeg->QuantTable1 = (uint8_t *)((uint32_t)JPEG_CHROM_QuantTable);
hjpeg->QuantTable2 = NULL;
hjpeg->QuantTable3 = NULL;

/* 初始化默认的霍夫曼表 */
if(JPEG_Set_HuffEnc_Mem(hjpeg, (JPEG_ACHuffTableTypeDef *)acLum_huffmanTableAddr, (JPEG_DCHuffTableTypeDef
                                *)dcLum_huffmanTableAddr, (JPEG_ACHuffTableTypeDef
                                *)acChrom_huffmanTableAddr, (JPEG_DCHuffTableTypeDef
                                *)dcChrom_huffmanTableAddr) != HAL_OK)
{
  hjpeg->ErrorCode = HAL_JPEG_ERROR_HUFF_TABLE;

  return HAL_ERROR;
}

/* 使能文件头处理 */
hjpeg->Instance->CONFR1 |= JPEG_CONFR1_HDR;

/* 复位 JPEG 输入输出计数 */
hjpeg->JpegInCount = 0;
hjpeg->JpegOutCount = 0;

/* 设置 JPEG 就绪 */
hjpeg->State = HAL_JPEG_STATE_READY;

/* 设置无错误 Reset the JPEG ErrorCode */
hjpeg->ErrorCode = HAL_JPEG_ERROR_NONE;

/* 清除上下文 */
hjpeg->Context = 0;
```

```
  /* 返回 HAL_OK */
  return HAL_OK;
}
```

函数描述：

此函数用于初始化 JPEG。

函数参数：

- 第 1 个参数是 JPEG_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数，结构体变量成员的详细介绍看本章 3.3 小节。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 函数 HAL_JPEG_MspInit 用于初始化 JPEG 的底层时钟、NVIC 等功能。需要用户自己在此函数里面实现具体的功能。由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
2. 如果形参 hjpeg 的结构体成员 State 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 JPEG_HandleTypeDef JpegHandle。

   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_JPEG_STATE_RESET = 0x00U。

   解决办法有三

   方法 1：用户自己初始 JPEG 底层。

   方法 2：定义 JPEG_HandleTypeDef JpegHandle 为全局变量。

   方法 3：下面的方法

```
if(HAL_JPEG_DeInit(&JpegHandle) != HAL_OK)
{
    Error_Handler();
}
if(HAL_JPEG_Init(&Dma2dHandle) != HAL_OK)
{
    Error_Handler();
}
```

使用举例：

```
JPEG_HandleTypeDef    JPEG_Handle;
JPEG_Handle.Instance = JPEG;
HAL_JPEG_Init(&JPEG_Handle);
```

57.4.2 函数 HAL_JPEG_GetInfo

函数原型：

```
HAL_StatusTypeDef HAL_JPEG_GetInfo(JPEG_HandleTypeDef *hjpeg, JPEG_ConfTypeDef *pInfo)
{
  uint32_t yblockNb, cBblockNb, cRblockNb;
```

```
  /* 检测句柄是否有效 */
  if((hjpeg == NULL) || (pInfo == NULL))
  {
    return HAL_ERROR;
  }

  /* 读取配置参数 */
  if((hjpeg->Instance->CONFR1 & JPEG_CONFR1_NF) == JPEG_CONFR1_NF_1)
  {
    pInfo->ColorSpace = JPEG_YCBCR_COLORSPACE;
  }
  else if((hjpeg->Instance->CONFR1 & JPEG_CONFR1_NF) == 0)
  {
    pInfo->ColorSpace = JPEG_GRAYSCALE_COLORSPACE;
  }
  else if((hjpeg->Instance->CONFR1 & JPEG_CONFR1_NF) == JPEG_CONFR1_NF)
  {
    pInfo->ColorSpace = JPEG_CMYK_COLORSPACE;
  }

  pInfo->ImageHeight = (hjpeg->Instance->CONFR1 & 0xFFFF0000U) >> 16;
  pInfo->ImageWidth  = (hjpeg->Instance->CONFR3 & 0xFFFF0000U) >> 16;

  if((pInfo->ColorSpace == JPEG_YCBCR_COLORSPACE) || (pInfo->ColorSpace == JPEG_CMYK_COLORSPACE))
  {
    yblockNb  = (hjpeg->Instance->CONFR4 & JPEG_CONFR4_NB) >> 4;
    cBblockNb = (hjpeg->Instance->CONFR5 & JPEG_CONFR5_NB) >> 4;
    cRblockNb = (hjpeg->Instance->CONFR6 & JPEG_CONFR6_NB) >> 4;

    if((yblockNb == 1) && (cBblockNb == 0) && (cRblockNb == 0))
    {
      pInfo->ChromaSubsampling = JPEG_422_SUBSAMPLING; /*16x8 block*/
    }
    else if((yblockNb == 0) && (cBblockNb == 0) && (cRblockNb == 0))
    {
      pInfo->ChromaSubsampling = JPEG_444_SUBSAMPLING;
    }
    else if((yblockNb == 3) && (cBblockNb == 0) && (cRblockNb == 0))
    {
      pInfo->ChromaSubsampling = JPEG_420_SUBSAMPLING;
    }
    else /* 默认是 4:4:4*/
    {
      pInfo->ChromaSubsampling = JPEG_444_SUBSAMPLING;
    }
  }
  else
  {
    pInfo->ChromaSubsampling = JPEG_444_SUBSAMPLING;
  }

  pInfo->ImageQuality = JPEG_GetQuality(hjpeg);

  /* 返回 HAL_OK */
  return HAL_OK;
}
```

函数描述：

此函数主要用于解码 JPEG 时获取相关图像信息，比如图像质量，图像长宽等。

函数参数：

- 第 1 个参数是 JPEG_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数，结构体变量成员的详细介绍看本章 3.3 小节。
- 第 2 个参数是 JPEG_ConfTypeDef 类型结构体指针变量，用于获取 JPEG 的配置信息，结构体变量成员的详细介绍看本章 3.2 小
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
JPEG_HandleTypeDef    JPEG_Handle;
JPEG_ConfTypeDef      JPEG_Info;

HAL_JPEG_GetInfo(&JPEG_Handle, &JPEG_Info);
```

57.4.3 函数 HAL_JPEG_Decode_DMA

函数原型：

```
HAL_StatusTypeDef  HAL_JPEG_Decode_DMA(JPEG_HandleTypeDef *hjpeg ,uint8_t *pDataIn ,uint32_t
                                InDataLength ,uint8_t *pDataOutMCU ,uint32_t OutDataLength)
{
  /* 检测参数 */
  assert_param((InDataLength >= 4));
  assert_param((OutDataLength >= 4));

  /* 检测参数 */
  if((hjpeg == NULL) || (pDataIn == NULL) || (pDataOutMCU == NULL))
  {
    return HAL_ERROR;
  }

  /* 上锁 */
  __HAL_LOCK(hjpeg);

  if(hjpeg->State == HAL_JPEG_STATE_READY)
  {
    /* 设置 JPEG 忙 */
    hjpeg->State = HAL_JPEG_STATE_BUSY_DECODING;

    /* 设置 JPEG 上下文，工作在 DMA 界面状态 */
    hjpeg->Context &= ~(JPEG_CONTEXT_OPERATION_MASK | JPEG_CONTEXT_METHOD_MASK);
    hjpeg->Context |= (JPEG_CONTEXT_DECODE | JPEG_CONTEXT_DMA);

    /* 设置输入输出缓冲地址和大小 */
    hjpeg->pJpegInBuffPtr = pDataIn;
    hjpeg->pJpegOutBuffPtr = pDataOutMCU;
    hjpeg->InDataLength = InDataLength;
    hjpeg->OutDataLength = OutDataLength;

    /* 复位输入输出缓冲计数 */
```

```
    hjpeg->JpegInCount = 0;
    hjpeg->JpegOutCount = 0;

    /* 初始化解码处理 */
    JPEG_Init_Process(hjpeg);

    /* 启动 JPEG 解码处理，使用 DMA 方式 */
    JPEG_DMA_StartProcess(hjpeg);

  }
  else
  {
    /* 解锁 */
    __HAL_UNLOCK(hjpeg);

    return HAL_BUSY;
  }

  /* 返回 HAL_OK */
  return HAL_OK;
}
```

函数描述：

此函数用于启动 JPEG 的 DMA 方式解码。

函数参数：

- 第 1 个参数是 JPEG_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数，结构体变量成员的详细介绍看本章 3.3 小节。
- 第 2 个参数是输入数据缓冲地址。
- 第 3 个参数输入数据大小，单位字节。
- 第 4 个参数是输出缓冲地址。
- 第 5 个参数是输出缓冲大小，单位字节。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: JPEG_Decode_DMA
*    功能说明: JPEG 解码
*    形    参: hjpeg               JPEG_HandleTypeDef 句柄指针
*              FrameSourceAddress  数据地址
*              FrameSize           数据大小
*              DestAddress         目的数据地址
*    返 回 值: HAL_ERROR 表示配置失败，HAL_OK 表示配置成功
*              HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出
*********************************************************************************************************
*/
uint32_t JPEG_Decode_DMA(JPEG_HandleTypeDef *hjpeg, uint32_t FrameSourceAddress ,uint32_t FrameSize, uint32_t
DestAddress)
{
    JPEGSourceAddress =  FrameSourceAddress ;
    FrameBufferAddress = DestAddress;
```

```
    Input_frameIndex = 0;
    Input_frameSize = FrameSize;

    /* 设置标志，0 表示开始解码，1 表示解码完成 */
    Jpeg_HWDecodingEnd = 0;

    /* 启动 JPEG 解码 */
    HAL_JPEG_Decode_DMA(hjpeg ,(uint8_t *)JPEGSourceAddress ,CHUNK_SIZE_IN ,
                        (uint8_t *)FrameBufferAddress ,CHUNK_SIZE_OUT);

    return HAL_OK;
}
```

57.4.4函数 HAL_JPEG_ConfigInputBuffer

函数原型：

```
void HAL_JPEG_ConfigInputBuffer(JPEG_HandleTypeDef *hjpeg, uint8_t *pNewInputBuffer, uint32_t InDataLength)
{
  hjpeg->pJpegInBuffPtr =  pNewInputBuffer;
  hjpeg->InDataLength = InDataLength;
}
```

函数描述：

此函数用于配置编解码输入缓冲地址和数据大小。

函数参数：

- 第 1 个参数是 JPEG_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数，结构体变量成员的详细介绍看本章 3.3 小节。
- 第 2 个参数是输入缓冲地址。
- 第 3 个参数是输入缓冲大小，单位字节。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: HAL_JPEG_GetDataCallback
*    功能说明: JPEG 回调函数，用于从输入地址获取新数据继续解码
*    形    参: hjpeg          JPEG_HandleTypeDef 句柄指针
*              NbDecodedData  上一轮已经解码的数据大小，单位字节
*    返 回 值: 无
*********************************************************************************************************
*/
void HAL_JPEG_GetDataCallback(JPEG_HandleTypeDef *hjpeg, uint32_t NbDecodedData)
{
    uint32_t inDataLength;

    /* 更新已经解码的数据大小 */
    Input_frameIndex += NbDecodedData;

    /* 如果当前已经解码的数据小于总文件大小，继续解码 */
```

```
    if( Input_frameIndex < Input_frameSize)
    {
        /* 更新解码数据位置 */
        JPEGSourceAddress = JPEGSourceAddress + NbDecodedData;

        /* 更新下一轮要解码的数据大小 */
        if((Input_frameSize - Input_frameIndex) >= CHUNK_SIZE_IN)
        {
            inDataLength = CHUNK_SIZE_IN;
        }
        else
        {
            inDataLength = Input_frameSize - Input_frameIndex;
        }
    }
    else
    {
        inDataLength = 0;
    }

    /* 更新输入缓冲 */
    HAL_JPEG_ConfigInputBuffer(hjpeg,(uint8_t *)JPEGSourceAddress, inDataLength);
}
```

57.4.5 函数 HAL_JPEG_ConfigOutputBuffer

函数原型：

```
void HAL_JPEG_ConfigOutputBuffer(JPEG_HandleTypeDef *hjpeg, uint8_t *pNewOutputBuffer, uint32_t OutDataLength)
{
  hjpeg->pJpegOutBuffPtr = pNewOutputBuffer;
  hjpeg->OutDataLength = OutDataLength;
}
```

函数描述：

此函数用于配置编解码输出缓冲地址和数据大小。

函数参数：

- 第 1 个参数是 JPEG_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数，结构体变量成员的详细介绍看本章 3.3 小节。
- 第 2 个参数是输出缓冲地址。
- 第 3 个参数是输出缓冲大小，单位字节。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: HAL_JPEG_DataReadyCallback
*    功能说明: JPEG 回调函数，用于输出缓冲地址更新
*    形    参: hjpeg         JPEG_HandleTypeDef 句柄指针
*              pDataOut      输出数据缓冲
```

```
*             OutDataLength 输出数据大小，单位字节
*    返 回 值: 无
*********************************************************************************************************
*/
void HAL_JPEG_DataReadyCallback (JPEG_HandleTypeDef *hjpeg, uint8_t *pDataOut, uint32_t OutDataLength)
{
    /* 更新 JPEG 输出地址 */
    FrameBufferAddress += OutDataLength;

    HAL_JPEG_ConfigOutputBuffer(hjpeg, (uint8_t *)FrameBufferAddress, CHUNK_SIZE_OUT);
}
```

57.5 总结

本章节就为大家讲解这么多，JPEG 功能用到的地方还是比较多的，建议熟练使用。

第58章 STM32H7 的硬件 JPEG 应用之图片解码显示

本章节为大家讲解硬件 JPEG 的图片解码功能，实际项目用到的地方比较多，一方面通过硬件 JPEG 可以加速图片解码，另一方面是 JPEG 图片比较小。

58.1 初学者重要提示
58.2 硬件 JPEG 驱动设计
58.3 硬件 JPEG 驱动移植和使用
58.4 实验例程设计框架
58.5 实验例程说明（MDK）
58.6 实验例程说明（IAR）
58.7 总结

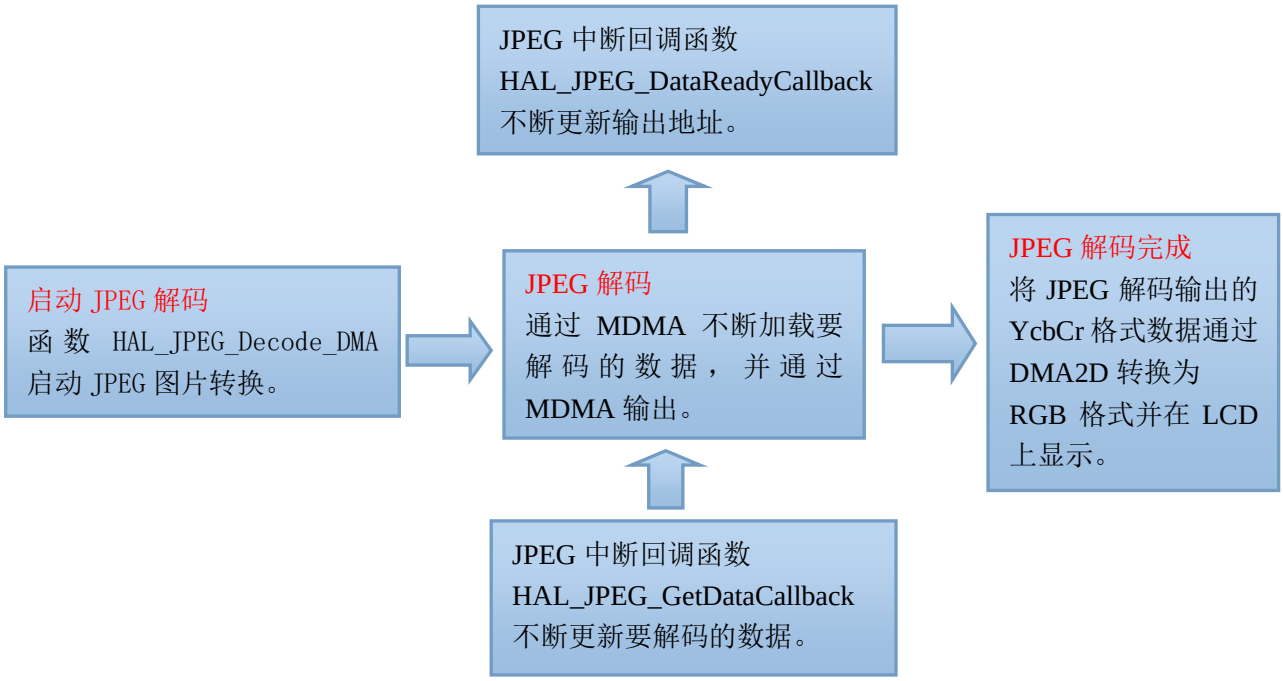
58.1 初学者重要提示

- 学习本章节前，务必优先学习第 57 章，需要对 JPEG 的基础知识有个认识。
- 测试 STM32H7 硬件 JPEG 解码 800*480 图片性能，全部通过 SDRAM 缓存数据，解码 10ms，显示 9ms：http://www.armbbs.cn/forum.php?mod=viewthread&tid=93598 。
- 由于 JPEG 图片较小，本章配套例程是将其转换为 C 数组后，直接添加到工程里面。
  文件转 C 数组小软件下载：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=93662 。
  使用方法看 emWin 教程第 17 章的 3.3 小节：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=19834 。

58.2 硬件 JPEG 驱动设计

58.2.1 硬件 JPEG 驱动设计框架

为了方便大家理解 JPEG 驱动的实现，先看下面 JPEG 的驱动设计框架：



下面为大家讲解具体的驱动实现。

58.2.2第 1 步：JPEG 配置

JPEG 的配置比较简单，仅需如下代码即可：

```
JPEG_HandleTypeDef    JPEG_Handle;
JPEG_Handle.Instance = JPEG;
HAL_JPEG_Init(&JPEG_Handle);
```

58.2.3第 2 步：启动 JPEG 解码

启动 JPEG 的代码如下：

```
1.  #define CHUNK_SIZE_IN  ((uint32_t)(64 * 1024))    /* 输入数据大小，单位字节 */
2.  #define CHUNK_SIZE_OUT ((uint32_t)(64 * 1024))    /* 输出数据大小，单位字节 */
3.
4.  /*
5.  *********************************************************************************************************
6.  *    函 数 名: JPEG_Decode_DMA
7.  *    功能说明: JPEG 解码
8.  *    形    参: hjpeg                JPEG_HandleTypeDef 句柄指针
9.  *              FrameSourceAddress  数据地址
10. *              FrameSize           数据大小
11. *              DestAddress         目的数据地址
12. *    返 回 值: HAL_ERROR 表示配置失败，HAL_OK 表示配置成功
13. *              HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出
14. *********************************************************************************************************
15. */
16. uint32_t JPEG_Decode_DMA(JPEG_HandleTypeDef *hjpeg, uint32_t FrameSourceAddress ,uint32_t FrameSize,
17. uint32_t DestAddress)
```

```
18.  {
19.      JPEGSourceAddress =  FrameSourceAddress ;
20.      FrameBufferAddress = DestAddress;
21.      Input_frameIndex = 0;
22.      Input_frameSize = FrameSize;
23.
24.      /* 设置标志，0 表示开始解码，1 表示解码完成 */
25.      Jpeg_HWDecodingEnd = 0;
26.
27.      /* 启动 JPEG 解码 */
28.      HAL_JPEG_Decode_DMA(hjpeg ,(uint8_t *)JPEGSourceAddress ,CHUNK_SIZE_IN ,
29.                                  (uint8_t *)FrameBufferAddress ,CHUNK_SIZE_OUT);
30.
31.      return HAL_OK;
32.  }
```

下面将程序设计中几个关键地方做个阐释：

- 第 1 行，解码过程中，每次加载的数据大小，如果解码的图片大小比这个数值小是没关系的。
- 第 2 行，解码过程中，每次输出的数据大小。
- 第 25 行，用此变量做解码完成标志。
- 第 28 行，启动 JPEG 解码，关于此函数的讲解在第 57 章的 4.3 小节有详细说明。

58.2.4第 3 步：MDMA 配置

硬件 JPEG 数据的加载和输出都是通过 MDMA 做数据传输，代码如下：

```
1.   /*
2.   *********************************************************************************************************
3.   *    函 数 名: HAL_JPEG_MspInit
4.   *    功能说明: 初始化 JEPG 所需要的底层
5.   *    形    参: JPEG_HandleTypeDef 句柄指针
6.   *    返 回 值: 无
7.   *********************************************************************************************************
8.   */
9.   void HAL_JPEG_MspInit(JPEG_HandleTypeDef *hjpeg)
10.  {
11.      /* 这两个变量务必设置为静态局部变量或者全局变量，因为退出后，JPEG 句柄还要使用 */
12.      static MDMA_HandleTypeDef   hmdmaIn;
13.      static MDMA_HandleTypeDef   hmdmaOut;
14.
15.      /* 使能 JPEG 时钟 */
16.      __HAL_RCC_JPGDECEN_CLK_ENABLE();
17.
18.      /* 使能 MDMA 时钟 */
19.      __HAL_RCC_MDMA_CLK_ENABLE();
20.
21.      /* 使能 JPEG 中断并配置优先级 */
22.      HAL_NVIC_SetPriority(JPEG_IRQn, 0x07, 0x00);
23.      HAL_NVIC_EnableIRQ(JPEG_IRQn);
24.
25.      /* JPEG 输入的 MDMA 配置 ##############################################*/
26.      hmdmaIn.Instance                = MDMA_Channel7;                    /* 使用 MDMA 通道 7 */
```

```
27.       hmdmaIn.Init.Priority           = MDMA_PRIORITY_HIGH;              /* 优先级高 */
28.       hmdmaIn.Init.Endianness         = MDMA_LITTLE_ENDIANNESS_PRESERVE; /* 小端格式 */
29.       hmdmaIn.Init.SourceInc          = MDMA_SRC_INC_BYTE;               /* 源地址字节递增 */
30.       hmdmaIn.Init.DestinationInc     = MDMA_DEST_INC_DISABLE;           /* 目的地址无自增 */
31.       hmdmaIn.Init.SourceDataSize     = MDMA_SRC_DATASIZE_BYTE;          /* 源地址数据宽度字节 */
32.       hmdmaIn.Init.DestDataSize       = MDMA_DEST_DATASIZE_WORD;         /* 目的地址数据宽度字节 */
33.       hmdmaIn.Init.DataAlignment      = MDMA_DATAALIGN_PACKENABLE;       /* 小端，右对齐 */
34.       hmdmaIn.Init.SourceBurst        = MDMA_SOURCE_BURST_32BEATS;       /* 源数据突发传输，32 次 */
35.       hmdmaIn.Init.DestBurst          = MDMA_DEST_BURST_16BEATS;         /* 目的数据突发传输，16 次 */
36.
37.       hmdmaIn.Init.SourceBlockAddressOffset = 0;  /* 用于 block 传输，buffer 传输用不到 */
38.       hmdmaIn.Init.DestBlockAddressOffset   = 0;  /* 用于 block 传输，buffer 传输用不到 */
39.
40.       hmdmaIn.Init.Request = MDMA_REQUEST_JPEG_INFIFO_TH;      /* JPEG 的 FIFO 阀值触发中断 */
41.       hmdmaIn.Init.TransferTriggerMode = MDMA_BUFFER_TRANSFER; /* 使用 MDMA 的 buffer 传输 */
42.       hmdmaIn.Init.BufferTransferLength = 32;           /* 每次传输 32 个字节，JPEG 的 FIFO 阀值是 32 字节 */
43.
44.       /* 关联 MDMA 的句柄到 JPEG */
45.       __HAL_LINKDMA(hjpeg, hdmain, hmdmaIn);
46.
47.       /* 先复位，然后配置 MDMA */
48.       HAL_MDMA_DeInit(&hmdmaIn);
49.       HAL_MDMA_Init(&hmdmaIn);
50.
51.       /* JPEG 输出的 MDMA 配置 ##########################################*/
52.       hmdmaOut.Instance             = MDMA_Channel6;                   /* 使用 MDMA 通道 6 */
53.       hmdmaOut.Init.Priority        = MDMA_PRIORITY_VERY_HIGH;         /* 优先级最高 */
54.       hmdmaOut.Init.Endianness      = MDMA_LITTLE_ENDIANNESS_PRESERVE; /* 小端格式 */
55.       hmdmaOut.Init.SourceInc       = MDMA_SRC_INC_DISABLE;            /* 源数据地址禁止自增 */
56.       hmdmaOut.Init.DestinationInc  = MDMA_DEST_INC_BYTE;              /* 目的数据地址字节自增 */
57.       hmdmaOut.Init.SourceDataSize  = MDMA_SRC_DATASIZE_WORD;          /* 源地址数据宽度字 */
58.       hmdmaOut.Init.DestDataSize    = MDMA_DEST_DATASIZE_BYTE;         /* 目的地址数据宽度字节 */
59.       hmdmaOut.Init.DataAlignment   = MDMA_DATAALIGN_PACKENABLE;       /* 小端，右对齐 */
60.       hmdmaOut.Init.SourceBurst     = MDMA_SOURCE_BURST_32BEATS;       /* 源数据突发传输，32 次 */
61.       hmdmaOut.Init.DestBurst       = MDMA_DEST_BURST_32BEATS;         /* 目的数据突发传输，16 次 */
62.
63.       hmdmaOut.Init.SourceBlockAddressOffset = 0;  /* 用于 block 传输，buffer 传输用不到 */
64.       hmdmaOut.Init.DestBlockAddressOffset   = 0;  /* 用于 block 传输，buffer 传输用不到 */
65.
66.       hmdmaOut.Init.Request             = MDMA_REQUEST_JPEG_OUTFIFO_TH;  /* JPEG 的 FIFO 阀值触发中断 */
67.       hmdmaOut.Init.TransferTriggerMode = MDMA_BUFFER_TRANSFER;          /* 使用 MDMA 的 buffer 传输 */
68.       hmdmaOut.Init.BufferTransferLength = 32;     /* 每次传输 32 个字节，JPEG 的 FIFO 阀值是 32 字节 */
69.
70.       /* 先复位，然后配置 MDMA */
71.       HAL_MDMA_DeInit(&hmdmaOut);
72.       HAL_MDMA_Init(&hmdmaOut);
73.
74.       /* 关联 MDMA 的句柄到 JPEG */
75.       __HAL_LINKDMA(hjpeg, hdmaout, hmdmaOut);
76.
77.       /* 使能 MDMA 中断并配置优先级 */
78.       HAL_NVIC_SetPriority(MDMA_IRQn, 0x08, 0x00);
79.       HAL_NVIC_EnableIRQ(MDMA_IRQn);
80.   }
```

下面将程序设计中几个关键地方做个阐释：

- 这个函数在用户调用 HAL_JPEG_Init 时会被调用到。

- 第 12-13 行，这两个变量一定要设置为静态局部变量或者全局变量，因为此函数退出后，JPEG 句柄还要使用。如果设置为局部变量，退出函数后，这两个变量占用的栈空间会被释放。
- 第 16-23 行，使能 JPEG 时钟，MDMA 时钟以及 JPEG 中断。
- 第 26-49 行，JPEG 通过 MDMA 实现输入数据配置，这部分知识点会在后面章节专门为大家讲解。当前直接调用即可。
- 第 52-75 行，同上，这里是 JPEG 通过 MDMA 实现输出数据配置。
- 第 78-79 行，这步别忘了，要用到 MDMA 中断。

58.2.5第 4 步：JPEG 输入数据更新

如果用户设置每次解码的数据小于 JPEG 文件大小，那么就需要通过此函数更新：

```
/*
*********************************************************************************************************
*    函 数 名: HAL_JPEG_GetDataCallback
*    功能说明: JPEG 回调函数，用于从输入地址获取新数据继续解码
*    形    参: hjpeg          JPEG_HandleTypeDef 句柄指针
*              NbDecodedData  上一轮已经解码的数据大小，单位字节
*    返 回 值: 无
*********************************************************************************************************
*/
void HAL_JPEG_GetDataCallback(JPEG_HandleTypeDef *hjpeg, uint32_t NbDecodedData)
{
    uint32_t inDataLength;

    /* 更新已经解码的数据大小 */
    Input_frameIndex += NbDecodedData;

    /* 如果当前已经解码的数据小于总文件大小，继续解码 */
    if( Input_frameIndex < Input_frameSize)
    {
        /* 更新解码数据位置 */
        JPEGSourceAddress = JPEGSourceAddress + NbDecodedData;

        /* 更新下一轮要解码的数据大小 */
        if((Input_frameSize - Input_frameIndex) >= CHUNK_SIZE_IN)
        {
            inDataLength = CHUNK_SIZE_IN;
        }
        else
        {
            inDataLength = Input_frameSize - Input_frameIndex;
        }
    }
    else
    {
        inDataLength = 0;
    }

    /* 更新输入缓冲 */
    HAL_JPEG_ConfigInputBuffer(hjpeg,(uint8_t *)JPEGSourceAddress, inDataLength);
```

```
}
```

58.2.6第 5 步：JPEG 输出数据更新

根据用户在函数 HAL_JPEG_Decode_DMA 中配置每次解码时输出的数据大小，需要通过此函数更新输出地址：

```
/*
*********************************************************************************************************
*    函 数 名: HAL_JPEG_DataReadyCallback
*    功能说明: JPEG 回调函数，用于输出缓冲地址更新
*    形    参: hjpeg         JPEG_HandleTypeDef 句柄指针
*              pDataOut      输出数据缓冲
*              OutDataLength 输出数据大小，单位字节
*    返 回 值: 无
*********************************************************************************************************
*/
void HAL_JPEG_DataReadyCallback (JPEG_HandleTypeDef *hjpeg, uint8_t *pDataOut, uint32_t OutDataLength)
{
    /* 更新 JPEG 输出地址 */
    FrameBufferAddress += OutDataLength;

    HAL_JPEG_ConfigOutputBuffer(hjpeg, (uint8_t *)FrameBufferAddress, CHUNK_SIZE_OUT);
}
```

58.2.7第 6 步：YCbCr 格式转 RGB 并显示

JPEG 解码输出数据是 YCbCr 格式，要显示到显示屏上需要使用 RGB 格式。通过 STM32H7 的 DMA2D 即可实现：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: DMA2D_Copy_YCbCr_To_RGB
4.  *    功能说明: YCbCr 转 RGB 输出
5.  *    形    参: pSrc:     数据源地址
6.  *              pDst:     数据目的地址
7.  *              x:        X 轴首地址
8.  *              y:        Y 轴首地址
9.  *              xsize:    目的区域的 X 轴大小，即每行像素数
10. *              ysize:    目的区域的 Y 轴大小，即行数
11. *              PixelFormat:    目标区颜色格式
12. *              ChromaSampling : YCbCr Chroma sampling : 4:2:0, 4:2:2 or 4:4:4
13. *    返 回 值: 无
14. *********************************************************************************************************
15. */
16. static void DMA2D_Copy_YCbCr_To_RGB(uint32_t *pSrc,
17.                                     uint32_t *pDst,
18.                                     uint16_t x,
19.                                     uint16_t y,
20.                                     uint16_t xsize,
21.                                     uint16_t ysize,
22.                                     uint32_t PixelFormat,
23.                                     uint32_t ChromaSampling)
```

```
{
    uint32_t cssMode = DMA2D_CSS_420;
    uint32_t inputLineOffset = 0;
    uint32_t destination = 0;

    /* 处理输入行偏移 */
    if(ChromaSampling == JPEG_420_SUBSAMPLING)
    {
        cssMode = DMA2D_CSS_420;

        inputLineOffset = xsize % 16;
        if(inputLineOffset != 0)
        {
            inputLineOffset = 16 - inputLineOffset;
        }
    }
    else if(ChromaSampling == JPEG_444_SUBSAMPLING)
    {
        cssMode = DMA2D_NO_CSS;

        inputLineOffset = xsize % 8;
        if(inputLineOffset != 0)
        {
            inputLineOffset = 8 - inputLineOffset;
        }
    }
    else if(ChromaSampling == JPEG_422_SUBSAMPLING)
    {
        cssMode = DMA2D_CSS_422;

        inputLineOffset = xsize % 16;
        if(inputLineOffset != 0)
        {
            inputLineOffset = 16 - inputLineOffset;
        }
    }

    /* 输出地址，特别注意末尾乘以 2 对应 RGB565，如果输出格式是 ARGB8888，需要乘以 4 */
    destination = (uint32_t)pDst + ((y * g_LcdWidth) + x) * 2;


    /* DMA2D 采用存储器到存储器模式，并且执行 FPC 颜色格式转换，这种模式是前景层作为 DMA2D 输入 */
    DMA2D->CR      = 0x00010000UL | (1 << 9);
    DMA2D->OOR     = g_LcdWidth - xsize;

    /* 输出格式 */
    DMA2D->OPFCCR  = PixelFormat
                   | (DMA2D_REGULAR_ALPHA << 20)
                   | (DMA2D_RB_REGULAR << 21);

    /* 前景层输入格式 */
    DMA2D->FGPFCCR = DMA2D_INPUT_YCBCR
                   | (DMA2D_REPLACE_ALPHA << 16)
                   | (DMA2D_REGULAR_ALPHA << 20)
                   | (DMA2D_RB_REGULAR << 21)
                   | (0xFFU << 24)
                   | (cssMode << 18);

```

```
82.       DMA2D->FGOR    = inputLineOffset;
83.       DMA2D->NLR     = (uint32_t)(xsize << 16) | (uint16_t)ysize;
84.       DMA2D->OMAR    = (uint32_t)destination;
85.       DMA2D->FGMAR   = (uint32_t)pSrc;
86.
87.       /* 启动传输 */
88.       DMA2D->CR    |= DMA2D_CR_START;
89.
90.       /* 等待 DMA2D 传输完成 */
91.       while (DMA2D->CR & DMA2D_CR_START) {}
92.   }
```

下面将程序设计中几个关键地方做个阐释：

- 第 30-59 行，获取输入行偏移。
- 第 62 行,计算输出地址,特别注意末尾乘以 2 对应 RGB565 颜色格式,如果输出格式是 ARGB8888，需要乘以 4。变量 g_LcdWidth 表示显示屏的宽度，单位像素。
- 第 66 行，DMA2D 采用存储器到存储器模式，并且执行 FPC 颜色格式转换，这种模式是前景层作为 DMA2D 输入。
- 第 67 行，输出行偏移，行偏移的意思就是一行结束到下一行开始的距离，单位像素个数。
- 第 70-72 行，输出格式配置。
- 第 75-80 行，前景层输入格式。
- 第 82 行，输入行偏移。
- 第 83 行，总传输次数。
- 第 84 行，数据传输目的地址。
- 第 85 行，数据传输源地址。
- 第 88-91 行，启动 DMA2D 传输，并等待传输完成。

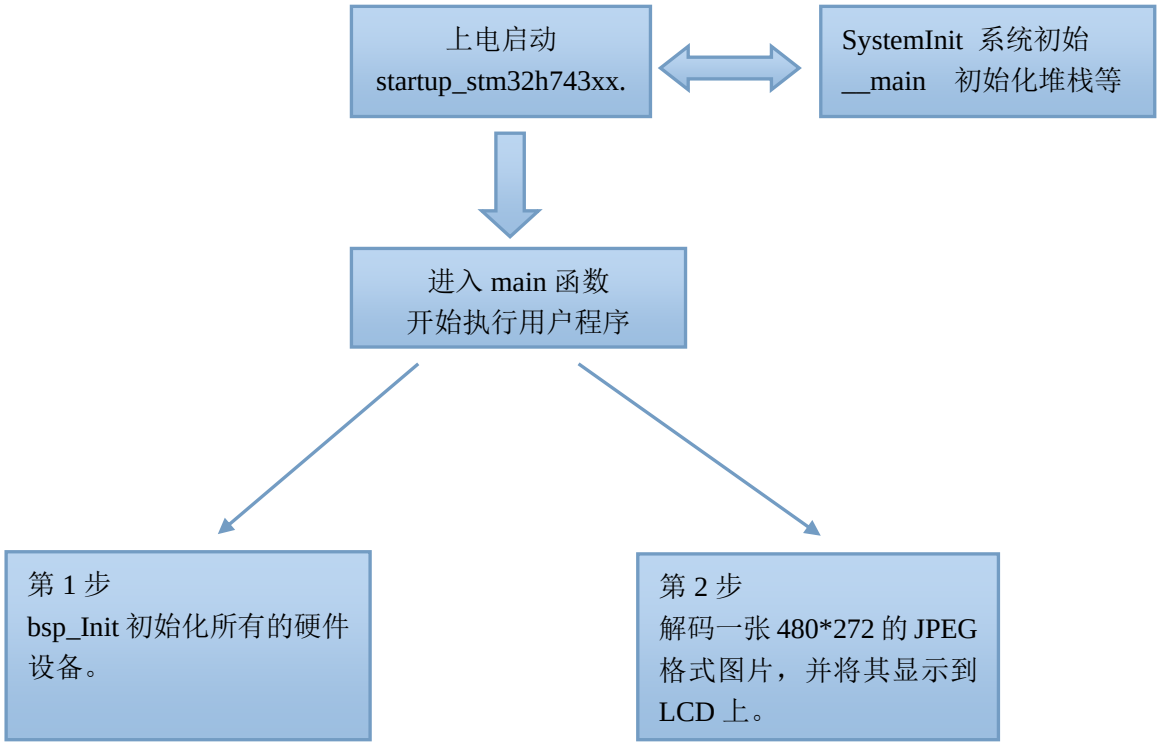
58.3 硬件 JPEG 驱动移植和使用

JPEG 移植比较简单：

- 第 1 步：复制 decode_dma.c 和 decode_dma.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：这几个驱动文件主要用到 HAL 库的 JPEG 和 MDMA 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 3 步，应用方法看本章节配套例子即可。

58.4 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED ，LCD，和 SDRAM。
  - 第 2 步，解码一张 480*272 的 JPEG 格式图片，并将其显示到 LCD 上。

58.5 实验例程说明（MDK）

配套例子：

V7-036_硬件 JPEG 实现图片解码显示

实验目的：

1. 学习 STM32H7 的硬件 JPEG 解码。

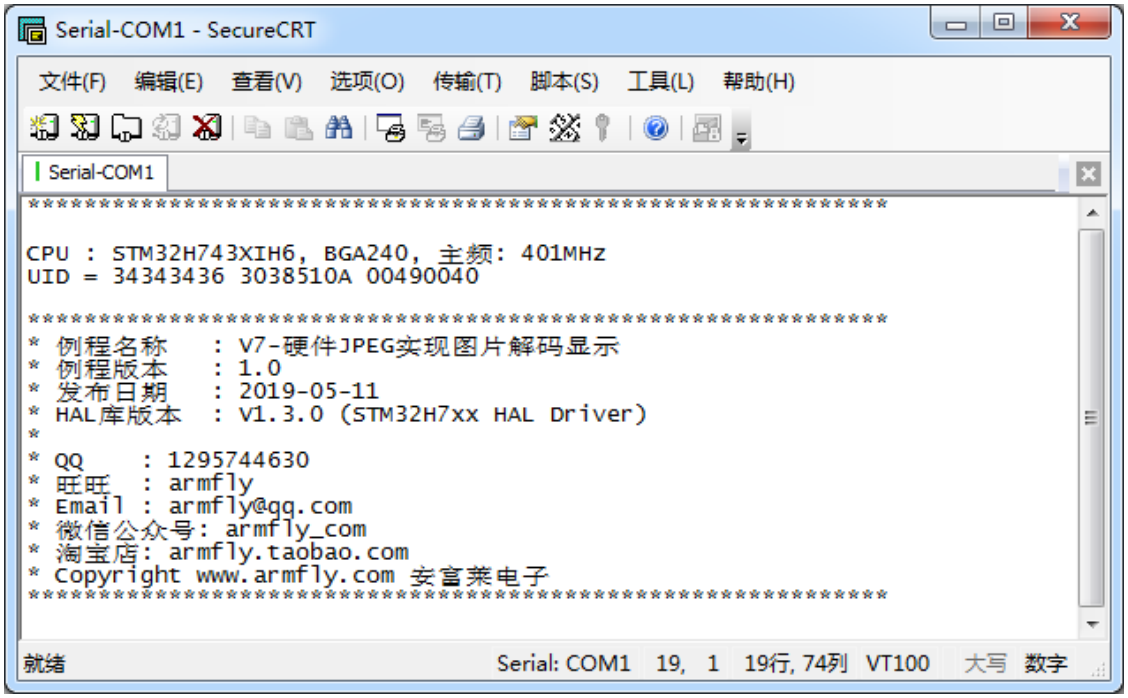
实验内容：

1. 解码一张 480*272 大小的 JPEG 图片并显示。

LCD 界面显示效果如下：

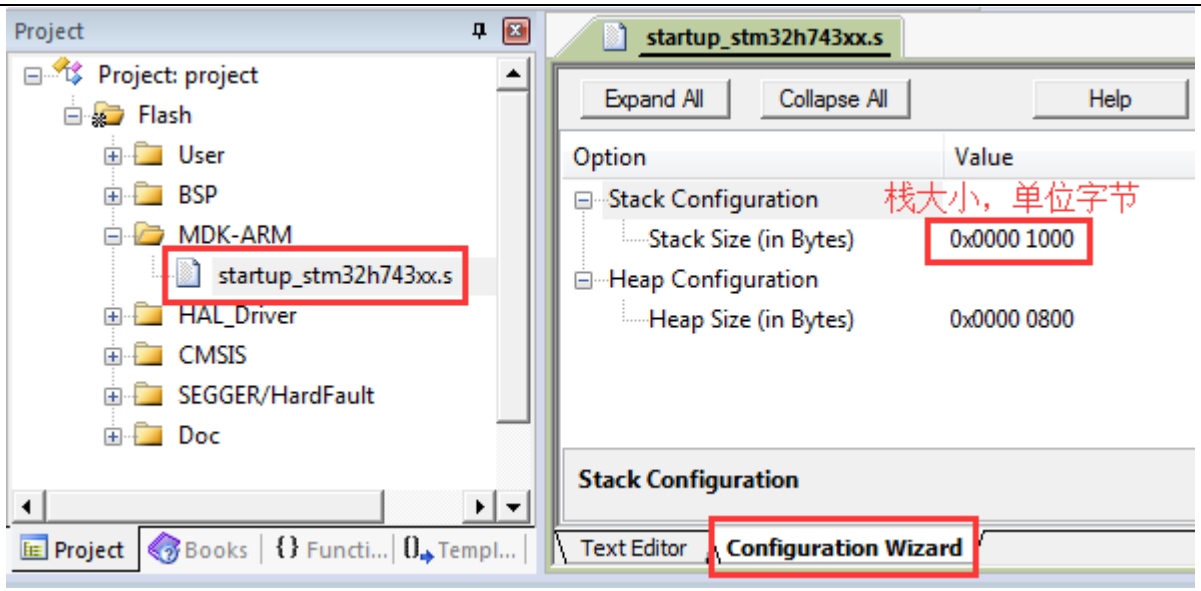
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

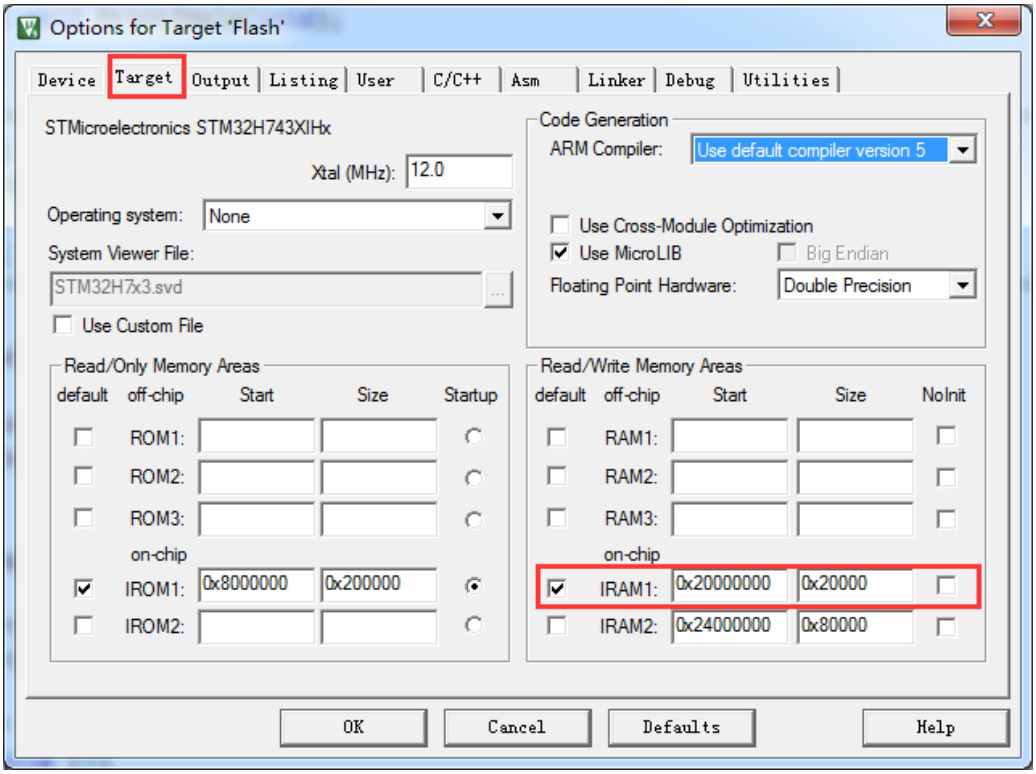


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */

    bsp_InitI2C();    /* 初始化 I2C 总线 */
    TOUCH_InitHard();  /* 初始化触摸芯片，LCD 面板型号的检查也在此函数，所以要在函数 LCD_InitHard 前调用 */
    LCD_InitHard();    /* 初始化 LCD */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SDRAM。由于 SDRAM 要用于 LCD 的显存，方便起见，直接将其配置为 WT 模式。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;
```

```
    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SDRAM 的 MPU 属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0xC0000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_32MB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
```

```
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能:**

主程序实现如下操作:

- 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。
- LCD 第 1 个图：使用 DMA2D 刷色块。
- LCD 第 2 个图：显示 ARGB8888 位图。
- LCD 第 3 个图：显示 RGB565 位图。
- LCD 第 4 个图：两个位图混合。
- LCD 第 5 个图：Alpha 透明度 200 的位图显示。
- LCD 第 6 个图：Alpha 透明度 100 的位图显示。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint16_t ucBright;      /* 背光亮度(0-255) */
    FONT_T tFont;           /* 定义一个字体结构体变量，用于设置字体参数 */


    /* 设置字体参数 */
    {
        tFont.FontCode = FC_ST_16;       /* 字体代码 16 点阵 */
        tFont.FrontColor = CL_WHITE;     /* 字体颜色 */
        tFont.BackColor = CL_BLUE;       /* 文字背景颜色 */
        tFont.Space = 0;                 /* 文字间距，单位 = 像素 */
    }

    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    /* 延迟 200ms 再点亮背光，避免瞬间高亮 */
    bsp_DelayMS(200);

    LCD_ClrScr(CL_BLUE);
```

```
/* 界面整体显示完毕后，再打开背光，设置为缺省亮度 */
bsp_DelayMS(100);
ucBright = BRIGHT_DEFAULT;
LCD_SetBackLight(ucBright);

/* 第 1 个图：使用 DMA2D 刷色块 ###################################################################*/
LCD_DispStr(24, 2, "DMA2D 刷色块", &tFont);
_DMA2D_Fill((void *)(SDRAM_LCD_BUF1 + g_LcdWidth*20*2 + 24*2), /* 显示起始地址(24, 20) */
            128,                                               /* 色块长 */
              128,                                               /* 色块高 */
              g_LcdWidth-128,                                    /* 色块行偏移 */
              CL_RED,                                            /* 色块颜色 */
              LTDC_PIXEL_FORMAT_RGB565);                         /* 色块颜色格式 */

/* 第 2 个图：显示 ARGB8888 位图 ###################################################################*/
LCD_DispStr(176, 2, "刷 ARGB8888 位图", &tFont);
_DMA2D_DrawAlphaBitmap((void *)(SDRAM_LCD_BUF1 + g_LcdWidth*20*2 + 176*2), /* 显示起始地址(176, 20) */
                       (void *)_aclufei,                                   /* 位图地址 */
                       128,                                                /* 位图长 */
                       128,                                                /* 位图高 */
                       0,                                                  /* 位图行偏移 */
                       g_LcdWidth-128,                                     /* 目标区行偏移 */
                       LTDC_PIXEL_FORMAT_RGB565);                          /* 目标区颜色格式 */

/* 第 3 个图：显示 RGB565 位图 ###################################################################*/
LCD_DispStr(328, 2, "刷 RGB565 位图", &tFont);
_DMA2D_Copy((uint32_t *)_acmickey,                                 /* 位图地址 */
              (uint32_t *)(SDRAM_LCD_BUF1 + g_LcdWidth*20*2 + 328*2),  /* 显示起始地址(328, 20) */
              128,                                                 /* 位图长 */
              128,                                                 /* 位图高 */
              0,                                                   /* 位图行偏移 */
              g_LcdWidth-128,                                      /* 目标区行偏移 */
               LTDC_PIXEL_FORMAT_RGB565);                           /* 目标区颜色格式 */


/* 第 4 个图：两个位图混合 ###################################################################*/
LCD_DispStr(24, 150, "两个位图混合", &tFont);
_DMA2D_AlphaBlendingBulk((uint32_t *)_aclufei,                          /* 前景层位图地址 */
                            0,                                          /* 前景层行偏移  */
                         (uint32_t *)_acsuolong,                        /* 背景层位图地址  */
                            0,                                          /* 背景层行偏移  */
                   (uint32_t *)(SDRAM_LCD_BUF1 +  g_LcdWidth*168*2 + 24*2), /* 显示起始地址(24, 168) */
                            g_LcdWidth-128,                             /* 目标区行偏移 */
                            128,                                        /* 目标区长 */
                            128);                                       /* 目标区高 */

/* 第 5 个图：Alpha 透明度 200 的位图显示 ###################################################################*/
LCD_DispStr(176, 150, "Alpha 透明度 200", &tFont);
_DMA2D_MixColorsBulk((uint32_t *)_achuoying,                              /* 位图地址 */
                     0,                                                   /* 位图行偏移 */
                    (uint32_t *)(SDRAM_LCD_BUF1 + g_LcdWidth*168*2 + 176*2), /* 显示起始地址(176, 168) */
                        g_LcdWidth-128,                                   /* 目标区行偏移 */
                        128,                                              /* 目标区长 */
                        128,                                              /* 目标区高 */
                        200);                                             /* 位图显示透明度 200 */

/* 第 6 个图：Alpha 透明度 100 的位图显示 ###################################################################*/
```

```
    LCD_DispStr(328, 150, "Alpha 透明度 100", &tFont);
    _DMA2D_MixColorsBulk((uint32_t *)_achuoying,                                    /* 位图地址 */
                         0,                                                         /* 位图行偏移 */
                        (uint32_t *)(SDRAM_LCD_BUF1 + g_LcdWidth*168*2 + 328*2), /* 显示起始地址(328, 168) */
                            g_LcdWidth-128,                                         /* 目标区行偏移 */
                            128,                                                    /* 目标区长 */
                            128,                                                    /* 目标区高 */
                            100);                                                   /* 位图显示透明度200 */


    bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */

    while (1)
    {
      bsp_Idle();

        /* 判断软件定时器 0 是否超时 */
        if(bsp_CheckTimer(0))
        {
            /* 每隔 200ms 进来一次 */
            bsp_LedToggle(2);
        }
    }
}
```

58.6 实验例程说明（IAR）

配套例子：

V7-036_硬件 JPEG 实现图片解码显示

实验目的：

1. 学习 STM32H7 的硬件 JPEG 解码。

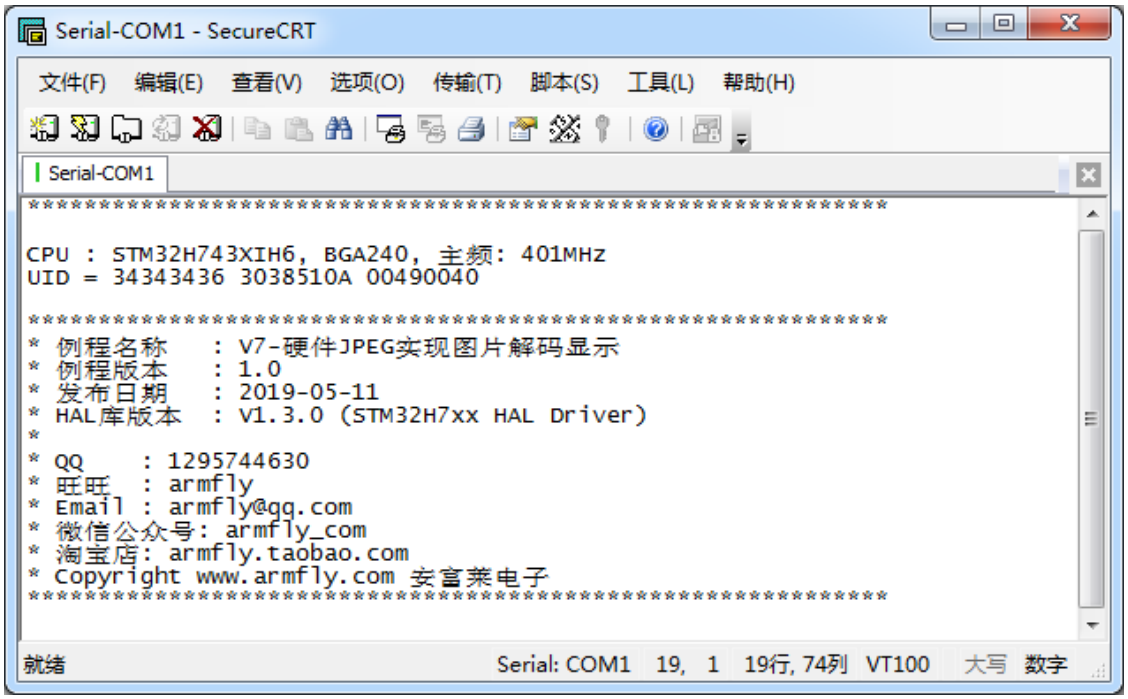
实验内容：

1. 解码一张 480*272 大小的 JPEG 图片并显示。

LCD 界面显示效果如下：

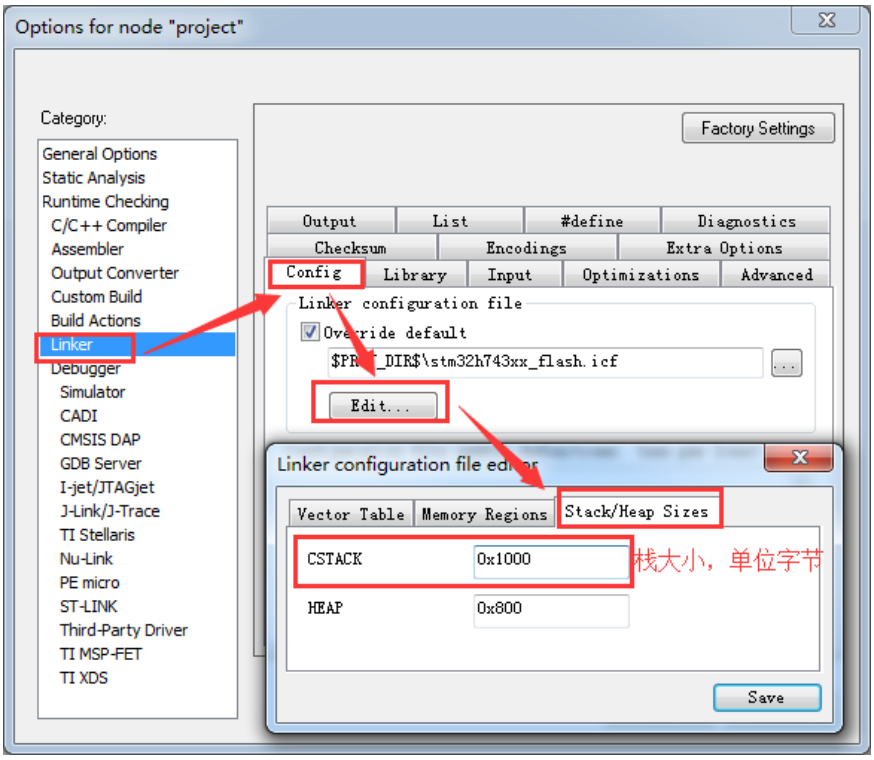
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

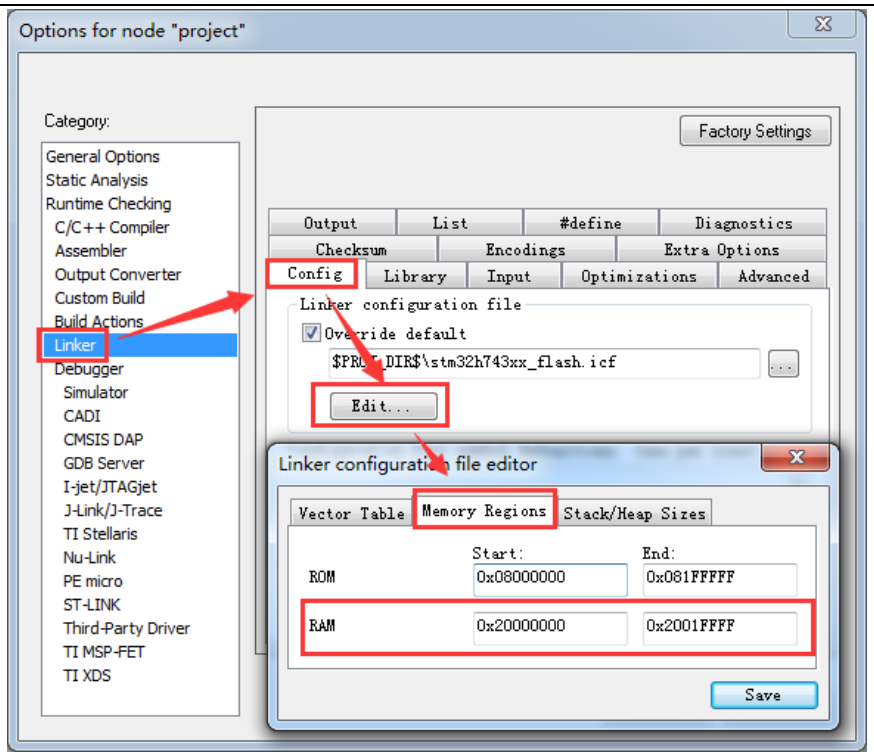


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */

    bsp_InitI2C();      /* 初始化 I2C 总线 */
    TOUCH_InitHard();   /* 初始化触摸芯片，LCD 面板型号的检查也在此函数，所以要在函数 LCD_InitHard 前调用 */
    LCD_InitHard();     /* 初始化 LCD */
}
```

- **MPU 配置和 Cache 配置：**

数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 SDRAM。由于 SDRAM 要用于 LCD 的显存，方便起见，直接将其配置为 WT 模式。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
```

```
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SDRAM 的 MPU 属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0xC0000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_32MB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- 启动 1 个 200ms 的自动重装定时器，让 LED2 每 200ms 翻转一次。
- 解码一张 480*272 大小的 JPEG 图片并显示。

```
/*
*********************************************************************************************************
```

```
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint16_t ucBright;      /* 背光亮度(0-255) */
    FONT_T tFont;           /* 定义一个字体结构体变量，用于设置字体参数 */


    /* 设置字体参数 */
    {
        tFont.FontCode = FC_ST_16;       /* 字体代码 16 点阵 */
        tFont.FrontColor = CL_WHITE;     /* 字体颜色 */
        tFont.BackColor = CL_BLUE;       /* 文字背景颜色 */
        tFont.Space = 0;                 /* 文字间距，单位 = 像素 */
    }

    bsp_Init();         /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    /* 延迟 200ms 再点亮背光，避免瞬间高亮 */
    bsp_DelayMS(200);

    LCD_ClrScr(CL_BLUE);

    /* 界面整体显示完毕后，再打开背光，设置为缺省亮度 */
    bsp_DelayMS(100);
    ucBright = BRIGHT_DEFAULT;
    LCD_SetBackLight(ucBright);

    /* 第 1 个图：使用 DMA2D 刷色块 ###########################################################*/
    LCD_DispStr(24, 2, "DMA2D 刷色块", &tFont);
    _DMA2D_Fill((void *)(SDRAM_LCD_BUF1 + g_LcdWidth*20*2 + 24*2), /* 显示起始地址(24, 20) */
                128,                                               /* 色块长 */
                  128,                                               /* 色块高 */
                  g_LcdWidth-128,                                    /* 色块行偏移 */
                  CL_RED,                                            /* 色块颜色 */
                  LTDC_PIXEL_FORMAT_RGB565);                         /* 色块颜色格式 */

    /* 第 2 个图：显示 ARGB8888 位图 ###########################################################*/
    LCD_DispStr(176, 2, "刷 ARGB8888 位图", &tFont);
    _DMA2D_DrawAlphaBitmap((void *)(SDRAM_LCD_BUF1 + g_LcdWidth*20*2 + 176*2), /* 显示起始地址(176, 20) */
                           (void *)_aclufei,                                   /* 位图地址 */
                           128,                                                /* 位图长 */
                           128,                                                /* 位图高 */
                           0,                                                  /* 位图行偏移 */
                           g_LcdWidth-128,                                     /* 目标区行偏移 */
                           LTDC_PIXEL_FORMAT_RGB565);                          /* 目标区颜色格式 */

    /* 第 3 个图：显示 RGB565 位图 ###########################################################*/
    LCD_DispStr(328, 2, "刷 RGB565 位图", &tFont);
    _DMA2D_Copy((uint32_t *)_acmickey,                                  /* 位图地址 */
                (uint32_t *)(SDRAM_LCD_BUF1 + g_LcdWidth*20*2 + 328*2),  /* 显示起始地址(328, 20) */
                128,                                                    /* 位图长 */
```

```
                128,                                                    /* 位图高 */
                0,                                                      /* 位图行偏移 */
                g_LcdWidth-128,                                         /* 目标区行偏移 */
                 LTDC_PIXEL_FORMAT_RGB565);                             /* 目标区颜色格式 */


    /* 第 4 个图：两个位图混合 ###########################################################*/
    LCD_DispStr(24, 150, "两个位图混合", &tFont);
    _DMA2D_AlphaBlendingBulk((uint32_t *)_aclufei,                      /* 前景层位图地址 */
                             0,                                         /* 前景层行偏移  */
                            (uint32_t *)_acsuolong,                     /* 背景层位图地址  */
                             0,                                         /* 背景层行偏移  */
                    (uint32_t *)(SDRAM_LCD_BUF1 +  g_LcdWidth*168*2 + 24*2), /* 显示起始地址(24, 168) */
                             g_LcdWidth-128,                            /* 目标区行偏移 */
                             128,                                       /* 目标区长 */
                             128);                                      /* 目标区高 */

    /* 第 5 个图：Alpha 透明度 200 的位图显示 ###################################################*/
    LCD_DispStr(176, 150, "Alpha 透明度 200", &tFont);
    _DMA2D_MixColorsBulk((uint32_t *)_achuoying,                        /* 位图地址 */
                         0,                                             /* 位图行偏移 */
                        (uint32_t *)(SDRAM_LCD_BUF1 + g_LcdWidth*168*2 + 176*2), /* 显示起始地址(176, 168) */
                            g_LcdWidth-128,                             /* 目标区行偏移 */
                            128,                                        /* 目标区长 */
                            128,                                        /* 目标区高 */
                            200);                                       /* 位图显示透明度200 */

    /* 第 6 个图：Alpha 透明度 100 的位图显示 ###################################################*/
    LCD_DispStr(328, 150, "Alpha 透明度 100", &tFont);
    _DMA2D_MixColorsBulk((uint32_t *)_achuoying,                        /* 位图地址 */
                         0,                                             /* 位图行偏移 */
                        (uint32_t *)(SDRAM_LCD_BUF1 + g_LcdWidth*168*2 + 328*2), /* 显示起始地址(328, 168) */
                            g_LcdWidth-128,                             /* 目标区行偏移 */
                            128,                                        /* 目标区长 */
                            128,                                        /* 目标区高 */
                            100);                                       /* 位图显示透明度200 */


    bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */

    while (1)
    {
      bsp_Idle();

        /* 判断软件定时器 0 是否超时 */
        if(bsp_CheckTimer(0))
        {
            /* 每隔 200ms 进来一次 */
            bsp_LedToggle(2);
        }
    }
}

/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
```

```
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */

    /* 延迟 200ms 再点亮背光，避免瞬间高亮 */
    bsp_DelayMS(200);

    LCD_ClrScr(CL_BLUE);

    /* 界面整体显示完毕后，再打开背光，设置为缺省亮度 */
    bsp_DelayMS(100);
    LCD_SetBackLight(BRIGHT_DEFAULT);

    TestJpeg(); /* JPEG 测试 */
}

/*
*********************************************************************************************************
*    函 数 名: TestJpeg
*    功能说明: 硬件 JPEG 测试
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void TestJpeg(void)
{
    int iTimeStart, iTimeEnd;
    FONT_T tFont;      /* 定义一个字体结构体变量，用于设置字体参数 */
    char buf0[100];
    char buf1[100];


    /* 设置字体属性 */
    tFont.FontCode = FC_ST_16;       /* 字体选择宋体 16 点阵，高 16x 宽 15) */
    tFont.FrontColor = CL_RED;       /* 字体颜色设置为红色 */
    tFont.BackColor = CL_MASK;       /* 文字背景颜色，透明 */
    tFont.Space = 0;                 /* 字符水平间距，单位 = 像素 */

    /* 第 1 步：JPEG 初始化 ############################################*/
    JPEG_Handle.Instance = JPEG;
    HAL_JPEG_Init(&JPEG_Handle);

    iTimeStart = bsp_GetRunTime();
    JPEG_Decode_DMA(&JPEG_Handle, (uint32_t)_ac1, sizeof(_ac1) , SDRAM_APP_BUF);

    /* 第 2 步：等待 JPEG 解码完成 ############################################*/
    while(Jpeg_HWDecodingEnd == 0){}
    iTimeEnd = bsp_GetRunTime();
    sprintf(buf0, "STM32H7 硬件 JPEG 解码 480*272 图片时间=%dms", iTimeEnd- iTimeStart);

    /* 第 3 步：获取 JPEG 图片信息#############################################*/
    HAL_JPEG_GetInfo(&JPEG_Handle, &JPEG_Info);

    /* 第 4 步：绘制 JPEG 图片到显示屏#############################################*/
```

```
    iTimeStart = bsp_GetRunTime();
    DMA2D_Copy_YCbCr_To_RGB((uint32_t *)SDRAM_APP_BUF,  /* JEPG 解码后的数据 */
                            (uint32_t *)SDRAM_LCD_BUF1, /* 这里是显存地址 */
                            0 ,
                            0,
                            JPEG_Info.ImageWidth,
                            JPEG_Info.ImageHeight,
                              LTDC_PIXEL_FORMAT_RGB565,
                            JPEG_Info.ChromaSubsampling);


    iTimeEnd = bsp_GetRunTime();
    LCD_DispStr(0, 0, buf0, &tFont);

    sprintf(buf1, "STM32H7 硬件 JPEG 显示 480*272 图片时间=%dms", iTimeEnd- iTimeStart);
    LCD_DispStr(0, 18, buf1, &tFont);

    bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();

        /* 判断软件定时器 0 是否超时 */
        if(bsp_CheckTimer(0))
        {
            /* 每隔 200ms 进来一次 */
            bsp_LedToggle(2);
        }
    }
}
```

58.7 总结

本章节涉及到的知识点比较重要，以后做 GUI 移植也要用到，可以大大加速 GUI 的 JPEG 解码速度。

第59章 STM32H7 的 DAC 基础知识和 HAL 库 API

本章节为大家讲解 DAC，实际项目用到 DAC 的地方比较多，而且 H7 的 DAC 性能也比较给力。

59.1 初学者重要提示
59.2 DAC 基础知识
59.3 DAC 的 HAL 库用法
59.4 源文件 stm32h7xx_hal_dac.c
59.5 总结

59.1 初学者重要提示

- 注意 STM32H7 只有一个 DAC，但有两个独立的通道，跟 F4 的略不同，F4 是两个 DAC。
- 如果仅使用 STM32H7 的一个通道，即 PA4 或者 PA5 引脚，另一个引脚没有做任何配置，这个引脚上会有波形效应。
- STM32H7 的 DAC 支持出厂校准和用户校准模式。特别注意一点，校准是建立在用户使能了输出缓冲的情况下才有效。
- STM32H7 的 DAC 支持正常模式和采样保持模式，其中采样保持模式用于低功耗状态使用。
- DAC 的输出除了可以连接 PA4 或者 PA5 引脚，也可以连接到片上外设，比如运放，比较器。

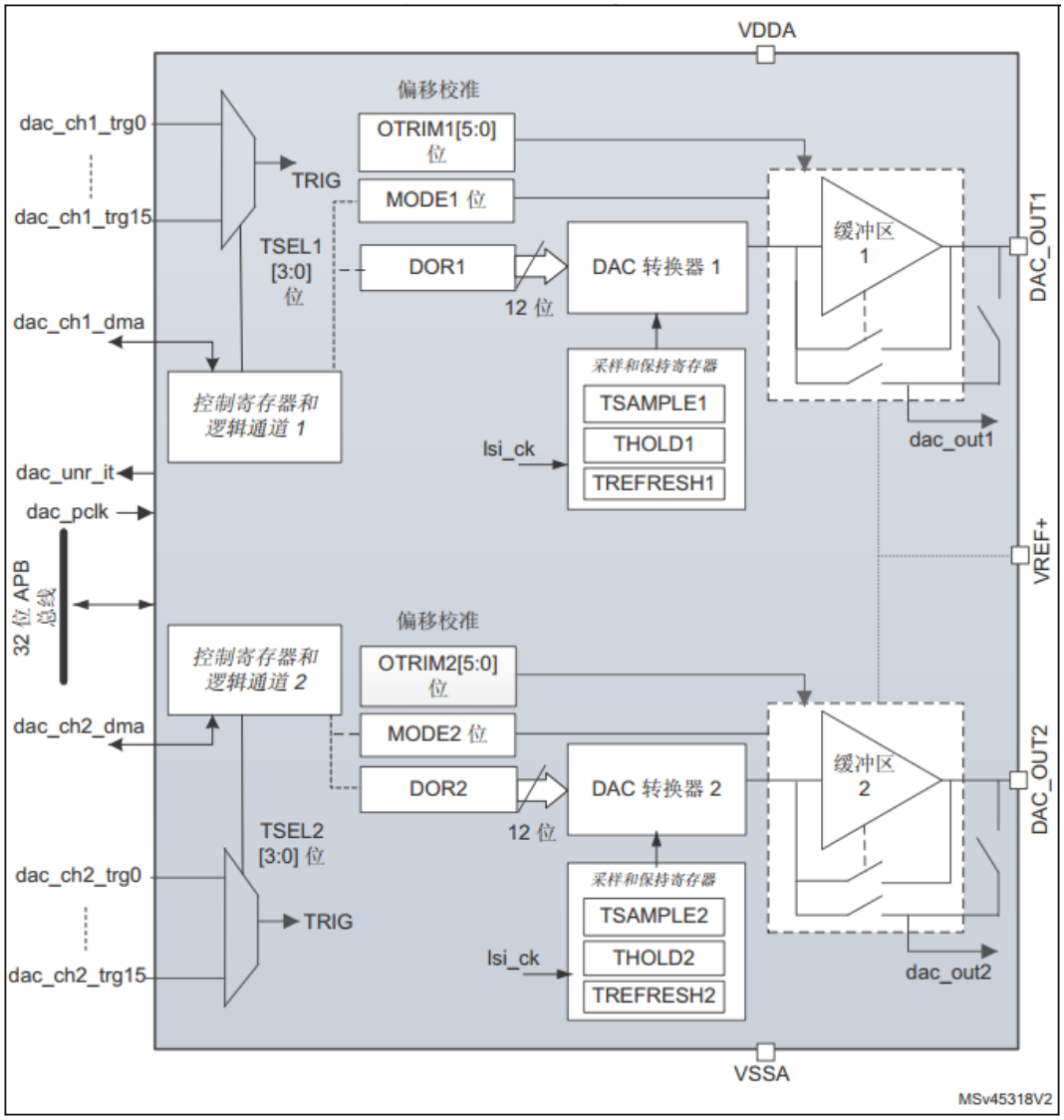
59.2 DAC 基础知识

对于 STM32H7 的 DAC 了解到以下几点即可：

- STM32H7 的 DAC 只有一个，但有两个独立的通道，跟 F4 的略不同，F4 是两个 DAC
- 12 位分辨率，双通道，支持独立或者同时使用。
- 两个 DAC 通道均支持 DMA。
- 每路 DAC 输出均可与 DAC_OUTx 输出引脚断开连接，而且 DAC 输出可与片上外设连接。
- 支持偏移校准，参考电压可以使用内部的 VREFBUF，也可以使用 VREF+引脚外接的电压基准。
- 支持噪声波和三角波生成。这两种方案不够灵活，所以基本都采用定时器触发+DMA 方式生成任意波形。

59.2.1 DAC 硬件框图

认识一个外设，最好的方式就是看它的框图，方便我们快速地了解 DAC 的基本功能，然后再看手册了解细节。框图如下所示：



通过这个框图，我们可以得到如下信息：

- VDDA

  用于 ADC、DAC、运放、比较器和电压基准供电，这部分供电是独立的。

- VREF+

  用于 ADC 和 DAC 的基准电压，当使能了 STM32H7 内部的电压基准，将使用内部基准供 VREF+，VREF-。如果没有使能的话，通过外置电压基准提供。

- VSSA

  所有电源和模拟稳压器的地端。

- dac_ch1_dma

DAC 通道 1 的 DMA 请求。

- dac_ch2_dma

  DAC 通道 2 的 DMA 请求。

- dac_ch1_trg[0:15]

  DAC 通道 1 的输入触发。

- dac_ch2_trg[0:15]

  DAC 通道 2 的输入触发。

- dac_unr_it

  DAC 输出的下溢中断信号。

- dac_pclk

  DAC 时钟输入

- dac_out1

  DAC 通道 1 输出。

- dac_out2

  DAC 通道 2 输出。

- lsi_ck

  使用 LSI 时钟源，可以让 DAC 在停止模式下运行。

59.2.2 DAC 数据格式和输出电压

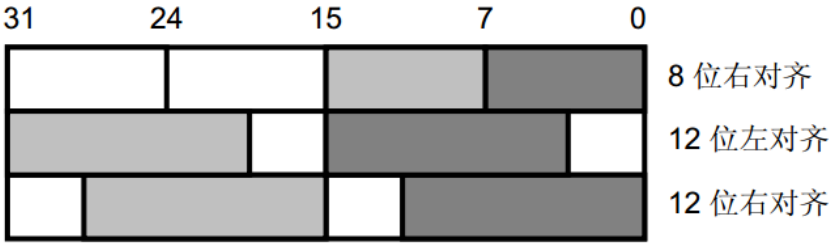
DAC 的数据寄存器设计比较灵活，每个通道都有一组单独的寄存器（下面是通道 1 的寄存器）：

- 8 位右对齐数据保持寄存器 DACx_DHR8R1。
- 12 位右对齐数据保持寄存器 DACx_DHR12R1。
- 12 位左对齐数据保持寄存器 DACx_DHR12L1。

除了这种单独寄存器，为了降低带宽，也支持两个通道公用一个寄存器。

- 8 位右对齐数据保持寄存器 DACx_DHR8RD。
- 12 位左对齐数据保持寄存器 DACx_DHR12LD。
- 12 位右对齐数据保持寄存器 DACx_DHR12RD。

通道 1 和通道 2 共用的效果如下：



由于 DAC 是 12bit 的 DAC，那么范围就是 0-4095，对应的输出电压如下：

DAC Output = Vref *（DOR / 4095），其中 Vref 是参考电压，DOR 是数据输出寄存器。

比如需要 DAC 输出 0.7V，那么假设 VREF+ = 3.3V， DAC_OUT1 = (3.3 * 868) / 4095 = 0.7V。

59.2.3 DAC 支持的触发源

DAC 支持软件触发和硬件触发，具体支持的触发源如下：

```
#define DAC_TRIGGER_NONE          ((uint32_t)0x00000000)
#define DAC_TRIGGER_SOFTWARE      ((uint32_t)(DAC_CR_TEN1))
#define DAC_TRIGGER_T1_TRGO       ((uint32_t)(DAC_CR_TSEL1_0 | DAC_CR_TEN1))
#define DAC_TRIGGER_T2_TRGO       ((uint32_t)(DAC_CR_TSEL1_1 | DAC_CR_TEN1))
#define DAC_TRIGGER_T4_TRGO       ((uint32_t)(DAC_CR_TSEL1_1 | DAC_CR_TSEL1_0 | DAC_CR_TEN1)
#define DAC_TRIGGER_T5_TRGO       ((uint32_t)(DAC_CR_TSEL1_2 |DAC_CR_TEN1))
#define DAC_TRIGGER_T6_TRGO       ((uint32_t)(DAC_CR_TSEL1_2 | DAC_CR_TSEL1_0 | DAC_CR_TEN1))
#define DAC_TRIGGER_T7_TRGO       ((uint32_t)(DAC_CR_TSEL1_2 | DAC_CR_TSEL1_1 | DAC_CR_TEN1))
#define DAC_TRIGGER_T8_TRGO       ((uint32_t)(DAC_CR_TSEL1_2 | DAC_CR_TSEL1_1 | DAC_CR_TSEL1_0 | DAC_CR_TEN1))
#define DAC_TRIGGER_T15_TRGO      ((uint32_t)(DAC_CR_TSEL1_3 | DAC_CR_TEN1))
#define DAC_TRIGGER_HR1_TRGO1     ((uint32_t)(DAC_CR_TSEL1_3 | DAC_CR_TSEL1_0 | DAC_CR_TEN1))
#define DAC_TRIGGER_HR1_TRGO2     ((uint32_t)(DAC_CR_TSEL1_3 | DAC_CR_TSEL1_1 | DAC_CR_TEN1))
#define DAC_TRIGGER_LP1_OUT       ((uint32_t)(DAC_CR_TSEL1_3 | DAC_CR_TSEL1_1 | DAC_CR_TSEL1_0 | DAC_CR_TEN1))
#define DAC_TRIGGER_LP2_OUT       ((uint32_t)(DAC_CR_TSEL1_3 | DAC_CR_TSEL1_2 | DAC_CR_TEN1))
#define DAC_TRIGGER_EXT_IT9       ((uint32_t)(DAC_CR_TSEL1_3 | DAC_CR_TSEL1_2 | DAC_CR_TSEL1_0 | DAC_CR_TEN1))
```

59.2.4 DAC 正常模式和采样保持模式

关于正常模式和采样保持模式，注意以下几点：

- 正常模式是平时最常用的方式，比较好理解。而采样保持模式用于低功耗方式。
- 在采样保持模式下，DAC 内核转换数据，然后保持电容上的电压。不转换时，DAC 内核和样本之间的缓冲器完全关闭，DAC 输出为三态，因此降低了整体功耗，但每次新转换前都需要一段稳定期。
- 采样保持模式可修改内部或者外部参考电压。
- 采样保持部分可以用 LSI 时钟，也可以运行在几种低功耗模式下，如 RUN 模式, SLEEP& STOP 模式。

59.2.5 DAC 的出厂校准和用户校准

一般情况下，使用出厂校准即可，芯片上电后自动完成出厂校准。而用户校准略麻烦，暂不做研究。这里特别注意一点，校准是建立在用户使能了输出缓冲的情况下才有效。

59.3 DAC 的 HAL 库用法

DAC 的 HAL 库用法其实就是几个结构体变量成员的配置和使用，然后配置时钟，并根据需要配置 NVIC、中断和 DMA。下面我们逐一展开为大家做个说明。

59.3.1 DAC 寄存器结构体 DAC_TypeDef

DAC 相关的寄存器是通过 HAL 库中的结构体 DAC_TypeDef 定义的，在 stm32h743xx.h 中可以找到它们的具体定义：

```
typedef struct
{
  __IO uint32_t CR;
  __IO uint32_t SWTRIGR;
  __IO uint32_t DHR12R1;
  __IO uint32_t DHR12L1;
  __IO uint32_t DHR8R1;
  __IO uint32_t DHR12R2;
  __IO uint32_t DHR12L2;
  __IO uint32_t DHR8R2;
  __IO uint32_t DHR12RD;
  __IO uint32_t DHR12LD;
  __IO uint32_t DHR8RD;
  __IO uint32_t DOR1;
  __IO uint32_t DOR2;
  __IO uint32_t SR;
  __IO uint32_t CCR;
  __IO uint32_t MCR;
  __IO uint32_t SHSR1;
  __IO uint32_t SHSR2;
  __IO uint32_t SHHR;
  __IO uint32_t SHRR;
} DAC_TypeDef;
```

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

下面我们再看 DAC 的定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE           ((uint32_t)0x40000000)
#define D2_APB1PERIPH_BASE    PERIPH_BASE
#define DAC1_BASE             (D2_APB1PERIPH_BASE + 0x7400)
#define DAC1                  ((DAC_TypeDef *) DAC1_BASE) <----- 展开这个宏，(DAC_TypeDef *) 0x40007400
```

我们访问 DAC1 的 CR 寄存器可以采用这种形式：DAC1->CR = 0。

59.3.2 DAC 的采样保持 DAC_SampleAndHoldConfTypeDef

此结构体用于 DAC 的采样保持参数，具体定义如下：

```
typedef struct
{
  uint32_t DAC_SampleTime ;
  uint32_t DAC_HoldTime ;
  uint32_t DAC_RefreshTime ;        }
DAC_SampleAndHoldConfTypeDef;
```

下面将这几个参数逐一为大家做个说明：

- uint32_t DAC_SampleTime

  此参数用于设置 DAC 的采样时间，范围 0 - 1023。

- uint32_t DAC_HoldTime

  此参数用于设置 DAC 的保持时间，范围 0 – 1023。

- uint32_t DAC_RefreshTime

  此参数用于设置 DAC 的刷新时间，范围 0 – 255。

59.3.3 DAC 的通道参数结构体 DAC_ChannelConfTypeDef

此结构体用于 DAC 的通道参数配置，具体定义如下：

```
typedef struct
{
  uint32_t DAC_SampleAndHold;
  uint32_t DAC_Trigger;
  uint32_t DAC_OutputBuffer;
  uint32_t DAC_ConnectOnChipPeripheral ;
  uint32_t DAC_UserTrimming;
  uint32_t DAC_TrimmingValue;
  DAC_SampleAndHoldConfTypeDef  DAC_SampleAndHoldConfig;
}DAC_ChannelConfTypeDef;
```

下面将这几个参数逐一为大家做个说明：

- uint32_t DAC_SampleAndHold

  此参数用于使能采样保持模式，具体支持的参数如下：

```
#define DAC_SAMPLEANDHOLD_DISABLE     ((uint32_t)0x00000000)
#define DAC_SAMPLEANDHOLD_ENABLE      ((uint32_t)DAC_MCR_MODE1_2)
```

- uint32_t DAC_Trigger

  此参数用于 DAC 触发源的选择，具体支持的参数如下：

```
#define DAC_TRIGGER_NONE
#define DAC_TRIGGER_SOFTWARE
#define DAC_TRIGGER_T1_TRGO
#define DAC_TRIGGER_T2_TRGO
#define DAC_TRIGGER_T5_TRGO
#define DAC_TRIGGER_T6_TRGO
#define DAC_TRIGGER_T7_TRGO
#define DAC_TRIGGER_T8_TRGO
#define DAC_TRIGGER_T15_TRGO
#define DAC_TRIGGER_HR1_TRGO1
#define DAC_TRIGGER_HR1_TRGO2
#define DAC_TRIGGER_LP1_OUT
#define DAC_TRIGGER_LP2_OUT
#define DAC_TRIGGER_EXT_IT9
```

- uint32_t DAC_OutputBuffer

  此参数用于使能或者关闭 DAC 的输出缓冲，使能输出缓冲后，可以增加 DAC 的驱动能力，具体支持的参数如下：

```
#define DAC_OUTPUTBUFFER_ENABLE           ((uint32_t)0x00000000)
#define DAC_OUTPUTBUFFER_DISABLE          ((uint32_t)DAC_MCR_MODE1_1)
```

- uint32_t DAC_ConnectOnChipPeripheral

  此参数用于 DAC 是否连接片上外设（运放，比较器等），具体支持的参数如下：

```
#define DAC_CHIPCONNECT_DISABLE      ((uint32_t)0x00000000)
#define DAC_CHIPCONNECT_ENABLE       ((uint32_t)DAC_MCR_MODE1_0)
```

- uint32_t DAC_UserTrimming

  此参数用于设置 DAC 的校准方式，采用出厂模式还是用户模式，具体支持的参数如下：

```
#define DAC_TRIMMING_FACTORY         ((uint32_t)0x00000000)
#define DAC_TRIMMING_USER            ((uint32_t)0x00000001)
```

- uint32_t DAC_TrimmingValue

  此参数用于设置用户校准模式的偏移值，参数范围 1-31。

- DAC_SampleAndHoldConfTypeDef DAC_SampleAndHoldConfig

  此参数用于采样保持具体参数设置，详解本章 3.2 小节的说明。

59.3.4 DAC 结构体句柄 DAC_HandleTypeDef

HAL 库在 DAC_TypeDef 的基础上封装了一个结构体 DAC_HandleTypeDef，定义如下：

```
typedef struct
{
  DAC_TypeDef                 *Instance;
  __IO HAL_DAC_StateTypeDef   State;
  HAL_LockTypeDef             Lock;
  DMA_HandleTypeDef           *DMA_Handle1;
  DMA_HandleTypeDef           *DMA_Handle2;
  __IO uint32_t               ErrorCode;
}DAC_HandleTypeDef;
```

下面将这几个参数逐一做个说明。

- **DAC_TypeDef *Instance**

  这个参数是寄存器的例化，方便操作寄存器，详见本章 3.1 小节。

- **DMA_HandleTypeDef *DMA_Handle1**
- **DMA_HandleTypeDef *DMA_Handle2**

  DMA 句柄结构体指针变量，用于关联 DAC 句柄，方便调用。

- **HAL_LockTypeDef Lock**
- **__IO HAL_DAC_STATETypeDef State**
- **__IO uint32_t ErrorCode**

  这三个变量主要供函数内部使用。Lock 用于设置锁状态，State 用于设置 DAC 状态，而 ErrorCode 用于配置代码错误。

59.3.5 DAC 初始化流程总结

使用方法由 HAL 库提供：

- 第 1 步：基本的初始化。
  - 函数 HAL_DAC_Init 初始化。
  - 配置 DAC_OUT1: PA4, DAC_OUT2: PA5 引脚为模拟模式。
  - 函数 HAL_DAC_ConfigChannel 配置通道参数。
  - 函数 HAL_DAC_Start() or HAL_DAC_Start_DMA()使能 DAC。

- 第 2 步：DAC 校准。

  出厂校准比较简单，芯片上电后自动完成，而用户校准需要依次调用函数 HAL_DACEx_GetTrimOffset，HAL_DACEx_SelfCalibrate 和 HAL_DACEx_SetUserTrimming。

- 第 3 步：查询模式。
  - 函数 HAL_DAC_Start() 启动。
  - 函数 HAL_DAC_GetValue()可以读取输出值。
  - 函数 HAL_DAC_Stop 可以停止 DAC。

- 第 4 步：DMA 方式。
  - 函数 HAL_DAC_Start_DMA()启动 DMA 方式转换。
  - DAC 的数据传输一半的时候， HAL_DAC_ConvHalfCpltCallbackCh1() 或者 HAL_DACEx_ConvHalfCpltCallbackCh2() 会被调用。
  - DAC 的数据传输完成的时候，HAL_DAC_ConvCpltCallbackCh1() 或者 HAL_DACEx_ConvHalfCpltCallbackCh2() 会被调用。
  - 传输错误时，函数 HAL_DAC_ErrorCallbackCh1 会被调用。
  - DMA 下溢错误，会调用函数 HAL_DAC_DMAUnderrunCallbackCh1()或者 HAL_DACEx_DMAUnderrunCallbackCh2()。
  - 停止 DAC 的 DMA 方式，可以调用函数 HAL_DAC_Stop_DMA

59.4源文件 stm32h7xx_hal_dac.c

这里把我们把如下几个常用到的函数做个说明：

- HAL_DAC_Init
- HAL_DAC_ConfigChannel
- HAL_DAC_Start_DMA

59.4.1函数 HAL_DAC_Init

函数原型：

```
HAL_StatusTypeDef HAL_DAC_Init(DAC_HandleTypeDef* hdac)
{
  /* 检测 DAC 句柄 */
  if(hdac == NULL)
  {
     return HAL_ERROR;
  }
  assert_param(IS_DAC_ALL_INSTANCE(hdac->Instance));

  if(hdac->State == HAL_DAC_STATE_RESET)
  {
    hdac->Lock = HAL_UNLOCKED;
     /* 初始化 GPIO，NVIC 等 */
    HAL_DAC_MspInit(hdac);
  }

  /* 设置 DAC 状态忙 */
  hdac->State = HAL_DAC_STATE_BUSY;

  /* 设置 DAC 无错误 */
  hdac->ErrorCode = HAL_DAC_ERROR_NONE;

  /* 设置 DAC 就绪 */
  hdac->State = HAL_DAC_STATE_READY;

  /* 返回 HAL_OK */
  return HAL_OK;
}
```

函数描述：

此函数用于初始化 DAC。

函数参数：

- 第 1 个参数是 DAC_HandleTypeDef 类型结构体指针变量，结构体变量成员的详细介绍看本章 3.4 小节。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 函数 HAL_DAC_MspInit 用于初始化 DAC 的底层时钟、NVIC 等功能。需要用户自己在此函数里面实现具体的功能。由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
2. 如果形参 hdac 的结构体成员 State 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 DAC_HandleTypeDef DacHandle。

对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_DAC_STATE_RESET = 0x00U。

解决办法有三

方法 1：用户自己初始 DAC 底层。

方法 2：定义 DAC_HandleTypeDef DacHandle 为全局变量。

方法 3：下面的方法

```
if(HAL_DAC_DeInit(&DacHandle) != HAL_OK)
{
    Error_Handler();
}
if(HAL_DAC_Init(&DacHandle) != HAL_OK)
{
    Error_Handler();
}
```

使用举例：

```
DAC_HandleTypeDef    DAC_Handle;
DacHandle.Instance = DAC1;
if (HAL_DAC_Init(&DacHandle) != HAL_OK)
{
     Error_Handler(__FILE__, __LINE__);
}
```

59.4.2 函数 HAL_DAC_ConfigChannel

函数原型：

```
HAL_StatusTypeDef HAL_DAC_ConfigChannel(DAC_HandleTypeDef* hdac, DAC_ChannelConfTypeDef* sConfig, uint32_t
Channel)
{
  uint32_t tmpreg1 = 0, tmpreg2 = 0;
  uint32_t tickstart = 0;

  /* 部分省略，未贴出 */

  /* 上锁 */
  __HAL_LOCK(hdac);

  /* 设置 DAC 忙 */
  hdac->State = HAL_DAC_STATE_BUSY;

  if(sConfig->DAC_SampleAndHold == DAC_SAMPLEANDHOLD_ENABLE)
  {
    /* 通道 1 设置 */
    if (Channel == DAC_CHANNEL_1)
    {


    }
    else /* 通道 2 设置 */
    {
```

```
    }
  }

  if(sConfig->DAC_UserTrimming == DAC_TRIMMING_USER)
  /* 用户校准配置 */
  {


  }
  /* 出厂模式无需配置，复位后自动设置 */

  /* 获取 DAC MCR 数值 */
  tmpreg1 = hdac->Instance->MCR;
  /* 清除 DAC_MCR_MODE2_0, DAC_MCR_MODE2_1 和  DAC_MCR_MODE2_2 位 */
  tmpreg1 &= ~(((uint32_t)(DAC_MCR_MODE1)) << Channel);
  /* 配置 DAC 通道 */
  tmpreg2 = (sConfig->DAC_SampleAndHold | sConfig->DAC_OutputBuffer | sConfig->DAC_ConnectOnChipPeripheral);
  tmpreg1 |= tmpreg2 << Channel;
  /* 设置 MCR 数值 */
  hdac->Instance->MCR = tmpreg1;

  /* DAC 工作在正常模式 */
  CLEAR_BIT (hdac->Instance->CR, DAC_CR_CEN1 << Channel);

  /* 获取 DAC CR 值 */
  tmpreg1 = hdac->Instance->CR;
  tmpreg1 &= ~(((uint32_t)(DAC_CR_MAMP1 | DAC_CR_WAVE1 | DAC_CR_TSEL1 | DAC_CR_TEN1)) << Channel);
  tmpreg2 = (sConfig->DAC_Trigger);
  tmpreg1 |= tmpreg2 << Channel;

  /* 写 DAC CR 值 */
  hdac->Instance->CR = tmpreg1;

  /* 禁止波形生成 */
  hdac->Instance->CR &= ~(DAC_CR_WAVE1 << Channel);

  /* 设置 DAC 就绪 */
  hdac->State = HAL_DAC_STATE_READY;

  /* 解锁 */
  __HAL_UNLOCK(hdac);

  /* 返回 HAL_OK */
  return HAL_OK;
}
```

函数描述：

此函数主要用于配置 DAC 的通道参数。

函数参数：

- 第 1 个参数是 DAC_HandleTypeDef 类型结构体指针变量，结构体变量成员的详细介绍看本章 3.4 小节。
- 第 2 个参数是 DAC_ChannelConfTypeDef 类型结构体指针变量，用于 DAC 的通道参数配置，结构体变量成员的详细介绍看本章 3.3 小

- 第 3 个参数用于选择要配置那个通道，DAC_CHANNEL_1 表示配置通道 1，DAC_CHANNEL_2 表示配置通道 2。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
static DAC_ChannelConfTypeDef sConfig;
static DAC_HandleTypeDef      DacHandle;

sConfig.DAC_SampleAndHold = DAC_SAMPLEANDHOLD_DISABLE;        /* 关闭采样保持模式，这个模式主要用于低功耗 */
sConfig.DAC_Trigger = DAC_TRIGGER_T6_TRGO;                    /* 采用定时器 6 触发 */
sConfig.DAC_OutputBuffer = DAC_OUTPUTBUFFER_ENABLE;           /* 使能输出缓冲 */
sConfig.DAC_ConnectOnChipPeripheral = DAC_CHIPCONNECT_DISABLE;/* 不将 DAC 连接到片上外设 */
sConfig.DAC_UserTrimming = DAC_TRIMMING_FACTORY;              /* 使用出厂校准 */

if (HAL_DAC_ConfigChannel(&DacHandle, &sConfig, DAC_CHANNEL_1) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

59.4.3 函数 HAL_DAC_Start_DMA

函数原型：

```
HAL_StatusTypeDef HAL_DAC_Start_DMA(DAC_HandleTypeDef* hdac, uint32_t Channel, uint32_t* pData, uint32_t Length,
                                    uint32_t Alignment)
{
  uint32_t tmpreg = 0;

  /* 部分省略，未贴出 */

  /* 检测参数 Check the parameters */
  assert_param(IS_DAC_CHANNEL(Channel));
  assert_param(IS_DAC_ALIGN(Alignment));

  /* 上锁 Process locked */
  __HAL_LOCK(hdac);

  /* 设置 DAC 忙 Change DAC state */
  hdac->State = HAL_DAC_STATE_BUSY;

  /* 配置通道 1 */
  if(Channel == DAC_CHANNEL_1)
  {
    /* DMA 传输完成回调 */
    hdac->DMA_Handle1->XferCpltCallback = DAC_DMAConvCpltCh1;

    /* DMA 半传输完成回调 */
    hdac->DMA_Handle1->XferHalfCpltCallback = DAC_DMAHalfConvCpltCh1;

    /* DMA 传输错误回调 */
    hdac->DMA_Handle1->XferErrorCallback = DAC_DMAErrorCh1;
```

```
    /* 使能 DAC DMA */
    SET_BIT(hdac->Instance->CR, DAC_CR_DMAEN1);

    /* 数据对齐方式设置 */
    switch(Alignment)
    {
      case DAC_ALIGN_12B_R:
        tmpreg = (uint32_t)&hdac->Instance->DHR12R1;
        break;
      case DAC_ALIGN_12B_L:
        tmpreg = (uint32_t)&hdac->Instance->DHR12L1;
        break;
      case DAC_ALIGN_8B_R:
        tmpreg = (uint32_t)&hdac->Instance->DHR8R1;
        break;
      default:
        break;
    }
  }
  else
  {

    }
  }

  /* 使能 DMA Stream  */
  if(Channel == DAC_CHANNEL_1)
  {
    /* 使能 DAC DMA 下溢中断 */
    __HAL_DAC_ENABLE_IT(hdac, DAC_IT_DMAUDR1);

    /* 启动传输 */
    HAL_DMA_Start_IT(hdac->DMA_Handle1, (uint32_t)pData, tmpreg, Length);
  }
  else
  {
    __HAL_DAC_ENABLE_IT(hdac, DAC_IT_DMAUDR2);
    HAL_DMA_Start_IT(hdac->DMA_Handle2, (uint32_t)pData, tmpreg, Length);
  }

  /* 解锁 */
  __HAL_UNLOCK(hdac);
  /* 使能 DAC 通道 */
  __HAL_DAC_ENABLE(hdac, Channel);

  /* 返回 HAL_OK */
  return HAL_OK;
}
```

函数描述：

此函数用于启动 DAC 的 DMA 方式

函数参数：

- 第 1 个参数是 DAC_HandleTypeDef 类型结构体指针变量，结构体变量成员的详细介绍看本章 3.4 小节。

- 第 2 个参数用于选择要配置那个通道，DAC_CHANNEL_1 表示配置通道 1，DAC_CHANNEL_2 表示配置通道 2。
- 第 3 个参数是波形数据地址。
- 第 4 个参数是传输的数据长度。
- 第 5 个参数是数据对齐方式设置。
  - DAC_ALIGN_8B_R 表示 8bit 右对齐。
  - DAC_ALIGN_12B_L 表示 12bit 左对齐。
  - DAC_ALIGN_12B_R 表示 12bit 右对齐。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
static DAC_HandleTypeDef      DacHandle;

/* 启动 DAC DMA */
if (HAL_DAC_Start_DMA(&DacHandle, DAC_CHANNEL_2, (uint32_t *)g_usWaveBuff, 64, DAC_ALIGN_12B_R) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

59.5 总结

本章节就为大家讲解这么多，DAC 功能用到的地方还是比较多的，建议熟练使用。

第60章 STM32H7 的 DAC 应用之定时器触发实现 DMA 方式双通道波形

本章节为大家讲解 DAC 采用定时器触发方式实现 DMA 双通道波形输出，实际输出效果也比较好，项目使用价值也比较大。

60.1 初学者重要提示
60.2 H7 和 F4 的 DAC 输出效果对比
60.3 DAC 驱动设计
60.4 DAC 驱动移植和使用
60.5 实验例程设计框架
60.6 实验例程说明（MDK）
60.7 实验例程说明（IAR）
60.8 总结

60.1 初学者重要提示

- 学习本章节前，务必优先学习第 59 章，需要对 DAC 的基础知识有个认识。
- 开发板右上角有个跳线帽，可以让 ADC 的稳压基准接 3.3V 或者 2.5V，本章例子是接到 3.3V。
- 注意 STM32H7 只有一个 DAC，但有两个独立的通道，跟 F4 的略不同，F4 是两个 DAC。
- 如果仅使用 STM32H7 的一个通道，即 PA4 或者 PA5 引脚，另一个引脚没有做任何配置，这个引脚上会有波形效应。
- STM32H7 的 DAC 支持出厂校准和用户校准模式。特别注意一点，校准是建立在用户使能了输出缓冲的情况下才有效。
- STM32H7 的 DAC 支持正常模式和采样保持模式，其中采样保持模式用于低功耗状态使用。
- DAC 的输出除了可以连接 PA4 或者 PA5 引脚，也可以连接到片上外设，比如运放，比较器。

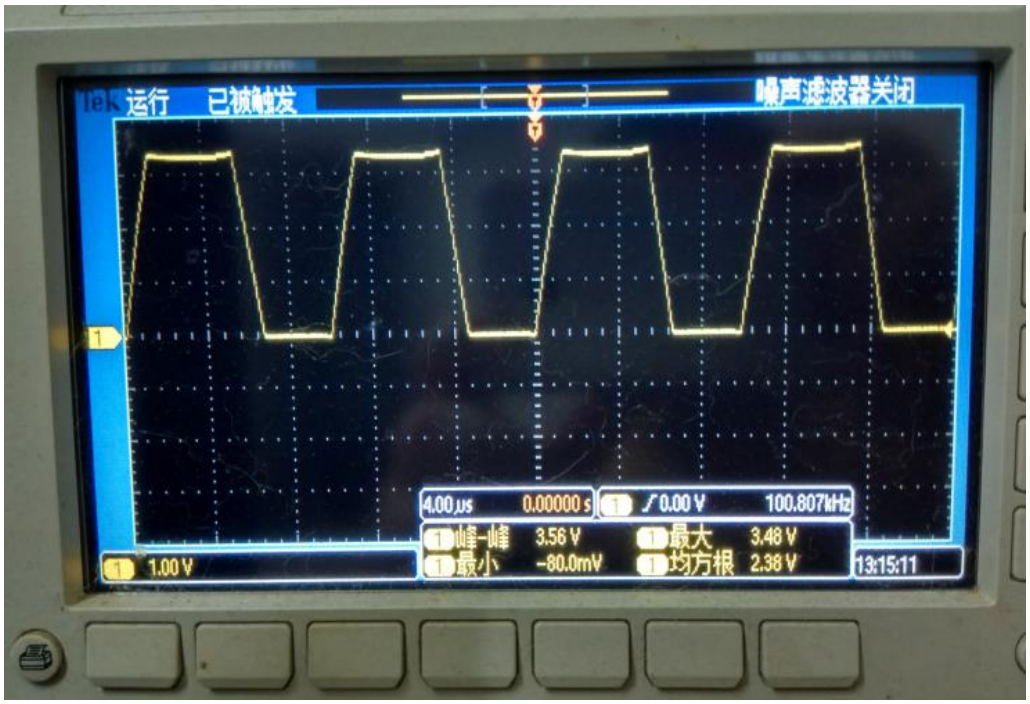
60.2 DAC 稳压基准硬件设计

详见第 46 章的第 2 小节，有详细说明，ADC 和 DAC 使用的基准源是一样的。

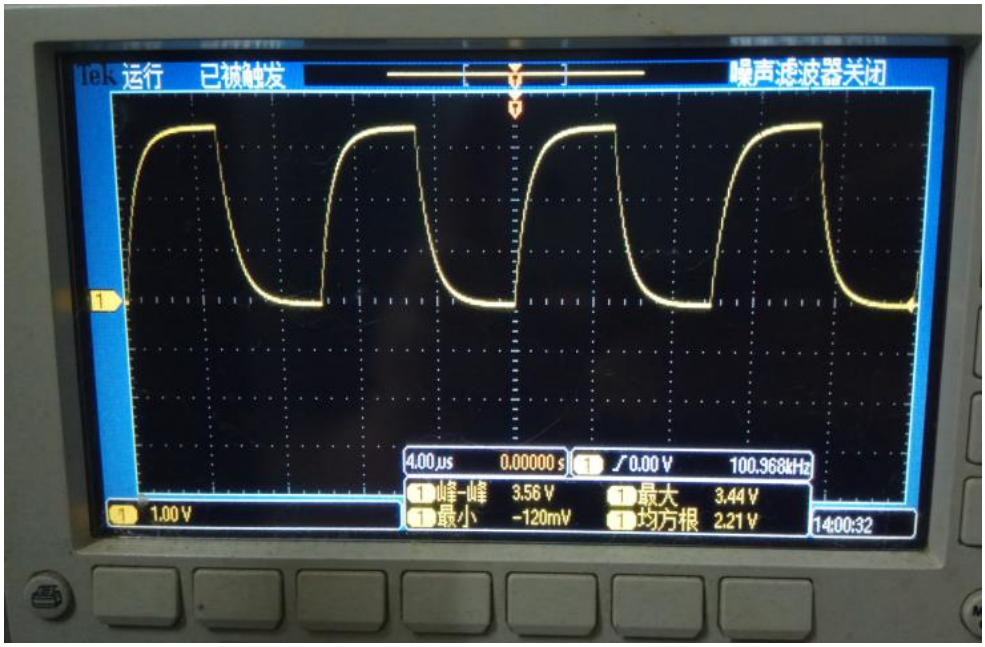
60.3 H7 和 F4 的 DAC 输出效果对比

STM32H7 的 DAC 输出 100KHz 方波的效果比 F429 好不少，满幅输出。

STM32H743 输出 100KHz 的效果如下：



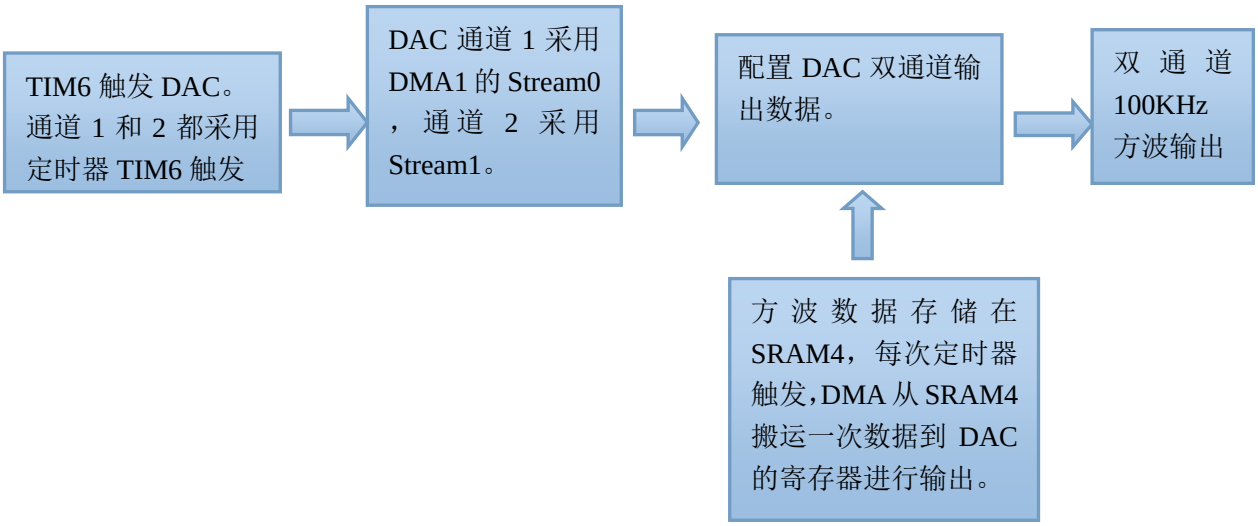
STM32F429 输出 100KHz 的效果如下：



60.4 DAC 驱动设计

60.4.1 DAC 驱动设计框架

为了方便大家理解 DAC 驱动的实现，先看下面 DAC 的驱动设计框架：DAC 做定时器触发 + DMA 数据传输的实现思路框图如下：



下面为大家讲解具体的驱动实现。

60.4.2 第 1 步：DAC 配置

DAC 的配置比较简单，仅需如下代码即可：

```
DAC_HandleTypeDef    DAC_Handle;
DacHandle.Instance = DAC1;
if (HAL_DAC_Init(&DacHandle) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

60.4.3 第 2 步：DAC 通道配置

下面是 DAC 通道 1 的配置，如果配置通道 2 的话，也是同样的方式：

```
1.  static DAC_HandleTypeDef      DacHandle;
2.  static DAC_ChannelConfTypeDef sConfig;
3.
4.  sConfig.DAC_SampleAndHold = DAC_SAMPLEANDHOLD_DISABLE;
5.  sConfig.DAC_Trigger = DAC_TRIGGER_T6_TRGO;
6.  sConfig.DAC_OutputBuffer = DAC_OUTPUTBUFFER_ENABLE;
7.  sConfig.DAC_ConnectOnChipPeripheral = DAC_CHIPCONNECT_DISABLE;
8.  sConfig.DAC_UserTrimming = DAC_TRIMMING_FACTORY;
```

```
9.
10.  if (HAL_DAC_ConfigChannel(&DacHandle, &sConfig, DAC_CHANNEL_1) != HAL_OK)
11.  {
12.      Error_Handler(__FILE__, __LINE__);
13.  }
```

下面将程序设计中几个关键地方做个阐释：

- 第 4 行，关闭采样保持模式，这个模式主要用于低功耗。
- 第 5 行，采用 TIM6 作为触发源。
- 第 6 行，使能 DAC 输出缓冲，增加驱动能力。
- 第 7 行，关闭 DAC 的输出连接片上外设，这样 DAC 的输出是连接的 PA4 或者 PA5 引脚。
- 第 8 行，采用出厂校准。
- 第 10 行，配置 DAC 的通道 1。

60.4.4第 3 步：DMA 配置

DAC 通道 1 的 DMA 配置如下，如果使用通道 2，配置是类似的，代码如下：

```
1.   static DMA_HandleTypeDef      hdma_dac1;
2.
3.   hdma_dac1.Instance = DMA1_Stream0;                 /* 使用的 DAM1 Stream0 */
4.   hdma_dac1.Init.Request  = DMA_REQUEST_DAC1;       /* DAC 触发 DMA 传输 */
5.   hdma_dac1.Init.Direction = DMA_MEMORY_TO_PERIPH;/* 存储器到外设 */
6.   hdma_dac1.Init.PeriphInc = DMA_PINC_DISABLE;     /* 外设地址禁止自增 */
7.   hdma_dac1.Init.MemInc = DMA_MINC_ENABLE;         /* 存储器地址自增 */
8.   hdma_dac1.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD; /* 外输操作数据宽度，半字 */
9.   hdma_dac1.Init.MemDataAlignment = DMA_MDATAALIGN_HALFWORD;    /* 存储器操作数据宽度，半字 */
10.  hdma_dac1.Init.Mode = DMA_CIRCULAR;                          /* 循环模式 */
11.  hdma_dac1.Init.Priority = DMA_PRIORITY_HIGH;                 /* 优先级高 */
12.
13.  HAL_DMA_Init(&hdma_dac1);
14.
15.  /* 关联 DMA 句柄到 DAC 句柄下 */
16.  __HAL_LINKDMA(&DacHandle, DMA_Handle1, hdma_dac1);
17.
18.  /* 启动 DAC DMA */
19.  if (HAL_DAC_Start_DMA(&DacHandle, DAC_CHANNEL_1, (uint32_t *)g_usWaveBuff, 64, DAC_ALIGN_12B_R) != HAL_OK)
20.  {
21.      Error_Handler(__FILE__, __LINE__);
22.  }
```

下面将程序设计中几个关键地方做个阐释：

- 第 3-11 行，配置 DAC 触发 DMA 传输，这里是采用循环模式，让 DMA 做连续的数据传输。
- 第 16 行，关联 DMA 的句柄到 DAC，方便 DAC 后期调用。此时就要特别注意，变量 hdma_dac1 如果是局部变量的话，一定要设置为静态 static，否则退出函数后，此变量会被释放掉。
- 第 19-22 行，启动 DAC 的 DMA 方式传输。

60.4.5 第 4 步：定时器触发

为了方便控制 DAC 输出波形的频率，我们采用定时器触发：

```
/*
*********************************************************************************************************
*    函 数 名: TIM6_Config
*    功能说明: 配置定时器 6，用于触发 DAC。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void TIM6_Config(void)
{
    /*----------------------------------------------------------------------
        bsp.c 文件中 void SystemClock_Config(void) 函数对时钟的配置如下:

       System Clock source       = PLL (HSE)
       SYSCLK(Hz)                = 400000000 (CPU Clock)
       HCLK(Hz)                  = 200000000 (AXI and AHBs Clock)
       AHB Prescaler             = 2
       D1 APB3 Prescaler         = 2 (APB3 Clock  100MHz)
       D2 APB1 Prescaler         = 2 (APB1 Clock  100MHz)
       D2 APB2 Prescaler         = 2 (APB2 Clock  100MHz)
       D3 APB4 Prescaler         = 2 (APB4 Clock  100MHz)

       因为 APB1 prescaler != 1, 所以 APB1 上的 TIMxCLK = APB1 x 2 = 200MHz;
       因为 APB2 prescaler != 1, 所以 APB2 上的 TIMxCLK = APB2 x 2 = 200MHz;
       APB4 上面的 TIMxCLK 没有分频，所以就是 100MHz;

       APB1 定时器有 TIM2, TIM3 ,TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14, LPTIM1
       APB2 定时器有 TIM1, TIM8 , TIM15, TIM16, TIM17

       APB4 定时器有 LPTIM2, LPTIM3, LPTIM4, LPTIM5

    TIM6 更新周期是 = TIM6CLK / （Period + 1）/（Prescaler + 1）
    根据如下的配置，更新周期是:
    TIM6CLK / （Period + 1）/（Prescaler + 1）
    = 200MHz /（30+1）/（0+1）
    ≈ 6.45MHz
    ---------------------------------------------------------------------- */
    TIM_MasterConfigTypeDef sMasterConfig;

    /* TIM6 时钟使能 */
    __HAL_RCC_TIM6_CLK_ENABLE();

    /* 配置定时器外设 */
    htim.Instance = TIM6;

    htim.Init.Period            = 30;
    htim.Init.Prescaler         = 0;
    htim.Init.ClockDivision     = 0;
    htim.Init.CounterMode       = TIM_COUNTERMODE_UP;
    htim.Init.RepetitionCounter = 0;
    HAL_TIM_Base_Init(&htim);
```

```
    /* TIM6 TRGO 选择 */
    sMasterConfig.MasterOutputTrigger = TIM_TRGO_UPDATE;
    sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;

    HAL_TIMEx_MasterConfigSynchronization(&htim, &sMasterConfig);

    /* 使能定时器 */
    HAL_TIM_Base_Start(&htim);
}
```

定时器部分在前面章节有详细介绍，这里主要是定时器触发频率：

TIM6 触发频率 = TIM6CLK / （Period + 1）/（Prescaler + 1）

根据如下的配置，触发频率为：

TIM6CLK /（Period + 1）/（Prescaler + 1）

= 200MHz /（30+1）/（0+1）

≈ 6.45MHz

TIM6 每次触发都会启动一次数据传输，通过 DMA 方式将存储器中的数据传输到 DAC 寄存器中。如此以来，比如我们设置 64 个数据为一个波形周期，那么输出波形的频率就是 6.45MHz / 64 ≈ 100KHz。

60.4.6第 5 步：波形数据生成

测试 DAC 的输出波形效果，最好的方式就是输出高频率的方波，然后查看方波的棱角是否直，如果直的话，说明高频成分越丰富，方波效果越好。

所以程序这里是直接设置 64 个点为一个周期的方波。

```
#if defined ( __ICCARM__ )
#pragma location = ".RAM_D3"
ALIGN_32BYTES(uint16_t g_usWaveBuff[64]);

#elif defined ( __CC_ARM )
ALIGN_32BYTES(__attribute__((section (".RAM_D3"))) uint16_t g_usWaveBuff[64]);
#endif

/*
*********************************************************************************************************
*    函 数 名: bsp_InitDAC
*    功能说明: DAC 初始化
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitDAC(void)
{
    uint8_t i;

    /* 一个周期的方波 */
    for(i =0; i < 32; i++)
    {
        g_usWaveBuff[i] = 0;
```

```
25.       }
26.
27.       for(i =0; i < 32; i++)
28.       {
29.            g_usWaveBuff[i+32] = 4095;
30.       }
31.
32.       DAC_WaveConfig();
33.       TIM6_Config();
34.  }
```

下面将程序设计中几个关键地方做个阐释：

- 第 2-3 行，用于 IAR 编译器，这里是在 D3 域的 SRAM4 中定义一个数组。这种定义方式在第 26 章有详细说明。另外注意，由于工程里面是将 TCM 作为主 RAM 空间，而这个空间是不支持 DMA1 和 DMA2 进行操作的，所以我们这里是在 SRAM4 中定义一个数组。

  这里还通过 ALIGN_32BYTES 做了一个 32 字节对齐，主要是方便 Cache 相关的 API 调用。原始定义如下：

```
#if defined   (__GNUC__)      /* GNU Compiler */
  #define ALIGN_32BYTES(buf)  buf __attribute__ ((aligned (32)))
#elif defined (__ICCARM__)    /* IAR Compiler */
  #define ALIGN_32BYTES(buf) _Pragma("data_alignment=32") buf
#elif defined   (__CC_ARM)    /* ARM Compiler */
  #define ALIGN_32BYTES(buf) __align(32) buf
#endif
```

- 第 6 行，同上，用于 MDK 编译器。
- 第 22-30 行，将 64 点数据一半设置为 0，一半设置为 12bit DAC 的最大值 4095。这里特别注意一点，程序里面是配置 SRAM4 的 MPU 属性为 Write through，即数据就直接写入到 SRAM4 里面，无需再调用 Cache 的 Clean 函数。

```
/* 配置 SRAM4 的 MPU 属性为 Write through, read allocate, no write allocate */
MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress      = 0x38000000;
MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

60.5 DAC 驱动移植和使用

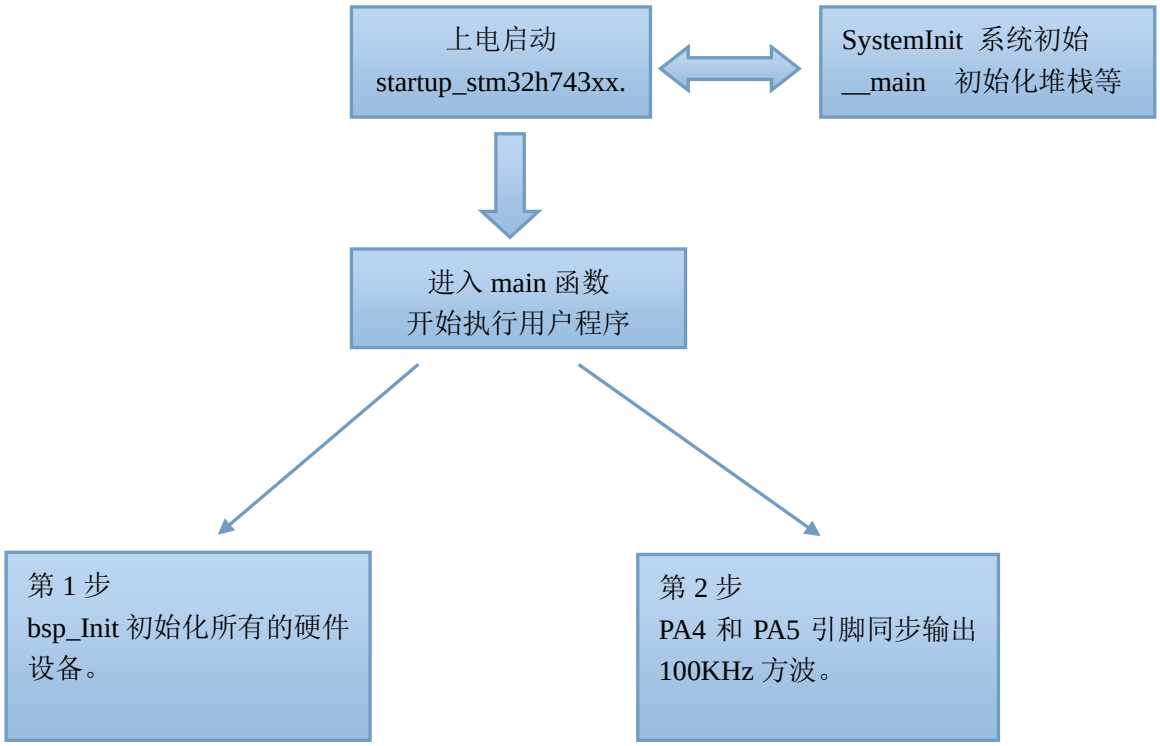
DAC 驱动的移植比较方便：

- 第 1 步：复制 bsp_dac.c 和 bsp_dac.h 到自己的工程目录，并添加到工程里面。

- 第 2 步：这几个驱动文件主要用到 HAL 库的 GPIO、TIM，DMA 和 DAC 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 3 步，应用方法看本章节配套例子即可，另外就是根据自己的需要做配置修改。

60.6 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。
- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED ，LCD，和 SDRAM。
  - 第 2 步，PA4 和 PA5 引脚同步输出 100KHz 方波。

60.7 实验例程说明（MDK）

配套例子：

V7-037_DAC 定时器触发+DMA 方式双通道同步输出

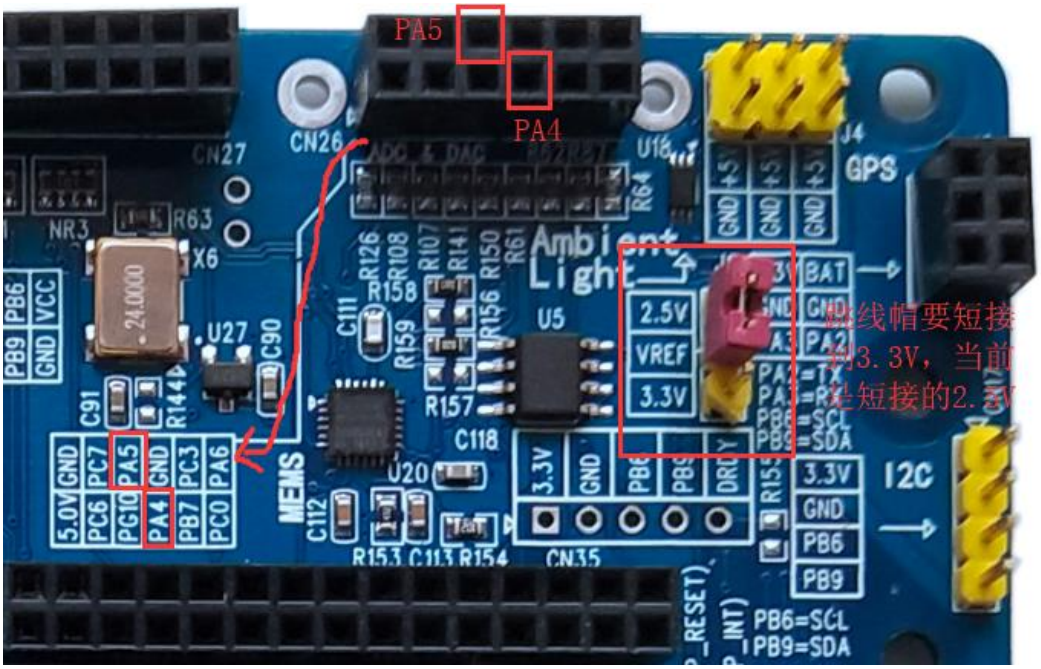
实验目的：

1. 学习 DAC 定时器触发 + DMA 方式双通道同步输出

实验内容：

2. 创建 1 个 500ms 的自动重载软定时器，每 500ms 翻转一次 LED2。
3. PA4 和 PA5 引脚输出 100KHz 的方波。

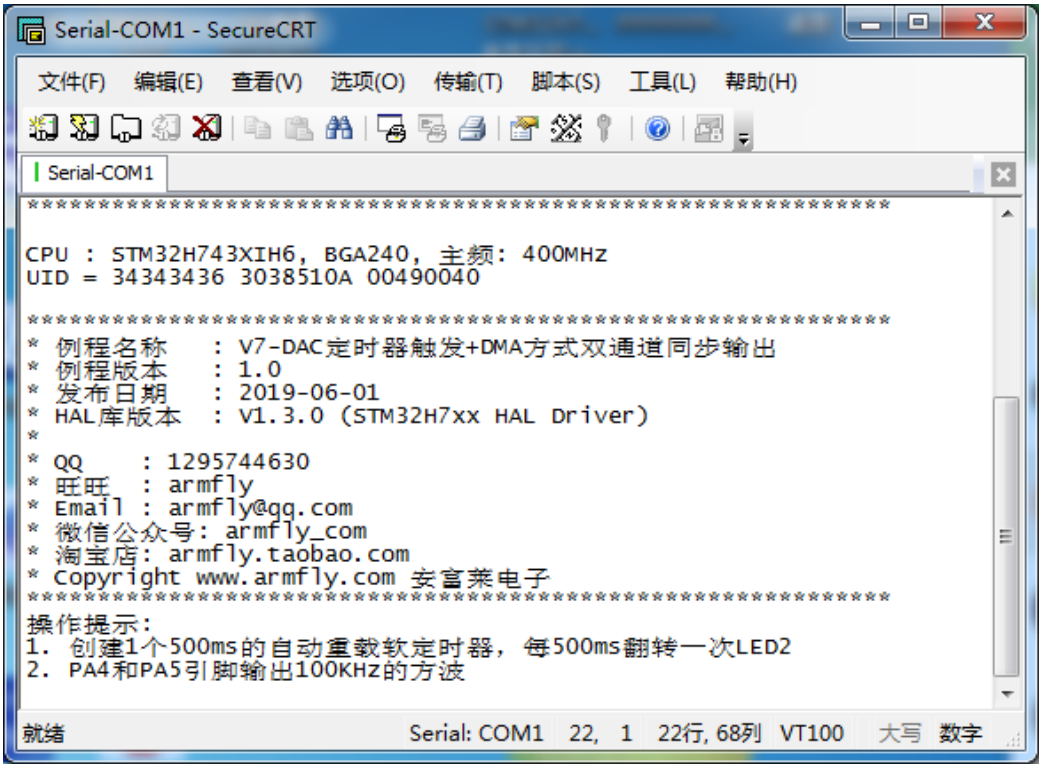
PA4 和 PA5 引脚位置（稳压基准要短接 3.3V）：



双通道 100KHz 方波效果：

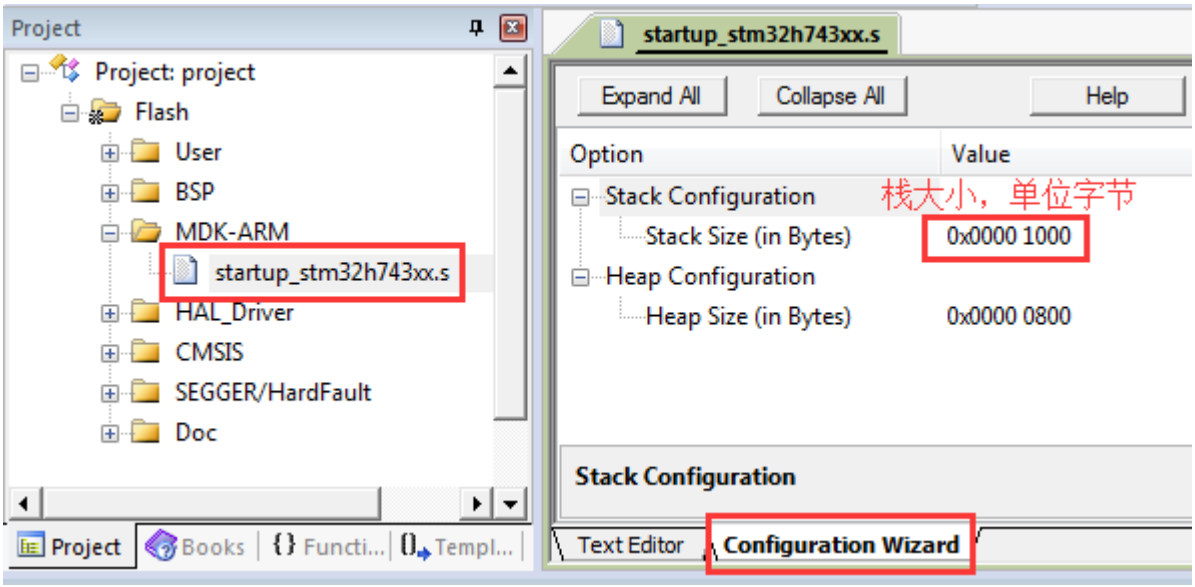
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

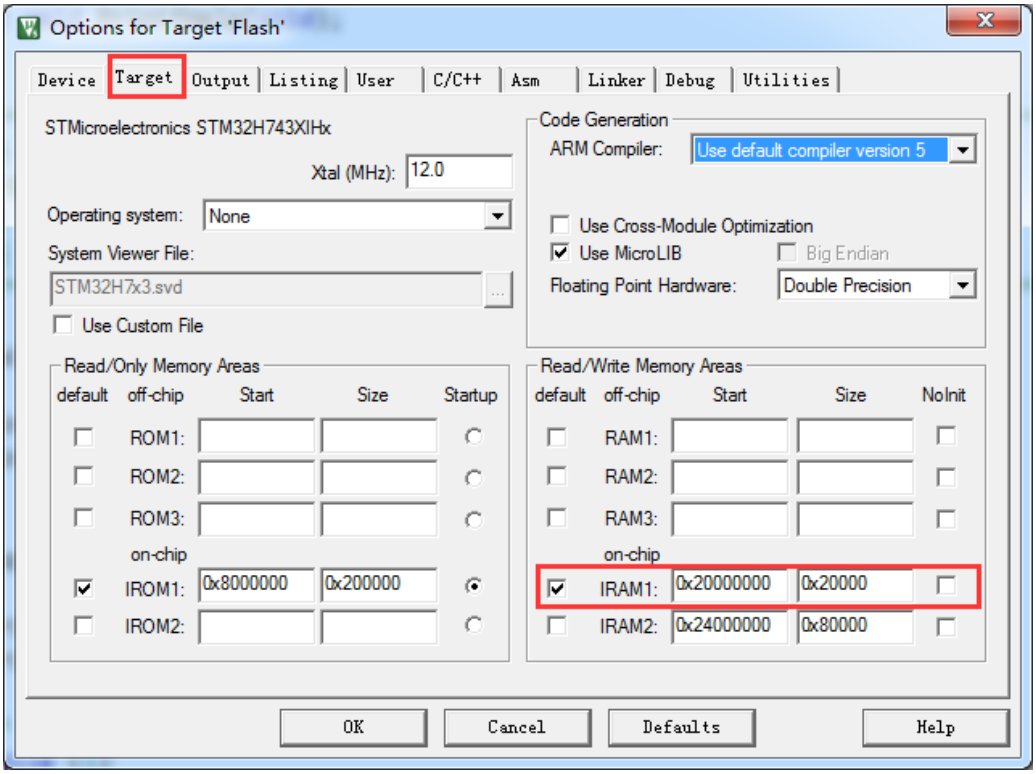


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */

    bsp_InitDAC();     /* 初始化 DAC */

}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 D3 域的 SRAM4。DAC 的数据缓存开在了 SRAM4。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
```

```
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM4 的 MPU 属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- 启动 1 个 500ms 的自动重装定时器，让 LED2 每 500ms 翻转一次。
- PA4 和 PA5 引脚输出 100KHz 的方波。

```
/*
```

```
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */

#if defined ( __CC_ARM )
    TempValues1 = 0; /* 避免 MDK 警告 */
    TempValues2 = 0;
#endif

    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 500); /* 启动 1 个 500ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:       /* K1 键按下 */
                    printf("K1 按键按下\r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

60.8 实验例程说明（IAR）

配套例子：

V7-037_DAC 定时器触发+DMA 方式双通道同步输出

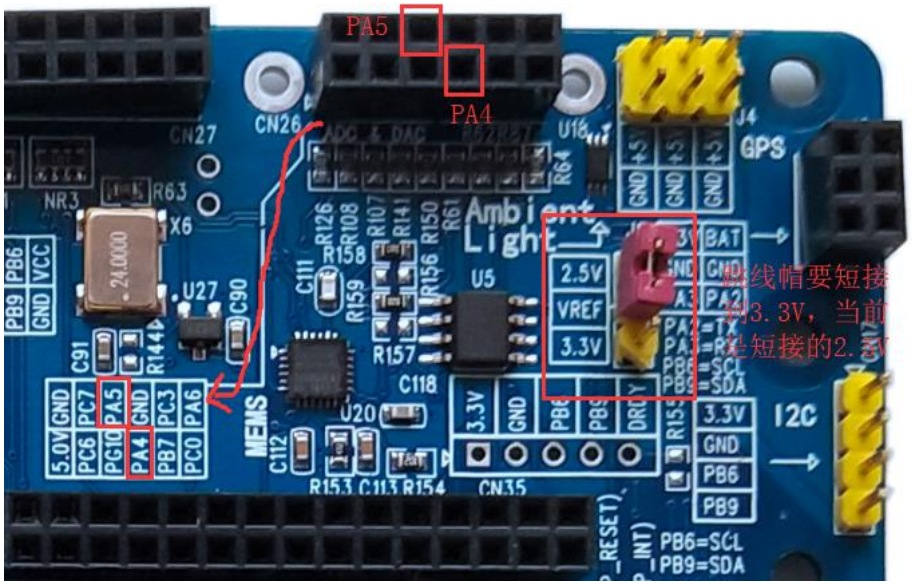
实验目的：

1. 学习 DAC 定时器触发 + DMA 方式双通道同步输出

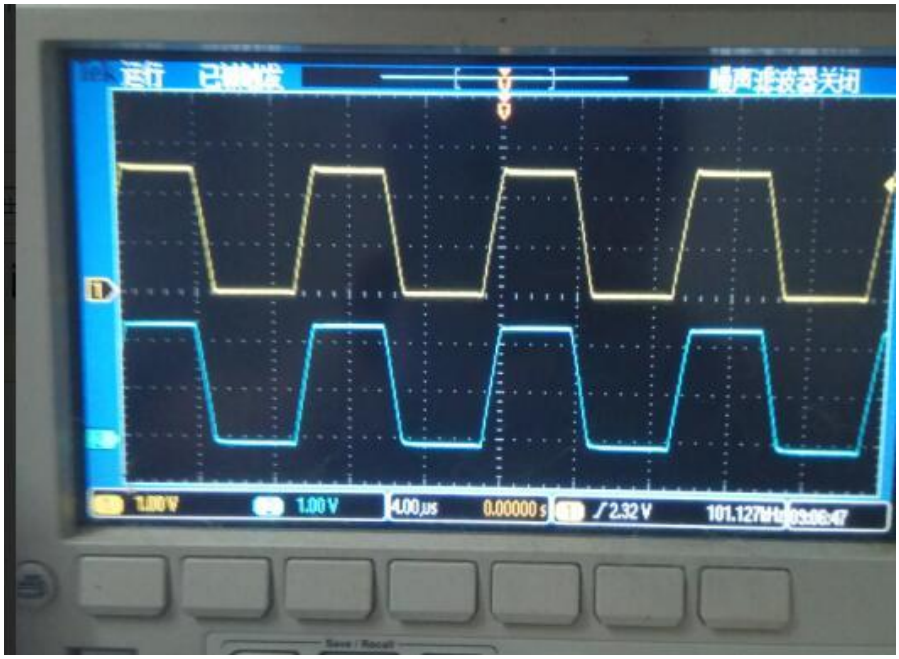
实验内容：

1. 创建 1 个 500ms 的自动重载软定时器，每 500ms 翻转一次 LED2。
2. PA4 和 PA5 引脚输出 100KHz 的方波。

PA4 和 PA5 引脚位置（稳压基准要短接 3.3V）：

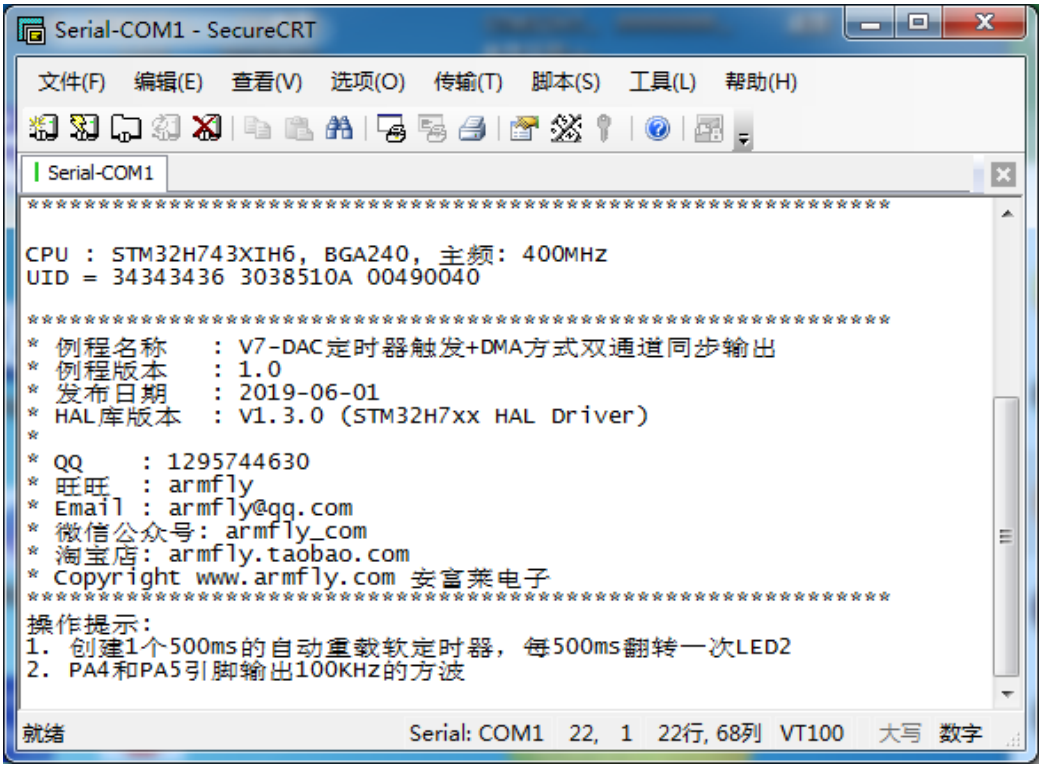


双通道 100KHz 方波效果：



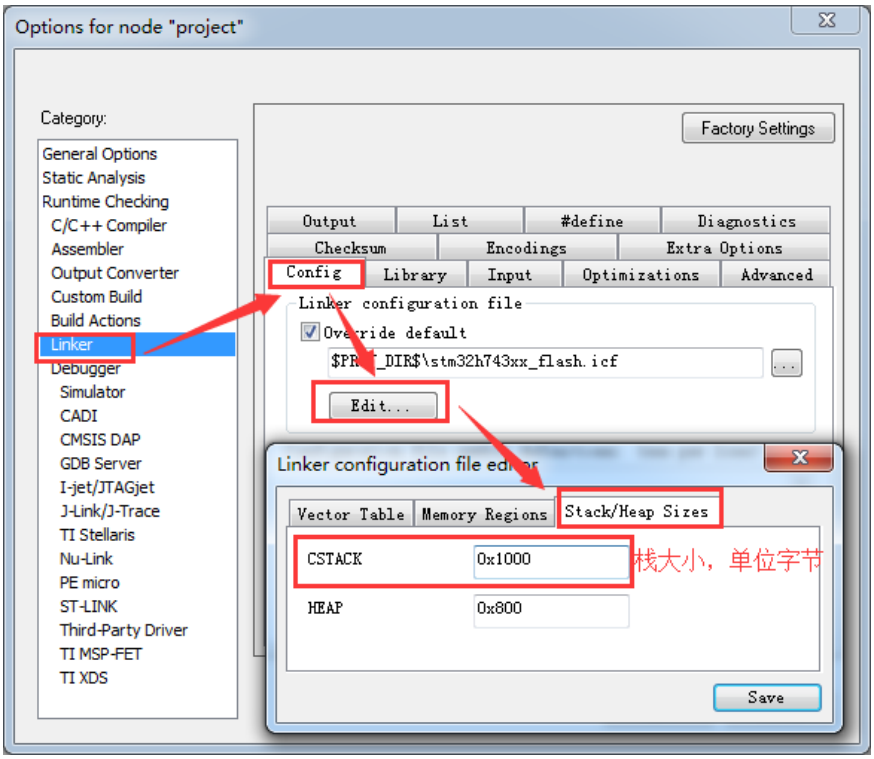
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

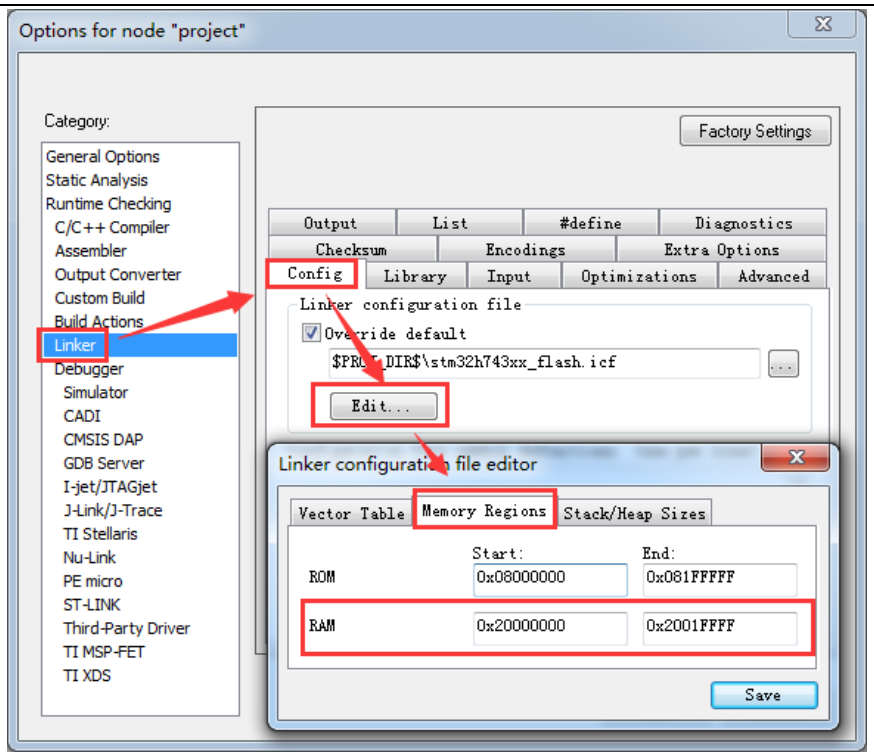


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */

    bsp_InitDAC();      /* 初始化 DAC */

}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 D3 域的 SRAM4。DAC 的数据缓存开在了 SRAM4。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
```

```
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM4 的 MPU 属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能:**

主程序实现如下操作:

- 启动 1 个 500ms 的自动重装定时器，让 LED2 每 500ms 翻转一次。
- PA4 和 PA5 引脚输出 100KHz 的方波。

```
/*
*********************************************************************************************************
*    函 数 名: main
```

```
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */

#if defined ( __CC_ARM )
    TempValues1 = 0; /* 避免 MDK 警告 */
    TempValues2 = 0;
#endif

    bsp_Init();         /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 500); /* 启动 1 个 500ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:       /* K1 键按下 */
                    printf("K1 按键按下\r\n");
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

60.9 总结

本章节涉及到的知识点比较重要，以后用到 DAC 的地方也比较多，并且 H7 的 DAC 性能比较给力。

第61章 STM32H7 的 MDMA 基础知识和 HAL 库 API

本章节为大家讲解 MDMA（Master direct memory access）基础知识，主要用于提供高速的数据传输。

61.1 初学者重要提示

61.2 MDMA 基础知识

61.3 MDMA 的 HAL 库用法

61.4 源文件 stm32h7xx_hal_mdma.c

61.5 总结

61.1 初学者重要提示

- MDMA 位于 D1 域，使用的 64 位的 AXI 总线，可以操作 TCM 空间（前面章节讲解的 DMA2D，通用 DMA 和 BDMA 不支持操作 TCM 空间）。
- 对于本章节，重点要了解 MDMA 的 buffer 缓冲传输，block 块传输和 list 列表模式的区别，详情见本章 2.2 小节。

61.2 MDMA 基础知识

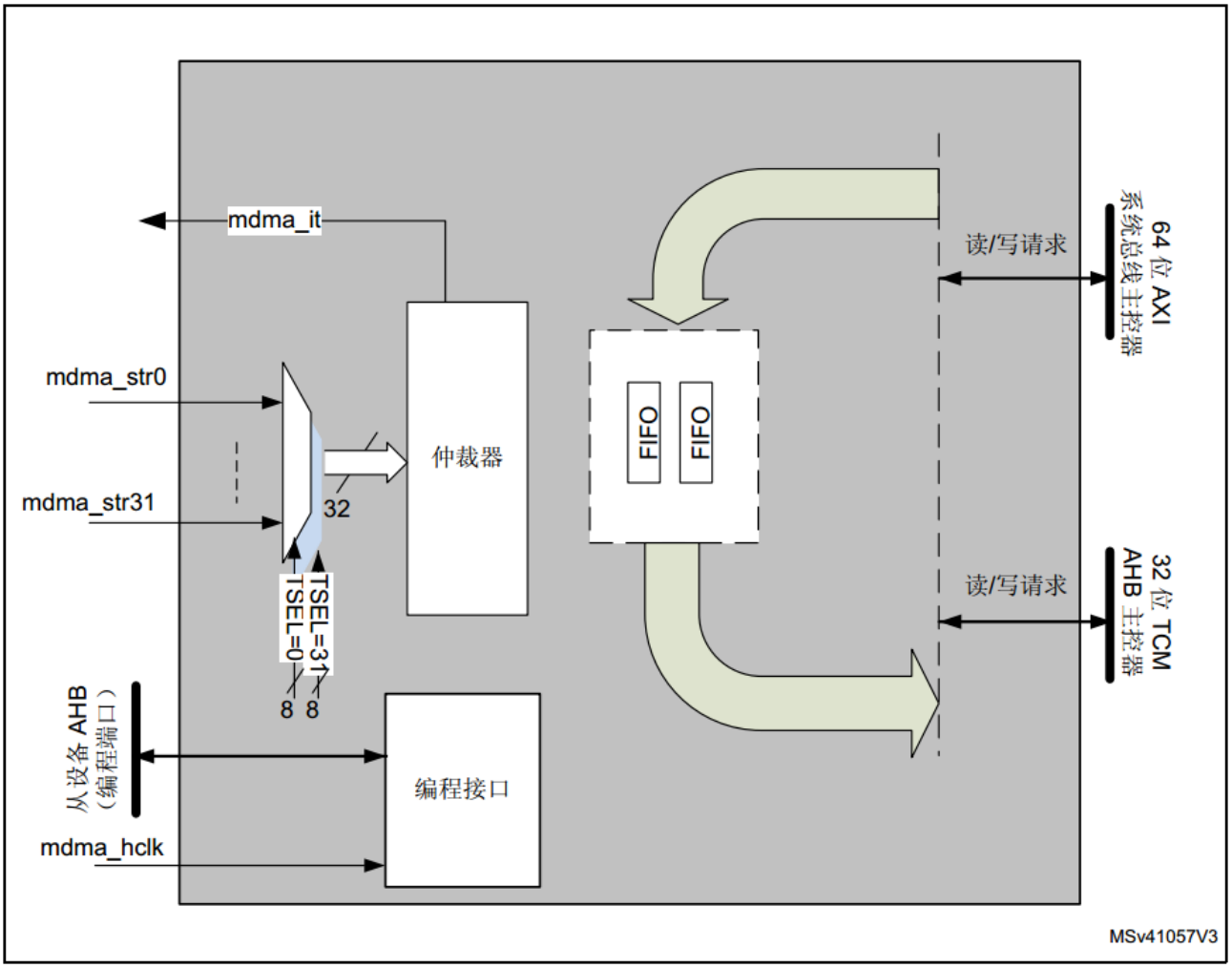
对于 STM32H7 的 MDMA 了解到以下几点即可：

- 由于总线矩阵的存在，各个主控的道路四通八达，从而可以让 DMA 和 CPU 同时开工，但是注意一点，如果他们同时访问的同一个外设，会有一点性能影响的。
- MDMA 有两个主控总线接口，一个是 AXI/AHB 总线接口，主要用存储器或者外设访问，另一个是 AHBS 总线接口，仅用于 TCM 空间访问。
- 有个 16 个通道，32 个硬件触发源。每个通道都可以选择 1 个触发源，当然，也可以通过软件触发。
- 16 个通道的传输请求，既可以外设，也可以来自 DMA1 或 DMA2
- MDMA 具有一个 256 级的 DMA 空间，被分为两个 128 级空间使用。
- MDMA 的优先级可通过软件配置，支持 very high, high, medium, low 四个等级，如果配置的优先级相同，则由 channel 的序号决定，channel0 最高，channel15 最低

- 数据宽度可以设置字节，半字，字和双字。源地址和目的地址的数据宽度可不同。
- 源地址和目标地址的大小和地址增量可以独立选择。
- 数据的打包和拆解是采用的小端格式。
- 支持突发模式，最大可传输 128 字节。
- 当源地址和目的地址的增量和数据大小相同，且位宽小于等于 32bit 时，才允许 TCM 使用突发模式。

61.2.1 MDMA 硬件框图

认识一个外设，最好的方式就是看它的框图，方便我们快速地了解 MDMA 的基本功能，然后再看手册了解细节。框图如下所示：



通过这个框图，我们可以得到如下信息：

- mdma_it

  mdma 的中断触发信号。

- mdma_str0 到 mdma_str31 触发源

  mdma 的输入请求。

- mdma_hclk

  MDMA 的 HCLK 时钟。

61.2.2 MDMA 块传输，缓冲传输和列表传输区别

初学 MDMA，要搞清除 MDMA 支持的块传输，缓冲传输和列表传输的区别。

- 缓冲传输（MDMA_BUFFER_TRANSFER）

  这个模式主要用于 QSPI，DCMI，硬件 JPEG 等外设上。每个请求都会触发 BufferTransferLength（最大 128 字节）大小的数据传输，此大小由 HAL_MDMA_Init 调用的参数配置。

- 块传输（MDMA_BLOCK_TRANSFER）

  此方式与 DMA1 和 DMA2 的数据传输相似，每次请求，触发一次块传输，块大小由 HAL_MDMA_Start/HAL_MDMA_Start_IT 定义，或者列表模式里面的参数。

- 多块传输（MDMA_REPEAT_BLOCK_TRANSFER）

  顾名思义，多块传输就是执行多次块传输，每次请求，触发多次的块传输，块大小和块数由 HAL_MDMA_Start/HAL_MDMA_Start_IT 定义，或者列表模式里面的参数。

- 列表传输（MDMA_FULL_TRANSFER）

  这种模式可以方便的实现多种 MDMA 配置进行切换，轮番实现，而且可以实现列表的循环方式。每次请求，将触发所有块和节点的传输（如果用户调用了函数 HAL_MDMA_LinkedList_CreateNode \ HAL_MDMA_LinkedList_AddNode），

61.2.3 MDMA 列表模式及其循环方式

列表模式包含多种 MDMA 的配置表，每个表包含一组完整 MDMA 配置。用户可以使用函数 HAL_MDMA_LinkedList_CreateNode 创建节点，再通过函数 HAL_MDMA_LinkedList_AddNode 将节点添加到列表里面。

- 使用列表模式的话，函数 HAL_MDMA_Init 创建的是节点 0。
- 使用函数 HAL_MDMA_LinkedList_EnableCircularMode 使能循环模式，注意是从节点 1 开始循环的，将节点 1 和末尾的节点相连，不包含 HAL_MDMA_Init 创建的节点 0。
- 节点 0 是初始配置，仅在初始化 MDMA 传输时使用一次，比如调用函数 HAL_MDMA_Start_IT。
- 如果要禁止循环模式，可以调用函数 HAL_MDMA_LinkedList_DisableCircularMode。
- 通过函数 HAL_MDMA_LinkedList_RemoveNode 还可以删除指定节点。

61.2.4 MDMA 的触发源

MDMA 支持的触发源如下，主要包含 DMA1，DMA2，DMA2D，LTDC，JPEG，QSPI，DSI，SDMMC 和软件触发：

```
#define MDMA_REQUEST_DMA1_Stream0_TC      ((uint32_t)0x00000000U)
#define MDMA_REQUEST_DMA1_Stream1_TC      ((uint32_t)0x00000001U)
#define MDMA_REQUEST_DMA1_Stream2_TC      ((uint32_t)0x00000002U)
#define MDMA_REQUEST_DMA1_Stream3_TC      ((uint32_t)0x00000003U)
#define MDMA_REQUEST_DMA1_Stream4_TC      ((uint32_t)0x00000004U)
#define MDMA_REQUEST_DMA1_Stream5_TC      ((uint32_t)0x00000005U)
#define MDMA_REQUEST_DMA1_Stream6_TC      ((uint32_t)0x00000006U)
#define MDMA_REQUEST_DMA1_Stream7_TC      ((uint32_t)0x00000007U)
#define MDMA_REQUEST_DMA2_Stream0_TC      ((uint32_t)0x00000008U)
#define MDMA_REQUEST_DMA2_Stream1_TC      ((uint32_t)0x00000009U)
#define MDMA_REQUEST_DMA2_Stream2_TC      ((uint32_t)0x0000000AU)
#define MDMA_REQUEST_DMA2_Stream3_TC      ((uint32_t)0x0000000BU)
#define MDMA_REQUEST_DMA2_Stream4_TC      ((uint32_t)0x0000000CU)
#define MDMA_REQUEST_DMA2_Stream5_TC      ((uint32_t)0x0000000DU)
#define MDMA_REQUEST_DMA2_Stream6_TC      ((uint32_t)0x0000000EU)
#define MDMA_REQUEST_DMA2_Stream7_TC      ((uint32_t)0x0000000FU)
#define MDMA_REQUEST_LTDC_LINE_IT         ((uint32_t)0x00000010U)
#define MDMA_REQUEST_JPEG_INFIFO_TH       ((uint32_t)0x00000011U)
#define MDMA_REQUEST_JPEG_INFIFO_NF       ((uint32_t)0x00000012U)
#define MDMA_REQUEST_JPEG_OUTFIFO_TH      ((uint32_t)0x00000013U)
#define MDMA_REQUEST_JPEG_OUTFIFO_NE      ((uint32_t)0x00000014U)
#define MDMA_REQUEST_JPEG_END_CONVERSION  ((uint32_t)0x00000015U)
#define MDMA_REQUEST_QUADSPI_FIFO_TH      ((uint32_t)0x00000016U)
#define MDMA_REQUEST_QUADSPI_TC           ((uint32_t)0x00000017U)
#define MDMA_REQUEST_DMA2D_CLUT_TC        ((uint32_t)0x00000018U)
#define MDMA_REQUEST_DMA2D_TC             ((uint32_t)0x00000019U)
#define MDMA_REQUEST_DMA2D_TW             ((uint32_t)0x0000001AU)

#if defined(DSI)
#define MDMA_REQUEST_DSI_TEARINGE_FFECT   ((uint32_t)0x0000001BU)
#define MDMA_REQUEST_DSI_END_REFRESH      ((uint32_t)0x0000001CU)
#endif /* DSI */

#define MDMA_REQUEST_SDMMC1_ED_DATA      ((uint32_t)0x0000001DU)

#define MDMA_REQUEST_SW                   ((uint32_t)0x40000000U)
```

61.2.5 MDMA 的软件触发方式

MDMA 配置为 MDMA_REQUEST_SW 软件触发时，可以通过函数 HAL_MDMA_GenerateSWRequest 产生触发请求，使用此函数要注意以下两个问题：

- 如果传输结束或者传输还没有启动，调用此函数会返回 error。
- 如果传输还在进行中断，调用此函数会返回 error，这次请求会被忽略。

应用示例 1：

- MDMA 配置为 MDMA_BUFFER_TRANSFER 模式，软件触发。
- 注册回调函数 HAL_MDMA_XFER_BUFFERCPLT_CB_ID。
- 调用函数 HAL_MDMA_Start_IT 会触发一次 BufferTransferLength 长度的数据传输。
- 传输结束会进入回调函数，用户可以在回调函数里面再次调用 HAL_MDMA_GenerateSWRequest 启动传输。

应用示例 2：

- MDMA 配置为 MDMA_BLOCK_TRANSFER 模式，软件触发。
- 注册回调函数 HAL_MDMA_XFER_BLOCKCPLT_CB_ID。
- 调用函数 HAL_MDMA_Start_IT 会触发一次块传输。
- 传输结束会进入回调函数，用户可以再回调函数里面再次调用 HAL_MDMA_GenerateSWRequest 再次启动传输。

61.3 MDMA 的 HAL 库用法

MDMA 的 HAL 库用法其实就是几个结构体变量成员的配置和使用，然后配置时钟，并根据需要配置 NVIC、中断。下面我们逐一展开为大家做个说明。

61.3.1 MDMA 寄存器结构体 MDMA_Channel_TypeDef 和 MDMA_TypeDef

MDMA 相关的寄存器是通过 HAL 库中的结构体 MDMA_Channel_TypeDef 和 MDMA_TypeDef 定义的，在 stm32h743xx.h 中可以找到它们的具体定义：

```
typedef struct
{
  __IO uint32_t  CISR;      /*!< MDMA channel x interrupt/status register,              Address offset: 0x40 */
  __IO uint32_t  CIFCR;     /*!< MDMA channel x interrupt flag clear register,          Address offset: 0x44 */
  __IO uint32_t  CESR;      /*!< MDMA Channel x error status register,                  Address offset: 0x48 */
  __IO uint32_t  CCR;       /*!< MDMA channel x control register,                       Address offset: 0x4C */
  __IO uint32_t  CTCR;      /*!< MDMA channel x Transfer Configuration register,        Address offset: 0x50 */
  __IO uint32_t  CBNDTR;    /*!< MDMA Channel x block number of data register,          Address offset: 0x54 */
  __IO uint32_t  CSAR;      /*!< MDMA channel x source address register,                Address offset: 0x58 */
  __IO uint32_t  CDAR;      /*!< MDMA channel x destination address register,           Address offset: 0x5C */
  __IO uint32_t  CBRUR;     /*!< MDMA channel x Block Repeat address Update register,  Address offset: 0x60 */
  __IO uint32_t  CLAR;      /*!< MDMA channel x Link Address register,                  Address offset: 0x64 */
  __IO uint32_t  CTBR;      /*!< MDMA channel x Trigger and Bus selection Register,     Address offset: 0x68 */
  uint32_t      RESERVED0;  /*!< Reserved, 0x68                                                              */
 __IO uint32_t    CMAR;      /*!< MDMA channel x Mask address register,                 Address offset: 0x70 */
 __IO uint32_t   CMDR;      /*!< MDMA channel x Mask Data register,                    Address offset: 0x74 */
}MDMA_Channel_TypeDef;

typedef struct
```

```
{
  __IO uint32_t  GISR0;   /*!< MDMA Global Interrupt/Status Register 0,          Address offset: 0x00 */
}MDMA_TypeDef;
```

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

下面我们再看 MDMA 的定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE           ((uint32_t)0x40000000)
#define D1_AHB1PERIPH_BASE    (PERIPH_BASE + 0x12000000)
#define MDMA_BASE             (D1_AHB1PERIPH_BASE + 0x0000)

#define MDMA_Channel0_BASE    (MDMA_BASE + 0x00000040)
#define MDMA_Channel1_BASE    (MDMA_BASE + 0x00000080)
#define MDMA_Channel2_BASE    (MDMA_BASE + 0x000000C0)
#define MDMA_Channel3_BASE    (MDMA_BASE + 0x00000100)
#define MDMA_Channel4_BASE    (MDMA_BASE + 0x00000140)
#define MDMA_Channel5_BASE    (MDMA_BASE + 0x00000180)
#define MDMA_Channel6_BASE    (MDMA_BASE + 0x000001C0)
#define MDMA_Channel7_BASE    (MDMA_BASE + 0x00000200)
#define MDMA_Channel8_BASE    (MDMA_BASE + 0x00000240)
#define MDMA_Channel9_BASE    (MDMA_BASE + 0x00000280)
#define MDMA_Channel10_BASE   (MDMA_BASE + 0x000002C0)
#define MDMA_Channel11_BASE   (MDMA_BASE + 0x00000300)
#define MDMA_Channel12_BASE   (MDMA_BASE + 0x00000340)
#define MDMA_Channel13_BASE   (MDMA_BASE + 0x00000380)
#define MDMA_Channel14_BASE   (MDMA_BASE + 0x000003C0)
#define MDMA_Channel15_BASE   (MDMA_BASE + 0x00000400)

#define MDMA                ((MDMA_TypeDef *)MDMA_BASE) <----- 展开这个宏，(MDMA_TypeDef *) 0x52000000
#define MDMA_Channel0       ((MDMA_Channel_TypeDef *)MDMA_Channel0_BASE)
#define MDMA_Channel1       ((MDMA_Channel_TypeDef *)MDMA_Channel1_BASE)
#define MDMA_Channel2       ((MDMA_Channel_TypeDef *)MDMA_Channel2_BASE)
#define MDMA_Channel3       ((MDMA_Channel_TypeDef *)MDMA_Channel3_BASE)
#define MDMA_Channel4       ((MDMA_Channel_TypeDef *)MDMA_Channel4_BASE)
#define MDMA_Channel5       ((MDMA_Channel_TypeDef *)MDMA_Channel5_BASE)
#define MDMA_Channel6       ((MDMA_Channel_TypeDef *)MDMA_Channel6_BASE)
#define MDMA_Channel7       ((MDMA_Channel_TypeDef *)MDMA_Channel7_BASE)
#define MDMA_Channel8       ((MDMA_Channel_TypeDef *)MDMA_Channel8_BASE)
#define MDMA_Channel9       ((MDMA_Channel_TypeDef *)MDMA_Channel9_BASE)
#define MDMA_Channel10      ((MDMA_Channel_TypeDef *)MDMA_Channel10_BASE)
#define MDMA_Channel11      ((MDMA_Channel_TypeDef *)MDMA_Channel11_BASE)
#define MDMA_Channel12      ((MDMA_Channel_TypeDef *)MDMA_Channel12_BASE)
#define MDMA_Channel13      ((MDMA_Channel_TypeDef *)MDMA_Channel13_BASE)
#define MDMA_Channel14      ((MDMA_Channel_TypeDef *)MDMA_Channel14_BASE)
#define MDMA_Channel15      ((MDMA_Channel_TypeDef *)MDMA_Channel15_BASE)
```

我们访问 MDMA 的 GISR0 寄存器可以采用这种形式：MDMA->GISR0 = 0。

61.3.2 MDMA 的参数结构体 MDMA_InitTypeDef

此结构体用于 MDMA 的参数配置，具体定义如下：

```
typedef struct
```

```
{
  uint32_t Request;
  uint32_t TransferTriggerMode;
  uint32_t Priority;
  uint32_t Endianness;
  uint32_t SourceInc;
  uint32_t DestinationInc;
  uint32_t SourceDataSize;
  uint32_t DestDataSize;
  uint32_t DataAlignment;
  uint32_t BufferTransferLength;
  uint32_t SourceBurst;
  uint32_t DestBurst;
  int32_t SourceBlockAddressOffset;
  int32_t DestBlockAddressOffset;
}MDMA_InitTypeDef;
```

下面将这几个参数逐一为大家做个说明：

- uint32_t Request

  触发 MDMA 的请求源，支持请求源如下：

```
#define MDMA_REQUEST_DMA1_Stream0_TC      ((uint32_t)0x00000000U)
#define MDMA_REQUEST_DMA1_Stream1_TC      ((uint32_t)0x00000001U)
#define MDMA_REQUEST_DMA1_Stream2_TC      ((uint32_t)0x00000002U)
#define MDMA_REQUEST_DMA1_Stream3_TC      ((uint32_t)0x00000003U)
#define MDMA_REQUEST_DMA1_Stream4_TC      ((uint32_t)0x00000004U)
#define MDMA_REQUEST_DMA1_Stream5_TC      ((uint32_t)0x00000005U)
#define MDMA_REQUEST_DMA1_Stream6_TC      ((uint32_t)0x00000006U)
#define MDMA_REQUEST_DMA1_Stream7_TC      ((uint32_t)0x00000007U)
#define MDMA_REQUEST_DMA2_Stream0_TC      ((uint32_t)0x00000008U)
#define MDMA_REQUEST_DMA2_Stream1_TC      ((uint32_t)0x00000009U)
#define MDMA_REQUEST_DMA2_Stream2_TC      ((uint32_t)0x0000000AU)
#define MDMA_REQUEST_DMA2_Stream3_TC      ((uint32_t)0x0000000BU)
 #define MDMA_REQUEST_DMA2_Stream4_TC      ((uint32_t)0x0000000CU)
#define MDMA_REQUEST_DMA2_Stream5_TC      ((uint32_t)0x0000000DU)
#define MDMA_REQUEST_DMA2_Stream6_TC      ((uint32_t)0x0000000EU)
#define MDMA_REQUEST_DMA2_Stream7_TC      ((uint32_t)0x0000000FU)
#define MDMA_REQUEST_LTDC_LINE_IT         ((uint32_t)0x00000010U)
#define MDMA_REQUEST_JPEG_INFIFO_TH       ((uint32_t)0x00000011U)
#define MDMA_REQUEST_JPEG_INFIFO_NF       ((uint32_t)0x00000012U)
#define MDMA_REQUEST_JPEG_OUTFIFO_TH      ((uint32_t)0x00000013U)
#define MDMA_REQUEST_JPEG_OUTFIFO_NE      ((uint32_t)0x00000014U)
#define MDMA_REQUEST_JPEG_END_CONVERSION  ((uint32_t)0x00000015U)
#define MDMA_REQUEST_QUADSPI_FIFO_TH      ((uint32_t)0x00000016U)
#define MDMA_REQUEST_QUADSPI_TC           ((uint32_t)0x00000017U)
#define MDMA_REQUEST_DMA2D_CLUT_TC        ((uint32_t)0x00000018U)
#define MDMA_REQUEST_DMA2D_TC             ((uint32_t)0x00000019U)
#define MDMA_REQUEST_DMA2D_TW             ((uint32_t)0x0000001AU)

#if defined(DSI)
#define MDMA_REQUEST_DSI_TEARINGE_FFECT   ((uint32_t)0x0000001BU)
#define MDMA_REQUEST_DSI_END_REFRESH      ((uint32_t)0x0000001CU)
#endif /* DSI */

#define MDMA_REQUEST_SDMMC1_END_DATA      ((uint32_t)0x0000001DU)
#define MDMA_REQUEST_SW                   ((uint32_t)0x40000000U)
```

- uint32_t TransferTriggerMode

MDMA 的传输模式，具体支持的参数如下：

```
#define MDMA_BUFFER_TRANSFER          ((uint32_t)0x00000000U)       /* 缓冲传输 */
#define MDMA_BLOCK_TRANSFER           ((uint32_t)MDMA_CTCR_TRGM_0) /* 块传输 */
#define MDMA_REPEAT_BLOCK_TRANSFER    ((uint32_t)MDMA_CTCR_TRGM_1) /* 多块传输 */
#define MDMA_FULL_TRANSFER            ((uint32_t)MDMA_CTCR_TRGM)    /* 链表传输 */
```

- uint32_t Priority

MDMA 通道的优先级设置，具体支持的参数如下：

```
#define MDMA_PRIORITY_LOW             ((uint32_t)0x00000000U)
#define MDMA_PRIORITY_MEDIUM          ((uint32_t)MDMA_CCR_PL_0)
#define MDMA_PRIORITY_HIGH            ((uint32_t)MDMA_CCR_PL_1)
#define MDMA_PRIORITY_VERY_HIGH       ((uint32_t)MDMA_CCR_PL)
```

- uint32_t Endianness

MDMA 数据传输格式，具体支持的参数如下：

```
#define MDMA_LITTLE_ENDIANNESS_PRESERVE          ((uint32_t)0x00000000U)
#define MDMA_LITTLE_BYTE_ENDIANNESS_EXCHANGE     ((uint32_t)MDMA_CCR_BEX)
#define MDMA_LITTLE_HALFWORD_ENDIANNESS_EXCHANGE ((uint32_t)MDMA_CCR_HEX)
#define MDMA_LITTLE_WORD_ENDIANNESS_EXCHANGE     ((uint32_t)MDMA_CCR_WEX)
```

MDMA_LITTLE_ENDIANNESS_PRESERVE：正常的小端格式。

MDMA_LITTLE_BYTE_ENDIANNESS_EXCHANGE：以半字为单位，交互高低字节。

MDMA_LITTLE_HALFWORD_ENDIANNESS_EXCHANGE：以字为单位，交互半字。

MDMA_LITTLE_WORD_ENDIANNESS_EXCHANGE：以双字为单位，交换字。

- uint32_t SourceInc

MDMA 源数据自增或者自减模式，具体支持的参数如下：

```
#define MDMA_SRC_INC_DISABLE       ((uint32_t)0x00000000U)
#define MDMA_SRC_INC_BYTE          ((uint32_t)MDMA_CTCR_SINC_1)
#define MDMA_SRC_INC_HALFWORD      ((uint32_t)MDMA_CTCR_SINC_1 | (uint32_t)MDMA_CTCR_SINCOS_0)
#define MDMA_SRC_INC_WORD          ((uint32_t)MDMA_CTCR_SINC_1 | (uint32_t)MDMA_CTCR_SINCOS_1)
#define MDMA_SRC_INC_DOUBLEWORD    ((uint32_t)MDMA_CTCR_SINC_1 | (uint32_t)MDMA_CTCR_SINCOS)
#define MDMA_SRC_DEC_BYTE          ((uint32_t)MDMA_CTCR_SINC)
#define MDMA_SRC_DEC_HALFWORD      ((uint32_t)MDMA_CTCR_SINC | (uint32_t)MDMA_CTCR_SINCOS_0)
#define MDMA_SRC_DEC_WORD          ((uint32_t)MDMA_CTCR_SINC | (uint32_t)MDMA_CTCR_SINCOS_1)
#define MDMA_SRC_DEC_DOUBLEWORD    ((uint32_t)MDMA_CTCR_SINC | (uint32_t)MDMA_CTCR_SINCOS)
```

- DestinationInc

MDMA 目的数据自增或者自减模式，具体支持的参数如下：

```
#define MDMA_DEST_INC_DISABLE       ((uint32_t)0x00000000U)
#define MDMA_DEST_INC_BYTE          ((uint32_t)MDMA_CTCR_DINC_1)
#define MDMA_DEST_INC_HALFWORD      ((uint32_t)MDMA_CTCR_DINC_1 | (uint32_t)MDMA_CTCR_DINCOS_0)
 #define MDMA_DEST_INC_WORD          ((uint32_t)MDMA_CTCR_DINC_1 | (uint32_t)MDMA_CTCR_DINCOS_1)
#define MDMA_DEST_INC_DOUBLEWORD    ((uint32_t)MDMA_CTCR_DINC_1 | (uint32_t)MDMA_CTCR_DINCOS)
#define MDMA_DEST_DEC_BYTE          ((uint32_t)MDMA_CTCR_DINC)
#define MDMA_DEST_DEC_HALFWORD      ((uint32_t)MDMA_CTCR_DINC | (uint32_t)MDMA_CTCR_DINCOS_0)
#define MDMA_DEST_DEC_WORD          ((uint32_t)MDMA_CTCR_DINC | (uint32_t)MDMA_CTCR_DINCOS_1)
#define MDMA_DEST_DEC_DOUBLEWORD    ((uint32_t)MDMA_CTCR_DINC | (uint32_t)MDMA_CTCR_DINCOS)
```

- SourceDataSize

MDMA 源数据带宽，具体支持的参数如下：

```
#define MDMA_SRC_DATASIZE_BYTE        ((uint32_t)0x00000000U)
#define MDMA_SRC_DATASIZE_HALFWORD    ((uint32_t)MDMA_CTCR_SSIZE_0)
#define MDMA_SRC_DATASIZE_WORD        ((uint32_t)MDMA_CTCR_SSIZE_1)
```

```
#define MDMA_SRC_DATASIZE_DOUBLEWORD  ((uint32_t)MDMA_CTCR_SSIZE)
```

- DestDataSize

  MDMA 目的数据带宽，具体支持的参数如下：

```
#define MDMA_DEST_DATASIZE_BYTE       ((uint32_t)0x00000000U)
#define MDMA_DEST_DATASIZE_HALFWORD   ((uint32_t)MDMA_CTCR_DSIZE_0)
#define MDMA_DEST_DATASIZE_WORD       ((uint32_t)MDMA_CTCR_DSIZE_1)
#define MDMA_DEST_DATASIZE_DOUBLEWORD ((uint32_t)MDMA_CTCR_DSIZE)
```

- DataAlignment

  MDMA 的填充/对齐设置，具体支持的参数如下：

```
#define MDMA_DATAALIGN_PACKENABLE       (uint32_t)MDMA_CTCR_PKE)
#define MDMA_DATAALIGN_RIGHT           ((uint32_t)0x00000000U)
#define MDMA_DATAALIGN_RIGHT_SIGNED    ((uint32_t)MDMA_CTCR_PAM_0)
#define MDMA_DATAALIGN_LEFT            ((uint32_t)MDMA_CTCR_PAM_1)
```

MDMA_DATAALIGN_PACKENABLE：

将源数据封装/解封为目标数据大小。所有数据均右对齐（采用小端模式）。

MDMA_DATAALIGN_RIGHT

右对齐，用 0 进行填充（默认值）

MDMA_DATAALIGN_RIGHT_SIGNED

右对齐，符号扩展

MDMA_DATAALIGN_LEFT

左对齐（用 0 进行填充）

- BufferTransferLength

  单次传输长度设置，范围 1-128，单位字节。

- SourceBurst

  源数据端突发设置，特别注意突发传输的数量不能超过 BufferTransferLength 参数设置的大小，具体支持的参数如下：

```
#define MDMA_SOURCE_BURST_SINGLE      ((uint32_t)0x00000000U)
#define MDMA_SOURCE_BURST_2BEATS      ((uint32_t)MDMA_CTCR_SBURST_0)
#define MDMA_SOURCE_BURST_4BEATS      ((uint32_t)MDMA_CTCR_SBURST_1)
#define MDMA_SOURCE_BURST_8BEATS      ((uint32_t)MDMA_CTCR_SBURST_0 | (uint32_t)MDMA_CTCR_SBURST_1)
#define MDMA_SOURCE_BURST_16BEATS     ((uint32_t)MDMA_CTCR_SBURST_2)
#define MDMA_SOURCE_BURST_32BEATS     ((uint32_t)MDMA_CTCR_SBURST_0 | (uint32_t)MDMA_CTCR_SBURST_2)
#define MDMA_SOURCE_BURST_64BEATS     ((uint32_t)MDMA_CTCR_SBURST_1 | (uint32_t)MDMA_CTCR_SBURST_2)
#define MDMA_SOURCE_BURST_128BEATS    ((uint32_t)MDMA_CTCR_SBURST)
```

- DestBurst

  目的数据端突发设置，特别注意突发传输的数量不能超过 BufferTransferLength 参数设置的大小，具体支持的参数如下

```
#define MDMA_DEST_BURST_SINGLE      ((uint32_t)0x00000000U)
#define MDMA_DEST_BURST_2BEATS      ((uint32_t)MDMA_CTCR_DBURST_0)
#define MDMA_DEST_BURST_4BEATS      ((uint32_t)MDMA_CTCR_DBURST_1)
#define MDMA_DEST_BURST_8BEATS      ((uint32_t)MDMA_CTCR_DBURST_0 | (uint32_t)MDMA_CTCR_DBURST_1)
#define MDMA_DEST_BURST_16BEATS     ((uint32_t)MDMA_CTCR_DBURST_2)
#define MDMA_DEST_BURST_32BEATS     ((uint32_t)MDMA_CTCR_DBURST_0 | (uint32_t)MDMA_CTCR_DBURST_2)
#define MDMA_DEST_BURST_64BEATS     ((uint32_t)MDMA_CTCR_DBURST_1 | (uint32_t)MDMA_CTCR_DBURST_2)
```

```
#define MDMA_DEST_BURST_128BEATS       ((uint32_t)MDMA_CTCR_DBURST)
```

- SourceBlockAddressOffset

  用于设置源地址端数据偏移。

- DestBlockAddressOffset

  用于设置目的地址端数据偏移。

61.3.3 MDMA 结构体句柄 MDMA_HandleTypeDef

HAL 库在 MDMA_Channel_TypeDef 和 MDMA_InitTypeDef 的基础上封装了一个结构体 MDMA_HandleTypeDef，定义如下：

```
typedef struct __MDMA_HandleTypeDef
{
  MDMA_Channel_TypeDef *Instance;
  MDMA_InitTypeDef      Init;
  HAL_LockTypeDef       Lock;
  __IO HAL_MDMA_StateTypeDef  State;
  void                  *Parent;
  void                  (* XferCpltCallback)( struct __MDMA_HandleTypeDef * hmdma);
  void                  (* XferBufferCpltCallback)( struct __MDMA_HandleTypeDef * hmdma);
  void                  (* XferBlockCpltCallback)( struct __MDMA_HandleTypeDef * hmdma);
  void                  (* XferRepeatBlockCpltCallback)( struct __MDMA_HandleTypeDef * hmdma);
  void                  (* XferErrorCallback)( struct __MDMA_HandleTypeDef * hmdma);
  void                  (* XferAbortCallback)( struct __MDMA_HandleTypeDef * hmdma);
  MDMA_LinkNodeTypeDef *FirstLinkedListNodeAddress;
  MDMA_LinkNodeTypeDef *LastLinkedListNodeAddress;
  uint32_t LinkedListNodeCounter;
  __IO uint32_t          ErrorCode;
} MDMA_HandleTypeDef;
```

下面将这几个参数逐一做个说明。

- **MDMA_Channel_TypeDef *Instance**

  这个参数 MDMA 通道的例化，方便操作寄存器，详见本章 3.1 小节。

- **MDMA_InitTypeDef Init**

  这个参数用于 MDMA 的通道参数初始化，详见本章 3.2 小节。

- **void *Parent**

  用于记录 MDMA 关联的外设。

- **void (* XferCpltCallback)(struct __MDMA_HandleTypeDef * hmdma);**
- **void (* XferBufferCpltCallback)(struct __MDMA_HandleTypeDef * hmdma);**
- **void (* XferBlockCpltCallback)(struct __MDMA_HandleTypeDef * hmdma);**
- **void (* XferRepeatBlockCpltCallback)(struct __MDMA_HandleTypeDef * hmdma);**
- **void (* XferErrorCallback)(struct __MDMA_HandleTypeDef * hmdma);**
- **void (* XferAbortCallback)(struct __MDMA_HandleTypeDef * hmdma);**

  MDMA 的各种回调函数。

- **MDMA_LinkNodeTypeDef *FirstLinkedListNodeAddress;**
- **MDMA_LinkNodeTypeDef *LastLinkedListNodeAddress;**
- **uint32_t LinkedListNodeCounter;**

  用于 MDMA 的列表模式。
- **HAL_LockTypeDef Lock**
- **__IO HAL_DAC_STATETypeDef State**
- **__IO uint32_t ErrorCode**

  这三个变量主要供函数内部使用。Lock 用于设置锁状态，State 用于设置 DAC 状态，而 ErrorCode 用于配置代码错误。

61.3.4MDMA 初始化流程总结

使用方法由 HAL 库提供：

- 第 1 步：基本的初始化。
  - 函数 HAL_MDMA_Init 配置 MDMA 的基本参数。
- 第 2 步：列表模式。

  如果使用列表模式，用户可以使用函数 HAL_MDMA_LinkedList_CreateNode 创建节点，再通过函数 HAL_MDMA_LinkedList_AddNode 将节点添加到列表里面。
- 第 3 步：查询模式。
  - 函数 HAL_MDMA_Start 启动传输。
  - 函数 HAL_MDMA_PollForTransfer 查询传输完成。
  - 函数 HAL_MDMA_Abort 终止传输。
- 第 4 步：中断方式。
  - 函数 HAL_NVIC_SetPriority 设置 MDMA 中断优先级。
  - 函数 HAL_NVIC_EnableIRQ 使能中断。
  - 函数 HAL_MDMA_Start_IT 启动中断传输。
  - MDMA 的中断服务程序 MDMA_IRQHandler 里面调用 HAL_MDMA_IRQHandler，如果用户注册了各种回调函数，会在此函数里面执行。
  - 函数 HAL_MDMA_Abort_IT 可以终止 MDMA 中断传输，终止完成后，会调用回调函数 XferAbortCallback（如果设置了的话）
- 第 5 步：中断回调函数。

函数 HAL_MDMA_RegisterCallback 注册回调函数，函数 HAL_MDMA_UnRegisterCallback 取消注册回调函数。

- XferCpltCallback ：传输完成回调。
- XferBufferCpltCallback ：buffer 传输完成回调。
- XferBlockCpltCallback ：block 传输完成回调。
- XferRepeatBlockCpltCallback : repeated block 传输完成回调。
- XferErrorCallback ：传输错误回调。
- XferAbortCallback ：传输终止回调。

61.4 源文件 stm32h7xx_hal_mdma.c

这里把我们把如下几个常用到的函数做个说明：

- HAL_MDMA_Init
- HAL_MDMA_Start_IT

61.4.1 函数 HAL_MDMA_Init

函数原型：

```
HAL_StatusTypeDef HAL_MDMA_Init(MDMA_HandleTypeDef *hmdma)
{
  uint32_t tickstart = HAL_GetTick();

  /* 检测句柄是否有效 */
  if(hmdma == NULL)
  {
    return HAL_ERROR;
  }

  /* 检测参数 */
  assert_param(IS_MDMA_STREAM_ALL_INSTANCE(hmdma->Instance));
  assert_param(IS_MDMA_PRIORITY(hmdma->Init.Priority));
  assert_param(IS_MDMA_ENDIANNESS_MODE(hmdma->Init.Endianness));
  assert_param(IS_MDMA_REQUEST(hmdma->Init.Request));
  assert_param(IS_MDMA_SOURCE_INC(hmdma->Init.SourceInc));
  assert_param(IS_MDMA_DESTINATION_INC(hmdma->Init.DestinationInc));
  assert_param(IS_MDMA_SOURCE_DATASIZE(hmdma->Init.SourceDataSize));
  assert_param(IS_MDMA_DESTINATION_DATASIZE(hmdma->Init.DestDataSize));
  assert_param(IS_MDMA_DATA_ALIGNMENT(hmdma->Init.DataAlignment));
  assert_param(IS_MDMA_SOURCE_BURST(hmdma->Init.SourceBurst));
  assert_param(IS_MDMA_DESTINATION_BURST(hmdma->Init.DestBurst));
  assert_param(IS_MDMA_BUFFER_TRANSFER_LENGTH(hmdma->Init.BufferTransferLength));
  assert_param(IS_MDMA_TRANSFER_TRIGGER_MODE(hmdma->Init.TransferTriggerMode));
  assert_param(IS_MDMA_BLOCK_ADDR_OFFSET(hmdma->Init.SourceBlockAddressOffset));
  assert_param(IS_MDMA_BLOCK_ADDR_OFFSET(hmdma->Init.DestBlockAddressOffset));
```

```
  /* 解锁 */
  __HAL_UNLOCK(hmdma);

  /* Change MDMA peripheral state */
  hmdma->State = HAL_MDMA_STATE_BUSY;

  /* 禁止 MDMA */
  __HAL_MDMA_DISABLE(hmdma);

  /* 等待禁止成功 */
  while((hmdma->Instance->CCR & MDMA_CCR_EN) != RESET)
  {
    if((HAL_GetTick() - tickstart ) > HAL_TIMEOUT_MDMA_ABORT)
    {
      hmdma->ErrorCode = HAL_MDMA_ERROR_TIMEOUT;

      hmdma->State = HAL_MDMA_STATE_ERROR;

      return HAL_ERROR;
    }
  }

  /* 初始化 MDMA  */
  MDMA_Init(hmdma);

  /* 复位列表模式相关设置 */
  hmdma->FirstLinkedListNodeAddress  = 0;
  hmdma->LastLinkedListNodeAddress   = 0;
  hmdma->LinkedListNodeCounter  = 0;

  /* 设置无错误 */
  hmdma->ErrorCode = HAL_MDMA_ERROR_NONE;

  /* 设置就绪 */
  hmdma->State = HAL_MDMA_STATE_READY;

  return HAL_OK;
}
```

函数描述：

此函数用于初始化 MDMA。

函数参数：

- 第 1 个参数是 MDMA_HandleTypeDef 类型结构体指针变量，结构体变量成员的详细介绍看本章 3.3 小节。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
MDMA_HandleTypeDef  MDMA_Handle;

__HAL_RCC_MDMA_CLK_ENABLE();

MDMA_Handle.Instance = MDMA_Channel0;
```

```
MDMA_Handle.Init.Request              = MDMA_REQUEST_SW;          /* 软件触发 */
MDMA_Handle.Init.TransferTriggerMode  = MDMA_BLOCK_TRANSFER;      /* 块传输 */
MDMA_Handle.Init.Priority             = MDMA_PRIORITY_HIGH;       /* 优先级高*/
MDMA_Handle.Init.Endianness           = MDMA_LITTLE_ENDIANNESS_PRESERVE; /* 小端 */
MDMA_Handle.Init.SourceInc            = MDMA_SRC_INC_DOUBLEWORD;          /* 源地址自增，双字，即 8 字节 */
MDMA_Handle.Init.DestinationInc       = MDMA_DEST_INC_DOUBLEWORD;         /* 目的地址自增，双字，即 8 字节 */
MDMA_Handle.Init.SourceDataSize       = MDMA_SRC_DATASIZE_DOUBLEWORD;     /* 源地址数据宽度双字，即 8 字节 */
MDMA_Handle.Init.DestDataSize         =MDMA_DEST_DATASIZE_DOUBLEWORD;    /* 目的地址数据宽度双字,即8字节 */
MDMA_Handle.Init.DataAlignment        = MDMA_DATAALIGN_PACKENABLE;        /* 小端，右对齐 */
MDMA_Handle.Init.SourceBurst          = MDMA_SOURCE_BURST_16BEATS;       /* 源数据突发传输 */
MDMA_Handle.Init.DestBurst            = MDMA_DEST_BURST_16BEATS;         /* 目的数据突发传输 */

MDMA_Handle.Init.BufferTransferLength = 128;    /* 每次传输 128 个字节 */

MDMA_Handle.Init.SourceBlockAddressOffset  = 0; /* 用于 block 传输，地址偏移 0 */
MDMA_Handle.Init.DestBlockAddressOffset    = 0; /* 用于 block 传输，地址偏移 0 */

/* 初始化 MDMA */
if(HAL_MDMA_Init(&MDMA_Handle) != HAL_OK)
{
      Error_Handler(__FILE__, __LINE__);
}
```

61.4.2 函数 HAL_MDMA_Start_IT

函数原型：

```
HAL_StatusTypeDef HAL_MDMA_Start_IT(MDMA_HandleTypeDef *hmdma, uint32_t SrcAddress, uint32_t DstAddress,
uint32_t BlockDataLength, uint32_t BlockCount)
{
  /* 检测参数 */
  assert_param(IS_MDMA_TRANSFER_LENGTH(BlockDataLength));
  assert_param(IS_MDMA_BLOCK_COUNT(BlockCount));

  /* 检测句柄是否有效 */
  if(hmdma == NULL)
  {
    return HAL_ERROR;
  }

  /* 上锁 */
  __HAL_LOCK(hmdma);

  if(HAL_MDMA_STATE_READY == hmdma->State)
  {
    hmdma->State = HAL_MDMA_STATE_BUSY;

    hmdma->ErrorCode = HAL_MDMA_ERROR_NONE;

    __HAL_MDMA_DISABLE(hmdma);

    /* 配置 MDMA */
    MDMA_SetConfig(hmdma, SrcAddress, DstAddress, BlockDataLength, BlockCount);

    /* 使能传输错误和传输完成中断 */
```

```
    __HAL_MDMA_ENABLE_IT(hmdma, (MDMA_IT_TE | MDMA_IT_CTC));

    if(hmdma->XferBlockCpltCallback != NULL)
    {
      /* 使能块传输完成中断 */
      __HAL_MDMA_ENABLE_IT(hmdma, MDMA_IT_BT);
    }

    if(hmdma->XferRepeatBlockCpltCallback != NULL)
    {
      /* 使能多块传输完成中断 */
      __HAL_MDMA_ENABLE_IT(hmdma, MDMA_IT_BRT);
    }

    if(hmdma->XferBufferCpltCallback != NULL)
    {
      /* 使能传输完成中断 */
      __HAL_MDMA_ENABLE_IT(hmdma, MDMA_IT_BFTC);
    }

    /* 使能外设 */
    __HAL_MDMA_ENABLE(hmdma);

    if(hmdma->Init.Request == MDMA_REQUEST_SW)
    {
      /* 使能软件请求 */
      hmdma->Instance->CCR |=  MDMA_CCR_SWRQ;
    }
  }
  else
  {
    /* 解锁 */
    __HAL_UNLOCK(hmdma);

    /* 返回忙 */
    return HAL_BUSY;
  }

  return HAL_OK;
}
```

函数描述：

此函数主要用于配置 MDMA 的中断方式传输。

函数参数：

- 第 1 个参数是 MDMA_HandleTypeDef 类型结构体指针变量，结构体变量成员的详细介绍看本章 3.3 小节。
- 第 2 个参数是源缓冲地址。
- 第 3 个参数是目的地址。
- 第 4 个参数是传输的块大小，单位字节，范围不可以超过 65536 字节。
- 第 5 个参数是传输的块数，范围不可以超过 4095 块。

- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
/* 将地址 0x24000000 开始的 64KB 数据复制到地址 0x24000000 + 64*1024 */
HAL_MDMA_Start_IT(&MDMA_Handle,
                  (uint32_t)0x24000000,
                  (uint32_t)(0x24000000 + 64*1024),
                  64*1024,
                  1);
```

61.5 总结

本章节就为大家讲解这么多，MDMA 功能用到的地方还是比较多的，后面的章节会为大家做讲解。

第62章 STM32H7 的 MDMA，DMA2D 和通用 DMA 性能比较

本章节为大家比较 MDMA，DMA2D 和通用 DMA 的性能，方便大家在实际应用中选择合适的 DMA 方式。

62.1 初学者重要提示
62.2 测试条件说明
62.3 MDMA 性能测试程序设计
62.4 DMA2D 性能测试程序设计
62.5 通用 DMA 性能测试程序设计
62.6 MDMA、DMA2D 和通用 DMA 性能比较
62.7 MDMA 驱动移植和使用
62.8 实验例程设计框架
62.9 实验例程说明（MDK）
62.10 实验例程说明（IAR）
62.11 总结

62.1 初学者重要提示

- 学习本章节前，务必优先学习第 61 章，需要对 MDMA 的基础知识有个认识。
- 官方各种 MDMA 例子简易分析，方便大家更好的了解 MDMA 应用场景：http://www.armbbs.cn/forum.php?mod=viewthread&tid=88905 。
- 合理配置 STM32H7 的 MDMA 突发传输次数和源数据以及目的数据位宽可以再提升一点性能 http://www.armbbs.cn/forum.php?mod=viewthread&tid=94071。

62.2 测试条件说明

MDMA，DMA2D 和每个都测试了四种情况

- 64 位带宽的 AXI SRAM 内部做 64KB 数据传输。
- 32 位带宽的 D2 域 SRAM1 内部 64KB 数据传输。
- AXI SRAM 向 SDRAM 传输 64KB 的数据传输。

◆ 32 位带宽的 SDRAM 内部做 64KB 数据传输。

MDMA：

在 D1 域，支持 64 位带宽的 DMA 数据传输。

DMA2D：

在 D1 域，主要用图形 2D 加速。

DMA1 和 DMA2：

在 D2 域，支持 32 位带宽的 DMA 数据传输。

62.3 MDMA 性能测试程序设计

这里将 MDMA 的程序设计分为以下几部分，逐一为大家做个说明：

62.3.1 第 1 步，MDMA 初始化

程序代码如下，采用块传输，源地址和目的地址都是 64bit 数据传输，并设置 16beat 突发，也就是连续传输 16 组 64bit 数据。

```
1.  __HAL_RCC_MDMA_CLK_ENABLE();
2.
3.  MDMA_Handle.Instance = MDMA_Channel0;
4.
5.  MDMA_Handle.Init.Request              = MDMA_REQUEST_SW;          /* 软件触发 */
6.  MDMA_Handle.Init.TransferTriggerMode  = MDMA_BLOCK_TRANSFER;      /* 块传输 */
7.  MDMA_Handle.Init.Priority             = MDMA_PRIORITY_HIGH;       /* 优先级高*/
8.  MDMA_Handle.Init.Endianness           = MDMA_LITTLE_ENDIANNESS_PRESERVE; /* 小端 */
9.  MDMA_Handle.Init.SourceInc            = MDMA_SRC_INC_DOUBLEWORD;       /* 源地址自增，双字，即 8 字节 */
10. MDMA_Handle.Init.DestinationInc       =MDMA_DEST_INC_DOUBLEWORD;      /* 目的地址自增，双字，即 8 字节 */
11. MDMA_Handle.Init.SourceDataSize       =MDMA_SRC_DATASIZE_DOUBLEWORD; /* 源地址数据宽度双字，即 8 字节 */
12. MDMA_Handle.Init.DestDataSize        =MDMA_DEST_DATASIZE_DOUBLEWORD;/* 目的地址数据宽度双字,即 8 字节 */
13. MDMA_Handle.Init.DataAlignment       = MDMA_DATAALIGN_PACKENABLE;     /* 小端，右对齐 */
14. MDMA_Handle.Init.SourceBurst         = MDMA_SOURCE_BURST_16BEATS;     /* 源数据突发传输 */
15. MDMA_Handle.Init.DestBurst           = MDMA_DEST_BURST_16BEATS;       /* 目的数据突发传输 */
16.
17. MDMA_Handle.Init.BufferTransferLength = 128;     /* 每次传输 128 个字节 */
18.
19. MDMA_Handle.Init.SourceBlockAddressOffset  = 0; /* 用于 block 传输，地址偏移 0 */
20. MDMA_Handle.Init.DestBlockAddressOffset    = 0; /* 用于 block 传输，地址偏移 0 */
21.
22. /* 初始化 MDMA */
23. if(HAL_MDMA_Init(&MDMA_Handle) != HAL_OK)
24. {
25.       Error_Handler(__FILE__, __LINE__);
26. }
```

下面将程序设计中几个关键地方做个阐释：

- 第 1 行,务必优先初始化 MDMA 时钟,测试发现没有使能时钟的情况下就配置 MDMA 很容易失败。
- 第 14-15 行，突发传输的配置非常考究，每次突发传输的总数据大小不能超过 128 字节。
  - 对于源地址就是 SourceBurst * SourceDataSize <= BufferTransferLength。
  - 对于目的地址就是 DestBurst*DestDataSize <= BufferTransferLength。

比如当前的程序配置：

SourceBurst * SourceDataSize = 16*8 =128 字节

DestBurst*DestDataSize = 16*8 =128 字节

这里要特别注意一点，如果实际应用中最好小于 BufferTransferLength，防止不稳定。

62.3.2第 2 步，MDMA 中断配置

MDMA 的中断设置比较简单，代码如下，注册了 MDMA 的传输完成回调：

```
HAL_MDMA_RegisterCallback(&MDMA_Handle, HAL_MDMA_XFER_CPLT_CB_ID, MDMA_TransferCompleteCallback);
HAL_NVIC_SetPriority(MDMA_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(MDMA_IRQn);

void MDMA_IRQHandler(void)
{
    HAL_MDMA_IRQHandler(&MDMA_Handle);
}
static void MDMA_TransferCompleteCallback(MDMA_HandleTypeDef *hmdma)
{
    TransferCompleteDetected = 1;
}
```

在传输完成回调里面设置了一个变量标志 TransferCompleteDetected，方便指示传输完成。

62.3.3第 3 步，AXI SRAM 内部互传 64KB 数据

通过下面的程序实现将地址 0x2400 0000 开始的 64KB 数据复制到地址 0x2400 0000 + 64*1024 里面：

```
TransferCompleteDetected = 0;
HAL_MDMA_Start_IT(&MDMA_Handle,
                  (uint32_t)0x24000000,
                  (uint32_t)(0x24000000 + 64*1024),
                  64*1024,
                  1);


start = DWT_CYCCNT;
while(TransferCompleteDetected == 0) {}
end = DWT_CYCCNT;
cnt = end - start;

//64*1024/(cnt/400/1000/1000)/1024/1024 = 64*1000*1000*400/1024/cnt = 25000000/cnt
```

```
printf("MDMA---AXI SRAM 内部互传 64KB 数据耗时 =  %dus %dMB/S\r\n", cnt/400, 25000000/cnt);
```

通过时钟周期计数器测量执行时间，单位 2.5ns。

62.3.4第 4 步，D2 域 SRAM1 内部互传 64KB 数据

通过下面的程序实现将地址 0x3000 0000 开始的 64KB 数据复制到地址 0x3000 0000 + 64*1024 里面：

```
TransferCompleteDetected = 0;
HAL_MDMA_Start_IT(&MDMA_Handle,
                  (uint32_t)0x30000000,
                  (uint32_t)(0x30000000 + 64*1024),
                  64*1024,
                  1);

start = DWT_CYCCNT;
while(TransferCompleteDetected == 0) {}
end = DWT_CYCCNT;
cnt = end - start;

printf("MDMA---D2 域 SRAM1 内部互传 64KB 数据耗时 =  %dus %dMB/S\r\n", cnt/400, 25000000/cnt);
```

通过时钟周期计数器测量执行时间，单位 2.5ns。

62.3.5第 5 步，AXI SRAM 传输 64KB 数据到 SDRAM

通过下面的程序实现将地址 0x2400 0000 开始的 64KB 数据复制到地址 0xC000 0000 里面：

```
TransferCompleteDetected = 0;
HAL_MDMA_Start_IT(&MDMA_Handle,
                  (uint32_t)0x24000000,
                  (uint32_t)0xC0000000,
                  64*1024,
                  1);

start = DWT_CYCCNT;
while(TransferCompleteDetected == 0) {}
end = DWT_CYCCNT;
cnt = end - start;

printf("MDMA---AXI SRAM 传输 64KB 数据到 SDRAM 耗时 =  %dus %dMB/S\r\n", cnt/400, 25000000/cnt);
```

通过时钟周期计数器测量执行时间，单位 2.5n。

62.3.6第 6 步，SDRAM 内部互传 64KB 数据

通过下面的程序实现将地址 0xC000 0000 开始的 64KB 数据复制到地址 0xC000 0000 + 64*1024 里面：

```
TransferCompleteDetected = 0;
HAL_MDMA_Start_IT(&MDMA_Handle,
```

```
                    (uint32_t)0xC0000000,
                    (uint32_t)(0xC0000000 + 64*1024),
                    64*1024,
                    1);


start = DWT_CYCCNT;
while(TransferCompleteDetected == 0) {}
end = DWT_CYCCNT;
cnt = end - start;

printf("MDMA---SDRAM 内部互传 64KB 数据耗时 =  %dus %dMB/S\r\n", cnt/400, 25000000/cnt);
```

通过时钟周期计数器测量执行时间，单位 2.5n。

62.4 DMA2D 性能测试程序设计

这里将 DMA2D 的程序设计分为以下几部分，逐一为大家做个说明：

62.4.1 第 1 步，DMA2D 初始化

配置 DMA2D 采用存储器到存储器模式，前景区和输出区都采用 ARGB8888 格式，传输 64*256 次，每次 4 字节，即 64*256*4 = 64KB 数据。

```
__HAL_RCC_DMA2D_CLK_ENABLE();

/* DMA2D 采用存储器到存储器模式, 这种模式是前景层作为 DMA2D 输入 */
DMA2D->CR      = 0x00000000UL;
DMA2D->FGOR    = 0;
DMA2D->OOR     = 0;

/* 前景层和输出区域都采用的 ARGB8888 颜色格式 */
DMA2D->FGPFCCR = LTDC_PIXEL_FORMAT_ARGB8888;
DMA2D->OPFCCR  = LTDC_PIXEL_FORMAT_ARGB8888;

DMA2D->NLR     = (uint32_t)(64 << 16) | (uint16_t)256;
```

62.4.2 第 2 步，AXI SRAM 内部互传 64KB 数据

通过下面的程序实现将地址 0x2400 0000 开始的 64KB 数据复制到地址 0x2400 0000 + 64*1024 里面：

```
/* AXI SRAM 的 64KB 数据传输测试 ********************************************/
DMA2D->FGMAR = (uint32_t)0x24000000;
DMA2D->OMAR  = (uint32_t)(0x24000000 + 64*1024);
DMA2D->CR    |= DMA2D_CR_START;

start = DWT_CYCCNT;
/* 等待 DMA2D 传输完成 */
while (DMA2D->CR & DMA2D_CR_START) {}
end = DWT_CYCCNT;
cnt = end - start;
```

```
printf("DMA2D---AXI SRAM 内部互传 64KB 数据耗时 =  %dus %dMB/S\r\n", cnt/400, 25000000/cnt);
```

通过时钟周期计数器测量执行时间，单位 2.5ns。

62.4.3第 3 步，D2 域 SRAM1 内部互传 64KB 数据

通过下面的程序实现将地址 0x3000 0000 开始的 64KB 数据复制到地址 0x3000 0000 + 64*1024 里面：

```
/* D2 域 SRAM1 的 64KB 数据传输测试 ********************************************/
DMA2D->FGMAR = (uint32_t)0x30000000;
DMA2D->OMAR  = (uint32_t)(0x30000000 + 64*1024);
DMA2D->CR    |= DMA2D_CR_START;

start = DWT_CYCCNT;
/* 等待 DMA2D 传输完成 */
while (DMA2D->CR & DMA2D_CR_START) {}
end = DWT_CYCCNT;
cnt = end - start;

printf("DMA2D---D2 域 SRAM1 内部互传 64KB 数据耗时 =  %dus %dMB/S\r\n", cnt/400, 25000000/cnt);
```

通过时钟周期计数器测量执行时间，单位 2.5ns。

62.4.4第 4 步，AXI SRAM 传输 64KB 数据到 SDRAM

通过下面的程序实现将地址 0x2400 0000 开始的 64KB 数据复制到地址 0xC000 0000 里面：

```
/* AXI SRAM 向 SDRAM 的 64KB 数据传输测试 ********************************************/
DMA2D->FGMAR = (uint32_t)0x24000000;
DMA2D->OMAR  = (uint32_t)0xC0000000;
DMA2D->CR    |= DMA2D_CR_START;

start = DWT_CYCCNT;
/* 等待 DMA2D 传输完成 */
while (DMA2D->CR & DMA2D_CR_START) {}
end = DWT_CYCCNT;
cnt = end - start;

printf("DMA2D---AXI SRAM 传输 64KB 数据到 SDRAM 耗时 =  %dus %dMB/S\r\n", cnt/400, 25000000/cnt);
```

通过时钟周期计数器测量执行时间，单位 2.5n。

62.4.5第 5 步，SDRAM 内部互传 64KB 数据

通过下面的程序实现将地址 0xC000 0000 开始的 64KB 数据复制到地址 0xC000 0000 + 64*1024 里面：

```
/* SDRAM 的 64KB 数据传输测试 ********************************************/
DMA2D->FGMAR = (uint32_t)0xC0000000;
DMA2D->OMAR  = (uint32_t)(0xC0000000 + 64*1024);
DMA2D->CR    |= DMA2D_CR_START;
```

```
start = DWT_CYCCNT;
/* 等待 DMA2D 传输完成 */
while (DMA2D->CR & DMA2D_CR_START) {}
end = DWT_CYCCNT;
cnt = end - start;

printf("DMA2D---SDRAM 内部互传 64KB 数据耗时 =  %dus %dMB/S\r\n", cnt/400, 25000000/cnt);
```

通过时钟周期计数器测量执行时间，单位 2.5n

62.5 通用 DMA 性能测试程序设计

这里将 DMA1 的程序设计分为以下几部分，逐一为大家做个说明：

62.5.1 第 1 步，DMA1 初始化

程序代码如下，采用存储区到存储区传输方式，源地址和目的地址都是 32bit 数据传输，并设置 4beat 突发，也就是连续传输 4 组 32bit 数据。

```
1.  __HAL_RCC_DMA1_CLK_ENABLE();
2.
3.  DMA_Handle.Instance                 = DMA1_Stream1;
4.  DMA_Handle.Init.Request             = DMA_REQUEST_MEM2MEM;
5.  DMA_Handle.Init.Direction           = DMA_MEMORY_TO_MEMORY;
6.  DMA_Handle.Init.PeriphInc           = DMA_PINC_ENABLE;
7.  DMA_Handle.Init.MemInc              = DMA_MINC_ENABLE;
8.  DMA_Handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;
9.  DMA_Handle.Init.MemDataAlignment    = DMA_PDATAALIGN_WORD;
10. DMA_Handle.Init.Mode                = DMA_NORMAL;
11. DMA_Handle.Init.Priority            = DMA_PRIORITY_VERY_HIGH;
12. DMA_Handle.Init.FIFOMode            = DMA_FIFOMODE_ENABLE;
13. DMA_Handle.Init.FIFOThreshold       = DMA_FIFO_THRESHOLD_FULL;
14. DMA_Handle.Init.MemBurst            = DMA_MBURST_INC4;     /*WORD 方式，仅支持 4 次突发 */
15. DMA_Handle.Init.PeriphBurst         = DMA_PBURST_INC4;     /*WORD 方式，仅支持 4 次突发 */
16. DMA_Handle.XferCpltCallback         = DMA_TransferCompleteCallback;
17.
18. HAL_DMA_Init(&DMA_Handle);
```

下面将程序设计中几个关键地方做个阐释：

- 第 1 行，务必优先初始化 DMA 时钟，测试发现没有使能时钟的情况下就配置 DMA 很容易失败。
- 第 14-15 行，突发传输的配置非常考究，这里要特别注意数据位宽，FIFO 以及突发的配置。

MSIZE	FIFO 级别	MBURST = INCR4	MBURST = INCR8	MBURST = INCR16
字节	1/4	4 个节拍的 1 次突发	禁止	禁止
	1/2	4 个节拍的 2 次突发	8 个节拍的 1 次突发	
	3/4	4 个节拍的 3 次突发	禁止	
	满	4 个节拍的 4 次突发	8 个节拍的 2 次突发	16 个节拍的 1 次突发
半字	1/4	禁止	禁止	禁止
	1/2	4 个节拍的 1 次突发		
	3/4	禁止		
	满	4 个节拍的 2 次突发	8 个节拍的 1 次突发	
字	1/4	禁止	禁止	
	1/2			
	3/4			
	满	4 个节拍的 1 次突发		

程序中数据位宽是配置为 32bit，FIFO 配置为满，那么突发仅可以配置为 4beat，即 DMA_MBURST_INC4。

- 第 16 行，设置传输完成回调函数。

62.5.2 第 2 步，DMA1 中断配置

DMA1 的中断设置比较简单，代码如下：

```
HAL_NVIC_SetPriority(DMA1_Stream1_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(DMA1_Stream1_IRQn);

void DMA1_Stream1_IRQHandler(void)
{
    HAL_DMA_IRQHandler(&DMA_Handle);
}
static void DMA_TransferCompleteCallback(DMA_HandleTypeDef *hdma)
{
     TransferCompleteDetected = 1;
}
```

在传输完成回调里面设置了一个变量标志 TransferCompleteDetected，方便指示传输完成。

62.5.3 第 3 步，AXI SRAM 内部互传 64KB 数据

通过下面的程序实现将地址 0x2400 0000 开始的 64KB 数据复制到地址 0x2400 0000 + 64*1024 里面：

```
/* AXI SRAM 的 64KB 数据传输测试 ********************************************/
TransferCompleteDetected = 0;
HAL_DMA_Start_IT(&DMA_Handle, (uint32_t)0x24000000, (uint32_t)(0x24000000 + 64*1024), 64*256);

start = DWT_CYCCNT;
while(TransferCompleteDetected == 0) {}
```

```
end = DWT_CYCCNT;
cnt = end - start;

//64*1024/(cnt/400/1000/1000)/1024/1024 = 64*1000*1000*400/1024/cnt = 25000000/cnt
printf("DMA1---AXI SRAM 内部互传 64KB 数据耗时 =  %dus %dMB/S\r\n", cnt/400, 25000000/cnt);
```

通过时钟周期计数器测量执行时间，单位 2.5ns。

62.5.4第 4 步，D2 域 SRAM1 内部互传 64KB 数据

通过下面的程序实现将地址 0x3000 0000 开始的 64KB 数据复制到地址 0x3000 0000 + 64*1024 里面：

```
/* D2 域 SRAM1 的 64KB 数据传输测试 ***********************************************/
TransferCompleteDetected = 0;
HAL_DMA_Start_IT(&DMA_Handle, (uint32_t)0x30000000, (uint32_t)(0x30000000 + 64*1024), 64*256);

start = DWT_CYCCNT;
while(TransferCompleteDetected == 0) {}
end = DWT_CYCCNT;
cnt = end - start;

printf("DMA1---D2 域 SRAM1 内部互传 64KB 数据耗时 =  %dus %dMB/S\r\n", cnt/400, 25000000/cnt);
```

通过时钟周期计数器测量执行时间，单位 2.5ns。

62.5.5第 5 步，AXI SRAM 传输 64KB 数据到 SDRAM

通过下面的程序实现将地址 0x2400 0000 开始的 64KB 数据复制到地址 0xC000 0000 里面：

```
/* AXI SRAM 向 SDRAM 的 64KB 数据传输测试 ***********************************************/
TransferCompleteDetected = 0;
HAL_DMA_Start_IT(&DMA_Handle, (uint32_t)0x24000000, (uint32_t)0xC0000000, 64*256);

start = DWT_CYCCNT;
while(TransferCompleteDetected == 0) {}
end = DWT_CYCCNT;
cnt = end - start;

printf("DMA1---AXI SRAM 传输 64KB 数据到 SDRAM 耗时 =  %dus %dMB/S\r\n", cnt/400, 25000000/cnt);
```

通过时钟周期计数器测量执行时间，单位 2.5n。

62.5.6第 6 步，SDRAM 内部互传 64KB 数据

通过下面的程序实现将地址 0xC000 0000 开始的 64KB 数据复制到地址 0xC000 0000 + 64*1024 里面：

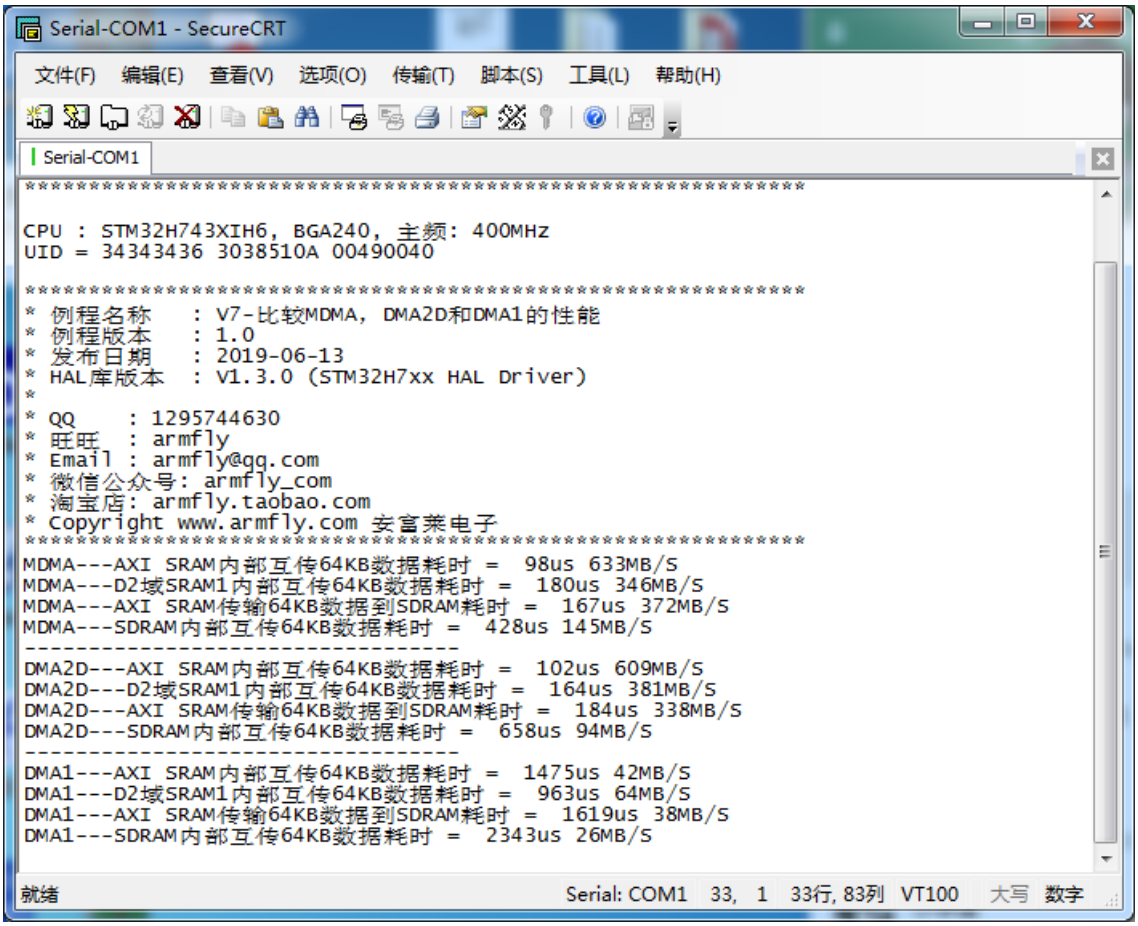
```
/* SDRAM 的 64KB 数据传输测试 ***********************************************/
TransferCompleteDetected = 0;
HAL_DMA_Start_IT(&DMA_Handle, (uint32_t)0xC0000000, (uint32_t)(0xC0000000 + 64*1024), 64*256);

start = DWT_CYCCNT;
```

```
while(TransferCompleteDetected == 0) {}
end = DWT_CYCCNT;
cnt = end - start;

printf("DMA1---SDRAM 内部互传 64KB 数据耗时 =  %dus %dMB/S\r\n", cnt/400, 25000000/cnt);
```

通过时钟周期计数器测量执行时间，单位 2.5n。

62.6 MDMA，DMA2D 和通用 DMA 性能比较

最终测试的性能如下：



可以看到 DMA1 的性能跟其它两个不是一个级别的，适合搞搞低速的外设。

DMA2D 和 MDMA 互有高低。

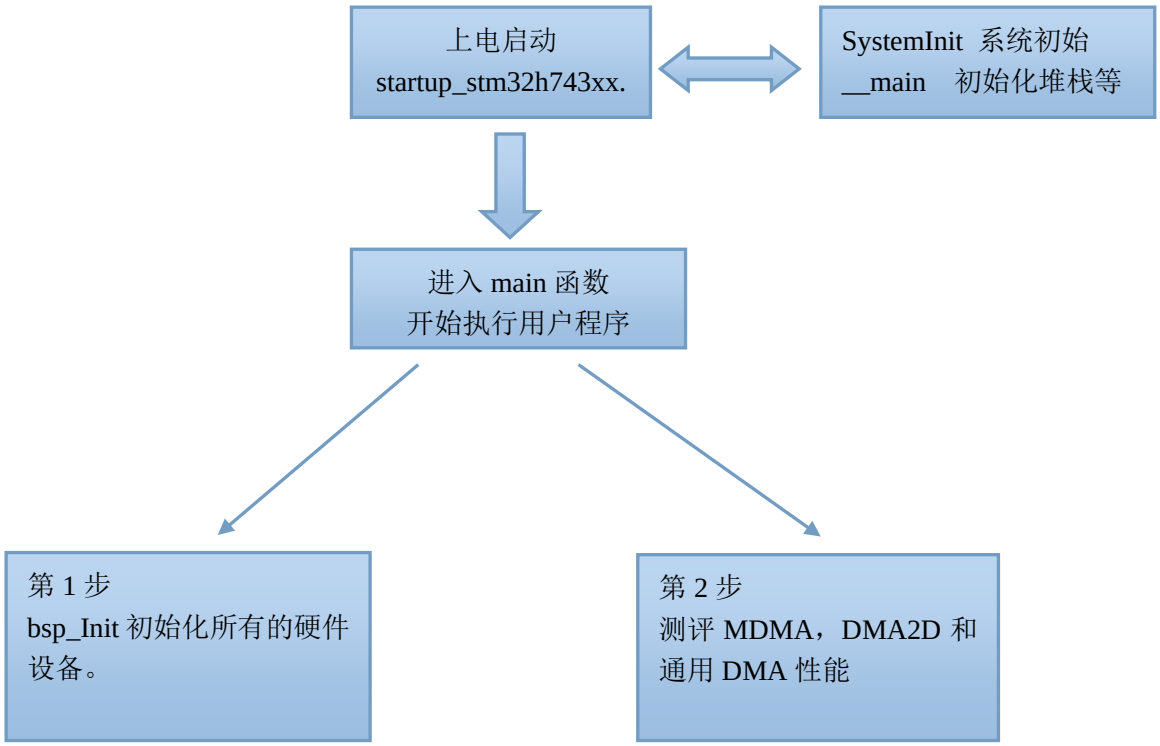
62.7 MDMA 驱动移植和使用

MDMA 驱动的移植比较方便：

- 第 1 步：添加 MDMA 的 HAL 库文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 2 步，应用方法看本章节配套例子即可，另外就是根据自己的需要做配置修改。

62.8 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED 和 SDRAM。
  - 第 2 步，测评 MDMA，DMA2D 和通用 DMA 性能。

62.9 实验例程说明（MDK）

配套例子：

V7-038_MDMA，DMA2D 和通用 DMA 性能比较

实验目的：

1. 比较 MDMA，DMA2D 和 DMA1 的性能

实验内容：

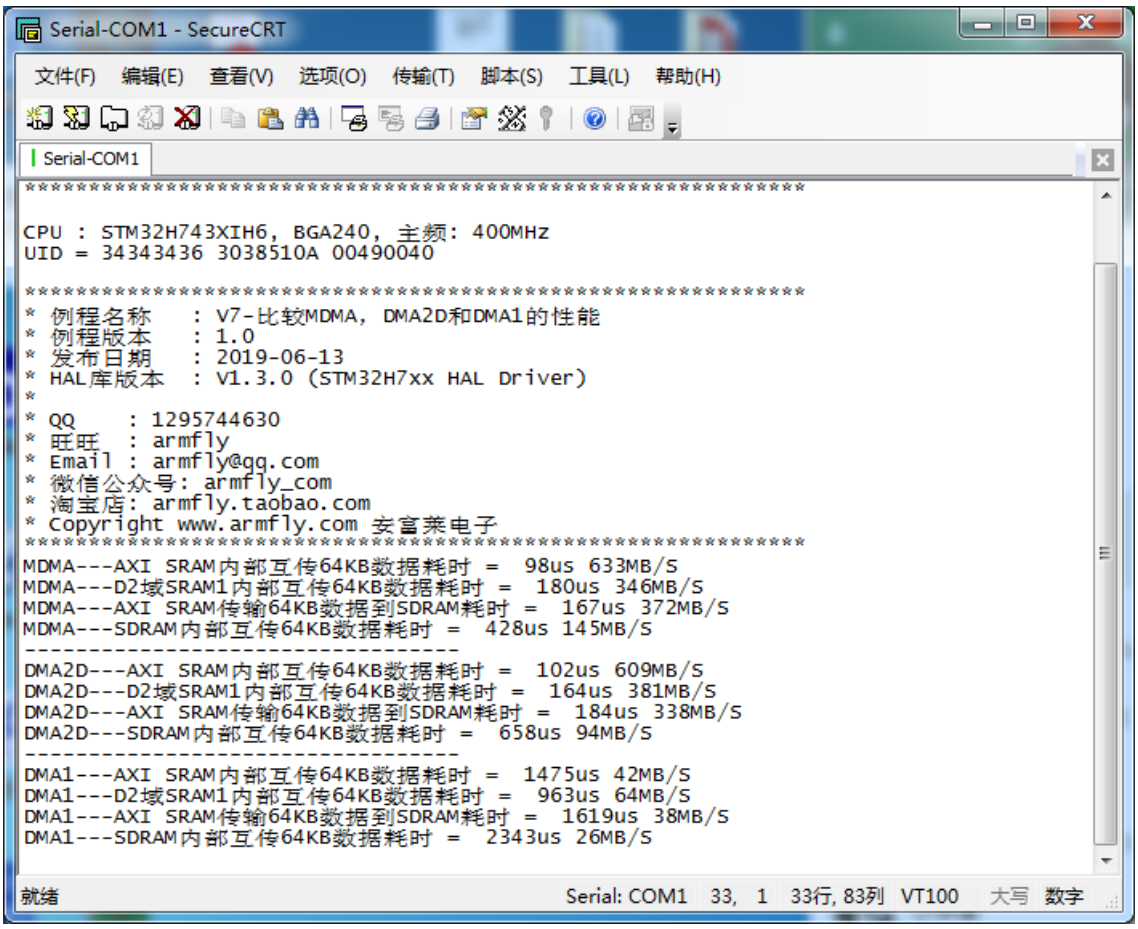
MDMA，DMA2D 和 DMA1 都测试了如下四种情况：

1. 64 位带宽的 AXI SRAM 内部做 64KB 数据传输。
2. 32 位带宽的 D2 域 SRAM1 内部 64KB 数据传输。

3. AXI SRAM 向 SDRAM 传输 64KB 的数据传输。
4. 32 位带宽的 SDRAM 内部做 64KB 数据传输。

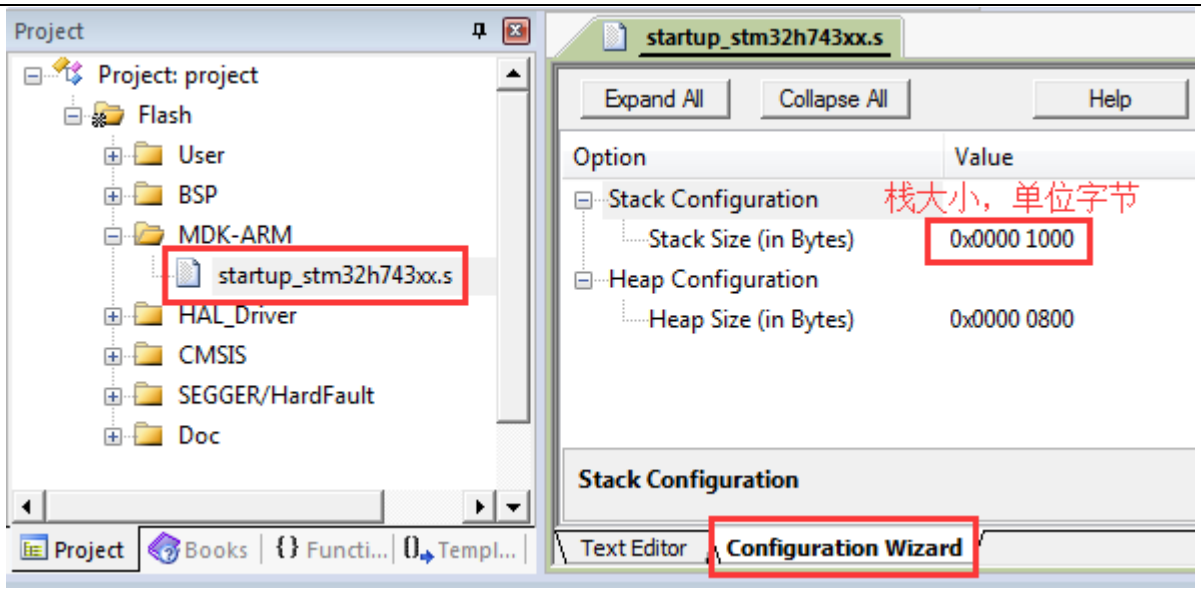
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

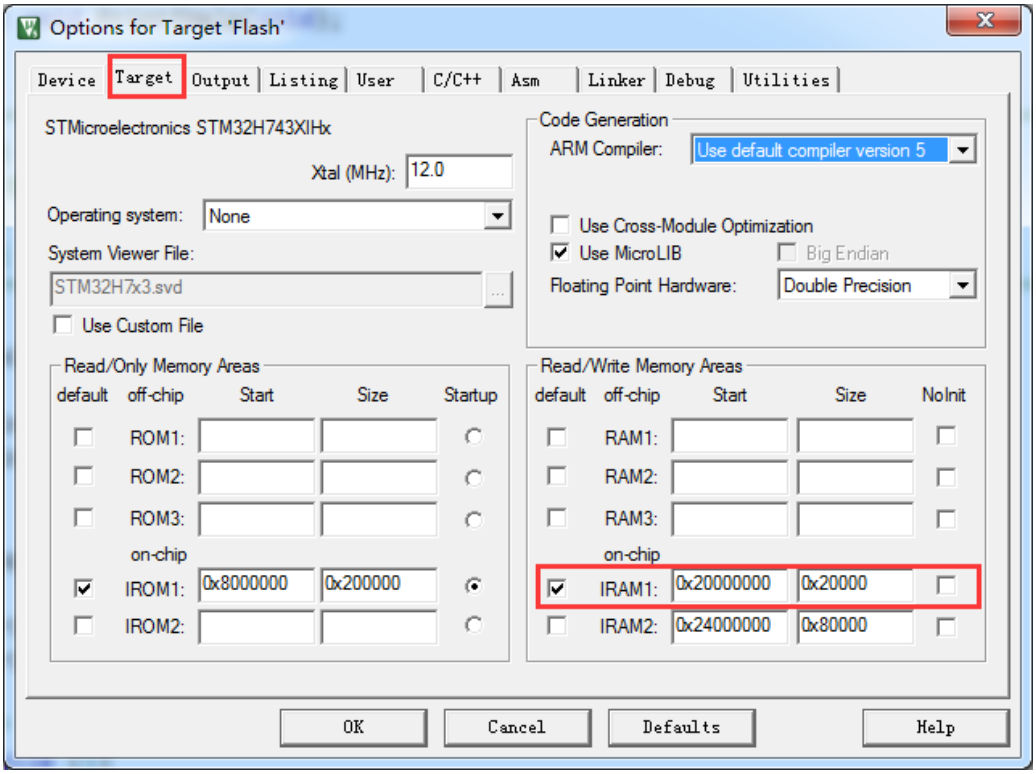


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    bsp_InitI2C();      /* 初始化 I2C 总线 */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 D3 域的 SRAM4。DAC 的数据缓存开在了 SRAM4。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;
```

```
    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SDRAM 的 MPU 属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0xC0000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_32MB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
```

```
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能:**

主程序实现如下操作:

- 测试了 MDMA，DMA2D 和 DMA1。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    bsp_Init();   /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */

    MDMA_SpeedTest();
    printf("---------------------------------\n\r");
    DMA2D_SpeedTest();
    printf("---------------------------------\n\r");
    DMA1_SpeedTest();

    bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */

      /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();

        /* 判断软件定时器 0 是否超时 */
        if(bsp_CheckTimer(0))
        {
            /* 每隔 200ms 进来一次 */
            bsp_LedToggle(2);
        }
    }
}
```

62.10 实验例程说明（IAR)

配套例子:

V7-038_MDMA，DMA2D 和通用 DMA 性能比较

实验目的：

1. 比较 MDMA，DMA2D 和 DMA1 的性能

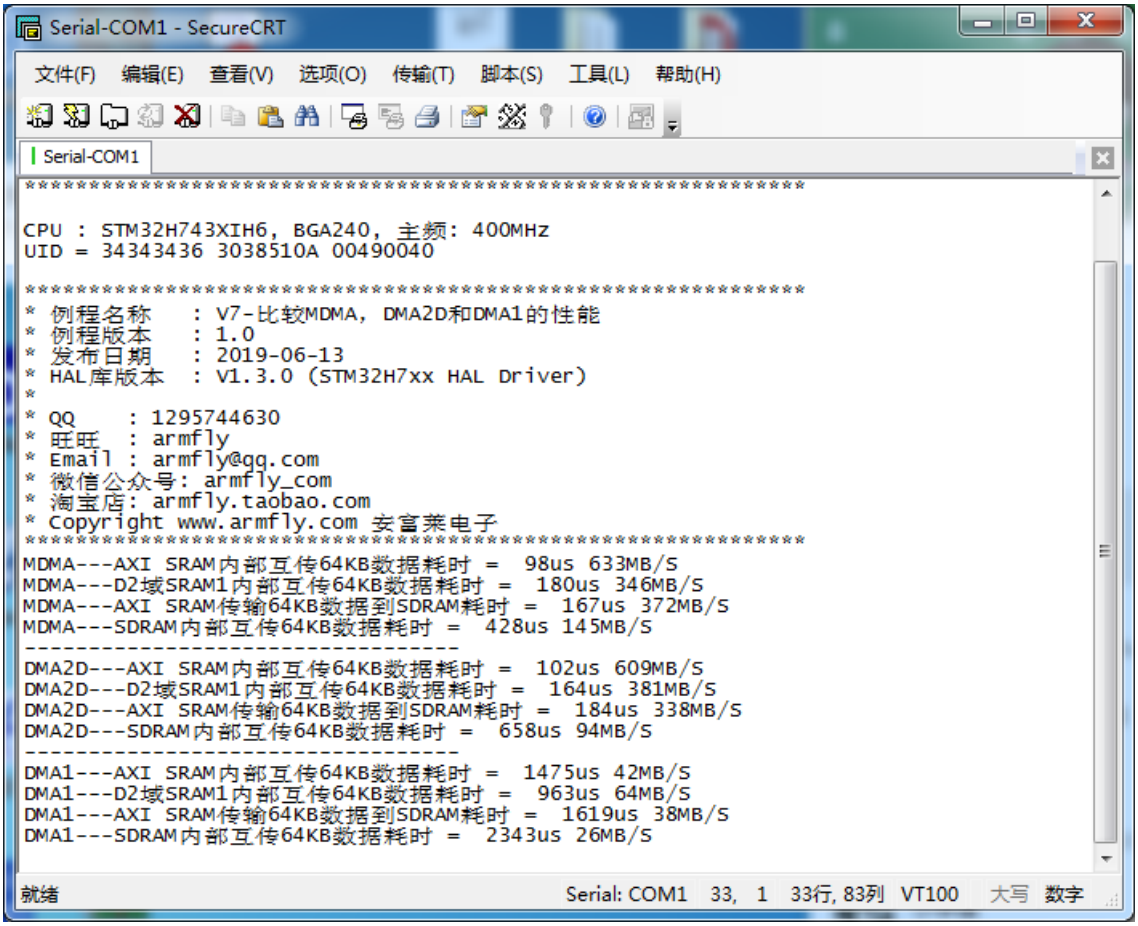
实验内容：

MDMA，DMA2D 和 DMA1 都测试了如下四种情况：

1. 64 位带宽的 AXI SRAM 内部做 64KB 数据传输。
2. 32 位带宽的 D2 域 SRAM1 内部 64KB 数据传输。
3. AXI SRAM 向 SDRAM 传输 64KB 的数据传输。
4. 32 位带宽的 SDRAM 内部做 64KB 数据传输。

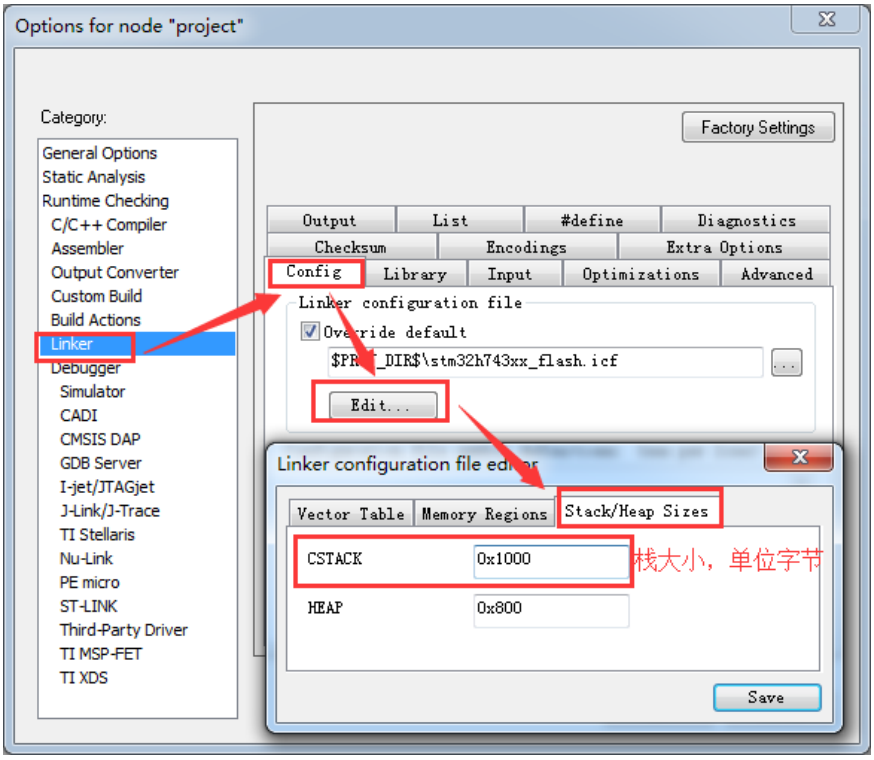
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

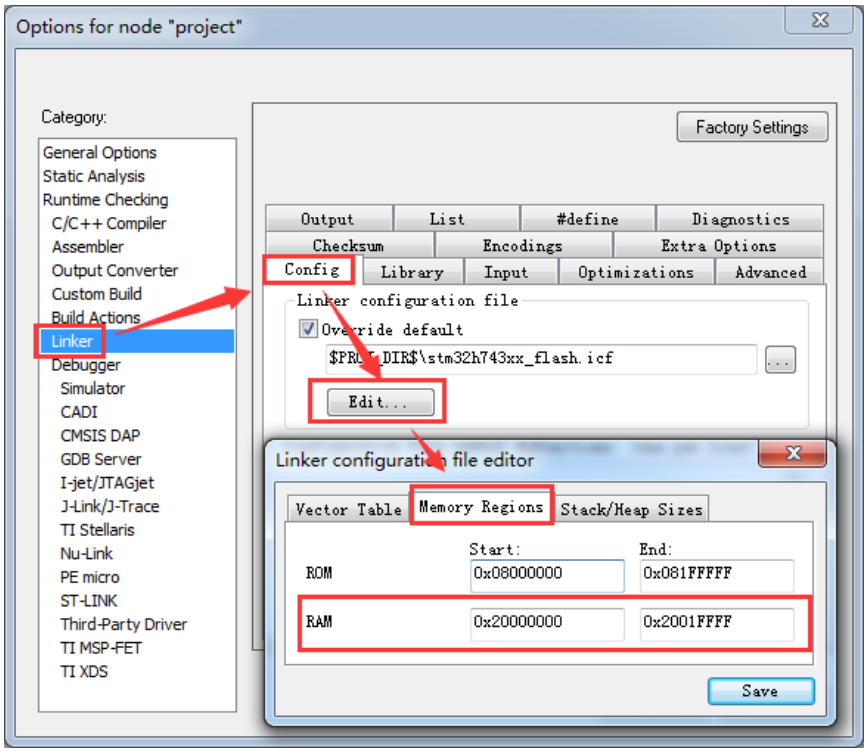


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
```

```
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    bsp_InitI2C();      /* 初始化 I2C 总线 */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM），FMC 的扩展 IO 区和 D3 域的 SRAM4。DAC 的数据缓存开在了 SRAM4。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
```

```
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

   /* 配置 SDRAM 的 MPU 属性为 Write through, read allocate, no write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0xC0000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_32MB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
```

```
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能:**

主程序实现如下操作:

- 测试了 MDMA，DMA2D 和 DMA1。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    bsp_Init();   /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */

    MDMA_SpeedTest();
    printf("---------------------------------\n\r");
    DMA2D_SpeedTest();
    printf("---------------------------------\n\r");
    DMA1_SpeedTest();

    bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */

      /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();

        /* 判断软件定时器 0 是否超时 */
        if(bsp_CheckTimer(0))
        {
            /* 每隔 200ms 进来一次 */
            bsp_LedToggle(2);
        }
    }
}
```

62.11 总结

本章节涉及到的知识点比较重要，以后用到 DMA 的地方比较多，可以根据性能选择合适的 DMA。

第63章 STM32H7 的高分辨率定时器 HRTIM 基础知识和 HAL 库 API

本章节为大家讲解高分辨率定时器（High-Resolution Timer）的基础知识和对应的 HAL 库 API。

63.1 初学者重要提示

63.2 HRTIM 基础知识

63.3 HRTIM 定时器的 HAL 库用法

63.4 源文件 stm32h7xx_hal_hrtim.c

63.5 总结

63.1 初学者重要提示

- 高分辨率定时器可以产生多达 10 路高分辨率信号。它主要用数字电源、照明，电源耗材，太阳能逆变器和无线充电等应用场合，当然，也可以作为通用目的。
- HRTIM 模块化的架构允许产生独立或者叠加波形。通过带的时序控制单元和各种外部事件的配合，可以产生各种信号 PWM，相移等。
- HRTIM 定时器还具有定时测量功能和链接到内置 ADC 和 DAC 转换器。并且它还具有轻载管理模式和能够处理各种故障的方案以实现安全关闭。
- HRTIM 有多个中断入口函数，使用的时候别搞错了：

```
HRTIM1_Master_IRQHandler
HRTIM1_TIMA_IRQHandler
HRTIM1_TIMB_IRQHandler
HRTIM1_TIMC_IRQHandler
HRTIM1_TIMD_IRQHandler
HRTIM1_TIME_IRQHandler
HRTIM1_FLT_IRQHandler
```

63.2 HRTIM 基础知识

- HRTIM 由 6 个定时器组成，分别是 1 个主定时器（Master Timer）和 5 个独立的定时器单元，从 Timer A 到 Timer E。
  - 这 6 个都是 16bit 定时器，每个定时器都包含 1 个独立计数器和 4 个比较单元。
  - 主定时器（Master Timer）：

基于 16 位递增计数器。它可通过 4 个比较单元置位/ 复位 10 路输出中的任何一路，并向 5 个定时器单元提供同步信号。其主要用途是使定时器单元受唯一的时钟源控制。交错降压转换器是一个典型的应用示例，主定时器在其中管理多个单元之间的相移。

- 5 个定时器 Timer A 到 Timer E：

  既可以独立工作，也可以与其它定时器（包括主定时器）配合工作。每个定时器都可控制两路输出。输出置位/复位事件可以由定时单元比较寄存器触发，或者由主定时器事件、其他定时器的事件或外部事件触发。

- 每个定时器的两路输出：
  - 支持 PWM 互补输出，支持添加死区时间。
  - 将载波频率添加到调制信号上。
  - 通过将异步输出置为预定义的安全电平来管理故障事件。

◆ 10 个外部事件，可用于任何定时器单元。
- 可编程极性和边沿有效性。
- 5 个事件用于快速异步模式。
- 5 个事件用于可编程数字滤波器。
- 利用消隐和窗口模式实现伪事件过滤。

◆ 多条通道可连接到内置模拟外设。
- 4 个用于 ADC 转换器的触发信号。
- 3 个用于 DAC 转换器的触发信号。
- 3 个用于比较器。

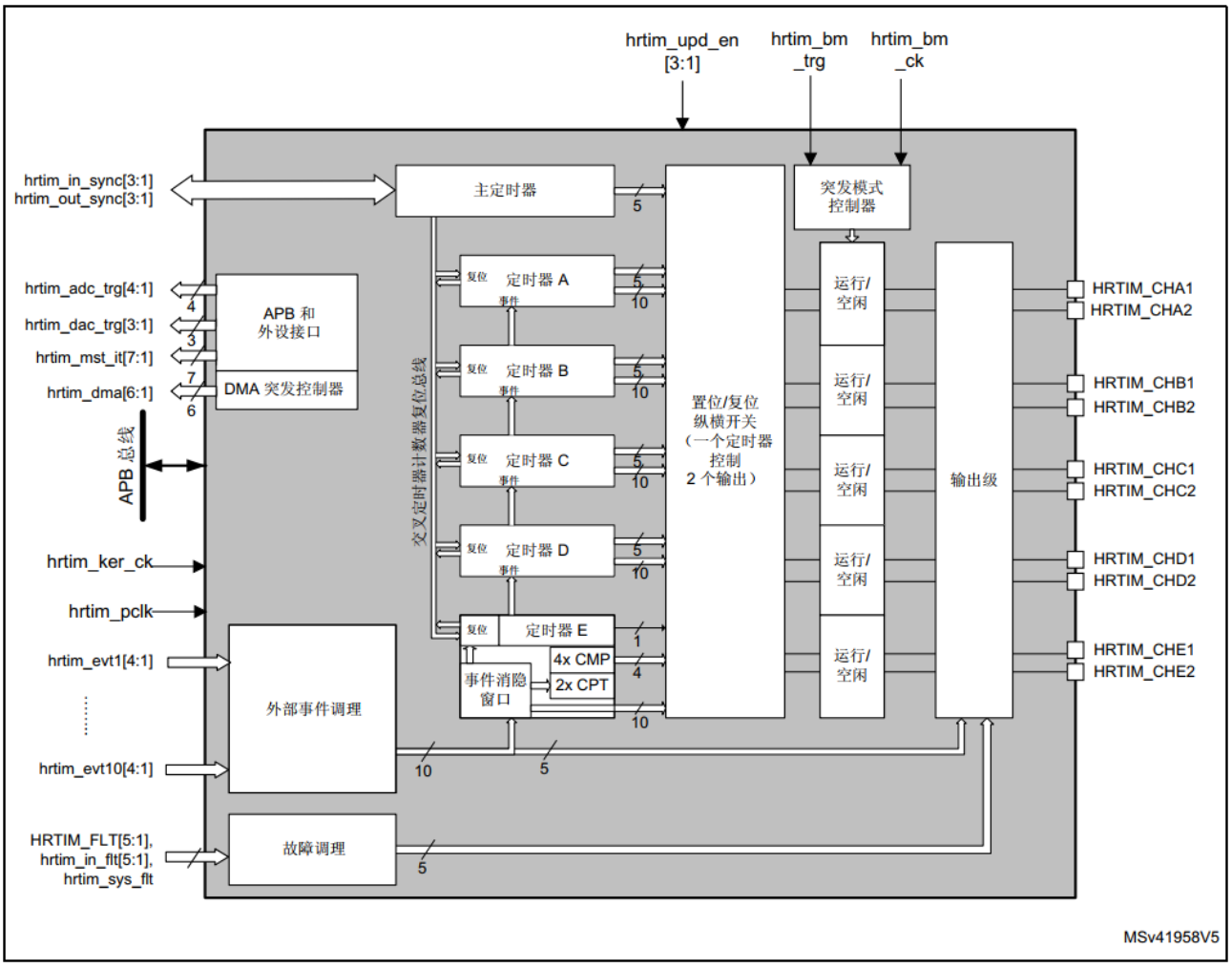
◆ 丰富的保护机制。
- 5 路故障输入可组合使用，而且可以关联到任何定时单元。
- 可编程极性和边沿有效性。
- 对谐振变换器配有专门的延时保护。

◆ 不同 HRTIM 之间可以做同步输入/输出。

63.2.1 定时器的硬件框图

认识一个外设，最好的方式就是看他的框图，方便我们快速的了解 HRTIM 的基本功能，然后再看手册了解细节。

下面我们直接看 HRTIM 的框图：



通过这个框图，我们可以得到如下信息：

- hrtim_in_sync[3:1]

  同步输入端，将 HRTIM 与其他内部或外部定时器资源进行同步输入：

  hrtim_in_sync[1]：保留。

  hrtim_in_sync[2]：时钟源为通用 TIMx 定时器（通过片上互连）。

  hrtim_in_sync[3]：时钟源为外部 HRTIM（通过 HRTIM_SCIN 输入引脚）。

- hrtim_out_sync[2:1]

  同步输出端，用于级联或同步多个片上或片外 HRTIM，由于 H7 只有一个 HRTIM，所以只能同步其它器件上带的 HRTIM。

  hrtim_out_sync[1]：保留。

  hrtim_out_sync[2]：目标为片外 HRTIM 或外设（通过 HRTIM_SCOUT 输出引脚）。

- hrtim_adc_trg[4:1]

  输出端，用于触发 ADC 转换。

- hrtim_dac_trg[3:1]

  输出端，用于触发 DAC 转换。

- hrtim_mst_it[7:1]

  输出端，用于发出中断请求。

- hrtim_dma[6:1]

  输出端，用于发出 DMA 请求。

- hrtim_pclk

  输入端，APB 时钟。

- hrtim_ker_ck

  输出端， HRTIM 内核时钟。

- hrtim_evtX[4:1] 大写字母 X 的范围是 1 到 10。

  输入端，用于接收外部事件，支持接收 10 个事件，每个事件有 4 种输入源可以选择，可选择片上（来自其他内置外设：比较器、ADC 模拟看门狗、TIMx 定时器、触发输出等）或片外（HRTIM_EEVx 输入引脚）。

外部事件通道	快速模式	数字滤波器	均衡故障定时器 A、B、C	均衡故障定时器 D、E、F	Src1	Src2	Src3	Src4
1	有	-	-	-	PC10	COMP1	TIM1_TRGO	ADC1_AWD1
2	有	-	-	-	PC12	COMP2	TIM2_TRGO	ADC1_AWD2
3	有	-	-	-	PD5	-	TIM3_TRGO	ADC1_AWD3
4	有	-	-	-	PG11	OPAMP1(1)	TIM7_TRGO	ADC2_AWD1
5	有	-	-	-	PG12	-	LPTIM1 OUT	ADC2_AWD2
6	-	有	有	-	PB4	COMP1	TIM6_TRGO	ADC2_AWD3
7	-	有	有	-	PB5	COMP2	TIM7_TRGO	-
8	-	有	-	有	PB6	-	TIM6_TRGO	TTCAN_TMP
9	-	有	-	有	PB7	OPAMP1(1)	TIM15_TRGO	TTCAN_RTP
10	-	有	-	-	PG13	-	LPTIM2 OUT	TTCAN_SOC

- HRTIM_FLT[5:1]

  hrtim_in_flt[5:1]

  故障输入端，5 路片上输入和 5 路片外 HRTIM_FLTx 输入。

  当输入信号有效时立即禁止 HRTIM 输出。

- hrtim_sys_flt

  系统故障输入，比如时钟安全系统、 SRAM 奇偶校验错误、 Cortex®-M7 LOCKUP (HardFault)、PVD 输出等。

- hrtim_upd_en[3:1]

  输入端，HRTIM 寄存器更新使能（片上互连）会触发从影子寄存器到活动寄存器的传输。

- hrtim_bm_trg

  输入端，突发模式触发事件。

- hrtim_bm_ck

  输入端，突发模式时钟。

- HRTIM_CHA1

  HRTIM_CHA2

  HRTIM_CHB1

  HRTIM_CHB2

  HRTIM_CHC1

  HRTIM_CHC2

  HRTIM_CHD1

  HRTIM_CHD2

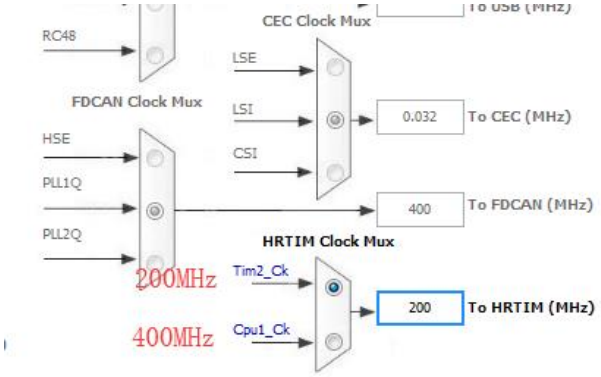
  HRTIM_CHE1

  HRTIM_CHE2

  对应定时器单元 Timer A，B，C，D，E 的输出，每个定时器有两路。

63.2.2 HRTIM 时钟选择

HRTIM 支持两种时钟输入，一个是通用定时器时钟输入，另一个是 CPU 主频时钟输入：



反映到程序代码上，通过下两个宏定义参数区分：

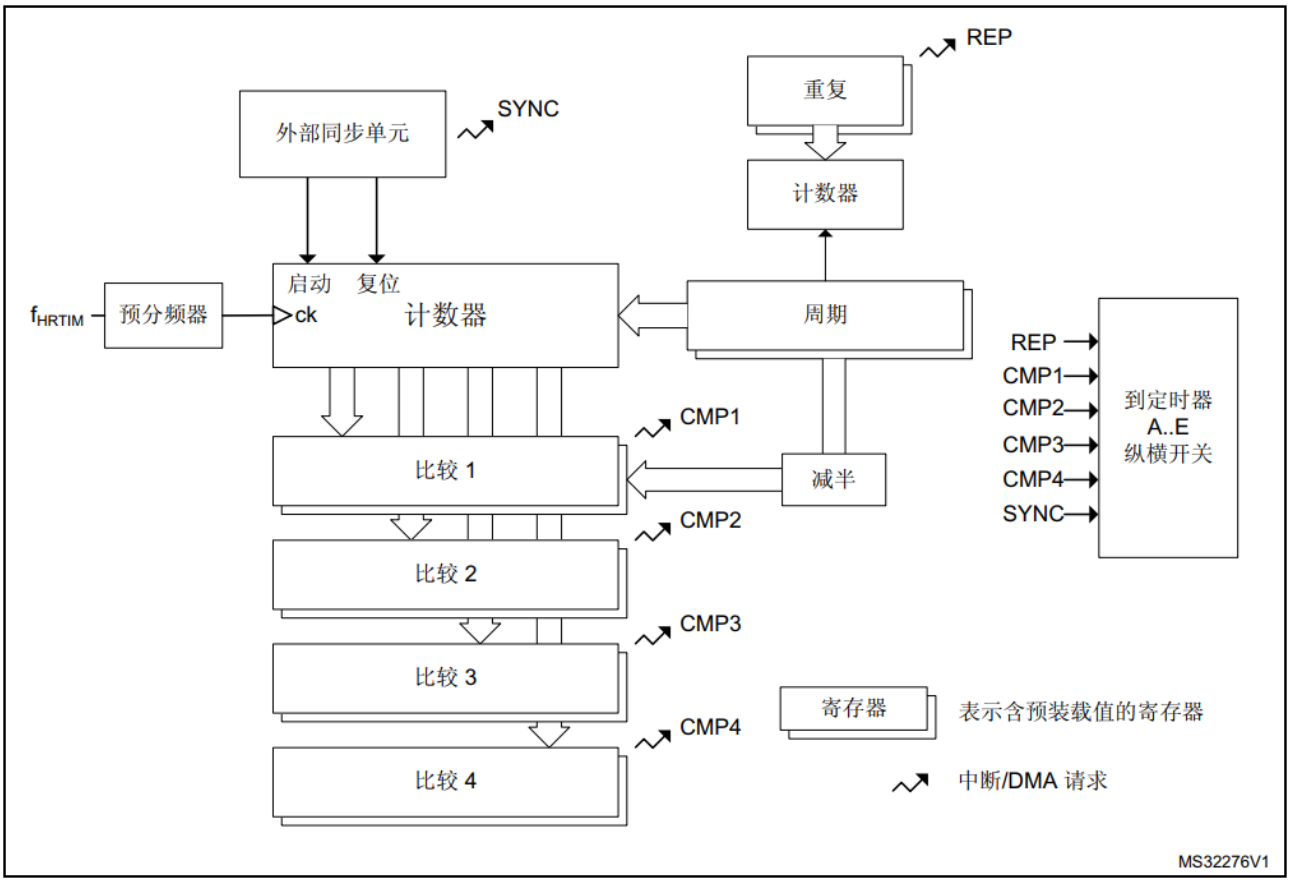
```
#define RCC_HRTIM1CLK_TIMCLK    ((uint32_t)0x00000000)
#define RCC_HRTIM1CLK_CPUCLK    RCC_CFGR_HRTIMSEL
```

- 使用 CPU 主频时钟，对应参数 RCC_HRTIM1CLK_CPUCLK
- 使用通用定时器时钟，对应参数 RCC_HRTIM1CLK_TIMCLK。如果 CPU 主频时钟是 400MHz 的话，通用定时器时钟就是 200MHz。
- 具体实现，通过函数 HAL_RCCEx_PeriphCLKConfig 来配置：

```
PeriphClkInitStruct.PeriphClockSelection = RCC_PERIPHCLK_HRTIM1;
PeriphClkInitStruct.Hrtim1ClockSelection = RCC_HRTIM1CLK_TIMCLK;
if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInitStruct) != HAL_OK)
{
    _Error_Handler(__FILE__, __LINE__);
}
```

63.2.3 HRTIM 主定时器

HRTIM 的主定时器框图：

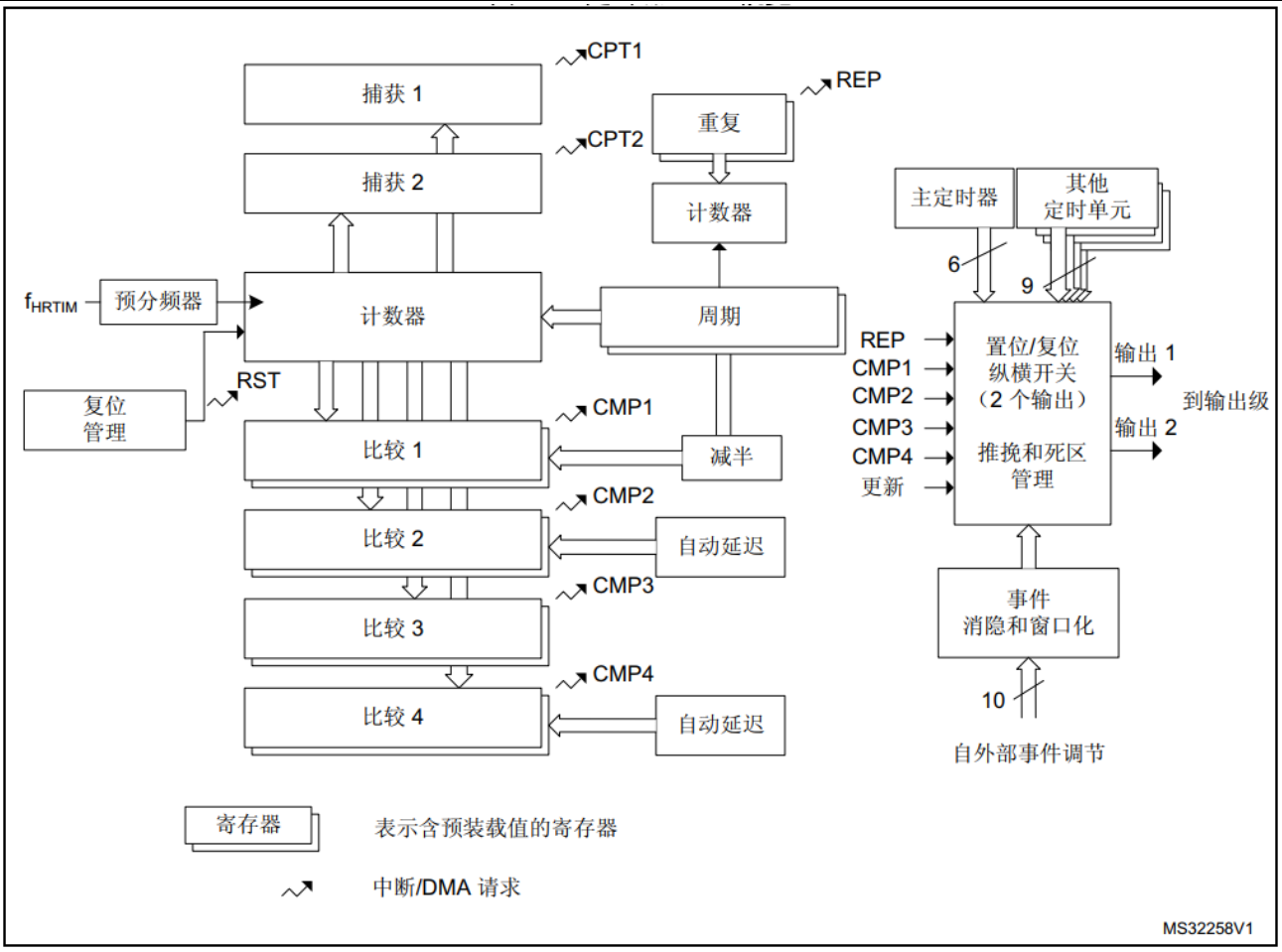


对于这个框图，了解到以下几点即可：

- 主定时器有 1 个计数器，4 个比较单元，但没有捕获单元和输出。
- 主定时器的控制寄存器包含主定时器 Master Timer 和定时单元 Timer A 到 E 的所有定时器使能位。

63.2.4 HRTIM 定时器单元（Timer A 到 E）

5 个定时器单元的 Timer A 到 E 都是一样的，框图如下：



对于这个框图，了解到以下几点即可：

- 有 1 个计数器，4 个比较单元，2 个捕获单元，带两个输出。
- 比较单元 2 和比较单元 4 支持自带延迟。
- 事件消隐和窗口模式可以实现伪事件过滤。
- REP，CMP1，CMP2，CMP3，CMP 和定时器更新这几个信号可以设置输出端的置位/复位。

63.2.5 HRTIM 的分辨率

STM32H7 的主频配置为 400MHz 时，那么 HRTIM 的时钟最高就是 400MHz。对于 H7 系列，HRTIM 不支持倍频设置，仅支持 1 分频，2 分频和 4 分频：

CKPSC[2:0](1)	Prescaling ratio	$f_{COUNTER}$	Resolution	Min PWM frequency
101	1	400 MHz	2.5 ns	6.1 kHz
110	2	400/2 MHz = 200 MHz	5 ns	3.05 kHz
111	4	400/4 MHz = 100MHz	10 ns	1.5 kHz

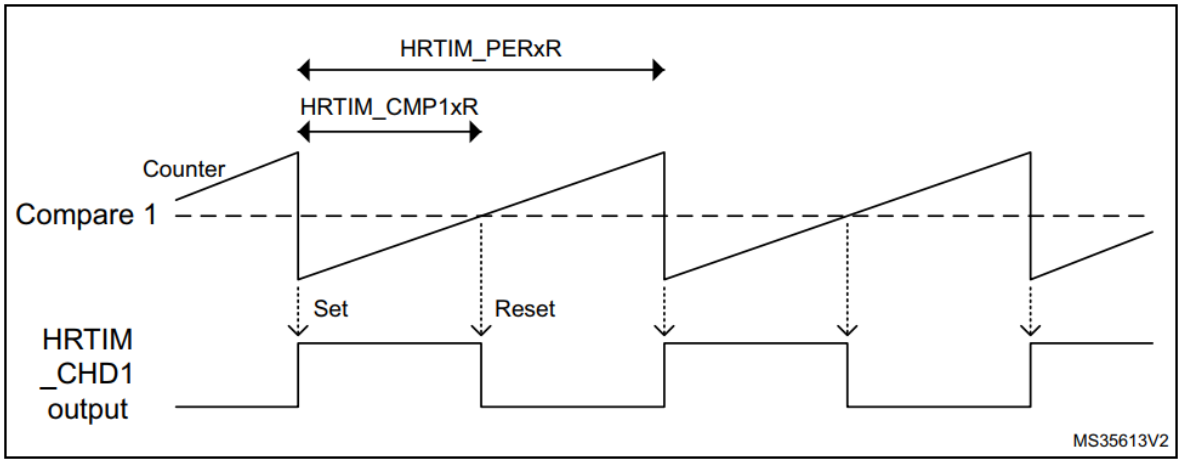
H7 系列与 F3 以及 G4 的对比：

Part numbers	PLL input	Input frequency (MHz)		HRTIM frequency (GHz)		Resolution (ps)	
		Min	Max	Min	Max	Min	Max
STM32F334xx	HSE	128	144	4.096	4.608	217	244
	HSI	128		4.096			
STM32H74xxx STM32H75xxx	Any	-	480	-	0.480	2.08 ns (no DLL)	-
STM32G47xxx STM32G48xxx		100	170	3.20	5.44	184	312

分辨率最高是 G4 系列，可以做到 184ps。

63.2.6 HRTIM 的 PWM 多路输出实现

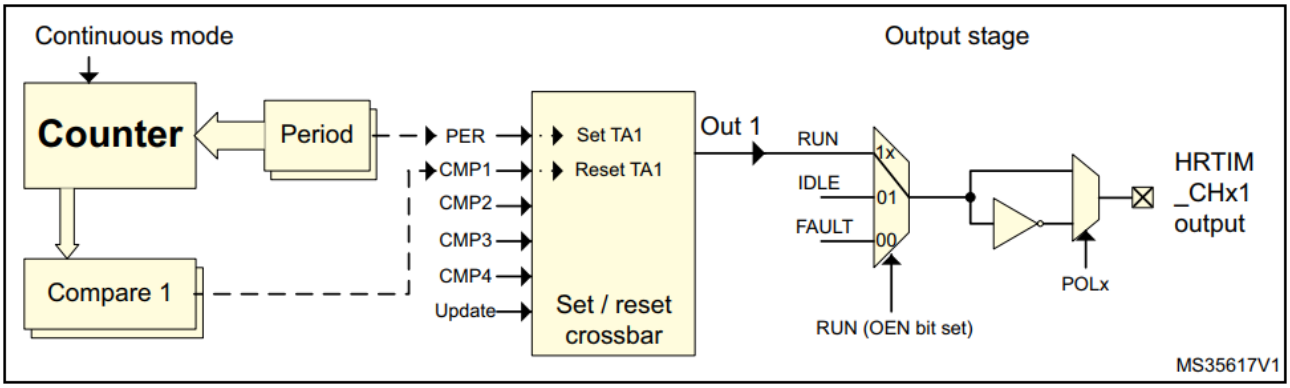
这里我们先来看下单路 PWM 输出：



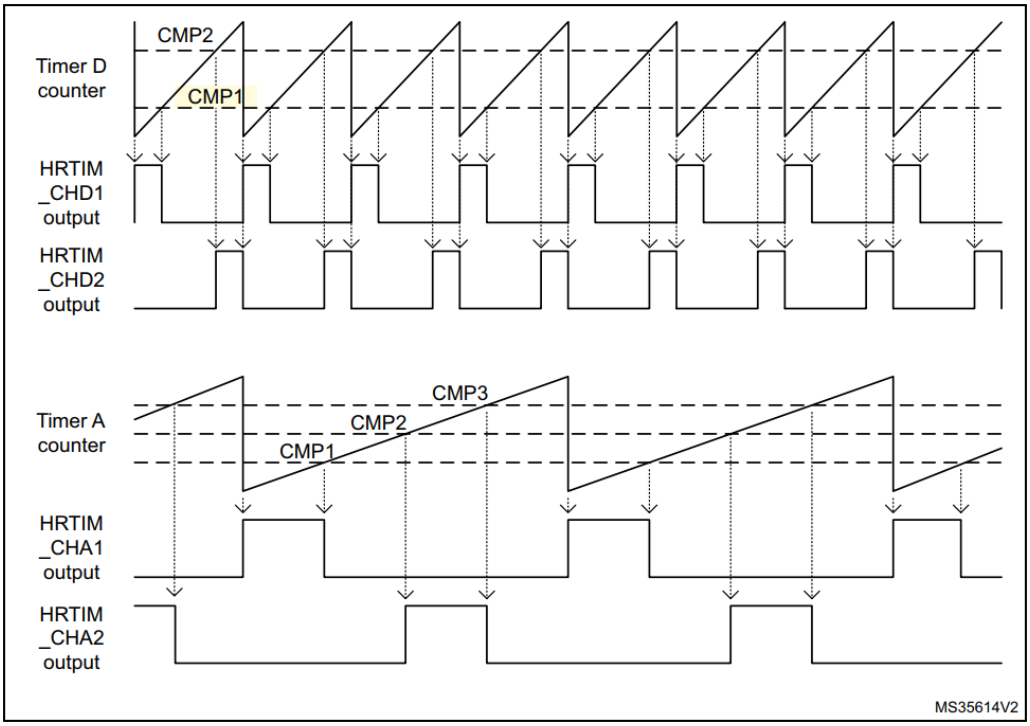
寄存器 HRTIM_PERxR 存储定时器的周期，寄存器 HRTIM_CMP1xR 存储比较值。执行过程如下：

- 计数器 Counter 计数到 HRTIM_CMP1xR 存储的比较值时，HRTIM_CHD1 输出端复位。
- 计数到 HRTIM_PERxR 时，HRTIM_CHD1 输出端置位，并且 Counter 重新从头开始计数。

理解了单路 PWM 输出后，再看实现这个功能对应的框图就很好理解了：



接下来再来看多路 PWM 输出：



截图上半部分波形是 Timer D 输出两路：

- HRTIM_CHD1 输出是通过 CMP1 以及计数更新实现输出的置位/复位。
- HRTIM_CHD2 输出是通过 CMP2 以及计数更新实现输出的置位/复位。

截图下半部分波形是 Timer A 输出两路：

- HRTIM_CHA1 输出是通过 CMP1 以及计数更新实现输出的置位/复位。
- HRTIM_CHA2 输出是通过 CMP2，CMP3 以及计数更新实现输出的置位/复位。

63.3 HRTIM 定时器的 HAL 库用法

HRTIM 的 HAL 库用法其实就是几个结构体变量成员的配置和使用，然后配置 GPIO、时钟，并根据需要配置 NVIC、中断和 DMA。下面我们逐一展开为大家做个说明。

63.3.1 定时器寄存器结构体 HRTIM_TypeDef

定时器相关的寄存器是通过 HAL 库中的结构体 HRTIM_TypeDef 定义的，在 stm32h743xx.h 中可以找到这个定义类型：

```
typedef struct {
  HRTIM_Master_TypeDef sMasterRegs;
  HRTIM_Timerx_TypeDef sTimerxRegs[5];
  uint32_t             RESERVED0[32];
  HRTIM_Common_TypeDef sCommonRegs;
}HRTIM_TypeDef;
```

```
typedef struct
{
  __IO uint32_t MCR;
  __IO uint32_t MISR;
  __IO uint32_t MICR;
  __IO uint32_t MDIER;
  __IO uint32_t MCNTR;
  __IO uint32_t MPER;
  __IO uint32_t MREP;
  __IO uint32_t MCMP1R;
  uint32_t      RESERVED0;
  __IO uint32_t MCMP2R;
  __IO uint32_t MCMP3R;
  __IO uint32_t MCMP4R;
  uint32_t      RESERVED1[20];
}HRTIM_Master_TypeDef;

typedef struct
{
  __IO uint32_t TIMxCR;
  __IO uint32_t TIMxISR;
  __IO uint32_t TIMxICR;
  __IO uint32_t TIMxDIER;
  __IO uint32_t CNTxR;
  __IO uint32_t PERxR;
  __IO uint32_t REPxR;
  __IO uint32_t CMP1xR;
  __IO uint32_t CMP1CxR;
  __IO uint32_t CMP2xR;
  __IO uint32_t CMP3xR;
  __IO uint32_t CMP4xR;
  __IO uint32_t CPT1xR;
  __IO uint32_t CPT2xR;
  __IO uint32_t DTxR;
  __IO uint32_t SETx1R;
  __IO uint32_t RSTx1R;
  __IO uint32_t SETx2R;
  __IO uint32_t RSTx2R;
  __IO uint32_t EEFxR1;
  __IO uint32_t EEFxR2;
  __IO uint32_t RSTxR;
  __IO uint32_t CHPxR;
  __IO uint32_t CPT1xCR;
  __IO uint32_t CPT2xCR;
  __IO uint32_t OUTxR;
  __IO uint32_t FLTxR;
  uint32_t      RESERVED0[5];
}HRTIM_Timerx_TypeDef;

typedef struct
{
  __IO uint32_t CR1;
  __IO uint32_t CR2;
  __IO uint32_t ISR;
  __IO uint32_t ICR;
  __IO uint32_t IER;
  __IO uint32_t OENR;
```

```
  __IO uint32_t ODISR;
  __IO uint32_t ODSR;
  __IO uint32_t BMCR;
  __IO uint32_t BMTRGR;
  __IO uint32_t BMCMPR;
  __IO uint32_t BMPER;
  __IO uint32_t EECR1;
  __IO uint32_t EECR2;
  __IO uint32_t EECR3;
  __IO uint32_t ADC1R;
  __IO uint32_t ADC2R;
  __IO uint32_t ADC3R;
  __IO uint32_t ADC4R;
  __IO uint32_t RESERVED0;
  __IO uint32_t FLTINR1;
  __IO uint32_t FLTINR2;
  __IO uint32_t BDMUPR;
  __IO uint32_t BDTAUPR;
  __IO uint32_t BDTBUPR;
  __IO uint32_t BDTCUPR;
  __IO uint32_t BDTDUPR;
  __IO uint32_t BDTEUPR;
  __IO uint32_t BDMADR;
}HRTIM_Common_TypeDef;
```

这个结构体的成员名称和排列次序和 CPU 的定时器寄存器是一 一对应的。

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

下面我们看下定时器的定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE                 (0x40000000UL)
#define D2_APB2PERIPH_BASE          (PERIPH_BASE + 0x00010000UL)

#define HRTIM1_BASE              (D2_APB2PERIPH_BASE + 0x7400UL)
#define HRTIM1_TIMA_BASE         (HRTIM1_BASE + 0x00000080UL)
#define HRTIM1_TIMB_BASE         (HRTIM1_BASE + 0x00000100UL)
#define HRTIM1_TIMC_BASE         (HRTIM1_BASE + 0x00000180UL)
#define HRTIM1_TIMD_BASE         (HRTIM1_BASE + 0x00000200UL)
#define HRTIM1_TIME_BASE         (HRTIM1_BASE + 0x00000280UL)
#define HRTIM1_COMMON_BASE       (HRTIM1_BASE + 0x00000380UL)

#define HRTIM1                  ((HRTIM_TypeDef *) HRTIM1_BASE)<----- 展开这个宏，(HRTIM_TypeDef *)0x40017400
#define HRTIM1_TIMA             ((HRTIM_Timerx_TypeDef *) HRTIM1_TIMA_BASE)
#define HRTIM1_TIMB             ((HRTIM_Timerx_TypeDef *) HRTIM1_TIMB_BASE)
#define HRTIM1_TIMC             ((HRTIM_Timerx_TypeDef *) HRTIM1_TIMC_BASE)
#define HRTIM1_TIMD             ((HRTIM_Timerx_TypeDef *) HRTIM1_TIMD_BASE)
#define HRTIM1_TIME             ((HRTIM_Timerx_TypeDef *) HRTIM1_TIME_BASE)
#define HRTIM1_COMMON           ((HRTIM_Common_TypeDef *) HRTIM1_COMMON_BASE)
```

我们访问 HRTIM 的 MCR 寄存器可以采用这种形式：HRTIM1->sMasterRegs.MCR = 0;

63.3.2 定时器初始化结构体 HRTIM_InitTypeDef

此结构体主要用于 HRTIM 初始化，定义如下：

```
typedef struct
{
  uint32_t HRTIMInterruptResquests;
  uint32_t SyncOptions;
  uint32_t SyncInputSource;
  uint32_t SyncOutputSource;
  uint32_t SyncOutputPolarity;
} HRTIM_InitTypeDef;
```

下面将这几个参数一一做个说明。

◆ HRTIMInterruptResquests

用于配置支持的中断请求，支持的模式较多：

```
#define HRTIM_IT_NONE           0x00000000U          /*!< No interrupt enabled */
#define HRTIM_IT_FLT1           HRTIM_IER_FLT1       /*!< Fault 1 interrupt enable */
#define HRTIM_IT_FLT2           HRTIM_IER_FLT2       /*!< Fault 2 interrupt enable */
#define HRTIM_IT_FLT3           HRTIM_IER_FLT3       /*!< Fault 3 interrupt enable */
#define HRTIM_IT_FLT4           HRTIM_IER_FLT4       /*!< Fault 4 interrupt enable */
#define HRTIM_IT_FLT5           HRTIM_IER_FLT5       /*!< Fault 5 interrupt enable */
#define HRTIM_IT_SYSFLT         HRTIM_IER_SYSFLT     /*!< System Fault interrupt enable */
#define HRTIM_IT_BMPER          HRTIM_IER_BMPER      /*!<  Burst mode period interrupt enable */
```

◆ SyncOptions

配置 HRTIM 作为 Master，发送同步信号，或者作为 Slave，接收同步信号。

```
#define HRTIM_SYNCOPTION_NONE   0x00000000U
#define HRTIM_SYNCOPTION_MASTER 0x00000001U   /* 作为 MASTER, 同步输出 SYNCOUT */
#define HRTIM_SYNCOPTION_SLAVE  0x00000002U   /* 作为 SLAVE,  同步输入 SYNCIN  */
```

◆ SyncInputSource

设置外部同步输入源，只有当 HRTIM 配置为 Slave 时才可以使用。

```
#define HRTIM_SYNCINPUTSOURCE_NONE           0x00000000U
/* 片上定时器做同步输入源 */
#define HRTIM_SYNCINPUTSOURCE_INTERNALEVENT  HRTIM_MCR_SYNC_IN_1
/* 外部输入引脚 SYNCIN 做同步输入源 */
#define HRTIM_SYNCINPUTSOURCE_EXTERNALEVENT  (HRTIM_MCR_SYNC_IN_1 | HRTIM_MCR_SYNC_IN_0)
```

◆ SyncOutputSource

设置同步脉冲输出源，只有当 HRTIM 配置为 Master 时才可以使用。

```
/* 主定时器启动事件触发 SYNCOUT 引脚发送脉冲 */
#define HRTIM_SYNCOUTPUTSOURCE_MASTER_START 0x00000000U

/* 主定时器比较 1 事件触发 SYNCOUT 引脚发送脉冲 */
#define HRTIM_SYNCOUTPUTSOURCE_MASTER_CMP1  (HRTIM_MCR_SYNC_SRC_0)

/* Timer A 启动或者复位事件触发 SYNCOUT 引脚发送脉冲 */
#define HRTIM_SYNCOUTPUTSOURCE_TIMA_START   (HRTIM_MCR_SYNC_SRC_1)

/* Timer A 比较 1 事件触发 SYNCOUT 引脚发送脉冲 */
#define HRTIM_SYNCOUTPUTSOURCE_TIMA_CMP1    (HRTIM_MCR_SYNC_SRC_1 | HRTIM_MCR_SYNC_SRC_0)
```

◆ SyncOutputPolarity

同步输出极性。

```
/* 同步输出关闭 */
#define HRTIM_SYNCOUTPUTPOLARITY_NONE       0x00000000U

/* SCOUT 引脚空闲状态是低电平，发送一个长度为 16 个 HRTIM 时钟周期的正脉冲 */
#define HRTIM_SYNCOUTPUTPOLARITY_POSITIVE   (HRTIM_MCR_SYNC_OUT_1)

/* SCOUT 引脚空闲状态是高电平，发送一个长度为 16 个 HRTIM 时钟周期的负脉冲 */
#define HRTIM_SYNCOUTPUTPOLARITY_NEGATIVE   (HRTIM_MCR_SYNC_OUT_1 | HRTIM_MCR_SYNC_OUT_0)
```

63.3.3定时器参数结构体 HRTIM_TimerParamTypeDef

此结构体主要用于参数定义：

```
typedef struct
{
  uint32_t CaptureTrigger1;
  uint32_t CaptureTrigger2;
  uint32_t InterruptRequests;
  uint32_t DMARequests;
  uint32_t DMASrcAddress;
  uint32_t DMADstAddress;
  uint32_t DMASize;
} HRTIM_TimerParamTypeDef;
```

下面将这几个参数一 一做个说明。

- **CaptureTrigger1**

触发捕获单元 1。

当 HRTIM 工作在 Simple Mode，支持的参数如下：

```
#define HRTIM_EVENT_NONE     (0x00000000U)     /*!< Undefined event channel */
#define HRTIM_EVENT_1        (0x00000001U)     /*!< External event channel 1  identifier */
#define HRTIM_EVENT_2        (0x00000002U)     /*!< External event channel 2  identifier */
#define HRTIM_EVENT_3        (0x00000003U)     /*!< External event channel 3  identifier */
#define HRTIM_EVENT_4        (0x00000004U)     /*!< External event channel 4  identifier */
#define HRTIM_EVENT_5        (0x00000005U)     /*!< External event channel 5  identifier */
#define HRTIM_EVENT_6        (0x00000006U)     /*!< External event channel 6  identifier */
#define HRTIM_EVENT_7        (0x00000007U)     /*!< External event channel 7  identifier */
#define HRTIM_EVENT_8        (0x00000008U)     /*!< External event channel 8  identifier */
#define HRTIM_EVENT_9        (0x00000009U)     /*!< External event channel 9  identifier */
#define HRTIM_EVENT_10       (0x0000000AU)     /*!< External event channel 10 identifier */
```

当 HRTIM 工作在 Waveform Mode，支持的参数如下：

```
#define HRTIM_CAPTURETRIGGER_NONE          0x00000000U
#define HRTIM_CAPTURETRIGGER_UPDATE        (HRTIM_CPT1CR_UPDCPT)
#define HRTIM_CAPTURETRIGGER_EEV_1         (HRTIM_CPT1CR_EXEV1CPT)
#define HRTIM_CAPTURETRIGGER_EEV_2         (HRTIM_CPT1CR_EXEV2CPT)
#define HRTIM_CAPTURETRIGGER_EEV_3         (HRTIM_CPT1CR_EXEV3CPT)
#define HRTIM_CAPTURETRIGGER_EEV_4         (HRTIM_CPT1CR_EXEV4CPT)
#define HRTIM_CAPTURETRIGGER_EEV_5         (HRTIM_CPT1CR_EXEV5CPT)
#define HRTIM_CAPTURETRIGGER_EEV_6         (HRTIM_CPT1CR_EXEV6CPT)
#define HRTIM_CAPTURETRIGGER_EEV_7         (HRTIM_CPT1CR_EXEV7CPT)
#define HRTIM_CAPTURETRIGGER_EEV_8         (HRTIM_CPT1CR_EXEV8CPT)
#define HRTIM_CAPTURETRIGGER_EEV_9         (HRTIM_CPT1CR_EXEV9CPT)
#define HRTIM_CAPTURETRIGGER_EEV_10        (HRTIM_CPT1CR_EXEV10CPT)
```

```
#define HRTIM_CAPTURETRIGGER_TA1_SET      (HRTIM_CPT1CR_TA1SET)
#define HRTIM_CAPTURETRIGGER_TA1_RESET    (HRTIM_CPT1CR_TA1RST)
#define HRTIM_CAPTURETRIGGER_TIMERA_CMP1  (HRTIM_CPT1CR_TIMACMP1)
#define HRTIM_CAPTURETRIGGER_TIMERA_CMP2  (HRTIM_CPT1CR_TIMACMP2)
#define HRTIM_CAPTURETRIGGER_TB1_SET      (HRTIM_CPT1CR_TB1SET)
#define HRTIM_CAPTURETRIGGER_TB1_RESET    (HRTIM_CPT1CR_TB1RST)
#define HRTIM_CAPTURETRIGGER_TIMERB_CMP1  (HRTIM_CPT1CR_TIMBCMP1)
#define HRTIM_CAPTURETRIGGER_TIMERB_CMP2  (HRTIM_CPT1CR_TIMBCMP2)
#define HRTIM_CAPTURETRIGGER_TC1_SET      (HRTIM_CPT1CR_TC1SET)
#define HRTIM_CAPTURETRIGGER_TC1_RESET    (HRTIM_CPT1CR_TC1RST)
#define HRTIM_CAPTURETRIGGER_TIMERC_CMP1  (HRTIM_CPT1CR_TIMCCMP1)
#define HRTIM_CAPTURETRIGGER_TIMERC_CMP2  (HRTIM_CPT1CR_TIMCCMP2)
#define HRTIM_CAPTURETRIGGER_TD1_SET      (HRTIM_CPT1CR_TD1SET)
#define HRTIM_CAPTURETRIGGER_TD1_RESET    (HRTIM_CPT1CR_TD1RST)
#define HRTIM_CAPTURETRIGGER_TIMERD_CMP1  (HRTIM_CPT1CR_TIMDCMP1)
#define HRTIM_CAPTURETRIGGER_TIMERD_CMP2  (HRTIM_CPT1CR_TIMDCMP2)
#define HRTIM_CAPTURETRIGGER_TE1_SET      (HRTIM_CPT1CR_TE1SET)
#define HRTIM_CAPTURETRIGGER_TE1_RESET    (HRTIM_CPT1CR_TE1RST)
#define HRTIM_CAPTURETRIGGER_TIMERE_CMP1  (HRTIM_CPT1CR_TIMECMP1)
#define HRTIM_CAPTURETRIGGER_TIMERE_CMP2  (HRTIM_CPT1CR_TIMECMP2)
```

- **CaptureTrigger2**

  触发捕获单元 2，支持的参数与 CaptureTrigger1 一样。

- **InterruptRequests**

  定时器的中断请求使能。

- **DMASrcAddress**

  DMA 源地址。

- **DMADstAddress**

  DMA 目的地址。

- **DMASize**

  DMA 传输大小。

63.3.4定时器时基配置 HRTIM_TimeBaseCfgTypeDef

此结构体主要用于定时器时间基准配置：

```
typedef struct
{
  uint32_t Period;
  uint32_t RepetitionCounter;
  uint32_t PrescalerRatio;
  uint32_t Mode;
} HRTIM_TimeBaseCfgTypeDef;
```

下面将这几个参数一 一做个说：

- **Period**

  定时器周期，至少 3 个 HRTIM 时钟周期，最大值 0xFFDF。

- **RepetitionCounter**

重复周期计数器，最小值 0x00，最大值 0xFF。

- **PrescalerRatio**

用于设置定时器分频，对于 H7 系列，此参数仅支持分频，也就是下面参数的后三个：

```
#define HRTIM_PRESCALERRATIO_MUL32     (0x00000000U)
#define HRTIM_PRESCALERRATIO_MUL16     (0x00000001U)
#define HRTIM_PRESCALERRATIO_MUL8      (0x00000002U)
#define HRTIM_PRESCALERRATIO_MUL4      (0x00000003U)
#define HRTIM_PRESCALERRATIO_MUL2      (0x00000004U)
/* STM32H7 仅支持下面这三种 */
#define HRTIM_PRESCALERRATIO_DIV1      (0x00000005U)
#define HRTIM_PRESCALERRATIO_DIV2      (0x00000006U)
#define HRTIM_PRESCALERRATIO_DIV4      (0x00000007U)
```

- **Mode**

设置工作模式，支持的参数如下：

```
#define HRTIM_MODE_CONTINUOUS                (0x00000008U)
#define HRTIM_MODE_SINGLESHOT                (0x00000000U)
#define HRTIM_MODE_SINGLESHOT_RETRIGGERABLE  (0x00000010U)
```

63.3.5 定时器配置 HRTIM_TimerCfgTypeDef

此结构体主要用于定时器配置：

```
typedef struct
{
  /* 下面这些参数支持所有定时器，含主定时器和 Timer A 到 Timer E*/
  uint32_t InterruptRequests;
  uint32_t DMARequests;
  uint32_t DMASrcAddress;
  uint32_t DMADstAddress;
  uint32_t DMASize;
  uint32_t HalfModeEnable;
  uint32_t StartOnSync;
  uint32_t ResetOnSync;
  uint32_t DACSynchro;
  uint32_t PreloadEnable;
  uint32_t UpdateGating;
  uint32_t BurstMode;
  uint32_t RepetitionUpdate;
  /* 下面这些参数仅支持 Timer A 到 Timer E */
  uint32_t PushPull;
  uint32_t FaultEnable;
  uint32_t FaultLock;
  uint32_t DeadTimeInsertion;
  uint32_t DelayedProtectionMode;
  uint32_t UpdateTrigger;
  uint32_t ResetTrigger;
  uint32_t ResetUpdate;
} HRTIM_TimerCfgTypeDef;
```

下面将这几个参数一 一做个说：

- **InterruptRequests**

用于使能 HRTIM 的中断，支持的中断如下：

```
#define HRTIM_MASTER_IT_NONE          0x00000000U          /*!< No interrupt enabled */
#define HRTIM_MASTER_IT_MCMP1         HRTIM_MDIER_MCMP1IE  /*!< Master compare 1 interrupt enable */
#define HRTIM_MASTER_IT_MCMP2         HRTIM_MDIER_MCMP2IE  /*!< Master compare 2 interrupt enable */
#define HRTIM_MASTER_IT_MCMP3         HRTIM_MDIER_MCMP3IE  /*!< Master compare 3 interrupt enable */
#define HRTIM_MASTER_IT_MCMP4         HRTIM_MDIER_MCMP4IE  /*!< Master compare 4 interrupt enable */
#define HRTIM_MASTER_IT_MREP          HRTIM_MDIER_MREPIE   /*!< Master Repetition interrupt enable */
#define HRTIM_MASTER_IT_SYNC          HRTIM_MDIER_SYNCIE   /*!< Synchronization input interrupt enable */
#define HRTIM_MASTER_IT_MUPD          HRTIM_MDIER_MUPDIE   /*!< Master update interrupt enable */
```

- **DMARequests**

用于设置支持的 DMA 请求，此参数支持两种类型的参数，下面是 HRTIM Master 支持的请求

```
#define HRTIM_MASTER_DMA_NONE          0x00000000U           /*!< No DMA request enable */
#define HRTIM_MASTER_DMA_MCMP1         HRTIM_MDIER_MCMP1DE   /*!< Master compare 1 DMA request enable */
#define HRTIM_MASTER_DMA_MCMP2         HRTIM_MDIER_MCMP2DE   /*!< Master compare 2 DMA request enable */
#define HRTIM_MASTER_DMA_MCMP3         HRTIM_MDIER_MCMP3DE   /*!< Master compare 3 DMA request enable */
#define HRTIM_MASTER_DMA_MCMP4         HRTIM_MDIER_MCMP4DE   /*!< Master compare 4 DMA request enable */
#define HRTIM_MASTER_DMA_MREP          HRTIM_MDIER_MREPDE    /*!< Master Repetition DMA request enable */
#define HRTIM_MASTER_DMA_SYNC          HRTIM_MDIER_SYNCDE    /*!< Synchronization input DMA request enable */
#define HRTIM_MASTER_DMA_MUPD          HRTIM_MDIER_MUPDDE    /*!< Master update DMA request enable */
```

下面是 HRTIM 的 TIMER A 到 TIMER E 支持的请求:

```
#define HRTIM_TIM_DMA_NONE          0x00000000U           /*!< No DMA request enable */
#define HRTIM_TIM_DMA_CMP1          HRTIM_TIMDIER_CMP1DE  /*!< Timer compare 1 DMA request enable */
#define HRTIM_TIM_DMA_CMP2          HRTIM_TIMDIER_CMP2DE  /*!< Timer compare 2 DMA request enable */
#define HRTIM_TIM_DMA_CMP3          HRTIM_TIMDIER_CMP3DE  /*!< Timer compare 3 DMA request enable */
#define HRTIM_TIM_DMA_CMP4          HRTIM_TIMDIER_CMP4DE  /*!< Timer compare 4 DMA request enable */
#define HRTIM_TIM_DMA_REP           HRTIM_TIMDIER_REPDE   /*!< Timer repetition DMA request enable */
#define HRTIM_TIM_DMA_UPD           HRTIM_TIMDIER_UPDDE   /*!< Timer update DMA request enable */
#define HRTIM_TIM_DMA_CPT1          HRTIM_TIMDIER_CPT1DE  /*!< Timer capture 1 DMA request enable */
#define HRTIM_TIM_DMA_CPT2          HRTIM_TIMDIER_CPT2DE  /*!< Timer capture 2 DMA request enable */
#define HRTIM_TIM_DMA_SET1          HRTIM_TIMDIER_SET1DE  /*!< Timer output 1 set DMA request enable */
#define HRTIM_TIM_DMA_RST1          HRTIM_TIMDIER_RST1DE  /*!< Timer output 1 reset DMA request enable */
#define HRTIM_TIM_DMA_SET2          HRTIM_TIMDIER_SET2DE  /*!< Timer output 2 set DMA request enable */
#define HRTIM_TIM_DMA_RST2          HRTIM_TIMDIER_RST2DE  /*!< Timer output 2 reset DMA request enable */
#define HRTIM_TIM_DMA_RST           HRTIM_TIMDIER_RSTDE   /*!< Timer reset DMA request enable */
#define HRTIM_TIM_DMA_DLYPRT        HRTIM_TIMDIER_DLYPRTDE /*!< Timer delay protection DMA request enable */
```

- **DMASrcAddress**

用于设置 DMA 源地址。

- **DMADstAddress**

用于设置 DMA 目的地址。

- **DMASize**

用于设置 DMA 大小。

- **HalfModeEnable**

用于使能 Half Mode。

```
#define HRTIM_HALFMODE_DISABLED (0x00000000U)  /* 禁止 */
#define HRTIM_HALFMODE_ENABLED  (0x00000020U)  /* 使能 */
```

- **StartOnSync**

设置同步输入端接收到上升沿信号后，是否启动定时器:

```
#define HRTIM_SYNCSTART_DISABLED (0x00000000U)   /* 禁止 */
#define HRTIM_SYNCSTART_ENABLED  (HRTIM_MCR_SYNCSTRTM)   /* 使能 */
```

◆ **ResetOnSync**

设置同步输入端接收到上升沿信号后，是否复位定时器。

```
#define HRTIM_SYNCRESET_DISABLED (0x00000000U) /* 禁止 */
#define HRTIM_SYNCRESET_ENABLED  (HRTIM_MCR_SYNCRSTM) /* 使能 */
```

◆ **DACSynchro**

用于设置是否使能 DAC 同步事件。

```
/* 禁止 DAC 同步事件 */
#define HRTIM_DACSYNC_NONE          0x00000000U

/* 定时器更新时，DACTrigOut1 输出同步事件 */
#define HRTIM_DACSYNC_DACTRIGOUT_1  (HRTIM_MCR_DACSYNC_0)

/* 定时器更新时，DACTrigOut2 输出同步事件 */
#define HRTIM_DACSYNC_DACTRIGOUT_2  (HRTIM_MCR_DACSYNC_1)

/* 定时器更新时，DACTrigOut3 输出同步事件 */
#define HRTIM_DACSYNC_DACTRIGOUT_3  (HRTIM_MCR_DACSYNC_1 | HRTIM_MCR_DACSYNC_0)
```

◆ **PreloadEnable**

用于设置是否使能寄存器预加载。

```
/* 禁止预加载，写操作是直接写入到寄存器里面 */
#define HRTIM_PRELOAD_DISABLED (0x00000000U)

/* 使能预加载，写操作是写入到预加载寄存器里面 */
#define HRTIM_PRELOAD_ENABLED  (HRTIM_MCR_PREEN)
```

◆ **UpdateGating**

用于设置 HRTIM 更新方式来源。

```
/* 独立更新，不受 DMA 突发传输完成影响 */
#define HRTIM_UPDATEGATING_INDEPENDENT     0x00000000U

/* 突发传输完成时，完成更新!< Update done when the DMA burst transfer is completed */
#define HRTIM_UPDATEGATING_DMABURST        (HRTIM_TIMCR_UPDGAT_0)

/* 代码太长，其它定义省略未写 */
```

◆ **BurstMode**

设置定时器在突发模式下表现。

```
/* 定时器计数器时钟保持，定时器正常操作 */
#define HRTIM_TIMERBURSTMODE_MAINTAINCLOCK 0x00000000U

/* 定时器计数时钟停止，计数器复位 */
#define HRTIM_TIMERBURSTMODE_RESETCOUNTER  (HRTIM_BMCR_MTBM)
```

◆ **RepetitionUpdate**

设置重复计数事件是否可以触发寄存器更新。

```
#define HRTIM_UPDATEONREPETITION_DISABLED  0x00000000U       /* 禁止 */
#define HRTIM_UPDATEONREPETITION_ENABLED   (HRTIM_MCR_MREPU) /* 使能 */
```

◆ **PushPull**

设置是否使能 HRTIM 的 TIMER（A 到 E）的推挽模式。

```
#define HRTIM_TIMPUSHPULLMODE_DISABLED     0x00000000U          /* 禁止推挽输出 */
#define HRTIM_TIMPUSHPULLMODE_ENABLED      (HRTIM_TIMCR_PSHPLL) /* 使能推挽输出 */
```

◆ **FaultEnable**

使能 HRTIM TIMER 的 Fault 通道。

```
#define HRTIM_TIMFAULTENABLE_NONE      0x00000000U
#define HRTIM_TIMFAULTENABLE_FAULT1    (HRTIM_FLTR_FLT1EN)
#define HRTIM_TIMFAULTENABLE_FAULT2    (HRTIM_FLTR_FLT2EN)
#define HRTIM_TIMFAULTENABLE_FAULT3    (HRTIM_FLTR_FLT3EN)
#define HRTIM_TIMFAULTENABLE_FAULT4    (HRTIM_FLTR_FLT4EN)
#define HRTIM_TIMFAULTENABLE_FAULT5    (HRTIM_FLTR_FLT5EN)
```

◆ **FaultLock**

设置 HRTIM TIMER 的异常使能状态是否写保护。

```
#define HRTIM_TIMFAULTLOCK_READWRITE (0x00000000U)       /* 可读可写 */
#define HRTIM_TIMFAULTLOCK_READONLY  (HRTIM_FLTR_FLTLCK) /* 只读 */
```

◆ **DeadTimeInsertion**

设置 HRTIM TIMER 的死区插入。

```
#define HRTIM_TIMDEADTIMEINSERTION_DISABLED   (0x00000000U)   /* 输出 1 和输出 2 信号是独立的 */
#define HRTIM_TIMDEADTIMEINSERTION_ENABLED    HRTIM_OUTR_DTEN /* 输出 1 和输出 2 之间插入死区时间 */
```

◆ **DelayedProtectionMode**

设置 HRTIM TIMER 的延迟保护模式。

```
#define HRTIM_TIMER_A_B_C_DELAYEDPROTECTION_DISABLED          (0x00000000U)
/* 代码太长，其它定义省略未写 */
```

◆ **UpdateTrigger**

设置 Master 或者 TIMER（A 到 E）更新时，同步更新寄存器。

```
#define HRTIM_TIMUPDATETRIGGER_NONE     0x00000000U
#define HRTIM_TIMUPDATETRIGGER_MASTER   (HRTIM_TIMCR_MSTU)
#define HRTIM_TIMUPDATETRIGGER_TIMER_A  (HRTIM_TIMCR_TAU)
#define HRTIM_TIMUPDATETRIGGER_TIMER_B  (HRTIM_TIMCR_TBU)
#define HRTIM_TIMUPDATETRIGGER_TIMER_C  (HRTIM_TIMCR_TCU)
#define HRTIM_TIMUPDATETRIGGER_TIMER_D  (HRTIM_TIMCR_TDU)
#define HRTIM_TIMUPDATETRIGGER_TIMER_E  (HRTIM_TIMCR_TEU)
```

◆ **ResetTrigger**

设置那个事件可以触发定时器计数复位。

```
#define HRTIM_TIMRESETTRIGGER_NONE        0x00000000U    /* 无触发 */
/* 代码太长，其它定义省略未写 */
```

◆ **ResetUpdate**

当 HRTIM TIMER 的计数器复位时或者计数回滚到 0 时，是否触发寄存器更新。

```
#define HRTIM_TIMUPDATEONRESET_DISABLED 0x00000000U        /* 禁止 */
#define HRTIM_TIMUPDATEONRESET_ENABLED (HRTIM_TIMCR_TRSTU) /* 使能 */
```

63.3.6定时器输出配置 HRTIM_OutputCfgTypeDef

此结构体主要用于定时器输出配置：

```
typedef struct
{
  uint32_t Polarity;
  uint32_t SetSource;
```

```
  uint32_t ResetSource;
  uint32_t IdleMode;
  uint32_t IdleLevel;
  uint32_t FaultLevel;
  uint32_t ChopperModeEnable;
  uint32_t BurstModeEntryDelayed;
} HRTIM_OutputCfgTypeDef;
```

下面将这几个参数一 一做个说：

- **Polarity**

输出极性是用来设置激活状态 Active 对应的高电平还是低电平

```
#define HRTIM_OUTPUTPOLARITY_HIGH     (0x00000000U)
#define HRTIM_OUTPUTPOLARITY_LOW      (HRTIM_OUTR_POL1)
```

- **SetSource**

可以将输出由 Inactive 状态转为 Active 状态的事件，即输出置位。

```
#define HRTIM_OUTPUTSET_NONE        0x00000000U
#define HRTIM_OUTPUTSET_RESYNC      (HRTIM_SET1R_RESYNC)
#define HRTIM_OUTPUTSET_TIMPER      (HRTIM_SET1R_PER)
#define HRTIM_OUTPUTSET_TIMCMP1     (HRTIM_SET1R_CMP1)
#define HRTIM_OUTPUTSET_TIMCMP2     (HRTIM_SET1R_CMP2)
#define HRTIM_OUTPUTSET_TIMCMP3     (HRTIM_SET1R_CMP3)
#define HRTIM_OUTPUTSET_TIMCMP4     (HRTIM_SET1R_CMP4)
/* 代码太长，其它定义省略未写 */
```

- **ResetSource**

可以将输出由 Active 状态转为 Inactive 状态的事件，即输出复位。

```
#define HRTIM_OUTPUTRESET_NONE        0x00000000U
#define HRTIM_OUTPUTRESET_RESYNC      (HRTIM_RST1R_RESYNC)
#define HRTIM_OUTPUTRESET_TIMPER      (HRTIM_RST1R_PER)
#define HRTIM_OUTPUTRESET_TIMCMP1     (HRTIM_RST1R_CMP1)
#define HRTIM_OUTPUTRESET_TIMCMP2     (HRTIM_RST1R_CMP2)
#define HRTIM_OUTPUTRESET_TIMCMP3     (HRTIM_RST1R_CMP3)
#define HRTIM_OUTPUTRESET_TIMCMP4     (HRTIM_RST1R_CMP4)

/* 代码太长，其它定义省略未写 */
```

- **IdleMode**

设置进入突发模式后，输出空闲状态。

```
#define HRTIM_OUTPUTIDLEMODE_NONE   0x00000000U         /* 输出不受突发模式影响 */
#define HRTIM_OUTPUTIDLEMODE_IDLE   (HRTIM_OUTR_IDLM1) /* 进入突发模式后，输出空闲状态*/
```

- **IdleLevel**

设置输出为空闲状态时，输出电平为 Inactive 或者 Active。

```
#define HRTIM_OUTPUTIDLELEVEL_INACTIVE    0x00000000U
#define HRTIM_OUTPUTIDLELEVEL_ACTIVE    (HRTIM_OUTR_IDLES1)
```

- **FaultLevel**

设置输出为 Fault 状态时，输出电平为 Inactive，Active 或者 HIGHZ（高阻）。

```
#define HRTIM_OUTPUTFAULTLEVEL_NONE       0x00000000U
#define HRTIM_OUTPUTFAULTLEVEL_ACTIVE     (HRTIM_OUTR_FAULT1_0)
#define HRTIM_OUTPUTFAULTLEVEL_INACTIVE   (HRTIM_OUTR_FAULT1_1)
#define HRTIM_OUTPUTFAULTLEVEL_HIGHZ      (HRTIM_OUTR_FAULT1_1 | HRTIM_OUTR_FAULT1_0)
```

- **ChopperModeEnable**

设置是否使能 chopper 模式

```
#define HRTIM_OUTPUTCHOPPERMODE_DISABLED    0x00000000U       /* 禁止 */
#define HRTIM_OUTPUTCHOPPERMODE_ENABLED     (HRTIM_OUTR_CHP1) /* 使能  */
```

- **BurstModeEntryDelayed**

设置从突发模式切换到空闲模式前，是否插入一个死区时间。

```
#define HRTIM_OUTPUTBURSTMODEENTRY_REGULAR    0x00000000U         /* 立即进入空闲状态 */
#define HRTIM_OUTPUTBURSTMODEENTRY_DELAYED    (HRTIM_OUTR_DIDL1) /* 进入空闲状态前，插入死区时间 */
```

63.3.7定时器比较配置 HRTIM_CompareCfgTypeDef

此结构体主要用于定时器比较配置：

```
typedef struct
{
  uint32_t CompareValue;
  uint32_t AutoDelayedMode;
  uint32_t AutoDelayedTimeout;
} HRTIM_CompareCfgTypeDef;
```

下面将这几个参数一 一做个说：

- **CompareValue**

设置定时器比较单元的比较值。

最小值要大于等于 3 个 HRTIM 时钟周期。

最大值要小于等于 0xFFFF – 1。

- **AutoDelayedMode**

设置比较单元 2 和 4 的自动延迟模式。

```
#define HRTIM_AUTODELAYEDMODE_REGULAR                  (0x00000000U)
#define HRTIM_AUTODELAYEDMODE_AUTODELAYED_NOTIMEOUT    (HRTIM_TIMCR_DELCMP2_0)
#define HRTIM_AUTODELAYEDMODE_AUTODELAYED_TIMEOUTCMP1  (HRTIM_TIMCR_DELCMP2_1)
#define HRTIM_AUTODELAYEDMODE_AUTODELAYED_TIMEOUTCMP3  (HRTIM_TIMCR_DELCMP2_1 | HRTIM_TIMCR_DELCMP2_0)
```

- **AutoDelayedTimeout**

当选择了溢出时间的自动延迟模式时，指定计时单元 1 或 3 的比较值。

注意：CompareValue + AutoDelayedTimeout 必须小于 0xFFFFU

63.3.8定时器句柄结构体 HRTIM_HandleTypeDef

HAL 库在前面几个结构体的基础上封装了一个结构体 HRTIM_HandleTypeDef，定义如下：

```
#if (USE_HAL_HRTIM_REGISTER_CALLBACKS == 1)
typedef struct __HRTIM_HandleTypeDef
#else
typedef struct
#endif
{
  HRTIM_TypeDef *             Instance;                     /*!< Register base address */

  HRTIM_InitTypeDef           Init;                         /*!< HRTIM required parameters */
```

```
  HRTIM_TimerParamTypeDef      TimerParam[MAX_HRTIM_TIMER];  /*!< HRTIM timers              */

  HAL_LockTypeDef              Lock;                         /*!< Locking object           */

  __IO HAL_HRTIM_StateTypeDef  State;                        /*!< HRTIM communication state */

  DMA_HandleTypeDef *          hdmaMaster;                   /*!< Master timer DMA handle parameters */
  DMA_HandleTypeDef *          hdmaTimerA;                   /*!< Timer A DMA handle parameters */
  DMA_HandleTypeDef *          hdmaTimerB;                   /*!< Timer B DMA handle parameters */
  DMA_HandleTypeDef *          hdmaTimerC;                   /*!< Timer C DMA handle parameters */
  DMA_HandleTypeDef *          hdmaTimerD;                   /*!< Timer D DMA handle parameters */
  DMA_HandleTypeDef *          hdmaTimerE;                   /*!< Timer E DMA handle parameters */

#if (USE_HAL_HRTIM_REGISTER_CALLBACKS == 1)
  void (* Fault1Callback)(struct __HRTIM_HandleTypeDef *hhrtim);
  void (* Fault2Callback)(struct __HRTIM_HandleTypeDef *hhrtim);
  void (* Fault3Callback)(struct __HRTIM_HandleTypeDef *hhrtim);
  void (* Fault4Callback)(struct __HRTIM_HandleTypeDef *hhrtim);
  void (* Fault5Callback)(struct __HRTIM_HandleTypeDef *hhrtim);
  void (* SystemFaultCallback)(struct __HRTIM_HandleTypeDef *hhrtim);
  void (* BurstModePeriodCallback)(struct __HRTIM_HandleTypeDef *hhrtim);
  void (* SynchronizationEventCallback)(struct __HRTIM_HandleTypeDef *hhrtim);
  void (* ErrorCallback)(struct __HRTIM_HandleTypeDef *hhrtim);
  void (* RegistersUpdateCallback)(struct __HRTIM_HandleTypeDef *hhrtim, uint32_t TimerIdx);
  void (* RepetitionEventCallback)(struct __HRTIM_HandleTypeDef *hhrtim, uint32_t TimerIdx);
  void (* Compare1EventCallback)(struct __HRTIM_HandleTypeDef *hhrtim, uint32_t TimerIdx);
  void (* Compare2EventCallback)(struct __HRTIM_HandleTypeDef *hhrtim, uint32_t TimerIdx);
  void (* Compare3EventCallback)(struct __HRTIM_HandleTypeDef *hhrtim, uint32_t TimerIdx);
  void (* Compare4EventCallback)(struct __HRTIM_HandleTypeDef *hhrtim, uint32_t TimerIdx);
  void (* Capture1EventCallback)(struct __HRTIM_HandleTypeDef *hhrtim, uint32_t TimerIdx);
  void (* Capture2EventCallback)(struct __HRTIM_HandleTypeDef *hhrtim, uint32_t TimerIdx);
  void (* DelayedProtectionCallback)(struct __HRTIM_HandleTypeDef *hhrtim, uint32_t TimerIdx);
  void (* CounterResetCallback)(struct __HRTIM_HandleTypeDef *hhrtim, uint32_t TimerIdx);
  void (* Output1SetCallback)(struct __HRTIM_HandleTypeDef *hhrtim, uint32_t TimerIdx);
  void (* Output1ResetCallback)(struct __HRTIM_HandleTypeDef *hhrtim, uint32_t TimerIdx);
  void (* Output2SetCallback)(struct __HRTIM_HandleTypeDef *hhrtim, uint32_t TimerIdx);
  void (* Output2ResetCallback)(struct __HRTIM_HandleTypeDef *hhrtim, uint32_t TimerIdx);
  void (* BurstDMATransferCallback)(struct __HRTIM_HandleTypeDef *hhrtim, uint32_t TimerIdx);
  void (* MspInitCallback)(struct __HRTIM_HandleTypeDef *hhrtim);
  void (* MspDeInitCallback)(struct __HRTIM_HandleTypeDef *hhrtim);
#endif
} HRTIM_HandleTypeDef;
```

这里注意条件编译 USE_HAL_HRTIM_REGISTER_CALLBACKS,用来设置使用自定义回调还是使用默认回调，此定义一般放在 stm32h7xx_hal_conf.h 文件里面设置：

```
#define    USE_HAL_HRTIM_REGISTER_CALLBACKS    1
```

下面将这几个参数一 一做个说：

- **HRTIM_TypeDef * Instance;**

  这个参数是寄存器的例化，方便操作寄存器，比如设置 MCR 寄存器。

  SET_BIT(hrtim->sMasterRegs.MCR, 0)。

- **HRTIM_InitTypeDef Init**

  HRTIM 初始化配置，在本章的 63.3.2 小节进行了说明。

- **HRTIM_TimerParamTypeDef TimerParam[MAX_HRTIM_TIMER]**

  HRTIM 的主定时器和 5 个定时器单元 Timer A 到 E 的配置。在本章的 63.3.3 小节进行了说明。

- **HAL_LockTypeDef Lock**
- **__IO HAL_HRTIM_StateTypeDef State**

  这两个变量主要供函数内部使用。Lock 用于设置锁状态，State 用于设置 HRTIM 状态。

- **DMA_HandleTypeDef * hdmaMaster;**
- **DMA_HandleTypeDef * hdmaTimerA;**
- **DMA_HandleTypeDef * hdmaTimerB;**
- **DMA_HandleTypeDef * hdmaTimerC;**
- **DMA_HandleTypeDef * hdmaTimerD;**
- **DMA_HandleTypeDef * hdmaTimerE;**

  用于关联 DMA。

- **其它所有参数**

  用于自定义回调。

63.3.9定时器的底层配置（GPIO，时钟，中断等）

HAL 库有自己的底层初始化回调函数，用于 GPIO，时钟，中断等初始化，比如调用函数 HAL_HRTIM_Init 就会调用 HAL_HRTIM_MspInit，此函数是弱定义的。

```
__weak void HAL_HRTIM_MspInit(HRTIM_HandleTypeDef * hhrtim)
{
  /* Prevent unused argument(s) compilation warning */
  UNUSED(hhrtim);

  /* NOTE: This function should not be modified, when the callback is needed,
           the HAL_HRTIM_MspInit could be implemented in the user file
   */
}
```

用户可以在其它的 C 文件重定向，并将相对的底层初始化在里面实现。对应的底层复位初始化函数 HAL_HRTIM_DeInit 是在函数 HAL_HRTIM_MspDeInit 里面被调用的，也是弱定义的。

除了默认回调，用户也可为 HRTIM 句柄成员 MspInitCallback 注册一个回调函数，然后设置宏定义 USE_HAL_HRTIM_REGISTER_CALLBACKS 为 1 即可。这样以来，用户调用了 HAL_HRTIM_Init 后，此函数就会调用用户注册的回调函数。使用注册回调好处是每个定时器可以有独立的回调，无需多少个定时器共用

当然，用户也可以自己初始化，不限制必须在两个函数里面实现。

关于这个底层配置有以下几点要着重说明下：

- 定时器所使用引脚的复用模式选择已经被 HAL 库定义好，放在了 stm32h7xx_hal_gpio_ex.h 文件里

面。但是里面却有三个复用：

```
#define GPIO_AF1_HRTIM1         ((uint8_t)0x01)
#define GPIO_AF2_HRTIM1         ((uint8_t)0x02)
#define GPIO_AF3_HRTIM1         ((uint8_t)0x03)
```

具体使用哪个，配置对应引脚的复用即可：

	Port	AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7	AF8	AF9	AF10	AF11	AF12	AF13	AF14	AF15
		SYS	TIM1/2/16/17/LPTIM1/HRTIM1	SAI1/TIM3/4/5/12/HRTIM1	LPUART/TIM8/LPTIM2/3/4/5/HRTIM1/DFSDM	I2C1/2/3/4/USART1/TIM15/LPTIM2/DFSDM/CEC	SPI1/2/3/4/5/6/CEC	SPI2/3/SAI1/3/I2C4/UART4/DFSDM	SPI2/3/6/USART1/2/3/6/UART7/SDMMC1	SPI6/SAI2/4/UART4/5/8/LPUART/SDMMC1/SPDIFRX	SAI4/FDCAN1/2/TIM13/14/QUADSPI/FMC/SDMMC2/LCD/SPDIFRX	SAI2/4/TIM8/QUADSPI/SDMMC2/OTG1_HS/OTG2_FS/LCD	I2C4/UART7/SWPMI1/TIM1/8/DFSDM/SDMMC2/MDIOS/ETH	TIM1/8/FMC/SDMMC1/MDIOS/OTG1_FS/LCD	TIM1/DCMI/LCD/COMP	UART5/LCD	SYS
Port B	PB0	-	TIM1_CH2N	TIM3_CH3	TIM8_CH2N	-	-	DFSDM_CKOUT	-	UART4_CTS	LCD_R3	OTG_HS_ULPI_D1	ETH_MII_RXD2	-	-	LCD_G1	EVENTOUT
	PB1	-	TIM1_CH3N	TIM3_CH4	TIM8_CH3N	-	-	DFSDM_DATIN1	-	-	LCD_R6	OTG_HS_ULPI_D2	ETH_MII_RXD3	-	-	LCD_G0	EVENTOUT
	PB2	-	-	SAI1_D1	-	DFSDM_CKIN1	-	SAI1_SD_A	SPI3_MOSI/I2S3_SDO	SAI4_SD_A	QUADSPI_CLK	SAI4_D1	-	-	-	-	EVENTOUT
	PB3	JTDO/TRACESWO	TIM2_CH2	HRTIM_FLT4	-	-	SPI1_SCK/I2S1_CK	SPI3_SCK/I2S3_CK	-	SPI6_SCK	SDMMC2_D2	-	UART7_RX	-	-	-	EVENTOUT
	PB4	NJTRST	TIM16_BKIN	TIM3_CH1	HRTIM_EEV6	-	SPI1_MISO/I2S1_SDI	SPI3_MISO/I2S3_SDI	SPI2_NSS/I2S2_WS	SPI6_MISO	SDMMC2_D3	-	UART7_TX	-	-	-	EVENTOUT
	PB5	-	TIM17_BKIN	TIM3_CH2	HRTIM_EEV7	I2C1_SMBA	SPI1_MOSI/I2S1_SDO	I2C4_SMBA	SPI3_MOSI/I2S3_SDO	SPI6_MOSI	FDCAN2_RX	OTG_HS_ULPI_D7	ETH_PPS_OUT	FMC_SDCKE1	DCMI_D10	UART5_RX	EVENTOUT
	PB6	-	TIM16_CH1N	TIM4_CH1	HRTIM_EEV8	I2C1_SCL	HDMI_CEC	I2C4_SCL	USART1_TX	LPUART1_TX	FDCAN2_TX	QUADSPI_BK1_NCS	DFSDM_DATIN5	FMC_SDNE1	DCMI_D5	UART5_TX	EVENTOUT
	PB7	-	TIM17_CH1N	TIM4_CH2	HRTIM_EEV9	I2C1_SDA	-	I2C4_SDA	USART1_RX	LPUART1_RX	FDCAN2_TXFD_MODE	-	DFSDM_CKIN5	FMC_NL	DCMI_VSYNC	-	EVENTOUT
	PB8	-	TIM16_CH1	TIM4_CH3	DFSDM_CKIN7	I2C1_SCL	-	I2C4_SCL	SDMMC1_CKIN	UART4_RX	FDCAN1_RX	SDMMC2_D4	ETH_MII_TXD3	SDMMC1_D4	DCMI_D6	LCD_B6	EVENTOUT

63.3.10 定时器初始化流程总结

使用方法由 HAL 库提供：

- HRTIM 的 API 主要分为两类，Simple mode 和 waveform mode
  - Simple mode

    与通用定时器的相同，只是分辨率更高，支持的模式如下：

    a、输出比较模式。

    b、PWM 输出模式。

    c、输入捕获模式。

    d、单脉冲模式。

    函数以 HAL_HRTIM_Simple 开头。
  - Waveform mode

    这个模式下的 API 可以充分发挥 HRTIM 的所有特性。所有功能都可以使用，而且没有任何限制。

    函数以 HAL_HRTIM_Waveform 开头。
- 第 1 步：通过函数 HAL_HRTIM_MspInit 实现 HRTIM 的底层初始化是
  - 函数__HRTIMx_CLK_ENABLE()使能时钟。
  - 初始化使用了的 IO 引脚，__HAL_RCC_GPIOx_CLK_ENABLE 和 HAL_GPIO_Init。
  - 使用 DMA 方式控制数据传输，比如 HAL_HRTIM_SimpleBaseStart_DMA。

    a、使能 DMA 接口时钟__DMAx_CLK_ENABLE。

b、初始化 DMA 句柄。

c、调用函数__HAL_LINKDMA 关联 DMA 和 HRTIM 句柄。

d、调用函数 HAL_DMA_Init 初始化 DMA。

e、HAL_NVIC_SetPriority 配置 DMA 优先级，HAL_NVIC_EnableIRQ 使能 DMA 中断。

- 使用中断方式控制数据传输，比如 HAL_HRTIM_SimpleBaseStart_IT，
  HAL_NVIC_SetPriority 配置 DMA 优先级，HAL_NVIC_EnableIRQ 使能 DMA 中断。

◆ 第 2 步：初始化 HRTIM 使用函数 HAL_HRTIM_Init

此函数除了配置 HRTIM 句柄的结构体成员，还包括外部同步配置，HRTIM 既可以作为 master 发出同步信号，也可以作为 slave，等待接收同步信号。

◆ 第 3 步：配置 HRTIM

- 突发模式控制器
  HAL_HRTIM_BurstModeConfig 突发模式配置
- 外部事件条件
  HAL_HRTIM_EventConfig 配置外部事件条件
  HAL_HRTIM_EventPrescalerConfig 配置外部事件采样时钟
- 异常条件
  HAL_HRTIM_FaultConfig 配置异常通道
  HAL_HRTIM_FaultPrescalerConfig 预分频配置
  HAL_HRTIM_FaultModeCtl 使能或者禁止输入电路
- ADC 触发
  HAL_HRTIM_ADCTriggerConfig 配置触发 ADC

◆ 第 4 步，配置 HRTIM 时基

使用函数 HAL_HRTIM_TimeBaseConfig 配置定时器时间基准，不管用于 Simple Mode 还是 Waveform Mode，都必须调用这个函数进行配置。

- HRTIM 定时器计数操作模式。
- HRTIM 定时器时钟分频。
- HRTIM 定时器周期。
- HRTIM 重复计数器。

◆ 第 5 步，如果 HRTIM 工作在 Simple Mode 可以调用的 API

- 时间基准模式：

HAL_HRTIM_SimpleBaseStart(),HAL_HRTIM_SimpleBaseStop(),
HAL_HRTIM_SimpleBaseStart_IT(),HAL_HRTIM_SimpleBaseStop_IT(),
HAL_HRTIM_SimpleBaseStart_DMA(),HAL_HRTIM_SimpleBaseStop_DMA()。

- 输出比较:
  HAL_HRTIM_SimpleOCChannelConfig(),
  HAL_HRTIM_SimpleOCStart(),HAL_HRTIM_SimpleOCStop(),
  HAL_HRTIM_SimpleOCStart_IT(),HAL_HRTIM_SimpleOCStop_IT(),
  HAL_HRTIM_SimpleOCStart_DMA(),HAL_HRTIM_SimpleOCStop_DMA()。
- PWM 输出:
  HAL_HRTIM_SimplePWMChannelConfig(),
  HAL_HRTIM_SimplePWMStart(),HAL_HRTIM_SimplePWMStop(),
  HAL_HRTIM_SimplePWMStart_IT(),HAL_HRTIM_SimplePWMStop_IT(),
  HAL_HRTIM_SimplePWMStart_DMA(),HAL_HRTIM_SimplePWMStop_DMA()。
- 输入捕获:
  HAL_HRTIM_SimpleCaptureChannelConfig(),
  HAL_HRTIM_SimpleCaptureStart(),HAL_HRTIM_SimpleCaptureStop(),
  HAL_HRTIM_SimpleCaptureStart_IT(),HAL_HRTIM_SimpleCaptureStop_IT()。
- 单脉冲模式:
  HAL_HRTIM_SimpleOnePulseChannelConfig(),
  HAL_HRTIM_SimpleOnePulseStart(),HAL_HRTIM_SimpleOnePulseStop(),
  HAL_HRTIM_SimpleOnePulseStart_IT(),HAL_HRTIM_SimpleOnePulseStop_It()。

◆ 第 6 步，如果 HRTIM 工作在 waveform 模式可以调用的 API

- HAL_HRTIM_WaveformTimerConfig
  使能 HRTIM 中断和 DMA 请求。
  使能 HRTIM 的 HALF Mode。
  定义 HRTIM 如何响应外部同步输入。
  使能 HRTIM 的推挽模式。
  使能 HRTIM 的异常通道。
  使能 dead-time 插入。
  设置 HRTIM 的延迟保护。
  设置 HRTIM 的更新和复位触发。
  设置 HRTIM 寄存器的更新策略。
  配置外部事件消隐和开窗电路。

Blanking：在规定的时间内屏蔽外部事件。

Windowing：只能在规定的时间内使能外部事件。

- HAL_HRTIM_DeadTimeConfig

  配置 HRTIM 的死区插入，用于配置互补信号。

- HAL_HRTIM_ChopperModeConfig

  用于配置在定时之上添加的高频载波信号输出，HRTIM 的每个输出都可以单独使能或者禁止 Chopper。

- HAL_HRTIM_BurstDMAConfig

  配置 HRTIM 的 DMA 突发模式。

- HAL_HRTIM_WaveformCompareConfig

  配置 HRTIM 的比较单元。

- HAL_HRTIM_WaveformCaptureConfig

  配置 HRTIM 的捕获单元。

- HAL_HRTIM_WaveformOutputConfig

  配置 HRTIM 输出。

- HAL_HRTIM_WaveformSetOutputLevel

  配置激活或者非激活态。

- 使能或者禁止 waveform timer 输出

  HAL_HRTIM_WaveformOutputStart(),HAL_HRTIM_WaveformOutputStop()。

- 启动或者禁止 HRTIM

  HAL_HRTIM_WaveformCountStart(), HAL_HRTIM_WaveformCountStop(),
  HAL_HRTIM_WaveformCountStart_IT(), HAL_HRTIM_WaveformCountStop_IT(),
  HAL_HRTIM_WaveformCountStart_DMA(), HAL_HRTIM_WaveformCountStop_DMA()。

- 突发模式控制器使能:

  HAL_HRTIM_BurstModeCtl()。

- 软件触发 ：

  HAL_HRTIM_BurstModeSoftwareTrigger()
  HAL_HRTIM_SoftwareCapture()
  HAL_HRTIM_SoftwareUpdate()
  HAL_HRTIM_SoftwareReset()。

◆ 第 7 步，根据需要可以做动态注册回调

首先使能宏定义 USE_HAL_HRTIM_REGISTER_CALLBACKS。

然后调用函数 HAL_HRTIM_RegisterCallback() 就可以注册如下回调函数：

(+) Fault1Callback
(+) Fault2Callback
(+) Fault3Callback
(+) Fault4Callback
(+) Fault5Callback
(+) SystemFaultCallback
(+) BurstModePeriodCallback
(+) SynchronizationEventCallback
(+) ErrorCallback
(+) MspInitCallback
(+) MspDeInitCallback

函数 HAL_HRTIM_TIMxRegisterCallback 允许注册的回调函数如下:

(+) RegistersUpdateCallback
(+) RepetitionEventCallback
(+) Compare1EventCallback
(+) Compare2EventCallback
(+) Compare3EventCallback
(+) Compare4EventCallback
(+) Capture1EventCallback
(+) Capture2EventCallback
(+) DelayedProtectionCallback
(+) CounterResetCallback
(+) Output1SetCallback
(+) Output1ResetCallback
(+) Output2SetCallback
(+) Output2ResetCallback
(+) BurstDMATransferCallback

关于动态注册回调函数注意以下几点:

- 卸载注册使用 HAL_HRTIM_UnRegisterCallback 和 HAL_HRTIM_TIMxUnRegisterCallback
- 默认情况下，HAL_HRTIM_Init 调用后将使用默认的弱定义回调，如果用户注册了回调，将使用用户设置的。
- 回调函数只能在 HAL_HRTIM_STATE_READY 状态下才可以注册/注销。

- 回调函数 MspInit 和 MspDeInit 除外，这两个函数可以在 HAL_HRTIM_STATE_READY 或 HAL_HRTIM_STATE_RESET 状态下注册，这样的话，用户调用函数 HAL_HRTIM_DeInit()或者 HAL_HRTIM_Init()时，就可以在其函数内运行 MspInit/DeInit。
- 用户可以在调用 HAL_HRTIM_DeInit()或者 HAL_HRTIM_Init()之前调用 HAL_HRTIM_RegisterCallback()为 MspInit/MspDeInit 注册回调。

HRTIM 常用的功能，通过上面这几步即可实现。

63.4 源文件 stm32h7xx_hal_hrtim.c

此文件涉及到的函数非常多，这里把几个常用的函数做个说明：

- HAL_HRTIM_Init
- HAL_HRTIM_TimeBaseConfig
- HAL_HRTIM_WaveformTimerConfig
- HAL_HRTIM_WaveformCompareConfig
- HAL_HRTIM_WaveformOutputConfig
- HAL_HRTIM_WaveformOutputStart
- HAL_HRTIM_WaveformCounterStart

63.4.1 函数 HAL_HRTIM_Init

函数原型：

```
HAL_StatusTypeDef HAL_HRTIM_Init(HRTIM_HandleTypeDef * hhrtim)
{
  uint8_t timer_idx;
  uint32_t hrtim_mcr;

  /* 检查句柄是否有效 */
  if(hhrtim == NULL)
  {
    return HAL_ERROR;
  }

  /* 检测参数 */
  assert_param(IS_HRTIM_ALL_INSTANCE(hhrtim->Instance));
  assert_param(IS_HRTIM_IT(hhrtim->Init.HRTIMInterruptResquests));

  /* 使用注册回调，这样每个定时器可以有独立的回调，无需多个定时器共用 */
#if (USE_HAL_HRTIM_REGISTER_CALLBACKS == 1)
  if (hhrtim->State == HAL_HRTIM_STATE_RESET)
  {
    /* 初始化回调函数到默认值，
       也就是说，如果用户使能了回调函数的注册功能，但是没有指定具体回调，就是用默认的 */
    hhrtim->Fault1Callback              = HAL_HRTIM_Fault1Callback;
```

```
    hhrtim->Fault2Callback                = HAL_HRTIM_Fault2Callback;
    hhrtim->Fault3Callback                = HAL_HRTIM_Fault3Callback;
    hhrtim->Fault4Callback                = HAL_HRTIM_Fault4Callback;
    hhrtim->Fault5Callback                = HAL_HRTIM_Fault5Callback;
    hhrtim->SystemFaultCallback           = HAL_HRTIM_SystemFaultCallback;
    hhrtim->BurstModePeriodCallback       = HAL_HRTIM_BurstModePeriodCallback;
    hhrtim->SynchronizationEventCallback = HAL_HRTIM_SynchronizationEventCallback;
    hhrtim->ErrorCallback                 = HAL_HRTIM_ErrorCallback;
    hhrtim->RegistersUpdateCallback       = HAL_HRTIM_RegistersUpdateCallback;
    hhrtim->RepetitionEventCallback       = HAL_HRTIM_RepetitionEventCallback;
    hhrtim->Compare1EventCallback         = HAL_HRTIM_Compare1EventCallback;
    hhrtim->Compare2EventCallback         = HAL_HRTIM_Compare2EventCallback;
    hhrtim->Compare3EventCallback         = HAL_HRTIM_Compare3EventCallback;
    hhrtim->Compare4EventCallback         = HAL_HRTIM_Compare4EventCallback;
    hhrtim->Capture1EventCallback         = HAL_HRTIM_Capture1EventCallback;
    hhrtim->Capture2EventCallback         = HAL_HRTIM_Capture2EventCallback;
    hhrtim->DelayedProtectionCallback     = HAL_HRTIM_DelayedProtectionCallback;
    hhrtim->CounterResetCallback          = HAL_HRTIM_CounterResetCallback;
    hhrtim->Output1SetCallback            = HAL_HRTIM_Output1SetCallback;
    hhrtim->Output1ResetCallback          = HAL_HRTIM_Output1ResetCallback;
    hhrtim->Output2SetCallback            = HAL_HRTIM_Output2SetCallback;
    hhrtim->Output2ResetCallback          = HAL_HRTIM_Output2ResetCallback;
    hhrtim->BurstDMATransferCallback      = HAL_HRTIM_BurstDMATransferCallback;

    if (hhrtim->MspInitCallback == NULL)
    {
      hhrtim->MspInitCallback = HAL_HRTIM_MspInit;
    }
  }
#endif /* USE_HAL_HRTIM_REGISTER_CALLBACKS */

  /* 设置 HRTIM 忙 */
  hhrtim->State = HAL_HRTIM_STATE_BUSY;

  /* 初始化 DMA 句柄 */
  hhrtim->hdmaMaster = (DMA_HandleTypeDef *)NULL;
  hhrtim->hdmaTimerA = (DMA_HandleTypeDef *)NULL;
  hhrtim->hdmaTimerB = (DMA_HandleTypeDef *)NULL;
  hhrtim->hdmaTimerC = (DMA_HandleTypeDef *)NULL;
  hhrtim->hdmaTimerD = (DMA_HandleTypeDef *)NULL;
  hhrtim->hdmaTimerE = (DMA_HandleTypeDef *)NULL;

  /* HRTIM 输出同步配置 */
  if ((hhrtim->Init.SyncOptions & HRTIM_SYNCOPTION_MASTER) != (uint32_t)RESET)
  {
    /* 检测参数 */
    assert_param(IS_HRTIM_SYNCOUTPUTSOURCE(hhrtim->Init.SyncOutputSource));
    assert_param(IS_HRTIM_SYNCOUTPUTPOLARITY(hhrtim->Init.SyncOutputPolarity));

    /* 同步输出初始化必须优先于 MCU 输出配置(通过 HAL_HRTIM_MspInit) */
    if (hhrtim->Instance == HRTIM1)
    {
      /* 使能 HRTIM 外设时钟 */
      __HAL_RCC_HRTIM1_CLK_ENABLE();
    }

    hrtim_mcr = hhrtim->Instance->sMasterRegs.MCR;
```

```
    /* 设置同步输出事件 */
    hrtim_mcr &= ~(HRTIM_MCR_SYNC_SRC);
    hrtim_mcr |= (hhrtim->Init.SyncOutputSource & HRTIM_MCR_SYNC_SRC);

    /* 设置同步输出极性 */
    hrtim_mcr &= ~(HRTIM_MCR_SYNC_OUT);
    hrtim_mcr |= (hhrtim->Init.SyncOutputPolarity & HRTIM_MCR_SYNC_OUT);

    /* 更新 HRTIM 寄存器 */
    hhrtim->Instance->sMasterRegs.MCR = hrtim_mcr;
  }

  /* 初始化底层硬件: GPIO, CLOCK, NVIC 和 DMA */
#if (USE_HAL_HRTIM_REGISTER_CALLBACKS == 1)
  hhrtim->MspInitCallback(hhrtim);
#else
  HAL_HRTIM_MspInit(hhrtim);
#endif /* USE_HAL_HRTIM_REGISTER_CALLBACKS */

  /* HRTIM 同步输入配置 HRTIM */
  if ((hhrtim->Init.SyncOptions & HRTIM_SYNCOPTION_SLAVE) != (uint32_t)RESET)
  {
    /* 检测参数 */
    assert_param(IS_HRTIM_SYNCINPUTSOURCE(hhrtim->Init.SyncInputSource));

    hrtim_mcr = hhrtim->Instance->sMasterRegs.MCR;

    /* 设置同步输入源 */
    hrtim_mcr &= ~(HRTIM_MCR_SYNC_IN);
    hrtim_mcr |= (hhrtim->Init.SyncInputSource & HRTIM_MCR_SYNC_IN);

    /* 更新 HRTIM 寄存器 */
    hhrtim->Instance->sMasterRegs.MCR = hrtim_mcr;
  }

  /* 设置 HRTIM 寄存器就绪 */
  hhrtim->State = HAL_HRTIM_STATE_READY;

  /* 初始化 HRTIM HAL API 的锁状态，即开锁 */
  __HAL_UNLOCK(hhrtim);

  /* 初始化 HRTIM TIMER 相关参数 */
  for (timer_idx = HRTIM_TIMERINDEX_TIMER_A ;
       timer_idx <= HRTIM_TIMERINDEX_MASTER ;
       timer_idx++)
  {
    hhrtim->TimerParam[timer_idx].CaptureTrigger1 = HRTIM_CAPTURETRIGGER_NONE;
    hhrtim->TimerParam[timer_idx].CaptureTrigger2 = HRTIM_CAPTURETRIGGER_NONE;
    hhrtim->TimerParam[timer_idx].InterruptRequests = HRTIM_IT_NONE;
    hhrtim->TimerParam[timer_idx].DMARequests = HRTIM_IT_NONE;
    hhrtim->TimerParam[timer_idx].DMASrcAddress = 0U;
    hhrtim->TimerParam[timer_idx].DMASize = 0U;
  }

  return HAL_OK;
}
```

函数描述:

此函数用于初始化 HRTIM。

函数参数:

- 第 1 个参数是 HRTIM_HandleTypeDef 类型结构体指针变量。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项:

1. 宏定义 USE_HAL_HRTIM_REGISTER_CALLBACKS 可以放在 stm32h7xx_hal_conf.h 文件里面使能
   #define USE_HAL_HRTIM_REGISTER_CALLBACKS 1
   使用注册回调好处是每个定时器可以有独立的回调，无需多个定时器共用。
2. 函数 HAL_HRTIM_MspInit 用于初始化定时器的底层时钟、引脚等功能。需要用户自己在此函数里面实现具体的功能，由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
3. 如果形参 htim 的结构体成员 State 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 HRTIM_HandleTypeDef HRTimHandle。
   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_HRTIM_STATE_RESET = 0x00U。
   解决办法有三:
   方法 1：用户自己初始定时器和涉及到的 GPIO 等。
   方法 2：定义 HRTIM_HandleTypeDef HRTimHandle 为全局变量。
   方法 3：下面的方法

```
if(HAL_HRTIM_DeInit(&HRTimHandle) != HAL_OK)
{
    Error_Handler();
}
if(HAL_HRTIM_Init(&HRTimHandle) != HAL_OK)
{
    Error_Handler();
}
```

使用举例:

```
HRTIM_HandleTypeDef  HrtimHandle = {0};

/* 例化，使用的 HRTIM1 */
HrtimHandle.Instance = HRTIM1;

/* 用于配置支持的中断请求，当前配置无中断 */
HrtimHandle.Init.HRTIMInterruptResquests = HRTIM_IT_NONE;

/* 配置 HRTIM 作为 Master，发送同步信号，或者作为 Slave，接收同步信号，当前配置没有做同步功能 */
HrtimHandle.Init.SyncOptions = HRTIM_SYNCOPTION_NONE;

HAL_HRTIM_Init(&HrtimHandle);
```

63.4.2 函数 HAL_HRTIM_TimeBaseConfig

函数原型：

```
HAL_StatusTypeDef HAL_HRTIM_TimeBaseConfig(HRTIM_HandleTypeDef *hhrtim,
                                           uint32_t TimerIdx,
                                           HRTIM_TimeBaseCfgTypeDef * pTimeBaseCfg)
{
  /* 检查参数 */
  assert_param(IS_HRTIM_TIMERINDEX(TimerIdx));
  assert_param(IS_HRTIM_PRESCALERRATIO(pTimeBaseCfg->PrescalerRatio));
  assert_param(IS_HRTIM_MODE(pTimeBaseCfg->Mode));

  if(hhrtim->State == HAL_HRTIM_STATE_BUSY)
  {
     return HAL_BUSY;
  }

  /* 设置 HRTIM 状态忙 */
  hhrtim->State = HAL_HRTIM_STATE_BUSY;

  if (TimerIdx == HRTIM_TIMERINDEX_MASTER)
  {
    /* 配置 MASTER TIMER 时基 */
    HRTIM_MasterBase_Config(hhrtim, pTimeBaseCfg);
  }
  else
  {
    /* 配置 Timing 单元时基（Timer A 到 Timer E） */
    HRTIM_TimingUnitBase_Config(hhrtim, TimerIdx, pTimeBaseCfg);
  }

  /* 设置 HRTIM 就绪 */
  hhrtim->State = HAL_HRTIM_STATE_READY;

  return HAL_OK;
}
```

函数描述：

配置定时器时基单元。

函数参数：

- 第 1 个参数是 HRTIM_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是使用的定时器索引，支持的参数如下：
  - HRTIM_TIMERINDEX_MASTER。
  - HRTIM_TIMERINDEX_TIMER_A。
  - HRTIM_TIMERINDEX_TIMER_B。
  - HRTIM_TIMERINDEX_TIMER_C。
  - HRTIM_TIMERINDEX_TIMER_D。
  - HRTIM_TIMERINDEX_TIMER_E。

- 第 3 个参数是 HRTIM_TimeBaseCfgTypeDef 类型结构指针变量。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 启动定时器前，优先调用此函数。
2. 配置定时器计数器工作在连续或者单次模式以及时钟分频，周期和重复计数器。

使用举例：

```
HRTIM_TimeBaseCfgTypeDef     sConfig_time_base = {0};

/*
    PWM 的频率 = 400MHz / HRTIM_TIMD_PERIOD
               = 400000000 / 4000
               = 100KHz
*/
sConfig_time_base.Mode = HRTIM_MODE_CONTINUOUS; /* 连续工作模式 */
sConfig_time_base.Period = HRTIM_TIMD_PERIOD;   /* 设置周期 */

/* 设置 HRTIM 分频，当前设置的 1 分频，也就是不分频 */
sConfig_time_base.PrescalerRatio = HRTIM_PRESCALERRATIO_DIV1;
sConfig_time_base.RepetitionCounter = 0; /* 设置重复计数器为 0，即不做重复计数 */

HAL_HRTIM_TimeBaseConfig(&HrtimHandle, HRTIM_TIMERINDEX_TIMER_D, &sConfig_time_base);
```

63.4.3 函数 HAL_HRTIM_WaveformTimerConfig

函数原型：

```
HAL_StatusTypeDef HAL_HRTIM_WaveformTimerConfig(HRTIM_HandleTypeDef * hhrtim,
                                                uint32_t TimerIdx,
                                                HRTIM_TimerCfgTypeDef * pTimerCfg)
{
  /* 检查参数是否有效  */
  assert_param(IS_HRTIM_TIMERINDEX(TimerIdx));
  assert_param(IS_HRTIM_HALFMODE(pTimerCfg->HalfModeEnable));
  assert_param(IS_HRTIM_SYNCSTART(pTimerCfg->StartOnSync));
  assert_param(IS_HRTIM_SYNCRESET(pTimerCfg->ResetOnSync));
  assert_param(IS_HRTIM_DACSYNC(pTimerCfg->DACSynchro));
  assert_param(IS_HRTIM_PRELOAD(pTimerCfg->PreloadEnable));
  assert_param(IS_HRTIM_TIMERBURSTMODE(pTimerCfg->BurstMode));
  assert_param(IS_HRTIM_UPDATEONREPETITION(pTimerCfg->RepetitionUpdate));

  /* 检测 HRTIM 是否处于忙状态 */
  if(hhrtim->State == HAL_HRTIM_STATE_BUSY)
  {
     return HAL_BUSY;
  }

  /* 操作期间，上锁 */
  __HAL_LOCK(hhrtim);

  hhrtim->State = HAL_HRTIM_STATE_BUSY;
```

```
  if (TimerIdx == HRTIM_TIMERINDEX_MASTER)
  {
    /* 检查参数 */
    assert_param(IS_HRTIM_UPDATEGATING_MASTER(pTimerCfg->UpdateGating));
    assert_param(IS_HRTIM_MASTER_IT(pTimerCfg->InterruptRequests));
    assert_param(IS_HRTIM_MASTER_DMA(pTimerCfg->DMARequests));

    /* 配置 Master Timer */
    HRTIM_MasterWaveform_Config(hhrtim, pTimerCfg);
  }
  else
  {
    /* 检查参数 */
    assert_param(IS_HRTIM_UPDATEGATING_TIM(pTimerCfg->UpdateGating));
    assert_param(IS_HRTIM_TIM_IT(pTimerCfg->InterruptRequests));
    assert_param(IS_HRTIM_TIM_DMA(pTimerCfg->DMARequests));
    assert_param(IS_HRTIM_TIMPUSHPULLMODE(pTimerCfg->PushPull));
    assert_param(IS_HRTIM_TIMFAULTENABLE(pTimerCfg->FaultEnable));
    assert_param(IS_HRTIM_TIMFAULTLOCK(pTimerCfg->FaultLock));
    assert_param(IS_HRTIM_TIMDEADTIMEINSERTION(pTimerCfg->PushPull,
                                               pTimerCfg->DeadTimeInsertion));
    assert_param(IS_HRTIM_TIMDELAYEDPROTECTION(pTimerCfg->PushPull,
                                               pTimerCfg->DelayedProtectionMode));
    assert_param(IS_HRTIM_TIMUPDATETRIGGER(pTimerCfg->UpdateTrigger));
    assert_param(IS_HRTIM_TIMRESETTRIGGER(pTimerCfg->ResetTrigger));
    assert_param(IS_HRTIM_TIMUPDATEONRESET(pTimerCfg->ResetUpdate));

    /* 配置时序单元，从 TIMER A 到 TIMER D */
    HRTIM_TimingUnitWaveform_Config(hhrtim, TimerIdx, pTimerCfg);
  }

  /* 更新定时器参数 */
  hhrtim->TimerParam[TimerIdx].InterruptRequests = pTimerCfg->InterruptRequests;
  hhrtim->TimerParam[TimerIdx].DMARequests = pTimerCfg->DMARequests;
  hhrtim->TimerParam[TimerIdx].DMASrcAddress = pTimerCfg->DMASrcAddress;
  hhrtim->TimerParam[TimerIdx].DMADstAddress = pTimerCfg->DMADstAddress;
  hhrtim->TimerParam[TimerIdx].DMASize = pTimerCfg->DMASize;

  /* 强制软件更新 */
  HRTIM_ForceRegistersUpdate(hhrtim, TimerIdx);

  hhrtim->State = HAL_HRTIM_STATE_READY;

  /* 解锁 */
  __HAL_UNLOCK(hhrtim);

  return HAL_OK;
}
```

函数描述：

此函数用于配置 HRTIM 在 waveform 模式的工作方式。

函数参数：

- 第 1 个参数是 HRTIM_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是使用的定时器索引，支持的参数如下：

- HRTIM_TIMERINDEX_MASTER。
- HRTIM_TIMERINDEX_TIMER_A。
- HRTIM_TIMERINDEX_TIMER_B。
- HRTIM_TIMERINDEX_TIMER_C。
- HRTIM_TIMERINDEX_TIMER_D。
- HRTIM_TIMERINDEX_TIMER_E。

◆ 第 3 个参数是 HRTIM_TimerCfgTypeDef 类型结构指针变量。

◆ 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 启动定时器前，优先调用此函数。
2. HRTIM 工作在 waveform 模式才能发挥 HRTIM 全功能特性。

使用举例：

```
HRTIM_TimerCfgTypeDef        sConfig_timerD = {0};

sConfig_timerD.DMARequests = HRTIM_TIM_DMA_NONE;         /* 不使用 DMA */
sConfig_timerD.HalfModeEnable = HRTIM_HALFMODE_DISABLED;/* 关闭 HALF 模式 */
sConfig_timerD.StartOnSync = HRTIM_SYNCSTART_DISABLED;   /* 设置同步输入端接收到上升沿信号后,不启动定时器 */
sConfig_timerD.ResetOnSync = HRTIM_SYNCRESET_DISABLED;   /* 设置同步输入端接收到上升沿信号后,不复位定时器 */
sConfig_timerD.DACSynchro = HRTIM_DACSYNC_NONE;          /* 不使用 DAC 同步事件 */
sConfig_timerD.PreloadEnable = HRTIM_PRELOAD_ENABLED;     /* 使能寄存器预加载 */
sConfig_timerD.UpdateGating = HRTIM_UPDATEGATING_INDEPENDENT;      /* 独立更新，与 DMA 突发传输完成无关 */
sConfig_timerD.BurstMode = HRTIM_TIMERBURSTMODE_MAINTAINCLOCK;     /* 在突发模式下，定时器正常运行 */
sConfig_timerD.RepetitionUpdate = HRTIM_UPDATEONREPETITION_ENABLED;/* 设置重计数器事件可以触发寄存器更新 */
/* 当 HRTIM TIMER 的计数器复位时或者计数回滚到 0 时，不触发寄存器更新 */
sConfig_timerD.ResetUpdate = HRTIM_TIMUPDATEONRESET_DISABLED;
sConfig_timerD.InterruptRequests = HRTIM_TIM_IT_NONE;            /* 不使用中断 */
sConfig_timerD.PushPull = HRTIM_TIMPUSHPULLMODE_DISABLED;        /* 不开启推挽模式 */
sConfig_timerD.FaultEnable = HRTIM_TIMFAULTENABLE_NONE;          /* 不使用 HRTIM TIMER 的 Fault 通道 */
sConfig_timerD.FaultLock = HRTIM_TIMFAULTLOCK_READWRITE;         /* 不开启 HRTIM TIMER 的异常使能状态写保护 */
sConfig_timerD.DeadTimeInsertion = HRTIM_TIMDEADTIMEINSERTION_DISABLED;/* 不开启死区时间插入 */
/* 不开启 HRTIM TIMER 的延迟保护模式 */
sConfig_timerD.DelayedProtectionMode = HRTIM_TIMER_D_E_DELAYEDPROTECTION_DISABLED;
/* Master 或 TIMER（A 到 E）更新时，不同步更新寄存器 */
sConfig_timerD.UpdateTrigger= HRTIM_TIMUPDATETRIGGER_NONE;
sConfig_timerD.ResetTrigger = HRTIM_TIMRESETTRIGGER_NONE; /* 无复位触发 */
HAL_HRTIM_WaveformTimerConfig(&HrtimHandle, HRTIM_TIMERINDEX_TIMER_D, &sConfig_timerD);
```

63.4.4函数 HAL_HRTIM_WaveformCompareConfig

函数原型：

```
HAL_StatusTypeDef HAL_HRTIM_WaveformCompareConfig(HRTIM_HandleTypeDef * hhrtim,
                                                  uint32_t TimerIdx,
                                                  uint32_t CompareUnit,
                                                  HRTIM_CompareCfgTypeDef* pCompareCfg)
{
```

```
/* 检查参数 */
assert_param(IS_HRTIM_TIMERINDEX(TimerIdx));

if(hhrtim->State == HAL_HRTIM_STATE_BUSY)
{
   return HAL_BUSY;
}

/* 处理过程上锁 */
__HAL_LOCK(hhrtim);

hhrtim->State = HAL_HRTIM_STATE_BUSY;

/* 配置比较单元 */
if (TimerIdx == HRTIM_TIMERINDEX_MASTER)
{
  /* 省略 switch 里面的具体内容 */
  switch (CompareUnit)
  {
    case HRTIM_COMPAREUNIT_1:

    case HRTIM_COMPAREUNIT_2:

    case HRTIM_COMPAREUNIT_3:

    case HRTIM_COMPAREUNIT_4:

    default:

  }

  if(hhrtim->State == HAL_HRTIM_STATE_ERROR)
  {
   return HAL_ERROR;
  }

}
else
{
   /* 省略 switch 里面的具体内容 */
  switch (CompareUnit)
  {
  case HRTIM_COMPAREUNIT_1:

  case HRTIM_COMPAREUNIT_2:

  case HRTIM_COMPAREUNIT_3:

  case HRTIM_COMPAREUNIT_4:


  default:
 }

 if(hhrtim->State == HAL_HRTIM_STATE_ERROR)
 {
   return HAL_ERROR;
 }
```

```
  }
  hhrtim->State = HAL_HRTIM_STATE_READY;

  /* 解锁 */
  __HAL_UNLOCK(hhrtim);

  return HAL_OK;
}
```

函数描述：

此函数用于配置 HRTIM 的定时器比较单元。

函数参数：

- 第 1 个参数是 HRTIM_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是使用的定时器索引，支持的参数如下：
  - HRTIM_TIMERINDEX_MASTER。
  - HRTIM_TIMERINDEX_TIMER_A。
  - HRTIM_TIMERINDEX_TIMER_B。
  - HRTIM_TIMERINDEX_TIMER_C。
  - HRTIM_TIMERINDEX_TIMER_D。
  - HRTIM_TIMERINDEX_TIMER_E
- 第 3 个参数是比较单元，支持的参数如下：
  - HRTIM_COMPAREUNIT_1。
  - HRTIM_COMPAREUNIT_2。
  - HRTIM_COMPAREUNIT_3。
  - HRTIM_COMPAREUNIT_4。
- 第 4 个参数是 HRTIM_CompareCfgTypeDef 类型结构体变量。
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 启动定时器前，优先调用此函数。
2. 如果比较单元 2 或比较单元 4 需要自动延迟模式，应用程序必须单独配置捕获单元。
   - 输出比较 2 的自动延迟：捕获单元 1 必须配置。
   - 输出比较 4 的自定延迟：捕获单元 2 必须配置。

使用举例：

```
HRTIM_CompareCfgTypeDef     sConfig_compare = {0};

sConfig_compare.AutoDelayedMode = HRTIM_AUTODELAYEDMODE_REGULAR; /* 这里使用标准模式，即未使用自动延迟 */
sConfig_compare.AutoDelayedTimeout = 0;     /* 由于前面的参数未使用自动延迟模式，此参数无作用 */
/*
    设置定时器比较单元的比较值：
    最小值要大于等于 3 个 HRTIM 时钟周期。
```

```
    最大值要小于等于 0xFFFF - 1
*/
sConfig_compare.CompareValue = 4000 / 2;  /* 占空比 50% */
HAL_HRTIM_WaveformCompareConfig(&HrtimHandle, HRTIM_TIMERINDEX_TIMER_D, HRTIM_COMPAREUNIT_1,
                                                                        &sConfig_compare);
```

63.4.5 函数 HAL_HRTIM_WaveformOutputConfig

函数原型：

```
HAL_StatusTypeDef HAL_HRTIM_WaveformOutputConfig(HRTIM_HandleTypeDef * hhrtim,
                                                 uint32_t TimerIdx,
                                                 uint32_t Output,
                                                 HRTIM_OutputCfgTypeDef * pOutputCfg)
{
  /* 检查参数 */
  assert_param(IS_HRTIM_TIMER_OUTPUT(TimerIdx, Output));
  assert_param(IS_HRTIM_OUTPUTPOLARITY(pOutputCfg->Polarity));
  assert_param(IS_HRTIM_OUTPUTIDLELEVEL(pOutputCfg->IdleLevel));
  assert_param(IS_HRTIM_OUTPUTIDLEMODE(pOutputCfg->IdleMode));
  assert_param(IS_HRTIM_OUTPUTFAULTLEVEL(pOutputCfg->FaultLevel));
  assert_param(IS_HRTIM_OUTPUTCHOPPERMODE(pOutputCfg->ChopperModeEnable));
  assert_param(IS_HRTIM_OUTPUTBURSTMODEENTRY(pOutputCfg->BurstModeEntryDelayed));

  if(hhrtim->State == HAL_HRTIM_STATE_BUSY)
  {
     return HAL_BUSY;
  }

  /* 处理过程上锁 */
  __HAL_LOCK(hhrtim);

  hhrtim->State = HAL_HRTIM_STATE_BUSY;

  /* 配置定时器输出 */
  HRTIM_OutputConfig(hhrtim,
                     TimerIdx,
                     Output,
                     pOutputCfg);

  hhrtim->State = HAL_HRTIM_STATE_READY;

  /* 解锁 */
  __HAL_UNLOCK(hhrtim);

  return HAL_OK;
}
```

函数描述：

此函数用于配置 HRTIM 的定时器（TIMER A 到 TIMER E）在 waveform 模式下的输出。

函数参数：

- 第 1 个参数是 HRTIM_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是使用的定时器索引，支持的参数如下：

- HRTIM_TIMERINDEX_TIMER_A。
- HRTIM_TIMERINDEX_TIMER_B。
- HRTIM_TIMERINDEX_TIMER_C。
- HRTIM_TIMERINDEX_TIMER_D。
- HRTIM_TIMERINDEX_TIMER_E

◆ 第 3 个参数是具体的输出通道，支持的参数如下：

- HRTIM_OUTPUT_TA1: Timer A - Output 1
- HRTIM_OUTPUT_TA2: Timer A - Output 2
- HRTIM_OUTPUT_TB1: Timer B - Output 1
- HRTIM_OUTPUT_TB2: Timer B - Output 2
- HRTIM_OUTPUT_TC1: Timer C - Output 1
- HRTIM_OUTPUT_TC2: Timer C - Output 2
- HRTIM_OUTPUT_TD1: Timer D - Output 1
- HRTIM_OUTPUT_TD2: Timer D - Output 2
- HRTIM_OUTPUT_TE1: Timer E - Output 1
- HRTIM_OUTPUT_TE2: Timer E - Output 2

◆ 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

注意事项：

1. 配置定时器前，必须调用此函数。并且在配置死区时间插入之前调用（如果需要配置死区时间）。

使用举例：

```
HRTIM_OutputCfgTypeDef  sConfig_output_config = {0};

sConfig_output_config.Polarity = HRTIM_OUTPUTPOLARITY_LOW;     /* 设置定时器输出极性 */
sConfig_output_config.SetSource = HRTIM_OUTPUTRESET_TIMCMP1;   /* 定时器比较事件 1 可以将输出置位 */
sConfig_output_config.ResetSource = HRTIM_OUTPUTSET_TIMPER;    /* 定时器周期性更新事件可以将输出清零 */
sConfig_output_config.IdleMode = HRTIM_OUTPUTIDLEMODE_NONE;    /* 输出不受突发模式影响 */
sConfig_output_config.IdleLevel = HRTIM_OUTPUTIDLELEVEL_INACTIVE; /* 设置空闲状态输出低电平 */
sConfig_output_config.FaultLevel = HRTIM_OUTPUTFAULTLEVEL_NONE;    /* 输出不受异常输入影响 */
sConfig_output_config.ChopperModeEnable = HRTIM_OUTPUTCHOPPERMODE_DISABLED; /* 关闭 Chopper 模式 */
sConfig_output_config.BurstModeEntryDelayed = HRTIM_OUTPUTBURSTMODEENTRY_REGULAR; /* 设置从突发模式切换到空
                                                                                    闲模式，不插入死区时间 */

HAL_HRTIM_WaveformOutputConfig(&HrtimHandle, HRTIM_TIMERINDEX_TIMER_D, HRTIM_OUTPUT_TD1,
                                                                       &sConfig_output_config);
```

63.4.6函数 HAL_HRTIM_WaveformOutputStart

函数原型：

```
HAL_StatusTypeDef HAL_HRTIM_WaveformOutputStart(HRTIM_HandleTypeDef * hhrtim,
                                                uint32_t OutputsToStart)
```

```
{
   /* 检查参数 */
  assert_param(IS_HRTIM_OUTPUT(OutputsToStart));

  /* 处理过程上锁 */
  __HAL_LOCK(hhrtim);

  hhrtim->State = HAL_HRTIM_STATE_BUSY;

  /* 使能 HRTIM 输出 */
  hhrtim->Instance->sCommonRegs.OENR |= (OutputsToStart);

  hhrtim->State = HAL_HRTIM_STATE_READY;

  /* 处理完毕解锁 */
  __HAL_UNLOCK(hhrtim);

  return HAL_OK;
}
```

函数描述：

此函数用于启动 HRTIM 的 waveform 模式输出。支持多个通道同时配置，即参数支持或操作。

函数参数：

- 第 1 个参数是 HRTIM_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数用于使能要输出的通道，这些参数支持或操作。
  - HRTIM_OUTPUT_TA1: Timer A - Output 1
  - HRTIM_OUTPUT_TA2: Timer A - Output 2
  - HRTIM_OUTPUT_TB1: Timer B - Output 1
  - HRTIM_OUTPUT_TB2: Timer B - Output 2
  - HRTIM_OUTPUT_TC1: Timer C - Output 1
  - HRTIM_OUTPUT_TC2: Timer C - Output 2
  - HRTIM_OUTPUT_TD1: Timer D - Output 1
  - HRTIM_OUTPUT_TD2: Timer D - Output 2
  - HRTIM_OUTPUT_TE1: Timer E - Output 1
  - HRTIM_OUTPUT_TE2: Timer E - Output 2
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
/* 启动 Timer D 的通道 1 和通道 2 的 PWM */
if (HAL_HRTIM_WaveformOutputStart(&HrtimHandle,  HRTIM_OUTPUT_TD1 + HRTIM_OUTPUT_TD2) != HAL_OK)
{
     Error_Handler(__FILE__, __LINE__);
}
```

63.4.7函数 HAL_HRTIM_WaveformCounterStart

函数原型：

```
#define HAL_HRTIM_WaveformCounterStart         HAL_HRTIM_WaveformCountStart

HAL_StatusTypeDef HAL_HRTIM_WaveformCountStart(HRTIM_HandleTypeDef * hhrtim,
                                               uint32_t Timers)
{
  /* 检查参数 */
  assert_param(IS_HRTIM_TIMERID(Timers));

  /* 处理过程上锁 */
  __HAL_LOCK(hhrtim);

  hhrtim->State = HAL_HRTIM_STATE_BUSY;

  /* 使能定时器计数器 */
  hhrtim->Instance->sMasterRegs.MCR |= (Timers);

  hhrtim->State = HAL_HRTIM_STATE_READY;

  /* 处理过程解锁 */
  __HAL_UNLOCK(hhrtim);

  return HAL_OK;
}
```

函数描述：

此函数用于启动定时器计数。

函数参数：

- 第 1 个参数是 HRTIM_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是定时器索引，支持以下参数，支持或操作：
  - HRTIM_TIMERID_MASTER
  - HRTIM_TIMERID_TIMER_A
  - HRTIM_TIMERID_TIMER_B
  - HRTIM_TIMERID_TIMER_C
  - HRTIM_TIMERID_TIMER_D
  - HRTIM_TIMERID_TIMER_E
- 返回值，返回 HAL_ERROR 表示配置失败，HAL_OK 表示配置成功，HAL_BUSY 表示忙（操作中），HAL_TIMEOUT 表示时间溢出。

使用举例：

```
/* 启动 Timer D 的计数器 */
if (HAL_HRTIM_WaveformCounterStart(&HrtimHandle, HRTIM_TIMERID_TIMER_D) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

63.5 总结

本章节就为大家讲解这么多，HRTIM 涉及到的知识点非常多，大家看完本章节后可以再学习 STM32H7 的参考手册做进一步了解。

第64章 STM32H7 的高分辨率定时器 HRTIM 应用之 PWM 实现

本章教程为大家讲解高分辨率定时器 HRTIM 的 PWM 实现。

64.1 初学者重要提示
64.2 HRTIM 的 PWM 驱动设计
64.3 HRTIM 板级支持包（bsp_tim_pwm.c）
64.4 HRTIM 驱动移植和使用
64.5 实验例程设计框架
64.6 实验例程说明（MDK）
64.7 实验例程说明（IAR）
64.8 总结

64.1 初学者重要提示

- 学习本章节前，务必优先学习第 63 章，HAL 库的几个常用 API 均作了讲解和举例。
- 设置 PWM 周期时，注意结构体 HRTIM_TimeBaseCfgTypeDef 中的 Period 周期参数范围，至少 3 个 HRTIM 时钟周期，最大值 0xFFDF。
- HRTIM 的输出极性可以设置激活状态 Active 和非激活状态 Inactive，这里要注意一点，激活状态既可以设置为高电平输出，也可以设置为低电平输出。
- HRTIM 其它几个例子执行效果展示，方便大家有个感性认识：
  - STM32H7 的 HRTIM 配置输出 5 组不同频率，不同占空比的波形，同时 5 组互补输出也是没问题的。
    http://www.armbbs.cn/forum.php?mod=viewthread&tid=89211 。
  - STM32H7 的 HRTIM 触发 ADC 和 DAC 转换
    http://www.armbbs.cn/forum.php?mod=viewthread&tid=89233 。
  - STM32H7 的 HRTIM Fault 故障保护功能，可在输出故障时禁止输出
    http://www.armbbs.cn/forum.php?mod=viewthread&tid=89232 。
  - STM32H7 的 HRTIM 生成任意波形
    http://www.armbbs.cn/forum.php?mod=viewthread&tid=89220 。

64.2 HRTIM 的 PWM 驱动设计

HRTIM 的 PWM 实现相对比较简单，只是涉及到的 API 比较多。

64.2.1 HRTIM 时钟设置

HRTIM 支持两种时钟源，一个是来自 CPU 主频时钟，另一个是来自通用定时器。大家可以通过函数 HAL_RCCEx_PeriphCLKConfig 来设置使用那个时钟。具体实现代码如下：

```
1.  RCC_PeriphCLKInitTypeDef PeriphClkInitStruct = {0};
2.
3.  PeriphClkInitStruct.PeriphClockSelection = RCC_PERIPHCLK_HRTIM1;
4.  PeriphClkInitStruct.Hrtim1ClockSelection = RCC_HRTIM1CLK_CPUCLK;
5.  if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInitStruct) != HAL_OK)
6.  {
7.      Error_Handler(__FILE__, __LINE__);
8.  }
```

这里把几个关键的地方再阐释下：

- 第 1 行，这个变量务必要做 0 初始化，防止不必要的麻烦。
- 第 4 行，用于配置 HRTIM 使用的时钟源，这里有两种选择：
  - 使用 CPU 主频时钟，对应参数 RCC_HRTIM1CLK_CPUCLK。
  - 使用通用定时器时钟，对应参数 RCC_HRTIM1CLK_TIMCLK。如果 CPU 主频时钟是 400MHz 的话，通用定时器时钟就是 200MHz。

64.2.2 HRTIM 的 PWM 输出引脚

HRTIM 的涉及到的输入输出引脚如下：

```
FTL = FAULT INPUT Lines
PA15        HRTIM_FLT1
PC11        HRTIM_FLT2
PD4         HRTIM_FLT3
PB3         HRTIM_FLT4
PG10        HRTIM_FLT5

EEV = EXTERN EVENT Lines
PG13        HRTIM_EEV10
PB7         HRTIM_EEV9
PB6         HRTIM_EEV8
PB5         HRTIM_EEV7
PB4         HRTIM_EEV6
PG12        HRTIM_EEV5
PG11        HRTIM_EEV4
PD5         HRTIM_EEV3
PC12        HRTIM_EEV2
PC10        HRTIM_EEV1

PC6         HRTIM_CHA1
```

```
PC7         HRTIM_CHA2
PC8         HRTIM_CHB1
PA8         HRTIM_CHB2
PA9         HRTIM_CHC1
PA10        HRTIM_CHC2
PA11        HRTIM_CHD1
PA12        HRTIM_CHD2
PG6         HRTIM_CHE1
PG7         HRTIM_CHE2

PE0         HRTIM_SCIN
PE1         HRTIM_SCOUT
PB10        HRTIM_SCOUT
PB11        HRTIM_SCIN
```

当前程序里面使用的 Timer D 的 HRTIM_CHD1 和 HRTIM_CHD2,即 PA11 和 PA12 引脚输出 PWM。

程序配置如下：

```
GPIO_InitTypeDef   GPIO_InitStruct;

GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_PULLUP;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;

GPIO_InitStruct.Alternate = GPIO_AF2_HRTIM1;
GPIO_InitStruct.Pin = GPIO_PIN_11;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

GPIO_InitStruct.Alternate = GPIO_AF2_HRTIM1;
GPIO_InitStruct.Pin = GPIO_PIN_12;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
```

64.2.3 HRTIM 初始化和时基配置

HRTIM 的初始化和时基配置如下：

```
1.  /*##- 初始化 HRTIM ##################################################*/
2.  HrtimHandle.Instance = HRTIM1;  /* 例化，使用的 HRTIM1 */
3.  HrtimHandle.Init.HRTIMInterruptResquests = HRTIM_IT_NONE;/* 用于配置支持的中断请求，当前配置无中断 */
4.  HrtimHandle.Init.SyncOptions = HRTIM_SYNCOPTION_NONE;    /* 配置 HRTIM 作为 Master，发送同步信号，或者作
5.                                                          为 Slave，接收同步信号，当前配置没有做同步功能 */
6.
7.  HAL_HRTIM_Init(&HrtimHandle);
8.
9.  /*##- 配置 HRTIM 的 TIMER D 时基 ##########################################*/
10. sConfig_time_base.Mode = HRTIM_MODE_CONTINUOUS; /* 连续工作模式 */
11. sConfig_time_base.Period = HRTIM_TIMD_PERIOD;   /* 设置周期 */
12. sConfig_time_base.PrescalerRatio = HRTIM_PRESCALERRATIO_DIV1; /* 设置 HRTIM 分频，当前设置的 1 分频，也就
13.                                                                 是不分频 */
14. sConfig_time_base.RepetitionCounter = 0;                     /* 设置重复计数器为 0，即不做重复计数 */
15.
16. HAL_HRTIM_TimeBaseConfig(&HrtimHandle, HRTIM_TIMERINDEX_TIMER_D, &sConfig_time_base);
```

这里把几个关键的地方再阐释下：

- 第 2-4 行，初始化 HRTIM。
- 第 10-16 行，配置 HRTIM 的 Timer D 时基。

- 第 11 行，设置 Timer D 的周期。

  比如 HRTIM 主频是 400MHz，HRTIM_TIMD_PERIOD = 4000，那么 Timer D 的输出频率如下：

  PWM 的频率 = 400MHz / HRTIM_TIMD_PERIOD

  = 400000000 / 4000

  = 100KHz

- 第 12 行，对于 STM32H7 系列，仅支持下面选项中最后三个参数，也就是 1 分频，2 分频和 4 分频。

```
#define HRTIM_PRESCALERRATIO_MUL32     (0x00000000U)
#define HRTIM_PRESCALERRATIO_MUL16     (0x00000001U)
#define HRTIM_PRESCALERRATIO_MUL8      (0x00000002U)
#define HRTIM_PRESCALERRATIO_MUL4      (0x00000003U)
#define HRTIM_PRESCALERRATIO_MUL2      (0x00000004U)
#define HRTIM_PRESCALERRATIO_DIV1      (0x00000005U)
#define HRTIM_PRESCALERRATIO_DIV2      (0x00000006U)
#define HRTIM_PRESCALERRATIO_DIV4      (0x00000007U)
```

64.2.4 HRTIM 的 Timer D 配置

Timer D 的配置成员非常多，对于 PWM 输出功能来说，这些成员的功能有个了解即可：

```
HRTIM_TimerCfgTypeDef        sConfig_timerD;
sConfig_timerD.DMARequests = HRTIM_TIM_DMA_NONE;         /* 不使用 DMA */
sConfig_timerD.HalfModeEnable = HRTIM_HALFMODE_DISABLED;/* 关闭 HALF 模式 */
sConfig_timerD.StartOnSync = HRTIM_SYNCSTART_DISABLED;   /* 设置同步输入端接收到上升沿信号后，不启动定时器 */
sConfig_timerD.ResetOnSync = HRTIM_SYNCRESET_DISABLED;   /* 设置同步输入端接收到上升沿信号后，不复位定时器 */
sConfig_timerD.DACSynchro = HRTIM_DACSYNC_NONE;          /* 不使用 DAC 同步事件 */
sConfig_timerD.PreloadEnable = HRTIM_PRELOAD_ENABLED;     /* 使能寄存器预加载 */
sConfig_timerD.UpdateGating = HRTIM_UPDATEGATING_INDEPENDENT;       /* 独立更新，与 DMA 突发传输完成无关 */
sConfig_timerD.BurstMode = HRTIM_TIMERBURSTMODE_MAINTAINCLOCK;      /* 在突发模式下，定时器正常运行 */
sConfig_timerD.RepetitionUpdate = HRTIM_UPDATEONREPETITION_ENABLED;/* 设置重计数器事件可以触发寄存器更新 */
/* 当 HRTIM TIMER 的计数器复位时或者计数回滚到 0 时，不触发寄存器更新 */
sConfig_timerD.ResetUpdate = HRTIM_TIMUPDATEONRESET_DISABLED;
sConfig_timerD.InterruptRequests = HRTIM_TIM_IT_NONE;            /* 不使用中断 */
sConfig_timerD.PushPull = HRTIM_TIMPUSHPULLMODE_DISABLED;        /* 不开启推挽模式 */
sConfig_timerD.FaultEnable = HRTIM_TIMFAULTENABLE_NONE;          /* 不使用 HRTIM TIMER 的 Fault 通道 */
sConfig_timerD.FaultLock = HRTIM_TIMFAULTLOCK_READWRITE;         /* 不开启 HRTIM TIMER 的异常使能状态写保护 */
sConfig_timerD.DeadTimeInsertion = HRTIM_TIMDEADTIMEINSERTION_DISABLED;/* 不开启死区时间插入 */
/* 不开启 HRTIM TIMER 的延迟保护模式 */
sConfig_timerD.DelayedProtectionMode = HRTIM_TIMER_D_E_DELAYEDPROTECTION_DISABLED;
/* Master 或 TIMER（A 到 E）更新时，不同步更新寄存器 */
sConfig_timerD.UpdateTrigger= HRTIM_TIMUPDATETRIGGER_NONE;
sConfig_timerD.ResetTrigger = HRTIM_TIMRESETTRIGGER_NONE; /* 无复位触发 */
HAL_HRTIM_WaveformTimerConfig(&HrtimHandle, HRTIM_TIMERINDEX_TIMER_D, &sConfig_timerD);
```

注意，如果 HRTIM_TimerCfgTypeDef sConfig_timerD 做局部变量，务必记得清零。

64.2.5 Timer D 的输出比较配置

HRTIM 用于 PWM 功能时，比较输出用于设置 PWM 占空比：

```
HRTIM_CompareCfgTypeDef       sConfig_compare;

sConfig_compare.AutoDelayedMode = HRTIM_AUTODELAYEDMODE_REGULAR; /* 这里使用标准模式，即未使用自动延迟 */
sConfig_compare.AutoDelayedTimeout = 0;              /* 由于前面的参数未使用自动延迟模式，此参数无作用 */
/*
     设置定时器比较单元的比较值：
     最小值要大于等于 3 个 HRTIM 时钟周期。
     最大值要小于等于 0xFFFF - 1
*/
sConfig_compare.CompareValue = HRTIM_TIMD_PERIOD / 2;  /* 占空比 50% */
HAL_HRTIM_WaveformCompareConfig(&HrtimHandle, HRTIM_TIMERINDEX_TIMER_D, HRTIM_COMPAREUNIT_1,
                                                                        &sConfig_compare);
sConfig_compare.CompareValue = HRTIM_TIMD_PERIOD / 4;  /* 占空比 25% */
HAL_HRTIM_WaveformCompareConfig(&HrtimHandle, HRTIM_TIMERINDEX_TIMER_D, HRTIM_COMPAREUNIT_2,
                                                                        &sConfig_compare);
```

注意事项：

- 如果 HRTIM_CompareCfgTypeDef sConfig_compare 做局部变量，务必记得清零。
- 配置占空比就是配置成员 CompareValue，范围是 0 到 HRTIM_TIMD_PERIOD（这个参数就是前面配置的 PWM 周期）。比如配置为 HRTIM_TIMD_PERIOD/2 就表示占空比 50%，配置为 HRTIM_TIMD_PERIOD/4 就表示占空比 25%。

64.2.6启动 PWM 输出和 Timer D 的计数

这部分的实现代码如下：

```
1.  HRTIM_OutputCfgTypeDef       sConfig_output_config;
2.
3.  sConfig_output_config.Polarity = HRTIM_OUTPUTPOLARITY_LOW;    /* 设置定时器输出极性 */
4.  sConfig_output_config.SetSource = HRTIM_OUTPUTRESET_TIMCMP1;  /* 定时器比较事件 1 可以将输出置位 */
5.  sConfig_output_config.ResetSource = HRTIM_OUTPUTSET_TIMPER;   /* 定时器周期性更新事件可以将输出清零 */
6.  sConfig_output_config.IdleMode = HRTIM_OUTPUTIDLEMODE_NONE;   /* 输出不受突发模式影响 */
7.  sConfig_output_config.IdleLevel = HRTIM_OUTPUTIDLELEVEL_INACTIVE; /* 设置空闲状态输出低电平 */
8.  sConfig_output_config.FaultLevel = HRTIM_OUTPUTFAULTLEVEL_NONE;   /* 输出不受异常输入影响 */
9.  sConfig_output_config.ChopperModeEnable = HRTIM_OUTPUTCHOPPERMODE_DISABLED; /* 关闭 Chopper 模式 */
10. sConfig_output_config.BurstModeEntryDelayed = HRTIM_OUTPUTBURSTMODEENTRY_REGULAR; /* 设置从突发模式切换
11.                                                                    到空闲模式，不插入死区时间 */
12.
13. HAL_HRTIM_WaveformOutputConfig(&HrtimHandle, HRTIM_TIMERINDEX_TIMER_D, HRTIM_OUTPUT_TD1,
14.                                                          &sConfig_output_config);
15.
16. sConfig_output_config.SetSource = HRTIM_OUTPUTRESET_TIMCMP2;  /* 定时器比较事件 2 可以将输出置位 */
17. HAL_HRTIM_WaveformOutputConfig(&HrtimHandle, HRTIM_TIMERINDEX_TIMER_D, HRTIM_OUTPUT_TD2,
18.                                                           &sConfig_output_config);
19.
20. /*##-9- 启动 PWM 输出 ##################################################*/
21. if (HAL_HRTIM_WaveformOutputStart(&HrtimHandle,  HRTIM_OUTPUT_TD1 + HRTIM_OUTPUT_TD2) != HAL_OK)
22. {
23.     Error_Handler(__FILE__, __LINE__);
24. }
25.
26. /*##-10- 启动计数器 ##################################################*/
27. if (HAL_HRTIM_WaveformCounterStart(&HrtimHandle, HRTIM_TIMERID_TIMER_D) != HAL_OK)
28. {
```

```
29.       Error_Handler(__FILE__, __LINE__);
30.   }
```

这里把几个关键的地方再阐释下：

- 第 1 行，如果 HRTIM_OutputCfgTypeDef sConfig_output_config 做局部变量，务必记得清零。
- 第 3 行，输出极性是用来设置激活状态 Active 对应的高电平还是低电平。
- 第 4 行，用来实现置位源（SetSource）设置，这里是设置满足比较事件 1 时，输出置位。
- 第 5 行，用来实现复位源（ResetSource）设置，这里是设置产生周期性更新事件时，输出清零。通过第 4 行和第 5 行，就实现了 Timer D 中通道 1 的高低电平输出方式，
- 第 16 行，设置 Timer D 中通道 2 的置位源，即通道 2 的高低电平输出方式。

64.3 HRTIM 板级支持包（bsp_hrtim_pwm.c）

定时器驱动文件 bsp_hrtim_pwm.c 主要实现了如下一个 API 供用户调用：

- bsp_SetHRTIMOutPWM

64.3.1 函数 bsp_SetHRTIMforInt

函数原型：

```
void bsp_SetHRTIMOutPWM(void)
```

函数描述：

这个函数的源码实现在本章 64.2 小节里面已经进行了详细说明。

当前这个函数通过配置 HRTIM 的 TIMER D 输出两路 PWM，周期都是 100KHz，PA11 引脚输出占空比 50%，PA12 引脚输出的占空比 25%。

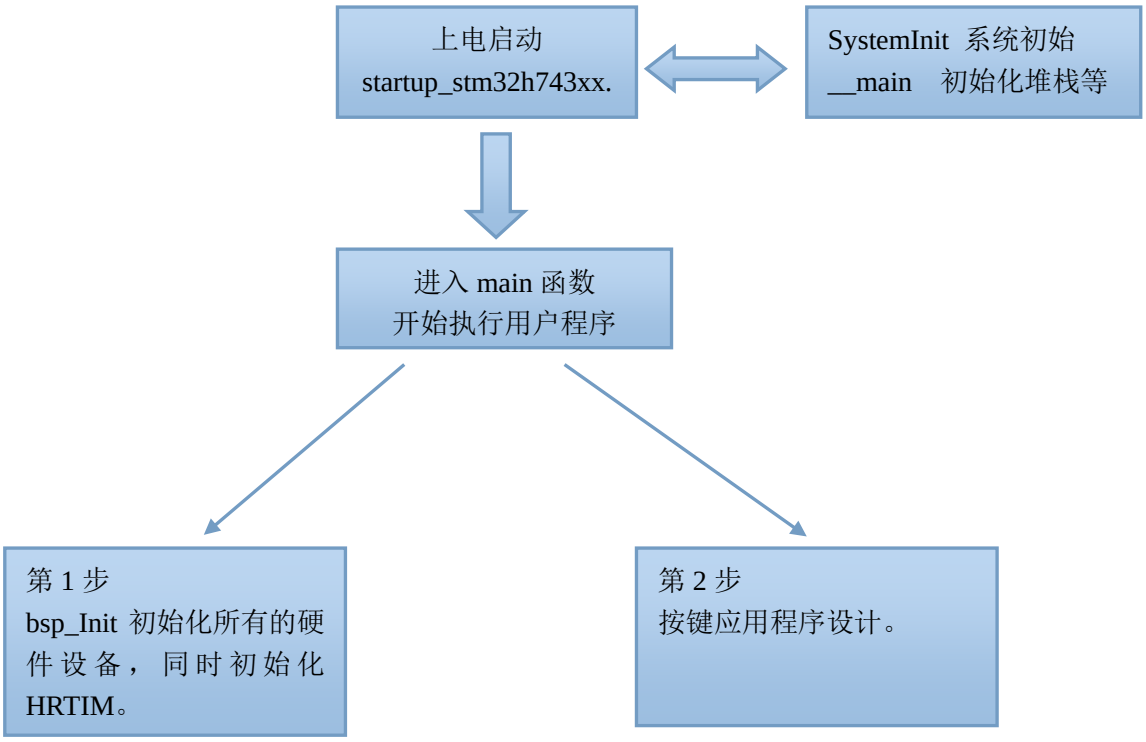
64.4 HRTIM 驱动移植和使用

定时器的移植比较简单：

- 第 1 步：复制 bsp_hrtim_pwm.c 和 bsp_hrtim_pwm.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：这几个驱动文件主要用到 HAL 库的 GPIO 和 HRTIM 驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 3 步，应用方法看本章节配套例子即可。

64.5 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED 和串口。同时 HRTIM 也做了配置，将 HRTIM 的 TIMER D 输出两路 PWM，周期都是 100KHz，PA11 引脚输出占空比 50%，PA12 引脚输出的占空比 25%。
  - 第 2 步，按键应用程序设计部分。

64.6 实验例程说明（MDK）

配套例子：

V7-045_高分辨率定时器 HRTIM 实现 PWM 输出

实验目的：

1. 学习高分辨率定时器 HRTIM 的 PWM 实现。

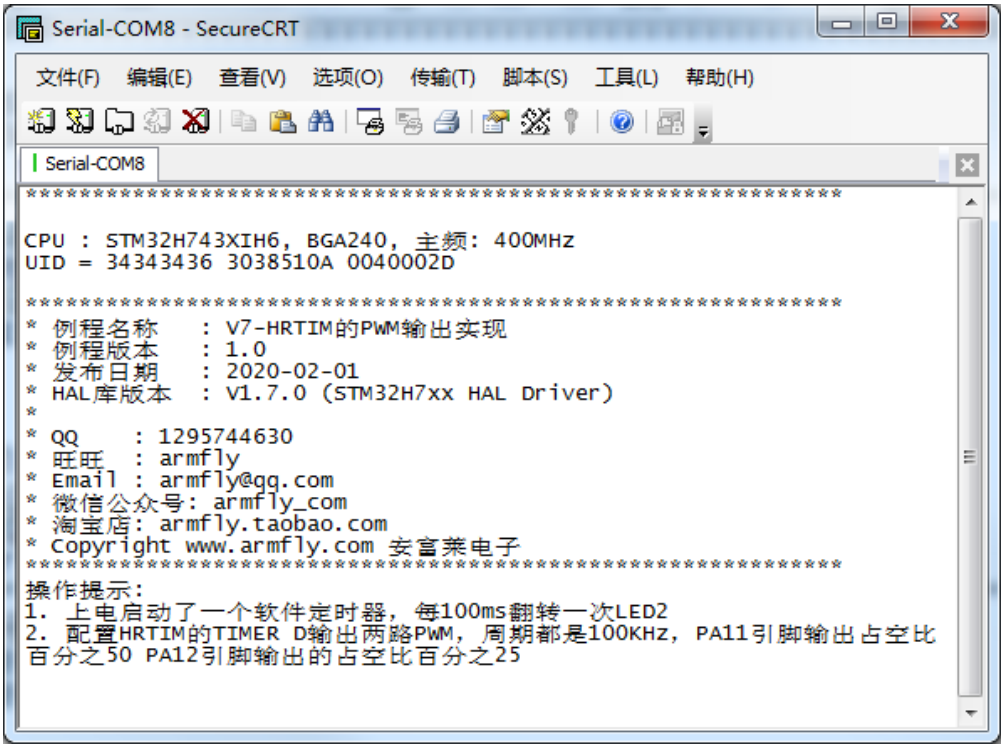
实验内容：

1. 上电启动了一个软件定时器，每 100ms 翻转一次 LED2。
2. 配置 HRTIM 的 TIMER D 输出两路 PWM，周期都是 100KHz，PA11 引脚输出占空比 50%，PA12 引脚输出的占空比 25%。

PWM 输出引脚 PA11 和 PA12 位置：

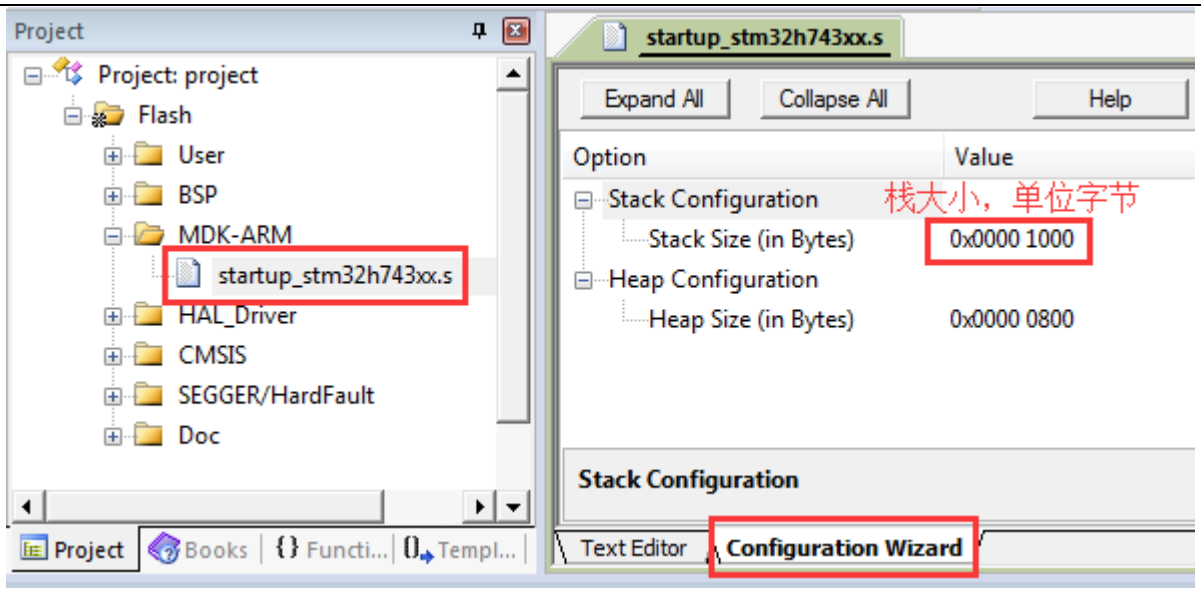
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

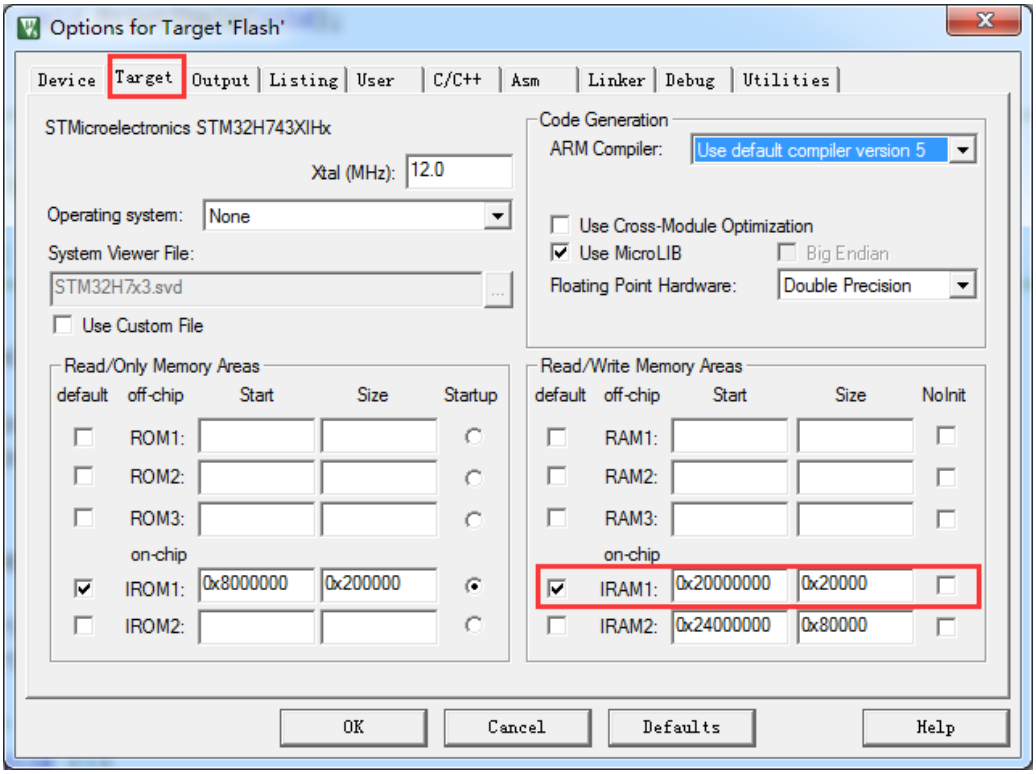


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();
```

```
    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **主功能:**

  主程序实现如下操作:

  - 上电启动了一个软件定时器，每 100ms 翻转一次 LED2。
  - 配置 HRTIM 的 TIMER D 输出两路 PWM,周期都是 100KHz,PA11 引脚输出占空比 50%,PA12

引脚输出的占空比 25%

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode; /* 按键代码 */

    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */


    bsp_SetHRTIMOutPWM();

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下 */
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

64.7 实验例程说明（IAR）

配套例子：

V7-045_高分辨率定时器 HRTIM 实现 PWM 输出

实验目的：

1. 学习高分辨率定时器 HRTIM 的 PWM 实现。

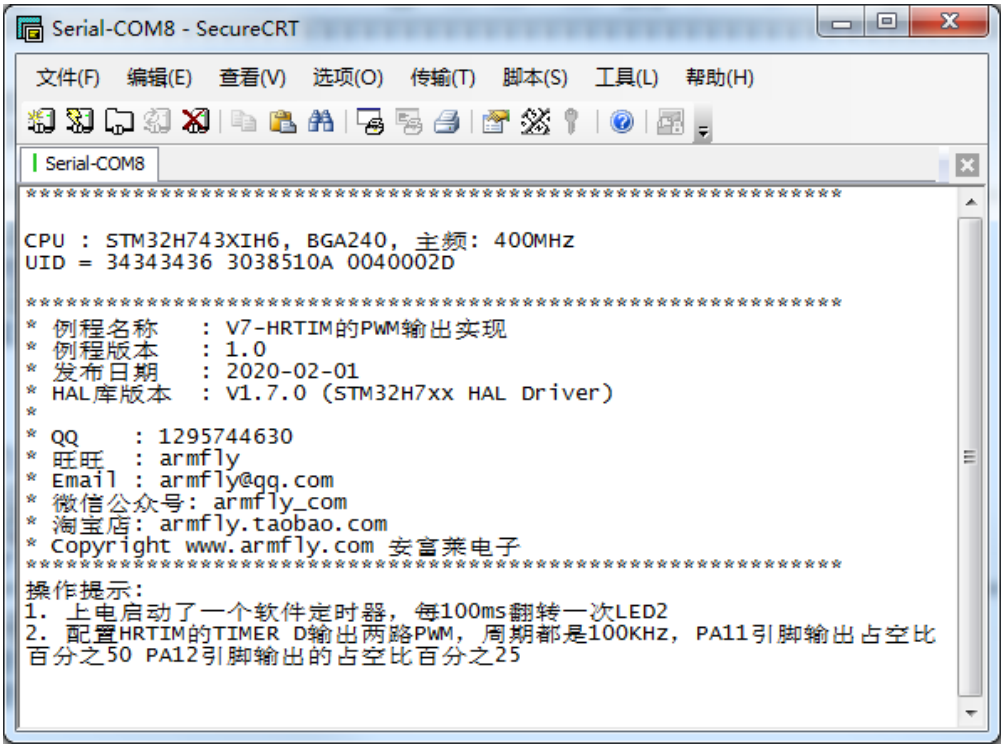
实验内容：

1. 上电启动了一个软件定时器，每 100ms 翻转一次 LED2。
2. 配置 HRTIM 的 TIMER D 输出两路 PWM，周期都是 100KHz，PA11 引脚输出占空比 50%，PA12 引脚输出的占空比 25%。

PWM 输出引脚 PA11 和 PA12 位置：

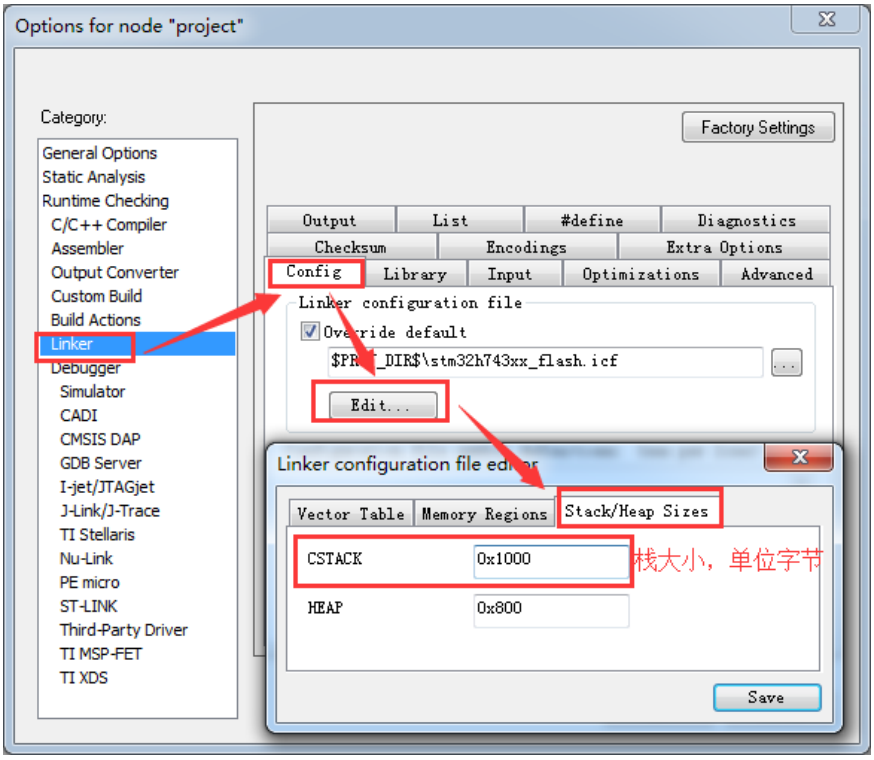
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

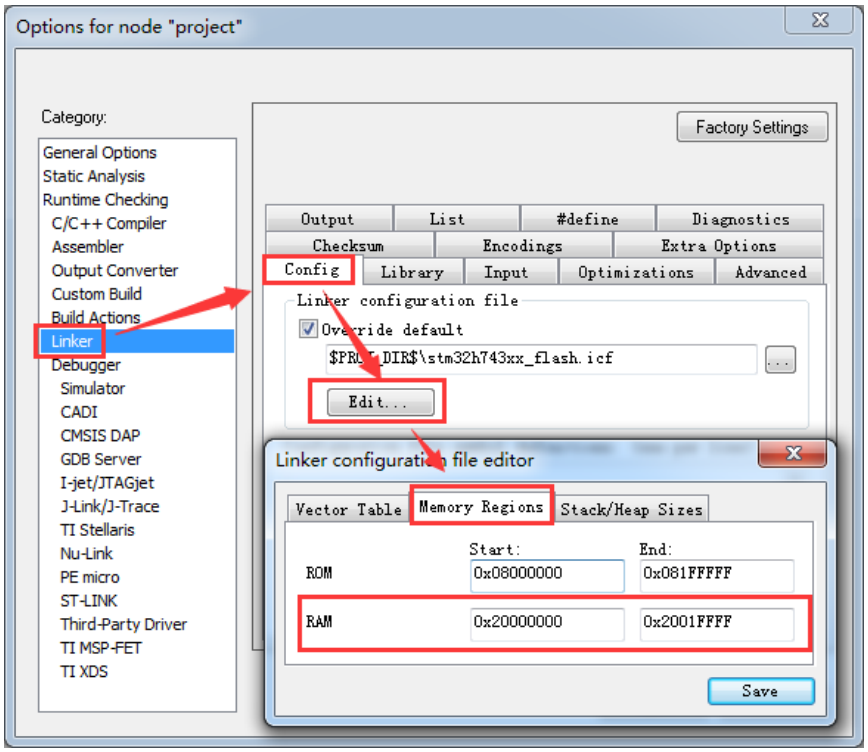


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
```

```
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
```

```
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能：**

主程序实现如下操作：

- 上电启动了一个软件定时器，每 100ms 翻转一次 LED2。
- 配置 HRTIM 的 TIMER D 输出两路 PWM，周期都是 100KHz，PA11 引脚输出占空比 50%，PA12 引脚输出的占空比 25%

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode; /* 按键代码 */

    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */


    bsp_SetHRTIMOutPWM();

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下 */
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

64.8 总结

本章节就为大家讲解这么多，PWM 是 HRTIM 里面相对比较容易掌握，还有一些高级玩法，后续章节为大家做介绍。

第65章 STM32H7 的低功耗串口 LPUART 基础知识和 HAL 库 API

本章节为大家讲解 LPUART（Low power universal asynchronous receiver transmitter，低功耗通用异步收发器）的基础知识和对应的 HAL 库 API。相比第 29 章的通用串口，增加了低功耗特性。

65.1 初学者重要提示
65.2 低功耗串口基础知识
65.3 低功耗串口的 HAL 库用法
65.4 源文件 stm32h7xx_hal_uart.c
65.5 总结

65.1 初学者重要提示

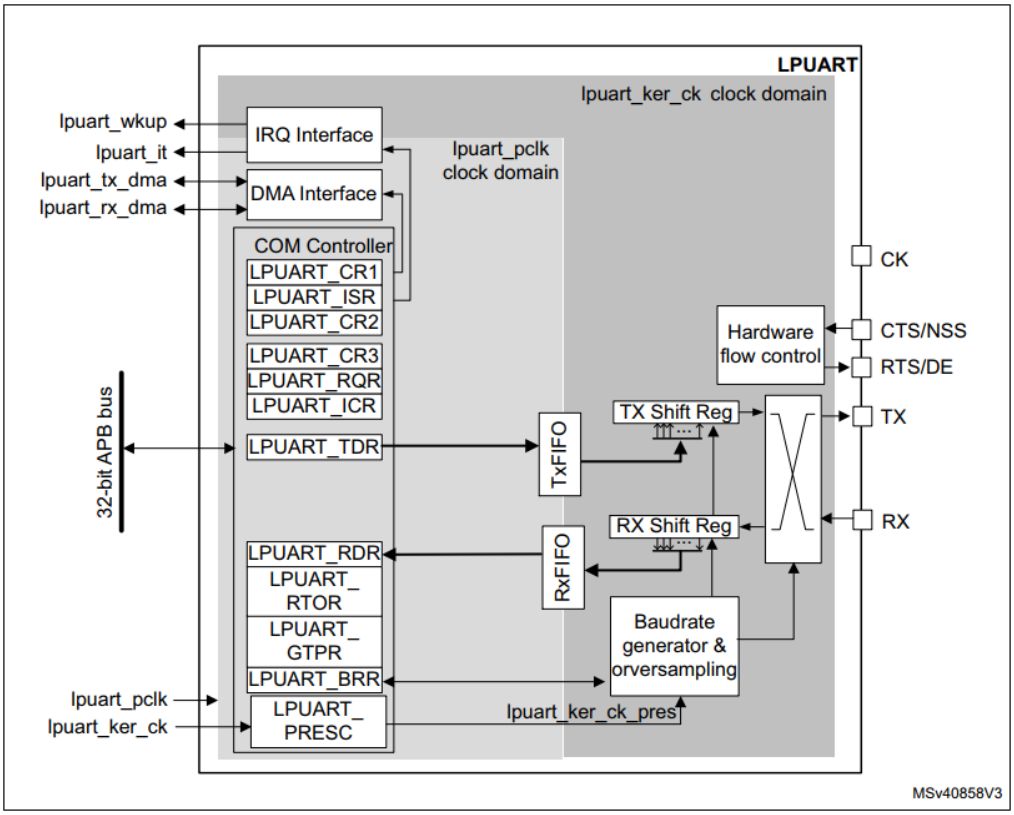
- 特别注意，LPUART 没有自己的 HAL 库驱动文件，是跟通用串口公用的驱动文件。
- 学习串口外设推荐从硬件框图开始了解基本的功能特性，然后逐步深入了解各种特性，这种方式方便记忆和以后查阅。而串口的通信学习，推荐看时序图。

65.2 低功耗串口基础知识

LPUART 的全称是 Low power universal synchronous asynchronous receiver transmitter，中文意思是低功耗通用异步收发器，简称 LPUART。

65.2.1 低功耗串口的硬件框图

认识一个外设，最好的方式就是看它的框图，方便我们快速的了解串口的基本功能，然后再看手册了解细节。



通过这个框图，我们可以得到如下信息：

- IRQ Interface 中断接口

  用于实现中断方式的串口唤醒 lpusart_wkup 和串口的相关中断 lpusart_it。

- DMA Interface DMA 接口

  实现串口发送 lpuart_tx_dma 和接收 lpuart_rx_dma 的 DMA 方式。

- COM Contronller 串口控制器

  串口相关的寄存器基本都在这部分。

- TxFIFO 和 RxFIFO

  串口的发送和接收都支持了硬件 FIFO 功能。

- TX 和 RX 引脚的互换功能

  发送偏移寄存器（TX Shift Reg）和接收偏移寄存器（RX Shift Reg）与 TX 引脚，RX 引脚之间弄了个交叉连接，这里的意思是支持了引脚互换功能，这样大家在设计 PCB 的时候就可以比较随性了，接反了也没有关系。

- 发送过程经过的寄存器

  依次是 LPUART_TDR -> TxFIFO ->Tx Shift Reg 偏移寄存器 –> TX 或者 RX 引脚。

- 接收经过的寄存器

  依次是 TX 或者 RX 引脚-> Rx Shift Reg 偏移寄存器->RxFIFO –>LPUART_RDR。

- 两个时钟 lpuart_pclk 和 lpuart_ker_ck

  这两个时钟是独立的，作用如下：

- lpuart_pclk
  用于为外设总线提供时钟。
- lpuart_ker_ck
  串口外设的时钟源。

65.2.2 低功耗串口的基本功能

STM32 的串口功能很强大，支持太多的模式。我们只需关心我们最常用的特性即可。我们的串口驱动使用的串口中断+FIFO 结构，没有使用 DMA。因此我们只讨论和串口中断、串口常规参数有关的知识。

STM32 串口的优越特性：（只列了举常用的）

- 各种波特率。硬件采用分数波特率发生器系统，可以设置各种需要的波特率
- 可编程数据字长度，支持 7bit，8bit 和 9bit。
- 可配置的停止位。支持 1 或 2 个停止位。
- 发送器和接收器可以单独使能。比如 GPS 应用只需要串口接收，那么发送的 GPIO 就可以节省出来用作其他功能。
- 检测标志和中断：
  - 接收缓冲器满，可产生中断。串口中断服务程序据此判断是否接收到数据。
  - 发送缓冲器空，可产生中断。串口中断服务程序据此启动发送下一个数据。
  - 传输结束标志，可产生中断。用于 RS485 通信，等最后一个字节发送完毕后，需要控制 RS485 收发器芯片切换为接收模式。

其它中断不常用，包括：CTS 改变、LIN 断开符检测、检测到总线为空闲（在 DMA 不定长接收方式会用到）、溢出错误、帧错误、噪音错误、校验错误。

65.2.3 低功耗串口的高级特性

H7 系列的串口支持了一些高级特性，比如：

- 数据逻辑电平翻转。
- 低功耗特性。
- RX 和 TX 引脚交换。
- MSB 位先发送。
- 外接 485 的 PHY 芯片时，硬件支持收发切换，无需用户手动控制 DE 引脚。

相比第 29 章的通用串口，低功耗串口不支持超时接收和自适应波特率。

65.2.4 低功耗串口的数据帧格式

串口支持的帧格式如下（M 和 PCE 都是 LPUART_CR1 寄存器的位，其中 M 位用于控制帧长度，PCE 用于使能奇偶校验位）：

M 位	PCE 位	LPUART 帧
00	0	|SB|8 位数据|STB|
00	1	|SB|7 位数据|PB|STB|
01	0	|SB|9 位数据|STB|
01	1	|SB|8 位数据|PB|STB|
10	0	|SB|7 位数据|STB|
10	1	|SB|6 位数据|PB|STB|

这里特别注意奇偶校验位，用户在配置的时候可以选择奇校验和偶校验，校验位是占据的最高位。比如选择 M=00，PCE=1，即 7bit 的数据位。

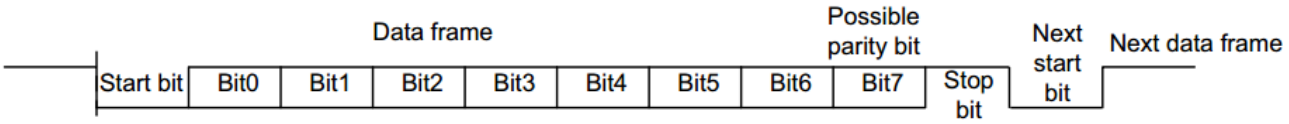
- 串口发送数据：

  如果发送的 7bit 数据是 111 0011，这个里面有奇数个 1，那么选择偶校验的情况下，校验位 = 1，凑够偶数个 1，而选择奇校验的情况下，校验位 = 0，因为已经是奇数个 1。校验位不需要用户去计算，是硬件自动生成的。

- 串口接收数据：

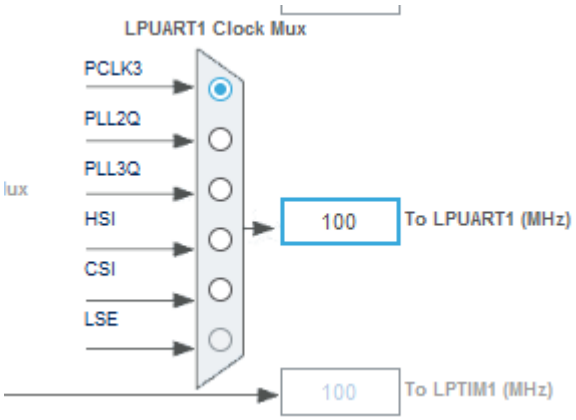
  根据用户设置的奇校验或者偶校验类型，串口硬件会对接收到的数据做校验，如果失败，LPUART_ISR 寄存器的 PE 位会被置 1。如果使能了对应的中断 PEIE，那么失败的时候还会产生中断。

了解到帧格式后，再来看一下实际数据发送时，数据位的先后顺序：



65.2.5 低功耗串口的支持的时钟和波特率

低功耗定时器支持如下几种时钟：



这里我们重点关注 PCLK3（D3PCLK1），HSI 和 LSE。

- LPUART 时钟选择 LSE(32768Hz)

  最高速度是 10922bps，最低 8bps（计算方法 3x < 32768 < 4096x，x 表示波特率）。

- LPUART 时钟选择 HSI(64MHz)

  最高值是 21MHz，最小值 15625bps（计算方法 3x < 64MHz < 4096x，x 表示波特率）。

- LPUART 时钟选择 D3PCLK1(100MHz)

  最大值 33Mbps，最小值 24414bps（计算方法 3x < 100MHz < 4096x，x 表示波特率）。

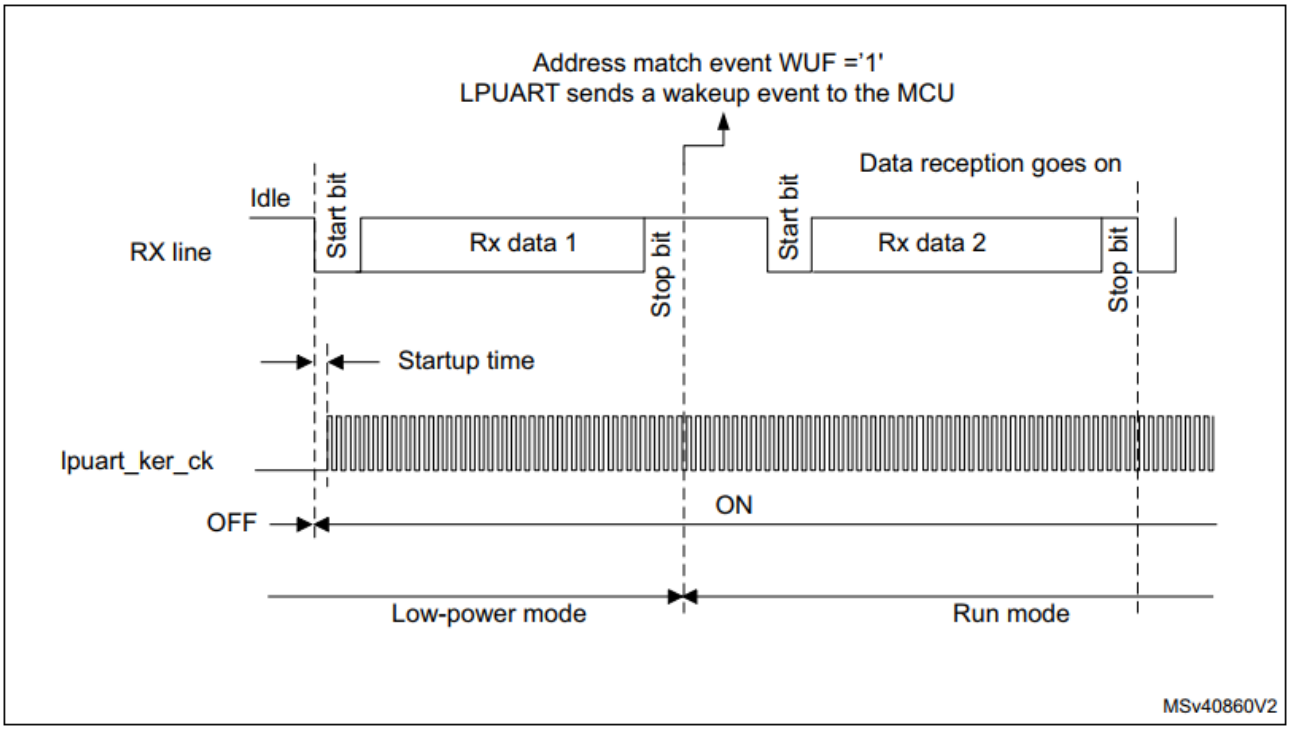
65.2.6低功耗串口的支持唤醒方式

低功耗串口的唤醒主要是通过接收数据来唤醒，具体唤醒的方如下：

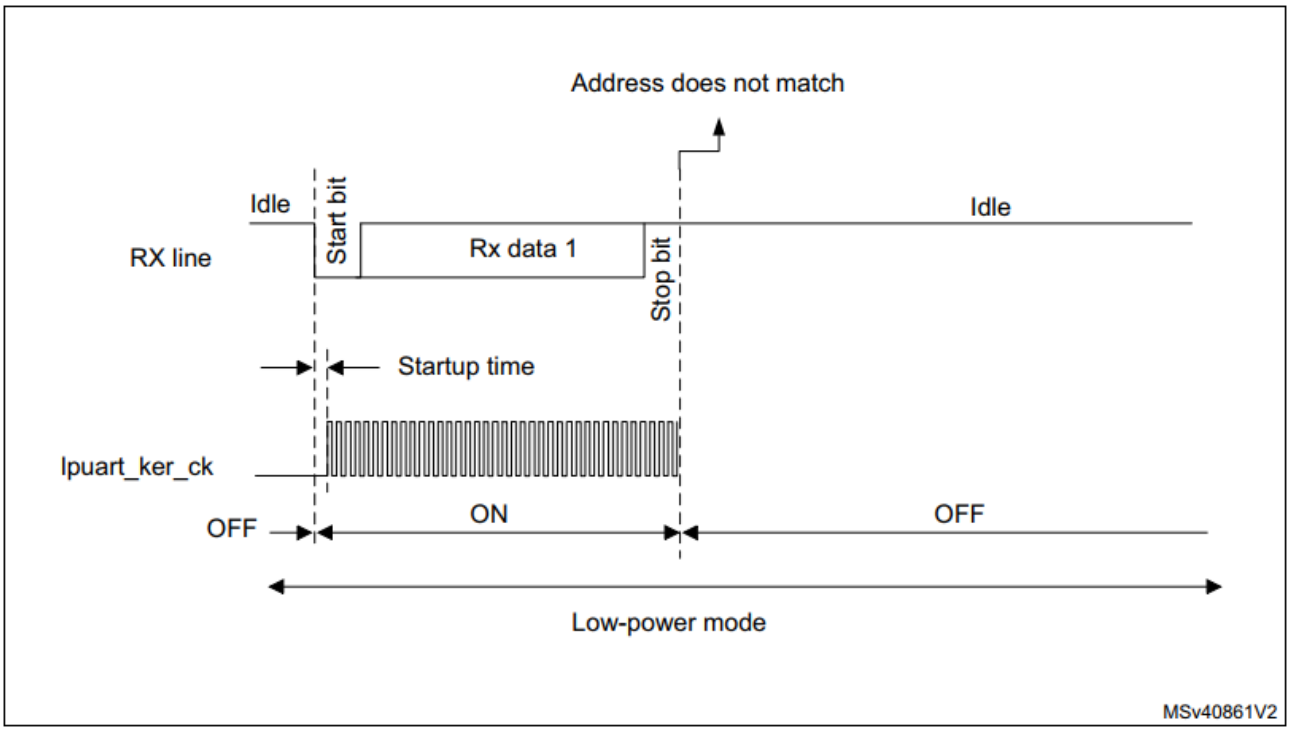
- 检测到起始位唤醒。
- 检测到 RXNE 标志唤醒，即接收到数据。
- 检测到匹配地址时唤醒。

  匹配地址支持 7bit 和 4bit 匹配两种方式，比如我们采用 7bit 匹配，设置地址是 0x19，那么用户唤醒的时候要将最高 bit 设置为 1，即发送地址 0x99（0b1001 1001）才可以唤醒。

唤醒成功时的时序效果如下，特别注意唤醒信号位置：

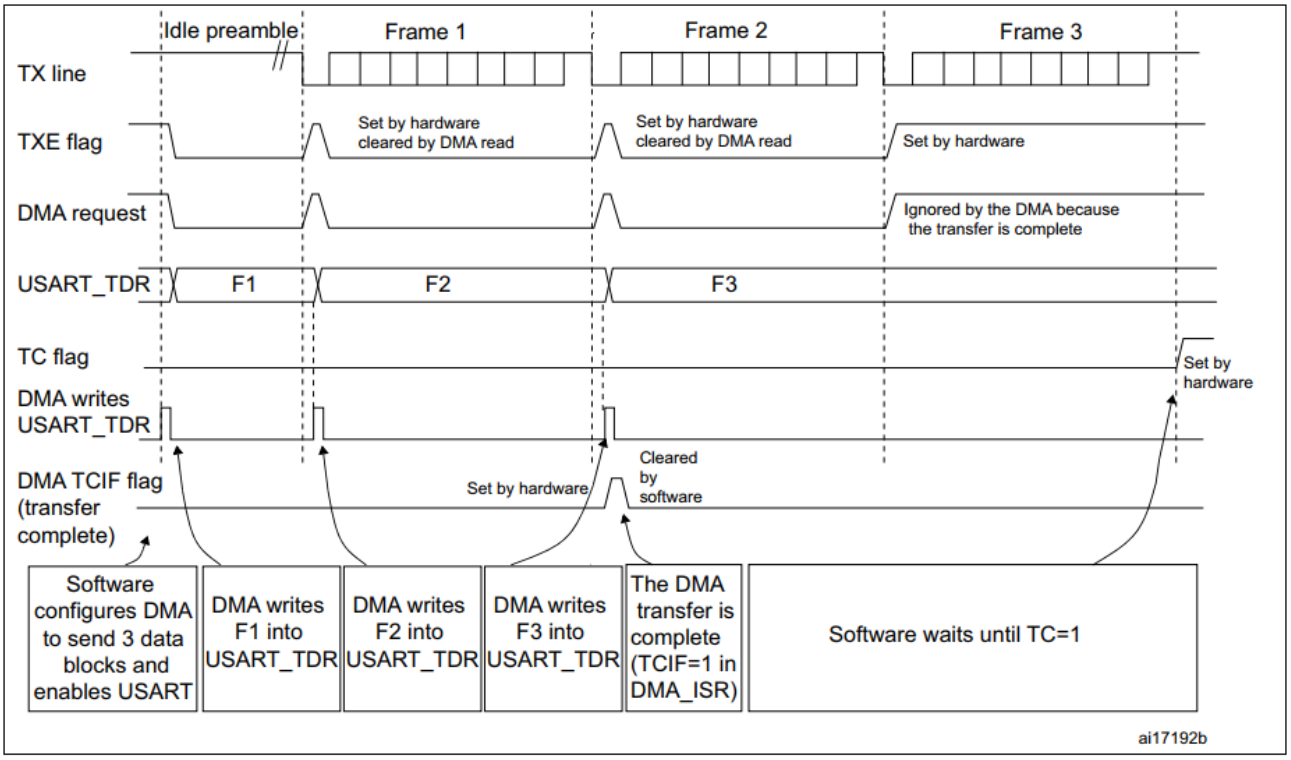


唤醒失败时的时序效果：



65.2.7 低功耗串口发送时序图

这个时序图非常具有代表性，可以帮助大家很好的理解 TC 发送完成中断和 TXE 空中断。



Note: When FIFO management is enabled, the DMA request is triggered by Transmit FIFO not full (i.e. TXFNF = 1).

65.2.8单工，半双工和全双工通讯

单工：在一个单工的串行通讯系统中，一般至少有两根线（信号线和地线），数据传送只有一个方向，例如可以使用单工数据传送将数据从一个简单的数据监测系统传送到 PC 上。

半双工：在半双工串行通信系统中，一般同样要求至少有两根线。这里的数据传送是双向的。然而，同一个时刻只能为一个方向。在上面的数据监测的例子中做了一些变化，可以使用半双工通讯机制发送信息到嵌入式模块（来设置参数，比如采样率）。此外，在其他时候，可以使用这个种连接将嵌入式装置上的数据下载到 PC 中。

全双工：在一个全双工的串行通信系统中，一般要求至少有三根线（信号线 A，信号线 B 和地线）。信号线 A 将传输一个方向上的数据，同时信号线 B 传送另一个方向上的数据。

65.3低功耗串口的 HAL 库用法

串口的 HAL 库用法其实就是几个结构体变量成员的配置和使用，然后配置 GPIO、时钟，并根据需要配置 NVIC、中断和 DMA。下面我们逐一展开为大家做个说明。

65.3.1 低功耗串口寄存器结构体 USART_TypeDef

USART 相关的寄存器是通过 HAL 库中的结构体 USART_TypeDef 定义的，在 stm32h743xx.h 中可以找到这个类型定义：

```
typedef struct
{
  __IO uint32_t CR1;    /*!< USART Control register 1,                 Address offset: 0x00 */
  __IO uint32_t CR2;    /*!< USART Control register 2,                 Address offset: 0x04 */
  __IO uint32_t CR3;    /*!< USART Control register 3,                 Address offset: 0x08 */
  __IO uint32_t BRR;    /*!< USART Baud rate register,                 Address offset: 0x0C */
  __IO uint16_t GTPR;   /*!< USART Guard time and prescaler register,  Address offset: 0x10 */
  uint16_t  RESERVED2;  /*!< Reserved, 0x12                                                 */
  __IO uint32_t RTOR;   /*!< USART Receiver Time Out register,         Address offset: 0x14 */
  __IO uint16_t RQR;    /*!< USART Request register,                   Address offset: 0x18 */
  uint16_t  RESERVED3;  /*!< Reserved, 0x1A                                                 */
  __IO uint32_t ISR;    /*!< USART Interrupt and status register,      Address offset: 0x1C */
  __IO uint32_t ICR;    /*!< USART Interrupt flag Clear register,      Address offset: 0x20 */
  __IO uint16_t RDR;    /*!< USART Receive Data register,              Address offset: 0x24 */
  uint16_t  RESERVED4;  /*!< Reserved, 0x26                                                 */
  __IO uint16_t TDR;    /*!< USART Transmit Data register,             Address offset: 0x28 */
  uint16_t  RESERVED5;  /*!< Reserved, 0x2A                                                 */
  __IO uint32_t PRESC;  /*!< USART clock Prescaler register,           Address offset: 0x2C */
} USART_TypeDef;
```

这个结构体的成员名称和排列次序和 CPU 的 USART 寄存器是一 一对应的。

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

下面我们看下 LPUART 的定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE           (0x40000000UL)
#define D3_APB1PERIPH_BASE    (PERIPH_BASE + 0x18000000UL)
#define LPUART1_BASE          (D3_APB1PERIPH_BASE + 0x0C00UL)
#define LPUART1               ((USART_TypeDef *) LPUART1_BASE) <----- 展开这个宏，(USART_TypeDef *) 0x58000C00
```

我们访问 LPUART1 的 CR1 寄存器可以采用这种形式：LPUART1->CR1 = 0。

65.3.2 低功耗串口句柄结构体 UART_HandleTypeDef

HAL 库在 USART_TypeDef 的基础上封装了一个结构体 UART_HandleTypeDef，定义如下：

```
typedef struct __UART_HandleTypeDef
{
  USART_TypeDef            *Instance;
  UART_InitTypeDef         Init;
  UART_AdvFeatureInitTypeDef AdvancedInit;
  uint8_t                  *pTxBuffPtr;
  uint16_t                 TxXferSize;
  __IO uint16_t            TxXferCount;
  uint8_t                  *pRxBuffPtr;
  uint16_t                 RxXferSize;
  __IO uint16_t            RxXferCount;
```

```
  uint16_t                Mask;
  uint32_t                FifoMode;
  uint16_t                NbRxDataToProcess;
  uint16_t                NbTxDataToProcess;
  void (*RxISR)(struct __UART_HandleTypeDef *huart);
  void (*TxISR)(struct __UART_HandleTypeDef *huart);
  DMA_HandleTypeDef       *hdmatx;
  DMA_HandleTypeDef       *hdmarx;
  HAL_LockTypeDef          Lock;                  /
  __IO HAL_UART_StateTypeDef    gState;
  __IO HAL_UART_StateTypeDef    RxState;
  __IO uint32_t                 ErrorCode;

#if (USE_HAL_UART_REGISTER_CALLBACKS == 1)
  void (* TxHalfCpltCallback)(struct __UART_HandleTypeDef *huart);
  void (* TxCpltCallback)(struct __UART_HandleTypeDef *huart);
  void (* RxHalfCpltCallback)(struct __UART_HandleTypeDef *huart);
  void (* RxCpltCallback)(struct __UART_HandleTypeDef *huart);
  void (* ErrorCallback)(struct __UART_HandleTypeDef *huart);
  void (* AbortCpltCallback)(struct __UART_HandleTypeDef *huart);
  void (* AbortTransmitCpltCallback)(struct __UART_HandleTypeDef *huart);
  void (* AbortReceiveCpltCallback)(struct __UART_HandleTypeDef *huart);
  void (* WakeupCallback)(struct __UART_HandleTypeDef *huart);
  void (* RxFifoFullCallback)(struct __UART_HandleTypeDef *huart);
  void (* TxFifoEmptyCallback)(struct __UART_HandleTypeDef *huart);
  void (* MspInitCallback)(struct __UART_HandleTypeDef *huart);
  void (* MspDeInitCallback)(struct __UART_HandleTypeDef *huart);
#endif
} UART_HandleTypeDef;
```

注意事项：

条件编译 USE_HAL_HRTIM_REGISTER_CALLBACKS 用来设置使用自定义回调还是使用默认回调，此定义一般放在 stm32h7xx_hal_conf.h 文件里面设置：

```
#define    USE_HAL_UAR_REGISTER_CALLBACKS    1
```

通过函数 HAL_UART_RegisterCallback 注册回调,取消注册使用函数 HAL_UART_UnRegisterCallback。

这里重点介绍前三个参数，其它参数主要是 HAL 库内部使用和自定义回调函数。

- USART_TypeDef *Instance

  这个参数是寄存器的例化，方便操作寄存器，比如使能串口的发送空中断。

  SET_BIT(huart->Instance->CR1, USART_CR1_TXEIE)。

- UART_InitTypeDef Init

  这个参数是用户接触最多的，用于配置串口的基本参数，像波特率、奇偶校验、停止位等。

  UART_InitTypeDef 结构体的定义如下：

```
typedef struct
{
  uint32_t BaudRate;       /* 波特率 */
  uint32_t WordLength;     /* 数据位长度 */
  uint32_t StopBits;       /* 停止位 */
  uint32_t Parity;         /* 奇偶校验位 */
  uint32_t Mode;           /* 发送模式和接收模式使能 */
  uint32_t HwFlowCtl;      /* 硬件流控制 */
  uint32_t OverSampling;   /* 过采样，可以选择 8 倍和 16 倍过采样 */
```

```
  uint32_t Prescaler;      /* 串口分频 */
  uint32_t FIFOMode;         /* 串口 FIFO 使能 */
  uint32_t TXFIFOThreshold;  /* 发送 FIFO 的阀值 */
  uint32_t RXFIFOThreshold;  /* 接收 FIFO 的阀值 */
}UART_InitTypeDef;
```

- UART_AdvFeatureInitTypeDef AdvancedInit

这个参数用于配置串口的高级特性。具体支持的功能参数如下：

```
typedef struct
{
  uint32_t AdvFeatureInit;       /* 初始化的高级特性类别 */
  uint32_t TxPinLevelInvert;     /* Tx 引脚电平翻转 */
  uint32_t RxPinLevelInvert;     /* Rx 引脚电平翻转 */
  uint32_t DataInvert;           /* 数据逻辑电平翻转 */
  uint32_t Swap;                 /* Tx 和 Rx 引脚交换 */
  uint32_t OverrunDisable;       /* 接收超时检测禁止 */
  uint32_t DMADisableonRxError;  /* 接收出错，禁止 DMA */
  uint32_t AutoBaudRateEnable;   /* 自适应波特率使能 */
  uint32_t AutoBaudRateMode;     /* 自适应波特率的四种检测模式选择 */
  uint32_t MSBFirst;             /* 发送或者接收数据时，高位在前 */
} UART_AdvFeatureInitTypeDef;
```

配置串口参数，其实就是配置结构体 UART_HandleTypeDef 的成员。比如下面配置为波特率 115200，8 个数据位，无奇偶校验，1 个停止位。

```
UART_HandleTypeDef UartHandle;

/* 配置如下:
  - 数据位 = 8 Bits
  - 停止位 = 1 bit
  - 奇偶校验位 = 无
  - 波特率 = 115200bsp
  - 硬件流控制 (RTS 和 CTS 信号) */
UartHandle.Instance        = LPUART1;

UartHandle.Init.BaudRate     = 115200;
UartHandle.Init.WordLength   = UART_WORDLENGTH_8B;
UartHandle.Init.StopBits     = UART_STOPBITS_1;
UartHandle.Init.Parity       = UART_PARITY_NONE;
UartHandle.Init.HwFlowCtl    = UART_HWCONTROL_NONE;
UartHandle.Init.Mode         = UART_MODE_TX_RX;
UartHandle.Init.OverSampling = UART_OVERSAMPLING_16;
UartHandle.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;

if(HAL_UART_Init(&UartHandle) != HAL_OK)
{
    Error_Handler();
}
```

65.3.3 低功耗串口的底层配置（GPIO、时钟、中断等）

串口外设的基本参数配置完毕后还不能使用，还需要配置 GPIO、时钟、中断等参数，比如下面配置使用引脚 PA9 和 PA10。

```
/* LPUART1 的 GPIO  PA9, PA10 */
#define LPUART1_CLK_ENABLE()              __HAL_RCC_LPUART1_CLK_ENABLE()
```

```
#define LPUART1_TX_GPIO_CLK_ENABLE()      __HAL_RCC_GPIOA_CLK_ENABLE()
#define LPUART1_TX_GPIO_PORT              GPIOA
#define LPUART1_TX_PIN                    GPIO_PIN_9
#define LPUART1_TX_AF                     GPIO_AF3_LPUART

#define LPUART1_RX_GPIO_CLK_ENABLE()      __HAL_RCC_GPIOA_CLK_ENABLE()
#define LPUART1_RX_GPIO_PORT              GPIOA
#define LPUART1_RX_PIN                    GPIO_PIN_10
#define LPUART1_RX_AF                     GPIO_AF3_LPUART

/*
*********************************************************************************************************
*    函 数 名: InitHardUart
*    功能说明: 配置串口的硬件参数和底层
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void InitHardUart(void)
{
    GPIO_InitTypeDef  GPIO_InitStruct;
    RCC_PeriphCLKInitTypeDef RCC_PeriphClkInit;

#if LPUART1_FIFO_EN == 1
    /* 使能 GPIO TX/RX 时钟 */
    LPUART1_TX_GPIO_CLK_ENABLE();
    LPUART1_RX_GPIO_CLK_ENABLE();

    /* 使能 USARTx 时钟 */
    LPUART1_CLK_ENABLE();

    /* 配置 TX 引脚 */
    GPIO_InitStruct.Pin       = LPUART1_TX_PIN;
    GPIO_InitStruct.Mode      = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Pull      = GPIO_PULLUP;
    GPIO_InitStruct.Speed     = GPIO_SPEED_FREQ_HIGH;
    GPIO_InitStruct.Alternate = LPUART1_TX_AF;
    HAL_GPIO_Init(LPUART1_TX_GPIO_PORT, &GPIO_InitStruct);

    /* 配置 RX 引脚 */
    GPIO_InitStruct.Pin = LPUART1_RX_PIN;
    GPIO_InitStruct.Alternate = LPUART1_RX_AF;
    HAL_GPIO_Init(LPUART1_RX_GPIO_PORT, &GPIO_InitStruct);

    /* 配置 NVIC the NVIC for UART */
    HAL_NVIC_SetPriority(LPUART1_IRQn, 0, 1);
    HAL_NVIC_EnableIRQ(LPUART1_IRQn);

    /* 配置波特率、奇偶校验 */
    bsp_SetLPUartParam(LPUART1,  LPUART1_BAUD, UART_PARITY_NONE, UART_MODE_TX_RX);

    SET_BIT(LPUART1->ICR, USART_ICR_TCCF);    /* 清除 TC 发送完成标志 */
    SET_BIT(LPUART1->RQR, USART_RQR_RXFRQ);   /* 清除 RXNE 接收标志 */
    SET_BIT(LPUART1->CR1, USART_CR1_RXNEIE); /* 使能 PE. RX 接受中断 */
#endif
}
```

总结下来就是以下几点：

- 配置 GPIO 引脚时钟。
- 配置 LPUART 时钟。
- 配置 LPUART 的发送和接收引脚。
- 通过 NVIC 配置中断。
- 配置波特率，奇偶校验等，在上一小节有讲。
- 清除 TC 和 RXNE 标志，使能接收中断。

关于这个底层配置有以下几点要着重说明下：

- 串口发送和接收引脚的复用模式选择已经被 HAL 库定义好，放在了 stm32h7xx_hal_gpio_ex.h 文件里面。比如串口 1 有两个复用

```
#define GPIO_AF3_LPUART    ((uint8_t)0x03) /* LPUART Alternate Function mapping */
#define GPIO_AF8_LPUART    ((uint8_t)0x08) /* LPUART Alternate Function mapping */
```

具体使用那个，要看数据手册，比如我们这里使用引脚 PA9 和 PA10，对应的复用如下：

Port		AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7	AF8	AF9	AF10	AF11	AF12	AF13	AF14	AF15
		SYS	TIM1/2/16/17/LPTIM1/HRTIM1	SAI1/TIM3/4/5/12/HRTIM1	LPUART/TIM8/LPTIM2/3/4/5/HRTIM1/DFSDM1	I2C1/2/3/4/USART1/TIM15/LPTIM2/DFSDM1/CEC	SPI1/2/3/4/5/6/CEC	SPI2/3/SAI1/3/I2C4/UART4/DFSDM1	SPI2/3/6/USART1/2/3/6/UART7/SDMMC1	SPI6/SAI2/4/UART4/5/8/LPUART/SDMMC1/SPDIFRX1	SAI4/FDCAN1/2/TIM13/14/QUADSPI/FMC/SDMMC2/LCD/SPDIFRX1	SAI2/4/TIM8/QUADSPI/SDMMC2/OTG1_HS/OTG2_FS/LCD	I2C4/UART7/SWPMI1/TIM1/8/DFSDM1/SDMMC2/MDIOS/ETH	TIM1/8/FMC/SDMMC1/MDIOS/OTG1_FS/LCD	TIM1/DCMI/LCD/COMP	UART5/LCD	SYS
Port A	PA0	-	TIM2_CH1/TIM2_ETR	TIM5_CH1	TIM8_ETR	TIM15_BKIN	-	-	USART2_CTS/USART2_NSS	UART4_TX	SDMMC2_CMD	SAI2_SD_B	ETH_MII_CRS	-	-	-	EVENTOUT
	PA1	-	TIM2_CH2	TIM5_CH2	LPTIM3_OUT	TIM15_CH1N	-	-	USART2_RTS/USART2_DE	UART4_RX	QUADSPI_BK1_IO3	SAI2_MCLK_B	ETH_MII_RX_CLK/ETH_RMII_REF_CLK	-	-	LCD_R2	EVENTOUT
	PA2	-	TIM2_CH3	TIM5_CH3	LPTIM4_OUT	TIM15_CH1	-	-	USART2_TX	SAI2_SCK_B	-	-	ETH_MDIO	MDIOS_MDIO	-	LCD_R1	EVENTOUT
	PA3	-	TIM2_CH4	TIM5_CH4	LPTIM5_OUT	TIM15_CH2	-	-	USART2_RX	-	LCD_B2	OTG_HS_ULPI_D0	ETH_MII_COL	-	-	LCD_B5	EVENTOUT
	PA4	D1PWREN	-	TIM5_ETR	-	-	SPI1_NSS/I2S1_WS	SPI3_NSS/I2S3_WS	USART2_CK	SPI6_NSS	-	-	-	OTG_HS_SOF	DCMI_HSYNC	LCD_VSYNC	EVENTOUT
	PA5	D2PWREN	TIM2_CH1/TIM2_ETR	-	TIM8_CH1N	-	SPI1_SCK/I2S1_CK	-	-	SPI6_SCK	-	OTG_HS_ULPI_CK	-	-	-	LCD_R4	EVENTOUT
	PA6	-	TIM1_BKIN	TIM3_CH1	TIM8_BKIN	-	SPI1_MISO/I2S1_SDI	-	-	SPI6_MISO	TIM13_CH1	TIM8_BKIN_COMP12	MDIOS_MDC	TIM1_BKIN_COMP12	DCMI_PIXCLK	LCD_G2	EVENTOUT
	PA7	-	TIM1_CH1N	TIM3_CH2	TIM8_CH1N	-	SPI1_MOSI/I2S1_SDO	-	-	SPI6_MOSI	TIM14_CH1	-	ETH_MII_RX_DV/ETH_RMII_CRS_DV	FMC_SDNWE	-	-	EVENTOUT
	PA8	MCO1	TIM1_CH1	HRTIM_CHB2	TIM8_BKIN2	I2C3_SCL	-	-	USART1_CK	-	-	OTG_FS_SOF	UART7_RX	TIM8_BKIN2_COMP12	LCD_B3	LCD_R6	EVENTOUT
	PA9	-	TIM1_CH2	HRTIM_CHC1	LPUART1_TX	I2C3_SMBA	SPI2_SCK/I2S2_CK	-	USART1_TX	-	FDCAN1_RXFD_MODE	-	-	-	DCMI_D0	LCD_R5	EVENTOUT
	PA10	-	TIM1_CH3	HRTIM_CHC2	LPUART1_RX	-	-	-	USART1_RX	-	FDCAN1_TXFD_MODE	OTG_FS_ID	MDIOS_MDIO	LCD_B4	DCMI_D1	LCD_B1	EVENTOUT
	PA11	-	TIM1_CH4	HRTIM_CHD1	LPUART1_CTS	-	SPI2_NSS/I2S2_WS	UART4_RX	USART1_CTS/USART1_NSS	-	FDCAN1_RX	OTG_FS_DM	-	-	-	LCD_R4	EVENTOUT
	PA12	-	TIM1_ETR	HRTIM_CHD2	LPUART1_RTS/LPUART1_DE	-	SPI2_SCK/I2S2_CK	UART4_TX	USART1_RTS/USART1_DE	SAI2_FS_B	FDCAN1_TX	OTG_FS_DP	-	-	-	LCD_R5	EVENTOUT

那么使用 GPIO_AF3_LPUART1 即可。

- 根据情况要清除 TC 发送完成标志和 RXNE 接收数据标志，因为这两个标志位在使能了串口后就已经置位，所以当用户使用了 TC 或者 RX 中断后，就会进入一次中断服务程序，这点要特别注意。
- HAL 库有个自己的底层初始化回调函数 HAL_UART_MspInit，是弱定义的，用户可以在其它的 C 文

件里面实现，并将相对的底层初始化在里面实现。当用户调用 HAL_UART_Init 后，会在此函数里面调用 HAL_UART_MspInit，对应的底层复位函数 HAL_UART_MspDeInit 是在函数 HAL_UART_DeInit 里面被调用的。

当然，用户也可以自己初始化，不限制必须在两个函数里面实现。

- 上面举的例子里面没有用到 DMA，如果用到了 DMA，也是要初始化的。

65.3.4低功耗串口的状态标志清除问题

注，早前使用 F1 和 F4 时候，经常会有网友咨询为什么串口中断服务程序里面没有做清除标志。

下面我们介绍__HAL_USART_GET_FLAG 函数。这个函数用来检查 LPUART/USART 标志位是否被设置。

```
/** @brief  Check whether the specified USART flag is set or not.
  * @param  __HANDLE__: specifies the USART Handle
  * @param  __FLAG__: specifies the flag to check.
  *        This parameter can be one of the following values:
  *           @arg USART_FLAG_TXFT: TXFIFO threshold flag
  *           @arg USART_FLAG_RXFT: RXFIFO threshold flag
  *           @arg USART_FLAG_RXFF:  RXFIFO Full flag
  *           @arg USART_FLAG_TXFE:  TXFIFO Empty flag
  *           @arg USART_FLAG_REACK: Receive enable ackowledge flag
  *           @arg USART_FLAG_TEACK: Transmit enable ackowledge flag
  *           @arg USART_FLAG_BUSY:  Busy flag
  *           @arg USART_FLAG_TXE:   Transmit data register empty flag
  *           @arg USART_FLAG_TC:    Transmission Complete flag
  *           @arg USART_FLAG_RXNE:  Receive data register not empty flag
  *           @arg USART_FLAG_IDLE:  Idle Line detection flag
  *           @arg USART_FLAG_ORE:   OverRun Error flag
  *           @arg USART_FLAG_UDR:   UnderRun Error flag
  *           @arg USART_FLAG_NE:    Noise Error flag
  *           @arg USART_FLAG_FE:    Framing Error flag
  *           @arg USART_FLAG_PE:    Parity Error flag
  * @retval The new state of __FLAG__ (TRUE or FALSE).
  */
#define __HAL_USART_GET_FLAG(__HANDLE__, __FLAG__) (((__HANDLE__)->Instance->ISR & (__FLAG__)) == (__FLAG__))
```

USART_FLAG 有如下几种取值：

USART_FLAG_CTS	CTS 标志 (CTS flag) 如果设置了CTSE位，当nCTS输入变化状态时，该位被硬件置高，由软件将其清零。如果USART_CR3中的CTSIE为‘1’，则产生中断。 0： nCTS状态线上没有变化； 1： nCTS状态线上发生变化。 注： 异步串口不支持这一位。
USART_FLAG_LBD	LBD: LIN断开检测标志 (LIN break detection flag) 当探测到LIN断开时，该位由硬件置‘1’，由软件清‘0’(将USART_ICR的LBDCF位置1)。如果USART_CR2中的LBDIE = 1，则产生中断。 0：没有检测到LIN断开；

	1：检测到LIN断开。 注意：若LBDIE=1，当LBD为’1’时要产生中断。
USART_FLAG_TXE **(必须了解的)**	TXE:发送数据寄存器空（Transmit data register empty） 当TDR寄存器中的数据被硬件转移到移位寄存器的时候，该位被硬件置位。如果USART_CR1寄存器中的TXEIE为1，则产生中断。对USART_TDR的写操作，将该位清零，也可以将USART_RQR的TXFRQ位置1实现清除标志。 0：数据还没有被转移到移位寄存器； 1：数据已经被转移到移位寄存器。 注意：单缓冲器传输中使用该位。
USART_FLAG_TC **(必须了解的)**	TC：发送完成（Transmission complete） 当包含有数据的一帧发送完成后，并且TXE=1时，由硬件将该位置’1’。如果USART_CR1中的TCIE为’1’，则产生中断。对USART_TDR的写操作可以清零该位，也可以将USART_ICR的TCCF位置1实现清除标志。 0：发送还未完成； 1：发送完成。
USART_FLAG_RXNE **(必须了解的)**	RXNE：接收数据寄存器非空（Read data register not empty） 当RDR移位寄存器中的数据被转移到USART_DR寄存器中，该位被硬件置位。如果USART_CR1寄存器中的RXNEIE为1，则产生中断。对USART_RDR的读操作可以将该位清零，也可以将USART_RQR的RXFRQ位置1实现清除标志。 0：数据没有收到； 1：收到数据，可以读出。
USART_FLAG_IDLE	IDLE：监测到总线空闲（IDLE line detected） 当检测到总线空闲时，该位被硬件置位。如果USART_CR1中的IDLEIE为’1’，则产生中断。将USART_ICR的IDLECF位置1实现清除标志。 0：没有检测到空闲总线； 1：检测到空闲总线。 注意： IDLE位不会再次被置高直到RXNE位被置起(即又检测到一次空闲总线)
USART_FLAG_ORE	ORE：过载错误（Overrun error） 当RXNE仍然是’1’的时候，当前被接收在移位寄存器中的数据，需要传送至RDR寄存器时，硬件将该位置位。如果USART_CR1中的RXNEIE为’1’的话，则产生中断。将USART_ICR的ORECF位置1实现清除标志。 0：没有过载错误； 1：检测到过载错误。

	注意：该位被置位时， RDR寄存器中的值不会丢失，但是移位寄存器中的数据会被覆盖。如果设置了EIE位，在多缓冲器通信模式下， ORE标志置位会产生中断的。
USART_FLAG_NE	NE: 噪声错误标志 (Noise error flag) 在接收到的帧检测到噪音时，由硬件对该位置位。将USART_ICR的NFCF位置1实现清除标志。 0：没有检测到噪声； 1：检测到噪声。 注意：该位不会产生中断，因为它和RXNE一起出现，硬件会在设置RXNE标志时产生中断。在多缓冲区通信模式下，如果设置了EIE位，则设置NE标志时会产生中断。
USART_FLAG_FE	FE: 帧错误 (Framing error) 当检测到同步错位，过多的噪声或者检测到断开符，该位被硬件置位。将USART_ICR的FECF位置1实现清除标志。 0：没有检测到帧错误； 1：检测到帧错误或者break符。
USART_FLAG_PE	PE: 校验错误 (Parity error) 在接收模式下，如果出现奇偶校验错误，硬件对该位置位。将USART_ICR的PECF位置1实现清除标志。如果USART_CR1中的PEIE为’1’，则产生中断。 0：没有奇偶校验错误； 1：奇偶校验错误。

请大家重点关注上表中红字部分，LPUART/USART 标志是需要软件主动清零的。清零有两种方式：一种是调用__HAL_USART_CLEAR_FLAG 函数，另一种是操作相关寄存器后自动清零。

```
/** @brief  Clear the specified USART pending flag.
  * @param  __HANDLE__: specifies the USART Handle.
  * @param  __FLAG__: specifies the flag to check.
  *          This parameter can be any combination of the following values:
  *            @arg USART_FLAG_TXFT:  TXFIFO threshold flag
  *            @arg USART_FLAG_RXFT:  RXFIFO threshold flag
  *            @arg USART_FLAG_RXFF:  RXFIFO Full flag
  *            @arg USART_FLAG_TXFE:  TXFIFO Empty flag
  *            @arg USART_FLAG_REACK: Receive enable ackowledge flag
  *            @arg USART_FLAG_TEACK: Transmit enable ackowledge flag
  *            @arg USART_FLAG_WUF:   Wake up from stop mode flag
  *            @arg USART_FLAG_RWU:   Receiver wake up flag (is the USART in mute mode)
  *            @arg USART_FLAG_SBKF:  Send Break flag
  *            @arg USART_FLAG_CMF:   Character match flag
  *            @arg USART_FLAG_BUSY:  Busy flag
  *            @arg USART_FLAG_ABRF:  Auto Baud rate detection flag
  *            @arg USART_FLAG_ABRE:  Auto Baud rate detection error flag
  *            @arg USART_FLAG_RTOF:  Receiver timeout flag
  *            @arg USART_FLAG_LBD:   LIN Break detection flag
```

```
  *            @arg USART_FLAG_TXE:   Transmit data register empty flag
  *            @arg USART_FLAG_TC:    Transmission Complete flag
  *            @arg USART_FLAG_RXNE:  Receive data register not empty flag
  *            @arg USART_FLAG_IDLE:  Idle Line detection flag
  *            @arg USART_FLAG_ORE:   OverRun Error flag
  *            @arg USART_FLAG_NE:    Noise Error flag
  *            @arg USART_FLAG_FE:    Framing Error flag
  *            @arg USART_FLAG_PE:    Parity Error flag
  * @retval The new state of __FLAG__ (TRUE or FALSE).
  */
#define __HAL_USART_CLEAR_FLAG(__HANDLE__, __FLAG__) ((__HANDLE__)->Instance->ICR = (__FLAG__))
```

上面介绍的 USART 标志大部分能够设置为产生中断，也就是有对应的 USART 中断标志。我们只介绍几个串口驱动要用到的中断标志：

USART_IT_TXE： TXE:发送数据寄存器空（此时数据可能正在发送）。

USART_IT_TC： 发送完成 。

USART_IT_RXNE： 接收数据寄存器非空。

中断缺省都是关闭的，通过__HAL_USART_ENABLE_IT 函数可以使能相应的中断标志。函数定义如下：

```
/** @brief  Enable the specified USART interrupt.
  * @param  __HANDLE__: specifies the USART Handle.
  * @param  __INTERRUPT__: specifies the USART interrupt source to enable.
  *          This parameter can be one of the following values:
  *            @arg USART_IT_RXFF: RXFIFO Full interrupt
  *            @arg USART_IT_TXFE: TXFIFO Empty interrupt
  *            @arg USART_IT_RXFT: RXFIFO threshold interrupt
  *            @arg USART_IT_TXFT: TXFIFO threshold interrupt
  *            @arg USART_IT_TXE : Transmit Data Register empty interrupt
  *            @arg USART_IT_TC  : Transmission complete interrupt
  *            @arg USART_IT_RXNE: Receive Data register not empty interrupt
  *            @arg USART_IT_IDLE: Idle line detection interrupt
  *            @arg USART_IT_PE  : Parity Error interrupt
  *            @arg USART_IT_ERR : Error interrupt(Frame error, noise error, overrun error)
  * @retval None
  */
#define __HAL_USART_ENABLE_IT(__HANDLE__, __INTERRUPT__)   (((((uint8_t)(__INTERRUPT__)) >> 5U) == 1)?
                        ((__HANDLE__)->Instance->CR1 |= (1U << ((__INTERRUPT__) & USART_IT_MASK))): \
                                                        ((((uint8_t)(__INTERRUPT__)) >> 5U) == 2)?
                         ((__HANDLE__)->Instance->CR2 |= (1U << ((__INTERRUPT__) & USART_IT_MASK))): \
                          ((__HANDLE__)->Instance->CR3 |= (1U << ((__INTERRUPT__) & USART_IT_MASK))))
```

STM32 一个串口的中断服务程序入口地址只有一个，进入中断服务程序后，我们需要判断是什么原因进入的中断，因此需要调用一个函数来检测中断标志。函数原型如下：

```
#define __HAL_USART_GET_IT(__HANDLE__, __IT__) ((__HANDLE__)->Instance->ISR & ((uint32_t)1 << ((__IT__)>>
0x08)))
```

中断处理完毕后，必须软件清除中断标志，否则中断返回后，会重入中断。清中断标志位的函数为：

```
#define __HAL_USART_CLEAR_IT(__HANDLE__, __IT_CLEAR__) ((__HANDLE__)->Instance->ICR =
(uint32_t)(__IT_CLEAR__))
```

正如前面介绍的，不是所有的标志都需要用这个函数清零。

注意：操作串口的寄存器不限制必须要用 HAL 库提供的 API，比如要操作寄存器 CR1，直接调用 LPUART1->CR1 操作即可。

65.3.5 低功耗串口初始化流程总结

使用方法由 HAL 库提供：

- 第 1 步：定义 UART_HandleTypeDef 类型串口结构体变量，比如 UART_HandleTypeDef huart。
- 第 2 步：使用函数 HAL_UART_MspInit 初始化串口底层，不限制一定要用此函数里面初始化，用户也可以自己实现。
  - 使能串口时钟。
  - 引脚配置。
    a、使能串口所使用的 GPIO 时钟。
    b、配置 GPIO 的复用模式。
  - 如果使用中断方式函数 HAL_UART_Transmit_IT 和 HAL_UART_Receive_IT 需要做如下配置。
    a、配置串口中断优先级。
    b、使能串口中断。
  - 串口中断的开关是通过函数__HAL_UART_ENABLE_IT() 和 __HAL_UART_DISABLE_IT()来实现，这两个函数被嵌套到串口的发送和接收函数中调用。
  - 如果使用 DMA 方式函数 HAL_UART_Transmit_DMA 和 HAL_UART_Receive_DMA 需要做如下配置。
    a、声明串口的发送和接收 DMA 结构体变量，注意发送和接收是独立的，如果都使用，那就都需要配置。
    b、使能 DMA 接口时钟。
    c、配置串口的发送和接收 DMA 结构体变量。
    d、配置 DMA 发送和接收通道。
    e、关联 DMA 和串口的句柄。
    f、配置发送 DMA 和接收 DMA 的传输完成中断和中断优先级。
- 第 3 步：配置串口的波特率，位长，停止位，奇偶校验位，流控制和发送接收模式。
- 第 4 步：如果需要，可以编程高级特性，比如 TX/RX 交换引脚，自动波特率检测。通过第 1 步串口结构体变量 huart 的结构体成员 AdvancedInit 来设置。
- 第 5 步：串口初始化调用的函数 HAL_UART_Init 初始化。
- 第 6 步：根据需要可以做动态注册回调。

  首先使能宏定义 USE_HAL_UART_REGISTER_CALLBACKS。

  然后调用函数 HAL_UART_RegisterCallback() 就可以注册如下回调函数：

(+) TxHalfCpltCallback

(+) TxCpltCallback

(+) RxHalfCpltCallback

(+) RxCpltCallback
(+) ErrorCallback
(+) AbortCpltCallback
(+) AbortTransmitCpltCallback
(+) AbortReceiveCpltCallback
(+) WakeupCallback
(+) RxFifoFullCallback
(+) TxFifoEmptyCallback
(+) MspInitCallback
(+) MspDeInitCallback

函数 HAL_UART_UnRegisterCallback 允许取消注册的回调函数如下：

(+) TxHalfCpltCallback
(+) TxCpltCallback
(+) RxHalfCpltCallback
(+) RxCpltCallback
(+) ErrorCallback
(+) AbortCpltCallback
(+) AbortTransmitCpltCallback
(+) AbortReceiveCpltCallback
(+) WakeupCallback
(+) RxFifoFullCallback
(+) TxFifoEmptyCallback
(+) MspInitCallback
(+) MspDeInitCallback

关于动态注册回调函数注意以下几点：

- 默认情况下，HAL_UART_Init 调用后将使用默认的弱定义回调，如果用户注册了回调，将使用用户设置的。
- 回调函数只能在 HAL_UART_STATE_READY 状态下才可以注册/注销。
  - 回调函数 MspInit 和 MspDeInit 除外，这两个函数可以在 HAL_HRTIM_STATE_READY 或 HAL_HRTIM_STATE_RESET 状态下注册，这样的话，用户调用函数 HAL_HRTIM_DeInit()或者 HAL_HRTIM_Init()时，就可以在其函数内运行 MspInit/DeInit。
  - 用户可以在调用 HAL_HRTIM_DeInit()或者 HAL_HRTIM_Init()之前调用 HAL_HRTIM_RegisterCallback()为 MspInit/MspDeInit 注册回调。

65.4 低功耗源文件 stm32h7xx_hal_uart.c

此文件涉及到的函数较多，这里把几个常用的函数做个说明：

- HAL_UART_Init
- HAL_UART_Transmit
- HAL_UART_Receive
- HAL_UART_Transmit_IT
- HAL_UART_Receive_IT
- HAL_UART_Transmit_DMA
- HAL_UART_Receive_DMA

其实 V7 开发板设计的低功耗串口 FIFO 驱动文件 bsp_lpuart_fifo.c 仅用到了函数 HAL_UART_Init，其它函数都没有用到，不过这里也为大家做个说明。

65.4.1 函数 HAL_UART_Init

函数原型：

```
HAL_StatusTypeDef HAL_UART_Init(UART_HandleTypeDef *huart)
{

   /* 省略 */

  if(huart->gState == HAL_UART_STATE_RESET)
  {
    huart->Lock = HAL_UNLOCKED;

    /* 初始化硬件: GPIO, CLOCK */
    HAL_UART_MspInit(huart);
  }

  huart->gState = HAL_UART_STATE_BUSY;

  /* 禁止串口 */
  __HAL_UART_DISABLE(huart);

  /* 配置串口参数 */
  if (UART_SetConfig(huart) == HAL_ERROR)
  {
    return HAL_ERROR;
  }

   /* 配置串口高级特性 */
  if (huart->AdvancedInit.AdvFeatureInit != UART_ADVFEATURE_NO_INIT)
  {
    UART_AdvFeatureConfig(huart);
  }
```

```
  /* 清寄存器的一些标志位 */
  CLEAR_BIT(huart->Instance->CR2, (USART_CR2_LINEN | USART_CR2_CLKEN));
  CLEAR_BIT(huart->Instance->CR3, (USART_CR3_SCEN | USART_CR3_HDSEL | USART_CR3_IREN));

  /* 使能串口 */
  __HAL_UART_ENABLE(huart);

  return (UART_CheckIdleState(huart));
}
```

函数描述：

此函数用于初始化串口的基础特性和高级特性。

函数参数：

- 第 1 个参数是 UART_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

注意事项：

1. 函数 HAL_UART_MspInit 用于初始化 USART 的底层时钟、引脚等功能。需要用户自己在此函数里面实现具体的功能。由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
2. 如果形参 huart 的结构体成员 gState 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 UART_HandleTypeDef UartHandle。

   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_UART_STATE_RESET = 0x00U。

   解决办法有三

   方法 1：用户自己初始串口和涉及到的 GPIO 等。

   方法 2：定义 UART_HandleTypeDef UartHandle 为全局变量。

   方法 3：下面的方法

```
if(HAL_UART_DeInit(&UartHandle) != HAL_OK)
{
    Error_Handler();
}
if(HAL_UART_Init(&UartHandle) != HAL_OK)
{
    Error_Handler();
}
```

3. 注意串口的中断状态寄存器 USART_ISR 复位后，TC 发送完成状态和 RXNE 接收状态都被置 1，如果用户使能这两个中断前，最好优先清除中断标志。

使用举例：

```
UART_HandleTypeDef UartHandle;

/* USART3 工作在 UART 模式 */
/* 配置如下:
  - 数据位 = 8 Bits
  - 停止位 = 1 bit
```

```
  - 奇偶校验位 = 无
  - 波特率 = 115200bsp
  - 硬件流控制 (RTS 和 CTS 信号) */
UartHandle.Instance        = USART3;

UartHandle.Init.BaudRate     = 115200;
UartHandle.Init.WordLength   = UART_WORDLENGTH_8B;
UartHandle.Init.StopBits     = UART_STOPBITS_1;
UartHandle.Init.Parity       = UART_PARITY_NONE;
UartHandle.Init.HwFlowCtl    = UART_HWCONTROL_NONE;
UartHandle.Init.Mode         = UART_MODE_TX_RX;
UartHandle.Init.OverSampling = UART_OVERSAMPLING_16;
UartHandle.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;

if(HAL_UART_Init(&UartHandle) != HAL_OK)
{
    Error_Handler();
}
```

65.4.2 函数 HAL_UART_Transmit

函数原型：

```
HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout)
{
  /* 省略 */

  if(huart->gState == HAL_UART_STATE_READY)
  {
    /* 省略 */

    while(huart->TxXferCount > 0U)
    {
      huart->TxXferCount--;
      /* 等待发送空中断标志 */
      if(UART_WaitOnFlagUntilTimeout(huart, UART_FLAG_TXE, RESET, tickstart, Timeout) != HAL_OK)
      {
        return HAL_TIMEOUT;
      }
    }
    /* 等待发送完成中断 */
    if(UART_WaitOnFlagUntilTimeout(huart, UART_FLAG_TC, RESET, tickstart, Timeout) != HAL_OK)
    {
      return HAL_TIMEOUT;
    }

    /* 省略 */
    return HAL_OK;
  }
  else
  {
    return HAL_BUSY;
  }
}
```

函数描述：

此函数以查询的方式发送指定字节。看源码的话，程序里面最重要的就是上面代码中置红的两个标志，发送空标志和发送完成标志。发送空标志表示发送数据寄存器为空，数据还在移位寄存器里面，而发送完成标志表示数据已经从移位寄存器发送出去。

函数参数：

- 第 1 个参数是 UART_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是要发送的数据地址。
- 第 3 个参数是要发送的数据大小，单位字节。
- 第 4 个参数是溢出时间，单位 ms。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: fputc
*    功能说明: 重定义 putc 函数，这样可以使用 printf 函数从串口 1 打印输出
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
int fputc(int ch, FILE *f)
{
    HAL_UART_Transmit(&UartHandle, (uint8_t *)&ch, 1, HAL_MAX_DELAY);

    return ch;
}
```

65.4.3 函数 HAL_UART_Receive

函数原型：

```
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout)
{

  /* 省略 */
  if(huart->RxState == HAL_UART_STATE_READY)
  {

    /* 省略 */
    while(huart->RxXferCount > 0U)
    {
      huart->RxXferCount--;
      if(UART_WaitOnFlagUntilTimeout(huart, UART_FLAG_RXNE, RESET, tickstart, Timeout) != HAL_OK)
      {
        return HAL_TIMEOUT;
      }


    }
    /* 省略 */
```

```
    return HAL_OK;
  }
  else
  {
    /* 省略 */
    return HAL_BUSY;
  }
}
```

函数描述：

此函数以查询的方式接收指定字节。这个函数相对比较好理解，就是等待上面程序中的 RXNE 标志，置位了表示接收数据寄存器已经存入数据。

函数参数：

- 第 1 个参数是 UART_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是要接收的数据地址。
- 第 3 个参数是要接收的数据大小，单位字节。
- 第 4 个参数是溢出时间，单位 ms。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: fgetc
*    功能说明: 重定义 getc 函数，这样可以使用 scanff 函数从串口 1 输入数据
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
int fgetc(FILE *f)
{
    int ret;

    HAL_UART_Receive(&UartHandle, (uint8_t *)&ret, 1, HAL_MAX_DELAY);

    return ret;
}
```

65.4.4函数 HAL_UART_Transmit_IT

函数原型：

```
HAL_StatusTypeDef HAL_UART_Transmit_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)
{

  if(huart->gState == HAL_UART_STATE_READY)
  {
    if((pData == NULL ) || (Size == 0U))
    {
      return HAL_ERROR;
    }
```

```
    __HAL_LOCK(huart);

    huart->pTxBuffPtr = pData;
    huart->TxXferSize = Size;
    huart->TxXferCount = Size;

    huart->ErrorCode = HAL_UART_ERROR_NONE;
    huart->gState = HAL_UART_STATE_BUSY_TX;


    __HAL_UNLOCK(huart);

    if (READ_BIT(huart->Instance->CR1, USART_CR1_FIFOEN) != RESET)
    {
      /* 使能 FIFO 发送中断 */
      SET_BIT(huart->Instance->CR3, USART_CR3_TXFTIE);
    }
    else
    {
      /* 使能发空中断 */
      SET_BIT(huart->Instance->CR1, USART_CR1_TXEIE);
    }

    return HAL_OK;
  }
  else
  {
    return HAL_BUSY;
  }
}
```

函数描述：

此函数以中断的方式发送指定字节，可以选择使能 FIFO 中断方式或者发送空中断方式。具体数据的发送是在中断处理函数 HAL_UART_IRQHandler 里面实现。

函数参数：

- 第 1 个参数是 UART_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是要发送的数据地址。
- 第 3 个参数是要发送的数据大小，单位字节。
- 返回值，返回 HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

使用中断方式要使能串口中断，此贴有完整例子：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=86245 。

```
UART_HandleTypeDef UartHandle;
uint8_t s_ucBuf[5];

/* 数据发送 */
HAL_UART_Transmit_IT(&UartHandle, s_ucBuf, 1);
HAL_UART_Transmit_IT(&UartHandle, (uint8_t*)"KEY_DOWN_K1\r\n", 13);
```

65.4.5函数 HAL_UART_Receive_IT

函数原型：

```
HAL_StatusTypeDef HAL_UART_Receive_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)
{

  if(huart->RxState == HAL_UART_STATE_READY)
  {
    if((pData == NULL ) || (Size == 0U))
    {
      return HAL_ERROR;
    }

    __HAL_LOCK(huart);

    huart->pRxBuffPtr = pData;
    huart->RxXferSize = Size;
    huart->RxXferCount = Size;

    UART_MASK_COMPUTATION(huart);

    huart->ErrorCode = HAL_UART_ERROR_NONE;
    huart->RxState = HAL_UART_STATE_BUSY_RX;

    __HAL_UNLOCK(huart);

    /* 使能错误中断: (Frame error, noise error, overrun error) */
    SET_BIT(huart->Instance->CR3, USART_CR3_EIE);

    if (READ_BIT(huart->Instance->CR1, USART_CR1_FIFOEN) != RESET)
    {
      /* 使能奇偶校验失败中断 */
      SET_BIT(huart->Instance->CR1, USART_CR1_PEIE);
      /* 使能 FIFO 接收中断 */
      SET_BIT(huart->Instance->CR3, USART_CR3_RXFTIE);
    }
    else
    {
      /* 使能奇偶校验失败中断和接收中断 */
      SET_BIT(huart->Instance->CR1, USART_CR1_PEIE | USART_CR1_RXNEIE);
    }

    return HAL_OK;
  }
  else
  {
    return HAL_BUSY;
  }
}
```

函数描述：

此函数以中断的方式接收指定字节，可以选择使能 FIFO 中断方式或者普通中断方式，两种方式使能了奇偶校验中断失败和错误中断。具体数据的接收是在中断处理函数 HAL_UART_IRQHandler 里面实现。

函数参数：

- 第 1 个参数是 UART_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是要接收的数据地址。
- 第 3 个参数是要接收的数据大小，单位字节。
- 返回值，返回 HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

使用中断方式要使能串口中断，此贴有完整例子：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=86245 。

```
UART_HandleTypeDef UartHandle;
uint8_t s_ucBuf[5];

/* 数据接收*/
HAL_UART_Receive_IT(&UartHandle, s_ucBuf, 1);
```

65.4.6函数 HAL_UART_Transmit_DMA

函数原型：

```
HAL_StatusTypeDef HAL_UART_Transmit_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)
{
  if(huart->gState == HAL_UART_STATE_READY)
  {
    if((pData == NULL ) || (Size == 0U))
    {
      return HAL_ERROR;
    }

    __HAL_LOCK(huart);

    huart->pTxBuffPtr = pData;
    huart->TxXferSize = Size;
    huart->TxXferCount = Size;

    huart->ErrorCode = HAL_UART_ERROR_NONE;
    huart->gState = HAL_UART_STATE_BUSY_TX;

    /* 注册各种 DMA 回调函数 */
    huart->hdmatx->XferCpltCallback = UART_DMATransmitCplt;
    huart->hdmatx->XferHalfCpltCallback = UART_DMATxHalfCplt;
    huart->hdmatx->XferErrorCallback = UART_DMAError;
    huart->hdmatx->XferAbortCallback = NULL;

    /* 使能串口发送 DMA 通道 */
    HAL_DMA_Start_IT(huart->hdmatx, (uint32_t)huart->pTxBuffPtr, (uint32_t)&huart->Instance->TDR, Size);

    /* 清除传输 TC 完成标志 */
    __HAL_UART_CLEAR_FLAG(huart, UART_CLEAR_TCF);

    __HAL_UNLOCK(huart);

    /* 使能串口发送 DMA 传输 */
```

```
    SET_BIT(huart->Instance->CR3, USART_CR3_DMAT);

    return HAL_OK;
  }
  else
  {
    return HAL_BUSY;
  }
}
```

函数描述：

此函数以 DMA 的方式发送指定字节。这里是用的 DMA 中断方式 HAL_DMA_Start_IT 进行的发送。所以使用此函数的话，不要忘了写 DMA 中断服务程序。而且 DMA 的配置也是需要用户实现的，可以直接在函数 HAL_UART_MspInit 里面实现，也可以放在其它位置。

函数参数：

- 第 1 个参数是 UART_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是要发送的数据地址。
- 第 3 个参数是要发送的数据大小，单位字节。
- 返回值，返回 HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

使用 DMA 方式，此贴有完整例子：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=86271 。

65.4.7 函数 HAL_UART_Receive_DMA

函数原型：

```
HAL_StatusTypeDef HAL_UART_Receive_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)
{
  if(huart->RxState == HAL_UART_STATE_READY)
  {
    if((pData == NULL ) || (Size == 0U))
    {
      return HAL_ERROR;
    }

    __HAL_LOCK(huart);

    huart->pRxBuffPtr = pData;
    huart->RxXferSize = Size;

    huart->ErrorCode = HAL_UART_ERROR_NONE;
    huart->RxState = HAL_UART_STATE_BUSY_RX;

    /* 注册各种 DMA 回调函数 */
    huart->hdmarx->XferCpltCallback = UART_DMAReceiveCplt;
```

```
    huart->hdmarx->XferHalfCpltCallback = UART_DMARxHalfCplt;
    huart->hdmarx->XferErrorCallback = UART_DMAError;
    huart->hdmarx->XferAbortCallback = NULL;

    /* 使能串口接收 DMA 通道 */
    HAL_DMA_Start_IT(huart->hdmarx, (uint32_t)&huart->Instance->RDR, (uint32_t)huart->pRxBuffPtr, Size);

    __HAL_UNLOCK(huart);

    /* 使能串口校验错误中断 */
    SET_BIT(huart->Instance->CR1, USART_CR1_PEIE);

    /* 使能串口错误中断:(Frame error, noise error, overrun error) */
    SET_BIT(huart->Instance->CR3, USART_CR3_EIE);

    /* 使能串口接收 DMA 传输 */
    SET_BIT(huart->Instance->CR3, USART_CR3_DMAR);

    return HAL_OK;
  }
  else
  {
    return HAL_BUSY;
  }
}
```

函数描述:

此函数以 DMA 的方式接收指定字节。这里是用的 DMA 中断方式 HAL_DMA_Start_IT 进行的接收。所以使用此函数的话，不要忘了写 DMA 中断服务程序。而且 DMA 的配置也是需要用户实现的，可以直接在函数 HAL_UART_MspInit 里面实现，也可以放在其它位置。

函数参数:

- 第 1 个参数是 UART_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是要接收的数据地址。
- 第 3 个参数是要接收的数据大小，单位字节。
- 返回值，返回 HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例:

使用 DMA 方式，此贴有完整例子：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=86271 。

65.5 总结

本章节就为大家讲解这么多，涉及到的知识点和 API 函数比较多，需要花点时间消化，后面用到的多了，就可以熟练掌握了。

第66章 STM32H7 的低功耗串口 LPUART 应用之串口 FIFO 和停机唤醒实现

本章节为大家讲解 STM32H7 的低功耗串口 FIFO 驱动实现和停机唤醒。

66.1 初学者重要提示
66.2 硬件设计
66.3 串口驱动设计
66.4 串口 FIFO 板级支持包（bsp_lpuart_fifo.c）
66.5 串口 FIFO 驱动移植和使用
66.6 实验例程设计框架
66.7 实验例程说明（MDK）
66.8 实验例程说明（IAR）
66.9 总结

66.1 初学者重要提示

- 学习本章节前，务必优先学习第 65 章。
- 低功耗串口 FIFO 的实现跟前面章节通用串口 FIFO 的机制是一样的。
- 大家自己做的板子，测试串口收发是乱码的话，重点看 stm32h7xx_hal_conf.h 文件中的 HSE_VALUE 的大小跟板子上实际晶振大小是否一致，然后再看 PLL 配置。

66.2 硬件设计

STM32H743XIH6 最多可以支持 8 个独立的通用串口和一个低功耗串口 LPUART1。其中串口 4 和串口 5 和 SDIO 的 GPIO 是共用的，也就是说，如果要用到 SD 卡，那么串口 4 和串口 5 将不能使用。串口 7 和 SPI3 共用，串口 8 和 RGB 硬件接口共用。串口功能可以分配到不同的 GPIO。我们常用的引脚分配如下：

低功耗串口 LPUART TX = PA9，　RX = PA10
串口 USART1　TX = PA9，　RX = PA10　（低功耗串口和 USART1 用的相同引脚）
串口 USART2　TX = PA2，　RX = PA3
串口 USART3　TX = PB10，　RX = PB11

串口 UART4　TX = PC10，　RX = PC11　(和 SDIO 共用)

串口 UART5　TX = PC12，　RX = PD2　(和 SDIO 共用)

串口 USART6　TX = PG14，　RX = PC7

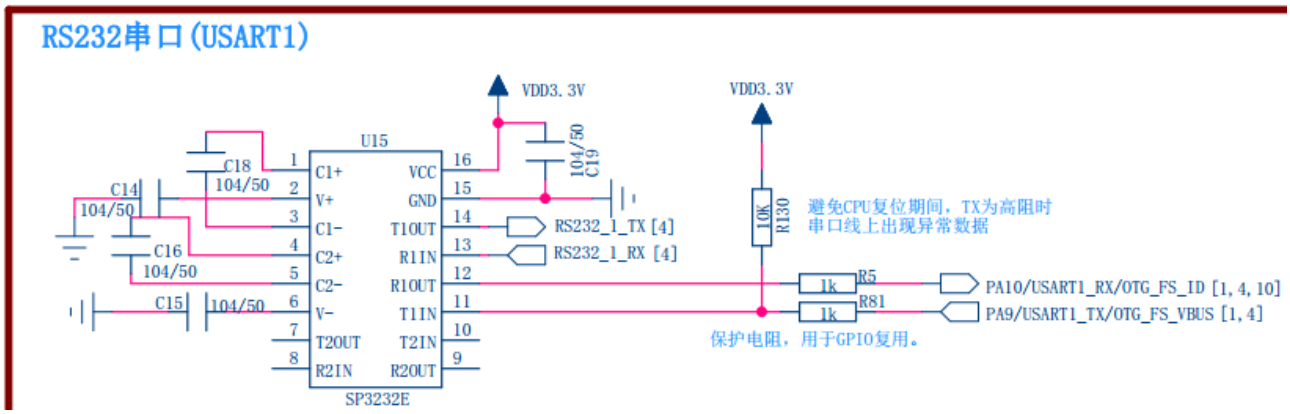
串口 UART7　TX = PB4，　RX = PB3　(和 SPI1/3 共用)

串口 UART8　TX = PJ8，　RX =PJ9　(和 RGB 硬件接口共用)

STM32-V7 开发板使用了 4 个串口设备。

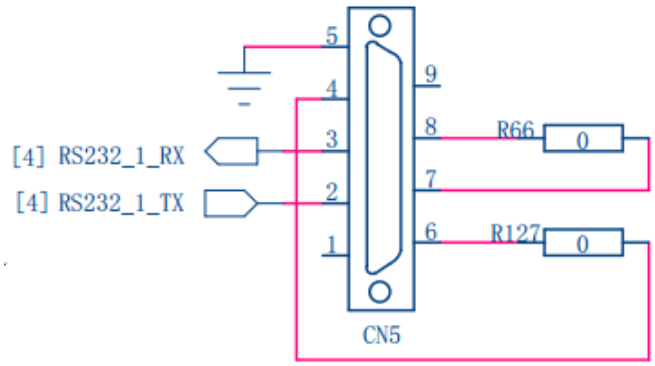
- 串口 1 用于 RS232 接口，很多例子的 pritnf 结果就是输出到串口 1
- 串口 2 用于 GPS
- 串口 3 用于 RS485 接口
- 串口 6 用于 TTL 串口插座，板子上有 GPRS 插座和串口 WIFI 插座。

下面是 RS232 的原理图：



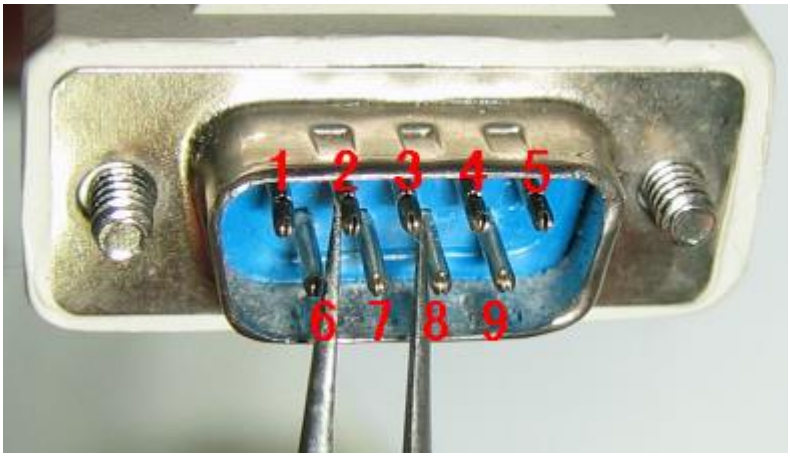
关于 232 的 PHY 芯片 SP3232E 要注意以下几个问题：

- SP3232E 的作用是 TTL 电平转 RS232 电平。
- 电阻 R130 的作用是避免 CPU 复位期间，TX 为高阻时串口线上出现异常数据。
- 检测 SP3232E 的好坏可以采用回环的方式，即短接 T1OUT 和 R1IN，对应到 DB9 插座上就是短接引脚 2 和引脚 3。



DB9 RS232插座 9孔

实际效果如下：



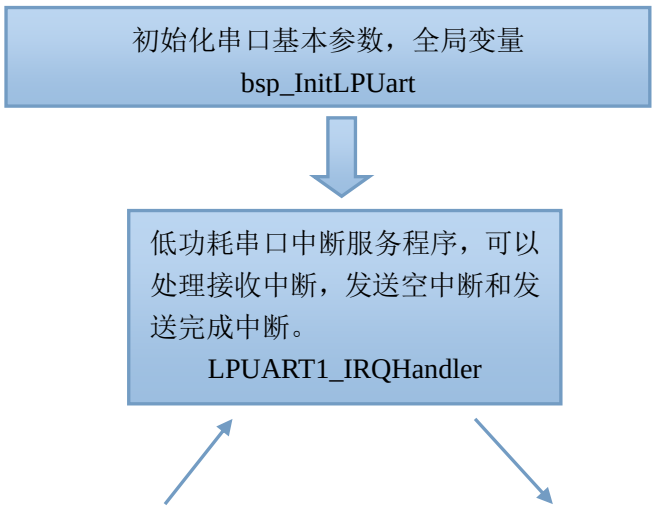
通过这种方式，可以在应用程序中通过串口发送几个字符，查看是否可以正确接收来判断 232 PHY 芯片是否有问题。

- 由于这里是 TTL 转 RS232，如果电脑端自带 DB9 串口，可以找根交叉线直接接上。如果电脑端没有，就需要用 RS232 转 USB 的串口线。这里要注意是 RS232 转 USB，不是 TTL 转 USB。像我们用的 CH340 就是 RS232 转 USB 芯片。
- 检测串口线的好坏跟板子上的 232 PHY 一样，将电脑端的串口助手打开，串口线接到电脑端并短接串口线的 2 脚和 3 脚，然后使用串口助手进行自收发测试即可。

66.3 低功耗串口 FIFO 驱动设计

66.3.1 低功耗串口 FIFO 框架

为了方便大家理解，先来看下低功耗串口 FIFO 的实现框图：



低功耗串口数据发送 lpcomSendBuf	低功耗串口数据接收 lpcomGetChar

- 第 1 阶段，初始化：

  通过函数 bsp_InitLPUart 初始化低功耗串口结构体，低功耗串口硬件参数。

- 第 2 阶段，低功耗串口中断服务程序：
  - 接收中断是一直开启的。
  - 做了发送空中断和发送完成中断的消息处理。
- 第 3 阶段，低功耗串口数据的收发：
  - 低功耗串口发送函数会开启发送空中断。
  - 低功耗串口接收中断接收到函数后，可以使用函数 lpcomGetChar 获取数据。

66.3.2 低功耗串口时钟选择

我们这里实现了三种时钟选择：

- LPUART 时钟选择 LSE(32768Hz)

  最高速度是 10922bps，最低 8bps（计算方法 3x < 32768 < 4096x，x 表示波特率）。

- LPUART 时钟选择 HSI(64MHz)

  最高值是 21MHz，最小值 15625bps（计算方法 3x < 64MHz < 4096x，x 表示波特率）。

- LPUART 时钟选择 D3PCLK1(100MHz)

  最大值 33Mbps，最小值 24414bps（计算方法 3x < 100MHz < 4096x，x 表示波特率）。

如果需要低功耗模式唤醒，必须使用 LSE 或者 HSI 时钟，程序代码如下，用户可以根据需要，使能相应的宏定义：

```
//#define LPUART_CLOCK_SOURCE_LSE
#define LPUART_CLOCK_SOURCE_HSI
//#define LPUART_CLOCK_SOURCE_D3PCLK1

/*
*********************************************************************************************************
*    函 数 名: InitHardLPUart
*    功能说明: 配置串口的硬件参数（波特率，数据位，停止位，起始位，校验位，中断使能）适合于 STM32-H7 开发板
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void InitHardLPUart(void)
{
    GPIO_InitTypeDef  GPIO_InitStruct;
    RCC_PeriphCLKInitTypeDef   RCC_PeriphCLKInitStruct = {0};

/* 使用 LSE(32768Hz)，最高速度是 10922bps，最低 8bps */
```

```
#if defined (LPUART_CLOCK_SOURCE_LSE)
    {
        RCC_OscInitTypeDef RCC_OscInitStruct = {0};

        RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_LSE;
        RCC_OscInitStruct.LSEState = RCC_LSE_ON;
        RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;

        if (HAL_RCC_OscConfig(&RCC_OscInitStruct)!= HAL_OK)
        {
            Error_Handler(__FILE__, __LINE__);
        }

        RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPUART1;
        RCC_PeriphCLKInitStruct.Lpuart1ClockSelection = RCC_LPUART1CLKSOURCE_LSE;
        HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
    }
/* LPUART 时钟选择 HSI(64MHz)，最高值是 21MHz，最小值 15625bps */
#elif defined (LPUART_CLOCK_SOURCE_HSI)
    {

        RCC_OscInitTypeDef RCC_OscInitStruct = {0};

          RCC_OscInitStruct.OscillatorType      = RCC_OSCILLATORTYPE_HSI;
          RCC_OscInitStruct.HSIState            = RCC_HSI_ON;
          RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
          RCC_OscInitStruct.PLL.PLLState        = RCC_PLL_NONE;

        if (HAL_RCC_OscConfig(&RCC_OscInitStruct)!= HAL_OK)
        {
            Error_Handler(__FILE__, __LINE__);
        }

        RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPUART1;
        RCC_PeriphCLKInitStruct.Lpuart1ClockSelection = RCC_LPUART1CLKSOURCE_HSI;
        HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
    }
/* LPUART 时钟选择 D3PCLK1（100MHz），最大值 33Mbps，最小值 24414bps */
#elif defined (LPUART_CLOCK_SOURCE_D3PCLK1)

    RCC_PeriphCLKInitStruct.PeriphClockSelection = RCC_PERIPHCLK_LPUART1;
    RCC_PeriphCLKInitStruct.Lptim1ClockSelection = RCC_LPUART1CLKSOURCE_D3PCLK1;
    HAL_RCCEx_PeriphCLKConfig(&RCC_PeriphCLKInitStruct);
#else
    #error Please select the LPTIM Clock source inside the bsp_lpuart_fifo.c file
#endif

#if LPUART1_FIFO_EN ==
    /* 其它省略未写 */
#endif
}
```

66.3.3 低功耗串口 FIFO 之相关的变量定义

低功耗串口驱动的核心文件为：bsp_lpuart_fifo.c, bsp_lpuart_fifo.h。

这里面包括有串口硬件的配置函数、中断处理函数，以及串口的读写接口函数。还有 printf 函数的实现。

每个串口都有 2 个 FIFO 缓冲区，一个是用于发送数据的 TX_FIFO，一个用于保存接收数据的 RX_FIFO。

我们来看下这个 FIFO 的定义，在 bsp_lpuart_fifo.h 文件。

```
/* 定义串口波特率和 FIFO 缓冲区大小，分为发送缓冲区和接收缓冲区，支持全双工 */
#if LPUART1_FIFO_EN == 1
	#define LPUART1_BAUD			115200
	#define LPUART1_TX_BUF_SIZE	1*1024
	#define LPUART1_RX_BUF_SIZE	1*1024
#endif

/* 串口设备结构体 */
typedef struct
{
	USART_TypeDef *uart;		/* STM32 内部串口设备指针 */
	uint8_t *pTxBuf;			/* 发送缓冲区 */
	uint8_t *pRxBuf;			/* 接收缓冲区 */
	uint16_t usTxBufSize;		/* 发送缓冲区大小 */
	uint16_t usRxBufSize;		/* 接收缓冲区大小 */
	__IO uint16_t usTxWrite;	/* 发送缓冲区写指针 */
	__IO uint16_t usTxRead;		/* 发送缓冲区读指针 */
	__IO uint16_t usTxCount;	/* 等待发送的数据个数 */

	__IO uint16_t usRxWrite;	/* 接收缓冲区写指针 */
	__IO uint16_t usRxRead;		/* 接收缓冲区读指针 */
	__IO uint16_t usRxCount;	/* 还未读取的新数据个数 */

	void (*SendBefor)(void);	/* 开始发送之前的回调函数指针（主要用于 RS485 切换到发送模式） */
	void (*SendOver)(void);		/* 发送完毕的回调函数指针（主要用于 RS485 将发送模式切换为接收模式） */
	void (*ReciveNew)(uint8_t _byte);	/* 串口收到数据的回调函数指针 */
	uint8_t Sending;			/* 正在发送中 */
}UART_T;
```

bsp_lpuart_fifo.c 文件定义变量。

```
/* 定义低功耗串口结构体变量 */
#if LPUART1_FIFO_EN == 1
	static LPUART_T g_tLPUart1;
	static uint8_t g_TxBuf1[LPUART1_TX_BUF_SIZE];		/* 发送缓冲区 */
	static uint8_t g_RxBuf1[LPUART1_RX_BUF_SIZE];		/* 接收缓冲区 */
#endif
```

关于 FIFO 的机制，我们在按键 FIFO 驱动已经做过详细的介绍，这个地方就不赘述了。每个串口有两个 FIFO 缓冲区，每个 FIFO 对应一个写指针和一个读指针。这个结构中还有三个回调函数。回调函数就是一个通过函数指针调用的函数。如果你把函数的指针（地址）作为参数传递给另一个函数，当这个指针被用为调用它所指向的函数时，我们就说这是回调函数。

66.3.4 低功耗串口 FIFO 初始化

低功耗串口的初始化代码如下；

```
/*
*********************************************************************************************************
```

```
*    函 数 名: bsp_InitLPUart
*    功能说明: 初始化串口硬件，并对全局变量赋初值.
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitLPUart(void)
{
    LPUartVarInit();        /* 必须先初始化全局变量,再配置硬件 */
    InitHardLPUart();       /* 配置串口的硬件参数(波特率等) */
}
```

下面将初始化代码实现的功能依次为大家做个说明。

- 函数 LPUartVarInit

这个函数实现的功能比较好理解，主要是串口设备结构体变量的初始化，代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: LPUartVarInit
*    功能说明: 初始化串口相关的变量
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void LPUartVarInit(void)
{
#if LPUART1_FIFO_EN == 1
    g_tLPUart1.uart = LPUART1;                      /* STM32 串口设备 */
    g_tLPUart1.pTxBuf = g_TxBuf1;                   /* 发送缓冲区指针 */
    g_tLPUart1.pRxBuf = g_RxBuf1;                   /* 接收缓冲区指针 */
    g_tLPUart1.usTxBufSize = LPUART1_TX_BUF_SIZE;   /* 发送缓冲区大小 */
    g_tLPUart1.usRxBufSize = LPUART1_RX_BUF_SIZE;   /* 接收缓冲区大小 */
    g_tLPUart1.usTxWrite = 0;                       /* 发送 FIFO 写索引 */
    g_tLPUart1.usTxRead = 0;                        /* 发送 FIFO 读索引 */
    g_tLPUart1.usRxWrite = 0;                       /* 接收 FIFO 写索引 */
    g_tLPUart1.usRxRead = 0;                        /* 接收 FIFO 读索引 */
    g_tLPUart1.usRxCount = 0;                       /* 接收到的新数据个数 */
    g_tLPUart1.usTxCount = 0;                       /* 待发送的数据个数 */
    g_tLPUart1.SendBefor = 0;                       /* 发送数据前的回调函数 */
    g_tLPUart1.SendOver = 0;                        /* 发送完毕后的回调函数 */
    g_tLPUart1.ReciveNew = 0;                       /* 接收到新数据后的回调函数 */
    g_tLPUart1.Sending = 0;                         /* 正在发送中标志 */
#endif
}
```

- 函数 InitHardLPUart

此函数主要用于串口的 GPIO，中断和相关参数的配置。

```
/* LPUART1 的 GPIO  PA9, PA10 */
#define LPUART1_CLK_ENABLE()              __HAL_RCC_LPUART1_CLK_ENABLE()

#define LPUART1_TX_GPIO_CLK_ENABLE()      __HAL_RCC_GPIOA_CLK_ENABLE()
#define LPUART1_TX_GPIO_PORT              GPIOA
#define LPUART1_TX_PIN                    GPIO_PIN_9
#define LPUART1_TX_AF                     GPIO_AF3_LPUART

#define LPUART1_RX_GPIO_CLK_ENABLE()      __HAL_RCC_GPIOA_CLK_ENABLE()
#define LPUART1_RX_GPIO_PORT              GPIOA
```

```
#define LPUART1_RX_PIN                  GPIO_PIN_10
#define LPUART1_RX_AF                   GPIO_AF3_LPUART

/*
*********************************************************************************************************
*    函 数 名: InitHardUart
*    功能说明: 配置串口的硬件参数和底层
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void InitHardUart(void)
{
    GPIO_InitTypeDef  GPIO_InitStruct;
    RCC_PeriphCLKInitTypeDef RCC_PeriphClkInit;

    /* 时钟初始化省略未写 */
#if LPUART1_FIFO_EN == 1
    /* 使能 GPIO TX/RX 时钟 */
    LPUART1_TX_GPIO_CLK_ENABLE();
    LPUART1_RX_GPIO_CLK_ENABLE();

    /* 使能 USARTx 时钟 */
    LPUART1_CLK_ENABLE();

    /* 配置 TX 引脚 */
    GPIO_InitStruct.Pin       = LPUART1_TX_PIN;
    GPIO_InitStruct.Mode      = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Pull      = GPIO_PULLUP;
    GPIO_InitStruct.Speed     = GPIO_SPEED_FREQ_HIGH;
    GPIO_InitStruct.Alternate = LPUART1_TX_AF;
    HAL_GPIO_Init(LPUART1_TX_GPIO_PORT, &GPIO_InitStruct);

    /* 配置 RX 引脚 */
    GPIO_InitStruct.Pin = LPUART1_RX_PIN;
    GPIO_InitStruct.Alternate = LPUART1_RX_AF;
    HAL_GPIO_Init(LPUART1_RX_GPIO_PORT, &GPIO_InitStruct);

    /* 配置 NVIC the NVIC for UART */
    HAL_NVIC_SetPriority(LPUART1_IRQn, 0, 1);
    HAL_NVIC_EnableIRQ(LPUART1_IRQn);

    /* 配置波特率、奇偶校验 */
    bsp_SetLPUartParam(LPUART1,  LPUART1_BAUD, UART_PARITY_NONE, UART_MODE_TX_RX);

    SET_BIT(LPUART1->ICR, USART_ICR_TCCF);    /* 清除 TC 发送完成标志 */
    SET_BIT(LPUART1->RQR, USART_RQR_RXFRQ);   /* 清除 RXNE 接收标志 */
    SET_BIT(LPUART1->CR1, USART_CR1_RXNEIE); /* 使能 PE. RX 接受中断 */
#endif
}
```

低功耗定时器的参数配置 API 如下：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_SetLPUartParam
*    功能说明: 配置串口的硬件参数（波特率，数据位，停止位，起始位，校验位，中断使能）适合于 STM32- H7 开发板
*    形    参: Instance   USART_TypeDef 类型结构体
*              BaudRate   波特率
```

```
*             Parity     校验类型，奇校验或者偶校验
*             Mode       发送和接收模式使能
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_SetLPUartParam(USART_TypeDef *Instance,  uint32_t BaudRate, uint32_t Parity, uint32_t Mode)
{
    /*##-1- 配置串口硬件参数 ######################################*/
    /* 异步串口模式 (UART Mode) */
    /* 配置如下:
      - 字长     = 8 位
      - 停止位   = 1 个停止位
      - 校验     = 参数 Parity
      - 波特率   = 参数 BaudRate
      - 硬件流控制关闭 (RTS and CTS signals) */

    UartHandle.Instance        = Instance;
    UartHandle.Init.BaudRate   = BaudRate;
    UartHandle.Init.WordLength = UART_WORDLENGTH_8B;
    UartHandle.Init.StopBits   = UART_STOPBITS_1;
    UartHandle.Init.Parity     = Parity;
    UartHandle.Init.HwFlowCtl  = UART_HWCONTROL_NONE;
    UartHandle.Init.Mode       = Mode;
    UartHandle.Init.OverSampling = UART_OVERSAMPLING_16;
    UartHandle.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
    UartHandle.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;

    if (HAL_UART_Init(&UartHandle) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

66.3.5 低功耗串口中断服务程序工作流程

串口中断服务程序是最核心的部分，主要实现如下三个功能

- 收到新的数据后，会将数据压入 RX_FIFO。
- 检测到发送缓冲区空后，会从 TX_FIFO 中取下一个数据并发送。
- 如果是 RS485 半双工串口，发送前会设置一个 GPIO=1 控制 RS485 收发器进入发送状态，当最后一个字节的最后一个 bit 传送完毕后，设置这个 GPIO=0 让 RS485 收发器进入接收状态。

下面我们分析一下串口中断处理的完整过程。

当产生串口中断后，CPU 会查找中断向量表，获得中断服务程序的入口地址。入口函数为 LPUART1_IRQHandler，这个函数在启动文件 startup_stm32h743xx.s 汇编代码中已经有实现。我们在 c 代码中需要重写一个同样名字的函数就可以重载它。如果不重载，启动文件中缺省的中断服务程序就是一个死循环，等于 while(1);

我们将串口中断服务程序放在 bsp_lpuart_fifo.c 文件，没有放到 stm32h7xx_it.c。当应用不需要串

口功能时，直接从工程中删除 bsp_lpuart_fifo.c 接口，不必再去整理 stm32h7xx_it.c 这个文件。下面展示的代码是低功耗串口的中断服务程序：

```
#if LPUART1_FIFO_EN == 1
void LPUART1_IRQHandler(void)
{
    LPUartIRQ(&g_tLPUart1);
}
#endif
```

下面，我们来看看 UartIRQ 函数的实现代码。

```
/*
*********************************************************************************************************
*    函 数 名: UartIRQ
*    功能说明: 供中断服务程序调用，通用串口中断处理函数
*    形    参: _pUart : 串口设备
*    返 回 值: 无
*********************************************************************************************************
*/
static void UartIRQ(UART_T *_pUart)
{
    uint32_t isrflags   = READ_REG(_pUart->uart->ISR);
    uint32_t crlits     = READ_REG(_pUart->uart->CR1);
    uint32_t cr3its     = READ_REG(_pUart->uart->CR3);

    /* 处理接收中断  */
    if ((isrflags & USART_ISR_RXNE) != RESET)
    {
        /* 从串口接收数据寄存器读取数据存放到接收 FIFO */
        uint8_t ch;

        ch = READ_REG(_pUart->uart->RDR);               /* 读串口接收数据寄存器 */
        _pUart->pRxBuf[_pUart->usRxWrite] = ch;         /* 填入串口接收 FIFO */
        if (++_pUart->usRxWrite >= _pUart->usRxBufSize) /* 接收 FIFO 的写指针+1 */
        {
            _pUart->usRxWrite = 0;
        }
        if (_pUart->usRxCount < _pUart->usRxBufSize)    /* 统计未处理的字节个数 */
        {
            _pUart->usRxCount++;
        }

        /* 回调函数,通知应用程序收到新数据,一般是发送 1 个消息或者设置一个标记 */
        //if (_pUart->usRxWrite == _pUart->usRxRead)
        //if (_pUart->usRxCount == 1)
        {
            if (_pUart->ReciveNew)
            {
                _pUart->ReciveNew(ch); /* 比如，交给 MODBUS 解码程序处理字节流 */
            }
        }
    }

    /* 处理发送缓冲区空中断 */
    if ( ((isrflags & USART_ISR_TXE) != RESET) && (crlits & USART_CR1_TXEIE) != RESET)
    {
        //if (_pUart->usTxRead == _pUart->usTxWrite)
        if (_pUart->usTxCount == 0)  /* 发送缓冲区已无数据可取 */
```

```
        {
        /* 发送缓冲区的数据已取完时， 禁止发送缓冲区空中断 （注意：此时最后 1 个数据还未真正发送完毕）*/
            //USART_ITConfig(_pUart->uart, USART_IT_TXE, DISABLE);
            CLEAR_BIT(_pUart->uart->CR1, USART_CR1_TXEIE);

            /* 使能数据发送完毕中断 */
            //USART_ITConfig(_pUart->uart, USART_IT_TC, ENABLE);
            SET_BIT(_pUart->uart->CR1, USART_CR1_TCIE);
        }
        Else  /*  还有数据等待发送 */
        {
            _pUart->Sending = 1;

            /* 从发送 FIFO 取 1 个字节写入串口发送数据寄存器 */
            //USART_SendData(_pUart->uart, _pUart->pTxBuf[_pUart->usTxRead]);
            _pUart->uart->TDR = _pUart->pTxBuf[_pUart->usTxRead];
            if (++_pUart->usTxRead >= _pUart->usTxBufSize)
            {
                _pUart->usTxRead = 0;
            }
            _pUart->usTxCount--;
        }

    }
    /* 数据 bit 位全部发送完毕的中断 */
    if (((isrflags & USART_ISR_TC) != RESET) && ((cr1its & USART_CR1_TCIE) != RESET))
    {
        //if (_pUart->usTxRead == _pUart->usTxWrite)
        if (_pUart->usTxCount == 0)
        {
            /* 如果发送 FIFO 的数据全部发送完毕，禁止数据发送完毕中断 */
            //USART_ITConfig(_pUart->uart, USART_IT_TC, DISABLE);
            CLEAR_BIT(_pUart->uart->CR1, USART_CR1_TCIE);

            /* 回调函数, 一般用来处理 RS485 通信，将 RS485 芯片设置为接收模式，避免抢占总线 */
            if (_pUart->SendOver)
            {
                _pUart->SendOver();
            }

            _pUart->Sending = 0;
        }
        else
        {
            /* 正常情况下，不会进入此分支 */

            /* 如果发送 FIFO 的数据还未完毕，则从发送 FIFO 取 1 个数据写入发送数据寄存器 */
            //USART_SendData(_pUart->uart, _pUart->pTxBuf[_pUart->usTxRead]);
            _pUart->uart->TDR = _pUart->pTxBuf[_pUart->usTxRead];
            if (++_pUart->usTxRead >= _pUart->usTxBufSize)
            {
                _pUart->usTxRead = 0;
            }
            _pUart->usTxCount--;
        }
    }

    /* 清除中断标志 */
```

```
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_PEF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_FEF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_NEF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_OREF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_IDLEF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_TCF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_LBDF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_CTSF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_CMF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_WUF);
    SET_BIT(_pUart->uart->ICR, UART_CLEAR_TXFECF);
}
```

中断服务程序的处理主要分为两部分，接收数据的处理和发送数据的处理，详情看程序注释即可，已经比较详细，下面重点把思路说一下。

- 接收数据处理

接收数据的处理是判断 ISR 寄存器的 USART_ISR_RXNE 标志是否置位，如果置位表示 RDR 接收寄存器已经存入数据。然后将数据读入到接收 FIFO 空间。

特别注意里面的 ReciveNew 处理，这个在 Modbus 协议里面要用到。

- 发送数据处理

发送数据主要是发送空中断 TEX 和发送完成中断 TC 的处理，当 TXE=1 时，只是表示发送数据寄存器为空了，此时可以填充下一个准备发送的数据了。当为 TDR 发送寄存器赋值后，硬件启动发送，等所有的 bit 传送完毕后，TC 标志设置为 1。如果是 RS232 全双工通信，可以只用 TXE 标志控制发送过程。如果是 RS485 半双工通信，就需要利用 TC 标志了，因为在最后一个 bit 传送完毕后，需要设置 RS485 收发器进入到接收状态。

66.3.6低功耗串口数据发送

低功耗串口数据的发送主要涉及到下面三个函数：

```
/*
*********************************************************************************************************
*    函 数 名: lpcomSendBuf
*    功能说明: 向串口发送一组数据。数据放到发送缓冲区后立即返回，由中断服务程序在后台完成发送
*    形    参: _ucPort: 端口号(LPCOM1)
*              _ucaBuf: 待发送的数据缓冲区
*              _usLen : 数据长度
*    返 回 值: 无
*********************************************************************************************************
*/
void lpcomSendBuf(LPCOM_PORT_E _ucPort, uint8_t *_ucaBuf, uint16_t _usLen)
{
    LPUART_T *pUart;

    pUart = ComToLPUart(_ucPort);
    if (pUart == 0)
    {
        return;
    }
```

```
    if (pUart->SendBefor != 0)
    {
        pUart->SendBefor();         /* 如果是 RS485 通信，可以在这个函数中将 RS485 设置为发送模式 */
    }

    LPUartSend(pUart, _ucaBuf, _usLen);
}

/*
*********************************************************************************************************
*    函 数 名: lpcomSendChar
*    功能说明: 向串口发送 1 个字节。数据放到发送缓冲区后立即返回，由中断服务程序在后台完成发送
*    形    参: _ucPort: 端口号(LPCOM1)
*              _ucByte: 待发送的数据
*    返 回 值: 无
*********************************************************************************************************
*/
void lpcomSendChar(LPCOM_PORT_E _ucPort, uint8_t _ucByte)
{
    lpcomSendBuf(_ucPort, &_ucByte, 1);
}

/*
*********************************************************************************************************
*    函 数 名: LPUartSend
*    功能说明: 填写数据到 UART 发送缓冲区,并启动发送中断。中断处理函数发送完毕后，自动关闭发送中断
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void LPUartSend(LPUART_T *_pUart, uint8_t *_ucaBuf, uint16_t _usLen)
{
    uint16_t i;

    for (i = 0; i < _usLen; i++)
    {
        /* 如果发送缓冲区已经满了，则等待缓冲区空 */
        while (1)
        {
            __IO uint16_t usCount;

            DISABLE_INT();
            usCount = _pUart->usTxCount;
            ENABLE_INT();

            if (usCount < _pUart->usTxBufSize)
            {
                break;
            }
            else if(usCount == _pUart->usTxBufSize)/* 数据已填满缓冲区 */
            {
                if((_pUart->uart->CR1 & USART_CR1_TXEIE) == 0)
                {
                    SET_BIT(_pUart->uart->CR1, USART_CR1_TXEIE);
                }
            }
        }
```

```
        /* 将新数据填入发送缓冲区 */
        _pUart->pTxBuf[_pUart->usTxWrite] = _ucaBuf[i];

        DISABLE_INT();
        if (++_pUart->usTxWrite >= _pUart->usTxBufSize)
        {
            _pUart->usTxWrite = 0;
        }
        _pUart->usTxCount++;
        ENABLE_INT();
    }

    SET_BIT(_pUart->uart->CR1, USART_CR1_TXEIE);   /* 使能发送中断（缓冲区空） */
}
```

函数 lpcomSendChar 是发送一个字节，通过调用函数 lpcomSendBuf 实现，而函数 lpcomSendBuf 又是通过调用函数 LPUartSend 实现，这个函数是重点。

函数 LPUartSend 的作用就是把要发送的数据填到发送缓冲区里面，并使能发送空中断。

- 如果要发送的数据没有超过发送缓冲区大小，实现起来还比较容易，直接把数据填到 FIFO 里面，并使能发送空中断即可。
- 如果超过了 FIFO 大小，就需要等待有空间可用，针对这种情况有个重要的知识点，就是当缓冲刚刚填满的时候要判断发送空中断是否开启了，如果填满了还没有开启，就会卡死在 while 循环中，所以多了一个刚填满时的判断，填满了还没有开启发送空中断，要开启下。

注意：由于函数 LPUartSend 做了 static 作用域限制，仅可在 bsp_lpuart_fifo.c 文件中调用。函数 lpcomSendChar 和 lpcomSendBuf 是供用户调用的。

函数 lpcomSendBuf 中调用了一个函数 pUart = ComToLPUart(_ucPort)，这个函数是将整数的 COM 端口号转换为 LPUART 结构体指针。

```
/*
*********************************************************************************************************
*    函 数 名: ComToLPUart
*    功能说明: 将 COM 端口号转换为 LPUART 指针
*    形    参: _ucPort: 端口号(LPCOM1)
*    返 回 值: uart 指针
*********************************************************************************************************
*/
LPUART_T *ComToLPUart(LPCOM_PORT_E _ucPort)
{
    if (_ucPort == LPCOM1)
    {
        #if LPUART1_FIFO_EN == 1
            return &g_tLPUart1;
        #else
            return 0;
        #endif
    }
    else
```

```
    {
        Error_Handler(__FILE__, __LINE__);
        return 0;
    }
}
```

66.3.7 低功耗串口数据接收

下面我们再来看看接收的函数：

```
/*
*********************************************************************************************************
*    函 数 名: lpcomGetChar
*    功能说明: 从接收缓冲区读取 1 字节，非阻塞。无论有无数据均立即返回。
*    形    参: _ucPort: 端口号(LPCOM1)
*              _pByte: 接收到的数据存放在这个地址
*    返 回 值: 0 表示无数据，1 表示读取到有效字节
*********************************************************************************************************
*/
uint8_t lpcomGetChar(LPCOM_PORT_E _ucPort, uint8_t *_pByte)
{
    LPUART_T *pUart;

    pUart = ComToLPUart(_ucPort);
    if (pUart == 0)
    {
        return 0;
    }

    return LPUartGetChar(pUart, _pByte);
}

/*
*********************************************************************************************************
*    函 数 名: LPUartGetChar
*    功能说明: 从串口接收缓冲区读取 1 字节数据 （用于主程序调用）
*    形    参: _pUart : 串口设备
*              _pByte : 存放读取数据的指针
*    返 回 值: 0 表示无数据  1 表示读取到数据
*********************************************************************************************************
*/
static uint8_t LPUartGetChar(LPUART_T *_pUart, uint8_t *_pByte)
{
    uint16_t usCount;

    /* usRxWrite 变量在中断函数中被改写，主程序读取该变量时，必须进行临界区保护 */
    DISABLE_INT();
    usCount = _pUart->usRxCount;
    ENABLE_INT();

    /* 如果读和写索引相同，则返回 0 */
    //if (_pUart->usRxRead == usRxWrite)
    if (usCount == 0)  /* 已经没有数据 */
    {
        return 0;
    }
```

```
    else
    {
        *_pByte = _pUart->pRxBuf[_pUart->usRxRead];         /* 从串口接收 FIFO 取 1 个数据 */

        /* 改写 FIFO 读索引 */
        DISABLE_INT();
        if (++_pUart->usRxRead >= _pUart->usRxBufSize)
        {
            _pUart->usRxRead = 0;
        }
        _pUart->usRxCount--;
        ENABLE_INT();
        return 1;
    }
}
```

函数 lpcomGetChar 是专门供用户调用的，用于从接收 FIFO 中读取 1 个数据。具体代码的实现也比较好理解，主要是接收 FIFO 的空间调整。

注意：由于函数 LPUartGetChar 做了 static 作用域限制，仅可在 bsp_lpuart_fifo.c 文件中调用。

66.3.8低功耗串口 printf 实现

printf 函数是标准 c 库函数。最原来的意思是打印输出到显示器。在单片机，我们常用它来打印调试信息到串口，通过计算机上运行的串口软件来监视程序的运行状态。

为什么要用 printf 函数，而不用串口发送的函数。因为 printf 函数的形参功能很强大，它支持各种数值转换。比如将整数、浮点数转换为字符串，支持整数左对齐、右对齐显示等。

我们设计的很多裸机例子都是用 printf 函数输出运行结果的。因为如果加上显示屏驱动后，会将程序搞的很复杂，显示部分的代码量超过了例程本身要演示的核心功能代码。用串口做输出，移植很方便，现在很少有不带串口的单片机。

实现 printf 输出到串口，只需要在工程中添加两个函数：

```
/*
*********************************************************************************************************
*    函 数 名: fputc
*    功能说明: 重定义 putc 函数，这样可以使用 printf 函数从串口 1 打印输出
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
int fputc(int ch, FILE *f)
{
#if 0    /* 将需要 printf 的字符通过串口中断 FIFO 发送出去，printf 函数会立即返回 */
    lpcomSendChar(LPCOM1, ch);

    return ch;
#else    /* 采用阻塞方式发送每个字符,等待数据发送完毕 */
    /* 写一个字节到 USART1 */
    LPUART1->TDR = ch;

    /* 等待发送结束 */
```

```
    while((LPUART1->ISR & USART_ISR_TC) == 0)
    {}

    return ch;
#endif
}

/*
*********************************************************************************************************
*    函 数 名: fgetc
*    功能说明: 重定义 getc 函数，这样可以使用 getchar 函数从串口 1 输入数据
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
int fgetc(FILE *f)
{

#if 1    /* 从串口接收 FIFO 中取 1 个数据，只有取到数据才返回 */
    uint8_t ucData;

    while(lpcomGetChar(LPCOM1, &ucData) == 0);

    return ucData;
#else
    /* 等待接收到数据 */
    while((LPUART1->ISR & USART_ISR_RXNE) == 0)
    {}

    return (int)LPUART1->RDR;
#endif
}
```

通过上面代码中的条件编译，可以设置 printf 函数阻塞和非阻塞方式，如果采用非阻塞方式，执行后会立即返回，串口中断服务程序会陆续将数据发送出去。

66.3.9 低功耗串口停机唤醒方式

低功耗串口的唤醒主要是通过接收数据来唤醒，具体唤醒的方如下：

- 检测到起始位唤醒。

低功耗串口设置为起始位检测方式如下，并且设置进入停机模式。

如果想唤醒 H7，发一个起始位即可，简单些也可以任意发送一个数据：

```
/* 使能 LPUART 的停机唤醒 */
HAL_UARTEx_EnableStopMode(&UartHandle);

/* 确保 LPUART 没有在通信中 */
while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_BUSY) == SET){}
while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_REACK) == RESET){}

/* 接收起始位唤醒 */
WakeUpSelection.WakeUpEvent = UART_WAKEUP_ON_STARTBIT;
```

```
if (HAL_UARTEx_StopModeWakeUpSourceConfig(&UartHandle, WakeUpSelection)!= HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}

/* 进入停机模式 */
HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);

/* 退出停机模式要重新配置 HSE 和 PLL*/
SystemClock_Config();

/* 关闭 LPUART 的停机唤醒 */
HAL_UARTEx_DisableStopMode(&UartHandle);
```

◆ 检测到 RXNE 标志唤醒，即接收到数据。

低功耗串口设置为 RXNE 检测方式如下，并且设置进入停机模式。

如果想唤醒 H7，发一个任意数据即可。

```
/* 使能 LPUART 的停机唤醒 */
HAL_UARTEx_EnableStopMode(&UartHandle);

/* 确保 LPUART 没有在通信中 */
while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_BUSY) == SET){}
while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_REACK) == RESET){}

/* 接收到数据唤醒，即 RXNE 标志置位 */
WakeUpSelection.WakeUpEvent = UART_WAKEUP_ON_READDATA_NONEMPTY;
if (HAL_UARTEx_StopModeWakeUpSourceConfig(&UartHandle, WakeUpSelection)!= HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}

/* 进入停机模式 */
HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);

/* 退出停机模式要重新配置 HSE 和 PLL*/
SystemClock_Config();

/* 关闭 LPUART 的停机唤醒 */
HAL_UARTEx_DisableStopMode(&UartHandle);
```

◆ 检测到匹配地址时唤醒。

低功耗串口设置为地址匹配检测方式如下，并且设置进入停机模式。

如果想唤醒 H7，必须发送指定的匹配地址。匹配地址支持 7bit 和 4bit 匹配两种方式，比如我们采用 7bit 匹配，设置地址是 0x19，那么用户唤醒的时候要将最高 bit 设置为 1，即发生地址 0x99（0b1001 1001）才可以唤醒。

```
/* 使能 LPUART 的停机唤醒 */
HAL_UARTEx_EnableStopMode(&UartHandle);

/* 确保 LPUART 没有在通信中 */
while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_BUSY) == SET){}
while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_REACK) == RESET){}

/* 接收地址 0x99（发送的数据 MSB 位要为 1），可以唤醒 */
WakeUpSelection.WakeUpEvent    = UART_WAKEUP_ON_ADDRESS;
```

```
WakeUpSelection.AddressLength = UART_ADDRESS_DETECT_7B;
WakeUpSelection.Address       = 0x19;
if (HAL_UARTEx_StopModeWakeUpSourceConfig(&UartHandle, WakeUpSelection)!= HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}

CLEAR_BIT(LPUART1->CR1, USART_CR1_RXNEIE); /* 关闭串口接收中断 */

/* 进入停机模式 */
HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);

/* 退出停机模式要重新配置 HSE 和 PLL*/
SystemClock_Config();

SET_BIT(LPUART1->CR1, USART_CR1_RXNEIE);  /* 使能串口接收中断 */

/* 关闭 LPUART 的停机唤醒 */
HAL_UARTEx_DisableStopMode(&UartHandle);
```

这里有一点要特别注意，程序启动后，调用下面两个函数：

```
__HAL_RCC_LPUART1_CLKAM_ENABLE();     /* 激活 LPUART 的自主模式，即停机状态下可以继续接收消息 */
__HAL_UART_ENABLE_IT(&UartHandle, UART_IT_WUF);/* 使能唤醒中断 */
```

66.4 低功耗串口 FIFO 板级支持包（bsp_lpuart_fifo.c)

串口驱动文件 bsp_lpuart_fifo.c 主要实现了如下几个 API 供用户调用：

- bsp_InitLPUart
- lpcomSendBuf
- lpcomSendChar
- lpcomGetChar

66.4.1 函数 bsp_InitLPUart

函数原型：

```
void bsp_InitLPUart(void)
```

函数描述：

此函数主要用于串口的初始化，使用所有其它 API 之前，务必优先调用此函数。

使用举例：

串口的初始化函数在 bsp.c 文件的 bsp_Init 函数里面调用。

66.4.2 函数 lpcomSendBuf

函数原型：

```
void lpcomSendBuf(LPCOM_PORT_E _ucPort, uint8_t *_ucaBuf, uint16_t _usLen);
```

函数描述：

此函数用于向串口发送一组数据，非阻塞方式，数据放到发送缓冲区后立即返回，由中断服务程序在后台完成发送。

函数参数：

- 第 1 个参数_ucPort 是端口号。
- 第 2 个参数_ucaBuf 是待发送的数据缓冲区地址。
- 第 3 个参数_usLen 是要发送数据的字节数。

注意事项：

- 此函数的解读在本章 66.3.5 小节。
- 发送的数据最好不要超过 bsp_lpuart_fifo.h 文件中定义的发送缓冲区大小，从而实现最优的工作方式。因为超过后需要在发送函数等待有发送空间可用。

使用举例：

调用此函数前，务必优先调用函数 bsp_InitLPUart 进行初始化。

```
const char buf1[] = "接收到串口命令 1\r\n";
lpcomSendBuf(LPCOM1, (uint8_t *)buf1, strlen(buf1));
```

66.4.3 函数 lpcomSendChar

函数原型：

```
void lpcomSendChar(LPCOM_PORT_E _ucPort, uint8_t _ucByte);
```

函数描述：

此函数用于向串口发送 1 个字节，非阻塞方式，数据放到发送缓冲区后立即返回，由中断服务程序在后台完成发送。此函数是通过调用函数 lpcomSendBuf 实现的。

函数参数：

- 第 1 个参数_ucPort 是端口号。
- 第 2 个参数_ucByte 是待发送的数据。

注意事项：

- 此函数的解读在本章 66.3.2 小节。

使用举例：

调用此函数前，务必优先调用函数 bsp_InitLPUart 进行初始化。比如通过串口 1 发送一个字符 c：lpcomSendChar(LPCOM1, 'c')。

66.4.4 函数 lpcomGetChar

函数原型：

```
uint8_t lpcomGetChar(LPCOM_PORT_E _ucPort, uint8_t *_pByte)
```

函数描述：

此函数用于从接收缓冲区读取 1 字节，非阻塞。无论有无数据均立即返回。

函数参数：

- 第 1 个参数_ucPort 是端口号。
- 第 2 个参数_pByte 用于存放接收到的数据。
- 返回值，返回 0 表示无数据，1 表示读取到有效字节。

注意事项：

- 此函数的解读在本章 66.3.6 小节。

使用举例：

调用此函数前，务必优先调用函数 bsp_InitLPUart 进行初始化。

比如从串口 1 读取一个字符就是：lpcomGetChar(LPCOM1, &read)。

66.5 低功耗串口 FIFO 驱动移植和使用

串口 FIFO 移植步骤如下：

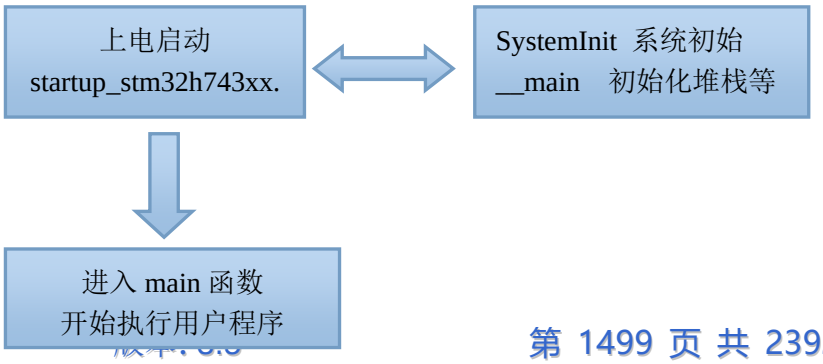
- 第 1 步：复制 bsp_lpuart_fifo.h 和 bsp_lpuart_fifo.c 到自己的工程目录，并添加到工程里面。
- 第 2 步：根据自己要使用的串口和收发缓冲大小，修改下面的宏定义即可。

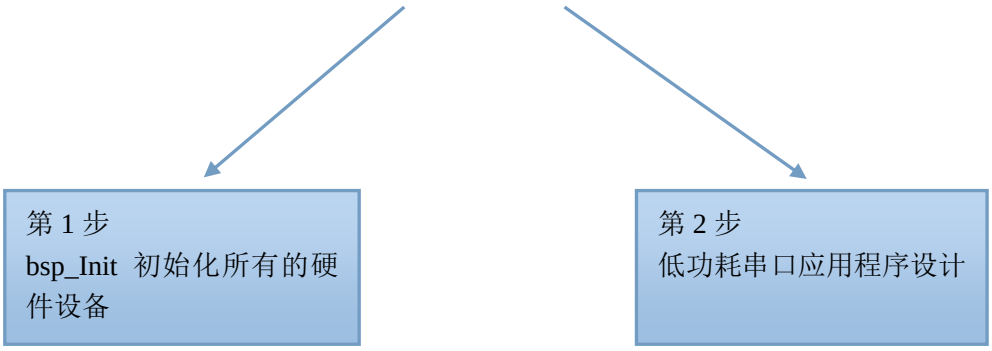
```
#define  LPUART1_FIFO_EN    1

/* 定义串口波特率和 FIFO 缓冲区大小，分为发送缓冲区和接收缓冲区，支持全双工 */
#if LPUART1_FIFO_EN == 1
    #define LPUART1_BAUD         115200
    #define LPUART1_TX_BUF_SIZE  1*1024
    #define LPUART1_RX_BUF_SIZE  1*1024
#endif
```

- 第 3 步：这几个驱动文件主要用到 HAL 库的 GPIO 和串口驱动文件，简单省事些可以添加所有 HAL 库.C 源文件进来。
- 第 4 步，应用方法看本章节配套例子即可。

66.6 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：





- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器和 LED。
  - 第 2 部分，应用程序设计部分，实现了三种停机唤醒方法。

66.7 实验例程说明（MDK）

配套例子：

V7-046_低功耗串口的停机唤醒（串口 FIFO 方式）

实验目的：

1. 学习低功耗串口的停机唤醒。

实验内容：

1. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
2. 当前程序使用的串口打印就是用的低功耗串口，即 USART1 和 LPUART1 都可以使用 PA9 和 PA10。
3. 上电启动了一个软件定时器，每 100ms 翻转一次 LED2。
4. USART1 和 LPUART 都可以使用 PA9 和 PA10 引脚做串口打印功能，本例子是用的 LPUART 做开发板串口打印。
5. LPUART 可以选择 HSI 时钟，LSE 时钟和 D3PCLK1 时钟，在 bsp_lpuart_fifo.c 文件开头可以配置。如果需要低功耗模式唤醒，必须使用 LSE 或者 HSI 时钟，波特率在 bsp_lpuart_fifo.h 定义，本例子是用的 HSI 时钟。

   LPUART 时钟选择 LSE(32768Hz)，最高速度是 10922bps，最低 8bps。

   LPUART 时钟选择 HSI(64MHz)，最高值是 21MHz，最小值 15625bps。

   LPUART 时钟选择 D3PCLK1(100MHz)，最大值 33Mbps，最小值 24414bps。

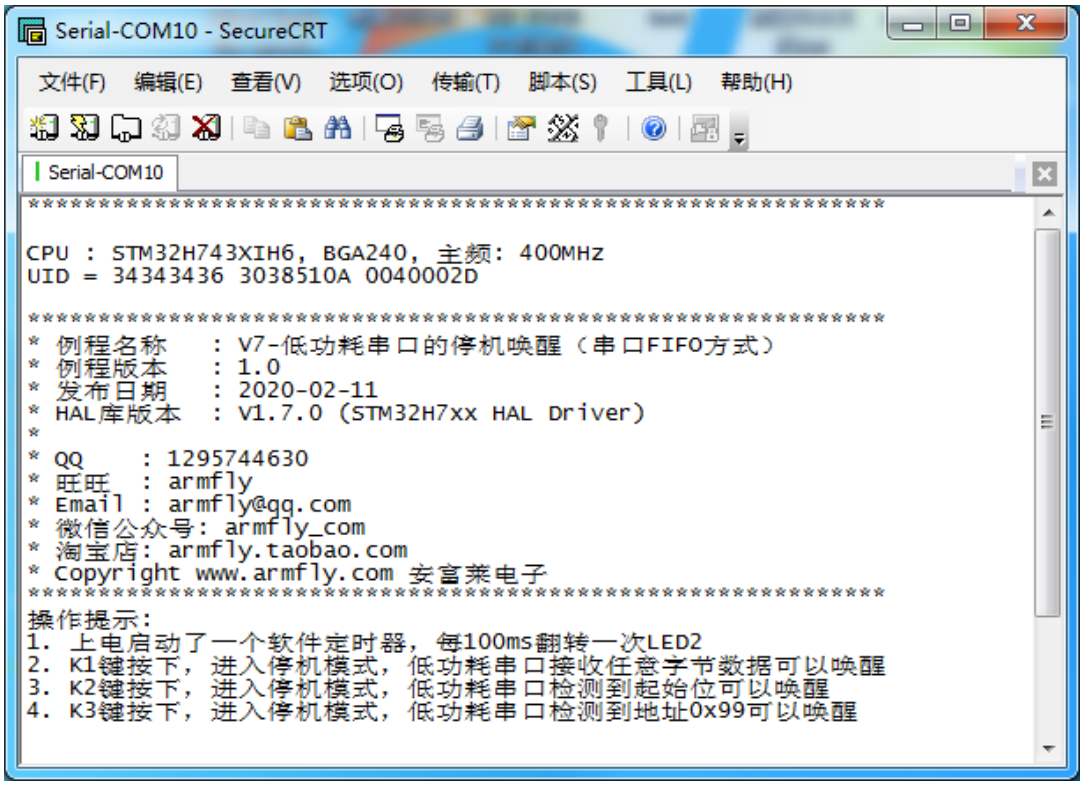
实验操作：

1. K1 键按下，进入停机模式，低功耗串口接收任意字节数据可以唤醒。

2. K2 键按下，进入停机模式，低功耗串口检测到起始位可以唤醒。
3. K3 键按下，进入停机模式，低功耗串口检测到地址 0x99 可以唤醒。

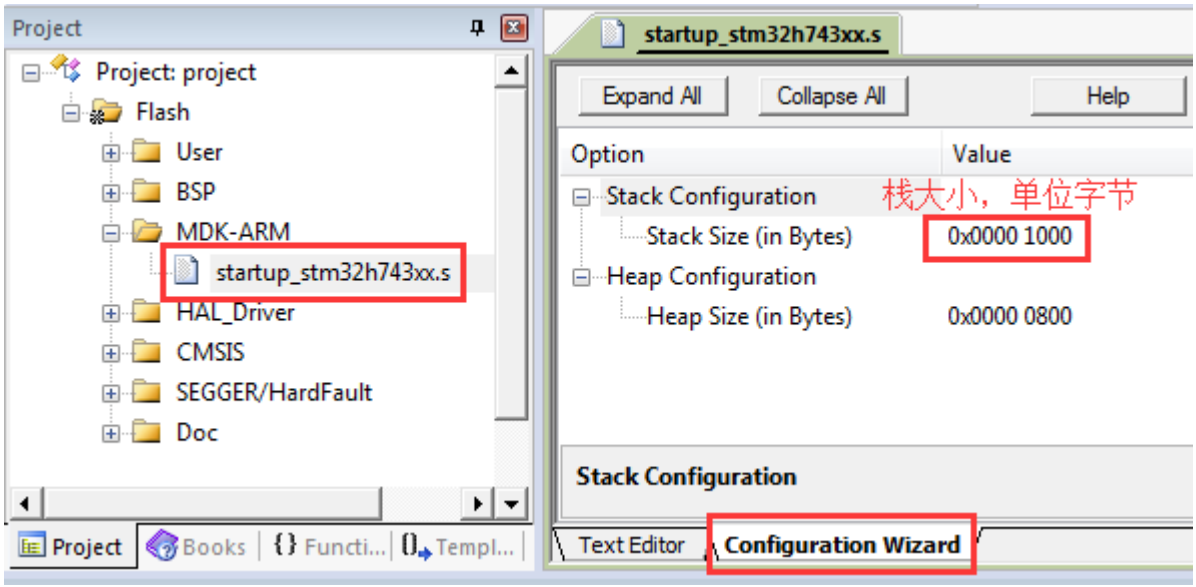
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1。

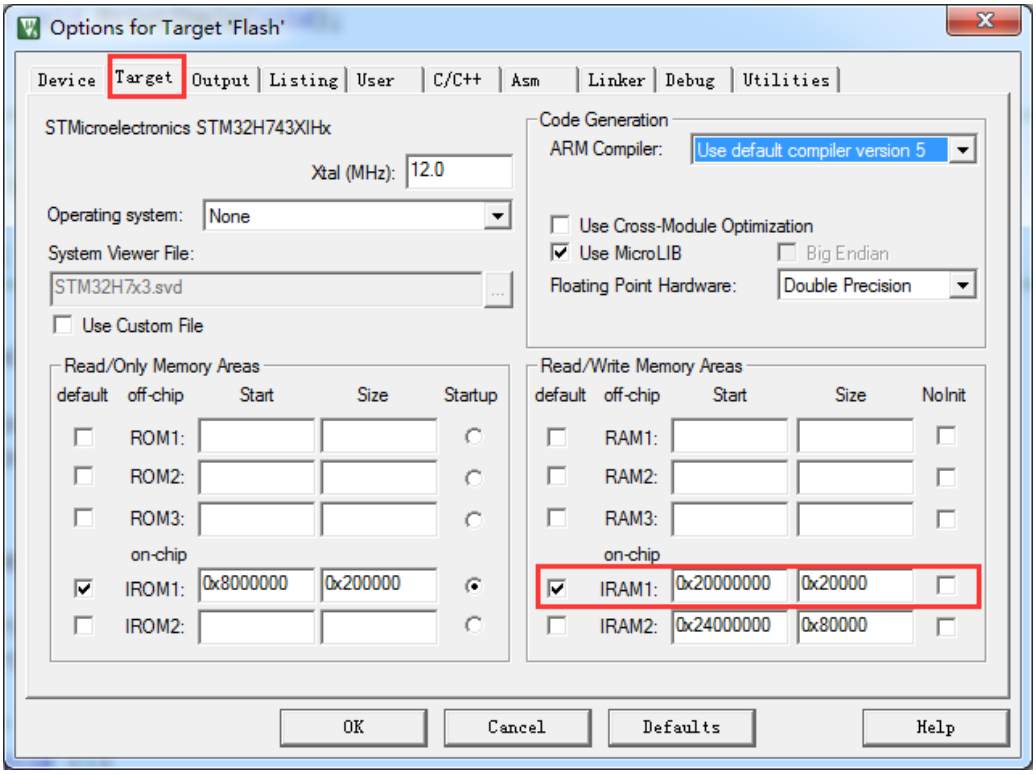


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitLPUart();  /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
   bsp_InitExtSDRAM(); /* 初始化 SDRAM */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
```

```
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ 每 10ms 调用一次蜂鸣器处理：

蜂鸣器处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ 主功能：

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- K1 键按下，进入停机模式，低功耗串口接收任意字节数据可以唤醒。
- K2 键按下，进入停机模式，低功耗串口检测到起始位可以唤醒。

- K3 键按下，进入停机模式，低功耗串口检测到地址 0x99 可以唤醒。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode; /* 按键代码 */
    uint8_t ucReceive;


    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    HAL_EnableDBGStopMode(); /* 使能停机模式下，LPUART 工程可以继续调试 */
    __HAL_RCC_WAKEUPSTOP_CLK_CONFIG(RCC_STOP_WAKEUPCLOCK_HSI); /* 从停机模式唤醒后使用 HSI 时钟 */
    __HAL_RCC_LPUART1_CLKAM_ENABLE();     /* 激活 LPUART 的自主模式，即停机状态下可以继续接收消息 */
    __HAL_UART_ENABLE_IT(&UartHandle, UART_IT_WUF);/* 使能唤醒中断 */


    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:  /* K1 键按下，进入停机模式，低功耗串口接收任意字节数据可以唤醒 */
                    /* 使能 LPUART 的停机唤醒 */
                    HAL_UARTEx_EnableStopMode(&UartHandle);

                    /* 确保 LPUART 没有在通信中 */
                    while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_BUSY) == SET){}
                    while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_REACK) == RESET){}

                    /* 接收到数据唤醒，即 RXNE 标志置位 */
                    WakeUpSelection.WakeUpEvent = UART_WAKEUP_ON_READDATA_NONEMPTY;
                    if (HAL_UARTEx_StopModeWakeUpSourceConfig(&UartHandle, WakeUpSelection)!= HAL_OK)
                    {
                        Error_Handler(__FILE__, __LINE__);
```

```
            }

            /* 进入停机模式 */
            HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);

            /* 退出停机模式要重新配置 HSE 和 PLL*/
            SystemClock_Config();

            /* 关闭 LPUART 的停机唤醒 */
            HAL_UARTEx_DisableStopMode(&UartHandle);

            lpcomGetChar(LPCOM1, &ucReceive);

            printf("低功耗串口接收到数据 %x 后唤醒\r\n", ucReceive);
            break;

        case KEY_DOWN_K2:   /* K2 键按下，进入停机模式，低功耗串口检测到起始位可以唤醒 */
            /* 使能 LPUART 的停机唤醒 */
            HAL_UARTEx_EnableStopMode(&UartHandle);

            /* 确保 LPUART 没有在通信中 */
            while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_BUSY) == SET){}
            while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_REACK) == RESET){}

            /* 接收起始位唤醒 */
            WakeUpSelection.WakeUpEvent = UART_WAKEUP_ON_STARTBIT;
            if (HAL_UARTEx_StopModeWakeUpSourceConfig(&UartHandle, WakeUpSelection)!= HAL_OK)
            {
                Error_Handler(__FILE__, __LINE__);
            }

            /* 进入停机模式 */
            HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);

            /* 退出停机模式要重新配置 HSE 和 PLL*/
            SystemClock_Config();

            /* 关闭 LPUART 的停机唤醒 */
            HAL_UARTEx_DisableStopMode(&UartHandle);

            lpcomGetChar(LPCOM1, &ucReceive);

            printf("低功耗串口检测到起始位（数据） %x 后唤醒\r\n", ucReceive);
            break;

        case KEY_DOWN_K3:  /* K3 键按下，进入停机模式，低功耗串口检测到地址 0x99 可以唤醒 */
            /* 使能 LPUART 的停机唤醒 */
            HAL_UARTEx_EnableStopMode(&UartHandle);

            /* 确保 LPUART 没有在通信中 */
            while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_BUSY) == SET){}
            while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_REACK) == RESET){}

            /* 接收地址 0x99（发送的数据 MSB 位要为 1），可以唤醒 */
            WakeUpSelection.WakeUpEvent    = UART_WAKEUP_ON_ADDRESS;
            WakeUpSelection.AddressLength = UART_ADDRESS_DETECT_7B;
            WakeUpSelection.Address        = 0x19;
            if (HAL_UARTEx_StopModeWakeUpSourceConfig(&UartHandle, WakeUpSelection)!= HAL_OK)
```

```
                        {
                            Error_Handler(__FILE__, __LINE__);
                        }

                        CLEAR_BIT(LPUART1->CR1, USART_CR1_RXNEIE); /* 关闭串口接收中断 */

                        /* 进入停机模式 */
                        HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);

                        /* 退出停机模式要重新配置 HSE 和 PLL*/
                        SystemClock_Config();

                        SET_BIT(LPUART1->CR1, USART_CR1_RXNEIE);  /* 使能串口接收中断 */

                        /* 关闭 LPUART 的停机唤醒 */
                        HAL_UARTEx_DisableStopMode(&UartHandle);

                        break;

                    default:
                        /* 其它的键值不处理 */
                        break;
                }
            }
        }
}
```

66.8 实验例程说明（IAR）

配套例子：

V7-046_低功耗串口的停机唤醒（串口 FIFO 方式）

实验目的：

1. 学习低功耗串口的停机唤醒。

实验内容：

1. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
2. 当前程序使用的串口打印就是用的低功耗串口，即 USART1 和 LPUART1 都可以使用 PA9 和 PA10。
3. 上电启动了一个软件定时器，每 100ms 翻转一次 LED2。
4. USART1 和 LPUART 都可以使用 PA9 和 PA10 引脚做串口打印功能，本例子是用的 LPUART 做开发板串口打印。
5. LPUART 可以选择 HSI 时钟，LSE 时钟和 D3PCLK1 时钟，在 bsp_lpuart_fifo.c 文件开头可以配置。如果需要低功耗模式唤醒，必须使用 LSE 或者 HSI 时钟，波特率在 bsp_lpuart_fifo.h 定义，本例子是用的 HSI 时钟。

   LPUART 时钟选择 LSE(32768Hz)，最高速度是 10922bps，最低 8bps。

   LPUART 时钟选择 HSI(64MHz)，最高值是 21MHz，最小值 15625bps。

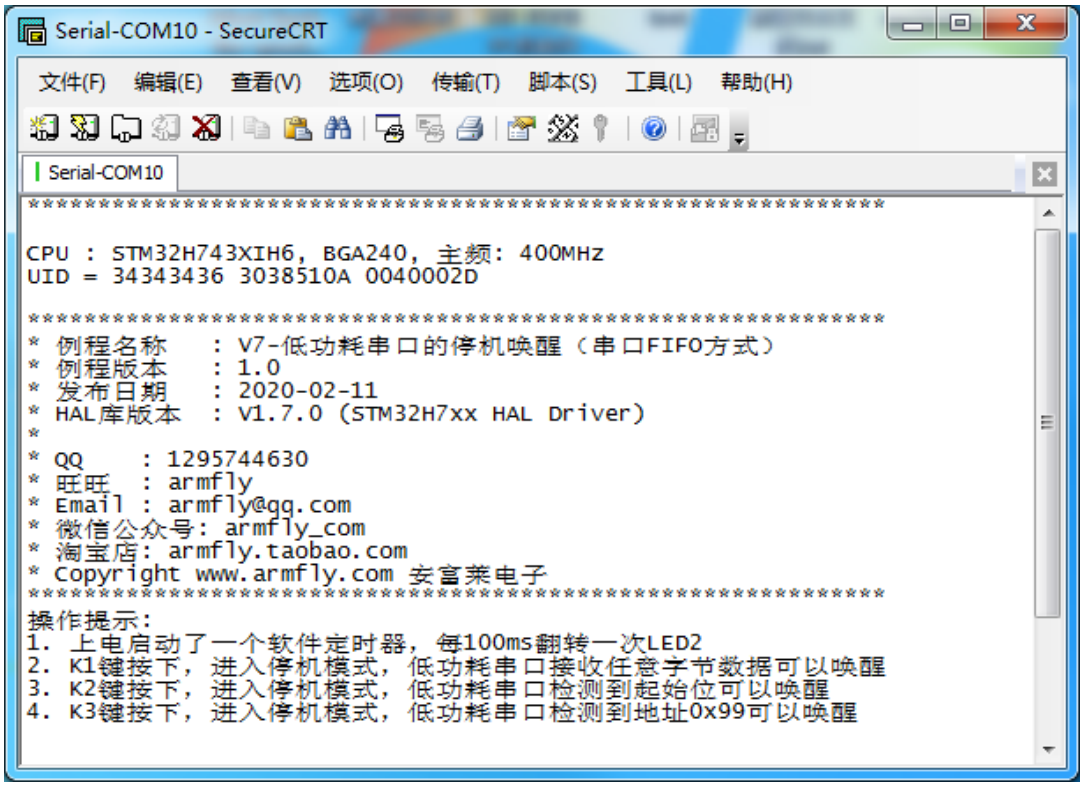
   LPUART 时钟选择 D3PCLK1(100MHz)，最大值 33Mbps，最小值 24414bps。

实验操作：

1. K1 键按下，进入停机模式，低功耗串口接收任意字节数据可以唤醒。
2. K2 键按下，进入停机模式，低功耗串口检测到起始位可以唤醒。
3. K3 键按下，进入停机模式，低功耗串口检测到地址 0x99 可以唤醒。

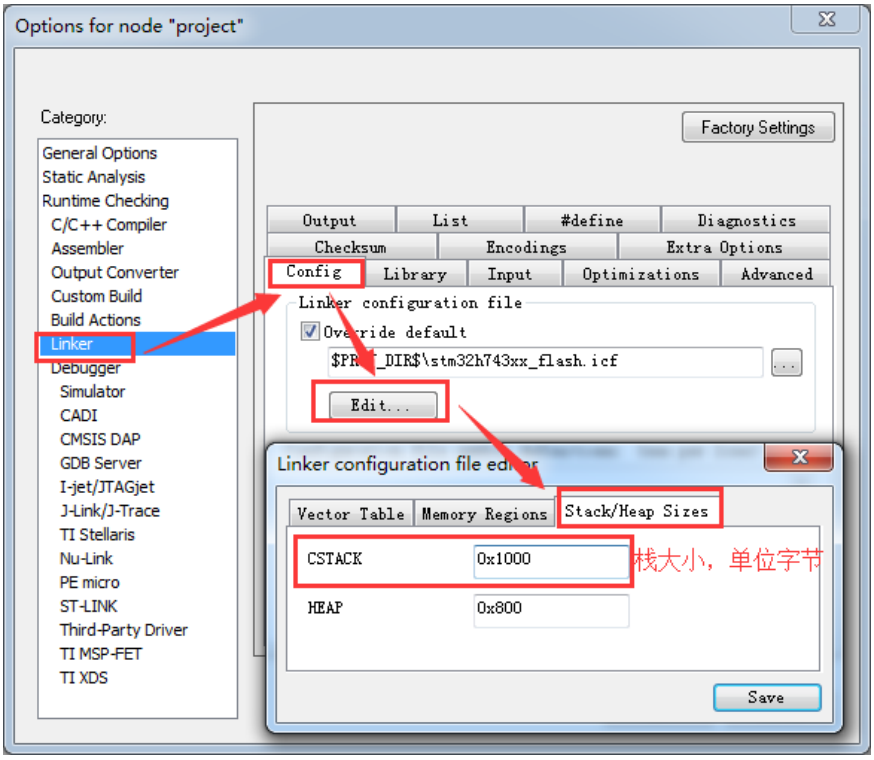
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

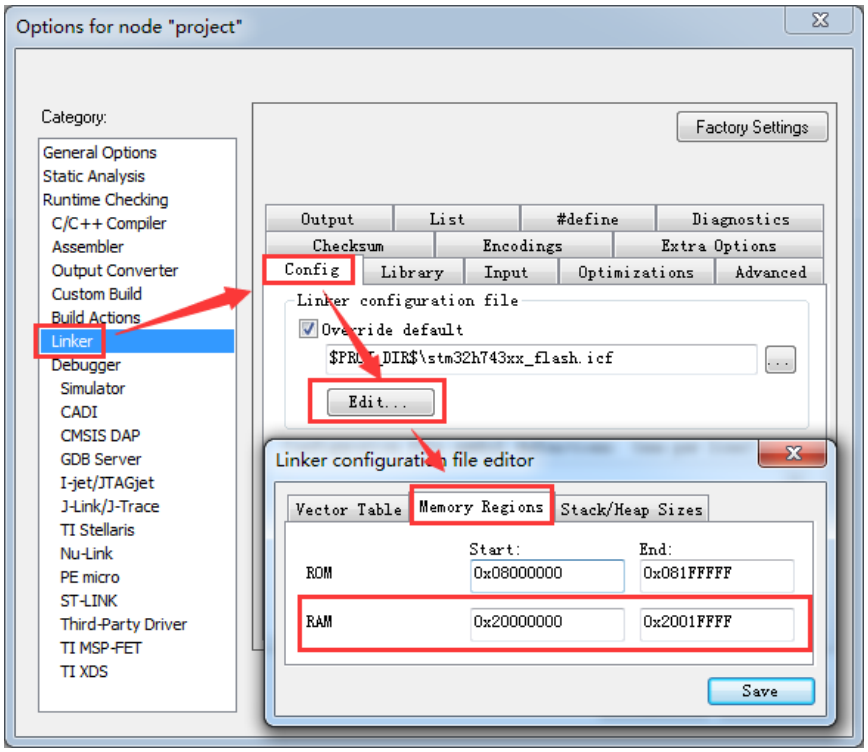


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
```

```
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitLPUart();  /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
```

```
}
```

◆ **每 10ms 调用一次蜂鸣器处理：**

蜂鸣器处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能：**

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- K1 键按下，进入停机模式，低功耗串口接收任意字节数据可以唤醒。
- K2 键按下，进入停机模式，低功耗串口检测到起始位可以唤醒。
- K3 键按下，进入停机模式，低功耗串口检测到地址 0x99 可以唤醒。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode; /* 按键代码 */
    uint8_t ucReceive;


    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    HAL_EnableDBGStopMode(); /* 使能停机模式下，LPUART 工程可以继续调试 */
    __HAL_RCC_WAKEUPSTOP_CLK_CONFIG(RCC_STOP_WAKEUPCLOCK_HSI); /* 从停机模式唤醒后使用 HSI 时钟 */
    __HAL_RCC_LPUART1_CLKAM_ENABLE();      /* 激活 LPUART 的自主模式，即停机状态下可以继续接收消息 */
    __HAL_UART_ENABLE_IT(&UartHandle, UART_IT_WUF);/* 使能唤醒中断 */


    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */
```

```
/* 判断定时器超时时间 */
if (bsp_CheckTimer(0))
{
    /* 每隔 100ms 进来一次 */
    bsp_LedToggle(2);
}

/* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
if (ucKeyCode != KEY_NONE)
{
    switch (ucKeyCode)
    {
        case KEY_DOWN_K1:  /* K1 键按下，进入停机模式，低功耗串口接收任意字节数据可以唤醒 */
            /* 使能 LPUART 的停机唤醒 */
            HAL_UARTEx_EnableStopMode(&UartHandle);

            /* 确保 LPUART 没有在通信中 */
            while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_BUSY) == SET){}
            while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_REACK) == RESET){}

            /* 接收到数据唤醒，即 RXNE 标志置位 */
            WakeUpSelection.WakeUpEvent = UART_WAKEUP_ON_READDATA_NONEMPTY;
            if (HAL_UARTEx_StopModeWakeUpSourceConfig(&UartHandle, WakeUpSelection)!= HAL_OK)
            {
                Error_Handler(__FILE__, __LINE__);
            }

            /* 进入停机模式 */
            HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);

            /* 退出停机模式要重新配置 HSE 和 PLL*/
            SystemClock_Config();

            /* 关闭 LPUART 的停机唤醒 */
            HAL_UARTEx_DisableStopMode(&UartHandle);

            lpcomGetChar(LPCOM1, &ucReceive);

            printf("低功耗串口接收到数据 %x 后唤醒\r\n", ucReceive);
            break;

        case KEY_DOWN_K2:   /* K2 键按下，进入停机模式，低功耗串口检测到起始位可以唤醒 */
            /* 使能 LPUART 的停机唤醒 */
            HAL_UARTEx_EnableStopMode(&UartHandle);

            /* 确保 LPUART 没有在通信中 */
            while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_BUSY) == SET){}
            while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_REACK) == RESET){}

            /* 接收起始位唤醒 */
            WakeUpSelection.WakeUpEvent = UART_WAKEUP_ON_STARTBIT;
            if (HAL_UARTEx_StopModeWakeUpSourceConfig(&UartHandle, WakeUpSelection)!= HAL_OK)
            {
                Error_Handler(__FILE__, __LINE__);
            }

            /* 进入停机模式 */
```

```
                    HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);

                    /* 退出停机模式要重新配置 HSE 和 PLL*/
                    SystemClock_Config();

                    /* 关闭 LPUART 的停机唤醒 */
                    HAL_UARTEx_DisableStopMode(&UartHandle);

                    lpcomGetChar(LPCOM1, &ucReceive);

                    printf("低功耗串口检测到起始位（数据） %x 后唤醒\r\n", ucReceive);
                    break;

                case KEY_DOWN_K3:  /* K3 键按下，进入停机模式，低功耗串口检测到地址 0x99 可以唤醒 */
                    /* 使能 LPUART 的停机唤醒 */
                    HAL_UARTEx_EnableStopMode(&UartHandle);

                    /* 确保 LPUART 没有在通信中 */
                    while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_BUSY) == SET){}
                    while(__HAL_UART_GET_FLAG(&UartHandle, USART_ISR_REACK) == RESET){}

                    /* 接收地址 0x99（发送的数据 MSB 位要为 1），可以唤醒 */
                    WakeUpSelection.WakeUpEvent    = UART_WAKEUP_ON_ADDRESS;
                    WakeUpSelection.AddressLength = UART_ADDRESS_DETECT_7B;
                    WakeUpSelection.Address       = 0x19;
                    if (HAL_UARTEx_StopModeWakeUpSourceConfig(&UartHandle, WakeUpSelection)!= HAL_OK)
                    {
                        Error_Handler(__FILE__, __LINE__);
                    }

                    CLEAR_BIT(LPUART1->CR1, USART_CR1_RXNEIE); /* 关闭串口接收中断 */

                    /* 进入停机模式 */
                    HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);

                    /* 退出停机模式要重新配置 HSE 和 PLL*/
                    SystemClock_Config();

                    SET_BIT(LPUART1->CR1, USART_CR1_RXNEIE);  /* 使能串口接收中断 */

                    /* 关闭 LPUART 的停机唤醒 */
                    HAL_UARTEx_DisableStopMode(&UartHandle);

                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

66.9 总结

本章节就为大家讲解这么多， 重点是低功耗串口的三种唤醒方式。

第67章 STM32H7 的系统 bootloader 基础知识

本章为大家介绍 STM32H7 内置的系统 bootloader 的基础知识。

67.1 初学者重要提示

67.2 系统 bootloader 基础知识

67.3 进入系统 bootloader 的方法

67.4 退出系统 bootloader 的方法

67.5 系统 bootloader 的擦写管理

67.6 总结

67.1 初学者重要提示

- 本章主要为大家介绍系统 bootloader 的理论知识，下个章节为大家实战。
- 更多系统 bootloader 的基础知识看本帖的 AN2606 应用笔记：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=96573 。

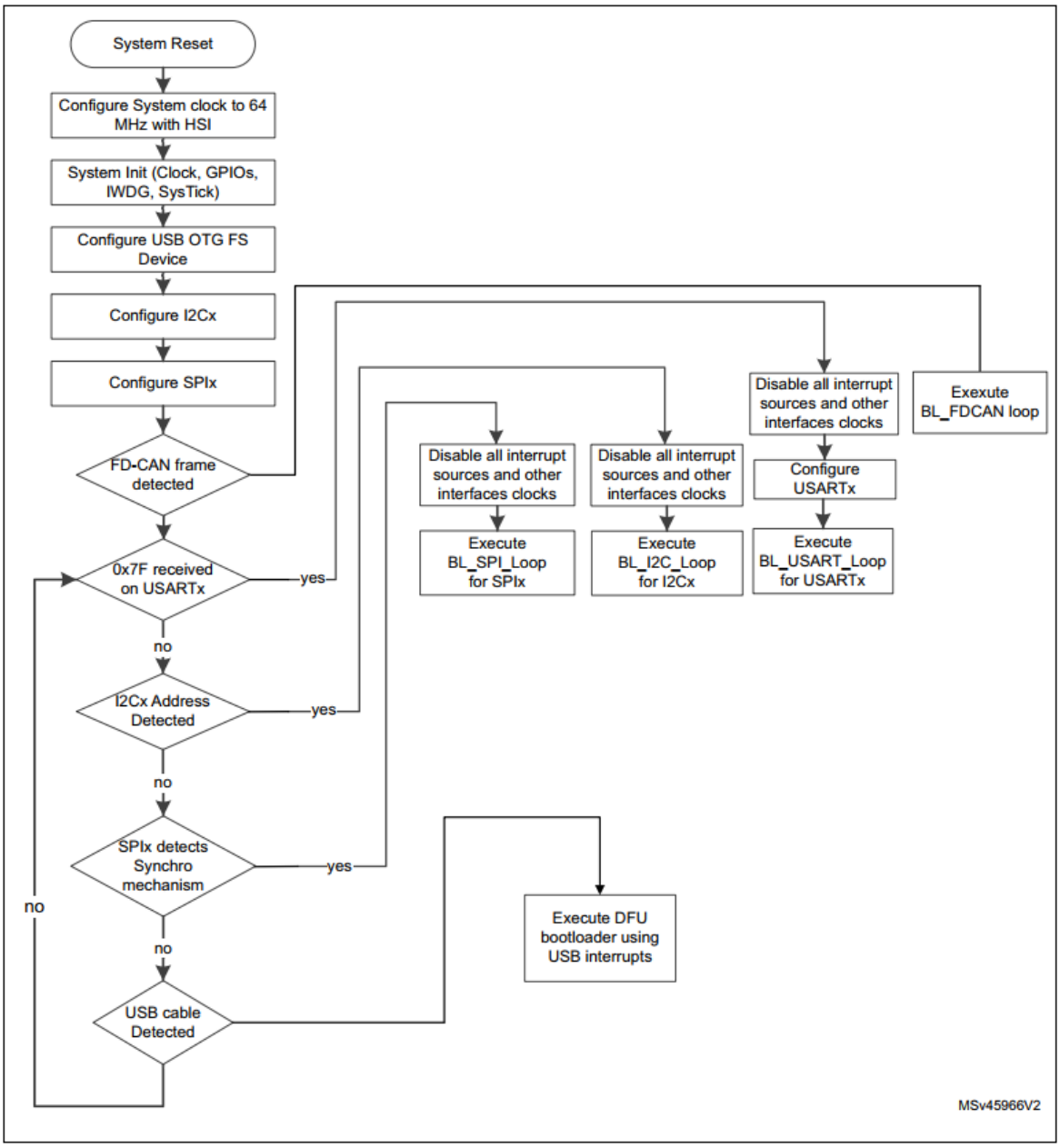
67.2 系统 bootLoader 基础知识

STM32 的系统存储区自带 bootloader，此程序是 ST 在芯片出厂时烧录进去的，主要用于将用户应用程序下载到芯片内部 Flash。支持 USB，SPI，I2C，CAN，UART 等接口方式下载。如果大家的应用程序打算采用这种接口方式进行升级，可以考虑采用系统 bootloader，简单易用，无需用户自己写 bootLoader 了。

更重要的是，使用系统 BootLoader 可以不依赖硬件 boot 引脚，可以直接从应用程序跳转到系统 BootLoader，这样灵活性就很大了。

67.2.1 系统 bootLoader 执行流程

下面是 STM32H7 的 BootLoader 程序执行流程，简单了解下即可：



关于这个执行流程，要特别注意一点：如果在进入系统 BootLoader 前就插入了 USB 线，会导致进入系统 BootLoader 后优先执行 USB DFU，其它的接口方式将没有机会执行。

67.2.2 系统 bootLoader 使用的引脚

STM32H7 的 Bootloader 使用到的引脚分配如下：

Bootloader peripherals	Bootloader pins
DFU	USB OTG FS (PA11/PA12) in device mode
USART1	PA9/PA10
USART2	PA2/PA3
USART3	PB10/PB11
CAN2	PH13/PH14
I2C1	PB6/PB9
I2C2	PF0/PF1
I2C3	PA8/PC9
I2C4	PD12/PD13
SPI1	PA4/PA5/PA6/PA7
SPI2	PI0/PI1/PI2/PI3

67.3 进入系统 bootLoader 的方法

STM32 进入系统 BootLoader 主要有两种方法，一种是设置 boot 引脚，另一种是应用程序直接跳转到系统 BootLoader。

67.3.1 设置硬件 boot 引脚进入系统 bootLoader

H7 的启动方式比较更灵活，只需一个 boot 引脚。但是一个引脚只能区分出两个状态，为了解决这个问题，H7 专门配套了两个 option bytes 选项字节配置，如此以来就可以方便设置各种存储器地址了。

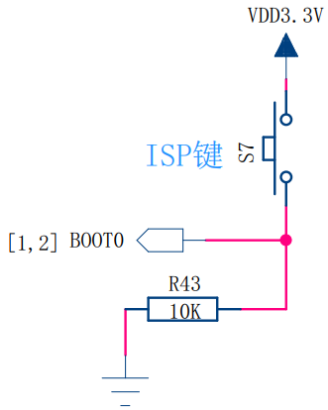
BOOT 模式选择		BOOT 区域
BOOT 引脚	BOOT 地址，选项字节	
0（低电平）	BOOT_ADD0[15:0]	启动地址由 BOOT_ADD0 定义，默认值是 0x0800，对应 Flash 首地址 0x0800 0000。
1（高电平）	BOOT_ADD1[15:0]	启动地址由 BOOT_ADD1 定义，默认值是 0x1FF0，对应系统 bootloader 的首地址 0x1FF0 0000。

BOOT_ADD0 和 BOOT_ADD1 对应 32 位地址到高 16 位，这点要特别注意。通过这两个选项字节，所有 0x0000 0000 到 0x3FFF 0000 的存储器地址都可以设置，包括：

- 所有 Flash 地址空间。
- 所有 RAM 地址空间，ITCM，DTCM 和 SRAM。

设置了选项字节后，掉电不会丢失，下次上电或者复位后，会根据 BOOT 引脚状态从 BOOT_ADD0，或 BOOT_ADD1 所设置的地址进行启动。也就是说是，H7 可以从 0x0000 0000 到 0x3FFF 0000 所有地址进行启动，这点与 F4 完全不同，F4 系列是固定从 0x0000 0000 启动的。

STM32H7 的 boot 引脚设计：



使用 BOOT 功能，注意以下几个问题：

- 如果用户不慎，设置的地址范围不在有效的存储器地址，那么 BOOT = 0 时，会从 Flash 首地址 0x0800 0000 启动，BOOT = 1 时，会从 ITCM 首地址 0x0000 0000 启动。
- 如果用户使能了 Flash Level 2 保护，那么只能从 Flash 地址空间进行启动。

67.3.2 用户应用程序跳转到系统 bootLoader

除了使用 boot 引脚控制运行系统 BootLoader，也可以上电后跳转，跳转前注意以下问题：

- 禁止所有外设时钟。
- 禁止使用的 PLL。
- 禁止所有中断。
- 清除所有中断挂起标志。
- 如果使用 Go 命令，对于 BootLoader 中使用的硬件外设寄存器，跳转前是不会设置到复位值的，如果用户代码中恰好也用到这些寄存器，需要重新配置。这里要特别注意的是 BootLoader 会用到看门狗，并且喂狗时间设为最长了，如果用户代码里面要用到看门狗请根据需要重新配置，并且看门狗一旦开启是无法关闭的。
- 对于具有双 bank 特性的 STM32，为了能够从用户代码跳转到系统 boot，需要将系统 boot 代码区使用寄存器 SYSCFG 重映射到 0x0000 0000（除了 F7 和 H7 系列）。对于 STM32F7 系列，需要禁止 nDBOOT / nDBANK 特性，然后跳转到系统 boot 区。
- 如果用到系统 bootloader 的 DFU/CAN 接口，需要用到 HSE 时钟，这个时钟的频率是通过内部 HSI/MSI 检测出具体频率。因此，由于外部温度等各种情况，内部 HSI 的精度会受到影响，从而影响检测出来的 HSE 时钟有较大误差，最终导致 DFU/CAN 运行失败。

具体实现会在下一章节为大家讲解。

67.4 退出系统 bootLoader 的方法

当前主要研究了 USB DFU 和串口 IAP 退出 bootLoader。

- USB DFU

  当芯片工作在系统 bootLoader 的 USB DFU 模式，更新完毕程序后，不会自动退出 USB DFU，需要重新复位芯片后才会退出。由于 DFU 模式会用到 USB 线，插拔 USB 线是难以避免的，所以是否支持自动退出，并不影响。

- 串口 IAP

  当芯片工作在系统 bootLoader 的串口升级模式，更新完毕程序后，可以自动退出。所以基于串口的组网设备，使用系统 bootloader 非常方便。

67.5 系统 bootLoader 的擦写管理

注：这部分知识点有个了解即可。

使用 bootloader 命令进行的所有写操作都只能字对齐（地址应该是 4 的倍数）。要写入的数据数量也必须是 4 的倍数（接受未对齐的半页写地址）。

- 有些产品不支持批量擦除操作。使用 BootLoader 进行批量擦除时，有两种方法可用：
  - 使用扇区擦除命令一个一个删除。
  - 将保护级别设置为 1，然后设置为 0（使用读保护命令，然后使用读非保护命令），将导致大批量擦除操作。
- STM32 L1 和 L0 系列的 Bootloader 除了支持操作内部 Flash，内部 SRAM，可选字节等，还支持操作 Data Memeory（数据存储区，貌似是指的 EEPROM）。数据存储区支持读写操作，而不支持擦除命令，如果要擦除，写 0 即可。另外对此存储区的写操作必须是 4 字节对齐（写地址），并且写入的数据也是 4 的倍数。
- F2, F4, F7 和 L4 除了支持操作内部 Flash，内部 SRAM，可选字节等，还支持操作 OTP 存储区。仅支持读写操作，不支持擦除命令。
- F2, F4 和 F7 系列的内部 Flash 写格式依赖于供电电压范围，默认的写操作只支持字节（半字，字和双字是不支持的），为了增加写操作速度，用户施加足够的电压范围以允许写操作按半字，字或双字，并通过虚拟内存位置的 boot 程序更新此配置。该内存位置不是物理地址，但可以根据协议使用常规的 BootLoader 读写操作。该存储位置包含 4 个字节，分别为如下表所述：

Address	Size	Description
0xFFFF0000	1 byte	This byte controls the current value of the voltage range. 0x00: voltage range [1.8 V, 2.1 V] 0x01: voltage range [2.1 V, 2.4 V] 0x02: voltage range [2.4 V, 2.7 V] 0x03: voltage range [2.7 V, 3.6 V] 0x04: voltage range [2.7 V, 3.6 V] and double word write/erase operation is used. In this case it is mandatory to supply 9 V through the VPP pin (refer to the product reference manual for more details about the double-word write procedure). Other: all other values are not supported and will be NACKed.
0xFFFF0001	1 byte	Reserved. 0xFF: default value. Other: all other values are not supported and will be NACKed.
0xFFFF0002	1 byte	Reserved. 0xFF: default value. Other: all other values are not supported and will be NACKed.
0xFFFF0003	1 byte	Reserved. 0xFF: default value. Other: all other values are not supported and will be NACKed.

67.6 总结

本章节就为大家讲解这么多，更新相关的知识看 ST 的应用笔记 AN2606。

第68章 STM32H7 的系统 bootloader 之 USB DFU 方式固件升级

本章节为大家讲解使用系统 bootloader 做程序升级的方法，即使不依赖外部 boot 引脚也可以方便升级。

DFU 的全称是 Device Firmware Upgrade,即设备固件升级

68.1 初学者重要提示
68.2 跳转到系统 bootloader 的程序设计
68.3 STM32CubeProg 的安装说明
68.3 STM32CubeProg 的程序下载说明
68.4 USB DFU 方式系统 bootloader 驱动移植和使用
68.6 实验例程设计框架
68.7 实验例程说明（MDK）
68.8 实验例程说明（IAR）
68.9 总结

68.1 初学者重要提示

- 学习本章节前，务必优先学习第 67 章。
- 特别注意 STM32H7 的系统 bootLoader 地址并不是 0x1FFF 0000。
- 本章用到的相关软件和文档下载：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=96573 。
- 软件 STM32CubeProg 和 DfuSe 都支持 USB DFU，但是两个软件不能都安装使用，因为这两个软件的 USB 驱动不同，导致工作在系统 bootloader 模式的板子通过 USB 线接到电脑端时，只有一个软件的驱动被识别。
- DfuSe 是老版的 USB DFU 软件，不推荐大家使用了。建议使用 STM32CubeProg，此软件实现了之前的 DfuSe，STLINK 小软件和 Flashloader 三合一，并且支持外部 EEPROM，NOR Flash，SPI Flash，NAND Flash 等烧写，也支持 OTA 编程。
- 本章节的 USB DFU 的下载软件采用 STM32CubeProg，如果想使用 DfuSe 的话，此贴有详细说明：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=11185 。

- 当芯片工作在系统 bootLoader 的 USB DFU 模式，更新完毕程序后，不会自动退出 USB DFU，需要重新复位芯片后才会退出。由于 DFU 模式会用到 USB 线，插拔 USB 线是难以避免的，所以是否支持自动退出，并不影响。

68.2 跳转到系统 bootLoader 的程序设计

程序设计如下，基本是按照第 67 章 3.2 小节的方法进行设计

```
/*
*********************************************************************************************************
*    函 数 名: JumpToBootloader
*    功能说明: 跳转到系统 BootLoader
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void JumpToBootloader(void)
{
    uint32_t i=0;
    void (*SysMemBootJump)(void);        /* 声明一个函数指针 */
    __IO uint32_t BootAddr = 0x1FF09800; /* STM32H7 的系统 BootLoader 地址 */

    /* 关闭全局中断 */
    DISABLE_INT();

    /* 关闭滴答定时器，复位到默认值 */
    SysTick->CTRL = 0;
    SysTick->LOAD = 0;
    SysTick->VAL = 0;

    /* 设置所有时钟到默认状态，使用 HSI 时钟 */
    HAL_RCC_DeInit();

    /* 关闭所有中断，清除所有中断挂起标志 */
    for (i = 0; i < 8; i++)
    {
        NVIC->ICER[i]=0xFFFFFFFF;
        NVIC->ICPR[i]=0xFFFFFFFF;
    }

    /* 使能全局中断 */
    ENABLE_INT();

    /* 跳转到系统 BootLoader，首地址是 MSP，地址+4 是复位中断服务程序地址 */
    SysMemBootJump = (void (*)(void)) (*((uint32_t *) (BootAddr + 4)));

    /* 设置主堆栈指针 */
    __set_MSP(*(uint32_t *)BootAddr);

    /* 在 RTOS 工程，这条语句很重要，设置为特权级模式，使用 MSP 指针 */
    __set_CONTROL(0);

    /* 跳转到系统 BootLoader */
    SysMemBootJump();

```

```
48.       /* 跳转成功的话，不会执行到这里，用户可以在这里添加代码 */
49.       while (1)
50.       {
51.
52.       }
53.   }
```

这里把程序设计中的几个关键地方做个说明：

- 第 12 行，声明一个函数指针。
- 第 13 行，这个要特别注意，H7 的系统 Bootloader 地址在 0x1FF09800。
- 第 19 到 21 行，设置滴答定时器到复位值。
- 第 24 行，此函数比较省事，可以方便的设置 H7 所有时钟到复位值，内部时钟使用 HSI。
- 第 27 到 31 行，清除所有中断挂起标志并关闭中断，这里是直接通过一个 for 循环设置了 NVIC 所有配置位，共 8 组。

Address	Name	Type	Required privilege	Reset value	Description
0xE000E100-0xE000E11C	NVIC_ISER0-NVIC_ISER7	RW	Privileged	0x00000000	*Interrupt set-enable registers on page 185*
0XE000E180-0xE000E19C	NVIC_ICER0-NVIC_ICER7	RW	Privileged	0x00000000	*Interrupt clear-enable registers on page 186*
0XE000E200-0xE000E21C	NVIC_ISPR0-NVIC_ISPR7	RW	Privileged	0x00000000	*Interrupt set-pending registers on page 186*
0XE000E280-0xE000E29C	NVIC_ICPR0-NVIC_ICPR7	RW	Privileged	0x00000000	*Interrupt clear-pending registers on page 187*
0xE000E300-0xE000E31C	NVIC_IABR0-NVIC_IABR7	RW	Privileged	0x00000000	*Interrupt active bit registers on page 188*
0xE000E400-0xE000E4EF	NVIC_IPR0-NVIC_IPR59	RW	Privileged	0x00000000	*Interrupt priority registers on page 188*
0xE000EF00	STIR	WO	Configurable(1)	0x00000000	*Software trigger interrupt register on page 189*

- 第 37 行，将系统 bootLoader 的中断复位服务程序的入口地址赋给第 12 行声明的函数，用户执行这个函数时，就会直接跳转过去。
- 第 40 行，设置主堆栈指针位置，即系统 bootloader 的首地址存储的就是栈地址。
- 第 43 行，这个设置在 RTOS 应用程序中比较重要，因为基于 Cortex-M 内核的 RTOS 任务堆栈基本都是使用线程堆栈指针 PSP。但系统 bootLoader 使用的是主堆栈指针 MSP，所以务必要设置下，同时让 M 内核工作于特权级。此寄存器的作用如下：

Table 4.3 Bit Fields in CONTROL Register

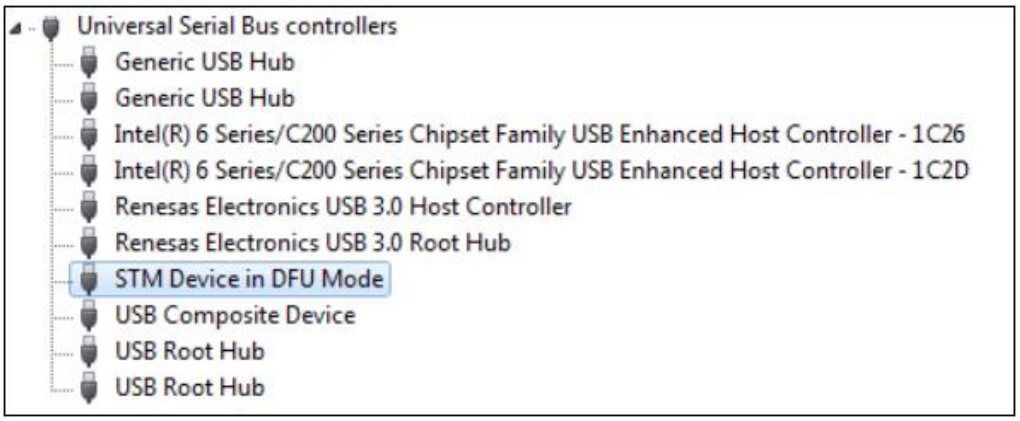
Bit	Function
nPRIV (bit 0)	Defines the privileged level in Thread mode: When this bit is 0 (default), it is privileged level when in Thread mode. When this bit is 1, it is unprivileged when in Thread mode. In Handler mode, the processor is always in privileged access level.
SPSEL (bit 1)	Defines the Stack Pointer selection: When this bit is 0 (default), Thread mode uses Main Stack Pointer (MSP). When this bit is 1, Thread mode uses Process Stack Pointer (PSP). In Handler mode, this bit is always 0 and write to this bit is ignored.
FPCA (bit 2)	Floating Point Context Active – This bit is only available in Cortex-M4 with floating point unit implemented. The exception handling mechanism uses this bit to determine if registers in the floating point unit need to be saved when an exception has occurred. When this bit is 0 (default), the floating point unit has not been used in the current context and therefore there is no need to save floating point registers. When this bit is 1, the current context has used floating point instructions and therefore need to save floating point registers. The FPCA bit is set automatically when a floating point instruction is executed. This bit is clear by hardware on exception entry. There are several options for handling saving of floating point registers. This will be covered in Chapter 13.

◆ 第 46 行，跳转到系统 bootLoader。

68.3 STM32CubeProg 的安装说明

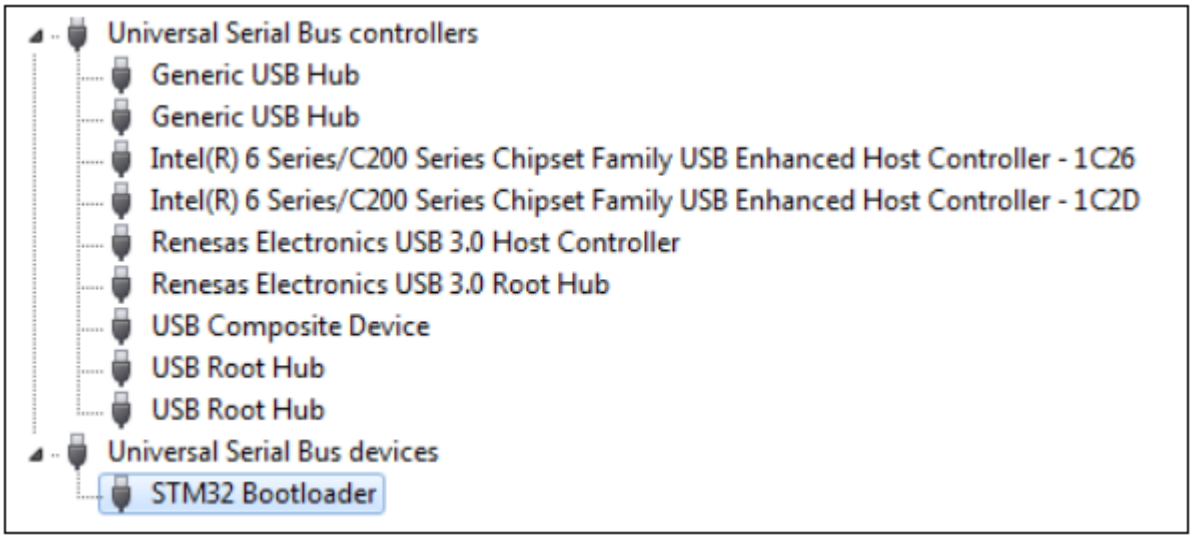
STM32CubeProg 的安装比较简单，如果大家的电脑中缺少 JAVA 环境，会提示安装，按照提示操作即可。

这里特别注意 USB DFU 驱动的安装，如果大家的电脑上安装了 DfuSe 软件，那边板子工作在系统 bootLoader 模式时，电脑端的设备管理器识别出来的标识是这样的：



如果用 STM32CubeProg 的话，务必要将此驱动删掉，鼠标右击此标识，选择卸载，弹出如下对话框：

卸载完毕后，重启电脑，然后运行 STM32CubeProg 安装目录里面的 STM32Bootloader.bat 即可，最后插上设备就可以正常识别了。识别后的标识：

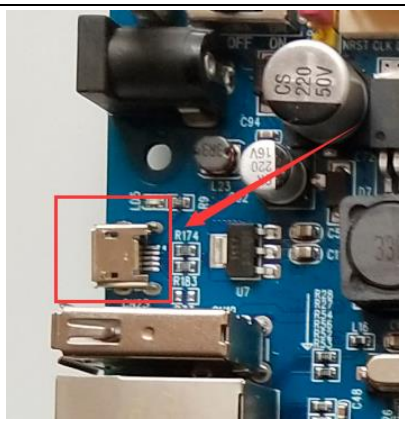


68.4 STM32CubeProg 的程序下载说明

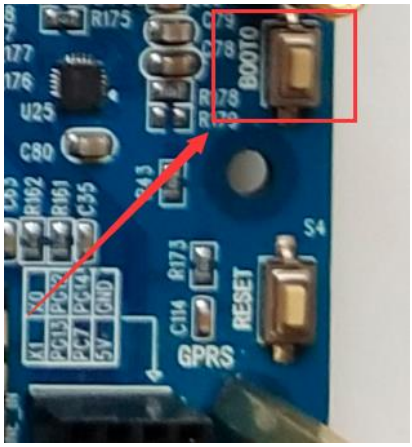
这里把两种下载方式都做个说明，一种是设置外部 boot 引脚进行下载，另一种是设置程序跳转到系统 bootloader 进行下载。

68.4.1 设置 boot 引脚跳转到系统 bootLoader

- 第 1 步：此接口插上 USB 线：

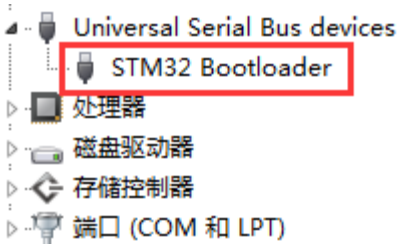


- 第 2 步：板子上电前按住右下角的 BOOT 引脚。



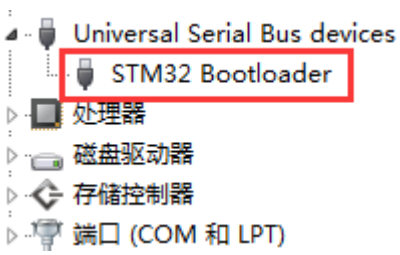
- 第 3 步：板子上电 3 秒左右，松手。

在电脑端设备管理器就可以看到已经识别出来：



68.4.2 应用程序跳转到系统 bootloader

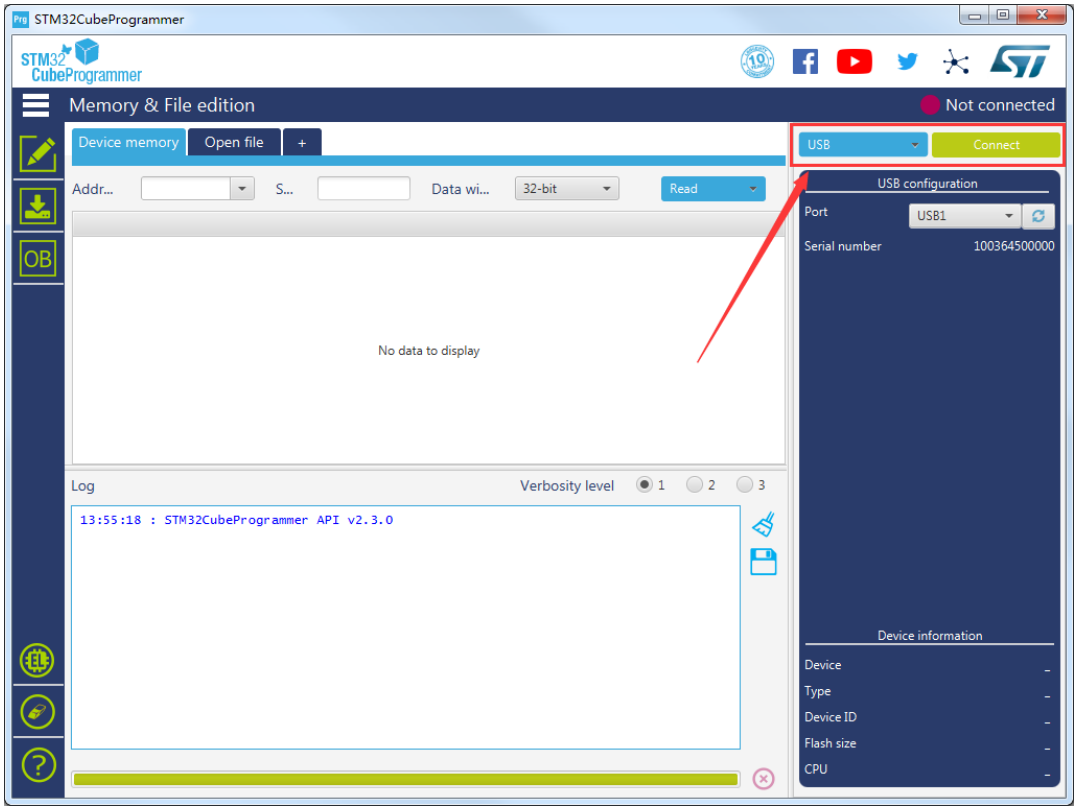
应用程序跳转到系统 bootLoader 比较方便，无需用户操作外置的 boot 引脚了，只需调用本章第 2 小节的程序就可以跳转。本章配套的例子是用户按下按键 K1 后执行跳转程序，大家可以根据需要实现各种触发跳转的方式。跳转成功后，在电脑端设备管理器里面也会看到 bootloader 标识：



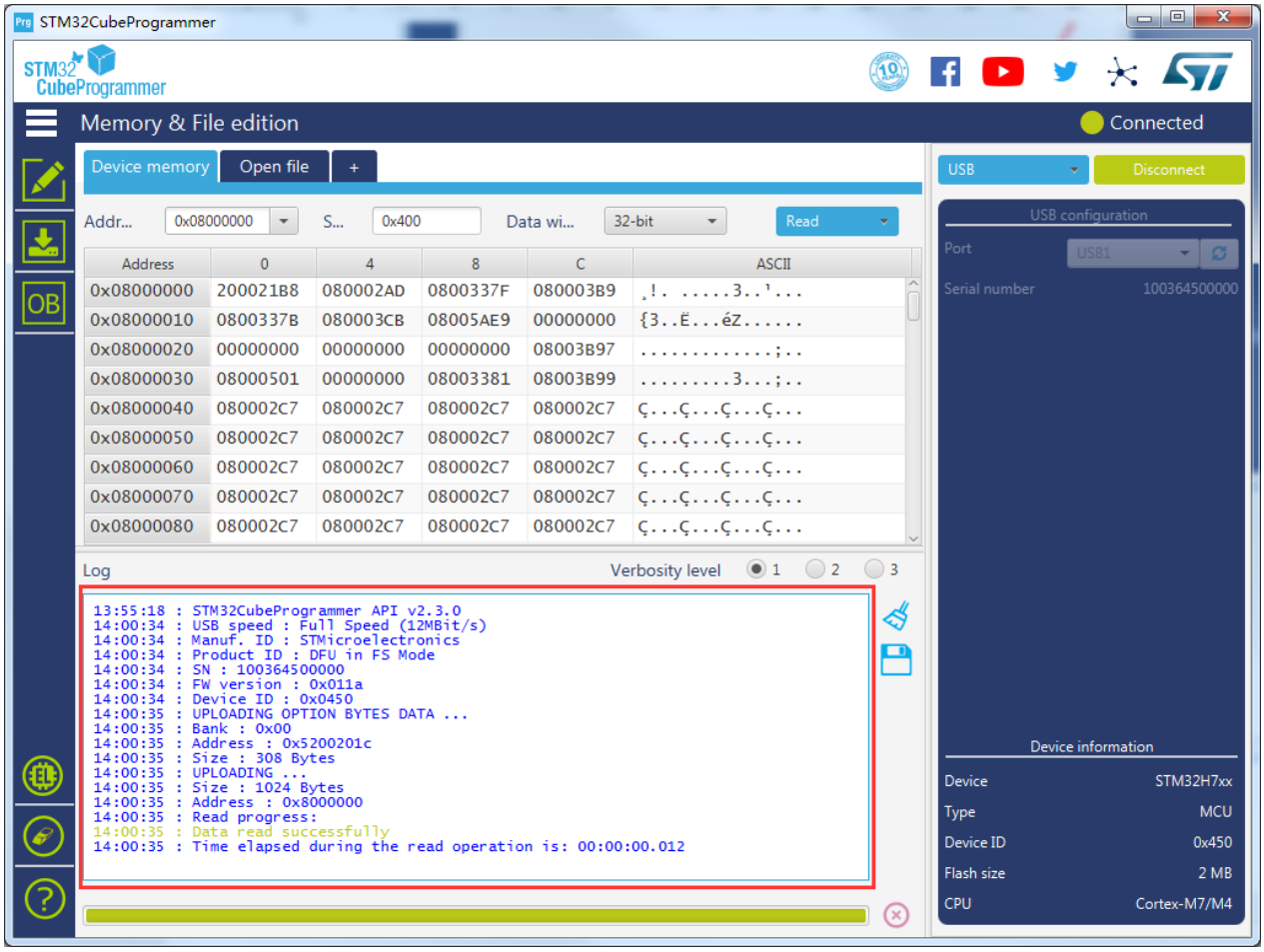
68.4.3 STM32CubeProg 下载程序设置

识别成功后就可以下载程序了。

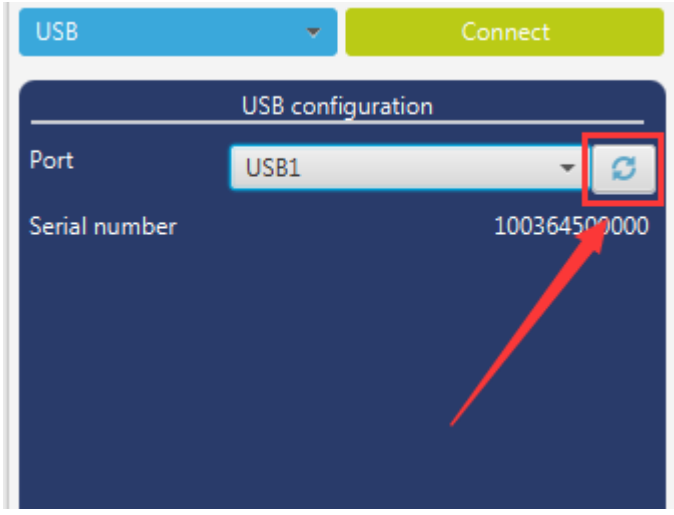
- 第 1 步，选择 USB 方式，点击 Connect 按钮。



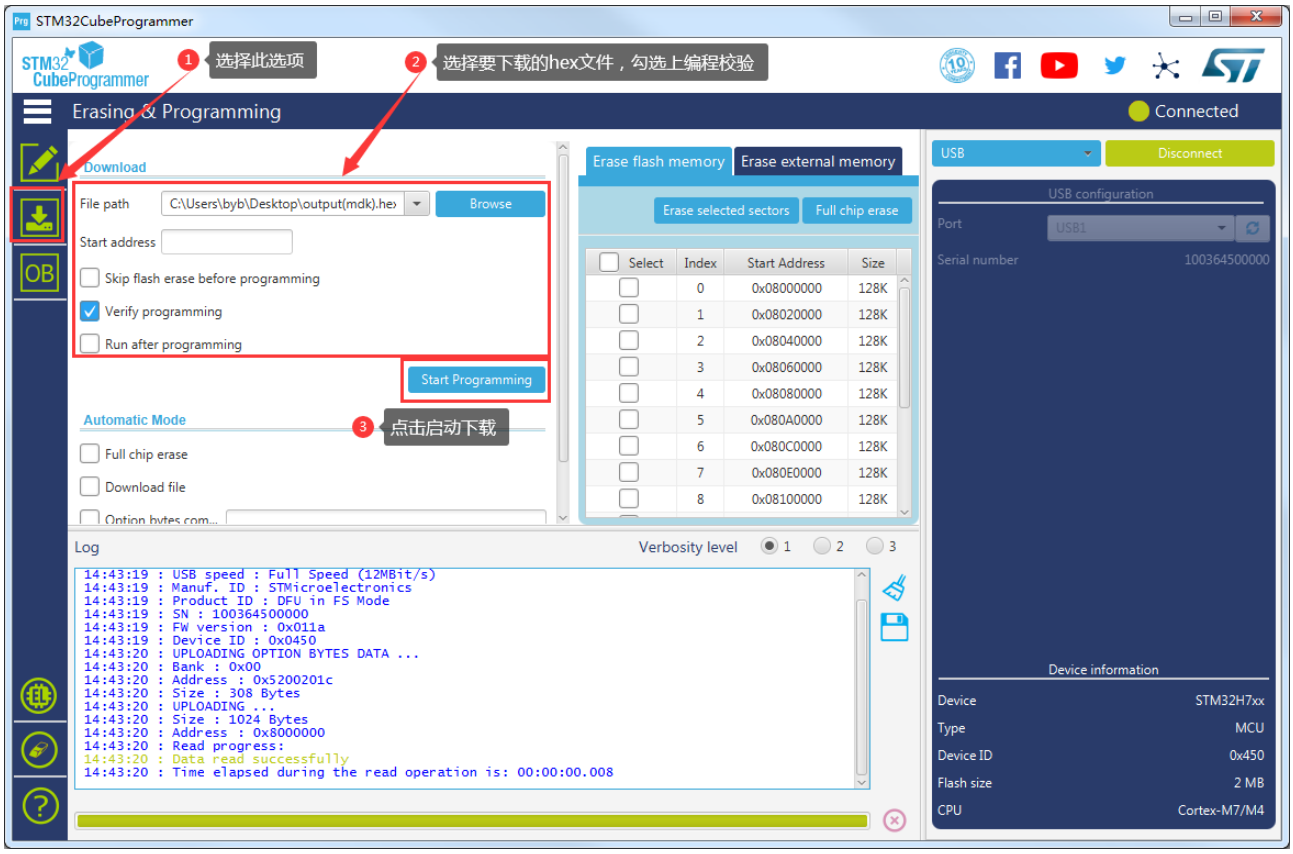
识别成功后的效果如下：



这里要特别注意一点，如果用户没有关闭这个软件，多次插拔 USB 线时，记得点击这里的刷新按钮，因为有时候这个软件不会自动显示出来，点击刷新按钮才行。



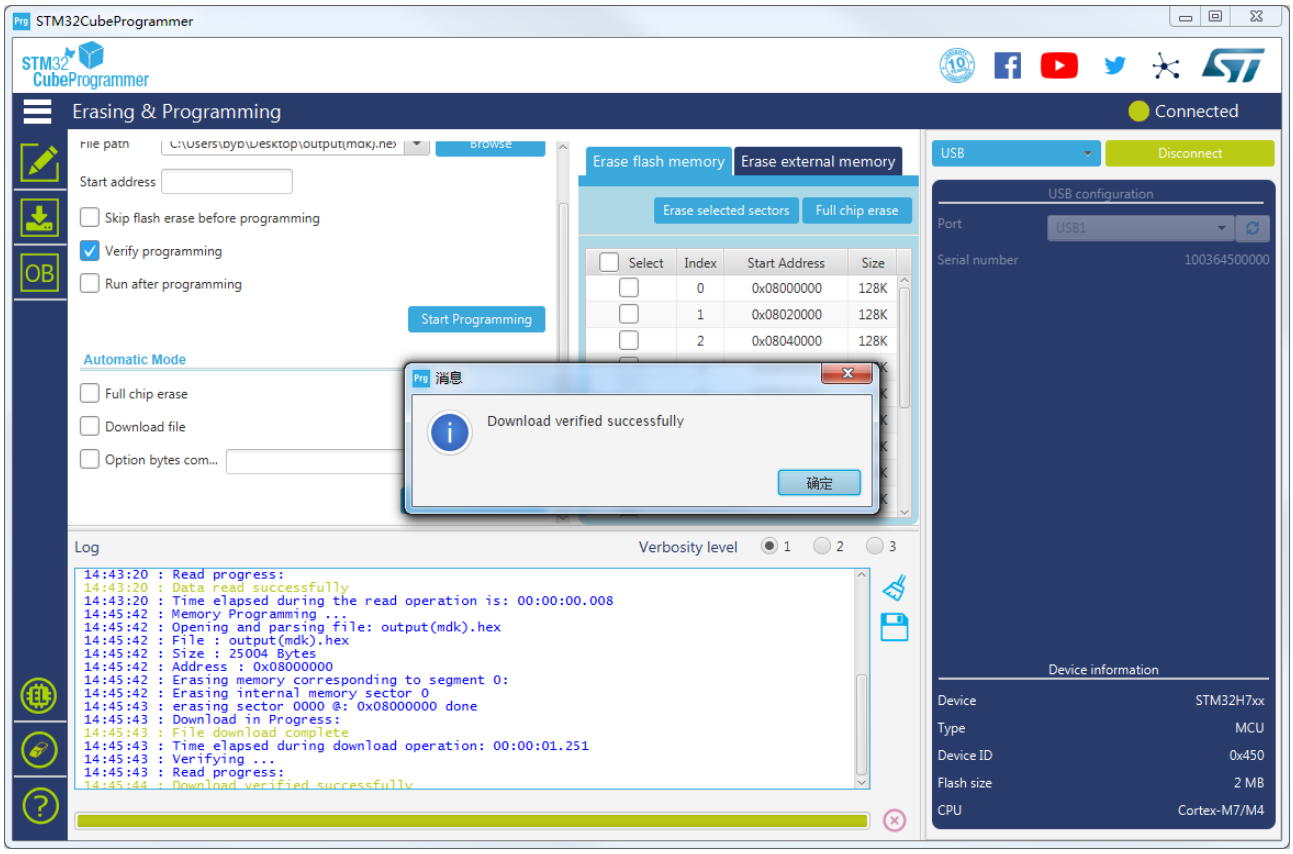
◆ 第 2 步，添加要下载的 hex 文件，勾选需要设置的选项，点击启动编程。



- Start address 选项不填的话，默认会下载到内部 Flash 的首地址，保险起见，大家也可以填上首地址 0x0800 0000，或者其它要下载的地址。
- Run after programming 选项勾选或者不勾选均可，因为测试发现 STM32CubeProg 不支持 USB DFU 编程后运行。这样特别说一点，如果勾上此选项后，下载完毕程序后，会自动断开连接，并弹出一些列窗口，最终弹出下面这个窗口：

弹出这个窗口并不是表示下载失败了，而是下载完成后退出了系统 bootloader。

◆ 第 3 步，完成下载后的效果如下：



下载完成后板子重新上电就可以看到程序已经成功下载了。

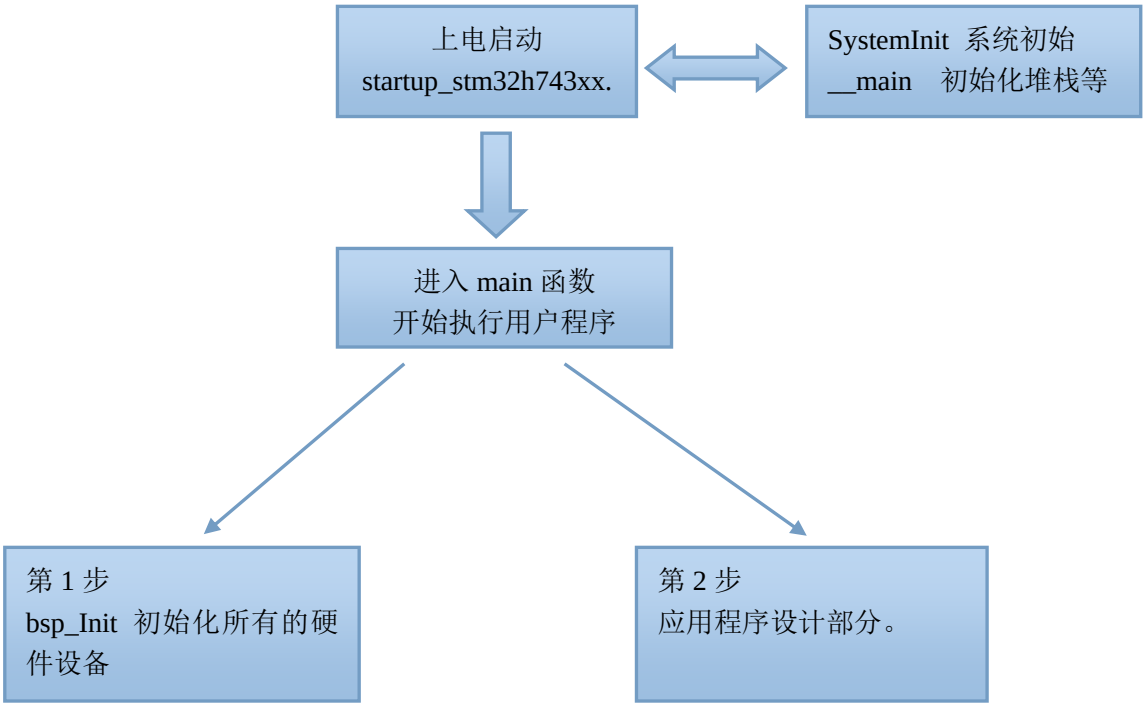
68.5 USB DFU 方式系统 Bootloader 驱动移植和使用

系统 bootloader 的移植比较简单，仅需添加本章第 2 小节的程序到自己工程里面即可。里面有个开关中断 API，是在 bsp.h 文件里面定义的：

```
/* 开关全局中断的宏 */
#define ENABLE_INT()    __set_PRIMASK(0)   /* 使能全局中断 */
#define DISABLE_INT()   __set_PRIMASK(1)   /* 禁止全局中断 */
```

68.6 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器和 LED。
  - 第 2 部分，应用程序设计部分，K1 按键按下后跳转到系统 bootloader。。

68.7 实验例程说明（MDK）

配套例子：

V7-047_基于系统 bootloader 的 USB 接口方式 IAP 升级（USB DFU）

实验目的：

1. 学习基于系统 bootloader 的 USB 接口方式 IAP 升级。

实验内容：

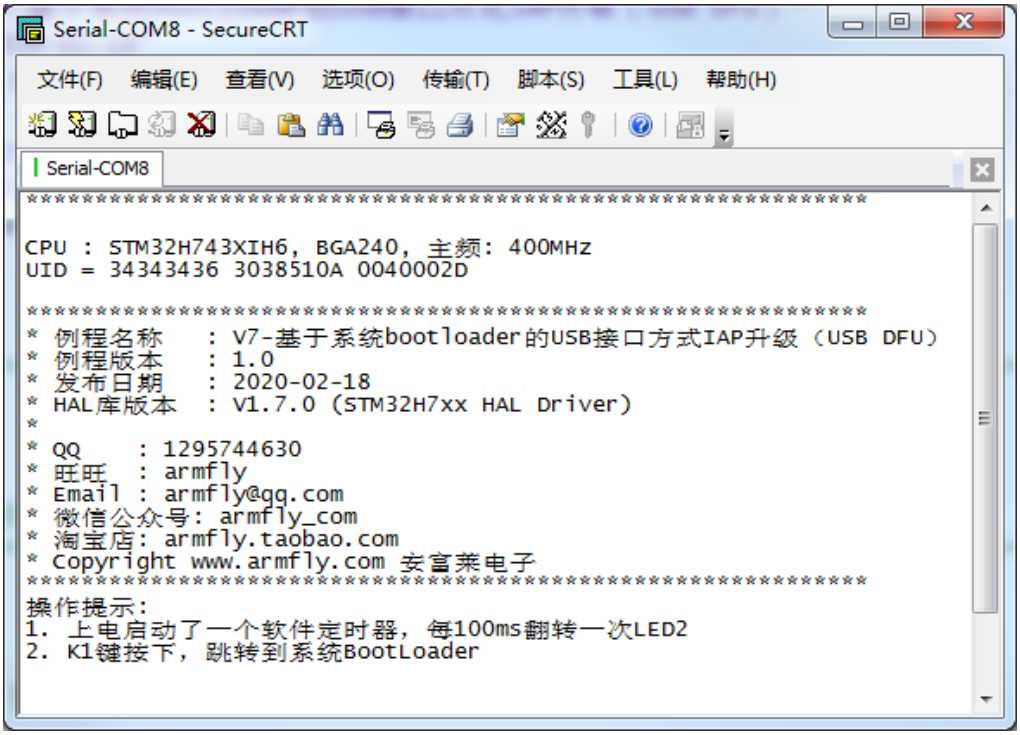
1. STM32 的系统存储区自带 bootLoader，可以方便的实现串口，I2C，CAN，SPI，USB 等接口方式的程序升级。
2. 如果使用系统 bootLoader 支持的接口升级方式，基本就不需要用户自己做 bootLoader 了。
3. 除了通过 boot 引脚控制启动地址，也可以直接从应用程序里面跳转到系统存储区。

实验操作：

1. K1 键按下，跳转到系统 bootLoader。

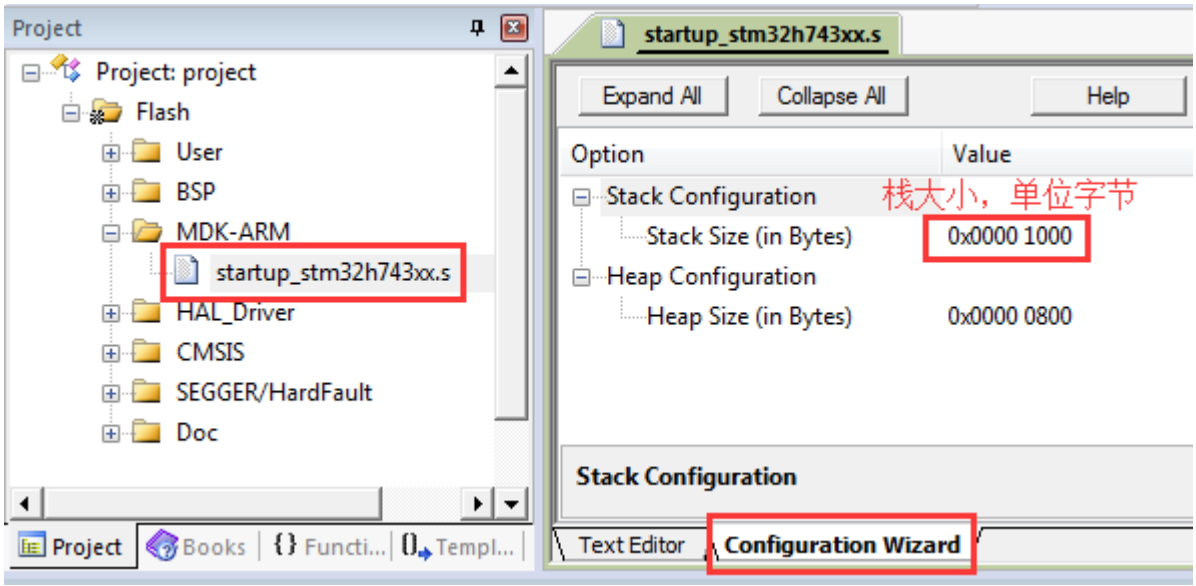
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1。

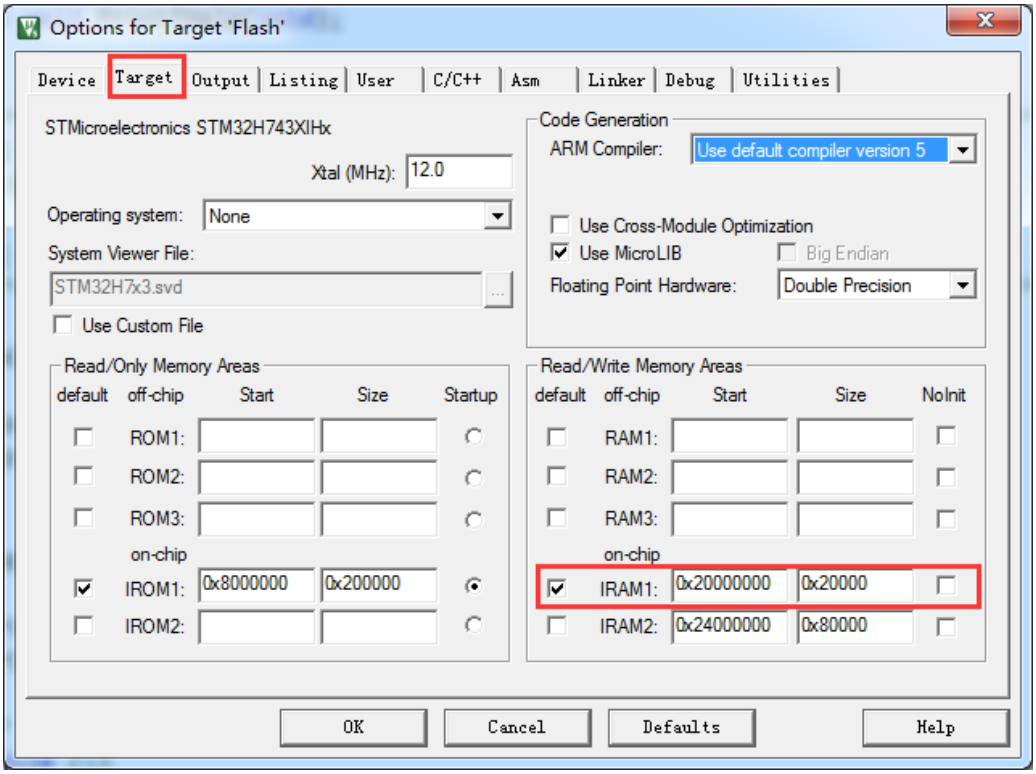


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
   bsp_InitExtSDRAM(); /* 初始化 SDRAM */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
```

```
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ 每 10ms 调用一次蜂鸣器处理：

蜂鸣器处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ 主功能：

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- K1 键按下，跳转到系统 BootLoader。

```
/*
*********************************************************************************************************
```

```
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode; /* 按键代码 */


    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，K1 键按下，跳转到系统 BootLoader */
                    JumpToBootloader();
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

68.8 实验例程说明（IAR）

配套例子：

V7-047_基于系统 bootloader 的 USB 接口方式 IAP 升级（USB DFU）

实验目的：

1. 学习基于系统 bootloader 的 USB 接口方式 IAP 升级。

实验内容：

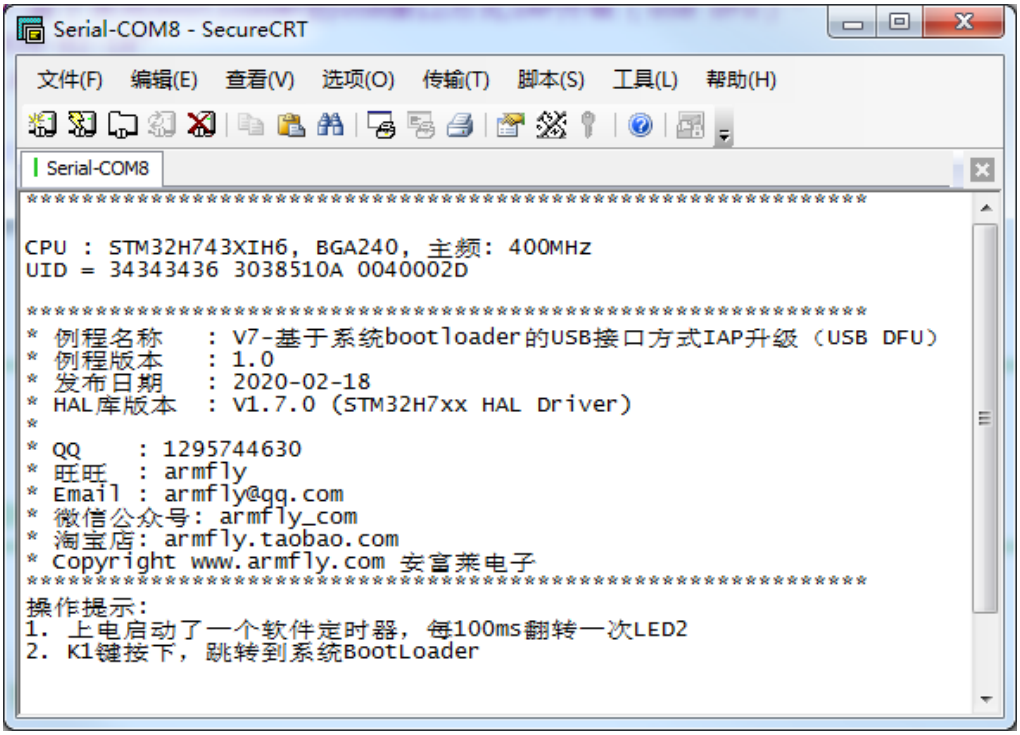
1. STM32 的系统存储区自带 bootLoader，可以方便的实现串口，I2C，CAN，SPI，USB 等接口方式的程序升级。
2. 如果使用系统 bootLoader 支持的接口升级方式，基本就不需要用户自己做 bootLoader 了。
3. 除了通过 boot 引脚控制启动地址，也可以直接从应用程序里面跳转到系统存储区。

实验操作：

1. K1 键按下，跳转到系统 bootLoader。

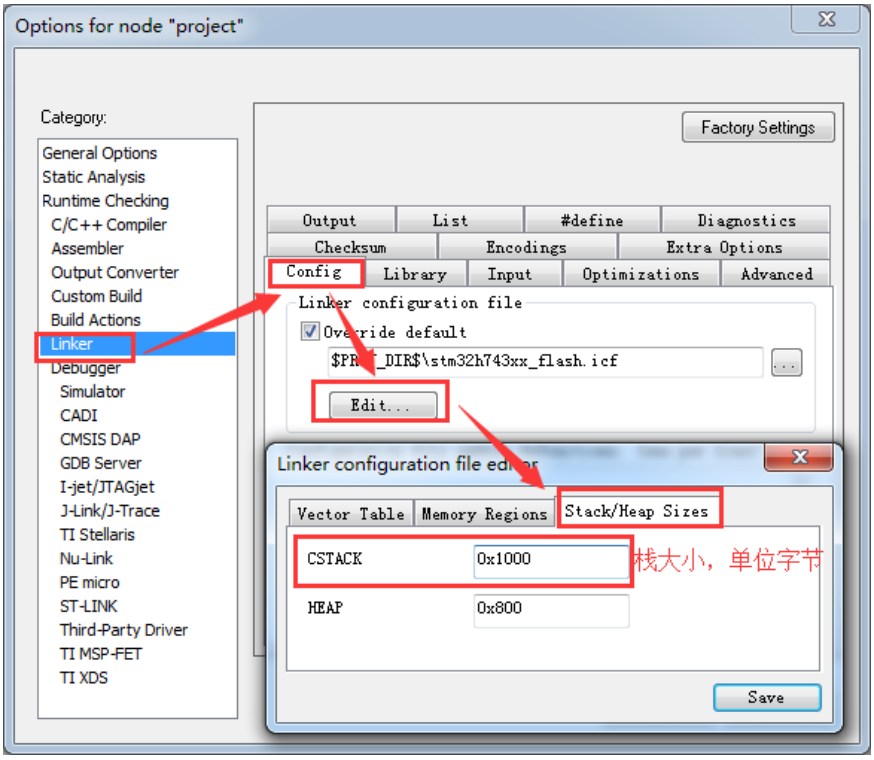
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1。

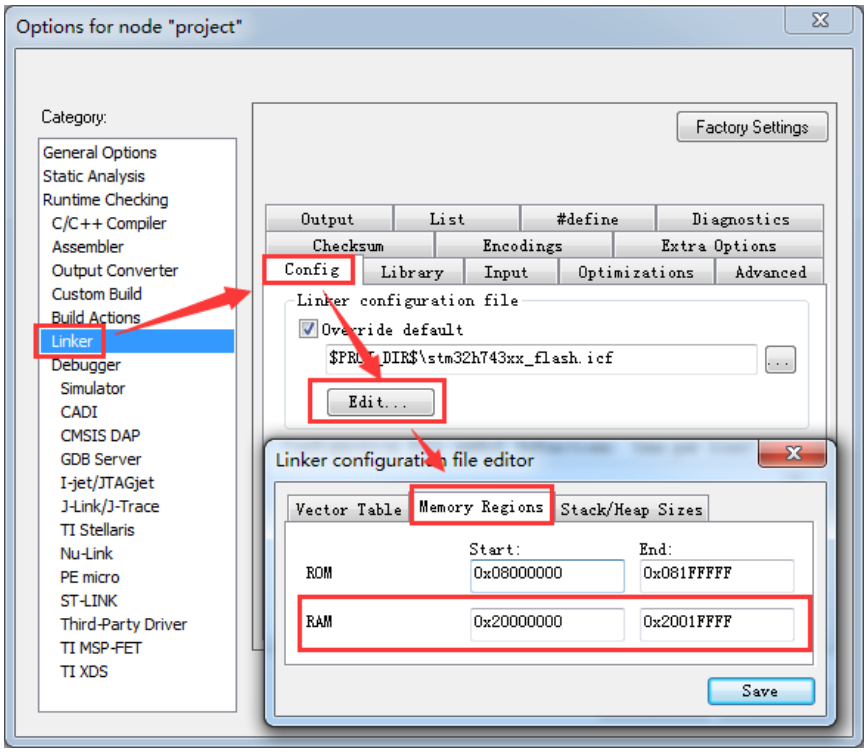


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
```

```
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
```

```
}
```

◆ **每 10ms 调用一次蜂鸣器处理：**

蜂鸣器处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能：**

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- K1 键按下，跳转到系统 BootLoader。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode; /* 按键代码 */


    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
```

```
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，K1 键按下，跳转到系统 BootLoader */
                    JumpToBootloader();
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

68.9 总结

本章节为大家介绍的 USB DFU 方式还是非常实用的，特别是产品硬件不带 boot 引脚时。

第69章 STM32H7 的系统 bootloader 之串口 IAP 固件升级

本章节为大家讲解使用系统 bootloader 做程序升级的方法，即使不依赖外部 boot 引脚也可以方便升级。

IAP 的全称是 In Application Programming，即在线应用编程

69.1 初学者重要提示
69.2 跳转到系统 bootloader 的程序设计
69.3 STM32CubeProg 的安装说明
69.3 STM32CubeProg 的程序下载说明
69.4 串口方式系统 bootloader 驱动移植和使用
69.6 实验例程设计框架
69.7 实验例程说明（MDK）
69.8 实验例程说明（IAR）
69.9 总结

69.1 初学者重要提示

- 学习本章节前，务必优先学习第 67 章。
- 特别注意 STM32H7 的系统 BootLoader 地址并不是 0x1FFF 0000。
- 本章用到的相关软件和文档下载：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=96573 。
- 本章节的串口 IAP 下载软件使用 STM32CubeProg，此软件实现了之前的 DfuSe，STLINK 小软件和 Flashloader 三合一，并且支持外部 EEPROM，NOR Flash，SPI Flash，NAND Flash 等烧写，也支持 OTA 编程。
- 使用系统 bootloader 做串口 IAP 升级时，MicroUSB 接口不要接线到电脑端，因为这会导致系统 bootloader 工作在 USB DFU 模式，无法再使用串口 IAP。

69.2 跳转到系统 bootLoader 的程序设计

程序设计如下，基本是按照第 67 章 3.2 小节的方法进行设计

```
54.  /*
55.  *********************************************************************************************************
56.  *    函 数 名: JumpToBootloader
57.  *    功能说明: 跳转到系统 BootLoader
58.  *    形    参: 无
59.  *    返 回 值: 无
60.  *********************************************************************************************************
61.  */
62.  static void JumpToBootloader(void)
63.  {
64.      uint32_t i=0;
65.      void (*SysMemBootJump)(void);        /* 声明一个函数指针 */
66.      __IO uint32_t BootAddr = 0x1FF09800; /* STM32H7 的系统 BootLoader 地址 */
67.
68.      /* 关闭全局中断 */
69.      DISABLE_INT();
70.
71.      /* 关闭滴答定时器，复位到默认值 */
72.      SysTick->CTRL = 0;
73.     SysTick->LOAD = 0;
74.     SysTick->VAL = 0;
75.
76.      /* 设置所有时钟到默认状态，使用 HSI 时钟 */
77.      HAL_RCC_DeInit();
78.
79.      /* 关闭所有中断，清除所有中断挂起标志 */
80.      for (i = 0; i < 8; i++)
81.      {
82.          NVIC->ICER[i]=0xFFFFFFFF;
83.          NVIC->ICPR[i]=0xFFFFFFFF;
84.      }
85.
86.      /* 使能全局中断 */
87.      ENABLE_INT();
88.
89.      /* 跳转到系统 BootLoader，首地址是 MSP，地址+4 是复位中断服务程序地址 */
90.      SysMemBootJump = (void (*)(void)) (*((uint32_t *) (BootAddr + 4)));
91.
92.      /* 设置主堆栈指针 */
93.      __set_MSP(*(uint32_t *)BootAddr);
94.
95.      /* 在 RTOS 工程，这条语句很重要，设置为特权级模式，使用 MSP 指针 */
96.      __set_CONTROL(0);
97.
98.      /* 跳转到系统 BootLoader */
99.      SysMemBootJump();
100.
101.     /* 跳转成功的话，不会执行到这里，用户可以在这里添加代码 */
102.     while (1)
103.     {
104.
105.     }
106. }
```

这里把程序设计中的几个关键地方做个说明：

- 第 12 行，声明一个函数指针。
- 第 13 行，这个要特别注意，H7 的系统 Bootloader 地址在 0x1FF09800。

- 第 19 到 21 行，设置滴答定时器到复位值。
- 第 24 行，此函数比较省事，可以方便的设置 H7 所有时钟到复位值，内部时钟使用 HSI。
- 第 27 到 31 行，清除所有中断挂起标志并关闭中断，这里是直接通过一个 for 循环设置了 NVIC 所有配置位，共 8 组。

Address	Name	Type	Required privilege	Reset value	Description
0xE000E100-0xE000E11C	NVIC_ISER0-NVIC_ISER7	RW	Privileged	0x00000000	*Interrupt set-enable registers on page 185*
0XE000E180-0xE000E19C	NVIC_ICER0-NVIC_ICER7	RW	Privileged	0x00000000	*Interrupt clear-enable registers on page 186*
0XE000E200-0xE000E21C	NVIC_ISPR0-NVIC_ISPR7	RW	Privileged	0x00000000	*Interrupt set-pending registers on page 186*
0XE000E280-0xE000E29C	NVIC_ICPR0-NVIC_ICPR7	RW	Privileged	0x00000000	*Interrupt clear-pending registers on page 187*
0xE000E300-0xE000E31C	NVIC_IABR0-NVIC_IABR7	RW	Privileged	0x00000000	*Interrupt active bit registers on page 188*
0xE000E400-0xE000E4EF	NVIC_IPR0-NVIC_IPR59	RW	Privileged	0x00000000	*Interrupt priority registers on page 188*
0xE000EF00	STIR	WO	Configurable(1)	0x00000000	*Software trigger interrupt register on page 189*

- 第 37 行，将系统 bootLoader 的中断复位服务程序的入口地址赋给第 12 行声明的函数，用户执行这个函数时，就会直接跳转过去。
- 第 40 行，设置主堆栈指针位置，即系统 bootloader 的首地址存储的就是栈地址。
- 第 43 行，这个设置在 RTOS 应用程序中比较重要，因为基于 Cortex-M 内核的 RTOS 任务堆栈基本都是使用线程堆栈指针 PSP。但系统 bootLoader 使用的是主堆栈指针 MSP，所以务必要设置下，同时让 M 内核工作于特权级。此寄存器的作用：

Table 4.3 Bit Fields in CONTROL Register

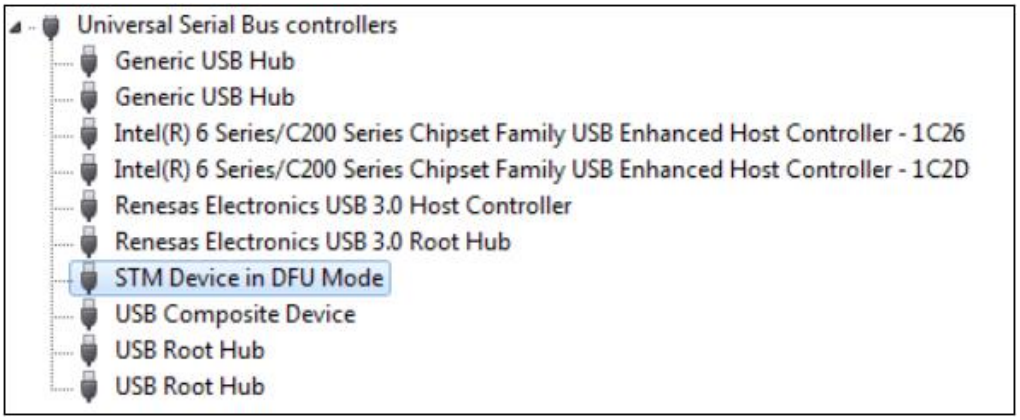
Bit	Function
nPRIV (bit 0)	Defines the privileged level in Thread mode: When this bit is 0 (default), it is privileged level when in Thread mode. When this bit is 1, it is unprivileged when in Thread mode. In Handler mode, the processor is always in privileged access level.
SPSEL (bit 1)	Defines the Stack Pointer selection: When this bit is 0 (default), Thread mode uses Main Stack Pointer (MSP). When this bit is 1, Thread mode uses Process Stack Pointer (PSP). In Handler mode, this bit is always 0 and write to this bit is ignored.
FPCA (bit 2)	Floating Point Context Active – This bit is only available in Cortex-M4 with floating point unit implemented. The exception handling mechanism uses this bit to determine if registers in the floating point unit need to be saved when an exception has occurred. When this bit is 0 (default), the floating point unit has not been used in the current context and therefore there is no need to save floating point registers. When this bit is 1, the current context has used floating point instructions and therefore need to save floating point registers. The FPCA bit is set automatically when a floating point instruction is executed. This bit is clear by hardware on exception entry. There are several options for handling saving of floating point registers. This will be covered in Chapter 13.

- 第 46 行，跳转到系统 bootLoader。

69.3 STM32CubeProg 的安装说明

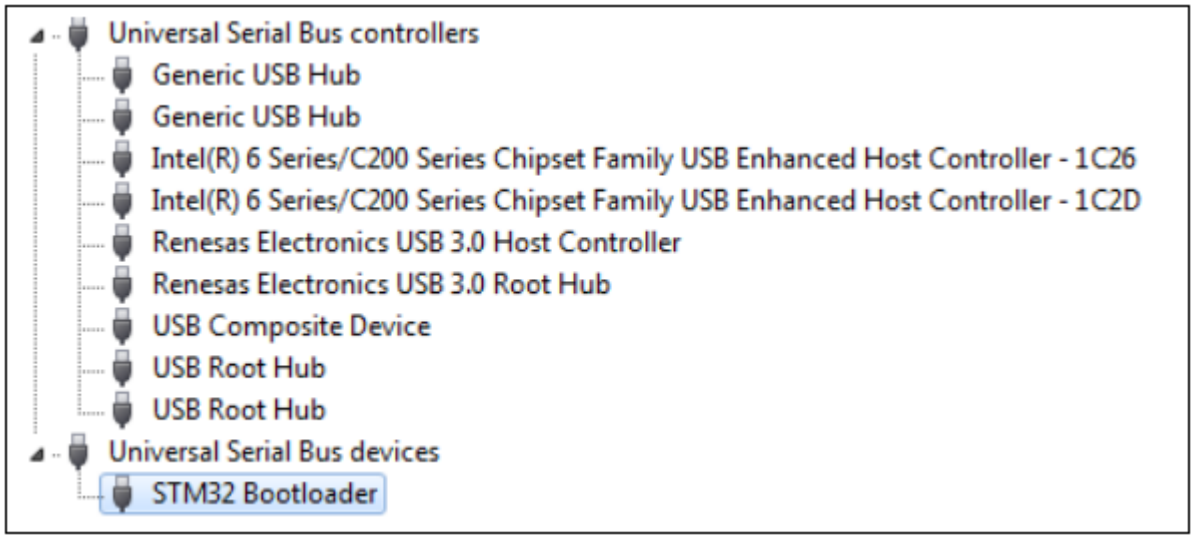
STM32CubeProg 的安装比较简单，如果大家的电脑中缺少 JAVA 环境，会提示安装，按照提示操作即可。

这里特别注意 USB DFU 驱动的安装，如果大家的电脑上安装了 DfuSe 软件，那边板子工作在系统 bootLoader 模式时，电脑端的设备管理器识别出来的标识是这样的：



如果用 STM32CubeProg 的话，务必要将此驱动删掉，鼠标右击此标识，选择卸载，弹出如下对话框：

卸载完毕后，重启电脑，然后运行 STM32CubeProg 安装目录里面的 STM32Bootloader.bat 即可，最后插上设备就可以正常识别了。识别后的标识：



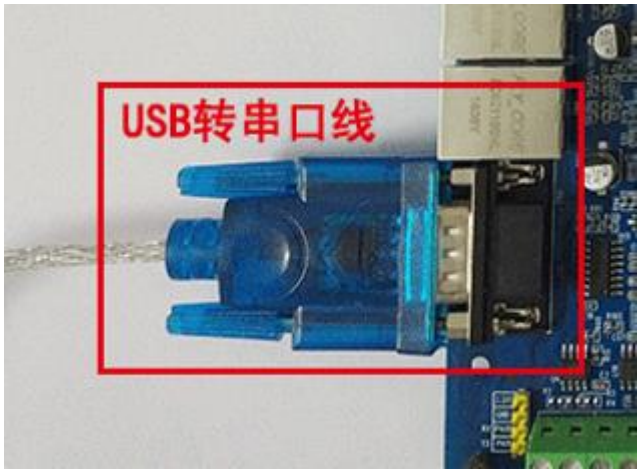
69.4 STM32CubeProg 的程序下载说明

这里把两种下载方式都做个说明，一种是设置外部 boot 引脚进行下载，另一种是设置程序跳转到系统 botloader 进行下载。

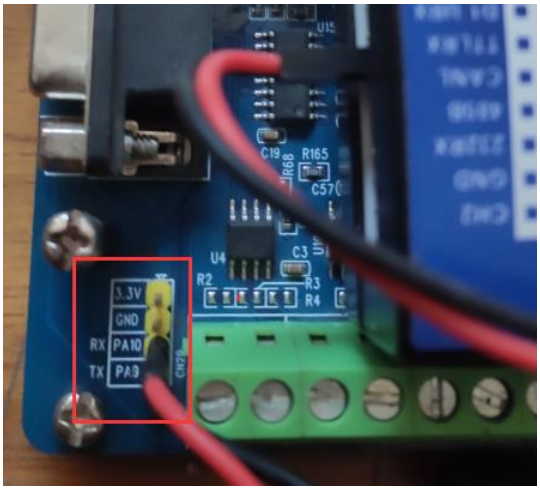
69.4.1 选择好用的串口线

（注：MicroUSB 接口不要接线到电脑端，因为这会导致系统 bootloader 工作在 USB DFU 模式，无法再使用串口 IAP）

选择好用的 USB 线很重要，比如我们开发板赠送的那根 USB 转 RS232 串口线是不可以用在这里做串口 IAP 的，这根线只能用于一般的串口通信和串口打印功能。



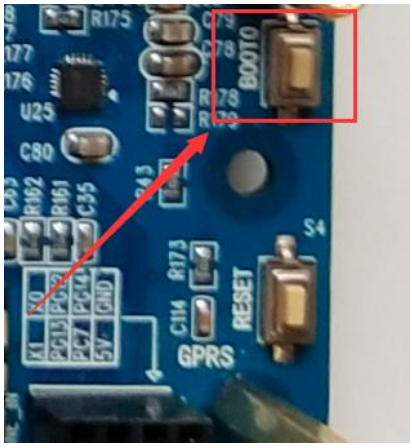
当前我这里是用的我们 H7-TOOL 的 USB 转 TTL 输出，注意交叉方式连接，即 RX 接 TX，TX 接 RX。GNG 接 GND。



注，我这里没有接共地线，推荐大家接上，3.3V 可以不接。

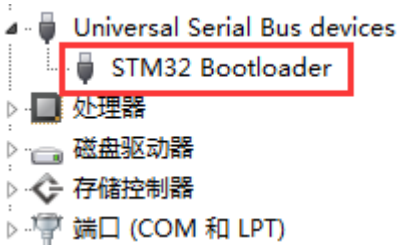
69.4.2 设置 boot 引脚跳转到系统 botLoader

- 第 1 步：板子上电前按住右下角的 BOOT 引脚。



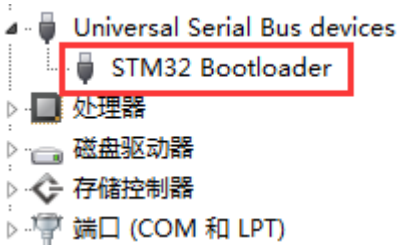
- 第 2 步：板子上电 3 秒左右，松手。

  在电脑端设备管理器就可以看到已经识别出来：



69.4.3 应用程序跳转到系统 bootloader

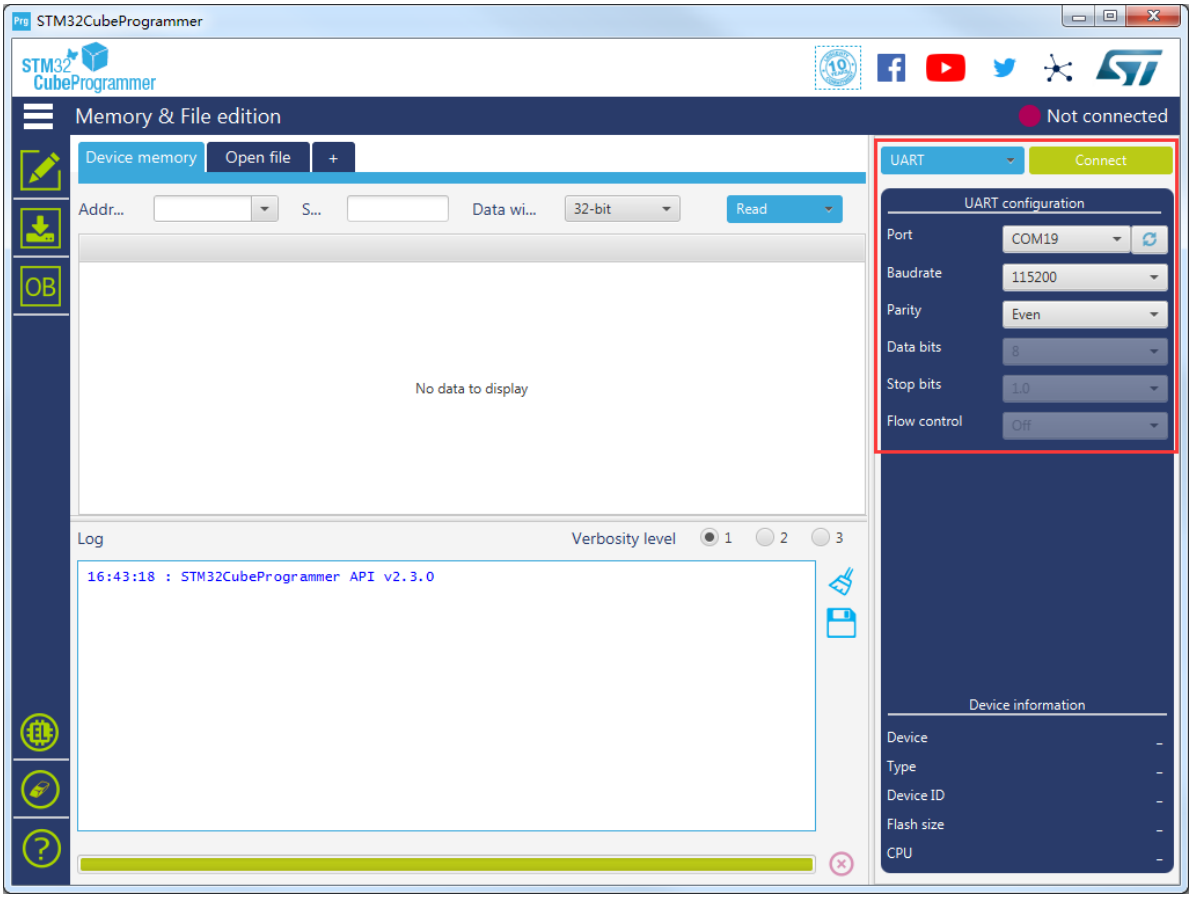
应用程序跳转到系统 bootLoader 比较方便，无需用户操作外置的 boot 引脚了，只需调用本章第 2 小节的程序就可以跳转。本章配套的例子是用户按下按键 K1 后执行跳转程序，大家可以根据需要实现各种触发跳转的方式。跳转成功后，在电脑端设备管理器里面也会看到 bootloader 标识：



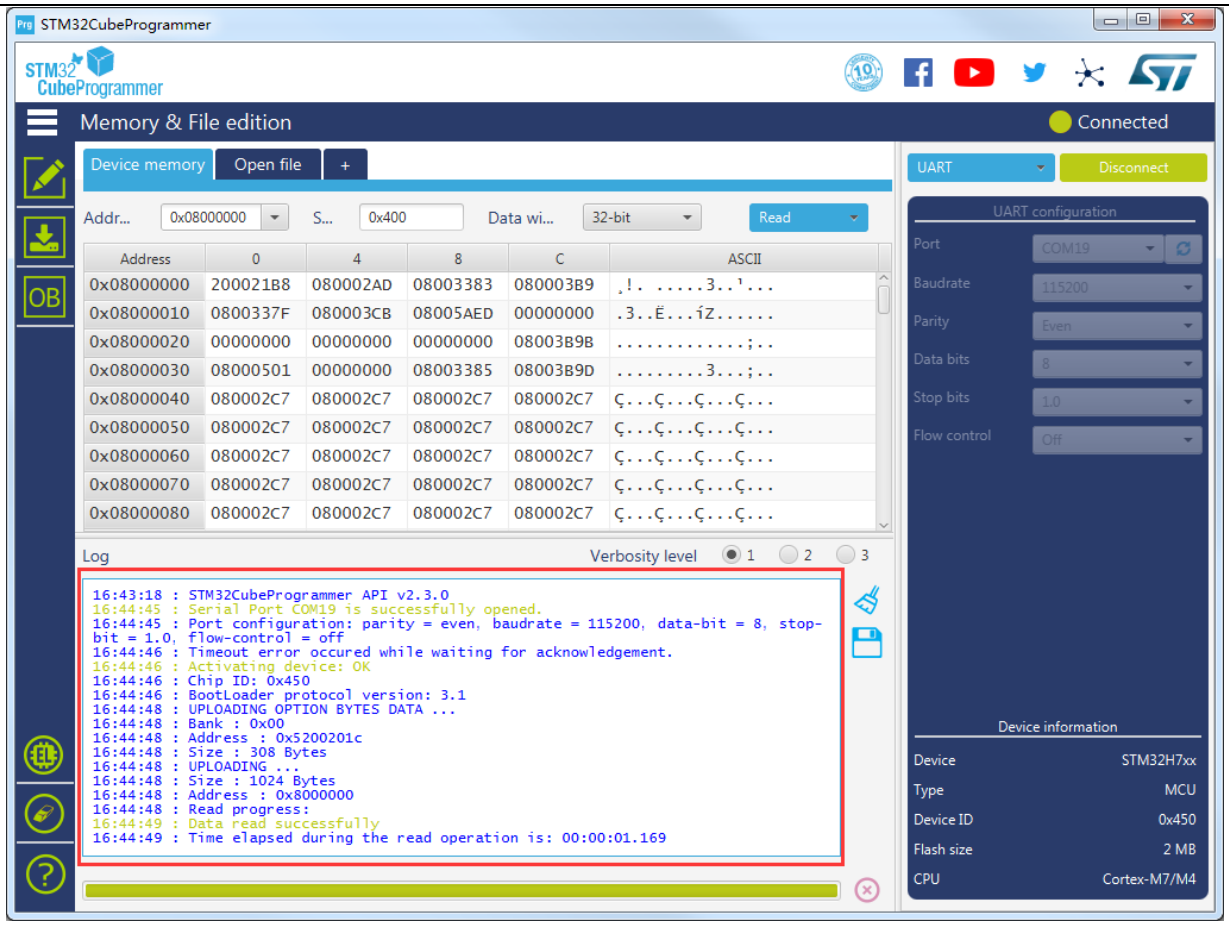
69.4.4 STM32CubeProg 下载程序设置

识别成功后就可以下载程序了。

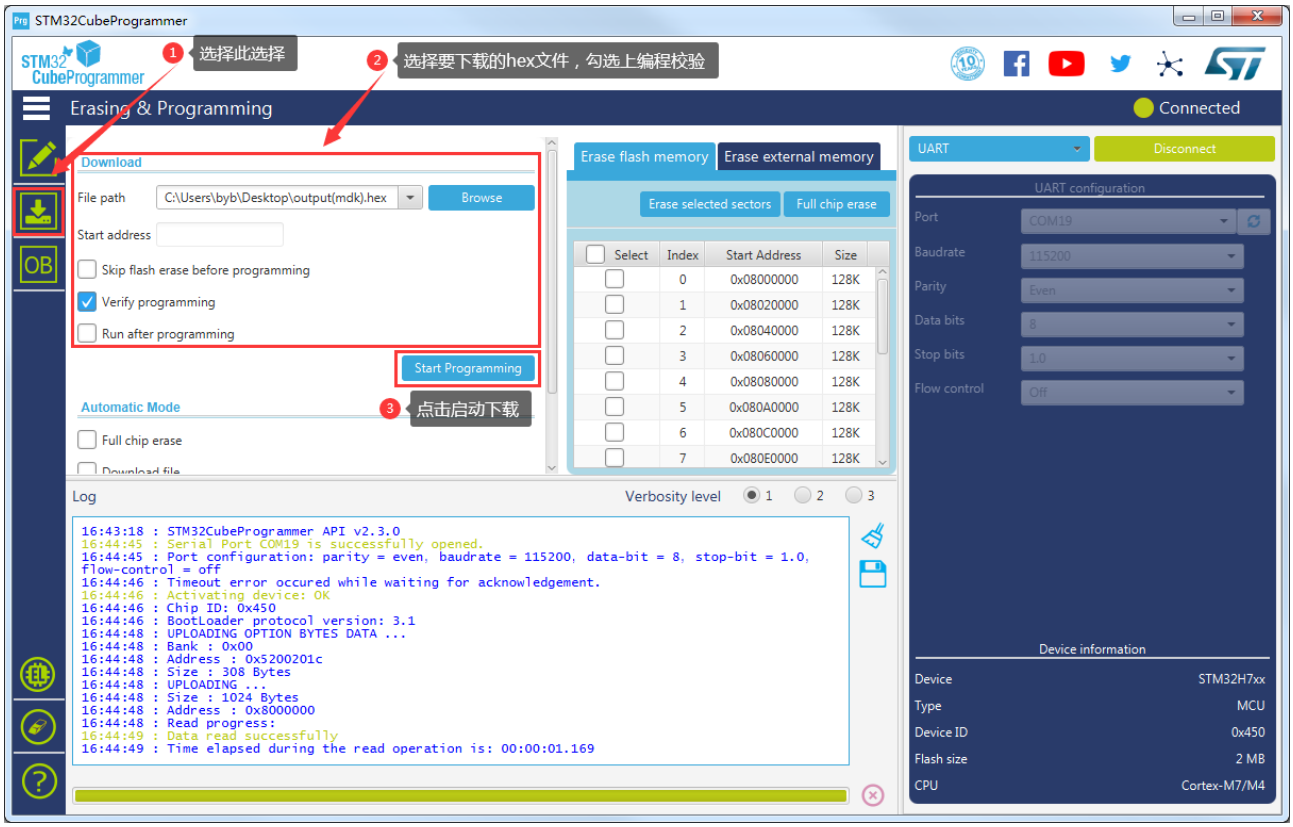
- 第 1 步，选择 UART 方式，配置使用的串口号，串口波特率 115200 和偶校验，点击 Connect 按钮。



识别成功后的效果如下：



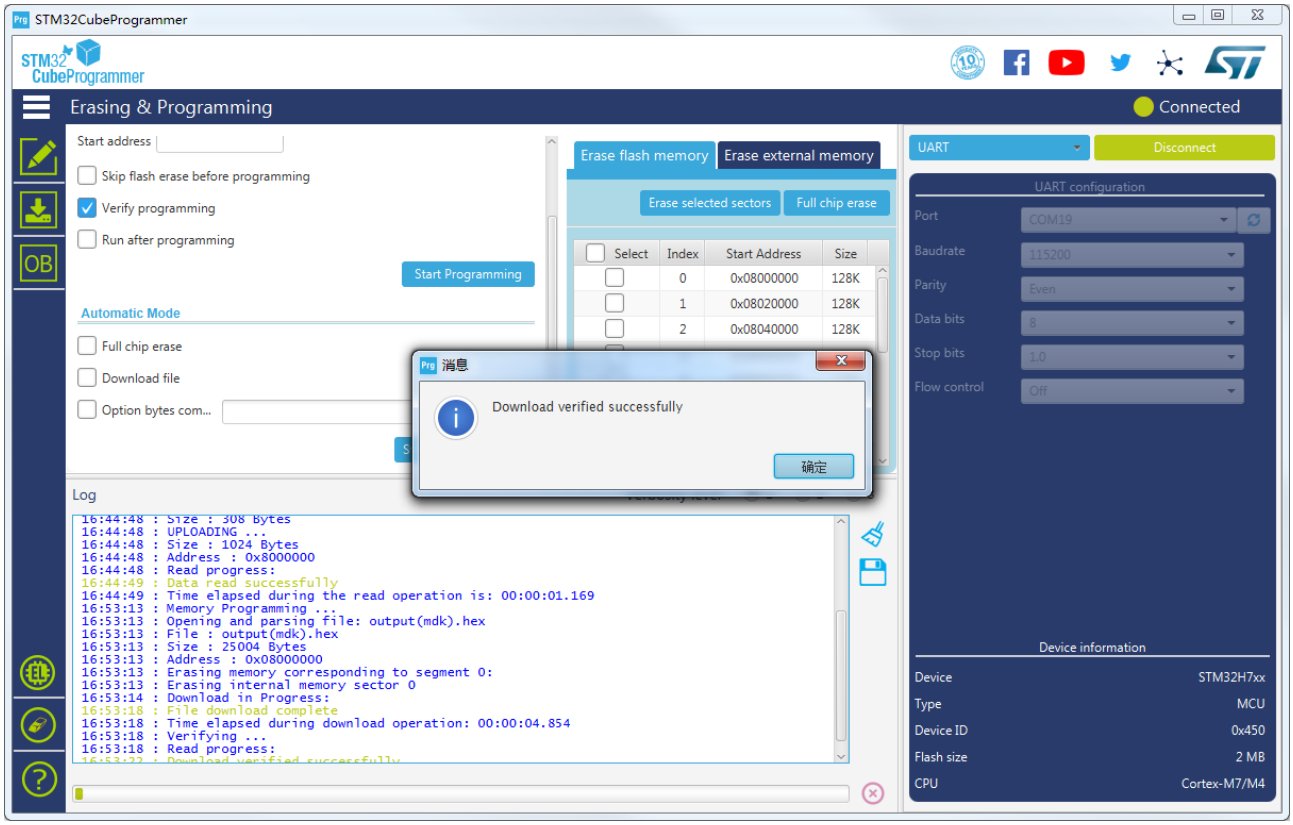
◆ 第 2 步，添加要下载的 hex 文件，勾选需要设置的选项，点击启动编程。



- Start address 选项不填的话，默认会下载到内部 Flash 的首地址，保险起见，大家也可以填上首地址 0x0800 0000，或者其它要下载的地址。
- Run after programming 选项可以根据需要勾上，如果勾上此选项后，下载完毕程序后，会自动断开连接，并弹出一些列窗口，最终弹出下面这个窗口：

弹出这个窗口并不是表示下载失败了，而是下载完成后退出了系统 bootloader，启动用刚刚下载的程序了。

◆ 第 3 步，完成下载后的效果如下：



下载完成后板子重新上电就可以看到程序已经成功下载了。

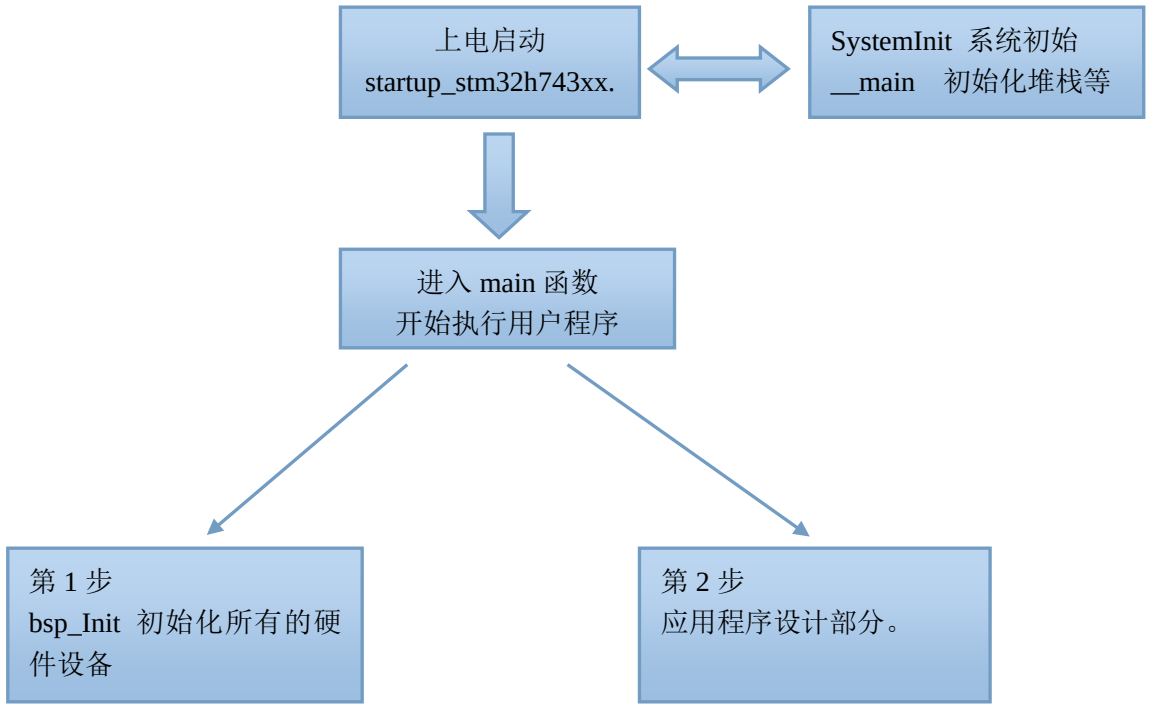
69.5 串口方式系统 Bootloader 驱动移植和使用

系统 bootloader 的移植比较简单，仅需添加本章第 2 小节的程序到自己工程里面即可。里面有个开关中断 API，是在 bsp.h 文件里面定义的：

```
/* 开关全局中断的宏 */
#define ENABLE_INT()    __set_PRIMASK(0)   /* 使能全局中断 */
#define DISABLE_INT()   __set_PRIMASK(1)   /* 禁止全局中断 */
```

69.6 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器和 LED。
  - 第 2 部分，应用程序设计部分，K1 按键按下后跳转到系统 bootloader。。

69.7 实验例程说明（MDK）

配套例子：

V7-048_基于系统 bootloader 的串口 IAP 方式固件升级

实验目的：

1. 学习基于系统 bootloader 的串口 IAP 方式固件升级。

实验内容：

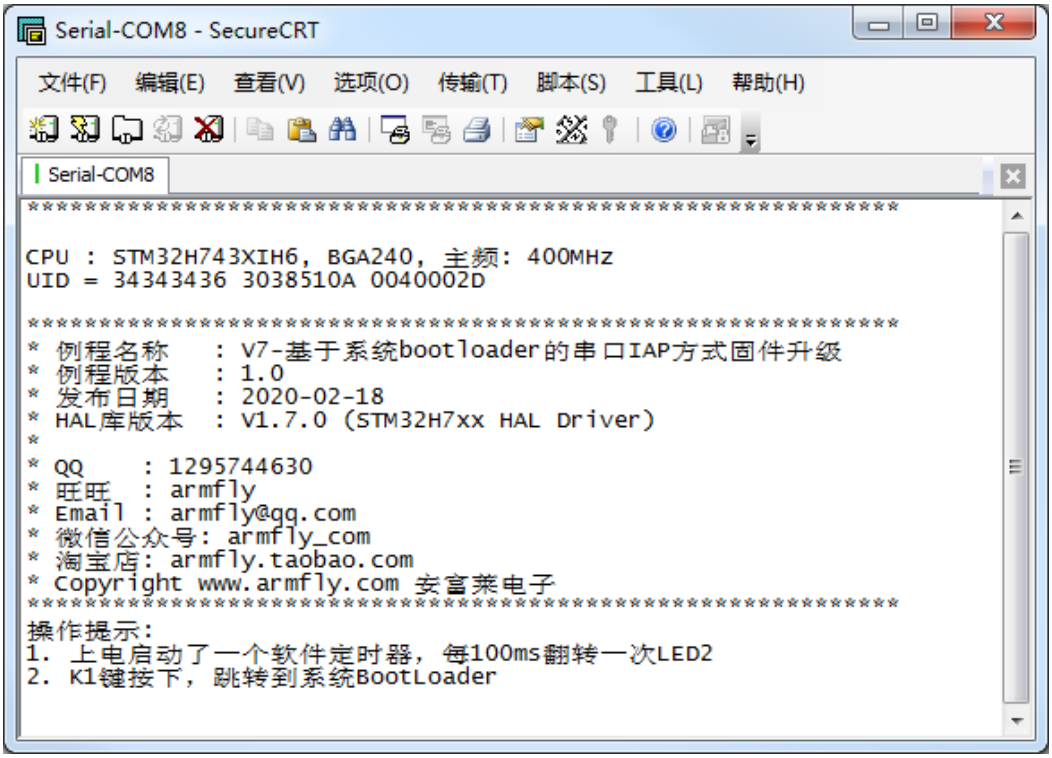
1. STM32 的系统存储区自带 bootLoader，可以方便的实现串口，I2C，CAN，SPI，USB 等接口方式的程序升级。
2. 如果使用系统 bootLoader 支持的接口升级方式，基本就不需要用户自己做 bootLoader 了。
3. 除了通过 boot 引脚控制启动地址，也可以直接从应用程序里面跳转到系统存储区。

实验操作：

1. K1 键按下，跳转到系统 bootLoader。

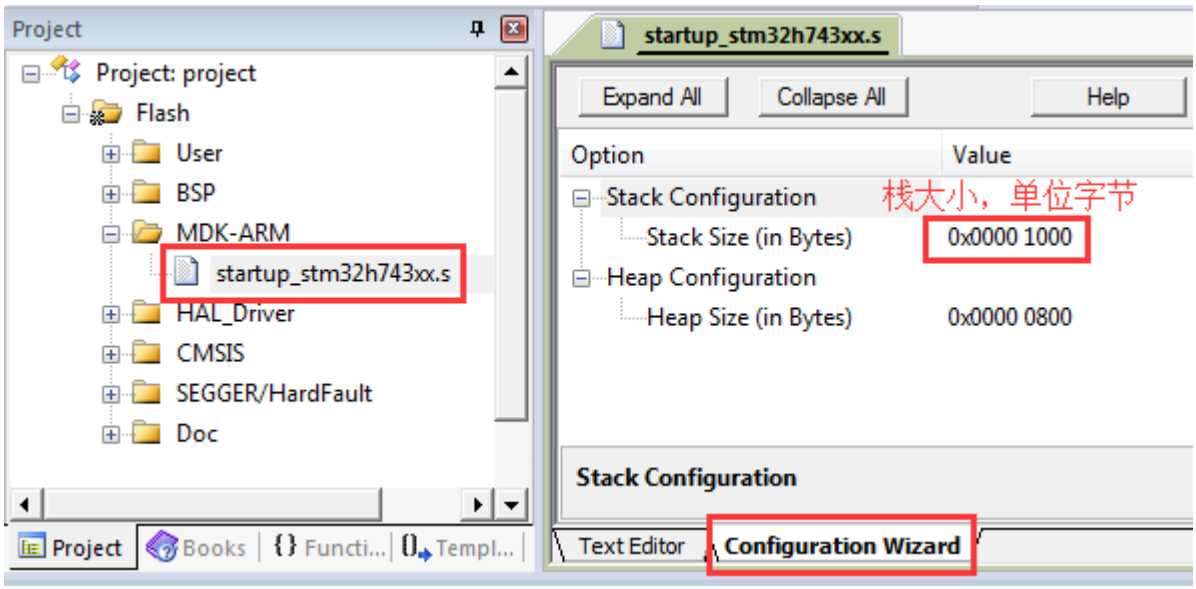
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1。

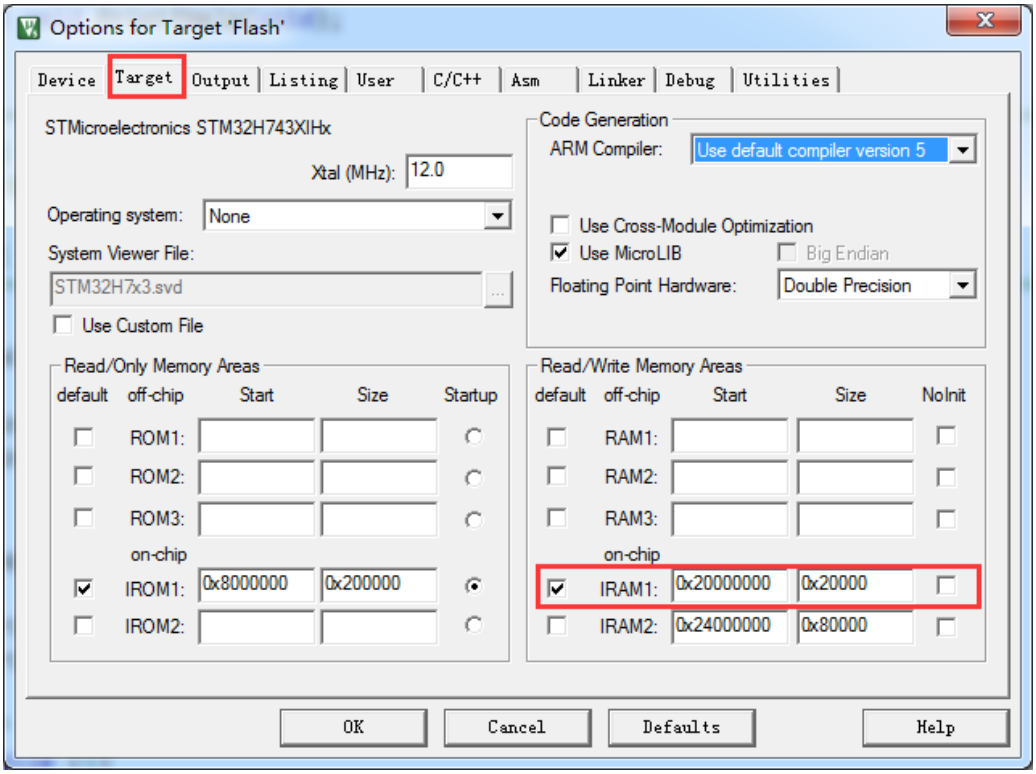


程序设计：

◆ **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
   bsp_InitExtSDRAM(); /* 初始化 SDRAM */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
```

```
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ 每 10ms 调用一次蜂鸣器处理：

蜂鸣器处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ 主功能：

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- K1 键按下，跳转到系统 BootLoader。

```
/*
*********************************************************************************************************
```

```
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode; /* 按键代码 */


    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，K1 键按下，跳转到系统 BootLoader */
                    JumpToBootloader();
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

69.8 实验例程说明（IAR）

配套例子：

V7-048_基于系统 bootloader 的串口 IAP 方式固件升级

实验目的：

1. 学习基于系统 bootloader 的 USB 接口方式 IAP 升级。

实验内容：

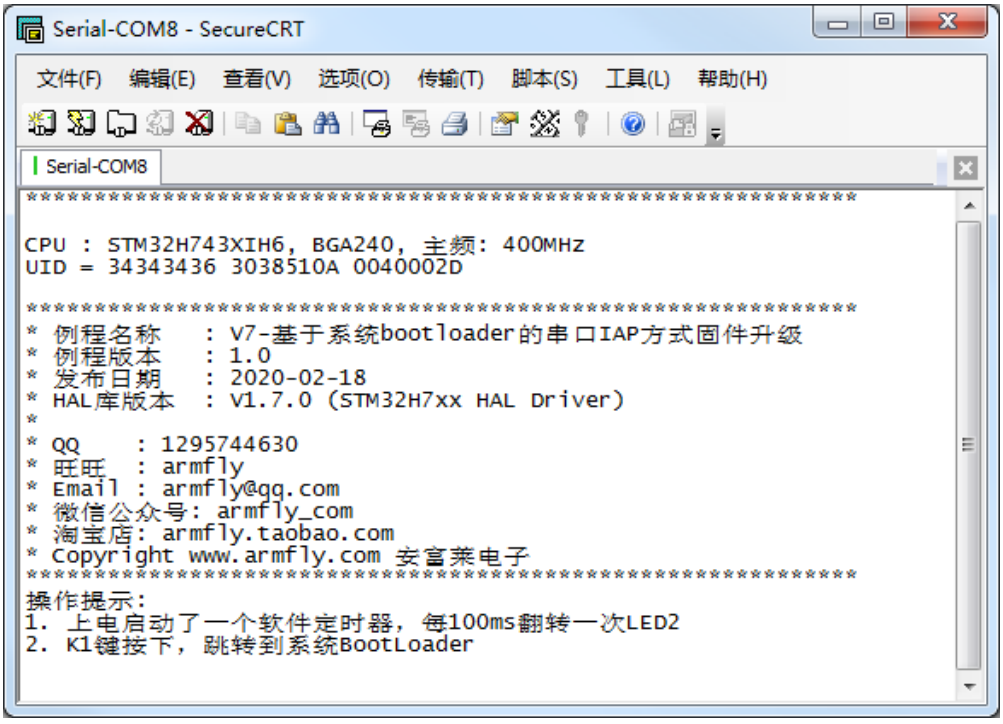
1. STM32 的系统存储区自带 bootLoader，可以方便的实现串口，I2C，CAN，SPI，USB 等接口方式的程序升级。
2. 如果使用系统 bootLoader 支持的接口升级方式，基本就不需要用户自己做 bootLoader 了。
3. 除了通过 boot 引脚控制启动地址，也可以直接从应用程序里面跳转到系统存储区。

实验操作：

1. K1 键按下，跳转到系统 bootLoader。

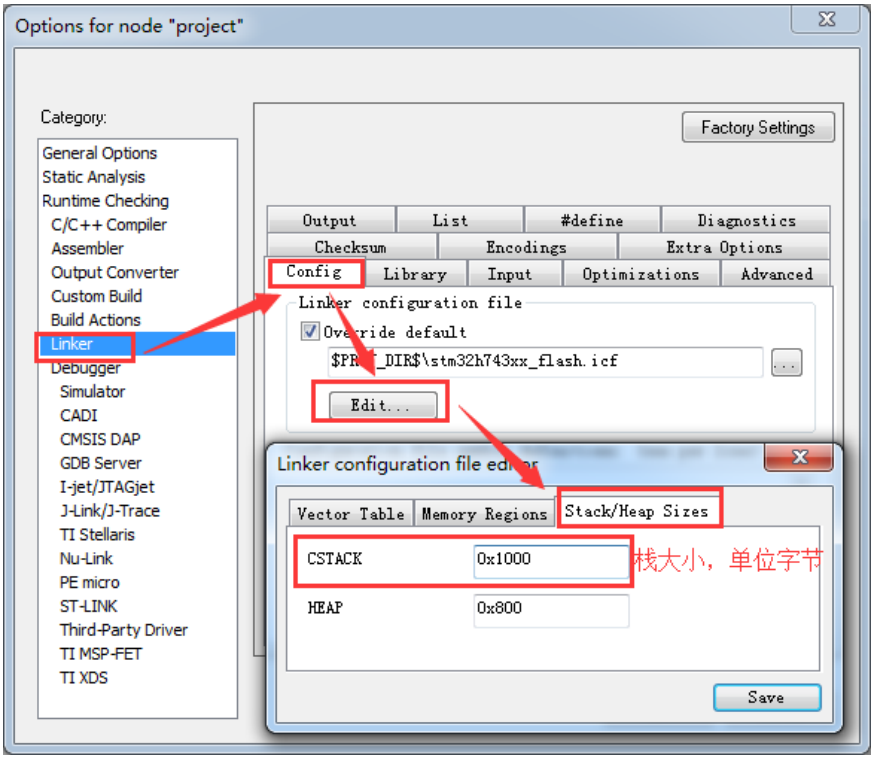
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1。

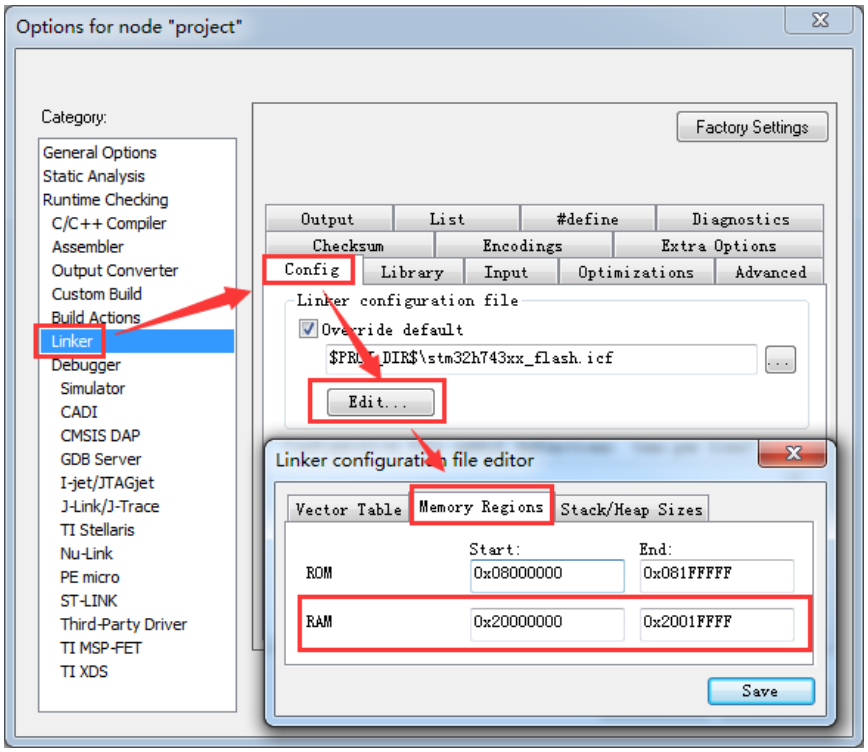


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
```

```
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();    /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
```

```
}
```

◆ **每 10ms 调用一次蜂鸣器处理：**

蜂鸣器处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能：**

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- K1 键按下，跳转到系统 BootLoader。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode; /* 按键代码 */


    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
```

```
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，K1 键按下，跳转到系统 BootLoader */
                    JumpToBootloader();
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

69.9 总结

本章节为大家介绍的串口 IAP 方式还是非常实用的，特别是产品硬件不带 boot 引脚时。

第70章 STM32H7 的内部 Flash 基础知识和 HAL 库 API

本章节为大家讲解内部 Flash 的基础知识和对应的 HAL 库 API。

70.1 初学者重要提示

70.2 内部 Flash 基础知识

70.3 内部 Flash 的 HAL 库用法

70.4 源文件 stm32h7xx_hal_flash.c

70.5 总结

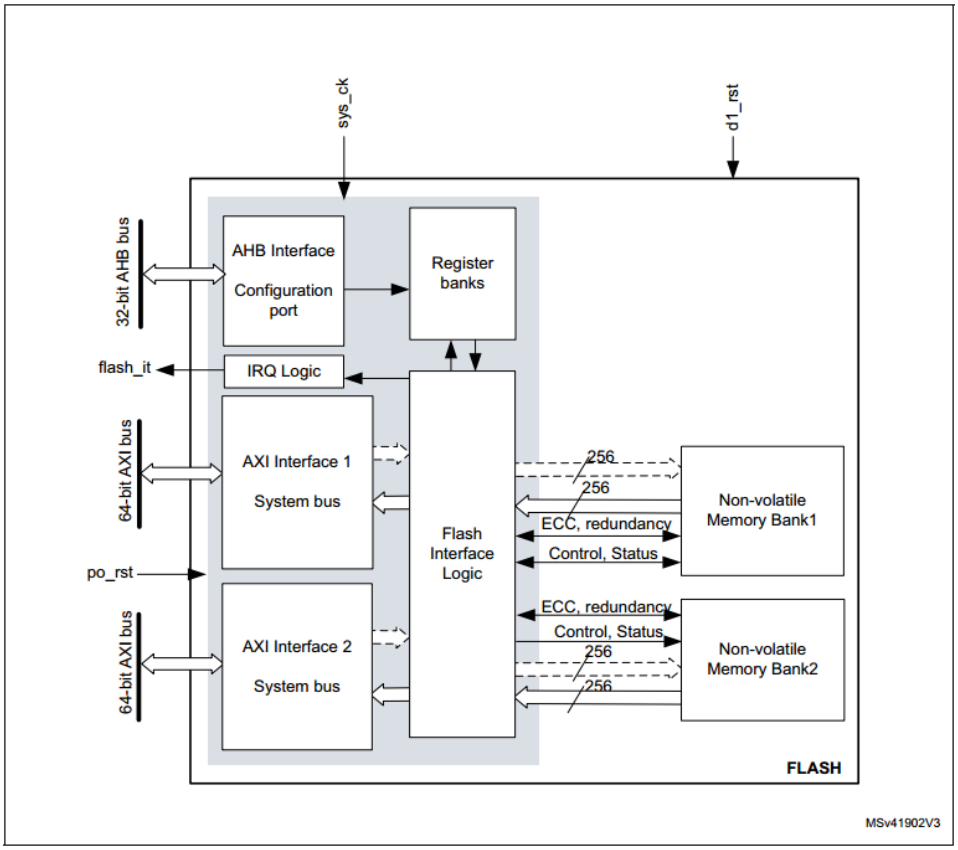
70.1 初学者重要提示

- 本章 2.5 小节里面的 Flash 三级读保护是重点，务必要掌握明白。
- STM32H743XI 有两个独立的 BANK,一个 BANK 的编程和擦除操作对另一个 BANK 没有任何影响。但是如果用户应用程序和要擦写的 Flash 扇区在同一个 BANK，在执行擦写操作时，用户应用程序将停止运行，包括中断服务程序。
- STM32H7 的两个 Flash BANK 是 256bit 带宽，CPU 访问是采用的两个 64bit AXI 总线。
- HAL 库的内部 Flash 编程函数 HAL_FLASH_Program 固定编写 32 字节数据。

70.2 内部 Flash 基础知识

70.2.1 内部 Flash 的硬件框图

认识一个外设，最好的方式就是看它的框图，方便我们快速的了解内部 Flash 的基本功能，然后再看手册了解细节。



通过这个框图，我们可以得到如下信息：

- sys_ck 时钟输入

  D1 域总线时钟。

- po_rst 输入

  Power on reset 上电复位。

- d1_rst 输入

  D1 域系统复位。

- flash_it 输出

  flash 中断请求输出。

STM32H7 的两个 Flash BANK 是独立的，读写和擦除互补影响，256bit 带宽，CPU 访问是采用的两个 64bit AXI 总线。

70.2.2 内部 Flash 框架

关于内部 Flash 的框架，了解以下几个知识点即可：

- 256bit 为单位，即 32 字节，并且每个单位配 10bit 的 ECC 校验位。正是这个原因要求大家对 Flash 进行编程时，必须以 32 字节为单位。

◆ 两个独立的 BANK，每个 BANK 有 1MB 容量。并且每个 BANK 的扇区大小固定为 128KB，即 8 个扇区。

Flash memory area		Address range	Size (bytes)	Region name	Access interface	SNB1/2 (1)
User main memory	Bank 1	0x0800 0000-0x0801 FFFF	128 K	Sector 0	AXI ports	0x0
		0x0802 0000-0x0803 FFFF	128 K	Sector 1		0x1
		...	...	...		...
		0x080E 0000-0x080F FFFF	128 K	Sector 7		0x7
	Bank 2	0x0810 0000-0x0811 FFFF	128 K	Sector 0		0x0
		0x0812 0000-0x0813 FFFF	128 K	Sector 1		0x1
		...	...	...		...
		0x081E 0000-0x081F FFFF	128 K	Sector 7		0x7
System memory	Bank 1	0x1FF0 0000-0x1FF1 FFFF	128 K	System Flash (read-only)	AXI ports	N/A(2)
	Bank 2	0x1FF4 0000-0x1FF5 FFFF	128 K	System Flash		N/A(2)
Option bytes		N/A	-	User option bytes	Registers only	N/A

BANK1 的地址范围：0x0800 0000 到 0x080F FFFF。

BANK2 的地址范围：0x0810 0000 到 0x081F FFFF。

70.2.3 内部 Flash 读操作

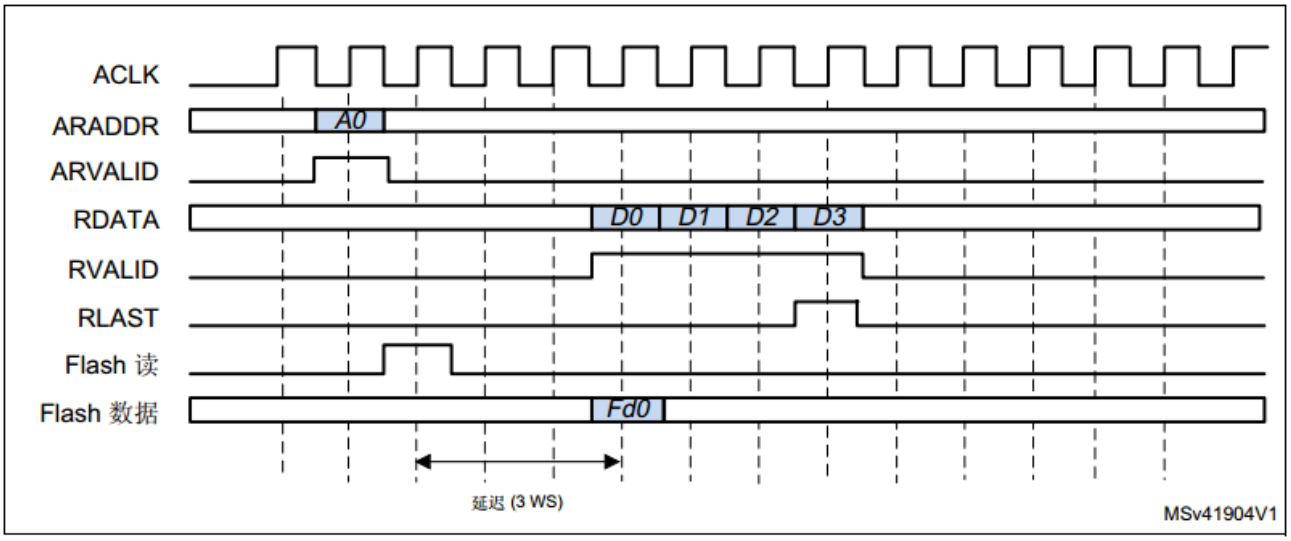
STM32H7 的内部 Flash 读操作跟内部 RAM 的读操作是一样的，支持 64bit，32bit，16bt 和 8bit，使用比较简单。这里我们重点普及一个知识点，H7 的内部 Flash 在不同主频下需要做的延迟参数：

Number of wait states (LATENCY)	Programming delay (WRHIGHFREQ)	AXI Interface clock frequency vs V_{CORE} range		
		VOS3 range 0.95 V - 1.05 V	VOS2 range 1.05 V - 1.15 V	VOS1 range 1.15 V - 1.26 V
0 WS (1 FLASH clock cycle)	00	[0 ; 45 MHz]	[0 ; 55 MHz]	[0 ; 70 MHz]
1 WS (2 FLASH clock cycles)	01	]45 MHz ; 90 MHz]	]55 MHz ; 110 MHz]	]70 MHz ; 140 MHz]
2 WS (3 FLASH clock cycles)	01	]90 MHz ; 135 MHz]	]110 MHz ; 165 MHz]	]140 MHz ; 185 MHz]
	10	-	-	]185 MHz ; 210 MHz]
3 WS (4 FLASH clock cycles)	10	]135 MHz ; 180 MHz]	]165 MHz ; 225 MHz]	]210 MHz ; 225 MHz]
4 WS (5 FLASH clock cycles)	10	]180 MHz ; 225 MHz]	225 MHz	-

对于上面的表格，大家可以看到，当延迟等待设置为 0 的时候，即无等待，单周期访问，速度可以做到 70MHz。增加 1 个 Flash 周期后，访问速度可以做到 140MHz。当增加到 3 个或 4 个 Flash 周期后，最高速度可以做到 225MHz。

了解了这个知识点后，再来看下面的时序，非常具有参考意义：

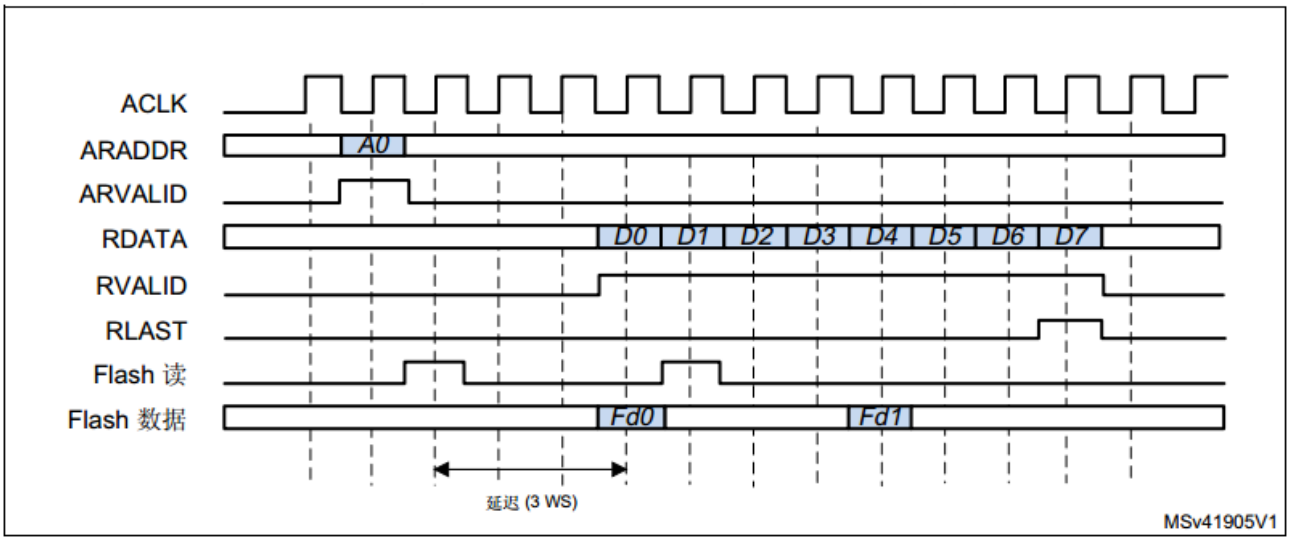
注：ACLK、ARADDR、ARVALID、RDATA、RVALID 和 RLAST 是 AXI 总线信号。Flash 读和 Flash 数据是 Flash 接口信号。



关于这个时序要要认识到以下几点：

- AXI 总线发起读取信号后，Flash 端等待了 3 个时钟周期（注意延迟三个周期，支持的 Flash 速度），之后连续读取了 4 个 64bit 数据。
- 由于 AXI 总线是 64bit 的，所以 1 次读取就可以读出 64bit 数据，连续读取 4 次后，就是 256bit，即 Flash 接口的一组数据，因为 H7 的 Flash 接口带宽是 256bit 的。
- 如果不开 Flash Cache 的情况下，连续读可以提升性能。

下面是连续读取 8 个 64bit 数据的时序图，跟连续读取 4 个 64bit 数据基本是一样的，只是多读取了 4 组数据。



70.2.4内部 Flash 写入和擦除操作

最重要的知识点放在开头说：STM32H7 内部 Flash 的写操作地址必须是 32 字节对齐（此地址对 32 求余数为 0），写入的数据量也必须是 32 字节整数倍，不足 32 字节整数倍，补 0 也要是整数倍。

这里我们重点了解 Flash 的写入和擦除流程。Flash 的写入扇区流程如下：

1. 先保证这块扇区空间之前已经擦除过了。
2. 解锁 Flash，通过 HAL 库的函数 HAL_FLASH_Unlock 实现。
3. 检查是否写保护，使能 Flash 可以编程，然后对其进行编程操作，编程完毕后，等待编程完成，然后禁止 Flash 编程位。具体操作可以通过 HAL 库的函数 HAL_FLASH_Program 实现。

Flash 的擦除流程如下：

1. 解锁 Flash，通过 HAL 库的函数 HAL_FLASH_Unlock 实现。
2. 如果是 BANK1 或者 BANK2 需要擦除，调用函数 FLASH_MassErase，然后等待擦除完成，完成之后关闭 BANK1 和 BANK2 的擦除请求位 BER1/BER2
3. 如果是扇区擦除，调用函数 FLASH_Erase_Sector，然后等待擦除完成，完成之后关闭扇区的擦除请求位 SER。

70.2.5内部 Flash 读保护

内部 Flash 支持三级读保护 RDP（read out protection）。

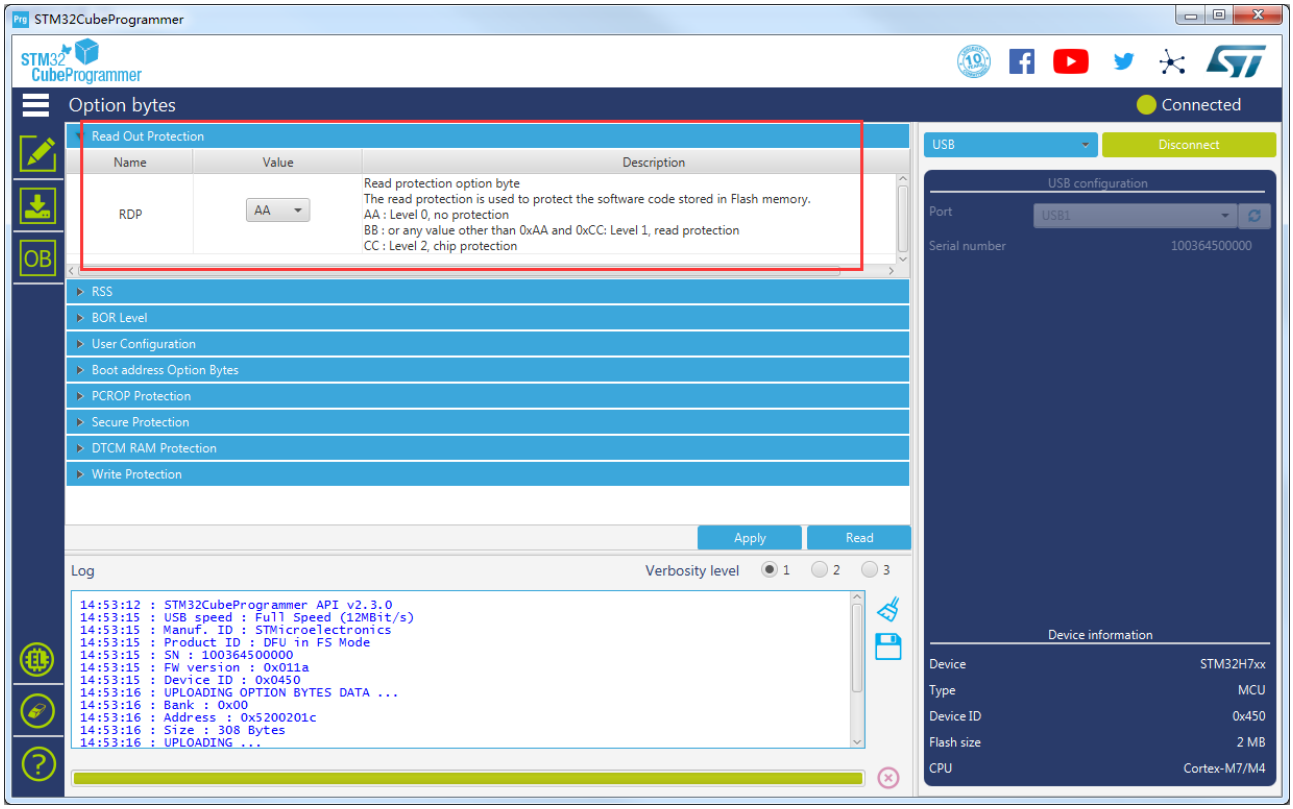
- Level 0（无保护）

默认设置，所有读写和擦除操作都可以正常支持。

- Level 1 （Flash 连接保护）
  - 可以防止连接调试器时读取 Flash 内容，或者 RAM 中存有恶意获取代码，也是禁止的。因此只要调试器连接芯片，或者从内部 RAM 启动运行代码，都是禁止访问内部 Flash 的。
  - 如果没有检测到从内部 RAM 启动和系统 bootloader 启动且没有连接调试器，对用户 Flash 的读写和擦除操作都是允许的，并且其它安全存储区也是可以访问的。否则是禁止访问的，一旦检测到对 Flash 的读请求，将产生总线错误。
  - 如果将 Level 1 切换到 Level 0 时，用户 Flash 区和安全区域将被擦除。
- Level 2（设备保护和自举保护）
  - 所有调试特性被关闭。
  - 禁止从 RAM 启动。
  - 除了选项字节里面的 SWAP 位可以配置，其它位都无法更改。
  - 禁止了调试功能，且禁止了从 RAM 和系统 bootloader 启动，用户 Flash 区是可以执行读写和擦除操作的，访问其它安全存储区也是可以的。

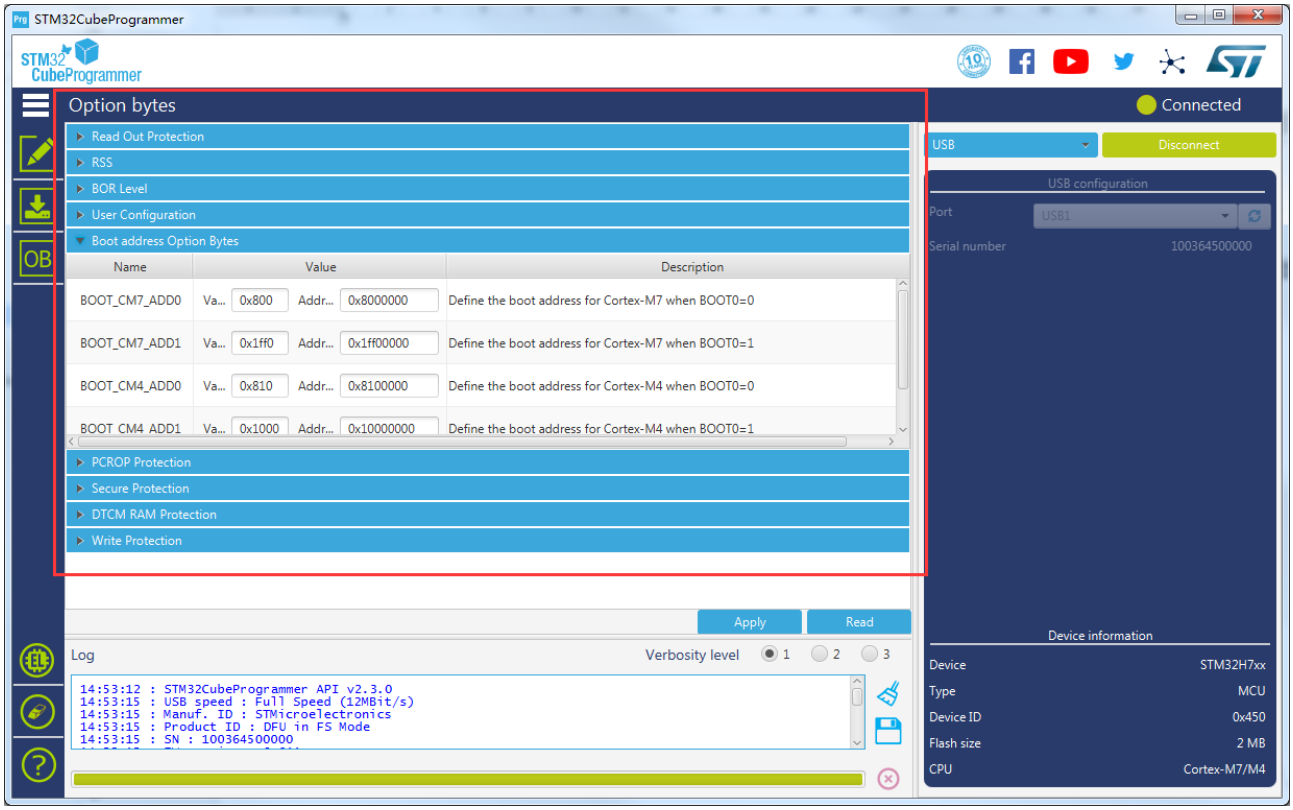
特别注意：Level2 修改是永久性的，一旦配置为 Level2 将不再支持被修改。

如果大家要设置读保护的话，使用 HAL 的 API 可以设置，也可以使用 STM32CubeProg 设置：



70.2.6内部 Flash 选项字节

Flash 选项字节主要用于 boot 地址设置，安全保护，Flash 扇区保护等，涉及到的选项比较多。如果大家打算了解这一部分的话，使用 STM32CubeProg 进行设置即可，也比较方便。



70.2.7内部 Flash 的 ECC 校验

这里先说下为什么内部 Flash 要带 ECC 校验，因为随着芯片的制造工艺水平越高，带电粒子产生的位翻转就越多，此时的 ECC 是必须要有的，一般可以纠正 1-2 个 bit，安全等级高的 Flash 类存储器和 RAM 类都是必须要带 ECC 的。

对于 STM32H7 带的 ECC 校验，一般不需要用户去管理。

- ECC 相关知识

关于 ECC 方面的知识，专门整理了一个帖子：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=86777

70.3 内部 Flash 的 HAL 库用法

70.3.1 内部 Flash 结构体 FLASH_TypeDef

内部 Flash 相关的寄存器是通过 HAL 库中的结构体 FLASH_TypeDef 定义的，在 stm32h743xx.h 中可以找到这个类型定义：

```
typedef struct
{
  __IO uint32_t ACR;
  __IO uint32_t KEYR1;
  __IO uint32_t OPTKEYR;
  __IO uint32_t CR1;
  __IO uint32_t SR1;
  __IO uint32_t CCR1;
  __IO uint32_t OPTCR;
  __IO uint32_t OPTSR_CUR;
  __IO uint32_t OPTSR_PRG;
  __IO uint32_t OPTCCR;
  __IO uint32_t PRAR_CUR1;
  __IO uint32_t PRAR_PRG1;
  __IO uint32_t SCAR_CUR1;
  __IO uint32_t SCAR_PRG1;
  __IO uint32_t WPSN_CUR1;
  __IO uint32_t WPSN_PRG1;
  __IO uint32_t BOOT_CUR;
  __IO uint32_t BOOT_PRG;
  uint32_t      RESERVED0[2];
  __IO uint32_t CRCCR1;
  __IO uint32_t CRCSADD1;
  __IO uint32_t CRCEADD1;
  __IO uint32_t CRCDATA;
  __IO uint32_t ECC_FA1;
  uint32_t      RESERVED1[40];
  __IO uint32_t KEYR2;
  uint32_t      RESERVED2;
  __IO uint32_t CR2;
  __IO uint32_t SR2;
  __IO uint32_t CCR2;
  uint32_t      RESERVED3[4];
  __IO uint32_t PRAR_CUR2;
  __IO uint32_t PRAR_PRG2;
  __IO uint32_t SCAR_CUR2;
  __IO uint32_t SCAR_PRG2;
  __IO uint32_t WPSN_CUR2;
  __IO uint32_t WPSN_PRG2;
  uint32_t      RESERVED4[4];
  __IO uint32_t CRCCR2;
  __IO uint32_t CRCSADD2;
  __IO uint32_t CRCEADD2;
  __IO uint32_t CRCDATA2;
  __IO uint32_t ECC_FA2;
} FLASH_TypeDef;
```

这个结构体的成员名称和排列次序和 CPU 的寄存器是一 一对应的。

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

下面我们看下 Flash 的定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE              (0x40000000UL)
#define D1_AHB1PERIPH_BASE       (PERIPH_BASE + 0x12000000UL)
#define FLASH_R_BASE             (D1_AHB1PERIPH_BASE + 0x2000UL)
#define FLASH                    ((FLASH_TypeDef *) FLASH_R_BASE) <----- 展开这个宏，(FLASH_TypeDef *)0x52002000
```

我们访问 Flash 的 CR1 寄存器可以采用这种形式：FLASH->CR1 = 0。

70.3.2 内部 Flash 擦除结构体 FLASH_EraseInitTypeDef

下面是做内部 Flash 擦除的结构体，用到的地方比较多：

```
typedef struct
{
  uint32_t TypeErase;
  uint32_t Banks;
  uint32_t Sector;
  uint32_t NbSectors;
  uint32_t VoltageRange;
} FLASH_EraseInitTypeDef;
```

下面将结构体成员逐一做个说明：

- TypeErase

  用于选择 BANK 擦除还是扇区擦除，H743 有两个 BANK，每个 BANK 有个 8 个扇区，每个扇区 128KB。

  具体支持的参数如下：

```
#define FLASH_TYPEERASE_SECTORS       0x00U  /* 扇区方式擦除 */
#define FLASH_TYPEERASE_MASSERASE     0x01U  /* BANK 方式擦除 */
```

- Banks

  用于选择要擦除的 BANK，或者两个 BANK 都选择：

```
#define FLASH_BANK_1              0x01U                         /* Bank 1 */
#define FLASH_BANK_2              0x02U                         /* Bank 2 */
#define FLASH_BANK_BOTH           (FLASH_BANK_1 | FLASH_BANK_2) /* Bank1 和 Bank2 */
```

- Sector

  用于选择要擦除的扇区：

```
#define FLASH_SECTOR_0            0U       /* Sector Number 0   */
#define FLASH_SECTOR_1            1U       /* Sector Number 1   */
#define FLASH_SECTOR_2            2U       /* Sector Number 2   */
#define FLASH_SECTOR_3            3U       /* Sector Number 3   */
#define FLASH_SECTOR_4            4U       /* Sector Number 4   */
#define FLASH_SECTOR_5            5U       /* Sector Number 5   */
#define FLASH_SECTOR_6            6U       /* Sector Number 6   */
#define FLASH_SECTOR_7            7U       /* Sector Number 7   */
```

- NbSectors

用于设置要擦除的扇区个数，对于 STM32H743 来说，范围 1 到 8。

- VoltageRange

用于设置编程的并行位数，电压不同，位数不同：

```
#define FLASH_VOLTAGE_RANGE_1        0x00000000U        /* Flash program/erase by 8 bits  */
#define FLASH_VOLTAGE_RANGE_2        FLASH_CR_PSIZE_0   /* Flash program/erase by 16 bits */
#define FLASH_VOLTAGE_RANGE_3        FLASH_CR_PSIZE_1   /* Flash program/erase by 32 bits */
#define FLASH_VOLTAGE_RANGE_4        FLASH_CR_PSIZE     /* Flash program/erase by 64 bits */
```

70.3.3 内部 Flash 的操作总结

使用方法由 HAL 库提供：

- Flash 编程函数操作流程
  - Flash 解锁函数 HAL_FLASH_Unlock。
  - Flash 查询方式编程 HAL_FLASH_Program。
  - Flash 中断方式编程 HAL_FLASH_Program_IT。
  - Flash 上锁函数 HAL_FLASH_Lock。
- 选项字节编程流程
  - 选项字节解锁函数 HAL_FLASH_OB_Unlock。
  - 选项字节加载函数 HAL_FLASH_OB_Launch。
  - 选项字节编程函数 HAL_FLASHEx_OBProgram。
  - 选项字节加锁函数 HAL_FLASH_OB_Lock。
- Flash 擦除流程
  - Flash 解锁函数 HAL_FLASH_Unlock。
  - Flash 查询方式擦除 HAL_FLASHEx_Erase。
  - Flash 中断方式擦除 HAL_FLASHEx_Erase_IT。
  - Flash 上锁函数 HAL_FLASH_Lock。

70.4 内部 Flash 源文件 stm32h7xx_hal_flash.c

此文件涉及到的函数较多，这里把几个常用的函数做个说明：

- HAL_FLASH_Unlock
- HAL_FLASH_Lock
- HAL_FLASHEx_Erase
- HAL_FLASH_Program

70.4.1 函数 HAL_FLASH_Lock

函数原型：

```
HAL_StatusTypeDef HAL_FLASH_Lock(void)
{
  /* 设置 FLASH Bank1 控制寄存器 Lock 位，即禁止访问 */
  SET_BIT(FLASH->CR1, FLASH_CR_LOCK);

  /* 验证 Flash Bank1 是否已经被锁住 */
  if (READ_BIT(FLASH->CR1, FLASH_CR_LOCK) == 0U)
  {
    return HAL_ERROR;
  }

  /* 设置 FLASH Bank2 控制寄存器 Lock 位，即禁止访问 */
  SET_BIT(FLASH->CR2, FLASH_CR_LOCK);

   /* 验证 Flash Bank2 是否已经被锁住 */
  if (READ_BIT(FLASH->CR2, FLASH_CR_LOCK) == 0U)
  {
    return HAL_ERROR;
  }

  return HAL_OK;
}
```

函数描述：

用于 Flash 加锁，加锁后将不能对 Flash 进行编程和擦除。

70.4.2 函数 HAL_FLASH_Unlock

函数原型：

```
HAL_StatusTypeDef HAL_FLASH_Unlock(void)
{
  if(READ_BIT(FLASH->CR1, FLASH_CR_LOCK) != 0U)
  {
    /* 允许访问 Flash Bank1 */
    WRITE_REG(FLASH->KEYR1, FLASH_KEY1);
    WRITE_REG(FLASH->KEYR1, FLASH_KEY2);

    /* 验证是否已经解锁 */
    if (READ_BIT(FLASH->CR1, FLASH_CR_LOCK) != 0U)
    {
      return HAL_ERROR;
    }
  }

  if(READ_BIT(FLASH->CR2, FLASH_CR_LOCK) != 0U)
  {
    /* 允许访问 Flash Bank2 */
    WRITE_REG(FLASH->KEYR2, FLASH_KEY1);
    WRITE_REG(FLASH->KEYR2, FLASH_KEY2);
```

```
    /* 验证是否已经解锁 */
    if (READ_BIT(FLASH->CR2, FLASH_CR_LOCK) != 0U)
    {
      return HAL_ERROR;
    }
  }

  return HAL_OK;
}
```

函数描述：

此函数用于 Flash 解锁，解锁后可以对 Flash 进行擦除和编程。

70.4.3 函数 HAL_FLASH_Program

函数原型：

```
HAL_StatusTypeDef HAL_FLASH_Program(uint32_t TypeProgram, uint32_t FlashAddress, uint32_t DataAddress)
{
  HAL_StatusTypeDef status;
  __IO uint32_t *dest_addr = (__IO uint32_t *)FlashAddress;
  __IO uint32_t *src_addr = (__IO uint32_t*)DataAddress;
  uint32_t bank;
  uint8_t row_index = FLASH_NB_32BITWORD_IN_FLASHWORD;

  /* 检测参数 */
  assert_param(IS_FLASH_TYPEPROGRAM(TypeProgram));
  assert_param(IS_FLASH_PROGRAM_ADDRESS(FlashAddress));

  /* 上锁 */
  __HAL_LOCK(&pFlash);

#if defined (FLASH_OPTCR_PG_OTP)
  if((IS_FLASH_PROGRAM_ADDRESS_BANK1(FlashAddress)) || (IS_FLASH_PROGRAM_ADDRESS_OTP(FlashAddress)))
#else
  if(IS_FLASH_PROGRAM_ADDRESS_BANK1(FlashAddress))
#endif
  {
    bank = FLASH_BANK_1;
  }
  else
  {
    bank = FLASH_BANK_2;
  }

  /* 错误标识，无错误 */
  pFlash.ErrorCode = HAL_FLASH_ERROR_NONE;

  /* 等待操作完成 */
  status = FLASH_WaitForLastOperation((uint32_t)FLASH_TIMEOUT_VALUE, bank);

  if(status == HAL_OK)
  {
    if(bank == FLASH_BANK_1)
    {
```

```
#if defined (FLASH_OPTCR_PG_OTP)
      if (TypeProgram == FLASH_TYPEPROGRAM_OTPWORD)
      {
        /* 设置 OTP 寄存器的 PG 位，使能可以编程 */
        SET_BIT(FLASH->OPTCR, FLASH_OPTCR_PG_OTP);
      }
      else
#endif
      {
        /* 设置 PG 位，使能可编程 */
        SET_BIT(FLASH->CR1, FLASH_CR_PG);
      }
    }
    else
    {
      /* 设置 PG 位 */
      SET_BIT(FLASH->CR2, FLASH_CR_PG);
    }

    __ISB();
    __DSB();

#if defined (FLASH_OPTCR_PG_OTP)
    if (TypeProgram == FLASH_TYPEPROGRAM_OTPWORD)
    {
      /* 编程 OTP(16 bits) */
      *(__IO uint16_t *)FlashAddress = *(__IO uint16_t*)DataAddress;
    }
    else
#endif
    {
      /* 对 Flash 进行编程 */
      do
      {
        *dest_addr = *src_addr;
        dest_addr++;
        src_addr++;
        row_index--;
     } while (row_index != 0U);
    }

    __ISB();
    __DSB();

    /* 等待最后一次操作完成 */
    status = FLASH_WaitForLastOperation((uint32_t)FLASH_TIMEOUT_VALUE, bank);

#if defined (FLASH_OPTCR_PG_OTP)
    if (TypeProgram == FLASH_TYPEPROGRAM_OTPWORD)
    {
      /* 如果编程操作完成，关闭 OTP PG 位 */
      CLEAR_BIT(FLASH->OPTCR, FLASH_OPTCR_PG_OTP);
    }
    else
#endif
    {
      if(bank == FLASH_BANK_1)
      {
```

```
        /* 如果操作完成，关闭 PG 位 */
        CLEAR_BIT(FLASH->CR1, FLASH_CR_PG);
      }
      else
      {
        /* 如果操作完成，关闭 PG 位 */
        CLEAR_BIT(FLASH->CR2, FLASH_CR_PG);
      }
    }
  }

  /* 解锁 */
  __HAL_UNLOCK(&pFlash);

  return status;
}
```

函数描述：

此函数主要用于 Flash 编程，固定编程 32 个字节数据。

函数参数：

- 第 1 个参数是要编程的 Flash 类型，支持两种参数：
  - FLASH_TYPEPROGRAM_FLASHWORD，用于芯片内部 Flash 编程。
  - FLASH_TYPEPROGRAM_OTPWORD，用于芯片内部 OTP 存储区编程，当前的 H743 并没有这个区域，所以可以忽略。
- 第 2 个参数是要编程的 Flash 地址。
- 第 3 个参数是要编程到 Flash 的数据地址。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

注意事项：

- 第 2 个参数的 Flash 地址要是 32 字节对齐的，即此地址对 32 求余等于 0。
- 第 3 个参数务必要是 32 字节的整数倍。

70.4.4函数 HAL_FLASHEx_Erase

函数原型：

```
HAL_StatusTypeDef HAL_FLASHEx_Erase(FLASH_EraseInitTypeDef *pEraseInit, uint32_t *SectorError)
{
  HAL_StatusTypeDef status = HAL_OK;
  uint32_t sector_index;

  /* 检查参数 */
  assert_param(IS_FLASH_TYPEERASE(pEraseInit->TypeErase));
  assert_param(IS_FLASH_BANK(pEraseInit->Banks));

  /* 上锁 */
  __HAL_LOCK(&pFlash);
```

```
/* 无错误 */
pFlash.ErrorCode = HAL_FLASH_ERROR_NONE;

 /* 等待 BANK1 的操作完成 */
if((pEraseInit->Banks & FLASH_BANK_1) == FLASH_BANK_1)
{
  if(FLASH_WaitForLastOperation((uint32_t)FLASH_TIMEOUT_VALUE, FLASH_BANK_1) != HAL_OK)
  {
    status = HAL_ERROR;
  }
}

/* 等待 BANK2 的操作完成 */
if((pEraseInit->Banks & FLASH_BANK_2) == FLASH_BANK_2)
{
  if(FLASH_WaitForLastOperation((uint32_t)FLASH_TIMEOUT_VALUE, FLASH_BANK_2) != HAL_OK)
  {
    status = HAL_ERROR;
  }
}

if(status == HAL_OK)
{
  if(pEraseInit->TypeErase == FLASH_TYPEERASE_MASSERASE)
  {
    /* 整个 BANK1 或者 BANK2 擦除 */
    FLASH_MassErase(pEraseInit->VoltageRange, pEraseInit->Banks);

    /* 等待操作完成 */
    if((pEraseInit->Banks & FLASH_BANK_1) == FLASH_BANK_1)
    {
      if(FLASH_WaitForLastOperation((uint32_t)FLASH_TIMEOUT_VALUE, FLASH_BANK_1) != HAL_OK)
      {
        status = HAL_ERROR;
      }
      /* 如果擦除完成，关闭 BANK1 的 BER 位 */
      FLASH->CR1 &= (~FLASH_CR_BER);
    }
    if((pEraseInit->Banks & FLASH_BANK_2) == FLASH_BANK_2)
    {
      if(FLASH_WaitForLastOperation((uint32_t)FLASH_TIMEOUT_VALUE, FLASH_BANK_2) != HAL_OK)
      {
        status = HAL_ERROR;
      }
      /* 如果擦除操作完成，关闭 BANK2 的 BER 位 */
      FLASH->CR2 &= (~FLASH_CR_BER);
    }
  }
  else
  {
    /* 初始化扇区错误码 */
    *SectorError = 0xFFFFFFFFU;

    /* 扇区方式擦除 */
    for(sector_index = pEraseInit->Sector; sector_index < (pEraseInit->NbSectors + pEraseInit->Sector);
                                                                                    sector_index++)
    {
```

```
        FLASH_Erase_Sector(sector_index, pEraseInit->Banks, pEraseInit->VoltageRange);

        if((pEraseInit->Banks & FLASH_BANK_1) == FLASH_BANK_1)
        {
          /* 等待 BANK1 操作完成 */
          status = FLASH_WaitForLastOperation((uint32_t)FLASH_TIMEOUT_VALUE, FLASH_BANK_1);

          /* 如果擦除操作完成，禁止 SER 位 */
          FLASH->CR1 &= (~(FLASH_CR_SER | FLASH_CR_SNB));
        }
        if((pEraseInit->Banks & FLASH_BANK_2) == FLASH_BANK_2)
        {
          /* 等待 BANK2 操作完成 */
          status = FLASH_WaitForLastOperation((uint32_t)FLASH_TIMEOUT_VALUE, FLASH_BANK_2);

          /* 如果擦除操作完成，禁止 SER 位 */
          FLASH->CR2 &= (~(FLASH_CR_SER | FLASH_CR_SNB));
        }

        if(status != HAL_OK)
        {
          /* 如果擦除出错，停止后续擦除，返回擦除异常的扇区号 */
          *SectorError = sector_index;
          break;
        }
      }
    }
  }

  /* 解锁 */
  __HAL_UNLOCK(&pFlash);

  return status;
}
```

函数描述：

用于内部 Flash 的批量擦除（BANK 擦除）和扇区方式擦除。

函数参数：

- 第 1 个参数是 FLASH_EraseInitTypeDef 类型结构体指针变量。
- 第 2 个参数是错误码返回，返回 0xFFFFFFFF 表示全部正确，返回其它值是擦除过程中的错误扇区。
- 返回值，返回 HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

70.5 总结

本章节就为大家讲解这么多，对于内部 Flash 编程来说，掌握本章节的这些知识点就够用了，更多的知识点可以看 STM32H7 的参考手册学习。

第71章 STM32H7 的内部 Flash 应用之模拟 EEPROM

本章节为大家讲解 STM32H7 的内部 Flash 模拟 EEPROM，主要应用到板子没有外置 EERPOM 的场合，而且 H7 的内部 Flash 比较大，可以开辟一个扇区用于模拟 EEPROM。

71.1 初学者重要提示
71.2 模拟 EEPROM 驱动设计
71.4 模拟 EEPROM 板级支持包（bsp_cpu_flash.c）
71.5 模拟 EERPOM 驱动移植和使用
71.6 实验例程设计框架
71.7 实验例程说明（MDK）
71.8 实验例程说明（IAR）
71.9 总结

71.1 初学者重要提示

- 学习本章节前，务必优先学习第 70 章。
- 使用内部 Flash 模拟 EEPROM，务必告诉编译要使用的存储空间，防止这个空间存入了程序。
- STM32H7 的 Flash 编程时，务必保证要编程的地址是 32 字节对齐的，即此地址对 32 求余为 0。并且编程的数据必须 32 字节整数倍。
- STM32H743XI 有两个独立的 BANK，一个 BANK 的编程和擦除操作对另一个 BANK 没有任何影响。但是用户应用程序和要擦写的 Flash 扇区在同一个 BANK，在执行擦写操作时，应用应用程序将停止运行，包括中断服务程序。
- 使用内部 Flash 模拟 EEPROM 要做到先擦除后使用。

71.2 模拟 EEPROM 驱动设计

这里重点把内部 Flash 的读取，编程和擦除做个说明。

71.2.1 内部 Flash 擦除的实现

内部 Flash 的擦除思路如下：

- 第 1 步，获取擦除地址所处的扇区。
- 第 2 步，调用函数 HAL_FLASH_Unlock 解锁。
- 第 3 步，调用函数 HAL_FLASHEx_Erase 擦除一个扇区。
- 第 4 步，调用函数 HAL_FLASH_Lock 上锁。

按照这个思路，程序实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_EraseCpuFlash
*    功能说明: 擦除 CPU FLASH 一个扇区 （128KB）
*    形    参: _ulFlashAddr : Flash 地址
*    返 回 值: 0 成功， 1 失败
*              HAL_OK       = 0x00,
*              HAL_ERROR    = 0x01,
*              HAL_BUSY     = 0x02,
*              HAL_TIMEOUT  = 0x03
*
*********************************************************************************************************
*/
uint8_t bsp_EraseCpuFlash(uint32_t _ulFlashAddr)
{
    uint32_t FirstSector = 0, NbOfSectors = 0;
    FLASH_EraseInitTypeDef EraseInitStruct;
    uint32_t SECTORError = 0;
    uint8_t re;

    /* 解锁 */
    HAL_FLASH_Unlock();

    /* 获取此地址所在的扇区 */
    FirstSector = bsp_GetSector(_ulFlashAddr);

    /* 固定 1 个扇区 */
    NbOfSectors = 1;

    /* 擦除扇区配置 */
    EraseInitStruct.TypeErase     = FLASH_TYPEERASE_SECTORS;
    EraseInitStruct.VoltageRange  = FLASH_VOLTAGE_RANGE_3;

    if (_ulFlashAddr >= ADDR_FLASH_SECTOR_0_BANK2)
    {
        EraseInitStruct.Banks         = FLASH_BANK_2;
    }
    else
    {
        EraseInitStruct.Banks         = FLASH_BANK_1;
    }

    EraseInitStruct.Sector        = FirstSector;
    EraseInitStruct.NbSectors     = NbOfSectors;
```

```
58.
59.       /* 扇区擦除 */
60.       re = HAL_FLASHEx_Erase(&EraseInitStruct, &SECTORError);
61.
62.       /* 擦除完毕后，上锁 */
63.       HAL_FLASH_Lock();
64.
65.       return re;
66.   }
```

这里将此程序设计的关键点为大家做个说明：

◆ 第 25 行函数是通过函数 bsp_GetSector 获取要擦除地址所处的扇区。这个函数的实现比较简单，代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_GetSector
*    功能说明: 根据地址计算扇区首地址
*    形    参: 无
*    返 回 值: 扇区号（0-7）
*********************************************************************************************************
*/
uint32_t bsp_GetSector(uint32_t Address)
{
    uint32_t sector = 0;

    if (((Address < ADDR_FLASH_SECTOR_1_BANK1) && (Address >= ADDR_FLASH_SECTOR_0_BANK1)) || \
        ((Address < ADDR_FLASH_SECTOR_1_BANK2) && (Address >= ADDR_FLASH_SECTOR_0_BANK2)))
    {
        sector = FLASH_SECTOR_0;
    }
    else if (((Address < ADDR_FLASH_SECTOR_2_BANK1) && (Address >= ADDR_FLASH_SECTOR_1_BANK1)) || \
      ((Address < ADDR_FLASH_SECTOR_2_BANK2) && (Address >= ADDR_FLASH_SECTOR_1_BANK2)))
    {
        sector = FLASH_SECTOR_1;
    }
    else if (((Address < ADDR_FLASH_SECTOR_3_BANK1) && (Address >= ADDR_FLASH_SECTOR_2_BANK1)) || \
      ((Address < ADDR_FLASH_SECTOR_3_BANK2) && (Address >= ADDR_FLASH_SECTOR_2_BANK2)))
    {
        sector = FLASH_SECTOR_2;
    }
    else if (((Address < ADDR_FLASH_SECTOR_4_BANK1) && (Address >= ADDR_FLASH_SECTOR_3_BANK1)) || \
      ((Address < ADDR_FLASH_SECTOR_4_BANK2) && (Address >= ADDR_FLASH_SECTOR_3_BANK2)))
    {
        sector = FLASH_SECTOR_3;
    }
    else if (((Address < ADDR_FLASH_SECTOR_5_BANK1) && (Address >= ADDR_FLASH_SECTOR_4_BANK1)) || \
      ((Address < ADDR_FLASH_SECTOR_5_BANK2) && (Address >= ADDR_FLASH_SECTOR_4_BANK2)))
    {
        sector = FLASH_SECTOR_4;
    }
    else if (((Address < ADDR_FLASH_SECTOR_6_BANK1) && (Address >= ADDR_FLASH_SECTOR_5_BANK1)) || \
      ((Address < ADDR_FLASH_SECTOR_6_BANK2) && (Address >= ADDR_FLASH_SECTOR_5_BANK2)))
    {
        sector = FLASH_SECTOR_5;
    }
    else if (((Address < ADDR_FLASH_SECTOR_7_BANK1) && (Address >= ADDR_FLASH_SECTOR_6_BANK1)) || \
      ((Address < ADDR_FLASH_SECTOR_7_BANK2) && (Address >= ADDR_FLASH_SECTOR_6_BANK2)))
```

```
    {
        sector = FLASH_SECTOR_6;
    }
    else if (((Address < ADDR_FLASH_SECTOR_0_BANK2) && (Address >= ADDR_FLASH_SECTOR_7_BANK1)) || \
      ((Address < CPU_FLASH_END_ADDR) && (Address >= ADDR_FLASH_SECTOR_7_BANK2)))
    {
        sector = FLASH_SECTOR_7;
    }
    else
    {
        sector = FLASH_SECTOR_7;
    }

    return sector;
}
```

由于 STM32H7 的 BANK1 和 BANK2 是独立的，都有 8 个扇区，所以程序里面只需返回要擦除地址所处的扇区号即可。

- 第 47 行的擦除函数 HAL_FLASHEx_Erase 在第 70 章的 4.4 小节有说明。

71.2.2 内部 Flash 编程的实现

内部 Flash 的编程思路如下：

- 第 1 步，判断是否要编写数据进去，如果数据已经在内部 Flash 里面。
- 第 2 步，调用函数 HAL_FLASH_Unlock 解锁。
- 第 3 步，调用函数 HAL_FLASH_Program 对内部 Flash 编程数据。
- 第 4 步，调用函数 HAL_FLASH_Lock 上锁。

按照这个思路，程序实现如下：

```
1.  /*
2.  *********************************************************************************************************
3.  *    函 数 名: bsp_WriteCpuFlash
4.  *    功能说明: 写数据到 CPU 内部 Flash。 必须按 32 字节整数倍写。不支持跨扇区。扇区大小 128KB. \
5.  *              写之前需要擦除扇区. 长度不是 32 字节整数倍时，最后几个字节末尾补 0 写入.
6.  *    形    参: _ulFlashAddr : Flash 地址
7.  *              _ucpSrc : 数据缓冲区
8.  *              _ulSize : 数据大小（单位是字节, 必须是 32 字节整数倍）
9.  *    返 回 值: 0-成功，1-数据长度或地址溢出，2-写 Flash 出错(估计 Flash 寿命到)
10. *********************************************************************************************************
11. */
12. uint8_t bsp_WriteCpuFlash(uint32_t _ulFlashAddr, uint8_t *_ucpSrc, uint32_t _ulSize)
13. {
14.     uint32_t i;
15.     uint8_t ucRet;
16.
17.     /* 如果偏移地址超过芯片容量，则不改写输出缓冲区 */
18.     if (_ulFlashAddr + _ulSize > CPU_FLASH_BASE_ADDR + CPU_FLASH_SIZE)
19.     {
20.         return 1;
21.     }
22.
23.     /* 长度为 0 时不继续操作  */
```

```
24.       if (_ulSize == 0)
25.       {
26.           return 0;
27.       }
28.
29.       ucRet = bsp_CmpCpuFlash(_ulFlashAddr, _ucpSrc, _ulSize);
30.
31.       if (ucRet == FLASH_IS_EQU)
32.       {
33.           return 0;
34.       }
35.
36.       __set_PRIMASK(1);           /* 关中断 */
37.
38.       /* FLASH 解锁 */
39.       HAL_FLASH_Unlock();
40.
41.       for (i = 0; i < _ulSize / 32; i++)
42.       {
43.           uint64_t FlashWord[4];
44.
45.           memcpy((char *)FlashWord, _ucpSrc, 32);
46.           _ucpSrc += 32;
47.
48.           if (HAL_FLASH_Program(FLASH_TYPEPROGRAM_FLASHWORD, _ulFlashAddr,
49.                                 (uint64_t)((uint32_t)FlashWord)) == HAL_OK)
50.           {
51.               _ulFlashAddr = _ulFlashAddr + 32; /* 递增，操作下一个 256bit */
52.           }
53.           else
54.           {
55.               goto err;
56.           }
57.       }
58.
59.       /* 长度不是 32 字节整数倍 */
60.       if (_ulSize % 32)
61.       {
62.           uint64_t FlashWord[4];
63.
64.           FlashWord[0] = 0;
65.           FlashWord[1] = 0;
66.           FlashWord[2] = 0;
67.           FlashWord[3] = 0;
68.           memcpy((char *)FlashWord, _ucpSrc, _ulSize % 32);
69.           if (HAL_FLASH_Program(FLASH_TYPEPROGRAM_FLASHWORD, _ulFlashAddr,
70.                                       (uint64_t)((uint32_t)FlashWord)) == HAL_OK)
71.           {
72.               ; // _ulFlashAddr = _ulFlashAddr + 32;
73.
74.           }
75.           else
76.           {
77.               goto err;
78.           }
79.       }
80.
81.       /* Flash 加锁，禁止写 Flash 控制寄存器 */
```

```
82.       HAL_FLASH_Lock();
83.
84.       __set_PRIMASK(0);           /* 开中断 */
85.
86.       return 0;
87.
88.   err:
89.       /* Flash 加锁，禁止写 Flash 控制寄存器 */
90.       HAL_FLASH_Lock();
91.
92.       __set_PRIMASK(0);           /* 开中断 */
93.
94.       return 1;
95.   }
```

关于此函数有几个要点：

- 第 1 个参数必须 32 字节对齐，即要编程的 Flash 地址对 32 求余为 0。
- 第 3 个参数必须是 32 字节的整数倍，长度不是 32 字节整数倍时，最后几个字节补 0 写入。
- 第 29 行，函数 bsp_CmpCpuFlash 放在这里只有一个作用，判断将要写入的数据是否已经在内部 Flash 存在，如果已经存在，无需重复写入，直接返回。
- 第 36 行，做了一个关中断操作，这里有个知识点要给大家普及下，由于 H7 的 BANK1 和 BANK2 是独立的，当前正在擦除的扇区所处的 BANK 会停止所有在此 BANK 执行的程序，包含中断也会停止执行。比如当前的应用程序都在 BANK1，如果要擦除或者编程 BANK2，是不会不影响 BANK1 里面执行的程序。这里安全起见加上了开关中断。
- 第 41 到 57 行，先将 32 字节整数倍的数据通过函数 HAL_FLASH_Program 编程，此函数每次可以固定编程 32 字节数据。
- 第 60 到 79 行，将剩余不足 32 字节的数据补 0，凑齐 32 字节编程。

71.2.3 内部 Flash 读取的实现

内部 Flash 数据读取比较简单，采用总线方式读取，跟访问内部 RAM 是一样的。比如要读取地址 0x08100000 里面的一个 32bit 变量，我们就可以：

变量 = *(uint32_t *)(0x08100000)

为了方便起见，也专门准备了一个函数：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_ReadCpuFlash
*    功能说明: 读取 CPU Flash 的内容
*    形    参:  _ucpDst : 目标缓冲区
*              _ulFlashAddr : 起始地址
*              _ulSize : 数据大小（单位是字节）
*    返 回 值: 0=成功，1=失败
*********************************************************************************************************
*/
uint8_t bsp_ReadCpuFlash(uint32_t _ulFlashAddr, uint8_t *_ucpDst, uint32_t _ulSize)
{
```

```
    uint32_t i;

    if (_ulFlashAddr + _ulSize > CPU_FLASH_BASE_ADDR + CPU_FLASH_SIZE)
    {
        return 1;
    }

    /* 长度为 0 时不继续操作,否则起始地址为奇地址会出错 */
    if (_ulSize == 0)
    {
        return 1;
    }

    for (i = 0; i < _ulSize; i++)
    {
        *_ucpDst++ = *(uint8_t *)_ulFlashAddr++;
    }

    return 0;
}
```

71.2.4 告诉编译器使用的扇区（重要）

使用内部 Flash 模拟 EEPROM 切不可随意定义一个扇区使用。因为编译器并不知道用户使用了这个扇区，导致应用程序也会编程到此扇区里面，所以就需要告诉编译器。

告诉 MDK 的方法如下（0x0810 0000 是 H7 的 BANK2 首地址）：

```
const uint8_t para_flash_area[128*1024] __attribute__((at(0x08100000)));
```

告诉 IAR 的方法如下：

```
#pragma location=0x08100000
const uint8_t para_flash_area[128*1024];
```

这里有两点特别注意：

- 模拟 EEPROM 的扇区可以定义到从第 2 个扇区开始的任何扇区，但不可以定义到首扇区，因为这个扇区是默认的 boot 启动地址。
- 如果应用程序不大的话，不推荐定义到末尾扇区，以 MDK 为例，定义到末尾扇区后，会导致整个 Flash 空间都被使用，从而让程序下载下载时间变长。

71.3 模拟 EEPROM 板级支持包（bsp_cpu_flash.c）

模拟 EEPROM 的驱动文件 bsp_cpu_flash.c 主要实现了如下几个 API 供用户调用：

- bsp_GetSector
- bsp_ReadCpuFlash
- bsp_CmpCpuFlash
- bsp_EraseCpuFlash

- bsp_WriteCpuFlash

71.3.1 函数 bsp_GetSector

函数原型：

```
uint32_t bsp_GetSector(uint32_t Address)
```

函数描述：

此函数主要用于获取给定地址所处的扇区。

函数参数：

- 第 1 个参数是用户给定的地址。
- 返回值，范围 FLASH_SECTOR_0 到 FLASH_SECTOR_7。

71.3.2 函数 bsp_ReadCpuFlash

函数原型：

```
uint8_t bsp_ReadCpuFlash(uint32_t _ulFlashAddr, uint8_t *_ucpDst, uint32_t _ulSize)
```

函数描述：

此函数用于从内部 Flash 读取数据

函数参数：

- 第 1 个参数读取的起始地址。
- 第 2 个参数是读取数据的存储地址
- 第 3 个参数是读取数据的大小，单位字节。
- 返回值，0 表示成功，1 表示失败。

71.3.3 函数 bsp_CmpCpuFlash

函数原型：

```
uint8_t bsp_CmpCpuFlash(uint32_t _ulFlashAddr, uint8_t *_ucpBuf, uint32_t _ulSize)
```

函数描述：

要编程的数据是否在内部 Flash 已经存在。

函数参数：

- 第 1 个参数是内部 Flash 地址。
- 第 2 个参数是缓冲区地址。
- 第 3 个参数是数据大小，单位字节。
- 返回值：

  FLASH_IS_EQU　　0　　Flash 内容和待写入的数据相等，不需要擦除和写操作。

FLASH_REQ_WRITE	1	Flash 不需要擦除，直接写。
FLASH_REQ_ERASE	2	Flash 需要先擦除,再写。
FLASH_PARAM_ERR	3	函数参数错误。

71.3.4函数 bsp_EraseCpuFlash

函数原型：

```
uint8_t bsp_EraseCpuFlash(uint32_t _ulFlashAddr)
```

函数描述：

此函数用于擦除一个扇区，大小 128KB

函数参数：

- 第 1 个参数要擦除的扇区地址，可以是此扇区范围内的任意值，一般填扇区首地址即可。
- 返回值：

  HAL_OK = 0x00
  HAL_ERROR = 0x01
  HAL_BUSY = 0x02
  HAL_TIMEOUT = 0x03

71.3.5函数 bsp_WriteCpuFlash

函数原型：

```
uint8_t bsp_WriteCpuFlash(uint32_t _ulFlashAddr, uint8_t *_ucpSrc, uint32_t _ulSize)
```

函数描述：

此函数用于编程数据到内部 Flash。

函数参数：

- 第 1 个参数是要编程的内部 Flash 地址。
- 第 2 个参数是数据缓冲区地址。
- 第 3 个参数是数据大小，单位字节。
- 返回值，0-成功，1-数据长度或地址溢出，2-写 Flash 出错(估计 Flash 寿命到)。

注意事项：

- 第 1 个参数必须 32 字节对齐，即要编程的 Flash 地址对 32 求余为 0。
- 第 3 个参数必须是 32 字节的整数倍，长度不是 32 字节整数倍时，此函数会将几个字节补 0 写入

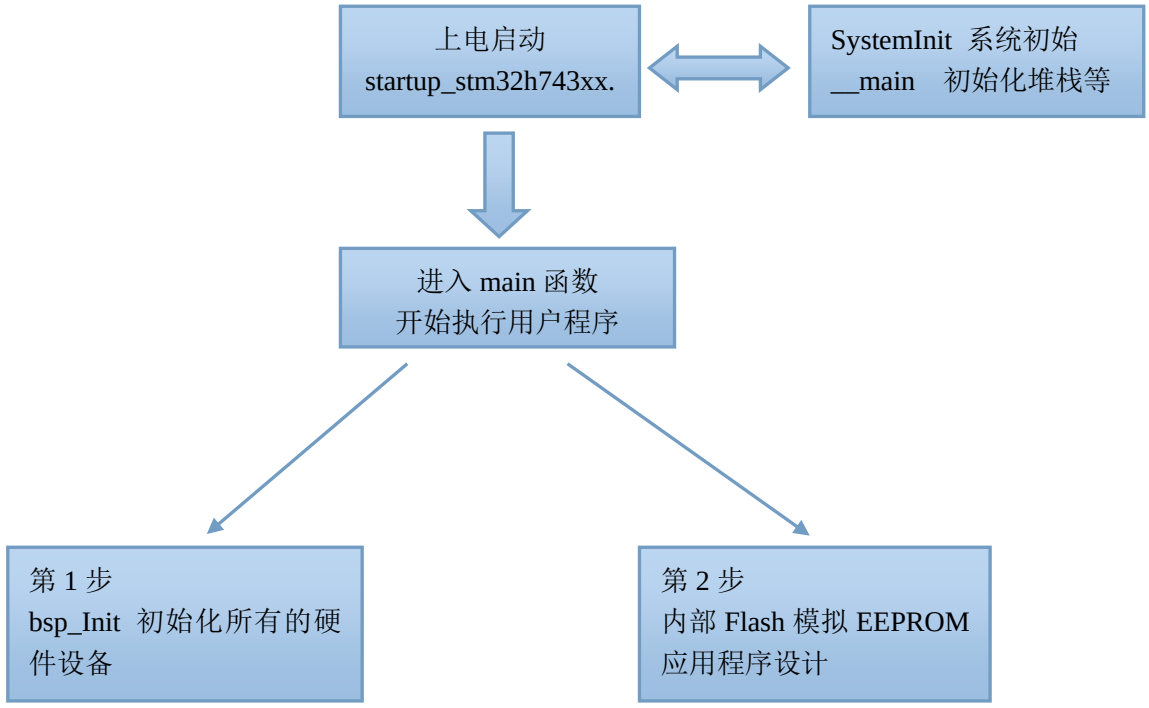
71.4模拟 EEPROM 驱动移植和使用

模拟 EEPROM 移植步骤如下：

- 第 1 步：复制 bsp_cpu_flash.c 和 bsp_cpu_flash.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：Flash 驱动文件主要用到 HAL 库的 Flash 驱动文件，简单省事些可以添加所有 HAL 库 C 源文件进来。
- 第 3 步，应用方法看本章节配套例子即可。

71.5 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。
- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器和 LED。
  - 第 2 部分，应用程序设计部分，实现内部 Flash 模拟 EEPROM。

71.6 实验例程说明（MDK）

配套例子：

V7-049_内部 Flash 模拟 EEPROM

实验目的：

2. 学习内部 Flash 模拟 EEPROM。

实验内容：

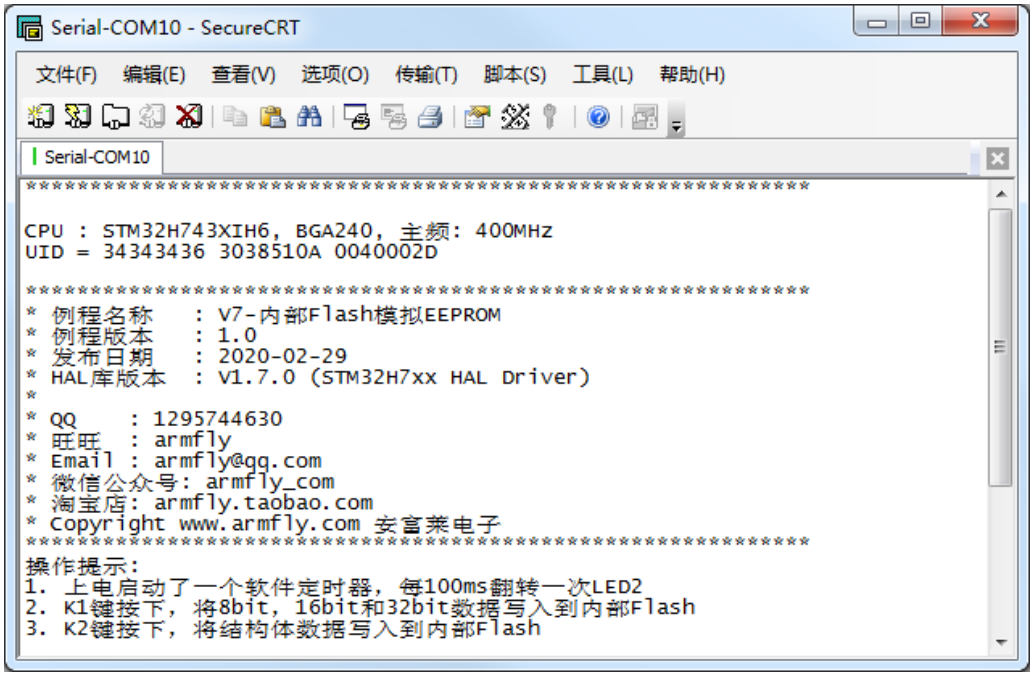
6. 使用内部 Flash 模拟 EEPROM，务必告诉编译要使用的存储空间，防止这个空间存入了程序。
7. 对于同一个地址空间，仅支持一次编程(不推荐二次编程，即使是将相应 bit 由数值 1 编程 0)。
8. 只能对已经擦除的空间做编程，擦除 1 个扇区是 128KB。
9. H7 的 Flash 编程时，务必保证要编程的地址是 32 字节对齐的，即此地址对 32 求余为 0。并且编程的数据必须 32 字节整数倍，函数 bsp_WriteCpuFlash 对字节数不够 32 字节整数倍的情况自动补 0。

实验操作：

4. K1 键按下，将 8bit，16bit 和 32bit 数据写入到内部 Flash。
5. K2 键按下，将结构体数据写入到内部 Flash。

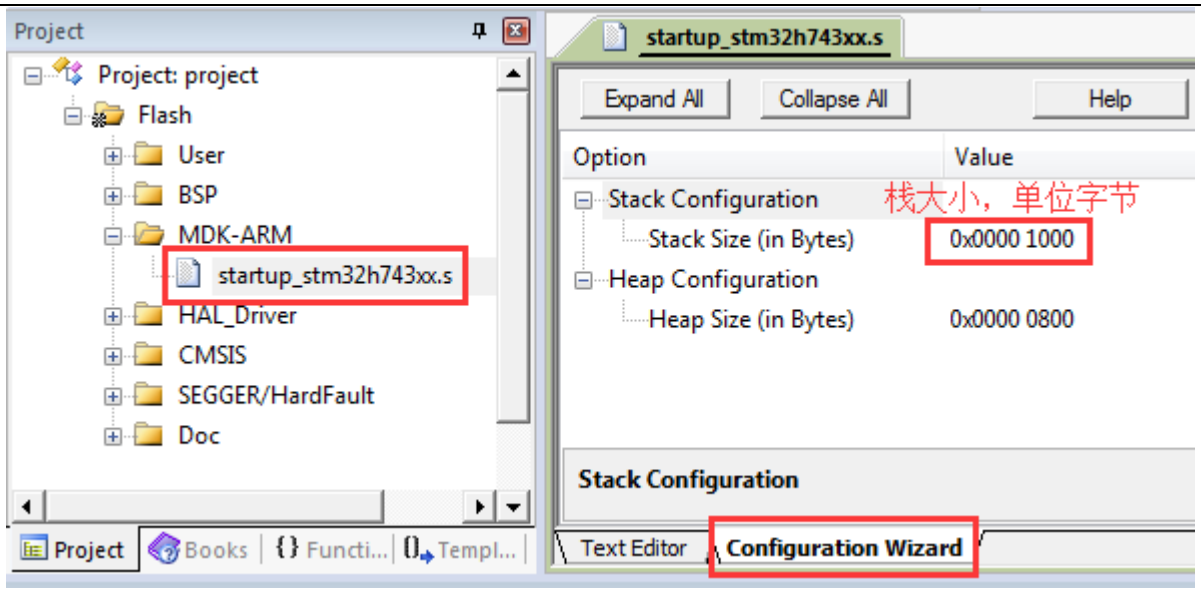
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1。

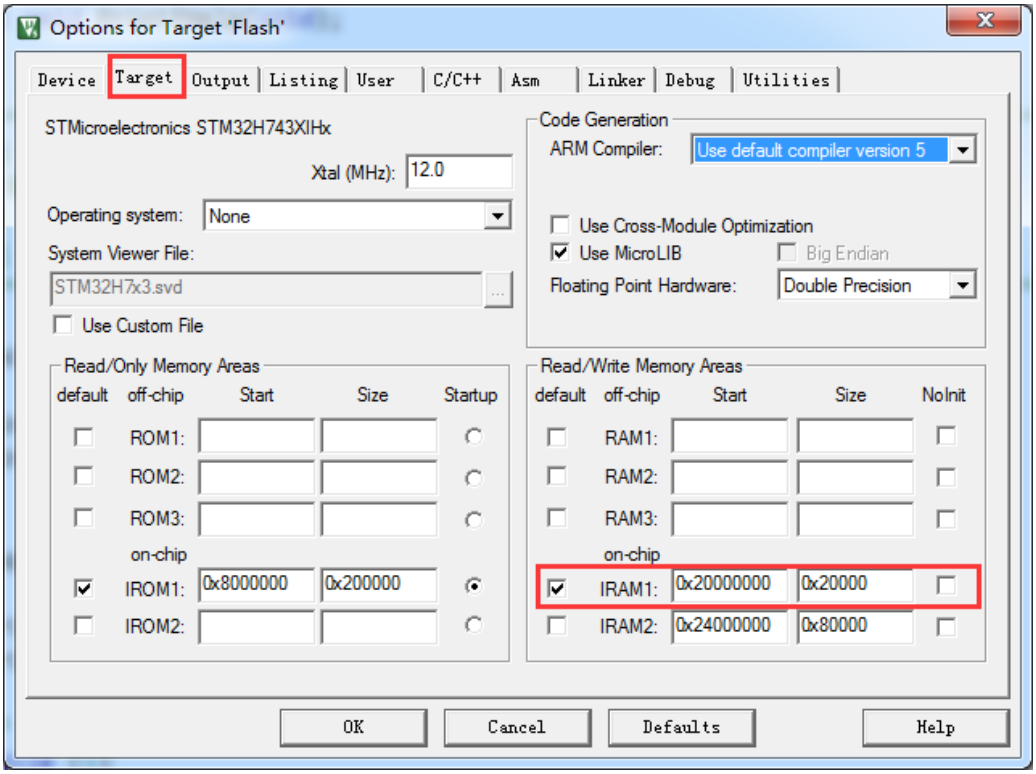


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitLPUart();  /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
```

```
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **每 10ms 调用一次蜂鸣器处理：**

  蜂鸣器处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
```

```
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能：**

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- K1 键按下，将 8bit，16bit 和 32bit 数据写入到内部 Flash。
- K2 键按下，将结构体数据写入到内部 Flash。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode; /* 按键代码 */
    uint8_t  ucTest, *ptr8;
    uint16_t uiTest, *ptr16;
    uint32_t ulTest, *ptr32;
    PARAM_T tPara, *paraptr;


    /* 初始化数据 */
    tPara.Baud485 = 0x5555AAAA;
    tPara.ParamVer = 0x99;
    tPara.ucBackLight = 0x7788;
    tPara.ucRadioMode = 99.99f;


    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
```

```
        bsp_LedToggle(2);
    }

    /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
    ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
    if (ucKeyCode != KEY_NONE)
    {
        switch (ucKeyCode)
        {
            case KEY_DOWN_K1:           /* K1 键按下，将 8bit，16bit 和 32bit 数据写入到内部 Flash */

            /*
             1、对于同一个地址空间，仅支持一次编程(不推荐二次编程，即使是将相应 bit 由数值 1 编程 0)。
             2、只能对已经擦除的空间做编程，擦除 1 个扇区是 128KB。
             3、H7 的 Flash 编程时，务必保证要编程的地址是 32 字节对齐的，即此地址对 32 求余为 0。并且编
                程的数据必须 32 字节整数倍。函数 bsp_WriteCpuFlash 对字节数不够 32 字节整数倍的情况自动补
                0。
            */
                /* 擦除扇区 */
                bsp_EraseCpuFlash((uint32_t)para_flash_area);

                ucTest = 0xAA;
                uiTest = 0x55AA;
                ulTest = 0x11223344;

                /* 扇区写入数据 */
                bsp_WriteCpuFlash((uint32_t)para_flash_area + 32*0,  (uint8_t *)&ucTest,
                                                                        sizeof(ucTest));
                bsp_WriteCpuFlash((uint32_t)para_flash_area + 32*1,  (uint8_t *)&uiTest,
                                                                        sizeof(uiTest));
                bsp_WriteCpuFlash((uint32_t)para_flash_area + 32*2,  (uint8_t *)&ulTest,
                                                                       sizeof(ulTest));


                /* 读出数据并打印 */
                ptr8  = (uint8_t  *)(para_flash_area + 32*0);
                ptr16 = (uint16_t *)(para_flash_area + 32*1);
                ptr32 = (uint32_t *)(para_flash_area + 32*2);

                printf("写入数据：ucTest = %x, uiTest = %x, ulTest = %x\r\n", ucTest, uiTest, ulTest);
                printf("读取数据：ptr8 = %x, ptr16 = %x, ptr32 = %x\r\n", *ptr8, *ptr16, *ptr32);

                break;

            case KEY_DOWN_K2:           /* K2 键按下， 将结构体数据写入到内部 Flash */
                /* 擦除扇区 */
                bsp_EraseCpuFlash((uint32_t)para_flash_area);

                /* 扇区写入数据 */
                bsp_WriteCpuFlash((uint32_t)para_flash_area,  (uint8_t *)&tPara, sizeof(tPara));


                /* 读出数据并打印 */
                paraptr  = (PARAM_T  *)((uint32_t)para_flash_area);


                printf("写入数据：Baud485=%x, ParamVer=%x, ucBackLight=%x, ucRadioMode=%f\r\n",
                                                                        tPara.Baud485,
```

```
                                                                    tPara.ParamVer,
                                                                    tPara.ucBackLight,
                                                                paraptr->ucRadioMode);

                        printf("读取数据：Baud485=%x, ParamVer=%x, ucBackLight=%x, ucRadioMode=%f\r\n",
                                                                     paraptr->Baud485,
                                                                     paraptr->ParamVer,
                                                                     paraptr->ucBackLight,
                                                                    paraptr->ucRadioMode);
                        break;
                    default:
                        /* 其它的键值不处理 */
                        break;
                }
            }
        }
}
```

71.7 实验例程说明（IAR）

配套例子：

V7-049_内部 Flash 模拟 EEPROM

实验目的：

1. 学习内部 Flash 模拟 EEPROM。

实验内容：

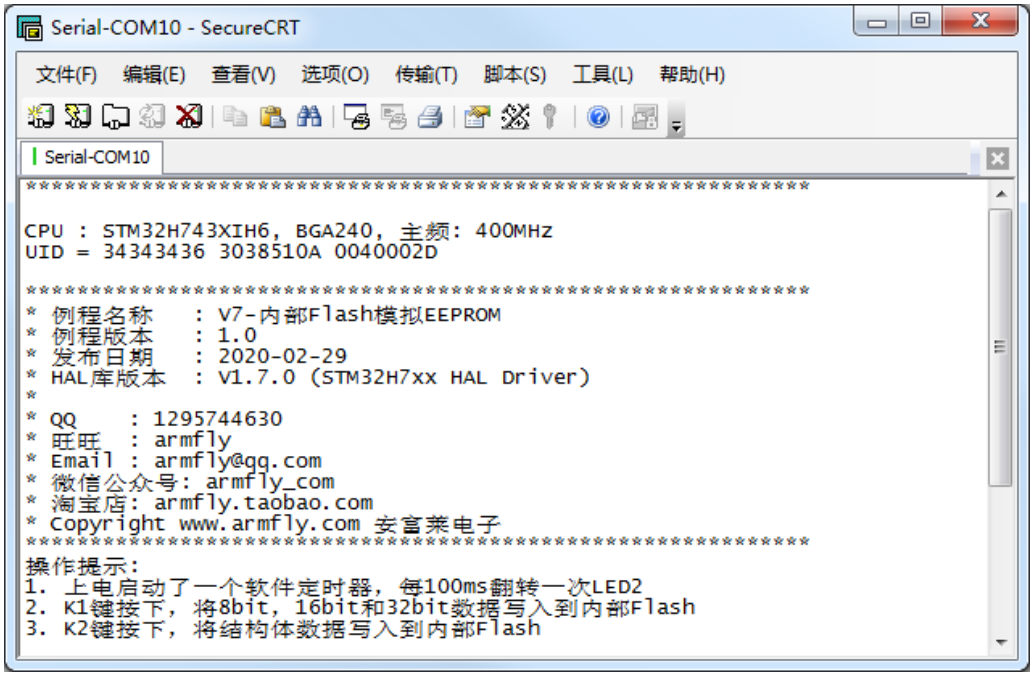
1. 使用内部 Flash 模拟 EEPROM，务必告诉编译要使用的存储空间，防止这个空间存入了程序。
2. 对于同一个地址空间，仅支持一次编程(不推荐二次编程，即使是将相应 bit 由数值 1 编程 0)。
3. 只能对已经擦除的空间做编程，擦除 1 个扇区是 128KB。
4. H7 的 Flash 编程时，务必保证要编程的地址是 32 字节对齐的，即此地址对 32 求余为 0。并且编程的数据必须 32 字节整数倍，函数 bsp_WriteCpuFlash 对字节数不够 32 字节整数倍的情况自动补 0。

实验操作：

1. K1 键按下，将 8bit，16bit 和 32bit 数据写入到内部 Flash。
2. K2 键按下，将结构体数据写入到内部 Flash。

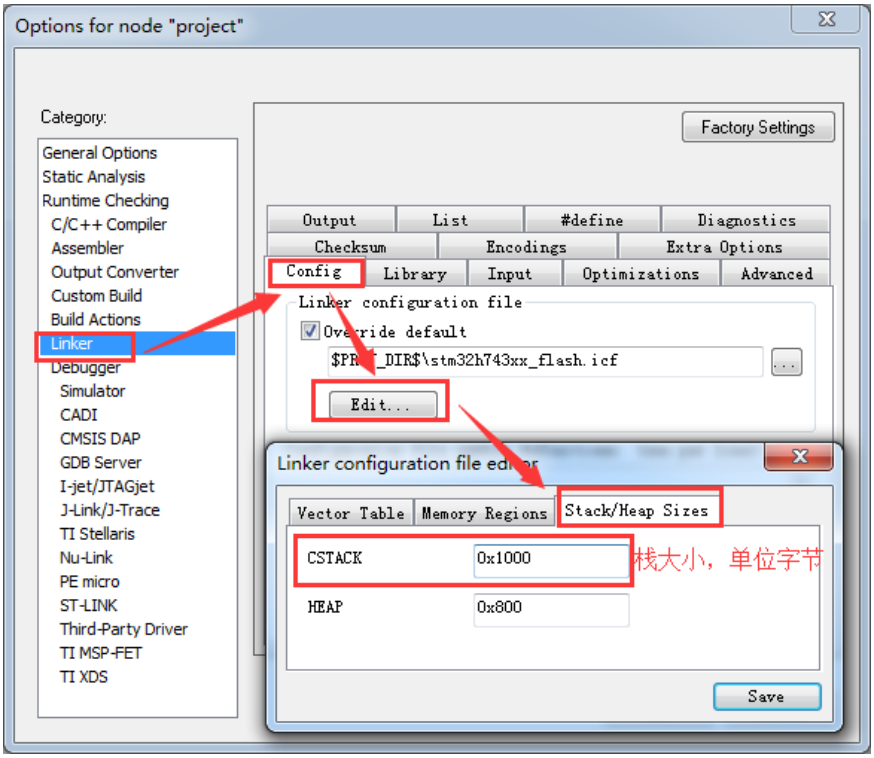
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1。

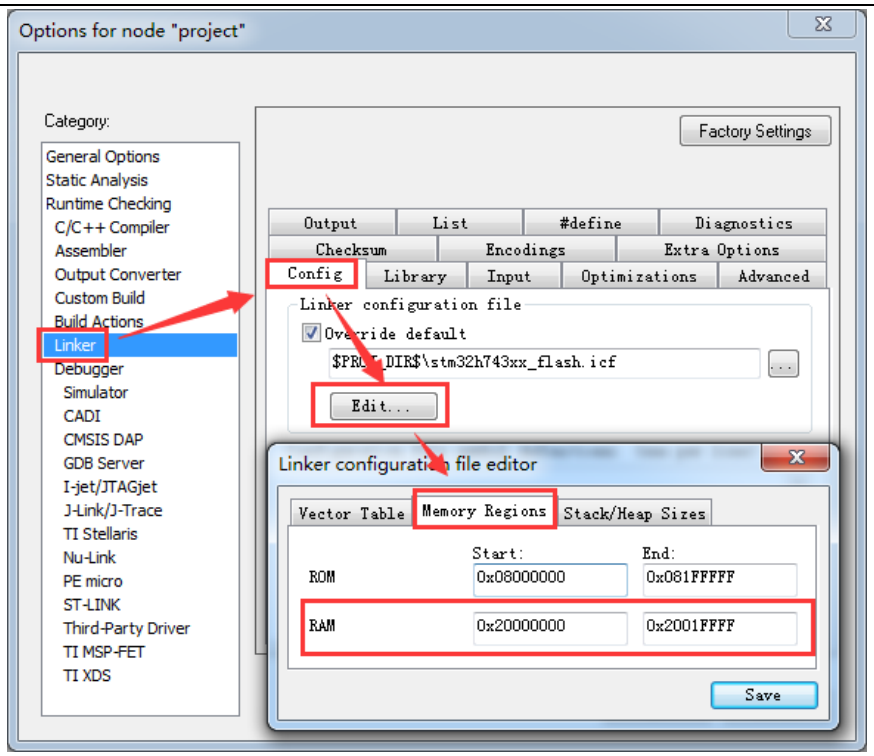


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitLPUart();  /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
   bsp_InitExtSDRAM(); /* 初始化 SDRAM */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
```

```
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ 每 10ms 调用一次蜂鸣器处理：

蜂鸣器处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ 主功能：

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- K1 键按下，将 8bit，16bit 和 32bit 数据写入到内部 Flash。
- K2 键按下，将结构体数据写入到内部 Flash。

```
/*
```

```
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t ucKeyCode; /* 按键代码 */
    uint8_t  ucTest, *ptr8;
    uint16_t uiTest, *ptr16;
    uint32_t ulTest, *ptr32;
    PARAM_T tPara, *paraptr;


    /* 初始化数据 */
    tPara.Baud485 = 0x5555AAAA;
    tPara.ParamVer = 0x99;
    tPara.ucBackLight = 0x7788;
    tPara.ucRadioMode = 99.99f;


    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，将 8bit，16bit 和 32bit 数据写入到内部 Flash */

                /*
                 1、对于同一个地址空间，仅支持一次编程(不推荐二次编程，即使是将相应 bit 由数值 1 编程 0)。
                 2、只能对已经擦除的空间做编程，擦除 1 个扇区是 128KB。
                 3、H7 的 Flash 编程时，务必保证要编程的地址是 32 字节对齐的，即此地址对 32 求余为 0。并且编
                    程的数据必须 32 字节整数倍。函数 bsp_WriteCpuFlash 对字节数不够 32 字节整数倍的情况自动补
                    0。
                */
                     /* 擦除扇区 */
                     bsp_EraseCpuFlash((uint32_t)para_flash_area);

                     ucTest = 0xAA;
```

```
                    uiTest = 0x55AA;
                    ulTest = 0x11223344;

                    /* 扇区写入数据 */
                    bsp_WriteCpuFlash((uint32_t)para_flash_area + 32*0,  (uint8_t *)&ucTest,
                                                                          sizeof(ucTest));
                    bsp_WriteCpuFlash((uint32_t)para_flash_area + 32*1,  (uint8_t *)&uiTest,
                                                                          sizeof(uiTest));
                    bsp_WriteCpuFlash((uint32_t)para_flash_area + 32*2,  (uint8_t *)&ulTest,
                                                                         sizeof(ulTest));


                    /* 读出数据并打印 */
                    ptr8  = (uint8_t  *)(para_flash_area + 32*0);
                    ptr16 = (uint16_t *)(para_flash_area + 32*1);
                    ptr32 = (uint32_t *)(para_flash_area + 32*2);

                    printf("写入数据：ucTest = %x, uiTest = %x, ulTest = %x\r\n", ucTest, uiTest, ulTest);
                    printf("读取数据：ptr8 = %x, ptr16 = %x, ptr32 = %x\r\n", *ptr8, *ptr16, *ptr32);

                    break;

                case KEY_DOWN_K2:           /* K2 键按下， 将结构体数据写入到内部 Flash */
                    /* 擦除扇区 */
                    bsp_EraseCpuFlash((uint32_t)para_flash_area);

                    /* 扇区写入数据 */
                    bsp_WriteCpuFlash((uint32_t)para_flash_area,  (uint8_t *)&tPara, sizeof(tPara));


                    /* 读出数据并打印 */
                    paraptr  = (PARAM_T  *)((uint32_t)para_flash_area);


                    printf("写入数据：Baud485=%x, ParamVer=%x, ucBackLight=%x, ucRadioMode=%f\r\n",
                                                                          tPara.Baud485,
                                                                          tPara.ParamVer,
                                                                          tPara.ucBackLight,
                                                                    paraptr->ucRadioMode);

                    printf("读取数据：Baud485=%x, ParamVer=%x, ucBackLight=%x, ucRadioMode=%f\r\n",
                                                                           paraptr->Baud485,
                                                                           paraptr->ParamVer,
                                                                           paraptr->ucBackLight,
                                                                          paraptr->ucRadioMode);
                    break;
                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

71.8 总结

本章节就为大家讲解这么多， 实际应用中的注意事项比较多，应用到项目之前务必实际测试熟悉下。

第72章 STM32H7 的 SPI 总线基础知识和 HAL 库 API

本章节为大家讲解 SPI（Serial peripheral interface）总线的基础知识和对应的 HAL 库 API。

72.1 初学者重要提示
72.2 SPI 总线基础知识
72.3 SPI 总线的 HAL 库用法
72.4 源文件 stm32h7xx_hal_spi.c
72.5 总结

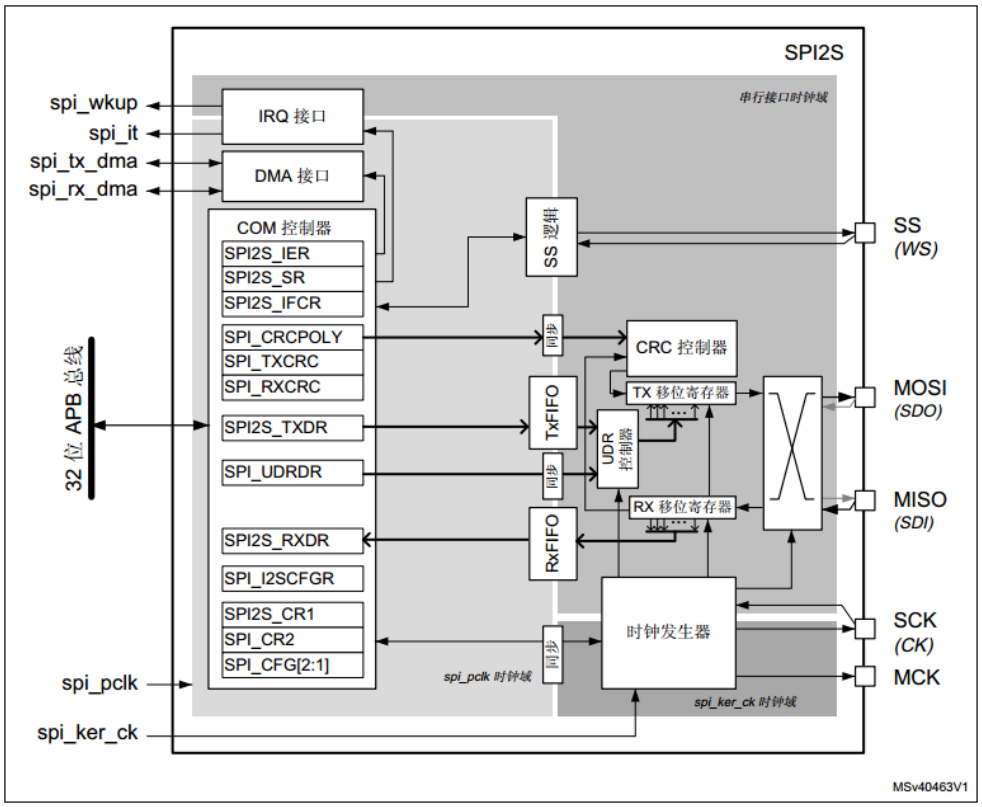
72.1 初学者重要提示

- STM32H7 的 SPI 支持 4 到 32bit 数据传输，而 STM32F1 和 F4 系列仅支持 8bit 或者 16bit。
- STM32H7 的主频 400MHz 时，SPI1, 2, 3 最高通信时钟是 100MHz，而 SPI4, 5, 6 是 50MHz。
- STM32H7 的 MISO 和 MOSI 引脚功能可以互换，使用比较灵活。
- SPI 总线的片选引脚 SS 在单一的主从器件配置下是可选的，一般情况下可以不使用。
- 搜集了几篇质量比较高的 SPI 总线介绍帖：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=96788。

72.2 SPI 总线基础知识

72.2.1 SPI 总线的硬件框图

认识一个外设，最好的方式就是看它的框图，方便我们快速的了解 SPI 的基本功能，然后再看手册了解细节。



通过这个框图，我们可以得到如下信息：

- spi_wkup 输出

  低功耗唤醒信号。

- spi_it 输出

  spi 的中断请求信号。

- spi_tx_dma

  spi_rx_dma

  spi 的 DMA 发送和接收请求信号。

- spi_pclk

  为寄存器提供时钟。

- spi_ker_ck

  为 spi 内核时钟。

- SCK（CK），Serial Clock

  此引脚在主机模式下用于时钟输出，从机模式下用于时钟输入。

- MISO（SDI），Master In / Slave Out data

  此引脚在从机模式下用于发送数据，主机模式下接收数据。

- MOSI（SDO），Master Out / Slave In data

  此引脚在从机模式下用于数据接收，主机模式下发送数据。

- SS（WS），Slave select pin

根据 SPI 和 SS 设置，此引脚可用于：

- 选择三个从器件进行通信。
- 同步数据帧。
- 检测多个主器件之间是否存在冲突。

通过这个框图还要认识到一点，SPI 有三个时钟域，分别是寄存器所在的 ABP 总线时钟域，内核时钟发生器时钟域以及内核时钟发生器分频后的串行时钟域。

72.2.2 SPI 接口的区别和时钟源（SPI1 到 SPI6）

这个知识点在初学的时候容易忽视，所以我们这里整理下。

◆ SPI1 到 SPI6 的区别

- SPI1，SPI2 和 SPI3 支持 4 到 32bit 数据传输，SPI4，SPI5 和 SPI6 是 4 到 16bit 数据传输。
- SPI1，SPI2 和 SPI3 的 FIFO 大小是 16*8bit，而 SPI4，SPI5 和 SPI6 的 FIFO 大小是 8*8bit。

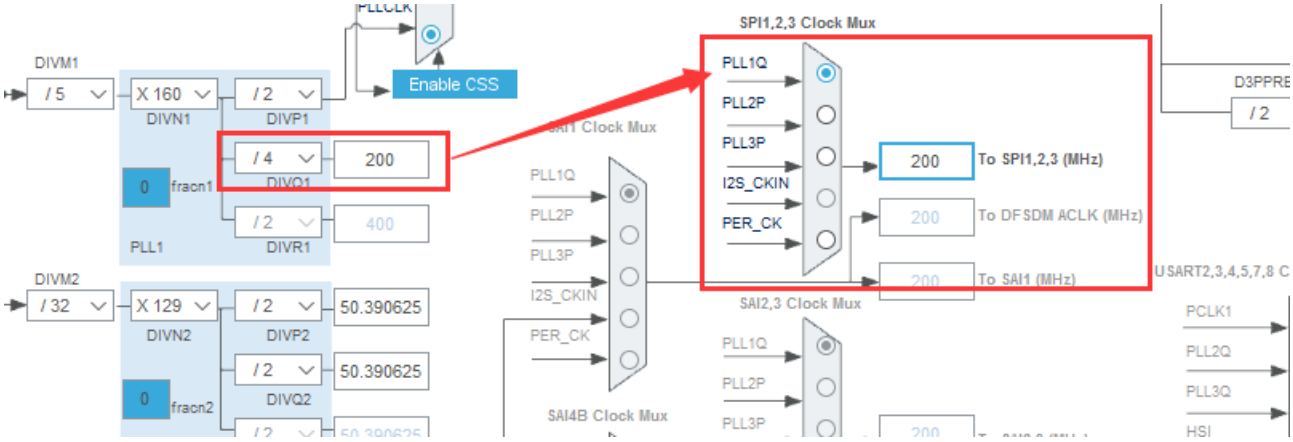
SPI modes/features	SPI2S1	SPI2S2	SPI2S3	SPI4	SPI5	SPI6
Rx & TxFIFO size (N) [x 8-bit]	16	16	16	8	8	8
Maximum configurable data size [bits]	32	32	32	16	16	16
I2S feature	Yes	Yes	Yes	No	No	No

◆ SPI1 到 SPI6 的所在的总线（对应 SPI 框图的 SPI_CLK 时钟域）

SPI1，SPI4 和 SPI5 在 APB2 总线，SPI2，SPI3 在 APB1 总线，SPI6 在 APB4 总线。注意，SPI 的最高时钟不是由这些总线决定的。

◆ SPI1 到 SPI6 的支持的最高时钟（对应 SPI 框图的 SPI_KER_CK）

STM32H7 主频在 400MHz 下，SPI1，SPI2 和 SPI3 的最高时钟是 200MHz，而 SPI4,5,6 是 100MHz，以 SPI1 为了，可以选择的时钟源如下：



这里特别注意一点，SPI 工作时最少选择二分频，也就是说 SPI1,2,3 实际通信时钟是 100MHz，而 SPI4,5,6

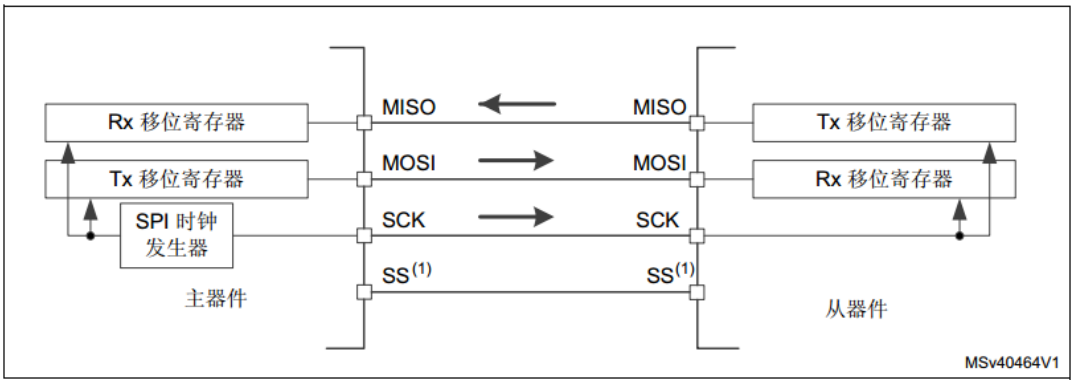
是 50MHz。

72.2.3 SPI 总线全双工，单工和半双工通信

片选信号 SS 在单一的主从器件配置下是可选的，一般情况下可以不使用。但需要同步数据流，或者用于 TI 模式时需要此信号。

◆ 全双工通信

全双工就是主从器件之间同时互传数据，SPI 总线的全双工模式接线方式如下：

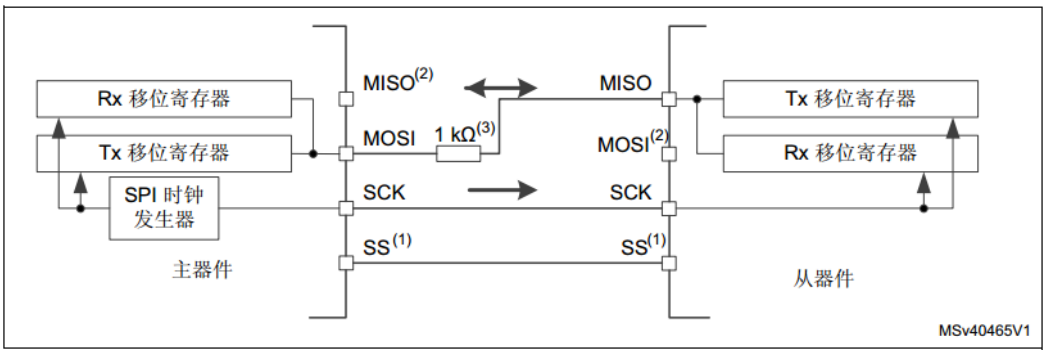


关于这个接线图要认识到以下几点：

- 注意接线方式，对于主器件来说 MISO 引脚就是输入端，从器件的 MISO 是输出端，即 Master In / Slave Out data。MOSI 也是同样道理。
- 每个时钟信号 SCK 的作用了，主器件的 MISO 引脚接收 1 个 bit 数据，MOSI 引脚输出 1 个 bit 数据。
- 这种单一的主从接线模式下，SS 引脚可以不使用。

◆ 半双工通信

半双工就是同一个时刻只能为一个方向传输数据，SPI 总线的半工模式接线方式如下：



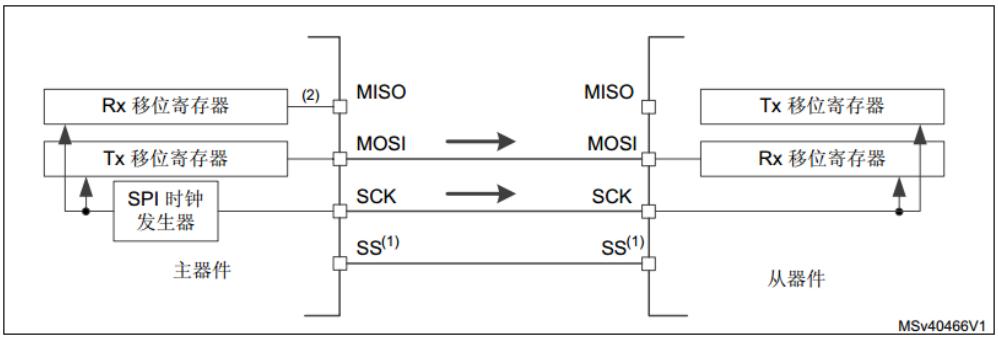
关于这个接线图要认识到以下几点：

- 更改通信方式时，要先禁止 SPI。
- 主器件的 MISO 和从器件的 MISO 不使用，可以继续用作标准 GPIO。

- 1KΩ 的接线电阻很有必要，因为当主器件和从器件的通信方向不是同步变化时，容易出现其中一个输出低电平，另一个输出高电平，造成短路。
- 这种单一的主从接线模式下，SS 引脚可以不使用。

◆ 单工模式

单工就是只有一种通信方向，即发送或者接收，SPI 总线的单工模式接线方式如下：

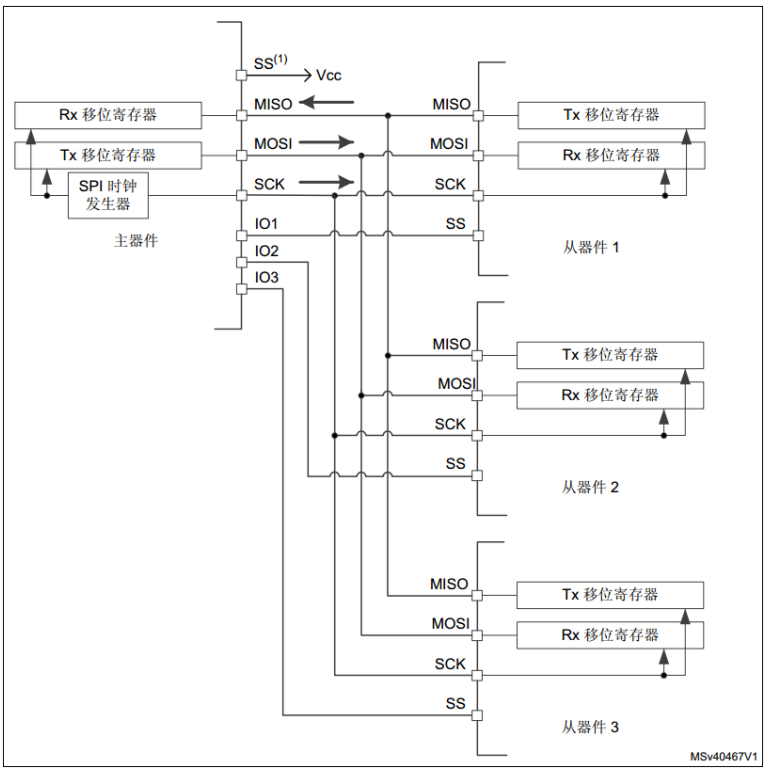


关于这个接线图要认识到以下几点：

- 未用到的 MOSI 或者 MISO 可以用作标准 GPIO。
- 这种单一的主从接线模式下，SS 引脚可以不使用。

72.2.4SPI 总线星型拓扑

SPI 总线星型拓扑用到的地方比较多，V7 开发板就是用的星型拓扑外接多种 SPI 器件：

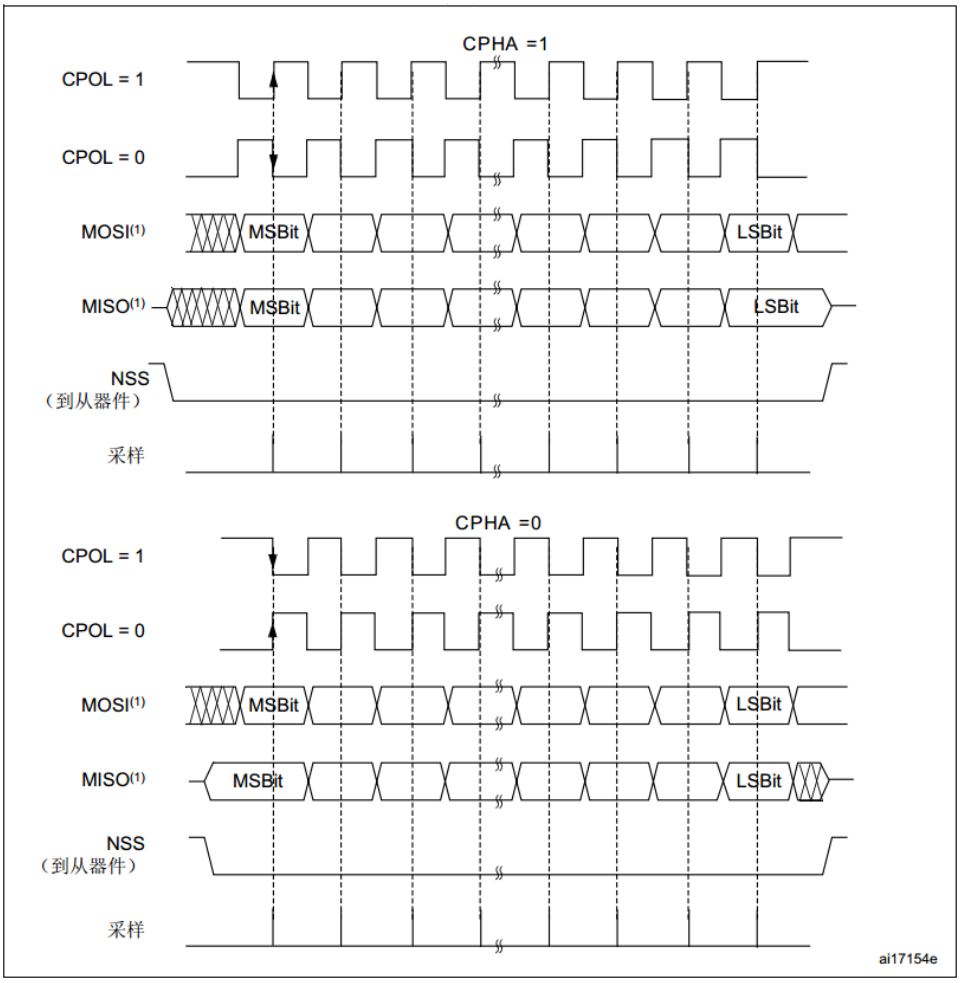


关于这个接线图，有以下几点需要大家了解：

- 主器件的 SS 引脚不使用，使用通用 GPIO 控制。为每个器件配一个 SS 引脚，方便单独片选控制。
- 从器件的 MISO 引脚要配置为复用开漏输出（很多外部芯片在未片选时，数据引脚是呈现高阻态）。

72.2.5 SPI 总线通信格式

SPI 总线主要有四种通信格式，由 CPOL 时钟极性和 CPHA 时钟相位控制：



四种通信格式如下：

- 当 CPOL = 1， CPHA = 1 时

  SCK 引脚在空闲状态处于高电平，SCK 引脚的第 2 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 0， CPHA = 1 时

  SCK 引脚在空闲状态处于低电平，SCK 引脚的第 2 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 1， CPHA = 0 时

  SCK 引脚在空闲状态处于高电平，SCK 引脚的第 1 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 0， CPHA = 0 时

  SCK 引脚在空闲状态处于低电平，SCK 引脚的第 1 个边沿捕获传输的第 1 个数据。

72.3 SPI 总线的 HAL 库用法

72.3.1 SPI 总线结构体 SPI_TypeDef

SPI 总线相关的寄存器是通过 HAL 库中的结构体 SPI_TypeDef 定义的，在 stm32h743xx.h 中可以找到这个类型定义：

```
typedef struct
{
  __IO uint32_t CR1;          /*!< SPI/I2S Control register 1,                    Address offset: 0x00 */
  __IO uint32_t CR2;          /*!< SPI Control register 2,                        Address offset: 0x04 */
  __IO uint32_t CFG1;         /*!< SPI Configuration register 1,                  Address offset: 0x08 */
  __IO uint32_t CFG2;         /*!< SPI Configuration register 2,                  Address offset: 0x0C */
  __IO uint32_t IER;          /*!< SPI/I2S Interrupt Enable register,             Address offset: 0x10 */
  __IO uint32_t SR;           /*!< SPI/I2S Status register,                       Address offset: 0x14 */
  __IO uint32_t IFCR;         /*!< SPI/I2S Interrupt/Status flags clear register, Address offset: 0x18 */
  uint32_t      RESERVED0;    /*!< Reserved, 0x1C                                                      */
  __IO uint32_t TXDR;         /*!< SPI/I2S Transmit data register,                Address offset: 0x20 */
  uint32_t      RESERVED1[3]; /*!< Reserved, 0x24-0x2C                                                 */
  __IO uint32_t RXDR;         /*!< SPI/I2S Receive data register,                 Address offset: 0x30 */
  uint32_t      RESERVED2[3]; /*!< Reserved, 0x34-0x3C                                                 */
  __IO uint32_t CRCPOLY;      /*!< SPI CRC Polynomial register,                   Address offset: 0x40 */
  __IO uint32_t TXCRC;        /*!< SPI Transmitter CRC register,                  Address offset: 0x44 */
  __IO uint32_t RXCRC;        /*!< SPI Receiver CRC register,                     Address offset: 0x48 */
  __IO uint32_t UDRDR;        /*!< SPI Underrun data register,                    Address offset: 0x4C */
  __IO uint32_t I2SCFGR;      /*!< I2S Configuration register,                    Address offset: 0x50 */

} SPI_TypeDef;
```

这个结构体的成员名称和排列次序和 CPU 的寄存器是一 一对应的。

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

下面我们看下 SPI 的定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE           (0x40000000UL)
#define D2_APB1PERIPH_BASE     PERIPH_BASE
#define D2_APB2PERIPH_BASE    (PERIPH_BASE + 0x00010000UL)
#define D3_APB1PERIPH_BASE    (PERIPH_BASE + 0x18000000UL)

#define SPI2_BASE             (D2_APB1PERIPH_BASE + 0x3800UL)
#define SPI3_BASE             (D2_APB1PERIPH_BASE + 0x3C00UL)
#define SPI1_BASE             (D2_APB2PERIPH_BASE + 0x3000UL)
#define SPI4_BASE             (D2_APB2PERIPH_BASE + 0x3400UL)
#define SPI5_BASE             (D2_APB2PERIPH_BASE + 0x5000UL)
#define SPI6_BASE             (D3_APB1PERIPH_BASE + 0x1400UL)

#define SPI1                ((SPI_TypeDef *) SPI1_BASE)
#define SPI2                ((SPI_TypeDef *) SPI2_BASE)
#define SPI3                ((SPI_TypeDef *) SPI3_BASE)
#define SPI4                ((SPI_TypeDef *) SPI4_BASE)
#define SPI5                ((SPI_TypeDef *) SPI5_BASE)
```

```
#define SPI6                (SPI_TypeDef *) SPI6_BASE) <----- 展开这个宏，(FLASH_TypeDef *)0x58001400
```

我们访问 SPI 的 CR1 寄存器可以采用这种形式：SPI->CR1 = 0。

72.3.2 SPI 总线初始化结构体 SPI_InitTypeDef

下面是 SPI 总线的初始化结构体，用到的地方比较多：

```
typedef struct
{
  uint32_t Mode;
  uint32_t Direction;
  uint32_t DataSize;
  uint32_t CLKPolarity;
  uint32_t CLKPhase;
  uint32_t NSS;
  uint32_t BaudRatePrescaler;
  uint32_t FirstBit;
  uint32_t TIMode;
  uint32_t CRCCalculation;
  uint32_t CRCPolynomial;
  uint32_t CRCLength;
  uint32_t NSSPMode;
  uint32_t NSSPolarity;
  uint32_t TxCRCInitializationPattern;
  uint32_t RxCRCInitializationPattern;
  uint32_t MasterSSIdleness;
  uint32_t MasterInterDataIdleness;
  uint32_t MasterReceiverAutoSusp;
  uint32_t MasterKeepIOState;
  uint32_t IOSwap;
} SPI_InitTypeDef;
```

下面将结构体成员逐一做个说明：

- Mode

用于设置工作在主机模式还是从机模式。

```
#define SPI_MODE_SLAVE                  (0x00000000UL)
#define SPI_MODE_MASTER                 SPI_CFG2_MASTER
```

- Direction

用于设置 SPI 工作在全双工，单工，还是半双工模式。

```
#define SPI_DIRECTION_2LINES            (0x00000000UL)      /* 全双工 */
#define SPI_DIRECTION_2LINES_TXONLY     SPI_CFG2_COMM_0     /* 单工，仅发送 */
#define SPI_DIRECTION_2LINES_RXONLY     SPI_CFG2_COMM_1     /* 单工，仅接收 */
#define SPI_DIRECTION_1LINE             SPI_CFG2_COMM       /* 半双工 */
```

- DataSize

用于设置 SPI 总线数据收发的位宽，支持 4-32bit。

```
#define SPI_DATASIZE_4BIT                               (0x00000003UL)
#define SPI_DATASIZE_5BIT                               (0x00000004UL)
#define SPI_DATASIZE_6BIT                               (0x00000005UL)
#define SPI_DATASIZE_7BIT                               (0x00000006UL)
#define SPI_DATASIZE_8BIT                               (0x00000007UL)
#define SPI_DATASIZE_9BIT                               (0x00000008UL)
#define SPI_DATASIZE_10BIT                              (0x00000009UL)
```

```
#define SPI_DATASIZE_11BIT                      (0x0000000AUL)
#define SPI_DATASIZE_12BIT                      (0x0000000BUL)
#define SPI_DATASIZE_13BIT                      (0x0000000CUL)
#define SPI_DATASIZE_14BIT                      (0x0000000DUL)
#define SPI_DATASIZE_15BIT                      (0x0000000EUL)
#define SPI_DATASIZE_16BIT                      (0x0000000FUL)
#define SPI_DATASIZE_17BIT                      (0x00000010UL)
#define SPI_DATASIZE_18BIT                      (0x00000011UL)
#define SPI_DATASIZE_19BIT                      (0x00000012UL)
#define SPI_DATASIZE_20BIT                      (0x00000013UL)
#define SPI_DATASIZE_21BIT                      (0x00000014UL)
#define SPI_DATASIZE_22BIT                      (0x00000015UL)
#define SPI_DATASIZE_23BIT                      (0x00000016UL)
#define SPI_DATASIZE_24BIT                      (0x00000017UL)
#define SPI_DATASIZE_25BIT                      (0x00000018UL)
#define SPI_DATASIZE_26BIT                      (0x00000019UL)
#define SPI_DATASIZE_27BIT                      (0x0000001AUL)
#define SPI_DATASIZE_28BIT                      (0x0000001BUL)
#define SPI_DATASIZE_29BIT                      (0x0000001CUL)
#define SPI_DATASIZE_30BIT                      (0x0000001DUL)
#define SPI_DATASIZE_31BIT                      (0x0000001EUL)
#define SPI_DATASIZE_32BIT                      (0x0000001FUL)
```

- CLKPolarity

用于设置空闲状态时，CLK 是高电平还是低电平。

```
#define SPI_POLARITY_LOW        (0x00000000UL)
#define SPI_POLARITY_HIGH       SPI_CFG2_CPOL
```

- NSS

用于设置 NSS 信号由硬件 NSS 引脚管理或者软件 SSI 位管理。

```
#define SPI_NSS_SOFT                            SPI_CFG2_SSM
#define SPI_NSS_HARD_INPUT                      (0x00000000UL)
#define SPI_NSS_HARD_OUTPUT                     SPI_CFG2_SSOE
```

- BaudRatePrescaler

用于设置 SPI 时钟分频，仅 SPI 工作在主控模式下起作用，对 SPI 从机模式不起作用。

```
#define SPI_BAUDRATEPRESCALER_2                 (0x00000000UL)
#define SPI_BAUDRATEPRESCALER_4                 (0x10000000UL)
#define SPI_BAUDRATEPRESCALER_8                 (0x20000000UL)
#define SPI_BAUDRATEPRESCALER_16                (0x30000000UL)
#define SPI_BAUDRATEPRESCALER_32                (0x40000000UL)
#define SPI_BAUDRATEPRESCALER_64                (0x50000000UL)
#define SPI_BAUDRATEPRESCALER_128               (0x60000000UL)
#define SPI_BAUDRATEPRESCALER_256               (0x70000000UL)
```

- FirstBit

用于设置数据传输从最高 bit 开始还是从最低 bit 开始。

```
#define SPI_FIRSTBIT_MSB                        (0x00000000UL)
#define SPI_FIRSTBIT_LSB                        SPI_CFG2_LSBFRST
```

- TIMode

用于设置是否使能 SPI 总线的 TI 模式。

```
#define SPI_TIMODE_DISABLE              (0x00000000UL)
#define SPI_TIMODE_ENABLE               SPI_CFG2_SP_0
```

- CRCCalculation

用于设置是否使能 CRC 计算。

```
#define SPI_CRCCALCULATION_DISABLE              (0x00000000UL)
#define SPI_CRCCALCULATION_ENABLE               SPI_CFG1_CRCEN
```

- CRCPolynomial

用于设置 CRC 计算使用的多项式，必须是奇数，范围 0 到 65535。

- CRCLength

用于设置 CRC 计算时的 CRC 长度。大小要与同属此结构体的 DataSize 一致。或是 DataSize 的整数倍。

```
#define SPI_CRC_LENGTH_DATASIZE                 (0x00000000UL)
#define SPI_CRC_LENGTH_4BIT                     (0x00030000UL)
#define SPI_CRC_LENGTH_5BIT                     (0x00040000UL)
#define SPI_CRC_LENGTH_6BIT                     (0x00050000UL)
#define SPI_CRC_LENGTH_7BIT                     (0x00060000UL)
#define SPI_CRC_LENGTH_8BIT                     (0x00070000UL)
#define SPI_CRC_LENGTH_9BIT                     (0x00080000UL)
#define SPI_CRC_LENGTH_10BIT                    (0x00090000UL)
#define SPI_CRC_LENGTH_11BIT                    (0x000A0000UL)
#define SPI_CRC_LENGTH_12BIT                    (0x000B0000UL)
#define SPI_CRC_LENGTH_13BIT                    (0x000C0000UL)
#define SPI_CRC_LENGTH_14BIT                    (0x000D0000UL)
#define SPI_CRC_LENGTH_15BIT                    (0x000E0000UL)
#define SPI_CRC_LENGTH_16BIT                    (0x000F0000UL)
#define SPI_CRC_LENGTH_17BIT                    (0x00100000UL)
#define SPI_CRC_LENGTH_18BIT                    (0x00110000UL)
#define SPI_CRC_LENGTH_19BIT                    (0x00120000UL)
#define SPI_CRC_LENGTH_20BIT                    (0x00130000UL)
#define SPI_CRC_LENGTH_21BIT                    (0x00140000UL)
#define SPI_CRC_LENGTH_22BIT                    (0x00150000UL)
#define SPI_CRC_LENGTH_23BIT                    (0x00160000UL)
#define SPI_CRC_LENGTH_24BIT                    (0x00170000UL)
#define SPI_CRC_LENGTH_25BIT                    (0x00180000UL)
#define SPI_CRC_LENGTH_26BIT                    (0x00190000UL)
#define SPI_CRC_LENGTH_27BIT                    (0x001A0000UL)
#define SPI_CRC_LENGTH_28BIT                    (0x001B0000UL)
#define SPI_CRC_LENGTH_29BIT                    (0x001C0000UL)
#define SPI_CRC_LENGTH_30BIT                    (0x001D0000UL)
#define SPI_CRC_LENGTH_31BIT                    (0x001E0000UL)
#define SPI_CRC_LENGTH_32BIT                    (0x001F0000UL)
```

- NSSPMode

用于设置是否使能 NSSP 信号，可以通过 SPIx_CR2 寄存器的 SSOM 位使能。注意，只有配置为摩托罗拉 SPI 主控模式时设置此成员才有用。

```
#define SPI_NSS_PULSE_DISABLE                   (0x00000000UL)
#define SPI_NSS_PULSE_ENABLE                    SPI_CFG2_SSOM
```

- NSSPolarity

用于设置 NSS 引脚上的高电平或者低电平作为激活电平。

```
#define SPI_NSS_POLARITY_LOW                    (0x00000000UL)
#define SPI_NSS_POLARITY_HIGH                    SPI_CFG2_SSIOP
```

- FifoThreshold

用于设置 SPI 的 FIFO 阀值。

```
#define SPI_FIFO_THRESHOLD_01DATA                (0x00000000UL)
#define SPI_FIFO_THRESHOLD_02DATA                (0x00000020UL)
#define SPI_FIFO_THRESHOLD_03DATA                (0x00000040UL)
#define SPI_FIFO_THRESHOLD_04DATA                (0x00000060UL)
#define SPI_FIFO_THRESHOLD_05DATA                (0x00000080UL)
#define SPI_FIFO_THRESHOLD_06DATA                (0x000000A0UL)
#define SPI_FIFO_THRESHOLD_07DATA                (0x000000C0UL)
#define SPI_FIFO_THRESHOLD_08DATA                (0x000000E0UL)
#define SPI_FIFO_THRESHOLD_09DATA                (0x00000100UL)
#define SPI_FIFO_THRESHOLD_10DATA                (0x00000120UL)
#define SPI_FIFO_THRESHOLD_11DATA                (0x00000140UL)
#define SPI_FIFO_THRESHOLD_12DATA                (0x00000160UL)
#define SPI_FIFO_THRESHOLD_13DATA                (0x00000180UL)
#define SPI_FIFO_THRESHOLD_14DATA                (0x000001A0UL)
#define SPI_FIFO_THRESHOLD_15DATA                (0x000001C0UL)
#define SPI_FIFO_THRESHOLD_16DATA                (0x000001E0UL)
```

- TxCRCInitializationPattern

发送 CRC 初始化模式。

```
#define SPI_CRC_INITIALIZATION_ALL_ZERO_PATTERN  (0x00000000UL)
#define SPI_CRC_INITIALIZATION_ALL_ONE_PATTERN   (0x00000001UL)
```

- RxCRCInitializationPattern

接收 CRC 初始化模式

```
#define SPI_CRC_INITIALIZATION_ALL_ZERO_PATTERN  (0x00000000UL)
#define SPI_CRC_INITIALIZATION_ALL_ONE_PATTERN   (0x00000001UL)
```

- MasterSSIdleness

在主模式下插入到 SS 有效边沿和第一个数据开始之间的额外延迟，单位 SPI 时钟周期个数。

```
#define SPI_MASTER_SS_IDLENESS_00CYCLE           (0x00000000UL)
#define SPI_MASTER_SS_IDLENESS_01CYCLE           (0x00000001UL)
#define SPI_MASTER_SS_IDLENESS_02CYCLE           (0x00000002UL)
#define SPI_MASTER_SS_IDLENESS_03CYCLE           (0x00000003UL)
#define SPI_MASTER_SS_IDLENESS_04CYCLE           (0x00000004UL)
#define SPI_MASTER_SS_IDLENESS_05CYCLE           (0x00000005UL)
#define SPI_MASTER_SS_IDLENESS_06CYCLE           (0x00000006UL)
#define SPI_MASTER_SS_IDLENESS_07CYCLE           (0x00000007UL)
#define SPI_MASTER_SS_IDLENESS_08CYCLE           (0x00000008UL)
#define SPI_MASTER_SS_IDLENESS_09CYCLE           (0x00000009UL)
#define SPI_MASTER_SS_IDLENESS_10CYCLE           (0x0000000AUL)
#define SPI_MASTER_SS_IDLENESS_11CYCLE           (0x0000000BUL)
#define SPI_MASTER_SS_IDLENESS_12CYCLE           (0x0000000CUL)
#define SPI_MASTER_SS_IDLENESS_13CYCLE           (0x0000000DUL)
#define SPI_MASTER_SS_IDLENESS_14CYCLE           (0x0000000EUL)
#define SPI_MASTER_SS_IDLENESS_15CYCLE           (0x0000000FUL)
```

- MasterInterDataIdleness

主模式下在两个连续数据帧之间插入的最小时间延迟，单位 SPI 时钟周期个数。

```
#define SPI_MASTER_RX_AUTOSUSP_DISABLE           (0x00000000UL)
#define SPI_MASTER_RX_AUTOSUSP_ENABLE            SPI_CR1_MASRX
```

- MasterReceiverAutoSusp

用于控制主器件接收器模式下的连续 SPI 传输以及自动管理，以避免出现上溢情况。

```
#define SPI_MASTER_RX_AUTOSUSP_DISABLE           (0x00000000UL)
#define SPI_MASTER_RX_AUTOSUSP_ENABLE            SPI_CR1_MASRX
```

- MasterKeepIOState

禁止 SPI 后，SPI 相关引脚保持当前状态，以防止出现毛刺。在从模式下，该位不应该使用。

```
#define SPI_MASTER_KEEP_IO_STATE_DISABLE              (0x00000000UL)
#define SPI_MASTER_KEEP_IO_STATE_ENABLE               SPI_CFG2_AFCNTR
```

- IOSwap

用于交换 MISO 和 MOSI 引脚。

```
#define SPI_IO_SWAP_DISABLE                           (0x00000000UL)
#define SPI_IO_SWAP_ENABLE                            SPI_CFG2_IOSWP
```

72.3.3SPI 总线句柄结构体 SPI_HandleTypeDef

下面是 SPI 总线的初始化结构体，用到的地方比较多：

```
typedef struct __SPI_HandleTypeDef
{
  SPI_TypeDef                *Instance;
  SPI_InitTypeDef            Init;
  uint8_t                    *pTxBuffPtr;
  uint16_t                   TxXferSize;
  __IO uint16_t              TxXferCount;
  uint8_t                    *pRxBuffPtr;
  uint16_t                   RxXferSize;
  __IO uint16_t              RxXferCount;
  uint32_t                   CRCSize;
  void (*RxISR)(struct __SPI_HandleTypeDef *hspi);
  void (*TxISR)(struct __SPI_HandleTypeDef *hspi);
  DMA_HandleTypeDef          *hdmatx;
  DMA_HandleTypeDef          *hdmarx;
  HAL_LockTypeDef            Lock;
  __IO HAL_SPI_StateTypeDef  State;
  __IO uint32_t              ErrorCode;
#if defined(USE_SPI_RELOAD_TRANSFER)
  SPI_ReloadTypeDef          Reload;
#endif

#if (USE_HAL_SPI_REGISTER_CALLBACKS == 1UL)
  void (* TxCpltCallback)(struct __SPI_HandleTypeDef *hspi);
  void (* RxCpltCallback)(struct __SPI_HandleTypeDef *hspi);
  void (* TxRxCpltCallback)(struct __SPI_HandleTypeDef *hspi);
  void (* TxHalfCpltCallback)(struct __SPI_HandleTypeDef *hspi);
  void (* RxHalfCpltCallback)(struct __SPI_HandleTypeDef *hspi);
  void (* TxRxHalfCpltCallback)(struct __SPI_HandleTypeDef *hspi);
  void (* ErrorCallback)(struct __SPI_HandleTypeDef *hspi);
  void (* AbortCpltCallback)(struct __SPI_HandleTypeDef *hspi);
  void (* MspInitCallback)(struct __SPI_HandleTypeDef *hspi);
  void (* MspDeInitCallback)(struct __SPI_HandleTypeDef *hspi);
#endif
} SPI_HandleTypeDef;
```

注意事项：

条件编译 USE_HAL_SPI_REGISTER_CALLBACKS 用来设置使用自定义回调还是使用默认回调，此定义一般放在 stm32h7xx_hal_conf.h 文件里面设置：

```
#define  USE_HAL_SPI_REGISTER_CALLBACKS  1
```

通过函数 HAL_SPI_RegisterCallback 注册回调，取消注册使用函数 HAL_SPI_UnRegisterCallback。

这里重点介绍下面几个参数，其它参数主要是 HAL 库内部使用和自定义回调函数。

- SPI_TypeDef *Instance

  这个参数是寄存器的例化，方便操作寄存器，比如使能 SPI1。

  SET_BIT(SPI1 ->CR1， SPI_CR1_SPE)。

- SPI_InitTypeDef Init

  这个参数是用户接触最多的，在本章节 3.2 小节已经进行了详细说明。

- DMA_HandleTypeDef *hdmatx
- DMA_HandleTypeDef *hdmarx

  用于 SPI 句柄关联 DMA 句柄，方便操作调用。

72.4 SPI 总线源文件 stm32h7xx_hal_spi.c

此文件涉及到的函数较多，这里把几个常用的函数做个说明：

- HAL_SPI_Init
- HAL_SPI_DeInit
- HAL_SPI_TransmitReceive
- HAL_SPI_TransmitReceive_IT
- HAL_SPI_TransmitReceive_DMA

72.4.1 函数 HAL_SPI_Init

函数原型：

```
HAL_StatusTypeDef HAL_SPI_Init(SPI_HandleTypeDef *hspi)
{
  uint32_t crc_length = 0UL;
  uint32_t packet_length;

  /* 省略未写 */

  /* 如果数据位宽大于 16bit，必须是 SPI1，SPI2 或者 SPI3，而 SPI4，SPI5 和 SPI6 不支持大于 16bit */
  if ((!IS_SPI_HIGHEND_INSTANCE(hspi->Instance)) && (hspi->Init.DataSize > SPI_DATASIZE_16BIT))
  {
    return HAL_ERROR;
  }

  /* SPI1，SPI2 和 SPI3 的 FIFO 大小是 16*8bit，而 SPI4，SPI5 和 SPI6 的 FIFO 大小是 8*8bit
     这里是查看设置的缓冲大小是否超出了 FIFO 支持的大小。
   */
  packet_length = SPI_GetPacketSize(hspi);
  if ((((!IS_SPI_HIGHEND_INSTANCE(hspi->Instance)) && (packet_length > SPI_LOWEND_FIFO_SIZE)) ||
       ((IS_SPI_HIGHEND_INSTANCE(hspi->Instance)) && (packet_length > SPI_HIGHEND_FIFO_SIZE)))
```

```
  {
    return HAL_ERROR;
  }

#if (USE_SPI_CRC != 0UL)
    /* 省略未写 */
#endif

  if (hspi->State == HAL_SPI_STATE_RESET)
  {
    /* 解锁 */
    hspi->Lock = HAL_UNLOCKED;

    /* 使用自定义回调 */
#if (USE_HAL_SPI_REGISTER_CALLBACKS == 1UL)
    /* 设置默认回调函数 */
    hspi->TxCpltCallback       = HAL_SPI_TxCpltCallback;       /* Legacy weak TxCpltCallback       */
    hspi->RxCpltCallback       = HAL_SPI_RxCpltCallback;       /* Legacy weak RxCpltCallback       */
    hspi->TxRxCpltCallback     = HAL_SPI_TxRxCpltCallback;     /* Legacy weak TxRxCpltCallback     */
    hspi->TxHalfCpltCallback   = HAL_SPI_TxHalfCpltCallback;   /* Legacy weak TxHalfCpltCallback   */
    hspi->RxHalfCpltCallback   = HAL_SPI_RxHalfCpltCallback;   /* Legacy weak RxHalfCpltCallback   */
    hspi->TxRxHalfCpltCallback = HAL_SPI_TxRxHalfCpltCallback; /* Legacy weak TxRxHalfCpltCallback */
    hspi->ErrorCallback        = HAL_SPI_ErrorCallback;        /* Legacy weak ErrorCallback        */
    hspi->AbortCpltCallback    = HAL_SPI_AbortCpltCallback;    /* Legacy weak AbortCpltCallback    */

    if (hspi->MspInitCallback == NULL)
    {
      hspi->MspInitCallback = HAL_SPI_MspInit;
    }

    /* 初始化地址硬件: GPIO, CLOCK, NVIC... */
    hspi->MspInitCallback(hspi);
#else
    /* 初始化底层硬件: GPIO, CLOCK, NVIC... */
    HAL_SPI_MspInit(hspi);
#endif
  }

  hspi->State = HAL_SPI_STATE_BUSY;

  /* 禁止 SPI 外设 */
  __HAL_SPI_DISABLE(hspi);

  /*---------------------- SPIx CR1 & CR2 配置---------------------*/
  if ((hspi->Init.NSS == SPI_NSS_SOFT) && (hspi->Init.Mode == SPI_MODE_MASTER) && (hspi->Init.NSSPolarity ==
       SPI_NSS_POLARITY_LOW))
  {
      SET_BIT(hspi->Instance->CR1, SPI_CR1_SSI);
  }

  /* SPIx CFG1 配置 */
  WRITE_REG(hspi->Instance->CFG1, (hspi->Init.BaudRatePrescaler | hspi->Init.CRCCalculation | crc_length |
                                   hspi->Init.FifoThreshold     | hspi->Init.DataSize));

  /* SPIx CFG2 配置 */
  WRITE_REG(hspi->Instance->CFG2, (hspi->Init.NSSPMode      | hspi->Init.TIMode           |
                                   hspi->Init.NSSPolarity   |
                                   hspi->Init.NSS           | hspi->Init.CLKPolarity      |
```

```
                                  hspi->Init.CLKPhase      |
                                  hspi->Init.FirstBit      | hspi->Init.Mode             |
                                  hspi->Init.MasterInterDataIdleness |
                                  hspi->Init.Direction     | hspi->Init.MasterSSIdleness |
                                  hspi->Init.IOSwap));

#if (USE_SPI_CRC != 0UL)
  /*---------------------------- SPIx CRC 配置 ------------------*/
  /* 配置 SPI CRC */
  if (hspi->Init.CRCCalculation == SPI_CRCCALCULATION_ENABLE)
  {
    /* 初始化 TX CRC 初始值 */
    if (hspi->Init.TxCRCInitializationPattern == SPI_CRC_INITIALIZATION_ALL_ONE_PATTERN)
    {
      SET_BIT(hspi->Instance->CR1, SPI_CR1_TCRCINI);
    }
    else
    {
      CLEAR_BIT(hspi->Instance->CR1, SPI_CR1_TCRCINI);
    }

    /* 初始化 RXCRC 初始值 */
    if (hspi->Init.RxCRCInitializationPattern == SPI_CRC_INITIALIZATION_ALL_ONE_PATTERN)
    {
      SET_BIT(hspi->Instance->CR1, SPI_CR1_RCRCINI);
    }
    else
    {
      CLEAR_BIT(hspi->Instance->CR1, SPI_CR1_RCRCINI);
    }

    /* 使能 33/17 bit CRC 计算 */
    if ((((!IS_SPI_HIGHEND_INSTANCE(hspi->Instance)) && (crc_length == SPI_CRC_LENGTH_16BIT)) ||
         ((IS_SPI_HIGHEND_INSTANCE(hspi->Instance))  && (crc_length == SPI_CRC_LENGTH_32BIT)))
    {
      SET_BIT(hspi->Instance->CR1, SPI_CR1_CRC33_17);
    }
    else
    {
      CLEAR_BIT(hspi->Instance->CR1, SPI_CR1_CRC33_17);
    }

    /* 写 CRC 多项式到 SPI 寄存器 */
    WRITE_REG(hspi->Instance->CRCPOLY, hspi->Init.CRCPolynomial);
  }
#endif

  /* SPI 从模式，下溢配置 */
  if (hspi->Init.Mode == SPI_MODE_SLAVE)
  {
    /* 设置默认下溢配置 */
#if (USE_SPI_CRC != 0UL)
    if (hspi->Init.CRCCalculation == SPI_CRCCALCULATION_DISABLE)
#endif
    {
      MODIFY_REG(hspi->Instance->CFG1, SPI_CFG1_UDRDET, SPI_CFG1_UDRDET_0);
    }
    MODIFY_REG(hspi->Instance->CFG1, SPI_CFG1_UDRCFG, SPI_CFG1_UDRCFG_1);
```

```
  }

#if defined(SPI_I2SCFGR_I2SMOD)
  CLEAR_BIT(hspi->Instance->I2SCFGR, SPI_I2SCFGR_I2SMOD);
#endif

  /* 确保 AFCNTR bit 由 SPI 主机模式管理 */
  if ((hspi->Init.Mode & SPI_MODE_MASTER) == SPI_MODE_MASTER)
  {
    /* Alternate function GPIOs control */
    MODIFY_REG(hspi->Instance->CFG2, SPI_CFG2_AFCNTR, (hspi->Init.MasterKeepIOState));
  }

  hspi->ErrorCode = HAL_SPI_ERROR_NONE;
  hspi->State     = HAL_SPI_STATE_READY;

  return HAL_OK;
}
```

函数描述：

此函数用于初始化 SPI。

函数参数：

- 第 1 个参数是 SPI_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

注意事项：

1. 函数 HAL_SPI_MspInit 用于初始化 SPI 的底层时钟、引脚等功能。需要用户自己在此函数里面实现具体的功能。由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
2. 如果形参 hspi 的结构体成员 State 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 SPI_HandleTypeDef SpiHandle。

   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_SPI_STATE_RESET = 0x00U。

   解决办法有三

   方法 1：用户自己初始化 SPI 和涉及到的 GPIO 等。

   方法 2：定义 SPI_HandleTypeDef SpiHandle 为全局变量。

   方法 3：下面的方法

```
if(HAL_SPI_DeInit(&SpiHandle) != HAL_OK)
{
    Error_Handler();
}
if(HAL_SPI_Init(&SpiHandle) != HAL_OK)
{
    Error_Handler();
}
```

使用举例：

```
SPI_HandleTypeDef hspi = {0};

/* 设置 SPI 参数 */
hspi.Instance               = SPIx;                     /* 例化 SPI */
hspi.Init.BaudRatePrescaler = _BaudRatePrescaler;       /* 设置波特率 */
hspi.Init.Direction         = SPI_DIRECTION_2LINES;     /* 全双工 */
hspi.Init.CLKPhase          = _CLKPhase;                /* 配置时钟相位 */
hspi.Init.CLKPolarity       = _CLKPolarity;             /* 配置时钟极性 */
hspi.Init.DataSize          = SPI_DATASIZE_8BIT;        /* 设置数据宽度 */
hspi.Init.FirstBit          = SPI_FIRSTBIT_MSB;         /* 数据传输先传高位 */
hspi.Init.TIMode            = SPI_TIMODE_DISABLE;       /* 禁止 TI 模式  */
hspi.Init.CRCCalculation    = SPI_CRCCALCULATION_DISABLE;       /* 禁止 CRC */
hspi.Init.CRCPolynomial     = 7;                                /* 禁止 CRC 后，此位无效 */
hspi.Init.CRCLength         = SPI_CRC_LENGTH_8BIT;              /* 禁止 CRC 后，此位无效 */
hspi.Init.NSS               = SPI_NSS_SOFT;                     /* 使用软件方式管理片选引脚 */
hspi.Init.FifoThreshold     = SPI_FIFO_THRESHOLD_01DATA;        /* 设置 FIFO 大小是一个数据项 */
hspi.Init.NSSPMode          = SPI_NSS_PULSE_DISABLE;            /* 禁止脉冲输出 */
hspi.Init.MasterKeepIOState = SPI_MASTER_KEEP_IO_STATE_ENABLE; /* 禁止 SPI 后，SPI 相关引脚保持当前状态 */
hspi.Init.Mode                  = SPI_MODE_MASTER;             /* SPI 工作在主控模式 */

if (HAL_SPI_Init(&hspi) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

72.4.2 函数 HAL_SPI_DeInit

函数原型：

```
HAL_StatusTypeDef HAL_SPI_DeInit(SPI_HandleTypeDef *hspi)
{
  /* 检测 SPI 句柄是否有效 */
  if (hspi == NULL)
  {
    return HAL_ERROR;
  }

  /* 检查 SPI 例化参数 */
  assert_param(IS_SPI_ALL_INSTANCE(hspi->Instance));

  hspi->State = HAL_SPI_STATE_BUSY;

  /* 禁止 SPI 外设时钟 */
  __HAL_SPI_DISABLE(hspi);

#if (USE_HAL_SPI_REGISTER_CALLBACKS == 1UL)
  if (hspi->MspDeInitCallback == NULL)
  {
    hspi->MspDeInitCallback = HAL_SPI_MspDeInit;
  }

  /* 复位底层硬件: GPIO, CLOCK, NVIC... */
  hspi->MspDeInitCallback(hspi);
#else
  /* 复位底层硬件: GPIO, CLOCK, NVIC... */
  HAL_SPI_MspDeInit(hspi);
```

```
#endif

  /* 设置无错误，复位状态标记 */
  hspi->ErrorCode = HAL_SPI_ERROR_NONE;
  hspi->State = HAL_SPI_STATE_RESET;

  /* 解锁 SPI */
  __HAL_UNLOCK(hspi);

  return HAL_OK;
}
```

函数描述：

用于复位 SPI 总线初始化。

函数参数：

- 第 1 个参数是 SPI_HandleTypeDef 类型结构体指针变量。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中

72.4.3 函数 HAL_SPI_TransmitReceive

函数原型：

```
HAL_StatusTypeDef HAL_SPI_TransmitReceive(SPI_HandleTypeDef *hspi, uint8_t *pTxData, uint8_t *pRxData,
                                          uint16_t Size, uint32_t Timeout)
{

   /* 省略未写 */

  /* 大于 16bit 的数据收发 */
  if (hspi->Init.DataSize > SPI_DATASIZE_16BIT)
  {
      /* 省略未写 */
  }
  /* 大于 8bit，小于 16bi 的数据收发 */
  else if (hspi->Init.DataSize > SPI_DATASIZE_8BIT)
  {
     /* 省略未写 */
  }
  /* 小于等于 8bit 的数据收发 */
  else
  {
      /* 省略未写 */
  }

}
```

函数描述：

此函数主要用于 SPI 数据收发，全双工查询方式。

函数参数：

- 第 1 个参数是 SPI_HandleTypeDef 类型结构体指针变量。

- 第 2 个参数是发送数据缓冲地址。
- 第 3 个参数是接收数据缓冲地址。
- 第 4 个参数是传输的数据大小，单位字节个数。
- 第 5 个参数是传输过程的溢出时间，单位 ms。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

使用举例：

```
SPI_HandleTypeDef hspi = {0};

if(HAL_SPI_TransmitReceive(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen, 1000000) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

72.4.4函数 HAL_SPI_TransmitReceive_IT

函数原型：

```
HAL_StatusTypeDef HAL_SPI_TransmitReceive_IT(SPI_HandleTypeDef *hspi, uint8_t *pTxData, uint8_t *pRxData,
                                             uint16_t Size)
{
  /* 省略未写 */

  /* 设置传输参数 */
  hspi->ErrorCode   = HAL_SPI_ERROR_NONE;
  hspi->pTxBuffPtr  = (uint8_t *)pTxData;
  hspi->TxXferSize  = Size;
  hspi->TxXferCount = Size;
  hspi->pRxBuffPtr  = (uint8_t *)pRxData;
  hspi->RxXferSize  = Size;
  hspi->RxXferCount = Size;

  /* 设置中断处理 */
  if (hspi->Init.DataSize > SPI_DATASIZE_16BIT)
  {
    hspi->TxISR     = SPI_TxISR_32BIT;
    hspi->RxISR     = SPI_RxISR_32BIT;
  }
  else if (hspi->Init.DataSize > SPI_DATASIZE_8BIT)
  {
    hspi->RxISR     = SPI_RxISR_16BIT;
    hspi->TxISR     = SPI_TxISR_16BIT;
  }
  else
  {
    hspi->RxISR     = SPI_RxISR_8BIT;
    hspi->TxISR     = SPI_TxISR_8BIT;
  }

  /* 设置当前传输数据大小 */
  MODIFY_REG(hspi->Instance->CR2, SPI_CR2_TSIZE, Size);
```

```
  /* 使能 SPI 外设 */
  __HAL_SPI_ENABLE(hspi);

  /* 使能各种中断标志 */
  __HAL_SPI_ENABLE_IT(hspi, (SPI_IT_EOT | SPI_IT_RXP | SPI_IT_TXP | SPI_IT_DXP | SPI_IT_UDR | SPI_IT_OVR |
                                                                  SPI_IT_FRE | SPI_IT_MODF | SPI_IT_TSERF));

  if (hspi->Init.Mode == SPI_MODE_MASTER)
  {
    /* 启动传输 */
    SET_BIT(hspi->Instance->CR1, SPI_CR1_CSTART);
  }

  /* 解锁 */
  __HAL_UNLOCK(hspi);
  return errorcode;
}
```

函数描述：

此函数主要用于 SPI 数据收发，全双工中断方式。

函数参数：

- 第 1 个参数是 SPI_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是发送数据缓冲地址。
- 第 3 个参数是接收数据缓冲地址。
- 第 4 个参数是传输的数据大小，单位字节个数。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

使用举例：

```
SPI_HandleTypeDef hspi = {0};

if(HAL_SPI_TransmitReceive_IT(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen) != HAL_OK)
{
    Error_Handler(__FILE__, __LINE__);
}
```

72.4.5 函数 HAL_SPI_TransmitReceive_DMA

函数原型：

```
HAL_StatusTypeDef HAL_SPI_TransmitReceive_DMA(SPI_HandleTypeDef *hspi, uint8_t *pTxData, uint8_t *pRxData,
                                              uint16_t Size)
{
  /* 省略未写 */

  /* 注意 DMA 的位宽和对齐设置 */
  if (((hspi->Init.DataSize > SPI_DATASIZE_16BIT) && (hspi->hdmarx->Init.MemDataAlignment !=
                                                      DMA_MDATAALIGN_WORD))  ||
      ((hspi->Init.DataSize > SPI_DATASIZE_8BIT) && ((hspi->hdmarx->Init.MemDataAlignment !=
      DMA_MDATAALIGN_HALFWORD) && (hspi->hdmarx->Init.MemDataAlignment != DMA_MDATAALIGN_WORD))))
```

```
  {
  }

  /* 调整 DMA 对齐和数据大小 */
  if (hspi->Init.DataSize <= SPI_DATASIZE_8BIT)
  {
     /* 省略未写 */
  }
  else if (hspi->Init.DataSize <= SPI_DATASIZE_16BIT)
  {
     /* 省略未写 */
  }
  else
  {
      /* 省略未写 */
  }

  /*  DMA 接收配置 */
  if (HAL_OK != HAL_DMA_Start_IT(hspi->hdmarx, (uint32_t)&hspi->Instance->RXDR, (uint32_t)hspi->pRxBuffPtr,
                                                                                hspi->RxXferCount))
  {

  }

  /* DMA 发送配置 */
  if (HAL_OK != HAL_DMA_Start_IT(hspi->hdmatx, (uint32_t)hspi->pTxBuffPtr, (uint32_t)&hspi->Instance->TXDR,
                                                                            hspi->TxXferCount))
  {
  }

  /* 省略未写 */
}
```

函数描述：

此函数主要用于 SPI 数据收发，全双工 DMA 方式。

函数参数：

- 第 1 个参数是 SPI_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是发送数据缓冲地址。
- 第 3 个参数是接收数据缓冲地址。
- 第 4 个参数是传输的数据大小，单位字节个数。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

使用举例：

```
SPI_HandleTypeDef hspi = {0};

if(HAL_SPI_TransmitReceive_DMA(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen) != HAL_OK)

{
    Error_Handler(__FILE__, __LINE__);
}
```

72.5 总结

本章节就为大家讲解这么多，要熟练掌握 SPI 总线的查询，中断和 DMA 方式的实现，因为基于 SPI 接口的外设芯片很多，熟练后，可以方便的驱动各种 SPI 接口芯片，以便选择合适的驱动方式。

第73章 STM32H7 的 SPI 总线应用之驱动 W25QXX（支持查询，中断和 DMA）

本章节为大家讲解标准 SPI 接线方式驱动 W25QXX，实现了查询，中断和 DMA 三种方式。

73.1 初学者重要提示
73.2 W25QXX 硬件设计
73.4 W25QXX 关键知识点整理（重要）
73.5 W25QXX 驱动设计
73.6 SPI 总线板级支持包（bsp_spi_bus.c）
73.7 W25QXX 板级支持包（bsp_spi_flash.c）
73.8 使用例程设计框架
73.9 实验例程说明（MDK）
73.10 实验例程说明（IAR）
73.11 总结

73.1 初学者重要提示

- 学习本章节前，务必优先学习第 72 章。
- W25Q64FV 属于 NOR 型 Flash 存储芯片。
- W25Q64JV 手册下载地址：链接 （这是一个超链接），当前章节配套例子的 Doc 文件件里面也有存放。

2Gb | 1Gb | 512Mb | 256Mb | 128Mb | 64Mb | 32Mb | 16Mb | 8Mb | 4Mb | 2Mb | 1Mb | 512Kb

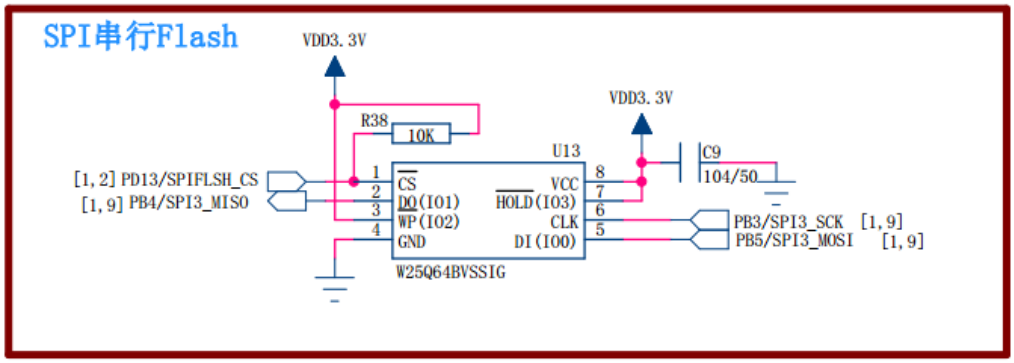
Part No.	Voltage	Speed	Temp.	Features	Automotive	Status	Datasheet
W25Q64JV	2.7V - 3.6V	133MHz	-40°C ~ 85°C / -40°C ~ 105°C	SPI 4 I/O Fixed	P	P	
W25Q64JV_DTR	2.7V - 3.6V	133MHz	-40°C ~ 85°C / -40°C ~ 105°C	SPI / QPI / DTR	P	P	
W25Q64FV	2.7V - 3.6V	104MHz	-40°C ~ 85°C	SPI / QPI	N	N	
W25Q64JW	1.7V - 1.95V	133MHz	-40°C ~ 85°C / -40°C ~ 105°C	SPI 4 I/O Fixed	P	S	
W25Q64JW_DTR	1.7V - 1.95V	133MHz	-40°C ~ 85°C / -40°C ~ 105°C	SPI / QPI / DTR	P	S	
W25R64JW	1.7V - 1.95V	104MHz	-40°C ~ 85°C	SPI with Single, Dual, Quad I/O RPMC	-	S	Contact us
W25R64JV	2.7V - 3.6V	133MHz	-40°C ~ 85°C	SPI with Single, Dual, Quad I/O RPMC	-	P	
W25R64FV	2.7V - 3.6V	104MHz	-40°C ~ 85°C	SPI with Single, Dual, Quad I/O RPMC	-	N	
W25Q64FW	1.65V - 1.95V	104MHz	-40°C ~ 85°C	SPI / QPI	N	N	

- 本章第 3 小节整理的知识点比较重要，务必要了解下，特别是页编程和页回卷。

- 对 SPI Flash W25QXX 的不同接线方式（1 线，2 线或者 4 线，这里的线是指的数据线），编程命令是不同的。
- W25Q64JV 最高支持 133MHz，但最高读命令 03H 速度是 50MHz。
- 文件 bsp_spi_bus.c 文件公共的总线驱动文件，支持串行 FLASH、TSC2046、VS1053、AD7705、ADS1256 等 SPI 设备的配置。
- 函数 sf_WriteBuffer 不需要用户做擦除，会自动执行擦除功能，支持任意大小，任意地址，不超过芯片容量即可。

73.2 W25QXX 硬件设计

STM32H7 驱动 W25QXX 的硬件设计如下：

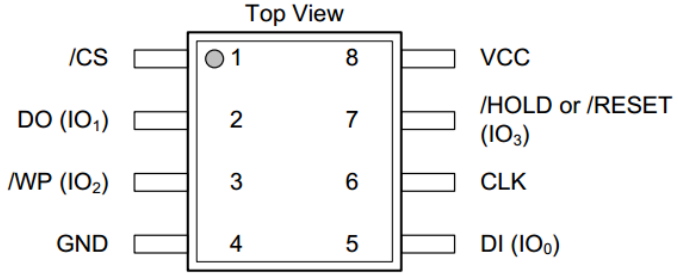


关于这个原理图，要了解到以下几个知识：

- V7 开发板实际外接的芯片是 W25Q64JV。
- CS 片选最好接上拉电阻，防止意外操作。
- 这里的 PB3，PB4 和 PB5 引脚可以复用 SPI1，SPI3 和 SPI6。实际应用中是复用的 SPI1。
- W25Q64 的 WP 引脚用于写保护，低电平有效性，当前是直接高电平。
- HOLD 引脚也是低电平有效，当前是将其接到高电平。此引脚的作用是 CS 片选低电平时，DO 引脚输出高阻，忽略 CLK 和 DI 引脚上的信号。

73.3 W25QXX 关键知识点整理（重要）

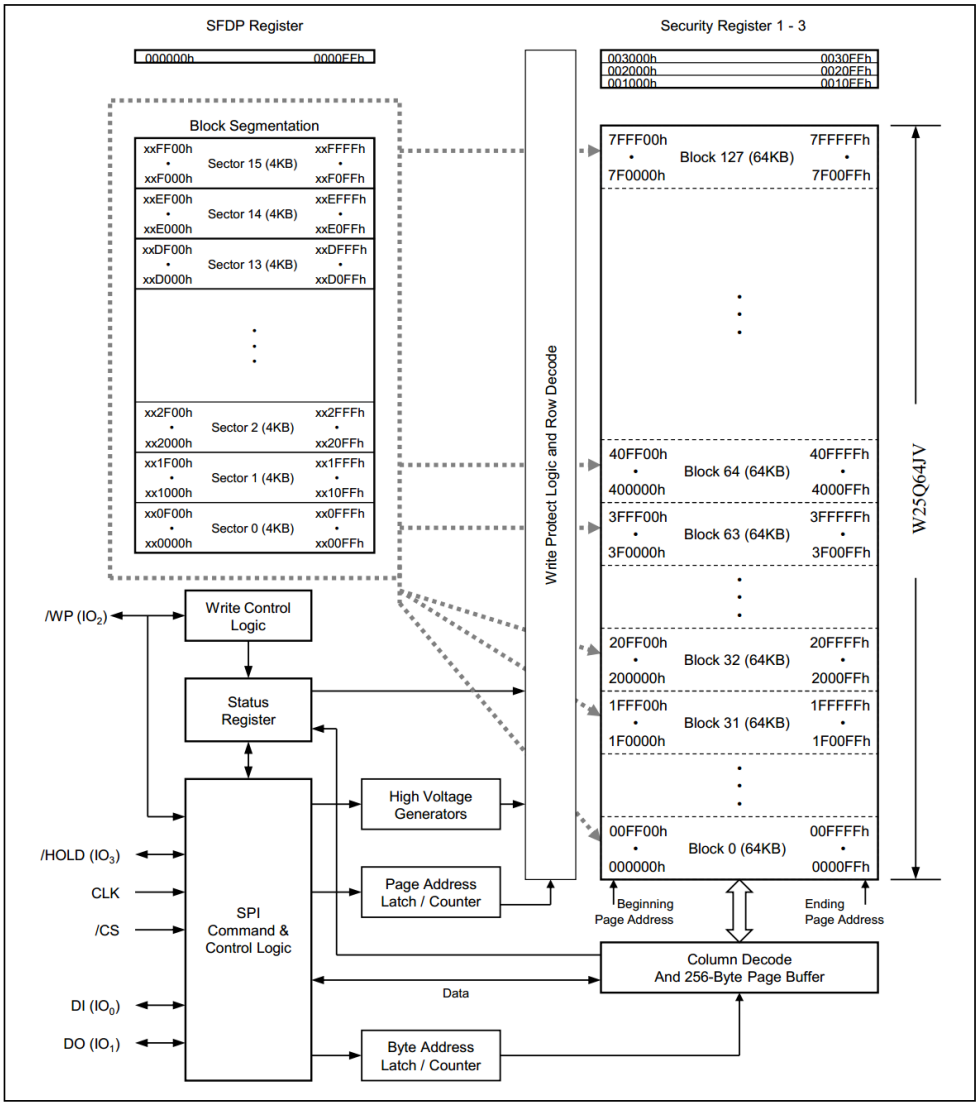
驱动 W25QXX 前要先了解下这个芯片的相关信息。



73.3.1W25QXX 基础信息

- W25Q64FV 的容量是 8MB（256Mbit）。
- W25Q64FV 支持标准 SPI（单线 SPI），用到引脚 CLK、CS，DI 和 DO 引脚。
  支持两线 SPI，用到引脚 CLK、CS、IO_0、IO_1 。
  支持四线 SPI，用到引脚 CLK、CS、IO_0、IO_1，IO_2、IO_3。
  （注：这里几线的意思是几个数据线）。
- W25Q64FV 支持的最高时钟是 133MHz。
- 每个扇区最少支持 10 万次擦写，可以保存 20 年数据。
- 页大小是 256 字节，支持页编程，也就是一次编写 256 个字节，也可以一个一个编写。
- 支持 4KB 为单位的扇区擦除，也可以 32KB 或者 64KB 为单位的擦除。

整体框图如下：



W25Q64FV：

- 有 128 个 Block，每个 Block 大小 64KB。
- 每个 Block 有 16 个 Sector，每个 Sector 大小 4KB。
- 每个 Sector 有 16 个 Page，每个 Page 大小是 256 字节。

73.3.2 W25QXX 命令

使用 W25Q 的接线方式不同，使用的命令也有所不同，使用的时候务必要注意，当前我们使用的标准 SPI，即单线 SPI，使用的命令如下：

Data Input Output	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
Number of Clock(1-1-1)	8	8	8	8	8	8	8
Write Enable	06h						
Volatile SR Write Enable	50h						
Write Disable	04h						
Release Power-down / ID	ABh	Dummy	Dummy	Dummy	(ID7-ID0)(2)		
Manufacturer/Device ID	90h	Dummy	Dummy	00h	(MF7-MF0)	(ID7-ID0)	
JEDEC ID	9Fh	(MF7-MF0)	(ID15-ID8)	(ID7-ID0)			
Read Unique ID	4Bh	Dummy	Dummy	Dummy	Dummy	(UID63-0)	
Read Data	03h	A23-A16	A15-A8	A7-A0	(D7-D0)		
Fast Read	0Bh	A23-A16	A15-A8	A7-A0	Dummy	(D7-D0)	
Page Program	02h	A23-A16	A15-A8	A7-A0	D7-D0	D7-D0(3)	
Sector Erase (4KB)	20h	A23-A16	A15-A8	A7-A0			
Block Erase (32KB)	52h	A23-A16	A15-A8	A7-A0			
Block Erase (64KB)	D8h	A23-A16	A15-A8	A7-A0			
Chip Erase	C7h/60h						
Read Status Register-1	05h	(S7-S0)(2)					
Write Status Register-1(4)	01h	(S7-S0)(4)					
Read Status Register-2	35h	(S15-S8)(2)					
Write Status Register-2	31h	(S15-S8)					
Read Status Register-3	15h	(S23-S16)(2)					
Write Status Register-3	11h	(S23-S16)					
Read SFDP Register	5Ah	00h	00h	A7–A0	dummy	(D7-0)	
Erase Security Register(5)	44h	A23-A16	A15-A8	A7-A0			
Program Security Register(5)	42h	A23-A16	A15-A8	A7-A0	D7-D0	D7-D0(3)	
Read Security Register(5)	48h	A23-A16	A15-A8	A7-A0	Dummy	(D7-D0)	
Global Block Lock	7Eh						
Global Block Unlock	98h						
Read Block Lock	3Dh	A23-A16	A15-A8	A7-A0	(L7-L0)		
Individual Block Lock	36h	A23-A16	A15-A8	A7-A0			
Individual Block Unlock	39h	A23-A16	A15-A8	A7-A0			
Erase / Program Suspend	75h						
Erase / Program Resume	7Ah						
Power-down	B9h						
Enable Reset	66h						
Reset Device	99h						

当前主要用到如下几个命令：

```
#define CMD_EWRSR      0x50  /* 允许写状态寄存器的命令 */
#define CMD_WRSR       0x01  /* 写状态寄存器命令 */
#define CMD_WREN       0x06  /* 写使能命令 */
#define CMD_READ       0x03  /* 读数据区命令 */
#define CMD_RDSR       0x05  /* 读状态寄存器命令 */
#define CMD_RDID       0x9F  /* 读器件 ID 命令 */
#define CMD_SE         0x20  /* 擦除扇区命令 */
#define CMD_BE         0xC7  /* 批量擦除命令 */
```

```
#define WIP_FLAG      0x01  /* 状态寄存器中的正在编程标志（WIP）*/
```

73.3.3 W25QXX 页编程和页回卷

SPI Flash 仅支持页编程（页大小 256 字节），所有其它大批量数据的写入都是以页为单位。这里注意所说的页编程含义，页编程分为以下三步（伪代码）：

```
bsp_spiWrite1(0x02);                                ----------第 1 步发送页编程命令
bsp_spiWrite1((_uiWriteAddr & 0xFF0000) >> 16);     ----------第 2 步发送地址
bsp_spiWrite1((_uiWriteAddr & 0xFF00) >> 8);
bsp_spiWrite1(_uiWriteAddr & 0xFF);

    for (i = 0; i < _usSize; i++)
    {
        bsp_spiWrite1(*_pBuf++);    ----------第 3 步写数据，此时就可以连续写入数据了，
                                              不需要再重新设置地址，地址会自增。这样可以大大加快写入速度。
    }
```

页编程的含义恰恰就体现在第 3 步了，如果用户设置的“起始地址+数据长度”所确定的地址范围超过了此起始地址所在的页，地址自增不会超过页范围，而是重新回到了此页的首地进行编写。这一点要特别的注意。如果用户不需要使用地址自增效果，那么直接指定地址进行编写即可。可以任意指定地址进行编写，编写前一定要进行擦除。

比如下面就是页内操作（使用前已经进行了扇区擦除，每次擦除最少擦除一个扇区 4KB）：

```
uint8_t tempbuf[10] = {0x11,0x22,0x33,0x44,0x55,0x66,0x77,0x88,0x99,0x00};
uint8_t temp1 = 0x10, temp2 = 0x29, temp3 = 0x48;
```

- 从 250 地址开始写入 10 个字节数据 PageWrite(tempbuf, 250, 10);(因为一旦写入超过地址 255，就会从 0 地址开始重新写)。
- 向地址 20 写入 1 个字节数据：PageWrite(&temp1, 20, 1);
- 向地址 30 写入 1 个字节数据：PageWrite(&temp2, 30, 1);
- 向地址 510 写入 1 个字节数据：PageWrite(&temp3, 510, 1) (这里已经是写到下一页了)

下面是将从 0 地址到 511 地址读取出来的 512 个字节数据，一行 32 字节。

```
地址：0x00000000；数据长度 = 512
77 88 99 00 FF FF FF FF FF FF FF FF FF FF FF FF -  FF FF FF FF 10 FF FF FF FF FF FF FF FF FF 29 FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF -  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF -  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF -  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF -  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF -  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF -  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF -  FF FF FF FF FF FF FF FF FF FF FF 11 22 33 44 55 66
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF -  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF -  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF -  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF -  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF -  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF -  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF -  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 48 FF
```

73.3.4 W25QXX 扇区擦除

SPI Flash 的擦除支持扇区擦除（4KB），块擦除（32KB 或者 64KB）以及整个芯片擦除。对于扇区

擦除和块擦除，使用的时候要注意一点，一般情况下，只需用户给出扇区或者块的首地址即可。

如果给的不是扇区或者块的首地址也没有关系的，只要此地址是在扇区或者块的范围内，此扇区或者块也可以被正确擦除。不过建议使用时给首地址，方便管理。

73.3.5 W25QXX 规格参数

这里我们主要了解擦写耗时和支持的时钟速度，下面是擦写时间参数：

DESCRIPTION	SYMBOL	ALT	SPEC			UNIT
			MIN	TYP	MAX	
Write Protect Setup Time Before /CS Low	t_{WHSL}(3)		20			ns
Write Protect Hold Time After /CS High	t_{SHWL}(3)		100			ns
/CS High to Power-down Mode	t_{DP}(2)				3	µs
/CS High to Standby Mode without ID Read	t_{RES1}(2)				3	µs
/CS High to Standby Mode with ID Read	t_{RES2}(2)				1.8	µs
/CS High to next Instruction after Suspend	t_{SUS}(2)				20	µs
/CS High to next Instruction after Reset	t_{RST}(2)				30	µs
/RESET pin Low period to reset the device	t_{RESET}(2)		1(5)			µs
Write Status Register Time	t_W			10	15	ms
Page Program Time	t_{PP}			0.4	3	ms
Sector Erase Time (4KB)	t_{SE}			45	400	ms
Block Erase Time (32KB)	t_{BE1}			120	1,600	ms
Block Erase Time (64KB)	t_{BE2}			150	2,000	ms
Chip Erase Time	t_{CE}			20	100	s

- 页编程时间：典型值 0.4ms，最大值 3ms。
- 扇区擦除时间（4KB）：典型值 45ms，最大值 400ms。
- 块擦除时间（32KB）：典型值 120ms，最大值 1600ms。
- 块擦除时间（64KB）：典型值 150ms，最大值 2000ms。
- 整个芯片擦除时间：典型值 20s，最大值 100s。

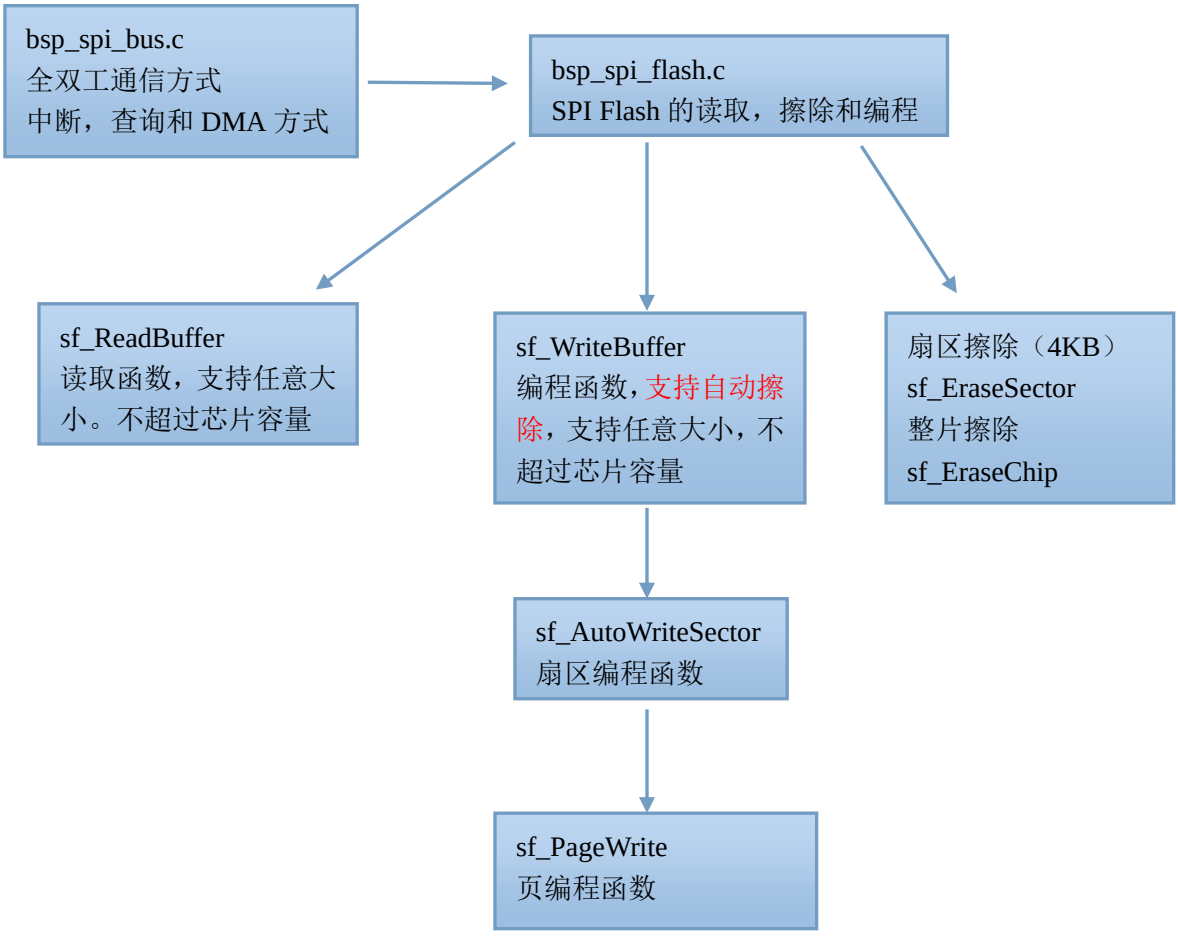
支持的速度参数如下：

DESCRIPTION	SYMBOL	ALT	SPEC			UNIT
			MIN	TYP	MAX	
Clock frequency except for Read Data (03h) instructions (3.0V-3.6V)	F_R	f_{C1}	D.C.		133	MHz
Clock frequency except for Read Data (03h) instructions(2.7V-3.0V)	F_R	f_{C2}	D.C.		104	MHz
Clock frequency for Read Data instruction (03h)	fR		D.C.		50	MHz
Clock High, Low Time for all instructions except for Read Data (03h)	t_{CLH}, t_{CLL}(1)		45% PC			ns
Clock High, Low Time for Read Data (03h) instruction	t_{CRLH}, t_{CRLL}(1)		45% PC			ns

可以看到最高支持的读时钟（使用命令 03H）速度是 50MHz，其它命令速度可以做到 133MHz。

73.4 W25QXX 驱动设计

W25QXX 的程序驱动框架设计如下：



有了这个框图，程序设计就比较好理解了。

73.4.1第 1 步：SPI 总线配置

spi 总线配置通过如下两个函数实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitSPIBus
*    功能说明: 配置 SPI 总线。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitSPIBus(void)
{
    g_spi_busy = 0;

    bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_8, SPI_PHASE_1EDGE, SPI_POLARITY_LOW);
}

/*
*********************************************************************************************************
*    函 数 名: bsp_InitSPIParam
*    功能说明: 配置 SPI 总线参数，时钟分频，时钟相位和时钟极性。
*    形    参: _BaudRatePrescaler  SPI 总线时钟分频设置，支持的参数如下：
*                                 SPI_BAUDRATEPRESCALER_2    2 分频
*                                 SPI_BAUDRATEPRESCALER_4    4 分频
*                                 SPI_BAUDRATEPRESCALER_8    8 分频
*                                 SPI_BAUDRATEPRESCALER_16   16 分频
*                                 SPI_BAUDRATEPRESCALER_32   32 分频
*                                 SPI_BAUDRATEPRESCALER_64   64 分频
*                                 SPI_BAUDRATEPRESCALER_128  128 分频
*                                 SPI_BAUDRATEPRESCALER_256  256 分频
*
*              _CLKPhase          时钟相位，支持的参数如下：
*                                 SPI_PHASE_1EDGE     SCK 引脚的第 1 个边沿捕获传输的第 1 个数据
*                                 SPI_PHASE_2EDGE     SCK 引脚的第 2 个边沿捕获传输的第 1 个数据
*
*              _CLKPolarity       时钟极性，支持的参数如下：
*                                 SPI_POLARITY_LOW    SCK 引脚在空闲状态处于低电平
*                                 SPI_POLARITY_HIGH   SCK 引脚在空闲状态处于高电平
*
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitSPIParam(uint32_t _BaudRatePrescaler, uint32_t _CLKPhase, uint32_t _CLKPolarity)
{
    /* 提高执行效率，只有在 SPI 硬件参数发生变化时，才执行 HAL_Init */
    if (s_BaudRatePrescaler == _BaudRatePrescaler && s_CLKPhase == _CLKPhase && s_CLKPolarity == _CLKPolarity)
    {
        return;
    }

    s_BaudRatePrescaler = _BaudRatePrescaler;
    s_CLKPhase = _CLKPhase;
    s_CLKPolarity = _CLKPolarity;
```

```
    /* 设置 SPI 参数 */
    hspi.Instance               = SPIx;                     /* 例化 SPI */
    hspi.Init.BaudRatePrescaler = _BaudRatePrescaler;       /* 设置波特率 */
    hspi.Init.Direction         = SPI_DIRECTION_2LINES;     /* 全双工 */
    hspi.Init.CLKPhase          = _CLKPhase;                /* 配置时钟相位 */
    hspi.Init.CLKPolarity       = _CLKPolarity;             /* 配置时钟极性 */
    hspi.Init.DataSize          = SPI_DATASIZE_8BIT;        /* 设置数据宽度 */
    hspi.Init.FirstBit          = SPI_FIRSTBIT_MSB;         /* 数据传输先传高位 */
    hspi.Init.TIMode            = SPI_TIMODE_DISABLE;       /* 禁止 TI 模式  */
    hspi.Init.CRCCalculation    = SPI_CRCCALCULATION_DISABLE; /* 禁止 CRC */
    hspi.Init.CRCPolynomial     = 7;                         /* 禁止 CRC 后，此位无效 */
    hspi.Init.CRCLength         = SPI_CRC_LENGTH_8BIT;       /* 禁止 CRC 后，此位无效 */
    hspi.Init.NSS               = SPI_NSS_SOFT;              /* 使用软件方式管理片选引脚 */
    hspi.Init.FifoThreshold     = SPI_FIFO_THRESHOLD_01DATA;  /* 设置 FIFO 大小是一个数据项 */
    hspi.Init.NSSPMode          = SPI_NSS_PULSE_DISABLE;      /* 禁止脉冲输出 */
    hspi.Init.MasterKeepIOState = SPI_MASTER_KEEP_IO_STATE_ENABLE; /* 禁止 SPI 后，SPI 相关引脚保持当前状态 */
    hspi.Init.Mode              = SPI_MODE_MASTER;           /* SPI 工作在主控模式 */

    /* 复位配置 */
    if (HAL_SPI_DeInit(&hspi) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 初始化配置 */
    if (HAL_SPI_Init(&hspi) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

关于这两个函数有以下两点要做个说明：

- 函数 bsp_InitSPIBus 里面的配置是个初始设置。实际驱动芯片时，会通过函数 bsp_InitSPIParam 做再配置。
- 函数 bsp_InitSPIParam 提供了时钟分频，时钟相位和时钟极性配置。驱动不同外设芯片时，基本上调整这三个参数就够。当 SPI 接口上接了多个不同类型的芯片时，通过此函数可以方便的切换配置。

73.4.2 第 2 步：SPI 总线的查询，中断和 DMA 方式设置

SPI 驱动的查询，中断和 DMA 方式主要通过函数 bsp_spiTransfer 实现数据传输：

```
/*
*********************************************************************************************************
*                                选择 DMA，中断或者查询方式
*********************************************************************************************************
*/
//#define USE_SPI_DMA    /* DMA 方式  */
//#define USE_SPI_INT    /* 中断方式 */
#define USE_SPI_POLL   /* 查询方式 */

/* 查询模式 */
#if defined (USE_SPI_POLL)
```

```
uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];

/* 中断模式 */
#elif defined (USE_SPI_INT)

uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];

/* DMA 模式使用的 SRAM4 */
#elif defined (USE_SPI_DMA)
    #if defined ( __CC_ARM )    /* IAR *******/
        __attribute__((section (".RAM_D3"))) uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
        __attribute__((section (".RAM_D3"))) uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];
    #elif defined (__ICCARM__)   /* MDK ********/
        #pragma location = ".RAM_D3"
        uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
        #pragma location = ".RAM_D3"
        uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];
    #endif
#endif

/*
*********************************************************************************************************
*    函 数 名: bsp_spiTransfer
*    功能说明: 启动数据传输
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_spiTransfer(void)
{
    if (g_spiLen > SPI_BUFFER_SIZE)
    {
        return;
    }

    /* DMA 方式传输 */
#ifdef USE_SPI_DMA
    wTransferState = TRANSFER_WAIT;

    if(HAL_SPI_TransmitReceive_DMA(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen) != HAL_OK)

    {
        Error_Handler(__FILE__, __LINE__);
    }

    while (wTransferState == TRANSFER_WAIT)
    {
        ;
    }
#endif

    /* 中断方式传输 */
#ifdef USE_SPI_INT
    wTransferState = TRANSFER_WAIT;

    if(HAL_SPI_TransmitReceive_IT(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen) != HAL_OK)
```

```
    {
        Error_Handler(__FILE__, __LINE__);
    }

    while (wTransferState == TRANSFER_WAIT)
    {
        ;
    }
#endif

    /* 查询方式传输 */
#ifdef USE_SPI_POLL
    if(HAL_SPI_TransmitReceive(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen, 1000000) !=
HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
#endif
}
```

通过开头宏定义可以方便的切换中断，查询和 DMA 方式。其中查询和中断方式比较好理解，而 DMA 方式要特别注意两点：

- 通过本手册第 26 章的内存块超方便使用方式，将 DMA 缓冲定义到 SRAM4 上。因为本工程是用的 DTCM 做的主 RAM 空间，这个空间无法使用通用 DMA1 和 DMA2。
- 由于程序里面开启了数据 Cache，会造成 DMA 和 CPU 访问 SRAM4 数据不一致的问题，特此将 SRAM4 空间关闭 Cache。

```
    /* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

73.4.3第 3 步：W25QXX 的时钟极性和时钟相位配置

首先回忆下 STM32H7 支持的 4 种时序配置。

- 当 CPOL = 1， CPHA = 1 时

  SCK 引脚在空闲状态处于低高平，SCK 引脚的第 2 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 0， CPHA = 1 时

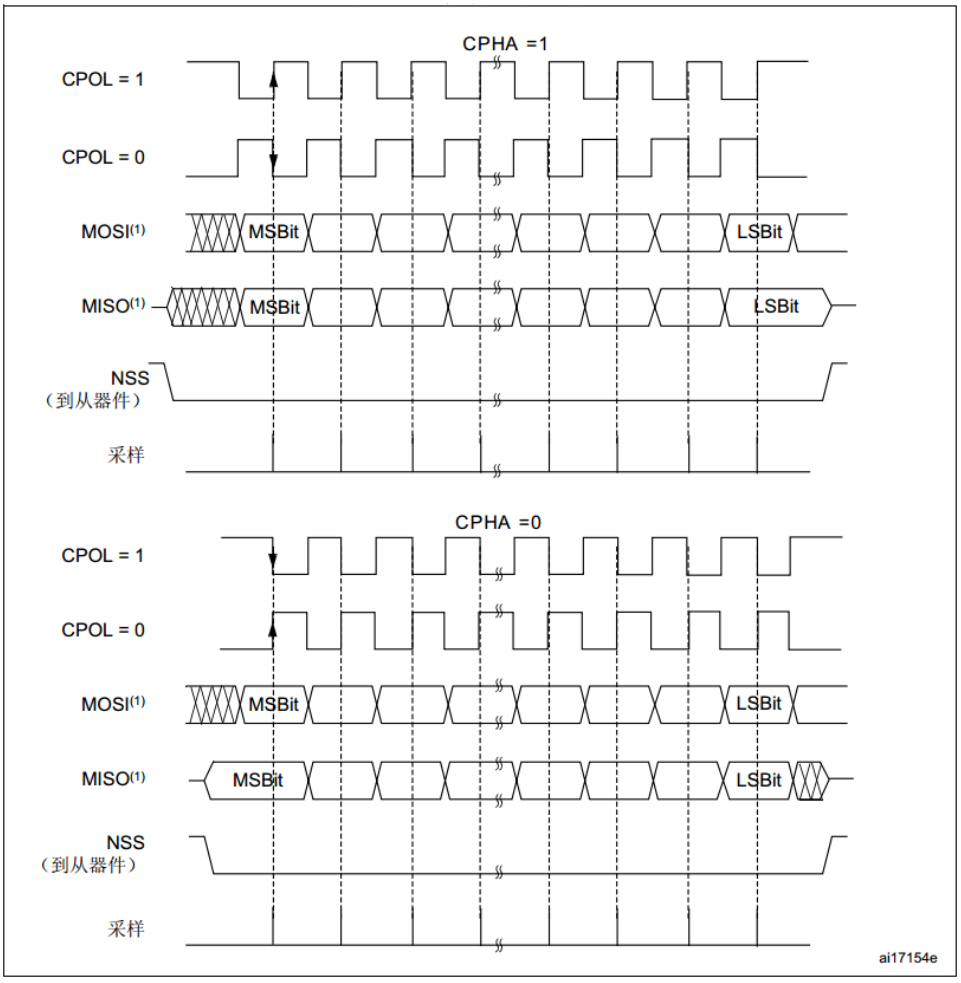
SCK 引脚在空闲状态处于低电平，SCK 引脚的第 2 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 1， CPHA = 0 时

  SCK 引脚在空闲状态处于高电平，SCK 引脚的第 1 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 0， CPHA = 0 时

  SCK 引脚在空闲状态处于低电平，SCK 引脚的第 1 个边沿捕获传输的第 1 个数据。



有了 H7 支持的时序配置，再来看下 W25Q 的时序图：

Mode0 ： 空闲状态的 sck 是低电平。

Mode1 ： 空闲状态的 sck 是高电平。

instruction is entered by driving /CS low, shifting the instruction code "06h" into the Data Input (DI) pin on the rising edge of CLK, and then driving /CS high.

注意这里是时钟上升沿

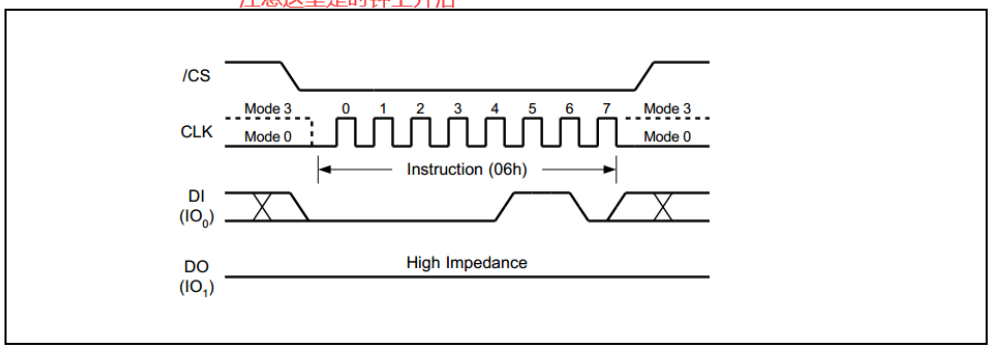


Figure 5. Write Enable Instruction for SPI Mode

首先 W25Q 是上升沿做数据采集，所以 STM32H7 的可选的配置就是：

CHOL = 1， CPHA = 1

CHOL = 0， CPHA = 0

对于这两种情况，具体选择哪种，继续往下看。W25Q 有两种 SCK 模式，分别是 Mode0 和 Mode3，也就是空闲状态下，SCK 既可以是高电平也可以是低电平。这样的话，这两种情况都可以使用，经过实际测试，STM32H7 使用这两个配置均可以配置驱动 W25Q。

73.4.4第 4 步：单 SPI 接口管理多个 SPI 设备的切换机制

单 SPI 接口管理多个 SPI 设备最麻烦的地方是不同设备的时钟分配，时钟极性和时钟相位并不相同。对此的解决解决办法是在片选阶段配置切换，比如 SPI Flash 的片选：

```
/*
*********************************************************************************************************
*    函 数 名: sf_SetCS
*    功能说明: 串行 FALSH 片选控制函数
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void sf_SetCS(uint8_t _Level)
{
    if (_Level == 0)
    {
        bsp_SpiBusEnter();
        bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_4, SPI_PHASE_1EDGE, SPI_POLARITY_LOW);
        SF_CS_0();
    }
    else
    {
        SF_CS_1();
        bsp_SpiBusExit();
    }
}
```

通过这种方式就有效的解决了单 SPI 接口管理多设备的问题。因为给每个设备都配了一个独立的片选引脚，这样就可以为每个设备都配置这么一个片选配置。

但是频繁配置也比较繁琐，所以函数 bsp_InitSPIParam 里面做了特别处理。当前配置与之前配置相同的情况下无需重复配置。

73.4.5第 5 步：W25QXX 的读取实现

W25QXX 的读取功能比较好实现，发送 03H 命令后，设置任意地址都可以读取数据，只要不超过芯片容量即可。

```
/*
*********************************************************************************************************
*    函 数 名: sf_ReadBuffer
*    功能说明: 连续读取若干字节，字节个数不能超出芯片容量。
*    形    参:     _pBuf : 数据源缓冲区;
*                 _uiReadAddr : 首地址
*                 _usSize : 数据个数, 不能超出芯片总容量
*    返 回 值: 无
*********************************************************************************************************
*/
void sf_ReadBuffer(uint8_t * _pBuf, uint32_t _uiReadAddr, uint32_t _uiSize)
{
    uint16_t rem;
    uint16_t i;

    /* 如果读取的数据长度为 0 或者超出串行 Flash 地址空间，则直接返回 */
    if ((_uiSize == 0) ||(_uiReadAddr + _uiSize) > g_tSF.TotalSize)
    {
        return;
    }

    /* 擦除扇区操作 */
    sf_SetCS(0);                                        /* 使能片选 */
    g_spiLen = 0;
    g_spiTxBuf[g_spiLen++] = (CMD_READ);                                /* 发送读命令 */
    g_spiTxBuf[g_spiLen++] = ((_uiReadAddr & 0xFF0000) >> 16);  /* 发送扇区地址的高 8bit */
    g_spiTxBuf[g_spiLen++] = ((_uiReadAddr & 0xFF00) >> 8);     /* 发送扇区地址中间 8bit */
    g_spiTxBuf[g_spiLen++] = (_uiReadAddr & 0xFF);              /* 发送扇区地址低 8bit */
    bsp_spiTransfer();

    /* 开始读数据，因为底层 DMA 缓冲区有限，必须分包读 */
    for (i = 0; i < _uiSize / SPI_BUFFER_SIZE; i++)
    {
        g_spiLen = SPI_BUFFER_SIZE;
        bsp_spiTransfer();

        memcpy(_pBuf, g_spiRxBuf, SPI_BUFFER_SIZE);
        _pBuf += SPI_BUFFER_SIZE;
    }

    rem = _uiSize % SPI_BUFFER_SIZE; /* 剩余字节 */
    if (rem > 0)
    {
        g_spiLen = rem;
        bsp_spiTransfer();
```

```
        memcpy(_pBuf, g_spiRxBuf, rem);
    }

    sf_SetCS(1);                                        /* 禁能片选 */
}
```

这个函数对 DMA 传输做了特别处理，方便分包进行。

73.4.6第 6 步：W25QXX 的扇区擦除实现

扇区擦除的实现也比较简单，发送“扇区擦除命令+扇区地址”即可完成相应扇区的擦除。擦除的扇区大小是 4KB。

```
/*
*********************************************************************************************************
*    函 数 名: sf_EraseSector
*    功能说明: 擦除指定的扇区
*    形    参: _uiSectorAddr : 扇区地址
*    返 回 值: 无
*********************************************************************************************************
*/
void sf_EraseSector(uint32_t _uiSectorAddr)
{
    sf_WriteEnable();                                   /* 发送写使能命令 */

    /* 擦除扇区操作 */
    sf_SetCS(0);                                        /* 使能片选 */
    g_spiLen = 0;
    g_spiTxBuf[g_spiLen++] = CMD_SE;                    /* 发送擦除命令 */
    g_spiTxBuf[g_spiLen++] = ((_uiSectorAddr & 0xFF0000) >> 16); /* 发送扇区地址的高 8bit */
    g_spiTxBuf[g_spiLen++] = ((_uiSectorAddr & 0xFF00) >> 8);    /* 发送扇区地址中间 8bit */
    g_spiTxBuf[g_spiLen++] = (_uiSectorAddr & 0xFF);             /* 发送扇区地址低 8bit */
    bsp_spiTransfer();
    sf_SetCS(1);                                        /* 禁能片选 */

    sf_WaitForWriteEnd();                               /* 等待串行 Flash 内部写操作完成 */
}
```

整个芯片的擦除更省事些，仅发送整个芯片擦除命令即可：

```
/*
*********************************************************************************************************
*    函 数 名: sf_EraseChip
*    功能说明: 擦除整个芯片
*    形    参:  无
*    返 回 值: 无
*********************************************************************************************************
*/
void sf_EraseChip(void)
{
    sf_WriteEnable();                                   /* 发送写使能命令 */

    /* 擦除扇区操作 */
    sf_SetCS(0);       /* 使能片选 */
    g_spiLen = 0;
    g_spiTxBuf[g_spiLen++] = CMD_BE;      /* 发送整片擦除命令 */
    bsp_spiTransfer();
```

```
    sf_SetCS(1);                        /* 禁能片选 */

    sf_WaitForWriteEnd();               /* 等待串行 Flash 内部写操作完成 */
}
```

73.4.7第 7 步：W25QXX 的编程实现

W25QXX 的编程实现略复杂，因为做了自动擦除支持，大家可以在任意地址，写任意大小的数据，只要不超过芯片容量即可。我们这里就不做展开讨论了，大家有兴趣可以研究下：

```
/*
*********************************************************************************************************
*    函 数 名: sf_WriteBuffer
*    功能说明: 写 1 个扇区并校验,如果不正确则再重写两次，本函数自动完成擦除操作。
*    形    参:  _pBuf : 数据源缓冲区;
*               _uiWrAddr : 目标区域首地址
*               _usSize : 数据个数，任意大小，但不能超过芯片容量。
*    返 回 值: 1 : 成功， 0 ： 失败
*********************************************************************************************************
*/
uint8_t sf_WriteBuffer(uint8_t* _pBuf, uint32_t _uiWriteAddr, uint32_t _usWriteSize)
{
    uint32_t NumOfPage = 0, NumOfSingle = 0, Addr = 0, count = 0, temp = 0;

    Addr = _uiWriteAddr % g_tSF.SectorSize;
    count = g_tSF.SectorSize - Addr;
    NumOfPage =  _usWriteSize / g_tSF.SectorSize;
    NumOfSingle = _usWriteSize % g_tSF.SectorSize;

    if (Addr == 0) /* 起始地址是扇区首地址  */
    {
        if (NumOfPage == 0) /* 数据长度小于扇区大小 */
        {
            if (sf_AutoWriteSector(_pBuf, _uiWriteAddr, _usWriteSize) == 0)
            {
                return 0;
            }
        }
        else      /* 数据长度大于等于扇区大小 */
        {
            while (NumOfPage--)
            {
                if (sf_AutoWriteSector(_pBuf, _uiWriteAddr, g_tSF.SectorSize) == 0)
                {
                    return 0;
                }
                _uiWriteAddr +=  g_tSF.SectorSize;
                _pBuf += g_tSF.SectorSize;
            }
            if (sf_AutoWriteSector(_pBuf, _uiWriteAddr, NumOfSingle) == 0)
            {
                return 0;
            }
        }
    }
```

```
    else  /* 起始地址不是扇区首地址  */
    {
        if (NumOfPage == 0) /* 数据长度小于扇区大小 */
        {
            if (NumOfSingle > count)  /* (_usWriteSize + _uiWriteAddr) > SPI_FLASH_PAGESIZE */
            {
                temp = NumOfSingle - count;

                if (sf_AutoWriteSector(_pBuf, _uiWriteAddr, count) == 0)
                {
                    return 0;
                }

                _uiWriteAddr +=  count;
                _pBuf += count;

                if (sf_AutoWriteSector(_pBuf, _uiWriteAddr, temp) == 0)
                {
                    return 0;
                }
            }
            else
            {
                if (sf_AutoWriteSector(_pBuf, _uiWriteAddr, _usWriteSize) == 0)
                {
                    return 0;
                }
            }
        }
        else /* 数据长度大于等于扇区大小 */
        {
            _usWriteSize -= count;
            NumOfPage =  _usWriteSize / g_tSF.SectorSize;
            NumOfSingle = _usWriteSize % g_tSF.SectorSize;
            if (sf_AutoWriteSector(_pBuf, _uiWriteAddr, count) == 0)
            {
                return 0;
            }

            _uiWriteAddr +=  count;
            _pBuf += count;

            while (NumOfPage--)
            {
                if (sf_AutoWriteSector(_pBuf, _uiWriteAddr, g_tSF.SectorSize) == 0)
                {
                    return 0;
                }
                _uiWriteAddr +=  g_tSF.SectorSize;
                _pBuf += g_tSF.SectorSize;
            }

            if (NumOfSingle != 0)
            {
                if (sf_AutoWriteSector(_pBuf, _uiWriteAddr, NumOfSingle) == 0)
                {
                    return 0;
                }
```

```
            }
        }
    }
    return 1;/* 成功 */
}
```

73.5 SPI 总线板级支持包（bsp_spi_bus.c)

SPI 总线驱动文件 bsp_spi_bus.c 主要实现了如下几个 API 供用户调用：

- bsp_InitSPIBus
- bsp_InitSPIParam
- bsp_spiTransfer

73.5.1 函数 bsp_InitSPIBus

函数原型：

```
void bsp_InitSPIBus(void)
```

函数描述：

此函数主要用于 SPI 总线的初始化，在 bsp.c 文件调用一次即可。

73.5.2 函数 bsp_InitSPIParam

函数原型：

```
void bsp_InitSPIParam(uint32_t _BaudRatePrescaler, uint32_t _CLKPhase, uint32_t _CLKPolarity)
```

函数描述：

此函数用于 SPI 总线的配置。

函数参数：

- 第 1 个参数 SPI 总线的分频设置，支持的参数如下：

  SPI_BAUDRATEPRESCALER_2　　2 分频
  SPI_BAUDRATEPRESCALER_4　　4 分频
  SPI_BAUDRATEPRESCALER_8　　8 分频
  SPI_BAUDRATEPRESCALER_16　16 分频
  SPI_BAUDRATEPRESCALER_32　32 分频
  SPI_BAUDRATEPRESCALER_64　64 分频
  SPI_BAUDRATEPRESCALER_128　128 分频
  SPI_BAUDRATEPRESCALER_256　256 分频

- 第 2 个参数用于时钟相位配置，支持的参数如下：

  SPI_PHASE_1EDGE　　SCK 引脚的第 1 个边沿捕获传输的第 1 个数据

SPI_PHASE_2EDGE　　SCK 引脚的第 2 个边沿捕获传输的第 1 个数据

- 第 3 个参数是时钟极性配置，支持的参数如下：

SPI_POLARITY_LOW　　SCK 引脚在空闲状态处于低电平

SPI_POLARITY_HIGH　　SCK 引脚在空闲状态处于高电平

73.5.3 函数 bsp_spiTransfer

函数原型：

```
void bsp_spiTransfer(void)
```

函数描述：

此函数用于启动 SPI 数据传输，支持查询，中断和 DMA 方式传输。

73.6 W25QXX 板级支持包（bsp_spi_flash.c）

W25QXX 驱动文件 bsp_spi_flash.c 主要实现了如下几个 API 供用户调用：

- sf_ReadBuffer
- sf_WriteBuffer
- sf_EraseSector
- sf_EraseChip
- sf_EraseSector

73.6.1 函数 sf_ReadBuffer

函数原型：

```
void sf_ReadBuffer(uint8_t * _pBuf, uint32_t _uiReadAddr, uint32_t _uiSize)
```

函数描述：

此函数主要用于从 SPI Flash 读取数据，支持任意大小，任意地址，不超过芯片容量即可。

函数参数：

- 第 1 个参数用于存储从 SPI Flash 读取的数据。
- 第 2 个参数是读取地址，不可以超过芯片容量。
- 第 3 个参数是读取的数据大小，读取范围不可以超过芯片容量。

73.6.2 函数 sf_WriteBuffer（自动执行擦除）

函数原型：

```
uint8_t sf_WriteBuffer(uint8_t* _pBuf, uint32_t _uiWriteAddr, uint32_t _usWriteSize)
```

函数描述：

此函数主要用于 SPI Flash 读取数据，支持任意大小，任意地址，不超过芯片容量即可。特别注意，此函数会自动执行擦除，无需用户处理。

函数参数：

- 第 1 个参数是源数据缓冲区。
- 第 2 个参数是目标区域首地址。
- 第 3 个参数是数据个数，支持任意大小，但不能超过芯片容量。单位字节个数。
- 返回值，返回 1 表示成功，返回 0 表示失败。

73.6.3 函数 sf_EraseSector

函数原型：

```
void sf_EraseSector(uint32_t _uiSectorAddr)
```

函数描述：

此函数主要用于扇区擦除，一个扇区大小是 4KB。

函数参数：

- 第 1 个参数是扇区地址，比如擦除扇区 0，此处填 0x0000，擦除扇区 1，此处填 0x1000，擦除扇区 2，此处填 0x2000，以此类推。

73.6.4 函数 sf_EraseChip

函数原型：

```
void sf_EraseChip(void)
```

函数描述：

此函数主要用于整个芯片擦除。

73.6.5 函数 sf_PageWrite（不推荐）

函数原型：

```
void sf_PageWrite(uint8_t * _pBuf, uint32_t _uiWriteAddr, uint16_t _usSize)
```

函数描述：

此函数主要用于页编程，一次可以编程多个页，只要不超过芯片容量即可。不推荐大家调用此函数，因为调用这个函数前，需要大家调用函数 sf_EraseSector 进行扇区擦除。

函数参数：

- 第 1 个参数是数据源缓冲区。

- 第 2 个参数目标区域首地址，比如编程页 0，此处填 0x0000，编程页 1，此处填 0x0100，编程页 2，此处填 0x0200，以此类推。
- 第 3 个参数是编程的数据大小，务必是 256 字节的整数倍，单位字节个数。

73.7 W25QXX 驱动移植和使用

W25QXX 移植步骤如下：

- 第 1 步：复制 bsp_spi_bus.c，bsp_spi_bus.h，bsp_spi_flash.c，bsp_spi_flash.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：根据使用的第几个 SPI，SPI 时钟，SPI 引脚和 DMA 通道等，修改 bsp_spi_bus.c 文件开头的宏定义

```
/*
*********************************************************************************************************
*                                    时钟，引脚，DMA，中断等宏定义
*********************************************************************************************************
*/
#define SPIx                             SPI1
#define SPIx_CLK_ENABLE()                __HAL_RCC_SPI1_CLK_ENABLE()
#define DMAx_CLK_ENABLE()                __HAL_RCC_DMA2_CLK_ENABLE()

#define SPIx_FORCE_RESET()               __HAL_RCC_SPI1_FORCE_RESET()
#define SPIx_RELEASE_RESET()             __HAL_RCC_SPI1_RELEASE_RESET()

#define SPIx_SCK_CLK_ENABLE()            __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_SCK_GPIO                    GPIOB
#define SPIx_SCK_PIN                     GPIO_PIN_3
#define SPIx_SCK_AF                          GPIO_AF5_SPI1

#define SPIx_MISO_CLK_ENABLE()           __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MISO_GPIO                   GPIOB
#define SPIx_MISO_PIN                    GPIO_PIN_4
#define SPIx_MISO_AF                     GPIO_AF5_SPI1

#define SPIx_MOSI_CLK_ENABLE()           __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MOSI_GPIO                   GPIOB
#define SPIx_MOSI_PIN                    GPIO_PIN_5
#define SPIx_MOSI_AF                     GPIO_AF5_SPI1

#define SPIx_TX_DMA_STREAM               DMA2_Stream3
#define SPIx_RX_DMA_STREAM               DMA2_Stream2

#define SPIx_TX_DMA_REQUEST              DMA_REQUEST_SPI1_TX
#define SPIx_RX_DMA_REQUEST              DMA_REQUEST_SPI1_RX

#define SPIx_DMA_TX_IRQn                 DMA2_Stream3_IRQn
#define SPIx_DMA_RX_IRQn                 DMA2_Stream2_IRQn

#define SPIx_DMA_TX_IRQHandler           DMA2_Stream3_IRQHandler
#define SPIx_DMA_RX_IRQHandler           DMA2_Stream2_IRQHandler

#define SPIx_IRQn                        SPI1_IRQn
```

```
#define SPIx_IRQHandler                 SPI1_IRQHandler
```

◆ 第 3 步：根据使用的 SPI ID，添加定义到文件 bsp_spi_flash.h

```
/* 定义串行 Flash ID */
enum
{
    SST25VF016B_ID = 0xBF2541,
    MX25L1606E_ID  = 0xC22015,
    W25Q64BV_ID    = 0xEF4017, /* BV, JV, FV */
    W25Q128_ID     = 0xEF4018
};
```

◆ 第 4 步：添加相应型号到 bsp_spi_flash.c 文件的函数 sf_ReadInfo 里面。

```
/*
*********************************************************************************************************
*    函 数 名: sf_ReadInfo
*    功能说明: 读取器件 ID,并填充器件参数
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void sf_ReadInfo(void)
{
    /* 自动识别串行 Flash 型号 */
    {
        g_tSF.ChipID = sf_ReadID(); /* 芯片 ID */

        switch (g_tSF.ChipID)
        {
            case SST25VF016B_ID:
                strcpy(g_tSF.ChipName, "SST25VF016B");
                g_tSF.TotalSize = 2 * 1024 * 1024;    /* 总容量 = 2M */
                g_tSF.SectorSize = 4 * 1024;          /* 扇区大小 = 4K */
                break;

            case MX25L1606E_ID:
                strcpy(g_tSF.ChipName, "MX25L1606E");
                g_tSF.TotalSize = 2 * 1024 * 1024;    /* 总容量 = 2M */
                g_tSF.SectorSize = 4 * 1024;          /* 扇区大小 = 4K */
                break;

            case W25Q64BV_ID:
                strcpy(g_tSF.ChipName, "W25Q64");
                g_tSF.TotalSize = 8 * 1024 * 1024;    /* 总容量 = 8M */
                g_tSF.SectorSize = 4 * 1024;          /* 扇区大小 = 4K */
                break;

            case W25Q128_ID:
                strcpy(g_tSF.ChipName, "W25Q128");
                g_tSF.TotalSize = 16 * 1024 * 1024;   /* 总容量 = 8M */
                g_tSF.SectorSize = 4 * 1024;          /* 扇区大小 = 4K */
                break;

            default:
                strcpy(g_tSF.ChipName, "Unknow Flash");
                g_tSF.TotalSize = 2 * 1024 * 1024;
                g_tSF.SectorSize = 4 * 1024;
                break;
```

```
        }
    }
}
```

◆ 第 5 步：根据芯片支持的时钟速度，时钟相位和时钟极性配置函数 sf_SetCS。

```
/*
*********************************************************************************************************
*    函 数 名: sf_SetCS
*    功能说明: 串行 FALSH 片选控制函数
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void sf_SetCS(uint8_t _Level)
{
    if (_Level == 0)
    {
        bsp_SpiBusEnter();
        bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_4, SPI_PHASE_1EDGE, SPI_POLARITY_LOW);
        SF_CS_0();
    }
    else
    {
        SF_CS_1();
        bsp_SpiBusExit();
    }
}
```

◆ 第 6 步：根据使用的 SPI Flash 片选引脚修改 bsp_spi_bus.c 文件开头的宏定义。

```
/* 串行 Flash 的片选 GPIO 端口，  PD13  */
#define SF_CS_CLK_ENABLE()            __HAL_RCC_GPIOD_CLK_ENABLE()
#define SF_CS_GPIO                    GPIOD
#define SF_CS_PIN                     GPIO_PIN_13

#define SF_CS_0()                     SF_CS_GPIO->BSRR = ((uint32_t)SF_CS_PIN << 16U)
#define SF_CS_1()                     SF_CS_GPIO->BSRR = SF_CS_PIN
```

◆ 第 7 步：如果使用 DMA 方式的话，请不要使用 TCM RAM，因为通用 DMA1 和 DMA2 不支持。并为了防止 DMA 和 CPU 同时访问 DMA 缓冲造成的数据一致性问题，将这块空间关闭 Cache 处理，比如使用的 SRAM4：

```
/* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress      = 0x38000000;
MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

◆ 第 8 步：初始化 SPI。

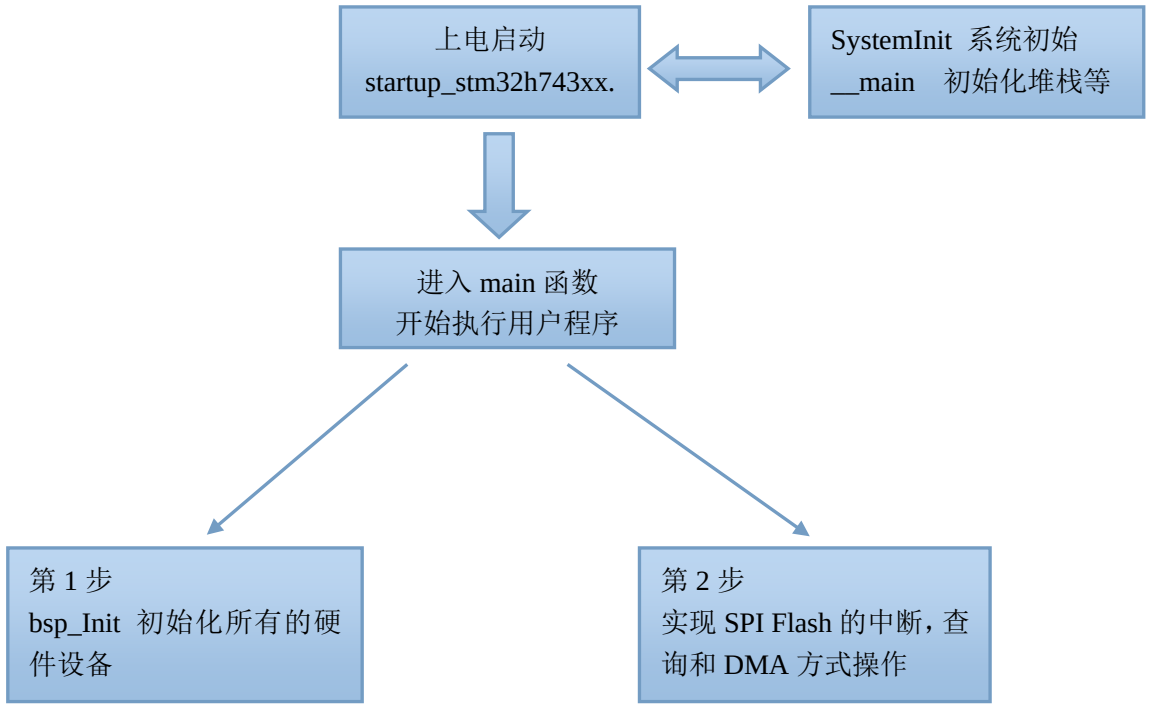
```
/* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
bsp_InitSPIBus();  /* 配置 SPI 总线 */
```

```
bsp_InitSFlash();  /* 初始化 SPI 串行 Flash */
```

- 第 9 步：SPI Flash 驱动主要用到 HAL 库的 SPI 驱动文件，简单省事些可以添加所有 HAL 库 C 源文件进来。
- 第 10 步：应用方法看本章节配套例子即可。

73.8 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。
- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器和 LED。
  - 第 2 部分，应用程序设计部分，实现 SPI Flash 的中断，查询和 DMA 方式操作。

73.9 实验例程说明（MDK）

配套例子：

V7-028_串行 SPI Flash W25QXX 读写例程（查询方式 V1.1）

V7-050_串行 SPI Flash W25QXX 读写例程（DMA 方式）

V7-051_串行 SPI Flash W25QXX 读写例程（中断方式）

实验目的：

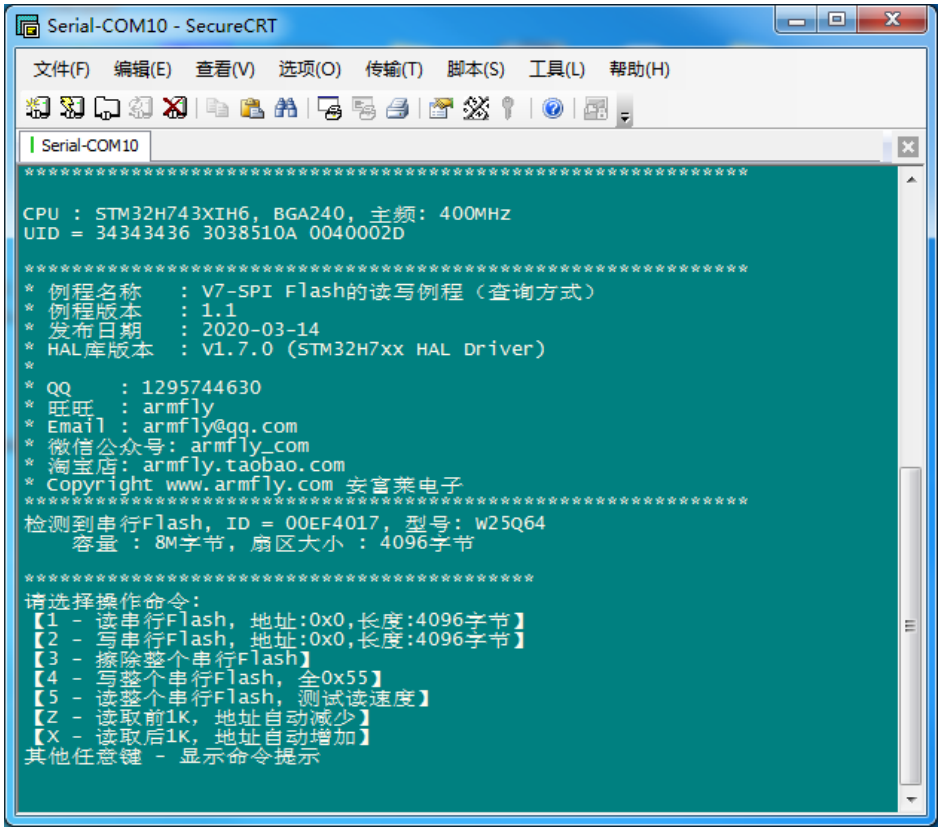
1. 学习 SPI Flash 的读写实现，支持查询，中断和 DMA 方式。

实验操作：

1. 支持以下 7 个功能，用户通过电脑端串口软件发送命令给开发板即可
2. printf("请选择操作命令:\r\n");
3. printf("【1 - 读串行 Flash，地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
4. printf("【2 - 写串行 Flash，地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
5. printf("【3 - 擦除整个串行 Flash】\r\n");
6. printf("【4 - 写整个串行 Flash，全 0x55】\r\n");
7. printf("【5 - 读整个串行 Flash，测试读速度】\r\n");
8. printf("【Z - 读取前 1K，地址自动减少】\r\n");
9. printf("【X - 读取后 1K，地址自动增加】\r\n");
10. printf("其他任意键 - 显示命令提示\r\n");

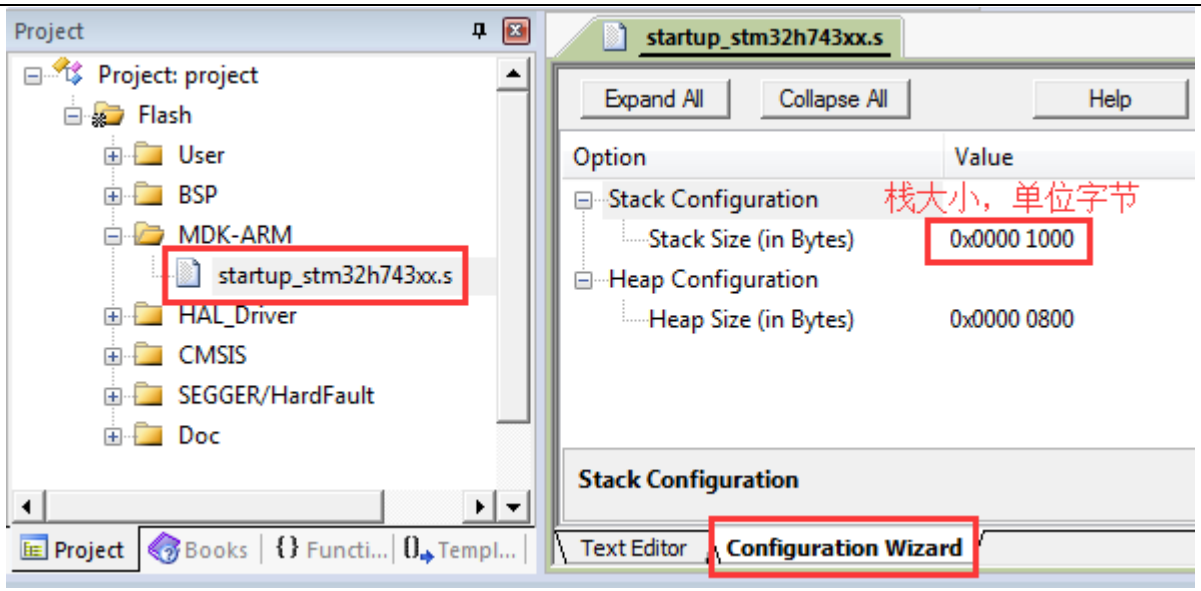
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1。

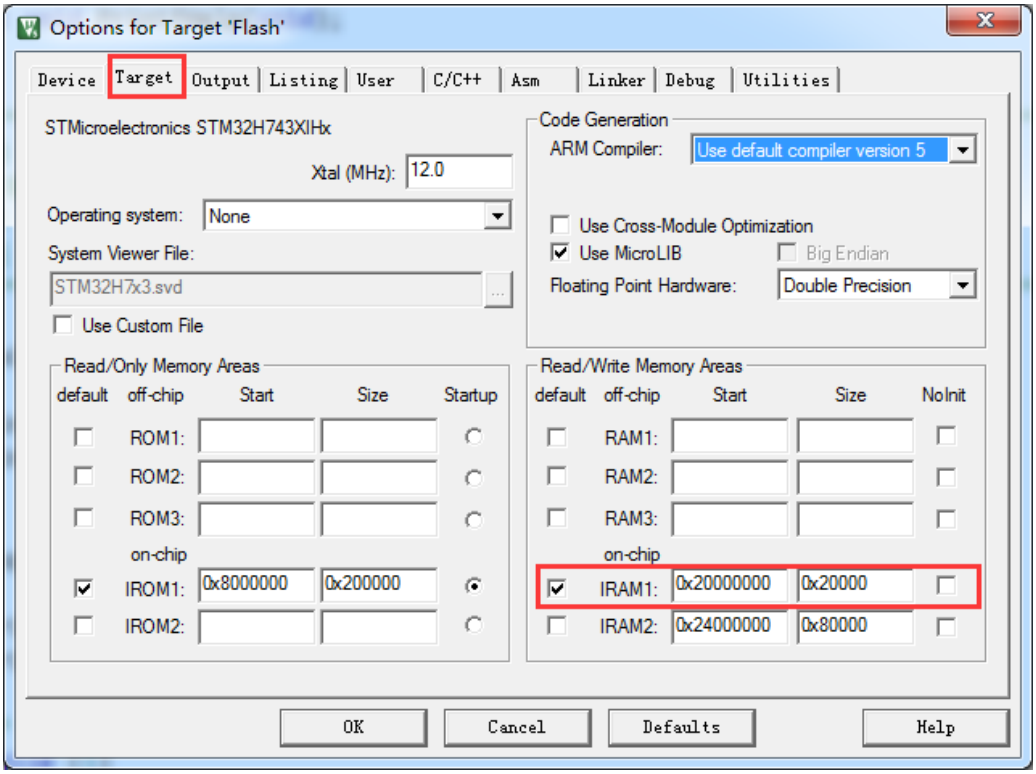


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();      /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();    /* 初始化滴答定时器 */
    bsp_InitLPUart();   /* 初始化串口 */
    bsp_InitExtIO();    /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();      /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitSPIBus();  /* 配置 SPI 总线 */
    bsp_InitSFlash();  /* 初始化 SPI 串行 Flash */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区以及 SRAM4

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
```

```
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
```

```
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **每 10ms 调用一次按键处理：**

按键处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能：**

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- 支持以下 7 个功能，用户通过电脑端串口软件发送命令给开发板即可
- 请选择操作命令:
- 1 - 读串行 Flash
- 2 - 写串行 Flash
- 3 - 擦除整个串行 Flash
- 4 - 写整个串行 Flash
- 5 - 读整个串行 Flash
- Z - 读取前 1K
- X - 读取后 1K

```
/*
*********************************************************************************************************
*    函 数 名: DemoSpiFlash
*    功能说明: 串行 EEPROM 读写例程
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
void DemoSpiFlash(void)
{
    uint8_t cmd;
    uint32_t uiReadPageNo = 0;


    /* 检测串行 Flash OK */
    printf("检测到串行 Flash, ID = %08X, 型号: %s \r\n", g_tSF.ChipID , g_tSF.ChipName);
    printf("    容量 : %dM 字节, 扇区大小 : %d 字节\r\n", g_tSF.TotalSize/(1024*1024), g_tSF.SectorSize);

    sfDispMenu();       /* 打印命令提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    while(1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        if (comGetChar(COM1, &cmd)) /* 从串口读入一个字符(非阻塞方式) */
        {
            switch (cmd)
            {
                case '1':
                    printf("\r\n【1 - 读串行 Flash, 地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
                    sfReadTest(); /* 读串行 Flash 数据，并打印出来数据内容 */
                    break;

                case '2':
                    printf("\r\n【2 - 写串行 Flash, 地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
                    sfWriteTest();     /* 写串行 Flash 数据，并打印写入速度 */
                    break;

                case '3':
                    printf("\r\n【3 - 擦除整个串行 Flash】\r\n");
                    printf("整个 Flash 擦除完毕大概需要 20 秒左右，请耐心等待");
                    sfErase();          /* 擦除串行 Flash 数据，实际上就是写入全 0xFF */
                    break;

                case '4':
                    printf("\r\n【4 - 写整个串行 Flash, 全 0x55】\r\n");
                    printf("整个 Flash 写入完毕大概需要 20 秒左右，请耐心等待");
                    sfWriteAll(0x55);/* 擦除串行 Flash 数据，实际上就是写入全 0xFF */
                    break;

                case '5':
                    printf("\r\n【5 - 读整个串行 Flash, %dM 字节】\r\n", g_tSF.TotalSize/(1024*1024));
                    sfTestReadSpeed(); /* 读整个串行 Flash 数据，测试速度 */
                    break;

                case 'z':
```

```
                case 'Z': /* 读取前 1K */
                    if (uiReadPageNo > 0)
                    {
                        uiReadPageNo--;
                    }
                    else
                    {
                        printf("已经是最前\r\n");
                    }
                    sfViewData(uiReadPageNo * 1024);
                    break;

                case 'x':
                case 'X': /* 读取后 1K */
                    if (uiReadPageNo < g_tSF.TotalSize / 1024 - 1)
                    {
                        uiReadPageNo++;
                    }
                    else
                    {
                        printf("已经是最后\r\n");
                    }
                    sfViewData(uiReadPageNo * 1024);
                    break;

                default:
                    sfDispMenu(); /* 无效命令，重新打印命令提示 */
                    break;

            }
        }
    }
}
```

73.10 实验例程说明（IAR）

配套例子：

V7-028_串行 SPI Flash W25QXX 读写例程（查询方式 V1.1）

V7-050_串行 SPI Flash W25QXX 读写例程（DMA 方式）

V7-051_串行 SPI Flash W25QXX 读写例程（中断方式）

实验目的：

1. 学习 SPI Flash 的读写实现，支持查询，中断和 DMA 方式。

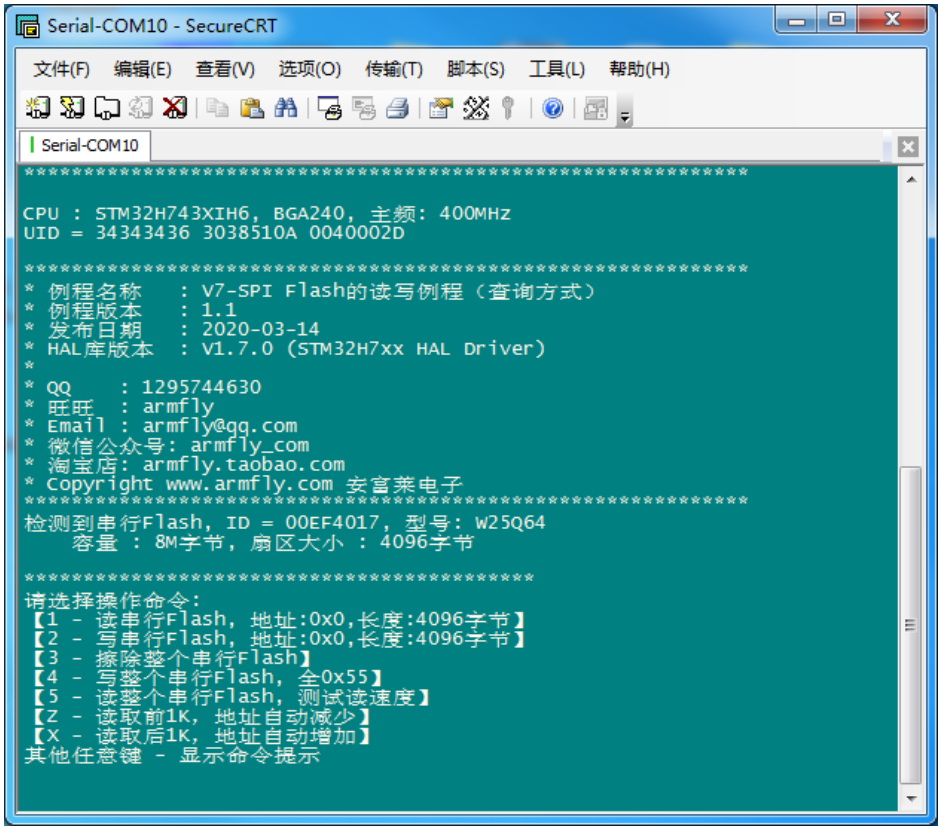
实验操作：

1. 支持以下 7 个功能，用户通过电脑端串口软件发送命令给开发板即可
2. printf("请选择操作命令:\r\n");
3. printf("【1 - 读串行 Flash, 地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
4. printf("【2 - 写串行 Flash, 地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
5. printf("【3 - 擦除整个串行 Flash】\r\n");

6. printf("【4 - 写整个串行 Flash，全 0x55】\r\n");
7. printf("【5 - 读整个串行 Flash，测试读速度】\r\n");
8. printf("【Z - 读取前 1K，地址自动减少】\r\n");
9. printf("【X - 读取后 1K，地址自动增加】\r\n");
10. printf("其他任意键 - 显示命令提示\r\n");

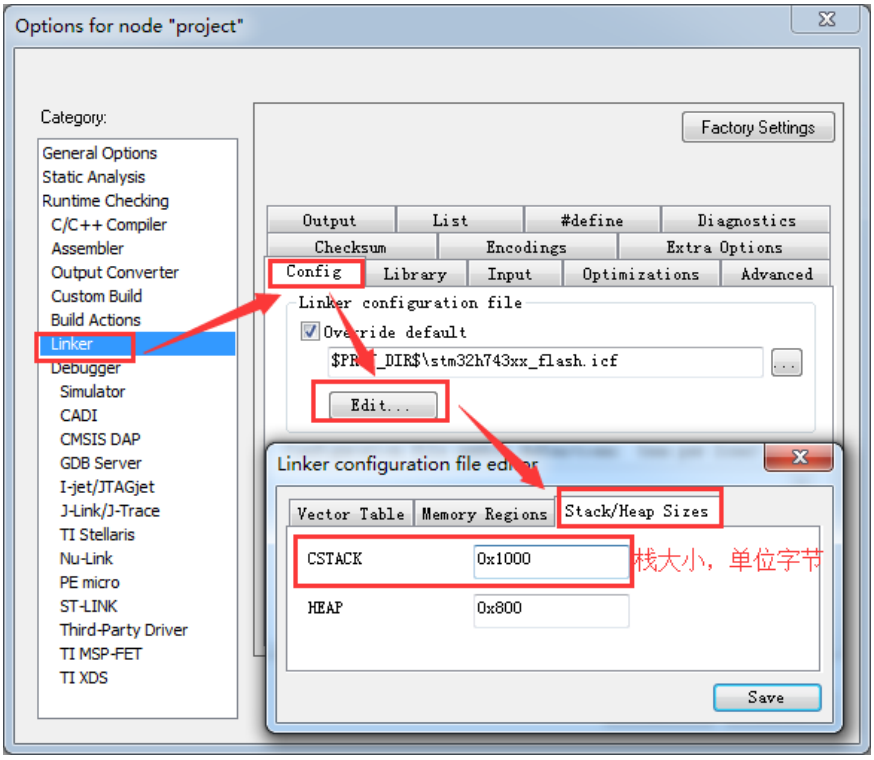
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1。

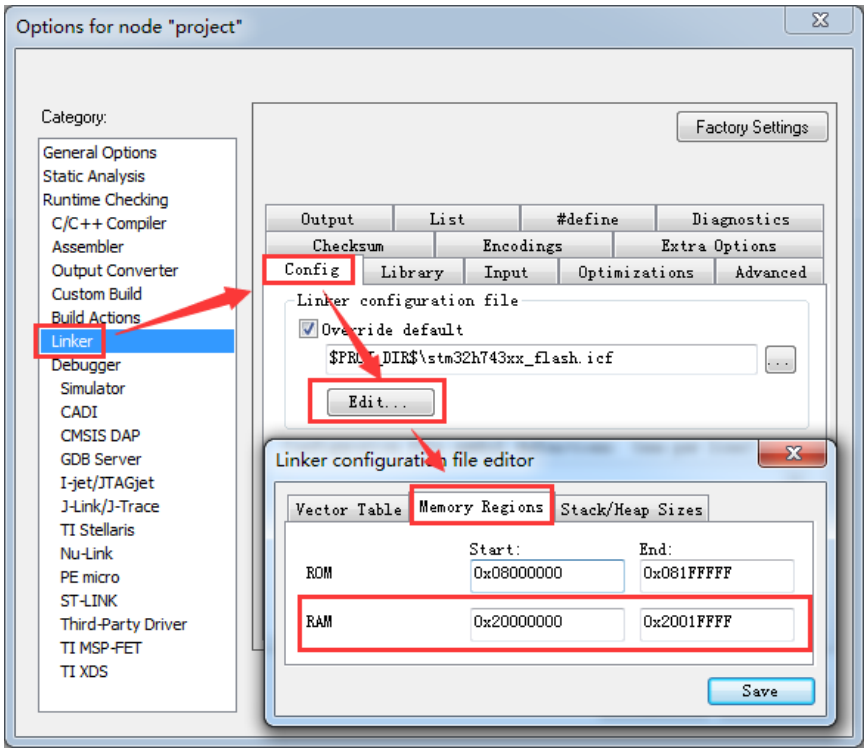


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
```

```
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitLPUart();  /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitSPIBus();  /* 配置 SPI 总线 */
    bsp_InitSFlash();  /* 初始化 SPI 串行 Flash */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
```

```
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

```
    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **每 10ms 调用一次按键处理：**

按键处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能：**

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- 支持以下 7 个功能，用户通过电脑端串口软件发送命令给开发板即可
- 请选择操作命令:
- 1 - 读串行 Flash
- 2 - 写串行 Flash
- 3 - 擦除整个串行 Flash
- 4 - 写整个串行 Flash
- 5 - 读整个串行 Flash
- Z - 读取前 1K
- X - 读取后 1K

```
/*
```

```
*********************************************************************************************************
*    函 数 名: DemoSpiFlash
*    功能说明: 串行 EEPROM 读写例程
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoSpiFlash(void)
{
    uint8_t cmd;
    uint32_t uiReadPageNo = 0;


    /* 检测串行 Flash OK */
    printf("检测到串行 Flash, ID = %08X, 型号: %s \r\n", g_tSF.ChipID , g_tSF.ChipName);
    printf("    容量 : %dM 字节, 扇区大小 : %d 字节\r\n", g_tSF.TotalSize/(1024*1024), g_tSF.SectorSize);

    sfDispMenu();      /* 打印命令提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    while(1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        if (comGetChar(COM1, &cmd)) /* 从串口读入一个字符(非阻塞方式) */
        {
            switch (cmd)
            {
                case '1':
                    printf("\r\n【1 - 读串行 Flash, 地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
                    sfReadTest(); /* 读串行 Flash 数据，并打印出来数据内容 */
                    break;

                case '2':
                    printf("\r\n【2 - 写串行 Flash, 地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
                    sfWriteTest();     /* 写串行 Flash 数据，并打印写入速度 */
                    break;

                case '3':
                    printf("\r\n【3 - 擦除整个串行 Flash】\r\n");
                    printf("整个 Flash 擦除完毕大概需要 20 秒左右，请耐心等待");
                    sfErase();         /* 擦除串行 Flash 数据，实际上就是写入全 0xFF */
                    break;

                case '4':
                    printf("\r\n【4 - 写整个串行 Flash, 全 0x55】\r\n");
                    printf("整个 Flash 写入完毕大概需要 20 秒左右，请耐心等待");
                    sfWriteAll(0x55);/* 擦除串行 Flash 数据，实际上就是写入全 0xFF */
                    break;
```

```
                case '5':
                    printf("\r\n【5 - 读整个串行 Flash, %dM 字节】\r\n", g_tSF.TotalSize/(1024*1024));
                    sfTestReadSpeed(); /* 读整个串行 Flash 数据，测试速度 */
                    break;

                case 'z':
                case 'Z': /* 读取前 1K */
                    if (uiReadPageNo > 0)
                    {
                        uiReadPageNo--;
                    }
                    else
                    {
                        printf("已经是最前\r\n");
                    }
                    sfViewData(uiReadPageNo * 1024);
                    break;

                case 'x':
                case 'X': /* 读取后 1K */
                    if (uiReadPageNo < g_tSF.TotalSize / 1024 - 1)
                    {
                        uiReadPageNo++;
                    }
                    else
                    {
                        printf("已经是最后\r\n");
                    }
                    sfViewData(uiReadPageNo * 1024);
                    break;

                default:
                    sfDispMenu(); /* 无效命令，重新打印命令提示 */
                    break;

            }
        }
    }
}
```

73.11 总结

本章节就为大家讲解这么多，实际应用中根据需要选择 DMA，中断和查询方式。

第74章 STM32H7 的 SPI 总线应用之驱动 DAC8563 (双通道，16bit 分辨率，正负 10V)

本章节为大家讲解标准 SPI 接线方式驱动模数转换器 DAC856X，制作了中断和 DMA 两种驱动方式。

74.1 初学者重要提示
74.2 DAC 结构分类
74.3 DAC 技术术语
74.4 DAC856X 硬件设计
74.5 DAC856X 关键知识点整理（重要）
74.6 DAC856X 驱动设计（中断更新方式）
74.7 DAC856X 驱动设计（SPI DMA 更新方式）
74.8 SPI 总线板级支持包（bsp_spi_bus.c）
74.9 DAC856X 支持包中断方式（bsp_spi_dac8562.c）
74.10 DAC856X 支持包 DMA 方式（bsp_spidma_dac8562.c）
74.11 DAC856X 驱动移植和使用（中断更新方式）
74.12 DAC856X 驱动移植和使用（SPI DMA 更新方式）
74.13 实验例程设计框架
74.14 实验例程说明（MDK）
74.15 实验例程说明（IAR）
74.16 总结

74.1 初学者重要提示

- 学习本章节前，务必优先学习第 72 章。
- 对于 DAC8562 和 DAC8563，教程中不做区分，因为 DAC8562 和 DAC8563 完全兼容，区别仅仅在于 CLR 引脚有效时，DAC8562 数据设置为 0， DAC8563 数据设置为 32767。
- 本章涉及到的知识点比较多，需要大家掌握 STM32H7 的 SPI，DMA，TIM，DMAMUX 和 DAC8563 的一些细节用法。
- H7 的 SPI + DMA 驱动这类外设的灵活度，绝对可以媲美 FPGA 去控制：

- H7 的 SPI 外设比 F4 系列的灵活性强太多了,主要表现在两个方面:数据的传输支持了 4-32bit,特别是那个 NSS 片选引脚,超强劲,可以做各种时间插入,灵活应对了市场上这类芯片的需求。
- DMA 这块相比 F4 系列,有了质的飞跃,支持了 DMAMUX,这个 DMAMUX 除了带来灵活的触发源选择,还支持了各种触发事件和同步触发功能。本章配套例子的触发周期控制就是利用了 DMAMUX 的同步触发功能。

◆ 本章配套了中断和 DMA 两种更新方式的案例,DMA 实现方式与中断更新方式完全不同,因为 DMA 方式要使用硬件 SPI1 NSS 片选引脚驱动 DAC856X。而中断更新方式使用公共的总线驱动文件 bsp_spi_bus.c,片选是通过通用 IO 方式控制,支持串行 FLASH、TSC2046、VS1053、AD7705、ADS1256 等 SPI 设备。大家在看例子的时候要注意。

◆ 对于本章教程配套例子的 SPI DMA 方式,这里特别注意一点,定时器触发一次,就会让 SPI 以 DMA 方式传输 24bit 数据。

◆ DAC856X 数据手册,模块原理图和接线图都已经放到本章教程配置例子的 Doc 文件里。

74.2 DAC 结构分类

这里将三种 DAC 结构为大家做个普及:R2R 型 MDAC,R2R 型 backDAC 和 Srting 型 DAC。

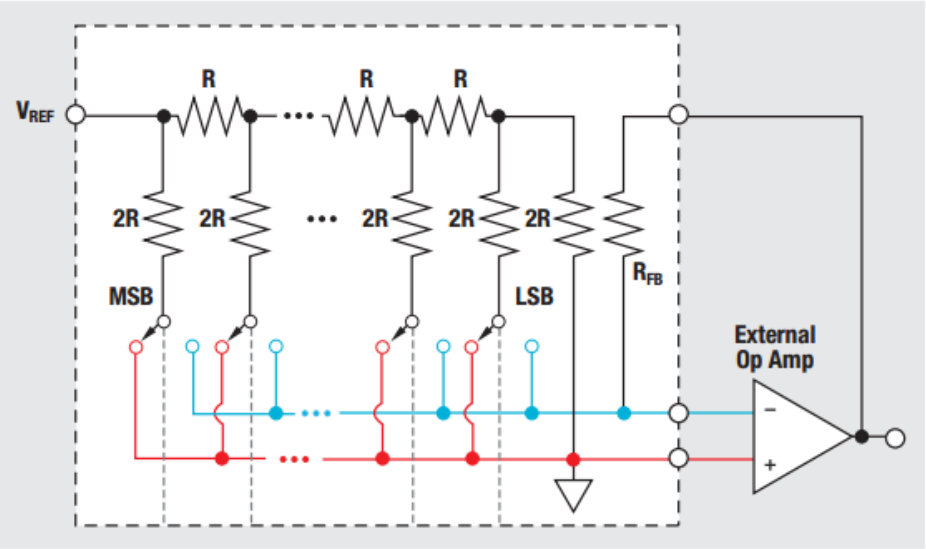
注,这些知识翻译自 TI 的英文技术手册。

74.2.1 R2R 型 MDAC

自动测试设备或仪器通常使用 R2R MDAC。MDAC 型制造商能够设计具有±1 LSB 的高分辨率积分非线性(INL)和差分非线性(DNL)DAC。通过使用合适的外部放大器,MDAC 能够实现较短的建立时间(<0.3 ms)和大于 10 MHz 的带宽。并且通过为 MDAC 的外部运算放大器提供不同电源电压和高输出电流可以增强 DAC 功能。

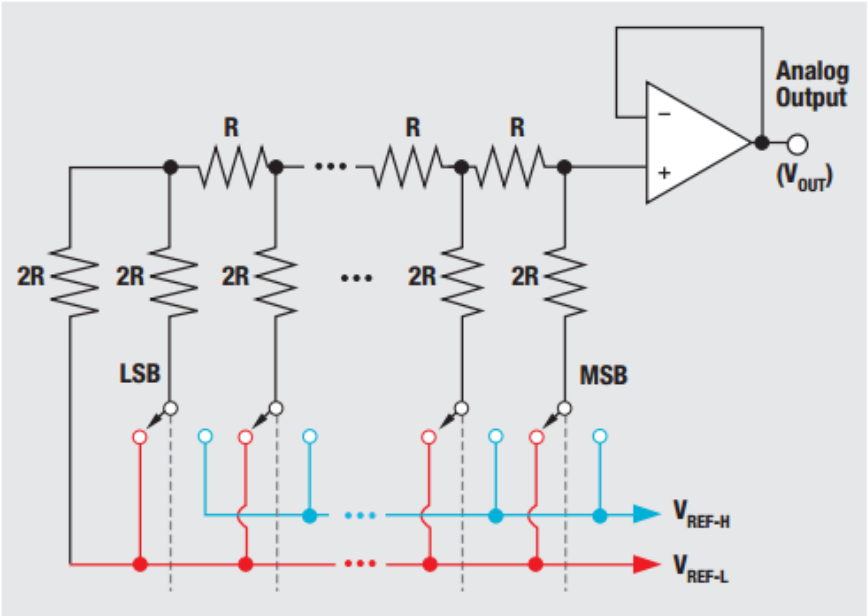
MDAC 产生的电流与用户设置的数字编码,外部放大器以及 RFB(在 MDAC 内部)将 DAC 的电流输出信号转换为可用的电压。

这类 DAC 的缺点是会有稳定性问题。



74.2.2 R2R 型 backDAC

通常在工业应用中使用 R2R backDAC。其它一些应用还包括仪器和数字控制校准。使用这类 DAC，每次新更新会将 2R 支路切换到参考电压高（VREF-H）或参考电压低（VREF-L）。注意 R-2R 梯子的布置与 MDAC 相比是倒置的。这就是名字 backDAC 的由来，这种架构很容易制造。



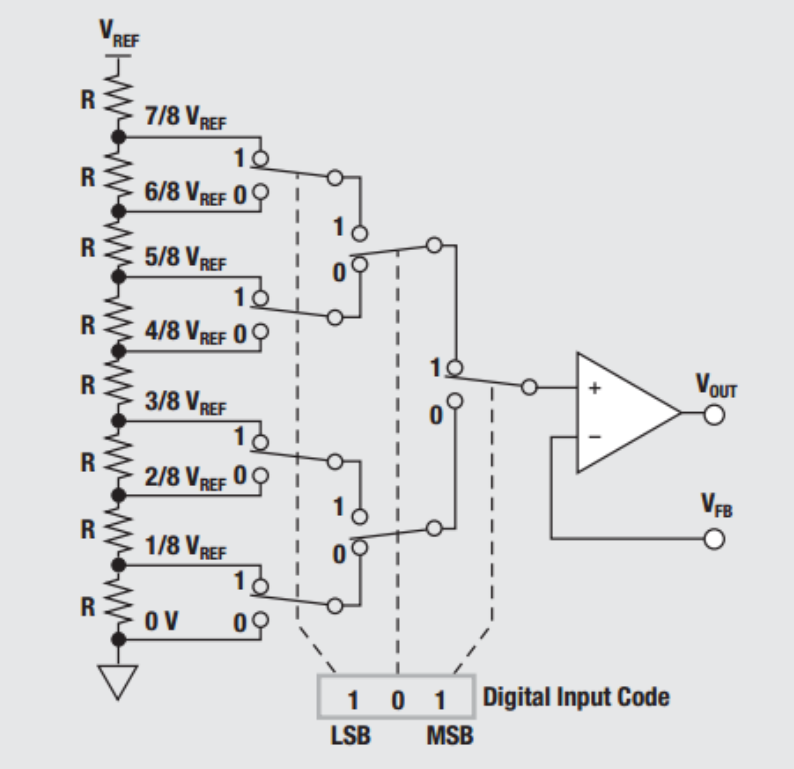
这类 DAC 的缺点是毛刺脉冲问题（注，此贴有详细解释：链接）：

74.2.3 String 型 DAC

String 型 DAC 最适合便携式仪器，闭环伺服控制和过程控制。下图显示了一个 3bit String DAC 的模型，数字输入代码 101b 被解码为 5/8 VREF。String DAC 的输出级放大器隔离了来自输出负载的内部电阻元件。

String DAC 是一种低功耗解决方案，可确保单调性在整个输入代码中具有良好的 DNL(差分非线性)性能范围。毛刺能量通常低于其它 DAC 类型。

但是，INL（积分非线性）通常较大，具体取决于电阻式片上匹配，另一方面，控制回路中的 DAC 可减轻线性度影响。String DAC 的噪声也相对较大，因为电阻串的阻抗很高，所以该值很高。但 String DAC 功耗低且非常小的故障能量。



74.3 DAC 技术术语

一些常见的 DAC 技术术语需要大家见到了，大概了解是什么意思。

74.3.1 单位 ppm℃（ppm/℃)

这个参数是专门用来定义温飘的，ppm 全称是 parts per million，即百万分之一。比如 2ppm℃就是 2 x 10^-6 ，反映到 DAC8563 上，定义如下:

```
Input or 2.5-V Output
4-ppm° C Temperature Drift (Typ)
```

也就是说，当输出 2.5V 时，每变化一度，输出电压的变化是 2.5V x （4 x 10^-6） = 10uV

类似的定义还有很多：

Measure	SI units	Named parts-per ratio (short scale)	Parts-per abbreviation or symbol	Value in scientific notation
A strain of...	2 cm/m	2 parts per hundred	2%[13]	2×10^{-2}
A sensitivity of...	2 mV/V	2 parts per thousand	2 ‰	2×10^{-3}
A sensitivity of...	0.2 mV/V	2 parts per ten thousand	2 ‱	2×10^{-4}
A sensitivity of...	2 µV/V	2 parts per million	2 ppm	2×10^{-6}
A sensitivity of...	2 nV/V	2 parts per billion	2 ppb	2×10^{-9}
A sensitivity of...	2 pV/V	2 parts per trillion	2 ppt	2×10^{-12}
A mass fraction of...	2 mg/kg	2 parts per million	2 ppm	2×10^{-6}
A mass fraction of...	2 µg/kg	2 parts per billion	2 ppb	2×10^{-9}
A mass fraction of...	2 ng/kg	2 parts per trillion	2 ppt	2×10^{-12}
A mass fraction of...	2 pg/kg	2 parts per quadrillion	2 ppq	2×10^{-15}
A volume fraction of...	5.2 µL/L	5.2 parts per million	5.2 ppm	5.2×10^{-6}
A mole fraction of...	5.24 µmol/mol	5.24 parts per million	5.24 ppm	5.24×10^{-6}
A mole fraction of...	5.24 nmol/mol	5.24 parts per billion	5.24 ppb	5.24×10^{-9}
A mole fraction of...	5.24 pmol/mol	5.24 parts per trillion	5.24 ppt	5.24×10^{-12}
A stability of...	1 (µA/A)/min.	1 part per million per min.	1 ppm/min.	1×10^{-6}/min.
A change of...	5 nΩ/Ω	5 parts per billion	5 ppb	5×10^{-9}
An uncertainty of...	9 µg/kg	9 parts per billion	9 ppb	9×10^{-9}
A shift of...	1 nm/m	1 part per billion	1 ppb	1×10^{-9}
A strain of...	1 µm/m	1 part per million	1 ppm	1×10^{-6}
A temperature coefficient of...	0.3 (µHz/Hz)/°C	0.3 part per million per °C	0.3 ppm/°C	0.3×10^{-6}/°C
A frequency change of...	0.35×10^{-9} f	0.35 part per billion	0.35 ppb	0.35×10^{-9}

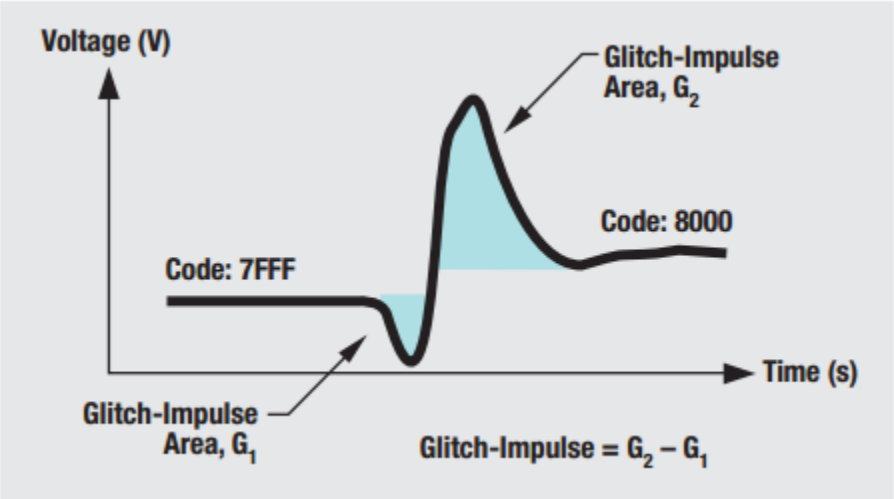
ppb，ppt，ppq 所代表的含义：

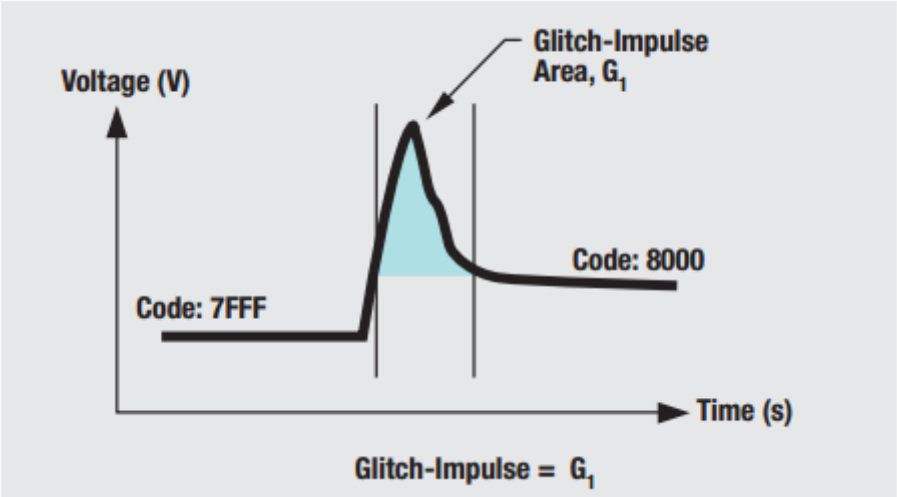
IUPAP's "uno" proposal

Coefficient	Name	Parts-per example	Uno equiv.	Symbol form	Value of quantity
10^{-2}	Percent	2%	2 centiuno	2 cU	2×10^{-2}
10^{-3}	Permille	2 ‰	2 milliuno	2 mU	2×10^{-3}
10^{-4}	Per ten thousand	2 ‱	0.2 milliuno	0.2 mU	2×10^{-4}
10^{-6}	Parts per million	2 ppm	2 microuno	2 µU	2×10^{-6}
10^{-9}	Parts per billion	2 ppb	2 nanouno	2 nU	2×10^{-9}
10^{-12}	Parts per trillion	2 ppt	2 picouno	2 pU	2×10^{-12}
10^{-15}	Parts per quadrillion	2 ppq	2 femtouno	2 fU	2×10^{-15}

74.3.2 毛刺脉冲（Glitch impulse）

使用 DAC 进行设计时，您期望输出从一个值单调移至下一个值，但实际电路并非总是如此。在某些代码范围内，出现过冲或下冲（量化为毛刺脉冲）并不少见。主要以下面两种形式呈现：

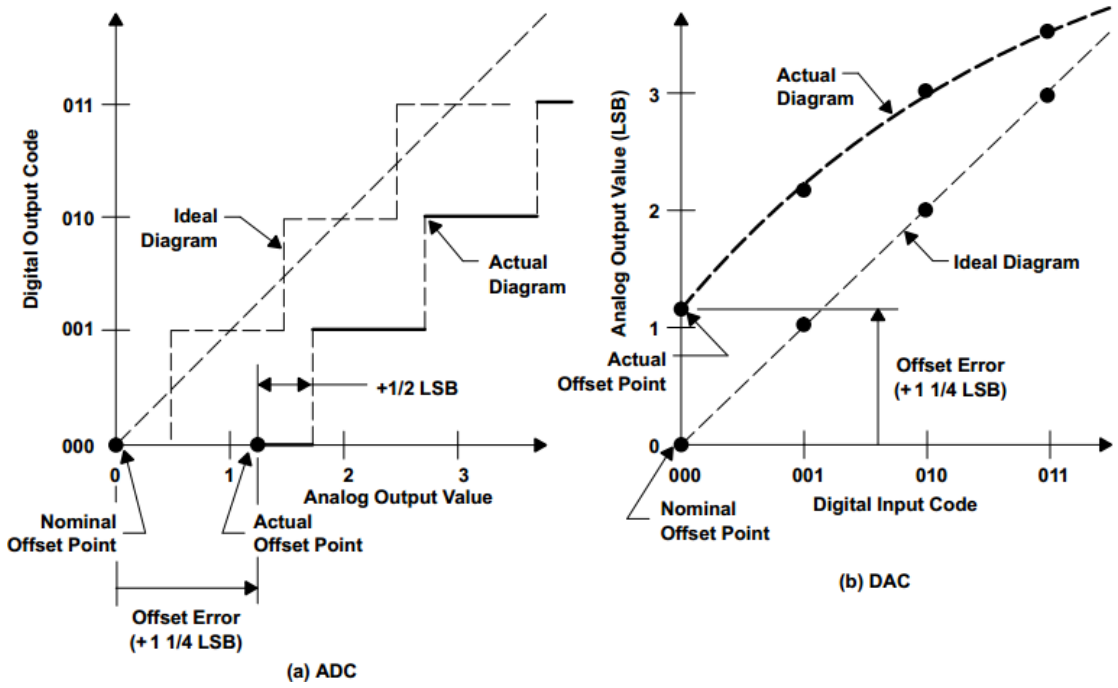




具体原因分析在这个帖子里面进行了讲解（内容较多，就不整理到教程里面了）：链接。

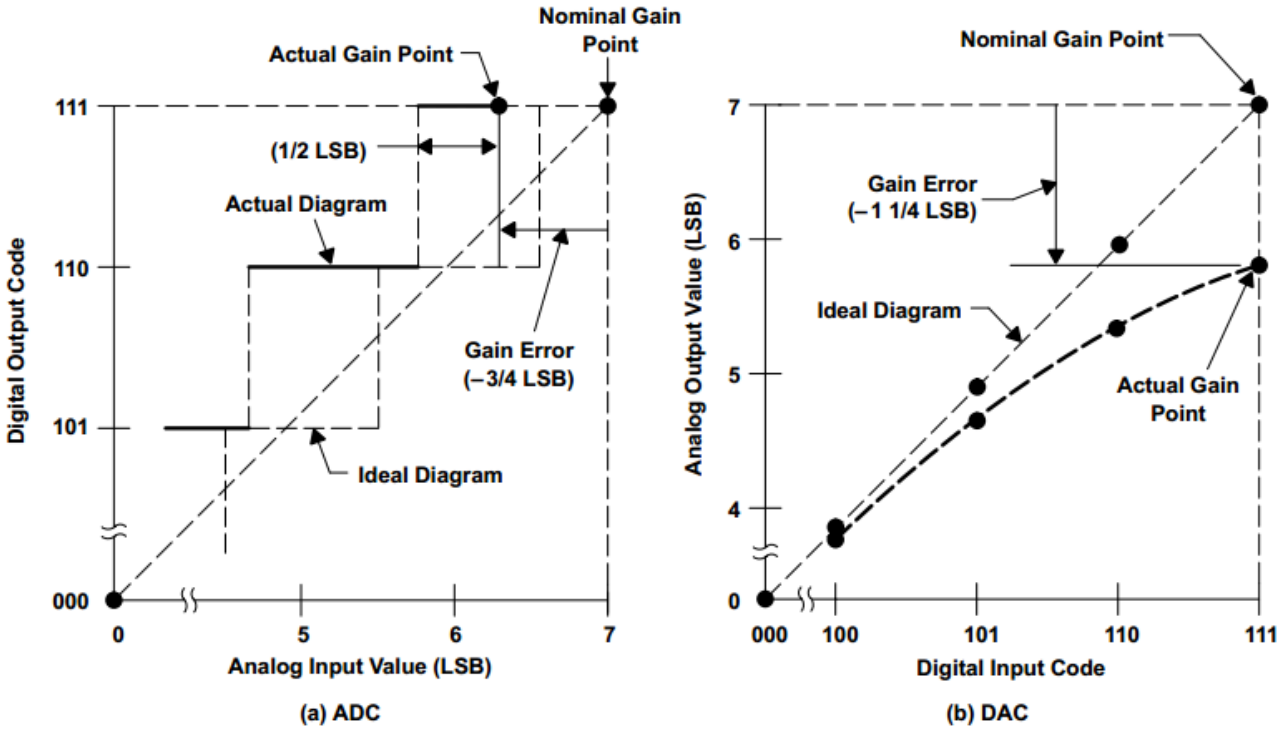
74.3.3偏移误差（Offset Error）

偏移误差为标称偏移点与实际偏移点之间的差。此错误以相同的数量影响所有代码，通常可以通过修正来补偿处理。如果无法修正，则该误差称为零刻度误差。



74.3.4增益误差（Gain Error)

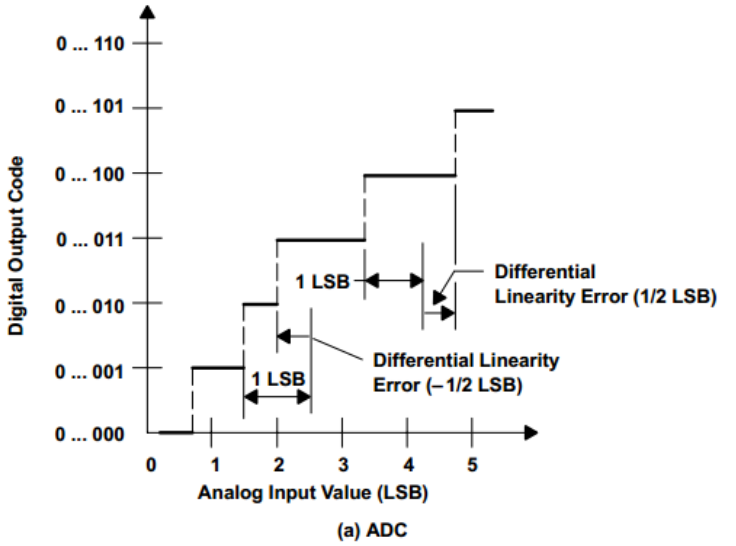
增益误差定义为传输时标称增益点与实际增益点之差。

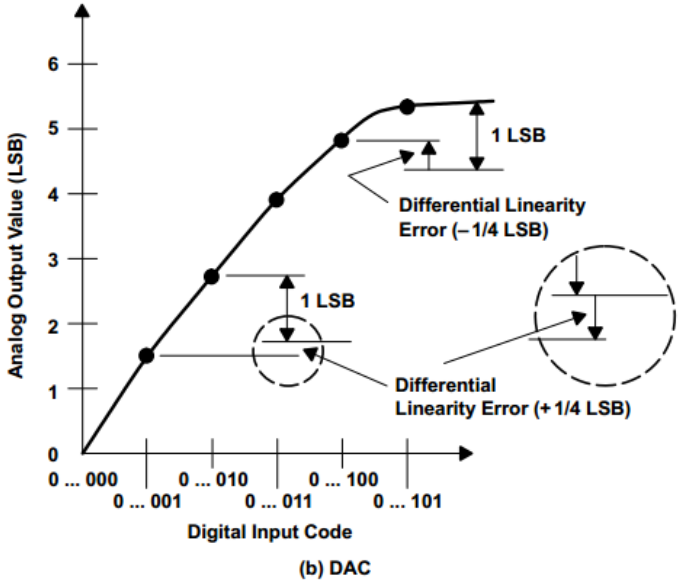


74.3.5差分非线性误差（DNL）

DNL 全称 Differential Nonlinearity。

差分非线性误差为实际步长宽度（对于 ADC）或步长高度（对于 DAC）与 1 LSB 的理想值之间的差值。 因此，如果阶跃宽度或高度恰好为 1 LSB，则差分非线性误差为零。 如果 DNL 超过 1 LSB，转换器可能变得非单调。这意味着增加了输入的幅度但输出的大小可能变小。



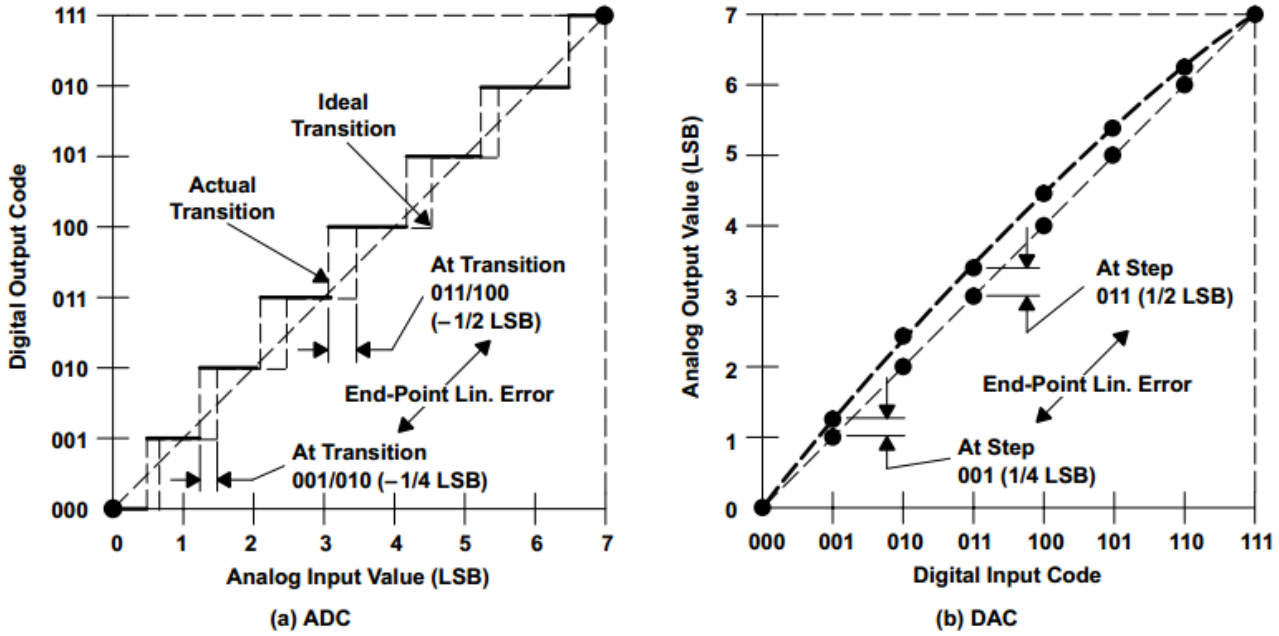


Differential Linearity Error of a Linear ADC or DAC

74.3.6积分非线性误差（INL）

INL 全称 Integral Nonlinearity

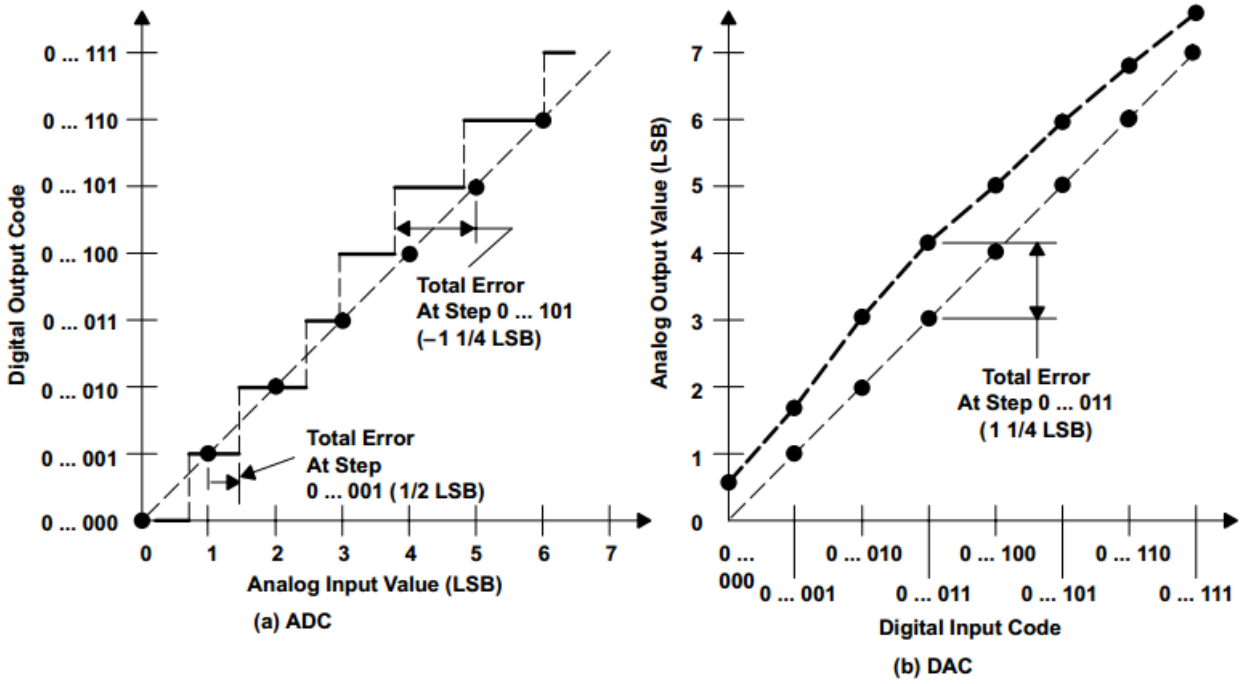
积分非线性误差是从一个传输点到相对应的理想传输曲线的最大偏差距离，不考虑偏置误差和增益误差。 这个参数对最佳传输函数或端点传输函数有一定参考意义。



End-Point Linearity Error of a Linear 3-Bit Natural Binary-Coded ADC or DAC
(Offset Error and Gain Error are Adjusted to the Value Zero)

74.3.7绝对精度误差（Absolute Accuracy Error)

绝对精度误差是包括偏移，增益，积分线性等误差的总体误差。



Absolute Accuracy or Total Error of a Linear ADC or DAC

74.4 DAC856X 硬件设计

DAC 的输出量可以为 0 到 2.5V 或者 0 到 5V，通过外置运放，实现了±10V 输出。原理图下载：http://www.armbbs.cn/forum.php?mod=viewthread&tid=97082 。

74.4.1 DAC856X 模块规格

产品规格：

1、供电电压 ： 2.7 - 5.5V 【3.3V 供电时，输出电压也可以到正负 10V】

2、通道数： 2 路 （通过 1 片 DAC8563 实现）

3、输出电压范围 ： -10V ~ +10V 【客户可以自己更改为 0-10V 输出范围。使用烙铁切换 2 个焊点即可，无需更换元器件】

4、输出驱动能力：带运放驱动，最大输出电流 10mA，负载电阻>1K 欧姆

5、分辨率： 16 位

6、功耗 ：小于 20mA

7、MCU 接口 ： 高速 SPI （50M）支持 3.3V 和 5V 单片机

8、DAC 输出模拟带宽：350KHz

9、DAC 输出响应： 10uS 到 0.003% FSR

产品特点：

1、输出和供电电压无关；模块内带正负 12V 升压电路

2、自适应单片机的电平（2.7 - 5V 均可以）

3、输出电压可抵达正负 10V

4、上电时缺省输出 0V (在软件未启动时)

5、引出正负 12V 电源排针，方便客户使用

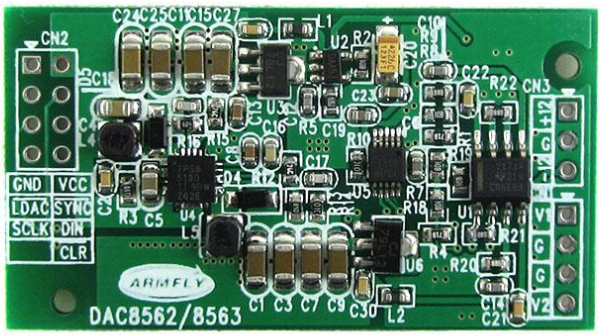
重要提示：

1、DAC8562 和 DAC8563 完全兼容，区别仅仅在于 CLR 引脚有效时，DAC8562 数据设置为 0，DAC8563 数据设置为 32767。注意这是 DAC 的内部数据，不表示输出电压。 对于-10 ~ +10V 输出的模块，DAC8562 输出-10V，DAC8563 输出 0V。

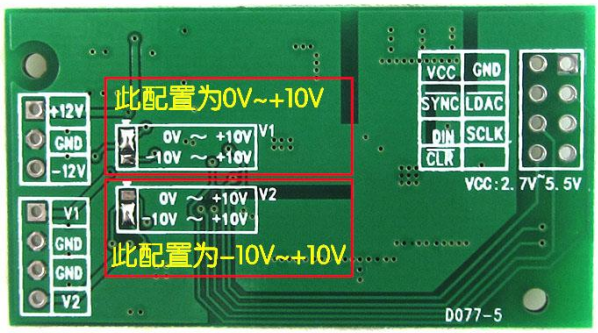
2、无论是用 DAC8562 还是 DAC8563 芯片，只要软件不启动，本模块输出电压缺省状态都是 0V。

3、CLR 脚悬浮时，电压在 1.9V 左右，容易受到干扰导致输出被清零。因此即使不用 CLR 控制功能，这个 CLR 脚也需要接固定电平（推荐接 GND）。CLR 是边沿触发，仅在下降沿信号出现执行清零。

产品效果：

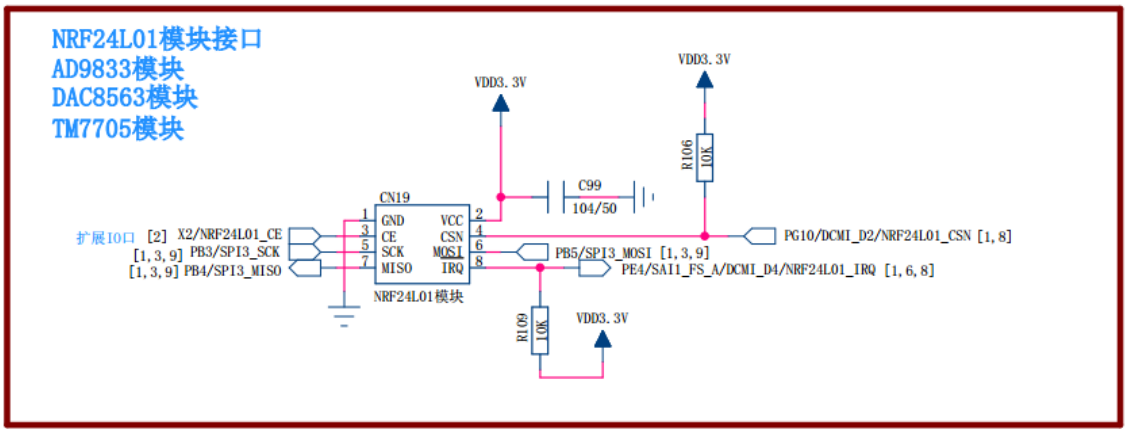


用户可使用烙铁切换焊点，自行改变输出范围

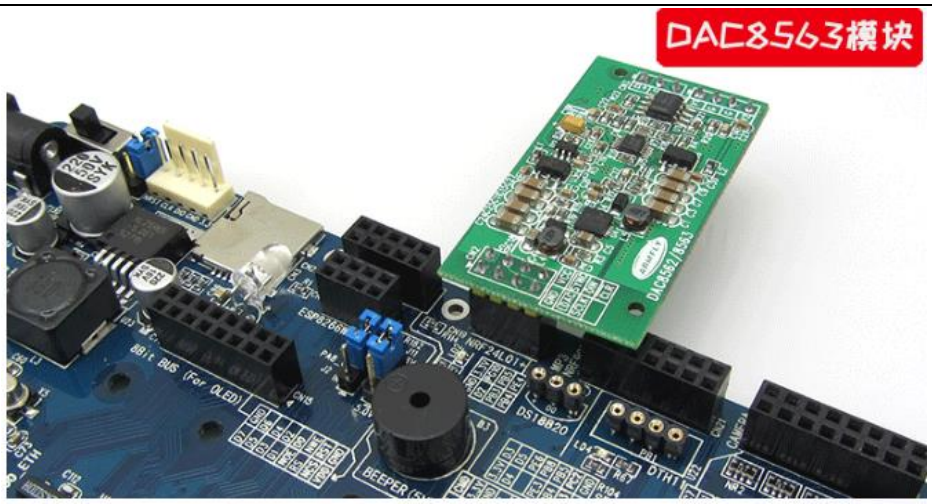


74.4.2 DAC856X 硬件接口

V7 板子上 DAC856X 模块的插座的原理图如下：



实际对应开发的位置如下：

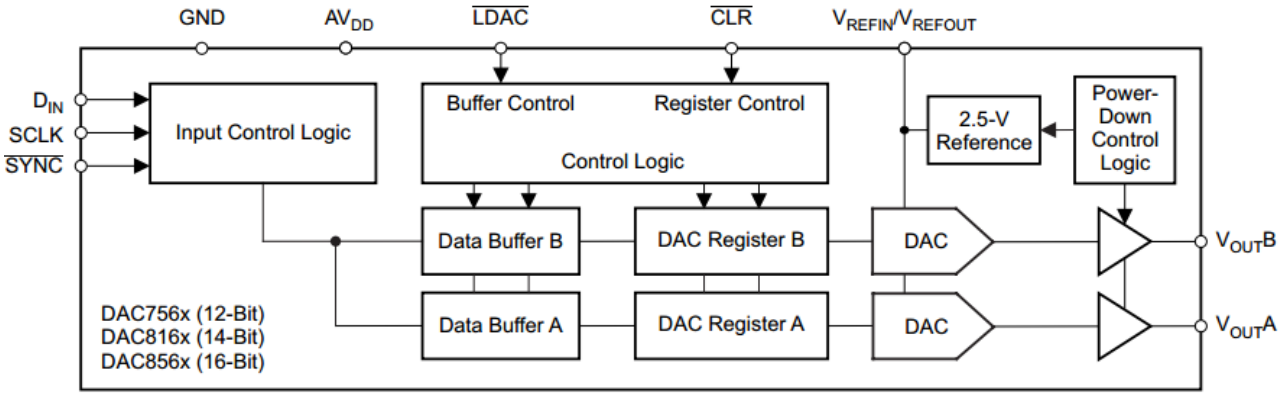


74.5 DAC856X 关键知识点整理（重要）

驱动 DAC856X 需要对下面这些知识点有个认识。

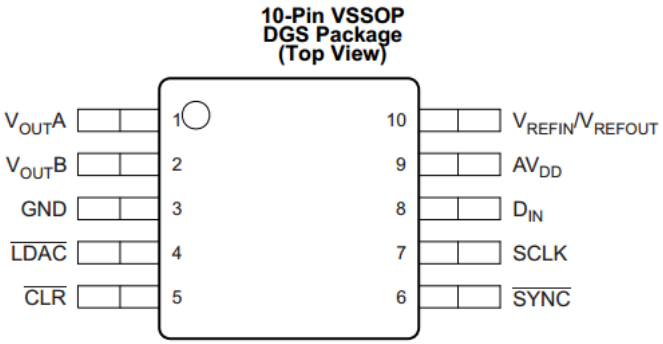
74.5.1 DAC856X 基础信息

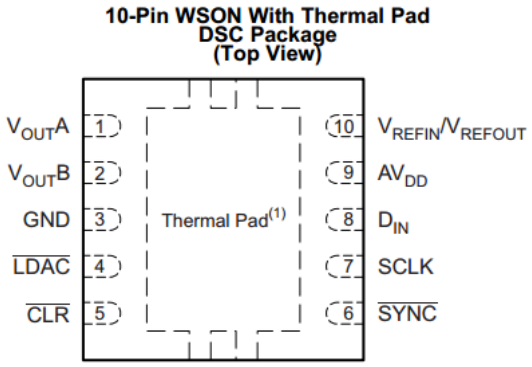
- 双通道 DAC，轨到轨输出，16bit 分辨率，支持 50MHz 的 SPI 时钟速度。
- 自带 2.5V 的内部参考基准，典型的温飘是 4ppm/℃。使用内部 2.5V 参考基准的情况下，根据增益设置不同，DAC 的输出量可以为 0 到 2.5V 或者 0 到 5V。
- 用户可以根据需要外接运放实现常用的±5V，±10V 或者±15V 输出。
- 相对精度误差 4LSB INL。
- 毛刺脉冲 0.1nV-s
- 上电复位数值 0V 或者中间值。



74.5.2 DAC856X 每个引脚的作用

DAC856X 主要有下面两种封装形式：





- Avdd

  供电范围 2.7-5.5V

- CLR

  异步清除输入，下降沿有效，触发后，DAC8562 输出最低电压值，DAC8563 输出中间值。用户写入操作的的第 24 个时钟下降沿将退出清除模式，激活清除模式将终止写操作。

- Din

  串行时钟输入，每个时钟下降沿将数据写到的 24bit 的输入移位寄存器。

- GND

  接地端。

- LDAC

  同步模式下，数据更新发生在第 24 个 SCLK 周期的下降沿，之后伴随着 SYNC 的下降沿。 这种同步更新不需要 LDAC，而 LDAC 必须永久接地，或者将命令发送到设备时保持低电平。异步模式下，LDAC 是低电平触发，用于同步 DAC 更新，可以编写多个单通道命令进行设置，然后在 LDAC 引脚上产生一个下降沿将同步更新 DAC 输出寄存器。

- SCLK

  时钟输入端，支持 50MHz。

- SYNC （片选）

  低电平有效，当 SYNC 变为低电平时，它使能输入移位寄存器，并且数据采样在随后的时钟下降沿。DAC 输出在第 24 个时钟下降沿之后更新。 如果 SYNC 在第 23 个时钟沿之前变高，SYNC 的上升沿将充当中断，并且 DAC756x，DAC816x 和 DAC856x 器件将忽略写序列。

- VoutA

  模拟电压输出 A。

- VoutB

  模拟电压输出 B。

- Vrefin/Vrefout

  双向电压参考引脚，如果使用内部电压基准，此引脚是输出 2.5V。

74.5.3 DAC856X 输出电压计算公式

DAC856X 的计算公式如下：

$$V_{OUT} = \left(\frac{D_{IN}}{2^n}\right) \times V_{REF} \times Gain$$

- D_{IN}

  配置 DAC856X 数据输出寄存器的数值，范围 0 到 2^16 – 1，即 0 到 65535。

- 2^n

  对于 DAC856X 来说，n 是 16。

- V_{REF}

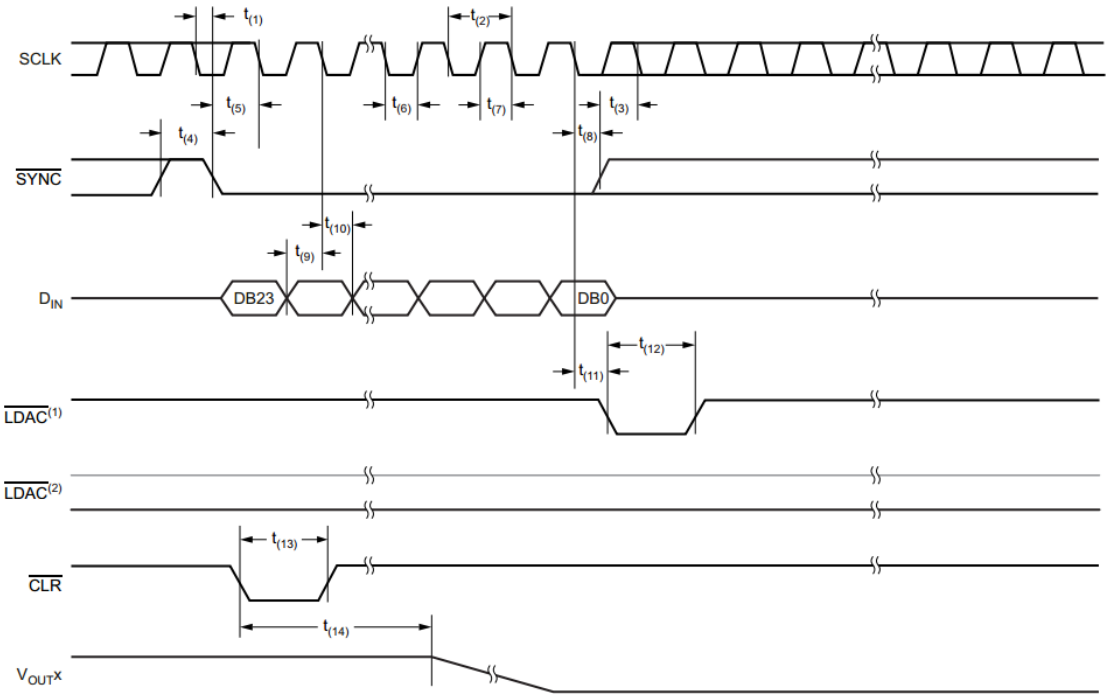
  如果使用内部参考电压，那么此数值是 2.5V，如果使用外部参考电压，由 V_{REFIN} 引脚的输入决定。

- Gain

  增益设置。禁止内部电压基准后，默认增益是 1。如果使能内部电压基准后，默认增益是 2。具体增益是 1 还是 2，可以通过 DAC856X 的寄存器设置。

74.5.4 DAC856X 时序图

DAC856X 的时序图如下：



(1) Asynchronous LDAC update mode. For more information, see the *LDAC Functionality* section.
(2) Synchronous LDAC update mode; LDAC remains low. For more information, see the *LDAC Functionality* section.

		DAC756x, DAC816x, DAC856x			UNIT
		MIN	TYP	MAX	
$f_{(SCLK)}$	Serial clock frequency			50	MHz
$t_{(1)}$	SCLK falling edge to $\overline{SYNC}$ falling edge (for successful write operation)	10			ns
$t_{(2)}$	SCLK cycle time	20			ns
$t_{(3)}$	$\overline{SYNC}$ rising edge to 23rd SCLK falling edge (for successful $\overline{SYNC}$ interrupt)	13			ns
$t_{(4)}$	Minimum $\overline{SYNC}$ HIGH time	80			ns
$t_{(5)}$	$\overline{SYNC}$ to SCLK falling edge setup time	13			ns
$t_{(6)}$	SCLK LOW time	8			ns
$t_{(7)}$	SCLK HIGH time	8			ns
$t_{(8)}$	SCLK falling edge to $\overline{SYNC}$ rising edge	10			ns
$t_{(9)}$	Data setup time	6			ns
$t_{(10)}$	Data hold time	5			ns
$t_{(11)}$	SCLK falling edge to $\overline{LDAC}$ falling edge for asynchronous LDAC update mode	5			ns
$t_{(12)}$	$\overline{LDAC}$ pulse duration, LOW time	10			ns
$t_{(13)}$	$\overline{CLR}$ pulse duration, LOW time	80			ns
$t_{(14)}$	$\overline{CLR}$ falling edge to start of V_{OUT} transition			100	ns

(1) All input signals are specified with $t_r = t_f = 3$ ns (10% to 90% of AV_{DD}) and timed from a voltage level of $(V_{IL} + V_{IH}) / 2$.
(2) See the *Serial Write Operation* timing diagram (Figure 1).

这个时序里面有三个参数尤其重要，后面时序配置要用到。

- $f_{(SCLK)}$

  支持最高的串行时钟是 50MHz。

- $t_{(4)}$

  每传输 24bit 数据后，SYNC 要保持一段时间的高电平，DAC856X 要求至少要 80ns。

- $t_{(5)}$

  SYNC 低电平有效到 SCLK 第 1 个下降沿信号的时间，最小值 13ns。

74.5.5 DAC856X 寄存器配置

DAC856X 的寄存器配置看下面的图表即可，一目了然（X 表示为 0 或者为 1 均可）：

DB23-DB22	Command			Address			Data						DESCRIPTION
	C2	C1	C0	A2	A1	A0	DB15-DB6	DB5	DB4	DB3-DB2	DB1	DB0	
X(1)	0	0	0	0	0	0	16-, 14-, or 12-bit DAC data						Write to DAC-A input register
				0	0	1	16-, 14-, or 12-bit DAC data						Write to DAC-B input register
				1	1	1	16-, 14-, or 12-bit DAC data						Write to DAC-A and DAC-B input registers
X	0	1	0	0	0	0	16-, 14-, or 12-bit DAC data						Write to DAC-A input register and update all DACs
				0	0	1	16-, 14-, or 12-bit DAC data						Write to DAC-B input register and update all DACs
				1	1	1	16-, 14-, or 12-bit DAC data						Write to DAC-A and DAC-B input register and update all DACs
X	0	1	1	0	0	0	16-, 14-, or 12-bit DAC data						Write to DAC-A input register and update DAC-A
				0	0	1	16-, 14-, or 12-bit DAC data						Write to DAC-B input register and update DAC-B
				1	1	1	16-, 14-, or 12-bit DAC data						Write to DAC-A and DAC-B input register and update all DACs
X	0	0	1	0	0	0	X						Update DAC-A
				0	0	1	X						Update DAC-B
				1	1	1	X						Update all DACs
X	0	0	0	0	1	0	X				0	0	Gain: DAC-B gain = 2, DAC-A gain = 2 (default with internal V_{REF})
											0	1	Gain: DAC-B gain = 2, DAC-A gain = 1
											1	0	Gain: DAC-B gain = 1, DAC-A gain = 2
											1	1	Gain: DAC-B gain = 1, DAC-A gain = 1 (power-on default)
X	1	0	0	X			X	0	0	X	0	1	Power up DAC-A
											1	0	Power up DAC-B
											1	1	Power up DAC-A and DAC-B
X	1	0	0	X			X	0	1	X	0	1	Power down DAC-A; 1 kΩ to GND
											1	0	Power down DAC-B; 1 kΩ to GND
											1	1	Power down DAC-A and DAC-B; 1 kΩ to GND
X	1	0	0	X			X	1	0	X	0	1	Power down DAC-A; 100 kΩ to GND
											1	0	Power down DAC-B; 100 kΩ to GND
											1	1	Power down DAC-A and DAC-B; 100 kΩ to GND
X	1	0	0	X			X	1	1	X	0	1	Power down DAC-A; Hi-Z
											1	0	Power down DAC-B; Hi-Z
											1	1	Power down DAC-A and DAC-B; Hi-Z
X	1	0	1	X			X				X	0	Reset DAC-A and DAC-B input register and update all DACs
											X	1	Reset all registers and update all DACs (Power-on-reset update)
X	1	1	0	X			X				0	0	$\overline{LDAC}$ pin active for DAC-B and DAC-A
											0	1	$\overline{LDAC}$ pin active for DAC-B; inactive for DAC-A
											1	0	$\overline{LDAC}$ pin inactive for DAC-B; active for DAC-A
											1	1	$\overline{LDAC}$ pin inactive for DAC-B and DAC-A
X	1	1	1	X			X				X	0	Disable internal reference and reset DACs to gain = 1
											X	1	Enable internal reference and reset DACs to gain = 2

(1) *X* denotes *don't care* bits.

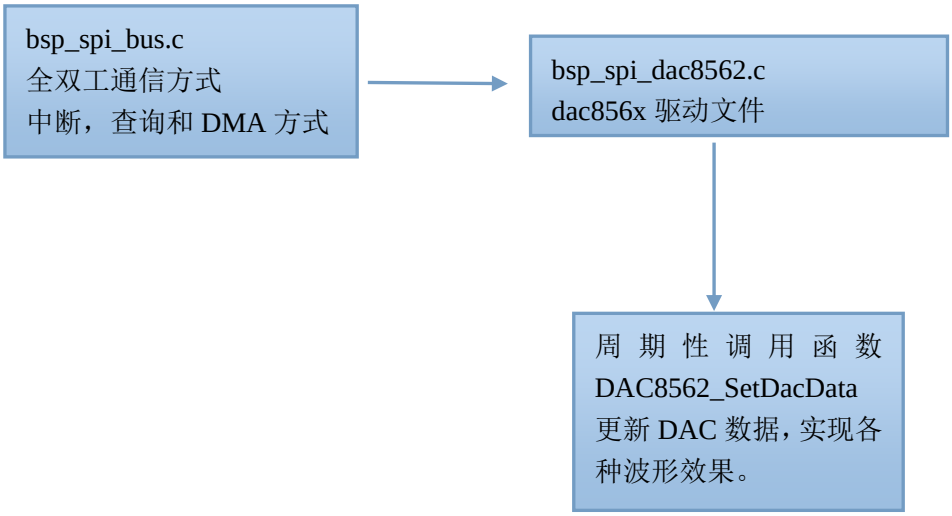
控制 DAC856X 每次要传输 24bit 数据，高 8bit 控制位 + 16bit 数据位。

比如 Power up DAC-A and DAC-B：

DAC8562_WriteCmd((4 << 19) | (0 << 16) | (3 << 0))

74.6 DAC856X 驱动设计（中断更新方式）

DAC856X 的程序驱动框架设计如下：



有了这个框图，程序设计就比较好理解了。

74.6.1第 1 步：SPI 总线配置

spi 总线配置通过如下两个函数实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitSPIBus
*    功能说明: 配置 SPI 总线。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitSPIBus(void)
{
    g_spi_busy = 0;

    bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_8, SPI_PHASE_1EDGE, SPI_POLARITY_LOW);
}

/*
*********************************************************************************************************
*    函 数 名: bsp_InitSPIParam
*    功能说明: 配置 SPI 总线参数，时钟分频，时钟相位和时钟极性。
*    形    参: _BaudRatePrescaler  SPI 总线时钟分频设置，支持的参数如下：
*                                 SPI_BAUDRATEPRESCALER_2    2 分频
*                                 SPI_BAUDRATEPRESCALER_4    4 分频
*                                 SPI_BAUDRATEPRESCALER_8    8 分频
*                                 SPI_BAUDRATEPRESCALER_16   16 分频
*                                 SPI_BAUDRATEPRESCALER_32   32 分频
*                                 SPI_BAUDRATEPRESCALER_64   64 分频
*                                 SPI_BAUDRATEPRESCALER_128  128 分频
*                                 SPI_BAUDRATEPRESCALER_256  256 分频
*
*              _CLKPhase          时钟相位，支持的参数如下：
*                                 SPI_PHASE_1EDGE     SCK 引脚的第 1 个边沿捕获传输的第 1 个数据
```

```
*                                SPI_PHASE_2EDGE     SCK 引脚的第 2 个边沿捕获传输的第 1 个数据
*
*             _CLKPolarity       时钟极性，支持的参数如下：
*                                SPI_POLARITY_LOW    SCK 引脚在空闲状态处于低电平
*                                SPI_POLARITY_HIGH   SCK 引脚在空闲状态处于高电平
*
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitSPIParam(uint32_t _BaudRatePrescaler, uint32_t _CLKPhase, uint32_t _CLKPolarity)
{
    /* 提高执行效率，只有在 SPI 硬件参数发生变化时，才执行 HAL_Init */
    if (s_BaudRatePrescaler == _BaudRatePrescaler && s_CLKPhase == _CLKPhase && s_CLKPolarity == _CLKPolarity)
    {
        return;
    }

    s_BaudRatePrescaler = _BaudRatePrescaler;
    s_CLKPhase = _CLKPhase;
    s_CLKPolarity = _CLKPolarity;


    /* 设置 SPI 参数 */
    hspi.Instance               = SPIx;                     /* 例化 SPI */
    hspi.Init.BaudRatePrescaler = _BaudRatePrescaler;       /* 设置波特率 */
    hspi.Init.Direction         = SPI_DIRECTION_2LINES;     /* 全双工 */
    hspi.Init.CLKPhase          = _CLKPhase;                /* 配置时钟相位 */
    hspi.Init.CLKPolarity       = _CLKPolarity;             /* 配置时钟极性 */
    hspi.Init.DataSize          = SPI_DATASIZE_8BIT;        /* 设置数据宽度 */
    hspi.Init.FirstBit          = SPI_FIRSTBIT_MSB;         /* 数据传输先传高位 */
    hspi.Init.TIMode            = SPI_TIMODE_DISABLE;       /* 禁止 TI 模式  */
    hspi.Init.CRCCalculation    = SPI_CRCCALCULATION_DISABLE; /* 禁止 CRC */
    hspi.Init.CRCPolynomial     = 7;                         /* 禁止 CRC 后，此位无效 */
    hspi.Init.CRCLength         = SPI_CRC_LENGTH_8BIT;      /* 禁止 CRC 后，此位无效 */
    hspi.Init.NSS               = SPI_NSS_SOFT;              /* 使用软件方式管理片选引脚 */
    hspi.Init.FifoThreshold     = SPI_FIFO_THRESHOLD_01DATA;  /* 设置 FIFO 大小是一个数据项 */
    hspi.Init.NSSPMode          = SPI_NSS_PULSE_DISABLE;      /* 禁止脉冲输出 */
    hspi.Init.MasterKeepIOState = SPI_MASTER_KEEP_IO_STATE_ENABLE; /* 禁止 SPI 后，SPI 相关引脚保持当前状态 */
    hspi.Init.Mode              = SPI_MODE_MASTER;           /* SPI 工作在主控模式 */

    /* 复位配置 */
    if (HAL_SPI_DeInit(&hspi) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 初始化配置 */
    if (HAL_SPI_Init(&hspi) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

关于这两个函数有以下两点要做个说明：

- 函数 bsp_InitSPIBus 里面的配置是个初始设置。实际驱动芯片时，会通过函数 bsp_InitSPIParam 做再配置。

- 函数 bsp_InitSPIParam 提供了时钟分频，时钟相位和时钟极性配置。驱动不同外设芯片时，基本上调整这三个参数就够。当 SPI 接口上接了多个不同类型的芯片时，通过此函数可以方便的切换配置。

74.6.2 第 2 步：SPI 总线的查询，中断和 DMA 方式设置

注：对于 DAC8563，请使用查询方式。

SPI 驱动的查询，中断和 DMA 方式主要通过函数 bsp_spiTransfer 实现数据传输：

```
/*
*********************************************************************************************************
*                                    选择 DMA，中断或者查询方式
*********************************************************************************************************
*/
//#define USE_SPI_DMA    /* DMA 方式  */
//#define USE_SPI_INT    /* 中断方式 */
#define USE_SPI_POLL   /* 查询方式 */

/* 查询模式 */
#if defined (USE_SPI_POLL)

uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];

/* 中断模式 */
#elif defined (USE_SPI_INT)

uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];

/* DMA 模式使用的 SRAM4 */
#elif defined (USE_SPI_DMA)
    #if defined ( __CC_ARM )    /* IAR *******/
        __attribute__((section (".RAM_D3"))) uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
        __attribute__((section (".RAM_D3"))) uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];
    #elif defined (__ICCARM__)   /* MDK ********/
        #pragma location = ".RAM_D3"
        uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
        #pragma location = ".RAM_D3"
        uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];
    #endif
#endif

/*
*********************************************************************************************************
*    函 数 名: bsp_spiTransfer
*    功能说明: 启动数据传输
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_spiTransfer(void)
{
    if (g_spiLen > SPI_BUFFER_SIZE)
    {
```

```
        return;
    }

    /* DMA 方式传输 */
#ifdef USE_SPI_DMA
    wTransferState = TRANSFER_WAIT;

    if(HAL_SPI_TransmitReceive_DMA(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen) != HAL_OK)

    {
        Error_Handler(__FILE__, __LINE__);
    }

    while (wTransferState == TRANSFER_WAIT)
    {
        ;
    }
#endif

    /* 中断方式传输 */
#ifdef USE_SPI_INT
    wTransferState = TRANSFER_WAIT;

    if(HAL_SPI_TransmitReceive_IT(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen) != HAL_OK)

    {
        Error_Handler(__FILE__, __LINE__);
    }

    while (wTransferState == TRANSFER_WAIT)
    {
        ;
    }
#endif

    /* 查询方式传输 */
#ifdef USE_SPI_POLL
    if(HAL_SPI_TransmitReceive(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen, 1000000) !=
HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
#endif
}
```

通过开头宏定义可以方便的切换中断，查询和 DMA 方式。其中查询和中断方式比较好理解，而 DMA 方式要特别注意两点：

- 通过本手册第 26 章的内存块超方便使用方式，将 DMA 缓冲定义到 SRAM4 上。因为本工程是用的 DTCM 做的主 RAM 空间，这个空间无法使用通用 DMA1 和 DMA2。
- 由于程序里面开启了数据 Cache，会造成 DMA 和 CPU 访问 SRAM4 数据不一致的问题，特此将 SRAM4 空间关闭 Cache。

```
    /* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
```

```
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

74.6.3第 3 步：DAC856X 的时钟极性和时钟相位配置

首先回忆下 STM32H7 支持的 4 种时序配置。

- 当 CPOL = 1， CPHA = 1 时

  SCK 引脚在空闲状态处于高电平，SCK 引脚的第 2 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 0， CPHA = 1 时

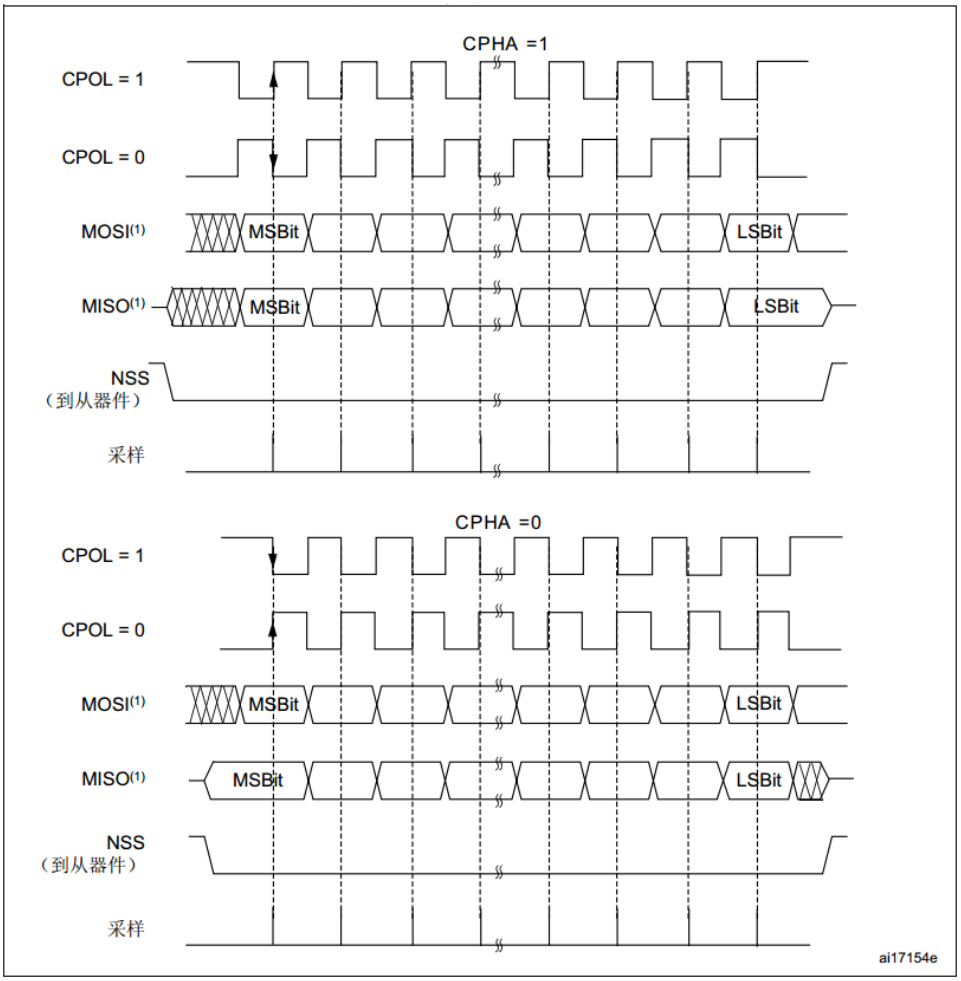
  SCK 引脚在空闲状态处于低电平，SCK 引脚的第 2 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 1， CPHA = 0 时

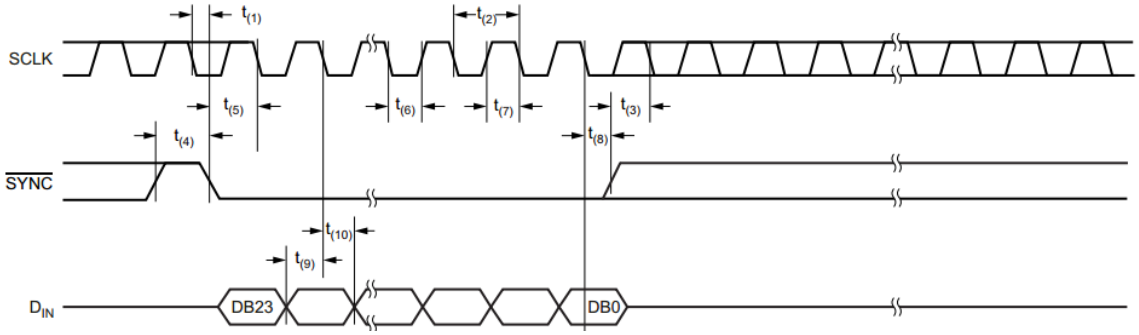
  SCK 引脚在空闲状态处于高电平，SCK 引脚的第 1 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 0 ，CPHA= 0 时

  SCK 引脚在空闲状态处于低电平，SCK 引脚的第 1 个边沿捕获传输的第 1 个数据。



有了 H7 支持的时序配置，再来看下 DAC856X 的时序图：



首先 DAC856X 是下降升沿做数据采集，所以 STM32H7 的可选的配置就是：

CHOL = 0， CPHA = 1

CHOL = 1， CPHA = 0

对于这两种情况的主要区别是空闲状态下 SCLK 时钟选择高电平还是低电平，根据上面的时序图和 DAC856X 的数据手册，两种情况下都可以正常运行。经过实际测试，STM32H7 使用这两个配置确实都可以正常运行。程序里面默认是选择 CHOL = 0， CPHA = 1。

74.6.4第 4 步：单 SPI 接口管理多个 SPI 设备的切换机制

单 SPI 接口管理多个 SPI 设备最麻烦的地方是不同设备的时钟分配，时钟极性和时钟相位并不相同。对此的解决解决办法是在片选阶段配置切换，比如 DAC856X 的片选：

```
/*
*********************************************************************************************************
*    函 数 名: DAC8562_SetCS
*    功能说明: DAC8562 片选控制函数
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DAC8562_SetCS(uint8_t _Level)
{
    if (_Level == 0)
    {
        bsp_SpiBusEnter(); /* 占用 SPI 总线  */
        bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_4, SPI_PHASE_2EDGE, SPI_POLARITY_LOW);
        CS_0();
    }
    else
    {
        CS_1();
        bsp_SpiBusExit();  /* 释放 SPI 总线 */
    }
}
```

通过这种方式就有效的解决了单 SPI 接口管理多设备的问题。因为给每个设备都配了一个独立的片选引脚，这样就可以为每个设备都配置这么一个片选配置。

但是频繁配置也比较繁琐，所以函数 bsp_InitSPIParam 里面做了特别处理。当前配置与之前配置相同的情况下无需重复配置。

74.6.5第 5 步：DAC856X 的数据更新

DAC856X 的双通道数据更新通过下面的函数实现：

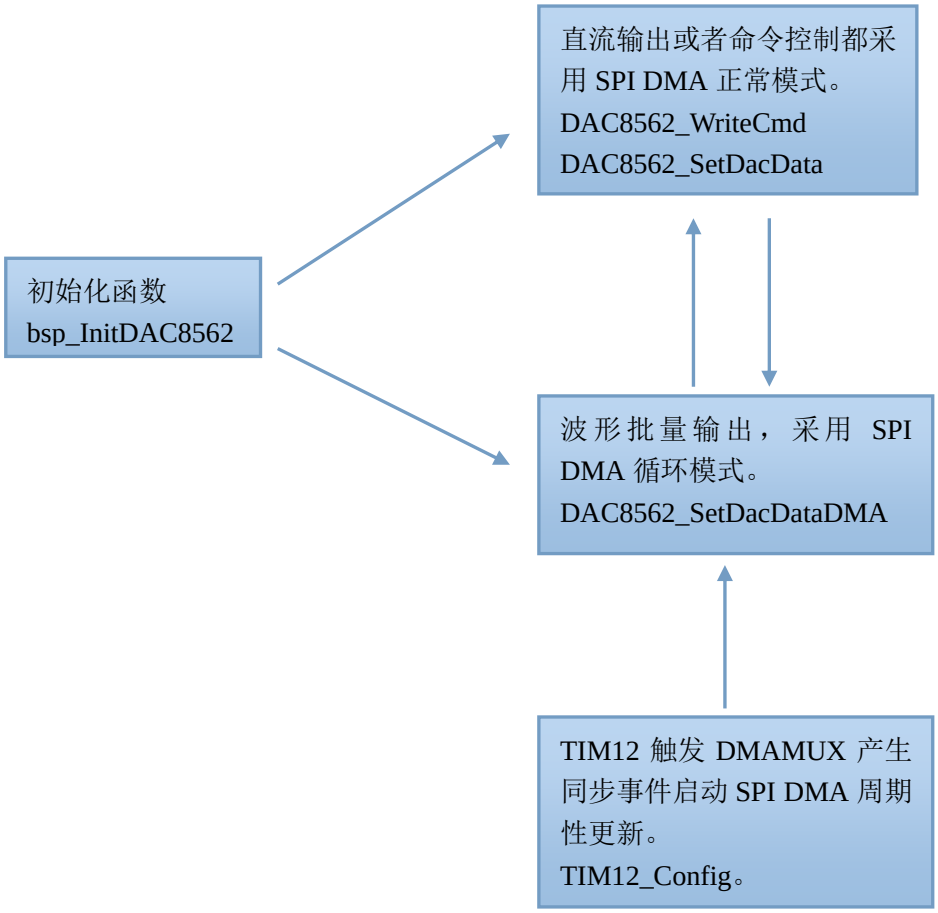
```
/*
*********************************************************************************************************
*    函 数 名: DAC8562_SetDacData
*    功能说明: 设置 DAC 输出，并立即更新。
*    形    参: _ch, 通道, 0 , 1
*              _data : 数据
*    返 回 值: 无
*********************************************************************************************************
*/
void DAC8562_SetDacData(uint8_t _ch, uint16_t _dac)
{
    if (_ch == 0)
    {
        /* Write to DAC-A input register and update DAC-A; */
        DAC8562_WriteCmd((3 << 19) | (0 << 16) | (_dac << 0));
```

```
    }
    else if (_ch == 1)
    {
        /* Write to DAC-B input register and update DAC-A; */
        DAC8562_WriteCmd((3 << 19) | (1 << 16) | (_dac << 0));
    }
}
```

函数实现比较简单，每次更新发送 24bit 数据即可。

74.7 DAC856X 驱动设计（SPI DMA 更新方式）

DAC856X 的 DMA 驱动方式略复杂，跟中断更新方式完全不同，要使用硬件 SPI1 NSS 引脚驱动 DAC8562 的片选，所有专门做了一个驱动文件来实现，程序驱动框架设计如下：



有了这个框图，程序设计就比较好理解了。

74.7.1 第 1 步：SPI 总线配置

spi 总线配置通过如下两个函数实现：

```
/*
```

```
*********************************************************************************************************
*    函 数 名: bsp_InitDAC8562
*    功能说明: 配置 GPIO 并初始化 DAC8562 寄存器
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitDAC8562(void)
{
    /* 配置 GPIO */
    GPIO_InitTypeDef GPIO_InitStruct;

    s_SpiDmaMode = 0;

    /*##-1- 配置 SPI DMA ######################################################################*/
    bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_4, SPI_PHASE_2EDGE, SPI_POLARITY_LOW);

    /*##-2- 配置 CLR 引脚 #####################################################################*/
    CLR_CLK_ENABLE();

    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;         /* 设置推挽输出 */
    GPIO_InitStruct.Pull = GPIO_NOPULL;                 /* 上下拉电阻不使能 */
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_MEDIUM; /* GPIO 速度等级 */

    GPIO_InitStruct.Pin = CLR_PIN;
    HAL_GPIO_Init(CLR_GPIO, &GPIO_InitStruct);

    CLR_0();      /* CLR 接 GND 可靠一些，CLR 是下降沿触发 */
    LDAC_0();     /* 不用异步更新模式，此引脚接 GND */

    /*##-3- 配置 DAC8562 ######################################################################*/
    /* Power up DAC-A and DAC-B */
    DAC8562_WriteCmd((4 << 19) | (0 << 16) | (3 << 0));

    /* LDAC pin inactive for DAC-B and DAC-A  不使用 LDAC 引脚更新数据 */
    DAC8562_WriteCmd((6 << 19) | (0 << 16) | (3 << 0));

    /* 复位 2 个 DAC 到中间值，输出 0V */
    DAC8562_SetDacData(0, 32767);
    DAC8562_SetDacData(1, 32767);

    /* 选择内部参考并复位 2 个 DAC 的增益=2 （复位时，内部参考是禁止的） */
    DAC8562_WriteCmd((7 << 19) | (0 << 16) | (1 << 0));
}

/*
*********************************************************************************************************
*    函 数 名: bsp_InitSPIParam
*    功能说明: 配置 SPI 总线参数，时钟分频，时钟相位和时钟极性。
*    形    参: _BaudRatePrescaler  SPI 总线时钟分频设置，支持的参数如下：
*                                 SPI_BAUDRATEPRESCALER_2    2 分频
*                                 SPI_BAUDRATEPRESCALER_4    4 分频
*                                 SPI_BAUDRATEPRESCALER_8    8 分频
*                                 SPI_BAUDRATEPRESCALER_16   16 分频
*                                 SPI_BAUDRATEPRESCALER_32   32 分频
*                                 SPI_BAUDRATEPRESCALER_64   64 分频
*                                 SPI_BAUDRATEPRESCALER_128  128 分频
*                                 SPI_BAUDRATEPRESCALER_256  256 分频
```

```
*
*             _CLKPhase           时钟相位，支持的参数如下：
*                                 SPI_PHASE_1EDGE     SCK 引脚的第 1 个边沿捕获传输的第 1 个数据
*                                 SPI_PHASE_2EDGE     SCK 引脚的第 2 个边沿捕获传输的第 1 个数据
*
*             _CLKPolarity        时钟极性，支持的参数如下：
*                                 SPI_POLARITY_LOW    SCK 引脚在空闲状态处于低电平
*                                 SPI_POLARITY_HIGH   SCK 引脚在空闲状态处于高电平
*
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitSPIParam(uint32_t _BaudRatePrescaler, uint32_t _CLKPhase, uint32_t _CLKPolarity)
{

    /* 设置 SPI 参数 */
    hspi.Instance               = SPIx;                         /* 例化 SPI */
    hspi.Init.BaudRatePrescaler = _BaudRatePrescaler;           /* 设置波特率 */
    hspi.Init.Direction         = SPI_DIRECTION_2LINES_TXONLY;  /* 全双工 */
    hspi.Init.CLKPhase          = _CLKPhase;                    /* 配置时钟相位 */
    hspi.Init.CLKPolarity       = _CLKPolarity;                 /* 配置时钟极性 */
    hspi.Init.DataSize          = SPI_DATASIZE_24BIT;           /* 设置数据宽度 */
    hspi.Init.FirstBit          = SPI_FIRSTBIT_MSB;             /* 数据传输先传高位 */
    hspi.Init.TIMode            = SPI_TIMODE_DISABLE;           /* 禁止 TI 模式  */
    hspi.Init.CRCCalculation    = SPI_CRCCALCULATION_DISABLE;   /* 禁止 CRC */
    hspi.Init.CRCPolynomial     = 7;                            /* 禁止 CRC 后，此位无效 */
    hspi.Init.CRCLength         = SPI_CRC_LENGTH_8BIT;          /* 禁止 CRC 后，此位无效 */
    hspi.Init.FifoThreshold     = SPI_FIFO_THRESHOLD_05DATA;    /* 设置 FIFO 大小是一个数据项 */

    hspi.Init.NSS         = SPI_NSS_HARD_OUTPUT;                /* 使用软件方式管理片选引脚 */
    hspi.Init.NSSPMode    = SPI_NSS_PULSE_ENABLE;               /* 使能脉冲输出 */
    hspi.Init.NSSPolarity = SPI_NSS_POLARITY_LOW;               /* 低电平有效 */

   /* MSS, 插入到 NSS 有效边沿和第一个数据开始之间的额外延迟，单位 SPI 时钟周期个数 */
    hspi.Init.MasterSSIdleness        = SPI_MASTER_SS_IDLENESS_00CYCLE;

    /* MIDI, 两个连续数据帧之间插入的最小时间延迟，单位 SPI 时钟周期个数 */
    hspi.Init.MasterInterDataIdleness = SPI_MASTER_INTERDATA_IDLENESS_10CYCLE;

    hspi.Init.MasterKeepIOState = SPI_MASTER_KEEP_IO_STATE_ENABLE; /* 禁止 SPI 后，SPI 相关引脚保持当前状态 */
    hspi.Init.Mode              = SPI_MODE_MASTER;                 /* SPI 工作在主控模式 */

    /* 复位配置 */
    if (HAL_SPI_DeInit(&hspi) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 初始化配置 */
    if (HAL_SPI_Init(&hspi) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

这两个配置函数里面最重要的是置红的几个配置选项，这里依次为大家做个说明：

- SPI_DIRECTION_2LINES_TXONLY

驱动 DAC856X 仅需要 SPI 写操作。

- SPI_DATASIZE_24BIT

  STM32H7 的 SPI 支持 4-32bit 数据传输，由于 DAC856X 需要 24bit 数据，所以这里配置为 24 即可。

- SPI_FIFO_THRESHOLD_05DATA

  对于 SPI1 来说，里面的 FIFO 大小是 16 字节，那么 SPI 数据传输配置为 24bit 的话，FIFO 最多可以存储 5 个 24bit，因此这个 fifo 阀值要设置为 5。

- SPI_NSS_HARD_OUTPUT

  我们这里要使用 SPI 的硬件片选引脚 SPI_NSS。

- SPI_MASTER_SS_IDLENESS_00CYCLE

  插入到 NSS 有效边沿和第一个数据开始之间的额外延迟，单位 SPI 时钟周期个数。

  根据本章 4.5.4 小节里面的 $t_{(5)}$要求，片选有效到 SCLK 第 1 个下降沿信号的时间，最小值 13ns。由于 DAC856X 的最高时钟是 50MHz，即 20ns 的分辨率，并且实际程序中，我们选择的是第 2 个边沿做数据采集，所以这里配置为 0 即可，也就是无需插入时间。

- SPI_MASTER_INTERDATA_IDLENESS_10CYCLE

  两个连续数据帧之间插入的最小时间延迟，单位 SPI 时钟周期个数。

  根据本章 4.5.4 小节里面的 $t_{(4)}$要求，每传输 24bit 数据后，片选要保持一段时间的高电平，DAC856X 要求至少要 80ns，也是说，如果我们以 50MHz 驱动 DAC856X，这里至少要配置为 4 个时钟周期，推荐值为 5 及其以上即可，我们这里直接配置为 10 个时钟周期（配置为 5 也没问题的）。

74.7.2 第 2 步：TIM12 周期性触发配置

这里特别注意一点，定时器触发一次，就会让 SPI 以 DMA 方式传输 24bit 输出。

TIM12 的触发配置如下：

```
/*
*********************************************************************************************************
*    函 数 名: TIM12_Config
*    功能说明: 配置 TIM12，用于触发 DMAMUX 的请求发生器
*    形    参: _ulFreq  触发频率，推荐范围 100Hz - 1MHz
*    返 回 值: 无
*********************************************************************************************************
*/
TIM_HandleTypeDef  htim ={0};
TIM_MasterConfigTypeDef sMasterConfig = {0};
TIM_OC_InitTypeDef sConfig = {0};
void TIM12_Config(uint32_t _ulFreq)
{
    uint16_t usPeriod;
    uint16_t usPrescaler;
    uint32_t uiTIMxCLK;
```

```
    /* 使能时钟 */
    __HAL_RCC_TIM12_CLK_ENABLE();

    /*-----------------------------------------------------------------------
        bsp.c 文件中 void SystemClock_Config(void) 函数对时钟的配置如下:

       System Clock source       = PLL (HSE)
       SYSCLK(Hz)                = 400000000 (CPU Clock)
       HCLK(Hz)                  = 200000000 (AXI and AHBs Clock)
       AHB Prescaler             = 2
       D1 APB3 Prescaler         = 2 (APB3 Clock  100MHz)
       D2 APB1 Prescaler         = 2 (APB1 Clock  100MHz)
       D2 APB2 Prescaler         = 2 (APB2 Clock  100MHz)
       D3 APB4 Prescaler         = 2 (APB4 Clock  100MHz)

       因为 APB1 prescaler != 1, 所以 APB1 上的 TIMxCLK = APB1 x 2 = 200MHz; 不含这个总线下的 LPTIM1
       因为 APB2 prescaler != 1, 所以 APB2 上的 TIMxCLK = APB2 x 2 = 200MHz;
       APB4 上面的 TIMxCLK 没有分频，所以就是 100MHz;

       APB1 定时器有 TIM2, TIM3 ,TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14，LPTIM1
       APB2 定时器有 TIM1, TIM8 , TIM15, TIM16，TIM17

       APB4 定时器有 LPTIM2，LPTIM3，LPTIM4，LPTIM5
    ----------------------------------------------------------------------- */
    uiTIMxCLK = SystemCoreClock / 2;

    if (_ulFreq < 100)
    {
        usPrescaler = 10000 - 1;                        /* 分频比 = 10000 */
        usPeriod =  (uiTIMxCLK / 10000) / _ulFreq  - 1; /* 自动重装的值 */
    }
    else if (_ulFreq < 3000)
    {
        usPrescaler = 100 - 1;                    /* 分频比 = 100 */
        usPeriod =  (uiTIMxCLK / 100) / _ulFreq  - 1;/* 自动重装的值 */
    }
    else /* 大于 4K 的频率，无需分频 */
    {
        usPrescaler = 0;                        /* 分频比 = 1 */
        usPeriod = uiTIMxCLK / _ulFreq - 1;   /* 自动重装的值 */
    }

   htim.Instance = TIM12;
    htim.Init.Period            = usPeriod;
    htim.Init.Prescaler         = usPrescaler;
    htim.Init.ClockDivision     = 0;
    htim.Init.CounterMode       = TIM_COUNTERMODE_UP;
    htim.Init.RepetitionCounter = 0;

    if(HAL_TIM_Base_DeInit(&htim) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    if(HAL_TIM_Base_Init(&htim) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
```

```
    sConfig.OCMode     = TIM_OCMODE_PWM1;
    sConfig.OCPolarity = TIM_OCPOLARITY_LOW;
    sConfig.Pulse = usPeriod / 2;     /* 占空比 50% */
    if(HAL_TIM_OC_ConfigChannel(&htim, &sConfig, TIM_CHANNEL_1) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 启动 OC1 */
    if(HAL_TIM_OC_Start(&htim, TIM_CHANNEL_1) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* TIM12 的 TRGO 用于触发 DMAMUX 的请求发生器 */
     sMasterConfig.MasterOutputTrigger = TIM_TRGO_OC1REF;
    sMasterConfig.MasterOutputTrigger2 = TIM_TRGO2_RESET;
     sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;

     HAL_TIMEx_MasterConfigSynchronization(&htim, &sMasterConfig);
}
#endif
```

这个函数支持的触发频率很宽，对于 DAC856X 来说，如果样本点设置为 100 个的话，此函数推荐的触发频率是 100Hz 到 1MHz，具体可以支持到最高触发速度计算看本章 4.7.7 小节即可。

74.7.3第 3 步：DMAMUX 同步触发 SPI DMA 传输

DMA 和 DMAMUX 的配置如下：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_spiDamStart
*    功能说明: 启动 SPI DMA 传输
*    形    参: _ulFreq 范围推荐 100Hz-1MHz
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_spiDamStart(uint32_t _ulFreq)
{
    /* 设置模式，要切换到 DMA CIRCULAR 模式 */
    s_SpiDmaMode = 1;

    bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_4, SPI_PHASE_2EDGE, SPI_POLARITY_LOW);

    /* 使能 DMA 时钟 */
    DMAx_CLK_ENABLE();

    /* SPI DMA 发送配置 */
    hdma_tx.Instance                 = SPIx_TX_DMA_STREAM;      /* 例化使用的 DMA 数据流 */
    hdma_tx.Init.FIFOMode            = DMA_FIFOMODE_ENABLE;     /* 使能 FIFO */
    hdma_tx.Init.FIFOThreshold       = DMA_FIFO_THRESHOLD_FULL; /* 用于设置阀值 */
    hdma_tx.Init.MemBurst            = DMA_MBURST_SINGLE;       /* 用于存储器突发 */
    hdma_tx.Init.PeriphBurst         = DMA_PBURST_SINGLE;       /* 用于外设突发 */
    hdma_tx.Init.Request             = SPIx_TX_DMA_REQUEST;     /* 请求类型 */
```

```
    hdma_tx.Init.Direction           = DMA_MEMORY_TO_PERIPH;     /* 传输方向是从存储器到外设 */
    hdma_tx.Init.PeriphInc           = DMA_PINC_DISABLE;         /* 外设地址自增禁止 */
    hdma_tx.Init.MemInc              = DMA_MINC_ENABLE;          /* 存储器地址自增使能 */
    hdma_tx.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;      /* 外设数据传输位宽选择字节，即 8bit */
    hdma_tx.Init.MemDataAlignment    = DMA_MDATAALIGN_WORD;      /* 存储器数据传输位宽选择字节，即 8bit */
    hdma_tx.Init.Mode                = DMA_CIRCULAR;             /* 正常模式 */
    hdma_tx.Init.Priority            = DMA_PRIORITY_LOW;         /* 优先级低 */

     /* 复位 DMA */
    if(HAL_DMA_DeInit(&hdma_tx) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

     /* 初始化 DMA */
    if(HAL_DMA_Init(&hdma_tx) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 关联 DMA 句柄到 SPI */
    __HAL_LINKDMA(&hspi, hdmatx, hdma_tx);


    /* 关闭 DMA 发送中断 */
    HAL_NVIC_SetPriority(SPIx_DMA_TX_IRQn, 1, 0);
    HAL_NVIC_DisableIRQ(SPIx_DMA_TX_IRQn);

    /* 关闭 SPI 中断 */
    HAL_NVIC_SetPriority(SPIx_IRQn, 1, 0);
    HAL_NVIC_DisableIRQ(SPIx_IRQn);

    /* 同步触发配置 */
    dmamux_syncParams.EventEnable   = ENABLE;
    dmamux_syncParams.SyncPolarity  = HAL_DMAMUX_SYNC_RISING;
    dmamux_syncParams.RequestNumber = 1;
    dmamux_syncParams.SyncSignalID  = HAL_DMAMUX1_SYNC_TIM12_TRGO;
    dmamux_syncParams.SyncEnable    = ENABLE;

    HAL_DMAEx_ConfigMuxSync(&hdma_tx, &dmamux_syncParams);

    //LPTIM_Config(_ulFreq);

    TIM12_Config(_ulFreq);

    /* 启动 DMA 传输 */
    if(HAL_SPI_Transmit_DMA(&hspi, (uint8_t*)g_spiTxBuf, g_spiLen/4)!= HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

这段程序里面最关键的就是置红的部分。作用是配置 DMAMUX 的同步触发功能，触发周期由 TIM12 控制。

74.7.4第 4 步：24bit 数据的 DMA 传输解决办法

由于通用 DMA1 和 DMA2 仅支持 8bit，16bit 和 32bit 数据传输，我们这里要传输 24bit 数据，解决的关键就是配置 DMA 为传输宽度为 32bit,并将传输的数据由 24bit 再补一个 8bit 的任意值组成 32bit 即可，实际的传输会由 SPI 完成。

```
/*
*********************************************************************************************************
*    函 数 名: DAC8562_SetDacDataDMA
*    功能说明: DAC8562 数据发送，DMA 方式
*    形    参: _ch          1 表示通道 1 输出
*                           2 表示通道 2 输出
*                           3 表示通道 1 和 2 都输出
*                           4 表示通道 1 和 2 都输出，并且附加一个控制命令，有效防止传输错误时恢复。
*              _pbufch1     通道 1 数据缓冲地址
*              _pbufch2     通道 2 数据缓冲地址
*              _sizech1     通道 1 数据大小
*              _sizech2     通道 2 数据大小
*              _ulFreq      触发频率，范围 2KB- 1MHz，注意这个参数是触发频率，并不是波形周期，触发一次，SPI DMA
*                           传输一次 24bit 数据。
*    返 回 值: 无
*********************************************************************************************************
*/
void DAC8562_SetDacDataDMA(uint8_t _ch, uint16_t *_pbufch1, uint16_t *_pbufch2, uint32_t _sizech1, uint32_t
_sizech2, uint32_t _ulFreq)
{
    uint32_t i;
    uint32_t _cmd;

    g_spiLen = 0;

    switch (_ch)
    {
        /* 通道 1 数据发送 */
        case 1:
            for(i = 0; i < _sizech1; i++)
            {
                _cmd = (3 << 19) | (0 << 16) | (_pbufch1[i] << 0);

                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
                g_spiTxBuf[g_spiLen++] = 0xff;
            }
            break;

        /* 通道 2 数据发送 */
        case 2:
            for(i = 0; i < _sizech2; i++)
            {
                _cmd = (3 << 19) | (1 << 16) | (_pbufch2[i] << 0);

                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
```

```
            g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
            g_spiTxBuf[g_spiLen++] = 0xff;
        }
        break;

    /* 通道 1 和 2 混合发送 */
    case 3:
        if(_sizech1 == _sizech2)
        {

            for(i = 0; i < _sizech1; i++)
            {
                _cmd = (3 << 19) | (0 << 16) | (_pbufch1[i] << 0);

                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
                g_spiTxBuf[g_spiLen++] = 0xff;

                _cmd = (3 << 19) | (1 << 16) | (_pbufch2[i] << 0);

                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
                g_spiTxBuf[g_spiLen++] = 0xff;
            }
        }
        else
        {
            for(i = 0; i < _sizech1; i++)
            {
                _cmd = (3 << 19) | (0 << 16) | (_pbufch1[i] << 0);

                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
                g_spiTxBuf[g_spiLen++] = 0xff;
            }
            for(i = 0; i < _sizech2; i++)
            {
                _cmd = (3 << 19) | (1 << 16) | (_pbufch2[i] << 0);

                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
                g_spiTxBuf[g_spiLen++] = 0xff;
            }
        }
        break;

    /* 插入关键命令，防止传输错误 */
    case 4:
        if(_sizech1 == _sizech2)
        {

            for(i = 0; i < _sizech1; i++)
            {
                _cmd = (3 << 19) | (0 << 16) | (_pbufch1[i] << 0);
```

```
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
                g_spiTxBuf[g_spiLen++] = 0xff;

                _cmd = (3 << 19) | (1 << 16) | (_pbufch2[i] << 0);

                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
                g_spiTxBuf[g_spiLen++] = 0xff;
            }
        }
        else
        {
            for(i = 0; i < _sizech1; i++)
            {
                _cmd = (3 << 19) | (0 << 16) | (_pbufch1[i] << 0);

                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
                g_spiTxBuf[g_spiLen++] = 0xff;
            }
            for(i = 0; i < _sizech2; i++)
            {
                _cmd = (3 << 19) | (1 << 16) | (_pbufch2[i] << 0);

                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
                g_spiTxBuf[g_spiLen++] = 0xff;
            }
        }

        /* 数据填充完毕后，插入关键命令，数据输出过程中被 8256 误识别为命令处理*/
        _cmd = (7 << 19) | (0 << 16) | (1 << 0);

        g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
        g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
        g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
        g_spiTxBuf[g_spiLen++] = 0xff;
        break;

    default:
        break;

    }

    bsp_spiDamStart(_ulFreq);
}
```

74.7.5 第 5 步：DMA 缓冲区的 MPU 配置

因为工程是用的 DTCM 做的主 RAM 空间，这个空间无法使用通用 DMA1 和 DMA2，通过本手册第 26 章的内存块超方便使用方式，将 DMA 缓冲定义到 SRAM4 上：

```
#if defined ( __CC_ARM )    /* IAR *******/
    __attribute__((section (".RAM_D3"))) uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
    __attribute__((section (".RAM_D3"))) uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];
#elif defined (__ICCARM__)  /* MDK ********/
    #pragma location = ".RAM_D3"
    uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
    #pragma location = ".RAM_D3"
    uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];
#endif
```

由于程序里面开启了数据 Cache，会造成 DMA 和 CPU 访问 SRAM4 数据不一致的问题，特此将 SRAM4 空间关闭 Cache。

```
    /* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

74.7.6 第 6 步：DAC856X 的时钟极性和时钟相位配置

注：与本章 4.6.3 小节内容是一样的。

首先回忆下 STM32H7 支持的 4 种时序配置。

- 当 CPOL = 1， CPHA = 1 时

  SCK 引脚在空闲状态处于高电平，SCK 引脚的第 2 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 0， CPHA = 1 时

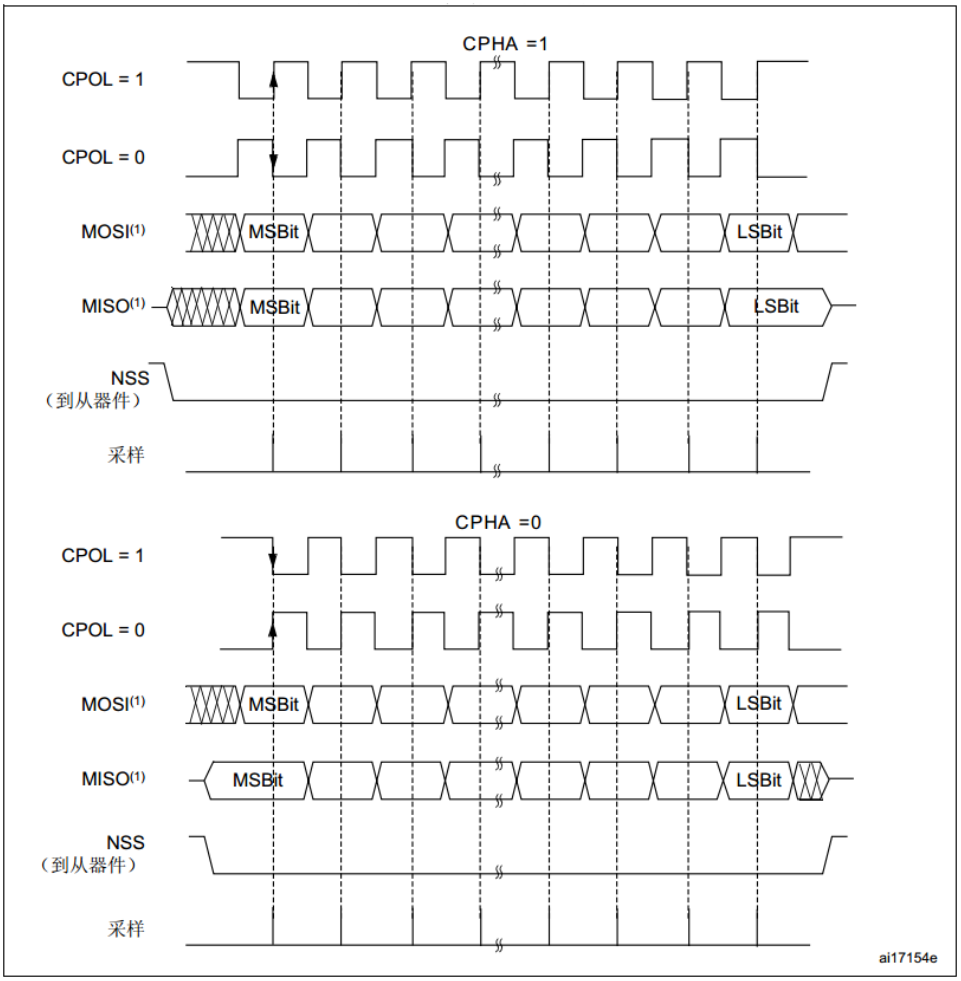
  SCK 引脚在空闲状态处于低电平，SCK 引脚的第 2 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 1， CPHA = 0 时

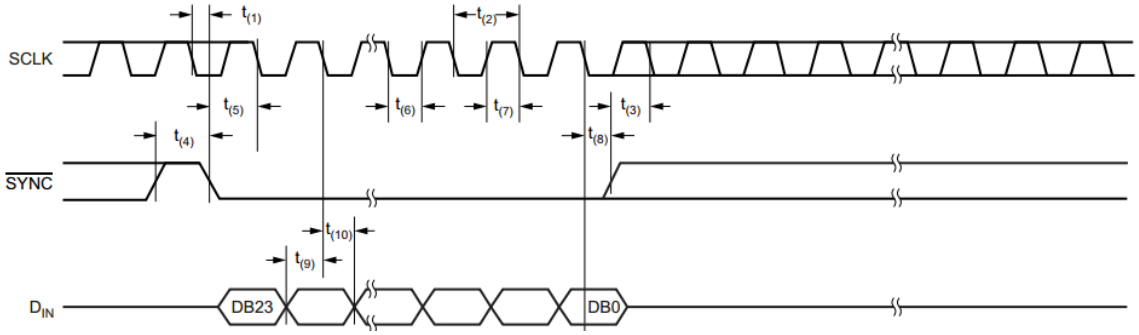
  SCK 引脚在空闲状态处于高电平，SCK 引脚的第 1 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 0 ，CPHA= 0 时

  SCK 引脚在空闲状态处于低电平，SCK 引脚的第 1 个边沿捕获传输的第 1 个数据。



有了 H7 支持的时序配置，再来看下 DAC856X 的时序图：



首先 DAC856X 是下降升沿做数据采集，所以 STM32H7 的可选的配置就是：

CHOL = 0， CPHA = 1

CHOL = 1， CPHA = 0

对于这两种情况的主要区别是空闲状态下 SCLK 时钟选择高电平还是低电平，根据上面的时序图和 DAC856X 的数据手册，两种情况下都可以正常运行。经过实际测试，STM32H7 使用这两个配置确实都可以正常运行。程序里面默认是选择 CHOL = 0， CPHA = 1。

74.7.7第 7 步：DAC856X 的最高更新速度计算

这里特别注意一点，定时器触发一次，就会让 SPI 以 DMA 方式传输 24bit 数据。

配置条件：

- SPI 时钟是 50MHz。
- SPI 数据传为 24bit，每个 bit 需要时间 20ns。
- hspi.Init.MasterSSIdleness = SPI_MASTER_SS_IDLENESS_00CYCLE
  插入到 NSS 有效边沿和第一个数据开始之间的额外延迟，单位 SPI 时钟周期个数，即 20ns。
- hspi.Init.MasterInterDataIdleness = SPI_MASTER_INTERDATA_IDLENESS_10CYCLE
  两个连续数据帧之间插入的最小时间延迟，单位 SPI 时钟周期个数，即 20ns。

根据上面的配置，传输一帧（24bit）数据需要的时间：

24bit * 20ns+ SPI_MASTER_SS_IDLENESS_00CYCLE * 20ns
  + SPI_MASTER_INTERDATA_IDLENESS_10CYCLE * 20ns
= 24bit * 20ns + 0 * 20ns + 10 * 20ns
= 680ns。

那么这种配置下，可以支持最高触发速度是 1 / 680ns = 1.47MHz，如果想速度再提升些，可以降低参数 hspi.Init.MasterInterDataIdleness，推荐的最小值是 5 个时钟周期，那么可以支持的最高触发速度是 1/580ns = 1.7MHz。

认识到这些后，实际输出的波形周期也比较好算了，比如我们设置 10 个样本点为一个周期，那么触发速度为 1MHz 的时候，那么波形周期就是 100KHz。

74.7.8第 8 步：DAC 值和电压值互转

DAC856X 模块的输出电压范围是-10V 到 10V，对应的编码值范围是 0 到 65535，为了方便大家做互转，专门做了两个函数：

```
/*
*********************************************************************************************************
*    函 数 名: DAC8562_DacToVoltage
*    功能说明: 将 DAC 值换算为电压值，单位 0.1mV
*    形    参: _dac  16 位 DAC 字
*    返 回 值: 电压。单位 0.1mV
*********************************************************************************************************
*/
int32_t DAC8562_DacToVoltage(uint16_t _dac)
{
    int32_t y;
```

```
    /* CaculTwoPoint(int32_t x1, int32_t y1, int32_t x2, int32_t y2, int32_t x);*/
    y =  CaculTwoPoint(X1, Y1, X2, Y2, _dac);
    return y;
}

/*
*********************************************************************************************************
*    函 数 名: DAC8562_VoltageToDac
*    功能说明: 将电压值转换为 DAC 置
*    形    参: _volt 电压，单位 0.1mV
*    返 回 值: 16 位 DAC 字
*********************************************************************************************************
*/
uint32_t DAC8562_VoltageToDac(int32_t _volt)
{
    /* CaculTwoPoint(int32_t x1, int32_t y1, int32_t x2, int32_t y2, int32_t x);*/
    return CaculTwoPoint(Y1, X1, Y2, X2, _volt);
}
```

74.7.9第 9 步：防止 SPI DMA 批量数据传输错误解决办法

使用 SPI DMA 批量数据传输过程中，要防止一些数据被 DAC856X 错误识别成关键命令，从而造成 DAC856X 工作异常，其中最重要的一个关键命令就下面这个：

```
/* 选择内部参考并复位 2 个 DAC 的增益=2 （复位时，内部参考是禁止的） */
DAC8562_WriteCmd((7 << 19) | (0 << 16) | (1 << 0));
```

针对这个问题，函数 DAC8562_SetDacDataDMA 专门做了一个传输方式 4：

```
/* 插入关键命令，防止传输错误 */
case 4:
    if(_sizech1 == _sizech2)
    {

        for(i = 0; i < _sizech1; i++)
        {
            _cmd = (3 << 19) | (0 << 16) | (_pbufch1[i] << 0);

            g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
            g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
            g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
            g_spiTxBuf[g_spiLen++] = 0xff;

            _cmd = (3 << 19) | (1 << 16) | (_pbufch2[i] << 0);

            g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
            g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
            g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
            g_spiTxBuf[g_spiLen++] = 0xff;
        }
    }
    else
    {
        for(i = 0; i < _sizech1; i++)
        {
            _cmd = (3 << 19) | (0 << 16) | (_pbufch1[i] << 0);
```

```
            g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
            g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
            g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
            g_spiTxBuf[g_spiLen++] = 0xff;
        }
        for(i = 0; i < _sizech2; i++)
        {
            _cmd = (3 << 19) | (1 << 16) | (_pbufch2[i] << 0);

            g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
            g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
            g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
            g_spiTxBuf[g_spiLen++] = 0xff;
        }
    }

    /* 数据填充完毕后，插入关键命令，数据输出过程中被 8256 误识别为命令处理*/
    _cmd = (7 << 19) | (0 << 16) | (1 << 0);

    g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
    g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
    g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
    g_spiTxBuf[g_spiLen++] = 0xff;
    break;
```

解决办法是在批量数据的末尾附一个命令，通过这种方式可以有效防止 DAC856X 工作异常。

74.8 SPI 总线板级支持包（bsp_spi_bus.c）

SPI 总线驱动文件 bsp_spi_bus.c 主要实现了如下几个 API 供用户调用：

- bsp_InitSPIBus
- bsp_InitSPIParam
- bsp_spiTransfer

74.8.1 函数 bsp_InitSPIBus

函数原型：

```
void bsp_InitSPIBus(void)
```

函数描述：

此函数主要用于 SPI 总线的初始化，在 bsp.c 文件调用一次即可。

74.8.2 函数 bsp_InitSPIParam

函数原型：

```
void bsp_InitSPIParam(uint32_t _BaudRatePrescaler, uint32_t _CLKPhase, uint32_t _CLKPolarity)
```

函数描述：

此函数用于 SPI 总线的配置。

函数参数:

- 第 1 个参数 SPI 总线的分频设置，支持的参数如下:

  SPI_BAUDRATEPRESCALER_2　　2 分频
  SPI_BAUDRATEPRESCALER_4　　4 分频
  SPI_BAUDRATEPRESCALER_8　　8 分频
  SPI_BAUDRATEPRESCALER_16　　16 分频
  SPI_BAUDRATEPRESCALER_32　　32 分频
  SPI_BAUDRATEPRESCALER_64　　64 分频
  SPI_BAUDRATEPRESCALER_128　　128 分频
  SPI_BAUDRATEPRESCALER_256　　256 分频

- 第 2 个参数用于时钟相位配置，支持的参数如下:

  SPI_PHASE_1EDGE　　SCK 引脚的第 1 个边沿捕获传输的第 1 个数据
  SPI_PHASE_2EDGE　　SCK 引脚的第 2 个边沿捕获传输的第 1 个数据

- 第 3 个参数是时钟极性配置，支持的参数如下:

  SPI_POLARITY_LOW　　SCK 引脚在空闲状态处于低电平
  SPI_POLARITY_HIGH　　SCK 引脚在空闲状态处于高电平

74.8.3 函数 bsp_spiTransfer

函数原型:

```
void bsp_spiTransfer(void)
```

函数描述:

此函数用于启动 SPI 数据传输，支持查询，中断和 DMA 方式传输。

74.9 DAC856X 支持包中断方式（bsp_spi_dac8562.c）

DAC856X 驱动文件 bsp_spi_dac8562.c 主要实现了如下几个 API 供用户调用:

- bsp_InitDAC8562
- DAC8562_SetCS
- DAC8562_WriteCmd
- DAC8562_SetDacData
- DAC8562_DacToVoltage
- DAC8562_VoltageToDac

74.9.1 函数 bsp_InitDAC8562

函数原型：

```
void bsp_InitDAC8562(void)
```

函数描述：

主要用于 DAC856X 的初始化，调用前务必先调用函数 bsp_InitSPIBus 初始化 SPI 外设。

74.9.2 函数 DAC8562_SetCS

函数原型：

```
void DAC8562_SetCS(uint8_t _Level)
```

函数描述：

此函数用于片选 DAC8562。

函数参数：

- 第 1 个参数为 0 表示选中，为 1 表示取消选中。

74.9.3 函数 DAC8562_WriteCmd

函数原型：

```
void DAC8562_WriteCmd(uint32_t _cmd)
```

函数描述：

此函数用于向 SPI 总线发送 24 个 bit 数据。

函数参数：

- 第 1 个参数为 24bit 数据。

74.9.4 函数 DAC8562_SetDacData

函数原型：

```
void DAC8562_SetDacData(uint8_t _ch, uint16_t _dac)
```

函数描述：

此函数用于设置 DAC 输出，并立即更新。

函数参数：

- 第 1 个参数为 0 表示通道 1，为 1 表示通道 2。
- 第 2 个参数是 DAC 数值设置，范围 0 到 65535，0 对应最小电压值，65535 对应最大电压值。

74.9.5 函数 DAC8562_DacToVoltage

函数原型：

```
int32_t DAC8562_DacToVoltage(uint16_t _dac)
```

函数描述：

此函数用于将 DAC 值换算为电压值，单位 0.1mV。

函数参数：

- 第 1 个参数 DAC 数值，范围 0 到 65535。
- 返回值，返回电压值，单位 0.1mV。

74.9.6 函数 DAC8562_VoltageToDac

函数原型：

```
uint32_t DAC8562_VoltageToDac(int32_t _volt)
```

函数描述：

此函数用于将电压值转换为 DAC 值。

函数参数：

- 第 1 个参数是电压值，范围-100000 到 100000，单位 0.1mV。
- 返回值，返回 DAC 值。

74.10 DAC856X 支持包 DMA 方式（bsp_spidma_dac8562.c）

DAC856X 驱动文件 bsp_spidam_dac8562.c 涉及到的函数比较多，我们主要介绍用到的如下几个函数：

- bsp_InitDAC8562
- DAC8562_SetDacDataDMA
- DAC8562_WriteCmd
- DAC8562_SetDacData

74.10.1 函数 bsp_InitDAC8562

函数原型：

```
void bsp_InitDAC8562(void)
```

函数描述：

主要用于 DAC856X 的初始化。

74.10.2 函数 DAC8562_SetDacDataDMA

函数原型：

```
void DAC8562_SetDacDataDMA(uint8_t _ch, uint16_t *_pbufch1, uint16_t *_pbufch2, uint32_t _sizech1, uint32_t
                                                                                    _sizech2, uint32_t _ulFreq)
```

函数描述：

此函数用于 SPI DMA 方式数据发送。

函数参数：

- 第 1 个参数用于选择的通道：
  - 1 表示通道 1 输出
  - 2 表示通道 2 输出
  - 3 表示通道 1 和 2 都输出
  - 4 表示通道 1 和 2 都输出，并且附加一个控制命令，有效防止传输错误时恢复。
- 第 2 个参数表示通道 1 数据缓冲地址。
- 第 3 个参数表示通道 2 数据缓冲地址。
- 第 4 个参数表示通道 1 数据大小。
- 第 5 个参数表示通道 2 数据大小。
- 第 6 个参数表示触发频率，推荐范围 100Hz- 1MHz，注意这个参数是触发频率，并不是波形周期。这里触发一次，SPI DMA 传输一次 24bit 数据。

74.10.3 函数 DAC8562_WriteCmd

函数原型：

```
void DAC8562_WriteCmd(uint32_t _cmd)
```

函数描述：

此函数用于向 SPI 总线发送 24 个 bit 数据。

函数参数：

- 第 1 个参数为 24bit 数据。

74.10.4 函数 DAC8562_SetDacData

函数原型：

```
void DAC8562_SetDacData(uint8_t _ch, uint16_t _dac)
```

函数描述：

此函数用于设置 DAC 输出，并立即更新。

函数参数：

- 第 1 个参数为 0 表示通道 1，为 1 表示通道 2。
- 第 2 个参数是 DAC 数值设置，范围 0 到 65535，0 对应最小电压值，65535 对应最大电压值。

74.11 DAC856X 驱动移植和使用（中断更新方式）

DAC856X 移植步骤如下：

- 第 1 步：复制 bsp_spi_bus.c，bsp_spi_bus.h，bsp_spi_dac8562.c，bsp_spi_dac8562.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：根据使用的第几个 SPI，SPI 时钟，SPI 引脚和 DMA 通道等，修改 bsp_spi_bus.c 文件开头的宏定义

```
/*
*********************************************************************************************************
*                                    时钟，引脚，DMA，中断等宏定义
*********************************************************************************************************
*/
#define SPIx                             SPI1
#define SPIx_CLK_ENABLE()                __HAL_RCC_SPI1_CLK_ENABLE()
#define DMAx_CLK_ENABLE()                __HAL_RCC_DMA2_CLK_ENABLE()

#define SPIx_FORCE_RESET()               __HAL_RCC_SPI1_FORCE_RESET()
#define SPIx_RELEASE_RESET()             __HAL_RCC_SPI1_RELEASE_RESET()

#define SPIx_SCK_CLK_ENABLE()            __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_SCK_GPIO                    GPIOB
#define SPIx_SCK_PIN                     GPIO_PIN_3
#define SPIx_SCK_AF                          GPIO_AF5_SPI1

#define SPIx_MISO_CLK_ENABLE()           __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MISO_GPIO                   GPIOB
#define SPIx_MISO_PIN                    GPIO_PIN_4
#define SPIx_MISO_AF                     GPIO_AF5_SPI1

#define SPIx_MOSI_CLK_ENABLE()           __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MOSI_GPIO                   GPIOB
#define SPIx_MOSI_PIN                    GPIO_PIN_5
#define SPIx_MOSI_AF                     GPIO_AF5_SPI1

#define SPIx_TX_DMA_STREAM               DMA2_Stream3
#define SPIx_RX_DMA_STREAM               DMA2_Stream2

#define SPIx_TX_DMA_REQUEST              DMA_REQUEST_SPI1_TX
#define SPIx_RX_DMA_REQUEST              DMA_REQUEST_SPI1_RX

#define SPIx_DMA_TX_IRQn                 DMA2_Stream3_IRQn
#define SPIx_DMA_RX_IRQn                 DMA2_Stream2_IRQn

#define SPIx_DMA_TX_IRQHandler           DMA2_Stream3_IRQHandler
#define SPIx_DMA_RX_IRQHandler           DMA2_Stream2_IRQHandler

#define SPIx_IRQn                        SPI1_IRQn
#define SPIx_IRQHandler                  SPI1_IRQHandler
```

◆ 第 3 步：根据芯片支持的时钟速度，时钟相位和时钟极性配置函数 DAC8562_SetCS。

```
/*
*********************************************************************************************************
*    函 数 名: DAC8562_SetCS
*    功能说明: DAC8562 片选控制函数
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DAC8562_SetCS(uint8_t _Level)
{
    if (_Level == 0)
    {
        bsp_SpiBusEnter(); /* 占用 SPI 总线  */
        bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_4, SPI_PHASE_2EDGE, SPI_POLARITY_LOW);
        CS_0();
    }
    else
    {
        CS_1();
        bsp_SpiBusExit();  /* 释放 SPI 总线 */
    }
}
```

◆ 第 4 步：根据使用的片选，CLR 和 LDAC 引脚，修改 bsp_spi_dac8562.c 文件开头的宏定义。

```
/* SYNC, 也就是 CS 片选 */
#define CS_CLK_ENABLE()          __HAL_RCC_GPIOG_CLK_ENABLE()
#define CS_GPIO                  GPIOG
#define CS_PIN                   GPIO_PIN_10
#define CS_1()                   CS_GPIO->BSRR = CS_PIN
#define CS_0()                   CS_GPIO->BSRR = ((uint32_t)CS_PIN << 16U)

/* CLR */
#define CLR_CLK_ENABLE()         __HAL_RCC_GPIOE_CLK_ENABLE()
#define CLR_GPIO                 GPIOE
#define CLR_PIN                  GPIO_PIN_4
#define CLR_1()                  CLR_GPIO->BSRR = CLR_PIN
#define CLR_0()                  CLR_GPIO->BSRR = ((uint32_t)CLR_PIN << 16U)

/* LDAC 使用扩展 IO ，特别注意，我们这里是用的扩展 IO 控制的 */
#define LDAC_1()            HC574_SetPin(NRF24L01_CE, 1);
#define LDAC_0()            HC574_SetPin(NRF24L01_CE, 0);
```

◆ 第 5 步：如果使用 DMA 方式的话，请不要使用 TCM RAM，因为通用 DMA1 和 DMA2 不支持。并为了防止 DMA 和 CPU 同时访问 DMA 缓冲造成的数据一致性问题，将这块空间关闭 Cache 处理，比如使用的 SRAM4：

```
/* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress      = 0x38000000;
MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
```

```
MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

- 第 6 步：初始化 SPI。

```
/* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
bsp_InitSPIBus();  /* 配置 SPI 总线 */
bsp_InitDAC8562(); /* 初始化配置 DAC8562/8563 */
```

- 第 7 步：DAC856X 驱动主要用到 HAL 库的 SPI 驱动文件，简单省事些可以添加所有 HAL 库 C 源文件进来。
- 第 8 步：应用方法看本章节配套例子即可。

74.12 DAC856X 驱动移植和使用（SPI DMA 更新方式）

DAC856X 移植步骤如下：

- 第 1 步：复制 bsp_spidma_dac8562.c，bsp_spidma_dac8562.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：根据使用的第几个 SPI，SPI 时钟，SPI 引脚和 DMA 通道等，修改 bsp_spidma_dac8562.c 文件开头的宏定义

```
/*
*********************************************************************************************************
*                                    时钟，引脚，DMA，中断等宏定义
*********************************************************************************************************
*/
#define SPIx                             SPI1
#define SPIx_CLK_ENABLE()                __HAL_RCC_SPI1_CLK_ENABLE()
#define DMAx_CLK_ENABLE()                __HAL_RCC_DMA2_CLK_ENABLE()

#define SPIx_FORCE_RESET()               __HAL_RCC_SPI1_FORCE_RESET()
#define SPIx_RELEASE_RESET()             __HAL_RCC_SPI1_RELEASE_RESET()

/* SYNC, 也就是 CS 片选 */
#define SPIx_NSS_CLK_ENABLE()            __HAL_RCC_GPIOG_CLK_ENABLE()
#define SPIx_NSS_GPIO                    GPIOG
#define SPIx_NSS_PIN                     GPIO_PIN_10
#define SPIx_NSS_AF                      GPIO_AF5_SPI1

#define SPIx_SCK_CLK_ENABLE()            __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_SCK_GPIO                    GPIOB
#define SPIx_SCK_PIN                     GPIO_PIN_3
#define SPIx_SCK_AF                      GPIO_AF5_SPI1

#define SPIx_MISO_CLK_ENABLE()           __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MISO_GPIO                   GPIOB
#define SPIx_MISO_PIN                    GPIO_PIN_4
#define SPIx_MISO_AF                     GPIO_AF5_SPI1

#define SPIx_MOSI_CLK_ENABLE()          __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MOSI_GPIO                  GPIOB
```

```
#define SPIx_MOSI_PIN                   GPIO_PIN_5
#define SPIx_MOSI_AF                    GPIO_AF5_SPI1

#define SPIx_TX_DMA_STREAM               DMA2_Stream3
#define SPIx_RX_DMA_STREAM               DMA2_Stream2

#define SPIx_TX_DMA_REQUEST              DMA_REQUEST_SPI1_TX
#define SPIx_RX_DMA_REQUEST              DMA_REQUEST_SPI1_RX

#define SPIx_DMA_TX_IRQn                 DMA2_Stream3_IRQn
#define SPIx_DMA_RX_IRQn                 DMA2_Stream2_IRQn

#define SPIx_DMA_TX_IRQHandler           DMA2_Stream3_IRQHandler
#define SPIx_DMA_RX_IRQHandler           DMA2_Stream2_IRQHandler

#define SPIx_IRQn                        SPI1_IRQn
#define SPIx_IRQHandler                  SPI1_IRQHandler
```

◆ 第 3 步：根据使用的 CLR 和 LDAC 引脚,修改 bsp_spidma_dac8562.c 文件开头的宏定义。

```
/* CLR */
#define CLR_CLK_ENABLE()    __HAL_RCC_GPIOE_CLK_ENABLE()
#define CLR_GPIO            GPIOE
#define CLR_PIN                 GPIO_PIN_4
#define CLR_1()                 CLR_GPIO->BSRR = CLR_PIN
#define CLR_0()                 CLR_GPIO->BSRR = ((uint32_t)CLR_PIN << 16U)

/* LDAC 使用扩展 IO ，特别注意，我们这里是用的扩展 IO 控制的 */
#define LDAC_1()            HC574_SetPin(NRF24L01_CE, 1);
#define LDAC_0()            HC574_SetPin(NRF24L01_CE, 0);
```

◆ 第 4 步：如果使用 DMA 方式的话，请不要使用 TCM RAM，因为通用 DMA1 和 DMA2 不支持。并为了防止 DMA 和 CPU 同时访问 DMA 缓冲造成的数据一致性问题，将这块空间关闭 Cache 处理，比如使用的 SRAM4：

```
/* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress      = 0x38000000;
MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

◆ 第 5 步：初始化 SPI。

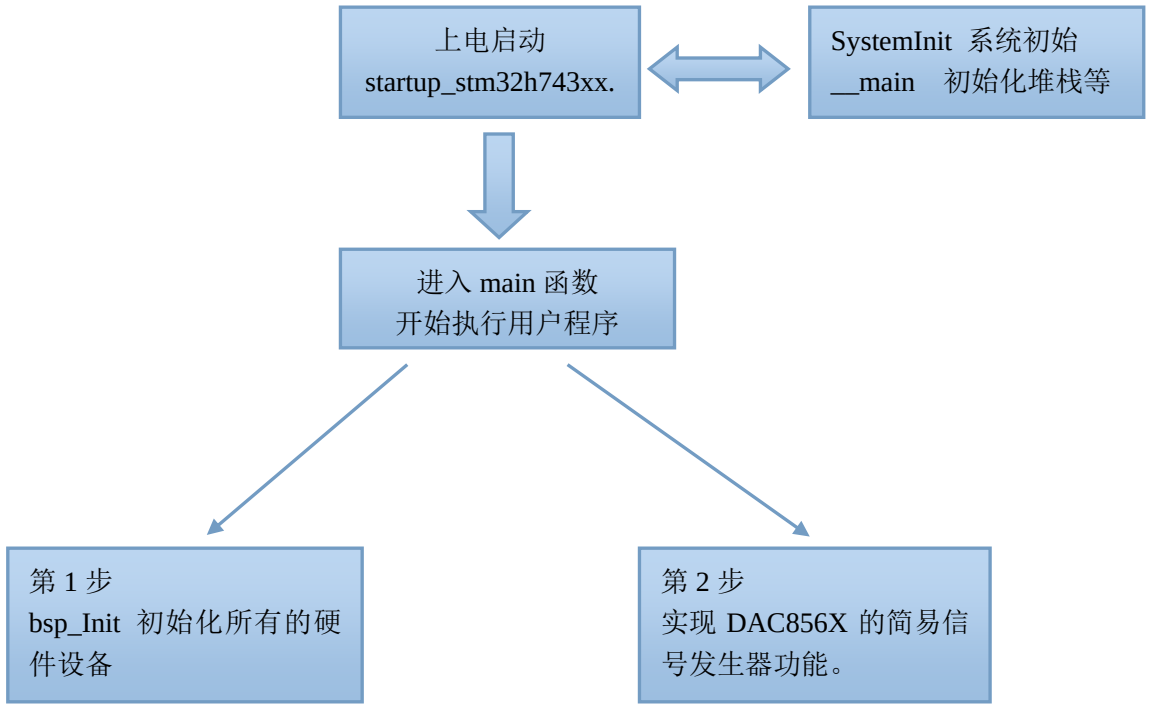
```
/* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
bsp_InitDAC8562(); /* 初始化配置 DAC8562/8563 */
```

◆ 第 6 步：DAC856X 驱动主要用到 HAL 库的 SPI 驱动文件，简单省事些可以添加所有 HAL 库 C 源文件进来。

◆ 第 7 步：应用方法看本章节配套例子即可

74.13 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器和 LED。
  - 第 2 部分，应用程序设计部分，实现 DAC856X 的简易信号发生器功能。。

74.14 实验例程说明（MDK）

注：本章是配套了两个例子的，这里我们以 SPI DMA 方式进行说明。

配套例子：

V7-052_DAC856x 简易信号发生器（双通道 SPI DMA 方式，16bit 分辨率，正负 10V 输出）

V7-053_DAC856x 简易信号发生器（双通道 SPI 查询方式，16bit 分辨率，正负 10V 输出）

实验目的：

3. 学习 SPI Flash 的 DAC8563 的 SPI DMA 驱动方式实现。

实验内容：

1. 双通道 DAC，轨到轨输出，16bit 分辨率，支持 50MHz 的 SPI 时钟速度。
2. 自带 2.5V 的内部参考基准，典型的温飘是 4ppm/℃，使用内部 2.5V 参考基准的情况下，

根据增益设置不同，DAC 的输出量可以为 0 到 2.5V 或者 0 到 5V。

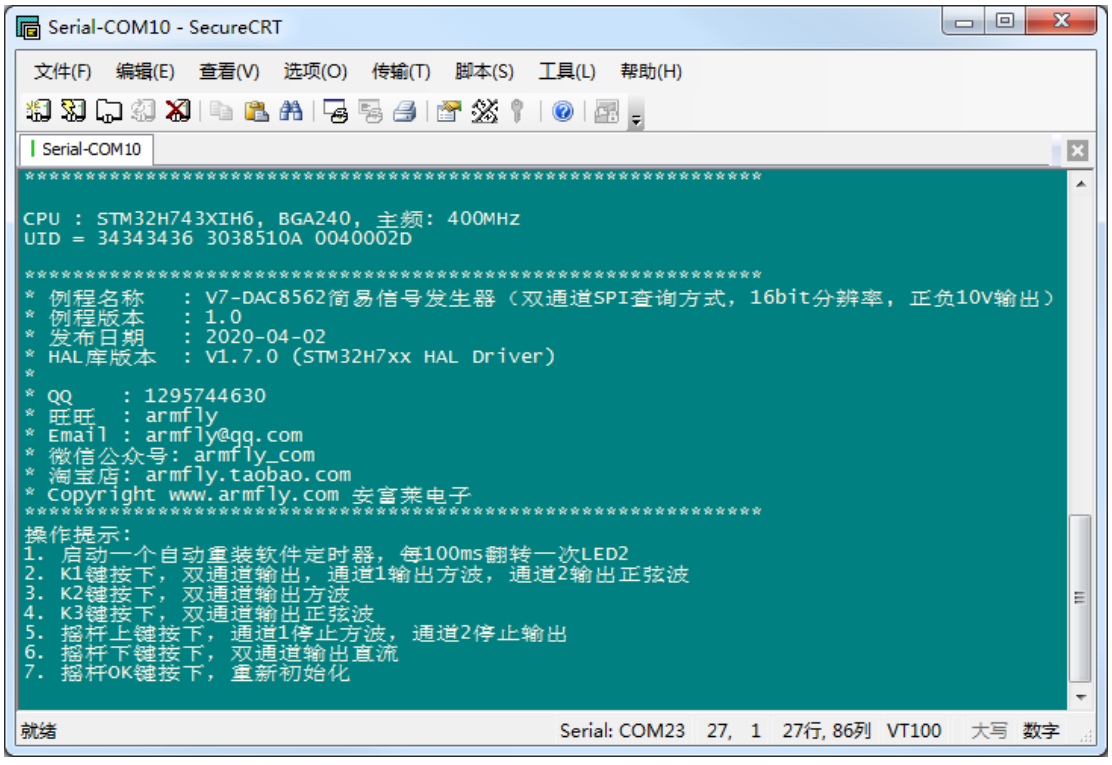
3. DAC8562 和 DAC8563 完全兼容，区别仅仅在于 CLR 引脚有效时，DAC8562 数据设置为 0，DAC8563 数据设置为 32767，注意这是 DAC 的内部数据，不表示输出电压。 对于-10 ~ +10V 输出的模块，DAC8562 输出-10V, DAC8563 输出 0V。
4. 无论是用 DAC8562 还是 DAC8563 芯片，只要软件不启动，本模块输出电压缺省状态都是 0V。
5. CLR 脚悬浮时，电压在 1.9V 左右，容易受到干扰导致输出被清零。因此即使不用 CLR 控制功能，这个 CLR 脚也需要接固定电平（推荐接 GND）。CLR 是边沿触发，仅在下降沿信号出现执行清零。

实验操作：

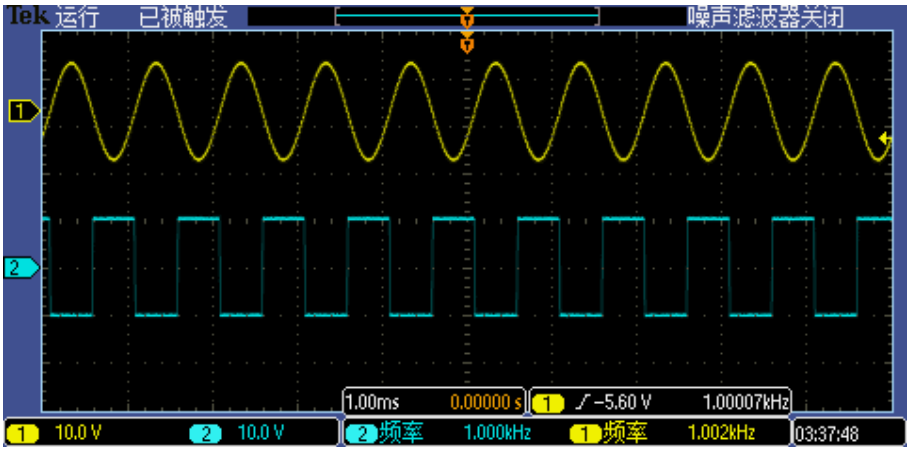
10. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
11. K1 键按下，双通道输出，通道 1 输出方波，通道 2 输出正弦波。
12. K2 键按下，双通道输出方波。
13. K3 键按下，双通道输出正弦波。
14. 摇杆上键按下，通道 1 停止方波，通道 2 停止输出。
15. 摇杆下键按下，双通道输出直流。
16. 摇杆 OK 键按下，重新初始化。

上电后串口打印的信息：

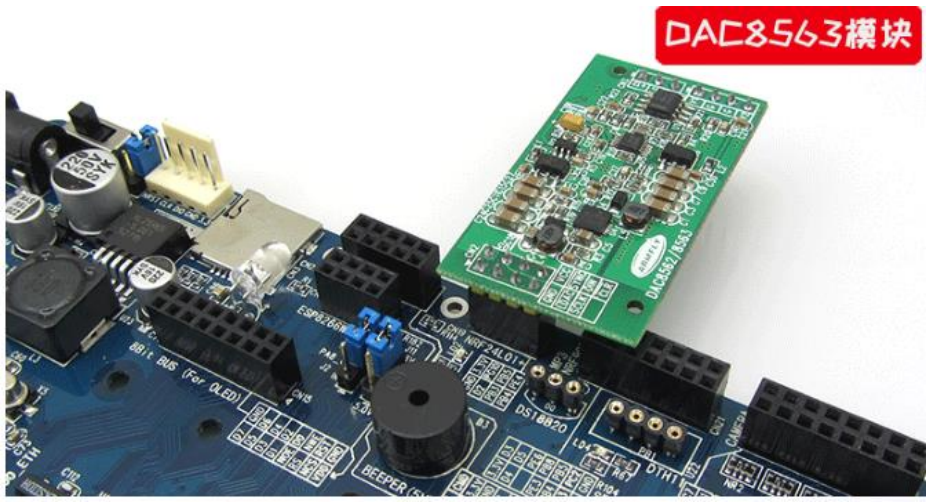
波特率 115200，数据位 8，奇偶校验位无，停止位 1。



波形效果：

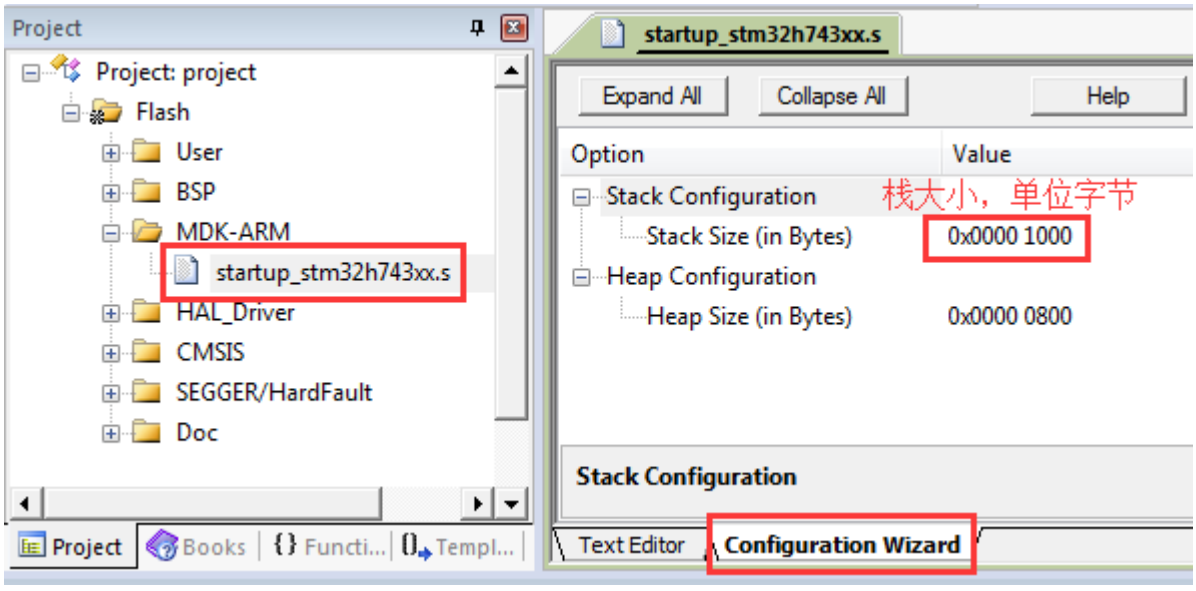


模块插入位置：

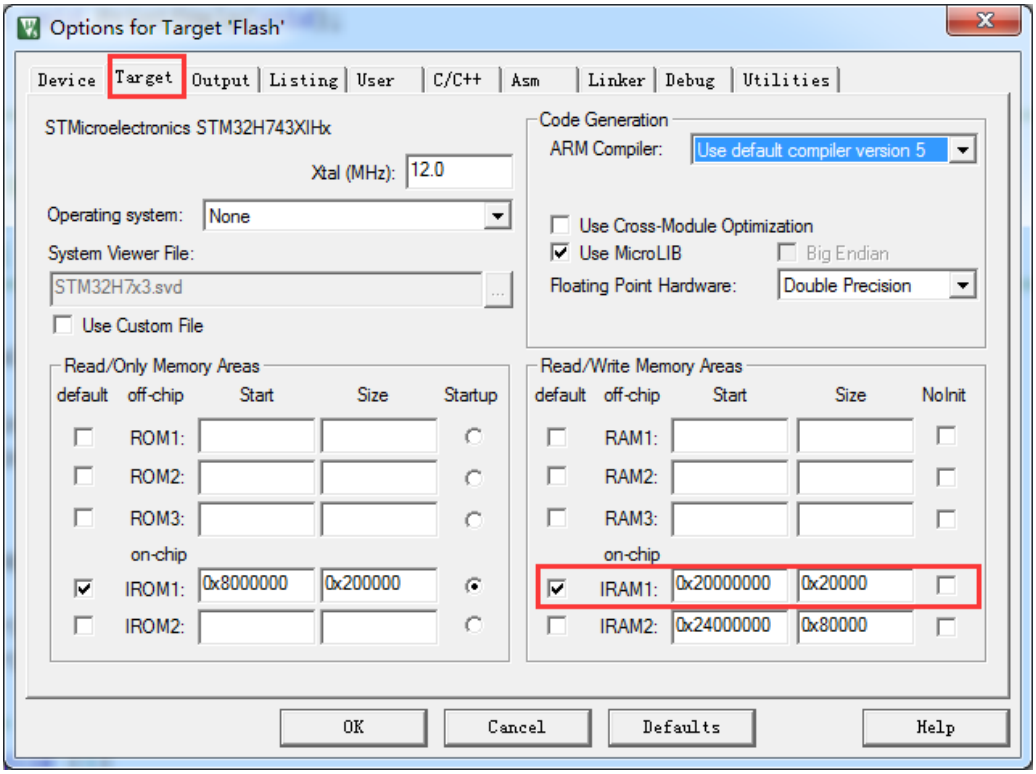


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();      /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();    /* 初始化滴答定时器 */
    bsp_InitLPUart();   /* 初始化串口 */
    bsp_InitExtIO();    /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();      /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitDAC8562(); /* 初始化配置 DAC8562/8563 */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区以及 SRAM4

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

```
    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **每 10ms 调用一次按键处理：**

  按键处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
```

```
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能:**

主程序实现如下操作:

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- K1 键按下，双通道输出，通道 1 输出方波，通道 2 输出正弦波。
- K2 键按下，双通道输出方波。
- K3 键按下，双通道输出正弦波。
- 摇杆上键按下，通道 1 停止方波，通道 2 停止输出。
- 摇杆下键按下，双通道输出直流。
- 摇杆 OK 键按下，重新初始化。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */

    DemoSpiDac();    /* SPI DAC 测试 */
}

/*
*********************************************************************************************************
*    函 数 名: DemoSpiDac
*    功能说明: DAC8562 测试
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoSpiDac(void)
{
    uint8_t i=0;
    uint8_t ucKeyCode; /* 按键代码 */

    sfDispMenu();      /* 打印命令提示 */

    bsp_StartAutoTimer(0, 200); /* 启动 1 个 100ms 的自动重装的定时器 */
```

```
/* 生成方波数据 */
for(i =0; i< 50; i++)
{
    ch1buf[i] = 0;
}

for(i =50; i< 100; i++)
{
    ch1buf[i] = 65535;
}

/* 生成正弦波数据 */
MakeSinTable(ch2buf, 100, 0, 65535);

/*
   上电默认双通道输出：
    第 1 个参数：
      1 表示通道 1 输出
      2 表示通道 2 输出
      3 表示通道 1 和 2 都输出
     4 表示通道 1 和 2 都输出，并且附加一个控制命令，有效防止传输错误时恢复，即使插拔模块也不影响。
    最后一个参数：
      定时器触发速度 1MHz，触发 1 次是一组 24bit 数据的传输。
      推荐范围 100Hz - 1MHz。
*/
DAC8562_SetDacDataDMA(4, ch1buf, ch2buf, sizeof(ch1buf)/sizeof(uint16_t),
                      sizeof(ch2buf)/sizeof(uint16_t), 1000000);

while(1)
{
    bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

    /* 判断定时器超时时间 */
    if (bsp_CheckTimer(0))
    {
        /* 每隔 100ms 进来一次 */
        bsp_LedToggle(2);
    }

    /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
    ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
    if (ucKeyCode != KEY_NONE)
    {
        switch (ucKeyCode)
        {
            case KEY_DOWN_K1:           /* K1 键按下，双通道输出，通道 1 输出方波，通道 2 输出正弦波 */
                /* 生成方波数据 */
                for(i =0; i< 50; i++)
                {
                    ch1buf[i] = 0;
                }

                for(i =50; i< 100; i++)
                {
                    ch1buf[i] = 65535;
                }
```

```
        /* 生成正弦波数据 */
        MakeSinTable(ch2buf, 100, 0, 65535);

        /* 上电默认双通道输出，触发速度 1MHz */
        DAC8562_SetDacDataDMA(3, ch1buf, ch2buf, sizeof(ch1buf)/sizeof(uint16_t),
                              sizeof(ch2buf)/sizeof(uint16_t), 1000000);
        break;

    case KEY_DOWN_K2:           /* K2 键按下，双通道输出方波 */
        /* 生成正弦波数据 */
        MakeSinTable(ch2buf, 100, 0, 65535);

        /* 上电默认双通道输出，触发速度 1MHz */
        DAC8562_SetDacDataDMA(3, ch2buf, ch2buf, sizeof(ch2buf)/sizeof(uint16_t),
                              sizeof(ch2buf)/sizeof(uint16_t), 1000000);
        break;

    case KEY_DOWN_K3:           /* K3 键按下，双通道输出正弦波 */
        /* 生成方波数据 */
        for(i =0; i< 50; i++)
        {
            ch1buf[i] = 0;
        }

        for(i =50; i< 100; i++)
        {
            ch1buf[i] = 65535;
        }

        /* 上电默认双通道输出，触发速度 1MHz */
        DAC8562_SetDacDataDMA(3, ch1buf, ch1buf, sizeof(ch1buf)/sizeof(uint16_t),
                              sizeof(ch1buf)/sizeof(uint16_t), 1000000);
        break;

    case JOY_DOWN_U:            /* 摇杆上键按下，通道 1 停止方波，通道 2 停止输出 */
        /* 通道 1 输出方波 */
        for(i =0; i< 50; i++)
        {
            ch1buf[i] = 0;
        }

        for(i =50; i< 100; i++)
        {
            ch1buf[i] = 65535;
        }

        /* 仅通道 1 输出 */
        DAC8562_SetDacDataDMA(1, ch1buf, ch2buf, sizeof(ch1buf)/sizeof(uint16_t),
                              sizeof(ch2buf)/sizeof(uint16_t), 1000000);
        break;

    case JOY_DOWN_D:            /* 摇杆下键按下，双通道输出直流 */
        /* 通道 1 输出负数 10V */
        DAC8562_SetDacData(0, 0);

        /* 通道 2 输出正 10V */
        DAC8562_SetDacData(1, 65535);
```

```
                    break;

                case JOY_DOWN_OK:           /* 摇杆 OK 键按下，重新初始化  */
                    /* 初始化配置 DAC8562/8563 */
                    //bsp_InitDAC8562();
                    DAC8562_WriteCmd((7 << 19) | (0 << 16) | (1 << 0));
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

74.15 实验例程说明（IAR）

注：本章是配套了两个例子的，这里我们以 SPI DMA 方式进行说明。

配套例子：

V7-052_DAC856x 简易信号发生器（双通道 SPI DMA 方式，16bit 分辨率，正负 10V 输出）

V7-053_DAC856x 简易信号发生器（双通道 SPI 查询方式，16bit 分辨率，正负 10V 输出）

实验目的：

1. 学习 SPI Flash 的 DAC8563 的 SPI DMA 驱动方式实现。

实验内容：

1. 双通道 DAC，轨到轨输出，16bit 分辨率，支持 50MHz 的 SPI 时钟速度。
2. 自带 2.5V 的内部参考基准，典型的温飘是 4ppm/℃，使用内部 2.5V 参考基准的情况下，根据增益设置不同，DAC 的输出量可以为 0 到 2.5V 或者 0 到 5V。
3. DAC8562 和 DAC8563 完全兼容，区别仅仅在于 CLR 引脚有效时，DAC8562 数据设置为 0，DAC8563 数据设置为 32767，注意这是 DAC 的内部数据，不表示输出电压。 对于-10 ~ +10V 输出的模块，DAC8562 输出-10V，DAC8563 输出 0V。
4. 无论是用 DAC8562 还是 DAC8563 芯片，只要软件不启动，本模块输出电压缺省状态都是 0V。
5. CLR 脚悬浮时，电压在 1.9V 左右，容易受到干扰导致输出被清零。因此即使不用 CLR 控制功能，这个 CLR 脚也需要接固定电平（推荐接 GND）。CLR 是边沿触发，仅在下降沿信号出现执行清零。

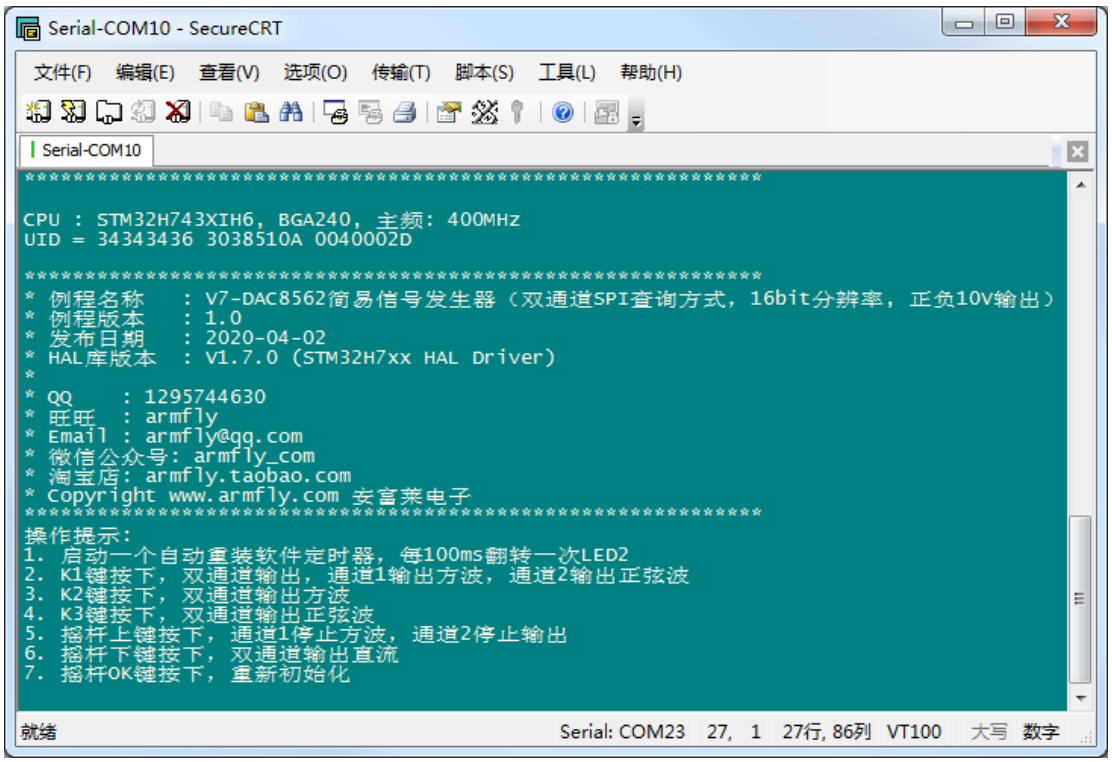
实验操作：

1. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
2. K1 键按下，双通道输出，通道 1 输出方波，通道 2 输出正弦波。
3. K2 键按下，双通道输出方波。
4. K3 键按下，双通道输出正弦波。
5. 摇杆上键按下，通道 1 停止方波，通道 2 停止输出。

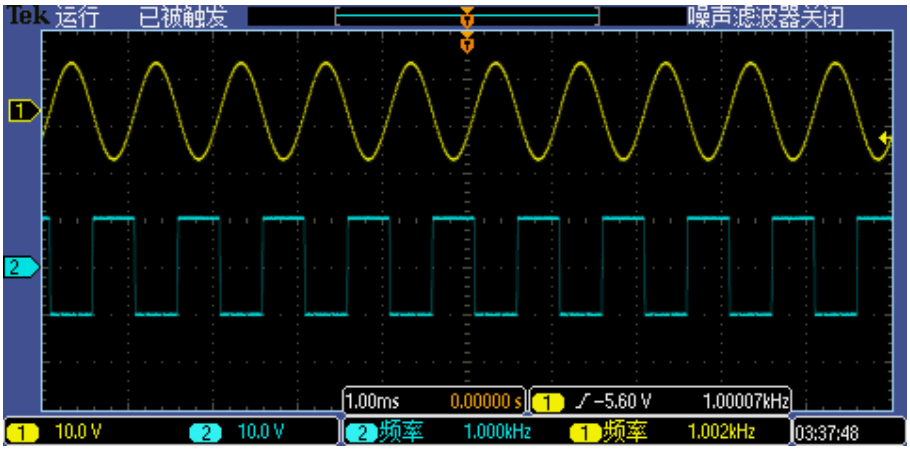
6. 摇杆下键按下，双通道输出直流。
7. 摇杆 OK 键按下，重新初始化。

上电后串口打印的信息：

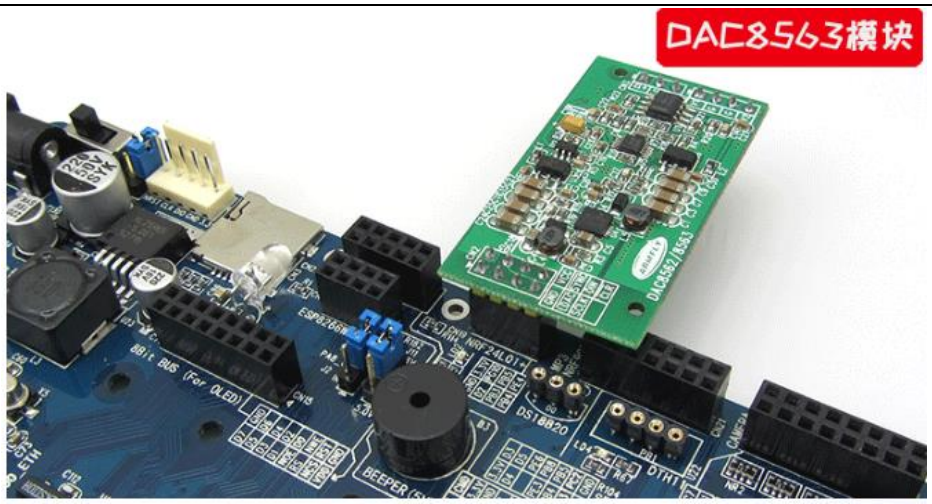
波特率 115200，数据位 8，奇偶校验位无，停止位 1。



波形效果：

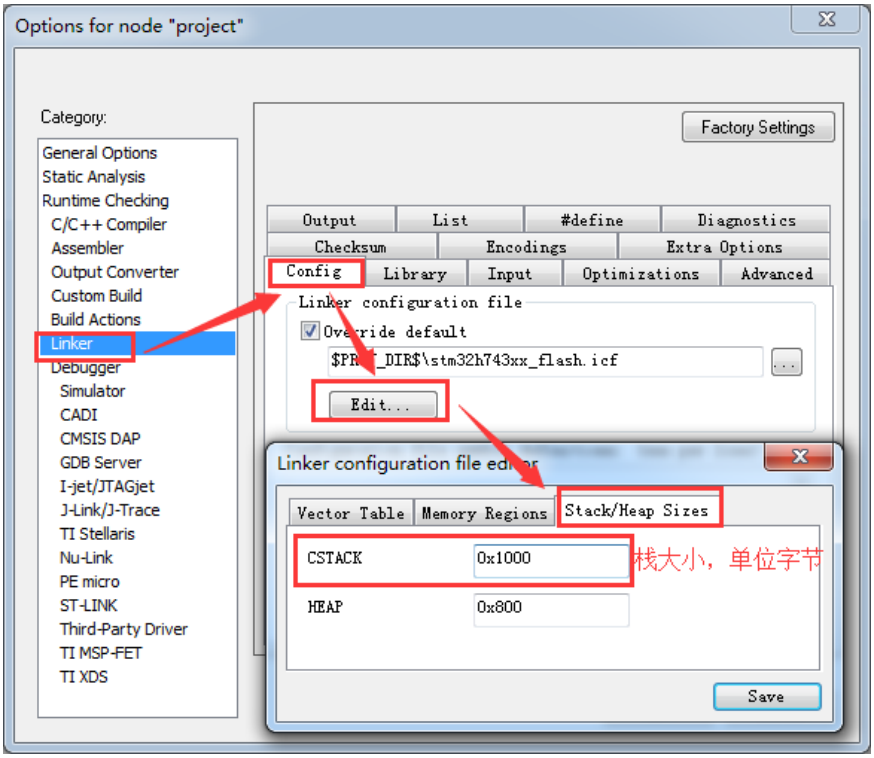


模块插入位置：

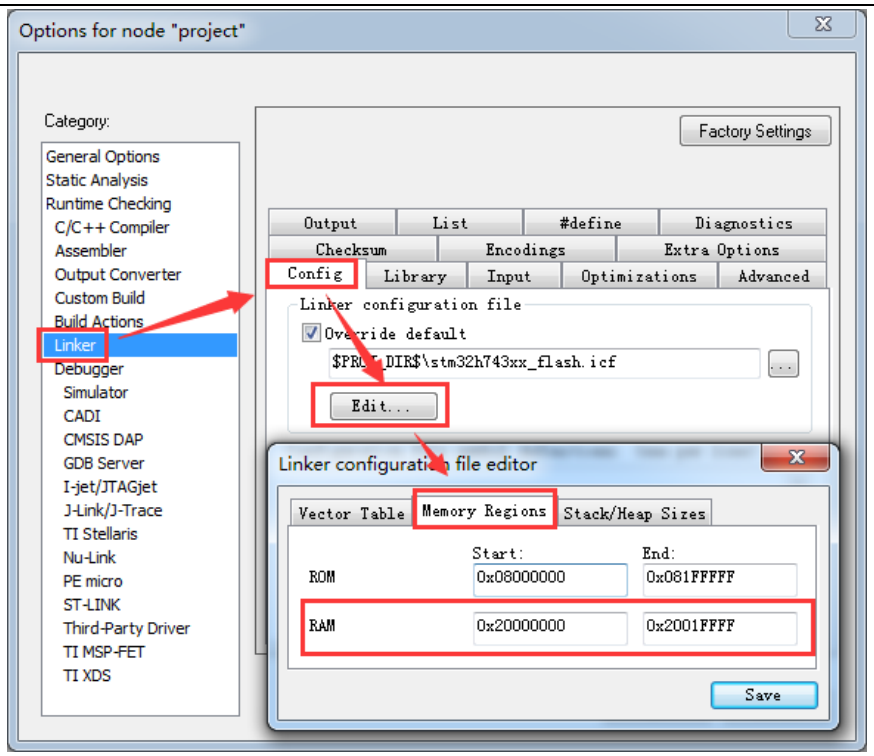


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();      /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();    /* 初始化滴答定时器 */
    bsp_InitLPUart();   /* 初始化串口 */
    bsp_InitExtIO();    /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();      /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitDAC8562(); /* 初始化配置 DAC8562/8563 */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区以及 SRAM4

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
```

```
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **每 10ms 调用一次按键处理：**

按键处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
```

```
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能:**

主程序实现如下操作:

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- K1 键按下，双通道输出，通道 1 输出方波，通道 2 输出正弦波。
- K2 键按下，双通道输出方波。
- K3 键按下，双通道输出正弦波。
- 摇杆上键按下，通道 1 停止方波，通道 2 停止输出。
- 摇杆下键按下，双通道输出直流。
- 摇杆 OK 键按下，重新初始化。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */

    DemoSpiDac();   /* SPI DAC 测试 */
}

/*
*********************************************************************************************************
*    函 数 名: DemoSpiDac
*    功能说明: DAC8562 测试
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoSpiDac(void)
{
    uint8_t i=0;
    uint8_t ucKeyCode; /* 按键代码 */

    sfDispMenu();      /* 打印命令提示 */

    bsp_StartAutoTimer(0, 200); /* 启动 1 个 100ms 的自动重装的定时器 */
```

```
    /* 生成方波数据 */
    for(i =0; i< 50; i++)
    {
        ch1buf[i] = 0;
    }

    for(i =50; i< 100; i++)
    {
        ch1buf[i] = 65535;
    }

    /* 生成正弦波数据 */
    MakeSinTable(ch2buf, 100, 0, 65535);

    /*
       上电默认双通道输出：
        第 1 个参数：
           1 表示通道 1 输出
           2 表示通道 2 输出
           3 表示通道 1 和 2 都输出
         4 表示通道 1 和 2 都输出，并且附加一个控制命令，有效防止传输错误时恢复，即使插拔模块也不影响。
        最后一个参数：
          定时器触发速度 1MHz，触发 1 次是一组 24bit 数据的传输。
          推荐范围 100Hz - 1MHz。
    */
    DAC8562_SetDacDataDMA(4, ch1buf, ch2buf, sizeof(ch1buf)/sizeof(uint16_t),
                          sizeof(ch2buf)/sizeof(uint16_t), 1000000);

    while(1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，双通道输出，通道 1 输出方波，通道 2 输出正弦波 */
                    /* 生成方波数据 */
                    for(i =0; i< 50; i++)
                    {
                        ch1buf[i] = 0;
                    }

                    for(i =50; i< 100; i++)
                    {
                        ch1buf[i] = 65535;
                    }

                    /* 生成正弦波数据 */
```

```
        MakeSinTable(ch2buf, 100, 0, 65535);

        /* 上电默认双通道输出，触发速度 1MHz */
        DAC8562_SetDacDataDMA(3, ch1buf, ch2buf, sizeof(ch1buf)/sizeof(uint16_t),
                              sizeof(ch2buf)/sizeof(uint16_t), 1000000);
        break;

    case KEY_DOWN_K2:           /* K2 键按下，双通道输出方波 */
        /* 生成正弦波数据 */
        MakeSinTable(ch2buf, 100, 0, 65535);

        /* 上电默认双通道输出，触发速度 1MHz */
        DAC8562_SetDacDataDMA(3, ch2buf, ch2buf, sizeof(ch2buf)/sizeof(uint16_t),
                              sizeof(ch2buf)/sizeof(uint16_t), 1000000);
        break;

    case KEY_DOWN_K3:           /* K3 键按下，双通道输出正弦波 */
        /* 生成方波数据 */
        for(i =0; i< 50; i++)
        {
            ch1buf[i] = 0;
        }

        for(i =50; i< 100; i++)
        {
            ch1buf[i] = 65535;
        }

        /* 上电默认双通道输出，触发速度 1MHz */
        DAC8562_SetDacDataDMA(3, ch1buf, ch1buf, sizeof(ch1buf)/sizeof(uint16_t),
                              sizeof(ch1buf)/sizeof(uint16_t), 1000000);
        break;

    case JOY_DOWN_U:            /* 摇杆上键按下，通道 1 停止方波，通道 2 停止输出 */
        /* 通道 1 输出方波 */
        for(i =0; i< 50; i++)
        {
            ch1buf[i] = 0;
        }

        for(i =50; i< 100; i++)
        {
            ch1buf[i] = 65535;
        }

        /* 仅通道 1 输出 */
        DAC8562_SetDacDataDMA(1, ch1buf, ch2buf, sizeof(ch1buf)/sizeof(uint16_t),
                              sizeof(ch2buf)/sizeof(uint16_t), 1000000);
        break;

    case JOY_DOWN_D:            /* 摇杆下键按下，双通道输出直流 */
        /* 通道 1 输出负数 10V */
        DAC8562_SetDacData(0, 0);

        /* 通道 2 输出正 10V */
        DAC8562_SetDacData(1, 65535);
        break;
```

```
                case JOY_DOWN_OK:           /* 摇杆 OK 键按下，重新初始化  */
                    /* 初始化配置 DAC8562/8563 */
                    //bsp_InitDAC8562();
                    DAC8562_WriteCmd((7 << 19) | (0 << 16) | (1 << 0));
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

74.16 总结

本章节涉及到的知识点非常多，特别是 SPI DMA 方式驱动的实现方法，需要大家稍花点精力去研究。

第75章 STM32H7 的 SPI 总线应用之驱动 DAC8501（双路输出，16bit 分辨率，0-5V）

本章节为大家讲解标准 SPI 接线方式驱动模数转换器 DAC8501，制作了中断和 DMA 两种驱动方式。

75.1 初学者重要提示
75.2 DAC 结构分类和技术术语
75.3 DAC8501 硬件设计
75.4 DAC8501 关键知识点整理（重要）
75.5 DAC8501 驱动设计（中断更新方式）
75.5 DAC8501 驱动设计（SPI DMA 更新方式）
75.6 SPI 总线板级支持包（bsp_spi_bus.c）
75.7 DAC8501 支持包中断方式（bsp_spi_dac8501.c）
75.8 DAC8501 支持包 DMA 方式（bsp_spidma_dac8501.c）
75.9 DAC8501 驱动移植和使用（中断更新方式）
75.10 DAC8501 驱动移植和使用（SPI DMA 更新方式）
75.11 实验例程设计框架
75.12 实验例程说明（MDK）
75.13 实验例程说明（IAR）
75.14 总结

75.1 初学者重要提示

- 学习本章节前，务必优先学习第 72 章。
- DAC8501 模块上带了两片 8501，每片是单通道 DAC，带片上输出缓冲运放，轨到轨输出，16bit 分辨率，支持 30MHz 的 SPI 时钟速度。
- 本章涉及到的知识点比较多，需要大家掌握 STM32H7 的 SPI，DMA，TIM，DMAMUX 和 DAC8501 的一些细节用法。
- H7 的 SPI + DMA 驱动这类外设的灵活度，绝对可以媲美 FPGA 去控制：
  - H7 的 SPI 外设比 F4 系列的灵活性强太多了，主要表现在两个方面：数据的传输支持了 4-32bit，特别是那个 NSS 片选引脚，超强劲，可以做各种时间插入，灵活应对了市场上这类芯片的需求。

- DMA 这块相比 F4 系列，有了质的飞跃，支持了 DMAMUX，这个 DMAMUX 除了带来灵活的触发源选择，还支持了各种触发事件和同步触发功能。本章配套例子的触发周期控制就是利用了 DMAMUX 的同步触发功能。

- 本章配套了中断和 DMA 两种更新方式的案例，DMA 实现方式与中断更新方式完全不同，因为 DMA 方式要使用硬件 SPI1 NSS 片选引脚驱动 DAC8501。而中断更新方式使用公共的总线驱动文件 bsp_spi_bus.c，片选是通过通用 IO 方式控制，支持串行 FLASH、TSC2046、VS1053、AD7705、ADS1256 等 SPI 设备。大家在看例子的时候要注意。
- 对于本章教程配套例子的 SPI DMA 方式，这里特别注意一点，定时器触发一次，就会让 SPI 以 DMA 方式传输 24bit 数据。
- DAC8501 数据手册，模块原理图和接线图都已经放到本章教程配置例子的 Doc 文件里。

75.2 DAC 结构分类和技术术语

在本教程的第 74 章进行了详细说明。

75.3 DAC8501 硬件设计

DAC 的原理图下载：
http://www.armbbs.cn/forum.php?mod=viewthread&tid=97082 。

75.3.1 DAC8501 模块规格

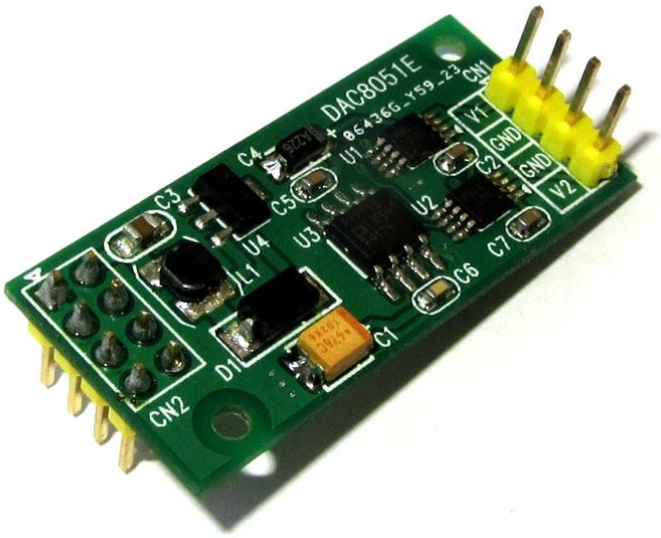
产品规格：

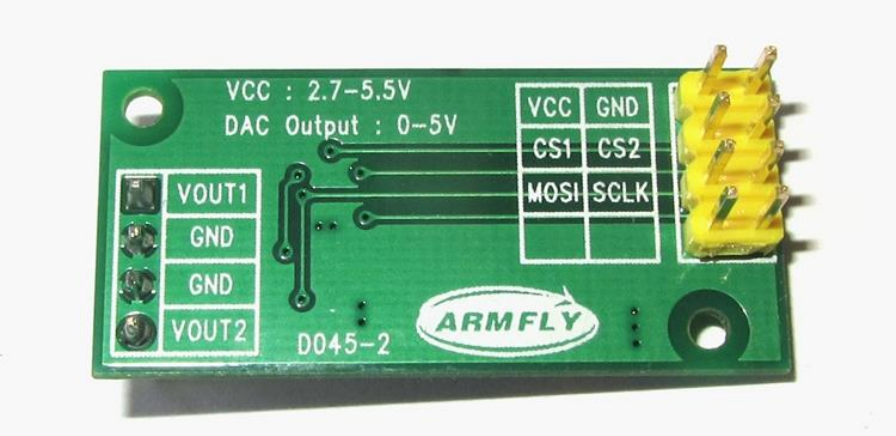
1、供电电压： 2.7 - 5.5V【3.3V 供电时，输出电压也可以到 5V】。
2、通道数： 2 路 （通过 2 片 DAC8501E 实现）。
3、输出电压范围 ： 0 - 5V【零位 < 0.020V， 满位 > 4.970V】。
4、分辨率： 16 位。
5、功耗 ： 小于 10mA。
6、MCU 接口 ： 高速 SPI （30M）支持 3.3V 和 5V 单片机。
7、DAC 输出模拟带宽：350KHz。
8、DAC 输出响应： 10uS 到 0.003% FSR。

产品特点：

1、输出和供电电压无关；模块内带升压电路和 5V 基准。
2、自适应单片机的电平（2.7 - 5V 均可以）。
3、输出电压轨到轨，最高电压可以到 4.970V 以上。

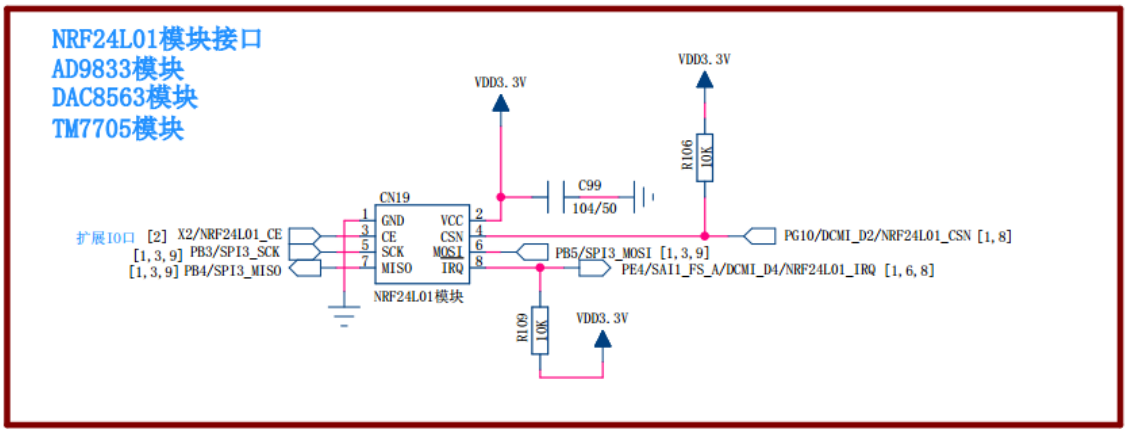
产品效果：



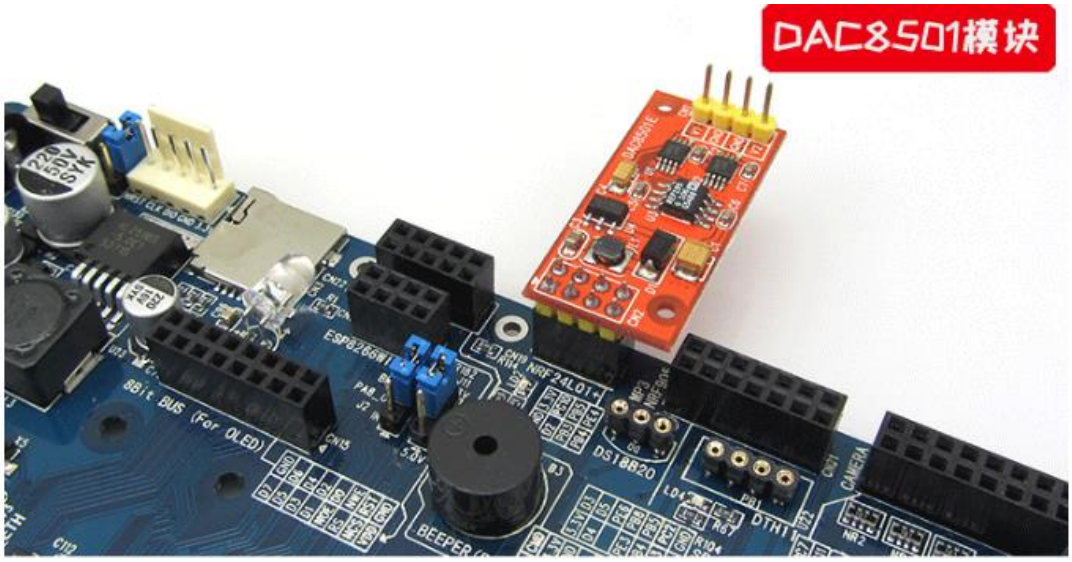


75.3.2 DAC8501 硬件接口

V7 板子上 DAC8501 模块的插座的原理图如下：



实际对应开发的位置如下：

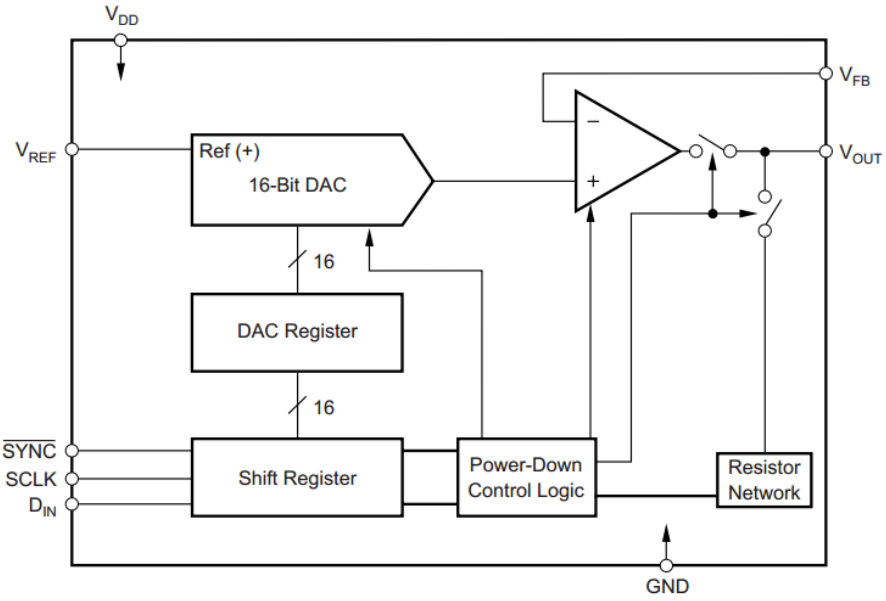


75.4 DAC8501 关键知识点整理（重要）

驱动 DAC8501 需要对下面这些知识点有个认识。

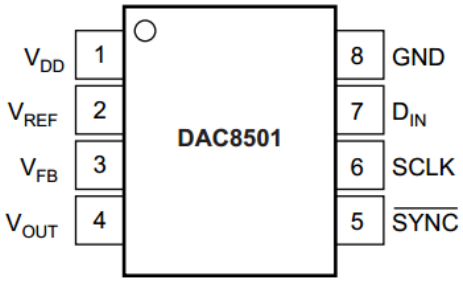
75.4.1 DAC8501 基础信息

- 单通道 DAC，带片上输出缓冲运放，轨到轨输出，16bit 分辨率，支持 30MHz 的 SPI 时钟速度。
- 模拟输出带宽 350KHz。
- 供电范围 2.7V 到 5.5V。
- 具有低功耗特性。
- 上电复位输出 0V。



75.4.2 DAC8501 每个引脚的作用

DAC8501 的封装形式：



- Vdd

  供电范围 2.7-5.5V。

- Vref

  稳压基准输入。

- Vfb

  输出运放的反馈。

- Vout

  模拟输出电压，输出运放具有轨到轨特性。

- SYNC （片选）

  低电平有效，当 SYNC 变为低电平时，它使能输入移位寄存器，并且数据采样在随后的时钟下降沿。DAC 输出在第 24 个时钟下降沿之后更新。 如果 SYNC 在第 23 个时钟沿之前变高，SYNC 的上升沿将充当中断，并且 DAC8501 将忽略写序列。

- SCLK

  时钟输入端，支持 30MHz。

- Din

  串行时钟输入，每个时钟下降沿将数据写到的 24bit 的输入移位寄存器。

- GND

  接地端。

75.4.3 DAC8501 输出电压计算公式

DAC8501 的计算公式如下：

$$V_{OUT} = V_{REF} \bullet \frac{D}{65536}$$

- D

  配置 DAC8501 数据输出寄存器的数值，范围 0 到 2^16 – 1，即 0 到 65535。

- V_{REF}

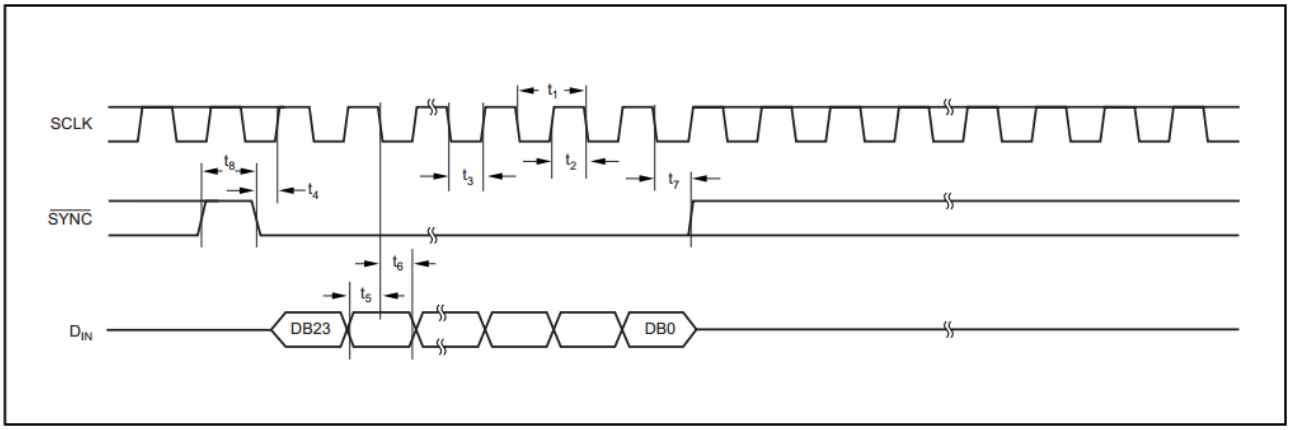
  使用外部参考电压，由 V_{REFIN} 引脚的输入决定。

- Vout

  输出电压。

75.4.4 DAC8501 时序图

DAC8501 的时序图如下：



PARAMETER	DESCRIPTION	CONDITIONS	DAC8501E MIN	DAC8501E TYP	DAC8501E MAX	UNITS
t_1(3)	SCLK Cycle Time	V_{DD} = 2.7V to 3.6V	50			ns
		V_{DD} = 3.6V to 5.5V	33			ns
t_2	SCLK HIGH Time	V_{DD} = 2.7V to 3.6V	13			ns
		V_{DD} = 3.6V to 5.5V	13			ns
t_3	SCLK LOW Time	V_{DD} = 2.7V to 3.6V	22.5			ns
		V_{DD} = 3.6V to 5.5V	13			ns
t_4	$\overline{SYNC}$ to SCLK Rising Edge Setup Time	V_{DD} = 2.7V to 3.6V	0			ns
		V_{DD} = 3.6V to 5.5V	0			ns
t_5	Data Setup Time	V_{DD} = 2.7V to 3.6V	5			ns
		V_{DD} = 3.6V to 5.5V	5			ns
t_6	Data Hold Time	V_{DD} = 2.7V to 3.6V	4.5			ns
		V_{DD} = 3.6V to 5.5V	4.5			ns
t_7	SCLK Falling Edge to $\overline{SYNC}$ Rising Edge	V_{DD} = 2.7V to 3.6V	0			ns
		V_{DD} = 3.6V to 5.5V	0			ns
t_8	Minimum $\overline{SYNC}$ HIGH Time	V_{DD} = 2.7V to 3.6V	50			ns
		V_{DD} = 3.6V to 5.5V	33			ns

NOTES: (1) All input signals are specified with $t_R = t_F$ = 5ns (10% to 90% of V_{DD}) and timed from a voltage level of $(V_{IL} + V_{IH})/2$. (2) See Serial Write Operation timing diagram, below. (3) Maximum SCLK frequency is 30MHz at V_{DD} = +3.6V to +5.5V and 20MHz at V_{DD} = +2.7V to +3.6V.

这个时序里面有三个参数尤其重要，后面时序配置要用到。

- f (1)

供电 2.7 到 3.6V 时，最高时钟 20MHz。

供电 3.6 到 5.5V 时，最高时钟 30MHz。

- t (4)

  SYNC 低电平有效到 SCLK 第 1 个上降沿信号的时间没有最小值限制，可以为 0。

- t (8)

  每传输 24bit 数据后，SYNC 要保持一段时间的高电平。

  供电 2.7 到 3.6V 时，最小要求 50ns。

  供电 3.6 到 5.5V 时，最小要求 33ns。

75.4.5 DAC8501 寄存器配置

DAC8501 的寄存器配置是 24bit 格式：

DB23																						DB0
X	X	X	X	X	X	PD1	PD0	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0

控制 DAC8501 每次要传输 24bit 数据，高 8bit 控制位 + 16bit 数据位。

控制位的 PD1 和 PD0 定义如下：

PD1 (DB17)	PD0 (DB16)	OPERATING MODE
0	0	Normal Operation
—	—	Power-Down Modes
0	1	Output 1kΩ to GND
1	0	Output 100kΩ to GND
1	1	High-Z

PD1 PD0 决定 4 种工作模式

0 0 ---> 正常工作模式

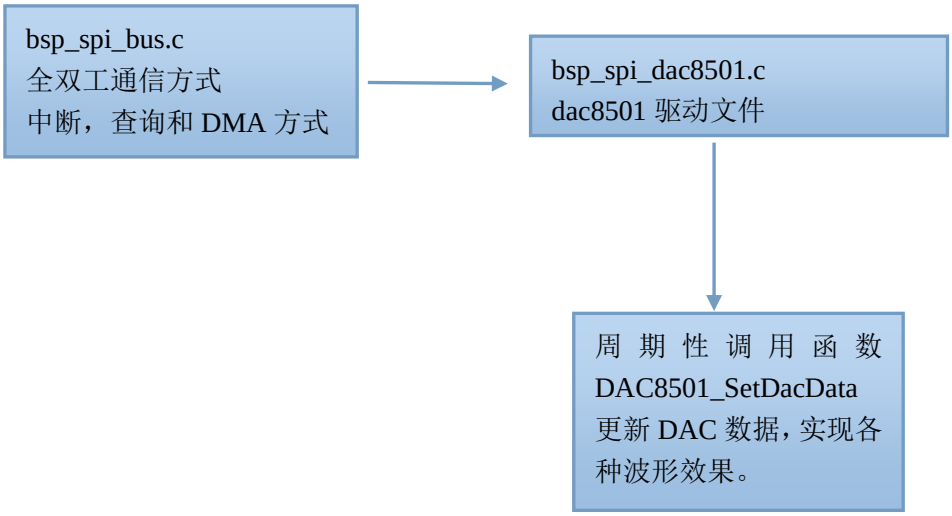
0 1 ---> 输出接 1K 欧到 GND

1 0 ---> 输出 100K 欧到 GND

1 1 ---> 输出高阻

75.5 DAC8501 驱动设计（中断更新方式）

DAC8501 的程序驱动框架设计如下：



有了这个框图，程序设计就比较好理解了。

75.5.1第 1 步：SPI 总线配置

spi 总线配置通过如下两个函数实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitSPIBus
*    功能说明: 配置 SPI 总线。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitSPIBus(void)
{
    g_spi_busy = 0;

    bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_8, SPI_PHASE_1EDGE, SPI_POLARITY_LOW);
}

/*
*********************************************************************************************************
*    函 数 名: bsp_InitSPIParam
*    功能说明: 配置 SPI 总线参数，时钟分频，时钟相位和时钟极性。
*    形    参: _BaudRatePrescaler  SPI 总线时钟分频设置，支持的参数如下：
*                                 SPI_BAUDRATEPRESCALER_2    2 分频
*                                 SPI_BAUDRATEPRESCALER_4    4 分频
*                                 SPI_BAUDRATEPRESCALER_8    8 分频
*                                 SPI_BAUDRATEPRESCALER_16   16 分频
*                                 SPI_BAUDRATEPRESCALER_32   32 分频
*                                 SPI_BAUDRATEPRESCALER_64   64 分频
*                                 SPI_BAUDRATEPRESCALER_128  128 分频
*                                 SPI_BAUDRATEPRESCALER_256  256 分频
*
*              _CLKPhase          时钟相位，支持的参数如下：
*                                 SPI_PHASE_1EDGE     SCK 引脚的第 1 个边沿捕获传输的第 1 个数据
```

```
*                                SPI_PHASE_2EDGE     SCK 引脚的第 2 个边沿捕获传输的第 1 个数据
*
*             _CLKPolarity       时钟极性，支持的参数如下：
*                                SPI_POLARITY_LOW    SCK 引脚在空闲状态处于低电平
*                                SPI_POLARITY_HIGH   SCK 引脚在空闲状态处于高电平
*
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitSPIParam(uint32_t _BaudRatePrescaler, uint32_t _CLKPhase, uint32_t _CLKPolarity)
{
    /* 提高执行效率，只有在 SPI 硬件参数发生变化时，才执行 HAL_Init */
    if (s_BaudRatePrescaler == _BaudRatePrescaler && s_CLKPhase == _CLKPhase && s_CLKPolarity == _CLKPolarity)
    {
        return;
    }

    s_BaudRatePrescaler = _BaudRatePrescaler;
    s_CLKPhase = _CLKPhase;
    s_CLKPolarity = _CLKPolarity;


    /* 设置 SPI 参数 */
    hspi.Instance               = SPIx;                     /* 例化 SPI */
    hspi.Init.BaudRatePrescaler = _BaudRatePrescaler;       /* 设置波特率 */
    hspi.Init.Direction         = SPI_DIRECTION_2LINES;     /* 全双工 */
    hspi.Init.CLKPhase          = _CLKPhase;                /* 配置时钟相位 */
    hspi.Init.CLKPolarity       = _CLKPolarity;             /* 配置时钟极性 */
    hspi.Init.DataSize          = SPI_DATASIZE_8BIT;        /* 设置数据宽度 */
    hspi.Init.FirstBit          = SPI_FIRSTBIT_MSB;         /* 数据传输先传高位 */
    hspi.Init.TIMode            = SPI_TIMODE_DISABLE;       /* 禁止 TI 模式  */
    hspi.Init.CRCCalculation    = SPI_CRCCALCULATION_DISABLE; /* 禁止 CRC */
    hspi.Init.CRCPolynomial     = 7;                         /* 禁止 CRC 后，此位无效 */
    hspi.Init.CRCLength         = SPI_CRC_LENGTH_8BIT;       /* 禁止 CRC 后，此位无效 */
    hspi.Init.NSS               = SPI_NSS_SOFT;              /* 使用软件方式管理片选引脚 */
    hspi.Init.FifoThreshold     = SPI_FIFO_THRESHOLD_01DATA;  /* 设置 FIFO 大小是一个数据项 */
    hspi.Init.NSSPMode          = SPI_NSS_PULSE_DISABLE;      /* 禁止脉冲输出 */
    hspi.Init.MasterKeepIOState = SPI_MASTER_KEEP_IO_STATE_ENABLE; /* 禁止 SPI 后，SPI 相关引脚保持当前状态 */
    hspi.Init.Mode              = SPI_MODE_MASTER;           /* SPI 工作在主控模式 */

    /* 复位配置 */
    if (HAL_SPI_DeInit(&hspi) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 初始化配置 */
    if (HAL_SPI_Init(&hspi) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

关于这两个函数有以下两点要做个说明：

- 函数 bsp_InitSPIBus 里面的配置是个初始设置。实际驱动芯片时，会通过函数 bsp_InitSPIParam 做再配置。

- 函数 bsp_InitSPIParam 提供了时钟分频，时钟相位和时钟极性配置。驱动不同外设芯片时，基本上调整这三个参数就够。当 SPI 接口上接了多个不同类型的芯片时，通过此函数可以方便的切换配置。

75.5.2第 2 步：SPI 总线的查询，中断和 DMA 方式设置

注：对于 DAC8501，请使用查询方式。

SPI 驱动的查询，中断和 DMA 方式主要通过函数 bsp_spiTransfer 实现数据传输：

```
/*
*********************************************************************************************************
*                                    选择 DMA，中断或者查询方式
*********************************************************************************************************
*/
//#define USE_SPI_DMA    /* DMA 方式  */
//#define USE_SPI_INT    /* 中断方式 */
#define USE_SPI_POLL   /* 查询方式 */

/* 查询模式 */
#if defined (USE_SPI_POLL)

uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];

/* 中断模式 */
#elif defined (USE_SPI_INT)

uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];

/* DMA 模式使用的 SRAM4 */
#elif defined (USE_SPI_DMA)
    #if defined ( __CC_ARM )    /* IAR *******/
        __attribute__((section (".RAM_D3"))) uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
        __attribute__((section (".RAM_D3"))) uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];
    #elif defined (__ICCARM__)   /* MDK ********/
        #pragma location = ".RAM_D3"
        uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
        #pragma location = ".RAM_D3"
        uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];
    #endif
#endif

/*
*********************************************************************************************************
*    函 数 名: bsp_spiTransfer
*    功能说明: 启动数据传输
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_spiTransfer(void)
{
    if (g_spiLen > SPI_BUFFER_SIZE)
    {
```

```
        return;
    }

    /* DMA 方式传输 */
#ifdef USE_SPI_DMA
    wTransferState = TRANSFER_WAIT;

    if(HAL_SPI_TransmitReceive_DMA(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen) != HAL_OK)

    {
        Error_Handler(__FILE__, __LINE__);
    }

    while (wTransferState == TRANSFER_WAIT)
    {
        ;
    }
#endif

    /* 中断方式传输 */
#ifdef USE_SPI_INT
    wTransferState = TRANSFER_WAIT;

    if(HAL_SPI_TransmitReceive_IT(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen) != HAL_OK)

    {
        Error_Handler(__FILE__, __LINE__);
    }

    while (wTransferState == TRANSFER_WAIT)
    {
        ;
    }
#endif

    /* 查询方式传输 */
#ifdef USE_SPI_POLL
    if(HAL_SPI_TransmitReceive(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen, 1000000) !=
HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
#endif
}
```

通过开头宏定义可以方便的切换中断，查询和 DMA 方式。其中查询和中断方式比较好理解，而 DMA 方式要特别注意两点：

- 通过本手册第 26 章的内存块超方便使用方式，将 DMA 缓冲定义到 SRAM4 上。因为本工程是用的 DTCM 做的主 RAM 空间，这个空间无法使用通用 DMA1 和 DMA2。
- 由于程序里面开启了数据 Cache，会造成 DMA 和 CPU 访问 SRAM4 数据不一致的问题，特此将 SRAM4 空间关闭 Cache。

```
    /* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
```

```
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

75.5.3第 3 步：DAC8501 的时钟极性和时钟相位配置

首先回忆下 STM32H7 支持的 4 种时序配置。

- 当 CPOL = 1， CPHA = 1 时

  SCK 引脚在空闲状态处于低电平，SCK 引脚的第 2 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 0， CPHA = 1 时

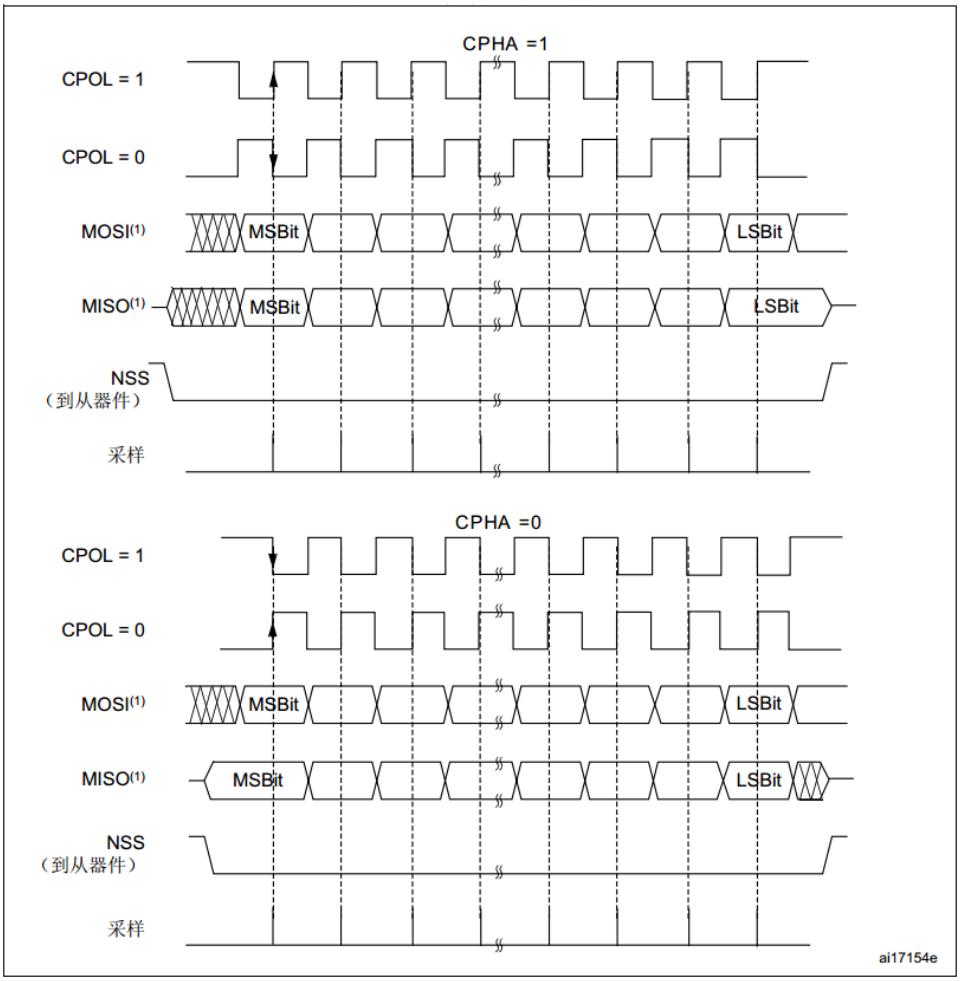
  SCK 引脚在空闲状态处于高电平，SCK 引脚的第 2 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 1， CPHA = 0 时

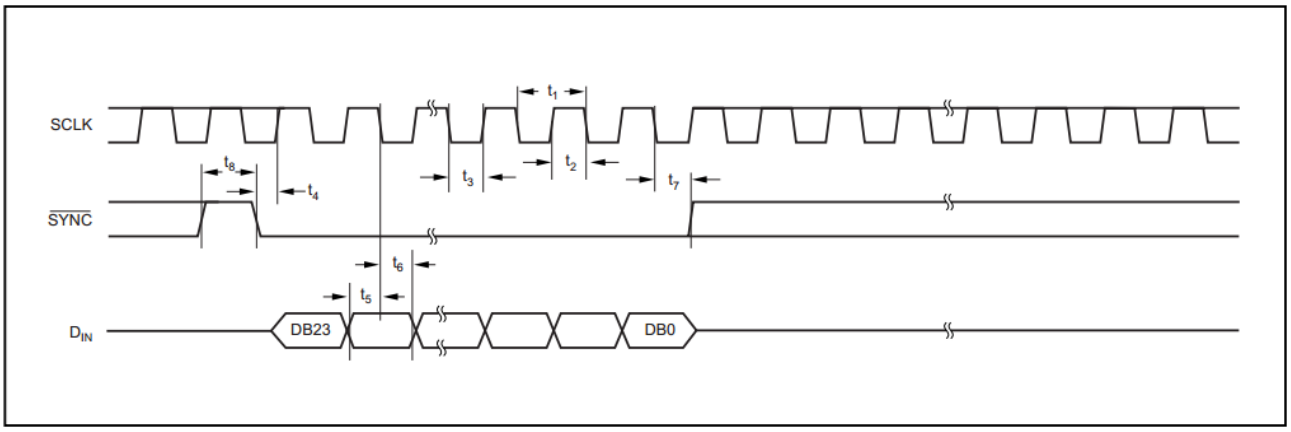
  SCK 引脚在空闲状态处于低电平，SCK 引脚的第 1 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 0 ，CPHA= 0 时

  SCK 引脚在空闲状态处于高电平，SCK 引脚的第 1 个边沿捕获传输的第 1 个数据。



有了 H7 支持的时序配置，再来看下 DAC8501 的时序图：



首先 DAC8501 是下降升沿做数据采集，所以 STM32H7 的可选的配置就是：

CHOL = 0， CPHA = 1

CHOL = 1， CPHA = 0

对于这两种情况的主要区别是空闲状态下 SCLK 时钟选择高电平还是低电平，根据上面的时序图和 DAC8501 的数据手册，两种情况下都可以正常运行。经过实际测试，STM32H7 使用这两个配置确实都可以正常运行。程序里面默认是选择 CHOL = 0， CPHA = 1。

75.5.4第 4 步：单 SPI 接口管理多个 SPI 设备的切换机制

单 SPI 接口管理多个 SPI 设备最麻烦的地方是不同设备的时钟分配，时钟极性和时钟相位并不相同。对此的解决解决办法是在片选阶段配置切换，比如 DAC8501 的片选：

```
/*
*********************************************************************************************************
*    函 数 名: DAC8501_SetCS1
*    功能说明: DAC8501 片选控制函数
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DAC8501_SetCS1(uint8_t _Level)
{
    if (_Level == 0)
    {
        bsp_SpiBusEnter(); /* 占用 SPI 总线  */
        bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_8, SPI_PHASE_2EDGE, SPI_POLARITY_LOW);
        CS1_0();
    }
    else
    {
        CS1_1();
        bsp_SpiBusExit(); /* 释放 SPI 总线 */
    }
}

/*
*********************************************************************************************************
*    函 数 名: DAC8501_SetCS2(0)
*    功能说明: 设置 CS2。 用于运行中 SPI 共享。
*    形    参: 无
    返 回 值: 无
*********************************************************************************************************
*/
void DAC8501_SetCS2(uint8_t _level)
{
    if (_level == 0)
    {
        bsp_SpiBusEnter(); /* 占用 SPI 总线  */
        bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_8, SPI_PHASE_2EDGE, SPI_POLARITY_LOW);
        CS2_0();
    }
    else
    {
        CS2_1();
        bsp_SpiBusExit(); /* 释放 SPI 总线 */
    }
}
```

通过这种方式就有效的解决了单 SPI 接口管理多设备的问题。因为给每个设备都配了一个独立的片选引脚，这样就可以为每个设备都配置这么一个片选配置。

但是频繁配置也比较繁琐，所以函数 bsp_InitSPIParam 里面做了特别处理。当前配置与之前配置相

同的情况下无需重复配置。

75.5.5第 5 步：DAC8501 的数据更新

DAC8501 的双通道数据更新通过下面的函数实现：

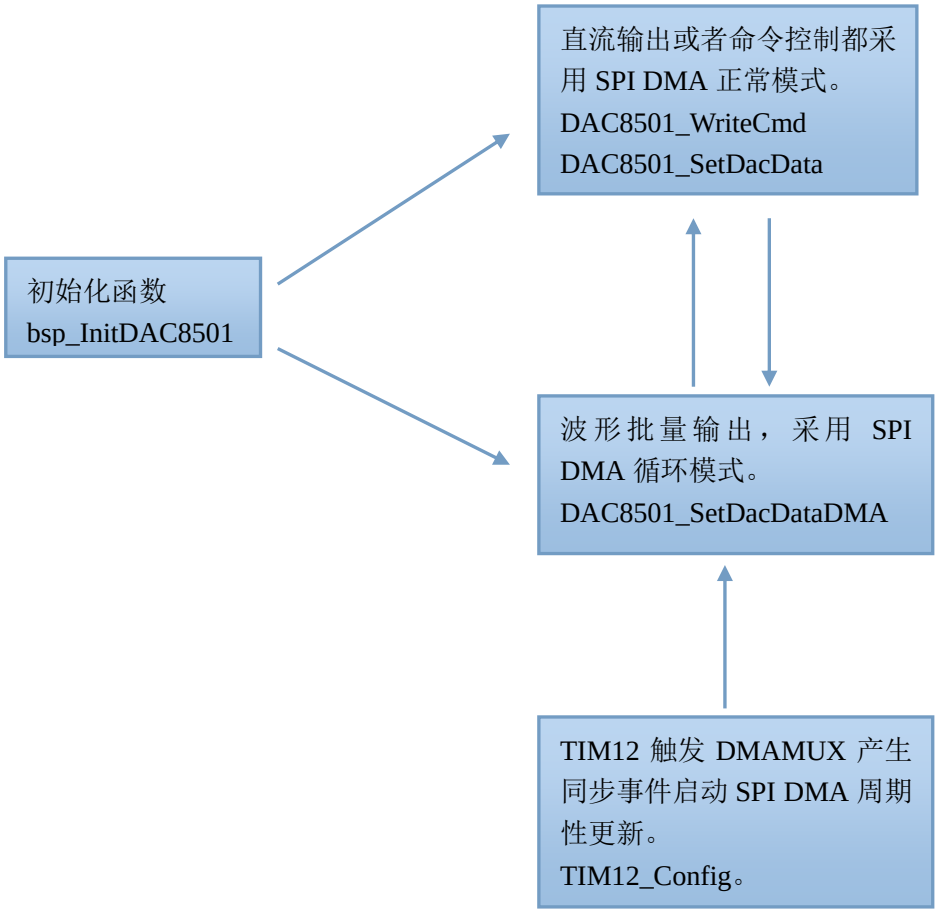
```
/*
*********************************************************************************************************
*    函 数 名: DAC8501_SetDacData
*    功能说明: 设置 DAC 数据
*    形    参: _ch, 通道,
*              _data : 数据
*    返 回 值: 无
*********************************************************************************************************
*/
void DAC8501_SetDacData(uint8_t _ch, uint16_t _dac)
{
    uint32_t data;

    /*
        DAC8501.pdf page 12 有 24bit 定义

        DB24:18 = xxxxx 保留
        DB17:  PD1
        DB16:  PD0

        DB15：0  16 位数据

        其中 PD1 PD0 决定 4 种工作模式
              0   0  ---> 正常工作模式
              0   1  ---> 输出接 1K 欧到 GND
              1   0  ---> 输出 100K 欧到 GND
              1   1  ---> 输出高阻
    */

    data = _dac; /* PD1 PD0 = 00 正常模式 */

    if (_ch == 0)
    {
        DAC8501_SetCS1(0);
    }
    else
    {
        DAC8501_SetCS2(0);
    }

    /*  DAC8501 SCLK 时钟高达 30M，因此可以不延迟 */
    g_spiLen = 0;
    g_spiTxBuf[g_spiLen++] = (data >> 16);
    g_spiTxBuf[g_spiLen++] = (data >> 8);
    g_spiTxBuf[g_spiLen++] = (data);
    bsp_spiTransfer();

    if (_ch == 0)
    {
```

```
        DAC8501_SetCS1(1);
    }
    else
    {
        DAC8501_SetCS2(1);
    }
}
```

函数实现比较简单，每次更新发送 24bit 数据即可。

75.6 DAC8501 驱动设计（SPI DMA 更新方式）

DAC8501 的 DMA 驱动方式略复杂，跟中断更新方式完全不同，要使用硬件 SPI1 NSS 引脚驱动 DAC8501 的片选，所有专门做了一个驱动文件来实现，程序驱动框架设计如下：



有了这个框图，程序设计就比较好理解了。

75.6.1 第 1 步：SPI 总线配置

spi 总线配置通过如下两个函数实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitDAC8501
```

```
*    功能说明: 配置 GPIO 并初始化 DAC8501 寄存器
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitDAC8501(void)
{
    s_SpiDmaMode = 0;

    /*##-1- 配置 SPI DMA ###################################################################*/
    bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_8, SPI_PHASE_2EDGE, SPI_POLARITY_LOW);

    /*##-2- 默认输出 0V ####################################################################*/
    DAC8501_SetDacData(0, 0);   /* CH1 输出 0 */
}

/*
*********************************************************************************************************
*    函 数 名: bsp_InitSPIParam
*    功能说明: 配置 SPI 总线参数，时钟分频，时钟相位和时钟极性。
*    形    参: _BaudRatePrescaler  SPI 总线时钟分频设置，支持的参数如下:
*                                 SPI_BAUDRATEPRESCALER_2    2 分频
*                                 SPI_BAUDRATEPRESCALER_4    4 分频
*                                 SPI_BAUDRATEPRESCALER_8    8 分频
*                                 SPI_BAUDRATEPRESCALER_16   16 分频
*                                 SPI_BAUDRATEPRESCALER_32   32 分频
*                                 SPI_BAUDRATEPRESCALER_64   64 分频
*                                 SPI_BAUDRATEPRESCALER_128  128 分频
*                                 SPI_BAUDRATEPRESCALER_256  256 分频
*
*              _CLKPhase          时钟相位，支持的参数如下:
*                                 SPI_PHASE_1EDGE     SCK 引脚的第 1 个边沿捕获传输的第 1 个数据
*                                 SPI_PHASE_2EDGE     SCK 引脚的第 2 个边沿捕获传输的第 1 个数据
*
*              _CLKPolarity       时钟极性，支持的参数如下:
*                                 SPI_POLARITY_LOW    SCK 引脚在空闲状态处于低电平
*                                 SPI_POLARITY_HIGH   SCK 引脚在空闲状态处于高电平
*
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitSPIParam(uint32_t _BaudRatePrescaler, uint32_t _CLKPhase, uint32_t _CLKPolarity)
{

    /* 设置 SPI 参数 */
    hspi.Instance               = SPIx;                         /* 例化 SPI */
    hspi.Init.BaudRatePrescaler = _BaudRatePrescaler;           /* 设置波特率 */
    hspi.Init.Direction         = SPI_DIRECTION_2LINES_TXONLY;  /* 全双工 */
    hspi.Init.CLKPhase          = _CLKPhase;                    /* 配置时钟相位 */
    hspi.Init.CLKPolarity       = _CLKPolarity;                 /* 配置时钟极性 */
    hspi.Init.DataSize          = SPI_DATASIZE_24BIT;           /* 设置数据宽度 */
    hspi.Init.FirstBit          = SPI_FIRSTBIT_MSB;             /* 数据传输先传高位 */
    hspi.Init.TIMode            = SPI_TIMODE_DISABLE;           /* 禁止 TI 模式  */
    hspi.Init.CRCCalculation    = SPI_CRCCALCULATION_DISABLE;   /* 禁止 CRC */
    hspi.Init.CRCPolynomial     = 7;                            /* 禁止 CRC 后，此位无效 */
    hspi.Init.CRCLength         = SPI_CRC_LENGTH_8BIT;          /* 禁止 CRC 后，此位无效 */
    hspi.Init.FifoThreshold     = SPI_FIFO_THRESHOLD_05DATA;    /* 设置 FIFO 大小是一个数据项 */
```

```
    hspi.Init.NSS          = SPI_NSS_HARD_OUTPUT;                /* 使用软件方式管理片选引脚 */
    hspi.Init.NSSPMode     = SPI_NSS_PULSE_ENABLE;               /* 使能脉冲输出 */
    hspi.Init.NSSPolarity = SPI_NSS_POLARITY_LOW;               /* 低电平有效 */

   /* MSS, 插入到 NSS 有效边沿和第一个数据开始之间的额外延迟，单位 SPI 时钟周期个数 */
    hspi.Init.MasterSSIdleness        = SPI_MASTER_SS_IDLENESS_00CYCLE;

    /* MIDI, 两个连续数据帧之间插入的最小时间延迟，单位 SPI 时钟周期个数 */
    hspi.Init.MasterInterDataIdleness = SPI_MASTER_INTERDATA_IDLENESS_02CYCLE;

    hspi.Init.MasterKeepIOState = SPI_MASTER_KEEP_IO_STATE_ENABLE; /* 禁止 SPI 后，SPI 相关引脚保持当前状态 */
    hspi.Init.Mode             = SPI_MODE_MASTER;                 /* SPI 工作在主控模式 */

    /* 复位配置 */
    if (HAL_SPI_DeInit(&hspi) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 初始化配置 */
    if (HAL_SPI_Init(&hspi) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

这两个配置函数里面最重要的是置红的几个配置选项，这里依次为大家做个说明：

- SPI_DIRECTION_2LINES_TXONLY

  驱动 DAC856X 仅需要 SPI 写操作。

- SPI_DATASIZE_24BIT

  STM32H7 的 SPI 支持 4-32bit 数据传输，由于 DAC856X 需要 24bit 数据，所以这里配置为 24 即可。

- SPI_FIFO_THRESHOLD_05DATA

  对于 SPI1 来说，里面的 FIFO 大小是 16 字节，那么 SPI 数据传输配置为 24bit 的话，FIFO 最多可以存储 5 个 24bit，因此这个 fifo 阀值要设置为 5。

- SPI_NSS_HARD_OUTPUT

  我们这里要使用 SPI 的硬件片选引脚 SPI_NSS。

- SPI_MASTER_SS_IDLENESS_00CYCLE

  插入到 NSS 有效边沿和第一个数据开始之间的额外延迟，单位 SPI 时钟周期个数。

  根据本章 75.4.4 小节里面的 $t_{(4)}$要求，片选有效到 SCLK 第 1 个下降沿信号的时间，最小值为 0。所以这里配置为 0 即可，也就是无需插入时间。

- SPI_MASTER_INTERDATA_IDLENESS_10CYCLE

  两个连续数据帧之间插入的最小时间延迟，单位 SPI 时钟周期个数。

  根据本章 75.4.4 小节里面的 $t_{(5)}$要求，每传输 24bit 数据后，片选要保持一段时间的高电平，DAC856X 要求至少要 33ns（供电 3.6 到 5.5V 时），也是说，如果我们以 25MHz 驱动 DAC856X，这里至少

要配置为 1 个时钟周期，推荐值为 2 及其以上即可，我们这里直接配置为 2 个时钟周期（配置为 1 也没问题的）。

75.6.2第 2 步：TIM12 周期性触发配置

这里特别注意一点，定时器触发一次，就会让 SPI 以 DMA 方式传输 24bit 输出。

TIM12 的触发配置如下：

```
/*
*********************************************************************************************************
*    函 数 名: TIM12_Config
*    功能说明: 配置 TIM12，用于触发 DMAMUX 的请求发生器
*    形    参: _ulFreq  触发频率，推荐范围 100Hz - 1MHz
*    返 回 值: 无
*********************************************************************************************************
*/
TIM_HandleTypeDef  htim ={0};
TIM_MasterConfigTypeDef sMasterConfig = {0};
TIM_OC_InitTypeDef sConfig = {0};
void TIM12_Config(uint32_t _ulFreq)
{
    uint16_t usPeriod;
    uint16_t usPrescaler;
    uint32_t uiTIMxCLK;


    /* 使能时钟 */
    __HAL_RCC_TIM12_CLK_ENABLE();

    /*-----------------------------------------------------------------------
        bsp.c 文件中 void SystemClock_Config(void) 函数对时钟的配置如下:

       System Clock source       = PLL (HSE)
       SYSCLK(Hz)                = 400000000 (CPU Clock)
       HCLK(Hz)                  = 200000000 (AXI and AHBs Clock)
       AHB Prescaler             = 2
       D1 APB3 Prescaler         = 2 (APB3 Clock  100MHz)
       D2 APB1 Prescaler         = 2 (APB1 Clock  100MHz)
       D2 APB2 Prescaler         = 2 (APB2 Clock  100MHz)
       D3 APB4 Prescaler         = 2 (APB4 Clock  100MHz)

       因为 APB1 prescaler != 1, 所以 APB1 上的 TIMxCLK = APB1 x 2 = 200MHz; 不含这个总线下的 LPTIM1
       因为 APB2 prescaler != 1, 所以 APB2 上的 TIMxCLK = APB2 x 2 = 200MHz;
       APB4 上面的 TIMxCLK 没有分频，所以就是 100MHz;

       APB1 定时器有 TIM2, TIM3 ,TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14，LPTIM1
       APB2 定时器有 TIM1, TIM8 , TIM15, TIM16，TIM17

       APB4 定时器有 LPTIM2，LPTIM3，LPTIM4，LPTIM5
    ----------------------------------------------------------------------- */
    uiTIMxCLK = SystemCoreClock / 2;

    if (_ulFreq < 100)
    {
```

```
        usPrescaler = 10000 - 1;                        /* 分频比 = 10000 */
        usPeriod =  (uiTIMxCLK / 10000) / _ulFreq  - 1; /* 自动重装的值 */
    }
    else if (_ulFreq < 3000)
    {
        usPrescaler = 100 - 1;                    /* 分频比 = 100 */
        usPeriod =  (uiTIMxCLK / 100) / _ulFreq  - 1;/* 自动重装的值 */
    }
    else /* 大于 4K 的频率，无需分频 */
    {
        usPrescaler = 0;                      /* 分频比 = 1 */
        usPeriod = uiTIMxCLK / _ulFreq - 1;  /* 自动重装的值 */
    }

   htim.Instance = TIM12;
    htim.Init.Period            = usPeriod;
    htim.Init.Prescaler         = usPrescaler;
    htim.Init.ClockDivision     = 0;
    htim.Init.CounterMode       = TIM_COUNTERMODE_UP;
    htim.Init.RepetitionCounter = 0;

    if(HAL_TIM_Base_DeInit(&htim) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    if(HAL_TIM_Base_Init(&htim) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

   sConfig.OCMode     = TIM_OCMODE_PWM1;
   sConfig.OCPolarity = TIM_OCPOLARITY_LOW;
   sConfig.Pulse = usPeriod / 2;     /* 占空比 50% */
   if(HAL_TIM_OC_ConfigChannel(&htim, &sConfig, TIM_CHANNEL_1) != HAL_OK)
   {
        Error_Handler(__FILE__, __LINE__);
   }

   /* 启动 OC1 */
   if(HAL_TIM_OC_Start(&htim, TIM_CHANNEL_1) != HAL_OK)
   {
        Error_Handler(__FILE__, __LINE__);
   }

   /* TIM12 的 TRGO 用于触发 DMAMUX 的请求发生器 */
    sMasterConfig.MasterOutputTrigger = TIM_TRGO_OC1REF;
   sMasterConfig.MasterOutputTrigger2 = TIM_TRGO2_RESET;
    sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;

    HAL_TIMEx_MasterConfigSynchronization(&htim, &sMasterConfig);
}
#endif
```

这个函数支持的触发频率很宽，对于 DAC856X 来说，如果样本点设置为 100 个的话，此函数推荐的触发频率是 100Hz 到 1MHz，具体可以支持到最高触发速度计算看本章 4.7.7 小节即可。

75.6.3第 3 步：DMAMUX 同步触发 SPI DMA 传输

DMA 和 DMAMUX 的配置如下：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_spiDamStart
*    功能说明: 启动 SPI DMA 传输
*    形    参: _ulFreq 范围推荐 100Hz-1MHz
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_spiDamStart(uint32_t _ulFreq)
{
    /* 设置模式，要切换到 DMA CIRCULAR 模式 */
    s_SpiDmaMode = 1;

    bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_4, SPI_PHASE_2EDGE, SPI_POLARITY_LOW);

    /* 使能 DMA 时钟 */
    DMAx_CLK_ENABLE();

    /* SPI DMA 发送配置 */
    hdma_tx.Instance                 = SPIx_TX_DMA_STREAM;      /* 例化使用的 DMA 数据流 */
    hdma_tx.Init.FIFOMode            = DMA_FIFOMODE_ENABLE;     /* 使能 FIFO */
    hdma_tx.Init.FIFOThreshold       = DMA_FIFO_THRESHOLD_FULL; /* 用于设置阀值 */
    hdma_tx.Init.MemBurst            = DMA_MBURST_SINGLE;       /* 用于存储器突发 */
    hdma_tx.Init.PeriphBurst         = DMA_PBURST_SINGLE;       /* 用于外设突发 */
    hdma_tx.Init.Request             = SPIx_TX_DMA_REQUEST;     /* 请求类型 */
    hdma_tx.Init.Direction           = DMA_MEMORY_TO_PERIPH;    /* 传输方向是从存储器到外设 */
    hdma_tx.Init.PeriphInc           = DMA_PINC_DISABLE;        /* 外设地址自增禁止 */
    hdma_tx.Init.MemInc              = DMA_MINC_ENABLE;         /* 存储器地址自增使能 */
    hdma_tx.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;     /* 外设数据传输位宽选择字节，即 8bit */
    hdma_tx.Init.MemDataAlignment    = DMA_MDATAALIGN_WORD;     /* 存储器数据传输位宽选择字节，即 8bit */
    hdma_tx.Init.Mode                = DMA_CIRCULAR;            /* 正常模式 */
    hdma_tx.Init.Priority            = DMA_PRIORITY_LOW;        /* 优先级低 */

     /* 复位 DMA */
    if(HAL_DMA_DeInit(&hdma_tx) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

     /* 初始化 DMA */
    if(HAL_DMA_Init(&hdma_tx) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 关联 DMA 句柄到 SPI */
    __HAL_LINKDMA(&hspi, hdmatx, hdma_tx);


    /* 关闭 DMA 发送中断 */
    HAL_NVIC_SetPriority(SPIx_DMA_TX_IRQn, 1, 0);
    HAL_NVIC_DisableIRQ(SPIx_DMA_TX_IRQn);
```

```
    /* 关闭 SPI 中断 */
    HAL_NVIC_SetPriority(SPIx_IRQn, 1, 0);
    HAL_NVIC_DisableIRQ(SPIx_IRQn);

    /* 同步触发配置 */
    dmamux_syncParams.EventEnable   = ENABLE;
    dmamux_syncParams.SyncPolarity  = HAL_DMAMUX_SYNC_RISING;
    dmamux_syncParams.RequestNumber = 1;
    dmamux_syncParams.SyncSignalID  = HAL_DMAMUX1_SYNC_TIM12_TRGO;
    dmamux_syncParams.SyncEnable    = ENABLE;

    HAL_DMAEx_ConfigMuxSync(&hdma_tx, &dmamux_syncParams);

    //LPTIM_Config(_ulFreq);

    TIM12_Config(_ulFreq);

    /* 启动 DMA 传输 */
    if(HAL_SPI_Transmit_DMA(&hspi, (uint8_t*)g_spiTxBuf, g_spiLen/4)!= HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

这段程序里面最关键的就是置红的部分。作用是配置 DMAMUX 的同步触发功能，触发周期由 TIM12 控制。

75.6.4第 4 步：24bit 数据的 DMA 传输解决办法

由于通用 DMA1 和 DMA2 仅支持 8bit，16bit 和 32bit 数据传输，我们这里要传输 24bit 数据，解决的关键就是配置 DMA 为传输宽度为 32bit,并将传输的数据由 24bit 再补一个 8bit 的任意值组成 32bit 即可，实际的传输会由 SPI 完成。

```
/*
*********************************************************************************************************
*    函 数 名: DAC8501_SetDacDataDMA
*    功能说明: DAC8501 数据发送，DMA 方式
*    形    参: _ch          1 表示通道 1 输出
*              _pbufch1     通道 1 数据缓冲地址
*              _sizech1     通道 1 数据大小
*              _ulFreq      触发频率，推荐范围 100Hz- 1MHz，注意这个参数是触发频率，并不是波形周期。
*                           这里触发一次，SPI DMA 传输一次 24bit 数据。
*    返 回 值: 无
*********************************************************************************************************
*/
void DAC8501_SetDacDataDMA(uint8_t _ch, uint16_t *_pbufch1, uint32_t _sizech1, uint32_t _ulFreq)
{
    uint32_t i;
    uint32_t _cmd;

    g_spiLen = 0;
```

```
    switch (_ch)
    {

        /*
            DAC8501.pdf page 12 有 24bit 定义

            DB24:18 = xxxxx 保留
            DB17:  PD1
            DB16:  PD0

            DB15: 0  16 位数据

            其中 PD1 PD0 决定 4 种工作模式
                  0   0  ---> 正常工作模式
                  0   1  ---> 输出接 1K 欧到 GND
                  1   0  ---> 输出 100K 欧到 GND
                  1   1  ---> 输出高阻
        */

        /* 通道 1 数据发送 */
        case 1:
            for(i = 0; i < _sizech1; i++)
            {
                /* 更新需要配置 PD1 和 PD0，当前是选择的正常工作模式 */
                _cmd = (0 << 16) | (_pbufch1[i] << 0);

                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 8);
                g_spiTxBuf[g_spiLen++] = (uint8_t)(_cmd >> 16);
                g_spiTxBuf[g_spiLen++] = 0;
            }
            break;

        default:
            break;

    }

    bsp_spiDamStart(_ulFreq);
}
```

75.6.5第 5 步：DMA 缓冲区的 MPU 配置

因为工程是用的 DTCM 做的主 RAM 空间，这个空间无法使用通用 DMA1 和 DMA2，通过本手册第 26 章的内存块超方便使用方式，将 DMA 缓冲定义到 SRAM4 上：

```
#if defined ( __CC_ARM )    /* MDK *******/
    __attribute__((section (".RAM_D3"))) uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
    __attribute__((section (".RAM_D3"))) uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];
#elif defined (__ICCARM__)  /* IAR ********/
    #pragma location = ".RAM_D3"
    uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
    #pragma location = ".RAM_D3"
    uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];
```

```
#endif
```

由于程序里面开启了数据 Cache，会造成 DMA 和 CPU 访问 SRAM4 数据不一致的问题，特此将 SRAM4 空间关闭 Cache。

```
    /* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

75.6.6第 6 步：DAC8501 的时钟极性和时钟相位配置

注：与本章 74.5.3 小节内容是一样的。

首先回忆下 STM32H7 支持的 4 种时序配置。

- 当 CPOL = 1， CPHA = 1 时

  SCK 引脚在空闲状态处于低电平，SCK 引脚的第 2 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 0， CPHA = 1 时

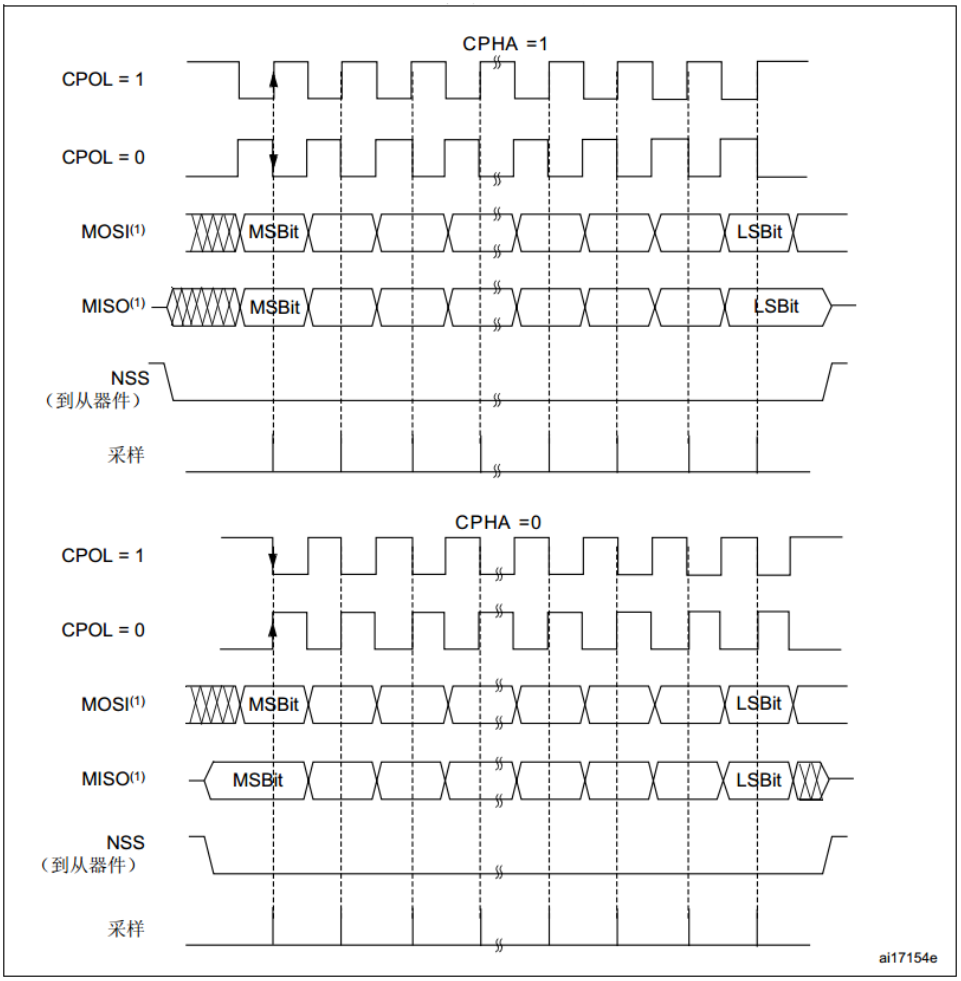
  SCK 引脚在空闲状态处于高电平，SCK 引脚的第 2 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 1， CPHA = 0 时

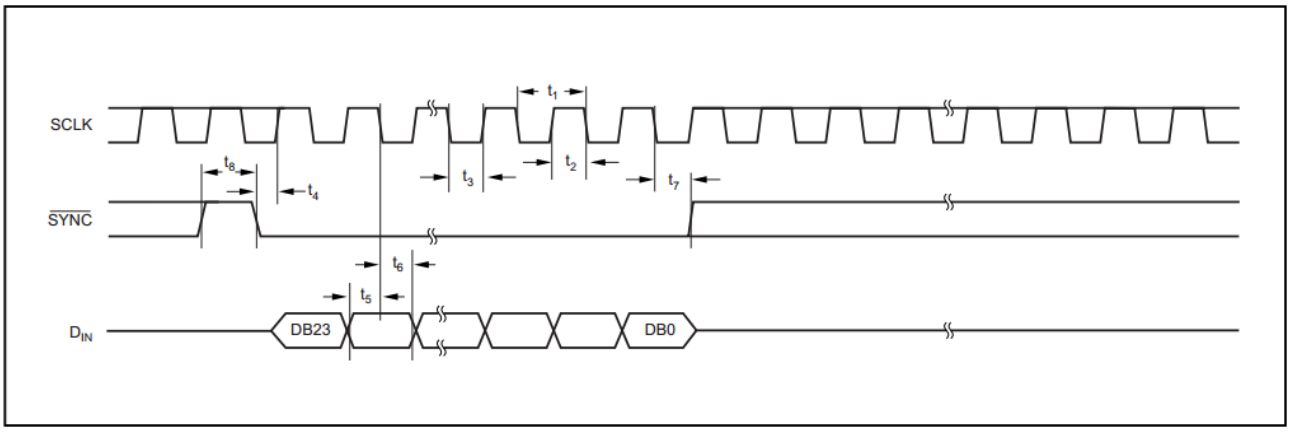
  SCK 引脚在空闲状态处于低电平，SCK 引脚的第 1 个边沿捕获传输的第 1 个数据。

- 当 CPOL = 0 ，CPHA= 0 时

  SCK 引脚在空闲状态处于高电平，SCK 引脚的第 1 个边沿捕获传输的第 1 个数据。



有了 H7 支持的时序配置，再来看下 DAC8501 的时序图：



首先 DAC8501 是下降升沿做数据采集，所以 STM32H7 的可选的配置就是：

CHOL = 0， CPHA = 1

CHOL = 1， CPHA = 0

对于这两种情况的主要区别是空闲状态下 SCLK 时钟选择高电平还是低电平，根据上面的时序图和 DAC8501 的数据手册，两种情况下都可以正常运行。经过实际测试，STM32H7 使用这两个配置确实都可以正常运行。程序里面默认是选择 CHOL = 0， CPHA = 1。

75.6.7第 7 步：DAC8501 的最高更新速度计算

这里特别注意一点，定时器触发一次，就会让 SPI 以 DMA 方式传输 24bit 数据。

配置条件：

- SPI 时钟是 25MHz，SPI 数据传为 24bit，每个 bit 需要时间 40ns。
- hspi.Init.MasterSSIdleness = SPI_MASTER_SS_IDLENESS_00CYCLE
  插入到 NSS 有效边沿和第一个数据开始之间的额外延迟，单位 SPI 时钟周期个数，即 40ns。
- hspi.Init.MasterInterDataIdleness = SPI_MASTER_INTERDATA_IDLENESS_10CYCLE
  两个连续数据帧之间插入的最小时间延迟，单位 SPI 时钟周期个数，即 40ns。

根据上面的配置，传输一帧（24bit）数据需要的时间：

24bit * 20ns+ SPI_MASTER_SS_IDLENESS_00CYCLE * 20ns
+ SPI_MASTER_INTERDATA_IDLENESS_02CYCLE * 20ns
= 24bit * 40ns + 0 * 40ns + 2 * 40ns
= 1040ns。

那么这种配置下，可以支持最高触发速度是 1 / 1040ns = 0.961MHz，如果想速度再提升些，可以降低参数 hspi.Init.MasterInterDataIdleness，推荐的最小值是 1 个时钟周期，那么可以支持的最高触发速度是 1/1000ns = 1MHz。

认识到这些后，实际输出的波形周期也比较好算了，比如我们设置 10 个样本点为一个周期，那么触发速度为 1MHz 的时候，那么波形周期就是 100KHz。

75.6.8第 8 步：DAC 值和电压值互转

DAC8501 模块的输出电压范围是 0V 到 5V,对应的编码值范围是 0 到 65535,为了方便大家做互转，专门做了两个函数：

```
/*
*********************************************************************************************************
*    函 数 名: DAC8501_DacToVoltage
*    功能说明: 将 DAC 值换算为电压值，单位 0.1mV
*    形    参: _dac  16 位 DAC 字
*    返 回 值: 电压，单位 0.1mV
*********************************************************************************************************
*/
int32_t DAC8501_DacToVoltage(uint16_t _dac)
{
    int32_t y;
```

```
    /* CaculTwoPoint(int32_t x1, int32_t y1, int32_t x2, int32_t y2, int32_t x);*/
    y =  CaculTwoPoint(X1, Y1, X2, Y2, _dac);
    if (y < 0)
    {
        y = 0;
    }
    return y;
}

/*
*********************************************************************************************************
*    函 数 名: DAC8501_DacToVoltage
*    功能说明: 将 DAC 值换算为电压值，单位 0.1mV
*    形    参: _volt 电压，单位 0.1mV
*    返 回 值: 16 位 DAC 字
*********************************************************************************************************
*/
uint32_t DAC8501_VoltageToDac(int32_t _volt)
{
    /* CaculTwoPoint(int32_t x1, int32_t y1, int32_t x2, int32_t y2, int32_t x);*/
    return CaculTwoPoint(Y1, X1, Y2, X2, _volt);
}
```

75.7 SPI 总线板级支持包（bsp_spi_bus.c)

SPI 总线驱动文件 bsp_spi_bus.c 主要实现了如下几个 API 供用户调用：

- bsp_InitSPIBus
- bsp_InitSPIParam
- bsp_spiTransfer

75.7.1 函数 bsp_InitSPIBus

函数原型：

```
void bsp_InitSPIBus(void)
```

函数描述：

此函数主要用于 SPI 总线的初始化，在 bsp.c 文件调用一次即可。

75.7.2 函数 bsp_InitSPIParam

函数原型：

```
void bsp_InitSPIParam(uint32_t _BaudRatePrescaler, uint32_t _CLKPhase, uint32_t _CLKPolarity)
```

函数描述：

此函数用于 SPI 总线的配置。

函数参数：

- 第 1 个参数 SPI 总线的分频设置，支持的参数如下：

SPI_BAUDRATEPRESCALER_2 2 分频
SPI_BAUDRATEPRESCALER_4 4 分频
SPI_BAUDRATEPRESCALER_8 8 分频
SPI_BAUDRATEPRESCALER_16 16 分频
SPI_BAUDRATEPRESCALER_32 32 分频
SPI_BAUDRATEPRESCALER_64 64 分频
SPI_BAUDRATEPRESCALER_128 128 分频
SPI_BAUDRATEPRESCALER_256 256 分频

- 第 2 个参数用于时钟相位配置，支持的参数如下：
  SPI_PHASE_1EDGE SCK 引脚的第 1 个边沿捕获传输的第 1 个数据
  SPI_PHASE_2EDGE SCK 引脚的第 2 个边沿捕获传输的第 1 个数据
- 第 3 个参数是时钟极性配置，支持的参数如下：
  SPI_POLARITY_LOW SCK 引脚在空闲状态处于低电平
  SPI_POLARITY_HIGH SCK 引脚在空闲状态处于高电平

75.7.3 函数 bsp_spiTransfer

函数原型：

```
void bsp_spiTransfer(void)
```

函数描述：

此函数用于启动 SPI 数据传输，支持查询，中断和 DMA 方式传输。

75.8 DAC8501 支持包中断方式（bsp_spi_dac8501.c）

DAC8501 驱动文件 bsp_spi_dac8501.c 主要实现了如下几个 API 供用户调用：

- bsp_InitDAC8501
- DAC8501_SetCS1
- DAC8501_SetCS2
- DAC8501_SetDacData
- DAC8501_DacToVoltage
- DAC8501_VoltageToDac

75.8.1 函数 bsp_InitDAC8501

函数原型：

```
void bsp_InitDAC8501(void)
```

函数描述：

主要用于 DAC8501 的初始化，调用前务必先调用函数 bsp_InitSPIBus 初始化 SPI 外设。

75.8.2 函数 DAC8501_SetCS1

函数原型：

```
void DAC8501_SetCS1(uint8_t _Level)
```

函数描述：

此函数用于片选 DAC8501 模块上的第 1 片 8501。

函数参数：

- 第 1 个参数为 0 表示选中，为 1 表示取消选中。

75.8.3 函数 DAC8501_SetCS2

函数原型：

```
void DAC8501_SetCS2(uint8_t _Level)
```

函数描述：

此函数用于片选 DAC8501 模块上的第 2 片 8501。

函数参数：

- 第 1 个参数为 0 表示选中，为 1 表示取消选中

75.8.4 函数 DAC8501_SetDacData

函数原型：

```
void DAC8501_SetDacData(uint8_t _ch, uint16_t _dac)
```

函数描述：

此函数用于设置 DAC 输出，并立即更新。

函数参数：

- 第 1 个参数为 0 表示通道 1，为 1 表示通道 2。
- 第 2 个参数是 DAC 数值设置，范围 0 到 65535，0 对应最小电压值，65535 对应最大电压值。

75.8.5 函数 DAC8501_DacToVoltage

函数原型：

```
int32_t DAC8501_DacToVoltage(uint16_t _dac)
```

函数描述：

此函数用于将 DAC 值换算为电压值，单位 0.1mV。

函数参数：

- 第 1 个参数 DAC 数值，范围 0 到 65535。
- 返回值，返回电压值，单位 0.1mV。

75.8.6 函数 DAC8501_VoltageToDac

函数原型：

```
uint32_t DAC8501_VoltageToDac(int32_t _volt)
```

函数描述：

此函数用于将电压值转换为 DAC 值。

函数参数：

- 第 1 个参数是电压值，范围 0 到 50000，单位 0.1mV。
- 返回值，返回 DAC 值。

75.9 DAC8501 支持包 DMA 方式（bsp_spidma_dac8501.c）

DAC8501 驱动文件 bsp_spidma_dac8501.c 涉及到的函数比较多，我们主要介绍用到的如下几个函数：

- bsp_InitDAC8501
- DAC8501_SetDacDataDMA
- DAC8501_SetDacData

75.9.1 函数 bsp_InitDAC8501

函数原型：

```
void bsp_InitDAC8501(void)
```

函数描述：

主要用于 DAC8501 的初始化。

75.9.2 函数 DAC8501_SetDacDataDMA

函数原型：

```
void DAC8501_SetDacDataDMA(uint8_t _ch, uint16_t *_pbufch1, uint32_t _sizech1, uint32_t _ulFreq)
```

函数描述：

此函数用于 SPI DMA 方式数据发送。

函数参数：

- 第 1 个参数用于选择的通道：
  - 1 表示通道 1 输出
- 第 2 个参数表示通道 1 数据缓冲地址。
- 第 3 个参数表示通道 1 数据大小。
- 第 4 个参数表示触发频率，推荐范围 100Hz- 1MHz，注意这个参数是触发频率，并不是波形周期。这里触发一次，SPI DMA 传输一次 24bit 数据。

75.9.3 函数 DAC8501_SetDacData

函数原型：

```
void DAC8501_SetDacData(uint8_t _ch, uint16_t _dac)
```

函数描述：

此函数用于设置 DAC 输出，并立即更新。

函数参数：

- 第 1 个参数为 0 表示通道 1，为 1 表示通道 2（对于 SPI DMA 方式，仅支持通道 1）。
- 第 2 个参数是 DAC 数值设置，范围 0 到 65535，0 对应最小电压值，65535 对应最大电压值。

75.10 DAC8501 驱动移植和使用（中断更新方式）

DAC8501 移植步骤如下：

- 第 1 步：复制 bsp_spi_bus.c，bsp_spi_bus.h，bsp_spi_dac8501.c，bsp_spi_dac8501.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：根据使用的第几个 SPI，SPI 时钟，SPI 引脚和 DMA 通道等，修改 bsp_spi_bus.c 文件开头的宏定义

```
/*
*********************************************************************************************************
*                                    时钟，引脚，DMA，中断等宏定义
*********************************************************************************************************
*/
#define SPIx                             SPI1
#define SPIx_CLK_ENABLE()                __HAL_RCC_SPI1_CLK_ENABLE()
#define DMAx_CLK_ENABLE()                __HAL_RCC_DMA2_CLK_ENABLE()

#define SPIx_FORCE_RESET()               __HAL_RCC_SPI1_FORCE_RESET()
#define SPIx_RELEASE_RESET()             __HAL_RCC_SPI1_RELEASE_RESET()

#define SPIx_SCK_CLK_ENABLE()            __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_SCK_GPIO                    GPIOB
#define SPIx_SCK_PIN                     GPIO_PIN_3
#define SPIx_SCK_AF                          GPIO_AF5_SPI1

#define SPIx_MISO_CLK_ENABLE()           __HAL_RCC_GPIOB_CLK_ENABLE()
```

```
#define SPIx_MISO_GPIO                  GPIOB
#define SPIx_MISO_PIN                   GPIO_PIN_4
#define SPIx_MISO_AF                    GPIO_AF5_SPI1

#define SPIx_MOSI_CLK_ENABLE()          __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MOSI_GPIO                  GPIOB
#define SPIx_MOSI_PIN                   GPIO_PIN_5
#define SPIx_MOSI_AF                    GPIO_AF5_SPI1

#define SPIx_TX_DMA_STREAM              DMA2_Stream3
#define SPIx_RX_DMA_STREAM              DMA2_Stream2

#define SPIx_TX_DMA_REQUEST             DMA_REQUEST_SPI1_TX
#define SPIx_RX_DMA_REQUEST             DMA_REQUEST_SPI1_RX

#define SPIx_DMA_TX_IRQn                DMA2_Stream3_IRQn
#define SPIx_DMA_RX_IRQn                DMA2_Stream2_IRQn

#define SPIx_DMA_TX_IRQHandler          DMA2_Stream3_IRQHandler
#define SPIx_DMA_RX_IRQHandler          DMA2_Stream2_IRQHandler

#define SPIx_IRQn                       SPI1_IRQn
#define SPIx_IRQHandler                 SPI1_IRQHandler
```

◆ 第 3 步：根据芯片支持的时钟速度，时钟相位和时钟极性配置函数 DAC8501_SetCS1 和 DAC8501_SetCS2。

```
/*
*********************************************************************************************************
*    函 数 名: DAC8501_SetCS1
*    功能说明: DAC8501 片选控制函数
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DAC8501_SetCS1(uint8_t _Level)
{
    if (_Level == 0)
    {
        bsp_SpiBusEnter(); /* 占用 SPI 总线  */
        bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_8, SPI_PHASE_2EDGE, SPI_POLARITY_LOW);
        CS1_0();
    }
    else
    {
        CS1_1();
        bsp_SpiBusExit();  /* 释放 SPI 总线 */
    }
}

/*
*********************************************************************************************************
*    函 数 名: DAC8501_SetCS2(0)
*    功能说明: 设置 CS2。 用于运行中 SPI 共享。
*    形    参: 无
*    返 回 值: 无
```

```
*********************************************************************************************************
*/
void DAC8501_SetCS2(uint8_t _level)
{
    if (_level == 0)
    {
        bsp_SpiBusEnter(); /* 占用 SPI 总线  */
        bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_8, SPI_PHASE_2EDGE, SPI_POLARITY_LOW);
        CS2_0();
    }
    else
    {
        CS2_1();
        bsp_SpiBusExit();  /* 释放 SPI 总线 */
    }
}
```

◆ 第 4 步：根据使用的片选引脚，修改 bsp_spi_dac8562.c 文件开头的宏定义。

```
#define CS1_CLK_ENABLE()    __HAL_RCC_GPIOG_CLK_ENABLE()
#define CS1_GPIO            GPIOG
#define CS1_PIN             GPIO_PIN_10

#define CS1_1()             CS1_GPIO->BSRR = CS1_PIN
#define CS1_0()             CS1_GPIO->BSRR = ((uint32_t)CS1_PIN << 16U)

/*特别注意，我们这里是用的扩展 IO 控制的 */
#define CS2_1()             HC574_SetPin(NRF24L01_CE, 1);
#define CS2_0()             HC574_SetPin(NRF24L01_CE, 0);
```

◆ 第 5 步：如果使用 DMA 方式的话，请不要使用 TCM RAM，因为通用 DMA1 和 DMA2 不支持。并为了防止 DMA 和 CPU 同时访问 DMA 缓冲造成的数据一致性问题，将这块空间关闭 Cache 处理，比如使用的 SRAM4：

```
/* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress      = 0x38000000;
MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

◆ 第 6 步：初始化 SPI。

```
/* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
bsp_InitSPIBus();  /* 配置 SPI 总线 */
bsp_InitDAC8501(); /* 初始化配置 DAC8501 */
```

◆ 第 7 步：DAC8501 驱动主要用到 HAL 库的 SPI 驱动文件，简单省事些可以添加所有 HAL 库 C 源文件进来。

◆ 第 8 步：应用方法看本章节配套例子即可。

75.11 DAC8501 驱动移植和使用（SPI DMA 更新方式）

DAC8501 移植步骤如下：

- 第 1 步：复制 bsp_spidma_dac8501.c，bsp_spidma_dac8501.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：根据使用的第几个 SPI，SPI 时钟，SPI 引脚和 DMA 通道等，修改 bsp_spidma_dac8501.c 文件开头的宏定义

```
/*
*********************************************************************************************************
*                                    时钟，引脚，DMA，中断等宏定义
*********************************************************************************************************
*/
#define SPIx                             SPI1
#define SPIx_CLK_ENABLE()                __HAL_RCC_SPI1_CLK_ENABLE()
#define DMAx_CLK_ENABLE()                __HAL_RCC_DMA2_CLK_ENABLE()

#define SPIx_FORCE_RESET()               __HAL_RCC_SPI1_FORCE_RESET()
#define SPIx_RELEASE_RESET()             __HAL_RCC_SPI1_RELEASE_RESET()

/* SYNC, 也就是 CS 片选 */
#define SPIx_NSS_CLK_ENABLE()            __HAL_RCC_GPIOG_CLK_ENABLE()
#define SPIx_NSS_GPIO                    GPIOG
#define SPIx_NSS_PIN                     GPIO_PIN_10
#define SPIx_NSS_AF                      GPIO_AF5_SPI1

#define SPIx_SCK_CLK_ENABLE()            __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_SCK_GPIO                    GPIOB
#define SPIx_SCK_PIN                     GPIO_PIN_3
#define SPIx_SCK_AF                      GPIO_AF5_SPI1

#define SPIx_MISO_CLK_ENABLE()           __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MISO_GPIO                   GPIOB
#define SPIx_MISO_PIN                    GPIO_PIN_4
#define SPIx_MISO_AF                     GPIO_AF5_SPI1

#define SPIx_MOSI_CLK_ENABLE()          __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MOSI_GPIO                  GPIOB
#define SPIx_MOSI_PIN                   GPIO_PIN_5
#define SPIx_MOSI_AF                    GPIO_AF5_SPI1

#define SPIx_TX_DMA_STREAM               DMA2_Stream3
#define SPIx_RX_DMA_STREAM               DMA2_Stream2

#define SPIx_TX_DMA_REQUEST              DMA_REQUEST_SPI1_TX
#define SPIx_RX_DMA_REQUEST              DMA_REQUEST_SPI1_RX

#define SPIx_DMA_TX_IRQn                 DMA2_Stream3_IRQn
#define SPIx_DMA_RX_IRQn                 DMA2_Stream2_IRQn

#define SPIx_DMA_TX_IRQHandler           DMA2_Stream3_IRQHandler
#define SPIx_DMA_RX_IRQHandler           DMA2_Stream2_IRQHandler

#define SPIx_IRQn                        SPI1_IRQn
```

```
#define SPIx_IRQHandler                 SPI1_IRQHandler
```

- 第 3 步：如果使用 DMA 方式的话，请不要使用 TCM RAM，因为通用 DMA1 和 DMA2 不支持。并为了防止 DMA 和 CPU 同时访问 DMA 缓冲造成的数据一致性问题，将这块空间关闭 Cache 处理，比如使用的 SRAM4：

```
/* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress      = 0x38000000;
MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

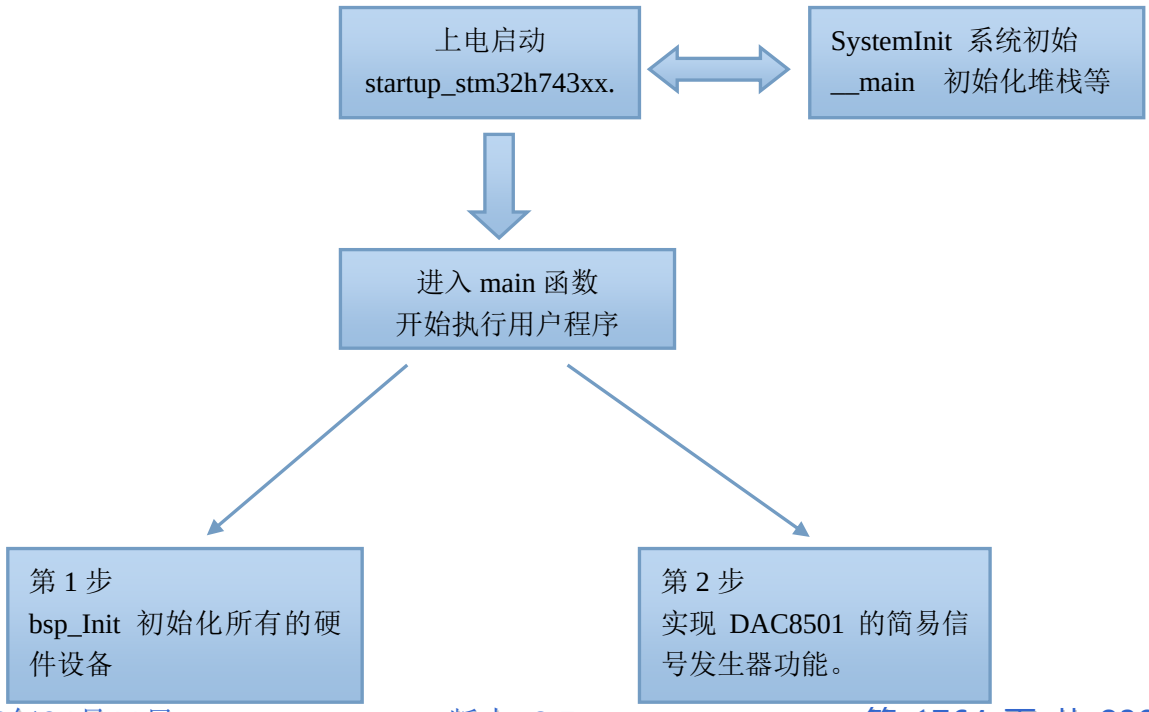
- 第 4 步：初始化 SPI。

```
/* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
bsp_InitDAC8501(); /* 初始化配置 DAC8501 */
```

- 第 5 步：DAC8501 驱动主要用到 HAL 库的 SPI 驱动文件，简单省事些可以添加所有 HAL 库 C 源文件进来。
- 第 6 步：应用方法看本章节配套例子即可

75.12 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器和 LED。
  - 第 2 部分，应用程序设计部分，实现 DAC8501 的简易信号发生器功能。。

75.13 实验例程说明（MDK）

注：本章是配套了两个例子的，这里我们以 SPI DMA 方式进行说明。

配套例子：

V7-054_DAC8501 简易信号发生器（单通道 SPI DMA 方式，16bit 分辨率, 0-5V 输出）

V7-055_DAC8501 简易信号发生器（双通道 SPI 查询方式，16bit 分辨率, 0-5V 输出）

实验目的：

1. 学习 DAC8501 的 SPI DMA 驱动方式实现。

实验内容：

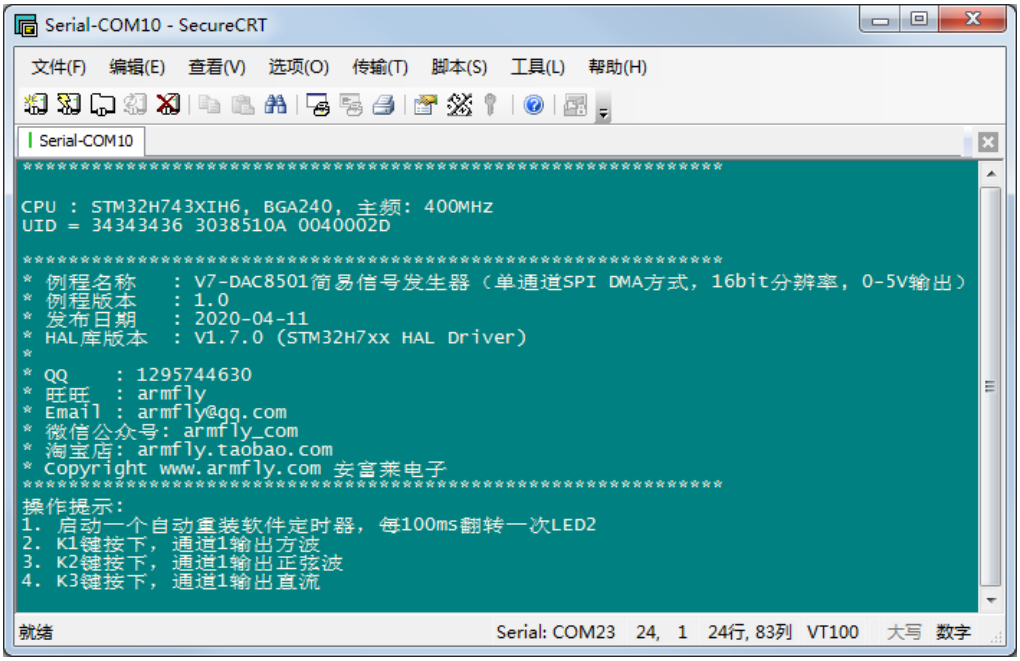
1. DAC8501 模块上带了两片 8501，每片是单通道 DAC，片上输出缓冲运放，轨到轨输出，16bit 分辨率，支持 30MHz 的 SPI 时钟速度。
2. DAC8501 本身仅支持一路输出，而模块上是带了两片 DAC8501，其中只有一路的片选可以支持 SPI NSS 复用， 所以只有一路支持 SPI DMA。
3. DAC8501 供电电压 2.7-5.5V，模拟输出带宽 350KHz。

实验操作：

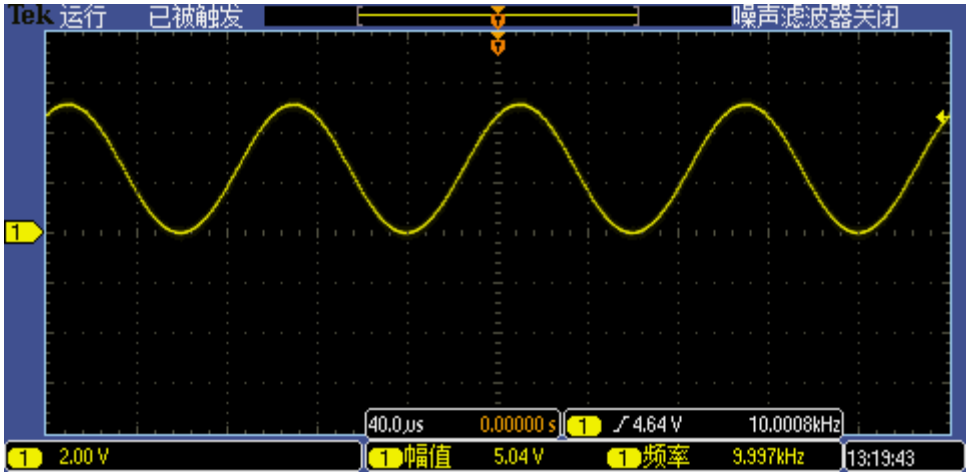
1. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
2. K1 键按下，通道 1 输出方波。
3. K2 键按下，通道 1 输出正弦波。
4. K3 键按下，通道 1 输出直流。

上电后串口打印的信息：

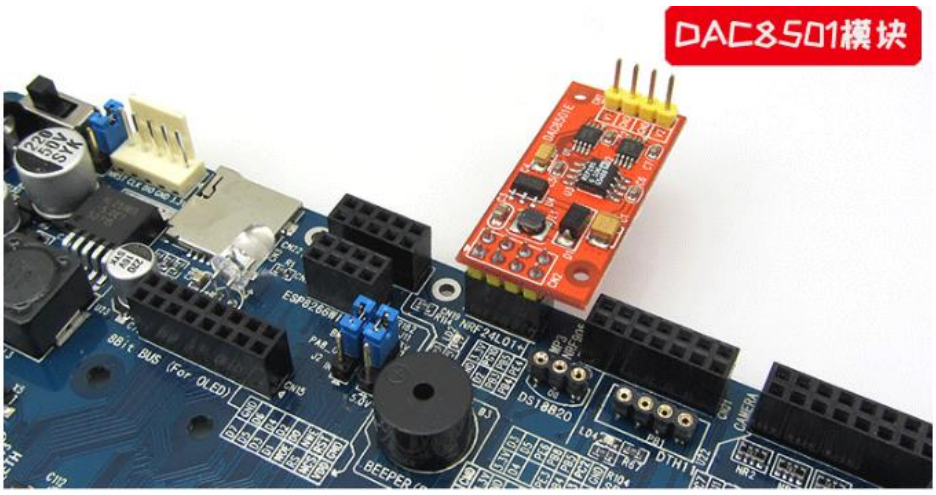
波特率 115200，数据位 8，奇偶校验位无，停止位 1。



波形效果：

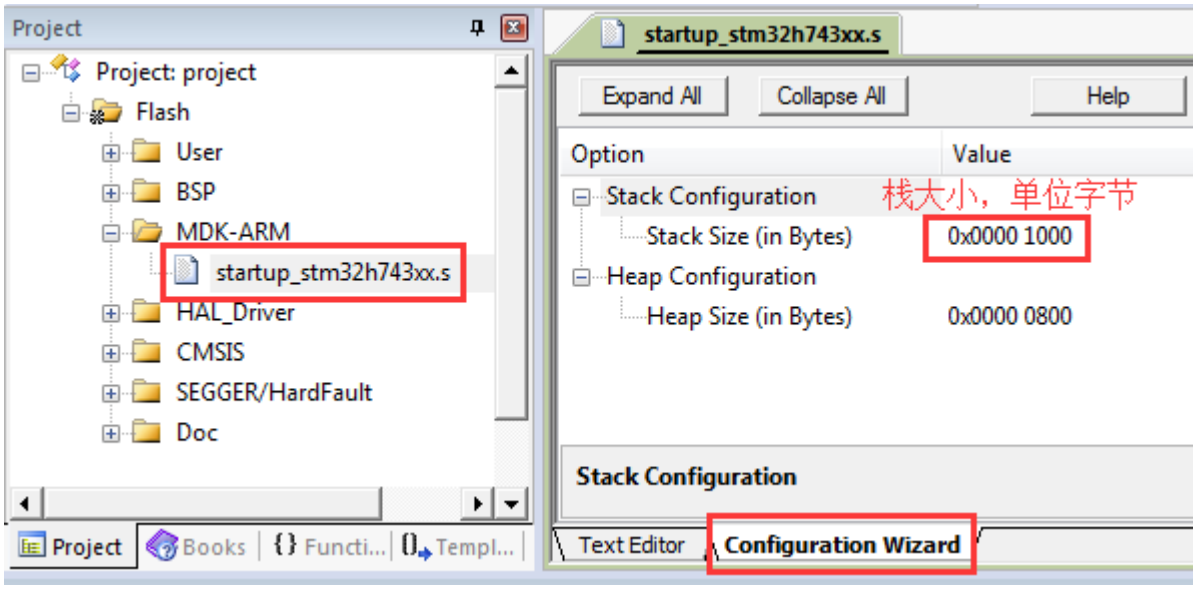


模块插入位置：

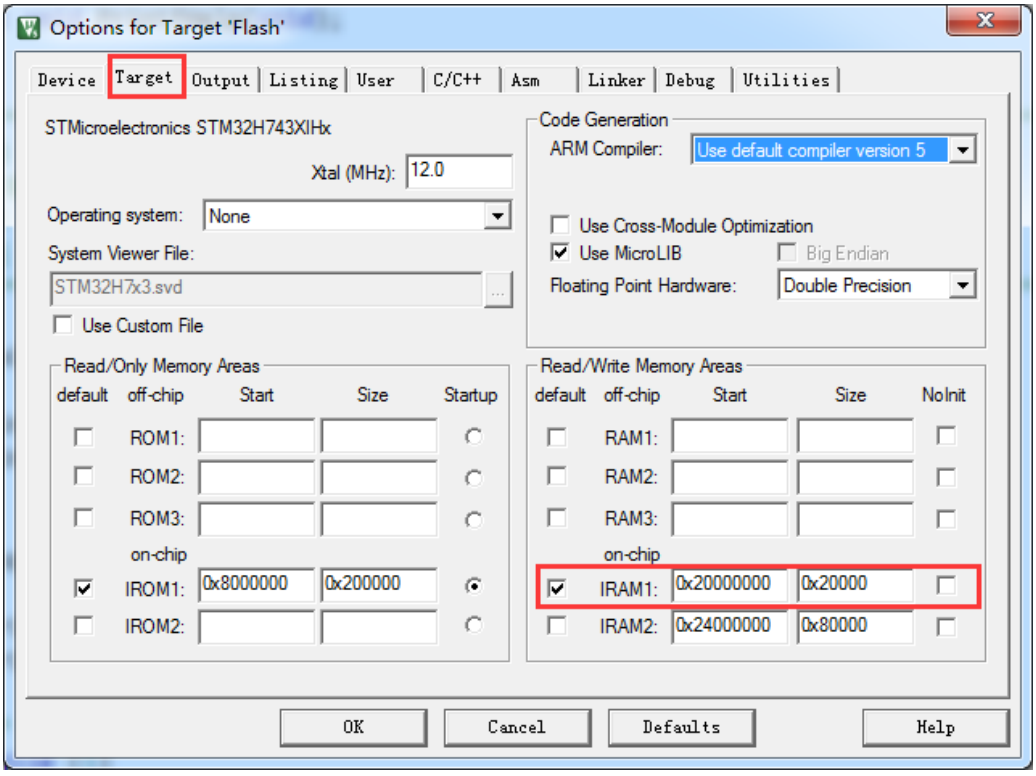


程序设计：

◆ **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();      /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();    /* 初始化滴答定时器 */
    bsp_InitLPUart();   /* 初始化串口 */
    bsp_InitExtIO();    /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();      /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitDAC8501();  /* 初始化配置 DAC8501 */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区以及 SRAM4

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
```

```
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
```

```
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **每 10ms 调用一次按键处理：**

按键处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能：**

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- K1 键按下，通道 1 输出方波。
- K2 键按下，通道 1 输出正弦波。
- K3 键按下，通道 1 输出直流。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */

    DemoSpiDac();    /* SPI DAC 测试 */
}
```

```
/*
*********************************************************************************************************
*    函 数 名: DemoSpiDac
*    功能说明: DAC8501 测试
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoSpiDac(void)
{
    uint8_t i=0;
    uint8_t ucKeyCode; /* 按键代码 */

    sfDispMenu();      /* 打印命令提示 */

    bsp_StartAutoTimer(0, 200); /* 启动 1 个 100ms 的自动重装的定时器 */


    /* 生成方波数据 */
    MakeSinTable(ch1buf, 100, 0, 65535);

    DAC8501_SetDacDataDMA(1, ch1buf, sizeof(ch1buf)/sizeof(uint16_t), 1000000);

    while(1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，通道 1 输出方波 */
                    /* 生成方波数据 */
                    for(i =0; i< 50; i++)
                    {
                        ch1buf[i] = 0;
                    }

                    for(i =50; i< 100; i++)
                    {
                        ch1buf[i] = 65535;
                    }

                    /* 触发速度 1MHz */
                    DAC8501_SetDacDataDMA(1, ch1buf, sizeof(ch1buf)/sizeof(uint16_t), 1000000);
                    break;

                case KEY_DOWN_K2:           /* K2 键按下，通道 1 输出正弦波 */
```

```
                    /* 生成正弦波数据 */
                    MakeSinTable(ch1buf, 100, 0, 65535);

                    /* 触发速度 1MHz */
                    DAC8501_SetDacDataDMA(1, ch1buf, sizeof(ch1buf)/sizeof(uint16_t), 1000000);
                    break;

                case KEY_DOWN_K3:           /* K3 键按下，通道 1 输出直流 */

                    /* 生成方波数据 */
                    for(i =0; i< 100; i++)
                    {
                        ch1buf[i] = 65535;
                    }

                    /* 触发速度 1MHz */
                    DAC8501_SetDacDataDMA(1, ch1buf, sizeof(ch1buf)/sizeof(uint16_t), 1000);
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
    }
}
```

75.14 实验例程说明（IAR）

注：本章是配套了两个例子的，这里我们以 SPI DMA 方式进行说明。

配套例子：

V7-054_DAC8501 简易信号发生器（单通道 SPI DMA 方式，16bit 分辨率, 0-5V 输出）

V7-055_DAC8501 简易信号发生器（双通道 SPI 查询方式，16bit 分辨率, 0-5V 输出）

实验目的：

1. 学习 DAC8501 的 SPI DMA 驱动方式实现。

实验内容：

1. DAC8501 模块上带了两片 8501，每片是单通道 DAC，片上输出缓冲运放，轨到轨输出，16bit 分辨率，支持 30MHz 的 SPI 时钟速度。
2. DAC8501 本身仅支持一路输出，而模块上是带了两片 DAC8501，其中只有一路的片选可以支持 SPI NSS 复用， 所以只有一路支持 SPI DMA。
3. DAC8501 供电电压 2.7-5.5V，模拟输出带宽 350KHz。

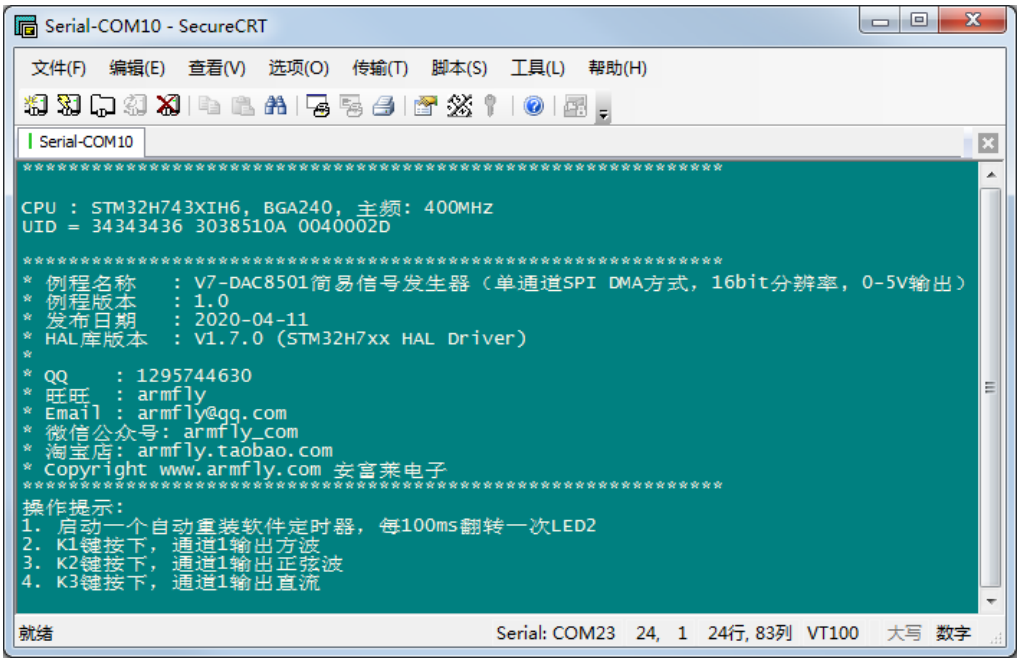
实验操作：

1. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
2. K1 键按下，通道 1 输出方波。
3. K2 键按下，通道 1 输出正弦波。

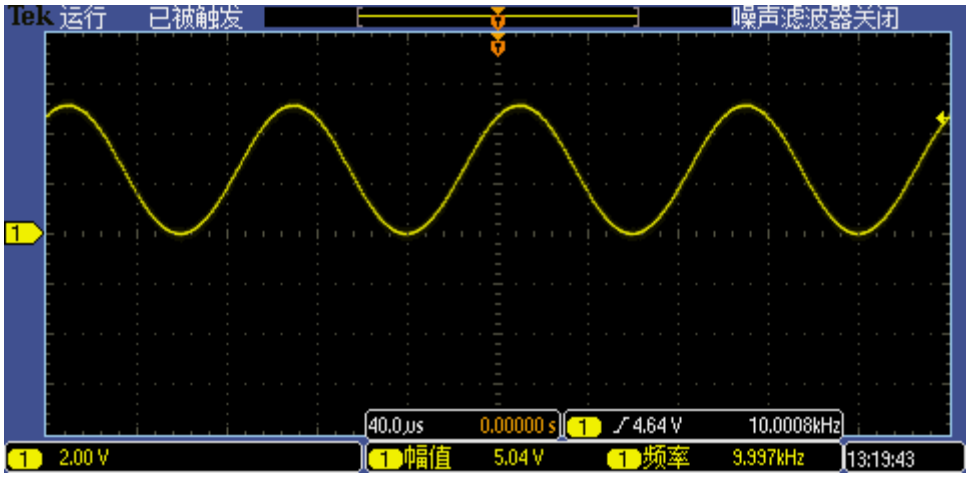
4. K3 键按下，通道 1 输出直流。

上电后串口打印的信息：

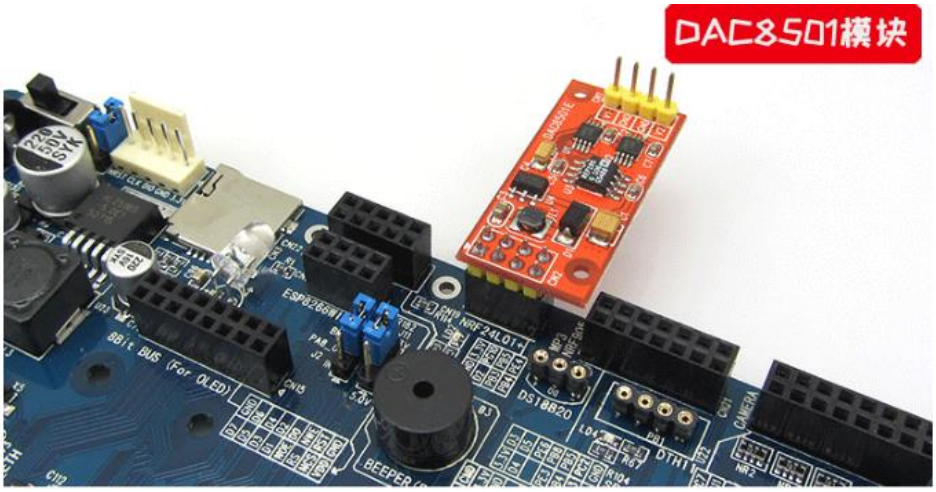
波特率 115200，数据位 8，奇偶校验位无，停止位 1。



波形效果：

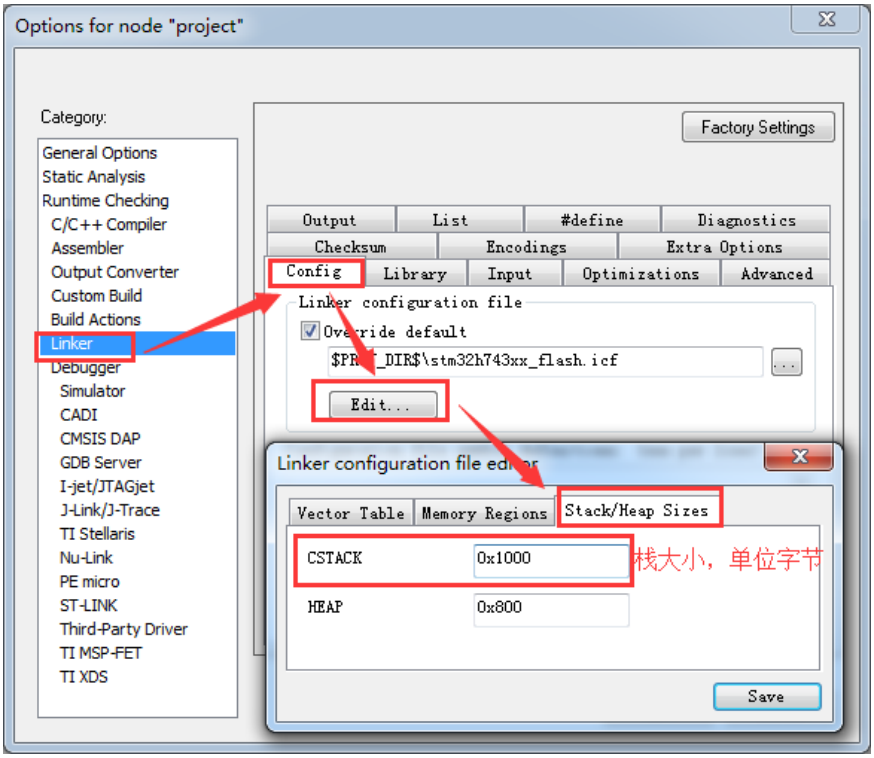


模块插入位置：

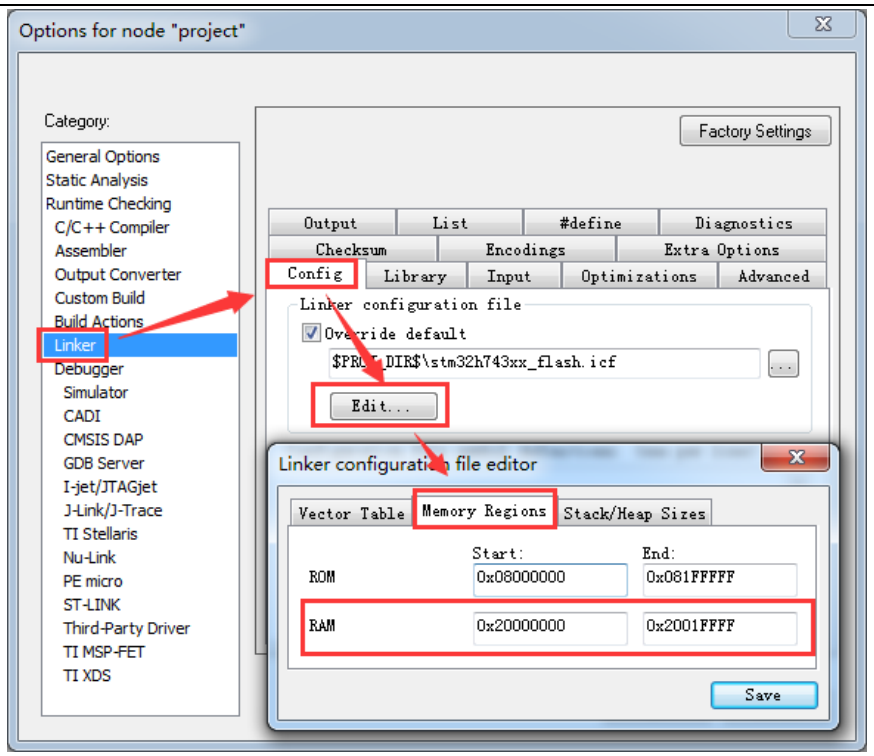


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();      /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();    /* 初始化滴答定时器 */
    bsp_InitLPUart();   /* 初始化串口 */
    bsp_InitExtIO();    /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();      /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitDAC8562(); /* 初始化配置 DAC8562/8563 */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区以及 SRAM4

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
```

```
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **每 10ms 调用一次按键处理：**

按键处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
```

```
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能:**

主程序实现如下操作:

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- K1 键按下，通道 1 输出方波。
- K2 键按下，通道 1 输出正弦波。
- K3 键按下，通道 1 输出直流。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */

    DemoSpiDac();    /* SPI DAC 测试 */
}

/*
*********************************************************************************************************
*    函 数 名: DemoSpiDac
*    功能说明: DAC8501 测试
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoSpiDac(void)
{
    uint8_t i=0;
    uint8_t ucKeyCode; /* 按键代码 */

    sfDispMenu();      /* 打印命令提示 */

    bsp_StartAutoTimer(0, 200); /* 启动 1 个 100ms 的自动重装的定时器 */


    /* 生成方波数据 */
    MakeSinTable(ch1buf, 100, 0, 65535);

    DAC8501_SetDacDataDMA(1, ch1buf, sizeof(ch1buf)/sizeof(uint16_t), 1000000);
```

```
    while(1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，通道 1 输出方波 */
                    /* 生成方波数据 */
                    for(i =0; i< 50; i++)
                    {
                        ch1buf[i] = 0;
                    }

                    for(i =50; i< 100; i++)
                    {
                        ch1buf[i] = 65535;
                    }

                    /* 触发速度 1MHz */
                    DAC8501_SetDacDataDMA(1, ch1buf, sizeof(ch1buf)/sizeof(uint16_t), 1000000);
                    break;

                case KEY_DOWN_K2:           /* K2 键按下，通道 1 输出正弦波 */
                    /* 生成正弦波数据 */
                    MakeSinTable(ch1buf, 100, 0, 65535);

                    /* 触发速度 1MHz */
                    DAC8501_SetDacDataDMA(1, ch1buf, sizeof(ch1buf)/sizeof(uint16_t), 1000000);
                    break;

                case KEY_DOWN_K3:           /* K3 键按下，通道 1 输出直流 */

                    /* 生成方波数据 */
                    for(i =0; i< 100; i++)
                    {
                        ch1buf[i] = 65535;
                    }

                    /* 触发速度 1MHz */
                    DAC8501_SetDacDataDMA(1, ch1buf, sizeof(ch1buf)/sizeof(uint16_t), 1000);
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }
        }
```

```
    }
}
```

75.15 总结

本章节涉及到的知识点非常多，特别是 SPI DMA 方式驱动的实现方法，需要大家稍花点精力去研究。

第76章 STM32H7 的 FMC 总线应用之驱动 AD7606（8 通道同步采样, 16bit，正负 10V）

本章节为大家讲解 FMC 总线驱动数模转换器 AD7606，实战性较强。

76.1 初学者重要提示
76.2 ADC 结构分类
76.3 AD7606 硬件设计
76.4 AD7606 关键知识点整理（重要）
76.5 AD7606 的 FMC 接口硬件设计
76.6 AD7606 的 FMC 接口驱动设计
76.7 AD7606 板级支持包（bsp_fmc_ad7606）
76.8 J-Scope 实时展示 AD7606 采集数据说明
76.9 AD7606 驱动移植和使用
76.10 实验例程设计框架
76.11 实验例程说明（MDK）
76.12 实验例程说明（IAR）
76.13 总结

76.1 初学者重要提示

- 学习本章节前，务必优先学习第 47 章，了解 FMC 总线的基础知识。
- AD7606 的配置很简单，它没有内部寄存器，量程范围和过采样参数是通过外部 IO 控制的，采样速率由 MCU 或 DSP 提供的脉冲频率控制。
- AD7606 必须使用单 5V 供电。而 AD7606 和 MCU 之间的通信接口电平由 VIO (V_{DRIVE}) 引脚控制。也就是说 VIO 必须接单片机的电源，可以是 3.3V 也可以是 5V（范围 2.3V – 5V）。
- 正确的理解过采样，比如我们设置是 1Ksps 采样率，64 倍过采样。意思是指每次采样，AD7606 会采样 64 次数据并求平均，相当于 AD7606 以 64Ksps 进行采样的，只是将每 64 个采样点的值做了平均，用户得到的值就是平均后的数值。因此，如果使用 AD7606 最高的 200Ksps 采样率，就不可以使用过采样了。
- STM32H7 驱动 AD7606 配合 J-Scope 实时输出，效果绝了，堪比示波器

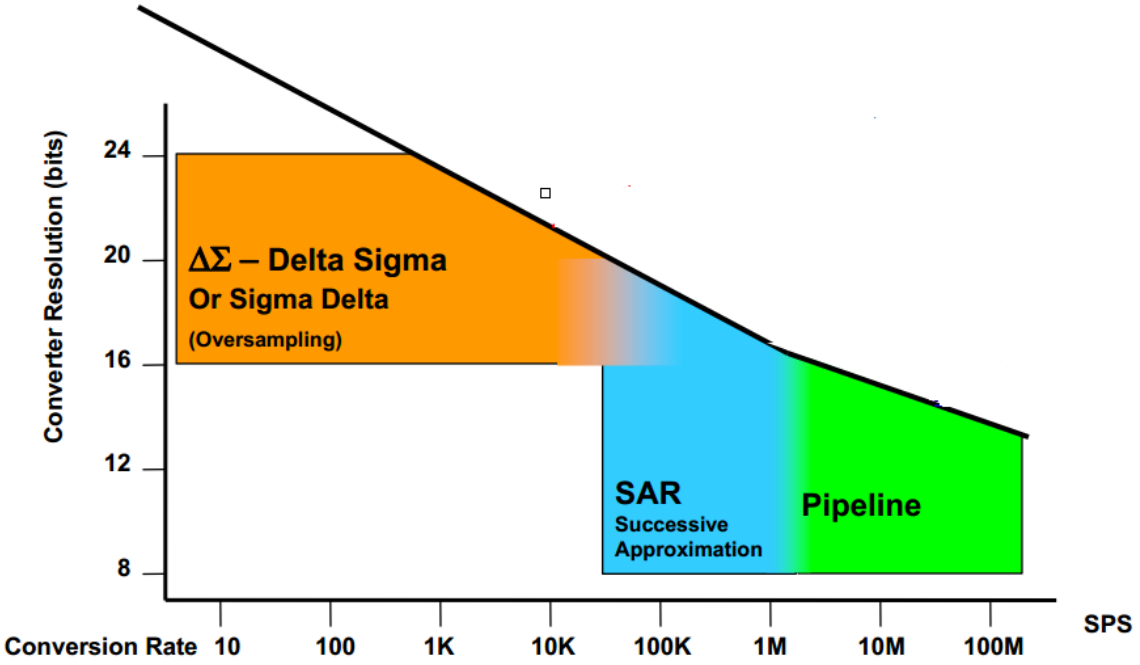
http://www.armbbs.cn/forum.php?mod=viewthread&tid=97393 。

使用方法详解本章节 77.8 小节。

- 本章配套例子的串口数据展示推荐使用 SecureCRT，因为数据展示做了特别处理，方便采集数据在串口软件同一个位置不断刷新。
- AD7606 数据手册，模块原理图（通用版）和接线图都已经放到本章教程配置例子的 Doc 文件里。
- ADC 的专业术语诠释文档，推荐大家看看：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=89414 。
- 测试本章配套例子前重要提示：
  - 测试时，务必使用外置电源为开发板供电，因为 AD7606 需要 5V 供电电压。板子上插入 AD7606 模块时，注意对齐。
  - 板子上电后，默认是软件定时采集，0.5 秒一次，适合串口展示数据。
  - 如果需要使用 J-Scope 实时展示采集的波形效果，需要按下 K2 按键切换到 FIFO 模式。
  - 如果使用的 JLINK 速度不够快，导致 J-Scope 无法最高速度实时上传，可以使用摇杆上下键设置过采样来降低上传速度。
  - 默认情况下，程序仅上传了 AD7606 通道 1 采集的数据。

76.2 ADC 结构分类

这里将六种 DAC 结构为大家做个普及。注，这些知识翻译自美信和 TI 的英文技术手册。



76.2.1 SAR ADC（逐次逼近型）

逐次逼近型 ADC 通常是中高分辨率的首选架构，采样速率通常低于 5Msps。SAR ADC 最常见的分辨率范围是 8 位到 20 位，并具有低功耗和小尺寸的特点。这种组合使其非常适合各种应用，例如自动测试设备，电池供电的设备，数据采集系统，医疗仪器，电机和过程控制，工业自动化，电信，测试和测量，便携式系统，高速闭环系统和窄带接收器。

76.2.2 Sigma-Delta ADC

Sigma-delta ADC 主要用于低速应用中，该应用需要通过过采样来权衡速度和分辨率，然后进行滤波以降低噪声。24 位 sigma-delta 转换器用于自动化测试设备，高精度便携式传感器，医疗和科学仪器以及地震数据采集等应用中。

76.2.3 Integrating ADC

集成 ADC 提供高分辨率，并且可以提供良好的线路频率和噪声抑制。集成架构提供了一种新颖且直接的方法，可将低带宽模拟信号转换为数字表示形式。这些类型的转换器通常包括用于 LCD 或 LED 显示器的内置驱动器，并且在许多便携式仪器应用中都可以找到，包括数字面板表和数字万用表。

76.2.4 FLASH ADC

Flash ADC 是将模拟信号转换为数字信号的最快方法。它们适用于需要非常大带宽的应用。然而，闪存转换器功率高，具有相对较低的分辨率，并且可能非常昂贵。这将它们限制在通常无法以其他任何方式解决的高频应用中。示例包括数据采集，卫星通信，雷达处理，示波器和高密度磁盘驱动器。

76.2.5 Pipelined ADC

流水线 ADC 已成为最受欢迎的 ADC 体系结构，其采样率从每秒几兆采样（MS / s）到最高 100MS / s +，分辨率为 8 至 16 位。它们提供的分辨率和采样率，可覆盖各种应用，包括 CCD 成像，超声医学成像，数字接收器，基站，数字视频（例如 HDTV），xDSL，电缆调制解调器和快速以太网。

76.2.6 Two Step ADC

两步 ADC 也称为子范围转换器，有时也称为多步或 half flash（比 Flash 架构慢）。这是 Flash ADC 和流水线 ADC 的交叉点。与 Flash ADC 相比，可以实现更高的分辨率或更小的裸片尺寸。

76.3 AD7606 硬件设计

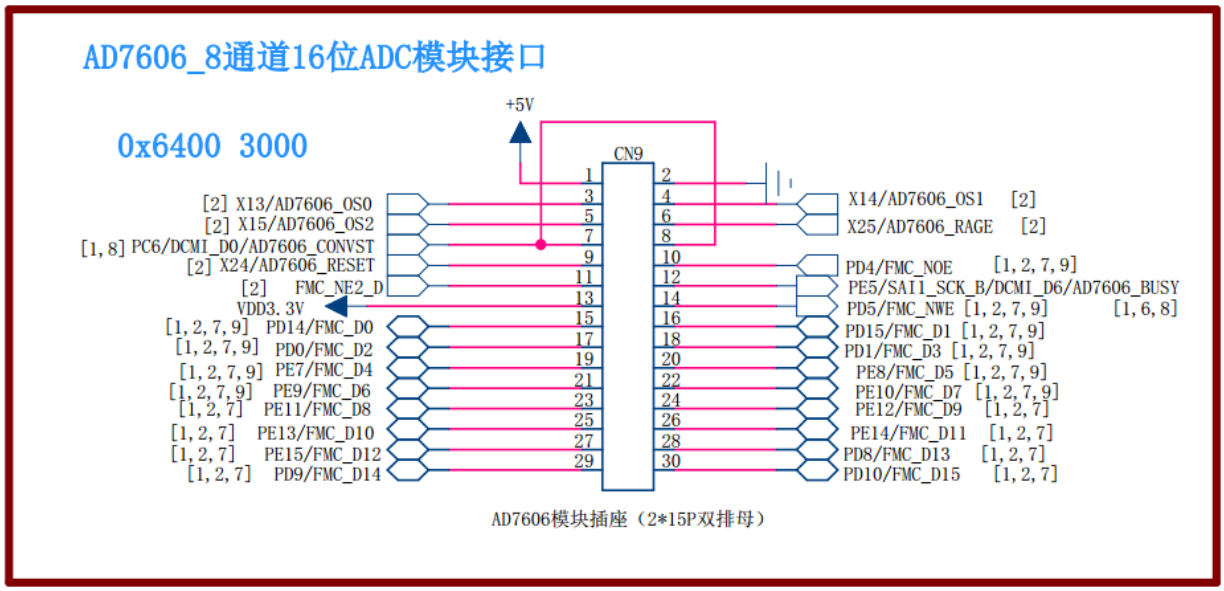
这里将开发板上的 AD7606 硬件接口，普通型 AD7606 模块，屏蔽型 AD7606 模块和磁耦高速隔离型 AD7606 模块为大家做个说明。

AD7606 的原理图下载：

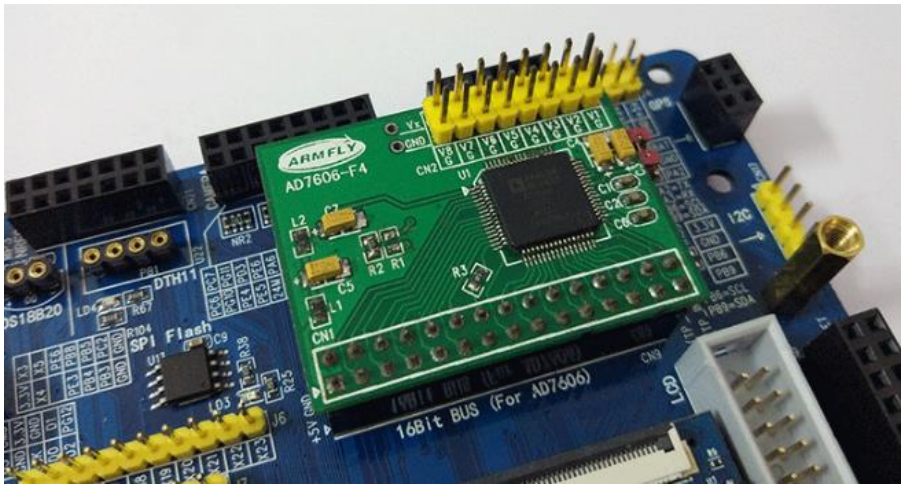
http://www.armbbs.cn/forum.php?mod=viewthread&tid=97547 。

76.3.1 AD7606 硬件接口

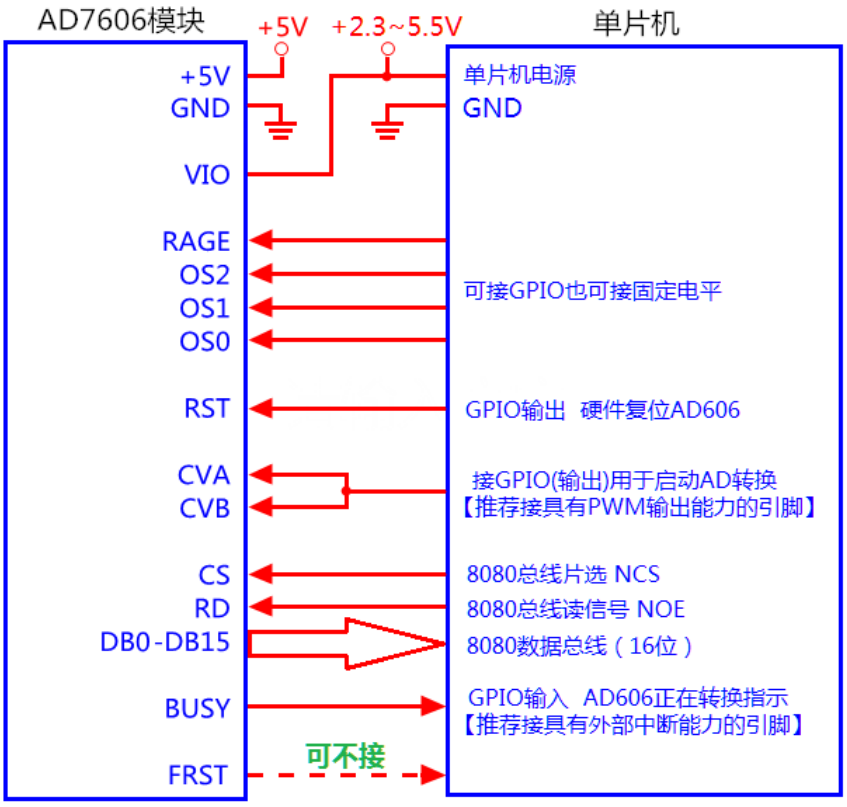
V7 板子上 AD7606 模块的插座的原理图如下：



实际对应开发板的位置如下：



为了方便大家更好的理解接线，下面是框图：



模块引脚说明：

- OS2 OS1 OS2 ：

  组合状态选择过采样模式。

  - 000 表示无过采样，最大 200Ksps 采样速率。
  - 001 表示 2 倍过采样， 也就是硬件内部采集 2 个样本求平均。
  - 010 表示 4 倍过采样， 也就是硬件内部采集 4 个样本求平均。
  - 011 表示 8 倍过采样， 也就是硬件内部采集 8 个样本求平均。
  - 100 表示 16 倍过采样， 也就是硬件内部采集 16 个样本求平均。
  - 101 表示 32 倍过采样， 也就是硬件内部采集 32 个样本求平均。
  - 110 表示 64 倍过采样， 也就是硬件内部采集 64 个样本求平均。

    过采样倍率越高，ADC 转换时间越长，可得到的最大采样频率就越低。

- CVA，CVB ：

  启动 AD 转换的控制信号。CVA 决定 1-4 通道，CVB 决定 5-8 通道。2 个信号可以错开短暂的时间。一般情况可以将 CVA，CVB 并联在一起。

- RAGE :

  量程范围选择。0 表示正负 5V, 1 表示正负 10V。

- RD :

  读信号。

- RST :

复位信号。

- BUSY：

  忙信号。

- CS：

  片选信号。

- FRST：

  第 1 个通道样本的指示信号。【注，此引脚可以省略不使用】

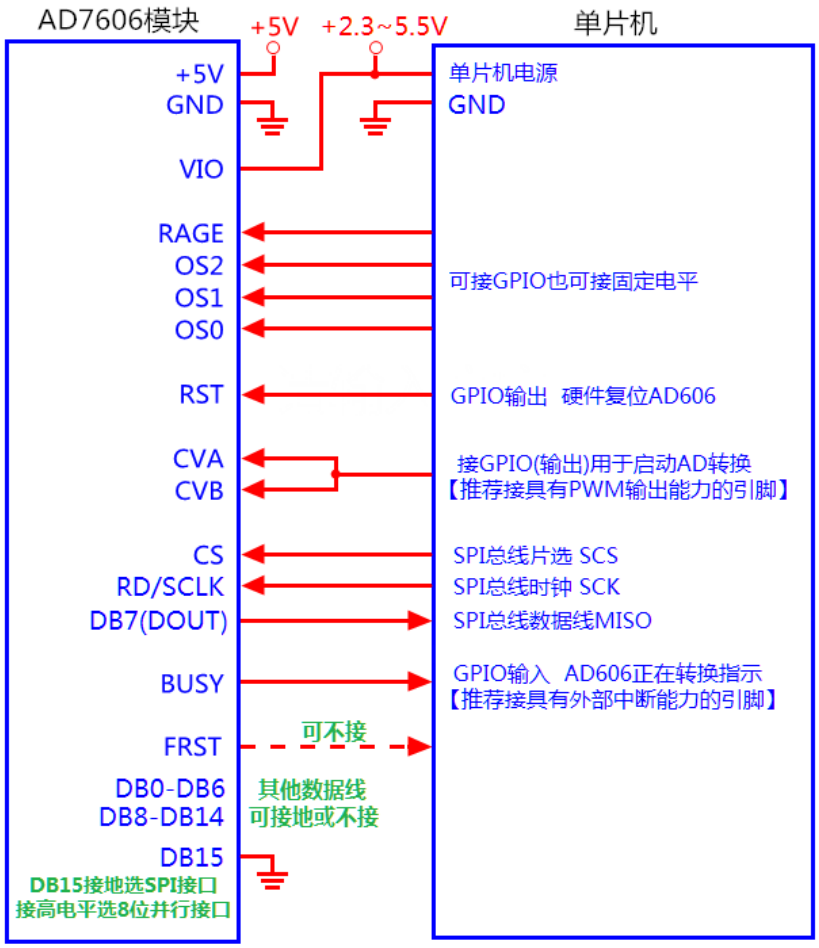
- VIO：

  通信接口电平。

- DB0-DB15：

  数据总线。

如果采用 SPI 接口方式，接线框图如下：



76.3.2 AD7606 模块（通用版）

产品规格：

1、16bit 分辨率，内置基准，单 5V 供电。

2、8 路模拟输入，阻抗 1M 欧姆。【无需负电源，无需前端模拟运放电路，可直接接传感器输出】

3、输入范围可以选择正负 5V 或者正负 10V，可通过 IO 控制量程。

4、最大采样频率 200Ksps，支持 8 档过采样设置（可以有效降低抖动）。

5、通信接口支持 SPI 或 16 位总线方式（也支持 8 位总线，一般用的比较少），接口 IO 电平可以是 5V 或 3.3V。

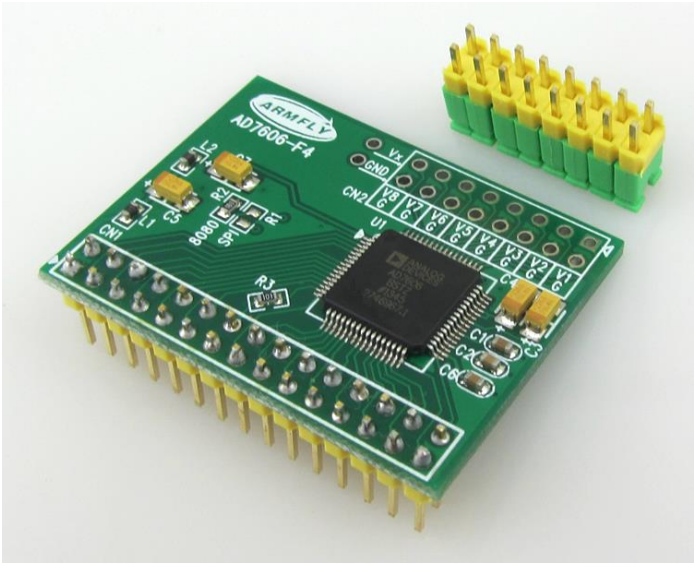
重要提示：

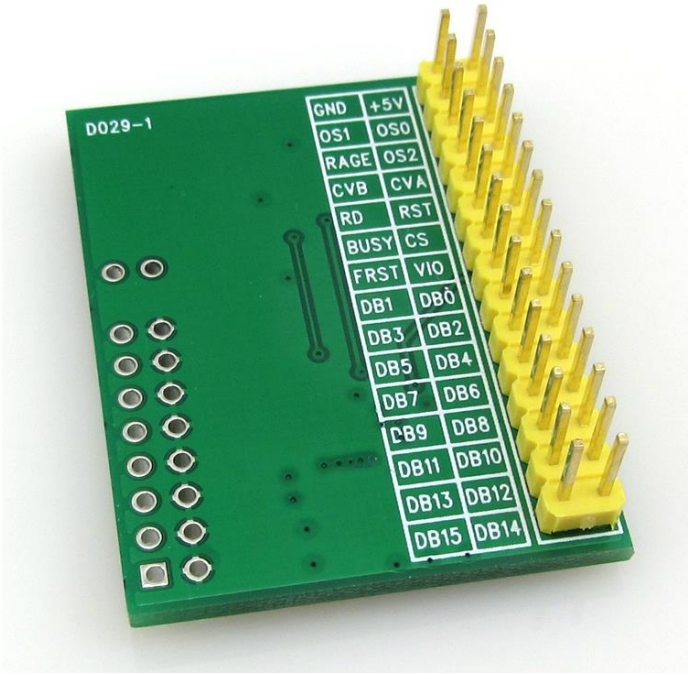
1、AD7606 的配置很简单，它没有内部寄存器。量程范围和过采样参数是通过外部 IO 控制的。采样速率由 MCU 或 DSP 提供的脉冲频率控制。

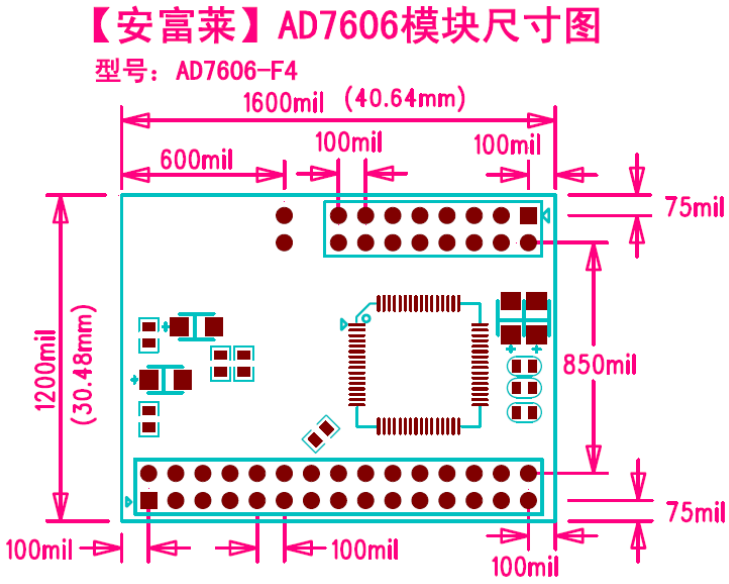
2、AD7606 必须使用单 5V 供电。

3、AD7606 和 MCU 之间的通信接口电平由 VIO（V_{DRIVE}）引脚控制。也就是说 VIO 必须接单片机的电源，可以是 3.3V 也可以是 5V。

产品效果：





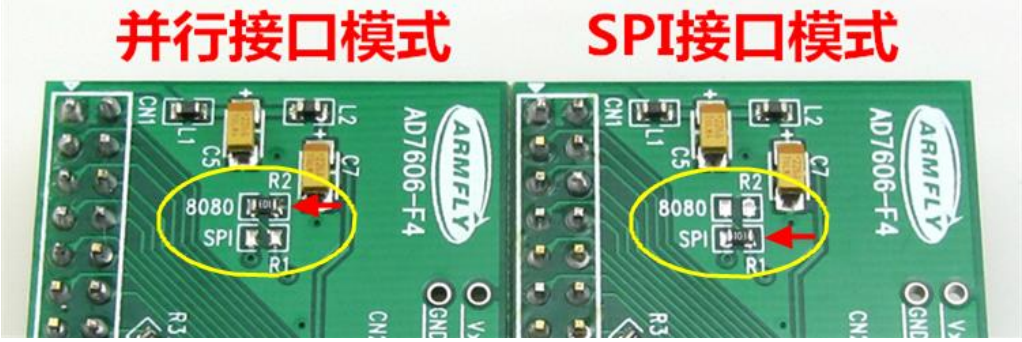


8080 或者 SPI 接口方式选择

出厂的 AD7606 模块缺省是 8080 并行接口，如果用 SPI 接口模式，需要修改 R1、R2 电阻配置。

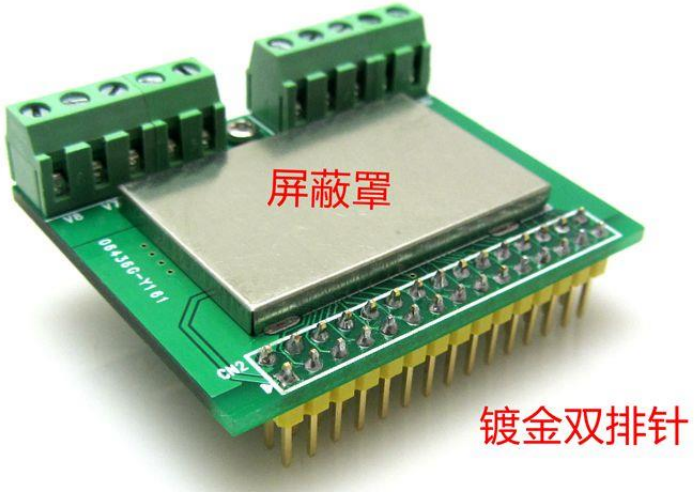
并口模式跳线：R1 悬空（不贴），R2 贴 10K 电阻。

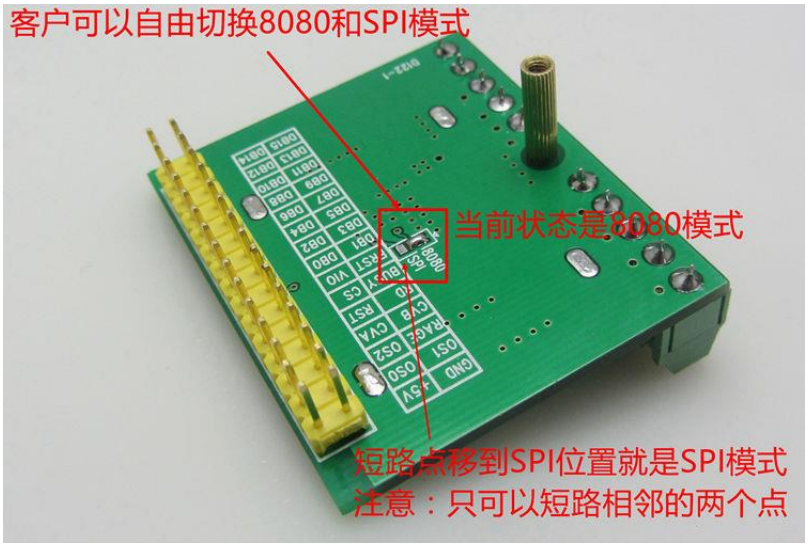
SPI 接口模式跳线：R1 贴 10K 电阻，R2 悬空（不贴）。

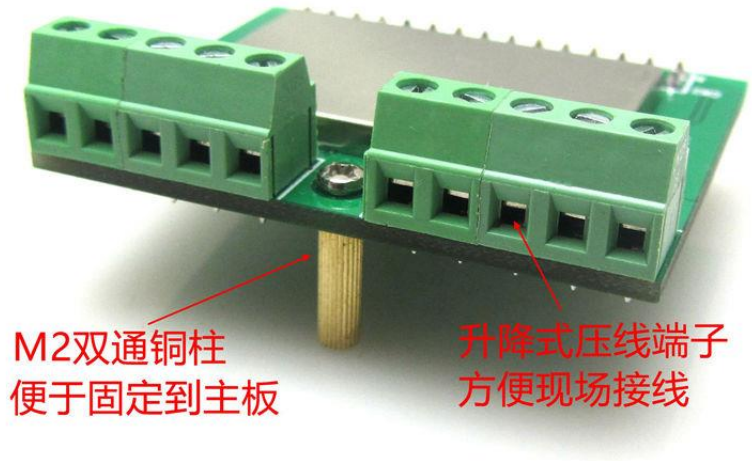


76.3.3 AD7606 模块（屏蔽版）

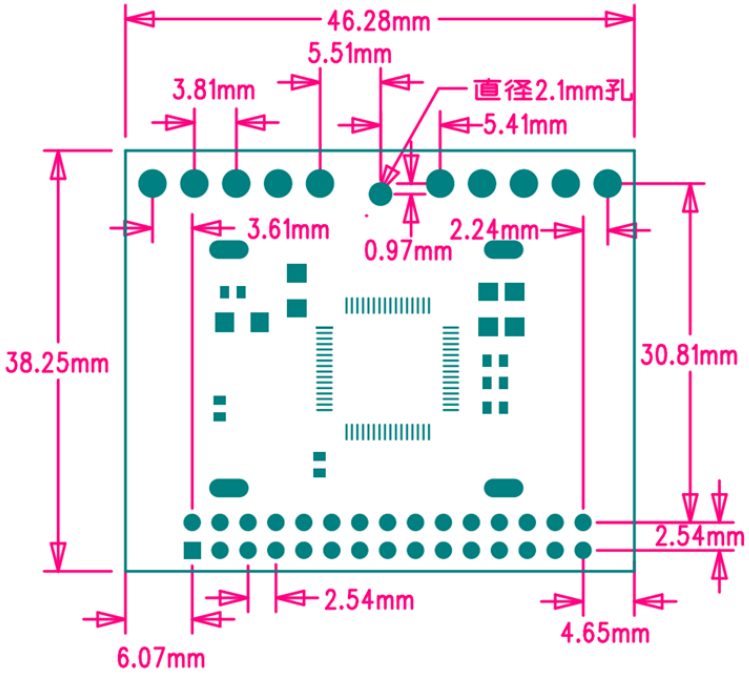
屏蔽版主要是为了更好的应对复杂的电磁工作，软件代码与非屏蔽版是一样的：






M2双通铜柱
便于固定到主板
升降式压线端子
方便现场接线

固定到底板


46.28mm
5.51mm
3.81mm
直径2.1mm孔
5.41mm
3.61mm
0.97mm
2.24mm
38.25mm
30.81mm
2.54mm
2.54mm
6.07mm
4.65mm

76.3.4AD7606 模块（磁耦高速隔离）

该款 ADC 模块采用磁耦隔离技术隔离 SPI 通信接口，采用 DC-DC 隔离电源模块隔离供电电源。高速 SPI 接口，ADC 主芯片采用 AD7606 芯片。8 通道 200KHz 采样。量程和滤波设置通过短路焊点设置。

产品规格

模拟通道 ： 8 路同步采集。

采样频率 ： 最大 200KHz。

ADC 分辨率 ： 16bit。

输入量程 ： 正负 5V 或正负 10V (通过焊点切换）。

滤波设置 ： 0 - 64 共 7 级硬件均值滤波。

供电电压 ： 5.0V， 耗电最大 50mA。

通信接口 ： SPI，最大时钟频率 16MHz。

接口电平 ： 3.3V 或 5V （3.3V 时，耗电 15mA）。

产品特点

1、电源隔离，隔离电压 1500V。

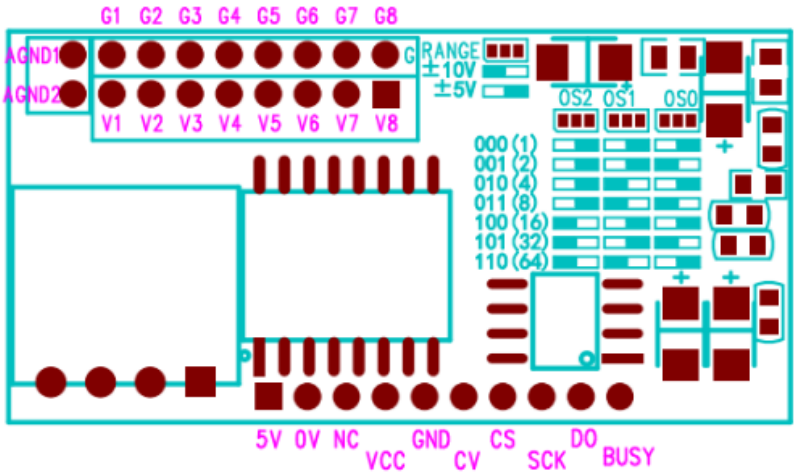
2、SPI 通信接口隔离，高速磁耦隔离技术。

3、短路点切换量程和过采样（滤波）参数。

4、体积小，2.0mm 间距排针，节约主板面积。

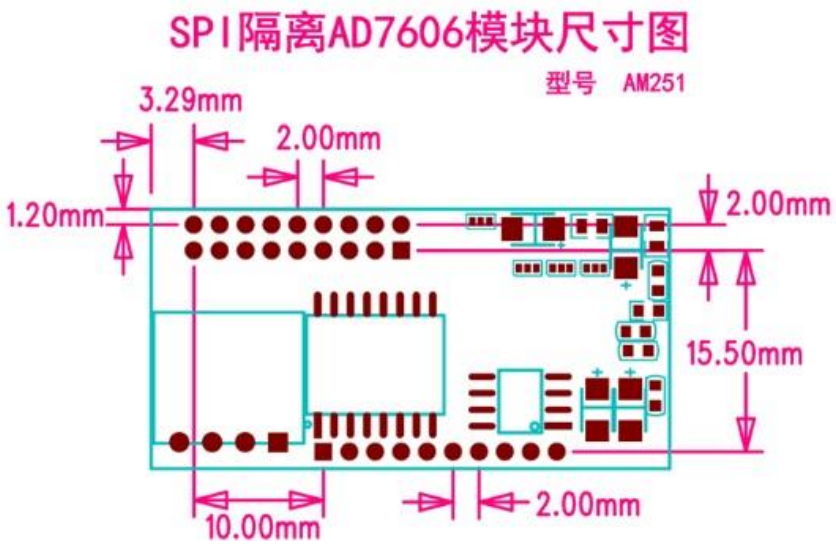
产品效果：

引脚定义和接线图：

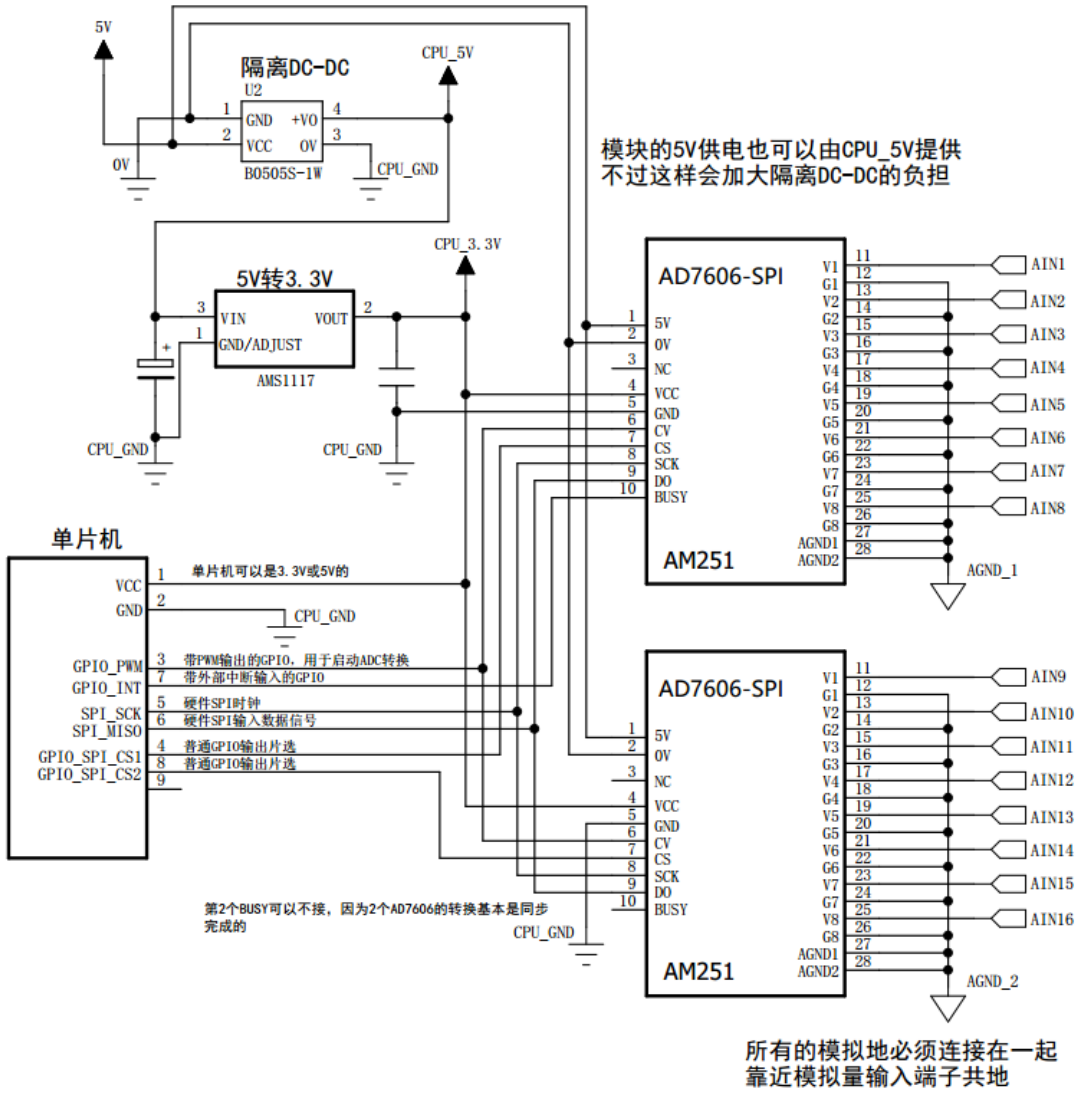


5V	供电电源5V
0V	供电电源地
NC	空脚
VCC	单片机电源
GND	单片机地
CV	AD7606 启动转换控制
CS	AD7606 SPI片选
SCK	AD7606 SPI时钟信号
DO	AD7606 SPI数据线（MISO）
BUSY	AD7606 BUSY 忙信号

V1 – V8	8路模拟信号输入
G1 – G8	8路模拟信号地
AGND1	AD7606模拟地，有2个脚模块内部是并联的
AGND2	



SPI隔离AD7606模块接线图

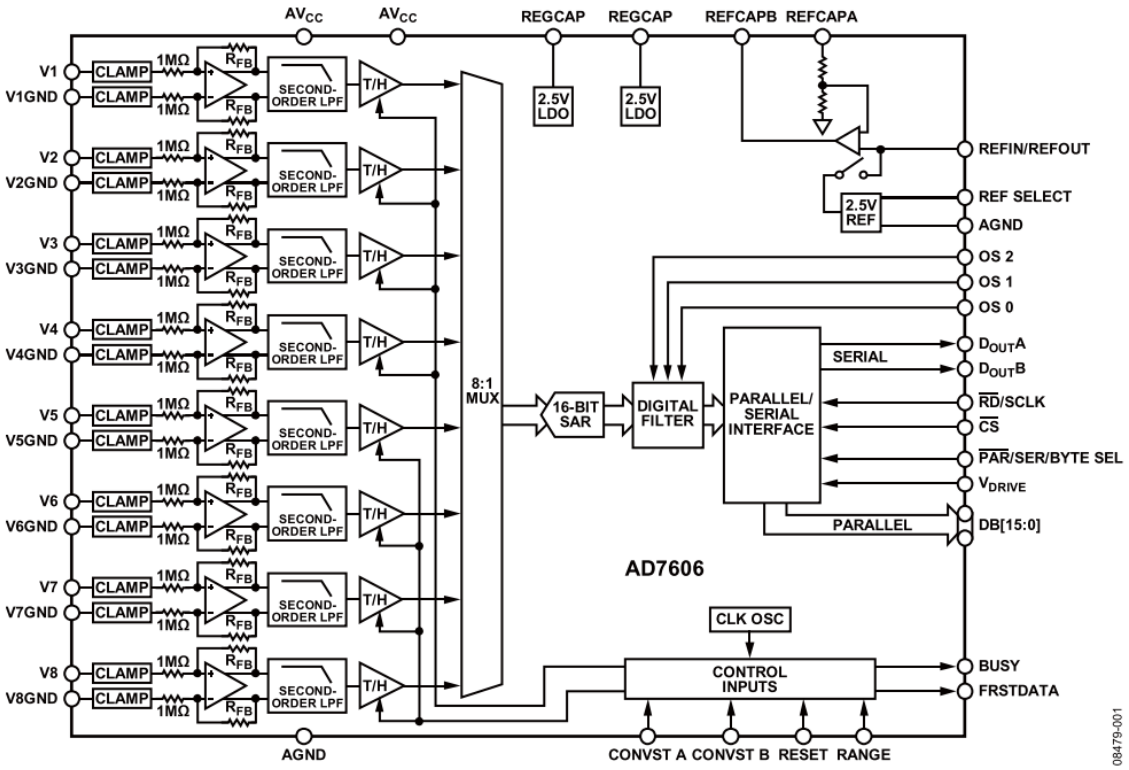


76.4 AD7606 关键知识点整理（重要）

驱动 AD7606 需要对下面这些知识点有个认识。

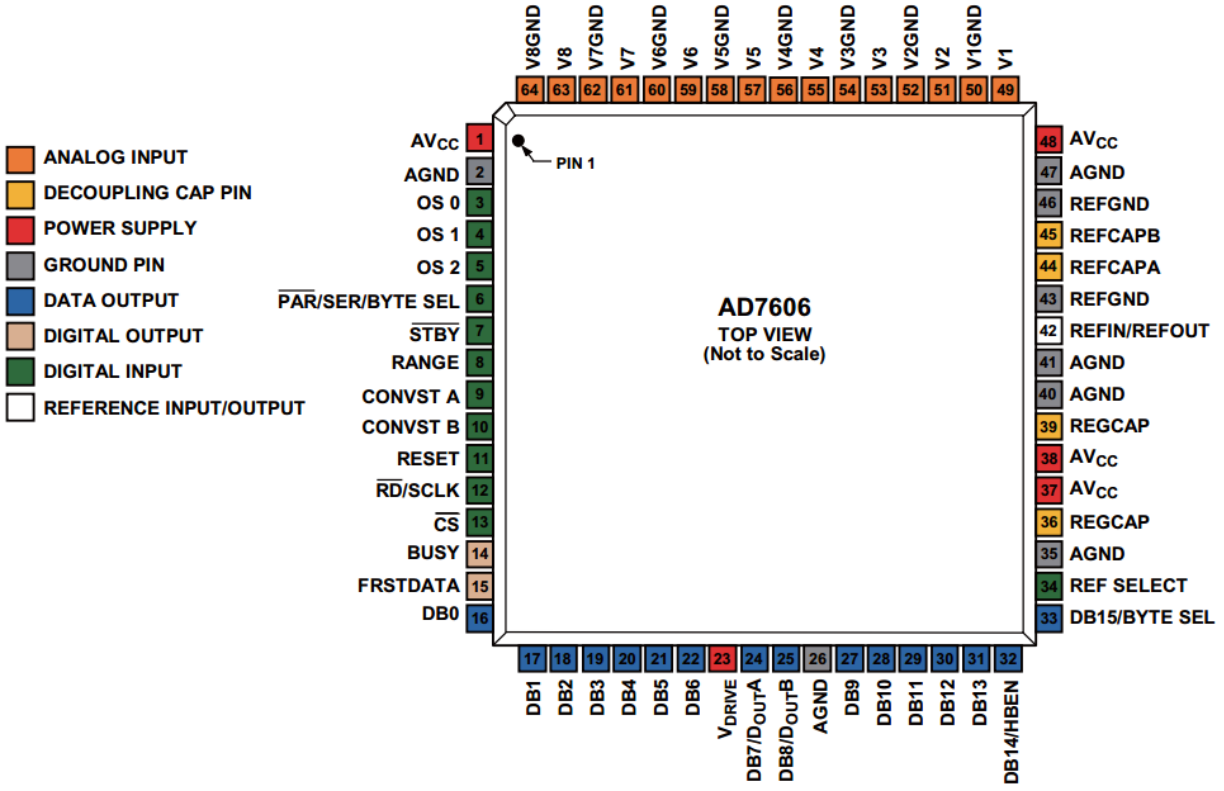
76.4.1 AD7606 基础信息

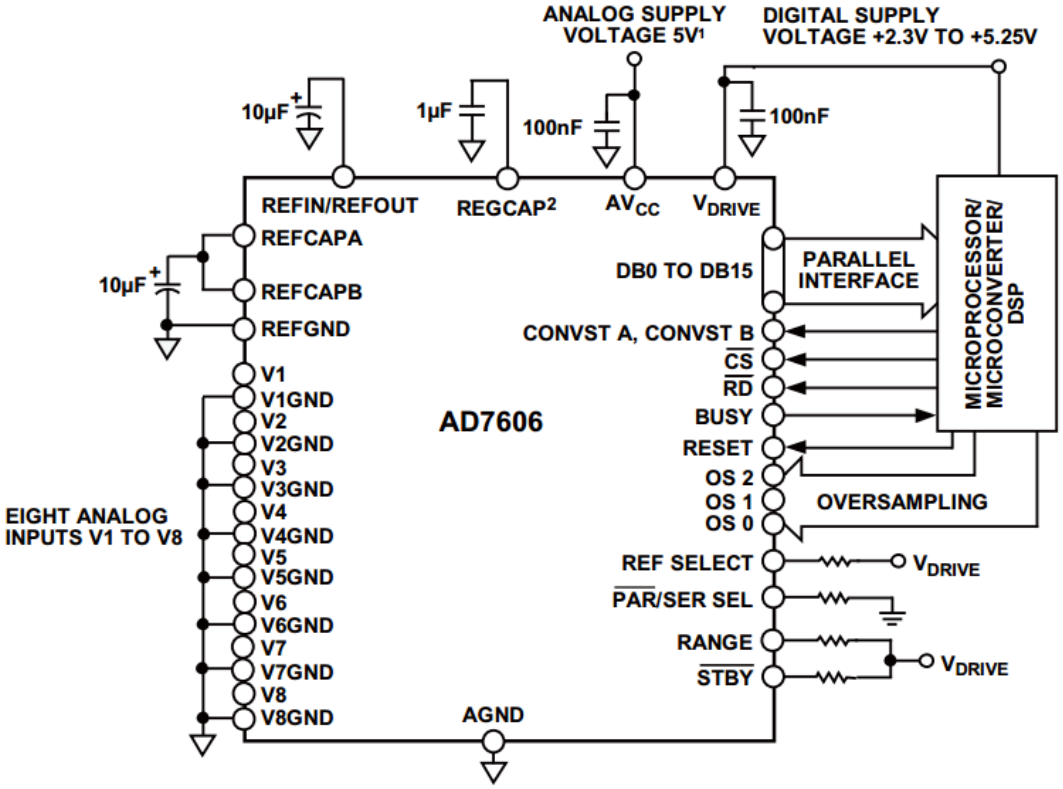
- 支持 8 通道同步采样，每个通道最高 200Ksps，16bit 分辨率。
- 真双极模拟输入范围：±10V、±5V。
- 5V 单模拟电源，V_{DRIVER} 支持 2.3V 到 5V。
- 完全集成的数据采集解决方案：
  - 模拟输入钳位保护，可以耐受±16.5V 的电压。
  - 具有 1MΩ 模拟输入阻抗的输入缓冲器。
  - 二阶抗混叠模拟滤波器。
  - 片内精密基准电压及缓冲。
  - 通过数字滤波器，提供过采样功能。
- 灵活的并行/串行即可，支持 SPI/QSPI/MICROWIRE/DSP 等。
- 性能
  - 模拟输入通道提供 7KV ESD。
  - 95.5dB SNR，-107dB THD，±0.5 LSB INL，±0.5 LSB DNL。
  - 低功耗：100mW。
  - 待机功耗：25mW。



76.4.2 AD7606 常用引脚的作用

AD7606 的封装形式：





这里把常用的几个引脚做个说明：

- AV_{CC}

  模拟电源电压，4.75V 到 5.25V。这是内部前端放大器和 ADC 内核的电源电压。应将这些电压引脚去偶接 AGND。

- AGND

  模拟地，这些引脚是 AD7606 上所有模拟电路的接地基准点。所有模拟输入信号和外部基准信号都应参考这些引脚。

- OS2 OS1 OS2 ：

  组合状态选择过采样模式。

  - 000 表示无过采样，最大 200Ksps 采样速率。
  - 001 表示 2 倍过采样， 也就是硬件内部采集 2 个样本求平均。
  - 010 表示 4 倍过采样， 也就是硬件内部采集 4 个样本求平均。
  - 011 表示 8 倍过采样， 也就是硬件内部采集 8 个样本求平均。
  - 100 表示 16 倍过采样， 也就是硬件内部采集 16 个样本求平均。
  - 101 表示 32 倍过采样， 也就是硬件内部采集 32 个样本求平均。
  - 110 表示 64 倍过采样， 也就是硬件内部采集 64 个样本求平均。

  过采样倍率越高，ADC 转换时间越长，可得到的最大采样频率就越低。

- CONVSTA，CONVSTB ：

启动 AD 转换的控制信号。CONVSTA 决定 1-4 通道，CONVSTB 决定 5-8 通道。2 个信号可以错开短暂的时间。一般情况可以将 CVA，CVB 并联在一起。

- RAGE :

  量程范围选择。0 表示正负 5V, 1 表示正负 10V.

- RD /SCL:

  读信号，低电平有效。

- RESET

  复位信号。

- BUSY :

  CONVST A 和 CONVST B 均达到上升沿后，此引脚变为逻辑高电平，表示转换过程已经开始，BUSY 输出保持高电平，直到所有通道的转换过程完成为止。BUSY 下降沿表示转换数据正被锁存至输出数据寄存器，此时用户就可以读取数据。

- CS :

  片选信号，低电平有效。

- FRST :

  第 1 个通道样本的指示信号。【注，此引脚可以省略不使用】

- V_{Driver}:

  通信接口电平。

- DB0-DB15 :

  数据总线。

- REF SELECT

  内部/外部基准电压选择。如果设置此引脚设为逻辑高电平，使用内部基准电压。如果此引脚设为逻辑低电平，则内部基准电压禁止，必须将外部基准电压加到 REFIN/REFOUT 引脚。

- REFIN/REFOUT

  基准电压输入（REFIN）/基准电压输出（REFOUT）引脚，如果 REF SELECT 引脚设置为逻辑高电平，此引脚将提供 2.5V 片内基准电压供外部使用。或者可以将 REF SELECT 引脚设置为逻辑低电平将禁止用内部基准电压。

- V1 到 V8

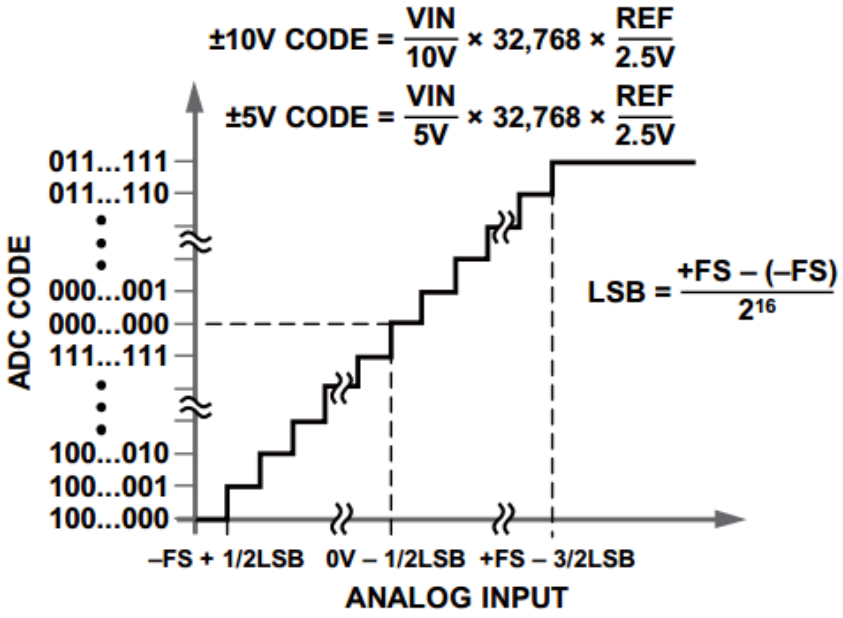
  模拟输入，此引脚为单端模拟输入，此通道的模拟输入范围由 RANGE 引脚决定。

- V1GND 到 V8GND

  模拟输入接地引脚，这些引脚与模拟输入引脚 V1 到 V8 对应，所有模拟输入 AGND 引脚都应连接到系统的 AGND 平面。

76.4.3 AD7606 输出电压计算公式

AD7606 的计算公式如下：



	+FS	MIDSCALE	–FS	LSB
±10V RANGE	+10V	0V	–10V	305μV
±5V RANGE	+5V	0V	–5V	152μV

08479-037

采用二进制补码（其实就是 16bit 有符号数，将转换结果定义为 int16_t 即可），因为 AD7606 支持正负压采集。

- VIN

  AD7606 采集到的电压值范围-32768 到 32767。

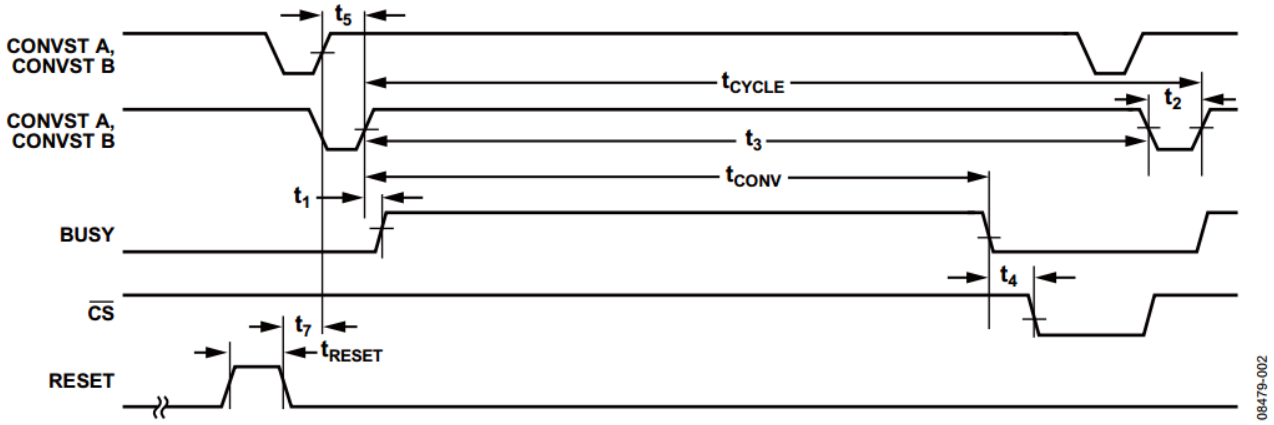
- REF

  一般使用内部基准，即 2.5V。

76.4.4 AD7606 时序图

了解时序参数是驱动 AD7606 能否成功的关键，我们这里对几个重要的参数做个说明。

1. AD7606 的 CONVST 转换时序（转换之后读取数据）：



- t_5

  CONVST A 和 CONVST B 上升沿之间最大允许的延迟时间。一般我们是用一根控制线同时控制 CONVST A 和 CONVST B，因此可以不用管这个时间。

- t_{CYCLE}

  并行模式，转换后并读取数据的最大值是 5us，即最高支持的时钟速度是 20MHz 及其以上。

- t_{CONV}

  转换时间。

参数	在T_{MIN}、T_{MAX}时的限值 最小值	典型值	最大值	单位	描述
并行/串行/字节模式					
t_{CYCLE}					1/吞吐速率
			5	μs	并行模式，转换期间或之后读取；或者串行模式：V_{DRIVE}=4.75 V至5.25 V，利用D_{OUT}A和D_{OUT}B线路在转换期间读取
		5		μs	串行模式，转换期间读取；V_{DRIVE}= 3.3 V
			9.7	μs	串行模式，转换之后读取；V_{DRIVE}= 2.3 V，D_{OUT}A和D_{OUT}B线路
t_{CONV}[2]					转换时间
	3.45	4	4.15	μs	过采样关闭；AD7606
		3		μs	过采样关闭；AD7606-6
		2		μs	过采样关闭；AD7606-4
	7.87		9.1	μs	2倍过采样；AD7606
	16.05		18.8	μs	4倍过采样；AD7606
	33		39	μs	8倍过采样；AD7606
	66		78	μs	16倍过采样；AD7606
	133		158	μs	32倍过采样；AD7606
	257		315	μs	64倍过采样；AD7606

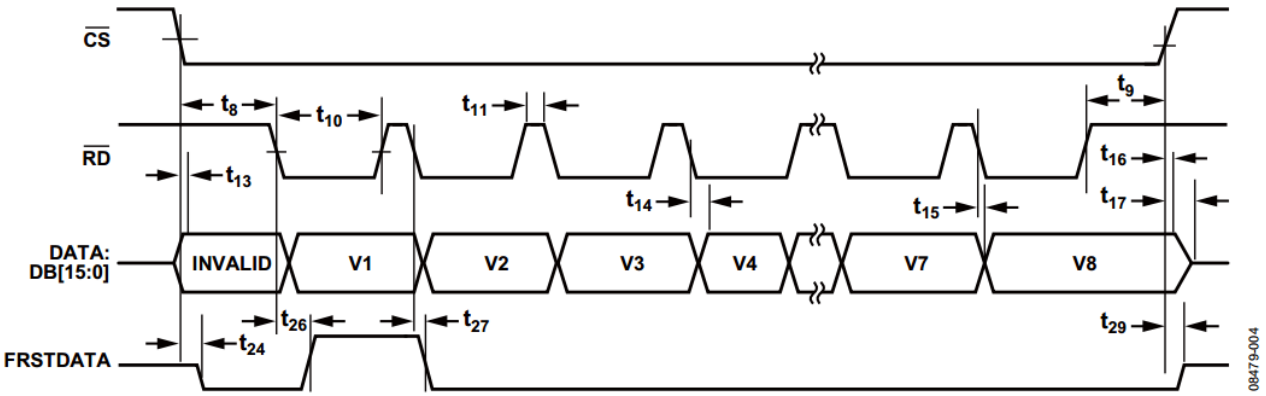
- t_3

  最短的 CONVST A/B 电平脉冲，最小值 25ns。

- t_4

  BUSY 下降沿到 CS 下降沿设置时间，最小值 0ns，所以可以忽略。

2. AD7606 的并行驱动模式有两种时序图，一个是独立的 CS 片选和 RD 读信号时序图：



- t_8

  CS 到 RD 的设置时间，最小值是 0ns，可以忽略。

- t_{10}

  RD 读信号的低电平脉冲宽度，通信电压不同，时间不同。对于 STM32 来说，FMC 通信电平一般是 3.3V，即最小值 21ns。

16	ns	V_{DRIVE}高于4.75 V
21	ns	V_{DRIVE}高于3.3 V
25	ns	V_{DRIVE}高于2.7 V
32	ns	V_{DRIVE}高于2.3 V
15	ns	$\overline{RD}$高电平脉冲宽度
22	ns	$\overline{CS}$高电平脉冲宽度(见图5)；$\overline{CS}$与$\overline{RD}$相连

- t_{11}

  RD 高电平脉冲宽度，最小值 15ns。

- t_9

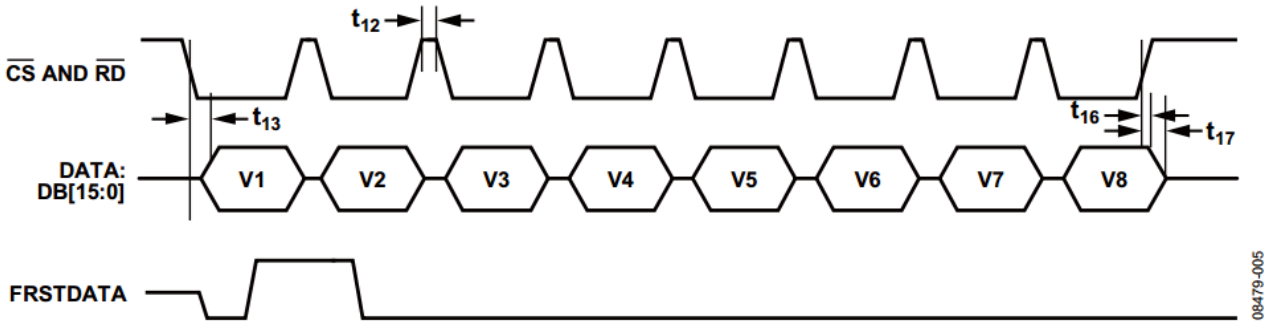
  CS 到 RD 保持时间，最小值 0ns，可以忽略。

- t_{13}到 t_{17}

  这几个参数了解下即可：

参数	在T_{MIN}、T_{MAX}时的限值 最小值	典型值	最大值	单位	描述
t_{13}					从$\overline{CS}$直到DB[15:0]三态禁用的延迟时间
			16	ns	V_{DRIVE}高于4.75 V
			20	ns	V_{DRIVE}高于3.3 V
			25	ns	V_{DRIVE}高于2.7 V
			30	ns	V_{DRIVE}高于2.3 V
t_{14}[4]					$\overline{RD}$下降沿后的数据访问时间
			16	ns	V_{DRIVE}高于4.75 V
			21	ns	V_{DRIVE}高于3.3 V
			25	ns	V_{DRIVE}高于2.7 V
			32	ns	V_{DRIVE}E高于2.3 V
t_{15}	6			ns	$\overline{RD}$下降沿后的数据保持时间
t_{16}	6			ns	$\overline{CS}$到DB[15:0]保持时间
t_{17}			22	ns	从$\overline{CS}$上升沿到DB[15:0]三态使能的延迟时间

3. 另一个是 CS 片选和 RD 相连的方式：



这个时序里面最重要的是 t_{12}。

- t_{12}

  CS 和 RD 的高电平脉冲宽度，最小值 22ns。

第 2 个和第 3 个时序图的主要区别是连续读取 8 路数据时，一个 CS 信号是全程低电平，另一个 CS 信号是与 RD 信号同步，每读取完一路，拉高一次。

76.4.5 AD7606 的过采样

使用过采样可以改善 SNR 信噪比。SNR 性能随着过采样倍率提高而改善，具体参数如下：

OS[2:0]	过采样倍率	5 V范围SNR(dB)	10 V范围SNR(dB)	5 V范围3 dB带宽(kHz)	10 V范围3 dB带宽(kHz)	最大吞吐量CONVST频率(kHz)
000	No OS	89	90	15	22	200
001	2	91.2	92	15	22	100
010	4	92.6	93.6	13.7	18.5	50
011	8	94.2	95	10.3	11.9	25
100	16	95.5	96	6	6	12.5
101	32	96.4	96.7	3	3	6.25
110	64	96.9	97	1.5	1.5	3.125
111	无效					

通过这个表，我们可以方便的了解不同过采样下的信噪比，3dB 带宽时的频率和最高支持的采样率。

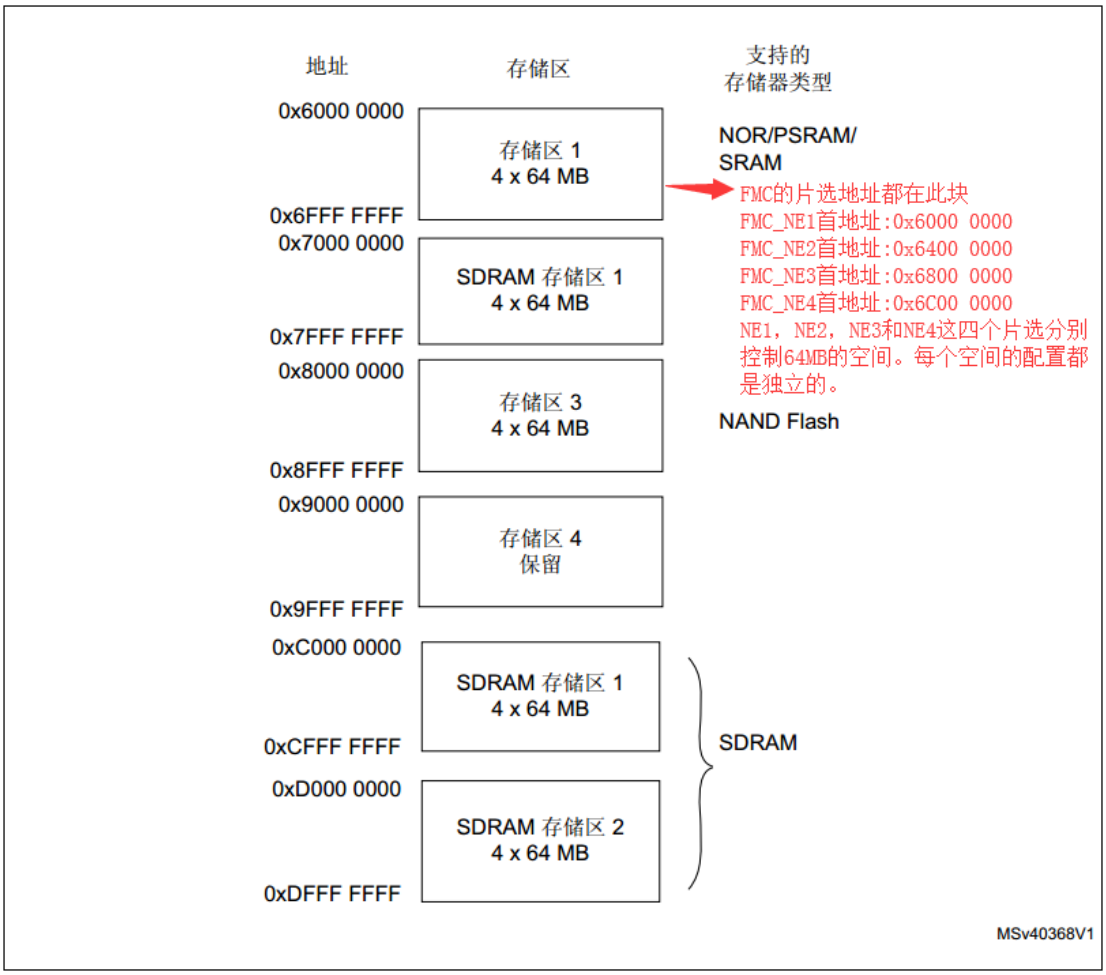
注意正确的理解过采样，比如我们设置是 1Ksps 采样率，64 倍过采样。意思是指每次采样，AD7606 会采样 64 次数据并求平均，相当于 AD7606 以 64Ksps 进行采样的，只是将每 64 个采样点的值做了平均，用户得到的值就是平均后的数值。因此，如果使用 AD7606 最高的 200Ksps 采样率，就不可以使用过采样了。

76.5 AD7606 的 FMC 接口硬件设计

FMC 硬件接口涉及到的知识点稍多，下面逐一为大家做个说明。

76.5.1 FMC 的块区分配

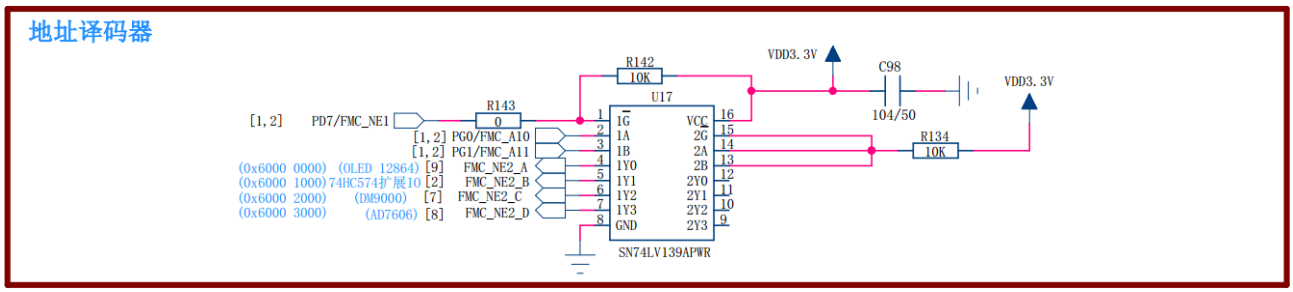
FMC 总线可操作的地址范围 0x60000000 到 0xDFFFFFFF，具体的框图如下：



从上面的框图可以看出，NOR/PSRAM/SRAM 块区有 4 个片选 NE1，NE2，NE3 和 NE4，但由于引脚复用，部分片选对应的引脚要用于其他功能，而且要控制的总线外设较多，导致片选不够用。因此需要增加译码器。

76.5.2 译码器及其地址计算

有了前面的认识之后再来看下面的译码器电路：



SN74LVC1G139APWR 是双 2-4 线地址译码器，也就是带了两个译码器。原理图上仅用了一个。下面是 139 的真值表和引脚功能：

INPUTS		OUTPUTS			
B	A	Y_0	Y_1	Y_2	Y_3
L	L	L	H	H	H
L	H	H	L	H	H
H	L	H	H	L	H
H	H	H	H	H	L

PIN		I/O	DESCRIPTION
NAME	NO.		
A	1	I	Adress input, bit 0
B	2	I	Adress input, bit 1
Y_3	3	O	Output 3, low when B is high and A is high
GND	4	—	Ground
Y_2	5	O	Output 2, low when B is high and A is low
Y_1	6	O	Output 1, low when B is low and A is high
Y_0	7	O	Output 0, low when B is low and A is low
V_{CC}	8	—	Power pin

通过上面的原理图和真值表就比较好理解了，真值表的输出是由片选 FMC_NE1 和地址线 FMC_A10、FMC_A11 控制。

FMC_NE1 输出低电平：

- FMC_A11（B），FMC_A10（A） = 00 时，1Y0 输出的低电平，选择的是 OLED。
- FMC_A11（B），FMC_A10（A） = 01 时，1Y1 输出的低电平，选择的是 74HC574。
- FMC_A11（B），FMC_A10（A） = 10 时，1Y2 输出的低电平，选择的是 DM9000。
- FMC_A11（B），FMC_A10（A） = 11 时，1Y3 输出的低电平，选择的是 AD7606。

然后我们再计算译码器的地址，注意，这里地址的计算都是按照 FMC 的 32bit 访问模式计算的，因为我们的 V7 程序中是将 NE1 对应的 FMC 配置为 32bit 模式了。

具体 FMC 的 32bit 访问模式，16bit 访问模式和 8bit 访问模式的区别在第 47 章的 2.4 小节有详细讲解。

表 248. NOR/PSRAM 外部存储器地址

存储器宽度(1)	向存储器发出的数据地址	最大存储器容量（位）
8 位	HADDR[25:0]	64 MB x 8 = 512 Mb
16 位	HADDR[25:1] >> 1	64 MB/2 x 16 = 512 Mb
32 位	HADDR[25:2] >> 2	64 MB/4 x 32 = 512 Mb

1. 如果外部存储器的宽度为 16 位，FMC 将使用内部的 HADDR[25:1] 地址来作为对外部存储器的寻址地址 FMC_A[24:0]。如果存储器宽度为 32 位，FMC 将使用内部的 HADDR[25:2] 地址进行外部寻址。
无论外部存储器的宽度是多少，FMC_A[0] 都应连接到外部存储器地址 A[0]。

32bit 模式下，我们计算 A10 和 A11 的时候，实际上需要按 HADDR12 和 HADDR13 计算的。

如果来算 NE1 + HADDR12 + HADDR13 的四种组合地址就是如下：

NE1 + HADDR13 + HADDR12 = 0x60000000 + 0<<13 + 0<<12 = 0x60000000

NE1 + HADDR13 + HADDR12 = 0x60000000 + 0<<13 + 1<<12 = 0x60001000

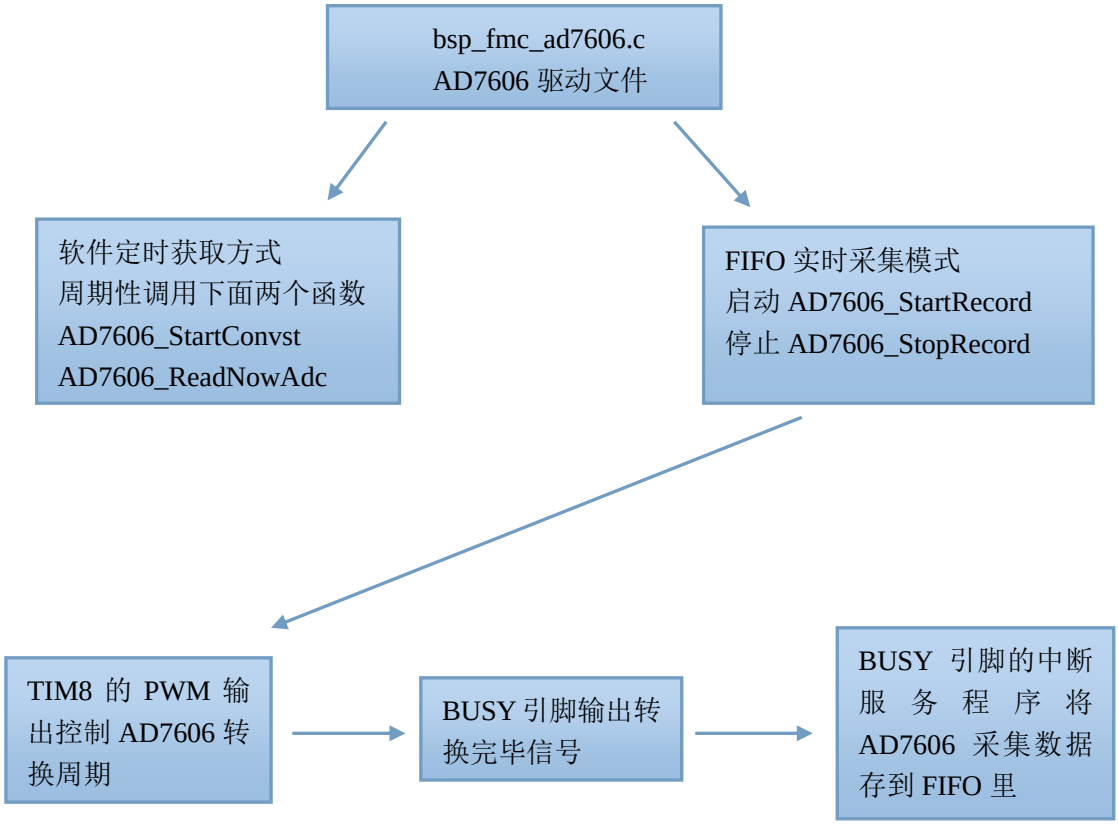
NE1 + HADDR13 + HADDR12 = 0x60000000 + 1<<13 + 0<<12 = 0x60002000

NE1 + HADDR13 + HADDR12 = 0x60000000 + 1<<13 + 1<<12 = 0x60003000

这样一来，原理图里面给的地址就对应上了。同理如果配置为 16 位模式和 8 位模式，大家应该也都会计算了。

76.6 AD7606 的 FMC 接口驱动设计

AD7606 的程序驱动框架设计如下：



有了这个框图，程序设计就比较好理解了。

76.6.1 第 1 步，AD7606 整体驱动框架设计

主要实现了两种采集方式：

（1）软件定时获取方式，适合低速查询获取。

（2）FIFO 工作模式，适合 8 路实时采集，支持最高采样率 200Ksps。

◆ 方案一：软件定时获取方式代码框架：

可以在硬件定时器中断服务程序或者软件定时器里面实现。

```
定时器中断 ISR：
{
    中断入口；
    读取 8 个通道的采样结果保存到 RAM；    ----> 读取的是上次的采集结果，对于连续采集来说，是没有关系的
    启动下次 ADC 采集；（翻转 CVA 和 CVB）
    中断返回；
}
```

定时器的频率就是 ADC 采样频率。这种模式可以不连接 BUSY 口线。

◆ 方案二：FIFO 工作模式框架：

配置 CVA、CVB 引脚为 PWM 输出模式，周期设置为需要的采样频率，之后 MCU 将产生周期非常稳定的 AD 转换信号

将 BUSY 口线设置为中断下降沿触发模式；

```
外部中断 ISR：
{
    中断入口；
    读取 8 个通道的采样结果保存到 RAM；
}
```

◆ 方案 1 和方案 2 的差异

（1）方案 1 可以少用 BUSY 口线，但是其他中断服务程序或者主程序临时关闭全局中断时，可能导致 ADC 转换周期存在轻微抖动。

（2）方案 2 可以确保采集时钟的稳定性，因为它是 MCU 硬件产生的，但是需要多接一根 BUSY 口线。

76.6.2 第 2 步，AD7606 所涉及到的 GPIO 配置

这里需要把用到的 GPIO 时钟、FMC 时钟、GPIO 引脚和复用配置好即可：

```
/*
*********************************************************************************************************
*    函 数 名: AD7606_CtrlLinesConfig
*    功能说明: 配置 GPIO 口线，FMC 管脚设置为复用功能
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
/*
    安富莱 STM32-H7 开发板接线方法：4 片 74HC574 挂在 FMC 32 位总线上。1 个地址端口可以扩展出 32 个 IO
    PD0/FMC_D2
    PD1/FMC_D3
    PD4/FMC_NOE        ---- 读控制信号，OE = Output Enable ， N 表示低有效
    PD5/FMC_NWE        -XX- 写控制信号，AD7606 只有读，无写信号
    PD8/FMC_D13
    PD9/FMC_D14
    PD10/FMC_D15
```

```
    PD14/FMC_D0
    PD15/FMC_D1

    PE7/FMC_D4
    PE8/FMC_D5
    PE9/FMC_D6
    PE10/FMC_D7
    PE11/FMC_D8
    PE12/FMC_D9
    PE13/FMC_D10
    PE14/FMC_D11
    PE15/FMC_D12

    PG0/FMC_A10         --- 和主片选 FMC_NE2 一起译码
    PG1/FMC_A11         --- 和主片选 FMC_NE2 一起译码
    PD7/FMC_NE1         --- 主片选（OLED, 74HC574, DM9000, AD7606）

     +------------------+------------------+
     +   32-bits Mode: D31-D16              +
     +------------------+------------------+
     | PH8 <-> FMC_D16   | PI0 <-> FMC_D24  |
     | PH9 <-> FMC_D17   | PI1 <-> FMC_D25  |
     | PH10 <-> FMC_D18  | PI2 <-> FMC_D26  |
     | PH11 <-> FMC_D19  | PI3 <-> FMC_D27  |
     | PH12 <-> FMC_D20  | PI6 <-> FMC_D28  |
     | PH13 <-> FMC_D21  | PI7 <-> FMC_D29  |
     | PH14 <-> FMC_D22  | PI9 <-> FMC_D30  |
     | PH15 <-> FMC_D23  | PI10 <-> FMC_D31 |
     +------------------+------------------+
*/

/*
    控制 AD7606 参数的其他 IO 分配在扩展的 74HC574 上
    X13 - AD7606_OS0
    X14 - AD7606_OS1
    X15 - AD7606_OS2
    X24 - AD7606_RESET
    X25 - AD7606_RAGE

    PE5 - AD7606_BUSY
*/
static void AD7606_CtrlLinesConfig(void)
{
    /* bsp_fm_io 已配置 fmc, bsp_InitExtIO();
       此处可以不必重复配置
    */

    GPIO_InitTypeDef gpio_init_structure;

    /* 使能 GPIO 时钟 */
    __HAL_RCC_GPIOD_CLK_ENABLE();
    __HAL_RCC_GPIOE_CLK_ENABLE();
    __HAL_RCC_GPIOF_CLK_ENABLE();
    __HAL_RCC_GPIOG_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOI_CLK_ENABLE();

    /* 使能 FMC 时钟 */
```

```
    __HAL_RCC_FMC_CLK_ENABLE();

    /* 设置 GPIOD 相关的 IO 为复用推挽输出 */
    gpio_init_structure.Mode = GPIO_MODE_AF_PP;
    gpio_init_structure.Pull = GPIO_PULLUP;
    gpio_init_structure.Speed = GPIO_SPEED_FREQ_HIGH;
    gpio_init_structure.Alternate = GPIO_AF12_FMC;

    /* 配置 GPIOD */
    gpio_init_structure.Pin = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_7 |
                              GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 | GPIO_PIN_14 |
                              GPIO_PIN_15;
    HAL_GPIO_Init(GPIOD, &gpio_init_structure);

    /* 配置 GPIOE */
    gpio_init_structure.Pin = GPIO_PIN_7 | GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 |
                              GPIO_PIN_11 | GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14 |
                              GPIO_PIN_15;
    HAL_GPIO_Init(GPIOE, &gpio_init_structure);

    /* 配置 GPIOG */
    gpio_init_structure.Pin = GPIO_PIN_0 | GPIO_PIN_1;
    HAL_GPIO_Init(GPIOG, &gpio_init_structure);

    /* 配置 GPIOH */
    gpio_init_structure.Pin = GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 | GPIO_PIN_11 | GPIO_PIN_12
                            | GPIO_PIN_13 | GPIO_PIN_14 | GPIO_PIN_15;
    HAL_GPIO_Init(GPIOH, &gpio_init_structure);

    /* 配置 GPIOI */
    gpio_init_structure.Pin = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3 | GPIO_PIN_6
                            | GPIO_PIN_7 | GPIO_PIN_9 | GPIO_PIN_10;
    HAL_GPIO_Init(GPIOI, &gpio_init_structure);


    /* 配置 BUSY 引脚，默认是普通 IO 状态 */
    {
        GPIO_InitTypeDef   GPIO_InitStructure;

        __HAL_RCC_SYSCFG_CLK_ENABLE();

        BUSY_RCC_GPIO_CLK_ENABLE();      /* 打开 GPIO 时钟 */

        /* BUSY 信号，使用的 PE5，用于转换完毕检测 */
        GPIO_InitStructure.Mode = GPIO_MODE_INPUT;   /* 设置推挽输出 */
        GPIO_InitStructure.Pull = GPIO_NOPULL;       /* 无上拉下拉 */
        GPIO_InitStructure.Pin = BUSY_PIN;
        HAL_GPIO_Init(BUSY_GPIO, &GPIO_InitStructure);
    }

    /* CONVST 启动 ADC 转换的 GPIO = PC6 */
    {
        GPIO_InitTypeDef   GPIO_InitStructure;
        CONVST_RCC_GPIO_CLK_ENABLE();

        /* 配置 PC6 */
        GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;      /* 设置推挽输出 */
        GPIO_InitStructure.Pull = GPIO_NOPULL;              /* 上下拉电阻不使能 */
```

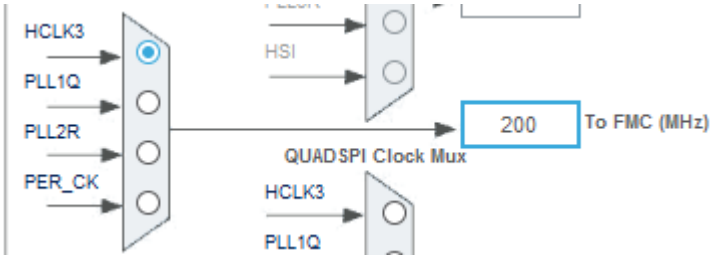
```
        GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_MEDIUM;  /* GPIO 速度等级 */

        GPIO_InitStructure.Pin = CONVST_PIN;
        HAL_GPIO_Init(CONVST_GPIO, &GPIO_InitStructure);
    }
}
```

这里重点注意 AD7606_CONVST 和 AD7606_BUSY 引脚，上电后的默认配置是普通 IO。另外还有过采样的 3 个引脚，量程配置的 1 个引脚和复位控制的 1 个引脚，均通过 V7 板子的扩展 IO 实现：

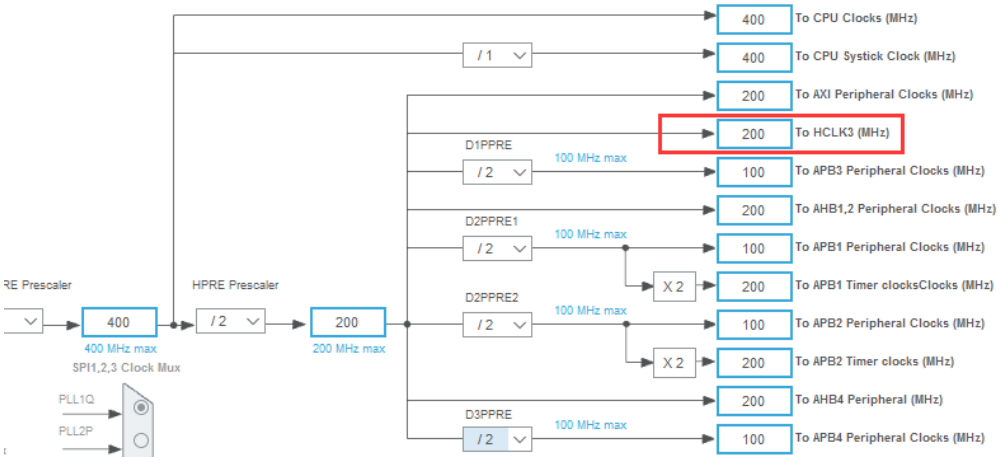
```
/* 设置过采样的 IO，在扩展的 74HC574 上 */
#define OS0_1()         HC574_SetPin(AD7606_OS0, 1)
#define OS0_0()         HC574_SetPin(AD7606_OS0, 0)
#define OS1_1()         HC574_SetPin(AD7606_OS1, 1)
#define OS1_0()         HC574_SetPin(AD7606_OS1, 0)
#define OS2_1()         HC574_SetPin(AD7606_OS2, 1)
#define OS2_0()         HC574_SetPin(AD7606_OS2, 0)

/* 设置输入量程的 GPIO，在扩展的 74HC574 上 */
#define RANGE_1()  HC574_SetPin(AD7606_RANGE, 1)
#define RANGE_0()  HC574_SetPin(AD7606_RANGE, 0)

/* AD7606 复位口线，在扩展的 74HC574 上 */
#define RESET_1()  HC574_SetPin(AD7606_RESET, 1)
#define RESET_0()  HC574_SetPin(AD7606_RESET, 0)
```

76.6.3第 3 步，FMC 的时钟源选择

使用 FMC 可以选择如下几种时钟源 HCLK3，PLL1Q，PLL2R 和 PER_CK：

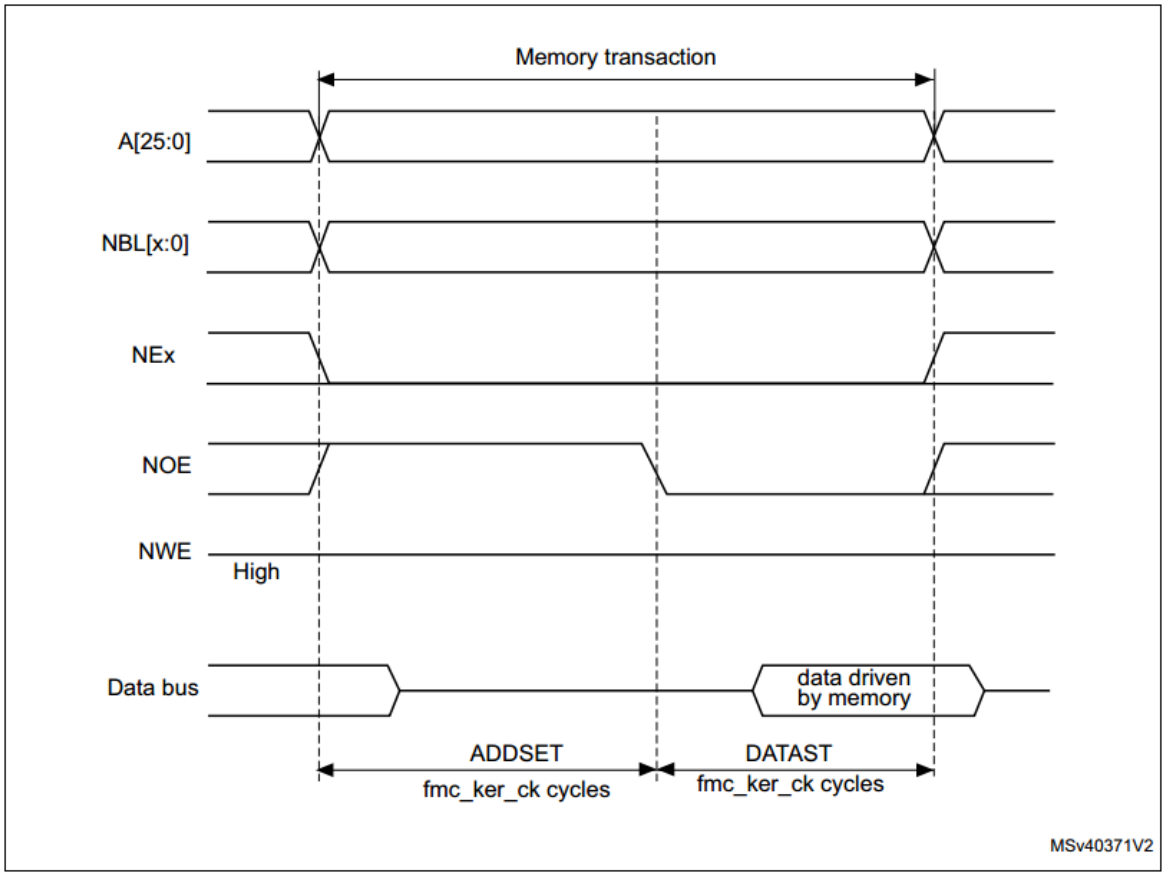


我们这里直接使用 HCLK3，配置 STM32H7 的主频为 400MHz 的时候，HCLK3 输出的 200MHz，这个速度是 FMC 支持的最高时钟，正好用于这里：



76.6.4第 4 步，FMC 的时序配置（重要）

由于操作 AD7606 仅需要读操作，而且使用的是 FMC 总线的 Mode_A，那么仅需按照如下时序图配置好即可：



根据这个时序图，重点配置好 ADDSET 地址建立时间和 DATAST 数据建立时间即可。

- DATAST（DataSetupTime，数据建立时间）

  DATAST 实际上对应的就是 76.4.4 小节里面的 t_{10} 。RD 读信号的低电平脉冲宽度，通信电压不同，时间不同，对于 STM32 来说，FMC 通信电平一般是 3.3V，即最小值 21ns。

16	ns	V_{DRIVE}高于4.75 V
21	ns	V_{DRIVE}高于3.3 V
25	ns	V_{DRIVE}高于2.7 V
32	ns	V_{DRIVE}高于2.3 V
15	ns	$\overline{RD}$高电平脉冲宽度
22	ns	$\overline{CS}$高电平脉冲宽度(见图5)；$\overline{CS}$与$\overline{RD}$相连

◆ ADDST（AddressSetupTime，地址建立时间）

DATAST 实际上对应的就是 76.4.4 小节里面的 t_{11} 或者 t_{12}。

- 如果采用 CS（NEx）片选和 RD（NOE）读信号独立方式，对应的时间最小 15ns，即 t_{11} 。
- 如果采用 CS（NEx）片选和 RD（NOE）读信号并联方式，对应的时间最小 22ns，即 t_{12} 。

我们这里将 t_{12} 作为最小值更合理，因为 CS（NEx）片选信号，每读取完毕一路，拉高一次。

有了这些认识后，再来看 FMC 的时序配置就比较好理解了：

```
/*
*********************************************************************************************************
*    函 数 名: AD7606_FSMCConfig
*    功能说明: 配置 FSMC 并口访问时序
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void AD7606_FSMCConfig(void)
{
    /*
       DM9000，扩展 IO，OLED 和 AD7606 公用一个 FMC 配置，如果都开启，请以 FMC 速度最慢的为准。
       从而保证所有外设都可以正常工作。
    */
    SRAM_HandleTypeDef hsram = {0};
    FMC_NORSRAM_TimingTypeDef SRAM_Timing = {0};

   /*
    AD7606 规格书要求(3.3V 时，通信电平 Vdriver)：RD 读信号低电平脉冲宽度最短 21ns，对应 DataSetupTime
    CS 片选和 RD 读信号独立方式的高电平脉冲最短宽度 15ns。
    CS 片选和 RD 读信号并联方式的高电平脉冲最短宽度 22ns。
    这里将 22ns 作为最小值更合理些，对应 FMC 的 AddressSetupTime。

         5-x-5-x-x-x  : RD 高持续 25ns， 低电平持续 25ns. 读取 8 路样本数据到内存差不多就是 400ns。
    */
    hsram.Instance  = FMC_NORSRAM_DEVICE;
    hsram.Extended  = FMC_NORSRAM_EXTENDED_DEVICE;

    /* FMC 使用的 HCLK3，主频 200MHz，1 个 FMC 时钟周期就是 5ns */
    SRAM_Timing.AddressSetupTime       = 5; /* 5*5ns=25ns，地址建立时间，范围 0 -15 个 FMC 时钟周期个数 */
    SRAM_Timing.AddressHoldTime        = 2; /* 地址保持时间，配置为模式 A 时，用不到此参数 范围 1 -15 个
                                               时钟周期个数 */
    SRAM_Timing.DataSetupTime          = 5;  /* 5*5ns=25ns，数据建立时间，范围 1 -255 个时钟周期个数 */
    SRAM_Timing.BusTurnAroundDuration  = 1;  /* 此配置用不到这个参数 */
    SRAM_Timing.CLKDivision            = 2;  /* 此配置用不到这个参数 */
    SRAM_Timing.DataLatency            = 2;  /* 此配置用不到这个参数 */
    SRAM_Timing.AccessMode             = FMC_ACCESS_MODE_A; /* 配置为模式 A */
```

```
89.     hsram.Init.NSBank             = FMC_NORSRAM_BANK1;               /* 使用的 BANK1，即使用的片选
90.                                                                        FMC_NE1 */
91.     hsram.Init.DataAddressMux     = FMC_DATA_ADDRESS_MUX_DISABLE;    /* 禁止地址数据复用 */
92.     hsram.Init.MemoryType         = FMC_MEMORY_TYPE_SRAM;            /* 存储器类型 SRAM */
93.     hsram.Init.MemoryDataWidth    = FMC_NORSRAM_MEM_BUS_WIDTH_32;    /* 32 位总线宽度 */
94.     hsram.Init.BurstAccessMode    = FMC_BURST_ACCESS_MODE_DISABLE;   /* 关闭突发模式 */
95.     hsram.Init.WaitSignalPolarity = FMC_WAIT_SIGNAL_POLARITY_LOW;    /* 用于设置等待信号的极性，关闭突
96.                                                                        发模式，此参数无效 */
97.     hsram.Init.WaitSignalActive   = FMC_WAIT_TIMING_BEFORE_WS;       /* 关闭突发模式，此参数无效 */
98.     hsram.Init.WriteOperation     = FMC_WRITE_OPERATION_ENABLE;      /* 用于使能或者禁止写保护 */
99.     hsram.Init.WaitSignal         = FMC_WAIT_SIGNAL_DISABLE;         /* 关闭突发模式，此参数无效 */
100.    hsram.Init.ExtendedMode       = FMC_EXTENDED_MODE_DISABLE;       /* 禁止扩展模式 */
101.    hsram.Init.AsynchronousWait   = FMC_ASYNCHRONOUS_WAIT_DISABLE;   /* 用于异步传输期间，使能或者禁止
102.                                                                       等待信号，这里选择关闭 */
103.    hsram.Init.WriteBurst         = FMC_WRITE_BURST_DISABLE;         /* 禁止写突发 */
104.    hsram.Init.ContinuousClock    = FMC_CONTINUOUS_CLOCK_SYNC_ONLY;  /* 仅同步模式才做时钟输出 */
105.    hsram.Init.WriteFifo          = FMC_WRITE_FIFO_ENABLE;           /* 使能写 FIFO */
106.
107.    /* 初始化 SRAM 控制器 */
108.    if (HAL_SRAM_Init(&hsram, &SRAM_Timing, &SRAM_Timing) != HAL_OK)
109.    {
110.        /* 初始化错误 */
111.        Error_Handler(__FILE__, __LINE__);
112.    }
113. }
```

这里把几个关键的地方阐释下：

- 第 15- 16 行，对作为局部变量的 HAL 库结构体做初始化，防止不确定值配置时出问题。
- 第 30 行，地址建立时间，对于 AD7606 来说，这个地方最小值 22ns。保险起见，这里取值 5 个 FMC 时钟周期，即 25ns。
- 第 31 行，地址保持时间，对于 FMC 模式 A 来说，此参数用不到。
- 第 33 行，数据建立时间，对于 AD7606 来说，这个地方最小值是 21ns，保险起见，这里取值 5 个 FMC 时钟周期，即 25ns。
- 第 34 – 36 行，当前配置用不到这三个参数。
- 第 38 行，使用的 BANK1，即使用的片选 FMC_NE1。

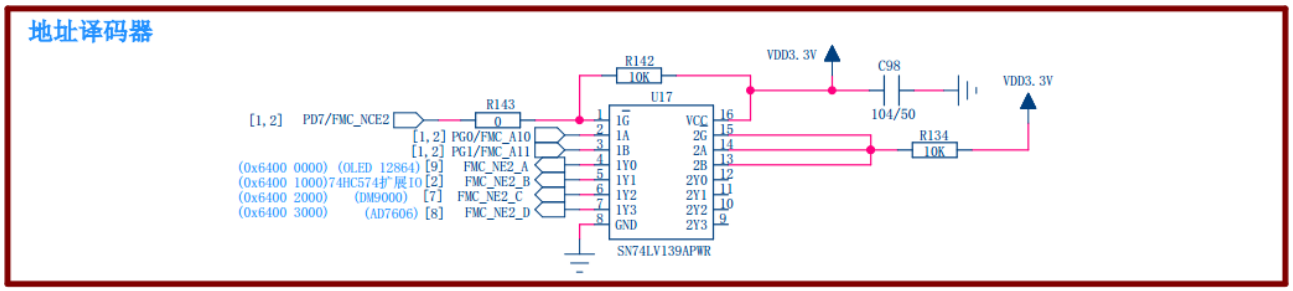
76.6.5第 5 步，FMC 的 MPU 配置

实际测试发现，使能 FMC_NE1 所管理的存储区的 Cache 功能后，会出现扩展 IO 的 NE 片选和 NWE 信号输出 2 次的问题。经过各种 Cache 方式配置、FMC 带宽配置、操作 FMC 时的数据位宽设置，发现禁止了 Cache 功能就正常了，也就是说，设置 FMC_NE1 所管理的存储区 MPU 属性为 Device 或者 Strongly Ordered 即可。

```
    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
```

```
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

MPU 配置中直接从 FMC_NE1 的首地址开始配置，设置了 64KB 空间的属性。将 FMC_NE1 通过译码器所管理的所有设备地址全部设置为此配置：



76.6.6第 6 步，AD7606 的软件定时器读取数据（方案一）

AD7606 的软件定时器读取方式比较简单，周期性调用下面两个函数即可：

```
AD7606_ReadNowAdc();        /* 读取采样结果 */
AD7606_StartConvst();       /* 启动下次转换 */
```

函数 AD7606_ReadNowAdc 的实现如下：

```
/* AD7606 FSMC 总线地址，只能读，无需写 */
#define AD7606_RESULT()*(__IO uint16_t *)0x60003000

void AD7606_ReadNowAdc(void)
{
    g_tAD7606.sNowAdc[0] = AD7606_RESULT();   /* 读第 1 路样本 */
    g_tAD7606.sNowAdc[1] = AD7606_RESULT();   /* 读第 2 路样本 */
    g_tAD7606.sNowAdc[2] = AD7606_RESULT();   /* 读第 3 路样本 */
    g_tAD7606.sNowAdc[3] = AD7606_RESULT();   /* 读第 4 路样本 */
    g_tAD7606.sNowAdc[4] = AD7606_RESULT();   /* 读第 5 路样本 */
    g_tAD7606.sNowAdc[5] = AD7606_RESULT();   /* 读第 6 路样本 */
    g_tAD7606.sNowAdc[6] = AD7606_RESULT();   /* 读第 7 路样本 */
    g_tAD7606.sNowAdc[7] = AD7606_RESULT();   /* 读第 8 路样本 */

    AD7606_SEGGER_RTTOUT();
}
```

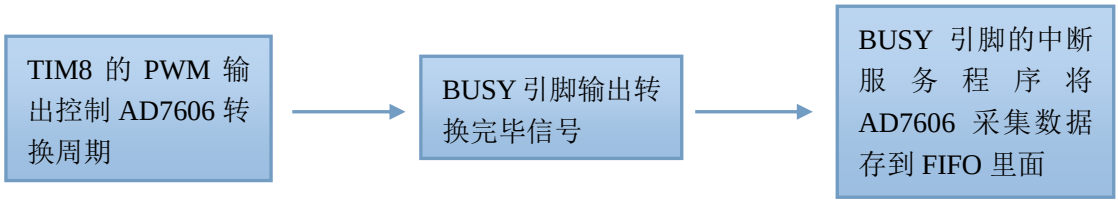
启动 ADC 转换的函数实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: AD7606_StartConvst
*    功能说明: 启动 1 次 ADC 转换
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
```

```
void AD7606_StartConvst(void)
{
    /* page 7:  CONVST 高电平脉冲宽度和低电平脉冲宽度最短 25ns */
    /* CONVST 平时为高 */
    CONVST_0();
    CONVST_0();
    CONVST_0();

    CONVST_1();
}
```

76.6.7第 7 步，AD7606 的 FIFO 方式实时读取数据（方案二）

通过下面的框图可以对 AD7606 的 FIFO 方式有个整体认识：



- 启动采集函数 AD7606_StartRecord

这个函数的主要作用是配置 TIM8 的 CH1 PWM 输出并使能 BUSY 引脚的 EXTI 中断。

```
/*
*********************************************************************************************************
*    函 数 名: AD7606_StartRecord
*    功能说明: 开始采集
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void AD7606_StartRecord(uint32_t _ulFreq)
{
    AD7606_StopRecord();

    AD7606_Reset();             /* 复位硬件 */
    AD7606_StartConvst();       /* 启动采样，避免第 1 组数据全 0 的问题 */

    g_tAdcFifo.usRead = 0;      /* 必须在开启定时器之前清 0 */
    g_tAdcFifo.usWrite = 0;
    g_tAdcFifo.usCount = 0;
    g_tAdcFifo.ucFull = 0;

    AD7606_EnterAutoMode(_ulFreq);
}
/*
*********************************************************************************************************
*    函 数 名: AD7606_EnterAutoMode
*    功能说明: 配置硬件工作在自动采集模式，结果存储在 FIFO 缓冲区。
*    形    参:  _ulFreq : 采样频率，单位 Hz，   1k，2k，5k，10k，20K，50k，100k，200k
*    返 回 值: 无
```

```
*********************************************************************************************************
*/
void AD7606_EnterAutoMode(uint32_t _ulFreq)
{
    /* 配置 PC6 为 TIM8_CH1 功能，输出占空比 50%的方波 */
    bsp_SetTIMOutPWM(CONVST_GPIO, CONVST_PIN, CONVST_TIMX,  CONVST_TIMCH, _ulFreq, 5000);

    /* 配置 PE5, BUSY 作为中断输入口，下降沿触发 */
    {
        GPIO_InitTypeDef   GPIO_InitStructure;

        CONVST_RCC_GPIO_CLK_ENABLE();    /* 打开 GPIO 时钟 */
        __HAL_RCC_SYSCFG_CLK_ENABLE();

        GPIO_InitStructure.Mode = GPIO_MODE_IT_FALLING;     /* 中断下降沿触发 */
        GPIO_InitStructure.Pull = GPIO_NOPULL;
        GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_LOW;
        GPIO_InitStructure.Pin = BUSY_PIN;
        HAL_GPIO_Init(BUSY_GPIO, &GPIO_InitStructure);

        HAL_NVIC_SetPriority(BUSY_IRQn, 2, 0);
        HAL_NVIC_EnableIRQ(BUSY_IRQn);
    }
}
```

- AD7606 转换完毕后，中断服务程序的处理。

  下面这几个函数的调用关系是

  - EXTI9_5_IRQHandler 调用 HAL_GPIO_EXTI_IRQHandler。
  - HAL_GPIO_EXTI_IRQHandler 调用 HAL_GPIO_EXTI_Callback。
  - HAL_GPIO_EXTI_Callback 调用 AD7606_ISR。
  - AD7606_ISR 调用 AD7606_ReadNowAdc。

```
/*
*********************************************************************************************************
*    函 数 名: EXTI9_5_IRQHandler
*    功能说明: 外部中断服务程序。
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void EXTI9_5_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(BUSY_PIN);
}

/*
*********************************************************************************************************
*    函 数 名: EXTI9_5_IRQHandler
*    功能说明: 外部中断服务程序入口, AD7606_BUSY 下降沿中断触发
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == BUSY_PIN)
```

```
    {
        AD7606_ISR();
    }
}

/*
*********************************************************************************************************
*    函 数 名: AD7606_ISR
*    功能说明: 定时采集中断服务程序
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void AD7606_ISR(void)
{
    uint8_t i;

    AD7606_ReadNowAdc();

    for (i = 0; i < 8; i++)
    {
        g_tAdcFifo.sBuf[g_tAdcFifo.usWrite] = g_tAD7606.sNowAdc[i];
        if (++g_tAdcFifo.usWrite >= ADC_FIFO_SIZE)
        {
            g_tAdcFifo.usWrite = 0;
        }
        if (g_tAdcFifo.usCount < ADC_FIFO_SIZE)
        {
            g_tAdcFifo.usCount++;
        }
        else
        {
            g_tAdcFifo.ucFull = 1;      /* FIFO 满，主程序来不及处理数据 */
        }
    }
}

/*
*********************************************************************************************************
*    函 数 名: AD7606_ReadNowAdc
*    功能说明: 读取 8 路采样结果。结果存储在全局变量 g_tAD7606
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void AD7606_ReadNowAdc(void)
{
    g_tAD7606.sNowAdc[0] = AD7606_RESULT();   /* 读第 1 路样本 */
    g_tAD7606.sNowAdc[1] = AD7606_RESULT();   /* 读第 2 路样本 */
    g_tAD7606.sNowAdc[2] = AD7606_RESULT();   /* 读第 3 路样本 */
    g_tAD7606.sNowAdc[3] = AD7606_RESULT();   /* 读第 4 路样本 */
    g_tAD7606.sNowAdc[4] = AD7606_RESULT();   /* 读第 5 路样本 */
    g_tAD7606.sNowAdc[5] = AD7606_RESULT();   /* 读第 6 路样本 */
    g_tAD7606.sNowAdc[6] = AD7606_RESULT();   /* 读第 7 路样本 */
    g_tAD7606.sNowAdc[7] = AD7606_RESULT();   /* 读第 8 路样本 */

    AD7606_SEGGER_RTTOUT();
}
```

这里的 FIFO 比较好理解，与前面按键 FIFO 章节的实现是一样的，详情可重温下按键 FIFO 的实现。

76.6.8第 8 步，AD7606 的双缓冲方式存储思路

为了方便大家实时处理采集的数据，专门预留了一个弱定义函数 AD7606_SEGGER_RTTOUT，方便大家将采集函数存储到双缓冲里面，这个函数是在中断服务程序里面调用的。

```
/*
*********************************************************************************************************
*    函 数 名: AD7606_ReadNowAdc
*    功能说明: 读取 8 路采样结果。结果存储在全局变量 g_tAD7606
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
/* 弱定义，方便用户将采集的结果实时输出 */
__weak void AD7606_SEGGER_RTTOUT(void)
{

}

void AD7606_ReadNowAdc(void)
{
    g_tAD7606.sNowAdc[0] = AD7606_RESULT();   /* 读第 1 路样本 */
    g_tAD7606.sNowAdc[1] = AD7606_RESULT();   /* 读第 2 路样本 */
    g_tAD7606.sNowAdc[2] = AD7606_RESULT();   /* 读第 3 路样本 */
    g_tAD7606.sNowAdc[3] = AD7606_RESULT();   /* 读第 4 路样本 */
    g_tAD7606.sNowAdc[4] = AD7606_RESULT();   /* 读第 5 路样本 */
    g_tAD7606.sNowAdc[5] = AD7606_RESULT();   /* 读第 6 路样本 */
    g_tAD7606.sNowAdc[6] = AD7606_RESULT();   /* 读第 7 路样本 */
    g_tAD7606.sNowAdc[7] = AD7606_RESULT();   /* 读第 8 路样本 */

    AD7606_SEGGER_RTTOUT();
}
```

本章是将此函数用于实时采集数据并输出到 J-Scope。

76.6.9第 9 步，AD7606 过采样设置

AD7606 的过采样实现比较简单，通过 IO 引脚就可以控制，支持 2 倍，4 倍，8 倍，16 倍，32 倍和 64 倍过采样设置。

```
/*
*********************************************************************************************************
*    函 数 名: AD7606_SetOS
*    功能说明: 配置 AD7606 数字滤波器，也就设置过采样倍率。
*              通过设置 AD7606_OS0、OS1、OS2 口线的电平组合状态决定过采样倍率。
*              启动 AD 转换之后，AD7606 内部自动实现剩余样本的采集，然后求平均值输出。
*
*              过采样倍率越高，转换时间越长。
*              0、无过采样时，AD 转换时间 = 3.45us - 4.15us
*              1、2 倍过采样时 = 7.87us - 9.1us
*              2、4 倍过采样时 = 16.05us - 18.8us
```

```
*                3、8 倍过采样时 = 33us - 39us
*                4、16 倍过采样时 = 66us - 78us
*                5、32 倍过采样时 = 133us - 158us
*                6、64 倍过采样时 = 257us - 315us
*
*    形    参: _ucOS : 过采样倍率, 0 - 6
*    返 回 值: 无
*********************************************************************************************************
*/
void AD7606_SetOS(uint8_t _ucOS)
{
    g_tAD7606.ucOS = _ucOS;
    switch (_ucOS)
    {
        case AD_OS_X2:
            OS2_0();
            OS1_0();
            OS0_1();
            break;

        case AD_OS_X4:
            OS2_0();
            OS1_1();
            OS0_0();
            break;

        case AD_OS_X8:
            OS2_0();
            OS1_1();
            OS0_1();
            break;

        case AD_OS_X16:
            OS2_1();
            OS1_0();
            OS0_0();
            break;

        case AD_OS_X32:
            OS2_1();
            OS1_0();
            OS0_1();
            break;

        case AD_OS_X64:
            OS2_1();
            OS1_1();
            OS0_0();
            break;

        case AD_OS_NO:
        default:
            g_tAD7606.ucOS = AD_OS_NO;
            OS2_0();
            OS1_0();
            OS0_0();
            break;
    }
```

```
}
```

76.6.10 第 10 步，AD7606 量程设置

AD7606 支持两种量程，±5V 和±10V，实现代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: AD7606_SetInputRange
*    功能说明: 配置 AD7606 模拟信号输入量程。
*    形    参: _ucRange : 0 表示正负 5V   1 表示正负 10V
*    返 回 值: 无
*********************************************************************************************************
*/
void AD7606_SetInputRange(uint8_t _ucRange)
{
    if (_ucRange == 0)
    {
        g_tAD7606.ucRange = 0;
        RANGE_0();    /* 设置为正负 5V */
    }
    else
    {
        g_tAD7606.ucRange = 1;
        RANGE_1();    /* 设置为正负 10V */
    }
}
```

76.6.11 第 11 步，操作数据位宽注意事项

在 bsp_fmc_ad7606.c 文件开头有个宏定义

```
#define AD7606_RESULT()*(__IO uint16_t *)0x60003000
```

特别注意，这里是要操作地址 0x60003000 上的 16 位数据空间，即做了一个强制转换 uint16_t *。

76.7 AD7606 板级支持包（bsp_fmc_ad7606.c)

AD7606 驱动文件 bsp_fmc_ad7606.c 主要实现了如下几个 API 供用户调用：

- bsp_InitAD7606
- AD7606_SetOS
- AD7606_SetInputRange
- AD7606_Reset
- AD7606_StartConvst
- AD7606_ReadNowAdc
- AD7606_EnterAutoMode
- AD7606_StartRecord

- AD7606_StopRecord
- AD7606_FifoNewData
- AD7606_ReadFifo
- AD7606_FifoFull

76.7.1 函数 bsp_InitAD7606

函数原型：

```
void bsp_InitAD7606(void)
```

函数描述：

主要用于 AD7606 的初始化。

76.7.2 函数 AD7606_SetOS

函数原型：

```
void AD7606_SetOS(uint8_t _ucOS)
```

函数描述：

此函数用于配置 AD7606 数字滤波器，也就设置过采样倍率。通过设置 AD7606_OS0、OS1、OS2 口线的电平组合状态决定过采样倍率。启动 AD 转换之后，AD7606 内部自动实现剩余样本的采集，然后求平均值输出。

过采样倍率越高，转换时间越长。

无过采样时，AD 转换时间 = 3.45us - 4.15us。

2 倍过采样时 = 7.87us - 9.1us。

4 倍过采样时 = 16.05us - 18.8us。

8 倍过采样时 = 33us - 39us。

16 倍过采样时 = 66us - 78us。

32 倍过采样时 = 133us - 158us。

64 倍过采样时 = 257us - 315us。

函数参数：

- 第 1 个参数为范围 0 – 6，分别对应无过采样，2 倍过采样，4 倍过采样，8 倍过采样，16 倍过采样，32 倍过采样和 64 倍过采样。

76.7.3 函数 AD7606_SetInputRange

函数原型：

```
void AD7606_SetInputRange(uint8_t _ucRange)
```

函数描述：

配置 AD7606 模拟信号输入量程。

函数参数：

- 第 1 个参数为 0 表示正负 5V ，1 表示正负 10V。

76.7.4函数 AD7606_Reset

函数原型：

```
void AD7606_Reset(void)
```

函数描述：

此函数用于硬件复位 AD7606，复位之后恢复到正常工作状态。

76.7.5函数 AD7606_StartConvst

函数原型：

```
void AD7606_StartConvst(void)
```

函数描述：

此函数用于启动 1 次 ADC 转换。

76.7.6函数 AD7606_ReadNowAdc

函数原型：

```
void AD7606_ReadNowAdc(void)
```

函数描述：

此函数用于读取 8 路采样结果，结果存储在全局变量 g_tAD7606。

76.7.7函数 AD7606_EnterAutoMode

函数原型：

```
void AD7606_EnterAutoMode(uint32_t _ulFreq)
```

函数描述：

此函数用于配置硬件工作在自动采集模式，结果存储在 FIFO 缓冲区。一般不单独调用，函数 AD7606_StartRecord 会调用。

函数参数：

- 第 1 个参数是采样频率，范围 1-200KHz，单位 Hz。

76.7.8函数 AD7606_StartRecord

函数原型：

```
void AD7606_StartRecord(uint32_t _ulFreq)
```

函数描述：

用于启动采集。

函数参数：

- 第 1 个参数是采样频率，范围 1-200KHz，单位 Hz。

76.7.9函数 AD7606_StopRecord

函数原型：

```
void AD7606_StopRecord(void)
```

函数描述：

此函数用于停止采集定时器。函数 AD7606_StartRecord 和 AD7606_StopRecord 是配套的。

76.7.10 函数 AD7606_FifoNewData

函数原型：

```
uint8_t AD7606_HasNewData(void)
```

函数描述：

此函数用于判断 FIFO 中是否有新数据。

函数参数：

- 返回值，1 表示有，0 表示暂无数据。

76.7.11 函数 AD7606_ReadFifo

函数原型：

```
uint8_t AD7606_ReadFifo(uint16_t *_usReadAdc)
```

函数描述：

此函数用于从 FIFO 中读取一个 ADC 值。

函数参数：

- 第 1 个参数是存放 ADC 结果的变量指针。
- 返回值，1 表示 OK，0 表示暂无数据。

76.7.12 函数 AD7606_FifoFull

函数原型：

```
uint8_t AD7606_FifoFull(void)
```

函数描述：

此函数用于判断 FIFO 是否满。

函数参数：

- 返回值，1 表示满，0 表示未满。

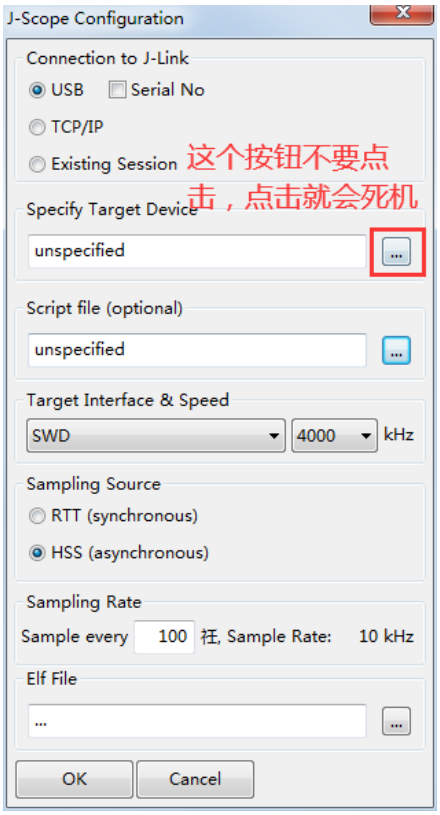
76.8 J-Scope 实时展示 AD7606 采集数据说明

J-Scope 专题教程（实时展示要用 J-Scope 的 RTT 模式），本章配套例子也做了支持：http://www.armbbs.cn/forum.php?mod=viewthread&tid=86881 。

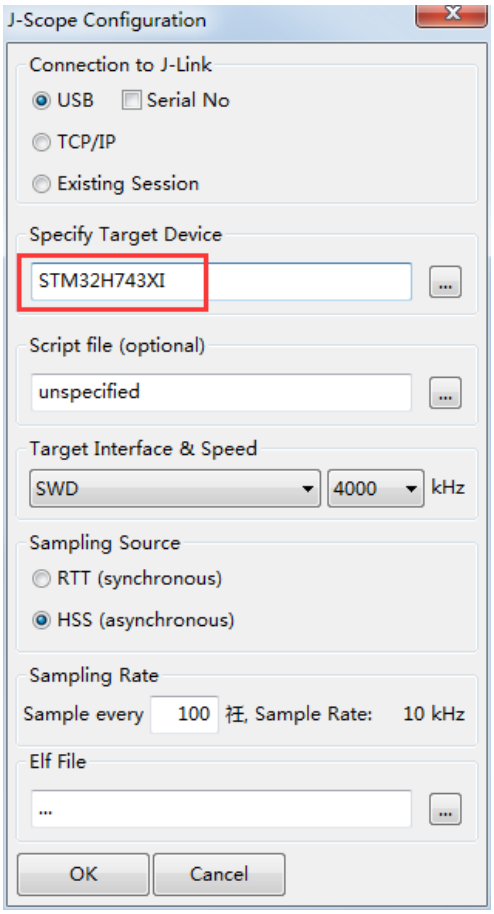
看完专题教程，基本就会操作了，这里有三点注意事项需要大家提前有个了解。另外，推荐使用 MDK 版工程做测试 J-Scope，IAR 版容易测试不正常。

76.8.1 J-Scope 闪退问题解决办法

如下界面，不要点击选择按钮，闪退就是因为点击了这个选择按钮。



直接手动填写型号即可，比如 STM32H743XI，STM32F429BI，STM32F407IG，STM32F103ZE 等。



76.8.2 J-Scope 多通道传输实现

J-Scope 的多通道传输配置好函数 SEGGER_RTT_ConfigUpBuffer 即可，主要是通过第 2 个参数实现的。

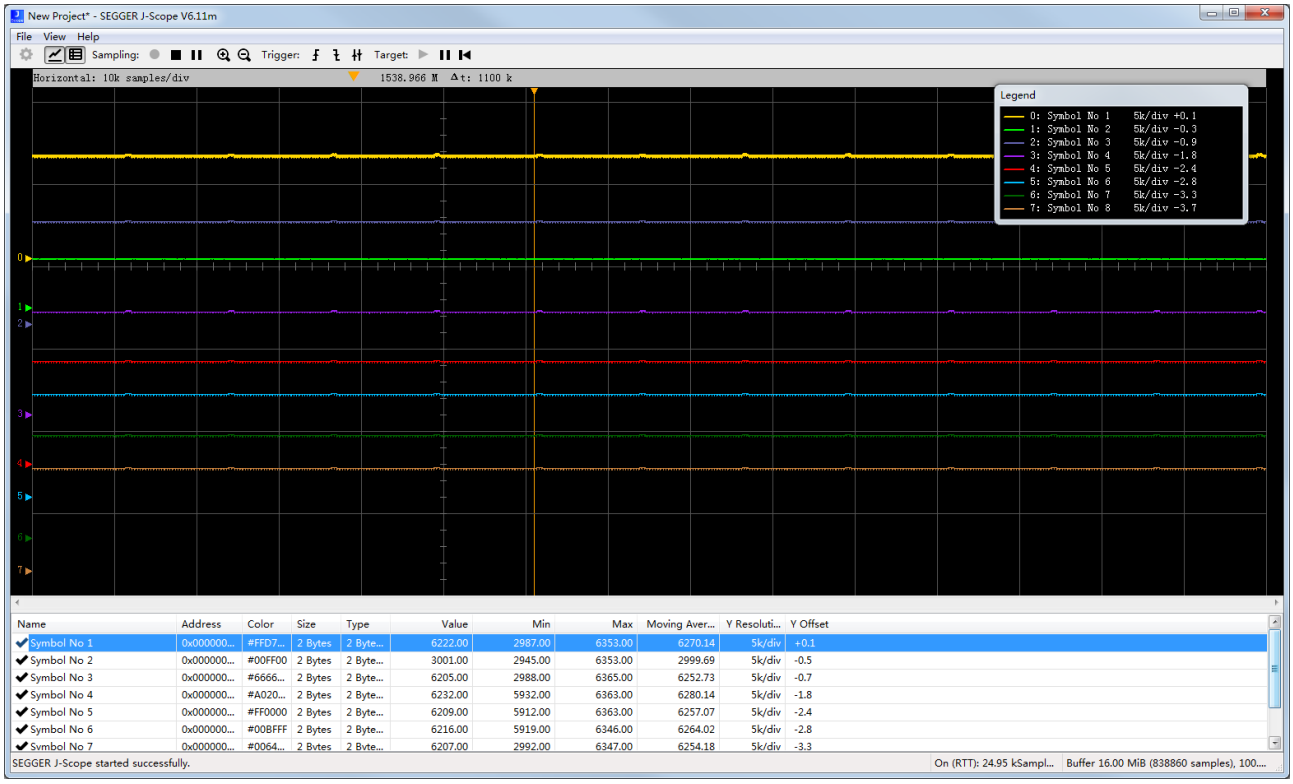
```
    /*
        配置通道 1，上行配置
        默认情况下，J-Scope 仅显示 1 个通道。
        上传 1 个通道的波形，配置第 2 个参数为 JScope_i2
        上传 2 个通道的波形，配置第 2 个参数为 JScope_i2i2
        上传 3 个通道的波形，配置第 2 个参数为 JScope_i2i2i2
        上传 4 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2
        上传 5 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2
        上传 6 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2i2
        上传 7 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2i2i2
        上传 8 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2i2i2i2
    */
    SEGGER_RTT_ConfigUpBuffer(1, "JScope_i2", buf, 20480, SEGGER_RTT_MODE_NO_BLOCK_SKIP);
```

使用函数 SEGGER_RTT_Write 上传数据时，要跟配置的通道数匹配，比如配置的三个通道，就需要调用三次函数：

```
SEGGER_RTT_Write(1, &(g_tAD7606.sNowAdc[0]), 2);
SEGGER_RTT_Write(1, &(g_tAD7606.sNowAdc[1]), 2);
```

```
SEGGER_RTT_Write(1, &(g_tAD7606.sNowAdc[2]), 2);
```

多路效果：

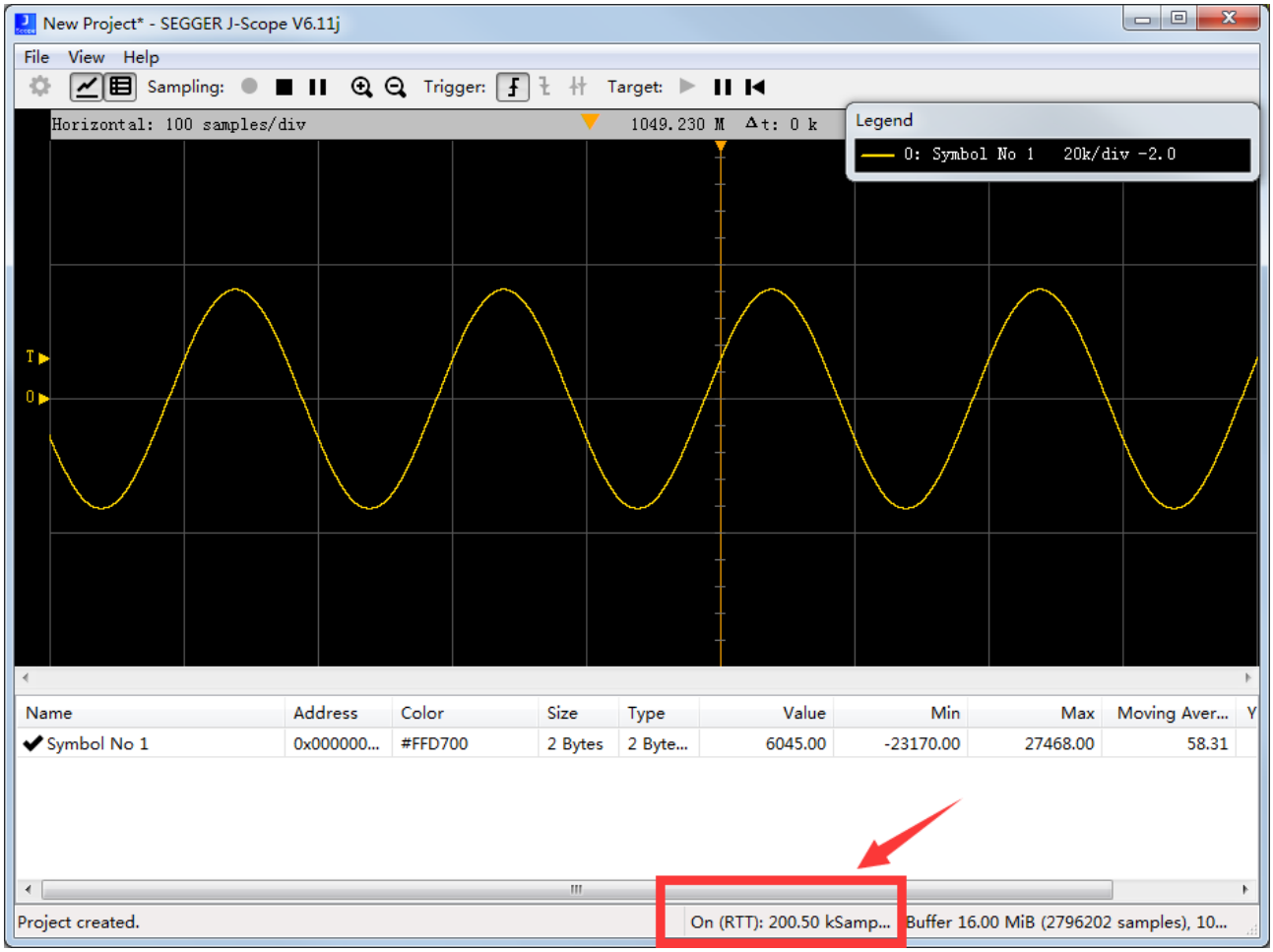


76.8.3 J-Scope 带宽问题

普通的 JLINK 时钟速度 8 - 12MHz 时， J-Scope 的速度基本可以达到 500KB/S（注意，单位是字节）AD7606 的最高采样率是 200Ksps，16bit，那么一路采集就有 400KB/S 的速速，所以要根据设置的采样率设置要显示的 J-Scope 通道数，如果超出了最高通信速度，波形显示会混乱。

200Ksps 时，实时显示 1 路
100Ksps 时，实时显示 2 路
50Ksps 时， 实时显示 4 路
25Ksps 时， 实时显示 8 路

实际速度以底栏的展示为准，如果与设置的速度差异较大，说明传输异常了。



76.9 AD7606 驱动移植和使用

AD7606 移植步骤如下：

- 第 1 步：复制 bsp_fmc_ad7606.c 和 bsp_fmc_ad7606.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：根据使用的 CONVST 引脚，BUSY 引脚，过采样引脚，量程控制引脚，复位引脚，修改 bsp_fmc_ad7606.c 开头的宏定义。

  这里要特别注意过采样引脚，量程控制引脚和复位引脚是采用的扩展 IO，需要大家根据自己的情况修改。

```
/* CONVST 启动 ADC 转换的 GPIO = PC6 */
#define CONVST_RCC_GPIO_CLK_ENABLE    __HAL_RCC_GPIOC_CLK_ENABLE
#define CONVST_TIM8_CLK_DISABLE      __HAL_RCC_TIM8_CLK_DISABLE
#define CONVST_GPIO          GPIOC
#define CONVST_PIN      GPIO_PIN_6
#define CONVST_TIMX          TIM8
#define CONVST_TIMCH    1

/* BUSY 转换完毕信号 = PE5 */
#define BUSY_RCC_GPIO_CLK_ENABLE __HAL_RCC_GPIOE_CLK_ENABLE
#define BUSY_GPIO       GPIOE
#define BUSY_PIN        GPIO_PIN_5
```

```
#define BUSY_IRQn       EXTI9_5_IRQn
#define BUSY_IRQHandler EXTI9_5_IRQHandler

/* 设置过采样的 IO，在扩展的 74HC574 上 */
#define OS0_1()         HC574_SetPin(AD7606_OS0, 1)
#define OS0_0()         HC574_SetPin(AD7606_OS0, 0)
#define OS1_1()         HC574_SetPin(AD7606_OS1, 1)
#define OS1_0()         HC574_SetPin(AD7606_OS1, 0)
#define OS2_1()         HC574_SetPin(AD7606_OS2, 1)
#define OS2_0()         HC574_SetPin(AD7606_OS2, 0)

/* 启动 AD 转换的 GPIO : PC6 */
#define CONVST_1()      CONVST_GPIO->BSRR = CONVST_PIN
#define CONVST_0()      CONVST_GPIO->BSRR = ((uint32_t)CONVST_PIN << 16U)

/* 设置输入量程的 GPIO，在扩展的 74HC574 上 */
#define RANGE_1()  HC574_SetPin(AD7606_RANGE, 1)
#define RANGE_0()  HC574_SetPin(AD7606_RANGE, 0)

/* AD7606 复位口线，在扩展的 74HC574 上 */
#define RESET_1()  HC574_SetPin(AD7606_RESET, 1)
#define RESET_0()  HC574_SetPin(AD7606_RESET, 0)
```

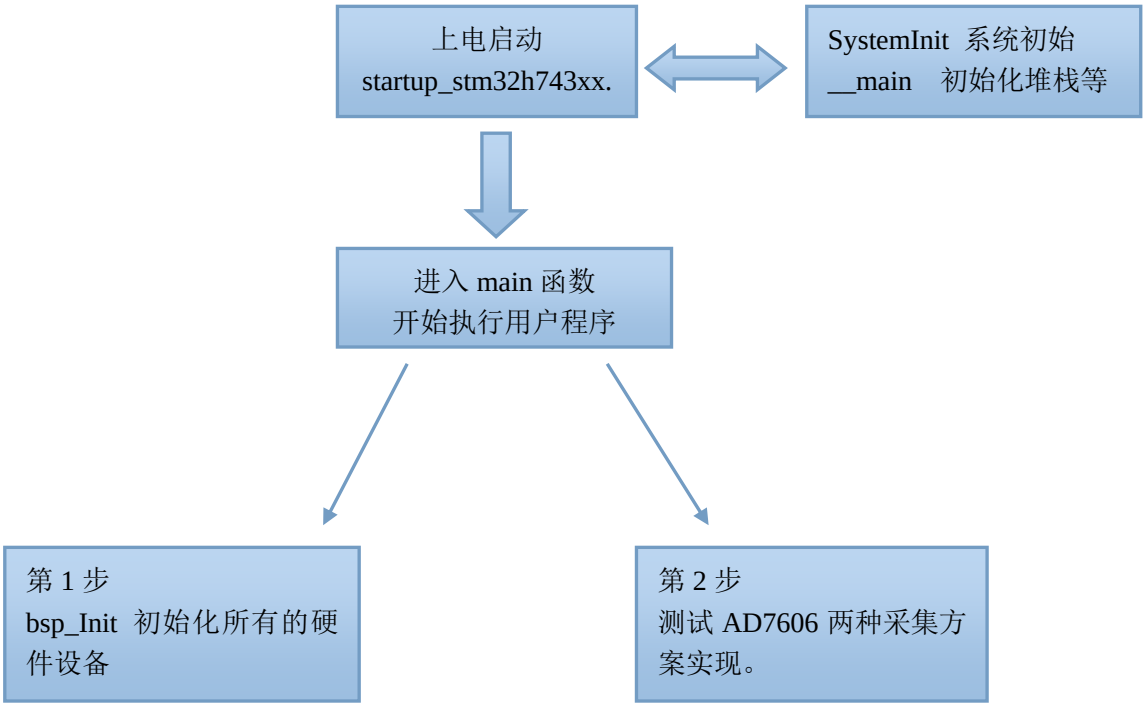
- 第 3 步：根据具体用到的 FMC 引脚，修改函数 AD7606_CtrlLinesConfig 里面做的 IO 配置。
- 第 4 步：根据使用的 FMC BANK，修改函数 AD7606_FSMCConfig 里面的 BANK 配置，这点非常容易疏忽。
- 第 5 步：注意 MPU 配置，详情见本章 77.7.5 小节。
- 第 6 步：初始化 AD7606。

```
bsp_InitAD7606();  /* 配置 AD7606 所用的 GPIO */
```

- 第 7 步：AD7606 驱动主要用到 HAL 库的 FMC 驱动文件，简单省事些可以添加所有 HAL 库 C 源文件进来。
- 第 8 步：应用方法看本章节配套例子即可。

76.10 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器和 LED。
  - 第 2 部分，应用程序设计部分，测试 AD7606 的两种采集方案。

76.11 实验例程说明（MDK）

配套例子：

V7-056_AD7606 的 FMC 总线驱动方式实现（8 通道同步采样, 16bit，正负 10V）

实验目的：

1. 学习 AD7606 的 FMC 驱动方式实现。

重要提示：

1. 板子上电后，默认是软件定时采集，0.5 秒一次，适合串口展示数据。
2. 如果需要使用 J-Scope 实时展示采集的波形效果，需要按下 K2 按键切换到 FIFO 模式。
3. 如果使用的 JLINK 速度不够快，导致 J-Scope 无法最高速度实时上传，可以使用摇杆上下键设置过采样来降低上传速度。
4. 默认情况下，程序仅上传了 AD7606 通道 1 采集的数据。
5. 串口数据展示推荐使用 SecureCRT，因为数据展示做了特别处理，方便采集数据在串口软件同一个位置不断刷新。

实验内容：

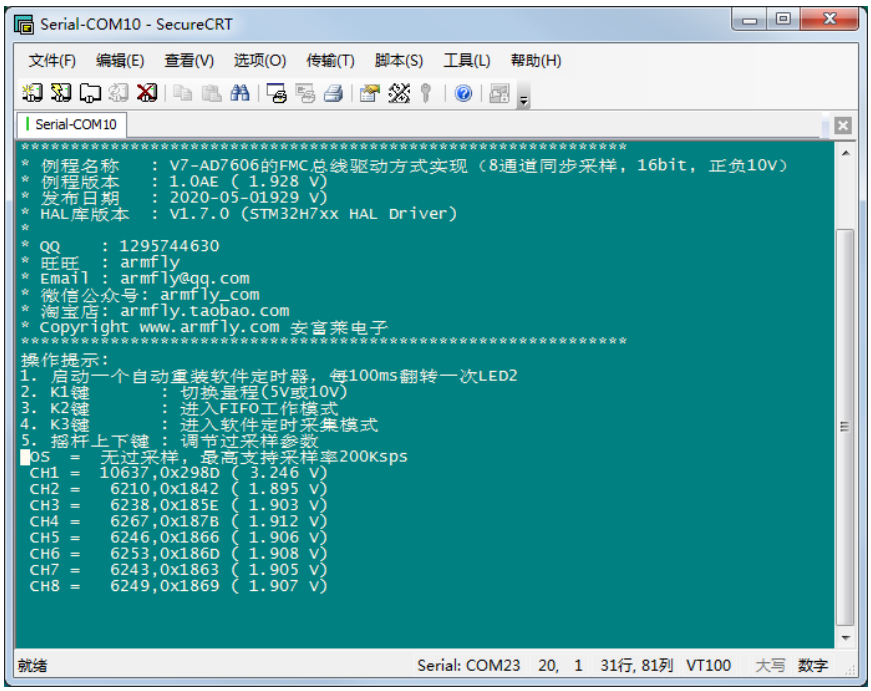
1. AD7606 的 FMC 驱动做了两种采集方式
   （1）软件定时获取方式，适合低速查询获取。
   （2）FIFO 工作模式，适合 8 路实时采集，支持最高采样率 200Ksps。
2. 数据展示方式：
   （1）软件查询方式，数据通过串口打印输出。
   （2）FIFO 工作模式，数据通过 J-Scope 实时输出。
   （3）J-Scope 的实时输出方法请看 V7 板子用户手册对应的 AD7606 章节。
3. 将模拟输入接地时，采样值是 0 左右。
4. 模拟输入端悬空时，采样值在某个范围浮动（这是正常的，这是 AD7606 内部输入电阻导致的浮动电压）。
5. 出厂的 AD7606 模块缺省是 8080 并行接口。如果用 SPI 接口模式，需要修改 R1 R2 电阻配置。
6. 配置 CVA CVB 引脚为 PWM 输出模式，周期设置为需要的采样频率，之后 MCU 将产生周期非常稳定的 AD 转换信号。

实验操作：

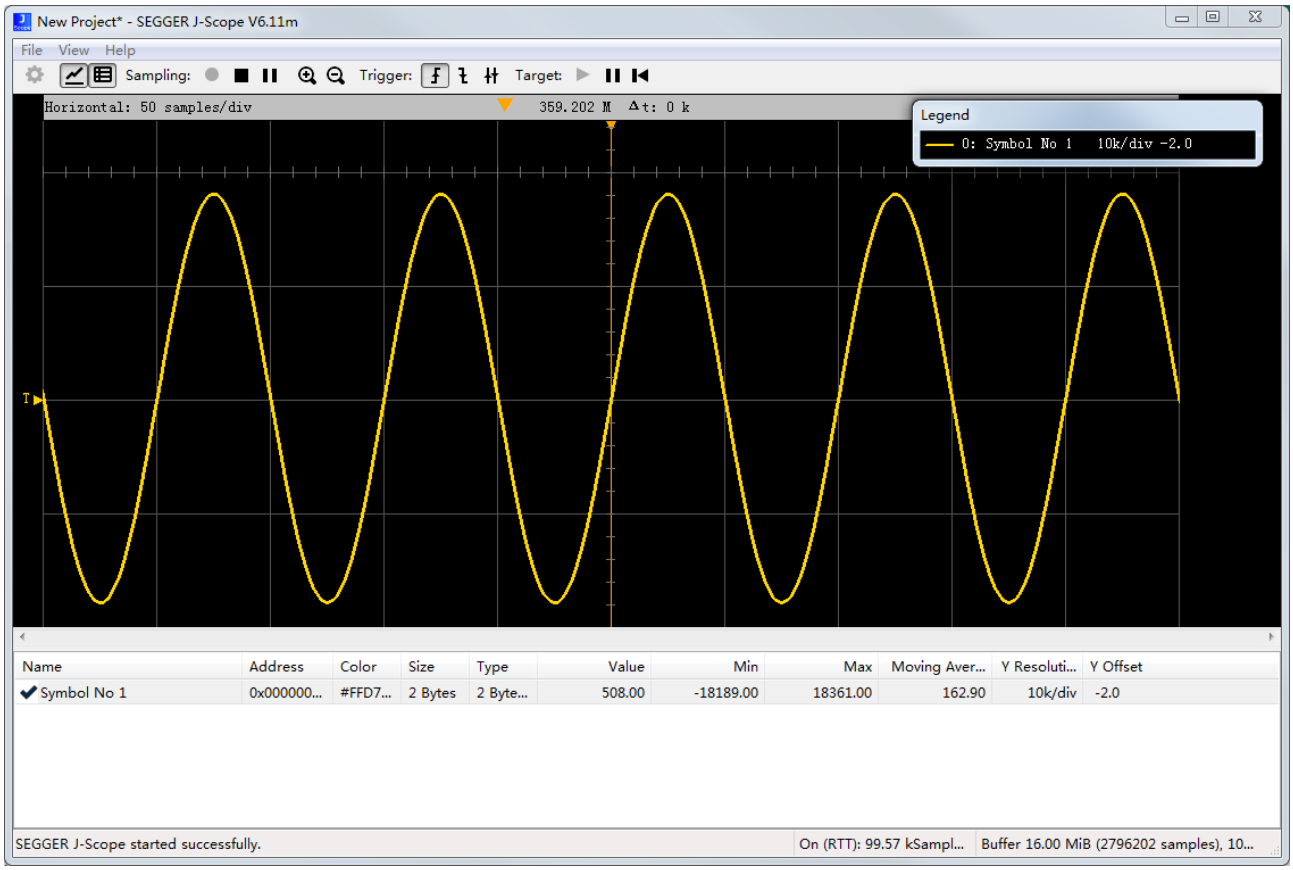
1. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
2. K1 键 ：切换量程(5V 或 10V)。
3. K2 键 ：进入 FIFO 工作模式。
4. K3 键 ：进入软件定时采集模式。
5. 摇杆上下键 ：调节过采样参数。

上电后串口打印的信息：

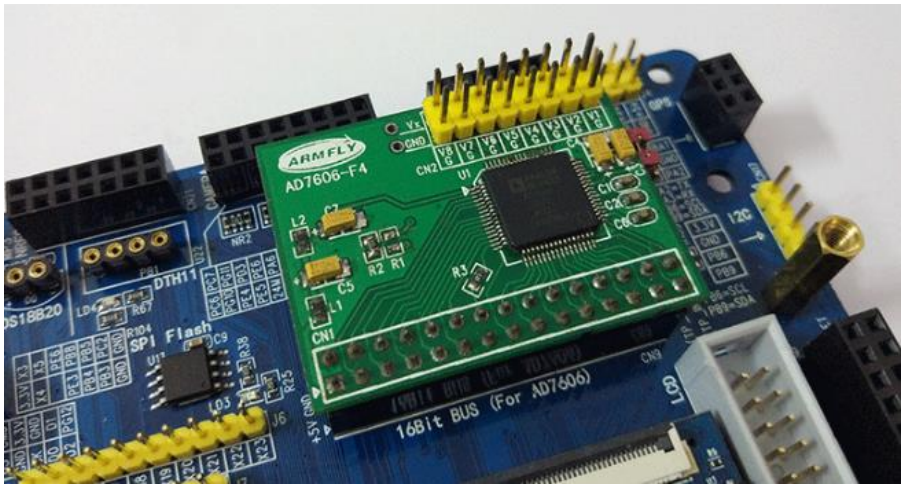
波特率 115200，数据位 8，奇偶校验位无，停止位 1。



J-Scope 波形效果：

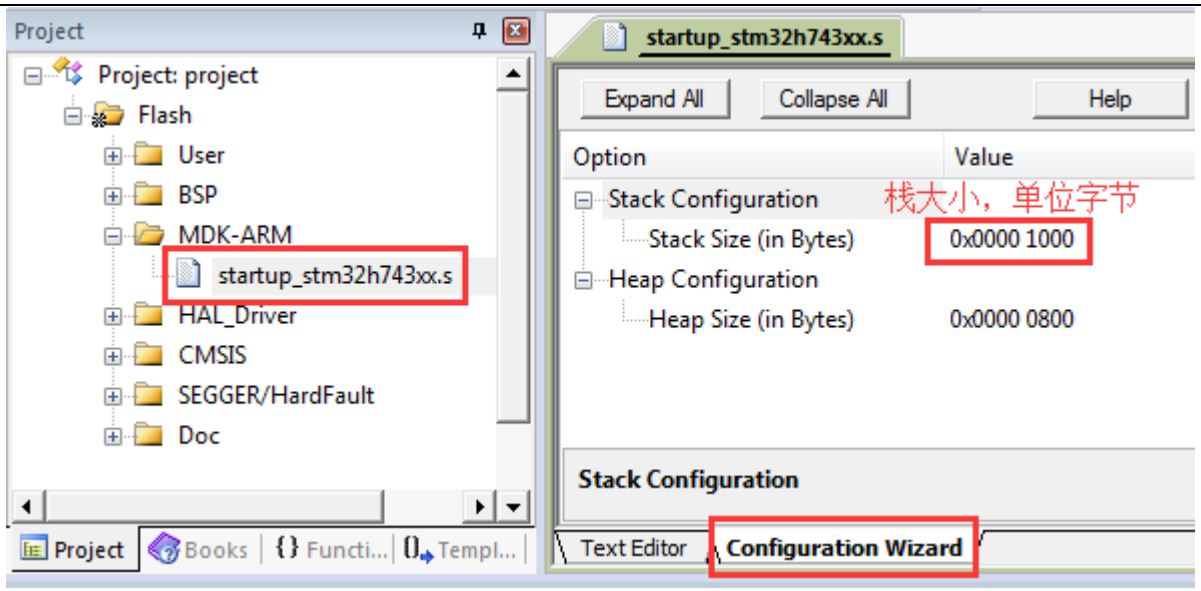


模块插入位置：

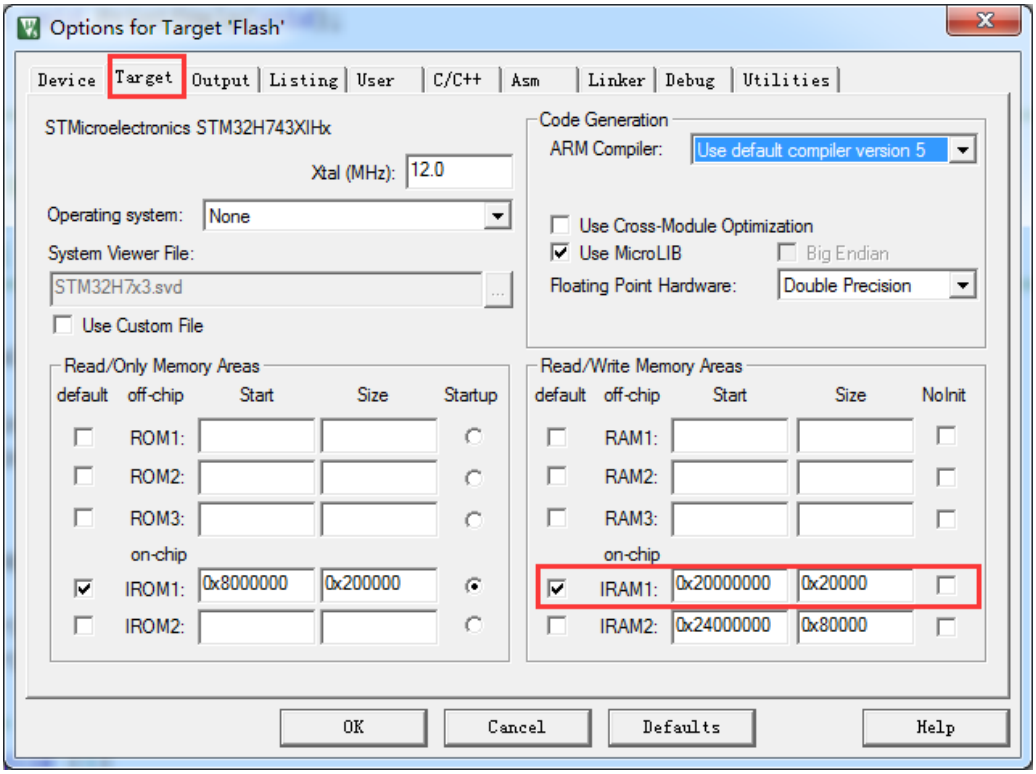


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();      /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();    /* 初始化滴答定时器 */
    bsp_InitLPUart();   /* 初始化串口 */
    bsp_InitExtIO();    /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();      /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitAD7606();  /* 配置 AD7606 所用的 GPIO */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
```

```
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **每 10ms 调用一次按键处理：**

按键处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能:**

主程序实现如下操作:

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- K1 键　　：切换量程(5V 或 10V)。
- K2 键　　：进入 FIFO 工作模式。
- K3 键　　：进入软件定时采集模式。
- 摇杆上下键 ：调节过采样参数。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */

    DemoFmcAD7606(); /* AD7606 测试 */
}

/*
*********************************************************************************************************
*    函 数 名: DemoFmcAD7606
*    功能说明: AD7606 测试
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoFmcAD7606(void)
{
    uint8_t ucKeyCode;
    uint8_t ucRefresh = 0;
    uint8_t ucFifoMode;
```

```
    sfDispMenu();       /* 打印命令提示 */

    ucFifoMode = 0;         /* AD7606 进入普通工作模式 */
    ucRefresh = 0;          /* 数据在串口刷新的标志 */

    AD7606_SetOS(AD_OS_NO);     /* 无过采样 */
    AD7606_SetInputRange(1);    /* 0 表示输入量程为正负 5V, 1 表示正负 10V */
    AD7606_StartConvst();       /* 启动 1 次转换 */

    bsp_StartAutoTimer(0, 500); /* 启动 1 个 500ms 的自动重装的定时器 */
    bsp_StartAutoTimer(3, 200); /* 启动 1 个 200ms 的自动重装的定时器 */

    /*
        配置通道 1，上行配置
        默认情况下，J-Scope 仅显示 1 个通道。
        上传 1 个通道的波形，配置第 2 个参数为 JScope_i2
        上传 2 个通道的波形，配置第 2 个参数为 JScope_i2i2
        上传 3 个通道的波形，配置第 2 个参数为 JScope_i2i2i2
        上传 4 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2
        上传 5 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2
        上传 6 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2i2
        上传 7 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2i2i2
        上传 8 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2i2i2i2
    */
    SEGGER_RTT_ConfigUpBuffer(1, "JScope_i2", buf, 20480, SEGGER_RTT_MODE_NO_BLOCK_SKIP);

    while(1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(3))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        if (ucRefresh == 1)
        {
            ucRefresh = 0;

            /* 处理数据 */
            AD7606_Mak();

            /* 打印 ADC 采样结果 */
            AD7606_Disp();
        }

        if (ucFifoMode == 0)    /* AD7606 普通工作模式 */
        {
            if (bsp_CheckTimer(0))
            {
                /* 每隔 500ms 进来一次. 由软件启动转换 */
                AD7606_ReadNowAdc();        /* 读取采样结果 */
                AD7606_StartConvst();       /* 启动下次转换 */

                ucRefresh = 1;      /* 刷新显示 */
            }
```

```
    }
    else
    {
        /*
            在 FIFO 工作模式，bsp_AD7606 自动进行采集，数据存储在 FIFO 缓冲区。
            结果可以通过下面的函数读取:
            uint8_t AD7606_ReadFifo(uint16_t *_usReadAdc)

            大家可以将数据保存到 SD 卡，或者保存到外部 SRAM。

            本例未对 FIFO 中的数据进行处理，进行打印当前最新的样本值和 J-Scope 的实时输出展示。

            如果主程序不能及时读取 FIFO 数据，那么 AD7606_FifoFull() 将返回真。

            8 通道 200K 采样时，数据传输率 = 200 000 * 2 * 8 = 3.2MB/S
        */

        if (bsp_CheckTimer(0))
        {
            ucRefresh = 1;     /* 刷新显示 */
        }
    }

    /* 按键检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。这个函数不会
    等待按键按下，这样我们可以在 while 循环内做其他的事情 */
    ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
    if (ucKeyCode != KEY_NONE)
    {

        switch (ucKeyCode)
        {
            case KEY_DOWN_K1:           /* K1 键按下 切换量程 */
                if (g_tAD7606.ucRange == 0)
                {
                    AD7606_SetInputRange(1);
                }
                else
                {
                    AD7606_SetInputRange(0);
                }
                ucRefresh = 1;
                break;

            case KEY_DOWN_K2:                           /* K2 键按下 */
                ucFifoMode = 1;                         /* AD7606 进入 FIFO 工作模式 */
                g_tAD7606.ucOS = 1;                     /* 无过采样 */
                AD7606_StartRecord(AD7606_SampleFreq[g_tAD7606.ucOS]);  /* 启动 100kHz 采样速率 */
                AD7606_SetOS(g_tAD7606.ucOS);          /* 设置无过采样 */
                printf("\33[%dA", (int)1);              /* 光标上移 n 行 */
                printf("AD7606 进入 FIFO 工作模式 (200KHz 8 通道同步采集)...\r\n");
                break;

            case KEY_DOWN_K3:           /* K3 键按下 */
                AD7606_StopRecord();    /* 停止记录 */
                ucFifoMode = 0;         /* AD7606 进入普通工作模式 */
                g_tAD7606.ucOS = 0;      /* 无过采样 */
                AD7606_SetOS(g_tAD7606.ucOS);
                printf("\33[%dA", (int)1);  /* 光标上移 n 行 */
```

```
                printf("AD7606 进入普通工作模式(0.5s 定时 8 通道同步采集)...\r\n");
                break;

            case JOY_DOWN_U:            /* 摇杆 UP 键按下 */
                if (g_tAD7606.ucOS < 6)
                {
                    g_tAD7606.ucOS++;
                }

                AD7606_SetOS(g_tAD7606.ucOS);

                /* 如果是 FIFO 模式，*/
                if(ucFifoMode == 1)
                {
                    /* 启动当前过采样下最高速度 */
                    AD7606_StartRecord(AD7606_SampleFreq[g_tAD7606.ucOS]);
                }

                ucRefresh = 1;
                break;

            case JOY_DOWN_D:            /* 摇杆 DOWN 键按下 */
                if (g_tAD7606.ucOS > 0)
                {
                    g_tAD7606.ucOS--;
                }
                AD7606_SetOS(g_tAD7606.ucOS);
                ucRefresh = 1;

                /* 如果是 FIFO 模式，*/
                if(ucFifoMode == 1)
                {
                    /* 启动当前过采样下最高速度 */
                    AD7606_StartRecord(AD7606_SampleFreq[g_tAD7606.ucOS]);
                }
                break;

            default:
                /* 其他的键值不处理 */
                break;
        }
    }
}
}
```

76.12 实验例程说明（IAR）

配套例子：

V7-056_AD7606 的 FMC 总线驱动方式实现（8 通道同步采样, 16bit, 正负 10V）

实验目的：

1. 学习 AD7606 的 FMC 驱动方式实现。

重要提示：

1. 板子上电后，默认是软件定时采集，0.5 秒一次，适合串口展示数据。
2. 如果需要使用 J-Scope 实时展示采集的波形效果，需要按下 K2 按键切换到 FIFO 模式。
3. 如果使用的 JLINK 速度不够快，导致 J-Scope 无法最高速度实时上传，可以使用摇杆上下键设置过采样来降低上传速度。
4. 默认情况下，程序仅上传了 AD7606 通道 1 采集的数据。
5. 串口数据展示推荐使用 SecureCRT，因为数据展示做了特别处理，方便采集数据在串口软件同一个位置不断刷新。

实验内容：

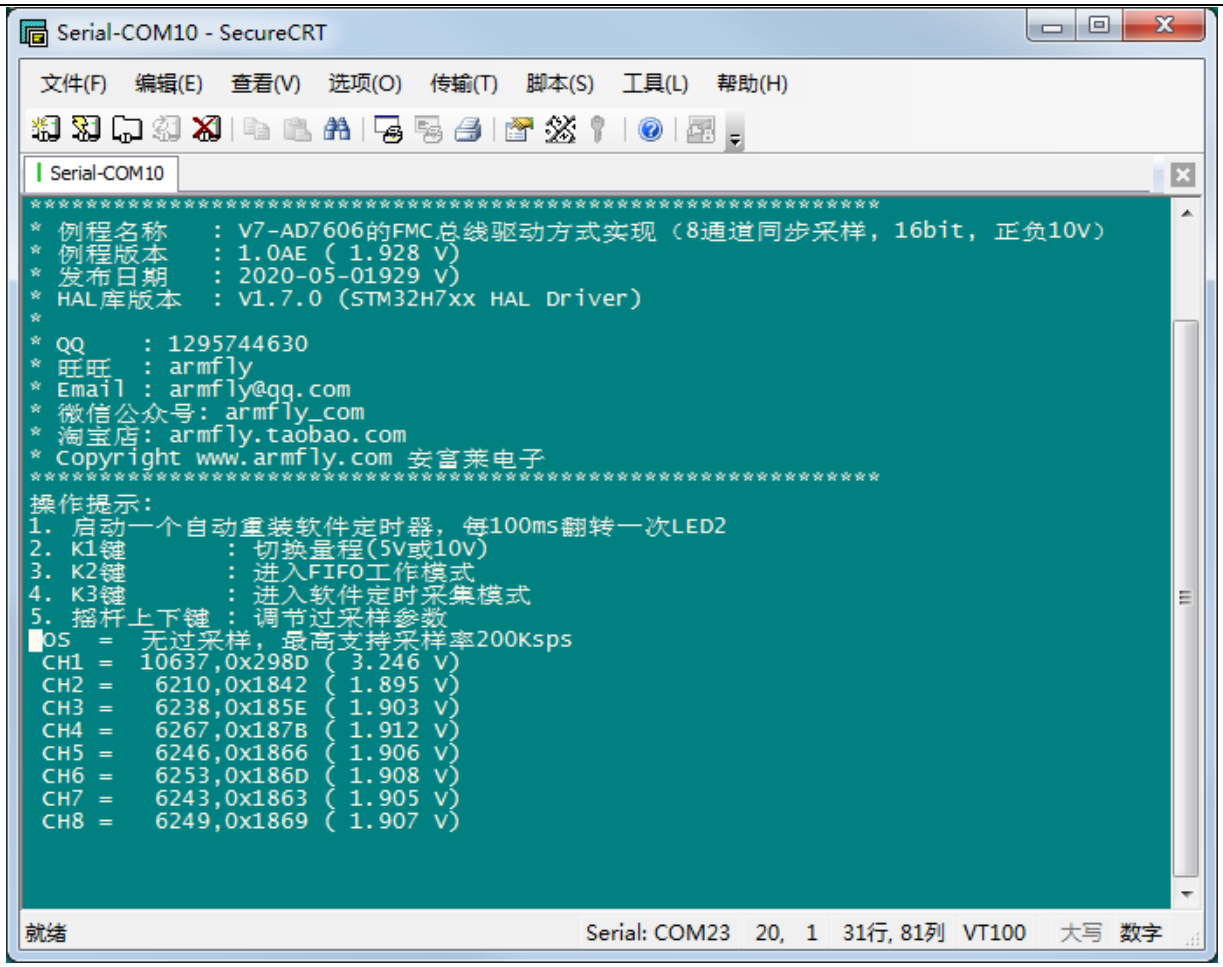
1. AD7606 的 FMC 驱动做了两种采集方式
   （1）软件定时获取方式，适合低速查询获取。
   （2）FIFO 工作模式，适合 8 路实时采集，支持最高采样率 200Ksps。
2. 数据展示方式：
   （1）软件查询方式，数据通过串口打印输出。
   （2）FIFO 工作模式，数据通过 J-Scope 实时输出。
   （3）J-Scope 的实时输出方法请看 V7 板子用户手册对应的 AD7606 章节。
3. 将模拟输入接地时，采样值是 0 左右。
4. 模拟输入端悬空时，采样值在某个范围浮动（这是正常的，这是 AD7606 内部输入电阻导致的浮动电压）。
5. 出厂的 AD7606 模块缺省是 8080 并行接口。如果用 SPI 接口模式，需要修改 R1 R2 电阻配置。
6. 配置 CVA CVB 引脚为 PWM 输出模式，周期设置为需要的采样频率，之后 MCU 将产生周期非常稳定的 AD 转换信号。

实验操作：

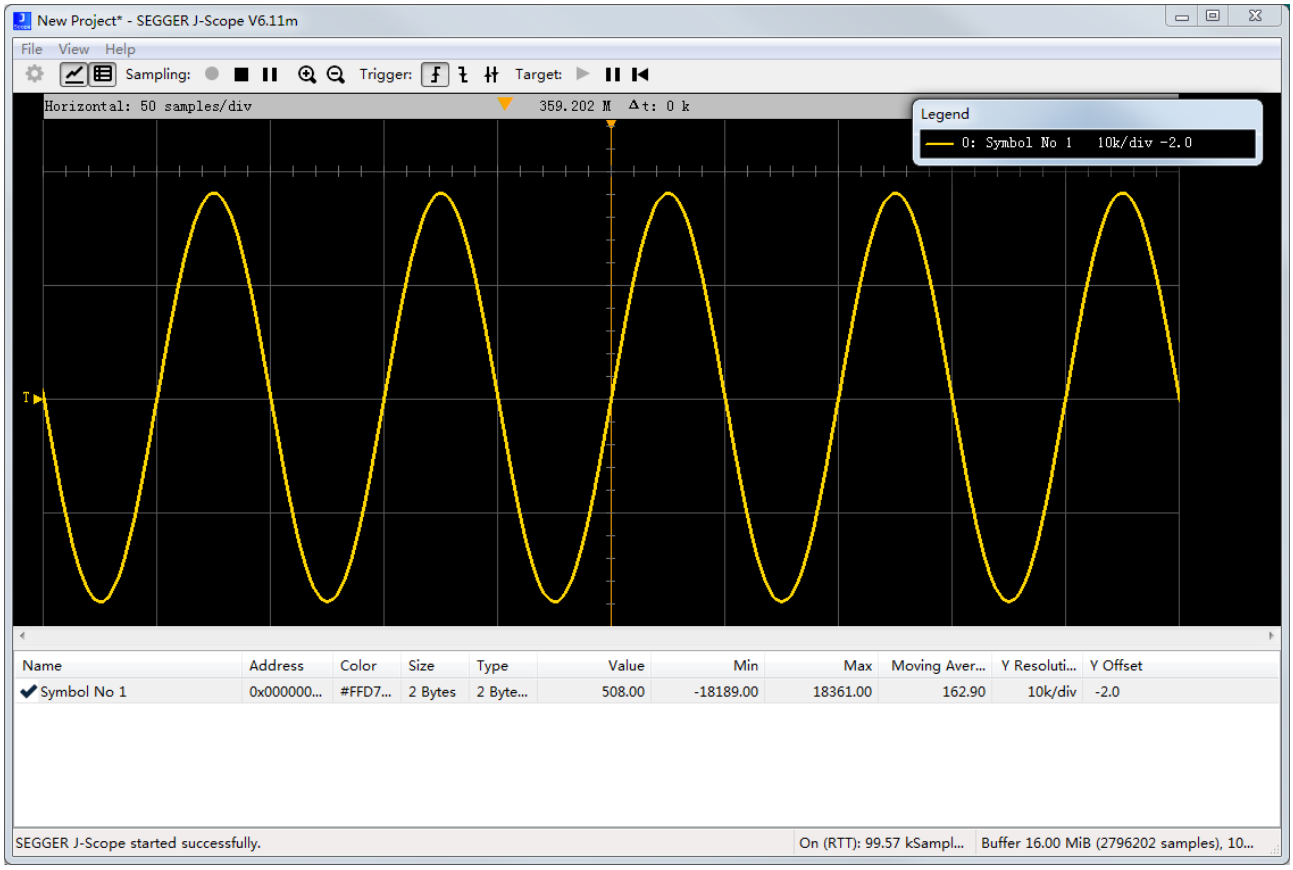
1. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
2. K1 键 ：切换量程(5V 或 10V)。
3. K2 键 ：进入 FIFO 工作模式。
4. K3 键 ：进入软件定时采集模式。
5. 摇杆上下键 ：调节过采样参数。

上电后串口打印的信息：

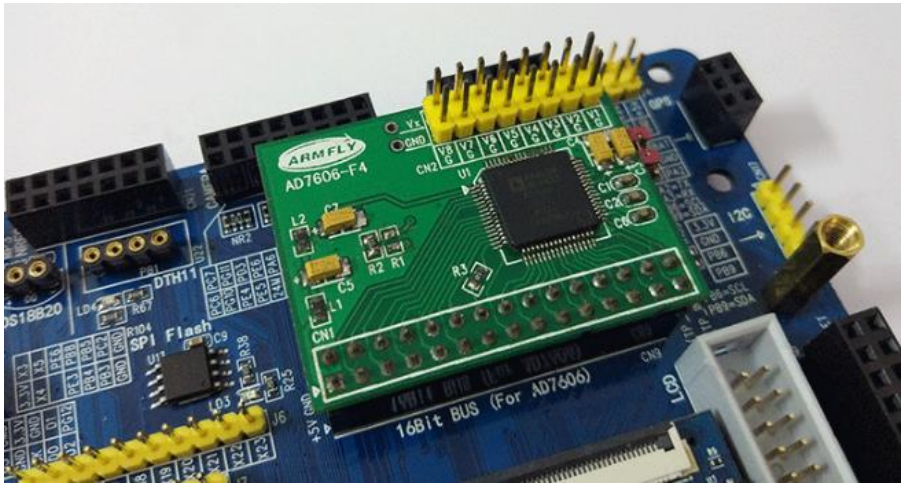
波特率 115200，数据位 8，奇偶校验位无，停止位 1。


Serial-COM10 - SecureCRT
文件(F) 编辑(E) 查看(V) 选项(O) 传输(T) 脚本(S) 工具(L) 帮助(H)
Serial-COM10
* 例程名称 : V7-AD7606的FMC总线驱动方式实现（8通道同步采样，16bit，正负10V）
* 例程版本 : 1.0AE (1.928 V)
* 发布日期 : 2020-05-01929 V)
* HAL库版本 : V1.7.0 (STM32H7xx HAL Driver)
* QQ : 1295744630
* 旺旺 : armfly
* Email : armfly@qq.com
* 微信公众号: armfly_com
* 淘宝店: armfly.taobao.com
* Copyright www.armfly.com 安富莱电子
操作提示:
1. 启动一个自动重装软件定时器，每100ms翻转一次LED2
2. K1键 : 切换量程(5V或10V)
3. K2键 : 进入FIFO工作模式
4. K3键 : 进入软件定时采集模式
5. 摇杆上下键 : 调节过采样参数
OS = 无过采样，最高支持采样率200Ksps
CH1 = 10637,0x298D (3.246 V)
CH2 = 6210,0x1842 (1.895 V)
CH3 = 6238,0x185E (1.903 V)
CH4 = 6267,0x187B (1.912 V)
CH5 = 6246,0x1866 (1.906 V)
CH6 = 6253,0x186D (1.908 V)
CH7 = 6243,0x1863 (1.905 V)
CH8 = 6249,0x1869 (1.907 V)
就绪
Serial: COM23 20, 1 31行, 81列 VT100 大写 数字

J-Scope 波形效果：

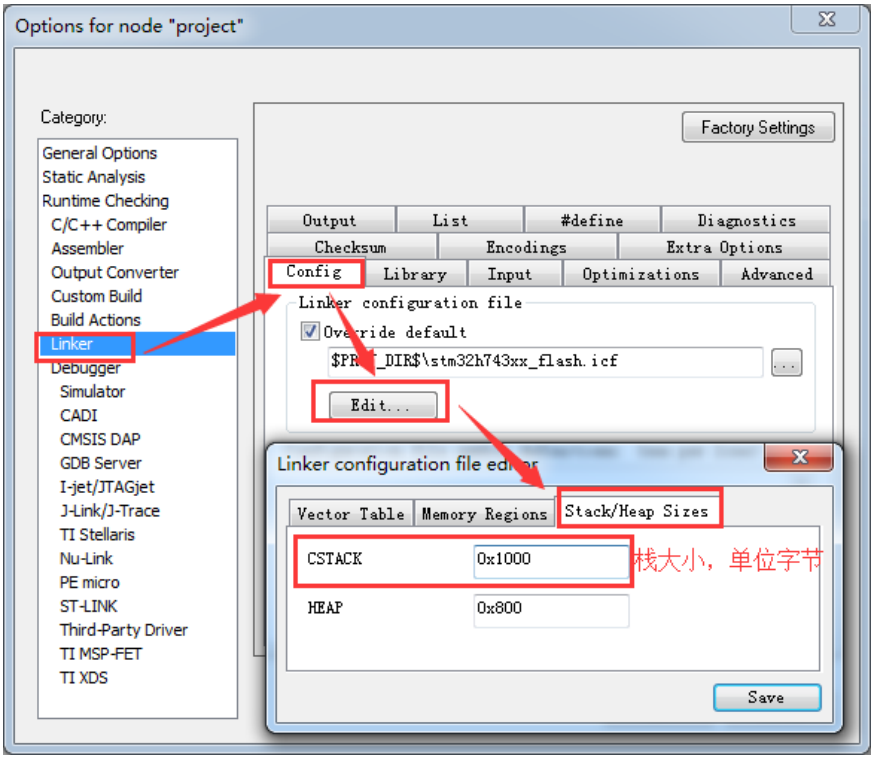


模块插入位置：

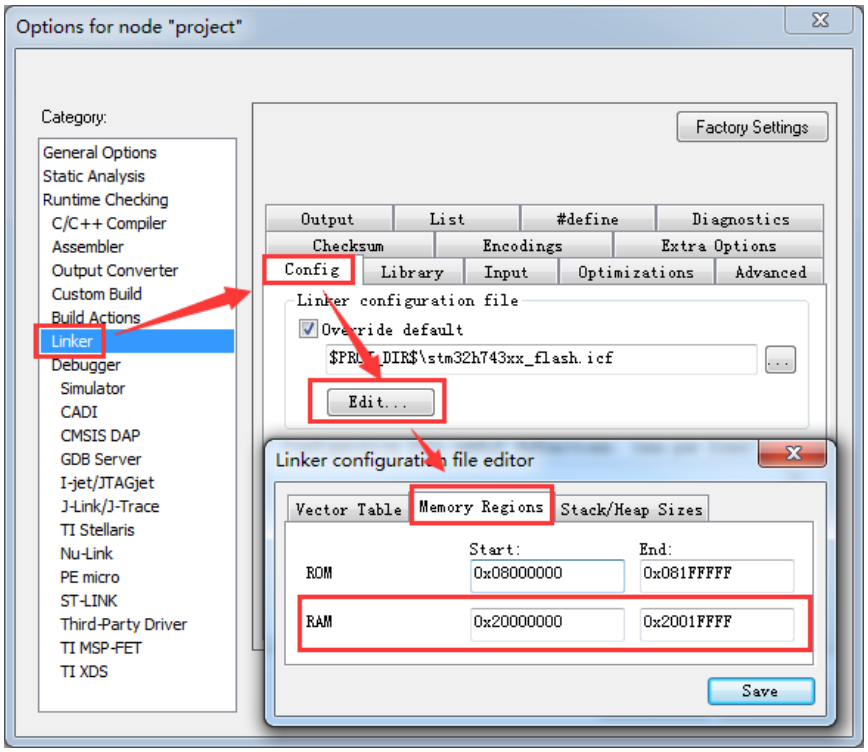


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
```

```
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();      /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();    /* 初始化滴答定时器 */
    bsp_InitLPUart();   /* 初始化串口 */
    bsp_InitExtIO();    /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();      /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitAD7606();  /* 配置 AD7606 所用的 GPIO */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
```

```
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();
```

```
    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **每 10ms 调用一次按键处理：**

按键处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能：**

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- K1 键　　：切换量程(5V 或 10V)。
- K2 键　　：进入 FIFO 工作模式。
- K3 键　　：进入软件定时采集模式。
- 摇杆上下键 ：调节过采样参数。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */

    DemoFmcAD7606(); /* AD7606 测试 */
}

/*
*********************************************************************************************************
*    函 数 名: DemoFmcAD7606
*    功能说明: AD7606 测试
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoFmcAD7606(void)
```

```
{
    uint8_t ucKeyCode;
    uint8_t ucRefresh = 0;
    uint8_t ucFifoMode;

    sfDispMenu();      /* 打印命令提示 */

    ucFifoMode = 0;         /* AD7606 进入普通工作模式 */
    ucRefresh = 0;          /* 数据在串口刷新的标志 */

    AD7606_SetOS(AD_OS_NO);     /* 无过采样 */
    AD7606_SetInputRange(1);    /* 0 表示输入量程为正负 5V, 1 表示正负 10V */
    AD7606_StartConvst();       /* 启动 1 次转换 */

    bsp_StartAutoTimer(0, 500); /* 启动 1 个 500ms 的自动重装的定时器 */
    bsp_StartAutoTimer(3, 200); /* 启动 1 个 200ms 的自动重装的定时器 */

    /*
        配置通道 1，上行配置
        默认情况下，J-Scope 仅显示 1 个通道。
        上传 1 个通道的波形，配置第 2 个参数为 JScope_i2
        上传 2 个通道的波形，配置第 2 个参数为 JScope_i2i2
        上传 3 个通道的波形，配置第 2 个参数为 JScope_i2i2i2
        上传 4 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2
        上传 5 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2
        上传 6 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2i2
        上传 7 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2i2i2
        上传 8 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2i2i2i2
    */
    SEGGER_RTT_ConfigUpBuffer(1, "JScope_i2", buf, 20480, SEGGER_RTT_MODE_NO_BLOCK_SKIP);

    while(1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(3))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        if (ucRefresh == 1)
        {
            ucRefresh = 0;

            /* 处理数据 */
            AD7606_Mak();

            /* 打印 ADC 采样结果 */
            AD7606_Disp();
        }

        if (ucFifoMode == 0)    /* AD7606 普通工作模式 */
        {
            if (bsp_CheckTimer(0))
            {
                /* 每隔 500ms 进来一次. 由软件启动转换 */
```

```
            AD7606_ReadNowAdc();        /* 读取采样结果 */
            AD7606_StartConvst();       /* 启动下次转换 */

            ucRefresh = 1;      /* 刷新显示 */
        }
    }
    else
    {
        /*
            在 FIFO 工作模式，bsp_AD7606 自动进行采集，数据存储在 FIFO 缓冲区。
            结果可以通过下面的函数读取:
            uint8_t AD7606_ReadFifo(uint16_t *_usReadAdc)

            大家可以将数据保存到 SD 卡，或者保存到外部 SRAM。

            本例未对 FIFO 中的数据进行处理，进行打印当前最新的样本值和 J-Scope 的实时输出展示。

            如果主程序不能及时读取 FIFO 数据，那么 AD7606_FifoFull() 将返回真。

            8 通道 200K 采样时，数据传输率 = 200 000 * 2 * 8 = 3.2MB/S
        */

        if (bsp_CheckTimer(0))
        {
            ucRefresh = 1;      /* 刷新显示 */
        }
    }

    /* 按键检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。这个函数不会
    等待按键按下，这样我们可以在 while 循环内做其他的事情 */
    ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
    if (ucKeyCode != KEY_NONE)
    {

        switch (ucKeyCode)
        {
            case KEY_DOWN_K1:           /* K1 键按下 切换量程 */
                if (g_tAD7606.ucRange == 0)
                {
                    AD7606_SetInputRange(1);
                }
                else
                {
                    AD7606_SetInputRange(0);
                }
                ucRefresh = 1;
                break;

            case KEY_DOWN_K2:                           /* K2 键按下 */
                ucFifoMode = 1;                         /* AD7606 进入 FIFO 工作模式 */
                g_tAD7606.ucOS = 1;                     /* 无过采样 */
                AD7606_StartRecord(AD7606_SampleFreq[g_tAD7606.ucOS]);  /* 启动 100kHz 采样速率 */
                AD7606_SetOS(g_tAD7606.ucOS);       /* 设置无过采样 */
                printf("\33[%dA", (int)1);          /* 光标上移 n 行 */
                printf("AD7606 进入 FIFO 工作模式 (200KHz 8 通道同步采集)...\r\n");
                break;

            case KEY_DOWN_K3:           /* K3 键按下 */
```

```
                AD7606_StopRecord();    /* 停止记录 */
                ucFifoMode = 0;         /* AD7606 进入普通工作模式 */
                g_tAD7606.ucOS = 0;      /* 无过采样 */
                AD7606_SetOS(g_tAD7606.ucOS);
                printf("\33[%dA", (int)1);  /* 光标上移 n 行 */
                printf("AD7606 进入普通工作模式(0.5s 定时 8 通道同步采集)...\r\n");
                break;

            case JOY_DOWN_U:            /* 摇杆 UP 键按下 */
                if (g_tAD7606.ucOS < 6)
                {
                    g_tAD7606.ucOS++;
                }

                AD7606_SetOS(g_tAD7606.ucOS);

                /* 如果是 FIFO 模式, */
                if(ucFifoMode == 1)
                {
                    /* 启动当前过采样下最高速度 */
                    AD7606_StartRecord(AD7606_SampleFreq[g_tAD7606.ucOS]);
                }

                ucRefresh = 1;
                break;

            case JOY_DOWN_D:            /* 摇杆 DOWN 键按下 */
                if (g_tAD7606.ucOS > 0)
                {
                    g_tAD7606.ucOS--;
                }
                AD7606_SetOS(g_tAD7606.ucOS);
                ucRefresh = 1;

                /* 如果是 FIFO 模式, */
                if(ucFifoMode == 1)
                {
                    /* 启动当前过采样下最高速度 */
                    AD7606_StartRecord(AD7606_SampleFreq[g_tAD7606.ucOS]);
                }
                break;

            default:
                /* 其他的键值不处理 */
                break;
        }
    }
}
```

76.13 总结

本章节涉及到的知识点非常多，实战性较强，需要大家稍花点精力去研究。

第77章 STM32H7 的 FMC 总线应用之 DMA 双缓冲驱动 AD7606（8 通道同步采样, 16bit，正负 10V）

本章节为大家讲解 FMC DMA 双缓冲方式驱动数模转换器 AD7606，实战性较强。

77.1 初学者重要提示

77.2 ADC 结构分类

77.3 AD7606 硬件设计

77.4 AD7606 关键知识点整理（重要）

77.5 AD7606 的 FMC DMA 实现思路

77.6 AD7606 的 FMC 接口硬件设计

77.7 AD7606 的 FMC 接口驱动设计

77.8 AD7606 板级支持包（bsp_fmc_ad7606）

77.9 J-Scope 实时展示 AD7606 采集数据说明

77.10 AD7606 驱动移植和使用

77.11 实验例程设计框架

77.12 实验例程说明（MDK）

77.13 总结

77.1 初学者重要提示

- 学习本章节前，务必优先学习第 76 章，本章是建立在 76 章的基础上。
- 本章 77.6 小节的知识点对于本章的理解尤其重要。
- AD7606 的配置很简单，它没有内部寄存器，量程范围和过采样参数是通过外部 IO 控制的，采样速率由 MCU 或 DSP 提供的脉冲频率控制。
- AD7606 必须使用单 5V 供电。而 AD7606 和 MCU 之间的通信接口电平由 VIO (V_{DRIVE}) 引脚控制。也就是说 VIO 必须接单片机的电源，可以是 3.3V 也可以是 5V（范围 2.3V – 5V）。
- 正确的理解过采样，比如我们设置是 1Ksps 采样率，64 倍过采样。意思是指每次采样，AD7606 会采样 64 次数据并求平均，相当于 AD7606 以 64Ksps 进行采样的，只是将每 64 个采样点的值做了

平均，用户得到的值就是平均后的数值。因此，如果使用 AD7606 最高的 200Ksps 采样率，就不可以使用过采样了。

- STM32H7 驱动 AD7606 配合 J-Scope 实时输出，效果绝了，堪比示波器 http://www.armbbs.cn/forum.php?mod=viewthread&tid=97393 。
  使用方法详解本章节 77.9 小节。
- 本章配套例子的串口数据展示推荐使用 SecureCRT，因为数据展示做了特别处理，方便采集数据在串口软件同一个位置不断刷新。
- AD7606 数据手册，模块原理图（通用版）和接线图都已经放到本章教程配置例子的 Doc 文件里。
- ADC 的专业术语诠释文档，推荐大家看看：http://www.armbbs.cn/forum.php?mod=viewthread&tid=89414 。
- 测试本章配套例子前重要提示：
  - 测试时，务必使用外置电源为开发板供电，因为 AD7606 需要 5V 供电电压。板子上插入 AD7606 模块时，注意对齐。
  - 板子上电后，默认是 100Ksps，2 倍过采样。
  - 如果使用的 JLINK 速度不够快，导致 J-Scope 无法最高速度实时上传，可以使用摇杆上下键设置过采样来降低上传速度。
  - 默认情况下，程序仅上传了 AD7606 通道 1 采集的数据。

77.2 ADC 结构分类

77.3 AD7606 硬件设计

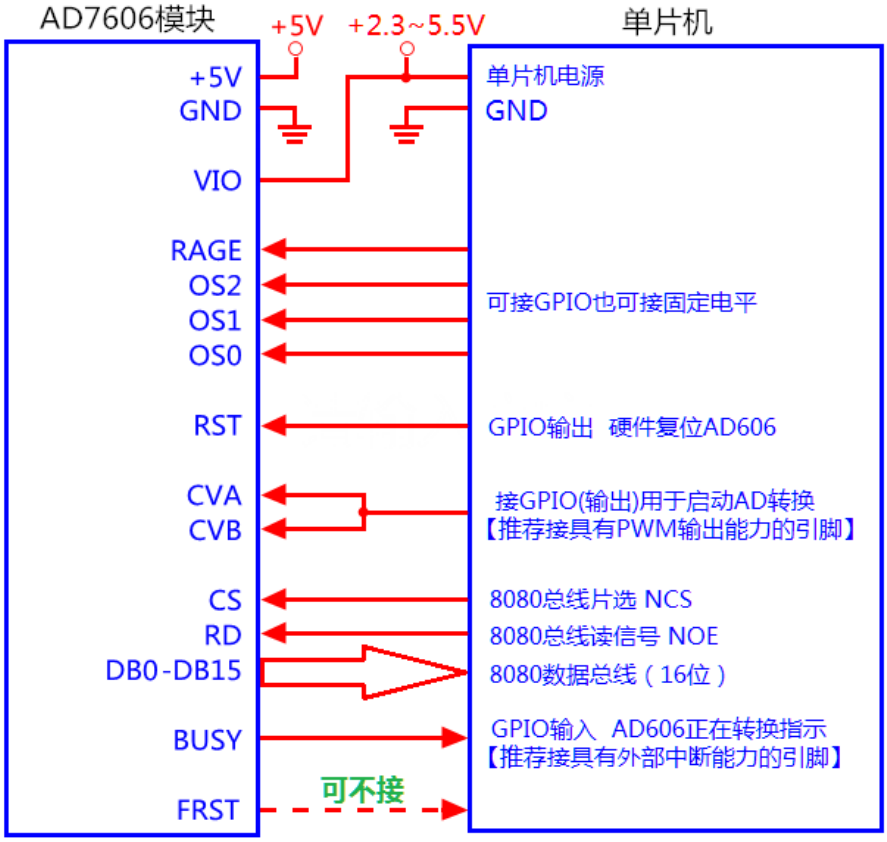
77.4 AD7606 关键知识点整理（重要）

77.5 AD7606 的 FMC 接口硬件设计

77.2，77.3，77.4 和 77.5 小节的知识在第 76 章节有详细说明，本章不再赘述。

77.6 AD7606 的 FMC DMA 实现思路

FMC 的并行接线方式如下：



这里实现 FMC DMA 方式的关键就是 BUSY 引脚去触发 DMA 控制，如果是单纯的 DMA 正常模式，实现比较简单，接收到 INT 引脚的就绪状态，使用 FMC DMA 将 8 路数据全部读取出来即可。

难点在于驱动 AD7606 不像 SRAM，SDRAM，仅需一个 FMC 接口就行，它还需要一个独立的时钟引脚，每次时钟触发要连续读取 8 次数据。针对这个问题，就可以使用 DMAMUX 的事件触发方式来实现，可以选择的主要是：

```
HAL_DMAMUX1_REQ_GEN_EXTI0
HAL_DMAMUX2_REQ_GEN_EXTI0
HAL_DMAMUX2_REQ_GEN_EXTI2
```

按照这个思路，尝试了下面三种方案，但实现都太复杂了。

- 方案 1：

  定时器配合 DMAMUX 两级级联，苦于找不到合理级联触发源。

- 方案 2：

  定时器触发 DMAMUX，然后 DMA 触发 MDMA，这个是可以实现的，就是 MDMA 玩起来有点复杂。

- 方案 3：

  两路 DMAMUX 控制，不限制必须用定时器的 PWM 引脚，然后配置定时器做同步触发源，也是可以实现的，占用太多硬件资源。

最终这三种方案全部否决了，实现的略麻烦。最终有个第 4 套方案，实现这套方案有如下几点：

77.6.1 定时器 PWM 输出控制 AD7606 转换

通过定时器 PWM 输出控制 AD7606 转换比较容易实现，我们上一个章节就是这种方式控制的。

77.6.2 定时器 UP 更新事件触发 DMA 实现突发传输

有了定时器 PWM 控制 AD7606 转换。还需要保证每个 PWM 脉冲读取一次数据，而且是连续读取 8 路。这就需要用到下面两个知识点，非常关键：

- 同时开启同一个定时器的 PWM 输出和 UP 更新事件。

  这样可以保证每个 PWM 后都配有一个 UP 更新，通过 UP 更新来触发 DMA 传输。

- DMA 突发功能实现每次触发连续读取 8 路数据。

  STM32H7 支持的突发方式如下，下面这个表格尤其重要，配置突发务必要按照这个表格来配置：

MSIZE	FIFO 级别	MBURST = INCR4	MBURST = INCR8	MBURST = INCR16
字节	1/4	4 个节拍的 1 次突发	禁止	禁止
	1/2	4 个节拍的 2 次突发	8 个节拍的 1 次突发	
	3/4	4 个节拍的 3 次突发	禁止	
	满	4 个节拍的 4 次突发	8 个节拍的 2 次突发	16 个节拍的 1 次突发
半字	1/4	禁止	禁止	禁止
	1/2	4 个节拍的 1 次突发		
	3/4	禁止		
	满	4 个节拍的 2 次突发	8 个节拍的 1 次突发	
字	1/4	禁止	禁止	
	1/2			
	3/4			
	满	4 个节拍的 1 次突发		

我们要实现的是连续读取 8 路 16bit 数据，上面表格中红色方框部分刚好支持。

77.6.3 不使用 BUSY 引脚如何保证读取正确的数据

解决这个问题的关键就是 AD7606 支持转换期间读取：

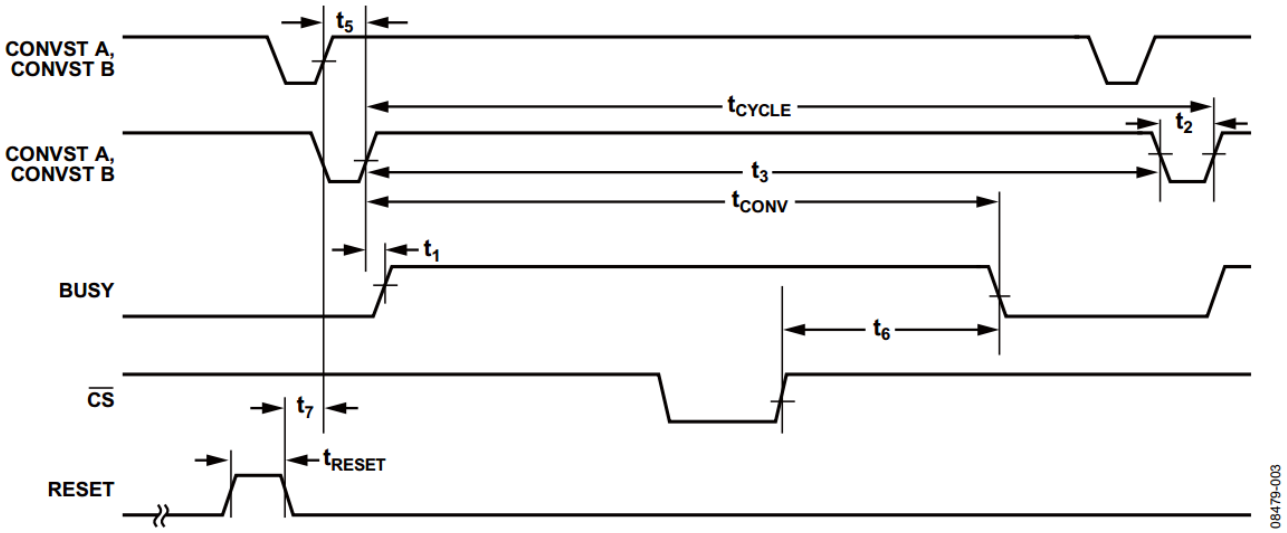


图3. CONVST时序—转换期间读取

这个功能正好用在本设计中，这里有四个关键时序参数：

- t_2

  表示最短的 CONVST 低电平脉冲，最小值 25ns，这个时间我们用 PWM 脉冲低电平控制。

- t_3

  表示最短的 CONVST 高电平脉冲，最小值 25ns，这个时间我们用 PWM 脉冲高电平控制。

- t_6

  表示 CS 上升沿和 BUSY 下降沿之间的最长时间，最大值 25ns。这个参数的主要作用是限制用户一定要在 BUSY 转换有效之前立即读取。

- t_{conv}

  表示 AD7606 转换时间，对于 AD7606-8 来说，最小值范围是 3.45us 到 4.15us。

参数	在T_{MIN}、T_{MAX}时的限值 最小值	典型值	最大值	单位	描述
并行/串行/字节模式					
t_{CYCLE}					1/吞吐速率
			5	μs	并行模式，转换期间或之后读取；或者串行模式：V_{DRIVE}=4.75 V至5.25 V，利用D_{OUT}A和D_{OUT}B线路在转换期间读取
		5		μs	串行模式，转换期间读取；V_{DRIVE}= 3.3 V
			9.7	μs	串行模式，转换之后读取；V_{DRIVE}= 2.3 V，D_{OUT}A和D_{OUT}B线路
t_{CONV}[2]					转换时间
	3.45	4	4.15	μs	过采样关闭；AD7606
		3		μs	过采样关闭；AD7606-6
		2		μs	过采样关闭；AD7606-4
	7.87		9.1	μs	2倍过采样；AD7606
	16.05		18.8	μs	4倍过采样；AD7606
	33		39	μs	8倍过采样；AD7606
	66		78	μs	16倍过采样；AD7606
	133		158	μs	32倍过采样；AD7606
	257		315	μs	64倍过采样；AD7606
$t_{WAKE-UP\ STANDBY}$			100	μs	$\overline{STBY}$上升沿到CONVST x上升沿；从待机模式上电的时间

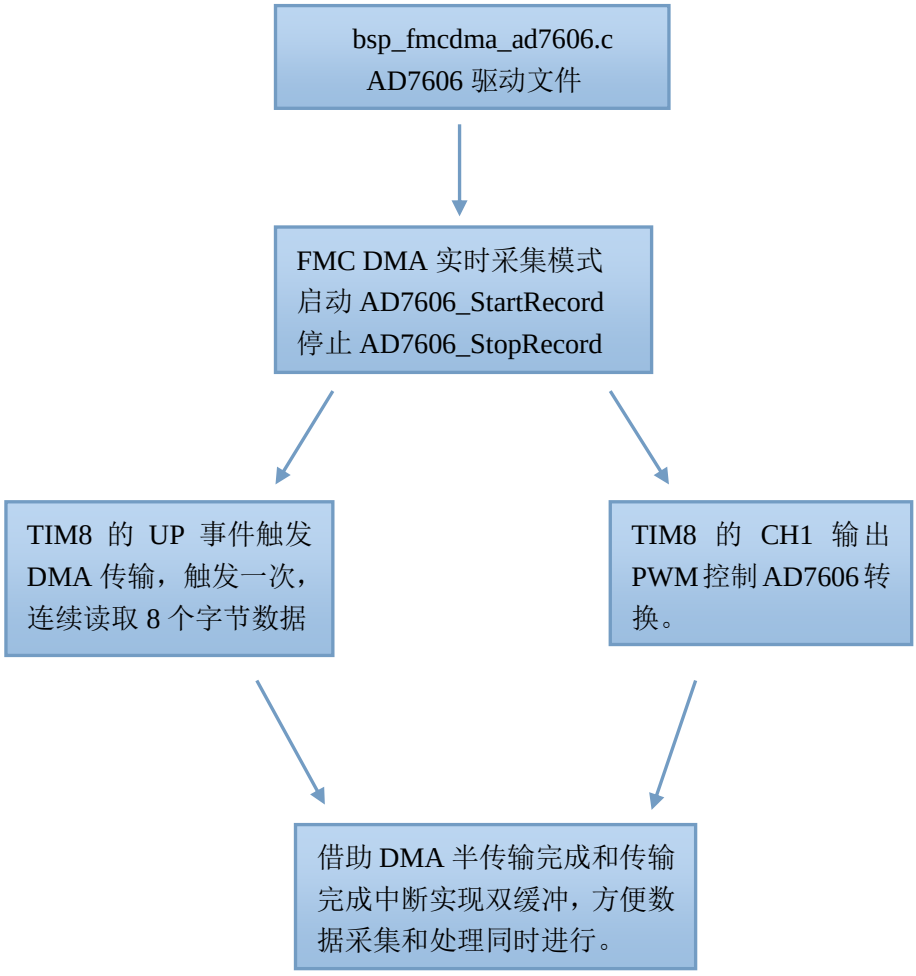
有了这三个参数，配置 PWM 的占空比就比较考究了，我们仅需配置好 PMW 低电平宽度，将其设置为接近于 25ns 的低电平脉宽时间，其余时间全是高电平即可。这样我们就保证了每次脉冲立即读取上一次的转换数据。

77.6.4 FMC DMA 双缓冲实现

DMA 双缓冲的实现比较简单，我们借助 DMA 半传输完成中断和 DMA 传输完成中断即可。其中半传输完成中断就是 DMA 数据传输完成一半的中断。

77.7 AD7606 的 FMC 接口驱动设计

AD7606 的程序驱动框架设计如下：



有了这个框图，程序设计就比较好理解了。

77.7.1第 1 步，AD7606 所涉及到的 GPIO 配置

这里需要把用到的 GPIO 时钟、FMC 时钟、GPIO 引脚和复用配置好即可：

```
/*
*********************************************************************************************************
*    函 数 名: AD7606_CtrlLinesConfig
*    功能说明: 配置 GPIO 口线，FMC 管脚设置为复用功能
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
/*
    安富莱 STM32-H7 开发板接线方法：4 片 74HC574 挂在 FMC 32 位总线上。1 个地址端口可以扩展出 32 个 IO
    PD0/FMC_D2
    PD1/FMC_D3
    PD4/FMC_NOE        ---- 读控制信号，OE = Output Enable ， N 表示低有效
    PD5/FMC_NWE        -XX- 写控制信号，AD7606 只有读，无写信号
    PD8/FMC_D13
    PD9/FMC_D14
    PD10/FMC_D15
    PD14/FMC_D0
    PD15/FMC_D1

    PE7/FMC_D4
    PE8/FMC_D5
    PE9/FMC_D6
    PE10/FMC_D7
    PE11/FMC_D8
    PE12/FMC_D9
    PE13/FMC_D10
    PE14/FMC_D11
    PE15/FMC_D12

    PG0/FMC_A10        --- 和主片选 FMC_NE2 一起译码
    PG1/FMC_A11        --- 和主片选 FMC_NE2 一起译码
    PD7/FMC_NE1        --- 主片选（OLED, 74HC574, DM9000, AD7606）

     +------------------+------------------+
     +   32-bits Mode: D31-D16              +
     +------------------+------------------+
     | PH8 <-> FMC_D16  | PI0 <-> FMC_D24  |
     | PH9 <-> FMC_D17  | PI1 <-> FMC_D25  |
     | PH10 <-> FMC_D18 | PI2 <-> FMC_D26  |
     | PH11 <-> FMC_D19 | PI3 <-> FMC_D27  |
     | PH12 <-> FMC_D20 | PI6 <-> FMC_D28  |
     | PH13 <-> FMC_D21 | PI7 <-> FMC_D29  |
     | PH14 <-> FMC_D22 | PI9 <-> FMC_D30  |
     | PH15 <-> FMC_D23 | PI10 <-> FMC_D31 |
     +------------------+------------------+
*/

/*
    控制 AD7606 参数的其他 IO 分配在扩展的 74HC574 上
    X13 - AD7606_OS0
    X14 - AD7606_OS1
```

```
    X15 - AD7606_OS2
    X24 - AD7606_RESET
    X25 - AD7606_RAGE

    PE5 - AD7606_BUSY
*/
static void AD7606_CtrlLinesConfig(void)
{
    /* bsp_fm_io 已配置 fmc, bsp_InitExtIO();
       此处可以不必重复配置
    */

    GPIO_InitTypeDef gpio_init_structure;

    /* 使能 GPIO 时钟 */
    __HAL_RCC_GPIOD_CLK_ENABLE();
    __HAL_RCC_GPIOE_CLK_ENABLE();
    __HAL_RCC_GPIOF_CLK_ENABLE();
    __HAL_RCC_GPIOG_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOI_CLK_ENABLE();

    /* 使能 FMC 时钟 */
    __HAL_RCC_FMC_CLK_ENABLE();

    /* 设置 GPIOD 相关的 IO 为复用推挽输出 */
    gpio_init_structure.Mode = GPIO_MODE_AF_PP;
    gpio_init_structure.Pull = GPIO_PULLUP;
    gpio_init_structure.Speed = GPIO_SPEED_FREQ_HIGH;
    gpio_init_structure.Alternate = GPIO_AF12_FMC;

    /* 配置 GPIOD */
    gpio_init_structure.Pin = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_7 |
                              GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 | GPIO_PIN_14 |
                              GPIO_PIN_15;
    HAL_GPIO_Init(GPIOD, &gpio_init_structure);

    /* 配置 GPIOE */
    gpio_init_structure.Pin = GPIO_PIN_7 | GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 |
                              GPIO_PIN_11 | GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14 |
                              GPIO_PIN_15;
    HAL_GPIO_Init(GPIOE, &gpio_init_structure);

    /* 配置 GPIOG */
    gpio_init_structure.Pin = GPIO_PIN_0 | GPIO_PIN_1;
    HAL_GPIO_Init(GPIOG, &gpio_init_structure);

    /* 配置 GPIOH */
    gpio_init_structure.Pin = GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 | GPIO_PIN_11 | GPIO_PIN_12
                            | GPIO_PIN_13 | GPIO_PIN_14 | GPIO_PIN_15;
    HAL_GPIO_Init(GPIOH, &gpio_init_structure);

    /* 配置 GPIOI */
    gpio_init_structure.Pin = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3 | GPIO_PIN_6
                            | GPIO_PIN_7 | GPIO_PIN_9 | GPIO_PIN_10;
    HAL_GPIO_Init(GPIOI, &gpio_init_structure);

    /* CONVST 启动 ADC 转换的 GPIO = PC6 */
```

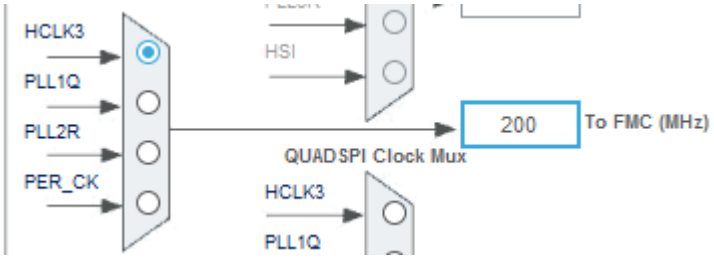
```
    {
        GPIO_InitTypeDef   GPIO_InitStructure;
        CONVST_RCC_GPIO_CLK_ENABLE();

        /* 配置 PC6 */
        GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;      /* 设置推挽输出 */
        GPIO_InitStructure.Pull = GPIO_NOPULL;              /* 上下拉电阻不使能 */
        GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_MEDIUM;  /* GPIO 速度等级 */

        GPIO_InitStructure.Pin = CONVST_PIN;
        HAL_GPIO_Init(CONVST_GPIO, &GPIO_InitStructure);
    }
}
```

这里重点注意 AD7606_CONVST，上电后的默认配置是普通 IO。另外还有过采样的 3 个引脚，量程配置的 1 个引脚和复位控制的 1 个引脚，均通过 V7 板子的扩展 IO 实现：

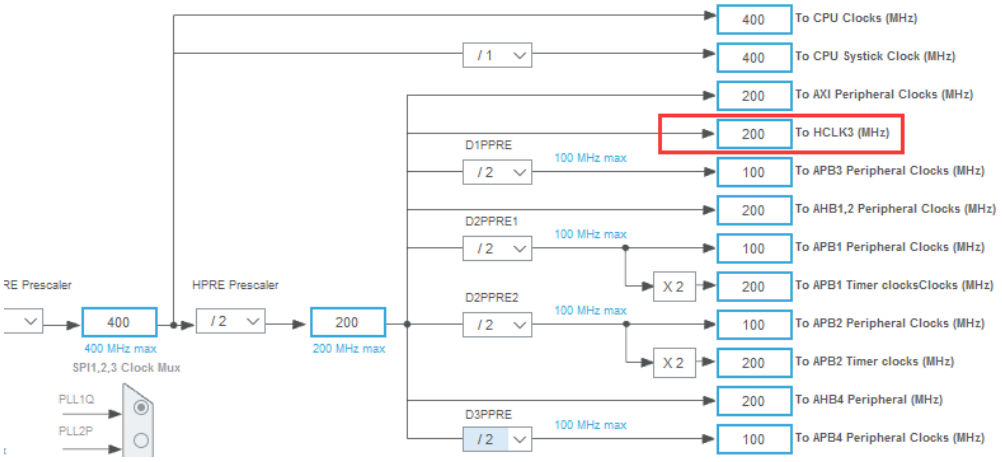
```
/* 设置过采样的 IO，在扩展的 74HC574 上 */
#define OS0_1()         HC574_SetPin(AD7606_OS0, 1)
#define OS0_0()         HC574_SetPin(AD7606_OS0, 0)
#define OS1_1()         HC574_SetPin(AD7606_OS1, 1)
#define OS1_0()         HC574_SetPin(AD7606_OS1, 0)
#define OS2_1()         HC574_SetPin(AD7606_OS2, 1)
#define OS2_0()         HC574_SetPin(AD7606_OS2, 0)

/* 设置输入量程的 GPIO，在扩展的 74HC574 上 */
#define RANGE_1()  HC574_SetPin(AD7606_RANGE, 1)
#define RANGE_0()  HC574_SetPin(AD7606_RANGE, 0)

/* AD7606 复位口线，在扩展的 74HC574 上 */
#define RESET_1()  HC574_SetPin(AD7606_RESET, 1)
#define RESET_0()  HC574_SetPin(AD7606_RESET, 0)
```

77.7.2 第 2 步，FMC 的时钟源选择

使用 FMC 可以选择如下几种时钟源 HCLK3，PLL1Q，PLL2R 和 PER_CK：

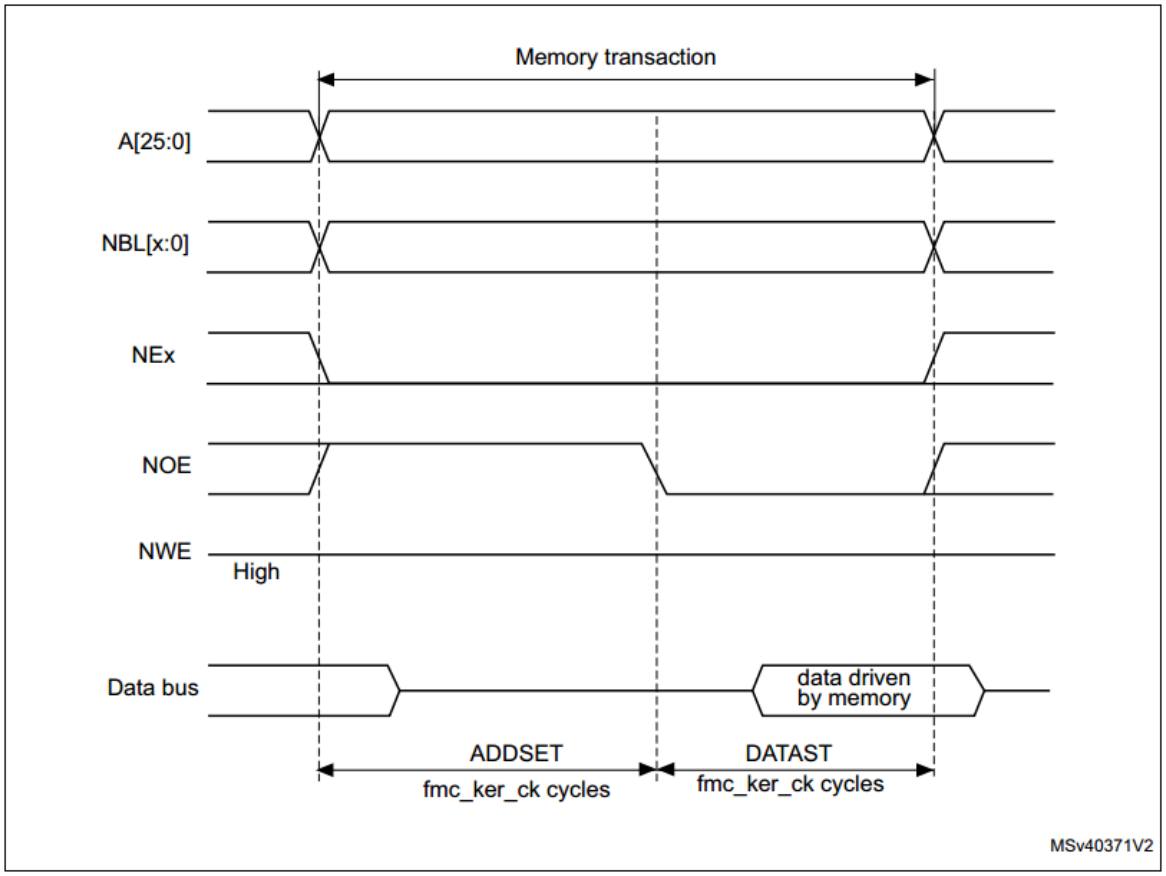


我们这里直接使用 HCLK3，配置 STM32H7 的主频为 400MHz 的时候，HCLK3 输出的 200MHz，这个速度是 FMC 支持的最高时钟，正好用于这里：



77.7.3第 3 步，FMC 的时序配置（重要）

由于操作 AD7606 仅需要读操作，而且使用的是 FMC 总线的 Mode_A，那么仅需按照如下时序图配置好即可：



根据这个时序图，重点配置好 ADDSET 地址建立时间和 DATAST 数据建立时间即可。

- DATAST（DataSetupTime，数据建立时间）

  DATAST 实际上对应的就是 76.4.4 小节里面的 t_{10} 。RD 读信号的低电平脉冲宽度，通信电压不同，时间不同，对于 STM32 来说，FMC 通信电平一般是 3.3V，即最小值 21ns。

16	ns	V_{DRIVE}高于4.75 V
21	ns	V_{DRIVE}高于3.3 V
25	ns	V_{DRIVE}高于2.7 V
32	ns	V_{DRIVE}高于2.3 V
15	ns	$\overline{RD}$高电平脉冲宽度
22	ns	$\overline{CS}$高电平脉冲宽度(见图5)；$\overline{CS}$与$\overline{RD}$相连

◆ ADDST（AddressSetupTime，地址建立时间）

DATAST 实际上对应的就是 76.4.4 小节里面的 t_{11} 或者 t_{12}。

- 如果采用 CS（NEx）片选和 RD（NOE）读信号独立方式，对应的时间最小 15ns，即 t_{11} 。
- 如果采用 CS（NEx）片选和 RD（NOE）读信号并联方式，对应的时间最小 22ns，即 t_{12} 。

我们这里将 t_{12} 作为最小值更合理，因为 CS（NEx）片选信号，每读取完毕一路，拉高一次。

有了这些认识后，再来看 FMC 的时序配置就比较好理解了：

```
/*
*********************************************************************************************************
*    函 数 名: AD7606_FSMCConfig
*    功能说明: 配置 FSMC 并口访问时序
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void AD7606_FSMCConfig(void)
{
    /*
       DM9000，扩展 IO，OLED 和 AD7606 公用一个 FMC 配置，如果都开启，请以 FMC 速度最慢的为准。
       从而保证所有外设都可以正常工作。
    */
    SRAM_HandleTypeDef hsram = {0};
    FMC_NORSRAM_TimingTypeDef SRAM_Timing = {0};

   /*
    AD7606 规格书要求(3.3V 时，通信电平 Vdriver)：RD 读信号低电平脉冲宽度最短 21ns，对应 DataSetupTime
    CS 片选和 RD 读信号独立方式的高电平脉冲最短宽度 15ns。
    CS 片选和 RD 读信号并联方式的高电平脉冲最短宽度 22ns。
    这里将 22ns 作为最小值更合理些，对应 FMC 的 AddressSetupTime。

         5-x-5-x-x-x  : RD 高持续 25ns， 低电平持续 25ns. 读取 8 路样本数据到内存差不多就是 400ns。
    */
    hsram.Instance  = FMC_NORSRAM_DEVICE;
    hsram.Extended  = FMC_NORSRAM_EXTENDED_DEVICE;

    /* FMC 使用的 HCLK3，主频 200MHz，1 个 FMC 时钟周期就是 5ns */
    SRAM_Timing.AddressSetupTime       = 5; /* 5*5ns=25ns，地址建立时间，范围 0 -15 个 FMC 时钟周期个数 */
    SRAM_Timing.AddressHoldTime        = 2; /* 地址保持时间，配置为模式 A 时，用不到此参数 范围 1 -15 个
                                               时钟周期个数 */
    SRAM_Timing.DataSetupTime          = 5;  /* 5*5ns=25ns，数据建立时间，范围 1 -255 个时钟周期个数 */
    SRAM_Timing.BusTurnAroundDuration  = 1;  /* 此配置用不到这个参数 */
    SRAM_Timing.CLKDivision            = 2;  /* 此配置用不到这个参数 */
    SRAM_Timing.DataLatency            = 2;  /* 此配置用不到这个参数 */
    SRAM_Timing.AccessMode             = FMC_ACCESS_MODE_A; /* 配置为模式 A */
```

```
38.     hsram.Init.NSBank             = FMC_NORSRAM_BANK1;               /* 使用的 BANK1，即使用的片选
39.                                                                        FMC_NE1 */
40.     hsram.Init.DataAddressMux     = FMC_DATA_ADDRESS_MUX_DISABLE;    /* 禁止地址数据复用 */
41.     hsram.Init.MemoryType         = FMC_MEMORY_TYPE_SRAM;            /* 存储器类型 SRAM */
42.     hsram.Init.MemoryDataWidth    = FMC_NORSRAM_MEM_BUS_WIDTH_32;    /* 32 位总线宽度 */
43.     hsram.Init.BurstAccessMode    = FMC_BURST_ACCESS_MODE_DISABLE;   /* 关闭突发模式 */
44.     hsram.Init.WaitSignalPolarity = FMC_WAIT_SIGNAL_POLARITY_LOW;    /* 用于设置等待信号的极性，关闭突
45.                                                                        发模式，此参数无效 */
46.     hsram.Init.WaitSignalActive   = FMC_WAIT_TIMING_BEFORE_WS;       /* 关闭突发模式，此参数无效 */
47.     hsram.Init.WriteOperation     = FMC_WRITE_OPERATION_ENABLE;      /* 用于使能或者禁止写保护 */
48.     hsram.Init.WaitSignal         = FMC_WAIT_SIGNAL_DISABLE;         /* 关闭突发模式，此参数无效 */
49.     hsram.Init.ExtendedMode       = FMC_EXTENDED_MODE_DISABLE;       /* 禁止扩展模式 */
50.     hsram.Init.AsynchronousWait   = FMC_ASYNCHRONOUS_WAIT_DISABLE;   /* 用于异步传输期间，使能或者禁止
51.                                                                        等待信号，这里选择关闭 */
52.     hsram.Init.WriteBurst         = FMC_WRITE_BURST_DISABLE;         /* 禁止写突发 */
53.     hsram.Init.ContinuousClock    = FMC_CONTINUOUS_CLOCK_SYNC_ONLY;  /* 仅同步模式才做时钟输出 */
54.     hsram.Init.WriteFifo          = FMC_WRITE_FIFO_ENABLE;           /* 使能写 FIFO */
55.
56.     /* 初始化 SRAM 控制器 */
57.     if (HAL_SRAM_Init(&hsram, &SRAM_Timing, &SRAM_Timing) != HAL_OK)
58.     {
59.         /* 初始化错误 */
60.         Error_Handler(__FILE__, __LINE__);
61.     }
62. }
```

这里把几个关键的地方阐释下：

- 第 15- 16 行，对作为局部变量的 HAL 库结构体做初始化，防止不确定值配置时出问题。
- 第 30 行，地址建立时间，对于 AD7606 来说，这个地方最小值 22ns。保险起见，这里取值 5 个 FMC 时钟周期，即 25ns。
- 第 31 行，地址保持时间，对于 FMC 模式 A 来说，此参数用不到。
- 第 33 行，数据建立时间，对于 AD7606 来说，这个地方最小值是 21ns，保险起见，这里取值 5 个 FMC 时钟周期，即 25ns。
- 第 34 – 36 行，当前配置用不到这三个参数。
- 第 38 行，使用的 BANK1，即使用的片选 FMC_NE1。

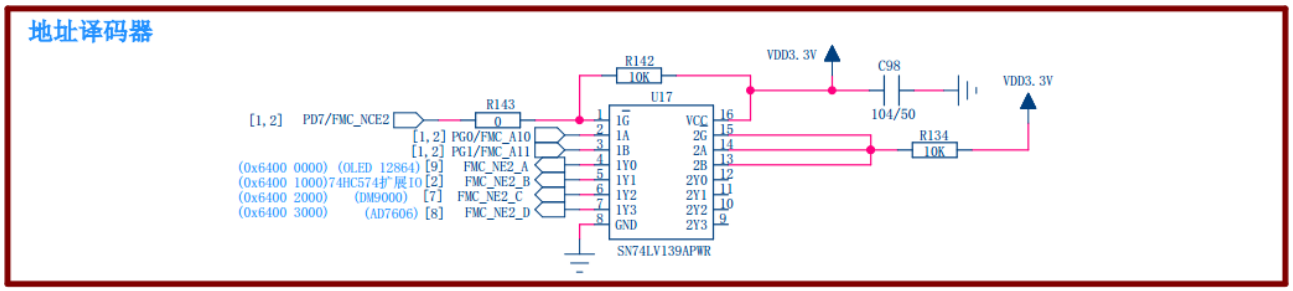
77.7.4第 4 步，FMC 的 MPU 配置

实际测试发现，使能 FMC_NE1 所管理的存储区的 Cache 功能后，会出现扩展 IO 的 NE 片选和 NWE 信号输出 2 次的问题。经过各种 Cache 方式配置、FMC 带宽配置、操作 FMC 时的数据位宽设置，发现禁止了 Cache 功能就正常了，也就是说，设置 FMC_NE1 所管理的存储区 MPU 属性为 Device 或者 Strongly Ordered 即可。

```
    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
```

```
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

MPU 配置中直接从 FMC_NE1 的首地址开始配置，设置了 64KB 空间的属性。将 FMC_NE1 通过译码器所管理的所有设备地址全部设置为此配置：



77.7.5第 5 步，AD7606 的 FMC DMA 实现（核心）

这部分代码是本章 77.6 小节的完美体现：

```
1.  static void AD7606_SetTIMOutPWM(TIM_TypeDef* TIMx, uint32_t _ulFreq)
2.  {
3.      TIM_OC_InitTypeDef sConfig = {0};
4.      GPIO_InitTypeDef   GPIO_InitStruct;
5.      uint16_t usPeriod;
6.      uint16_t usPrescaler;
7.      uint32_t uiTIMxCLK;
8.      uint32_t pulse;
9.
10.
11.     /* 配置时钟 */
12.     CONVST_RCC_GPIO_CLK_ENABLE();
13.     CONVST_TIM8_CLK_ENABLE();
14.     TIMx_UP_DMA_STREAM_CLK_ENABLE();
15.
16.     /* 配置引脚 */
17.     GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
18.     GPIO_InitStruct.Pull = GPIO_PULLUP;
19.     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
20.     GPIO_InitStruct.Alternate = CONVST_AF;
21.     GPIO_InitStruct.Pin = CONVST_PIN;
22.     HAL_GPIO_Init(CONVST_GPIO, &GPIO_InitStruct);
23.
24.     /*-----------------------------------------------------------------------
25.         bsp.c 文件中 void SystemClock_Config(void) 函数对时钟的配置如下:
26.
27.         System Clock source       = PLL (HSE)
28.         SYSCLK(Hz)                = 400000000 (CPU Clock)
29.         HCLK(Hz)                  = 200000000 (AXI and AHBs Clock)
30.         AHB Prescaler             = 2
31.         D1 APB3 Prescaler         = 2 (APB3 Clock  100MHz)
```

```
32.          D2 APB1 Prescaler         = 2 (APB1 Clock  100MHz)
33.          D2 APB2 Prescaler         = 2 (APB2 Clock  100MHz)
34.          D3 APB4 Prescaler         = 2 (APB4 Clock  100MHz)
35.
36.          因为 APB1 prescaler != 1, 所以 APB1 上的 TIMxCLK = APB1 x 2 = 200MHz;
37.          因为 APB2 prescaler != 1, 所以 APB2 上的 TIMxCLK = APB2 x 2 = 200MHz;
38.          APB4 上面的 TIMxCLK 没有分频，所以就是 100MHz;
39.
40.          APB1 定时器有 TIM2, TIM3 ,TIM4, TIM5, TIM6, TIM7, TIM12, TIM13, TIM14，LPTIM1
41.          APB2 定时器有 TIM1, TIM8 , TIM15, TIM16，TIM17
42.
43.          APB4 定时器有 LPTIM2，LPTIM3，LPTIM4，LPTIM5
44.
45.      ------------------------------------------------------------------------ */
46.      if ((TIMx == TIM1) || (TIMx == TIM8) || (TIMx == TIM15) || (TIMx == TIM16) || (TIMx == TIM17))
47.      {
48.          /* APB2 定时器时钟 = 200M */
49.          uiTIMxCLK = SystemCoreClock / 2;
50.      }
51.      else
52.      {
53.          /* APB1 定时器 = 200M */
54.          uiTIMxCLK = SystemCoreClock / 2;
55.      }
56.
57.      if (_ulFreq < 100)
58.      {
59.          usPrescaler = 10000 - 1;                    /* 分频比 = 10000 */
60.          usPeriod =  (uiTIMxCLK / 10000) / _ulFreq  - 1; /* 自动重装的值， usPeriod 最小值 200，单位 50us*/
61.          pulse = usPeriod;                  /* 设置低电平时间 50us，注意 usPeriod 已经进行了减 1 操作 */
62.      }
63.      else if (_ulFreq < 3000)
64.      {
65.          usPrescaler = 100 - 1;                    /* 分频比 = 100 */
66.          usPeriod =  (uiTIMxCLK / 100) / _ulFreq -1;/* 自动重装的值， usPeriod 最小值 666，单位 500ns */
67.          pulse = usPeriod-1;                  /* 设置低电平时间 1us，注意 usPeriod 已经进行了减 1 操作 */
68.      }
69.      else /* 大于 4K 的频率，无需分频 */
70.      {
71.          usPrescaler = 0;                      /* 分频比 = 1 */
72.          usPeriod = uiTIMxCLK / _ulFreq - 1;  /* 自动重装的值， usPeriod 最小值 1000，单位 5ns */
73.          pulse = usPeriod - 199;               /* 设置低电平时间 1us,注意 usPeriod 已经进行了减 1 操作 */
74.      }
75.
76.      /*  PWM 频率 = TIMxCLK / usPrescaler + 1）/usPeriod + 1）*/
77.      TimHandle.Instance = TIMx;
78.      TimHandle.Init.Prescaler         = usPrescaler;         /* 用于设置定时器分频 */
79.      TimHandle.Init.Period            = usPeriod;            /* 用于设置定时器周期 */
80.      TimHandle.Init.ClockDivision     = 0;                   /* 用于指示定时器时钟（CK_INT）频率与死区
81.                                                                 发生器以及数字滤波器（ETR、 TIx）所使用
82.                                                                 的死区及采样时钟 （tDTS）之间的分频比*/
83.      TimHandle.Init.CounterMode       = TIM_COUNTERMODE_UP;  /* 用于设置计数模式，向上计数模式 */
84.      TimHandle.Init.RepetitionCounter = 0;  /* 用于设置重复计数器，仅 TIM1 和 TIM8 有，其它定时器没有 */
85.      TimHandle.Init.AutoReloadPreload = 0; /* 用于设置定时器的 ARR 自动重装寄存器是更新事件产生时写入有
86.                                               效 */
87.
88.      if (HAL_TIM_PWM_DeInit(&TimHandle) != HAL_OK)
89.      {
```

```
90.           Error_Handler(__FILE__, __LINE__);
91.       }
92.
93.       if (HAL_TIM_PWM_Init(&TimHandle) != HAL_OK)
94.       {
95.           Error_Handler(__FILE__, __LINE__);
96.       }
97.
98.       /* 配置定时器 PWM 输出通道 */
99.       sConfig.OCMode       = TIM_OCMODE_PWM1;          /* 配置输出比较模式 */
100.      sConfig.OCPolarity   = TIM_OCPOLARITY_HIGH;      /* 设置输出高电平有效 */
101.      sConfig.OCFastMode   = TIM_OCFAST_DISABLE;       /* 关闭快速输出模式 */
102.      sConfig.OCNPolarity  = TIM_OCNPOLARITY_HIGH;     /* 配置互补输出高电平有效 */
103.      sConfig.OCIdleState  = TIM_OCIDLESTATE_SET;      /* 空闲状态时，设置输出比较引脚为高电平 */
104.      sConfig.OCNIdleState = TIM_OCNIDLESTATE_RESET;   /* 空闲状态时，设置互补输出比较引脚为低电平 */
105.
106.      /* 占空比 */
107.      sConfig.Pulse = pulse;
108.      if (HAL_TIM_PWM_ConfigChannel(&TimHandle, &sConfig, CONVST_TIMCH) != HAL_OK)
109.      {
110.          Error_Handler(__FILE__, __LINE__);
111.      }
112.
113.      /* 使能定时器中断  */
114.      __HAL_TIM_ENABLE_DMA(&TimHandle, TIM_DMA_UPDATE);
115.
116.      /* 启动 PWM 输出 */
117.      if (HAL_TIM_PWM_Start(&TimHandle, CONVST_TIMCH) != HAL_OK)
118.      {
119.          Error_Handler(__FILE__, __LINE__);
120.      }
121.
122.      /* 定时器 UP 更新触发 DMA 传输 */
123.      TIMDMA.Instance                 = TIMx_UP_DMA_STREAM;       /* 例化使用的 DMA 数据流 */
124.      TIMDMA.Init.FIFOMode            = DMA_FIFOMODE_ENABLE;      /* 使能 FIFO*/
125.      TIMDMA.Init.FIFOThreshold       = DMA_FIFO_THRESHOLD_FULL; /* 用于设置阀值 */
126.      TIMDMA.Init.MemBurst            = DMA_MBURST_INC8;         /* 用于存储器突发 */
127.      TIMDMA.Init.PeriphBurst         = DMA_PBURST_INC8;         /* 用于外设突发 */
128.      TIMDMA.Init.Request             = TIMx_UP_DMA_REQUEST;      /* 请求类型 */
129.      TIMDMA.Init.Direction           = DMA_PERIPH_TO_MEMORY;     /* 传输方向是从外设到存储器 */
130.      TIMDMA.Init.PeriphInc           = DMA_PINC_DISABLE;         /* 外设地址自增禁止 */
131.      TIMDMA.Init.MemInc              = DMA_MINC_ENABLE;          /* 存储器地址自增使能 */
132.      TIMDMA.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD; /* 外设数据传输位宽选择半字，即 16bit */
133.      TIMDMA.Init.MemDataAlignment    =DMA_MDATAALIGN_HALFWORD; /* 存储器数据传输位宽选择半字,即 16bit */
134.      TIMDMA.Init.Mode                = DMA_CIRCULAR;             /* 循环模式 */
135.      TIMDMA.Init.Priority            = DMA_PRIORITY_LOW;         /* 优先级低 */
136.
137.       /* 复位 DMA */
138.      if(HAL_DMA_DeInit(&TIMDMA) != HAL_OK)
139.      {
140.          Error_Handler(__FILE__, __LINE__);
141.      }
142.
143.       /* 初始化 DMA */
144.      if(HAL_DMA_Init(&TIMDMA) != HAL_OK)
145.      {
146.          Error_Handler(__FILE__, __LINE__);
147.      }
```

```
148.
149.      /* 关联 DMA 句柄到 TIM */
150.      //__HAL_LINKDMA(&TimHandle, hdma[TIM_DMA_ID_UPDATE], TIMDMA);
151.
152.      /* 配置 DMA 中断 */
153.      HAL_NVIC_SetPriority(TIMx_UP_DMA_IRQn, 1, 0);
154.      HAL_NVIC_EnableIRQ(TIMx_UP_DMA_IRQn);
155.
156.      /* 注册半传输完成中断和传输完成中断 */
157.      HAL_DMA_RegisterCallback(&TIMDMA, HAL_DMA_XFER_CPLT_CB_ID, AD7606_DmaCplCb);
158.      HAL_DMA_RegisterCallback(&TIMDMA, HAL_DMA_XFER_HALFCPLT_CB_ID, AD7606_DmaHalfCplCb);
159.
160.      /* 启动 DMA 传输 */
161.      HAL_DMA_Start_IT(&TIMDMA, (uint32_t)AD7606_BASE, (uint32_t)g_sAd7606Buf, AD7606_BUFSIZE);
162. }
```

- 第 46 – 74 行，配置 PWM 频率和占空比，特别是占比设计比较考究。
- 第 123-135 行，配置 DMA，特别注意突发和 FIFO 设置，完全按照 78.6.2 小节配置。
- 第 157-158 行，注册半传输完成中断和传输完成中断的回调函数。

77.7.6第 6 步，FMC DMA 双缓冲

通过注册半传输完成中断和传输完成中断回调函数实现双缓冲：

```
/* DMA 传输完成回调函数，弱定义 */
__weak void AD7606_DmaCplCb(DMA_HandleTypeDef *hdma)
{

}

/* DMA 半传输完成回调函数，弱定义 */
__weak void AD7606_DmaHalfCplCb(DMA_HandleTypeDef *hdma)
{

}
```

比如用户设置的 DMA 缓冲是 int16_t buf[16],那么进入半传输完成回调,用户就可以处理 buf[0]到 buf[7]里面的数据，进入传输完成中断里面，处理 buf[8]到 buf[15]里面的数据。

77.7.7第 7 步，AD7606 过采样设置

AD7606 的过采样实现比较简单，通过 IO 引脚就可以控制，支持 2 倍，4 倍，8 倍，16 倍，32 倍和 64 倍过采样设置。

```
/*
*********************************************************************************************************
*    函 数 名: AD7606_SetOS
*    功能说明: 配置 AD7606 数字滤波器，也就设置过采样倍率。
*              通过设置 AD7606_OS0、OS1、OS2 口线的电平组合状态决定过采样倍率。
*              启动 AD 转换之后，AD7606 内部自动实现剩余样本的采集，然后求平均值输出。
*
*              过采样倍率越高，转换时间越长。
*              0、无过采样时，AD 转换时间 = 3.45us - 4.15us
```

```
*                1、2 倍过采样时 = 7.87us - 9.1us
*                2、4 倍过采样时 = 16.05us - 18.8us
*                3、8 倍过采样时 = 33us - 39us
*                4、16 倍过采样时 = 66us - 78us
*                5、32 倍过采样时 = 133us - 158us
*                6、64 倍过采样时 = 257us - 315us
*
*    形    参: _ucOS : 过采样倍率, 0 - 6
*    返 回 值: 无
*********************************************************************************************************
*/
void AD7606_SetOS(uint8_t _ucOS)
{
    g_tAD7606.ucOS = _ucOS;
    switch (_ucOS)
    {
        case AD_OS_X2:
            OS2_0();
            OS1_0();
            OS0_1();
            break;

        case AD_OS_X4:
            OS2_0();
            OS1_1();
            OS0_0();
            break;

        case AD_OS_X8:
            OS2_0();
            OS1_1();
            OS0_1();
            break;

        case AD_OS_X16:
            OS2_1();
            OS1_0();
            OS0_0();
            break;

        case AD_OS_X32:
            OS2_1();
            OS1_0();
            OS0_1();
            break;

        case AD_OS_X64:
            OS2_1();
            OS1_1();
            OS0_0();
            break;

        case AD_OS_NO:
        default:
            g_tAD7606.ucOS = AD_OS_NO;
            OS2_0();
            OS1_0();
            OS0_0();
```

```
            break;
    }
}
```

77.7.8第 8 步，AD7606 量程设置

AD7606 支持两种量程，±5V 和±10V，实现代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: AD7606_SetInputRange
*    功能说明: 配置 AD7606 模拟信号输入量程。
*    形    参: _ucRange : 0 表示正负 5V   1 表示正负 10V
*    返 回 值: 无
*********************************************************************************************************
*/
void AD7606_SetInputRange(uint8_t _ucRange)
{
    if (_ucRange == 0)
    {
        g_tAD7606.ucRange = 0;
        RANGE_0();    /* 设置为正负 5V */
    }
    else
    {
        g_tAD7606.ucRange = 1;
        RANGE_1();    /* 设置为正负 10V */
    }
}
```

77.7.9第 9 步，DMA 突发传输的 1KB 边界处理

针对突发传输，参考手册 DMA 章节有如下说明：

> 选择突发配置必须要遵守 AHB 协议，即突发传输*不得*越过 1 KB 地址边界，因为可以分配给单个从设备的最小地址空间是 1 KB。这意味着突发块传输不应越过 1 KB 地址边界，否则就会产生一个 AHB 错误，并且 DMA 寄存器不会报告这个错误。

注意正确理解这段话的含义，意思是说突发传输期间，不可以跨越 1KB 对齐的地址，比如 0x2000 0400、0x2000 0800、0x2000 0C00 等地址。我们程序里面是设置的每次突发传输 16 个字节数据，这 16 个连续数据不能有跨越这些地址的情况。这对这个问题，有个比较巧妙的解决办法，直接设置 DMA 缓冲区 16 字节对齐即可，这样每次突发都不会有跨越这些地址的情况：

```
/* 8 路同步采集，每次采集 16 字节数据，防止 DMA 突发方式 1KB 边界问题，即每次采集不要有跨边界的情况 */
#define AD7606_BUFSIZE        16
__align(16) int16_t g_sAd7606Buf[AD7606_BUFSIZE];    /* DMA 双缓冲使用 */
```

77.8 AD7606 板级支持包（bsp_fmcdma_ad7606.c）

AD7606 驱动文件 bsp_fmcdma_ad7606.c 主要实现了如下几个 API 供用户调用：

- bsp_InitAD7606
- AD7606_SetOS
- AD7606_SetInputRange
- AD7606_Reset
- AD7606_StartConvst
- AD7606_ReadNowAdc
- AD7606_EnterAutoMode
- AD7606_StartRecord
- AD7606_StopRecord
- AD7606_FifoNewData
- AD7606_ReadFifo
- AD7606_FifoFull

77.8.1 函数 bsp_InitAD7606

函数原型：

```
void bsp_InitAD7606(void)
```

函数描述：

主要用于 AD7606 的初始化。

77.8.2 函数 AD7606_SetOS

函数原型：

```
void AD7606_SetOS(uint8_t _ucOS)
```

函数描述：

此函数用于配置 AD7606 数字滤波器，也就设置过采样倍率。通过设置 AD7606_OS0、OS1、OS2 口线的电平组合状态决定过采样倍率。启动 AD 转换之后，AD7606 内部自动实现剩余样本的采集，然后求平均值输出。

过采样倍率越高，转换时间越长。

无过采样时，AD 转换时间 = 3.45us - 4.15us。

2 倍过采样时 = 7.87us - 9.1us。

4 倍过采样时 = 16.05us - 18.8us。

8 倍过采样时 = 33us - 39us。

16 倍过采样时 = 66us - 78us。

32 倍过采样时 = 133us - 158us。

64 倍过采样时 = 257us - 315us。

函数参数:

- 第 1 个参数为范围 0 – 6，分别对应无过采样，2 倍过采样，4 倍过采样，8 倍过采样，16 倍过采样，32 倍过采样和 64 倍过采样。

77.8.3 函数 AD7606_SetInputRange

函数原型:

```
void AD7606_SetInputRange(uint8_t _ucRange)
```

函数描述:

配置 AD7606 模拟信号输入量程。

函数参数:

- 第 1 个参数为 0 表示正负 5V ， 1 表示正负 10V。

77.8.4 函数 AD7606_Reset

函数原型:

```
void AD7606_Reset(void)
```

函数描述:

此函数用于硬件复位 AD7606，复位之后恢复到正常工作状态。

77.8.5 函数 AD7606_StartRecord

函数原型:

```
void AD7606_StartRecord(uint32_t _ulFreq)
```

函数描述:

用于启动采集。

函数参数:

- 第 1 个参数是采样频率，范围 1-200KHz，单位 Hz。

77.8.6 函数 AD7606_StopRecord

函数原型:

```
void AD7606_StopRecord(void)
```

函数描述：

此函数用于停止采集定时器。函数 AD7606_StartRecord 和 AD7606_StopRecord 是配套的。

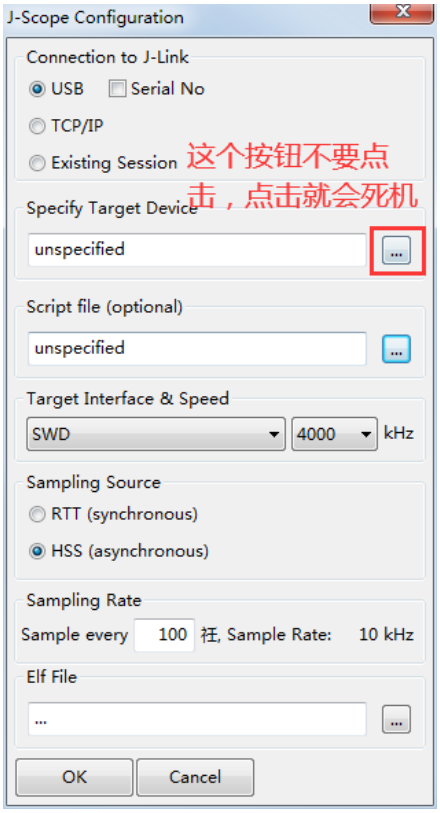
77.9 J-Scope 实时展示 AD7606 采集数据说明

J-Scope 专题教程（实时展示要用 J-Scope 的 RTT 模式），本章配套例子也做了支持：http://www.armbbs.cn/forum.php?mod=viewthread&tid=86881 。

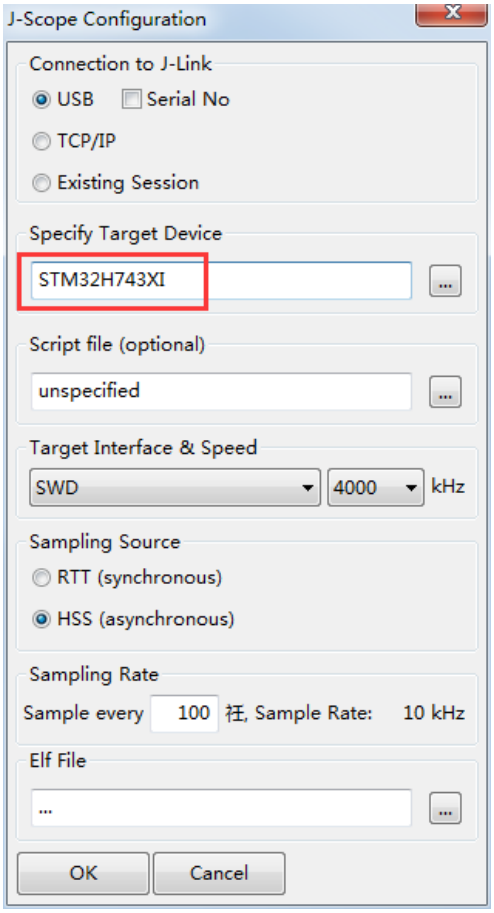
看完专题教程，基本就会操作了，这里有三点注意事项需要大家提前有个了解。另外，推荐使用 MDK 版工程做测试 J-Scope，IAR 版容易测试不正常。

77.9.1 J-Scope 闪退问题解决办法

如下界面，不要点击选择按钮，闪退就是因为点击了这个选择按钮。



直接手动填写型号即可，比如 STM32H743XI，STM32F429BI，STM32F407IG，STM32F103ZE 等。



77.9.2 J-Scope 多通道传输实现

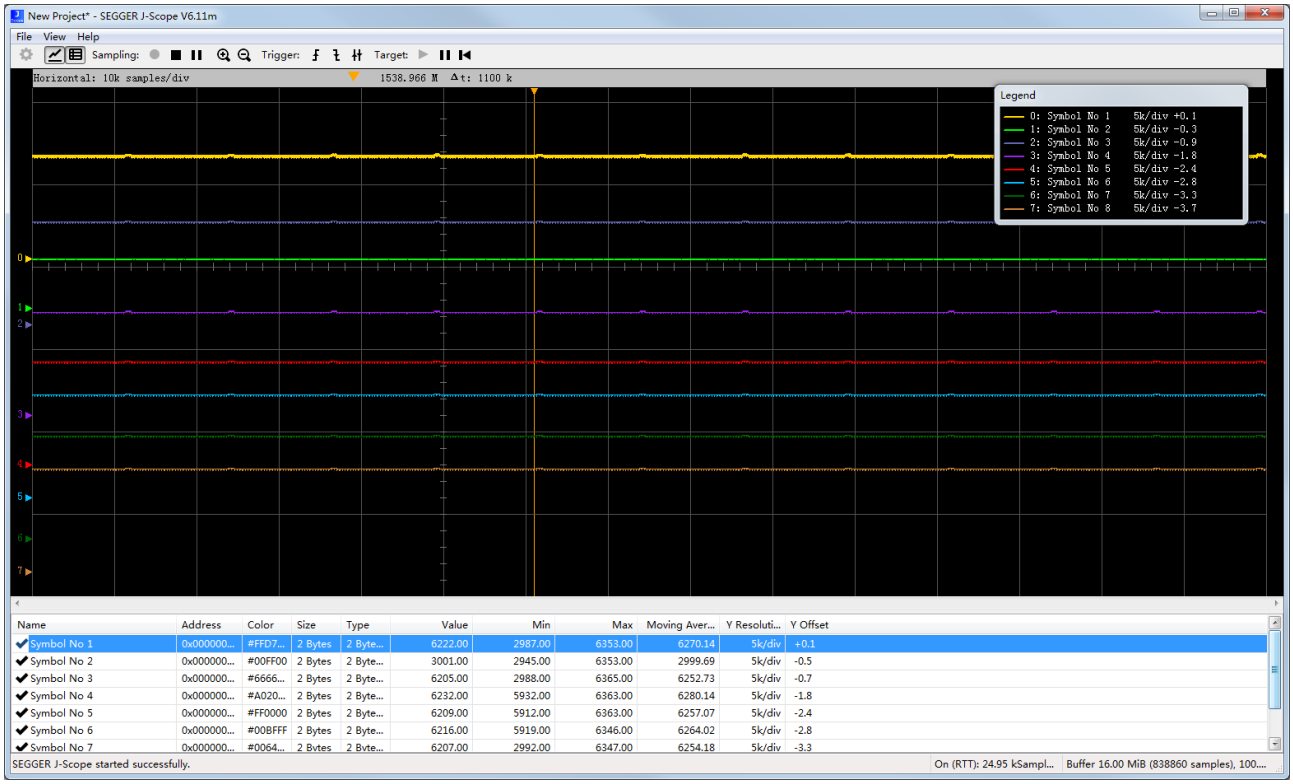
J-Scope 的多通道传输配置好函数 SEGGER_RTT_ConfigUpBuffer 即可，主要是通过第 2 个参数实现的。

```
    /*
        配置通道 1，上行配置
        默认情况下，J-Scope 仅显示 1 个通道。
        上传 1 个通道的波形，配置第 2 个参数为 JScope_i2
        上传 2 个通道的波形，配置第 2 个参数为 JScope_i2i2
        上传 3 个通道的波形，配置第 2 个参数为 JScope_i2i2i2
        上传 4 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2
        上传 5 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2
        上传 6 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2i2
        上传 7 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2i2i2
        上传 8 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2i2i2i2
    */
    SEGGER_RTT_ConfigUpBuffer(1, "JScope_i2", buf, 20480, SEGGER_RTT_MODE_NO_BLOCK_SKIP);
```

使用函数 SEGGER_RTT_Write 上传数据时，要跟配置的通道数匹配，比如配置的三个通道，就需要调用三次函数：

```
SEGGER_RTT_Write(1, &(g_tAD7606.sNowAdc[0]), 2);
SEGGER_RTT_Write(1, &(g_tAD7606.sNowAdc[1]), 2);
SEGGER_RTT_Write(1, &(g_tAD7606.sNowAdc[2]), 2);
```

多路效果：

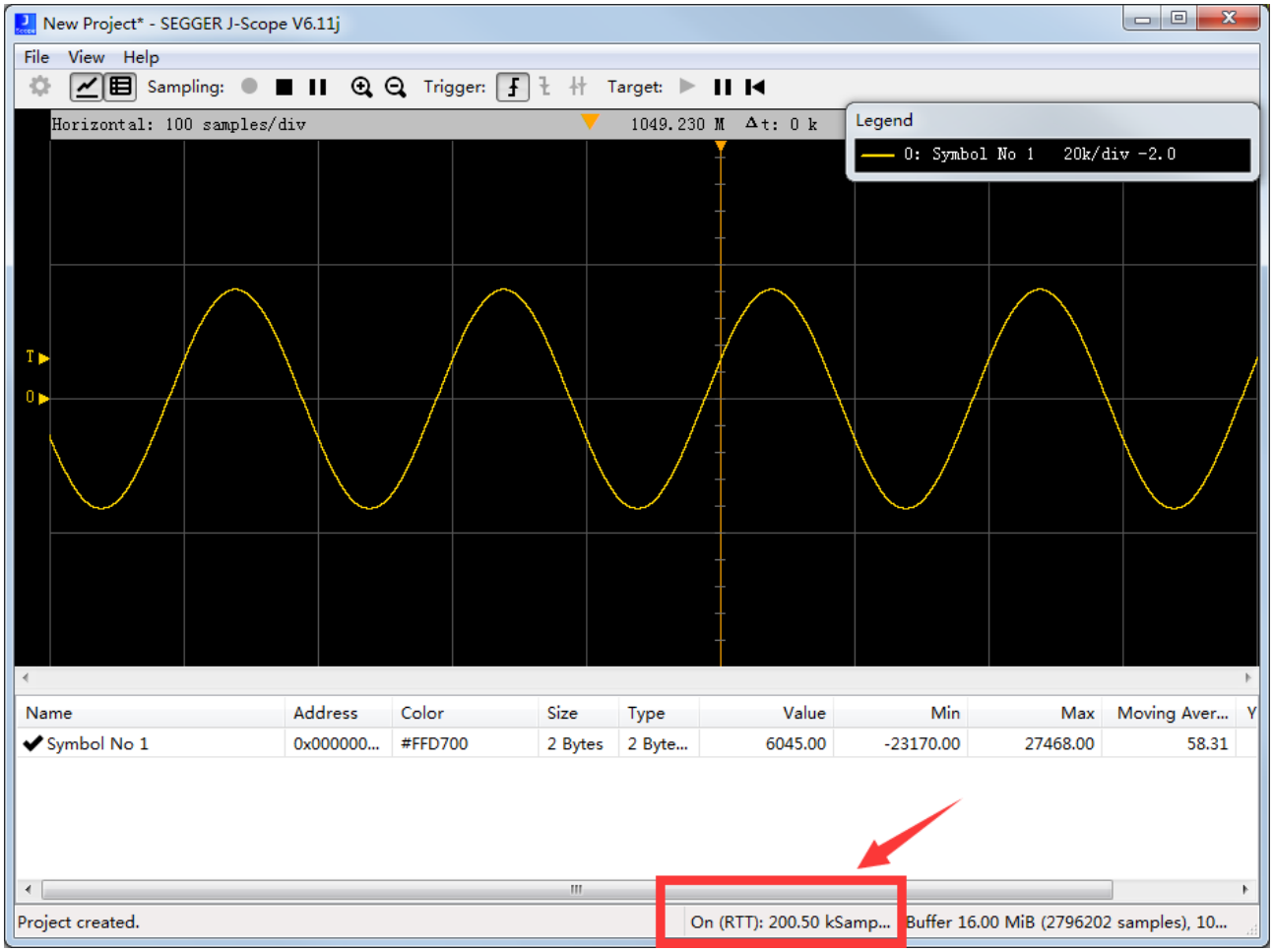


77.9.3 J-Scope 带宽问题

普通的 JLINK 时钟速度 8 - 12MHz 时， J-Scope 的速度基本可以达到 500KB/S（注意，单位是字节）AD7606 的最高采样率是 200Ksps，16bit，那么一路采集就有 400KB/S 的速速，所以要根据设置的采样率设置要显示的 J-Scope 通道数，如果超出了最高通信速度，波形显示会混乱。

200Ksps 时，实时显示 1 路
100Ksps 时，实时显示 2 路
50Ksps 时， 实时显示 4 路
25Ksps 时， 实时显示 8 路

实际速度以底栏的展示为准，如果与设置的速度差异较大，说明传输异常了。



77.10 AD7606 驱动移植和使用

AD7606 移植步骤如下：

- 第 1 步：复制 bsp_fmcdma_ad7606.c 和 bsp_fmcdma_ad7606.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：根据使用的 CONVST 引脚，FMC DMA，过采样引脚，量程控制引脚，复位引脚，修改 bsp_fmcdma_ad7606.c 开头的宏定义。

  这里要特别注意过采样引脚，量程控制引脚和复位引脚是采用的扩展 IO，需要大家根据自己的情况修改。

```
/* CONVST 启动 ADC 转换的 GPIO = PC6 */
#define CONVST_RCC_GPIO_CLK_ENABLE    __HAL_RCC_GPIOC_CLK_ENABLE
#define CONVST_TIM8_CLK_ENABLE        __HAL_RCC_TIM8_CLK_ENABLE
#define CONVST_RCC_GPIO_CLK_DISBALE   __HAL_RCC_GPIOC_CLK_DISABLE
#define CONVST_TIM8_CLK_DISABLE       __HAL_RCC_TIM8_CLK_DISABLE
#define CONVST_GPIO          GPIOC
#define CONVST_PIN           GPIO_PIN_6
#define CONVST_AF            GPIO_AF3_TIM8
#define CONVST_TIMX          TIM8
#define CONVST_TIMCH         TIM_CHANNEL_1
```

```
/* FMC DMA */
#define TIMx_UP_DMA_STREAM_CLK_ENABLE      __HAL_RCC_DMA2_CLK_ENABLE
#define TIMx_UP_DMA_STREAM_CLK_DISABLE   __HAL_RCC_DMA2_CLK_DISABLE
#define TIMx_UP_DMA_STREAM              DMA2_Stream1
#define TIMx_UP_DMA_CHANNEL             DMA_CHANNEL_7
#define TIMx_UP_DMA_IRQn                DMA2_Stream1_IRQn
#define TIMx_UP_DMA_IRQHandler          DMA2_Stream1_IRQHandler

/* BUSY 转换完毕信号 = PE5 */
#define BUSY_RCC_GPIO_CLK_ENABLE __HAL_RCC_GPIOE_CLK_ENABLE
#define BUSY_GPIO       GPIOE
#define BUSY_PIN        GPIO_PIN_5
#define BUSY_IRQn       EXTI9_5_IRQn
#define BUSY_IRQHandler EXTI9_5_IRQHandler

/* 设置过采样的 IO, 在扩展的 74HC574 上 */
#define OS0_1()         HC574_SetPin(AD7606_OS0, 1)
#define OS0_0()         HC574_SetPin(AD7606_OS0, 0)
#define OS1_1()         HC574_SetPin(AD7606_OS1, 1)
#define OS1_0()         HC574_SetPin(AD7606_OS1, 0)
#define OS2_1()         HC574_SetPin(AD7606_OS2, 1)
#define OS2_0()         HC574_SetPin(AD7606_OS2, 0)

/* 启动 AD 转换的 GPIO : PC6 */
#define CONVST_1()      CONVST_GPIO->BSRR = CONVST_PIN
#define CONVST_0()      CONVST_GPIO->BSRR = ((uint32_t)CONVST_PIN << 16U)

/* 设置输入量程的 GPIO, 在扩展的 74HC574 上 */
#define RANGE_1()  HC574_SetPin(AD7606_RANGE, 1)
#define RANGE_0()  HC574_SetPin(AD7606_RANGE, 0)

/* AD7606 复位口线, 在扩展的 74HC574 上 */
#define RESET_1()  HC574_SetPin(AD7606_RESET, 1)
#define RESET_0()  HC574_SetPin(AD7606_RESET, 0)
```

- 第 3 步：根据具体用到的 FMC 引脚，修改函数 AD7606_CtrlLinesConfig 里面做的 IO 配置
- 第 4 步：根据需要设置 DMA 缓冲大小：

```
/* 8 路同步采集，每次采集 16 字节数据，防止 DMA 突发方式 1KB 边界问题，即每次采集不要有跨边界的情况 */
#define AD7606_BUFSIZE        16
__align(16) int16_t g_sAd7606Buf[AD7606_BUFSIZE];   /* DMA 双缓冲使用 */
```

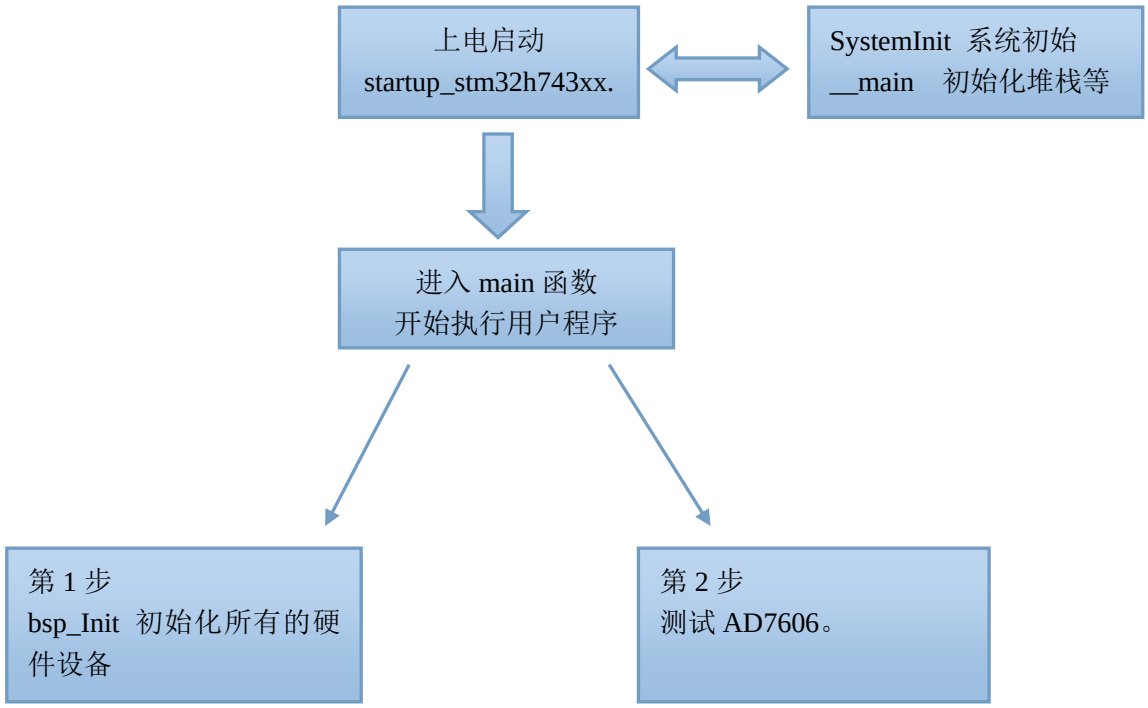
- 第 5 步：根据使用的 FMC BANK，修改函数 AD7606_FSMCConfig 里面的 BANK 配置，这点非常容易疏忽。
- 第 6 步：注意 MPU 配置，详情见本章 78.7.4 小节。
- 第 7 步：初始化 AD7606。

```
bsp_InitAD7606();  /* 配置 AD7606 所用的 GPIO */
```

- 第 8 步：AD7606 驱动主要用到 HAL 库的 FMC 驱动文件，简单省事些可以添加所有 HAL 库 C 源文件进来。
- 第 9 步：应用方法看本章节配套例子即可。

77.11 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器和 LED。
  - 第 2 部分，应用程序设计部分，测试 AD7606。

77.12 实验例程说明（MDK）

配套例子：

V7-057_ AD7606 的 FMC DMA 双缓冲总线驱动方式实现（8 通道同步采样, 16bit，正负 10V）

实验目的：

4. 学习 AD7606 的 FMC DMA 双缓冲驱动方式实现。

重要提示：

6. 板子上电后，默认是 100Ksps 的 2 倍过采样。
7. 如果使用的 JLINK 速度不够快，导致 J-Scope 无法最高速度实时上传，可以使用摇杆上下键设置过采样来降低上传速度。
8. 默认情况下，程序仅上传了 AD7606 通道 1 采集的数据。

9. 串口数据展示推荐使用 SecureCRT，因为数据展示做了特别处理，方便采集数据在串口软件同一个位置不断刷新。

实验内容：

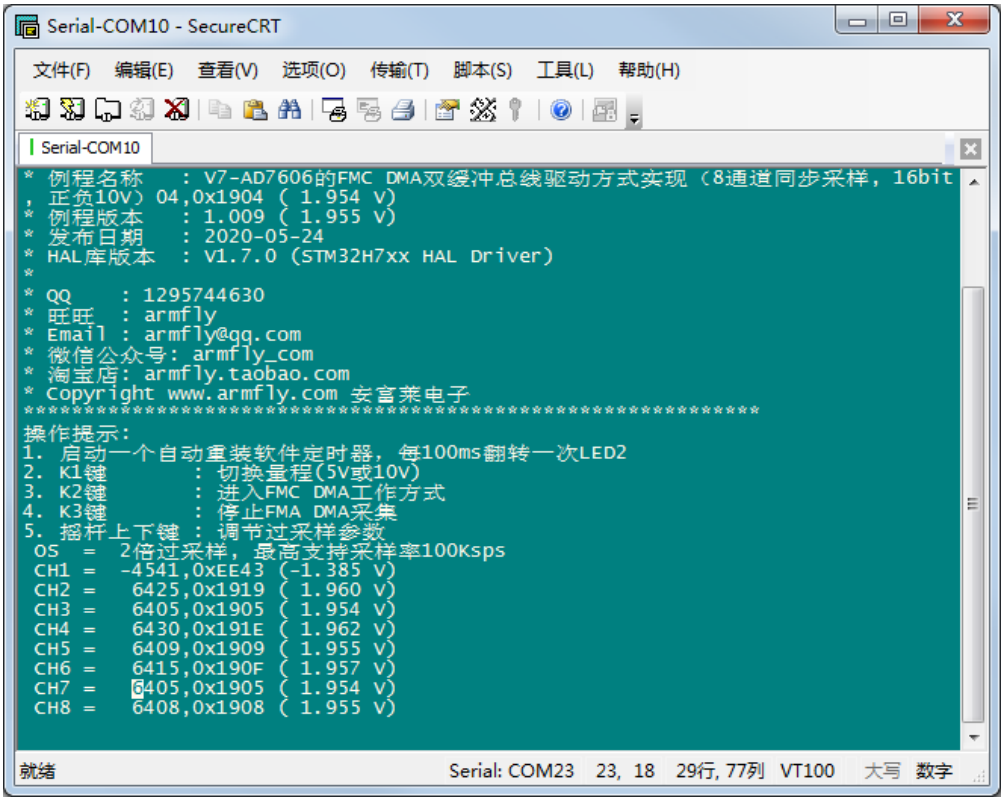
7. AD7606 的 FMC 驱动做了两种采集方式
   （1）软件定时获取方式，适合低速查询获取。
   （2）FIFO 工作模式，适合 8 路实时采集，支持最高采样率 200Ksps。
8. 将模拟输入接地时，采样值是 0 左右。
9. 模拟输入端悬空时，采样值在某个范围浮动（这是正常的，这是 AD7606 内部输入电阻导致的浮动电压）。
10. 出厂的 AD7606 模块缺省是 8080 并行接口。如果用 SPI 接口模式，需要修改 R1 R2 电阻配置。
11. 配置 CVA CVB 引脚为 PWM 输出模式，周期设置为需要的采样频率，之后 MCU 将产生周期非常稳定的 AD 转换信号。

实验操作：

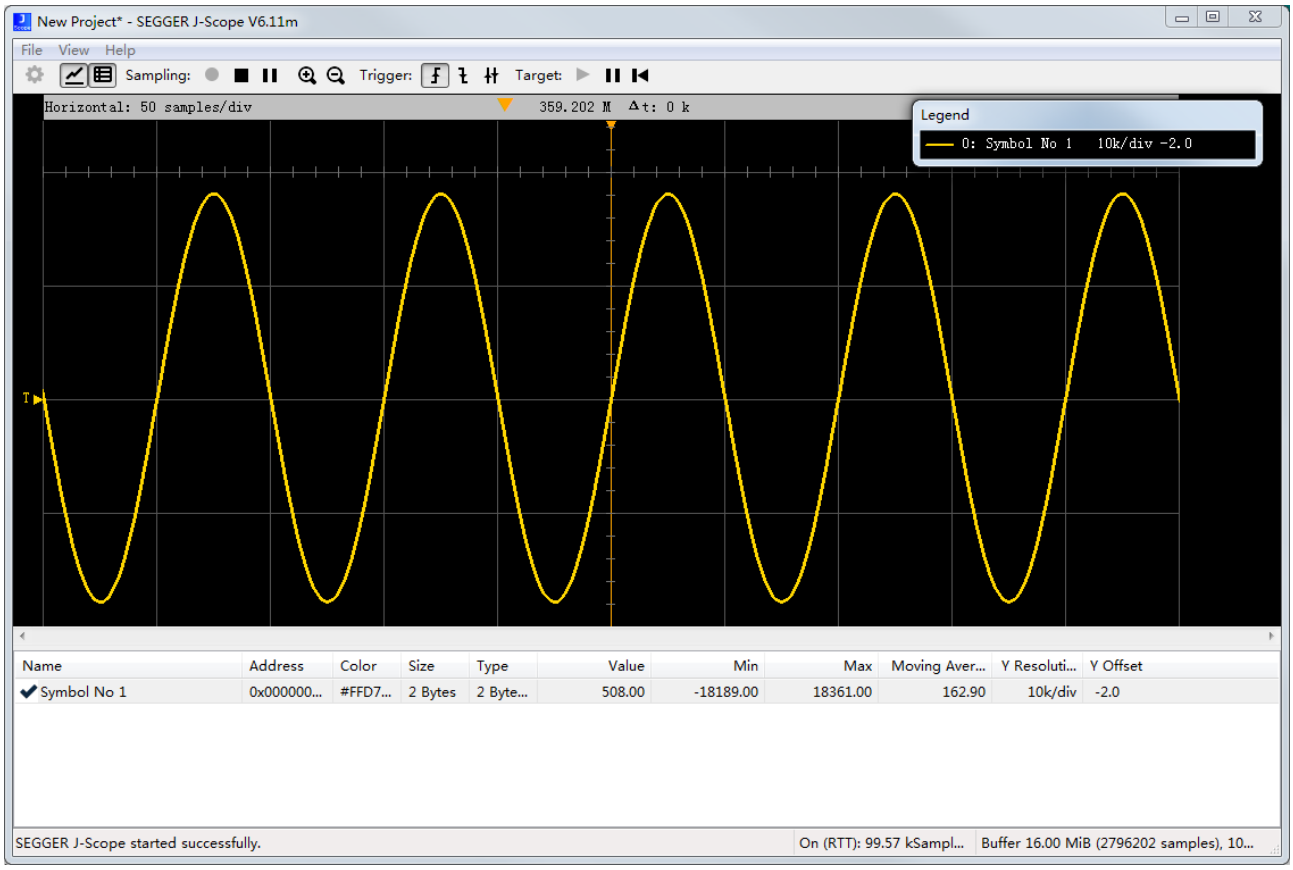
17. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
18. K1 键 ：切换量程(5V 或 10V)。
19. K2 键 ：进入 FIFO 工作模式。
20. K3 键 ：进入软件定时采集模式。
21. 摇杆上下键 ：调节过采样参数。

上电后串口打印的信息：

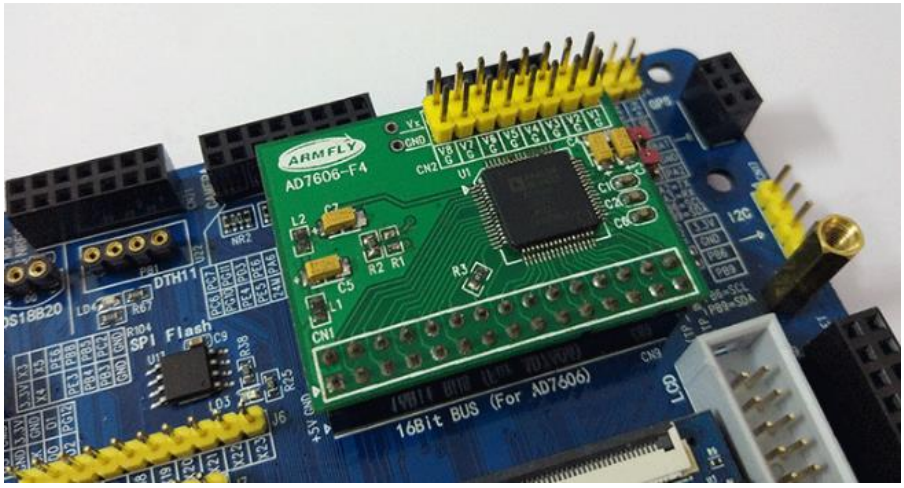
波特率 115200，数据位 8，奇偶校验位无，停止位 1。



J-Scope 波形效果：

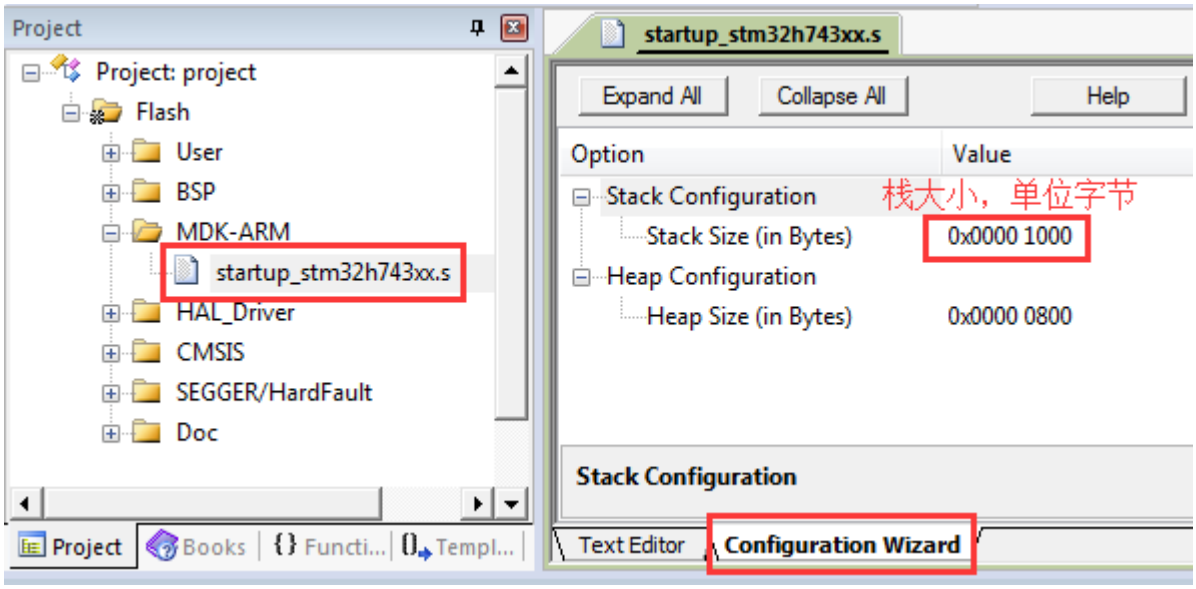


模块插入位置：

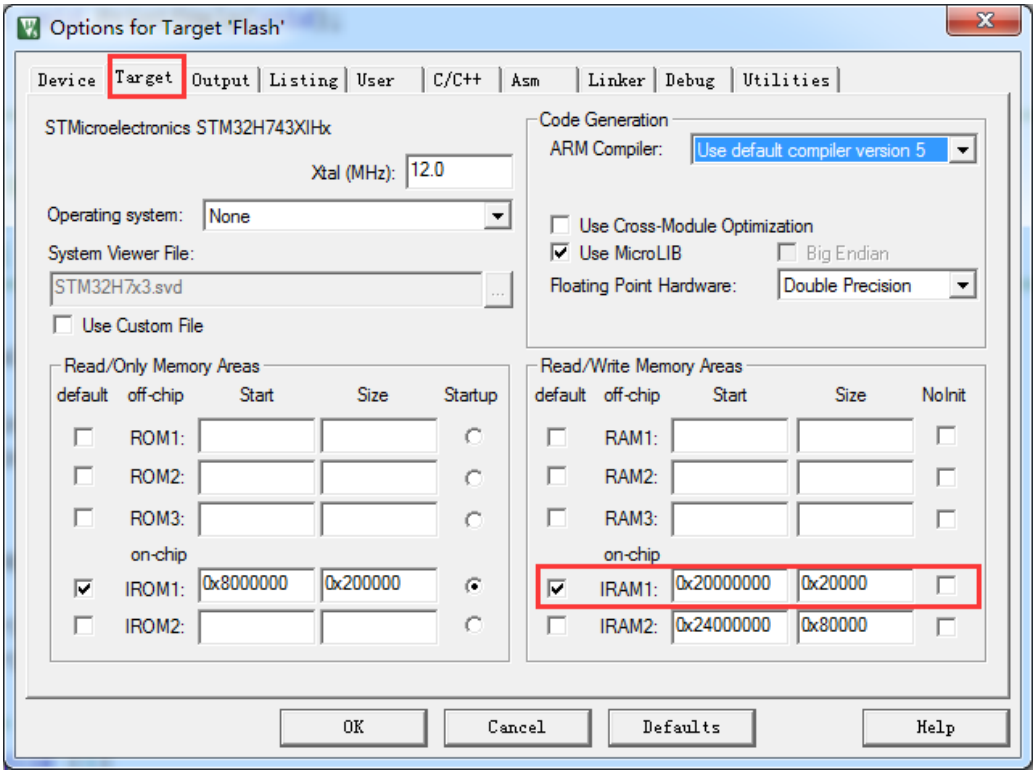


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();      /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();    /* 初始化滴答定时器 */
    bsp_InitLPUart();   /* 初始化串口 */
    bsp_InitExtIO();    /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();      /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitAD7606();  /* 配置 AD7606 所用的 GPIO */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

```
    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **每 10ms 调用一次按键处理：**

按键处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

- **主功能：**

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。

- K1 键 ：切换量程(5V 或 10V)。
- K2 键 ：进入 FIFO 工作模式。
- K3 键 ：进入软件定时采集模式。
- 摇杆上下键 ：调节过采样参数。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */

    DemoFmcAD7606(); /* AD7606 测试 */
}

/*
*********************************************************************************************************
*    函 数 名: DemoFmcAD7606
*    功能说明: AD7606 测试
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoFmcAD7606(void)
{
    uint8_t ucKeyCode;
    uint8_t ucRefresh = 0;


    sfDispMenu();      /* 打印命令提示 */

    ucRefresh = 0;          /* 数据在串口刷新的标志 */

    AD7606_SetOS(AD_OS_NO);      /* 无过采样 */
    AD7606_SetInputRange(1);     /* 0 表示输入量程为正负 5V, 1 表示正负 10V */
    AD7606_StartConvst();        /* 启动 1 次转换 */

    /* 上电默认采样率 */
    g_tAD7606.ucOS = 1;              /* 2 倍过采样 */
    AD7606_StartRecord(100000);      /* 启动 100kHz 采样速率 */
    AD7606_SetOS(g_tAD7606.ucOS);    /* 设置 2 倍过采样 */


    bsp_StartAutoTimer(0, 500); /* 启动 1 个 500ms 的自动重装的定时器 */
    bsp_StartAutoTimer(3, 200); /* 启动 1 个 200ms 的自动重装的定时器 */

    /*
        配置通道 1，上行配置
        默认情况下，J-Scope 仅显示 1 个通道。
```

```
        上传 1 个通道的波形，配置第 2 个参数为 JScope_i2
        上传 2 个通道的波形，配置第 2 个参数为 JScope_i2i2
        上传 3 个通道的波形，配置第 2 个参数为 JScope_i2i2i2
        上传 4 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2
        上传 5 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2
        上传 6 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2i2
        上传 7 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2i2i2
        上传 8 个通道的波形，配置第 2 个参数为 JScope_i2i2i2i2i2i2i2i2
    */
    SEGGER_RTT_ConfigUpBuffer(1, "JScope_i2", buf, 20480, SEGGER_RTT_MODE_NO_BLOCK_SKIP);

    while(1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(3))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(4);
        }

        if (bsp_CheckTimer(0))
        {
            ucRefresh = 1;     /* 刷新显示 */
        }

        if (ucRefresh == 1)
        {
            ucRefresh = 0;

            /* 处理数据 */
            AD7606_Mak();

            /* 打印 ADC 采样结果 */
            AD7606_Disp();
        }

        /* 按键检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。这个函数不会
        等待按键按下，这样我们可以在 while 循环内做其他的事情 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {

            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:                           /* K1 键按下 切换量程 */
                    if (g_tAD7606.ucRange == 0)
                    {
                        AD7606_SetInputRange(1);
                    }
                    else
                    {
                        AD7606_SetInputRange(0);
                    }
                    ucRefresh = 1;
                    break;
```

```
                case KEY_DOWN_K2:                           /* K2 键按下 */
                    g_tAD7606.ucOS = 1;                     /* 2 倍过采样 */
                    AD7606_StartRecord(100000);             /* 启动 100kHz 采样速率 */
                    AD7606_SetOS(g_tAD7606.ucOS);          /* 设置 2 倍过采样 */
                    break;

                case KEY_DOWN_K3:                           /* K3 键按下 */
                    AD7606_StopRecord();                    /* 停止记录 */
                    break;

                case JOY_DOWN_U:                            /* 摇杆 UP 键按下 */
                    if (g_tAD7606.ucOS < 6)
                    {
                        g_tAD7606.ucOS++;
                    }

                    AD7606_SetOS(g_tAD7606.ucOS);

                AD7606_StartRecord(AD7606_SampleFreq[g_tAD7606.ucOS]);/* 启动当前过采样下最高速度 */

                    ucRefresh = 1;
                    break;

                case JOY_DOWN_D:                            /* 摇杆 DOWN 键按下 */
                    if (g_tAD7606.ucOS > 0)
                    {
                        g_tAD7606.ucOS--;
                    }
                    AD7606_SetOS(g_tAD7606.ucOS);
                    ucRefresh = 1;

                AD7606_StartRecord(AD7606_SampleFreq[g_tAD7606.ucOS]);  /* 启动当前过采样下最高速度 */
                    break;

                default:
                    /* 其他的键值不处理 */
                    break;
            }
        }
    }
}
```

77.13 总结

本章节涉及到的知识点非常多，实战性较强，需要大家稍花点精力去研究。

第78章 STM32H7的QSPI总线基础知识和HAL库API

本章节为大家讲解 QSP(Quad Serial peripheral interface）总线的基础知识和对应的 HAL 库 API。

79.1 初学者重要提示

79.2 QSPI 总线基础知识

79.3 QSPI 总线的 HAL 库用法

79.4 源文件 stm32h7xx_hal_qspi.c

79.5 总结

78.1 初学者重要提示

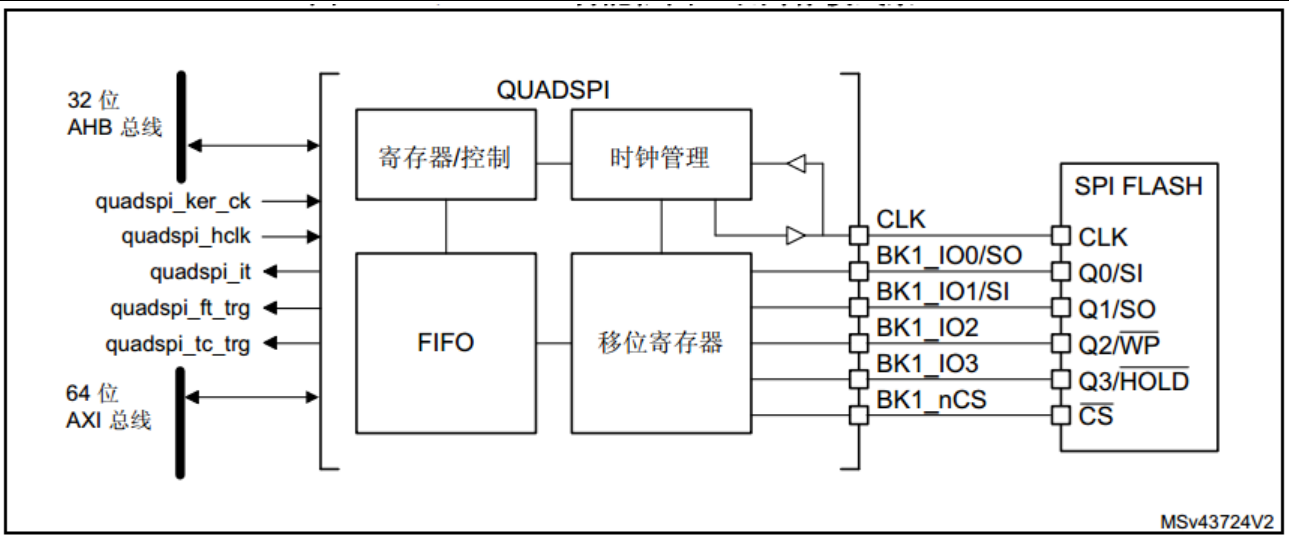
- QSPI 接口，可以做 1 线，2 线或者 4 线使用。
- 注意，QSPI 接口不分主从，QSPI 仅用于主控。
- 可以单独使用 BANK1 外接一个 Flash，也可以单独使用 BANK2 外接一个 Flash，不可以 BANK1 和 BANK2 同时独立使用。但可以两个 BANK 合起来做双 BANK（也称双 Flash，即 dual flash）使用，即 8 线模式。
- STM32H7 可用的单线，双线，四线和八线 SPI QSPI OctaFlash 等涵盖各大厂商，含性能测评：http://www.armbbs.cn/forum.php?mod=viewthread&tid=89669
- QSPI Flash 应用笔记：http://www.armbbs.cn/forum.php?mod=viewthread&tid=99489

78.2 QSPI 总线基础知识

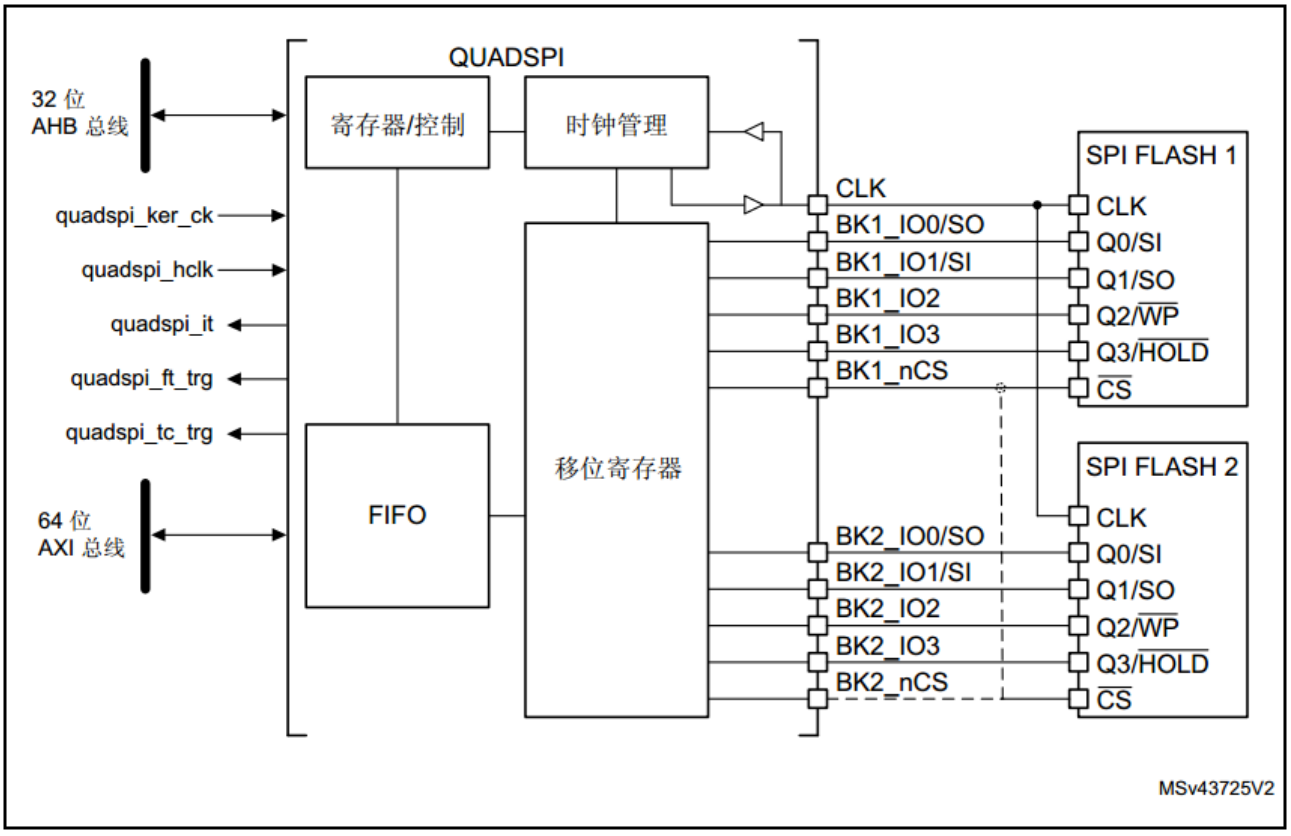
78.2.1 QSPI 总线的硬件框图

认识一个外设，最好的方式就是看它的框图，方便我们快速的了解 QSPI 的基本功能，然后再看手册了解细节。

BANK1 外接 Flash



BANK1 和 BANK2 都外接 Flash：



通过这个框图，我们可以得到如下信息：

- quadspi_ker_ck 时钟输入

  内核时钟。

- quadspi_hclk 时钟输入

  为寄存器提供时钟。

- quadspi_it 输出信号

  QSPI 全局中断。

- quadspi_ft_trg 输出信号

MDMA 的 QSPI FIFO 阀值触发信号

- quadspi_tc_trg 输出信号

  MDMA 的 QSPI 传输完成触发信号

- CLK

  为外接 Flash 提供的时钟。为外接的两块 Flash 同时提供时钟。

- BK1_IO0/SO

  在 2 线或者 4 线模式中作为双向 IO，1 线模式作为单向输出，供 Flash1 使用。

- BK1_IO1/SI

  在 2 线或者 4 线模式中作为双向 IO，1 线模式作为单向输入，供 Flash1 使用。

- BK1_IO2

  在 4 线模式中作为双向 IO，供 Flash1 使用。

- BK1_IO3

  在 4 线模式中作为双向 IO，供 Flash1 使用。

- BK12_IO0/SO

  在 2 线或者 4 线模式中作为双向 IO，1 线模式作为单向输出，供 Flash2 使用。

- BK2_IO1/SI

  在 2 线或者 4 线模式中作为双向 IO，1 线模式作为单向输入，供 Flash2 使用。

- BK2_IO2

  在 4 线模式中作为双向 IO，供 Flash2 使用。

- BK2_IO3

  在 4 线模式中作为双向 IO，供 Flash2 使用。

- BK1_nCS

  片选信号，低电平有效，供 Flash1 使用。如果工作在双 bank 模式下，也可用于 Flash2。

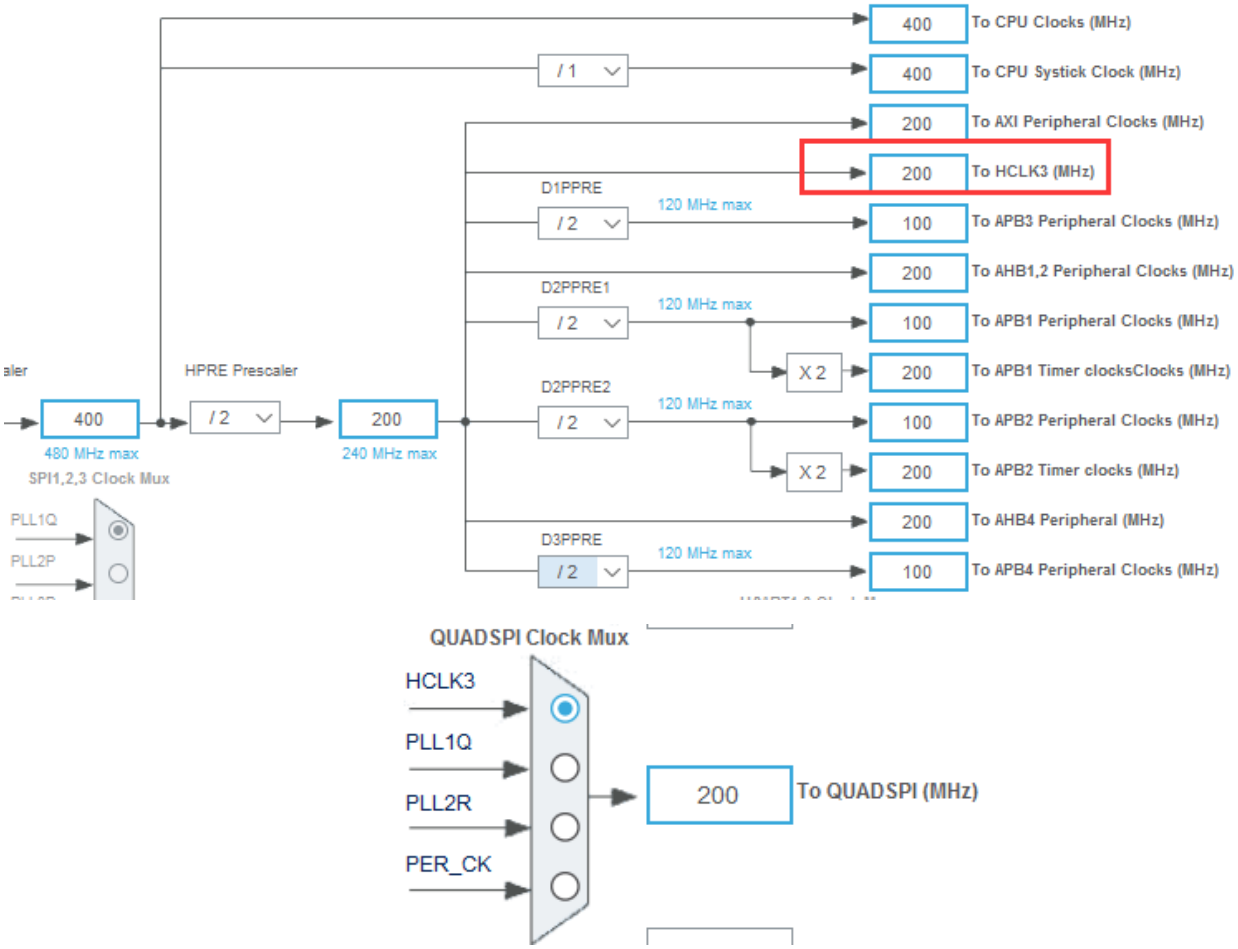
- BK2_nCS

  片选信号，低电平有效，供 Flash1 使用。如果工作在双 bank 模式下，也可用于 Flash1。

78.2.2 QSPI 的时钟源

这个知识点在初学的时候容易忽视，所以我们这里整理下。

STM32H7 主频在 400MHz 下，QSPI 的最高时钟是 200MHz，可以选择的时钟源如下：

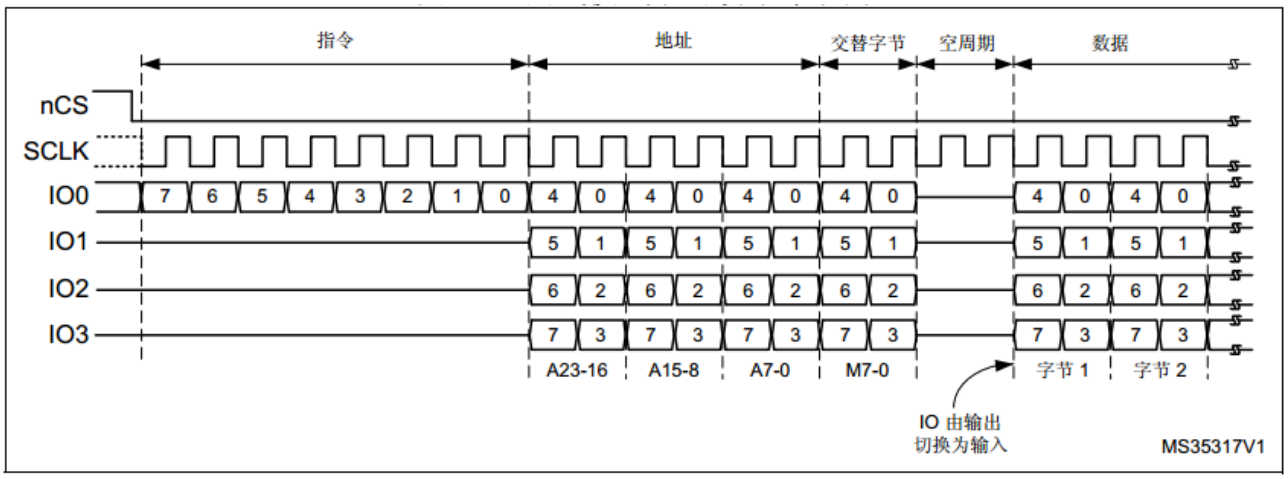


我们的程序中默认是采用的 HCLK3 作为 QSPI 时钟。

（QSPI Flash 应用笔记：http://www.armbbs.cn/forum.php?mod=viewthread&tid=99489 ）

78.2.3 QSPI 命令序列

QSPI 的命令序列主要包括以下几个阶段：指令阶段、地址阶段、交替字节阶段、空指令阶段和数据这五个阶段，任一阶段均可跳过，但至少要包含指令、地址、交替字节或数据阶段之一。



- 指令阶段

  在此阶段，将命令（8 位指令）发送到 Flash，允许发送任何值。用户可以简单地将所需的命令写在指令字段中。根据软件和硬件配置，可以 1 线，2 线或者 4 线方式发送。在某些只发送地址的案例中，指令阶段可以跳过。

- 地址阶段

  在此阶段，将地址发送到 Flash，从指定的地址读取或写入数据。 地址阶段是完全可配置的，允许发送 1、2、3 或 4 个字节的地址。在间接模式和自动轮询模式下，用户可以简单地将所需的地址写入 QUADSPI_AR 寄存器。据软件和硬件配置，可以 1 线，2 线或者 4 线方式方式发送。 在一些不需要地址的情况下，可以跳过地址阶段。

- 交替字节阶段

  QSPI 接口支持的一个额外阶段，具有更大的灵活性。它是通常用于控制操作模式。交替字节阶段是完全可配置的，并允许发送一，二，三或四字节。

- 空周期阶段

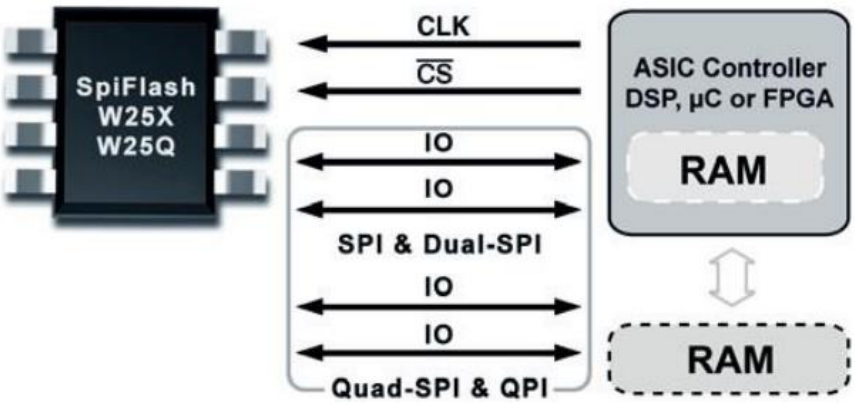
  在高速时钟下运行时，此阶段可以确保有足够的“周转”时间从输出模式切换到输入模式。

- 数据阶段

  这个阶段实现数据的收发。

78.2.4QSPI 的 1 线，2 线，4 线，SDR 和 DDR 模式

这里所说的线是指通信阶段使用的数据线个数。



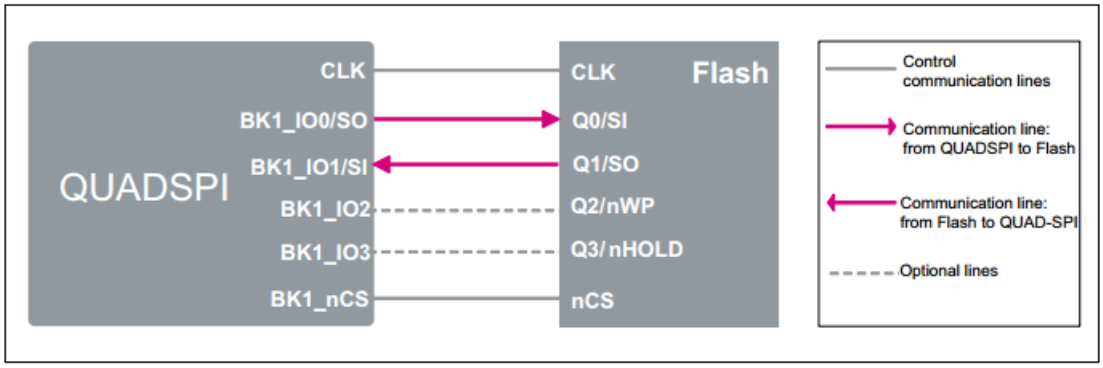
- 1 线（SingleSPI，虽然是一发一收，但属于 1 线方式）

  发送用的数据线 BK1_IO0/SO（BK2_IO0/SO），接收用的数据线 BK1_IO1/SI（BK2_IO1/SI）

  1 线模式下，所有线处于的状态：

  - BK1_IO0 / SO（BK2_IO0 / SO）处于输出模式。
  - BK1_IO1 / SI（BK2_IO1 / SI）处于输入模式（高阻抗）。
  - BK1_IO2（BK2_IO2）处于输出模式并强制置 0。

- BK1_IO3（BK2_IO3）处于输出模式并强制置 1。

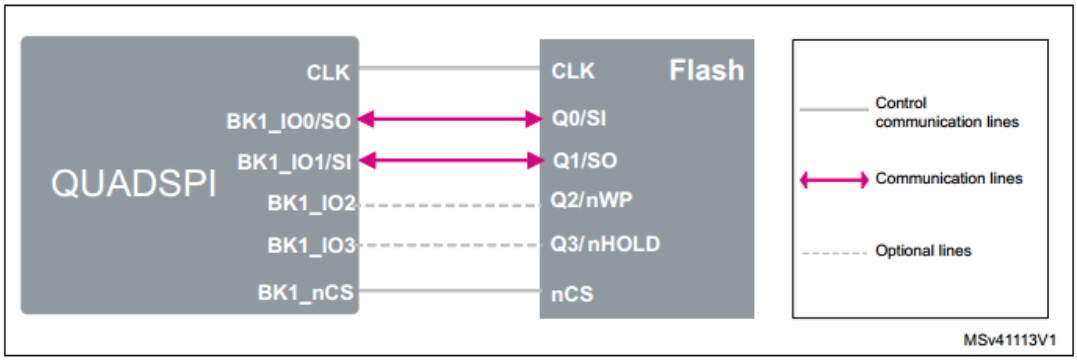


◆ 2 线（Dual-SPI）

同时使用 BK1_IO0（BK2_IO0），BK1_IO1（BK2_IO1）做输入输出。

2 线模式下，所有线处于的状态：

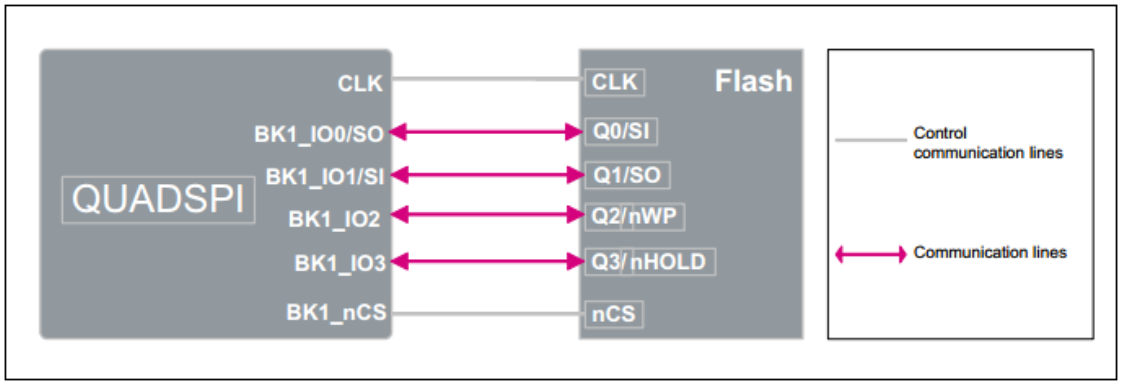
- BK1_IO0 （BK2_IO0 ）和 BK1_IO1（BK2_IO1）读取时处于输入（高阻）。其它情况下为输出。
- BK1_IO2（BK2_IO2）处于输出模式并强制置 0。
- BK1_IO3（BK2_IO3）处于输出模式并强制置 1



◆ 4 线（Quad-SPI）

同时使用 BK1_IO0（BK2_IO0），BK1_IO1（BK2_IO1），BK1_IO2（BK2_IO2），BK1_IO3（BK2_IO3）做输入输出。

4 线模式下，当读取数据时，所有线处于输入（高阻），其它情况作为输出。



◆ SDR

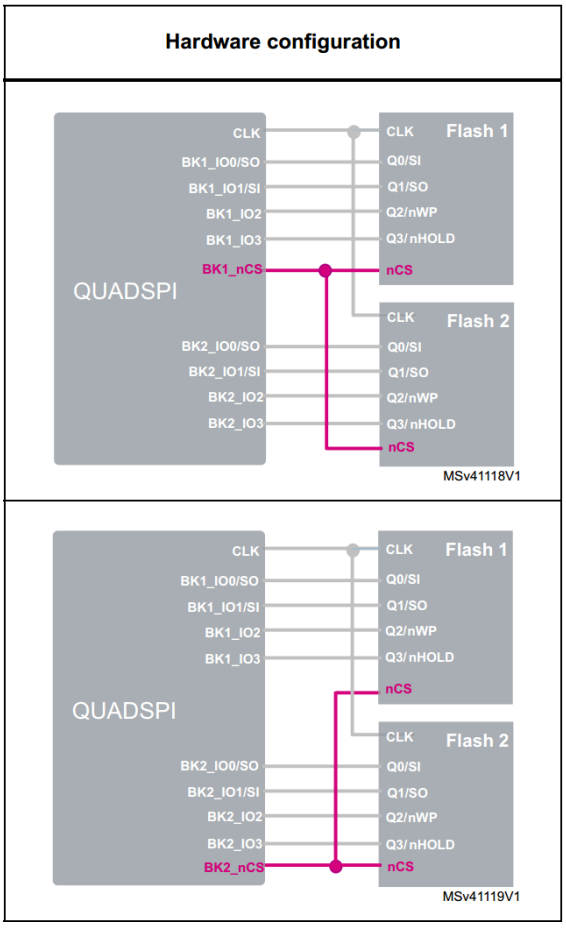
在 SDR 模式下，当 QSPI 驱动 BK1_IO0（BK2_IO0），BK1_IO1（BK2_IO1），BK1_IO2（BK2_IO2），BK1_IO3（BK2_IO3）信号时，这些信号仅在 CLK 的下降沿发生转变。

- DDR

在 SDR 模式下，当 QSPI 驱动 BK1_IO0（BK2_IO0），BK1_IO1（BK2_IO1），BK1_IO2（BK2_IO2），BK1_IO3（BK2_IO3）信号时，这些信号在 CLK 的上升沿和下降沿发生转变。

- 双 BANK（双 Flash）

双闪存就是将 QSPI 的两个 BANK 分别接一个 QSPI Flash，然后时钟公用，片选公用(也可以不公用)，从而实现 8 线模式。



78.2.5 QSPI 间接模式，查询模式和内存映射

QSPI 支持在以下三种模式下工作：

- 间接模式：

这里所谓的间接模式是指寄存器方式访问外设，就跟我们操作串口外设一样。间接模式主要用于以下场合：

- 用于读取，写入，擦除和配置 QSPI Flash。
- 如果不需要 AHB 总线访问 QSPI Flash（在内存映射模式用）。

- CPU 或者 DMA 通过 QSPI 数据寄存器执行所有操作。

在间接模式下，所有操作均通过 QSPI 寄存器执行，含读取和写入操作都由软件管理。 QSPI 接口类似于经典的 SPI 接口。传输的数据通过数据寄存器与 FIFO。在在此模式下，可以从大容量的外部 Flash 读取数据或向外部 Flash 写入数据，可以支持到 4GB 容量。

如果进行擦除或编程操作，则必须使用间接模式，并且所有操作必须由软件处理。在这种情况下，建议使用状态轮询模式，然后轮询闪存内部的状态寄存器以了解何时编程或擦除操作完成。

◆ 状态轮询模式

在以下情况下使用状态轮询模式：

- 读取 QSPI Flash 状态寄存器。
- 在操作结束时自动轮询状态寄存器。

可以自动轮询内存中的指定寄存器并减轻 CPU 负担，例如检查擦除操作何时完成。QSPI 也支持定期查询 QSPI Flash，并且可以屏蔽返回的数据位，将所选位与所需值进行比较，结果比较可以通过下面两种方式处理：

- AND 与操作模式：如果所有选定位都匹配，则产生中断。
- OR 或操作模式：如果所选位之一匹配，则产生中断

发生匹配时，QSPI 可以自动停止。

◆ 内存映射模式

在以下情况下使用内存映射模式：

- 用于阅读操作。
- 像使用内部 Flash 一样使用外部 QSPI Flash， 任何 AHB 总线主控都可以自主读取数据。
- 用于从外部 QSPI Flash 执行代码。

QSPI 接口能够管理多达 256 MB 的内存，在内存映射模式下地址范围是 0x9000 0000 到 0x9FFF FFFF。

78.3 QSPI 总线的 HAL 库用法

78.3.1 QSPI 总线结构体 QUADSPI_TypeDef

QSPI 总线相关的寄存器是通过 HAL 库中的结构体 QUADSPI_TypeDef 定义的，在 stm32h743xx.h 中可以找到这个类型定义：

```
typedef struct
```

```
{
  __IO uint32_t CR;       /*!< QUADSPI Control register,                         Address offset: 0x00 */
  __IO uint32_t DCR;      /*!< QUADSPI Device Configuration register,            Address offset: 0x04 */
  __IO uint32_t SR;       /*!< QUADSPI Status register,                          Address offset: 0x08 */
  __IO uint32_t FCR;      /*!< QUADSPI Flag Clear register,                      Address offset: 0x0C */
  __IO uint32_t DLR;      /*!< QUADSPI Data Length register,                     Address offset: 0x10 */
  __IO uint32_t CCR;      /*!< QUADSPI Communication Configuration register,     Address offset: 0x14 */
  __IO uint32_t AR;       /*!< QUADSPI Address register,                         Address offset: 0x18 */
  __IO uint32_t ABR;      /*!< QUADSPI Alternate Bytes register,                 Address offset: 0x1C */
  __IO uint32_t DR;       /*!< QUADSPI Data register,                            Address offset: 0x20 */
  __IO uint32_t PSMKR;    /*!< QUADSPI Polling Status Mask register,             Address offset: 0x24 */
  __IO uint32_t PSMAR;    /*!< QUADSPI Polling Status Match register,            Address offset: 0x28 */
  __IO uint32_t PIR;      /*!< QUADSPI Polling Interval register,                Address offset: 0x2C */
  __IO uint32_t LPTR;     /*!< QUADSPI Low Power Timeout register,               Address offset: 0x30 */
} QUADSPI_TypeDef;
```

这个结构体的成员名称和排列次序和 CPU 的寄存器是一 一对应的。

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

下面我们看下 QSPI 的定义，在 stm32h743xx.h 文件。

```
#define QSPI_BASE (uint32_t)0x90000000) /*!< Base address of : QSPI memories  accessible over AXI */

#define PERIPH_BASE           (0x40000000UL)
#define D1_AHB1PERIPH_BASE    (PERIPH_BASE + 0x12000000)

#define QSPI_R_BASE           (D1_AHB1PERIPH_BASE + 0x5000)
#define DLYB_QSPI_BASE        (D1_AHB1PERIPH_BASE + 0x6000)

#define QUADSPI               ((QUADSPI_TypeDef *) QSPI_R_BASE) <----- 展开这个宏，(FLASH_TypeDef *)0x52005000
#define DLYB_QUADSPI          ((DLYB_TypeDef *) DLYB_QSPI_BASE)
```

我们访问 QSPI 的 CR 寄存器可以采用这种形式：QUADSPI->CR= 0。

78.3.2 QSPI 总线初始化结构体 QSPI_InitTypeDef

下面是 QSPI 总线的初始化结构体：

```
typedef struct
{
  uint32_t ClockPrescaler;
  uint32_t FifoThreshold;
  uint32_t SampleShifting;
  uint32_t FlashSize;
  uint32_t ChipSelectHighTime;
  uint32_t ClockMode;
  uint32_t FlashID;
  uint32_t DualFlash;
}QSPI_InitTypeDef;
```

下面将结构体成员逐一做个说明：

- ClockPrescaler

设置时钟分频，参数范围 0 到 255。特别注意，这里是针对 HCLK3 作为 QSPI 时钟来说的。

- FifoThreshold

  用于设置 FIFO 阈值，仅用于间接模式，参数范围 1 到 32，单位字节。

- SampleShifting

  QUADSPI 在 FLASH 驱动信号后可以选择再过半个 CLK 周期后才对 FLASH 数据采样，这有利于推迟数据采样。支持的参数如下：

```
#define QSPI_SAMPLE_SHIFTING_NONE      ((uint32_t)0x00000000U)
#define QSPI_SAMPLE_SHIFTING_HALFCYCLE ((uint32_t)QUADSPI_CR_SSHIFT)
```

- FlashSize

  Flash 大小是 2^(FlashSize + 1)，单位字节。

  间接模式下，最大支持的 Flash 大小是 4GB，内存映射模式，最大支持 256MB。

```
#define SPI_POLARITY_LOW       (0x00000000UL)
#define SPI_POLARITY_HIGH      SPI_CFG2_CPOL
```

- ChipSelectHighTime

  命令之间的 CS 片选至少保持的高电平时钟周期 ChipSelectHighTime+1。支持的参数如下：

```
#define QSPI_CS_HIGH_TIME_1_CYCLE      ((uint32_t)0x00000000U)
#define QSPI_CS_HIGH_TIME_2_CYCLE      ((uint32_t)QUADSPI_DCR_CSHT_0)
#define QSPI_CS_HIGH_TIME_3_CYCLE      ((uint32_t)QUADSPI_DCR_CSHT_1)
#define QSPI_CS_HIGH_TIME_4_CYCLE      ((uint32_t)QUADSPI_DCR_CSHT_0 | QUADSPI_DCR_CSHT_1)
#define QSPI_CS_HIGH_TIME_5_CYCLE      ((uint32_t)QUADSPI_DCR_CSHT_2)
#define QSPI_CS_HIGH_TIME_6_CYCLE      ((uint32_t)QUADSPI_DCR_CSHT_2 | QUADSPI_DCR_CSHT_0)
#define QSPI_CS_HIGH_TIME_7_CYCLE      ((uint32_t)QUADSPI_DCR_CSHT_2 | QUADSPI_DCR_CSHT_1)
#define QSPI_CS_HIGH_TIME_8_CYCLE      ((uint32_t)QUADSPI_DCR_CSHT)
```

- FlashID

  用于选择要操作的 BANK，即用 BANK1 还是 BANK2 操作 Flash。

```
#define QSPI_FLASH_ID_1                ((uint32_t)0x00000000)
#define QSPI_FLASH_ID_2                ((uint32_t)QUADSPI_CR_FSEL)
```

- DualFlash

  用于选择是否使用双 BANK。

```
#define QSPI_DUALFLASH_ENABLE          ((uint32_t)QUADSPI_CR_DFM) /*!<Dual-flash mode enabled*/
#define QSPI_DUALFLASH_DISABLE         ((uint32_t)0x00000000)     /*!<Dual-flash mode disabled*/
```

78.3.3 QSPI 总线句柄结构体 QSPI_HandleTypeDef

下面是 QSPI 总线的初始化结构体：

```
typedef struct
{
  QUADSPI_TypeDef           *Instance;        /* QSPI registers base address        */
  QSPI_InitTypeDef          Init;             /* QSPI communication parameters      */
  uint8_t                   *pTxBuffPtr;      /* Pointer to QSPI Tx transfer Buffer */
  __IO uint32_t             TxXferSize;       /* QSPI Tx Transfer size              */
  __IO uint32_t             TxXferCount;      /* QSPI Tx Transfer Counter           */
  uint8_t                   *pRxBuffPtr;      /* Pointer to QSPI Rx transfer Buffer */
  __IO uint32_t             RxXferSize;       /* QSPI Rx Transfer size              */
  __IO uint32_t             RxXferCount;      /* QSPI Rx Transfer Counter           */
  MDMA_HandleTypeDef        *hmdma;           /* QSPI Rx/Tx MDMA Handle parameters  */
  __IO HAL_LockTypeDef      Lock;             /* Locking object                     */
```

```
  __IO HAL_QSPI_StateTypeDef State;           /* QSPI communication state          */
  __IO uint32_t              ErrorCode;       /* QSPI Error code                   */
  uint32_t                   Timeout;         /* Timeout for the QSPI memory access */
}QSPI_HandleTypeDef;
```

下面将结构体成员做个说明：

- QUADSPI_TypeDef *Instance

  这个参数是寄存器的例化，方便操作寄存器，比如使能 QUADSPI。

  SET_BIT(QUADSPI ->CR, QUADSPI_CR_EN)。

- QSPI_InitTypeDef Init

  这个参数是用户接触最多的，在本章节 3.2 小节已经进行了详细说明。

- MDMA_HandleTypeDef *hmdma

  用于 QSPI 句柄关联 MDMA 句柄，方便操作调用。

- 其它参数

  其它参数基本都是在函数内部调用，用户基本不用管。

78.3.4QSPI 命令结构体 QSPI_CommandTypeDef

下面是 QSPI 总线的命名结构体：

```
typedef struct
{
  uint32_t Instruction;
  uint32_t Address;
  uint32_t AlternateBytes;
  uint32_t AddressSize;
  uint32_t AlternateBytesSize;
  uint32_t DummyCycles;
  uint32_t InstructionMode;
  uint32_t AddressMode;
  uint32_t AlternateByteMode;
  uint32_t DataMode;
  uint32_t NbData;
  uint32_t DdrMode;
  uint32_t DdrHoldHalfCycle;
  uint32_t SIOOMode;
}QSPI_CommandTypeDef;
```

下面将结构体成员逐一做个说明：

- Instruction

  设置要发送的指令，参数范围 0x00 到 0xFF。

- Address

  设置要发送的地址，地址由是 1 个字节到 4 个字节来表示，参数范围 0x0 到 0xFFFFFFFF。

- AddressSize

  地址大小，即表示此地址需要的字节数，支持的参数如下：

```
#define QSPI_ADDRESS_8_BITS           ((uint32_t)0x00000000)          /*!<8-bit address*/
#define QSPI_ADDRESS_16_BITS          ((uint32_t)QUADSPI_CCR_ADSIZE_0) /*!<16-bit address*/
```

```
#define QSPI_ADDRESS_24_BITS          ((uint32_t)QUADSPI_CCR_ADSIZE_1) /*!<24-bit address*/
#define QSPI_ADDRESS_32_BITS          ((uint32_t)QUADSPI_CCR_ADSIZE)   /*!<32-bit address*/
```

- AlternateBytesSize

  交替字节大小，支持的参数如下：

```
#define QSPI_ALTERNATE_BYTES_8_BITS    ((uint32_t)0x00000000)           /*!<8-bit alternate bytes*/
#define QSPI_ALTERNATE_BYTES_16_BITS   ((uint32_t)QUADSPI_CCR_ABSIZE_0) /*!<16-bit alternate bytes*/
#define QSPI_ALTERNATE_BYTES_24_BITS   ((uint32_t)QUADSPI_CCR_ABSIZE_1) /*!<24-bit alternate bytes*/
#define QSPI_ALTERNATE_BYTES_32_BITS   ((uint32_t)QUADSPI_CCR_ABSIZE)   /*!<32-bit alternate bytes*/
```

- DummyCycles

  执行空周期个数，参数范围 0 到 31：

- InstructionMode

  指令阶段需要几线模式：

```
#define QSPI_INSTRUCTION_NONE          ((uint32_t)0x00000000)          /*!<No instruction*/
#define QSPI_INSTRUCTION_1_LINE        ((uint32_t)QUADSPI_CCR_IMODE_0) /*!<Instruction on a single line*/
#define QSPI_INSTRUCTION_2_LINES       ((uint32_t)QUADSPI_CCR_IMODE_1) /*!<Instruction on two lines*/
#define QSPI_INSTRUCTION_4_LINES       ((uint32_t)QUADSPI_CCR_IMODE)   /*!<Instruction on four lines*/
```

- AddressMode

  地址阶段需要几线模式：

```
#define QSPI_ADDRESS_NONE              ((uint32_t)0x00000000)           /*!<No address*/
#define QSPI_ADDRESS_1_LINE            ((uint32_t)QUADSPI_CCR_ADMODE_0) /*!<Address on a single line*/
#define QSPI_ADDRESS_2_LINES           ((uint32_t)QUADSPI_CCR_ADMODE_1) /*!<Address on two lines*/
#define QSPI_ADDRESS_4_LINES           ((uint32_t)QUADSPI_CCR_ADMODE)   /*!<Address on four lines*/
```

- AlternateByteMode

  交替字节阶段需要几线模式：

```
#define QSPI_ALTERNATE_BYTES_NONE      ((uint32_t)0x00000000)           /*!<No alternate bytes*/
#define QSPI_ALTERNATE_BYTES_1_LINE    ((uint32_t)QUADSPI_CCR_ABMODE_0) /*!<Alternate bytes on a single line*/
#define QSPI_ALTERNATE_BYTES_2_LINES   ((uint32_t)QUADSPI_CCR_ABMODE_1) /*!<Alternate bytes on two lines*/
#define QSPI_ALTERNATE_BYTES_4_LINES   ((uint32_t)QUADSPI_CCR_ABMODE)   /*!<Alternate bytes on four lines*/
```

- DataMode

  数据阶段需要几线模式：

```
#define QSPI_DATA_NONE                 ((uint32_t)0X00000000)           /*!<No data*/
#define QSPI_DATA_1_LINE               ((uint32_t)QUADSPI_CCR_DMODE_0) /*!<Data on a single line*/
#define QSPI_DATA_2_LINES              ((uint32_t)QUADSPI_CCR_DMODE_1) /*!<Data on two lines*/
#define QSPI_DATA_4_LINES              ((uint32_t)QUADSPI_CCR_DMODE)   /*!<Data on four lines*/
```

- NbData

  要传输的数据大小，参数范围 0 到 0xFFFFFFFF，如果设置为 0 表示不定长，直到存储器末尾。

- DdrMode

  用于设置是否使能 DDR 模式。数据阶段，交替字节阶段和数据传输阶段可以使用 DDR 模式。支持的参数如下：

```
#define QSPI_DDR_MODE_DISABLE          ((uint32_t)0x00000000)       /*!<Double data rate mode disabled*/
#define QSPI_DDR_MODE_ENABLE           ((uint32_t)QUADSPI_CCR_DDRM) /*!<Double data rate mode enabled*/
```

- DdrHoldHalfCycle

  DDR 模式下，用于设置延迟半个时钟周期再做数据输出。

```
#define QSPI_DDR_HHC_ANALOG_DELAY      ((uint32_t)0x00000000)
#define QSPI_DDR_HHC_HALF_CLK_DELAY    ((uint32_t)QUADSPI_CCR_DHHC)
```

- SIOOMode

设置仅发送一次指令还是每次操作都发送指令，支持的参数如下：

```
#define QSPI_SIOO_INST_EVERY_CMD       ((uint32_t)0x00000000)
#define QSPI_SIOO_INST_ONLY_FIRST_CMD  ((uint32_t)QUADSPI_CCR_SIOO)
```

78.3.5 QSPI 自动查询结构体 QSPI_AutoPollingTypeDef

下面是 QSPI 总线自动查询结构体：

```
typedef struct
{
  uint32_t Match;
  uint32_t Mask;
  uint32_t Interval;
  uint32_t StatusBytesSize;
  uint32_t MatchMode;
  uint32_t AutomaticStop;
}QSPI_AutoPollingTypeDef;
```

下面将结构体成员逐一做个说明：

- Match

参数成员 Mask 屏蔽了状态寄存器的某些位后，状态寄存器的值与此参数成员值做匹配。参数范围 0x0 到 0xFFFFFFFF。

- Mask

用于设置屏蔽位，比如 Mask = 0x01，表示仅保留 bit0 的数值，其它 bit 忽略。参数范围 0x0 到 0xFFFFFFFF。

- Interval

指定自动轮询阶段两次读取之间的时钟周期数。参数范围 0 到 0xFFFF。

- StatusBytesSize

用于设置状态寄存器大小，参数范围 1 到 4 个字节。

- MatchMode

参数成员 Mask 屏蔽了状态寄存器的某些位后，状态寄存器完全与参数成员 Match 一样（与操作的含义）或者任意一个 bit 的值与参数成员 Match 中一个 bit 的值一样（或操作的含义），比如 Mask = 0x01，Match=0x00，MatchMode=与操作，表示不断查询状态寄存器 bit0，等待其为 0。

MatchMode 支持的参数成员如下：

```
#define QSPI_MATCH_MODE_AND            ((uint32_t)0x00000000)     /*!<AND match mode between unmasked bits*/
#define QSPI_MATCH_MODE_OR             ((uint32_t)QUADSPI_CR_PMM) /*!<OR match mode between unmasked bits*/
```

- AutomaticStop

当与参数成员 Match 匹配时，自动停止检测。

78.3.6 QSPI 内存映射结构体 QSPI_MemoryMappedTypeDef

下面是 QSPI 总线的内存映射结构体：

```
typedef struct
{
  uint32_t TimeOutPeriod;
  uint32_t TimeOutActivation;
}QSPI_MemoryMappedTypeDef;
```

下面将结构体成员逐一做个说明：

- TimeOutPeriod

  FIFO 满时，释放芯片选择之前要等待的时钟周期数。参数范围 0 到 0xFFFF。

- TimeOutActivation

  指定是否启用超时计数器以释放芯片选择，支持的参数成员如下：

```
#define QSPI_TIMEOUT_COUNTER_DISABLE    ((uint32_t)0x00000000)
#define QSPI_TIMEOUT_COUNTER_ENABLE     ((uint32_t)QUADSPI_CR_TCEN)
```

78.4 QSPI 总线源文件 stm32h7xx_hal_qspi.c

此文件涉及到的函数较多，这里把几个常用的函数做个说明：

- HAL_QSPI_Init
- HAL_QSPI_DeInit
- HAL_QSPI_Command
- HAL_QSPI_Command_IT
- HAL_QSPI_AutoPolling
- HAL_QSPI_AutoPolling_IT
- HAL_QSPI_Transmit
- HAL_QSPI_Receive
- HAL_QSPI_Transmit_DMA
- HAL_QSPI_Receive_DMA
- HAL_QSPI_MemoryMapped

78.4.1 函数 HAL_QSPI_Init

函数原型：

```
HAL_StatusTypeDef HAL_QSPI_Init(QSPI_HandleTypeDef *hqspi)
{
  HAL_StatusTypeDef status;
  uint32_t tickstart = HAL_GetTick();

  /* 检测句柄是否有效 */
```

```
  if(hqspi == NULL)
  {
    return HAL_ERROR;
  }

  /* 检查参数是否有效 */
  assert_param(IS_QSPI_ALL_INSTANCE(hqspi->Instance));
  assert_param(IS_QSPI_CLOCK_PRESCALER(hqspi->Init.ClockPrescaler));
  assert_param(IS_QSPI_FIFO_THRESHOLD(hqspi->Init.FifoThreshold));
  assert_param(IS_QSPI_SSHIFT(hqspi->Init.SampleShifting));
  assert_param(IS_QSPI_FLASH_SIZE(hqspi->Init.FlashSize));
  assert_param(IS_QSPI_CS_HIGH_TIME(hqspi->Init.ChipSelectHighTime));
  assert_param(IS_QSPI_CLOCK_MODE(hqspi->Init.ClockMode));
  assert_param(IS_QSPI_DUAL_FLASH_MODE(hqspi->Init.DualFlash));

  if (hqspi->Init.DualFlash != QSPI_DUALFLASH_ENABLE )
  {
    assert_param(IS_QSPI_FLASH_ID(hqspi->Init.FlashID));
  }

  if(hqspi->State == HAL_QSPI_STATE_RESET)
  {

#if (USE_HAL_QSPI_REGISTER_CALLBACKS == 1)
    /* 复位状态，设置默认的回调 */
    hqspi->ErrorCallback         = HAL_QSPI_ErrorCallback;
    hqspi->AbortCpltCallback     = HAL_QSPI_AbortCpltCallback;
    hqspi->FifoThresholdCallback = HAL_QSPI_FifoThresholdCallback;
    hqspi->CmdCpltCallback       = HAL_QSPI_CmdCpltCallback;
    hqspi->RxCpltCallback        = HAL_QSPI_RxCpltCallback;
    hqspi->TxCpltCallback        = HAL_QSPI_TxCpltCallback;
    hqspi->StatusMatchCallback   = HAL_QSPI_StatusMatchCallback;
    hqspi->TimeOutCallback       = HAL_QSPI_TimeOutCallback;

    if(hqspi->MspInitCallback == NULL)
    {
      hqspi->MspInitCallback = HAL_QSPI_MspInit;
    }

    /* 初始化底层硬件 */
    hqspi->MspInitCallback(hqspi);
#else
    /* 初始化: GPIO, CLOCK */
    HAL_QSPI_MspInit(hqspi);
#endif

    /* 配置 QSPI 内存访问默认的溢出时间 */
    HAL_QSPI_SetTimeout(hqspi, HAL_QSPI_TIMEOUT_DEFAULT_VALUE);
  }

  /* 配置 QSPI FIFO 阈值 */
  MODIFY_REG(hqspi->Instance->CR, QUADSPI_CR_FTHRES,
             ((hqspi->Init.FifoThreshold - 1U) << QUADSPI_CR_FTHRES_Pos));

  /* 等 BUSY 标志复位 */
  status = QSPI_WaitFlagStateUntilTimeout(hqspi, QSPI_FLAG_BUSY, RESET, tickstart, hqspi->Timeout);

  if(status == HAL_OK)
```

```
  {
    /* 配置 QSPI 时钟分频和采样延迟 */
    MODIFY_REG(hqspi->Instance->CR, (QUADSPI_CR_PRESCALER | QUADSPI_CR_SSHIFT | QUADSPI_CR_FSEL |
                QUADSPI_CR_DFM),
               ((hqspi->Init.ClockPrescaler << QUADSPI_CR_PRESCALER_Pos) |
                hqspi->Init.SampleShifting  | hqspi->Init.FlashID | hqspi->Init.DualFlash));

    /* 配置 QSPI Flash 大小，CS 片选高电平时间和时钟模式 */
    MODIFY_REG(hqspi->Instance->DCR, (QUADSPI_DCR_FSIZE | QUADSPI_DCR_CSHT | QUADSPI_DCR_CKMODE),
               ((hqspi->Init.FlashSize << QUADSPI_DCR_FSIZE_Pos) |
                hqspi->Init.ChipSelectHighTime | hqspi->Init.ClockMode));

    /* 时钟 QSPI 外设 */
    __HAL_QSPI_ENABLE(hqspi);

    /* 设置 QSPI 无错误代码 */
    hqspi->ErrorCode = HAL_QSPI_ERROR_NONE;

    /* 初始化 QSPI 状态就绪 */
    hqspi->State = HAL_QSPI_STATE_READY;
  }

  /* 返回状态信息 */
  return status;
}
```

函数描述：

此函数用于初始化 QSPI。

函数参数：

- 第 1 个参数是 QSPI_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数，详见本章 3.3 小节。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

注意事项：

3. 函数 HAL_QSPI_MspInit 用于初始化 QSPI 的底层时钟、引脚等功能。需要用户自己在此函数里面实现具体的功能。由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
4. 如果形参 hqspi 的结构体成员 State 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 QSPI_HandleTypeDef SpiHandle。

   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_QSPI_STATE_RESET = 0x00U。

   解决办法有三

   方法 1：用户自己初始化 QSPI 和涉及到的 GPIO 等。

   方法 2：定义 QSPI_HandleTypeDef QspiHandle 为全局变量。

   方法 3：下面的方法

```
if(HAL_QSPI_DeInit(&QspiHandle) != HAL_OK)
```

```
{
    Error_Handler();
}
if(HAL_QSPI_Init(&QspiHandle) != HAL_OK)
{
    Error_Handler();
}
```

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitQSPI_W25Q256
*    功能说明: QSPI Flash 硬件初始化，配置基本参数
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitQSPI_W25Q256(void)
{
    /* 复位 QSPI */
    QSPIHandle.Instance = QUADSPI;
    if (HAL_QSPI_DeInit(&QSPIHandle) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 设置时钟速度，QSPI clock = 200MHz / (ClockPrescaler+1) = 100MHz */
    QSPIHandle.Init.ClockPrescaler  = 1;

    /* 设置 FIFO 阀值，范围 1 - 32 */
    QSPIHandle.Init.FifoThreshold   = 32;

    /*
        QUADSPI 在 FLASH 驱动信号后过半个 CLK 周期才对 FLASH 驱动的数据采样。
        在外部信号延迟时，这有利于推迟数据采样。
    */
    QSPIHandle.Init.SampleShifting  = QSPI_SAMPLE_SHIFTING_HALFCYCLE;

    /*Flash 大小是 2^(FlashSize + 1) = 2^25 = 32MB */
    //QSPI_FLASH_SIZE - 1; 需要扩大一倍，否则内存映射方位最后 1 个地址时，会异常。
    QSPIHandle.Init.FlashSize       = QSPI_FLASH_SIZE;

    /* 命令之间的 CS 片选至少保持 2 个时钟周期的高电平 */
    QSPIHandle.Init.ChipSelectHighTime = QSPI_CS_HIGH_TIME_2_CYCLE;

    /*
       MODE0: 表示片选信号空闲期间，CLK 时钟信号是低电平
       MODE3: 表示片选信号空闲期间，CLK 时钟信号是高电平
    */
    QSPIHandle.Init.ClockMode = QSPI_CLOCK_MODE_0;

    /* QSPI 有两个 BANK，这里使用的 BANK1 */
    QSPIHandle.Init.FlashID   = QSPI_FLASH_ID_1;

    /* V7 开发板仅使用了 BANK1，这里是禁止双 BANK */
    QSPIHandle.Init.DualFlash = QSPI_DUALFLASH_DISABLE;

    /* 初始化配置 QSPI */
```

```
    if (HAL_QSPI_Init(&QSPIHandle) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

78.4.2 函数 HAL_QSPI_DeInit

函数原型：

```
HAL_StatusTypeDef HAL_QSPI_DeInit(QSPI_HandleTypeDef *hqspi)
{
  /* 检测 QSPI 句柄 */
  if(hqspi == NULL)
  {
    return HAL_ERROR;
  }

  /* 禁止 QSPI 外设时钟 */
  __HAL_QSPI_DISABLE(hqspi);

#if (USE_HAL_QSPI_REGISTER_CALLBACKS == 1)
  if(hqspi->MspDeInitCallback == NULL)
  {
    hqspi->MspDeInitCallback = HAL_QSPI_MspDeInit;
  }

  /* 复位硬件底层 */
  hqspi->MspDeInitCallback(hqspi);
#else
  /* 复位: GPIO, CLOCK, NVIC... */
  HAL_QSPI_MspDeInit(hqspi);
#endif

  /* 设置无错误 Set QSPI error code to none */
  hqspi->ErrorCode = HAL_QSPI_ERROR_NONE;

  /* 设置 QSPI 状态为复位 */
  hqspi->State = HAL_QSPI_STATE_RESET;

  return HAL_OK;
}
```

函数描述：

用于复位 QSPI 总线初始化。

函数参数：

- 第 1 个参数是 QSPI_HandleTypeDef 类型结构体指针变量，详见本章 3.3 小节。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

78.4.3函数 HAL_QSPI_Command

函数原型：

```
HAL_StatusTypeDef HAL_QSPI_Command(QSPI_HandleTypeDef *hqspi, QSPI_CommandTypeDef *cmd, uint32_t Timeout)
{
  HAL_StatusTypeDef status;
  uint32_t tickstart = HAL_GetTick();

  /* 检测参数 */
  assert_param(IS_QSPI_INSTRUCTION_MODE(cmd->InstructionMode));
  if (cmd->InstructionMode != QSPI_INSTRUCTION_NONE)
  {
    assert_param(IS_QSPI_INSTRUCTION(cmd->Instruction));
  }

  assert_param(IS_QSPI_ADDRESS_MODE(cmd->AddressMode));
  if (cmd->AddressMode != QSPI_ADDRESS_NONE)
  {
    assert_param(IS_QSPI_ADDRESS_SIZE(cmd->AddressSize));
  }

  assert_param(IS_QSPI_ALTERNATE_BYTES_MODE(cmd->AlternateByteMode));
  if (cmd->AlternateByteMode != QSPI_ALTERNATE_BYTES_NONE)
  {
    assert_param(IS_QSPI_ALTERNATE_BYTES_SIZE(cmd->AlternateBytesSize));
  }

  assert_param(IS_QSPI_DUMMY_CYCLES(cmd->DummyCycles));
  assert_param(IS_QSPI_DATA_MODE(cmd->DataMode));

  assert_param(IS_QSPI_DDR_MODE(cmd->DdrMode));
  assert_param(IS_QSPI_DDR_HHC(cmd->DdrHoldHalfCycle));
  assert_param(IS_QSPI_SIOO_MODE(cmd->SIOOMode));

  /* 上锁 */
  __HAL_LOCK(hqspi);

  if(hqspi->State == HAL_QSPI_STATE_READY)
  {
    hqspi->ErrorCode = HAL_QSPI_ERROR_NONE;

    /* 更新 QSPI 状态 */
    hqspi->State = HAL_QSPI_STATE_BUSY;

    /* 等待 BUSY 标志复位 */
    status = QSPI_WaitFlagStateUntilTimeout(hqspi, QSPI_FLAG_BUSY, RESET, tickstart, Timeout);

    if (status == HAL_OK)
    {
      /* 配置 QSPI  */
      QSPI_Config(hqspi, cmd, QSPI_FUNCTIONAL_MODE_INDIRECT_WRITE);

      if (cmd->DataMode == QSPI_DATA_NONE)
      {
        /* 没有数据阶段时，配置完成后立即开始传输，所以请等到 TC 标志设置并返回到空闲状态 */
```

```
        status = QSPI_WaitFlagStateUntilTimeout(hqspi, QSPI_FLAG_TC, SET, tickstart, Timeout);

        if (status == HAL_OK)
        {
          __HAL_QSPI_CLEAR_FLAG(hqspi, QSPI_FLAG_TC);

          /* QSPI 就绪 */
          hqspi->State = HAL_QSPI_STATE_READY;
        }
      }
      else
      {
        /* QSPI 就绪 */
        hqspi->State = HAL_QSPI_STATE_READY;
      }
    }
  }
  else
  {
    status = HAL_BUSY;
  }

  /* 解锁 */
  __HAL_UNLOCK(hqspi);

  /* 返回函数状态 */
  return status;
}
```

函数描述：

此函数主要用于为 QSPI Flash 发送操作命令，查询方式。

函数参数：

- 第 1 个参数是 QSPI_HandleTypeDef 类型结构体指针变量，详见本章 3.3 小节。
- 第 2 个参数是 QSPI_CommandTypeDef 类型结构体变量，详见本章 3.4 小节。
- 第 3 个参数是溢出时间，单位 HAL 库时间基准，一般我们设置的是 1ms。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: QSPI_WriteEnable
*    功能说明: 写使能
*    形    参: hqspi  QSPI_HandleTypeDef 句柄。
*    返 回 值: 无
*********************************************************************************************************
*/
static void QSPI_WriteEnable(QSPI_HandleTypeDef *hqspi)
{
    QSPI_CommandTypeDef     sCommand = {0};

    /* 基本配置 */
    sCommand.InstructionMode   = QSPI_INSTRUCTION_1_LINE;     /* 1 线方式发送指令 */
    sCommand.AddressSize       = QSPI_ADDRESS_32_BITS;        /* 32 位地址 */
```

```
    sCommand.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;  /* 无交替字节 */
    sCommand.DdrMode           = QSPI_DDR_MODE_DISABLE;      /* W25Q256JV 不支持 DDR */
    sCommand.DdrHoldHalfCycle  = QSPI_DDR_HHC_ANALOG_DELAY;  /* DDR 模式，数据输出延迟 */
    sCommand.SIOOMode          = QSPI_SIOO_INST_EVERY_CMD;   /* 每次传输都发指令 */

    /* 写使能 */
    sCommand.Instruction       = WRITE_ENABLE_CMD;  /* 写使能指令 */
    sCommand.AddressMode       = QSPI_ADDRESS_NONE; /* 无需地址 */
    sCommand.DataMode          = QSPI_DATA_NONE;    /* 无需数据 */
    sCommand.DummyCycles       = 0;                 /* 空周期  */

    if (HAL_QSPI_Command(&QSPIHandle, &sCommand, HAL_QPSI_TIMEOUT_DEFAULT_VALUE) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

78.4.4函数 HAL_QSPI_Command_IT

函数原型：

```
HAL_StatusTypeDef HAL_QSPI_Command_IT(QSPI_HandleTypeDef *hqspi, QSPI_CommandTypeDef *cmd)
{
  HAL_StatusTypeDef status;
  uint32_t tickstart = HAL_GetTick();

  /* 检测参数 */
  assert_param(IS_QSPI_INSTRUCTION_MODE(cmd->InstructionMode));
  if (cmd->InstructionMode != QSPI_INSTRUCTION_NONE)
  {
    assert_param(IS_QSPI_INSTRUCTION(cmd->Instruction));
  }

  assert_param(IS_QSPI_ADDRESS_MODE(cmd->AddressMode));
  if (cmd->AddressMode != QSPI_ADDRESS_NONE)
  {
    assert_param(IS_QSPI_ADDRESS_SIZE(cmd->AddressSize));
  }

  assert_param(IS_QSPI_ALTERNATE_BYTES_MODE(cmd->AlternateByteMode));
  if (cmd->AlternateByteMode != QSPI_ALTERNATE_BYTES_NONE)
  {
    assert_param(IS_QSPI_ALTERNATE_BYTES_SIZE(cmd->AlternateBytesSize));
  }

  assert_param(IS_QSPI_DUMMY_CYCLES(cmd->DummyCycles));
  assert_param(IS_QSPI_DATA_MODE(cmd->DataMode));

  assert_param(IS_QSPI_DDR_MODE(cmd->DdrMode));
  assert_param(IS_QSPI_DDR_HHC(cmd->DdrHoldHalfCycle));
  assert_param(IS_QSPI_SIOO_MODE(cmd->SIOOMode));

  /* 上锁 */
  __HAL_LOCK(hqspi);

  if(hqspi->State == HAL_QSPI_STATE_READY)
```

```
{
  hqspi->ErrorCode = HAL_QSPI_ERROR_NONE;

  /* QSPI 忙 */
  hqspi->State = HAL_QSPI_STATE_BUSY;

  /* 等待 BUSY 标志复位 */
  status = QSPI_WaitFlagStateUntilTimeout(hqspi, QSPI_FLAG_BUSY, RESET, tickstart, hqspi->Timeout);

  if (status == HAL_OK)
  {
    if (cmd->DataMode == QSPI_DATA_NONE)
    {
      /* 清除中断 */
      __HAL_QSPI_CLEAR_FLAG(hqspi, QSPI_FLAG_TE | QSPI_FLAG_TC);
    }

    /* 调用所有配置函数 */
    QSPI_Config(hqspi, cmd, QSPI_FUNCTIONAL_MODE_INDIRECT_WRITE);

    if (cmd->DataMode == QSPI_DATA_NONE)
    {
      /* 无数据阶段，配置完毕后，立即开始传输，所以机会 TC 和 TE 中断 */
      /* 解锁 /
      __HAL_UNLOCK(hqspi);

      /* 使能 TE（Transfer Error）和 TC 中断 */
      __HAL_QSPI_ENABLE_IT(hqspi, QSPI_IT_TE | QSPI_IT_TC);
    }
    else
    {
      /* 更正 QSPI 状态 Update QSPI state */
      hqspi->State = HAL_QSPI_STATE_READY;

      /* 上锁 */
      __HAL_UNLOCK(hqspi);
    }
  }
  else
  {
    /* 解锁 */
    __HAL_UNLOCK(hqspi);
  }
}
else
{
  status = HAL_BUSY;

  /* 解锁 */
  __HAL_UNLOCK(hqspi);
}

/* 返回状态 */
return status;
}
```

函数描述：

此函数主要用于为 QSPI Flash 发送操作命令，中断方式。

函数参数：

- 第 1 个参数是 QSPI_HandleTypeDef 类型结构体指针变量，详见本章 3.3 小节。
- 第 2 个参数是 QSPI_CommandTypeDef 类型结构体变量，详见本章 3.4 小节。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: QSPI_WriteEnable
*    功能说明: 写使能
*    形    参: hqspi  QSPI_HandleTypeDef 句柄。
*    返 回 值: 无
*********************************************************************************************************
*/
static void QSPI_WriteEnable(QSPI_HandleTypeDef *hqspi)
{
    QSPI_CommandTypeDef     sCommand = {0};

    /* 基本配置 */
    sCommand.InstructionMode   = QSPI_INSTRUCTION_1_LINE;    /* 1 线方式发送指令 */
    sCommand.AddressSize       = QSPI_ADDRESS_32_BITS;       /* 32 位地址 */
    sCommand.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;  /* 无交替字节 */
    sCommand.DdrMode           = QSPI_DDR_MODE_DISABLE;      /* W25Q256JV 不支持 DDR */
    sCommand.DdrHoldHalfCycle  = QSPI_DDR_HHC_ANALOG_DELAY;  /* DDR 模式，数据输出延迟 */
    sCommand.SIOOMode          = QSPI_SIOO_INST_EVERY_CMD;   /* 每次传输都发指令 */

    /* 写使能 */
    sCommand.Instruction       = WRITE_ENABLE_CMD;  /* 写使能指令 */
    sCommand.AddressMode       = QSPI_ADDRESS_NONE; /* 无需地址 */
    sCommand.DataMode          = QSPI_DATA_NONE;    /* 无需数据 */
    sCommand.DummyCycles       = 0;                 /* 空周期  */

    if (HAL_QSPI_Command_IT(&QSPIHandle, &sCommand) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

78.4.5 函数 HAL_QSPI_AutoPolling

函数原型：

```
HAL_StatusTypeDef HAL_QSPI_AutoPolling(QSPI_HandleTypeDef *hqspi, QSPI_CommandTypeDef *cmd,
QSPI_AutoPollingTypeDef *cfg, uint32_t Timeout)
{
  HAL_StatusTypeDef status;
  uint32_t tickstart = HAL_GetTick();

    /* 检查参数 */
  assert_param(IS_QSPI_INSTRUCTION_MODE(cmd->InstructionMode));
```

```
  if (cmd->InstructionMode != QSPI_INSTRUCTION_NONE)
  {
    assert_param(IS_QSPI_INSTRUCTION(cmd->Instruction));
  }

  assert_param(IS_QSPI_ADDRESS_MODE(cmd->AddressMode));
  if (cmd->AddressMode != QSPI_ADDRESS_NONE)
  {
    assert_param(IS_QSPI_ADDRESS_SIZE(cmd->AddressSize));
  }

  assert_param(IS_QSPI_ALTERNATE_BYTES_MODE(cmd->AlternateByteMode));
  if (cmd->AlternateByteMode != QSPI_ALTERNATE_BYTES_NONE)
  {
    assert_param(IS_QSPI_ALTERNATE_BYTES_SIZE(cmd->AlternateBytesSize));
  }

  assert_param(IS_QSPI_DUMMY_CYCLES(cmd->DummyCycles));
  assert_param(IS_QSPI_DATA_MODE(cmd->DataMode));

  assert_param(IS_QSPI_DDR_MODE(cmd->DdrMode));
  assert_param(IS_QSPI_DDR_HHC(cmd->DdrHoldHalfCycle));
  assert_param(IS_QSPI_SIOO_MODE(cmd->SIOOMode));

  assert_param(IS_QSPI_INTERVAL(cfg->Interval));
  assert_param(IS_QSPI_STATUS_BYTES_SIZE(cfg->StatusBytesSize));
  assert_param(IS_QSPI_MATCH_MODE(cfg->MatchMode));

    /* 上锁 */
  __HAL_LOCK(hqspi);

  if(hqspi->State == HAL_QSPI_STATE_READY)
  {
    hqspi->ErrorCode = HAL_QSPI_ERROR_NONE;

      /* 更新状态 */
    hqspi->State = HAL_QSPI_STATE_BUSY_AUTO_POLLING;

      /* 等待 BUSY 复位标志 */
    status = QSPI_WaitFlagStateUntilTimeout(hqspi, QSPI_FLAG_BUSY, RESET, tickstart, Timeout);

    if (status == HAL_OK)
    {
        /* 配置 QSPI 匹配位 */
      WRITE_REG(hqspi->Instance->PSMAR, cfg->Match);

        /* 配置 QSPI 屏蔽位 */
      WRITE_REG(hqspi->Instance->PSMKR, cfg->Mask);

        /* 配置查询时间间隔 */
      WRITE_REG(hqspi->Instance->PIR, cfg->Interval);

        /* 配置匹配模式，使能自动停(否则阻塞方式无限等待) */
      MODIFY_REG(hqspi->Instance->CR, (QUADSPI_CR_PMM | QUADSPI_CR_APMS),
                 (cfg->MatchMode | QSPI_AUTOMATIC_STOP_ENABLE));

        /* 调用配置函数 */
      cmd->NbData = cfg->StatusBytesSize;
```

```
      QSPI_Config(hqspi, cmd, QSPI_FUNCTIONAL_MODE_AUTO_POLLING);

        /* 等待 SM 标志 */
      status = QSPI_WaitFlagStateUntilTimeout(hqspi, QSPI_FLAG_SM, SET, tickstart, Timeout);

      if (status == HAL_OK)
      {
        __HAL_QSPI_CLEAR_FLAG(hqspi, QSPI_FLAG_SM);

          /* 更新状态 */
        hqspi->State = HAL_QSPI_STATE_READY;
      }
    }
  }
  else
  {
    status = HAL_BUSY;
  }

  /* 解锁 */
  __HAL_UNLOCK(hqspi);

    /* 返回状态 */
  return status;
}
```

函数描述：

用于状态标志查询，函数采用的查询方式。

函数参数：

- 第 1 个参数是 QSPI_HandleTypeDef 类型结构体指针变量，详见本章 3.3 小节。
- 第 2 个参数是 QSPI_CommandTypeDef 类型结构体变量，详见本章 3.4 小节。
- 第 3 个参数是 QSPI_AutoPollingTypeDef 类型结构体变量，详见本章 3.5 小节。
- 第 4 个参数是溢出时间，单位 HAL 库时间基准，单位 1ms。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: QSPI_AutoPollingMemReady
*    功能说明: 等待 QSPI Flash 就绪，主要用于 Flash 擦除和页编程时使用
*    形    参: hqspi  QSPI_HandleTypeDef 句柄
*    返 回 值: 无
*********************************************************************************************************
*/
static void QSPI_AutoPollingMemReady(QSPI_HandleTypeDef *hqspi)
{
    QSPI_CommandTypeDef     sCommand = {0};
    QSPI_AutoPollingTypeDef sConfig = {0};


    /* 基本配置 */
    sCommand.InstructionMode   = QSPI_INSTRUCTION_1_LINE;    /* 1 线方式发送指令 */
```

```
    sCommand.AddressSize       = QSPI_ADDRESS_32_BITS;       /* 32 位地址 */
    sCommand.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;  /* 无交替字节 */
    sCommand.DdrMode           = QSPI_DDR_MODE_DISABLE;      /* W25Q256JV 不支持 DDR */
    sCommand.DdrHoldHalfCycle  = QSPI_DDR_HHC_ANALOG_DELAY;  /* DDR 模式，数据输出延迟 */
    sCommand.SIOOMode          = QSPI_SIOO_INST_EVERY_CMD;   /* 每次传输都发指令 */

    /* 读取状态*/
    sCommand.Instruction       = READ_STATUS_REG_CMD; /* 读取状态命令 */
    sCommand.AddressMode       = QSPI_ADDRESS_NONE;   /* 无需地址 */
    sCommand.DataMode          = QSPI_DATA_1_LINE;    /* 1 线数据 */
    sCommand.DummyCycles       = 0;                   /* 无需空周期 */

    /* 屏蔽位设置的 bit0，匹配位等待 bit0 为 0，即不断查询状态寄存器 bit0，等待其为 0 */
    sConfig.Mask            = 0x01;
    sConfig.Match           = 0x00;
    sConfig.MatchMode       = QSPI_MATCH_MODE_AND;
    sConfig.StatusBytesSize = 1;
    sConfig.Interval        = 0x10;
    sConfig.AutomaticStop   = QSPI_AUTOMATIC_STOP_ENABLE;

    if (HAL_QSPI_AutoPolling(&QSPIHandle, &sCommand, &sConfig, 10000) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

78.4.6函数 HAL_QSPI_AutoPolling_IT

函数原型：

```
HAL_StatusTypeDef HAL_QSPI_AutoPolling_IT(QSPI_HandleTypeDef *hqspi, QSPI_CommandTypeDef *cmd,
QSPI_AutoPollingTypeDef *cfg)
{
  HAL_StatusTypeDef status;
  uint32_t tickstart = HAL_GetTick();

    /* 检查参数 */
  assert_param(IS_QSPI_INSTRUCTION_MODE(cmd->InstructionMode));
  if (cmd->InstructionMode != QSPI_INSTRUCTION_NONE)
  {
    assert_param(IS_QSPI_INSTRUCTION(cmd->Instruction));
  }

  assert_param(IS_QSPI_ADDRESS_MODE(cmd->AddressMode));
  if (cmd->AddressMode != QSPI_ADDRESS_NONE)
  {
    assert_param(IS_QSPI_ADDRESS_SIZE(cmd->AddressSize));
  }

  assert_param(IS_QSPI_ALTERNATE_BYTES_MODE(cmd->AlternateByteMode));
  if (cmd->AlternateByteMode != QSPI_ALTERNATE_BYTES_NONE)
  {
    assert_param(IS_QSPI_ALTERNATE_BYTES_SIZE(cmd->AlternateBytesSize));
  }

  assert_param(IS_QSPI_DUMMY_CYCLES(cmd->DummyCycles));
```

```
  assert_param(IS_QSPI_DATA_MODE(cmd->DataMode));

  assert_param(IS_QSPI_DDR_MODE(cmd->DdrMode));
  assert_param(IS_QSPI_DDR_HHC(cmd->DdrHoldHalfCycle));
  assert_param(IS_QSPI_SIOO_MODE(cmd->SIOOMode));

  assert_param(IS_QSPI_INTERVAL(cfg->Interval));
  assert_param(IS_QSPI_STATUS_BYTES_SIZE(cfg->StatusBytesSize));
  assert_param(IS_QSPI_MATCH_MODE(cfg->MatchMode));
  assert_param(IS_QSPI_AUTOMATIC_STOP(cfg->AutomaticStop));

    /* 上锁 */
  __HAL_LOCK(hqspi);

  if(hqspi->State == HAL_QSPI_STATE_READY)
  {
    hqspi->ErrorCode = HAL_QSPI_ERROR_NONE;

      /* 更新状态 */
    hqspi->State = HAL_QSPI_STATE_BUSY_AUTO_POLLING;

      /* 等 BUSY 标志复位 */
    status = QSPI_WaitFlagStateUntilTimeout(hqspi, QSPI_FLAG_BUSY, RESET, tickstart, hqspi->Timeout);

    if (status == HAL_OK)
    {
        /* 配置匹配值 */
      WRITE_REG(hqspi->Instance->PSMAR, cfg->Match);

        /* 配置屏蔽值 */
      WRITE_REG(hqspi->Instance->PSMKR, cfg->Mask);

        /* 配置查询间隔 */
      WRITE_REG(hqspi->Instance->PIR, cfg->Interval);

        /* 配置匹配模式和自动停止位 */
      MODIFY_REG(hqspi->Instance->CR, (QUADSPI_CR_PMM | QUADSPI_CR_APMS),
                 (cfg->MatchMode | cfg->AutomaticStop));

        /* 清标志 */
      __HAL_QSPI_CLEAR_FLAG(hqspi, QSPI_FLAG_TE | QSPI_FLAG_SM);

        /* 调用配置函数 */
      cmd->NbData = cfg->StatusBytesSize;
      QSPI_Config(hqspi, cmd, QSPI_FUNCTIONAL_MODE_AUTO_POLLING);

        /* 解锁 */
      __HAL_UNLOCK(hqspi);

      /* 使能 SM（status match）和 TE（Transfer Error）标志 */
      __HAL_QSPI_ENABLE_IT(hqspi, (QSPI_IT_SM | QSPI_IT_TE));

    }
    else
    {
        /* 解锁 */
      __HAL_UNLOCK(hqspi);
    }
```

```
  }
  else
  {
    status = HAL_BUSY;

      /* 解锁 */
    __HAL_UNLOCK(hqspi);
  }

    /* 返回函数状态 */
  return status;
}
```

函数描述：

用于状态标志查询，函数采用的中断方式。

函数参数：

- 第 1 个参数是 QSPI_HandleTypeDef 类型结构体指针变量，详见本章 3.3 小节。
- 第 2 个参数是 QSPI_CommandTypeDef 类型结构体变量，详见本章 3.4 小节。
- 第 3 个参数是 QSPI_AutoPollingTypeDef 类型结构体变量，详见本章 3.5 小节。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: QSPI_AutoPollingMemReady
*    功能说明: 等待 QSPI Flash 就绪，主要用于 Flash 擦除和页编程时使用
*    形    参: hqspi  QSPI_HandleTypeDef 句柄
*    返 回 值: 无
*********************************************************************************************************
*/
static void QSPI_AutoPollingMemReady(QSPI_HandleTypeDef *hqspi)
{
    QSPI_CommandTypeDef     sCommand = {0};
    QSPI_AutoPollingTypeDef sConfig = {0};


    /* 基本配置 */
    sCommand.InstructionMode   = QSPI_INSTRUCTION_1_LINE;    /* 1 线方式发送指令 */
    sCommand.AddressSize       = QSPI_ADDRESS_32_BITS;       /* 32 位地址 */
    sCommand.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;  /* 无交替字节 */
    sCommand.DdrMode           = QSPI_DDR_MODE_DISABLE;      /* W25Q256JV 不支持 DDR */
    sCommand.DdrHoldHalfCycle  = QSPI_DDR_HHC_ANALOG_DELAY;  /* DDR 模式，数据输出延迟 */
    sCommand.SIOOMode          = QSPI_SIOO_INST_EVERY_CMD;   /* 每次传输都发指令 */

    /* 读取状态*/
    sCommand.Instruction       = READ_STATUS_REG_CMD; /* 读取状态命令 */
    sCommand.AddressMode       = QSPI_ADDRESS_NONE;   /* 无需地址 */
    sCommand.DataMode          = QSPI_DATA_1_LINE;    /* 1 线数据 */
    sCommand.DummyCycles       = 0;                   /* 无需空周期 */

    /* 屏蔽位设置的 bit0，匹配位等待 bit0 为 0，即不断查询状态寄存器 bit0，等待其为 0 */
    sConfig.Mask           = 0x01;
    sConfig.Match          = 0x00;
```

```
    sConfig.MatchMode       = QSPI_MATCH_MODE_AND;
    sConfig.StatusBytesSize = 1;
    sConfig.Interval        = 0x10;
    sConfig.AutomaticStop   = QSPI_AUTOMATIC_STOP_ENABLE;

    if (HAL_QSPI_AutoPolling_IT(&QSPIHandle, &sCommand, &sConfig) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

78.4.7 函数 HAL_QSPI_Transmit

函数原型：

```
HAL_StatusTypeDef HAL_QSPI_Transmit(QSPI_HandleTypeDef *hqspi, uint8_t *pData, uint32_t Timeout)
{
  HAL_StatusTypeDef status = HAL_OK;
  uint32_t tickstart = HAL_GetTick();
  __IO uint32_t *data_reg = &hqspi->Instance->DR;

    /* 上锁 */
  __HAL_LOCK(hqspi);

  if(hqspi->State == HAL_QSPI_STATE_READY)
  {
    hqspi->ErrorCode = HAL_QSPI_ERROR_NONE;

    if(pData != NULL )
    {
        /* 更新状态 */
      hqspi->State = HAL_QSPI_STATE_BUSY_INDIRECT_TX;

        /* 配置发送 */
      hqspi->TxXferCount = READ_REG(hqspi->Instance->DLR) + 1U;
      hqspi->TxXferSize = READ_REG(hqspi->Instance->DLR) + 1U;
      hqspi->pTxBuffPtr = pData;

        /* 配置 QSPI，间接模式 */
      MODIFY_REG(hqspi->Instance->CCR, QUADSPI_CCR_FMODE, QSPI_FUNCTIONAL_MODE_INDIRECT_WRITE);

      while(hqspi->TxXferCount > 0U)
      {
          /* 等待 FT 标志 */
        status = QSPI_WaitFlagStateUntilTimeout(hqspi, QSPI_FLAG_FT, SET, tickstart, Timeout);

        if (status != HAL_OK)
        {
          break;
        }

        *((__IO uint8_t *)data_reg) = *hqspi->pTxBuffPtr;
        hqspi->pTxBuffPtr++;
        hqspi->TxXferCount--;
      }
```

```
      if (status == HAL_OK)
      {
          /* 等待 TC 标志 */
        status = QSPI_WaitFlagStateUntilTimeout(hqspi, QSPI_FLAG_TC, SET, tickstart, Timeout);

        if (status == HAL_OK)
        {
            /* 清楚传输完成 */
          __HAL_QSPI_CLEAR_FLAG(hqspi, QSPI_FLAG_TC);

        }
      }

        /* 更新 */
      hqspi->State = HAL_QSPI_STATE_READY;
    }
    else
    {
      hqspi->ErrorCode |= HAL_QSPI_ERROR_INVALID_PARAM;
      status = HAL_ERROR;
    }
  }
  else
  {
    status = HAL_BUSY;
  }

    /* 解锁 */
  __HAL_UNLOCK(hqspi);

  return status;
}
```

函数描述：

此函数用于 QSPI 接口数据发送，函数采用查询方式。

函数参数：

- 第 1 个参数是 QSPI_HandleTypeDef 类型结构体指针变量，详见本章 3.3 小节。
- 第 2 个参数是要发送的数据地址。
- 第 3 个参数是溢出时间，单位 HAL 库时间基准，我们一般设置的是 1ms。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: QSPI_WriteBuffer
*    功能说明: 页编程，页大小 256 字节，任意页都可以写入
*    形    参: _pBuf : 数据源缓冲区;
*              _uiWriteAddr : 目标区域首地址，即页首地址，比如 0， 256，512 等。
*              _usWriteSize : 数据个数，不能超过页面大小，范围 1 - 256。
*    返 回 值: 1:成功， 0：失败
*********************************************************************************************************
*/
```

```
uint8_t QSPI_WriteBuffer(uint8_t *_pBuf, uint32_t _uiWriteAddr, uint16_t _usWriteSize)
{
    QSPI_CommandTypeDef sCommand={0};

    /* 写使能 */
    QSPI_WriteEnable(&QSPIHandle);

    /* 基本配置 */
    sCommand.InstructionMode   = QSPI_INSTRUCTION_1_LINE;    /* 1 线方式发送指令 */
    sCommand.AddressSize       = QSPI_ADDRESS_32_BITS;       /* 32 位地址 */
    sCommand.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;  /* 无交替字节 */
    sCommand.DdrMode           = QSPI_DDR_MODE_DISABLE;      /* W25Q256JV 不支持 DDR */
    sCommand.DdrHoldHalfCycle  = QSPI_DDR_HHC_ANALOG_DELAY;  /* DDR 模式，数据输出延迟 */
    sCommand.SIOOMode          = QSPI_SIOO_INST_ONLY_FIRST_CMD;   /* 仅发送一次命令 */

    /* 写序列配置 */
    sCommand.Instruction = QUAD_IN_FAST_PROG_4_BYTE_ADDR_CMD; /* 32bit 地址的 4 线快速写入命令 */
    sCommand.DummyCycles = 0;                       /* 不需要空周期 */
    sCommand.AddressMode = QSPI_ADDRESS_1_LINE;  /* 4 线地址方式 */
    sCommand.DataMode    = QSPI_DATA_4_LINES;    /* 4 线数据方式 */
    sCommand.NbData      = _usWriteSize;         /* 写数据大小 */
    sCommand.Address     = _uiWriteAddr;         /* 写入地址 */

    if (HAL_QSPI_Command(&QSPIHandle, &sCommand, 10000) != HAL_OK)
    {
       //return 0;
        Error_Handler(__FILE__, __LINE__);
    }

    /* 启动传输 */
    if (HAL_QSPI_Transmit(&QSPIHandle, _pBuf, 10000) != HAL_OK)
    {
       //return 0;
        Error_Handler(__FILE__, __LINE__);

    }

    QSPI_AutoPollingMemReady(&QSPIHandle);

    return 1;
}
```

78.4.8函数 HAL_QSPI_Receive

函数原型：

```
HAL_StatusTypeDef HAL_QSPI_Receive(QSPI_HandleTypeDef *hqspi, uint8_t *pData, uint32_t Timeout)
{
  HAL_StatusTypeDef status = HAL_OK;
  uint32_t tickstart = HAL_GetTick();
  uint32_t addr_reg = READ_REG(hqspi->Instance->AR);
  __IO uint32_t *data_reg = &hqspi->Instance->DR;

    /* 上锁 */
  __HAL_LOCK(hqspi);
```

```
if(hqspi->State == HAL_QSPI_STATE_READY)
{
  hqspi->ErrorCode = HAL_QSPI_ERROR_NONE;

  if(pData != NULL )
  {
      /* 更新状态 */
    hqspi->State = HAL_QSPI_STATE_BUSY_INDIRECT_RX;

      /* 配置接收 */
    hqspi->RxXferCount = READ_REG(hqspi->Instance->DLR) + 1U;
    hqspi->RxXferSize = READ_REG(hqspi->Instance->DLR) + 1U;
    hqspi->pRxBuffPtr = pData;

      /* 配置 QSPI，间接模式 */
    MODIFY_REG(hqspi->Instance->CCR, QUADSPI_CCR_FMODE, QSPI_FUNCTIONAL_MODE_INDIRECT_READ);

      /* 启动传输 */
    WRITE_REG(hqspi->Instance->AR, addr_reg);

    while(hqspi->RxXferCount > 0U)
    {
        /* 等 FT 和 TC 标志 */
      status = QSPI_WaitFlagStateUntilTimeout(hqspi, (QSPI_FLAG_FT | QSPI_FLAG_TC), SET, tickstart, Timeout);

      if  (status != HAL_OK)
      {
        break;
      }

      *hqspi->pRxBuffPtr = *((__IO uint8_t *)data_reg);
      hqspi->pRxBuffPtr++;
      hqspi->RxXferCount--;
    }

    if (status == HAL_OK)
    {
        /* 等待 TC 标志 */
      status = QSPI_WaitFlagStateUntilTimeout(hqspi, QSPI_FLAG_TC, SET, tickstart, Timeout);

      if  (status == HAL_OK)
      {
          /* 清楚传输完成 bit */
        __HAL_QSPI_CLEAR_FLAG(hqspi, QSPI_FLAG_TC);

      }
    }

      /* 更新 QSPI 状态 */
    hqspi->State = HAL_QSPI_STATE_READY;
  }
  else
  {
    hqspi->ErrorCode |= HAL_QSPI_ERROR_INVALID_PARAM;
    status = HAL_ERROR;
  }
}
else
```

```
  {
    status = HAL_BUSY;
  }

    /* 解锁 */
  __HAL_UNLOCK(hqspi);

  return status;
}
```

函数描述：

此函数用于 QSPI 接口数据接收，函数采用查询方式。

函数参数：

- 第 1 个参数是 QSPI_HandleTypeDef 类型结构体指针变量，详见本章 3.3 小节。
- 第 2 个参数是要接收的数据地址。
- 第 3 个参数是溢出时间，单位 HAL 库时间基准，我们一般设置的是 1ms。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: QSPI_ReadBuffer
*    功能说明: 连续读取若干字节，字节个数不能超出芯片容量。
*    形    参: _pBuf : 数据源缓冲区。
*              _uiReadAddr : 起始地址。
*              _usSize : 数据个数, 可以大于 PAGE_SIZE, 但是不能超出芯片总容量。
*    返 回 值: 无
*********************************************************************************************************
*/
void QSPI_ReadBuffer(uint8_t * _pBuf, uint32_t _uiReadAddr, uint32_t _uiSize)
{

    QSPI_CommandTypeDef sCommand = {0};


    /* 基本配置 */
    sCommand.InstructionMode   = QSPI_INSTRUCTION_1_LINE;        /* 1 线方式发送指令 */
    sCommand.AddressSize       = QSPI_ADDRESS_32_BITS;           /* 32 位地址 */
    sCommand.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;      /* 无交替字节 */
    sCommand.DdrMode           = QSPI_DDR_MODE_DISABLE;          /* W25Q256JV 不支持 DDR */
    sCommand.DdrHoldHalfCycle  = QSPI_DDR_HHC_ANALOG_DELAY;      /* DDR 模式，数据输出延迟 */
    sCommand.SIOOMode          = QSPI_SIOO_INST_EVERY_CMD;       /* 每次传输要发指令 */

    /* 读取数据 */
    sCommand.Instruction = QUAD_INOUT_FAST_READ_4_BYTE_ADDR_CMD; /* 32bit 地址的 4 线快速读取命令 */
    sCommand.DummyCycles = 6;                     /* 空周期 */
    sCommand.AddressMode = QSPI_ADDRESS_4_LINES; /* 4 线地址 */
    sCommand.DataMode    = QSPI_DATA_4_LINES;    /* 4 线数据 */
    sCommand.NbData      = _uiSize;              /* 读取的数据大小 */
    sCommand.Address     = _uiReadAddr;          /* 读取数据的起始地址 */

    if (HAL_QSPI_Command(&QSPIHandle, &sCommand, 10000) != HAL_OK)
```

```
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 读取 */
    if (HAL_QSPI_Receive(&QSPIHandle, _pBuf, 10000) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

78.4.9函数 HAL_QSPI_Transmit_DMA

函数原型：

```
HAL_StatusTypeDef HAL_QSPI_Transmit_DMA(QSPI_HandleTypeDef *hqspi, uint8_t *pData)
{
  HAL_StatusTypeDef status = HAL_OK;
  uint32_t data_size = (READ_REG(hqspi->Instance->DLR) + 1U);

    /* 上锁 */
  __HAL_LOCK(hqspi);

  if(hqspi->State == HAL_QSPI_STATE_READY)
  {
      /* 无错误 */
    hqspi->ErrorCode = HAL_QSPI_ERROR_NONE;

    if(pData != NULL )
    {
        /* 配置发送字节数 */
      hqspi->TxXferCount = data_size;

          /* 设置 QSPI 状态 */
        hqspi->State = HAL_QSPI_STATE_BUSY_INDIRECT_TX;

          /* 清除中断标志 */
        __HAL_QSPI_CLEAR_FLAG(hqspi, (QSPI_FLAG_TE | QSPI_FLAG_TC));

          /* 配置发送 */
        hqspi->TxXferSize = hqspi->TxXferCount;
        hqspi->pTxBuffPtr = pData;

          /* 配置间接写模式 */
        MODIFY_REG(hqspi->Instance->CCR, QUADSPI_CCR_FMODE, QSPI_FUNCTIONAL_MODE_INDIRECT_WRITE);

          /* 设置 MDMA 发送完回调 */
        hqspi->hmdma->XferCpltCallback = QSPI_DMATxCplt;

          /* 设置 MDMA 错误回调 */
        hqspi->hmdma->XferErrorCallback = QSPI_DMAError;

          /* 设置 MDMA 终止传输回调为 NULL */
        hqspi->hmdma->XferAbortCallback = NULL;

          /* MDMA 的目的地址是 QSPI DR 寄存器，设置禁止递增 */
```

```
      MODIFY_REG(hqspi->hmdma->Instance->CTCR, (MDMA_CTCR_DINC | MDMA_CTCR_DINCOS),MDMA_DEST_INC_DISABLE);

        /* 更新 MDMA 源地址配置 */
      if (hqspi->hmdma->Init.SourceDataSize == MDMA_SRC_DATASIZE_BYTE)
      {
        MODIFY_REG(hqspi->hmdma->Instance->CTCR, (MDMA_CTCR_SINC | MDMA_CTCR_SINCOS) , MDMA_SRC_INC_BYTE);
      }
      else if (hqspi->hmdma->Init.SourceDataSize == MDMA_SRC_DATASIZE_HALFWORD)
      {
        MODIFY_REG(hqspi->hmdma->Instance->CTCR, (MDMA_CTCR_SINC | MDMA_CTCR_SINCOS) ,
             MDMA_SRC_INC_HALFWORD);
      }
      else if (hqspi->hmdma->Init.SourceDataSize == MDMA_SRC_DATASIZE_WORD)
      {
        MODIFY_REG(hqspi->hmdma->Instance->CTCR, (MDMA_CTCR_SINC | MDMA_CTCR_SINCOS) , MDMA_SRC_INC_WORD);
      }
      else
      {
          /* 配置错误 */
        hqspi->ErrorCode |= HAL_QSPI_ERROR_DMA;
        status = HAL_ERROR;
      }

        /* 使能 QSPI MDMA 发送 */
      if (HAL_MDMA_Start_IT(hqspi->hmdma, (uint32_t)pData, (uint32_t)&hqspi->Instance->DR,
                            hqspi->TxXferSize, 1) == HAL_OK)
      {
          /* 解锁 */
        __HAL_UNLOCK(hqspi);

          /* 使能 QSPI 传输错误中断 */
        __HAL_QSPI_ENABLE_IT(hqspi, QSPI_IT_TE);

          /* 使能 MDMA 传输 */
        SET_BIT(hqspi->Instance->CR, QUADSPI_CR_DMAEN);
      }
      else
      {
        status = HAL_ERROR;
        hqspi->ErrorCode |= HAL_QSPI_ERROR_DMA;
        hqspi->State = HAL_QSPI_STATE_READY;

          /* 解锁 */
        __HAL_UNLOCK(hqspi);
      }
    }
    else
    {
      hqspi->ErrorCode |= HAL_QSPI_ERROR_INVALID_PARAM;
      status = HAL_ERROR;

        /* 解锁 */
      __HAL_UNLOCK(hqspi);
    }
  }
  else
  {
    status = HAL_BUSY;
```

```
    /* 解锁 */
    __HAL_UNLOCK(hqspi);
  }

  return status;
}
```

函数描述：

此函数用于 QSPI 接口数据发送，函数采用 DMA 方式。

函数参数：

- 第 1 个参数是 QSPI_HandleTypeDef 类型结构体指针变量，详见本章 3.3 小节。
- 第 2 个参数是要接收的数据地址。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: QSPI_WriteBuffer
*    功能说明: 页编程，页大小 256 字节，任意页都可以写入
*    形    参: _pBuf : 数据源缓冲区;
*              _uiWriteAddr : 目标区域首地址，即页首地址，比如 0， 256, 512 等。
*              _usWriteSize : 数据个数，不能超过页面大小，范围 1 - 256。
*    返 回 值: 1:成功， 0：失败
*********************************************************************************************************
*/
uint8_t QSPI_WriteBuffer(uint8_t *_pBuf, uint32_t _uiWriteAddr, uint16_t _usWriteSize)
{
    QSPI_CommandTypeDef sCommand={0};

    /* 写使能 */
    QSPI_WriteEnable(&QSPIHandle);

    /* 基本配置 */
    sCommand.InstructionMode   = QSPI_INSTRUCTION_1_LINE;    /* 1 线方式发送指令 */
    sCommand.AddressSize       = QSPI_ADDRESS_32_BITS;       /* 32 位地址 */
    sCommand.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;  /* 无交替字节 */
    sCommand.DdrMode           = QSPI_DDR_MODE_DISABLE;      /* W25Q256JV 不支持 DDR */
    sCommand.DdrHoldHalfCycle  = QSPI_DDR_HHC_ANALOG_DELAY;  /* DDR 模式，数据输出延迟 */
    sCommand.SIOOMode          = QSPI_SIOO_INST_ONLY_FIRST_CMD;   /* 仅发送一次命令 */

    /* 写序列配置 */
    sCommand.Instruction = QUAD_IN_FAST_PROG_4_BYTE_ADDR_CMD; /* 32bit 地址的 4 线快速写入命令 */
    sCommand.DummyCycles = 0;                       /* 不需要空周期 */
    sCommand.AddressMode = QSPI_ADDRESS_1_LINE;  /* 4 线地址方式 */
    sCommand.DataMode    = QSPI_DATA_4_LINES;    /* 4 线数据方式 */
    sCommand.NbData      = _usWriteSize;         /* 写数据大小 */
    sCommand.Address     = _uiWriteAddr;         /* 写入地址 */

    if (HAL_QSPI_Command(&QSPIHandle, &sCommand, 10000) != HAL_OK)
    {
        //return 0;
```

```
        Error_Handler(__FILE__, __LINE__);
    }

    /* 启动传输 */
    if (HAL_QSPI_Transmit_DMA(&QSPIHandle, _pBuf) != HAL_OK)
    {
       //return 0;
        Error_Handler(__FILE__, __LINE__);
    }

    QSPI_AutoPollingMemReady(&QSPIHandle);

    return 1;
}
```

78.4.10 函数 HAL_QSPI_Receive_DMA

函数原型：

```
HAL_StatusTypeDef HAL_QSPI_Receive_DMA(QSPI_HandleTypeDef *hqspi, uint8_t *pData)
{
  HAL_StatusTypeDef status = HAL_OK;
  uint32_t addr_reg = READ_REG(hqspi->Instance->AR);
  uint32_t data_size = (READ_REG(hqspi->Instance->DLR) + 1U);

    /* 上锁 */
  __HAL_LOCK(hqspi);

  if(hqspi->State == HAL_QSPI_STATE_READY)
  {
      /* 无错误 */
    hqspi->ErrorCode = HAL_QSPI_ERROR_NONE;

    if(pData != NULL )
    {
        /* 配置接收 */
      hqspi->RxXferCount = data_size;
          /* 更新状态 */
        hqspi->State = HAL_QSPI_STATE_BUSY_INDIRECT_RX;

          /* 清除中断标志 */
        __HAL_QSPI_CLEAR_FLAG(hqspi, (QSPI_FLAG_TE | QSPI_FLAG_TC));

          /* 配置接收 Configure */
        hqspi->RxXferSize = hqspi->RxXferCount;
        hqspi->pRxBuffPtr = pData;

          /* 设置接收完成中断 */
        hqspi->hmdma->XferCpltCallback = QSPI_DMARxCplt;

          /* 设置 MDMA 错误回调 */
        hqspi->hmdma->XferErrorCallback = QSPI_DMAError;

          /* 置空 MDMA 终止回调 */
        hqspi->hmdma->XferAbortCallback = NULL;
```

```
    /* 接收模式下，MDMA 的源地址是 QSPI DR，要禁止地址自增 */
  MODIFY_REG(hqspi->hmdma->Instance->CTCR, (MDMA_CTCR_SINC | MDMA_CTCR_SINCOS) , MDMA_SRC_INC_DISABLE);

    /* 更新目的地址配置 */
  if (hqspi->hmdma->Init.DestDataSize == MDMA_DEST_DATASIZE_BYTE)
  {
    MODIFY_REG(hqspi->hmdma->Instance->CTCR, (MDMA_CTCR_DINC | MDMA_CTCR_DINCOS) , MDMA_DEST_INC_BYTE);
  }
  else if (hqspi->hmdma->Init.DestDataSize == MDMA_DEST_DATASIZE_HALFWORD)
  {
    MODIFY_REG(hqspi->hmdma->Instance->CTCR, (MDMA_CTCR_DINC | MDMA_CTCR_DINCOS) ,
                           MDMA_DEST_INC_HALFWORD);
  }
  else if (hqspi->hmdma->Init.DestDataSize == MDMA_DEST_DATASIZE_WORD)
  {
    MODIFY_REG(hqspi->hmdma->Instance->CTCR, (MDMA_CTCR_DINC | MDMA_CTCR_DINCOS) , MDMA_DEST_INC_WORD);
  }
  else
  {
     /* 配置错误 */
    hqspi->ErrorCode |= HAL_QSPI_ERROR_DMA;
    status = HAL_ERROR;
  }
        /* 配置 CCR 寄存器，间接读 */
      MODIFY_REG(hqspi->Instance->CCR, QUADSPI_CCR_FMODE, QSPI_FUNCTIONAL_MODE_INDIRECT_READ);

        /* 启动传输 */
      WRITE_REG(hqspi->Instance->AR, addr_reg);

      /* 使能 MDMA */
    if (HAL_MDMA_Start_IT(hqspi->hmdma, (uint32_t)&hqspi->Instance->DR, (uint32_t)pData,
                          hqspi->RxXferSize, 1) == HAL_OK)
    {
        /* 解锁 */
      __HAL_UNLOCK(hqspi);

        /* 使能传输错误中断 */
      __HAL_QSPI_ENABLE_IT(hqspi, QSPI_IT_TE);

        /* 使能 MDMA 传输 */
      SET_BIT(hqspi->Instance->CR, QUADSPI_CR_DMAEN);
    }
    else
    {
      status = HAL_ERROR;
      hqspi->ErrorCode |= HAL_QSPI_ERROR_DMA;
      hqspi->State = HAL_QSPI_STATE_READY;

        /* 解锁 */
      __HAL_UNLOCK(hqspi);
    }
}
else
{
  hqspi->ErrorCode |= HAL_QSPI_ERROR_INVALID_PARAM;
  status = HAL_ERROR;

    /* 解锁 */
```

```
      __HAL_UNLOCK(hqspi);
    }
  }
  else
  {
    status = HAL_BUSY;

      /* 解锁 */
    __HAL_UNLOCK(hqspi);
  }

  return status;
}
```

函数描述：

此函数用于 QSPI 接口数据接收，函数采用 DMA 方式。

函数参数：

- 第 1 个参数是 QSPI_HandleTypeDef 类型结构体指针变量，详见本章 3.3 小节。
- 第 2 个参数是要接收的数据地址。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: QSPI_ReadBuffer
*    功能说明: 连续读取若干字节，字节个数不能超出芯片容量。
*    形    参: _pBuf : 数据源缓冲区。
*              _uiReadAddr : 起始地址。
*              _usSize : 数据个数, 可以大于 PAGE_SIZE, 但是不能超出芯片总容量。
*    返 回 值: 无
*********************************************************************************************************
*/
void QSPI_ReadBuffer(uint8_t * _pBuf, uint32_t _uiReadAddr, uint32_t _uiSize)
{

    QSPI_CommandTypeDef sCommand = {0};


    /* 基本配置 */
    sCommand.InstructionMode   = QSPI_INSTRUCTION_1_LINE;        /* 1 线方式发送指令 */
    sCommand.AddressSize       = QSPI_ADDRESS_32_BITS;           /* 32 位地址 */
    sCommand.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;      /* 无交替字节 */
    sCommand.DdrMode           = QSPI_DDR_MODE_DISABLE;          /* W25Q256JV 不支持 DDR */
    sCommand.DdrHoldHalfCycle  = QSPI_DDR_HHC_ANALOG_DELAY;      /* DDR 模式，数据输出延迟 */
    sCommand.SIOOMode          = QSPI_SIOO_INST_EVERY_CMD;       /* 每次传输要发指令 */

    /* 读取数据 */
    sCommand.Instruction = QUAD_INOUT_FAST_READ_4_BYTE_ADDR_CMD; /* 32bit 地址的 4 线快速读取命令 */
    sCommand.DummyCycles = 6;                     /* 空周期 */
    sCommand.AddressMode = QSPI_ADDRESS_4_LINES; /* 4 线地址 */
    sCommand.DataMode    = QSPI_DATA_4_LINES;    /* 4 线数据 */
    sCommand.NbData      = _uiSize;               /* 读取的数据大小 */
    sCommand.Address     = _uiReadAddr;           /* 读取数据的起始地址 */
```

```
    if (HAL_QSPI_Command(&QSPIHandle, &sCommand, 10000) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 读取 */
    if (HAL_QSPI_Receive_DMA(&QSPIHandle, _pBuf) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

78.4.11 函数 HAL_QSPI_MemoryMapped

函数原型：

```
HAL_StatusTypeDef HAL_QSPI_MemoryMapped(QSPI_HandleTypeDef *hqspi, QSPI_CommandTypeDef *cmd,
QSPI_MemoryMappedTypeDef *cfg)
{
  HAL_StatusTypeDef status;
  uint32_t tickstart = HAL_GetTick();

    /* 检查参数 */
  assert_param(IS_QSPI_INSTRUCTION_MODE(cmd->InstructionMode));
  if (cmd->InstructionMode != QSPI_INSTRUCTION_NONE)
  {
  assert_param(IS_QSPI_INSTRUCTION(cmd->Instruction));
  }

  assert_param(IS_QSPI_ADDRESS_MODE(cmd->AddressMode));
  if (cmd->AddressMode != QSPI_ADDRESS_NONE)
  {
    assert_param(IS_QSPI_ADDRESS_SIZE(cmd->AddressSize));
  }

  assert_param(IS_QSPI_ALTERNATE_BYTES_MODE(cmd->AlternateByteMode));
  if (cmd->AlternateByteMode != QSPI_ALTERNATE_BYTES_NONE)
  {
    assert_param(IS_QSPI_ALTERNATE_BYTES_SIZE(cmd->AlternateBytesSize));
  }

  assert_param(IS_QSPI_DUMMY_CYCLES(cmd->DummyCycles));
  assert_param(IS_QSPI_DATA_MODE(cmd->DataMode));

  assert_param(IS_QSPI_DDR_MODE(cmd->DdrMode));
  assert_param(IS_QSPI_DDR_HHC(cmd->DdrHoldHalfCycle));
  assert_param(IS_QSPI_SIOO_MODE(cmd->SIOOMode));

  assert_param(IS_QSPI_TIMEOUT_ACTIVATION(cfg->TimeOutActivation));

    /* 上锁 */
  __HAL_LOCK(hqspi);

  if(hqspi->State == HAL_QSPI_STATE_READY)
  {
```

```
    hqspi->ErrorCode = HAL_QSPI_ERROR_NONE;

      /* 更新状态 */
    hqspi->State = HAL_QSPI_STATE_BUSY_MEM_MAPPED;

      /* 等待 BUSY 标志复位 */
    status = QSPI_WaitFlagStateUntilTimeout(hqspi, QSPI_FLAG_BUSY, RESET, tickstart, hqspi->Timeout);

    if (status == HAL_OK)
    {
        /* 配置 QSPI CR 寄存器溢出时间计数 */
    MODIFY_REG(hqspi->Instance->CR, QUADSPI_CR_TCEN, cfg->TimeOutActivation);

    if (cfg->TimeOutActivation == QSPI_TIMEOUT_COUNTER_ENABLE)
      {
        assert_param(IS_QSPI_TIMEOUT_PERIOD(cfg->TimeOutPeriod));

          /* 配置溢出时间 */
        WRITE_REG(hqspi->Instance->LPTR, cfg->TimeOutPeriod);

          /* 清楚中断标志 */
        __HAL_QSPI_CLEAR_FLAG(hqspi, QSPI_FLAG_TO);

          /* 使能 QSPI 溢出中断 */
        __HAL_QSPI_ENABLE_IT(hqspi, QSPI_IT_TO);
      }

        /* 调用配置函数 */
      QSPI_Config(hqspi, cmd, QSPI_FUNCTIONAL_MODE_MEMORY_MAPPED);
    }
  }
  else
  {
    status = HAL_BUSY;
  }

    /* 解锁 */
  __HAL_UNLOCK(hqspi);

    /* 返回状态 */
  return status;
}
```

函数描述：

用于 QSPI 内存映射设置。

函数参数：

- 第 1 个参数是 QSPI_HandleTypeDef 类型结构体指针变量，详见本章 3.3 小节。
- 第 2 个参数是 QSPI_CommandTypeDef 类型结构体变量，详见本章 3.4 小节。
- 第 3 个参数是 QSPI_MemoryMappedTypeDef 类型结构体变量，详见本章 3.6 小节。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示串口忙，正在使用中。

使用举例：

```
/*
```

```
*********************************************************************************************************
*    函 数 名: QSPI_MemoryMapped
*    功能说明: QSPI 内存映射，地址 0x90000000
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void QSPI_MemoryMapped(void)
{
    QSPI_CommandTypeDef s_command = {0};
    QSPI_MemoryMappedTypeDef s_mem_mapped_cfg = {0};

    /* 基本配置 */
    s_command.InstructionMode = QSPI_INSTRUCTION_1_LINE;       /* 1 线方式发送指令 */
    s_command.AddressSize = QSPI_ADDRESS_32_BITS;              /* 32 位地址 */
    s_command.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;   /* 无交替字节 */
    s_command.DdrMode = QSPI_DDR_MODE_DISABLE;                 /* W25Q256JV 不支持 DDR */
    s_command.DdrHoldHalfCycle = QSPI_DDR_HHC_ANALOG_DELAY;    /* DDR 模式，数据输出延迟 */
    s_command.SIOOMode = QSPI_SIOO_INST_EVERY_CMD;             /* 每次传输都发指令 */

    /* 全部采用 4 线 */
    s_command.Instruction = QUAD_INOUT_FAST_READ_4_BYTE_ADDR_CMD; /* 快速读取命令 */
    s_command.AddressMode = QSPI_ADDRESS_4_LINES;                 /* 4 个地址线 */
    s_command.DataMode = QSPI_DATA_4_LINES;                       /* 4 个数据线 */
    s_command.DummyCycles = 6;                                    /* 空周期 */

    /* 关闭溢出计数 */
    s_mem_mapped_cfg.TimeOutActivation = QSPI_TIMEOUT_COUNTER_DISABLE;
    s_mem_mapped_cfg.TimeOutPeriod = 0;

    if (HAL_QSPI_MemoryMapped(&QSPIHandle, &s_command, &s_mem_mapped_cfg) != HAL_OK)
    {
       Error_Handler(__FILE__, __LINE__);
    }
}
```

78.5 总结

本章节就为大家讲解这么多，要熟练掌握 QSPI 总线的查询，中断和 DMA 方式的 API 用法。

第79章 STM32H7 的 QSPI 总线应用之驱动 W25QXX（支持查询和 MDMA）

本章节为大家讲解标准 QSPI 接线方式驱动 W25QXX，实现了查询和 MDMA 两种方式。

80.1 初学者重要提示
80.2 W25QXX 硬件设计
80.4 W25QXX 关键知识点整理（重要）
80.5 W25QXX 驱动设计
80.6 W25QXX 板级支持包（bsp_spi_flash.c）
80.7 使用例程设计框架
80.8 实验例程说明（MDK）
80.9 实验例程说明（IAR）
80.10 总结

79.1 初学者重要提示

- 学习本章节前，务必优先学习第 79 章。
- W25Q256JV 属于 NOR 型 Flash 存储芯片。
- W25Q256JV 手册下载地址：链接 （这是一个超链接），当前章节配套例子的 Doc 文件件里面也有存放。

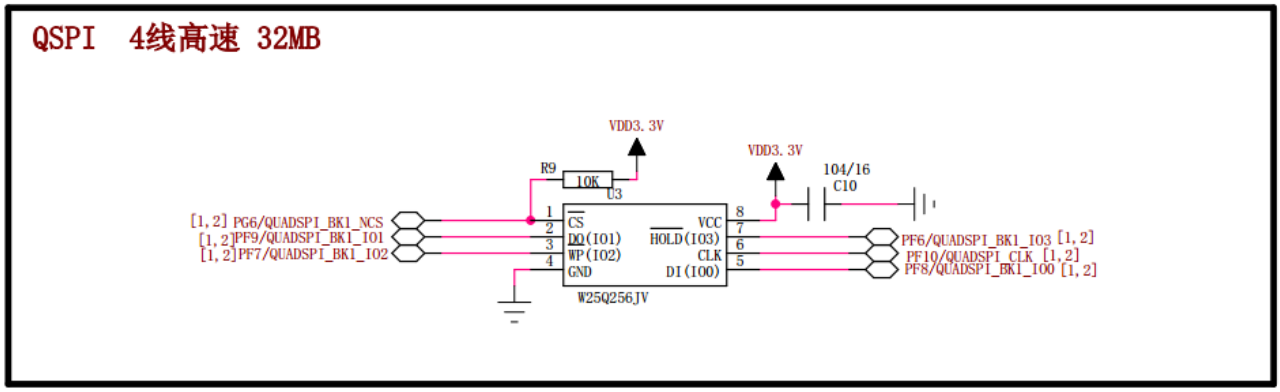
2Gb | 1Gb | 512Mb | 256Mb | 128Mb | 64Mb | 32Mb | 16Mb | 8Mb | 4Mb | 2Mb | 1Mb | 512Kb

Part No.	Voltage	Speed	Temp.	Features	Automotive	Status	Datasheet
W25H256JV	2.7V - 3.6V	104MHz	-40°C ~ 125°C	SPI / QPI / DTR On-Chip ECC	P	-	Contact us
W25Q256JV	2.7V - 3.6V	133MHz	-40°C ~ 85°C / -40°C ~ 105°C	SPI 4 I/O Fixed 4 I/O Fixed	P	P	
W25Q256JV_DTR	2.7V - 3.6V	133MHz	-40°C ~ 85°C / -40°C ~ 105°C	SPI / QPI / DTR	P	P	
W25Q257JV	2.7V - 3.6V	133MHz	-40°C ~ 85°C	4-Byte Addressing Mode 4 I/O Fixed	N	P	
W25Q256FV	2.7V - 3.6V	104MHz	-40°C ~ 85°C	SPI / QPI	N	N	
W25Q257FV	2.7V - 3.6V	104MHz	-40°C ~ 85°C	4-Byte Addressing Mode SPI / QPI	N	N	
W25Q256JW	1.7V - 1.95V	133MHz	-40°C ~ 85°C / -40°C ~ 105°C	SPI 4 I/O Fixed	P	P	
W25Q256JW_DTR	1.7V - 1.95V	133MHz	-40°C ~ 85°C / -40°C ~ 105°C	SPI / QPI / DTR	P	P	
W25R256JV	2.7V - 3.6V	104MHz	-40°C ~ 85°C	SPI with Single, Dual, Quad I/O RPMC	-	P	
W25R256JW	1.7V - 1.95V	104MHz	-40°C ~ 85°C	SPI with Single, Dual, Quad I/O RPMC	-	P	

- 本章第 3 小节整理的知识点比较重要，务必要了解下，特别是页编程和页回卷。
- 对 QSPI Flash W25Q256JV 的不同接线方式（1 线，2 线或者 4 线，这里的线是指的数据线），编程命令是不同的。
- W25Q256JV 最高支持 133MHz。
- STM32H7 驱动 QSPI Flash 的 4 线 DMA 模式，读速度 48MB/S 左右。
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=91616 。
- 内存映射模式下，最后一个字节无法正常读取的解决办法：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=96726 。
- 本章配套例子的 DMA 是采用性能最强的 MDMA。

79.2 W25QXX 硬件设计

STM32H7 驱动 W25Q256JV 的硬件设计如下：

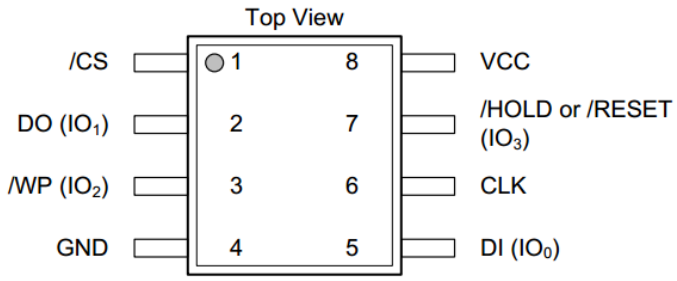


关于这个原理图，要了解到以下几个知识：

- V7 开发板实际外接的芯片是 W25Q256JV。
- CS 片选最好接上拉电阻，防止意外操作。
- W25Q256 的 WP 引脚用于写保护，低电平有效性，当前是将其作为 4 方式的 IO2。
- HOLD 引脚也是低电平有效，当前是将其接到高电平。此引脚的作用是 CS 片选低电平时，DO 引脚输出高阻，忽略 CLK 和 DI 引脚上的信号。当前是将其作为 4 线方式的 IO3。

79.3 W25QXX 关键知识点整理（重要）

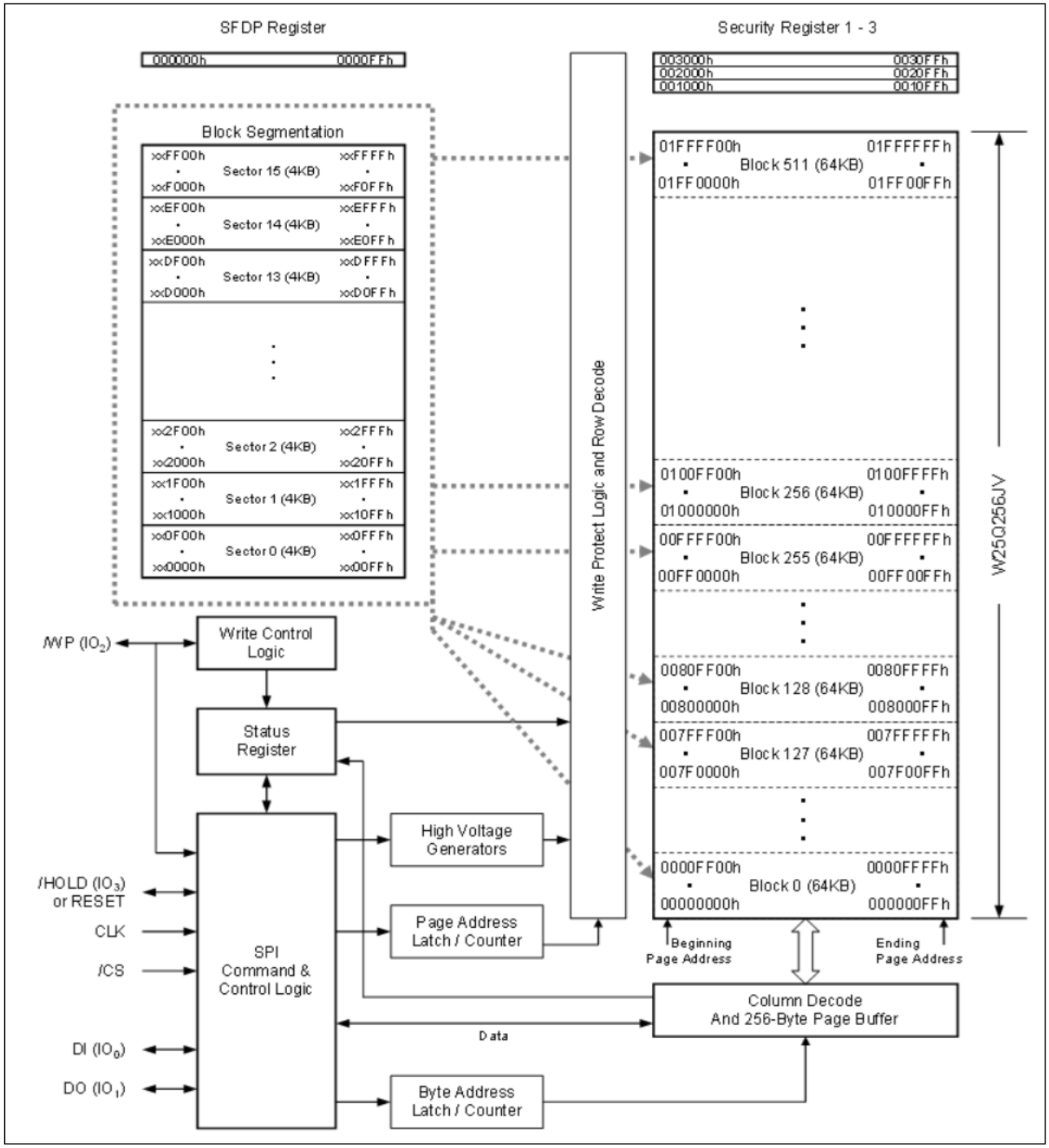
驱动 W25QXX 前要先了解下这个芯片的相关信息。



79.3.1W25QXX 基础信息

- W25Q256JV 是 32MB（256Mbit）。
- W25Q256JV 支持标准 SPI（单线 SPI），用到引脚 CLK、CS，DI 和 DO 引脚。
  支持两线 SPI，用到引脚 CLK、CS、IO_0、IO_1 。
  支持四线 SPI，用到引脚 CLK、CS、IO_0、IO_1，IO_2、IO_3。
  （注：这里几线的意思是几个数据线）。
- W25Q256JV 支持的最高时钟是 133MHz。
- 每个扇区最少支持 10 万次擦写，可以保存 20 年数据。
- 页大小是 256 字节，支持页编程，也就是一次编写 256 个字节，也可以一个一个编写。
- 支持 4KB 为单位的扇区擦除，也可以 32KB 或者 64KB 为单位的擦除。

整体框图如下：



W25Q256JV：

- 有 512 个 Block，每个 Block 大小 64KB。
- 每个 Block 有 16 个 Sector，每个 Sector 大小 4KB。
- 每个 Sector 有 16 个 Page，每个 Page 大小是 256 字节。

79.3.2 W25QXX 命令

使用 W25Q256 的接线方式不同，使用的命令也有所不同，使用的时候务必要注意，当前我们使用的

QSPI，即 4 线 SPI，并且用的 4 字节地址模式，使用的命令如下：

Data Input Output	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
Number of Clock(1-1-1)	**8**	**8**	**8**	**8**	**8**	**8**	**8**
Write Enable	**06h**						
Volatile SR Write Enable	**50h**						
Write Disable	**04h**						
Release Power-down / ID	**ABh**	Dummy	Dummy	Dummy	(ID7-ID0)(2)		
Manufacturer/Device ID	**90h**	Dummy	Dummy	00h	(MF7-MF0)	(ID7-ID0)	
JEDEC ID	**9Fh**	(MF7-MF0)	(ID15-ID8)	(ID7-ID0)			
Read Unique ID	**4Bh**	Dummy	Dummy	Dummy	Dummy	Dummy	(UID63-0)
Read Data	**03h**	**A31-A24**	A23-A16	A15-A8	A7-A0	(D7-D0)	
Read Data with 4-Byte Address	**13h**	A31-A24	A23-A16	A15-A8	A7-A0	(D7-D0)	
Fast Read	**0Bh**	A31-A24	A23-A16	A15-A8	A7-A0	Dummy	(D7-D0)
Fast Read with 4-Byte Address	**0Ch**	A31-A24	A23-A16	A15-A8	A7-A0	Dummy	(D7-D0)
Page Program	**02h**	A31-A24	A23-A16	A15-A8	A7-A0	D7-D0	D7-D0(3)
Page Program with 4-Byte Address	**12h**	A31-A24	A23-A16	A15-A8	A7-A0	D7-D0	D7-D0(3)
Sector Erase (4KB)	**20h**	A31-A24	A23-A16	A15-A8	A7-A0		
Sector Erase (4KB) with 4-Byte Address	**21h**	A31-A24	A23-A16	A15-A8	A7-A0		
Block Erase (32KB)	**52h**	A31-A24	A23-A16	A15-A8	A7-A0		
Block Erase (64KB)	**D8h**	A31-A24	A23-A16	A15-A8	A7-A0		
Block Erase (64KB) with 4-Byte Address	**DCh**	A31-A24	A23-A16	A15-A8	A7-A0		
Chip Erase	**C7h/60h**						
Read Status Register-1	**05h**	(S7-S0)(2)					
Write Status Register-1(4)	**01h**	(S7-S0)(4)					
Read Status Register-2	**35h**	(S15-S8)(2)					
Write Status Register-2	**31h**	(S15-S8)					
Read Status Register-3	**15h**	(S23-S16)(2)					
Write Status Register-3	**11h**	(S23-S16)					
Read Extended Addr. Reg.	**C8h**	(EA7-EA0)(2)					
Write Extended Addr. Reg.	**C5h**	(EA7-EA0)					
Read SFDP Register	**5Ah**	A23-A16	A15-A8	A7-A0	Dummy	(D7-D0)	
Erase Security Register(5)	**44h**	A31-A24	A23-A16	A15-A8	A7-A0		
Program Security Register(5)	**42h**	A31-A24	A23-A16	A15-A8	A7-A0	D7-D0	D7-D0(3)
Read Security Register(5)	**48h**	A31-A24	A23-A16	A15-A8	A7-A0	Dummy	(D7-D0)
Global Block Lock	**7Eh**						
Global Block Unlock	**98h**						
Read Block Lock	**3Dh**	A31-A24	A23-A16	A15-A8	A7-A0	(L7-L0)	
Individual Block Lock	**36h**	A31-A24	A23-A16	A15-A8	A7-A0		
Individual Block Unlock	**39h**	A31-A24	A23-A16	A15-A8	A7-A0		
Erase / Program Suspend	**75h**						
Erase / Program Resume	**7Ah**						
Power-down	**B9h**						
Enter 4-Byte Address Mode	**B7h**						
Exit 4-Byte Address Mode	**E9h**						
Enable Reset	**66h**						
Reset Device	**99h**						

Data Input Output	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7	Byte 8	Byte9
Number of Clock(1-1-2)	**8**	**8**	**8**	**8**	**8**	**8**	**4**	**4**	
Fast Read Dual Output	**3Bh**	A31-A24	A23-A16	A15-A8	A7-A0	Dummy	(D7-D0,...)(7)		
Fast Read Dual Output with 4-Byte Address	**3Ch**	A31-A24	A23-A16	A15-A8	A7-A0	Dummy	(D7-D0,...)(7)		
Number of Clock(1-2-2)	**8**	**4**	**4**	**4**	**4**	**4**	**4**	**4**	
Mftr./Device ID Dual I/O	**92h**	A31-A24	A23-A16	A15-A8	00	Dummy(11)	(MF7-MF0)	(ID7-ID0)	
Fast Read Dual I/O	**BBh**	A31-A24	A23-A16	A15-A8	A7-A0	Dummy(11)	(D7-D0)		
Fast Read Dual I/O with 4-Byte Address	**BCh**	A31-A24	A23-A16	A15-A8	A7-A0	Dummy(11)	(D7-D0)		
Number of Clock(1-1-4)	**8**	**8**	**8**	**8**	**8**	**4**	**4**	**4**	
Quad Input Page Program	**32h**	A31-A24	A23-A16	A15-A8	A7-A0	(D7-D0)(9)	(D7-D0)(3)..		
Quad Page Program with 4-Byte Address	**34h**	A31-A24	A23-A16	A15-A8	A7-A0	D7-D0	D7-D0	D7-D0	D7-D0
Fast Read Quad Output	**6Bh**	A31-A24	A23-A16	A15-A8	A7-A0	Dummy	Dummy	(D7-D0)(9)	
Fast Read Quad Output with 4-Byte Address	**6Ch**	A31-A24	A23-A16	A15-A8	A7-A0	Dummy	Dummy	(D7-D0)(9)	
Number of Clock(1-4-4)	**8**	**2**	**2**	**2**	**2**	**2**	**4**	**2**	**2**
Mftr./Device ID Quad I/O	**94h**	A31-A24	A23-A16	A15-A8	00	Dummy(11)	Dummy	(MF7-MF0)	(ID7-ID0)
Fast Read Quad I/O	**EBh**	A31-A24	A23-A16	A15-A8	A7-A0	Dummy(11)	Dummy	(D7-D0)	
Fast Read Quad I/O with 4-Byte Address	**ECh**	A31-A24	A23-A16	A15-A8	A7-A0	Dummy(11)	Dummy	(D7-D0)	
Set Burst with Wrap	**77h**	Dummy	Dummy	Dummy	Dummy	W7-W0			

当前主要用到如下几个命令：

```
#define WRITE_ENABLE_CMD        0x06          /* 写使能指令 */
#define READ_ID_CMD2            0x9F          /* 读取 ID 命令 */
#define READ_STATUS_REG_CMD     0x05          /* 读取状态命令 */
#define BULK_ERASE_CMD          0xC7          /* 整个芯片擦除命令 */
#define SUBSECTOR_ERASE_4_BYTE_ADDR_CMD      0x21     /* 32bit 地址扇区擦除指令, 4KB */
#define QUAD_IN_FAST_PROG_4_BYTE_ADDR_CMD    0x34     /* 32bit 地址的 4 线快速写入命令 */
#define QUAD_INOUT_FAST_READ_4_BYTE_ADDR_CMD 0xEC     /* 32bit 地址的 4 线快速读取命令 */
```

79.3.3W25QXX 页编程和页回卷

SPI Flash 仅支持页编程（页大小 256 字节），所有其它大批量数据的写入都是以页为单位。这里注意所说的页编程含义，页编程分为以下三步（伪代码）：

```
bsp_spiWrite1(0x02);                                  ----------第 1 步发送页编程命令
bsp_spiWrite1((_uiWriteAddr & 0xFF0000) >> 16);       ----------第 2 步发送地址
bsp_spiWrite1((_uiWriteAddr & 0xFF00) >> 8);
bsp_spiWrite1(_uiWriteAddr & 0xFF);

    for (i = 0; i < _usSize; i++)
    {
        bsp_spiWrite1(*_pBuf++);     ----------第 3 步写数据，此时就可以连续写入数据了，
                                              不需要再重新设置地址，地址会自增。这样可以大大加快写入速度。
    }
```

页编程的含义恰恰就体现在第 3 步了，如果用户设置的“起始地址+数据长度”所确定的地址范围超过了此起始地址所在的页，地址自增不会超过页范围，而是重新回到了此页的首地进行编写。这一点要特别的

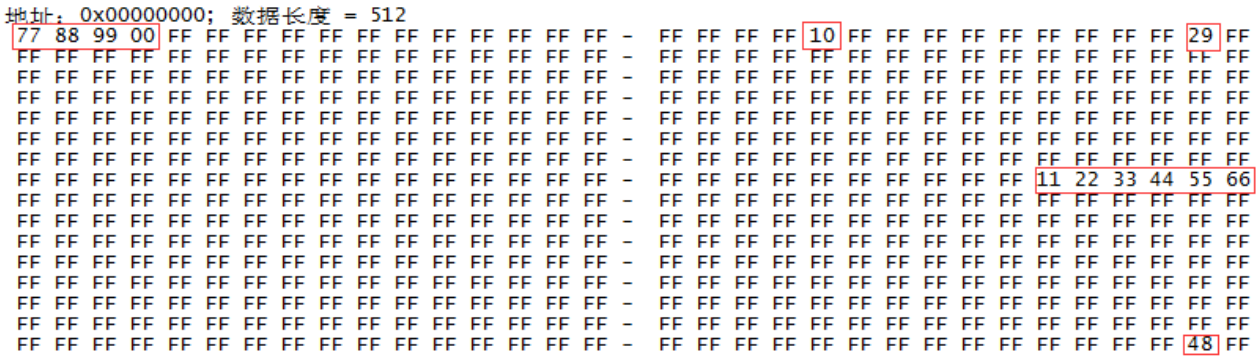
注意。如果用户不需要使用地址自增效果，那么直接指定地址进行编写即可。可以任意指定地址进行编写，编写前一定要进行擦除。

比如下面就是页内操作（使用前已经进行了扇区擦除，每次擦除最少擦除一个扇区 4KB）：

```
uint8_t tempbuf[10] = {0x11,0x22,0x33,0x44,0x55,0x66,0x77,0x88,0x99,0x00};
uint8_t temp1 = 0x10, temp2 = 0x29, temp3 = 0x48;
```

- 从 250 地址开始写入 10 个字节数据 PageWrite(tempbuf, 250, 10);(因为一旦写入超过地址 255，就会从 0 地址开始重新写)。
- 向地址 20 写入 1 个字节数据：PageWrite(&temp1, 20, 1);
- 向地址 30 写入 1 个字节数据：PageWrite(&temp2, 30, 1);
- 向地址 510 写入 1 个字节数据：PageWrite(&temp3, 510, 1) (这里已经是写到下一页了)

下面是将从 0 地址到 511 地址读取出来的 512 个字节数据，一行 32 字节。



79.3.4W25QXX 扇区擦除

SPI Flash 的擦除支持扇区擦除（4KB），块擦除（32KB 或者 64KB）以及整个芯片擦除。对于扇区擦除和块擦除，使用的时候要注意一点，一般情况下，只需用户给出扇区或者块的首地址即可。

如果给的不是扇区或者块的首地址也没有关系的，只要此地址是在扇区或者块的范围内，此扇区或者块也可以被正确擦除。不过建议使用时给首地址，方便管理。

79.3.5W25QXX 规格参数

这里我们主要了解擦写耗时和支持的时钟速度，下面是擦写时间参数：

DESCRIPTION	SYMBOL	ALT	SPEC			UNIT
			MIN	TYP	MAX	
Write Protect Setup Time Before /CS Low	tWHSL(3)		20			ns
Write Protect Hold Time After /CS High	tSHWL(3)		100			ns
/CS High to Power-down Mode	tDP(2)				3	µs
/CS High to Standby Mode without ID Read	tRES1(2)				3	µs
/CS High to Standby Mode with ID Read	tRES2(2)				1.8	µs
/CS High to next Instruction after Suspend	tSUS(2)				20	µs
/CS High to next Instruction after Reset	tRST(2)				30	µs
/RESET pin Low period to reset the device	tRESET(2)		1(4)			µs
Write Status Register Time	tW			10	15	ms
Page Program Time	tPP			0.4	3	ms
Sector Erase Time (4KB)	tSE			50	400	ms
Block Erase Time (32KB)	tBE1			120	1,600	ms
Block Erase Time (64KB)	tBE2			150	2,000	ms
Chip Erase Time	tCE			80	400	s

- 页编程时间：典型值 0.4ms，最大值 3ms。
- 扇区擦除时间（4KB）：典型值 50ms，最大值 400ms。
- 块擦除时间（32KB）：典型值 120ms，最大值 1600ms。
- 块擦除时间（64KB）：典型值 150ms，最大值 2000ms。
- 整个芯片擦除时间：典型值 80s，最大值 400s。

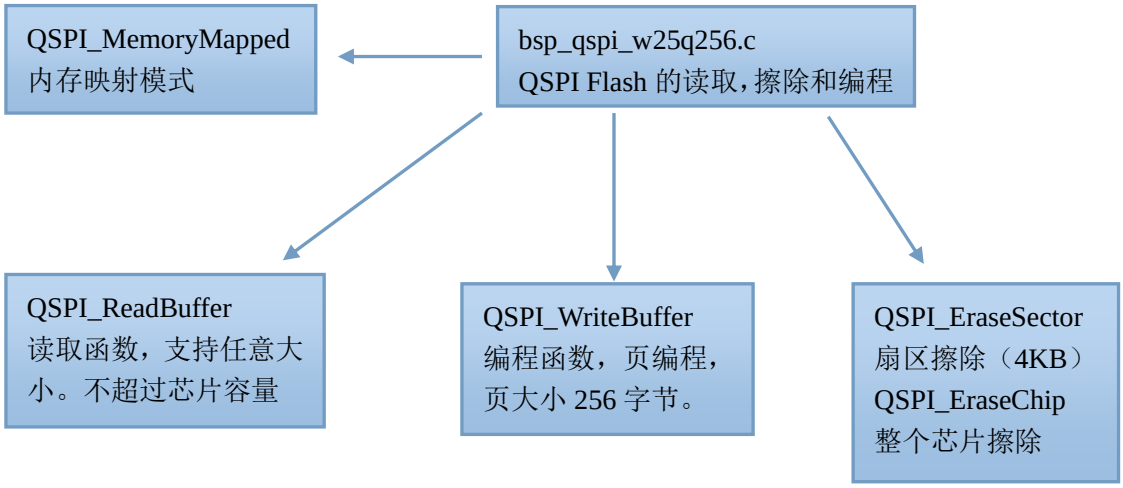
支持的速度参数如下：

DESCRIPTION	SYMBOL	ALT	SPEC			UNIT
			MIN	TYP	MAX	
Clock frequency except for Read Data (03h) & DTR instructions (3.0V-3.6V)	FR	fC1	D.C.		133	MHz
Clock frequency except for Read Data (03h) & DTR instructions(2.7V-3.0V)	FR	fC1	D.C.		104	MHz
Clock frequency for Read Data instruction (03h)	fR		D.C.		50	MHz
Clock High, Low Time for all instructions except for Read Data (03h)	tCLH, tCLL(1)		45%PC			ns
Clock High, Low Time for Read Data (03h) instruction	tCRLH, tCRLL(1)		45%PC			ns

可以看到最高支持的读时钟（使用命令 03H）速度是 50MHz，其它命令速度可以做到 133MHz。

79.4 W25QXX 驱动设计

W25QXX 的程序驱动框架设计如下：



有了这个框图，程序设计就比较好理解了。

79.4.1 第 1 步：QSPI 总线配置

QSPI 总线配置如下：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitQSPI_W25Q256
*    功能说明: QSPI Flash 硬件初始化，配置基本参数
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitQSPI_W25Q256(void)
{
    /* 复位 QSPI */
    QSPIHandle.Instance = QUADSPI;
    if (HAL_QSPI_DeInit(&QSPIHandle) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 设置时钟速度，QSPI clock = 200MHz / (ClockPrescaler+1) = 100MHz */
    QSPIHandle.Init.ClockPrescaler  = 1;

    /* 设置 FIFO 阀值，范围 1 - 32 */
    QSPIHandle.Init.FifoThreshold   = 32;

    /*
        QUADSPI 在 FLASH 驱动信号后过半个 CLK 周期才对 FLASH 驱动的数据采样。
        在外部信号延迟时，这有利于推迟数据采样。
    */
    QSPIHandle.Init.SampleShifting  = QSPI_SAMPLE_SHIFTING_HALFCYCLE;
```

```
    /*Flash 大小是 2^(FlashSize + 1) = 2^25 = 32MB */
    //QSPI_FLASH_SIZE - 1; 需要扩大一倍，否则内存映射方位最后 1 个地址时，会异常。
    QSPIHandle.Init.FlashSize       = QSPI_FLASH_SIZE;

    /* 命令之间的 CS 片选至少保持 2 个时钟周期的高电平 */
    QSPIHandle.Init.ChipSelectHighTime = QSPI_CS_HIGH_TIME_2_CYCLE;

    /*
       MODE0: 表示片选信号空闲期间，CLK 时钟信号是低电平
       MODE3: 表示片选信号空闲期间，CLK 时钟信号是高电平
    */
    QSPIHandle.Init.ClockMode = QSPI_CLOCK_MODE_0;

    /* QSPI 有两个 BANK，这里使用的 BANK1 */
    QSPIHandle.Init.FlashID   = QSPI_FLASH_ID_1;

    /* V7 开发板仅使用了 BANK1，这里是禁止双 BANK */
    QSPIHandle.Init.DualFlash = QSPI_DUALFLASH_DISABLE;

    /* 初始化配置 QSPI */
    if (HAL_QSPI_Init(&QSPIHandle) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

QSPI 这部分配置，要特别注意 Flash 大小的设置，这里做了特别处理，本来是应该填入 QSPI_FLASH_SIZE-1，而我们实际上填入的是 QSPI_FLASH_SIZE，主要是因为内存映射模式下，最后一个字节访问有问题。

79.4.2 第 2 步：QSPI 总线的查询和 MDMA 方式设置

本章提供了 QSPI Flash 的查询和 MDMA 两种方式的例子，驱动的区别是调用的 API 不同，查询方式调用的 API 是 HAL_QSPI_Transmit 和 HAL_QSPI_Receive，而 DMA 方式使用的 API 是 HAL_QSPI_Transmit_DMA 和 HAL_QSPI_Receive_DMA。

79.4.3 第 3 步：W25QXX 的读取实现

注：这里以查询方式的 API 进行说明，DMA 方式是一样的。

W25QXX 的读取功能是发送的 4 线快速读取指令 0xEC，设置任意地址都可以读取数据，只要不超过芯片容量即可（如果采用的 DMA 方式，限制每次最大读取 65536 字节）。

```
/*
*********************************************************************************************************
*    函 数 名: QSPI_ReadBuffer
*    功能说明: 连续读取若干字节，字节个数不能超出芯片容量。
*    形    参: _pBuf : 数据源缓冲区。
*              _uiReadAddr : 起始地址。
*              _usSize : 数据个数, 可以大于 PAGE_SIZE, 但是不能超出芯片总容量。
```

```
*    返 回 值: 无
*********************************************************************************************************
*/
void QSPI_ReadBuffer(uint8_t * _pBuf, uint32_t _uiReadAddr, uint32_t _uiSize)
{

    QSPI_CommandTypeDef sCommand = {0};


    /* 基本配置 */
    sCommand.InstructionMode   = QSPI_INSTRUCTION_1_LINE;       /* 1 线方式发送指令 */
    sCommand.AddressSize       = QSPI_ADDRESS_32_BITS;          /* 32 位地址 */
    sCommand.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;     /* 无交替字节 */
    sCommand.DdrMode           = QSPI_DDR_MODE_DISABLE;         /* W25Q256JV 不支持 DDR */
    sCommand.DdrHoldHalfCycle  = QSPI_DDR_HHC_ANALOG_DELAY;     /* DDR 模式，数据输出延迟 */
    sCommand.SIOOMode          = QSPI_SIOO_INST_EVERY_CMD;      /* 每次传输要发指令 */

    /* 读取数据 */
    sCommand.Instruction = QUAD_INOUT_FAST_READ_4_BYTE_ADDR_CMD; /* 32bit 地址的 4 线快速读取命令 */
    sCommand.DummyCycles = 6;                       /* 空周期 */
    sCommand.AddressMode = QSPI_ADDRESS_4_LINES;    /* 4 线地址 */
    sCommand.DataMode    = QSPI_DATA_4_LINES;       /* 4 线数据 */
    sCommand.NbData      = _uiSize;                 /* 读取的数据大小 */
    sCommand.Address     = _uiReadAddr;             /* 读取数据的起始地址 */

    if (HAL_QSPI_Command(&QSPIHandle, &sCommand, 10000) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 读取 */
    if (HAL_QSPI_Receive(&QSPIHandle, _pBuf, 10000) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

此时函数使用的指令 0xEC 对应的 W25Q256JV 手册说明，注意红色方框位置：

Number of Clock(1-4-4)	8	2	2	2	2	2	4	2	2
Mftr./Device ID Quad I/O	94h	A31-A24	A23-A16	A15-A8	00	Dummy(11)	Dummy	(MF7-MF0)	(ID7-ID0)
Fast Read Quad I/O	EBh	A31-A24	A23-A16	A15-A8	A7-A0	Dummy(11)	Dummy	(D7-D0)	
Fast Read Quad I/O with 4-Byte Address	ECh	A31-A24	A23-A16	A15-A8	A7-A0	Dummy(11)	Dummy	(D7-D0)	
Set Burst with Wrap	77h	Dummy	Dummy	Dummy	Dummy	W7-W0			

左上角的 1-4-4 就是指令阶段使用 1 个 IO，地址阶段使用 4 个 IO，数据阶段也是使用 4 个 IO，并且采用的 4 字节地址方式，反映到程序里面就是：

sCommand.InstructionMode = QSPI_INSTRUCTION_1_LINE;

sCommand.AddressMode = QSPI_ADDRESS_4_LINES;

sCommand.AddressSize = QSPI_ADDRESS_32_BITS;

sCommand.DataMode = QSPI_DATA_4_LINES;

79.4.4第 4 步：W25QXX 的编程实现

注：这里以查询方式的 API 进行说明，DMA 方式是一样的。

下面实现了一个页编程函数：

```
/*
*********************************************************************************************************
*    函 数 名: QSPI_WriteBuffer
*    功能说明: 页编程，页大小 256 字节，任意页都可以写入
*    形    参: _pBuf : 数据源缓冲区;
*              _uiWriteAddr : 目标区域首地址，即页首地址，比如 0， 256, 512 等。
*              _usWriteSize : 数据个数，不能超过页面大小，范围 1 - 256。
*    返 回 值: 1:成功， 0：失败
*********************************************************************************************************
*/
uint8_t QSPI_WriteBuffer(uint8_t *_pBuf, uint32_t _uiWriteAddr, uint16_t _usWriteSize)
{
    QSPI_CommandTypeDef sCommand={0};

    /* 写使能 */
    QSPI_WriteEnable(&QSPIHandle);

    /* 基本配置 */
    sCommand.InstructionMode   = QSPI_INSTRUCTION_1_LINE;    /* 1 线方式发送指令 */
    sCommand.AddressSize       = QSPI_ADDRESS_32_BITS;       /* 32 位地址 */
    sCommand.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;  /* 无交替字节 */
    sCommand.DdrMode           = QSPI_DDR_MODE_DISABLE;      /* W25Q256JV 不支持 DDR */
    sCommand.DdrHoldHalfCycle  = QSPI_DDR_HHC_ANALOG_DELAY;  /* DDR 模式，数据输出延迟 */
    sCommand.SIOOMode          = QSPI_SIOO_INST_ONLY_FIRST_CMD;   /* 仅发送一次命令 */

    /* 写序列配置 */
    sCommand.Instruction = QUAD_IN_FAST_PROG_4_BYTE_ADDR_CMD; /* 32bit 地址的 4 线快速写入命令 */
    sCommand.DummyCycles = 0;                     /* 不需要空周期 */
    sCommand.AddressMode = QSPI_ADDRESS_1_LINE;   /* 4 线地址方式 */
    sCommand.DataMode    = QSPI_DATA_4_LINES;     /* 4 线数据方式 */
    sCommand.NbData      = _usWriteSize;          /* 写数据大小 */
    sCommand.Address     = _uiWriteAddr;          /* 写入地址 */

    if (HAL_QSPI_Command(&QSPIHandle, &sCommand, 10000) != HAL_OK)
    {
       //return 0;
        Error_Handler(__FILE__, __LINE__);
    }

    /* 启动传输 */
    if (HAL_QSPI_Transmit(&QSPIHandle, _pBuf, 10000) != HAL_OK)
    {
       //return 0;
        Error_Handler(__FILE__, __LINE__);

    }

    QSPI_AutoPollingMemReady(&QSPIHandle);

    return 1;
}
```

此时函数使用的指令 0x34 对应的 W25Q256JV 手册说明，注意红色方框位置：

Number of Clock(1-1-4)	**8**	**8**	**8**	**8**	**8**	**4**	**4**	**4**	
Quad Input Page Program	**32h**	A31-A24	A23-A16	A15-A8	A7-A0	(D7-D0)(9)	(D7-D0)(3)-		
Quad Page Program with 4-Byte Address	**34h**	A31-A24	A23-A16	A15-A8	A7-A0	D7-D0	D7-D0	D7-D0	D7-D0
Fast Read Quad Output	**6Bh**	A31-A24	A23-A16	A15-A8	A7-A0	Dummy	Dummy	(D7-D0)(9)	
Fast Read Quad Output with 4-Byte Address	**6Ch**	A31-A24	A23-A16	A15-A8	A7-A0	Dummy	Dummy	(D7-D0)(9)	
Number of Clock(1-4-4)	**8**	**2**	**2**	**2**	**2**	**2**	**4**	**2**	**2**
Mftr./Device ID Quad I/O	**94h**	A31-A24	A23-A16	A15-A8	00	Dummy(11)	Dummy	(MF7-MF0)	(ID7-ID0)

左上角的 1-4-4 就是指令阶段使用 1 个 IO，地址阶段使用 4 个 IO，数据阶段也是使用 4 个 IO，并且采用的 4 字节地址方式，反映到程序里面就是：

sCommand.InstructionMode = QSPI_INSTRUCTION_1_LINE;

sCommand.AddressMode = QSPI_ADDRESS_4_LINES;

sCommand.AddressSize = QSPI_ADDRESS_32_BITS;

sCommand.DataMode = QSPI_DATA_4_LINES;

79.4.5第 5 步：W25QXX 的扇区擦除实现

注：这里以查询方式的 API 进行说明，DMA 方式是一样的。

通过发送“扇区擦除命令+扇区地址”即可完成相应扇区的擦除，擦除的扇区大小是 4KB。

```
/*
*********************************************************************************************************
*    函 数 名: QSPI_EraseSector
*    功能说明: 擦除指定的扇区，扇区大小 4KB
*    形    参: _uiSectorAddr : 扇区地址，以 4KB 为单位的地址，比如 0，4096，8192 等，
*    返 回 值: 无
*********************************************************************************************************
*/
void QSPI_EraseSector(uint32_t _uiSectorAddr)
{
    QSPI_CommandTypeDef sCommand={0};

    /* 写使能 */
    QSPI_WriteEnable(&QSPIHandle);

    /* 基本配置 */
    sCommand.InstructionMode   = QSPI_INSTRUCTION_1_LINE;    /* 1 线方式发送指令 */
    sCommand.AddressSize       = QSPI_ADDRESS_32_BITS;       /* 32 位地址 */
    sCommand.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;  /* 无交替字节 */
    sCommand.DdrMode           = QSPI_DDR_MODE_DISABLE;      /* W25Q256JV 不支持 DDR */
    sCommand.DdrHoldHalfCycle  = QSPI_DDR_HHC_ANALOG_DELAY;  /* DDR 模式，数据输出延迟 */
    sCommand.SIOOMode          = QSPI_SIOO_INST_EVERY_CMD;   /* 每次传输都发指令 */

    /* 擦除配置 */
    sCommand.Instruction = SUBSECTOR_ERASE_4_BYTE_ADDR_CMD;   /*32bit 地址方式的扇区擦除命令，扇区大小 4KB*/
    sCommand.AddressMode = QSPI_ADDRESS_1_LINE;  /* 地址发送是 1 线方式 */
    sCommand.Address     = _uiSectorAddr;        /* 扇区首地址，保证是 4KB 整数倍 */
```

```
    sCommand.DataMode    = QSPI_DATA_NONE;       /* 无需发送数据 */
    sCommand.DummyCycles = 0;                    /* 无需空周期 */

    if (HAL_QSPI_Command(&QSPIHandle, &sCommand, 10000) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    QSPI_AutoPollingMemReady(&QSPIHandle);
}
```

此时函数使用的指令 0x21 对应的 W25Q256JV 手册说明，注意红色方框位置：

Data Input Output	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
Number of Clock(1-1-1)	8	8	8	8	8	8	8
Write Enable	06h						
Volatile SR Write Enable	50h						
Write Disable	04h						
Release Power-down / ID	ABh	Dummy	Dummy	Dummy	(ID7-ID0)(2)		
Manufacturer/Device ID	90h	Dummy	Dummy	00h	(MF7-MF0)	(ID7-ID0)	
JEDEC ID	9Fh	(MF7-MF0)	(ID15-ID8)	(ID7-ID0)			
Read Unique ID	4Bh	Dummy	Dummy	Dummy	Dummy	Dummy	(UID63-0)
Read Data	03h	A23-A16	A15-A8	A7-A0	(D7-D0)		
Read Data with 4-Byte Address	13h	A31-A24	A23-A16	A15-A8	A7-A0	(D7-D0)	
Fast Read	0Bh	A31-A24	A23-A16	A15-A8	A7-A0	Dummy	(D7-D0)
Fast Read with 4-Byte Address	0Ch	A31-A24	A23-A16	A15-A8	A7-A0	Dummy	(D7-D0)
Page Program	02h	A31-A24	A23-A16	A15-A8	A7-A0	D7-D0	D7-D0(3)
Page Program with 4-Byte Address	12h	A31-A24	A23-A16	A15-A8	A7-A0	D7-D0	D7-D0(3)
Sector Erase (4KB)	20h	A31-A24	A23-A16	A15-A8	A7-A0		
Sector Erase (4KB) with 4-Byte Address	21h	A31-A24	A23-A16	A15-A8	A7-A0		
Block Erase (32KB)	52h	A31-A24	A23-A16	A15-A8	A7-A0		
Block Erase (64KB)	D8h	A31-A24	A23-A16	A15-A8	A7-A0		
Block Erase (64KB) with 4-Byte Address	DCh	A31-A24	A23-A16	A15-A8	A7-A0		
Chip Erase	C7h/60h						

左上角的 1-1-1 就是指令阶段使用 1 个 IO，地址阶段使用 1 个 IO，数据阶段也是使用 1 个 IO，并且采用的 4 字节地址方式，反映到程序里面就是：

```
sCommand.InstructionMode = QSPI_INSTRUCTION_1_LINE;
 sCommand.AddressMode = QSPI_ADDRESS_1_LINES;
 sCommand.AddressSize = QSPI_ADDRESS_32_BITS;
sCommand.DataMode = QSPI_DATA_1_LINES;
```

79.4.6第 6 步：W25QXX 的整个芯片擦除实现

注：这里以查询方式的 API 进行说明，DMA 方式是一样的。

整个芯片的擦除可以通过擦除各个扇区来实现，也可以调用专门的整个芯片擦除指令实现。下面实现方法是发送整个芯片擦除命令实现：

```
/*
*********************************************************************************************************
```

```
*    函 数 名: QSPI_EraseChip
*    功能说明: 整个芯片擦除
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void QSPI_EraseChip(void)
{
    QSPI_CommandTypeDef sCommand={0};

    /* 写使能 */
    QSPI_WriteEnable(&QSPIHandle);

    /* 基本配置 */
    sCommand.InstructionMode   = QSPI_INSTRUCTION_1_LINE;    /* 1 线方式发送指令 */
    sCommand.AddressSize       = QSPI_ADDRESS_32_BITS;       /* 32 位地址 */
    sCommand.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;  /* 无交替字节 */
    sCommand.DdrMode           = QSPI_DDR_MODE_DISABLE;      /* W25Q256JV 不支持 DDR */
    sCommand.DdrHoldHalfCycle  = QSPI_DDR_HHC_ANALOG_DELAY;  /* DDR 模式，数据输出延迟 */
    sCommand.SIOOMode          = QSPI_SIOO_INST_EVERY_CMD;   /* 每次传输都发指令 */

    /* 擦除配置 */
    sCommand.Instruction = BULK_ERASE_CMD;        /* 整个芯片擦除命令*/
    sCommand.AddressMode = QSPI_ADDRESS_1_LINE;   /* 地址发送是 1 线方式 */
    sCommand.Address     = 0;                     /* 地址 */
    sCommand.DataMode    = QSPI_DATA_NONE;        /* 无需发送数据 */
    sCommand.DummyCycles = 0;                     /* 无需空周期 */

    if (HAL_QSPI_Command(&QSPIHandle, &sCommand, 10000) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    QSPI_AutoPollingMemReady(&QSPIHandle);
}
```

此时函数使用的指令 0xC7 对应的 W25Q256JV 手册说明，注意红色方框位置：

Data Input Output	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
Number of Cloc(1-1-1)	8	8	8	8	8	8	8
Write Enable	06h						
Volatile SR Write Enable	50h						
Write Disable	04h						
Release Power-down / ID	ABh	Dummy	Dummy	Dummy	(ID7-ID0)(2)		
Manufacturer/Device ID	90h	Dummy	Dummy	00h	(MF7-MF0)	(ID7-ID0)	
JEDEC ID	9Fh	(MF7-MF0)	(ID15-ID8)	(ID7-ID0)			
Read Unique ID	4Bh	Dummy	Dummy	Dummy	Dummy	Dummy	(UID63-0)
Read Data	03h	A31-A24	A23-A16	A15-A8	A7-A0	(D7-D0)	
Read Data with 4-Byte Address	13h	A31-A24	A23-A16	A15-A8	A7-A0	(D7-D0)	
Fast Read	0Bh	A31-A24	A23-A16	A15-A8	A7-A0	Dummy	(D7-D0)
Fast Read with 4-Byte Address	0Ch	A31-A24	A23-A16	A15-A8	A7-A0	Dummy	(D7-D0)
Page Program	02h	A31-A24	A23-A16	A15-A8	A7-A0	D7-D0	D7-D0(3)
Page Program with 4-Byte Address	12h	A31-A24	A23-A16	A15-A8	A7-A0	D7-D0	D7-D0(3)
Sector Erase (4KB)	20h	A31-A24	A23-A16	A15-A8	A7-A0		
Sector Erase (4KB) with 4-Byte Address	21h	A31-A24	A23-A16	A15-A8	A7-A0		
Block Erase (32KB)	52h	A31-A24	A23-A16	A15-A8	A7-A0		
Block Erase (64KB)	D8h	A31-A24	A23-A16	A15-A8	A7-A0		
Block Erase (64KB) with 4-Byte Address	DCh	A31-A24	A23-A16	A15-A8	A7-A0		
Chip Erase	C7h/60h						

左上角的 1-1-1 就是指令阶段使用 1 个 IO，地址阶段使用 1 个 IO，数据阶段也是使用 1 个 IO，并且采用的 4 字节地址方式，反应到程序里面就是：

sCommand.InstructionMode = QSPI_INSTRUCTION_1_LINE;

sCommand.AddressMode = QSPI_ADDRESS_1_LINES;

sCommand.AddressSize = QSPI_ADDRESS_32_BITS;

sCommand.DataMode = QSPI_DATA_1_LINES;

擦除用不到数据阶段，sCommand.DataMode = QSPI_DATA_NONE 即可。

79.4.7第 7 步：W25QXX 内存映射实现

注：这里以查询方式的 API 进行说明，DMA 方式是一样的。

通过内存映射模式，就可以像使用内部 Flash 一样使用 W25QXX，代码实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: QSPI_MemoryMapped
*    功能说明: QSPI 内存映射，地址 0x90000000
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void QSPI_MemoryMapped(void)
{
    QSPI_CommandTypeDef s_command = {0};
    QSPI_MemoryMappedTypeDef s_mem_mapped_cfg = {0};

    /* 基本配置 */
```

```
    s_command.InstructionMode = QSPI_INSTRUCTION_1_LINE;       /* 1 线方式发送指令 */
    s_command.AddressSize = QSPI_ADDRESS_32_BITS;              /* 32 位地址 */
    s_command.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;   /* 无交替字节 */
    s_command.DdrMode = QSPI_DDR_MODE_DISABLE;                 /* W25Q256JV 不支持 DDR */
    s_command.DdrHoldHalfCycle = QSPI_DDR_HHC_ANALOG_DELAY;    /* DDR 模式，数据输出延迟 */
    s_command.SIOOMode = QSPI_SIOO_INST_EVERY_CMD;             /* 每次传输都发指令 */

    /* 全部采用 4 线 */
    s_command.Instruction = QUAD_INOUT_FAST_READ_4_BYTE_ADDR_CMD; /* 快速读取命令 */
    s_command.AddressMode = QSPI_ADDRESS_4_LINES;                 /* 4 个地址线 */
    s_command.DataMode = QSPI_DATA_4_LINES;                       /* 4 个数据线 */
    s_command.DummyCycles = 6;                                    /* 空周期 */

    /* 关闭溢出计数 */
    s_mem_mapped_cfg.TimeOutActivation = QSPI_TIMEOUT_COUNTER_DISABLE;
    s_mem_mapped_cfg.TimeOutPeriod = 0;

    if (HAL_QSPI_MemoryMapped(&QSPIHandle, &s_command, &s_mem_mapped_cfg) != HAL_OK)
    {
       Error_Handler(__FILE__, __LINE__);
    }
}
```

此时函数使用的指令 0xEC 对应的 W25Q256JV 手册说明，注意红色方框位置：

Number of Clock(1-4-4)	8	2	2	2	2	2	4	2	2
Mftr./Device ID Quad I/O	94h	A31-A24	A23-A16	A15-A8	00	Dummy(11)	Dummy	(MF7-MF0)	(ID7-ID0)
Fast Read Quad I/O	EBh	A31-A24	A23-A16	A15-A8	A7-A0	Dummy(11)	Dummy	(D7-D0)	
Fast Read Quad I/O with 4-Byte Address	ECh	A31-A24	A23-A16	A15-A8	A7-A0	Dummy(11)	Dummy	(D7-D0)	
Set Burst with Wrap	77h	Dummy	Dummy	Dummy	Dummy	W7-W0			

左上角的 1-4-4 就是指令阶段使用 1 个 IO，地址阶段使用 4 个 IO，数据阶段也是使用 4 个 IO，采用的 4 字节地址方式，反应到程序里面就是：

sCommand.InstructionMode = QSPI_INSTRUCTION_1_LINE;

sCommand.AddressMode = QSPI_ADDRESS_4_LINES;

sCommand.AddressSize = QSPI_ADDRESS_32_BITS;

sCommand.DataMode = QSPI_DATA_4_LINES;

79.4.8第 8 步：使用 MDMA 方式要注意 Cache 问题

如果使用 MDMA 方式的话，可以使用 TCM RAM，此时不用考虑 Cache 问题。如果使用的是其它 RAM 空间，要考虑 Cache 问题。因为 MDMA 和 CPU 同时访问 DMA 缓冲造成的数据一致性问题，将这块空间关闭读 Cache 和写 Cache，比如使用的 AXI SRAM，这样可以方便大家做测试，测试通过后，再根据需要开启 Cache。

```
/* 配置 AXI SRAM 的 MPU 属性为 NORMAL, NO Read allocate, NO Write allocate */
MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress      = 0x24000000;
MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
```

```
MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

79.5 W25QXX 板级支持包（bsp_qspi_w25q256.c）

W25QXX 驱动文件 bsp_qspi_w25q256.c 主要实现了如下几个 API 供用户调用：

- QSPI_ReadBuffer
- QSPI_WriteBuffer
- QSPI_EraseSector
- QSPI_EraseChip
- QSPI_MemoryMapped

79.5.1 函数 QSPI_ReadBuffer

函数原型：

```
void QSPI_ReadBuffer(uint8_t * _pBuf, uint32_t _uiReadAddr, uint32_t _uiSize);
```

函数描述：

此函数主要用于从 QSPI Flash 读取数据，支持任意大小，任意地址，不超过芯片容量即可（如果使用 DMA 方式，每次最大 65536 字节）。

函数参数：

- 第 1 个参数用于存储从 QSPI Flash 读取的数据。
- 第 2 个参数是读取地址，不可以超过芯片容量。
- 第 3 个参数是读取的数据大小，读取范围不可以超过芯片容量。

79.5.2 函数 QSPI_WriteBuffer

函数原型：

```
uint8_t QSPI_WriteBuffer(uint8_t *_pBuf, uint32_t _uiWriteAddr, uint16_t _usWriteSize);
```

函数描述：

页编程，页大小 256 字节，任意页都可以写入。注意使用前，务必保证相应页已经做了擦除操作。

函数参数：

- 第 1 个参数是源数据缓冲区。

- 第 2 个参数是目标区域首地址，即页首地址，比如 0， 256, 512 等。
- 第 3 个参数是数据个数，不能超过页面大小，范围 1 – 256，单位字节个数。
- 返回值，返回 1 表示成功，返回 0 表示失败。

79.5.3 函数 QSPI_EraseSector

函数原型：

```
void QSPI_EraseSector(uint32_t _uiSectorAddr)
```

函数描述：

此函数主要用于扇区擦除，一个扇区大小是 4KB。

函数参数：

- 第 1 个参数是扇区地址，比如擦除扇区 0，此处填 0x0000，擦除扇区 1，此处填 0x1000，擦除扇区 2，此处填 0x2000，以此类推。

79.5.4 函数 QSPI_EraseChip

函数原型：

```
void QSPI_EraseChip(void)
```

函数描述：

此函数主要用于整个芯片擦除。

79.5.5 函数 QSPI_MemoryMapped

函数原型：

```
void QSPI_MemoryMapped(void)
```

函数描述：

调用了此函数就可以像使用内部 Flash 一样使用外部 Flash。

79.6 W25QXX 驱动移植和使用

W25QXX 移植步骤如下：

- 第 1 步：复制 bsp_qspi_w25q256.c，bsp_qspi_w25q256.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：根据使用的 QSPI 引脚，时钟等，修改 bsp_qspi_w25q256.c 文件开头的宏定义。

```
/*
    STM32-V7 开发板接线

    PG6/QUADSPI_BK1_NCS AF10
    PF10/QUADSPI_CLK   AF9
    PF8/QUADSPI_BK1_IO0 AF10
```

```
    PF9/QUADSPI_BK1_IO1 AF10
    PF7/QUADSPI_BK1_IO2 AF9
    PF6/QUADSPI_BK1_IO3 AF9

    W25Q256JV 有 512 块，每块有 16 个扇区，每个扇区 Sector 有 16 页，每页有 256 字节，共计 32MB
*/

/* QSPI 引脚和时钟相关配置宏定义 */
#define QSPI_CLK_ENABLE()               __HAL_RCC_QSPI_CLK_ENABLE()
#define QSPI_CLK_DISABLE()              __HAL_RCC_QSPI_CLK_DISABLE()
#define QSPI_CS_GPIO_CLK_ENABLE()       __HAL_RCC_GPIOG_CLK_ENABLE()
#define QSPI_CLK_GPIO_CLK_ENABLE()      __HAL_RCC_GPIOF_CLK_ENABLE()
#define QSPI_BK1_D0_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOF_CLK_ENABLE()
#define QSPI_BK1_D1_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOF_CLK_ENABLE()
#define QSPI_BK1_D2_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOF_CLK_ENABLE()
#define QSPI_BK1_D3_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOF_CLK_ENABLE()

#define QSPI_MDMA_CLK_ENABLE()          __HAL_RCC_MDMA_CLK_ENABLE()
#define QSPI_FORCE_RESET()              __HAL_RCC_QSPI_FORCE_RESET()
#define QSPI_RELEASE_RESET()            __HAL_RCC_QSPI_RELEASE_RESET()

#define QSPI_CS_PIN                     GPIO_PIN_6
#define QSPI_CS_GPIO_PORT               GPIOG

#define QSPI_CLK_PIN                    GPIO_PIN_10
#define QSPI_CLK_GPIO_PORT              GPIOF

#define QSPI_BK1_D0_PIN                 GPIO_PIN_8
#define QSPI_BK1_D0_GPIO_PORT           GPIOF

#define QSPI_BK1_D1_PIN                 GPIO_PIN_9
#define QSPI_BK1_D1_GPIO_PORT           GPIOF

#define QSPI_BK1_D2_PIN                 GPIO_PIN_7
#define QSPI_BK1_D2_GPIO_PORT           GPIOF

#define QSPI_BK1_D3_PIN                 GPIO_PIN_6
#define QSPI_BK1_D3_GPIO_PORT           GPIOF
```

◆ 根据使用的 QSPI 命令不同，容量不同等，修改 bsp_qspi_w25q256.h 头文件

```
/* W25Q256JV 基本信息 */
#define QSPI_FLASH_SIZE      25                     /* Flash 大小，2^25 = 32MB*/
#define QSPI_SECTOR_SIZE     (4 * 1024)             /* 扇区大小，4KB */
#define QSPI_PAGE_SIZE       256                        /* 页大小，256 字节 */
#define QSPI_END_ADDR        (1 << QSPI_FLASH_SIZE) /* 末尾地址 */
#define QSPI_FLASH_SIZES     32*1024*1024           /* Flash 大小，2^25 = 32MB*/

/* W25Q256JV 相关命令 */
#define WRITE_ENABLE_CMD       0x06          /* 写使能指令 */
#define READ_ID_CMD2           0x9F          /* 读取 ID 命令 */
#define READ_STATUS_REG_CMD    0x05          /* 读取状态命令 */
#define BULK_ERASE_CMD         0xC7          /* 整个芯片擦除命令 */
#define SUBSECTOR_ERASE_4_BYTE_ADDR_CMD      0x21     /* 32bit 地址扇区擦除指令，4KB */
#define QUAD_IN_FAST_PROG_4_BYTE_ADDR_CMD    0x34     /* 32bit 地址的 4 线快速写入命令 */
#define QUAD_INOUT_FAST_READ_4_BYTE_ADDR_CMD 0xEC     /* 32bit 地址的 4 线快速读取命令 */
```

- 第 7 步：如果使用 MDMA 方式的话，可以使用 TCM RAM，此时不用考虑 Cache 问题。如果使用的是其它 RAM 空间，要考虑 Cache 问题。因为 MDMA 和 CPU 同时访问 DMA 缓冲造成的数据一致性问题，将这块空间关闭读 Cache 和写 Cache，比如使用的 AXI SRAM：

```
/* 配置 AXI SRAM 的 MPU 属性为 NORMAL, NO Read allocate, NO Write allocate */
MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress      = 0x24000000;
MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

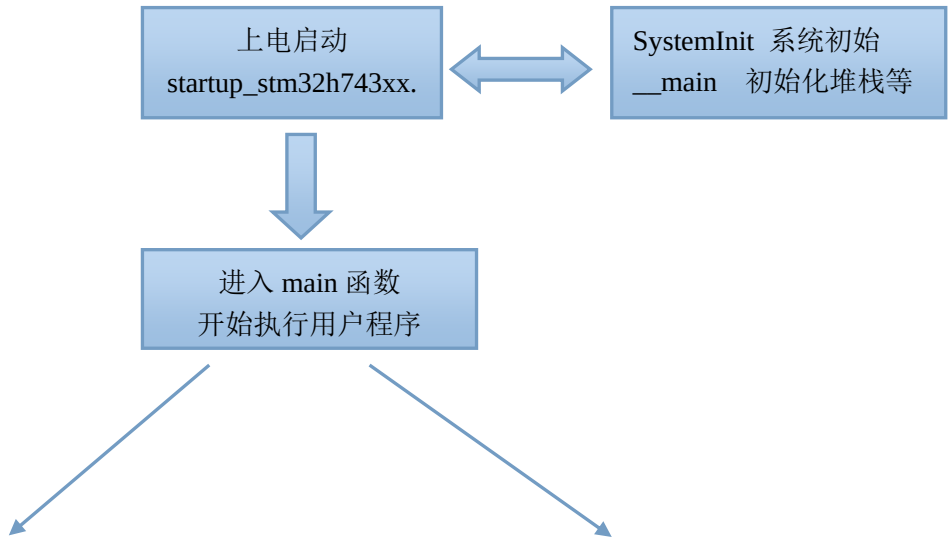
- 第 8 步：初始化 QSPI。

```
/* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
bsp_InitQSPI_W25Q256();  /* 配置 SPI 总线 */
```

- 第 9 步：QSPI Flash 驱动主要用到 HAL 库的 SPI 驱动文件，简单省事些可以添加所有 HAL 库 C 源文件进来。
- 第 10 步：应用方法看本章节配套例子即可。

79.7 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



第 1 步 bsp_Init 初始化所有的硬件设备	第 2 步 实现 QSPI Flash 的中断，查询和 MDMA 方式操作

- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器和 LED。
  - 第 2 部分，应用程序设计部分，实现 QSPI Flash 的查询和 MDMA 方式操作。

79.8 实验例程说明（MDK）

配套例子：

V7-029_QSPI 读写例程，四线 DMA 方式，读每秒 48MB（V1.1）

V7-059_QSPI 读写例程，查询方式

实验目的：

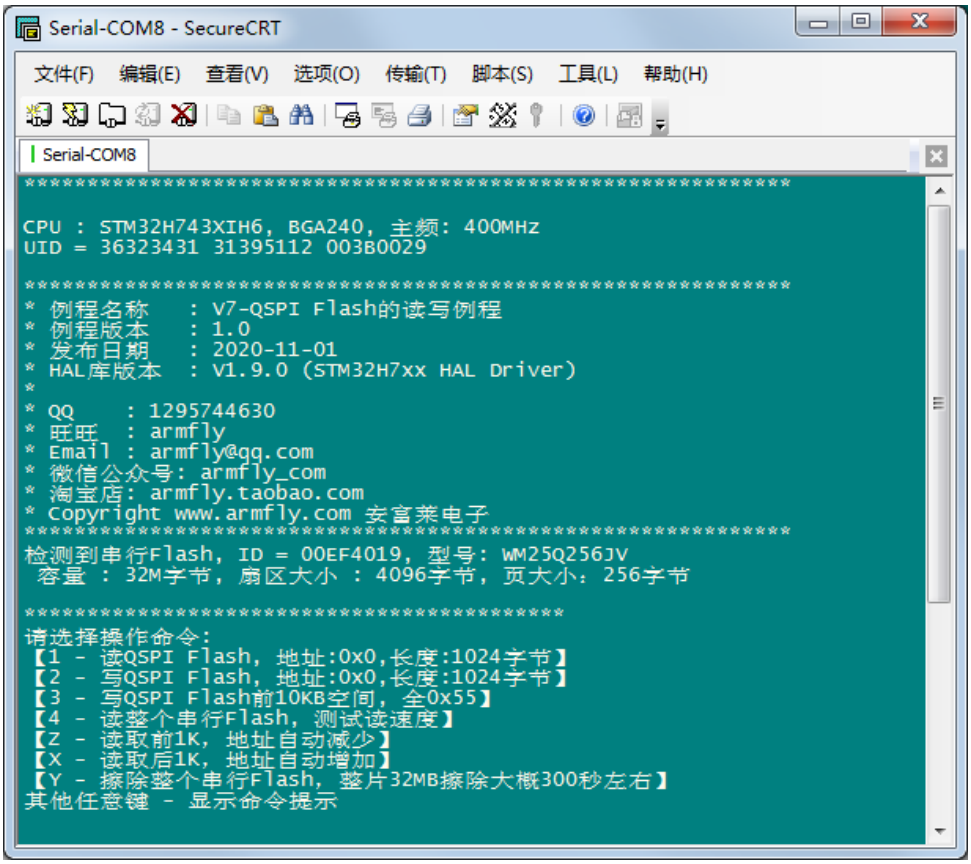
2. 学习 QSPI Flash 的读写测试例程

实验操作：

11. 支持以下 7 个功能，用户通过电脑端串口软件发送数字 1-6 给开发板即可
12. printf("请选择操作命令:\r\n");
13. printf("【1 - 读 QSPI Flash，地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
14. printf("【2 - 写 QSPI Flash，地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
15. printf("【3 - 写 QSPI Flash 前 10KB 空间，全 0x55】\r\n");
16. printf("【4 - 读整个串行 Flash，测试读速度】\r\n");
17. printf("【Z - 读取前 1K，地址自动减少】\r\n");
18. printf("【X - 读取后 1K，地址自动增加】\r\n");
19. printf("【Y - 擦除整个串行 Flash，整片 32MB 擦除大概 300 秒左右】\r\n");
20. printf("其他任意键 - 显示命令提示\r\n");

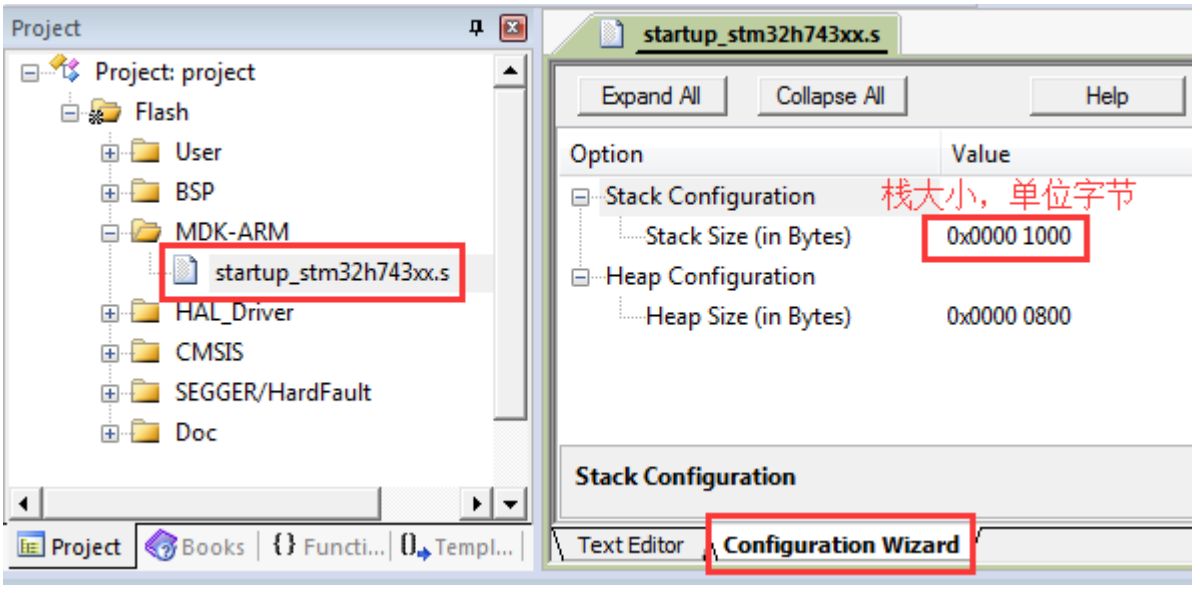
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1。

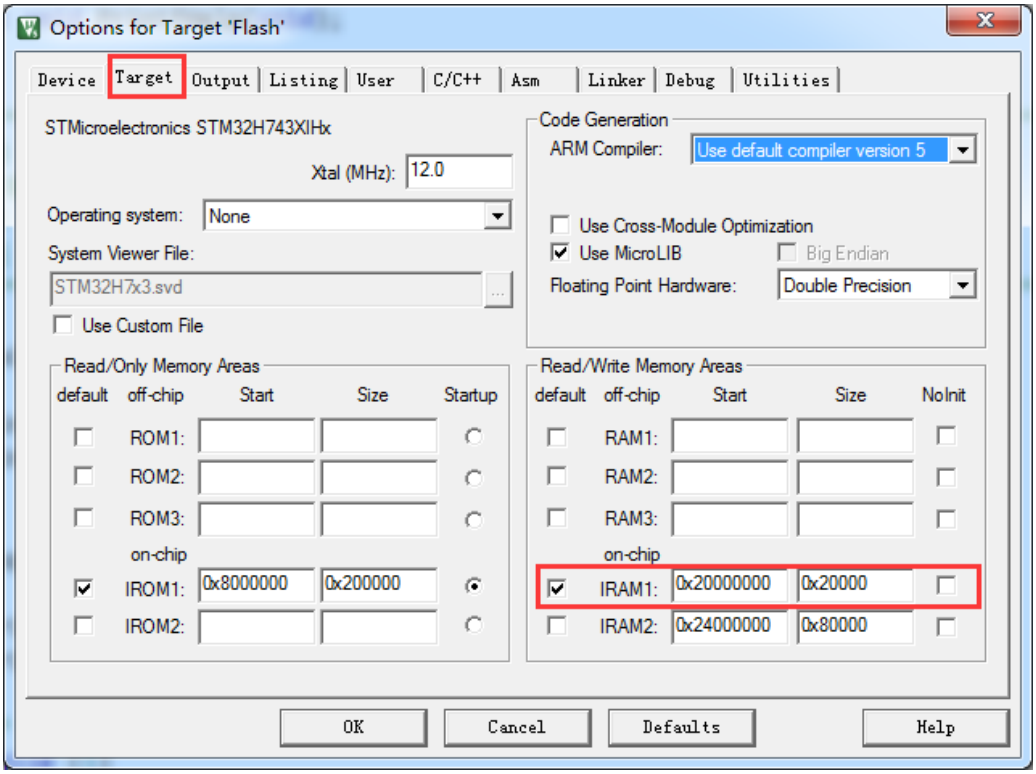


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();      /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();    /* 初始化滴答定时器 */
    bsp_InitLPUart();   /* 初始化串口 */
    bsp_InitExtIO();    /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();      /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitQSPI_W25Q256();  /* 配置 SPI 总线 */}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

#if 0
    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

```
#else
    /* 配置 AXI SRAM 的 MPU 属性为 NORMAL, NO Read allocate, NO Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
#endif

    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **每 10ms 调用一次按键处理：**

  按键处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
```

```
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能:**

主程序实现如下操作:

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- 支持以下 7 个功能，用户通过电脑端串口软件发送命令给开发板即可
- printf("【1 - 读 QSPI Flash，地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
- printf("【2 - 写 QSPI Flash，地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
- printf("【3 - 写 QSPI Flash 前 10KB 空间，全 0x55】\r\n");
- printf("【4 - 读整个串行 Flash，测试读速度】\r\n");
- printf("【Z - 读取前 1K，地址自动减少】\r\n");
- printf("【X - 读取后 1K，地址自动增加】\r\n");
- printf("【Y - 擦除整个串行 Flash，整片 32MB 擦除大概 300 秒左右】\r\n");
- printf("其他任意键 - 显示命令提示\r\n");
-

```
/*
*********************************************************************************************************
*    函 数 名: DemoSpiFlash
*    功能说明: QSPI 读写例程
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoSpiFlash(void)
{
    uint8_t cmd;
    uint32_t uiReadPageNo = 0, id;

    /* 检测串行 Flash OK */
    id = QSPI_ReadID();
    printf("检测到串行 Flash, ID = %08X, 型号: WM25Q256JV\r\n", id);
    printf(" 容量 : 32M 字节, 扇区大小 : 4096 字节, 页大小：256 字节\r\n");

    sfDispMenu();      /* 打印命令提示 */

   bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */
    while(1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */
```

```
    /* 判断软件定时器 0 是否超时 */
    if(bsp_CheckTimer(0))
    {
        /* 每隔 200ms 进来一次 */
        bsp_LedToggle(2);
    }

     if (comGetChar(COM1, &cmd)) /* 从串口读入一个字符(非阻塞方式) */
     {
          switch (cmd)
          {
               case '1':
                    printf("\r\n【1 - 读 QSPI Flash, 地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
                    sfReadTest(); /* 读串行 Flash 数据，并打印出来数据内容 */
                    break;

               case '2':
                    printf("\r\n【2 - 写 QSPFlash, 地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
                    sfWriteTest();     /* 写串行 Flash 数据，并打印写入速度 */
                    break;

               case '3':
                    printf("\r\n【3 - 写 QSPI Flash 前 10KB 空间, 全 0x55】\r\n");
                    sfWriteAll(0x55);/* 擦除串行 Flash 数据，实际上就是写入全 0xFF */
                    break;

               case '4':
                    printf("\r\n【4 - 读整个 QSPI Flash, %dM 字节】\r\n", QSPI_FLASH_SIZES/(1024*1024));
                    sfTestReadSpeed(); /* 读整个串行 Flash 数据，测试速度 */
                    break;

               case 'y':
               case 'Y':
                    printf("\r\n【Y - 擦除整个 QSPI Flash】\r\n");
                    printf("整个 Flash 擦除完毕大概需要 300 秒左右，请耐心等待");
                    sfErase();         /* 擦除串行 Flash 数据，实际上就是写入全 0xFF */
                    break;

               case 'z':
               case 'Z': /* 读取前 1K */
                    if (uiReadPageNo > 0)
                    {
                         uiReadPageNo--;
                    }
                    else
                    {
                         printf("已经是最前\r\n");
                    }
                    sfViewData(uiReadPageNo * 1024);
                    break;

               case 'x':
               case 'X': /* 读取后 1K */
                    if (uiReadPageNo < QSPI_FLASH_SIZES / 1024 - 1)
                    {
                         uiReadPageNo++;
                    }
                    else
```

```
                        {
                            printf("已经是最后\r\n");
                        }
                        sfViewData(uiReadPageNo * 1024);
                        break;

                    default:
                        sfDispMenu(); /* 无效命令，重新打印命令提示 */
                        break;
                }
            }
        }
}
```

79.9 实验例程说明（IAR）

配套例子：

V7-029_QSPI 读写例程，四线 DMA 方式，读每秒 48MB（V1.1）

V7-059_QSPI 读写例程，查询方式

实验目的：

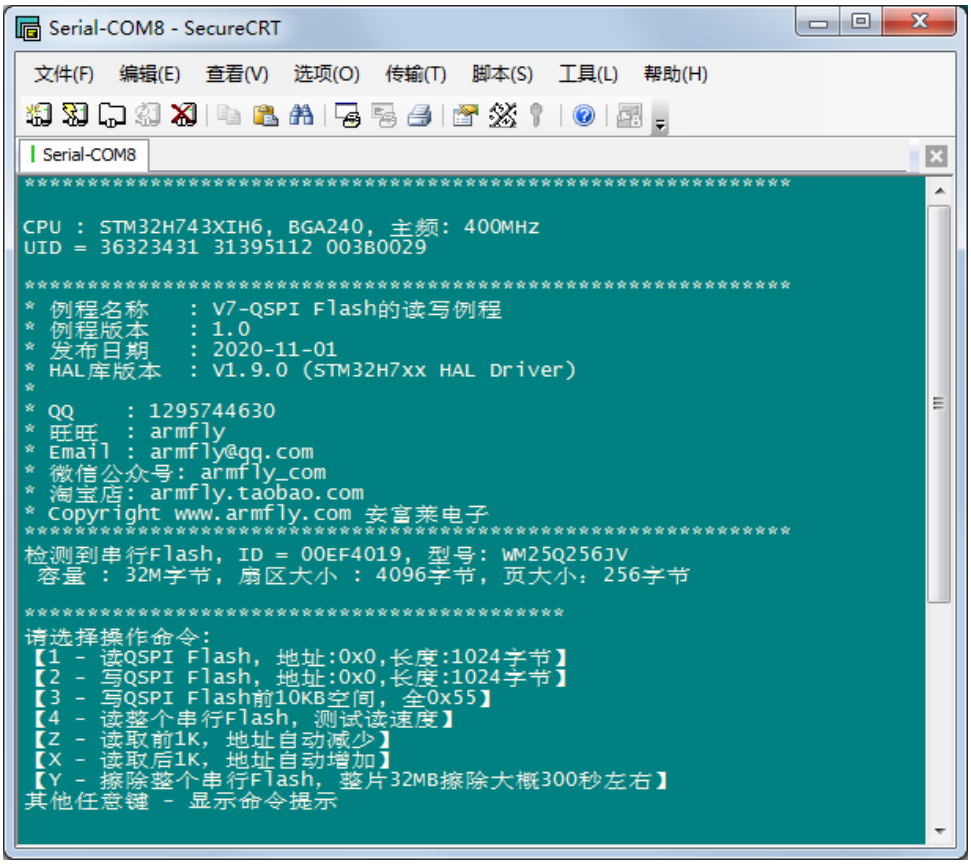
1. 学习 QSPI Flash 的读写测试例程

实验操作：

1. 支持以下 7 个功能，用户通过电脑端串口软件发送数字 1-6 给开发板即可
2. printf("请选择操作命令:\r\n");
3. printf("【1 - 读 QSPI Flash，地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
4. printf("【2 - 写 QSPI Flash，地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
5. printf("【3 - 写 QSPI Flash 前 10KB 空间，全 0x55】\r\n");
6. printf("【4 - 读整个串行 Flash，测试读速度】\r\n");
7. printf("【Z - 读取前 1K，地址自动减少】\r\n");
8. printf("【X - 读取后 1K，地址自动增加】\r\n");
9. printf("【Y - 擦除整个串行 Flash，整片 32MB 擦除大概 300 秒左右】\r\n");
10. printf("其他任意键 - 显示命令提示\r\n");

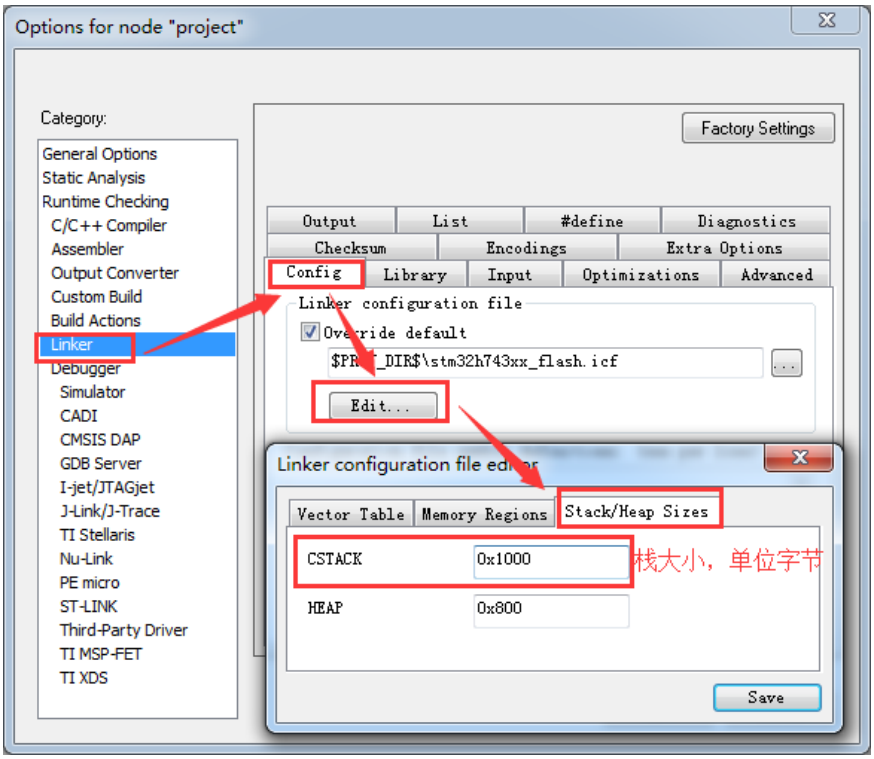
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1。

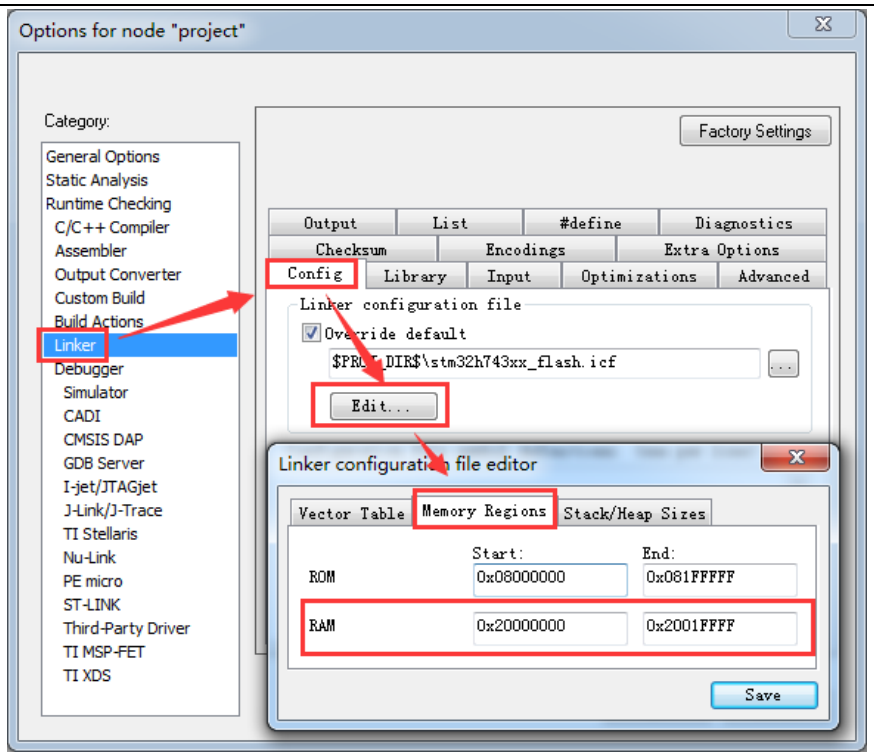


程序设计：

◆ 系统栈大小分配：



◆ RAM 空间用的 DTCM：



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();      /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();    /* 初始化滴答定时器 */
    bsp_InitLPUart();   /* 初始化串口 */
    bsp_InitExtIO();    /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();      /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitQSPI_W25Q256();  /* 配置 SPI 总线 */}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

#if 0
    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

 #else
    /* 配置 AXI SRAM 的 MPU 属性为 NORMAL, NO Read allocate, NO Write allocate */
```

```
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
#endif

    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **每 10ms 调用一次按键处理：**

  按键处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
```

```
*                不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能:**

主程序实现如下操作:

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- 支持以下 7 个功能，用户通过电脑端串口软件发送命令给开发板即可
- printf("【1 - 读 QSPI Flash，地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
- printf("【2 - 写 QSPI Flash，地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
- printf("【3 - 写 QSPI Flash 前 10KB 空间，全 0x55】\r\n");
- printf("【4 - 读整个串行 Flash，测试读速度】\r\n");
- printf("【Z - 读取前 1K，地址自动减少】\r\n");
- printf("【X - 读取后 1K，地址自动增加】\r\n");
- printf("【Y - 擦除整个串行 Flash，整片 32MB 擦除大概 300 秒左右】\r\n");
- printf("其他任意键 - 显示命令提示\r\n");
-

```
/*
*********************************************************************************************************
*    函 数 名: DemoSpiFlash
*    功能说明: QSPI 读写例程
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoSpiFlash(void)
{
    uint8_t cmd;
    uint32_t uiReadPageNo = 0, id;

    /* 检测串行 Flash OK */
    id = QSPI_ReadID();
    printf("检测到串行 Flash, ID = %08X, 型号: WM25Q256JV\r\n", id);
    printf(" 容量 : 32M 字节, 扇区大小 : 4096 字节, 页大小：256 字节\r\n");

    sfDispMenu();       /* 打印命令提示 */

   bsp_StartAutoTimer(0, 200); /* 启动 1 个 200ms 的自动重装的定时器，软件定时器 0 */
    while(1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

       /* 判断软件定时器 0 是否超时 */
       if(bsp_CheckTimer(0))
```

```
    {
        /* 每隔 200ms 进来一次 */
        bsp_LedToggle(2);
    }

    if (comGetChar(COM1, &cmd)) /* 从串口读入一个字符(非阻塞方式) */
    {
        switch (cmd)
        {
            case '1':
                printf("\r\n【1 - 读 QSPI Flash, 地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
                sfReadTest(); /* 读串行 Flash 数据，并打印出来数据内容 */
                break;

            case '2':
                printf("\r\n【2 - 写 QSPFlash, 地址:0x%X,长度:%d 字节】\r\n", TEST_ADDR, TEST_SIZE);
                sfWriteTest();     /* 写串行 Flash 数据，并打印写入速度 */
                break;

            case '3':
                printf("\r\n【3 - 写 QSPI Flash 前 10KB 空间, 全 0x55】\r\n");
                sfWriteAll(0x55);/* 擦除串行 Flash 数据，实际上就是写入全 0xFF */
                break;

            case '4':
                printf("\r\n【4 - 读整个 QSPI Flash, %dM 字节】\r\n", QSPI_FLASH_SIZES/(1024*1024));
                sfTestReadSpeed(); /* 读整个串行 Flash 数据，测试速度 */
                break;

            case 'y':
            case 'Y':
                printf("\r\n【Y - 擦除整个 QSPI Flash】\r\n");
                printf("整个 Flash 擦除完毕大概需要 300 秒左右，请耐心等待");
                sfErase();          /* 擦除串行 Flash 数据，实际上就是写入全 0xFF */
                break;

            case 'z':
            case 'Z': /* 读取前 1K */
                if (uiReadPageNo > 0)
                {
                    uiReadPageNo--;
                }
                else
                {
                    printf("已经是最前\r\n");
                }
                sfViewData(uiReadPageNo * 1024);
                break;

            case 'x':
            case 'X': /* 读取后 1K */
                if (uiReadPageNo < QSPI_FLASH_SIZES / 1024 - 1)
                {
                    uiReadPageNo++;
                }
                else
                {
                    printf("已经是最后\r\n");
```

```
                        }
                        sfViewData(uiReadPageNo * 1024);
                        break;

                    default:
                        sfDispMenu(); /* 无效命令，重新打印命令提示 */
                        break;
                }
            }
        }
}
```

79.10 总结

本章节就为大家讲解这么多，实际应用中根据需要选择 DMA 和查询方式。

第80章 STM32H7 的 QSPI 总线应用之 QSPI Flash 的 MDK 下载算法制作

本章节为大家讲解 MDK 下载算法制作方法。

80.1 初学者重要提示
80.2 MDK 下载算法基础知识
80.3 创建 MDK 下载算法通用流程
80.4 QSPI Flash 的 MDK 下载算法制作
80.5 QSPI Flash 的 MDK 下载算法使用方法
80.6 实验例程说明
80.7 总结

80.1 初学者重要提示

- QSPI Flash 的相关知识点可以看第 78 章和 79 章。
- QSPI Flash 下载算法文件直接采用 HAL 库制作，方便大家自己修改。

80.2 MDK 下载算法基础知识

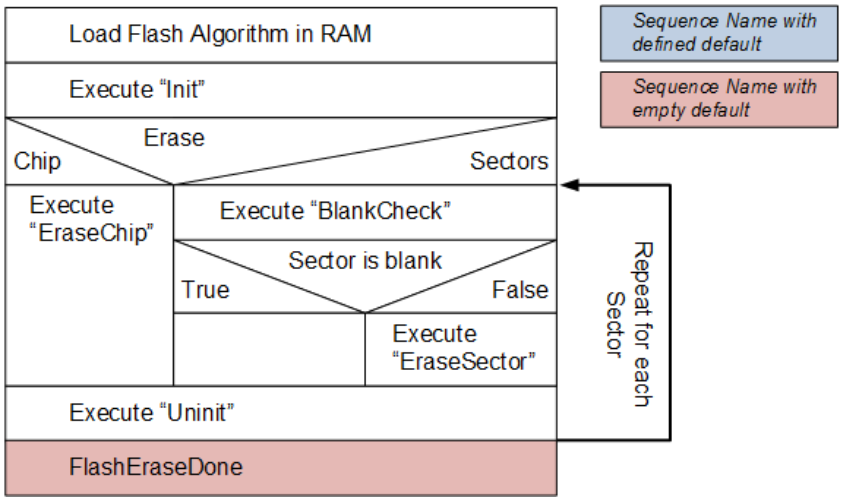
Flash 编程算法是一种用于擦除应用程序或将应用程序下载到 Flash 的程序代码。MDK 本身支持的各种器件都自带下载算法，存放在 MDK 各种器件的软件包里面，以 STM32H7 为例，算法存放在\Keil\STM32H7xx_DFP\2.6.0\CMSIS\Flash（软件包版本不同，数值 2.6.0 不同），但不支持的需要我们自己制作，本章教程为此而生。

80.2.1 程序能够通过下载算法下载到芯片的核心思想

认识到这点很重要：通过 MDK 创建一批与地址信息无关的函数，实现的功能主要有初始化，擦除，编程，读取，校验等，然后 MDK 调试下载阶段，会将算法文件加载到芯片的内部 RAM 里面（加载地址可以通过 MDK 设置），然后 MDK 通过与这个算法文件的交互，实现程序下载，调试阶段数据读取等操作。

80.2.2 算法程序中擦除操作执行流程

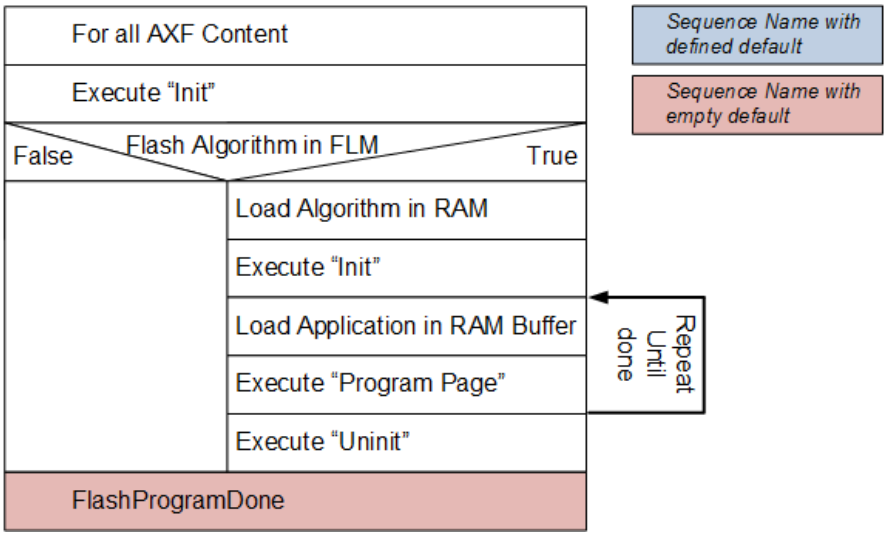
擦除操作大致流程：



- 加载算法到芯片 RAM。
- 执行初始化函数 Init。
- 执行擦除操作，根据用户的 MDK 配置，这里可以选择整个芯片擦除或者扇区擦除。
- 执行 Uinit 函数。
- 操作完毕。

80.2.3 算法程序中编程操作执行流程

编程操作大致流程：



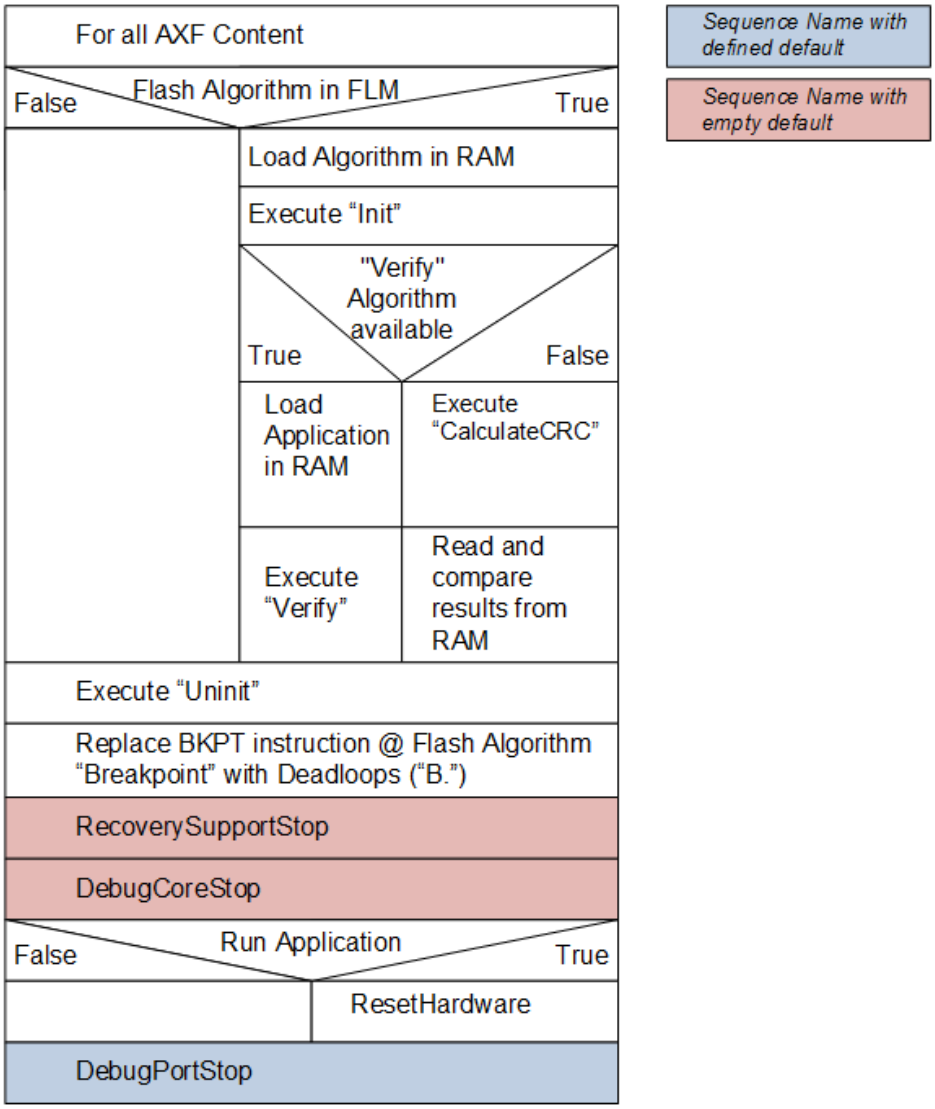
- 针对 MDK 生成的 axf 可执行文件做 Init 初始化，这个 axf 文件是指的大家自己创建应用程序生成的。
- 查看 Flash 算法是否在 FLM 文件。如果没有在，操作失败。如果在：
  - 加载算法到 RAM。

- 执行 Init 函数。
- 加载用户到 RAM 缓冲。
- 执行 Program Page 页编程函数。
- 执行 Uninit 函数。

◆ 操作完毕。

80.2.4算法程序中校验操作执行流程

校验操作大致流程：



◆ 校验要用到 MDK 生成的 axf 可执行文件。校验就是 axf 文件中下载到芯片的程序和实际下载的程序读出来做比较。

◆ 查看 Flash 算法是否在 FLM 文件。如果没有在，操作失败。如果在：

- 加载算法到 RAM。

- 执行 Init 函数。
- 查看校验算法是否存在
  - 如果有，加载应用程序到 RAM 并执行校验。
  - 如果没有，计算 CRC，将芯片中读取出来的数据和 RAM 中加载应用计算输出的 CRC 值做比较。
- 执行 Uninit 函数。
- 替换 BKPT（BreakPoint 断点指令）为 B. 死循环指令。
- 执行 RecoverySupportStop，恢复支持停止。
- 执行 DebugCoreStop，调试内核停止。

◆ 运行应用：

- 执行失败。
- 执行成功，再执行硬件复位。

◆ 操作完毕，停止调试端口。

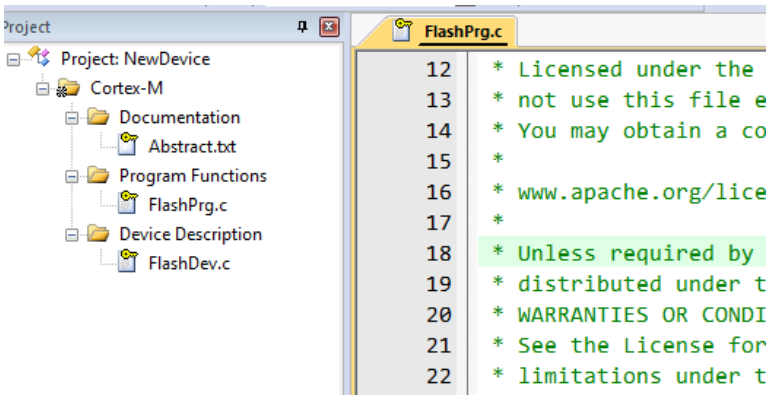
80.3 创建 MDK 下载算法通用流程

下面是 MDK 给的一种大致操作流程，不限制必须采用这种方法，自己创建也可以的。

80.3.1 第 1 步，使用 MDK 提供好的程序模板

位于路径：\Keil\ARM\Pack\ARM\CMSIS\version\Device_Template_Flash。

效果如下：

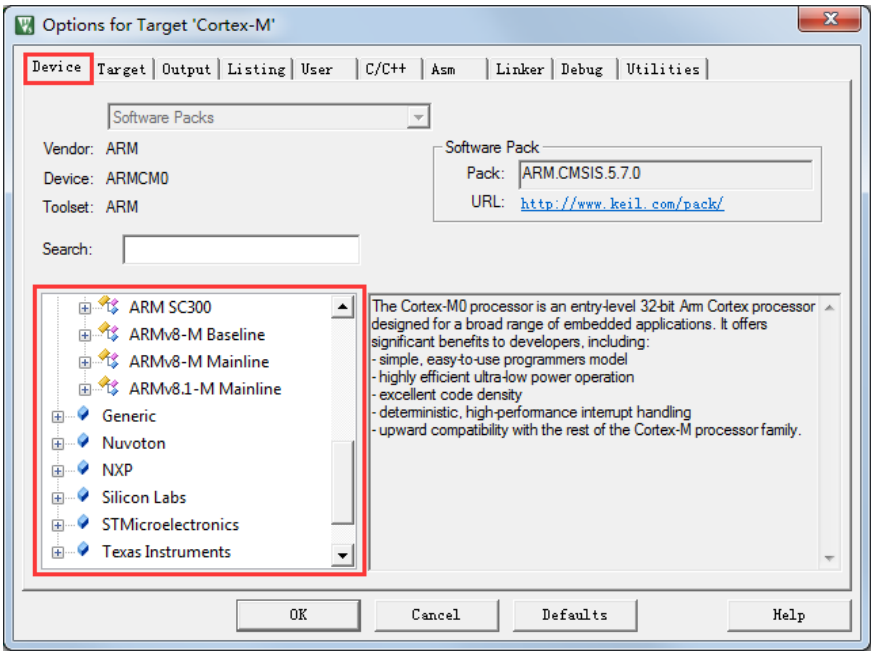


80.3.2 第 2 步，修改工程名

MDK 提供的工程模板原始名字是 NewDevice.uvprojx，大家可以根据自己的需要做修改。比如修改为 MyDevice.uvprojx。

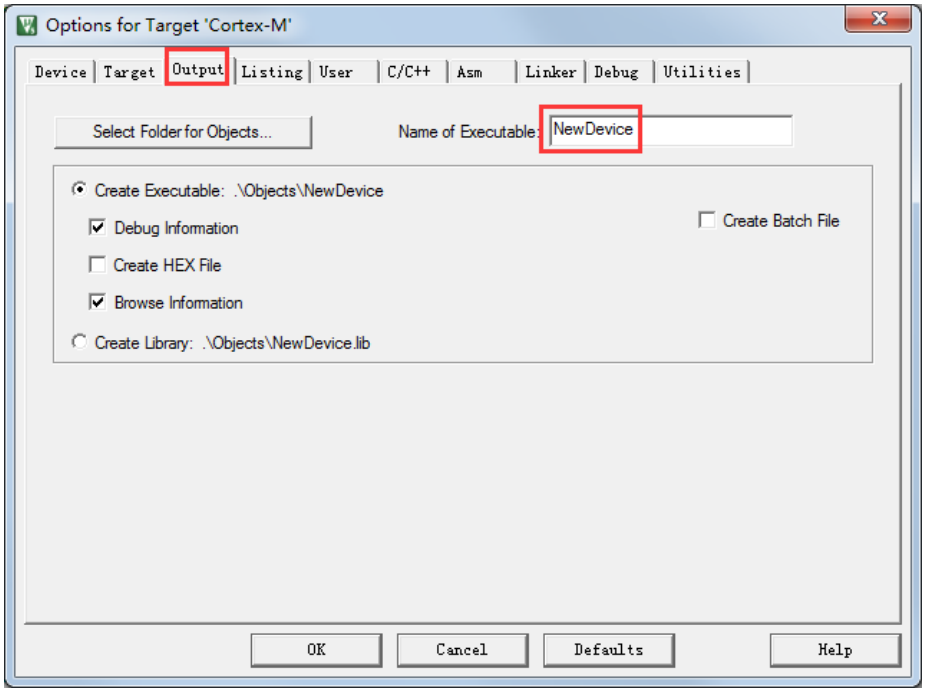
80.3.3第 3 步，修改使用的器件

在 MDK 的 Option 选项里面设置使用的器件。

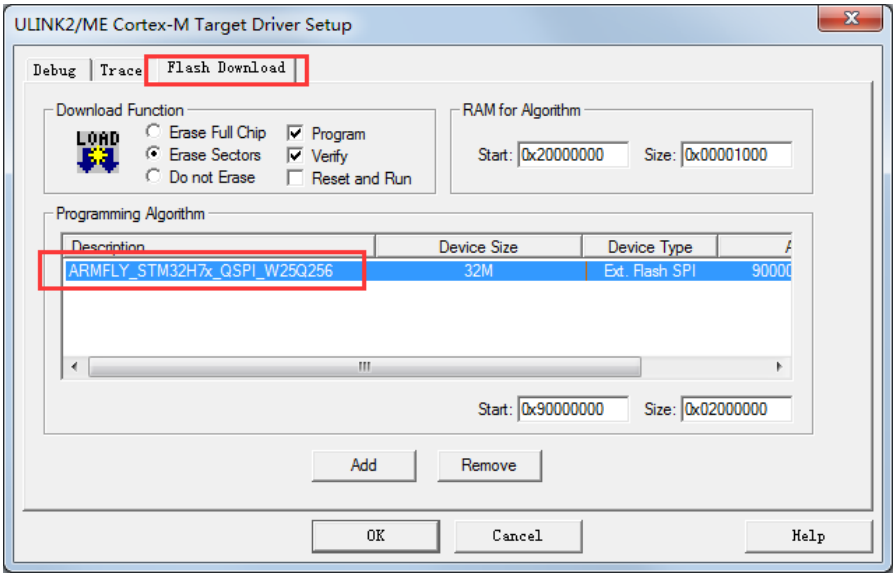


80.3.4第 4 步，修改输出算法文件的名字

这个名字是方便用户查看的，比如设置为 stm32h7，那么输出的算法文件就是 stm32h7.flm。



注：MDK 这里设置的名字与下面位置识别出来的算法名无关：



这个名字是在 FlashDev.c 里面定义的。

80.3.5第 5 步，修改编程算法文件 FlashPrg.c

模板工程里面仅提供了接口函数，内容需要用户自己填。

```
/*
   Mandatory Flash Programming Functions (Called by FlashOS):
               int Init        (unsigned long adr,   // Initialize Flash
                                unsigned long clk,
                                unsigned long fnc);
               int UnInit      (unsigned long fnc);  // De-initialize Flash
               int EraseSector (unsigned long adr);  // Erase Sector Function
               int ProgramPage (unsigned long adr,   // Program Page Function
                                unsigned long sz,
                                unsigned char *buf);

   Optional  Flash Programming Functions (Called by FlashOS):
               int BlankCheck  (unsigned long adr,   // Blank Check
                                unsigned long sz,
                                unsigned char pat);
               int EraseChip   (void);               // Erase complete Device
      unsigned long Verify     (unsigned long adr,   // Verify Function
                                unsigned long sz,
                                unsigned char *buf);

       - BlanckCheck  is necessary if Flash space is not mapped into CPU memory space
       - Verify       is necessary if Flash space is not mapped into CPU memory space
       - if EraseChip is not provided than EraseSector for all sectors is called
*/

/*
 *  Initialize Flash Programming Functions
 *    Parameter:      adr:  Device Base Address
 *                    clk:  Clock Frequency (Hz)
 *                    fnc:  Function Code (1 - Erase, 2 - Program, 3 - Verify)
```

```
 *    Return Value:   0 - OK,  1 - Failed
 */

int Init (unsigned long adr, unsigned long clk, unsigned long fnc) {

  /* Add your Code */
  return (0);                                  // Finished without Errors
}


/*
 *  De-Initialize Flash Programming Functions
 *    Parameter:      fnc:  Function Code (1 - Erase, 2 - Program, 3 - Verify)
 *    Return Value:   0 - OK,  1 - Failed
 */

int UnInit (unsigned long fnc) {

  /* Add your Code */
  return (0);                                  // Finished without Errors
}


/*
 *  Erase complete Flash Memory
 *    Return Value:   0 - OK,  1 - Failed
 */

int EraseChip (void) {

  /* Add your Code */
  return (0);                                  // Finished without Errors
}


/*
 *  Erase Sector in Flash Memory
 *    Parameter:      adr:  Sector Address
 *    Return Value:   0 - OK,  1 - Failed
 */

int EraseSector (unsigned long adr) {

  /* Add your Code */
  return (0);                                  // Finished without Errors
}


/*
 *  Program Page in Flash Memory
 *    Parameter:      adr:  Page Start Address
 *                    sz:   Page Size
 *                    buf:  Page Data
 *    Return Value:   0 - OK,  1 - Failed
 */

int ProgramPage (unsigned long adr, unsigned long sz, unsigned char *buf) {
```

```
  /* Add your Code */
  return (0);                                  // Finished without Errors
}
```

80.3.6第 6 步，修改配置文件 FlashDev.c

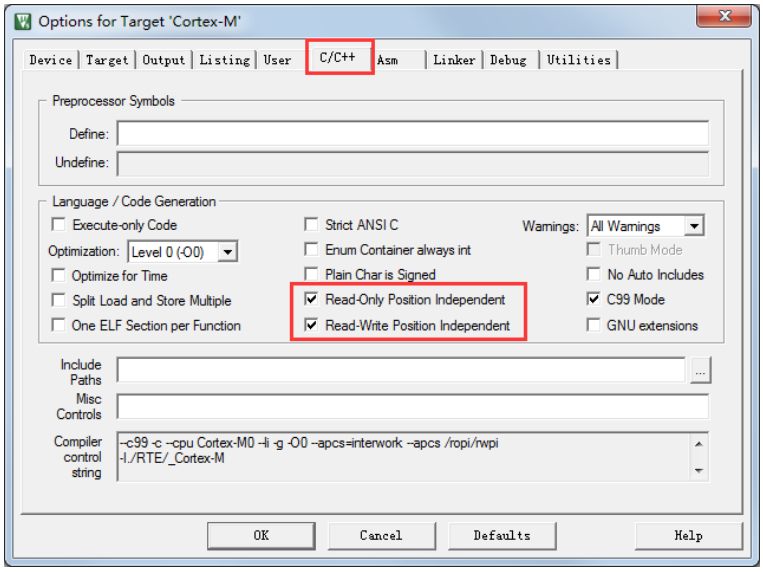
模板工程里面提供简单的配置说明：

```
struct FlashDevice const FlashDevice  =  {
   FLASH_DRV_VERS,             // Driver Version, do not modify!
   "New Device 256kB Flash",   // Device Name
   ONCHIP,                     // Device Type
   0x00000000,                 // Device Start Address
   0x00040000,                 // Device Size in Bytes (256kB)
   1024,                       // Programming Page Size
   0,                          // Reserved, must be 0
   0xFF,                       // Initial Content of Erased Memory
   100,                        // Program Page Timeout 100 mSec
   3000,                       // Erase Sector Timeout 3000 mSec

// Specify Size and Address of Sectors
   0x002000, 0x000000,         // Sector Size  8kB (8 Sectors)
   0x010000, 0x010000,         // Sector Size 64kB (2 Sectors)
   0x002000, 0x030000,         // Sector Size  8kB (8 Sectors)
   SECTOR_END
};
```

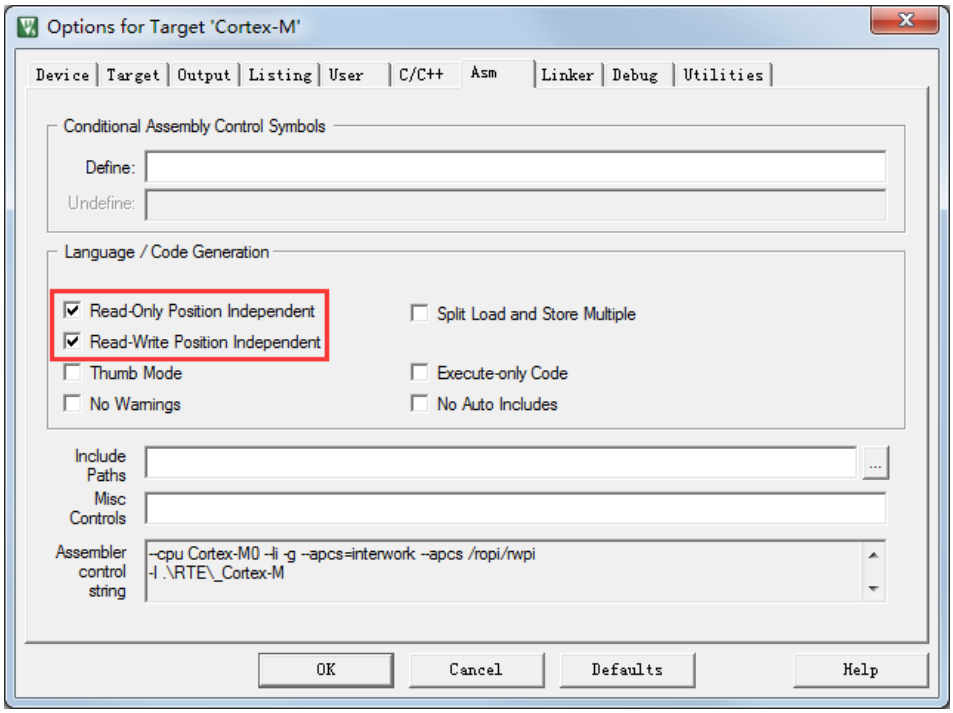
注：名字 New Device 256kB Flash 就是我们第 4 步所说的。MDK 的 Option 选项里面会识别出这个名字。

80.3.7第 7 步，保证生成的算法文件中 RO 和 RW 段的独立性，即与地址无关

C 和汇编的配置都勾选上：



汇编：



如果程序的所有只读段都与位置无关，则该程序为只读位置无关（ROPI, Read-only position independence）。ROPI 段通常是位置无关代码（PIC,position-independent code），但可以是只读数据，也可以是 PIC 和只读数据的组合。选择“ ROPI”选项，可以避免用户不得不将代码加载到内存中的特定位置。这对于以下例程特别有用：

（1）加载以响应运行事件。

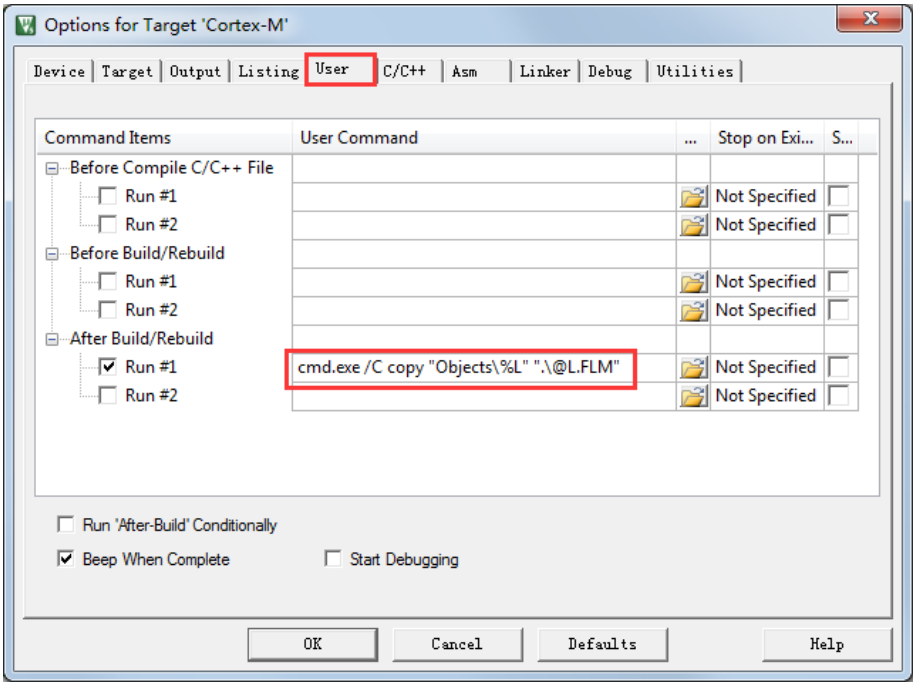
（2）在不同情况下使用其他例程的不同组合加载到内存中。

（3）在执行期间映射到不同的地址。

使用 Read-Write position independence 同理，表示的可读可写数据段。

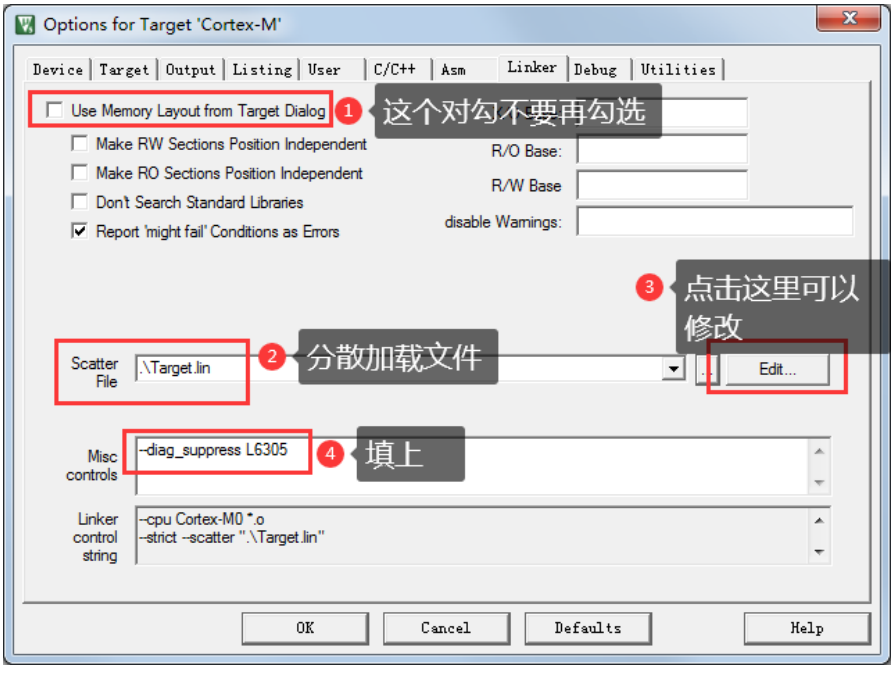
80.3.8第 8 步，将程序可执行文件 axf 修改为 flm 格式

通过下面的命令就可以将生成的 axf 可执行文件修改为 flm。



80.3.9第 9 步，分散加载设置

我们这里的分散加载文件直接使用 MDK 模板工程里提供好的即可，无需任何修改。



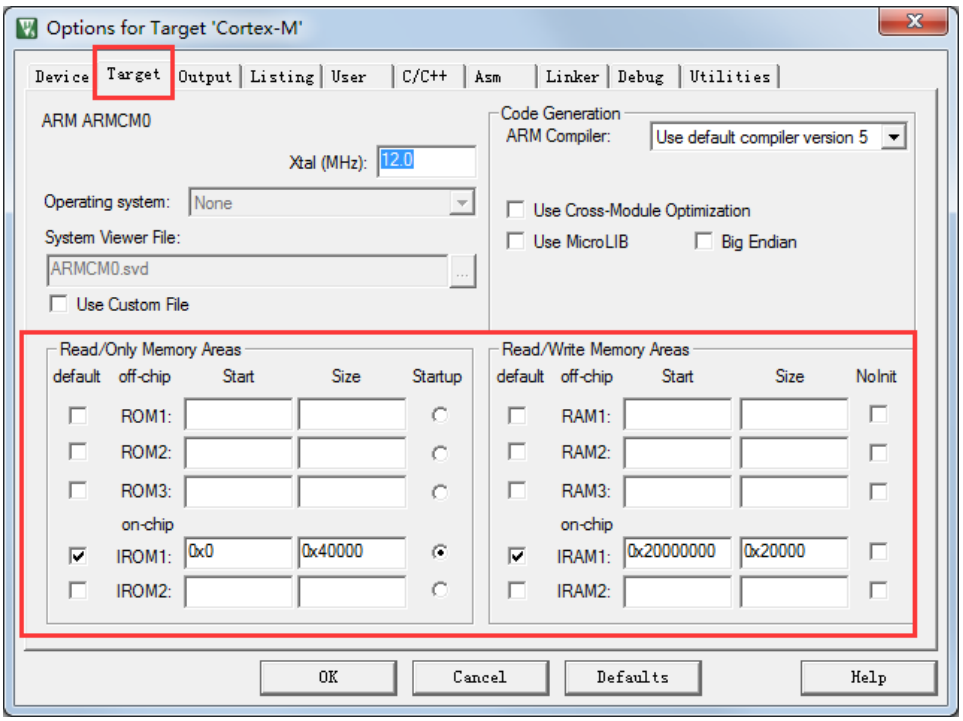
分散加载文件中的内容如下：

```
; Linker Control File (scatter-loading)
;

PRG 0 PI                ; Programming Functions
{
  PrgCode +0            ; Code
```

```
  {
    * (+RO)
  }
  PrgData +0            ; Data
  {
    * (+RW,+ZI)
  }
}

DSCR +0                 ; Device Description
{
  DevDscr +0
  {
    FlashDev.o
  }
}
```

--diag_suppress L6305 用于屏蔽 L6503 类型警告信息。

特别注意，设置了分散加载后，此处的配置就不再起作用了：



80.4 QSPI Flash 的 MDK 下载算法制作

下面将 QSPI Flash 算法制作过程中的几个关键点为大家做个说明。

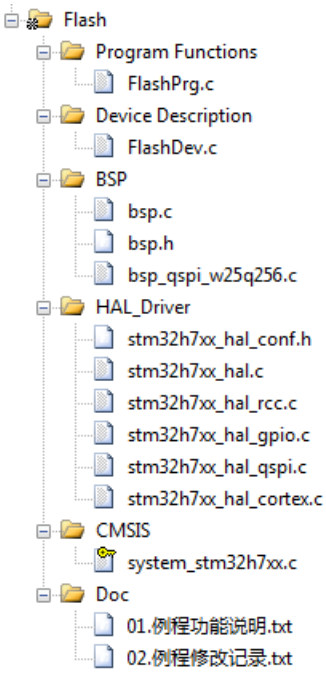
80.4.1 第 1 步，制作前重要提示

这两点非常重要：

- 程序里面不要开启任何中断，全部查询方式。
- HAL 库里面各种时间基准相关的 API 全部处理掉。简单省事些，我们这里是直接注释，采用死等即可。无需做超时等待，因为超时后，已经意味着操作失败了，跟死等没有区别。

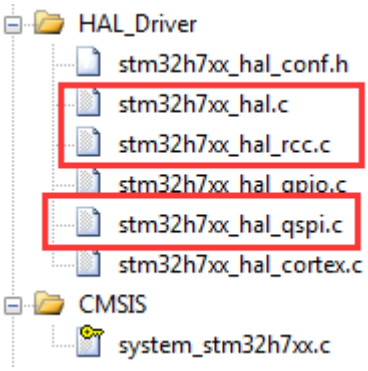
80.4.2 第 2 步，准备一个工程模板

推荐大家直接使用我们本章工程准备好的模板即可，如果大家自己制作，注意一点，请使用当前最新的 HAL 库。



80.4.3 第 3 步，修改 HAL 库

这一步比较重要，主要修改了以下三个文件：



主要是修改了 HAL 库时间基准相关的几个 API，并注释掉了一批无关的 API。具体修改内容，大家可以找个比较软件，对比修改后的这个文件和 CubeH7 软件包 V1.8.0（软件包里面的 HAL 库版本是 V1.9.0）的

差异即可。

80.4.4第 4 步，时钟初始化

我们已经用不到滴答定时器了，直接在 bsp.c 文件里面对滴答初始化函数做重定向：

```
/*
*********************************************************************************************************
*    函 数 名: HAL_InitTick
*    功能说明: 重定向，不使用
*    形    参: TickPriority
*    返 回 值: 无
*********************************************************************************************************
*/
HAL_StatusTypeDef HAL_InitTick(uint32_t TickPriority)
{
    return HAL_OK;
}
```

然后就是 HSE 外置晶振的配置，大家根据自己的板子实际外挂晶振大小，修改 stm32h7xx_hal_conf.h 文件中 HSE_VALUE 大小，实际晶振多大，这里就修改为多大：

```
#if !defined  (HSE_VALUE)
#define HSE_VALUE    ((uint32_t)25000000) /*!< Value of the External oscillator in Hz */
#endif /* HSE_VALUE */
```

最后修改 PLL：

```
/*
*********************************************************************************************************
*    函 数 名: SystemClock_Config
*    功能说明: 初始化系统时钟
*            System Clock source            = PLL (HSE)
*            SYSCLK(Hz)                     = 400000000 (CPU Clock)
*            HCLK(Hz)                       = 200000000 (AXI and AHBs Clock)
*            AHB Prescaler                  = 2
*            D1 APB3 Prescaler              = 2 (APB3 Clock  100MHz)
*            D2 APB1 Prescaler              = 2 (APB1 Clock  100MHz)
*            D2 APB2 Prescaler              = 2 (APB2 Clock  100MHz)
*            D3 APB4 Prescaler              = 2 (APB4 Clock  100MHz)
*            HSE Frequency(Hz)              = 25000000
*            PLL_M                          = 5
*            PLL_N                          = 160
*            PLL_P                          = 2
*            PLL_Q                          = 4
*            PLL_R                          = 2
*            VDD(V)                         = 3.3
*            Flash Latency(WS)              = 4
*    形    参: 无
*    返 回 值: 1 表示失败，0 表示成功
*********************************************************************************************************
*/
int SystemClock_Config(void)
{
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    HAL_StatusTypeDef ret = HAL_OK;
```

```
    /* 锁住 SCU(Supply configuration update) */
    MODIFY_REG(PWR->CR3, PWR_CR3_SCUEN, 0);

    /*
      1、芯片内部的 LDO 稳压器输出的电压范围，可选 VOS1，VOS2 和 VOS3，不同范围对应不同的 Flash 读速度，
         详情看参考手册的 Table 12 的表格。
      2、这里选择使用 VOS1，电压范围 1.15V - 1.26V。
    */
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

    while(!__HAL_PWR_GET_FLAG(PWR_FLAG_VOSRDY)) {}

    /* 使能 HSE，并选择 HSE 作为 PLL 时钟源 */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.HSIState = RCC_HSI_OFF;
    RCC_OscInitStruct.CSIState = RCC_CSI_OFF;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;

    RCC_OscInitStruct.PLL.PLLM = 5;
    RCC_OscInitStruct.PLL.PLLN = 160;
    RCC_OscInitStruct.PLL.PLLP = 2;
    RCC_OscInitStruct.PLL.PLLR = 2;
    RCC_OscInitStruct.PLL.PLLQ = 4;

    RCC_OscInitStruct.PLL.PLLVCOSEL = RCC_PLL1VCOWIDE;
    RCC_OscInitStruct.PLL.PLLRGE = RCC_PLL1VCIRANGE_2;
    ret = HAL_RCC_OscConfig(&RCC_OscInitStruct);
    if(ret != HAL_OK)
    {
        return 1;
    }

    /*
       选择 PLL 的输出作为系统时钟
       配置 RCC_CLOCKTYPE_SYSCLK 系统时钟
       配置 RCC_CLOCKTYPE_HCLK 时钟，对应 AHB1，AHB2，AHB3 和 AHB4 总线
       配置 RCC_CLOCKTYPE_PCLK1 时钟，对应 APB1 总线
       配置 RCC_CLOCKTYPE_PCLK2 时钟，对应 APB2 总线
       配置 RCC_CLOCKTYPE_D1PCLK1 时钟，对应 APB3 总线
       配置 RCC_CLOCKTYPE_D3PCLK1 时钟，对应 APB4 总线
    */
    RCC_ClkInitStruct.ClockType = (RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_D1PCLK1 |
RCC_CLOCKTYPE_PCLK1 | \
                                   RCC_CLOCKTYPE_PCLK2  | RCC_CLOCKTYPE_D3PCLK1);

    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.SYSCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB3CLKDivider = RCC_APB3_DIV2;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_APB1_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_APB2_DIV2;
    RCC_ClkInitStruct.APB4CLKDivider = RCC_APB4_DIV2;

    /* 此函数会更新 SystemCoreClock，并重新配置 HAL_InitTick */
    ret = HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4);
```

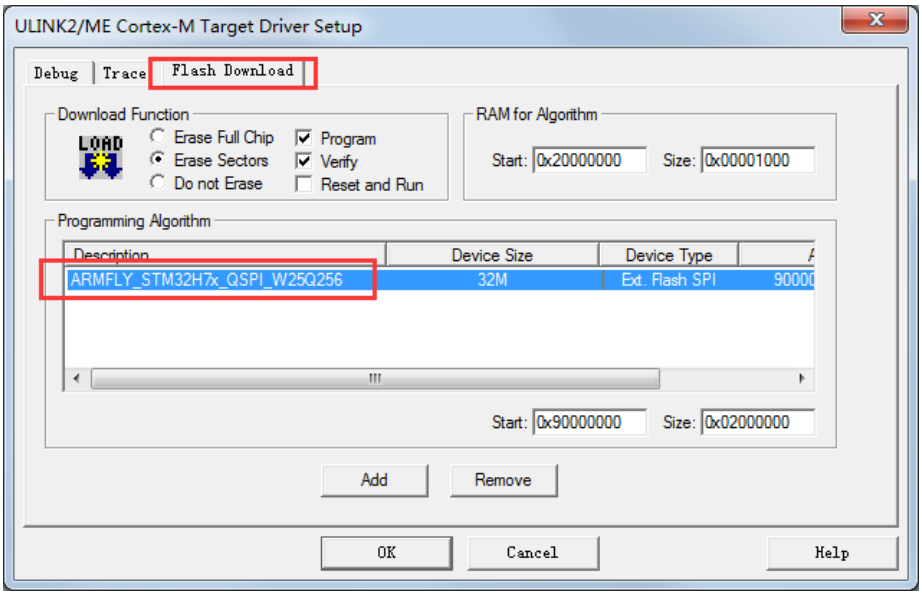
```
    if(ret != HAL_OK)
    {
        return 1;
    }

    /*
      使用 IO 的高速模式，要使能 IO 补偿，即调用下面三个函数
      （1）使能 CSI clock
      （2）使能 SYSCFG clock
      （3）使能 I/O 补偿单元， 设置 SYSCFG_CCCSR 寄存器的 bit0
    */
    __HAL_RCC_CSI_ENABLE() ;

    __HAL_RCC_SYSCFG_CLK_ENABLE() ;

    HAL_EnableCompensationCell();

    __HAL_RCC_D2SRAM1_CLK_ENABLE();
    __HAL_RCC_D2SRAM2_CLK_ENABLE();
    __HAL_RCC_D2SRAM3_CLK_ENABLE();

    return 0;
}
```

80.4.5第 5 步，配置文件 FlashDev.c 的实现

配置如下：

```
struct FlashDevice const FlashDevice  =  {
    FLASH_DRV_VERS,                 /* 驱动版本，勿修改，这个是 MDK 定的 */
    "ARMFLY_STM32H7x_QSPI_W25Q256",  /* 算法名，添加算法到 MDK 安装目录会显示此名字 */
    EXTSPI,                         /* 设备类型 */
    0x90000000,                     /* Flash 起始地址 */
    32 * 1024 * 1024,               /* Flash 大小，32MB */
    4 * 1024,                       /* 编程页大小 */
    0,                              /* 保留，必须为 0 */
    0xFF,                           /* 擦除后的数值 */
    1000,                           /* 页编程等待时间 */
    6000,                           /* 扇区擦除等待时间 */
    64 * 1024, 0x000000,            /* 扇区大小，扇区地址 */
    SECTOR_END
};
```

注释已经比较详细，大家根据自己的需要做修改即可。注意一点，算法名 ARMFLY_STM32H7x_QSPI_W25Q256 会反馈到这个地方：



80.4.6第 6 步，编程文件 FlashPrg.c 的实现

下面将文件中实现的几个函数为大家做个说明：

◆ **初始化函数 Init**

```
/*
*********************************************************************************************************
*    函 数 名: Init
*    功能说明: Flash 编程初始化
*    形    参: adr Flash 基地址，芯片首地址。
*              clk 时钟频率
*              fnc 函数代码，1 - Erase, 2 - Program, 3 - Verify
*    返 回 值: 0 表示成功， 1 表示失败
*********************************************************************************************************
*/
int Init (unsigned long adr, unsigned long clk, unsigned long fnc)
{
    int result = 0;

    /* 系统初始化 */
    SystemInit();

    /* 时钟初始化 */
    result = SystemClock_Config();
    if (result  != 0)
    {
        return 1;
    }

    /* W25Q256 初始化 */
    result = bsp_InitQSPI_W25Q256();
    if (result != 0)
    {
        return 1;
    }
```

```
    /* 内存映射 */
    result = QSPI_MemoryMapped();
    if (result != 0)
    {
        return 1;
    }

    return 0;
}
```

初始化完毕后将其设置为内存映射模式。

◆ 复位初始化函数 Uinit

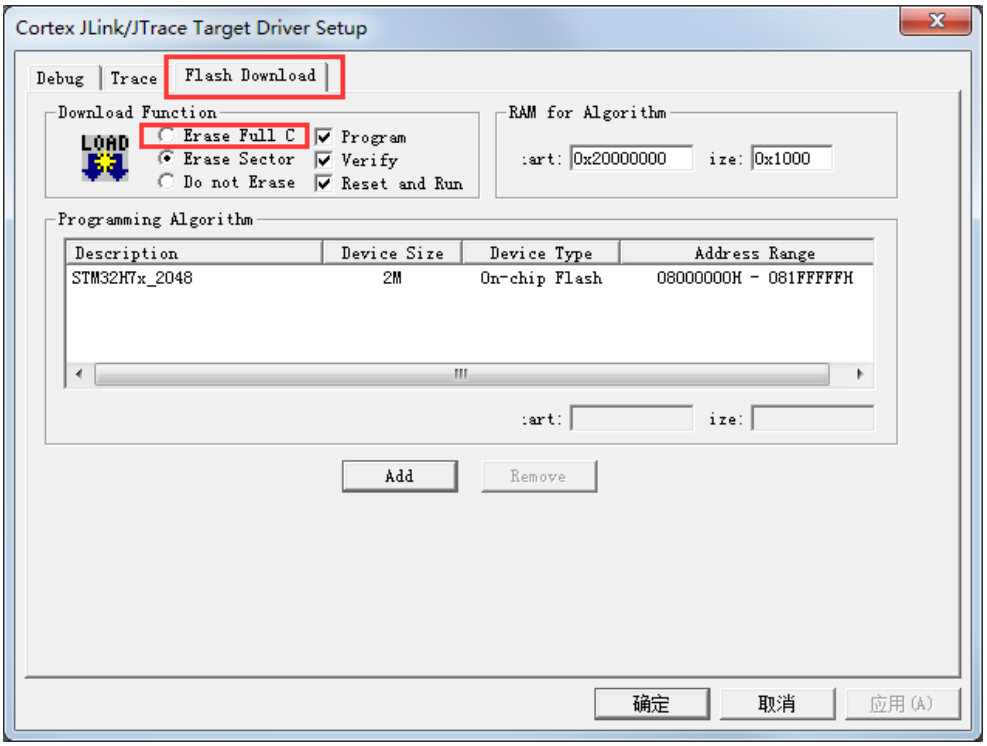
擦除，编程和校验函数后都会调用此函数。

```
/*
*********************************************************************************************************
*    函 数 名: UnInit
*    功能说明: 复位初始化
*    形    参: fnc 函数代码, 1 - Erase, 2 - Program, 3 - Verify
*    返 回 值: 0 表示成功， 1 表示失败
*********************************************************************************************************
*/
int UnInit (unsigned long fnc)
{
    int result = 0;

    /* W25Q256 初始化 */
    result = bsp_InitQSPI_W25Q256();
    if (result != 0)
    {
        return 1;
    }

    /* 内存映射 */
    result = QSPI_MemoryMapped();
    if (result != 0)
    {
        return 1;
    }

    return (0);
}
```

复位初始化这里，直接将其设置为内存映射模式。

◆ 整个芯片擦除函数 EraseChip

如果大家配置勾选了 MDK Option 选项中此处的配置，会调用的整个芯片擦除：



实际应用中不推荐大家勾选这里，因为整个芯片擦除太耽误时间，比如 32MB QSPI Flash 整个芯片擦除需要 300 秒左右。

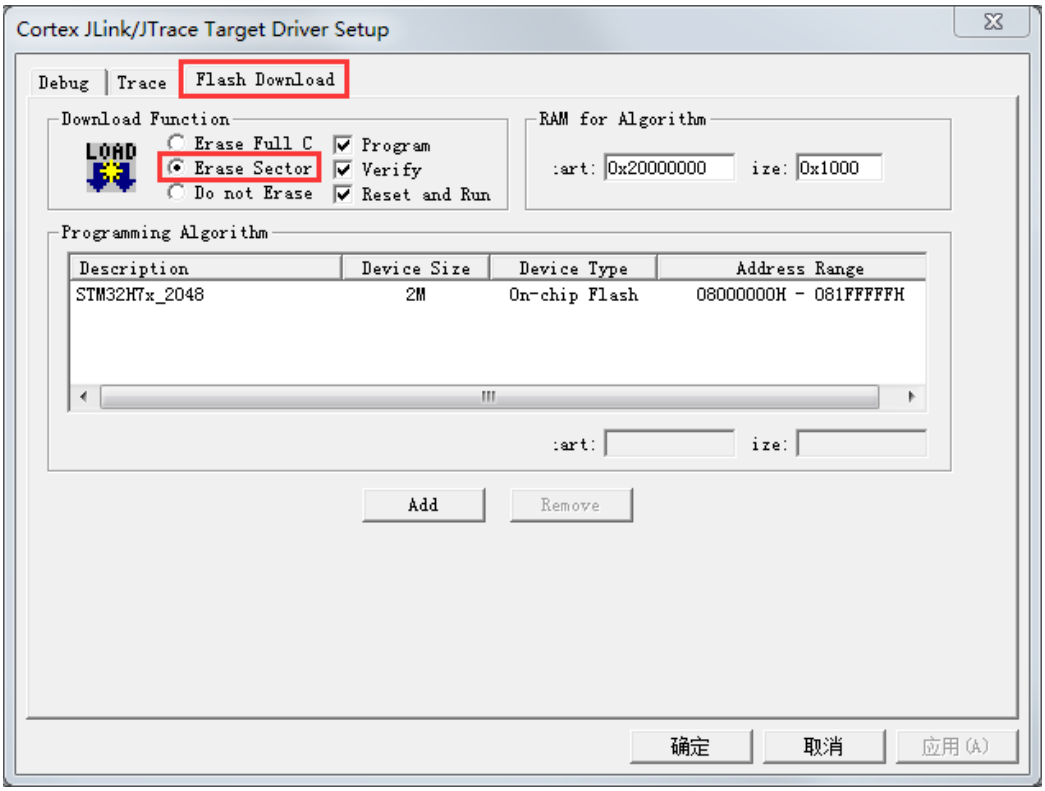
另外，如果大家的算法工程里面没有添加此函数，MDK 会调用扇区擦除函数来实现，直到所有扇区擦除完毕。

```
/*
*********************************************************************************************************
*    函 数 名: UnInit
*    功能说明: 复位初始化
*    形    参: fnc 函数代码，1 - Erase, 2 - Program, 3 - Verify
*    返 回 值: 0 表示成功， 1 表示失败
*********************************************************************************************************
*/
int UnInit (unsigned long fnc)
{
    int result = 0;

    /* W25Q256 初始化 */
    result = bsp_InitQSPI_W25Q256();
    if (result != 0)
    {
        return 1;
    }

    /* 内存映射 */
    result = QSPI_MemoryMapped();
    if (result != 0)
    {
        return 1;
    }

    return (0);
```

```
}
```

◆ 扇区擦除函数 EraseSector

如果大家配置勾选了 MDK Option 选项中此处的配置，会调用扇区擦除：




```
/*
*********************************************************************************************************
*    函 数 名: EraseSector
*    功能说明: 扇区擦除
*    形    参: adr 擦除地址
*    返 回 值: 无
*********************************************************************************************************
*/
int EraseSector (unsigned long adr)
{
    int result = 0;

    /* 地址要在操作的芯片范围内 */
    if (adr < QSPI_FLASH_MEM_ADDR || adr >= QSPI_FLASH_MEM_ADDR + QSPI_FLASH_SIZES)
    {
        return 1;
    }

    adr -= QSPI_FLASH_MEM_ADDR;

    /* W25Q256 初始化 */
    result = bsp_InitQSPI_W25Q256();
    if (result != 0)
    {
        return 1;
    }

    /* 扇区擦除 */
```

```
    result = QSPI_EraseSector(adr);
    if (result != 0)
    {
        return 1;
    }

    /* 内存映射 */
    result = QSPI_MemoryMapped();
    if (result != 0)
    {
        return 1;
    }

    return 0;
}
```

这里要注意两点：

（1） 程序里面的操作 adr -= QSPI_FLASH_MEM_ADDR，实际传递进来的地址是带了首地址的，即 0x90000000。

（2） 这里执行的擦除大小要前面 FlashDev.c 文件中配置的扇区大小一致，这里是执行的 64KB 为扇区进行擦除。

◆ 页编程函数 ProgramPage

页编程函数实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: ProgramPage
*    功能说明: 页编程
*    形    参: adr 页起始地址
*              sz  页大小
*              buf 要写入的数据地址
*    返 回 值: 无
*********************************************************************************************************
*/
int ProgramPage (unsigned long adr, unsigned long sz, unsigned char *buf)
{
    int size;
    int result = 0;

    /* 地址要在操作的芯片范围内 */
    if (adr < QSPI_FLASH_MEM_ADDR || adr >= QSPI_FLASH_MEM_ADDR + QSPI_FLASH_SIZES)
    {
        return 1;
    }

    /* W25Q256 初始化 */
    result = bsp_InitQSPI_W25Q256();
    if (result != 0)
    {
        return 1;
    }

    adr -= QSPI_FLASH_MEM_ADDR;
    size =  sz;
```

```
    /* 页编程 */
    while(size > 0)
    {
        if (QSPI_WriteBuffer(buf, adr, 256) == 1)
        {
            QSPI_MemoryMapped();

            return 1;
        }
        size -= 256;
        adr += 256;
        buf += 256;
    }

    /* 内存映射 */
    result = QSPI_MemoryMapped();
    if (result != 0)
    {
        return 1;
    }

    return (0);
}
```

这里注意两点：

（1） W25Q256 的页大小是 256 字节，前面 FlashDev.c 中将页编程大小设置为 4096 字节，所以此程序要做处理。

（2） 程序里面的操作 adr -= QSPI_FLASH_MEM_ADDR，实际传递进来的地址是带了首地址的，即 0x90000000。

◆ **读取和校验函数**

我们程序中未做读取和校验函数。

（1） 如果程序中未做读取函数，那么 MDK 会以总线方式进行读取，这也是为什么每个函数执行完毕都设置为内存映射模式的原因。

（2） 如果程序中未做校验函数，那么 MDK 会读取数据做 CRC 校验。

80.4.7第 7 步，修改 QSPI Flash 驱动文件（引脚，命令等）

最后一步就是 QSPI Flash（W25Q256）的驱动修改，大家可以根据自己的需求做修改。使用的引脚定义在文件 bsp_qspi_w25q256.c（做了条件编译，包含了 H7-TOOL 和 STM32-V7 板子）：

```
/*
    STM32-V7 开发板接线

    PG6/QUADSPI_BK1_NCS     AF10
    PF10/QUADSPI_CLK        AF9
    PF8/QUADSPI_BK1_IO0     AF10
    PF9/QUADSPI_BK1_IO1     AF10
```

```
    PF7/QUADSPI_BK1_IO2     AF9
    PF6/QUADSPI_BK1_IO3     AF9

    W25Q256JV 有 512 块，每块有 16 个扇区，每个扇区 Sector 有 16 页，每页有 256 字节，共计 32MB

    H7-TOOL 开发板接线

    PG6/QUADSPI_BK1_NCS     AF10
    PB2/QUADSPI_CLK         AF9
    PD11/QUADSPI_BK1_IO0    AF10
    PD12/QUADSPI_BK1_IO1    AF10
    PF7/QUADSPI_BK1_IO2     AF9
    PD13/QUADSPI_BK1_IO3    AF9
*/

/* QSPI 引脚和时钟相关配置宏定义 */
#if 0
#define QSPI_CLK_ENABLE()               __HAL_RCC_QSPI_CLK_ENABLE()
#define QSPI_CLK_DISABLE()              __HAL_RCC_QSPI_CLK_DISABLE()
#define QSPI_CS_GPIO_CLK_ENABLE()       __HAL_RCC_GPIOG_CLK_ENABLE()
#define QSPI_CLK_GPIO_CLK_ENABLE()      __HAL_RCC_GPIOB_CLK_ENABLE()
#define QSPI_BK1_D0_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOD_CLK_ENABLE()
#define QSPI_BK1_D1_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOD_CLK_ENABLE()
#define QSPI_BK1_D2_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOF_CLK_ENABLE()
#define QSPI_BK1_D3_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOD_CLK_ENABLE()

#define QSPI_MDMA_CLK_ENABLE()          __HAL_RCC_MDMA_CLK_ENABLE()
#define QSPI_FORCE_RESET()              __HAL_RCC_QSPI_FORCE_RESET()
#define QSPI_RELEASE_RESET()            __HAL_RCC_QSPI_RELEASE_RESET()

#define QSPI_CS_PIN                     GPIO_PIN_6
#define QSPI_CS_GPIO_PORT               GPIOG
#define QSPI_CS_GPIO_AF                 GPIO_AF10_QUADSPI

#define QSPI_CLK_PIN                    GPIO_PIN_2
#define QSPI_CLK_GPIO_PORT              GPIOB
#define QSPI_CLK_GPIO_AF                GPIO_AF9_QUADSPI

#define QSPI_BK1_D0_PIN                 GPIO_PIN_11
#define QSPI_BK1_D0_GPIO_PORT           GPIOD
#define QSPI_BK1_D0_GPIO_AF             GPIO_AF9_QUADSPI

#define QSPI_BK1_D1_PIN                 GPIO_PIN_12
#define QSPI_BK1_D1_GPIO_PORT           GPIOD
#define QSPI_BK1_D1_GPIO_AF             GPIO_AF9_QUADSPI

#define QSPI_BK1_D2_PIN                 GPIO_PIN_7
#define QSPI_BK1_D2_GPIO_PORT           GPIOF
#define QSPI_BK1_D2_GPIO_AF             GPIO_AF9_QUADSPI

#define QSPI_BK1_D3_PIN                 GPIO_PIN_13
#define QSPI_BK1_D3_GPIO_PORT           GPIOD
#define QSPI_BK1_D3_GPIO_AF             GPIO_AF9_QUADSPI
#else
#define QSPI_CLK_ENABLE()               __HAL_RCC_QSPI_CLK_ENABLE()
#define QSPI_CLK_DISABLE()              __HAL_RCC_QSPI_CLK_DISABLE()
#define QSPI_CS_GPIO_CLK_ENABLE()       __HAL_RCC_GPIOG_CLK_ENABLE()
#define QSPI_CLK_GPIO_CLK_ENABLE()      __HAL_RCC_GPIOF_CLK_ENABLE()
```

```
#define QSPI_BK1_DO_GPIO_CLK_ENABLE()    __HAL_RCC_GPIOF_CLK_ENABLE()
#define QSPI_BK1_D1_GPIO_CLK_ENABLE()    __HAL_RCC_GPIOF_CLK_ENABLE()
#define QSPI_BK1_D2_GPIO_CLK_ENABLE()    __HAL_RCC_GPIOF_CLK_ENABLE()
#define QSPI_BK1_D3_GPIO_CLK_ENABLE()    __HAL_RCC_GPIOF_CLK_ENABLE()

#define QSPI_MDMA_CLK_ENABLE()           __HAL_RCC_MDMA_CLK_ENABLE()
#define QSPI_FORCE_RESET()               __HAL_RCC_QSPI_FORCE_RESET()
#define QSPI_RELEASE_RESET()             __HAL_RCC_QSPI_RELEASE_RESET()

#define QSPI_CS_PIN                      GPIO_PIN_6
#define QSPI_CS_GPIO_PORT                GPIOG
#define QSPI_CS_GPIO_AF                  GPIO_AF10_QUADSPI

#define QSPI_CLK_PIN                     GPIO_PIN_10
#define QSPI_CLK_GPIO_PORT               GPIOF
#define QSPI_CLK_GPIO_AF                 GPIO_AF9_QUADSPI

#define QSPI_BK1_DO_PIN                  GPIO_PIN_8
#define QSPI_BK1_DO_GPIO_PORT            GPIOF
#define QSPI_BK1_DO_GPIO_AF              GPIO_AF10_QUADSPI

#define QSPI_BK1_D1_PIN                  GPIO_PIN_9
#define QSPI_BK1_D1_GPIO_PORT            GPIOF
#define QSPI_BK1_D1_GPIO_AF              GPIO_AF10_QUADSPI

#define QSPI_BK1_D2_PIN                  GPIO_PIN_7
#define QSPI_BK1_D2_GPIO_PORT            GPIOF
#define QSPI_BK1_D2_GPIO_AF              GPIO_AF9_QUADSPI

#define QSPI_BK1_D3_PIN                  GPIO_PIN_6
#define QSPI_BK1_D3_GPIO_PORT            GPIOF
#define QSPI_BK1_D3_GPIO_AF              GPIO_AF9_QUADSPI
#endif
```

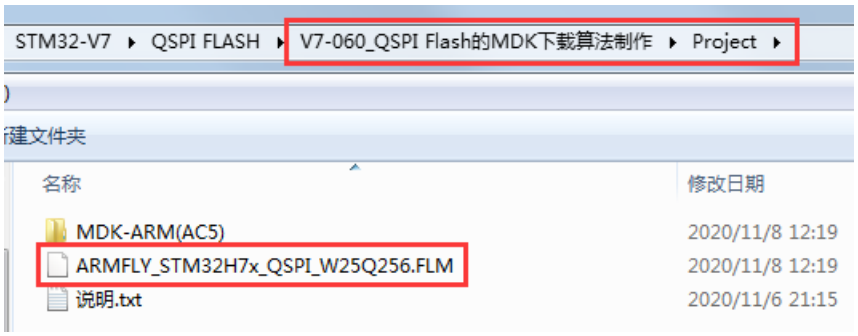
硬件设置了之后，剩下就是 QSPI Flash 相关的几个配置，在文件 bsp_qspi_w25q256.h：

主要是下面这几个：

```
#define QSPI_FLASH_MEM_ADDR          0x90000000

/* W25Q256JV 基本信息 */
#define QSPI_FLASH_SIZE      25                     /* Flash 大小，2^25 = 32MB*/
#define QSPI_SECTOR_SIZE     (4 * 1024)             /* 扇区大小，4KB */
#define QSPI_PAGE_SIZE       256                    /* 页大小，256 字节 */
#define QSPI_END_ADDR        (1 << QSPI_FLASH_SIZE) /* 末尾地址 */
#define QSPI_FLASH_SIZES     32 * 1024 * 1024        /* Flash 大小，2^25 = 32MB*/

/* W25Q256JV 相关命令 */
#define WRITE_ENABLE_CMD                        0x06    /* 写使能指令 */
#define READ_ID_CMD2                            0x9F    /* 读取 ID 命令 */
#define READ_STATUS_REG_CMD                     0x05    /* 读取状态命令 */
#define SUBSECTOR_ERASE_4_BYTE_ADDR_CMD         0x21    /* 32bit 地址扇区擦除指令，4KB */
#define QUAD_IN_FAST_PROG_4_BYTE_ADDR_CMD       0x34    /* 32bit 地址的 4 线快速写入命令 */
#define QUAD_INOUT_FAST_READ_4_BYTE_ADDR_CMD    0xEC    /* 32bit 地址的 4 线快速读取命令 */

#define BLOCK_ERASE_64K_4_BYTE_ADDR_CMD         0xDC    /* 4 字节地址，64K 扇区 */

#define BULK_ERASE_CMD                          0xC7    /* 整片擦除 */
```

80.5 QSPI Flash 的 MDK 下载算法使用方法

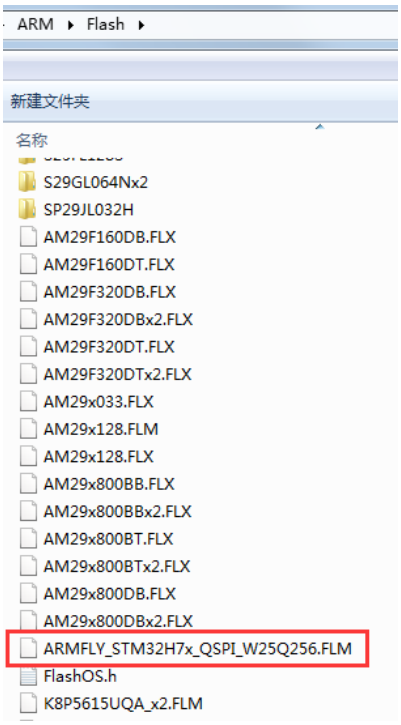
编译本章教程配套的例子，生成的算法文件位于此路径下：



80.5.1 下载算法存放位置

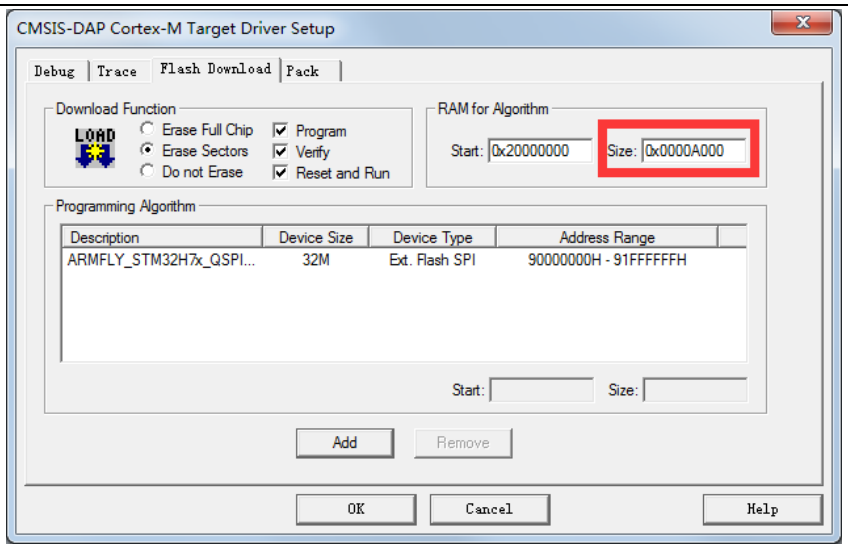
生成算法文件后，需要大家将其存到 MDK 安装目录，有两个位置可以存放，任选其一，推荐第 2 种：

- 第 1 种：存放到 MDK 的 STM32H7 软包安装目录里面：\Keil\STM32H7xx_DFP\2.6.0\CMSIS\Flash（软包版本不同，数值 2.6.0 不同）。
- 第 2 种：MDK 的安装目录 \ARM\Flash 里面。



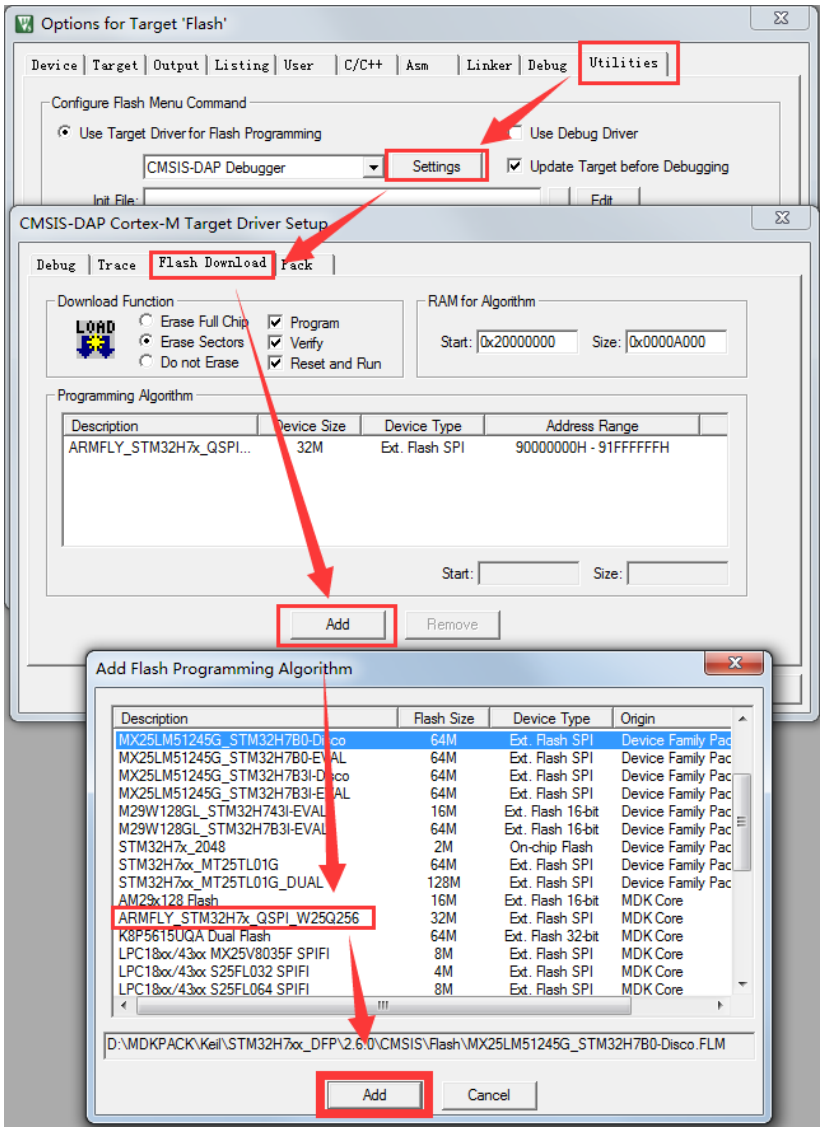
80.5.2 下载配置

注意这里一定要够大，否则会提示算法文件无法加载：



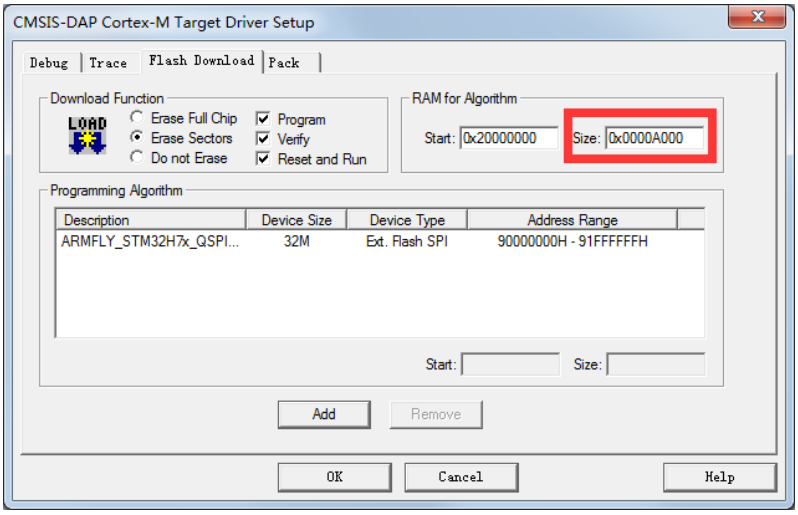
我们这里是将其加到 DTCM 中，即首地址为 0x20000000，大家也可以存储到任意其它 RAM 地址，只要空间还够加载算法文件即可。推荐使用 AXI SRAM（地址 0x24000000），因为这块 RAM 空间足够大。

如果要下载程序到 QSPI Flash 里面，需要做如下配置：



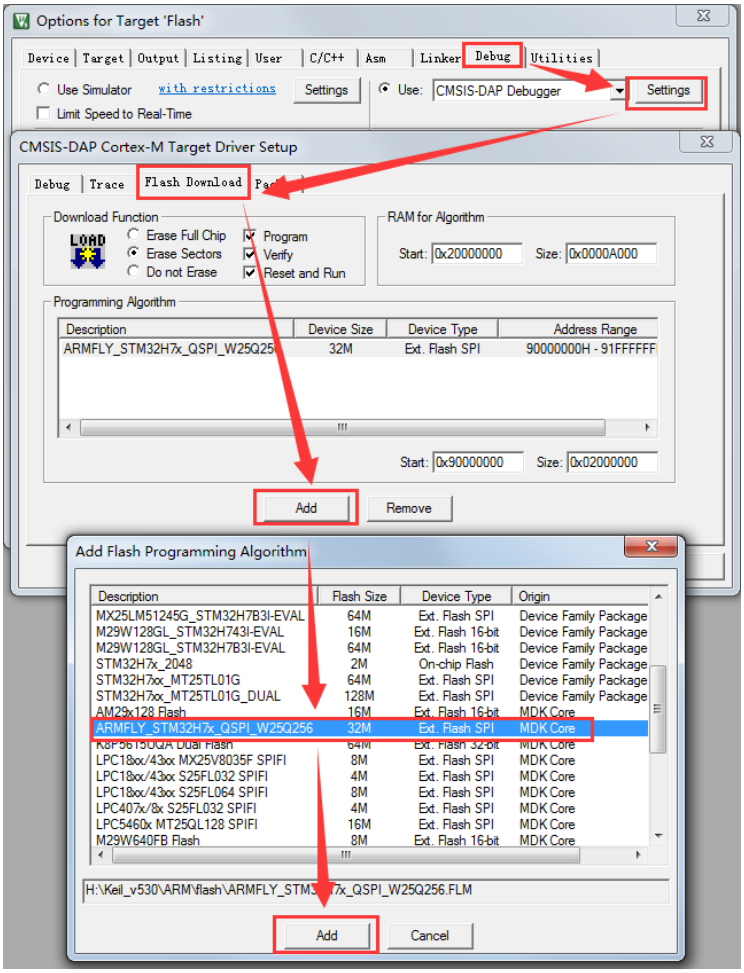
80.5.3调试配置

注意这里一定要够大，否则会提示算法文件无法加载：



我们这里是将其加到 DTCM 中，即首地址为 0x20000000，大家也可以存储到任意其它 RAM 地址，只要空间还够加载算法文件即可。

如果要做调试下载，需要做如下配置：



80.5.4验证算法文件是否可以正常使用

为了验证算法文件是否可以正常使用，大家可以运行本教程第 82 章或者 83 章配套的例子。

80.6 实验例程说明

本章配套例子：V7-060_QSPI Flash 的 MDK 下载算法制作。

编译后，算法文件会存到此路径下：

80.7 总结

本章节就为大家讲解这么多，为了熟练掌握，大家可以尝试自己实现一个 Flash 下载算法。

第81章 STM32H7 的 QSPI 总线应用之 QSPI Flash 的 STM32CubeProg 下载算法制作

本章节为大家讲解 STM32CubeProg 下载算法制作方法。

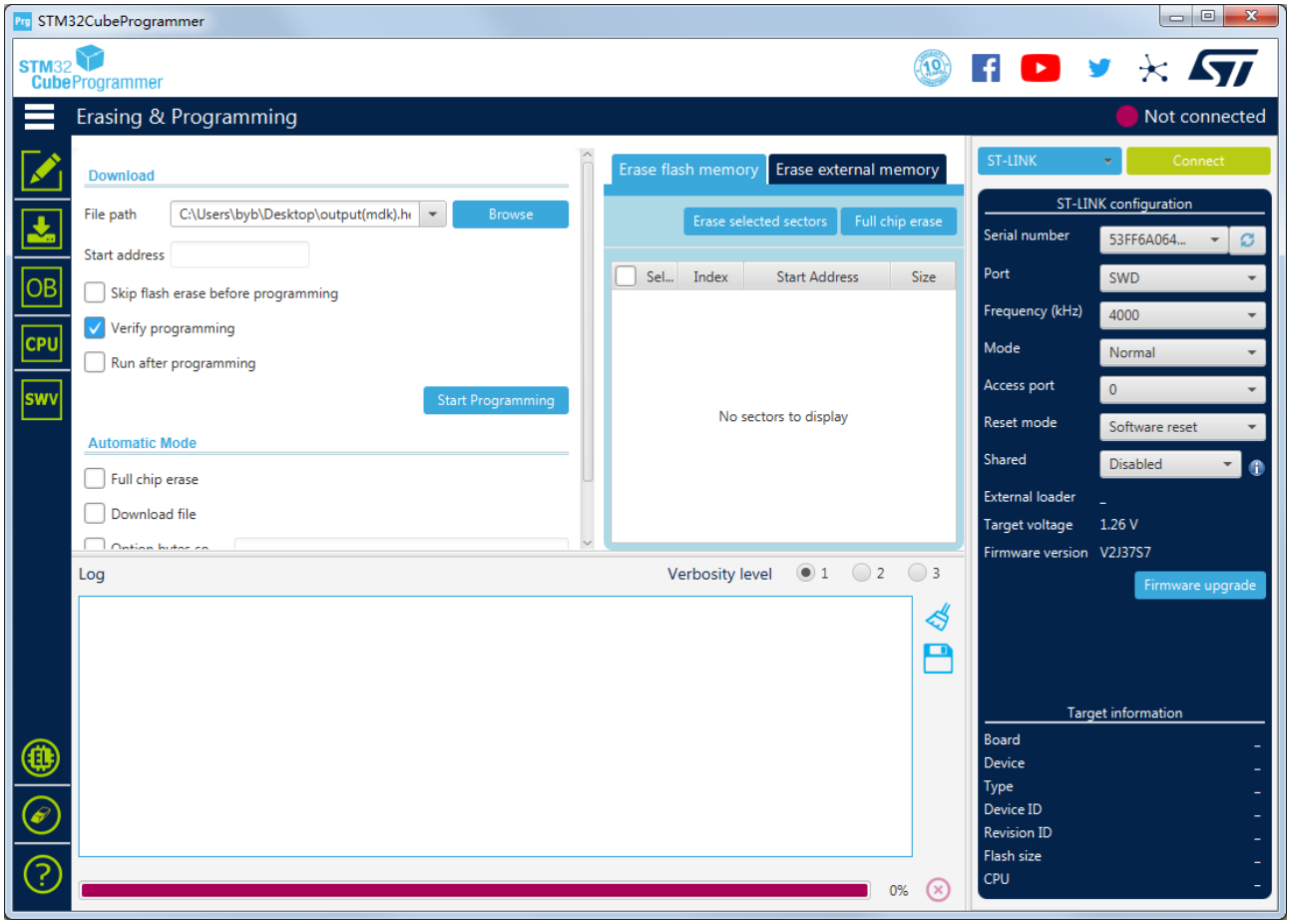
81.1 初学者重要提示
81.2 STM32CubeProg 简介
80.3 STM32CubeProg 下载算法基础知识
80.4 创建 STM32CubeProg 下载算法通用流程
80.4 QSPI Flash 的 STM32CubeProg 下载算法制作
80.5 QSPI Flash 的 STM32CubeProg 下载算法使用方法
80.6 实验例程说明
80.7 总结

81.1 初学者重要提示

- QSPI Flash 的相关知识点可以看第 78 章和 79 章。
- QSPI Flash 下载算法文件直接采用 HAL 库制作，方便大家自己修改。
- STM32CubeProg 下载算法制作和 MDK 下载算法制作基本是一样
- 本教程的第 68 章 USB DFU 和第 69 章串口 IAP 章节为大家介绍过 STM32CubeProg 的用法。
  STM32CubeProg 下载
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=97370 。

81.2 STM32CubeProg 简介

STM32CubeProg，此软件实现了之前的 DfuSe，STLINK 小软件和 Flashloader 三合一，并且支持外部 EEPROM，NOR Flash，SPI Flash，NAND Flash 等烧写，也支持 OTA 编程。



81.3 STM32CubeProg 下载算法基础知识

STM32CubeProg 下载算法是一种用于擦除应用程序或将应用程序下载到 Flash 的程序代码。ST 自家的芯片都自带下载算法，存放在 STM32CubeProg 安装目录里面，但不支持的需要我们自己制作，本章教程为此而生。

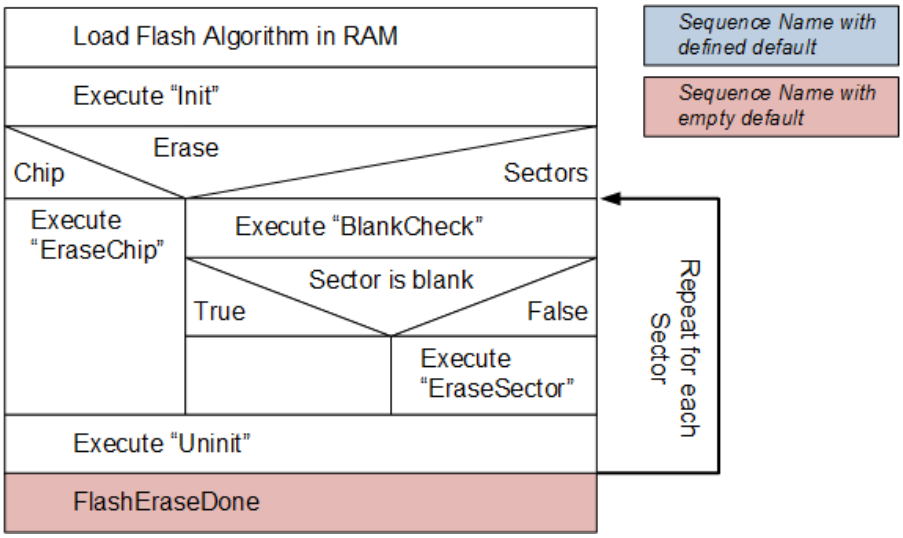
81.3.1 程序能够通过下载算法下载到芯片的核心思想

认识到这点很重要：通过 IDE 开发环境创建一批与地址信息无关的算法文件，实现的功能主要有初始化，擦除，编程，读取，校验等，然后 STM32CubeProg 下载阶段，会将算法文件加载到芯片的内部 RAM 里面，然后 STM32CubeProg 通过与这个算法文件的交互，实现程序下载，数据读取等操作。

81.3.2 算法程序中擦除操作执行流程

注：下面是 MDK 的算法执行流程，STM32CubeProg 是类似的。

擦除操作大致流程：

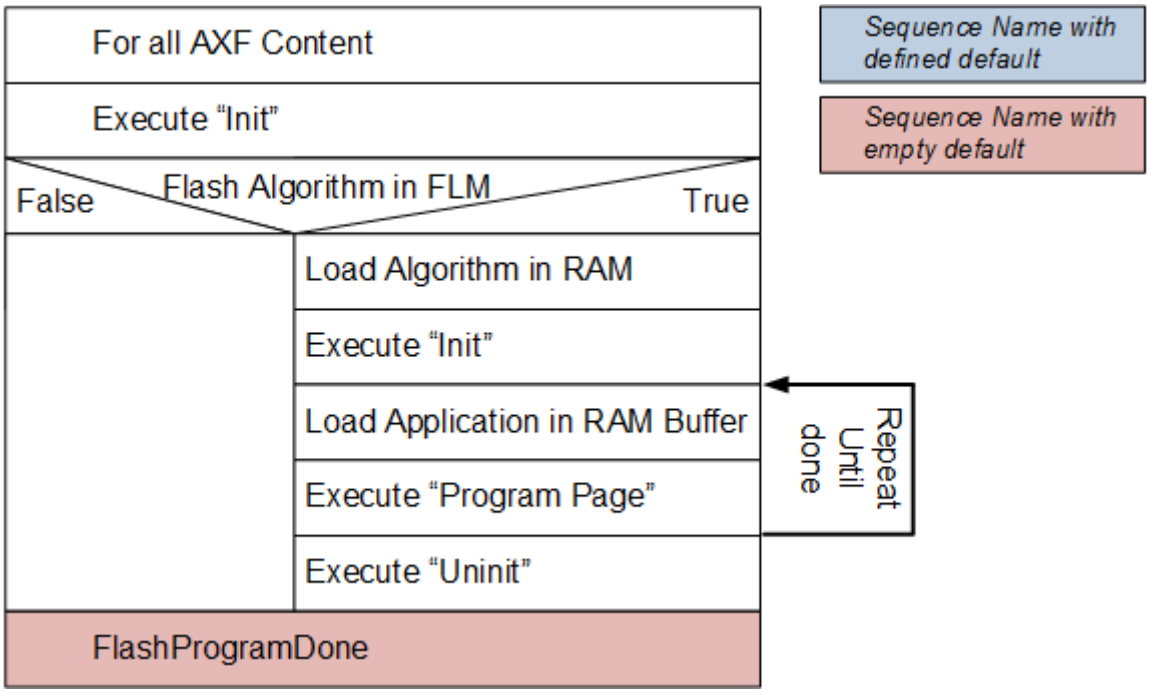


- 加载算法到芯片 RAM。
- 执行初始化函数 Init。
- 执行擦除操作，根据用户配置，这里可以选择整个芯片擦除或者扇区擦除。
- 执行 Uinit 函数。
- 操作完毕。

81.3.3 算法程序中编程操作执行流程

注：下面是 MDK 的算法执行流程，STM32CubeProg 是类似的（没有 Unint 函数）。

编程操作大致流程：



- 针对 IDE 生成的 axf（elf）可执行文件做 Init 初始化，这个 axf（elf）文件是指的大家自己创建应用

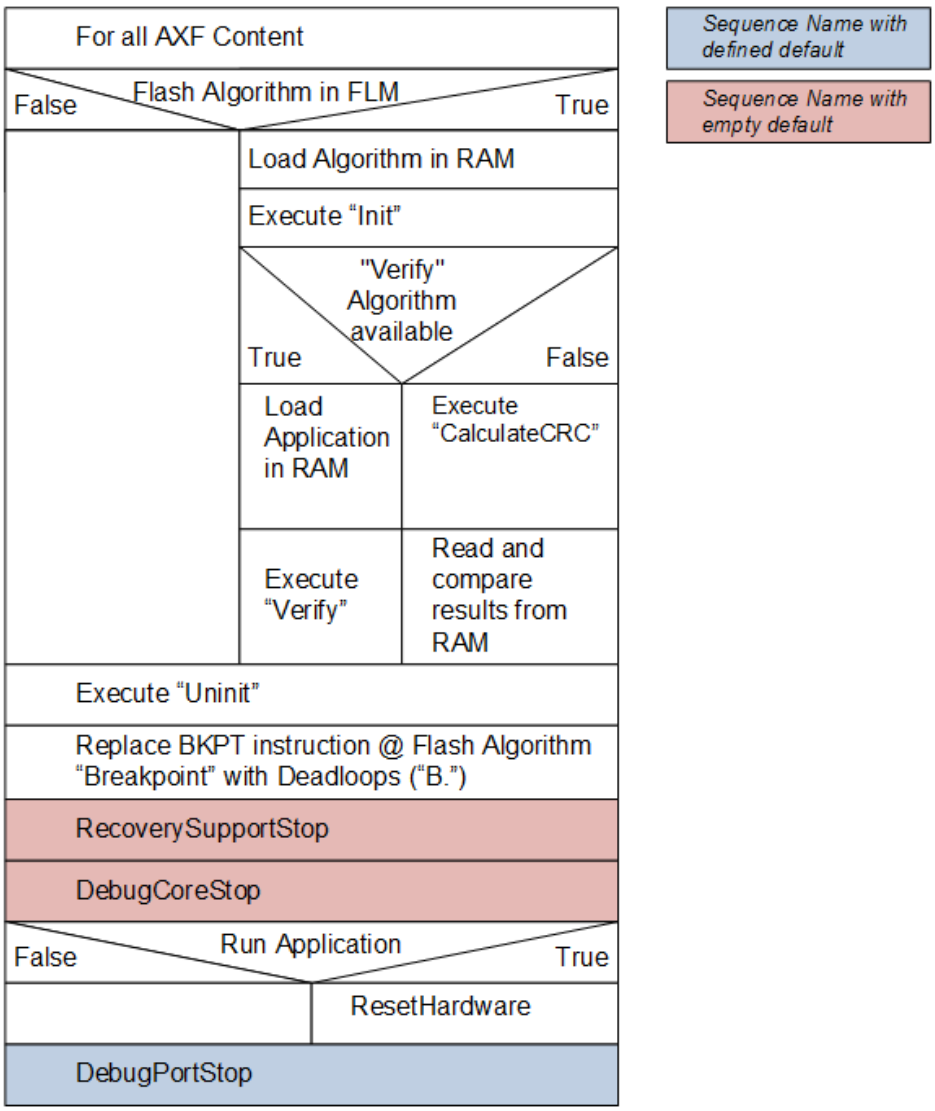
程序生成的。

- 查看 Flash 算法是否在 FLM 文件。如果没有在，操作失败。如果在：
  - 加载算法到 RAM。
  - 执行 Init 函数。
  - 加载用户到 RAM 缓冲。
  - 执行 Program Page 页编程函数。
  - 执行 Uninit 函数。
- 操作完毕。

81.3.4算法程序中校验操作执行流程

注：下面是 MDK 的算法执行流程，STM32CubeProg 是类似的（没有 Unint 函数）。

校验操作大致流程：



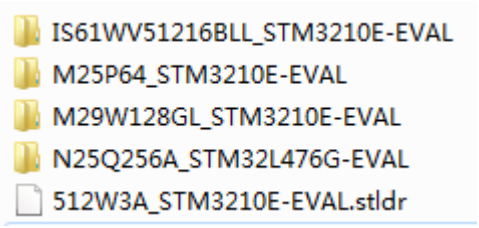
- 校验要用到 IDE 生成的 axf（elf）可执行文件。校验就是 axf（elf）文件中下载到芯片的程序和实际下载的程序读出来做比较。
- 查看 Flash 算法是否在 FLM 文件。如果没有在，操作失败。如果在：
  - 加载算法到 RAM。
  - 执行 Init 函数。
  - 查看校验算法是否存在
    - 如果有，加载应用程序到 RAM 并执行校验。
    - 如果没有，执行计算 CRC，将芯片中读取数据出来和 RAM 中加载应用计算输出的 CRC 值做比较。
  - 执行 Uninit 函数。
  - 替换 BKPT（BreakPoint 断点指令）为 B. 死循环指令。
  - 执行 RecoverySupportStop，回复支持停止。
  - 执行 DebugCoreStop，调试内核停止。
- 运行应用：
  - 执行失败
  - 执行成功，再执行硬件复位。
- 操作完毕，停止调试端口。

81.4 创建 STM32CubeProg 下载算法通用流程

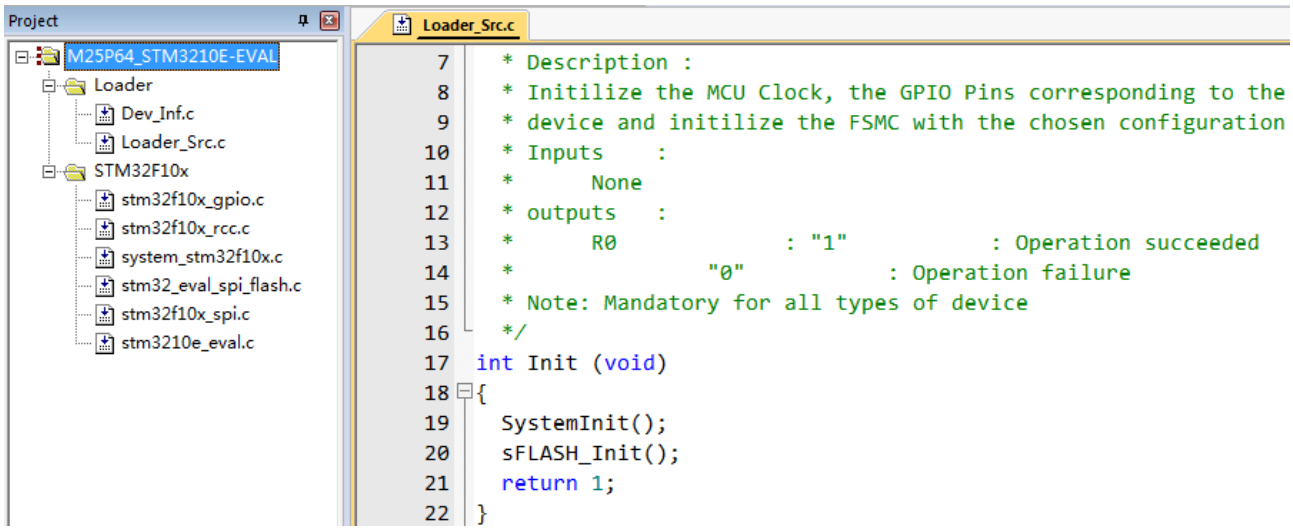
下面是 STM32CubeProg 给的一种大致操作流程，不限制必须采用这种方法，自己创建也可以的。

81.4.1 第 1 步，使用 STM32CubeProg 提供好的程序模板

位于路径：STMicroelectronics\STM32Cube\STM32CubeProgrammer\bin\ExternalLoader



以 M25P64 为例（注，其余步骤就以这个为例子进行说明）：

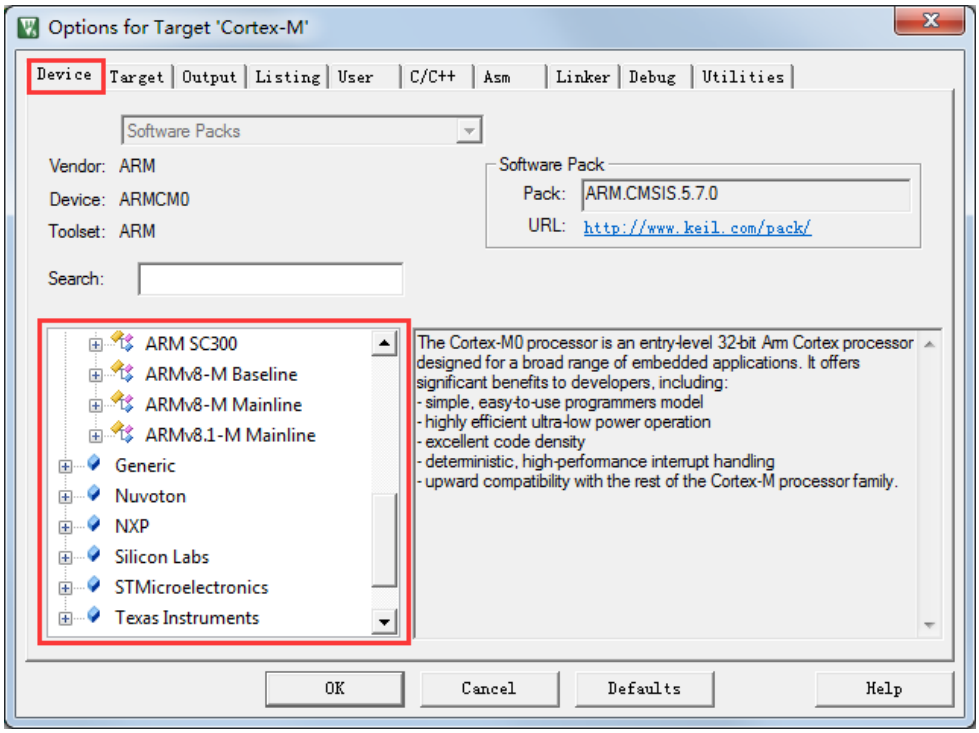


81.4.2 第 2 步，修改工程名

STM32CubeProg 提供的工程模板原始名字是 M25P64_STM3210E-EVAL.uvproj，大家可以根据自己的需要做修改。比如修改为 MyDevice.uvproj（MDK4 的后缀）或者 MyDevice.uvprojx（MDK5 的后缀）。

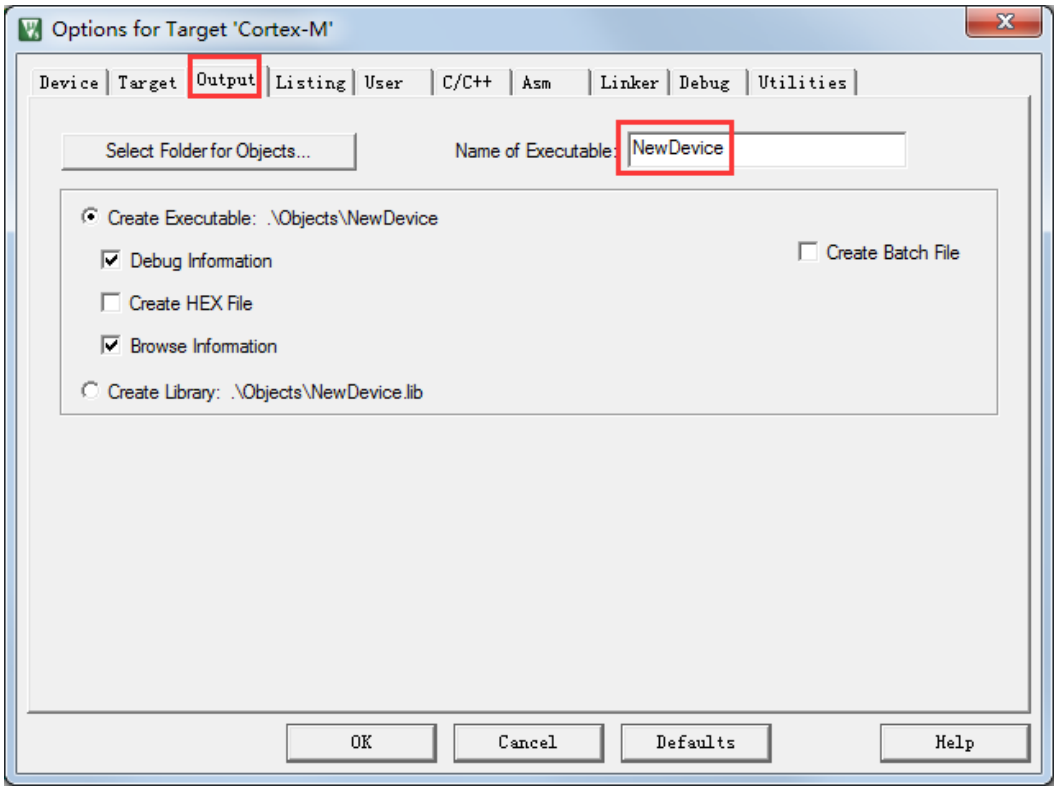
81.4.3 第 3 步，修改使用的器件

在 MDK 的 Option 选项里面设置使用的器件。

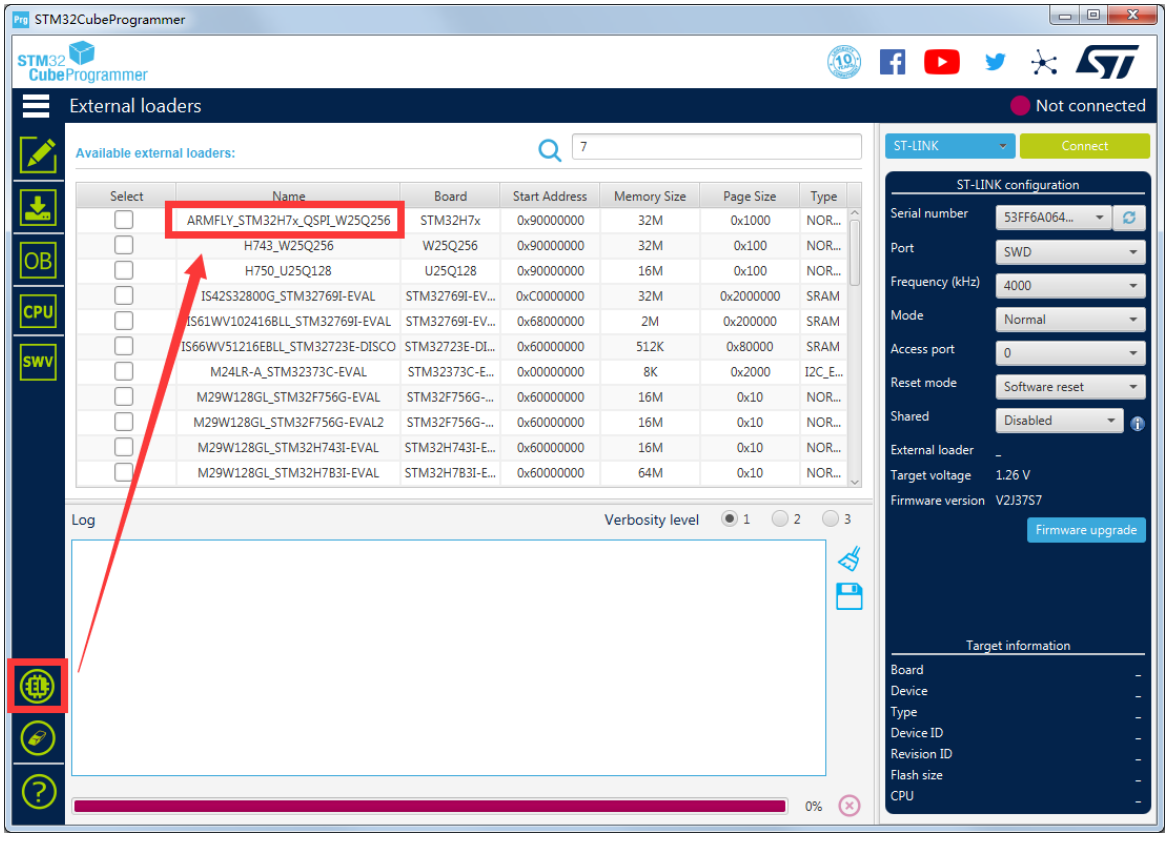


81.4.4第 4 步，修改输出算法文件的名字

这个名字是方便用户查看的，比如设置为 stm32h7，那么输出的算法文件就是 stm32h7.stldr。



注:STM32CubeProg 软件里面别的文件名并不采用这个,而是由用户在文件 Dev_Inf.c 里面定义的:



81.4.5第 5 步，修改使用的库文件和头文件路径

根据大家使用的主控芯片，添加相应的库文件和头文件路径。

81.4.6第 5 步，修改编程算法文件 Loader_Src.c

模板工程里面提供的是 M25P64，部分代码如下：

```
/**
  * Description :
  * Initilize the MCU Clock, the GPIO Pins corresponding to the
  * device and initilize the FSMC with the chosen configuration
  * Inputs    :
  *      None
  * outputs   :
  *      R0             : "1"             : Operation succeeded
  *             "0"             : Operation failure
  * Note: Mandatory for all types of device
  */
int Init (void)
{
  SystemInit();
  sFLASH_Init();
  return 1;
}


/**
  * Description :
  * Read data from the device
  * Inputs    :
  *      Address       : Write location
  *      Size          : Length in bytes
  *      buffer        : Address where to get the data to write
  * outputs   :
  *      R0             : "1"             : Operation succeeded
  *             "0"             : Operation failure
  * Note: Mandatory for all types except SRAM and PSRAM
  */
int Read (uint32_t Address, uint32_t Size, uint8_t* buffer)
{
  sFLASH_ReadBuffer(buffer, Address, Size);
  return 1;
}


/**
  * Description :
  * Write data from the device
  * Inputs    :
  *      Address       : Write location
  *      Size          : Length in bytes
  *      buffer        : Address where to get the data to write
  * outputs   :
```

```
  *      RO           : "1"              : Operation succeeded
  *                     "0"              : Operation failure
  * Note: Mandatory for all types except SRAM and PSRAM
  */
int Write (uint32_t Address, uint32_t Size, uint8_t* buffer)
{
  sFLASH_WriteBuffer(buffer, Address, Size);
  return 1;
}


/**
  * Description :
  * Erase a full sector in the device
  * Inputs    :
  *     None
  * outputs   :
  *     RO             : "1" : Operation succeeded
  *            "0" : Operation failure
  * Note: Not Mandatory for SRAM PSRAM and NOR_FLASH
  */
int MassErase (void)
{
  sFLASH_EraseBulk();
  return 1;
}
```

81.4.7第 6 步，修改配置文件 Dev_Inf.c

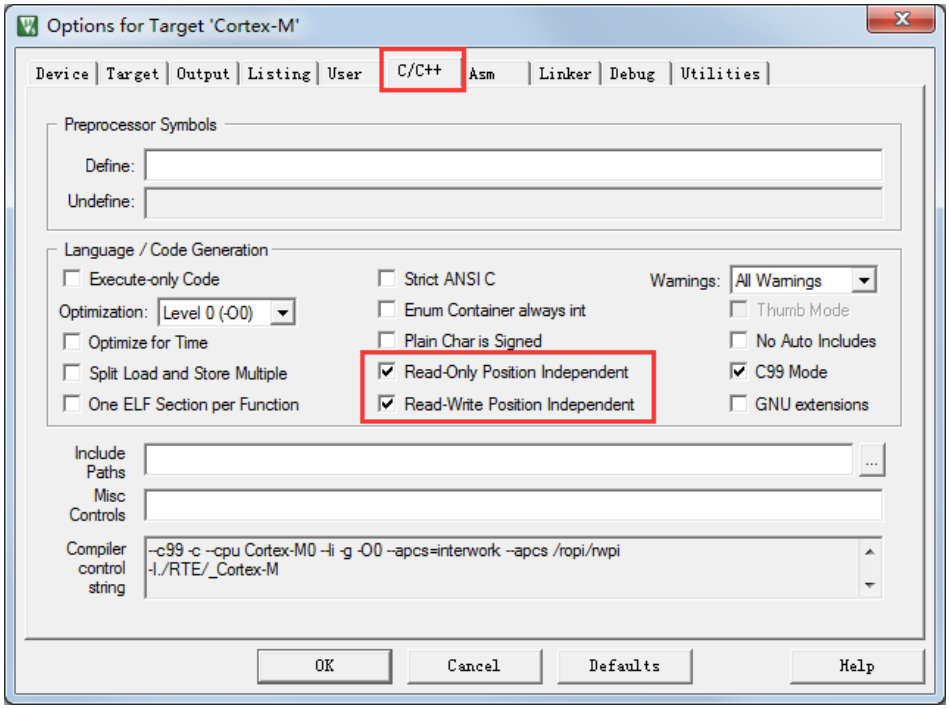
模板工程里面提供简单的配置说明：

```
#if defined (__ICCARM__)
__root struct StorageInfo const StorageInfo  =  {
#else
struct StorageInfo const StorageInfo  =  {
#endif
   "M25P64_STM3210E-EVAL",        // Device Name + version number
   SPI_FLASH,                     // Device Type
   0x00000000,                    // Device Start Address
   0x00800000,                    // Device Size in Bytes (8MBytes/64Mbits)
   0x00000100,                    // Programming Page Size 16Bytes
   0xFF,                          // Initial Content of Erased Memory
// Specify Size and Address of Sectors (view example below)
   0x00000080, 0x00010000,        // Sector Num : 128 ,Sector Size: 64KBytes
   0x00000000, 0x00000000,
};
```

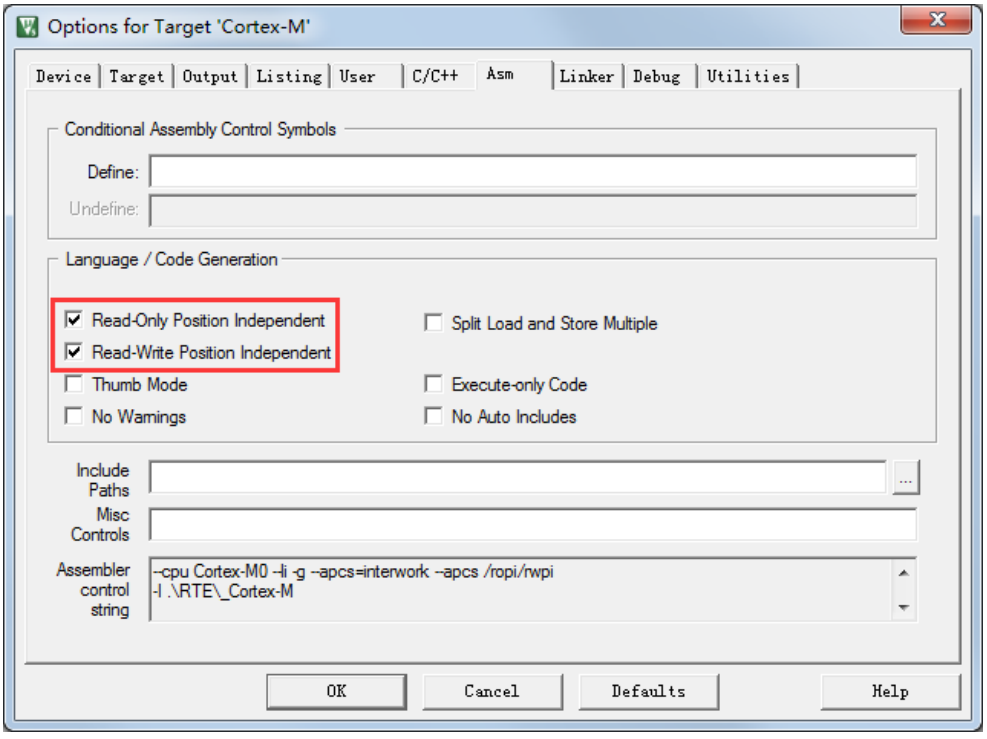
注：名字 M25P64_STM3210E-EVAL 就是我们第 4 步所说的。STM32CubeProg 会识别出这个名字。

81.4.8第 7 步，保证生成的算法文件中 RO 和 RW 段的独立性，即与地址无关。

C 和汇编的配置都勾选上：



汇编：



如果程序的所有只读段都与位置无关，则该程序为只读位置无关（ROPI, Read-only position independence）。ROPI 段通常是位置无关代码（PIC,position-independent code），但可以是只读数据，也可以是 PIC 和只读数据的组合。选择 “ ROPI”选项，可以避免不得不将代码加载到内存中的特定位置。这对于以下例程特别有用：

（1）加载以响应运行事件。

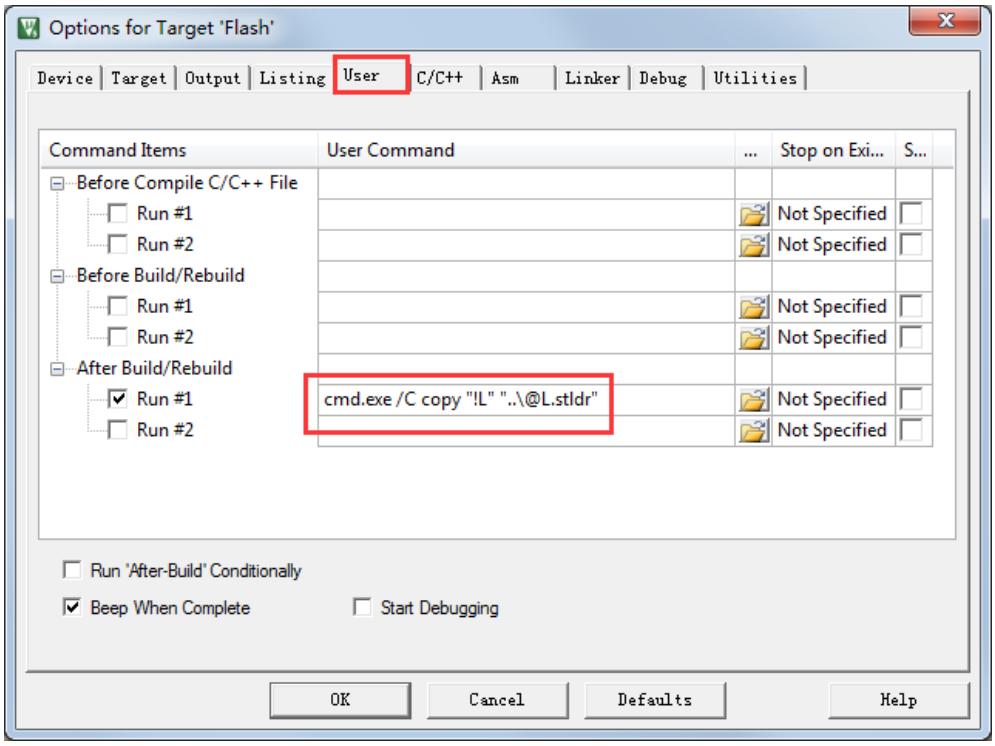
（2）在不同情况下使用其他例程的不同组合加载到内存中。

（3）在执行期间映射到不同的地址。

使用 Read-Write position independence 同理，表示的可读可写数据段。

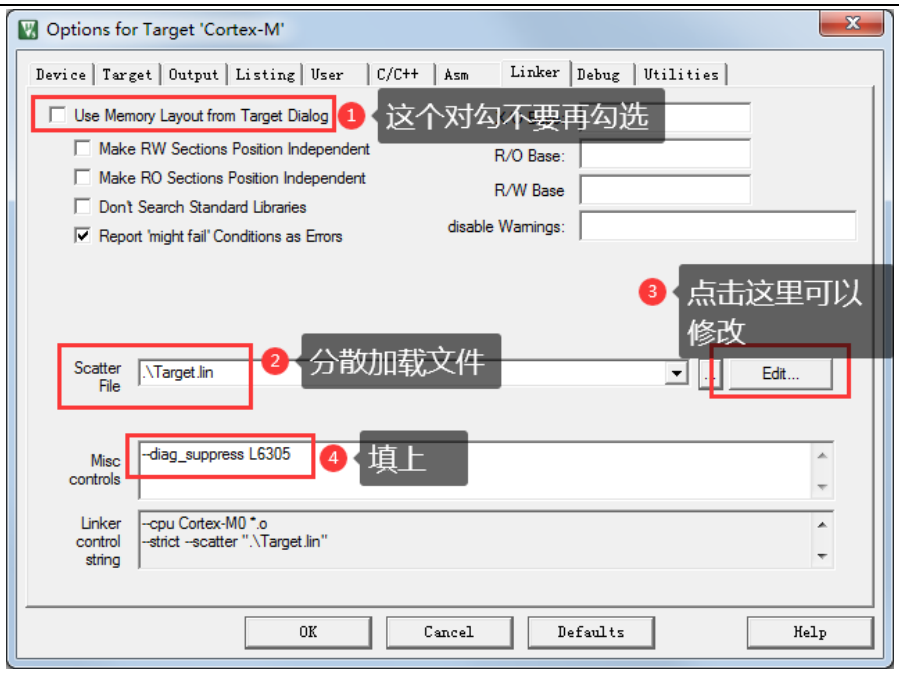
81.4.9第 8 步，将程序可执行文件 axf 修改为 stldr 格式

通过下面的命令就可以将生成的 axf 可执行文件修改为 stldr。



81.4.10 第 9 步，分散加载设置

我们这里的分散加载文件直接使用 MDK 模板工程里提供好的即可。



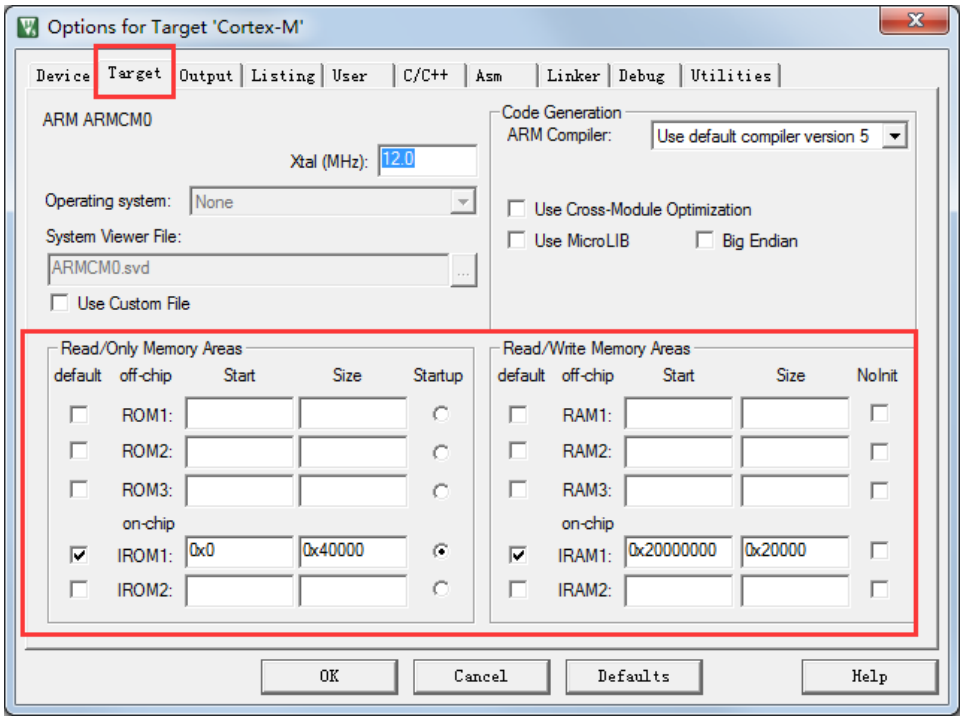
分散加载文件中的内容如下：

```
FLASH_LOADER 0x20000004 PI    ; FlashLoader Functions
{
  PrgCode +0            ; Code
  {
    * (+RO)
  }
  PrgData +0            ; Data
  {
    * (+RW,+ZI)
  }
}

DEVICE_INFO +0                ; Device Info
{
  DevInfo +0            ; Info structure
  {
    dev_inf.o
  }
}
```

这里要修改下 Flash 算法加载地址，将 0x20000004 修改为 STM32H7 的 RAM 地址，任何 RAM 块地址均可，只要够存储 Flash 算法。推荐设置为 AXI SRAM 地址 0x24000004，因为空间够大，有 512KB。

--diag_suppress L6305 用于屏蔽 L6503 类型警告信息。

特别注意，设置了分散加载后，此处的配置就不再起作用了：



81.5 QSPI Flash 的 STM32CubeProg 下载算法制作

下面将 QSPI Flash 算法制作过程中的几个关键点为大家做个说明。

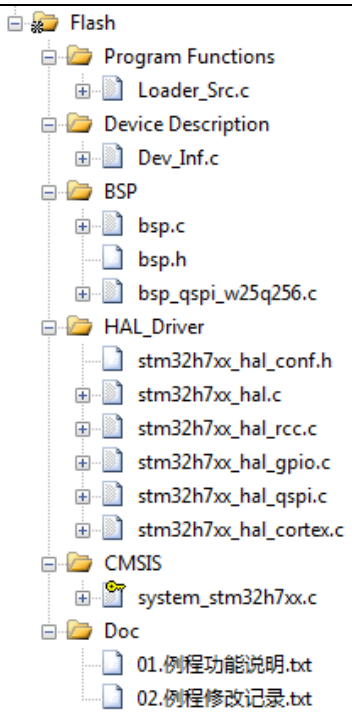
81.5.1 第 1 步，制作前重要提示

这两点非常重要：

- 程序里面不要开启任何中断，全部查询方式。
- HAL 库里面各种时间基准相关的 API 全部处理掉。简单省事些，我们这里是直接注释，采用死等即可。无需做超时等待，因为超时后，已经意味着操作失败了，跟死等没有区别。

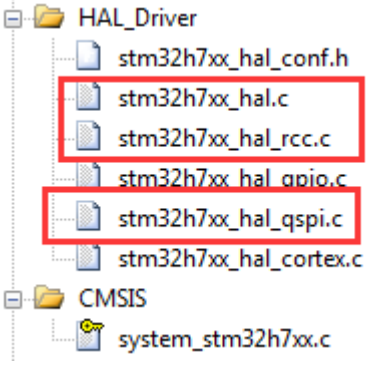
81.5.2 第 2 步，准备一个工程模板

推荐大家直接使用我们本章工程准备好的模板即可，如果大家自己制作，注意一点，请使用当前最新的 HAL 库。



81.5.3第 3 步，修改 HAL 库

这一步比较重要，主要修改了以下三个文件：



主要是修改了 HAL 库时间基准相关的几个 API，并注释掉了一批无关的 API。具体修改内容，大家可以找个比较软件，对比修改后的这个文件和 CubeH7 软件包 V1.8.0（软件包里面的 HAL 库版本是 V1.9.0）的差异即可。

81.5.4第 4 步，时钟初始化

我们已经用不到滴答定时器了，直接在 bsp.c 文件里面对滴答初始化函数做重定向：

```
/*
*********************************************************************************************************
*    函 数 名: HAL_InitTick
*    功能说明: 重定向，不使用
*    形    参: TickPriority
*    返 回 值: 无
```

```
*****************************************************************************************************
*/
HAL_StatusTypeDef HAL_InitTick(uint32_t TickPriority)
{
    return HAL_OK;
}
```

然后就是 HSE 外置晶振的配置，大家根据自己的板子实际外挂晶振大小，修改 stm32h7xx_hal_conf.h 文件中 HSE_VALUE 大小，实际晶振多大，这里就修改为多大：

```
#if !defined  (HSE_VALUE)
#define HSE_VALUE    ((uint32_t)25000000) /*!< Value of the External oscillator in Hz */
#endif /* HSE_VALUE */
```

最后修改 PLL：

```
/*
*****************************************************************************************************
*    函 数 名: SystemClock_Config
*    功能说明: 初始化系统时钟
*            System Clock source            = PLL (HSE)
*            SYSCLK(Hz)                     = 400000000 (CPU Clock)
*            HCLK(Hz)                       = 200000000 (AXI and AHBs Clock)
*            AHB Prescaler                  = 2
*            D1 APB3 Prescaler              = 2 (APB3 Clock  100MHz)
*            D2 APB1 Prescaler              = 2 (APB1 Clock  100MHz)
*            D2 APB2 Prescaler              = 2 (APB2 Clock  100MHz)
*            D3 APB4 Prescaler              = 2 (APB4 Clock  100MHz)
*            HSE Frequency(Hz)              = 25000000
*            PLL_M                          = 5
*            PLL_N                          = 160
*            PLL_P                          = 2
*            PLL_Q                          = 4
*            PLL_R                          = 2
*            VDD(V)                         = 3.3
*            Flash Latency(WS)              = 4
*    形    参: 无
*    返 回 值: 1 表示失败，0 表示成功
*****************************************************************************************************
*/
int SystemClock_Config(void)
{
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    HAL_StatusTypeDef ret = HAL_OK;

    /* 锁住 SCU(Supply configuration update) */
    MODIFY_REG(PWR->CR3, PWR_CR3_SCUEN, 0);

    /*
      1、芯片内部的 LDO 稳压器输出的电压范围，可选 VOS1，VOS2 和 VOS3，不同范围对应不同的 Flash 读速度，
         详情看参考手册的 Table 12 的表格。
      2、这里选择使用 VOS1，电压范围 1.15V - 1.26V。
    */
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

    while(!__HAL_PWR_GET_FLAG(PWR_FLAG_VOSRDY)) {}

    /* 使能 HSE，并选择 HSE 作为 PLL 时钟源 */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
```

```
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.HSIState = RCC_HSI_OFF;
    RCC_OscInitStruct.CSIState = RCC_CSI_OFF;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;

    RCC_OscInitStruct.PLL.PLLM = 5;
    RCC_OscInitStruct.PLL.PLLN = 160;
    RCC_OscInitStruct.PLL.PLLP = 2;
    RCC_OscInitStruct.PLL.PLLR = 2;
    RCC_OscInitStruct.PLL.PLLQ = 4;

    RCC_OscInitStruct.PLL.PLLVCOSEL = RCC_PLL1VCOWIDE;
    RCC_OscInitStruct.PLL.PLLRGE = RCC_PLL1VCIRANGE_2;
    ret = HAL_RCC_OscConfig(&RCC_OscInitStruct);
    if(ret != HAL_OK)
    {
        return 1;
    }

    /*
      选择 PLL 的输出作为系统时钟
      配置 RCC_CLOCKTYPE_SYSCLK 系统时钟
      配置 RCC_CLOCKTYPE_HCLK 时钟，对应 AHB1，AHB2，AHB3 和 AHB4 总线
      配置 RCC_CLOCKTYPE_PCLK1 时钟，对应 APB1 总线
      配置 RCC_CLOCKTYPE_PCLK2 时钟，对应 APB2 总线
      配置 RCC_CLOCKTYPE_D1PCLK1 时钟，对应 APB3 总线
      配置 RCC_CLOCKTYPE_D3PCLK1 时钟，对应 APB4 总线
    */
    RCC_ClkInitStruct.ClockType = (RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_D1PCLK1 |
RCC_CLOCKTYPE_PCLK1 | \
                                   RCC_CLOCKTYPE_PCLK2  | RCC_CLOCKTYPE_D3PCLK1);

    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.SYSCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB3CLKDivider = RCC_APB3_DIV2;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_APB1_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_APB2_DIV2;
    RCC_ClkInitStruct.APB4CLKDivider = RCC_APB4_DIV2;

    /* 此函数会更新 SystemCoreClock，并重新配置 HAL_InitTick */
    ret = HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4);
    if(ret != HAL_OK)
    {
        return 1;
    }

    /*
      使用 IO 的高速模式，要使能 IO 补偿，即调用下面三个函数
      （1）使能 CSI clock
      （2）使能 SYSCFG clock
      （3）使能 I/O 补偿单元， 设置 SYSCFG_CCCSR 寄存器的 bit0
    */
    __HAL_RCC_CSI_ENABLE() ;

    __HAL_RCC_SYSCFG_CLK_ENABLE() ;
```

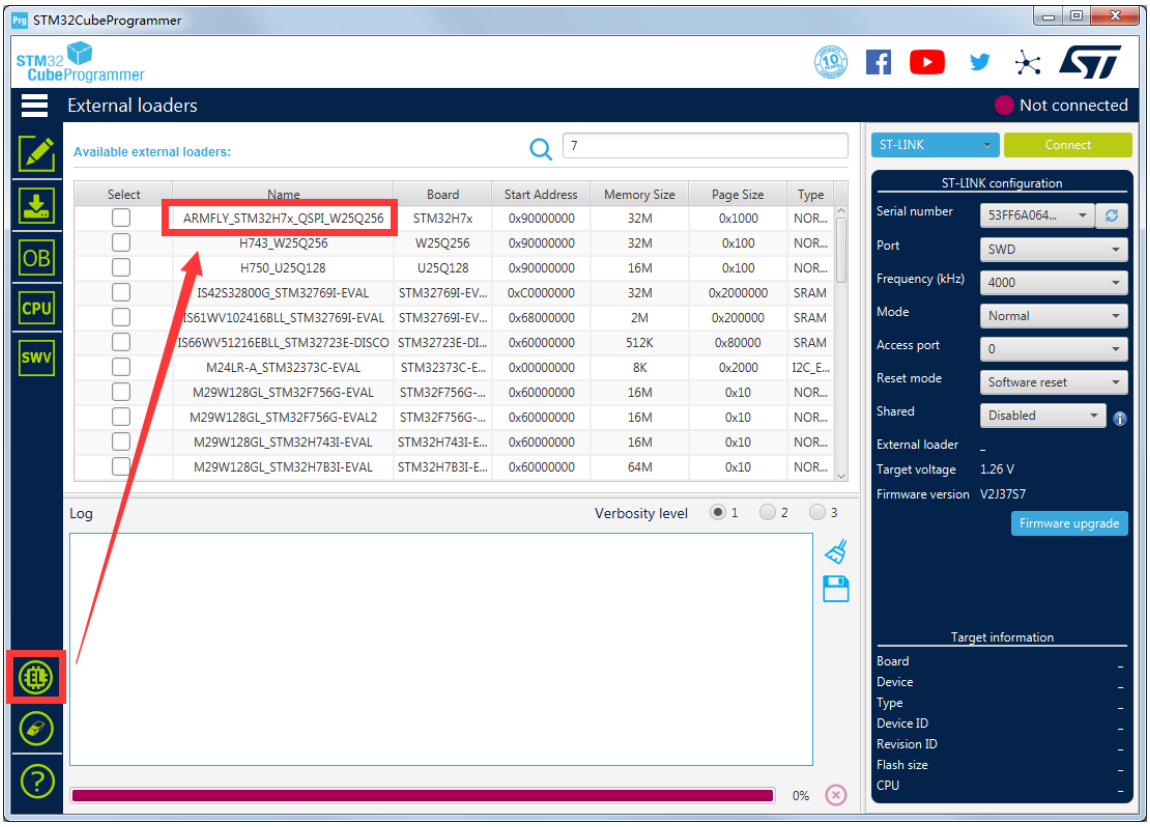
```
    HAL_EnableCompensationCell();

    __HAL_RCC_D2SRAM1_CLK_ENABLE();
    __HAL_RCC_D2SRAM2_CLK_ENABLE();
    __HAL_RCC_D2SRAM3_CLK_ENABLE();

    return 0;
}
```

81.5.5 第 5 步，配置文件 Dev_Inf.c 的实现

配置如下：

```
#if defined (__ICCARM__)
__root struct StorageInfo const StorageInfo  =  {
#else
struct StorageInfo const StorageInfo =  {
#endif
    "ARMFLY_STM32H7x_QSPI_W25Q256", /* 算法名，添加算法到 STM32CubeProg 安装目录会显示此名字 */
    NOR_FLASH,                      /* 设备类型 */
    0x90000000,                     /* Flash 起始地址 */
    32 * 1024 * 1024,               /* Flash 大小，32MB */
    4*1024,                         /* 编程页大小 */
    0xFF,                           /* 擦除后的数值 */
    512 , 64 * 1024,                /* 块个数和块大小 */
    0x00000000, 0x00000000,
};
```

注释已经比较详细，大家根据自己的需要做修改即可。注意一点，算法名 ARMFLY_STM32H7x_QSPI_W25Q256 会反馈到这个地方：



81.5.6第 6 步，编程文件 Loader_Src.c 的实现

下面将变成文件中实现的几个函数为大家做个说明：

◆ **初始化函数 Init**

```
/*
*********************************************************************************************************
*    函 数 名: Init
*    功能说明: Flash 编程初始化
*    形    参: 无
*    返 回 值: 0 表示失败， 1 表示成功
*********************************************************************************************************
*/
int Init(void)
{
    int result = 1;

    /* 系统初始化 */
    SystemInit();

    /* 时钟初始化 */
    result = SystemClock_Config();
    if (result == 1)
    {
        return 0;
    }

    /* W25Q256 初始化 */
    result = bsp_InitQSPI_W25Q256();
    if (result == 1)
    {
        return 0;
    }

    /* 内存映射 */
    result = QSPI_MemoryMapped();
    if (result == 1)
    {
        return 0;
    }

    return 1;
}
```

初始化完毕后将其设置为内存映射模式。

◆ **整个芯片擦除函数 MassErase**

整个芯片的擦除实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: MassErase
*    功能说明: 整个芯片擦除
*    形    参: 无
*    返 回 值: 1 表示成功，0 表示失败
*********************************************************************************************************
*/
```

```
int MassErase(void)
{
    int result = 1;

    /* W25Q256 初始化 */
    result = bsp_InitQSPI_W25Q256();
    if (result == 1)
    {
        return 0;
    }

    result = QSPI_EraseChip();
    if (result == 1)
    {
        return 0;
    }

    /* 内存映射 */
    result = QSPI_MemoryMapped();
    if (result == 1)
    {
        return 0;
    }

    return result;
}
```

◆ 扇区擦除函数 SectorErase

扇区擦除实现：

```
/*
*********************************************************************************************************
*    函 数 名: SectorErase
*    功能说明: EraseStartAddress 擦除起始地址
*              EraseEndAddress   擦除结束地址
*    形    参: adr 擦除地址
*    返 回 值: 1 表示成功，0 表示失败
*********************************************************************************************************
*/
int SectorErase (uint32_t EraseStartAddress ,uint32_t EraseEndAddress)
{
    uint32_t BlockAddr, result;

    EraseStartAddress -= QSPI_FLASH_MEM_ADDR;
    EraseEndAddress -= QSPI_FLASH_MEM_ADDR;
    EraseStartAddress = EraseStartAddress -  EraseStartAddress % 0x10000; /* 64KB 首地址 */

    /* W25Q256 初始化 */
    result = bsp_InitQSPI_W25Q256();
    if (result == 1)
    {
        return 0;
    }

    while (EraseEndAddress >= EraseStartAddress)
    {
        /* 防止超出 256MB 空间 */
```

```
        BlockAddr = EraseStartAddress & 0x0FFFFFFF;

        result = QSPI_EraseSector(BlockAddr);
        if (result == 1)
        {
            QSPI_MemoryMapped();
            return 0;
        }

        EraseStartAddress += 0x10000;
    }

    /* 内存映射 */
    result = QSPI_MemoryMapped();
    if (result == 1)
    {
        return 0;
    }

    return 1;
}
```

这里要注意两点：

（1） 程序里面的操作 EraseStartAddress-= QSPI_FLASH_MEM_ADDR，实际传递进来的地址是带了首地址的，即 0x90000000。

（2） 这里执行的擦除大小要前面 Dev_Inf.c 文件中配置的扇区大小一致，这里是执行的 64KB 为扇区进行擦除。

◆ 页编程函数 Write

页编程函数实现如下：

```
int Write(uint32_t Address, uint32_t Size, uint8_t* buffer)
{
    int result = 0;
    uint32_t end_addr, current_size, current_addr;


    if (Address < QSPI_FLASH_MEM_ADDR || Address >= QSPI_FLASH_MEM_ADDR + QSPI_FLASH_SIZES)
    {
        return 0;
    }

    Address -= QSPI_FLASH_MEM_ADDR;

    /* 计算写入地址和页末的字节大小 */
    current_size = QSPI_PAGE_SIZE - (Address % QSPI_PAGE_SIZE);

    /* 检测页剩余空间是否够用 */
    if (current_size > Size)
    {
        current_size = Size;
    }

    /* W25Q256 初始化 */
    result = bsp_InitQSPI_W25Q256();
```

```
    if (result == 1)
    {
        return 0;
    }

    current_addr = Address;
    end_addr = Address + Size;

    do{
        if (QSPI_WriteBuffer(buffer, current_addr, current_size) == 1)
        {
            QSPI_MemoryMapped();
            return 0;
        }

        /* 更新地址 */
        current_addr += current_size;
        buffer += current_size;
        current_size = ((current_addr + QSPI_PAGE_SIZE) > end_addr) ? (end_addr - current_addr) : QSPI_PAGE_SIZE;

    }while(current_addr < end_addr);

    /* 内存映射 */
    result = QSPI_MemoryMapped();
    if (result == 1)
    {
        return 0;
    }

    return (1);
}
```

这里注意两点：

（1） W25Q256 的页大小是 256 字节，前面 FlashDev.c 中将页编程大小设置为 4096 字节，所以此程序要做处理。

（2） 程序里面的操作 Address-= QSPI_FLASH_MEM_ADDR，实际传递进来的地址是带了首地址的，即 0x90000000。

◆ **读取和校验函数**

我们程序中未做读取和校验函数。

（1） 如果程序中未做读取函数，那么 STM32CubeProg 会以总线方式进行读取，这也是为什么每个函数执行完毕都设置为内存映射模式的原因。

（2） 如果程序中未做校验函数，那么 STM32CubeProg 会读取数据做校验。

81.5.7第 7 步，修改 QSPI Flash 驱动文件（引脚，命令等）

最后一步就是 QSPI Flash（W25Q256）的驱动修改，大家可以根据自己的需求做修改。使用的引脚定义在文件 bsp_qspi_w25q256.c（做了条件编译，包含了 H7-TOOL 和 STM32-V7 板子）：

```
/*
    STM32-V7 开发板接线
```

```
    PG6/QUADSPI_BK1_NCS     AF10
    PF10/QUADSPI_CLK        AF9
    PF8/QUADSPI_BK1_IO0     AF10
    PF9/QUADSPI_BK1_IO1     AF10
    PF7/QUADSPI_BK1_IO2     AF9
    PF6/QUADSPI_BK1_IO3     AF9

    W25Q256JV 有 512 块，每块有 16 个扇区，每个扇区 Sector 有 16 页，每页有 256 字节，共计 32MB

    H7-TOOL 开发板接线

    PG6/QUADSPI_BK1_NCS     AF10
    PB2/QUADSPI_CLK         AF9
    PD11/QUADSPI_BK1_IO0    AF10
    PD12/QUADSPI_BK1_IO1    AF10
    PF7/QUADSPI_BK1_IO2     AF9
    PD13/QUADSPI_BK1_IO3    AF9
*/

/* QSPI 引脚和时钟相关配置宏定义 */
#if 0
#define QSPI_CLK_ENABLE()               __HAL_RCC_QSPI_CLK_ENABLE()
#define QSPI_CLK_DISABLE()              __HAL_RCC_QSPI_CLK_DISABLE()
#define QSPI_CS_GPIO_CLK_ENABLE()       __HAL_RCC_GPIOG_CLK_ENABLE()
#define QSPI_CLK_GPIO_CLK_ENABLE()      __HAL_RCC_GPIOB_CLK_ENABLE()
#define QSPI_BK1_D0_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOD_CLK_ENABLE()
#define QSPI_BK1_D1_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOD_CLK_ENABLE()
#define QSPI_BK1_D2_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOF_CLK_ENABLE()
#define QSPI_BK1_D3_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOD_CLK_ENABLE()

#define QSPI_MDMA_CLK_ENABLE()          __HAL_RCC_MDMA_CLK_ENABLE()
#define QSPI_FORCE_RESET()              __HAL_RCC_QSPI_FORCE_RESET()
#define QSPI_RELEASE_RESET()            __HAL_RCC_QSPI_RELEASE_RESET()

#define QSPI_CS_PIN                     GPIO_PIN_6
#define QSPI_CS_GPIO_PORT               GPIOG
#define QSPI_CS_GPIO_AF                 GPIO_AF10_QUADSPI

#define QSPI_CLK_PIN                    GPIO_PIN_2
#define QSPI_CLK_GPIO_PORT              GPIOB
#define QSPI_CLK_GPIO_AF                GPIO_AF9_QUADSPI

#define QSPI_BK1_D0_PIN                 GPIO_PIN_11
#define QSPI_BK1_D0_GPIO_PORT           GPIOD
#define QSPI_BK1_D0_GPIO_AF             GPIO_AF9_QUADSPI

#define QSPI_BK1_D1_PIN                 GPIO_PIN_12
#define QSPI_BK1_D1_GPIO_PORT           GPIOD
#define QSPI_BK1_D1_GPIO_AF             GPIO_AF9_QUADSPI

#define QSPI_BK1_D2_PIN                 GPIO_PIN_7
#define QSPI_BK1_D2_GPIO_PORT           GPIOF
#define QSPI_BK1_D2_GPIO_AF             GPIO_AF9_QUADSPI

#define QSPI_BK1_D3_PIN                 GPIO_PIN_13
#define QSPI_BK1_D3_GPIO_PORT           GPIOD
#define QSPI_BK1_D3_GPIO_AF             GPIO_AF9_QUADSPI
```

```
#else
#define QSPI_CLK_ENABLE()               __HAL_RCC_QSPI_CLK_ENABLE()
#define QSPI_CLK_DISABLE()              __HAL_RCC_QSPI_CLK_DISABLE()
#define QSPI_CS_GPIO_CLK_ENABLE()       __HAL_RCC_GPIOG_CLK_ENABLE()
#define QSPI_CLK_GPIO_CLK_ENABLE()      __HAL_RCC_GPIOF_CLK_ENABLE()
#define QSPI_BK1_DO_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOF_CLK_ENABLE()
#define QSPI_BK1_D1_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOF_CLK_ENABLE()
#define QSPI_BK1_D2_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOF_CLK_ENABLE()
#define QSPI_BK1_D3_GPIO_CLK_ENABLE()   __HAL_RCC_GPIOF_CLK_ENABLE()

#define QSPI_MDMA_CLK_ENABLE()          __HAL_RCC_MDMA_CLK_ENABLE()
#define QSPI_FORCE_RESET()              __HAL_RCC_QSPI_FORCE_RESET()
#define QSPI_RELEASE_RESET()            __HAL_RCC_QSPI_RELEASE_RESET()

#define QSPI_CS_PIN                     GPIO_PIN_6
#define QSPI_CS_GPIO_PORT               GPIOG
#define QSPI_CS_GPIO_AF                 GPIO_AF10_QUADSPI

#define QSPI_CLK_PIN                    GPIO_PIN_10
#define QSPI_CLK_GPIO_PORT              GPIOF
#define QSPI_CLK_GPIO_AF                GPIO_AF9_QUADSPI

#define QSPI_BK1_DO_PIN                 GPIO_PIN_8
#define QSPI_BK1_DO_GPIO_PORT           GPIOF
#define QSPI_BK1_DO_GPIO_AF             GPIO_AF10_QUADSPI

#define QSPI_BK1_D1_PIN                 GPIO_PIN_9
#define QSPI_BK1_D1_GPIO_PORT           GPIOF
#define QSPI_BK1_D1_GPIO_AF             GPIO_AF10_QUADSPI

#define QSPI_BK1_D2_PIN                 GPIO_PIN_7
#define QSPI_BK1_D2_GPIO_PORT           GPIOF
#define QSPI_BK1_D2_GPIO_AF             GPIO_AF9_QUADSPI

#define QSPI_BK1_D3_PIN                 GPIO_PIN_6
#define QSPI_BK1_D3_GPIO_PORT           GPIOF
#define QSPI_BK1_D3_GPIO_AF             GPIO_AF9_QUADSPI
#endif
```

硬件设置了之后，剩下就是 QSPI Flash 相关的几个配置，在文件 bsp_qspi_w25q256.h：

主要是下面这几个：

```
#define QSPI_FLASH_MEM_ADDR         0x90000000

/* W25Q256JV 基本信息 */
#define QSPI_FLASH_SIZE     25                      /* Flash 大小，2^25 = 32MB*/
#define QSPI_SECTOR_SIZE    (4 * 1024)              /* 扇区大小，4KB */
#define QSPI_PAGE_SIZE      256                     /* 页大小，256 字节 */
#define QSPI_END_ADDR       (1 << QSPI_FLASH_SIZE)  /* 末尾地址 */
#define QSPI_FLASH_SIZES    32 * 1024 * 1024         /* Flash 大小，2^25 = 32MB*/

/* W25Q256JV 相关命令 */
#define WRITE_ENABLE_CMD                        0x06    /* 写使能指令 */
#define READ_ID_CMD2                            0x9F    /* 读取 ID 命令 */
#define READ_STATUS_REG_CMD                     0x05    /* 读取状态命令 */
#define SUBSECTOR_ERASE_4_BYTE_ADDR_CMD         0x21    /* 32bit 地址扇区擦除指令，4KB */
#define QUAD_IN_FAST_PROG_4_BYTE_ADDR_CMD       0x34    /* 32bit 地址的 4 线快速写入命令 */
#define QUAD_INOUT_FAST_READ_4_BYTE_ADDR_CMD    0xEC    /* 32bit 地址的 4 线快速读取命令 */
```

```
#define BLOCK_ERASE_64K_4_BYTE_ADDR_CMD         0xDC    /* 4 字节地址，64K 扇区 */
#define BULK_ERASE_CMD                          0xC7    /* 整片擦除 */
```

81.6 QSPI Flash 的 STM32CubeProg 下载算法使用方法

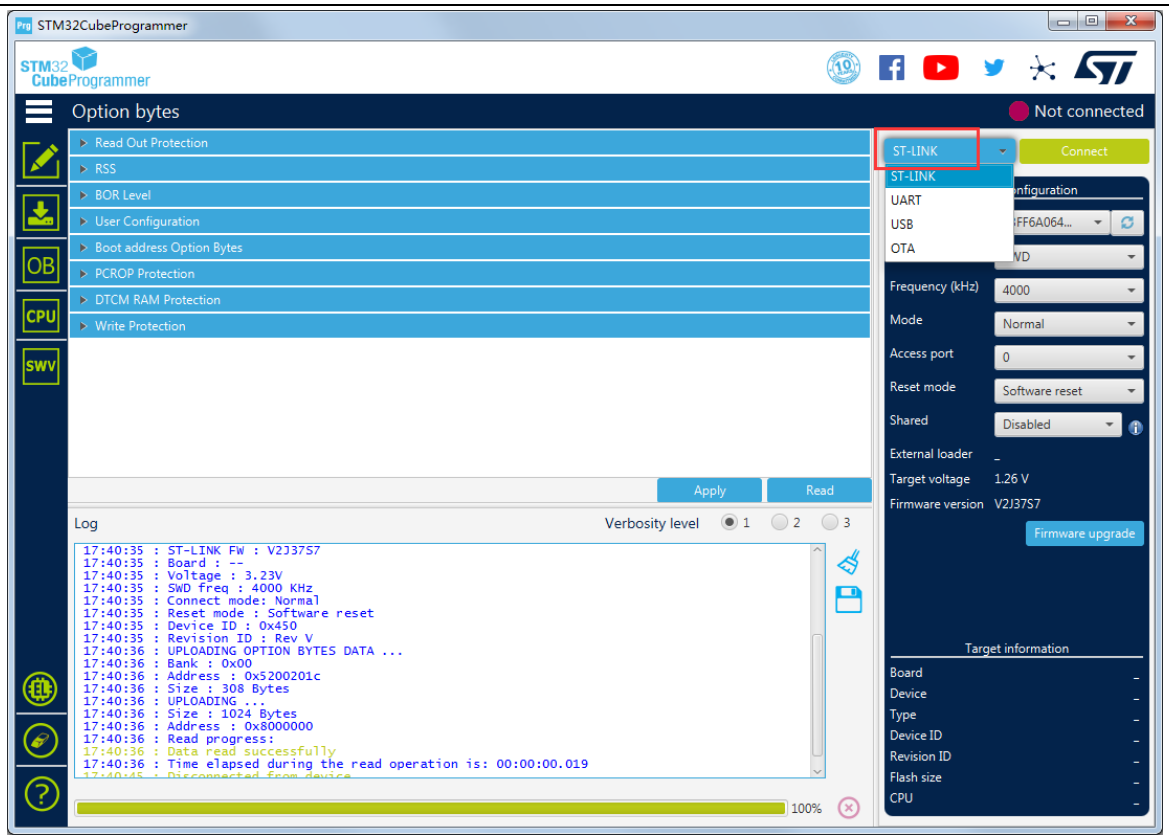
编译本章教程配套的例子，生成的算法文件位于此路径下：

81.6.1 下载算法存放位置

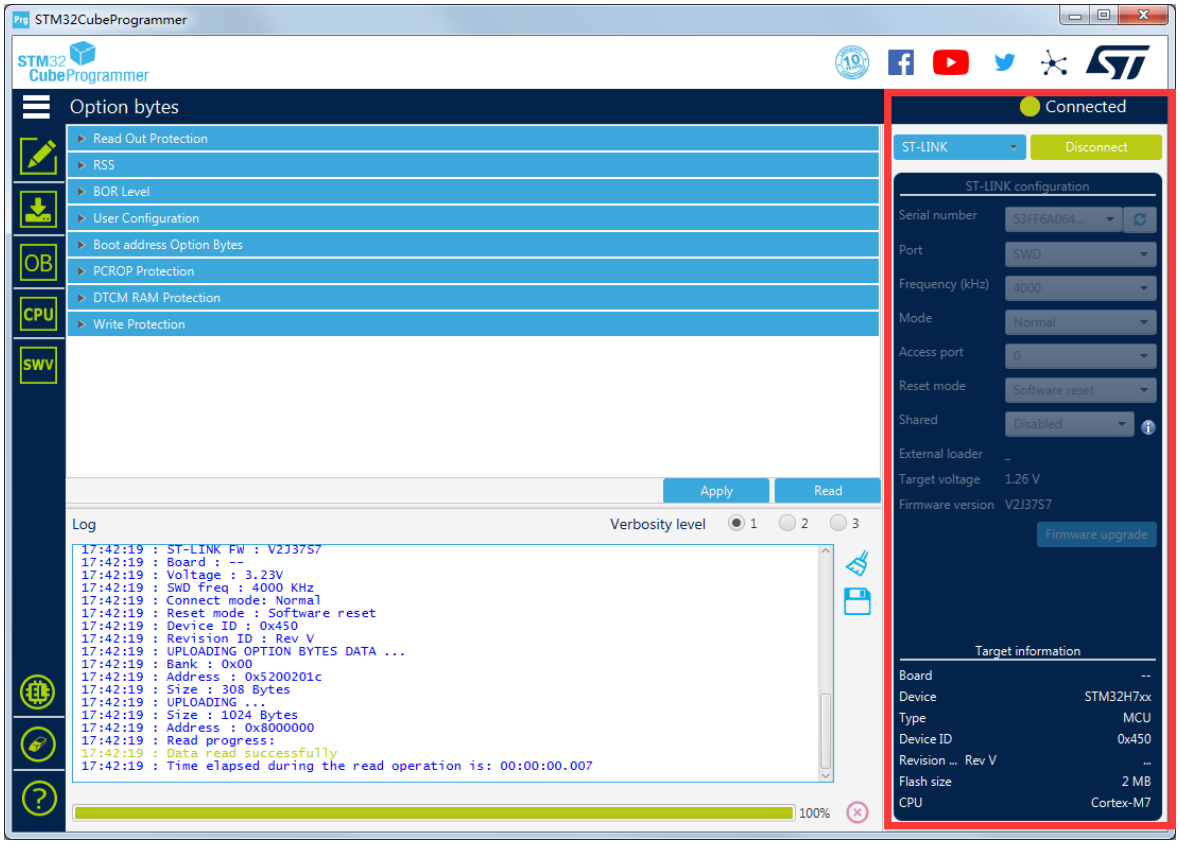
生成此文件后，需要大家将其存放到 STM32CubeProg 安装目录路径：
\STMicroelectronics\STM32Cube\STM32CubeProgrammer\bin\ExternalLoader

81.6.2 STM32CubeProg 下载配置

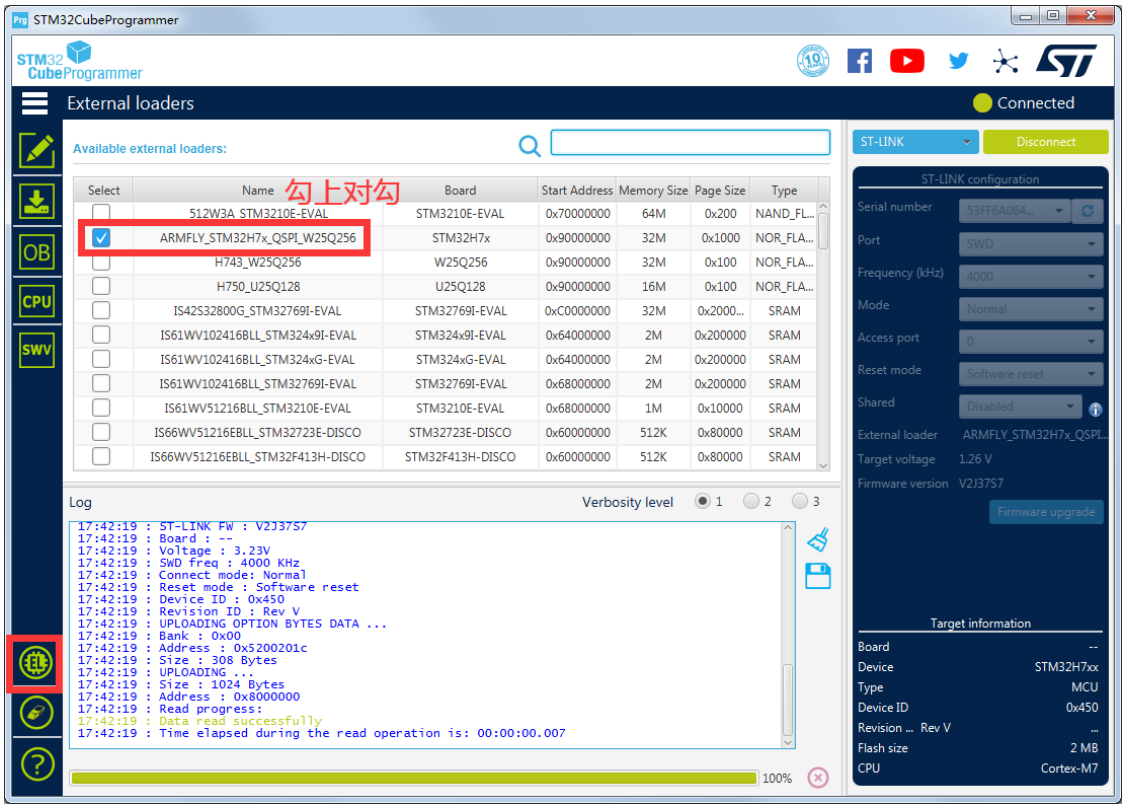
我们这里以 STLINK 连接开发板为例进行说明（USB DFU 或者串口方式不支持下载外部 Flash）。



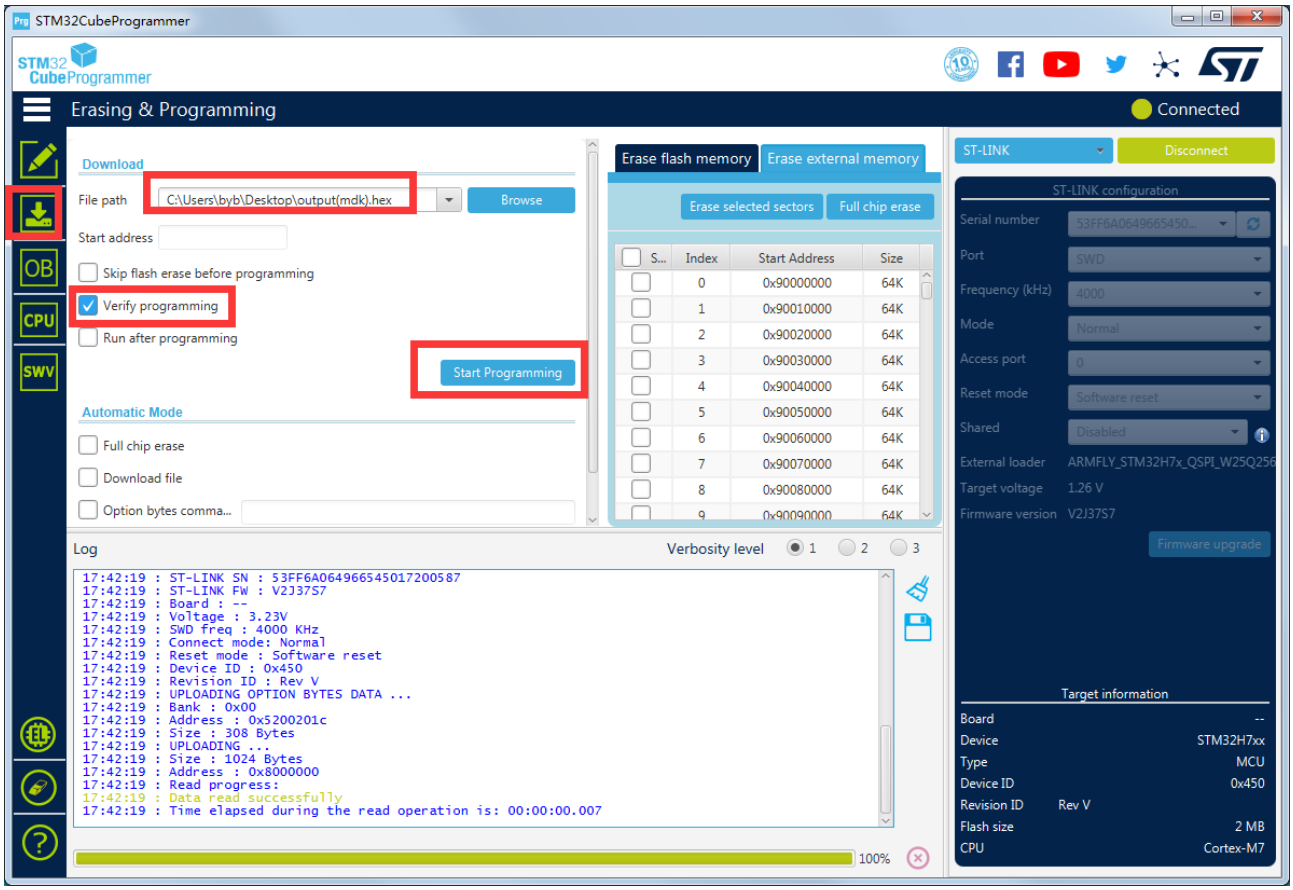
点击 Connect 后效果如下：



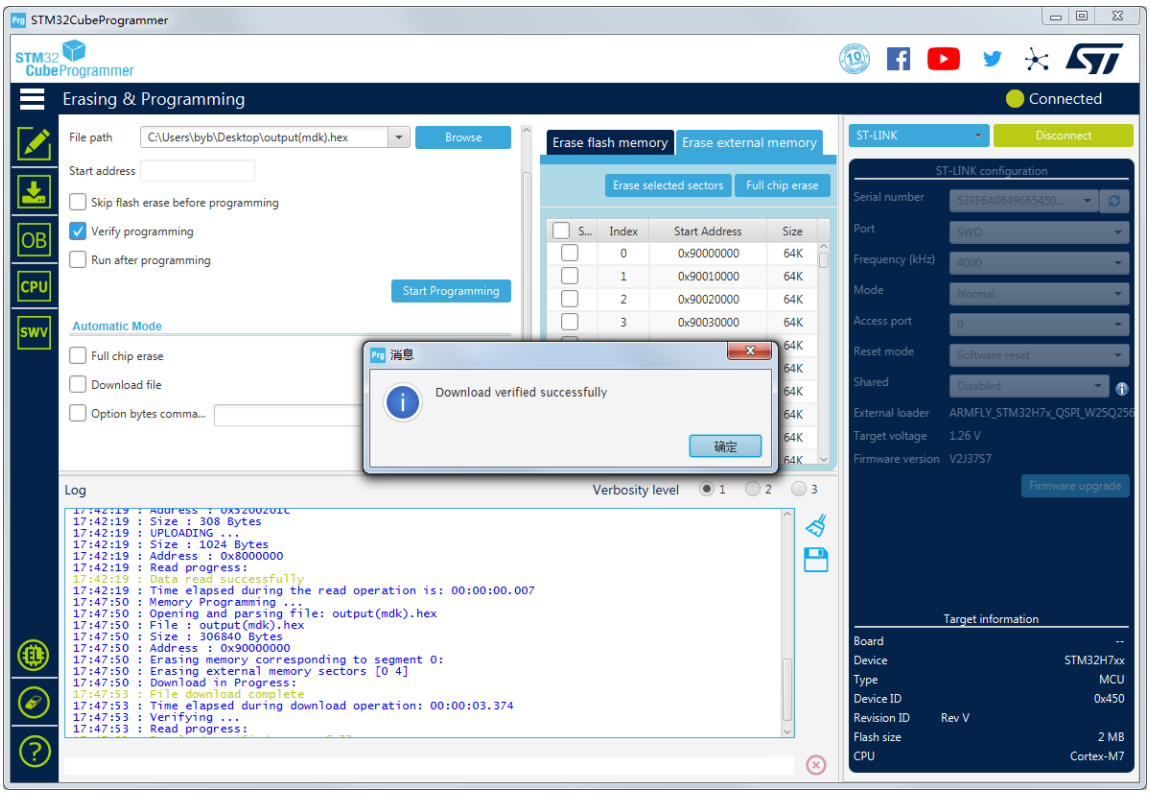
在这里选择我们制作的下载算法：



任意加载个 hex 或者 bin 文件，并按照如下配置，然后点击 Start Programming

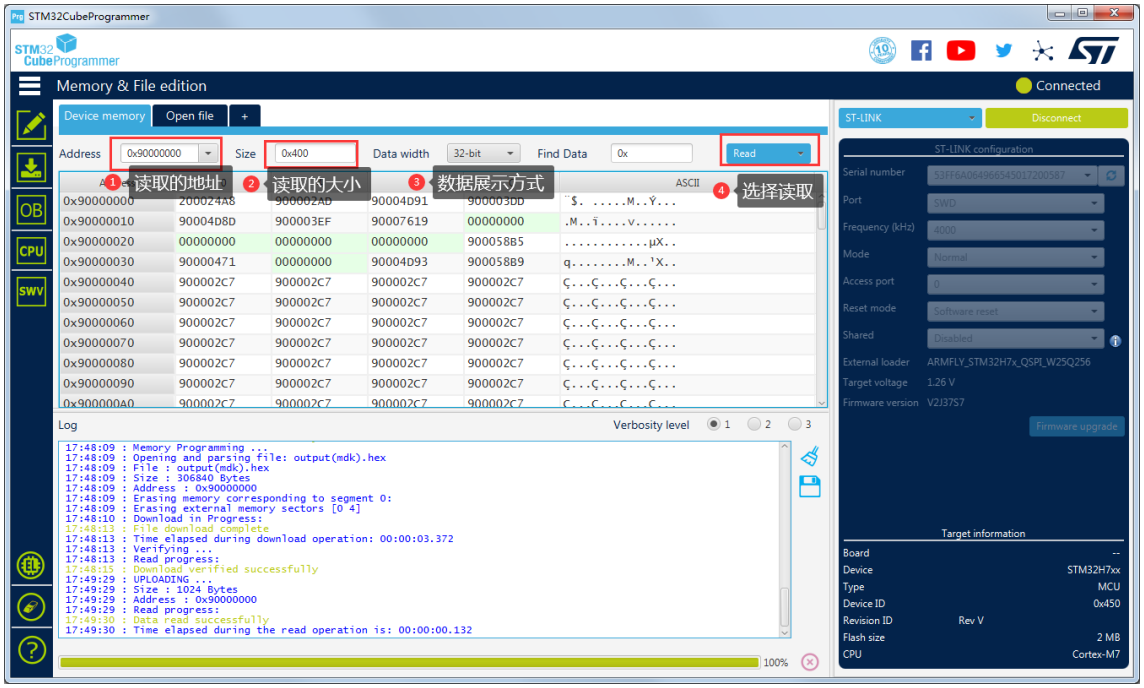


下载完成后的效果如下：



81.6.3验证算法文件是否可以正常使用

为了验证算法文件是否可以正常使用，大家可以运行本教程第 82 章或者 83 章配套的例子。也可以使用 STM32CubeProg 直接读取：



81.7 实验例程说明

本章配套例子：V7-061_QSPI Flash 的 STM32CubeProg 下载算法制作。

编译本章教程配套的例子，生成的算法文件位于此路径下：

81.8 总结

本章节就为大家讲解这么多，为了熟练掌握，大家可以尝试自己实现一个 Flash 下载算法。

第82章 STM32H7 的 QSPI 总线应用之 QSPI Flash 执行程序（BOOT+APP 方式）

本章节为大家讲解 QSPI Flash 执行程序。

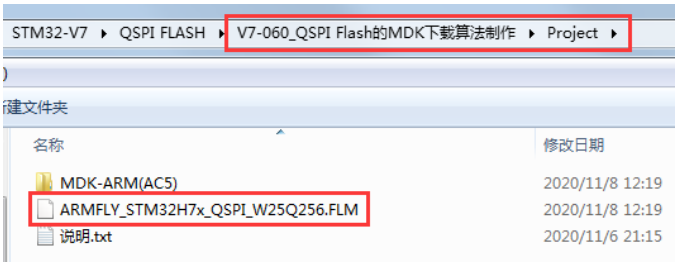
82.1 初学者重要提示
82.2 下载算法存放位置
82.3 QSPI Flash 的 Bootloader 说明
82.4 QSPI Flash 的 APP 应用程序说明
82.5 QSPI Flash 的 APP 应用程序调试下载配置
82.6 实验例程说明
82.7 总结

82.1 初学者重要提示

- QSPI Flash 的相关知识点可以看第 78 章和 79 章。
- 操作本章配套例子前务必先将 QSPI Flash 的下载算法放到 MDK 安装目录。
- QSPI Flash 执行程序的 BOOT 和 APP 作用：
  - BOOT 程序需要下载到内部 Flash。主要用于跳转到外部 QSPI Flash 执行程序。那么问题来了，我们可不可以不使用 BOOT，上电就直接执行 QSPI Flash 的程序？不行，因为 QSPI Flash 不像内部 Flash，无需初始化，上电就可以使用，而 QSPI Flash 不行。
  - APP 应用程序要下载到 QSPI Flash 里面。APP 应用程序可以在 QSPI Flash 里面执行主要是因为 W25Q256 支持 XIP（Execution In Place），并且 STM32H7 的 QSPI Flash 支持内存映射。

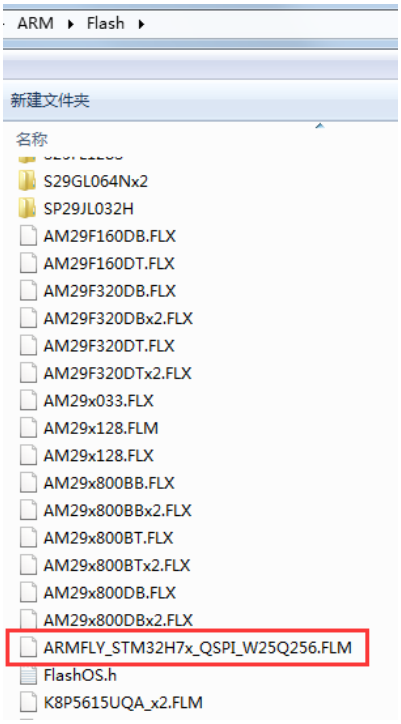
82.2 下载算法存放位置

编译例子：V7-060_QSPI Flash 的 MDK 下载算法制作，生成的算法文件位于此路径下：



生成算法文件后，需要大家将其存到到 MDK 安装目录，有两个位置可以存放，任选其一，推荐第 2 种：

- 第 1 种：存放到 MDK 的 STM32H7 软包安装目录里面：\Keil\STM32H7xx_DFP\2.6.0\CMSIS\Flash（软包版本不同，数值 2.6.0 不同）。
- 第 2 种：MDK 的安装目录 \ARM\Flash 里面。



82.3 QSPI Flash 的 Bootloader 说明

Bootloader 的实现比较简单，需要大家将其下载到内部 Flash。程序实现上主要注意以下两点即可。

82.3.1 初始化 QSPI Flash 并设置内存映射模式

在 bsp.c 文件中初始 QSPI Flash 并设置为内存映射模式。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();
```

```
    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitQSPI_W25Q256();  /* 配置 SPI 总线 */
    QSPI_MemoryMapped();
}
```

82.3.2 程序跳转的实现

从内部 Flash 跳转到外部 QSPI Flash 的实现代码如下

```
107. /*
108. *********************************************************************************************************
109. *    函 数 名: JumpToApp
110. *    功能说明: 跳转到应用 JumpToApp
111. *    形    参: 无
112. *    返 回 值: 无
113. *********************************************************************************************************
114. */
115. static void JumpToApp(void)
116. {
117.     uint32_t i=0;
118.     void (*AppJump)(void);          /* 声明一个函数指针 */
119.     __IO uint32_t AppAddr = 0x90000000;  /* APP 地址 */
120.
121.
122.    /* 关闭全局中断 */
123.     DISABLE_INT();
124.
```

```
125.     /* 设置所有时钟到默认状态，使用 HSI 时钟 */
126.     HAL_RCC_DeInit();
127.
128.     /* 关闭滴答定时器，复位到默认值 */
129.     SysTick->CTRL = 0;
130.     SysTick->LOAD = 0;
131.     SysTick->VAL = 0;
132.
133.     /* 关闭所有中断，清除所有中断挂起标志 */
134.     for (i = 0; i < 8; i++)
135.     {
136.         NVIC->ICER[i]=0xFFFFFFFF;
137.         NVIC->ICPR[i]=0xFFFFFFFF;
138.     }
139.
140.     /* 使能全局中断 */
141.     ENABLE_INT();
142.
143.     /* 跳转到应用程序，首地址是 MSP，地址+4 是复位中断服务程序地址 */
144.     AppJump = (void (*)(void)) (*((uint32_t *) (AppAddr + 4)));
145.
146.     /* 设置主堆栈指针 */
147.     __set_MSP(*(uint32_t *)AppAddr);
148.
149.     /* 在 RTOS 工程，这条语句很重要，设置为特权级模式，使用 MSP 指针 */
150.     __set_CONTROL(0);
151.
152.     /* 跳转到系统 BootLoader */
153.     AppJump();
154.
155.     /* 跳转成功的话，不会执行到这里，用户可以在这里添加代码 */
156.     while (1)
157.     {
158.
159.     }
160. }
```

这里把程序设计中的几个关键地方做个说明：

- 第 12 行，声明一个函数指针。
- 第 13 行，QSPI Flash 地址在 0x90000000。
- 第 20 行，此函数比较省事，可以方便的设置 H7 所有时钟到复位值，内部时钟使用 HSI。
- 第 23 到 25 行，设置滴答定时器到复位值。
- 第 28 到 32 行，清除所有中断挂起标志并关闭中断，这里是直接通过一个 for 循环设置了 NVIC 所有配置位，共 8 组。

Address	Name	Type	Required privilege	Reset value	Description
0xE000E100-0xE000E11C	NVIC_ISER0-NVIC_ISER7	RW	Privileged	0x00000000	*Interrupt set-enable registers on page 185*
0XE000E180-0xE000E19C	NVIC_ICER0-NVIC_ICER7	RW	Privileged	0x00000000	*Interrupt clear-enable registers on page 186*
0XE000E200-0xE000E21C	NVIC_ISPR0-NVIC_ISPR7	RW	Privileged	0x00000000	*Interrupt set-pending registers on page 186*
0XE000E280-0xE000E29C	NVIC_ICPR0-NVIC_ICPR7	RW	Privileged	0x00000000	*Interrupt clear-pending registers on page 187*
0xE000E300-0xE000E31C	NVIC_IABR0-NVIC_IABR7	RW	Privileged	0x00000000	*Interrupt active bit registers on page 188*
0xE000E400-0xE000E4EF	NVIC_IPR0-NVIC_IPR59	RW	Privileged	0x00000000	*Interrupt priority registers on page 188*
0xE000EF00	STIR	WO	Configurable(1)	0x00000000	*Software trigger interrupt register on page 189*

- 第 38 行，将系统 bootLoader 的中断复位服务程序的入口地址赋给第 12 行声明的函数，用户执行这个函数时，就会直接跳转过去。
- 第 41 行，设置主堆栈指针位置，即 QSPI Flash 应用程序首地址存储的就是栈地址。
- 第 44 行，这个设置在 RTOS 应用程序中比较重要，因为基于 Cortex-M 内核的 RTOS 任务堆栈基本都是使用线程堆栈指针 PSP。但系统 bootLoader 使用的是主堆栈指针 MSP，所以务必要设置下，同时让 M 内核工作于特权级。此寄存器的作用如下：

Table 4.3 Bit Fields in CONTROL Register

Bit	Function
nPRIV (bit 0)	Defines the privileged level in Thread mode: When this bit is 0 (default), it is privileged level when in Thread mode. When this bit is 1, it is unprivileged when in Thread mode. In Handler mode, the processor is always in privileged access level.
SPSEL (bit 1)	Defines the Stack Pointer selection: When this bit is 0 (default), Thread mode uses Main Stack Pointer (MSP). When this bit is 1, Thread mode uses Process Stack Pointer (PSP). In Handler mode, this bit is always 0 and write to this bit is ignored.
FPCA (bit 2)	Floating Point Context Active – This bit is only available in Cortex-M4 with floating point unit implemented. The exception handling mechanism uses this bit to determine if registers in the floating point unit need to be saved when an exception has occurred. When this bit is 0 (default), the floating point unit has not been used in the current context and therefore there is no need to save floating point registers. When this bit is 1, the current context has used floating point instructions and therefore need to save floating point registers. The FPCA bit is set automatically when a floating point instruction is executed. This bit is clear by hardware on exception entry. There are several options for handling saving of floating point registers. This will be covered in Chapter 13.

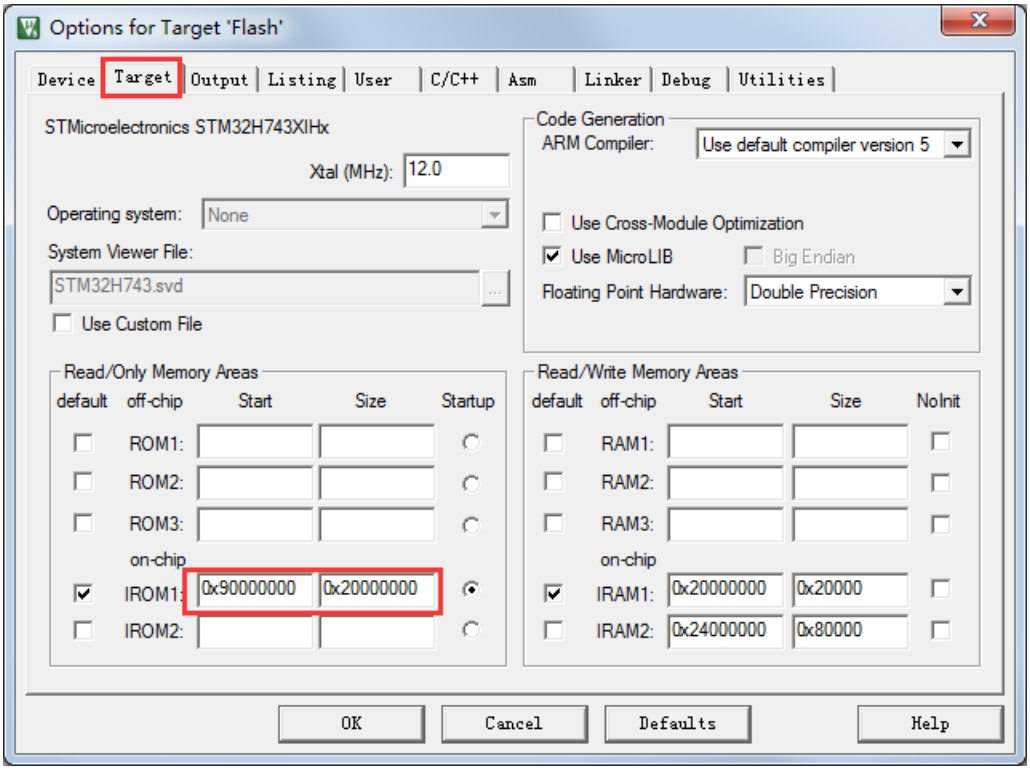
- 第 47 行，跳转到系统 bootLoader。

82.4 QSPI Flash 的 APP 应用程序说明

APP 应用程序是由第 53 章配套例子：V7-033_LCD 的汉字小字库和全字库制作 简单修改而来，主要修改如下两个地方：

82.4.1 设置 Flash 地址

设置 Flash 地址为 QSPI Flash 首地址 0x90000000：



82.4.2 设置中断向量表

在 main 函数最开始就设置中断向量表到 QSPI Flash 首地址 0x90000000，也可以按照本教程第 28 章的说明，将中断向量表设置到 DTCM 里面。

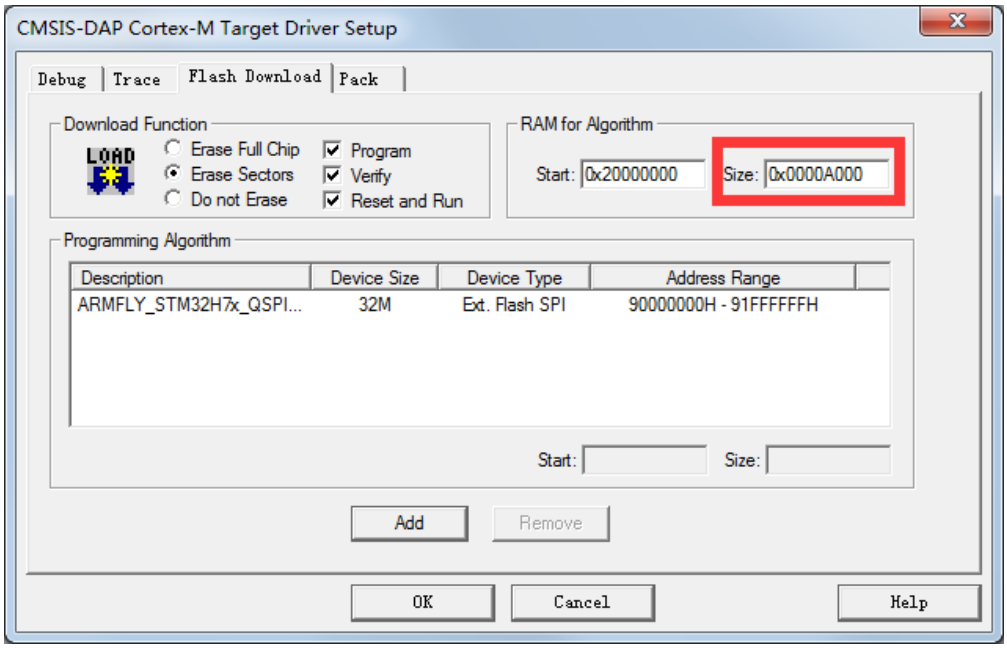
```
int main(void)
{
    uint16_t ucBright;      /* 背光亮度(0-255) */
    uint8_t ucKeyCode;      /* 按键代码 */
    uint8_t ucStatus;       /* 主程序状态字 */
    uint8_t fRefresh;       /* 刷屏请求标志,1 表示需要刷新 */

    SCB->VTOR = 0x90000000; /* 设置中断向量表地址 */

    bsp_Init();   /* 硬件初始化 */
    PrintfLogo(); /* 打印例程名称和版本等信息 */
    PrintfHelp(); /* 打印操作提示 */

    省略未写

}
```

82.5 QSPI Flash 的 APP 应用程序调试下载配置

将下面两个地方配置后，就可以像使用内部 Flash 一样使用 QSPI Flash 进行调试了。

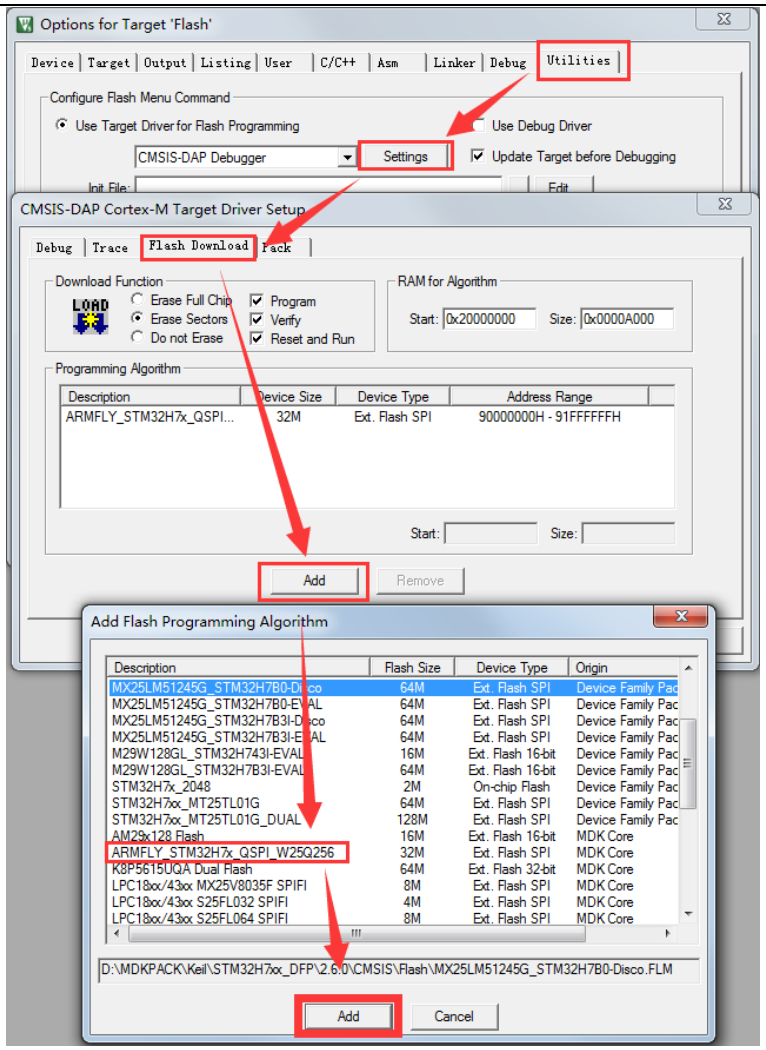
82.5.1 下载配置

注意这里一定要够大，否则会提示算法文件无法加载：



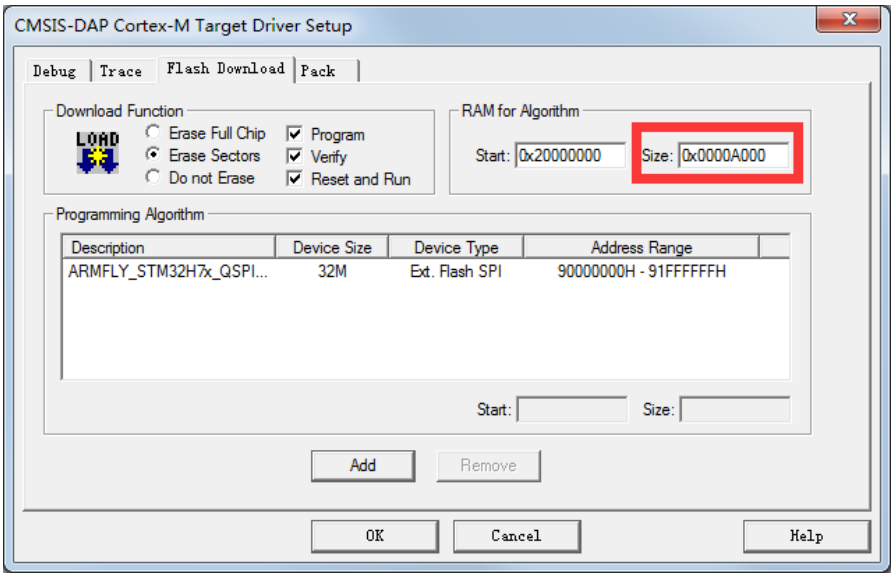
我们这里是将其加到 DTCM 中，即首地址为 0x20000000，大家也可以存储到任意其它 RAM 地址，只要空间还够加载算法文件即可。推荐使用 AXI SRAM（地址 0x24000000），因为这块 RAM 空间足够大。

如果要下载程序到 QSPI Flash 里面，需要做如下配置：



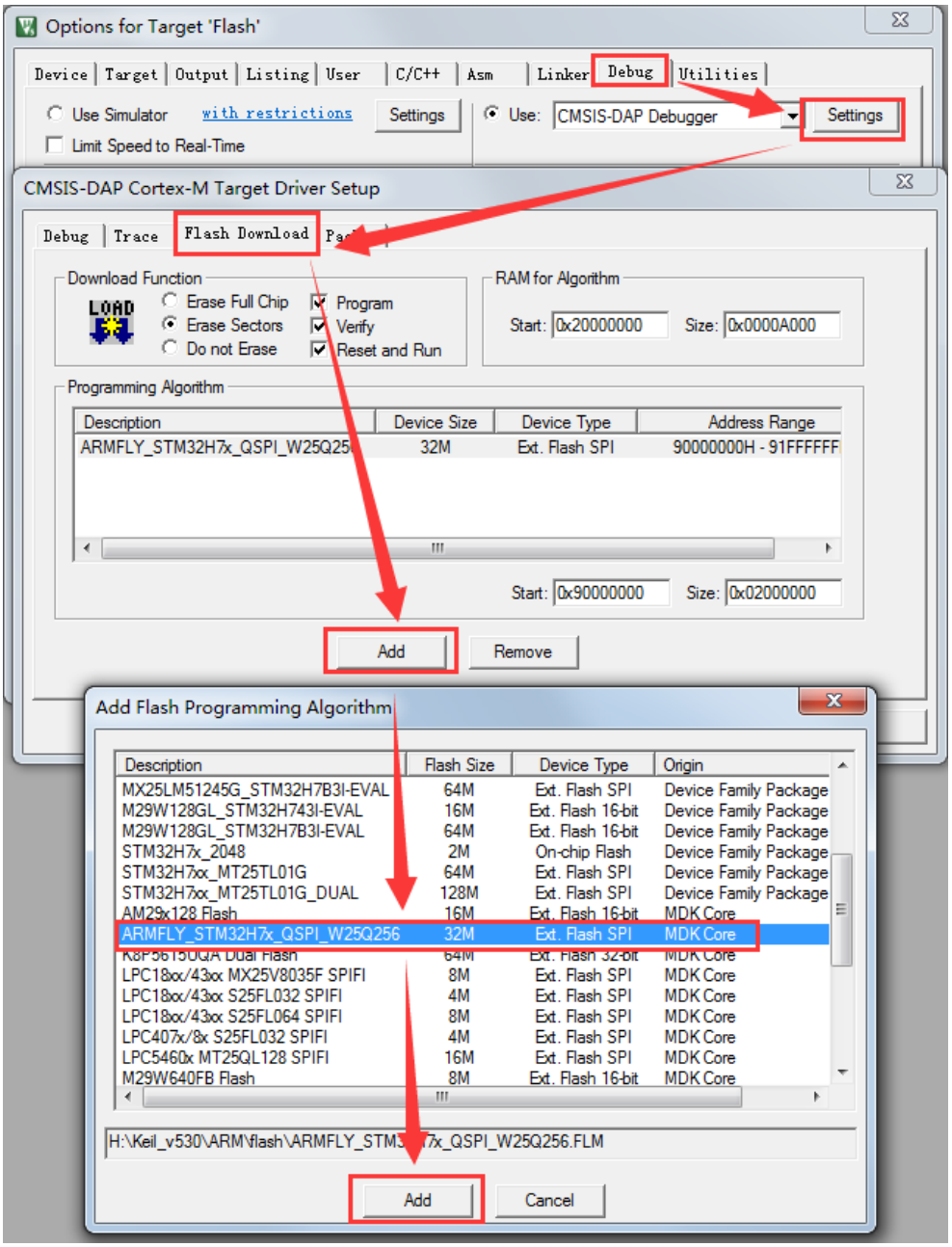
82.5.2 调试配置

注意这里一定要够大，否则会提示算法文件无法加载：



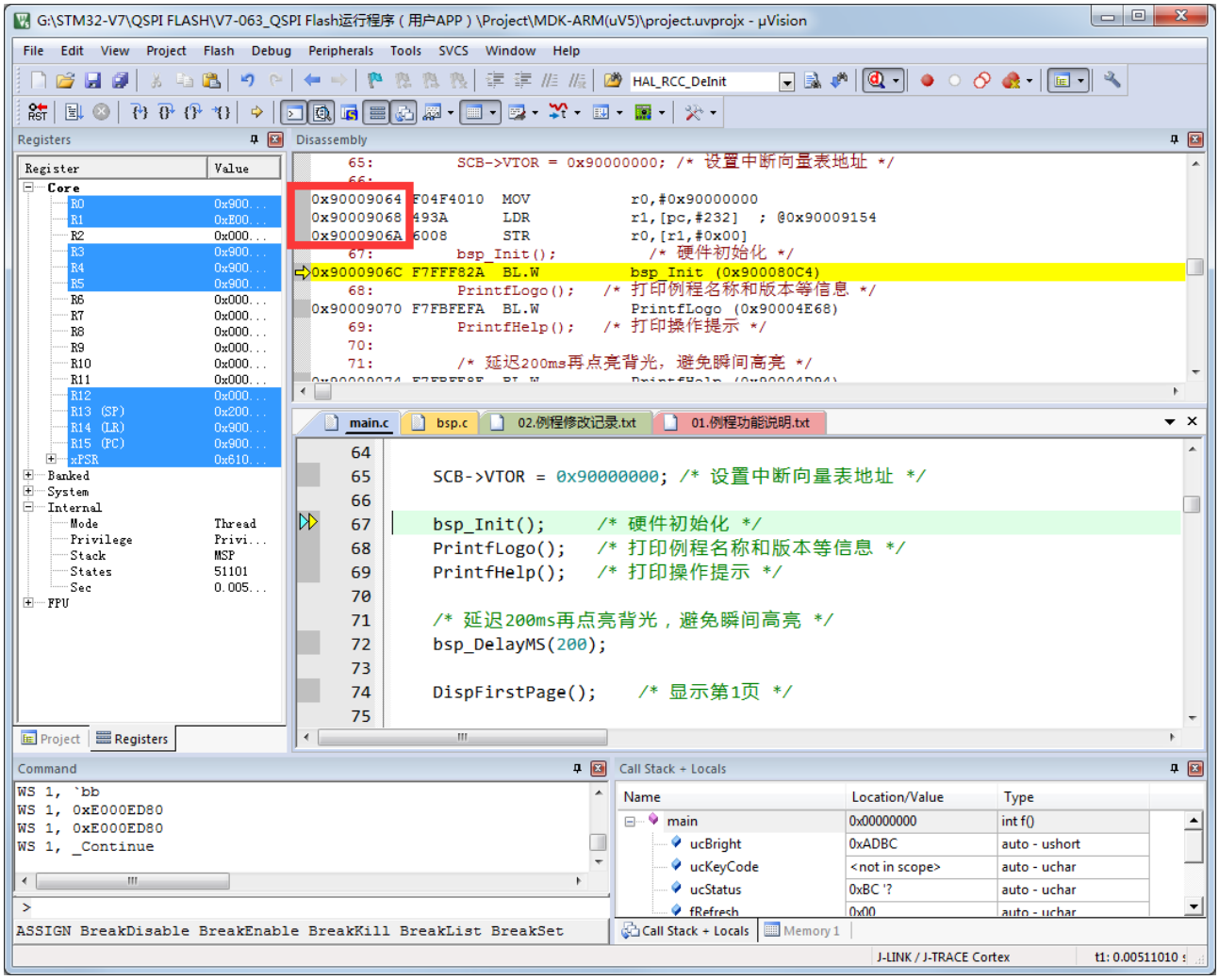
我们这里是将其加到 DTCM 中，即首地址为 0x20000000，大家也可以存储到任意其它 RAM 地址，只要空间还够加载算法文件即可。

如果要做调试下载，需要做如下配置：



82.5.3 程序调试效果

APP 应用程序调试效果如下：



82.6 实验例程说明

本章配套了两个例子：

- V7-062_QSPI Flash 运行程序（Bootloader）。
- V7-063_QSPI Flash 运行程序（用户 APP）。

Bootloader 例子需要大家先下载到内部 Flash 里面，然后按照本章第 5 小节的说明配置后，就可以像使用内部 Flash 一样调试下载 QSPI Flash 了。

82.7 总结

本章节就为大家讲解这么，为了熟练掌握，建议大家是操作练习下。

第83章 STM32H7 的内部 Flash 和 QSPI Flash 都运行程序的混合方式（强烈推荐）

本章节为大家讲解内部 Flash 和外部 QSPI Flash 都运行程序的混合方式。

83.1 初学者重要提示
83.2 内部 Fkash 和 QSPI Flash 混合执行程序的优势
83.3 下载算法存储存放位置
83.4 内部 Flash 和 QSPI Flash 混合执行程序说明
83.5 内部 Flash 和 QSPI Flash 程序调试下载配置
83.6 实验例程说明
83.7 总结

83.1 初学者重要提示

- QSPI Flash 的相关知识点可以看第 78 章和 79 章。
- 操作本章配套例子前务必先将 QSPI Flash 的下载算法放到 MDK 安装目录。
- 本章节与第 28 章的时间关键代码在 ITCM 执行的设置思路是一样的，可以让大家对本章节的方法有个更全面的认识。

83.2 内部 Flash 和 QSPI Flash 混合执行程序的优势

现在大家使用 H750 或 H743 在外置 QSPI Flash 执行程序，都倾向于制作个 BOOT，再制作一个运行在外置 QSPI Flash 的 APP 程序。即本教程第 82 章讲解的内容。

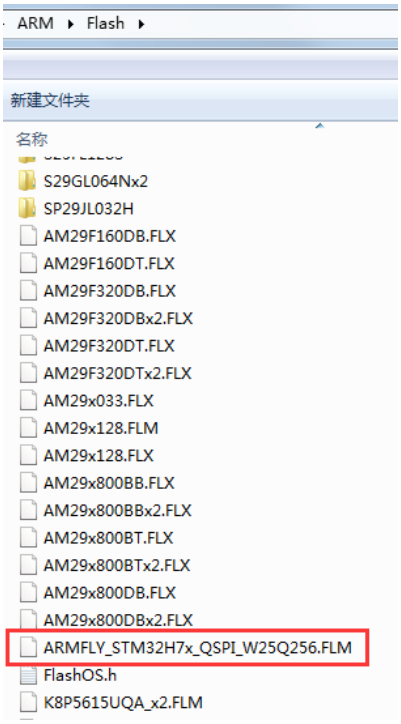
本章要实现的这种方式对于我们想外置 QSPI Flash 运行程序，在调试测试阶段更加方便，甚至是项目使用，可以性能最大化。

83.3 下载算法存放位置

编译例子：V7-060_QSPI Flash 的 MDK 下载算法制作，生成的算法文件位于此路径下：

生成算法文件后，需要大家将其存到到 MDK 安装目录，有两个位置可以存放，任选其一，推荐第 2 种：

- 第 1 种：存放到 MDK 的 STM32H7 软包安装目录里面：\Keil\STM32H7xx_DFP\2.6.0\CMSIS\Flash（软包版本不同，数值 2.6.0 不同）。
- 第 2 种：MDK 的安装目录 \ARM\Flash 里面。



83.4 内部 Flash 和 QSPI Flash 混合执行程序说明

混合程序的设置注意以下两点即可。

83.4.1 初始化 QSPI Flash 并设置内存映射模式

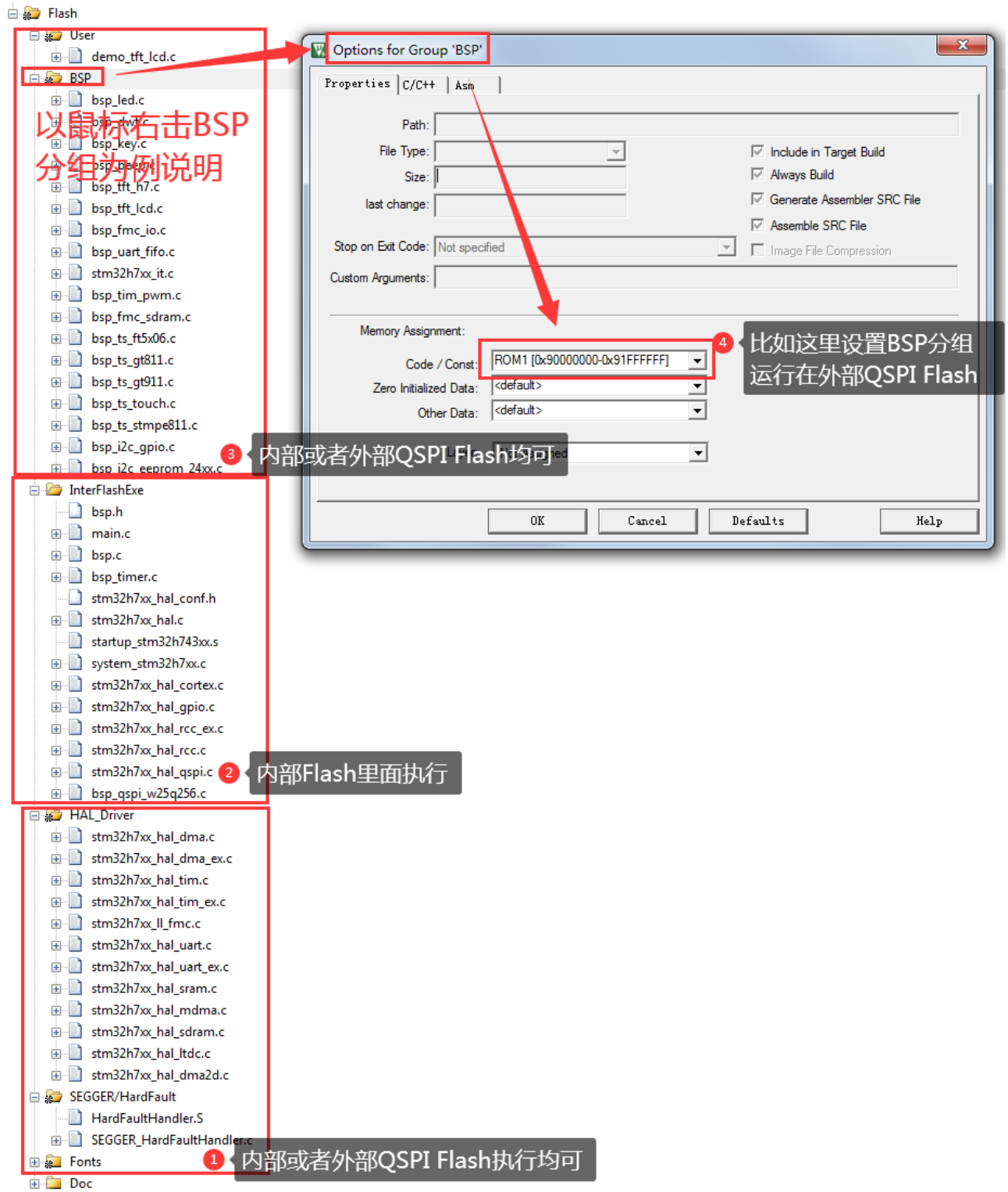
在 bsp.c 文件中初始在 bsp.c 文件中初始 QSPI Flash 并设置为内存映射模式。注意，配置好时钟后，优先初始化 QSPI Flash 并设置为内存映射模式：

```
/*
```

```
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 8 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitQSPI_W25Q256();  /* 配置 SPI 总线 */
    QSPI_MemoryMapped();

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();     /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();   /* 初始化滴答定时器 */
    bsp_InitUart();        /* 初始化串口 */
    bsp_InitExtIO();   /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();     /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    bsp_InitI2C();          /* 初始化 I2C 总线 */
    TOUCH_InitHard();    /* 初始化触摸芯片，LCD 面板型号的检查也在此函数，所以要在函数 LCD_InitHard 前调用 */
    LCD_InitHard();     /* 初始化 LCD */
}
```

83.4.2 设置运行在内部 Flash 和 QSPI Flash 的程序

由于我们要把 QSPI Flash 初始化并设置为内存映射模式后才可以使用 QSPI Flash 运行程序，所以要在 QSPI Flash 运行程序之前的所有操作都在内部 Flash 里面实现，主要是滴答定时器，RCC，QSPI 和 HAL 库等相关配置。其它所有程序都可以在外部 QSPI Flash 执行：

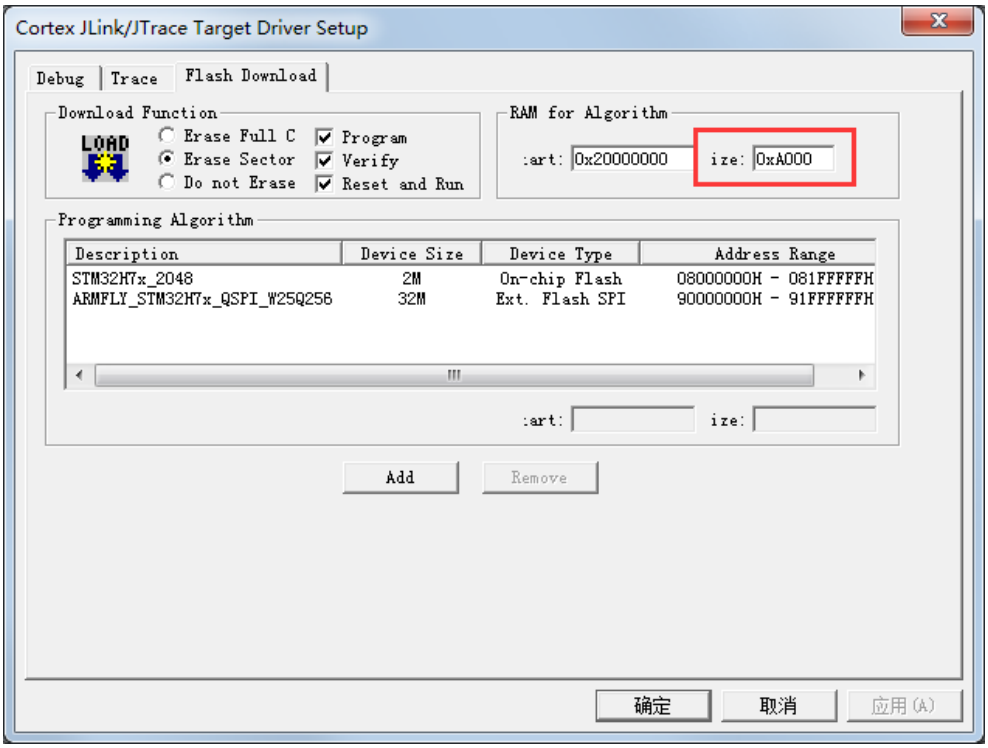


83.5 内部 Flash 和 QSPI Flash 程序调试下载配置

将下面两个地方配置后，就可以像使用内部 Flash 一样使用 QSPI Flash 进行调试了。并且这种方式可以方便的调试程序，内部 Flash 和外部 Flash 都做调试。

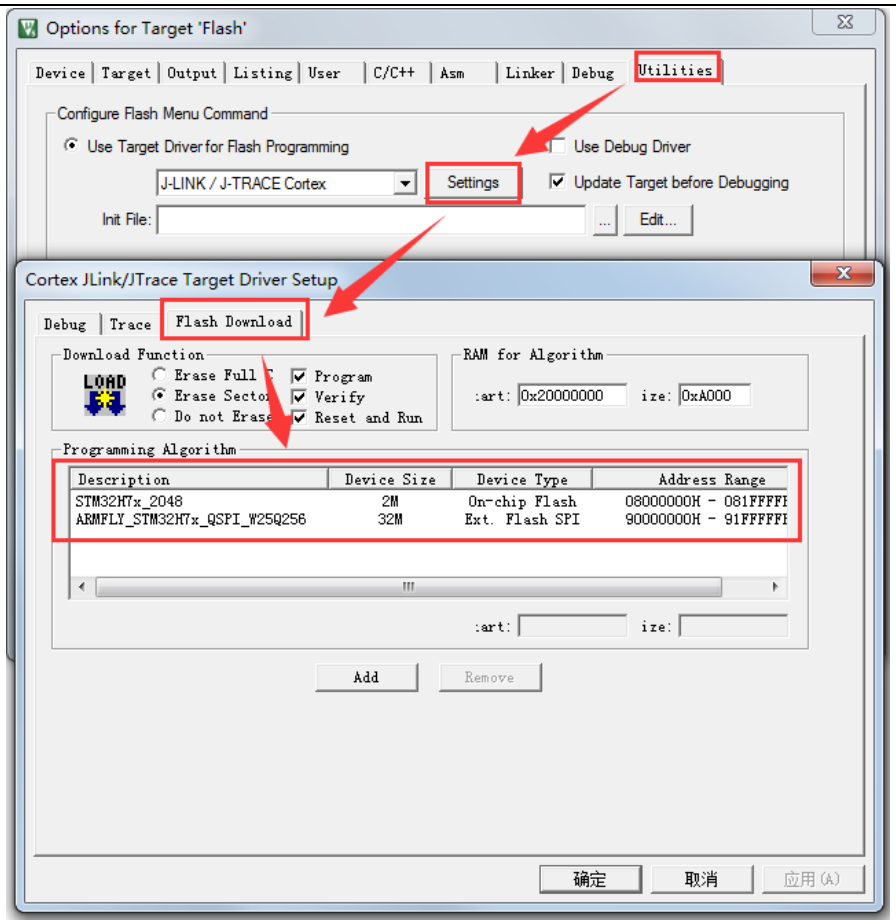
83.5.1 下载配置

注意这里一定要够大，否则会提示算法文件无法加载：



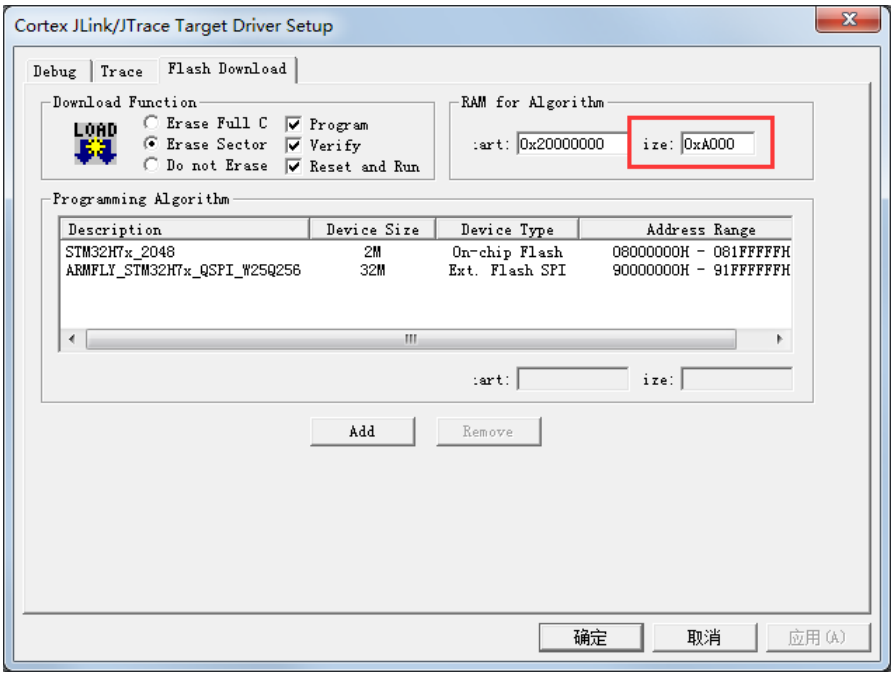
我们这里是将其加到 DTCM 中，即首地址为 0x20000000，大家也可以存储到任意其它 RAM 地址，只要空间还够加载算法文件即可。推荐使用 AXI SRAM（地址 0x24000000），因为这块 RAM 空间足够大。

如果要下载程序到内部 Flash 和外部 QSPI Flash 里面，需要做如下配置，两个下载算法都要添加进来：



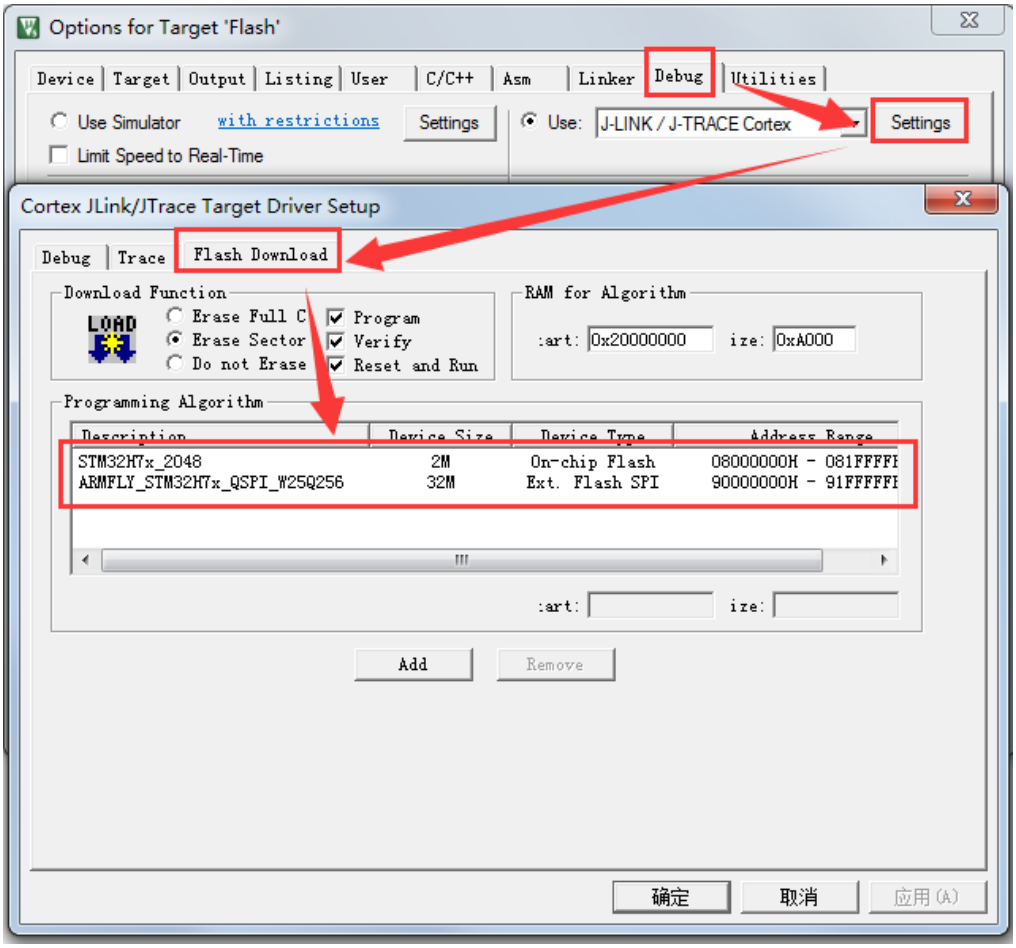
83.5.2 调试配置

注意这里一定要够大，否则会提示算法文件无法加载：



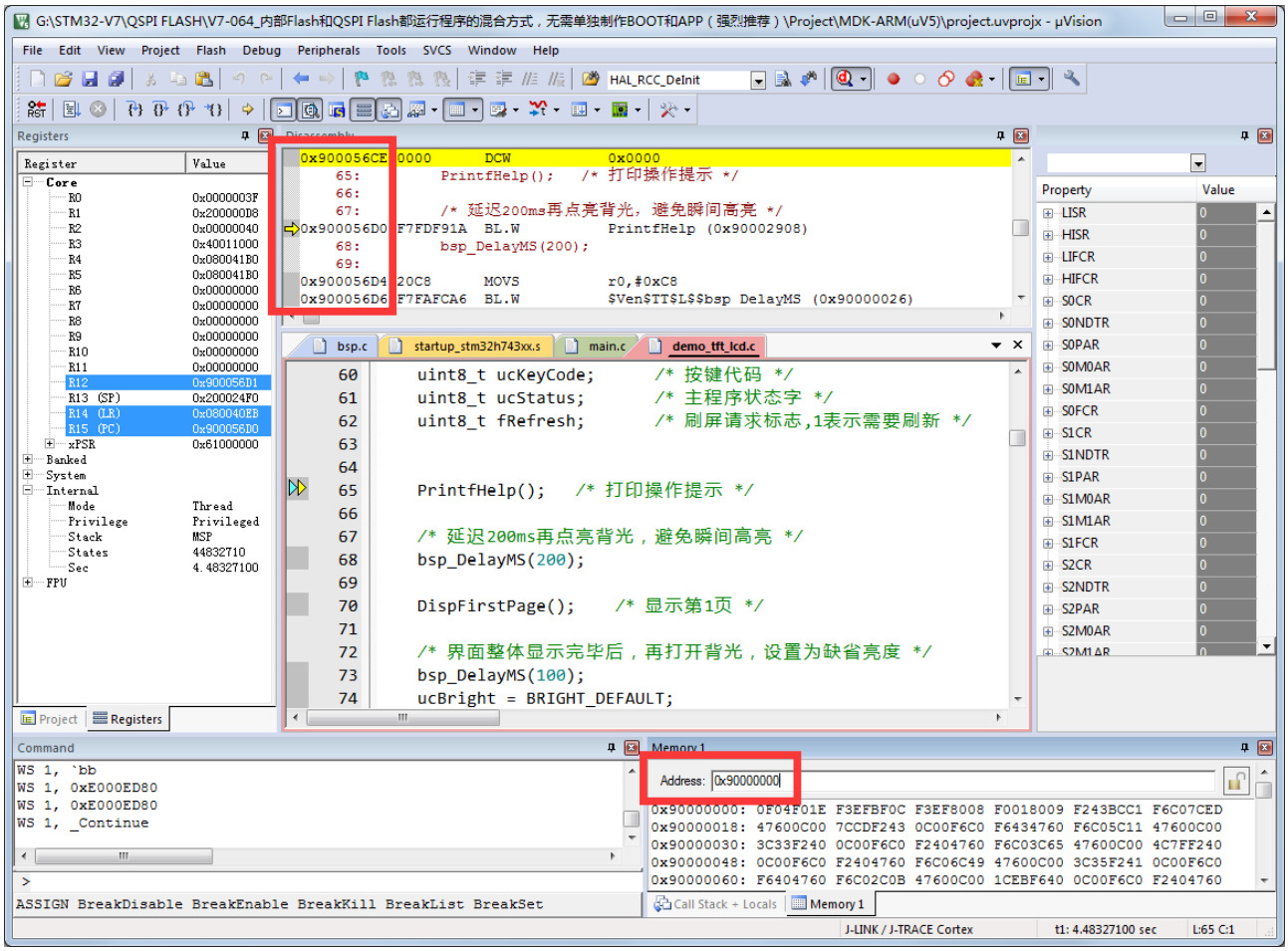
我们这里是将其加到 DTCM 中，即首地址为 0x20000000，大家也可以存储到任意其它 RAM 地址，只要空间还够加载算法文件即可。

如果要做调试下载，需要做如下配置：



83.5.3 程序调试效果

调试效果如下：



83.6 实验例程说明

本章节配套例子：V7-064_内部 Flash 和 QSPI Flash 都运行程序的混合方式，无需单独制作 BOOT 和 APP（强烈推荐）

编译此程序弹出的警告不用管，可以选择屏蔽此警告：

```
compiling hz24.c...
compiling hz32.c...
compiling ra8875_asc_width.c...
compiling hz16.c...
linking...
.\Objects\output.sct(23): warning: L6329W: Pattern bsp_beep.o(RO) only matches removed unused sections.
.\Objects\output.sct(38): warning: L6329W: Pattern stm32h7xx_hal_dma.o(RO) only matches removed unused sections.
.\Objects\output.sct(39): warning: L6329W: Pattern stm32h7xx_hal_dma_ex.o(RO) only matches removed unused sections.
.\Objects\output.sct(41): warning: L6329W: Pattern stm32h7xx_hal_tim_ex.o(RO) only matches removed unused sections.
.\Objects\output.sct(44): warning: L6329W: Pattern stm32h7xx_hal_uart_ex.o(RO) only matches removed unused sections.
.\Objects\output.sct(46): warning: L6329W: Pattern stm32h7xx_hal_mdma.o(RO) only matches removed unused sections.
.\Objects\output.sct(54): warning: L6329W: Pattern asc24.o(RO) only matches removed unused sections.
.\Objects\output.sct(55): warning: L6329W: Pattern asc32.o(RO) only matches removed unused sections.
Program Size: Code=39258 RO-data=270054 RW-data=220 ZI-data=9244
Finished: 0 information, 8 warning and 0 error messages.
```

83.7 总结

本章节就为大家讲解这么，为了熟练掌握，大家可以尝试自己实现一个内部 Flash 和外部 Flash 混合运行的程序。

第84章 STM32H7 的 SPI 总线应用之 SPI Flash 的 MDK 下载算法制作

本章节为大家讲解 MDK 下载算法制作方法。

84.1 初学者重要提示
84.2 MDK 下载算法基础知识
84.3 创建 MDK 下载算法通用流程
84.4 SPI Flash 的 MDK 下载算法制作
84.5 SPI Flash 的 MDK 下载算法使用方法
84.6 实验例程说明
84.7 总结

84.1 初学者重要提示

- SPI Flash 的相关知识点可以看第 78 章和 79 章。
- SPI Flash 下载算法文件直接采用 HAL 库制作，方便大家自己修改。

84.2 MDK 下载算法基础知识

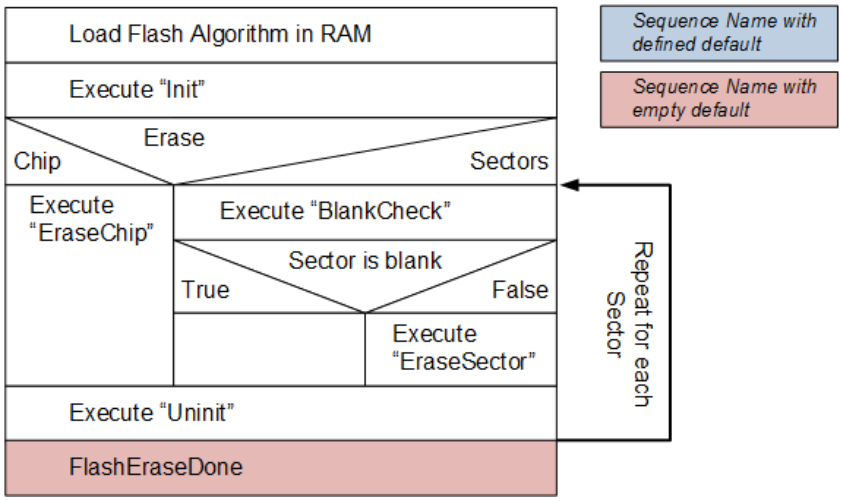
Flash 编程算法是一种用于擦除应用程序或将应用程序下载到 Flash 的程序代码。MDK 本身支持的各种器件都自带下载算法，存放在 MDK 各种器件的软件包里面，以 STM32H7 为例，算法存放在\Keil\STM32H7xx_DFP\2.6.0\CMSIS\Flash（软件包版本不同，数值 2.6.0 不同），但不支持的需要我们自己制作，本章教程为此而生。

84.2.1 程序能够通过下载算法下载到芯片的核心思想

认识到这点很重要：通过 MDK 创建一批与地址信息无关的函数，实现的功能主要有初始化，擦除，编程，读取，校验等，然后 MDK 调试下载阶段，会将算法文件加载到芯片的内部 RAM 里面（加载地址可以通过 MDK 设置），然后 MDK 通过与这个算法文件的交互，实现程序下载，调试阶段数据读取等操作。

84.2.2 算法程序中擦除操作执行流程

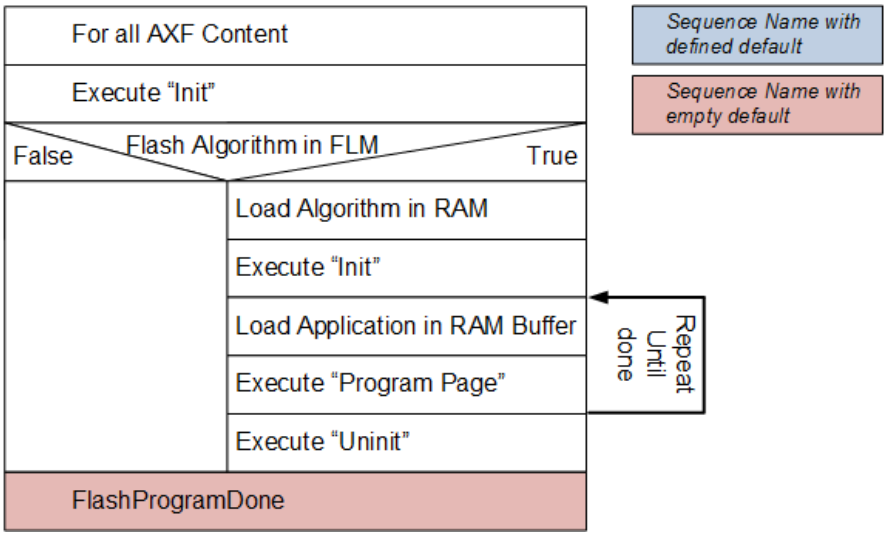
擦除操作大致流程：



- 加载算法到芯片 RAM。
- 执行初始化函数 Init。
- 执行擦除操作，根据用户的 MDK 配置，这里可以选择整个芯片擦除或者扇区擦除。
- 执行 Uinit 函数。
- 操作完毕。

84.2.3 算法程序中编程操作执行流程

编程操作大致流程：



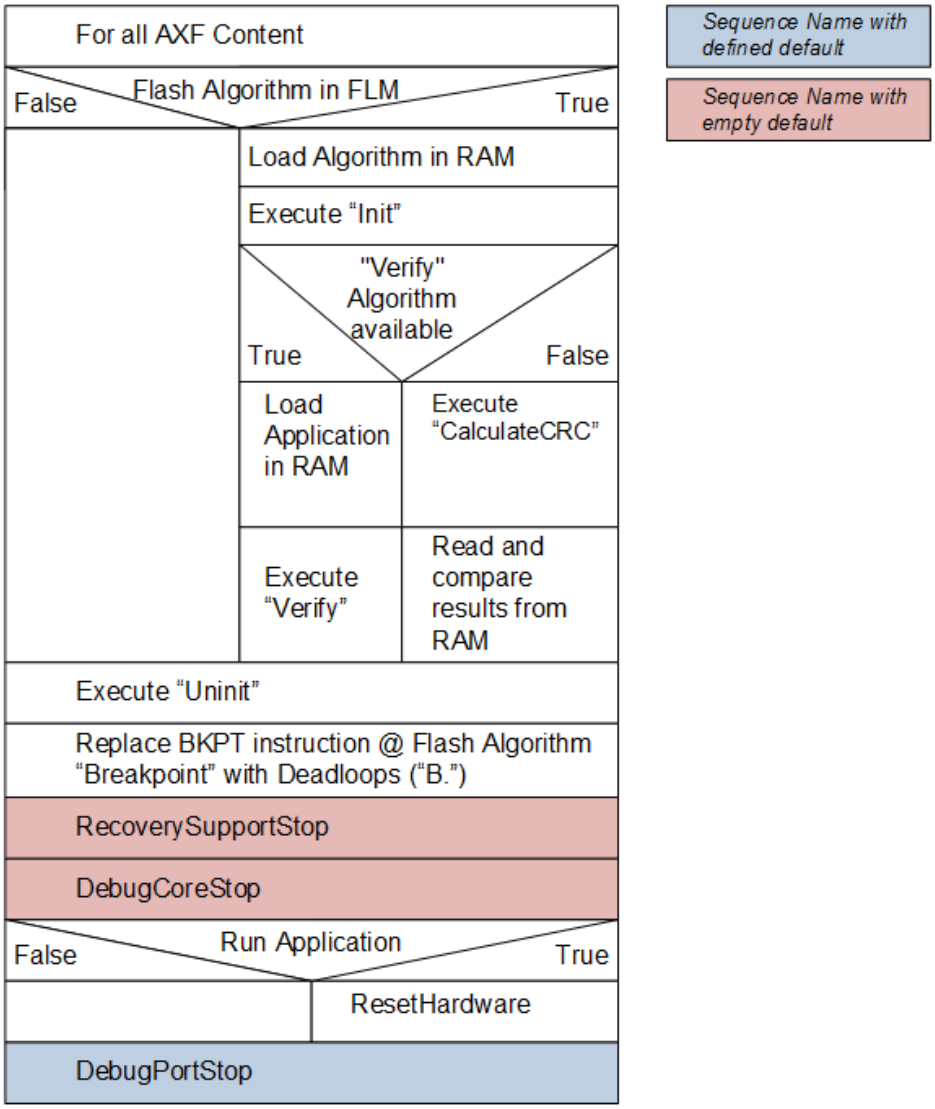
- 针对 MDK 生成的 axf 可执行文件做 Init 初始化，这个 axf 文件是指的大家自己创建应用程序生成的。
- 查看 Flash 算法是否在 FLM 文件。如果没有在，操作失败。如果在：
  - 加载算法到 RAM。

- 执行 Init 函数。
- 加载用户到 RAM 缓冲。
- 执行 Program Page 页编程函数。
- 执行 Uninit 函数。

◆ 操作完毕。

84.2.4算法程序中校验操作执行流程

校验操作大致流程：



◆ 校验要用到 MDK 生成的 axf 可执行文件。校验就是 axf 文件中下载到芯片的程序和实际下载的程序读出来做比较。

◆ 查看 Flash 算法是否在 FLM 文件。如果没有在，操作失败。如果在：

- 加载算法到 RAM。

- 执行 Init 函数。
- 查看校验算法是否存在
  - 如果有，加载应用程序到 RAM 并执行校验。
  - 如果没有，计算 CRC，将芯片中读取出来的数据和 RAM 中加载应用计算输出的 CRC 值做比较。
- 执行 Uninit 函数。
- 替换 BKPT（BreakPoint 断点指令）为 B. 死循环指令。
- 执行 RecoverySupportStop，恢复支持停止。
- 执行 DebugCoreStop，调试内核停止。

◆ 运行应用：

- 执行失败。
- 执行成功，再执行硬件复位。

◆ 操作完毕，停止调试端口。

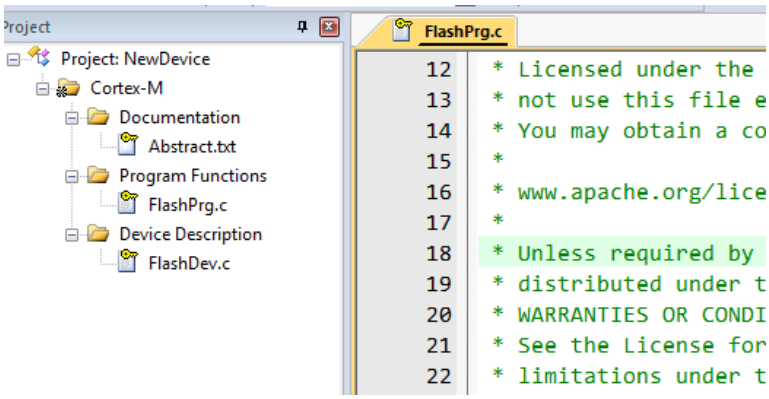
84.3 创建 MDK 下载算法通用流程

下面是 MDK 给的一种大致操作流程，不限制必须采用这种方法，自己创建也可以的。

84.3.1 第 1 步，使用 MDK 提供好的程序模板

位于路径：\Keil\ARM\Pack\ARM\CMSIS\version\Device_Template_Flash。

效果如下：

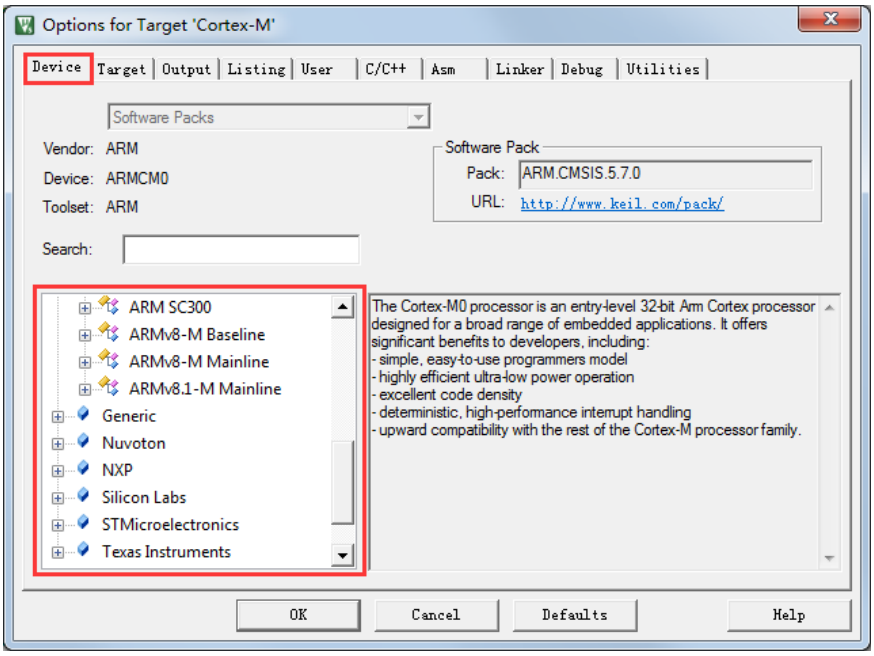


84.3.2 第 2 步，修改工程名

MDK 提供的工程模板原始名字是 NewDevice.uvprojx，大家可以根据自己的需要做修改。比如修改为 MyDevice.uvprojx。

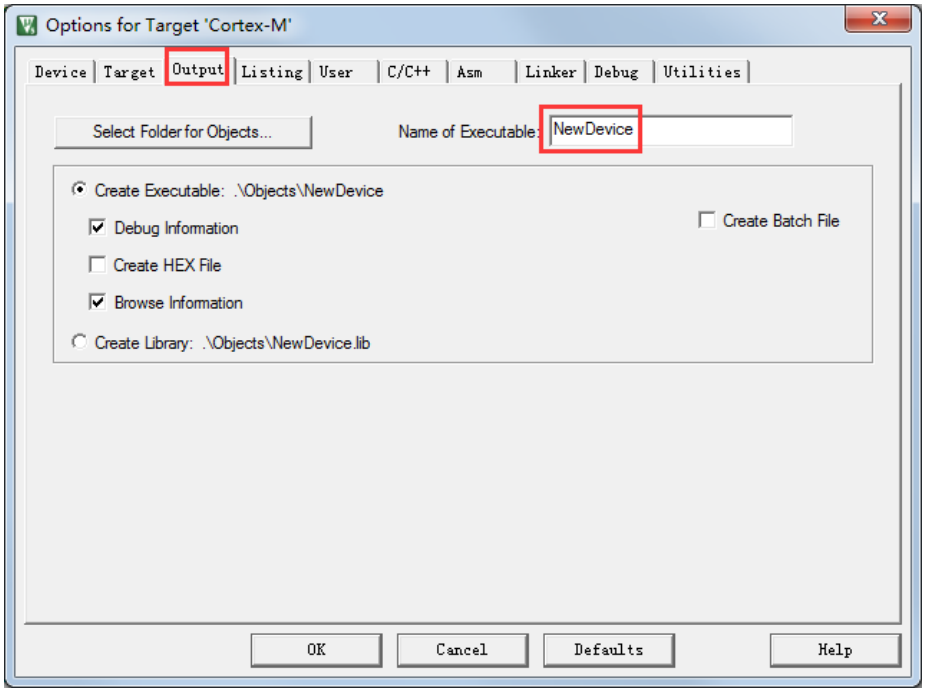
84.3.3第 3 步，修改使用的器件

在 MDK 的 Option 选项里面设置使用的器件。

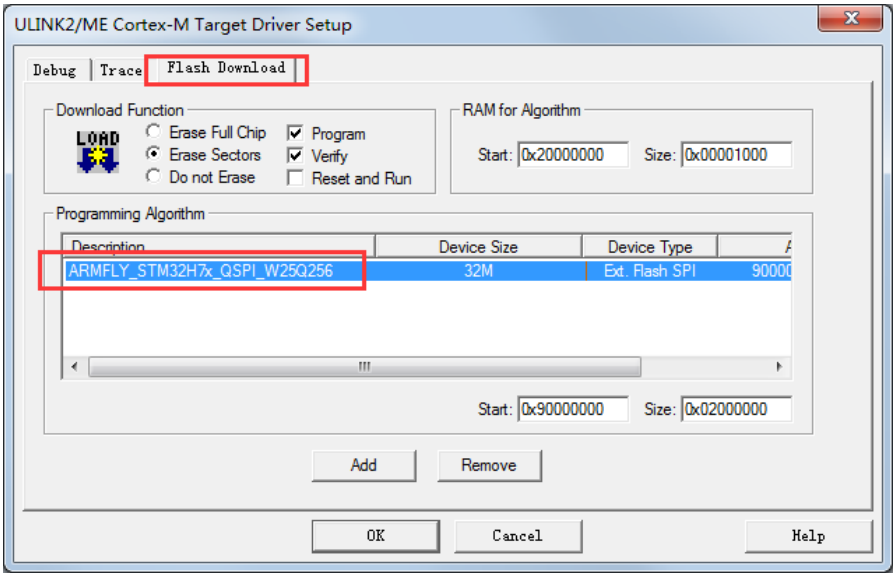


84.3.4第 4 步，修改输出算法文件的名字

这个名字是方便用户查看的，比如设置为 stm32h7，那么输出的算法文件就是 stm32h7.flm。



注：MDK 这里设置的名字与下面位置识别出来的算法名无关：



这个名字是在 FlashDev.c 里面定义的。

84.3.5 第 5 步，修改编程算法文件 FlashPrg.c

模板工程里面仅提供了接口函数，内容需要用户自己填。

```
/*
   Mandatory Flash Programming Functions (Called by FlashOS):
               int Init        (unsigned long adr,   // Initialize Flash
                                unsigned long clk,
                                unsigned long fnc);
               int UnInit      (unsigned long fnc);  // De-initialize Flash
               int EraseSector (unsigned long adr);  // Erase Sector Function
               int ProgramPage (unsigned long adr,   // Program Page Function
                                unsigned long sz,
                                unsigned char *buf);

   Optional  Flash Programming Functions (Called by FlashOS):
               int BlankCheck  (unsigned long adr,   // Blank Check
                                unsigned long sz,
                                unsigned char pat);
               int EraseChip   (void);               // Erase complete Device
      unsigned long Verify     (unsigned long adr,   // Verify Function
                                unsigned long sz,
                                unsigned char *buf);

       - BlanckCheck  is necessary if Flash space is not mapped into CPU memory space
       - Verify       is necessary if Flash space is not mapped into CPU memory space
       - if EraseChip is not provided than EraseSector for all sectors is called
*/

/*
 *  Initialize Flash Programming Functions
 *    Parameter:      adr:  Device Base Address
 *                    clk:  Clock Frequency (Hz)
 *                    fnc:  Function Code (1 - Erase, 2 - Program, 3 - Verify)
```

```
 *    Return Value:   0 - OK,  1 - Failed
 */

int Init (unsigned long adr, unsigned long clk, unsigned long fnc) {

  /* Add your Code */
  return (0);                                  // Finished without Errors
}


/*
 *  De-Initialize Flash Programming Functions
 *    Parameter:      fnc:  Function Code (1 - Erase, 2 - Program, 3 - Verify)
 *    Return Value:   0 - OK,  1 - Failed
 */

int UnInit (unsigned long fnc) {

  /* Add your Code */
  return (0);                                  // Finished without Errors
}


/*
 *  Erase complete Flash Memory
 *    Return Value:   0 - OK,  1 - Failed
 */

int EraseChip (void) {

  /* Add your Code */
  return (0);                                  // Finished without Errors
}


/*
 *  Erase Sector in Flash Memory
 *    Parameter:      adr:  Sector Address
 *    Return Value:   0 - OK,  1 - Failed
 */

int EraseSector (unsigned long adr) {

  /* Add your Code */
  return (0);                                  // Finished without Errors
}


/*
 *  Program Page in Flash Memory
 *    Parameter:      adr:  Page Start Address
 *                    sz:   Page Size
 *                    buf:  Page Data
 *    Return Value:   0 - OK,  1 - Failed
 */

int ProgramPage (unsigned long adr, unsigned long sz, unsigned char *buf) {
```

```
  /* Add your Code */
  return (0);                                  // Finished without Errors
}
```

84.3.6第 6 步，修改配置文件 FlashDev.c

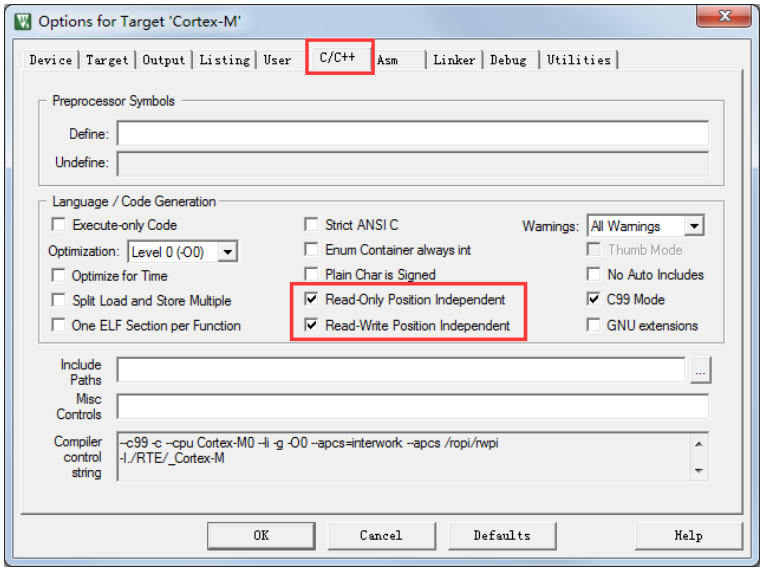
模板工程里面提供简单的配置说明：

```
struct FlashDevice const FlashDevice  =  {
   FLASH_DRV_VERS,             // Driver Version, do not modify!
   "New Device 256kB Flash",   // Device Name
   ONCHIP,                     // Device Type
   0x00000000,                 // Device Start Address
   0x00040000,                 // Device Size in Bytes (256kB)
   1024,                       // Programming Page Size
   0,                          // Reserved, must be 0
   0xFF,                       // Initial Content of Erased Memory
   100,                        // Program Page Timeout 100 mSec
   3000,                       // Erase Sector Timeout 3000 mSec

// Specify Size and Address of Sectors
   0x002000, 0x000000,         // Sector Size  8kB (8 Sectors)
   0x010000, 0x010000,         // Sector Size 64kB (2 Sectors)
   0x002000, 0x030000,         // Sector Size  8kB (8 Sectors)
   SECTOR_END
};
```

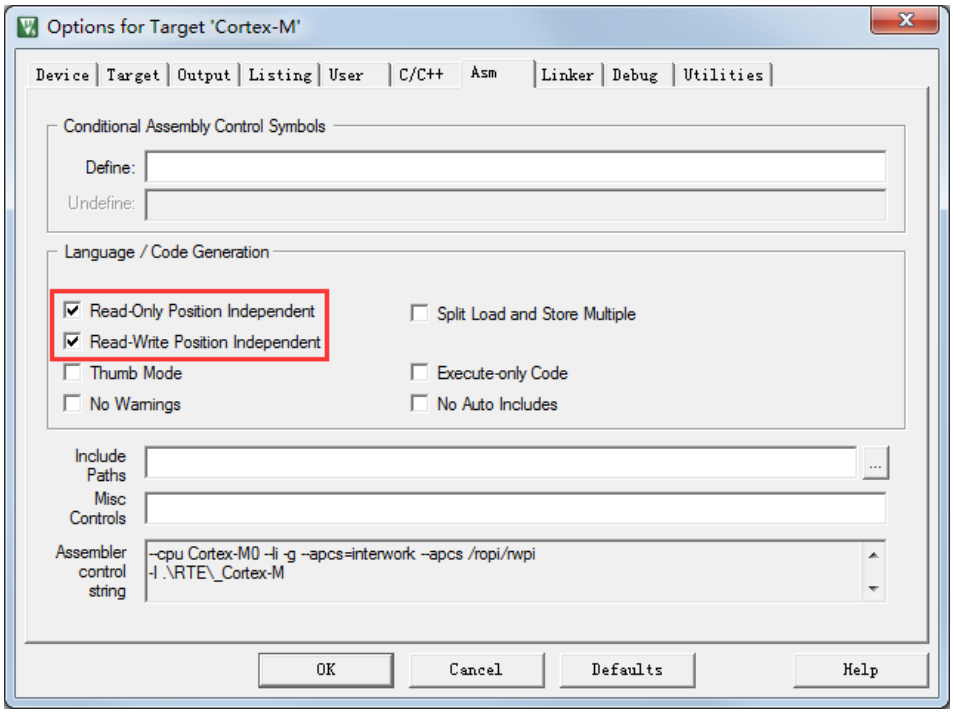
注：名字 New Device 256kB Flash 就是我们第 4 步所说的。MDK 的 Option 选项里面会识别出这个名字。

84.3.7第 7 步，保证生成的算法文件中 RO 和 RW 段的独立性，即与地址无关

C 和汇编的配置都勾选上：



汇编：



如果程序的所有只读段都与位置无关，则该程序为只读位置无关（ROPI, Read-only position independence）。ROPI 段通常是位置无关代码（PIC,position-independent code），但可以是只读数据，也可以是 PIC 和只读数据的组合。选择“ ROPI”选项，可以避免用户不得不将代码加载到内存中的特定位置。这对于以下例程特别有用：

（1）加载以响应运行事件。

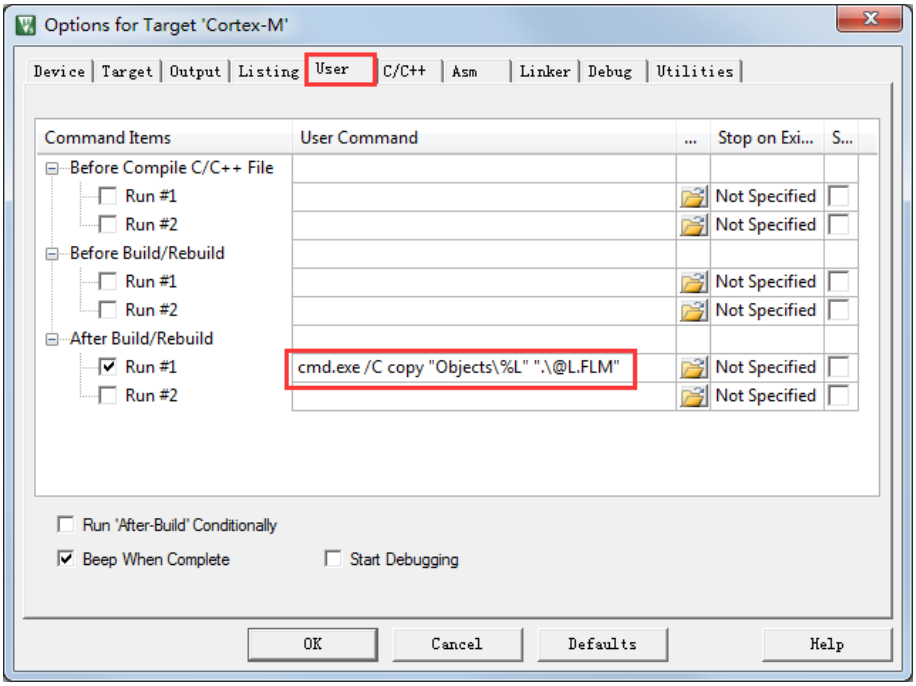
（2）在不同情况下使用其他例程的不同组合加载到内存中。

（3）在执行期间映射到不同的地址。

使用 Read-Write position independence 同理，表示的可读可写数据段。

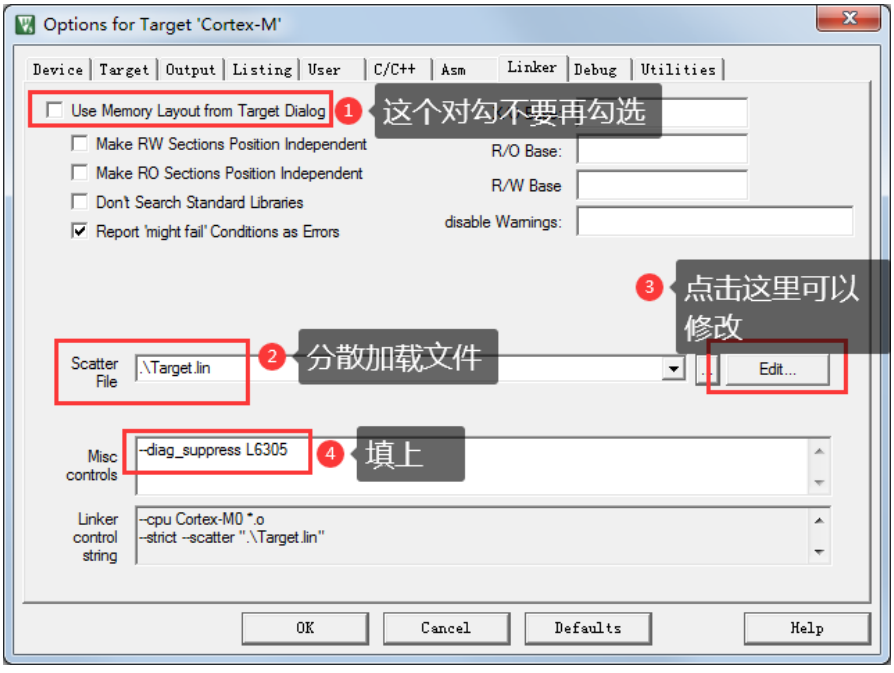
84.3.8第 8 步，将程序可执行文件 axf 修改为 flm 格式

通过下面的命令就可以将生成的 axf 可执行文件修改为 flm。



84.3.9第 9 步，分散加载设置

我们这里的分散加载文件直接使用 MDK 模板工程里提供好的即可，无需任何修改。



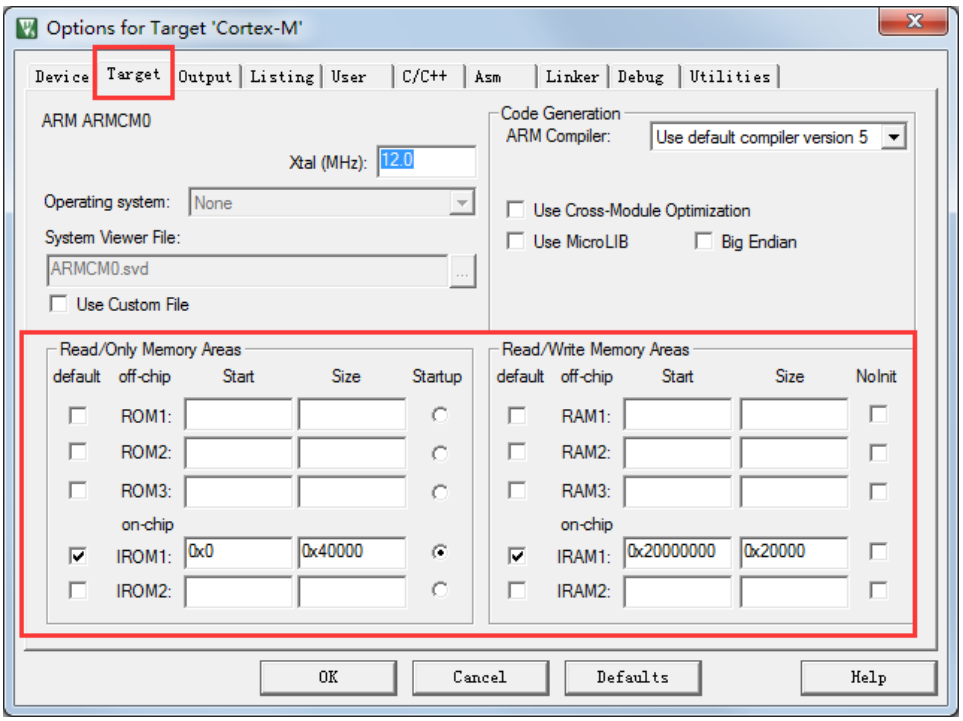
分散加载文件中的内容如下：

```
; Linker Control File (scatter-loading)
;

PRG 0 PI                ; Programming Functions
{
  PrgCode +0            ; Code
```

```
  {
    * (+RO)
  }
  PrgData +0            ; Data
  {
    * (+RW,+ZI)
  }
}

DSCR +0                 ; Device Description
{
  DevDscr +0
  {
    FlashDev.o
  }
}
```

--diag_suppress L6305 用于屏蔽 L6503 类型警告信息。

特别注意，设置了分散加载后，此处的配置就不再起作用了：



84.4 SPI Flash 的 MDK 下载算法制作

下面将 QSPI Flash 算法制作过程中的几个关键点为大家做个说明。

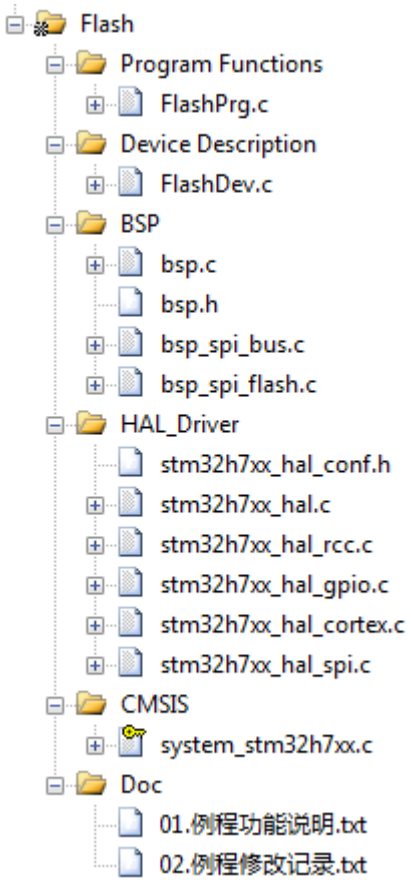
84.4.1 第 1 步，制作前重要提示

这两点非常重要：

- 程序里面不要开启任何中断，全部查询方式。
- HAL 库里面各种时间基准相关的 API 全部处理掉。简单省事些，我们这里是直接注释，采用死等即可。无需做超时等待，因为超时后，已经意味着操作失败了，跟死等没有区别。

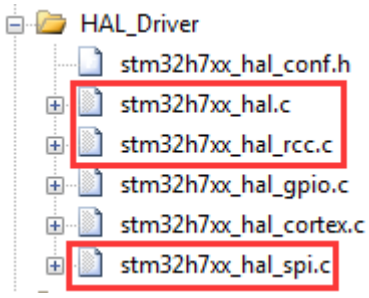
84.4.2 第 2 步，准备一个工程模板

推荐大家直接使用我们本章工程准备好的模板即可，如果大家自己制作，注意一点，请使用当前最新的 HAL 库。



84.4.3 第 3 步，修改 HAL 库

这一步比较重要，主要修改了以下三个文件：



主要是修改了 HAL 库时间基准相关的几个 API，并注释掉了一批无关的 API。具体修改内容，大家可以找个比较软件，对比修改后的这个文件和 CubeH7 软件包 V1.8.0（软件包里面的 HAL 库版本是 V1.9.0）的差异即可。

84.4.4第 4 步，时钟初始化

我们已经用不到滴答定时器了，直接在 bsp.c 文件里面对滴答初始化函数做重定向：

```
/*
*********************************************************************************************************
*    函 数 名: HAL_InitTick
*    功能说明: 重定向，不使用
*    形    参: TickPriority
*    返 回 值: 无
*********************************************************************************************************
*/
HAL_StatusTypeDef HAL_InitTick(uint32_t TickPriority)
{
    return HAL_OK;
}
```

然后就是 HSE 外置晶振的配置，大家根据自己的板子实际外挂晶振大小，修改 stm32h7xx_hal_conf.h 文件中 HSE_VALUE 大小，实际晶振多大，这里就修改为多大：

```
#if !defined  (HSE_VALUE)
#define HSE_VALUE    ((uint32_t)25000000) /*!< Value of the External oscillator in Hz */
#endif /* HSE_VALUE */
```

最后修改 PLL：

```
/*
*********************************************************************************************************
*    函 数 名: SystemClock_Config
*    功能说明: 初始化系统时钟
*            System Clock source            = PLL (HSE)
*            SYSCLK(Hz)                     = 400000000 (CPU Clock)
*            HCLK(Hz)                       = 200000000 (AXI and AHBs Clock)
*            AHB Prescaler                  = 2
*            D1 APB3 Prescaler              = 2 (APB3 Clock  100MHz)
*            D2 APB1 Prescaler              = 2 (APB1 Clock  100MHz)
*            D2 APB2 Prescaler              = 2 (APB2 Clock  100MHz)
*            D3 APB4 Prescaler              = 2 (APB4 Clock  100MHz)
*            HSE Frequency(Hz)              = 25000000
*            PLL_M                          = 5
*            PLL_N                          = 160
*            PLL_P                          = 2
*            PLL_Q                          = 4
*            PLL_R                          = 2
*            VDD(V)                         = 3.3
*            Flash Latency(WS)              = 4
*    形    参: 无
*    返 回 值: 1 表示失败，0 表示成功
*********************************************************************************************************
*/
int SystemClock_Config(void)
{
```

```
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    HAL_StatusTypeDef ret = HAL_OK;

    /* 锁住 SCU(Supply configuration update) */
    MODIFY_REG(PWR->CR3, PWR_CR3_SCUEN, 0);

    /*
      1、芯片内部的 LDO 稳压器输出的电压范围，可选 VOS1，VOS2 和 VOS3，不同范围对应不同的 Flash 读速度，
         详情看参考手册的 Table 12 的表格。
      2、这里选择使用 VOS1，电压范围 1.15V - 1.26V。
    */
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

    while(!__HAL_PWR_GET_FLAG(PWR_FLAG_VOSRDY)) {}

    /* 使能 HSE，并选择 HSE 作为 PLL 时钟源 */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.HSIState = RCC_HSI_OFF;
    RCC_OscInitStruct.CSIState = RCC_CSI_OFF;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;

    RCC_OscInitStruct.PLL.PLLM = 5;
    RCC_OscInitStruct.PLL.PLLN = 160;
    RCC_OscInitStruct.PLL.PLLP = 2;
    RCC_OscInitStruct.PLL.PLLR = 2;
    RCC_OscInitStruct.PLL.PLLQ = 4;

    RCC_OscInitStruct.PLL.PLLVCOSEL = RCC_PLL1VCOWIDE;
    RCC_OscInitStruct.PLL.PLLRGE = RCC_PLL1VCIRANGE_2;
    ret = HAL_RCC_OscConfig(&RCC_OscInitStruct);
    if(ret != HAL_OK)
    {
        return 1;
    }

    /*
      选择 PLL 的输出作为系统时钟
      配置 RCC_CLOCKTYPE_SYSCLK 系统时钟
      配置 RCC_CLOCKTYPE_HCLK 时钟，对应 AHB1，AHB2，AHB3 和 AHB4 总线
      配置 RCC_CLOCKTYPE_PCLK1 时钟，对应 APB1 总线
      配置 RCC_CLOCKTYPE_PCLK2 时钟，对应 APB2 总线
      配置 RCC_CLOCKTYPE_D1PCLK1 时钟，对应 APB3 总线
      配置 RCC_CLOCKTYPE_D3PCLK1 时钟，对应 APB4 总线
    */
    RCC_ClkInitStruct.ClockType = (RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_D1PCLK1 |
RCC_CLOCKTYPE_PCLK1 | \
                                   RCC_CLOCKTYPE_PCLK2  | RCC_CLOCKTYPE_D3PCLK1);

    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.SYSCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB3CLKDivider = RCC_APB3_DIV2;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_APB1_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_APB2_DIV2;
    RCC_ClkInitStruct.APB4CLKDivider = RCC_APB4_DIV2;
```

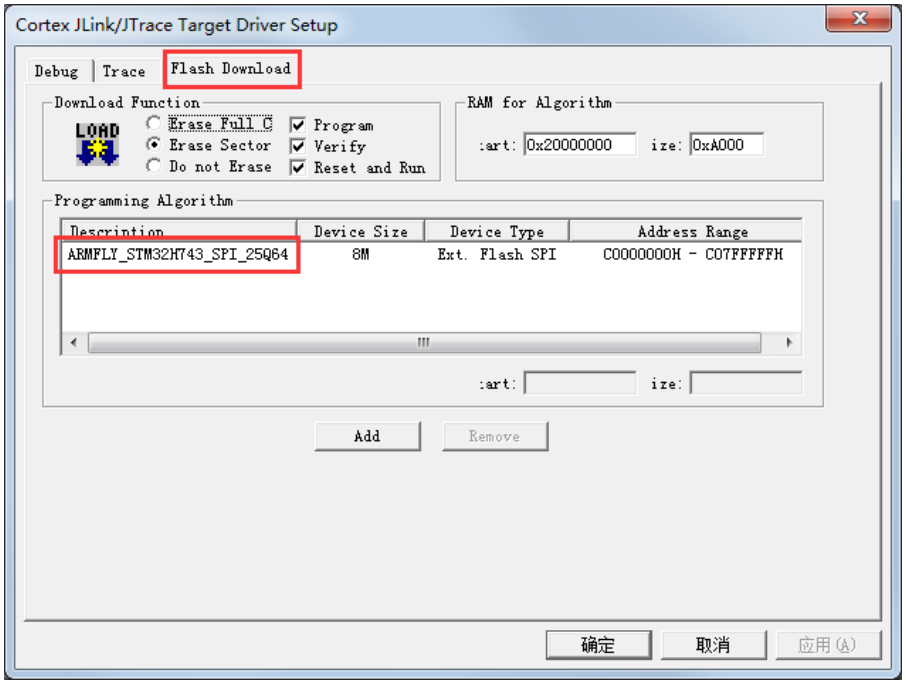
```
    /* 此函数会更新 SystemCoreClock，并重新配置 HAL_InitTick */
    ret = HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4);
    if(ret != HAL_OK)
    {
        return 1;
    }

    /*
      使用 IO 的高速模式，要使能 IO 补偿，即调用下面三个函数
      （1）使能 CSI clock
      （2）使能 SYSCFG clock
      （3）使能 I/O 补偿单元， 设置 SYSCFG_CCCSR 寄存器的 bit0
    */
    __HAL_RCC_CSI_ENABLE() ;

    __HAL_RCC_SYSCFG_CLK_ENABLE() ;

    HAL_EnableCompensationCell();

    __HAL_RCC_D2SRAM1_CLK_ENABLE();
    __HAL_RCC_D2SRAM2_CLK_ENABLE();
    __HAL_RCC_D2SRAM3_CLK_ENABLE();

    return 0;
}
```

84.4.5第 5 步，配置文件 FlashDev.c 的实现

配置如下：

```
struct FlashDevice const FlashDevice  =  {
    FLASH_DRV_VERS,                 /* 驱动版本，勿修改，这个是 MDK 定的 */
    "ARMFLY_STM32H743_SPI_25Q64", /* 算法名，添加算法到 MDK 安装目录会显示此名字 */
    EXTSPI,                         /* 设备类型 */
    0xC0000000,                     /* Flash 起始地址 */
    8 * 1024 * 1024,                /* Flash 大小，8MB */
    4096,                           /* 编程页大小 */
    0,                              /* 保留，必须为 0 */
    0xFF,                           /* 擦除后的数值 */
    6000,                           /* 页编程等待时间 */
    6000,                           /* 扇区擦除等待时间 */
    4 * 1024, 0x000000,            /* 扇区大小，扇区地址 */
    SECTOR_END
};
```

注释已经比较详细，大家根据自己的需要做修改即可。注意一点，算法名 ARMFLY_STM32H743_SPI_W25Q64 会反馈到这个地方：



84.4.6第 6 步，编程文件 FlashPrg.c 的实现

下面将文件中实现的几个函数为大家做个说明：

◆ **初始化函数 Init**

```
/*
*********************************************************************************************************
*    函 数 名: Init
*    功能说明: Flash 编程初始化
*    形    参: adr Flash 基地址，芯片首地址。
*              clk 时钟频率
*              fnc 函数代码，1 - Erase, 2 - Program, 3 - Verify
*    返 回 值: 0 表示成功， 1 表示失败
*********************************************************************************************************
*/
int Init (unsigned long adr, unsigned long clk, unsigned long fnc)
{
    int result = 0;

    /* 系统初始化 */
    SystemInit();

    /* 时钟初始化 */
    result = SystemClock_Config();
    if (result  != 0)
    {
        return 1;
    }

    bsp_InitSPIBus();
    bsp_InitSFlash();
```

```
    return 0;
}
```

◆ 复位初始化函数 Uinit

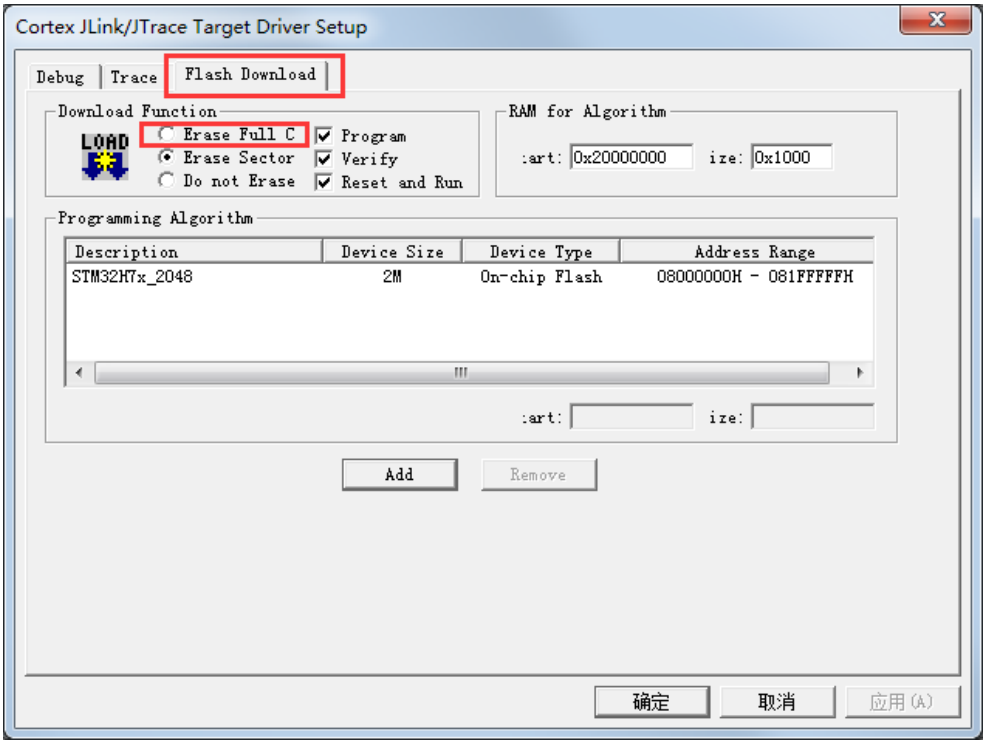
擦除，编程和校验函数后都会调用此函数，我们这里未使用。

```
/*
*********************************************************************************************************
*    函 数 名: UnInit
*    功能说明: 复位初始化
*    形    参: fnc 函数代码，1 - Erase, 2 - Program, 3 - Verify
*    返 回 值: 0 表示成功， 1 表示失败
*********************************************************************************************************
*/
int UnInit (unsigned long fnc)
{
    return (0);
}
```

复位初始化这里，直接将其设置为内存映射模式。

◆ 整个芯片擦除函数 EraseChip

如果大家配置勾选了 MDK Option 选项中此处的配置，会调用的整个芯片擦除：



实际应用中不推荐大家勾选这里，因为整个芯片擦除太耽误时间。

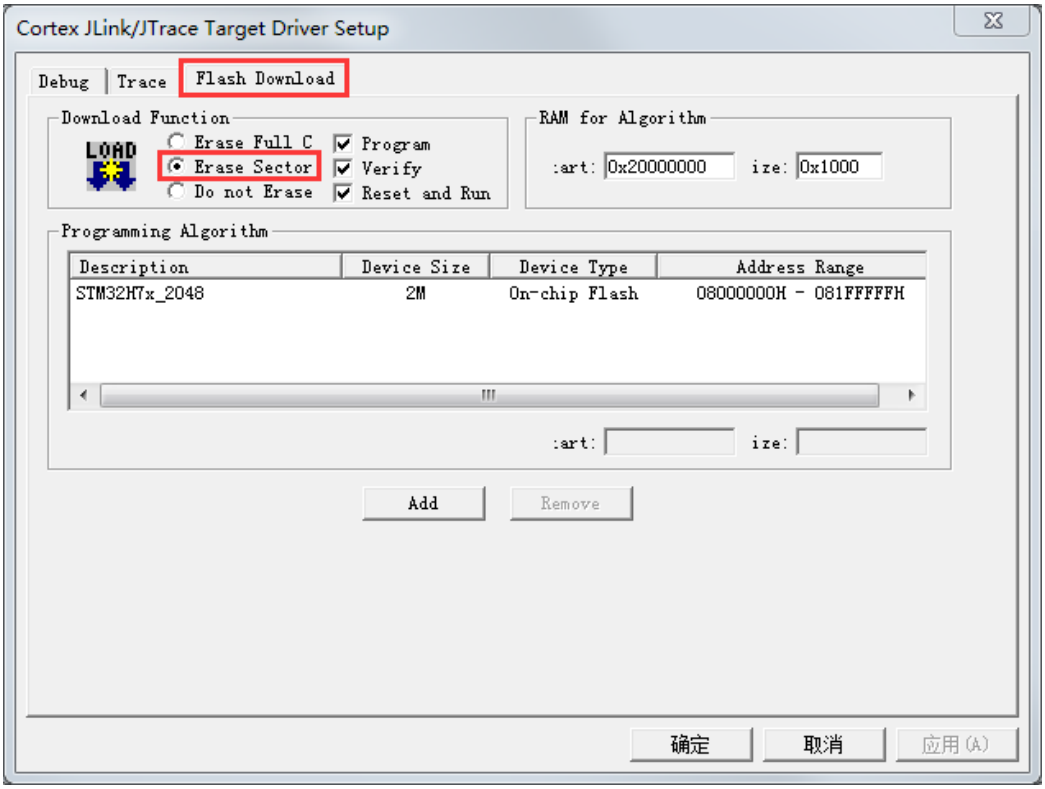
另外，如果大家的算法工程里面没有添加此函数，MDK 会调用扇区擦除函数来实现，直到所有扇区擦除完毕。

```
/*
*********************************************************************************************************
*    函 数 名: EraseChip
*    功能说明: 整个芯片擦除
*    形    参: 无
```

```
*    返 回 值: 0 表示成功， 1 表示失败
*********************************************************************************************************
*/
int EraseChip (void)
{
    sf_EraseChip();
    return 0;
}
```

◆ 扇区擦除函数 EraseSector

如果大家配置勾选了 MDK Option 选项中此处的配置，会调用扇区擦除：




```
/*
*********************************************************************************************************
*    函 数 名: EraseSector
*    功能说明: 扇区擦除
*    形    参: adr 擦除地址
*    返 回 值: 无
*********************************************************************************************************
*/
int EraseSector (unsigned long adr)
{
    adr -= SPI_FLASH_MEM_ADDR;

    sf_EraseSector(adr);

    return 0;
}
```

这里要注意两点：

（3） 程序里面的操作 adr -= SPI_FLASH_MEM_ADDR，实际传递进来的地址是带了首地址的，即

0xC0000000。特别注意，我们这里的 0xC0000000 是随意设置的，因为 STM32H7 的标准 SPI 外设并不支持内存映射。

(4) 这里执行的擦除大小要前面 FlashDev.c 文件中配置的扇区大小一致，这里是执行的 4KB 为扇区进行擦除。

◆ 页编程函数 ProgramPage

页编程函数实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: ProgramPage
*    功能说明: 页编程
*    形    参: adr 页起始地址
*              sz  页大小
*              buf 要写入的数据地址
*    返 回 值: 无
*********************************************************************************************************
*/
int ProgramPage (unsigned long adr, unsigned long sz, unsigned char *buf)
{
    adr -= SPI_FLASH_MEM_ADDR;

    sf_WriteBuffer(buf, adr, sz);

    return (0);
}
```

这里注意两点：

(3) W25Q64 的页大小是 256 字节，前面 FlashDev.c 中将页编程大小设置为 4096 字节，主要是方便擦除操作。

(4) 程序里面的操作 adr -= SPI_FLASH_MEM_ADDR，实际传递进来的地址是带了首地址的，即 0xC0000000。

◆ 读取和校验函数

校验函数实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: Verify
*    功能说明: 校验
*    形    参: adr 起始地址
*              sz  数据大小
*              buf 要校验的数据缓冲地址
*    返 回 值: -
*********************************************************************************************************
*/
unsigned char aux_buf[4096];
unsigned long Verify (unsigned long adr, unsigned long sz, unsigned char *buf)
{
    int i;

    adr -= SPI_FLASH_MEM_ADDR;
```

```
    sf_ReadBuffer(aux_buf, adr, sz);

    for (i = 0; i< sz; i++)
    {
        if (aux_buf[i] != buf[i])
        return (adr+i);              /* 校验失败 */
    }

    adr += SPI_FLASH_MEM_ADDR;
    return (adr+sz);                 /* 校验成功 */
}
```

84.4.7第 7 步，修改 SPI Flash 驱动文件（引脚，命令等）

最后一步就是 SPI Flash（W25Q64）的驱动修改，大家可以根据自己的需求做修改。使用的引脚定义在文件 bsp_spi_bus.c：

```
/*
*********************************************************************************************************
*                                    时钟，引脚，DMA，中断等宏定义
*********************************************************************************************************
*/
#define SPIx                             SPI1
#define SPIx_CLK_ENABLE()                __HAL_RCC_SPI1_CLK_ENABLE()
#define DMAx_CLK_ENABLE()                __HAL_RCC_DMA2_CLK_ENABLE()

#define SPIx_FORCE_RESET()               __HAL_RCC_SPI1_FORCE_RESET()
#define SPIx_RELEASE_RESET()             __HAL_RCC_SPI1_RELEASE_RESET()

#define SPIx_SCK_CLK_ENABLE()            __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_SCK_GPIO                    GPIOB
#define SPIx_SCK_PIN                     GPIO_PIN_3
#define SPIx_SCK_AF                          GPIO_AF5_SPI1

#define SPIx_MISO_CLK_ENABLE()           __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MISO_GPIO                   GPIOB
#define SPIx_MISO_PIN                    GPIO_PIN_4
#define SPIx_MISO_AF                     GPIO_AF5_SPI1

#define SPIx_MOSI_CLK_ENABLE()           __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MOSI_GPIO                   GPIOB
#define SPIx_MOSI_PIN                    GPIO_PIN_5
#define SPIx_MOSI_AF                     GPIO_AF5_SPI1
```

硬件设置了之后，剩下就是 SPI Flash 相关的几个配置和片选引脚配置，在文件 bsp_spi_flash.c：

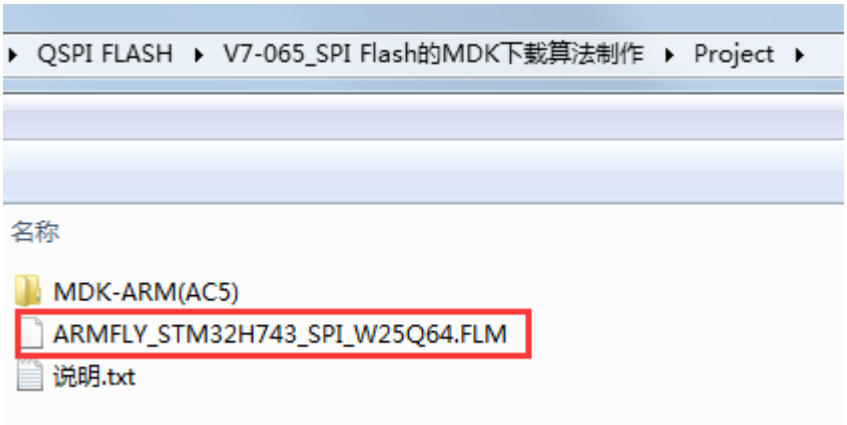
主要是下面这几个：

```
/* 串行 Flash 的片选 GPIO 端口，  PD13  */
#define SF_CS_CLK_ENABLE()              __HAL_RCC_GPIOD_CLK_ENABLE()
#define SF_CS_GPIO                      GPIOD
#define SF_CS_PIN                       GPIO_PIN_13

#define SF_CS_0()                       SF_CS_GPIO->BSRR = ((uint32_t)SF_CS_PIN << 16U)
#define SF_CS_1()                       SF_CS_GPIO->BSRR = SF_CS_PIN
```

```
#define CMD_AAI       0xAD     /* AAI 连续编程指令(FOR SST25VF016B) */
#define CMD_DISWR     0x04     /* 禁止写, 退出 AAI 状态 */
#define CMD_EWRSR     0x50     /* 允许写状态寄存器的命令 */
#define CMD_WRSR      0x01     /* 写状态寄存器命令 */
#define CMD_WREN      0x06     /* 写使能命令 */
#define CMD_READ      0x03     /* 读数据区命令 */
#define CMD_RDSR      0x05     /* 读状态寄存器命令 */
#define CMD_RDID      0x9F     /* 读器件 ID 命令 */
#define CMD_SE        0x20     /* 擦除扇区命令 */
#define CMD_BE        0xC7     /* 批量擦除命令 */
#define DUMMY_BYTE    0xA5     /* 哑命令，可以为任意值，用于读操作 */

#define WIP_FLAG      0x01     /* 状态寄存器中的正在编程标志（WIP） */
```

84.5 SPI Flash 的 MDK 下载算法使用方法

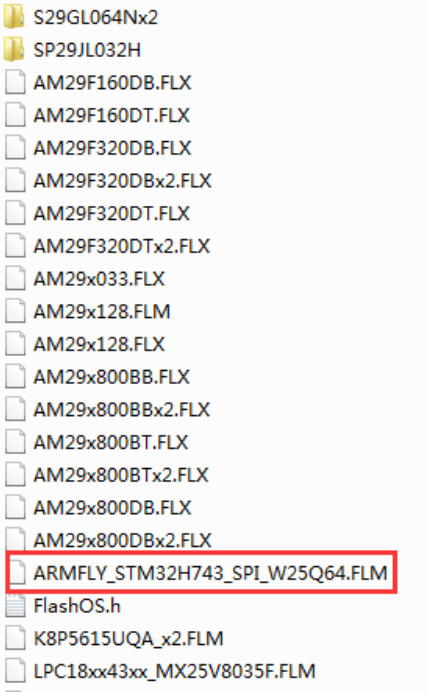
编译本章教程配套的例子，生成的算法文件位于此路径下：



84.5.1 下载算法存放位置

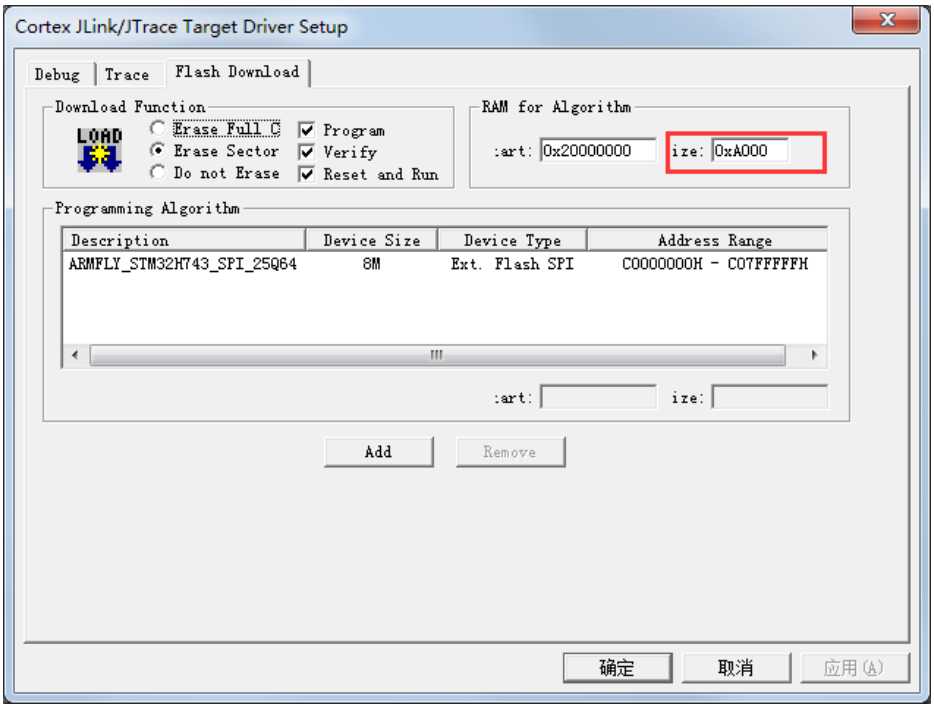
生成算法文件后，需要大家将其存到 MDK 安装目录，有两个位置可以存放，任选其一，推荐第 2 种：

- 第 1 种：存放到 MDK 的 STM32H7 软包安装目录里面：\Keil\STM32H7xx_DFP\2.6.0\CMSIS\Flash（软包版本不同，数值 2.6.0 不同）。
- 第 2 种：MDK 的安装目录 \ARM\Flash 里面。



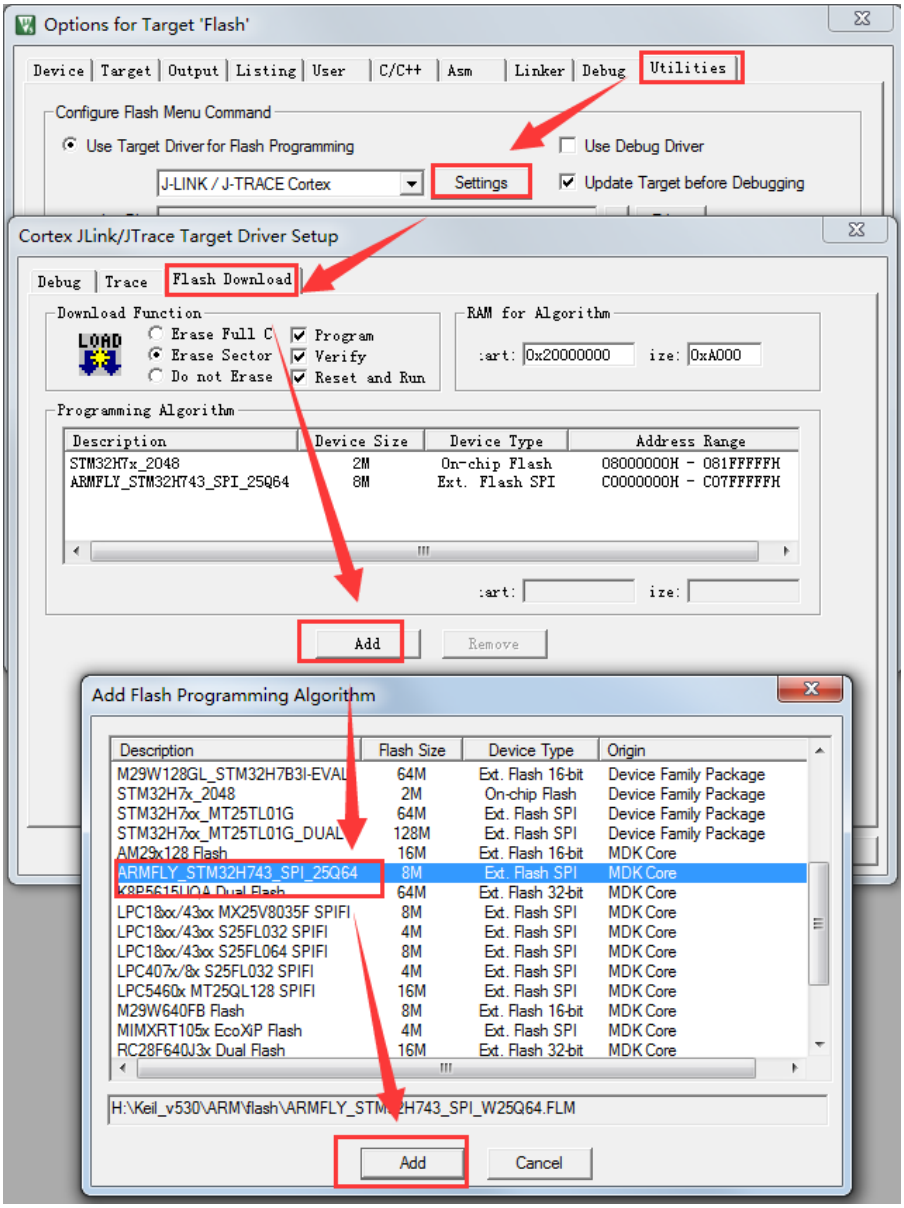
84.5.2 下载配置

注意这里一定要够大，否则会提示算法文件无法加载：



我们这里是将其加到 DTCM 中，即首地址为 0x20000000，大家也可以存储到任意其它 RAM 地址，只要空间还够加载算法文件即可。推荐使用 AXI SRAM（地址 0x24000000），因为这块 RAM 空间足够大。

如果要下载程序到 QSPI Flash 里面，需要做如下配置：



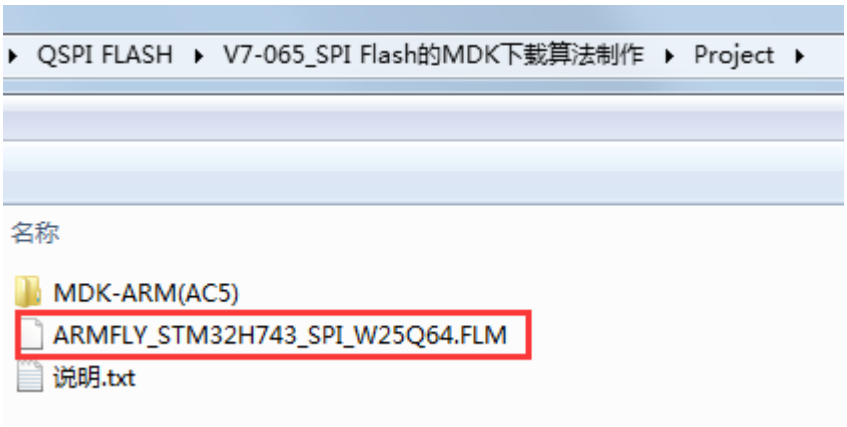
84.5.3 验证算法文件是否可以正常使用

为了验证算法文件是否可以正常使用，大家可以运行本教程第 86 章配套的例子。

84.6 实验例程说明

本章配套例子：V7-065_SPI Flash 的 MDK 下载算法制作

编译后，算法文件会存到此路径下：



84.7 总结

本章节就为大家讲解这么多，为了熟练掌握，大家可以尝试自己实现一个 Flash 下载算法。

第85章 STM32H7 的 SPI 总线应用之 SPI Flash 的 STM32CubeProg 下载算法制作

本章节为大家讲解 STM32CubeProg 下载算法制作方法。

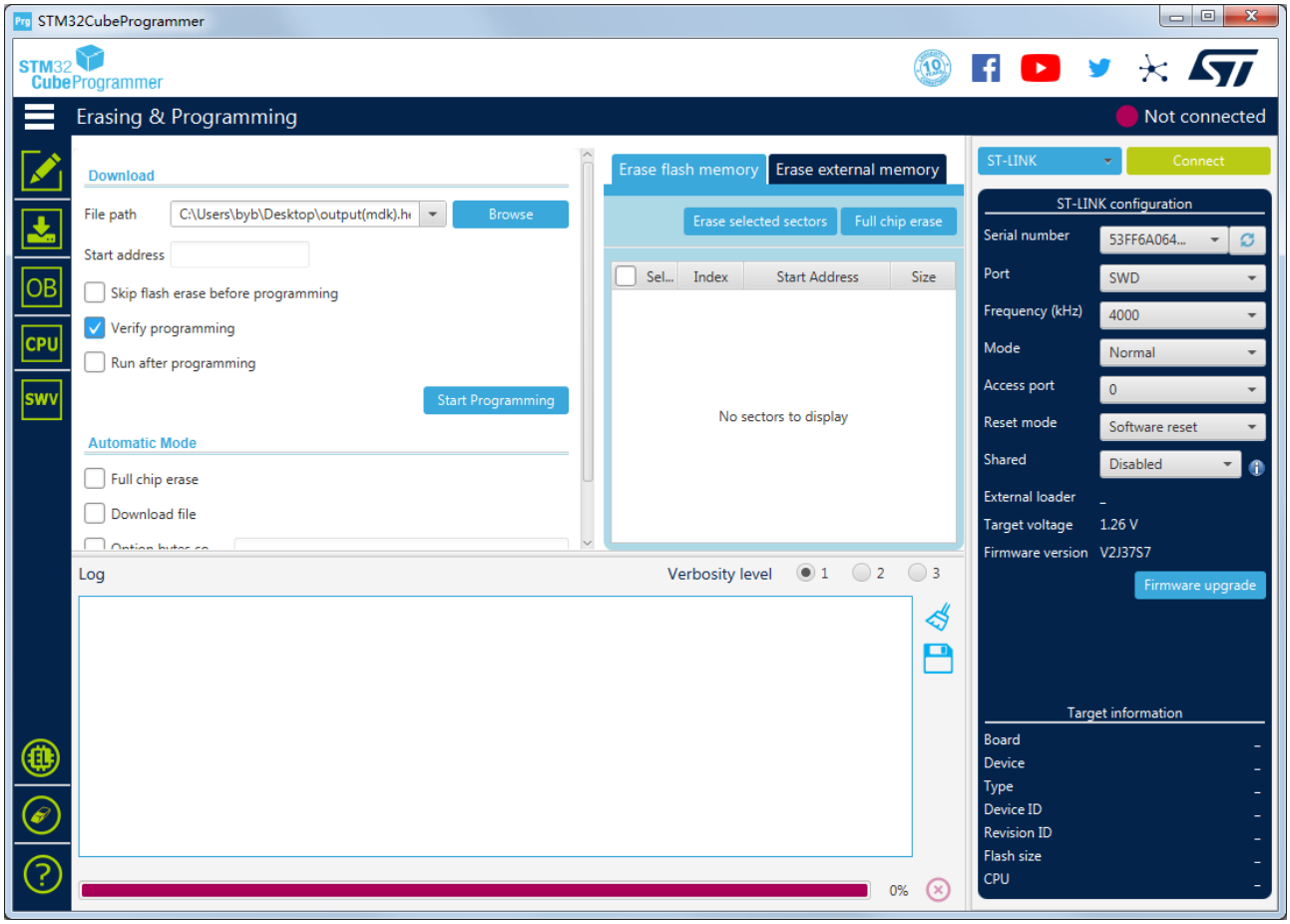
85.1 初学者重要提示
85.2 STM32CubeProg 简介
85.3 STM32CubeProg 下载算法基础知识
85.4 创建 STM32CubeProg 下载算法通用流程
85.4 SPI Flash 的 STM32CubeProg 下载算法制作
85.5 SPI Flash 的 STM32CubeProg 下载算法使用方法
85.6 实验例程说明
85.7 总结

85.1 初学者重要提示

- QSPI Flash 的相关知识点可以看第 78 章和 79 章。
- QSPI Flash 下载算法文件直接采用 HAL 库制作，方便大家自己修改。
- STM32CubeProg 下载算法制作和 MDK 下载算法制作基本是一样
- 本教程的第 68 章 USB DFU 和第 69 章串口 IAP 章节为大家介绍过 STM32CubeProg 的用法。
  STM32CubeProg 下载
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=97370 。

85.2 STM32CubeProg 简介

STM32CubeProg，此软件实现了之前的 DfuSe，STLINK 小软件和 Flashloader 三合一，并且支持外部 EEPROM，NOR Flash，SPI Flash，NAND Flash 等烧写，也支持 OTA 编程。



85.3 STM32CubeProg 下载算法基础知识

STM32CubeProg 下载算法是一种用于擦除应用程序或将应用程序下载到 Flash 的程序代码。ST 自家的芯片都自带下载算法，存放在 STM32CubeProg 安装目录里面，但不支持的需要我们自己制作，本章教程为此而生。

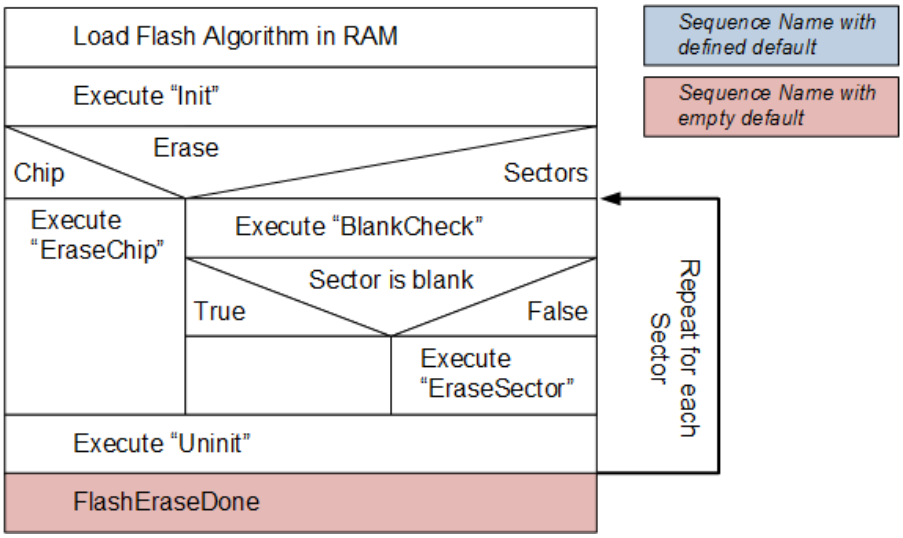
85.3.1 程序能够通过下载算法下载到芯片的核心思想

认识到这点很重要：通过 IDE 开发环境创建一批与地址信息无关的算法文件，实现的功能主要有初始化，擦除，编程，读取，校验等，然后 STM32CubeProg 下载阶段，会将算法文件加载到芯片的内部 RAM 里面，然后 STM32CubeProg 通过与这个算法文件的交互，实现程序下载，数据读取等操作。

85.3.2 算法程序中擦除操作执行流程

注：下面是 MDK 的算法执行流程，STM32CubeProg 是类似的。

擦除操作大致流程：

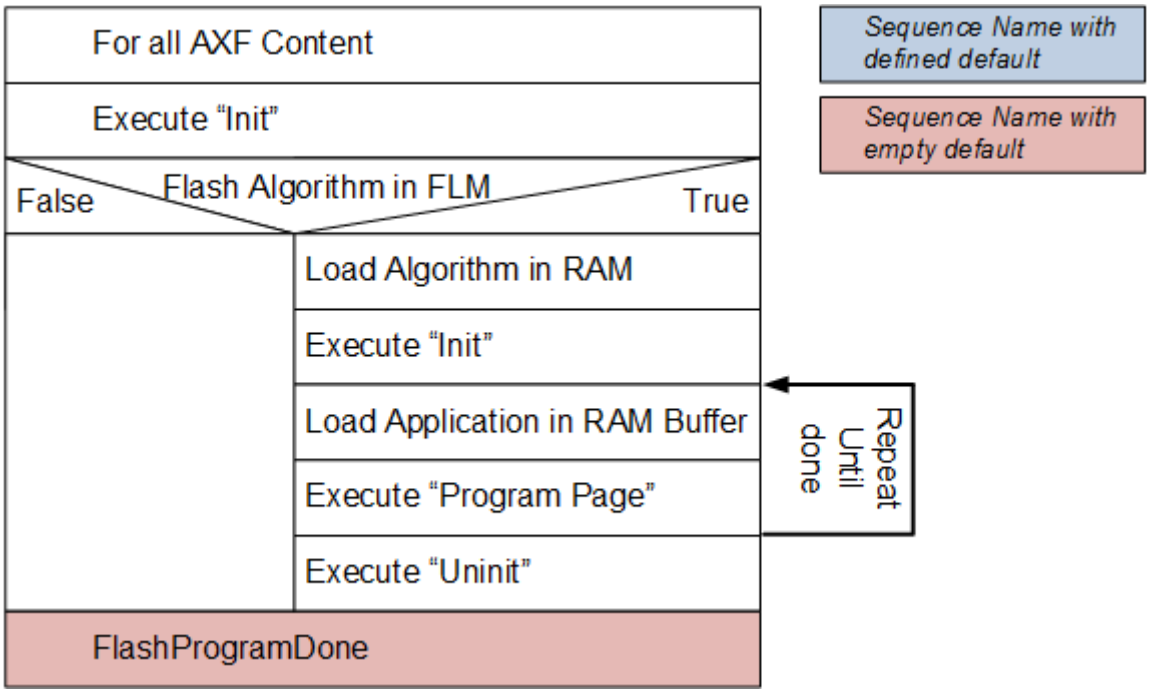


- 加载算法到芯片 RAM。
- 执行初始化函数 Init。
- 执行擦除操作，根据用户配置，这里可以选择整个芯片擦除或者扇区擦除。
- 执行 Uinit 函数。
- 操作完毕。

85.3.3算法程序中编程操作执行流程

注：下面是 MDK 的算法执行流程，STM32CubeProg 是类似的（没有 Unint 函数）。

编程操作大致流程：



- 针对 IDE 生成的 axf（elf）可执行文件做 Init 初始化，这个 axf（elf）文件是指的大家自己创建应用

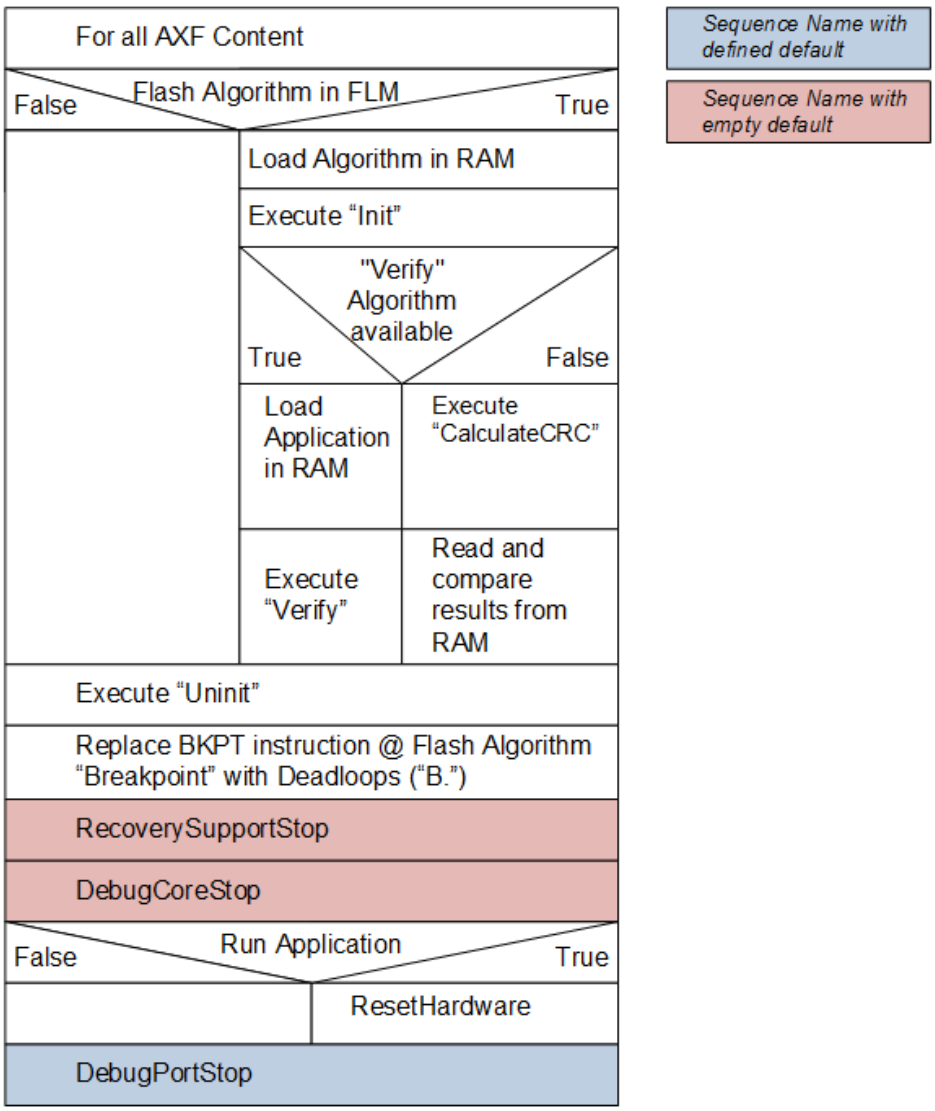
程序生成的。

- 查看 Flash 算法是否在 FLM 文件。如果没有在，操作失败。如果在：
  - 加载算法到 RAM。
  - 执行 Init 函数。
  - 加载用户到 RAM 缓冲。
  - 执行 Program Page 页编程函数。
  - 执行 Uninit 函数。
- 操作完毕。

85.3.4算法程序中校验操作执行流程

注：下面是 MDK 的算法执行流程，STM32CubeProg 是类似的（没有 Unint 函数）。

校验操作大致流程：



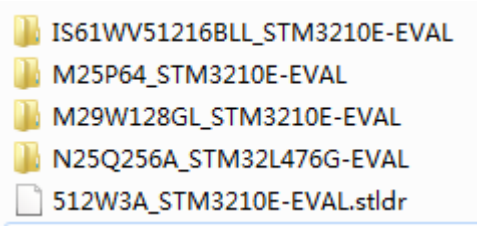
- 校验要用到 IDE 生成的 axf（elf）可执行文件。校验就是 axf（elf）文件中下载到芯片的程序和实际下载的程序读出来做比较。
- 查看 Flash 算法是否在 FLM 文件。如果没有在，操作失败。如果在：
  - 加载算法到 RAM。
  - 执行 Init 函数。
  - 查看校验算法是否存在
    - 如果有，加载应用程序到 RAM 并执行校验。
    - 如果没有，执行计算 CRC，将芯片中读取数据出来和 RAM 中加载应用计算输出的 CRC 值做比较。
  - 执行 Uninit 函数。
  - 替换 BKPT（BreakPoint 断点指令）为 B. 死循环指令。
  - 执行 RecoverySupportStop，回复支持停止。
  - 执行 DebugCoreStop，调试内核停止。
- 运行应用：
  - 执行失败
  - 执行成功，再执行硬件复位。
- 操作完毕，停止调试端口。

85.4 创建 STM32CubeProg 下载算法通用流程

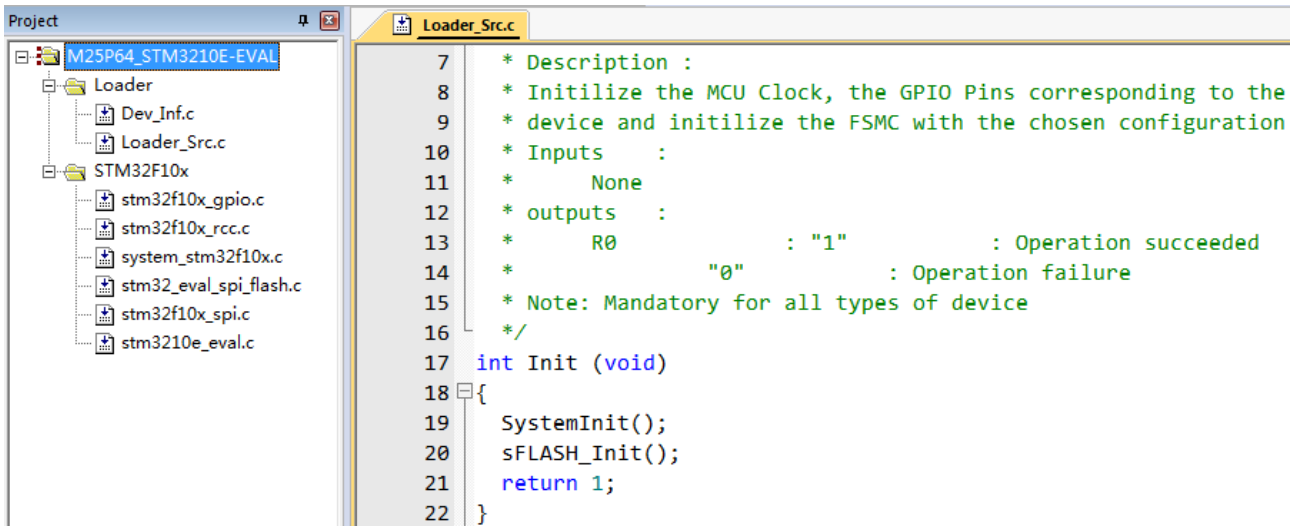
下面是 STM32CubeProg 给的一种大致操作流程，不限制必须采用这种方法，自己创建也可以的。

85.4.1 第 1 步，使用 STM32CubeProg 提供好的程序模板

位于路径：STMicroelectronics\STM32Cube\STM32CubeProgrammer\bin\ExternalLoader



以 M25P64 为例（注，其余步骤就以这个为例子进行说明）：

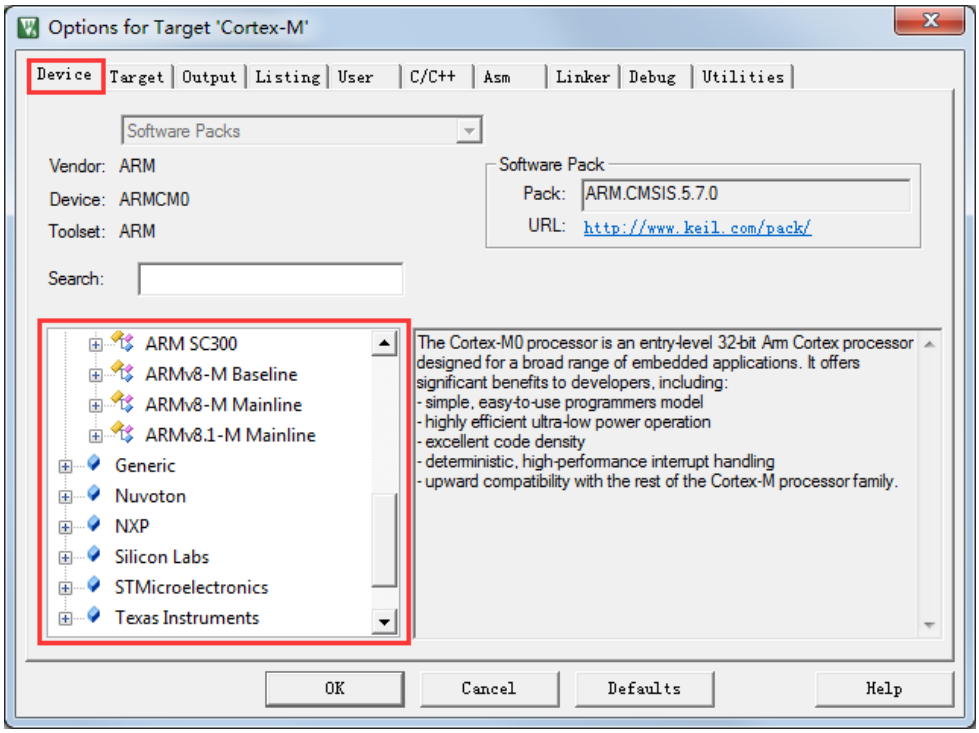


85.4.2 第 2 步，修改工程名

STM32CubeProg 提供的工程模板原始名字是 M25P64_STM3210E-EVAL.uvproj，大家可以根据自己的需要做修改。比如修改为 MyDevice.uvproj（MDK4 的后缀）或者 MyDevice.uvprojx（MDK5 的后缀）。

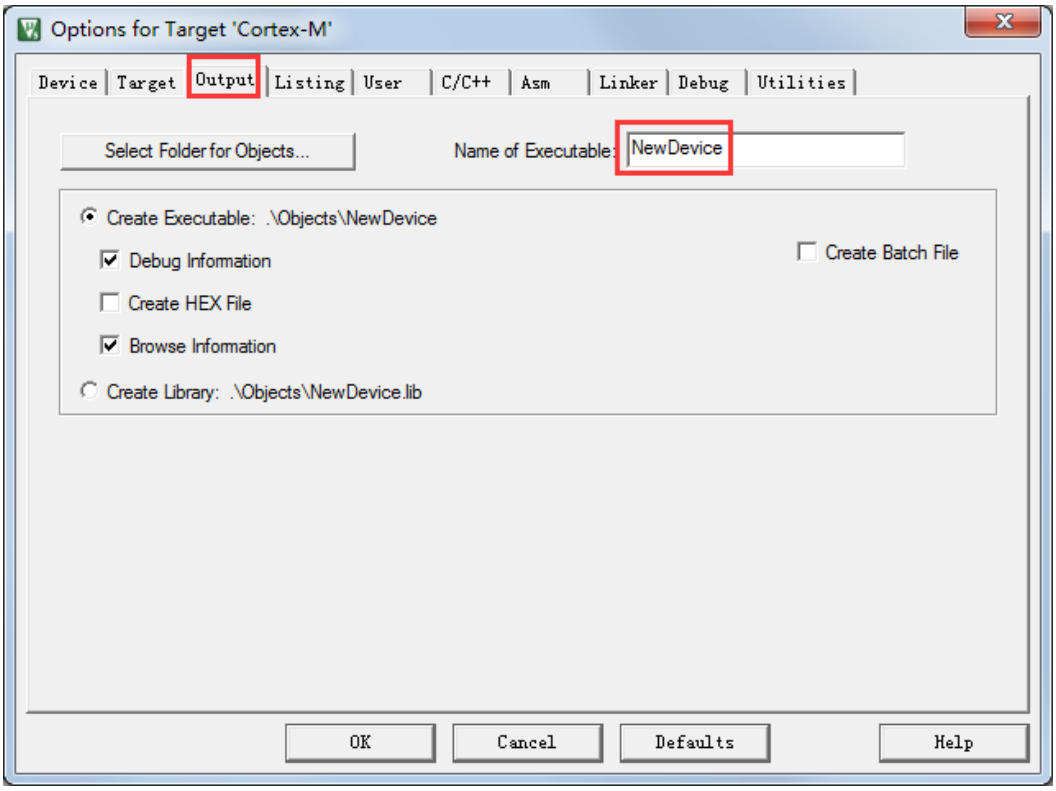
85.4.3 第 3 步，修改使用的器件

在 MDK 的 Option 选项里面设置使用的器件。

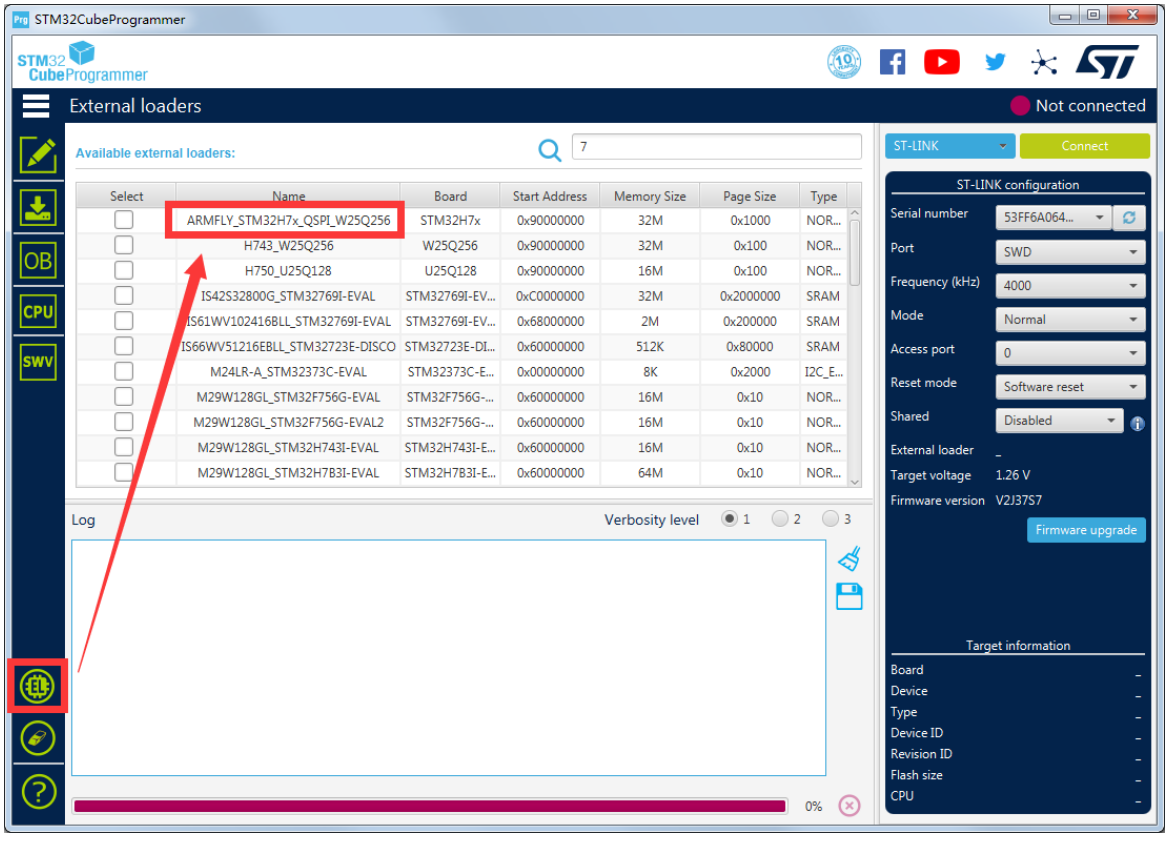


85.4.4第 4 步，修改输出算法文件的名字

这个名字是方便用户查看的，比如设置为 stm32h7，那么输出的算法文件就是 stm32h7.stldr。



注:STM32CubeProg 软件里面别的文件名并不采用这个,而是由用户在文件 Dev_Inf.c 里面定义的:



85.4.5第 5 步，修改使用的库文件和头文件路径

根据大家使用的主控芯片，添加相应的库文件和头文件路径。

85.4.6第 5 步，修改编程算法文件 Loader_Src.c

模板工程里面提供的是 M25P64，部分代码如下：

```
/**
  * Description :
  * Initilize the MCU Clock, the GPIO Pins corresponding to the
  * device and initilize the FSMC with the chosen configuration
  * Inputs    :
  *      None
  * outputs   :
  *      R0             : "1"             : Operation succeeded
  *             "0"             : Operation failure
  * Note: Mandatory for all types of device
  */
int Init (void)
{
  SystemInit();
  sFLASH_Init();
  return 1;
}


/**
  * Description :
  * Read data from the device
  * Inputs    :
  *      Address       : Write location
  *      Size          : Length in bytes
  *      buffer        : Address where to get the data to write
  * outputs   :
  *      R0             : "1"             : Operation succeeded
  *             "0"             : Operation failure
  * Note: Mandatory for all types except SRAM and PSRAM
  */
int Read (uint32_t Address, uint32_t Size, uint8_t* buffer)
{
  sFLASH_ReadBuffer(buffer, Address, Size);
  return 1;
}


/**
  * Description :
  * Write data from the device
  * Inputs    :
  *      Address       : Write location
  *      Size          : Length in bytes
  *      buffer        : Address where to get the data to write
  * outputs   :
```

```
  *      RO           : "1"              : Operation succeeded
  *                     "0"              : Operation failure
  * Note: Mandatory for all types except SRAM and PSRAM
  */
int Write (uint32_t Address, uint32_t Size, uint8_t* buffer)
{
  sFLASH_WriteBuffer(buffer, Address, Size);
  return 1;
}


/**
  * Description :
  * Erase a full sector in the device
  * Inputs    :
  *     None
  * outputs   :
  *     RO             : "1" : Operation succeeded
  *            "0" : Operation failure
  * Note: Not Mandatory for SRAM PSRAM and NOR_FLASH
  */
int MassErase (void)
{
  sFLASH_EraseBulk();
  return 1;
}
```

85.4.7第 6 步，修改配置文件 Dev_Inf.c

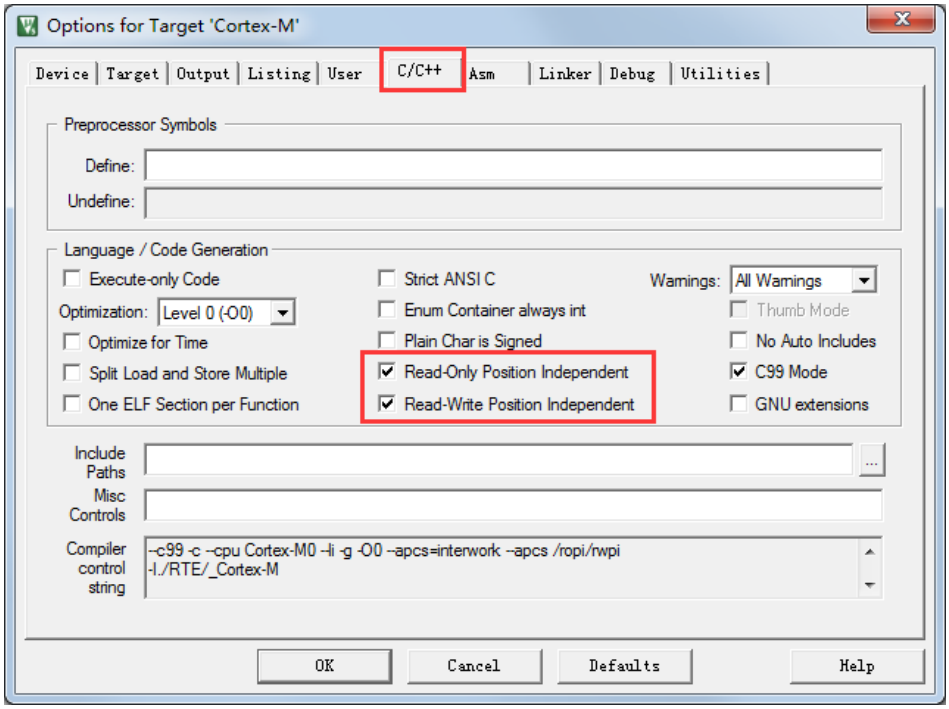
模板工程里面提供简单的配置说明：

```
#if defined (__ICCARM__)
__root struct StorageInfo const StorageInfo  =  {
#else
struct StorageInfo const StorageInfo  =  {
#endif
   "M25P64_STM3210E-EVAL",        // Device Name + version number
   SPI_FLASH,                     // Device Type
   0x00000000,                    // Device Start Address
   0x00800000,                    // Device Size in Bytes (8MBytes/64Mbits)
   0x00000100,                    // Programming Page Size 16Bytes
   0xFF,                          // Initial Content of Erased Memory
// Specify Size and Address of Sectors (view example below)
   0x00000080, 0x00010000,        // Sector Num : 128 ,Sector Size: 64KBytes
   0x00000000, 0x00000000,
};
```

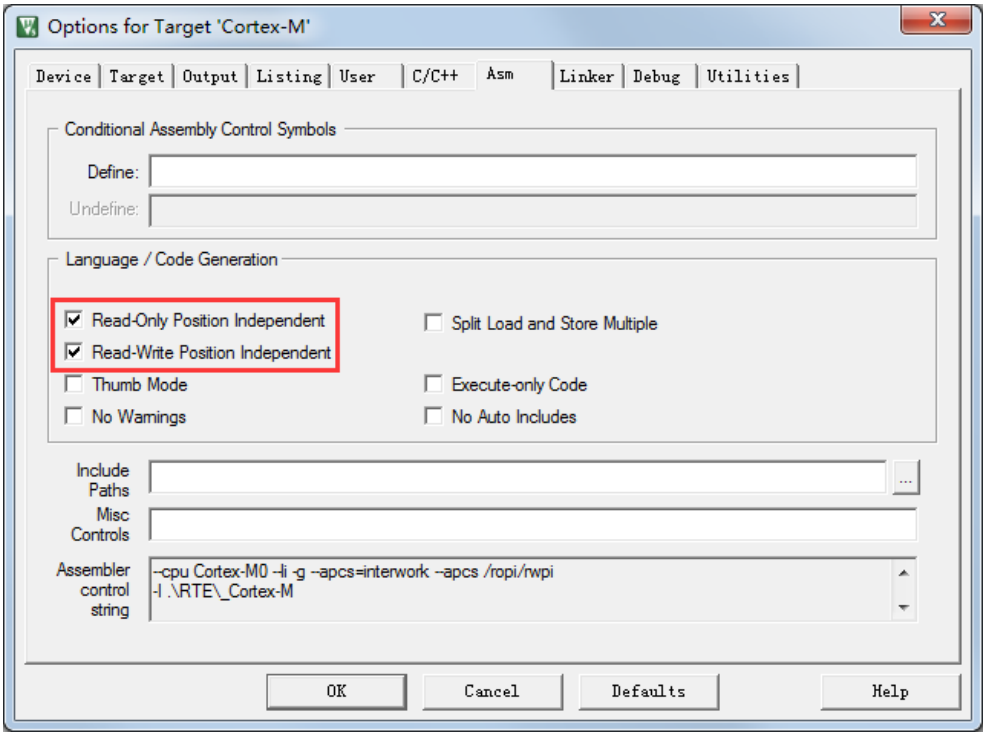
注：名字 M25P64_STM3210E-EVAL 就是我们第 4 步所说的。STM32CubeProg 会识别出这个名字。

85.4.8第 7 步，保证生成的算法文件中 RO 和 RW 段的独立性，即与地址无关。

C 和汇编的配置都勾选上：



汇编：



如果程序的所有只读段都与位置无关，则该程序为只读位置无关（ROPI, Read-only position independence）。ROPI 段通常是位置无关代码（PIC,position-independent code），但可以是只读数据，也可以是 PIC 和只读数据的组合。选择 “ ROPI”选项，可以避免不得不将代码加载到内存中的特定位置。这对于以下例程特别有用：

（1）加载以响应运行事件。

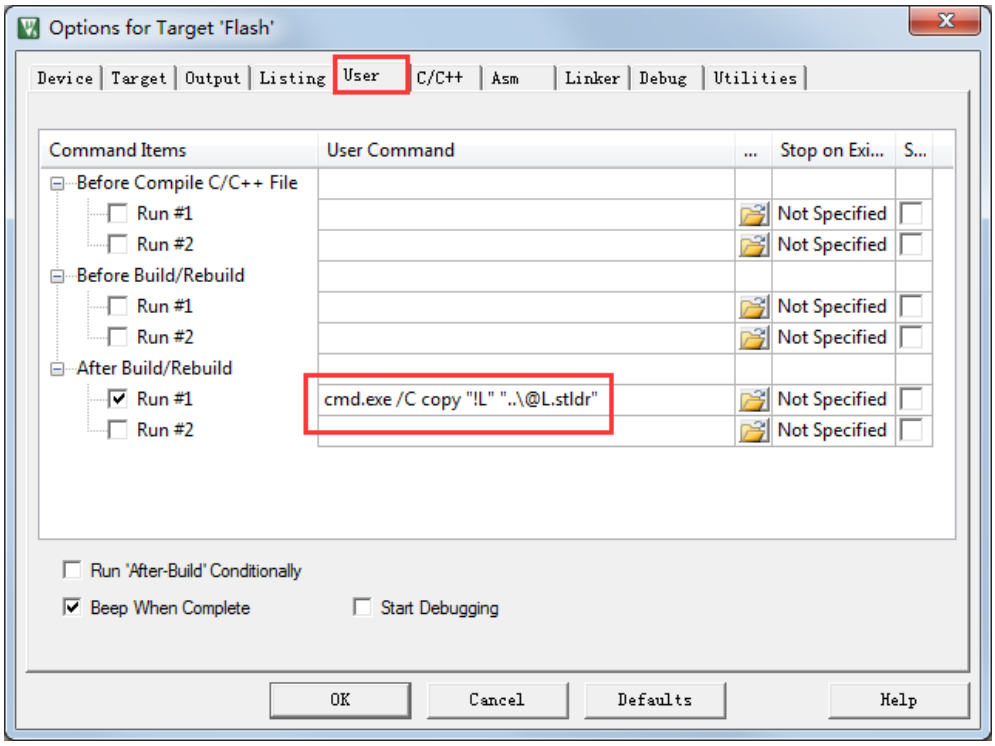
（2）在不同情况下使用其他例程的不同组合加载到内存中。

（3）在执行期间映射到不同的地址。

使用 Read-Write position independence 同理，表示的可读可写数据段。

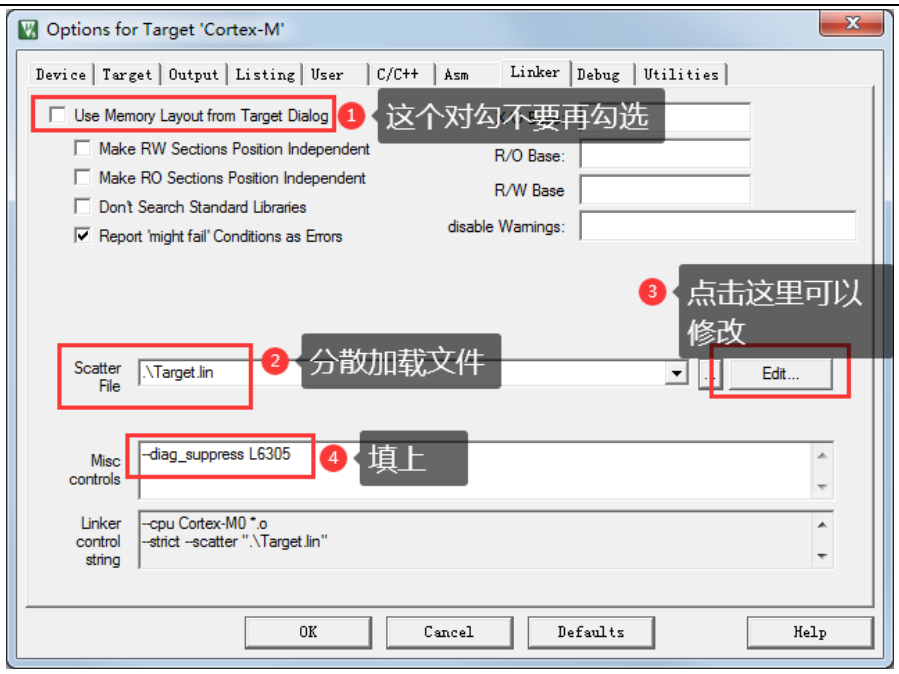
85.4.9第 8 步，将程序可执行文件 axf 修改为 stldr 格式

通过下面的命令就可以将生成的 axf 可执行文件修改为 stldr。



85.4.10 第 9 步，分散加载设置

我们这里的分散加载文件直接使用 MDK 模板工程里提供好的即可。



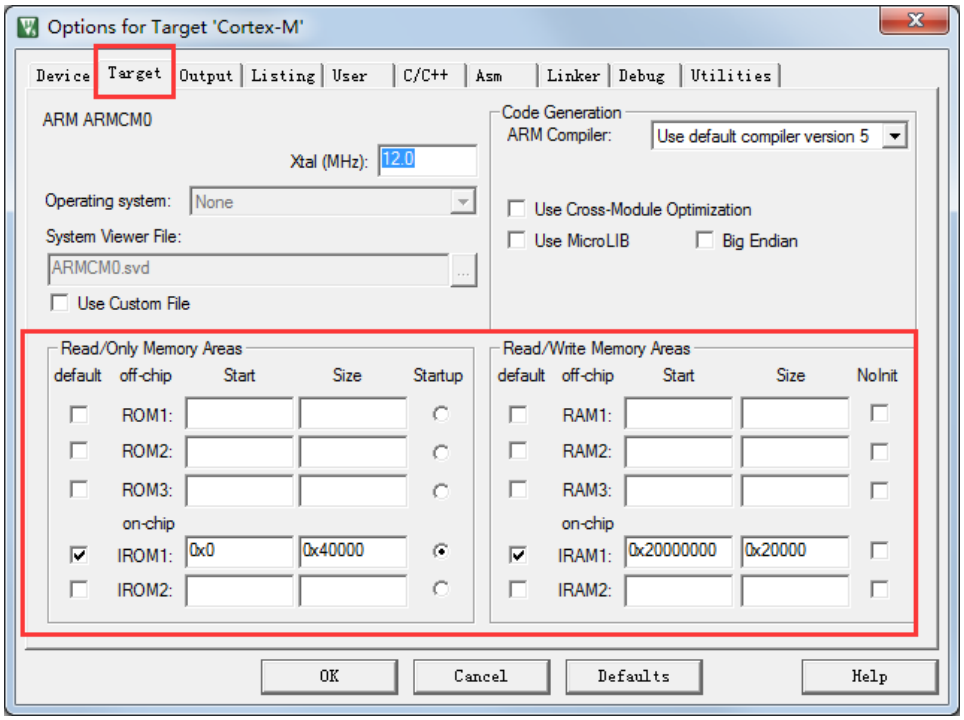
分散加载文件中的内容如下：

```
FLASH_LOADER 0x20000004 PI    ; FlashLoader Functions
{
  PrgCode +0            ; Code
  {
    * (+RO)
  }
  PrgData +0            ; Data
  {
    * (+RW,+ZI)
  }
}

DEVICE_INFO +0                ; Device Info
{
  DevInfo +0            ; Info structure
  {
    dev_inf.o
  }
}
```

这里要修改下 Flash 算法加载地址，将 0x20000004 修改为 STM32H7 的 RAM 地址，任何 RAM 块地址均可，只要够存储 Flash 算法。推荐设置为 AXI SRAM 地址 0x24000004，因为空间够大，有 512KB。

--diag_suppress L6305 用于屏蔽 L6503 类型警告信息。

特别注意，设置了分散加载后，此处的配置就不再起作用了：



85.5 SPI Flash 的 STM32CubeProg 下载算法制作

下面将 SPI Flash 算法制作过程中的几个关键点为大家做个说明。

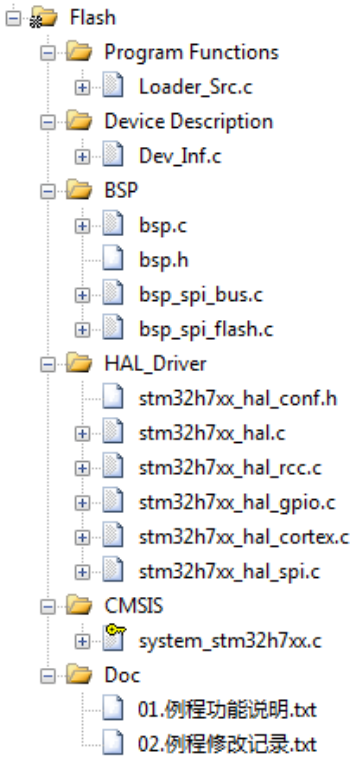
85.5.1 第 1 步，制作前重要提示

这两点非常重要：

- 程序里面不要开启任何中断，全部查询方式。
- HAL 库里面各种时间基准相关的 API 全部处理掉。简单省事些，我们这里是直接注释，采用死等即可。无需做超时等待，因为超时后，已经意味着操作失败了，跟死等没有区别。

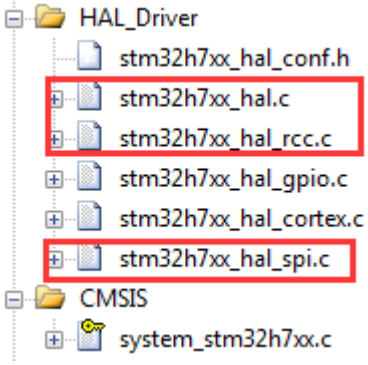
85.5.2 第 2 步，准备一个工程模板

推荐大家直接使用我们本章工程准备好的模板即可，如果大家自己制作，注意一点，请使用当前最新的 HAL 库。



85.5.3第 3 步，修改 HAL 库

这一步比较重要，主要修改了以下三个文件：



主要是修改了 HAL 库时间基准相关的几个 API，并注释掉了一批无关的 API。具体修改内容，大家可以找个比较软件，对比修改后的这个文件和 CubeH7 软件包 V1.8.0（软件包里面的 HAL 库版本是 V1.9.0）的差异即可。

85.5.4第 4 步，时钟初始化

我们已经用不到滴答定时器了，直接在 bsp.c 文件里面对滴答初始化函数做重定向：

```
/*
*********************************************************************************************************
*    函 数 名: HAL_InitTick
```

```
*    功能说明: 重定向，不使用
*    形    参: TickPriority
*    返 回 值: 无
*********************************************************************************************************
*/
HAL_StatusTypeDef HAL_InitTick(uint32_t TickPriority)
{
    return HAL_OK;
}
```

然后就是 HSE 外置晶振的配置，大家根据自己的板子实际外挂晶振大小，修改 stm32h7xx_hal_conf.h 文件中 HSE_VALUE 大小，实际晶振多大，这里就修改为多大：

```
#if !defined  (HSE_VALUE)
#define HSE_VALUE    ((uint32_t)25000000) /*!< Value of the External oscillator in Hz */
#endif /* HSE_VALUE */
```

最后修改 PLL：

```
/*
*********************************************************************************************************
*    函 数 名: SystemClock_Config
*    功能说明: 初始化系统时钟
*            System Clock source            = PLL (HSE)
*            SYSCLK(Hz)                     = 400000000 (CPU Clock)
*            HCLK(Hz)                       = 200000000 (AXI and AHBs Clock)
*            AHB Prescaler                  = 2
*            D1 APB3 Prescaler              = 2 (APB3 Clock  100MHz)
*            D2 APB1 Prescaler              = 2 (APB1 Clock  100MHz)
*            D2 APB2 Prescaler              = 2 (APB2 Clock  100MHz)
*            D3 APB4 Prescaler              = 2 (APB4 Clock  100MHz)
*            HSE Frequency(Hz)              = 25000000
*            PLL_M                          = 5
*            PLL_N                          = 160
*            PLL_P                          = 2
*            PLL_Q                          = 4
*            PLL_R                          = 2
*            VDD(V)                         = 3.3
*            Flash Latency(WS)              = 4
*    形    参: 无
*    返 回 值: 1 表示失败，0 表示成功
*********************************************************************************************************
*/
int SystemClock_Config(void)
{
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    HAL_StatusTypeDef ret = HAL_OK;

    /* 锁住 SCU(Supply configuration update) */
    MODIFY_REG(PWR->CR3, PWR_CR3_SCUEN, 0);

    /*
      1、芯片内部的 LDO 稳压器输出的电压范围，可选 VOS1，VOS2 和 VOS3，不同范围对应不同的 Flash 读速度，
         详情看参考手册的 Table 12 的表格。
      2、这里选择使用 VOS1，电压范围 1.15V - 1.26V。
    */
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

    while(!__HAL_PWR_GET_FLAG(PWR_FLAG_VOSRDY)) {}
```

```
    /* 使能 HSE，并选择 HSE 作为 PLL 时钟源 */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.HSIState = RCC_HSI_OFF;
    RCC_OscInitStruct.CSIState = RCC_CSI_OFF;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;

    RCC_OscInitStruct.PLL.PLLM = 5;
    RCC_OscInitStruct.PLL.PLLN = 160;
    RCC_OscInitStruct.PLL.PLLP = 2;
    RCC_OscInitStruct.PLL.PLLR = 2;
    RCC_OscInitStruct.PLL.PLLQ = 4;

    RCC_OscInitStruct.PLL.PLLVCOSEL = RCC_PLL1VCOWIDE;
    RCC_OscInitStruct.PLL.PLLRGE = RCC_PLL1VCIRANGE_2;
    ret = HAL_RCC_OscConfig(&RCC_OscInitStruct);
    if(ret != HAL_OK)
    {
        return 1;
    }

    /*
      选择 PLL 的输出作为系统时钟
      配置 RCC_CLOCKTYPE_SYSCLK 系统时钟
      配置 RCC_CLOCKTYPE_HCLK 时钟，对应 AHB1，AHB2，AHB3 和 AHB4 总线
      配置 RCC_CLOCKTYPE_PCLK1 时钟，对应 APB1 总线
      配置 RCC_CLOCKTYPE_PCLK2 时钟，对应 APB2 总线
      配置 RCC_CLOCKTYPE_D1PCLK1 时钟，对应 APB3 总线
      配置 RCC_CLOCKTYPE_D3PCLK1 时钟，对应 APB4 总线
    */
    RCC_ClkInitStruct.ClockType = (RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_D1PCLK1 |
RCC_CLOCKTYPE_PCLK1 | \
                                   RCC_CLOCKTYPE_PCLK2  | RCC_CLOCKTYPE_D3PCLK1);

    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.SYSCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB3CLKDivider = RCC_APB3_DIV2;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_APB1_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_APB2_DIV2;
    RCC_ClkInitStruct.APB4CLKDivider = RCC_APB4_DIV2;

    /* 此函数会更新 SystemCoreClock，并重新配置 HAL_InitTick */
    ret = HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4);
    if(ret != HAL_OK)
    {
        return 1;
    }

    /*
      使用 IO 的高速模式，要使能 IO 补偿，即调用下面三个函数
      （1）使能 CSI clock
      （2）使能 SYSCFG clock
      （3）使能 I/O 补偿单元， 设置 SYSCFG_CCCSR 寄存器的 bit0
    */
    __HAL_RCC_CSI_ENABLE() ;
```

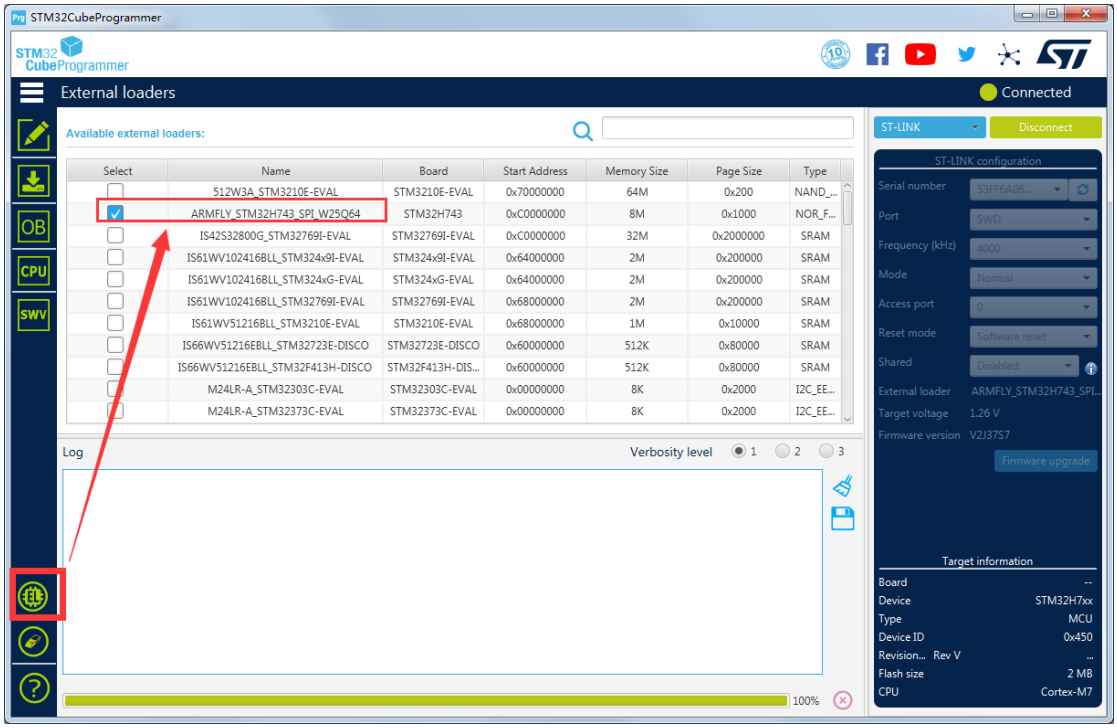
```
    __HAL_RCC_SYSCFG_CLK_ENABLE() ;

    HAL_EnableCompensationCell();

    __HAL_RCC_D2SRAM1_CLK_ENABLE();
    __HAL_RCC_D2SRAM2_CLK_ENABLE();
    __HAL_RCC_D2SRAM3_CLK_ENABLE();

    return 0;
}
```

85.5.5第 5 步，配置文件 Dev_Inf.c 的实现

配置如下：

```
#if defined (__ICCARM__)
__root struct StorageInfo const StorageInfo  =  {
#else
struct StorageInfo const StorageInfo =  {
#endif
    "ARMFLY_STM32H743_SPI_W25Q64", /* 算法名，添加算法到 STM32CubeProg 安装目录会显示此名字 */
    NOR_FLASH,                     /* 设备类型 */
    0xC0000000,                    /* Flash 起始地址 */
    8 * 1024 * 1024,               /* Flash 大小，8MB */
    4096,                          /* 编程页大小 */
    0xFF,                          /* 擦除后的数值 */
    2048 , 4 * 1024,               /* 块个数和块大小 */
    0x00000000, 0x00000000,
};
```

注释已经比较详细，大家根据自己的需要做修改即可。注意一点，算法名 ARMFLY_STM32H743_SPI_W25Q64 会反馈到这个地方：



85.5.6第 6 步，编程文件 Loader_Src.c 的实现

下面将变成文件中实现的几个函数为大家做个说明：

◆ **初始化函数 Init**

```
/*
*********************************************************************************************************
*    函 数 名: Init
*    功能说明: Flash 编程初始化
*    形    参: 无
*    返 回 值: 0 表示失败， 1 表示成功
*********************************************************************************************************
*/
int Init(void)
{
    int result = 0;

    /* 系统初始化 */
    SystemInit();

    /* 时钟初始化 */
    result = SystemClock_Config();
    if (result == 1)
    {
        return 0;
    }

    /* SPI Flash 初始化 */
    bsp_InitSPIBus();
    bsp_InitSFlash();

    return 1;
}
```

◆ **整个芯片擦除函数 MassErase**

整个芯片的擦除实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: MassErase
*    功能说明: 整个芯片擦除
*    形    参: 无
*    返 回 值: 1 表示成功，0 表示失败
*********************************************************************************************************
*/
int MassErase(void)
{
    sf_EraseChip();

    return 1;
}
```

◆ **扇区擦除函数 SectorErase**

扇区擦除实现：

```
/*
*********************************************************************************************************
```

```
*    函 数 名: SectorErase
*    功能说明: EraseStartAddress 擦除起始地址
*              EraseEndAddress   擦除结束地址
*    形    参: adr 擦除地址
*    返 回 值: 1 表示成功，0 表示失败
*********************************************************************************************************
*/
int SectorErase (uint32_t EraseStartAddress ,uint32_t EraseEndAddress)
{
    uint32_t BlockAddr;

    EraseStartAddress -= SPI_FLASH_MEM_ADDR;
    EraseEndAddress -= SPI_FLASH_MEM_ADDR;
    EraseStartAddress = EraseStartAddress -  EraseStartAddress % 0x1000; /* 4KB 首地址 */

    while (EraseEndAddress >= EraseStartAddress)
    {
        BlockAddr = EraseStartAddress & 0x0FFFFFFF;

        sf_EraseSector(BlockAddr);

        EraseStartAddress += 0x1000;
    }

    return 1;
}
```

这里要注意两点：

（3） 程序里面的操作 EraseStartAddress-= SPI_FLASH_MEM_ADDR，实际传递进来的地址是带了首地址的，即 0xC0000000。

（4） 这里执行的擦除大小要前面 Dev_Inf.c 文件中配置的扇区大小一致，这里是执行的 4KB 为扇区进行擦除。

◆ 页编程函数 Write

页编程函数实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: Write
*    功能说明: 写数据到 Device
*    形    参: Address 写入地址
*              Size   写入大小，单位字节
*              buffer 要写入的数据地址
*    返 回 值: 1 表示成功，0 表示失败
*********************************************************************************************************
*/
int Write(uint32_t Address, uint32_t Size, uint8_t* buffer)
{
    Address -= SPI_FLASH_MEM_ADDR;

    sf_WriteBuffer(buffer, Address, Size);

    return 1;
}
```

这里注意两点：

（1） W25Q256 的页大小是 256 字节，前面 FlashDev.c 中将页编程大小设置为 4096 字节，主要是方便擦除操作。

（2） 程序里面的操作 Address-= SPI_FLASH_MEM_ADDR，实际传递进来的地址是带了首地址的，即 0xC0000000。

◆ **读取函数**

```
/*
*********************************************************************************************************
*    函 数 名: Read
*    功能说明: 从 SPI Flash 读取数据
*    形    参: Address 读取地址
*              Size   读取大小，单位字节
*              buffer 读取存放的数据缓冲
*    返 回 值: 1 表示成功，0 表示失败
*********************************************************************************************************
*/
int Read(uint32_t Address, uint32_t Size, uint8_t* Buffer)
{
    Address -= SPI_FLASH_MEM_ADDR;
    sf_ReadBuffer(Buffer, Address, Size);

    return 1;
}
```

◆ **读取和校验函数**

我们程序中未做校验函数。如果程序中未做校验函数，那么 STM32CubeProg 会读取数据做校验。

85.5.7第 7 步，修改 SPI Flash 驱动文件（引脚，命令等）

最后一步就是 SPI Flash（W25Q64）的驱动修改，大家可以根据自己的需求做修改。使用的引脚定义在文件 bsp_spi_bus.c：

```
/*
*********************************************************************************************************
*                                    时钟，引脚，DMA，中断等宏定义
*********************************************************************************************************
*/
#define SPIx                             SPI1
#define SPIx_CLK_ENABLE()                __HAL_RCC_SPI1_CLK_ENABLE()
#define DMAx_CLK_ENABLE()                __HAL_RCC_DMA2_CLK_ENABLE()

#define SPIx_FORCE_RESET()               __HAL_RCC_SPI1_FORCE_RESET()
#define SPIx_RELEASE_RESET()             __HAL_RCC_SPI1_RELEASE_RESET()

#define SPIx_SCK_CLK_ENABLE()            __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_SCK_GPIO                    GPIOB
#define SPIx_SCK_PIN                     GPIO_PIN_3
#define SPIx_SCK_AF                          GPIO_AF5_SPI1

#define SPIx_MISO_CLK_ENABLE()           __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MISO_GPIO                   GPIOB
#define SPIx_MISO_PIN                    GPIO_PIN_4
#define SPIx_MISO_AF                     GPIO_AF5_SPI1
```

```
#define SPIx_MOSI_CLK_ENABLE()              __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MOSI_GPIO                      GPIOB
#define SPIx_MOSI_PIN                       GPIO_PIN_5
#define SPIx_MOSI_AF                        GPIO_AF5_SPI1
```

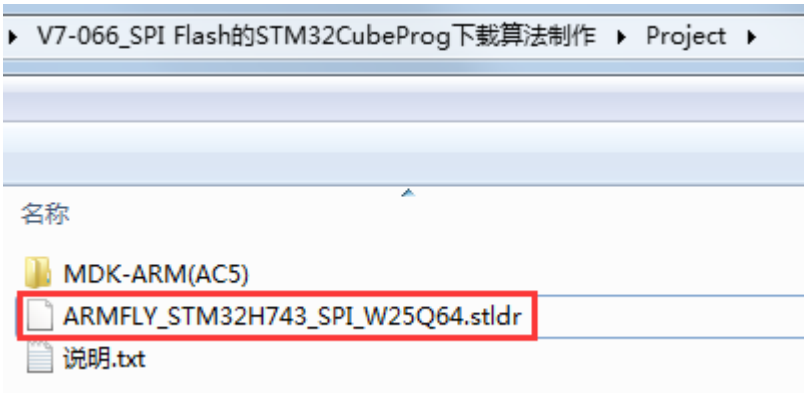
硬件设置了之后，剩下就是 SPI Flash 相关的几个配置和片选引脚配置，在文件 bsp_spi_flash.c：

主要是下面这几个：

```
/* 串行 Flash 的片选 GPIO 端口， PD13 */
#define SF_CS_CLK_ENABLE()              __HAL_RCC_GPIOD_CLK_ENABLE()
#define SF_CS_GPIO                      GPIOD
#define SF_CS_PIN                       GPIO_PIN_13

#define SF_CS_0()                       SF_CS_GPIO->BSRR = ((uint32_t)SF_CS_PIN << 16U)
#define SF_CS_1()                       SF_CS_GPIO->BSRR = SF_CS_PIN

#define CMD_AAI       0xAD      /* AAI 连续编程指令(FOR SST25VF016B) */
#define CMD_DISWR     0x04      /* 禁止写, 退出 AAI 状态 */
#define CMD_EWRSR     0x50      /* 允许写状态寄存器的命令 */
#define CMD_WRSR      0x01      /* 写状态寄存器命令 */
#define CMD_WREN      0x06      /* 写使能命令 */
#define CMD_READ      0x03      /* 读数据区命令 */
#define CMD_RDSR      0x05      /* 读状态寄存器命令 */
#define CMD_RDID      0x9F      /* 读器件 ID 命令 */
#define CMD_SE        0x20      /* 擦除扇区命令 */
#define CMD_BE        0xC7      /* 批量擦除命令 */
#define DUMMY_BYTE    0xA5      /* 哑命令，可以为任意值，用于读操作 */

#define WIP_FLAG      0x01      /* 状态寄存器中的正在编程标志（WIP） */
```

85.6 QSPI Flash 的 STM32CubeProg 下载算法使用方法

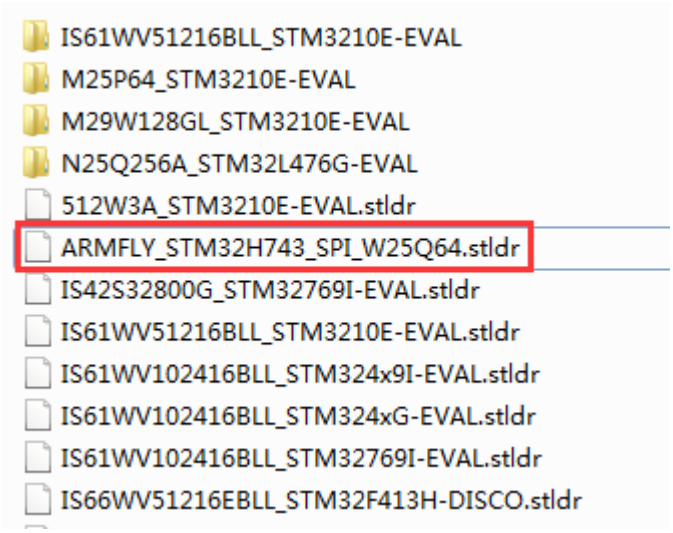
编译本章教程配套的例子，生成的算法文件位于此路径下：



85.6.1 下载算法存放位置

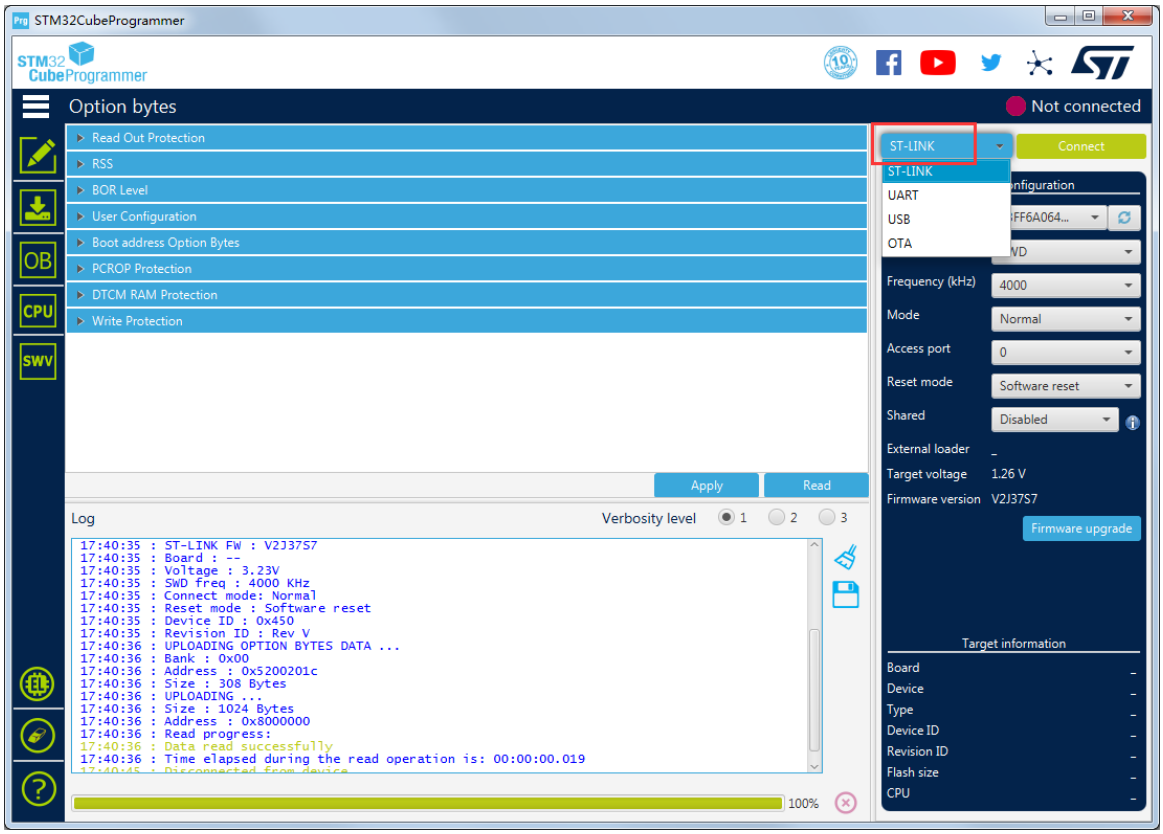
生成此文件后，需要大家将其存放到 STM32CubeProg 安装目录路径：

\STMicroelectronics\STM32Cube\STM32CubeProgrammer\bin\ExternalLoader

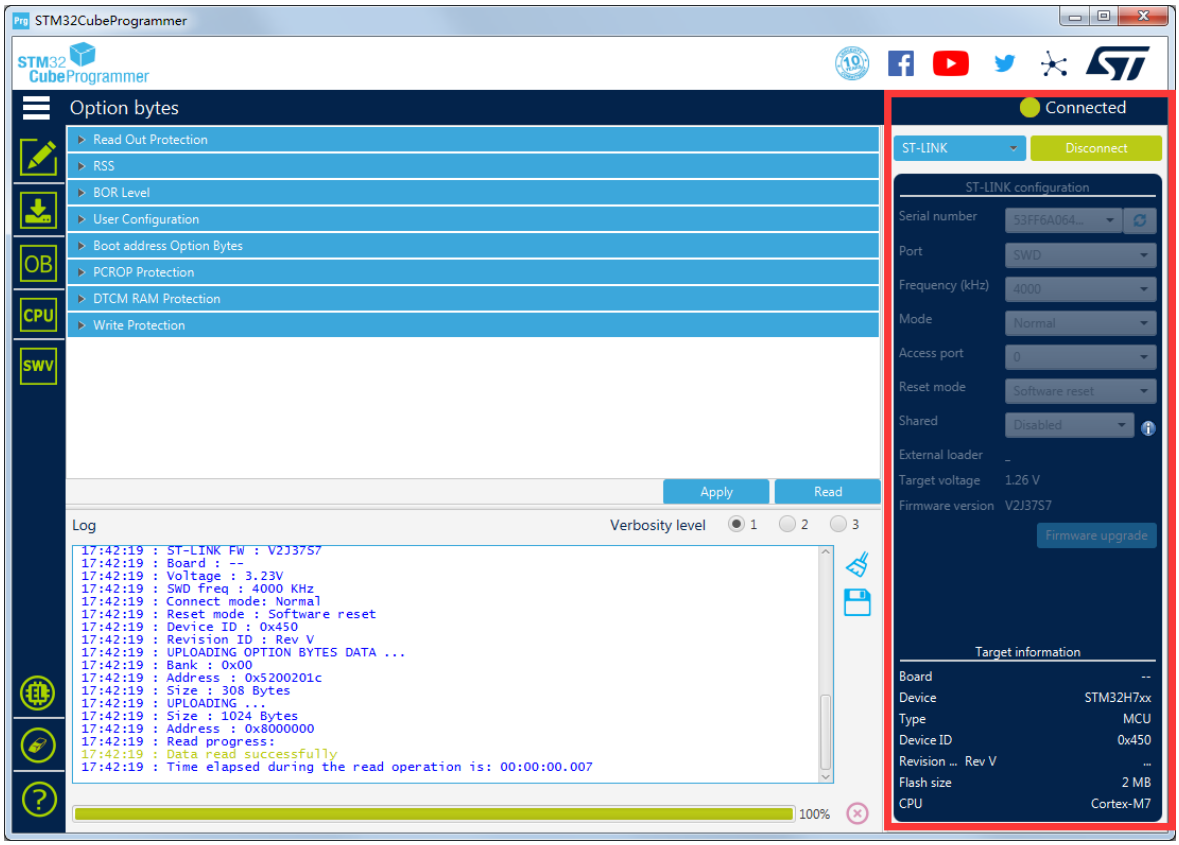


85.6.2 STM32CubeProg 下载配置

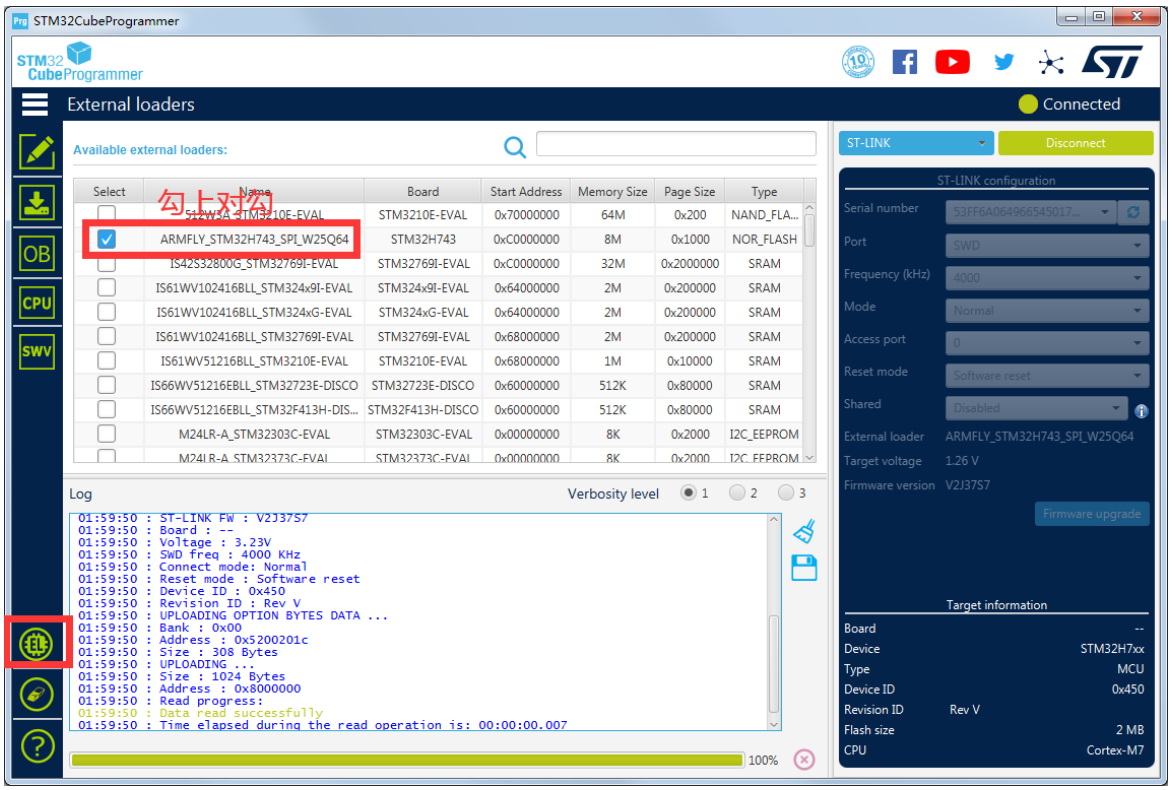
我们这里以 STLINK 连接开发板为例进行说明（USB DFU 或者串口方式不支持下载外部 Flash）。



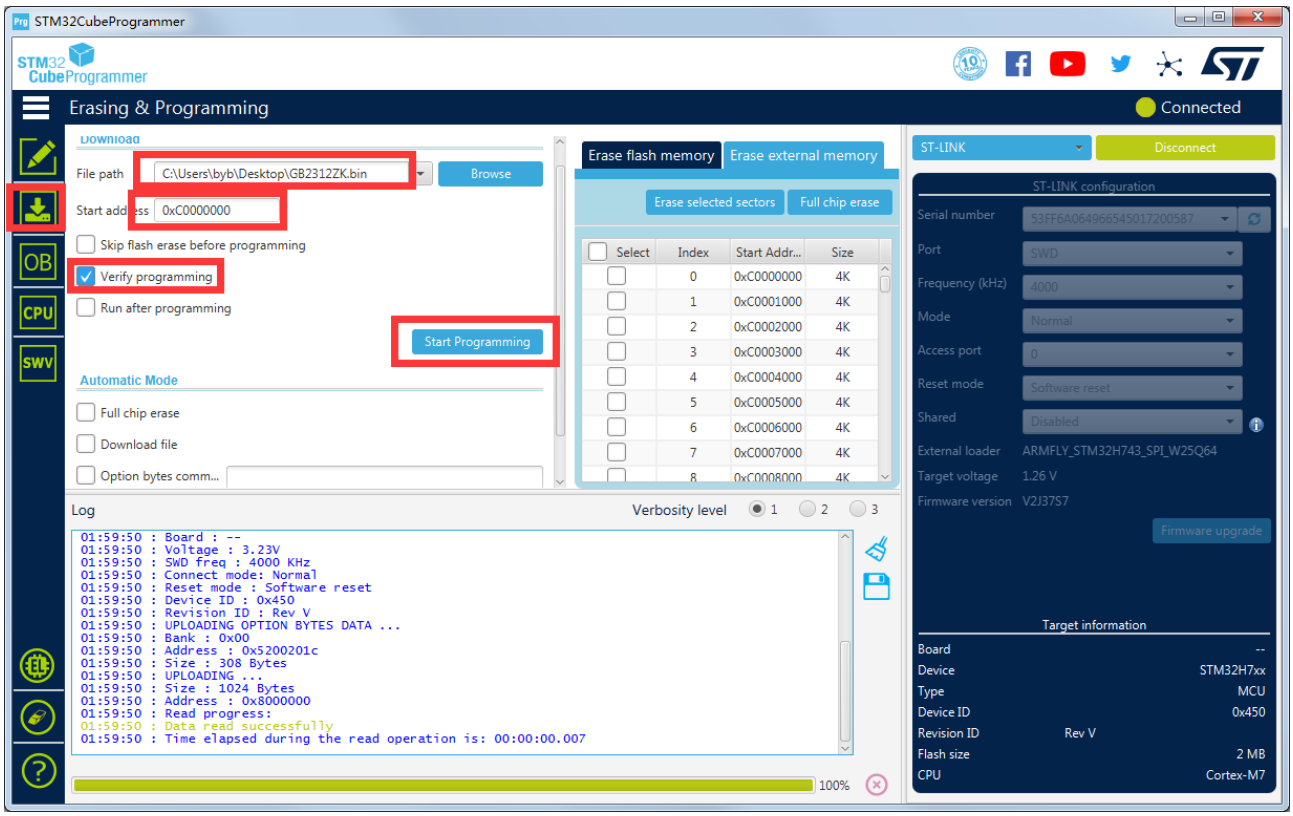
点击 Connect 后效果如下：



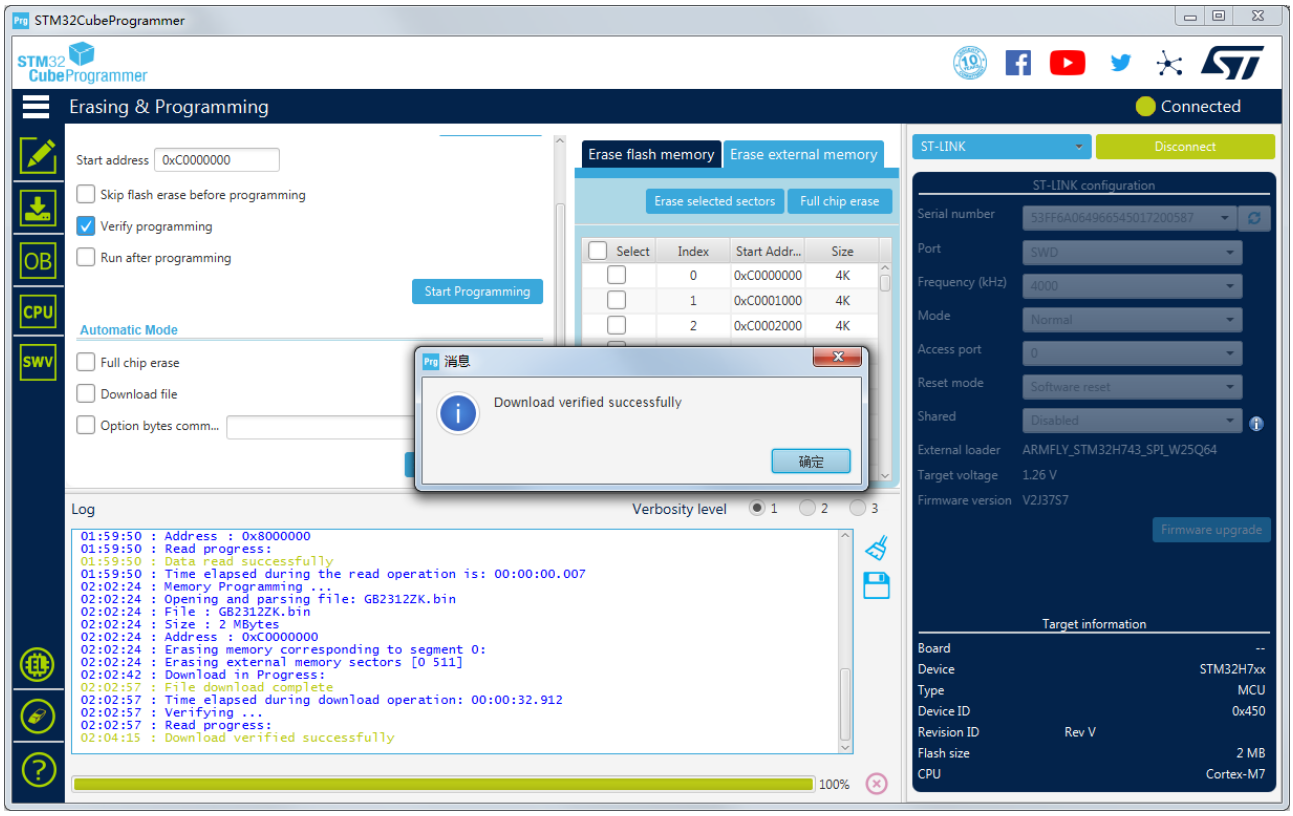
在这里选择我们制作的下载算法：



任意加载个 hex 或者 bin 文件，并按照如下配置，然后点击 Start Programming

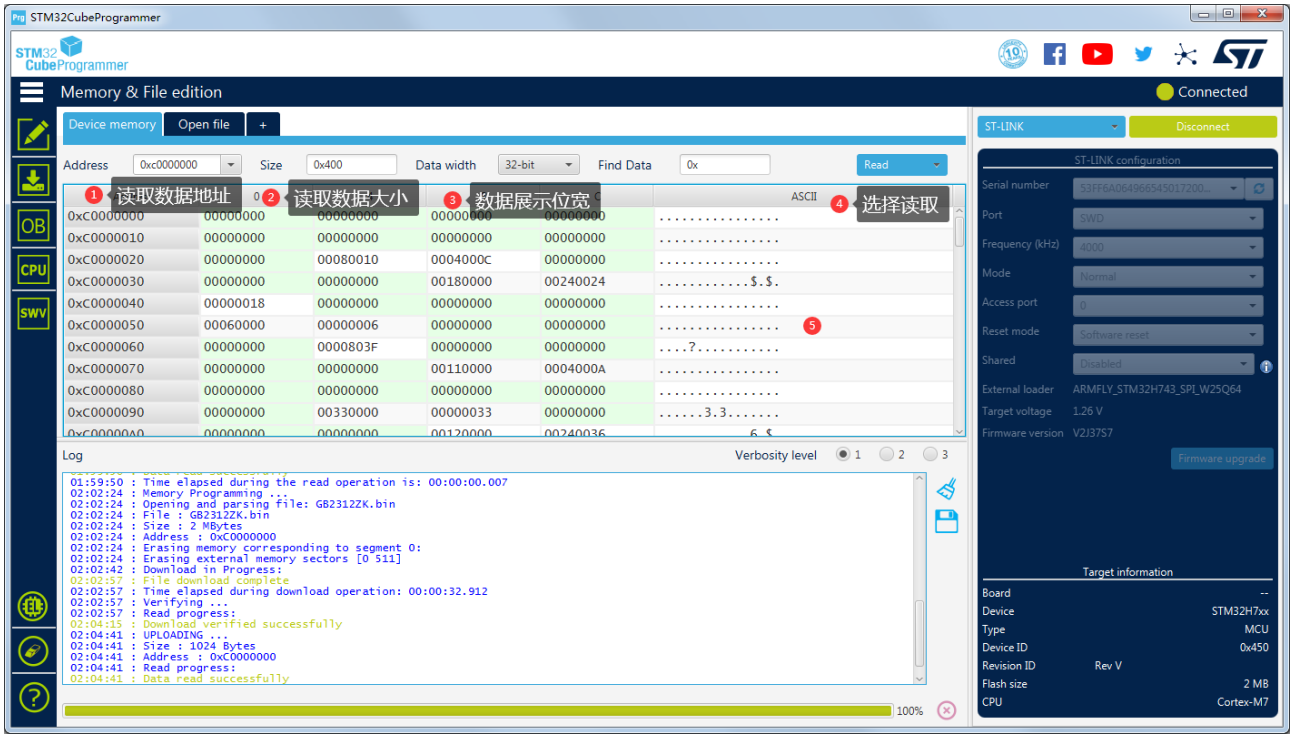

STM32CubeProgrammer
Erasing & Programming
Connected
File path
C:\Users\byb\Desktop\GB2312ZK.bin
Browse
Start address
0xC0000000
Skip flash erase before programming
Verify programming
Run after programming
Start Programming
Automatic Mode
Full chip erase
Download file
Option bytes comm...
Erase flash memory
Erase external memory
Erase selected sectors
Full chip erase
Select
Index
Start Addr...
Size
ST-LINK
Disconnect
ST-LINK configuration
Serial number
Port
Frequency (kHz)
Mode
Access port
Reset mode
Shared
External loader
ARMFLY_STM32H743_SPI_W25Q64
Target voltage
1.26 V
Firmware version
V2J37S7
Firmware upgrade
Target information
Board
Device
STM32H7xx
Type
MCU
Device ID
0x450
Revision ID
Rev V
Flash size
2 MB
CPU
Cortex-M7
Log
Verbosity level
01:59:50 : Board : --
01:59:50 : Voltage : 3.23V
01:59:50 : SWD freq : 4000 KHz
01:59:50 : Connect mode: Normal
01:59:50 : Reset mode : Software reset
01:59:50 : Device ID : 0x450
01:59:50 : Revision ID : Rev V
01:59:50 : UPLOADING OPTION BYTES DATA ...
01:59:50 : Bank : 0x00
01:59:50 : Address : 0x5200201c
01:59:50 : Size : 308 Bytes
01:59:50 : UPLOADING ...
01:59:50 : Size : 1024 Bytes
01:59:50 : Address : 0x8000000
01:59:50 : Read progress:
01:59:50 : Data read successfully
01:59:50 : Time elapsed during the read operation is: 00:00:00.007
100%

下载完成后的效果如下：


STM32CubeProgrammer
Erasing & Programming
Connected
Start address
0xC0000000
Skip flash erase before programming
Verify programming
Run after programming
Start Programming
Automatic Mode
Full chip erase
Download file
Option bytes comm...
Erase flash memory
Erase external memory
Erase selected sectors
Full chip erase
Select
Index
Start Addr...
Size
消息
Download verified successfully
确定
ST-LINK
Disconnect
ST-LINK configuration
Serial number
Port
Frequency (kHz)
Mode
Access port
Reset mode
Shared
External loader
ARMFLY_STM32H743_SPI_W25Q64
Target voltage
1.26 V
Firmware version
V2J37S7
Firmware upgrade
Target information
Board
Device
STM32H7xx
Type
MCU
Device ID
0x450
Revision ID
Rev V
Flash size
2 MB
CPU
Cortex-M7
Log
Verbosity level
01:59:50 : Address : 0x8000000
01:59:50 : Read progress:
01:59:50 : Data read successfully
01:59:50 : Time elapsed during the read operation is: 00:00:00.007
02:02:24 : Memory Programming ...
02:02:24 : Opening and parsing file: GB2312ZK.bin
02:02:24 : File : GB2312ZK.bin
02:02:24 : Size : 2 MBytes
02:02:24 : Address : 0xC0000000
02:02:24 : Erasing memory corresponding to segment 0:
02:02:24 : Erasing external memory sectors [0 511]
02:02:42 : Download in Progress:
02:02:57 : File download complete
02:02:57 : Time elapsed during download operation: 00:00:32.912
02:02:57 : Verifying ...
02:02:57 : Read progress:
02:04:15 : Download verified successfully
100%

85.6.3 验证算法文件是否可以正常使用

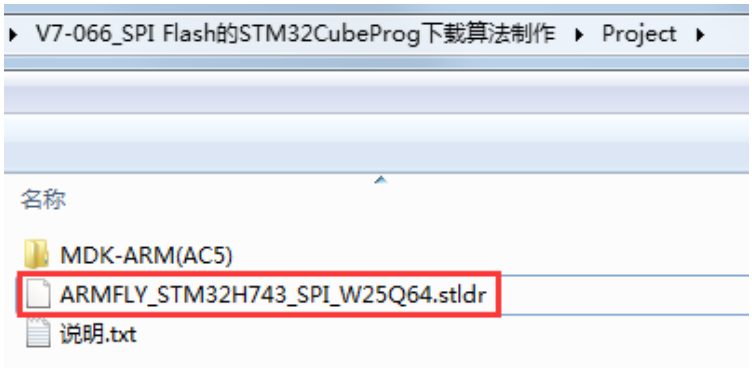
为了验证算法文件是否可以正常使用，大家可以运行本教程第 86 章配套的例子。也可以使用 STM32CubeProg 直接读取：



85.7 实验例程说明

本章配套例子：V7-066_SPI Flash 的 STM32CubeProg 下载算法制作。

编译本章教程配套的例子，生成的算法文件位于此路径下：



85.8 总结

本章节就为大家讲解这么多，为了熟练掌握，大家可以尝试自己实现一个 Flash 下载算法。

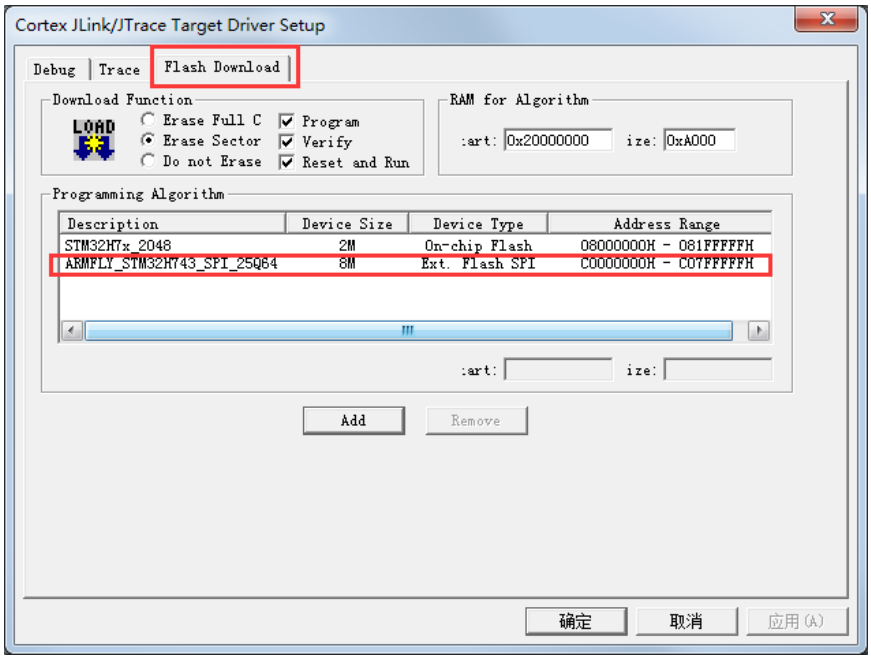
第86章 STM32H7 的内部 Flash 和 SPI Flash 同时使用 MDK 一键下载

本章节为大家讲解内部 Flash 和外部 SPI Flash 同时一键下载的实现。

86.1 初学者重要提示
86.2 内部 Flash 和 SPI Flash 同时一键下载的优势
86.3 下载算法存储存放位置
86.4 下载全字库到外部 SPI Flash 说明
86.5 内部 Flash 和 SPI Flash 程序下载配置
86.6 实验例程说明
86.7 总结

86.1 初学者重要提示

- SPI Flash 的相关知识点可以看第 78 章和 79 章。
- 操作本章配套例子前务必先将 SPI Flash 的下载算法放到 MDK 安装目录。
- 本章节配套例子实现了将 12 点阵，16 点阵，24 点阵和 32 点阵中文全字库下载到外置 SPI Flash。
- 全字库文件仅需下载一次即可，下载完毕后，大家可以删除工程中添加的算法文件，从这里删除即可：

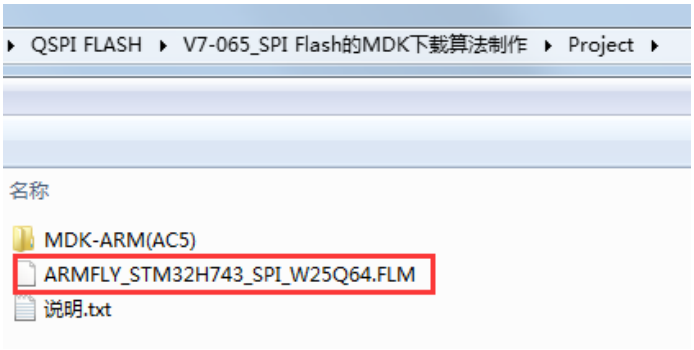


86.2 内部 Flash 和 SPI Flash 同时一键下载的优势

通过这种方式，可以方便的更新 SPI Flash 中的数据，特别是全字库，图库，主题等资源文件的下载。这样就可以通过 MDK 进行一键下载。

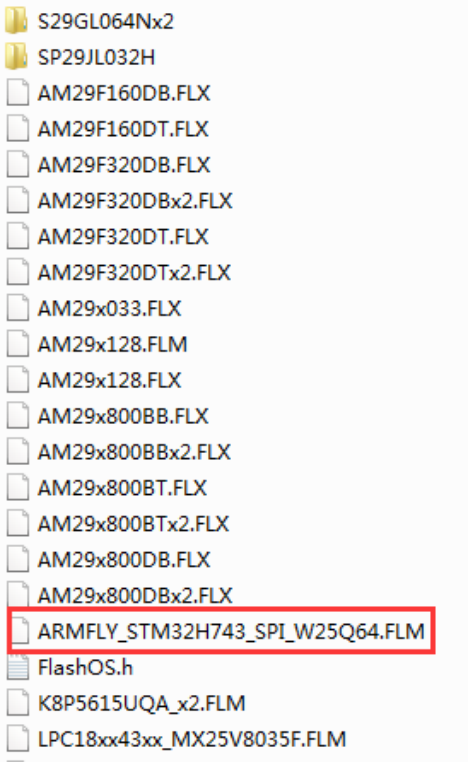
86.3 下载算法存放位置

编译例子：V7-065_SPI Flash 的 MDK 下载算法制作，生成的算法文件位于此路径下：



生成算法文件后，需要大家将其存到到 MDK 安装目录，有两个位置可以存放，任选其一，推荐第 2 种：

- 第 1 种：存放到 MDK 的 STM32H7 软包安装目录里面：\Keil\STM32H7xx_DFP\2.6.0\CMSIS\Flash（软包版本不同，数值 2.6.0 不同）。
- 第 2 种：MDK 的安装目录 \ARM\Flash 里面。

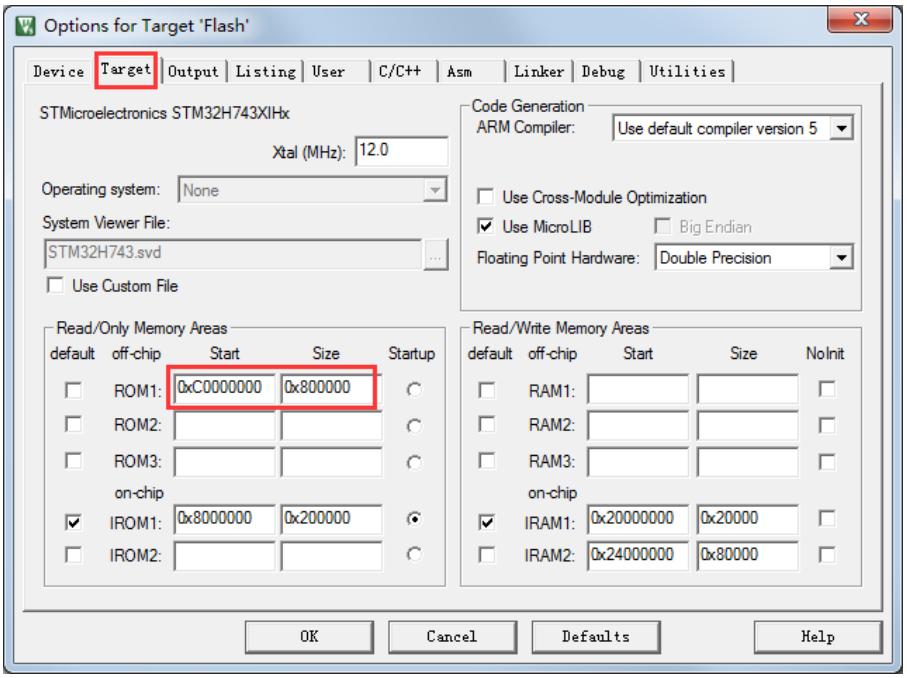


86.4 下载全字库到外部 SPI Flash 说明

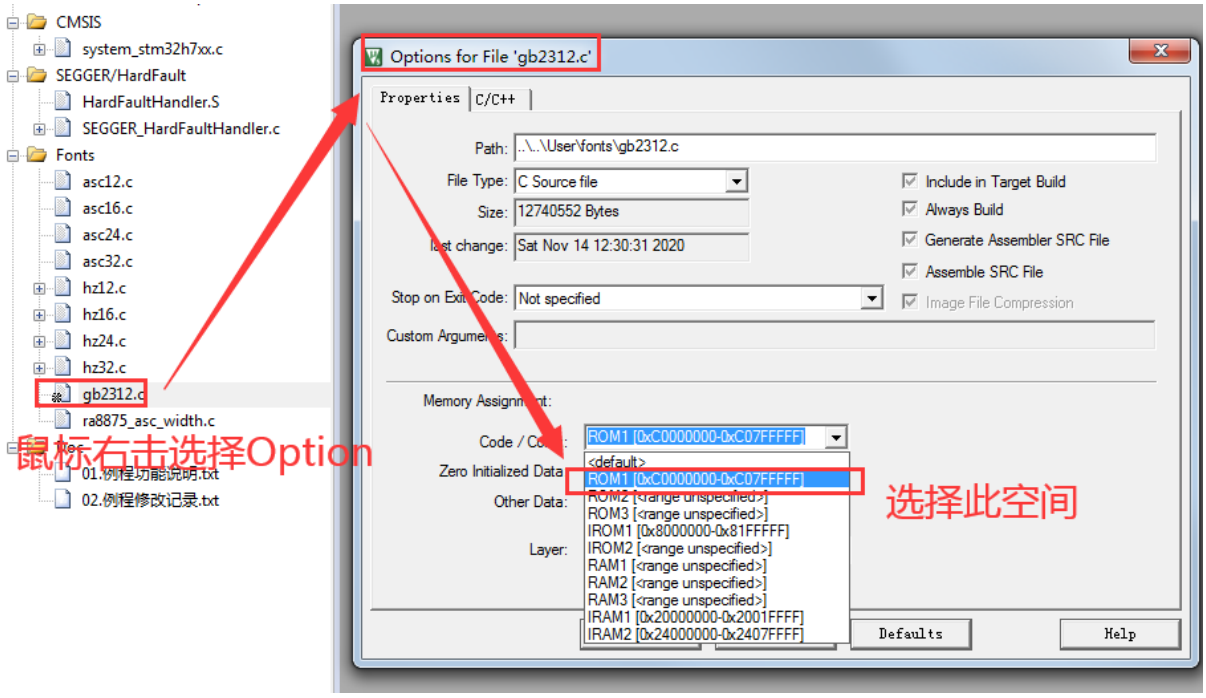
全字库的下载注意以下两点即可。

86.4.1 设置全字库文件到外置 SPI Flash

增加地址空间：

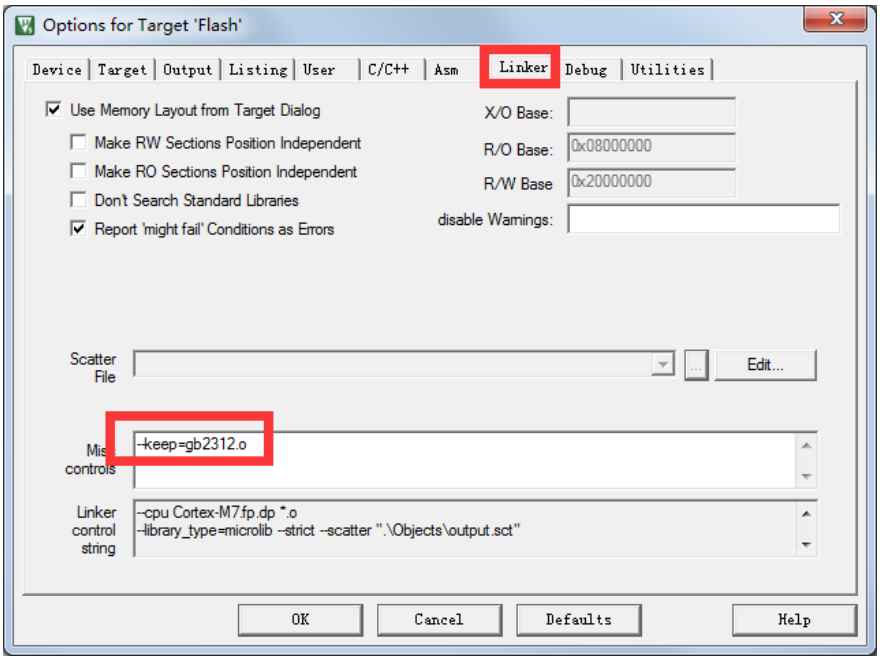


设置文件 gb3212 到外部 SPI Flash，此文件记录了 12 点阵，16 点阵，24 点阵和 32 点阵字体。



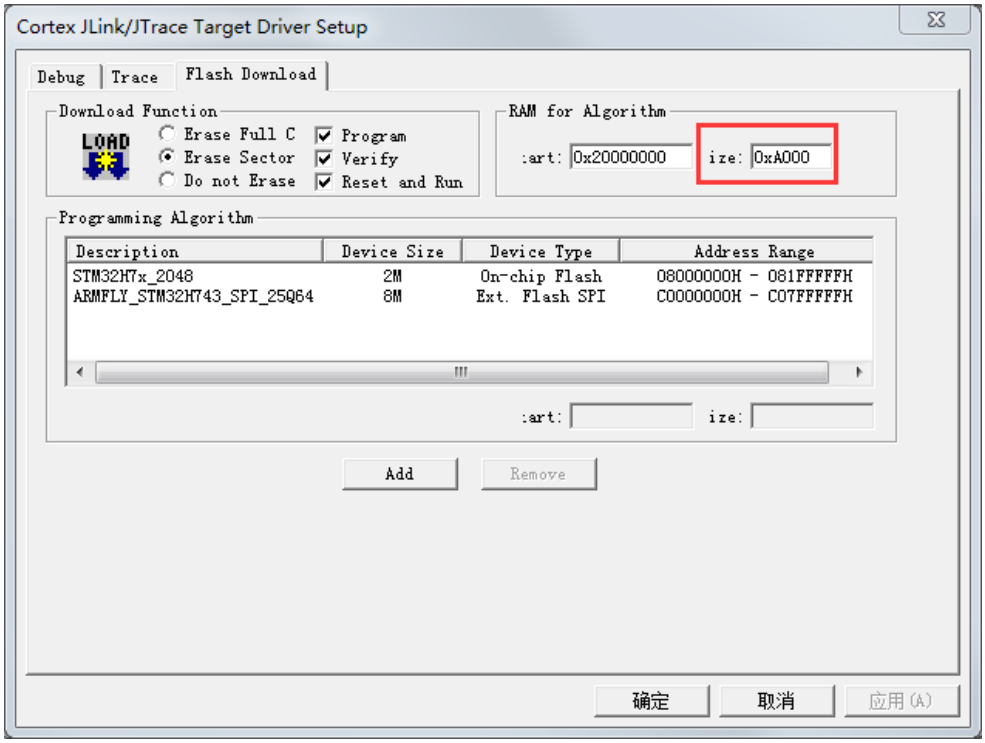
86.4.2 防止编译优化全字库文件

为了防止编译将文件 gb2312 优化掉（因为我们没有调用此文件），需要大家在 Option->Linker 选项里面设置保留此文件：



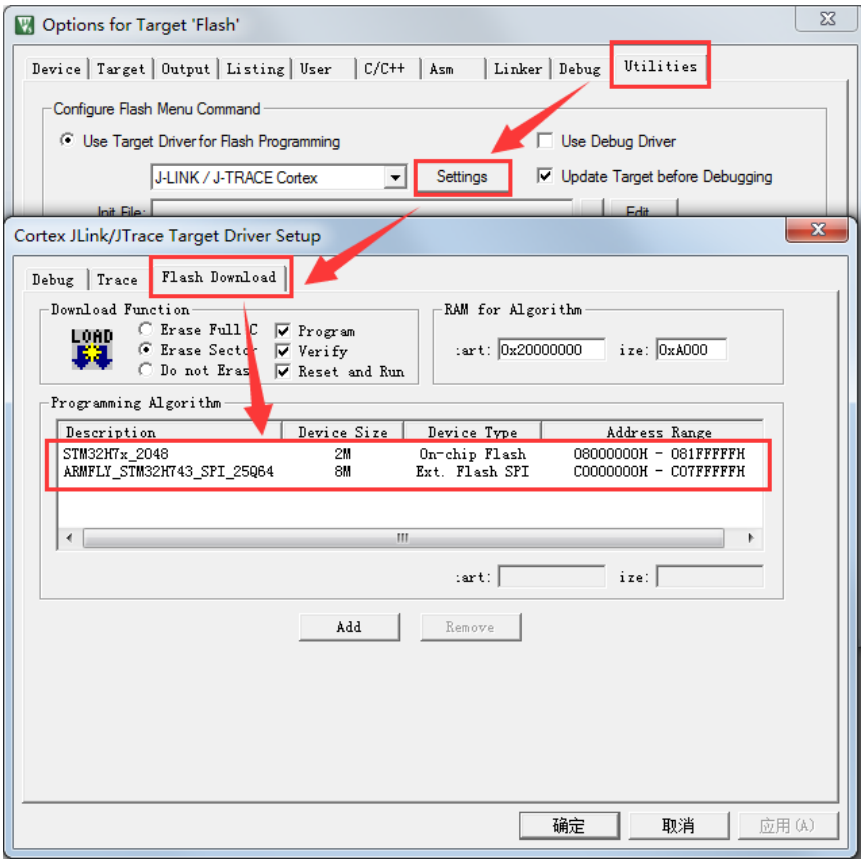
86.5 内部 Flash 和 QSPI Flash 程序下载配置

配置方法如下，注意这里一定要够大，否则会提示算法文件无法加载：



我们这里是将其加到 DTCM 中，即首地址为 0x20000000，大家也可以存储到任意其它 RAM 地址，只要空间还够加载算法文件即可。推荐使用 AXI SRAM（地址 0x24000000），因为这块 RAM 空间足够大。

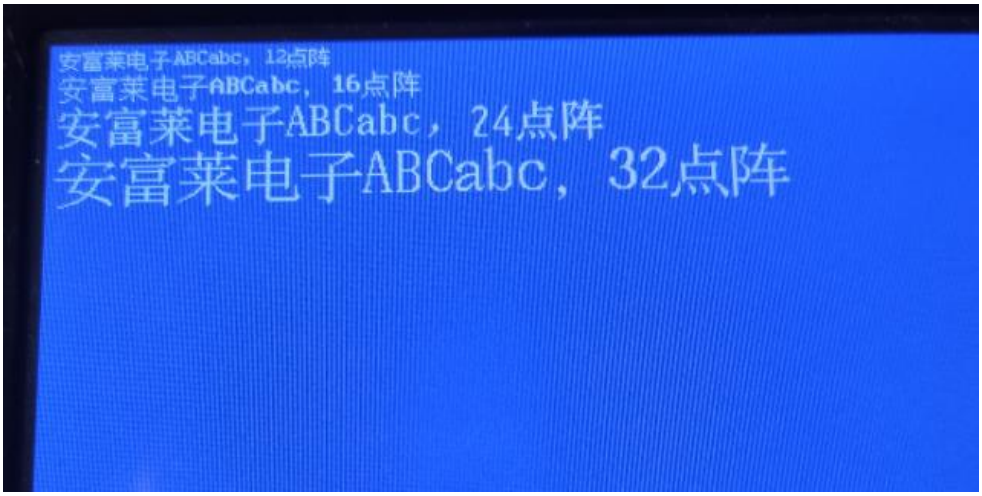
如果要下载程序到内部 Flash 和外部 SPI Flash 里面，需要做如下配置，两个下载算法都要添加进来：



86.6 实验例程说明

本章节配套例子：V7-067_MDK 一键下载内部 Flash 和外部 SPI Flash（下载的全字库文件支持 12,16,24 和 32 点阵）

效果如下：



86.7 总结

本章节就为大家讲解这么，为了熟练掌握，大家可以尝试自己实现外部 SPI Flash 数据更新。

第87章 STM32H7 的 SDMMC 总线基础知识和 HAL 库 API

本章节为大家讲解 SDMMC（Secure digital input/output MultiMediaCard interface）总线的基础知识和对应的 HAL 库 API。

87.1 初学者重要提示
87.2 SDMMC 总线基础知识
87.3 SDMMC 总线的 HAL 库用法
87.4 SDMMC 总线源文件 stm32h7xx_hal_sd.c
87.5 总结

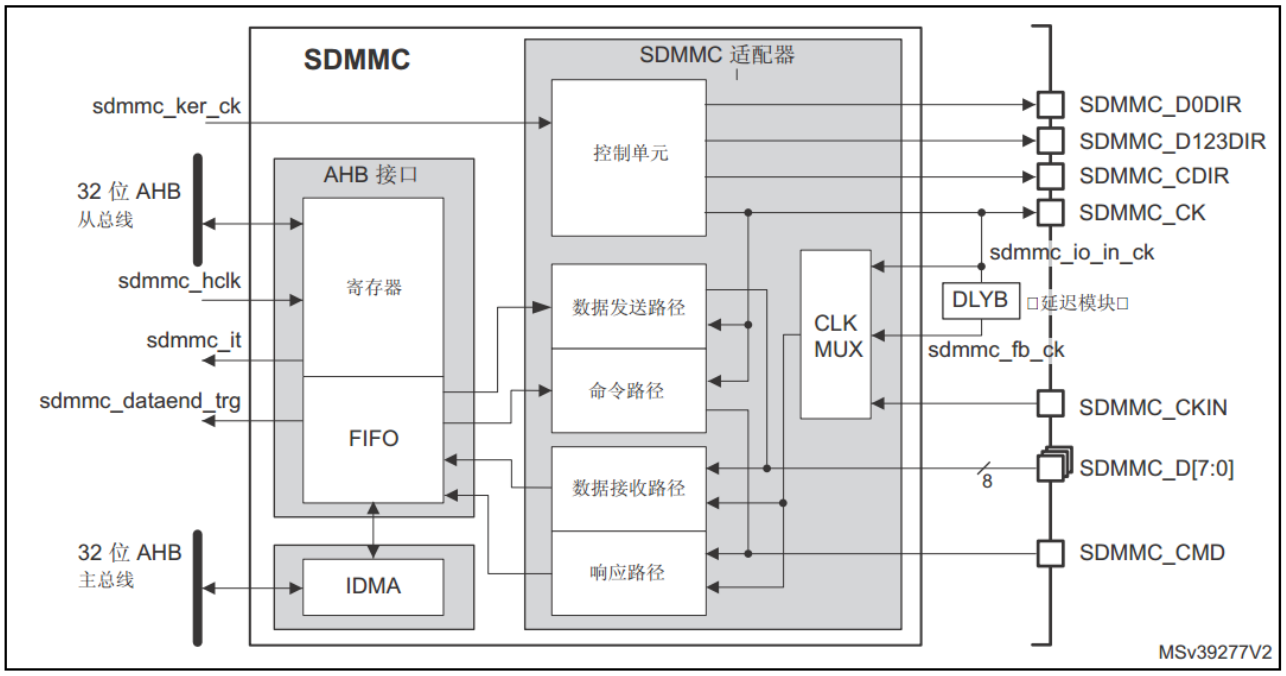
87.1 初学者重要提示

- 对于 SDMMC 控制 SD 卡或者 eMMC，掌握本章的知识点就够用了，更深入的认识可以看 STM32H7 的参考手册。
- 注意，操作 SD 卡是采用的函数 HAL_SD_XXXX，而操作 eMMC 是采用的函数 HAL_MMC_XXXX，也就是说他们采用的函数前缀是不同的。
- SD 卡官网： www.sdcard.org 。
- SDMMC 驱动 eMMC 支持 1 线，4 线和 8 线模式，其中 8 线模式的最高速度可达 208MB/S，实际速度受 IO 最大速度限制。
- SDMMC 驱动 SD 卡支持 1 线和 4 线模式。
- STM32H7 的 SDMMC 也支持 eMMC：
  - 【普及贴】各个厂家 eMMC 读写速度，镁光，东芝，三星，ISSI 和旺宏
    http://www.armbbs.cn/forum.php?mod=viewthread&tid=95954
  - H7 的 8 线 SDIO DMA 驱动 eMMC 的裸机性能，读 43MB/S，写 18.8MB/S
    http://www.armbbs.cn/forum.php?mod=viewthread&tid=95953

87.2 SDMMC 总线基础知识

87.2.1 SDMMC 总线的硬件框图

认识一个外设，最好的方式就是看它的框图，方便我们快速的了解 SDMMC 的基本功能，然后再看手册了解细节。



通过这个框图，我们可以得到如下信息：

- sdmmc_ker_ck 输入

  SDMMC 内核时钟。

- sdmmc_hclk 输入

  AHB 时钟。

- sdmmc_it 输出

  SDMMC 全局中断。

- sdmmc_dataend_trg 输出

  MDMA 的 SDMMC 数据接收触发信号。

- SDMMC_CMD

  SD/SDIO/MMC 卡双向/响应信号。

- SDMMC_D[7:0]

  SD/SDIO/MMC 卡双向数据线。

- SDMMC_CKIN

  来自 SD/SDIO/MMC 卡的外部驱动器的时钟反馈（用于 SDR12，SDR25，SDR50 和 DDR50）。

- SDMMC_CK

  SD/SDIO/MMC 卡的时钟。

- SDMMC_CDIR

  SDMMC_CMD 信号的 SD/SDIO/MMC 卡 I/O 方向指示。

- SDMMC_D123DIR

  SDMMC_D[3:1]数据线的 SD/SDIO/MMC 卡 I/O 方向指示。

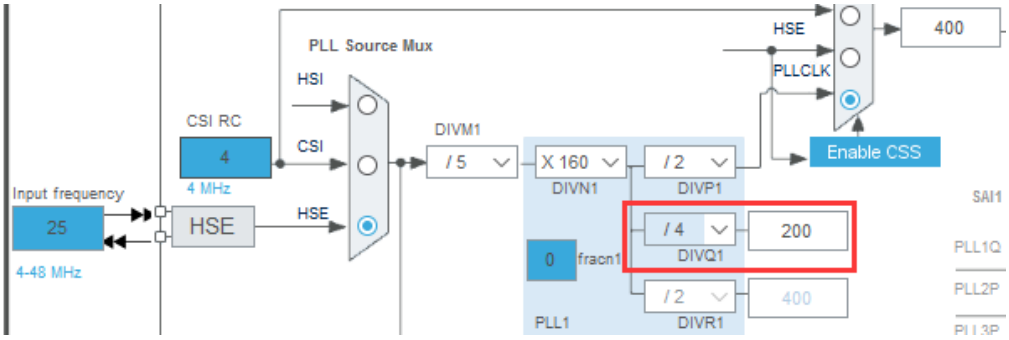
- SDMMC_D0DIR

  SDMMC_D0 数据线的 SD/SDIO/MMC 卡 I/O 方向指示。

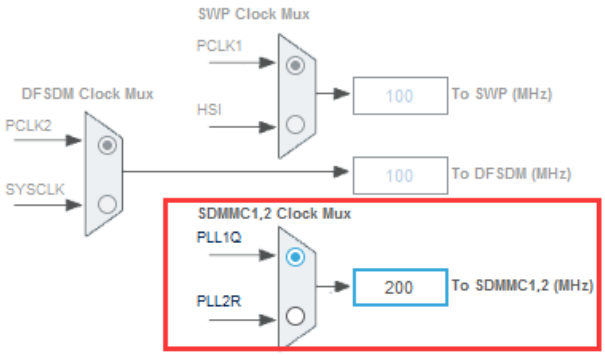
STM32H7 有两个 SDMMC 控制器，SDMMC1 和 SDMMC2，这两个控制器支持的功能是一样的。

87.2.2 SDMMC 时钟

SDMMC 控制器的时钟来源：



SDMMC1 和 SDMMC2 时钟源是一样的：



87.2.3 SDMMC1 和 SDMMC2 支持的 RAM 空间区别

注：大家应用时要特别注意这个问题。

使用 STM32H7 的 SDIO1 仅支持 AXI SRAM，而 SDIO2 是 AXI，SRAM1,SRAM2 和 SRAM3 都支持的

Table 2. Bus-master-to-bus-slave interconnect

Bus slave / type(1)	Bus master / type(1)																	
	Cortex-M7 - AXIM	Cortex-M7 - AHBP	Cortex-M7 - ITCM	Cortex-M7 -DTCM	SDMMC1	MDMA - AXI	MDMA - AHBS	DMA2D	LTDC	DMA1 - MEM	DMA1 - PERIPH	DMA2 - MEM	DMA2 - PERIPH	Eth. MAC - AHB	SDMMC2 - AHB	USBHS1 - AHB	USBHS2 - AHB	BDMA - AHB
	Interconnect path and type(2)																	
ITCM	-	-	x	-	-	-	x	-	-	-	-	-	-	-	-	-	-	-
DTCM	-	-	-	x	-	-	x	-	-	-	-	-	-	-	-	-	-	-
AHB3 peripherals	x	-	-	-	-	x	-	-	-	x	x	x	x	x	x	x	x	-
APB3 peripherals	x	-	-	-	-	x	-	-	-	x	x	x	x	x	x	x	x	-
Flash bank 1	x	-	-	-	x	x	-	x	x	x	x	x	x	x	x	x	x	-
Flash bank 2(3)	x	-	-	-	x	x	-	x	x	x	x	x	x	x	x	x	x	-
AXI SRAM	x	-	-	-	x	x	-	x	x	x	x	x	x	x	x	x	x	-
QUADSPI	x	-	-	-	x	x	-	x	x	x	x	x	x	x	x	x	x	-
FMC	x	-	-	-	x	x	-	x	x	x	x	x	x	x	x	x	x	-
SRAM 1	x	-	-	-	-	x	-	x	-	x	x	x	x	x	x	x	x	-
SRAM 2	x	-	-	-	-	x	-	x	-	x	x	x	x	x	x	x	x	-
SRAM 3	x	-	-	-	-	x	-	x	-	x	x	x	x	x	x	x	x	-

87.2.4SDMMC 支持的速度

驱动 SD 卡支持的最大总线速度：

SDIO 总线速度模式(1)(2)	最大总线速度 (3) [MB/s]	最大时钟频率 [MHz]	信号电压 [V]
DS（默认速度）	12.5	25	3.3
HS（高速）	25	50	3.3
SDR12	12.5	25	1.8
SDR25	25	50	1.8
DDR50	50	50	1.8
SDR50	50	100	1.8
SDR104	104	208(4)	1.8

1. SDR 单倍数据速率信号传输。
2. DDR 双倍数据速率信号传输。（在 SDMMC_CK 的两种时钟边沿上均采样数据。）
3. SDIO 总线速度，4 位总线宽度。
4. 最大频率取决于允许的最大 I/O 速度。

驱动 eMMC 支持的最大总线速度：

eMMC 总线速度模式 (1)(2)	最大总线速度 (3) [MB/s]	最大时钟频率 [MHz]	信号电压 [V]
传统兼容型	26	26	3/1.8/1.2V
高速 SDR	52	52	3/1.8/1.2V
高速 DDR	104	52	3/1.8/1.2V
高速 HS200	200	200(4)	1.8/1.2V

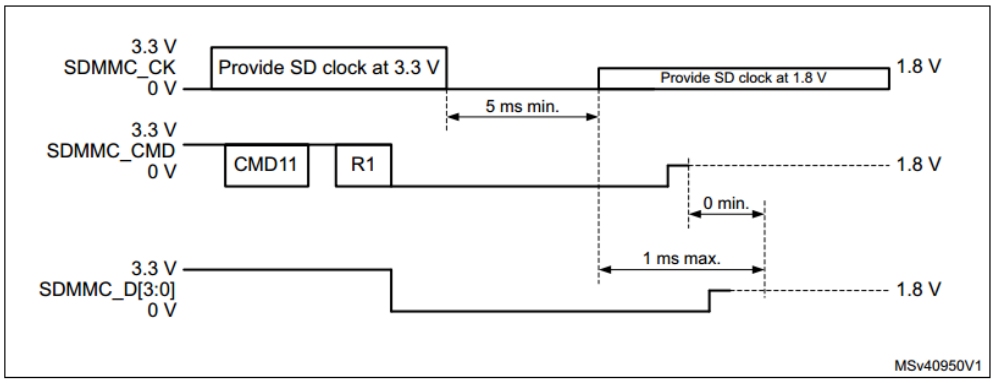
1. SDR 单倍数据速率信号传输。
2. DDR 双倍数据速率信号传输。（在 SDMMC_CK 的两种时钟边沿上均采样数据。）
3. eMMC 总线速度，8 位总线宽度。
4. 最大频率取决于允许的最大 I/O 速度。

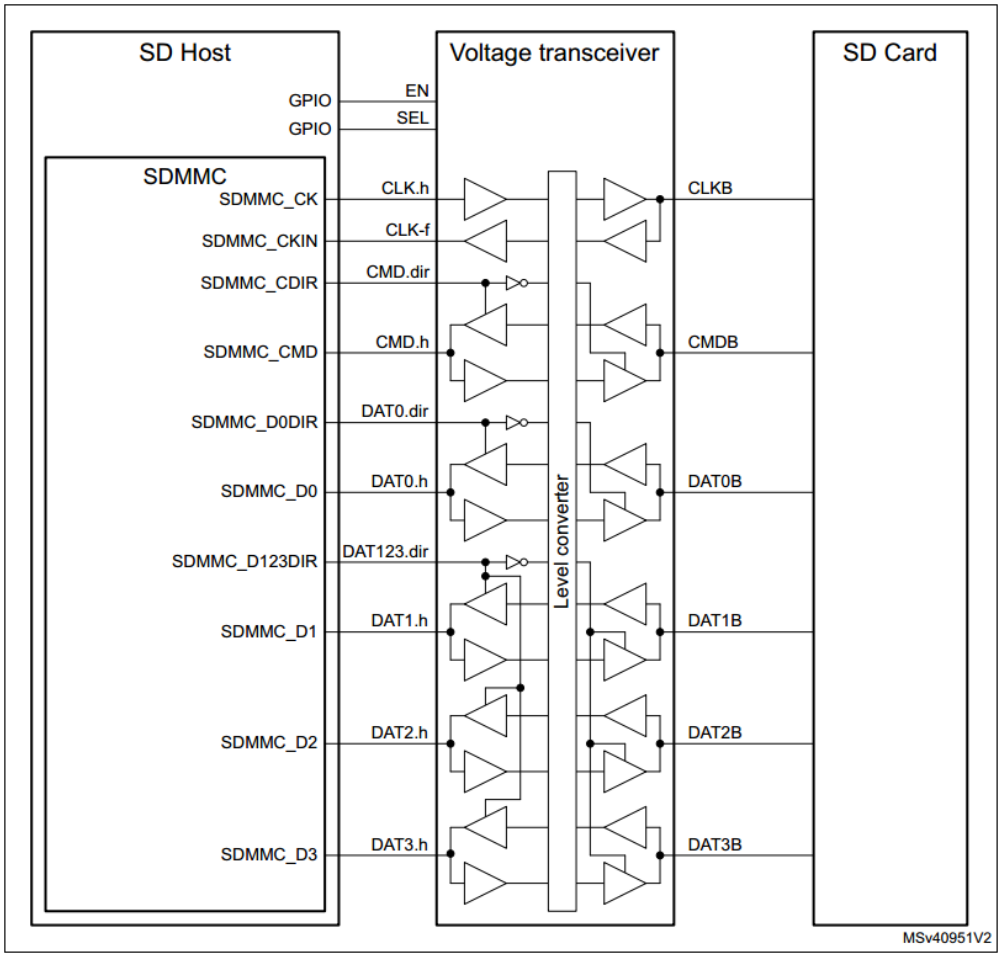
关于这两个数据表，注意以下几点：

- 驱动 SD 卡最大支持 4bit，驱动 eMMC 最大支持 8bit。
- 针对信号电压 1.8V 或者 1.2V，STM32H7 需要外接专门的 PHY 芯片才可以驱动。
- 最大 IO 翻转限制说的是 SDR50，SDR104 这种高速通信。平时用的 DS，HS 这种，无压力，刷满速不成问题。

87.2.5 SDMMC 支持 UHS-I 模式

STM32H7 的 SDIO 外接支持 UHS-I 模式 (SDR12, SDR25, SDR50, SDR104 和 DDR50)需要 1.8 的电平转换器。STM32H7 参考手册给了一个型号 ST6G3244ME：





87.2.6 SDMMC 自带的 DMA 控制器 IDMA

STM32H7 的 SDMMC 自带了专用的 DMA 控制器 IDMA，支持突发，也支持双缓冲。为什么要自带 DMA 控制器？ 主要原因是 STM32H7 的通用 DMA1 和 DMA2 已经无法满足 SDMMC 高速通信速度。在本教程的第 62 章专门为大家测试过。通过让 SDMMC 自带控制器，这个问题就迎刃而解。

87.3 SDMMC 总线的 HAL 库用法

87.3.1 SDMMC 总线结构体 SD_TypeDef

SDMMC 总线相关的寄存器是通过 HAL 库中的结构体 SD_TypeDef 定义，在 stm32h743xx.h 中可以找到这个类型定义：

```
#define SD_TypeDef          SDMMC_TypeDef
typedef struct
{
  __IO uint32_t POWER;          /*!< SDMMC power control register,          Address offset: 0x00  */
  __IO uint32_t CLKCR;          /*!< SDMMC clock control register,          Address offset: 0x04  */
  __IO uint32_t ARG;            /*!< SDMMC argument register,               Address offset: 0x08  */
```

```
  __IO uint32_t CMD;            /*!< SDMMC command register,                  Address offset: 0x0C  */
  __I uint32_t  RESPCMD;        /*!< SDMMC command response register,         Address offset: 0x10  */
  __I uint32_t  RESP1;          /*!< SDMMC response 1 register,               Address offset: 0x14  */
  __I uint32_t  RESP2;          /*!< SDMMC response 2 register,               Address offset: 0x18  */
  __I uint32_t  RESP3;          /*!< SDMMC response 3 register,               Address offset: 0x1C  */
  __I uint32_t  RESP4;          /*!< SDMMC response 4 register,               Address offset: 0x20  */
  __IO uint32_t DTIMER;         /*!< SDMMC data timer register,               Address offset: 0x24  */
  __IO uint32_t DLEN;           /*!< SDMMC data length register,              Address offset: 0x28  */
  __IO uint32_t DCTRL;          /*!< SDMMC data control register,             Address offset: 0x2C  */
  __I uint32_t  DCOUNT;         /*!< SDMMC data counter register,             Address offset: 0x30  */
  __I uint32_t  STA;            /*!< SDMMC status register,                   Address offset: 0x34  */
  __IO uint32_t ICR;            /*!< SDMMC interrupt clear register,          Address offset: 0x38  */
  __IO uint32_t MASK;           /*!< SDMMC mask register,                     Address offset: 0x3C  */
  __IO uint32_t ACKTIME;        /*!< SDMMC Acknowledgement timer register,    Address offset: 0x40  */
  uint32_t      RESERVED0[3];   /*!< Reserved, 0x44 - 0x4C - 0x4C                                   */
  __IO uint32_t IDMACTRL;       /*!< SDMMC DMA control register,              Address offset: 0x50  */
  __IO uint32_t IDMABSIZE;      /*!< SDMMC DMA buffer size register,          Address offset: 0x54  */
  __IO uint32_t IDMABASE0;      /*!< SDMMC DMA buffer 0 base address register, Address offset: 0x58  */
  __IO uint32_t IDMABASE1;      /*!< SDMMC DMA buffer 1 base address register, Address offset: 0x5C  */
  uint32_t      RESERVED1[8];   /*!< Reserved, 0x60-0x7C                                            */
  __IO uint32_t FIFO;           /*!< SDMMC data FIFO register,                Address offset: 0x80  */
  uint32_t      RESERVED2[222]; /*!< Reserved, 0x84-0x3F8                                           */
  __IO uint32_t IPVR;           /*!< SDMMC data FIFO register,                Address offset: 0x3FC */
} SDMMC_TypeDef;
```

这个结构体的成员名称和排列次序和 CPU 的寄存器是一 一对应的。

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

下面我们看下 SDMMC 的定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE           (0x40000000UL)
#define D1_AHB1PERIPH_BASE    (PERIPH_BASE + 0x12000000UL)
#define D2_AHB2PERIPH_BASE    (PERIPH_BASE + 0x08020000UL)

#define SDMMC1_BASE           (D1_AHB1PERIPH_BASE + 0x7000UL)
#define SDMMC2_BASE           (D2_AHB2PERIPH_BASE + 0x2400UL)

#define SDMMC1              ((SDMMC_TypeDef *) SDMMC1_BASE)
#define SDMMC2              ((SDMMC_TypeDef *) SDMMC2_BASE) <----- 展开这个宏，(SDMMC_TypeDef *)0x48022400
```

我们访问 SDMMC1 的 CMD 寄存器可以采用这种形式：SDMMC1->CMD = 0。

87.3.2 SDMMC 总线初始化结构体 SD_InitTypeDef

下面是 SDMMC 总线的初始化结构体：

```
#define SD_InitTypeDef      SDMMC_InitTypeDef
typedef struct
{
  uint32_t ClockEdge;
  uint32_t ClockPowerSave;
  uint32_t BusWide;
  uint32_t HardwareFlowControl;
  uint32_t ClockDiv;
```

```
#if (USE_SD_TRANSCEIVER != 0U)
  uint32_t TranceiverPresent;
#endif
}SDMMC_InitTypeDef;
```

下面将结构体成员逐一做个说明：

- ClockEdge

  用于设置 SDMMC 的数据或者命令变化的时钟沿。

```
#define SDMMC_CLOCK_EDGE_RISING              ((uint32_t)0x00000000U)
#define SDMMC_CLOCK_EDGE_FALLING             SDMMC_CLKCR_NEGEDGE
```

- ClockPowerSave

  用于设置空闲状态，是否输出时钟。

```
#define SDMMC_CLOCK_POWER_SAVE_DISABLE       ((uint32_t)0x00000000U)
#define SDMMC_CLOCK_POWER_SAVE_ENABLE        SDMMC_CLKCR_PWRSAV
```

- BusWide

  用于设置 SDMMC 总线位宽。

```
#define SDMMC_BUS_WIDE_1B                    ((uint32_t)0x00000000U)
#define SDMMC_BUS_WIDE_4B                    SDMMC_CLKCR_WIDBUS_0
#define SDMMC_BUS_WIDE_8B                    SDMMC_CLKCR_WIDBUS_1
```

- HardwareFlowControl

  用于设置时候使能硬件流控制。

```
#define SDMMC_HARDWARE_FLOW_CONTROL_DISABLE  ((uint32_t)0x00000000U)
#define SDMMC_HARDWARE_FLOW_CONTROL_ENABLE   SDMMC_CLKCR_HWFC_EN
```

- ClockDiv

  用于设置 SDMMC 时钟分频，参数范围 0 到 1023。

- TranceiverPresent

  用于设置是否带 1.8V 收发器。

```
#define SDMMC_TRANSCEIVER_UNKNOWN            ((uint32_t)0x00000000U)
#define SDMMC_TRANSCEIVER_NOT_PRESENT        ((uint32_t)0x00000001U)
#define SDMMC_TRANSCEIVER_PRESENT            ((uint32_t)0x00000002U)
```

87.3.3SDMMC 接 SD 卡信息结构体 HAL_SD_CardInfoTypeDef

下面是 SDMMC 总线的卡信息结构体：

```
typedef struct
{
  uint32_t CardType;                  /*!< Specifies the card Type                         */
  uint32_t CardVersion;               /*!< Specifies the card version                      */
  uint32_t Class;                     /*!< Specifies the class of the card class           */
  uint32_t RelCardAdd;                /*!< Specifies the Relative Card Address             */
  uint32_t BlockNbr;                  /*!< Specifies the Card Capacity in blocks           */
  uint32_t BlockSize;                 /*!< Specifies one block size in bytes               */
  uint32_t LogBlockNbr;               /*!< Specifies the Card logical Capacity in blocks   */
  uint32_t LogBlockSize;              /*!< Specifies logical block size in bytes           */
  uint32_t CardSpeed;                 /*!< Specifies the card Speed                        */
}HAL_SD_CardInfoTypeDef;
```

下面将结构体成员逐一做个说明：

- CardType

  卡类型。

```
/*!< SD Standard Capacity <2Go                          */
#define CARD_SDSC                  ((uint32_t)0x00000000U)
/*!< SD High Capacity <32Go, SD Extended Capacity <2To  */
#define CARD_SDHC_SDXC             ((uint32_t)0x00000001U)
#define CARD_SECURED               ((uint32_t)0x00000003U)
```

- CardVersion

  卡版本。

```
#define CARD_V1_X                  ((uint32_t)0x00000000U)
#define CARD_V2_X                  ((uint32_t)0x00000001U)
```

- Class

  卡类型。

- RelCardAdd

  卡相对地址。

- BlockNbr

  整个卡的块数。

- BlockSize

  每个块的字节数。

- LogBlockNbr

  整个卡的逻辑块数。

- LogBlockSize

  逻辑块大小

```
#define SPI_FIRSTBIT_MSB                            (0x00000000UL)
#define SPI_FIRSTBIT_LSB                            SPI_CFG2_LSBFRST
```

- CardSpeed

  用于设置是否使能 SPI 总线的 TI 模式。

```
/*!< Normal Speed Card <12.5Mo/s , Spec Version 1.01    */
#define CARD_NORMAL_SPEED          ((uint32_t)0x00000000U)

/*!< High Speed Card <25Mo/s , Spec version 2.00        */
#define CARD_HIGH_SPEED            ((uint32_t)0x00000100U)

/*!< UHS-I SD Card <50Mo/s for SDR50, DDR5 Cards
     and <104Mo/s for SDR104, Spec version 3.01         */
#define CARD_ULTRA_HIGH_SPEED      ((uint32_t)0x00000200U)
```

87.3.4SDMMC 总线句柄结构体 SD_HandleTypeDef

下面是 SDMMC 句柄结构体：

```
#if defined (USE_HAL_SD_REGISTER_CALLBACKS) && (USE_HAL_SD_REGISTER_CALLBACKS == 1U)
typedef struct __SD_HandleTypeDef
#else
```

```
typedef struct
#endif /* USE_HAL_SD_REGISTER_CALLBACKS */
{
  SD_TypeDef                   *Instance;        /*!< SD registers base address           */
  SD_InitTypeDef               Init;             /*!< SD required parameters              */
  HAL_LockTypeDef              Lock;             /*!< SD locking object                   */
  uint8_t                      *pTxBuffPtr;      /*!< Pointer to SD Tx transfer Buffer    */
  uint32_t                     TxXferSize;       /*!< SD Tx Transfer size                 */
  uint8_t                      *pRxBuffPtr;      /*!< Pointer to SD Rx transfer Buffer    */
  uint32_t                     RxXferSize;       /*!< SD Rx Transfer size                 */
  __IO uint32_t                Context;          /*!< SD transfer context                 */
  __IO HAL_SD_StateTypeDef     State;            /*!< SD card State                       */
  __IO uint32_t                ErrorCode;        /*!< SD Card Error codes                 */
  HAL_SD_CardInfoTypeDef       SdCard;           /*!< SD Card information                 */
  uint32_t                     CSD[4];           /*!< SD card specific data table         */
  uint32_t                     CID[4];           /*!< SD card identification number table */

#if defined (USE_HAL_SD_REGISTER_CALLBACKS) && (USE_HAL_SD_REGISTER_CALLBACKS == 1U)
  void (* TxCpltCallback)                (struct __SD_HandleTypeDef *hsd);
  void (* RxCpltCallback)                (struct __SD_HandleTypeDef *hsd);
  void (* ErrorCallback)                 (struct __SD_HandleTypeDef *hsd);
  void (* AbortCpltCallback)             (struct __SD_HandleTypeDef *hsd);
  void (* Read_DMADblBuf0CpltCallback)   (struct __SD_HandleTypeDef *hsd);
  void (* Read_DMADblBuf1CpltCallback)   (struct __SD_HandleTypeDef *hsd);
  void (* Write_DMADblBuf0CpltCallback)  (struct __SD_HandleTypeDef *hsd);
  void (* Write_DMADblBuf1CpltCallback)  (struct __SD_HandleTypeDef *hsd);
#if (USE_SD_TRANSCEIVER != 0U)
  void (* DriveTransceiver_1_8V_Callback) (FlagStatus status);
#endif /* USE_SD_TRANSCEIVER */

  void (* MspInitCallback)               (struct __SD_HandleTypeDef *hsd);
  void (* MspDeInitCallback)             (struct __SD_HandleTypeDef *hsd);
#endif /* USE_HAL_SD_REGISTER_CALLBACKS */
}SD_HandleTypeDef;
```

注意事项：

条件编译 USE_HAL_SD_REGISTER_CALLBACKS 用来设置使用自定义回调还是使用默认回调，此定义一般放在 stm32h7xx_hal_conf.h 文件里面设置：

```
#define    USE_HAL_SD_REGISTER_CALLBACKS    1
```

通过函数 HAL_SD_RegisterCallback 注册回调，取消注册使用函数 HAL_SD_UnRegisterCallback。

这里重点介绍下面几个参数，其它参数主要是 HAL 库内部使用和自定义回调函数。

- SD_TypeDef *Instance

  这个参数是寄存器的例化，方便操作寄存器。

- SD_InitTypeDef Init

  这个参数在本章节 3.2 小节已经进行了详细说明。

87.4 SDMMC 总线源文件 stm32h7xx_hal_sd.c

此文件涉及到的函数较多，这里把几个常用的函数做个说明：

- HAL_SD_Init
- HAL_SD_DeInit
- HAL_SD_ReadBlocks
- HAL_SD_WriteBlocks
- HAL_SD_ReadBlocks_DMA
- HAL_SD_WriteBlocks_DMA
- HAL_SD_Erase

87.4.1 函数 HAL_SD_Init

函数原型：

```
HAL_StatusTypeDef HAL_SD_Init(SD_HandleTypeDef *hsd)
{
  HAL_SD_CardStatusTypeDef CardStatus;
  uint32_t speedgrade, unitsize;
  uint32_t tickstart;

  /* 检查句柄是否有效 */
  if(hsd == NULL)
  {
    return HAL_ERROR;
  }

  /* 检查参数 */
  assert_param(IS_SDMMC_ALL_INSTANCE(hsd->Instance));
  assert_param(IS_SDMMC_CLOCK_EDGE(hsd->Init.ClockEdge));
  assert_param(IS_SDMMC_CLOCK_POWER_SAVE(hsd->Init.ClockPowerSave));
  assert_param(IS_SDMMC_BUS_WIDE(hsd->Init.BusWide));
  assert_param(IS_SDMMC_HARDWARE_FLOW_CONTROL(hsd->Init.HardwareFlowControl));
  assert_param(IS_SDMMC_CLKDIV(hsd->Init.ClockDiv));

  if(hsd->State == HAL_SD_STATE_RESET)
  {
    /* 开锁 */
    hsd->Lock = HAL_UNLOCKED;

#if (USE_SD_TRANSCEIVER != 0U)
    /* 兼容 */
    if (hsd->Init.TranceiverPresent == SDMMC_TRANSCEIVER_UNKNOWN)
    {
      hsd->Init.TranceiverPresent = SDMMC_TRANSCEIVER_PRESENT;
    }
#endif
#if defined (USE_HAL_SD_REGISTER_CALLBACKS) && (USE_HAL_SD_REGISTER_CALLBACKS == 1U)
    /* 复位回调 */
    hsd->TxCpltCallback    = HAL_SD_TxCpltCallback;
    hsd->RxCpltCallback    = HAL_SD_RxCpltCallback;
    hsd->ErrorCallback     = HAL_SD_ErrorCallback;
    hsd->AbortCpltCallback = HAL_SD_AbortCallback;
    hsd->Read_DMADblBuf0CpltCallback = HAL_SDEx_Read_DMADoubleBuf0CpltCallback;
```

```
    hsd->Read_DMADblBuf1CpltCallback = HAL_SDEx_Read_DMADoubleBuf1CpltCallback;
    hsd->Write_DMADblBuf0CpltCallback = HAL_SDEx_Write_DMADoubleBuf0CpltCallback;
    hsd->Write_DMADblBuf1CpltCallback = HAL_SDEx_Write_DMADoubleBuf1CpltCallback;
#if (USE_SD_TRANSCEIVER != 0U)
    if (hsd->Init.TranceiverPresent == SDMMC_TRANSCEIVER_PRESENT)
    {
      hsd->DriveTransceiver_1_8V_Callback = HAL_SD_DriveTransceiver_1_8V_Callback;
    }
#endif

    if(hsd->MspInitCallback == NULL)
    {
      hsd->MspInitCallback = HAL_SD_MspInit;
    }

    /* 初始化底层 */
    hsd->MspInitCallback(hsd);
#else
    /* 初始化底层硬件 GPIO, CLOCK, CORTEX...etc */
    HAL_SD_MspInit(hsd);
#endif /* USE_HAL_SD_REGISTER_CALLBACKS */
  }

  hsd->State = HAL_SD_STATE_BUSY;

  /* 初始化卡参数 */
  if (HAL_SD_InitCard(hsd) != HAL_OK)
  {
    return HAL_ERROR;
  }

  if( HAL_SD_GetCardStatus(hsd, &CardStatus) != HAL_OK)
  {
    return HAL_ERROR;
  }
  /* 获取卡速度等信息 */
  speedgrade = CardStatus.UhsSpeedGrade;
  unitsize = CardStatus.UhsAllocationUnitSize;
  if ((hsd->SdCard.CardType == CARD_SDHC_SDXC) && ((speedgrade != 0U) || (unitsize != 0U)))
  {
    hsd->SdCard.CardSpeed = CARD_ULTRA_HIGH_SPEED;
  }
  else
  {
    if (hsd->SdCard.CardType == CARD_SDHC_SDXC)
    {
      hsd->SdCard.CardSpeed  = CARD_HIGH_SPEED;
    }
    else
    {
      hsd->SdCard.CardSpeed  = CARD_NORMAL_SPEED;
    }

  }
  /* 配置总线位宽 */
  if(HAL_SD_ConfigWideBusOperation(hsd, hsd->Init.BusWide) != HAL_OK)
  {
    return HAL_ERROR;
```

```
  }

  /* 验证卡初始化后是否就绪 */
  tickstart = HAL_GetTick();
  while((HAL_SD_GetCardState(hsd) != HAL_SD_CARD_TRANSFER))
  {
    if((HAL_GetTick()-tickstart) >=  SDMMC_DATATIMEOUT)
    {
      hsd->ErrorCode = HAL_SD_ERROR_TIMEOUT;
      hsd->State= HAL_SD_STATE_READY;
      return HAL_TIMEOUT;
    }
  }

  hsd->ErrorCode = HAL_SD_ERROR_NONE;

  hsd->Context = SD_CONTEXT_NONE;

  hsd->State = HAL_SD_STATE_READY;

  return HAL_OK;
}
```

函数描述：

此函数用于初始化 SD 卡。

函数参数：

- 第 1 个参数是 SD_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

注意事项：

5. 函数 HAL_SD_MspInit 用于初始化 SD 的底层时钟、引脚等功能。需要用户自己在此函数里面实现具体的功能。由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
6. 如果形参 hsd 的结构体成员 State 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 SD_HandleTypeDef SdHandle。

   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_SD_STATE_RESET = 0x00U。

   解决办法有三

   方法 1：用户自己初始化 SD 和涉及到的 GPIO 等。

   方法 2：定义 SD_HandleTypeDef SdHandle 为全局变量。

   方法 3：下面的方法

```
if(HAL_SD_DeInit(&SdHandle) != HAL_OK)
{
    Error_Handler();
}
if(HAL_SD_Init(&SdHandle) != HAL_OK)
{
```

```
    Error_Handler();
}
```

使用举例：

```
SD_HandleTypeDef uSdHandle;

uSdHandle.Instance = SDMMC1;

/* if CLKDIV = 0 then SDMMC Clock frequency = SDMMC Kernel Clock
     else SDMMC Clock frequency = SDMMC Kernel Clock / [2 * CLKDIV].
     200MHz / (2*2) = 50MHz
*/
uSdHandle.Init.ClockDiv            = 2;
uSdHandle.Init.ClockPowerSave      = SDMMC_CLOCK_POWER_SAVE_DISABLE;
uSdHandle.Init.ClockEdge           = SDMMC_CLOCK_EDGE_RISING;
uSdHandle.Init.HardwareFlowControl = SDMMC_HARDWARE_FLOW_CONTROL_DISABLE;
uSdHandle.Init.BusWide             = SDMMC_BUS_WIDE_4B;
if(HAL_SD_Init(&uSdHandle) != HAL_OK)
{
   sd_state = MSD_ERROR;
}
```

87.4.2 函数 HAL_SD_DeInit

函数原型：

```
HAL_StatusTypeDef HAL_SD_DeInit(SD_HandleTypeDef *hsd)
{
  /* 检查 SD 卡句柄是否有效 */
  if(hsd == NULL)
  {
    return HAL_ERROR;
  }

  /* 检查参数 */
  assert_param(IS_SDMMC_ALL_INSTANCE(hsd->Instance));

  hsd->State = HAL_SD_STATE_BUSY;

#if (USE_SD_TRANSCEIVER != 0U)
  /* 关闭 1.8V 模式 */
  if (hsd->Init.TranceiverPresent == SDMMC_TRANSCEIVER_PRESENT)
  {
#if defined (USE_HAL_SD_REGISTER_CALLBACKS) && (USE_HAL_SD_REGISTER_CALLBACKS == 1U)
    if(hsd->DriveTransceiver_1_8V_Callback == NULL)
    {
      hsd->DriveTransceiver_1_8V_Callback = HAL_SD_DriveTransceiver_1_8V_Callback;
    }
    hsd->DriveTransceiver_1_8V_Callback(RESET);
#else
    HAL_SD_DriveTransceiver_1_8V_Callback(RESET);
#endif
  }
#endif

  /* 关闭 SD 卡电源 */
```

```
  SD_PowerOFF(hsd);

#if defined (USE_HAL_SD_REGISTER_CALLBACKS) && (USE_HAL_SD_REGISTER_CALLBACKS == 1U)
  if(hsd->MspDeInitCallback == NULL)
  {
    hsd->MspDeInitCallback = HAL_SD_MspDeInit;
  }

  /* 复位底层硬件 */
  hsd->MspDeInitCallback(hsd);
#else
  /* 复位底层硬件 */
  HAL_SD_MspDeInit(hsd);
#endif

  hsd->ErrorCode = HAL_SD_ERROR_NONE;
  hsd->State = HAL_SD_STATE_RESET;

  return HAL_OK;
}
```

函数描述：

用于复位 SD 总线初始化。

函数参数：

- 第 1 个参数是 SD_HandleTypeDef 类型结构体指针变量。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

87.4.3 函数 HAL_SD_ReadBlocks

函数原型：

```
HAL_StatusTypeDef HAL_SD_ReadBlocks(SD_HandleTypeDef *hsd, uint8_t *pData, uint32_t BlockAdd, uint32_t
NumberOfBlocks, uint32_t Timeout)
{
  SDMMC_DataInitTypeDef config;
  uint32_t errorstate;
  uint32_t tickstart = HAL_GetTick();
  uint32_t count, data, dataremaining;
  uint32_t add = BlockAdd;
  uint8_t *tempbuff = pData;

  if(NULL == pData)
  {
    hsd->ErrorCode |= HAL_SD_ERROR_PARAM;
    return HAL_ERROR;
  }

  if(hsd->State == HAL_SD_STATE_READY)
  {
    hsd->ErrorCode = HAL_SD_ERROR_NONE;
```

```
if((add + NumberOfBlocks) > (hsd->SdCard.LogBlockNbr))
{
  hsd->ErrorCode |= HAL_SD_ERROR_ADDR_OUT_OF_RANGE;
  return HAL_ERROR;
}

hsd->State = HAL_SD_STATE_BUSY;

/* 初始化数据控制寄存器 */
hsd->Instance->DCTRL = 0U;

if(hsd->SdCard.CardType != CARD_SDHC_SDXC)
{
  add *= 512U;
}

/* 配置 SD DPSM (Data Path State Machine) */
config.DataTimeOut   = SDMMC_DATATIMEOUT;
config.DataLength    = NumberOfBlocks * BLOCKSIZE;
config.DataBlockSize = SDMMC_DATABLOCK_SIZE_512B;
config.TransferDir   = SDMMC_TRANSFER_DIR_TO_SDMMC;
config.TransferMode  = SDMMC_TRANSFER_MODE_BLOCK;
config.DPSM          = SDMMC_DPSM_DISABLE;
(void)SDMMC_ConfigData(hsd->Instance, &config);
__SDMMC_CMDTRANS_ENABLE( hsd->Instance);

/* 查询方式块读取 */
if(NumberOfBlocks > 1U)
{
  hsd->Context = SD_CONTEXT_READ_MULTIPLE_BLOCK;

  /* 多块读取命令 */
  errorstate = SDMMC_CmdReadMultiBlock(hsd->Instance, add);
}
else
{
  hsd->Context = SD_CONTEXT_READ_SINGLE_BLOCK;

  /* 单块读取命令 */
  errorstate = SDMMC_CmdReadSingleBlock(hsd->Instance, add);
}
if(errorstate != HAL_SD_ERROR_NONE)
{
  /* 清除所有静态标志 */
  __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
  hsd->ErrorCode |= errorstate;
  hsd->State = HAL_SD_STATE_READY;
  hsd->Context = SD_CONTEXT_NONE;
  return HAL_ERROR;
}

/* 查询 SDMMC 标志 */
dataremaining = config.DataLength;
while(!__HAL_SD_GET_FLAG(hsd, SDMMC_FLAG_RXOVERR | SDMMC_FLAG_DCRCFAIL | SDMMC_FLAG_DTIMEOUT |
      SDMMC_FLAG_DATAEND))
{
  if(__HAL_SD_GET_FLAG(hsd, SDMMC_FLAG_RXFIFOHF) && (dataremaining >= 32U))
```

```
    {
      /* 从 SDMMC Rx FIFO 读取数据 */
      for(count = 0U; count < 8U; count++)
      {
        data = SDMMC_ReadFIFO(hsd->Instance);
        *tempbuff = (uint8_t)(data & 0xFFU);
        tempbuff++;
        *tempbuff = (uint8_t)((data >> 8U) & 0xFFU);
        tempbuff++;
        *tempbuff = (uint8_t)((data >> 16U) & 0xFFU);
        tempbuff++;
        *tempbuff = (uint8_t)((data >> 24U) & 0xFFU);
        tempbuff++;
      }
      dataremaining -= 32U;
    }

    if(((HAL_GetTick()-tickstart) >=  Timeout) || (Timeout == 0U))
    {
      /* 清除所有静态标志 */
      __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
      hsd->ErrorCode |= HAL_SD_ERROR_TIMEOUT;
      hsd->State= HAL_SD_STATE_READY;
      hsd->Context = SD_CONTEXT_NONE;
      return HAL_TIMEOUT;
    }
  }
  __SDMMC_CMDTRANS_DISABLE( hsd->Instance);

  /* 多块读取发送停止传输命令 */
  if(__HAL_SD_GET_FLAG(hsd, SDMMC_FLAG_DATAEND) && (NumberOfBlocks > 1U))
  {
    if(hsd->SdCard.CardType != CARD_SECURED)
    {
      /* 发送停止传输命令 */
      errorstate = SDMMC_CmdStopTransfer(hsd->Instance);
      if(errorstate != HAL_SD_ERROR_NONE)
      {
        /* 清除所有静态标志 */
        __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
        hsd->ErrorCode |= errorstate;
        hsd->State = HAL_SD_STATE_READY;
        hsd->Context = SD_CONTEXT_NONE;
        return HAL_ERROR;
      }
    }
  }

  /* 获取错误状态 */
  if(__HAL_SD_GET_FLAG(hsd, SDMMC_FLAG_DTIMEOUT))
  {
    /* 清除所有静态标志 */
    __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
    hsd->ErrorCode |= HAL_SD_ERROR_DATA_TIMEOUT;
    hsd->State = HAL_SD_STATE_READY;
    hsd->Context = SD_CONTEXT_NONE;
    return HAL_ERROR;
  }
```

```
    else if(__HAL_SD_GET_FLAG(hsd, SDMMC_FLAG_DCRCFAIL))
    {
      /* 清除所有静态标志 */
      __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
      hsd->ErrorCode |= HAL_SD_ERROR_DATA_CRC_FAIL;
      hsd->State = HAL_SD_STATE_READY;
      hsd->Context = SD_CONTEXT_NONE;
      return HAL_ERROR;
    }
    else if(__HAL_SD_GET_FLAG(hsd, SDMMC_FLAG_RXOVERR))
    {
      /* 清除所有静态标志 */
      __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
      hsd->ErrorCode |= HAL_SD_ERROR_RX_OVERRUN;
      hsd->State = HAL_SD_STATE_READY;
      hsd->Context = SD_CONTEXT_NONE;
      return HAL_ERROR;
    }
    else
    {
      /* 什么都不做 */
    }

    /* 清除所有静态标志 */
    __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_DATA_FLAGS);

    hsd->State = HAL_SD_STATE_READY;

    return HAL_OK;
  }
  else
  {
    hsd->ErrorCode |= HAL_SD_ERROR_BUSY;
    return HAL_ERROR;
  }
}
```

函数描述：

此函数主要用于 SD 卡数据读取。

函数参数：

- 第 1 个参数是 SD_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是接收数据的缓冲地址。
- 第 3 个参数是要读取的扇区地址，即从第几个扇区开始读取（512 字节为一个扇区）。
- 第 4 个参数是读取的扇区数。
- 第 5 个参数是传输过程的溢出时间，单位 ms。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

使用举例：

```
/**
```

```
  * @brief  Reads block(s) from a specified address in an SD card, in polling mode.
  * @param  pData: Pointer to the buffer that will contain the data to transmit
  * @param  ReadAddr: Address from where data is to be read
  * @param  NumOfBlocks: Number of SD blocks to read
  * @param  Timeout: Timeout for read operation
  * @retval SD status
  */
uint8_t BSP_SD_ReadBlocks(uint32_t *pData, uint32_t ReadAddr, uint32_t NumOfBlocks, uint32_t Timeout)
{

  if( HAL_SD_ReadBlocks(&uSdHandle, (uint8_t *)pData, ReadAddr, NumOfBlocks, Timeout) == HAL_OK)
  {
    return MSD_OK;
  }
  else
  {
    return MSD_ERROR;
  }

}
```

87.4.4函数 HAL_SD_WriteBlocks

函数原型：

```
HAL_StatusTypeDef HAL_SD_WriteBlocks(SD_HandleTypeDef *hsd, uint8_t *pData, uint32_t BlockAdd, uint32_t
NumberOfBlocks, uint32_t Timeout)
{
  SDMMC_DataInitTypeDef config;
  uint32_t errorstate;
  uint32_t tickstart = HAL_GetTick();
  uint32_t count, data, dataremaining;
  uint32_t add = BlockAdd;
  uint8_t *tempbuff = pData;

  if(NULL == pData)
  {
    hsd->ErrorCode |= HAL_SD_ERROR_PARAM;
    return HAL_ERROR;
  }

  if(hsd->State == HAL_SD_STATE_READY)
  {
    hsd->ErrorCode = HAL_SD_ERROR_NONE;

    if((add + NumberOfBlocks) > (hsd->SdCard.LogBlockNbr))
    {
      hsd->ErrorCode |= HAL_SD_ERROR_ADDR_OUT_OF_RANGE;
      return HAL_ERROR;
    }

    hsd->State = HAL_SD_STATE_BUSY;

    /* 初始化数据控制寄存器 */
    hsd->Instance->DCTRL = 0U;
```

```
    if(hsd->SdCard.CardType != CARD_SDHC_SDXC)
    {
      add *= 512U;
    }

    /* 配置 SD DPSM */
    config.DataTimeOut   = SDMMC_DATATIMEOUT;
    config.DataLength    = NumberOfBlocks * BLOCKSIZE;
    config.DataBlockSize = SDMMC_DATABLOCK_SIZE_512B;
    config.TransferDir   = SDMMC_TRANSFER_DIR_TO_CARD;
    config.TransferMode  = SDMMC_TRANSFER_MODE_BLOCK;
    config.DPSM          = SDMMC_DPSM_DISABLE;
    (void)SDMMC_ConfigData(hsd->Instance, &config);
    __SDMMC_CMDTRANS_ENABLE( hsd->Instance);

    /* 查询方式块写操作 */
    if(NumberOfBlocks > 1U)
    {
      hsd->Context = SD_CONTEXT_WRITE_MULTIPLE_BLOCK;

      /* 写多块命令 */
      errorstate = SDMMC_CmdWriteMultiBlock(hsd->Instance, add);
    }
    else
    {
      hsd->Context = SD_CONTEXT_WRITE_SINGLE_BLOCK;

      /* 写单块命令 */
      errorstate = SDMMC_CmdWriteSingleBlock(hsd->Instance, add);
    }
    if(errorstate != HAL_SD_ERROR_NONE)
    {
      /* 清除所有静态命令 */
      __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
      hsd->ErrorCode |= errorstate;
      hsd->State = HAL_SD_STATE_READY;
      hsd->Context = SD_CONTEXT_NONE;
      return HAL_ERROR;
    }

    /* 查询方式块写操作 */
    dataremaining = config.DataLength;
    while(!__HAL_SD_GET_FLAG(hsd, SDMMC_FLAG_TXUNDERR | SDMMC_FLAG_DCRCFAIL | SDMMC_FLAG_DTIMEOUT |
SDMMC_FLAG_DATAEND))
    {
      if(__HAL_SD_GET_FLAG(hsd, SDMMC_FLAG_TXFIFOHE) && (dataremaining >= 32U))
      {
        /* 写数据到 SDMMC Tx FIFO */
        for(count = 0U; count < 8U; count++)
        {
          data = (uint32_t)(*tempbuff);
          tempbuff++;
          data |= ((uint32_t)(*tempbuff) << 8U);
          tempbuff++;
          data |= ((uint32_t)(*tempbuff) << 16U);
          tempbuff++;
          data |= ((uint32_t)(*tempbuff) << 24U);
          tempbuff++;
```

```
        (void)SDMMC_WriteFIFO(hsd->Instance, &data);
      }
      dataremaining -= 32U;
    }

    if(((HAL_GetTick()-tickstart) >=  Timeout) || (Timeout == 0U))
    {
      /* 清除所有静态标志 */
      __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
      hsd->ErrorCode |= errorstate;
      hsd->State = HAL_SD_STATE_READY;
      hsd->Context = SD_CONTEXT_NONE;
      return HAL_TIMEOUT;
    }
  }
  __SDMMC_CMDTRANS_DISABLE( hsd->Instance);

  /* 多块写操作，发送停止传输命令 */
  if(__HAL_SD_GET_FLAG(hsd, SDMMC_FLAG_DATAEND) && (NumberOfBlocks > 1U))
  {
    if(hsd->SdCard.CardType != CARD_SECURED)
    {
      /* 发送停止传输命令 */
      errorstate = SDMMC_CmdStopTransfer(hsd->Instance);
      if(errorstate != HAL_SD_ERROR_NONE)
      {
        /* 清除所有静态传输标志 */
        __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
        hsd->ErrorCode |= errorstate;
        hsd->State = HAL_SD_STATE_READY;
        hsd->Context = SD_CONTEXT_NONE;
        return HAL_ERROR;
      }
    }
  }

  /* Get error state */
  if(__HAL_SD_GET_FLAG(hsd, SDMMC_FLAG_DTIMEOUT))
  {
    /* 清除所有静态传输标志 */
    __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
    hsd->ErrorCode |= HAL_SD_ERROR_DATA_TIMEOUT;
    hsd->State = HAL_SD_STATE_READY;
    hsd->Context = SD_CONTEXT_NONE;
    return HAL_ERROR;
  }
  else if(__HAL_SD_GET_FLAG(hsd, SDMMC_FLAG_DCRCFAIL))
  {
    /* 清除所有静态传输标志 */
    __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
    hsd->ErrorCode |= HAL_SD_ERROR_DATA_CRC_FAIL;
    hsd->State = HAL_SD_STATE_READY;
    hsd->Context = SD_CONTEXT_NONE;
    return HAL_ERROR;
  }
  else if(__HAL_SD_GET_FLAG(hsd, SDMMC_FLAG_TXUNDERR))
  {
    /* 清除所有静态传输标志 */
```

```
      __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
      hsd->ErrorCode |= HAL_SD_ERROR_TX_UNDERRUN;
      hsd->State = HAL_SD_STATE_READY;
      hsd->Context = SD_CONTEXT_NONE;
      return HAL_ERROR;
    }
    else
    {
      /* 什么都不做 */
    }

      /* 清除所有静态传输标志 */
    __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_DATA_FLAGS);

    hsd->State = HAL_SD_STATE_READY;

    return HAL_OK;
  }
  else
  {
    hsd->ErrorCode |= HAL_SD_ERROR_BUSY;
    return HAL_ERROR;
  }
}
```

函数描述：

此函数主要用于向 SD 卡写入数据。

函数参数：

- 第 1 个参数是 SD_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是要写入到 SD 卡的数据缓冲地址。
- 第 3 个参数是要写入的扇区地址，即从第几个扇区开始写入（512 字节为一个扇区）。
- 第 4 个参数是读取的扇区数。
- 第 5 个参数是传输过程的溢出时间，单位 ms。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

使用举例：

```
/**
  * @brief  Writes block(s) to a specified address in an SD card, in polling mode.
  * @param  pData: Pointer to the buffer that will contain the data to transmit
  * @param  WriteAddr: Address from where data is to be written
  * @param  NumOfBlocks: Number of SD blocks to write
  * @param  Timeout: Timeout for write operation
  * @retval SD status
  */
uint8_t BSP_SD_WriteBlocks(uint32_t *pData, uint32_t WriteAddr, uint32_t NumOfBlocks, uint32_t Timeout)
{

  if( HAL_SD_WriteBlocks(&uSdHandle, (uint8_t *)pData, WriteAddr, NumOfBlocks, Timeout) == HAL_OK)
  {
```

```
    return MSD_OK;
  }
  else
  {
    return MSD_ERROR;
  }
}
```

87.4.5 函数 HAL_SD_ReadBlocks_DMA

函数原型：

```
HAL_StatusTypeDef HAL_SD_ReadBlocks_DMA(SD_HandleTypeDef *hsd, uint8_t *pData, uint32_t BlockAdd, uint32_t
NumberOfBlocks)
{
  SDMMC_DataInitTypeDef config;
  uint32_t errorstate;
  uint32_t add = BlockAdd;

  if(NULL == pData)
  {
    hsd->ErrorCode |= HAL_SD_ERROR_PARAM;
    return HAL_ERROR;
  }

  if(hsd->State == HAL_SD_STATE_READY)
  {
    hsd->ErrorCode = HAL_SD_ERROR_NONE;

    if((add + NumberOfBlocks) > (hsd->SdCard.LogBlockNbr))
    {
      hsd->ErrorCode |= HAL_SD_ERROR_ADDR_OUT_OF_RANGE;
      return HAL_ERROR;
    }

    hsd->State = HAL_SD_STATE_BUSY;

    /* 初始化数据控制寄存器 */
    hsd->Instance->DCTRL = 0U;

    hsd->pRxBuffPtr = pData;
    hsd->RxXferSize = BLOCKSIZE * NumberOfBlocks;

    if(hsd->SdCard.CardType != CARD_SDHC_SDXC)
    {
      add *= 512U;
    }

    /* 配置 SD DPSM (Data Path State Machine) */
    config.DataTimeOut   = SDMMC_DATATIMEOUT;
    config.DataLength    = BLOCKSIZE * NumberOfBlocks;
    config.DataBlockSize = SDMMC_DATABLOCK_SIZE_512B;
    config.TransferDir   = SDMMC_TRANSFER_DIR_TO_SDMMC;
    config.TransferMode  = SDMMC_TRANSFER_MODE_BLOCK;
    config.DPSM          = SDMMC_DPSM_DISABLE;
    (void)SDMMC_ConfigData(hsd->Instance, &config);
```

```
    __SDMMC_CMDTRANS_ENABLE( hsd->Instance);
    hsd->Instance->IDMABASE0 = (uint32_t) pData ;
    hsd->Instance->IDMACTRL  = SDMMC_ENABLE_IDMA_SINGLE_BUFF;

    /* DMA 方式读取多个块 */
    if(NumberOfBlocks > 1U)
    {
      hsd->Context = (SD_CONTEXT_READ_MULTIPLE_BLOCK | SD_CONTEXT_DMA);

     /* DMA 方式读取多块命令 */
      errorstate = SDMMC_CmdReadMultiBlock(hsd->Instance, add);
    }
    else
    {
      hsd->Context = (SD_CONTEXT_READ_SINGLE_BLOCK | SD_CONTEXT_DMA);

     /* 读取单块命令 */
      errorstate = SDMMC_CmdReadSingleBlock(hsd->Instance, add);
    }
    if(errorstate != HAL_SD_ERROR_NONE)
    {
     /* 清除所有静态标志 */
      __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
      hsd->ErrorCode |= errorstate;
      hsd->State = HAL_SD_STATE_READY;
      hsd->Context = SD_CONTEXT_NONE;
      return HAL_ERROR;
    }

    /* 使能传输中断 */
    __HAL_SD_ENABLE_IT(hsd, (SDMMC_IT_DCRCFAIL | SDMMC_IT_DTIMEOUT | SDMMC_IT_RXOVERR | SDMMC_IT_DATAEND));


    return HAL_OK;
  }
  else
  {
    return HAL_BUSY;
  }
}
```

函数描述：

此函数主要用于 SD 卡数据读取，DMA 方式。

函数参数：

- 第 1 个参数是 SD_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是接收数据的缓冲地址。
- 第 3 个参数是要读取的扇区地址，即从第几个扇区开始读取（512 字节为一个扇区）。
- 第 4 个参数是读取的扇区数。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

使用举例：

```
/**
  * @brief  Reads block(s) from a specified address in an SD card, in DMA mode.
  * @param  pData: Pointer to the buffer that will contain the data to transmit
  * @param  ReadAddr: Address from where data is to be read
  * @param  NumOfBlocks: Number of SD blocks to read
  * @retval SD status
*/
uint8_t BSP_SD_ReadBlocks_DMA(uint32_t *pData, uint32_t ReadAddr, uint32_t NumOfBlocks)
{

  if( HAL_SD_ReadBlocks_DMA(&uSdHandle, (uint8_t *)pData, ReadAddr, NumOfBlocks) == HAL_OK)
  {
    return MSD_OK;
  }
  else
  {
    return MSD_ERROR;
  }
}
```

87.4.6函数 HAL_SD_WriteBlocks_DMA

函数原型：

```
HAL_StatusTypeDef HAL_SD_WriteBlocks_DMA(SD_HandleTypeDef *hsd, uint8_t *pData, uint32_t BlockAdd, uint32_t
NumberOfBlocks)
{
  SDMMC_DataInitTypeDef config;
  uint32_t errorstate;
  uint32_t add = BlockAdd;

  if(NULL == pData)
  {
    hsd->ErrorCode |= HAL_SD_ERROR_PARAM;
    return HAL_ERROR;
  }

  if(hsd->State == HAL_SD_STATE_READY)
  {
    hsd->ErrorCode = HAL_SD_ERROR_NONE;

    if((add + NumberOfBlocks) > (hsd->SdCard.LogBlockNbr))
    {
      hsd->ErrorCode |= HAL_SD_ERROR_ADDR_OUT_OF_RANGE;
      return HAL_ERROR;
    }

    hsd->State = HAL_SD_STATE_BUSY;

    /* 初始化数据控制寄存器 */
    hsd->Instance->DCTRL = 0U;

    hsd->pTxBuffPtr = pData;
    hsd->TxXferSize = BLOCKSIZE * NumberOfBlocks;

    if(hsd->SdCard.CardType != CARD_SDHC_SDXC)
```

```
    {
      add *= 512U;
    }

    /* 配置 SD DPSM (Data Path State Machine) */
    config.DataTimeOut   = SDMMC_DATATIMEOUT;
    config.DataLength    = BLOCKSIZE * NumberOfBlocks;
    config.DataBlockSize = SDMMC_DATABLOCK_SIZE_512B;
    config.TransferDir   = SDMMC_TRANSFER_DIR_TO_CARD;
    config.TransferMode  = SDMMC_TRANSFER_MODE_BLOCK;
    config.DPSM          = SDMMC_DPSM_DISABLE;
    (void)SDMMC_ConfigData(hsd->Instance, &config);


    __SDMMC_CMDTRANS_ENABLE( hsd->Instance);

    hsd->Instance->IDMABASE0 = (uint32_t) pData ;
    hsd->Instance->IDMACTRL  = SDMMC_ENABLE_IDMA_SINGLE_BUFF;

    /* 查询模式写块 */
    if(NumberOfBlocks > 1U)
    {
      hsd->Context = (SD_CONTEXT_WRITE_MULTIPLE_BLOCK | SD_CONTEXT_DMA);

      /* 多块写命令 */
      errorstate = SDMMC_CmdWriteMultiBlock(hsd->Instance, add);
    }
    else
    {
      hsd->Context = (SD_CONTEXT_WRITE_SINGLE_BLOCK | SD_CONTEXT_DMA);

      /* 单块写命令 */
      errorstate = SDMMC_CmdWriteSingleBlock(hsd->Instance, add);
    }
    if(errorstate != HAL_SD_ERROR_NONE)
    {
      /* 清除静态标志 */
      __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
      hsd->ErrorCode |= errorstate;
      hsd->State = HAL_SD_STATE_READY;
      hsd->Context = SD_CONTEXT_NONE;
      return HAL_ERROR;
    }

    /* 使能传输中断 Enable */
    __HAL_SD_ENABLE_IT(hsd, (SDMMC_IT_DCRCFAIL | SDMMC_IT_DTIMEOUT | SDMMC_IT_TXUNDERR | SDMMC_IT_DATAEND));

    return HAL_OK;
  }
  else
  {
    return HAL_BUSY;
  }
}
```

函数描述：

此函数主要用于向 SD 卡写入数据，DMA 方式。

函数参数：

- 第 1 个参数是 SD_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是要写入到 SD 卡的数据缓冲地址。
- 第 3 个参数是要写入的扇区地址，即从第几个扇区开始写入（512 字节为一个扇区）。
- 第 4 个参数是读取的扇区数。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

使用举例：

```
/**
  * @brief  Writes block(s) to a specified address in an SD card, in DMA mode.
  * @param  pData: Pointer to the buffer that will contain the data to transmit
  * @param  WriteAddr: Address from where data is to be written
  * @param  NumOfBlocks: Number of SD blocks to write
  * @retval SD status
*/
uint8_t BSP_SD_WriteBlocks_DMA(uint32_t *pData, uint32_t WriteAddr, uint32_t NumOfBlocks)
{

  if( HAL_SD_WriteBlocks_DMA(&uSdHandle, (uint8_t *)pData, WriteAddr, NumOfBlocks) == HAL_OK)
  {
    return MSD_OK;
  }
  else
  {
    return MSD_ERROR;
  }
}
```

87.4.7 函数 HAL_SD_Erase

函数原型：

```
HAL_StatusTypeDef HAL_SD_Erase(SD_HandleTypeDef *hsd, uint32_t BlockStartAdd, uint32_t BlockEndAdd)
{
  uint32_t errorstate;
  uint32_t start_add = BlockStartAdd;
  uint32_t end_add = BlockEndAdd;

  if(hsd->State == HAL_SD_STATE_READY)
  {
    hsd->ErrorCode = HAL_SD_ERROR_NONE;

    if(end_add < start_add)
    {
      hsd->ErrorCode |= HAL_SD_ERROR_PARAM;
      return HAL_ERROR;
    }

    if(end_add > (hsd->SdCard.LogBlockNbr))
    {
      hsd->ErrorCode |= HAL_SD_ERROR_ADDR_OUT_OF_RANGE;
```

```
    return HAL_ERROR;
  }

  hsd->State = HAL_SD_STATE_BUSY;

  /* 检测是否支持擦除命令 */
  if(((hsd->SdCard.Class) & SDMMC_CCCC_ERASE) == 0U)
  {
    /* 清除所有静态标志 */
    __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
    hsd->ErrorCode |= HAL_SD_ERROR_REQUEST_NOT_APPLICABLE;
    hsd->State = HAL_SD_STATE_READY;
    return HAL_ERROR;
  }

  if((SDMMC_GetResponse(hsd->Instance, SDMMC_RESP1) & SDMMC_CARD_LOCKED) == SDMMC_CARD_LOCKED)
  {
    /* 清除所有静态标志 */
    __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
    hsd->ErrorCode |= HAL_SD_ERROR_LOCK_UNLOCK_FAILED;
    hsd->State = HAL_SD_STATE_READY;
    return HAL_ERROR;
  }

  /* 对于高容量卡，获取起始块和结束块 */
  if(hsd->SdCard.CardType != CARD_SDHC_SDXC)
  {
    start_add *= 512U;
    end_add   *= 512U;
  }

    /* 根据 sd-card spec 1.0 ERASE_GROUP_START (CMD32) 和 erase_group_end(CMD33) */
  if(hsd->SdCard.CardType != CARD_SECURED)
  {
    /* 发送 CMD32 SD_ERASE_GRP_START 命令带地址参数 */
    errorstate = SDMMC_CmdSDEraseStartAdd(hsd->Instance, start_add);
    if(errorstate != HAL_SD_ERROR_NONE)
    {
        /* 清除所有静态标志 */
      __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
      hsd->ErrorCode |= errorstate;
      hsd->State = HAL_SD_STATE_READY;
      return HAL_ERROR;
    }

      /* 发送 CMD33 SD_ERASE_GRP_END 命令，带地址参数 */
    errorstate = SDMMC_CmdSDEraseEndAdd(hsd->Instance, end_add);
    if(errorstate != HAL_SD_ERROR_NONE)
    {
        /* 清除所有静态标志 */
      __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
      hsd->ErrorCode |= errorstate;
      hsd->State = HAL_SD_STATE_READY;
      return HAL_ERROR;
    }
  }

    /* 发送 CMD38 ERASE 命令 */
```

```
    errorstate = SDMMC_CmdErase(hsd->Instance, 0UL);
    if(errorstate != HAL_SD_ERROR_NONE)
    {
        /* 清除所有静态标志 */
      __HAL_SD_CLEAR_FLAG(hsd, SDMMC_STATIC_FLAGS);
      hsd->ErrorCode |= errorstate;
      hsd->State = HAL_SD_STATE_READY;
      return HAL_ERROR;
    }

    hsd->State = HAL_SD_STATE_READY;

    return HAL_OK;
  }
  else
  {
    return HAL_BUSY;
  }
}
```

函数描述：

此函数主要用于 SD 卡擦除。

函数参数：

- 第 1 个参数是 SD_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是擦除的起始扇区地址，地址单位是第几个扇区（512 字节为一个扇区）。
- 第 3 个参数是擦除的结束扇区地址，地址单位是第几个扇区（512 字节为一个扇区）。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

使用举例：

```
/**
 * @brief  Erases the specified memory area of the given SD card.
 * @param  StartAddr: Start byte address
 * @param  EndAddr: End byte address
 * @retval SD status
*/
uint8_t BSP_SD_Erase(uint32_t StartAddr, uint32_t EndAddr)
{

  if( HAL_SD_Erase(&uSdHandle, StartAddr, EndAddr) == HAL_OK)
  {
    return MSD_OK;
  }
  else
  {
    return MSD_ERROR;
  }
}
```

87.5 总结

本章节就为大家讲解这么多，更多 SDMMC 知识可以看 STM32H7 的参考手册。

第88章 STM32H7 的 SDMMC 总线应用之 SD 卡移植 FatFs 文件系统

本章节为大家讲解 SD 卡的 FatFs 文件系统移植。

88.1 初学者重要提示
88.2 SD 卡硬件接口设计
88.3 SD 卡基础知识
88.4 各种存储卡区别
88.5 关于 SD 卡内部是否自带擦写均衡
88.6 FatFs 文件系统简介
88.7 FatFs 移植步骤
88.8 FatFs 应用代码测试
88.9 FatFs 移植接口文件 diskio.c 说明
88.10 SDMMMC 自带 IDMA 的 4 字节对齐问题（重要）
88.11 实验例程设计框架
88.12 实验例程说明（MDK）
88.13 实验例程说明（IAR）
88.14 总结

88.1 初学者重要提示

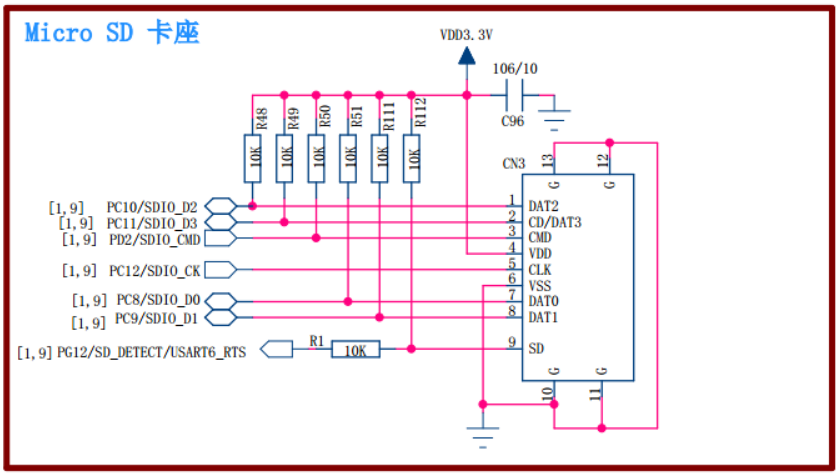
- SDMMC 的相关知识点可以看第 87 章。
- 操作 SD 卡是以扇区（512 字节）为单位进行操作。
- SD 卡联盟强烈强烈建议使用此软件来格式化 SD/SDHC/SDXC 卡，而不要使用各个操作系统随附的格式化工具。通常，操作系统附带的格式化工具可以格式化包括 SD/SDHC/SDXC 卡在内的各种存储介质，但是可能无法针对 SD/SDHC/SDXC 卡进行优化，并且可能导致性能降低。
  软件下载：http://www.armbbs.cn/forum.php?mod=viewthread&tid=96181 。
- STM32H7 的 SDMMC 也支持 eMMC：
  - 【普及贴】各个厂家 eMMC 读写速度，镁光，东芝，三星，ISSI 和旺宏
    http://www.armbbs.cn/forum.php?mod=viewthread&tid=95954

- H7 的 8 线 SDIO DMA 驱动 eMMC 的裸机性能，读 43MB/S，写 18.8MB/S
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=95953

- 支持 128GB 的大容量 SD 卡，需要大家使能 FatFS 的 exFAT 即可。
- FatFs 的挂载函数 f_mount 可以上电后仅调用一次，本章配套例子为了测试方式，使用前挂载，使用完毕后卸载。
- FAT 文件系统基础：
  http://elm-chan.org/fsw/ff/00index_e.html
- exFAT 文件系统基础：
  http://elm-chan.org/fsw/ff/00index_e.html
- FatFS API 列表：
  http://elm-chan.org/fsw/ff/00index_e.html
- FatFS 应用笔记（含移植说明）
  http://elm-chan.org/fsw/ff/doc/appnote.html
- FatFS 中 ffconf.h 文件中各种配置说明：
  http://elm-chan.org/fsw/ff/doc/config.html
- FatFs 的 RTOS 版本移植：
  - 基于 STM32H7 的 uCOS-III + FatFS + emWin + ST USB 的综合模板下载
    http://www.armbbs.cn/forum.php?mod=viewthread&tid=100125
  - 基于 STM32H7 的 FreeRTOS+ FatFS + emWin + ST USB 的综合模板下载
    http://www.armbbs.cn/forum.php?mod=viewthread&tid=100127
  - 基于 STM32H7 的 uCOS-II + FatFS + emWin + ST USB 的综合模板下载
    http://www.armbbs.cn/forum.php?mod=viewthread&tid=100126

88.2 SD 卡硬件接口设计

STM32H7 驱动 SD 卡设计如下：



关于这个原理图，要了解到以下几个知识：

- 大家自己设计推荐也接上拉电阻。
- 这里采用 SDMMC 的 4 线方式。

88.3 SD 卡基础知识

这里将 SD 卡相关的基础知识为大家做个普及。

88.3.1 SD 卡分类

根据不同容量做的区分，主要包括 Full SD，miniSD 和 microSD。

	Full SD	miniSD	microSD	SD Card Capacity
SD				up to 2GB
SDHC				High Capacity 4GB to 32GB
SDXC				Extended Capacity Over 32GB up to 2TB

88.3.2 SD 卡容量及其使用的文件系统

容量小于 2GB(SD 卡)使用 FAT12 或者 FAT16,容量在 2GB 和 32GB 之间(SDHC 卡)使用 FAT32，容量大于 32GB 小于 2TB（SDXC 卡）使用 exFAT。

		SD Standard	SDHC Standard	SDXC Standard
Capacity		up to 2GB	more than 2GB up to 32GB	more than 32GB up to 2TB
File System		FAT 12, 16	FAT 32	exFAT
SD Logo				
Card Specifications	SD	32 x 24 x 2.1 mm, Approx 2g		
	miniSD	20 x 21.5 x 1.4 mm, Approx 1g		
	microSD	11 x 15 x 1.0 mm, Approx 0.5g		
Bus Speed		Normal Speed (NS) High Speed (HS)	NS, HS UHS-I	NS, HS UHS-I
Speed Classes	NS, HS mode	Speed Class	Speed Class	Speed Class
	UHS-I mode		Speed Class UHS Speed Class	Speed Class UHS Speed Class

88.3.3SD 卡总线速度和速度等级

SD 卡速度：

Bus Interface	Card Type	Bus Mark	Bus Speed	Spec Version
Normal Speed	SD, SDHC and SDXC	---	12.5MB/s	1.01
High Speed	SD, SDHC and SDXC	---	25MB/s	2.00
UHS-I	SDHC and SDXC		50MB/s (SDR50, DDR50) 104MB/s (SDR104)	3.01
UHS-II	SDHC and SDXC		156MB/s 312MB/s	4.00

SD 卡速度等级：

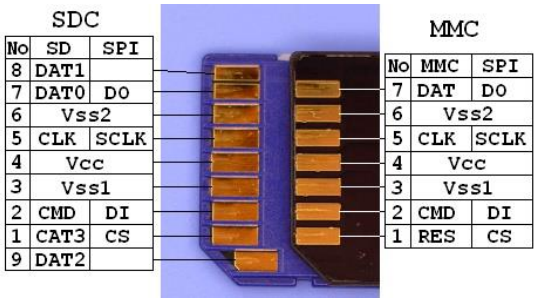
	Mark	Minimum Serial Data	SD Bus Mode	Application
UHS Speed Class	U3	30MB/s	UHS-II UHS-I	4K2K Video Recording
	U1	10MB/s		Full HD Video Recording HD Still Image Continuous Shooting
Speed Class	CLASS10	10MB/s	High Speed	
	CLASS6	6MB/s	Normal Speed	HD and Full HD Video Recording
	CLASS4	4MB/s		
	CLASS2	2MB/s		Standard Video Recording

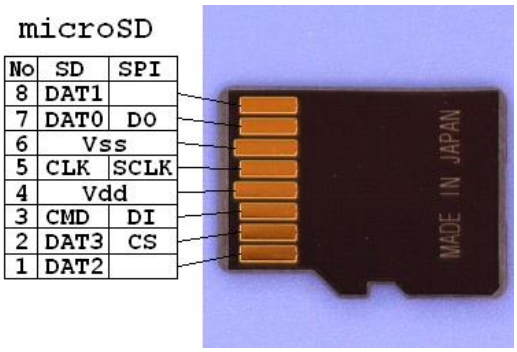
88.4 各种存储卡区别

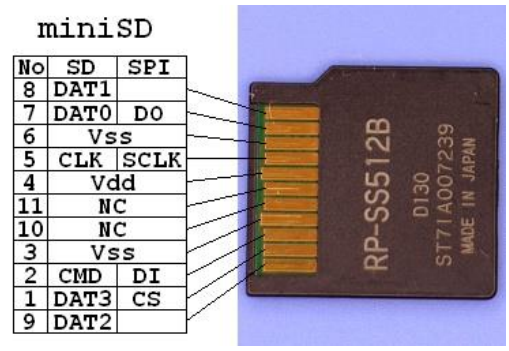
市面上的卡种类非常多，容易把人搞糊涂，这里将这些卡种类为大家做个区分：

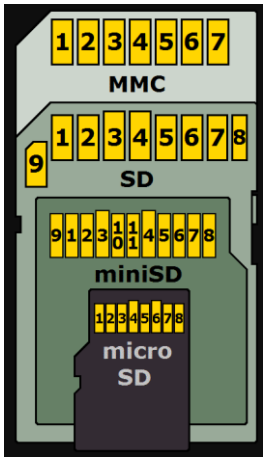
88.4.1 SD 卡，miniSD 卡，TF 卡，MircoSD 卡

TF 卡是 MicroSD 卡的另一种叫法，无需做区分。SD 卡，miniSD 卡，MircoSD 卡其实是一种卡，区别是引脚使用上。









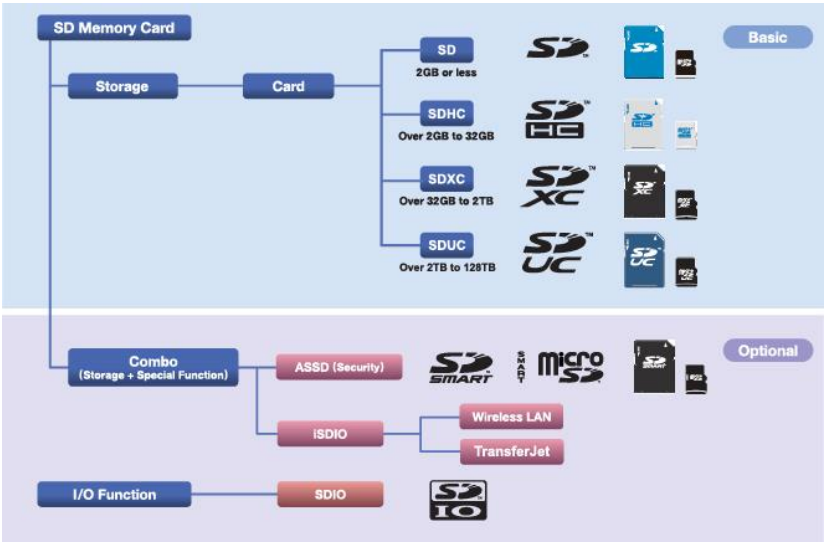
88.4.2 SDIO 卡

SDIO 卡就是使用 SDIO 外设来接 SD 卡。

而为什么叫 SDIO，根据 wiki 百科说明，其实就是 SD 卡接口规范的扩展，带了输入输出功能，这个接口不仅可以接 SD 卡，还可以接其它外设，如条形码读卡器，WiFi，蓝牙，调制解调器等。

https://en.wikipedia.org/wiki/SD_card#SDIO_cards

对于 STM32 的 SDIO 来说，他就是指 STM32 的一个外设接口，不仅能够来接 SD 卡，还可以接其它外设。



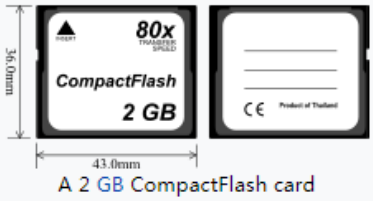
88.4.3 MMC 卡，eMMC

截止 2018 年，市场上已经没有设备内置 MMC 卡槽，但 eMMC 得到了广泛应用 https://en.wikipedia.org/wiki/MultiMediaCard 。

88.4.4 CF 卡

CF 卡是早期最成功的存储卡格式之一，像 MMC/SD 卡都是后来才推出的。CF 卡仍然很受欢迎卡之一，并得到许多专业设备和高端消费类设备的支持。截至 2017 年，佳能和尼康都将 CompactFlash 用于其旗舰数码相机。佳能还选择了 CompactFlash 作为其专业高清无带摄像机的记录介质。

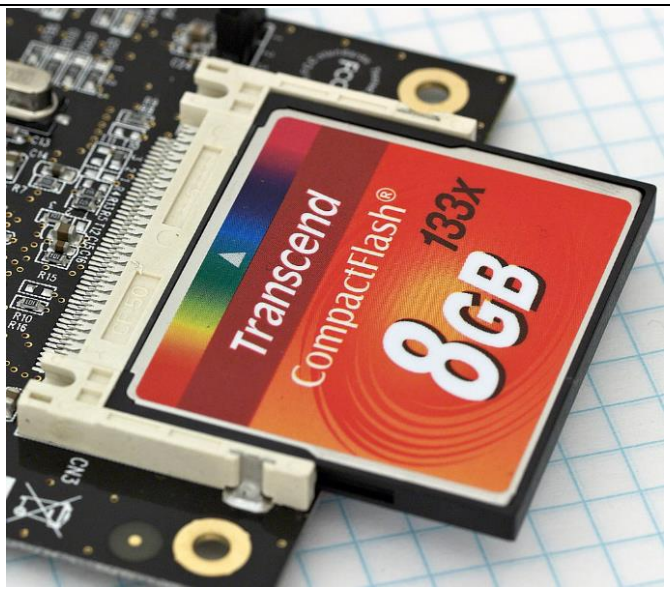
基础规格：



A 2 GB CompactFlash card

Media type	Mass storage device format
Encoding	Various file systems
Capacity	2 MB to 512 GB[1][2] CF5.0: up to 128 PB
Developed by	SanDisk
Dimensions	43×36×3.3 mm (Type I) 43×36×5 mm (Type II)
Weight	10 grams (typical)
Usage	Digital cameras and other mass storage devices
Extended from	PCMCIA / PC Card

实际效果：



88.4.5 总体区别

来自 Wiki：https://en.wikipedia.org/wiki/SD_card#Micro

Comparison of technical features of MMC and SD card variants

Type	MMC	RS-MMC	MMCplus	MMCmobile	SecureMMC	SDIO	SD	miniSD	microSD
SD-socket compatible	Yes	Extender	Yes	Extender	Yes	Yes	Yes	Adapter	Adapter
Pins	7	7	13	13	7	9	9	11	8
Width	24 mm	24 mm	24 mm	24 mm	24 mm	24 mm	24 mm	20 mm	11 mm
Length	32 mm	18 mm	32 mm	18 mm	32 mm	32 mm+	32 mm	21.5 mm	15 mm
Thickness	1.4 mm	1.4 mm	1.4 mm	1.4 mm	1.4 mm	2.1 mm	2.1 mm (most) 1.4 mm (rare)	1.4 mm	1 mm
1-bit SPI-bus mode	Optional	Optional	Optional	Optional	Yes	Yes	Yes	Yes	Yes
Max SPI bus clock	20 MHz	20 MHz	52 MHz	52 MHz	20 MHz	50 MHz	25 MHz	50 MHz	50 MHz
1-bit MMC/SD bus mode	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
4-bit MMC/SD bus mode	No	No	Yes	Yes	No	Optional	Yes	Yes	Yes
8-bit MMC bus mode	No	No	Yes	Yes	No	No	No	No	No
DDR mode	No	No	Yes	Yes	Unknown	Unknown	Unknown	Unknown	Unknown
Max MMC/SD bus clock	20 MHz	20 MHz	52 MHz	52 MHz	20 MHz?	50 MHz	208 MHz	208 MHz	208 MHz
Max MMC/SD transfer rate	20 Mbit/s	20 Mbit/s	832 Mbit/s	832 Mbit/s	20 Mbit/s?	200 Mbit/s	832 Mbit/s	832 Mbit/s	832 Mbit/s
Interrupts	No	No	No	No	No	Optional	No	No	No
DRM support	No	No	No	No	Yes	N/A	Yes	Yes	Yes
User encrypt	No	No	No	No	Yes	No	No	No	No
Simplified spec.	Yes	Yes	No	No	Unknown	Yes	Yes	No	No
Membership cost	JEDEC: US$4,400/yr, optional					SD Card Association: US$2,000/year, general; US$4,500/year, executive			
Specification cost	Free				Unknown	Simplified: free. Full: membership, or US$1,000/year to R&D non-members			
Host license	No	No	No	No	No	US$1,000/year, excepting SPI-mode only use			
Card royalties	Yes	Yes	Yes	Yes	Yes	Yes, US$1,000/year	Yes	Yes	Yes
Open-source compatible	Yes	Yes	Unknown	Unknown	Unknown	Yes	Yes	Yes	Yes
Nominal voltage	3.3 V	3.3 V	3.3 V[163][164]	1.8 V/3.3 V	1.8 V/3.3 V	3.3 V	3.3 V (SDSC), 1.8/3.3 V (SDHC, SDXC & SDUC)	3.3 V (miniSD), 1.8/3.3 V (miniSDHC)	3.3 V (SDSC), 1.8/3.3 V (microSDHC, microSDXC & microSDUC)
Max capacity	128 GB	2 GB	128 GB?	2 GB	128 GB?	?	2 GB (SD), 32 GB (SDHC), 1 TB (SDXC), 2 TB (SDXC, theoretical), 128 TB (SDUC, theoretical)	2 GB (miniSD), 16 GB (miniSDHC)	2 GB (microSD), 32 GB (microSDHC), 1 TB (microSDXC), 2 TB (microSDXC, theoretical), 128 TB (microSDUC, theoretical)
Type	MMC	RS-MMC	MMCplus	MMCmobile	SecureMMC	SDIO	SD	miniSD	microSD

88.5 关于 SD 卡内部是否自带擦写均衡

根据网上搜的一个闪迪的规格书，里面说是带擦写均衡的：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=102891

1.5.3 Content Protection

A detailed description of the content protection mechanism and related security SD commands can be found in the SD Security Specification from the SDA. All SD security-related commands in the SanDisk microSD Card Product Family operate in the data transfer mode.

1.5.4 Wear Leveling

Wear leveling is an intrinsic part of the erase pooling functionality of cards in the SanDisk microSD Card Product Family using NAND memory.

03/10, Version 2.0. © 2008 - 2010 SanDisk Corporation. SanDisk Confidential, subject to all applicable non-disclosure agreements

1.5.5 Automatic Sleep Mode

A unique feature of cards in the SanDisk microSD Card Product Family is automatic entrance and exit from sleep mode. Upon completion of user operations and subsequently needed flash management, cards enter sleep mode to conserve power. The host does not have to take any action for this to occur. However, in order to achieve the lowest sleep current, the host needs to shut down its clock to the card. In most systems, cards are in sleep mode except when accessed by the host, thus conserving power.

When the host is ready to access a card in sleep mode, any command issued to it will cause it to exit sleep, and respond.

1.5.6 Hot Insertion

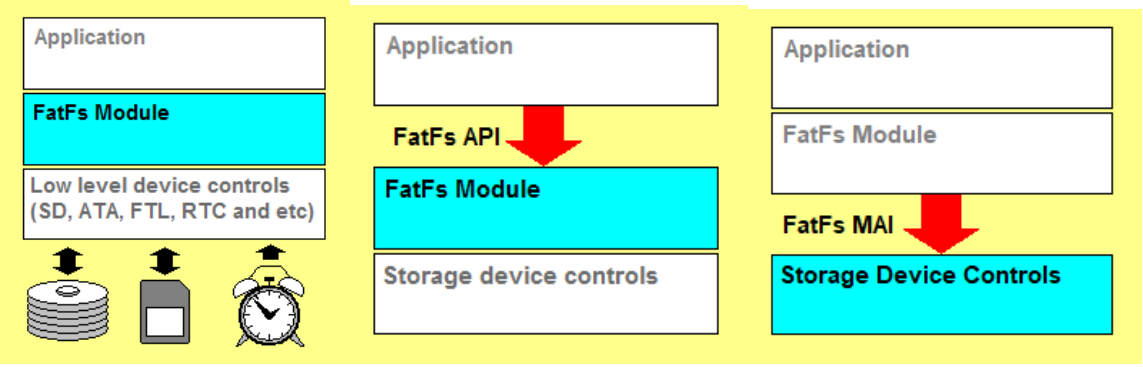
Support for hot insertion will be required on the host but will be supported through the connector. Connector manufacturers will provide connectors that have power pins long enough to be powered before contact is made with the other pins. This approach is similar to that used in PCMCIA devices to allow for hot insertion.

88.6 FatFs 文件系统简介

FatFs 是用于小型嵌入式系统的通用 FAT / exFAT 文件系统模块。FatFs 是按照 ANSI C（C89）编写的并且与磁盘 I / O 层完全分开。因此，它独立于平台。它可以并入资源有限的小型 MCU 中，例如 8051，PIC，AVR，ARM，Z80，RX 等。此处还提供了适用于小型 MCU 的 Petit FatFs 模块。

特征：

- DOS / Windows 兼容的 FAT / exFAT 文件系统。
- 平台无关，容易移植。
- 程序代码和工作区的占用空间非常小。
- 支持以下各种配置选项：
  - ANSI / OEM 或 Unicode 中的长文件名。
  - exFAT 文件系统，64 位 LBA 和 GPT 可存储大量数据。
  - RTOS 的线程安全。
  - 多个卷（物理驱动器和分区）。
  - 可变扇区大小。
  - 多个代码页，包括 DBCS。
  - 只读，可选 API，I / O 缓冲区等



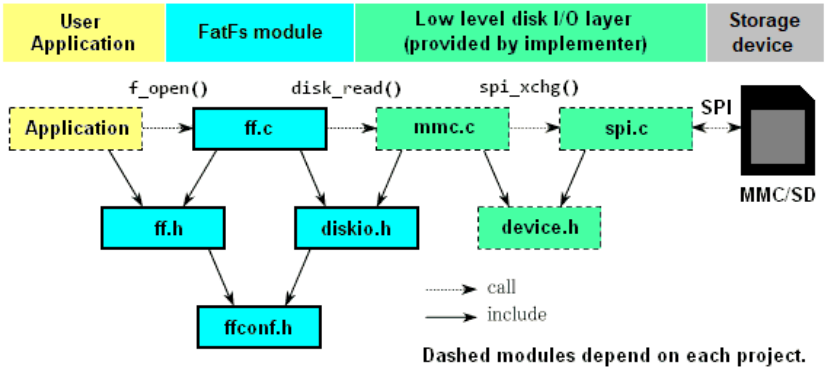
FatFS 的官网的资料介绍非常详细，大家哪里不清楚的都可以找到说明，如果打算使用 FatFS，建议把官方的这些资料了解下：

- FAT 文件系统基础：
  http://elm-chan.org/fsw/ff/00index_e.html
- exFAT 文件系统基础：
  http://elm-chan.org/fsw/ff/00index_e.html
- FatFS API 列表：
  http://elm-chan.org/fsw/ff/00index_e.html
- FatFS 应用笔记（含移植说明）
  http://elm-chan.org/fsw/ff/doc/appnote.html
- FatFS 中 ffconf.h 文件中各种配置说明：
  http://elm-chan.org/fsw/ff/doc/config.html

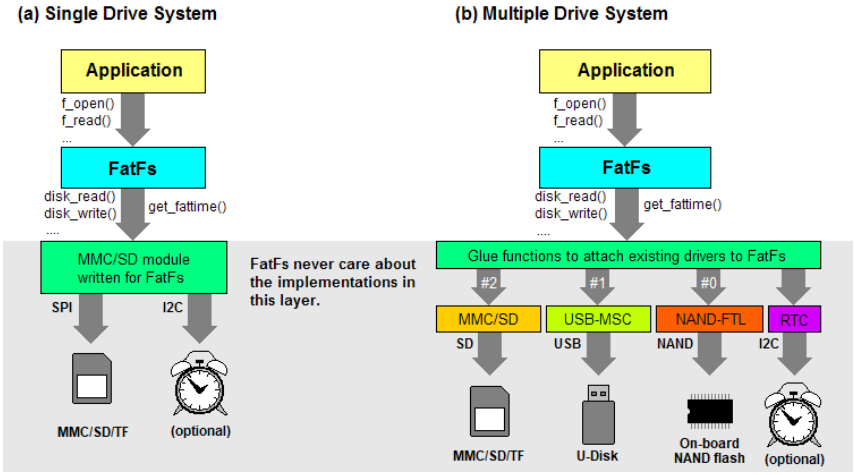
88.7 FatFs 移植步骤

这里将 FatFs 的移植步骤为大家做个说明。

FatFs 各个文件的依赖关系：

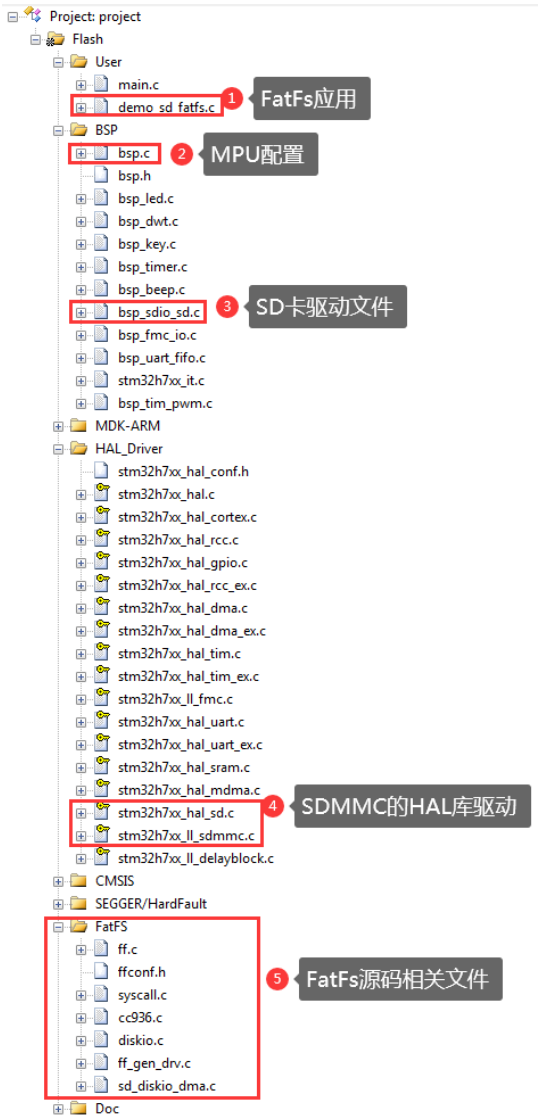


驱动一个磁盘或者多个磁盘的框图：



88.7.1第 1 步，了解整体设计框架

为了方便大家移植，需要大家先对移植好的工程有个整体认识：



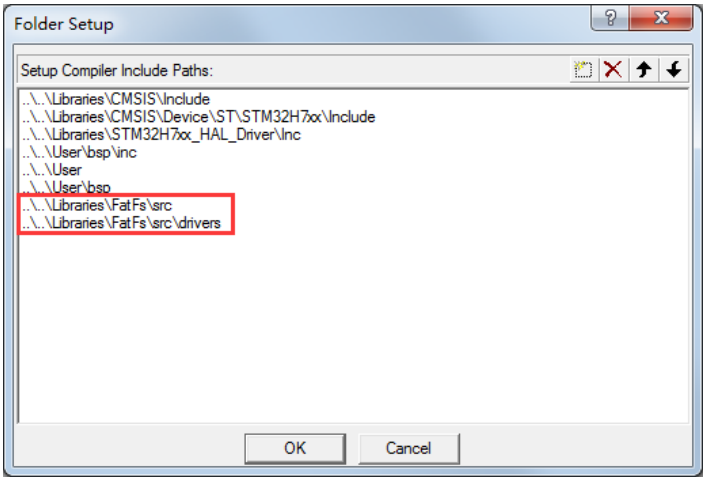
88.7.2 第 2 步，添加 FatFs 和 SDMMC 驱动到工程

本教程前面章节配套的例子都可以作为模板使用，在模板的基础上需要添加 FatFs 文件，SDMMC 驱动文件和 SD 卡驱动文件，大家可以直接从本章教程提供的例子里面复制。

- SD 卡驱动文件 bsp_sdio_sd.c 和 bsp_sdio_sd.h 添加到自己的工程里面，路径不限。
  配套例子是放在\User\bsp\src 和\User\bsp\inc 文件。
- SDMMMC 驱动文件 stm32h7xx_hal_sd.c 和 stm32h7xx_ll_sdmmc.c
  这个是 STM32H7 的 HAL 库自带的。
- FatFs 相关源文件。
  大家可以将所有相关文件都复制到自己的工程里面，配套例子是放在\Libraries\FatFs。

88.7.3 第 3 步，添加工程路径

当前需要添加的两个 FatFs 路径，大家根据自己添加的源文件位置，添加相关路径即可：



88.7.4 第 4 步，配置 GPIO 和时钟

根据大家使用 SDMMC1 或者 SDMMC2 配置相应时钟和 GPIO，当前 V7 板子是用的 SDMMC1:

```
__weak void BSP_SD_MspInit(SD_HandleTypeDef *hsd, void *Params)
{
  GPIO_InitTypeDef gpio_init_structure;

  /* Enable SDIO clock */
  __HAL_RCC_SDMMC1_CLK_ENABLE();

  /* Enable GPIOs clock */
  __HAL_RCC_GPIOB_CLK_ENABLE();
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOD_CLK_ENABLE();
```

```
  gpio_init_structure.Mode      = GPIO_MODE_AF_PP;
  gpio_init_structure.Pull      = GPIO_NOPULL;
  gpio_init_structure.Speed     = GPIO_SPEED_FREQ_VERY_HIGH;

  /* D0(PC8), D1(PC9), D2(PC10), D3(PC11), CK(PC12), CMD(PD2) */
  /* Common GPIO configuration */
  gpio_init_structure.Alternate = GPIO_AF12_SDIO1;

  /* GPIOC configuration */
  gpio_init_structure.Pin = GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 | GPIO_PIN_11 | GPIO_PIN_12;
  HAL_GPIO_Init(GPIOC, &gpio_init_structure);

  /* GPIOD configuration */
  gpio_init_structure.Pin = GPIO_PIN_2;
  HAL_GPIO_Init(GPIOD, &gpio_init_structure);

  __HAL_RCC_SDMMC1_FORCE_RESET();
  __HAL_RCC_SDMMC1_RELEASE_RESET();

  /* NVIC configuration for SDIO interrupts */
  HAL_NVIC_SetPriority(SDMMC1_IRQn, 5, 0);
  HAL_NVIC_EnableIRQ(SDMMC1_IRQn);
}
```

88.7.5第 5 步，MPU 配置

为了方便大家移植测试，我们这里直接关闭 AXI SRAM 的读 Cache 和写 Cache（这样就跟使用 STM32F1 或者 STM32F4 系列里面的 SRAM 一样）。此配置是在 bsp.c 文件的 MPU_Config 函数里面实现：

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

#if 0
    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
```

```
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

#else
    /* 当前是采用下面的配置 */
    /* 配置 AXI SRAM 的 MPU 属性为 NORMAL, NO Read allocate, NO Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
#endif
    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}
```

88.7.6第 6 步，FatFs 的配置文件 ffconf.h 设置

移植阶段，这个里面的配置可以不用动，无需任何修改。需要注意的是我们这里开启了长文件名支持，对应的宏定义：

#define _USE_LFN   3

88.7.7 第 7 步，添加应用代码

这里将 FatFs 大部分操作函数都做了应用，专门整理到了文件 demo_sd_fatfs.c。通过串口命令进行操作，大家可以直接将这个文件添加到自己的工程里面。

另外注意，如果自己的工程里面没有移植我们其它的驱动，可以直接调用 FatFs 的测试函数，比如浏览 SD 根目录文件，可以直接调用函数 ViewRootDir。

88.8 FatFs 应用代码测试

这里将 FatFs 大部分函数都做了测试。注意，所有用到的函数在 FatFs 官网都有详细说明。

88.8.1 注册 SD 卡驱动

注册 SD 卡功能是 ST 简单封装的一个函数，方便用户实现 FatFs 驱动多个磁盘。

代码如下：

```
char DiskPath[4]; /* SD 卡逻辑驱动路径，比盘符 0，就是"0:/" */
/* 注册 SD 卡驱动 */
FATFS_LinkDriver(&SD_Driver, DiskPath);
```

88.8.2 SD 卡文件浏览

SD 卡根目录的文件浏览代码实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: ViewRootDir
*    功能说明: 显示 SD 卡根目录下的文件名
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
extern SD_HandleTypeDef uSdHandle;
static void ViewRootDir(void)
{
    FRESULT result;
    uint32_t cnt = 0;
    FILINFO fno;

    /* 挂载文件系统 */
    result = f_mount(&fs, DiskPath, 0);  /* Mount a logical drive */
    if (result != FR_OK)
    {
        printf("挂载文件系统失败 (%s)\r\n", FR_Table[result]);
    }

    /* 打开根文件夹 */
    result = f_opendir(&DirInf, DiskPath); /* 如果不带参数，则从当前目录开始 */
```

```
    if (result != FR_OK)
    {
        printf("打开根目录失败  (%s)\r\n", FR_Table[result]);
        return;
    }

    printf("属性        |  文件大小 | 短文件名 | 长文件名\r\n");
    for (cnt = 0; ;cnt++)
    {
        result = f_readdir(&DirInf, &FileInf);         /* 读取目录项，索引会自动下移 */
        if (result != FR_OK || FileInf.fname[0] == 0)
        {
            break;
        }

        if (FileInf.fname[0] == '.')
        {
            continue;
        }

        /* 判断是文件还是子目录 */
        if (FileInf.fattrib & AM_DIR)
        {
            printf("(0x%02d)目录  ", FileInf.fattrib);
        }
        else
        {
            printf("(0x%02d)文件  ", FileInf.fattrib);
        }

        f_stat(FileInf.fname, &fno);

        /* 打印文件大小, 最大 4G */
        printf(" %10d", (int)fno.fsize);


        printf("  %s\r\n", (char *)FileInf.fname);     /* 长文件名 */
    }

    /* 打印卡速度信息 */
    if(uSdHandle.SdCard.CardSpeed == CARD_NORMAL_SPEED)
    {
        printf("Normal Speed Card <12.5MB/S, MAX Clock < 25MHz, Spec Version 1.01\r\n");
    }
    else if (uSdHandle.SdCard.CardSpeed == CARD_HIGH_SPEED)
    {
        printf("High Speed Card <25MB/s, MAX Clock < 50MHz, Spec Version 2.00\r\n");
    }
    else if (uSdHandle.SdCard.CardSpeed == CARD_ULTRA_HIGH_SPEED)
    {
        printf("UHS-I SD Card <50MB/S for SDR50, DDR50 Cards, MAX Clock < 50MHz OR 100MHz\r\n");
        printf("UHS-I SD Card <104MB/S for SDR104, MAX Clock < 108MHz, Spec version 3.01\r\n");
    }


    /* 卸载文件系统 */
     f_mount(NULL, DiskPath, 0);
}
```

- f_mount 可以上电后仅调用一次，我们这里是为了测试方式，使用前挂载，使用完毕后卸载。
- 代码里面加入了 SD 卡速度信息打印功能，方便大家了解自己的卡类型。通过查询全局结构体变量 uSdHandle 来实现。
- 文件浏览通过函数 f_readdir 实现。

88.8.3SD 卡创建 txt 文件并写入数据

代码实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: CreateNewFile
*    功能说明: 在 SD 卡创建一个新文件，文件内容填写“www.armfly.com”
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CreateNewFile(void)
{
    FRESULT result;
    uint32_t bw;
    char path[32];


    /* 挂载文件系统 */
    result = f_mount(&fs, DiskPath, 0);            /* Mount a logical drive */
    if (result != FR_OK)
    {
        printf("挂载文件系统失败 (%s)\r\n", FR_Table[result]);
    }

    /* 打开文件 */
    sprintf(path, "%sarmfly.txt", DiskPath);
    result = f_open(&file, path, FA_CREATE_ALWAYS | FA_WRITE);
    if (result == FR_OK)
    {
        printf("armfly.txt 文件打开成功\r\n");
    }
    else
    {
        printf("armfly.txt 文件打开失败  (%s)\r\n", FR_Table[result]);
    }

    /* 写一串数据 */
    result = f_write(&file, FsWriteBuf, strlen(FsWriteBuf), &bw);
    if (result == FR_OK)
    {
        printf("armfly.txt 文件写入成功\r\n");
    }
    else
    {
        printf("armfly.txt 文件写入失败  (%s)\r\n", FR_Table[result]);
    }
```

```
    /* 关闭文件*/
    f_close(&file);

    /* 卸载文件系统 */
    f_mount(NULL, DiskPath, 0);
}
```

- f_mount 可以上电后仅调用一次，我们这里是为了测试方式，使用前挂载，使用完毕后卸载。
- 函数 f_open 用来创建并打开文件。
- 函数 f_write 用来写入数据。
- 函数 f_close 用来关闭文件，注意调用完函数 f_write 后，内容还没有实际写入到 SD 卡中，调用了 f_close 后，数据才真正的写入到 SD 卡。当然也可以调用函数 f_sync，内容也会实际的写入。

88.8.4SD 卡文件读取

代码实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: ReadFileData
*    功能说明: 读取文件 armfly.txt 前 128 个字符，并打印到串口
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
static void ReadFileData(void)
{
    FRESULT result;
    uint32_t bw;
    char path[64];


    /* 挂载文件系统 */
    result = f_mount(&fs, DiskPath, 0);            /* Mount a logical drive */
    if (result != FR_OK)
    {
        printf("挂载文件系统失败 (%s)\r\n", FR_Table[result]);
    }

    /* 打开文件 */
    sprintf(path, "%sarmfly.txt", DiskPath);
    result = f_open(&file, path, FA_OPEN_EXISTING | FA_READ);
    if (result !=  FR_OK)
    {
        printf("Don't Find File : armfly.txt\r\n");
        return;
    }

    /* 读取文件 */
    result = f_read(&file, FsReadBuf, sizeof(FsReadBuf), &bw);
    if (bw > 0)
    {
        FsReadBuf[bw] = 0;
```

```
        printf("\r\narmfly.txt 文件内容 : \r\n%s\r\n", FsReadBuf);
    }
    else
    {
        printf("\r\narmfly.txt 文件内容 : \r\n");
    }

    /* 关闭文件*/
    f_close(&file);

    /* 卸载文件系统 */
    f_mount(NULL, DiskPath, 0);
}
```

- f_mount 可以上电后仅调用一次，我们这里是为了测试方式，使用前挂载，使用完毕后卸载。
- 函数 f_open 用来打开文件。
- 函数 f_read 用来读取文件中的内容。
- 函数 f_close 用来关闭打开的文件。

88.8.5 SD 卡创建文件夹

代码实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: CreateDir
*    功能说明: 在 SD 卡根目录创建 Dir1 和 Dir2 目录，在 Dir1 目录下创建子目录 Dir1_1
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CreateDir(void)
{
    FRESULT result;
    char path[64];


    /* 挂载文件系统 */
    result = f_mount(&fs, DiskPath, 0);            /* Mount a logical drive */
    if (result != FR_OK)
    {
        printf("挂载文件系统失败 (%s)\r\n", FR_Table[result]);
    }

    /* 创建目录/Dir1 */
    sprintf(path, "%sDir1", DiskPath);
    result = f_mkdir(path);
    if (result == FR_OK)
    {
        printf("f_mkdir Dir1 Ok\r\n");
    }
    else if (result == FR_EXIST)
    {
        printf("Dir1 目录已经存在(%d)\r\n", result);
```

```
    }
    else
    {
        printf("f_mkdir Dir1 失败 (%s)\r\n", FR_Table[result]);
        return;
    }

    /* 创建目录/Dir2 */
    sprintf(path, "%sDir2", DiskPath);
    result = f_mkdir(path);
    if (result == FR_OK)
    {
        printf("f_mkdir Dir2 Ok\r\n");
    }
    else if (result == FR_EXIST)
    {
        printf("Dir2 目录已经存在(%d)\r\n", result);
    }
    else
    {
        printf("f_mkdir Dir2 失败 (%s)\r\n", FR_Table[result]);
        return;
    }

    /* 创建子目录 /Dir1/Dir1_1     注意：创建子目录 Dir1_1 时，必须先创建好 Dir1 */
    sprintf(path, "%sDir1/Dir1_1", DiskPath);
    result = f_mkdir(path); /* */
    if (result == FR_OK)
    {
        printf("f_mkdir Dir1_1 成功\r\n");
    }
    else if (result == FR_EXIST)
    {
        printf("Dir1_1 目录已经存在 (%d)\r\n", result);
    }
    else
    {
        printf("f_mkdir Dir1_1 失败 (%s)\r\n", FR_Table[result]);
        return;
    }

    /* 卸载文件系统 */
    f_mount(NULL, DiskPath, 0);
}
```

- f_mount 可以上电后仅调用一次，我们这里是为了测试方式，使用前挂载，使用完毕后卸载。
- 创建目录通过函数 f_mkdir。

88.8.6SD 卡文件和文件夹删除

代码实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: DeleteDirFile
*    功能说明: 删除 SD 卡根目录下的 armfly.txt 文件和 Dir1, Dir2 目录
```

```
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
static void DeleteDirFile(void)
{
    FRESULT result;
    uint8_t i;
    char path[64];

    /* 挂载文件系统 */
    result = f_mount(&fs, DiskPath, 0);            /* Mount a logical drive */
    if (result != FR_OK)
    {
        printf("挂载文件系统失败 (%s)\r\n", FR_Table[result]);
    }

    /* 删除目录/Dir1 【因为还存在目录非空（存在子目录），所以这次删除会失败】*/
    sprintf(path, "%sDir1", DiskPath);
    result = f_unlink(path);
    if (result == FR_OK)
    {
        printf("删除目录 Dir1 成功\r\n");
    }
    else if (result == FR_NO_FILE)
    {
        printf("没有发现文件或目录 :%s\r\n", "/Dir1");
    }
    else
    {
        printf("删除 Dir1 失败(错误代码 = %d) 文件只读或目录非空\r\n", result);
    }

    /* 先删除目录/Dir1/Dir1_1 */
    sprintf(path, "%sDir1/Dir1_1", DiskPath);
    result = f_unlink(path);
    if (result == FR_OK)
    {
        printf("删除子目录/Dir1/Dir1_1 成功\r\n");
    }
    else if ((result == FR_NO_FILE) || (result == FR_NO_PATH))
    {
        printf("没有发现文件或目录 :%s\r\n", "/Dir1/Dir1_1");
    }
    else
    {
        printf("删除子目录/Dir1/Dir1_1 失败(错误代码 = %d) 文件只读或目录非空\r\n", result);
    }

    /* 先删除目录/Dir1 */
    sprintf(path, "%sDir1", DiskPath);
    result = f_unlink(path);
    if (result == FR_OK)
    {
        printf("删除目录 Dir1 成功\r\n");
    }
    else if (result == FR_NO_FILE)
    {
```

```
        printf("没有发现文件或目录 :%s\r\n", "/Dir1");
    }
    else
    {
        printf("删除 Dir1 失败(错误代码 = %d) 文件只读或目录非空\r\n", result);
    }

    /* 删除目录/Dir2 */
    sprintf(path, "%sDir2", DiskPath);
    result = f_unlink(path);
    if (result == FR_OK)
    {
        printf("删除目录 Dir2 成功\r\n");
    }
    else if (result == FR_NO_FILE)
    {
        printf("没有发现文件或目录 :%s\r\n", "/Dir2");
    }
    else
    {
        printf("删除 Dir2 失败(错误代码 = %d) 文件只读或目录非空\r\n", result);
    }

    /* 删除文件 armfly.txt */
    sprintf(path, "%sarmfly.txt", DiskPath);
    result = f_unlink(path);
    if (result == FR_OK)
    {
        printf("删除文件 armfly.txt 成功\r\n");
    }
    else if (result == FR_NO_FILE)
    {
        printf("没有发现文件或目录 :%s\r\n", "armfly.txt");
    }
    else
    {
        printf("删除 armfly.txt 失败(错误代码 = %d) 文件只读或目录非空\r\n", result);
    }

    /* 删除文件 speed1.txt */
    for (i = 0; i < 20; i++)
    {
        sprintf(path, "%sSpeed%02d.txt", DiskPath, i);/* 每写 1 次，序号递增 */
        result = f_unlink(path);
        if (result == FR_OK)
        {
            printf("删除文件%s 成功\r\n", path);
        }
        else if (result == FR_NO_FILE)
        {
            printf("没有发现文件:%s\r\n", path);
        }
        else
        {
            printf("删除%s 文件失败(错误代码 = %d) 文件只读或目录非空\r\n", path, result);
        }
    }
```

```
    /* 卸载文件系统 */
    f_mount(NULL, DiskPath, 0);
}
```

- f_mount 可以上电后仅调用一次，我们这里是为了测试方式，使用前挂载，使用完毕后卸载。
- 文件夹和文件的删除都是通过函数 f_unlink 实现，这里注意一点，删除文件夹时，只有文件夹中的内容为空时，才可以删除文件夹。

88.8.7SD 卡读写速度测试

代码实现如下，主要是方便大家测试 SD 卡的读写性能。

```
/*
*********************************************************************************************************
*    函 数 名: WriteFileTest
*    功能说明: 测试文件读写速度
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
static void WriteFileTest(void)
{
    FRESULT result;
    char path[64];
    uint32_t bw;
    uint32_t i,k;
    uint32_t runtime1,runtime2,timelen;
    uint8_t err = 0;
    static uint8_t s_ucTestSn = 0;


    for (i = 0; i < sizeof(g_TestBuf); i++)
    {
        g_TestBuf[i] = (i / 512) + '0';
    }

    /* 挂载文件系统 */
    result = f_mount(&fs, DiskPath, 0);            /* Mount a logical drive */
    if (result != FR_OK)
    {
        printf("挂载文件系统失败 (%s)\r\n", FR_Table[result]);
    }

    /* 打开文件 */
    sprintf(path, "%sSpeed%02d.txt", DiskPath, s_ucTestSn++); /* 每写 1 次，序号递增 */
    result = f_open(&file, path, FA_CREATE_ALWAYS | FA_WRITE);

    /* 写一串数据 */
    printf("开始写文件%s %dKB ...\r\n", path, TEST_FILE_LEN / 1024);

    runtime1 = bsp_GetRunTime();     /* 读取系统运行时间 */
    for (i = 0; i < TEST_FILE_LEN / BUF_SIZE; i++)
    {
        result = f_write(&file, g_TestBuf, sizeof(g_TestBuf), &bw);
        if (result == FR_OK)
```

```
        {
            if (((i + 1) % 8) == 0)
            {
                printf(".");
            }
        }
        else
        {
            err = 1;
            printf("%s 文件写失败\r\n", path);
            break;
        }
    }
    runtime2 = bsp_GetRunTime();    /* 读取系统运行时间 */

    if (err == 0)
    {
        timelen = (runtime2 - runtime1);
        printf("\r\n  写耗时 : %dms   平均写速度 : %dB/S (%dKB/S)\r\n",
            timelen,
            (TEST_FILE_LEN * 1000) / timelen,
            ((TEST_FILE_LEN / 1024) * 1000) / timelen);
    }

    f_close(&file);         /* 关闭文件*/


    /* 开始读文件测试 */
    result = f_open(&file, path, FA_OPEN_EXISTING | FA_READ);
    if (result !=  FR_OK)
    {
        printf("没有找到文件: %s\r\n", path);
        return;
    }

    printf("开始读文件 %dKB ...\r\n", TEST_FILE_LEN / 1024);

    runtime1 = bsp_GetRunTime();    /* 读取系统运行时间 */
    for (i = 0; i < TEST_FILE_LEN / BUF_SIZE; i++)
    {
        result = f_read(&file, g_TestBuf, sizeof(g_TestBuf), &bw);
        if (result == FR_OK)
        {
            if (((i + 1) % 8) == 0)
            {
                printf(".");
            }

            /* 比较写入的数据是否正确，此语句会导致读卡速度结果降低到 3.5MBytes/S */
            for (k = 0; k < sizeof(g_TestBuf); k++)
            {
                if (g_TestBuf[k] != (k / 512) + '0')
                {
                    err = 1;
                    printf("Speed1.txt 文件读成功，但是数据出错\r\n");
                    break;
                }
            }
```

```
            if (err == 1)
            {
                break;
            }
        }
        else
        {
            err = 1;
            printf("Speed1.txt 文件读失败\r\n");
            break;
        }
    }

    runtime2 = bsp_GetRunTime();    /* 读取系统运行时间 */

    if (err == 0)
    {
        timelen = (runtime2 - runtime1);
        printf("\r\n  读耗时 : %dms   平均读速度 : %dB/S (%dKB/S)\r\n", timelen,
            (TEST_FILE_LEN * 1000) / timelen, ((TEST_FILE_LEN / 1024) * 1000) / timelen);
    }

    /* 关闭文件*/
    f_close(&file);

    /* 卸载文件系统 */
    f_mount(NULL, DiskPath, 0);
}
```

- f_mount 可以上电后仅调用一次，我们这里是为了测试方式，使用前挂载，使用完毕后卸载。
- 为了实现更高性能的测试，大家可以加大宏定义

  #define BUF_SIZE (4*1024) /* 每次读写 SD 卡的最大数据长度 */

  设置的缓冲大小，比如设置为 64KB 进行测试。

88.9 FatFs 移植接口文件 diskio.c 说明

这里将 FatFs 的底层接口文件 diskio.c 的实现为大家简单做个说明。

88.9.1 磁盘状态函数 disk_status

代码如下：

```
/**
  * @brief  Gets Disk Status
  * @param  pdrv: Physical drive number (0..)
  * @retval DSTATUS: Operation status
  */
DSTATUS disk_status (
    BYTE pdrv     /* Physical drive number to identify the drive */
)
{
  DSTATUS stat;
```

```
  stat = disk.drv[pdrv]->disk_status(disk.lun[pdrv]);
  return stat;
}
```

实际对应的函数在文件 sd_diskio_dma.c

```
/**
  * @brief  Gets Disk Status
  * @param  lun : not used
  * @retval DSTATUS: Operation status
  */
DSTATUS SD_status(BYTE lun)
{
  return SD_CheckStatus(lun);
}

static DSTATUS SD_CheckStatus(BYTE lun)
{
  Stat = STA_NOINIT;

  if(BSP_SD_GetCardState() == MSD_OK)
  {
    Stat &= ~STA_NOINIT;
  }

  return Stat;
}
```

88.9.2 磁盘初始化函数 disk_initialize

代码如下：

```
/**
  * @brief  Initializes a Drive
  * @param  pdrv: Physical drive number (0..)
  * @retval DSTATUS: Operation status
  */
DSTATUS disk_initialize (
    BYTE pdrv               /* Physical drive nmuber to identify the drive */
)
{
  DSTATUS stat = RES_OK;

  if(disk.is_initialized[pdrv] == 0)
  {
    disk.is_initialized[pdrv] = 1;
    stat = disk.drv[pdrv]->disk_initialize(disk.lun[pdrv]);
  }
  return stat;
}
```

实际对应的函数在文件 sd_diskio_dma.c：

```
/**
  * @brief  Initializes a Drive
  * @param  lun : not used
  * @retval DSTATUS: Operation status
  */
DSTATUS SD_initialize(BYTE lun)
```

```
{
#if !defined(DISABLE_SD_INIT)

  if(BSP_SD_Init() == MSD_OK)
  {
    Stat = SD_CheckStatus(lun);
  }

#else
  Stat = SD_CheckStatus(lun);
#endif
  return Stat;
}
```

88.9.3磁盘读函数 disk_read

代码如下：

```
/**
  * @brief  Reads Sector(s)
  * @param  pdrv: Physical drive number (0..)
  * @param  *buff: Data buffer to store read data
  * @param  sector: Sector address (LBA)
  * @param  count: Number of sectors to read (1..128)
  * @retval DRESULT: Operation result
  */
DRESULT disk_read (
    BYTE pdrv,          /* Physical drive nmuber to identify the drive */
    BYTE *buff,         /* Data buffer to store read data */
    DWORD sector,          /* Sector address in LBA */
    UINT count          /* Number of sectors to read */
)
{
  DRESULT res;

  res = disk.drv[pdrv]->disk_read(disk.lun[pdrv], buff, sector, count);
  return res;
}
```

实际对应的函数在文件 sd_diskio_dma.c：

下面代码中最关键的处理是形参 buff 的 4 字节对齐问题（SDMMC 自带的 IDMA 需要 4 字节对齐），如果 buff 地址是 4 字节对齐的，不做处理，如果不是对齐，通过复制到一个 4 字节对齐的缓冲里面做 DMA 传递。这个是理解下面代码的关键。

```
/**
  * @brief  Reads Sector(s)
  * @param  lun : not used
  * @param  *buff: Data buffer to store read data
  * @param  sector: Sector address (LBA)
  * @param  count: Number of sectors to read (1..128)
  * @retval DRESULT: Operation result
  */
DRESULT SD_read(BYTE lun, BYTE *buff, DWORD sector, UINT count)
{
    DRESULT res = RES_ERROR;
```

```
uint32_t timeout;
ReadStatus = 0;

if (!((uint32_t)buff & 0x3))
{
    if(BSP_SD_ReadBlocks_DMA((uint32_t*)buff,
                             (uint32_t) (sector),
                             count) == MSD_OK)
    {
        /* Wait that the reading process is completed or a timeout occurs */
        timeout = HAL_GetTick();
        while((ReadStatus == 0) && ((HAL_GetTick() - timeout) < SD_TIMEOUT))
        {
        }

        /* incase of a timeout return error */
        if (ReadStatus == 0)
        {
            res = RES_ERROR;
        }
        else
        {
            ReadStatus = 0;
            timeout = HAL_GetTick();

            while((HAL_GetTick() - timeout) < SD_TIMEOUT)
            {
                if (BSP_SD_GetCardState() == SD_TRANSFER_OK)
                {
                    res = RES_OK;

                    #if (ENABLE_SD_DMA_CACHE_MAINTENANCE_READ == 1)
                       SCB_CleanInvalidateDCache();
                    #endif
                    break;
                }
            }
        }
    }
}
else
{
    uint8_t ret;
    int i;

    for (i = 0; i < count; i++)
    {

        ret = BSP_SD_ReadBlocks_DMA((uint32_t*)scratch, (uint32_t)sector++, 1);

        if(ret == MSD_OK)
        {
            /* Wait that the reading process is completed or a timeout occurs */
            timeout = HAL_GetTick();
            while((ReadStatus == 0) && ((HAL_GetTick() - timeout) < SD_TIMEOUT))
            {

            }
```

```
                /* incase of a timeout return error */
                if (ReadStatus == 0)
                {
                    break;
                }
                else
                {
                    ReadStatus = 0;
                    timeout = HAL_GetTick();

                    while((HAL_GetTick() - timeout) < SD_TIMEOUT)
                    {
                        if (BSP_SD_GetCardState() == SD_TRANSFER_OK)
                        {
                            #if (ENABLE_SD_DMA_CACHE_MAINTENANCE_READ == 1)
                                SCB_CleanInvalidateDCache();
                            #endif

                            memcpy(buff, scratch, BLOCKSIZE);
                            buff += BLOCKSIZE;

                            break;
                        }
                    }
                }
            }
            else
            {
                break;
            }
        }
        if ((i == count) && (ret == MSD_OK))
        {
           res = RES_OK;
        }
    }
    return res;
}
```

88.9.4磁盘写函数 disk_write

代码如下：

```
/**
  * @brief  Writes Sector(s)
  * @param  pdrv: Physical drive number (0..)
  * @param  *buff: Data to be written
  * @param  sector: Sector address (LBA)
  * @param  count: Number of sectors to write (1..128)
  * @retval DRESULT: Operation result
  */
DRESULT disk_write (
    BYTE pdrv,          /* Physical drive nmuber to identify the drive */
    const BYTE *buff,   /* Data to be written */
    DWORD sector,       /* Sector address in LBA */
    UINT count          /* Number of sectors to write */
```

```
)
{
  DRESULT res;

  res = disk.drv[pdrv]->disk_write(disk.lun[pdrv], buff, sector, count);
  return res;
}
```

实际对应的函数在文件 sd_diskio_dma.c：

下面代码中最关键的处理是形参 buff 的 4 字节对齐问题（SDMMC 自带的 IDMA 需要 4 字节对齐），如果 buff 地址是 4 字节对齐的，不做处理，如果不是对齐，通过复制到一个 4 字节对齐的缓冲里面做 DMA 传递。这个是理解下面代码的关键。

```
/**
  * @brief  Writes Sector(s)
  * @param  lun : not used
  * @param  *buff: Data to be written
  * @param  sector: Sector address (LBA)
  * @param  count: Number of sectors to write (1..128)
  * @retval DRESULT: Operation result
  */
#if _USE_WRITE == 1
DRESULT SD_write(BYTE lun, const BYTE *buff, DWORD sector, UINT count)
{
    DRESULT res = RES_ERROR;
    uint32_t timeout;
    WriteStatus = 0;

#if (ENABLE_SD_DMA_CACHE_MAINTENANCE_WRITE == 1)
   SCB_CleanInvalidateDCache();
#endif

    if (!((uint32_t)buff & 0x3))
    {
        if(BSP_SD_WriteBlocks_DMA((uint32_t*)buff,
                                  (uint32_t)(sector),
                                  count) == MSD_OK)
        {
            /* Wait that writing process is completed or a timeout occurs */
            timeout = HAL_GetTick();
            while((WriteStatus == 0) && ((HAL_GetTick() - timeout) < SD_TIMEOUT))
            {
            }

            /* incase of a timeout return error */
            if (WriteStatus == 0)
            {
                res = RES_ERROR;
            }
            else
            {
                WriteStatus = 0;
                timeout = HAL_GetTick();

                while((HAL_GetTick() - timeout) < SD_TIMEOUT)
                {
                    if (BSP_SD_GetCardState() == SD_TRANSFER_OK)
```

```
                {
                    res = RES_OK;
                    break;
                }
            }
        }
    }
}
else
{
    int i;
    uint8_t ret;

    for (i = 0; i < count; i++)
    {
        WriteStatus = 0;

        memcpy((void *)scratch, (void *)buff, BLOCKSIZE);
        buff += BLOCKSIZE;

        ret = BSP_SD_WriteBlocks_DMA((uint32_t*)scratch, (uint32_t)sector++, 1);
        if(ret == MSD_OK)
        {
            /* Wait that writing process is completed or a timeout occurs */

            timeout = HAL_GetTick();
            while((WriteStatus == 0) && ((HAL_GetTick() - timeout) < SD_TIMEOUT))
            {
            }

            /* incase of a timeout return error */
            if (WriteStatus == 0)
            {
                break;
            }
            else
            {
                WriteStatus = 0;
                timeout = HAL_GetTick();

                while((HAL_GetTick() - timeout) < SD_TIMEOUT)
                {
                    if (BSP_SD_GetCardState() == SD_TRANSFER_OK)
                    {
                        break;
                    }
                }
            }
        }
        else
        {
            break;
        }
    }

    if ((i == count) && (ret == MSD_OK))
    {
        res = RES_OK;
```

```
        }
    }

    return res;
}
```

88.9.5磁盘 I/O 控制函数 disk_ioctl

代码如下：

```
/**
  * @brief  I/O control operation
  * @param  pdrv: Physical drive number (0..)
  * @param  cmd: Control code
  * @param  *buff: Buffer to send/receive control data
  * @retval DRESULT: Operation result
  */
#if _USE_IOCTL == 1
DRESULT disk_ioctl (
    BYTE pdrv,          /* Physical drive nmuber (0..) */
    BYTE cmd,      /* Control code */
    void *buff          /* Buffer to send/receive control data */
)
{
  DRESULT res;

  res = disk.drv[pdrv]->disk_ioctl(disk.lun[pdrv], cmd, buff);
  return res;
}
#endif /* _USE_IOCTL == 1 */
```

实际对应的函数在文件 sd_diskio_dma.c

特别注意，如果大家要调用 FatFs 的 API 格式化 SD 卡，此函数比较重要。下面几个 cmd 一定实现：

```
/**
  * @brief  I/O control operation
  * @param  lun : not used
  * @param  cmd: Control code
  * @param  *buff: Buffer to send/receive control data
  * @retval DRESULT: Operation result
  */
#if _USE_IOCTL == 1
DRESULT SD_ioctl(BYTE lun, BYTE cmd, void *buff)
{
  DRESULT res = RES_ERROR;
  BSP_SD_CardInfo CardInfo;

  if (Stat & STA_NOINIT) return RES_NOTRDY;

  switch (cmd)
  {
  /* Make sure that no pending write process */
  case CTRL_SYNC :
    res = RES_OK;
    break;

  /* Get number of sectors on the disk (DWORD) */
```

```
  case GET_SECTOR_COUNT :
    BSP_SD_GetCardInfo(&CardInfo);
    *(DWORD*)buff = CardInfo.LogBlockNbr;
    res = RES_OK;
    break;

  /* Get R/W sector size (WORD) */
  case GET_SECTOR_SIZE :
    BSP_SD_GetCardInfo(&CardInfo);
    *(WORD*)buff = CardInfo.LogBlockSize;
    res = RES_OK;
    break;

  /* Get erase block size in unit of sector (DWORD) */
  case GET_BLOCK_SIZE :
    BSP_SD_GetCardInfo(&CardInfo);
    *(DWORD*)buff = CardInfo.LogBlockSize / SD_DEFAULT_BLOCK_SIZE;
     res = RES_OK;
    break;

  default:
    res = RES_PARERR;
  }

  return res;
}
#endif /* _USE_IOCTL == 1 */
```

88.9.6 RTC 时间获取函数 get_fattime

我们这里未使用这个函数，此函数的作用是用户创建文件时，可以将创建文件时间设置为此函数的获取值

```
/**
  * @brief  Gets Time from RTC
  * @param  None
  * @retval Time in DWORD
  */
__weak DWORD get_fattime (void)
{
  return 0;
}
```

88.10 SDMMC 自带 IDMA 的 4 字节对齐问题（重要）

由于本章教程配套例子使用了 SDMMC 自带的 IDMA，所以也专门做了 4 字节对齐处理。处理思路就是底层的读写函数里面如果地址是 4 字节对齐的，不做处理，如果不是对齐，通过复制到一个 4 字节对齐的缓冲里面做 DMA 传递。

其实有个更简单，性能也最高的解决办法，核心思想如下（ffconf.h 文件里面设置的扇区大小基本都是 512 字节）：

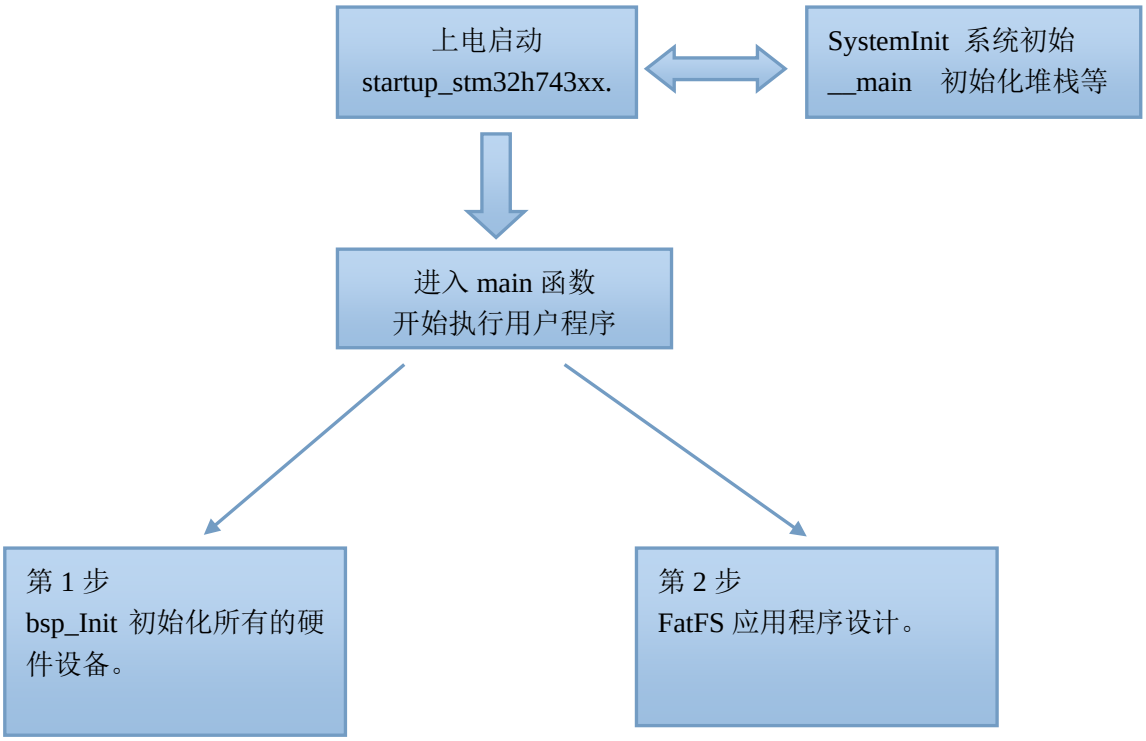
- 当要写入和读取的数据小于扇区大小时，会直接使用 FATFS 结构体里面的数组 win[_MAX_SS]做 DMA 写操作到，正好 1 个扇区大小。由于数组 win[_MAX_SS]的地址是 4 字节对齐的，所以无需做处理。
- 当要写入和读取的数据大于等于扇区大小时，扇区整数倍的地方将直接使用用户提供的收发缓冲区发送，而不足一个扇区的地方将使用 FATFS 结构体里面的数组。这种情况下用户要做的就是直接定义个 4 字节对齐的读写缓冲区即可。

针对本章教程配套的例子，我们直接做了 32 字节对齐，同时也方便了 Cache 处理：

```
ALIGN_32BYTES(char FsReadBuf[1024]);
ALIGN_32BYTES(char FsWriteBuf[1024]) = {"FatFS Write Demo \r\n www.armfly.com \r\n"};
ALIGN_32BYTES(uint8_t g_TestBuf[BUF_SIZE]);
```

88.11 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。
- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED 和串口。

- 第 2 步，FatFs 应用程序设计部分。

88.12 实验例程说明（MDK）

配套例子：

V7-025_FatFS 文件系统例子（SD 卡 V1.1）

实验目的：

2. 学习 SD 卡的 FatFS 移植实现。

实验内容：

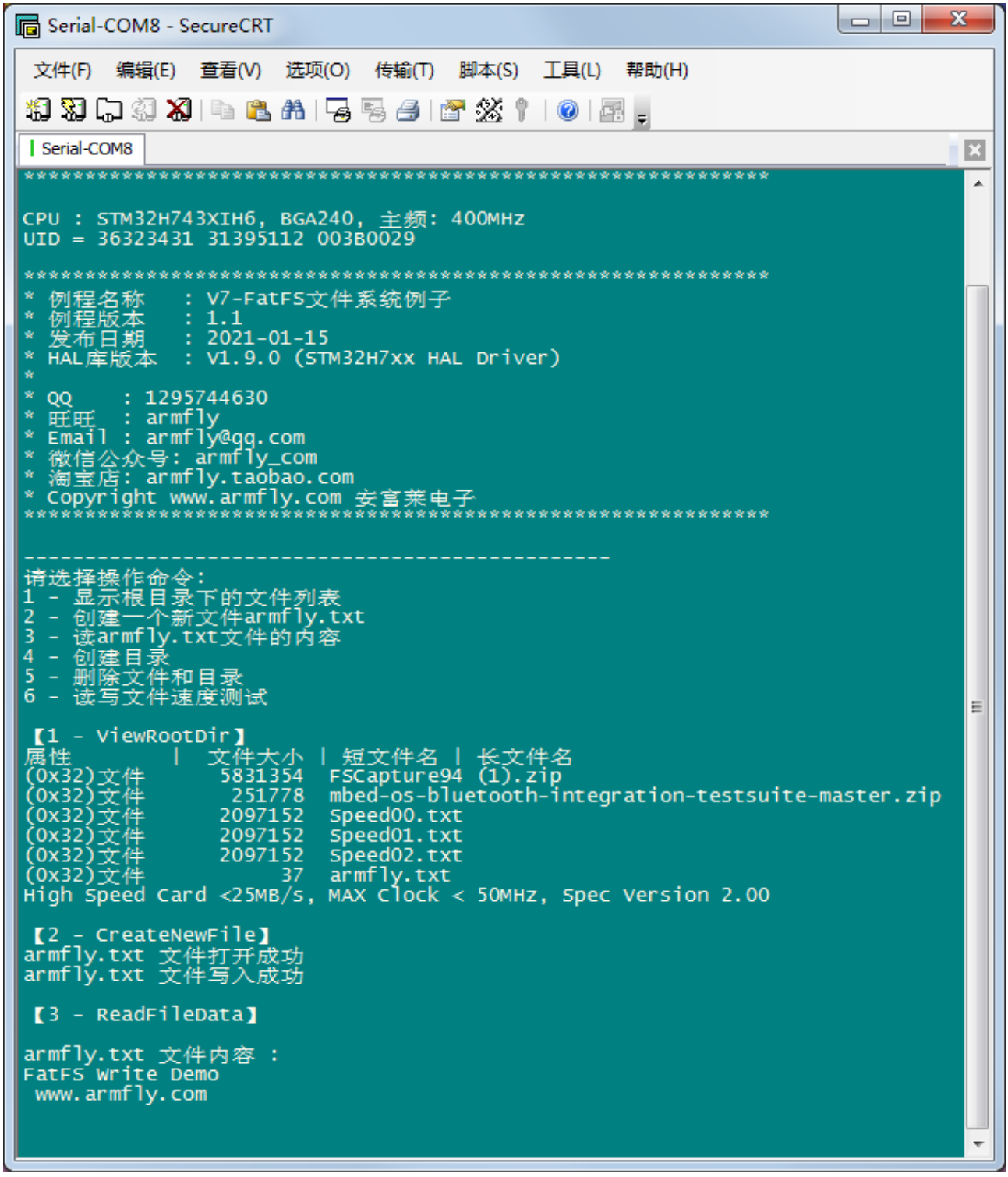
3. 上电启动了一个软件定时器，每 100ms 翻转一次 LED2。
4. V7 开发板的 SD 卡接口是用的 SDMMC1，而这个接口仅支持 AXI SRAM 区访问，其它 SRAM 和 TCP 均不支持。

实验操作：

1. 测试前务必将 SD 卡插入到开发板左上角的卡座中。
2. 支持以下 6 个功能，用户通过电脑端串口软件发送数字 1-6 给开发板即可
3. printf("1 - 显示根目录下的文件列表\r\n");
4. printf("2 - 创建一个新文件 armfly.txt\r\n");
5. printf("3 - 读 armfly.txt 文件的内容\r\n");
6. printf("4 - 创建目录\r\n");
7. printf("5 - 删除文件和目录\r\n");
8. printf("6 - 读写文件速度测试\r\n");

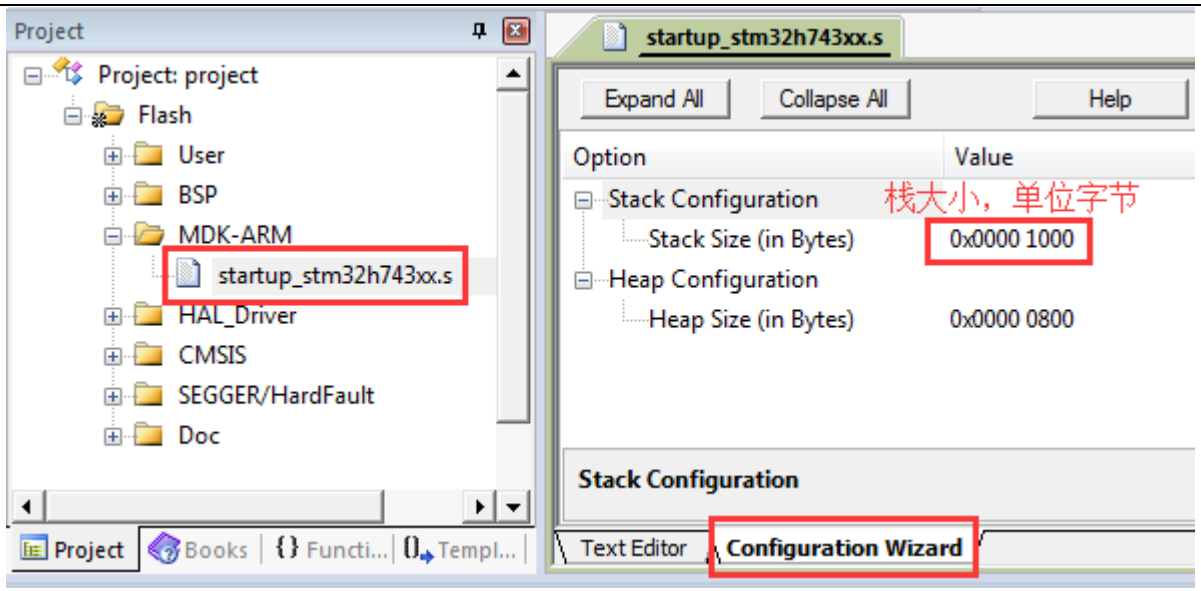
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

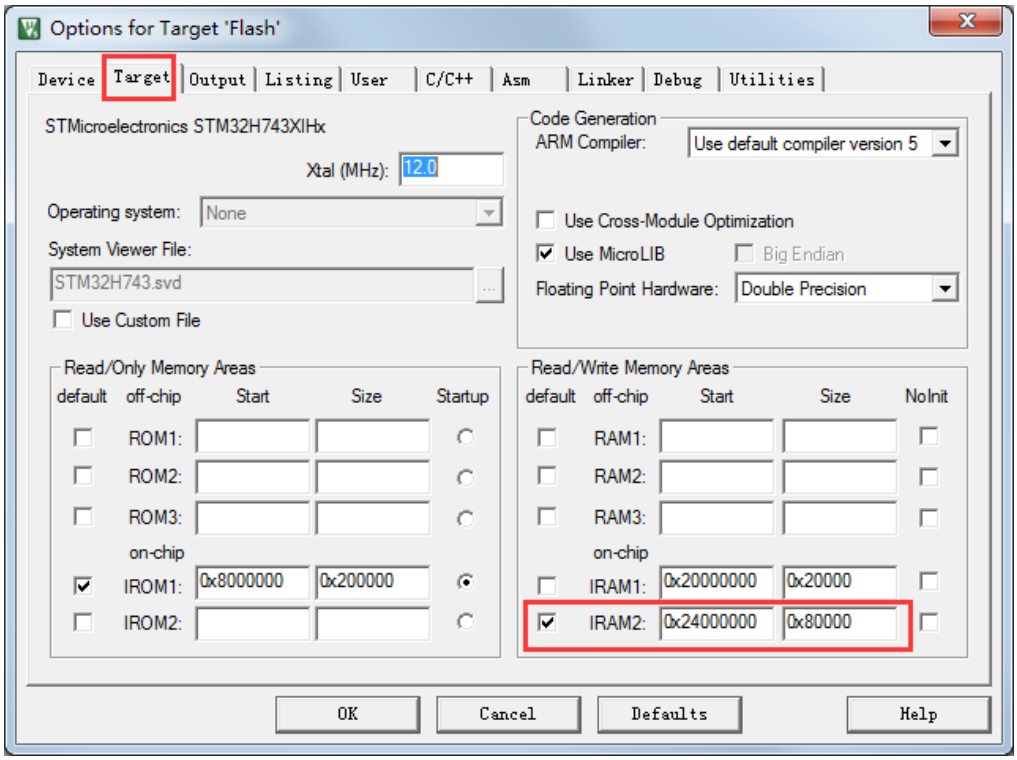


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 AXI SRAM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
```

```
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

#if 0
```

```
    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

#else
    /* 当前是采用下面的配置 */
    /* 配置 AXI SRAM 的 MPU 属性为 NORMAL, NO Read allocate, NO Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
#endif
    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
```

```
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能:**

主程序实现如下操作:

- 上电启动了一个软件定时器，每 100ms 翻转一次 LED2。
- 支持以下 6 个功能，用户通过电脑端串口软件发送数字给开发板即可:
- 1 - 显示根目录下的文件列表
- 2 - 创建一个新文件 armfly.txt
- 3 - 读 armfly.txt 文件的内容
- 4 - 创建目录
- 5 - 删除文件和目录
- 6 - 读写文件速度测试

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */

    DemoFatFS();    /* SD 卡测试 */
}

/*
*********************************************************************************************************
*    函 数 名: DemoFatFS
*    功能说明: FatFS 文件系统演示主程序
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoFatFS(void)
{
    uint8_t cmd;

    /* 打印命令列表，用户可以通过串口操作指令 */
    DispMenu();
```

```
    /* 注册 SD 卡驱动 */
    FATFS_LinkDriver(&SD_Driver, DiskPath);

    bsp_StartAutoTimer(0, 500); /* 启动 1 个 500ms 的自动重装的定时器 */

    while (1)
    {

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 500ms 进来一次 */
            bsp_LedToggle(2);
        }

        if (comGetChar(COM1, &cmd)) /* 从串口读入一个字符(非阻塞方式) */
        {
            printf("\r\n");
            switch (cmd)
            {
                case '1':
                    printf("【1 - ViewRootDir】\r\n");
                    ViewRootDir();          /* 显示 SD 卡根目录下的文件名 */
                    break;

                case '2':
                    printf("【2 - CreateNewFile】\r\n");
                    CreateNewFile();    /* 创建一个新文件,写入一个字符串 */
                    break;

                case '3':
                    printf("【3 - ReadFileData】\r\n");
                    ReadFileData();         /* 读取根目录下 armfly.txt 的内容 */
                    break;

                case '4':
                    printf("【4 - CreateDir】\r\n");
                    CreateDir();        /* 创建目录 */
                    break;

                case '5':
                    printf("【5 - DeleteDirFile】\r\n");
                    DeleteDirFile();    /* 删除目录和文件 */
                    break;

                case '6':
                    printf("【6 - TestSpeed】\r\n");
                    WriteFileTest();    /* 速度测试 */
                    break;

                default:
                    DispMenu();
                    break;
            }
        }
    }
}
```

88.13 实验例程说明（IAR）

配套例子：

V7-025_FatFS 文件系统例子（SD 卡 V1.1）

实验目的：

1. 学习 SD 卡的 FatFS 移植实现。

实验内容：

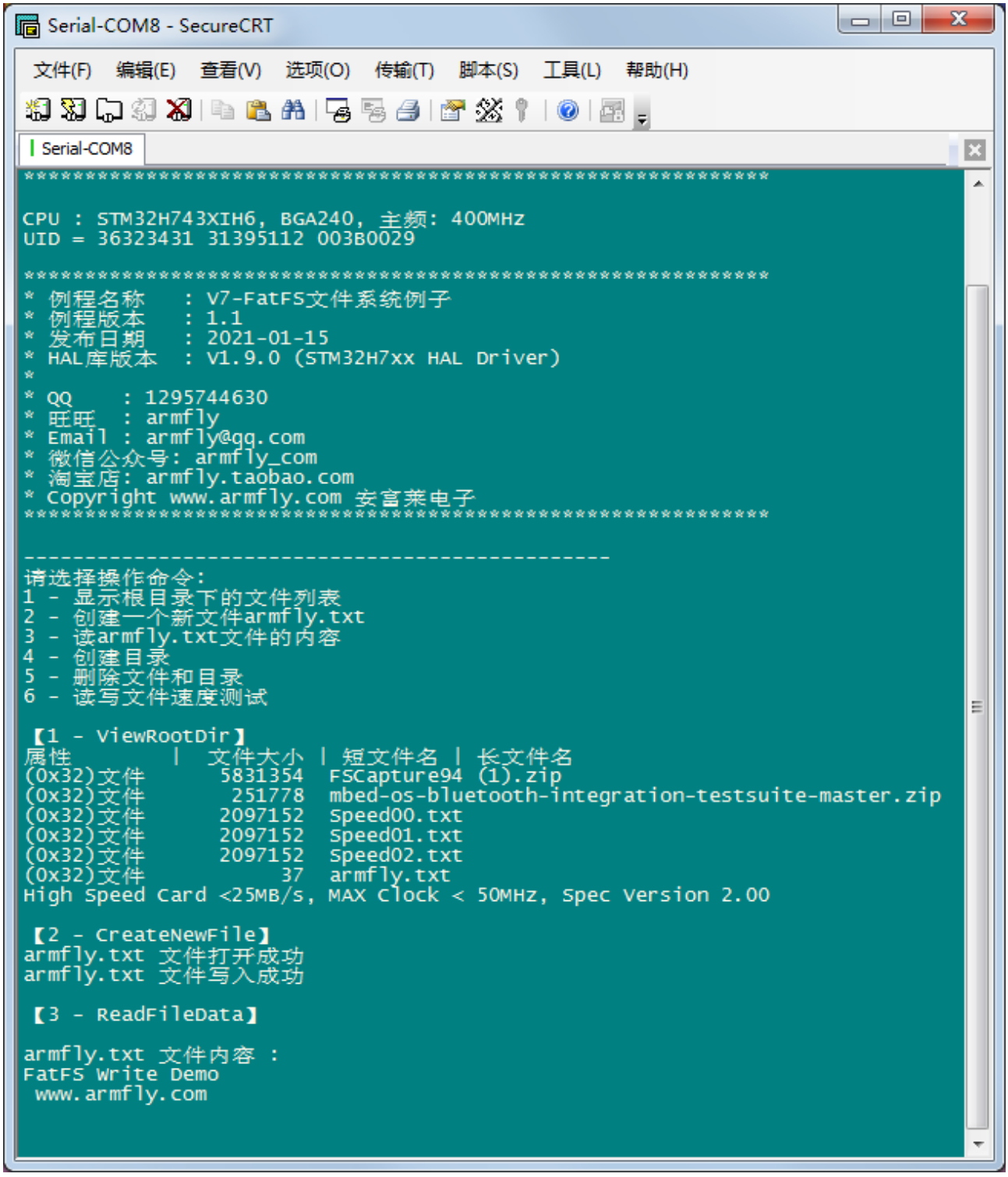
1. 上电启动了一个软件定时器，每 100ms 翻转一次 LED2。
2. V7 开发板的 SD 卡接口是用的 SDMMC1，而这个接口仅支持 AXI SRAM 区访问，其它 SRAM 和 TCP 均不支持。

实验操作：

1. 测试前务必将 SD 卡插入到开发板左上角的卡座中。
2. 支持以下 6 个功能，用户通过电脑端串口软件发送数字 1-6 给开发板即可
3. printf("1 - 显示根目录下的文件列表\r\n");
4. printf("2 - 创建一个新文件 armfly.txt\r\n");
5. printf("3 - 读 armfly.txt 文件的内容\r\n");
6. printf("4 - 创建目录\r\n");
7. printf("5 - 删除文件和目录\r\n");
8. printf("6 - 读写文件速度测试\r\n");

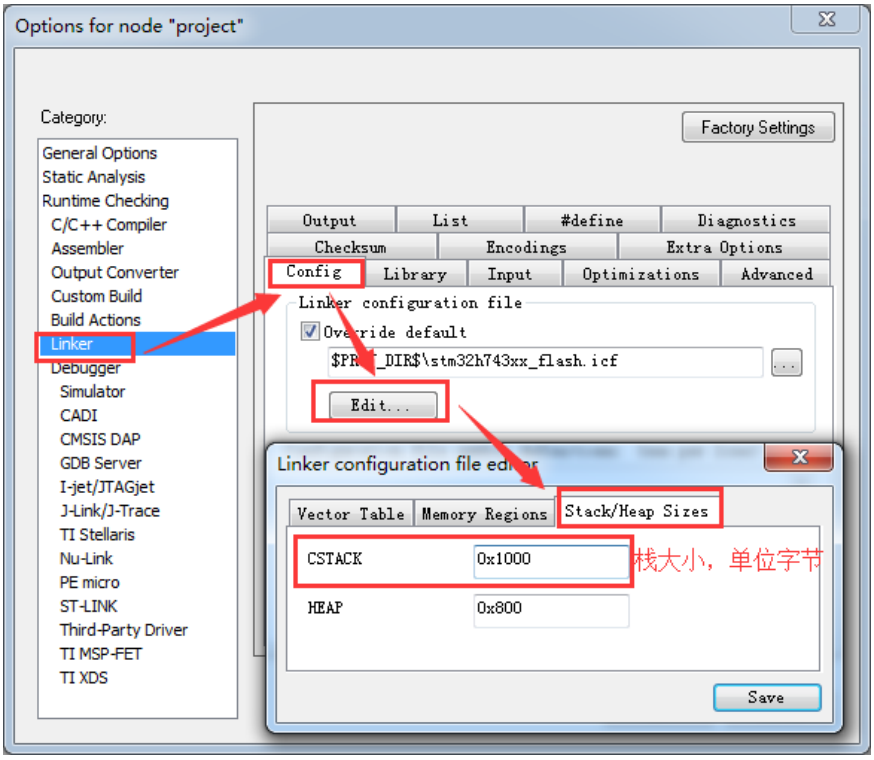
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

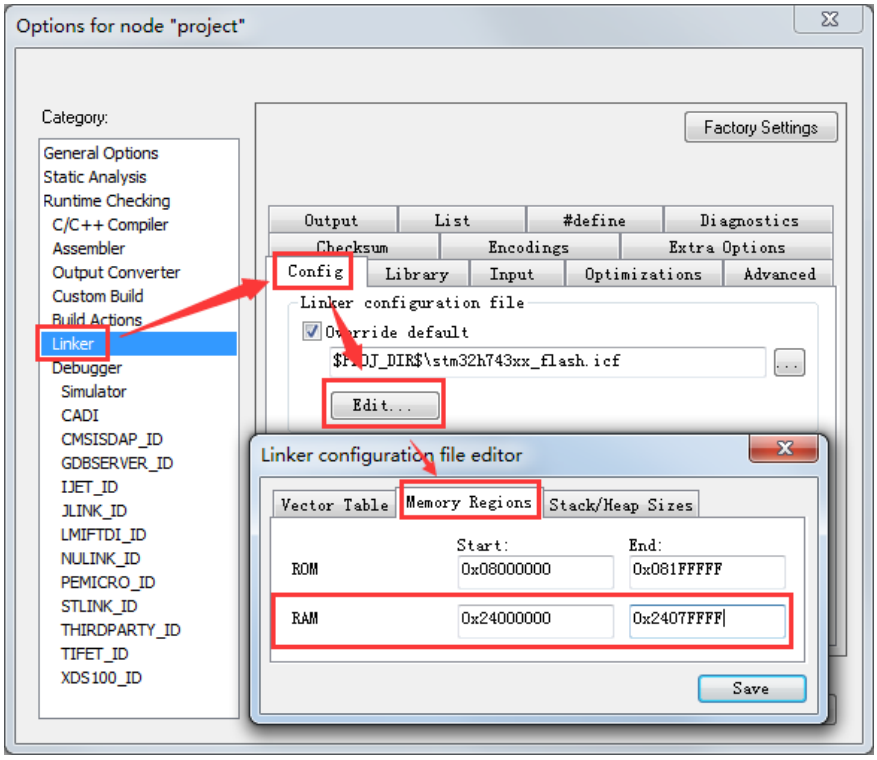


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 AXI SRAM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
```

```
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

◆ MPU 配置和 Cache 配置：

数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
```

```
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

#if 0
    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

 #else
    /* 当前是采用下面的配置 */
    /* 配置 AXI SRAM 的 MPU 属性为 NORMAL, NO Read allocate, NO Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
#endif
    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}
```

```
/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **主功能:**

主程序实现如下操作:

- 上电启动了一个软件定时器，每 100ms 翻转一次 LED2。
- 支持以下 6 个功能，用户通过电脑端串口软件发送数字给开发板即可:
- 1 - 显示根目录下的文件列表
- 2 - 创建一个新文件 armfly.txt
- 3 - 读 armfly.txt 文件的内容
- 4 - 创建目录
- 5 - 删除文件和目录
- 6 - 读写文件速度测试

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    bsp_Init();        /* 硬件初始化 */

    PrintfLogo(); /* 打印例程名称和版本等信息 */

    DemoFatFS();    /* SD 卡测试 */
}

/*
*********************************************************************************************************
*    函 数 名: DemoFatFS
*    功能说明: FatFS 文件系统演示主程序
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
```

```
void DemoFatFS(void)
{
    uint8_t cmd;

    /* 打印命令列表，用户可以通过串口操作指令 */
    DispMenu();

    /* 注册 SD 卡驱动 */
    FATFS_LinkDriver(&SD_Driver, DiskPath);

    bsp_StartAutoTimer(0, 500); /* 启动 1 个 500ms 的自动重装的定时器 */

    while (1)
    {

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 500ms 进来一次 */
            bsp_LedToggle(2);
        }

        if (comGetChar(COM1, &cmd)) /* 从串口读入一个字符(非阻塞方式) */
        {
            printf("\r\n");
            switch (cmd)
            {
                case '1':
                    printf("【1 - ViewRootDir】\r\n");
                    ViewRootDir();          /* 显示 SD 卡根目录下的文件名 */
                    break;

                case '2':
                    printf("【2 - CreateNewFile】\r\n");
                    CreateNewFile();    /* 创建一个新文件,写入一个字符串 */
                    break;

                case '3':
                    printf("【3 - ReadFileData】\r\n");
                    ReadFileData();         /* 读取根目录下 armfly.txt 的内容 */
                    break;

                case '4':
                    printf("【4 - CreateDir】\r\n");
                    CreateDir();        /* 创建目录 */
                    break;

                case '5':
                    printf("【5 - DeleteDirFile】\r\n");
                    DeleteDirFile();    /* 删除目录和文件 */
                    break;

                case '6':
                    printf("【6 - TestSpeed】\r\n");
                    WriteFileTest();    /* 速度测试 */
                    break;

                default:
```

```
                        DispMenu();
                        break;
                }
            }
        }
}
```

88.14 总结

本章节就为大家讲解这么多，需要大家实现操作一遍来熟练掌握 FatFs 的移植，然后 FatFs 相关的知识点可以到 FatFs 官网查看，资料非常详细。

第89章 STM32H7 的 CAN FD 总线基础之前世今生

本文整理翻译自 CAN in Automation（CiA），通过这个帖子大家会对当前市场上各种 CAN 名词有个全面系统的认识。

89.1 初学者重要提示
89.2 CAN 从构思到第一个芯片
89.3 CAN 标准化和规格
89.4 CAN 时代的先锋
89.5 CAN 从理论到实践
89.6 CAN 时间触发通信
89.7 CAN 总线被当局批准
88.8 CAN FD 开发
89.9 CAN 的未来是光明的
89.10 第三代 CAN

89.1 初学者重要提示

- 1986 年 2 月，Robert Bosch GmbH 在汽车工程师协会（SAE）大会上推出了控制器局域网（CAN）串行总线系统。这是有史以来最成功的网络协议之一诞生的时刻。如今，欧洲几乎所有汽车都配备了至少一个 CAN 网络。CAN 还用于其他类型的车辆，从火车到轮船，以及工业控制中，CAN 是最主要的总线协议之一，甚至可能是全球领先的串行总线系统。

89.2 CAN 从构思到第一个芯片

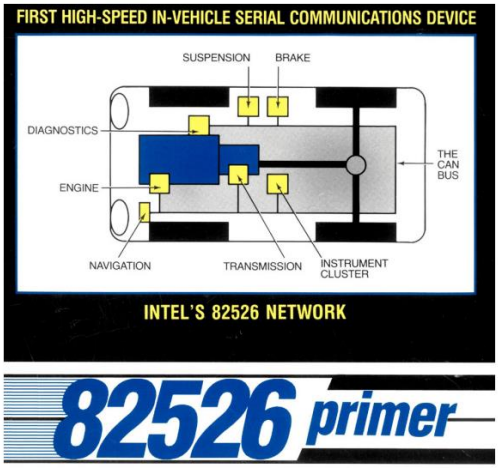
在 1980 年代初期，博世的工程师正在评估现有的串行总线系统，以探讨其在汽车中的可能用途。由于没有可用的网络协议能够满足汽车工程师的要求，因此 Uwe Kiencke 于 1983 年开始开发新的串行总线系统。

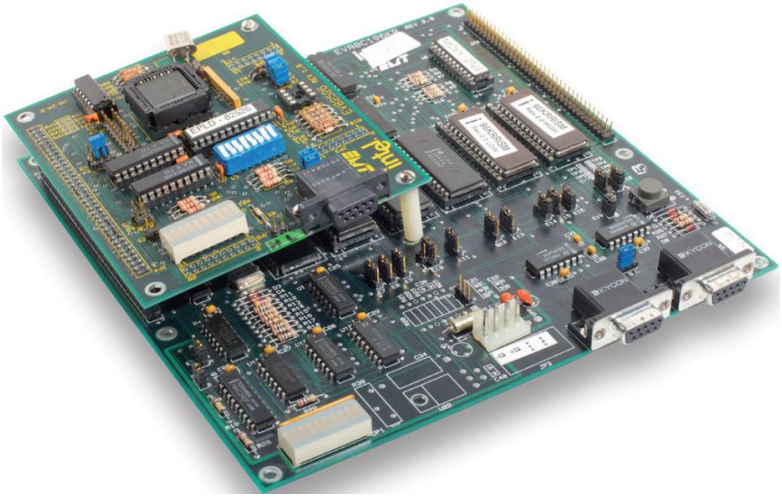
新的总线协议主要是要增加新的功能 --- 减少线束不是 CAN 发展背后的驱动力。奔驰的工程师很早就参与了新的串行总线系统的规范制定阶段，英特尔作为潜在的主要半导体供应商也参与了进来。来自德国 Braunschweig-Wolfenbüttel 的应用科学大学（现为 Ostfalia University of Applied Science）的

Wolfhard Lawrenz 教授被聘为顾问，他将新的网络协议命名为“Controller Area Network（控制器区域网络)”。来自卡尔斯鲁厄大学的 Horst Wettstein 教授也提供了学术帮助。

1986 年 2 月，CAN 诞生了：在底特律的 SAE 大会上，新的总线系统被称为“Automotive Serial Controller Area Network (汽车串行控制器局域网)”。Uwe Kiencke，Siegfried Dais 和 Martin Litschel 介绍了多主网络协议。它基于一种非破坏性仲裁机制，该机制允许总线访问具有最高优先级的帧而没有任何延迟。没有中心控制器。此外，CAN 之父（上述人员以及博世员工 Wolfgang Borst，Wolfgang Botzenhard，Otto Karl，Helmut Schelling 和 Jan Unruh）已经实现了几种错误检测机制。错误处理还包括自动断开故障总线节点，以保持其余节点之间的通信。传输的帧不是由帧发送器或帧接收器的节点地址（几乎在所有其他总线系统中）识别的，而是由它们的内容识别的。表示帧有效载荷的标识符，还具有指定帧在网络段内的优先级。

随后，许多介绍该创新通信协议的演讲和出版物，直到 1987 年中期（比计划提前两个月），英特尔才交付了第一款 CAN 控制器芯片 82526。这是 CAN 协议的第一个硬件实现。在短短四年内，一个想法就变成了现实。此后不久，飞利浦半导体推出了 82C200。CAN 控制器的这两个最早的祖先在接受过滤和帧处理方面有很大的不同。一方面，与飞利浦选择的 BasicCAN 实施相比，英特尔所青睐的 FullCAN 概念要求所连接的微控制器的 CPU 负载更少。另一方面，FullCAN 设备在可以接收的帧数方面受到限制。BasicCAN 控制器还需要更少的硅。在当今的 CAN 控制器中，实现了接受过滤和帧处理这两种概念的混合。这使 BasicCAN 和 FullCAN 的误导性术语过时了。



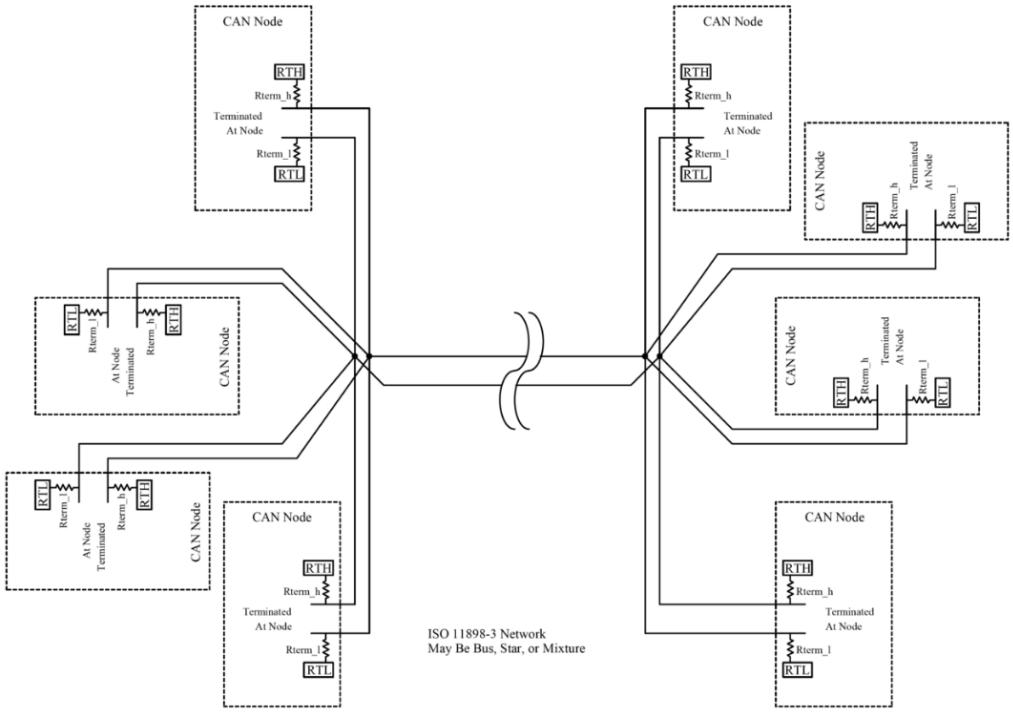


89.3 CAN 标准化和规格

博世 CAN 规范（2.0 版）已于 1990 年代初提交国际标准化。在发生了几场政治纠纷之后，尤其是涉及到法国一些主要汽车制造商开发的“车载区域网络”（VAN，Vehicle Area Network），ISO 11898 标准于 1993 年 11 月发布。除了 CAN 协议外，它还标准化了物理层，速率高达 1 Mbit / s。同时，ISO 11519-2 对通过 CAN 的低功耗，容错数据传输方式进行了标准化。由于该标准存在缺陷，因此从未实施。在 1995 年，ISO 11898 标准通过附录进行了扩展，该附录描述了使用 29 位 CAN 标识符的扩展帧格式。

不幸的是，所有已发布的 CAN 规范和标准化都包含错误或不完整。为避免不兼容的 CAN 实施方案，Bosch 确保所有 CAN 芯片均符合 Bosch CAN 参考模型。此外，由 Wolfhard Lawrenz 教授领导的德国 Braunschweig-Wolfenbüttel 应用科学大学已经进行了几年的 CAN 一致性测试。所使用的测试模式基于 ISO 16845 一致性测试计划标准系列。今天，多家测试机构可提供 CAN 一致性测试服务。

修订后的 CAN 规范已标准化。ISO 11898-1 描述了“ CAN 数据链路层”，ISO 11898-2 标准化了“非容错” CAN 物理层，而 ISO 11898-3 指定了“容错 CAN 物理层”。ISO 11992 系列（卡车和拖车接口）和 ISO 11783（农业和林业机械）系列指定了基于 SAE J1939 网络方法的应用程序配置文件。它们不兼容，因为物理层规范不同。



89.4 CAN 时代的先锋

尽管 CAN 最初被开发用于汽车，但最初的应用来自不同的市场领域。特别是在北欧，CAN 在早期已经非常流行。在芬兰，电梯制造商 Kone 使用了 CAN 总线。瑞典工程办公室 Kvaser 向一些纺织机械制造商（Lindauer Dornier 和 Sulzer）及其供应商建议将 CAN 作为机器内部的通信协议。在 Lars-Berno Fredriksson 的领导下，这些公司成立了“ CAN 纺织品用户小组”。到 1989 年，他们制定了沟通原则，有助于在 1990 年代初塑造发展环境“ CAN Kingdom”。尽管相对于 OSI 参考模型，CAN Kingdom 不是应用程序层，

在荷兰，飞利浦医疗系统决定将 CAN 用于其 X 射线机的内部联网，从而加入了工业 CAN 用户。主要由 Tom Suters 开发的“ Philips 消息规范” （PMS）代表了 CAN 网络的第一个应用层。来自德国 Weingarten 应用科学大学的 Konrad Etschberger 教授也有相同的想法，在他负责的 Steinbeis 过程自动化转移中心（STZP）中，他开发了类似的协议。

尽管第一个标准化的高层协议开始出现，但大多数 CAN 开拓者还是采用了整体法。通信功能，网络管理和应用程序代码是一种软件。即使某些用户希望采用模块化的方法，但他们仍有专有解决方案的缺点。增强和维护 CAN 高层协议的必要努力被低估了 ---今天仍然部分是正确的。

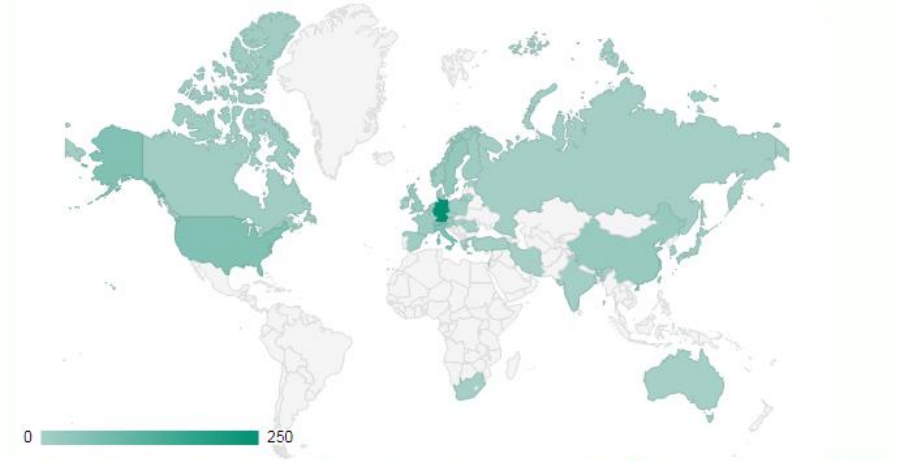
在 1990 年代初，是时候成立了一个用户小组，以推广 CAN 协议并促进其在许多应用中的使用。1992 年 1 月，当时的《 VMEbus》杂志编辑（出版商：Franzis）的 Holger Zeltwanger 吸引了用户和制造商一起建立了一个中立的平台，用于 CAN 的技术增强以及串行总线系统的市场营销。两个月后，“ CAN in Automation” （CiA）国际用户和制造商小组正式成立。在早期，CAN 通讯已经出版。

仅几周后发布的第一份技术出版物涉及物理层：CiA 建议仅使用符合 ISO 11898 的 CAN 收发器。如今，制造商专用的 EIA-485 收发器在 CAN 网络中非常普遍，那个时候，并不总是兼容，应该已经完全消失了。

CiA 的首要任务之一是规范 CAN 应用层。利用飞利浦医疗系统公司和 STZP 提供的现有材料，并在 CiA 其他成员的帮助下，开发了“ CAN 应用程序层” （CAL），也称为“绿皮书”。在使用 CAN 制定规范时，CiA 的主要任务之一是组织 CAN 专家与那些希望在 CAN 上变得更多知识的专家之间的信息交换。因此，自 1994 年以来，举行了国际 CAN 会议（iCC）。

LAV（德国农业车辆协会）追求另一种学术方法。自 1980 年代后期以来，已经开发了基于 CAN 的农用车辆总线系统（LBS）。但是在这项工作可以成功完成之前，国际委员会已决定支持美国的解决方案 J1939（ISO 11783）。此应用程序配置文件也基于 CAN，由 SAE 卡车和公共汽车协会的委员会定义。J1939 是一种非模块化方法，非常易于使用，但也非常不灵活。

还为卡车开发了 CAN 的标准化。卡车和拖车之间的网络标准化为 ISO11992。该协议基于 J1939，从 2006 年起必须在欧洲使用。汽车的趋势是向 OSEK-COM 和 OSEK-NM，通信协议和网络管理迈进。两者均已提交国际标准化。到目前为止，汽车制造商一直在使用专有软件解决方案。



89.5 CAN 从理论到实践

当然，将 CAN 内核实现到其微控制器中的半导体供应商主要集中在汽车行业。自 1990 年代中期以来，英飞凌科技公司（以前为西门子半导体公司）和摩托罗拉公司（外包为飞思卡尔，后来被 NXP 收购）已经向欧洲汽车制造商及其供应商运送了大量 CAN 控制器。自 1990 年代末以来，远东半导体供应商也开始提供 CAN 控制器。NEC 在 1994 年推出了他们传奇的 CAN 芯片 72005，但为时过早-该组件在商业上并不成功。

自 1991 年以来，奔驰一直在其汽车中使用 CAN。第一步，通过 CAN 连接负责发动机管理的电子控制单元。1995 年，宝马在其 7 系汽车中使用了带有五个 ECU（电子控制单元）的树/星拓扑 CAN 网络。第二步，跟随人体电子设备所需的控制单元。实现了两个物理上分离的 CAN 网络，通常通过网关连接。

其他汽车制造商也沿用了奔驰的方式，通常在乘用车中实现两个 CAN 网络。如今，他们都在自己的车辆中实现了多个 CAN 网络。

在 1990 年代初期，美国机械工程公司 Cincinnati Milacron 的工程师与 Allen-Bradley 和 Honeywell Microswitch 共同成立了一家合资公司，以基于 CAN 的控制和通信项目。但是，不久之后，重要的项目成员换了工作，合资企业瓦解了。但是 Allen-Bradley 和 Honeywell 分别继续工作。这导致了至少在较低的通信层中非常相似的两个高层协议“ DeviceNet”和“智能分布式系统”（SDS）。1994 年初，Allen-Bradley 将 DeviceNet 规范移交给了“开放 DeviceNet 供应商协会”(ODVA)，这促进了 DeviceNet 的普及。Honeywell 未能在 SDS 上采取类似的方式，这使 SDS 看起来更像是 Honeywell 微动的内部解决方案。DeviceNet 是专为工厂自动化而开发的，因此，它本身就是 Profibus-DP 和 Interbus 等协议的直接对手。提供现成的即插即用功能，DeviceNet 已成为美国这个特定细分市场中领先的总线系统。

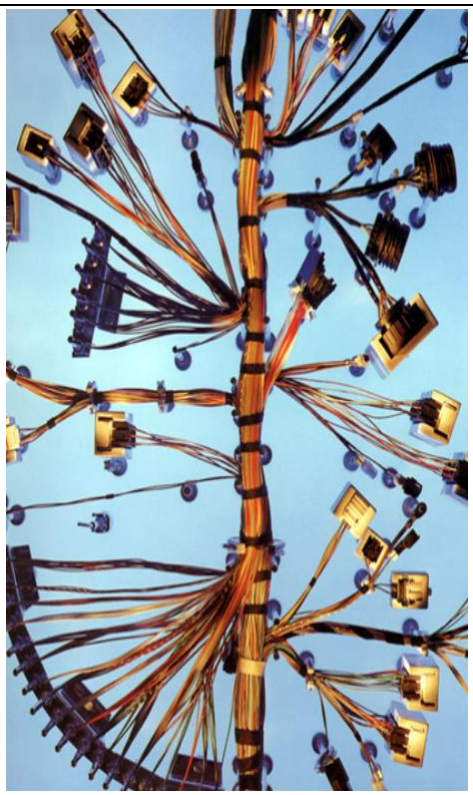
在欧洲，几家公司尝试使用 CAL。尽管 CAL 方法在学术上是正确的，并且可以在工业应用中使用它，但是每个用户都需要设计一个新的配置文件，因为 CAL 是真正的应用程序层。CAL 可以看作是独立于应用程序的 CAN 解决方案必不可少的学术步骤，但它从未在该领域获得广泛认可。

自 1993 年以来，在 Esprit 项目 ASPIC 的范围内，由博世领导的欧洲财团一直在开发 CANopen 的原型。它是用于生产单元内部网络的 CAL 配置文件。在学术方面，来自德国罗伊特林根应用科学大学的 Gerhard Gruhler 教授和来自英国纽卡斯尔大学的 Mohammed Farsi 博士参加了有史以来最成功的 Esprit 活动之一。项目完成后，CANopen 规范移交给了 CiA，以进行进一步的开发和维护。1995 年，发布了经过完全修订的 CANopen 通信配置文件，并在短短五年内成为欧洲最重要的标准化嵌入式网络。

第一个 CANopen 网络用于内部机器通信，尤其是驱动器。CANopen 提供了很高的灵活性和可配置性。高层协议已用于多个非常不同的应用领域（工业自动化，航海电子，军用车辆等），同时其国际标准为 EN 50325-4（2003）。CANopen 尤其在欧洲使用。意大利的注塑机，德国的木锯和机器，英国的卷烟机，法国的起重机，奥地利的搬运机以及瑞士的钟表制造机只是工业自动化和机械制造中的几个例子。在美国，推荐将 CANopen 用于叉车并将其用于信件分拣机中。

CANopen 不仅定义了应用程序层和通信配置文件，而且还定义了可编程系统以及不同设备，接口和应用程序配置文件的框架。这是整个行业（例如印刷机，海事应用，医疗系统）决定在 1990 年代后期使用 CANopen 的重要原因。

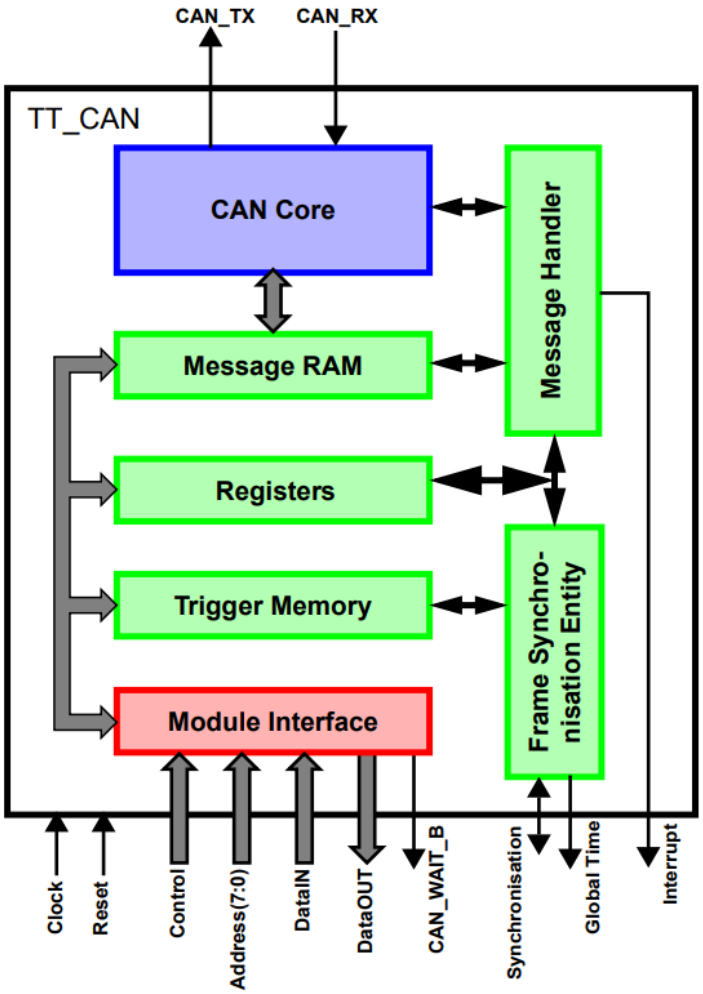
借助 DeviceNet 和 CANopen，可以使用两个标准化的 (IEC 62026-3 或 EN 50325-4 / 5) 应用层，以满足不同的工业自动化市场的需求。DeviceNet 针对工厂自动化进行了优化，而 CANopen 特别适合于各种机器控制中的嵌入式网络。这使得专有的应用程序层已过时。定义特定于应用程序的应用程序层的必要性已成为历史（也许，对于某些专用的大容量嵌入式系统而言除外）。



89.6 CAN 时间触通信

在 2000 年初，一个由数家公司组成的 ISO 特别工作组为 CAN 帧的时间触发传输定义了一个协议。Bernd Mueller 博士，Thomas Fuehrer 博士和其他博世员工，以及半导体行业和学术研究专家共同定义了协议“ CAN 上的时间触发通信”（TTCAN，Time-triggered communication on CAN）。

这种 CAN 扩展可以实现时间等量的帧传输和通过 CAN 的闭环控制的实现，而且还可以在 x-by-wire 的应用中使用 CAN。由于 CAN 协议未更改，因此可以通过同一物理总线系统传输时间触发帧和事件触发帧。但是，汽车行业尚未采用 TTCAN。而且，工业用户很少使用时间触发的协议扩展。他们改用 CANopen 中指定的同步传输功能，可以说是一种软时间触发方法。



89.7 CAN 总线被当局批准

在 90 年代后期，发明了几种基于 CAN 的专有安全协议。幸存下来的是德国 Pilz 生产的 Safetybus p。在 1999 年，CiA 开始开发 CANopen-Safety 协议，该协议已获得德国 TüV 的批准。经过标准化机构的大量参与之后，此 CANopen 扩展（CiA 304）在 EN 50325-5（2009）中进行了国际标准化。

DeviceNet 使用 CIP 安全协议扩展。全球领先的船级社之一的德国劳埃德船级社（Germanischer Lloyd）已批准了适用于海事应用的 CANopen 框架（CiA 307）。除其他外，该框架规定了从默认的 CANopen 网络到冗余总线系统的自动切换。如今，这些功能已在 CiA 302 系列其他 CANopen 应用层功能中得到了概括和指定。

89.8 CAN FD 开发

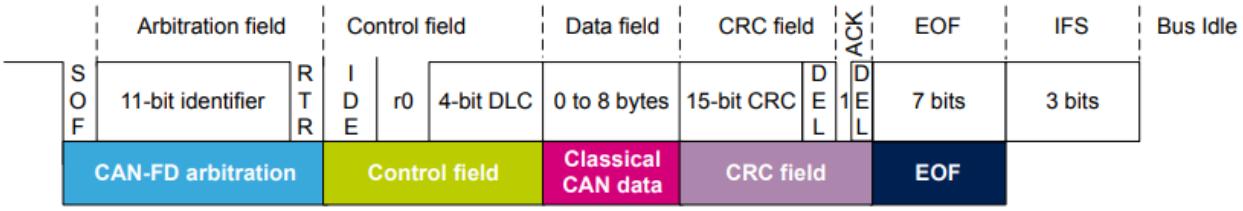
2011 年初，通用汽车和博世开始着手开发一些有关提高吞吐量的 CAN 协议。当将越来越多的软件包下载到电子控制单元（ECU）时，汽车行业尤其受苦。高性能通信系统必须缩短这项耗时的任务。通过引

入第二个比特率来提高 CAN 传输速度的想法并不新鲜。自 2000 年初以来，已有几位学者发表了研究方法。但是，没有一个研究者能够说服汽车制造商。博世与其他 CAN 专家合作，预先开发了 CAN FD 规范，该规范于 2012 年在德国哈姆巴赫城堡举行的第 13 届国际 CAN 会议上正式推出。

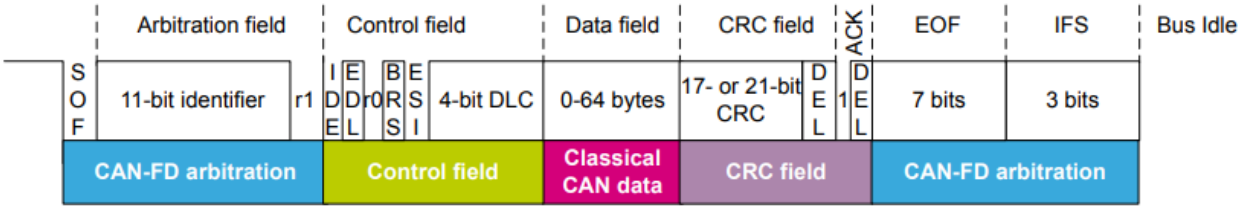
在 ISO 的标准化过程中，发现了所提出的错误检测机制中的一些学术缺陷。这需要对 CAN FD 协议进行审查，并引入其他防护措施（例如，填充位计数器）。这就是为什么存在非 ISO CAN FD 协议的原因，该协议与 ISO 11898-1 中标准化的 ISO CAN FD 协议不兼容。

戴姆勒公司的 Mark Schreiner 博士为设计 CAN FD 网络提供了很多提示和知识。他的许多想法都包含在 CAN FD 节点的 CiA 601 系列以及系统设计建议和规范中。

CAN 2.0: Classical base frame format



CAN-FD: CAN flexible data rate base frame format



DEL = Deliminator RTR = Remote transmission request

89.9 CAN 的未来是光明的

随着 CAN FD 协议的引入，CAN 技术的寿命得以延长。汽车行业已经开始为下一代车载网络采用 CAN FD 协议。可以预期，所有未来的应用程序都将使用 CAN FD 协议。不管它们是否需要更高的带宽都没关系。您仍然可以将 CAN FD 与单个位定时设置一起使用。有效负载长度无论如何都可以在 0 到 64 字节之间配置。

对于那些需要更多带宽并需要混合拓扑的用户，CiA 已开发了所谓的 SIC（信号改善电路）收发器规范（CiA 601-4）。最初的想法来自日本 Tier-1 供应商 Denso。

CiA 还开发了基于 CAN FD 下层的 CANopen FD 协议。特别是对于工业运动控制应用，非常适合更高的传输速率和更长的有效载荷（最大 64 字节）。CiA 还参与了使用 SAE J1939 系列中指定的现有参数组的商用车，基于 CAN FD 的应用层的开发。

89.10 第三代 CAN

2018 年底，CiA 开始开发第三代基于 CAN 的数据链路层协议 CAN XL。它是应大众汽车的要求启动的。Carsten Schanze 和 Alexander Mueller 提供了许多最初的想法。

2020 年由于疫情，取消了国际 CAN 会议的执行，今年的国际 CAN 会议改为线上举行，主题是 CAN XL。CAN XL 速度提升至 10Mbps。

同时保持 CAN 协议的优势以及与 CAN 和 CAN FD 的兼容性。下面是 CAN FD 和 CAN XL 的混合网络：

补充：

- CiA CAN XL 规范于 2018 年 12 月开始，目标是在 2019 年底之前拥有 OSI 第 2 层规范（称为 CAN XL 协议），第二步，将开始进行 ISO 标准化。
- 经典 CAN 是 1Mbps，CAN FD 最高 2Mbps，CAN FD-SiC 是 5-8Mbps，CAN XL 是 10Mbps。

89.11 总结

更多 CAN 的历史知识，大家可以上 CAN in Automation（CiA）官网搜索了解。

第90章 STM32H7 的 CAN FD 总线之关键知识点整理

CAN FD（CAN with flexible data-rate）是 CAN2.0 协议的扩展，CAN-FD 由博世开发，并由 ISO 11898-1:2015 标准化。

90.1 初学者重要提示
90.2 CAN FD 协议介绍
90.3 CAN FD 相比 CAN2.0 的提升
90.4 CAN FD 协议介绍
90.5 CAN FD 的 RAM 区管理
90.6 CAN FD 过滤区
90.7 CAN FD 接收区
90.8 CAN FD 发送区
90.9 CAN FD 测试模式
90.10 FDCAN 时钟校准
90.11 FDCAN 相比标准 bxCAN 的提示
90.12 总结

90.1 初学者重要提示

- 本章是对 ST 官方 30 多页的英文文档翻译而来（翻译不准确的地方，大家以英文原版为准），通过本章可以对 CAN FD 有个系统性的认识。原始的英文文档下载地址：http://www.armbbs.cn/forum.php?mod=viewthread&tid=104728 。

90.2 CAN FD 协议介绍

90.2.1 CAN FD 特性

（1）兼容经典 CAN，可以遵循 ISO 11898-1 做数据收发。
（2）提升错误检测，支持高达 CRC 21 位的校验和。

（3）消息优先级。

（4）保证延迟时间。

（5）配置灵活性。

（6）具有时间同步的组播接收。

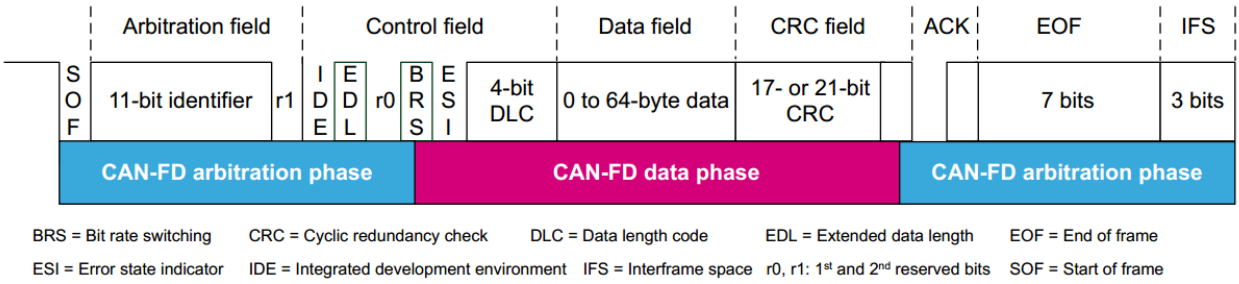
（7）系统范围内的数据一致性，每条消息最多 64 个字节。

（8）多主机。

（9）错误检测和信号。

（10）区分节点的临时错误和永久性故障以及自动关闭缺陷节点。

90.2.2 CAN FD 格式



第一个仲裁阶段（The first arbitration phase）是一条消息，其中包含：

（1）帧开始（SOF）。

（2）ID 号和其他位，指示消息的目的（提供或请求数据），以及速度和格式配置（CAN 或 CAN-FD）。

数据传输阶段（The data transmission phase）包括：

（1）数据长度代码（DLC），指示消息包含多少数据字节。

（2）用户希望发送的数据。

（3）检查循环冗余序列（CRC）。

（4）显性位。

第二个仲裁阶段（The second arbitration phase）包含：

（1）总线上其他节点发送的确认（ACK）的接收器（如果至少有一个接收器成功收到消息）

（2）帧尾（EOF），在 IFS 期间不传输任何消息：目标是将当前帧与下一帧分开。

注意：对于 29bit 标识符帧，当添加 18bit 标识到第 1 个仲裁阶段的 IDE bit 之后与标准 CAN FD 是类似的。

90.3 CAN FD 相比 CAN2.0 的提升

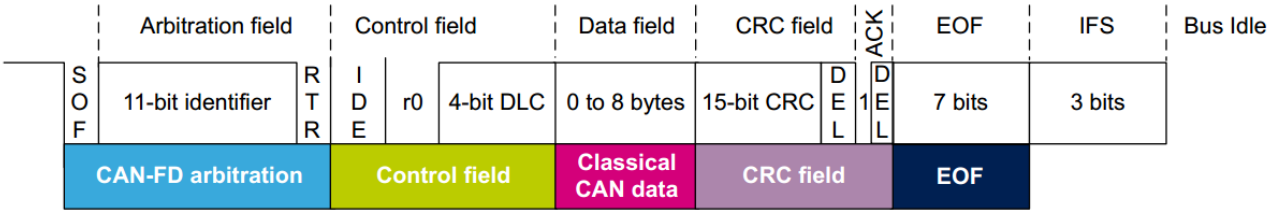
CAN-FD 的开发可以满足需要更高带宽的通信网络需求。每帧最多具有 64 个字节的 CAN-FD 以及将比特率提高到最大的可能性，使数据阶段要快 8 倍，在第二个仲裁阶段要恢复到正常的比特率。通过以下方式确保数据传输的完整性：

（1）17 级多项式对最大 16 字节的有效载荷进行 CRC。

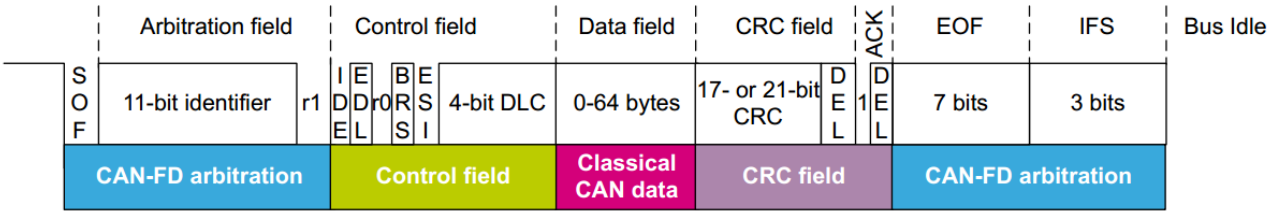
（2）21 级多项式对 16 到 64 字节之间的有效载荷进行校验。

标准帧和 CAN FD 的区别：

CAN 2.0: Classical base frame format



CAN-FD: CAN flexible data rate base frame format



DEL = Deliminator　RTR = Remote transmission request

标识符后，CAN 2.0 和 CAN-FD 具有不同的作用：

（1）CAN 2.0 发送 RTR 位以精确确定帧类型：数据帧（RTR 为主要）或远程帧（RTR）是隐性的）。

（2）由于 CAN-FD 仅支持数据帧，因此始终发送占优势的 RRS（保留）。

IDE 位保持在相同位置，并以相同的动作来区分基本格式（11 位标识符）。请注意，在扩展格式的情况下，IDE 位以显性或隐性方式传输（29 位标识符）。

与 CAN 2.0 相比，在 CAN-FD 帧中，在控制字段中添加了三个新位：

（1）扩展数据长度（EDL）位：隐性表示帧为 CAN-FD，否则该位为显性（称为 R0）在 CAN 2.0 帧中。

（2）比特率切换（BRS）：指示是否启用两个比特率（例如，当数据阶段位以不同的比特率传输到仲裁阶段）。

（3）错误状态指示器（ESI）：指示节点处于错误活动模式还是错误被动模式。

控制字段的最后一部分是数据长度代码（DLC），它具有相同的位置和相同的长度（4 位），用于 CAN 2.0 和 CAN-FD。 DLC 功能在 CAN-FD 和 CAN 2.0 中相同，但 CAN-FD 有很小变化（下表中的详细信息）。CAN-FD 扩展帧允许单个消息中发送 64 个数据字节，而 CAN 2.0 有效负载数据最多可以发送 8 个字节。

DLC (Dec)	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
CAN 2.0	0	1	2	3	4	5	6	7	8	8	8	8	8	8	8	8
CAN-FD	0	1	2	3	4	5	6	7	8	12	16	20	24	32	48	64

通过增加有效载荷数据的数据字段来改善网络带宽，因为需要更少的包处理。 同时，通过为 CRC 添加更多位来增强消息完整性：

（1）如果有效载荷数据最多为 16 个字节，则 CRC 以 17 位编码。

（2）如果有效载荷数据大于 20（16）个字节，则 CRC 以 21 位编码。

另外，为了确保 CAN-FD 帧的鲁棒性，填充位机制支持 CRC 字段。下表总结了 CAN-FD 和 CAN 2.0 之间的主要区别。 提供的主要功能与 CAN 2.0 相比，CAN FD 的改进之处在于数据有效负载的增加和速度的提高由 CAN-FD 中可用的 BRS，EDL 和 ESI 位来确保。

Features	CAN 2.0	CAN-FD
Compatibility	Does not support CAN-FD	Supports CAN 2.0 A/B
Maximum bit rate (Mbit/s)	Frame bitrate: up to 1	Arbitration bitrate: up to 1s Data bitrate: up to 8
DLC field (4 bits) code	Coded in 0 to 8	Coded in 0 to 64
Maximum data bytes in one message	8 bytes of data	64 bytes of data
BRS support	No	Yes
EDL support	No	Yes
ASI support	No	Yes
CRC bits check codes	Bits not included in CRC calculation	Bits included in CRC calculation
Remote frame support	Yes	No

90.4 CAN FD 协议介绍

STM32H7 的 CAN FD 符合 ISO 11898-12015 标准。STM32 器件上的 FDCAN 的功能如下所示：

（1）符合 CAN 协议 2.0 版 A，B 和 ISO 11898-1：2015，-4。

（2）可访问的 10 KB RAM 内存，最多可分配 2560 个字。

（3）改进了接收过滤。

（4）两个可配置的接收 FIFO。

（5）多达 64 个专用接收缓冲区。

（6）接收高优先级消息时的单独信令。

（7）多达 32 个专用发送缓冲区。

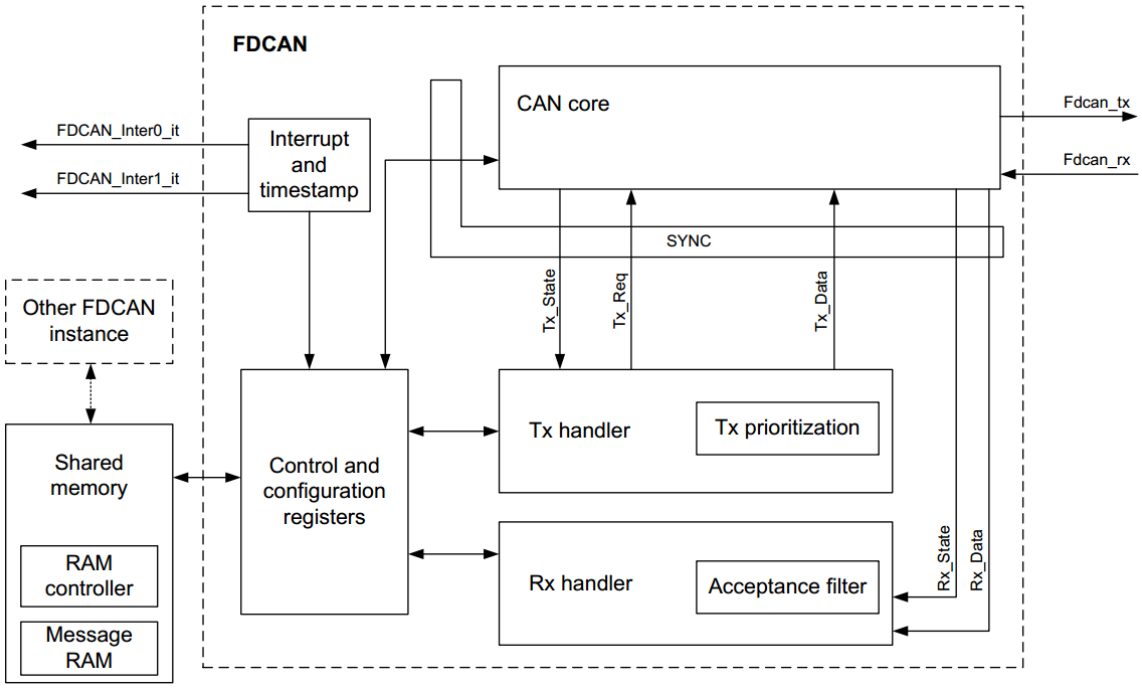
（8）可配置的发送 FIFO 和发送队列。

（9）可配置的发送事件 FIFO。

（10）时钟校准单元。

（11）收发器延迟补偿。

下图说明了 FDCAN 框图。



通过这个框图要了解到以下信息：

（1）CANFD1 和 CANFD2 共用一个 RAM 空间。

（2）每个 CANFD 都有自己的内核。

（3）CAN 内核实现协议控制和收发移位寄存器。

（4）Tx handler 控制消息从 CAN 消息 RAM 到 CAN 内核。

（5）Rx handler 控制 CAN 内核到 CAN 消息 RAM。

90.5 CAN FD 的 RAM 区管理

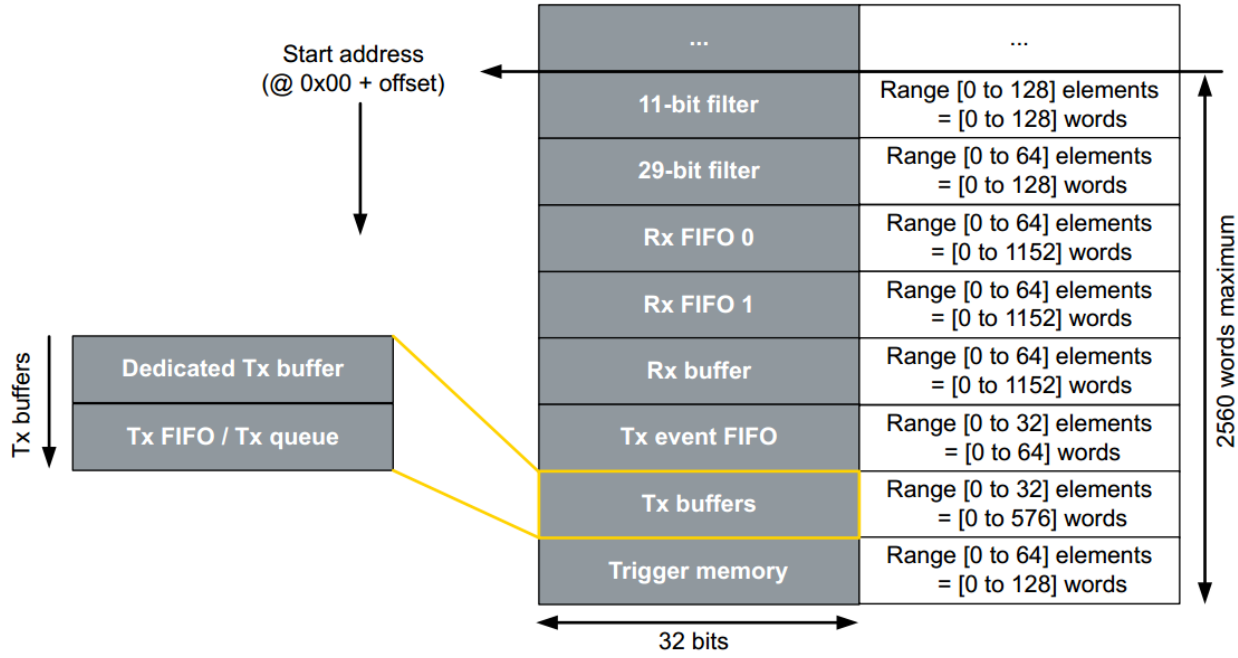
必须配置每条消息的字节数，以确定每条消息所需的内存大小。 CAN-FD 上有效负载的增加可以更有效的将内存都利用起来，并允许更多消息要存储在分配的存储空间中。总的专用 RAM 大小是 2560 个 32bit 数据。

（1）过滤区（11bit 过滤和 29bit 过滤）

（2）接收区（Rx FIFO 0，Rx FIFO 1 和 Rx Buffer）

（3）发送区（Tx event FIFO，Tx Buffers）

（4）触发空间（Trigger memory）



FDCAN 外设的所有部分都可以由用户配置。所有部分的所有元素之和必须不能超过 CAN 消息 RAM 的总大小。 该 RAM 通过以下方式提高了灵活性和性能：从而有可能消除未使用的部分，并为其他部分扩展足够的内存。在 CAN 消息中以动态且连续的方式分配每个部分（按照上图顺序）； 但是为了避免超出 RAM 的风险并且出于可靠性的原因，没有将特定的开始和结束地址分配给每个部分。

FDCAN外设可以配置三种传输机制：Tx buffer ，Tx queue 和 Tx FIFO并可以通过Rx buffer 和 Rx FIFO 接收。它们配置有起始地址偏移量和要存储的存储元素。起始地址是在配置中预定义的(0到2560)，用户必须确保每个内存空间中的元素数量不会导致它们重叠。

注：消息的接收和发送意味着在 RAM 级别存储“元素”。 这个“元素”仅包含标识符，DLC，控制位（ESI，XTD，RTR，BRS，FDF），数据字段和特定的发送/接收位字段进行控制。 CAN 消息的其余位会自动由硬件处理而不保存在 RAM 中 。

用于控制接收的特定位字段是滤波器索引，可接受的不匹配帧和 Rx 时间戳。用于传输的特定位字段是消息标记 (message marker) 和 event FIFO 控制位。无论 Tx buffer, Tx FIFO, Tx queue 或 Rx buffer 中的每个元素分配的 32bit 的数量是多少，要计算保留：

（1）标头信息（两个保留的 32bit），用于分配标识符，DLC 字段，控制位和特定的发送/接收位字段。

（2）数据（足够的 32bit 数据个数）以包含每个数据字段的字节数。

下面的公式确定为每个元素分配的 32bit 的数量：

元素大小（以字为单位）=标头信息（2 个字）+数据（数据字段/ 4）

其中数据字段是每条消息的数据字节数

如果数据字段的范围是 0 到 8，则每个元素为数据分配 2 个字。下表详细介绍了必要的 “元素” 大小，具体取决于数据字段范围：

Data field (bytes)	Element size (RAM words)
0 to 8	4
12	5
16	6
20	7
24	8
32	10
48	14
64	18

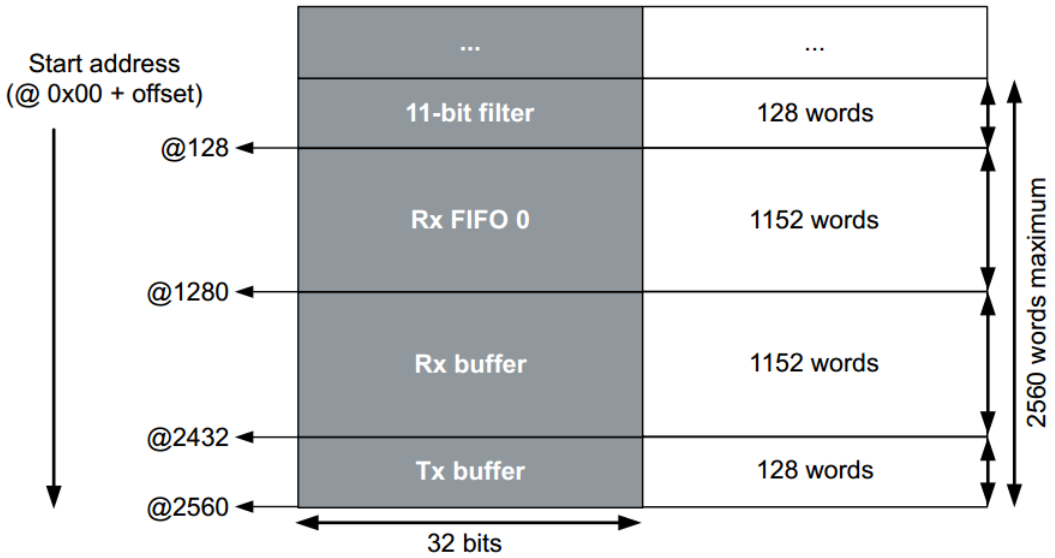
有效使用 CAN 消息 RAM 的示例。 这个例子假设配置了 FDCAN 外设的应用程序：

（1）发送带有专用 Tx buffer 的 32 条消息（每个消息在数据字段中包含 8 个字节）。

（2）具有 128 个 11 位过滤器，用于接收消息。

（3）接收 64 条消息，其中每个消息在专用 Rx buffers 的数据字段中包含 64 个字节。

（4）接收 64 条消息，其中每个消息在 Rx FIFO 0 的数据字段中包含 64 个字节。



在此示例中，RAM 中的分配按以下顺序进行：

（1）在 ID-11 位部分分配 128 个字。

（2）保留 1152 个字用于接收 Rx FIFO 0 部分中的元素。

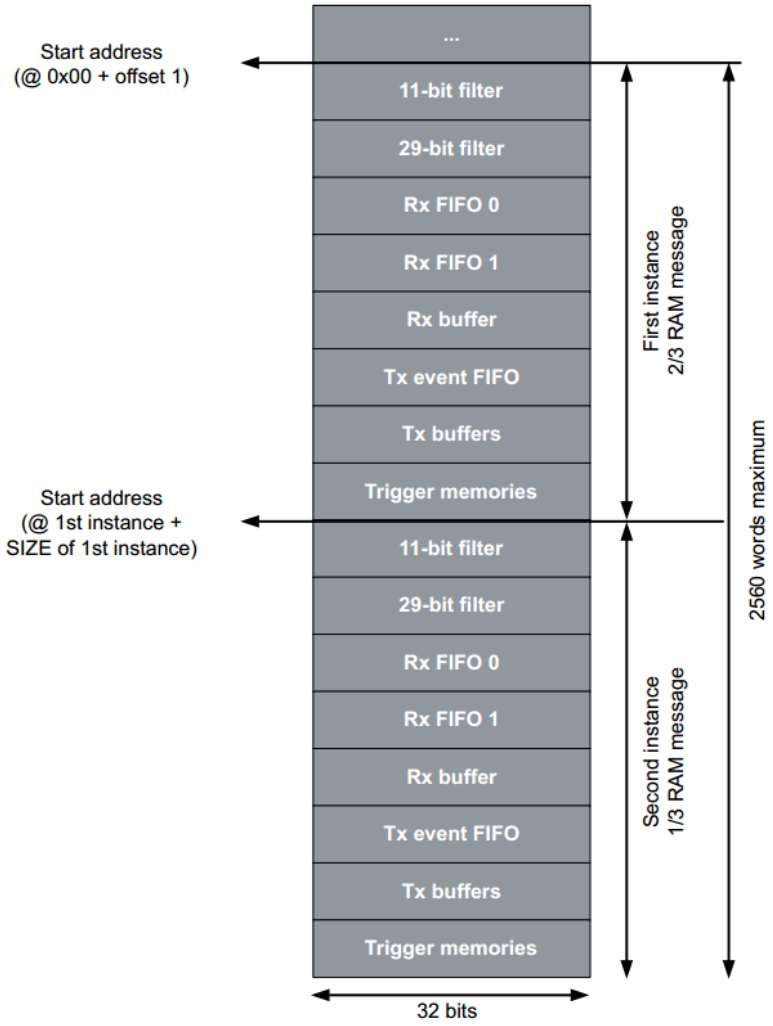
（3）保留 1152 个字，用于接收 Rx 缓冲区中的元素。

（4）为发送缓冲区中发送的元素保留 128 个字。

由于采用了动态分配，并且不对未使用的段进行任何分配，因此整个内存大小有效利用了 RAM：在此应用程序中分配了所有 2560 个字。

注意：配置后，分配的地址范围初始化为零。

大多数 STM32 设备支持一个以上的 FDCAN 外设来满足所有应用程序要求。在这种情况下，重要的是要说 RAM 是在不同外设之间共享的。 用户可以划分各种外设上的 RAM（每个外设的大小通过指示其起始偏移地址来选择）。下图显示了使用多个 FDCAN 实例的 CAN 消息 RAM 的示例。 这该示例假定用户将 CAN 消息 RAM 分为两个外设：第一个外设的大小为第二个外设的两倍。

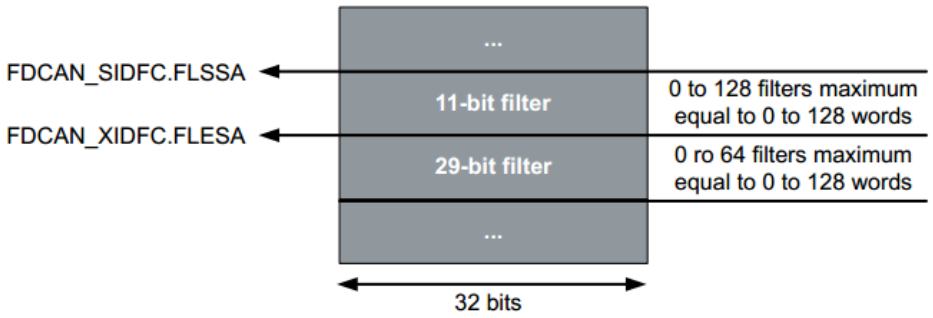


90.6 CAN FD 过滤区

CAN 总线的标识符过滤器很重要。

过滤器是用于接收的，发送的时候无需设置过滤器，这点一定要整明白。根据发送端发送来的帧数据，通过标识符过滤器就可以设置仅接收需要接收的帧数据有效降低 CPU 多余的处理时间。

FDCAN 外设可以配置两套验收滤波器：一套用于标准标识符，一种是扩展标识符，用于存储或拒绝接收到的消息。128 个过滤元素可以配置用来配置 128 个 11bit 标准 ID 和 64 个过滤元素可以用来配置 64 个 29bit 扩展 ID。11 位过滤器部分的起始地址通过 FDCAN_SIDFC 寄存器中的 FLSSA [13：0]位进行配置，并且通过 FDCAN_XIDFC 寄存器中的 FLESA [13：0]配置 29 位滤波器部分。下图显示了 CAN 消息 RAM 的一部分，其中包含过滤器元件的数量及其开始位置地址。



可以将这些过滤器分配给 Rx FIFO 0/1 或专用的 Rx buffers。 FDCAN 执行接受过滤，它总是从过滤器元素＃0 开始，然后遍历过滤器列表以找到匹配项元素。验收过滤在第一个匹配元素处停止，而随后的过滤元素不在针对此消息进行了检索。因此，配置过滤器元素的顺序对过滤性能有很重要的影响。用户选择启用或禁用每个过滤器元素，并可以配置每个元素以接受或拒绝过滤。每个过滤器元素可以配置为：

（1）范围过滤器（Range filter）：该过滤器匹配标识符在两个 ID 定义的范围内的所有消息。

（2）专用 ID 的过滤器（Filter for dedicated IDs）：可以将过滤器配置为匹配一个或两个特定的标识符。

（3）经典位屏蔽过滤器（Classic bit mask filter）：通过对接收到的标识符的位进行屏蔽来匹配标识符组。第一个 ID 配置为消息 ID 过滤器，第二个 ID 为过滤器屏蔽。过滤器屏蔽的每个零位屏蔽已配置的 ID 过滤器的相应位位置。

注：如果所有位均等于 1，则仅当接收到的消息 ID 和消息 ID 过滤器相同时，才会发生匹配。 如果所有掩码位均等于 0，所有消息 ID 均匹配。

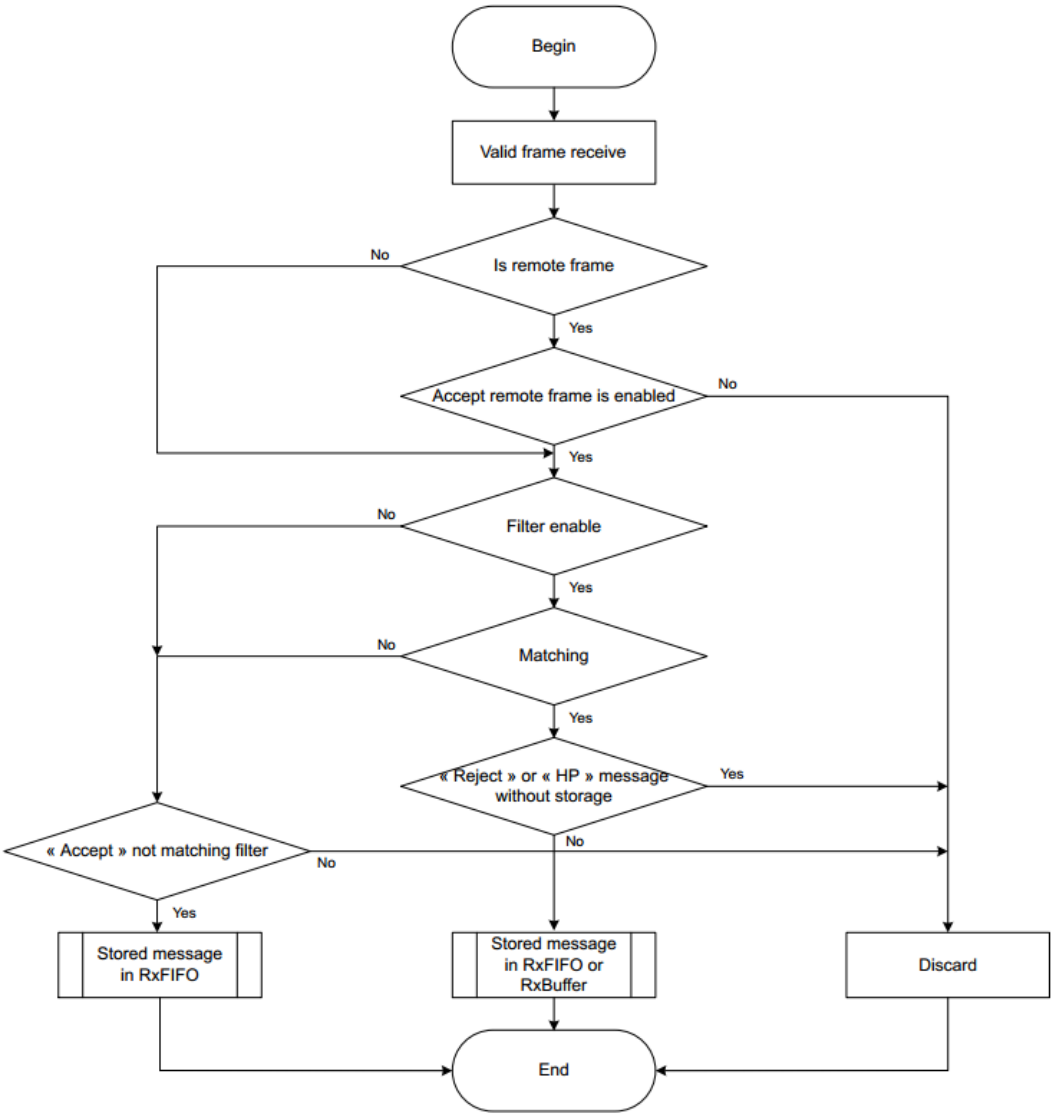
90.6.1 高优先级消息

当收到高优先级消息时，FDCAN 可以通知用户。 此通知可用于监视传入的高优先级消息的状态并启用对这些元素的快速访问。FDCAN 在消息过滤器的帮助下检测到高优先级消息。 过滤器元素提供以下内容与高优先级邮件相关的设置：

（1）设置优先级并在过滤器匹配的情况下存储在 FIFO 0/1 中：如果此消息过滤器匹配，则 FDCAN 通知高优先级消息到达的信息，并将元素存储在 Rx FIFO 0/1 中。

（2）如果过滤器匹配，则设置优先级：如果此消息过滤器匹配，则 FDCAN 会通知有关高优先级的信息消息到达，但不存储元素。

接收过滤流程图：



90.6.2 举例说明

为了说明可以使用的不同类型的过滤器以及每种类型的结果，我们假设用户想要配置 FDCAN：

（1）拒绝所有标识符在[0x16 到 0x20]范围内的消息

（2）接受所有标识符等于 0x15 或 0x120 的消息，并将它们存储在 FIFO 1 中

（3）接受标识符等于 0x130 的消息并将其存储在 Rx 缓冲区索引 4 中

（4）接受具有与以下内容相对应的标识符的消息：

–bit[10..6] = 0b111 00

–bit[5..4] = 不管，任意值

–bit[3..0] = 0b00000

在这种情况下，必须将过滤器配置为经典位屏蔽过滤器，因为可接受的标识符对应于 0b11100XX0000 (其中 x 可以是 0 或 1 中的任何值）。 可接受的标识符是：

– 0b111 0000 0000（0x700）
– 0b111 0001 0000（0x710）
– 0b111 0010 0000（0x720）
– 0b111 0011 0000（0x730）

基本过滤器 ID 可以是 0x700、0x710、0x720、0x730 中的任何值。屏蔽过滤器 ID 等于 0b111 1100 1111（0x7CF）。下表介绍了标准 11 位消息 ID 过滤器的不同配置，如上面的例子。 每个标准过滤器元素包含：

（1）SFT 位（标准过滤器类型）
（2）SFEC 位（标准过滤器元素配置）。
（3）SFID1 位（标准过滤器 ID1）
（4）SFID2 位（标准过滤器 ID2）

Filter	Standard filter type SFT [31:30]	Standard filter element configuration SFEC [29:27]	Standard filter ID 1 SFID1 [26:16]	Standard filter ID 2 SFID2 [15:0]
First	00 - Range filter	011 - Reject	0x16	0x20
Second	01 - Dual ID	010 - Store in FIFO 1	0x15	0x120
Third	xx - Don't care	111 - Store in Rx buffer	0x130	0x04 (buffer index)
Fourth	10 - Classic bit mask filter	001 - Store in FIFO 0	0x700	0x7CF

第一个过滤器配置为拒绝 ID 为[0x16 ... 0x20]范围内的消息。
第二个过滤器配置为将 ID 等于双 ID 0x15 或 0x120 的消息存储在 Rx FIFO 1 中。
第三过滤器被配置为将 ID 等于 0x130 的消息存储在 Rx 缓冲区索引 4 中。

注意：如果将 SFEC 配置为“存储到"Rx buffer”，则将忽略 SFT 的配置。 验收过滤器停在第一次匹配。 因此，筛选器的顺序很重要。本示例以与用户配置扩展过滤器相同的方式配置标准过滤器（请参阅产品数据表以了解更多详细信息）。FDCAN 的众多过滤器可能性允许在硬件中进行复杂的消息过滤，这使得软件过滤冗余并节省 CPU 资源。

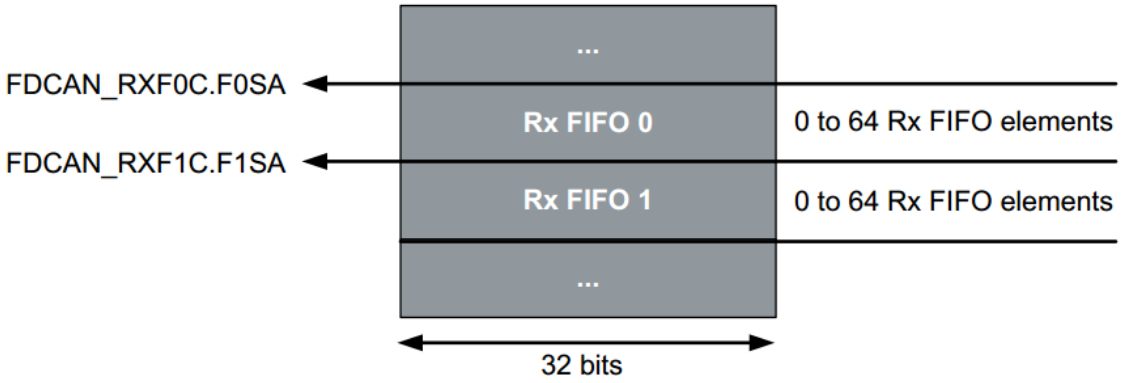
90.7 CAN FD 接收区

90.7.1 Rx FIFO 0 和 Rx FIFO 1

可以在 CAN 消息 RAM 中配置两个 Rx FIFO。 每个 Rx FIFO 部分最多可存储 64 个元素。每个元素存储在一个 Rx FIFO 元素中。可以通过 FDCAN_RXESC 寄存器分别为每个 Rx FIFO 配置 Rx FIFO 元素的

大小。Rx FIFO 元素大小定义了可以存储一个接收元素的数据字段字节数。 一个大小 Rx FIFO 元素由前面 RAM 管理中指定的公式定义。

标头信息包含标识符，DLC 字段，控制位和位字段（过滤器索引，可接受的不匹配帧，Rx 时间戳）。通过 FDCAN_RXESC 寄存器中的 F1DS [2：0]字段配置元素大小后，Rx FIFO 1 的元素个数和起始地址必须分别通过 FDCAN_RXF1C 寄存器中的 F1S [6：0]和 F1SA [13：0]进行配置。下图显示了 CAN 消息 RAM 上的 Rx FIFO 部分，其中包含可以支持和每个部分的开始地址。



Rx FIFO 的起始地址是第一个 Rx FIFO 元素的第一个字的地址。收到的元素通过匹配过滤的数据将根据匹配的过滤器元素存储在适当的 Rx FIFO 中。如果 Rx FIFO 已满，则可以根据两种不同模式来处理新到达的元素：

（1）阻塞模式：这是 Rx FIFO 的默认操作模式，没有其他元素写入 RxFIFO，直到至少一个元素已被读出。

（2）覆盖模式：Rx FIFO 中接受的新元素将覆盖 Rx FIFO 中最旧（最先接收的数据）的元素并且 FIFO 的 put 和 get 索引加 1。

要从 Rx FIFO 读取元素，CPU 必须执行以下步骤：

（1）读取寄存器 FDCAN_RXF1S 以了解 Rx FIFO 的状态。

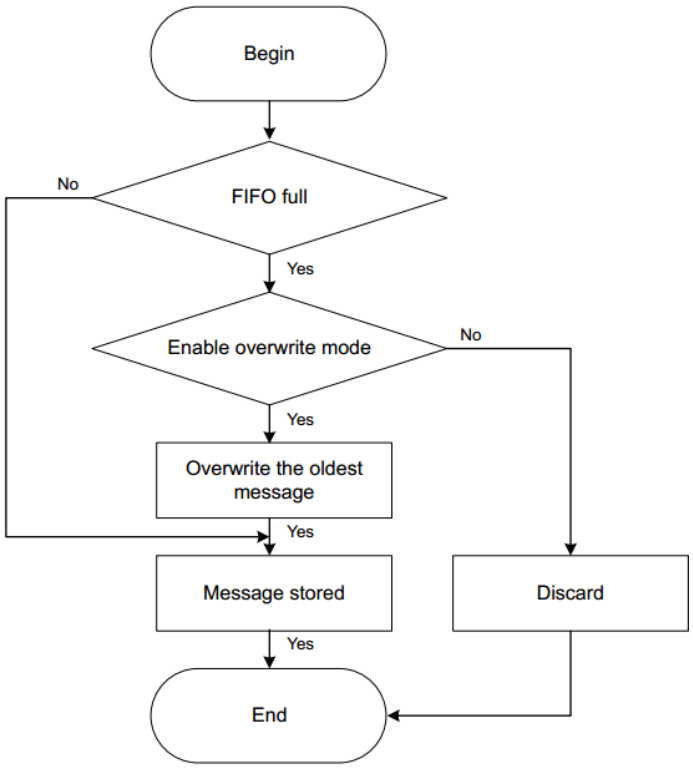
（2）按照以下公式计算 RAM 中最旧的元素的地址：

Oldest element address = CAN_message_RAM_base_address + FDCAN_RXF1C.F1SA (start address) + FDCAN_RXF1S.F1GI (get Index) x Rx FIFO_element_size.

（3）从计算出的地址中读取元素。

CPU 从 Rx FIFO 读取一个元素或一系列元素后，它必须确认读取。确认后，FDCAN 可以将相应的 Rx FIFO 缓冲区重新用于新元素。为了确认一个或多个元素，则 CPU 必须将从 Rx FIFO 读取的最后一个元素的缓冲区索引写入 FDCAN_RXF1A 寄存器。结果，FDCAN 更新了 FIFO 填充级别和 get 索引。

RX FIFO 流程：

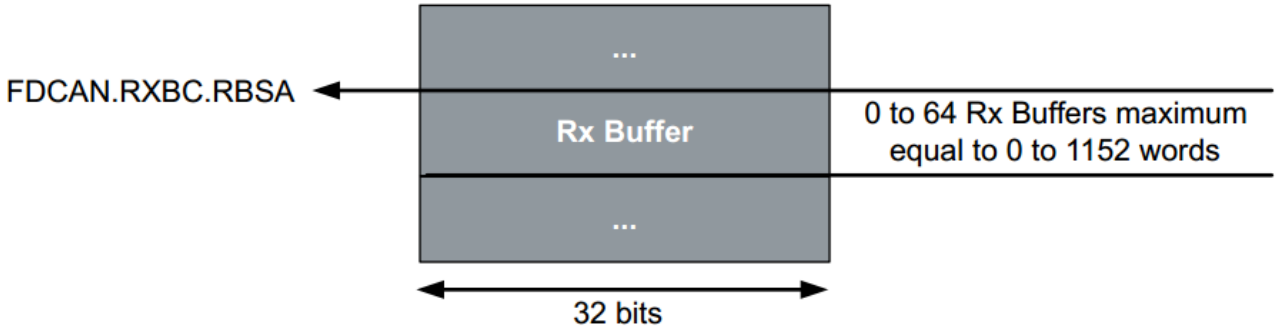


Rx FIFO 0 和 Rx FIFO 1 具有相同的寄存器，它们的名称有意义，方法是更改每次的 FIFO 数量。

90.7.2 专用的 Rx buffer

FDCAN 支持多达 64 个专用 Rx buffers。每个专用的 Rx buffers 可以存储一个元素。专用 Rx buffers 的大小可以通过 FDCAN_RXESC 寄存器进行配置。 Rx buffers 大小定义可以存储多少个接收元素的数据字段字节。 专用 Rx buffers 的大小由前面 RAM 组织中描述的公式定义。

通过 FDCAN_RXESC 寄存器中的 RBDS [2：0]字段配置元素大小后，启动必须通过 FDCAN_RXBC 寄存器中的 RBSA [13：0]字段配置地址。下图显示了 CAN 消息 RAM 上的 Rx buffers 部分，其中有可以支持的最大数量的 Rx buffers 元素和起始地址。



当将元素存储在专用 Rx 缓冲区中时，FDCAN 通过 FDCAN_IR 寄存器的 DRX 位设置中断标志以及新数据标志 FDCAN_NDAT1 或 FDCAN_NDAT2 寄存器中的相应位。当 FDCAN_NDAT1 / 2 中的位置 1 时，

相应的 Rx 缓冲区将被锁定（不会被新元素覆盖），并且相应的过滤器不匹配。 读取元素后，CPU 必须将相应的位复位 FDCAN_NDAT1 / 2，以解锁相应的 Rx 缓冲区。

要从专用 Rx 缓冲区读取元素，CPU 必须执行以下步骤：

（1）检查 FDCAN_NDAT1 / 2 中的位，以了解是否有新元素到达专用 Rx 缓冲区。

（2）计算 CAN 消息 RAM 中元素的地址，如以下公式所示：

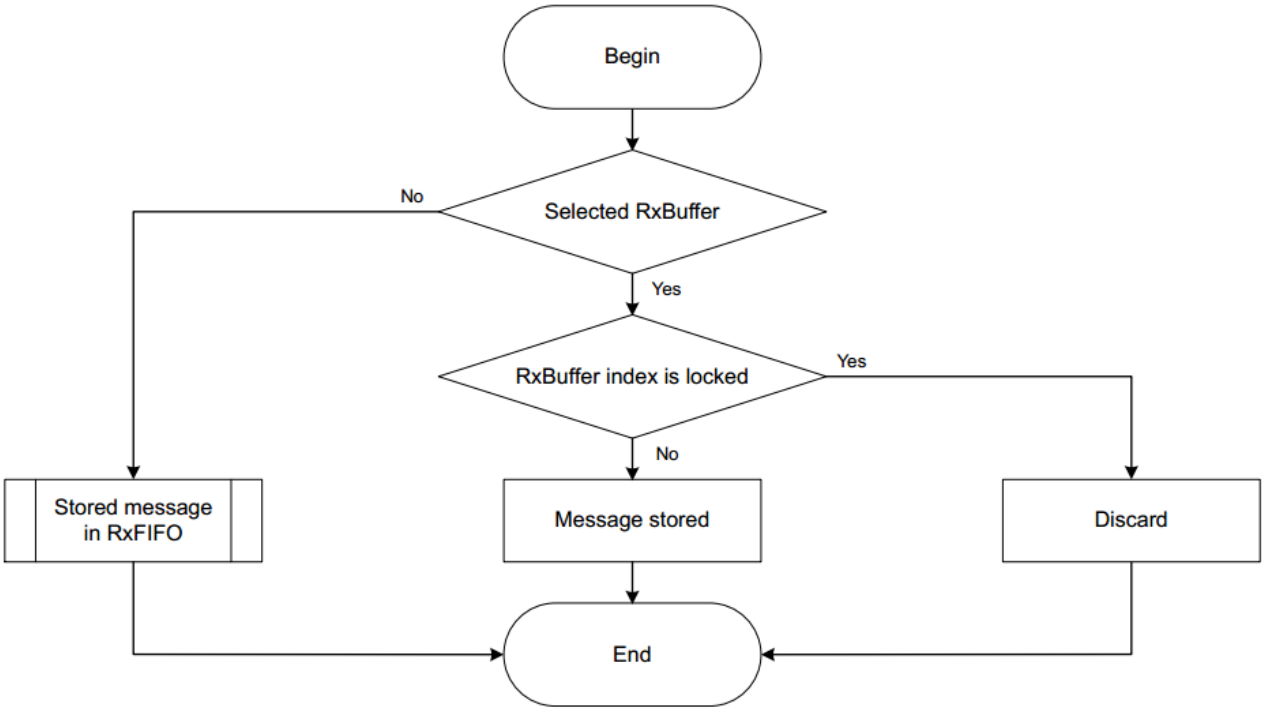
Reference Rx buffer address = CAN_message_RAM_base_address + FDCAN_RXBC.RBSA (startaddress) + dedicated Rx buffer index x Rx_Buffer_element_size.

（3）从计算出的地址中读取消息。

过滤器元素可以引用 Rx 缓冲区索引（0 到 63）作为接收元素的目的地。如果相应的过滤器匹配，则 FDCAN 仅执行对引用的 Rx 缓冲区位置的写操作。 换一种说法，FDCAN 不会写入未引用的 Rx 缓冲区位置。

90.7.3举例说明（含 Rx FIFO 和 Rx buffer 的差异)

为了配置过滤器元素以引用 Rx 缓冲区索引 60，必须至少配置 61 个 Rx 缓冲区。注意：用户必须选择最佳配置，以避免浪费 RAM。下图显示了简化 Rx 缓冲区操作的流程图：



如前面所述，FDCAN 具有两种机制：专用 Rx buffer 或 Rx FIFO 0/1 可以将配置为存储接收到的元素。下表描述了专用 Rx 缓冲区和 Rx FIFO 之间的差异。

Feature	Dedicated Rx buffer	Rx FIFO
Sections to configure	64 dedicated Rx buffers can be configured .	Two Rx FIFO can be configured.
Elements per section	Configured to contain only one element per buffer	May contain one or more elements (up to 64 elements) per section
Position in the RAM	The user chooses the buffer index.	The position in the RAM is automatically and dynamically managed (using the incrementing of get/put index).
Discards newly arriving element configuration	Discards the newly arriving element when buffer is locked. *Note: The user must reset the corresponding bit in FDCAN_NDAT1/2.*	Discards the newly arriving element when Rx FIFO is full (blocking mode by default). *Note: Overwrite mode option to receive the new element and overwrite the oldest element.*

90.8 CAN FD 发送区

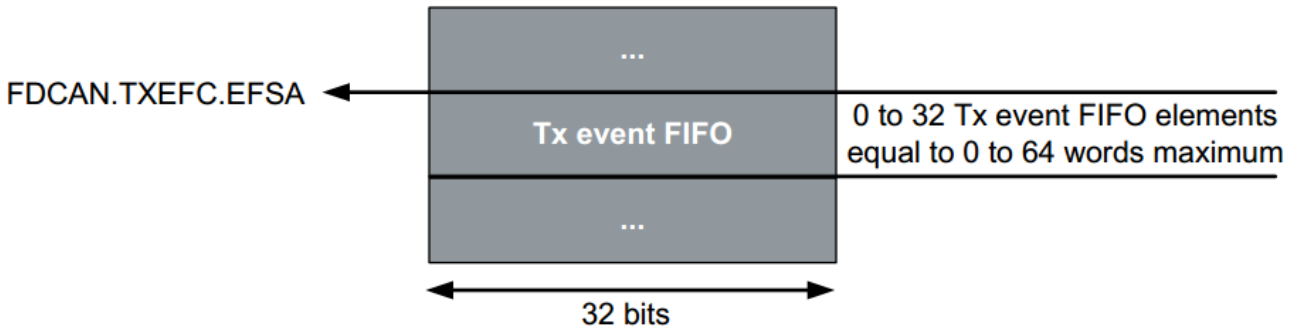
90.8.1 Tx event FIFO 区

通过使用 Tx event FIFO，CPU 获得有关已发送元素的以下信息：

（1）元素传输的顺序.

（2）每个元素传输帧的本地时间。

FDCAN 提供发送 event FIFO。 该 Tx event FIFO 的使用是可选的。 FDCAN 在 CAN 总线上传输了一个元素成功后，它可以将消息 ID 和时间戳存储在 Tx event FIFO 中元素。 Tx event FIFO 元素是一种数据结构，用于存储已传输消息。 Tx 可以通过 FDCAN_TXEFC 寄存器配置事件 FIFO（Tx 事件 FIFO 配置）。 FIFO 可以存储最多 32 个元素。下图显示了存储 Tx 事件 FIFO 元素的 CAN 消息 RAM 的示例 EFSA [13：0]字段包含起始地址。



CAN 消息 RAM 中的 Tx 事件 FIFO 元素的地址由以下公式确定：

Tx event FIFO element address = CAN_message_RAM_base_address + FDCAN_TXEFC.EFSA (startaddress) + FDCAN_TXEFS.EFGI (get index) x Tx_event_FIFO_element_size

为了将 Tx event 链接到 Tx event FIFO 元素，将来自已发送 Tx buffer 的消息标记复制到 Tx event FIFO 元素。

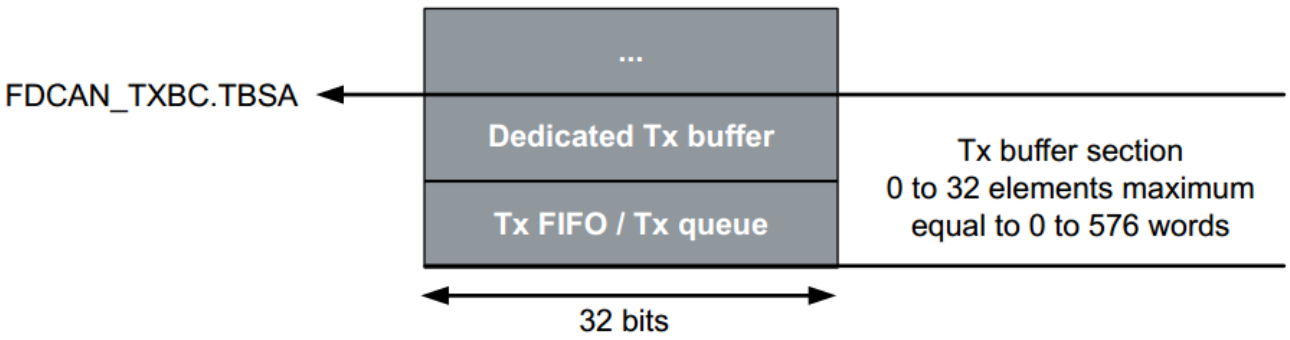
仅当 Tx buffer 元素中的 EFC 位（存储 Tx 事件）等于 1 时，事件才存储在 Tx event FIFO 中。当 Tx event FIFO 已满时，不会再有其他元素写入 Tx event FIFO，直到至少有一个元素被写入为止。读出后，Tx event FIFO 获取索引增加。如果在 Tx event FIFO 已满时发生 Tx event，则这事件被丢弃。为避免 Tx event FIFO 溢出，可以使用 Tx event FIFO 水印。CPU 从 Tx event FIFO 读取一个元素或元素序列后，CPU 必须确认阅读。因此，它会通过 FDCAN_TXEFA 寄存器 EFAI [4：0]字段写入从 Tx event FIFO 读取的最后一个元素的索引。

90.8.2 Tx buffer 区（含 Tx buffer，Tx FIFO 和 Tx queue)

为了使外设传输元素，该元素在定义的存储空间内，并且传输开始。 传输的元素存储在 Tx buffer 中，用户可以选择使用的机制：专用的 Tx buffer 或 Tx queue 或 Tx FIFO。FDCAN 最多支持 32 个元素。每个元素存储标识符，DLC，控制位（ESI，XTD，RTR，BRS，FDF），数据字段，位字段消息标记和事件 FIFO 控制位，仅一条消息。

在 RAM 上的分配按以下顺序进行：如果应用程序使用了专用的 Tx buffer，则它们在 Tx FIFO 和 Tx queue.之前分配。 用户只能在同一队列中选择 Tx queue 或 Tx FIFO 应用程序：FDCAN 不支持它们的组合。

如下图所示，Tx buffer 的起始地址是通过 FDCAN_TXBC 寄存器的 TBSA [13：0]字段配置：



如前一节所述，RAM 中的分配是以动态且连续的方式进行的，因此，如果用户未配置 Dedicated Tx buffer。 Tx buffer 部分仅包含配置的 Tx queue 或 Tx FIFO，并将其存储在起始段地址中。

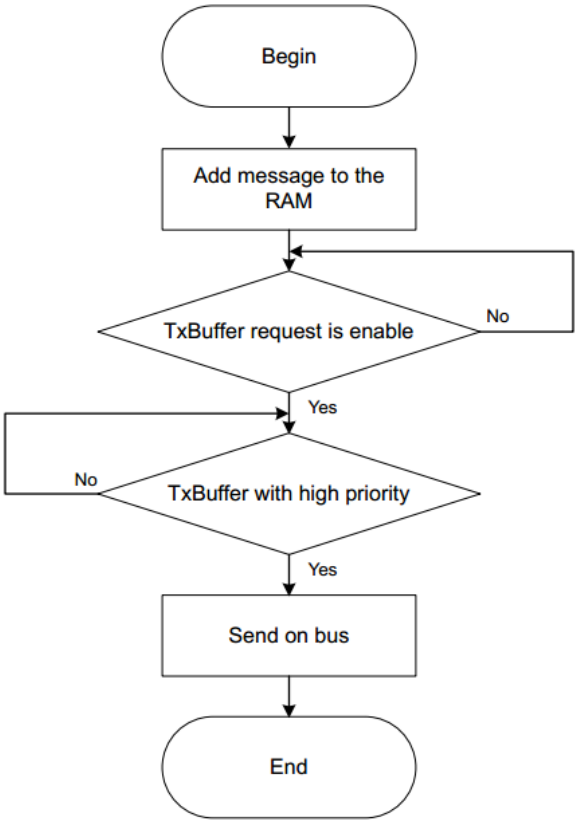
90.8.3 专用 Tx buffers 区（Dedicated Tx buffers)

专用 Tx buffer 的数量是通过 FDCAN_TXBC 寄存器中的 NTDB [5:0]字段配置的。每个专用 Tx buffer 配置有特定的标识符，以仅存储一个元素。 传输是通过 FDCAN_TXBAR 寄存器的添加请求。 所请求的消息在外部进行仲裁的 CAN 总线上，并根据最低标识符（最高优先级）发送出去。专用 Tx buffer 的内

存要求取决于 Tx buffer 元素的大小。Tx buffer 元素大小定义了属于 Tx buffer 的数据字节数。使用以下公式计算 CAN 消息 RAM 中专用 Tx 缓冲区的地址：

Dedicated Tx buffer address = CAN_message_RAM_base_address + FDCAN_TXBC[TBSA] (start address)+ Tx_buffer_index x Tx_buffer_element_size

Tx buffer 操作机制：



如果多个专用 Tx buffer 配置了相同的 ID，则具有第一个传输请求的 Tx buffer 首先发送。

90.8.4Tx FIFO

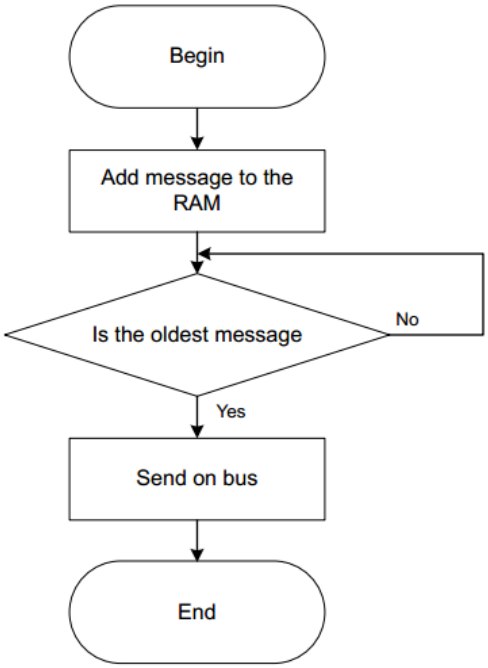
通过将 0 写入 FDCAN_TXBC 中的 TFQM 位来配置 Tx FIFO 操作。元素存储在通过 get 索引中 FDCAN_TXFQS 的 TFG1 [4：0]字段，从 get 索引引用的元素开始发送 Tx FIFO。每次发送后，get 索引都会循环递增，直到 Tx FIFO 缓冲区为空。Tx FIFO 可以按照已将元素写入 Tx FIFO 的顺序传输元素。传输与各个标识符的优先级无关，因为 FIFO 首先发送出去。

FDCAN 通过 FDCAN_TXFQS 中的 TFFL [5：0]字段计算 Tx FIFO 缓冲区的空闲级别，作为 get 和 put 索引之间的差值（get 和 set 索引之间有两种机制递增每个事务以指示下一个事务 RAM 上要读取或写入元素的缓冲区位置）。该值指示可用（空闲）Tx FIFO 的数量元素。

必须从放置索引引用的 Tx 缓冲区开始，将新的发送元素通过 FDCAN_TXFQS 中的 TFAQPI [4：0]字段指示写入 Tx FIFO。CAN 消息 RAM 中的下一个空闲 Tx FIFO 缓冲区的地址是使用以下公式计算的：

Next free Tx FIFO buffer address = CAN_message_RAM_base_address + FDCAN_TXBC.TBSA (startaddress) + FDCAN_TXFQS.TFQPI (put Index) x Tx_FIFO_element_size

Tx FIFO 机制：



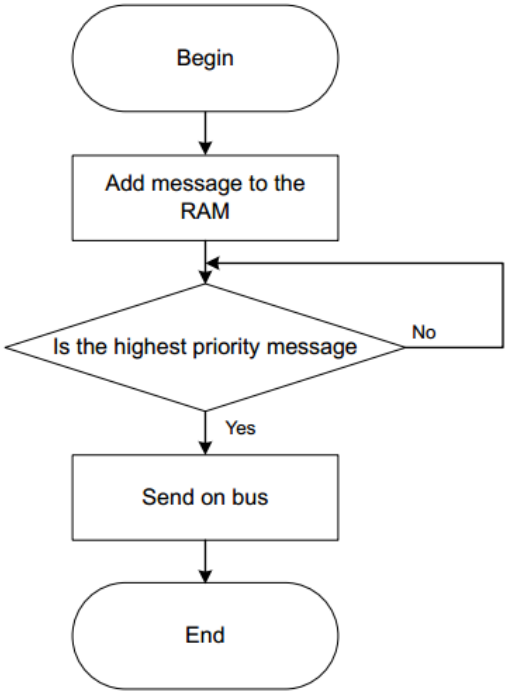
90.8.5 Tx queue

通过将 1 写入 FDCAN_TXBC 中的 TFQM 位来配置 Tx 队列操作。 Tx 中存储的元素从具有最低标识符（最高优先级）的 Tx 队列缓冲区开始传输队列。与专用 Tx 缓冲区相比，RAM 上的位置是自动动态管理的，因此消息标识符不固定为预定义的 Tx 缓冲区索引。

新消息必须写入到放置索引引用的 Tx 队列缓冲区中。周期性添加请求将 put 索引递增到下一个空闲 Tx 队列缓冲区。FDCAN_TXFQS 中的 TFQF 位设置为 1 表示 Tx 队列已满。在至少一个以下元素之前，不得将其他元素写入 Tx 队列中。发送请求的元素或取消挂起的传输请求。Tx 队列缓冲区的内存要求取决于属于 Tx 队列的数据字节数元素。CAN 消息 RAM 中下一个可用的空闲 Tx 队列缓冲区的地址可以通过以下方式计算：

Next free Tx queue buffer address = CAN_message_RAM_base_address + FDCAN_TXBC.TBSA (startaddress) + FDCAN_TXFQS.TFQPI (put index) x Tx_Buffer_element_size

Tx queue 机制



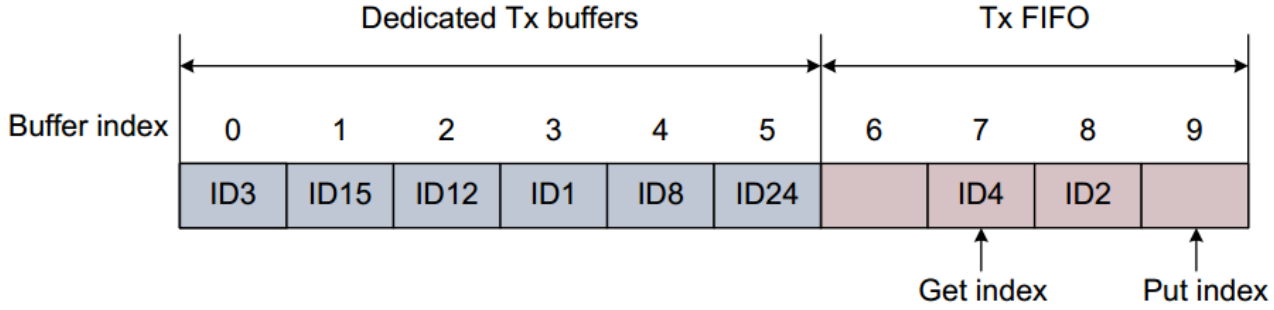
90.8.6 Tx FIFO , dedicated Tx buffer, and Tx queue 的区别

Feature	Dedicated Tx buffer	Tx FIFO	Tx queue
Sections that can be configured	32 dedicated Tx buffers can be configured.	One Tx FIFO can configured.	One Tx queue can configured.
Elements per section	Tx buffer is configured to contain only one element per buffer.	May contain one or more elements per section (up to 32)	May contain one or more elements per section (up to 32)
Element to be sent first	Element with the lowest ID is sent first.	Elements transmitted in the FIFO order	Element with the lowest ID is sent first.
Behaviour if multiple elements with the same ID are present	The first transmission request is sent first.	No prioritization with identifier	FIFO order
Position in the RAM	User chooses the buffer index.	Automatically and dynamically managed (using the incrementing of *get* and *put* index)	
Pending elements management	Elements become pending after being added to the RAM or after the tx buffer request is enabled.	Elements become pending after being added to the RAM (the request is automatically enabled).	

90.8.7 Tx FIFO 和 dedicated Tx buffers 混合配置

可以使用混合配置来配置 CAN 消息 RAM 的 Tx 缓冲区部分，其中 CAN 消息 RAM 中的 Tx 缓冲区部分可细分为一组专用 Tx 缓冲区和一个 Tx FIFO。专用 Tx 缓冲区的数量是通过 FDCAN_TXBC 中的 NDTB [5：0]配置的。 Tx 缓冲区的数量通过 FDCAN_TXBC 中的 TFQS [5：0]配置分配给 Tx FIFO 的地址。

Tx 处理程序使用 activated transmission request 和 the oldest pending 扫描所有专用 Tx 缓冲区 get 索引引用的 Tx FIFO 缓冲区。 具有最低标识符的缓冲区将获得最高优先级，并且是接下来传送。下图说明了使用混合专用 Tx 缓冲区和 Tx FIFO 的用例：

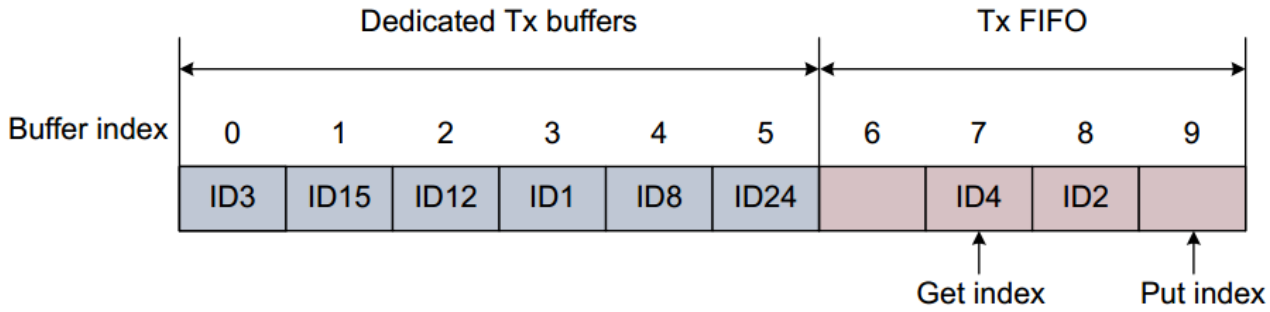


在此示例中，元素按以下顺序发送（假设所有专用 Tx 缓冲区请求已启用）：

1. Tx buffer 3 (identifier = 1: it is the highest priority between all other dedicated Tx buffers and it has a higher priority than the oldest pending Tx FIFO: Tx buffer 7)
2. Tx buffer 0 (identifier = 3: it is the highest priority between all other dedicated Tx buffers and it has higher priority than the oldest pending Tx FIFO: Tx buffer 7)
3. Tx buffer 7 (because it is the oldest pending Tx FIFO with identifier =4 and has higher priority between all dedicated Tx buffers)
4. Tx buffer 8 (because it is the oldest pending Tx FIFO with identifier =2 and has the highest priority between all dedicated Tx buffers)
5. Tx buffer 4 (identifier = 8: it has the highest priority between all other dedicated Tx buffers and the Tx FIFO is empty)
6. Tx buffer 2 (identifier = 12: it has the highest priority between all other dedicated Tx buffers and the Tx FIFO is empty)
7. Tx buffer 1 (identifier = 15: it has the highest priority between all other dedicated Tx buffers and the Tx FIFO is empty)
8. Tx buffer 5 (because it is the only pending dedicated Tx buffer)。

90.8.8 Tx queue 和 dedicated Tx buffers 混合配置

可以使用混合配置来配置 CAN 消息 RAM 的 Tx buffer，其中 CAN 消息 RAM 中的“ Tx 缓冲区”部分可细分为一组专用 Tx buffer 和一个 Tx queue。通过 FDCAN_TXBC 中的 NDTB [5：0]配置专用 Tx buffer 的数量和 Tx queue 的数量缓冲区是通过 FDCAN_TXBC 中的 TFQS [5：0]配置的。Tx 处理程序扫描具有激活的传输请求的所有专用 Tx 缓冲区和 Tx 队列缓冲区。标识符最低的缓冲区获得最高优先级，然后发送。下图说明了使用混合专用 Tx 缓冲区和 Tx 队列的用例。



在此示例中，元素按以下顺序发送（假设所有专用 Tx 缓冲区请求已启用）：

1. Tx buffer 3 (identifier = 1: it is the highest priority between all other dedicated Tx buffers and it has a higher priority than the oldest pending Tx FIFO: Tx buffer 7)
2. Tx buffer 0 (identifier = 3: it is the highest priority between all other dedicated Tx buffers and it has higher priority than the oldest pending Tx FIFO: Tx buffer 7)
3. Tx buffer 7 (because it is the oldest pending Tx FIFO with identifier =4 and has higher priority between all dedicated Tx buffers)
4. Tx buffer 8 (because it is the oldest pending Tx FIFO with identifier =2 and has the highest priority between all dedicated Tx buffers)
5. Tx buffer 4 (identifier = 8: it has the highest priority between all other dedicated Tx buffers and the Tx FIFO is empty)
6. Tx buffer 2 (identifier = 12: it has the highest priority between all other dedicated Tx buffers and the Tx FIFO is empty)
7. Tx buffer 1 (identifier = 15: it has the highest priority between all other dedicated Tx buffers and the Tx FIFO is empty)
8. Tx buffer 5 (because it is the only pending dedicated Tx buffer)

不支持带有 Tx FIFO 和 Tx 队列的混合配置。

dedicated buffer + Tx FIFO 和 Tx buffer + Tx queue 的区别如下：

Feature	Mixed configuration: Tx buffer + Tx queue	Mixed configuration: Tx buffer + Tx FIFO
Description	A combination of dedicated Tx buffers and Tx queue	A combination of dedicated Tx buffers and Tx FIFO
Element to be sent first	Element with the lowest ID between all dedicated Tx buffers and Tx queue	Element with the lowest ID between all dedicated Tx buffers and oldest element in Tx
Management of multiple elements with same ID	First request is sent first.	Order request FIFO

在 Tx FIFO + Tx 队列中，元素在“添加到 RAM”操作之后立即变为挂起状态。

90.9 CAN FD 测试模式

在 FDCAN 的操作模式下，除了正常操作外，还有几种测试模式可用。 那里测试模式只能用于生产测试或自测以及校准单元。必须将 FDCAN_CCCR 中的 TEST 位设置为 1，以允许对 FDCAN 测试寄存器和存储器的写访问。测试模式和功能的配置。FDCAN 以下列模式之一工作：

（1）限制操作模式 Restricted-operation mode

（2）总线监控模式 Bus-monitoring mode

（3）外部环回模式 External loop-back mode

（4）内部环回模式 Internal loop-back mode

90.9.1 限制操作模式 Restricted-operation mode

在受限操作模式下，FDCAN 能够：

--接收数据帧

--接收远程帧

--确认有效帧

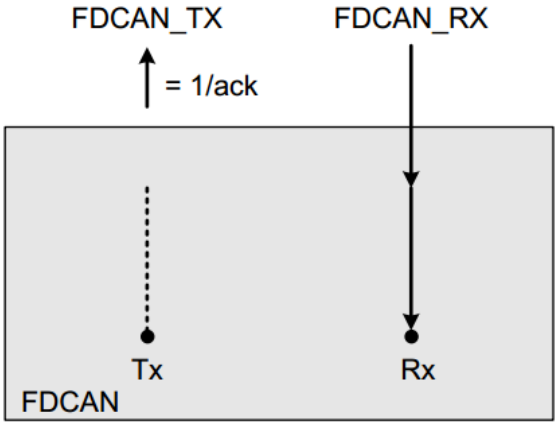
该模式不支持：

--数据帧发送

--远程帧发送

--活动错误帧或过载帧发送

通过 FDCAN_CCCR 中的 ASM 位将 FDCAN 设置为受限操作模式。当 Tx handler 无法从 CAN 读取消息 RAM 准时或时钟校准激活时，将自动进入受限操作模式。在这种模式下，应用程序测试不同的比特率，并在应用程序收到有效的帧之后退出受限操作模式。下图说明了在受限操作模式下 FDCAN_TX 和 FDCAN_RX 引脚的连接：



注：只要 FDCAN 处于受限操作模式，FDCAN_TX 引脚就处于隐性状态。 显性位被传输，以此确认接收到有效帧。

90.9.2 总线监控模式 Bus-monitoring mode

为了分析总线上的通信量而又不受主要位传输的影响，用户可以设置 FDCAN_CCCR 中的 MON 位，可以在总线监视模式下使用 FDCAN。在总线监视模式下，FDCAN 能够：

--接收有效的数据帧

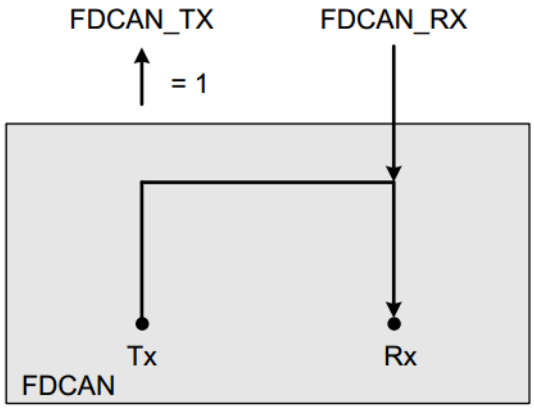
--接收有效的远程帧

该模式不支持：

--传输开始

--确认有效帧（与受限操作模式不同）

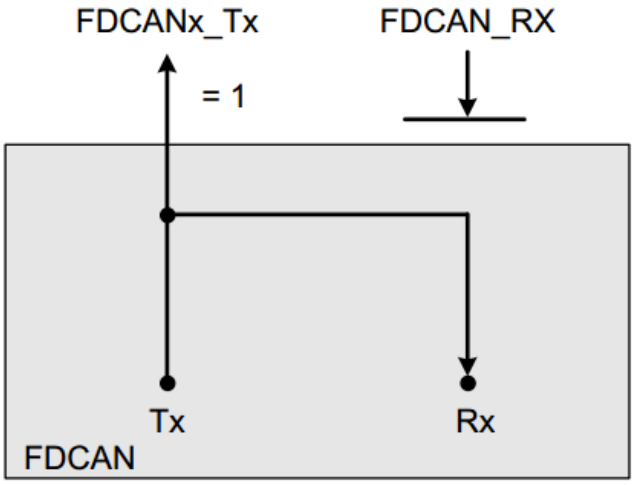
在总线监视模式下，FDCAN 仅在总线上发送隐性位。 下图显示了总线监视模式下 FDCAN_TX 和 FDCAN_RX 引脚的连接。



90.9.3 外部环回模式 External loop-back mode

提供此模式用于硬件自检。 用户可以通过以下方式将 FDCAN 设置为外部环回模式：将 1 写入 FDCAN_TEST 中的 LBCK 位，并将 0 写入 FDCAN_CCCR 中的 MON 位。 FDCAN 对待自己的发送的消息作为已接收的消息，如果它们通过 Rx FIFO 中的接受过滤，则将其存储。为了独立于外部信号，FDCAN 忽略确认错误（隐性位在确认插槽中采样）。 FDCAN 实现内部 TX 和 RX 连接。

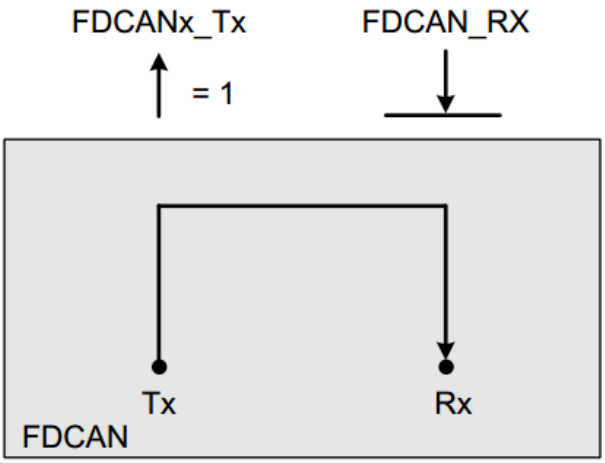
下图显示了外部环回模式下的 FDCAN_TX 和 FDCAN_RX 引脚的连接。



90.9.4内部环回模式 Internal loop-back mode

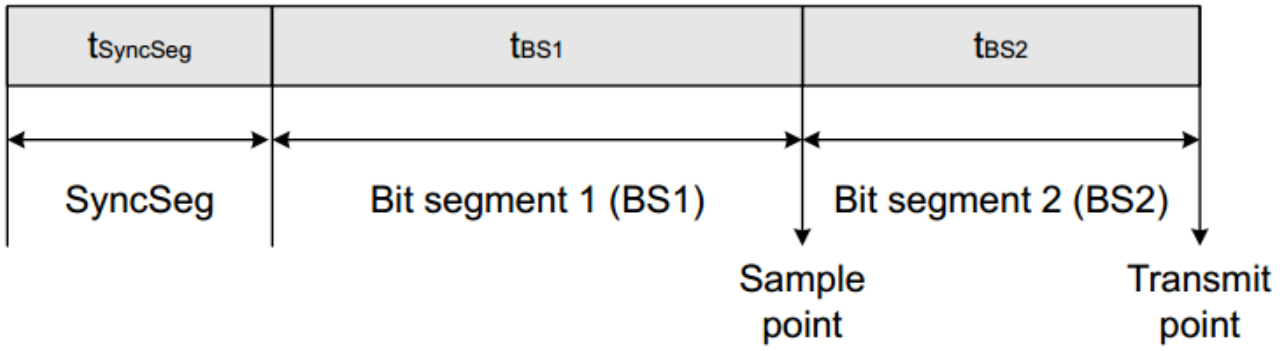
提供此模式用于硬件自检。用户可以通过以下方式将 FDCAN 设置为内部环回模式:向 FDCAN_TEST 中的 LBCK 位写入 1，并向 FDCAN_CCCR 中的 MON 位写入 1。可以测试 FDCAN，而不会影响连接到 FDCAN_TX 和 FDCAN_RX 的正在运行的 CAN 系统针脚。 FDCAN_RX 引脚与 FDCAN 断开连接，并且 FDCAN_TX 引脚保持隐性。

下图显示了内部环回模式下 FDCAN_TX 和 FDCAN_RX 引脚的连接:



90.9.5收发延迟器补偿

在采样点，所有发送器都检查先前发送的位是否被正确采样。 这需要一种机制来检查问题并检测其他节点错误帧。由于发射器看到了自己的发送位由收发器环路延迟而延迟,此延迟为 TSEG1 设置了下限，如图所示。下图(采样点之前的时间段),这也是数据比特率的上限。 这是引入收发器延迟补偿机制(TDC)的原因。

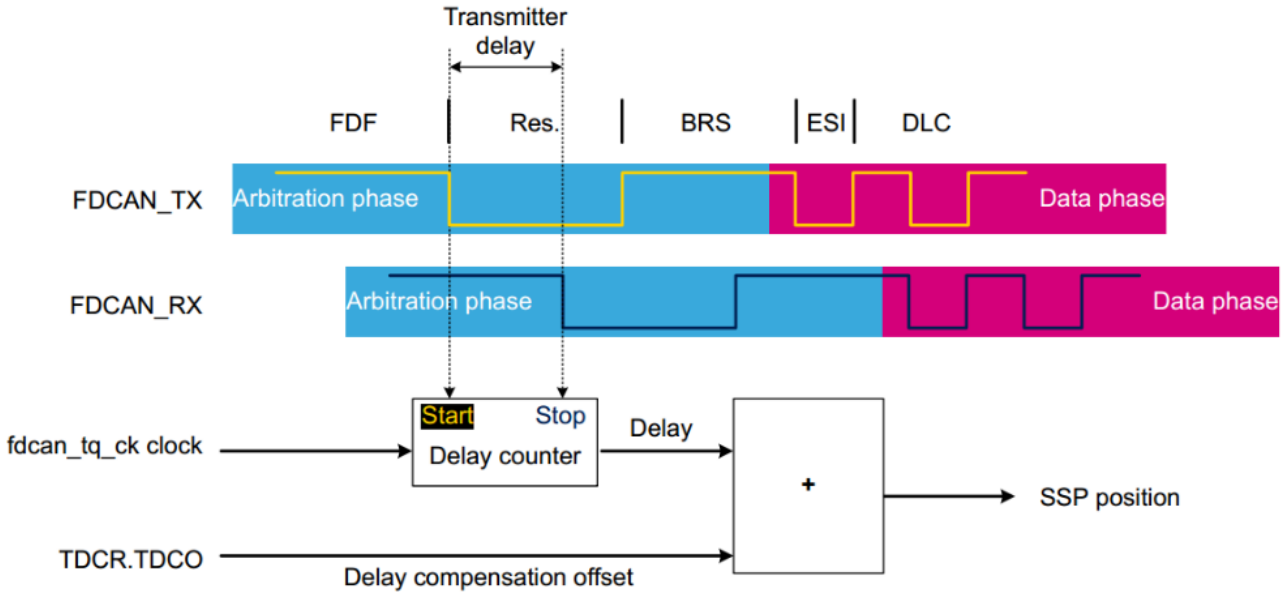


为了在检查误码时补偿此环路延迟，定义了一个辅助采样点（SSP）。而不是在采样点完成，而是在 SSP 中检查传输的位。 该检查的结果是直到达下一个采样点为止。

为数据阶段中发送的每个位生成一个 SSP。 收发器不对称和总线振铃必须考虑 SSP 位置，但是没有时钟容限，因为收发器会监视自己的位置位流。

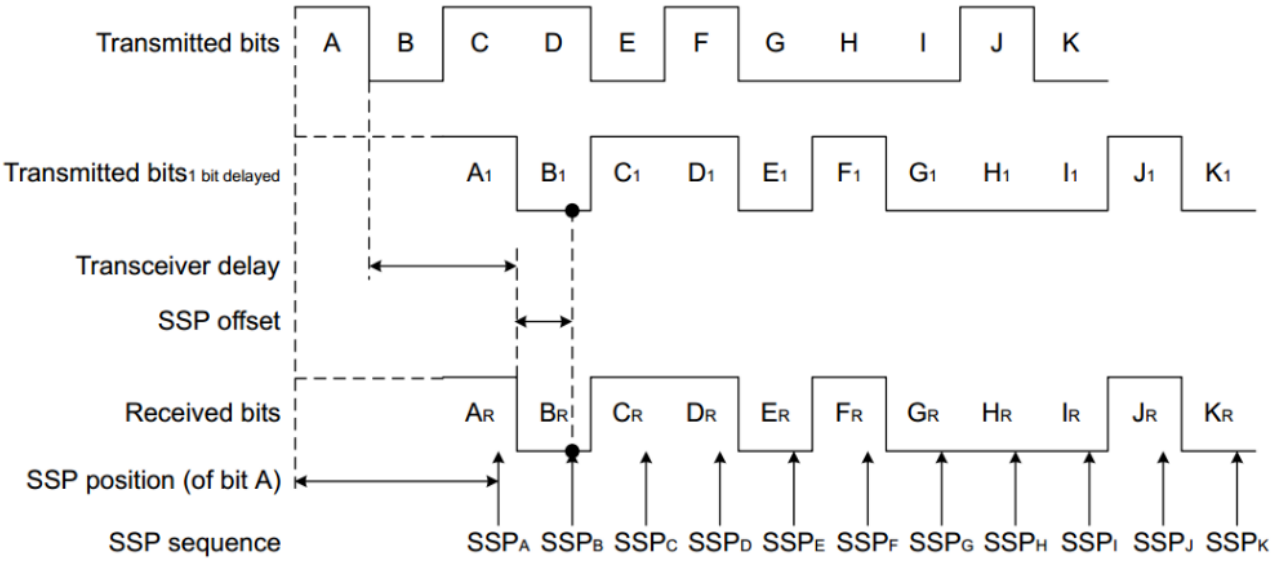
通过将 1 写入 FDCAN_DBTP 中的 TDC 位来启用收发器延迟补偿。 测量在数据阶段开始之前（在信号的下降沿之前）在每个发送的 FDCAN 帧内开始 FDF 逐位解析。 当在示波器的“接收”输入引脚 FDCAN_RX 看到该边沿时，测量将停止。 此测量的分辨率为 1 mtq（最小时间量）

下图显示了环路延迟的测量：



注 1：在仲裁阶段，始终禁用延迟补偿。SSP 位置定义为从 FDCAN_TX 引脚到 FDCAN_RX 引脚测得的延迟之和，加上通过 FDCAN_TDCR 中的 TDCO [6：0]字段配置的发送器延迟补偿偏移。

注 2：发送器延迟补偿偏移量用于调整 SSP 在接收位内部的位置。存储已发送位的值，直到达到其 SSP，然后将其与实际接收到的位进行比较，如下图所示，该值表示发送的比特序列 A 到 K 和接收的比特序列 AR 到 KR，以及从 SSPA 到 SSPK 的一系列 SSP。 通过比较 SSBR 来检查接收到的比特 BR 与延迟位 B1。 SSPB 的位置在发送的比特 B 开始之后的特定时间。特定时间是测得的收发器延迟与配置的 SSP 偏移之和。



根据博世（Bosch）文档确定，必须考虑以下边界条件 FDCAN 中实现的发射机延迟补偿：

----从 FDCAN_Tx 到 FDCAN_Rx 测得的延迟与已组态的发送器延迟之和在数据阶段，补偿偏移量必须小于六位时间。

----从 FDCAN_Tx 到 FDCAN_Rx 测得的延迟与已组态的发送器延迟之和补偿偏移不得超过 127 mtq。

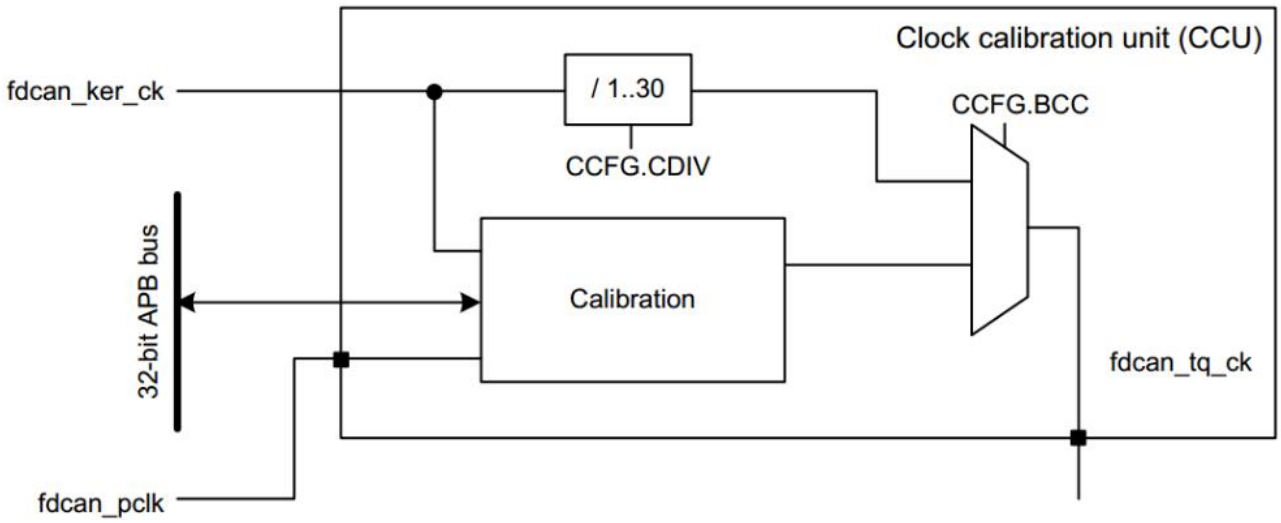
注 3：如果总和超过 127 mtq，则最大值（127 mtq）用于发射机延迟补偿。数据阶段在 CRC 分隔符的采样点结束，该采样点停止检查 SSP 处的接收位。本文档的下一部分将说明时钟校准单元，并对其功能和操作进行说明。

90.10 CAN FD 时钟校准

FDCAN 支持时钟校准单元（CCU）功能。 该功能允许用户校准 FDCAN，FDCAN 发送器（主机）接收器（设备）。 例如，当 FDCAN 设备与主机的最新比特率。此功能允许用户在总线中添加新实例，并且比特率的存在是未知的。也是在 FDCAN 接收器没有精确石英的情况下很有用(可能导致准时错误)。

90.10.1 CCU 介绍

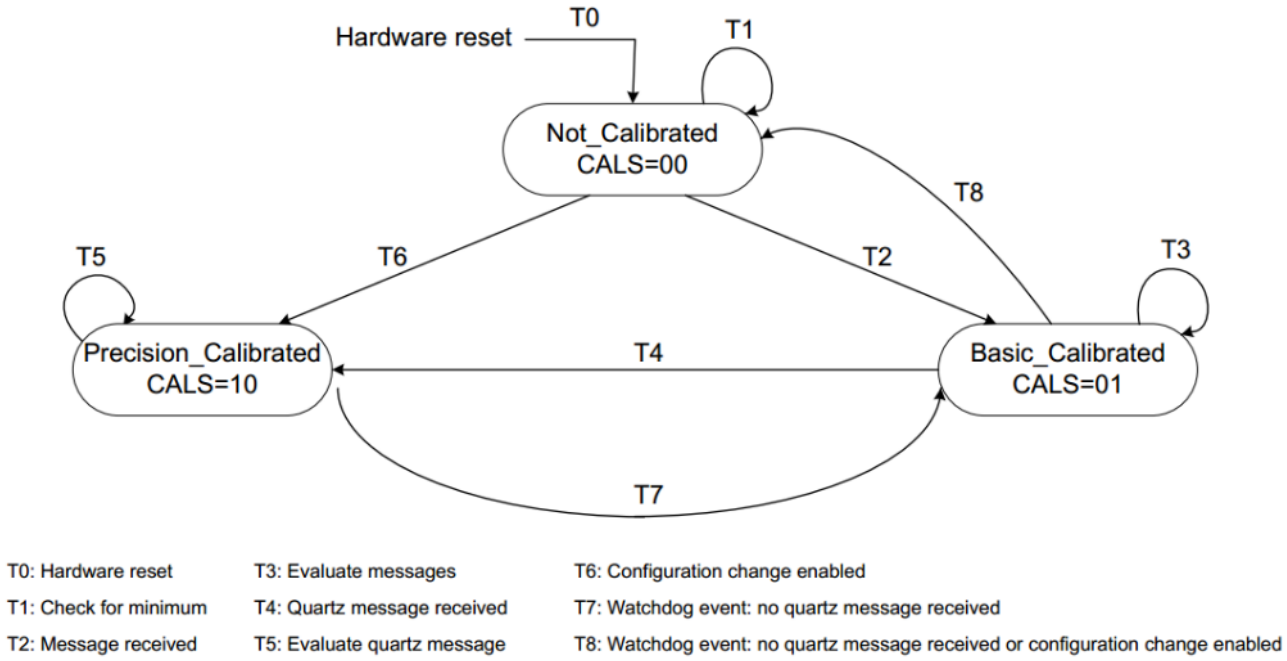
时钟校准单元是通过 FDCAN_CCU_CCFG 寄存器初始化的，只有当两个 FDCAN_CCCR 中的 CCE 和 INIT 位设置为 1。仅当 FDCAN 在 CAN 2.0 模式下运行时，才可以使用 CCU。当 FDCAN_CCU_CCFG 中的 BCC = 1 时，时钟校准被绕过。 下图显示了旁路操作。



CCU 操作条件：仅当 FDCAN 比特率在 125 Kbit / s 和 1 Mbit / s 之间时，CCU 才能运行

CCU 功能说明：通过适配时钟分频器，可以通过 CAN 消息对 fdcan_tq_ck（时间量子时钟）进行校准，它从 fdcan_ker_ck 时钟生成 CAN 协议时间量 tq。

校准状态机：功能状态机的校准如下图所示。



90.10.2 基础校准

测量了从隐性到显性的两个连续下降沿之间的最小距离。这该措施假设两个 CAN 位时间以 PLL 时钟周期计。 每当新的时钟分频器通过 FDCAN_CCU_CCFG 中的 CDIV [3：0]字段找到边缘之间的较小距离来更新一次测量。 当 CAN 协议控制器检测到有效的 CAN 消息时，便完成了校准。

90.10.3 精确校准

校准状态机通过计算以下内容来测量 CAN 帧内较长位序列的长度：fdcan_ker_ck 周期数。该位序列的长度可以通过在 FDCAN_CCU_CCFG 中 CFL 位配置为 32 位或 64 位。精度校准基于新的时钟分频器值，该值由较长位序列的测量。校准框通过 FDCAN 验收过滤检测。过滤器元素和 Rx 缓冲区必须为在 FDCAN 中配置以识别和存储校准消息。如果 fdcan_ker_clk 校准是通过软件完成的（请从 CALS [1：0]字段中评估校准状态 FDCAN_CCU_CSTAT），必须将 FDCAN 设置为受限操作模式，直到校准完成 Precision_Calibrated 状态（无帧，无错误或过载标志传输，无错误计数）。

注意：收到校准消息后，必须重新设置 Rx 缓冲区新数据标志以启用发信号通知。校准消息的数据字段必须至少为 1010 二进制序列，以确保设备节点可以进入 Basic_Calibrated 状态，并且主机节点消息得到确认。只能在由稳定的石英时钟控制的主机节点发送的有效 CAN 帧上执行精确校准。精密校准必须在预定义的最大间隔内重复进行，并由校准看门狗监督。

90.10.4 校准看门狗

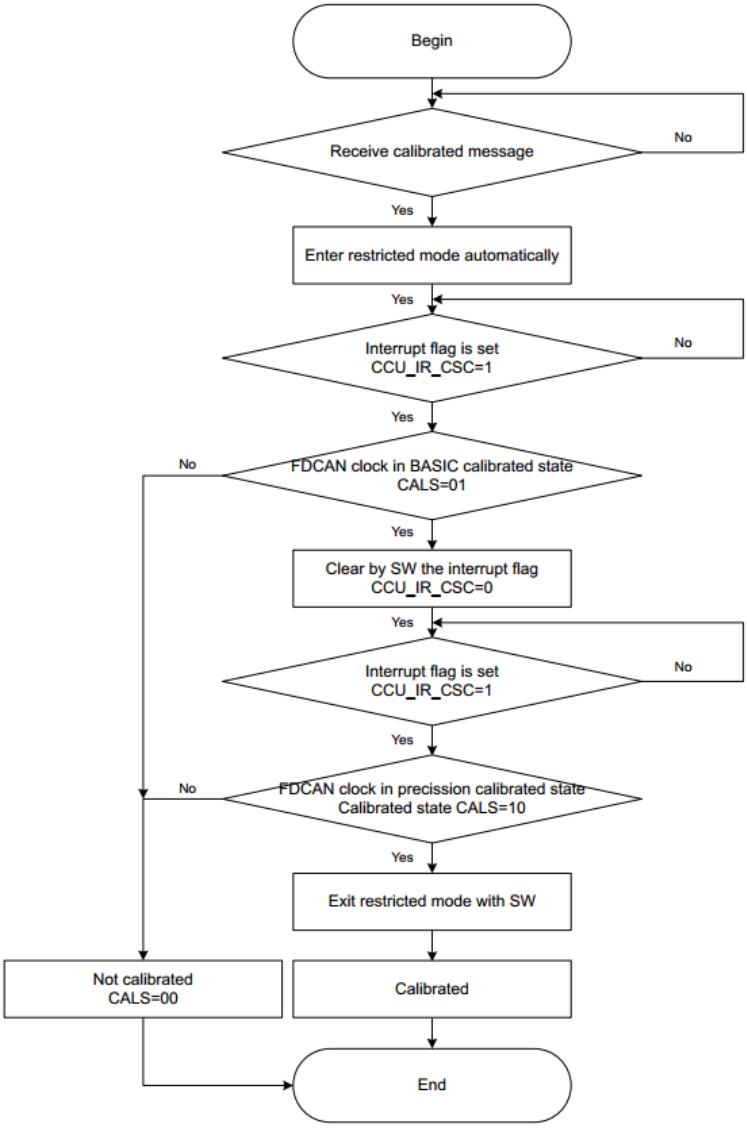
校准看门狗是一个递减计数器，它以 Not_Calibrated 状态开始，并监视接收到的信号消息。当处于 Basic_Calibrated 状态时，校准监视程序随每个收到的消息重新启动。

注意：如果在校准看门狗递减计数到零之前未收到任何消息，则 FSM 校准保持不变处于 Not_Calibrated 状态。计数器被重新加载，并且基本校准重新开始。进入 Precision_Calibrated 状态时，将重新启动校准看门狗。 在这种状态下，校准看门狗监视收到的石英消息。

注意：如果 FDCAN 直到校准看门狗都没有收到来自石英控制节点的消息，当计数降至零时，FSM 校准将转换回“ Basic_Calibrated”状态。

90.10.5 校准例子

本示例介绍了一个用例，用于通过 FDCAN 主机（发送器）校准 FDCAN 设备（接收器）。 这以下流程图说明了校准前通过 FDCAN 设备的步骤：



校准成功通过后，FDCAN 设备准备好进行接收和发送带有新 FDCAN 主机比特率的消息。

90.11 FDCAN 相比标准 bxCAN 的提升

下表可帮助用户简化将 STM32 设备中的 CAN 2.0 协议升级到 CAN-FD 协议的过程。该表还指定了 FDCAN 的改进。与传统的 BxCAN（基本扩展 CAN）相比，FDCAN 具有许多优势，包括更快的数据传输速度。速率和数据字节数的扩展，减少了帧开销。 总线负载也可以减少。 传输和接收中消息数量的增加要求 RAM 存储器的改进：

Features		Supported by BxCAN	Supported by FDCAN
Compatibility	Compatibility with BxCAN	Yes	Yes
	Compatibility with CAN-FD	No	No
	Remote frame	No	Supported to become compatible with BxCAN
Frame	Arbitration bitrate /data rate (in Mbit/s)	Up to 1 (only bitrate per frame)	Up to 1 /Up to 8
	Data length per frame (in bytes)	0 to 8	0 to 64
RAM	RAM	512 bytes	10 Kbytes
	Accessible RAM	No	Yes
Transmission	Dedicated Tx buffer/Tx queue/Tx FIFO	No/Yes/Yes Maximum three elements can be used as queue **or** FIFO.	Yes/Yes/Yes Maximum 32 elements: user chooses the transmit mechanism.
	Transmit pause	No	Yes
Reception	Dedicated Rx buffer/Rx FIFO	No/2 Up to three elements in each Rx FIFO Up to six elements maximum	64/2 Up to 64 elements in each Rx FIFO Up to 192 elements maximum
	Overwrite mode option	Overwrite the last element received in FIFO	Overwrite the oldest element in FIFO
	Improvements acceptance filters	Classical CAL (Can 2.0) acceptance filters	Some features are added as: • Discards matching filter. • Accepts non matching filters.
Other	Restricted test mode	No	Yes
	Transceiver delay compensation	No	Yes
	Clock calibration unit	No	Yes

鉴于 BxCAN 的兼容性，BxCAN 开发人员可以轻松地迁移到 FDCAN，因为 FDCAN 可以无需对整个系统设计进行修订即可实施。 FDCAN 包含所有 BxCAN 功能，并满足新应用程序的要求。

90.12 总结

本章节的知识点比较多，如果要理解透彻需要反复多读几遍，然后再看看英文原版。

第91章 STM32H7 的 FDCAN 总线基础知识和 HAL 库 API

本章节为大家讲解 FDCAN 的基础知识和对应的 HAL 库 API。CAN FD 中的 FD 含义就是 flexible data，灵活数据通信，且波特率可以和仲裁阶段波特率不同

91.1 初学者重要提示

91.2 FDCAN 基础知识

91.3 FDCAN 的 HAL 库用法

91.4 FDCAN 源文件 stm32h7xx_hal_fdcan.c

91.5 总结

91.1 初学者重要提示

- FDCAN 基础知识点可以看第 90 章，已经进行了详细说明
- 特别推荐瑞萨的 CAN 入门中英文手册，做的非常好：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=14546

91.2 FDCAN 基础知识

FDCAN 的基础知识在本教程的第 90 章进行了非常详细的说明。我们这里将本章用到的几个知识点再做个说明，详情推荐看第 90 章。

91.2.1 CAN FD 协议介绍

STM32H7 的 CAN FD 符合 ISO 11898-12015 标准。STM32 器件上的 FDCAN 的功能如下所示：

（1）符合 CAN 协议 2.0 版 A，B 和 ISO 11898-1：2015，-4。

（2）可访问的 10 KB RAM 内存，最多可分配 2560 个字。

（3）改进了接收过滤。

（4）两个可配置的接收 FIFO。

（5）多达 64 个专用接收缓冲区。

（6）接收高优先级消息时的单独信令。

（7）多达 32 个专用发送缓冲区。

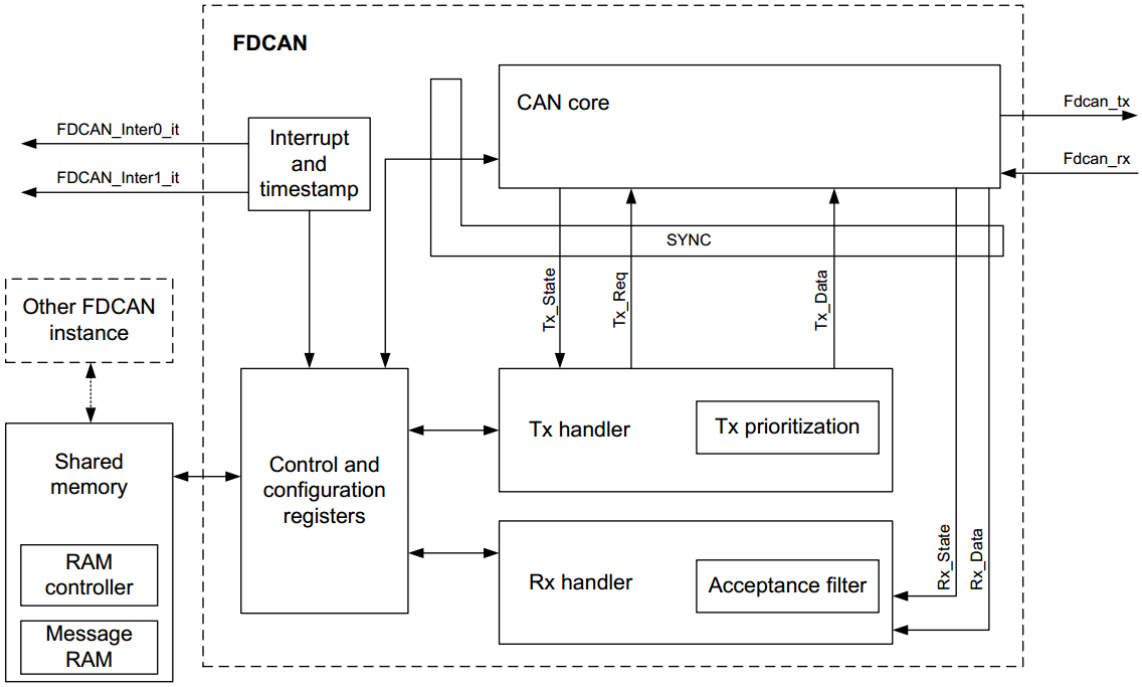
（8）可配置的发送 FIFO 和发送队列。

（9）可配置的发送事件 FIFO。

（10）时钟校准单元。

（11）收发器延迟补偿。

下图说明了 FDCAN 框图。



通过这个框图要了解到以下信息：

（1）CANFD1 和 CANFD2 共用一个 RAM 空间。

（2）每个 CANFD 都有自己的内核。

（3）CAN 内核实现协议控制和收发移位寄存器。

（4）Tx handler 控制消息从 CAN 消息 RAM 到 CAN 内核。

（5）Rx handler 控制 CAN 内核到 CAN 消息 RAM。

91.2.2 CAN FD 特性

（1）兼容经典 CAN，可以遵循 ISO 11898-1 做数据收发。

（2）提升错误检测，支持高达 CRC 21 位的校验和。

（3）消息优先级。

（4）保证延迟时间。

（5）配置灵活性。

（6）具有时间同步的组播接收。

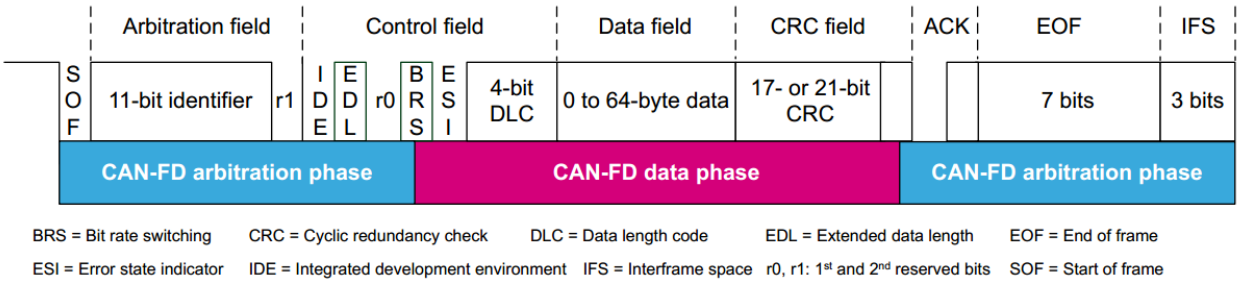
（7）系统范围内的数据一致性，每条消息最多 64 个字节。

（8）多主机。

（9）错误检测和信号。

（10）区分节点的临时错误和永久性故障以及自动关闭缺陷节点。

91.2.3 CAN FD 格式



第一个仲裁阶段（The first arbitration phase）是一条消息，其中包含：

（1）帧开始（SOF）。

（2）ID 号和其他位，指示消息的目的（提供或请求数据），以及速度和格式配置（CAN 或 CAN-FD）。

数据传输阶段（The data transmission phase）包括：

（1）数据长度代码（DLC），指示消息包含多少数据字节。

（2）用户希望发送的数据。

（3）检查循环冗余序列（CRC）。

（4）显性位。

第二个仲裁阶段（The second arbitration phase）包含：

（1）总线上其他节点发送的确认（ACK）的接收器（如果至少有一个接收器成功收到消息）

（2）帧尾（EOF），在 IFS 期间不传输任何消息：目标是将当前帧与下一帧分开。

注意：对于 29bit 标识符帧，当添加 18bit 标识到第 1 个仲裁阶段的 IDE bit 之后与标准 CAN FD 是类似的。

91.2.4CAN FD 相比 CAN2.0 的提升

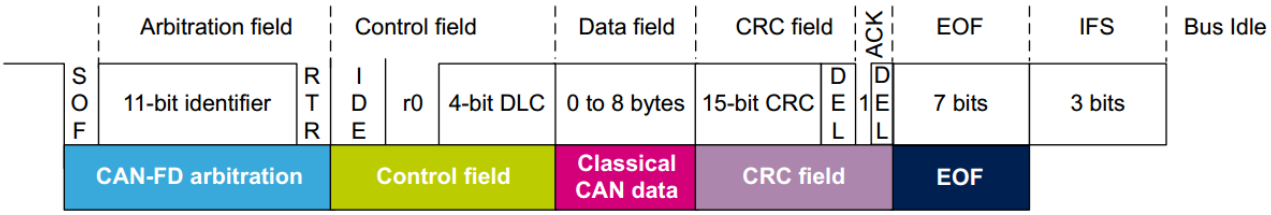
CAN-FD 的开发可以满足需要更高带宽的通信网络需求。每帧最多具有 64 个字节的 CAN-FD 以及将比特率提高到最大的可能性，使数据阶段要快 8 倍，在第二个仲裁阶段要恢复到正常的比特率。通过以下方式确保数据传输的完整性：

（1）17 级多项式对最大 16 字节的有效载荷进行 CRC。

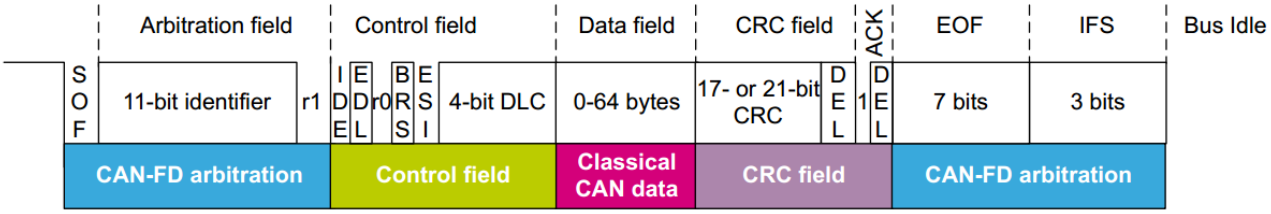
（2）21 级多项式对 16 到 64 字节之间的有效载荷进行校验。

标准帧和 CAN FD 的区别：

CAN 2.0: Classical base frame format



CAN-FD: CAN flexible data rate base frame format



DEL = Deliminator　RTR = Remote transmission request

标识符后，CAN 2.0 和 CAN-FD 具有不同的作用：

（1）CAN 2.0 发送 RTR 位以精确确定帧类型：数据帧（RTR 为主要）或远程帧（RTR）是隐性的）。

（2）由于 CAN-FD 仅支持数据帧，因此始终发送占优势的 RRS（保留）。

IDE 位保持在相同位置，并以相同的动作来区分基本格式（11 位标识符）。请注意，在扩展格式的情况下，IDE 位以显性或隐性方式传输（29 位标识符）。

与 CAN 2.0 相比，在 CAN-FD 帧中，在控制字段中添加了三个新位：

（1）扩展数据长度（EDL）位：隐性表示帧为 CAN-FD，否则该位为显性（称为 R0）在 CAN 2.0 帧中。

（2）比特率切换（BRS）：指示是否启用两个比特率（例如，当数据阶段位以不同的比特率传输到仲裁阶段）。

（3）错误状态指示器（ESI）：指示节点处于错误活动模式还是错误被动模式。

控制字段的最后一部分是数据长度代码（DLC），它具有相同的位置和相同的长度（4 位），用于 CAN 2.0 和 CAN-FD。 DLC 功能在 CAN-FD 和 CAN 2.0 中相同，但 CAN-FD 有很小变化（下表中的详细信息）。CAN-FD 扩展帧允许单个消息中发送 64 个数据字节，而 CAN 2.0 有效负载数据最多可以发送 8 个字节。

DLC (Dec)	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
CAN 2.0	0	1	2	3	4	5	6	7	8	8	8	8	8	8	8	8
CAN-FD	0	1	2	3	4	5	6	7	8	12	16	20	24	32	48	64

通过增加有效载荷数据的数据字段来改善网络带宽，因为需要更少的包处理。 同时，通过为 CRC 添加更多位来增强消息完整性：

（1）如果有效载荷数据最多为 16 个字节，则 CRC 以 17 位编码。

（2）如果有效载荷数据大于 20（16）个字节，则 CRC 以 21 位编码。

另外，为了确保 CAN-FD 帧的鲁棒性，填充位机制支持 CRC 字段。下表总结了 CAN-FD 和 CAN 2.0 之间的主要区别。 提供的主要功能与 CAN 2.0 相比，CAN FD 的改进之处在于数据有效负载的增加和速度的提高由 CAN-FD 中可用的 BRS，EDL 和 ESI 位来确保。

Features	CAN 2.0	CAN-FD
Compatibility	Does not support CAN-FD	Supports CAN 2.0 A/B
Maximum bit rate (Mbit/s)	Frame bitrate: up to 1	Arbitration bitrate: up to 1s Data bitrate: up to 8
DLC field (4 bits) code	Coded in 0 to 8	Coded in 0 to 64
Maximum data bytes in one message	8 bytes of data	64 bytes of data
BRS support	No	Yes
EDL support	No	Yes
ASI support	No	Yes
CRC bits check codes	Bits not included in CRC calculation	Bits included in CRC calculation
Remote frame support	Yes	No

91.3 FDCAN 总线的 HAL 库用法

91.3.1 FDCAN 总线结构体 FDCAN_GlobalTypeDef

FD CAN 总线相关的寄存器是通过 HAL 库中的结构体 FDCAN_GlobalTypeDef 定义，在 stm32h743xx.h 中可以找到这个类型定义：

```
typedef struct
{
  __IO uint32_t CREL;
  __IO uint32_t ENDN;
  __IO uint32_t RESERVED1;
  __IO uint32_t DBTP;
  __IO uint32_t TEST;
```

```
  __IO uint32_t RWD;
  __IO uint32_t CCCR;
  __IO uint32_t NBTP;
  __IO uint32_t TSCC;
  __IO uint32_t TSCV;
  __IO uint32_t TOCC;
  __IO uint32_t TOCV;
  __IO uint32_t RESERVED2[4];
  __IO uint32_t ECR;
  __IO uint32_t PSR;
  __IO uint32_t TDCR;
  __IO uint32_t RESERVED3;
  __IO uint32_t IR;
  __IO uint32_t IE;
  __IO uint32_t ILS;
  __IO uint32_t ILE;
  __IO uint32_t RESERVED4[8];
  __IO uint32_t GFC;
  __IO uint32_t SIDFC;
  __IO uint32_t XIDFC;
  __IO uint32_t RESERVED5;
  __IO uint32_t XIDAM;
  __IO uint32_t HPMS;
  __IO uint32_t NDAT1;
  __IO uint32_t NDAT2;
  __IO uint32_t RXF0C;
  __IO uint32_t RXF0S;
  __IO uint32_t RXF0A;
  __IO uint32_t RXBC;
  __IO uint32_t RXF1C;
  __IO uint32_t RXF1S;
  __IO uint32_t RXF1A;
  __IO uint32_t RXESC;
  __IO uint32_t TXBC;
  __IO uint32_t TXFQS;
  __IO uint32_t TXESC;
  __IO uint32_t TXBRP;
  __IO uint32_t TXBAR;
  __IO uint32_t TXBCR;
  __IO uint32_t TXBTO;
  __IO uint32_t TXBCF;
  __IO uint32_t TXBTIE;
  __IO uint32_t TXBCIE;
  __IO uint32_t RESERVED6[2];
  __IO uint32_t TXEFC;
  __IO uint32_t TXEFS;
  __IO uint32_t TXEFA;
  __IO uint32_t RESERVED7;
} FDCAN_GlobalTypeDef;
```

这个结构体的成员名称和排列次序和 CPU 的寄存器是一 一对应的。

__IO 表示 volatile，这是标准 C 语言中的一个修饰字，表示这个变量是非易失性的，编译器不要将其优化掉。core_m7.h 文件定义了这个宏：

```
#define     __O     volatile             /*!< Defines 'write only' permissions */
#define     __IO    volatile             /*!< Defines 'read / write' permissions */
```

下面我们看下 FDCAN 的定义，在 stm32h743xx.h 文件。

```
#define PERIPH_BASE           (0x40000000UL)
#define D2_APB1PERIPH_BASE     PERIPH_BASE

#define FDCAN1_BASE           (D2_APB1PERIPH_BASE + 0xA000UL)
#define FDCAN2_BASE           (D2_APB1PERIPH_BASE + 0xA400UL)
#define FDCAN_CCU_BASE        (D2_APB1PERIPH_BASE + 0xA800UL)

#define FDCAN1       ((FDCAN_GlobalTypeDef *) FDCAN1_BASE) <----- 展开这个宏，(FDCAN_GlobalTypeDef *)0x4000A000
#define FDCAN2       ((FDCAN_GlobalTypeDef *) FDCAN2_BASE)
#define FDCAN_CCU    ((FDCAN_ClockCalibrationUnit_TypeDef *) FDCAN_CCU_BASE)
```

我们访问 FDCAN1 的 CCCR 寄存器可以采用这种形式：FDCAN1->CCCR = 0。

91.3.2 FDCAN 总线时间触发结构体 TTCAN_TypeDef

FDCAN 总线时间触发相关的寄存器是通过 HAL 库中的结构体 TTCAN_TypeDef 定义，在 stm32h743xx.h 中可以找到这个类型定义：

```
typedef struct
{
  __IO uint32_t TTTMC;          /*!< TT Trigger Memory Configuration register,    Address offset: 0x100 */
  __IO uint32_t TTRMC;          /*!< TT Reference Message Configuration register, Address offset: 0x104 */
  __IO uint32_t TTOCF;          /*!< TT Operation Configuration register,         Address offset: 0x108 */
  __IO uint32_t TTMLM;          /*!< TT Matrix Limits register,                   Address offset: 0x10C */
  __IO uint32_t TURCF;          /*!< TUR Configuration register,                  Address offset: 0x110 */
  __IO uint32_t TTOCN;          /*!< TT Operation Control register,               Address offset: 0x114 */
  __IO uint32_t TTGTP;          /*!< TT Global Time Preset register,              Address offset: 0x118 */
  __IO uint32_t TTTMK;          /*!< TT Time Mark register,                       Address offset: 0x11C */
  __IO uint32_t TTIR;           /*!< TT Interrupt register,                       Address offset: 0x120 */
  __IO uint32_t TTIE;           /*!< TT Interrupt Enable register,                Address offset: 0x124 */
  __IO uint32_t TTILS;          /*!< TT Interrupt Line Select register,           Address offset: 0x128 */
  __IO uint32_t TTOST;          /*!< TT Operation Status register,                Address offset: 0x12C */
  __IO uint32_t TURNA;          /*!< TT TUR Numerator Actual register,            Address offset: 0x130 */
  __IO uint32_t TTLGT;          /*!< TT Local and Global Time register,           Address offset: 0x134 */
  __IO uint32_t TTCTC;          /*!< TT Cycle Time and Count register,            Address offset: 0x138 */
  __IO uint32_t TTCPT;          /*!< TT Capture Time register,                    Address offset: 0x13C */
  __IO uint32_t TTCSM;          /*!< TT Cycle Sync Mark register,                 Address offset: 0x140 */
  __IO uint32_t RESERVED1[111]; /*!< Reserved,                                                 0x144 - 0x2FC */
  __IO uint32_t TTTS;           /*!< TT Trigger Select register,                  Address offset: 0x300 */
} TTCAN_TypeDef;
```

91.3.3 FDCAN 总线初始化结构体 FDCAN_InitTypeDef

下面是 FDCAN 总线的初始化结构体：

```
typedef struct
{
  uint32_t FrameFormat;                 /*!< Specifies the FDCAN frame format.
                                             This parameter can be a value of @ref FDCAN_frame_format     */

  uint32_t Mode;                        /*!< Specifies the FDCAN mode.
                                             This parameter can be a value of @ref FDCAN_operating_mode   */

  FunctionalState AutoRetransmission;   /*!< Enable or disable the automatic retransmission mode.
```

```
                                    This parameter can be set to ENABLE or DISABLE                */

  FunctionalState TransmitPause;    /*!< Enable or disable the Transmit Pause feature.
                                    This parameter can be set to ENABLE or DISABLE                */

  FunctionalState ProtocolException;  /*!< Enable or disable the Protocol Exception Handling.
                                    This parameter can be set to ENABLE or DISABLE                */

  uint32_t NominalPrescaler;        /*!< Specifies the value by which the oscillator frequency is
                                    divided for generating the nominal bit time quanta.
                                    This parameter must be a number between 1 and 512             */

  uint32_t NominalSyncJumpWidth;    /*!< Specifies the maximum number of time quanta the FDCAN
                                    hardware is allowed to lengthen or shorten a bit to perform
                                    resynchronization.
                                    This parameter must be a number between 1 and 128             */

  uint32_t NominalTimeSeg1;         /*!< Specifies the number of time quanta in Bit Segment 1.
                                    This parameter must be a number between 2 and 256             */

  uint32_t NominalTimeSeg2;         /*!< Specifies the number of time quanta in Bit Segment 2.
                                    This parameter must be a number between 2 and 128             */

  uint32_t DataPrescaler;           /*!< Specifies the value by which the oscillator frequency is
                                    divided for generating the data bit time quanta.
                                    This parameter must be a number between 1 and 32              */

  uint32_t DataSyncJumpWidth;       /*!< Specifies the maximum number of time quanta the FDCAN
                                    hardware is allowed to lengthen or shorten a data bit to
                                    perform resynchronization.
                                    This parameter must be a number between 1 and 16              */

  uint32_t DataTimeSeg1;            /*!< Specifies the number of time quanta in Data Bit Segment 1.
                                    This parameter must be a number between 1 and 32              */

  uint32_t DataTimeSeg2;            /*!< Specifies the number of time quanta in Data Bit Segment 2.
                                    This parameter must be a number between 1 and 16              */

  uint32_t MessageRAMOffset;        /*!< Specifies the message RAM start address.
                                    This parameter must be a number between 0 and 2560            */

  uint32_t StdFiltersNbr;           /*!< Specifies the number of standard Message ID filters.
                                    This parameter must be a number between 0 and 128             */

  uint32_t ExtFiltersNbr;           /*!< Specifies the number of extended Message ID filters.
                                    This parameter must be a number between 0 and 64              */

  uint32_t RxFifo0ElmtsNbr;         /*!< Specifies the number of Rx FIFO0 Elements.
                                    This parameter must be a number between 0 and 64              */

  uint32_t RxFifo0ElmtSize;         /*!< Specifies the Data Field Size in an Rx FIFO 0 element.
                                    This parameter can be a value of @ref FDCAN_data_field_size   */

  uint32_t RxFifo1ElmtsNbr;         /*!< Specifies the number of Rx FIFO 1 Elements.
                                    This parameter must be a number between 0 and 64              */

  uint32_t RxFifo1ElmtSize;         /*!< Specifies the Data Field Size in an Rx FIFO 1 element.
                                    This parameter can be a value of @ref FDCAN_data_field_size   */
```

```
  uint32_t RxBuffersNbr;                /*!< Specifies the number of Dedicated Rx Buffer elements.
                                             This parameter must be a number between 0 and 64             */

  uint32_t RxBufferSize;                /*!< Specifies the Data Field Size in an Rx Buffer element.
                                             This parameter can be a value of @ref FDCAN_data_field_size  */

  uint32_t TxEventsNbr;                 /*!< Specifies the number of Tx Event FIFO elements.
                                             This parameter must be a number between 0 and 32             */

  uint32_t TxBuffersNbr;                /*!< Specifies the number of Dedicated Tx Buffers.
                                             This parameter must be a number between 0 and 32             */

  uint32_t TxFifoQueueElmtsNbr;         /*!< Specifies the number of Tx Buffers used for Tx FIFO/Queue.
                                             This parameter must be a number between 0 and 32             */

  uint32_t TxFifoQueueMode;             /*!< Tx FIFO/Queue Mode selection.
                                             This parameter can be a value of @ref FDCAN_txFifoQueue_Mode */

  uint32_t TxElmtSize;                  /*!< Specifies the Data Field Size in a Tx Element.
                                             This parameter can be a value of @ref FDCAN_data_field_size  */

} FDCAN_InitTypeDef;
```

下面将结构体成员逐一做个说明：

- FrameFormat

  用于设置 CAN 帧格式。

```
#define FDCAN_FRAME_CLASSIC    ((uint32_t)0x00000000U)        /* 经典 CAN 模式 */
#define FDCAN_FRAME_FD_NO_BRS  ((uint32_t)FDCAN_CCCR_FDOE)    /* FD CAN 不带可变波特率 */
#define FDCAN_FRAME_FD_BRS     ((uint32_t)(FDCAN_CCCR_FDOE | FDCAN_CCCR_BRSE)) /* FD CAN 带可变波特率 */
```

- Mode

  用于设置 CAN 操作模式。

```
#define FDCAN_MODE_NORMAL                ((uint32_t)0x00000000U) /*!< 正常模式                 */
#define FDCAN_MODE_RESTRICTED_OPERATION  ((uint32_t)0x00000001U) /*!< 有限制的操作模式          */
#define FDCAN_MODE_BUS_MONITORING        ((uint32_t)0x00000002U) /*!< 总线监测模式              */
#define FDCAN_MODE_INTERNAL_LOOPBACK     ((uint32_t)0x00000003U) /*!< 内部环回模式              */
#define FDCAN_MODE_EXTERNAL_LOOPBACK     ((uint32_t)0x00000004U) /*!< 外部环回模式             */
```

- AutoRetransmission

  使能自动重传模式。

  使能 ENABLE 或者禁止 DISABLE。

- TransmitPause

  使能或者禁止传输暂停特性。ENABLE 使能或者 DISABLE 禁止。

- ProtocolException

  使能或者禁止协议异常管理。ENABLE 表示使能，DISABLE 表示禁止。

- NominalPrescaler

  用于 CAN FD 仲裁阶段分频设置，产生标称位时间量，参数范围 1-512。

- NominalSyncJumpWidth

  设置 FD CAN 仲裁阶段最大支持的时间量来加长或者缩短一个 bit 来实现再同步，参数范围 1-128。

- NominalTimeSeg1

  设置仲裁阶段 Bit Segment 1 的时间量，范围 2 – 256。

- NominalTimeSeg2

  设置仲裁阶段 Bit Segment 2 的时间量，范围 2 – 128。

- DataPrescaler

  用于 CAN FD 数据阶段分频设置，范围 1-32。

- DataSyncJumpWidth

  设置 FD CAN 数据阶段最大支持的时间量来加长或者缩短一个 bit 来实现数据再同步，参数范围 1-16。

- DataTimeSeg1

  设置数据阶段 Data Bit Segment 1 的时间量，范围 1 – 32。

- DataTimeSeg2

  设置数据阶段 Data Bit Segment 2 的时间量，范围 1 – 16。

- MessageRAMOffset

  设置消息 RAM 起始地址，范围 0 到 2560。

- StdFiltersNbr

  标准 ID 过滤个数，范围 0 到 128。

- ExtFiltersNbr

  扩展 ID 过滤个数，范围 0 到 64。

- RxFifo0ElmtsNbr

  RX FIFO0 元素个数，范围 0 到 64。

- RxFifo0ElmtSize

  RX FIFO0 每个元素中数据大小，支持参数如下：

```
#define FDCAN_DATA_BYTES_8  ((uint32_t)0x00000004U) /*!< 8 bytes data field  */
#define FDCAN_DATA_BYTES_12 ((uint32_t)0x00000005U) /*!< 12 bytes data field */
#define FDCAN_DATA_BYTES_16 ((uint32_t)0x00000006U) /*!< 16 bytes data field */
#define FDCAN_DATA_BYTES_20 ((uint32_t)0x00000007U) /*!< 20 bytes data field */
#define FDCAN_DATA_BYTES_24 ((uint32_t)0x00000008U) /*!< 24 bytes data field */
#define FDCAN_DATA_BYTES_32 ((uint32_t)0x0000000AU) /*!< 32 bytes data field */
#define FDCAN_DATA_BYTES_48 ((uint32_t)0x0000000EU) /*!< 48 bytes data field */
#define FDCAN_DATA_BYTES_64 ((uint32_t)0x00000012U) /*!< 64 bytes data field */
```

- RxFifo1ElmtsNbr

  RX FIFO1 个数，范围 0 到 64。

- RxFifo1ElmtSize

  RX FIFO1 每个元素中数据大小，支持参数如下：

```
#define FDCAN_DATA_BYTES_8  ((uint32_t)0x00000004U) /*!< 8 bytes data field  */
#define FDCAN_DATA_BYTES_12 ((uint32_t)0x00000005U) /*!< 12 bytes data field */
#define FDCAN_DATA_BYTES_16 ((uint32_t)0x00000006U) /*!< 16 bytes data field */
#define FDCAN_DATA_BYTES_20 ((uint32_t)0x00000007U) /*!< 20 bytes data field */
#define FDCAN_DATA_BYTES_24 ((uint32_t)0x00000008U) /*!< 24 bytes data field */
#define FDCAN_DATA_BYTES_32 ((uint32_t)0x0000000AU) /*!< 32 bytes data field */
```

```
#define FDCAN_DATA_BYTES_48 ((uint32_t)0x0000000EU) /*!< 48 bytes data field */
#define FDCAN_DATA_BYTES_64 ((uint32_t)0x00000012U) /*!< 64 bytes data field */
```

- RxBuffersNbr

  设置 Rx Buffer 元素个数，范围 0 - 64：

- RxBuffersSize

  设置 Rx Buffer 元素中每个数据大小，范围 0 - 64：

```
#define FDCAN_DATA_BYTES_8  ((uint32_t)0x00000004U) /*!< 8 bytes data field  */
#define FDCAN_DATA_BYTES_12 ((uint32_t)0x00000005U) /*!< 12 bytes data field */
#define FDCAN_DATA_BYTES_16 ((uint32_t)0x00000006U) /*!< 16 bytes data field */
#define FDCAN_DATA_BYTES_20 ((uint32_t)0x00000007U) /*!< 20 bytes data field */
#define FDCAN_DATA_BYTES_24 ((uint32_t)0x00000008U) /*!< 24 bytes data field */
#define FDCAN_DATA_BYTES_32 ((uint32_t)0x0000000AU) /*!< 32 bytes data field */
#define FDCAN_DATA_BYTES_48 ((uint32_t)0x0000000EU) /*!< 48 bytes data field */
#define FDCAN_DATA_BYTES_64 ((uint32_t)0x00000012U) /*!< 64 bytes data field */
```

- TxEventsNbr

  Tx Event FIFO 元素个数，范围 0 到 32。

- TxBuffersNbr

  设置专用的 Tx Buffer 元素个数，范围 0 到 32。

- TxFifoQueueElmtsNbr

  设置用于 Tx FIFO/Queue 的 Tx Buffers 个数。范围 0 到 32。

- TxFifoQueueMode

  设置 FIFO 模式或者 QUEUE 队列模式。

```
#define FDCAN_TX_FIFO_OPERATION  ((uint32_t)0x00000000U)     /*!< FIFO mode  */
#define FDCAN_TX_QUEUE_OPERATION ((uint32_t)FDCAN_TXBC_TFQM) /*!< Queue mode */
```

- TxElmtSize

  设置 Tx Element 中的数据域大小。支持参数如下：

```
#define FDCAN_DATA_BYTES_8  ((uint32_t)0x00000004U) /*!< 8 bytes data field  */
#define FDCAN_DATA_BYTES_12 ((uint32_t)0x00000005U) /*!< 12 bytes data field */
#define FDCAN_DATA_BYTES_16 ((uint32_t)0x00000006U) /*!< 16 bytes data field */
#define FDCAN_DATA_BYTES_20 ((uint32_t)0x00000007U) /*!< 20 bytes data field */
#define FDCAN_DATA_BYTES_24 ((uint32_t)0x00000008U) /*!< 24 bytes data field */
#define FDCAN_DATA_BYTES_32 ((uint32_t)0x0000000AU) /*!< 32 bytes data field */
#define FDCAN_DATA_BYTES_48 ((uint32_t)0x0000000EU) /*!< 48 bytes data field */
#define FDCAN_DATA_BYTES_64 ((uint32_t)0x00000012U) /*!< 64 bytes data field */
```

91.3.4FDCAN 总线消息 RAM 地址

FDCAN_MsgRamAddressTypeDef

下面是消息 RAM 结构体：

```
typedef struct
{
  uint32_t StandardFilterSA; /*!< Specifies the Standard Filter List Start Address.
                                  This parameter must be a 32-bit word address      */
```

```
  uint32_t ExtendedFilterSA; /*!< Specifies the Extended Filter List Start Address.
                                  This parameter must be a 32-bit word address      */

  uint32_t RxFIFO0SA;        /*!< Specifies the Rx FIFO 0 Start Address.
                                  This parameter must be a 32-bit word address      */

  uint32_t RxFIFO1SA;        /*!< Specifies the Rx FIFO 1 Start Address.
                                  This parameter must be a 32-bit word address      */

  uint32_t RxBufferSA;       /*!< Specifies the Rx Buffer Start Address.
                                  This parameter must be a 32-bit word address      */

  uint32_t TxEventFIFOSA;    /*!< Specifies the Tx Event FIFO Start Address.
                                  This parameter must be a 32-bit word address      */

  uint32_t TxBufferSA;       /*!< Specifies the Tx Buffers Start Address.
                                  This parameter must be a 32-bit word address      */

  uint32_t TxFIFOQSA;        /*!< Specifies the Tx FIFO/Queue Start Address.
                                  This parameter must be a 32-bit word address      */

  uint32_t TTMemorySA;       /*!< Specifies the Trigger Memory Start Address.
                                  This parameter must be a 32-bit word address      */

  uint32_t EndAddress;       /*!< Specifies the End Address of the allocated RAM.
                                  This parameter must be a 32-bit word address      */

} FDCAN_MsgRamAddressTypeDef;
```

下面将结构体成员逐一做个说明：

- StandardFilterSA

  设置标准过滤器起始地址，必须是 32bit 地址。

- ExtendedFilterSA

  设置扩展过滤器起始地址，必须是 32bit 地址。

- RxFIFO0SA

  设置 RX FIFO 0 起始地址，必须是 32bit 地址。

- RxFIFO1SA

  设置 RX FIFO 1 起始地址，必须是 32bit 地址。

- RxBufferSA

  设置 RX Buffer 起始地址，必须是 32bit 地址。

- TxEventFIFOSA

  设置 Tx Event FIFO 起始地址，必须是 32bit 地址。

- TTMemorySA

  设置触发内存起始地址，必须是 32bit 地址。

- EndAddress

  设置申请 RAM 空间的结束地址，必须是 32bit 地址。

91.3.5 FDCAN 总线过滤结构体 FDCAN_FilterTypeDef

下面是过滤结构体：

```
typedef struct
{
  uint32_t IdType;           /*!< Specifies the identifier type.
                                  This parameter can be a value of @ref FDCAN_id_type       */

  uint32_t FilterIndex;      /*!< Specifies the filter which will be initialized.
                                  This parameter must be a number between:
                                   - 0 and 127, if IdType is FDCAN_STANDARD_ID
                                   - 0 and 63, if IdType is FDCAN_EXTENDED_ID               */

  uint32_t FilterType;       /*!< Specifies the filter type.
                                  This parameter can be a value of @ref FDCAN_filter_type.
                                  The value FDCAN_EXT_FILTER_RANGE_NO_EIDM is permitted
                                  only when IdType is FDCAN_EXTENDED_ID.
                                  This parameter is ignored if FilterConfig is set to
                                  FDCAN_FILTER_TO_RXBUFFER                                 */

  uint32_t FilterConfig;     /*!< Specifies the filter configuration.
                                  This parameter can be a value of @ref FDCAN_filter_config */

  uint32_t FilterID1;        /*!< Specifies the filter identification 1.
                                  This parameter must be a number between:
                                   - 0 and 0x7FF, if IdType is FDCAN_STANDARD_ID
                                   - 0 and 0x1FFFFFFF, if IdType is FDCAN_EXTENDED_ID       */

  uint32_t FilterID2;        /*!< Specifies the filter identification 2.
                                  This parameter is ignored if FilterConfig is set to
                                  FDCAN_FILTER_TO_RXBUFFER.
                                  This parameter must be a number between:
                                   - 0 and 0x7FF, if IdType is FDCAN_STANDARD_ID
                                   - 0 and 0x1FFFFFFF, if IdType is FDCAN_EXTENDED_ID       */

  uint32_t RxBufferIndex;    /*!< Contains the index of the Rx buffer in which the
                                  matching message will be stored.
                                  This parameter must be a number between 0 and 63.
                                  This parameter is ignored if FilterConfig is different
                                  from FDCAN_FILTER_TO_RXBUFFER                            */

  uint32_t IsCalibrationMsg; /*!< Specifies whether the filter is configured for
                                  calibration messages.
                                  This parameter is ignored if FilterConfig is different
                                  from FDCAN_FILTER_TO_RXBUFFER.
                                  This parameter can be:
                                   - 0 : ordinary message
                                   - 1 : calibration message                               */

} FDCAN_FilterTypeDef;
```

- IdType

用于设置标准 ID 和扩展 ID。

```
#define FDCAN_STANDARD_ID ((uint32_t)0x00000000U) /*!< 标准 ID */
#define FDCAN_EXTENDED_ID ((uint32_t)0x40000000U) /*!< 扩展 ID */
```

- FilterIndex

  用于过滤索引，如果是标准 ID，范围 0 到 127。如果是扩展 ID，范围 0 到 64。

- FilterType

  用于设置过滤类型。如果成员 FilterConfig 设置为 FDCAN_FILTER_TO_RXBUFFER，本参数将不起作用。

```
#define FDCAN_FILTER_RANGE  ((uint32_t)0x00000000U) /*!< 范围过滤从 FilterID1 到 FilterID2  */
#define FDCAN_FILTER_DUAL   ((uint32_t)0x00000001U) /*!< 专用 ID 过滤，FilterID1 或者 FilterID2  */

/*!< 精度屏蔽过滤，FilterID1 = filter, FilterID2 = mask */
#define FDCAN_FILTER_MASK   ((uint32_t)0x00000002U)

/*!< 仅 ID 扩展模式支持此参数，范围从 FilterID1 到 FilterID2, EIDM mask not applied */
#define FDCAN_FILTER_RANGE_NO_EIDM ((uint32_t)0x00000003U)
```

- FilterConfig

  用于设置过滤类型。

```
#define FDCAN_FILTER_DISABLE        ((uint32_t)0x00000000U)  禁止过滤
#define FDCAN_FILTER_TO_RXFIFO0     ((uint32_t)0x00000001U)  如果过滤匹配，将数据保存到 Rx FIFO 0
#define FDCAN_FILTER_TO_RXFIFO1     ((uint32_t)0x00000002U)  如果过滤匹配，将数据保存到 Rx FIFO 1
#define FDCAN_FILTER_REJECT         ((uint32_t)0x00000003U)  如果过滤匹配，拒绝此 ID
#define FDCAN_FILTER_HP             ((uint32_t)0x00000004U)  如果过滤匹配，设置高优先级
#define FDCAN_FILTER_TO_RXFIFO0_HP  ((uint32_t)0x00000005U)  如果过滤匹配，设置高优先级并保存到 FIFO 0
#define FDCAN_FILTER_TO_RXFIFO1_HP  ((uint32_t)0x00000006U)  如果过滤匹配，设置高优先级并保存到 FIFO 1
#define FDCAN_FILTER_TO_RXBUFFER    ((uint32_t)0x00000007U)  如果过滤匹配，保存到 Rx Buffer，并忽略 FilterType
                                                             配置
```

- FilterID1

  用于设置过滤 ID1。如果 ID 类型是 FDCAN_STANDARD_ID，范围 0 到 0x7FF。如果 ID 类型是 FDCAN_EXTENDED_ID，范围是 0 到 0x1FFFFFFF。

- FilterID2

  用于设置过滤 ID2。如果 FilterConfig 设置为 FDCAN_FILTER_TO_RXBUFFER，此参数不起作用。如果 ID 类型是 FDCAN_STANDARD_ID,范围 0 到 0x7FF。如果 ID 类型是 FDCAN_EXTENDED_ID，范围是 0 到 0x1FFFFFFF。

- RxBufferIndex

  匹配消息存储到 Rx buffer 中的索引。参数范围 0 到 63。如果 FilterConfig 设置为 FDCAN_FILTER_TO_RXBUFFER，此参数不起作用。

- IsCalibrationMsg

  用于设置是否配置校准消息。如果 FilterConfig 设置为 FDCAN_FILTER_TO_RXBUFFER，此参数不起作用。

  0 ： 表示正常消息。

  1 ： 标志校准消息。

91.3.6 FDCAN 总线消息发送结构体 FDCAN_TxHeaderTypeDef

下面是 CAN FD 发送消息结构体：

```
typedef struct
{
  uint32_t Identifier;          /*!< Specifies the identifier.
                                     This parameter must be a number between:
                                      - 0 and 0x7FF, if IdType is FDCAN_STANDARD_ID
                                      - 0 and 0x1FFFFFFF, if IdType is FDCAN_EXTENDED_ID              */

  uint32_t IdType;              /*!< Specifies the identifier type for the message that will be
                                     transmitted.
                                     This parameter can be a value of @ref FDCAN_id_type             */

  uint32_t TxFrameType;         /*!< Specifies the frame type of the message that will be transmitted.
                                     This parameter can be a value of @ref FDCAN_frame_type          */

  uint32_t DataLength;          /*!< Specifies the length of the frame that will be transmitted.
                                      This parameter can be a value of @ref FDCAN_data_length_code   */

  uint32_t ErrorStateIndicator; /*!< Specifies the error state indicator.
                                     This parameter can be a value of @ref FDCAN_error_state_indicator */

  uint32_t BitRateSwitch;       /*!< Specifies whether the Tx frame will be transmitted with or without
                                     bit rate switching.
                                     This parameter can be a value of @ref FDCAN_bit_rate_switching  */

  uint32_t FDFormat;            /*!< Specifies whether the Tx frame will be transmitted in classic or
                                     FD format.
                                     This parameter can be a value of @ref FDCAN_format              */

  uint32_t TxEventFifoControl;  /*!< Specifies the event FIFO control.
                                     This parameter can be a value of @ref FDCAN_EFC                 */

  uint32_t MessageMarker;       /*!< Specifies the message marker to be copied into Tx Event FIFO
                                     element for identification of Tx message status.
                                     This parameter must be a number between 0 and 0xFF              */

} FDCAN_TxHeaderTypeDef;
```

- Identifier

  用于设置 ID，如果 IdType 是标准 FDCAN_STANDARD_ID，范围 0 到 0x7FF，如果 IdType 是 FDCAN_EXTENDED_ID 扩展 ID，范围 0 到 0x1FFFFFFF。

- IdType

  用于设置标准 ID 或者扩展 ID。

```
#define FDCAN_STANDARD_ID ((uint32_t)0x00000000U) /*!< 标准 ID */
#define FDCAN_EXTENDED_ID ((uint32_t)0x40000000U) /*!< 扩展 ID */
```

- TxFrameType

  用于设置帧类型，数据帧或遥控帧。

```
#define FDCAN_DATA_FRAME   ((uint32_t)0x00000000U)  /*!< 数据帧 */
#define FDCAN_REMOTE_FRAME ((uint32_t)0x20000000U)  /*!< 遥控帧 */
```

- DataLength

用于设置数据长度。

```
#define FDCAN_DLC_BYTES_0  ((uint32_t)0x00000000U) /*!< 0 bytes data field  */
#define FDCAN_DLC_BYTES_1  ((uint32_t)0x00010000U) /*!< 1 bytes data field  */
#define FDCAN_DLC_BYTES_2  ((uint32_t)0x00020000U) /*!< 2 bytes data field  */
#define FDCAN_DLC_BYTES_3  ((uint32_t)0x00030000U) /*!< 3 bytes data field  */
#define FDCAN_DLC_BYTES_4  ((uint32_t)0x00040000U) /*!< 4 bytes data field  */
#define FDCAN_DLC_BYTES_5  ((uint32_t)0x00050000U) /*!< 5 bytes data field  */
#define FDCAN_DLC_BYTES_6  ((uint32_t)0x00060000U) /*!< 6 bytes data field  */
#define FDCAN_DLC_BYTES_7  ((uint32_t)0x00070000U) /*!< 7 bytes data field  */
#define FDCAN_DLC_BYTES_8  ((uint32_t)0x00080000U) /*!< 8 bytes data field  */
#define FDCAN_DLC_BYTES_12 ((uint32_t)0x00090000U) /*!< 12 bytes data field */
#define FDCAN_DLC_BYTES_16 ((uint32_t)0x000A0000U) /*!< 16 bytes data field */
#define FDCAN_DLC_BYTES_20 ((uint32_t)0x000B0000U) /*!< 20 bytes data field */
#define FDCAN_DLC_BYTES_24 ((uint32_t)0x000C0000U) /*!< 24 bytes data field */
#define FDCAN_DLC_BYTES_32 ((uint32_t)0x000D0000U) /*!< 32 bytes data field */
#define FDCAN_DLC_BYTES_48 ((uint32_t)0x000E0000U) /*!< 48 bytes data field */
#define FDCAN_DLC_BYTES_64 ((uint32_t)0x000F0000U) /*!< 64 bytes data field */
```

- ErrorStateIndicator

用于设置错误状态指示：

```
#define FDCAN_ESI_ACTIVE  ((uint32_t)0x00000000U) /*!< 传输节点 error active  */
#define FDCAN_ESI_PASSIVE ((uint32_t)0x80000000U) /*!< 传输节点 error passive */
```

- BitRateSwitch

用于设置发送是否波特率可变。

```
#define FDCAN_BRS_OFF ((uint32_t)0x00000000U) /*!< FDCAN 帧发送/接收不带波特率可变 */
#define FDCAN_BRS_ON  ((uint32_t)0x00100000U) /*!< FDCAN 帧发送/接收带波特率可变 */
```

- FDFormat

用于设置发送帧是经典格式还是 CANFD 格式。

```
#define FDCAN_CLASSIC_CAN ((uint32_t)0x00000000U) /*!< 帧发送/接收使用经典 CAN */
#define FDCAN_FD_CAN      ((uint32_t)0x00200000U) /*!< 帧发送/接收使用 FDCAN 格式 */
```

- TxEventFifoControl

用于设置发送事件 FIFO 控制。

```
#define FDCAN_NO_TX_EVENTS    ((uint32_t)0x00000000U) /*!< 不存储 Tx events */
#define FDCAN_STORE_TX_EVENTS ((uint32_t)0x00800000U) /*!< 存储 Tx events  */
```

- MessageMarker

用于设置复制到 TX EVENT FIFO 的消息 Maker，来识别消息状态，范围 0 到 0xFF。

91.3.7 FDCAN 总线消息接收结构体 FDCAN_RxHeaderTypeDef

下面是 CAN FD 接收消息结构体：

```
typedef struct
{
  uint32_t Identifier;            /*!< Specifies the identifier.
                                       This parameter must be a number between:
                                        - 0 and 0x7FF, if IdType is FDCAN_STANDARD_ID
                                        - 0 and 0x1FFFFFFF, if IdType is FDCAN_EXTENDED_ID              */
```

```
  uint32_t IdType;                /*!< Specifies the identifier type of the received message.
                                       This parameter can be a value of @ref FDCAN_id_type              */

  uint32_t RxFrameType;           /*!< Specifies the the received message frame type.
                                       This parameter can be a value of @ref FDCAN_frame_type           */

  uint32_t DataLength;            /*!< Specifies the received frame length.
                                        This parameter can be a value of @ref FDCAN_data_length_code    */

  uint32_t ErrorStateIndicator;   /*!< Specifies the error state indicator.
                                       This parameter can be a value of @ref FDCAN_error_state_indicator */

  uint32_t BitRateSwitch;         /*!< Specifies whether the Rx frame is received with or without bit
                                       rate switching.
                                       This parameter can be a value of @ref FDCAN_bit_rate_switching   */

  uint32_t FDFormat;              /*!< Specifies whether the Rx frame is received in classic or FD
                                       format.
                                       This parameter can be a value of @ref FDCAN_format               */

  uint32_t RxTimestamp;           /*!< Specifies the timestamp counter value captured on start of frame
                                       reception.
                                       This parameter must be a number between 0 and 0xFFFF             */

  uint32_t FilterIndex;           /*!< Specifies the index of matching Rx acceptance filter element.
                                       This parameter must be a number between:
                                        - 0 and 127, if IdType is FDCAN_STANDARD_ID
                                        - 0 and 63, if IdType is FDCAN_EXTENDED_ID                       */

  uint32_t IsFilterMatchingFrame; /*!< Specifies whether the accepted frame did not match any Rx filter.
                                       Acceptance of non-matching frames may be enabled via
                                       HAL_FDCAN_ConfigGlobalFilter().
                                       This parameter can be 0 or 1                                     */

} FDCAN_RxHeaderTypeDef;
```

- Identifier

  用于设置 ID，如果 IdType 是标准 FDCAN_STANDARD_ID，范围 0 到 0x7FF，如果 IdType 是 FDCAN_EXTENDED_ID 扩展 ID，范围 0 到 0x1FFFFFFF。

- IdType

  用于设置标志 ID 或者扩展 ID。

```
#define FDCAN_STANDARD_ID ((uint32_t)0x00000000U) /*!< 标准 ID */
#define FDCAN_EXTENDED_ID ((uint32_t)0x40000000U) /*!< 扩展 ID */
```

- RxFrameType

  用于设置接收帧类型，数据帧或遥控帧

```
#define FDCAN_DATA_FRAME   ((uint32_t)0x00000000U)  /*!< 数据帧 */
#define FDCAN_REMOTE_FRAME ((uint32_t)0x20000000U)  /*!< 遥控帧 */
```

- DataLength

  用于设置数据长度。

```
#define FDCAN_DLC_BYTES_0  ((uint32_t)0x00000000U) /*!< 0 bytes data field  */
#define FDCAN_DLC_BYTES_1  ((uint32_t)0x00010000U) /*!< 1 bytes data field  */
#define FDCAN_DLC_BYTES_2  ((uint32_t)0x00020000U) /*!< 2 bytes data field  */
#define FDCAN_DLC_BYTES_3  ((uint32_t)0x00030000U) /*!< 3 bytes data field  */
```

```
#define FDCAN_DLC_BYTES_4  ((uint32_t)0x00040000U) /*!< 4 bytes data field  */
#define FDCAN_DLC_BYTES_5  ((uint32_t)0x00050000U) /*!< 5 bytes data field  */
#define FDCAN_DLC_BYTES_6  ((uint32_t)0x00060000U) /*!< 6 bytes data field  */
#define FDCAN_DLC_BYTES_7  ((uint32_t)0x00070000U) /*!< 7 bytes data field  */
#define FDCAN_DLC_BYTES_8  ((uint32_t)0x00080000U) /*!< 8 bytes data field  */
#define FDCAN_DLC_BYTES_12 ((uint32_t)0x00090000U) /*!< 12 bytes data field */
#define FDCAN_DLC_BYTES_16 ((uint32_t)0x000A0000U) /*!< 16 bytes data field */
#define FDCAN_DLC_BYTES_20 ((uint32_t)0x000B0000U) /*!< 20 bytes data field */
#define FDCAN_DLC_BYTES_24 ((uint32_t)0x000C0000U) /*!< 24 bytes data field */
#define FDCAN_DLC_BYTES_32 ((uint32_t)0x000D0000U) /*!< 32 bytes data field */
#define FDCAN_DLC_BYTES_48 ((uint32_t)0x000E0000U) /*!< 48 bytes data field */
#define FDCAN_DLC_BYTES_64 ((uint32_t)0x000F0000U) /*!< 64 bytes data field */
```

- ErrorStateIndicator

  用于设置错误状态指示：

```
#define FDCAN_ESI_ACTIVE  ((uint32_t)0x00000000U) /*!< 传输节点 error active  */
#define FDCAN_ESI_PASSIVE ((uint32_t)0x80000000U) /*!< 传输节点 error passive */
```

- BitRateSwitch

  用于设置接收是否带波特率切换

```
#define FDCAN_BRS_OFF ((uint32_t)0x00000000U) /*!< FDCAN 帧发送/接收不支持波特率可变*/
#define FDCAN_BRS_ON  ((uint32_t)0x00100000U) /*!< FDCAN 帧发送/接收支持波特率可变 */
```

- FDFormat

  用于设置接收帧是经典格式还是 CANFD 格式

```
#define FDCAN_CLASSIC_CAN ((uint32_t)0x00000000U) /*!< 经典帧 */
#define FDCAN_FD_CAN      ((uint32_t)0x00200000U) /*!< FDCAN 帧 */
```

- RxTimestamp

  用于设置帧接收时间戳，范围 0 到 0xFFFF。

- FilterIndex

  用于设置接收过滤索引。如果是标准 ID，范围 0 到 127，如果是扩展 ID，范围 0 到 63。

- IsFilterMatchingFrame

  用于设置是否接收非匹配帧，通过函数 HAL_FDCAN_ConfigGlobalFilter()可以使能。

  0：表示不接受。

  1：表示接收。

91.3.8 FDCAN 总线句柄结构体 FDCAN_HandleTypeDef

下面是 CANFD 句柄结构体：

```
#if USE_HAL_FDCAN_REGISTER_CALLBACKS == 1
typedef struct __FDCAN_HandleTypeDef
#else
typedef struct
#endif /* USE_HAL_FDCAN_REGISTER_CALLBACKS */
{
  FDCAN_GlobalTypeDef         *Instance;        /*!< Register base address     */

  TTCAN_TypeDef               *ttcan;           /*!< TT register base address  */
```

```
  FDCAN_InitTypeDef           Init;             /*!< FDCAN required parameters */

  FDCAN_MsgRamAddressTypeDef  msgRam;           /*!< FDCAN Message RAM blocks  */

  uint32_t                    LatestTxFifoQRequest; /*!< FDCAN Tx buffer index
                                                     of latest Tx FIFO/Queue request */

  __IO HAL_FDCAN_StateTypeDef State;            /*!< FDCAN communication state */

  HAL_LockTypeDef             Lock;             /*!< FDCAN locking object      */

  __IO uint32_t               ErrorCode;        /*!< FDCAN Error code          */

#if USE_HAL_FDCAN_REGISTER_CALLBACKS == 1
  void (* ClockCalibrationCallback)(struct __FDCAN_HandleTypeDef *hfdcan, uint32_t ClkCalibrationITs);
/*!< FDCAN Clock Calibration callback          */
  void (* TxEventFifoCallback)(struct __FDCAN_HandleTypeDef *hfdcan, uint32_t TxEventFifoITs);
/*!< FDCAN Tx Event Fifo callback              */
  void (* RxFifo0Callback)(struct __FDCAN_HandleTypeDef *hfdcan, uint32_t RxFifo0ITs);
/*!< FDCAN Rx Fifo 0 callback                  */
  void (* RxFifo1Callback)(struct __FDCAN_HandleTypeDef *hfdcan, uint32_t RxFifo1ITs);
/*!< FDCAN Rx Fifo 1 callback                  */
  void (* TxFifoEmptyCallback)(struct __FDCAN_HandleTypeDef *hfdcan);
/*!< FDCAN Tx Fifo Empty callback              */
  void (* TxBufferCompleteCallback)(struct __FDCAN_HandleTypeDef *hfdcan, uint32_t BufferIndexes);
/*!< FDCAN Tx Buffer complete callback         */
  void (* TxBufferAbortCallback)(struct __FDCAN_HandleTypeDef *hfdcan, uint32_t BufferIndexes);
/*!< FDCAN Tx Buffer abort callback            */
  void (* RxBufferNewMessageCallback)(struct __FDCAN_HandleTypeDef *hfdcan);
/*!< FDCAN Rx Buffer New Message callback      */
  void (* HighPriorityMessageCallback)(struct __FDCAN_HandleTypeDef *hfdcan);
/*!< FDCAN High priority message callback      */
  void (* TimestampWraparoundCallback)(struct __FDCAN_HandleTypeDef *hfdcan);
/*!< FDCAN Timestamp wraparound callback       */
  void (* TimeoutOccurredCallback)(struct __FDCAN_HandleTypeDef *hfdcan);
/*!< FDCAN Timeout occurred callback           */
  void (* ErrorCallback)(struct __FDCAN_HandleTypeDef *hfdcan);
/*!< FDCAN Error callback                      */
  void (* ErrorStatusCallback)(struct __FDCAN_HandleTypeDef *hfdcan, uint32_t ErrorStatusITs);
/*!< FDCAN Error status callback               */
  void (* TT_ScheduleSyncCallback)(struct __FDCAN_HandleTypeDef *hfdcan, uint32_t TTSchedSyncITs);
/*!< FDCAN T Schedule Synchronization callback */
  void (* TT_TimeMarkCallback)(struct __FDCAN_HandleTypeDef *hfdcan, uint32_t TTTimeMarkITs);
/*!< FDCAN TT Time Mark callback               */
  void (* TT_StopWatchCallback)(struct __FDCAN_HandleTypeDef *hfdcan, uint32_t SWTime, uint32_t SWCycleCount);
/*!< FDCAN TT Stop Watch callback              */
  void (* TT_GlobalTimeCallback)(struct __FDCAN_HandleTypeDef *hfdcan, uint32_t TTGlobTimeITs);
/*!< FDCAN TT Global Time callback             */

  void (* MspInitCallback)(struct __FDCAN_HandleTypeDef *hfdcan);
/*!< FDCAN Msp Init callback                   */
  void (* MspDeInitCallback)(struct __FDCAN_HandleTypeDef *hfdcan);
/*!< FDCAN Msp DeInit callback                 */
#endif /* USE_HAL_FDCAN_REGISTER_CALLBACKS */

} FDCAN_HandleTypeDef;
```

注意事项：

条件编译 USE_HAL_FDCAN_REGISTER_CALLBACKS 用来设置使用自定义回调还是使用默认回调，此定义一般放在 stm32h7xx_hal_conf.h 文件里面设置：

```
#define    USE_HAL_FDCAN_REGISTER_CALLBACKS    1
```

函数 HAL_FDCAN_RegisterCallback 注册回调，取消注册使用函数 HAL_FDCAN_UnRegisterCallback。

这里重点介绍下面几个参数，其它参数主要是 HAL 库内部使用和自定义回调函数。

- FDCAN_GlobalTypeDef *Instance
  这个参数是寄存器的例化，方便操作寄存器。
- TTCAN_TypeDef *ttcan
  TT CAN 时间触发寄存器基地址。
- FDCAN_InitTypeDef Init
  FDCAN 相关的初始化参数。
- FDCAN_MsgRamAddressTypeDef msgRam
  消息 RAM 地址。
- uint32_t LatestTxFifoQRequest
  Tx buffer 索引最后一个 Tx FIFO/Queue 参数。
- __IO HAL_FDCAN_StateTypeDef State
  HAL_LockTypeDef Lock
  __IO uint32_t ErrorCode
  程序内部使用的状态参数。
- 剩下的都是回调函数

91.4 FD CAN 总线源文件 stm32h7xx_hal_fdcan.c

此文件涉及到的函数较多，这里把几个常用的函数做个说明：

91.4.1 函数 HAL_FDCAN_Init

函数原型：

```
HAL_StatusTypeDef HAL_FDCAN_Init(FDCAN_HandleTypeDef *hfdcan)
{
  uint32_t tickstart;
  HAL_StatusTypeDef status;
  const uint32_t CvtEltSize[] = {0, 0, 0, 0, 0, 1, 2, 3, 4, 0, 5, 0, 0, 0, 6, 0, 0, 0, 7};

  /* 检测 FDCAN 句柄是否有效 */
  if (hfdcan == NULL)
  {
    return HAL_ERROR;
```

```
  }

  /* 检查 FDCAN 例化 */
  if (hfdcan->Instance == FDCAN1)
  {
    hfdcan->ttcan = (TTCAN_TypeDef *)((uint32_t)hfdcan->Instance + 0x100U);
  }

  /* 检查函数参数 */
  assert_param(IS_FDCAN_ALL_INSTANCE(hfdcan->Instance));
  assert_param(IS_FDCAN_FRAME_FORMAT(hfdcan->Init.FrameFormat));
  assert_param(IS_FDCAN_MODE(hfdcan->Init.Mode));
  assert_param(IS_FUNCTIONAL_STATE(hfdcan->Init.AutoRetransmission));
  assert_param(IS_FUNCTIONAL_STATE(hfdcan->Init.TransmitPause));
  assert_param(IS_FUNCTIONAL_STATE(hfdcan->Init.ProtocolException));
  assert_param(IS_FDCAN_NOMINAL_PRESCALER(hfdcan->Init.NominalPrescaler));
  assert_param(IS_FDCAN_NOMINAL_SJW(hfdcan->Init.NominalSyncJumpWidth));
  assert_param(IS_FDCAN_NOMINAL_TSEG1(hfdcan->Init.NominalTimeSeg1));
  assert_param(IS_FDCAN_NOMINAL_TSEG2(hfdcan->Init.NominalTimeSeg2));
  if (hfdcan->Init.FrameFormat == FDCAN_FRAME_FD_BRS)
  {
    assert_param(IS_FDCAN_DATA_PRESCALER(hfdcan->Init.DataPrescaler));
    assert_param(IS_FDCAN_DATA_SJW(hfdcan->Init.DataSyncJumpWidth));
    assert_param(IS_FDCAN_DATA_TSEG1(hfdcan->Init.DataTimeSeg1));
    assert_param(IS_FDCAN_DATA_TSEG2(hfdcan->Init.DataTimeSeg2));
  }
  assert_param(IS_FDCAN_MAX_VALUE(hfdcan->Init.StdFiltersNbr, 128U));
  assert_param(IS_FDCAN_MAX_VALUE(hfdcan->Init.ExtFiltersNbr, 64U));
  assert_param(IS_FDCAN_MAX_VALUE(hfdcan->Init.RxFifo0ElmtsNbr, 64U));
  if (hfdcan->Init.RxFifo0ElmtsNbr > 0U)
  {
    assert_param(IS_FDCAN_DATA_SIZE(hfdcan->Init.RxFifo0ElmtSize));
  }
  assert_param(IS_FDCAN_MAX_VALUE(hfdcan->Init.RxFifo1ElmtsNbr, 64U));
  if (hfdcan->Init.RxFifo1ElmtsNbr > 0U)
  {
    assert_param(IS_FDCAN_DATA_SIZE(hfdcan->Init.RxFifo1ElmtSize));
  }
  assert_param(IS_FDCAN_MAX_VALUE(hfdcan->Init.RxBuffersNbr, 64U));
  if (hfdcan->Init.RxBuffersNbr > 0U)
  {
    assert_param(IS_FDCAN_DATA_SIZE(hfdcan->Init.RxBufferSize));
  }
  assert_param(IS_FDCAN_MAX_VALUE(hfdcan->Init.TxEventsNbr, 32U));
  assert_param(IS_FDCAN_MAX_VALUE((hfdcan->Init.TxBuffersNbr + hfdcan->Init.TxFifoQueueElmtsNbr), 32U));
  if (hfdcan->Init.TxFifoQueueElmtsNbr > 0U)
  {
    assert_param(IS_FDCAN_TX_FIFO_QUEUE_MODE(hfdcan->Init.TxFifoQueueMode));
  }
  if ((hfdcan->Init.TxBuffersNbr + hfdcan->Init.TxFifoQueueElmtsNbr) > 0U)
  {
    assert_param(IS_FDCAN_DATA_SIZE(hfdcan->Init.TxElmtSize));
  }

#if USE_HAL_FDCAN_REGISTER_CALLBACKS == 1
  if (hfdcan->State == HAL_FDCAN_STATE_RESET)
  {
    /* 解锁*/
```

```
    hfdcan->Lock = HAL_UNLOCKED;

    /* 复位设置默认回调 */
    hfdcan->ClockCalibrationCallback    = HAL_FDCAN_ClockCalibrationCallback;    /* Legacy weak
ClockCalibrationCallback    */
    hfdcan->TxEventFifoCallback         = HAL_FDCAN_TxEventFifoCallback;         /* Legacy weak
TxEventFifoCallback         */
    hfdcan->RxFifo0Callback             = HAL_FDCAN_RxFifo0Callback;             /* Legacy weak
RxFifo0Callback             */
    hfdcan->RxFifo1Callback             = HAL_FDCAN_RxFifo1Callback;             /* Legacy weak
RxFifo1Callback             */
    hfdcan->TxFifoEmptyCallback         = HAL_FDCAN_TxFifoEmptyCallback;         /* Legacy weak
TxFifoEmptyCallback         */
    hfdcan->TxBufferCompleteCallback    = HAL_FDCAN_TxBufferCompleteCallback;    /* Legacy weak
TxBufferCompleteCallback    */
    hfdcan->TxBufferAbortCallback       = HAL_FDCAN_TxBufferAbortCallback;       /* Legacy weak
TxBufferAbortCallback       */
    hfdcan->RxBufferNewMessageCallback  = HAL_FDCAN_RxBufferNewMessageCallback;  /* Legacy weak
RxBufferNewMessageCallback  */
    hfdcan->HighPriorityMessageCallback = HAL_FDCAN_HighPriorityMessageCallback; /* Legacy weak
HighPriorityMessageCallback */
    hfdcan->TimestampWraparoundCallback = HAL_FDCAN_TimestampWraparoundCallback; /* Legacy weak
TimestampWraparoundCallback */
    hfdcan->TimeoutOccurredCallback     = HAL_FDCAN_TimeoutOccurredCallback;     /* Legacy weak
TimeoutOccurredCallback     */
    hfdcan->ErrorCallback               = HAL_FDCAN_ErrorCallback;               /* Legacy weak ErrorCallback
*/
    hfdcan->ErrorStatusCallback         = HAL_FDCAN_ErrorStatusCallback;         /* Legacy weak
ErrorStatusCallback         */
    hfdcan->TT_ScheduleSyncCallback     = HAL_FDCAN_TT_ScheduleSyncCallback;     /* Legacy weak
TT_ScheduleSyncCallback     */
    hfdcan->TT_TimeMarkCallback         = HAL_FDCAN_TT_TimeMarkCallback;         /* Legacy weak
TT_TimeMarkCallback         */
    hfdcan->TT_StopWatchCallback        = HAL_FDCAN_TT_StopWatchCallback;        /* Legacy weak
TT_StopWatchCallback        */
    hfdcan->TT_GlobalTimeCallback       = HAL_FDCAN_TT_GlobalTimeCallback;       /* Legacy weak
TT_GlobalTimeCallback       */

    if (hfdcan->MspInitCallback == NULL)
    {
      hfdcan->MspInitCallback = HAL_FDCAN_MspInit;  /* Legacy weak MspInit */
    }

    /* 初始化 CLOCK 和 NVIC  */
    hfdcan->MspInitCallback(hfdcan);
  }
#else
  if (hfdcan->State == HAL_FDCAN_STATE_RESET)
  {
    /* 解锁 */
    hfdcan->Lock = HAL_UNLOCKED;

    /* 初始化底层硬件 */
    HAL_FDCAN_MspInit(hfdcan);
  }
#endif /* USE_HAL_FDCAN_REGISTER_CALLBACKS */

  /* 退出 Sleep  */
```

```
  CLEAR_BIT(hfdcan->Instance->CCCR, FDCAN_CCCR_CSR);

  /* 获取时钟 */
  tickstart = HAL_GetTick();

  /* 等待 Sleep 模式确认 */
  while ((hfdcan->Instance->CCCR & FDCAN_CCCR_CSA) == FDCAN_CCCR_CSA)
  {
    if ((HAL_GetTick() - tickstart) > FDCAN_TIMEOUT_VALUE)
    {
      /* 更新错误码 */
      hfdcan->ErrorCode |= HAL_FDCAN_ERROR_TIMEOUT;

      /* 改变 FDCAN 状态 */
      hfdcan->State = HAL_FDCAN_STATE_ERROR;

      return HAL_ERROR;
    }
  }

  /* 请求初始化 */
  SET_BIT(hfdcan->Instance->CCCR, FDCAN_CCCR_INIT);

  /* 获取时钟 */
  tickstart = HAL_GetTick();

  /* 等待 CCCR 寄存器的 INIT 位 */
  while ((hfdcan->Instance->CCCR & FDCAN_CCCR_INIT) == 0U)
  {
    /* 检查溢出时间 */
    if ((HAL_GetTick() - tickstart) > FDCAN_TIMEOUT_VALUE)
    {
      /* 更新错误码 */
      hfdcan->ErrorCode |= HAL_FDCAN_ERROR_TIMEOUT;

      /* 设置 FDCAN 状态 */
      hfdcan->State = HAL_FDCAN_STATE_ERROR;

      return HAL_ERROR;
    }
  }

  /* 使能配置修改 */
  SET_BIT(hfdcan->Instance->CCCR, FDCAN_CCCR_CCE);

  /* 设置是否自动重传 */
  if (hfdcan->Init.AutoRetransmission == ENABLE)
  {
    CLEAR_BIT(hfdcan->Instance->CCCR, FDCAN_CCCR_DAR);
  }
  else
  {
    SET_BIT(hfdcan->Instance->CCCR, FDCAN_CCCR_DAR);
  }

  /* 设置传输暂停特性 */
  if (hfdcan->Init.TransmitPause == ENABLE)
  {
```

```
  SET_BIT(hfdcan->Instance->CCCR, FDCAN_CCCR_TXP);
}
else
{
  CLEAR_BIT(hfdcan->Instance->CCCR, FDCAN_CCCR_TXP);
}

/* 设置协议异常处理 */
if (hfdcan->Init.ProtocolException == ENABLE)
{
  CLEAR_BIT(hfdcan->Instance->CCCR, FDCAN_CCCR_PXHD);
}
else
{
  SET_BIT(hfdcan->Instance->CCCR, FDCAN_CCCR_PXHD);
}

/* 设置 FDCAN 帧格式 */
MODIFY_REG(hfdcan->Instance->CCCR, FDCAN_FRAME_FD_BRS, hfdcan->Init.FrameFormat);

/* 复位 FDCAN 操作模式 */
CLEAR_BIT(hfdcan->Instance->CCCR, (FDCAN_CCCR_TEST | FDCAN_CCCR_MON | FDCAN_CCCR_ASM));
CLEAR_BIT(hfdcan->Instance->TEST, FDCAN_TEST_LBCK);

/* 设置 FDCAN 操作模式:
             | Normal | Restricted |    Bus     | Internal | External
             |        | Operation  | Monitoring | LoopBack | LoopBack
   CCCR.TEST |   0    |     0      |     0      |    1     |    1
   CCCR.MON  |   0    |     0      |     1      |    1     |    0
   TEST.LBCK |   0    |     0      |     0      |    1     |    1
   CCCR.ASM  |   0    |     1      |     0      |    0     |    0
*/
if (hfdcan->Init.Mode == FDCAN_MODE_RESTRICTED_OPERATION)
{
  /* 使能限制操作模式 */
  SET_BIT(hfdcan->Instance->CCCR, FDCAN_CCCR_ASM);
}
else if (hfdcan->Init.Mode != FDCAN_MODE_NORMAL)
{
  if (hfdcan->Init.Mode != FDCAN_MODE_BUS_MONITORING)
  {
    /* TEST 寄存器写访问使能 */
    SET_BIT(hfdcan->Instance->CCCR, FDCAN_CCCR_TEST);

    /* 使能回环模式 */
    SET_BIT(hfdcan->Instance->TEST, FDCAN_TEST_LBCK);

    if (hfdcan->Init.Mode == FDCAN_MODE_INTERNAL_LOOPBACK)
    {
      SET_BIT(hfdcan->Instance->CCCR, FDCAN_CCCR_MON);
    }
  }
  else
  {
    /* 使能检测模式 */
    SET_BIT(hfdcan->Instance->CCCR, FDCAN_CCCR_MON);
  }
}
```

```
  else
  {
    /* Nothing to do: normal mode */
  }

  /* 设置 nominal bit timing 寄存器 */
  hfdcan->Instance->NBTP = ((((uint32_t)hfdcan->Init.NominalSyncJumpWidth - 1U) << FDCAN_NBTP_NSJW_Pos) | \
                            (((uint32_t)hfdcan->Init.NominalTimeSeg1 - 1U) << FDCAN_NBTP_NTSEG1_Pos)    | \
                            (((uint32_t)hfdcan->Init.NominalTimeSeg2 - 1U) << FDCAN_NBTP_NTSEG2_Pos)    | \
                            (((uint32_t)hfdcan->Init.NominalPrescaler - 1U) << FDCAN_NBTP_NBRP_Pos));

  /* 如果使能 BRS（BitRate Switching 可变波特率），设置 data bit timing 寄存器*/
  if (hfdcan->Init.FrameFormat == FDCAN_FRAME_FD_BRS)
  {
    hfdcan->Instance->DBTP = ((((uint32_t)hfdcan->Init.DataSyncJumpWidth - 1U) << FDCAN_DBTP_DSJW_Pos) | \
                              (((uint32_t)hfdcan->Init.DataTimeSeg1 - 1U) << FDCAN_DBTP_DTSEG1_Pos)    | \
                              (((uint32_t)hfdcan->Init.DataTimeSeg2 - 1U) << FDCAN_DBTP_DTSEG2_Pos)    | \
                              (((uint32_t)hfdcan->Init.DataPrescaler - 1U) << FDCAN_DBTP_DBRP_Pos));
  }

  if (hfdcan->Init.TxFifoQueueElmtsNbr > 0U)
  {
    /* 设置 Tx FIFO 或 Tx Queue 操作模式 */
    SET_BIT(hfdcan->Instance->TXBC, hfdcan->Init.TxFifoQueueMode);
  }

  /* 配置 Tx element 大小 */
  if ((hfdcan->Init.TxBuffersNbr + hfdcan->Init.TxFifoQueueElmtsNbr) > 0U)
  {
    MODIFY_REG(hfdcan->Instance->TXESC, FDCAN_TXESC_TBDS, CvtEltSize[hfdcan->Init.TxElmtSize]);
  }

  /* 配置 Rx FIFO 0 大小 */
  if (hfdcan->Init.RxFifo0ElmtsNbr > 0U)
  {
    MODIFY_REG(hfdcan->Instance->RXESC, FDCAN_RXESC_F0DS, (CvtEltSize[hfdcan->Init.RxFifo0ElmtSize] <<
FDCAN_RXESC_F0DS_Pos));
  }

  /* 配置 Rx FIFO 1 大小 */
  if (hfdcan->Init.RxFifo1ElmtsNbr > 0U)
  {
    MODIFY_REG(hfdcan->Instance->RXESC, FDCAN_RXESC_F1DS, (CvtEltSize[hfdcan->Init.RxFifo1ElmtSize] <<
FDCAN_RXESC_F1DS_Pos));
  }

  /* 配置 Rx buffer element 大小 */
  if (hfdcan->Init.RxBuffersNbr > 0U)
  {
    MODIFY_REG(hfdcan->Instance->RXESC, FDCAN_RXESC_RBDS, (CvtEltSize[hfdcan->Init.RxBufferSize] <<
FDCAN_RXESC_RBDS_Pos));
  }

  /* 默认是的事件触发模式，如果使用时间触发模式，用户需要在 HAL_FDCAN_Init
     后调用 HAL_FDCAN_TT_ConfigOperation */
  if (hfdcan->Instance == FDCAN1)
  {
    CLEAR_BIT(hfdcan->ttcan->TTOCF, FDCAN_TTOCF_OM);
```

```
  }

  /* 初始化 Tx FIFO/Queue */
  hfdcan->LatestTxFifoQRequest = 0U;

  /* 初始化错误码 */
  hfdcan->ErrorCode = HAL_FDCAN_ERROR_NONE;

  /* 初始化 FDCAN 状态 */
  hfdcan->State = HAL_FDCAN_STATE_READY;

  /* 计算每个 RAM 块地址 */
  status = FDCAN_CalcultateRamBlockAddresses(hfdcan);

  /* 返回函数状态 */
  return status;
}
```

函数描述：

此函数用于初始化 FDCAN。

函数参数：

- 第 1 个参数是 FDCAN_HandleTypeDef 类型结构体指针变量，用于配置要初始化的参数。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

注意事项：

7. 函数 HAL_FDCAN_MspInit 用于初始化 FDCAN 的底层时钟、引脚等功能。需要用户自己在此函数里面实现具体的功能。由于这个函数是弱定义的，允许用户在工程其它源文件里面重新实现此函数。当然，不限制一定要在此函数里面实现，也可以像早期的标准库那样，用户自己初始化即可，更灵活些。
8. 如果形参 hfdcan 的结构体成员 State 没有做初始状态，这个地方就是个坑。特别是用户搞了一个局部变量 FDCAN_HandleTypeDef FdCANHandle。

   对于局部变量来说，这个参数就是一个随机值，如果是全局变量还好，一般 MDK 和 IAR 都会将全部变量初始化为 0，而恰好这个 HAL_FDCAN_STATE_RESET = 0x00U。

   解决办法有三

   方法 1：用户自己初始化 FDCAN 和涉及到的 GPIO 等。

   方法 2：定义 FDCAN_HandleTypeDef FdCANHandle 为全局变量。

   方法 3：下面的方法

```
if(HAL_FDCAN_DeInit(&FdCANHandle) != HAL_OK)
{
    Error_Handler();
}
if(HAL_FDCAN_Init(&FdCANHandle) != HAL_OK)
{
    Error_Handler();
}
```

使用举例：

```
FDCAN_HandleTypeDef hfdcan1;
```

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitCan1
*    功能说明: 初始 CAN1
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitCan1(void)
{
    /*                    位时间特性配置
        Bit time parameter          | Nominal      |  Data
        ----------------------------|--------------|---------------
        fdcan_ker_ck                | 20 MHz       | 20 MHz
        Time_quantum (tq)           | 50 ns        | 50 ns
        Synchronization_segment     | 1 tq         | 1 tq
        Propagation_segment         | 23 tq        | 1 tq
        Phase_segment_1             | 8 tq         | 4 tq
        Phase_segment_2             | 8 tq         | 4 tq
        Synchronization_Jump_width  | 8 tq         | 4 tq
        Bit_length                  | 40 tq = 2us  | 10 tq = 0.5us
        Bit_rate                    | 0.5 MBit/s   | 2 MBit/s
    */
    hfdcan1.Instance = FDCAN1;                      /* 配置 FDCAN1 */
    hfdcan1.Init.FrameFormat = FDCAN_FRAME_FD_BRS; /* 配置使用 FDCAN 可变波特率 */
    hfdcan1.Init.Mode = FDCAN_MODE_NORMAL;          /* 配置使用正常模式 */
    hfdcan1.Init.AutoRetransmission = ENABLE;       /*使能自动重发 */
    hfdcan1.Init.TransmitPause = DISABLE;           /* 配置禁止传输暂停特性 */
    hfdcan1.Init.ProtocolException = ENABLE;        /* 协议异常处理使能 */

    /*
        配置仲裁阶段波特率
        CAN 时钟 20MHz 时，仲裁阶段的波特率就是
        CAN FD Freq / (Sync_Seg + Pro_Seg + Phase_Seg1 + Phase_Seg2) = 20MHz / （1+0x1F + 8） = 0.5Mbps


        其中 Sync_Seg 是固定值 = 1 ， Pro_Seg + Phase_Seg1 = NominalTimeSeg1， Phase_Seg2 = NominalTimeSeg2
    */
    /* CAN 时钟分配设置，一般设置为 1 即可，全部由 PLL 配置好，tq = NominalPrescaler x (1/ fdcan_ker_ck) */
    hfdcan1.Init.NominalPrescaler = 0x01;

    /* 用于动态调节  Phase_Seg1 和 Phase_Seg1，所以不可以比 Phase_Seg1 和 Phase_Seg1 大 */
    hfdcan1.Init.NominalSyncJumpWidth = 0x08;

    /* 特别注意这里的 Seg1，这里是两个参数之和，对应位时间特性图的 Pro_Seg + Phase_Seg1 */
    hfdcan1.Init.NominalTimeSeg1 = 0x1F;

    /* 对应位时间特性图的 Phase_Seg2 */
    hfdcan1.Init.NominalTimeSeg2 = 0x08;


    /*
        配置数据阶段波特率
        CAN 时钟 20MHz 时，数据阶段的波特率就是
        CAN FD Freq / (Sync_Seg + Pro_Seg + Phase_Seg1 + Phase_Seg2) = 20MHz / （1+5+ 4） = 2Mbps

        其中 Sync_Seg 是固定值 = 1 ， Pro_Seg + Phase_Seg1 = DataTimeSeg1， Phase_Seg2 = DataTimeSeg2
```

```
    */
    /* CAN 时钟分配设置，一般设置为 1 即可，全部由 PLL 配置好，tq = NominalPrescaler x (1/ fdcan_ker_ck)，
       范围 1-32 */
    hfdcan1.Init.DataPrescaler = 0x01;

    /* 用于动态调节  Phase_Seg1 和 Phase_Seg1，所以不可以比 Phase_Seg1 和 Phase_Seg1 大，范围 1-16 */
    hfdcan1.Init.DataSyncJumpWidth = 0x04;

    /* 特别注意这里的 Seg1，这里是两个参数之和，对应位时间特性图的 Pro_Seg + Phase_Seg1，范围 */
    hfdcan1.Init.DataTimeSeg1 = 0x05;

    /* 对应位时间特性图的 Phase_Seg2 */
    hfdcan1.Init.DataTimeSeg2 = 0x04;


    hfdcan1.Init.MessageRAMOffset = 0;      /* CAN1 和 CAN2 共享 2560 个字，这里 CAN1 分配前 1280 字 */


    hfdcan1.Init.StdFiltersNbr = 1;                    /* 设置标准 ID 过滤器个数，范围 0-128 */
    hfdcan1.Init.ExtFiltersNbr = 0;                    /* 设置扩展 ID 过滤器个数，范围 0-64 */
    hfdcan1.Init.RxFifo0ElmtsNbr = 2;                   /* 设置 Rx FIFO0 的元素个数，范围 0-64 */
    /* 设置 Rx FIFO0 中每个元素大小，支持 8,12,16,20,24,32,48 或者 64 字节 */
    hfdcan1.Init.RxFifo0ElmtSize = FDCAN_DATA_BYTES_8;
    hfdcan1.Init.RxFifo1ElmtsNbr = 0;                   /* 设置 Rx FIFO1 的元素个数，范围 0-64 */

    /* 设置 Rx FIFO1 中每个元素大小，支持 8,12,16,20,24,32,48 或者 64 字节 */
    hfdcan1.Init.RxFifo1ElmtSize = FDCAN_DATA_BYTES_8;
    hfdcan1.Init.RxBuffersNbr = 0;                      /* 设置 Rx Buffer 个数，范围 0-64 */

    /* 设置 Rx Buffer 中每个元素大小，支持 8,12,16,20,24,32,48 或者 64 字节 */
    hfdcan1.Init.RxBufferSize = 0;


    hfdcan1.Init.TxEventsNbr = 0;          /* 设置 Tx Event FIFO 中元素个数，范围 0-32 */
    hfdcan1.Init.TxBuffersNbr = 0;         /* 设置 Tx Buffer 中元素个数，范围 0-32 */
    hfdcan1.Init.TxFifoQueueElmtsNbr = 2; /* 设置用于 Tx FIFO/Queue 的 Tx Buffers 个数。范围 0 到 32 */
    hfdcan1.Init.TxFifoQueueMode = FDCAN_TX_FIFO_OPERATION; /* 设置 FIFO 模式或者 QUEUE 队列模式 */

    /* 设置 Tx Element 中的数据域大小，支持 8,12,16,20,24,32,48 或者 64 字节 */
    hfdcan1.Init.TxElmtSize = FDCAN_DATA_BYTES_8;
    HAL_FDCAN_Init(&hfdcan1);


    //省略未写

}
```

91.4.2 函数 HAL_FDCAN_DeInit

函数原型：

```
HAL_StatusTypeDef HAL_FDCAN_DeInit(FDCAN_HandleTypeDef *hfdcan)
{
  /* 检测句柄 */
  if (hfdcan == NULL)
```

```
  {
    return HAL_ERROR;
  }

  /* 检查函数形参 */
  assert_param(IS_FDCAN_ALL_INSTANCE(hfdcan->Instance));

  /* 停止 FDCAN */
  (void)HAL_FDCAN_Stop(hfdcan);

  /* 禁止中断 */
  CLEAR_BIT(hfdcan->Instance->ILE, (FDCAN_INTERRUPT_LINE0 | FDCAN_INTERRUPT_LINE1));

#if USE_HAL_FDCAN_REGISTER_CALLBACKS == 1
  if (hfdcan->MspDeInitCallback == NULL)
  {
    hfdcan->MspDeInitCallback = HAL_FDCAN_MspDeInit; /* Legacy weak MspDeInit */
  }

  /* 复位底层硬件，如时钟，NVIC 等 */
  hfdcan->MspDeInitCallback(hfdcan);
#else
 /* 复位底层硬件，如时钟，NVIC 等 */
  HAL_FDCAN_MspDeInit(hfdcan);
#endif

  /* 复位错误码 */
  hfdcan->ErrorCode = HAL_FDCAN_ERROR_NONE;

  /* 设置 FDCAN 状态 */
  hfdcan->State = HAL_FDCAN_STATE_RESET;

  /* 返回 HAL_OK */
  return HAL_OK;
}
```

函数描述：

用于复位 FDCAN。

函数参数：

- 第 1 个参数是 FDCAN_HandleTypeDef 类型结构体指针变量。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

91.4.3 函数 HAL_FDCAN_ConfigFilter

函数原型：

```
HAL_StatusTypeDef HAL_FDCAN_ConfigFilter(FDCAN_HandleTypeDef *hfdcan, FDCAN_FilterTypeDef *sFilterConfig)
{
  uint32_t FilterElementW1;
  uint32_t FilterElementW2;
  uint32_t *FilterAddress;
  HAL_FDCAN_StateTypeDef state = hfdcan->State;
```

```
if ((state == HAL_FDCAN_STATE_READY) || (state == HAL_FDCAN_STATE_BUSY))
{
  /* 检测函数参数 */
  assert_param(IS_FDCAN_ID_TYPE(sFilterConfig->IdType));
  assert_param(IS_FDCAN_FILTER_CFG(sFilterConfig->FilterConfig));
  if (sFilterConfig->FilterConfig == FDCAN_FILTER_TO_RXBUFFER)
  {
    assert_param(IS_FDCAN_MAX_VALUE(sFilterConfig->RxBufferIndex, 63U));
    assert_param(IS_FDCAN_MAX_VALUE(sFilterConfig->IsCalibrationMsg, 1U));
  }

  /* 标准 ID */
  if (sFilterConfig->IdType == FDCAN_STANDARD_ID)
  {
    /* 检测函数形参 */
    assert_param(IS_FDCAN_MAX_VALUE(sFilterConfig->FilterIndex, (hfdcan->Init.StdFiltersNbr - 1U)));
    assert_param(IS_FDCAN_MAX_VALUE(sFilterConfig->FilterID1, 0x7FFU));
    if (sFilterConfig->FilterConfig != FDCAN_FILTER_TO_RXBUFFER)
    {
      assert_param(IS_FDCAN_MAX_VALUE(sFilterConfig->FilterID2, 0x7FFU));
      assert_param(IS_FDCAN_STD_FILTER_TYPE(sFilterConfig->FilterType));
    }

    /* 构建过滤元素 */
    if (sFilterConfig->FilterConfig == FDCAN_FILTER_TO_RXBUFFER)
    {
      FilterElementW1 = ((FDCAN_FILTER_TO_RXBUFFER << 27U)       |
                         (sFilterConfig->FilterID1 << 16U)       |
                         (sFilterConfig->IsCalibrationMsg << 8U) |
                         sFilterConfig->RxBufferIndex);
    }
    else
    {
      FilterElementW1 = ((sFilterConfig->FilterType << 30U)   |
                         (sFilterConfig->FilterConfig << 27U) |
                         (sFilterConfig->FilterID1 << 16U)    |
                         sFilterConfig->FilterID2);
    }

    /* 计算过滤地址 */
    FilterAddress = (uint32_t *)(hfdcan->msgRam.StandardFilterSA + (sFilterConfig->FilterIndex * 4U));

    /* 将过滤元素写到消息 RAM 中 */
    *FilterAddress = FilterElementW1;
  }
  /* 扩展 ID */
  else
  {
    /* 检测函数参数 */
    assert_param(IS_FDCAN_MAX_VALUE(sFilterConfig->FilterIndex, (hfdcan->Init.ExtFiltersNbr - 1U)));
    assert_param(IS_FDCAN_MAX_VALUE(sFilterConfig->FilterID1, 0x1FFFFFFFU));
    if (sFilterConfig->FilterConfig != FDCAN_FILTER_TO_RXBUFFER)
    {
      assert_param(IS_FDCAN_MAX_VALUE(sFilterConfig->FilterID2, 0x1FFFFFFFU));
      assert_param(IS_FDCAN_EXT_FILTER_TYPE(sFilterConfig->FilterType));
    }
```

```
      /* 构建第 1 个 word 的过滤元素 */
      FilterElementW1 = ((sFilterConfig->FilterConfig << 29U) | sFilterConfig->FilterID1);

      /* 构建第 2 个 word 的过滤元素 */
      if (sFilterConfig->FilterConfig == FDCAN_FILTER_TO_RXBUFFER)
      {
        FilterElementW2 = sFilterConfig->RxBufferIndex;
      }
      else
      {
        FilterElementW2 = ((sFilterConfig->FilterType << 30U) | sFilterConfig->FilterID2);
      }

      /* 计算过滤地址 */
      FilterAddress = (uint32_t *)(hfdcan->msgRam.ExtendedFilterSA + (sFilterConfig->FilterIndex * 4U * 2U));

      /* 写过滤元素到消息 RAM */
      *FilterAddress = FilterElementW1;
      FilterAddress++;
      *FilterAddress = FilterElementW2;
    }

    return HAL_OK;
  }
  else
  {
    /* 更新错误码 e */
    hfdcan->ErrorCode |= HAL_FDCAN_ERROR_NOT_INITIALIZED;

    return HAL_ERROR;
  }
}
```

函数描述：

此函数主要用于设置 FDCAN 过滤。

函数参数：

- 第 1 个参数是 FDCAN_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是 FDCAN_FilterTypeDef 类型结构体变量，主要用于过滤参数配置。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

使用举例：

```
FDCAN_HandleTypeDef hfdcan1;

/*
*********************************************************************************************************
*    函 数 名: bsp_InitCan1
*    功能说明: 初始 CAN1
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitCan1(void)
{
```

```
    /*
        配置过滤器，过滤器主要用于接收，这里采样屏蔽位模式。
        FilterID1 = filter
        FilterID2 = mask

        FilterID2 的 mask 每个 bit 含义
        0: 不关心，该位不用于比较;
        1: 必须匹配，接收到的 ID 必须与滤波器对应的 ID 位相一致。

        举例说明:
        FilterID1 = 0x111
        FilterID2 = 0x7FF
        表示仅接收 ID 为 0x111 的 FDCAN 帧。

    */
    sFilterConfig1.IdType = FDCAN_STANDARD_ID;              /* 设置标准 ID 或者扩展 ID */
    /* 用于过滤索引，如果是标准 ID，范围 0 到 127。如果是扩展 ID，范围 0 到 64 */
    sFilterConfig1.FilterIndex = 0;
    sFilterConfig1.FilterType = FDCAN_FILTER_MASK;          /* 过滤器采样屏蔽位模式 */
    sFilterConfig1.FilterConfig = FDCAN_FILTER_TO_RXFIFO0;  /* 如果过滤匹配，将数据保存到 Rx FIFO 0 */
    sFilterConfig1.FilterID1 = 0x111;                       /* 屏蔽位模式下，FilterID1 是消息 ID */
    sFilterConfig1.FilterID2 = 0x7FF;                       /* 屏蔽位模式下，FilterID2 是消息屏蔽位 */
    HAL_FDCAN_ConfigFilter(&hfdcan1, &sFilterConfig1);      /* 配置过滤器 */

}
```

91.4.4函数 HAL_FDCAN_ConfigFifoWatermark

函数原型:

```
HAL_StatusTypeDef HAL_FDCAN_ConfigFifoWatermark(FDCAN_HandleTypeDef *hfdcan, uint32_t FIFO, uint32_t
Watermark)
{
  /* 检测参数 */
  assert_param(IS_FDCAN_FIFO_WATERMARK(FIFO));
  if (FIFO == FDCAN_CFG_TX_EVENT_FIFO)
  {
    assert_param(IS_FDCAN_MAX_VALUE(Watermark, 32U));
  }
  else /* (FIFO == FDCAN_CFG_RX_FIFO0) || (FIFO == FDCAN_CFG_RX_FIFO1) */
  {
    assert_param(IS_FDCAN_MAX_VALUE(Watermark, 64U));
  }

  if (hfdcan->State == HAL_FDCAN_STATE_READY)
  {
    /* 设置发送事件 FIFO */
    if (FIFO == FDCAN_CFG_TX_EVENT_FIFO)
    {
      MODIFY_REG(hfdcan->Instance->TXEFC, FDCAN_TXEFC_EFWM, (Watermark << FDCAN_TXEFC_EFWM_Pos));
    }
    /* 设置接收 FIFO0 */
    else if (FIFO == FDCAN_CFG_RX_FIFO0)
    {
      MODIFY_REG(hfdcan->Instance->RXF0C, FDCAN_RXF0C_F0WM, (Watermark << FDCAN_RXF0C_F0WM_Pos));
```

```
    }
    /* 设置接收 FIFO1 */
    else
    {
      MODIFY_REG(hfdcan->Instance->RXF1C, FDCAN_RXF1C_F1WM, (Watermark << FDCAN_RXF1C_F1WM_Pos));
    }

    /* 返回状态 */
    return HAL_OK;
  }
  else
  {
    /* 更新错误码 */
    hfdcan->ErrorCode |= HAL_FDCAN_ERROR_NOT_READY;

    return HAL_ERROR;
  }
}
```

函数描述：

此函数主要用于配置 FIFO Watermark

函数参数：

- 第 1 个参数是 FDCAN_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是 FDCAN FIFO Watermark

```
#define FDCAN_CFG_TX_EVENT_FIFO ((uint32_t)0x00000000U) /*!< Tx event FIFO */
#define FDCAN_CFG_RX_FIFO0      ((uint32_t)0x00000001U) /*!< Rx FIFO0      */
#define FDCAN_CFG_RX_FIFO1      ((uint32_t)0x00000002U) /*!< Rx FIFO1      */
```

- 第 3 个参数 FIFO Watermark 中断位置，如果是 FDCAN_CFG_TX_EVENT_FIFO，范围 0 到 32，如果是 FDCAN_CFG_RX_FIFO0 或 FDCAN_CFG_RX_FIFO1，范围 0 到 64。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

使用举例：

```
FDCAN_HandleTypeDef hfdcan1;

/* 设置 Rx FIFO0 的 wartermark 为 1 */
HAL_FDCAN_ConfigFifoWatermark(&hfdcan1, FDCAN_CFG_RX_FIFO0, 1);
```

91.4.5 函数 HAL_FDCAN_ActivateNotification

函数原型：

```
HAL_StatusTypeDef HAL_FDCAN_ActivateNotification(FDCAN_HandleTypeDef *hfdcan, uint32_t ActiveITs, uint32_t
BufferIndexes)
{
  HAL_FDCAN_StateTypeDef state = hfdcan->State;

  /* 检测函数形参 */
  assert_param(IS_FDCAN_IT(ActiveITs));
```

```
  if ((state == HAL_FDCAN_STATE_READY) || (state == HAL_FDCAN_STATE_BUSY))
  {
    /* 使能中断行 */
    if ((ActiveITs & hfdcan->Instance->ILS) == 0U)
    {
      /* 使能中断行 0 */
      SET_BIT(hfdcan->Instance->ILE, FDCAN_INTERRUPT_LINE0);
    }
    else if ((ActiveITs & hfdcan->Instance->ILS) == ActiveITs)
    {
      /* 使能中断行 1 */
      SET_BIT(hfdcan->Instance->ILE, FDCAN_INTERRUPT_LINE1);
    }
    else
    {
      /* 使能中断行 0 和中断行 1 */
      hfdcan->Instance->ILE = (FDCAN_INTERRUPT_LINE0 | FDCAN_INTERRUPT_LINE1);
    }

    if ((ActiveITs & FDCAN_IT_TX_COMPLETE) != 0U)
    {
      /* 使能 Tx Buffer Transmission 中断 */
      SET_BIT(hfdcan->Instance->TXBTIE, BufferIndexes);
    }

    if ((ActiveITs & FDCAN_IT_TX_ABORT_COMPLETE) != 0U)
    {
      /* 使能 Tx Buffer Cancellation Finished 中断 */
      SET_BIT(hfdcan->Instance->TXBCIE, BufferIndexes);
    }

    /* 使能选择的中断 */
    __HAL_FDCAN_ENABLE_IT(hfdcan, ActiveITs);

    /* 返回函数状态 */
    return HAL_OK;
  }
  else
  {
    /* 更新错误码 */
    hfdcan->ErrorCode |= HAL_FDCAN_ERROR_NOT_INITIALIZED;

    return HAL_ERROR;
  }
}
```

函数描述：

此函数主要用于使能中断。

函数参数：

- 第 1 个参数是 FDCAN_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数用于设置要使能的中断，支持的参数如下：

```
#define FDCAN_IT_TX_COMPLETE         FDCAN_IE_TCE   /*!< Transmission Completed
#define FDCAN_IT_TX_ABORT_COMPLETE   FDCAN_IE_TCFE  /*!< Transmission Cancellation Finished      */
#define FDCAN_IT_TX_FIFO_EMPTY       FDCAN_IE_TFEE  /*!< Tx FIFO Empty
```

```
#define FDCAN_IT_RX_HIGH_PRIORITY_MSG  FDCAN_IE_HPME  /*!< High priority message received       */
#define FDCAN_IT_RX_BUFFER_NEW_MESSAGE FDCAN_IE_DRXE  /*!< At least one received message stored into a Rx Buffer

#define FDCAN_IT_TIMESTAMP_WRAPAROUND  FDCAN_IE_TSWE  /*!< Timestamp counter wrapped around     */
#define FDCAN_IT_TIMEOUT_OCCURRED      FDCAN_IE_TOOE  /*!< Timeout reached                 */

#define FDCAN_IT_CALIB_STATE_CHANGED  (FDCANCCU_IE_CSCE << 30) /*!< Clock calibration state changed    */
#define FDCAN_IT_CALIB_WATCHDOG_EVENT (FDCANCCU_IE_CWEE << 30) /*!< Clock calibration watchdog event occurred

#define FDCAN_IT_TX_EVT_FIFO_ELT_LOST  FDCAN_IE_TEFLE /*!< Tx Event FIFO element lost                */
#define FDCAN_IT_TX_EVT_FIFO_FULL      FDCAN_IE_TEFFE /*!< Tx Event FIFO full                        */
#define FDCAN_IT_TX_EVT_FIFO_WATERMARK FDCAN_IE_TEFWE /*!< Tx Event FIFO fill level reached watermark */
#define FDCAN_IT_TX_EVT_FIFO_NEW_DATA  FDCAN_IE_TEFNE /*!< Tx Handler wrote Tx Event FIFO element     */

#define FDCAN_IT_RX_FIFO0_MESSAGE_LOST FDCAN_IE_RFOLE /*!< Rx FIFO 0 message lost                */
#define FDCAN_IT_RX_FIFO0_FULL         FDCAN_IE_RFOFE /*!< Rx FIFO 0 full                        */
#define FDCAN_IT_RX_FIFO0_WATERMARK    FDCAN_IE_RFOWE /*!< Rx FIFO 0 fill level reached watermark */
#define FDCAN_IT_RX_FIFO0_NEW_MESSAGE  FDCAN_IE_RFONE /*!< New message written to Rx FIFO 0       */

#define FDCAN_IT_RX_FIFO1_MESSAGE_LOST FDCAN_IE_RF1LE /*!< Rx FIFO 1 message lost                */
#define FDCAN_IT_RX_FIFO1_FULL         FDCAN_IE_RF1FE /*!< Rx FIFO 1 full                        */
#define FDCAN_IT_RX_FIFO1_WATERMARK    FDCAN_IE_RF1WE /*!< Rx FIFO 1 fill level reached watermark */
#define FDCAN_IT_RX_FIFO1_NEW_MESSAGE  FDCAN_IE_RF1NE /*!< New message written to Rx FIFO 1       */

#define FDCAN_IT_RAM_ACCESS_FAILURE      FDCAN_IE_MRAFE /*!< Message RAM access failure occurred
#define FDCAN_IT_ERROR_LOGGING_OVERFLOW  FDCAN_IE_ELOE  /*!< Overflow of FDCAN Error Logging Counter occurred
#define FDCAN_IT_RAM_WATCHDOG            FDCAN_IE_WDIE  /*!< Message RAM Watchdog event due to missing READY
#define FDCAN_IT_ARB_PROTOCOL_ERROR      FDCAN_IE_PEAE  /*!< Protocol error in arbitration phase detected
#define FDCAN_IT_DATA_PROTOCOL_ERROR     FDCAN_IE_PEDE  /*!< Protocol error in data phase detected
#define FDCAN_IT_RESERVED_ADDRESS_ACCESS FDCAN_IE_ARAE  /*!< Access to reserved address occurred

#define FDCAN_IT_ERROR_PASSIVE FDCAN_IE_EPE /*!< Error_Passive status changed */
#define FDCAN_IT_ERROR_WARNING FDCAN_IE_EWE /*!< Error_Warning status changed */
#define FDCAN_IT_BUS_OFF       FDCAN_IE_BOE /*!< Bus_Off status changed       */
```

◆ 第 3 个参数是 Tx Buffer Indexes，可以如下参数的任意组合：

```
#define FDCAN_TX_BUFFER0  ((uint32_t)0x00000001U) /*!< Add message to Tx Buffer 0  */
#define FDCAN_TX_BUFFER1  ((uint32_t)0x00000002U) /*!< Add message to Tx Buffer 1  */
#define FDCAN_TX_BUFFER2  ((uint32_t)0x00000004U) /*!< Add message to Tx Buffer 2  */
#define FDCAN_TX_BUFFER3  ((uint32_t)0x00000008U) /*!< Add message to Tx Buffer 3  */
#define FDCAN_TX_BUFFER4  ((uint32_t)0x00000010U) /*!< Add message to Tx Buffer 4  */
#define FDCAN_TX_BUFFER5  ((uint32_t)0x00000020U) /*!< Add message to Tx Buffer 5  */
#define FDCAN_TX_BUFFER6  ((uint32_t)0x00000040U) /*!< Add message to Tx Buffer 6  */
#define FDCAN_TX_BUFFER7  ((uint32_t)0x00000080U) /*!< Add message to Tx Buffer 7  */
#define FDCAN_TX_BUFFER8  ((uint32_t)0x00000100U) /*!< Add message to Tx Buffer 8  */
#define FDCAN_TX_BUFFER9  ((uint32_t)0x00000200U) /*!< Add message to Tx Buffer 9  */
#define FDCAN_TX_BUFFER10 ((uint32_t)0x00000400U) /*!< Add message to Tx Buffer 10 */
#define FDCAN_TX_BUFFER11 ((uint32_t)0x00000800U) /*!< Add message to Tx Buffer 11 */
#define FDCAN_TX_BUFFER12 ((uint32_t)0x00001000U) /*!< Add message to Tx Buffer 12 */
#define FDCAN_TX_BUFFER13 ((uint32_t)0x00002000U) /*!< Add message to Tx Buffer 13 */
#define FDCAN_TX_BUFFER14 ((uint32_t)0x00004000U) /*!< Add message to Tx Buffer 14 */
#define FDCAN_TX_BUFFER15 ((uint32_t)0x00008000U) /*!< Add message to Tx Buffer 15 */
#define FDCAN_TX_BUFFER16 ((uint32_t)0x00010000U) /*!< Add message to Tx Buffer 16 */
#define FDCAN_TX_BUFFER17 ((uint32_t)0x00020000U) /*!< Add message to Tx Buffer 17 */
#define FDCAN_TX_BUFFER18 ((uint32_t)0x00040000U) /*!< Add message to Tx Buffer 18 */
#define FDCAN_TX_BUFFER19 ((uint32_t)0x00080000U) /*!< Add message to Tx Buffer 19 */
#define FDCAN_TX_BUFFER20 ((uint32_t)0x00100000U) /*!< Add message to Tx Buffer 20 */
#define FDCAN_TX_BUFFER21 ((uint32_t)0x00200000U) /*!< Add message to Tx Buffer 21 */
```

```
#define FDCAN_TX_BUFFER22 ((uint32_t)0x00400000U) /*!< Add message to Tx Buffer 22 */
#define FDCAN_TX_BUFFER23 ((uint32_t)0x00800000U) /*!< Add message to Tx Buffer 23 */
#define FDCAN_TX_BUFFER24 ((uint32_t)0x01000000U) /*!< Add message to Tx Buffer 24 */
#define FDCAN_TX_BUFFER25 ((uint32_t)0x02000000U) /*!< Add message to Tx Buffer 25 */
#define FDCAN_TX_BUFFER26 ((uint32_t)0x04000000U) /*!< Add message to Tx Buffer 26 */
#define FDCAN_TX_BUFFER27 ((uint32_t)0x08000000U) /*!< Add message to Tx Buffer 27 */
#define FDCAN_TX_BUFFER28 ((uint32_t)0x10000000U) /*!< Add message to Tx Buffer 28 */
#define FDCAN_TX_BUFFER29 ((uint32_t)0x20000000U) /*!< Add message to Tx Buffer 29 */
#define FDCAN_TX_BUFFER30 ((uint32_t)0x40000000U) /*!< Add message to Tx Buffer 30 */
#define FDCAN_TX_BUFFER31 ((uint32_t)0x80000000U) /*!< Add message to Tx Buffer 31 */
```

如果第 2 个参数不包括 FDCAN_IT_TX_COMPLETE 或 FDCAN_IT_TX_ABORT_COMPLETE，此参数将被忽略。

- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

使用举例：

```
FDCAN_HandleTypeDef hfdcan1;

/* 激活 RX FIFO0 的 watermark 通知中断，位开启 Tx Buffer 中断*/
HAL_FDCAN_ActivateNotification(&hfdcan1, FDCAN_IT_RX_FIFO0_WATERMARK, 0);
```

91.4.6函数 HAL_FDCAN_Start

函数原型：

```
HAL_StatusTypeDef HAL_FDCAN_Start(FDCAN_HandleTypeDef *hfdcan)
{
  if (hfdcan->State == HAL_FDCAN_STATE_READY)
  {
      /* 设置 FDCAN 外设状态 */
    hfdcan->State = HAL_FDCAN_STATE_BUSY;

      /* 请求离开初始化状态 */
    CLEAR_BIT(hfdcan->Instance->CCCR, FDCAN_CCCR_INIT);

      /* 设置错误码 */
    hfdcan->ErrorCode = HAL_FDCAN_ERROR_NONE;

    return HAL_OK;
  }
  else
  {
    hfdcan->ErrorCode |= HAL_FDCAN_ERROR_NOT_READY;

    return HAL_ERROR;
  }
}
```

函数描述：

此函数主要用于启动 FDCAN。

函数参数：

- 第 1 个参数是 FDCAN_HandleTypeDef 类型结构体指针变量。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

使用举例：

```
/**
  * @brief  Writes block(s) to a specified address in an SD card, in DMA mode.
  * @param  pData: Pointer to the buffer that will contain the data to transmit
  * @param  WriteAddr: Address from where data is to be written
  * @param  NumOfBlocks: Number of SD blocks to write
  * @retval SD status
*/
uint8_t BSP_SD_WriteBlocks_DMA(uint32_t *pData, uint32_t WriteAddr, uint32_t NumOfBlocks)
{

  if( HAL_SD_WriteBlocks_DMA(&uSdHandle, (uint8_t *)pData, WriteAddr, NumOfBlocks) == HAL_OK)
  {
    return MSD_OK;
  }
  else
  {
    return MSD_ERROR;
  }
}
```

91.4.7 函数 HAL_FDCAN_AddMessageToTxFifoQ

函数原型：

```
HAL_StatusTypeDef HAL_FDCAN_AddMessageToTxFifoQ(FDCAN_HandleTypeDef *hfdcan, FDCAN_TxHeaderTypeDef *pTxHeader,
uint8_t *pTxData)
{
  uint32_t PutIndex;

    /* 检查函数形参 */
  assert_param(IS_FDCAN_ID_TYPE(pTxHeader->IdType));
  if (pTxHeader->IdType == FDCAN_STANDARD_ID)
  {
    assert_param(IS_FDCAN_MAX_VALUE(pTxHeader->Identifier, 0x7FFU));
  }
  else /* pTxHeader->IdType == FDCAN_EXTENDED_ID */
  {
    assert_param(IS_FDCAN_MAX_VALUE(pTxHeader->Identifier, 0x1FFFFFFFU));
  }
  assert_param(IS_FDCAN_FRAME_TYPE(pTxHeader->TxFrameType));
  assert_param(IS_FDCAN_DLC(pTxHeader->DataLength));
  assert_param(IS_FDCAN_ESI(pTxHeader->ErrorStateIndicator));
  assert_param(IS_FDCAN_BRS(pTxHeader->BitRateSwitch));
  assert_param(IS_FDCAN_FDF(pTxHeader->FDFormat));
  assert_param(IS_FDCAN_EFC(pTxHeader->TxEventFifoControl));
  assert_param(IS_FDCAN_MAX_VALUE(pTxHeader->MessageMarker, 0xFFU));
```

```
  if (hfdcan->State == HAL_FDCAN_STATE_BUSY)
  {
    /* 检测 Tx FIFO/Queue 是否在 RAM 中分配到空间 */
    if ((hfdcan->Instance->TXBC & FDCAN_TXBC_TFQS) == 0U)
    {
        /* 更新错误码 */
      hfdcan->ErrorCode |= HAL_FDCAN_ERROR_PARAM;

      return HAL_ERROR;
    }

      /* 检查 Tx FIFO/Queue 是否满 */
    if ((hfdcan->Instance->TXFQS & FDCAN_TXFQS_TFQF) != 0U)
    {
        /* 更新错误码 */
      hfdcan->ErrorCode |= HAL_FDCAN_ERROR_FIFO_FULL;

      return HAL_ERROR;
    }
    else
    {
        /* 获取 Tx FIFO PutIndex */
      PutIndex = ((hfdcan->Instance->TXFQS & FDCAN_TXFQS_TFQPI) >> FDCAN_TXFQS_TFQPI_Pos);

        /* 添加消息到 Tx FIFO/Queue */
      FDCAN_CopyMessageToRAM(hfdcan, pTxHeader, pTxData, PutIndex);

        /* 激活相应的传输请求 */
      hfdcan->Instance->TXBAR = ((uint32_t)1 << PutIndex);

        /* 存储最近的 Tx FIFO/Queue Request Buffer Index */
      hfdcan->LatestTxFifoQRequest = ((uint32_t)1 << PutIndex);
    }

      /* 返回错误状态 */
    return HAL_OK;
  }
  else
  {
      /* 更新错误码 */
    hfdcan->ErrorCode |= HAL_FDCAN_ERROR_NOT_STARTED;

    return HAL_ERROR;
  }
}
```

函数描述：

此函数用于添加消息到 Tx FIFO/Queue 并激活相应的传输请求。

函数参数：

- 第 1 个参数是 FDCAN_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是 FDCAN_TxHeaderTypeDef 类型结构体变量，用于消息发送。
- 第 3 个参数是要发送的数据地址。程序里面会将数据复制到发送缓冲区。

- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: can1_SendPacket
*    功能说明: 发送一包数据
*    形    参：_DataBuf 数据缓冲区
*              _Len 数据长度，支持 8,12,16,20,24,32,48 或者 64 字节
*    返 回 值: 无
*********************************************************************************************************
*/
void can1_SendPacket(uint8_t *_DataBuf, uint8_t _Len)
{
    FDCAN_TxHeaderTypeDef TxHeader = {0};

    /* 配置发送参数 */
    TxHeader.Identifier = 0x222;                    /* 设置接收帧消息的 ID */
    TxHeader.IdType = FDCAN_STANDARD_ID;            /* 标准 ID */
    TxHeader.TxFrameType = FDCAN_DATA_FRAME;        /* 数据帧 */
    TxHeader.DataLength = (uint32_t)_Len << 16;      /* 发送数据长度 */
    TxHeader.ErrorStateIndicator = FDCAN_ESI_ACTIVE; /* 设置错误状态指示 */
    TxHeader.BitRateSwitch = FDCAN_BRS_ON;           /* 开启可变波特率 */
    TxHeader.FDFormat = FDCAN_FD_CAN;                /* FDCAN 格式 */
    TxHeader.TxEventFifoControl = FDCAN_NO_TX_EVENTS;/* 用于发送事件 FIFO 控制，不存储 */
    TxHeader.MessageMarker = 0;     /* 用于复制到 TX EVENT FIFO 的消息 Maker 来识别消息状态，范围 0 到 0xFF */

   /* 添加数据到 TX FIFO */
   HAL_FDCAN_AddMessageToTxFifoQ(&hfdcan1, &TxHeader, _DataBuf);
}
```

91.4.8函数 HAL_FDCAN_GetRxMessage

函数原型：

```
HAL_StatusTypeDef HAL_FDCAN_GetRxMessage(FDCAN_HandleTypeDef *hfdcan, uint32_t RxLocation,
FDCAN_RxHeaderTypeDef *pRxHeader, uint8_t *pRxData)
{
  uint32_t *RxAddress;
  uint8_t  *pData;
  uint32_t ByteCounter;
  uint32_t GetIndex = 0;
  HAL_FDCAN_StateTypeDef state = hfdcan->State;

  if (state == HAL_FDCAN_STATE_BUSY)
  {
    if (RxLocation == FDCAN_RX_FIFO0) /* Rx element 分配到 Rx FIFO 0 */
    {
      /* 检查 Rx FIFO 0 分配了 RAM 空间  */
      if ((hfdcan->Instance->RXF0C & FDCAN_RXF0C_F0S) == 0U)
      {
        /* 更新错误码*/
        hfdcan->ErrorCode |= HAL_FDCAN_ERROR_PARAM;
```

```
        return HAL_ERROR;
      }

      /* 检查 Rx FIFO 0 非空 */
      if ((hfdcan->Instance->RXF0S & FDCAN_RXF0S_F0FL) == 0U)
      {
        /* 更新错误码*/
        hfdcan->ErrorCode |= HAL_FDCAN_ERROR_FIFO_EMPTY;

        return HAL_ERROR;
      }
      else
      {
        /* 检查 Rx FIFO 0 满且开了覆盖模式 */
        if(((hfdcan->Instance->RXF0S & FDCAN_RXF0S_F0F) >> FDCAN_RXF0S_F0F_Pos) == 1U)
        {
          if(((hfdcan->Instance->RXF0C & FDCAN_RXF0C_F0OM) >> FDCAN_RXF0C_F0OM_Pos) ==
FDCAN_RX_FIFO_OVERWRITE)
          {
            /* 开启了覆盖模式，丢弃第 1 个数据 */
            GetIndex = 1U;
          }
        }

        /* 计算 Rx FIFO 0 element 索引 */
        GetIndex += ((hfdcan->Instance->RXF0S & FDCAN_RXF0S_F0GI) >> FDCAN_RXF0S_F0GI_Pos);

        /* 计算 Rx FIFO 0 element 地址  */
        RxAddress = (uint32_t *)(hfdcan->msgRam.RxFIFO0SA + (GetIndex * hfdcan->Init.RxFifo0ElmtSize * 4U));
      }
    }
    else if (RxLocation == FDCAN_RX_FIFO1) /* Rx element is assigned to the Rx FIFO 1 */
    {
       /* 检查 Rx FIFO 1 分配了 RAM 空间  */
      if ((hfdcan->Instance->RXF1C & FDCAN_RXF1C_F1S) == 0U)
      {
        /* 更新错误码 */
        hfdcan->ErrorCode |= HAL_FDCAN_ERROR_PARAM;

        return HAL_ERROR;
      }

      /* 检查 Rx FIFO 0 非空 */
      if ((hfdcan->Instance->RXF1S & FDCAN_RXF1S_F1FL) == 0U)
      {
       /* 更新错误码 */
        hfdcan->ErrorCode |= HAL_FDCAN_ERROR_FIFO_EMPTY;

        return HAL_ERROR;
      }
      else
      {
         /* 检查 Rx FIFO 1 满且开了覆盖模式 */
        if(((hfdcan->Instance->RXF1S & FDCAN_RXF1S_F1F) >> FDCAN_RXF1S_F1F_Pos) == 1U)
        {
          if(((hfdcan->Instance->RXF1C & FDCAN_RXF1C_F1OM) >> FDCAN_RXF1C_F1OM_Pos) ==
FDCAN_RX_FIFO_OVERWRITE)
```

```
        {
          /* 开启了覆盖模式，丢弃第 1 个数据 */
          GetIndex = 1U;
        }
      }

      /* 计算  Rx FIFO 1 element 索引 */
      GetIndex += ((hfdcan->Instance->RXF1S & FDCAN_RXF1S_F1GI) >> FDCAN_RXF1S_F1GI_Pos);

      /* 计算 Rx FIFO 1 element 地址 */
      RxAddress = (uint32_t *)(hfdcan->msgRam.RxFIFO1SA + (GetIndex * hfdcan->Init.RxFifo1ElmtSize * 4U));
    }
  }
  else /* Rx element 分配了专用 Rx buffer */
  {
    /* 检查选择的 buffer 分配了 RAM 空间 */
    if (RxLocation >= hfdcan->Init.RxBuffersNbr)
    {
      /* 更新错误码 */
      hfdcan->ErrorCode |= HAL_FDCAN_ERROR_PARAM;

      return HAL_ERROR;
    }
    else
    {
      /* 计算 Rx buffer 地址 */
      RxAddress = (uint32_t *)(hfdcan->msgRam.RxBufferSA + (RxLocation * hfdcan->Init.RxBufferSize * 4U));
    }
  }

  /* 接收 IdType */
  pRxHeader->IdType = *RxAddress & FDCAN_ELEMENT_MASK_XTD;

  /* 接收 Identifier */
  if (pRxHeader->IdType == FDCAN_STANDARD_ID) /* Standard ID element */
  {
    pRxHeader->Identifier = ((*RxAddress & FDCAN_ELEMENT_MASK_STDID) >> 18);
  }
  else /* Extended ID element */
  {
    pRxHeader->Identifier = (*RxAddress & FDCAN_ELEMENT_MASK_EXTID);
  }

  /* 接收 RxFrameType */
  pRxHeader->RxFrameType = (*RxAddress & FDCAN_ELEMENT_MASK_RTR);

  /* 接收 ErrorStateIndicator */
  pRxHeader->ErrorStateIndicator = (*RxAddress & FDCAN_ELEMENT_MASK_ESI);

  /* Increment RxAddress pointer to second word of Rx FIFO element */
  RxAddress++;

  /* 接收 RxTimestamp */
  pRxHeader->RxTimestamp = (*RxAddress & FDCAN_ELEMENT_MASK_TS);

  /* 接收 DataLength */
  pRxHeader->DataLength = (*RxAddress & FDCAN_ELEMENT_MASK_DLC);
```

```
    /* 接收 BitRateSwitch */
    pRxHeader->BitRateSwitch = (*RxAddress & FDCAN_ELEMENT_MASK_BRS);

    /* 接收 FDFormat */
    pRxHeader->FDFormat = (*RxAddress & FDCAN_ELEMENT_MASK_FDF);

    /* 接收 FilterIndex */
    pRxHeader->FilterIndex = ((*RxAddress & FDCAN_ELEMENT_MASK_FIDX) >> 24);

    /* 接收 NonMatchingFrame */
    pRxHeader->IsFilterMatchingFrame = ((*RxAddress & FDCAN_ELEMENT_MASK_ANMF) >> 31);

    /* 增加地址，指向 Rx FIFO element 的 payload */
    RxAddress++;

    /* 接收 Rx payload */
    pData = (uint8_t *)RxAddress;
    for (ByteCounter = 0; ByteCounter < DLCtoBytes[pRxHeader->DataLength >> 16]; ByteCounter++)
    {
      pRxData[ByteCounter] = pData[ByteCounter];
    }

    if (RxLocation == FDCAN_RX_FIFO0) /* Rx element is assigned to the Rx FIFO 0 */
    {
      /* 确认 Rx FIFO 0 先进的数据已经读出 */
      hfdcan->Instance->RXF0A = GetIndex;
    }
    else if (RxLocation == FDCAN_RX_FIFO1) /* Rx element is assigned to the Rx FIFO 1 */
    {
     /* 确认 Rx FIFO 1 先进的数据已经读出 */
      hfdcan->Instance->RXF1A = GetIndex;
    }
    else /* Rx element is assigned to a dedicated Rx buffer */
    {
      /* 清除当前 Rx buffer 的新数据标志 */
      if (RxLocation < FDCAN_RX_BUFFER32)
      {
        hfdcan->Instance->NDAT1 = ((uint32_t)1 << RxLocation);
      }
      else /* FDCAN_RX_BUFFER32 <= RxLocation <= FDCAN_RX_BUFFER63 */
      {
        hfdcan->Instance->NDAT2 = ((uint32_t)1 << (RxLocation & 0x1FU));
      }
    }

    /* 返回状态 */
    return HAL_OK;
  }
  else
  {
    /* 更新错误码 */
    hfdcan->ErrorCode |= HAL_FDCAN_ERROR_NOT_STARTED;

    return HAL_ERROR;
  }
}
```

函数描述：

此函数主要用于获取接收到的数据。

函数参数：

- 第 1 个参数是 FDCAN_HandleTypeDef 类型结构体指针变量。
- 第 2 个参数是读取位置，支持如下几种参数：

```
#define FDCAN_RX_FIFO0    ((uint32_t)0x00000040U) /*!< Get received message from Rx FIFO 0    */
#define FDCAN_RX_FIFO1    ((uint32_t)0x00000041U) /*!< Get received message from Rx FIFO 1    */
#define FDCAN_RX_BUFFER0  ((uint32_t)0x00000000U) /*!< Get received message from Rx Buffer 0  */
#define FDCAN_RX_BUFFER1  ((uint32_t)0x00000001U) /*!< Get received message from Rx Buffer 1  */
#define FDCAN_RX_BUFFER2  ((uint32_t)0x00000002U) /*!< Get received message from Rx Buffer 2  */
#define FDCAN_RX_BUFFER3  ((uint32_t)0x00000003U) /*!< Get received message from Rx Buffer 3  */
#define FDCAN_RX_BUFFER4  ((uint32_t)0x00000004U) /*!< Get received message from Rx Buffer 4  */
#define FDCAN_RX_BUFFER5  ((uint32_t)0x00000005U) /*!< Get received message from Rx Buffer 5  */
#define FDCAN_RX_BUFFER6  ((uint32_t)0x00000006U) /*!< Get received message from Rx Buffer 6  */
#define FDCAN_RX_BUFFER7  ((uint32_t)0x00000007U) /*!< Get received message from Rx Buffer 7  */
#define FDCAN_RX_BUFFER8  ((uint32_t)0x00000008U) /*!< Get received message from Rx Buffer 8  */
#define FDCAN_RX_BUFFER9  ((uint32_t)0x00000009U) /*!< Get received message from Rx Buffer 9  */
#define FDCAN_RX_BUFFER10 ((uint32_t)0x0000000AU) /*!< Get received message from Rx Buffer 10 */
#define FDCAN_RX_BUFFER11 ((uint32_t)0x0000000BU) /*!< Get received message from Rx Buffer 11 */
#define FDCAN_RX_BUFFER12 ((uint32_t)0x0000000CU) /*!< Get received message from Rx Buffer 12 */
#define FDCAN_RX_BUFFER13 ((uint32_t)0x0000000DU) /*!< Get received message from Rx Buffer 13 */
#define FDCAN_RX_BUFFER14 ((uint32_t)0x0000000EU) /*!< Get received message from Rx Buffer 14 */
#define FDCAN_RX_BUFFER15 ((uint32_t)0x0000000FU) /*!< Get received message from Rx Buffer 15 */
#define FDCAN_RX_BUFFER16 ((uint32_t)0x00000010U) /*!< Get received message from Rx Buffer 16 */
#define FDCAN_RX_BUFFER17 ((uint32_t)0x00000011U) /*!< Get received message from Rx Buffer 17 */
#define FDCAN_RX_BUFFER18 ((uint32_t)0x00000012U) /*!< Get received message from Rx Buffer 18 */
#define FDCAN_RX_BUFFER19 ((uint32_t)0x00000013U) /*!< Get received message from Rx Buffer 19 */
#define FDCAN_RX_BUFFER20 ((uint32_t)0x00000014U) /*!< Get received message from Rx Buffer 20 */
#define FDCAN_RX_BUFFER21 ((uint32_t)0x00000015U) /*!< Get received message from Rx Buffer 21 */
#define FDCAN_RX_BUFFER22 ((uint32_t)0x00000016U) /*!< Get received message from Rx Buffer 22 */
#define FDCAN_RX_BUFFER23 ((uint32_t)0x00000017U) /*!< Get received message from Rx Buffer 23 */
#define FDCAN_RX_BUFFER24 ((uint32_t)0x00000018U) /*!< Get received message from Rx Buffer 24 */
#define FDCAN_RX_BUFFER25 ((uint32_t)0x00000019U) /*!< Get received message from Rx Buffer 25 */
#define FDCAN_RX_BUFFER26 ((uint32_t)0x0000001AU) /*!< Get received message from Rx Buffer 26 */
#define FDCAN_RX_BUFFER27 ((uint32_t)0x0000001BU) /*!< Get received message from Rx Buffer 27 */
#define FDCAN_RX_BUFFER28 ((uint32_t)0x0000001CU) /*!< Get received message from Rx Buffer 28 */
#define FDCAN_RX_BUFFER29 ((uint32_t)0x0000001DU) /*!< Get received message from Rx Buffer 29 */
#define FDCAN_RX_BUFFER30 ((uint32_t)0x0000001EU) /*!< Get received message from Rx Buffer 30 */
#define FDCAN_RX_BUFFER31 ((uint32_t)0x0000001FU) /*!< Get received message from Rx Buffer 31 */
#define FDCAN_RX_BUFFER32 ((uint32_t)0x00000020U) /*!< Get received message from Rx Buffer 32 */
#define FDCAN_RX_BUFFER33 ((uint32_t)0x00000021U) /*!< Get received message from Rx Buffer 33 */
#define FDCAN_RX_BUFFER34 ((uint32_t)0x00000022U) /*!< Get received message from Rx Buffer 34 */
#define FDCAN_RX_BUFFER35 ((uint32_t)0x00000023U) /*!< Get received message from Rx Buffer 35 */
#define FDCAN_RX_BUFFER36 ((uint32_t)0x00000024U) /*!< Get received message from Rx Buffer 36 */
#define FDCAN_RX_BUFFER37 ((uint32_t)0x00000025U) /*!< Get received message from Rx Buffer 37 */
#define FDCAN_RX_BUFFER38 ((uint32_t)0x00000026U) /*!< Get received message from Rx Buffer 38 */
#define FDCAN_RX_BUFFER39 ((uint32_t)0x00000027U) /*!< Get received message from Rx Buffer 39 */
#define FDCAN_RX_BUFFER40 ((uint32_t)0x00000028U) /*!< Get received message from Rx Buffer 40 */
#define FDCAN_RX_BUFFER41 ((uint32_t)0x00000029U) /*!< Get received message from Rx Buffer 41 */
#define FDCAN_RX_BUFFER42 ((uint32_t)0x0000002AU) /*!< Get received message from Rx Buffer 42 */
#define FDCAN_RX_BUFFER43 ((uint32_t)0x0000002BU) /*!< Get received message from Rx Buffer 43 */
#define FDCAN_RX_BUFFER44 ((uint32_t)0x0000002CU) /*!< Get received message from Rx Buffer 44 */
#define FDCAN_RX_BUFFER45 ((uint32_t)0x0000002DU) /*!< Get received message from Rx Buffer 45 */
#define FDCAN_RX_BUFFER46 ((uint32_t)0x0000002EU) /*!< Get received message from Rx Buffer 46 */
#define FDCAN_RX_BUFFER47 ((uint32_t)0x0000002FU) /*!< Get received message from Rx Buffer 47 */
#define FDCAN_RX_BUFFER48 ((uint32_t)0x00000030U) /*!< Get received message from Rx Buffer 48 */
```

```
#define FDCAN_RX_BUFFER49 ((uint32_t)0x00000031U) /*!< Get received message from Rx Buffer 49 */
#define FDCAN_RX_BUFFER50 ((uint32_t)0x00000032U) /*!< Get received message from Rx Buffer 50 */
#define FDCAN_RX_BUFFER51 ((uint32_t)0x00000033U) /*!< Get received message from Rx Buffer 51 */
#define FDCAN_RX_BUFFER52 ((uint32_t)0x00000034U) /*!< Get received message from Rx Buffer 52 */
#define FDCAN_RX_BUFFER53 ((uint32_t)0x00000035U) /*!< Get received message from Rx Buffer 53 */
#define FDCAN_RX_BUFFER54 ((uint32_t)0x00000036U) /*!< Get received message from Rx Buffer 54 */
#define FDCAN_RX_BUFFER55 ((uint32_t)0x00000037U) /*!< Get received message from Rx Buffer 55 */
#define FDCAN_RX_BUFFER56 ((uint32_t)0x00000038U) /*!< Get received message from Rx Buffer 56 */
#define FDCAN_RX_BUFFER57 ((uint32_t)0x00000039U) /*!< Get received message from Rx Buffer 57 */
#define FDCAN_RX_BUFFER58 ((uint32_t)0x0000003AU) /*!< Get received message from Rx Buffer 58 */
#define FDCAN_RX_BUFFER59 ((uint32_t)0x0000003BU) /*!< Get received message from Rx Buffer 59 */
#define FDCAN_RX_BUFFER60 ((uint32_t)0x0000003CU) /*!< Get received message from Rx Buffer 60 */
#define FDCAN_RX_BUFFER61 ((uint32_t)0x0000003DU) /*!< Get received message from Rx Buffer 61 */
#define FDCAN_RX_BUFFER62 ((uint32_t)0x0000003EU) /*!< Get received message from Rx Buffer 62 */
#define FDCAN_RX_BUFFER63 ((uint32_t)0x0000003FU) /*!< Get received message from Rx Buffer 63 */
```

- 第 3 个参数是 FDCAN_RxHeaderTypeDef 类型结构体变量，主要用于消息接收。
- 第 4 个参数是数据接收缓冲地址。
- 返回值，返回 HAL_TIMEOUT 表示超时，HAL_ERROR 表示参数错误，HAL_OK 表示发送成功，HAL_BUSY 表示忙，正在使用中。

使用举例：

```
/*
*********************************************************************************************************
*    函 数 名: HAL_FDCAN_RxFifo0Callback
*    功能说明: CAN 中断服务程序-回调函数
*    形    参: hfdcan
*    返 回 值: 无
*********************************************************************************************************
*/
void HAL_FDCAN_RxFifo0Callback(FDCAN_HandleTypeDef *hfdcan, uint32_t RxFifo0ITs)
{
    if (hfdcan == &hfdcan1)
    {
        if ((RxFifo0ITs & FDCAN_IT_RX_FIFO0_WATERMARK) != RESET)
        {
            /* 从 RX FIFO0 读取数据 */
            HAL_FDCAN_GetRxMessage(hfdcan, FDCAN_RX_FIFO0, &g_Can1RxHeader, g_Can1RxData);

            /* 激活 Rx FIFO0 watermark notification */
            HAL_FDCAN_ActivateNotification(hfdcan, FDCAN_IT_RX_FIFO0_WATERMARK, 0);

            if (g_Can1RxHeader.Identifier == 0x111 && g_Can1RxHeader.IdType == FDCAN_STANDARD_ID)
            {
                bsp_PutMsg(MSG_CAN1_RX, 0); /* 发消息收到数据包，结果在 g_Can1RxHeader，  g_Can1RxData */
            }
        }
    }
}
```

91.5 总结

本章节就为大家讲解这么多，更多 FDCAN 知识可以看本教程的第 90 章。

第92章 STM32H7 的 FDCAN 总线应用之双 FDCAN 实现（支持经典 CAN）

本章节为大家讲解 STM32H7 的带两个 FDCAN 控制器使用方法。CAN FD 中的 FD 含义就是 flexible data，灵活数据通信，且波特率可以和仲裁阶段波特率不同。

92.1 初学者重要提示
92.2 FDCAN 硬件接口设计
92.3 FDCAN 基础知识
92.4 FDCAN 驱动代码实现
92.5 双 FDCAN 测试的接线盒跳线帽说明
92.6 开发板和 H7-TOOL 的 FDCAN 助手测试
92.7 实验例程设计框架
92.8 实验例程说明（MDK）
92.9 实验例程说明（IAR）
92.10 总结

92.1 初学者重要提示

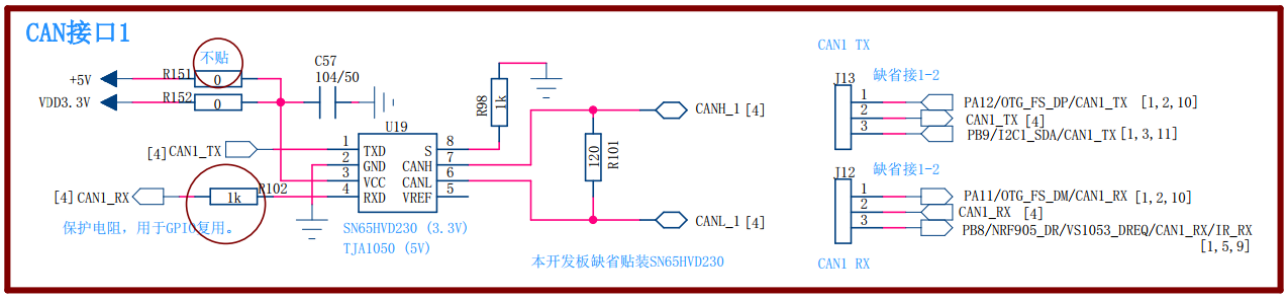
- FDCAN 相关知识点可以看第 90 和 91 章。
- CAN 菊花链组网时，在两端接分别接 120 Ω的终端电阻。
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=104793 。
- FDCAN 控制器外接的 PHY 芯片输出的是差分信号，组网接线时，注意是 CANL 接 CANL，CANH 接 CANH。
- 经典 CAN 每帧最大 8 字节，FDCAN 每帧最大 64 字节。
- CAN 不接外置 PHY 芯片，通信测试方法：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=104684 。
- 关于 CAN 总线是否需要供地的问题：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=104704
- CAN 组网只有一个节点的情况下，接示波器看不到 FDCAN 数据帧波形。
- 特别推荐瑞萨的 CAN 入门中英文手册，做的非常好：

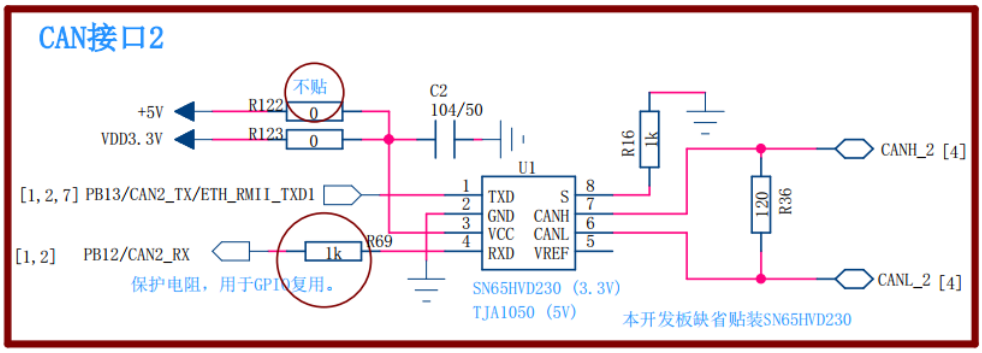
http://www.armbbs.cn/forum.php?mod=viewthread&tid=14546

- 带隔离功能的 FDCAN 芯片搜集：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=90420
- 除了本章提供的基于 ST HAL 库实现的双 FDCAN 通信，再提供个基于 CMSIS-Driver 的：
  基于 STM32H7 的 CMSIS-Driver 驱动实现双 CAN FD 和双经典 CAN 两种方式案例发布
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=105369

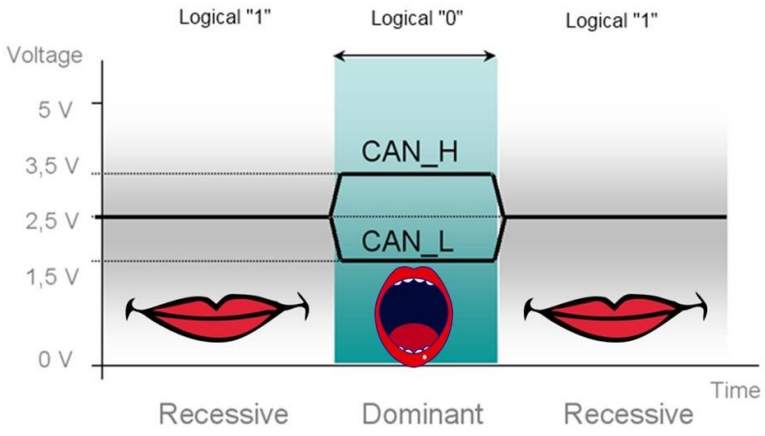
92.2 FDCAN 硬件接口设计

STM32H7 带了两个 FDCAN 控制器，然后外接物理层 PHY 芯片就可以使用了。FDCAN1 和 FDCAN2 外接芯片原理图如下：





使用的 PHY 芯片 SN65HVD230 即支持经典 CAN，也支持 FDCAN。PHY 芯片输出的是差分信号，逻辑 0 或者逻辑 1 的电平效果如下：http://www.armbbs.cn/forum.php?mod=viewthread&tid=30855



92.3 FDCAN 基础知识

FDCAN 的基础知识在第 90 已经做了详细说，这里补充些本章要用到的知识点。

92.3.1 经典 CAN 和 FDCAN 的区别

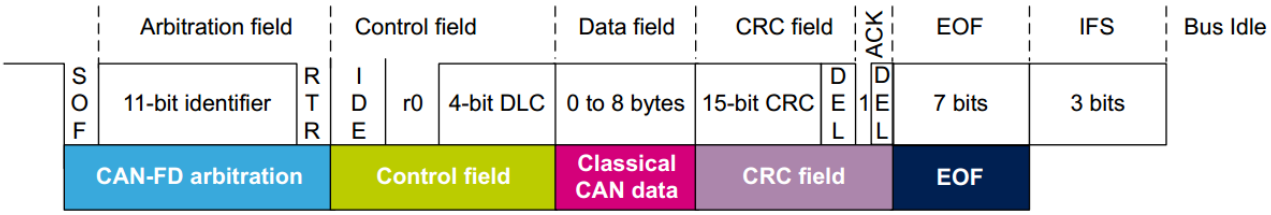
CAN-FD 的开发可以满足需要更高带宽的通信网络需求。每帧最多具有 64 个字节的 CAN-FD 以及将比特率提高到最大的可能性，使数据阶段要快 8 倍，在第二个仲裁阶段要恢复到正常的比特率。通过以下方式确保数据传输的完整性：

（1）17 级多项式对最大 16 字节的有效载荷进行 CRC。

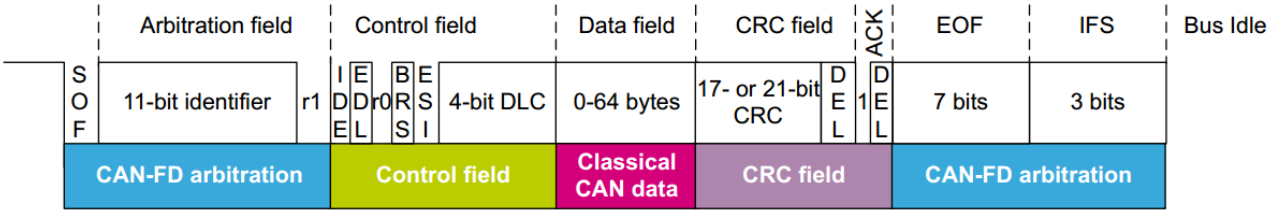
（2）21 级多项式对 16 到 64 字节之间的有效载荷进行校验。

标准帧和 CAN FD 的区别：

CAN 2.0: Classical base frame format



CAN-FD: CAN flexible data rate base frame format



DEL = Deliminator RTR = Remote transmission request

标识符后，CAN 2.0 和 CAN-FD 具有不同的作用：

（1）CAN 2.0 发送 RTR 位以精确确定帧类型：数据帧（RTR 为主要）或远程帧（RTR）是隐性的）。

（2）由于 CAN-FD 仅支持数据帧，因此始终发送占优势的 RRS（保留）。

IDE 位保持在相同位置，并以相同的动作来区分基本格式 (11 位标识符)。请注意，在扩展格式的情况下，IDE 位以显性或隐性方式传输（29 位标识符）。

与 CAN 2.0 相比，在 CAN-FD 帧中，在控制字段中添加了三个新位：

（1）扩展数据长度（EDL）位：隐性表示帧为 CAN-FD，否则该位为显性（称为 R0）在 CAN 2.0 帧中。

（2）比特率切换（BRS）：指示是否启用两个比特率（例如，当数据阶段位以不同的比特率传输到仲裁阶段）。

（3）错误状态指示器（ESI）：指示节点处于错误活动模式还是错误被动模式。

控制字段的最后一部分是数据长度代码（DLC），它具有相同的位置和相同的长度（4 位），用于 CAN 2.0 和 CAN-FD。 DLC 功能在 CAN-FD 和 CAN 2.0 中相同，但 CAN-FD 有很小变化（下表中的详细信息）。CAN-FD 扩展帧允许单个消息中发送 64 个数据字节，而 CAN 2.0 有效负载数据最多可以发送 8 个字节。

DLC (Dec)	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
CAN 2.0	0	1	2	3	4	5	6	7	8	8	8	8	8	8	8	8
CAN-FD	0	1	2	3	4	5	6	7	8	12	16	20	24	32	48	64

通过增加有效载荷数据的数据字段来改善网络带宽，因为需要更少的包处理。 同时，通过为 CRC 添加更多位来增强消息完整性：

（1）如果有效载荷数据最多为 16 个字节，则 CRC 以 17 位编码。

（2）如果有效载荷数据大于 20（16）个字节，则 CRC 以 21 位编码。

另外，为了确保 CAN-FD 帧的鲁棒性，填充位机制支持 CRC 字段。下表总结了 CAN-FD 和 CAN 2.0 之间的主要区别。 提供的主要功能与 CAN 2.0 相比，CAN FD 的改进之处在于数据有效负载的增加和速度的提高由 CAN-FD 中可用的 BRS，EDL 和 ESI 位来确保。

Features	CAN 2.0	CAN-FD
Compatibility	Does not support CAN-FD	Supports CAN 2.0 A/B
Maximum bit rate (Mbit/s)	Frame bitrate: up to 1	Arbitration bitrate: up to 1s Data bitrate: up to 8
DLC field (4 bits) code	Coded in 0 to 8	Coded in 0 to 64
Maximum data bytes in one message	8 bytes of data	64 bytes of data
BRS support	No	Yes
EDL support	No	Yes
ASI support	No	Yes
CRC bits check codes	Bits not included in CRC calculation	Bits included in CRC calculation
Remote frame support	Yes	No

下表可帮助用户简化将 STM32 设备中的 CAN 2.0 协议升级到 CAN-FD 协议的过程。该表还指定了 FDCAN 的改进。与传统的 BxCAN（基本扩展 CAN）相比，FDCAN 具有许多优势，包括更快的数据传输速度。速率和数据字节数的扩展，减少了帧开销。 总线负载也可以减少。 传输和接收中消息数量的增加要求 RAM 存储器的改进：

Features		Supported by BxCAN	Supported by FDCAN
Compatibility	Compatibility with BxCAN	Yes	Yes
	Compatibility with CAN-FD	No	No
	Remote frame	No	Supported to become compatible with BxCAN
Frame	Arbitration bitrate /data rate (in Mbit/s)	Up to 1 (only bitrate per frame)	Up to 1 /Up to 8
	Data length per frame (in bytes)	0 to 8	0 to 64
RAM	RAM	512 bytes	10 Kbytes
	Accessible RAM	No	Yes
Transmission	Dedicated Tx buffer/Tx queue/Tx FIFO	No/Yes/Yes Maximum three elements can be used as queue **or** FIFO.	Yes/Yes/Yes Maximum 32 elements: user chooses the transmit mechanism.
	Transmit pause	No	Yes
Reception	Dedicated Rx buffer/Rx FIFO	No/2 Up to three elements in each Rx FIFO Up to six elements maximum	64/2 Up to 64 elements in each Rx FIFO Up to 192 elements maximum
	Overwrite mode option	Overwrite the last element received in FIFO	Overwrite the oldest element in FIFO
	Improvements acceptance filters	Classical CAL (Can 2.0) acceptance filters	Some features are added as: • Discards matching filter. • Accepts non matching filters.
Other	Restricted test mode	No	Yes
	Transceiver delay compensation	No	Yes
	Clock calibration unit	No	Yes

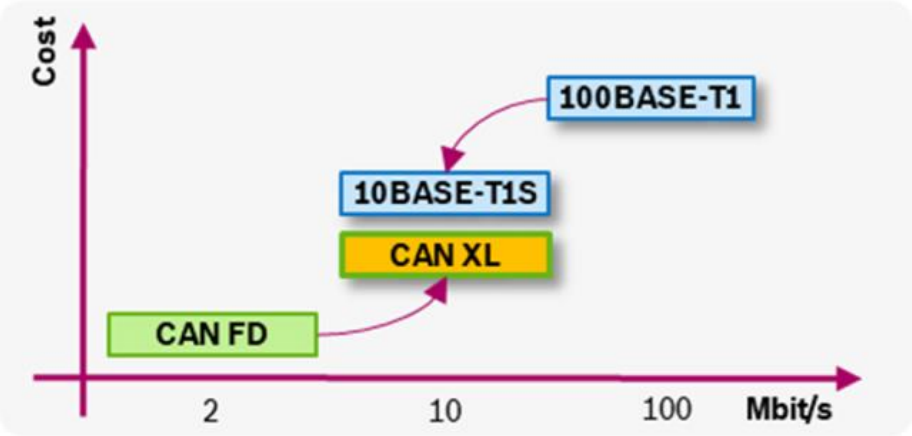
鉴于 BxCAN 的兼容性，BxCAN 开发人员可以轻松地迁移到 FDCAN，因为 FDCAN 可以无需对整个系统设计进行修订即可实施。 FDCAN 包含所有 BxCAN 功能，并满足新应用程序的要求。

92.3.2 STM32H7 的 FDCAN1 和 FDCAN2 的区别

仅 FDCAN1 支持 TTCAN 时间触发通信，而 FDCAN2 不支持。

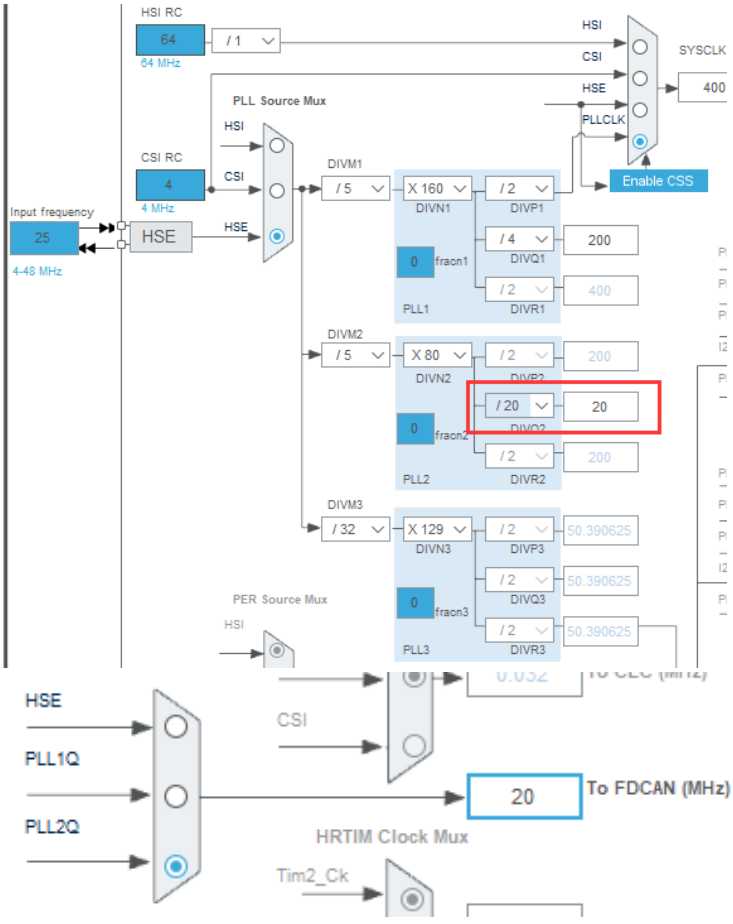
92.3.3 FDCAN 支持的最高速度

经典 CAN 是 1Mbps，CAN FD 最高 2Mbps，CAN FD-SiC 是 5-8Mbps，CAN XL 是 10Mbps。



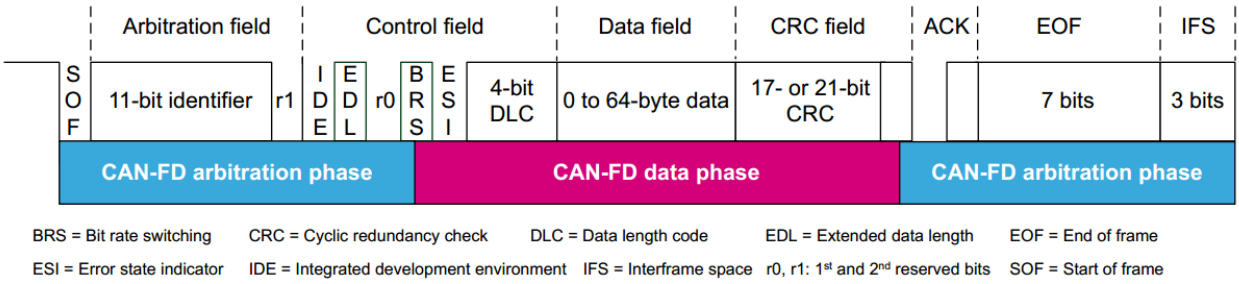
92.3.4 FDCAN 的主时钟选择

FDCAN1 和 FDCAN2 支持三种时钟源 HSE，PLL1Q 和 PLL2Q，我们这里选择的 PLL2Q 输出 20MHz。



92.3.5 FDCAN 仲裁阶段和通信阶段波特率配置（重要）

CAN FD 中的 FD 含义就是 flexible data，灵活数据通信，且波特率可以和仲裁阶段波特率不同。看下面的图，意思就是红色部分可以是一个波特率，蓝色部分也可以是一个波特率。



波特率计算公式，看如下的位时间特性图：

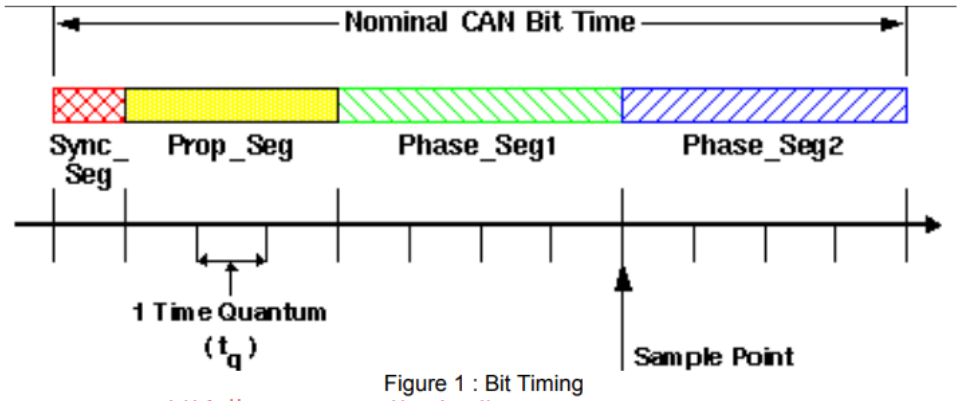


Figure 1 : Bit Timing

下面是最小编程范围，STM32H7的要大一些

Parameter	Range	Remark
BRP	[1 .. 32]	defines the length of the time quantum t_q
Sync_Seg	1 t_q	fixed length, synchronization of bus input to system clock
Prop_Seg	[1 .. 8] t_q	compensates for the physical delay times
Phase_Seg1	[1 .. 8] t_q	may be lengthened temporarily by synchronization
Phase_Seg2	[1 .. 8] t_q	may be shortened temporarily by synchronization
SJW	[1 .. 4] t_q	may not be longer than either Phase Buffer Segment
This table describes the minimum programmable ranges required by the CAN protocol		

Table 1 : Parameters of the CAN Bit Time

1bit 的 CAN FD 数据需要时间由 Sync_Seg + Pro_Seg + Phase_Seg1 + Phase_Seg2 组成。

◆ 仲裁阶段波特率对应的 HAL 库配置如下：

```
/* CAN 时钟分配设置，一般设置为 1 即可，全部由 PLL 配置好，tq = NominalPrescaler x (1/ fdcan_ker_ck) */
hfdcan2.Init.NominalPrescaler = 0x1;

/* 特别注意这里的 Seg1，这里是两个参数之和，对应位时间特性图的 Pro_Seg + Phase_Seg1 */
hfdcan2.Init.NominalTimeSeg1 = 0x1F;

/* 对应位时间特性图的 Phase_Seg2 */
hfdcan2.Init.NominalTimeSeg2 = 0x8;
```

```
/* 用于动态调节  Phase_Seg1 和 Phase_Seg1，所以不可以比 Phase_Seg1 和 Phase_Seg1 大 */
hfdcan2.Init.NominalSyncJumpWidth = 0x8;
```

其中：

```
Sync_Seg 是固定值 = 1
Pro_Seg + Phase_Seg1 = DataTimeSeg1
hase_Seg2 = DataTimeSeg2
```

FDCAN 时钟是 20MHz 时，仲裁阶段的波特率就是

FDCAN Freq / (Sync_Seg + Pro_Seg + Phase_Seg1 + Phase_Seg2)

= 20MHz / (1+0x1F + 8)

= 0.5Mbps

◆ 数据阶段波特率对应的 HAL 库配置如下：

```
/* CAN 时钟分配设置，一般设置为 1 即可，全部由 PLL 配置好，tq = NominalPrescaler x (1/ fdcan_ker_ck) */
hfdcan2.Init.DataPrescaler = 0x1;

/* 特别注意这里的 Seg1，这里是两个参数之和，对应位时间特性图的 Pro_Seg + Phase_Seg1 */
hfdcan2.Init.DataTimeSeg1 = 0x5;

/* 对应位时间特性图的 Phase_Seg2 */
hfdcan2.Init.DataTimeSeg2 = 0x4;

/* 用于动态调节  Phase_Seg1 和 Phase_Seg1，所以不可以比 Phase_Seg1 和 Phase_Seg1 大 */
hfdcan2.Init.DataSyncJumpWidth = 0x4;
```

其中：

```
Sync_Seg 是固定值 = 1
Pro_Seg + Phase_Seg1 = DataTimeSeg1
hase_Seg2 = DataTimeSeg2
```

FDCAN 时钟是 20MHz 时，数据阶段的波特率就是

CAN FD Freq / (Sync_Seg + Pro_Seg + Phase_Seg1 + Phase_Seg2)

= 20MHz / (1+5+ 4)

= 2Mbps

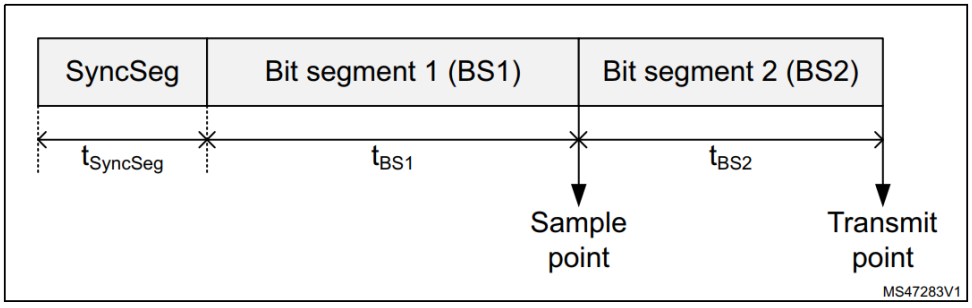
STM32H7 在 400MHz 情况下，PLL 配置输出 20MHz 的 CAN FD 时钟（大家可以使用 STM32CubeMX 来配置的）：

```
/* 选择 PLL2Q 作为 FDCANx 时钟 */
PeriphClkInitStruct.PeriphClockSelection = RCC_PERIPHCLK_FDCAN;
PeriphClkInitStruct.PLL2.PLL2M = 5;
PeriphClkInitStruct.PLL2.PLL2N = 80;
PeriphClkInitStruct.PLL2.PLL2P = 2;
PeriphClkInitStruct.PLL2.PLL2Q = 20;
PeriphClkInitStruct.PLL2.PLL2R = 2;
PeriphClkInitStruct.PLL2.PLL2RGE = RCC_PLL2VCIRANGE_2;
PeriphClkInitStruct.PLL2.PLL2VCOSEL = RCC_PLL2VCOWIDE;
PeriphClkInitStruct.PLL2.PLL2FRACN = 0;
PeriphClkInitStruct.FdcanClockSelection = RCC_FDCANCLKSOURCE_PLL2;
if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInitStruct) != HAL_OK)
{
```

```
    Error_Handler(__FILE__, __LINE__);
}
```

92.3.6 FDCAN 的采样点位置推荐设置为 85% - 90%

FDCAN 每个 bit 的时间组成如下：



为了实现更好的稳定性，在满足指定波特率的情况下，让采样点的位置满足 85%到 90%是推荐的方式。主要用到 HAL 库的如下两个成员：

```
hfdcan2.Init.NominalTimeSeg1
hfdcan2.Init.NominalTimeSeg2
```

SyncSeg 是固定 1，也就是：

(SyncSeg + NominalTimeSeg1)/ (SyncSeg + NominalTimeSeg1 + NominalTimeSeg2)满足这个条件。

92.3.7 FDCAN 的 2560 字 RAM 空间分配问题

关于 FDCAN 的 2560 字 RAM 空间在本教程第 90 章的第 5 小节有详细说明。本章配套程序将前 1280 字分配给 FDCAN1，后 1280 字分配给 FDCAN2。

92.3.8 FDCAN 过滤器常用的范围过滤器和经典的位屏蔽过滤器

关于 FDCAN 的过滤器类型在本教程第 90 章的第 6 小节有详细说明，我们这里重点了解范围过滤器和经典位屏蔽过滤器。

- 范围过滤器

范围过滤器比较简单，也比较好理解，对应的 HAL 库配置代码如下：

```
sFilterConfig2.IdType = FDCAN_STANDARD_ID;
sFilterConfig2.FilterIndex = 0;
sFilterConfig2.FilterType = FDCAN_FILTER_RANGE;
sFilterConfig2.FilterConfig = FDCAN_FILTER_TO_RXFIFO0;
sFilterConfig2.FilterID1 = 0x111;
sFilterConfig2.FilterID2 = 0x555;
HAL_FDCAN_ConfigFilter(&hfdcan2, &sFilterConfig2);
```

FilterType 类型配置为 FDCAN_FILTER_RANGE 表示范围过滤器。

FilterID1 = 0x111 和 FilterID2 = 0x555 表示仅接收 ≥0x111 且 ≤ 0x555 的 ID。

◆ 经典的位屏蔽过滤

对应的 HAL 库配置代码如下：

```
sFilterConfig2.IdType = FDCAN_STANDARD_ID;
sFilterConfig2.FilterIndex = 0;
sFilterConfig2.FilterType = FDCAN_FILTER_MASK;
sFilterConfig2.FilterConfig = FDCAN_FILTER_TO_RXFIFO0;
sFilterConfig2.FilterID1 = 0x222;
sFilterConfig2.FilterID2 = 0x7FF;
HAL_FDCAN_ConfigFilter(&hfdcan2, &sFilterConfig2);
```

FilterType 类型配置为 FDCAN_FILTER_MASK 表示经典的位屏蔽过滤。

FilterID1 = filter 表示 ID。

FilterID2 = mask 表示 ID 屏蔽位，mask 每个 bit 含义：

0: 表示 FilterID1 相应 bit 不关心，该位不用于比较。

1: 表示 FilterID1 相应 bit 必须匹配，即接收到的 ID 位必须与 FilterID1 的相应位一致。

我们这里 FilterID1 = 0x222，FilterID2 = 0x7FF 表示仅接收 ID 为 0x222 的 FDCAN 帧。

92.4 FDCAN 驱动代码实现

这里以 FDCAN1 为例进行说明，FDCAN2 的驱动程序在本章配套例子里面也提供了。

92.4.1 FDCAN 配置

代码里面对每个成员都进行了详细注释说明：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitCan1
*    功能说明: 初始 CAN1
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitCan1(void)
{
    /*                    位时间特性配置
        Bit time parameter          | Nominal      |  Data
        ----------------------------|--------------|---------------
        fdcan_ker_ck                | 20 MHz       | 20 MHz
        Time_quantum (tq)           | 50 ns        | 50 ns
        Synchronization_segment     | 1 tq         | 1 tq
        Propagation_segment         | 23 tq        | 1 tq
        Phase_segment_1             | 8 tq         | 4 tq
        Phase_segment_2             | 8 tq         | 4 tq
        Synchronization_Jump_width  | 8 tq         | 4 tq
        Bit_length                  | 40 tq = 2us  | 10 tq = 0.5us
        Bit_rate                    | 0.5 MBit/s   | 2 MBit/s
```

```
*/
hfdcan1.Instance = FDCAN1;                      /* 配置 FDCAN1 */
hfdcan1.Init.FrameFormat = FDCAN_FRAME_FD_BRS; /* 配置使用 FDCAN 可变波特率 */
hfdcan1.Init.Mode = FDCAN_MODE_NORMAL;          /* 配置使用正常模式 */
hfdcan1.Init.AutoRetransmission = ENABLE;       /*使能自动重发 */
hfdcan1.Init.TransmitPause = DISABLE;           /* 配置禁止传输暂停特性 */
hfdcan1.Init.ProtocolException = ENABLE;        /* 协议异常处理使能 */

/*
     配置仲裁阶段波特率
     CAN 时钟 20MHz 时，仲裁阶段的波特率就是
     CAN FD Freq / (Sync_Seg + Pro_Seg + Phase_Seg1 + Phase_Seg2) = 20MHz / （1+0x1F + 8） = 0.5Mbps


     其中 Sync_Seg 是固定值 = 1 ， Pro_Seg + Phase_Seg1 = NominalTimeSeg1， Phase_Seg2 = NominalTimeSeg2
*/
/* CAN 时钟分配设置，一般设置为 1 即可，全部由 PLL 配置好，tq = NominalPrescaler x (1/ fdcan_ker_ck) */
hfdcan1.Init.NominalPrescaler = 0x01;

/* 用于动态调节  Phase_Seg1 和 Phase_Seg1，所以不可以比 Phase_Seg1 和 Phase_Seg1 大 */
hfdcan1.Init.NominalSyncJumpWidth = 0x08;

/* 特别注意这里的 Seg1，这里是两个参数之和，对应位时间特性图的 Pro_Seg + Phase_Seg1 */
hfdcan1.Init.NominalTimeSeg1 = 0x1F;

/* 对应位时间特性图的 Phase_Seg2 */
hfdcan1.Init.NominalTimeSeg2 = 0x08;


/*
     配置数据阶段波特率
     CAN 时钟 20MHz 时，数据阶段的波特率就是
     CAN FD Freq / (Sync_Seg + Pro_Seg + Phase_Seg1 + Phase_Seg2) = 20MHz / （1+5+ 4） = 2Mbps

     其中 Sync_Seg 是固定值 = 1 ， Pro_Seg + Phase_Seg1 = DataTimeSeg1， Phase_Seg2 = DataTimeSeg2
*/
/* CAN 时钟分配设置，一般设置为 1 即可，全部由 PLL 配置好，tq = NominalPrescaler x (1/ fdcan_ker_ck)，
   范围 1-32 */
hfdcan1.Init.DataPrescaler = 0x01;

/* 用于动态调节  Phase_Seg1 和 Phase_Seg1，所以不可以比 Phase_Seg1 和 Phase_Seg1 大，范围 1-16 */
hfdcan1.Init.DataSyncJumpWidth = 0x04;

/* 特别注意这里的 Seg1，这里是两个参数之和，对应位时间特性图的 Pro_Seg + Phase_Seg1，范围 */
hfdcan1.Init.DataTimeSeg1 = 0x05;

/* 对应位时间特性图的 Phase_Seg2 */
hfdcan1.Init.DataTimeSeg2 = 0x04;


hfdcan1.Init.MessageRAMOffset = 0;      /* CAN1 和 CAN2 共享 2560 个字，这里 CAN1 分配前 1280 字 */


hfdcan1.Init.StdFiltersNbr = 1;                  /* 设置标准 ID 过滤器个数，范围 0-128 */
hfdcan1.Init.ExtFiltersNbr = 0;                  /* 设置扩展 ID 过滤器个数，范围 0-64 */
hfdcan1.Init.RxFifo0ElmtsNbr = 2;                /* 设置 Rx FIFO0 的元素个数，范围 0-64 */
/* 设置 Rx FIFO0 中每个元素大小，支持 8,12,16,20,24,32,48 或者 64 字节 */
hfdcan1.Init.RxFifo0ElmtSize = FDCAN_DATA_BYTES_8;
```

```
    hfdcan1.Init.RxFifo1ElmtsNbr = 0;                       /* 设置 Rx FIFO1 的元素个数，范围 0-64 */

    /* 设置 Rx FIFO1 中每个元素大小，支持 8,12,16,20,24,32,48 或者 64 字节 */
    hfdcan1.Init.RxFifo1ElmtSize = FDCAN_DATA_BYTES_8;
    hfdcan1.Init.RxBuffersNbr = 0;                          /* 设置 Rx Buffer 个数，范围 0-64 */

    /* 设置 Rx Buffer 中每个元素大小，支持 8,12,16,20,24,32,48 或者 64 字节 */
    hfdcan1.Init.RxBufferSize = 0;


    hfdcan1.Init.TxEventsNbr = 0;           /* 设置 Tx Event FIFO 中元素个数，范围 0-32 */
    hfdcan1.Init.TxBuffersNbr = 0;          /* 设置 Tx Buffer 中元素个数，范围 0-32 */
    hfdcan1.Init.TxFifoQueueElmtsNbr = 2; /* 设置用于 Tx FIFO/Queue 的 Tx Buffers 个数。范围 0 到 32 */
    hfdcan1.Init.TxFifoQueueMode = FDCAN_TX_FIFO_OPERATION; /* 设置 FIFO 模式或者 QUEUE 队列模式 */

    /* 设置 Tx Element 中的数据域大小，支持 8,12,16,20,24,32,48 或者 64 字节 */
    hfdcan1.Init.TxElmtSize = FDCAN_DATA_BYTES_8;
    HAL_FDCAN_Init(&hfdcan1);


    /*
         配置过滤器，过滤器主要用于接收，这里采样屏蔽位模式。
         FilterID1 = filter
         FilterID2 = mask

         FilterID2 的 mask 每个 bit 含义
         0: 不关心，该位不用于比较；
         1: 必须匹配，接收到的 ID 必须与滤波器对应的 ID 位相一致。

         举例说明：
         FilterID1 = 0x111
         FilterID2 = 0x7FF
         表示仅接收 ID 为 0x111 的 FDCAN 帧。

    */
    sFilterConfig1.IdType = FDCAN_STANDARD_ID;              /* 设置标准 ID 或者扩展 ID */
    /* 用于过滤索引，如果是标准 ID，范围 0 到 127。如果是扩展 ID，范围 0 到 64 */
    sFilterConfig1.FilterIndex = 0;
    sFilterConfig1.FilterType = FDCAN_FILTER_MASK;          /* 过滤器采样屏蔽位模式 */
    sFilterConfig1.FilterConfig = FDCAN_FILTER_TO_RXFIFO0;  /* 如果过滤匹配，将数据保存到 Rx FIFO 0 */
    sFilterConfig1.FilterID1 = 0x111;                       /* 屏蔽位模式下，FilterID1 是消息 ID */
    sFilterConfig1.FilterID2 = 0x7FF;                       /* 屏蔽位模式下，FilterID2 是消息屏蔽位 */
    HAL_FDCAN_ConfigFilter(&hfdcan1, &sFilterConfig1);      /* 配置过滤器 */

    /* 设置 Rx FIFO0 的 wartermark 为 1 */
    HAL_FDCAN_ConfigFifoWatermark(&hfdcan1, FDCAN_CFG_RX_FIFO0, 1);

    /* 激活 RX FIFO0 的 watermark 通知中断，位开启 Tx Buffer 中断*/
    HAL_FDCAN_ActivateNotification(&hfdcan1, FDCAN_IT_RX_FIFO0_WATERMARK, 0);

    /* 启动 FDCAN */
    HAL_FDCAN_Start(&hfdcan1);
}
```

92.4.2 FDCAN 的 GPIO，NVIC 等配置

主要配置了 FDCAN 的 GPIO，GPIO 时钟，FDCAN 时钟和 NVIC：

```
/*
*********************************************************************************************************
*    函 数 名: HAL_FDCAN_MspInit
*    功能说明: 配置 CAN gpio
*    形    参: hfdcan
*    返 回 值: 无
*********************************************************************************************************
*/
void HAL_FDCAN_MspInit(FDCAN_HandleTypeDef* hfdcan)
{
    GPIO_InitTypeDef  GPIO_InitStruct;
    RCC_PeriphCLKInitTypeDef PeriphClkInitStruct = {0};

    if (hfdcan == &hfdcan1)
    {
        /*##-1- 使能外设这个 GPIO 时钟 ###################################*/
        FDCAN1_TX_GPIO_CLK_ENABLE();
        FDCAN1_RX_GPIO_CLK_ENABLE();

        /* 选择 PLL2Q 作为 FDCANx 时钟 */
       PeriphClkInitStruct.PeriphClockSelection = RCC_PERIPHCLK_FDCAN;
       PeriphClkInitStruct.PLL2.PLL2M = 5;
       PeriphClkInitStruct.PLL2.PLL2N = 80;
       PeriphClkInitStruct.PLL2.PLL2P = 2;
       PeriphClkInitStruct.PLL2.PLL2Q = 20;
       PeriphClkInitStruct.PLL2.PLL2R = 2;
       PeriphClkInitStruct.PLL2.PLL2RGE = RCC_PLL2VCIRANGE_2;
       PeriphClkInitStruct.PLL2.PLL2VCOSEL = RCC_PLL2VCOWIDE;
       PeriphClkInitStruct.PLL2.PLL2FRACN = 0;
       PeriphClkInitStruct.FdcanClockSelection = RCC_FDCANCLKSOURCE_PLL2;
       if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInitStruct) != HAL_OK)
       {
           Error_Handler(__FILE__, __LINE__);
       }

        __HAL_RCC_FDCAN_CLK_ENABLE();

        GPIO_InitStruct.Pin       = FDCAN1_TX_PIN;
        GPIO_InitStruct.Mode      = GPIO_MODE_AF_PP;
        GPIO_InitStruct.Pull      = GPIO_PULLUP;
        GPIO_InitStruct.Speed     = GPIO_SPEED_FREQ_HIGH;
        GPIO_InitStruct.Alternate = FDCAN1_TX_AF;
        HAL_GPIO_Init(FDCAN1_TX_GPIO_PORT, &GPIO_InitStruct);

        GPIO_InitStruct.Pin       = FDCAN1_RX_PIN;
        GPIO_InitStruct.Alternate = FDCAN1_RX_AF;
        HAL_GPIO_Init(FDCAN1_RX_GPIO_PORT, &GPIO_InitStruct);

        HAL_NVIC_SetPriority(FDCAN1_IT0_IRQn, 2, 0);
        HAL_NVIC_SetPriority(FDCAN1_IT1_IRQn, 2, 0);
        HAL_NVIC_SetPriority(FDCAN_CAL_IRQn, 2, 0);
        HAL_NVIC_EnableIRQ(FDCAN1_IT0_IRQn);
```

```
        HAL_NVIC_EnableIRQ(FDCAN1_IT1_IRQn);
        HAL_NVIC_EnableIRQ(FDCAN_CAL_IRQn);
    }
}
```

92.4.3 FDCAN 的数据发送

FDCAN 每帧数据最大 64 字节：

```
/*
*********************************************************************************************************
*    函 数 名: can1_SendPacket
*    功能说明: 发送一包数据
*    形    参：_DataBuf 数据缓冲区
*              _Len 数据长度，支持 8,12,16,20,24,32,48 或者 64 字节
*    返 回 值: 无
*********************************************************************************************************
*/
void can1_SendPacket(uint8_t *_DataBuf, uint8_t _Len)
{
    FDCAN_TxHeaderTypeDef TxHeader = {0};

    /* 配置发送参数 */
    TxHeader.Identifier = 0x222;                    /* 设置接收帧消息的 ID */
    TxHeader.IdType = FDCAN_STANDARD_ID;            /* 标准 ID */
    TxHeader.TxFrameType = FDCAN_DATA_FRAME;        /* 数据帧 */
    TxHeader.DataLength = (uint32_t)_Len << 16;      /* 发送数据长度 */
    TxHeader.ErrorStateIndicator = FDCAN_ESI_ACTIVE; /* 设置错误状态指示 */
    TxHeader.BitRateSwitch = FDCAN_BRS_ON;           /* 开启可变波特率 */
    TxHeader.FDFormat = FDCAN_FD_CAN;                /* FDCAN 格式 */
    TxHeader.TxEventFifoControl = FDCAN_NO_TX_EVENTS;/* 用于发送事件 FIFO 控制，不存储 */
    TxHeader.MessageMarker = 0;     /* 用于复制到 TX EVENT FIFO 的消息 Maker 来识别消息状态，范围 0 到 0xFF */

    /* 添加数据到 TX FIFO */
    HAL_FDCAN_AddMessageToTxFifoQ(&hfdcan1, &TxHeader, _DataBuf);
}
```

这里特别注意两点：

- 发送的数据幅值给 DataLength，需要右移 16bit。
- 要发送的数据会被复制到 TX FIFO 硬件缓存中。

92.4.4 FDCAN 的中断服务程序处理

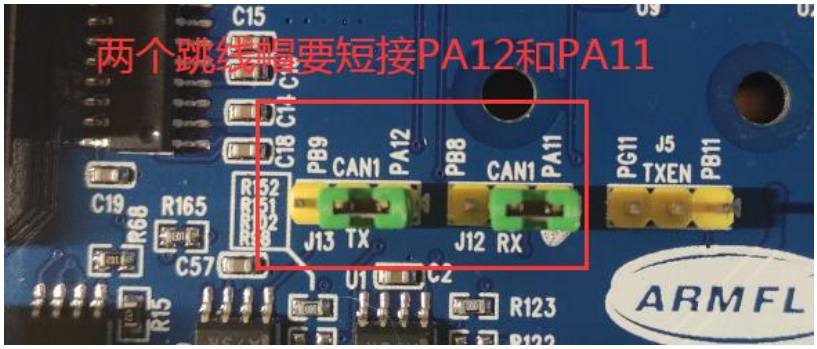
这里 FDCAN 中断主要用于数据接收：

```
/*
*********************************************************************************************************
*    函 数 名: FDCAN1_IT0_IRQHandler
*    功能说明: CAN 中断服务程序
*    形    参: hfdcan
*    返 回 值: 无
*********************************************************************************************************
*/
void FDCAN1_IT0_IRQHandler(void)
```

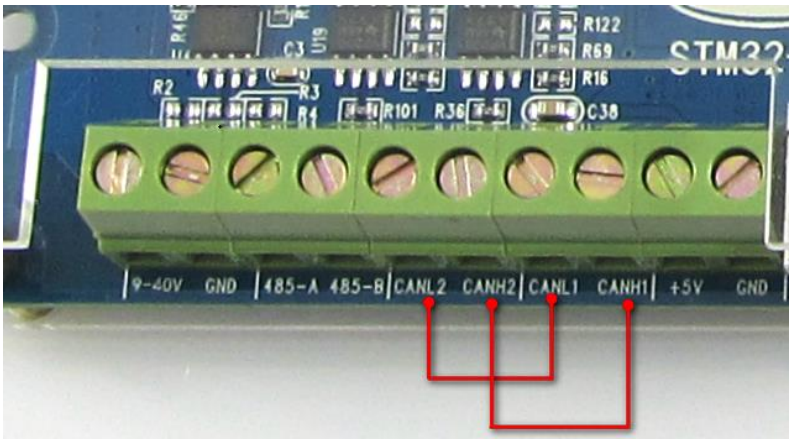
```
{
    HAL_FDCAN_IRQHandler(&hfdcan1);
}

void FDCAN1_IT1_IRQHandler(void)
{
    HAL_FDCAN_IRQHandler(&hfdcan1);
}

void FDCAN_CAL_IRQHandler(void)
{
    HAL_FDCAN_IRQHandler(&hfdcan1);
}
/*
*********************************************************************************************************
*    函 数 名: HAL_FDCAN_RxFifo0Callback
*    功能说明: CAN 中断服务程序-回调函数
*    形    参: hfdcan
*    返 回 值: 无
*********************************************************************************************************
*/
void HAL_FDCAN_RxFifo0Callback(FDCAN_HandleTypeDef *hfdcan, uint32_t RxFifo0ITs)
{
    if (hfdcan == &hfdcan1)
    {
        if ((RxFifo0ITs & FDCAN_IT_RX_FIFO0_WATERMARK) != RESET)
        {
            /* 从 RX FIFO0 读取数据 */
            HAL_FDCAN_GetRxMessage(hfdcan, FDCAN_RX_FIFO0, &g_Can1RxHeader, g_Can1RxData);

            /* 激活 Rx FIFO0 watermark notification */
            HAL_FDCAN_ActivateNotification(hfdcan, FDCAN_IT_RX_FIFO0_WATERMARK, 0);

            if (g_Can1RxHeader.Identifier == 0x111 && g_Can1RxHeader.IdType == FDCAN_STANDARD_ID)
            {
                bsp_PutMsg(MSG_CAN1_RX, 0); /* 发消息收到数据包，结果在 g_Can1RxHeader，  g_Can1RxData */
            }
        }
    }
}
```

92.5 双 FDCAN 测试的接线和跳线帽说明

大家可以直接使用板载的两个 FDCAN 进行通信测试。跳线帽设置如下：



双 CAN FD 接线：

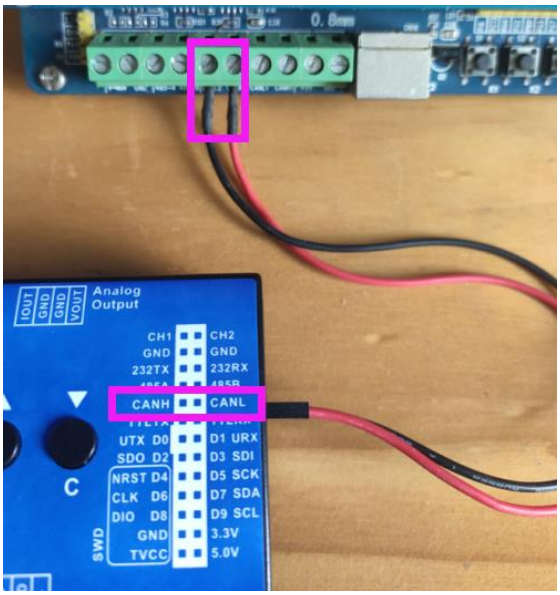


92.6 开发板和 H7-TOOL 的 FDCAN 助手测试

H7-TOOL 的 CAN/CANFD 助手支持以太网，USB 和 WiFi 三种通信方式。大家可以用使用本章配套的双 FDCAN 例子与 H7-TOOL 的 FDCAN 助手进行通信。

92.6.1 接线说明

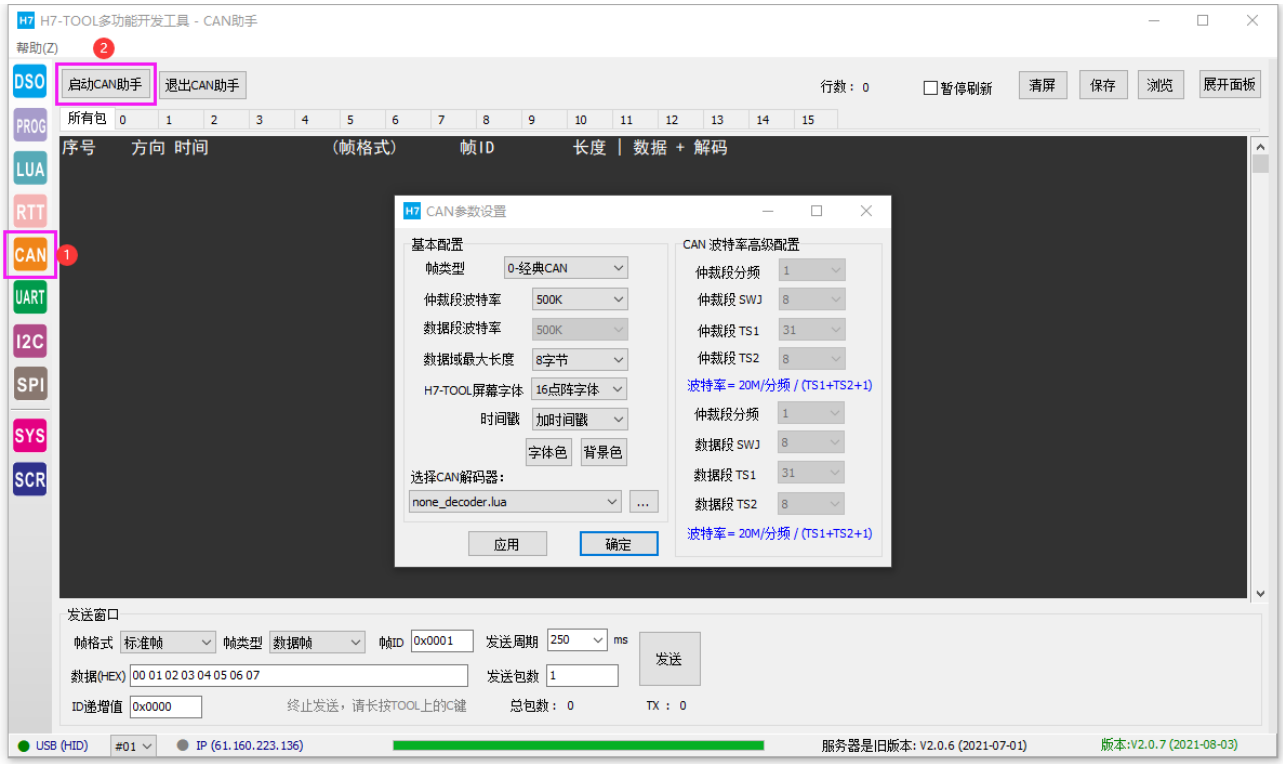
H7-TOOL 和 STM32H7 的 FDCAN2 进行通信，注意是 CANH 接 CANH， CANL 接 CANL



92.6.2 启动 H7-TOOL 上位机

打开最新版的 H7-TOOL 上位机，使用 USB，以太网或者 WiFi 方式均支持。选择左侧的 CAN 助手 ->

启动 CAN 助手：



参数介绍：

（1）帧类型 ： 0 - 经典 CAN，最大收发 8 字节

1 - CAN FD， 仲裁段和数据点波特率相同，最大收发 64 字节

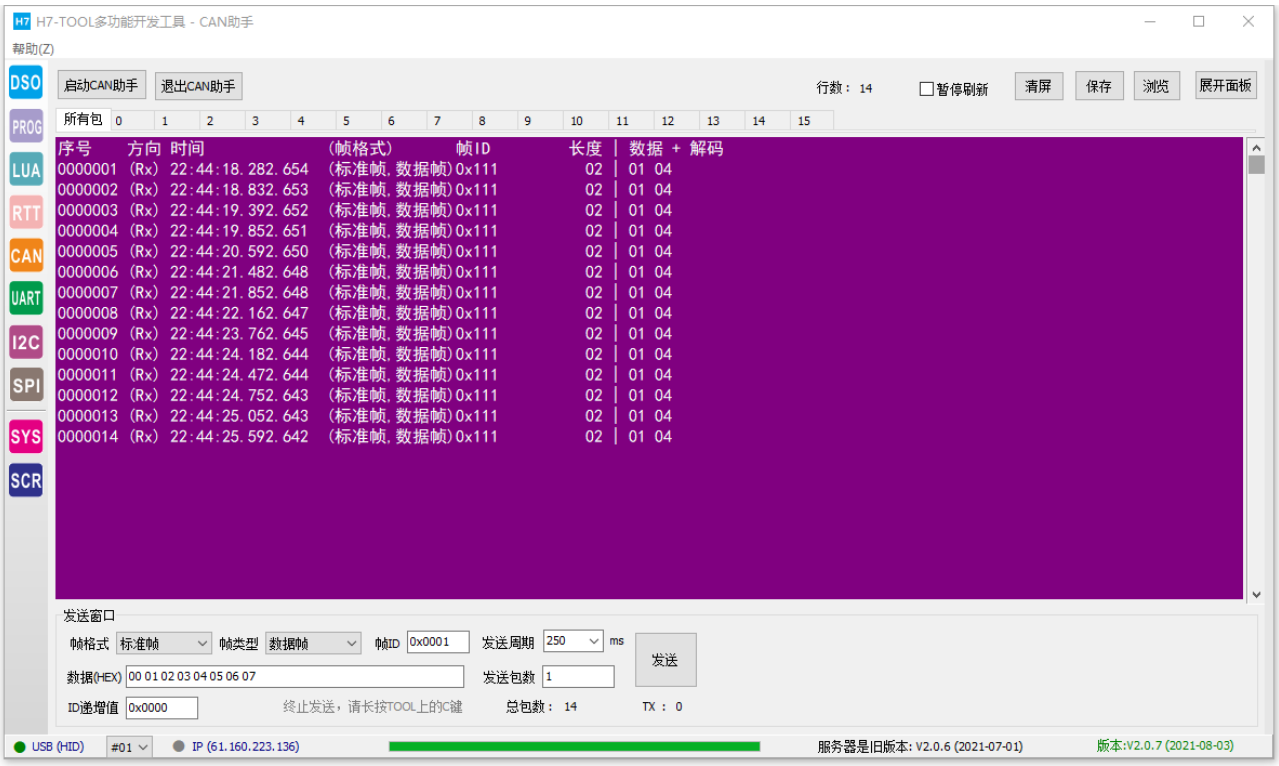
2 - CAN FD 双波特率，仲裁段和数据点波特率不同，最大收发 64 字节

（2）仲裁段和数据段波特率 ：除了提供常用的波特率，还提供了用户可自定义配置模式，需要用户选择如下选项，这样就可以配置右侧的“CAN 波特率高级配置”

我们这里选择 CAN FD 双波特率，仲裁段设置为 500Kbps，数据段设置为 2Mbps，最大数据设置为 8 字节， CAN 解码器设置为 none_decoder.lua

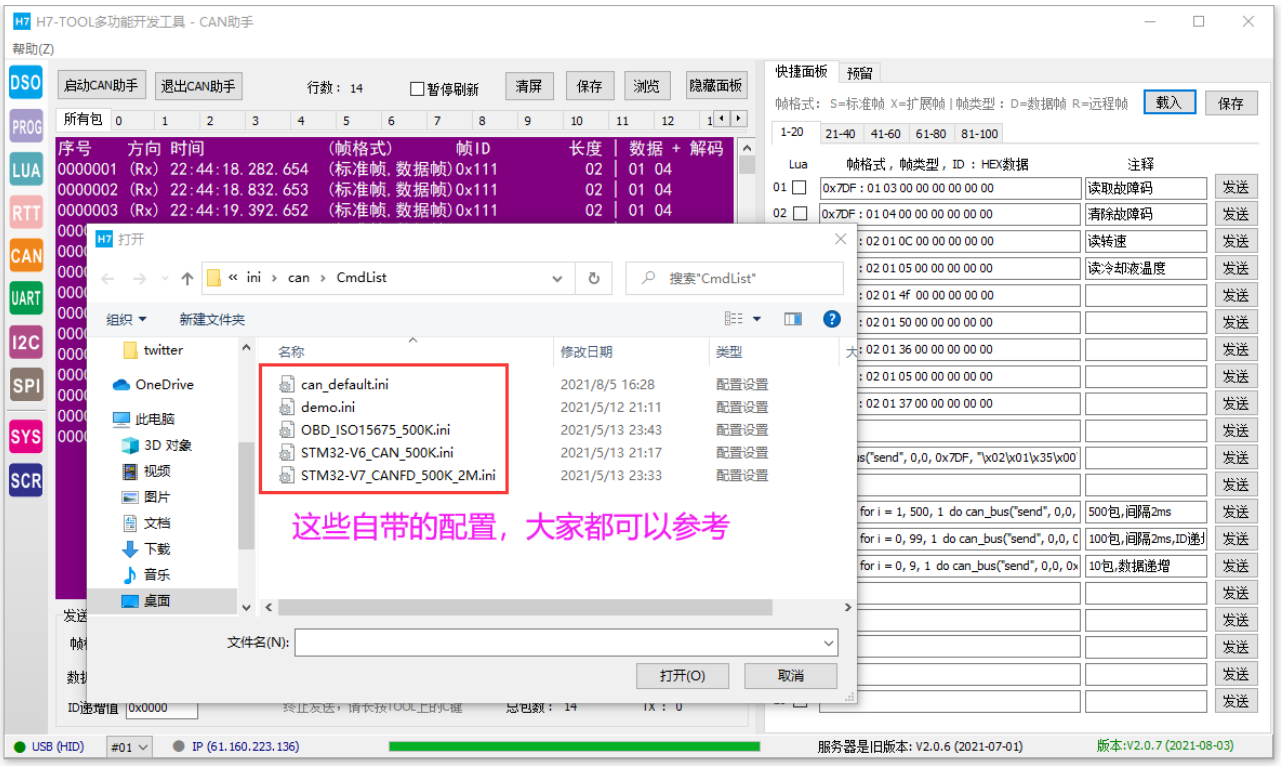
92.6.3 H7-TOOL 的 FDCAN 接收数据测试

第 2 步设置完毕参数后，按下几次 STM32H7 板子的 K1， 可以看到 H7-TOOL 上位机接收到了数据：

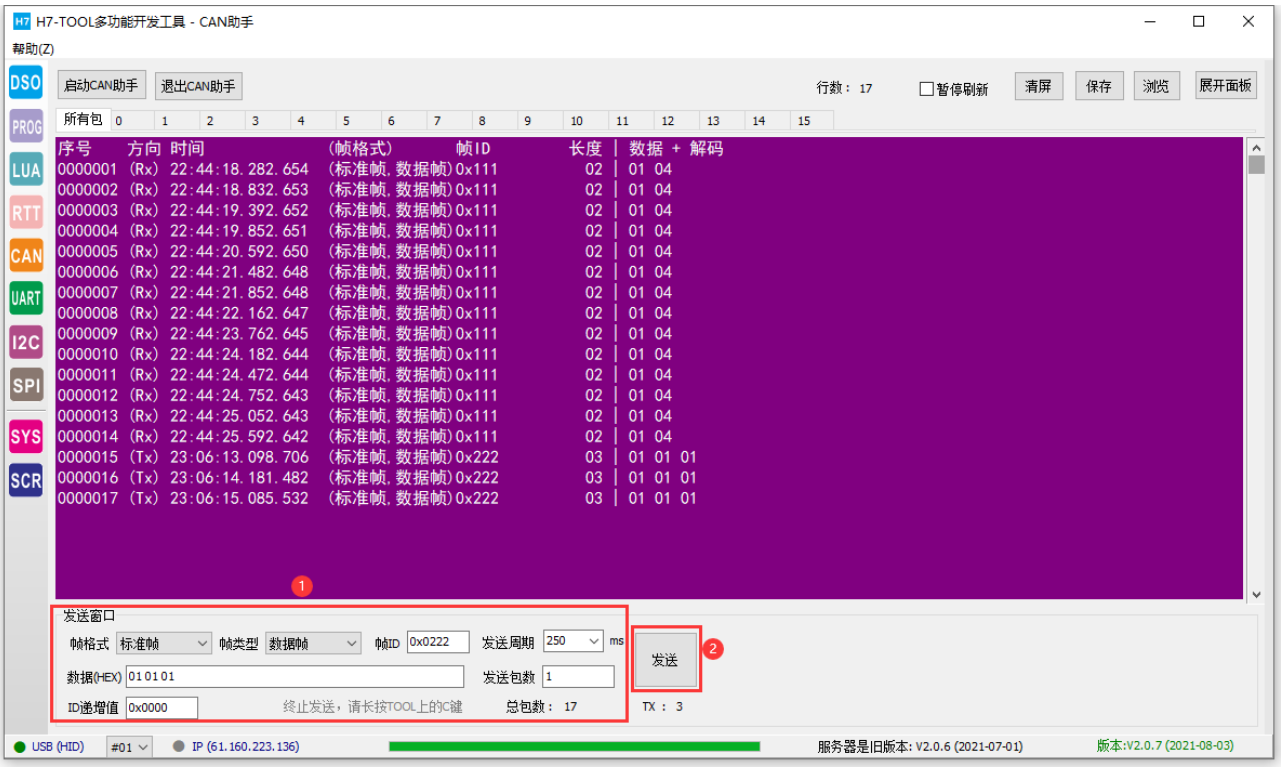


92.6.4 H7-TOOL 的 FDCAN 发送数据测试

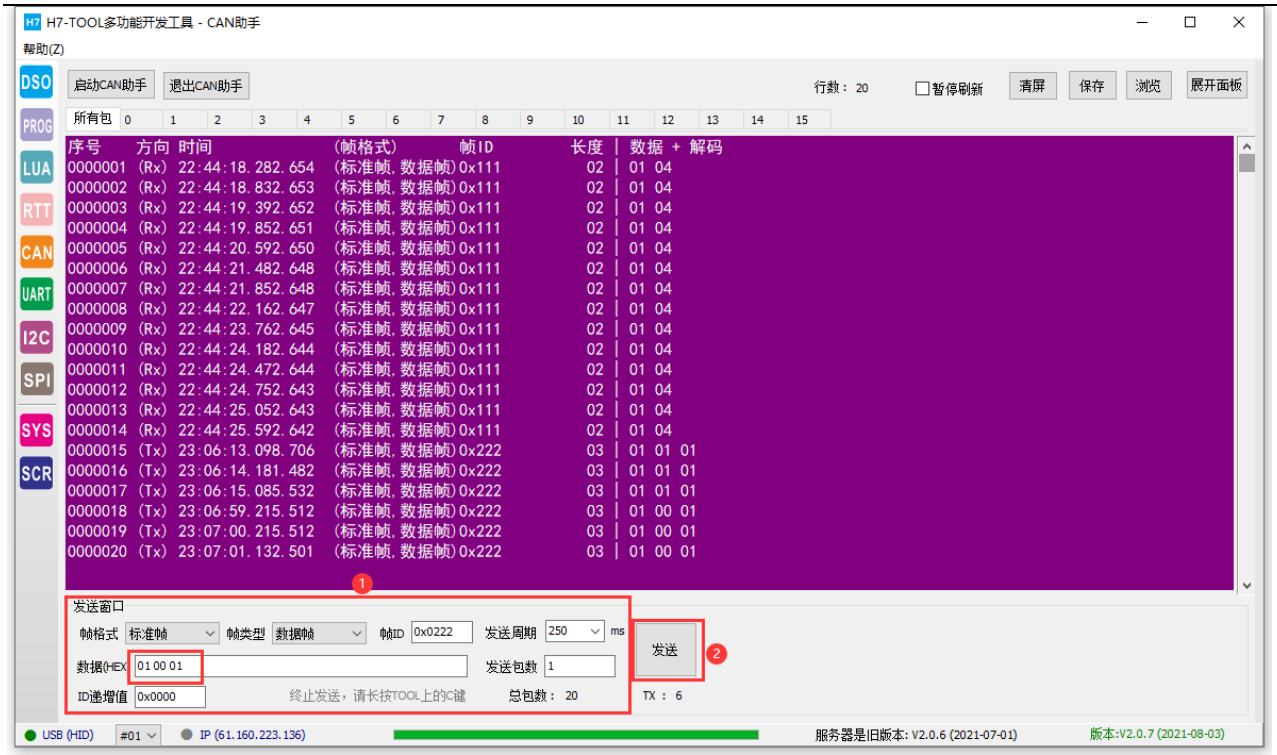
如果有多种发送格式，用户可以在快捷面板里面发送，这个面板也支持用户加载专门的配置文件，不用每次都设置。



如下的配置，点击发送，可以控制 STM32H7 板子的 LED1 点亮：

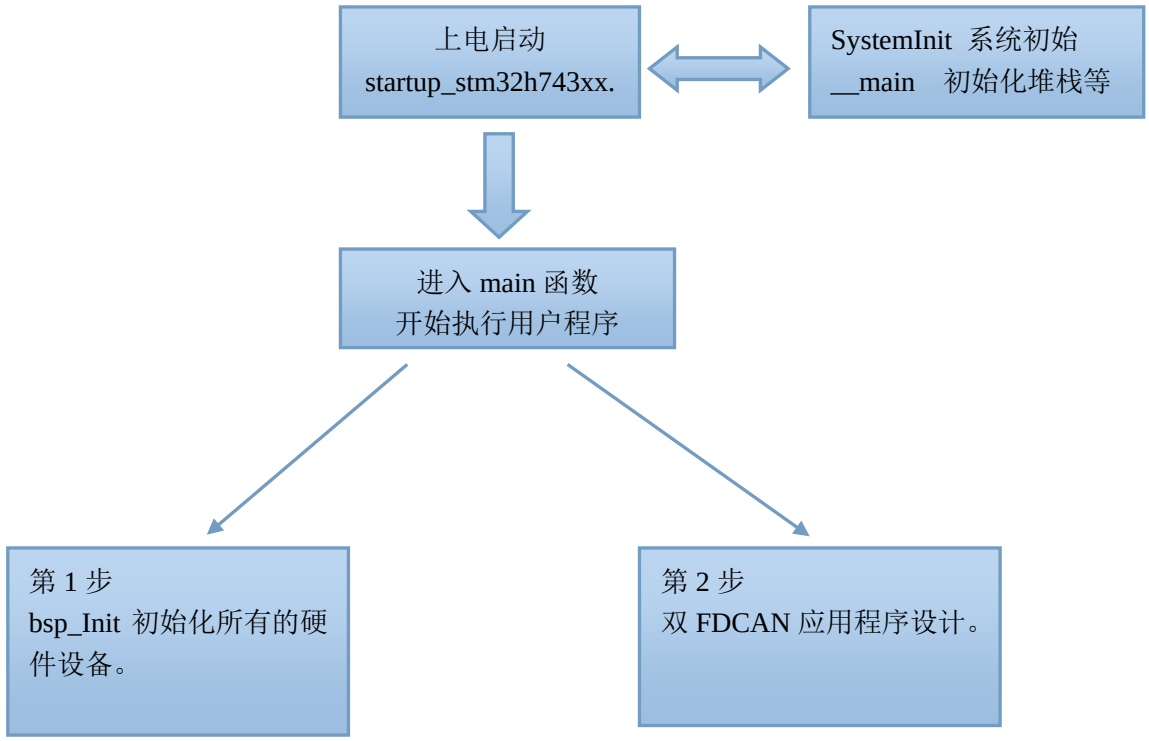


如下的配置，点击发送，可以控制 STM32H7 板子的 LED1 熄灭：



92.7 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



◆ 第 1 阶段，上电启动阶段：

这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 步，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器，LED 和串口。
  - 第 2 步，FDCAN 应用程序设计部分。

92.8 实验例程说明（MDK）

配套例子：

V7-069_双 CAN FD 之间通信

实验目的：

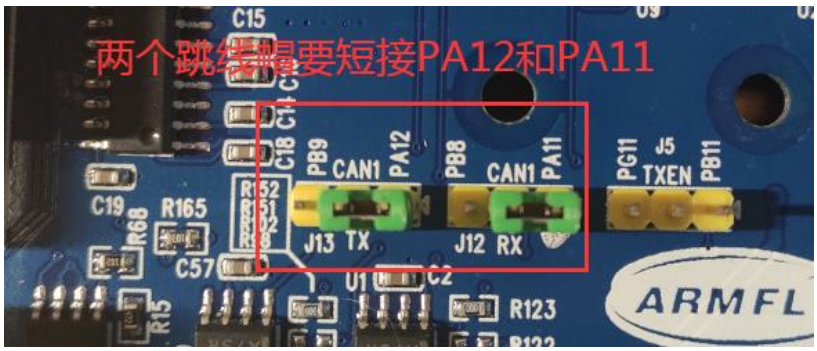
1. 学习双 CAN FD 的实现。

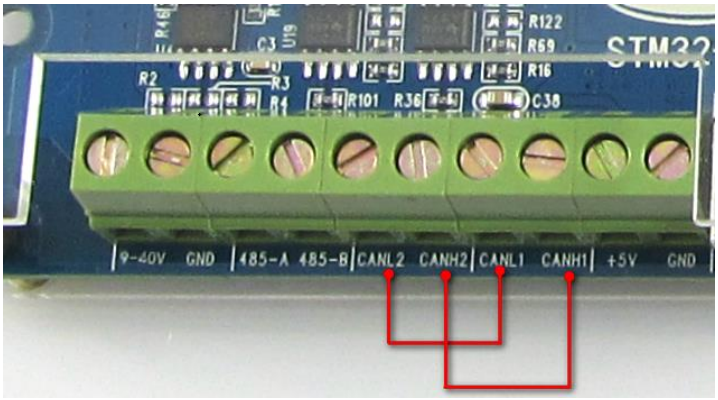
实验内容：

1. K1 按键按下，CAN2 发送消息给 CAN1，蜂鸣器鸣响 4 次。
2. K2 按键按下，CAN1 发送消息给 CAN2，点亮 LED1。
3. K2 按键按下，CAN1 发送消息给 CAN2，熄灭 LED1。

注意事项：

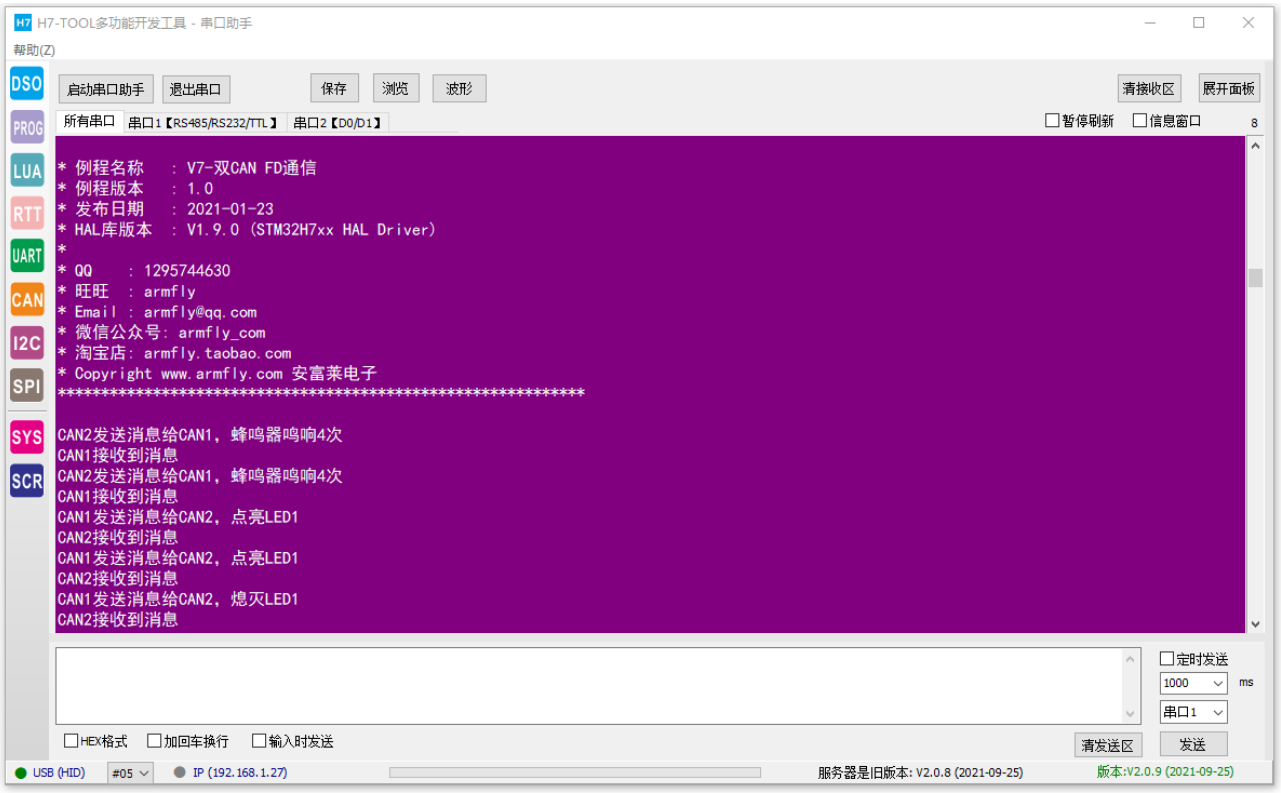
1. 接线方式是 CAN1 的 CANL1 和 CAN2 的 CANL2 连接，CAN1 的 CANH2 和 CAN2 的 CANH2 连接，具体接线看本工程 Doc 文件中的截图。
2. 启用 CAN1，需要将 V7 主板上的 J12 跳线帽短接 PA11，J13 跳线帽短接 PA12。
3. 启用 CNA2，硬件无需跳线，要禁止使用以太网功能（有引脚复用）。





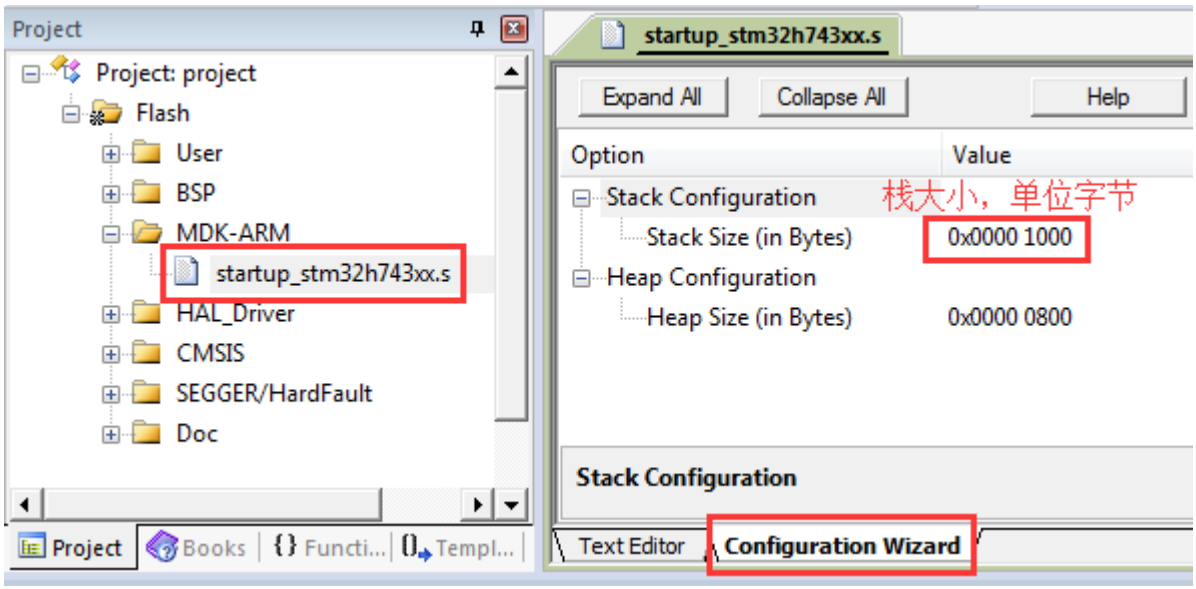
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

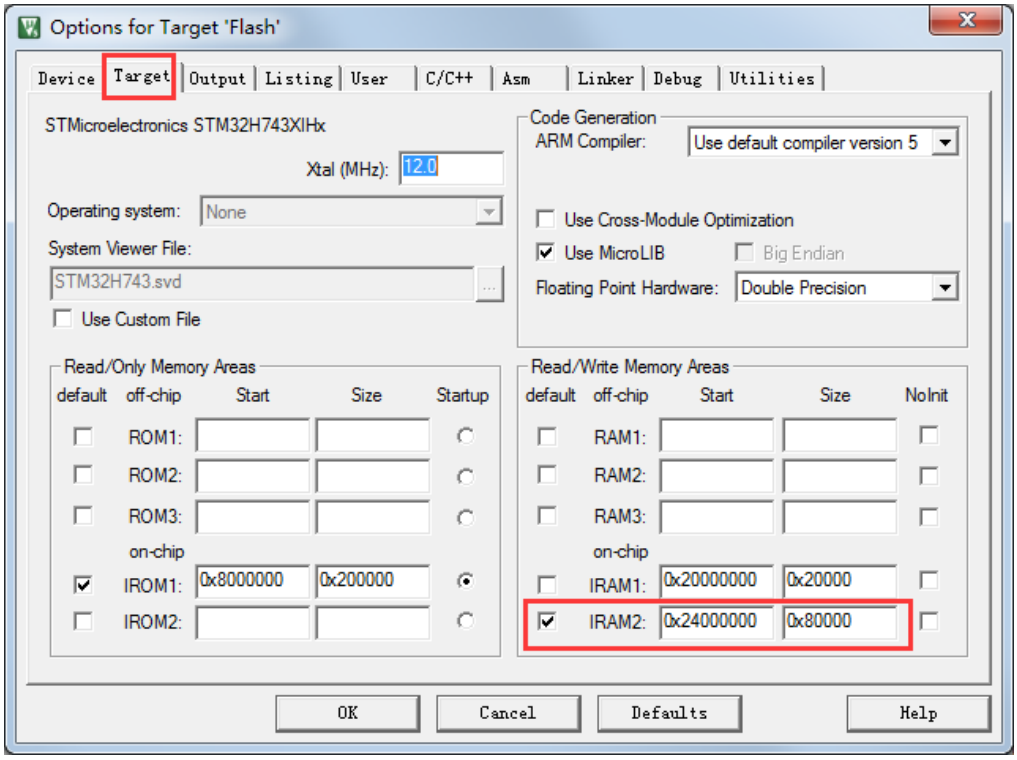


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 AXI SRAM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

◆ MPU 配置和 Cache 配置：

数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

#if 0
    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

 #else
    /* 当前是采用下面的配置 */
    /* 配置 AXI SRAM 的 MPU 属性为 NORMAL, NO Read allocate, NO Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
```

```
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
#endif
    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **主功能：**

  主程序实现如下操作：

  - 上电启动了一个软件定时器，每 100ms 翻转一次 LED2。
  - K1 按键按下，CAN2 发送消息给 CAN1，蜂鸣器鸣响 4 次。
  - K2 按键按下，CAN1 发送消息给 CAN2，点亮 LED1。
  - K2 按键按下，CAN1 发送消息给 CAN2，熄灭 LED1。

```
/*
```

```
*********************************************************************************************************
*    函 数 名: DemoCANFD
*    功能说明: CAN 测试
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoCANFD(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */

    can_Init();    /* 初始化 CAN */

    bsp_StartAutoTimer(0, 500); /* 启动 1 个 500ms 的自动重装的定时器 */

    while (1)
    {
        {
            MSG_T msg;

            if (bsp_GetMsg(&msg))
            {
                switch (msg.MsgCode)
                {
                    case MSG_CAN1_RX:       /* 接收到 CAN 设备的应答 */
                     printf("CAN1 接收到消息\r\n");
                    can1_Analyze();
                        break;

                    case MSG_CAN2_RX:     /* 接收到 CAN 设备的应答 */
                      printf("CAN2 接收到消息\r\n");
                        can2_Analyze();
                        break;
                }
            }
        }

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 500ms 进来一次 */
            bsp_LedToggle(2);
        }

       /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下 */
                    printf("CAN2 发送消息给 CAN1，蜂鸣器鸣响 4 次\r\n");
                  can_BeepCtrl(1, 4);
                    break;

                case KEY_DOWN_K2:           /* K2 键按下 */
                    printf("CAN1 发送消息给 CAN2，点亮 LED1\r\n");
                  can_LedOn(1, 1);
```

```
                    break;


                case KEY_DOWN_K3:       /* K3 键按下 */
                    printf("CAN1 发送消息给 CAN2，熄灭 LED1\r\n");
                 can_LedOff(1, 1);
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }

        }
    }
}
```

92.9 实验例程说明（IAR）

配套例子：

V7-069_双 CAN FD 之间通信

实验目的：

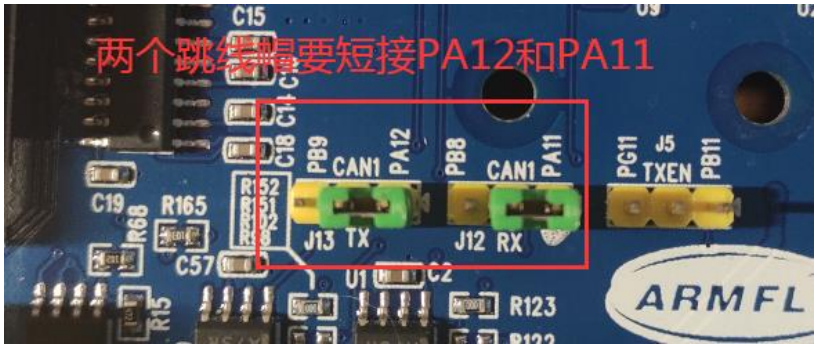
1. 学习双 CAN FD 的实现。

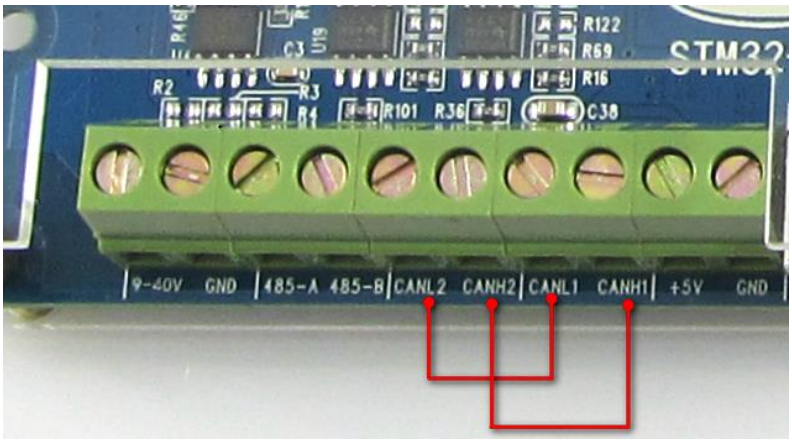
实验内容：

1. K1 按键按下，CAN2 发送消息给 CAN1，蜂鸣器鸣响 4 次。
2. K2 按键按下，CAN1 发送消息给 CAN2，点亮 LED1。
3. K2 按键按下，CAN1 发送消息给 CAN2，熄灭 LED1。

注意事项：

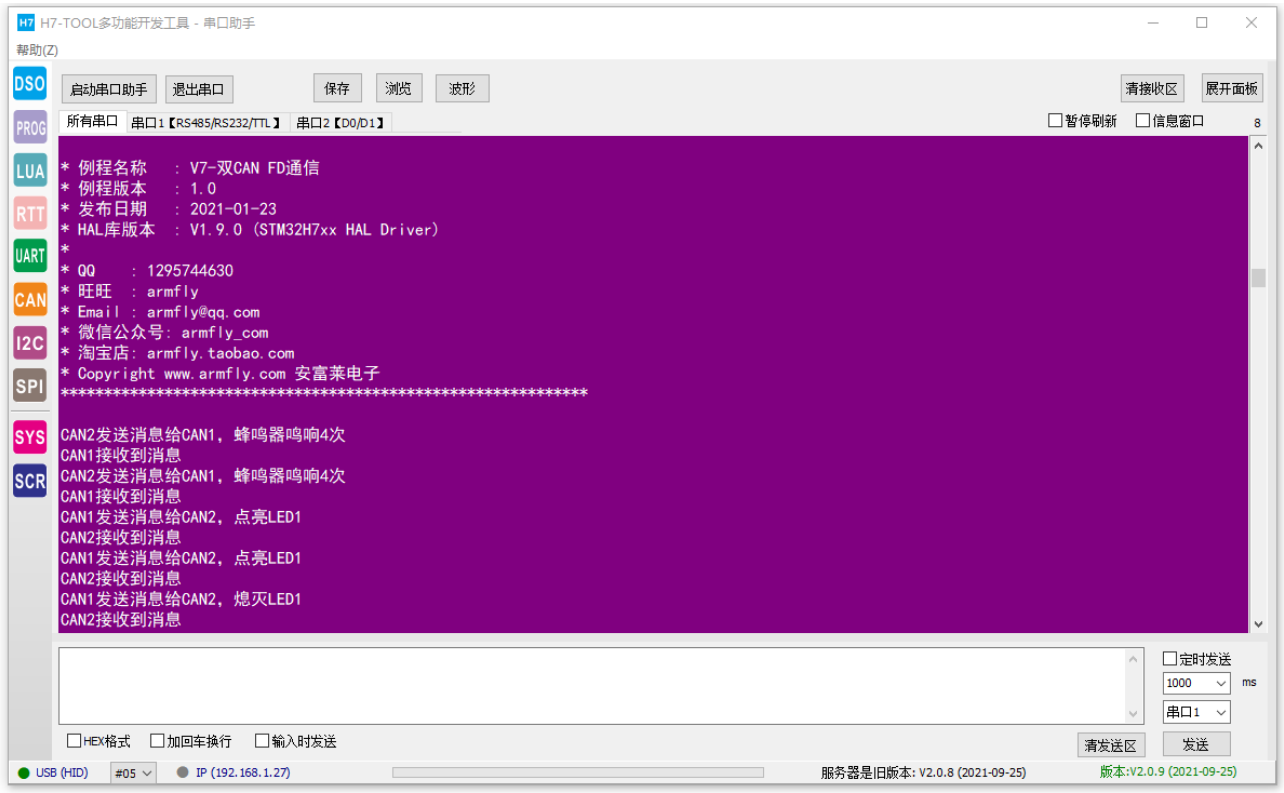
1. 接线方式是 CAN1 的 CANL1 和 CAN2 的 CANL2 连接，CAN1 的 CANH2 和 CAN2 的 CANH2 连接，具体接线看本工程 Doc 文件中的截图。
2. 启用 CAN1，需要将 V7 主板上的 J12 跳线帽短接 PA11，J13 跳线帽短接 PA12。
3. 启用 CNA2，硬件无需跳线，要禁止使用以太网功能（有引脚复用）。





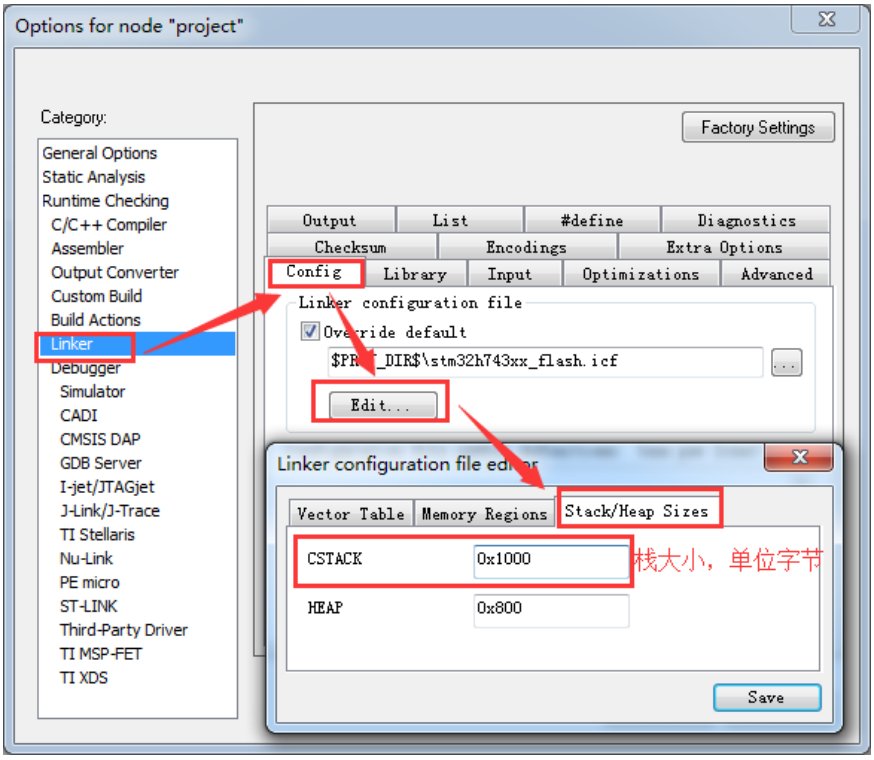
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1

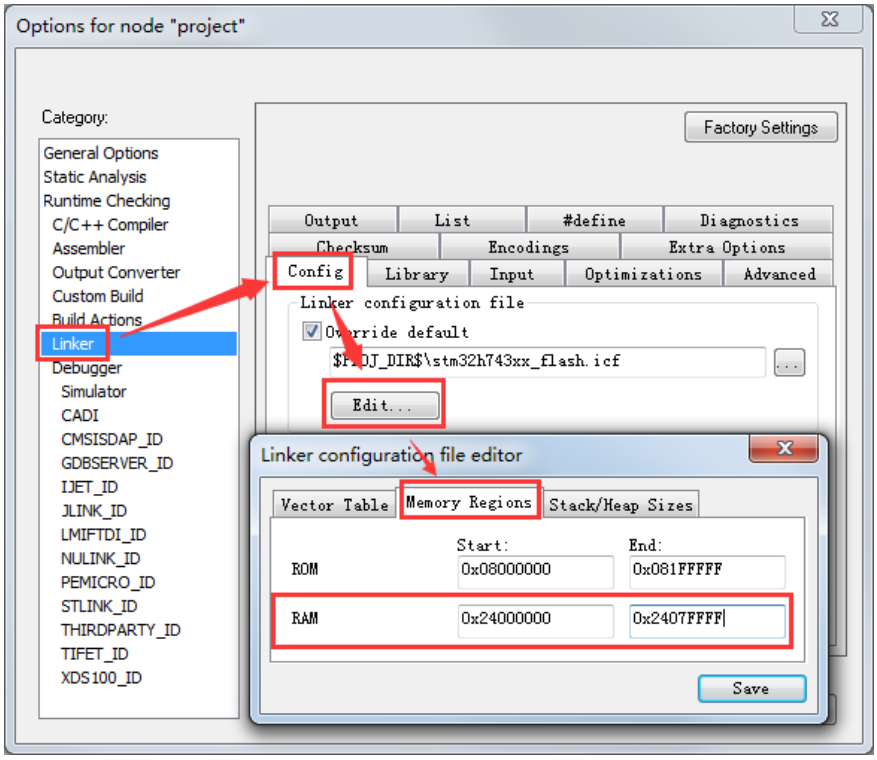


程序设计：

- **系统栈大小分配：**



◆ **RAM 空间用的 AXI SRAM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
```

```
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitKey();    /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();  /* 初始化滴答定时器 */
    bsp_InitUart();   /* 初始化串口 */
    bsp_InitExtIO();  /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();    /* 初始化 LED */
}
```

◆ MPU 配置和 Cache 配置：

数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
```

```
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

#if 0
    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

 #else
    /* 当前是采用下面的配置 */
    /* 配置 AXI SRAM 的 MPU 属性为 NORMAL, NO Read allocate, NO Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
#endif
    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}
```

```
/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **主功能:**

主程序实现如下操作:

- 上电启动了一个软件定时器，每 100ms 翻转一次 LED2。
- K1 按键按下，CAN2 发送消息给 CAN1，蜂鸣器鸣响 4 次。
- K2 按键按下，CAN1 发送消息给 CAN2，点亮 LED1。
- K2 按键按下，CAN1 发送消息给 CAN2，熄灭 LED1。

```
/*
*********************************************************************************************************
*    函 数 名: DemoCANFD
*    功能说明: CAN 测试
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoCANFD(void)
{
    uint8_t ucKeyCode;      /* 按键代码 */

    can_Init();    /* 初始化 CAN */

    bsp_StartAutoTimer(0, 500); /* 启动 1 个 500ms 的自动重装的定时器 */

    while (1)
    {
       {
            MSG_T msg;

            if (bsp_GetMsg(&msg))
            {
                switch (msg.MsgCode)
                {
                    case MSG_CAN1_RX:       /* 接收到 CAN 设备的应答 */
                     printf("CAN1 接收到消息\r\n");
                    can1_Analyze();
                        break;

                    case MSG_CAN2_RX:     /* 接收到 CAN 设备的应答 */
                      printf("CAN2 接收到消息\r\n");
```

```
                        can2_Analyze();
                        break;
                }
            }
        }

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 500ms 进来一次 */
            bsp_LedToggle(2);
        }

       /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下 */
                    printf("CAN2 发送消息给 CAN1，蜂鸣器鸣响 4 次\r\n");
                 can_BeepCtrl(1, 4);
                    break;

                case KEY_DOWN_K2:           /* K2 键按下 */
                    printf("CAN1 发送消息给 CAN2，点亮 LED1\r\n");
                 can_LedOn(1, 1);
                    break;


                case KEY_DOWN_K3:       /* K3 键按下 */
                    printf("CAN1 发送消息给 CAN2，熄灭 LED1\r\n");
                 can_LedOff(1, 1);
                    break;

                default:
                    /* 其它的键值不处理 */
                    break;
            }

        }
    }
}
```

92.10 总结

本章节就为大家讲解这么多，FDCAN 涉及到的知识点非常多，建议先将例子熟练运用，然后再深入了解。

第93章 STM32H7 的 SPI 总线应用之驱动 ADS1256（8 通道 24bit ADC，增益可编程）

本章节为大家讲解 8 通道 24bit ADC 芯片驱动实现。

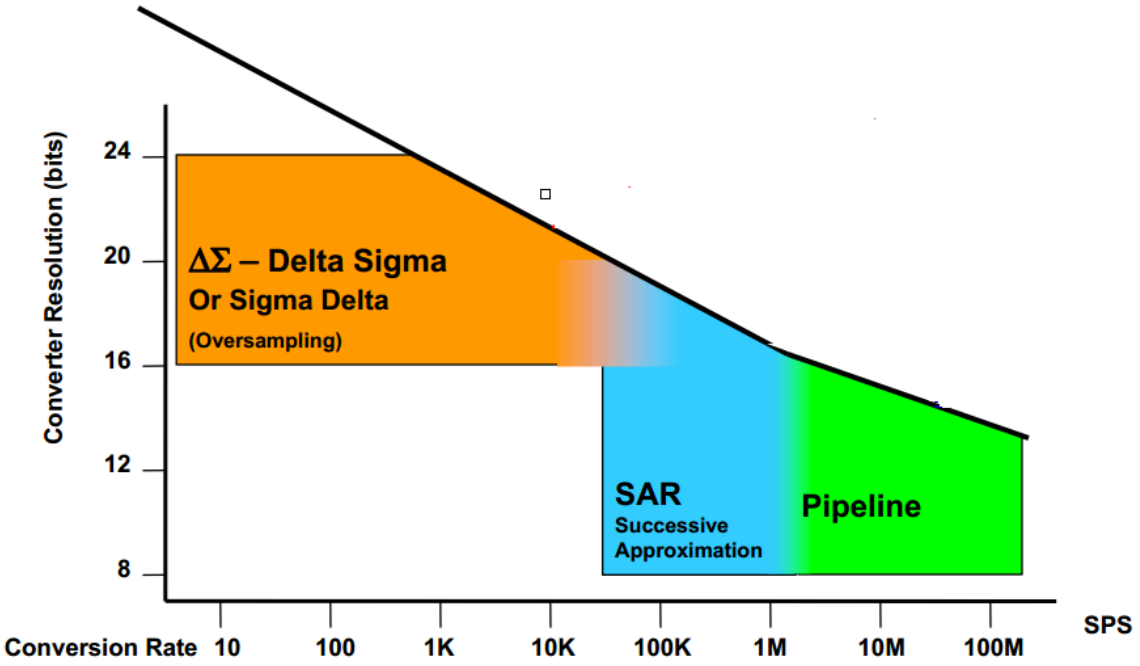
93.1 初学者重要提示
93.2 ADC 结构分类
93.3 ADS1256 硬件设计
93.4 ADS1256 关键知识点整理（重要）
93.5 ADS1256 驱动设计
93.6 ADS1256 板级支持包（bsp_spi_ads1256）
93.7 ADS1256 实际测试效果（10uV 抖动）
93.8 ADS1256 驱动移植和使用
93.9 实验例程设计框架
93.10 实验例程说明（MDK）
93.11 实验例程说明（IAR）
93.12 总结

93.1 初学者重要提示

- ADS1256 的模拟部分供电 5V，SPI 数字接口电平 3.3V。
- ADS1256 的 PGA 可以编程增益支持： 1、2、4、8、16、32、64。
- ADS1256 支持自动校准 （当设置了 PGA,BUF 使能、数据采样率时，会启动自校准)。
- ADS1256 支持 8 通道单端 ADC 采集或者 4 通道差分采集。
- ADS1256 支持正负 5V 差分采集，但不支持负压，使用时要注意。
- ADS1256 时序操作稍有点特殊，所以本章是采用的模拟 SPI 控制。
- ADS1256 数据手册，模块原理图（通用版）和接线图都已经放到本章教程配置例子的 Doc 文件里。
- ADC 的专业术语诠释文档，推荐大家看看：
  http://www.armbbs.cn/forum.php?mod=viewthread&tid=89414 。
- 测试时，务必使用外置电源为开发板供电，因为 ADS1256 需要 5V 供电电压。板子上插入 ADS1256 模块时，注意对齐。

93.2 ADC 结构分类

这里将六种 DAC 结构为大家做个普及。注，这些知识翻译自美信和 TI 的英文技术手册。



93.2.1 SAR ADC（逐次逼近型）

逐次逼近型 ADC 通常是中高分辨率的首选架构，采样速率通常低于 5Msps。SAR ADC 最常见的分辨率范围是 8 位到 20 位，并具有低功耗和小尺寸的特点。这种组合使其非常适合各种应用，例如自动测试设备，电池供电的设备，数据采集系统，医疗仪器，电机和过程控制，工业自动化，电信，测试和测量，便携式系统，高速闭环系统和窄带接收器。

93.2.2 Sigma-Delta ADC

Sigma-delta ADC 主要用于低速应用中，该应用需要通过过采样来权衡速度和分辨率，然后进行滤波以降低噪声。24 位 sigma-delta 转换器用于自动化测试设备，高精度便携式传感器，医疗和科学仪器以及地震数据采集等应用中。

93.2.3 Integrating ADC

集成 ADC 提供高分辨率，并且可以提供良好的线路频率和噪声抑制。集成架构提供了一种新颖且直接的方法，可将低带宽模拟信号转换为数字表示形式。这些类型的转换器通常包括用于 LCD 或 LED 显示器的内置驱动器，并且在许多便携式仪器应用中都可以找到，包括数字面板表和数字万用表。

93.2.4 FLASH ADC

Flash ADC 是将模拟信号转换为数字信号的最快方法。它们适用于需要非常大带宽的应用。然而，闪存转换器功率高，具有相对较低的分辨率，并且可能非常昂贵。这将它们限制在通常无法以其他任何方式解决的高频应用中。示例包括数据采集，卫星通信，雷达处理，示波器和高密度磁盘驱动器。

93.2.5 Pipelined ADC

流水线 ADC 已成为最受欢迎的 ADC 体系结构，其采样率从每秒几兆采样（MS / s）到最高 100MS / s +，分辨率为 8 至 16 位。它们提供的分辨率和采样率，可覆盖各种应用，包括 CCD 成像，超声医学成像，数字接收器，基站，数字视频（例如 HDTV），xDSL，电缆调制解调器和快速以太网。

93.2.6 Two Step ADC

两步 ADC 也称为子范围转换器，有时也称为多步或 half flash（比 Flash 架构慢）。这是 Flash ADC 和流水线 ADC 的交叉点。与 Flash ADC 相比，可以实现更高的分辨率或更小的裸片尺寸。

93.3 ADS1256 硬件设计

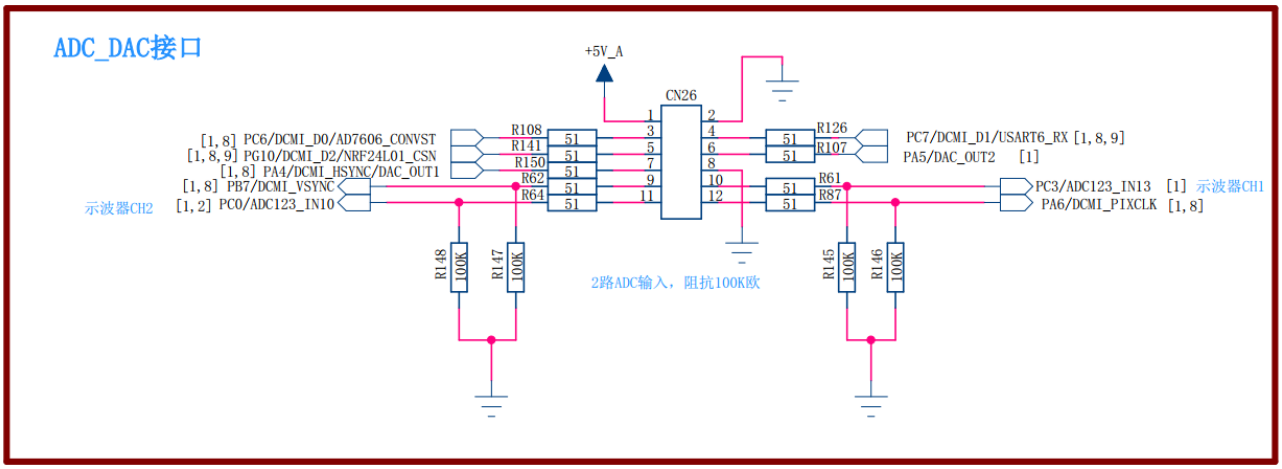
这里将开发板上的 ADS1256 硬件接口，ADS1256 模块为大家做个说明。

ADS1256 的原理图论坛下载：

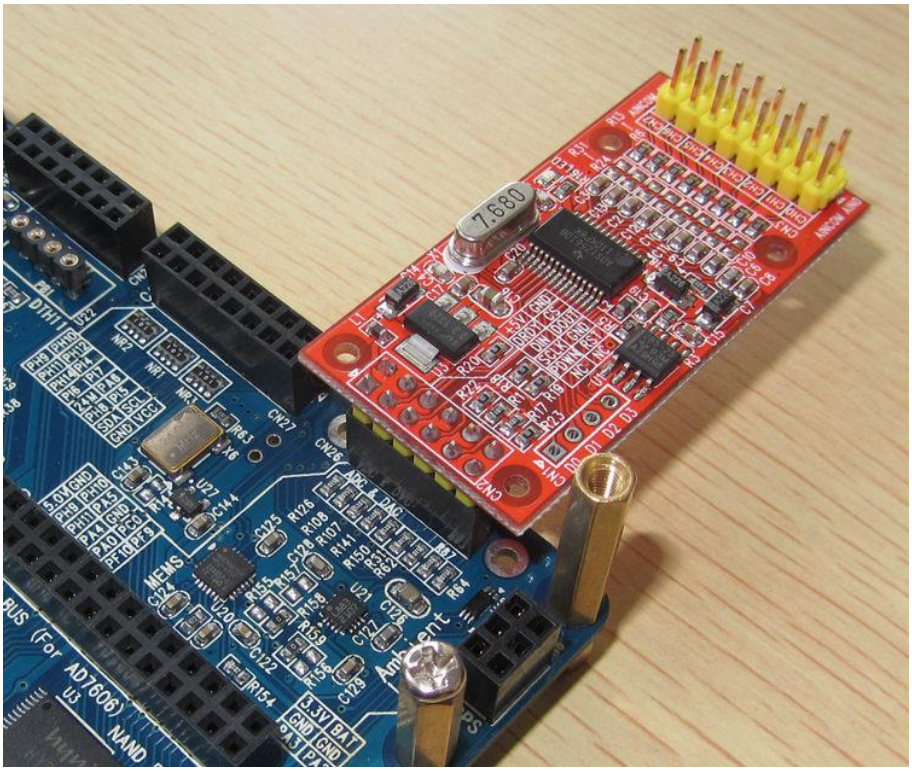
http://www.armbbs.cn/forum.php?mod=viewthread&tid=97547 。

93.3.1 ADS1256 硬件接口

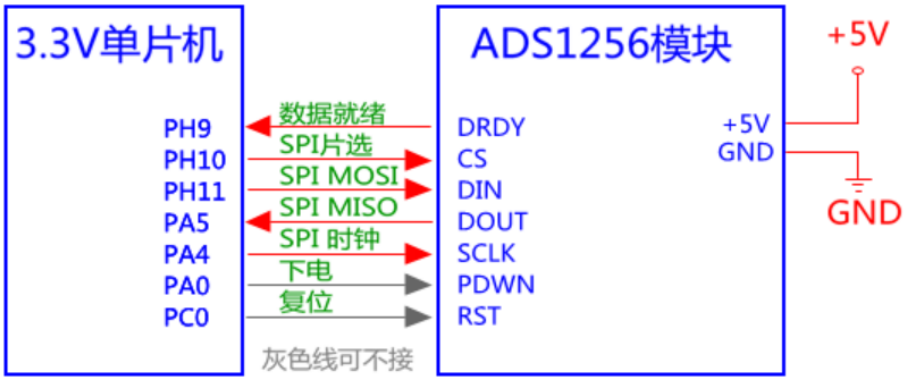
V7 板子上 ADS1256 模块的插座的原理图如下：



实际对应开发板的位置如下：



为了方便大家更好的理解接线，下面是框图：



93.3.2 ADS1256 模块

产品规格：

6、单电源 5.0V DC 供电，提供正负 5V 信号采样功能。

7、MCU 接口：SPI。

8、主 ADC 芯片：ADS1256 （全新进口原装正品）。

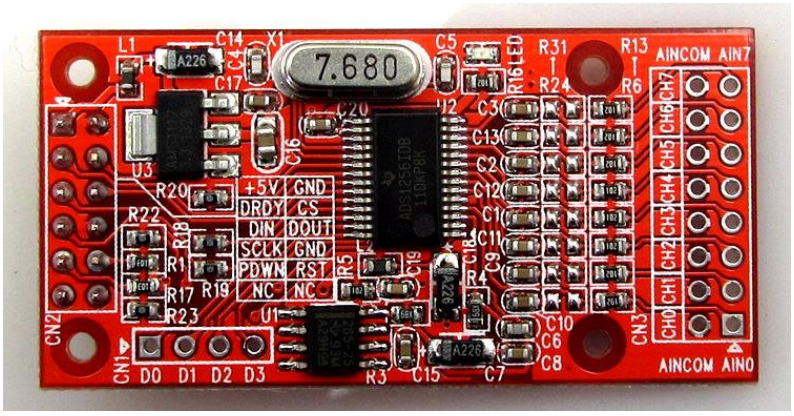
9、电压基准，之前采用的 LM285-2.5，现在采用的 REF5025（全新进口原装正品）。

10、 输入电路带分压电阻和 R-C 滤波，方便客户自己变更增益范围。

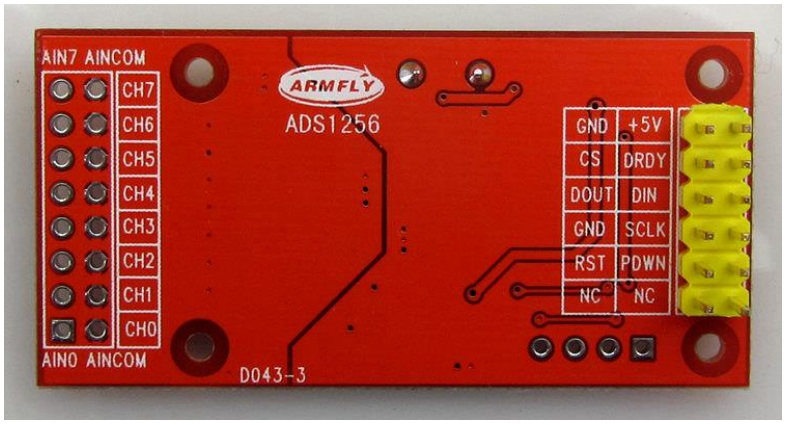
11、 芯片内带可编程增益放大器，增益范围：1-64 倍。

12、 芯片内部输入带缓冲放大器，可以直接连接传感器。

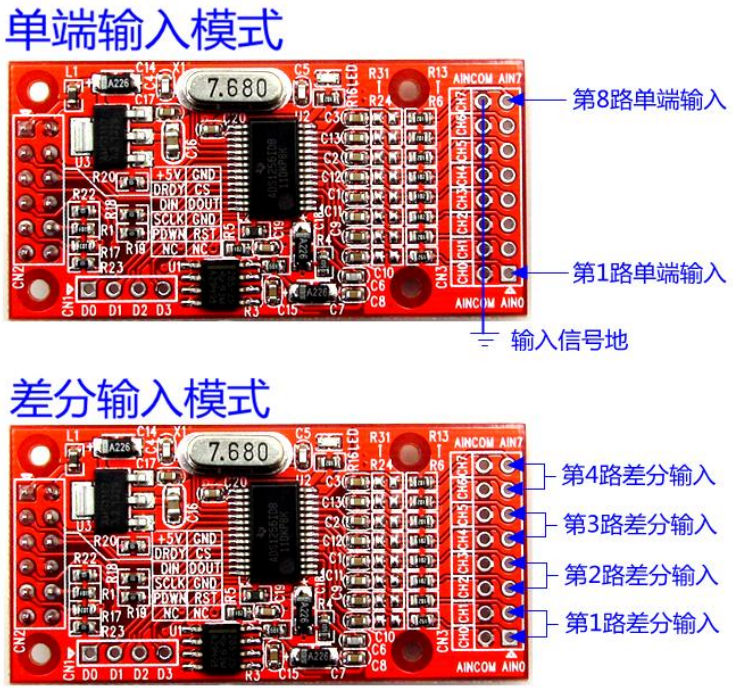
正面：



反面：



接线图：



93.4 ADS1256 关键知识点整理（重要）

驱动 ADS1256 需要对下面这些知识点有个认识。

93.4.1 ADS1256 基础信息

ADS1256 是 TI 公司推出的微功耗、高精度、8 通道、 24 位△-∑型高性能模数转换器（ADC）。该器件提供高达 23 比特的无噪声精度、数据速率高达 30kSPS（次采样/秒）、0.0010%非线性特性(最大值)以及众多的板上外设（输入模拟多路开关、输入缓冲器、可编程增益放大器和可编程数字滤波器等），可为设计人员带来完整而高分辨率的量测解决方案。

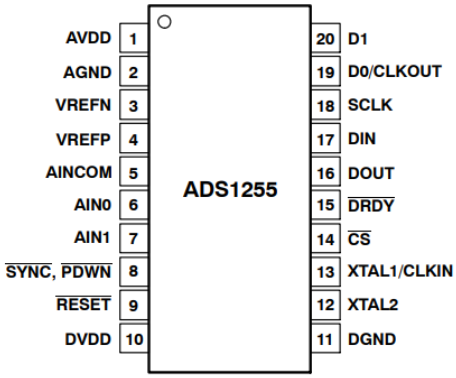
- 24 位无数据丢失;
- 高达 23 比特的无噪声精度;
- 低非线性度: ±0.0010% ;
- 数据采样率可达 30kSPS ;
- 采用单周期转换模式;
- 带有模拟多路开关,具有传感器接口（可配置为 4 路差动输入或 8 路单极输入） ;
- 带有输入缓冲器(BUF) ;
- 带有串行外设接口(SPI) ;
- 内含低噪声可编程增益放大器(PGA) ,所有的 PGA 均具有自校准和系统校准;
  - PGA= 1 时,可提供高达 25.3 位的有效分辨率;
  - PGA = 64 时,可提供高达 22.5 位的有效分辨率;
- 模拟输入电压为 5V ,数字电压为 1.8~3.6V ;
- 正常模式下功耗低至 38mW ,备用模式下功耗为 0.4mW。

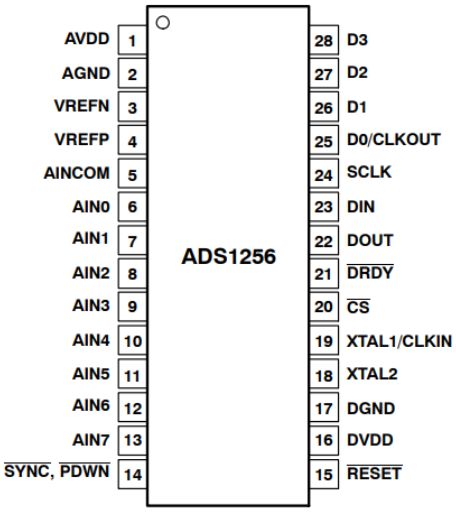
93.4.2 ADS1256 常用引脚的作用

ADS1256 的封装形式:

PIN ASSIGNMENTS

SSOP PACKAGE
(TOP VIEW)





这里把常用的几个引脚做个说明：

- AVDD

  模拟电源供电。

- AGND

  模拟地。

- VREFN

  负参考电压输入。

- VREFP

  正参考电压输入。

- AINCOM

  模拟公共输入。

- AIN0 – AIN7

  模拟输入通道 0 到通道 7

- SYNC/PDWN

  同步，掉电输入。

- RESET

  复位引脚。

- DVDD

  数字电源。

- DGND

  数字地。

- XTAL1，XTAL2

  晶振输入端。

- CS

  片选输入端。

- DRDY

  数据就绪输出信号引脚。

- DOUT

  数据输出。

- DIN

  数据输入。

- SCLK

  时钟引脚。

- D0,D1,D2,D3

  通用 GPIO

93.4.3 ADS1256 输出电压计算公式

ADS1256 的计算公式如下：

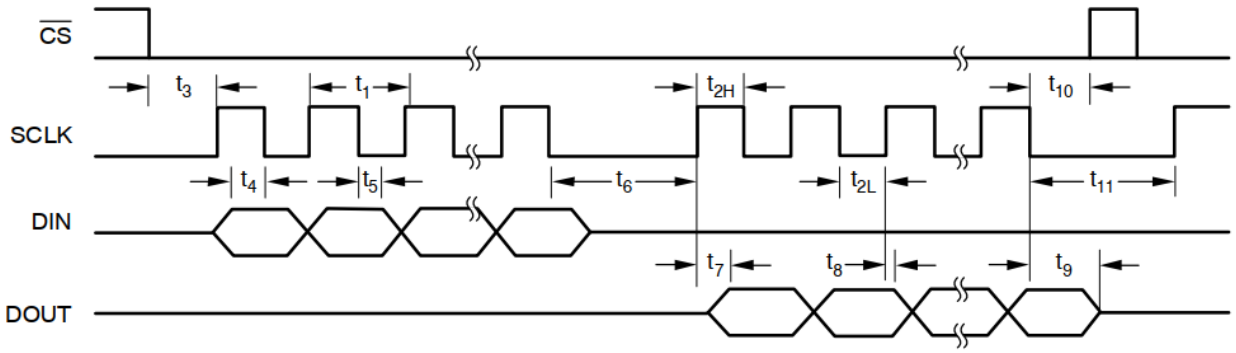
INPUT SIGNAL V_{IN} ($AIN_P - AIN_N$)	IDEAL OUTPUT CODE(1)
$\geq \frac{+2V_{REF}}{PGA}$	7FFFFFh
$\frac{+2V_{REF}}{PGA(2^{23}-1)}$	000001h
0	000000h
$\frac{-2V_{REF}}{PGA(2^{23}-1)}$	FFFFFFh
$\leq \frac{-2V_{REF}}{PGA}\left(\frac{2^{23}}{2^{23}-1}\right)$	800000h

最小单位值是 2 * VREF/(PGA * (2^23 – 1))

采用二进制补码表示（其实就是 24bit 有符号数，我们将转换结果定义为 int32_t 即可）。

93.4.4 ADS1256 时序图

驱动 ADS1256 主要是两个时序图需要了解，读写时序：



◆ t_1：SPI 时钟周期：

外部晶振频率 = 7.68MHz,

时钟频率 tCLK = 1/7.68M = 0.13uS

输出数据周期 tDATA = 1 / 30K = 0.033mS (按 30Ksps 计算)

对 SPI 的时钟速度要求:

最快 4 个 tCLK = 0.52uS

最慢 10 个 tDATA = 0.3mS (按 30Ksps 计算)

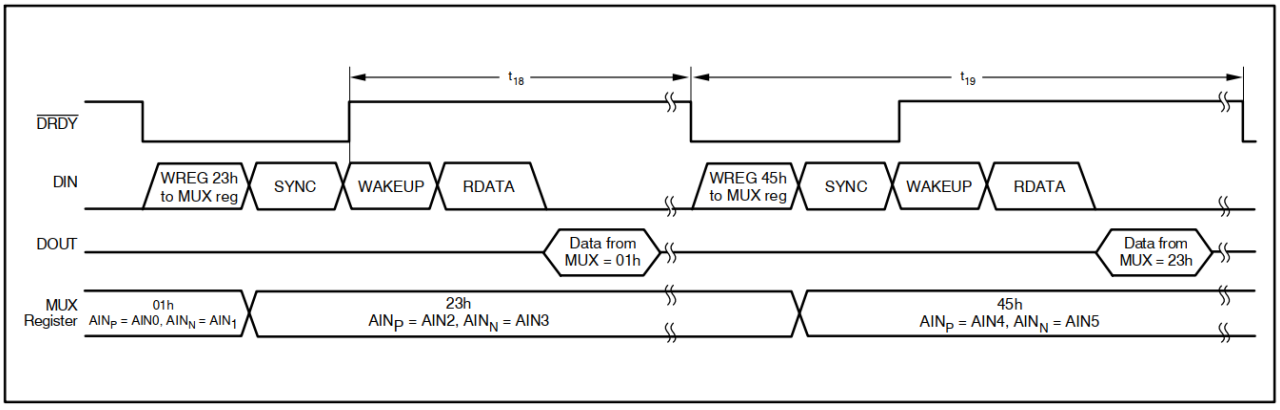
SYMBOL	DESCRIPTION	MIN	MAX	UNIT
t_1	SCLK period	4		τ_{CLKIN} (1)
			10	τ_{DATA} (2)

◆ t_{2H}，t_{2L} ：脉冲高低电平：

SCL 高电平和低电平持续时间最小 200ns。

SYMBOL	DESCRIPTION	MIN	MAX	UNIT
t_1	SCLK period	4		τ_{CLKIN} (1)
			10	τ_{DATA} (2)
t_{2H}	SCLK pulse width: high	200		ns
			9	τ_{DATA}
t_{2L}	SCLK pulse width: low	200		ns

数据读取流程：



- 第 1 步：ADS1256_SetChannal(g_tADS1256.Channel)切换模拟通道。
- 第 2 步：发送 SYNC 同步命令。
- 第 3 步：唤醒。
- 第 4 步：读取数据。

  这个过程中特别注意读取的是上次转换的数据。

93.4.5 ADS1256 的增益和测量范围问题

ADS1256 的增益和测量范围关系如下：

PGA SETTING	FULL-SCALE INPUT VOLTAGE V_{IN}(1) (V_{REF} = 2.5V)
1	±5V
2	±2.5V
4	±1.25V
8	±0.625V
16	±312.5mV
32	±156.25mV
64	±78.125mV

比如增益是 1 时，测量范围是正负 5V，增益是 64 时，测量范围是正负 78.125mV。

93.4.6 ADS1256 输入缓冲器

开关输入缓冲器时，影响到的几个参数，大家需要做个了解。

开缓冲的情况下，输入参考值噪声。

DATA RATE (SPS)	PGA						
	1	2	4	8	16	32	64
2.5	0.247	0.156	0.080	0.056	0.043	0.037	0.033
5	0.301	0.175	0.102	0.076	0.061	0.045	0.044
10	0.339	0.214	0.138	0.106	0.082	0.061	0.061
15	0.401	0.264	0.169	0.126	0.107	0.085	0.073
25	0.494	0.305	0.224	0.149	0.134	0.102	0.093
30	0.533	0.335	0.245	0.176	0.138	0.104	0.106
50	0.629	0.393	0.292	0.216	0.168	0.136	0.122
60	0.692	0.438	0.321	0.233	0.184	0.146	0.131
100	0.875	0.589	0.409	0.305	0.229	0.170	0.169
500	1.946	1.250	0.630	0.648	0.497	0.390	0.367
1000	2.931	1.891	1.325	1.070	0.689	0.512	0.486
2000	4.173	2.589	1.827	1.492	0.943	0.692	0.654
3750	5.394	3.460	2.376	1.865	1.224	0.912	0.906
7500	7.249	4.593	3.149	2.436	1.691	1.234	1.187
15,000	9.074	5.921	3.961	2.984	2.125	1.517	1.515
30,000	10.728	6.705	4.446	3.280	2.416	1.785	1.742

关闭缓冲时的输入参考噪声：

DATA RATE (SPS)	PGA						
	1	2	4	8	16	32	64
2.5	0.247	0.149	0.097	0.058	0.036	0.031	0.027
5	0.275	0.176	0.109	0.070	0.046	0.039	0.038
10	0.338	0.201	0.129	0.084	0.063	0.048	0.047
15	0.401	0.221	0.150	0.109	0.070	0.063	0.057
25	0.485	0.279	0.177	0.136	0.093	0.076	0.076
30	0.559	0.315	0.202	0.142	0.107	0.093	0.082
50	0.644	0.390	0.238	0.187	0.129	0.108	0.103
60	0.688	0.417	0.281	0.204	0.134	0.109	0.111
100	0.815	0.530	0.360	0.233	0.169	0.123	0.122
500	1.957	1.148	0.772	0.531	0.375	0.276	0.259
1000	2.803	1.797	1.191	0.940	0.518	0.392	0.365
2000	4.025	2.444	1.615	1.310	0.700	0.526	0.461
3750	5.413	3.250	2.061	1.578	0.914	0.693	0.625
7500	7.017	4.143	2.722	1.998	1.241	0.914	0.857
15,000	8.862	5.432	3.378	2.411	1.569	1.149	1.051
30,000	10.341	6.137	3.873	2.775	1.805	1.313	1.211

开缓冲的情况下，有效分辨率：

DATA RATE (SPS)	PGA						
	1	2	4	8	16	32	64
2.5	25.3	24.9	24.9	24.4	23.8	23.0	22.2
5	25.0	24.8	24.5	24.0	23.3	22.7	21.8
10	24.8	24.5	24.1	23.5	22.9	22.3	21.3
15	24.6	24.2	23.8	23.2	22.5	21.8	21.0
25	24.3	24.0	23.4	23.0	22.2	21.5	20.7
30	24.2	23.8	23.3	22.8	22.1	21.5	20.5
50	23.9	23.6	23.0	22.5	21.8	21.1	20.3
60	23.8	23.4	22.9	22.4	21.7	21.0	20.2
100	23.4	23.0	22.5	22.0	21.4	20.8	19.8
500	22.3	21.9	21.5	20.9	20.3	19.6	18.7
1000	21.7	21.3	20.8	20.2	19.8	19.2	18.3
2000	21.2	20.9	20.4	19.7	19.3	18.8	17.9
3750	20.8	20.5	20.0	19.4	19.0	18.4	17.4
7500	20.4	20.1	19.6	19.0	18.5	17.9	17.0
15,000	20.1	19.7	19.3	18.7	18.2	17.7	16.7
30,000	19.8	19.5	19.1	18.5	18.0	17.4	16.5

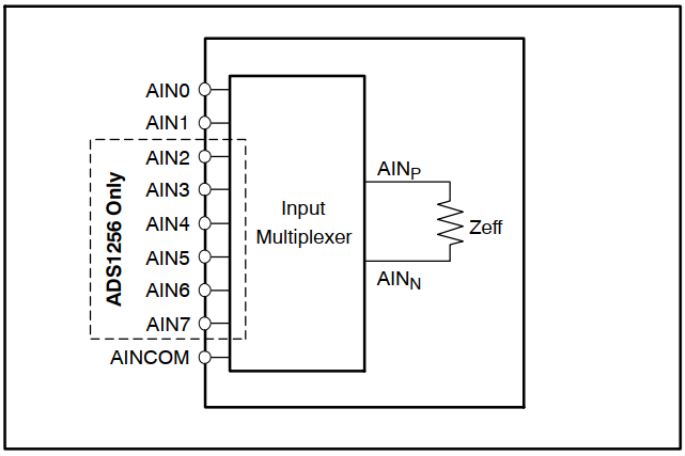
关闭缓冲时的有效位数：

DATA RATE (SPS)	PGA						
	1	2	4	8	16	32	64
2.5	25.3	25.0	24.6	24.4	24.0	23.2	22.5
5	25.1	24.8	24.5	24.1	23.7	22.9	22.0
10	24.8	24.6	24.2	23.8	23.2	22.6	21.7
15	24.6	24.4	24.0	23.4	23.1	22.2	21.4
25	24.3	24.1	23.8	23.1	22.7	22.0	21.0
30	24.1	23.9	23.6	23.1	22.5	21.7	20.9
50	23.9	23.6	23.3	22.7	22.2	21.5	20.5
60	23.8	23.5	23.1	22.5	22.1	21.5	20.4
100	23.5	23.2	22.7	22.4	21.8	21.3	20.3
500	22.3	22.1	21.6	21.2	20.7	20.1	19.2
1000	21.8	21.4	21.0	20.3	20.2	19.6	18.7
2000	21.2	21.0	20.6	19.9	19.8	19.2	18.4
3750	20.8	20.6	20.2	19.6	19.4	18.8	17.9
7500	20.4	20.2	19.8	19.3	18.9	18.4	17.5
15,000	20.1	19.8	19.5	19.0	18.6	18.1	17.2
30,000	19.9	19.6	19.3	18.8	18.4	17.9	17.0

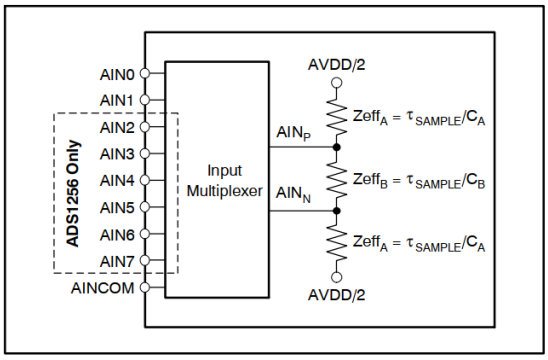
打开缓冲器后的输入阻抗：

DATA RATE (SPS)	Zeff (MΩ)
30,000	10
15,000	10
7,500	10
3,750	10
2,000	10
1,000	20
500	40
100	40
60	40
≤ 50	80

NOTE: f_{CLKIN} = 7.68MHz.



关闭缓冲器后的输入阻抗：



PGA SETTING	$Zeff_A$ (kΩ)	$Zeff_B$ (kΩ)
1	260	220
2	130	110
4	65	55
8	33	28
16	16	14
32	8	7
64	8	7

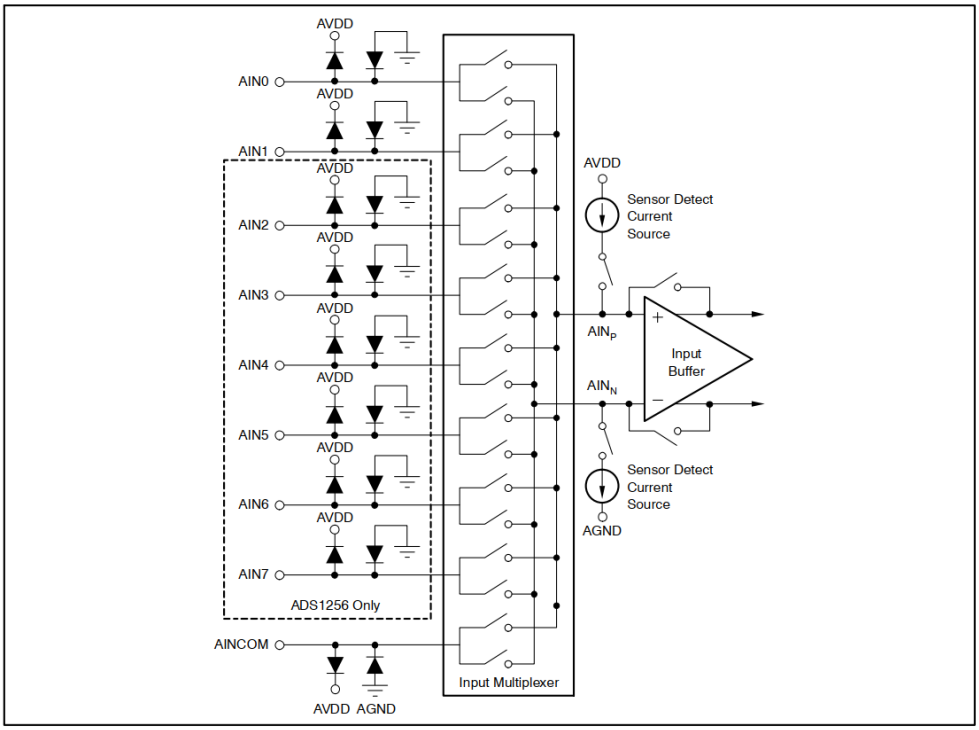
93.4.7 ADS1256 支持的采样率

ADS1256 支持的采样率如下，这里特别注意，因为切换通道和读数据耗时 123 微秒，因此扫描中断模式工作时，最大速率 = DRATE_1000SPS。

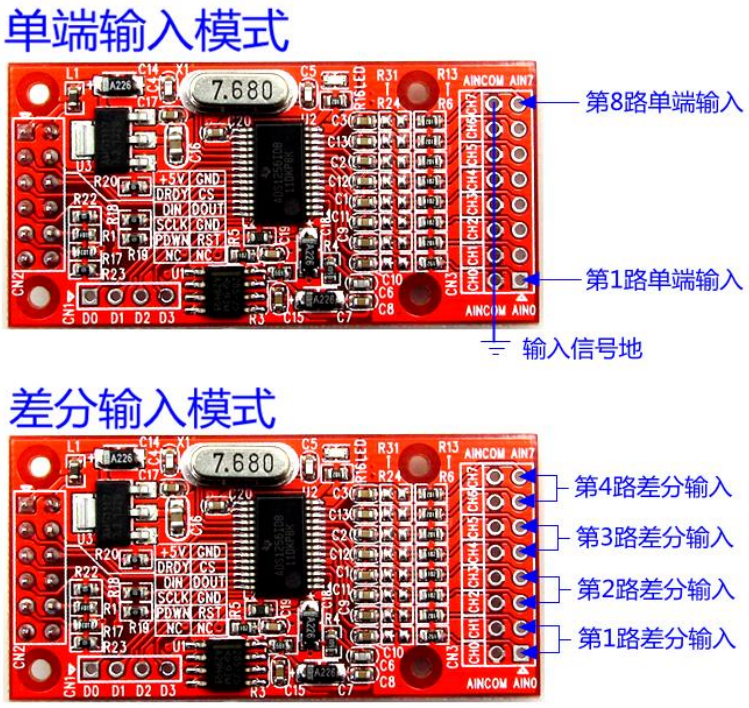
DRATE DR[7:0]	NUMBER OF AVERAGES FOR PROGRAMMABLE FILTER (Num_Ave)	DATA RATE(1) (SPS)
11110000	1 (averager bypassed)	30,000
11100000	2	15,000
11010000	4	7500
11000000	8	3750
10110000	15	2000
10100001	30	1000
10010010	60	500
10000010	300	100
01110010	500	60
01100011	600	50
01010011	1000	30
01000011	1200	25
00110011	2000	15
00100011	3000	10
00010011	6000	5
00000011	12,000	2.5

93.4.8 ADS1256 的多路选择器，单端和多端输入

ADS1256 的 8 路支持是通过多路选择器实现，8 路采样时是选择相应通道进行采样：

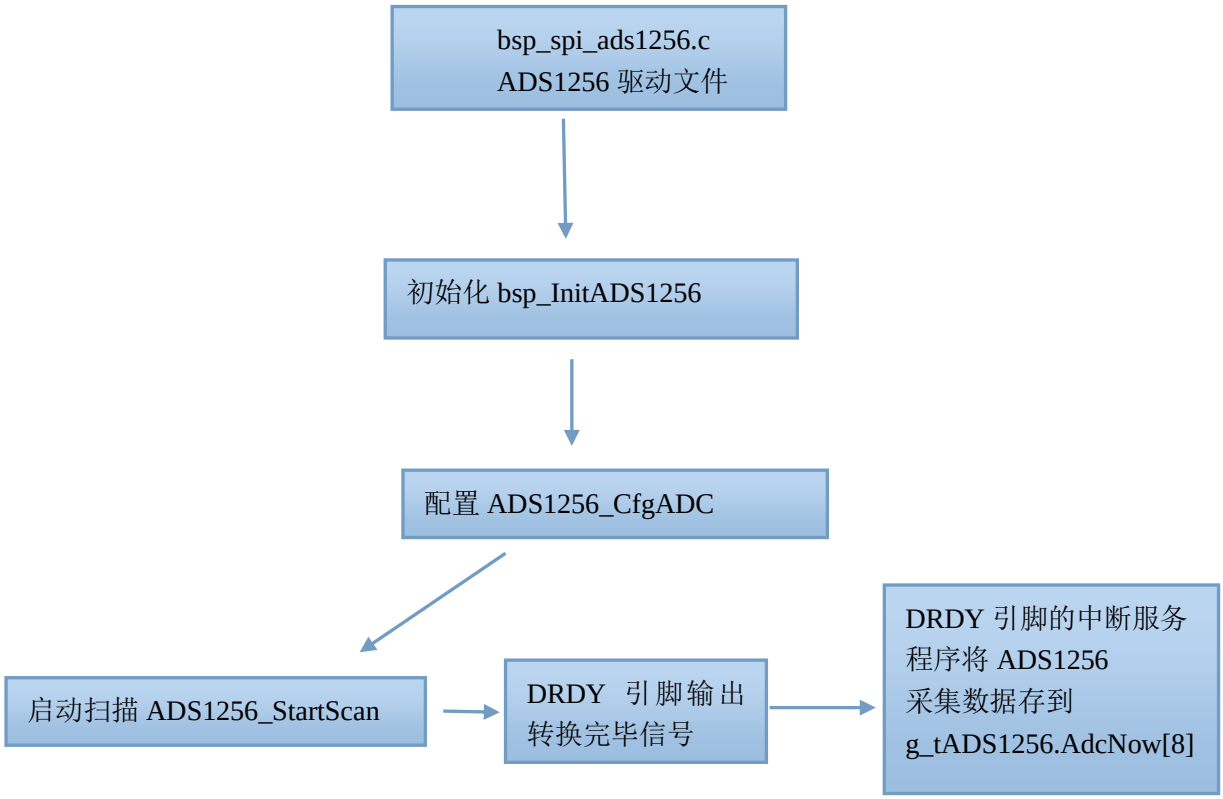


实际应用的的 8 路单端采样和 4 路差分采样效果如下：



93.5 ADS1256 驱动设计

ADS1256 的程序驱动框架设计如下：



有了这个框图，程序设计就比较好理解了。

93.5.1第 1 步，ADS1256 所涉及到的 GPIO 配置

这里需要把用到的 GPIO 时钟、GPIO 引脚配置好：

```
/*
    ADS1256 模块     STM32-V7 开发板(示波器接口)
      +5V   <------  5.0V      5V 供电
      GND   -------  GND       地
      DRDY  ------>  PC6       准备就绪
      CS    <------  PC7       SPI_CS
      DIN   <------  PG10      SPI_MOSI
      DOUT  ------>  PA5       SPI_MISO
      SCLK  <------  PA4       SPI 时钟
      GND   -------  GND       地
      PDWN  <------  PB7       掉电控制
      RST   <------  PC3       复位信号
      NC    空脚
      NC    空脚
*/

#ifdef SOFT_SPI         /* 软件 SPI */
    /* 定义 GPIO 端口 */
    #define SCK_CLK_ENABLE()    __HAL_RCC_GPIOA_CLK_ENABLE()
    #define SCK_GPIO            GPIOA
    #define SCK_PIN             GPIO_PIN_4
    #define SCK_1()             SCK_GPIO->BSRR = SCK_PIN
    #define SCK_0()             SCK_GPIO->BSRR = ((uint32_t)SCK_PIN << 16U)

    #define DIN_CLK_ENABLE()    __HAL_RCC_GPIOG_CLK_ENABLE()
    #define DIN_GPIO            GPIOG
    #define DIN_PIN             GPIO_PIN_10
    #define DIN_1()             DIN_GPIO->BSRR = DIN_PIN
    #define DIN_0()             DIN_GPIO->BSRR = ((uint32_t)DIN_PIN << 16U)

    #define CS_CLK_ENABLE()     __HAL_RCC_GPIOC_CLK_ENABLE()
    #define CS_GPIO             GPIOC
    #define CS_PIN              GPIO_PIN_7
    #define CS_1()              CS_GPIO->BSRR = CS_PIN
    #define CS_0()              CS_GPIO->BSRR = ((uint32_t)CS_PIN << 16U)

    #define DOUT_CLK_ENABLE()   __HAL_RCC_GPIOA_CLK_ENABLE()
    #define DOUT_GPIO           GPIOA
    #define DOUT_PIN            GPIO_PIN_5
    #define DOUT_IS_HIGH()      ((DOUT_GPIO->IDR & DOUT_PIN) != 0)

    #define DRDY_CLK_ENABLE()   __HAL_RCC_GPIOC_CLK_ENABLE()
    #define DRDY_GPIO           GPIOC
    #define DRDY_PIN            GPIO_PIN_6
    #define DRDY_IS_LOW()       ((DRDY_GPIO->IDR & DRDY_PIN) == 0)
    #define DRDY_IRQn           EXTI9_5_IRQn
    #define DRDY_IRQHandler     EXTI9_5_IRQHandler

    /* PDWN  <------  PB7       掉电控制 */
```

```
    #define PWDN_CLK_ENABLE()    __HAL_RCC_GPIOB_CLK_ENABLE()
    #define PWDN_GPIO            GPIOB
    #define PWDN_PIN             GPIO_PIN_7
    #define PWDN_1()             PWDN_GPIO->BSRR = PWDN_PIN
    #define PWDN_0()             PWDN_GPIO->BSRR = ((uint32_t)PWDN_PIN << 16U)

    /*  RST   <------  PC3       复位信号 */
    #define RST_CLK_ENABLE()     __HAL_RCC_GPIOC_CLK_ENABLE()
    #define RST_GPIO             GPIOC
    #define RST_PIN              GPIO_PIN_3
    #define RST_1()              RST_GPIO->BSRR = RST_PIN
    #define RST_0()              RST_GPIO->BSRR = ((uint32_t)RST_PIN << 16U)
#endif

/*
*********************************************************************************************************
*    函 数 名: bsp_InitADS1256
*    功能说明: 配置 STM32 的 GPIO 和 SPI 接口，用于连接 ADS1256
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitADS1256(void)
{
#ifdef SOFT_SPI
    GPIO_InitTypeDef gpio_init;

    RST_1();
    PWDN_1();
    CS_1();
    SCK_0();      /* SPI 总线空闲时，钟线是低电平 */
    DIN_1();

    /* 打开 GPIO 时钟 */
    SCK_CLK_ENABLE();
    DIN_CLK_ENABLE();
    CS_CLK_ENABLE();
    DOUT_CLK_ENABLE();
    DRDY_CLK_ENABLE();
    PWDN_CLK_ENABLE();
    RST_CLK_ENABLE();

    /* 配置几个推完输出 IO */
    gpio_init.Mode = GPIO_MODE_OUTPUT_PP;     /* 设置推挽输出 */
    gpio_init.Pull = GPIO_NOPULL;                 /* 上下拉电阻不使能 */
    gpio_init.Speed = GPIO_SPEED_FREQ_HIGH;   /* GPIO 速度等级 */

    gpio_init.Pin = SCK_PIN;
    HAL_GPIO_Init(SCK_GPIO, &gpio_init);

    gpio_init.Pin = DIN_PIN;
    HAL_GPIO_Init(DIN_GPIO, &gpio_init);

    gpio_init.Pin = CS_PIN;
    HAL_GPIO_Init(CS_GPIO, &gpio_init);

    gpio_init.Pin = PWDN_PIN;
    HAL_GPIO_Init(PWDN_GPIO, &gpio_init);
```

```
    /* DRDY 设置为输入 */
    gpio_init.Mode = GPIO_MODE_INPUT;          /* 设置输入 */
    gpio_init.Pull = GPIO_NOPULL;              /* 上下拉电阻不使能 */
    gpio_init.Speed = GPIO_SPEED_FREQ_HIGH;    /* GPIO 速度等级 */

    gpio_init.Pin = DRDY_PIN;
    HAL_GPIO_Init(DRDY_GPIO, &gpio_init);

    gpio_init.Pin = DOUT_PIN;
    HAL_GPIO_Init(DOUT_GPIO, &gpio_init);
#endif
}
```

这里重点注意 DRDY 转换就绪引脚的配置，DRDY_IRQn 和 DRDY_IRQHandler 不要配置错了。

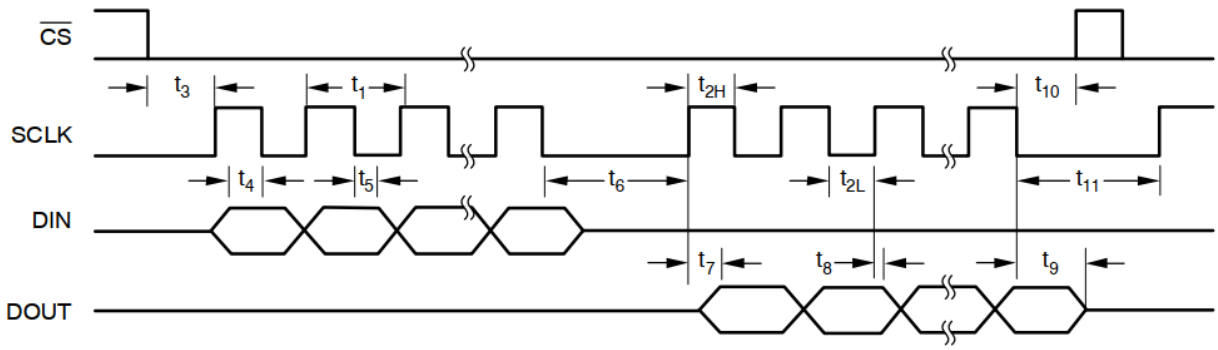
93.5.2第 2 步，ADS1256 的 8bit 读写函数实现

读写函数实现如下：

```
/*
*********************************************************************************************************
*    函 数 名: ADS1256_Send8Bit
*    功能说明: 向 SPI 总线发送 8 个 bit 数据。 不带 CS 控制。
*    形    参: _data : 数据
*    返 回 值: 无
*********************************************************************************************************
*/
static void ADS1256_Send8Bit(uint8_t _data)
{
    uint8_t i;

    /* 连续发送多个字节时，需要延迟一下 */
    ADS1256_DelaySCLK();
    ADS1256_DelaySCLK();

    /*  ADS1256 要求 SCL 高电平和低电平持续时间最小 200ns  */
    for(i = 0; i < 8; i++)
    {
        if (_data & 0x80)
        {
            DIN_1();
        }
        else
        {
            DIN_0();
        }
        SCK_1();
        ADS1256_DelaySCLK();
        _data <<= 1;
        SCK_0();           /* <----  ADS1256 是在 SCK 下降沿采样 DIN 数据，数据必须维持 50nS */
        ADS1256_DelaySCLK();
    }
}

/*
```

```
*********************************************************************************************************
*    函 数 名: ADS1256_Recive8Bit
*    功能说明: 从 SPI 总线接收 8 个 bit 数据。 不带 CS 控制。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static uint8_t ADS1256_Recive8Bit(void)
{
    uint8_t i;
    uint8_t read = 0;

    ADS1256_DelaySCLK();
    /* ADS1256 要求 SCL 高电平和低电平持续时间最小 200ns */
    for (i = 0; i < 8; i++)
    {
        SCK_1();
        ADS1256_DelaySCLK();
        read = read<<1;
        SCK_0();
        if (DOUT_IS_HIGH())
        {
            read++;
        }
        ADS1256_DelaySCLK();
    }
    return read;
}
```

读写的实现完全按照下面的时序实现：



这里主要注意时间实现：

t_1：SPI 时钟周期：

外部晶振频率 = 7.68MHz,

时钟频率 tCLK = 1/7.68M = 0.13uS

输出数据周期 tDATA = 1 / 30K = 0.033mS (按 30Ksps 计算)

对 SPI 的时钟速度要求:

最快 4 个 tCLK = 0.52uS

最慢 10 个 tDATA = 0.3mS (按 30Ksps 计算)

SYMBOL	DESCRIPTION	MIN	MAX	UNIT
t_1	SCLK period	4		τ_{CLKIN}(1)
			10	τ_{DATA}(2)

t_{2H}，t_{2L} ：脉冲高低电平：

SCL 高电平和低电平持续时间最小 200ns。

SYMBOL	DESCRIPTION	MIN	MAX	UNIT
t_1	SCLK period	4		τ_{CLKIN}(1)
			10	τ_{DATA}(2)
t_{2H}	SCLK pulse width: high	200		ns
			9	τ_{DATA}
t_{2L}	SCLK pulse width: low	200		ns

93.5.3第 3 步，ADS1256 的 24bit ADC 数据读取

实现代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: ADS1256_ReadData
*    功能说明: 读 ADC 数据
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static int32_t ADS1256_ReadData(void)
{
    uint32_t read = 0;

    CS_0();  /* SPI 片选 = 0 */

    ADS1256_Send8Bit(CMD_RDATA);     /* 读数据的命令 */

    ADS1256_DelayDATA();    /* 必须延迟才能读取芯片返回数据 */

    /* 读采样结果，3 个字节，高字节在前 */
    read = ADS1256_Recive8Bit() << 16;
    read += ADS1256_Recive8Bit() << 8;
    read += ADS1256_Recive8Bit() << 0;

    CS_1();  /* SPI 片选 = 1 */

    /* 负数进行扩展。24 位有符号数扩展为 32 位有符号数 */
    if (read & 0x800000)
    {
        read += 0xFF000000;
    }

    return (int32_t)read;
}
```

这段代码里面关键是 24bit 数据的补码处理。对负数进行扩展，24 位有符号数扩展为 32 位有符号数。

93.5.4第 3 步，ADS1256 增益和采样率配置

代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: ADS1256_CfgADC
*    功能说明: 配置 ADC 参数，增益和数据输出速率
*    形    参: _gain : 支持增益参数。
*                    ADS1256_GAIN_1
*                    ADS1256_GAIN_2
*                    ADS1256_GAIN_4
*                    ADS1256_GAIN_8
*                    ADS1256_GAIN_16
*                    ADS1256_GAIN_32
*                    ADS1256_GAIN_64
*
*              _drate : 数据输出速率，不推荐超过 1000SPS
*                     ADS1256_30000SPS
*                     ADS1256_15000SPS
*                     ADS1256_7500SPS
*                     ADS1256_3750SPS
*                     ADS1256_2000SPS
*                     ADS1256_1000SPS
*                     ADS1256_500SPS
*                     ADS1256_100SPS
*                     ADS1256_60SPS
*                     ADS1256_50SPS
*                     ADS1256_30SPS
*                     ADS1256_25SPS
*                     ADS1256_15SPS
*                     ADS1256_10SPS
*                     ADS1256_5SPS
*                     ADS1256_2d5SPS
*    返 回 值: 无
*********************************************************************************************************
*/
void ADS1256_CfgADC(ADS1256_GAIN_E _gain, ADS1256_DRATE_E _drate)
{
    g_tADS1256.Gain = _gain;
    g_tADS1256.DataRate = _drate;

    ADS1256_StopScan();             /* 暂停 CPU 中断 */

    ADS1256_ResetHard();        /* 硬件复位 */

    ADS1256_WaitDRDY();

    {
        uint8_t buf[4];         /* 暂存 ADS1256 寄存器配置参数，之后连续写 4 个寄存器 */

        buf[0] = (0 << 3) | (1 << 2) | (1 << 1);

        buf[1] = 0x08;      /* 高四位 0 表示 AINP 接 AIN0,  低四位 8 表示 AINN 固定接 AINCOM */

        buf[2] = (0 << 5) | (0 << 2) | (_gain << 1);
```

```
        /* 因为切换通道和读数据耗时 123uS，因此扫描中断模式工作时，最大速率 = DRATE_1000SPS */
        buf[3] = s_tabDataRate[_drate];  // DRATE_10SPS;    /* 选择数据输出速率 */

        CS_0();  /* SPI 片选 = 0 */
        ADS1256_Send8Bit(CMD_WREG | 0);  /* 写寄存器的命令，并发送寄存器地址 */
        ADS1256_Send8Bit(0x03);          /* 寄存器个数 - 1，此处 3 表示写 4 个寄存器 */

        ADS1256_Send8Bit(buf[0]);   /* 设置状态寄存器 */
        ADS1256_Send8Bit(buf[1]);   /* 设置输入通道参数 */
        ADS1256_Send8Bit(buf[2]);   /* 设置 ADCON 控制寄存器，增益 */
        ADS1256_Send8Bit(buf[3]);   /* 设置输出数据速率 */

        CS_1();  /* SPI 片选 = 1 */
    }

    bsp_DelayUS(50);
}
```

这个函数主要配置了 4 个 ADS1256 寄存器。

◆ **设置状态寄存器：**

STATUS : STATUS REGISTER (ADDRESS 00h)

Reset Value = x1h

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
ID	ID	ID	ID	ORDER	ACAL	BUFEN	$\overline{\text{DRDY}}$

Bits 7-4 **ID3, ID2, ID1, ID0** Factory Programmed Identification Bits (Read Only)

Bit 3 **ORDER:** Data Output Bit Order
0 = Most Significant Bit First (default)
1 = Least Significant Bit First
Input data is always shifted in most significant byte and bit first. Output data is always shifted out most significant byte first. The ORDER bit only controls the bit order of the output data within the byte.

Bit 2 **ACAL**: Auto-Calibration
0 = Auto-Calibration Disabled (default)
1 = Auto-Calibration Enabled
When Auto-Calibration is enabled, self-calibration begins at the completion of the WREG command that changes the PGA (bits 0-2 of ADCON register), DR (bits 7-0 in the DRATE register) or BUFEN (bit 1 in the STATUS register) values.

Bit 1 **BUFEN:** Analog Input Buffer Enable
0 = Buffer Disabled (default)
1 = Buffer Enabled

Bit 0 **$\overline{\text{DRDY}}$:** Data Ready (Read Only)
This bit duplicates the state of the $\overline{\text{DRDY}}$ pin.

程序中的配置为：buf[0] = (0 << 3) | (1 << 2) | (1 << 1) ，意思是 LSB 传输，自动校准，使能模拟输入缓冲。

◆ **设置输入通道参数**

MUX : Input Multiplexer Control Register (Address 01h)

Reset Value = 01h

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
PSEL3	PSEL2	PSEL1	PSEL0	NSEL3	NSEL2	NSEL1	NSEL0

Bits 7-4 **PSEL3, PSEL2, PSEL1, PSEL0**: Positive Input Channel (AIN_P) Select

0000 = AIN0 (default)
0001 = AIN1
0010 = AIN2 (ADS1256 only)
0011 = AIN3 (ADS1256 only)
0100 = AIN4 (ADS1256 only)
0101 = AIN5 (ADS1256 only)
0110 = AIN6 (ADS1256 only)
0111 = AIN7 (ADS1256 only)
1xxx = AINCOM (when PSEL3 = 1, PSEL2, PSEL1, PSEL0 are “don’t care”)

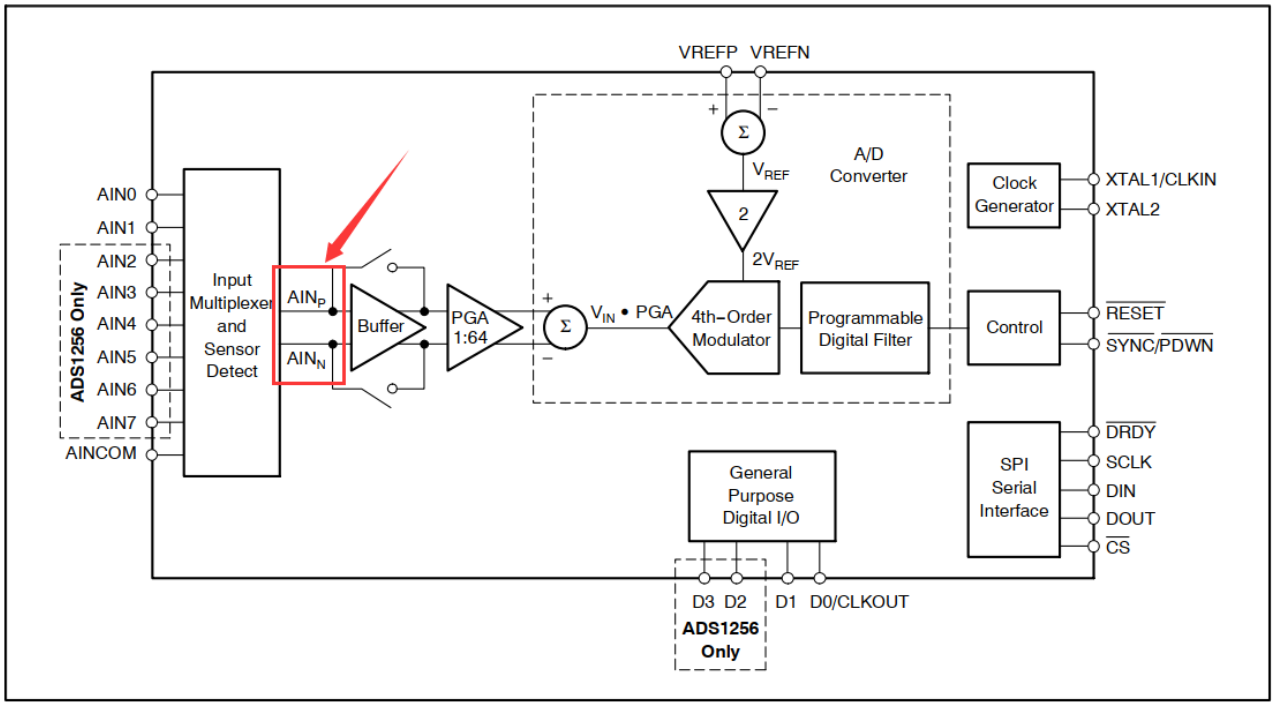
NOTE: When using an ADS1255 make sure to only select the available inputs.

Bits 3-0 **NSEL3, NSEL2, NSEL1, NSEL0**: Negative Input Channel (AIN_N)Select

0000 = AIN0
0001 = AIN1 (default)
0010 = AIN2 (ADS1256 only)
0011 = AIN3 (ADS1256 only)
0100 = AIN4 (ADS1256 only)
0101 = AIN5 (ADS1256 only)
0110 = AIN6 (ADS1256 only)
0111 = AIN7 (ADS1256 only)
1xxx = AINCOM (when NSEL3 = 1, NSEL2, NSEL1, NSEL0 are “don’t care”)

NOTE: When using an ADS1255 make sure to only select the available inputs.

程序中的配置为：buf[1] = 0x08， 高四位 0 表示 AINp 接通的 AIN0， 低四位 8 表示 AINn 接通的 AINCOM。AINp 和 AINn 表示当前的多路选择器选通的 AIN0:



◆ 设置 ADCON 控制寄存器，主要用于增益设置

ADCON: A/D Control Register (Address 02h)

Reset Value = 20h

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
0	CLK1	CLK0	SDCS1	SDCS0	PGA2	PGA1	PGA0

Bit 7 Reserved, always 0 (Read Only)

Bits 6-5 **CLK1, CLK0**: D0/CLKOUT Clock Out Rate Setting
00 = Clock Out OFF
01 = Clock Out Frequency = f_{CLKIN} (default)
10 = Clock Out Frequency = $f_{CLKIN}/2$
11 = Clock Out Frequency = $f_{CLKIN}/4$
When not using CLKOUT, it is recommended that it be turned off. These bits can only be reset using the $\overline{RESET}$ pin.

Bits 4-2 **SDCS1, SCDS0**: Sensor Detect Current Sources
00 = Sensor Detect OFF (default)
01 = Sensor Detect Current = 0.5µA
10 = Sensor Detect Current = 2µA
11 = Sensor Detect Current = 10µA

The Sensor Detect Current Sources can be activated to verify the integrity of an external sensor supplying a signal to the ADS1255/6. A shorted sensor produces a very small signal while an open-circuit sensor produces a very large signal.

Bits 2-0 **PGA2, PGA1, PGA0**: Programmable Gain Amplifier Setting
000 = 1 (default)
001 = 2
010 = 4
011 = 8
100 = 16
101 = 32
110 = 64
111 = 64

程序中的配置为：buf[2] = (0 << 5) | (0 << 2) | (_gain << 1)， 意思是关闭 CLKOUT 引脚输出，关闭传感器检测，设置增益为形参_gain。

◆ 设置 ADC 采样率

DRATE: A/D Data Rate (Address 03h)

Reset Value = F0h

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
DR7	DR6	DR5	DR4	DR3	DR2	DR1	DR0

The 16 valid Data Rate settings are shown below. Make sure to select a valid setting as the invalid settings may produce unpredictable results.

Bits 7-0 **DR[7: 0]**: Data Rate Setting(1)
11110000 = 30,000SPS (default)
11100000 = 15,000SPS
11010000 = 7,500SPS
11000000 = 3,750SPS
10110000 = 2,000SPS
10100001 = 1,000SPS
10010010 = 500SPS
10000010 = 100SPS
01110010 = 60SPS
01100011 = 50SPS
01010011 = 30SPS
01000011 = 25SPS
00110011 = 15SPS
00100011 = 10SPS
00010011 = 5SPS
00000011 = 2.5SPS

(1) for f_{CLKIN} = 7.68MHz. Data rates scale linearly with f_{CLKIN}.

程序中的配置为：buf[3] = s_tabDataRate[_drate]，用于设置波特率：

```
static const uint8_t s_tabDataRate[ADS1256_DRATE_MAX] =
{
	0xF0,		/* 复位时缺省值 */
	0xE0,
	0xD0,
	0xC0,
	0xB0,
	0xA1,
	0x92,
	0x82,
	0x72,
	0x63,
	0x53,
	0x43,
	0x33,
	0x20,
	0x13,
	0x03
};
typedef enum
{
	ADS1256_30000SPS = 0,
	ADS1256_15000SPS,
	ADS1256_7500SPS,
	ADS1256_3750SPS,
	ADS1256_2000SPS,
	ADS1256_1000SPS,
	ADS1256_500SPS,
	ADS1256_100SPS,
	ADS1256_60SPS,
	ADS1256_50SPS,
	ADS1256_30SPS,
	ADS1256_25SPS,
	ADS1256_15SPS,
	ADS1256_10SPS,
	ADS1256_5SPS,
	ADS1256_2d5SPS,

	ADS1256_DRATE_MAX
}ADS1256_DRATE_E;
```

93.5.5第 4 步，ADS1256 启动采样

代码实现如下

```
/*
*********************************************************************************************************
*	函 数 名: ADS1256_StartScan
*	功能说明: 将 DRDY 引脚 （PC6 ）配置成外部中断触发方式， 中断服务程序中扫描 8 个通道的数据。
*	形	参: 无
*	返 回 值: 无
*********************************************************************************************************
*/
void ADS1256_StartScan(void)
```

```
{
    /* PC6 外部中断，BUSY
        配置 BUSY 作为中断输入口，下降沿触发 */
    {
        GPIO_InitTypeDef   GPIO_InitStructure;

        DRDY_CLK_ENABLE(); /* 打开 GPIO 时钟 */

        GPIO_InitStructure.Mode = GPIO_MODE_IT_FALLING;
        GPIO_InitStructure.Pull = GPIO_NOPULL;
        GPIO_InitStructure.Pin = DRDY_PIN;
        HAL_GPIO_Init(DRDY_GPIO, &GPIO_InitStructure);

        HAL_NVIC_SetPriority(DRDY_IRQn, 2, 0);
        HAL_NVIC_EnableIRQ(DRDY_IRQn);
    }

    /* 开始扫描前，清零结果缓冲区 */
    {
        uint8_t i;

        g_tADS1256.Channel = 0;

        for (i = 0; i < 8; i++)
        {
            g_tADS1256.AdcNow[i] = 0;
        }
    }
}
```

代码比较简单，主要是配置 PC6 的 EXTI 外部中断，并初始化变量。

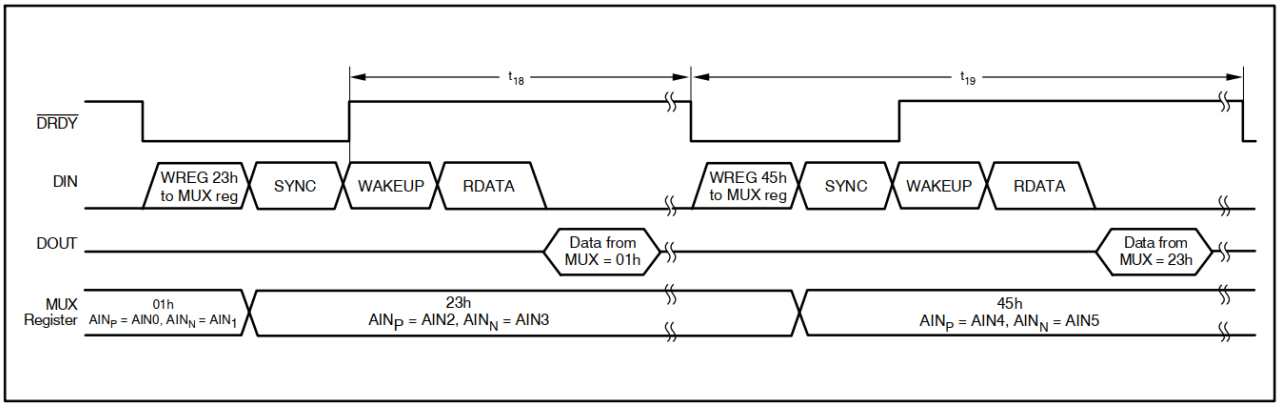
93.5.6第 5 步，ADS1256 的中断处理（8 通道数据读取）

代码如下：

```
/*
*********************************************************************************************************
*    函 数 名: EXTI9_5_IRQHandler
*    功能说明: 外部中断服务程序.  此程序执行时间约 123uS
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
#ifdef EXTI9_5_ISR_MOVE_OUT      /* bsp.h 中定义此行，表示本函数移到 stam32f4xx_it.c。 避免重复定义 */
void EXTI9_5_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_6);
}

/*
*********************************************************************************************************
*    函 数 名: EXTI9_5_IRQHandler
*    功能说明: 外部中断服务程序入口。PI6 / AD7606_BUSY 下降沿中断触发
*    形    参: 无
```

```
*    返 回 值: 无
*********************************************************************************************************
*/
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_6)
    {

        ADS1256_ISR();
    }
}
#endif

/*
*********************************************************************************************************
*    函 数 名: ADS1256_ISR
*    功能说明: 定时采集中断服务程序
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void ADS1256_ISR(void)
{
    /* 读取采集结构，保存在全局变量 */
    ADS1256_SetChannal(g_tADS1256.Channel);   /* 切换模拟通道 */
    bsp_DelayUS(5);

    ADS1256_WriteCmd(CMD_SYNC);
    bsp_DelayUS(5);

    ADS1256_WriteCmd(CMD_WAKEUP);
    bsp_DelayUS(25);

    if (g_tADS1256.Channel == 0)
    {
        g_tADS1256.AdcNow[7] = ADS1256_ReadData();     /* 注意保存的是上一个通道的数据 */
    }
    else
    {
        g_tADS1256.AdcNow[g_tADS1256.Channel-1] = ADS1256_ReadData(); /* 注意保存的是上一个通道的数据 */
    }

    if (++g_tADS1256.Channel >= 8)
    {
        g_tADS1256.Channel = 0;
    }
}
```

中断服务程序的代码完全是按照下面的时序时序：



- 第 1 步：ADS1256_SetChannal(g_tADS1256.Channel)切换模拟通道。
- 第 2 步：发送 SYNC 同步命令。
- 第 3 步：唤醒。
- 第 4 步：读取数据。

  这个过程中特别注意读取的是上次转换的数据。

93.6 ADS1256 板级支持包（bsp_spi_ads1256.c）

ADS1256 驱动文件 bsp_spi_ads1256.c 主要实现了如下几个 API 供用户调用：

- bsp_InitADS1256
- ADS1256_CfgADC
- ADS1256_StartScan
- ADS1256_SetChannal
- ADS1256_SetDiffChannal

93.6.1 函数 bsp_InitADS1256

函数原型：

```
void bsp_InitADS1256(void)
```

函数描述：

主要用于 ADS1256 的初始化。

93.6.2 函数 ADS1256_CfgADC

函数原型：

```
void ADS1256_CfgADC(ADS1256_GAIN_E _gain, ADS1256_DRATE_E _drate)
```

函数描述：

用于配置 ADS1256 的增益和采样率。

函数参数:

- 第 1 个参数用于设置增益，支持的参数如下:

  ADS1256_GAIN_1
  ADS1256_GAIN_2
  ADS1256_GAIN_4
  ADS1256_GAIN_8
  ADS1256_GAIN_16
  ADS1256_GAIN_32
  ADS1256_GAIN_64

- 第 2 个参数用于设置采样率，支持的参数如下:

  ADS1256_30000SPS
  ADS1256_15000SPS
  ADS1256_7500SPS
  ADS1256_3750SPS
  ADS1256_2000SPS
  ADS1256_1000SPS
  ADS1256_500SPS
  ADS1256_100SPS
  ADS1256_60SPS
  ADS1256_50SPS
  ADS1256_30SPS
  ADS1256_25SPS
  ADS1256_15SPS
  ADS1256_10SPS
  ADS1256_5SPS
  ADS1256_2d5SPS

93.6.3函数 ADS1256_StartScan

函数原型:

```
void ADS1256_StartScan(void)
```

函数描述:

此函数用于启动扫描，采样的中断方式。

93.6.4函数 ADS1256_SetChannal

函数原型：

```
static void ADS1256_SetChannal(uint8_t _ch)
```

函数描述：

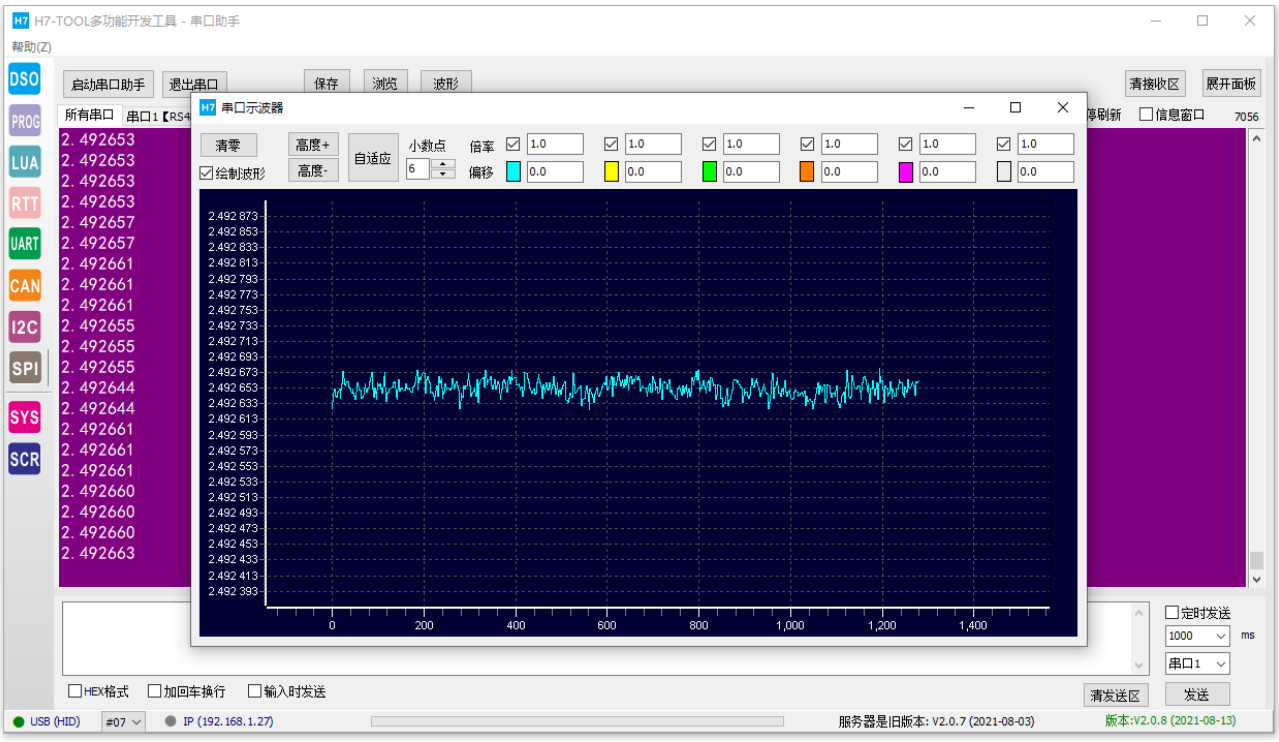
此函数用于设置单端采样的通道。

函数参数：

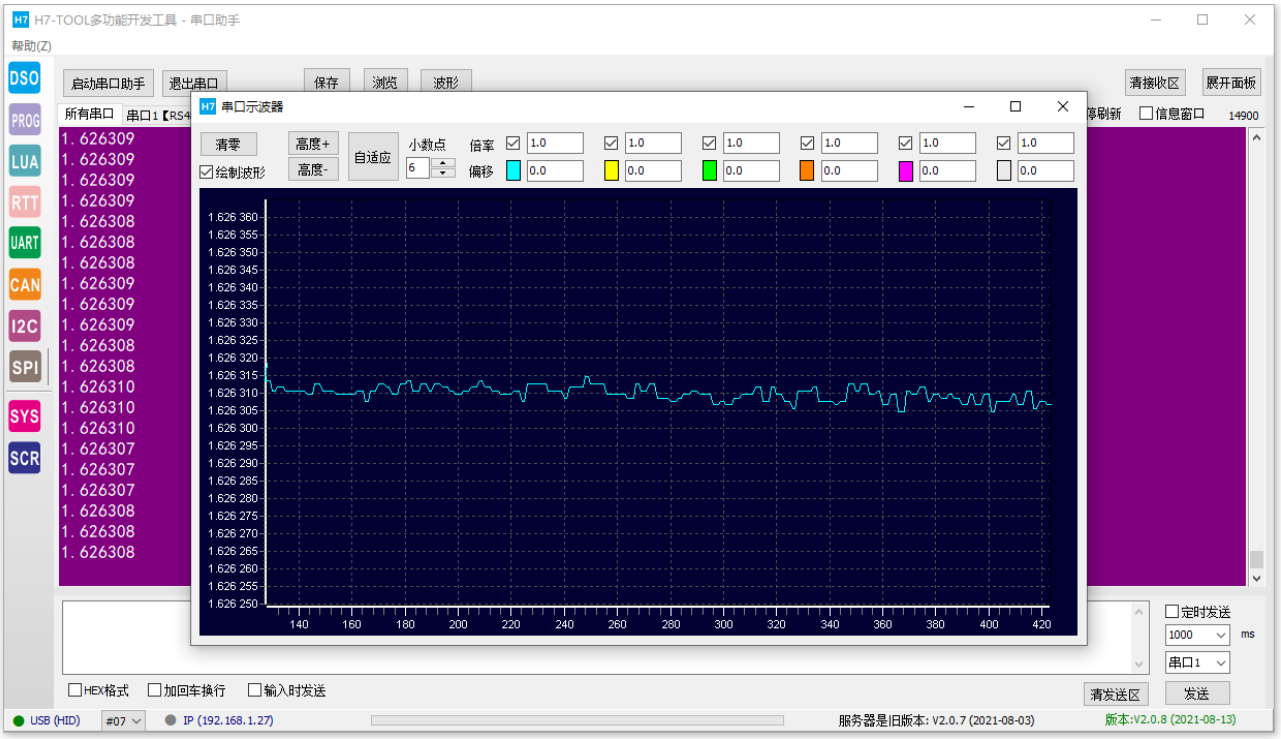
- 第 1 个参数支持 0 到 7，0 表示采样的通道 0, 1 表示采样的通道 1，依次类推，范围 0-7，共 8 个通道。

93.7 ADS1256 实际测量效果（10uV 抖动）

测试 LM285-2.5V 稳压效果，抖动 40uV：



测试干电池效果，抖动 10uV 左右，注意，这个级别的抖动容易受环境温度的影响，特别是开关空调，最明显。



93.8 ADS1256 驱动移植和使用

移植步骤如下：

- 第 1 步：复制 bsp_spi_ads1256.c 和 bsp_spi_ads1256.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：根据使用的 SPI 引脚，DRDY 就绪引脚，RST 复位引脚，修改 bsp_spi_ads1256.c 开头的宏定义。

```
/* 定义 GPIO 端口 */
#define SCK_CLK_ENABLE()     __HAL_RCC_GPIOA_CLK_ENABLE()
#define SCK_GPIO             GPIOA
#define SCK_PIN                  GPIO_PIN_4
#define SCK_1()                  SCK_GPIO->BSRR = SCK_PIN
#define SCK_0()                  SCK_GPIO->BSRR = ((uint32_t)SCK_PIN << 16U)

#define DIN_CLK_ENABLE()     __HAL_RCC_GPIOG_CLK_ENABLE()
#define DIN_GPIO             GPIOG
#define DIN_PIN                  GPIO_PIN_10
#define DIN_1()                  DIN_GPIO->BSRR = DIN_PIN
#define DIN_0()                  DIN_GPIO->BSRR = ((uint32_t)DIN_PIN << 16U)

#define CS_CLK_ENABLE()      __HAL_RCC_GPIOC_CLK_ENABLE()
#define CS_GPIO                  GPIOC
#define CS_PIN                   GPIO_PIN_7
#define CS_1()                   CS_GPIO->BSRR = CS_PIN
#define CS_0()                   CS_GPIO->BSRR = ((uint32_t)CS_PIN << 16U)

#define DOUT_CLK_ENABLE()    __HAL_RCC_GPIOA_CLK_ENABLE()
#define DOUT_GPIO            GPIOA
#define DOUT_PIN             GPIO_PIN_5
#define DOUT_IS_HIGH()       ((DOUT_GPIO->IDR & DOUT_PIN) != 0)
```

```
#define DRDY_CLK_ENABLE()    __HAL_RCC_GPIOC_CLK_ENABLE()
#define DRDY_GPIO            GPIOC
#define DRDY_PIN             GPIO_PIN_6
#define DRDY_IS_LOW()        ((DRDY_GPIO->IDR & DRDY_PIN) == 0)
#define DRDY_IRQn            EXTI9_5_IRQn
#define DRDY_IRQHandler      EXTI9_5_IRQHandler

/* PDWN  <------  PB7       掉电控制 */
#define PWDN_CLK_ENABLE()    __HAL_RCC_GPIOB_CLK_ENABLE()
#define PWDN_GPIO            GPIOB
#define PWDN_PIN             GPIO_PIN_7
#define PWDN_1()             PWDN_GPIO->BSRR = PWDN_PIN
#define PWDN_0()             PWDN_GPIO->BSRR = ((uint32_t)PWDN_PIN << 16U)

/*  RST   <------  PC3       复位信号 */
#define RST_CLK_ENABLE()     __HAL_RCC_GPIOC_CLK_ENABLE()
#define RST_GPIO             GPIOC
#define RST_PIN                  GPIO_PIN_3
#define RST_1()                  RST_GPIO->BSRR = RST_PIN
#define RST_0()                  RST_GPIO->BSRR = ((uint32_t)RST_PIN << 16U)
```

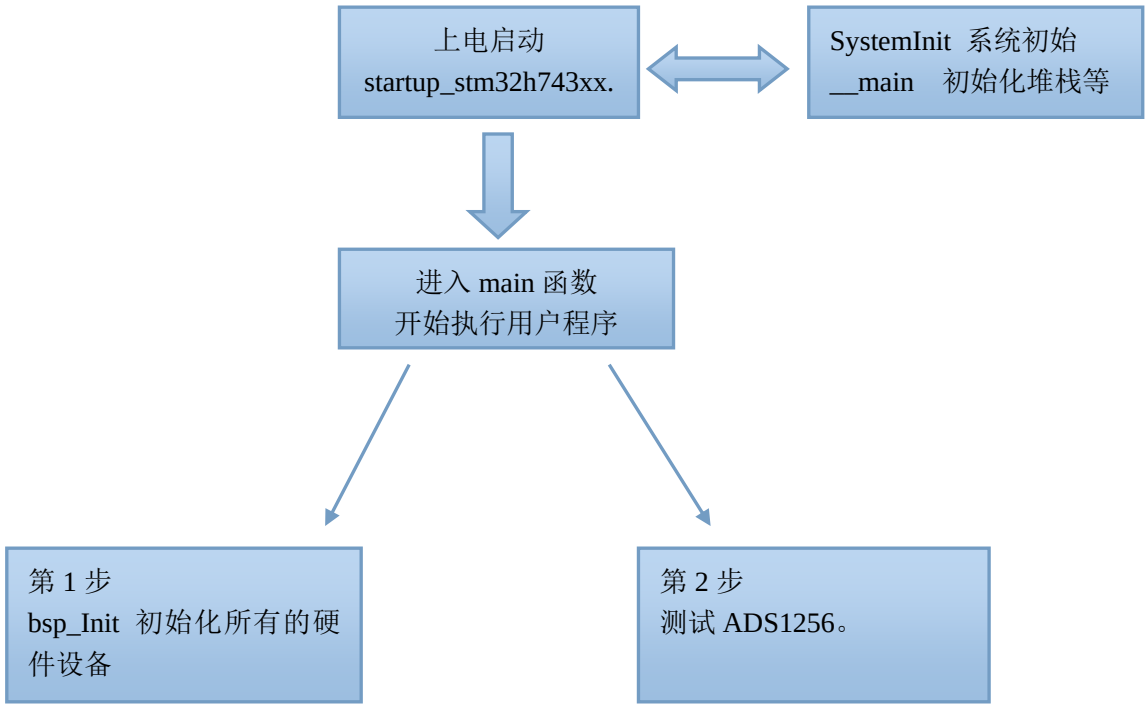
◆ 第 3 步：特别注意中断服务程序的入口要根据使用的 DRDY 引脚修改。

```
/*
*********************************************************************************************************
*    函 数 名: EXTI9_5_IRQHandler
*    功能说明: 外部中断服务程序.  此程序执行时间约 123uS
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
#ifdef EXTI9_5_ISR_MOVE_OUT      /* bsp.h 中定义此行，表示本函数移到 stam32f4xx_it.c。 避免重复定义 */
void EXTI9_5_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_6);
}

/*
*********************************************************************************************************
*    函 数 名: EXTI9_5_IRQHandler
*    功能说明: 外部中断服务程序入口。PI6 / AD7606_BUSY 下降沿中断触发
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_6)
    {

        ADS1256_ISR();
    }
}
#endif
```

◆ 第 4 步：应用方法看本章节配套例子即可。

93.9 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器和 LED。
  - 第 2 部分，应用程序设计部分，测试 ADS1256。

93.10 实验例程说明（MDK）

配套例子：

V7-068_ADS1256（8 通道带 PGA 的 24 位 ADC）

实验目的：

2. 学习 ADS1256, 8 通道带 PGA 的 24bit ADC。

重要提示：

6. 开发板请使用外置电源供电。

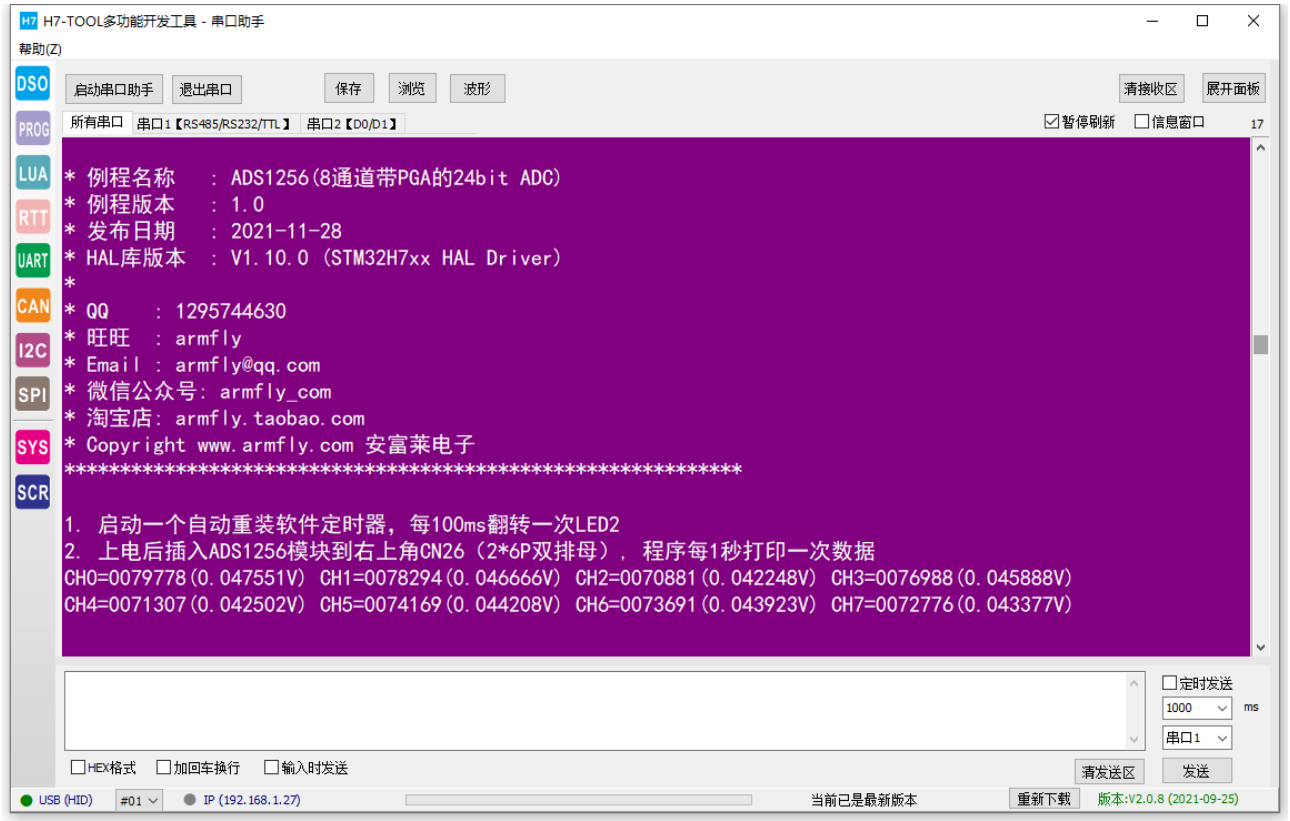
实验内容：

12. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
13. 上电后插入 ADS1256 模块到右上角 CN26（2*6P 双排母），程序每 1 秒打印一次 8 通道采样数

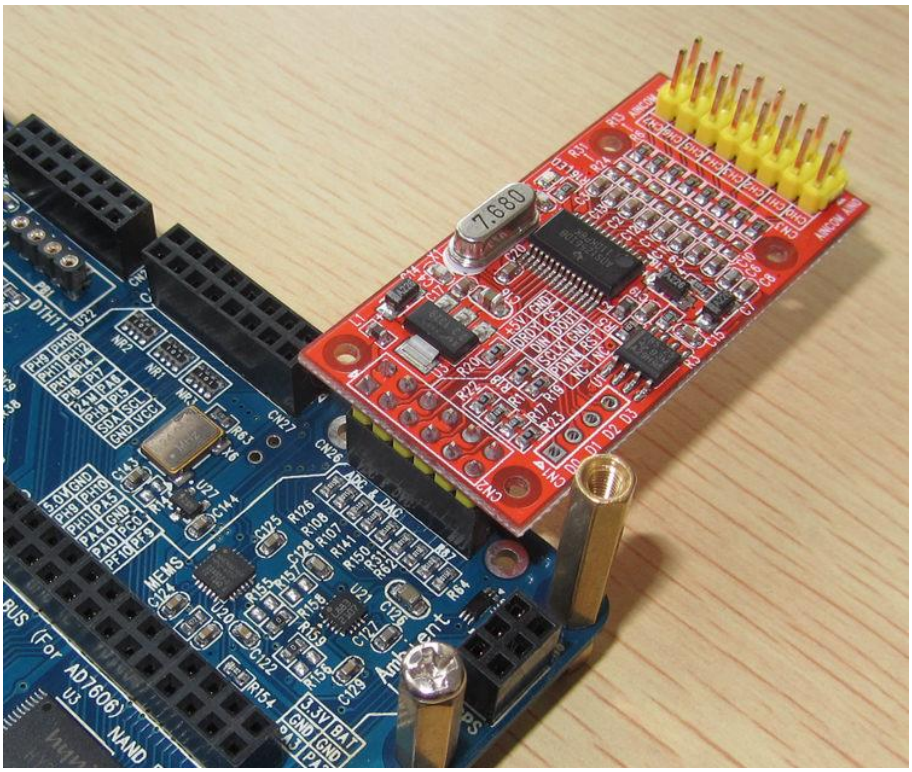
据。

上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1。

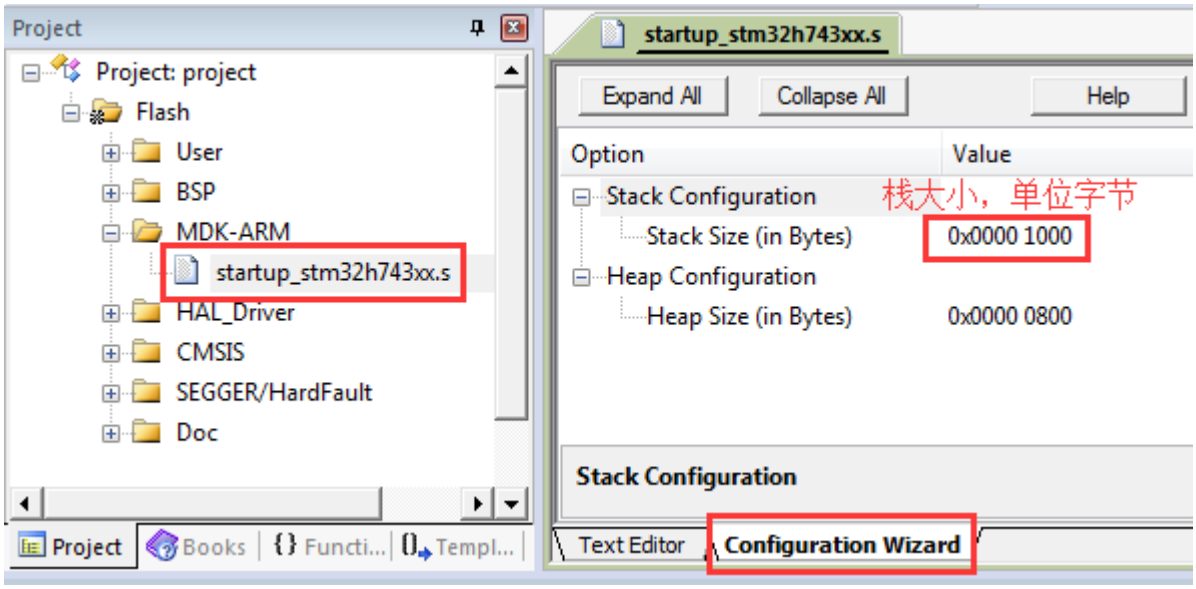


模块插入位置：

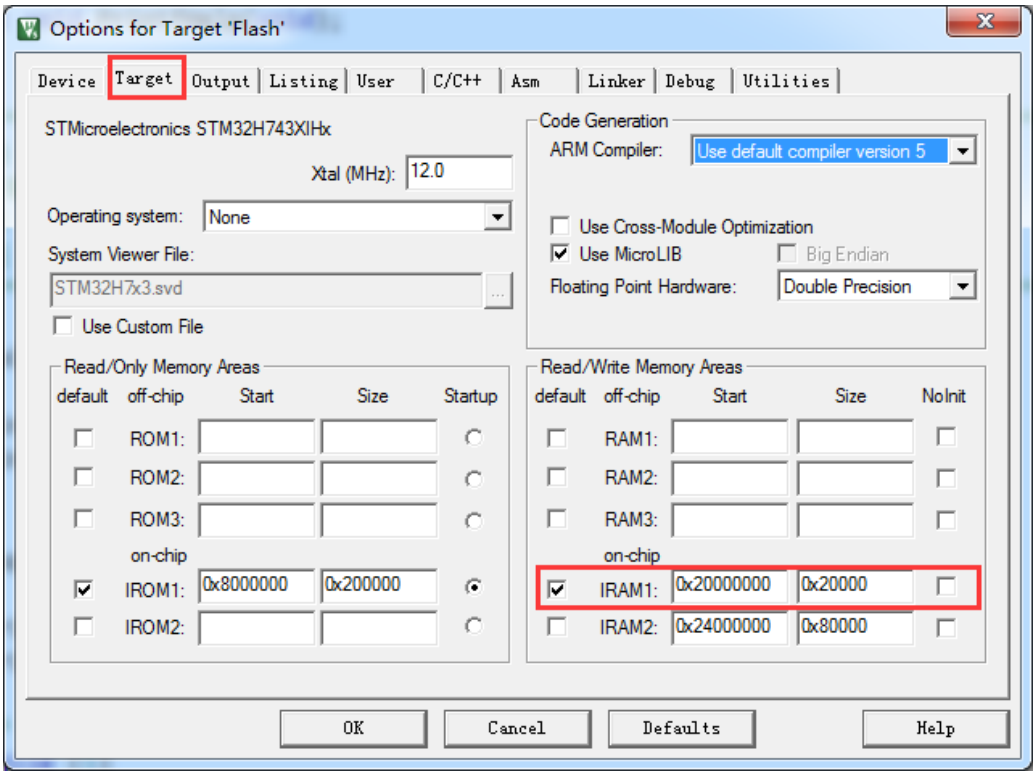


程序设计：

◆ 系统栈大小分配：



◆ RAM 空间用的 DTCM：



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
```

```
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();      /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();    /* 初始化滴答定时器 */
    bsp_InitLPUart();   /* 初始化串口 */
    bsp_InitExtIO();    /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();      /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */
}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
```

```
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **每 10ms 调用一次按键处理：**

按键处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能：**

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- 上电后插入 ADS1256 模块到右上角 CN26（2*6P 双排母），程序每 1 秒打印一次 8 通道采样数据。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t i;
    int32_t iTemp;
    float fTemp;


    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程信息到串口 1 */

    PrintfHelp(); /* 打印操作提示信息 */


    bsp_DelayMS(500);  /* 等待上电稳定，等基准电压电路稳定，bsp_InitADS1256() 内部会进行自校准 */

    bsp_InitADS1256(); /* 初始化配置 ADS1256.  PGA=1, DRATE=30KSPS, BUFEN=1, 输入正负 5V */


    /* 打印芯片 ID (通过读 ID 可以判断硬件接口是否正常)，正常时状态寄存器的高 4bit = 3 */
#if 0
    {
        uint8_t id;

        id = ADS1256_ReadChipID();
```

```
        if (id != 3)
        {
            printf("Error, ASD1256 Chip ID = 0x%X\r\n", id);
        }
        else
        {
            printf("Ok, ASD1256 Chip ID = 0x%X\r\n", id);
        }
    }
#endif

    ADS1256_CfgADC(ADS1256_GAIN_1, ADS1256_30SPS); /* 配置 ADC 参数： 增益 1:1, 数据输出速率 30Hz */

/* 启动中断扫描模式, 轮流采集 8 个通道的 ADC 数据. 通过 ADS1256_GetAdc() 函数来读取这些数据 */
    ADS1256_StartScan();

    bsp_StartAutoTimer(0, 1000);     /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */


        if (bsp_CheckTimer(0)) /* 判断定时器超时时间 */
        {
            /* 每隔 1000ms 进来一次 */
            bsp_LedToggle(2);  /* 翻转 LED 的状态 */

            /* 打印采集数据 */
            for (i = 0; i < 8; i++)
            {

                /*
                  计算公式：2 * VREF/(PGA * (2^23 - 1)) ，这里 VREF 是 2.5V，PGA = 1
                */
                 /* 计算实际电压值（近似估算的），如需准确，请进行校准 */
                 iTemp = ((int64_t)g_tADS1256.AdcNow[i] * 2500000) / 4194303;


                 fTemp = (float)iTemp / 1000000;

                 printf("CH%d=%07d(%fV) ", i, g_tADS1256.AdcNow[i], fTemp);

                 if(i == 3)
                 {
                     printf("\r\n");
                 }
            }

            printf("\r\n\r\n");
        }
    }
}
```

93.11 实验例程说明（IAR）

配套例子：

V7-068_ADS1256（8 通道带 PGA 的 24 位 ADC）

实验目的：

1. 学习 ADS1256, 8 通道带 PGA 的 24bit ADC。

重要提示：

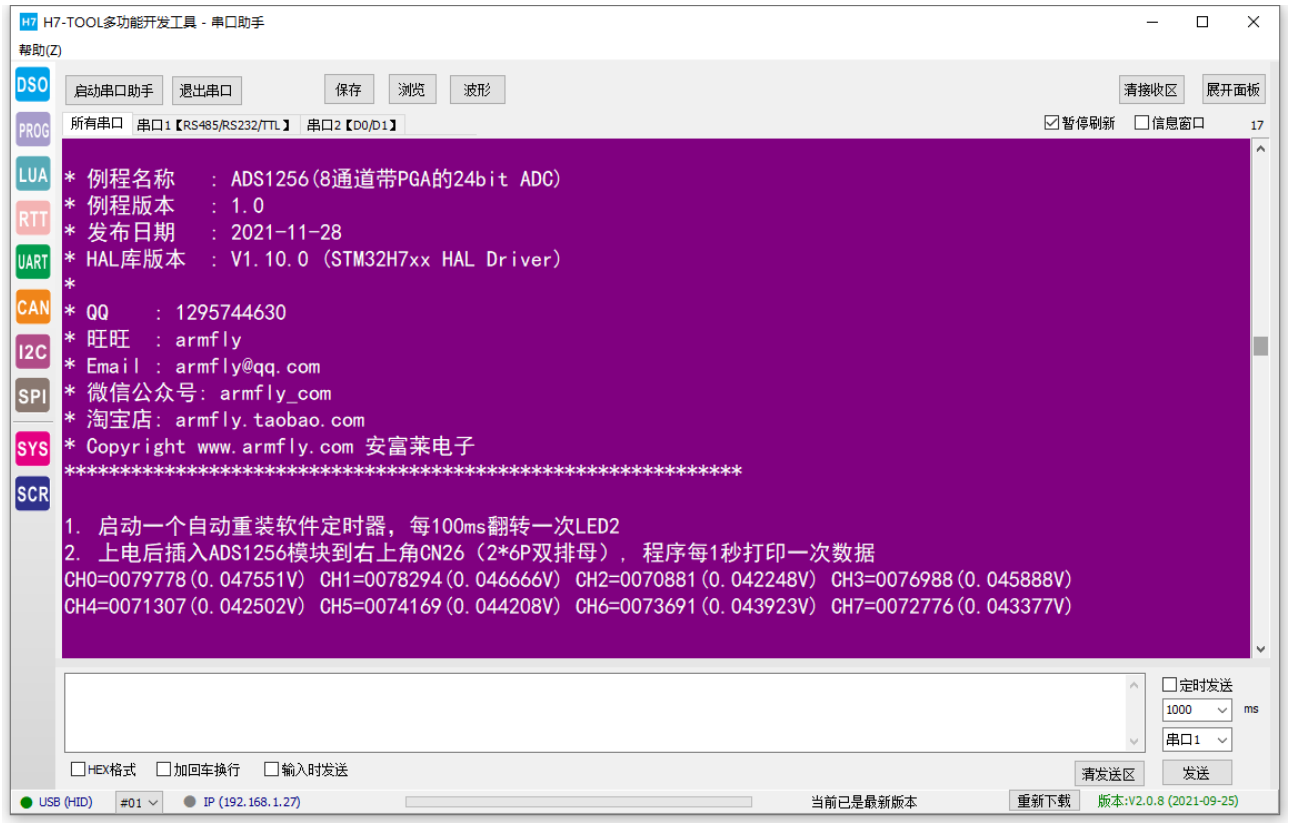
1. 开发板请使用外置电源供电。

实验内容：

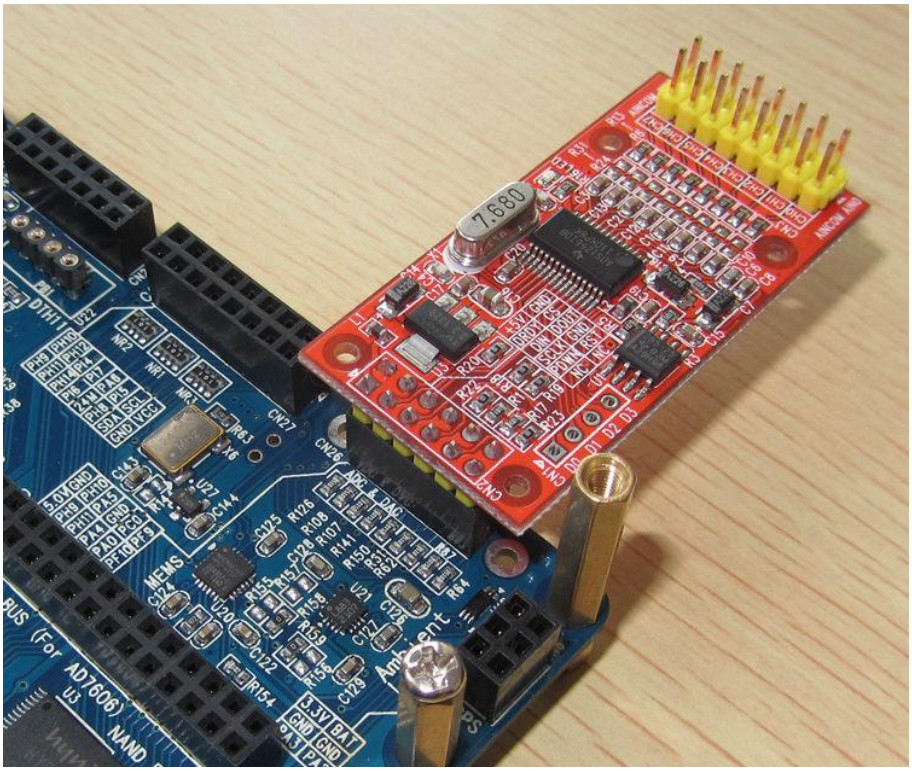
1. 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
2. 上电后插入 ADS1256 模块到右上角 CN26（2*6P 双排母），程序每 1 秒打印一次 8 通道采样数据。

上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1。

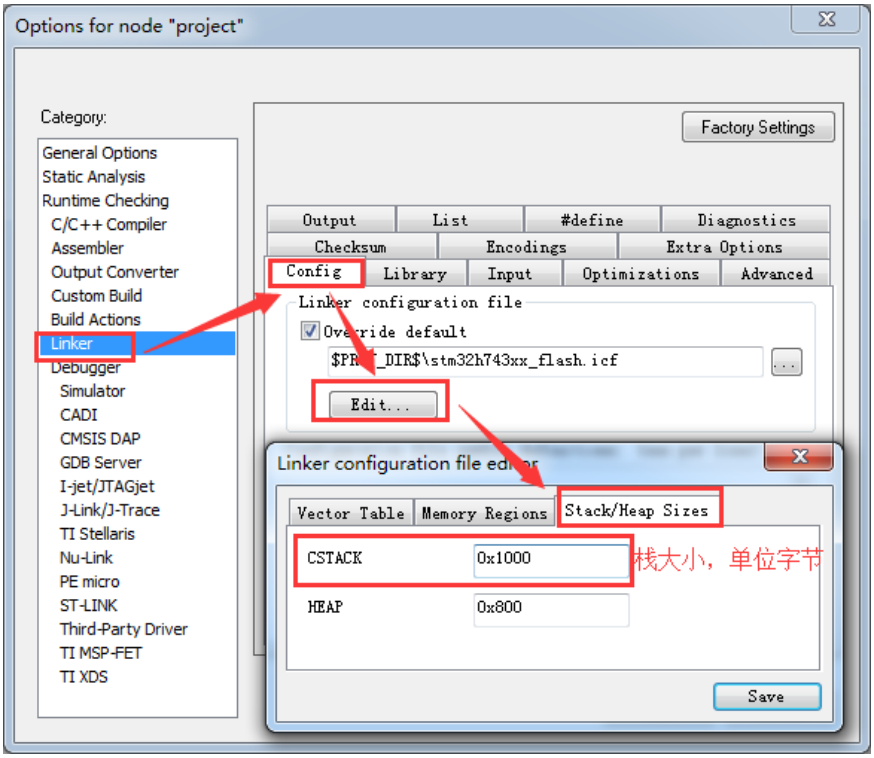


模块插入位置：

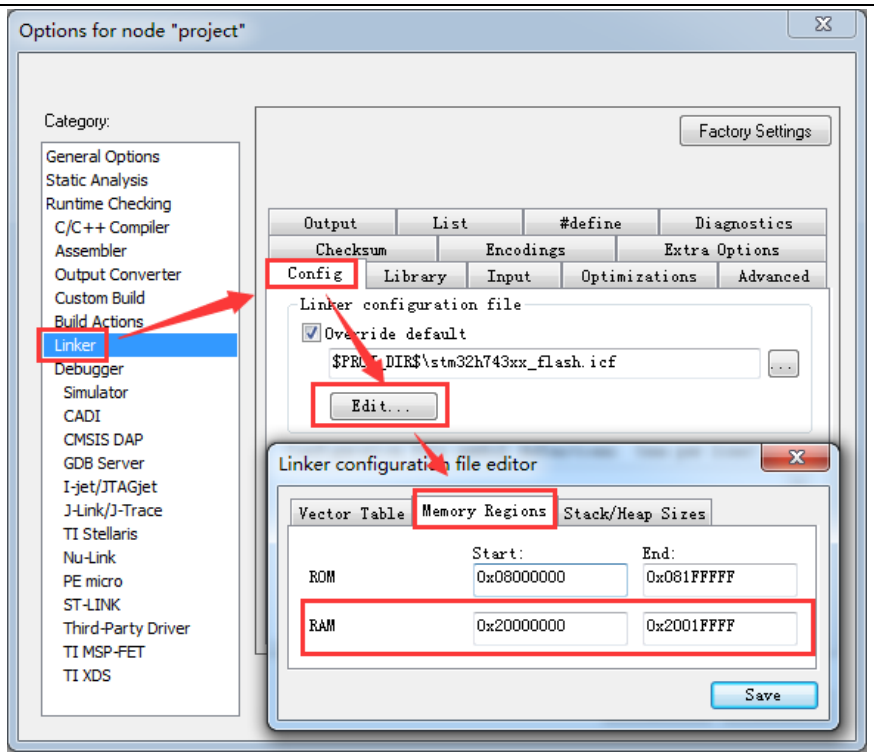


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
```

```
        Event Recorder:
        - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
        - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();      /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();    /* 初始化滴答定时器 */
    bsp_InitLPUart();   /* 初始化串口 */
    bsp_InitExtIO();    /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();      /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitAD7606();  /* 配置 AD7606 所用的 GPIO */
}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区。

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
```

```
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ 每 10ms 调用一次按键处理：

按键处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ 主功能：

主程序实现如下操作：

- 启动一个自动重装软件定时器，每 100ms 翻转一次 LED2。
- 上电后插入 ADS1256 模块到右上角 CN26（2*6P 双排母），程序每 1 秒打印一次 8 通道采样数

据。

```
/*
*********************************************************************************************************
*    函 数 名: main
*    功能说明: c 程序入口
*    形    参: 无
*    返 回 值: 错误代码(无需处理)
*********************************************************************************************************
*/
int main(void)
{
    uint8_t i;
    int32_t iTemp;
    float fTemp;


    bsp_Init();        /* 硬件初始化 */
    PrintfLogo(); /* 打印例程信息到串口 1 */

    PrintfHelp(); /* 打印操作提示信息 */


    bsp_DelayMS(500);  /* 等待上电稳定，等基准电压电路稳定, bsp_InitADS1256() 内部会进行自校准 */

    bsp_InitADS1256(); /* 初始化配置 ADS1256.  PGA=1, DRATE=30KSPS, BUFEN=1, 输入正负 5V */


    /* 打印芯片 ID (通过读 ID 可以判断硬件接口是否正常)， 正常时状态寄存器的高 4bit = 3 */
#if 0
    {
        uint8_t id;

        id = ADS1256_ReadChipID();

        if (id != 3)
        {
            printf("Error, ASD1256 Chip ID = 0x%X\r\n", id);
        }
        else
        {
            printf("Ok, ASD1256 Chip ID = 0x%X\r\n", id);
        }
    }
#endif

    ADS1256_CfgADC(ADS1256_GAIN_1, ADS1256_30SPS); /* 配置 ADC 参数:  增益 1:1, 数据输出速率 30Hz */

/* 启动中断扫描模式, 轮流采集 8 个通道的 ADC 数据. 通过 ADS1256_GetAdc() 函数来读取这些数据 */
    ADS1256_StartScan();

    bsp_StartAutoTimer(0, 1000);     /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 进入主程序循环体 */
    while (1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */
```

```
        if (bsp_CheckTimer(0)) /* 判断定时器超时时间 */
        {
            /* 每隔 1000ms 进来一次 */
            bsp_LedToggle(2);  /* 翻转 LED 的状态 */

            /* 打印采集数据 */
            for (i = 0; i < 8; i++)
            {

                /*
                  计算公式：2 * VREF/(PGA * (2^23 - 1)) ，这里 VREF 是 2.5V，PGA = 1
                */
                 /* 计算实际电压值（近似估算的），如需准确，请进行校准 */
                 iTemp = ((int64_t)g_tADS1256.AdcNow[i] * 2500000) / 4194303;


                 fTemp = (float)iTemp / 1000000;

                 printf("CH%d=%07d(%fV) ", i, g_tADS1256.AdcNow[i], fTemp);

                 if(i == 3)
                 {
                     printf("\r\n");
                 }
            }

            printf("\r\n\r\n");
        }
    }
}
```

93.12 总结

本章节涉及到的知识点非常多，主要为大家讲解了 ADS1256 的常用玩法，如果实际项目中用到此芯片需要熟练运用。

第94章 STM32H7 的 SPI 总线应用之双机通信（DMA 方式）

本章节为大家讲解 SPI DMA 方式双机通信。

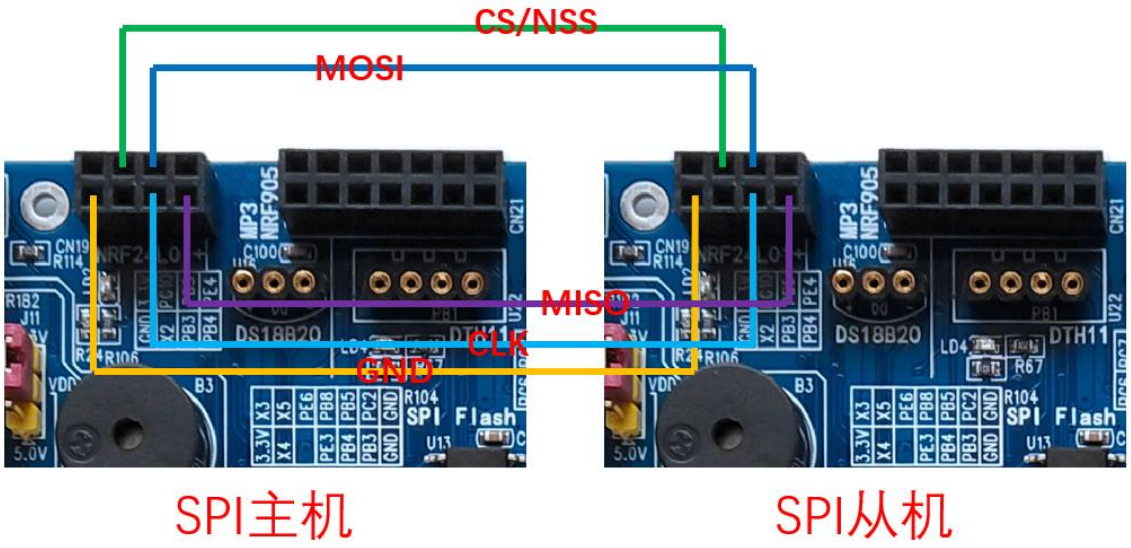
94.1 初学者重要提示
94.2 SPI DMA 主从机硬件接线
94.3 SPI DMA 主机程序设计
94.4 SPI DMA 从机程序设计
94.5 SPI DMA 主从机使用注意事项
94.6 SPI DMA 主从机驱动移植和使用
94.7 实验例程设计框架
94.8 实验例程说明（MDK）
94.9 实验例程说明（IAR）
94.10 总结

94.1 初学者重要提示

- 学习本章节前，务必优先学习第 72 章 SPI 基础和第 73 章 SPI Flash 的 DMA 玩法方式。本章实现的 SPI DMA 通信方式的主机和从机，跟 SPI DMA 方式驱动 SPI Flash 是类似的。
- 本章是采用的 SPI DMA 全双工通信方式。
- 大家根据自己接线的稳定性，可以适当调节 SPI 主机和从机的时钟速度，其中从机的时钟速度是可以高于主机速度的，这样通信的容错性更好些。

94.2 SPI DMA 主从机硬件接线

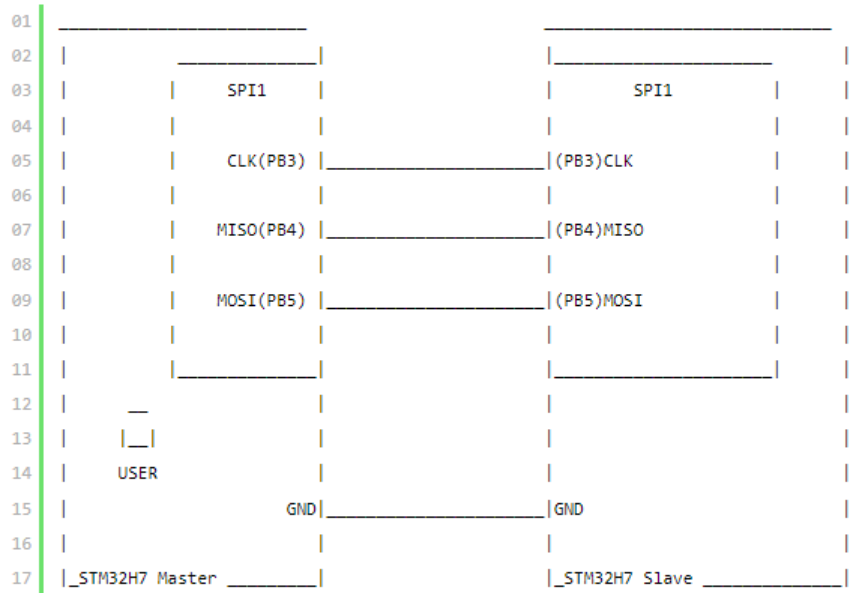
接线方式如下，使用的两块 V7 板子，一块板子做主机，一块板子做从机。



对应的引脚信息如下：

```
_____________________                    _____________________________
|          _____________|                  |_____________________       |
|         |     SPI1    |                  |       SPI1          |      |
|         |             |                  |                     |      |
|         |    CLK(PB3) |__________________|(PB3)CLK             |      |
|         |             |                  |                     |      |
|         |   MISO(PB4) |__________________|(PB4)MISO            |      |
|         |             |                  |                     |      |
|         |   MOSI(PB5) |__________________|(PB5)MOSI            |      |
|         |             |                  |                     |      |
|         |   NSS(PG10) |__________________|(PG10)NSS            |      |
|         |_____________|                  |_____________________|      |
|                       |                  |                            |
|                       |                  |                            |
|                       |                  |                            |
|                   GND|__________________|GND                         |
|                       |                  |                            |
|_STM32H7 Master ________|                  |_STM32H7 Slave _____________|
```

实际项目中使用，推荐大家务必比将硬件片选引脚 NSS 接上，实现全程硬件控制收发。如果大家不使用硬件片选，而使用下面的方式：



这种方式有个比较明显的缺点，主从机上电次序不同，很容易造成从机 CLK 识别错误，即高低电平变化导致数据传输错位。改成加入硬件 SPI 片选 NSS 引脚后，完美解决了这个问题

94.3 SPI DMA 主机程序设计

SPI DMA 主机程序实现和本教程 72 的 SPI DMA 配置是一样的，只是多了 SPI 硬件片选引脚 NSS 配置。

94.3.1 第 1 步：SPI 总线配置

SPI 总线配置通过如下两个函数实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitSPIBus
*    功能说明: 配置 SPI 总线。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitSPIBus(void)
{
    g_spi_busy = 0;

    bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_16, SPI_PHASE_1EDGE, SPI_POLARITY_LOW);
}

/*
*********************************************************************************************************
*    函 数 名: bsp_InitSPIParam
*    功能说明: 配置 SPI 总线参数，时钟分频，时钟相位和时钟极性。
*    形    参: _BaudRatePrescaler  SPI 总线时钟分频设置，支持的参数如下：
*                                 SPI_BAUDRATEPRESCALER_2    2 分频
```

```
*                                SPI_BAUDRATEPRESCALER_4    4 分频
*                                SPI_BAUDRATEPRESCALER_8    8 分频
*                                SPI_BAUDRATEPRESCALER_16   16 分频
*                                SPI_BAUDRATEPRESCALER_32   32 分频
*                                SPI_BAUDRATEPRESCALER_64   64 分频
*                                SPI_BAUDRATEPRESCALER_128  128 分频
*                                SPI_BAUDRATEPRESCALER_256  256 分频
*
*            _CLKPhase           时钟相位，支持的参数如下：
*                                SPI_PHASE_1EDGE     SCK 引脚的第 1 个边沿捕获传输的第 1 个数据
*                                SPI_PHASE_2EDGE     SCK 引脚的第 2 个边沿捕获传输的第 1 个数据
*
*            _CLKPolarity        时钟极性，支持的参数如下：
*                                SPI_POLARITY_LOW    SCK 引脚在空闲状态处于低电平
*                                SPI_POLARITY_HIGH   SCK 引脚在空闲状态处于高电平
*
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitSPIParam(uint32_t _BaudRatePrescaler, uint32_t _CLKPhase, uint32_t _CLKPolarity)
{
    /* 提高执行效率，只有在 SPI 硬件参数发生变化时，才执行 HAL_Init */
    if (s_BaudRatePrescaler == _BaudRatePrescaler && s_CLKPhase == _CLKPhase && s_CLKPolarity == _CLKPolarity)
    {
        return;
    }

    s_BaudRatePrescaler = _BaudRatePrescaler;
    s_CLKPhase = _CLKPhase;
    s_CLKPolarity = _CLKPolarity;


    /* 设置 SPI 参数 */
    hspi.Instance               = SPIx;                     /* 例化 SPI */
    hspi.Init.BaudRatePrescaler = _BaudRatePrescaler;       /* 设置波特率 */
    hspi.Init.Direction         = SPI_DIRECTION_2LINES;     /* 全双工 */
    hspi.Init.CLKPhase          = _CLKPhase;                /* 配置时钟相位 */
    hspi.Init.CLKPolarity       = _CLKPolarity;             /* 配置时钟极性 */
    hspi.Init.DataSize          = SPI_DATASIZE_8BIT;        /* 设置数据宽度 */
    hspi.Init.FirstBit          = SPI_FIRSTBIT_MSB;         /* 数据传输先传高位 */
    hspi.Init.TIMode            = SPI_TIMODE_DISABLE;       /* 禁止 TI 模式  */
    hspi.Init.CRCCalculation    = SPI_CRCCALCULATION_DISABLE; /* 禁止 CRC */
    hspi.Init.CRCPolynomial     = 7;                         /* 禁止 CRC 后，此位无效 */
    hspi.Init.CRCLength         = SPI_CRC_LENGTH_8BIT;       /* 禁止 CRC 后，此位无效 */
    hspi.Init.NSS               = SPI_NSS_SOFT;              /* 使用软件方式管理片选引脚 */
    hspi.Init.FifoThreshold     = SPI_FIFO_THRESHOLD_01DATA;  /* 设置 FIFO 大小是一个数据项 */
    hspi.Init.NSSPMode          = SPI_NSS_PULSE_DISABLE;      /* 禁止脉冲输出 */
    hspi.Init.MasterKeepIOState = SPI_MASTER_KEEP_IO_STATE_ENABLE; /* 禁止 SPI 后，SPI 相关引脚保持当前状态 */
    hspi.Init.Mode              = SPI_MODE_MASTER;           /* SPI 工作在主控模式 */

    /* 设置 SPI 参数 */
    hspi.Instance               = SPIx;                      /* 例化 SPI */
    hspi.Init.BaudRatePrescaler = _BaudRatePrescaler;        /* 设置波特率 */
    hspi.Init.Direction         = SPI_DIRECTION_2LINES;      /* 全双工 */
    hspi.Init.CLKPhase          = _CLKPhase;                 /* 配置时钟相位 */
    hspi.Init.CLKPolarity       = _CLKPolarity;              /* 配置时钟极性 */
    hspi.Init.DataSize          = SPI_DATASIZE_8BIT;         /* 设置数据宽度 */
    hspi.Init.FirstBit          = SPI_FIRSTBIT_MSB;          /* 数据传输先传高位 */
```

```
    hspi.Init.TIMode            = SPI_TIMODE_DISABLE;           /* 禁止 TI 模式  */
    hspi.Init.CRCCalculation    = SPI_CRCCALCULATION_DISABLE;   /* 禁止 CRC */
    hspi.Init.CRCPolynomial     = 7;                            /* 禁止 CRC 后，此位无效 */
    hspi.Init.CRCLength         = SPI_CRC_LENGTH_8BIT;          /* 禁止 CRC 后，此位无效 */
    hspi.Init.FifoThreshold     = SPI_FIFO_THRESHOLD_16DATA;    /* 设置 FIFO 大小是一个数据项 */

    hspi.Init.NSS         = SPI_NSS_HARD_OUTPUT;                /* 片选引脚 */
    hspi.Init.NSSPMode    = SPI_NSS_PULSE_DISABLE;              /* 禁止脉冲输出 */
    hspi.Init.NSSPolarity = SPI_NSS_POLARITY_LOW;               /* 低电平有效 */
    hspi.Init.MasterSSIdleness        = SPI_MASTER_SS_IDLENESS_00CYCLE; /* MSS，插入到 NSS 有效边沿和第一个
                                                                   数据开始之间的额外延迟，单位 SPI 时钟周期个数 */

    hspi.Init.MasterInterDataIdleness = SPI_MASTER_INTERDATA_IDLENESS_10CYCLE; /* MIDI，两个连续数据帧之间
                                                                   插入的最小时间延迟，单位 SPI 时钟周期个数 */

    hspi.Init.MasterKeepIOState = SPI_MASTER_KEEP_IO_STATE_ENABLE; /* 禁止 SPI 后，SPI 相关引脚保持当前状态 */
    hspi.Init.Mode              = SPI_MODE_MASTER;

    /* 复位配置 */
    if (HAL_SPI_DeInit(&hspi) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 初始化配置 */
    if (HAL_SPI_Init(&hspi) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

关于这两个函数有以下四点要做个说明：

- 函数 bsp_InitSPIBus 里面的配置是个初始设置。实际驱动芯片时，会通过函数 bsp_InitSPIParam 做再配置。
- 函数 bsp_InitSPIParam 提供了时钟分频，时钟相位和时钟极性配置。一般主机和从机此处设置为一样即可，但推荐
- SPI 硬件片选 NSS 设置为 SPI_NSS_HARD_OUTPUT。
- 这里特别注意主机是 hspi.Init.Mode = SPI_MODE_MASTER。

94.3.2第 2 步：SPI DMA 配置

配置代码实现如下，注释比较详细：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitSPIParam
*    功能说明: 配置 SPI 总线时钟，GPIO，中断，DMA 等
*    形    参: SPI_HandleTypeDef 类型指针变量
*    返 回 值: 无
*********************************************************************************************************
*/
void HAL_SPI_MspInit(SPI_HandleTypeDef *_hspi)
```

```
{
    /* 配置 SPI 总线 GPIO : SCK MOSI MISO */
    {
        GPIO_InitTypeDef  GPIO_InitStruct;

        /* SPI 和 GPIP 时钟 */
        SPIx_SCK_CLK_ENABLE();
        SPIx_MISO_CLK_ENABLE();
        SPIx_MOSI_CLK_ENABLE();
        SPIx_CLK_ENABLE();

        /* SPI SCK */
        GPIO_InitStruct.Pin       = SPIx_SCK_PIN;
        GPIO_InitStruct.Mode      = GPIO_MODE_AF_PP;
        GPIO_InitStruct.Pull      = GPIO_PULLUP;
        GPIO_InitStruct.Speed     = GPIO_SPEED_FREQ_HIGH;
        GPIO_InitStruct.Alternate = SPIx_SCK_AF;
        HAL_GPIO_Init(SPIx_SCK_GPIO, &GPIO_InitStruct);

        /* SPI MISO */
        GPIO_InitStruct.Pin = SPIx_MISO_PIN;
        GPIO_InitStruct.Alternate = SPIx_MISO_AF;
        HAL_GPIO_Init(SPIx_MISO_GPIO, &GPIO_InitStruct);

        /* SPI MOSI */
        GPIO_InitStruct.Pin = SPIx_MOSI_PIN;
        GPIO_InitStruct.Alternate = SPIx_MOSI_AF;
        HAL_GPIO_Init(SPIx_MOSI_GPIO, &GPIO_InitStruct);

       /* SPI NSS */
       GPIO_InitStruct.Pin = SPIx_NSS_PIN;
       GPIO_InitStruct.Alternate = SPIx_NSS_AF;
       HAL_GPIO_Init(SPIx_NSS_GPIO, &GPIO_InitStruct);
    }

    /* 配置 DMA 和 NVIC */
    #ifdef USE_SPI_DMA
    {
        /* 使能 DMA 时钟 */
        DMAx_CLK_ENABLE();

        /* SPI DMA 发送配置 */
        hdma_tx.Instance                 = SPIx_TX_DMA_STREAM;    /* 例化使用的 DMA 数据流 */
        hdma_tx.Init.FIFOMode            = DMA_FIFOMODE_ENABLE;   /* FIFO*/
        hdma_tx.Init.FIFOThreshold       = DMA_FIFO_THRESHOLD_FULL;/* 禁止 FIFO 此位不起作用,用于设置阀值 */
        hdma_tx.Init.MemBurst            = DMA_MBURST_SINGLE;   /* 禁止 FIFO 此位不起作用,用于存储器突发 */
        hdma_tx.Init.PeriphBurst         = DMA_PBURST_SINGLE;    /* 禁止 FIFO 此位不起作用，用于外设突发 */
        hdma_tx.Init.Request             = SPIx_TX_DMA_REQUEST;     /* 请求类型 */
        hdma_tx.Init.Direction           = DMA_MEMORY_TO_PERIPH;    /* 传输方向是从存储器到外设 */
        hdma_tx.Init.PeriphInc           = DMA_PINC_DISABLE;        /* 外设地址自增禁止 */
        hdma_tx.Init.MemInc              = DMA_MINC_ENABLE;         /* 存储器地址自增使能 */
        hdma_tx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;    /* 外设数据传输位宽选择字节，即 8bit */
        hdma_tx.Init.MemDataAlignment    = DMA_MDATAALIGN_BYTE;     /* 存储器数据传输位宽选择字节，即 8bit
*/
        hdma_tx.Init.Mode                = DMA_NORMAL;              /* 正常模式 */
        hdma_tx.Init.Priority            = DMA_PRIORITY_LOW;        /* 优先级低 */

         /* 复位 DMA */
```

```
    if(HAL_DMA_DeInit(&hdma_tx) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

     /* 初始化 DMA */
    if(HAL_DMA_Init(&hdma_tx) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 关联 DMA 句柄到 SPI */
    __HAL_LINKDMA(_hspi, hdmatx, hdma_tx);

    /* SPI DMA 接收配置 */
    hdma_rx.Instance                 = SPIx_RX_DMA_STREAM;      /* 例化使用的 DMA 数据流 */
    hdma_rx.Init.FIFOMode            = DMA_FIFOMODE_ENABLE;     /* FIFO*/
    hdma_rx.Init.FIFOThreshold       =DMA_FIFO_THRESHOLD_FULL;/* 禁止 FIFO 此位不起作用,用于设置阀值 */
    hdma_rx.Init.MemBurst             = DMA_MBURST_SINGLE;    /* 禁止FIFO此位不起作用,用于存储器突发 */
    hdma_rx.Init.PeriphBurst          = DMA_PBURST_SINGLE;    /* 禁止 FIFO 此位不起作用, 用于外设突发 */
    hdma_rx.Init.Request              = SPIx_RX_DMA_REQUEST;     /* 请求类型 */
    hdma_rx.Init.Direction            = DMA_PERIPH_TO_MEMORY;    /* 传输方向从外设到存储器 */
    hdma_rx.Init.PeriphInc            = DMA_PINC_DISABLE;        /* 外设地址自增禁止 */
    hdma_rx.Init.MemInc               = DMA_MINC_ENABLE;         /* 存储器地址自增使能 */
    hdma_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;     /* 外设数据传输位宽选择字节, 即 8bit */
    hdma_rx.Init.MemDataAlignment    =DMA_MDATAALIGN_BYTE;     /* 存储器数据传输位宽选择字节,即 8bit */
    hdma_rx.Init.Mode                 = DMA_NORMAL;              /* 正常模式 */
    hdma_rx.Init.Priority             = DMA_PRIORITY_HIGH;       /* 优先级高 */

     /* 复位 DMA */
    if(HAL_DMA_DeInit(&hdma_rx) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

     /* 初始化 DMA */
    if(HAL_DMA_Init(&hdma_rx) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 关联 DMA 句柄到 SPI */
   __HAL_LINKDMA(_hspi, hdmarx, hdma_rx);

    /* 配置 DMA 发送中断 */
    HAL_NVIC_SetPriority(SPIx_DMA_TX_IRQn, 1, 0);
    HAL_NVIC_EnableIRQ(SPIx_DMA_TX_IRQn);

    /* 配置 DMA 接收中断 */
    HAL_NVIC_SetPriority(SPIx_DMA_RX_IRQn, 1, 0);
    HAL_NVIC_EnableIRQ(SPIx_DMA_RX_IRQn);

    /* 配置 SPI 中断 */
    HAL_NVIC_SetPriority(SPIx_IRQn, 1, 0);
    HAL_NVIC_EnableIRQ(SPIx_IRQn);
}
#endif
```

```
    #ifdef USE_SPI_INT
        /* 配置 SPI 中断优先级并使能中断 */
        HAL_NVIC_SetPriority(SPIx_IRQn, 1, 0);
        HAL_NVIC_EnableIRQ(SPIx_IRQn);
    #endif
}
```

94.3.3第 3 步：SPI DMA 传输设置和 MPU 配置

SPI DMA 方式主要通过函数 bsp_spiTransfer 实现数据传输（代码里面的查询和中断方式请忽略）：

```
/*
*********************************************************************************************************
*                                    选择 DMA，中断或者查询方式
*********************************************************************************************************
*/
#define USE_SPI_DMA    /* DMA 方式  */
//#define USE_SPI_INT    /* 中断方式 */
//#define USE_SPI_POLL   /* 查询方式 */

/* 查询模式 */
#if defined (USE_SPI_POLL)

uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];

/* 中断模式 */
#elif defined (USE_SPI_INT)

uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];

/* DMA 模式使用的 SRAM4 */
#elif defined (USE_SPI_DMA)
    #if defined ( __CC_ARM )    /* IAR *******/
        __attribute__((section (".RAM_D3"))) uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
        __attribute__((section (".RAM_D3"))) uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];
    #elif defined (__ICCARM__)   /* MDK ********/
        #pragma location = ".RAM_D3"
        uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
        #pragma location = ".RAM_D3"
        uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];
    #endif
#endif

/*
*********************************************************************************************************
*    函 数 名: bsp_spiTransfer
*    功能说明: 启动数据传输
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_spiTransfer(void)
{
    if (g_spiLen > SPI_BUFFER_SIZE)
```

```
    {
        return;
    }

    /* DMA 方式传输 */
#ifdef USE_SPI_DMA
    wTransferState = TRANSFER_WAIT;

    if(HAL_SPI_TransmitReceive_DMA(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen) != HAL_OK)

    {
        Error_Handler(__FILE__, __LINE__);
    }

    while (wTransferState == TRANSFER_WAIT)
    {
        ;
    }
#endif

    /* 中断方式传输 */
#ifdef USE_SPI_INT
    wTransferState = TRANSFER_WAIT;

    if(HAL_SPI_TransmitReceive_IT(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen) != HAL_OK)

    {
        Error_Handler(__FILE__, __LINE__);
    }

    while (wTransferState == TRANSFER_WAIT)
    {
        ;
    }
#endif

    /* 查询方式传输 */
#ifdef USE_SPI_POLL
    if(HAL_SPI_TransmitReceive(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen, 1000000) !=
HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
#endif
}
```

DMA 方式要特别注意两点：

- 通过本手册第 26 章的内存块超方便使用方式，将 DMA 缓冲定义到 SRAM4 上。因为本工程是用的 DTCM 做的主 RAM 空间，这个空间无法使用通用 DMA1 和 DMA2。
- 程序这里 SPI DMA 方式主控用的是等待传输完成，大家根据自己实际应用可以做修改，详情大家可以看此贴作为拓展：

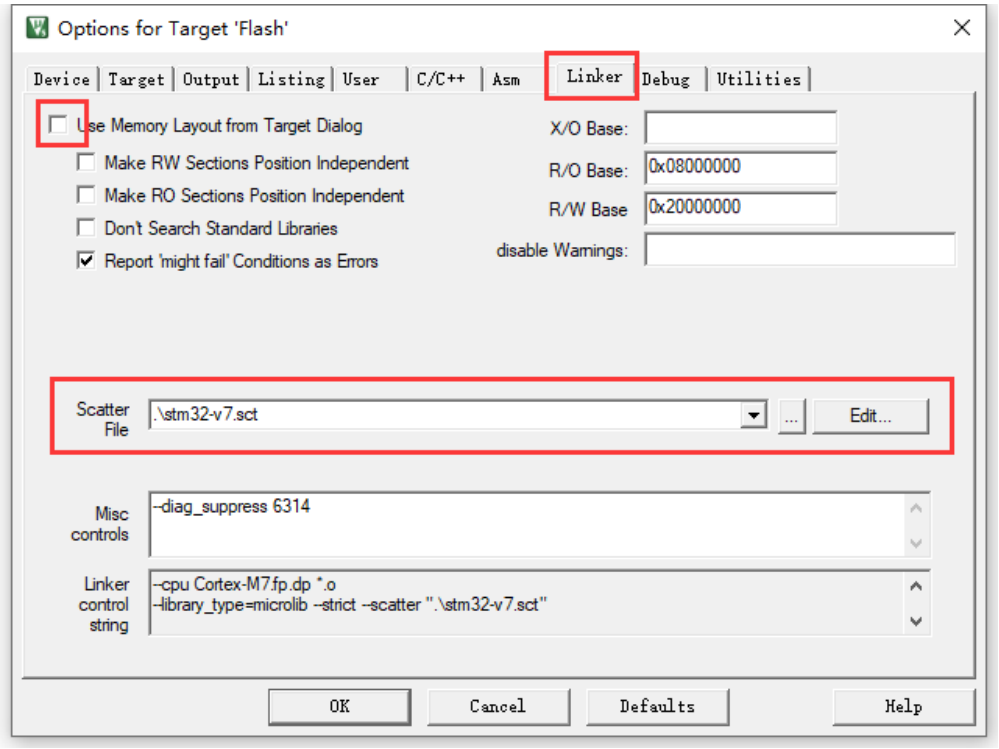
  【深入探讨】DMA 到底能不能起到加速程序执行的作用，DMA 死等操作是否合理，多个 DMA 数据流同时刷是否处理过来

https://www.armbbs.cn/forum.php?mod=viewthread&tid=109765 。

◆ 由于程序里面开启了数据 Cache，会造成 DMA 和 CPU 访问 SRAM4 数据不一致的问题，特此将 SRAM4 空间关闭 Cache。

```
/* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress      = 0x38000000;
MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

分散加载设置：



94.3.4第 4 步：应用代码设计

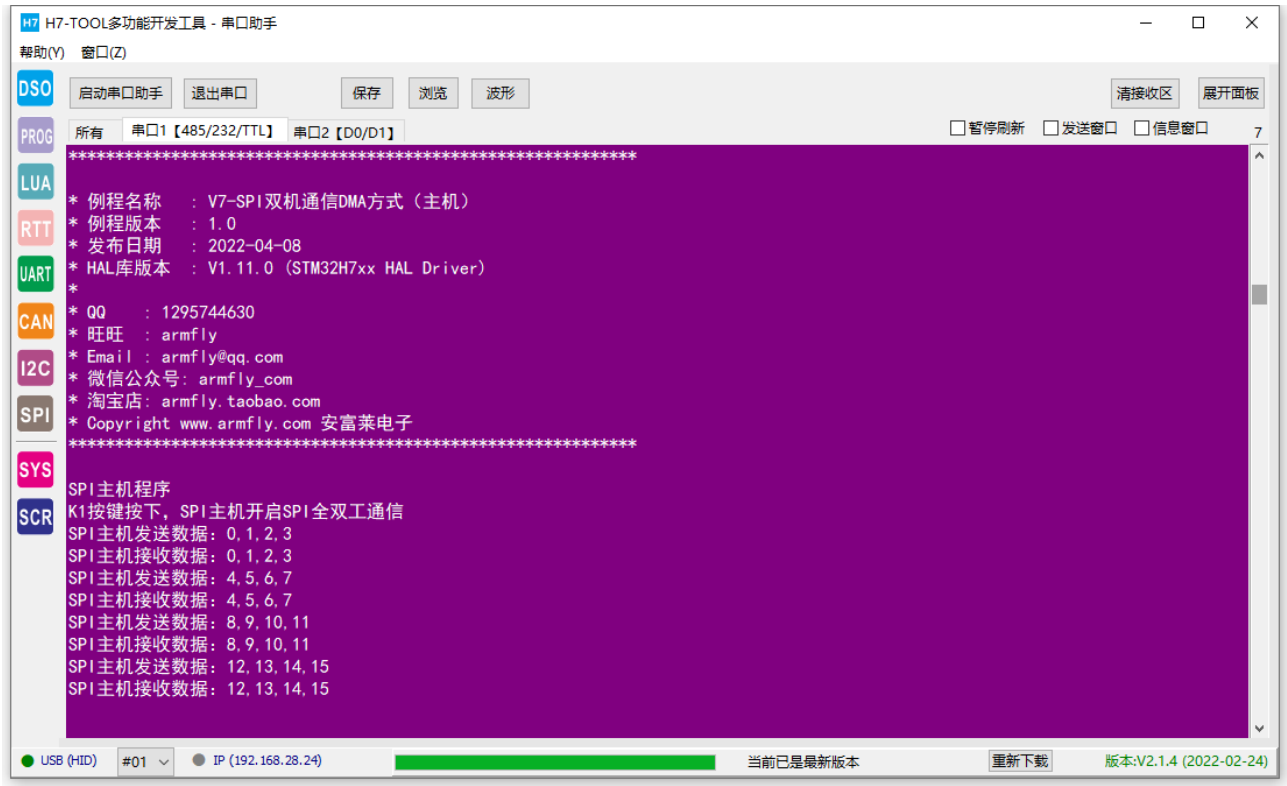
应用部分的代码设计如下：

```
/*
*********************************************************************************************************
*    函 数 名: DemoSpiMaster
*    功能说明: SPI 主机通信
*    形    参：无
*    返 回 值: 无
```

```
*********************************************************************************************************
*/
void DemoSpiMaster(void)
{
    uint8_t count = 0;
    uint8_t ucKeyCode;      /* 按键代码 */

    /***************设置 SPI Flash 片选上拉，防止影响 ***************/
    {
        GPIO_InitTypeDef gpio_init;

        /* 打开 GPIO 时钟 */
        __HAL_RCC_GPIOD_CLK_ENABLE();

        gpio_init.Mode = GPIO_MODE_OUTPUT_PP;
        gpio_init.Pull = GPIO_NOPULL;
        gpio_init.Speed = GPIO_SPEED_HIGH;
        gpio_init.Pin = GPIO_PIN_13;
        HAL_GPIO_Init(GPIOD, &gpio_init);

        GPIOD->BSRR = GPIO_PIN_13;
    }

    sfDispMenu();      /* 打印命令提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    while(1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值，无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，发送数据给从机*/
                  g_spiTxBuf[0] = count++;
                  g_spiTxBuf[1] = count++;
                  g_spiTxBuf[2] = count++;
                  g_spiTxBuf[3] = count++;
                  g_spiLen = 4;
                  printf("SPI 主机发送数据：%d,%d,%d,%d\r\n",
                         g_spiTxBuf[0],g_spiTxBuf[1],g_spiTxBuf[2],g_spiTxBuf[3]);

                  bsp_spiTransfer();
                  printf("SPI 主机接收数据：%d,%d,%d,%d\r\n",
                          g_spiRxBuf[0],g_spiRxBuf[1],g_spiRxBuf[2],g_spiRxBuf[3]);
                     break;
                default:
```

```
                        /* 其它的键值不处理 */
                        break;
                }

        }
    }
}
```

这部分代码比较好理解，大家按下 K1 按键后，会打印发送的数据并打印 SPI 从机设备返回的数据。



94.4 SPI DMA 从机程序设计

SPI DMA 从机设计程序如下，与主机不同的是部分配置选项要设置为从机方式。

94.4.1 第 1 步：SPI 总线配置

SPI 总线配置通过如下两个函数实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitSPIBus
*    功能说明: 配置 SPI 总线。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitSPIBus(void)
{
    g_spi_busy = 0;
```

```
    bsp_InitSPIParam(SPI_BAUDRATEPRESCALER_16, SPI_PHASE_1EDGE, SPI_POLARITY_LOW);
}

/*
*********************************************************************************************************
*    函 数 名: bsp_InitSPIParam
*    功能说明: 配置 SPI 总线参数，时钟分频，时钟相位和时钟极性。
*    形    参: _BaudRatePrescaler  SPI 总线时钟分频设置，支持的参数如下：
*                                 SPI_BAUDRATEPRESCALER_2    2 分频
*                                 SPI_BAUDRATEPRESCALER_4    4 分频
*                                 SPI_BAUDRATEPRESCALER_8    8 分频
*                                 SPI_BAUDRATEPRESCALER_16   16 分频
*                                 SPI_BAUDRATEPRESCALER_32   32 分频
*                                 SPI_BAUDRATEPRESCALER_64   64 分频
*                                 SPI_BAUDRATEPRESCALER_128  128 分频
*                                 SPI_BAUDRATEPRESCALER_256  256 分频
*
*              _CLKPhase          时钟相位，支持的参数如下：
*                                 SPI_PHASE_1EDGE     SCK 引脚的第 1 个边沿捕获传输的第 1 个数据
*                                 SPI_PHASE_2EDGE     SCK 引脚的第 2 个边沿捕获传输的第 1 个数据
*
*              _CLKPolarity       时钟极性，支持的参数如下：
*                                 SPI_POLARITY_LOW    SCK 引脚在空闲状态处于低电平
*                                 SPI_POLARITY_HIGH   SCK 引脚在空闲状态处于高电平
*
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_InitSPIParam(uint32_t _BaudRatePrescaler, uint32_t _CLKPhase, uint32_t _CLKPolarity)
{
    /* 提高执行效率，只有在 SPI 硬件参数发生变化时，才执行 HAL_Init */
    if (s_BaudRatePrescaler == _BaudRatePrescaler && s_CLKPhase == _CLKPhase && s_CLKPolarity == _CLKPolarity)
    {
        return;
    }

    s_BaudRatePrescaler = _BaudRatePrescaler;
    s_CLKPhase = _CLKPhase;
    s_CLKPolarity = _CLKPolarity;


    /* 设置 SPI 参数 */
    hspi.Instance               = SPIx;                     /* 例化 SPI */
    hspi.Init.BaudRatePrescaler = _BaudRatePrescaler;       /* 设置波特率 */
    hspi.Init.Direction         = SPI_DIRECTION_2LINES;     /* 全双工 */
    hspi.Init.CLKPhase          = _CLKPhase;                /* 配置时钟相位 */
    hspi.Init.CLKPolarity       = _CLKPolarity;             /* 配置时钟极性 */
    hspi.Init.DataSize          = SPI_DATASIZE_8BIT;        /* 设置数据宽度 */
    hspi.Init.FirstBit          = SPI_FIRSTBIT_MSB;         /* 数据传输先传高位 */
    hspi.Init.TIMode            = SPI_TIMODE_DISABLE;       /* 禁止 TI 模式  */
    hspi.Init.CRCCalculation    = SPI_CRCCALCULATION_DISABLE; /* 禁止 CRC */
    hspi.Init.CRCPolynomial     = 7;                         /* 禁止 CRC 后，此位无效 */
    hspi.Init.CRCLength         = SPI_CRC_LENGTH_8BIT;      /* 禁止 CRC 后，此位无效 */
    hspi.Init.NSS               = SPI_NSS_SOFT;              /* 使用软件方式管理片选引脚 */
    hspi.Init.FifoThreshold     = SPI_FIFO_THRESHOLD_01DATA;  /* 设置 FIFO 大小是一个数据项 */
    hspi.Init.NSSPMode          = SPI_NSS_PULSE_DISABLE;      /* 禁止脉冲输出 */
    hspi.Init.MasterKeepIOState = SPI_MASTER_KEEP_IO_STATE_ENABLE; /* 禁止 SPI 后，SPI 相关引脚保持当前状态 */
    hspi.Init.Mode              = SPI_MODE_MASTER;           /* SPI 工作在主控模式 */
```

```
    /* 设置 SPI 参数 */
    hspi.Instance               = SPIx;                           /* 例化 SPI */
    hspi.Init.BaudRatePrescaler = _BaudRatePrescaler;             /* 设置波特率 */
    hspi.Init.Direction         = SPI_DIRECTION_2LINES;           /* 全双工 */
    hspi.Init.CLKPhase          = _CLKPhase;                      /* 配置时钟相位 */
    hspi.Init.CLKPolarity       = _CLKPolarity;                   /* 配置时钟极性 */
    hspi.Init.DataSize          = SPI_DATASIZE_8BIT;              /* 设置数据宽度 */
    hspi.Init.FirstBit          = SPI_FIRSTBIT_MSB;               /* 数据传输先传高位 */
    hspi.Init.TIMode            = SPI_TIMODE_DISABLE;             /* 禁止 TI 模式  */
    hspi.Init.CRCCalculation    = SPI_CRCCALCULATION_DISABLE;     /* 禁止 CRC */
    hspi.Init.CRCPolynomial     = 7;                              /* 禁止 CRC 后，此位无效 */
    hspi.Init.CRCLength         = SPI_CRC_LENGTH_8BIT;            /* 禁止 CRC 后，此位无效 */
    hspi.Init.FifoThreshold     = SPI_FIFO_THRESHOLD_16DATA;      /* 设置 FIFO 大小是一个数据项 */

    hspi.Init.NSS         = SPI_NSS_HARD_INPUT;                   /* 片选引脚 */
    hspi.Init.NSSPMode    = SPI_NSS_PULSE_DISABLE;                /* 禁止脉冲输出 */
    hspi.Init.NSSPolarity = SPI_NSS_POLARITY_LOW;                 /* 低电平有效 */
    hspi.Init.MasterSSIdleness        = SPI_MASTER_SS_IDLENESS_00CYCLE; /* MSS，插入到 NSS 有效边沿和第一个
                                                                   数据开始之间的额外延迟，单位 SPI 时钟周期个数 */

    hspi.Init.MasterInterDataIdleness = SPI_MASTER_INTERDATA_IDLENESS_10CYCLE; /* MIDI，两个连续数据帧之间
                                                                   插入的最小时间延迟，单位 SPI 时钟周期个数 */

    hspi.Init.MasterKeepIOState = SPI_MASTER_KEEP_IO_STATE_ENABLE; /* 禁止 SPI 后，SPI 相关引脚保持当前状态 */
    hspi.Init.Mode              = SPI_MODE_SLAVE;

    /* 复位配置 */
    if (HAL_SPI_DeInit(&hspi) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 初始化配置 */
    if (HAL_SPI_Init(&hspi) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
}
```

关于这两个函数有以下三点要做个说明：

- 函数 bsp_InitSPIBus 里面的配置是个初始设置。实际驱动芯片时，会通过函数 bsp_InitSPIParam 做再配置。
- 函数 bsp_InitSPIParam 提供了时钟分频，时钟相位和时钟极性配置。一般主机和从机此处设置为一样即可。
- SPI 硬件片选 NSS 设置为 SPI_NSS_HARD_INPUT。
- 这里特别注意主机是 hspi.Init.Mode = SPI_MODE_SLAVE。

94.4.2 第 2 步：SPI DMA 配置

配置代码实现如下，注释比较详细：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_InitSPIParam
*    功能说明: 配置 SPI 总线时钟，GPIO，中断，DMA 等
*    形    参: SPI_HandleTypeDef 类型指针变量
*    返 回 值: 无
*********************************************************************************************************
*/
void HAL_SPI_MspInit(SPI_HandleTypeDef *_hspi)
{
    /* 配置 SPI 总线 GPIO : SCK MOSI MISO */
    {
        GPIO_InitTypeDef  GPIO_InitStruct;

        /* SPI 和 GPIP 时钟 */
        SPIx_SCK_CLK_ENABLE();
        SPIx_MISO_CLK_ENABLE();
        SPIx_MOSI_CLK_ENABLE();
        SPIx_CLK_ENABLE();

        /* SPI SCK */
        GPIO_InitStruct.Pin       = SPIx_SCK_PIN;
        GPIO_InitStruct.Mode      = GPIO_MODE_AF_PP;
        GPIO_InitStruct.Pull      = GPIO_PULLUP;
        GPIO_InitStruct.Speed     = GPIO_SPEED_FREQ_HIGH;
        GPIO_InitStruct.Alternate = SPIx_SCK_AF;
        HAL_GPIO_Init(SPIx_SCK_GPIO, &GPIO_InitStruct);

        /* SPI MISO */
        GPIO_InitStruct.Pin = SPIx_MISO_PIN;
        GPIO_InitStruct.Alternate = SPIx_MISO_AF;
        HAL_GPIO_Init(SPIx_MISO_GPIO, &GPIO_InitStruct);

        /* SPI MOSI */
        GPIO_InitStruct.Pin = SPIx_MOSI_PIN;
        GPIO_InitStruct.Alternate = SPIx_MOSI_AF;
        HAL_GPIO_Init(SPIx_MOSI_GPIO, &GPIO_InitStruct);

       /* SPI NSS */
       GPIO_InitStruct.Pin = SPIx_NSS_PIN;
       GPIO_InitStruct.Alternate = SPIx_NSS_AF;
       HAL_GPIO_Init(SPIx_NSS_GPIO, &GPIO_InitStruct);
    }

    /* 配置 DMA 和 NVIC */
    #ifdef USE_SPI_DMA
    {
        /* 使能 DMA 时钟 */
        DMAx_CLK_ENABLE();

        /* SPI DMA 发送配置 */
        hdma_tx.Instance                 = SPIx_TX_DMA_STREAM;     /* 例化使用的 DMA 数据流 */
        hdma_tx.Init.FIFOMode            = DMA_FIFOMODE_ENABLE;    /* FIFO*/
        hdma_tx.Init.FIFOThreshold       =DMA_FIFO_THRESHOLD_FULL;/* 禁止 FIFO 此位不起作用，用于设置阀值 */
        hdma_tx.Init.MemBurst             = DMA_MBURST_SINGLE;    /* 禁止 FIFO 此位不起作用，用于存储器突发 */
        hdma_tx.Init.PeriphBurst          = DMA_PBURST_SINGLE;    /* 禁止 FIFO 此位不起作用，用于外设突发 */
        hdma_tx.Init.Request              = SPIx_TX_DMA_REQUEST;      /* 请求类型 */
        hdma_tx.Init.Direction            = DMA_MEMORY_TO_PERIPH;     /* 传输方向是从存储器到外设 */
```

```
    hdma_tx.Init.PeriphInc           = DMA_PINC_DISABLE;         /* 外设地址自增禁止 */
    hdma_tx.Init.MemInc              = DMA_MINC_ENABLE;          /* 存储器地址自增使能 */
    hdma_tx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;     /* 外设数据传输位宽选择字节，即 8bit */
    hdma_tx.Init.MemDataAlignment    = DMA_MDATAALIGN_BYTE;     /* 存储器数据传输位宽选择字节，即 8bit
*/
    hdma_tx.Init.Mode                = DMA_NORMAL;              /* 正常模式 */
    hdma_tx.Init.Priority            = DMA_PRIORITY_LOW;        /* 优先级低 */

     /* 复位 DMA */
    if(HAL_DMA_DeInit(&hdma_tx) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

     /* 初始化 DMA */
    if(HAL_DMA_Init(&hdma_tx) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 关联 DMA 句柄到 SPI */
    __HAL_LINKDMA(_hspi, hdmatx, hdma_tx);

    /* SPI DMA 接收配置 */
    hdma_rx.Instance                 = SPIx_RX_DMA_STREAM;      /* 例化使用的 DMA 数据流 */
    hdma_rx.Init.FIFOMode            = DMA_FIFOMODE_ENABLE;     /* FIFO*/
    hdma_rx.Init.FIFOThreshold       = DMA_FIFO_THRESHOLD_FULL;/* 禁止 FIFO 此位不起作用，用于设置阀值 */
    hdma_rx.Init.MemBurst            = DMA_MBURST_SINGLE;    /* 禁止 FIFO 此位不起作用，用于存储器突发 */
    hdma_rx.Init.PeriphBurst         = DMA_PBURST_SINGLE;    /* 禁止 FIFO 此位不起作用，用于外设突发 */
    hdma_rx.Init.Request             = SPIx_RX_DMA_REQUEST;     /* 请求类型 */
    hdma_rx.Init.Direction           = DMA_PERIPH_TO_MEMORY;    /* 传输方向从外设到存储器 */
    hdma_rx.Init.PeriphInc           = DMA_PINC_DISABLE;        /* 外设地址自增禁止 */
    hdma_rx.Init.MemInc              = DMA_MINC_ENABLE;         /* 存储器地址自增使能 */
    hdma_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;     /* 外设数据传输位宽选择字节，即 8bit */
    hdma_rx.Init.MemDataAlignment    = DMA_MDATAALIGN_BYTE;     /* 存储器数据传输位宽选择字节，即 8bit */
    hdma_rx.Init.Mode                = DMA_NORMAL;              /* 正常模式 */
    hdma_rx.Init.Priority            = DMA_PRIORITY_HIGH;       /* 优先级高 */

     /* 复位 DMA */
    if(HAL_DMA_DeInit(&hdma_rx) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

     /* 初始化 DMA */
    if(HAL_DMA_Init(&hdma_rx) != HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }

    /* 关联 DMA 句柄到 SPI */
   __HAL_LINKDMA(_hspi, hdmarx, hdma_rx);

    /* 配置 DMA 发送中断 */
    HAL_NVIC_SetPriority(SPIx_DMA_TX_IRQn, 1, 0);
    HAL_NVIC_EnableIRQ(SPIx_DMA_TX_IRQn);

    /* 配置 DMA 接收中断 */
```

```
        HAL_NVIC_SetPriority(SPIx_DMA_RX_IRQn, 1, 0);
        HAL_NVIC_EnableIRQ(SPIx_DMA_RX_IRQn);

        /* 配置 SPI 中断 */
        HAL_NVIC_SetPriority(SPIx_IRQn, 1, 0);
        HAL_NVIC_EnableIRQ(SPIx_IRQn);
    }
    #endif

    #ifdef USE_SPI_INT
        /* 配置 SPI 中断优先级并使能中断 */
        HAL_NVIC_SetPriority(SPIx_IRQn, 1, 0);
        HAL_NVIC_EnableIRQ(SPIx_IRQn);
    #endif
}
```

94.4.3第 3 步：SPI DMA 传输设置和 MPU 配置

SPI DMA 方式主要通过函数 bsp_spiTransfer 实现数据传输（代码里面的查询和中断方式请忽略）：

```
/*
*********************************************************************************************************
*                                    选择 DMA，中断或者查询方式
*********************************************************************************************************
*/
#define USE_SPI_DMA    /* DMA 方式  */
//#define USE_SPI_INT    /* 中断方式 */
//#define USE_SPI_POLL   /* 查询方式 */

/* 查询模式 */
#if defined (USE_SPI_POLL)

uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];

/* 中断模式 */
#elif defined (USE_SPI_INT)

uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];

/* DMA 模式使用的 SRAM4 */
#elif defined (USE_SPI_DMA)
    #if defined ( __CC_ARM )    /* IAR *******/
        __attribute__((section (".RAM_D3"))) uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
        __attribute__((section (".RAM_D3"))) uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];
    #elif defined (__ICCARM__)   /* MDK ********/
        #pragma location = ".RAM_D3"
        uint8_t g_spiTxBuf[SPI_BUFFER_SIZE];
        #pragma location = ".RAM_D3"
        uint8_t g_spiRxBuf[SPI_BUFFER_SIZE];
    #endif
#endif

/*
*********************************************************************************************************
```

```
*    函 数 名: bsp_spiTransfer
*    功能说明: 启动数据传输
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_spiTransfer(void)
{
    if (g_spiLen > SPI_BUFFER_SIZE)
    {
        return;
    }

    /* DMA 方式传输 */
#ifdef USE_SPI_DMA
    wTransferState = TRANSFER_WAIT;

    if(HAL_SPI_TransmitReceive_DMA(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen) != HAL_OK)

    {
        Error_Handler(__FILE__, __LINE__);
    }

    //while (wTransferState == TRANSFER_WAIT)
    //{
        ;
    //}
#endif

    /* 中断方式传输 */
#ifdef USE_SPI_INT
    wTransferState = TRANSFER_WAIT;

    if(HAL_SPI_TransmitReceive_IT(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen) != HAL_OK)

    {
        Error_Handler(__FILE__, __LINE__);
    }

    //while (wTransferState == TRANSFER_WAIT)
    //{
        ;
    //}
#endif

    /* 查询方式传输 */
#ifdef USE_SPI_POLL
    if(HAL_SPI_TransmitReceive(&hspi, (uint8_t*)g_spiTxBuf, (uint8_t *)g_spiRxBuf, g_spiLen, 1000000) !=
HAL_OK)
    {
        Error_Handler(__FILE__, __LINE__);
    }
#endif
}
```

DMA 方式要特别注意两点：

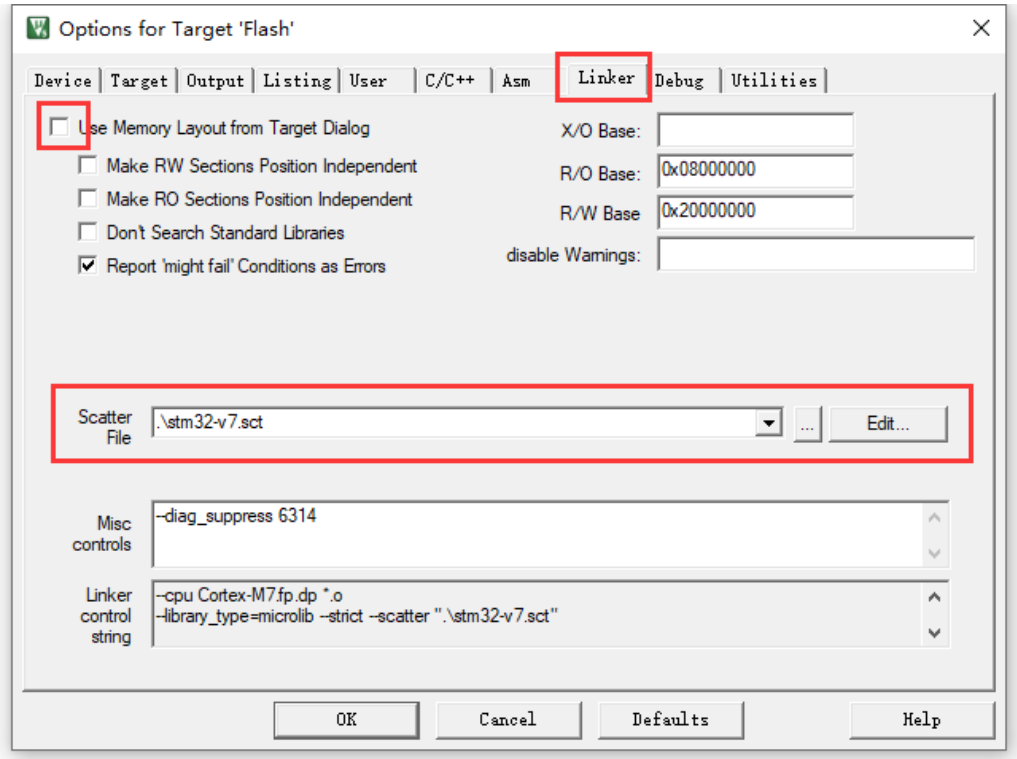
◆ 通过本手册第 26 章的内存块超方便使用方式，将 DMA 缓冲定义到 SRAM4 上。因为本工程是用的

DTCM 做的主 RAM 空间，这个空间无法使用通用 DMA1 和 DMA2。

- 由于程序里面开启了数据 Cache，会造成 DMA 和 CPU 访问 SRAM4 数据不一致的问题，特此将 SRAM4 空间关闭 Cache。

```
/* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress      = 0x38000000;
MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

分散加载设置：



94.4.4第 4 步：应用代码设计

应用部分的代码设计如下：

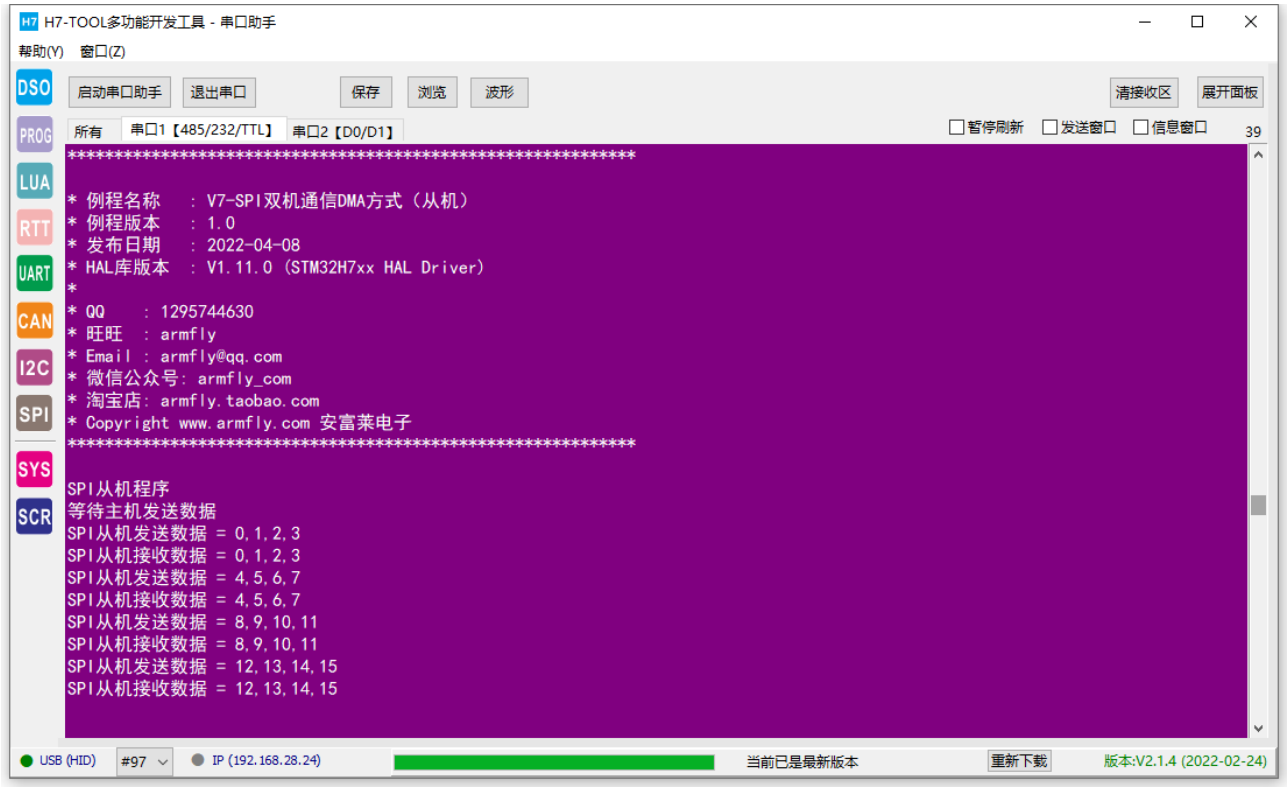
```
/*
*********************************************************************************************************
*    函 数 名: DemoSpiSlave
*    功能说明: SPI 从机通信
*    形    参：无
```

```
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoSpiSlave(void)
{
    uint8_t count = 0;

    /**************设置 SPI Flash 片选上拉，防止影响 **************/
    {
        GPIO_InitTypeDef gpio_init;

        /* 打开 GPIO 时钟 */
        __HAL_RCC_GPIOD_CLK_ENABLE();

        gpio_init.Mode = GPIO_MODE_OUTPUT_PP;  /* 设置推挽输出 */
        gpio_init.Pull = GPIO_NOPULL;          /* 上下拉电阻不使能 */
        gpio_init.Speed = GPIO_SPEED_HIGH;     /* GPIO 速度等级 */
        gpio_init.Pin = GPIO_PIN_13;
        HAL_GPIO_Init(GPIOD, &gpio_init);

        GPIOD->BSRR = GPIO_PIN_13;
    }

     sfDispMenu();      /* 打印命令提示 */

     bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 上电后，准备接收主机命令 */
    g_spiTxBuf[0] = count++;
    g_spiTxBuf[1] = count++;
    g_spiTxBuf[2] = count++;
    g_spiTxBuf[3] = count++;
    g_spiLen = 4;
    bsp_spiTransfer();

     while(1)
     {
         bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

         /* 判断定时器超时时间 */
         if (bsp_CheckTimer(0))
         {
             /* 每隔 100ms 进来一次 */
             bsp_LedToggle(2);
         }

        if (wTransferState != TRANSFER_WAIT)
        {
            printf("SPI 从机发送数据 = %d,%d,%d,%d\r\n", g_spiTxBuf[0], g_spiTxBuf[1], g_spiTxBuf[2],
                                                         g_spiTxBuf[3]);
            printf("SPI 从机接收数据 = %d,%d,%d,%d\r\n", g_spiRxBuf[0], g_spiRxBuf[1], g_spiRxBuf[2],
                                                         g_spiRxBuf[3]);
            g_spiTxBuf[0] = count++;
            g_spiTxBuf[1] = count++;
            g_spiTxBuf[2] = count++;
            g_spiTxBuf[3] = count++;
            g_spiLen = 4;
            bsp_spiTransfer();
```

```
        }
    }
}
```

从机设计这里主要注意两点：

- 上电后优先准备一次从机数据接收。
- 从机接收到主机发送的数据后，将接收到的数据打印出来并打印发送的数据。



94.5 SPI DMA 主从机使用注意事项

- 大家根据自己接线的稳定性，可以适当调节 SPI 主机和从机的时钟速度，其中从机的时钟速度是可以高于主机速度的，这样通信的容错性更好些。

94.6 SPI DMA 主从机驱动移植和使用

移植步骤如下：

- 第 1 步：复制 bsp_spi_bus.c，bsp_spi_bus.h 到自己的工程目录，并添加到工程里面。
- 第 2 步：根据使用的第几个 SPI，SPI 时钟，SPI 引脚和 DMA 通道等，修改 bsp_spi_bus.c 文件开头的宏定义

```
/*
*********************************************************************************************************
*                                    时钟，引脚，DMA，中断等宏定义
*********************************************************************************************************
*/
#define SPIx                                SPI1
```

```
#define SPIx_CLK_ENABLE()                __HAL_RCC_SPI1_CLK_ENABLE()
#define DMAx_CLK_ENABLE()                __HAL_RCC_DMA2_CLK_ENABLE()

#define SPIx_FORCE_RESET()               __HAL_RCC_SPI1_FORCE_RESET()
#define SPIx_RELEASE_RESET()             __HAL_RCC_SPI1_RELEASE_RESET()

#define SPIx_SCK_CLK_ENABLE()            __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_SCK_GPIO                    GPIOB
#define SPIx_SCK_PIN                     GPIO_PIN_3
#define SPIx_SCK_AF                          GPIO_AF5_SPI1

#define SPIx_MISO_CLK_ENABLE()           __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MISO_GPIO                   GPIOB
#define SPIx_MISO_PIN                    GPIO_PIN_4
#define SPIx_MISO_AF                     GPIO_AF5_SPI1

#define SPIx_MOSI_CLK_ENABLE()           __HAL_RCC_GPIOB_CLK_ENABLE()
#define SPIx_MOSI_GPIO                   GPIOB
#define SPIx_MOSI_PIN                    GPIO_PIN_5
#define SPIx_MOSI_AF                     GPIO_AF5_SPI1

#define SPIx_TX_DMA_STREAM               DMA2_Stream3
#define SPIx_RX_DMA_STREAM               DMA2_Stream2

#define SPIx_TX_DMA_REQUEST              DMA_REQUEST_SPI1_TX
#define SPIx_RX_DMA_REQUEST              DMA_REQUEST_SPI1_RX

#define SPIx_DMA_TX_IRQn                 DMA2_Stream3_IRQn
#define SPIx_DMA_RX_IRQn                 DMA2_Stream2_IRQn

#define SPIx_DMA_TX_IRQHandler           DMA2_Stream3_IRQHandler
#define SPIx_DMA_RX_IRQHandler           DMA2_Stream2_IRQHandler

#define SPIx_IRQn                        SPI1_IRQn
#define SPIx_IRQHandler                  SPI1_IRQHandler
```

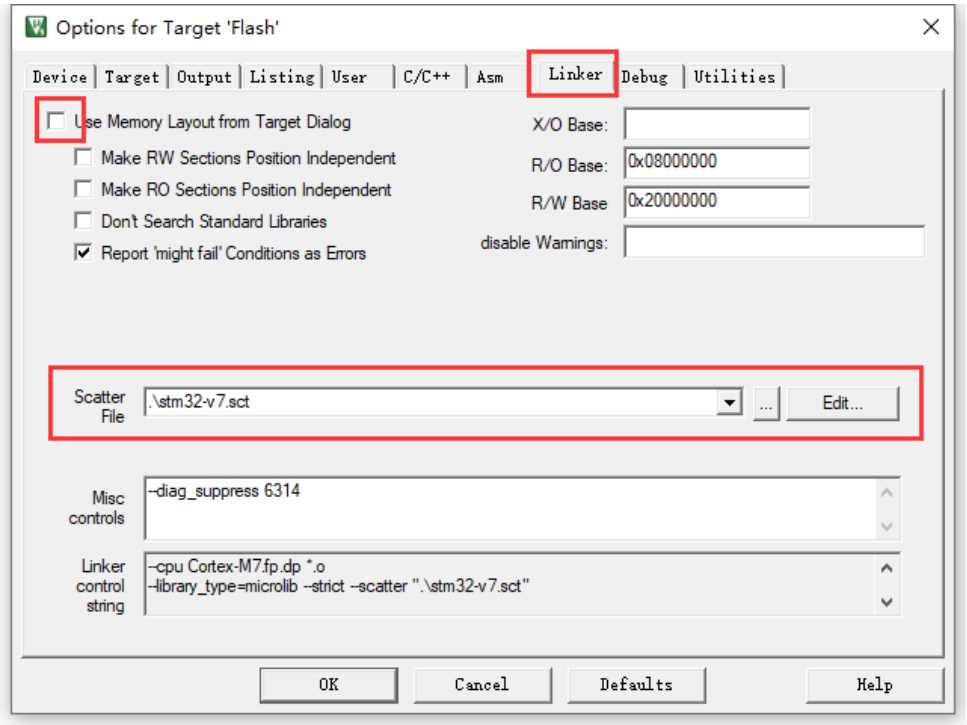
- 第 3 步：如果使用 DMA 方式的话，请不要使用 TCM RAM，因为通用 DMA1 和 DMA2 不支持。并为了防止 DMA 和 CPU 同时访问 DMA 缓冲造成的数据一致性问题，将这块空间关闭 Cache 处理，比如使用的 SRAM4：

```
/* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress      = 0x38000000;
MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

- 第 8 步：初始化 SPI。

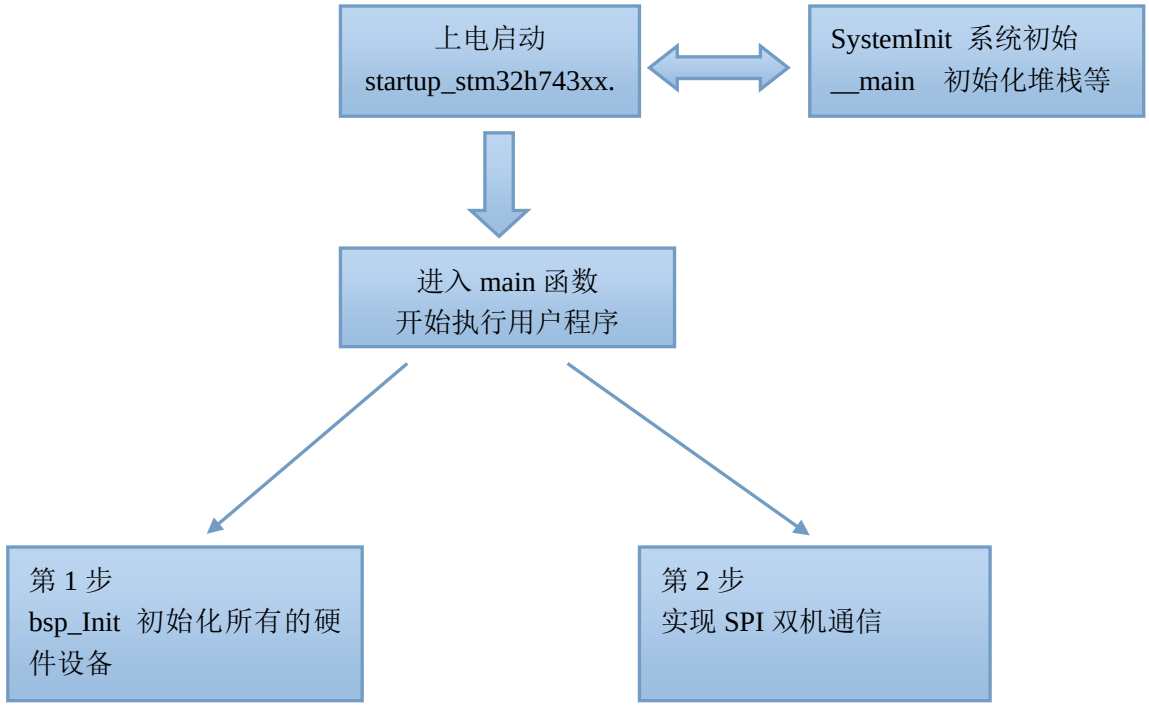
```
/* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
bsp_InitSPIBus();  /* 配置 SPI 总线 */
```

- 第 9 步：SPI Flash 驱动主要用到 HAL 库的 SPI 驱动文件，简单省事些可以添加所有 HAL 库 C 源文件进来。
- 第 10 步：应用方法看本章节配套例子即可。特别注意分散加载设置：



94.7 实验例程设计框架

通过程序设计框架，让大家先对配套例程有一个全面的认识，然后再理解细节，本次实验例程的设计框架如下：



- 第 1 阶段，上电启动阶段：

  这部分在第 14 章进行了详细说明。

- 第 2 阶段，进入 main 函数：
  - 第 1 部分，硬件初始化，主要是 MPU，Cache，HAL 库，系统时钟，滴答定时器和 LED。
  - 第 2 部分，应用程序设计部分，实现 SPI 双击通信。

94.8 实验例程说明（MDK）

配套例子：

V7-070_SPI DMA 双机通信(主机)

V7-071_SPI DMA 双机通信(从机)

实验目的：

3. 学习 SPI Flash 主从机通信实现。

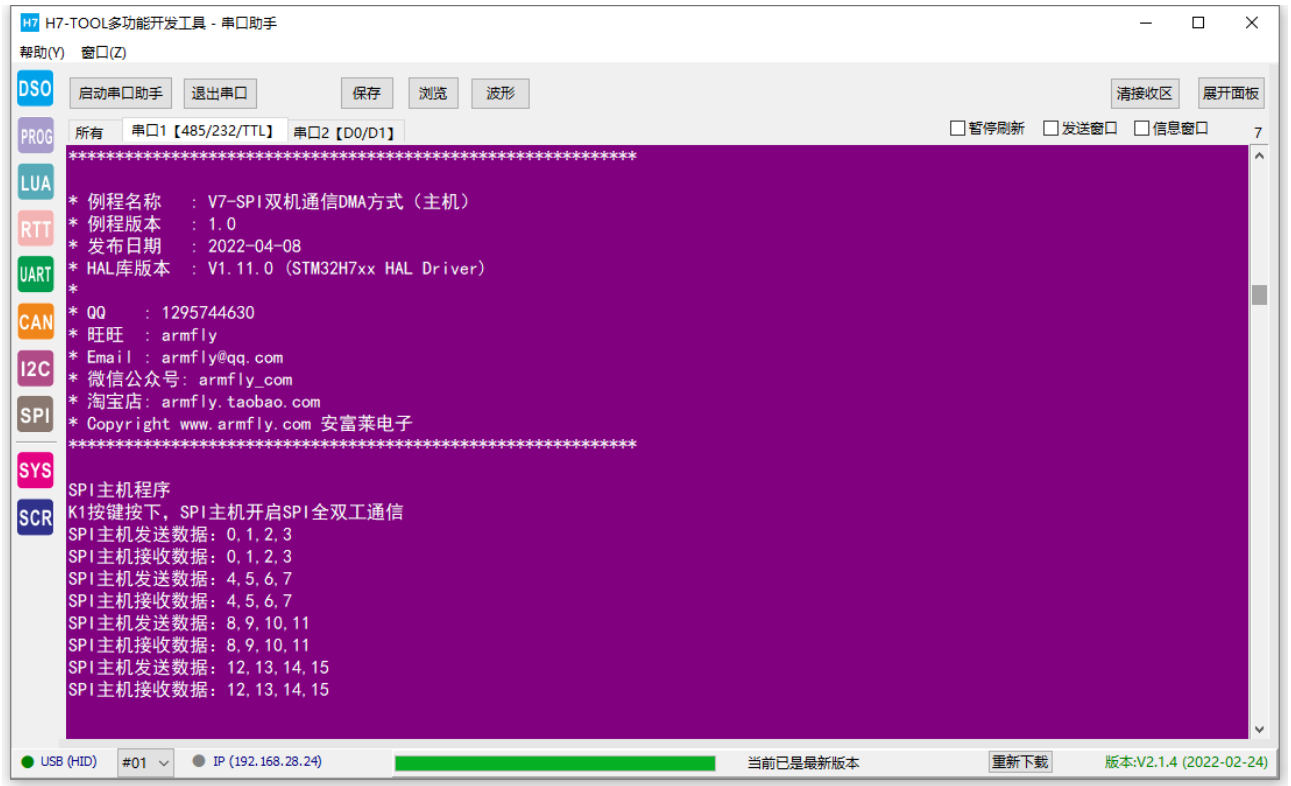
实验操作：

1. K1 按键按下，主机打印。
2. SPI 从机等待主机消息。

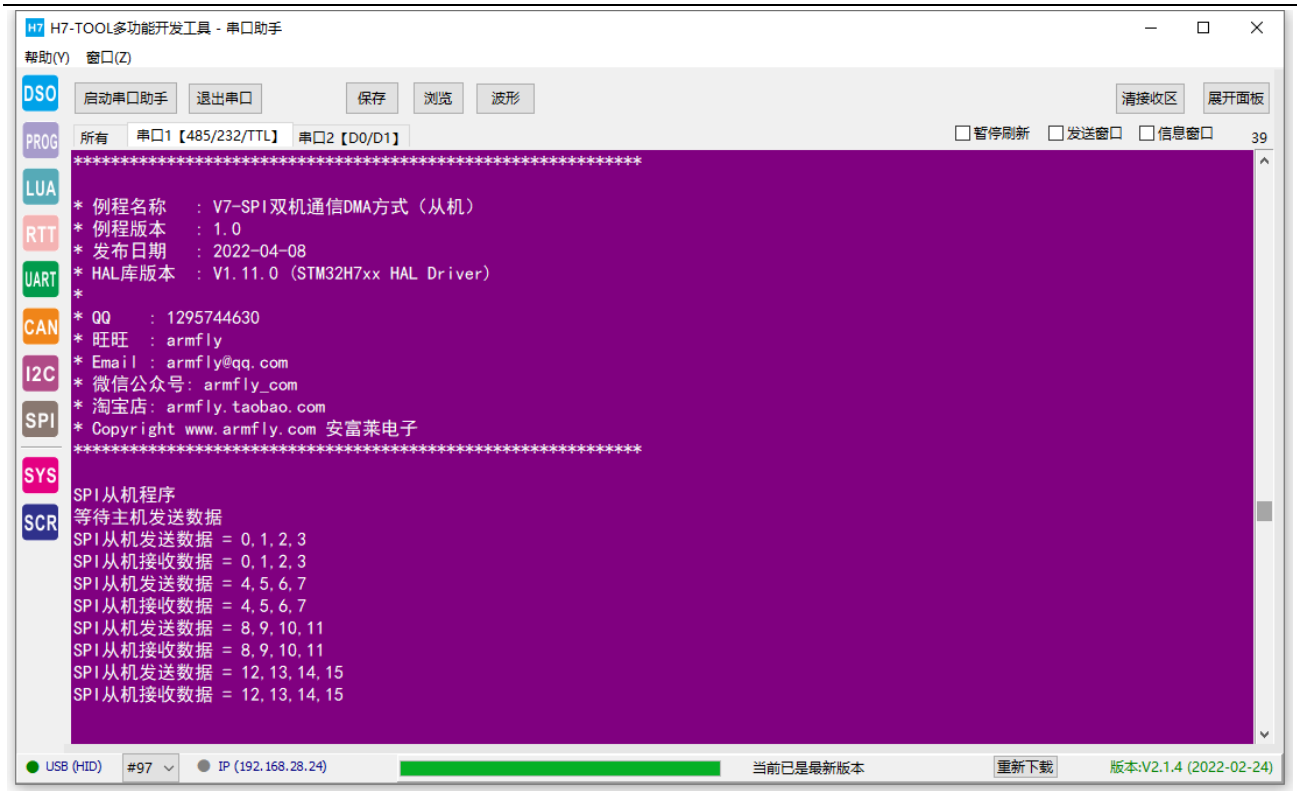
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1。

主机：

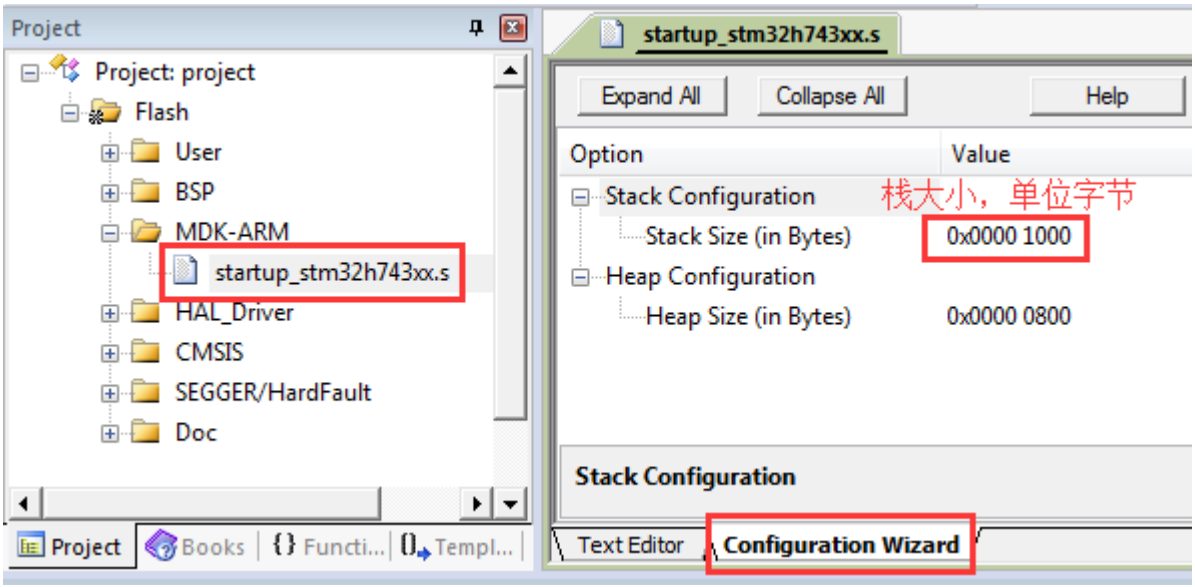


从机：

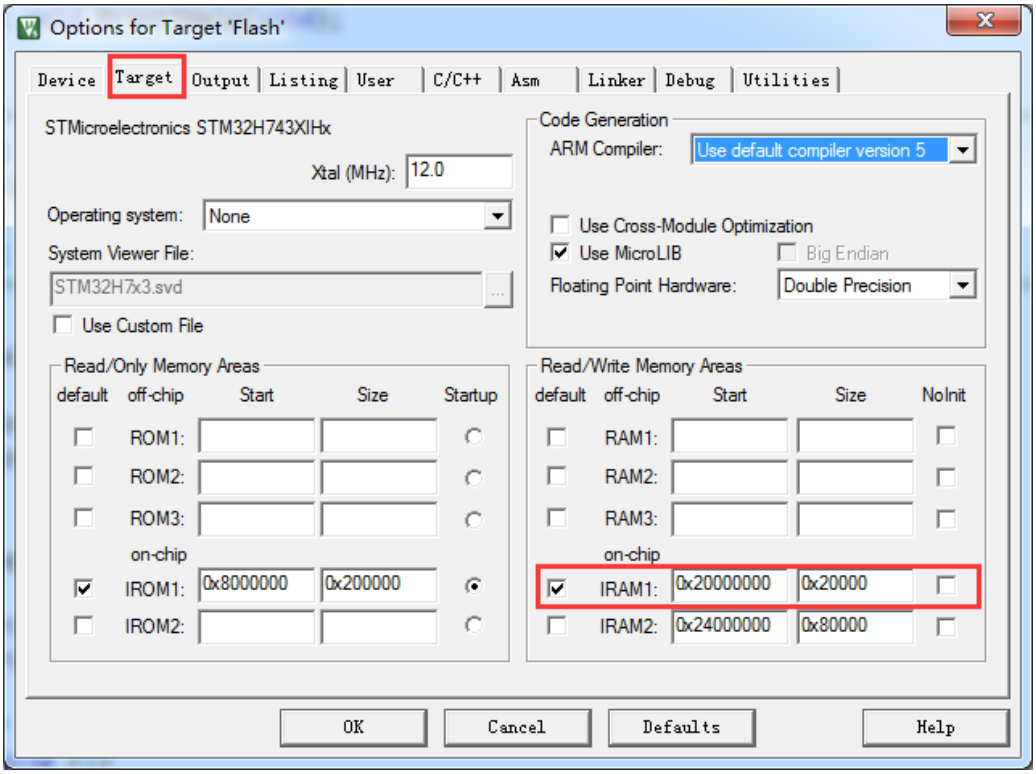


程序设计：

- **系统栈大小分配：**



- **RAM 空间用的 DTCM：**



◆ 硬件外设初始化

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();
```

```
    /*
       Event Recorder：
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();      /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();    /* 初始化滴答定时器 */
    bsp_InitLPUart();   /* 初始化串口 */
    bsp_InitExtIO();    /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();      /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitSPIBus();  /* 配置 SPI 总线 */

}
```

- **MPU 配置和 Cache 配置：**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区以及 SRAM4

```
/*
*********************************************************************************************************
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

```
    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

- **每 10ms 调用一次按键处理：**

  按键处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
```

```
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能：**

主机程序实现如下操作：

```
/*
*********************************************************************************************************
*    函 数 名: DemoSpiMaster
*    功能说明: SPI 主机通信
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoSpiMaster(void)
{
    uint8_t count = 0;
    uint8_t ucKeyCode;      /* 按键代码 */

    /**************设置 SPI Flash 片选上拉，防止影响 **************/
    {
        GPIO_InitTypeDef gpio_init;

        /* 打开 GPIO 时钟 */
        __HAL_RCC_GPIOD_CLK_ENABLE();

        gpio_init.Mode = GPIO_MODE_OUTPUT_PP;
        gpio_init.Pull = GPIO_NOPULL;
        gpio_init.Speed = GPIO_SPEED_HIGH;
        gpio_init.Pin = GPIO_PIN_13;
        HAL_GPIO_Init(GPIOD, &gpio_init);

        GPIOD->BSRR = GPIO_PIN_13;
    }

    sfDispMenu();       /* 打印命令提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    while(1)
    {
        bsp_Idle();         /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }
```

```
        /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
         ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
         if (ucKeyCode != KEY_NONE)
         {
              switch (ucKeyCode)
              {
                   case KEY_DOWN_K1:           /* K1 键按下，发送数据给从机*/
                    g_spiTxBuf[0] = count++;
                    g_spiTxBuf[1] = count++;
                    g_spiTxBuf[2] = count++;
                    g_spiTxBuf[3] = count++;
                    g_spiLen = 4;
                    printf("SPI 主机发送数据：%d,%d,%d,%d\r\n",
                           g_spiTxBuf[0],g_spiTxBuf[1],g_spiTxBuf[2],g_spiTxBuf[3]);

                    bsp_spiTransfer();
                    printf("SPI 主机接收数据：%d,%d,%d,%d\r\n",
                            g_spiRxBuf[0],g_spiRxBuf[1],g_spiRxBuf[2],g_spiRxBuf[3]);
                       break;
                   default:
                       /* 其它的键值不处理 */
                       break;
              }

         }
    }
}
```

从机实现程序如下：

```
/*
*********************************************************************************************************
*    函 数 名: DemoSpiSlave
*    功能说明: SPI 从机通信
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoSpiSlave(void)
{
    uint8_t count = 0;

    /**************设置 SPI Flash 片选上拉，防止影响 **************/
    {
        GPIO_InitTypeDef gpio_init;

        /* 打开 GPIO 时钟 */
        __HAL_RCC_GPIOD_CLK_ENABLE();

        gpio_init.Mode = GPIO_MODE_OUTPUT_PP;  /* 设置推挽输出 */
        gpio_init.Pull = GPIO_NOPULL;          /* 上下拉电阻不使能 */
        gpio_init.Speed = GPIO_SPEED_HIGH;     /* GPIO 速度等级 */
        gpio_init.Pin = GPIO_PIN_13;
        HAL_GPIO_Init(GPIOD, &gpio_init);

        GPIOD->BSRR = GPIO_PIN_13;
    }
```

```
    sfDispMenu();      /* 打印命令提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

   /* 上电后，准备接收主机命令 */
   g_spiTxBuf[0] = count++;
   g_spiTxBuf[1] = count++;
   g_spiTxBuf[2] = count++;
   g_spiTxBuf[3] = count++;
   g_spiLen = 4;
   bsp_spiTransfer();

    while(1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

       if (wTransferState != TRANSFER_WAIT)
       {
           printf("SPI 从机发送数据 = %d,%d,%d,%d\r\n", g_spiTxBuf[0], g_spiTxBuf[1], g_spiTxBuf[2],
                                                        g_spiTxBuf[3]);
           printf("SPI 从机接收数据 = %d,%d,%d,%d\r\n", g_spiRxBuf[0], g_spiRxBuf[1], g_spiRxBuf[2],
                                                        g_spiRxBuf[3]);
           g_spiTxBuf[0] = count++;
           g_spiTxBuf[1] = count++;
           g_spiTxBuf[2] = count++;
           g_spiTxBuf[3] = count++;
           g_spiLen = 4;
           bsp_spiTransfer();
       }
    }
}
```

94.9 实验例程说明（IAR）

配套例子：

V7-070_SPI DMA 双机通信(主机)

V7-071_SPI DMA 双机通信(从机)

实验目的：

1. 学习 SPI Flash 主从机通信实现。

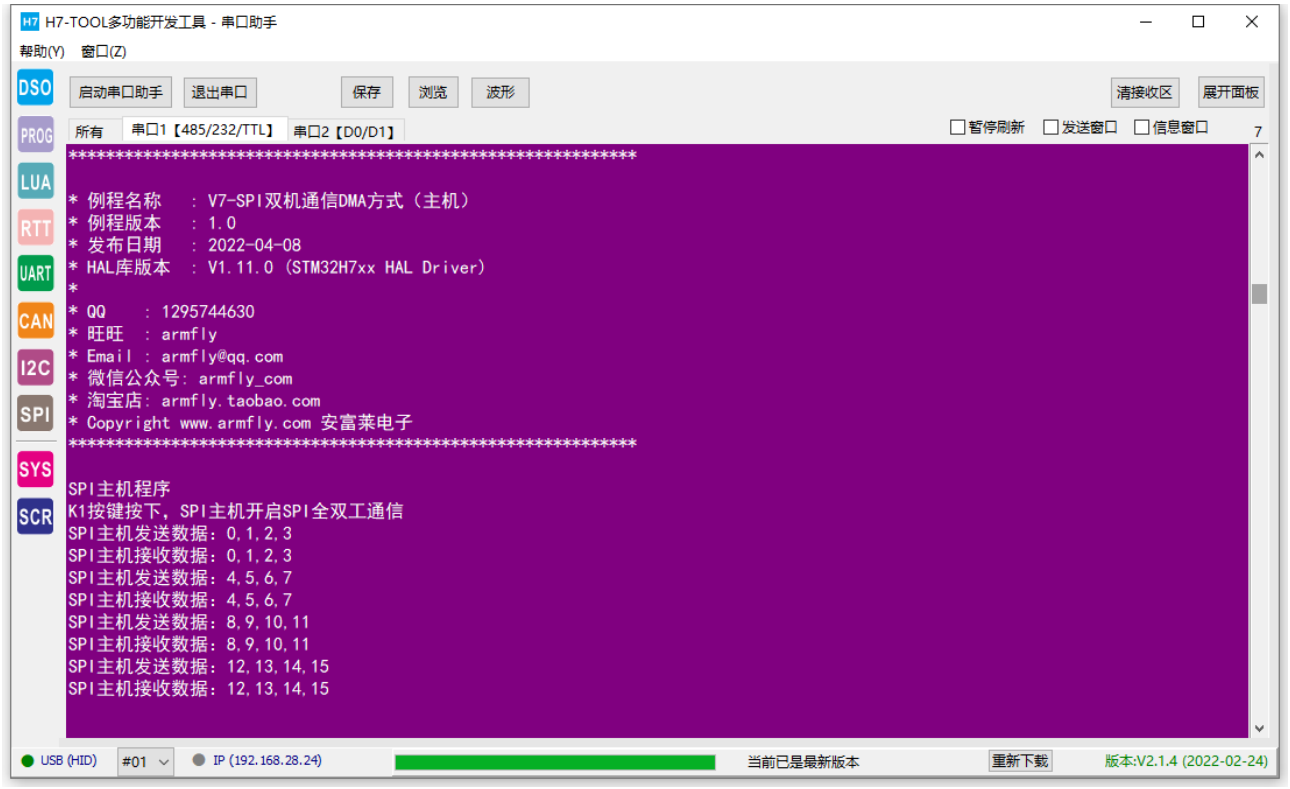
实验操作：

1. K1 按键按下，主机打印。
2. SPI 从机等待主机消息。

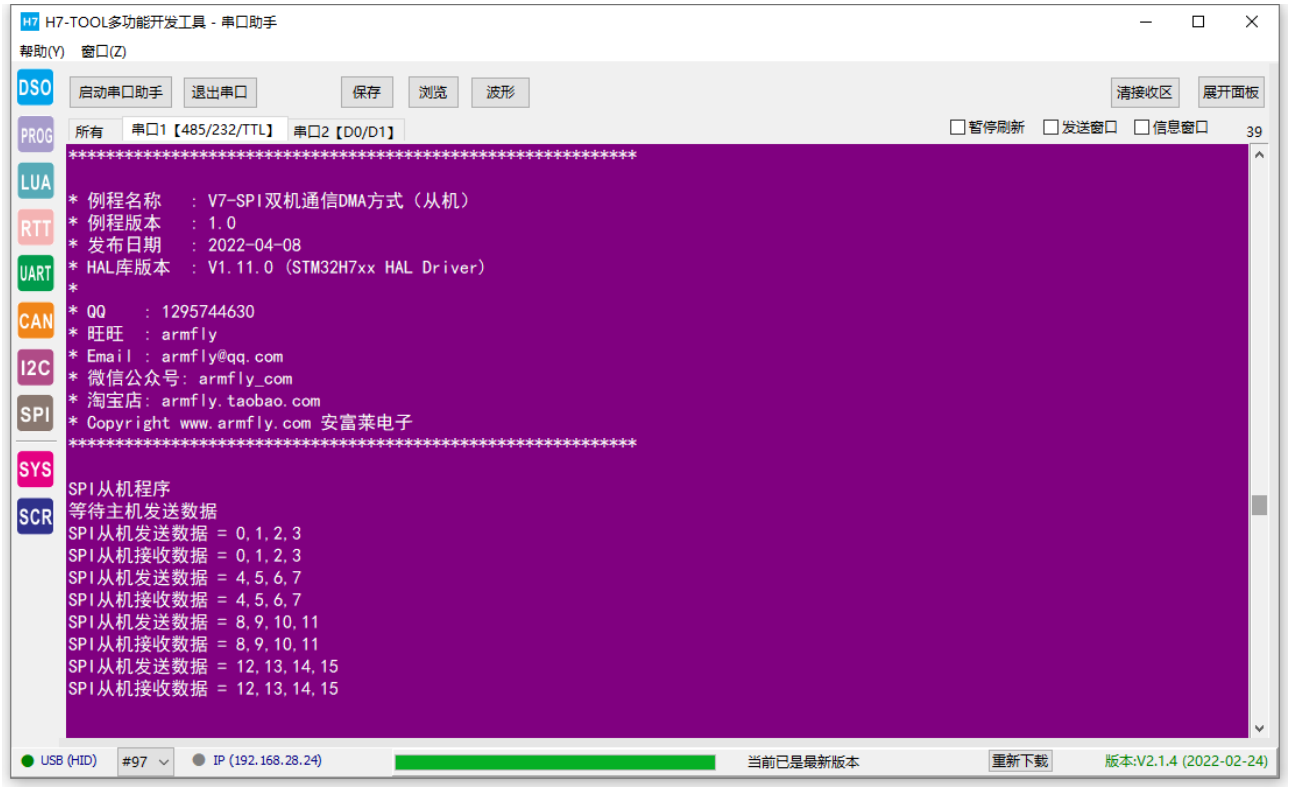
上电后串口打印的信息：

波特率 115200，数据位 8，奇偶校验位无，停止位 1。

主机：

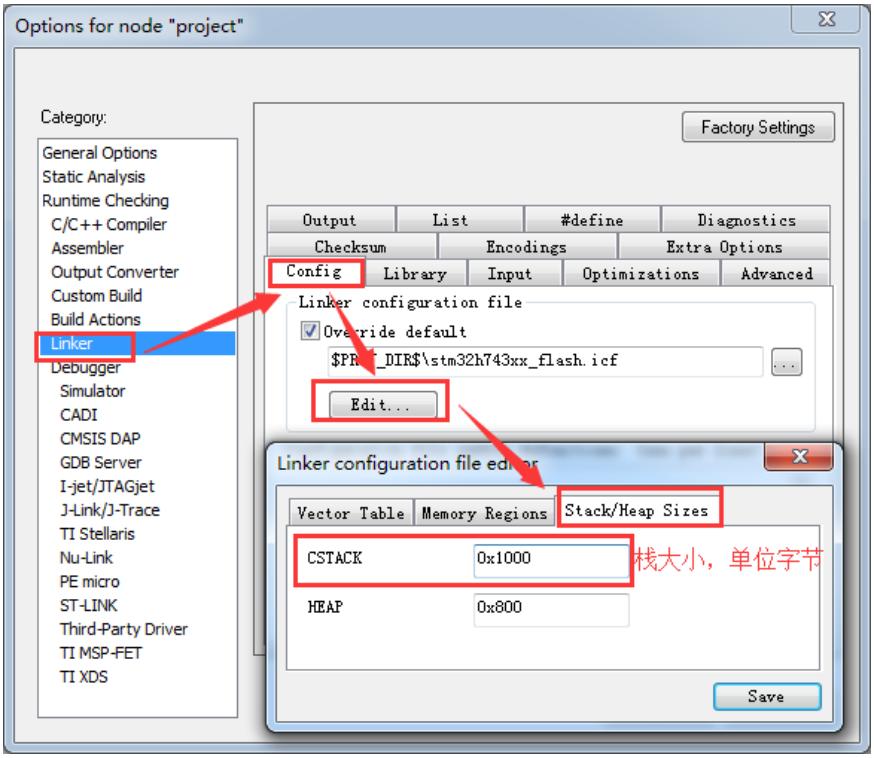


从机：

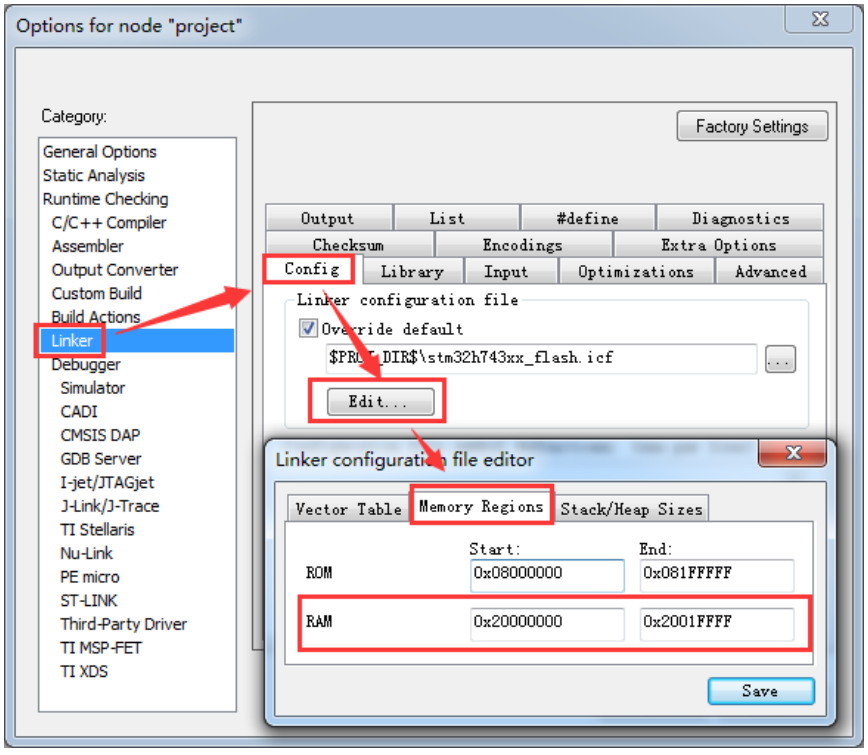


程序设计：

◆ **系统栈大小分配：**



◆ **RAM 空间用的 DTCM：**



◆ **硬件外设初始化**

硬件外设的初始化是在 bsp.c 文件实现：

```
/*
*********************************************************************************************************
```

```
*    函 数 名: bsp_Init
*    功能说明: 初始化所有的硬件设备。该函数配置 CPU 寄存器和外设的寄存器并初始化一些全局变量。只需要调用一次
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_Init(void)
{
    /* 配置 MPU */
    MPU_Config();

    /* 使能 L1 Cache */
    CPU_CACHE_Enable();

    /*
       STM32H7xx HAL 库初始化，此时系统用的还是 H7 自带的 64MHz，HSI 时钟:
       - 调用函数 HAL_InitTick，初始化滴答时钟中断 1ms。
       - 设置 NVIV 优先级分组为 4。
     */
    HAL_Init();

    /*
       配置系统时钟到 400MHz
       - 切换使用 HSE。
       - 此函数会更新全局变量 SystemCoreClock，并重新配置 HAL_InitTick。
    */
    SystemClock_Config();

    /*
       Event Recorder:
       - 可用于代码执行时间测量，MDK5.25 及其以上版本才支持，IAR 不支持。
       - 默认不开启，如果要使能此选项，务必看 V7 开发板用户手册第 xx 章
    */
#if Enable_EventRecorder == 1
    /* 初始化 EventRecorder 并开启 */
    EventRecorderInitialize(EventRecordAll, 1U);
    EventRecorderStart();
#endif

    bsp_InitDWT();      /* 初始化 DWT 时钟周期计数器 */
    bsp_InitKey();      /* 按键初始化，要放在滴答定时器之前，因为按钮检测是通过滴答定时器扫描 */
    bsp_InitTimer();    /* 初始化滴答定时器 */
    bsp_InitLPUart();   /* 初始化串口 */
    bsp_InitExtIO();    /* 初始化 FMC 总线 74HC574 扩展 IO. 必须在 bsp_InitLed()前执行 */
    bsp_InitLed();      /* 初始化 LED */
    bsp_InitExtSDRAM(); /* 初始化 SDRAM */

    /* 针对不同的应用程序，添加需要的底层驱动模块初始化函数 */
    bsp_InitSPIBus();  /* 配置 SPI 总线 */

}
```

- **MPU 配置和 Cache 配置:**

  数据 Cache 和指令 Cache 都开启。配置了 AXI SRAM 区（本例子未用到 AXI SRAM）和 FMC 的扩展 IO 区以及 SRAM4

```
/*
*********************************************************************************************************
```

```
*    函 数 名: MPU_Config
*    功能说明: 配置 MPU
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void MPU_Config( void )
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    /* 禁止 MPU */
    HAL_MPU_Disable();

    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x24000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);


    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x60000000;
    MPU_InitStruct.Size             = ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 SRAM4 的 MPU 属性为 Non-cacheable */
    MPU_InitStruct.Enable           = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress      = 0x38000000;
    MPU_InitStruct.Size             = MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable     = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable      = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable      = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number           = MPU_REGION_NUMBER2;
    MPU_InitStruct.TypeExtField     = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec      = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

```
    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

/*
*********************************************************************************************************
*    函 数 名: CPU_CACHE_Enable
*    功能说明: 使能 L1 Cache
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
static void CPU_CACHE_Enable(void)
{
    /* 使能 I-Cache */
    SCB_EnableICache();

    /* 使能 D-Cache */
    SCB_EnableDCache();
}
```

◆ **每 10ms 调用一次按键处理：**

按键处理是在滴答定时器中断里面实现，每 10ms 执行一次检测。

```
/*
*********************************************************************************************************
*    函 数 名: bsp_RunPer10ms
*    功能说明: 该函数每隔 10ms 被 Systick 中断调用 1 次。详见 bsp_timer.c 的定时中断服务程序。一些处理时间要求
*              不严格的任务可以放在此函数。比如：按键扫描、蜂鸣器鸣叫控制等。
*    形    参: 无
*    返 回 值: 无
*********************************************************************************************************
*/
void bsp_RunPer10ms(void)
{
    bsp_KeyScan10ms();
}
```

◆ **主功能：**

主机程序实现如下操作：

```
/*
*********************************************************************************************************
*    函 数 名: DemoSpiMaster
*    功能说明: SPI 主机通信
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoSpiMaster(void)
{
    uint8_t count = 0;
    uint8_t ucKeyCode;      /* 按键代码 */

    /**************设置 SPI Flash 片选上拉，防止影响 **************/
    {
        GPIO_InitTypeDef gpio_init;
```

```
        /* 打开 GPIO 时钟 */
        __HAL_RCC_GPIOD_CLK_ENABLE();

        gpio_init.Mode = GPIO_MODE_OUTPUT_PP;
        gpio_init.Pull = GPIO_NOPULL;
        gpio_init.Speed = GPIO_SPEED_HIGH;
        gpio_init.Pin = GPIO_PIN_13;
        HAL_GPIO_Init(GPIOD, &gpio_init);

        GPIOD->BSRR = GPIO_PIN_13;
    }

    sfDispMenu();      /* 打印命令提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    while(1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

       /* 按键滤波和检测由后台 systick 中断服务程序实现，我们只需要调用 bsp_GetKey 读取键值即可。 */
        ucKeyCode = bsp_GetKey();   /* 读取键值, 无键按下时返回 KEY_NONE = 0 */
        if (ucKeyCode != KEY_NONE)
        {
            switch (ucKeyCode)
            {
                case KEY_DOWN_K1:           /* K1 键按下，发送数据给从机*/
                  g_spiTxBuf[0] = count++;
                  g_spiTxBuf[1] = count++;
                  g_spiTxBuf[2] = count++;
                  g_spiTxBuf[3] = count++;
                  g_spiLen = 4;
                  printf("SPI 主机发送数据：%d,%d,%d,%d\r\n",
                         g_spiTxBuf[0],g_spiTxBuf[1],g_spiTxBuf[2],g_spiTxBuf[3]);

                  bsp_spiTransfer();
                  printf("SPI 主机接收数据：%d,%d,%d,%d\r\n",
                          g_spiRxBuf[0],g_spiRxBuf[1],g_spiRxBuf[2],g_spiRxBuf[3]);
                     break;
                default:
                    /* 其它的键值不处理 */
                    break;
            }

        }
    }
}
```

从机实现程序如下：

```
/*
```

```
*********************************************************************************************************
*    函 数 名: DemoSpiSlave
*    功能说明: SPI 从机通信
*    形    参：无
*    返 回 值: 无
*********************************************************************************************************
*/
void DemoSpiSlave(void)
{
    uint8_t count = 0;

    /**************设置 SPI Flash 片选上拉，防止影响 **************/
    {
        GPIO_InitTypeDef gpio_init;

        /* 打开 GPIO 时钟 */
        __HAL_RCC_GPIOD_CLK_ENABLE();

        gpio_init.Mode = GPIO_MODE_OUTPUT_PP;  /* 设置推挽输出 */
        gpio_init.Pull = GPIO_NOPULL;          /* 上下拉电阻不使能 */
        gpio_init.Speed = GPIO_SPEED_HIGH;     /* GPIO 速度等级 */
        gpio_init.Pin = GPIO_PIN_13;
        HAL_GPIO_Init(GPIOD, &gpio_init);

        GPIOD->BSRR = GPIO_PIN_13;
    }

    sfDispMenu();      /* 打印命令提示 */

    bsp_StartAutoTimer(0, 100); /* 启动 1 个 100ms 的自动重装的定时器 */

    /* 上电后，准备接收主机命令 */
    g_spiTxBuf[0] = count++;
    g_spiTxBuf[1] = count++;
    g_spiTxBuf[2] = count++;
    g_spiTxBuf[3] = count++;
    g_spiLen = 4;
    bsp_spiTransfer();

    while(1)
    {
        bsp_Idle();        /* 这个函数在 bsp.c 文件。用户可以修改这个函数实现 CPU 休眠和喂狗 */

        /* 判断定时器超时时间 */
        if (bsp_CheckTimer(0))
        {
            /* 每隔 100ms 进来一次 */
            bsp_LedToggle(2);
        }

        if (wTransferState != TRANSFER_WAIT)
        {
            printf("SPI 从机发送数据 = %d,%d,%d,%d\r\n", g_spiTxBuf[0], g_spiTxBuf[1], g_spiTxBuf[2],
                                                         g_spiTxBuf[3]);
            printf("SPI 从机接收数据 = %d,%d,%d,%d\r\n", g_spiRxBuf[0], g_spiRxBuf[1], g_spiRxBuf[2],
                                                         g_spiRxBuf[3]);
            g_spiTxBuf[0] = count++;
            g_spiTxBuf[1] = count++;
```

```
            g_spiTxBuf[2] = count++;
            g_spiTxBuf[3] = count++;
            g_spiLen = 4;
            bsp_spiTransfer();
        }
    }
}
```

94.10 总结

本章节就为大家讲解这么多，推荐大家做实际应用测试，实际项目中需要双机通信，可以考虑这种方式，比较实用。

第95章 附件 A---如何学习

写这个章节的目的，只是想帮助一些初学者提高学习效率，避免走太多的弯路。

1、学习观点

不要排斥英文资料

在有中文资料的情况下，我们当然是看中文资料学习起来效率高一些。但是大部分资料是没有中文译稿的。比如CPU的数据手册只有英文的，固件库源代码中的注释也都是英文的。因此一定要习惯阅读英文手册，不认识的单词可以安装翻译软件协助阅读。

选择性学习

STM32的功能实在太多，和STM32相关的软件组件也多如牛毛。而人的精力是有限的，你就算学习一辈子，也不可能学会所有的东西。

一些不常用的功能和知识点可以先放放，等待使用的时候再学习。几个必须了解的知识点：GPIO、TIM（基本定时器、脉宽测量、PWM输出）、UART（中断模式）、I2C、SPI、ADC（DMA）、DAC、FMC、看门狗等。

如果你已经接到一些项目和产品开发，那么恭喜你，你一定会选择性的学习，因为时间不允许你全面学习。

拿来主义观点

人的一生，时间是最为宝贵的，我们一定要将时间用在刀刃上。别人做好的开源的第三方的代码，如果你测试应用过没有问题，那就属于你的了，你只要学会怎么移植和使用它就足够了。如果别人已经移植好了，那就更方便了，直接拿来使用即可。没有必要再花精力深入代码去研究别人的程序结构了。比如FatFS、uIP、ST固件库、uCOS、emWin。除非你是做基础研究或者查找BUG。

我们一定要将时间腾出来去学习和掌握更多的软件模块。这样，当项目来临时，你就可以从容应对。好比盖房子，你可以直接拿砖砌，而不用在花时间去造砖。

构建自己的代码库

要学会收集整理自己验证过的代码，变为自己代码库的一部分，日积月累。等到做产品和项目时，80%以上的代码来自于代码库。做产品很多时候就是搭积木。你的代码库越丰富，做起产品来也就越快越好。

一些好的工程框架也要学会收集整理。我们在很多时候做项目，都是直接复制某个相近的工程。不可能每次都去重新新建一个工程。

多动手

光看不练是成为不了程序高手的。我们的资料只能帮助你尽快掌握STM32的开发方法，带你尽快入门，同时告诉你大家使用的常用组件，避免你闭门造车。大家一定要动手修改程序，独立设计自己的程序。

如果你没有机会参与项目开发，完全没有目标。那我们建议你多上上论坛，看下别人遇到的问题，尝试自己解决下，这也是一种不错的锻炼方法。

培养兴趣

做程序员这一行，一定得培养起兴趣来。在学习的前期是最容易打退堂鼓的阶段，如果这个时候你放弃了，你的板子可能会开始吃灰。在前期，千万不要被复杂的程序结构所吓倒，一定要反复阅读，多调试跟踪进行分析。当你看懂了，你就会有成就感，才有继续学习的动力。

学会分享

分享也是学习的一部分，养成和大家分享成果的习惯。你可以分享自己的学习心得、调通的代码、帮别人解答问题，当这些内容获得网友认可的时候，你就会有成就感。分享的过程也在培养你的文笔、归纳总结能力。当你分享的内容到达一定数量时，量变到质变，不知不自觉中，你就会成为某一行的专家，你就有可能独立接到项目开发。如果没人找你，那说明你的能力还不够级别，或者不知道通过分享来宣传自己。

2、学习步骤

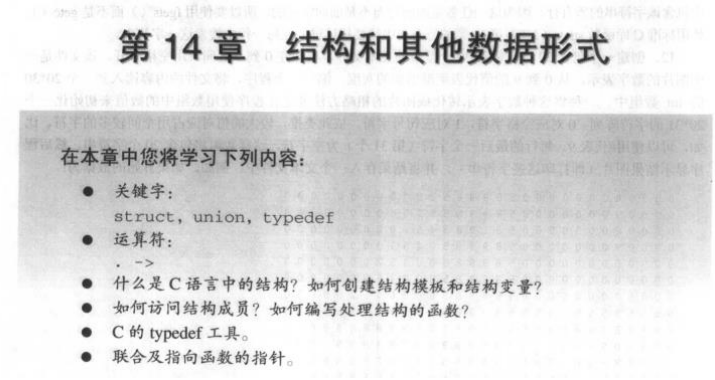
本教程的前10章是专门为初学者准备的。另外，如果对C语言不熟悉，可以阅读c语言书：C Primer Plus（第五版）中文版.pdf

论坛下载：http://www.armbbs.cn/forum.php?mod=viewthread&tid=91219 。

该书受到广大网友推荐，如果看电子档不方便，可以考虑购买一本。

因为STM32相对51单片机复杂了很多，寄存器数量是51单片机的好几倍。因此ST的固件库使用了大量的结构和指针来书写程序。固件库的书写和变量命名非常规范。但是对于一些c语言薄弱的同学来说，看固件库的源码，有点阅读天书的感觉，原因就是对结构和结构指针不熟悉。

对结构体和指针不熟悉的同学，推荐看看“第14章 结构和其他数据形式”【在pdf文档的第398页】


第 14 章　结构和其他数据形式

在本章中您将学习下列内容：

- 关键字：
  struct, union, typedef
- 运算符：
  . ->
- 什么是 C 语言中的结构？如何创建结构模板和结构变量？
- 如何访问结构成员？如何编写处理结构的函数？
- C 的 typedef 工具。
- 联合及指向函数的指针。

对于有一定C语言基础和ARM开发基础的人员，首要目的是快速熟悉STM32固件库的用法，推荐的学习步骤：

（1） 快速浏览CPU参考手册中的FSMC，PWR，RCC，GPIO、USART、interrupts and events等章节。虽然看CPU的DataSheet很枯燥，但是这是最权威的信息，挑重点章节多看几遍比去网上搜索答案更有效，可以毫不夸张地说“阅读DataSheet也是一种技能“。
ST的CPU手册分两个文档，一个叫数据手册（描述不同封装CPU间的差异以及管脚定义），另外一个叫参考手册（详细描述CPU内部结构、外设功能以及各个寄存器的用法）。
如果你对Cortex核心感兴趣，可以阅读 《STM32权威指南（中文）.pdf》

（2） 把一些自己感兴趣的例程跑一遍，主要是熟悉ST固件库中常用函数的用法。顺序可以按 GPIO、USART、TIM、ADC等。

（3） 在学习的过程中，要逐步掌握开发工具的用法。主要是KEIL或IAR集成环境、J-Link/STLINK仿真器。

有一定基础后，可以自己设计或者在别人的基础上设计一个有使用价值的完整的程序，然后共享你的成果，和大家一起分享成功的喜悦。

第96章 附件 B---Embedded Studio 工程模板设计

注，随着本教程的不断更新，后面将选做几个 Embedded Studio 版的例子。

- 模板设计方法：

  http://www.armbbs.cn/forum.php?mod=viewthread&tid=90433 。

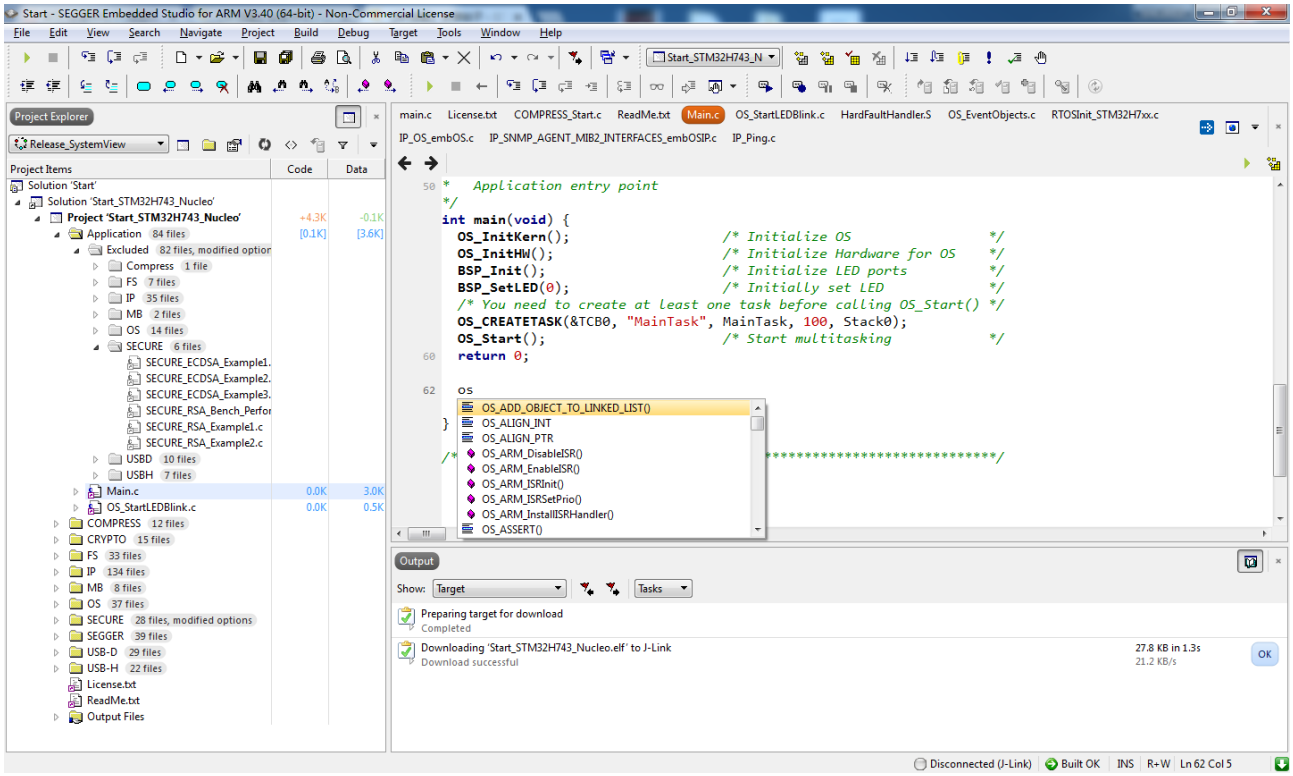
- Embedded Studio 与 MDK 和 IAR 性能比较
  - 优点：
    1. 编辑器功能比 MDK 和 IAR 强劲太多倍，几乎可以媲美 notepad++，VSCode，SI 等编辑器，像代码格式化，文件比较，代码提示，专业的代码高亮，各种风格主题等。
    2. 超快速的工程启动，现在 MDK5 和 IAR8 开机都得反应会，特别是超大工程时，使用 Embedded Studio 的效果非常明显。同时编译速度也飞快，不比 IAR 差，比 MDK 强很多。编译个 HAL 库，带 browse info，速度也是嗖嗖的。
    3. 调试工具方面，Embedded Studio 简直不要太丰富，因为有 JLINK 的支撑，像 JScope，RTT，SystemWiew，embOSView 等都很好用。
    4. 个人使用免费，支持全功能， 含 Windows, macOS 和 Linux 版本。
    5. 已经率先支持 RISC-V 内核。

  - 缺点：
    1. 仅支持自家的 JLINK。
    2. Embedded Studio 的配置用的不顺手，配置选项多，不像 MDK 那种直接图形化配置。
    3. 中文支持不给力。
    4. 都开最高等级优化，ES 使用的 GCC，相比 IAR 和 MDK 生成的代码大小还是稍大些（配置的例子同时支持了 IAR，MDK 和 ES，大家可以实测下）。

  - 待测：
    1. 编译器方面，Embedded Studio 是用的 GCC 和 CLANG，其中 CLANG 是个趋势，现在 MDK 的 AC6 也是用的 CLANG，具体性能还没有做过比较。

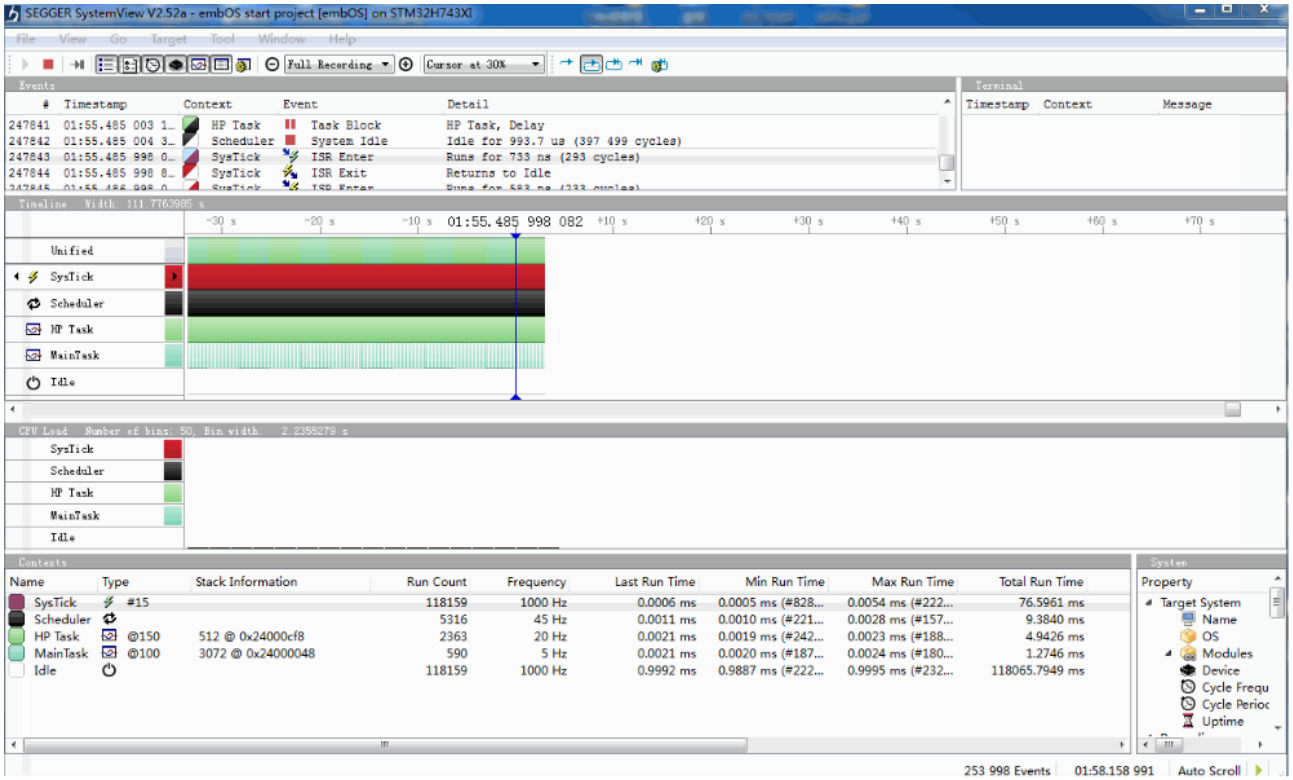
- Embedded Studio 效果：

  代码提示功能比较好用。

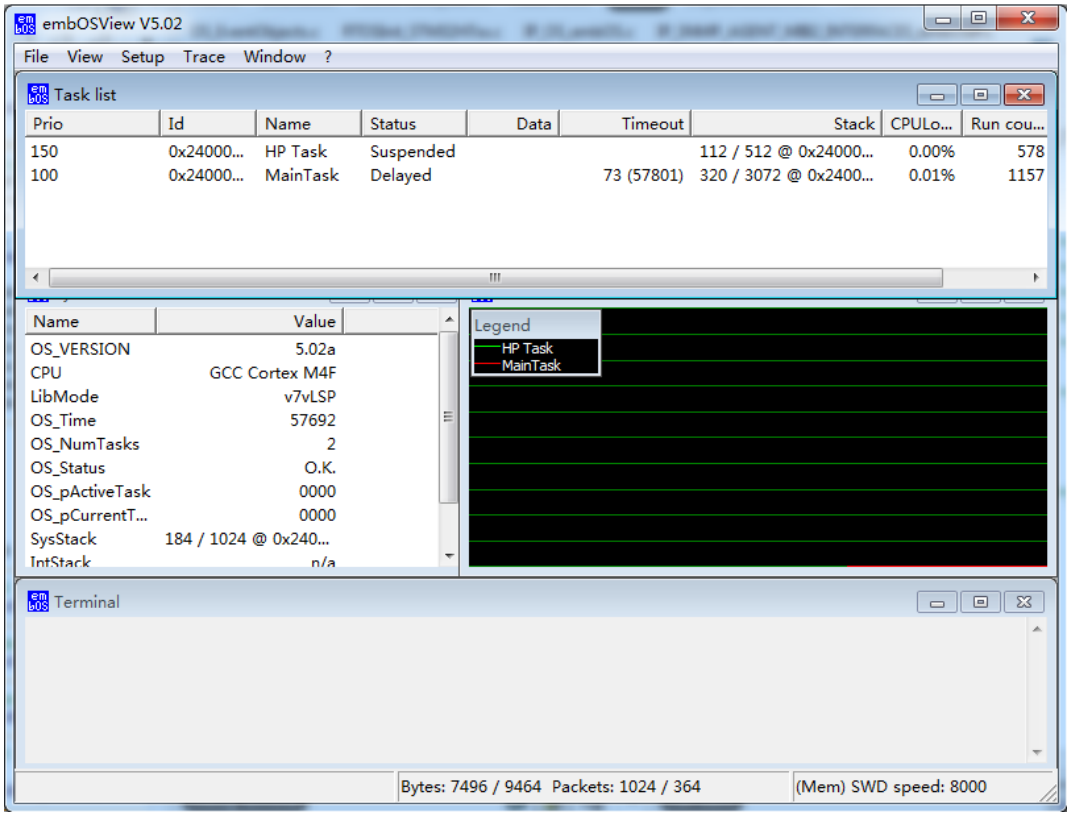


◆ 各种配套小软件：

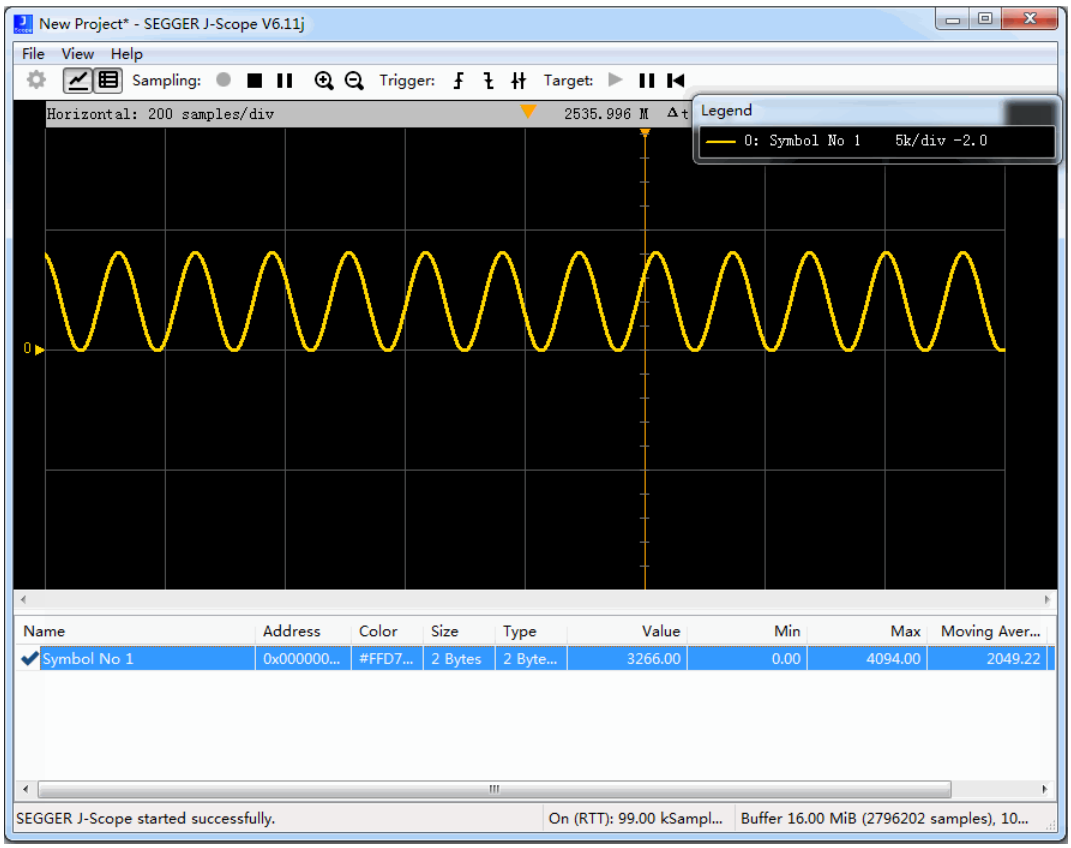
SystemView：



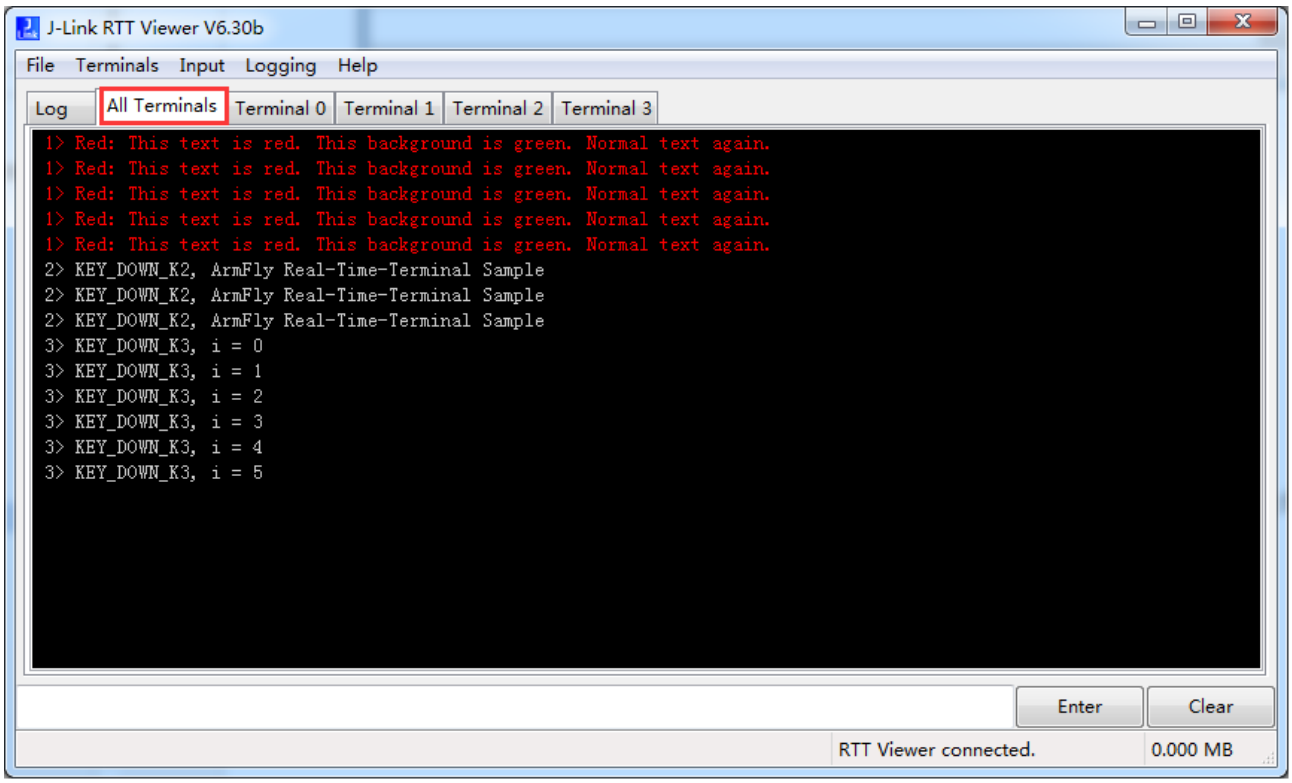
embOSView：


embOSView V5.02
File View Setup Trace Window ?
Task list
Prio Id Name Status Data Timeout Stack CPULo... Run cou...
150 0x24000... HP Task Suspended 112 / 512 @ 0x24000... 0.00% 578
100 0x24000... MainTask Delayed 73 (57801) 320 / 3072 @ 0x2400... 0.01% 1157
Name Value
OS_VERSION 5.02a
CPU GCC Cortex M4F
LibMode v7vLSP
OS_Time 57692
OS_NumTasks 2
OS_Status O.K.
OS_pActiveTask 0000
OS_pCurrentT... 0000
SysStack 184 / 1024 @ 0x240...
IntStack n/a
Legend
HP Task
MainTask
Terminal
Bytes: 7496 / 9464 Packets: 1024 / 364
(Mem) SWD speed: 8000

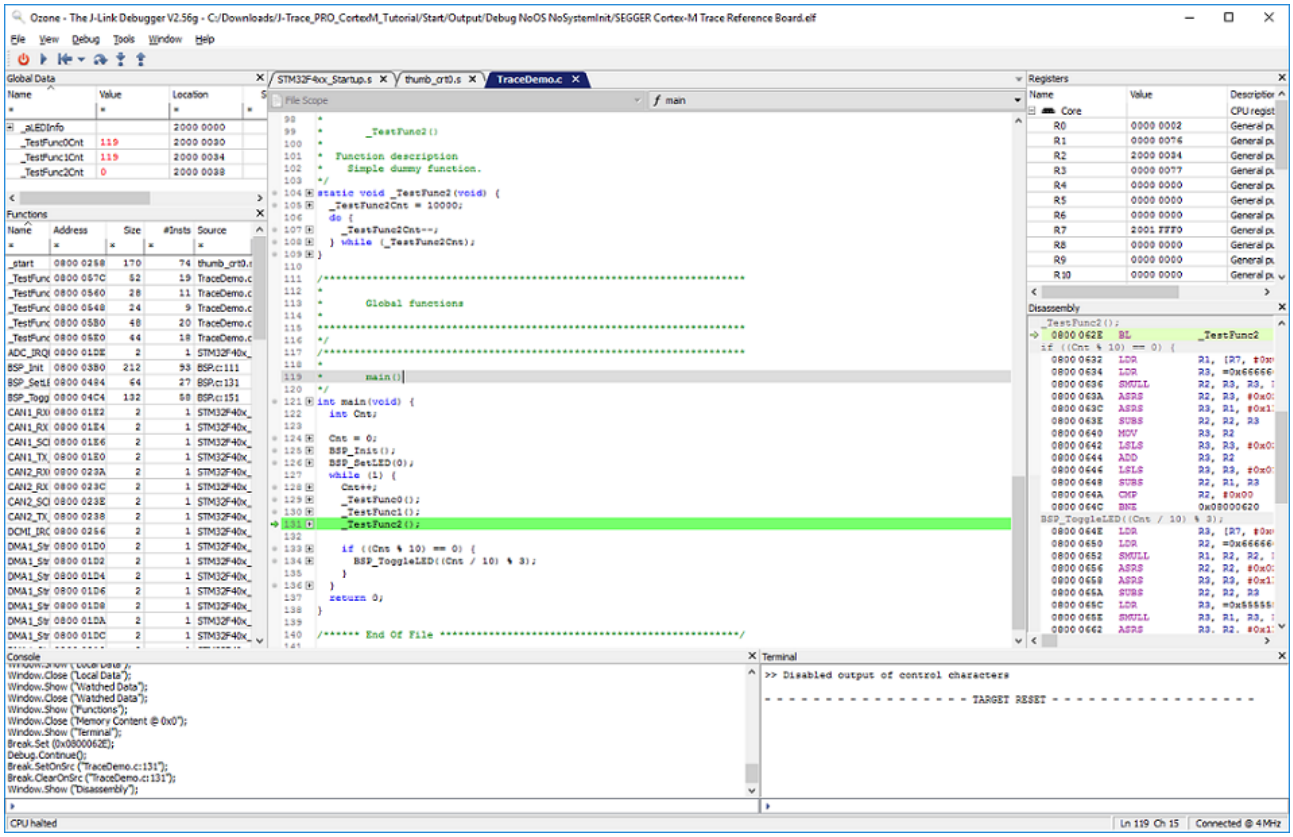
J-Scope：


New Project* - SEGGER J-Scope V6.11j
File View Help
Sampling: Trigger: Target:
Horizontal: 200 samples/div
2535.996 M Δt
Legend
0: Symbol No 1 5k/div -2.0
0
Name Address Color Size Type Value Min Max Moving Aver...
Symbol No 1 0x000000... #FFD7... 2 Bytes 2 Byte... 3266.00 0.00 4094.00 2049.22
SEGGER J-Scope started successfully.
On (RTT): 99.00 kSampl...
Buffer 16.00 MiB (2796202 samples), 10...

J-RTT：


J-Link RTT Viewer V6.30b
File Terminals Input Logging Help
Log
All Terminals
Terminal 0
Terminal 1
Terminal 2
Terminal 3
1> Red: This text is red. This background is green. Normal text again.
1> Red: This text is red. This background is green. Normal text again.
1> Red: This text is red. This background is green. Normal text again.
1> Red: This text is red. This background is green. Normal text again.
1> Red: This text is red. This background is green. Normal text again.
2> KEY_DOWN_K2, ArmFly Real-Time-Terminal Sample
2> KEY_DOWN_K2, ArmFly Real-Time-Terminal Sample
2> KEY_DOWN_K2, ArmFly Real-Time-Terminal Sample
3> KEY_DOWN_K3, i = 0
3> KEY_DOWN_K3, i = 1
3> KEY_DOWN_K3, i = 2
3> KEY_DOWN_K3, i = 3
3> KEY_DOWN_K3, i = 4
3> KEY_DOWN_K3, i = 5
Enter
Clear
RTT Viewer connected.
0.000 MB

Ozone：

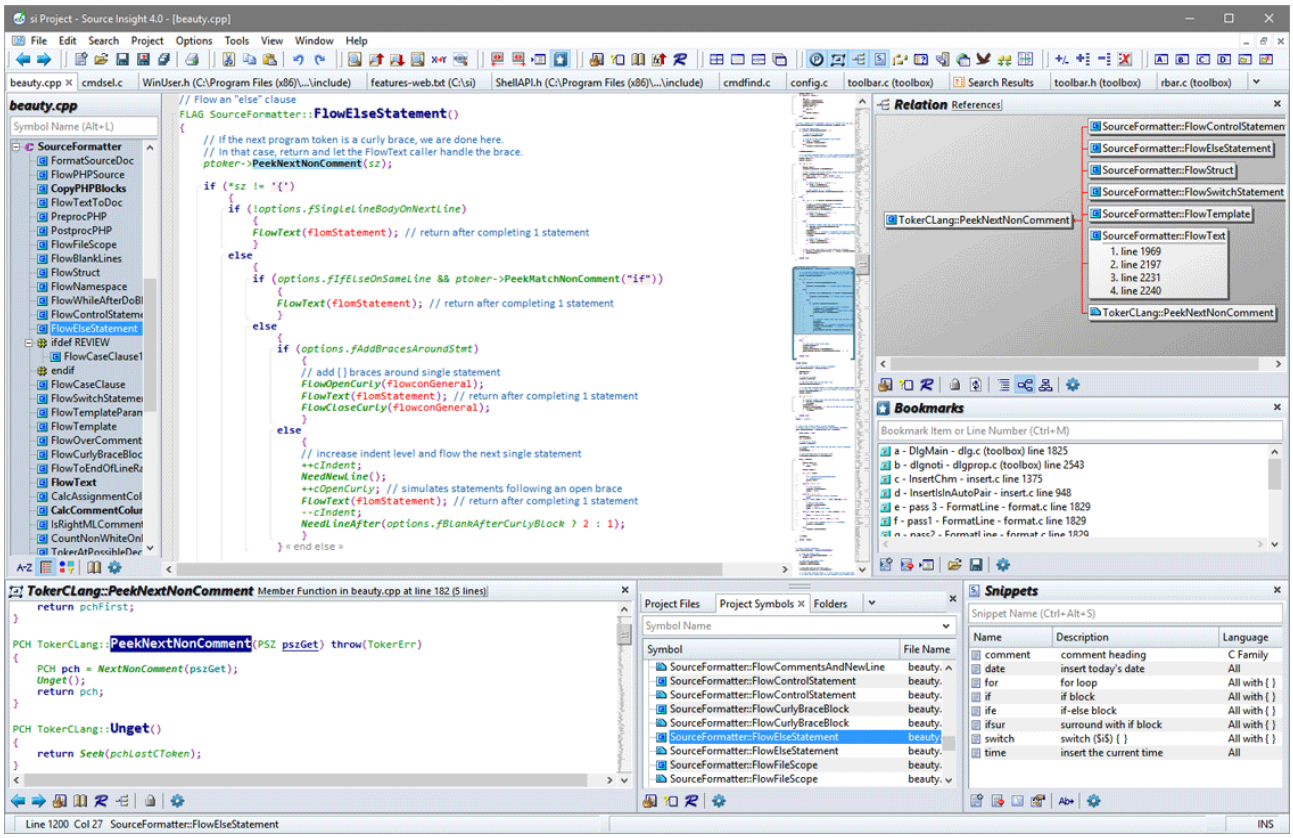

Ozone - The J-Link Debugger V2.56g - C:/Downloads/J-Trace_PRO_CortexM_Tutorial/Start/Output/Debug NoOS NoSystemInit/SEGGER Cortex-M Trace Reference Board.elf
File View Debug Tools Window Help
Global Data
Functions
Registers
Disassembly
Console
Terminal
>> Disabled output of control characters
- - - - - - - - - - - - - - - - - - TARGET RESET - - - - - - - - - - - - - - - - - -
CPU halted

第97章 附件 C---Source Insight 源代码浏览和编辑器

Source Insight 是一款面向项目开发的程序编辑器和代码浏览器。相对于一般的软件开发环境，Source Insight 能够更直观，更方便地浏览和编辑代码。

Source Insigh4.0 安装包和注册，含早年做的无声入门视频：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=82497 。



补充一个知识点：

同时测试，VS Code，Si，Sublime Text3 和 NotePad++，添加了 N 种插件后，那个编辑代码更溜

http://www.armbbs.cn/forum.php?mod=viewthread&tid=24001 。

第98章 附件 D---比较 IAR,MDK 的 AC5 和 AC6 以及 ES 的 CLANG 和 GCC 编译 HAL 库性能

鉴于 MDK AC5(ARM Compiler5)编译 STM32H7 的 HAL 库带 Browser Info 时速度巨卡，电脑性能稍差点，编译个大点的工程能编译出几十分钟。特此做一个完整的测试。

此贴持续更新中：http://www.armbbs.cn/forum.php?mod=viewthread&tid=93102 。

◆ **测试条件：**

1、IAR8.30 开最高等级的最小代码优化，Printf 选择 small 和 full 均做测试。

2、MDK5.27 正式版使用 AC5 开最高等级优化 3，使用的 MicroLib。

3、MDK5.27 正式版使用 AC6 开最小代码优化，使用的 MicroLib。

4、Embedded Studio4.15 beta 版使用 GCC 开最高等级代码大小优化。

5、Embedded Studio4.15 beta 版使用 CLANG 开最高等级代码大小优化。

6、全部开启多核并行编译

7、电脑配置是 i3-3220，机械硬盘

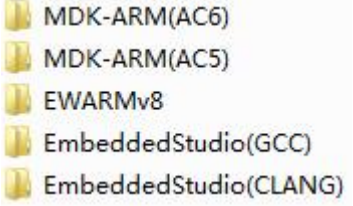
8、暂未测试芯片实际执行性能。

注意，现在 MDK 的 AC6 和 IAR 打开后都会主动创建 Browser Info，在窗口的最下方正中间有个提示。MDK 标识，要等待标识消失才可以使用 go to def：

IAR 标识，要等到进度条到头才可以使用 go to def：

◆ **基于 V7 开发板的测试例程下载**

地址：http://www.armbbs.cn/forum.php?mod=viewthread&tid=93102 。



◆ **编译时间测试**

先测试编译时间：

MDK5 使用 AC5 带 Browser Info

耗时 8 分 51 秒

MDK5 使用 AC6 带 Browser Info

耗时 1 分 07 秒

IAR 带 Browser Info（IAR 的编译速度很快，等 Browser Info 时间较长）

耗时 2 分 03 秒

Embedded Studio4.15 使用 GCC 带 Browser Info

耗时 38 秒

Embedded Studio4.15 使用 CLANG 带 Browser Info

耗时 21 秒

总结，Embedded Studio 使用 CLANG 完胜其它方式。

//

◆ **生成代码大小**

MDK5 使用 AC5：

```
Total RO  Size (Code + RO Data)                 17060 (16.66kB)
Total RW  Size (RW Data + ZI Data)               6536 (  6.38kB)
Total ROM Size (Code + RO Data + RW Data)   17120 (16.72kB)
```

MDK5 使用 AC6：

```
Total RO  Size (Code + RO Data)                 15960 (15.59kB)
Total RW  Size (RW Data + ZI Data)               6544 (  6.39kB)
Total ROM Size (Code + RO Data + RW Data)   15980 (15.61kB)
```

MDK5 使用 AC6，开启 Link-Time 优化（2019-03-29，晚 11 点）

```
Total RO  Size (Code + RO Data)                 11476 (  11.21kB)
Total RW  Size (RW Data + ZI Data)               6512 (   6.36kB)
Total ROM Size (Code + RO Data + RW Data)   11484 (  11.21kB)
```

IAR（printf 选择 full）：

20 290 bytes of readonly code memory

142 bytes of readonly data memory

6 677 bytes of readwrite data memory

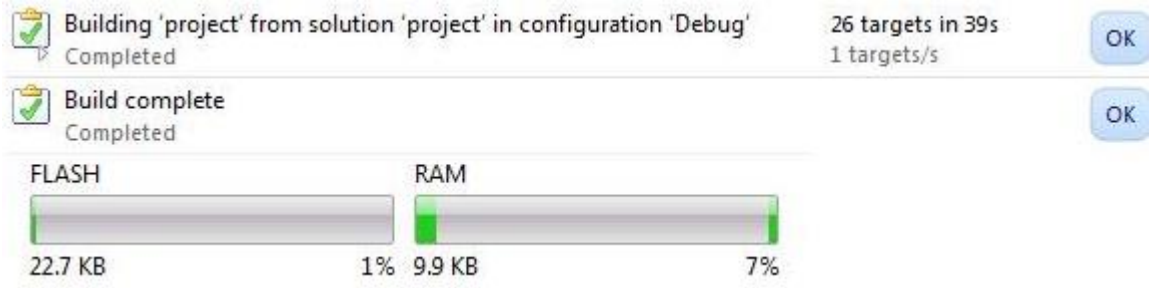
IAR（增加 Link 优化，printf 选择 small，新增 small 测试，2019-03-29，晚 10 点）：

16 734 bytes of readonly code memory

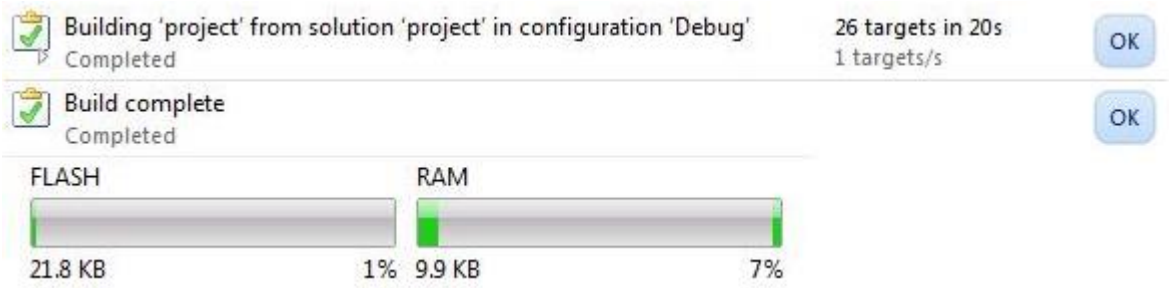
90 bytes of readonly data memory

6 621 bytes of readwrite data memory

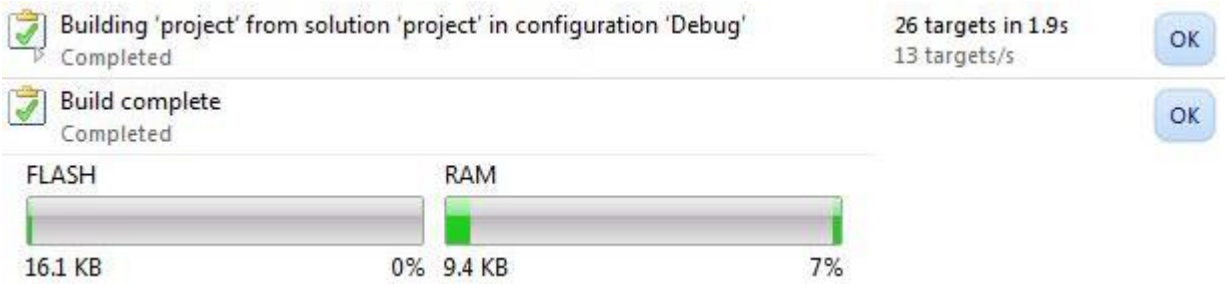
Embedded Studio4.15 使用 GCC



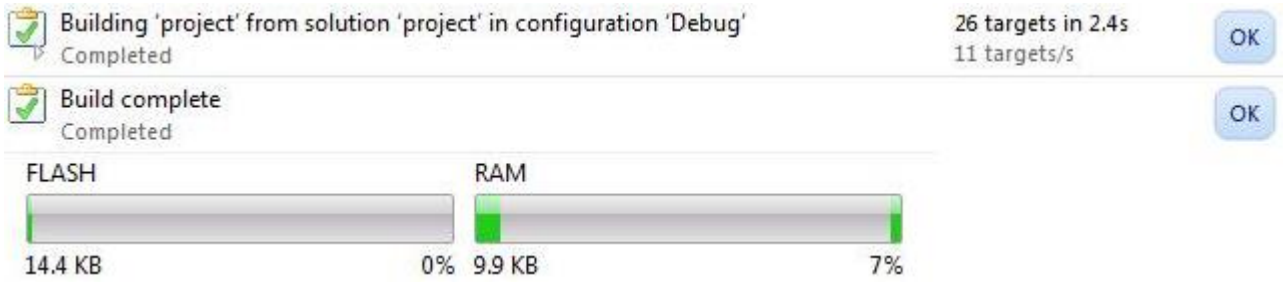
Embedded Studio4.15 使用 CLANG



Embedded Studio4.15 使用 CLANG 开 Link-Time 优化，效果明显



Embedded Studio4.16 使用 GCC 开 Link-Time 优化，继续减小（2019-04-06）



总结，MDK 的 AC6 生成的代码最小。

现在 MDK5 的 AC6 也是用的 CLANG，总的来说，无论是编译速度还是生成代码大小，CLANG 都表现出巨大优势。

当前 HAL 库使用 AC6 编译的警告有点多，可以选择使用 AC5 的警告临时屏蔽，提供的测试例子是采用的这种方式。

另外使用中文也会有很多警告，所以干脆使用 UTF-8 编码，就没有警告了，不过程序中的打印有中文的话，串口助手要使用支持 UTF-8 的，比如 SecureCRT 即可。

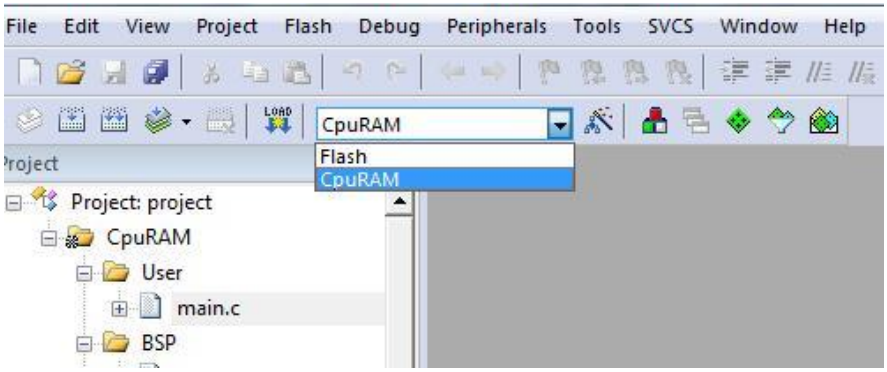
第99章 附件 E---STM32H7 内部 SRAM 调试工程模板

基于 STM32H7 的内部 SRAM 程序调试工程模板(512KB 的 AXI SRAM 存程序,DTCM 做主 RAM)。程序下载地址：http://www.armbbs.cn/forum.php?mod=viewthread&tid=93112 。

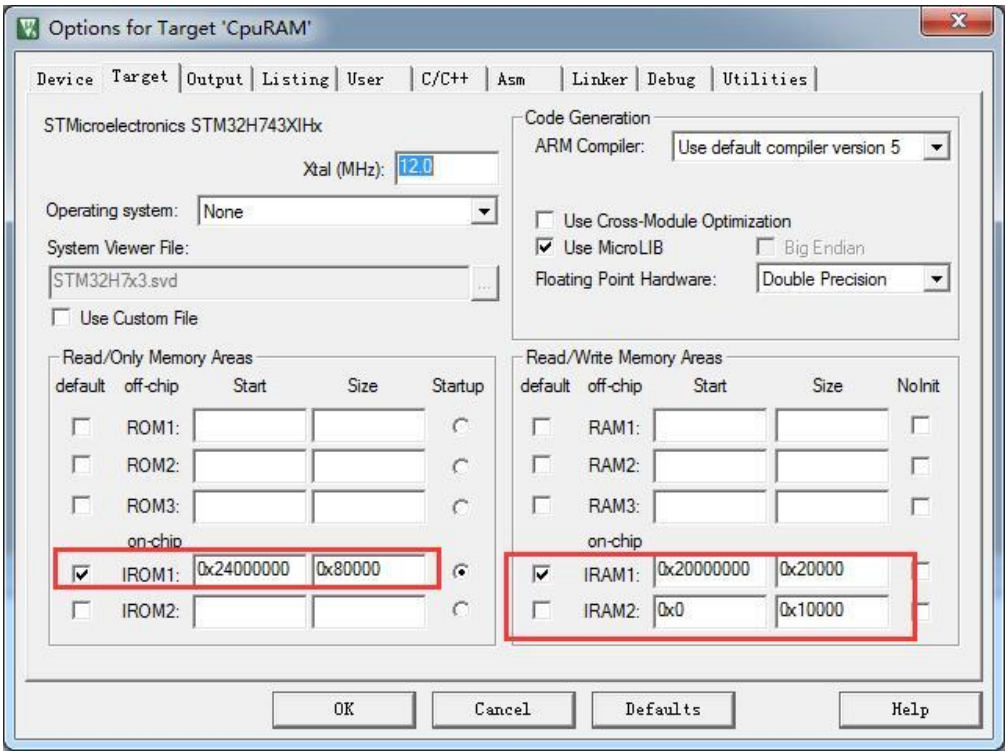
直接点击调试按键即可使用。后面大家使用 H750 时,这个方案很实用,而对于 H743 来说,内部 Flash 放心用即可，擦写次数足够用。

几个关键地方的设置：

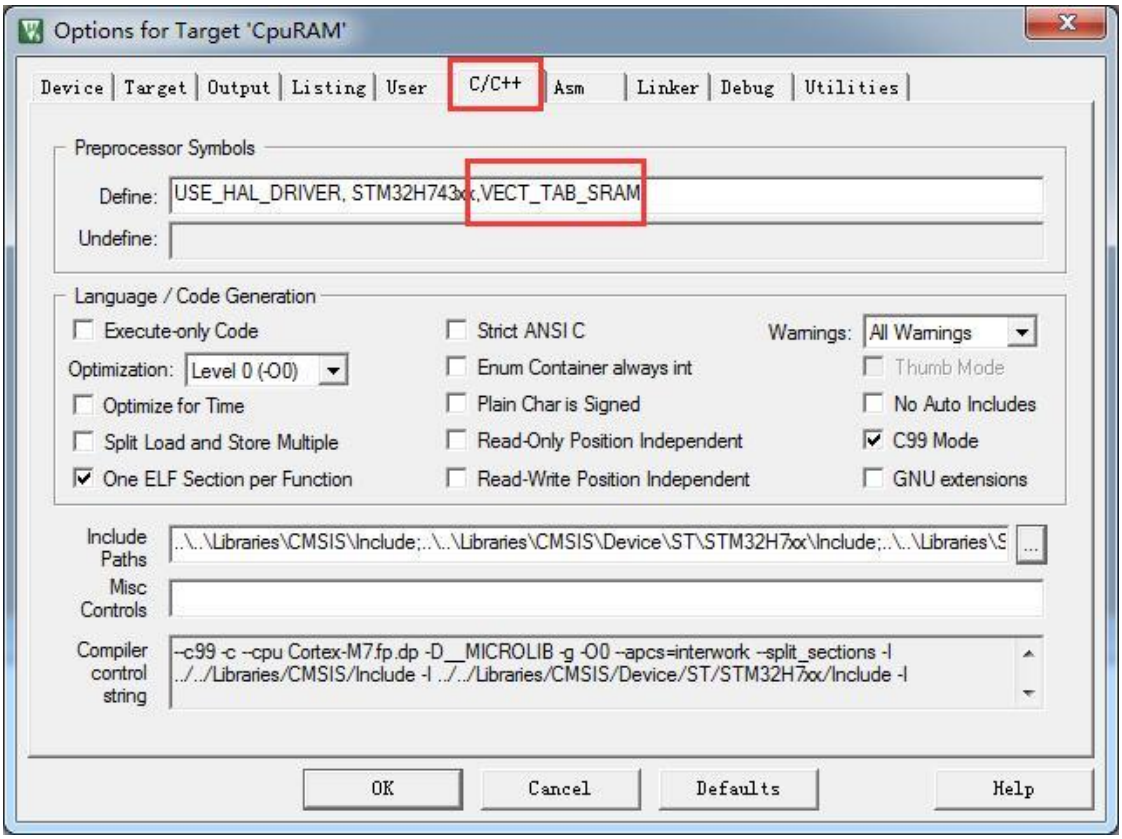
- 选择 CpuRAM。



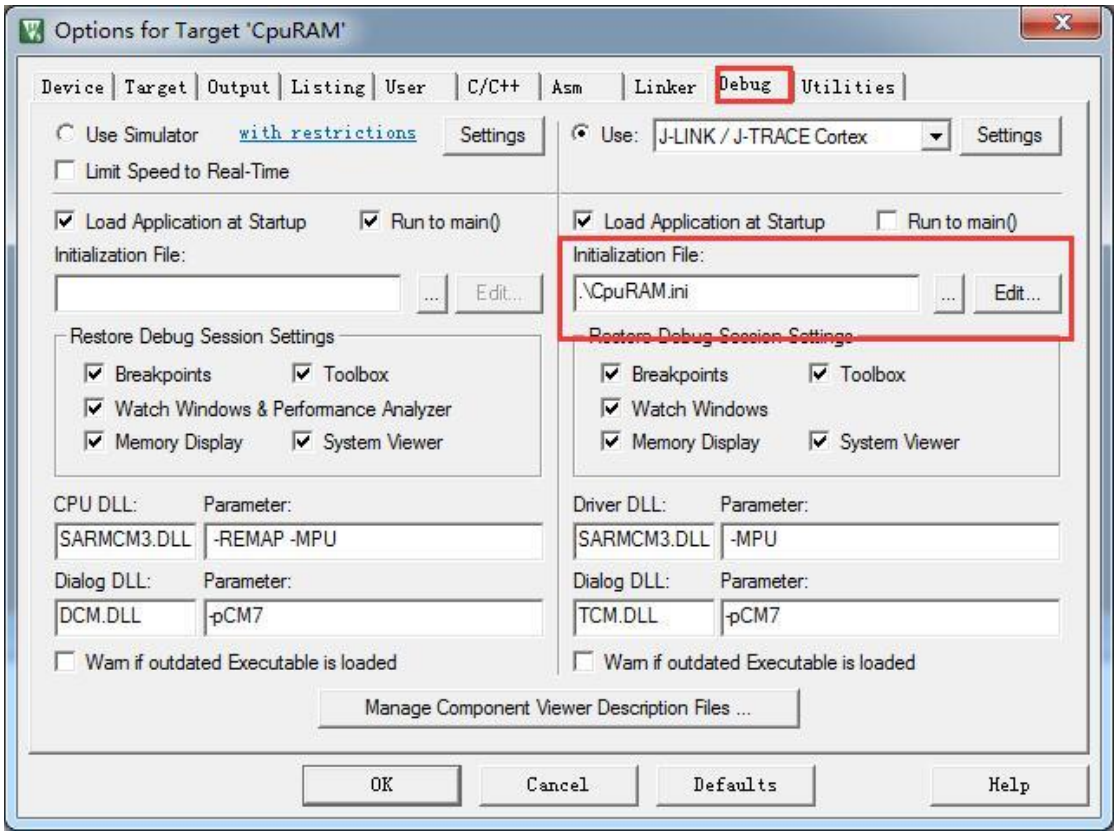
- 程序空间使用 AXI SRAM，大小 512KB，主 RAM 使用 DTCM，大小 128KB，其它未使用的用于 DMA 或者其它需要大数据量的场合。



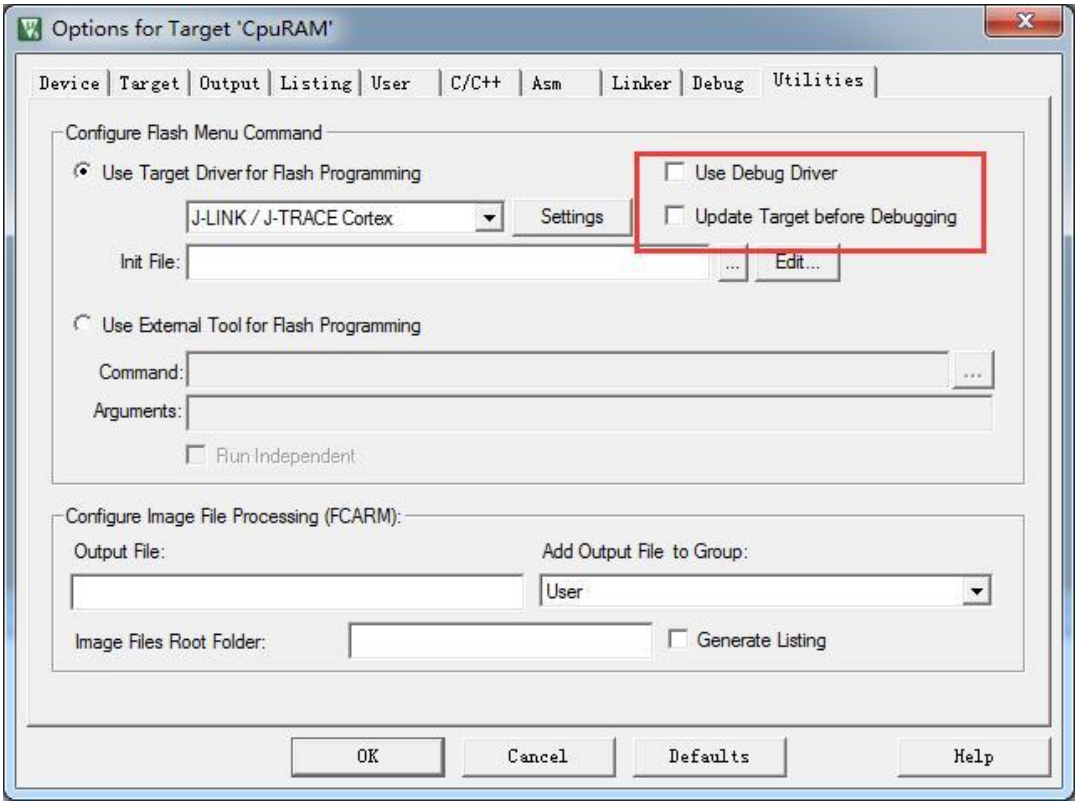
◆ 中断向量表设置到 RAM。



◆ 添加脚本。



◆ 这里取消对勾。


Options for Target 'CpuRAM'
Device
Target
Output
Listing
User
C/C++
Asm
Linker
Debug
Utilities
Configure Flash Menu Command
Use Target Driver for Flash Programming
Use Debug Driver
J-LINK / J-TRACE Cortex
Settings
Update Target before Debugging
Init File:
Edit...
Use External Tool for Flash Programming
Command:
Arguments:
Run Independent
Configure Image File Processing (FCARM):
Output File:
Add Output File to Group:
User
Image Files Root Folder:
Generate Listing
OK
Cancel
Defaults
Help

第100章　附件 F---安富莱 C 语言编程规范

后面会为大家专门整理出一个文档，当前已经整理发布到论坛：

安富莱 C 语言编码规范 1--文件与目录
http://www.armbbs.cn/forum.php?mod=viewthread&tid=85595

安富莱 C 语言编码规范 2--排版
http://www.armbbs.cn/forum.php?mod=viewthread&tid=85596

安富莱 C 语言编码规范 3 --注释
http://www.armbbs.cn/forum.php?mod=viewthread&tid=85709

安富莱 C 语言编码规范 4 --可读性
http://www.armbbs.cn/forum.php?mod=viewthread&tid=85710

安富莱 C 语言编码规范 5--变量、结构、常量、宏
http://www.armbbs.cn/forum.php?mod=viewthread&tid=85724

安富莱 C 语言编码规范 6--函数
http://www.armbbs.cn/forum.php?mod=viewthread&tid=85725

第101章 附件 G---无损压缩算法 miniLZO 移植实现

1、miniLZO 是采用 C 编写的无损压缩库。

2、提供了快速压缩和超快速解压缩能力。

3、比较耗内存，需要 64KB 内存用于压缩，对于 H7 这种大内存的，非常合适。或者有外置 SRAM/SDRAM 的也比较合适，相比内部 RAM 就是速度稍慢些。

4、用到数据压缩的场合非常多，像以太网，无线数据传输，USB 数传，固件升级等场合，通数据压缩可以大大提升实际传输的数据量。

例子和详细的说明看此贴：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=94710 。

第102章 附件 F---基于 mbedTLS 的 AES 加密实现

1、 mbedTLS 的前身是 PolarSSL，开源免费。

主要提供了的 SSL/TLS 支持（在传输层对网络进行加密），各种加密算法，各种哈希算法，随机数生成以及 X.509（密码学里公钥证书的格式标准）的支持。

2、 mbedTLS 支持的加密算法含对称加密和非对称加密。

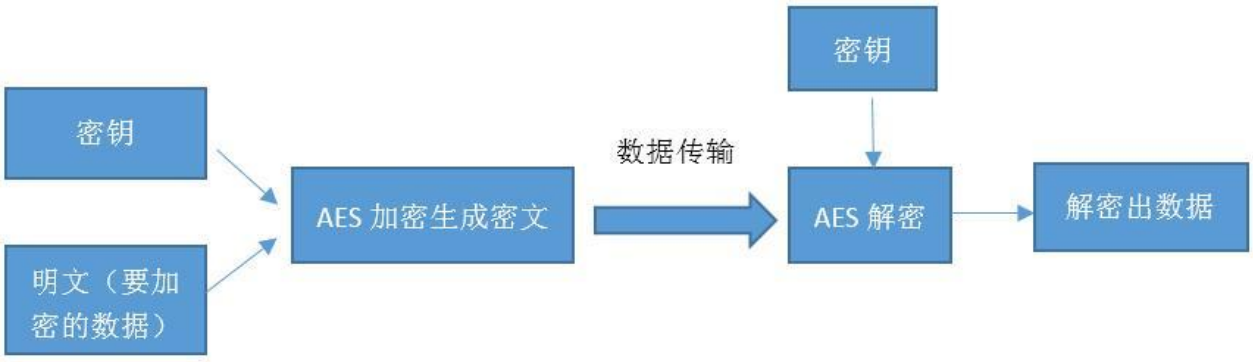
- mbedTLS 支持的对称加密算法有：

  AES，ARCFOUR，Blowfish/BF，Camellia，DES/3DES，GCM，XTEA

  加密和解密用到的密钥是相同的，加密速度快，适合数据收发通信，缺点是密钥的传输略麻烦。
- mbedTLS 支持的非对称加密算法有：

  Diffie-Hellman-Merkle，RSA，Elliptic Curves over GF(p)，Elliptic Curve Digital Signature Algorithm (ECDSA)和 Elliptic Curve Diffie Hellman (ECDH)加密和解密用的密钥是不同的，加密的速度慢，适合密钥的传输。

3、 关于 AES 加密，简单的说就是下面图示，由密钥和要解码的数据，通过算法生成密文，然后发给接收者，接收者使用相应的密钥解密数据。



例子和详细的说明看此贴：

http://www.armbbs.cn/forum.php?mod=viewthread&tid=94736。

第103章 文档更新记录

版本	更改说明	作者	发布日期
V1.0	初版。 从2018-06-04开始编写，2019-03-27发布首版。	白永斌 (Eric2013)	2019-03-27 (1057页)
V1.1	发布第50章和51章，添加一个附件章节。	白永斌 (Eric2013)	2019-04-06 (1132页)
V1.2	发布第52章和53章，添加一个附件章节。 第26章，增加IAR方式支持。	白永斌 (Eric2013)	2019-04-13 (1189页)
V1.3	发布第54章和55章。	白永斌 (Eric2013)	2019-04-21 (1257页)
V1.4	发布第56章。	白永斌 (Eric2013)	2019-04-28 (1290页)
V1.5	发布第57章和58章。	白永斌 (Eric2013)	2019-05-12 (1335页)
V1.6	发布第59章和60章	白永斌 (Eric2013)	2019-06-06 (1370页)
V1.7	发布第61章和62章	白永斌 (Eric2013)	2019-07-07 (1408页)
V1.8	更新所有章节的论坛链接，并增加两个附件章节	白永斌 (Eric2013)	2019-08-18 (1410页)
V1.9	发布第63和64章	白永斌	2020-02-06

		(Eric2013)	(1470页)
V2.0	发布第65和66章	白永斌 (Eric2013)	2020-02-15 (1535页)
V2.1	发布第67，68和69章	白永斌 (Eric2013)	2020-02-23 (1584页)
V2.2	发布第70和71章	白永斌 (Eric2013)	2020-03-01 (1624页)
V2.3	发布第72和73章	白永斌 (Eric2013)	2020-03-15 (1684页)
V2.4	发布第74章	白永斌 (Eric2013)	2020-04-05 (1749页)
V2.5	发布第75章	白永斌 (Eric2013)	2020-04-12 (1800页)
V2.6	发布第76章	白永斌 (Eric2013)	2020-05-03 (1867页)
V2.7	发布第77章	白永斌 (Eric2013)	2020-05-24 (1901页)
V2.8	发布第79章	白永斌 (Eric2013)	2020-11-02 (1978页)
V2.9	发布第80,81,82和83章	白永斌 (Eric2013)	2020-11-09 (2053页)
V3.0	发布第84,85和86章	白永斌	2020-11-16

		(Eric2013)	(2109页)
V3.1	发布第87和88章	白永斌 (Eric2013)	2021-01-18 (2188页)
V3.2	发布第89和90章	白永斌 (Eric2013)	2021-03-22 (2288页)
V3.3	发布第91和92章	白永斌 (Eric2013)	2021-11-01 (2306页)
V3.4	发布第93章	白永斌 (Eric2013)	2021-11-29 (2352页)
V3.5	发布第94章	白永斌 (Eric2013)	2022-04-11 (2392页)